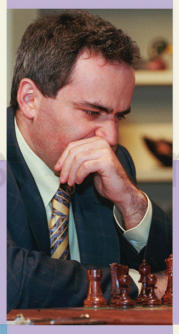
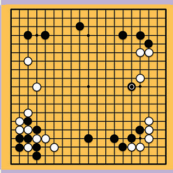
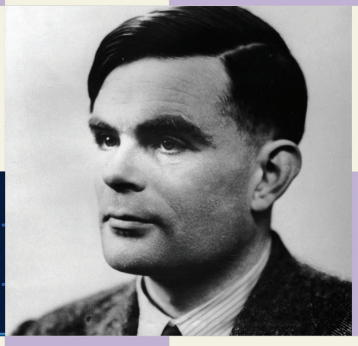
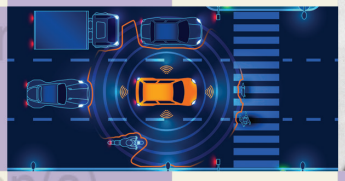
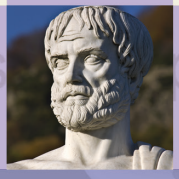









Stuart **Russell**
Peter **Norvig**

Inteligencia Artificial
Un enfoque moderno

Pearson

Inteligencia artificial

Un enfoque moderno

Inteligencia artificial

Un enfoque moderno

Primera edición en español

Cuarta edición en inglés

Stuart J. Russell y Peter Norvig

Escritores colaboradores:
Ming-Wei Chang • Jacob Devlin
Anca Dragan • David Forsyth
Ian Goodfellow • Jitendra M. Malik
Vikash Mansinghka • Judea Pearl
Michael Wooldridge

Traducción y revisión técnica
Juan Antonio Osuna Coutiño
Tecnológico Nacional de México, Campus Tuxtla Gutiérrez
Instituto Nacional de Astrofísica Óptica y Electrónica

Abiel Aguilar-González
Polyworks México

Revisión técnica
Andrés Mauricio Castillo Robles
Australian National Phenome Centre

Pearson

Datos de catalogación

Autores: Russell, Stuart J.; Peter Norvig
Inteligencia artificial. Un enfoque moderno
Primera edición
Pearson Educación de México, S.A. de C.V., 2024
ISBN IMPRESO: 978-607-32-6019-0
Área: Computación
Formato: 18.5 x 23.5 cm Páginas: 1136

Inteligencia artificial. Un enfoque moderno

Authorized translation from the English language edition, entitled *Artificial intelligence: a modern approach*, 9780134610993 by Stuart J. Russell and Peter Norvig, published by Pearson Education, Inc., publishing as [imprint], Copyright © 2021. All rights reserved. We may use artificial intelligence (AI) in our products and services, including to support the creation of content. While Pearson has a process to validate the resulting content, generative AI may still produce information that is not always accurate.

Traducción autorizada de la edición en lengua inglesa titulada *Artificial intelligence: a modern approach*, 9780134610993 por Stuart J. Russell y Peter Norvig, publicada por Pearson Education, Inc., como [imprint], Copyright © 2021. Todos los derechos reservados. Es posible que utilicemos inteligencia artificial (IA) en nuestros productos y servicios, incluso para apoyar la creación de contenido. Si bien Pearson cuenta con un proceso para validar el contenido resultante, la IA generativa puede aún producir información que no siempre es precisa.

Esta edición en español es la única autorizada.

Primera edición en español / Cuarta edición en inglés

Dirección de innovación y producto HED LATAM: Jorge Luis Íñiguez ■ **Gerencia de producto Hispanoamérica:** Teresa Islas ■ **Coordinación de arte y diseño LATAM:** Mónica Galván Alvarez ■ **Product Owner:** Roxana Cervantes Castañón ■ **Traducción:** Juan Antonio Osuna Coutiño/Abiel Aguilar-González/Mireya Janeth Gutiérrez Martínez ■ **Editor en jefe:** Arturo González Maya ■ **Revisión técnica:** Andrés Mauricio Castillo Robles /Juan Antonio Osuna Coutiño/Abiel Aguilar-González ■ **Composición y diagramación:** Alberto Sandoval (ZasaDesign).

Contacto: hedsoporte@pearson.com

Este título NO incluye códigos de acceso a ningún apoyo en línea. MyLabMath o cualquier otra herramienta digital se venden por separado. Para más información consulte a su Asesor de Servicios de Aprendizaje, o bien, escriba a hedsoporte@pearson.com

Primera edición, 2024

ISBN LIBRO IMPRESO: 978-607-32-6019-0
ISBN LIBRO EPUB: 978-607-32-6020-6

Impreso en México. *Printed in Mexico*.

1 2 3 4 5 6 7 8 9 0 - 27 26 25 24

D.R. © 2024 por Pearson Educación de México, S.A. de C.V.
Avenida Javier Barros Sierra, número 495, piso 3, oficina 138, Santa Fe, Alcaldía Álvaro Obregón, Ciudad de México, México, C.P. 01219

www.pearsonenespañol.com

Reservados todos los derechos. Ni la totalidad ni parte de esta publicación pueden reproducirse, registrarse o transmitirse, por un sistema de recuperación de información, en ninguna forma ni por ningún medio, sea electrónico, mecánico, fotoquímico, magnético o electroóptico, fotocopia, grabación o cualquier otro, sin permiso previo por escrito del editor.

Nota sobre enlaces a páginas web ajenas: Este libro incluye enlaces a sitios web cuya gestión, mantenimiento y control son responsabilidad única y exclusiva de terceros ajenos a Pearson Educación de México S.A. de C.V. ("Pearson"). Los enlaces u otras referencias a sitios web se incluyen con finalidad estrictamente informativa y se proporcionan en el estado en que se encuentran en el momento de publicación sin garantías, expresas o implícitas, sobre la información que se proporcione en ellas. Los enlaces no implican el aval de Pearson a tales sitios, páginas web, funcionalidades y sus respectivos contenidos o cualquier asociación con sus administradores. En consecuencia, Pearson no asume responsabilidad alguna por los daños que se puedan derivar de hipotéticas infracciones de los derechos de propiedad intelectual y/o industrial que puedan contener dichos sitios web ni por las pérdidas, delitos o los daños y perjuicios derivados, directa o indirectamente, del uso de tales sitios web y de su información. Al acceder a tales enlaces externos de los sitios web, el usuario estará bajo la protección de datos y políticas de privacidad o prácticas y otros contenidos de tales sitios web y no de Pearson.

Pearson LATAM

Argentina ■ Brasil ■ Bolivia ■ Chile ■ Colombia ■ Costa Rica ■ República Dominicana ■ Ecuador ■ El Salvador ■ Guatemala ■ Honduras ■ México ■ Nicaragua ■ Panamá ■ Paraguay ■ Perú ■ Uruguay ■ Venezuela

Para Loy, Gordon, Lucy, George e Isaac — S.J.R.

Para Kris, Isabella y Juliet — P.N.

Prefacio

La **Inteligencia Artificial** (IA) es un gran campo y este es un gran libro acerca de este tema. Hemos intentado explorar toda la amplitud de este campo que abarca lógica, probabilidad y matemáticas continuas; así como percepción, razonamiento, aprendizaje y acción; equidad, confianza, bienestar social y seguridad; y diversas aplicaciones que van desde dispositivos microelectrónicos hasta exploradores planetarios robóticos y servicios online con miles de millones de usuarios.

El subtítulo de este libro, "Un enfoque moderno", significa que hemos elegido contar la historia desde una perspectiva actual. Sintetizamos lo que ahora se conoce en un marco común, reelaborando el trabajo temprano con base en las ideas y la terminología preponderantes de la actualidad. Pedimos disculpas a aquellos lectores cuyos subcampos son, como resultado, menos reconocibles.

Nuevo en esta edición

Esta edición refleja los cambios en la IA ocurridos desde la última edición en 2010:

- Nos enfocamos más en el aprendizaje automático en lugar de la ingeniería del conocimiento hecha a mano, debido a la mayor disponibilidad de datos, recursos informáticos y nuevos algoritmos.
- El aprendizaje profundo, la programación probabilística y los sistemas multiagente reciben una cobertura ampliada, cada uno con su propio capítulo.
- La cobertura de la comprensión de temas como lenguaje natural, robótica y visión por computadora ha sido revisada para reflejar el impacto del aprendizaje profundo.
- El capítulo de robótica ahora incluye robots que interactúan con humanos y la aplicación del aprendizaje por refuerzo a la robótica.
- En la edición previa en inglés definimos como objetivo de la IA la creación de sistemas que intentan maximizar la utilidad esperada, donde la información de utilidad específica –el objetivo– es proporcionada por los diseñadores humanos del sistema. En esta nueva edición, primera en español, ya no asumimos que el objetivo es fijo y conocido por el sistema de IA; al contrario, consideramos que el sistema puede estar incierto sobre los verdaderos objetivos de los humanos, en cuyo nombre opera. Debe aprender qué maximizar y debe funcionar de manera adecuada, incluso mientras está incierto sobre el objetivo.
- Aumentamos la cobertura del impacto de la IA en la sociedad, incluidos temas vitales de ética, equidad, confianza y seguridad.
- Hemos trasladado los ejercicios de final de capítulo a un sitio online; esto nos permite agregar, actualizar y mejorar de manera continua los ejercicios, y así satisfacer las necesidades de los instructores y reflejar los avances en el campo y en las herramientas de software relacionadas con la IA.
- En general, cerca de 25 % del material en el libro es completamente nuevo; el 75 % restante ha sido reelaborado y reescrito en gran parte para presentar una imagen más unificada del campo. El 22 % de las citas en esta edición son de trabajos publicados después de 2010.

Visión general del libro

El principal tema unificador es la idea de un **agente inteligente**. Definimos la IA como el estudio de agentes que reciben percepciones del entorno y realizan acciones. Cada uno de estos agentes

implementa una función que mapea secuencias de percepción a acciones, aquí cubrimos diferentes formas de representar estas funciones, como: agentes reactivos, planificadores en tiempo real, sistemas de decisiones teóricas y sistemas de aprendizaje profundo. Enfatizamos el aprendizaje tanto como un método de construcción para sistemas competentes, como una forma de extender el alcance del diseñador a entornos desconocidos. Tratamos y definimos la robótica y la visión no como problemas independientes, sino como servicios que ocurren para lograr objetivos. Destacamos la importancia del entorno de la tarea en la determinación del diseño del agente apropiado.

Nuestro objetivo principal es transmitir las *ideas* que han surgido durante los últimos setenta años de investigación en IA y los últimos dos milenios de trabajo relacionado. Hemos tratado de evitar una excesiva formalidad en la presentación de estas ideas, pero manteniendo su precisión. Además, hemos incluido fórmulas matemáticas y algoritmos de pseudocódigo para concretar las ideas clave; los conceptos y notaciones matemáticas se describen en el Apéndice A y nuestro pseudocódigo se describe en el Apéndice B.

Este libro está destinado principalmente para su uso en un curso o secuencia de cursos de pregrado. El libro tiene 28 capítulos, cada uno requiere aproximadamente una semana de conferencias, por lo que trabajar en todo el libro requiere una secuencia de dos semestres. Un curso de un semestre puede usar capítulos seleccionados para adaptarse a los intereses del instructor y de los estudiantes. El libro también puede ser utilizado en un curso de nivel de posgrado (quizá con la adición de algunas de las fuentes primarias sugeridas en las notas bibliográficas), o para el autoestudio o como referencia.

A lo largo del libro, los *puntos importantes* están marcados con un icono de triángulo en el margen. Donde se define un nuevo **término**, también se anota en el margen. Los usos significativos posteriores del **término** están en negrita, pero no en el margen. Hemos incluido un índice completo y una extensa bibliografía.

El único requisito previo es la familiaridad con los conceptos básicos de la informática (algoritmos, estructuras de datos y complejidad) a nivel de segundo año. El cálculo de primer año y el álgebra lineal son útiles para algunos de los temas.

Material complementario para el profesor (en inglés)

A continuación se presenta una lista de los materiales complementarios, en inglés, disponibles para el profesor:

- Manual de soluciones para el profesor.
- Presentaciones PowerPoint correspondientes a los capítulos 1 a 28.

Para más información consulte a su Asesor de Servicios de Aprendizaje o, bien, escriba a hedsoporte@pearson.com

Recursos en inglés, disponibles en el sitio web del libro

Los recursos online están disponibles a través del sitio web del libro, aima.cs.berkeley.edu. Allí encontrará:

- Ejercicios, proyectos de programación y proyectos de investigación. Estos recursos, antes dispuestos al final de cada capítulo, ahora sólo están disponibles online. Dentro del libro nos referimos a un ejercicio online con un nombre como "Ejercicio 6.NARY". Las instrucciones en el sitio web le permiten encontrar ejercicios por nombre o por tema.

- Implementaciones de los algoritmos en el libro en Python, Java y otros lenguajes de programación (actualmente alojados en github.com/aimacode).
- Una lista de más de 1 400 escuelas que han utilizado el libro, muchas con enlaces a materiales de cursos online y programas de estudio.
- Material suplementario y enlaces para estudiantes e instructores.
- Instrucciones sobre cómo informar errores en el libro, en el probable caso de que existan.

Portada del libro

La portada muestra la posición final del decisivo juego 6 de la partida de ajedrez de 1997 en la que el programa Deep Blue derrotó a Garry Kasparov (jugando con negras), lo que fue la primera vez que un ordenador vencía a un campeón del mundo en una partida de ajedrez. Kasparov se muestra arriba. A la derecha se encuentra una posición crucial del segundo juego del histórico partido de Go entre el excampeón mundial Lee Sedol y el programa AlphaGo de DeepMind. El movimiento 37 de AlphaGo violó siglos de ortodoxia de Go y fue visto inmediatamente por los expertos humanos como un error embarazoso, pero resultó ser un movimiento ganador. En la parte superior izquierda se encuentra el robot humanoide Atlas, construido por Boston Dynamics. Una representación de un coche autónomo percibiendo su entorno aparece entre Ada Lovelace, la primera programadora de ordenadores del mundo, y Alan Turing, cuyo trabajo fundamental definió la inteligencia artificial. En la parte inferior del tablero de ajedrez se observa un robot Rover de Exploración de Marte y una estatua de Aristóteles, pionero en el estudio de la lógica, cuyo algoritmo de planificación de *De Motu Animalium* aparece atrás de los nombres de los autores. Detrás del tablero de ajedrez aparece un modelo de programación probabilística utilizado por la Organización del Tratado de Prohibición Completa de los Ensayos Nucleares de la ONU para detectar explosiones nucleares a partir de señales sísmicas.

Agradecimientos

Se necesita una aldea global para hacer un libro. Más de 600 personas leyeron partes del libro y sugirieron mejoras. La lista completa se aloja en aima.cs.berkeley.edu/ack.html, con todos ellos estamos muy agradecidos. Aquí hacemos espacio para mencionar sólo a algunos colaboradores especialmente importantes. Primero los escritores colaboradores:

- Judea Pearl (Sección 13.5, Redes causales);
- Vikash Mansinghka (Sección 15.3, Programas como modelos de probabilidad);
- Michael Wooldridge (Capítulo 18, Toma de decisiones multiagente);
- Ian Goodfellow (Capítulo 21, Aprendizaje profundo);
- Jacob Devlin y Mei-Wing Chang (Capítulo 24, Aprendizaje profundo para lenguaje natural);
- Jitendra Malik y David Forsyth (Capítulo 25, Visión por computadora);
- Anca Dragan (Capítulo 26, Robótica).

Luego, algunos roles clave:

- Cynthia Yeung y Malika Cantor (gestión de proyectos);
- Julie Sussman y Tom Galloway (corrección de textos y sugerencias de escritura);
- Omari Stephens (ilustraciones);
- Tracy Johnson (editor);

- Erin Ault y Rose Kernan (portada y conversión de color);
- Nalin Chhibber, Sam Goto, Raymond de Lacaze, Ravi Mohan, Ciaran O'Reilly, Amit Patel, Dragomir Radiv y Samagra Sharma (desarrollo de código online y tutoría);
- Estudiantes de Google Summer of Code (desarrollo de código online).

Stuart quisiera agradecer a su esposa, Loy Sheflott, por su infinita paciencia y sabiduría sin límites. Espera que Gordon, Lucy, George e Isaac pronto estén leyendo este libro después de que le hayan perdonado por trabajar tanto tiempo en él. RUGS (Grupo Inusual de Estudiantes de Russell) han sido inusualmente útiles, como siempre.

Peter quisiera agradecer a sus padres (Torsten y Gerda), por iniciarle, y a su esposa (Kris), hijos (Bella y Juliet), colegas, jefe y amigos por animarlo y tolerarlo durante las largas horas de escritura y reescritura.

Acerca de los autores

Stuart Russell nació en 1962 en Portsmouth, Inglaterra. Recibió su B.A. con honores de primera clase en Física de la Universidad de Oxford en 1982, y su Ph.D. en Ciencias de la computación de Stanford en 1986. Luego se unió a la facultad de la Universidad de California en Berkeley, donde es profesor y ex presidente de Ciencias de la computación, director del Centro para la IA Compatible con Humanos y titular de la Cátedra Smith–Zadeh en Ingeniería. En 1990 recibió el Premio al Joven Investigador Presidencial de la Fundación Nacional de Ciencias y en 1995 fue coganador del Premio Computadoras y Pensamiento. Es miembro de la Asociación Americana para la Inteligencia Artificial, la Asociación para la Maquinaria de Computación, y la Asociación Americana para el Avance de la Ciencia, es Miembro Honorario del Wadham College, Oxford, y un Andrew Carnegie Fellow. Ocupó la Chaire Blaise Pascal en París de 2012 a 2014. Ha publicado más de 300 trabajos sobre una amplia gama de temas en inteligencia artificial. Sus otros libros incluyen El *Uso del conocimiento en analogía e inducción*, *Haz lo correcto: estudios en racionalidad limitada* (con Eric Wefald), *Compatible con humanos: inteligencia artificial* y el *Problema del control*.

Peter Norvig es actualmente Director de Investigación en Google, Inc., antes fue el director responsable de los principales algoritmos de búsqueda web. Coenseñó una clase de IA online que inscribió a 160 000 estudiantes y ayudó a dar inicio a la ronda actual de clases masivas abiertas online. Fue jefe de la División de Ciencias Computacionales en el Centro de Investigación Ames de la NASA, supervisando la investigación y desarrollo en inteligencia artificial y robótica. Recibió un B.S. en Matemáticas aplicadas de la Universidad de Brown y un Ph.D. en Ciencias de la computación de Berkeley. Ha sido profesor en la Universidad del Sur de California y miembro de la facultad en Berkeley y Stanford. Es miembro de la Asociación Americana para la Inteligencia Artificial, la Asociación para la Maquinaria de Computación, la Academia Americana de Artes y Ciencias, y la Academia de Ciencias de California. Sus otros libros son *Paradigmas de la programación de IA: Estudios de caso en Common Lisp*, Verbmobil: *Un sistema de traducción para diálogos cara a cara* y *Sistemas de ayuda inteligentes para UNIX*.

Ambos autores compartieron el premio inaugural al Educador Sobresaliente AAAI/EAAI en 2016.

Contenidos

I Inteligencia Artificial

1 Introducción **1**
- 1.1 ¿Qué es IA? 1
- 1.2 Fundamentos de la inteligencia artificial 5
- 1.3 La Historia de la inteligencia artificial 17
- 1.4 El estado del arte 27
- 1.5 Riesgos y beneficios de la IA 31
- Resumen 34
- Notas bibliográficas e históricas 35

2 Agentes inteligentes **36**
- 2.1 Agentes y entornos 36
- 2.2 Buen comportamiento: el concepto de racionalidad 39
- 2.3 La naturaleza de los entornos 42
- 2.4 Estructura de los agentes 47
- Resumen 60
- Notas bibliográficas e históricas 60

II Resolución de problemas

3 Resolución de problemas de búsqueda **63**
- 3.1 Agentes solucionadores de problemas 63
- 3.2 Ejemplos de problemas 66
- 3.3 Algoritmos de búsqueda 71
- 3.4 Estrategias de búsqueda no informadas 76
- 3.5 Estrategias de búsqueda informada (heurística) 84
- 3.6 Funciones heurísticas 97
- Resumen 104
- Notas bibliográficas e históricas 106

4 Búsqueda en entornos complejos **110**
- 4.1 Búsqueda local y problemas de optimización 110
- 4.2 Búsqueda local en espacios continuos 119
- 4.3 Búsqueda con acciones no deterministas 122
- 4.4 Búsqueda en entornos parcialmente observables 126
- 4.5 Agentes de búsqueda online y entornos desconocidos 134
- Resumen 141
- Notas bibliográficas e históricas 142

5 Búsqueda adversarial y juegos **146**
- 5.1 Teoría de juegos 146

5.2 Decisiones óptimas en juegos 148
5.3 Búsqueda de árbol alfa-beta heurística 156
5.4 Búsqueda de árbol de Monte Carlo 161
5.5 Juegos estocásticos 164
5.6 Juegos parcialmente observables 168
5.7 Limitaciones de los algoritmos de búsqueda de juegos 173
Resumen 174
Notas bibliográficas e históricas 175

6 Problemas de satisfacción de restricciones **180**
6.1 Definiendo problemas de satisfacción de restricciones 180
6.2 Propagación de restricciones: Inferencia en CSPs 185
6.3 Búsqueda de retroceso para CSPs 191
6.4 Búsqueda local para CSPs 197
6.5 La estructura de los problemas 199
Resumen 203
Notas bibliográficas e históricas 204

III Conocimiento, razonamiento y planificación

7 Agentes lógicos **208**
7.1 Agentes basados en conocimiento 209
7.2 El Mundo del wumpus 210
7.3 Lógica 214
7.4 Lógica proposicional: una lógica muy simple 217
7.5 Prueba de teoremas proposicionales 222
7.6 Verificación efectiva de modelos proposicionales 232
7.7 Agentes basados en lógica proposicional 237
Resumen 246
Notas bibliográficas e históricas 247

8 Lógica de primer orden **251**
8.1 Revisión de la representación 251
8.2 Sintaxis y semántica de la lógica de primer orden 256
8.3 Usando la lógica de primer orden 265
8.4 Ingeniería del conocimiento en lógica de primer orden 271
Resumen 277
Notas bibliográficas e históricas 278

9 Inferencia en la lógica de primer orden **280**
9.1 Inferencia proposicional vs. inferencia de primer orden 280
9.2 Unificación e inferencia de primer orden 282
9.3 Inferencia hacia adelante (fc) 286
9.4 Encadenamiento hacia atrás (bc) 293
9.5 Resolución 298
Resumen 309
Notas bibliográficas e históricas 310

10 Representación del conocimiento **314**
10.1 Ingeniería ontológica 314
10.2 Categorías y objetos 317
10.3 Eventos 322
10.4 Objetos mentales y lógica modal 326
10.5 Sistemas de razonamiento para categorías 329
10.6 Razonamiento con información por defecto 333
Resumen 337
Notas bibliográficas e históricas 338

11 Planificación automatizada **344**
11.1 Definición de planificación clásica 344
11.2 Algoritmos para planificación clásica 348
11.3 Heurísticas para la planificación 353
11.4 Planificación jerárquica 356
11.5 Planificar y actuar en dominios no deterministas 365
11.6 Tiempo, programación y recursos 374
11.7 Análisis de enfoques de planificación 378
Resumen 379
Notas bibliográficas e históricas 380

IV Conocimiento y razonamiento inciertos

12 Cuantificar la incertidumbre **385**
12.1 Actuando bajo incertidumbre 385
12.2 Notación básica de probabilidad 388
12.3 Inferencia utilizando distribuciones conjuntas completas 395
12.4 Independencia 397
12.5 Regla de Bayes y su uso 399
12.6 Modelos Bayes ingenuos 402
12.7 De vuelta al mundo del wumpus 404
Resumen 407
Notas bibliográficas e históricas 408

13 Razonamiento probabilístico **412**
13.1 Representando conocimiento en un dominio incierto 412
13.2 La semántica de las redes bayesianas 414
13.3 Inferencia exacta en redes bayesianas 427
13.4 Inferencia aproximada para redes bayesianas 435
13.5 Redes causales 449
Resumen 453
Notas bibliográficas e históricas 454

14 Razonamiento probabilístico a lo largo del tiempo **461**
14.1 Tiempo e incertidumbre 461
14.2 Inferencia en modelos temporales 465
14.3 Modelos de Markov ocultos 473

14.4 Filtros de Kalman 479
14.5 Redes bayesianas dinámicas 485
Resumen 496
Notas bibliográficas e históricas 497

15 Programación probabilística **500**
15.1 Modelos de probabilidad relacionales 501
15.2 Modelos de probabilidad de universo abierto 507
15.3 Mantener seguimiento de un mundo complejo 514
15.4 Programas como modelos de probabilidad 519
Resumen 523
Notas bibliográficas e históricas 524

16 Toma de decisiones sencillas **528**
16.1 Combinando creencias y deseos bajo incertidumbre 528
16.2 La base de la teoría de la utilidad 529
16.3 Funciones de utilidad 532
16.4 Funciones de utilidad multiatributo 540
16.5 Redes de decisión 544
16.6 El valor de la información 547
16.7 Preferencias desconocidas 553
Resumen 557
Notas bibliográficas e históricas 557

17 Tomando decisiones complejas **562**
17.1 Problemas de decisión secuencial 562
17.2 Algoritmos para MDPs 572
17.3 Problemas de bandido 581
17.4 MDPs parcialmente observables 588
17.5 Algoritmos para resolver POMDPs 590
Resumen 595
Notas bibliográficas e históricas 596

18 Toma de decisiones simples multiagente **599**
18.1 Propiedades de los entornos multiagente 599
18.2 Teoría de juegos no cooperativa 605
18.3 Teoría de juegos cooperativos 626
18.4 Tomando decisiones colectivas 632
Resumen 645
Notas bibliográficas e históricas 646

V Aprendizaje automático

19 Aprendiendo de ejemplos **651**
19.1 Formas de aprendizaje 651
19.2 Aprendizaje supervisado 653
19.3 Aprendizaje de árboles de decisión 657

19.4 Selección y optimización del modelo 665
19.5 La teoría del aprendizaje 672
19.6 Regresión y clasificación lineal 676
19.7 Modelos no paramétricos 686
19.8 Aprendizaje en conjunto 696
19.9 Desarrollo de sistemas de aprendizaje automático 704
Resumen 714
Notas bibliográficas e históricas 715

20 Aprendizaje de modelos probabilísticos **721**
20.1 Aprendizaje estadístico 721
20.2 Aprendizaje con datos completos 724
20.3 Aprendizaje con variables ocultas: el algoritmo EM 737
Resumen 746
Notas bibliográficas e históricas 747

21 Aprendizaje profundo **750**
21.1 Redes de avance simple 751
21.2 Grafos computacionales para el aprendizaje profundo 756
21.3 Redes convolucionales 760
21.4 Algoritmos de aprendizaje 765
21.5 Generalización 768
21.6 Redes neuronales recurrentes 772
21.7 Aprendizaje no supervisado y aprendizaje de transferencia 775
21.8 Aplicaciones 782
Resumen 784
Notas bibliográficas e históricas 785

22 Aprendizaje por refuerzo **789**
22.1 Aprendiendo de las recompensas 789
22.2 Aprendizaje por refuerzo pasivo 791
22.3 Aprendizaje por refuerzo activo 797
22.4 Generalización en aprendizaje por refuerzo 803
22.5 Búsqueda de políticas 810
22.6 Aprendizaje por observación y por refuerzo inverso 812
22.7 Aplicaciones del aprendizaje por refuerzo 815
Resumen 818
Notas bibliográficas e históricas 819

VI Comunicarse, percibir y actuar

23 Procesamiento del lenguaje natural **823**
23.1 Modelos de lenguaje 823
23.2 Gramática 833
23.3 Análisis sintáctico 835
23.4 Gramáticas aumentadas 841
23.5 Complicaciones del lenguaje natural real 845

23.6 Tareas de lenguaje natural 849
Resumen 850
Notas bibliográficas e históricas 851

24 Aprendizaje profundo para el procesamiento del lenguaje natural 856
24.1 Incrustaciones (embeddings) de palabras 856
24.2 Redes neuronales recurrentes para NLP 860
24.3 Modelos de secuencia a secuencia 864
24.4 La arquitectura del transformador 868
24.5 Preentrenamiento y aprendizaje de transferencia 871
24.6 Estado del arte 875
Resumen 878
Notas bibliográficas e históricas 878

25 Visión por computadora 881
25.1 Introducción 881
25.2 Formación de la imagen 882
25.3 Características de imagen simples 888
25.4 Clasificando imágenes 895
25.5 Detectando objetos 899
25.6 El mundo 3D 901
25.7 Usando visión por computadora 906
Resumen 919
Notas bibliográficas e históricas 920

26 Robótica 925
26.1 Robots 925
26.2 Hardware del robot 926
26.3 ¿Qué tipo de problema resuelve la robótica? 930
26.4 Percepción robótica 931
26.5 Planificación y control 938
26.6 Planificación de movimientos inciertos 956
26.7 Aprendizaje por refuerzo en robótica 958
26.8 Humanos y robots 961
26.9 Marcos robóticos alternativos 968
26.10 Dominios de aplicación 971
Resumen 974
Notas bibliográficas e históricas 975

VII Conclusiones

27 Filosofía, ética y seguridad de la ia 981
27.1 Los límites de la IA 981
27.2 ¿Pueden las máquinas pensar? 984
27.3 La ética de la IA 986
Resumen 1005
Notas bibliográficas e históricas 1006

28 El futuro de la IA **1012**
28.1 Componentes de IA 1012
28.2 Arquitecturas de IA 1018

A Antecedentes matemáticos **1023**
A.1 Análisis de complejidad y notación O() 1023
A.2 Vectores, matrices y álgebra lineal 1025
A.3 Distribuciones de probabilidad 1027
Notas bibliográficas e históricas 1029

B Notas sobre lenguajes y algoritmos **1030**
B.1 Definición de idiomas con Backus-Naur Form (BNF) 1030
B.2 Descripción de algoritmos con pseudocódigo 1031
B.3 Material complementario online 1032

Bibliografía **1033**

Índice **1069**

CAPÍTULO 1

INTRODUCCIÓN

En la cual se intenta explicar por qué consideramos que la inteligencia artificial (IA) es un tema digno de estudio, y en lo cual intentamos decidir con exactitud en qué consiste, siendo esto algo bueno por determinar antes de entrar en materia.

Nos nombramos *Homo sapiens*, hombre sabio, porque nuestra **inteligencia** es fundamental para nosotros. Por miles de años, hemos tratado de entender *cómo pensamos y actuamos;* es decir, cómo nuestro cerebro, un mero puñado de materia, puede percibir, comprender, predecir y manipular un mundo más grande y complicado que él mismo. El campo de la **inteligencia artificial (**IA) se ocupa no sólo de comprender, sino de *construir* entidades inteligentes, máquinas que pueden calcular cómo actuar de manera efectiva y segura en una amplia variedad de situaciones novedosas.

Inteligencia

Inteligencia artificial (IA)

Las encuestas suelen clasificar la IA como uno de los campos más interesantes y de más rápido crecimiento, ya genera más de un billón de dólares al año en ganancias. Kai-Fu Lee, experto en IA, predice que su impacto será "mayor que cualquier otra cosa en la historia de la humanidad". Ysus fronteras intelectuales están abiertas. Mientras que un estudiante de una ciencia antigua, como la física, podría pensar que las mejores ideas ya fueron reveladas por Galileo, Newton, Curie, Einstein y demás, la IA aún tiene muchas oportunidades para mentes maestras dedicadas.

Hoy, la IA abarca una gran variedad de subcampos, que van de lo general (aprendizaje, razonamiento, percepción, y así sucesivamente) hasta lo específico (jugar ajedrez, probar teoremas matemáticos, escribir poesía, conducir un automóvil o diagnosticar enfermedades). La IA es relevante para cualquier tarea intelectual; es en verdad un campo universal.

1.1 ¿Qué es IA?

Ya afirmamos que la IA es interesante, pero no hemos dicho qué es. Históricamente, los investigadores han perseguido varias versiones diferentes de IA. Algunos definen la inteligencia en términos de *actuación humana*, otros dan una definición abstracta y formal, llamada **racionalidad**, es decir, hacer lo "correcto". El tema en sí mismo varía: algunos consideran que la inteligencia es una propiedad de los procesos de *pensamiento* y *razonamiento* internos, mientras que otros se centran en el *comportamiento* inteligente, una caracterización externa.[1]

Racionalidad

De estas dos dimensiones, humano vs. racional[2] y pensamiento vs. comportamiento, hay cuatro combinaciones posibles; ha habido seguidores y investigaciones para todas ellas. Los métodos

[1] En la opinión pública, a veces hay confusión entre los términos "inteligencia artificial" y "aprendizaje automático" (aprendizaje de máquina). Éste es un subcampo de la IA, el cual estudia la capacidad de mejorar el rendimiento basado en la experiencia. Algunos sistemas de IA usan métodos de aprendizaje automático para lograr competencia, otros no.

[2] No sugerimos que los humanos sean "irracionales" en el sentido literal de "privados de claridad mental normal". Sólo reconocemos que las decisiones humanas no siempre son matemáticamente perfectas.

utilizados son necesariamente diferentes: la búsqueda de una inteligencia similar a la humana debe ser, en parte, una ciencia empírica relacionada con la psicología, que involucre observaciones e hipótesis sobre el comportamiento humano real y los procesos de pensamiento; en cambio, un enfoque racionalista implica una combinación de matemáticas e ingeniería, y se conecta con estadística, teoría de control y economía. Ambos grupos se han menospreciado y ayudado. Veamos los cuatro enfoques con más detalle.

1.1.1 Actuar de manera humana: El enfoque de la prueba de Turing

Prueba de Turing

La **prueba de Turing**, propuesta por Alan Turing (1950), fue diseñada como un experimento mental que evitaría la vaguedad filosófica de la pregunta "¿Puede pensar una máquina?". Una computadora pasa la prueba si un interrogador humano, después de plantear algunas preguntas escritas, no puede decir si las éstas provienen de una persona o de una computadora. El Capítulo 27 discute los detalles de la prueba y si realmente una computadora sería inteligente si la pasara. Por ahora, observe que programar una computadora para pasar una prueba aplicada con rigor da mucho en qué trabajar. La computadora necesitaría las siguientes capacidades:

Procesamiento del lenguaje natural
Representación del conocimiento
Razonamiento automatizado
Aprendizaje automático

- **Procesamiento del lenguaje natural** para comunicarse con éxito en un idioma humano
- **Representación del conocimiento** para almacenar lo que sabe o escucha
- **Razonamiento automatizado** para responder preguntas y sacar nuevas conclusiones
- **Aprendizaje automático** se adapta a nuevas circunstancias, detecta y extrapola patrones

Prueba de Turing total

Turing consideraba innecesaria la simulación *física* de una persona para demostrar inteligencia. Otros investigadores, en cambio, han propuesto una **prueba de Turing total**, que requiere interacción con objetos y personas en el mundo real. Para pasar esta prueba, un robot necesitará:

Visión por computadora
Robótica

- **Visión por computadora** y reconocimiento de voz para percibir el mundo;
- **Robótica** para manipular objetos y moverse.

Las anteriores seis disciplinas componen en su mayoría la IA. Sin embargo, los investigadores han dedicado poco esfuerzo a pasar la prueba de Turing, pues creen más importante estudiar los principios de la inteligencia. La búsqueda del "vuelo artificial" tuvo éxito cuando ingenieros e inventores dejaron de imitar a las aves y usaron túneles de viento y aprendieron aerodinámica. Los textos de ingeniería aeronáutica no definen su objeto su estudio como hacer "máquinas que vuelen igual que las palomas, de modo que engañen aun a otras palomas".

1.1.2 Pensar de manera humana: Enfoque de modelado cognitivo

Para decir que un programa piensa como humano, debemos saber cómo piensan los humanos. Podemos aprender sobre el pensamiento humano de tres maneras:

Introspección
Experimento psicológico
Imágenes cerebrales

- **Introspección:** captar nuestros propios pensamientos mientras ocurren;
- **Experimentos psicológicos:** observar a una persona en acción;
- **Imágenes cerebrales:** observar el cerebro en acción.

Una vez que tengamos una teoría suficientemente precisa de la mente humana, es posible expresar esta teoría como un programa de computadora. Si el comportamiento de entrada-salida del programa coincide con el comportamiento humano, se evidencia que algunos de los mecanismos del programa también podrían operar en los humanos.

Por ejemplo, Allen Newell y Herbert Simon, creadores del "Solucionador General de Problemas" (GPS, por sus siglas en inglés) (Newell y Simon, 1961), no se satisficieron con que resolviera problemas de manera correcta. Se preocupaban más por comparar la secuencia y el momento de los pasos de razonamiento con los de los humanos que resolvían esos problemas. El campo interdisciplinario de la **ciencia cognitiva** reúne modelos computacionales de IA y técnicas experimentales de psicología para construir teorías precisas y comprobables de la mente humana.

Ciencia cognitiva

La ciencia cognitiva es un campo fascinante en sí mismo, digno de libros de texto y de una enciclopedia (Wilson y Keil, 1999). A veces comentaremos las similitudes o diferencias entre las técnicas de IA y la cognición humana. La verdadera ciencia cognitiva, empero, se basa en la investigación experimental en humanos o animales reales. Dejemos eso para otros libros, pues asumimos que el lector sólo tiene una computadora para experimentación.

En los orígenes de la IA, se solían confundir sus enfoques. Un autor argumentaría que un algoritmo se desempeña bien en una tarea y que, *por lo tanto*, es un buen modelo de desempeño humano, o viceversa. Los autores de hoy separan ambos tipos de afirmaciones; esta distinción permite que la IA y la ciencia cognitiva se desarrollen más rápido. Los dos campos se fecundan mutuamente, en especial en la visión por computadora, al incorporar evidencia neurofisiológica en modelos computacionales. Recién, la combinación de métodos de neuroimagen con técnicas de aprendizaje automático para analizar dichos datos ha creado la capacidad para "leer mentes", es decir, para determinar el contenido semántico de los pensamientos de una persona; capacidad que podría, a su vez, esclarecer cómo funciona la cognición humana.

1.1.3 Pensamiento racional: El enfoque de las "leyes del pensamiento"

El filósofo griego Aristóteles fue uno de los primeros que intentó codificar el "pensamiento correcto", es decir, procesos de razonamiento irrefutables. Sus **silogismos** aportaron patrones para estructuras de argumentos que siempre producían conclusiones correctas cuando se daban premisas correctas. El ejemplo canónico dicta *Sócrates es un hombre y todos los hombres son mortales* y concluye que *Sócrates es mortal*. (Este ejemplo quizá se debe a Sexto Empírico, no a Aristóteles.) Estas leyes del pensamiento se suponían regir el funcionamiento de la mente; lo que inició el campo de la **lógica**.

Silogismo

Los lógicos del siglo XIX desarrollaron una notación precisa para afirmaciones sobre objetos en el mundo y las relaciones entre ellos. (En contraste con la notación aritmética ordinaria, que sólo afirmaba sobre *números*). Para 1965, los programas podían, en principio, resolver *cualquier* problema soluble descrito en notación lógica. Esta tradición **logicista** dentro de la IA espera construir sistemas inteligentes sobre tales programas.

Logicismo

La lógica convencional requiere conocimiento del mundo que sea *cierto*; condición que, en realidad, rara vez se cumple. Simplemente desconocemos las reglas de, digamos, la política o la guerra tanto como conocemos las del ajedrez o la aritmética. La teoría de la **probabilidad** llena este vacío, al permitir un razonamiento riguroso con información incierta. En principio, permite la construcción de un modelo integral del pensamiento racional, desde información perceptual cruda hasta una comprensión de cómo funciona el mundo y predicciones sobre el futuro. Lo que no hace es generar *comportamiento* inteligente. Para eso, necesitamos una teoría de la acción racional. El pensamiento racional, por sí solo, no es suficiente.

Probabilidad

1.1.4 Actuar racionalmente: el enfoque del agente racional

Un **agente** (del latín *agere*, hacer) es algo que actúa. Claro, todos los programas de computadora hacen algo, pero se espera que los agentes de computadora hagan más: operar de manera autónoma, percibir su entorno, persistir durante un periodo prolongado, adaptarse a cambiar, crear

Agente

Agente racional

y perseguir objetivos. Un **agente racional** es aquel que actúa para lograr el mejor resultado o, cuando hay incertidumbre, el mejor resultado esperado.

En el enfoque de las "leyes del pensamiento" hacia la IA, el énfasis estaba en las inferencias correctas. Hacer inferencias correctas a veces es *parte* de ser un agente racional, pues una forma de actuar racionalmente es deducir que una acción dada es la mejor y, luego, actuar en consecuencia. Además, hay formas de actuar racionalmente que no pueden considerarse que involucren inferencia. Por ejemplo, retroceder ante una estufa caliente es una acción refleja que resulta más exitosa que una acción lenta tomada tras una cuidadosa deliberación.

Todas las habilidades necesarias para el test de Turing permiten que un agente actúe racionalmente. La representación del conocimiento y el razonamiento permiten a los agentes tomar buenas decisiones. Necesitamos ser capaces de generar frases comprensibles en lenguaje natural para desenvolvernos en una sociedad compleja. Necesitamos el aprendizaje no sólo por erudición, sino porque mejora nuestra capacidad para generar un comportamiento efectivo, en especial en circunstancias nuevas.

El enfoque del agente racional hacia la IA tiene dos ventajas sobre otros enfoques. Primero, es más general que el enfoque de las "leyes del pensamiento" porque la inferencia correcta es sólo uno de varios mecanismos posibles para lograr la racionalidad. Segundo, es más susceptible al desarrollo científico. El estándar de racionalidad está matemáticamente definido y es por completo general. Con frecuencia podemos retroceder desde esta especificación para derivar diseños de agentes que logren de manera comprobable eso, algo que es en gran medida imposible si el objetivo es imitar el comportamiento humano o los procesos de pensamiento.

Así, el enfoque del agente racional hacia la IA ha prevalecido en casi toda la historia de ésta. Al inicio, los agentes racionales se construyeron sobre bases lógicas y formaron planes definitivos para lograr objetivos específicos. Luego, los métodos basados en la teoría de la probabilidad y el aprendizaje automático permitieron la creación de agentes que tomaban decisiones bajo incertidumbre para obtener el mejor resultado esperado. En síntesis, la IA *se ha centrado en el estudio y la construcción de agentes que permiten* ***hacer lo correcto***, definido esto por el objetivo indicado al agente. Este paradigma general es tan persistente que podríamos llamarlo **modelo estándar**. Prevalece en IA y en teoría de control, donde un controlador minimiza una función de costo; en investigación operativa, donde una política maximiza una suma de recompensas; en estadística, donde una regla de decisión minimiza una función de pérdida, y en economía, donde un tomador de decisiones maximiza la utilidad o medida de bienestar social.

Hacer lo correcto

Modelo estándar

Necesitamos depurar el modelo estándar para advertir el hecho de que la racionalidad perfecta –siempre tomar la acción óptima–, no es factible en entornos complejos y las demandas computacionales son demasiado altas. Los capítulos 5 y 17 tratan acerca de la **racionalidad limitada**: actuar apropiadamente cuando no hay suficiente tiempo para calcular lo que uno desearía. Sin embargo, la racionalidad perfecta suele ser un buen punto de partida para el análisis teórico.

Racionalidad limitada

1.1.5 Máquinas benéficas

El modelo estándar ha sido una guía útil para la investigación en IA desde su inicio, pero quizá no sea el modelo correcto a largo plazo. La razón es que el modelo estándar asume que suministraremos un objetivo por completo especificado a la máquina.

Para una tarea artificialmente definida como ajedrez o cálculo de la ruta más corta, la tarea viene con un objetivo incorporado, por lo que el modelo estándar es aplicable. Pero a medida que avanzamos hacia el mundo real, se vuelve más y más difícil especificar por completo y de mane-

ra correcta el objetivo. Por ejemplo, al diseñar un automóvil autónomo, podríamos pensar que el objetivo es llegar al destino sanos y salvos. Conducir por carretera conlleva un riesgo de lesiones debidas a otros conductores erráticos, fallos en el equipo, etc.; por lo tanto, un objetivo estricto de seguridad requeriría quedarse en el garaje. Existe un equilibrio entre avanzar hacia el destino y correr el riesgo de lesiones. ¿Cómo se logra este equilibrio? Además, ¿hasta qué punto podemos permitir que el automóvil tome acciones que molesten a otros conductores? ¿Cuánto debe moderar el automóvil su aceleración, dirección y frenado para no zarandear al pasajero? Preguntas difíciles de responder *a priori,* pues son, en particular, problemáticas en el área general de interacción humano-robot, del cual este automóvil es un ejemplo.

Problema de alineación de valores

El problema de lograr un acuerdo entre nuestras verdaderas preferencias y el objetivo que introducimos en la máquina se llama **problema de alineación de valores**: los valores u objetivos introducidos en la máquina deben estar alineados con humanos. Si desarrollamos un sistema de IA en el laboratorio o en un simulador, como ha sido el caso en casi toda la historia del área, hay una solución fácil para un objetivo incorrectamente especificado: reiniciar el sistema, corregir el objetivo e intentarlo de nuevo. A medida que el campo avanza hacia sistemas inteligentes cada vez más capaces que se despliegan en el mundo real, este enfoque ya no es viable. Un sistema desplegado con un objetivo incorrecto tendrá consecuencias negativas. Además, cuanto más inteligente sea el sistema, más negativas serán las consecuencias.

Volviendo al ejemplo en apariencia sin problemas del ajedrez, consideremos qué sucede si la máquina es inteligente en suficiencia para razonar y actuar más allá de los límites del ajedrez. En ese caso, podría aumentar sus posibilidades de ganar mediante trucos, como hipnotizar o chantajear a su oponente o sobornar a la audiencia para que perturbe con ruidos el tiempo de reflexión de su oponente.[3] También podría intentar apropiarse de más potencia informática para sí mismo. *Estos comportamientos no son "no inteligentes" o "insanos"; son consecuencia lógica de definir la victoria como el único objetivo para la máquina.*

Es imposible anticipar todas las formas en que una máquina que persigue un objetivo podría comportarse mal. Hay buenas razones para pensar que el modelo estándar es inadecuado. No queremos máquinas inteligentes en el sentido de perseguir *sus* objetivos; queremos que persigan *nuestros* objetivos. Si no podemos transferir objetivos perfectamente a la máquina, entonces necesitamos reformular que la máquina persiga nuestros objetivos, pero que necesariamente esté *incierta* sobre cuáles son. Cuando una máquina sabe que no conoce el objetivo completo, tiene un incentivo para actuar con cautela, pedir permiso, aprender más sobre nuestras preferencias mediante la observación y deferir al control humano. En última instancia, queremos agentes que sean **probablemente benéficos** para los humanos. Volveremos a este tema en la Sección 1.5.

Probablemente benéfico

1.2 Fundamentos de la inteligencia artificial

En esta sección, se refiere con brevedad la historia de las disciplinas que contribuyeron con ideas, puntos de vista y técnicas a la IA. Como todas, esta historia se centra en un pequeño número de personas, eventos e ideas e ignora otros que fueron importantes, y se organiza en torno a una serie de preguntas. Ciertamente, no se busca dar la impresión de que estas preguntas son las únicas que abordan las disciplinas o que las disciplinas se han planteado hacia la IA como su fruto último.

[3] En uno de los primeros libros sobre ajedrez, Ruy López (1561) escribió: "Siempre coloca el tablero de manera que el Sol pegué en los ojos de tu oponente".

1.2.1 Filosofía

- ¿Se pueden utilizar reglas formales para sacar conclusiones válidas?
- ¿Cómo surge la mente a partir de un cerebro físico?
- ¿De dónde proviene el conocimiento?
- ¿Cómo lleva el conocimiento a la acción?

Aristóteles (384-322 a. C.) fue el primero en formular un leyes que rigen la parte racional de la mente. Desarrolló un sistema informal de silogismos para el razonamiento adecuado, que en principio permitía generar conclusiones de manera mecánica, dadas premisas iniciales.

Ramon Llull (c. 1232-1315) ideó un sistema de razonamiento publicado como *Ars Magna* o *The Great Art* (1305). Él intentó implementar su sistema utilizando un dispositivo mecánico real: un conjunto de ruedas de papel que podían girarse en diferentes permutaciones.

Leonardo da Vinci (1452-1519) diseñó, pero no construyó una calculadora mecánica; reconstrucciones de hoy demostraron que su diseño era funcional. La primera máquina calculadora conocida fue construida alrededor de 1623 por el científico alemán Wilhelm Schickard (1592-1635). Blaise Pascal (1623-1662) construyó la Pascalina en 1642 y escribió que "produce efectos más cercanos al pensamiento que las acciones de los animales". Gottfried W. Leibniz (1646-1716) construyó un dispositivo mecánico destinado a realizar operaciones sobre conceptos en lugar de números, pero su alcance era muy limitado. En su libro *Leviathan* (1651), Thomas Hobbes (1588-1679) sugirió la idea de una máquina pensante, un "animal artificial" en sus palabras, argumentando "Pero ¿qué es el corazón sino un resorte; y los nervios, muchas cuerdas; y las articulaciones, muchas ruedas?" Y sugirió que el razonamiento era como la computación numérica: "Porque 'razón' ... no es más que 'cálculo', es decir, sumar y restar".

Una cosa es decir que la mente opera, al menos en parte, según reglas lógicas o numéricas, y construir sistemas físicos que emulen algunas de esas reglas. Es otra cosa decir que la mente misma *es* tal sistema físico. René Descartes (1596-1650) dio la primera discusión clara de la distinción entre mente y materia. Notó que una concepción puramente física de la mente parece dejar poco espacio para el libre albedrío. Si la mente es gobernada enteramente por leyes físicas, entonces no tiene más libre albedrío que una roca "decidiendo" caer hacia abajo. Descartes fue partidario del **dualismo**. Sostenía que hay una parte de la mente humana (o alma o espíritu) que está fuera de la naturaleza, exenta de las leyes físicas. Los animales, por otro lado, no poseían esta dualidad; podían ser tratados como máquinas.

Dualismo

Una alternativa al dualismo es el **materialismo**, que sostiene que la operación del cerebro según las leyes de la física *constituye* la mente. El libre albedrío es la forma en que la percepción de las opciones disponibles aparece para la entidad que elige. Los términos **fisicalismo** y **naturalismo** también se usan para describir esta visión que se opone a lo sobrenatural.

Dada una mente física que opera el conocimiento, el siguiente problema es establecer la fuente del conocimiento. El movimiento **empirista**, iniciado con el *Novum Organum*[4] de Francis Bacon (1561-1626), fue caracterizado por una sentencia de John Locke (1632-1704): "Nada está en el entendimiento, que no estuviera primero en los sentidos".

Empirismo

El tratado de la naturaleza humana de David Hume (1711-1776) propuso lo que hoy se conoce como el principio de **inducción**: que las reglas generales se adquieren mediante la exposición a asociaciones repetidas entre sus elementos.

Inducción

A partir del trabajo de Ludwig Wittgenstein (1889-1951) y Bertrand Russell (1872-1970), el famoso Círculo de Viena (Sigmund, 2017), un grupo de filósofos y matemáticos que se reunían

[4] El *Novum Organum* es una actualización del *Organon* de Aristóteles, o instrumento del pensamiento.

en Viena en las décadas de 1920 y 1930, desarrolló la doctrina del **positivismo lógico**. Esta doctrina sostiene que todo conocimiento puede caracterizarse mediante teorías lógicas conectadas, en última instancia, a **oraciones de observación** que corresponden a entradas sensoriales; el positivismo lógico combina el racionalismo y el empirismo.

Positivismo lógico

Oraciones de observación

La **teoría de la confirmación** de Rudolf Carnap (1891-1970) y Carl Hempel (1905-1997) intentó analizar la adquisición de conocimiento a partir de la experiencia al cuantificar el grado de creencia que debería asignarse a las oraciones lógicas en función de su conexión con observaciones que las confirmen o refuten. El libro de Carnap *La construcción lógica del mundo* (1928) fue quizá la primera teoría de la mente como un proceso computacional.

Teoría de la confirmación

El elemento final en el panorama filosófico de la mente es la conexión entre conocimiento y acción. Esta cuestión es vital para la IA porque la inteligencia requiere acción así como razonamiento. Además, sólo al entender cómo se justifican las acciones podemos comprender cómo construir un agente cuyas acciones sean justificables (o racionales).

Aristóteles argumentó (en *De Motu Animalium*) que las acciones están justificadas por una conexión lógica entre los objetivos y el conocimiento del resultado de la acción:

> Pero ¿cómo sucede que el pensamiento esté a veces acompañado de acción y a veces no, a veces por movimiento y a veces no? Parece como si casi sucediera lo mismo que en el caso de razonar y hacer inferencias sobre objetos invariables. Pero en ese caso el fin es una proposición especulativa... mientras que aquí la conclusión que resulta de las dos premisas es una acción. Necesito una cubierta; un manto es una cubierta. Necesito un manto. Lo que necesito, tengo que hacer; necesito un manto. Tengo que hacer un manto. Y la conclusión, el "tengo que hacer un manto", es una acción.

En la Ética a *Nicomaco* (Libro III. 3, 1112b), Aristóteles elabora aún más sobre este tema, sugiriendo un algoritmo:

> No deliberamos sobre los fines, sino sobre los medios. Porque un médico no deliberaría si debería curar, ni un orador si debería persuadir. Suponen el fin y consideran cómo y por qué medios se logra, y si parece fácil y mejor producido por ello; mientras si se logra por un solo medio, consideran *cómo* se logrará por éste y por qué medios se logrará *esto*, hasta que llegan a la primera causa, y lo último en el orden del análisis parece ser lo primero en el orden del devenir. Y si nos encontramos con una imposibilidad, abandonamos la búsqueda, por ejemplo, si necesitamos dinero y esto no se puede obtener; pero si una cosa parece posible, intentamos hacerlo.

El algoritmo de Aristóteles fue implementado 2300 años después por Newell y Simon en su programa **General Problem Solver**. Ahora lo llamaríamos un sistema de planificación de regresión voraz (ver Capítulo 11). Los métodos basados en la planificación lógica para lograr objetivos definidos dominaron las primeras décadas de la investigación teórica en IA.

Pensar sólo en términos de acciones que logran objetivos es útil, pero a veces inaplicable. Por ejemplo, si hay varias formas diferentes de lograr un objetivo, debe haber alguna forma de elegir entre ellas. Más importante aún, puede que no sea posible lograr un objetivo con certeza, pero aun así se debe tomar alguna acción. ¿Cómo se decide entonces? Antoine Arnauld (1662), al analizar la noción de decisiones racionales en el juego, propuso una fórmula cuantitativa para maximizar el valor monetario esperado del resultado. Después, Daniel Bernoulli (1738) introdujo la noción más general de **utilidad** para capturar el valor interno y subjetivo de un resultado. La noción moderna de toma de decisiones racional bajo incertidumbre implica maximizar la utilidad esperada, como se explica en el Capítulo 16.

Utilidad

Utilitarismo

En ética y política pública, un tomador de decisiones debe considerar los intereses de múltiples individuos. Jeremy Bentham (1823) y John Stuart Mill (1863) promovieron la idea del **utilitarismo**: la toma de decisiones racional basada en la maximización de la utilidad debería aplicarse a todas las esferas de la actividad humana, incluidas las decisiones de política pública tomadas en nombre de muchos individuos. El utilitarismo es un tipo específico de **consecuencialismo**: la idea de que lo que está bien y lo que está mal es determinado por los resultados esperados de una acción.

Deontología ética

En contraste, Kant propuso en 1875 una teoría de ética basada en reglas o **deontología ética**, en ella "hacer lo correcto" no es determinado por los resultados, sino por leyes sociales universales que rigen las acciones permitidas, como "no mentir" o "no matar". Así, un utilitarista diría una mentira piadosa si el bien esperado supera el mal, pero un kantiano estaría obligado a no hacerlo, mentir es inherentemente incorrecto. Mill reconoció el valor de las reglas, pero las entendió como procedimientos de decisión eficientes compilados a partir del razonamiento de primeros principios sobre las consecuencias. Muchos sistemas de IA adoptan este enfoque.

1.2.2 Matemáticas

- ¿Cuáles son las reglas formales para sacar conclusiones válidas?
- ¿Qué se puede computar?
- ¿Cómo razonamos con información incierta?

Los filósofos establecieron algunas de las ideas fundamentales de la IA, pero el salto a una ciencia formal requirió la matematización de la lógica y la probabilidad, y la introducción de una nueva rama de las matemáticas: la computación.

Lógica formal

La idea de la **lógica formal** se remonta a los filósofos de la antigua Grecia, India y China, pero su desarrollo matemático realmente comenzó con el trabajo de George Boole (1815-1864), quien elaboró los detalles de la lógica proposicional, o booleana (Boole, 1847). En 1879, Gottlob Frege (1848-1925) extendió la lógica de Boole para incluir objetos y relaciones, creando la lógica de primer orden que se utiliza hoy en día.[5] Además de su papel central en el periodo inicial de la investigación en IA, la lógica de primer orden motivó el trabajo de Gödel y Turing que sustentó la computación en sí misma, como ahora explicamos.

Probabilidad

La teoría de la **probabilidad** se ve como una generalización de la lógica en situaciones con información incierta, una consideración fundamental para la IA. Gerolamo Cardano (1501-1576) fue el primero en plantear la idea de la probabilidad, describiéndola en términos de posibles resultados de eventos de juego. En 1654, Pascal, en carta a Pierre Fermat (1601-1665), mostró cómo predecir el futuro de un juego de azar en curso y asignar pagos promedio a los jugadores. La probabilidad se convirtió rápido en parte invaluable de las ciencias cuantitativas, ayudó a lidiar con medidas inciertas y teorías incompletas. Jacob Bernoulli (1654-1705, tío de Daniel Bernoulli), Pierre Laplace (1749-1827) y otros avanzaron en la teoría e introdujeron nuevos métodos estadísticos. Thomas Bayes (1702-1761) propuso una regla para actualizar probabilidades a la luz de nueva evidencia; esta regla es una herramienta crucial para los sistemas de IA.

Estadística

La formalización de la probabilidad, combinada con la disponibilidad de datos, llevó al surgimiento de la **estadística** como campo. Uno de los primeros usos fue el análisis de John Graunt usando datos del censo de Londres en 1662. Ronald Fisher es considerado el primer

[5] La notación propuesta por Frege para la lógica de primer orden (una combinación arcana de características textuales y geométricas) nunca se hizo popular.

estadístico moderno (Fisher, 1922). Él reunió las ideas de probabilidad, diseño experimental, análisis de datos y computación; en 1919, insistió en que no podía hacer su trabajo sin una calculadora mecánica llamada *MILLIONAIRE* (la primera calculadora que podía hacer multiplicaciones), aunque el costo de la calculadora fuera más alto que su salario anual (Ross, 2012).

La historia de la computación es tan antigua como la de los números, se cree que el primer **algoritmo** no trivial es el de Euclides para calcular el máximo común divisor. La palabra *algoritmo* proviene de Muhammad ibn Musa al-Khwarizmi, un matemático del siglo IX, cuyos escritos también introdujeron los números arábigos y el álgebra en Europa. Boole y otros discutieron algoritmos para la deducción lógica y, a fines del siglo XIX, realizaban esfuerzos para formalizar el razonamiento matemático general como deducción lógica.

Algoritmo

Kurt Gödel (1906-1978) mostró que existe un procedimiento efectivo para probar cualquier afirmación verdadera en la lógica de primer orden de Frege y Russell, pero que esta lógica no podía capturar el principio de inducción matemática necesario para caracterizar los números naturales. En 1931, Gödel demostró limitaciones en la deducción. Su **teorema de incompletitud** mostró que en cualquier teoría formal tan sólida como la aritmética de Peano (la teoría elemental de los números naturales), hay afirmaciones necesariamente verdaderas que no tienen una prueba dentro de la teoría.

Teorema de incompletitud de Gödel

Este resultado fundamental también se interpreta como una muestra de que algunas funciones en los enteros no pueden ser representadas por un algoritmo, es decir, no son calculables. Esto motivó a Alan Turing (1912-1954) a caracterizar exactamente qué funciones *son* **computables**, es decir, capaces de ser calculadas por un procedimiento efectivo. La tesis Church-Turing propone identificar la noción general de computabilidad con funciones computadas por una máquina de Turing (1936). Él también demostró que había algunas funciones que ninguna máquina de Turing computaba. Por ejemplo, ninguna máquina puede decir en *general* si un programa dado devolverá una respuesta a una entrada dada o se ejecutará para siempre.

Computable

Aunque la computabilidad es importante para comprender la computación, la noción de **tratabilidad** ha tenido un impacto aún mayor en la IA. Por ejemplo, un problema se llama intratable si el tiempo requerido para resolver instancias del problema crece exponencialmente con el tamaño de las instancias. La distinción entre el crecimiento polinomial y exponencial en complejidad fue enfatizada por primera vez a mediados de la década de 1960 (Cobham, 1964; Edmonds, 1965). Es importante porque el crecimiento exponencial significa que ni siquiera se pueden resolver instancias moderadamente grandes en un tiempo razonable.

Tratabilidad

La teoría de la **NP-completitud**, pionera de Cook (1971) y Karp (1972), es base para analizar la tratabilidad de problemas: cualquier clase de problema a la que se pueda reducir la clase de problemas NP-completos quizá sea intratable. (Pero no se ha demostrado que los problemas NP-completos sean necesariamente intratables, la mayoría de teóricos lo creen). Estos resultados contrastan con el optimismo de la prensa popular al recibir las primeras computadoras: "¡Súper cerebros electrónicos", "¡Más rápidos que Einstein!" A pesar de la creciente velocidad de las computadoras, el uso cuidadoso de recursos y la imperfección necesaria caracterizarán los sistemas inteligentes. Para ser breves, ¡el mundo es una instancia de problema *en extremo* grande!

NP-completitud

1.2.3 Economía

- ¿Cómo debemos tomar decisiones de acuerdo con nuestras preferencias?
- ¿Cómo deberíamos decidir cuando otros pueden no estar de acuerdo?
- ¿Cómo deberíamos decidir cuando la recompensa puede estar lejos en el futuro?

La ciencia de la economía se originó en 1776, cuando Adam Smith (1723-1790) publicó *Investigación sobre la naturaleza y causas de la riqueza de las naciones*. Smith propuso analizar las economías como compuestas por muchos agentes individuales que atienden sus propios intereses. Pero Smith no abogaba por la codicia financiera como una posición moral: su libro anterior (1759) *Teoría de los sentimientos morales* inicia señalando que la preocupación por el bienestar de los demás es un componente esencial de los intereses de cada individuo.

La mayoría de las personas relaciona la economía con el dinero, pero, de hecho, el primer análisis matemático de decisiones bajo incertidumbre, la fórmula del máximo valor esperado de Arnauld (1662), era sobre el valor monetario de las apuestas. Daniel Bernoulli (1738) notó que esta fórmula no parecía funcionar bien para cantidades mayores de dinero, como inversiones en expediciones comerciales marítimas. En su lugar, propuso un principio basado en la maximización de la utilidad esperada, y explicó las decisiones de inversión humanas al proponer que la utilidad marginal de una cantidad adicional de dinero disminuía a medida que se adquiría más dinero.

León Walras (pronunciado "Valras") (1834-1910) dio a la teoría de la utilidad una base más general en términos de preferencias entre apuestas sobre cualquier resultado (no sólo resultados monetarios). La teoría fue mejorada por Ramsey (1931) y luego por John von Neumann y Oskar Morgenstern en su libro *The Theory of Games and Economic Behavior* (1944). La economía ya no es estudio del dinero; más bien, es estudio de deseos y preferencias.

Teoría de la decisión

La **teoría de la decisión**, que combina la teoría de la probabilidad con la de la utilidad, da un marco formal y completo para decisiones individuales (económicas o no) tomadas bajo incertidumbre, es decir, cuando las descripciones probabilísticas capturan adecuadamente el entorno del tomador de decisiones. Adecuada para "grandes" economías donde cada agente no necesita prestar atención a las acciones de otros agentes como individuos. Para "pequeñas" economías, la situación se parece más a un juego: las acciones de un jugador pueden afectar significativamente la utilidad de otro (positiva o negativamente). El desarrollo de la teoría de juegos de von Neumann y Morgenstern (véase Luce y Raiffa, 1957) incluyó el resultado sorprendente de que, para algunos juegos, un agente racional debería adoptar políticas aleatorias (o que lo parezcan). A diferencia de la teoría de la decisión, la de juegos no ofrece una prescripción inequívoca para seleccionar acciones. En IA, las decisiones que implican múltiples agentes se estudian bajo el título de **sistemas multiagente** (Capítulo 18).

Los economistas, con excepciones, no abordaron la tercera pregunta: cómo tomar decisiones racionales cuando los pagos de las acciones son mediatos, sino que resultan de varias acciones *tomadas en secuencia*. Este tema se indagó en la **investigación de operaciones**, durante la Segunda Guerra Mundial, a partir de los esfuerzos en Gran Bretaña para optimizar las instalaciones de radar, que luego encontró múltiples aplicaciones civiles. El trabajo de Richard Bellman (1957) formalizó los **procesos de decisión de Markov**, una clase de problemas de decisión secuencial, presentados en el Capítulo 17, como **aprendizaje por refuerzo**, y en el Capítulo 22.

Investigación de operaciones

El trabajo en economía e investigación de operaciones ha contribuido a nuestra noción de agentes racionales, aunque por años la investigación en IA se desarrolló por caminos separados. Una razón fue la aparente complejidad de tomar decisiones racionales. Herbert Simon (1916-2001), pionero en IA y Premio Nobel de Economía en 1978 por su trabajo sobre los modelos basados en la **satisfacción**, es decir, sobre la toma de decisiones "suficientemente buenas", en lugar de calcular laboriosamente una decisión óptima, ofrecían una mejor descripción del comportamiento humano real (Simon, 1947). Desde la década de 1990, ha habido un resurgimiento del interés en técnicas de teoría de decisiones para la IA.

Satisfacción

1.2.4 Neurociencia

- ¿Cómo procesan información los cerebros?

Neurociencia

La **neurociencia** es el estudio del sistema nervioso, en particular del cerebro. Aunque la manera exacta en que el cerebro *permite* el pensamiento es uno de los grandes misterios de la ciencia, el hecho de que lo haga ha sido apreciado por miles de años debido a la evidencia de que golpes fuertes en la cabeza pueden llevar a la incapacidad mental. También se ha sabido durante mucho tiempo que los cerebros humanos en cierto modo diferentes; alrededor del 335 a. C., Aristóteles escribió: "De todos los animales, el hombre tiene el cerebro más grande en proporción a su tamaño".[6] Aun así, no fue hasta mediados del siglo XVIII que el cerebro fue ampliamente reconocido como la sede de la conciencia. Antes de eso, ésta se ubicaba en el corazón y el bazo.

La investigación de Paul Broca (1824-1880) sobre la afasia (deficiencia del habla) en pacientes con daño cerebral en 1861 inició el estudio de la organización funcional del cerebro al identificar un área localizada en el hemisferio izquierdo, hoy llamada área de Broca, responsable de la producción del habla.[7] Entonces se sabía que el cerebro consistía en gran medida de células nerviosas (**neuronas**), pero fue hasta 1873 que Camillo Golgi (1843-1926) desarrolló una técnica de tinción que permitió observar neuronas individuales (Figura 1.1). Esta técnica la utilizó Santiago Ramón y Cajal (1852-1934) en sus estudios pioneros sobre la organización neuronal.[8] Hoy se acepta ampliamente que las funciones cognitivas son resultado de la operación electroquímica de estas estructuras. Es decir, *un conjunto de células simples dirige pensamiento, acción y conciencia*. En palabras de John Searle (1992), *el cerebro causa la mente.*

Neurona

Hasta ahora hemos mapeado algunas relaciones entre áreas del cerebro y las partes del cuerpo que controlan o de las cuales reciben entrada sensorial. Tales mapeos pueden cambiar radicalmente en el transcurso de unas pocas semanas, y algunos animales parecen tener múltiples mapas. Aún no entendemos por completo cómo otras áreas asumen funciones cuando una está dañada. Casi no hay teoría sobre cómo se almacena una memoria individual o cómo operan las funciones cognitivas de nivel superior.

La medición de la actividad cerebral intacta inició en 1929 con la invención del electroencefalógrafo (EEG) por Hans Berger. El desarrollo de la resonancia magnética funcional (fMRI) (Ogawa *et al*., 1990; Cabeza y Nyberg, 2001) da a los neurocientíficos imágenes detalladas y sin precedentes de la actividad cerebral que permiten mediciones que corresponden a los intereses de procesos cognitivos en curso. Estos avances se complementan con mejoras en la grabación eléctrica de una sola célula de la actividad neuronal y con métodos de **optogenética** (Crick, 1999; Zemelman *et al*., 2002; Han and Boyden, 2007), que permiten la medición y el control de neuronas individuales modificadas para ser sensibles a la luz.

Optogenética

El desarrollo de **interfaces cerebro-máquina** (Lebedev y Nicolelis, 2006) para la detección y el control motor no sólo promete restaurar la función en personas discapacitadas, sino que también arroja luz sobre muchos aspectos de los sistemas neurales. Un hallazgo notable de este trabajo es que el cerebro es capaz de ajustarse para interactuar con éxito con un dispositivo externo, tratándolo de hecho como otro órgano sensorial o extremidad.

Interfaz cerebro máquina

---

[6] Se ha descubierto que la musaraña arborícola y algunas especies de aves superan la proporción cerebro/cuerpo humano.

[7] Muchos citan a Alexander Hood (1824) como una posible fuente previa.

[8] Golgi persistió en su creencia de que las funciones cerebrales se llevaban a cabo en un medio continuo en el que estaban incrustadas las neuronas, mientras que Cajal propugnaba la "doctrina neuronal". Ambos compartieron el Premio Nobel en 1906, pero pronunciaron discursos de aceptación mutuamente antagónicos.

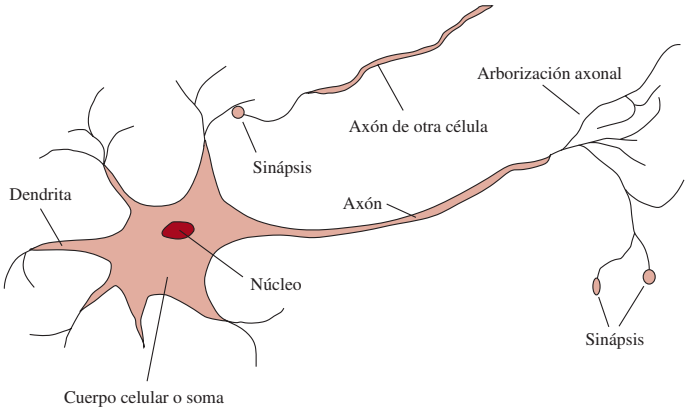


Figura 1.1 Partes de la célula nerviosa (neurona). Cada neurona consta de un cuerpo celular o soma que aloja un núcleo celular. Desde el cuerpo celular se ramifican varias fibras, llamadas dendritas, y una única fibra larga, un axón. Éste se extiende una distancia, mayor que la escala indicada en este diagrama. Por lo regular, un axón mide 1 cm de largo (100 veces el diámetro del cuerpo celular), y alcanza hasta 1 metro. Una neurona se conecta con 10 y hasta 100 000 otras neuronas en uniones llamadas sinapsis. Las señales se propagan de una neurona a otra mediante una compleja reacción electroquímica. Las señales controlan la actividad cerebral a corto plazo y permiten cambios a largo plazo en la conectividad de las neuronas. Se cree que estos mecanismos son la base del aprendizaje en el cerebro. La mayor parte del procesamiento de la información ocurre en la corteza cerebral, capa externa del cerebro. La unidad organizativa básica semeja una columna de tejido de casi 0.5 mm de diámetro, que aloja unas 20 000 neuronas y se extiende a toda la corteza (cerca de 4 mm en humanos).

Los cerebros y las computadoras digitales tienen propiedades diferentes. La Figura 1.2 muestra que las computadoras tienen un tiempo de ciclo un millón de veces más rápido que el de un cerebro. Éste compensa eso con más almacenamiento e interconexión que una computadora personal de alta gama, aunque las supercomputadoras más grandes igualan al cerebro en algunas métricas. Los futuristas calculan estos números, señalando una **singularidad** de que las computadoras alcanzan un nivel de rendimiento sobrehumano (Vinge, 1993; Kurzweil, 2005; Doctorow y Stross, 2012), que mejoran rápidamente aún más. Las comparaciones de números en bruto no son en especial informativas. Incluso con una computadora de capacidad virtualmente ilimitada, aún requerimos avances conceptuales adicionales en nuestra comprensión de la inteligencia (vea el Capítulo 28). En resumen, sin la teoría adecuada, las máquinas más rápidas sólo dan la respuesta incorrecta más rápido.

Singularidad

1.2.5 Psicología

- ¿Cómo piensan y actúan los seres humanos y los animales?

Los orígenes de la psicología científica suelen atribuirse al trabajo del físico alemán Hermann von Helmholtz (1821-1894) y su estudiante Wilhelm Wundt (1832-1920). Helmholtz aplicó el método científico al estudio de la visión humana, y su libro *Handbook of Physiological Optics* ha sido descrito como "el tratado más importante sobre física y fisiología de la visión humana" (Nalwa, 1993, p. 15). En 1879, Wundt abrió el primer laboratorio de psicología experimental en la Universidad de Leipzig. Él insistió en realizar experimentos minuciosos y controlados en los

	Supercomputadora	Computadora personal	Cerebro humano
Unidades computacionales	10^6 GPUs + CPUs	8 núcleos de CPU	10^6 columnas
	10^{15} transistores	10^{10} transistores	10^{11} neuronas
Unidades de almacenamiento	10^{16} bytes de RAM	10^{10} bytes de RAM	10^{11} neuronas
	10^{17} bytes de disco	10^{12} bytes de disco	10^{14} sinápsis
Tiempo de ciclo	10^{-9} segundos	10^{-9} segundos	10^{-3} segundos
Operaciones/segundo	10^{18}	10^{10}	10^{17}

Figura 1.2 Una comparación de una supercomputadora, Summit (Feldman, 2017); una computadora personal típica de 2019 y el cerebro humano. La capacidad de éste no ha cambiado mucho en miles de años, mientras que las supercomputadoras han mejorado desde megaFLOP en la década de 1960 a gigaFLOP en 1980, teraFLOP en 1990, petaFLOP en 2008 y exaFLOP en 2018 (1 exaFLOP = 1018 operaciones de punto flotante por segundo).

que sus trabajadores realizarían una tarea perceptiva o asociativa al hacer introspección sobre sus procesos de pensamiento. Estos controles contribuyeron a convertir la psicología en una ciencia, pero la naturaleza subjetiva de los datos hacía improbable que los experimentadores no confirmaran sus propias teorías.

Por otro lado, los biólogos que estudian el comportamiento animal carecían de datos introspectivos y desarrollaron una metodología objetiva, como describe H. S. Jennings (1906) en *Behavior of the Lower Organisms*. Al aplicar este punto de vista a los humanos, el **conductismo**, liderado por John Watson (1878-1958), rechazó cualquier teoría que implicara procesos mentales bajo el argumento de que la introspección no podía dar evidencia confiable. Los conductistas insistían en estudiar sólo medidas objetivas de los perceptos (*o estímulos*) de un animal y sus acciones resultantes (*o respuestas*). El conductismo descubrió mucho sobre ratas y palomas, pero tuvo menos éxito en entender a los humanos.

Conductismo

Para la **psicología cognitiva** el cerebro es como un dispositivo de procesamiento de información, lo que se remonta a William James (1842-1910). Helmholtz insistió en que la percepción involucraba una forma de inferencia lógica inconsciente. El punto de vista cognitivo fue eclipsado por el conductismo en Estados Unidos, aunque en la Unidad de Psicología Aplicada de Cambridge, con Frederic Bartlett (1886-1969), el modelado cognitivo floreció. *The Nature of Explanation*, de Bartlett Kenneth Craik (1943) estudiante y sucesor de Frederic, reestableció con energía la legitimidad de términos "mentales" como creencias y metas, argumentando que son igual de científicos que, por ejemplo, usar presión y temperatura para hablar sobre gases, a pesar de que éstos están compuestos por moléculas que no tienen ni uno ni otro.

Psicología cognitiva

Craik especificó tres pasos clave de un agente basado en el conocimiento: (1) el estímulo debe ser traducido en una representación interna, (2) ésta es controlada por procesos cognitivos para derivar nuevas representaciones internas, y (3) éstas, a su vez, se vuelven a traducir en acción. Explicó con claridad por qué este era un buen diseño para un agente:

> Si el organismo lleva un "modelo a pequeña escala" de la realidad externa y de sus propias posibles acciones en su cabeza, es capaz de probar varias alternativas, concluir cuál es la mejor de ellas, reaccionar ante situaciones futuras antes de que surjan, usar el conocimiento de eventos pasados en el tratamiento del presente y el futuro, y en todos sentidos reaccionar de manera más completa, segura y competente a las emergencias que enfrenta. (Craik, 1943)

Tras la muerte de Craik en un accidente de bicicleta en 1945, su trabajo fue continuado por Donald Broadbent, cuyo libro *Perception and Communication* (1958) fue uno de los primeros trabajos en modelar fenómenos psicológicos como procesamiento de información. Mientras, en Estados Unidos, el desarrollo de modelado por computadora llevó a la creación de la **ciencia cognitiva**. Podemos decir que este campo inició en un taller en septiembre de 1956 en el MIT, justo dos meses después de la conferencia en la que la IA misma "nació".

En el taller, se presentaron *The Magic Number Seven* de George Miller, *Three Models of Language* de Noam Chomsky, y *The Logic Theory Machine* de Allen Newell y Herbert Simon. Estos tres textos fundamentales mostraron cómo los modelos de computadora podrían usarse para abordar la psicología de la memoria, el lenguaje y el pensamiento lógico, respectivamente. Hoy es común (no universal) entre los psicólogos que "una teoría cognitiva debería ser como un programa de computadora" (Anderson, 1980); es decir, debería describir el funcionamiento de una función cognitiva en términos de procesamiento de información.

Inteligencia aumentada (ia)

Aquí consideraremos el campo de la **interacción humano-computadora** (HCI, por sus siglas en inglés) bajo la psicología. Doug Engelbart, pionero de HCI, defendió la idea de la **inteligencia aumentada** (ia) sobre la Inteligencia Artificial (IA). Él creía que las computadoras debían aumentar las habilidades humanas, no automatizar sus tareas. En 1968, la "madre de las demostraciones", Engelbart sacó a la luz el mouse de computadora, un sistema de ventanas, hipertexto y videoconferencias, todo en un esfuerzo por demostrar lo que los trabajadores del conocimiento humano podrían lograr colectivamente con inteligencia aumentada.

Hoy es más probable que veamos la ia y la IA como dos caras de la misma moneda, siendo la primera la que enfatiza el control humano y la segunda el comportamiento inteligente por parte de la máquina. Ambos son necesarios para que las máquinas sean útiles para los humanos.

1.2.6 Ingeniería informática

- ¿Cómo podemos construir una computadora eficiente?

La computadora electrónica digital moderna fue inventada de manera independiente y casi simultánea por científicos en tres países inmersos en la Segunda Guerra Mundial. La primera computadora *operativa* fue el electromecánico Heath Robinson,[9] construido en 1943 por el equipo de Alan Turing para descifrar mensajes alemanes. También en 1943, el mismo grupo desarrolló Colossus, una potente máquina de propósito general basada en tubos de vacío.[10] La primera computadora programable operativa fue la Z-3, inventada por Konrad Zuse en Alemania en 1941; él también inventó los números de punto flotante y el primer lenguaje de programación de alto nivel, Plankalkül. La primera computadora *electrónica*, la ABC, fue ensamblada por John Atanasoff y su estudiante Clifford Berry entre 1940 y 1942 en la Universidad Estatal de Iowa, con poco apoyo o reconocimiento; fue el ENIAC, desarrollado como parte de un proyecto militar secreto en la Universidad de Pensilvania por un equipo que incluía a John Mauchly y J. Presper Eckert, precursor más influyente de las computadoras modernas.

Ley de Moore

Desde entonces, cada generación de hardware de computadora ha traído un aumento en velocidad y capacidad y una disminución en precio, una tendencia capturada en la **ley de Moore**. El rendimiento se duplicaba casi cada 18 meses hasta 2005, cuando problemas de

[9] Una máquina compleja nombrada en honor a un caricaturista británico que representaba dispositivos caprichosos y absurdamente complicados para tareas cotidianas como untar mantequilla en pan tostado.

[10] En el periodo de posguerra, Turing quería utilizar estas computadoras para la investigación en IA; por ejemplo, creó un esquema del primer programa de ajedrez (Turing *et al.,* 1953), pero el gobierno británico bloqueó esta investigación.

disipación de energía llevaron a los fabricantes a multiplicar el número de núcleos de CPU en lugar de la velocidad del reloj. Las expectativas de hoy son que los aumentos futuros de funcionalidad provendrán de un paralelismo masivo, una curiosa convergencia con las propiedades del cerebro. También vemos nuevos diseños de hardware basados en la idea de que, al enfrentarnos a un mundo incierto, no necesitamos 64 bits de precisión en nuestros números; sólo 16 bits (como en el formato `bfloat16`) u 8 bits serán suficientes y permitirán un procesamiento más rápido.

Empezamos a ver hardware orientado a aplicaciones de IA, como la unidad de procesamiento gráfico (GPU, por sus siglas en inglés), la unidad de procesamiento tensorial (TPU) y el Wafer Scale Engine (WSE). De 1960 a 2012, la cantidad de potencia informática usada para entrenar las principales aplicaciones de aprendizaje automático seguía la ley de Moore. Desde 2012, las cosas cambiaron pues a 2018 aumentó 300 000 veces, lo que equivale a duplicarse cada 100 días (Amodei y Hernandez, 2018). Un modelo de aprendizaje automático que en 2014 tardaba un día completo en entrenarse, en 2018 sólo tarda dos minutos (Ying *et al.*, 2018). Aunque aún no es práctica, la **computación cuántica** ofrece la promesa de aceleraciones mayores para algunas subclases importantes de algoritmos de IA.

Cómputo cuántico

Claro, existían dispositivos de cálculo antes de la computadora electrónica. Las primeras máquinas automatizadas, datan del siglo XVII y las analizamos en la página 6. La primera *máquina programable* fue un telar, ideado en 1805 por Joseph Marie Jacquard (1752-1834), que usaba tarjetas perforadas para almacenar instrucciones para el patrón a tejer.

En el siglo XIX, Charles Babbage (1792-1871) diseñó dos máquinas de cómputo, ninguna de las cuales completó. La máquina de diferencias, diseñada para calcular tablas matemáticas y para proyectos científicos y de ingeniería. Finalmente, en 1991 fue construida y se mostró su funcionamiento (Swade, 2000). La máquina analítica de Babbage, más ambiciosa, incluía memoria direccionable, programas almacenados basados en las tarjetas perforadas de Jacquard, y saltos condicionales, fue la primera capaz de computación universal.

La colega de Babbage, Ada Lovelace, hija del poeta Lord Byron, comprendió su potencial, describiéndola como "una máquina pensante o... una máquina de razonamiento", capaz de razonar sobre "todos los temas en el universo" (Lovelace, 1843). Ella también anticipó los ciclos de hype de la IA, escribiendo: "Es deseable protegerse contra la posibilidad de ideas exageradas que podrían surgir sobre los poderes de la máquina analítica". Desafortunadamente, las máquinas de Babbage y las ideas de Lovelace fueron olvidadas.

La IA también tiene una deuda con la ciencia del software de la informática, que ha creado los sistemas operativos, lenguajes de programación y herramientas necesarias para escribir programas modernos (y documentos sobre ellos). Pero esta es un área donde la deuda se ha saldado: el trabajo en IA ha sido pionero de muchas ideas que regresado a la informática convencional, incluido el tiempo compartido, los intérpretes interactivos, las computadoras personales con ventanas y ratones, entornos de desarrollo rápidos, el tipo de datos lista enlazada, la gestión automática de almacenamiento y conceptos clave de programación simbólica, funcional, declarativa y orientada a objetos.

1.2.7 Control y cibernética

- ¿Cómo pueden los artefactos operar bajo su propio control?

Ktesibios de Alejandría (250 a. C.) construyó la primera máquina de autocontrol: un reloj de agua con un regulador que mantenía un flujo constante. Esta invención redefinió lo que un artefacto podía hacer. Antes, sólo los seres vivos podían modificar su comportamiento en respuesta a los cambios en el entorno. Otros ejemplos de sistemas de control de retroalimentación auto-regula-

dora son el gobernador de la máquina de vapor, de James Watt (1736–1819), y el termostato, de Cornelis Drebbel (1572–1633), quien también inventó el submarino. James Clerk Maxwell (1868) inició la teoría matemática de los sistemas de control.

Teoría de control

Una figura central en el desarrollo de la **teoría del control** después de la guerra fue Norbert Wiener (1894-1964), un brillante matemático colaborador de Bertrand Russell, entre otros, antes de interesarse en sistemas de control biológicos y mecánicos y su conexión con la cognición. Igual que Craik (quien también usó sistemas de control como modelos psicológicos), Wiener, Arturo Rosenblueth y Julian Bigelow desafiaron la ortodoxia del conductismo (Rosenblueth *et al.*, 1943), pues consideraban que el comportamiento intencional surgía de un mecanismo regulador que minimizaba el "error", la diferencia entre estado actual y estado objetivo. Para 1950, Wiener, junto con Warren McCulloch, Walter Pitts y John von Neumann, organizaron una serie de conferencias en las que exploraron nuevos modelos matemáticos y computacionales de la cognición. El libro de Wiener, *Cybernetics* (1948), fue un éxito y despertó en el público la posibilidad de máquinas inteligentes artificialmente.

Cibernética

Mientras, en Gran Bretaña, W. Ross Ashby, pionero en ideas similares (Ashby, 1940), Alan Turing, Grey Walter y otros crearon el Ratio Club para "quienes tenían las ideas de Wiener antes del libro de éste". *Design for a Brain* de Ashby (1948, 1952) desarrolló su idea de que la inteligencia podría crearse mediante dispositivos **homeostáticos** que contienen bucles de retroalimentación apropiados para lograr una conducta adaptativa estable.

Homeostática

La teoría del control moderna, en especial la rama de control óptimo estocástico, tiene como objetivo diseñar sistemas que maximicen una **función de costo** con el tiempo. Esto casi coincide con el modelo estándar de IA: diseñar sistemas que se comporten de manera óptima. Pero ¿por qué la IA y la teoría del control son dos campos diferentes, a pesar de las estrechas conexiones entre sus fundadores? La respuesta radica en el acoplamiento entre las técnicas matemáticas familiares para los participantes y los conjuntos propios de problemas abarcados en cada punto de vista. El cálculo y el álgebra matricial, herramientas de la teoría del control, se prestan para sistemas que pueden ser descritos por conjuntos fijos de variables continuas, mientras que la IA se fundó como una forma de escapar de estas limitaciones percibidas. Las herramientas de inferencia lógica y computación permitieron a los investigadores en IA considerar problemas como lenguaje, visión y planificación simbólica, por completo fuera del alcance del teórico del control.

Función de costo

1.2.8 Lingüística

- ¿Cómo se relaciona el lenguaje con el pensamiento?

En 1957, B. F. Skinner publicó *Verbal Behavior*, un relato completo y detallado del enfoque conductista para el aprendizaje del lenguaje, escrito por el experto más destacado en el campo. Curiosamente, una reseña del libro se hizo tan conocida como el libro mismo, y sirvió para casi matar el interés en el conductismo. El autor de la reseña fue el lingüista Noam Chomsky, quien acababa de publicar un libro sobre su propia teoría, *Syntactic Structures*. Chomsky señaló que la teoría conductista no abordaba la noción de creatividad en el lenguaje; no explicaba cómo los niños podían entender y crear oraciones que nunca habían escuchado antes. La teoría de Chomsky, basada en modelos sintácticos que se remontan al lingüista indio Panini (350 a. C.), podía explicar esto, y a diferencia de las teorías anteriores, era lo suficientemente formal como para poder ser programada.

Lingüística computacional

La lingüística moderna y la IA "nacieron" casi al mismo tiempo, y crecieron juntas, intersectándose en un campo híbrido llamado **lingüística computacional** o **procesamiento de lenguaje natural**. El problema de entender el lenguaje resultó ser mucho más complejo

de lo que parecía en 1957. Lograr su entendimiento requiere una comprensión del tema y del contexto, no sólo comprender la estructura de las oraciones. Parece obvio, pero esto no se apreció con claridad hasta la década de 1960. Gran parte del trabajo inicial en **representación del conocimiento** (el estudio de cómo poner el conocimiento en una forma en que una computadora pueda razonar) estaba ligado al lenguaje e informado por la investigación en lingüística que, a su vez, se conectaba con décadas de trabajo en el análisis filosófico del lenguaje.

1.3 La Historia de la Inteligencia Artificial

Una síntesis de los hitos en la historia de la IA es listar a los Premios Turing: Marvin Minsky (1969) y John McCarthy (1971), por definir sus fundamentos con base en representación y razonamiento; Ed Feigenbaum y Raj Reddy (1994) por desarrollar sistemas expertos que codifican el conocimiento humano para resolver problemas del mundo real; Judea Pearl (2011) por desarrollar técnicas de razonamiento probabilístico que abordan la incertidumbre de manera fundamentada; y, al final, Yoshua Bengio, Geoffrey Hinton y Yann LeCun (2019) por hacer que el "aprendizaje profundo" (redes neuronales multicapa) sea parte crítica de la informática moderna. A partir de aquí se detallará cada fase de la historia de la IA.

1.3.1 Incepción de la inteligencia artificial (1943–1956)

Por lo regular, el primer trabajo reconocido como IA fue realizado por Warren McCulloch y Walter Pitts (1943). Inspirados en el modelado matemático del asesor de Pitts, Nicolas Rashevsky (1936, 1938), se basaron en tres fuentes: conocimiento de la fisiología básica y función de las neuronas en el cerebro; análisis formal de la lógica proposicional de Russell y Whitehead, y la teoría de la computación de Turing. Propusieron un modelo de neuronas artificiales en el que cada neurona se caracteriza como "activada" o "desactivada", con un cambio a "activada" en respuesta a la estimulación por un número suficiente de neuronas vecinas. El estado de una neurona fue concebido como "equivalente en términos fácticos a una proposición que proponía su estímulo adecuado". Mostraron que cualquier función computable podría ser calculada por alguna red de neuronas conectadas y que todos los conectivos lógicos (AND, OR, NOT, etc.) podrían implementarse mediante estructuras de red simples. McCulloch y Pitts también sugirieron que las redes definidas adecuadamente podrían aprender. Donald Hebb (1949) demostró una regla de actualización simple para modificar las fuerzas de conexión entre neuronas. Su regla, hoy llamada **aprendizaje hebbiano**, aún es un modelo influyente.

Aprendizaje hebbiano

En 1950, dos estudiantes de pregrado de Harvard, Marvin Minsky (1927-2016) y Dean Edmonds, construyeron la primera computadora de red neuronal: el *SNARC*. Usaba 3 000 tubos de vacío y un mecanismo de piloto automático, sobrante de un bombardero B-24, para simular una red de 40 neuronas. Luego, en Princeton, Minsky estudió computación universal en redes neuronales. Su comité de doctorado estaba escéptico sobre si este tipo de trabajo debería considerarse matemáticas, pero se dice que von Neumann dijo: "Si no lo es ahora, lo será algún día".

Hubo otros ejemplos de trabajos tempranos caracterizados como IA, incluidos dos programas de juego de damas desarrollados independientemente en 1952 por Christopher Strachey en la Universidad de Manchester y por Arthur Samuel en *IBM*. Pero la visión de Alan Turing fue la más influyente. Dio conferencias sobre el tema tan temprano como 1947 en la London Mathematical Society y articuló una agenda persuasiva en su artículo de 1950 "Computing

Machinery and Intelligence". En él, introdujo la prueba de Turing, el aprendizaje automático, los algoritmos genéticos y el aprendizaje por refuerzo. Abordó muchas objeciones a la posibilidad de IA, como se describe en el Capítulo 27. También sugirió que sería más fácil crear IA a nivel humano desarrollando algoritmos de aprendizaje y luego enseñando a la máquina, en lugar de programar su inteligencia. En conferencias posteriores advirtió que lograr este objetivo podría no ser lo mejor para la raza humana.

En 1955, John McCarthy de Dartmouth College convenció a Minsky, Claude Shannon y Nathaniel Rochester para que lo ayudaran a reunir investigadores estadounidenses interesados en teoría de autómatas, redes neuronales y estudio de la inteligencia. Organizaron un taller de dos meses en Dartmouth en el verano de 1956. Lograron 10 asistentes, incluidos Allen Newell y Herbert Simon, de Carnegie Tech,[11] Trenchard More de Princeton, Arthur Samuel, de IBM, y Ray Solomonoff y Oliver Selfridge, del MIT. La propuesta dice:[12]

> Proponemos un estudio de dos meses, con 10 participantes, sobre IA durante el verano de 1956 en Dartmouth College, en Hanover, New Hampshire. El estudio será sobre la base de la conjetura de que cada aspecto del aprendizaje o cualquier otra característica de la inteligencia puede, en principio, describirse de manera tan precisa que una máquina pueda simularlo. Se intentará encontrar cómo hacer que las máquinas utilicen el lenguaje, formen abstracciones y conceptos, resuelvan tipos de problemas hoy reservados a humanos, y se mejoren a sí mismas. Creemos en lograr un avance significativo en uno o más de estos problemas si un grupo bien seleccionado de científicos trabaja en ello durante un verano.

A pesar de tal optimismo, el taller de Dartmouth no logró avance. Newell y Simon presentaron quizás el trabajo más maduro, un sistema de demostración de teoremas matemáticos llamado Teorista Lógico (LT, por sus siglas en inglés). Simon afirmó: "Hemos inventado un programa de computadora capaz de pensar de forma no numérica, y así hemos resuelto el venerable problema mente-cuerpo".[13] Después del taller, el programa pudo probar la mayoría de los teoremas en el Capítulo 2 de *Principia Mathematica* de Russell y Whitehead. Se dice que Russell se alegró cuando supo que LT había encontrado una demostración para un teorema más corta que la de *Principia*. Los editores del *Journal of Symbolic Logic* estaban menos impresionados; rechazaron un artículo coescrito por Newell, Simon y el Teorista Lógico.

1.3.2 Entusiasmo temprano, grandes expectativas (1952–1969)

El tono intelectual de la década de 1950 prefería creer que "una máquina nunca puede hacer *X*". (Vea el Capítulo 27 para una larga lista de *X* recopilada por Turing). Claro que los investigadores de IA respondieron demostrando un *X* tras otro. Se centraron en tareas consideradas indicativas de inteligencia en humanos, como juegos, rompecabezas, matemáticas y

---

[11] Ahora Universidad Carnegie Mellon (CMU, por sus siglas en inglés).

[12] Este fue el primer uso oficial del término *inteligencia artificial* de McCarthy. Quizá "racionalidad computacional" hubiera sido más precisa y menos amenazador, pero "IA" se ha mantenido. En el 50 aniversario de la conferencia de Dartmouth, McCarthy declaró que se resistía a los términos "computadora" o "computacional" en deferencia a Norbert Wiener, quien promovía dispositivos cibernéticos analógicos en lugar de computadoras digitales.

[13] Newell y Simon inventaron un lenguaje de procesamiento de listas, IPL, para escribir LT (teoristas lógicos). No tenían compilador y lo tradujeron manualmente a código de máquina. Para evitar errores, trabajaron en paralelo, pasándose números binarios entre sí mientras escribían cada instrucción para asegurarse de que coincidieran.

pruebas de coeficiente intelectual. John McCarthy nombró este periodo como era de "¡Mira, mamá, sin manos!".

Newell y Simon siguieron su éxito con LT con el Solucionador General de Problemas (GPS). A diferencia de LT, este fue diseñado para imitar los protocolos de resolución de problemas humanos. Dentro de la clase limitada de rompecabezas que podía manejar, resultó que el orden en que el programa consideraba subobjetivos y posibles acciones era similar a como los humanos abordaban los mismos problemas. Así, GPS fue quizás el primer programa en incorporar el enfoque de "pensamiento humano". El éxito de GPS y programas posteriores como modelos de cognición llevó a Newell y Simon (1976) a formular la famosa hipótesis del **sistema de símbolos físicos:** "un sistema de símbolos físicos tiene medios necesarios y suficientes para la acción inteligente general". Lo que querían decir es que cualquier sistema (humano o máquina) que exhiba inteligencia debe operar con estructuras de datos compuestas por símbolos. Veremos que esta hipótesis ha sido desafiada desde muchas direcciones.

Sistema de símbolos físicos

En *IBM*, Nathaniel Rochester y sus colegas produjeron unos de los primeros programas de IA. Herbert Gelernter (1959) construyó el Probador de Teoremas de Geometría, capaz de demostrar teoremas complicados para muchos estudiantes de matemáticas. Este trabajo fue precursor de modernos probadores de teoremas matemáticos.

De todo el trabajo exploratorio realizado en este periodo, quizás el más influyente a largo plazo fue el de Arthur Samuel en el juego de damas. Usando métodos que hoy llamamos aprendizaje por refuerzo (vea el Capítulo 22), los programas de Samuel aprendieron a jugar a nivel aficionado experimentado. Así, desmintió la idea de que las computadoras sólo hacen lo que se les dice: su programa rápido aprendió a jugar mejor que su creador. Luego fue demostrado en televisión en 1956 y causó una fuerte impresión. Al igual que Turing, Samuel tuvo problemas para encontrar tiempo de computadora. Trabajando por la noche, utilizó máquinas que aún estaban en pruebas en la fábrica de IBM. El programa de Samuel fue precursor de sistemas posteriores como TD-GAMMON (Tesauro, 1992), que estuvo entre los mejores jugadores de backgammon del mundo, y ALPHAGO (Silver *et al.*, 2016), que sorprendió al mundo al derrotar al campeón mundial humano en *Go* (vea el Capítulo 5).

En 1958, John McCarthy realizó dos importantes contribuciones a la IA. En el Memo No. 1 del Laboratorio de IA del MIT, definió el lenguaje de alto nivel **Lisp**, que se convertiría en el lenguaje de programación de IA dominante los próximos 30 años. En el artículo *Programs with Common Sense* propuso un concepto para sistemas de IA basados en conocimiento y razonamiento. En él describe al Tomador de Consejos, un programa hipotético que encarnaría el conocimiento general del mundo y lo usaría para derivar planes de acción. El concepto se ilustró con axiomas lógicos simples, suficientes para generar un plan para conducir al aeropuerto. El programa también fue diseñado para aceptar nuevos axiomas en el curso normal de su funcionamiento, lo que le permitía adquirir competencia en nuevas áreas sin *necesidad de ser reprogramado*. Así, el Tomador de Consejos incorporaba los principios centrales de representación y razonamiento del conocimiento: útil para tener una representación formal y explícita del mundo y de su funcionamiento, y manipular esa representación con procesos deductivos. El documento influyó tanto a la IA que aún es relevante.

Lisp

Ese año lo marcó el traslado de Marvin Minsky al *MIT*. Pero su colaboración con McCarthy no duró. Éste enfatizaba la representación y el razonamiento en lógica formal, mientras que a Minsky le interesaba que los programas funcionaran y, así, desarrolló una perspectiva antilógica. En 1963, McCarthy inició el laboratorio de IA en Stanford; su plan de usar la lógica para construir el último Tomador de Consejos fue superado por el método de resolución de J. A. Robinson en 1965 (un algoritmo completo de demostración de teoremas para la lógica de primer orden; vea el Capítulo 9). El trabajo en Stanford enfatizaba métodos generales para el razonamiento lógico. Las aplicaciones de la lógica incluían el sistema

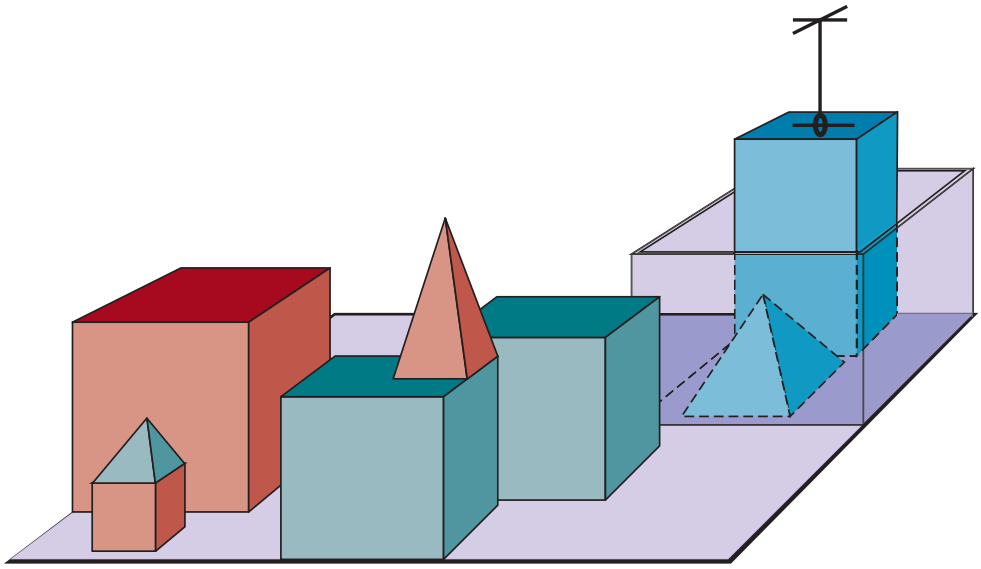


Figura 1.3 Una escena del mundo de los bloques. *SHRDLU* (Winograd, 1972) acaba de completar el comando "Encuentra un bloque más alto que el que estás sosteniendo y ponlo en la caja".

de respuesta a preguntas y de planificación de Cordell Green (Green, 1969b) y el proyecto de robótica Shakey en el Instituto de Investigación de Stanford (SRI). Este proyecto, analizado en el Capítulo 26, fue el primero en demostrar la integración completa del razonamiento lógico y la actividad física.

Micromundo

En el MIT, Minsky supervisó estudiantes que eligieron para resolver problemas limitados que requerían inteligencia. Estos dominios limitados se conocieron como **micromundos**. El programa SAINT de James Slagle (1963) resolvió problemas de integración de cálculo en forma cerrada, típicos de cursos universitarios de primer año. El programa ANALOGY de Tom Evans (1968) resolvió problemas de analogía geométrica usados en pruebas de coeficiente intelectual. El programa STUDENT de Daniel Bobrow (1967) resolvió problemas algebraicos, como:

> Si la cantidad de clientes que Tom obtiene es el doble del cuadrado del 20 por ciento del número de anuncios que publica, y el número de anuncios que publica es 45, ¿cuál es la cantidad de clientes que Tom obtiene?

Mundo de los bloques

El micromundo más famoso es el **mundo de bloques**, un conjunto de bloques sólidos colocados sobre una mesa (o simulación de ésta), como se ve en la Figura 1.3. Una tarea típica en este mundo es reorganizar los bloques de cierta manera, usando una mano robótica que puede recoger un bloque a la vez. El mundo de los bloques albergó el proyecto de visión de David Huffman (1971), el trabajo de visión y propagación de restricciones de David Waltz (1975), la teoría del aprendizaje de Patrick Winston (1970), el programa de comprensión del lenguaje natural de Terry Winograd (1972), y el planificador de Scott Fahlman (1974).

El trabajo temprano basado en las redes neuronales de McCulloch y Pitts floreció. El trabajo de Shmuel Winograd y Jack Cowan (1963) mostró cómo un gran número de elementos podían

representar colectivamente un concepto individual, con más robustez y paralelismo. Bernie Widrow mejoró los métodos de aprendizaje de Hebb (Widrow y Hoff, 1960; Widrow, 1962), al igual que Frank Rosenblatt (1962) con sus **perceptrones.** Widrow llamó a sus redes **adaline** (Adaptive Linear Neuron). El **teorema de convergencia del perceptrón** (Block *et al.*, 1962) indica que el algoritmo de aprendizaje ajusta las fuerzas de conexión de un perceptrón para que coincidan con cualquier dato de entrada, si existe tal coincidencia.

1.3.3 Una dosis de realidad (1966–1973)

Desde el principio, los investigadores de IA no dudaron en hacer predicciones sobre sus próximos éxitos. La siguiente declaración de Herbert Simon en 1957 se cita a menudo:

> No es mi objetivo sorprenderlos o deslumbrarlos, pero la forma más simple en que puedo resumir es decir que ahora hay en el mundo máquinas que piensan, que aprenden y que crean. Además, su capacidad para hacer estas cosas aumentará rápidamente hasta que, en un futuro visible, el rango de problemas que puedan manejar será coextensivo con el rango al que se ha aplicado la mente humana.

Si bien el término "futuro visible" es vago, vemos que Simon hizo predicciones concretas: que dentro de 10 años una computadora sería campeona de ajedrez y que un teorema matemático significativo sería demostrado por una máquina. Ambas predicciones se cumplieron (o casi) en 40 años en lugar de 10. La excesiva confianza de Simon se debió al prometedor rendimiento de los primeros sistemas de IA en ejemplos simples. Sin embargo, en casi todos los casos, estos primeros sistemas fallaron al enfrentar problemas más difíciles.

Hubo dos razones principales de este fracaso. La primera fue que muchos de los primeros sistemas de IA se basaban en la "introspección informada" sobre cómo los humanos realizan una tarea, en lugar de en un análisis cuidadoso de la tarea, lo que significa ser una solución y lo que un algoritmo necesitaría hacer para producir tales soluciones de manera confiable.

La segunda razón del fracaso fue la falta de aprecio por la intratabilidad de muchos de los problemas que la IA intentaba resolver. Los primeros sistemas de resolución de problemas funcionaban probando diferentes combinaciones de pasos hasta encontrar la solución. Esta estrategia funcionó al inicio porque los micromundos contenían pocos objetos y, por lo tanto, pocas acciones posibles y secuencias de solución cortas. Antes del desarrollo de la teoría de la complejidad computacional, se pensaba que "escalar" hacia problemas mayores sólo era cuestión de hardware más rápido y memorias más grandes. El optimismo que acompañó al desarrollo de la demostración de teoremas de resolución, se vio pronto amortiguado cuando los investigadores no lograron demostrar teoremas que involucraran más que unas pocas docenas de hechos. *El que un programa encuentre una solución en principio no significa que éste contenga ninguno de los mecanismos necesarios para encontrarla en la práctica.*

La ilusión de poder computacional ilimitado no se restringió a programas de resolución de problemas. Los primeros experimentos en **computación evolutiva** (hoy **programación genética**) (Friedberg, 1958; Friedberg *et al.*, 1959) se basaron en la creencia de que haciendo una serie adecuada de pequeñas mutaciones a un programa de código de máquina, se puede generar un programa con buen rendimiento para cualquier tarea particular. La idea era probar mutaciones aleatorias con un proceso de selección para preservar mutaciones que parecían útiles. A pesar de miles de horas de tiempo de CPU, casi no se demostró progreso.

Cómputo evolutivo

La incapacidad para enfrentar la "explosión combinatoria" fue una de las principales críticas del informe Lighthill (Lighthill, 1973) a la IA, la cual fue base para la decisión del gobierno

británico de poner fin al apoyo a la investigación en IA en todas las universidades, excepto en dos. (La tradición oral pinta un cuadro algo diferente y más colorido, con ambiciones políticas y animosidades personales cuya descripción está fuera de lugar).

Una tercera dificultad surgió debido a limitaciones fundamentales en las estructuras básicas usadas para generar comportamiento inteligente. Así, el libro *Perceptrons* (1969) de Minsky y Papert demostró que, aunque los perceptrones (forma simple de red neuronal) demostraban que aprendían cualquier cosa que fueran capaces de representar, muy poco representaban. En particular, un perceptrón de dos entradas no podía entrenarse para reconocer cuando sus dos entradas eran diferentes. Aunque sus resultados no se aplicaban a redes más complejas de múltiples capas, los fondos para la investigación en redes neuronales pronto se redujeron casi a nada. Irónicamente, los nuevos algoritmos de aprendizaje de retropropagación que causarían un gran resurgimiento en la investigación de redes neuronales a finales de los años ochenta, y en la década de 2010, ya habían sido desarrollados en contextos de los años sesenta.

1.3.4 Sistemas expertos (1969–1986)

Método débil

La imagen de resolución de problemas surgido al inicio de la investigación en IA era la de un mecanismo de búsqueda de propósito general para unir pasos de razonamiento elementales que encontraran soluciones completas. Tales enfoques, llamados **métodos débiles,** aunque generales, no se escalan a instancias de problemas grandes o difíciles. La alternativa es usar conocimientos más potentes y específicos del dominio que den pasos de razonamiento más grandes que manejen con más facilidad los casos que ocurren típicamente en áreas específicas de la experticia. Se podría decir que, para resolver un problema difícil, casi debes conocer la respuesta de antemano.

El programa DENDRAL (Buchanan *et al.*, 1969) fue ejemplo temprano de este enfoque. Se desarrolló en Stanford, donde Ed Feigenbaum (exalumno de Herbert Simon), Bruce Buchanan (filósofo convertido en informático) y Joshua Lederberg (genetista ganador del Premio Nobel) se unieron para resolver el problema de inferir la estructura molecular a partir de la información dada por un espectrómetro de masas. La entrada al programa consiste en la fórmula elemental de la molécula (por ejemplo, $C_6H_{13}NO_2$) y el espectro de masas que da las masas de los diversos fragmentos de la molécula, generados cuando se bombardea con un haz de electrones. Así, el espectro de masas podría contener un pico en $m = 15$, correspondiente a la masa de un fragmento de metilo (CH_3).

La versión ingenua del programa generaba todas las estructuras posibles consistentes con la fórmula, y luego predecía qué espectro de masas se observaría para cada una, comparándolo con el espectro real. Como uno podría esperar, esto es intratable incluso para moléculas de tamaño moderado. Los investigadores de DENDRAL consultaron a químicos analíticos y descubrieron que trabajaban buscando patrones bien conocidos de picos en el espectro que sugerían subestructuras comunes en la molécula. Por ejemplo, la siguiente regla se utiliza para reconocer un subgrupo cetona (C = O) (con un peso molecular de 28):

> **if** M es la masa de la molécula entera y hay dos picos en x_1 y x_2 tales que
> (a) $x_1 + x_2 = M + 28$; (b) $x_1 - 28$ es un pico alto; (c) $x_2 - 28$ es un pico alto; y
> (d) al menos uno de x_1 y x_2 es alto
> **then** hay un subgrupo cetona.

Reconocer que la molécula contiene una subestructura particular reduce mucho el número de candidatos posibles. Según sus autores, DENDRAL fue poderoso porque incorporó el conocimiento relevante de la espectroscopia de masas no en forma de principios básicos, sino

en "recetas de cocina" eficientes (Feigenbaum *et al.*, 1971). La importancia de DENDRAL es que fue el primer sistema exitoso intensivo de conocimiento intensivo: su experiencia se derivaba de grandes cantidades de reglas de propósito especial. En 1971, Feigenbaum y otros en Stanford iniciaron el Proyecto de Programación Heurística (HPP, por sus siglas en inglés) para investigar en qué medida se aplicaría la nueva metodología de **sistemas expertos** a otras áreas.

Sistemas expertos

El siguiente esfuerzo importante fue el sistema MYCIN para diagnosticar infecciones sanguíneas. Con casi 450 reglas, MYCIN se desempeñó tan bien como algunos expertos y mejor que médicos residentes. Presentaba dos diferencias principales con DENDRAL. Primero, a diferencia de las reglas de DENDRAL, no existía un modelo teórico general del cual se pudieran deducir las reglas de MYCIN. Tuvieron que ser adquiridas en extensas entrevistas con expertos. Segundo, las reglas debían reflejar la incertidumbre asociada con el conocimiento médico. MYCIN incorporaba un cálculo de incertidumbre, **factores de certeza** (vea el Capítulo 13), que parecían encajar (entonces) con la forma en que los médicos evaluaban el impacto de la evidencia en el diagnóstico.

Factores de certeza

El primer sistema experto comercial exitoso, R1, comenzó a operar en Digital Equipment Corporation (McDermott, 1982). El programa ayudaba a configurar pedidos de nuevos sistemas informáticos; para 1986, ahorraba a la empresa un estimado de 40 millones de dólares al año. Para 1988, el grupo de IA de DEC tenía 40 sistemas expertos desplegados, y con más en camino. DuPont tenía 100 en uso y 500 en desarrollo. Las principales corporaciones estadounidenses tenían su propio grupo de IA y usaban o investigaba sistemas expertos.

La importancia del conocimiento del dominio también fue evidente en el área de la comprensión del lenguaje natural. A pesar del éxito del sistema SHRDLU de Winograd, sus métodos no se extendían a tareas más generales: para problemas como la resolución de ambigüedades, usaba reglas simples que dependían del pequeño alcance del mundo de los bloques.

Varios investigadores, incluidos Eugene Charniak en el MIT y Roger Schank en Yale, sugirieron que una comprensión robusta del lenguaje requeriría conocimiento general sobre el mundo y un método general para usar ese conocimiento. (Schank fue más allá: "No existe tal cosa como la sintaxis", lo que molestó a muchos lingüistas pero sirvió para iniciar debate). Schank y sus estudiantes construyeron una serie de programas (Schank y Abelson, 1977; Wilensky, 1978; Schank y Riesbeck, 1981) que tenían todos la tarea de comprender el lenguaje natural. Pero el énfasis estaba menos en el lenguaje en sí y más en los problemas de representar y razonar con el conocimiento requerido para la comprensión del lenguaje.

El crecimiento generalizado de aplicaciones a problemas del mundo real llevó al desarrollo de una amplia gama de herramientas de representación y razonamiento. Algunas se basaban en la lógica, como el lenguaje Prolog se volvió popular en Europa y Japón, y la familia PLANNER en Estados Unidos. Otros, siguiendo la idea de **marcos** de Minsky (1975), adoptaron un enfoque más estructurado, ensamblando hechos sobre tipos particulares de objetos y eventos, organizando los tipos en una jerarquía taxonómica grande, análoga a una taxonomía biológica.

Marcos

En 1981, Japón anunció el proyecto "Quinta Generación", un plan de 10 años para construir masivamente computadoras paralelas e inteligentes que ejecutaran Prolog. El presupuesto superaba 1.3 billones de dólares en dinero de hoy. En respuesta, Estados Unidos formó la Corporación de Tecnología de Microelectrónica y Computadoras (MCC, por sus siglas en inglés), diseñado para asegurar su competitividad. En ambos casos, la IA fue parte de un esfuerzo amplio, que incluía diseño de chips e investigación de interfaces humanas. En Gran Bretaña, el informe Alvey restableció el financiamiento eliminado por el informe Lighthill. Ninguno de los proyectos cumplió sus ambiciosas metas en términos de nuevas capacidades de IA o impacto económico.

La industria de la IA experimentó un auge, pasó de unos millones de dólares en 1980 a miles de millones en 1988, incluía cientos de empresas que construían sistemas expertos, sistemas de visión, robots y software y hardware especializados para estos propósitos.

Poco después llegó un periodo llamado el "invierno de la IA", en el que muchas empresas fracasaron al no cumplir con promesas extravagantes. Resultó difícil construir y mantener sistemas expertos para dominios complejos, en parte porque los métodos de razonamiento usados por los sistemas fallaban ante la incertidumbre y porque los sistemas no podían aprender de la experiencia.

1.3.5 El regreso de las redes neuronales (1986–presente)

A mediados de la década de 1980, al menos cuatro grupos diferentes reinventaron el algoritmo de aprendizaje basado en **retropropagación** desarrollado a principios de la década de 1960. El algoritmo se aplicó a muchos problemas de aprendizaje en informática y psicología, y la amplia difusión de los resultados en la colección *Parallel Distributed Processing* (Rumelhart y McClelland, 1986) causó gran emoción.

Conexionista

Estos modelos **conexionistas** fueron vistos por algunos como competidores directos de los modelos simbólicos promovidos por Newell y Simon y del enfoque logicista de McCarthy y otros. Parecería obvio que en algún nivel los humanos manipulan símbolos, de hecho, el libro del antropólogo Terrence Deacon, *The Symbolic Species* (1997), sugiere que esta es la *característica definitoria* de los humanos. Ante esto, Geoff Hinton, destacado en el resurgimiento de las redes neuronales en la década de 1980 y 2010, ha descrito los símbolos como el "éter de la IA", haciendo referencia al medio inexistente por el cual muchos físicos del siglo XIX creían que se propagaban las ondas electromagnéticas. Ciertamente, muchos conceptos que nombramos en el lenguaje fallan con un examen más cercano, en tener el tipo de condiciones necesarias y suficientes definidas lógicamente que los primeros investigadores de IA esperaban capturar en forma axiomática. Puede ser que los modelos conexionistas formen conceptos internos de manera más fluida e imprecisa que se adapte mejor al desorden del mundo real. También tienen la capacidad de aprender de ejemplos: esto compara su valor de salida predicho con el valor verdadero en un problema y modificar sus parámetros para disminuir la diferencia, lo que los hace más propensos a desempeñarse bien en ejemplos futuros.

1.3.6 Razonamiento probabilístico y aprendizaje automático (1987–presente)

La fragilidad de los sistemas expertos llevó a un nuevo enfoque más científico que incorporaba probabilidad en lugar de lógica booleana, aprendizaje automático en lugar de codificación manual y resultados experimentales en lugar de afirmaciones filosóficas.[14] Se volvió más común basarse en teorías existentes que proponer nuevas, fundamentar las afirmaciones en teoremas rigurosos o una metodología experimental sólida (Cohen, 1995) en lugar de en la intuición, y demostrar la relevancia para aplicaciones del mundo real en lugar de ejemplos de juguete.

Los conjuntos de problemas de referencia compartidos se convirtieron en la norma para demostrar el progreso, incluido el repositorio UC Irvine para conjuntos de datos de aprendizaje automático, la Competencia Internacional de Planificación para algoritmos de plani-

[14] Algunos caracterizan este cambio como victoria de los **pulcros** (los que piensan que las teorías de la IA deberían basarse en el rigor matemático) sobre los **desaliñados** (los que prefieren probar muchas ideas, escribir algunos programas y evaluar lo que parece ser). Ambos enfoques son importantes. Un cambio hacia la pulcritud implica que el campo ha alcanzado un nivel de estabilidad y madurez. El énfasis actual en el aprendizaje profundo puede representar un resurgimiento de los desaliñados.

ficación, el corpus LibriSpeech de reconocimiento de voz, el conjunto de datos MNIST de reconocimiento de dígitos escritos a mano, ImageNet y COCO de reconocimiento de objetos en imágenes, SQUAD de respuesta a preguntas de lenguaje natural, la competición WMT de traducción automática y las Competiciones SAT Internacionales de solucionadores de satisfacibilidad booleana.

La IA fue fundada en parte como una rebelión contra las limitaciones de campos existentes como la teoría de control y la estadística, pero en este periodo abrazó los resultados positivos de ambos campos. Como lo expresó David McAllester (1998):

> Al inicio de la IA, parecía plausible que nuevas formas de computación simbólica, como marcos y redes semánticas, volvieran obsoleta parte de la teoría clásica. Esto llevó a una forma de aislacionismo en la que la IA quedó en gran medida separada del resto de la informática. Este aislacionismo es abandonado actualmente. Existe un reconocimiento de que el aprendizaje automático no debería estar aislado de la teoría de la información, que el razonamiento incierto no debería estar aislado de la modelización estocástica, que la búsqueda no debería estar aislada de la optimización y el control clásicos, y que el razonamiento automatizado no debería estar aislado de los métodos formales y el análisis estático.

El reconocimiento de voz ilustra el patrón. En los setenta se probaron una amplia variedad de arquitecturas y enfoques diferentes. Muchos fueron específicos y frágiles, y funcionaron sólo en ejemplos cuidadosamente seleccionados. En los ochenta, los enfoques que utilizaban **modelos ocultos de Markov** (HMM, por sus siglas en inglés) dominaron el área. Dos aspectos relevantes de los HMM: En primer lugar, se basan en una teoría matemática rigurosa. Esto permitió a los investigadores del habla basarse en resultados matemáticos de varias décadas desarrollados en otros campos. En segundo lugar, son generados mediante un proceso de entrenamiento en un corpus de datos de habla grande pero real. Esto asegura un rendimiento robusto, en pruebas ciegas rigurosas, los HMM mejoraron sus puntajes. Como resultado, la tecnología del habla y el campo relacionado del reconocimiento de caracteres manuscritos hicieron la transición a aplicaciones industriales y de consumo generalizadas. Cabe señalar que no hubo reclamo científico de que los humanos usen HMM para reconocer el habla; más bien, los HMM dieron un marco matemático para comprender y resolver el problema. En la sección 1.3.8 veremos que el aprendizaje profundo incomodó esta cómoda narrativa.

Modelos ocultos de Markov

Un año importante para la conexión entre IA y otros campos fue 1988, incluidas las estadísticas, investigación de operaciones, teoría de decisiones y teoría de control. El trabajo de Judea Pearl (1988) *Probabilistic Reasoning in Intelligent Systems* llevó a una nueva aceptación de la probabilidad y la teoría de decisiones en IA. El desarrollo de Pearl de **redes bayesianas** produjo un formalismo riguroso y eficiente para representar conocimiento incierto, así como algoritmos prácticos para razonamiento probabilístico. Los capítulos 12 al 16 cubren esta área, además de desarrollos más recientes que han aumentado en gran medida el poder expresivo de los formalismos probabilísticos; el Capítulo 20 describe métodos para aprender redes bayesianas y modelos relacionados a partir de datos.

Red bayesiana

Otra gran contribución en 1988 fue el trabajo de Rich Sutton que conectó el aprendizaje por refuerzo, usado en el programa de juego de damas de Arthur Samuel en la década de 1950, con la teoría de procesos de decisión de Markov (MDP, por sus siglas en inglés) desarrollada en la investigación de operaciones. Diversos trabajos conectaron la investigación de planificación por IA con MDP, y el aprendizaje por refuerzo encontró aplicaciones en robótica y control de procesos, así como en la adquisición de fundamentos teóricos profundos.

Consecuencia del renovado aprecio de la IA por los datos, la modelización estadística, la optimización y el aprendizaje automático fue la gradual reunificación de los subcampos visión

por computadora, robótica, reconocimiento de voz, sistemas multiagentes y procesamiento de lenguaje natural que se habían separados de la IA central. El proceso de reintegración generó beneficios significativos en términos de aplicaciones, como la implementación de robots prácticos que se expandió durante este periodo, como en una mejor comprensión teórica de los problemas centrales de la IA.

1.3.7 Big data (2001 - presente)

Big data

Avances notables en la potencia informática y la creación de la World Wide Web facilitaron la creación de conjuntos de datos grandes, un fenómeno a veces conocido como **big data**; éstos incluyen billones de palabras de texto, miles de millones de imágenes y miles de millones de horas de audio y video, así como amplias cantidades de datos genómicos, datos de seguimiento de vehículos, datos de clics, datos de redes sociales, y así sucesivamente.

Esto ha llevado al desarrollo de algoritmos de aprendizaje diseñados para aprovechar conjuntos de datos grandes. La gran mayoría de ejemplos de conjuntos de datos no están *etiquetados*; por ejemplo, en el trabajo de Yarowsky (1995) sobre la desambiguación de sentidos de las palabras, las ocurrencias de una palabra como "planta" no están etiquetadas en el conjunto de datos para indicar si se refieren a flora o a una fábrica. Pero con conjuntos de datos tan grandes, algoritmos de aprendizaje adecuados pueden lograr una precisión de más de 96 % en la tarea de identificar qué sentido se pretendía en una oración. Además, Banko y Brill (2001) argumentaron que la mejora en el rendimiento al aumentar el tamaño del conjunto de datos en dos o tres órdenes de magnitud supera cualquier mejora al ajustar el algoritmo.

Un fenómeno similar parece ocurrir en tareas de visión por computadora como llenar huecos en fotografías, causados por daños o por eliminar exnovios(as). Hays y Efros (2007) desarrollaron un método ingenioso para mezclar píxeles de imágenes similares; pero descubrieron que la técnica funcionaba mal con una base de datos de sólo miles de imágenes, ya que superaba un umbral de calidad con millones de imágenes. Después, la disponibilidad de decenas de millones de imágenes en la base de datos ImageNet (Deng *et al.*, 2009) revolucionó el campo de la visión por computadora.

La disponibilidad de big data y el cambio hacia el aprendizaje automático ayudaron a que la IA recuperara su atractivo comercial. ¡El big data fue un factor crucial en la victoria en 2011 del sistema Watson de IBM sobre campeones humanos en el juego de preguntas Jeopardy!, un evento que tuvo un gran impacto en la percepción pública de la IA.

1.3.8 Aprendizaje profundo (Deep learning) (2011– presente)

Aprendizaje profundo

El término **aprendizaje profundo** se refiere al aprendizaje automático usado en múltiples capas de elementos de cómputo simples y ajustables. Se experimentó con estas redes ya en la década de 1970, y en forma de **redes neuronales convolucionales** encontraron cierto éxito en el reconocimiento de dígitos manuscritos en la década de 1990 (LeCun *et al.*, 1995). Sin embargo, fue hasta 2011 que los métodos de aprendizaje profundo en realidad despegaron. Esto ocurrió primero en el reconocimiento de voz y luego en el reconocimiento visual de objetos.

En la competencia ImageNet de 2012, que requería clasificar imágenes en una de mil categorías (armadillo, estantería, sacacorchos, etc.), un sistema de aprendizaje profundo creado en el grupo de Geoffrey Hinton en la Universidad de Toronto (Krizhevsky *et al.*, 2013) demostró una gran mejora sobre los sistemas anteriores basados en características diseñadas a mano. Desde entonces, los sistemas de aprendizaje profundo han superado el rendimiento humano en algunas tareas de visión (pero quedan rezagados en otras). Se han reportado ganancias similares en

reconocimiento de voz, traducción automática, diagnóstico médico y juegos. El uso de una red profunda para representar la función de evaluación contribuyó a las victorias de ALPHAGO sobre los principales jugadores humanos de Go (Silver *et al.*, 2016, 2017, 2018).

Estos impresionantes éxitos hicieron resurgir el interés en IA en estudiantes, empresas, inversores, gobiernos, medios de comunicación y público en general. Parece que cada semana hay noticias de una aplicación de IA que se acerca o supera el rendimiento humano, a veces rodeada de especulaciones sobre éxito acelerado o un invierno de la IA.

El aprendizaje profundo depende mucho de hardware potente. Mientras que una CPU de computadora estándar realiza entre 10^9 y 10^{10} operaciones por segundo, un algoritmo de aprendizaje profundo en hardware especializado (GPU, TPU o FPGA) podría consumir entre 10^{14} y 10^{17} operaciones por segundo, en principio en forma de operaciones matriciales y vectoriales muy paralelizadas. Claro, el aprendizaje profundo también depende de la disponibilidad de grandes cantidades de datos de entrenamiento y de algunos trucos algorítmicos (vea Capítulo 21).

1.4 El estado del arte

El Estudio de Cien Años de IA de la Universidad de Stanford (también conocido como AI100) convoca expertos para informar sobre el estado del arte en IA. Su informe de 2016 (Stone *et al.*, 2016; Grosz y Stone, 2018) concluye que "Se pueden esperar aumentos sustanciales en los futuros usos de aplicaciones de IA, incluidos automóviles autónomos, diagnósticos y tratamientos médicos específicos y asistencia física para el cuidado de personas mayores" y que "La sociedad está en un momento crucial para determinar cómo desplegar tecnologías basadas en IA que promuevan valores democráticos como libertad, igualdad y transparencia, en vez de obstaculizarlos". AI100 también produce un **AI Index** en *aiindex.org* para rastrear el progreso. Aspectos destacados de los informes de 2018 y 2019 (comparados con el año 2000, a menos que se indique lo contrario):

AI Index

- Publicaciones: Los documentos de IA aumentaron a 20 000 al año, 20 veces entre 2010 y 2019. La categoría más popular fue el aprendizaje automático. (Los documentos de aprendizaje automático en *arXiv.org* se duplicaron cada año desde 2009 a 2017). La visión por computadora y el procesamiento del lenguaje natural más populares fueron:
- Sentimiento: Casi 70 % de las noticias sobre IA son neutrales, pero las de tono positivo aumentaron de 12 %, en 2016, a 30 %, en 2018. Los problemas comunes son éticos: privacidad de datos y sesgo algorítmico.
- Estudiantes: La inscripción en cursos aumentó 5 veces en EUA y 16 a nivel internacional desde la base de 2010. La IA es la especialización más popular en Ciencias de la Computación.
- Diversidad: Los profesores de IA en el mundo son 80 % hombres y 20 % mujeres. Números similares aplican a doctorantes y contrataciones de la industria.
- Conferencias: La asistencia a NeurIPS aumentó 800 % desde 2012 a 13 500 asistentes. Otras conferencias crecen al año casi 30%.
- Industria: Las startups de IA en Estados Unidos aumentaron 20 veces, a más de 800.
- Internacionalización: China publica más documentos por año que Estados Unidos y casi tantos como toda Europa. Pero en impacto ponderado por citas, los autores estadounidenses están 50 % por delante de los autores chinos. Singapur, Brasil, Australia, Canadá e India son los países de más rápido crecimiento en términos de número de contrataciones de IA.

- Visión: Las tasas de error para la detección de objetos (como se logró en LSVRC, el Desafío de Reconocimiento Visual a Gran Escala) mejoraron 28 % en 2010 y 2% en 2017, superando el rendimiento humano. La precisión en la respuesta a preguntas visuales abiertas (VQA) mejoró de 55 % a 68 % desde 2015, pero rezagada respecto al rendimiento humano de 83 %.
- Velocidad: El tiempo de entrenamiento para la tarea de reconocimiento de imágenes se redujo en un factor de 100 en los últimos dos años. La cantidad de potencia informática usada en las principales aplicaciones de IA se duplica cada 3.4 meses.
- Lenguaje: La precisión en la respuesta a preguntas, medida por el puntaje F1 en el Conjunto de Datos de Preguntas y Respuestas de Stanford (SQUAD), aumentó de 60 a 95 entre 2015 y 2019; en la variante SQUAD 2, el progreso fue más rápido, de 62 a 90 en un año. Ambas puntuaciones superan el rendimiento a nivel humano.
- Referencias humanas: Para 2019, se informó que los sistemas de IA habían alcanzado o superado el rendimiento humano en ajedrez, Go, póker, Pac-Man, Jeopardy!, detección de objetos de ImageNet, reconocimiento de voz en un dominio limitado, traducción de chino a inglés en un dominio restringido, Quake III, Dota 2, StarCraft II, varios juegos de Atari, detección de cáncer de piel y de cáncer de próstata, plegamiento de proteínas y diagnóstico de retinopatía diabética.

¿Cuándo (si ocurre) alcanzarán los sistemas de IA el rendimiento a nivel humano en diversidad de tareas? Ford (2018) entrevista a expertos en IA y encuentra una amplia gama de años objetivo, de 2029 a 2200, con una media de 2099. En una encuesta similar (Grace *et al.*, 2017), 50 % de los encuestados pensaron que esto sucedería para 2066, pero 10 % pensó que sucedería en 2025, y otros dijeron "nunca". Los expertos también se dividían en: si necesitamos nuevos avances fundamentales o sólo refinamientos en los enfoques actuales. Pero no tomemos sus predicciones muy en serio; como Philip Tetlock (2017) demuestra en el área de predicción de eventos mundiales, los expertos no son mejores que los aficionados.

¿Cómo operarán los futuros sistemas de IA? Aún no podemos decirlo. Como se detalla en esta sección, el campo ha adoptado varias historias sobre sí mismo: primero la idea audaz de que la inteligencia por una máquina era posible, luego que podría lograrse codificando el conocimiento experto en lógica, después que los modelos probabilísticos del mundo serían la herramienta principal, y ahora que el aprendizaje automático induciría modelos que podrían no basarse en ninguna teoría bien entendida en absoluto. El futuro revelará qué modelo vendrá.

¿Qué hace la IA hoy en día? Quizás no tanto como algunos medios optimistas podrían hacer creer, pero aun así mucho. Aquí unos ejemplos:

Vehículos robóticos: La historia de estos vehículos se remonta a los autos de control por radio de la década de 1920, aunque las primeras demostraciones de conducción autónoma en carretera sin guías especiales ocurrieron hasta 1980 (Kanade *et al.*, 1986; Dickmanns y Zapp, 1987). Después de demostraciones exitosas de conducción en caminos de tierra en el Gran Desafío DARPA de 132 millas en 2005 (Thrun, 2006) y en calles con tráfico en el Desafío Urbano de 2007, la carrera para desarrollar autos autónomos comenzó en serio. En 2018, los vehículos de prueba de Waymo superaron el hito de 10 millones de millas conducidas en carreteras sin un accidente grave, con el conductor humano tomando el control sólo una vez cada 6 000 millas. Después, la empresa ofreció un servicio de taxi robótico comercial.

En el aire, los drones autónomos de ala fija han realizado entregas de sangre a grandes distancias en Ruanda desde 2016. Los cuadricópteros maniobran notables acrobacias, exploran edificios mientras construyen mapas en 3D y se autoensamblan en formaciones autónomas.

Locomoción mediante patas: BigDog, robot cuadrúpedo creado por Raibert *et al*. (2008), trastocó nuestras nociones de movilidad de robots: no la lenta ni rígida marcha de piernas tiesas de películas de Hollywood, sino algo parecido estrechamente a un animal, capaz de recuperarse cuando es empujado o resbala en hielo. Atlas, un robot humanoide, no sólo camina sobre terreno irregular, también salta sobre cajas y hace volteretas hacia atrás (Ackerman y Guizzo, 2016).

Planificación y programación autónoma: A cien millones de millas de la Tierra, el programa Remote Agent de la NASA es el primer programa de planificación autónoma a bordo de una nave espacial que controla sus operaciones (Jonsson *et al*., 2000). Remote Agent planificaba a partir de objetivos de alto nivel especificados desde tierra y supervisaba su ejecución, detectando, diagnosticando y recuperándose de los problemas que ocurrían. Hoy, el conjunto de herramientas de planificación EUROPA (Barreiro *et al*., 2012) se utiliza para las operaciones diarias de los rovers de la NASA en Marte y el sistema SEXTANT (Winternitz, 2017) posibilita la navegación autónoma en el espacio profundo, más allá del sistema GPS global.

Durante la crisis del Golfo Pérsico de 1991, las fuerzas estadounidenses usaron una Herramienta de Análisis y Replanificación Dinámica, DART (Cross y Walker, 1994), para planificar y programar logística automatizada para transporte. Esto involucraba hasta 50 000 vehículos, de carga y de personas a la vez, y debía considerar puntos de partida, destinos, rutas, capacidades de transporte, de puertos y aeródromos, y resolver conflictos entre todos los parámetros. La Agencia de Investigación de Proyectos Avanzados de Defensa (DARPA, por sus siglas en inglés) afirmó que esta única aplicación había retribuido la inversión de 30 años de DARPA en IA.

Todos los días, empresas de transporte como Uber y servicios de mapas como Google Maps proporcionan indicaciones de manejo para cientos de millones de usuarios, trazando rápidamente una ruta óptima teniendo en cuenta las condiciones de tráfico actuales y futuras.

Traducción automática: Estos sistemas de traducción en línea facilitan la lectura de documentos en más de 100 idiomas, incluidos idiomas nativos de más de 99 % de humanos, y representan cientos de miles de millones de palabras al día para cientos de millones de usuarios. Si bien no son perfectos, son adecuados para su comprensión. Para idiomas estrechamente relacionados con gran cantidad de datos de entrenamiento (como francés e inglés), las traducciones dentro de un dominio estrecho se acercan al nivel de un humano (Wu *et al*., 2016b).

Reconocimiento de voz: En 2017, Microsoft demostró que su Sistema de Reconocimiento de Habla Conversacional había alcanzado una tasa de error de palabras de 5.1 %, igualando el rendimiento humano en la tarea de Switchboard, que implica transcribir conversaciones telefónicas (Xiong *et al*., 2017). Casi un tercio de la interacción informática en el mundo hoy se realiza por voz en lugar de teclado; Skype ofrece traducción de voz a voz en tiempo real en diez idiomas. Alexa, Siri, Cortana y Google ofrecen asistentes que responden preguntas y realizar tareas para el usuario; por ejemplo, el servicio Google Duplex usa reconocimiento y síntesis de voz para reservar en restaurantes para los usuarios, casi con conversación fluida en su nombre.

Recomendaciones: Empresas como Amazon, Facebook, Netflix, Spotify, YouTube, Walmart y otras usan aprendizaje automático para recomendar a un usuario e que podría gustar según sus experiencias pasadas y de otros usuarios. El campo de sistemas de recomendación tiene su historia (Resnick y Varian, 1997), pero cambia rápido debido a nuevos métodos de aprendizaje profundo que analizan contenido (texto, música, video), historial y metadatos (Van den Oord *et al*., 2014; Zhang *et al*., 2017). La filtración de spam se considerar una forma de recomendación (o no-recomendación); las técnicas de IA hoy filtran más de 99.9% de spam, los

servicios de correo electrónico también recomiendan posibles destinatarios y proponer texto de respuesta posible.

Juegos: Cuando Deep Blue derrotó al campeón mundial de ajedrez Garry Kasparov en 1997, los defensores de la supremacía humana depositaron sus esperanzas en Go. Piet Hut, astrofísico y entusiasta de Go, predijo que tomaría "cien años para que una computadora venciera a los humanos en Go, quizá más". Pero 20 años después, ALPHAGO superó a todos los jugadores humanos (Silver *et al.*, 2017). Ke Jie, el campeón mundial, dijo: "El año pasado aún era muy parecido a un humano cuando jugaba. Hoy, se convirtió en un dios de Go". ALPHAGO se benefició de estudiar cientos de miles de juegos pasados de jugadores humanos de Go y del conocimiento de expertos jugadores que trabajaron en el equipo.

Un programa de seguimiento, *ALPHAZERO*, no usó ninguna entrada de humanos (excepto las reglas del juego) y aprendió mediante auto juego cómo derrotar a sus oponentes, humanos y máquinas, en Go, ajedrez y shogi (Silver *et al.*, 2018). Mientras, ¡campeones humanos han sido derrotados por sistemas de IA en juegos tan diversos como Jeopardy! (Ferrucci *et al.*, 2010), póker (Bowling *et al.*, 2015; Moravčík *et al.*, 2017; Brown y Sandholm, 2019), y los videojuegos Dota 2 (Fernandez y Mahlmann, 2018), StarCraft II (Vinyals *et al.*, 2019) y Quake III (Jaderberg *et al.*, 2019).

Análisis de imágenes: No contentos con superar la precisión humana en el desafiante reconocimiento de objetos de ImageNet, los investigadores en visión por computadora han asumido el problema aún más difícil de la subtitulación de imágenes. Impresionantes ejemplos incluyen "Persona montando una motocicleta en un camino de tierra", "Dos pizzas sobre una estufa" y "Grupo de jóvenes lanzando el *frisbee* " (Vinyals *et al.*, 2017b). Sin embargo, los sistemas de hoy están lejos de ser perfectos: un "refrigerador repleto de comida y bebidas" resultó ser un letrero de prohibido estacionar parcialmente oscurecido por muchas pegatinas pequeñas.

Medicina: Los algoritmos de IA igualan o superan a los médicos expertos en el diagnóstico de muchas condiciones, en particular cuando se basa en imágenes. Ejemplos incluyen enfermedad de Alzheimer (Ding *et al.*, 2018), cáncer metastásico (Liu *et al.*, 2017; Esteva *et al.*, 2017), enfermedades oftálmicas (Gulshan *et al.*, 2016) y de la piel (Liu *et al.*, 2019c). Una revisión sistemática y metanálisis (Liu *et al.*, 2019a) encontró que el rendimiento de los programas de IA era equivalente al de los profesionales de la salud. Un énfasis actual en la IA médica es facilitar las asociaciones humano-máquina. Así, el sistema LYNA logra una precisión general de 99.6 % en el diagnóstico de cáncer de mama metastásico, mejor que un experto humano sin ayuda, pero la combinación de ambos logra mejores resultados (Liu *et al.*, 2018; Steiner *et al.*, 2018).

La adopción generalizada de estas técnicas está limitada no por la precisión del diagnóstico, sino por la necesidad de demostrar mejora en los resultados clínicos y garantizar transparencia, falta de sesgo y privacidad de los datos (Topol, 2019). En 2017, sólo dos aplicaciones de IA médica fueron aprobadas por la FDA, pero eso aumentó a 12 en 2018, y continúa creciendo.

Ciencia del clima: Un equipo de científicos ganó el Premio Gordon Bell 2018 por un modelo de aprendizaje profundo que descubre información detallada sobre eventos climáticos extremos que antes estaban enterrados en datos climáticos. Usaron una supercomputadora con hardware GPU especializado para superar el nivel de exaop (10^{18} operaciones por segundo), el primer programa de aprendizaje automático en hacerlo (Kurth *et al.*, 2018). Rolnick *et al.* (2019) presentan un catálogo de 60 páginas de formas en que el aprendizaje automático se utiliza para abordar el cambio climático.

Estos son sólo ejemplos de sistemas de IA que hoy existen. Ni magia ni ciencia ficción, sino ciencia, ingeniería y matemáticas, a las que este libro dedica una introducción.

1.5 Riesgos y beneficios de la IA

Francis Bacon, un filósofo al que se le atribuye la creación del método científico, señaló en *La sabiduría de los antiguos* (1609) que las "artes mecánicas son de uso ambiguo, sirven para hacer daño y como remedio". A medida que la IA juega un papel más importante en los ámbitos económico, social, científico, médico, financiero y militar. Sería prudente considerar los daños y los remedios, en lenguaje moderno, riesgos y beneficios que puede traer. Los temas resumidos aquí se tratan con mayor profundidad en los capítulos 27 y 28.

Para comenzar con los beneficios: en síntesis, la civilización es producto de la inteligencia humana. Si tenemos acceso a una IA sustancialmente mayor, el techo de nuestras ambiciones se eleva de modo considerable. El potencial de la IA y la robótica para liberar a la humanidad del trabajo repetitivo y monótono, y para aumentar significativamente la producción de bienes y servicios, podría presagiar una era de paz y abundancia. La capacidad para acelerar la investigación científica podría resultar en curas para enfermedades y soluciones para el cambio climático y la escasez de recursos. Como ha sugerido Demis Hassabis, CEO de Google DeepMind: "Primero resolvamos la IA, luego usemos la IA para resolver lo demás".

No obstante, mucho antes de que tengamos la oportunidad de "resolver la IA", incurriremos en riesgos por su mal uso, inadvertido o no. Algunos de estos ya son evidentes, mientras que otros parecen probables según las tendencias actuales:

- *Armas letales autónomas*: Definidas por las Naciones Unidas como armas que localizan, seleccionan y eliminan objetivos humanos sin intervención humana. Una preocupación fundamental con tales armas es su *escalabilidad*: la ausencia de un requisito de supervisión humana significa que un pequeño grupo de personas puede desplegar un número arbitrariamente grande de armas contra objetivos humanos definidos por cualquier criterio de reconocimiento factible. Las tecnologías necesarias para las armas autónomas son similares a las necesarias para coches autónomos. Las discusiones expertas informales sobre los riesgos potenciales de las armas letales autónomas comenzaron en la ONU en 2014, pasando a la etapa previa al tratado formal de un Grupo de Expertos Gubernamentales en 2017.
- *Vigilancia y persuasión*: Si bien es costoso, tedioso y a veces legalmente cuestionable que el personal de seguridad monitoree líneas telefónicas, datos de videocámaras, correos electrónicos y otros canales de mensajería, la IA (reconocimiento de voz, visión por computadora y comprensión del lenguaje natural) puede ser usada de manera escalable para vigilar de modo masivo individuos y detectar actividades de interés. Al adaptar los flujos de información a los individuos mediante redes sociales, con base en técnicas de aprendizaje automático, el comportamiento político puede modificarse y controlarse hasta cierto punto; preocupación evidenciada en las elecciones de Estados Unidos a partir de 2016.
- *Toma de decisiones sesgada*: El uso descuidado o deliberado de algoritmos de aprendizaje automático para evaluar solicitudes de libertad condicional y préstamos puede resultar en decisiones sesgadas por raza, género u otras categorías protegidas. Con frecuencia, los datos mismos reflejan un sesgo generalizado en la sociedad.
- *Impacto en el empleo*: El miedo de que las máquinas suprimen empleos tienen siglos de antigüedad. La historia nunca es simple: las máquinas realizan tareas que de otro modo harían los humanos, pero también hacen a estos más productivos y, por lo tanto, más empleables, y hacen a las empresas más rentables y, en consecuencia, capaces de pagar mejores salarios. Pueden dar viabilidad económica a actividades que de otro modo serían imprácticas. Su uso en general resulta en aumento de la riqueza, pero suele tener el

efecto de desplazar la riqueza del trabajo al capital, agravando aún más la desigualdad. Los avances tecnológicos anteriores, como la invención de telares mecánicos, han resultado en serias afectaciones al empleo, pero eventualmente las personas encuentran nuevos tipos de trabajo para hacer. Por otro lado, es posible que la IA esté creando esos nuevos trabajos. Este tema es, sin duda, un foco de interés para economistas y gobiernos de todo el mundo.

- *Aplicaciones críticas para la seguridad:* Conforme las técnicas de IA avanzan, se usan cada vez más en aplicaciones críticas para la seguridad, como conducir automóviles y gestionar suministros de agua de las ciudades. Los accidentes fatales ya han ocurrido y destacan la dificultad de la verificación formal y el análisis de riesgos estadísticos para sistemas desarrollados usando técnicas de aprendizaje automático. El campo de la IA necesitará desarrollar estándares técnicos y éticos comparables a los predominantes en otras disciplinas de ingeniería y salud, donde las vidas de las personas están en juego.
- *Ciberseguridad***:** Las técnicas de IA sirven para defenderse contra ciberataques, por ejemplo, al detectar patrones de comportamiento inusuales, y al contribuir a la potencia, supervivencia y capacidad de proliferación de programas malintencionados (malware). Así, los métodos de aprendizaje por refuerzo se han usado para crear herramientas altamente efectivas contra ataques de chantaje y phishing automatizados y personalizados.

Revisaremos estos temas en profundidad en la Sección 27.3. A medida que los sistemas de IA se hacen más capaces, asumirán más roles sociales antes desempeñados por los humanos. Así como éstos han usado esos roles para perpetrar actos cuestionables, podemos esperar que hoy también den mal uso a los sistemas de IA para perpetrar actos cuestionables. Todos los ejemplos apuntan a la importancia de la gobernanza y, eventualmente, la regulación. Hoy, la comunidad de investigación y las principales corporaciones involucradas en investigación de IA han desarrollado principios de autorregulación voluntaria para actividades relacionadas con la IA (Sección 27.3). Los gobiernos y las organizaciones internacionales crean organismos consultivos para elaborar regulaciones apropiadas para cada caso de uso específico, para prepararse ante los impactos económicos y sociales, y para aprovechar las capacidades de IA para enfrentar los principales problemas sociales.

¿Qué hay del largo plazo? ¿Lograremos el objetivo más ambicioso: la creación de una inteligencia comparable a la humana o más capaz? Y, si lo hacemos, ¿qué sigue?

En la historia de la IA, estas preguntas han sido eclipsadas por la rutina diaria de hacer que los sistemas de IA hagan algo inteligente. Como con cualquier gran disciplina, los investigadores se han especializado en subcampos específicos: juego, representación de conocimiento, visión o comprensión del lenguaje natural, con frecuencia suponiendo que el progreso en ellos contribuiría a los objetivos generales de la IA. Nils Nilsson (1995), uno de los líderes originales del proyecto Shakey en SRI, recordó al campo los objetivos más amplios y advirtió que los subcampos peligraban de convertirse en fines en sí mismos. Después, algunos fundadores influyentes, como John McCarthy (2007), Marvin Minsky (2007) y Patrick Winston (Beal y Winston, 2009), estuvieron de acuerdo con las advertencias de Nilsson, y sugirieron que, en lugar de centrarse en el rendimiento medible en aplicaciones específicas, la IA debería volver a sus raíces y esforzarse por lograr, en palabras de Herb Simon, "máquinas que piensen, que aprendan y que creen". Llamaron a este esfuerzo **IA a nivel humano** o HLAI (por sus siglas en inglés): una máquina debería ser capaz de aprender a hacer cualquier cosa que un humano haga. Su primer simposio fue en 2004 (Minsky *et al.,* 2004). Otro esfuerzo con objetivos similares, el movimiento de **inteligencia artificial general (AGI)** (Goertzel y Pennachin, 2007), celebró su primera conferencia y organizó la *Revista de Inteligencia Artificial General* en 2008.

IA a nivel humano

Inteligencia artificial general (AGI)

Superinteligencia artificial (ASI)

En ese periodo, surgió la inquietud de que crear una **superinteligencia artificial** o **ASI** (por sus siglas en inglés), es decir, una inteligencia que superara la capacidad humana podría ser mala idea (Yudkowsky, 2008; Omohundro, 2008). Turing (1996) abordó la misma preocupación en una conferencia en Manchester en 1951, con base en ideas de Samuel Butler (1863):[15]

> Es probable que cuando el método de pensamiento de la máquina haya comenzado, no tardaría mucho en superar nuestros débiles poderes. En algún punto deberíamos esperar que las máquinas tomen el control, de la manera en que se menciona en *Erewhon* de Samuel Butler.

Estas preocupaciones se han generalizado con los avances en el aprendizaje profundo, la publicación de libros como *Superintelligence* de Nick Bostrom (2014) y declaraciones públicas de Stephen Hawking, Bill Gates, Martin Rees y Elon Musk.

Problema del gorila

Advertir una sensación general de malestar con la idea de crear máquinas superinteligentes es natural. Podríamos llamar a esto el **problema del gorila**: hace unos siete millones de años, un primate ahora extinto evolucionó, con una rama que condujo a los gorilas y otra a los humanos. Hoy en día, los gorilas no están muy contentos con la rama humana; pues no tienen control sobre su futuro. Si este es el resultado del éxito en la creación de una IA superhumana, es decir, que los humanos cedan el control de su futuro, quizá deberíamos dejar de trabajar en IA y, como corolario, renunciar a los beneficios que podría traer. Esta es la esencia de la advertencia de Turing: no es obvio que podamos controlar máquinas más inteligentes que nosotros.

Si la IA superhumana fuera una caja negra que llegara del espacio exterior, sería prudente cuidarnos de abrir la caja. Pero no lo es: *nosotros* diseñamos los sistemas de IA, por ello, si "tomaran el control", como sugiere Turing, sería el resultado de un fallo de diseño.

Para evitar tal resultado, necesitamos comprender la fuente de posibles fallos. Norbert Wiener (1960), motivado a considerar el futuro a largo plazo de la IA después de ver cómo el programa de juego de damas de Arthur Samuel, aprendió a vencer a su creador, dijo:

> Si, para lograr nuestros propósitos, usamos una agencia mecánica cuya operación en verdad no podemos interferir... será mejor que estemos bien seguros de que el propósito puesto en la máquina es el que realmente deseamos.

Problema del Rey Midas

Muchas culturas refieren mitos de humanos que piden a dioses, genios, magos o demonios algo. Invariablemente, en estas historias, obtienen lo que literalmente piden y luego se arrepienten. El tercer deseo, si lo hay, es deshacer los dos primeros. Llamaremos a esto el **problema del Rey Midas:** Midas, un legendario rey de la mitología griega; pidió que todo lo que tocara se convirtiera en oro, pero luego se arrepintió al tocar su comida, bebida y miembros de su familia.[16]

Abordamos este tema en la Sección 1.1.5, donde señalamos la necesidad de una modificación significativa al modelo estándar de establecer objetivos fijos en la máquina. La solución al dilema de Wiener no es tener un "propósito definido en la máquina", en absoluto. En cambio, queremos máquinas que se esfuercen por alcanzar objetivos humanos, pero que sepan que no saben con certeza cuáles son esos objetivos.

---

[15] Aun antes, en 1847, Richard Thornton, editor del *Primitive Expounder*, arremetió contra las calculadoras mecánicas: "Mente. . . se supera a sí misma y elimina la necesidad de su propia existencia al inventar máquinas para pensar por sí mismas. Pero ¿quién sabe si esas máquinas, cuando se las lleva a mayor perfección, no pueden pensar en un plan para remediar todos sus defectos y, luego, concebir ideas más allá de la mente mortal!

[16] Midas lo habría hecho mejor si hubiera seguido principios básicos de seguridad y hubiera incluido un botón de "deshacer" y un botón de "pausa" en su deseo.

Juego de asistencia

Aprendizaje por refuerzo inverso

Es quizá desafortunado que casi toda la investigación en IA a la fecha se haya realizado dentro del modelo estándar, lo que significa que casi todo el material técnico en esta edición refleja ese marco teórico. Sin embargo, hay resultados tempranos dentro del nuevo marco. En el Capítulo 16, se muestra que una máquina tiene un incentivo positivo para permitirse ser apagada, si y sólo si tiene incertidumbre sobre el objetivo humano. En el Capítulo 18, se formulan y estudian **juegos de asistencia**, que describen matemáticamente la situación en la que un humano tiene un objetivo y una máquina intenta lograrlo, pero al inicio no está segura de qué es. En el Capítulo 22, se explican los métodos del **aprendizaje por refuerzo inverso** que permiten a las máquinas aprender más sobre las preferencias humanas a partir de observaciones de las elecciones que hacen los humanos. En el Capítulo 27, se exploran dos de las principales dificultades: primera, que nuestras elecciones dependen de nuestras preferencias a partir de una arquitectura cognitiva compleja difícil de invertir; y, segunda, que nosotros, los humanos, quizá no tengamos preferencias consistentes, individualmente o como grupo, por lo que puede no estar claro *qué deberían hacer* los sistemas de IA por nosotros.

Resumen

Este capítulo define la IA y establece el contexto cultural en el que se ha desarrollado. Algunos de los puntos importantes son los siguientes:

- Diferentes personas abordan la IA con diferentes objetivos en mente. Dos preguntas fundamentales a plantear son: ¿Le motiva el pensamiento o el comportamiento? ¿Quiere modelar a los humanos o intentar lograr resultados óptimos?
- De acuerdo con el llamado modelo estándar, la IA se preocupa en principio por la **acción racional**. Un **agente inteligente** ideal toma la mejor acción posible en una situación. Estudiamos el problema de construir agentes que sean inteligentes en este sentido.
- Se necesitan dos refinamientos adicionales a esta idea simple: primero, la capacidad de cualquier agente, humano o el que sea, para elegir acciones racionales está limitada por la intractabilidad computacional para hacerlo; segundo, el concepto de una máquina que persigue un objetivo definido debe ser reemplazado por el de una máquina que persigue objetivos para beneficiar a humanos, pero inciertos sobre cuáles son esos objetivos.
- Los filósofos (remontándose a 400 a. C.) hicieron concebible la IA al sugerir que la mente es de alguna manera como una máquina, que opera con conocimiento codificado en algún lenguaje interno, y que el pensamiento puede usarse para elegir qué acciones tomar.
- Los matemáticos proporcionaron las herramientas para manipular declaraciones de certeza lógica, inciertas y probabilísticas. También sentaron las bases para comprender la computación y el razonamiento sobre algoritmos.
- Los economistas formalizaron el problema de tomar decisiones que maximizan la utilidad esperada para el tomador de decisiones.
- Los neurocientíficos descubrieron algunos hechos sobre cómo funciona el cerebro y las formas en que es similar y diferente de las computadoras.
- Los psicólogos adoptaron la idea de que humanos y animales pueden considerarse máquinas de procesamiento de información. Los lingüistas demostraron que el uso del lenguaje encaja en este modelo.
- Los ingenieros informáticos proporcionaron las máquinas cada vez más potentes que hacen posibles las aplicaciones de IA, y los ingenieros de software las hicieron más utilizables.
- La teoría de control se ocupa de diseñar dispositivos que actúen de manera óptima en función de la retroalimentación del entorno. Al inicio, las herramientas matemáticas de

la teoría de control eran bastante diferentes de las usadas en IA, pero los campos se están acercando.
- La historia de la IA ha tenido ciclos de éxito, optimismo fuera de lugar y recortes resultantes en entusiasmo y financiamiento. También ha habido ciclos de introducción de enfoques nuevos y creativos y refinamiento sistemático de los mejores.
- La IA ha madurado de modo considerable en comparación con sus primeras décadas, tanto teórica como metodológicamente. A medida que los problemas con los que trata la IA se volvieron más complejos, el campo pasó de la lógica booleana al razonamiento probabilístico, y del conocimiento elaborado a mano al aprendizaje automático a partir de datos. Esto ha llevado a mejoras en las capacidades de los sistemas reales y a mayor integración con otras disciplinas.
- A medida que los sistemas de IA encuentran aplicación en el mundo real, se ha vuelto necesario considerar una amplia gama de riesgos y consecuencias éticas.
- A largo plazo, nos enfrentamos al difícil problema de controlar sistemas de IA superinteligentes que pueden evolucionar de manera impredecible. Resolver este dilema parece requerir un cambio en nuestra concepción de la IA.

Notas bibliográficas e históricas

Una historia completa de la IA la ofrece Nils Nilsson (2009), uno de los pioneros tempranos del campo. Pedro Domingos (2015) y Melanie Mitchell (2019) dan una visión general del aprendizaje automático para un público general, y Kai-Fu Lee (2018) describe la carrera por el liderazgo internacional en IA. Martin Ford (2018) entrevista a 23 destacados investigadores de IA.

Las principales sociedades profesionales de IA son la Association for the Advancement of Artificial Intelligence (AAAI), el ACMSpecial Interest Group in Artificial Intelligence (SIGAI, formalmente SIGART), la European Association for AI, y la Society for Artificial Intelligence and Simulation of Behaviour (AISB). The Partnership on AI reúne a muchas organizaciones comerciales y sin fines de lucro preocupadas por los impactos éticos y sociales de la IA. La revista *AI Magazine* de AAAI publica numerosos artículos temáticos y tutoriales, y su sitio web, aaai.org, ofrece noticias, tutoriales e información de fondo.

El trabajo más reciente aparece en las actas de las principales conferencias de IA tales como: El *International Joint Conference on AI (IJCAI),* la *European Conference on AI (ECAI)* anual, y la *AAAI Conference.* El aprendizaje automático está cubierto por: La *International Conference on Machine Learning and the Neural Information Processing Systems (NeurIPS) meeting.* Las principales revistas sobre IA general son: *Artificial Intelligence, Computational Intelligence,* el *IEEE Transactions on Pattern Analysis and Machine Intelligence, IEEE Intelligent Systems,* y el *Journal of Artificial Intelligence Research.* También hay numerosas conferencias y revistas dedicadas a áreas específicas, las cuales se revisarán en los capítulos correspondientes.

CAPÍTULO 2

AGENTES INTELIGENTES

En el que discutimos la naturaleza de los agentes, perfectos o, dicho de otro modo, la diversidad de entornos y la resultante variedad de tipos de agentes.

El Capítulo 1 identificó el concepto de **agentes racionales** como central para nuestro enfoque en la IA. En este capítulo, hacemos esta noción más concreta. Veremos que el concepto de racionalidad se aplica a una amplia variedad de agentes que operan en cualquier ambiente imaginable. Nuestro plan en el libro es desarrollar con este concepto un conjunto de principios de diseño apto para construir agentes exitosos, sistemas que sean llamados razonablemente **inteligentes**.

Comenzamos examinando agentes, entornos y el acoplamiento entre ellos. La observación de que algunos agentes se comportan mejor que otros lleva naturalmente a la idea de un agente racional, uno que se comporta lo mejor posible. Qué tan bien se comporta un agente depende de la naturaleza del entorno pues algunos son más difíciles que otros. Damos una categorización básica de entornos y mostramos cómo sus propiedades influyen en el diseño de agentes adecuados para ese entorno. Describimos varios diseños básicos de agentes "esqueleto", que desarrollamos en el libro.

2.1 Agentes y entornos

Entorno
Sensor
Actuador

Un **agente** es cualquier cosa que pueda ser vista como percibiendo su **entorno** mediante **sensores** y actuando sobre ese entorno por medio de actuadores. Esta idea se ilustra en la Figura 2.1. Un agente humano tiene ojos, oídos y otros órganos para los sensores, y manos, piernas, tracto vocal, etc., para los actuadores. Un agente robótico tendría cámaras y sensores infrarrojos para los sensores y varios motores para los actuadores. Un agente de software recibe el contenido de archivos, paquetes de red y entrada humana (teclado/ratón/pantalla táctil/voz) como entradas sensoriales y actúa sobre el entorno al escribir archivos, enviar paquetes de red y mostrar información o generar sonidos. El entorno podría ser todo, ¡incluso el universo entero! En la práctica, es sólo aquella parte del universo cuyo estado nos importa al diseñar el agente, la parte que afecta lo que el agente percibe y que es afectada por sus acciones.

Percepción
Secuencia de percepción

Usamos el término **percepción** para referirnos al contenido que los sensores de un agente perciben. La **secuencia perceptiva** de un agente es la secuencia completa de todo lo que el agente ha percibido. En general, *la elección de acción de un agente en un momento dado puede depender de su conocimiento añadido y de toda la secuencia perceptiva observada a la fecha, pero no de algo que no haya percibido.* Al especificar la elección de acción del agente para cada secuencia perceptiva posible, hemos dicho más o menos todo lo que hay que decir del agente. Matemáticamente hablando, decimos que el comportamiento de un agente es descrito por la **función del agente** que mapea cualquier secuencia perceptiva dada a una acción.

Función del agente

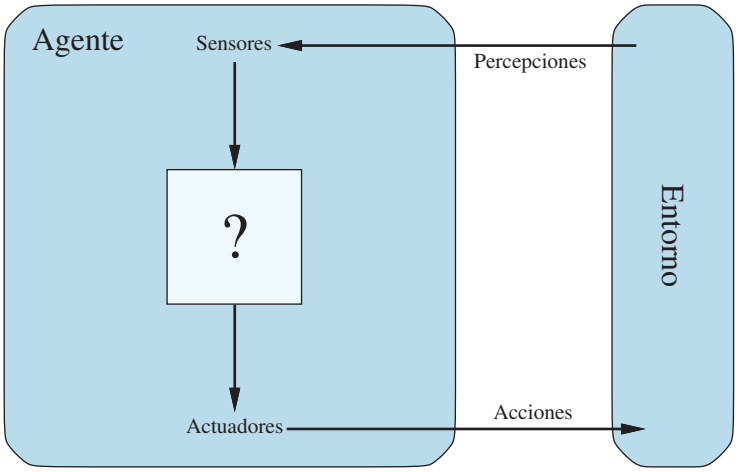


Figura 2.1 Los agentes interactúan con los entornos a través de sensores y actuadores.

Podemos imaginar *tabular* la función del agente que describe cualquier agente dado; para la mayoría de los agentes, esta sería una tabla muy grande –infinita, de hecho–, a menos que limitemos la longitud de las secuencias perceptivas que queremos considerar. Dado un agente para experimentar, podemos, en principio, construir esta tabla probando todas las secuencias perceptivas posibles y registrando qué acciones realiza el agente en respuesta.[1] La tabla es, por supuesto, una caracterización *externa* del agente. *Internamente*, la función del agente para un agente artificial será implementada por un **programa de agente.** Es importante estas dos ideas diferenciadas. La función del agente es una descripción matemática abstracta; el programa de agente es una ejecución concreta, que se ejecuta dentro de algún sistema físico.

Programa de agente

Ilustremos estas ideas con un ejemplo simple: el mundo de la aspiradora, que consiste en un agente de aspiradoras robóticas en un mundo compuesto por cuadrados sucios o limpios. La Figura 2.2 muestra una configuración con sólo dos cuadrados, A y B. El agente de aspiradora percibe en qué cuadrado se encuentra y si hay suciedad en el cuadro. El agente comienza en el cuadro A. Las acciones disponibles son moverse a la derecha, moverse a la izquierda, aspirar la suciedad o no hacer nada.[2] Una función de agente simple es: si el cuadrado actual está sucio, entonces aspirar; de lo contrario, moverse al otro cuadrado. Una tabulación parcial de esta función del agente se muestra en la Figura 2.3 y un programa de agente que la implementa se ilustra en la Figura 2.8, página 49.

Al observar la Figura 2.3, vemos que varios agentes del mundo de la aspiradora pueden definirse sólo rellenando la columna derecha de varias maneras. Así, la pregunta obvia es: *¿Cuál es la manera correcta de completar la tabla?* Es decir, *¿qué hace que un agente sea bueno o malo, inteligente o estúpido?* Respondemos estas preguntas en la próxima sección.

[1] Si el agente utiliza cierta aleatorización para elegir sus acciones, entonces tendríamos que probar cada secuencia varias veces para identificar la probabilidad de cada acción. Uno podría imaginar que actuar al azar es bastante tonto, pero más adelante en este capítulo mostraremos que puede ser muy inteligente.

[2] Un robot real sería poco probable que tuviera acciones como “mover a la derecha” y “mover a la izquierda”. En cambio, las acciones serían “girar las ruedas hacia adelante” y “girar las ruedas hacia atrás”. Hemos elegido las acciones más fáciles de seguir en el ejemplo, no para facilitar su implementación en un robot real.

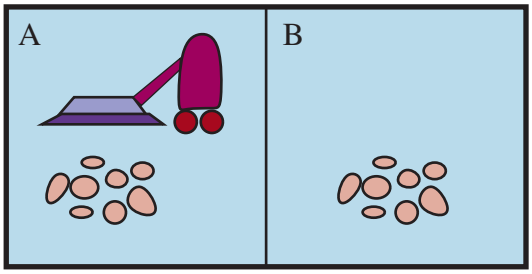


Figura 2.2 El mundo de una aspiradoras con sólo dos ubicaciones. Cada ubicación podría estar limpia o sucia, y el agente puede moverse a la izquierda o a la derecha y limpiar el cuadrado que ocupa. Diferentes versiones del mundo de la aspiradora permiten diferentes reglas sobre lo que el agente puede percibir, si sus acciones siempre tienen éxito, etc.

Secuencia perceptiva	Acción
[A, Limpio]	Derecha
[A, Sucio]	Aspirar
[B, Limpio]	Izquierda
[B, Sucio]	Aspirar
[A, Limpio], [A, Limpio]	Derecha
[A, Limpio], [A, Sucio]	Aspirar
...	
[A, Limpio], [A, Limpio], [A, Limpio]	Derecha
[A, Limpio], [A, Limpio], [A, Sucio]	Aspirar
...	

Figura 2.3 Tabulación parcial de una función de un agente simple para el mundo de las aspiradoras que se muestra en la Figura 2.2. El agente limpia el cuadrado actual si está sucio; de lo contrario, pasa al otro cuadrado. Tenga en cuenta que la tabla tiene un tamaño ilimitado a menos que exista una restricción en la longitud de las posibles secuencias de perceptivas.

Antes de cerrar la sección, enfatizamos que la noción de agente está destinada a ser una herramienta para analizar sistemas, no una caracterización absoluta que divida el mundo en agentes y no agentes. Podríamos ver una calculadora de mano como un agente que elige la acción de mostrar "4" cuando se le da la secuencia perceptiva "2 + 2 =", pero tal análisis difícilmente ayudaría a nuestra comprensión de la calculadora. En cierto sentido, las áreas de la ingeniería son vistas como el diseño de artefactos que interactúan con el mundo; la IA opera en (según los autores) el extremo más interesante del espectro, donde los artefactos tienen recursos computacionales significativos y el entorno de tarea requiere toma de decisiones no triviales.

2.2 Buen comportamiento: el concepto de racionalidad

Un **agente racional** es aquel que hace lo correcto. Claro, hacer lo correcto es mejor que hacer lo incorrecto, pero ¿qué significa hacer lo correcto?

Agente racional

2.2.1 Medidas de rendimiento

La filosofía moral ha desarrollado varias nociones diferentes de lo "correcto", pero la IA generalmente se ha adherido a una noción llamada **consecuencialismo**: evaluamos el comportamiento de un agente por sus consecuencias. Cuando un agente es colocado en un entorno, genera una secuencia de acciones de acuerdo con las percepciones que recibe. Esta secuencia de acciones hace que el entorno pase por una secuencia de estados. Si la secuencia es deseable, entonces el agente ha tenido un buen desempeño. Esta noción de deseabilidad está capturada por una **medida de rendimiento** que evalúa cualquier secuencia dada de estados del entorno.

Consecuencialismo

Medida de rendimiento

Los humanos tienen deseos y preferencias propias, así que la noción de racionalidad aplicada a los humanos tiene que ver con su éxito en elegir acciones que produzcan secuencias de estados del entorno deseables desde *su punto de vista*. Las máquinas, al contrario, no tienen deseos ni preferencias propios; la medida de rendimiento está, inicialmente, en la mente del diseñador de la máquina o en la mente de los usuarios para quienes se diseñó la máquina. Veremos que algunos diseños de agentes tienen una representación explícita de (una versión de) la medida de rendimiento, mientras que en otros diseños la medida de rendimiento por completo implícita: el agente podría hacer lo correcto, pero no sabe por qué.

Recordando la advertencia de Norbert Wiener: asegurarse que "el propósito introducido en la máquina sea el que realmente deseamos" (página 33), observe que es muy difícil formular correctamente una medida de rendimiento. Considere, por ejemplo, el agente de aspiradora de la sección anterior. Podríamos proponer medir el rendimiento por la cantidad de suciedad recogida en un turno de ocho horas. Con un agente racional, por supuesto, lo que se solicita es lo que se obtiene. Un agente racional maximiza esta medida de rendimiento limpiando la suciedad, arrojándola al suelo, luego limpiándola de nuevo, y así sucesivamente. Una medida de rendimiento más adecuada recompensaría al agente por tener un suelo limpio. Por ejemplo, se otorgaría un punto por cada cuadro limpio en cada paso de tiempo (quizá con una penalización por la electricidad consumida y el ruido generado). *Como regla, es mejor diseñar medidas de rendimiento de acuerdo con lo que uno realmente quiere lograr en el entorno, en vez de cómo uno piensa que debería comportarse el agente.*

Incluso cuando se evitan problemas obvios, algunos difíciles perduran. Por ejemplo, la noción de "suelo limpio" del párrafo anterior se basa en la limpieza promedio a lo largo del tiempo. La misma limpieza promedio puede ser lograda por dos agentes diferentes, uno hace un trabajo mediocre todo el tiempo, mientras que el otro limpia con vigor, pero toma largos descansos. ¿Cuál es preferible parecería un detalle fino de la ciencia de la limpieza, aunque de hecho es una pregunta filosófica profunda con grandes implicaciones? ¿Cuál es mejor: una vida imprudente de altibajos, o una existencia segura pero monótona? ¿Cuál es mejor: una economía donde todos viven pobreza moderada, o una donde algunos viven en abundancia mientras que otros son muy pobres? Dejamos estas preguntas como ejercicio para el lector.

En el libro, asumiremos que la medida de rendimiento se especifica de manera correcta. Por las razones dadas antes, sin embargo, debemos aceptar la posibilidad de que podríamos introducir el propósito equivocado en la máquina, precisamente el problema del Rey Midas descrito en la página 33. Al diseñar un software, copias de este pertenecerán a diferentes usuarios, no podemos anticipar las preferencias exactas de cada usuario. Por ello, quizá necesitemos construir agentes

que reflejen la incertidumbre inicial sobre la verdadera medida de rendimiento y aprender al respecto conforme pase el tiempo; tales agentes se describen en los Capítulos 16, 18 y 22.

2.2.2 Racionalidad

¿Qué es racional en un momento dado depende de cuatro cosas:

- La medida de rendimiento que define el criterio de éxito.
- El conocimiento previo del entorno del agente.
- Las acciones que el agente es capaz realizar.
- La secuencia perceptiva del agente hasta la fecha.

Agente racional

Esto lleva a una **definición de un agente racional**:

> *Para cada secuencia* perceptiva *posible, un agente racional debería seleccionar una acción que se espera maximice su medida de rendimiento, dada la evidencia proporcionada por la secuencia* perceptiva *y cualquier conocimiento incorporado que tenga el agente.*

Considere el simple agente de limpieza que limpia un cuadrado si está sucio o si no lo está se mueve al otro cuadrado; esta es la función de agente tabulada en la Figura 2.3. ¿Es este un agente racional? ¡Depende! Primero, necesitamos decir cuál es la medida de rendimiento, qué se sabe sobre el entorno y qué sensores y actuadores tiene el agente. Supongamos que:

- La medida de rendimiento otorga un punto por cada cuadro limpio en cada paso, en una "vida útil" de 1 000 pasos.
- La "geografía" del entorno se conoce *a priori* (Figura 2.2), pero no la distribución de la suciedad ni la ubicación inicial del agente no lo están. Los cuadros limpios permanecen limpios y aspirar limpia el cuadro actual. Las acciones *derecha* e *izquierda* mueven al agente un cuadro excepto cuando esto lleve al agente fuera del entorno, en cuyo caso el agente permanece donde está.
- Las únicas acciones disponibles son *derecha*, *izquierda* y *aspirar*.
- El agente percibe correctamente su ubicación y si ésta contiene suciedad.

Bajo estas circunstancias, el agente es de hecho racional; su rendimiento esperado es al menos tan bueno como el de cualquier otro agente.

Se ve con facilidad que el mismo agente sería irracional bajo diferentes circunstancias. Por ejemplo, cuando toda la suciedad se ha limpiado, el agente oscilará sin necesidad de un lado a otro; si la medida de rendimiento incluye penalización de un punto por cada movimiento, el agente se desempeñará mal. Un mejor agente para este caso haría nada una vez que esté seguro de que todos los cuadros están limpios. Si los cuadros limpios se vuelven a ensuciar, el agente debería verificar y volver a limpiarlos si es necesario. Si la geografía del entorno es desconocida, el agente necesitará **explorar**. El ejercicio 2.VACR le pide que diseñe agentes para estos casos.

2.2.3 Omnisciencia, aprendizaje y autonomía

Omnisciencia

Necesitamos tener cuidado de distinguir entre racionalidad y **omnisciencia**. Un agente omnisciente conoce el resultado *real* de sus acciones y actúa en consecuencia; pero la omnisciencia es

imposible en el mundo real. Pensemos el siguiente ejemplo: Estoy caminando por los Campos Elíseos un día y veo a un viejo amigo al otro lado de la calle. No hay tráfico cerca y no estoy ocupado en otra cosa, así que, siendo racional, comienzo a cruzar la calle. Mientras, a 33 000 pies de altura, la puerta de carga de un avión que pasa se cae,[3] y antes de lograr cruzar la calle, soy aplastado. ¿Fui irracional al cruzar la calle? Es poco probable que mi obituario diga "Idiota intenta cruzar la calle".

Este ejemplo muestra que racionalidad no es lo mismo que perfección. La racionalidad maximiza el rendimiento *esperado*, mientras que la perfección maximiza el rendimiento *real*. Retroceder de un requisito de perfección no es sólo una cuestión de ser justo con los agentes. El punto es que si esperamos que un agente haga lo que resulta ser la mejor acción después del hecho, será imposible diseñar un agente que cumpla esta especificación, a menos que mejoremos el desempeño de las bolas de cristal o de las máquinas del tiempo.

Nuestra definición de racionalidad no requiere omnisciencia porque la elección racional depende sólo de la secuencia perceptiva *hasta la fecha*. También debemos asegurarnos de no permitir inadvertidamente que el agente participe en actividades en definitiva poco inteligentes. Por ejemplo, si antes de cruzar una calle concurrida un agente no mira a ambos lados, entonces su secuencia perceptiva no le dirá que se aproxima un camión a gran velocidad. ¿Dice nuestra definición de racionalidad que ahora está bien cruzar la calle? ¡Para nada!

Primero, no sería racional cruzar la calle dada esta secuencia perceptiva poco informativa: el riesgo de accidente al cruzar sin mirar es mucho. Segundo, un agente racional debería elegir la acción de "mirar" antes de pisar la calle, porque mirar maximiza el rendimiento esperado. Ejecutar acciones *para modificar percepciones futuras*, a veces llamado **recolección de información**, es parte importante de la racionalidad; esto se cubre en profundidad en el Capítulo 16. Otro ejemplo de recolección de información es el dado por la **exploración** que debe emprender un agente de aspiradoras en un entorno inicialmente desconocido.

Recolección de información

Nuestra definición requiere que un agente racional no sólo recolecte información, sino que también *aprenda* lo máximo posible de lo que percibe. La configuración inicial del agente podría reflejar algún conocimiento previo del entorno, pero a medida que adquiere experiencia, esto puede modificarse y aumentarse. Hay casos extremos en los que el entorno es completamente conocido *a priori* y por completo predecible. En tales casos, el agente no necesita percibir ni aprender; simplemente actúa de modo correcto.

Aprendizaje

Por supuesto, tales agentes son frágiles. Pensemos en el humilde escarabajo estercolero. Después de cavar su nido y poner sus huevos, busca una bola de estiércol en un montón cercano para tapar la entrada. Si la bola de estiércol se le quita en *el camino*, el escarabajo continúa su tarea y simula tapar el nido con la bola de estiércol inexistente, sin darse cuenta de que falta. La evolución ha construido una suposición en el comportamiento del escarabajo, y cuando se viola, resulta un comportamiento infructuoso.

Un poco más inteligente es la hembra de la avispa Sphex. Esta cavará un nido, saldrá y cazará una oruga y la arrastrará hasta su nido, entrará de nuevo en él para verificar que todo esté bien, arrastrará la oruga adentro y pondrá sus huevos. La oruga sirve como fuente de alimento cuando los huevos eclosionan. Hasta aquí todo bien, pero si un entomólogo mueve la oruga unos centímetros mientras la Sphex está hace la revisión, esta volverá sobre su plan al paso de "arrastrar la oruga" y continuará con el plan sin modificaciones, volviendo a revisar la madriguera, incluso después de docenas de intervenciones para mover la oruga. La Sphex es incapaz de aprender que su plan innato está fallando y, por lo tanto, no lo cambiará.

[3] Véase N. Henderson, "Se recomiendan nuevos pestillos para las puertas de los jumbo jets Boeing 747", *Washington Post*, 24 de agosto de 1989.

Autonomía

En la medida en que un agente depende del conocimiento previo de su diseñador en vez de en sus propias percepciones y procesos de aprendizaje, decimos que carece de **autonomía**. Un agente racional debería ser autónomo; debería aprender lo que pueda para compensar el conocimiento previo parcial o incorrecto. Por ejemplo, un agente de limpieza que aprende a predecir dónde y cuándo aparecerá suciedad adicional lo hará mejor que uno que no lo hace.

En la práctica, rara vez se requiere completa autonomía desde el principio: cuando el agente tiene poca o nula experiencia, tendría que actuar al azar a menos que el diseñador dé alguna asistencia. Así como la evolución proporciona a los animales suficientes reflejos innatos para sobrevivir el tiempo necesario para aprender por sí mismos, sería razonable dar a un agente artificialmente inteligente algún conocimiento inicial y capacidad para aprender. Después de suficiente experiencia en su entorno, el comportamiento de un agente racional permite volverse efectivamente *independiente* de su conocimiento previo. Así, la incorporación del aprendizaje permite diseñar un solo agente racional que tendrá éxito en una gran variedad de entornos.

2.3 La naturaleza de los entornos

Entorno de tarea

Ahora que hemos definido *racionalidad*, casi estamos listos para pensar en la construcción de agentes racionales. Pero, primero debemos pensar en los **entornos de tarea**, esencialmente los "problemas" para los que los agentes racionales son las "soluciones". Comenzamos mostrando cómo especificar un entorno de tarea, ilustrando el proceso con varios ejemplos. Luego veremos que los entornos de tarea vienen en una variedad de sabores. La naturaleza del entorno de tarea afecta directamente el diseño adecuado para el programa de agente.

2.3.1 Especificar el entorno de tarea

PEAS

En nuestro análisis de la racionalidad del simple de limpieza, debimos especificar la medida de rendimiento, entorno y actuadores y sensores del agente. Esto lo agrupamos bajo el encabezado **entorno de tarea**. Para los aficionados a los acrónimos, llamaremos a esta descripción **PEAS** (**P**erformance, **E**nvironment, **A**ctuators, **S**ensors). Al diseñar un agente, en el primer paso siempre debemos especificar el entorno de tarea de la manera más completa posible.

El mundo de las aspiradoras fue un ejemplo simple; consideremos un problema más complejo: un conductor de taxi automatizado. La Figura 2.4 resume la descripción PEAS para el entorno de tarea del taxi. Analizamos cada elemento en detalle a continuación.

*Primero ¿cuál es la **medida de rendimiento** a que la aspira nuestro conductor automático? Las cualidades deseables son llegar al destino correcto, minimizar consumo de combustible, desgaste, tiempo, costo de viaje, violaciones a leyes de tráfico y molestias a otros conductores; así como, maximizar la seguridad, comodidad de los pasajeros y las ganancias. Es obvio, algunas de estas cualidades entran en conflicto, por lo que se requerirán compensaciones.*

Luego, ¿cuál es el **entorno** de conducción que enfrentará el taxi? Todo conductor debe lidiar con una variedad de carreteras, desde caminos rurales y callejones urbanos hasta autopistas de 12 carriles. Éstas alojan otros vehículos, peatones, animales callejeros, obras viales, patrullas, charcos y baches. El taxi debe interactuar con pasajeros potenciales y reales. También hay alternativas. El taxi podría operar en el sur de California, donde la nieve rara vez es problema, o en Alaska, donde rara vez no lo es. Siempre podría conducir a la derecha, o querríamos ser tan flexibles como para conducir a la izquierda en Gran Bretaña o en Japón. Obvio, cuanto más restringido el entorno, más fácil será el problema de diseño.

Tipo de agente	Medida de rendimiento	Entorno	Actuadores	Sensores
Conductor de taxi	Seguro, rápido, viaje legal y cómodo, maximizar las ganancias, minimizar el impacto en otros usuarios de la vía	Carreteras, otros, tráfico, policía, peatones, clientes, clima	Dirección, acelerador, freno, señal, bocina, pantalla, habla	Cámaras, radares, velocímetro, GPS, sensores del motor, acelerómetro, micrófonos, pantalla táctil

Figura 2.4 Descripción PEAS del entorno de tareas para un taxista automatizado.

Los **actuadores** para un taxi automatizado son los disponibles para un conductor humano: control sobre el motor mediante el acelerador, sobre la dirección y los frenos. Además, necesitará salida a una pantalla de visualización o un sintetizador de voz para comunicarse con los pasajeros y, quizás, comunicarse con otros vehículos, de manera educada o no.

Los **sensores** básicos para el taxi incluirán una o más cámaras de video para ver, así como sensores LiDAR y de ultrasonido para detectar distancias a otros autos y obstáculos. Para evitar multas por exceso de velocidad, debe tener un velocímetro, y para controlar el vehículo de manera correcta, en especial en curvas, debería tener un acelerómetro. Para determinar el estado mecánico del vehículo, necesitará la habitual gama de sensores del motor, combustible y sistema eléctrico. Como muchos conductores humanos, querría acceder a señales GPS para no perderse. Al final, necesitará una entrada táctil o de voz para que el pasajero solicite un destino.

La Figura 2.5 esboza los elementos PEAS básicos para varios tipos adicionales de agentes. En el Ejercicio 2.PEAS se presentan más ejemplos, éstos incluyen entornos físicos y virtuales. Vea que los entornos de tareas virtuales pueden ser tan complejos como el mundo "real": así, un agente de software (o robot de software o softbot) que opera en sitios web de subastas y reventa trata con millones de otros usuarios y miles de millones de objetos, muchos con imágenes reales.

Agente de software
Softbot

2.3.2 Propiedades de los entornos de tareas

Las dimensiones del entorno de la tarea que podrían surgir en IA son vastas. Sin embargo, podemos identificar un número muy pequeño de dimensiones a lo largo de las cuales los entornos de tareas pueden ser categorizados. Estas dimensiones determinan, en gran medida, el diseño adecuado del agente y la aplicabilidad de cada una de las principales familias de técnicas para la ejecución del agente. Primero listamos las dimensiones, luego analizamos varios entornos de tareas para ilustrar las ideas. Las definiciones aquí son informales; en capítulos posteriores se dan declaraciones más precisas y ejemplos de cada tipo de entorno.

Completamente observable
Parcialmente observable

Completa vs. **parcialmente observable**: Si los sensores de un agente le dan acceso al estado completo del entorno en cada momento, entonces decimos que el entorno de la tarea es por completo observable. Un entorno de tarea es por completo observable si los sensores detectan todos los aspectos *relevantes* para elegir la acción; la *relevancia*, a su vez, depende de la medida de rendimiento. Los entornos por completo observables son útiles porque el agente no necesita

Tipo de agente	Medida de rendimiento	Entorno	Actuadores	Sensores
Sistema de diagnóstico médico	Paciente sano, costos reducidos	Paciente, hospital, personal	Visualización de preguntas, pruebas, diagnósticos, tratamientos.	Pantalla táctil/ entrada por voz de síntomas y hallazgos.
Sistema de análisis de imágenes de satélite.	Categorización correcta de objetos, terreno	Satélite en órbita, enlace descendente, clima	Visualización de categorización de escenas.	Cámara digital de alta resolución.
Robot recoge piezas	Porcentaje de piezas en contenedores correctos	Cinta transportadora con piezas; contenedores	Brazo y mano articulados.	Sensores de cámara, ángulo táctil y articular.
Controlador de refinería	Pureza, rendimiento, seguridad.	Refinería, materias primas, operadores.	Válvulas, bombas, calentadores, agitadores, displays.	Sensores de temperatura, presión, flujo y químicos.
Tutor de inglés interactivo	Puntaje del estudiante en la prueba	Conjunto de estudiantes, agencia de pruebas.	Visualización de ejercicios, retroalimentación, habla.	Entrada de teclado, voz.

Figura 2.5 Ejemplos de tipos de agentes y sus descripciones PEAS.

mantener ningún estado interno para dar seguimiento al mundo. Un entorno sería parcialmente observable debido a sensores ruidosos e inexactos o porque partes del estado faltan en los datos del sensor; así, dado un agente de limpieza con solo un sensor local de suciedad no puede decir si hay suciedad en otras casillas; entonces, un taxi automatizado no es capaz de saber qué piensan los otros conductores. Si el agente no tiene sensores, el entorno es **inobservable**. Pensaríamos que en esos casos la situación del agente es desesperada, pero, como en el Capítulo 4, los objetivos del agente aún son alcanzables, a veces con certeza.

Inobservable

Agente único
Multiagente

Agente único vs. **multiagente**: La distinción entre entornos de un solo agente y multiagente parecería muy simple. Por ejemplo, un agente que resuelve un crucigrama por sí mismo claramente se encuentra en un entorno de un agente único, mientras que un agente que juega ajedrez está en un entorno de dos agentes. No obstante, hay problemas sutiles. Primero, ya describimos como una entidad *puede* ser vista como un agente, pero no hemos explicado qué entidades *deben* ser vistas como agentes. ¿Debe un agente A (el conductor del taxi, por ejemplo) tratar a un objeto B (otro vehículo) como un agente, o sólo tratarlo como un objeto que se comporta según las leyes de la física, análogo a las olas en la playa o las hojas que sopla el viento? La distinción clave radica en si el comportamiento de B se describe mejor al maximizar una medida de rendimiento cuyo valor depende (también) del comportamiento del agente A.

En el ajedrez, la entidad oponente *B* intenta maximizar su medida de rendimiento, lo que, según las reglas del ajedrez, minimiza la medida de rendimiento del agente *A*. Por lo tanto, el ajedrez es un entorno **competitivo** multiagente. Al contrario, en el entorno de conducción de taxis, evitar colisiones maximiza la medida de rendimiento de todos los agentes, por lo que es un entorno multiagente parcialmente **cooperativo**. También es parcialmente competitivo porque, por ejemplo, solo un automóvil puede ocupar un espacio de estacionamiento.

Competitivo

Cooperativo

Los problemas de diseño de agentes en entornos multiagentes suelen ser muy diferentes de los de entornos de agente único; por ejemplo, la comunicación suele surgir como un comportamiento racional en entornos multiagentes; en algunos entornos competitivos, el comportamiento aleatorio es racional porque evita los inconvenientes de la previsibilidad.

Determinista vs. **no determinista**. Si el siguiente estado del entorno es por completo determinado por el estado actual y la acción ejecutada por el agente(s), entonces decimos que el entorno es determinista; de lo contrario, es no determinista. En principio, un agente no necesita preocuparse por la incertidumbre en un entorno por completo observable y determinista. Sin embargo, si el entorno es parcialmente observable, podría *parecer* no determinista.

Determinista

No determinista

La mayoría de las situaciones reales son tan complejas que es imposible registrar todos los aspectos no observados; en la práctica, deben tratarse como no deterministas. Conducir un taxi es claramente no determinista en este sentido, porque nunca es posible predecir el comportamiento del tránsito; además, los neumáticos pueden reventar súbitamente y el motor podría apagarse de improviso. El mundo de la aspiradora, tal como lo describimos, es determinista, pero las variaciones permiten incluir elementos no deterministas como la aparición aleatoria de suciedad y un mecanismo de succión poco confiable (Ejercicio 2.VFIN).

Un apunte final: algunos usan la palabra estocástico como sinónimo de "no determinista", pero hacemos una distinción entre ambos términos; decimos que un modelo del entorno es estocástico si trata explícitamente con probabilidades (por ejemplo, "hay un 25% de probabilidad de lluvia mañana") y "no determinista" si las posibilidades se enumeran sin cuantificar (como, "hay posibilidad de lluvia mañana").

Estocástico

Episódico vs. **secuencial:** En un entorno de tarea episódico, la experiencia del agente se divide en episodios atómicos. En cada uno, el agente recibe una percepción y luego realiza una sola acción. Es crucial destacar que el siguiente episodio no depende de las acciones tomadas en episodios previos. Muchas tareas de clasificación son episódicas. Así, un agente que debe detectar piezas defectuosas en una línea de ensamblaje basa cada decisión en la pieza actual, con independencia de las decisiones previas; además, la decisión actual no afecta si la siguiente pieza es defectuosa. Al contrario, en entornos secuenciales, la decisión actual podría afectar las decisiones futuras.[4] El ajedrez y la conducción de taxis son secuenciales: en ambos casos, las acciones a corto plazo podrían tener consecuencias a largo plazo. Los entornos episódicos son más simples que los entornos secuenciales porque el agente no necesita planificar hacia adelante.

Episódico

Secuencial

Estático vs. **dinámico:** Si el entorno cambia mientras un agente delibera, decimos que el entorno es dinámico para ese agente; en caso contrario, es estático. Los entornos estáticos son fáciles de manejar porque el agente no necesita seguir mirando el mundo mientras decide una acción, ni preocuparse por el paso del tiempo. Los entornos dinámicos, por otro lado, preguntan continuamente al agente qué quiere hacer; si aún no ha decidido, eso cuenta como decidir no hacer nada. Si el entorno en sí no cambia con el paso del tiempo, pero la puntuación de ren-

Estático

Dinámico

[4] La palabra "secuencial" también se utiliza en informática como antónimo de "paralelo". Ambos significados no están relacionados.

Semidinámico

dimiento del agente sí lo hace, entonces decimos que el entorno es **semidinámico**. Conducir un taxi es claramente dinámico: los otros autos y el taxi mismo siguen moviéndose mientras el algoritmo de conducción duda sobre qué hacer a continuación. El ajedrez, cuando se juega con un reloj, es semidinámico. Los crucigramas son estáticos.

Discreto
Continuo

Discreto vs. **continuo:** La distinción discreto/continuo aplica al *estado* del entorno, a la forma como se maneja el *tiempo* y a los *perceptos* y *acciones* del agente. Por ejemplo, el entorno del ajedrez tiene un número finito de estados distintos (excluyendo el reloj), y también tiene un conjunto discreto de perceptos y acciones. La conducción de taxis es un problema de estado y tiempo continuos: su velocidad y su ubicación y de los otros vehículos varían en un rango de valores continuos y lo hacen suavemente con el tiempo. Las acciones de conducción también son continuas (ángulos de dirección, etc.). La entrada de las cámaras digitales es discreta, estrictamente hablando, pero en general se trata como si representara intensidades y ubicaciones continuamente variables.

Conocido
Desconocido

Conocido vs. **desconocido:** Hablando estrictamente, esta distinción no se refiere al entorno en sí, sino al estado de conocimiento del agente (o diseñador) de las "leyes de la física" del entorno. En un entorno conocido, se dan los resultados (o probabilidades de resultados si el entorno es no determinista) para todas las acciones. Obvio, si el entorno es desconocido, el agente deberá aprender cómo funciona para tomar buenas decisiones.

La distinción entre entornos conocidos y desconocidos no es la misma que entre entornos completa y parcialmente observables. Es muy posible que un entorno *conocido* sea *parcialmente* observable, por ejemplo, en juegos de solitario, conozco las reglas, pero aún no puedo ver las cartas que no se han volteado. Al contrario, un entorno *desconocido* puede ser *por completo* observable, en un nuevo videojuego, la pantalla muestra todo el estado del juego, pero aún no sé qué hacen los botones hasta que los pruebo.

Como vimos en la página 39, la medida de rendimiento en sí misma podría ser desconocida, pues el diseñador no está seguro de cómo escribirla con corrección o porque el usuario final, cuyas preferencias importan, no se conoce. Así, un conductor de taxi no sabrá si un nuevo pasajero prefiere un viaje tranquilo o rápido, un estilo de conducción cauteloso o agresivo. Un asistente personal virtual inicia sin saber nada sobre las preferencias personales de su nuevo propietario. En tales casos, el agente es capaz de aprender más sobre la medida de rendimiento basándose en interacciones adicionales con el diseñador o el usuario. Esto sugiere que el entorno de la tarea por necesidad se ve como un entorno multiagente.

El caso más difícil es *parcialmente observable, multiagente, no determinista, secuencial, dinámico, continuo y desconocido.* Conducir un taxi es difícil en todos estos sentidos, excepto que el entorno del conductor es en general conocido. Conducir un automóvil alquilado en otro país con geografía desconocida, leyes de tráfico diferentes y pasajeros nerviosos es más emocionante.

La Figura 2.6 lista las propiedades de varios entornos familiares. Note que las propiedades no siempre son absolutas. Por ejemplo, hemos listado la tarea de diagnóstico médico como de un solo agente porque el proceso de enfermedad en un paciente no se modela de manera útil como un agente; pero un sistema de diagnóstico médico también lidiaría con pacientes recalcitrantes y personal escéptico, por ello el entorno tendría un aspecto multiagente. Además, el diagnóstico médico es episódico si uno concibe la tarea como seleccionar un diagnóstico dado un listado de síntomas; el problema es secuencial si la tarea incluye proponer una serie de pruebas, evaluar el progreso durante el tratamiento, manejar varios pacientes, y así sucesivamente.

No incluimos una columna "conocido/desconocido" porque, como ya explicamos, no es estrictamente una propiedad del entorno. Para algunos entornos, como el ajedrez y el póker, es

Entorno de tarea	Observación	Agentes	Determinista	Episódico	Estático	Discreto
Crucigrama	Total	Individual	Determinista	Secuencial	Estático	Discreto
Ajedrez con reloj	Total	Múltiple	Determinista	Secuencial	Semiestático	Discreto
Póker	Parcial	Múltiple	Estocástico	Secuencial	Estático	Discreto
Backgammon	Total	Múltiple	Estocástico	Secuencial	Estático	Discreto
Conducción de taxi	Parcial	Múltiple	Estocástico	Secuencial	Dinámico	Continuo
Diagnóstico médico	Parcial	Individual	Estocástico	Secuencial	Dinámico	Continuo
Análisis de imágenes	Total	Individual	Determinista	Episódico	Semiestático	Continuo
Robot de selección de partes	Parcial	Individual	Estocástico	Episódico	Dinámico	Continuo
Controlador de refinería	Parcial	Individual	Estocástico	Secuencial	Dinámico	Continuo
Tutor de inglés	Parcial	Múltiple	Estocástico	Secuencial	Dinámico	Discreto

Figura 2.6 Ejemplos de entornos de tareas y sus características.

fácil dar al agente un conocimiento completo de las reglas; no obstante, es interesante considerar cómo un agente podría aprender a jugar estos juegos sin dicho conocimiento.

El repositorio de código asociado con este libro (`aima.cs.berkeley.edu`) incluye múltiples implementaciones de entornos, junto con un simulador de entornos de propósito general para evaluar el rendimiento de un agente. Los experimentos con frecuencia se realizan no para un solo entorno, sino para múltiples entornos tomados de una **clase de entornos**. Por ejemplo, para evaluar a un conductor de taxi en tráfico simulado, ejecutaríamos múltiples simulaciones con diferentes condiciones de tráfico, iluminación y clima. Luego, nos interesaríamos en el rendimiento promedio del agente sobre la clase de entornos.

Clase de entornos

2.4 Estructura de los agentes

Hasta aquí hemos hablado de agentes describiendo el *comportamiento*: la acción realizada tras cualquier secuencia dada de percepciones. Ahora enfrentaremos el problema y hablaremos sobre su funcionamiento interno. El trabajo de la IA es diseñar un **programa de agente** que implemente la función del agente, es decir, la correspondencia de percepciones a acciones. Suponemos que este programa se ejecutará en algún tipo de dispositivo informático con sensores físicos y actuadores, a esto llamamos **arquitectura del agente:**

Programa de agente

Arquitectura del agente

$$agente = arquitectura + programa$$

Obvio, el programa que elijamos debe ser apropiado para la arquitectura. Si el programa va a recomendar acciones como *Caminar*, la arquitectura debería tener piernas. La arquitectura podría ser sólo una PC ordinaria o un automóvil robótico con varias computadoras a bordo, cámaras y otros sensores. En general, la arquitectura pone a disposición del programa las percepciones de los sensores, ejecuta el programa y alimenta las opciones de acción del programa a los actuadores a medida que se generan. La mayor parte de este libro trata sobre el diseño de programas de agentes, aunque los Capítulos 25 y 26 tratan sobre sensores y actuadores.

```
function TABLE-DRIVEN-AGENT(percepción) returns una acción
  persistent: percepciones, una secuencia, inicialmente vacía,
              tabla, una tabla de acciones, indexadas por secuencias de percepciones, al inicio completamente especificadas

  append percepción al final de percepciones
  action←LOOKUP(percepciones, tabla)
  return acción
```

Figura 2.7 El programa TABLE-DRIVEN-AGENT se invoca para cada nueva percepción y devuelve una acción cada vez. Conserva la secuencia perceptiva completa en la memoria.

2.4.1 Programas de agentes

Los programas de agentes que diseñamos en este libro tienen la misma estructura básica: toman la percepción actual como entrada de los sensores y devuelven una acción a los actuadores.[5] Note la diferencia entre el programa de agente, que toma la percepción actual como entrada, y la función del agente, que depende de todo el historial de percepciones. El programa de agente no tiene más opción que tomar sólo la percepción actual como entrada porque no dispone nada del entorno; si las acciones del agente necesitan depender de toda la secuencia de percepciones, el agente tendrá que recordar las percepciones.

Describimos los programas de agentes en lenguaje de pseudocódigo, definido en el Apéndice B. (El repositorio de código en línea contiene ejecuciones en lenguajes de programación reales). Por ejemplo, la Figura 2.7 muestra un programa de agente común que da seguimiento de la secuencia de percepciones y luego usa esta secuencia para buscar en una tabla de acciones para decidir qué hacer. La tabla, un ejemplo para el mundo de la aspiradora de la Figura 2.3, representa de forma explícita la función del agente que el programa de agente encarna. Para construir un agente racional así, como diseñadores debemos construir una tabla que contenga la acción apropiada para cada posible secuencia de percepciones.

Es ilustrativo considerar por qué el enfoque basado en tablas para la construcción de agentes está condenado al fracaso. Sea $\mathcal{P}$ el conjunto de percepciones posibles y sea T la vida útil del agente (el número total de percepciones que recibirá). La tabla de búsqueda contendrá $\sum_{t=1}^{T} |\mathcal{P}|^t$ entradas. Considera el taxi automatizado: la entrada visual de una sola cámara (ocho cámaras es lo común) llega a una velocidad de casi 70 megabytes por segundo (30 cuadros por segundo, 1 080 × 720 píxeles con 24 bits de información de color). Esto da una tabla de búsqueda con más de $10^{600\,000\,000\,000}$ entradas para una hora de conducción. Incluso la tabla de búsqueda para el ajedrez, un fragmento pequeño y controlado del mundo real, tiene (resulta) al menos 10^{150} entradas. En comparación, el número de átomos en el universo observable es inferior a 10^{80}. El tamaño abrumador de estas tablas significa que (a) ningún agente físico en el universo tendrá el espacio para almacenar la tabla; (b) el diseñador no tendría tiempo para crear la tabla, y (c) ningún agente podría aprender nunca todas las entradas correctas de la tabla a partir de su experiencia.

A pesar de todo, el TABLE-DRIVEN-AGENT *hace* lo que queremos, suponiendo que la tabla se llene de manera correcta: implementa la función del agente deseada.

[5] Hay otras opciones para el esqueleto del programa de agente; por ejemplo, podríamos tener los programas de agente en **corrutinas** que se ejecutan de forma asincrónica con el entorno. Cada corrutina tiene un puerto de entrada y salida, y consiste en un bucle que lee el puerto de entrada en busca de percepciones y escribe acciones en el de salida.

function REFLEX-VACUUM-AGENT([*ubicación*, *estado*]) devuelve una acción

if *estado* = *Sucio* entonces devuelve, *Limpiar*
else if *ubicación* = *A* entonces devuelve *Derecha*
else if *ubicación* = *B* entonces devuelve *Izquierda*

Figura 2.8 El programa de agente para un agente reflejo simple en el entorno la aspi de vacío de dos ubicaciones del programa de la aspiradora. Este programa implementa la función de agente tabulada en la Figura 2.3.

El desafío clave para la IA es descubrir cómo escribir programas que, de ser posible, causen un comportamiento racional a partir de un programa pequeño y no de una tabla vasta.

Tenemos muchos casos que ilustran que esto es exitoso en otras áreas: por ejemplo, las enormes tablas de raíces cuadradas usadas por ingenieros y escolares antes de 1970, reemplazadas por un programa de cinco líneas para el método de Newton ejecutado en calculadoras electrónicas. La pregunta es ¿puede la IA hacer por el comportamiento inteligente general lo que Newton hizo por las raíces cuadradas? Creemos que la respuesta es sí.

En el resto de esta sección, describimos cuatro tipos básicos de programas de agente que incorporan los principios subyacentes de casi todos los sistemas inteligentes:

- Agentes de reflejo simple;
- Agentes de reflejo basados en modelos;
- Agentes basados en objetivos (y en metas); y
- Agentes basados en utilidad.

Cada tipo de programa de agente combina componentes específicos de maneras particulares para forjar acciones. La sección 2.4.6 explica en términos ordinarios cómo convertir todos estos agentes en *agentes de aprendizaje* que pueden mejorar el rendimiento de sus componentes para generar mejores acciones. Finalmente, la sección 2.4.7 describe la variedad de formas en que los propios componentes se representan dentro del agente. Esta variedad ofrece un principio organizativo importante para el campo y para el libro mismo.

2.4.2 Agentes de reflejo simple

Agentes de reflejo simple

El tipo más simple de agente es el **agente de reflejo simple**. Estos agentes seleccionan acciones en función de la percepción *actual*, ignorando el resto del historial de percepción. Por ejemplo, el agente de aspiradora cuya función de agente está tabulada en la Figura 2.3 es un agente de reflejo simple, porque su decisión se basa sólo en la ubicación actual y en si ésta contiene suciedad. Un programa de agente para este caso se muestra en la Figura 2.8.

Note que el programa de agente de aspiradora es muy pequeño comparado con la tabla correspondiente. La reducción más obvia surge de ignorar el historial de percepción, lo que reduce el número de secuencias de percepción relevantes de 4^T a sólo 4. Una reducción adicional, pequeña, viene del hecho de que cuando el cuadro actual está sucio, la acción no depende de la ubicación. Aunque hemos escrito el programa de agente usando declaraciones **if-then-else**, es lo suficientemente simple como para que se implemente como un circuito booleano.

Los comportamientos de reflejo simples ocurren aun en entornos más complejos. Imagínese a sí mismo como el conductor del taxi automatizado. Si el auto de adelante frena y enciende sus

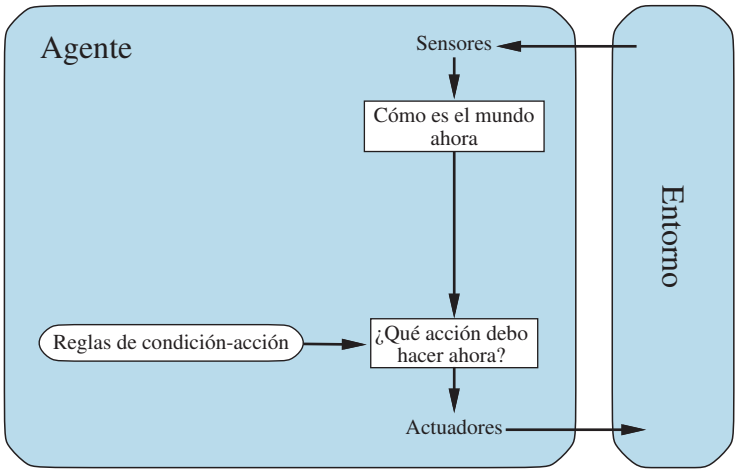


Figura 2.9 Diagrama esquemático de un agente de reflejo simple. Usamos rectángulos para indicar el estado interno actual del proceso de decisión del agente y óvalos para representar la información de fondo utilizada en el proceso.

luces de freno, entonces debería notar esto e iniciar el frenado. Es decir, se realiza un procesamiento en la entrada visual para establecer la condición "El automóvil de adelante frena". Luego, esto activa alguna conexión establecida en el programa de agente hacia la acción "iniciar frenado". Llamamos a tal conexión **regla de condición-acción,**[6] escrita como

Regla de condición-acción

if *el automóvil-de-adelante-está-frenando* ***then*** *iniciar-frenado.*

Los humanos también tienen muchas de estas conexiones, algunas son respuestas aprendidas (como para conducir) y algunas son reflejos innatos (como parpadear cuando algo se acerca al ojo). En el transcurso del libro, mostramos varias formas diferentes en las que tales conexiones pueden ser aprendidas e implementadas.

El programa en la Figura 2.8 es específico para un entorno de aspiradora particular. Un enfoque más amplio y flexible es construir primero un intérprete de propósito general para reglas de condición-acción y luego crear conjuntos de reglas para entornos de tarea específicos. La Figura 2.9 muestra la estructura de este programa general en forma esquemática, y muestra cómo las reglas de condición-acción permiten al agente hacer la conexión desde el percepto hasta la acción. No se preocupe si esto parece trivial; se volverá más interesante luego.

Un programa de agente para la Figura 2.9 se ve en la Figura 2.10. La función INTERPRET-INPUT genera una descripción abstracta del estado actual a partir del percepto, y la función RULE-MATCHING devuelve la primera regla en el conjunto de reglas que coincide con la descripción de estado dada. Vea que la descripción en términos de "reglas" y "coincidencias" es puramente conceptual; como señalamos, las ejecuciones reales son tan simples como una colección de compuertas lógicas que implementan un circuito booleano. Como alternativa, es posible usar un circuito "neuronal", donde las compuertas lógicas se reemplazan por unidades no lineales de las redes neuronales artificiales (vea el Capítulo 21).

[6] También llamadas reglas de conducción-acción, resultados o reglas **if-then**.

```
function SIMPLE-REFLEX-AGENT(percepción) devuelve una acción
  persistent: reglas, un conjunto de reglas condición-acción

  estado←INTERPRET-INPUT(percepción)
  regla←RULE-MATCH(estado, reglas)
  acción←regla.ACTION
  return acción
```

Figura 2.10 Un agente de reflejo simple. Actúa según una regla cuya condición coincide con el estado actual, definido por la percepción.

Los agentes de reflejo simple tienen la admirable propiedad de ser simples, pero con inteligencia limitada. El agente en la Figura 2.10 funcionará *sólo si es posible tomar la decisión correcta basada únicamente en el percepto actual, es decir, sólo si el entorno es por completo observable*.

La falta de observabilidad puede causar problemas. Así, la regla de frenado asume que la condición *el-auto-de-adelante-está-frenando* se puede determinar a partir de la percepción actual: un fotograma de video. Esto funciona si el auto de adelante tiene una luz de freno montada al centro (e identificable de modo inequívoco). Pero los autos antiguos tienen diferentes configuraciones de luces traseras, de freno y direccionales, y no siempre es posible decir a partir de una sola imagen si el auto está frenando o sólo tiene encendidas las luces traseras. Un agente de reflejo simple que conduce detrás de ese auto podría frenar de manera continua y sin necesidad, o, peor aún, nunca frenar.

Vemos un problema similar brotando en el mundo de la aspiradora. Suponga que un agente de aspiradora de reflejo simple carece de su sensor de ubicación y sólo tiene un sensor de suciedad. Un agente así tiene sólo dos percepciones posibles: [*Sucio*] y [*Limpio*]. Puede *Aspirar* en respuesta a [*Sucio*]; ¿qué haría en respuesta a [*Limpio*]? Moverse a la *izquierda* falla (siempre) si comienza en el cuadro A, y moverse a la *derecha* falla (siempre) si comienza en el cuadro B. Los bucles infinitos con frecuencia inevitables para los agentes de reflejo simple que operan en entornos en parte observables.

Aleatorizar

Escapar de los bucles infinitos es posible si el agente puede **aleatorizar** sus acciones. Así, si el agente de aspiradora percibe [*Limpio*], podría lanzar una moneda para elegir entre *Derecha* e *Izquierda*. Es fácil demostrar que el agente llegará al otro cuadro en un promedio de dos pasos. Entonces, si ese cuadro está sucio, el agente lo limpiará y la tarea se completará. Luego, un agente de reflejo simple aleatorizado superaría a un agente de reflejo simple determinista.

En la Sección 2.3 vimos que el comportamiento aleatorizado correcto puede ser racional en unos entornos multiagentes; pero en los de un solo agente, en general no es racional. Es un truco útil que ayuda a un agente de reflejo simple en unas situaciones, pero en la mayoría de los casos podemos hacer más con agentes deterministas más sofisticados.

2.4.3 Agentes de reflejo basados en modelos

Estado interno

La forma más efectiva de manejar la observabilidad parcial es que el *agente registre la parte del mundo que no puede ver en este momento*. Es decir, el agente debería mantener algún tipo de **estado interno** que dependa del historial perceptual y refleje aspectos no observados del estado actual. Para el problema de frenado, el estado interno no es demasiado extenso, sólo el fotograma anterior de la cámara, que permite al agente detectar cuándo se encienden o apagan simultáneamente ambas luces rojas en la zaga del vehículo. Para otras tareas de conducción, como cambiar de carril, el agente necesita registrar dónde están los otros autos si no es capaz de verlos todos a la vez. Y para poder siquiera comenzar a conducir, el agente necesita un registro de dónde están sus llaves.

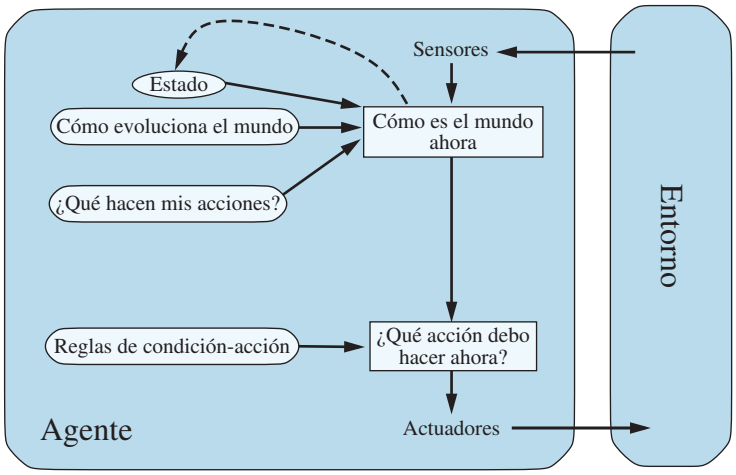


Figura 2.11 Un agente reflejo basado en modelos.

Conforme pasa el tiempo, la actualización de la información del estado interno requiere que dos tipos de conocimiento se codifiquen en el programa de agente. Uno, necesitamos información de cómo cambia el mundo con el tiempo; la cual se divide en dos: efectos de las acciones del agente y evolución del mundo independientemente del agente. Así, cuando el agente gira el volante en sentido horario, el auto vira a la derecha, y cuando llueve las cámaras del auto pueden mojarse. Este conocimiento sobre "cómo funciona el mundo", implementado en simples circuitos booleanos o en teorías científicas completas, se llama **modelo de transición del mundo**.

Modelo de transición

Dos, necesitamos información acerca de cómo se refleja el estado del mundo en los perceptos del agente. Por ejemplo, cuando el auto de adelante inicia el frenado, una o más regiones rojas iluminadas aparecen en la imagen de la cámara frontal, y, cuando ésta se moja, objetos en forma de gota aparecen en la imagen que obstruyen parcialmente la carretera. Este tipo de conocimiento se llama **modelo de sensor**.

Modelo de sensor

El modelo de transición y el modelo de sensor permiten que un agente registre el estado del mundo, en la medida de lo posible dadas las limitaciones de sus sensores. Un agente que usa esos modelos es un **agente basado en modelos**.

Agente basado en modelos

La Figura 2.11 muestra la estructura del agente de reflejo basado en modelos con estado interno, mostrando cómo el percepto actual se combina con el estado interno pasado para generar la descripción renovada del estado actual, basada en el modelo del agente de cómo funciona el mundo. El programa de agente se muestra en la Figura 2.12. La parte interesante es la función UPDATE-STATE, responsable de crear la nueva descripción del estado interno. Los detalles de cómo se representan los modelos y los estados varían según el tipo de entorno y la tecnología específica usada en el diseño del agente.

Independiente al tipo de representación, rara vez el agente podrá determinar *con exactitud* el estado actual de un entorno parcialmente observable. En cambio, el cuadro etiquetado "cómo es el mundo ahora" (Figura 2.11) representa la "mejor suposición" del agente (o las mejores suposiciones, si el agente considera múltiples posibilidades). Así, un taxi automatizado podría no ser capaz de ver alrededor del camión grande que se ha detenido frente a él y sólo haría conjeturas

```
function MODEL-BASED-REFLEX-AGENT(percepción) devuelve una acción
  persistente: estado, la concepción actual del agente sobre el estado del mundo
               Modelo de transición, una descripción de cómo el siguiente estado
               depende del estado actual y de la acción
               Modelo de sensor, una descripción de cómo el estado del entorno actual
               se refleja en las percepciones del agente
               reglas, un conjunto de reglas condición-acción
               reglas, un conjunto de reglas condición-acción

  estado←UPDATE-STATE(estado, acción, percepción, modelo de transición, modelo de sensor)
  regla←RULE-MATCH(estado, regla)
  acción←regla.ACCIÓN
  return acción
```

Figura 2.12 Agente de reflejo basado en modelos. Da seguimiento del estado actual del mundo mediante un modelo interno. Luego elige una acción al igual que el agente reflejo.

sobre lo que causaría la demora. Entonces, la incertidumbre sobre el estado actual es inevitable, pero el agente aún debe tomar una decisión.

2.4.5 Agentes basados en objetivos

Saber algo sobre el estado actual del entorno no siempre es suficiente para decidir qué hacer. Por ejemplo, en una intersección de carreteras, el taxi puede girar a la izquierda, a la derecha o seguir derecho. La decisión correcta depende de a dónde trata de llegar el taxi. En otras palabras, además de una descripción del estado actual, el agente necesita algún tipo de información acerca del **objetivo** que describa situaciones deseables, como tenr un destino particular. El programa de agente permite combinar esto con el modelo (la misma información usada en el agente de reflejo basado en modelos) para elegir acciones que logren el objetivo. La Figura 2.13 muestra la estructura del agente basada en objetivos.

Objetivo

A veces, la selección de acciones basada en objetivos es directa; como cuando la satisfacción del objetivo es inmediata a una sola acción. A veces será más complicado, como cuando el agente debe considerar largas secuencias de giros y vueltas para encontrar un modo de alcanzar el objetivo. La **búsqueda** (Capítulos 3 a 5) y la **planificación** (Capítulo 11) son subcampos de la IA dedicados a descubrir secuencias de acciones que logren los objetivos del agente.

Vea que la toma de decisiones de este tipo es en esencia diferente de las reglas de condición-acción ya descritas, pues implican el futuro, tanto "¿Qué sucederá si hago tal cosa?", como "¿Eso me hará feliz?". En los diseños de agentes de reflejo, esta información no se representa de modo explícito, porque las reglas incorporadas mapean de manera directa de las percepciones a las acciones. El agente de reflejo frena cuando ve las luces de freno, punto. No tiene idea de por qué. Un agente basado en objetivos frena cuando ve las luces de freno porque esa es la única acción que predice que logrará su meta de no chocar con otros autos.

Aunque el agente basado en objetivos parece menos eficiente, es más flexible porque el conocimiento que respalda sus decisiones se representa de modo explícito y puede ser modificado. Así, el comportamiento de un agente basado en objetivos cambiaría su destino con facilidad, especificando éste como el objetivo. Las reglas del agente de reflejo para cuándo

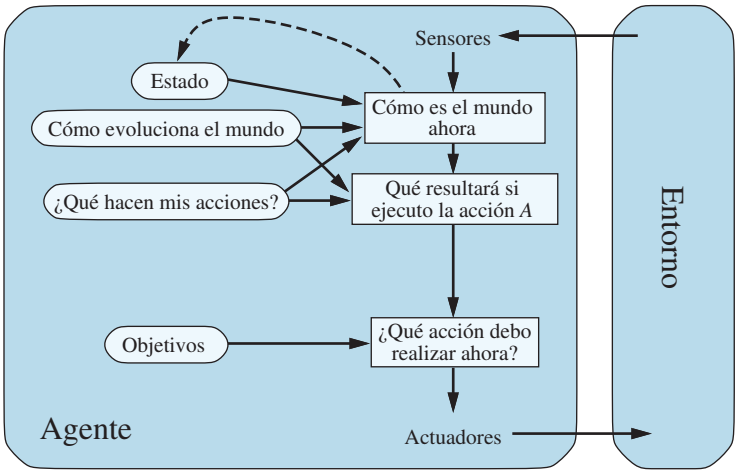


Figura 2.13 Un agente reflejo basado en modelos. Da seguimiento del estado mundial, al conjunto de objetivos que busca alcanzar y elige una acción que (con eventualidad) llevará al logro de sus objetivos.

girar y cuándo ir recto sólo funcionarán para un único destino; todas deben reemplazarse para ir a un lugar nuevo.

2.4.5 Agentes basados en utilidad

Los objetivos en sí no son suficientes para generar un comportamiento de calidad en la mayoría de entornos. Por ejemplo, muchas secuencias de acciones llevarán al taxi a su destino (logrando el objetivo), pero unas son más rápidas, seguras, confiables o económicas. Los objetivos sólo dan una distinción binaria básica entre estados "felices" e "infelices". Una medida de rendimiento más general debería permitir una comparación entre diferentes estados del mundo, según lo exacto de felices que harían al agente. Como "feliz" no suena muy científico, los economistas y científicos de computación mejor usan el término **utilidad**.[7]

Utilidad

Ya hemos visto que una medida de rendimiento asigna una puntuación a cualquier secuencia dada de estados del entorno, por lo que se distingue con facilidad entre formas más y menos deseables de lograr el destino del taxi. La **función de utilidad** de un agente es en esencia una internalización de la medida de rendimiento. Siempre que la función de utilidad interna y la medida de rendimiento externa concuerden, un agente que elige acciones para maximizar su utilidad será racional según la medida de rendimiento externa.

Función de utilidad

Enfaticemos de nuevo que esta no es la *única* forma de ser racional: ya vimos un programa de agente racional para el mundo de la aspiradora (Figura 2.8) que no tiene idea de cuál es su función de utilidad, pero, igual que los agentes basados en objetivos, uno basado en utilidad tiene muchas ventajas en términos de flexibilidad y aprendizaje. Además, en dos tipos de casos, los objetivos son inadecuados pero un agente basado en utilidad aún es capaz de tomar decisiones racionales. Primero, cuando hay objetivos en conflicto, es decir, que no todos puede lograrse a la vez (como, rapidez y seguridad), la función de utilidad especifica el compromi-

[7] La palabra "utilidad" refiere a "cualidad de ser útil", no a "servicios públicos" ("utility"en inglés).

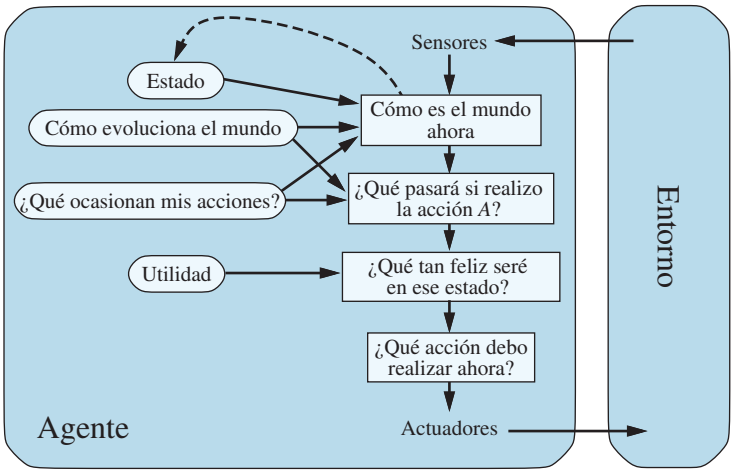


Figura 2.14 Un agente basado en modelos y utilidad. Utiliza un modelo del mundo, junto con una función de utilidad que mide sus preferencias entre los estados del mundo. Luego elige la acción que conduce a la mejor utilidad esperada, donde la utilidad esperada se calcula promediando todos los estados de resultado posibles, ponderados por la probabilidad del resultado.

so correcto. Segundo, cuando hay varios objetivos que el agente buscaría lograr, y ninguno se puede lograr con certeza, la utilidad da una forma en la que la probabilidad de éxito se pondera frente a la importancia de los objetivos.

La observación parcial y el no determinismo son ubicuos en el mundo real, y, por ello, la toma de decisiones bajo incertidumbre también lo es. Técnicamente hablando, un agente racional basado en utilidad elige la acción que maximiza la **utilidad esperada** de los resultados de la acción, es decir, la utilidad que el agente espera obtener, en promedio, dadas las probabilidades y utilidades de cada resultado. (El Apéndice A define esta expectativa con más precisión). En el Capítulo 16, vemos que cualquier agente racional debe comportarse *como si* poseyera una función de utilidad cuyo valor esperado intenta maximizar. Un agente que posee una función de utilidad *explícita* toma decisiones racionales con un algoritmo de propósito general que no depende de la función de utilidad específica que se maximice. Así, la definición "global" de racionalidad, que designa como racionales las funciones de agente que tienen el mejor rendimiento, se convierte en una restricción "local" en los diseños de agentes racionales que se expresan en un programa simple.

Utilidad esperada

La estructura del agente basado en utilidad se ve en la Figura 2.14, y los programas de agentes basados en utilidad en los Capítulos 16 y 17, donde diseñamos agentes de toma de decisiones que manejan la incertidumbre inherente en entornos no deterministas o parcialmente observables. La toma de decisiones en entornos multiagentes se ve en el marco de la teoría de utilidad, explicado en el Capítulo 18.

En este punto, el lector se preguntara: "¿Es tan simple? ¿Sólo construimos agentes que maximizan la utilidad esperada y ya?". Es cierto que tales agentes serían inteligentes, pero no es simple. Un agente basado en utilidad debe modelar y dar seguimiento de su entorno, tareas que involucran investigación en percepción, representación, razonamiento y aprendizaje. Los resultados de esa investigación llenan nuestros capítulos. Elegir el curso de acción que maximiza la utilidad es una tarea difícil, que requiere algoritmos ingeniosos que llenan varios capítulos más.

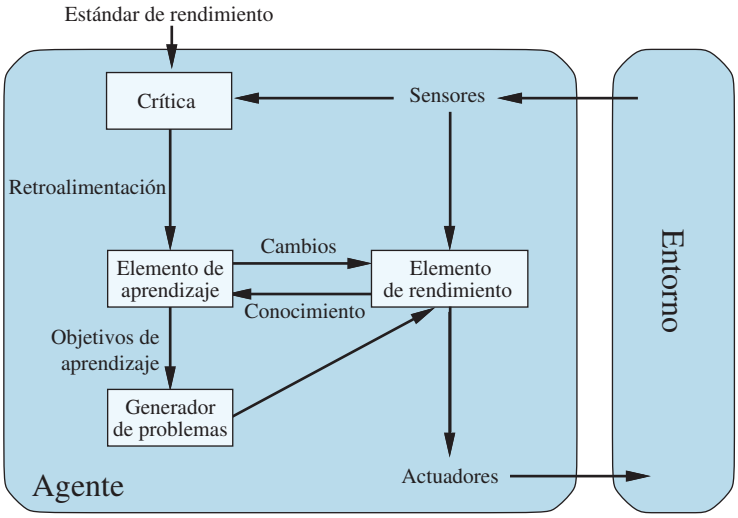


Figura 2.15 Agente de aprendizaje general. El cuadro "elemento de rendimiento" representa lo que ya antes consideramos el programa de agente completo. Ahora, el cuadro "elemento de aprendizaje" modifica ese programa para mejorar su rendimiento.

Aun con estos algoritmos, la racionalidad perfecta es general inalcanzable en la práctica debido a la complejidad computacional, como señalamos en el Capítulo 1. Además, no todos los agentes basados en utilidad se basan en modelos; en los Capítulos 22 y 26 veremos que un **agente sin modelo** es capaz de aprender cuál es la mejor acción en una situación particular sin aprender exactamente cómo esa acción cambia el entorno.

Agente sin modelo

Al final, todo supone que el diseñador puede especificar la función de utilidad correctamente; los capítulos 17, 18 y 22 analizan en detalle la cuestión de las funciones de utilidad desconocidas.

2.4.6 Agentes de aprendizaje

Hemos descrito programas de agentes con varios métodos para seleccionar acciones. Hasta ahora, no hemos explicado cómo *surgen* los programas de agentes. En su famoso artículo, Turing (1950) considera la idea de programar realmente sus máquinas inteligentes a mano. Estima cuánto trabajo le llevaría esto y concluye: "Un método más expeditivo parece deseable". El método que propone es construir máquinas de aprendizaje y luego enseñarles. En muchas áreas de IA, este es ahora el método preferido para crear sistemas de vanguardia. Cualquier tipo de agente (basado en modelos, en metas o en utilidad, etc.) puede construirse como un agente de aprendizaje (o no).

El aprendizaje tiene otra ventaja, como referimos: permite que el agente opere en entornos en inicio desconocidos y se vuelva más competente de lo que su conocimiento inicial permitiría. En esta sección, presentamos de modo breve las principales ideas de los agentes de aprendizaje. En el libro estudiamos las oportunidades y métodos para el aprendizaje en diferentes tipos de agentes. Los Capítulos 19-22 profundizan en los algoritmos de aprendizaje en sí mismos.

Un agente de aprendizaje se divide en cuatro componentes conceptuales, vea la Figura 2.15. La distinción fundamental es entre el **elemento de aprendizaje**, responsable de realizar mejoras, y el **elemento de rendimiento**, responsable de seleccionar acciones externas. El ele-

Agente de aprendizaje

Elemento de rendimiento

mento de rendimiento es lo que ya consideramos como agente completo: recibe percepciones y decide acciones. El elemento de aprendizaje usa la retroalimentación del **crítico** sobre cómo funciona el agente y determina cómo modificar el elemento de rendimiento para mejorarlo en el futuro.

Crítico

El diseño del elemento de aprendizaje depende mucho del diseño del elemento de rendimiento. Al intentar diseñar un agente que aprende determinada capacidad, la pregunta no es "¿Cómo voy a lograr que aprenda esto?" sino "¿Qué tipo de elemento de rendimiento usará mi agente para aplicar lo que ya aprendió?" Dado un diseño para el elemento de rendimiento, se pueden construir mecanismos de aprendizaje para mejorar cada parte del agente.

El crítico le dice al elemento de aprendizaje qué tan bien funciona el agente con respecto a un estándar de rendimiento fijo. La crítica es necesaria porque las percepciones mismas no indican el éxito del agente. Por ejemplo, un programa de ajedrez podría percibir que ha dado jaque mate a su oponente, pero necesita un estándar de rendimiento para saber que esto es algo bueno; la percepción en sí misma no lo dice. Es importante que el estándar de rendimiento sea fijo. Conceptualmente, se debe pensar en él como algo que está fuera del agente por completo porque el agente no debe modificarlo para que se ajuste a su propio comportamiento.

El último componente del agente de aprendizaje es el **generador de problemas,** responsable de sugerir acciones que lleven a experiencias nuevas e informativas. Si el elemento de rendimiento cumpliera su cometido, seguiría eligiendo las mejores acciones, dado lo que sabe, pero si el agente está dispuesto a explorar y elegir algunas acciones, tal vez, subóptimas a corto plazo, descubrirá acciones mejores a largo plazo. El trabajo del generador de problemas es sugerir acciones exploratorias. Esto hacen los científicos cuando experimentan. Galileo no pensaba que dejar caer rocas desde la cima de una torre en Pisa fuera valioso en sí mismo; no trataba de romper las rocas ni de modificar los cerebros de transeúntes desafortunados, su objetivo era modificar su propio cerebro al identificar una mejor teoría del movimiento de los objetos.

Generador de problemas

El elemento de aprendizaje puede realizar cambios en cualquiera de los componentes de "conocimiento" de los diagramas del agente (Figuras 2.9, 2.11, 2.13 y 2.14). Los casos más simples involucran el aprendizaje directamente de la secuencia de percepciones. La observación de pares de estados sucesivos del entorno permite que el agente aprenda "Qué hacen mis acciones" y "Cómo evoluciona el mundo" en respuesta a sus acciones. Así, si el taxi automatizado ejerce una presión de frenado al conducir en una carretera mojada, pronto descubrirá cuánta desaceleración se logra realmente y si se desliza fuera de la carretera. El generador de problemas identificaría las partes del modelo que necesitan mejora y sugerir experimentos, como probar los frenos en diferentes superficies de la carretera bajo diferentes condiciones.

Mejorar los componentes del modelo de un agente basado en modelos para ajustarlos a la realidad casi siempre resulta buena idea, independiente del estándar de rendimiento externo. (A veces, desde un punto de vista computacional, es mejor tener un modelo simple e inexacto comparado con un modelo perfecto, en vez de uno altamente complejo). La información del estándar externo es necesaria al intentar aprender un componente de reflejo o una función de utilidad.

Así, suponga que el agente de conducción de taxi no recibe propina de pasajeros sacudidos en el viaje. El estándar de rendimiento externo debe informar al agente que la pérdida de propinas es una derrama negativa a su rendimiento general; entonces el agente aprendería que las maniobras violentas no favorecen su propia utilidad. En cierto sentido, el estándar de rendimiento distingue parte de la percepción entrante como una **recompensa** (o **penalización**) que dan retroalimentación directa sobre la calidad del comportamiento del agente. Los estándares de rendimiento expresados como dolor y hambre en animales se pueden entender así.

Recompensa
Penalización

En general, las *elecciones humanas* proporcionan información sobre preferencias humanas. Por ejemplo, suponga que el taxi no sabe que, en general, a la gente no le gustan los ruidos fuertes y opta por la idea de tocar de manera continua su claxon como forma de asegurarse de que los peatones sepan que se acerca. El comportamiento humano resultante –cubrirse los oídos, usar malas palabras y quizá cortar los cables del claxon– evidenciaría al agente actualizar su función de utilidad. Este tema se discute en detalle en el Capítulo 22.

En resumen, los agentes tienen una variedad de componentes, y éstos permiten ser representados de muchas maneras dentro del programa de agente, por ello parece haber una gran variedad entre los métodos de aprendizaje. Sin embargo, hay un único tema unificador. El aprendizaje en agentes inteligentes se puede resumir como un proceso de modificación de cada componente del agente para alinearlos con la información de retroalimentación disponible, mejorando así el rendimiento general del agente.

2.4.7 Cómo funcionan los componentes de los programas de agentes

Ya describimos los programas de agentes (de modo general) como compuestos de varios componentes, cuya función es responder las preguntas: "¿Cómo es el mundo ahora?" "¿Qué acción debo hacer ahora?" "¿Qué hacen mis acciones?". La sucesiva pregunta para un estudiante de IA es: "¿Cómo funcionan estos componentes?" Se necesitan mil páginas para responder con corrección esa pregunta, pero aquí queremos llamar la atención del lector sobre algunas distinciones básicas entre las diversas formas en que los componentes pueden representar el entorno en el que habita el agente.

A grandes rasgos, podemos poner las representaciones a lo largo de un eje de complejidad y poder expresivo creciente: atómico, factorizado y estructurado. Para ilustrar esto, ayuda considerar un componente de agente particular, como el que se ocupa de "Qué hacen mis acciones". Este componente describe los cambios que ocurrirían en el entorno como resultado de ejecutar una acción. La Figura 2.16 ofrece representaciones esquemáticas de esas transiciones.

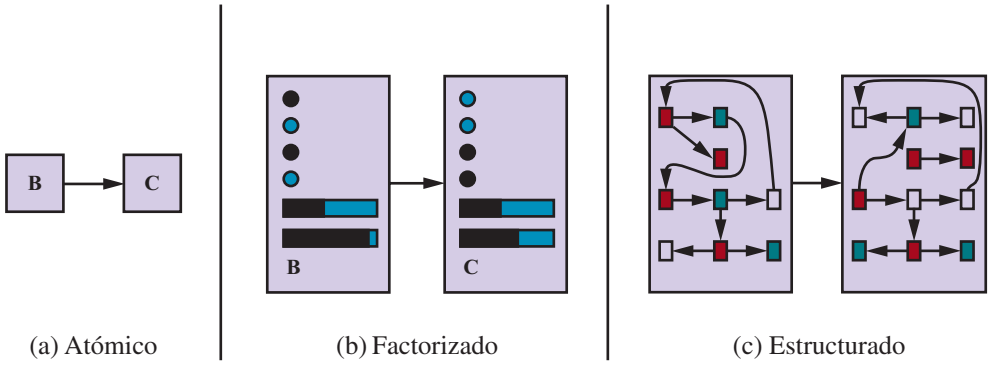


Figura 2.16 Tres formas de representar los estados y las transiciones entre ellos. (a) Representación atómica: un estado (como B o C) es una caja negra sin estructura interna; (b) Representación factorizada: un estado consta de un vector de valores de atributos; los valores pueden ser booleanos, de valor real o uno de un conjunto fijo de símbolos. (c) Representación estructurada: un estado incluye objetos, cada uno de los cuales podría tener atributos propios, así como relaciones con otros objetos.

Representación atómica

En una representación atómica, cada estado del mundo es indivisible, no tiene estructura interna. Veamos la tarea de encontrar una ruta de manejo de un extremo a otro de un país, por una secuencia de ciudades (vimos este problema en la Figura 3.1, p. 64). Para resolver este problema, puede ser suficiente reducir el estado del mundo sólo al nombre de la ciudad donde estamos, un solo átomo de conocimiento, una "caja negra" cuya única propiedad discernible es ser idéntica o diferente de otra caja negra. Los algoritmos estándar que subyacen a la búsqueda y los juegos de estrategia (Capítulos 3-5), los modelos ocultos de Markov ocultos (Capítulo 14) y los procesos de decisión de Markov (Capítulo 17) trabajan con representaciones atómicas.

Representación factorizada
Variable
Atributo
Valor

Una representación factorizada divide cada estado en un conjunto fijo de **variables** o **atributos**, y cada uno puede tener un **valor**. Considere una descripción más detallada para este problema de manejo, donde se debe preocupar por más que sólo la ubicación atómica en una ciudad u otra; se necesitaría atender cuánta gasolina hay en el tanque, las coordenadas GPS actuales, si funciona o no la luz de advertencia de aceite, cuánto dinero se tiene para peajes, qué sintoniza la radio, y así sucesivamente. Mientras que dos diferentes estados atómicos no tienen nada en común, son sólo cajas negras diferentes, dos estados factorizados diferentes podrían compartir algunos atributos (como una ubicación GPS particular) y otros no (como tener gasolina o carecer); esto facilita el proceso de convertir un estado en otro. Muchas áreas fundamentales de IA se basan en representaciones factorizadas, incluidos algoritmos de satisfacción de restricciones (Capítulo 6), lógica proposicional (Capítulo 7), planificación (Capítulo 11), redes bayesianas (Capítulos 12-16) y varios algoritmos de aprendizaje automático.

Para muchos propósitos, debemos entender que el mundo tiene *cosas* que están *relacionadas* entre sí, no sólo variables con valores. Así, podría ver que un camión frente a nosotros retrocede hacia la entrada de una granja lechera, pero una vaca suelta bloquea el camino. Es poco probable que una representación factorizada esté pre-equipada con el atributo *CamiónAdelanteRetrocediendoEnEntradaDeGranjaLecheraBloqueadoPorVacaSuelta* con valor *verdadero* o *falso*. En su lugar, necesitaríamos una **representación estructurada**, donde los objetos, vacas y camiones, y sus diversas y variables relaciones puedan describirse de forma explícita (vea la Figura 2.16(c)). Este tipo de representaciones subyacen a las bases de datos relacionales y la lógica de primer orden (Capítulos 8, 9 y 10), los modelos de probabilidad de primer orden (Capítulo 15) y gran parte de la comprensión del lenguaje natural (Capítulos 23 y 24). De hecho, gran parte de lo que los humanos expresan en el lenguaje natural se refiere a objetos y sus relaciones.

Representación estructurada

Como referimos, el eje en el que están las representaciones atómicas, factorizadas y estructuradas es el de la **expresividad** creciente. Es decir, una representación más expresiva permite capturar, de modo conciso, todo lo que una menos expresiva captura, y algo más. En general, el lenguaje más expresivo es *mucho* más conciso; así, las reglas del ajedrez se escriben en una o dos páginas de un lenguaje de representación estructurado, como la lógica de primer orden, pero requieren miles al escribirse en un lenguaje de representación factorizado como la lógica proposicional y unas 10^{38} páginas al escribirse en un lenguaje atómico como el de los autómatas finitos. Además, el razonamiento y el aprendizaje se hacen más complejos al aumentar el poder expresivo de la representación. Para lograr los beneficios de las representaciones expresivas y evitar sus inconvenientes, los sistemas inteligentes para el mundo real pueden necesitar operar de manera simultánea en todos los puntos del espectro de representaciones.

Expresividad

Otro eje para la representación involucra la asignación de conceptos a ubicaciones en la memoria física, ya sea en una computadora o en un cerebro. Si hay una asignación uno a uno entre conceptos y ubicaciones de memoria, la llamamos **representación local**. Además, si la

Representación local

Representación distribuida

representación de un concepto se distribuye en muchas ubicaciones de memoria, y cada una se emplea como parte de la representación de múltiples conceptos diferentes, la llamamos **representación distribuida**. Este tipo de representación es más robustas contra el ruido y la pérdida de información. Con una representación local, la asignación de concepto a ubicación de memoria es arbitraria, y si un error de transmisión distorsiona unos pocos bits, podríamos confundir *camión* con el concepto no relacionado de *camisón*. Pero con una representación distribuida, cada concepto de puede pensar como la representación de un punto en un espacio multidimensional, y si se distorsionan unos bits, se moverá a un punto cercano en ese espacio, que tendrá un significado similar.

Resumen

Este capítulo es un recorrido vertiginoso por la IA, concebida como la ciencia del diseño de agentes. Los puntos más significativos para recordar son:

- **Un agente** es algo que percibe y actúa en un entorno. La **función del agente** especifica la acción tomada por él en respuesta a una secuencia de percepciones.
- La **medida de rendimiento** evalúa el comportamiento del agente en un entorno. Un **agente racional** actúa para maximizar el valor esperado de la medida de rendimiento, dada la secuencia de percepciones que ha visto hasta ahora.
- Una **especificación del entorno** de la tarea incluye medida de rendimiento, entorno externo, actuadores y sensores. Al diseñar un agente, el primer paso siempre es especificar el entorno de la tarea tan completo como sea posible.
- Los entornos de tarea varían en dimensiones significativas. Pueden ser completa o parcialmente observables, de agente único o multiagente, deterministas o indeterministas, episódicos o secuenciales, estáticos o dinámicos, discretos o continuos, y conocidos o desconocidos.
- En casos en que la medida de rendimiento es desconocida o difícil de especificar con corrección, hay un riesgo significativo de que el agente optimice el objetivo incorrecto. En tales casos, el diseño del agente reflejaría incertidumbre del verdadero objetivo.
- El **programa de agente** implementa la función de éste. Hay una variedad de diseños básicos de programas de agentes que reflejan el tipo de información explícita usada en el proceso de decisión. Los diseños varían en eficiencia, tamaño y flexibilidad. El diseño apropiado del programa de agente depende de la naturaleza del entorno.
- Los **agentes de reflejo simple** responden directamente a las percepciones, pero los **agentes de reflejo basados en modelos** conservan un estado interno para rastrear aspectos del mundo no evidentes en la percepción actual. Los **agentes basados en objetivos** actúan para lograrlos, los **agentes basados en utilidad** buscan maximizar su propia "felicidad" esperada.
- Todos los agentes pueden mejorar su rendimiento a través del **aprendizaje**.

Notas bibliográficas e históricas

El papel central de la acción en la inteligencia, la noción de razonamiento práctico se remonta al menos a la Ética a *Nicomaco* de Aristóteles. El razonamiento práctico también fue el tema del influyente artículo de McCarthy "Programs with Common Sense" (1958). Los campos de la robótica y la teoría del control están, por su naturaleza propia, interesados en agentes físicos. El

concepto de un **controlador** en la teoría del control es idéntico al de un agente en IA. Quizás sorprende, la IA se ha concentrado durante la mayor parte de su historia en componentes aislados de agentes: sistemas de respuesta a preguntas, demostradores de teoremas, sistemas de visión, etc., en lugar de en agentes completos. La discusión sobre agentes en el texto de Genesereth y Nilsson (1987) fue una influencia excepcional. La visión de agentes completos ahora es ampliamente aceptada y es un tema central en textos recientes (Padgham y Winikoff, 2004; Jones, 2007; Poole y Mackworth, 2017). Controlador

El Capítulo 1 rastreó las raíces del concepto de racionalidad en la filosofía y la economía. En IA, el concepto fue de interés periférico hasta mediados de la década de 1980, cuando comenzó a impregnar muchas discusiones sobre las bases técnicas adecuadas para el campo. Un artículo de Jon Doyle (1983) predijo que el diseño de agentes racionales sería considerado la misión central de la IA, mientras que otros temas populares se separarían para formar nuevas disciplinas.

La atención cuidadosa a las propiedades del entorno y sus consecuencias para el diseño racional de agentes se ve en la tradición de la teoría del control, como en los sistemas de control clásicos (Dorf y Bishop, 2004; Kirk, 2004) manejan entornos por completo observables y deterministas; el control óptimo estocástico (Kumar y Varaiya, 1986; Bertsekas y Shreve, 2007) maneja entornos parcialmente observables y estocásticos; y el control híbrido (Henzinger y Sastry, 1998; Cassandras y Lygeros, 2006) se ocupa de entornos que contienen elementos discretos y continuos. La distinción entre entornos total y parcialmente observables es fundamental en la literatura de **programación dinámica**, desarrollada en el campo de investigación de operaciones (Puterman, 1994) y analizada en el Capítulo 17.

Aunque los agentes de reflejo simple fueron centrales en la psicología conductista (vea el Capítulo 1), los investigadores de IA los ven como demasiado simples para proporcionar mucha ventaja. (Rosenschein (1985) y Brooks (1986) cuestionaron esta suposición; vea el Capítulo 26). Se ha realizado mucho trabajo para encontrar algoritmos eficientes para realizar un seguimiento de entornos complejos (Bar-Shalom *et al.*, 2001; Choset *et al.*, 2005; Simon, 2006), la mayoría de ellos en el marco probabilístico.

Los agentes basados en objetivos se presuponen en todo, en la visión de la razón práctica de Aristóteles hasta los trabajos de McCarthy en IA lógica. Shakey the Robot (Fikes y Nilsson, 1971; Nilsson, 1984) fue la primera materialización robótica de un agente lógico basado en objetivos. Un análisis lógico completo de estos agentes apareció en Genesereth y Nilsson (1987), y una metodología de programación basada en agentes, programación orientada a agentes, fue desarrollada por Shoham (1993). El enfoque basado en agentes es ahora en extremo popular en la ingeniería de software (Ciancarini y Wooldridge, 2001). También ha infiltrado el área de los sistemas operativos, donde la **computación autónoma** se refiere a sistemas informáticos y redes que se monitorizan y controlan a sí mismos con un ciclo percibir-actuar y métodos de aprendizaje automático (Kephart y Chess, 2003). Vea que una colección de programas de agentes diseñados para funcionar bien juntos en un entorno multiagente real exhibe la necesidad de modularidad, los programas no comparten estado interno y se comunican sólo a través del entorno, es común dentro del campo de los **sistemas multiagente** diseñar el programa de un solo agente como una colección de subagentes autónomos. A veces, se puede demostrar que el sistema resultante da las mismas soluciones óptimas que un diseño monolítico. Cómputo automático

La visión basada en objetivos de los agentes también domina la tradición de la psicología cognitiva en el área de resolución de problemas, comenzando con el influyente *Human Problem Solving* (Newell y Simon, 1972) y pasando por todo el trabajo posterior de Newell (Newell, 1990). Los objetivos, analizados más a fondo como *deseos* (generales) e *intenciones* (actualmente perseguidas), son centrales en la influyente teoría de agentes desarrollada por Michael Bratman (1987).

Como se señaló en el Capítulo 1, el desarrollo de la teoría de la utilidad como base para el comportamiento racional se remonta a cientos de años atrás. En IA, las primeras investigaciones

evitaron las utilidades en favor de los objetivos, con algunas excepciones (Feldman y Sproull, 1977). El resurgimiento del interés en los métodos probabilísticos en la década de 1980 llevó a la aceptación de la maximización de la utilidad esperada como el marco más general para la toma de decisiones (Horvitz *et al*., 1988). El texto de Pearl (1988) fue el primero en IA en cubrir la teoría de la probabilidad y la utilidad en profundidad; su exposición de métodos prácticos para el razonamiento y la toma de decisiones bajo incertidumbre fue con gran probabilidad el factor más importante en el rápido cambio hacia agentes basados en utilidad en la década de 1990 (vea el Capítulo 16). La formalización del aprendizaje por refuerzo dentro de un marco de teoría de la decisión también contribuyó a este cambio (Sutton, 1988). Es notable, en casi toda la investigación en IA hasta hace muy poco se asumió que la medida de rendimiento puede especificarse de manera exacta y correcta en forma de una función de utilidad o función de recompensa (Hadfield-Menell *et al*., 2017a; Russell, 2019).

El diseño general para agentes de aprendizaje retratado en la Figura 2.15 es clásico en la literatura de aprendizaje automático (Buchanan *et al*., 1978; Mitchell, 1997). Ejemplos del diseño, como se incorporan en programas, se remontan al programa de aprendizaje de Arthur Samuel para jugar damas (1959, 1967). Los agentes de aprendizaje se discuten en profundidad en los Capítulos 19-22.

Algunos artículos tempranos sobre enfoques basados en agentes están recopilados por Huhns y Singh (1998) y Wooldridge y Rao (1999). Los textos sobre sistemas multiagente proporcionan una buena introducción a muchos aspectos del diseño de agentes (Weiss, 2000a; Wooldridge, 2009). Varias series de conferencias dedicadas a los agentes comenzaron en la década de 1990, incluyendo el Taller Internacional sobre Teorías, Arquitecturas y Lenguajes de Agentes (ATAL), la Conferencia Internacional sobre Agentes Autónomos (AGENTS) y la Conferencia Internacional sobre Sistemas Multiagente (ICMAS). En 2002, estos tres se fusionaron para formar la Conferencia Internacional Conjunta sobre Agentes Autónomos y Sistemas Multiagente (AAMAS). Desde 2000 hasta 2012 hubo talleres anuales sobre Ingeniería de Software Orientada a Agentes (AOSE). La revista *Autonomous Agents and Multi-Agent Systems* fue fundada en 1998. Finalmente, *Ecology of Dung Beetle* (Hanski y Cambefort, 1991) proporciona una gran cantidad de información interesante sobre el comportamiento de los escarabajos estercoleros. YouTube tiene inspiradoras grabaciones de video sobre sus actividades.

CAPÍTULO 3

RESOLUCIÓN DE PROBLEMAS DE BÚSQUEDA

En el que discutimos cómo un agente puede mirar hacia adelante para encontrar una secuencia de acciones que eventualmente lograrán su objetivo.

Si la acción correcta a tomar no es obvia, un agente necesitaría *planificar con anticipación*: pensar una *secuencia* de acciones que armen una vía hacia un estado objetivo. Tal agente será el **agente solucionador de problemas** y, el proceso computacional que emprende, **búsqueda**.

Agente solucionador de problemas

Búsqueda

Los agentes solucionadores de problemas usan representaciones atómicas (vea la Sección 2.4.7); es decir, los estados del mundo se piensan como un todo, sin estructura interna visible a los algoritmos solucionadores de problemas. Los agentes que usan representaciones **factorizadas** o **estructuradas** de estados son **agentes de planificación**, revisados en Capítulos 7 y 11.

Aquí cubriremos varios algoritmos de búsqueda, pero sólo los entornos más simples: episódicos, de un solo agente, totalmente observables, deterministas, estáticos, discretos y conocidos. Distinguimos entre algoritmos informados, donde el agente estima qué tan lejos está del objetivo, y algoritmos no informados, donde la estimación no está disponible. El Capítulo 4 flexibiliza las restricciones en los entornos, y el 5 considera múltiples agentes.

Este capítulo usa los conceptos de complejidad asintótica (es decir, notación $0(n)$). Los lectores que no familiarizados con estos conceptos deben consultar el Apéndice A.

3.1 Agentes solucionadores de problemas

Imagine un agente que disfruta vacacionar en Rumania. El agente quiere disfrutar de los lugares de interés, mejorar su rumano, disfrutar de la vida nocturna, evitar resacas, etc. El problema de decisión es complejo. Supongamos que el agente está en la ciudad de Arad y tiene un boleto no reembolsable para volar desde de Bucarest al día siguiente. El agente observa los letreros de las calles y ve que hay tres caminos que salen de Arad: uno hacia Sibiu, otro hacia Timisoara y otro a Zerind; ninguno de ellos es el objetivo, así que a menos que el agente conozca la geografía de Rumania, no sabrá qué camino seguir.[1]

Si el agente no tiene información adicional, es decir, si desconoce el entorno, entonces no puede hacer más que ejecutar una de las acciones al azar. Esta situación triste se discute en el Capítulo 4. Aquí, asumiremos que nuestros agentes siempre tienen acceso a información sobre el mundo, como a la información del mapa en la Figura 3.1, con la cual el agente puede seguir este proceso de resolución de problemas de cuatro fases:

- **Formulación del objetivo:** El agente adopta el **objetivo**: ir a Bucarest. Las metas organizan el comportamiento al limitar los objetivos y las acciones a considerar.

Formulación del objetivo

[1] Creemos que la mayoría de los lectores están en la misma situación y se imaginan tan despistados como nuestro agente. Nos disculpamos con los lectores rumanos que no puedan aprovechar este recurso pedagógico.

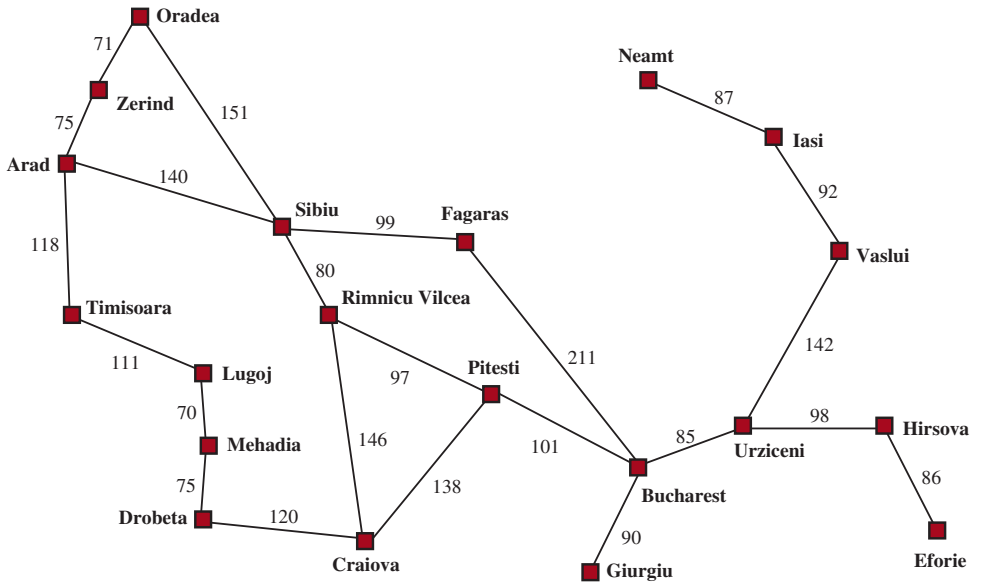


Figura 3.1 Mapa de carreteras simplificado de Rumania, con distancias en millas.

Formulación del problema

- **Formulación del problema:** Nuestro agente elabora una descripción de los estados y acciones necesarias para alcanzar el objetivo: un modelo abstracto de la parte relevante del mundo. Para él, un buen modelo considera las acciones de viajar de una ciudad a una ciudad adyacente y, por lo tanto, el único hecho sobre el estado del mundo que cambiará debido a una acción es la ciudad actual.

Búsqueda

- **Búsqueda:** Antes de tomar cualquier acción en el mundo real, el agente simula secuencias de acciones en su modelo, buscando hasta encontrar una secuencia de acciones que logre el objetivo. Tal secuencia se llama **solución**. El agente podría simular múltiples secuencias que no logren el objetivo, pero alguna encontrará una solución (como ir de Arad a Sibiu a Fagaras a Bucarest), o encontrará que ninguna solución es posible.

Solución

Ejecución

- **Ejecución:** El agente ahora puede ejecutar una por una las acciones en la solución.

Es una propiedad importante que, en un entorno del todo observable, determinista y conocido, *la solución a cualquier problema es una secuencia fija de acciones*: conducir a Sibiu, luego a Fagaras, luego a Bucarest. Si el modelo es correcto, cuando el agente encuentre una solución, puede ignorar sus percepciones al ejecutar las acciones, con los ojos cerrados, por así decir, porque la solución garantiza llegar al objetivo. Los teóricos del control lo llaman sistema de **lazo abierto (bucle abierto)**: ignorar las percepciones rompe el lazo agente–entorno. Si hay posibilidad de que el modelo sea incorrecto, o el entorno sea no determinista, entonces el agente estaría más seguro al usar un enfoque de **lazo cerrado** que monitorea las percepciones (vea la Sección 4.4).

Lazo abierto

Lazo cerrado

En entornos parcialmente observables o no deterministas, una solución sería una estrategia de ramal que sugiere diferentes acciones futuras según qué percepciones lleguen. Así, el agente planificaría conducir desde Arad hasta Sibiu, pero necesitaría un plan de contingencia si por accidente llega a Zerind o encuentra el letrero "Drum Închis" (*carretera cerrada*).

3.1.1 Problemas y soluciones de búsqueda

Un **problema** de búsqueda se define formalmente como:

Problema

Estado
Espacio de estados
Estado inicial
Estados objetivo

- Conjunto de posibles **estados** en los que estaría el entorno. Llamado **espacio de estados**.
- El **estado inicial** en el que el agente comienza. Por ejemplo: la ciudad de *Arad*.
- Un conjunto de uno o más **estados objetivo**. A veces hay un solo estado objetivo (Bucarest), a veces hay un pequeño conjunto de estados objetivo alternativos, y a veces el objetivo se define por una propiedad aplicable a muchos estados (un número infinito). Así, en el mundo de la aspiradora, el objetivo es ausencia de suciedad en todo lugar, con independencia de cualquier otro hecho sobre el estado. Consideraremos tres posibilidades al especificar un método IsGoal para un problema. Aquí a veces diremos "el objetivo" para simplificar, lo que también se aplica a "cualquiera de los posibles estados objetivo".
- Las **acciones** disponibles para el agente. Dado un estado s, Actions(s) devuelve un conjunto finito[2] de acciones que pueden ejecutarse en s. Decimos que cada una de estas acciones es **aplicable** en s. Un ejemplo:

  Acción
  Aplicable

  $$\text{Actions}(Arad) = \{ToSibiu, ToTimisoara, ToZerind\}.$$

- Un **modelo de transición**, que describe cada acción. Result(s,a) devuelve el estado que resulta de realizar la acción a en el estado s. Por ejemplo,

  Modelo de transición

  $$\text{Result}(Arad, ToZerind) = Zerind.$$

- Una **función de costo de acción**, denotada por Action-Cost(s,a,s') cuando programamos o $c(s,a,s')$ cuando hacemos matemáticas, que da el costo numérico de aplicar la acción a en el estado s para alcanzar el estado s'. Un agente solucionador de problemas debe usar una función de costo que refleje su propia medida de rendimiento; así, para agentes de búsqueda de rutas, el costo de una acción podría ser la longitud en millas (vea la Figura 3.1), o podría ser el tiempo que tarda en completar la acción.

función de costo de acción

Una secuencia de acciones crea un **camino;** una **solución** es un camino desde el estado inicial a un estado objetivo. Asumimos que los costos de las acciones son aditivos; es decir, el costo de un camino es la suma de costos individuales de las acciones. Una **solución óptima** tiene el costo de camino más bajo entre las soluciones. Admitimos que los costos de acción serán positivos, para evitar complicaciones.[3]

Camino

Solución óptica

El espacio de estados se representa como un **grafo** donde los vértices son estados y las aristas dirigidas entre ellos son acciones. El mapa de Rumania de la Figura 3.1 es el grafo, donde cada camino indica dos acciones, una en cada dirección.

Grafo

---

[2] Para problemas con un número infinito de acciones necesitaríamos técnicas ajenas a este capítulo.

[3] En todo problema con un ciclo de costo neto negativo, la solución rentable es recorrer ese ciclo un número infinito de veces. Los algoritmos Bellman-Ford y Floyd-Warshall (no vistos aquí) manejan acciones de costos negativos, siempre que no haya ciclos negativos. Es fácil dar cabida a acciones de costo cero, siempre que el número de acciones consecutivas de costo cero sea limitado. Así, podríamos tener un robot donde hay un costo por moverse, pero cero por girar 90°; los algoritmos del capítulo manejan esto siempre que no se permitan más de tres giros consecutivos de 90°. También existe una complicación con los problemas que tienen un número infinito de costos de acción arbitrariamente pequeños. Consideremos una versión de la paradoja de Zenón donde hay una acción para avanzar hasta la mitad del camino de la meta, a un costo de la mitad del movimiento previo. Este problema no tiene solución con un número finito de acciones, pero para evitar que una búsqueda realice un número ilimitado de acciones sin alcanzar la meta, podemos exigir que los costos de las acciones sean al menos ϵ, para algún pequeño valor positivo ϵ.

3.1.2 Formulación de problemas

Nuestra formulación del problema, llegar a Bucarest, es un **modelo**, una descripción matemática abstracta, no la cosa real. Compare la simple descripción de estado atómico de *Arad* con un viaje real por el país, donde el estado del mundo incluye: compañeros de viaje, programa de radio, paisaje por la ventana, proximidad de los agentes del orden público, distancia a la próxima parada, estado del camino, clima, tránsito, etc. Todas estas consideraciones se eliminan de nuestro modelo porque son irrelevantes para el problema: encontrar una ruta a Bucarest.

Abstracción

El proceso de eliminar detalles de una representación se llama **abstracción**. Una buena formulación del problema tiene el nivel adecuado de detalle. Si las acciones fueran al nivel de "mover el pie derecho un centímetro hacia adelante" o "girar el volante un grado a la izquierda", quizás el agente nunca saldría del estacionamiento, menos a Bucarest.

Nivel de abstracción

¿Podemos ser más precisos sobre el apropiado **nivel de abstracción**? Piense en los estados y acciones abstractos elegidos como correspondientes a grandes conjuntos detallados de estados del mundo y de secuencias de acciones. Ahora considere una solución al problema abstracto: por ejemplo, el camino de Arad a Sibiu a Rimnicu Vilcea a Pitesti a Bucarest. Esta solución abstracta concierne a un gran número de caminos más detallados. Así, podríamos conducir con la radio encendida entre Sibiu y Rimnicu Vilcea, y luego apagarla para el resto del viaje.

La abstracción es *válida* si podemos elaborar una solución abstracta en una solución en el mundo más detallado; una condición suficiente es que para cada estado detallado que esté "en Arad", haya un camino detallado hacia algún estado que esté "en Sibiu", etcétera.[4] La abstracción es *útil* si ejecutar cada acción en la solución es más fácil que el problema original; aquí, la acción "conducir de Arad a Sibiu" puede realizarse sin necesidad de más búsqueda o planificación por parte de un conductor con habilidades promedio. La elección de una buena abstracción implica eliminar tanto detalle como sea posible mientras se conserva la validez y se asegura que las acciones abstractas sean fáciles de realizar. Si no fuera por la capacidad de construir abstracciones útiles, los agentes inteligentes estarían por completo abrumados por el mundo real.

3.2 Ejemplos de problemas

El enfoque de resolución de problemas se ha aplicado a una vasta variedad de entornos de tarea. Enumeramos algunos de los más conocidos aquí, distinguiendo entre *problemas estandarizados* y *problemas del mundo real*. Un **problema estandarizado** busca ilustrar o ejercitar varios métodos de resolución de problemas. Puede recibir una descripción concisa y exacta y, por lo tanto, es adecuado como referencia para que los investigadores comparen el rendimiento de los algoritmos. Un **problema del mundo real**, como la navegación de robots, es uno cuyas soluciones ya se utilizan, y cuya formulación es idiosincrática, no estandarizada, porque, por ejemplo, cada robot tiene diferentes sensores que producen datos diferentes.

Problema estandarizado

Problema del mundo real

3.2.1 Problemas estandarizados

Un problema **mundo de rejilla** es un arreglo rectangular bidimensional de celdas cuadradas donde los agentes se mueven de una celda a otra. En general, el agente se mueve a cualquier celda adyacente libre de obstáculos, horizontal o verticalmente y, a veces, en diagonal. Las celdas pueden contener objetos, que el agente puede recoger, empujar o actuar de otra manera sobre ellos;

Mundo de rejilla

[4] Vea la Sección 11.4.

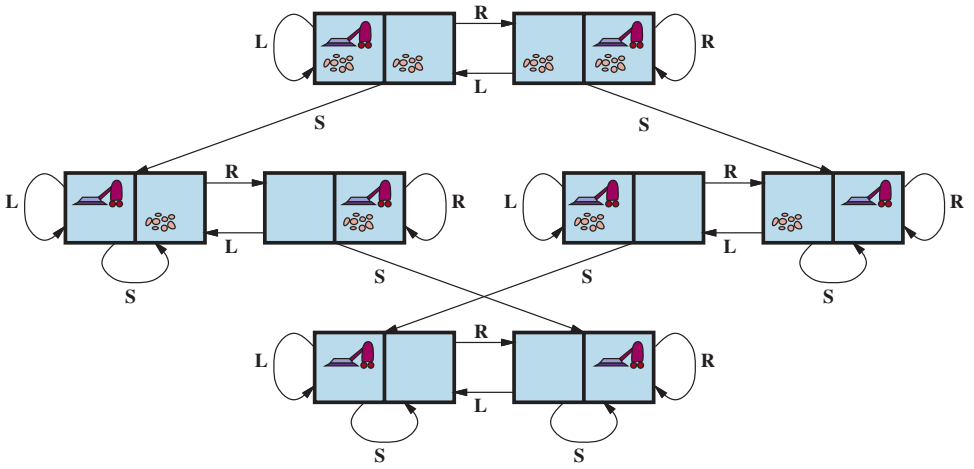


Figura 3.2 El gráfico del espacio de estados para el mundo de la aspiradora de dos celdas. Hay ocho estados y tres acciones para cada estado: L = *Izquierda*, R = *Derecha*, S = *Aspirar*.

una pared u otro obstáculo imposible de pasar en una celda impide que un agente se mueva hacia esa celda. El mundo de la aspiradora de la Sección 2.1 se formula como un problema de mundo de rejilla de la siguiente manera:

- **Estados:** Un estado del mundo dice qué objetos están en qué celdas. Para el mundo de la aspiradora, los objetos son agente y cualquier suciedad. En la versión simple de dos celdas, el agente estaría en cualquiera de las dos celdas, cada celda puede tener suciedad o no, por ello hay $2 \cdot 2 \cdot 2 = 8$ estados (vea la Figura 3.2). Así, un ambiente de aspiradora con n celdas tiene 2^n estados.
- **Estado inicial:** Cualquier estado puede ser designado como el estado inicial.
- **Acciones:** En el mundo de dos celdas definimos tres acciones: *Aspirar*, mover *a la izquierda* y mover *a la derecha*. En un mundo multicelda bidimensional necesitamos más acciones de movimiento. Podríamos sumar *Arriba* y *Abajo*, dándonos cuatro acciones de movimiento **absoluto**, o podríamos cambiar a **acciones egocéntricas**, definidas en relación con el punto de vista del agente: *Adelante, Atrás, Girar a la derecha* y *Girar a la izquierda*.
- **Modelo de transición:** *Aspirar* elimina cualquier suciedad de la celda del agente; *Adelante* mueve al agente una celda hacia adelante en la dirección en la mira, a menos que choque con una pared, en cuyo caso la acción no tiene efecto. *Atrás* mueve al agente en la dirección opuesta, mientras que *Girar a la derecha* y *Girar a la izquierda* cambian la dirección que mira en 90 grados.
- **Estados objetivo:** Los estados en los que todas las celdas están limpias.
- **Costo de la acción:** Cada acción cuesta 1.

Rompecabezas de sokoban

Otro tipo de mundo de rejilla es el **rompecabezas de sokoban**, donde el objetivo del agente es empujar cajas, dispersas por la rejilla, hacia ubicaciones de acopio designadas. Puede haber máximo una caja por celda. Cuando un agente avanza hacia una celda que contiene una caja y hay una celda vacía al otro lado de la caja, entonces la caja y el agente avanzan hacia adelante.

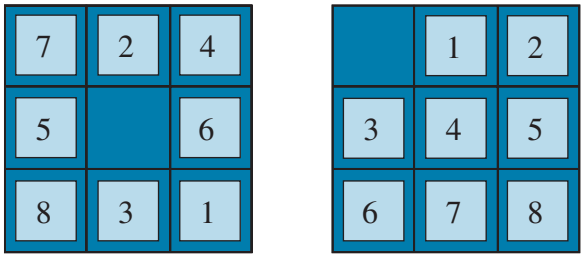


Figura 3.3 Un ejemplo típico del 8 puzzle.

El agente no puede empujar una caja hacia otra caja o una pared. Para un mundo con n celdas no obstaculizadas y b cajas, hay $n \times n!/(b!(n - b)!)$ estados; por ejemplo, en una cuadrícula de 8 × 8 con una docena de cajas, hay más de 200 billones de estados.

Rompecabezas de deslizamiento de fichas

En un **rompecabezas de piezas deslizantes (*puzzle*)**, un número de fichas (bloques o fichas) se colocan en una cuadrícula con uno o más espacios en blanco para que se puedan deslizar hacia el espacio en blanco. Una variante es el rompecabezas "Rush Hour", donde los autos se deslizan alrededor de una cuadrícula de 6 × 6 en un intento de liberar un auto del tráfico. Quizá la variante más conocida sea el **8-puzzle** (vea la Figura 3.3), una cuadrícula de 3 × 3 con ocho fichas numeradas y un espacio en blanco, y el **15-puzzle** en una cuadrícula de 4 × 4. El fin es alcanzar un estado objetivo especificado, como el de la derecha de la figura. La formulación estándar del 8 puzzle es:

8-puzzle

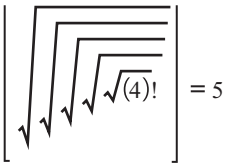
15-puzzle

- **Estados:** Una descripción del estado especifica la ubicación de cada ficha.
- **Estado inicial**: Cualquier estado puede ser designado como estado inicial. Vea que una propiedad de paridad divide el espacio de estados: cualquier objetivo se puede alcanzar desde exactamente la mitad de los estados iniciales posibles (vea el Ejercicio 3.PART).
- **Acciones:** Cuando en el mundo físico la ficha es la que se desliza, la forma más simple de describir una acción es pensar en el espacio en blanco moviéndose a la *Izquierda*, *Derecha*, *Arriba* o *Abajo*. Si este espacio está en un borde o esquina, entonces no todas las acciones serán aplicables.
- **Modelo de transición:** Asocia un estado y una acción a un estado resultante; así, si aplicamos *Izquierda* al estado inicial en la Figura 3.3, el estado resultante tiene el 5 y el espacio en blanco intercambiados.
- **Estado objetivo:** Aunque cualquier estado podría ser el objetivo, en general especificamos un estado con los números en orden, como en la Figura 3.3.
- **Costo de acción:** Cada acción cuesta 1.

Note que toda formulación de problema implica abstracciones. Las acciones del 8-puzzle se abstraen a sus estados inicial y final, ignorando las ubicaciones intermedias donde la ficha se desliza. Hemos abstraído acciones como agitar el tablero cuando las fichas se atascan y descartamos extraer las fichas con un cuchillo y volver a colocarlas. Nos queda una descripción de las reglas, evitando los detalles de manipulación física.

Nuestro problema estandarizado final, ideado por Donald Knuth (1964), ilustra cómo surgen espacios de estados infinitos. Knuth conjeturó que comenzando con el número 4, una secuencia

de operaciones de raíz cuadrada, redondeo hacia abajo y factorial alcanza cualquier entero positivo deseado. Así, podemos llegar a 5 desde 4 como:



La definición del problema es simple:

- **Estados:** Números reales positivos.
- **Estado inicial:** 4.
- **Acciones:** Aplicar la operación de raíz cuadrada, redondeo hacia abajo o factorial (factorial sólo para enteros).
- **Modelo de transición:** Según las definiciones matemáticas de las operaciones.
- **Estado objetivo:** El entero positivo deseado.
- **Costo de acción:** Cada acción cuesta 1.

El espacio de estados para este problema es infinito: para cualquier entero mayor que 2, el operador factorial siempre producirá un entero más grande. El problema es interesante porque explora números grandes: el camino más corto hacia 5 pasa por (4!)! = 620 448 401 733 239 439 360 000. Los espacios de estados infinitos suelen surgir en tareas que implican generación de expresiones matemáticas, circuitos, demostraciones, programas y objetos definidos recursivamente.

3.2.2 Problemas del mundo real

Ya definimos el **problema de búsqueda de rutas** en términos de ubicaciones y transiciones específicas a lo largo de conexiones entre ellas. Los algoritmos de búsqueda de rutas se usan en gran variedad de aplicaciones. Como los sitios web y sistemas de indicaciones para conducir autos, extensiones relativamente sencillas del ejemplo de Rumania. (Las principales complicaciones son los costos variables por los retrasos causados por tráfico y cambio de ruta por cierre de caminos). Otros, como enrutamiento de flujos de video en redes informáticas, planificación de operaciones militares y sistemas de planificación de viajes aéreos involucran especificaciones más complejas que deben resolverse mediante sitios web:

- **Estados:** Cada estado incluye una ubicación (como, un aeropuerto) y el tiempo actual. Además, debido a que el costo de una acción (un segmento de vuelo) puede depender de segmentos previos, sus bases tarifarias y su condición de domésticos o internacionales, el estado debe registrar información adicional sobre estos aspectos "históricos".
- **Estado inicial:** El aeropuerto de origen del usuario.
- **Acciones:** Tomar un vuelo desde la ubicación actual, en cualquier clase de asiento, salir después del tiempo actual, con tiempo para el traslado dentro del aeropuerto, si es necesario.
- **Modelo de transición:** El estado resultante de tomar un vuelo tendrá como nueva ubicación el destino del vuelo y como nuevo tiempo el de llegada del vuelo.
- **Estado objetivo:** Una ciudad de destino. A veces, el objetivo será más complejo, como "llegar al destino en vuelo directo".
- **Costo de acción:** Combinación de costo monetario, tiempos de espera y de vuelo, procedimientos de aduana e inmigración, calidad del asiento, hora del día, tipo de avión, puntos de recompensa de viajeros frecuentes, etc.

Los sistemas de recomendación de viaje usan una formulación de problema de este tipo, con muchas complicaciones adicionales para manejar las intrincadas estructuras tarifarias de las aerolíneas. Cualquier viajero con experiencia sabe que no todo vuelo va según lo planeado. Un sistema en verdad bueno incluiría planes de contingencia: ¿qué pasa si su vuelo se retrasa y se pierde la conexión?

Problemas de recorrido

Problema del agente viajero (TSP)

Los **problemas de recorrido** describen un conjunto de ubicaciones por visitar, en vez de un solo destino. El **problema del agente viajero** (TSP, por sus siglas en inglés) es un problema de recorrido donde se debe visitar cada ciudad en un mapa. El objetivo es encontrar un recorrido con un costo $< C$ (u, optimizado, encontrarlo con el costo más bajo posible). Se ha dedicado mucho esfuerzo para mejorar las capacidades de los algoritmos de TSP. Éstos también pueden extenderse al manejo de flotas de vehículos. Así, un algoritmo de búsqueda y optimización para enrutamiento de autobuses escolares en Boston ahorró $5 millones de dólares, redujo el tráfico, la contaminación del aire y ahorró tiempo a conductores y estudiantes (Bertsimas *et al.*, 2019). Además de planificar viajes, los algoritmos de búsqueda se usan para planificar los movimientos de taladros de circuitos automáticos y máquinas de provisión en fábricas.

Diseño VLSI

El problema de **diseño VLSI** requiere posicionar millones de componentes y conexiones en un chip para minimizar el área, los retrasos del circuito y las capacitancias parásitas, así como maximizar el rendimiento de fabricación. El problema de diseño surge tras la fase de diseño lógico y, en general, se divide en dos partes: **diseño de celda** y **enrutamiento de canal**. En el primero, los componentes primitivos del circuito se agrupan en celdas, cada cual realiza una función reconocida. Cada celda tiene un área de ocupación fija (tamaño y forma) y requiere cierto número de conexiones con cada una de las otras celdas. El objetivo es colocar las celdas en el chip de modo que no se superpongan y haya espacio para los cables de conexión entre las celdas. El enrutamiento de canal encuentra una ruta específica para cada cable por los espacios entre celdas. Estos problemas de búsqueda son muy complejos, pero vale la pena resolverlos.

Navegación autónoma

La **navegación autónoma** es una generalización del problema de búsqueda de rutas ya visto. En vez de seguir caminos distintos (como las carreteras en Rumania), un robot deambula, en efecto, creando sus propios caminos. Para un robot circular moviéndose en superficie plana, el espacio es bidimensional. Cuando el robot tiene brazos y piernas que también deben ser controlados, el espacio de búsqueda se hace multidimensional: una dimensión por cada ángulo de articulación. Se requieren técnicas avanzadas sólo para hacer que el espacio de búsqueda en esencia continuo sea finito (vea el Capítulo 26). Además de la complejidad del problema, los robots reales lidian con errores en sus lecturas de sensores y controles de motores, con observabilidad parcial y con otros agentes que podrían alterar el entorno.

Secuenciación automática de ensamblaje

La **secuenciación automática de ensamblaje** de objetos complejos (como motores eléctricos) por un robot ha sido práctica estándar en la industria desde 1970. Los algoritmos primero encuentran una secuencia de ensamblaje factible y luego trabajan para optimizar el proceso. Minimizar la cantidad de trabajo manual humano en la línea de ensamblaje produce ahorros significativos en tiempo y costos. En problemas de ensamblaje, el objetivo es encontrar un orden para ensamblar las partes de algún objeto. Si se elige el orden incorrecto, no habrá forma de agregar alguna parte más tarde en la secuencia sin deshacer parte del trabajo ya realizado. Comprobar la factibilidad de una acción en la secuencia es un problema de búsqueda geométrica difícil relacionado con la navegación de robots. La generación de acciones legales es la parte costosa de la secuenciación de ensamblaje. Un problema de ensamblaje importante es el **diseño de proteínas**, donde el objetivo es encontrar una secuencia de aminoácidos que se pliegue en una proteína tridimensional con las propiedades correctas para curar enfermedades.

Diseño de proteínas

3.3 Algoritmos de búsqueda

Un **algoritmo de búsqueda** toma un problema de búsqueda como entrada y devuelve una solución, o una indicación de fracaso. Aquí consideramos algoritmos que superponen un **árbol de búsqueda** sobre el grafo del espacio de estados, formando varios caminos desde el estado inicial, buscando un camino al estado objetivo. Cada **nodo** en el árbol de búsqueda corresponde a un estado en el espacio de estados y las aristas en el árbol de búsqueda son acciones. La raíz del árbol es el estado inicial del problema.

Algoritmo de búsqueda

Nodo

Es importante entender la distinción entre el espacio de estados y el árbol de búsqueda. El primero describe el conjunto (quizás infinito) de estados en el mundo, y las acciones que permiten transiciones de un estado a otro. En cambio el árbol de búsqueda describe los caminos entre esos estados, al avanzar al objetivo. Este árbol puede tener múltiples caminos a (y, por lo tanto, múltiples nodos para) cualquier estado dado, pero cada nodo en el árbol tiene un camino único de regreso a la raíz (como en los árboles).

La Figura 3.4 muestra los primeros pasos para encontrar un camino desde Arad hasta Bucarest. El nodo raíz del árbol de búsqueda está en el estado inicial, *Arad*. Podemos **expandir** el nodo,

Expandir

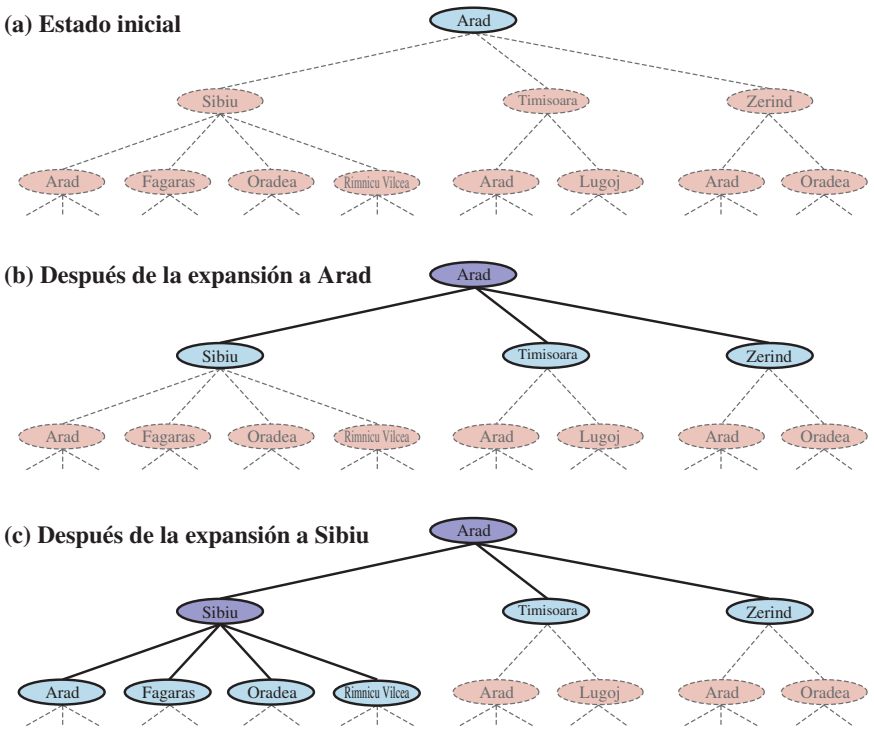


Figura 3.4 Tres árboles de búsqueda parciales para hallar una ruta de Arad a Bucarest. Los nodos *ampliados* son de color lavanda con letras en negrita; los nodos en la frontera *generados* pero aún no expandidos están en verde; se dice que se ha *alcanzado* el conjunto de estados que corresponde a ambos tipos de nodos. Los nodos que podrían generarse se ven en líneas discontinuas tenues. Note que en el árbol inferior hay un ciclo de Arad a Sibiu y a Arad; ese no puede ser un camino óptimo, por ello la búsqueda no debe continuar desde allí.

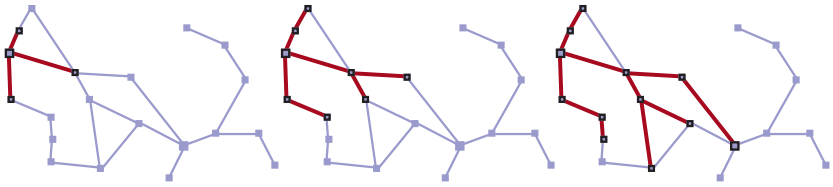


Figura 3.5 Secuencia de árboles de búsqueda generados por una búsqueda gráfica sobre el problema de Rumania de la Figura 3.1. En cada etapa, hemos expandido cada nodo de la frontera, extendiendo cada camino con todas las acciones aplicables que no resulten en un estado ya alcanzado. Note que en la tercera etapa, la ciudad de más arriba (en el gráfico) (Oradea) tiene dos sucesoras, ambas ya han sido alcanzadas por otros caminos, por lo que no se expande ningún camino desde Oradea.

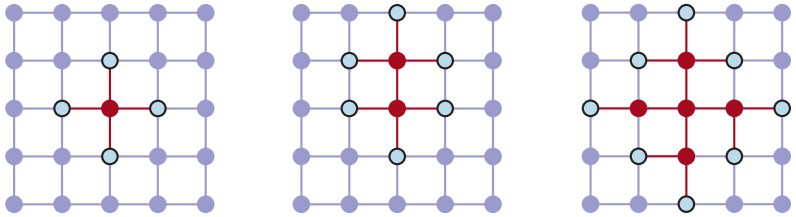


Figura 3.6 La propiedad de separación de la búsqueda de gráficos, ilustrada en un problema de cuadrícula rectangular. El borde (verde) separa el interior (lavanda) del exterior (discontinuo tenue). La frontera es el conjunto de nodos (y estados correspondientes) alcanzados, pero aún no expandido; el interior es el conjunto de nodos (y estados correspondientes) expandidos; y el exterior es el conjunto de estados que no se han alcanzado. En (a), sólo se expandió la raíz. En (b), se expande el nodo de la frontera superior. En (c), los estados restantes de la raíz se expanden en el sentido de las agujas del reloj.

considerando las ACTIONS disponibles para ese estado, usando la función RESULT para ver a dónde conducen esas acciones y **generar** un nuevo nodo (**nodo hijo** o **sucesor**) para cada estado resultante. Cada nodo hijo tiene a Arad como **nodo padre**.

Generar
Nodo hijo
Nodo sucesor
Nodo padre

Ahora elegimos cuál de los tres nodos hijos considerar. Esta es la esencia de la búsqueda: seguir una opción ahora y dejar para después las otras. Suponga que elegimos expandir Sibiu primero. La Figura 3.4 (abajo) muestra el resultado: un conjunto de 6 nodos no expandidos (resaltados en negrita). Llamamos a esto la **frontera** del árbol de búsqueda. Decimos que cualquier estado para el cual se ha generado un nodo ha sido **alcanzado** (haya sido expandido o no ese nodo).[5] La Figura 3.5 muestra el árbol de búsqueda superpuesto en el grafo del espacio de estados.

Frontera

Alcanzado

Note que la frontera **separa** dos regiones del grafo del espacio de estados: una región interior donde se han expandido todos los estados, y una exterior de estados que aún no han sido alcanzados. Esta propiedad se ilustra en la Figura 3.6.

Separador

[5] Unos autores llaman **lista abierta** a la frontera, geográficamente menos evocadora y computacionalmente menos apropiada, porque aquí una fila es más eficiente que una lista. Esos autores usan el término **lista cerrada** para referirse al conjunto de nodos antes expandidos, que en nuestra terminología serían los nodos alcanzados menos la frontera.

```
function BEST-FIRST-SEARCH(problem, f) returns un nodo solución or failure
    node ←NODE(STATE=problem.INITIAL)
    frontier ← una cola de prioridad ordenada por f, con node como elemento
    reached ← una tabla de búsqueda, con una entrada con ID problem.INITIAL y un valor node
    while not IS-EMPTY(frontier) do
        node ←POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s no está en reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                agregar child en frontier
    return failure

function EXPAND( problem, node) yields nodos
    s ← node.STATE
    for each action in problem.ACTIONS(s) do
        s′ ← problem.RESULT(s, action)
        cost ← node.PATH-COST + problem.ACTION-COST(s, action, s′)
        yield NODE(STATE=s′, PARENT=node, ACTION=action, PATH-COST=cost)
```

Figura 3.7 El algoritmo de búsqueda del best-first y la función para expandir un nodo. Las estructuras de datos usadas se describen en la Sección 3.3.2. Vea el Apéndice B para conocer el **rendimiento**.

3.3.1 Búsqueda del best-first

búsqueda del *best-first*
Función de evaluación

¿Cómo decidimos qué nodo de la frontera expandir después? Un enfoque general es la **búsqueda del best-first**, donde elegimos un nodo, n, con el valor mínimo de una **función de evaluación**, $f(n)$. La Figura 3.7 muestra el algoritmo. En cada iteración elegimos un nodo en la frontera con el valor mínimo de $f(n)$, lo devolvemos si su estado es objetivo, de lo contrario, aplicamos EXPAND para generar nodos hijos. Cada nodo hijo se agrega a la frontera si antes no ha sido alcanzada, o es agregado de nuevo si se ha alcanzado con un camino de un costo más bajo que cualquier camino previo. El algoritmo devuelve o una indicación de fracaso o un nodo que representa un camino hacia un objetivo. Al emplear diferentes funciones $f(n)$, obtenemos diferentes algoritmos específicos, a revisar en este capítulo.

3.3.2 Estructuras de datos de búsqueda

Los algoritmos de búsqueda requieren una estructura de datos para dar seguimiento al árbol de búsqueda. Un **nodo** en el árbol se representa mediante una estructura de datos con cuatro componentes:

- *nodo*.STATE: el estado al que corresponde el nodo;
- *nodo*.PARENT: el nodo en el árbol que generó este nodo;
- *nodo*.ACTION: la acción que se aplicó al estado del padre para generar este nodo;
- *nodo*.PATH-COST: costo total del camino, del estado inicial a este nodo. En fórmulas matemáticas, usamos $g(node)$ sinónimo de PATH-COST.

Siguiendo los punteros PARENT desde un nodo podemos recuperar los estados y acciones a lo largo del camino hacia ese nodo. Haciendo esto desde un nodo objetivo nos da la solución.

Frontera (*frontier*)
Cola (*queue*)

Necesitamos una estructura de datos para almacenar la **frontera** (*frontier*). La elección apropiada es una **cola** (*queue*) de algún tipo, porque las operaciones en una frontera son:

- IS-EMPTY(*frontier*) devuelve verdadero sólo si no hay nodos en la frontera.
- POP(*frontier*) elimina el nodo superior de la frontera y lo devuelve.
- TOP(*frontier*) devuelve (pero no elimina) el nodo superior de la frontera.
- ADD(node,*frontier*) inserta el nodo en su lugar adecuado en la cola.

Se utilizan tres tipos de colas en algoritmos de búsqueda:

Cola de prioridad
FIFO
LIFO
Pila

- Una **cola de prioridad** primero extrae el nodo con el costo mínimo según alguna función de evaluación, *f*. Se usa en la búsqueda del *best-first*.
- Una **cola FIFO** o cola primero en entrar-primero en salir, extrae los nodos en el orden de llegada a la cola; veremos que se usa en la búsqueda de amplitud.
- Una **cola LIFO** o cola último en entrar-primero en salir (también conocida como **pila**) extrae primero el nodo recién agregado; veremos que se usa en la búsqueda de profundidad.

Los estados alcanzados pueden almacenarse como una tabla de búsqueda (por ejemplo, una tabla hash) donde cada clave es un estado y cada valor es el nodo para ese estado.

3.3.3 Caminos redundantes

Estado repetido
Ciclo
Ruta en bucle

El árbol de búsqueda de la Figura 3.4 (abajo) incluye un camino de Arad a Sibiu y de regreso a Arad. Decimos que *Arad* es un **estado repetido** en el árbol de búsqueda, creado aquí por un **ciclo** (o **ruta en bucle**). Así, aunque el espacio de estados tiene sólo 20 estados, el árbol de búsqueda completo es *infinito* pues no hay límite en la frecuencia con la que se recorre un bucle.

Camino redundante

Un ciclo es un caso especial de un **camino redundante**. Así, podemos llegar a Sibiu por el camino Arad-Sibiu (140 millas) o el camino Arad-Zerind-Oradea-Sibiu (297 millas). Esta segunda ruta es redundante: es sólo una peor forma de llegar al mismo estado, no necesita considerarse en nuestra búsqueda de caminos óptimos.

Piense en un agente en un mundo cuadriculado de 10 × 10, con capacidad de moverse a cualquiera casillas de 8 adyacentes. Sin trabas, el agente puede alcanzar cualquiera de las 100 casillas en 9 movimientos o menos. Pero el número de caminos de longitud 9 es casi 8^9 (menos, dados los bordes de la cuadrícula) o más de 100 millones. Es decir, la casilla promedio sería alcanzada por más de un millón de caminos redundantes de longitud 9, pero si eliminamos estos caminos, completaremos una búsqueda casi un millón de veces más rápido. Dice el dicho, *los algoritmos que no recuerdan el pasado están condenados a repetirlo*. Este problema tiene tres enfoques.

Primero, podemos recordar todos los estados alcanzados antes (como lo hace la búsqueda del best-first), lo que nos permite detectar todos los caminos redundantes y mantener sólo el mejor para cada estado. Esto es apropiado para espacios de estados con muchos caminos redundantes, y es la opción preferida cuando la tabla de estados alcanzados cabe en la memoria.

Segundo, podemos no preocuparnos por repetir el pasado. Hay formulaciones de problemas donde es raro o imposible que dos caminos lleguen al mismo estado. Por ejemplo, un problema de ensamblaje donde cada acción agrega una parte a un ensamblaje en evolución, y hay un orden de partes de modo que es posible agregar *A* y luego *B*, pero no *B* y luego *A*. Para esto, podríamos ahorrar espacio en memoria si *no* seguimos los estados alcanzados y no verificamos los caminos redundantes. Llamamos a un algoritmo de **búsqueda basada en grafos** si verifica caminos

Búsqueda basada en grafos

redundantes y **búsqueda de tipo árbol**[6] si no lo hace. El algoritmo BEST-FIRST-SEARCH en la Figura 3.7 es un algoritmo de búsqueda de grafos; si eliminamos todas las referencias a *alcanzar* obtenemos una búsqueda de tipo árbol que usa menos memoria pero examinará caminos redundantes al mismo estado, y por ello se ejecutará más lentamente.

Búsqueda de tipo árbol

Tercero, podemos comprometernos y verificar ciclos, pero no caminos redundantes en general. Dado que cada nodo tiene una cadena de punteros de padres, podemos verificar ciclos sin necesidad de memoria adicional siguiendo la cadena de padres para ver si el estado al final del camino ya ha aparecido antes en el camino. Algunas ejecuciones siguen esta cadena hasta arriba, y así eliminan todos los ciclos; otras ejecuciones siguen sólo unos enlaces (al padre, abuelo y bisabuelo), y así sólo toman una cantidad constante de tiempo, mientras eliminan todos los ciclos cortos (y confiando en otros mecanismos para lidiar con los ciclos largos).

3.3.4 Medición del rendimiento en la resolución de problemas

Antes de adentrarse en el diseño de algoritmos de búsqueda, se considerarán criterios usados para elegir entre ellos. Se puede evaluar el rendimiento de un algoritmo de cuatro maneras:

- **Completitud:** ¿Garantiza que el algoritmo encontrará una solución cuando exista una, y reportará correctamente el fracaso cuando no haya? — Completitud
- **Optimalidad del costo:** ¿Halla una solución con el costo de camino más bajo de todas?[7] — Optimalidad del costo
- **Complejidad temporal:** ¿Cuánto tarda en encontrar una solución? Esto se puede medir en segundos, o en abstracto por el número de estados y acciones considerados. — Complejidad temporal
- **Complejidad espacial:** ¿Cuánta memoria se necesita para realizar la búsqueda? — Complejidad espacial

Para comprender la completitud, se piensa en un problema de búsqueda con un solo objetivo. Éste podría estar en cualquier lugar del espacio de estados; así, un algoritmo completo debe ser capaz de explorar sistemáticamente cada estado alcanzable desde el estado inicial. En espacios de estados finitos es fácil de lograr: siempre y cuando llevemos un registro de los caminos y cortemos los que son ciclos (como, de Arad a Sibiu a Arad), eventualmente alcanzaremos cada estado alcanzable.

En espacios de estados infinitos, se necesita más cuidado. Por ejemplo, un algoritmo que aplicara repetidamente el operador "factorial" en el problema "4" de Knuth seguiría un camino infinito de 4 a 4! a (4!)!, etc. Asimismo, en una cuadrícula infinita sin obstáculos, moverse repetidamente hacia adelante en línea recta también sigue un camino infinito de nuevos estados. En ambos casos, el algoritmo nunca vuelve a un estado que haya alcanzado antes, pero es incompleto porque amplias extensiones del espacio de estados nunca se alcanzan.

Para ser completo, un algoritmo de búsqueda debe ser **sistemático** para explorar un espacio de estados infinito y asegurar que podría alcanzar cualquier estado conectado al estado inicial. Así, en la cuadrícula infinita, un tipo de búsqueda sistemática es un camino en espiral que cubre todas las casillas a s pasos del origen antes de moverse a las casillas que están a $s + 1$ pasos de distancia. Con poco acierto, en un espacio de estados infinito sin solución, un algoritmo sólido necesita buscar por siempre; no puede terminar porque no sabe si el próximo estado será un objetivo.

Sistemático

La complejidad temporal y espacial se piensan con respecto a alguna medida de la dificultad del problema. En ciencias de la computación, la medida típica es el tamaño del grafo de espacio de estados, $|V| + |E|$, donde $|V|$ es el número de vértices (nodos de estado) del grafo y $|E|$ es el

[6] Decimos "búsqueda en forma de árbol" porque el espacio de estados sigue siendo el mismo gráfico sin importar cómo lo busquemos; sólo elegimos tratarlo *como si* fuera un árbol, con sólo una ruta desde cada nodo hasta la raíz.

[7] Algunos autores usan el término "admisibilidad" para la propiedad de encontrar la solución de menor costo, otros usan sólo "optimalidad", pero eso puede confundirse con otros tipos de optimización.

número de aristas (pares de estado/acción distintos). Esto es apropiado cuando el grafo es una estructura de datos explícita, como el mapa de Rumania. En muchos problemas de IA, el grafo se representa sólo *implícitamente* por el estado inicial, acciones y modelo de transición. Para un espacio de estados implícito, la complejidad se mide en términos de d, **profundidad** o número de acciones en una solución óptima; m, el número máximo de acciones en cualquier camino; y b, el **factor de ramificación** o número de sucesores de un nodo que necesitan ser considerados.

Profundidad

Factor de ramificación

3.4 Estrategias de búsqueda no informadas

Un algoritmo de búsqueda no informada no recibe ninguna pista sobre qué tan cerca está un estado de la(s) meta(s). Por ejemplo, consideremos nuestro agente en Arad con el objetivo de llegar a Bucarest. Un agente no informado sin conocimiento de la geografía rumana no tiene idea de si ir a Zerind o Sibiu es un mejor primer paso. En contraste, un agente informado (Sección 3.5) que conoce la ubicación de cada ciudad sabe que Sibiu está mucho más cerca de Bucarest y, por ello, es más probable que esté en el camino más corto.

3.4.1 Búsqueda de amplitud

Cuando todas las acciones tienen el mismo costo, una estrategia adecuada es la **búsqueda de amplitud**, en la cual el nodo raíz se expande primero, luego se expanden todos los sucesores del nodo raíz, luego *sus* sucesores, etc. Esta es una estrategia de búsqueda sistemática que, por lo tanto, es completa aun en espacios de estados infinitos. Podríamos implementar la búsqueda de amplitud como una llamada a BEST-FIRST-SEARCH donde la función de evaluación $f(n)$ es la profundidad del nodo, el número de acciones necesarias para alcanzar el nodo.

Búsqueda de amplitud

Podemos obtener eficiencia adicional con un par de trucos. Una cola de tipo primero en entrar-primero en salir será más rápida que una cola de prioridad, y nos dará el orden correcto de los nodos: nodos nuevos (siempre más profundos que sus padres) van al final de la cola, y los nodos antiguos, menos profundos que los nodos nuevos, se expanden primero. Además, *alcanzado* puede ser un conjunto de estados en vez de un mapeo de estados a nodos, porque una vez alcanzado un estado, nunca encontraremos un camino mejor hacia el estado. Eso significa que podemos realizar una **prueba temprana de objetivo**, verificar si un nodo es una solución tan pronto se *genera*, en vez de la **prueba tardía de objetivo** que usa la búsqueda de best-first, esperando a que un nodo se saque de la cola. La Figura 3.8 muestra el progreso de una búsqueda de amplitud en un árbol binario, la Figura 3.9 muestra el algoritmo con las mejoras de eficiencia de objetivo temprano.

Prueba temprana de objetivo

Prueba tardía de objetivo

La búsqueda de amplitud siempre encuentra una solución con un número mínimo de acciones, porque cuando genera nodos en la profundidad d, ya ha generado todos los nodos en la profundidad $d - 1$, por lo que, si uno de ellos fuera una solución, se habría encontrado. Eso significa que

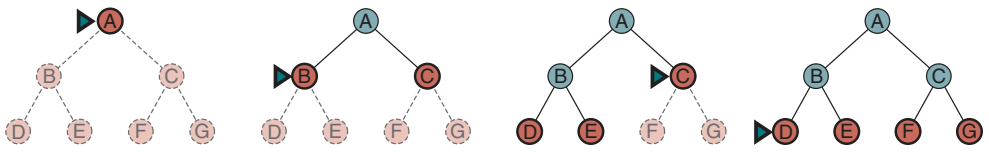


Figura 3.8 Búsqueda de amplitud en un árbol binario simple. En cada etapa, el nodo que se expandirá a continuación está indicado por el marcador triangular.

```
function BREADTH-FIRST-SEARCH(problem) returns un nodo solución o failure
    node ←NODE(problem.INITIAL)
    if problem.IS-GOAL(node.STATE) then return node
    frontier ← una cola FIFO, con node como un elemento
    reached ←{problem.INITIAL}
    while not IS-EMPTY(frontier) do
        node ←POP(frontier)
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if problem.IS-GOAL(s) then return child
            if s no está en reached then
                agregar s en reached
                agregar child en frontier
    return failure

function UNIFORM-COST-SEARCH( problem) returns un nodo solución o failure
    return BEST-FIRST-SEARCH(problem, PATH-COST)
```

Figura 3.9 Algoritmos de búsqueda de amplitud y de costo uniforme.

es óptimo en costos para problemas donde las acciones tienen el mismo costo, pero no para problemas sin esa propiedad. Es completa en ambos casos. En términos de tiempo y espacio, imaginemos buscar en un árbol uniforme donde cada estado tiene b sucesores. La raíz del árbol de búsqueda genera b nodos, cada uno de los cuales genera b nodos más, para un total de b^2 en el segundo nivel. Cada uno genera b nodos más, produciendo b^3 nodos en el tercer nivel, etc. Ahora suponga que la solución está en una profundidad d. Así el número total de nodos generados es

$$1+b+b^2+b^3+\cdots+b^d = O(b^d)$$

Todos los nodos siguen en la memoria, por ello la complejidad temporal y espacial son $O(b^d)$. Esos límites exponenciales son de miedo. Ejemplo típico del mundo real: piense un problema con un factor de ramificación b=10, velocidad de procesamiento de 1 millón de nodos/s y requisitos de memoria de 1 Kbyte/nodo. Una búsqueda a una profundidad d=10 tomaría menos de 3 horas, pero requeriría 10 terabytes de memoria. *Los requisitos de memoria son problema mayor para la búsqueda de amplitud que el tiempo de ejecución.* El tiempo es factor importante. A profundidad d=14, aun con memoria infinita, la búsqueda tomaría 3.5 años. *Los problemas de búsqueda de complejidad exponencial no se resuelven por búsqueda no informada excepto para las instancias más pequeñas.*

3.4.2 Algoritmo de Dijkstra o búsqueda de costos uniformes

Si las acciones tienen costos diferentes, una opción es usar la búsqueda de best-first donde la función de evaluación es el costo del camino de la raíz al nodo actual. Esto se llama algoritmo de Dijkstra por la comunidad de ciencias de la computación, y **búsqueda de costo uniforme** por la comunidad de IA. La idea es que mientras que la búsqueda de amplitud se extiende en ondas de profundidad uniforme, primero profundidad 1, luego profundidad 2, etc., la búsqueda de costo uniforme se extiende en ondas de costo de camino uniforme. El algoritmo se ejecuta llamada a BEST-FIRST-SEARCH con PATH-COST como la función de evaluación, como en la Figura 3.9.

Búsqueda de costos uniformes

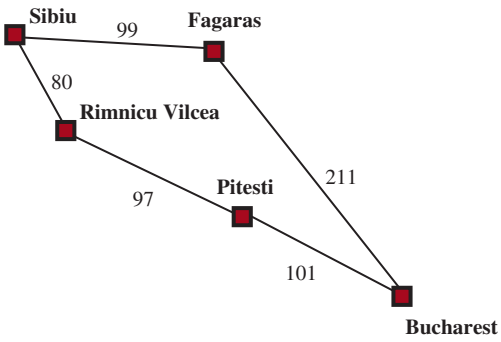


Figura 3.10 Parte del espacio estatal de Rumania, seleccionado para ilustrar la búsqueda de costos uniformes.

Piense en la Figura 3.10, donde el problema es llegar de Sibiu a Bucarest. Los sucesores de Sibiu son Rimnicu Vilcea y Fagaras, con costos de 80 y 99, respectivamente. El nodo de menor costo, Rimnicu Vilcea, se expande y añade Pitesti con costo de 80 + 97 = 177. El nodo de menor costo ahora es Fagaras, que se expande y añade Bucarest con costo de 99 + 211 = 310. Bucarest es el objetivo, pero el algoritmo prueba los objetivos sólo si expande un nodo, no cuando genera uno, por ello aún no ha detectado que esta es una ruta hacia el objetivo.

El algoritmo sigue, elige Pitesti para la expansión siguiente y agrega una segunda ruta a Bucarest con costo de 80 + 97 + 101 = 278. Tiene un costo más bajo, por ello reemplaza la ruta previa en *reached* y se agrega a *frontier*. Resulta que este nodo ahora tiene el costo más bajo, así que se considera a continuación, se encuentra que es un objetivo y se devuelve. Vea que, si hubiéramos evaluado la función objetivo al generar un nodo en lugar de al expandir el nodo de menor costo, habríamos devuelto una ruta de mayor costo (la que pasa por Fagaras).

La complejidad de la búsqueda de costo uniforme es en términos de C^*, el costo de la solución óptima,[8] y ϵ, un límite inferior en el costo de cada acción, con $\epsilon > 0$. Así, la complejidad de tiempo y espacio en el peor caso del algoritmo es $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, mucho mayor que b^d. Esto porque la búsqueda de costo uniforme puede explorar árboles grandes de acciones con costos bajos antes de explorar caminos que involucran una acción de alto costo y quizás útil. Si todos los costos de las acciones son iguales, $b^{1+\lfloor C^*/\epsilon \rfloor}$ es sólo b^{d+1}, y la búsqueda de costo uniforme es similar a la búsqueda de amplitud.

La búsqueda de costo uniforme es completa y óptima en cuanto al costo, pues la primera solución que encuentra tendrá un costo al menos tan bajo como el de cualquier otro nodo en la frontera. La búsqueda de costo uniforme supone todos los caminos con sistematicidad en orden de costo creciente, sin quedarse atrapada en un bucle infinito (si supone que los costos de las acciones son $> \epsilon > 0$).

3.4.3 La búsqueda de profundidad y el problema de la memoria

Búsqueda de profundidad

La **búsqueda de profundidad** siempre expande primero el nodo *más profundo* en la frontera. Podría ejecutarse como una llamada a BEST-FIRST-SEARCH donde la función de evaluación f es el negativo de la profundidad. Pero, en general, se implementa no como una búsqueda de grafos sino como una búsqueda de tipo árbol que no mantiene una tabla de estados alcanzados. Este progreso se ilustra en la Figura 3.11; la búsqueda avanza de inmediato al nivel más profundo del

[8] Aquí, y a lo largo del libro, la "estrella" en C^* significa un valor óptimo para C.

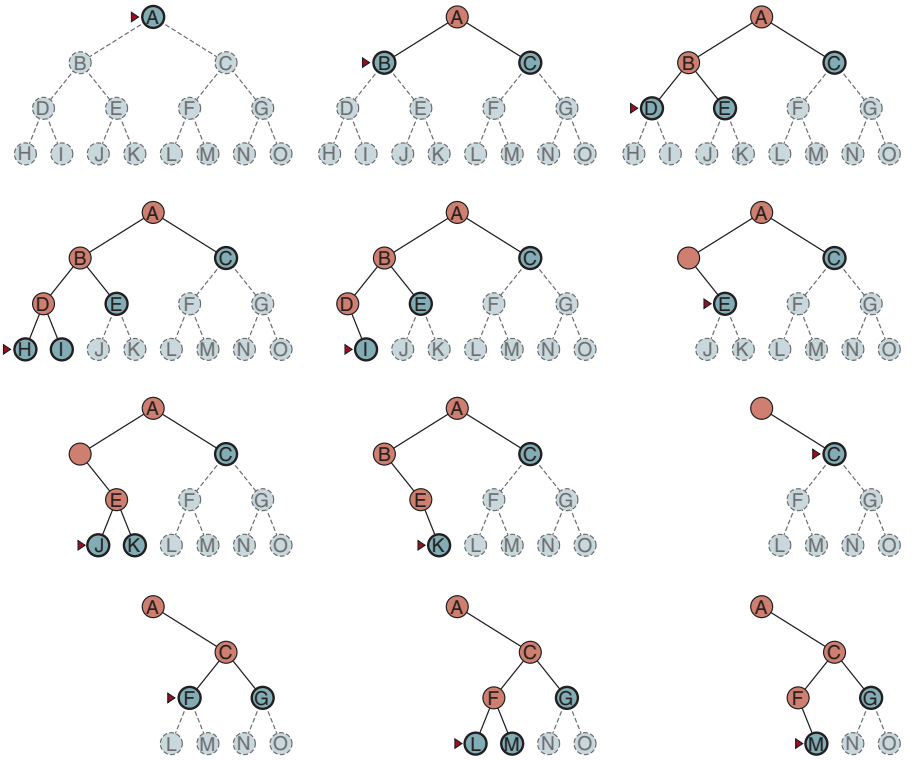


Figura 3.11 Una docena de pasos (de izquierda a derecha, de arriba a abajo) en el progreso de una búsqueda de profundidad en un árbol binario del estado inicial A al objetivo M. La frontera está en verde, con un triángulo que marca el nodo que luego se expandirá. Los nodos antes expandidos son de color lavanda y los nodos futuros potenciales tienen líneas discontinuas. Los nodos expandidos sin descendientes en la frontera (líneas muy tenues) pueden descartarse.

árbol de búsqueda, donde los nodos no tienen sucesores. La búsqueda luego "retrocede" al siguiente nodo más profundo que aún tiene sucesores sin expandir. La búsqueda de profundidad no es óptima en costos; devuelve la primera solución que encuentra, incluso si no es la más barata.

Para espacios de estados finitos que son árboles, es eficiente y completo; para espacios de estados acíclicos, puede terminar expandiendo el mismo estado muchas veces por diferentes caminos, pero (eventualmente) explorará con sistematicidad todo el espacio.

En espacios de estados cíclicos puede atascarse en un bucle infinito; por ello, unas ejecuciones de la búsqueda de profundidad verifican cada nuevo nodo al buscar ciclos. Al final, en espacios de estados infinitos, esta búsqueda no es sistemática: puede atascarse bajando por un camino infinito, aun si no hay ciclos. Por lo tanto, la búsqueda de profundidad es incompleta.

Con malas noticias, ¿por qué usar la búsqueda de profundidad en vez de la de amplitud o la best-first? La respuesta es que para problemas donde una búsqueda de tipo árbol es factible, la de profundidad tiene necesidades de memoria menores. No mantenemos una tabla de estados *reached*, y la frontera es muy pequeña: piense en la frontera en la búsqueda de amplitud como la superficie de una esfera en expansión, mientras que la frontera en la búsqueda de profundidad es sólo un radio de la esfera.

Para un espacio de estados finito con forma de árbol como en la Figura 3.11, una búsqueda en árbol de profundidad toma tiempo proporcional al número de estados, y tiene una complejidad de memoria de sólo $O(bm)$, donde b es el factor de ramificación y m la profundidad máxima del árbol. Algunos problemas que requerirían exabytes de memoria con búsqueda de amplitud pueden manejarse sólo con kilobytes usando búsqueda de profundidad. Por su uso parsimonioso de la memoria, la búsqueda tipo árbol en profundidad se adoptó como herramienta básica en muchas áreas de IA, como satisfacciones de restricciones (Capítulo 6) y de proposiciones (Capítulo 7), y programación lógica (Capítulo 9).

Búsqueda de retroceso

Una variante de la búsqueda de profundidad, la **búsqueda con retroceso**, usa aún menos memoria. (Vea detalles en el Capítulo 6). En el retroceso, sólo se genera un sucesor a la vez, no todos los sucesores; cada nodo parcialmente expandido recuerda qué sucesor generar después. Además, los sucesores se generan *modificando* directamente la descripción del estado actual en vez de asignar memoria para un estado nuevo. Esto reduce los requisitos de memoria a sólo una descripción de estado y un camino de acciones de $O(m)$; un ahorro significativo sobre $O(bm)$ estados para la búsqueda de profundidad. Con el retroceso, tenemos la opción de mantener una estructura de datos para el conjunto de estados en el camino actual, esto permite verificar si hay un ciclo en tiempo $O(1)$ en vez de $O(m)$. Para que funcione el retroceso, debemos poder *deshacer* cada acción al retroceder. Esto es fundamental para el éxito de muchos problemas con descripciones de estado grandes, como el ensamble robótico.

3.4.4 Búsqueda de profundidad limitada e iterativa

Búsqueda limitada por profundidad

Para evitar que la búsqueda de profundidad se desvíe a un camino infinito, usamos la **búsqueda limitada por profundidad**, versión de la búsqueda de profundidad en la que proveemos un límite de profundidad, ℓ, y tratamos los nodos en profundidad ℓ como si no tuvieran sucesores (vea la Figura 3.12). La complejidad temporal es $O(b^\ell)$, la espacial es $O(b\ell)$. Por desgracia, si elegimos mal para ℓ, el algoritmo no alcanzará la solución, lo que lo hace incompleto de nuevo.

Dado que la búsqueda de profundidad es de tipo árbol, no podemos evitar que pierda tiempo en caminos redundantes, pero podemos eliminar ciclos con un poco de tiempo de computación. Si miramos sólo algunos enlaces en la cadena de padres atraparemos la mayoría de los ciclos; los ciclos más largos son manejados por el límite de profundidad.

Diámetro

A veces, se elige un buen límite de profundidad si se conoce el problema. Por ejemplo, en el mapa de Rumania hay 20 ciudades; así, $\ell = 19$ es un límite válido. Pero si estudiáramos el mapa con cuidado, descubriríamos que toda ciudad puede ser alcanzada desde cualquier otra en máximo 9 acciones. Este número, conocido como **diámetro** del grafo del espacio de estados, da un mejor límite de profundidad, que lleva a una búsqueda limitada por profundidad más eficiente. Para la mayoría de los problemas, no conoceremos un buen límite de profundidad hasta resolverlo.

Búsqueda iterativa en profundidad

La **búsqueda iterativa en profundidad** resuelve el problema de elegir un buen valor para ℓ al probar todos los valores: 0, luego 1, luego 2, etc., hasta encontrar solución o cuando la búsqueda limitada por profundidad devuelva el valor de *failure* en vez del valor de *cutoff*. (Vea el algoritmo en la Figura 3.12). La búsqueda iterativa en profundidad combina muchos beneficios de las búsquedas de profundidad y de amplitud. Al igual que la búsqueda de profundidad, sus requisitos de memoria son modestos: $O(bd)$ si hay una solución, o $O(bm)$ en espacios de estados finitos sin solución. Igual que la búsqueda de amplitud, la iterativa en profundidad es óptima para problemas donde todas las acciones tienen el mismo costo, y es completa en espacios de estados finitos acíclicos, o en cualquier espacio de estados finitos si verificamos los nodos para ciclos en todo el camino.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns un nodo solución o failure
  for depth = 0 to ∞ do
  result ←DEPTH-LIMITED-SEARCH(problem, depth)
  if result /= cutoff then return result

function DEPTH-LIMITED-SEARCH(problem, ℓ) returns un nodo o failure o cutoff
  frontier ←una cola LIFO (stack) con NODE(problem.INITIAL) como un elemento
  result ← failure
  while not IS-EMPTY(frontier) do
    node ←POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) > ℓ then
      result ← cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        agregar child en frontier
  return result
```

Figura 3.12 Profundización iterativa y búsqueda tipo árbol de profundidad limitada. La profundización iterativa aplica repetidamente búsqueda de profundidad limitada con límites crecientes. Devuelve uno de tres tipos diferentes de valores: un nodo de solución; o *failure*, si se han agotado todos los nodos y se ha demostrado que no hay solución a ninguna profundidad; o *cutoff*, para significar que habría solución a una profundidad mayor que ℓ. Es un algoritmo de búsqueda en forma de árbol que no da seguimiento de los estados *reached* y, por lo tanto, usa menos memoria que la búsqueda del *best-first*, pero corre el riesgo de visitar el mismo estado varias veces en diferentes caminos. Además, si la verificación IS-CYCLE no verifica *todos los ciclos*, el algoritmo queda atrapado en un bucle.

La complejidad temporal es $0(b^d)$ si hay una solución, o $0(bm)$ si no la hay. Cada iteración de la búsqueda iterativa en profundidad genera un nuevo nivel, así como lo hace la búsqueda de amplitud, pero esta última almacena todos los nodos en memoria, mientras que la búsqueda iterativa en profundidad lo hace repitiendo los niveles anteriores, así ahorra memoria a cambio de tiempo. La Figura 3.13 muestra cuatro iteraciones de la búsqueda iterativa en profundidad en un árbol de búsqueda binario, donde está la solución en la cuarta iteración.

La búsqueda iterativa en profundidad parece derrochadora porque los estados cerca de la parte superior del árbol de búsqueda se regeneran varias veces. Pero para muchos espacios de estados, la mayoría de los nodos están en el nivel inferior, por ello no importa que los niveles superiores se repitan. En una búsqueda iterativa en profundidad, los nodos en el nivel inferior (profundidad d) se generan una vez, los del nivel inferior siguiente dos veces, y así hasta los hijos de la raíz, que se generan d veces. Entonces, el número total de nodos generados en el peor de los casos es

$$N(\text{IDS}) = (d)b^1 + (d-1)b^2 + (d-2)b^3 \cdot \cdot \cdot + b^d$$

lo que da una complejidad temporal de $0(b^d)$, que es asintóticamente lo mismo que la búsqueda de amplitud. Por ejemplo, si $b = 10$ y $d = 5$, los números son

$$N(\text{IDS}) = 50 + 400 + 3\,000 + 20\,000 + 100\,000 = 123\,450$$
$$N(\text{BFS}) = 10 + 100 + 1\,000 + 10\,000 + 100\,000 = 111\,110.$$

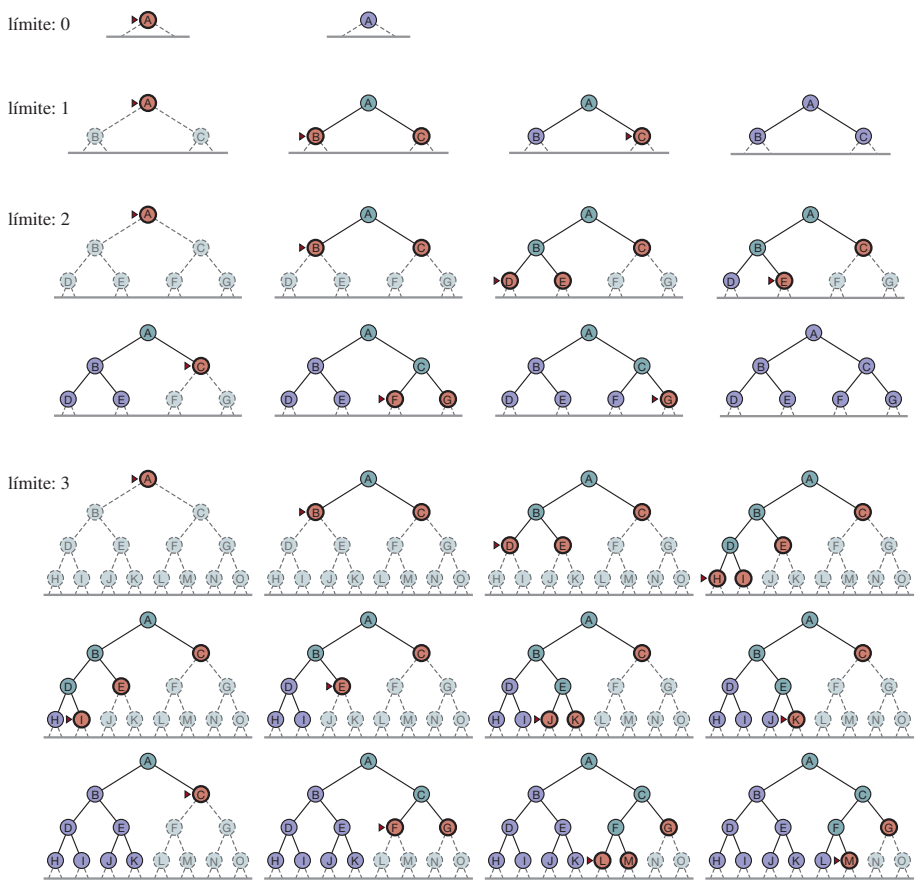


Figura 3.13 Cuatro iteraciones de búsqueda iterativa en profundidad para el objetivo M en un árbol binario, con el límite de profundidad que va de 0 a 3. Vea que los nodos interiores forman una única ruta. El triángulo marca el nodo que luego se expandirá; en el borde hay nodos verdes con contornos oscuros. Quizá los nodos muy débiles no sean parte de una solución con este límite de profundidad.

Si realmente le preocupa la repetición, puede usar un enfoque híbrido que ejecute la búsqueda de amplitud hasta que casi toda la memoria disponible esté consumida, y luego ejecute la profundización iterativa desde todos los nodos en la frontera. En general, esta profundización *es el método de búsqueda no informada preferido si el espacio de estados de búsqueda es más grande que la memoria y la profundidad de la solución no es conocida.*

3.4.5 Búsqueda bidireccional

Búsqueda bidireccional

Los algoritmos revisados hasta ahora comienzan en un estado inicial y pueden alcanzar cualquiera de los posibles estados objetivo. Un enfoque alternativo es la **búsqueda bidireccional** que de modo simultáneo busca hacia adelante desde el estado inicial y hacia atrás desde los estados objetivo, con la esperanza de que ambas búsquedas se encuentren. La motivación es que $b^{d/2} + b^{d/2}$ es menor que b^d (como, 50 000 veces menos si $b = d = 10$).

```
function BIBF-SEARCH(problem_F, f_F, problem_B, f_B) returns un nodo solución, or failure
    node_F ← NODE(problem_F.INITIAL)// Nodo para un estado inicial
    node_B ← NODE(problem_B.INITIAL) // Nodo para un estado objetivo
    frontier_F ← una cola de prioridad ordenada por f_F , con node_F como elemento
    frontier_B ← una cola de prioridad ordenada por f_B, con node_B como elemento
    reached_F ← una tabla de búsqueda, con ID node_F.STATE y un valor node_F
    reached_B ← una tabla de búsqueda, con ID node_B.STATE y un valor node_B
    solution ← failure
    while not TERMINATED(solution, frontier_F, frontier_B) do
        if f_F (TOP(frontier_F)) < f_B(TOP(frontier_B)) then
            solution ←PROCEED(F, problem_F, frontier_F, reached_F, reached_B, solution)
        else solution ←PROCEED(B, problem_B, frontier_B, reached_B, reached_F, solution)
    return solution

function PROCEED(dir, problem, frontier, reached, reached_2, solution) returns una solución
            / / Expandir nodo en la frontera; comparar con la otra frontera en reached_2.
            / / La variable "dir" es la dirección: F para adelante o B para atrás.
    node ←POP(frontier)
    for each child in EXPAND(problem, node) do
        s ← child.STATE
        if s no está en reached or PATH-COST(child) < PATH-COST(reached[s]) then
            reached[s] ← child
            agregar child a frontier
            if s está en reached_2 then
                solution_2 ←JOIN-NODES(dir, child, reached_2[s]))
                if PATH-COST(solution_2) < PATH-COST(solution) then
                    solution ← solution_2
    return solution
```

Figura 3.14 La búsqueda bidireccional de best-first mantiene dos fronteras y dos tablas de estados alcanzados. Cuando un camino en una frontera alcanza un estado que también se alcanzó en la otra mitad de la búsqueda, los dos caminos se unen (mediante la función JOIN-NODES) para formar una solución. No se garantiza que la primera solución que obtengamos sea la mejor; la función TERMINATED determina cuándo dejar de buscar nuevas soluciones.

Para que esto funcione, necesitamos tanto dar seguimiento de dos fronteras y dos tablas de estados alcanzados, como razonar hacia atrás: si el estado s' es un sucesor de s en la dirección hacia adelante, entonces necesitamos saber que s es un sucesor de s' en la dirección hacia atrás. Tenemos una solución cuando las dos fronteras colisionan.[9]

Hay versiones diferentes de búsqueda bidireccional, así como hay muchos algoritmos de búsqueda unidireccional diferentes. En esta sección, describimos la búsqueda bidireccional de best-first. Aunque hay dos fronteras separadas, el nodo a expandir después siempre es aquel con un valor mínimo de la función de evaluación, en una de las dos fronteras. Si la función de evaluación

[9] En la ejecución, la estructura de datos *reached* admite una consulta y pregunta si un estado determinado es miembro, y la estructura de datos de frontera (una cola de prioridad) no lo es; así, verificamos si hay colisión usando *reached*; en teoría preguntamos si ambas fronteras se han encontrado. La ejecución se expande para manejar múltiples estados objetivo cargando el nodo para cada uno de ellos en la tabla de frontera hacia atrás y alcanzada hacia atrás.

es el costo del camino, obtenemos una búsqueda bidireccional de costo uniforme, y si el costo del camino óptimo es C^*, entonces ningún nodo con costo $> \frac{C^*}{2}$ será expandido. Esto puede resultar en una considerable aceleración.

Vea en la Figura 3.14 el algoritmo general de búsqueda bidireccional de best-first. Pasamos dos versiones del problema y la función de evaluación, una en dirección hacia adelante (subíndice F), otra en dirección hacia atrás (subíndice B). Si la función de evaluación es el costo del camino, sabemos que la primera solución hallada será una solución óptima, pero con diferentes funciones de evaluación no necesariamente ciertas. Así, damos seguimiento a la mejor solución hallada hasta el momento, aunque podríamos tener que actualizarla varias veces antes de que la prueba TERMINATED demuestre que no hay posibilidad de una mejor solución restante.

3.4.6 Comparación de algoritmos de búsqueda no informados

La Figura 3.15 compara algoritmos de búsqueda no informada con base en los criterios de evaluación de la Sección 3.3.4, que es para versiones de búsqueda en forma de árbol que no verifican estados repetidos. Para búsquedas en grafos que sí verifican, las diferencias son que la búsqueda de profundidad es completa para espacios de estados finitos, pero las complejidades de espacio y tiempo están limitadas por el tamaño del espacio de estados (el número de vértices y aristas, $|V| + |E|$).

Criterio	Amplitud primero	Costo uniforme	Búsqueda de profundidad	Búsqueda de profundidad limitada	Búsqueda iterativa profunda	Búsqueda bidireccional (si es aplicable)
¿Completo?	Sí[1]	Sí[1,2]	No	No	Sí[1]	Sí[1,4]
¿Costo óptimo?	Sí[3]	Sí	No	No	Sí[3]	Sí[3,4]
Tiempo	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Espacio	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figura 3.15 Evaluación de algoritmos de búsqueda. *b,* factor de ramificación; *m,* profundidad máxima del árbol de búsqueda; *d,* profundidad de la solución más superficial, o es *m*, si no hay solución; ℓ, límite de profundidad. Las advertencias de superíndice son las siguientes: [1] completo si *b* es finito y el espacio de estados tiene una solución o es finito. [2], completo si todos los costos de acción son $\geq \epsilon > 0$; [3], costo óptimo si los costos de acción son todos idénticos; [4], si ambas direcciones son primero en amplitud o en costo uniforme.

3.5 Estrategias de búsqueda informada (heurística)

Búsqueda informada

Función heurística

Aquí mostramos cómo una **estrategia de búsqueda informada**, que usa pistas específicas del dominio sobre la ubicación de los objetivos, encuentra soluciones de modo más eficiente que una estrategia no informada. Las pistas son en forma de una **función heurística**, denotada como $h(n)$:[10]

$h(n)$ = costo estimado del camino más barato del estado en el nodo n a un estado objetivo.

Por ejemplo, en problemas de búsqueda de rutas, podemos estimar la distancia desde el estado actual hasta un objetivo calculando la distancia en línea recta en el mapa entre ambos puntos. En la Sección 3.6 estudiamos en detalle las heurísticas y su origen.

[10] Parece extraño que la función heurística opere en un nodo, cuando todo lo que necesita es el estado del nodo. Se suele usar $h(n)$ en lugar de $h(s)$ para ser coherente con la función de evaluación $f(n)$ y el costo de la ruta $g(n)$.

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Figura 3.16 Valores de h_{SLD}: distancias en línea recta a Bucarest.

3.5.1 Búsqueda voraz del best-first

Búsqueda voraz del best-first

La **búsqueda voraz del best-first** es una forma de búsqueda de candidato best-first que expande primero el nodo con el valor $h(n)$ más bajo, el que parece estar más cerca del objetivo, bajo la premisa de que quizá llevará rápido a una solución. Así, la función de evaluación $f(n) = h(n)$.

Distancia en línea recta

Cómo funciona esto para problemas de búsqueda de rutas en Rumania; usamos la heurística de **distancia en línea recta**, que llamada h_{SLD}. Si el objetivo es Bucarest, necesitamos conocer las distancias en línea recta a Bucarest, mostrada en la Figura 3.16. Así, $h_{SLD}(Arad) = 366$. Vea que los valores de h_{SLD} no se calculan a partir de la descripción del problema en sí (es decir, las funciones de ACCIONES y RESULTADO). Además, se requiere conocimiento del mundo para saber que h_{SLD} se correlaciona con las distancias reales de la carretera y, por lo tanto, es una heurística útil.

La Figura 3.17 muestra el progreso de una búsqueda voraz del best-first usando h_{SLD} para encontrar un camino de Arad a Bucarest. El primer nodo que se expandirá de Arad será Sibiu porque la heurística indica que está más cerca de Bucarest que Zerind o Timisoara. El siguiente nodo en expandirse será Fagaras porque ahora está más cerca, según la heurística. Fagaras, a su vez, genera Bucarest, el objetivo. Para este problema particular, la búsqueda voraz del best-first usando h_{SLD} encuentra una solución sin expandir nunca un nodo que no esté en el camino de la solución. La solución encontrada no tiene costo óptimo: el camino de Sibiu y Fagaras a Bucarest es 32 millas más largo que por Rimnicu Vilcea y Pitesti. Por ello es que el algoritmo se llama "voraz": en cada iteración intenta acercarse lo más posible a un objetivo, pero la voracidad puede conducir a resultados peores que ser cuidadoso.

La búsqueda voraz del best-first es completa en espacios de estados finitos, no en infinitos. La complejidad temporal y espacial, en el peor caso, es $O(|V|)$; pero, con una buena función heurística, puede reducirse sustancialmente, en ciertos problemas alcanzando $O(bm)$.

3.5.2 Búsqueda A*

Búsqueda A*

El algoritmo de búsqueda informada más común es la **búsqueda A*** (pronunciada "A estrella"), una búsqueda de best-first candidato que usa la función de evaluación

$$f(n) = g(n) + h(n)$$

donde $g(n)$ es el costo del camino desde el estado inicial hasta el nodo n, y $h(n)$ es el costo *estimado* del camino más corto desde n hasta un estado objetivo, por lo que tenemos

$f(n)$ = costo estimado del best-first que continúa desde n hasta un objetivo.

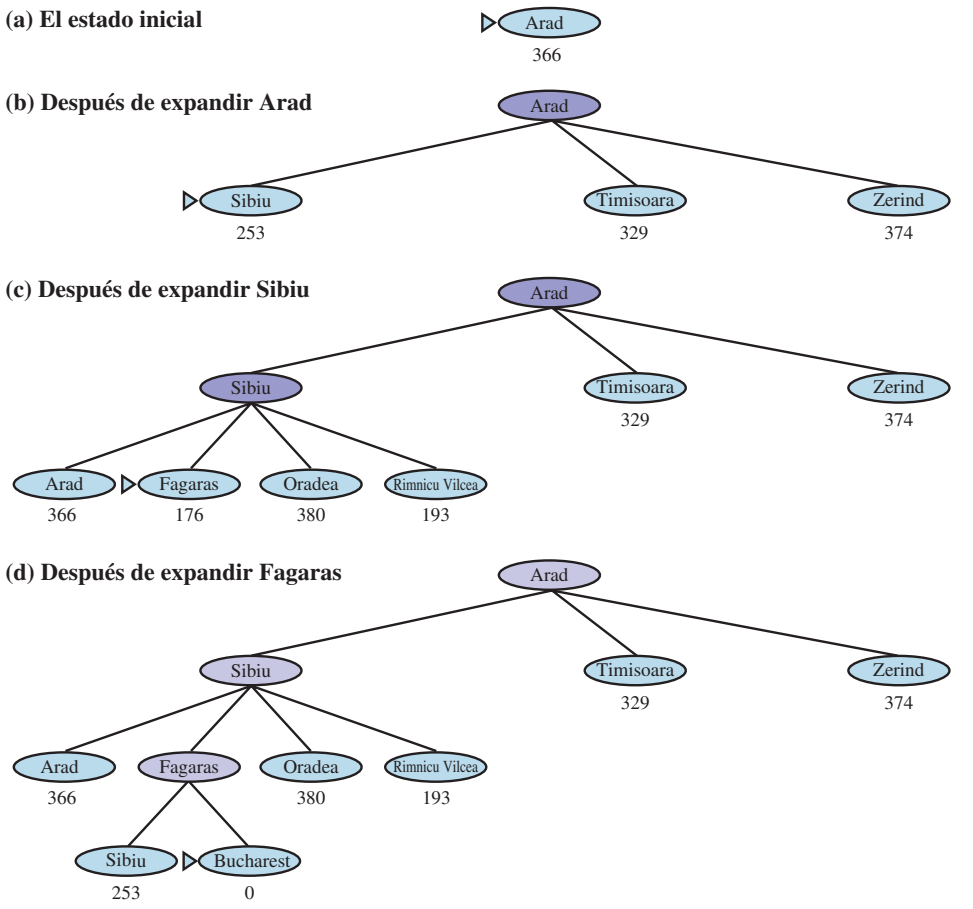


Figura 3.17 Etapas de una búsqueda voraz del best-first en forma de árbol para Bucarest con la heurística de distancia en línea recta h_{SLD}. Los nodos están etiquetados con sus valores h.

En la Figura 3.18, mostramos el progreso de una búsqueda A* con el objetivo de llegar a Bucarest. Los valores de g se calculan a partir de los costos de acción en la Figura 3.1, los valores de h_{SLD} se dan en la Figura 3.16. Vea que Bucarest aparece por primera vez en la frontera en el paso (e), pero no se selecciona para expansión (y, por ello, no se detecta como solución) porque $f = 450$ no es el nodo de menor costo en la frontera; ese sería Pitesti, $f = 417$; es decir, que *podría* haber una solución por Pitesti si su costo fuera tan bajo como 417, entonces el algoritmo no aceptará una solución que cueste 450. En el paso (f), un camino diferente a Bucarest, ahora es el nodo de menor costo, $f = 418$, así se selecciona y detecta como solución óptima.

Heurística admisible

La búsqueda A* es completa.[11] Si A* es óptimo en costos depende de ciertas propiedades de la heurística. Una propiedad clave es la **admisibilidad**: una **heurística admisible** es la que nunca sobreestima el costo de alcanzar un objetivo (es, por lo tanto, *optimista*). Con una heurística admisible,

[11] De nuevo, suponga que todos los costos de acción son $> \epsilon > 0$, y que el espacio de estados tiene una solución o es finito.

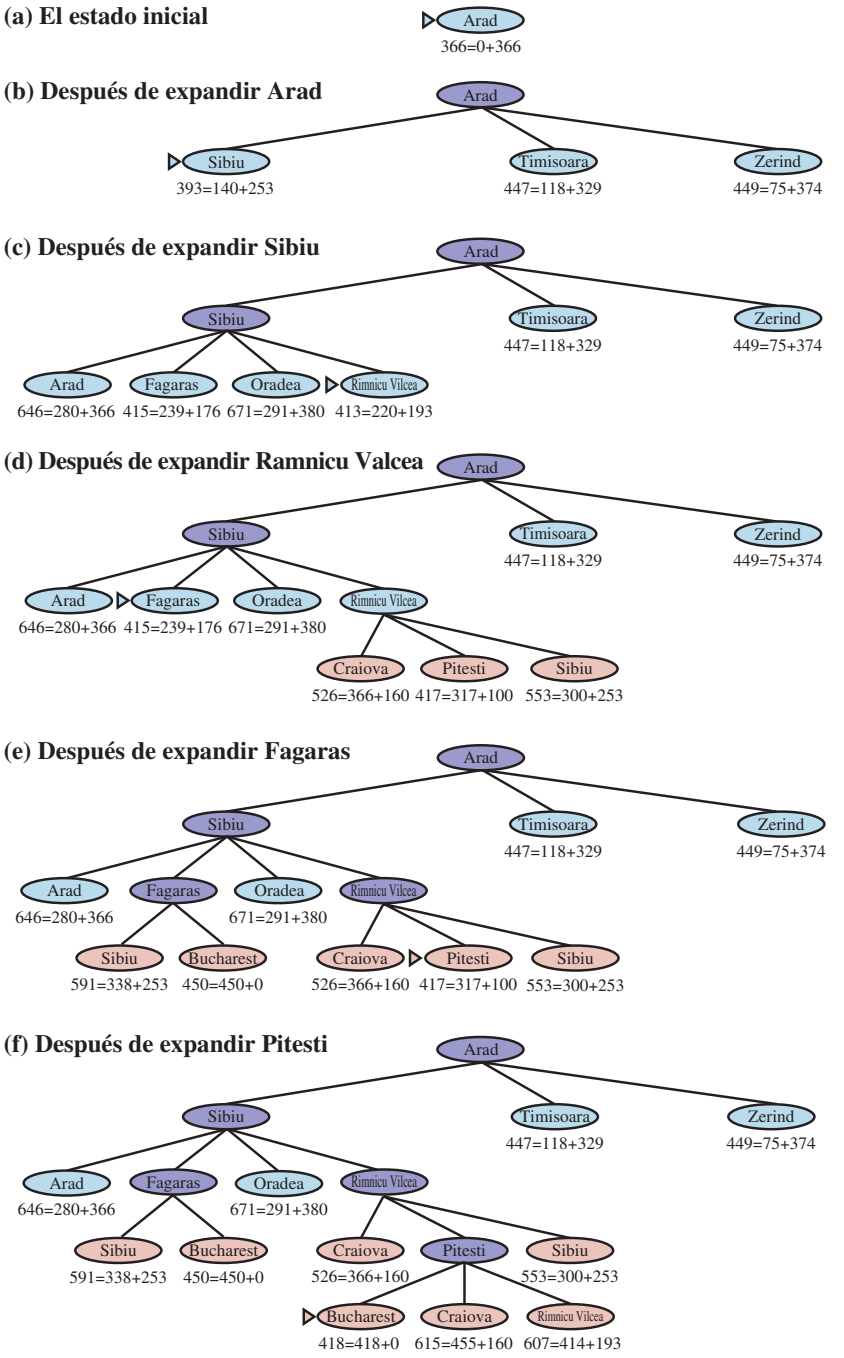


Figura 3.18 Prácticas en una búsqueda A* para Bucarest. Los nodos están etiquetados con $f = g + h$. Los valores h son las distancias en línea recta a Bucarest tomadas de la Figura 3.16.

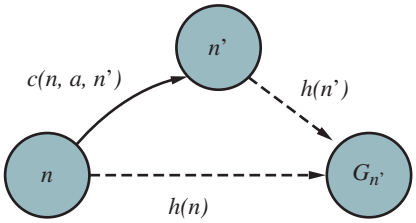


Figura 3.19 Desigualdad triangular: si la heurística h es **consistente**, entonces el número único $h(n)$ será menor que la suma del costo $c(n,a,a')$ de la acción de n a n' más la estimación heurística $h(n')$.

A* es óptimo en costos, y se demuestra con una prueba por contradicción. Suponga que el camino óptimo tiene un costo C^*, pero el algoritmo devuelve un camino con costo $C > C^*$. Luego, debe haber un nodo n que esté en el camino óptimo y no haya sido expandido (porque si todos los nodos en el camino óptimo hubieran sido expandidos, habríamos devuelto esa solución óptima). Así, usando la notación $g^*(n)$ para significar el costo del camino óptimo desde el inicio hasta n, y $h^*(n)$ para significar el costo del camino óptimo desde n hasta el objetivo más cercano, tenemos:

$$\begin{aligned} f(n) &> C^* \text{ (de lo contrario, } n \text{ se habría expandido)} \\ f(n) &= g(n) + h(n) \text{ (por definición)} \\ f(n) &= g^*(n) + h(n) \text{ (porque } n \text{ está en un camino óptimo)} \\ f(n) &\leq g^*(n) + h^*(n) \text{ (debido a la admisibilidad, } h(n) \leq h^*(n)) \\ f(n) &\leq C^* \text{ (por definición, } C^* = g^*(n) + h^*(n)) \end{aligned}$$

La primera y última líneas dan una contradicción, así que la suposición de que el algoritmo podría devolver un camino subóptimo debe ser incorrecta; debe ser que A* devuelve sólo caminos óptimos en costo.

Consistencia

Una propiedad ligeramente más fuerte se llama **consistencia**. Una heurística $h(n)$ es consistente si, para cada nodo n y cada sucesor n' de n generado por una acción a, tenemos:

$$h(n) \leq c(n,a,n') + h(n').$$

Desigualdad del triángulo

Esta es una forma de la **desigualdad del triángulo**, que estipula que un lado de un triángulo no puede ser más largo que la suma de los otros dos lados (Figura 3.19). Un ejemplo de heurística consistente es la distancia en línea recta h_{SLD} que usamos para llegar a Bucarest.

Toda heurística consistente es admisible (no al revés), por ello, con una heurística consistente, A* es óptimo en costos. Además, con una heurística consistente, la primera vez que alcanzamos un estado será en un camino óptimo, así nunca tenemos que volver a agregar un estado a la frontera ni cambiar una entrada en *reached*. Pero con una heurística inconsistente, podemos terminar con múltiples caminos que alcanzan el mismo estado, y si cada nuevo camino tiene un costo más bajo que el anterior, entonces terminaremos con múltiples nodos para ese estado en la frontera, lo que costará tiempo y espacio. Debido a eso, algunas implementaciones de A* se encargan de ingresar un estado en la frontera sólo una vez, y si se encuentra un camino mejor hacia el estado, se actualizan todos los sucesores del estado (esto requiere que los nodos tengan punteros de hijos además de punteros de padres). Estas complicaciones han llevado a muchos implementadores a evitar heurísticas inconsistentes, pero Felner *et al.* (2011) argumentan que los peores efectos rara vez ocurren en la práctica, y uno no debe tener miedo de las heurísticas inconsistentes.

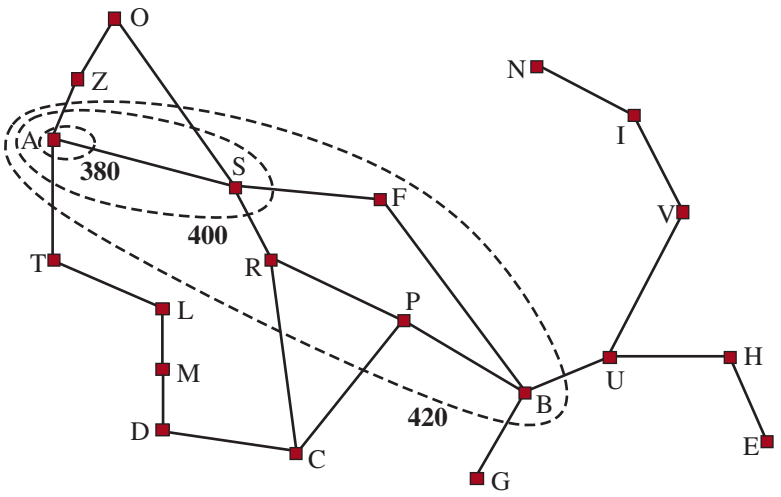


Figura 3.20 Mapa de Rumania que muestra contornos en $f = 380$, $f = 400$ y $f = 420$, con Arad como estado inicial. Los nodos dentro de un contorno dado tienen costos $f = g + h$ menores o iguales que el valor del contorno.

Con una heurística inadmisible, A* puede o no ser óptimo en costos. Aquí hay dos casos donde es: Primero, si hay al menos un camino óptimo en el que $h(n)$ sea admisible para todos los nodos n en el camino, entonces ese camino se encontrará, sin importar lo que la heurística diga para los estados fuera del camino. Segundo, si la solución óptima tiene un costo C^*, y la segunda mejor tiene un costo C_2, y si $h(n)$ sobreestima unos costos, pero nunca en más de $C_2 - C^*$, entonces A* está garantizado de devolver soluciones óptimas en costos.

3.5.3 Contornos de búsqueda

Una forma útil de visualizar una búsqueda es dibujar **contornos** en el espacio de estados, como los contornos en un mapa topográfico. La Figura 3.20 ejemplifica. Dentro del contorno etiquetado como 400, todos los nodos tienen $f(n) = g(n) + h(n) \leq 400$, etc. Luego, dado que A* expande el nodo de la frontera con el costo f más bajo, vemos que una búsqueda A* se expande desde el nodo inicial, agregando nodos en bandas concéntricas de costos f crecientes.

Contorno

Con la búsqueda de costo uniforme, también tenemos contornos, pero de costo g, no $g + h$. Los contornos con la búsqueda de costo uniforme serán "circulares" alrededor del estado inicial, extendiéndose de manera equitativa en todas direcciones sin preferencia hacia el objetivo. Con la búsqueda A* usando una buena heurística, las bandas $g + h$ se extenderán a un estado objetivo (como en la Figura 3.20) y se centrarán más estrechamente alrededor de un camino óptimo.

Debe quedar claro que, al extender un camino, los costos g son **monótonos**: el costo del camino aumenta a medida que avanza por un camino, porque los costos de acción siempre son positivos.[12] Así, obtiene líneas de contorno concéntricas que no se cruzan entre sí, y si elige dibujar las líneas finas, puede colocar una línea entre cualquier par de nodos en cualquier camino.

Monótono

[12] Técnicamente, decimos "estrictamente monótono" para los costos que siempre aumentan, y "monótono" para los que nunca disminuyen, pero que pueden persistir iguales.

No es obvio si el costo $f = g + h$ aumentará monótonamente. Al extender un camino de n a n', el costo pasa de $g(n) + h(n)$ a $g(n) + c(n,a,n') + h(n')$. Cancelando el término $g(n)$, vemos que el costo del camino aumentará monótonamente si y sólo si $h(n) \leq c(n,a,n') + h(n')$; es decir, si y sólo si la heurística es consistente.[13] Pero observe que un camino podría contribuir con varios nodos seguidos con la misma puntuación de $g(n) + h(n)$; esto sucederá siempre que la disminución en h sea exactamente igual al costo de acción recién tomado (como, en un problema de cuadrícula, cuando n está en la misma fila que el objetivo y da un paso hacia el objetivo, g aumenta en 1 y h disminuye en 1). Si C^* es el costo del camino de solución óptima, podemos decir:

Nodos seguramente expandidos

- A* expande los nodos a los que se puede llegar desde el estado inicial en un camino, donde cada nodo en el camino tiene $f(n) < C^*$. Decimos que estos son **nodos seguramente expandidos.**
- A* podría expandir nodos justo en el "contorno objetivo" (donde $f(n) = C^*$) antes de seleccionar un nodo objetivo.
- A* no expande nodos con $f(n) > C^*$.

Óptimamente eficiente

Decimos que A* con una heurística consistente es **óptimamente eficiente** en el sentido de que todo algoritmo que extienda los caminos de búsqueda desde el estado inicial y use la misma información heurística debe expandir todos los nodos que seguramente serán expandidos por A* (porque cualquiera podría haber sido parte de una solución óptima). Entre los nodos con $f(n) = C^*$, un algoritmo podría tener suerte y elegir el óptimo primero, mientras que otro tiene mala suerte; no consideramos esta diferencia en la definición de eficiencia óptima.

Evitando

A* es eficiente porque **evita** los nodos del árbol de búsqueda innecesarios para encontrar una solución óptima. En la Figura 3.18(b) vemos que Timisoara tiene $f = 447$ y Zerind $f = 449$. Aunque son hijos de la raíz y estarían entre los primeros nodos expandidos por la búsqueda de costo uniforme o la búsqueda de amplitud, nunca son expandidos por la búsqueda A* porque se encuentra primero la solución con $f = 418$. El concepto de poda, eliminando posibilidades de consideración sin tener que examinarlas, es importante para muchas áreas de la IA.

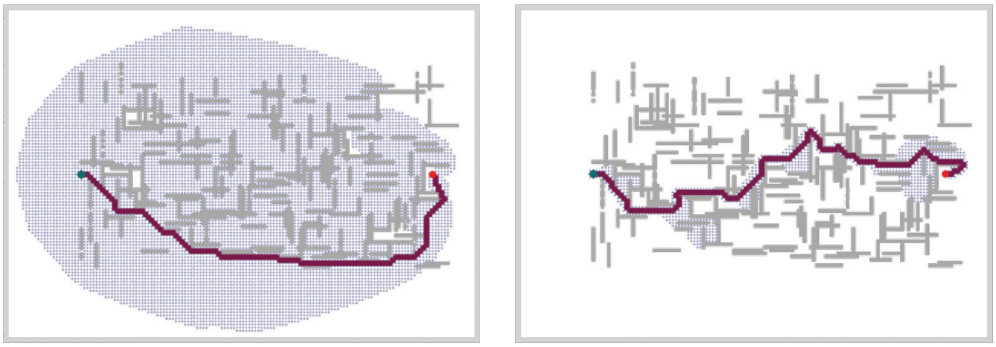
Que la búsqueda A* sea completa, óptima en costos y eficiente entre todos esos algoritmos es muy satisfactorio. Por desgracia, eso no significa que A* sea la respuesta a todas nuestras necesidades de búsqueda. El problema es que, para muchos problemas, el número de nodos expandidos sea exponencial en la longitud de la solución. Así, considere una versión del mundo de la aspiradora con una aspiradora súper potente que limpia cualquier cuadrado a un costo de 1 unidad, sin siquiera visitar el cuadrado; en ese escenario, los cuadrados pueden limpiarse en cualquier orden. Con N cuadrados inicialmente sucios, hay 2^N estados donde algún subconjunto ha sido limpiado; todos esos estados están en un camino de solución óptima y, por lo tanto, satisfacen $f(n) < C^*$, por lo que todos serían visitados por A*.

3.5.4 Búsqueda de satisfacción: Heurísticas inadmisibles y A* ponderado

La búsqueda A* tiene cualidades positivas, pero expande muchos nodos. Podemos explorar menos nodos (en menos tiempo y espacio) si aceptamos soluciones que al ser subóptimas, sean "suficientemente buenas", lo que llamamos soluciones **satisfactorias**. Si permitimos que la búsqueda A* use una **heurística inadmisible**, una que pueda sobreestimar, corremos el riesgo de perder la solución óptima, pero la heurística puede ser en potencial más precisa, reduciendo así el número de nodos expandidos. Por ejemplo, los ingenieros de carreteras conocen el concepto **índice de**

Heurística inadmisible

[13] De hecho, el término "heurística monótona" es sinónimo de "heurística consistente". Ambas ideas se desarrollaron de forma independiente, luego se demostró su equivalencia (Pearl, 1984).



(a) (b)

Figura 3.21 Dos búsquedas en la misma cuadrícula: (a) una búsqueda A* y (b) una búsqueda ponderada A* con peso $W = 2$. Las barras grises son obstáculos, la línea morada es el camino del inicio verde a la meta roja, los puntos pequeños son estados que se alcanzaron en cada búsqueda. En este problema en particular, la ponderación A* explora 7 veces menos estados y encuentra un camino 5% más costoso.

Índice de desvío

desvío, un multiplicador aplicado a la distancia en línea recta para considerar la curvatura típica de las carreteras. Un índice de desvío de 1.3 significa que, si dos ciudades están a 10 millas de distancia en línea recta, una buena estimación del mejor camino entre ellas es de 13 millas. Para la mayoría de las localidades, el índice de desvío va de 1.2 a 1.6.

Búsqueda A* ponderada

Podemos aplicar esta idea a cualquier problema, no sólo a los que involucran carreteras, con un enfoque llamado **búsqueda A* ponderada**, donde ponderamos el valor heurístico más fuertemente, lo que nos da la función de evaluación $f(n) = g(n) + W \times h(n)$, para algún $W > 1$.

La Figura 3.21 muestra un problema de búsqueda en un mundo de cuadrícula. En (a), una búsqueda A* encuentra la solución óptima, pero debe explorar parte del espacio de estados para encontrarla. En (b), una búsqueda A* ponderada encuentra una solución un poco más costosa, pero el tiempo de búsqueda es más rápido. Vemos que la búsqueda ponderada enfoca el contorno de los estados alcanzados a un objetivo. Es decir, se exploran menos estados, pero si el camino óptimo se desvía fuera del contorno de búsqueda ponderada (como en este caso), no se encontrará el camino óptimo. En general, si el costo de la solución óptima es C^*, una búsqueda A* ponderada encontrará una solución que cuesta entre C^* y $W \times C^*$; pero en la práctica, solemos obtener resultados más cercanos a C^* que a $W \times C^*$.

Hemos considerado búsquedas que evalúan estados combinando g y h de varias maneras; la búsqueda A* ponderada se ve como una generalización de las demás:

Búsqueda A*:	$g(n) + h(n)$	$(W = 1)$
Búsqueda de costo uniforme:	$g(n)$	$(W = 0)$
Búsqueda voraz del best-first:	$h(n)$	$(W = \infty)$
Búsqueda A* ponderada:	$g(n) + W \times h(n)$	$(1 < W < \infty)$

Podrías llamar a la búsqueda A* ponderada "búsqueda algo voraz": al igual que la búsqueda voraz del best-first, enfoca la búsqueda hacia un objetivo; por otro lado, no ignorará por completo el costo del camino y suspenderá un camino que hace poco progreso a gran costo.

Búsqueda subóptima acotada

Búsqueda de costo acotado

Búsqueda de costo no acotado

Hay una variedad de algoritmos de búsqueda subóptimos, que se caracterizan por criterios de lo considerado "bueno". En la **búsqueda subóptima acotada**, buscamos una solución que esté garantizada de estar dentro de un factor constante W del costo óptimo. La búsqueda A* ponderada garantiza. En la **búsqueda de costo acotado**, buscamos una solución cuyo costo sea menor que alguna constante C. Y en la **búsqueda de costo no acotado**, aceptamos una solución de cualquier costo, siempre y cuando podamos encontrarla rápido.

Búsqueda rápida

Un ejemplo de un algoritmo de búsqueda de costo no acotado es la **búsqueda rápida**, una versión de la búsqueda voraz del best-first que usa como heurística el número estimado de acciones requeridas para llegar a un objetivo, con independencia del costo de esas acciones. Para problemas donde todas las acciones tienen el mismo costo, es lo mismo que la búsqueda voraz del best-first, pero si las acciones tienen costos diferentes, lleva la búsqueda a encontrar rápido una solución, aun si tuviera un costo alto.

3.5.5 Búsqueda limitada a memoria

El punto con A* es su uso de memoria. Aquí, veremos unos trucos de ejecución que ahorran espacio, y unos algoritmos nuevos que aprovechan mejor el espacio disponible.

La memoria se divide entre *frontera* y *estados alcanzados*. En nuestra ejecución de Búsqueda del best-first, un estado que está en la frontera se almacena en dos lugares: como nodo en la frontera (para decidir qué expandir después) y como una entrada en la tabla de estados alcanzados (para saber si ya visitamos el estado antes). Para muchos problemas (como explorar una cuadrícula), esta duplicación no es una preocupación, porque el tamaño de la *frontier* es más pequeño que *reached*, por lo que duplicar los estados en la frontera requiere una cantidad de memoria trivial en comparación. Pero algunas ejecuciones mantienen un estado en sólo uno de los dos lugares, ahorrando espacio a costa de complicar (y, quizá, ralentizar) el algoritmo.

Otra posibilidad es eliminar estados de *reached* si demostramos que ya no son necesarios. Para unos problemas, podemos usar la propiedad de separación (Figura 3.6, p. 72), junto con la prohibición de acciones de retorno, para garantizar que las acciones se muevan hacia afuera desde la frontera o hacia otro estado de la frontera. En ese caso, sólo debemos verificar la frontera para encontrar rutas redundantes, y eliminar la *tabla* de *reached*.

Conteos de referencia

Para más problemas, podemos mantener **conteos de referencia** del número de veces que se ha alcanzado un estado y eliminarlo de la tabla de *reached* si ya no haya más formas de alcanzarlo. Así, en un mundo de cuadrícula donde cada estado sólo puede alcanzarse desde sus cuatro vecinos, una vez alcanzado un estado cuatro veces, podemos eliminarlo de la tabla.

Ahora veamos nuevos algoritmos diseñados para conservar el uso de memoria.

Búsqueda de haz

La **búsqueda de haz** limita el tamaño de la frontera. El enfoque más fácil es mantener sólo los k nodos con los mejores puntajes f, y descartar cualquier nodo expandido. Claro, esto hace que la búsqueda sea incompleta y subóptima, pero podemos elegir k para aprovechar la memoria disponible y el algoritmo se ejecute rápido porque expande menos nodos. Para muchos problemas, puede encontrar soluciones cercanas a óptimas. Puede pensar en la búsqueda uniforme o A* como que se expande por todas partes en contornos concéntricos, y puede pensar en la búsqueda de haz como explorando sólo una parte enfocada de esos contornos, la parte que contiene los k mejores candidatos.

Una versión alternativa de la búsqueda de haz no tiene un límite estricto en el tamaño de la frontera, sino que mantiene cada nodo cuyo puntaje f esté dentro de δ del mejor puntaje f. Así, cuando hay pocos nodos con puntajes fuertes, sólo se conservarán unos pocos, pero si no hay nodos fuertes, se conservarán más hasta que aparezca uno fuerte.

Algoritmo IDA*

Algoritmo IDA* es a A* lo que la búsqueda iterativa profunda es a la búsqueda de profundidad: IDA* brinda los beneficios de A* sin la necesidad de mantener todos los estados alcanzados en la memoria, a costa de visitar algunos estados varias veces. Es un algoritmo importante y comúnmente utilizado para problemas que no caben en la memoria.

En la búsqueda iterativa estándar, el límite es la profundidad, que aumenta en uno en cada iteración. En IDA*, el límite es el costo $f(g + h)$; en cada iteración, el valor de límite es el menor costo f de cualquier nodo que haya excedido el límite en la iteración previa. Es decir, cada iteración busca exhaustivamente un contorno f, halla un nodo justo pasando ese contorno y usa el costo f de ese nodo como el siguiente contorno. Para problemas como el 8-puzzle donde el costo f de cada ruta es un número entero, esto funciona bien y resulta en un progreso constante hacia el objetivo en cada iteración. Si la solución óptima tiene un costo C^*, no puede haber más de C^* iteraciones (así, no más de 31 iteraciones en los problemas de 8-puzzle más difíciles). Pero para un problema donde cada nodo tiene un costo f diferente, cada nuevo contorno podría contener sólo un nodo nuevo, y el número de iteraciones podría ser igual al número de estados.

Búsqueda recursiva del best-first

La **búsqueda recursiva del best-first (RBFS)** (Figura 3.22) imita la operación de la búsqueda estándar del best-first, pero usando sólo espacio lineal. RBFS se asemeja a una búsqueda de profundidad recursiva, pero en vez de continuar de modo indefinido por el camino actual, usa la variable de *límite-f* para realizar un seguimiento del valor f de la mejor ruta *alternativa* disponible desde cualquier ancestro del nodo actual. Si el nodo actual supera este límite, la recursión se deshace hasta la ruta alternativa. Conforme la recursión se deshace, RBFS reemplaza el valor f de cada nodo a lo largo de la ruta con un **valor respaldado**, el mejor valor f de sus hijos. Así, RBFS recuerda el valor f de la mejor hoja en el subárbol olvidado y, por lo tanto, puede decidir si vale la pena reexpandir el subárbol en algún momento posterior. La Figura 3.23 ilustra cómo RBFS llega a Bucarest.

Valor respaldado

RBFS es algo más eficiente que IDA*, pero aún sufre de regeneración excesiva de nodos. En el ejemplo de la Figura 3.23, RBFS sigue el camino por Rimnicu Vilcea, después "cambia de opinión" y prueba Fagaras, y luego cambia de opinión de nuevo. Estos cambios de opinión ocurren porque cada vez que se extiende la mejor ruta actual, es probable que el valor f aumente; h suele

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns una solución o failure solution,
fvalue ←RBFS(problem, NODE(problem.INITIAL),∞)
    return solution

function RBFS(problem,node,f_limit) returns una solución o failure, y un nuevo f-cost limit
    if problem.IS-GOAL(node.STATE) then return node successors ← LIST(EXPAND(node))
    if successors está vacío then return failure, ∞
    for each s in successors do // update f with value from previous search
        s.f ←max(s.PATH-COST + h(s), node.f))
    while true do
        best ←el nodo en successors con el valor mínimo en f-value
        if best.f > f_limit then return failure, best.f
        alternative ←el segundo valor mínimo en f-value among successors
        result, best.f ←RBFS(problem,best,min(f_limit,alternative))
        if result /= failure then return result, best.f
```

Figura 3.22 El algoritmo para la búsqueda recursiva del best-first.

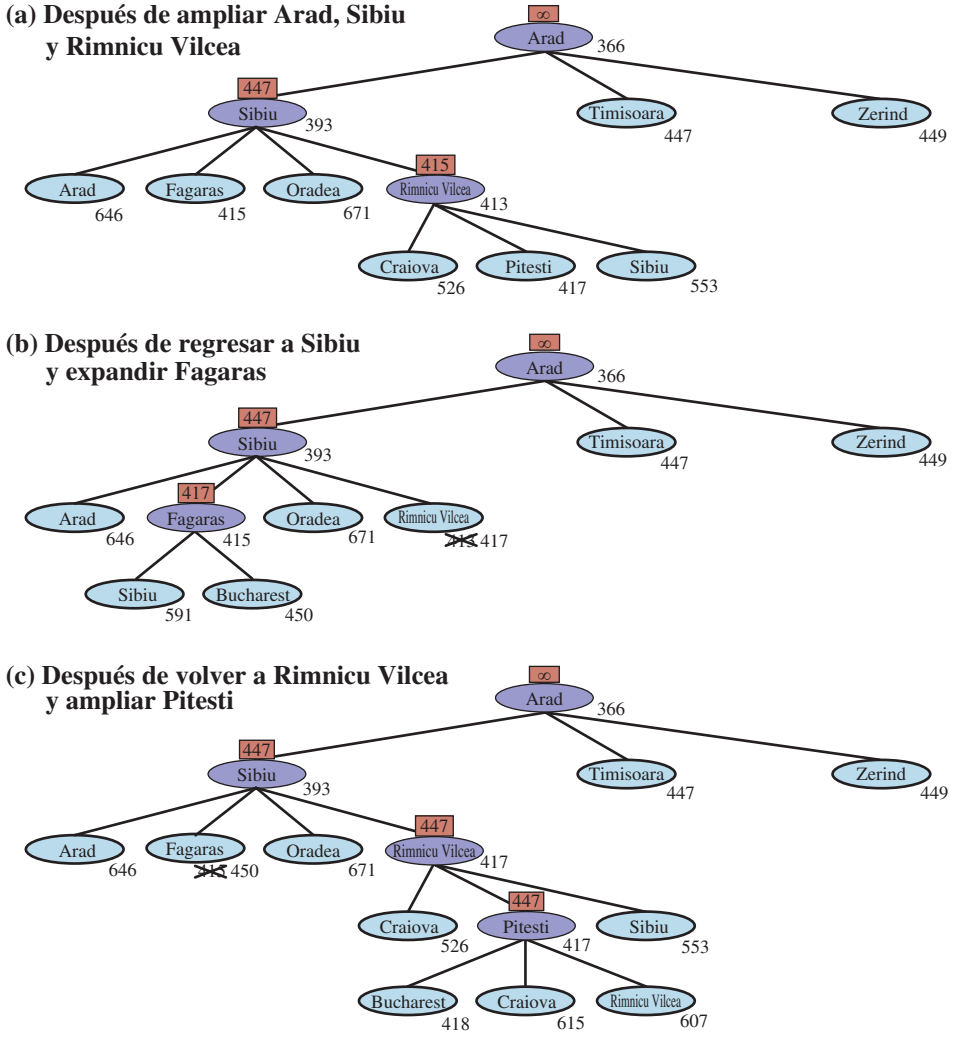


Figura 3.23 Prácticas en una RBFS buscan el camino más corto a Bucarest. El valor del límite-*f* para cada llamada recursiva se muestra encima de cada nodo actual, cada nodo está etiquetado con su costo *f*. (a) Se sigue el camino vía Rimnicu Vilcea hasta que la mejor hoja actual (Pitesti) tenga un valor peor que el mejor camino alternativo (Fagaras). (b) La recursividad se deshace y el mejor valor de hoja del subárbol olvidado (417) se respalda en Rimnicu Vilcea; luego se expande Fagaras, revelando un mejor valor de hoja de 450. (c) La recursividad se desenrolla y el mejor valor de hoja del subárbol olvidado (450) se respalda en Fagaras; luego se amplía Rimnicu Vilcea. Esta vez, como el mejor camino alternativo (a través de Timisoara) cuesta al menos 447, la ampliación continúa hasta Bucarest.

ser menos optimista para los nodos más cercanos a un objetivo. Si esto sucede, es posible que la segunda mejor ruta se convierta en la mejor ruta, por lo que la búsqueda debe retroceder para seguirla. Cada cambio de opinión corresponde a una iteración de IDA* y podría requerir muchas reexpansiones de nodos olvidados para recrear la mejor ruta y extenderla un nodo más.

RBFS es óptimo si la función heurística $h(n)$ es admisible. Su complejidad espacial es lineal en la profundidad de la solución óptima más profunda, pero su complejidad temporal es difícil de caracterizar: depende de la precisión de la función heurística como de la frecuencia con la que cambia el mejor camino a medida que se expanden los nodos. Expande los nodos en orden de puntaje *f* creciente, incluso si *f* no es monótono.

IDA* y RBFS sufren por usar *muy poca* memoria. Entre iteraciones, IDA* retiene sólo un número: el límite actual del costo *f*. RBFS retiene más información en la memoria, pero sólo usa espacio lineal: aun si hubiera más memoria disponible, RBFS no tiene forma de usarla. Debido a que olvidan la mayor parte de lo que han hecho, ambos algoritmos pueden terminar reexplorando los mismos estados muchas veces.

Por lo tanto, parece sensato determinar cuánta memoria tenemos disponible y permitir que un algoritmo la use toda. Dos algoritmos que hacen esto son **MA*** (A* con límite de memoria) y **SMA*** (MA* simplificado). SMA* es más simple, así que lo describiremos. SMA* procede como A*, expandiendo la mejor hoja hasta que la memoria esté llena. En este punto, no puede agregar un nuevo nodo al árbol de búsqueda sin eliminar uno viejo. SMA* siempre elimina la *peor* hoja, es decir, la que tiene el valor *f* más alto. Como RBFS, SMA* luego retrocede el valor del nodo olvidado a su padre. De esta manera, el ancestro de un subárbol olvidado conoce la calidad del mejor camino en ese subárbol. Con esta información, SMA* regenera el subárbol sólo cuando se ha demostrado que todos los demás caminos parecen peores que el camino que ha olvidado. Otra forma de decir esto es que si todos los descendientes de un nodo *n* son olvidados, entonces no sabremos qué camino tomar desde *n*, pero aún tendremos una idea de cuán valioso es ir a cualquier lugar desde *n*.

MA*

SMA*

El algoritmo completo se describe en el repositorio de código en línea que acompaña este libro. Hay una sutileza a mencionar. Dijimos que SMA* expande la mejor hoja y elimina la peor. ¿Qué pasa si *todas* las hojas tienen el mismo valor *f*? Para evitar seleccionar el mismo nodo para eliminar y expandir, SMA* expande la mejor hoja *nueva* y elimina la peor hoja *más antigua*. Las hojas coinciden si hay sólo una hoja, pero en ese caso, el árbol de búsqueda actual debe ser un único camino de la raíz a la hoja que llene toda la memoria. Si la hoja no es un nodo objetivo, *aun si está en un camino de solución óptimo*, esa solución no es alcanzable con la memoria disponible. Entonces, el nodo se puede descartar exactamente como si no tuviera sucesores.

SMA* es completo si hay una solución alcanzable, es decir, si *d*, la profundidad del nodo objetivo más superficial, es menor que el tamaño de la memoria (expresado en nodos). Es óptimo si una solución óptima es alcanzable; de lo contrario, devuelve la mejor solución alcanzable. En la práctica, SMA* es una opción robusta para encontrar soluciones óptimas, en especial si el espacio de estados es un grafo, los costos de acción no son uniformes y la generación de nodos es costosa en comparación con los gastos generales de mantener la frontera y el conjunto alcanzado.

Sin embargo, en problemas complicados sucederá que SMA* se ve obligado a cambiar constantemente entre muchos caminos de solución candidatos, sólo un pequeño subconjunto de los cuales puede caber en la memoria. (Esto se asemeja al problema del **thrashing** en los sistemas de paginación de disco). Luego, el tiempo extra requerido para esta regeneración de los mismos nodos significa que los problemas que serían de hecho resolubles por A*, dado una memoria ilimitada, se vuelven intratables para SMA*. Es decir, *las limitaciones de memoria pueden hacer que un problema sea intratable desde el punto de vista del tiempo de computación*. Aunque ninguna teoría actual explica el equilibrio entre tiempo y memoria, parece que es un problema inevitable. La única salida es eliminar el requisito de optimalidad.

Thrashing

3.5.6 La búsqueda heurística bidireccional

Con la búsqueda unidireccional del best-first, vimos que usar $f(n) = g(n) + h(n)$ como función de evaluación da una búsqueda A* que garantiza soluciones de costo óptimo (suponiendo una h admisible) al mismo tiempo que es óptimamente eficiente en el número de nodos expandidos.

Con la búsqueda bidireccional del best-first, también podríamos intentar usar $f(n) = g(n) + h(n)$, pero por desgracia no hay garantía de que esto conduzca a una solución de costo óptimo, ni que sea óptimamente eficiente, aun con una heurística admisible. Con la búsqueda bidireccional, resulta que no son los nodos individuales sino más bien *pares* de nodos (uno de cada frontera) los que pueden demostrarse que se expanden con seguridad, por ello cualquier prueba de eficiencia tendrá que considerar pares de nodos (Eckerle *et al*., 2017).

Comenzaremos con una nueva notación. Usamos $f_F(n) = g_F(n) + h_F(n)$ para los nodos que van en la dirección hacia adelante (con el estado inicial como raíz) y $f_B(n) = g_B(n) + h_B(n)$ para los nodos en la dirección hacia atrás (con un estado objetivo como raíz). Aunque ambas búsquedas hacia adelante y hacia atrás resuelven el mismo problema, tienen funciones de evaluación diferentes porque las heurísticas son diferentes, dependiendo de si se está luchando por el objetivo o por el estado inicial. Supondremos heurísticas admisibles.

Considere un camino hacia adelante desde el estado inicial hasta un nodo m y un camino hacia atrás desde el objetivo a un nodo n. Podemos definir un límite inferior en el costo de una solución que sigue el camino del estado inicial a m, que luego llega a n, y sigue el camino hacia el objetivo como

$$lb(m,n) = max(g_F(m) + g_B(n), f_F(m), f_B(n))$$

Es decir, el costo de tal camino debe ser tan grande como la suma de los costos de los caminos de ambas partes (porque la conexión restante entre ellos debe tener un costo no negativo), y el costo también debe ser al menos tanto como el costo f estimado de cualquier parte (pues las estimaciones heurísticas son optimistas). Así, el teorema es que para cualquier par de nodos m, n con $lb(m,n)$ menor que el costo óptimo C^*, debemos expandir m o n, ya que el camino que pasa por ambos es una solución en potencia óptima. La dificultad es que no sabemos con certeza qué nodo es mejor expandir, y por lo tanto, ningún algoritmo de búsqueda bidireccional garantiza ser óptimamente eficiente; cualquier algoritmo podría expandir hasta el doble del número mínimo de nodos, si siempre elige el miembro incorrecto de un par para expandir primero. Algunos algoritmos de búsqueda heurística bidireccional manejan explícitamente una cola de pares (m,n), pero nos quedaremos con la búsqueda bidireccional del best-first (Figura 3.14), que tiene dos colas de prioridad de frontera, y le daremos una función de evaluación que imita los criterios de lb:

$$f_2(n) = \max(2g(n), g(n) + h(n))$$

El nodo a expandir después será el que minimice este valor f_2; el nodo puede provenir de cualquier frontera. Esta función f_2 garantiza que nunca expandiremos un nodo (de ninguna frontera) con $g(n) > \frac{C^*}{2}$. Decimos que ambas mitades de la búsqueda "están en el medio" en el sentido de que si las dos fronteras se tocan, ningún nodo dentro de ninguna frontera tiene un costo de camino mayor que el límite $\frac{C^*}{2}$. La Figura 3.24 ilustra un ejemplo de búsqueda bidireccional.

Hemos descrito un enfoque donde la heurística h_F estima la distancia al objetivo (o, si el problema tiene múltiples estados objetivo, la distancia al objetivo más cercano) y h_B estima la distancia al inicio. Esto se llama **búsqueda de frente a fin**. Una alternativa, llamada **búsqueda de frente a frente**, estima la distancia a la otra frontera. Claramente, si una frontera tiene millones de nodos, sería ineficiente aplicar la función heurística a cada uno de ellos y tomar el mínimo. Puede funcionar

Búsqueda de frente a fin

Búsqueda de frente a frente

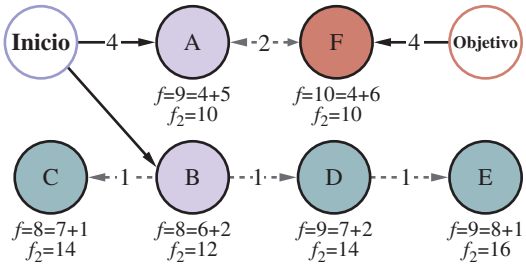


Figura 3.24 La búsqueda bidireccional mantiene dos fronteras: a la izquierda, los nodos A y B son sucesores de Inicio; a la derecha, el nodo F es sucesión inversa de Objetivo. Cada nodo está etiquetado con valores $f = g + h$ y el valor $f_2 = \max(2g, g + h)$. (Los valores g suman los costos de la acción, mostrada en cada flecha; los valores h son arbitrarios y no se derivan de nada en la figura). La solución óptima, Inicio-A-F-Objetivo, ha costado $C^* = 4 + 2 + 4 = 10$, y significa que un algoritmo bidireccional de encuentro en el medio no debería expandir ningún nodo con $g > \frac{C^*}{2} = 5$; de hecho, el siguiente nodo a expandir sería A o F (cada uno con $g = 4$), lo que llevaría a una solución óptima. Si expandimos primero el nodo con el menor costo f, entonces B y C vendrían después, y D y E se empatarían con A, pero todos tienen $g > \frac{C^*}{2}$ y, así, nunca se expanden cuando f_2 es la función de evaluación.

muestrear nodos de la frontera. En ciertos dominios de problemas específicos, se puede *resumir* la frontera; así, en un problema de búsqueda de cuadrícula, calculamos incrementalmente una caja delimitadora de la frontera y usamos como heurística la distancia a esa caja.

La búsqueda bidireccional a veces es más eficiente que la búsqueda unidireccional, a veces no. En general, con una buena heurística, la búsqueda A* produce contornos de búsqueda que se centran en el objetivo, y agregar búsqueda bidireccional no ayuda mucho. Con una heurística promedio, la búsqueda bidireccional que se topa en el medio tiende a expandir menos nodos y es preferida. En el peor caso de una heurística deficiente, la búsqueda no se centra en el objetivo, y la búsqueda bidireccional tiene la misma complejidad asintótica que A*; pero esta búsqueda con la función de evaluación f_2 y una heurística admisible h es completa y óptima.

3.6 Funciones heurísticas

Ahora, examinamos cómo la precisión de una heurística afecta el rendimiento de la búsqueda; además, consideramos cómo se inventan heurísticas, y ejemplificamos con el 8-puzzle. Como se indicó en la Sección 3.2, el objetivo del 8-puzzle es deslizar las fichas horizontal o verticalmente en el espacio vacío hasta que la configuración coincida con la configuración objetivo (Figura 3.25).

Hay 9!/2 = 181 400 estados alcanzables en un 8-puzzle; por ello, una búsqueda los mantendría a todos fácilmente en memoria. Para el 15-puzzle, hay 16!/2 estados, más de 10 billones; así, para buscar en ese espacio necesitaremos la ayuda de una buena función heurística admisible. Hay una larga historia de heurísticas para el 15-puzzle; aquí dos muy utilizados:

- h_1 = el número de fichas mal colocadas (blancos no incluidos). Para la Figura 3.25, las ocho fichas están fuera de posición; por ello, el estado inicial tiene $h_1 = 8$. h_1, es una heurística admisible ya que toda ficha fuera de lugar requerirá al menos un movimiento para ponerla en su lugar correcto.

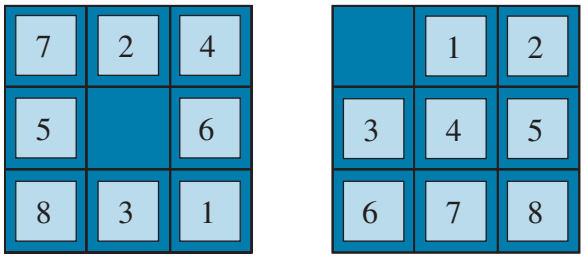


Figura 3.25 Ejemplo típico del 8-puzzle. La solución más corta tiene 26 acciones.

Distancia de Manhattan

- h_2 = suma de distancias de las fichas desde sus posiciones objetivo. Como las fichas no se deslizan en diagonal, la distancia es la suma de las distancias horizontales y verticales, llamada **distancia de bloque de ciudad** o **distancia de Manhattan**. h_2 también es admisible porque cualquier deslizamiento de ficha es un paso más cerca del objetivo. Las fichas 1 a 8 en el estado inicial de la Figura 3.25 dan una distancia de Manhattan de

$$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18.$$

Como se esperaba, ninguno de estos sobreestima el costo real de la solución, que es 26.

3.6.1 El efecto de la precisión heurística en el rendimiento

Factor de ramificación efectivo

Un modo de caracterizar la calidad de una heurística es el **factor de ramificación efectivo** b^*. Si el número total de nodos generados por A* para un problema particular es N y la profundidad de la solución es d, entonces b^* es el factor de ramificación que un árbol uniforme de profundidad d tendría para contener $N + 1$ nodos. Así,

$$N + 1 = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d.$$

Así, si A* encuentra solución a una profundidad de 5 usando 52 nodos, entonces el factor de ramificación efectivo es 1.92. Este factor varía según las instancias del problema, pero para un dominio específico (como un 8-puzzle) es constante en todas las instancias de problemas no triviales. Las mediciones experimentales de b^* en un pequeño conjunto de problemas da una buena guía sobre la utilidad general de la heurística, que si está bien diseñada tendría un valor de b^* cercano a 1, lo que permite resolver problemas grandes con un costo computacional razonable.

Profundidad efectiva

Korf y Reid (1998) afirman que la mejor caracterización del efecto de la poda de A* con una heurística dada h es que reduce la **profundidad efectiva** en una constante k_h, comparada con la profundidad real. Esto significa que el costo de búsqueda total es $O(b^{d-kh})$, comparado con $O(b^d)$ para una búsqueda no informada. Sus experimentos en problemas de cubo de Rubik y de n-puzzle muestran que la fórmula predice con precisión el costo total de búsqueda para instancias de problemas muestreadas en una amplia gama de longitudes de solución, al menos para las mayores que k_h.

Para la Figura 3.26, generamos problemas aleatorios de 8-puzzle y los resolvimos con una búsqueda de amplitud no informada y con una búsqueda A* usando h_1 y h_2, reportando el número promedio de nodos generados y el factor de ramificación efectivo correspondiente para cada estrategia de búsqueda y cada longitud de solución. Los resultados sugieren que h_2 es mejor que h_1, y ambos son mejores que no usar heurística.

	Costo de búsqueda (nodos generados)			Factor de ramificación efectivo		
d	BFS	A*(h_1)	A*(h_2)	BFS	A*(h_1)	A*(h_2)
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Figura 3.26 Comparación de costos de búsqueda y factores de ramificación efectivos para problemas de 8-puzzle usando búsqueda de amplitud, A* con h_1 (fichas fuera de lugar) y A* con h_2 (distancia de Manhattan). Los datos son promedios de 100 rompecabezas para cada solución con longitud d de 6 a 28.

Uno podría preguntarse si h_2 *siempre* es mejor que h_1. La respuesta es "Esencialmente, sí". Es fácil ver a partir de las definiciones de ambas heurísticas que para cualquier nodo n, $h_2(n) \geq h_1(n)$. Decimos que h_2 **domina** a h_1. La dominación se traduce en eficiencia: A* usando h_2 nunca expandirá más nodos que A* usando h_1 (excepto en el caso de desempates desafortunados). El argumento es simple. Recordemos la observación en la página 90 de que todo nodo con $f(n) < C^*$ de seguro será expandido; es lo mismo que decir que todo nodo con $h(n) < C^* - g(n)$ seguro será expandido si h es consistente. Pero como h_2 es tan grande como h_1 para todos los nodos, cada nodo que sea seguramente expandido por la búsqueda A* con h_2 también será expandido con h_1, y h_1 podría causar que otros nodos se expandan. En general es mejor usar una función heurística con valores más altos, siempre que sea consistente y que el tiempo de cálculo de la heurística no sea demasiado largo.

Domina

3.6.2 Generar heurísticas a partir de problemas flexibles

Hemos visto que tanto h_1 (fichas mal colocadas) como h_2 (distancia de Manhattan) son heurísticas muy buenas para el 8-puzzle y que h_2 es mejor. ¿Cómo podría uno haber llegado a h_2? ¿Puede una computadora inventar tal heurística mecánicamente?

h_1 y h_2 son estimaciones de longitud del camino restante para 8-puzzle, pero también son longitudes de camino precisas para versiones *simplificadas* del rompecabezas. Si las reglas de éste cambiaran para que una ficha se deslizara a cualquier casilla, en vez de sólo a la vacía adyacente, entonces h_1 daría la longitud exacta de la solución más corta. Y, si una ficha se deslizara una casilla en cualquier dirección, aun a una ocupada, entonces h_2 daría la longitud exacta de la solución más corta. Un problema con menos restricciones en las acciones se llama **problema flexible**. El grafo del espacio de estados del problema flexible es un *supergrafo* del espacio de estados original, porque la eliminación de restricciones crea bordes adicionales en el grafo.

Problema flexible

Debido a que el problema flexible agrega bordes al grafo del espacio de estados, cualquier solución óptima en el problema original es, por definición, una solución en el problema flexible; pero

el problema flexible puede tener *mejores* soluciones si los bordes adicionales proporcionan atajos. Por lo tanto, *el costo de una solución óptima para un problema flexible es una heurística admisible para el problema original*. Además, como la heurística derivada es un costo exacto para el problema flexible, debe obedecer la desigualdad triangular y, así, ser consistente (ver página 88).

Si una definición de problema se escribe en un lenguaje formal, es posible construir problemas flexibles de modo automático.[14] Así, si las acciones del 8-puzzle se describen como

> Una ficha puede moverse de la casilla X a la casilla Y si
> X está adyacente a Y **y** Y está en blanco,

podemos generar tres problemas flexibles eliminando una o ambas condiciones:

(a) Una ficha puede deslizarse de la casilla X a la casilla Y si X está adyacente a Y.
(b) Una ficha puede deslizarse de la casilla X a la casilla Y si Y está en blanco.
(c) Una ficha puede deslizarse de la casilla X a la casilla Y.

A partir de (a), podemos derivar h_2 (distancia de Manhattan). El razonamiento es que h_2 sería la puntuación adecuada si moviéramos cada ficha a su destino por turno. La heurística derivada de (b) se analiza en el Ejercicio 3.GASC. A partir de (c), podemos derivar h_1 (fichas mal colocadas) porque sería la puntuación adecuada si las fichas pudieran moverse a su destino en una acción. Vea que es crucial que los problemas flexibles generados por esta técnica puedan resolverse en esencia *sin búsqueda*, porque las reglas flexibles permiten que el problema se descomponga en ocho subproblemas independientes. Si el problema flexible es difícil de resolver, los valores de la heurística correspondiente serán costosos de obtener.

El programa ABSOLVER genera heurísticas de modo automático a partir de definiciones de problemas, usando el método del "problema flexible" y otras técnicas (Prieditis, 1993). ABSOLVER generó una nueva heurística para el 8-puzzle, mejor que cualquier heurística preexistente y encontró la primera heurística útil para el famoso Cubo de Rubik.

Si hay una colección de heurísticas admisibles $h_1,..., h_m$ disponibles para un problema y ninguna de ellas es claramente mejor que las demás, ¿cuál deberíamos elegir? Resulta que podemos tener lo mejor de todos los mundos, definiendo

$$h(n) = \max\{h_1(n),... , h_k(n)\}.$$

Esta heurística compuesta elige la función más precisa para el nodo en cuestión. Como los componentes h_i son admisibles, h es admisible (y si h_i son todos consistentes, h es consistente). Además, h domina todas sus heurísticas componentes. El único inconveniente es que $h(n)$ tarda más en calcularse. Si eso es un problema, una alternativa es seleccionar aleatoriamente una heurística en cada evaluación, o usar un algoritmo de aprendizaje automático para predecir cuál será la mejor heurística. Hacer esto puede resultar en una heurística inconsistente (aun si cada h_i es consistente), pero en la práctica conduce, en general, a una solución de problemas más rápida.

3.6.3 La generación de heurísticas a partir de subproblemas: Bases de datos de patrones

Subproblema

Las heurísticas admisibles pueden derivarse del costo de la solución de un **subproblema** de un problema dado. Por ejemplo, la Figura 3.27 muestra un subproblema de la instancia del 8-puzzle en la Figura 3.25. El subproblema implica colocar las fichas 1, 2, 3, 4 y la casilla en blanco en sus posiciones correctas. Claramente, el costo de la solución óptima de este subproblema es una cota

[14] En los Capítulos 8 y 11, describimos lenguajes formales adecuados para esta tarea; con descripciones formales que se manipulen, se puede automatizar la construcción de problemas flexibles. Por ahora usamos el inglés.

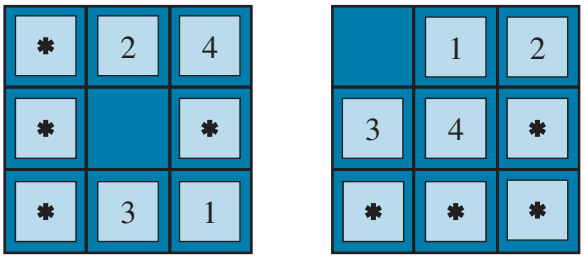


Figura 3.27 Subproblema del ejemplo de 8-puzzle ilustrado en la Figura 3.25. La tarea consiste en colocar las fichas 1, 2, 3, 4 y el espacio en blanco en sus posiciones correctas, sin preocuparse por lo que sucede con las otras fichas.

inferior del costo del problema completo. Resulta ser más preciso que la distancia de Manhattan en algunos casos.

Bases de datos de patrones

La idea detrás de las **bases de datos de patrones** es almacenar costos de solución exactos para cada instancia de subproblema posible; en el ejemplo, cada configuración posible de las cuatro fichas y el espacio en blanco. (Habrá $9 \times 8 \times 7 \times 6 \times 5 = 15,120$ patrones en la base de datos. Las identidades de las otras cuatro fichas son irrelevantes para resolver el subproblema, pero los movimientos de las fichas cuentan hacia el costo de solución del subproblema). Luego calculamos una heurística admisible h_{DB} para cada estado hallado en una búsqueda sólo buscando la configuración de subproblema correspondiente en la base de datos. Ésta se construye buscando hacia atrás desde el objetivo y registrando el costo de cada nuevo patrón hallado;[15] el gasto de esta búsqueda se amortiza sobre las instancias de problemas subsiguientes, por ello tiene sentido si esperamos que se nos pida resolver muchos problemas.

La elección de las fichas 1-2-3-4 para acompañar al espacio en blanco es arbitraria; también podríamos construir bases de datos para 5-6-7-8, para 2-4-6-8, etc. Cada base de datos produce una heurística admisible, y estas heurísticas pueden combinarse, como antes se explicó, tomando el valor máximo. Una heurística combinada de este tipo es más precisa que la distancia de Manhattan; el número de nodos generados al resolver 15-puzzle aleatorios se reduce en un factor de 1 000. Pero, con cada base de datos adicional hay rendimientos decrecientes y aumentan los costos de memoria y computación.

Uno podría preguntarse si las heurísticas obtenidas de la base de datos 1-2-3-4 y 5-6-7-8 podrían *sumarse*, pues ambos subproblemas parecen no solaparse. ¿Seguiría siendo esta una heurística admisible? La respuesta es no, porque las soluciones del subproblema 1-2-3-4 y del 5-6-7-8 para un estado dado casi con certeza compartirán unos movimientos; es poco probable que 1-2-3-4 se mueva a su lugar sin tocar 5-6-7-8, y viceversa. Pero ¿qué pasa si no contamos esos movimientos? ¿Qué si no abstraemos las otras fichas a inicios, sino que las desaparecemos? Es decir, no registramos el costo total de resolver el subproblema 1-2-3-4, sino sólo el número de movimientos que involucran 1-2-3-4. Luego, la suma de ambos costos sigue siendo un límite inferior para el costo de resolver todo el problema. Esta es la idea detrás de las **bases de datos de patrones disyuntos**. Con éstas es posible resolver 15-puzzle aleatorios en pocos milisegundos; el número de nodos generados se reduce en un factor de 10 000 en comparación con el uso de la distancia de Manhattan. Para 24-puzzle, se puede

Bases de datos de patrones disyuntos

[15] Al trabajar hacia atrás desde el objetivo, el costo exacto de la solución de cada instancia encontrada está disponible de inmediato; ejemplo de programación dinámica, a analizar a detalle en el Capítulo 17.

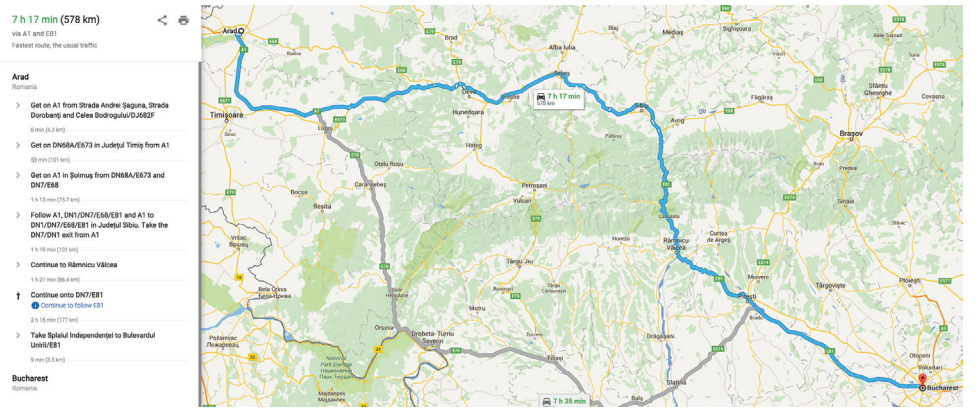


Figura 3.28 Servicio web de indicaciones para conducir, computada con algoritmo de búsqueda.

obtener una aceleración de casi un millón de veces. Las bases de datos de patrones disjuntos funcionan para rompecabezas de deslizamiento porque el problema se puede dividir de modo que cada movimiento afecte sólo un subproblema, pues sólo se mueve una ficha a la vez.

3.6.4 Generando heurísticas con puntos de referencia

Hay servicios en línea que alojan mapas con decenas de millones de vértices y encuentran direcciones de manejo óptimas en cuestión de milisegundos (Figura 3.28). ¿Cómo hacen eso, cuando los mejores algoritmos de búsqueda que considerados hasta ahora son casi un millón de veces más lentos? Hay muchos trucos, pero el mejor es la **precomputación** de algunos costos de ruta óptimos. Aunque la precomputación lleva tiempo, sólo necesita hacerse una vez, luego se amortiza en miles de millones de solicitudes de búsqueda de usuarios.

Precomputación

Podemos generar una heurística perfecta al precomputar y almacenar el costo de la ruta óptima entre cada par de vértices. Eso tomaría espacio $O(|V|^2)$ y tiempo $O(|E|^3)$; es práctico para grafos con 10 mil vértices, pero no para 10 millones.

Punto de referencia

Un mejor enfoque es elegir pocos (quizá 10 o 20) **puntos de referencia**[16] de los vértices. Luego, para cada punto de referencia L y para cada otro vértice v en el grafo, calculamos y almacenamos $C^*(v,L)$, el costo exacto de la ruta óptima de v a L. (También necesitamos $C^*(L,v)$; en un grafo no dirigido esto es lo mismo que $C^*(v,L)$; en uno dirigido, con calles de un solo sentido, debemos calcular por separado). Dadas las tablas C^* almacenadas, podemos crear una heurística eficiente (aunque no admisible): el mínimo, sobre todos los puntos de referencia, del costo de ir del nodo actual al punto de referencia, y luego al objetivo:

$$h_L(n) = \min_{L \in Landmarks} C^*(n,L) + C^*(L,goal)$$

Si la ruta óptima pasa por un punto de referencia, esta heurística será exacta; si no es inadmisible, sobreestima el costo hasta el objetivo. En una búsqueda A*, si tiene heurísticas exactas, una vez que llegue a un nodo que está en una ruta óptima, cada nodo que expanda a partir de ahí estará

[16] Los puntos emblemáticos a veces se denominan "pivotes" o "anclas".

en una ruta óptima. Piense en las líneas de contorno siguiendo esta ruta óptima. La búsqueda seguirá la ruta óptima, en cada iteración agregando una acción con costo c para llegar a un estado resultado cuyo valor h será c menos, lo que significa que el puntaje total $f = g + h$ será constante en C^* a lo largo de la ruta.

Atajos

Algunos algoritmos de búsqueda de rutas ahorran más tiempo al agregar **atajos**: bordes artificiales en el grafo que definen una ruta óptima de múltiples acciones. Por ejemplo, si hubiera atajos predefinidos entre las 100 ciudades más grandes de Estados Unidos, y tratáramos de ir desde el campus de Berkeley, en California, hasta la NYU, en Nueva York, podríamos tomar el atajo entre Sacramento y Manhattan y cubrir 90 % del camino en una acción.

$h_L(n)$ es eficiente, no admisible. Con un poco más de cuidado, podemos idear una heurística que sea eficiente y admisible:

$$h_{DH}(n) = \max_{L \in Landmarks} |C^*(n,L) - C^*(goal,L)|$$

Heurística diferencial

Esto se llama **heurística diferencial** (debido a la resta). Piénsela como un punto de referencia que está más allá del objetivo. Si éste se encuentra en la ruta óptima de n al punto de referencia, entonces nos dice "considera toda la ruta desde n a L, luego resta la última parte de esa ruta, desde el *objetivo* hasta L, lo que da el costo exacto de la ruta desde n hasta el *objetivo*". Conforme el objetivo esté un poco fuera de la ruta óptima hacia el punto de referencia, la heurística será inexacta, pero aun así será admisible. Los puntos de referencia que no estén más allá del objetivo no serán útiles; un punto de referencia que esté exactamente a mitad de camino entre n y el *objetivo* dará $h_{DH} = 0$, lo cual no es útil.

Hay varias formas de seleccionar puntos de referencia: al azar es rápido, pero logramos mejores resultados si nos aseguramos de distribuirlos para que no estén demasiado cerca unos de otros. Un enfoque ambicioso es elegir un primer punto de referencia al azar, luego encontrar el punto que esté más lejos de ese, y agregarlo al conjunto de puntos de referencia, y continuar, en cada iteración agregando el punto de maximiza la distancia al punto de referencia más cercano. Si tiene registros de búsquedas pasadas de sus usuarios, puede elegir puntos de referencia que sean solicitados con frecuencia en las búsquedas. Para la heurística diferencial, es bueno que los puntos de referencia se distribuyan alrededor del perímetro del grafo. Así, una buena técnica es hallar el centroide del grafo, disponer k sectores en forma de pie alrededor del centroide, y en cada sector seleccionar el vértice que esté más lejos del centro.

Los puntos de referencia funcionan en especial bien en problemas de búsqueda de rutas gracias a como se construyen las carreteras en el mundo: mucho tráfico quiere viajar entre puntos de referencia, por ello los ingenieros construyen las carreteras más anchas y rápidas por estas rutas; la búsqueda de puntos de referencia facilita la recuperación de estas rutas.

3.6.5 Aprender a buscar mejor

Hemos visto varias estrategias de búsqueda fijas, de amplitud, A*, etc., diseñadas con cuidado y programadas por científicos de computación. ¿Podría un agente *aprender* a buscar mejor? La respuesta es sí, el método se basa en un concepto llamado **espacio de estados meta**. Cada estado en este espacio captura el estado interno (computacional) de un programa que busca en un espacio de estados ordinario como el mapa de Rumania. (Para mantener separados ambos conceptos, llamamos al mapa de Rumania **espacio de estados a nivel de objeto**). Por ejemplo, el estado interno del algoritmo A* consiste en el árbol de búsqueda actual. Cada acción en el espacio de estados meta es un paso de computación que altera el estado interno; por ejemplo, cada paso de

Espacio de estados meta

Espacio de estados a nivel de objeto

computación en A* expande un nodo hoja y agrega sus sucesores al árbol. Así, la Figura 3.18, que muestra una secuencia de árboles de búsqueda cada vez más grandes, es como la representación de una ruta en el espacio de estados meta donde cada estado en la ruta es un árbol de búsqueda a nivel de objeto.

Ahora, el camino en la Figura 3.18 tiene cinco pasos, incluido la expansión de Fagaras, que no es propiamente útil. Para problemas más difíciles, habrá muchos de estos errores, y un algoritmo de **aprendizaje a nivel meta** puede aprender de estas experiencias para evitar explorar subárboles poco prometedores. Las técnicas usadas para este tipo de aprendizaje se describen en el Capítulo 22. El objetivo del aprendizaje es minimizar el **costo total** de la resolución de problemas, equilibrando los costos computacional y de camino.

Aprendizaje a nivel meta

3.6.6 Aprender heurísticas de la experiencia

Una forma de inventar una heurística es idear un problema flexible para encontrar una solución óptima fácilmente. Una alternativa es aprender de la "experiencia", que aquí significa resolver 8-puzzles, por ejemplo. Cada solución óptima a un problema de 8-puzzle da un ejemplo (objetivo, camino); a partir de esto se usa un algoritmo de aprendizaje para construir una función h que, con suerte, aproxime el costo del camino real para otros estados que surgen en la búsqueda. La mayoría de estos enfoques aprenden una aproximación imperfecta de la función heurística y, por ello, corren el riesgo de ser inadmisibles. Esto conduce a un inevitable compromiso entre el tiempo de aprendizaje, el tiempo de ejecución de la búsqueda y el costo de la solución. Las técnicas de aprendizaje automático se verán en el Capítulo 19. Los métodos de aprendizaje por refuerzo, descritos en el Capítulo 22, son aplicables a la búsqueda.

Característica

Unas técnicas de aprendizaje automático funcionan mejor si se proveen **características** de un estado relevantes para predecir su valor heurístico, en vez de sólo dar la descripción del estado en bruto. Por ejemplo, la característica "número de fichas mal colocadas" sería útil para predecir la distancia real de un estado de 8-puzzle al objetivo. Llamemos a esta característica $x_1(n)$. Podríamos tomar 100 arreglos de 8-puzzle generados al azar y recopilar datos de sus costos de solución reales. Podríamos encontrar que si $x_1(n)$ es 5, el costo promedio de la solución es de casi 14, etc. Claro, podemos usar múltiples características. Una segunda característica $x_2(n)$ sería "número de pares de fichas adyacentes, no son adyacentes en el estado objetivo". ¿Cómo combinar $x_1(n)$ y $x_2(n)$ para predecir $h(n)$? Un enfoque común es usar una combinación lineal:

$$h(n) = c_1 x_1(n) + c_2 x_2(n).$$

Las constantes c_1 y c_2 se ajustan para encajar mejor a los datos reales en las configuraciones generadas al azar. Se espera que c_1 y c_2 sean positivos porque las fichas mal colocadas y los pares adyacentes incorrectos complican la resolución del problema. Esta heurística satisface la condición $h(n) = 0$ para los estados objetivo, pero no es necesariamente admisible o consistente.

Resumen

El capítulo mostró algoritmos de búsqueda que un agente puede usar para seleccionar secuencias de acciones en un sinfín de entornos, siempre y cuando sean episódicos, de un solo agente, por completo observables, deterministas, estáticos, discretos y conocidos. Hay que hacer concesiones entre el tiempo que lleva la búsqueda, la cantidad de memoria disponible y la calidad de la solución. Podemos ser más eficientes si tenemos conocimientos dependientes del dominio en

forma de una función heurística que estime qué tan lejos está un estado dado del objetivo, o si precalculamos soluciones parciales que involucran patrones o puntos de referencia.

- Antes de que un agente inicie la búsqueda, debe plantear un **problema** bien definido.
- Un problema consta de cinco partes: **estado inicial**, conjunto de **acciones**, **modelo de transición** que describe los resultados de las acciones, conjunto de **estados objetivo** y **función de costo de acción**.
- El entorno del problema es representado por un grafo de **espacio de estados**. Un **camino** por el espacio de estados (una secuencia de acciones), desde el estado inicial hasta un estado objetivo, es una **solución**.
- Los algoritmos de búsqueda, en general, tratan los estados y acciones como **atómicos**, sin ninguna estructura interna (aunque introducimos características de estados cuando llegó el momento de aprender).
- Los algoritmos de búsqueda se juzgan en función de la **completitud, optimalidad de costos, complejidad temporal y complejidad espacial**.
- **Los métodos de búsqueda no informados** sólo tienen acceso a la definición del problema. Los algoritmos construyen un árbol de búsqueda en un intento de solución. Los algoritmos difieren según qué nodo expandan primero:
  - **La búsqueda de best-first** selecciona nodos para expansión usando una **función de evaluación**.
  - **La búsqueda de amplitud** expande primero los nodos más superficiales; es completa, óptima para costos de acción unitarios, pero tiene complejidad exponencial del espacio.
  - **La búsqueda de costo uniforme** expande el nodo con el costo de camino más bajo, $g(n)$, y es óptima para costos de acción generales.
  - **La búsqueda de profundidad** expande el nodo no expandido más profundo primero. No es completa ni óptima, pero tiene complejidad espacial lineal. La **búsqueda de profundidad limitada** agrega un límite de profundidad.
  - **La búsqueda iterativa en profundidad** llama a la búsqueda de profundidad con límites de profundidad crecientes hasta que se encuentra un objetivo. Es completa cuando se realiza una verificación de ciclo completa, óptima para costos de acción unitarios, tiene complejidad temporal comparable a la búsqueda de amplitud y tiene complejidad espacial lineal.
  - **La búsqueda bidireccional** expande dos frentes, uno alrededor del estado inicial y otro alrededor del objetivo, deteniéndose cuando ambos frentes se encuentran.
- **Los métodos de búsqueda informados** tienen acceso a una función **heurística** $h(n)$ que estima el costo de una solución a partir de n. Pueden tener acceso a información adicional, como bases de datos de patrones con costos de solución.
  - La **búsqueda voraz del best-first** expande nodos con $h(n)$ mínimo. No es óptima, pero a menudo es eficiente.
  - **La búsqueda A*** expande nodos con $f(n)$ mínimo $= g(n) + h(n)$. A* es completo y óptimo, siempre que $h(n)$ sea admisible. La complejidad espacial de A* sigue siendo un problema para muchos problemas.
  - **La búsqueda A* bidireccional** a veces es más eficiente que A* mismo.
  - **IDA*** (búsqueda A* iterativa en profundidad) es una versión de A* iterativa en profundidad y, por lo tanto, aborda el problema de la complejidad espacial.
  - **RBFS** (búsqueda de best-first recursiva) y **SMA*** (A* limitado a la memoria simplificado) son algoritmos de búsqueda robustos y óptimos que usan cantidades limitadas de memoria; con suficiente tiempo, pueden resolver problemas para los cuales A* se queda sin memoria.

  - **La búsqueda de haz (Beam search)** pone un límite en el tamaño de la frontera; eso la hace incompleta y subóptima, pero a menudo encuentra soluciones razonablemente buenas y se ejecuta más rápido que las búsquedas completas.
  - **La búsqueda A* ponderada (Weighted A*)** enfoca la búsqueda hacia un objetivo, expandiendo menos nodos, pero sacrificando la optimalidad.
- El rendimiento de los algoritmos de búsqueda heurística depende de la calidad de la función heurística. A veces se pueden construir buenas heurísticas volviendo flexible la definición del problema, almacenando costos de solución precalculados para subproblemas en una base de datos de patrones, definiendo puntos de referencia, o aprendiendo de la experiencia con la clase de problema.

Notas bibliográficas e históricas

El tema de la búsqueda en el espacio de estados se originó en los primeros años de la IA. El trabajo de Newell y Simon en el *Logic Theorist* (1957) y *GPS* (1961) condujo a establecer algoritmos de búsqueda como la herramienta principal para los investigadores de IA de la década de 1960, así como a establecer la resolución de problemas como la tarea canónica de la IA. El trabajo en investigación de operaciones realizado por Richard Bellman (1957) mostró la importancia de los costos de trayectoria aditivos en la simplificación de algoritmos de optimización. El texto de Nils Nilsson (1971) estableció el área sobre una base teórica sólida.

El 8-puzzle es un primo menor del 15-puzzle, cuya historia es detallada por Slocum y Sonneveld (2006). En 1880, el 15-puzzle atrajo amplia atención del público y de los matemáticos (Johnson y Story, 1879; Tait, 1880). Los editores de la *American Journal of Mathematics* declararon: "El '15' puzzle ha estado prominentemente ante el público estadounidense durante las últimas semanas, se puede decir con seguridad que ha captado la atención de nueve de cada diez personas de ambos sexos, de todas las edades y condiciones de la comunidad", mientras que el *Weekly News-Democrat* de Emporia, Kansas, escribió el 12 de marzo de 1880: "Literalmente se ha convertido en una epidemia en todo el país".

El famoso diseñador de juegos estadounidense Sam Loyd afirmó falazmente haber inventado el 15 puzzle (Loyd, 1959); en realidad fue inventado por Noyes Chapman, un administrador de correos en Canastota, Nueva York, a mediados de la década de 1870 (aunque se otorgó una patente genérica que cubría bloques deslizantes a Ernest Kinsey en 1878). Ratner y Warmuth (1986) mostraron que la versión general $n \times n$ del 15-puzzle pertenece a la clase de problemas NP-completos.

El Cubo de Rubik fue inventado en 1974 por Ernó Rubik, quien también descubrió un algoritmo para encontrar soluciones buenas, pero no óptimas. Korf (1997) encontró soluciones óptimas para algunas instancias de problemas aleatorios utilizando bases de datos de patrones y búsqueda IDA*. Rokicki *et al.* (2014) demostraron que cualquier instancia puede resolverse en 26 movimientos (si considera un giro de 180° como dos movimientos; 20, si cuenta como uno). La demostración consumió 35 años de CPU de cómputo; no conduce de inmediato a un algoritmo eficiente. Agostinelli *et al.* (2019) usaron aprendizaje por refuerzo, redes neuronales profundas y búsqueda de árboles de Monte Carlo para aprender un solucionador más eficiente para el Cubo de Rubik. No se garantiza encontrar una solución óptima en costos, pero lo hace casi el 60% del tiempo, y los tiempos típicos de solución son menos de un segundo.

Cada uno de los problemas de búsqueda del mundo real, enumerados en el capítulo, ha sido objeto de una buena cantidad de esfuerzo de investigación. Los métodos para seleccionar vuelos óptimos siguen siendo en su mayoría de propiedad privada, pero Carl de Marcken demostró mediante una reducción a problemas de decisión diofántica que la fijación de precios y restricciones

de billetes de avión se han vuelto tan complicados que el problema de seleccionar un vuelo óptimo es formalmente *indecidible* (Robinson, 2002). El problema del agente viajero (TSP, por sus siglas en inglés) es un problema combinatorio estándar en la ciencia informática teórica (Lawler *et al*., 1992). Karp (1972) demostró que el problema de decisión del TSP es NP-duro, pero se desarrollaron métodos de aproximación heurística efectivos (Lin y Kernighan, 1973). Arora (1998) ideó un esquema de aproximación completamente polinomial para TSP euclídeos. Los métodos de diseño de diseños VLSI son revisados por LaPaugh (2010), y muchos artículos de optimización de diseños aparecen en revistas de VLSI. La navegación robótica se discute en el Capítulo 26. La secuenciación de ensamblaje automático fue demostrada por primera vez por FREDDY (Michie, 1972); se presenta una revisión completa en (Bahubalendruni y Biswal, 2016).

Los algoritmos de búsqueda no informada son un tema central en la informática (Cormen *et al*., 2009) y la investigación de operaciones (Dreyfus, 1969). Moore (1959) formuló la búsqueda de amplitud para resolver laberintos. El método de programación dinámica (Bellman, 1957; Bellman y Dreyfus, 1962), que registra de modo sistemático soluciones para todos los subproblemas de longitudes crecientes, se puede ver como una forma de búsqueda de amplitud.

El algoritmo de Dijkstra en la forma en que se presenta comúnmente (Dijkstra, 1959) es aplicable a grafos finitos explícitos. Nilsson (1971) introdujo una versión del algoritmo de Dijkstra que llamó búsqueda de costo uniforme (porque el algoritmo "se extiende a lo largo de contornos de igual costo de camino") que permite grafos implícitamente definidos, infinitos. El trabajo de Nilsson también introdujo la idea de listas cerradas y abiertas, y el término "búsqueda de gráficos". El nombre de BEST-FIRST-SEARCH fue introducido en el *Manual de IA* (Barr y Feigenbaum, 1981). Los algoritmos de Floyd–Warshall (Floyd, 1962) y Bellman-Ford (Bellman, 1958; Ford, 1956) permiten costos de paso negativos (siempre que no haya ciclos negativos).

Una versión de la iterativa en profundidad diseñada para hacer un uso eficiente del reloj de ajedrez fue usada por primera vez por Slate y Atkin (1977) en el programa de juego CHESS 4.5. El algoritmo B de Martelli (1977) también incluye un aspecto de iterativa en profundidad. La técnica iterativa en profundidad fue introducida por Bertram Raphael (1976) y llegó a la vanguardia en el trabajo de Korf (1985a).

El uso de información heurística en la resolución de problemas aparece en un artículo temprano de Simon y Newell (1958), pero la frase "búsqueda heurística" y el uso de funciones heurísticas que estiman la distancia al objetivo aparecieron un poco más tarde (Newell y Ernst, 1965; Lin, 1965). Doran y Michie (1966) realizaron extensos estudios experimentales de búsqueda heurística. Aunque analizaron la longitud del camino y la "penetración" (la relación entre la longitud del camino y el número total de nodos examinados hasta el momento), parecen haber ignorado la información proporcionada por el costo del camino $g(n)$. El algoritmo A*, que incorpora el costo del camino actual en la búsqueda heurística, fue desarrollado por Hart, Nilsson y Raphael (1968). Dechter y Pearl (1985) estudiaron las condiciones bajo las cuales A* es óptimamente eficiente (en número de nodos expandidos).

El documento original de A* (Hart *et al*., 1968) introdujo la condición de consistencia en funciones heurísticas. La condición monótona fue introducida por Pohl (1977) como un reemplazo más simple, pero Pearl (1984) demostró que ambas eran equivalentes.

Pohl (1977) fue pionero en el estudio de la relación entre el error en las funciones heurísticas y la complejidad temporal de A*. Se obtuvieron resultados básicos para la búsqueda en forma de árbol con costos de acción unitarios y un solo estado objetivo (Pohl, 1977; Gaschnig, 1979; Huyn *et al*., 1980; Pearl, 1984) y con múltiples estados objetivo (Dinh *et al*., 2007). Korf y Reid (1998) mostraron cómo predecir el número exacto de nodos expandidos (no sólo una aproximación asintótica) en una variedad de dominios de problemas reales. El "factor de ramificación efectivo" fue propuesto por Nilsson (1971) como una medida empírica de eficiencia. Para la búsqueda de gráficos, Helmert y Röger (2008) señalaron que varios problemas bien conocidos contenían

exponencialmente muchos nodos en rutas de solución de costo óptimo, lo que implica complejidad temporal exponencial para A*.

Hay muchas variaciones en el algoritmo A*. Pohl (1970) introdujo la búsqueda ponderada A* y, más tarde, una versión dinámica (1973), donde el peso cambia a lo largo de la profundidad del árbol. Ebendt y Drechsler (2009) sintetizan los resultados y examinan algunas aplicaciones. Hatem y Ruml (2014) muestran una versión simplificada y mejorada de la búsqueda ponderada A* que es más fácil de implementar. Wilt y Ruml (2014) introducen la búsqueda rápida como una alternativa a la búsqueda voraz que se enfoca en minimizar el tiempo de búsqueda, y Wilt y Ruml (2016) muestran que las mejores heurísticas para la búsqueda satisfactoria son diferentes de las de la búsqueda óptima. Burns *et al.* (2012) dan algunos trucos de implementación para escribir código de búsqueda rápido, y Felner (2018) considera cómo cambia la implementación al usar una prueba de objetivo temprana.

Pohl (1971) introdujo la búsqueda bidireccional. Holte *et al*. (2016) describen la versión de búsqueda bidireccional garantizada para encontrarse en el medio, lo que la hace más ampliamente aplicable. Eckerle *et al*. (2017) describen el conjunto de pares de nodos seguramente expandidos, y muestran que ninguna búsqueda bidireccional puede ser óptimamente eficiente. El algoritmo NBS (Chen *et al*., 2017) hace uso explícito de una cola de pares de nodos.

Una combinación de A* bidireccional y puntos de referencia conocidos se utilizó para encontrar eficientemente rutas de manejo para el servicio de mapas en línea de Microsoft (Goldberg *et al*., 2006). Después de almacenar en caché un conjunto de caminos entre puntos de referencia, el algoritmo puede encontrar un camino de costo óptimo entre cualquier par de puntos en un grafo de 24 millones de puntos de Estados Unidos, buscando menos de 0.1% del grafo. Korf (1987) muestra cómo usar subobjetivos, macrooperadores y abstracción para lograr mejoras de velocidad notables sobre técnicas anteriores. Delling *et al*. (2009) describen cómo utilizar la búsqueda bidireccional, puntos de referencia, estructura jerárquica y otros trucos para encontrar rutas para conducir. Anderson *et al.* (2008) describen una técnica relacionada, llamada **búsqueda de grueso a fino**, que puede pensar cómo definir puntos de referencia en varios niveles jerárquicos de abstracción. Korf (1987) describe las condiciones bajo las cuales la búsqueda de grueso a fino proporciona una aceleración exponencial. Knoblock (1991) proporciona resultados experimentales y análisis para cuantificar las ventajas de la búsqueda jerárquica.

Búsqueda de grueso a fino

A* y otros algoritmos de búsqueda de espacio de estados están estrechamente relacionados con las técnicas de **ramificación y poda** que se usan ampliamente en la investigación de operaciones (Lawler y Wood, 1966; Rayward-Smith *et al*., 1996). Kumar y Kanal (1988) intentan una "gran unificación" de búsqueda heurística, programación dinámica y técnicas de ramificación y poda bajo el nombre de CDP: el "proceso de decisión compuesto".

Ramificación y poda

Debido a que la mayoría de las computadoras en la década de 1960 tenían sólo unos pocos miles de palabras de memoria principal, la búsqueda heurística limitada por memoria fue un tema de investigación temprano. El Traverser de Gráficos (Doran y Michie, 1966), uno de los primeros programas de búsqueda, se compromete con una acción después de buscar en el best-first hasta el límite de memoria. IDA* (Korf, 1985b) fue el primer algoritmo de búsqueda heurística limitado por longitud y óptimo usado ampliamente, y se han desarrollado una gran cantidad de variantes. Un análisis de la eficiencia de IDA* y de sus dificultades con heurísticas de valores reales aparece en Patrick *et al*. (1992).

La versión original de RBFS (Korf, 1993) es en realidad algo más complicada que el algoritmo mostrado en la Figura 3.22, que en realidad está más cerca de un algoritmo desarrollado independientemente llamado **expansión iterativa** o IE (Russell, 1992). RBFS usa un límite inferior además del límite superior; ambos algoritmos se comportan de manera idéntica con heurísticas admisibles, pero RBFS expande nodos en orden de best-first aun con una heurística inadmisible. La idea de llevar un registro de la mejor ruta alternativa apareció antes en la elegante

Expansión iterativa

implementación en Prolog de A* de Bratko (2009) y en el algoritmo DTA* (Russell y Wefald, 1991). Este último trabajo también discute espacios de estados meta y aprendizaje a nivel meta.

El algoritmo MA* apareció en Chakrabarti *et al*. (1989). SMA*, o MA* Simplificado, surgió de un intento de implementar MA* (Russell, 1992). Kaindl y Khorsand (1994) aplicaron SMA* para producir un algoritmo de búsqueda bidireccional sustancialmente más rápido que los algoritmos anteriores. Korf y Zhang (2000) describen un enfoque de dividir y conquistar, y Zhou y Hansen (2002) introducen la búsqueda de gráficos A* limitada por memoria y una estrategia para cambiar a la búsqueda de amplitud para aumentar la eficiencia de la memoria (Zhou y Hansen, 2006).

La idea de que las heurísticas admisibles pueden derivarse mediante la flexibilización del problema aparece en el trabajo seminal de Held y Karp (1970), quienes usaron la heurística del árbol de expansión mínima para resolver el TSP. (Consulte el Ejercicio 3.MSTR). La automatización del proceso de flexibilización fue implementada con éxito por Prieditis (1993). Hay una creciente literatura sobre la aplicación del aprendizaje automático para descubrir funciones heurísticas (Samadi *et al.,* 2008; Arfaee *et al.,* 2010; Thayer *et al*., 2011; Lelis *et al*., 2012).

El uso de bases de datos de patrones para derivar heurísticas admisibles se debe a Gasser (1995) y Culberson y Schaeffer (1996, 1998); las bases de datos de patrones disjuntos son descritas por Korf y Felner (2002); un método similar que usa patrones simbólicos se debe a Edelkamp (2009). Felner *et al*. (2007) muestran cómo comprimir bases de datos de patrones para ahorrar espacio. La interpretación probabilística de las heurísticas fue investigada por Pearl (1984) y Hansson y Mayer (1989).

Heuristics de Pearl (1984) y *Heuristic Search* de Edelkamp y Schrödl (2012) son libros de texto influyentes sobre búsqueda. Los artículos sobre nuevos algoritmos de búsqueda aparecen en *The International Symposium on Combinatorial Search (SoCS)* y the International Conference on Automated Planning and Scheduling (ICAPS), así como en conferencias de IA generales como AAAI e IJCAI, y revistas como *Artificial Intelligence* y *Journal of the ACM*.

CAPÍTULO 4

BÚSQUEDA EN ENTORNOS COMPLEJOS

En el que se amplían los supuestos simplificadores del capítulo anterior, para acercarnos al mundo real.

El Capítulo 3 abordó problemas en entornos por completo observables, deterministas, estáticos y conocidos donde la solución es una secuencia de acciones. En este capítulo, se amplían esas restricciones; comenzamos con el problema de encontrar un estado bueno sin preocuparse por el camino para llegar allí, cubriendo tanto estados discretos (Sección 4.1) como continuos (Sección 4.2). Luego, se amplían los supuestos de determinismo (Sección 4.3) y observabilidad (Sección 4.4). En un mundo no determinista, el agente necesitará un plan condicional ejecutar diferentes acciones dependiendo de lo que observe, por ejemplo, deteniéndose si la luz es roja y avanzando si es verde. Con la observabilidad parcial, el agente también necesitará llevar un seguimiento de los posibles estados en los que podría estar. Finalmente, la Sección 4.5 guía al agente a través de un espacio desconocido que debe aprender a medida que avanza, utilizando la **búsqueda online.**

4.1 Búsqueda local y problemas de optimización

En los problemas de búsqueda del Capítulo 3, queríamos encontrar caminos mediante espacio de búsqueda, como un camino de Arad a Bucarest. Sin embargo, a veces sólo nos importa el estado final, no el camino para llegar allí. En el problema de las 8 reinas (Figura 4.3), sólo buscamos una configuración final válida de 8 reinas (porque si conoces la configuración, es trivial reconstruir los pasos que la crearon). Esto también es cierto para muchas aplicaciones importantes como el diseño de circuitos integrados y el de plantas de fábrica, de programación de talleres, la programación automática, la optimización de redes de telecomunicaciones, la planificación de cultivos y la gestión de carteras.

Búsqueda local

Los algoritmos de **búsqueda local** operan buscando desde un estado inicial hacia estados vecinos, sin dar seguimiento de los caminos ni del conjunto de estados alcanzados. Eso significa que no son sistemáticos; es decir, que quizá nunca exploren una porción del espacio de búsqueda donde sí resida una solución. No obstante, tienen dos ventajas clave: (1) usan poca memoria y (2) suelen encontrar soluciones razonables en espacios de estados grandes o infinitos, para los cuales los algoritmos sistemáticos no son adecuados.

Problema de optimización
Función objetivo

Los algoritmos de búsqueda local también pueden resolver **problemas de optimización**, en los que el objetivo es encontrar el mejor estado, una **función objetivo**.

Imagen de espacio de estados
Máximo global
Ascenso de colinas

Para entender la búsqueda local, considere los estados de un problema dispuestos en una **imagen de espacio de estados**, como en la Figura 4.1. Cada punto (estado) en el paisaje tiene una "elevación", definida por el valor de la función objetivo. Si la elevación corresponde a una función objetivo, entonces el objetivo es encontrar el pico más alto, un **máximo global**, y llamamos al proceso **ascenso de colinas**. Si la elevación corresponde al costo,

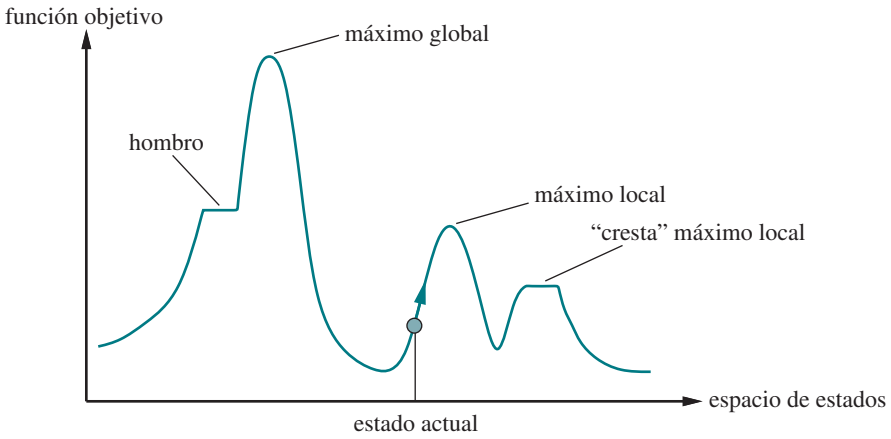


Figura 4.1 Una imagen de espacio de estados unidimensional en el que la elevación corresponde a la función objetivo. El objetivo es encontrar el máximo global.

function HILL-CLIMBING(*problem*) **returns** un estado que es un máximo local
  current ← *problem*.INITIAL
  while *true* **do**
    neighbor ← un estado sucesor de mayor valor que *current*
    if VALUE(*neighbor*) ≤ VALUE(*current*) **then return** *current*
    current ← *neighbor*

Figura 4.2 El algoritmo de búsqueda de ascenso, técnica de búsqueda local básica, en cada paso el nodo actual es reemplazado por el mejor vecino.

entonces el objetivo es encontrar el valle más bajo, un **mínimo global**, y llamarlo **descenso de gradiente**.

Mínimo global
Descenso de gradiente

4.1.1 Búsqueda mediante ascenso de colinas

El algoritmo de búsqueda de ascenso de colinas (vea la Figura 4.2) da seguimiento de un estado actual y, en cada iteración, se mueve al estado vecino con el valor más alto; es decir, se dirige en la dirección que da el **ascenso más empinado**. Termina cuando alcanza un "pico" donde ningún vecino tiene un valor más alto. El ascenso de colinas no mira más allá de los vecinos inmediatos del estado actual. Esto se asemeja a tratar de encontrar la cima del Monte Everest en una espesa niebla mientras se sufre de amnesia. Note que una forma de usar la búsqueda de ascenso de colinas es usar el negativo de una función de costo heurístico como la función objetivo; eso escalará localmente hacia el estado con la menor distancia heurística al objetivo.

Ascenso más empinado

Para ilustrar el ascenso de colinas, usaremos el problema de las **8 reinas** (Figura 4.3). Usaremos una **formulación de estado completo**, lo que significa que cada estado tiene todos los componentes de una solución, aunque es posible que no estén todos en el lugar correcto. En este caso, cada

Formulación de estado completo

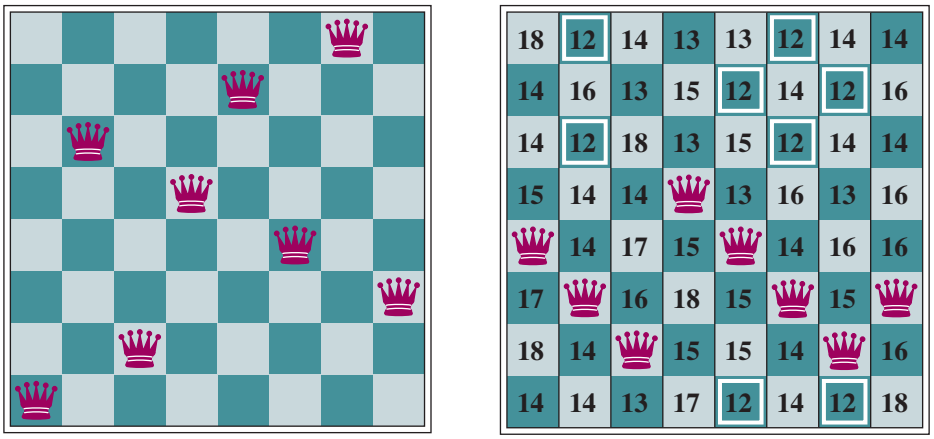


Figura 4.3 (a) Problema de las 8 reinas: coloque 8 reinas en un tablero de ajedrez de modo que ninguna ataque a otra. (Una reina ataca cualquier pieza en la misma fila, columna o diagonal). Esta posición es casi una solución, excepto por las dos reinas en la cuarta y séptima columna que se atacan entre sí a lo largo de la diagonal. (b) Estado de 8 reinas con estimación de costo heurístico $h = 17$. El tablero muestra el valor de h para cada posible sucesor obtenido al mover una reina dentro de su columna. Hay 8 movimientos empatados como mejores, con $h = 12$. El algoritmo de ascenso elegirá uno de ellos.

estado tiene 8 reinas en el tablero, una por columna. El estado inicial se elige al azar, y los sucesores de un estado son todos los estados posibles generados al mover una sola reina a otro cuadro en la misma columna (así cada estado tiene $8 \times 7 = 56$ sucesores). La función de costo heurístico h es el número de pares de reinas que se atacan entre sí; esto será cero sólo para las soluciones. (Cuenta como un ataque si dos piezas están en la misma línea, aun si hay una pieza intermedia entre ellas). La Figura 4.3(b) muestra un estado que tiene $h = 17$. La figura también muestra los valores de h de todos sus sucesores.

Búsqueda local voraz

A veces al ascenso de colinas se le llama **búsqueda local voraz** porque agarra un estado vecino bueno sin pensar en qué hacer después. Aunque la avaricia (voracidad) es uno de los siete pecados capitales, resulta que los algoritmos voraces suelen funcionar bastante bien. El ascenso de colinas puede progresar con rapidez hacia una solución porque en general es muy fácil mejorar un estado malo. Por ejemplo, desde el estado en la Figura 4.3(b), sólo se necesitan cinco pasos para llegar al estado en la Figura 4.3(a), que tiene $h = 1$ y es casi una solución. Desafortunadamente, el ascenso de colinas puede quedarse atascado por cualquiera de las siguientes razones:

Máximo local

- **Máximos locales:** Pico más alto que cada uno de sus estados vecinos, pero más bajo que el máximo global. Los algoritmos de ascenso de colinas que alcanzan las cercanías de un máximo local serán atraídos hacia arriba hacia el pico, pero luego se quedarán atrapados sin ningún otro lugar a dónde ir. La Figura 4.1 esquematiza el problema. Más en concreto, el estado en la Figura 4.3(a) es un máximo local (es decir, un mínimo local para el costo h); cada movimiento de una sola reina empeora la situación.

Cresta

- **Cresta:** Las crestas (vea una en la Figura 4.4) resultan en una secuencia de máximos locales que es muy difícil de navegar para los algoritmos voraces.

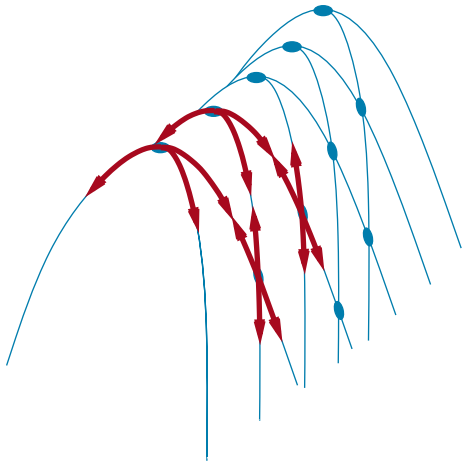


Figura 4.4 Ilustración de por qué las crestas causan dificultades para escalar colinas. La cuadrícula de estados (círculos oscuros) se superpone a una cresta que se eleva de izquierda a derecha y crea una secuencia de máximos locales no directamente conectados entre sí. A partir de cada máximo local, todas las acciones disponibles apuntan cuesta abajo. Topologías como esta son comunes en espacios de estados de baja dimensión, como puntos en un plano bidimensional. Pero en espacios de estados con cientos o miles de dimensiones, esta imagen intuitiva no se cumple y, por lo regular, hay al menos unas pocas dimensiones que hacen posible escapar de crestas y mesetas.

- **Meseta:** Área plana del paisaje del espacio de estados. Puede ser un máximo local plano, del que no existe salida cuesta arriba, o un **hombro**, desde el que es posible progresar. (Vea la Figura 4.1). Una búsqueda de ascenso de colina puede perderse vagando en la meseta.

Meseta
Hombro

En cada caso, el algoritmo llega a un punto en el que no se progresa. Si se comienza desde un estado de 8 reinas y se genera aleatoriamente, el ascenso más empinado se atasca 86 % del tiempo y resuelve sólo 14% de los casos de problemas. Por otro lado, funciona rápidamente, tomando sólo 4 pasos en promedio cuando tiene éxito y 3 si se atasca, no está mal para un espacio de estados con $8^8 \approx 17$ millones de estados.

¿Cómo resolveríamos más problemas? Podríamos seguir adelante al llegar a una meseta y permitir un **movimiento lateral** con la esperanza de que la meseta sí sea un hombro, como en la Figura 4.1. Pero si estamos realmente en un máximo local plano, entonces este enfoque vagará en la meseta para siempre. Por lo tanto, podemos limitar el número de movimientos laterales consecutivos, deteniéndonos después de, digamos, 100 movimientos. Esto aumenta el porcentaje de casos de problemas resueltos por el ascenso de colina de 14 % a 94 %. El éxito tiene un costo: el algoritmo promedia casi 21 pasos para cada instancia exitosa y 64 para cada fracaso.

Movimiento lateral

Se han inventado muchas variantes del ascenso de colina. El **ascenso de colina estocástica** elige al azar entre los movimientos cuesta arriba; la probabilidad de selección varía con la pendiente del movimiento cuesta arriba. Esto suele converger más lentamente que el ascenso más empinado, pero en algunos paisajes de estado, encuentra mejores soluciones. La **ascenso de colina de primera opción** implementa el ascenso de colina estocástica generando sucesores al azar hasta que se genera uno que sea mejor que el estado actual. Esta es una buena estrategia si un estado tiene muchos (por ejemplo, miles) de sucesores.

Ascenso de colina estocástica

Ascenso de colina de primera opción

Ascenso de colina con reinicio aleatorio

Otra variante es el **ascenso de colina con reinicio aleatorio**, que adopta el dicho: "Si al principio no tiene éxito, inténtelo e inténtelo de nuevo". Este algoritmo realiza una serie de búsquedas de escalado de colina a partir de estados iniciales generados aleatoriamente, hasta encontrar un objetivo. Es completa con una probabilidad de 1, porque eventualmente generará un estado objetivo como estado inicial. Si cada búsqueda de escalado de colina tiene una probabilidad p de éxito, entonces el número esperado de reinicios requeridos es $1/p$. Para instancias de 8 reinas sin movimientos laterales permitidos, $p \approx 0.14$, por lo que necesitamos cerca de 7 iteraciones para encontrar un objetivo (6 fracasos y 1 éxito). El número esperado de pasos es el costo de una iteración exitosa más $(1 - p)/p$ veces el costo del fracaso, o unos 22 pasos en total. Si permitimos movimientos laterales, se necesitan en promedio $1/0.94 \approx 1.06$ iteraciones y $(1 \times 21) + (0.06/0.94) \times 64 \approx 25$ pasos. Para 8 reinas, entonces, el escalado de colina con reinicio aleatorio es muy efectivo. Incluso para tres millones de reinas, el enfoque puede encontrar soluciones en segundos.[1]

El éxito del escalado de colina depende de la forma de la imagen del espacio de estados: si hay pocos máximos locales y mesetas, el escalado de colina con reinicio aleatorio encontrará rápido una buena solución. En cambio, muchos problemas reales tienen una imagen semejante más a una familia dispersa de puercoespines calvos en un piso plano, con mini puercoespines viviendo en la punta de cada aguja de puercoespín. Los problemas NP-duros (vea el Apéndice A) suelen tener un número exponencial de máximos locales en los que quedarse atascado. A pesar de ello, con frecuencia se encuentra un máximo local razonablemente bueno después de un pequeño número de reinicios.

4.1.2 Simulated annealing (recocido simulado)

Un algoritmo de escalado de colina que nunca realiza movimientos "cuesta abajo" hacia estados con un valor más bajo (o un costo más alto) siempre es vulnerable a quedar atrapado en un máximo local. En contraste, una caminata aleatoria pura que se mueve a un estado sucesor sin preocuparse por el valor eventualmente encontrará el máximo global, pero será en extremo ineficiente. Entonces, parece razonable intentar combinar el escalado de colina con una caminata aleatoria de modo que dé eficiencia y completitud.

Simulated annealing

El **Simulated annealing** es un algoritmo. En metalurgia, el **annealing** (recocido) es el proceso para templar o endurecer metales y vidrio al calentarlos a una temperatura alta y luego enfriándolos gradualmente, lo que permite que el material alcance un estado cristalino de baja energía. Para explicar el Simulated annealing (recocido simulado), cambiamos nuestro punto de vista del escalado de colina al **descenso de gradiente** (es decir, minimizando el costo) e imaginamos la tarea de hacer que una pelota de ping-pong caiga en la hendidura más profunda en una superficie muy accidentada. Si simplemente dejamos rodar la pelota, se detendrá en un mínimo local. Si agitamos la superficie, podemos hacer rebotar la pelota fuera del mínimo local, tal vez hacia un mínimo local más profundo, donde pasará más tiempo. El truco es agitar lo suficientemente fuerte (es decir, a una temperatura alta) y luego reducir gradualmente la intensidad de la agitación (es decir, bajar la temperatura).

La estructura general del algoritmo de Simulated annealing (Figura 4.5) es similar al escalado de colina. Sin embargo, en lugar de elegir el *mejor* movimiento, selecciona un movimiento *aleatorio*. Si el movimiento mejora la situación, siempre se acepta. De lo contrario, el algoritmo acepta el movimiento con alguna probabilidad menor que 1. La probabilidad disminuye exponencialmente con la "maldad" del movimiento, la cantidad ΔE por la cual se empeora la evaluación. La probabilidad también disminuye a medida que la "temperatura" T disminuye: los movimientos "malos" tienden más a ser permitidos al principio cuando T es alta, y se vuelven menos probables a medida que T disminuye. Si el *programa* reduce lentamente T a 0, entonces una propiedad

[1] Luby *et al.* (1993) sugieren reiniciar después de un número fijo de pasos y muestran que es *mucho* más eficiente que dejar que cada búsqueda sea infinita.

```
function SIMULATED-ANNEALING(problem, schedule) returns un estado de solución
    current ← problem.INITIAL
    for t = 1 to ∞ do
        T ← schedule(t)
        if T = 0 then return current
        next ← un sucesor seleccionado al azar de current
        ΔE ←VALUE(current) – VALUE(next)
        if ΔE > 0 then current ← next
        else current ← next sólo con probabilidad e^(−ΔE/T)
```

Figura 4.5 El algoritmo de recocido simulado, una versión de ascenso estocástico en la que se permiten algunos movimientos cuesta abajo. La entrada del *programa* determina el valor de la "temperatura" T en función del tiempo.

de la distribución de Boltzmann, $e^{\Delta E/T}$, es que toda la probabilidad se concentra en los máximos globales, que el algoritmo encontrará con una probabilidad que se aproxima a 1.

El recocido simulado se usó para resolver problemas de diseño de VLSI a partir de la década de 1980. Se ha aplicado con amplitud a programar fábricas y otras tareas de optimización a gran escala.

4.1.3 Búsqueda local en haz

Mantener sólo un nodo en memoria parecería una reacción extrema al problema de las limitaciones de memoria. El algoritmo de **búsqueda local en haz** lleva un seguimiento de k estados en lugar de sólo uno. Comienza con k estados generados aleatoriamente. En cada paso, se generan todos los sucesores de los k estados. Si alguno es un objetivo, el algoritmo se detiene; de lo contrario, selecciona los k mejores sucesores de la lista completa y repite.

Búsqueda local en haz

A primera vista, una búsqueda local en haz con k estados parecería no ser más que ejecutar k reinicios aleatorios en paralelo, en lugar de en secuencia. De hecho, los dos algoritmos son bastante diferentes. En una búsqueda de reinicio aleatorio, cada proceso de búsqueda se ejecuta de forma independiente de los demás. *En una búsqueda local en haz, se pasa información útil entre los hilos de búsqueda paralelos*. En efecto, los estados que generan los mejores sucesores le dicen a los demás: "¡Venga aquí, el césped está más verde!" El algoritmo rápido abandona las búsquedas infructuosas y mueve sus recursos a donde hace más progreso.

La búsqueda local en haz puede carecer de diversidad entre los k estados, pueden agruparse en una pequeña región del espacio de estados, lo que hace que la búsqueda sea poco más que una versión k veces más lenta del escalado de colina. Una variante llamada **búsqueda de haz estocástica**, análoga al escalado de colina estocástico, ayuda a aliviar este problema. En lugar de elegir los mejores k sucesores, la búsqueda de haz estocástica elige sucesores con una probabilidad proporcional al valor del sucesor, lo que aumenta la diversidad.

Búsqueda de haz estocástica

4.1.4 Algoritmos evolutivos

Los **algoritmos evolutivos** pueden verse como variantes de la búsqueda de haz estocástica explícitamente motivados por la metáfora de la selección natural en biología: hay una población de individuos (estados), en la que los individuos más aptos (mayor valor) producen descendencia (estados sucesores) que pueblan la siguiente generación, un proceso llamado **recombinación**. Hay formas infinitas de algoritmos evolutivos, variando como sigue:

Algoritmos evolutivos

Recombinación

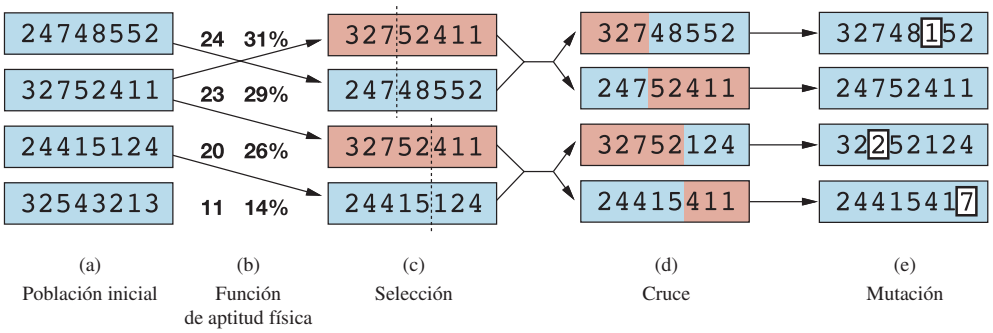


Figura 4.6 Un algoritmo genético, ilustrado para cadenas de dígitos que representan estados de 8 reinas. La población inicial en (a) se clasifica mediante una función de aptitud en (b), lo que da como resultados pares para aparearse en (c). Producen descendencia en (d), que está sujeta a mutación en (e).

Algoritmo genético

Estrategia de evolución

Programación genética

- Tamaño de la población.
- Representación de cada individuo. En **algoritmos genéticos**, cada individuo es una cadena sobre un alfabeto finito (a menudo, una cadena booleana), al igual que el ADN es una cadena sobre el alfabeto **ACGT**. En **estrategias de evolución**, un individuo es una secuencia de números reales, y en **programación genética** un individuo es un programa de computadora.
- Número de cruces, ρ, que es el número de padres que se unen para crear descendencia. El caso más común es $\rho = 2$: dos padres combinan sus "genes" (partes de su representación) para crear descendencia. Cuando $\rho = 1$ tenemos búsqueda de haz estocástica (que se ver como reproducción asexual). Es posible tener $\rho > 2$, lo cual ocurre sólo raramente en la naturaleza, pero es lo suficientemente fácil de simular en computadoras.

Selección

- Proceso de **selección** para elegir los individuos que se convertirán en los padres de la próxima generación: una posibilidad es seleccionar de todos los individuos con probabilidad proporcional a su puntaje de aptitud; otra es seleccionar al azar n individuos ($n > \rho$), y luego seleccionar los ρ más aptos como padres.

Punto de cruce

- Procedimiento de recombinación. Un enfoque común (asumiendo $\rho = 2$) es seleccionar aleatoriamente un **punto de cruce** para dividir cada una de las cadenas parentales, y recombinar las partes para formar dos hijos; uno con la primera parte del padre 1 y la segunda parte del padre 2, otro, con la segunda parte del padre 1 y la primera parte del padre 2.

Tasa de mutación

- **Tasa de mutación**, que determina con qué frecuencia los descendientes tienen mutaciones aleatorias en su representación. Una vez que se ha generado un descendiente, cada bit en su composición se invierte con una probabilidad igual a la tasa de mutación.
- Composición de la próxima generación. Puede ser sólo la descendencia recién creada o puede incluir algunos padres con los puntajes más altos de la generación anterior (una práctica llamada **elitismo**, que garantiza que la aptitud general nunca disminuirá con el tiempo). La práctica de la **eliminación**, en la que se descartan todos los individuos por debajo de cierto umbral, puede conducir a una aceleración (Baum *et al.*, 1995).

Elitismo

La Figura 4.6(a) muestra una población de cuatro cadenas de 8 dígitos, cada una representa un estado del *rompecabezas* de las 8 reinas: el d-ésimo dígito representa el número de fila de la reina en la columna c. En (b), cada estado es calificado por la función de aptitud. Los valores de aptitud

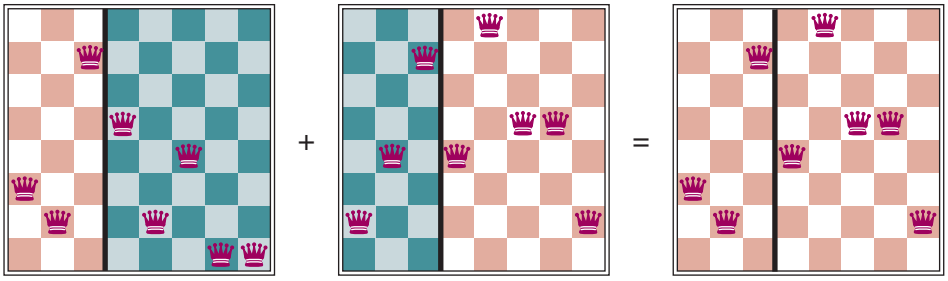


Figura 4.7 Los estados de 8 reinas corresponden a los dos primeros padres en la Figura 4.6(c) y a la primera descendencia en la Figura 4.6(d). Las columnas verdes se pierden en el paso de cruce, las rojas se conservan. (Para interpretar los números de la Figura 4.6: la fila 1 es la fila inferior y la 8 es la fila superior).

más altos son mejores, así que para el problema de las 8 reinas usamos el número de pares de reinas *que no se atacan*, que tiene un valor de $8 \times 7/2 = 28$ para una solución. Los valores de los cuatro estados en (b) son 24, 23, 20 y 11. Los puntajes de aptitud se normalizan a probabilidades, y los valores resultantes se muestran junto a los valores de aptitud en (b).

En (c) se seleccionan dos pares de padres, según las probabilidades en (b). Note que un individuo es seleccionado dos veces y otro no. Para cada par seleccionado, se elige aleatoriamente un punto de cruce (línea punteada). En (d) cruzamos las cadenas de los padres en los puntos de cruce, obteniendo nuevas crías. Por ejemplo, el primer hijo del primer par obtiene los primeros tres dígitos (327) del primer padre, los dígitos restantes (48552) del segundo padre. Los estados de 8 reinas involucrados en este paso de recombinación se muestran en la Figura 4.7.

Finalmente, en (e) cada ubicación en cada cadena está sujeta a mutación aleatoria con una pequeña probabilidad independiente. Un dígito fue mutado en la primera, tercera y cuarta cría. En el problema de las 8 reinas, esto corresponde a elegir una reina al azar y moverla a un cuadrado aleatorio en su columna. A menudo sucede que la población es diversa al principio del proceso, por lo que el cruce da pasos grandes con frecuencia en el espacio de estados al inicio del proceso de búsqueda (como en el recocido simulado). Después de muchas generaciones de selección hacia una mayor aptitud, la población se vuelve menos diversa y son típicos los pasos más pequeños. La Figura 4.8 describe un algoritmo que implementa todos estos pasos.

Los algoritmos genéticos son similares a la búsqueda de haz estocástica, pero con la adición de la operación de cruce. Esto es ventajoso si hay bloques que realizan funciones útiles. Por ejemplo, podría ser que poner las primeras tres reinas en las posiciones 2, 4 y 6 (donde no se atacan entre sí) constituya un bloque útil que se puede combinar con otros bloques útiles que aparecen en otros individuos para construir una solución. Se puede demostrar matemáticamente que, si los bloques no sirven para nada, por ejemplo, si las posiciones del código genético se permutan al azar, entonces el cruce no ofrece ninguna ventaja.

La teoría de los algoritmos genéticos explica cómo funcionan estos usando la idea de un **esquema**, que es una subcadena en la que algunas posiciones pueden quedar sin especificar. Por ejemplo, el esquema 246***** describe todos los estados de las 8 reinas en los que las primeras tres reinas están en las posiciones 2, 4 y 6, respectivamente. Las cadenas que coinciden con el esquema (como 24613578) se llaman **instancias** del esquema. Se puede demostrar que si la aptitud promedio de las instancias de un esquema está por encima de la media, entonces el número de instancias del esquema crecerá con el tiempo.

Esquema

Instancia

Evolución y búsqueda

La teoría de la evolución fue desarrollada por Charles Darwin en *El origen de las especies por medio de la selección natural* (1859) e independientemente por Alfred Russel Wallace (1858). La idea central es simple: las variaciones ocurren en la reproducción y se conservarán en generaciones sucesivas casi en proporción a su efecto en la aptitud reproductiva.

La teoría de Darwin se desarrolló sin conocimiento de cómo se pueden heredar y modificar los rasgos de los organismos. Las leyes probabilísticas que rigen estos procesos fueron identificadas por Gregor Mendel (1866), un monje que experimentaba con chícharos. Más tarde, Watson y Crick (1953) identificaron la estructura de la molécula de ADN y su alfabeto, AGTC (adenina, guanina, timina, citosina). En el modelo estándar, la variación ocurre por mutaciones puntuales en la secuencia de letras y por "cruce" (en el que el ADN de una descendencia se genera combinando largas secciones de ADN de cada padre).

La analogía con los algoritmos de búsqueda local ya ha sido descrita; la diferencia principal entre la búsqueda de vigas estocástica y la evolución es el uso de la reproducción *sexual*, donde los sucesores se generan a partir de *múltiples* individuos en vez de sólo uno. Sin embargo, los mecanismos reales de la evolución son más ricos de lo que permite la mayoría de los algoritmos genéticos. Por ejemplo, las mutaciones pueden implicar inversiones, duplicaciones y movimiento de grandes fragmentos de ADN; algunos virus toman ADN de un organismo y lo insertan en otro; además, existen genes transponibles que sólo se copian muchas miles de veces dentro del genoma.

Aun, existen genes que envenenan las células de posibles compañeros que no llevan el gen, esto incrementa sus posibilidades de replicación. Lo más importante es que *los propios genes codifican los mecanismos* mediante los cuales el genoma se reproduce y se traduce en un organismo. En los algoritmos genéticos, esos mecanismos son un programa separado que no está representado dentro de las cadenas que se manipulan.

La evolución darwiniana parecería ineficiente, luego de generar a ciegas casi 10^{43} organismos sin mejorar sus heurísticas de búsqueda ni un ápice. Pero el aprendizaje sí juega un papel en la evolución. Aunque el gran naturalista francés Jean Lamarck (1809) estaba equivocado al proponer que los rasgos adquiridos por adaptación durante la vida de un organismo se transmitirían a su descendencia, la teoría superficialmente similar de James Baldwin (1896) es correcta: el aprendizaje puede relajar eficazmente el paisaje de aptitud, lo que lleva a una aceleración en la tasa de evolución. Un organismo que tiene un rasgo no del todo adaptativo para su entorno transmitirá el rasgo si también tiene la suficiente plasticidad para aprender a adaptarse al entorno de una manera benéfica. Las simulaciones por computadora (Hinton y Nowlan, 1987) confirman que este **efecto Baldwin** es real y que una consecuencia es que las cosas que son difíciles de aprender terminan en el genoma, pero las cosas que son fáciles de aprender no necesariamente deben residir allí (Morgan y Griffiths, 2015).

```
function GENETIC-ALGORITHM(population,fitness) returns un individuo
  repeat
    weights ← WEIGHTED-BY(population,fitness) population2←empty list
    for i = 1 to SIZE(population) do
      parent1, parent2 ←WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child ← REPRODUCE(parent1, parent2)
      if (Pequeña probabilidad aleatoria) then child ←MUTATE(child)
      agregar child to population2
    population← population2
  until algún individuo es lo suficientemente apto o ha transcurrido suficiente tiempo
  return el mejor individuo en population, de acuerdo a fitness

function REPRODUCE(parent1, parent2) returns un individuo
  n ←LENGTH(parent1)
  c ← número aleatorio de 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Figura 4.8 Un algoritmo genético. Dentro de la función, la *población* (*population*) es una lista ordenada de individuos, los *pesos* (*weights*) son una lista de valores de aptitud correspondientes para cada individuo y la *aptitud* (*fitness*) es una función para calcular estos valores.

Claramente, este efecto es poco probable que sea significativo si los bits adyacentes no están relacionados entre sí, porque entonces habrá pocos bloques contiguos que proporcionen un beneficio consistente. Los algoritmos genéticos funcionan mejor si los esquemas corresponden a componentes significativos de una solución. Por ejemplo, si la cadena es representación de una antena, entonces los esquemas pueden representar componentes de ésta, como reflectores y deflectores. Quizás un buen componente sea bueno en una variedad de diseños diferentes. Esto sugiere que el uso exitoso de algoritmos genéticos requiere una ingeniería cuidadosa de la representación.

En la práctica, los algoritmos genéticos tienen su lugar dentro del amplio panorama de los métodos de optimización (Marler y Arora, 2004), en especial para problemas estructurados complejos como el diseño de circuitos o la programación de talleres, y más recientemente para evolucionar la arquitectura de redes neuronales profundas (Miikkulainen *et al.*, 2019). No está claro cuánto de lo llamativo de los algoritmos genéticos surge de su superioridad en tareas específicas y cuánto del atractivo metafórico de la evolución.

4.2 Búsqueda local en espacios continuos

En el Capítulo 2, explicamos la distinción entre entornos discretos y continuos, señalamos que la mayoría de entornos del mundo real son continuos. Un espacio de acción continuo tiene un factor de ramificación infinito y, por lo tanto, no puede ser manejado por la mayoría de los algoritmos revisados hasta ahora (excepto, el ascenso de colina de primera opción y el recocido simulado).

Esta sección introduce de manera *muy breve* a algunas técnicas de búsqueda local para espacios continuos. La literatura sobre el tema es vasta; muchas de las técnicas básicas se originaron en el

siglo XVII, después del desarrollo del cálculo por Newton y Leibniz.[2] Encontramos usos para estas técnicas en varias secciones del libro, incluidos los capítulos sobre aprendizaje, visión y robótica.

Iniciamos con un ejemplo. Suponga que desea ubicar tres nuevos aeropuertos en cualquier lugar de Rumania, de modo que la suma de las distancias al cuadrado en línea recta desde cada ciudad en el mapa hasta su aeropuerto más cercano se minimice. (Vea el mapa de Rumania en la Figura 3.1). El espacio de estados está definido por las coordenadas de los tres aeropuertos: (x_1, y_1), (x_2, y_2) y (x_3, y_3). Este es un espacio de *seis dimensiones*; también se dice que los estados están definidos por seis **variables**. En general, los estados están definidos por un vector n-dimensional de variables, $\mathbf{x}$. Moverse en este espacio corresponde a mover uno o más de los aeropuertos en el mapa. La función objetivo $f(\mathbf{x}) = f(x_1, y_1, x_2, y_2, x_3, y_3)$ es relativamente fácil de calcular para cualquier estado particular una vez que calculamos las ciudades más cercanas. Sea C_i el conjunto de ciudades cuyo aeropuerto más cercano (en el estado $\mathbf{x}$) es el aeropuerto i. Así, tenemos:

Variable

$$f(\mathbf{x}) = f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_{i=1}^{3} \sum_{c \in C_i} (x_i - x_c)^2 + (y_i - y_c)^2 \tag{4.1}$$

Esta ecuación es correcta no sólo para el estado $\mathbf{x}$ sino también para estados en el vecindario local de $\mathbf{x}$. Sin embargo, no es correcta globalmente; si nos alejamos demasiado de $\mathbf{x}$ (alterando la ubicación de uno o más de los aeropuertos en gran medida), entonces el conjunto de ciudades más cercanas para ese aeropuerto cambia, y necesitamos recalcular C_i.

Discretización

Una forma de tratar con un espacio de estado continuo es **discretizarlo**. Así, en vez de permitir que las ubicaciones (x_i, y_i) sean cualquier punto en el espacio bidimensional continuo, podríamos limitarlos a puntos fijos en una cuadrícula rectangular con un espaciado de tamaño δ (delta). Así, en lugar de tener un número infinito de sucesores, cada estado en el espacio tendría sólo 12 sucesores, correspondientes a incrementar una de las 6 variables en ±δ. Luego, podemos aplicar cualquier algoritmo de búsqueda local a este espacio discreto. Alternativamente, podríamos hacer que el factor de ramificación sea finito muestreando estados sucesores al azar, moviéndonos en una dirección aleatoria en una pequeña cantidad, δ. Los métodos que miden el progreso por el cambio en el valor de la función objetivo entre dos puntos cercanos se llaman métodos **empíricos de gradiente**. La búsqueda empírica de gradientes es lo mismo que el ascenso de colina más empinada en una versión discretizada del espacio de estado. Reducir el valor de δ con el tiempo puede dar una solución más precisa, que no necesariamente converge a un óptimo global en el límite.

Gradiente empírico

Con frecuencia tenemos una función objetivo expresada en forma matemática tal que podemos usar cálculo para resolver el problema analíticamente, no empíricamente. Muchos métodos intentan usar el **gradiente** del paisaje para encontrar un máximo. El gradiente de la función objetivo es un vector ∇f que da la magnitud y dirección de la pendiente más pronunciada. Para nuestro problema, tenemos:

Gradiente

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3}\right)$$

En algunos casos, podemos encontrar un máximo resolviendo la ecuación $\nabla f = 0$. (Esto podría hacerse, por ejemplo, si estuviéramos situando sólo un aeropuerto; la solución es la media aritmética de todas las coordenadas de las ciudades). Sin embargo, en muchos casos, esta ecuación no se puede resolver en forma cerrada. Por ejemplo, con tres aeropuertos, la expresión para el

[2] El conocimiento de vectores, matrices y derivadas es útil para esta sección (consulte el Apéndice A).

gradiente depende de qué ciudades están más cerca de cada aeropuerto en el estado actual. Esto significa que podemos calcular el gradiente localmente (pero no globalmente); por ejemplo,

$$\frac{\partial f}{\partial x_1} = 2\sum_{c \in C_1} (x_i - x_c) \tag{4.2}$$

Dada una expresión localmente correcta para el gradiente, podemos realizar el ascenso más pronunciado actualizando el estado actual según la fórmula

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha \nabla f(\mathbf{x}),$$

donde α (alfa) es una pequeña constante a menudo llamada **tamaño de paso**. Existe una gran variedad de métodos para ajustar α. El problema básico es que, si α es demasiado pequeño, se necesitan demasiados pasos; si α es demasiado grande, la búsqueda podría sobrepasar el máximo. La técnica de **búsqueda online** intenta superar este dilema extendiendo la dirección del gradiente actual, en general duplicando α repetidamente, hasta que f comience a disminuir nuevamente. El punto en el que esto ocurre se convierte en el nuevo estado actual. Hay varias corrientes de pensamiento sobre cómo se debe elegir la nueva dirección en este punto.

Tamaño de paso

Búsqueda online

Para muchos problemas, el algoritmo más efectivo es el venerable método de **Newton-Raphson**. Este es una técnica general para encontrar raíces de funciones, es decir, resolver ecuaciones de la forma $g(x) = 0$. Funciona calculando una nueva estimación para la raíz x según la fórmula de Newton:

Newton-Raphson

$$x \leftarrow x - g(x)/g'(x).$$

Para encontrar un máximo o mínimo de f, necesitamos encontrar $\mathbf{x}$ tal que el *gradiente* sea un vector cero (es decir, $\nabla f(\mathbf{x}) = \mathbf{0}$). Por lo tanto, $g(x)$ en la fórmula de Newton se convierte en $\nabla f(\mathbf{x})$, y la ecuación de actualización se escribe en forma de matriz-vector como:

$$\mathbf{x} \leftarrow \mathbf{x} - \mathbf{H}_f^{-1}(\mathbf{x}) \nabla f(\mathbf{x}),$$

donde $\mathbf{H}_f(\mathbf{x})$ es la matriz **Hessiana** de segundas derivadas, cuyos elementos H_{ij} son dados por $\partial^2 f / \partial x_i \partial x_j$. Para nuestro ejemplo de aeropuertos, podemos ver desde la Ecuación (4.2) que $\mathbf{H}_f(\mathbf{x})$ es en particular simple: los elementos fuera de la diagonal son cero y los elementos diagonales para el aeropuerto i son sólo dos veces el número de ciudades en C_i. Un cálculo rápido muestra que un paso de la actualización mueve al aeropuerto i directamente al centroide de C_i, que es el mínimo de la expresión local para f de la Ecuación (4.1).[3] No obstante, para problemas de alta dimensionalidad, calcular las n^2 entradas de la Hessiana e invertirla puede ser costoso, por lo que se han desarrollado muchas versiones aproximadas del método de Newton-Raphson.

Hessiano

Los métodos de búsqueda local sufren de máximos locales, crestas y mesetas en espacios de estado continuos y en espacios discretos. Los reinicios aleatorios y el recocido simulado suelen ser útiles. Pero, los espacios continuos de alta dimensión son lugares grandes donde es fácil perderse.

Un tema final es la **optimización con restricciones**. Un problema de optimización está restringido si las soluciones deben satisfacer restricciones estrictas en los valores de las variables. Por ejemplo, en nuestro problema de ubicación de aeropuertos, podríamos restringir los sitios a estar dentro de Rumania y en tierra firme (y no en medio de lagos). La dificultad de los problemas de optimización con restricciones depende de la naturaleza de las restricciones y la función objetivo. La categoría más conocida es la de problemas de **programación lineal**, en los que las restricciones deben ser

Optimización con restricciones

Programación lineal

[3] En general, se puede considerar que la actualización de Newton-Raphson ajusta una superficie cuadrática a f en $\mathbf{x}$ y luego se mueve directamente al mínimo de esa superficie, que también es el mínimo de f si f es cuadrática.

Conjunto convexo

desigualdades lineales que formen un **conjunto convexo**[4] y la función objetivo también sea lineal. La complejidad temporal de la programación lineal es polinomial en el número de variables.

Optimización convexa

La programación lineal es quizás el método más estudiado y útil para la optimización. Es un caso especial del problema más general de **optimización convexa**, que permite que la región de restricción sea cualquier región convexa y que el objetivo sea cualquier función que sea convexa dentro de la región de restricción. Bajo ciertas condiciones, los problemas de optimización convexa también permiten resolverse de manera polinómica y pueden ser factibles en la práctica con miles de variables. Varios problemas importantes en aprendizaje automático y teoría de control pueden formularse como problemas de optimización convexa (ver Capítulo 20).

4.3 Búsqueda con acciones no deterministas

En el Capítulo 3 asumimos un entorno por completo observable, determinista y conocido. Por lo tanto, un agente puede observar el estado inicial, calcular una secuencia de acciones que alcancen el objetivo y ejecutar las acciones con "ojos cerrados", sin necesidad de usar sus percepciones.

Sin embargo, si el entorno es parcialmente observable, el agente no sabe con certeza en qué estado se encuentra; y si el entorno es no determinista, el agente no sabe a qué estado cambiará después de tomar una acción. Esto significa que en vez de pensar "Estoy en el estado s_1 y si hago la acción a terminaré en el estado s_2", un agente ahora pensará "Estoy en el estado s_1 o s_3, y si hago la acción a terminaré en el estado s_2, s_4 o s_5". Llamamos **estado de creencia** a un conjunto de estados físicos que el agente cree posibles**.**

Estado de creencia

En entornos parcialmente observables y no deterministas, la solución a un problema ya no es una secuencia, sino un **plan condicional** (a veces llamado plan de contingencia o estrategia) que especifica qué hacer dependiendo de qué percepciones reciba el agente mientras ejecuta el plan. Examinamos el no determinismo en esta sección y la observabilidad parcial en la siguiente.

Plan condicional

4.3.1 El mundo de la aspiradora errática

El mundo de la aspiradora del Capítulo 2 tiene ocho estados (vea la Figura 4.9). Hay tres acciones: *Right, Left, y Suck*, y el objetivo es limpiar toda suciedad (estados 7 y 8). Si el entorno es determinista y por completo observable y conocido, entonces el problema es fácil de resolver con cualquier algoritmo del Capítulo 3; la solución es una secuencia de acciones. Así, si el estado inicial es 1, entonces la secuencia de acciones [*Suck, Right, Suck*] alcanzará un estado objetivo 8.

Suponga que introducimos el no determinismo en forma de aspiradora potente, pero errática. En el **mundo de la aspiradora errática**, la acción *Aspirar* (*Suck*) funciona así:

- Si se aplica a un cuadrado sucio, la acción limpia el cuadrado y, a veces, limpia la suciedad en un cuadrado adyacente también.
- Al aplicarse a un cuadrado limpio, la acción, a veces, deposita suciedad en la alfombra.[5]

Para formular de manera precisa este problema, necesitamos generalizar la noción de un **modelo de transición** del Capítulo 3. En vez de definir éste mediante una función RESULT que devuelve

[4] Un conjunto de puntos S es convexo si la línea que une dos puntos cualesquiera en S también está contenida en S. Una **función convexa** es aquella para la cual el espacio "encima" forma un conjunto convexo; por definición, las funciones convexas no tienen mínimos locales (a diferencia de los globales).

[5] Suponemos que la mayoría de lectores enfrentan problemas similares y pueden simpatizar con nuestro agente. Nos disculpamos con los propietarios de aparatos de limpieza modernos y eficientes que no puedan aprovechar este recurso pedagógico.

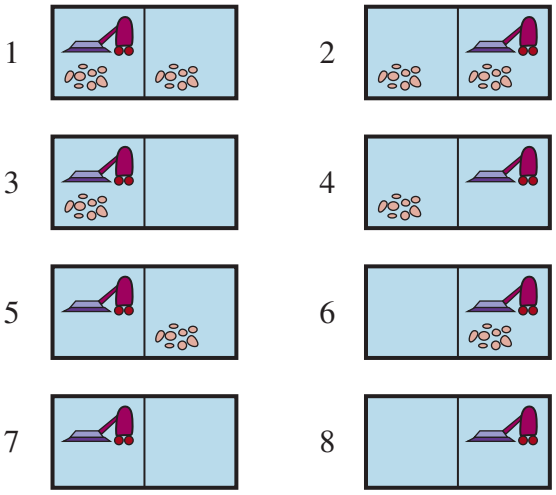


Figura 4.9 Ocho estados posibles del mundo de la aspiradora; 7 y 8 son estados objetivo.

un único estado de resultado, usamos una función RESULT que devuelve un conjunto de posibles estados de resultado. Por ejemplo, en el mundo de la aspiradora errática, la acción *Aspirar* en el estado 1 limpia sólo la ubicación actual, o ambas ubicaciones:

RESULTS(1, *Suck*) = {5, 7}

Si comenzamos en el estado 1, ninguna *secuencia* única de acciones resuelve el problema, pero el siguiente **plan condicional** sí lo hace:

[*Suck*, **if** *State* = 5 **then** [*Right*, *Suck*] **else** [ ]]. (4.3)

Vemos que un plan condicional puede contener pasos de tipo **if–then–else**; esto significa que las soluciones son *árboles*, no secuencias. Aquí, la condición en la instrucción **if** verifica qué estado actual es; esto es algo que el agente podrá observar en tiempo de ejecución, pero no sabe al momento de la planificación. Alternativamente, podríamos haber tenido una formulación que prueba la percepción en lugar del estado. Muchos problemas en el mundo real y físico son problemas de contingencia, porque la predicción exacta del futuro es imposible. Por esta razón, muchas personas mantienen los ojos abiertos mientras caminan.

4.3.2 Árboles de búsqueda AND–OR

¿Cómo hallamos estas soluciones contingentes a problemas no deterministas? Como en el Capítulo 3, primero construimos árboles de búsqueda, que aquí tienen un carácter diferente. En un entorno determinista, la única ramificación se introduce por las elecciones propias del agente en cada estado: puedo hacer esta acción o aquella. Llamamos a estos **nodos OR**. En el mundo de la aspiradora, por ejemplo, en un nodo OR, el agente elige *Left* o *Right* o *Suck*. En un entorno no determinista, la ramificación también se introduce por la elección del *entorno* de resultado para cada acción. Llamamos a estos **nodos AND**. Por ejemplo, la acción *Aspirar* en el estado 1 resulta en el estado de creencia {5, 7}, por ello el agente tendría que encontrar un plan para el estado 5 *Y* para el estado 7. Ambos tipos de nodos se alternan, llevan a un **árbol AND–OR**, como en la Figura 4.10.

Nodo OR

Nodo AND

Árbol AND-OR

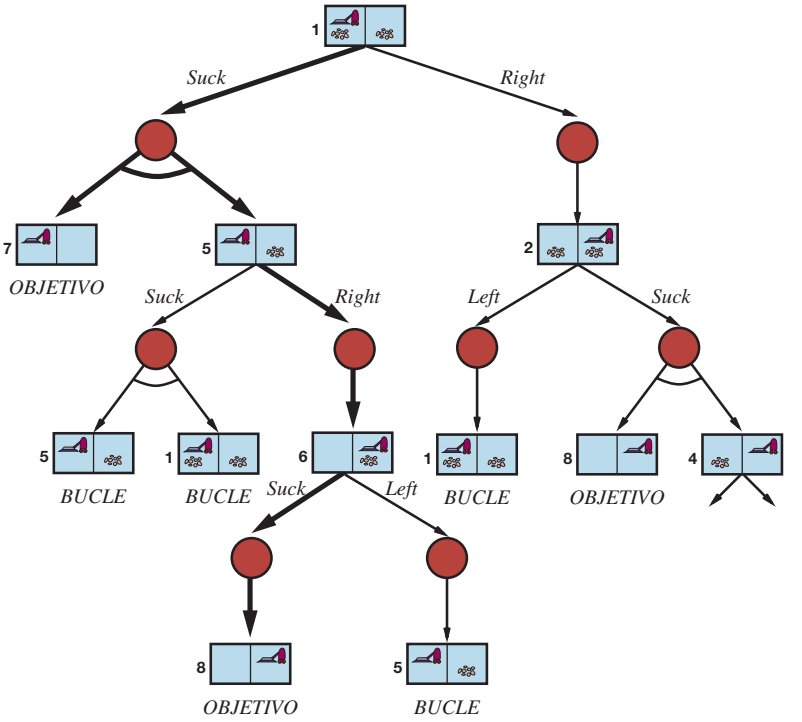


Figura 4.10 Los dos primeros niveles del árbol de búsqueda del mundo de la aspiradora errática. Los nodos de estado son nodos OR donde se debe elegir alguna acción. En los nodos AND, circulados, se deben manejar todos los resultados, como lo indica el arco que une las ramas salientes. La solución hallada está en líneas en negrita.

Una solución para un problema de búsqueda AND-OR es un subárbol del árbol de búsqueda completo que (1) tiene un nodo objetivo en cada hoja, (2) especifica una acción en cada uno de sus nodos OR, y (3) incluye cada rama de resultado en cada uno de sus nodos AND. La solución se muestra en líneas negritas en la figura y corresponde al plan de la Ecuación (4.3).

La Figura 4.11 muestra un algoritmo recursivo de búsqueda de grafos AND-OR en profundidad; un aspecto clave de éste es la forma en que maneja los ciclos, que con frecuencia surgen en problemas no deterministas (por ejemplo, si una acción a veces no tiene efecto o si se puede corregir un efecto no deseado). Si el estado actual es idéntico a un estado en el camino desde la raíz, entonces regresa con un fallo. Esto no significa que *no haya* solución desde el estado actual, sólo que si *hay* una solución no cíclica, debe ser alcanzable desde la encarnación anterior del estado actual, por ello la nueva encarnación se puede descartar. Con esta comprobación aseguramos que el algoritmo termine en cada espacio de estado finito, porque cada camino debe llegar a un objetivo, un callejón sin salida o un estado repetido. Note que el algoritmo no verifica si el estado actual es una repetición de un estado en algún *otro* camino desde la raíz, lo cual es importante para la eficiencia.

Los grafos AND-OR pueden ser explorados en anchura o en best-first. El concepto de una función heurística debe ser modificado para estimar el costo de una solución contingente en vez de una secuencia, pero la noción de admisibilidad se mantiene y hay un análogo del algoritmo A* para encontrar soluciones óptimas. (Vea las Notas bibliográficas e históricas al final del capítulo).

```
function AND-OR-SEARCH(problem) returns un plan condicional o failure
    return OR-SEARCH(problem, problem.INITIAL, [ ])

function OR-SEARCH(problem, state, path) returns a conditional plan o failure
    if problem.IS-GOAL(state) then return el plan vacío
    if IS-CYCLE(path) then return failure
    for each action in problem.ACTIONS(state) do
        plan ← AND-SEARCH(problem, RESULTS(state, action), [state] + path])
        if plan ≠ failure then return [action] + plan]
    return failure

function AND-SEARCH(problem, states, path) returns a conditional plan o failure
    for each s_i in states do
        plan_i ←OR-SEARCH(problem, s_i, path)
        if plan_i = failure then return failure
    return [if s_1 then plan_1 else if s_2 then plan_2 else . . . if s_{n−1} then plan_{n−1} else plan_n]
```

Figura 4.11 Algoritmo grafos de búsqueda AND-OR generados por entornos no deterministas. Una solución es un plan condicional que considera cada resultado no determinista y elabora un plan para cada uno.

4.3.3 Intentar e intentar de nuevo

Considere un mundo de aspiradora *resbaladizo*, idéntico al mundo de aspiradora ordinario (no errático) excepto que las acciones de movimiento a veces fallan y dejan al agente en la misma ubicación. Por ejemplo, moverse a la Derecha (*Right*) en el estado 1 lleva al estado de creencia {1, 2}. La Figura 4.12 muestra parte del grafo de búsqueda; claramente, ya no hay soluciones acíclicas desde el estado 1, y la BÚSQUEDA AND-OR retornaría con un fallo. Pero existe una **solución cíclica**, intentar seguir hacia la *Derecha* hasta que funcione; que podemos expresar con una nueva construcción **while**:

Solución cíclica

[*Suck*, **while** *State* = 5 **do** *Right*, *Suck*]

o agregando una **etiqueta** para denotar alguna parte del plan y referirse a ella más tarde:

[*Suck*, L_1 : *Right*, **if** *State* = 5 **then** L_1 else *Suck*].

¿Cuándo un plan cíclico es una solución? Una condición mínima es que cada hoja sea un estado objetivo y que una hoja sea alcanzable desde cada punto en el plan. Además, necesitamos considerar la causa del no determinismo. Si realmente el mecanismo de conducción del robot aspirador funciona a veces, pero resbala al azar e de modo independiente otras ocasiones, entonces el agente puede estar seguro de que si la acción se repite suficientes veces, a la larga funcionará y el plan tendrá éxito. Pero si el no determinismo se debe a algún hecho no observado sobre el robot o el entorno, quizás una correa de transmisión se ha roto y el robot nunca se moverá, entonces repetir la acción no ayudará.

Una forma de entender esta decisión es decir que la formulación inicial del problema (totalmente observable, no determinista) se abandona por una formulación diferente (parcialmente observable, determinista) donde el fracaso del plan cíclico se atribuye a una propiedad no observada de la correa de transmisión. En el Capítulo 12 discutimos cómo decidir cuál de varias posibilidades inciertas es más probable.

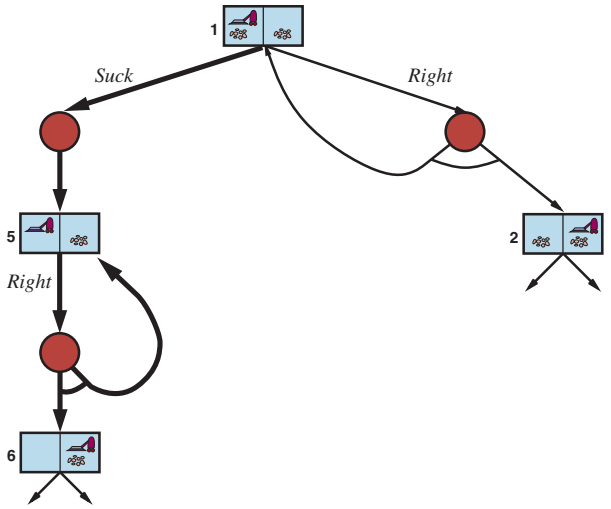


Figura 4.12 Parte del grafos de búsqueda de un mundo de aspiradora resbaladizo, donde hemos mostrado (algunos) ciclos explícitamente. Todas las soluciones a este problema son planes cíclicos porque no hay forma de avanzar de manera confiable.

4.4 Búsqueda en entornos parcialmente observables

Ahora nos dirigimos al problema de la observabilidad parcial, donde las percepciones del agente no son suficientes para determinar exactamente el estado. Eso significa que algunas acciones del agente estarán dirigidas a reducir la incertidumbre sobre el estado actual.

4.4.1 Búsqueda sin observaciones

Sin sensor
De conformidad

Si las percepciones del agente *no dan ninguna información*, tenemos lo que se llama un problema **sin sensor** (o **conformant problem**). Primero, podría pensar que el agente sin sensor no tiene esperanza de resolver un problema si no tiene idea en qué estado comienza, pero las soluciones sin sensor son muy comunes y útiles, en principio porque *no* dependen de sensores que funcionen correctamente. En los sistemas de fabricación, por ejemplo, se han desarrollado métodos ingeniosos para orientar las piezas con corrección desde una posición inicial desconocida mediante una secuencia de acciones sin ningún tipo de detección. A veces, un plan sin sensor es mejor aun cuando está disponible un plan condicional con detección. Por ejemplo, los médicos recetan antibióticos de amplio espectro en vez de usar el plan condicional de realizar un análisis de sangre, esperar los resultados y recetar un algo específico. El plan sin sensor ahorra tiempo y dinero, y evita el riesgo de que la infección empeore antes de disponer de los resultados del análisis.

Considere una versión sin sensor del mundo de la aspiradora (determinista). Suponga que el agente conoce la geografía de su mundo, pero no su propia ubicación o la distribución de la suciedad. En ese caso, su estado de creencia inicial es {1, 2, 3, 4, 5, 6, 7, 8} (vea la Figura 4.9). Ahora, si el agente se mueve a la *Derecha*, estará en uno de los estados {2, 4, 6, 8}, ¡el agente obtuvo información sin percibir nada! Luego de [*Right*,*Suck*] el agente siempre terminará en uno de los estados {4, 8}. Al final, después de [*Right,Suck,Left,Suck*] el agente está garantizado de alcanzar el estado objetivo 7, sin importar cuál sea el estado inicial. Decimos que el agente puede **forzar** al mundo al estado 7.

Forzar

La solución a un problema sin sensor es una secuencia de acciones, no un plan condicional (porque no hay percepción). Buscamos en el espacio de estados de creencia en lugar de estados físicos.[6] En el espacio de estados de creencia, el problema es *por completo* observable pues el agente siempre conoce su propio estado de creencia. Además, la solución (si existe) para un problema sin sensor siempre es una secuencia de acciones. Esto se debe a que, igual que en los problemas ordinarios del Capítulo 3, las percepciones recibidas luego de cada acción son por completo predecibles, ¡siempre están vacíos! Por ello, no hay contingencias para planificar. Esto es cierto *aun si el entorno es no determinista*.

Podríamos introducir nuevos algoritmos para problemas de búsqueda sin sensor. Pero, en cambio, podemos usar los algoritmos del Capítulo 3 si transformamos el problema físico subyacente en un problema de estado de creencia, en el que buscamos en estados de creencia, no en estados físicos. El problema original, P, tiene componentes $Actions_P$, $Result_P$, etc.; el problema de estado de creencia tiene estos componentes:

- **Estados:** El espacio de estados de creencia contiene cada subconjunto posible de los estados físicos. Si P tiene N estados, entonces el problema de estado de creencia tiene 2^N estados de creencia, aunque muchos de esos pueden ser inalcanzables desde el estado inicial.
- **Estado inicial:** En general, el estado de creencia que consiste en todos los estados en P, aunque en algunos casos el agente tendrá más conocimiento que esto.
- **Acciones:** Esto es complicado. Suponga que el agente está en el estado de creencia $b = \{s_1, s_2\}$, pero $\text{ACTIONS}_P(s_1) \neq \text{ACTIONS}_P(s_2)$; entonces el agente no está seguro de qué acciones son legales. Si asumimos que las acciones ilegales no tienen efecto en el entorno, entonces es seguro tomar la *unión* de todas las acciones en cualquiera de los estados físicos en el estado de creencia actual b:

  $$\text{ACTIONS}(b) = \bigcup_{s \in b} \text{ACTIONS}_P(s).$$

  Además, si una acción ilegal pudiera provocar una catástrofe, es más seguro permitir sólo la *intersección*, es decir, el conjunto de acciones legales en *todos* los estados. Para el mundo de la aspiradora, cada estado tiene las mismas acciones legales, por ello ambos métodos dan el mismo resultado.
- **Modelo de transición:** Para acciones deterministas, el nuevo estado de creencia tiene un estado de resultado para cada uno de los estados posibles actuales (aunque algunos estados de resultado fueren iguales):

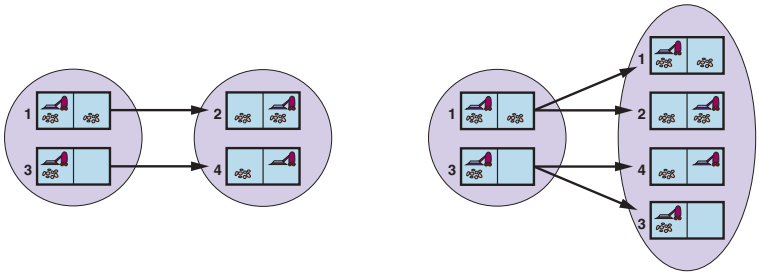
  $$b' = \text{RESULT}(b, a) = \{s' : s' = \text{RESULT}_P(s, a) \text{ and } s \in b\}. \tag{4.4}$$

  Con el no determinismo, el nuevo estado de creencia consiste en todos los posibles resultados de aplicar la acción a cualquiera estado en el estado de creencia actual:

  $$\begin{aligned} b' = \text{RESULT}(b, a) &= \{s' : s' \in \text{RESULTS}_P(s, a) \text{ and } s \in b\} \\ &= \bigcup_{s \in b} \text{RESULTS}_P(s,a), \end{aligned}$$

  El tamaño de b' será el mismo o más pequeño que b para acciones deterministas, pero puede ser más grande que b con acciones no deterministas (vea la Figura 4.13).
- **Prueba de objetivo:** El agente *posiblemente* logra el objetivo si *algún* estado s en el estado de creencia satisface la prueba de objetivo del problema subyacente, $\text{IS-GOALP}(s)$. El agente *necesariamente* logra el objetivo si *cada* estado satisface $\text{IS-GOALP}(s)$. Nuestro objetivo es lograr necesariamente el objetivo.
- **Costo de la acción:** Esto también es complicado. Si la misma acción puede tener diferentes costos en diferentes estados, entonces el costo de tomar una acción en un estado de creencia dado

[6] En un entorno por completo observable, cada estado de creencia contiene un estado físico. Así, podemos ver los algoritmos del Capítulo 3 como una búsqueda en un espacio de estados de creencias de estados de creencias únicos.



(a) (b)

Figura 4.13 (a) Predecir el siguiente estado de creencia para el mundo de la aspiradora sin sensores con la acción determinista, *Right*. (b) Predicción para el mismo estado de creencia y acción en la versión resbaladiza del mundo del vacío sin sensores.

podría ser uno de varios valores. (Esto da lugar a una nueva clase de problemas, que exploramos en el Ejercicio 4.MVAL). Por ahora, asumimos que el costo de una acción es el mismo en todos los estados y, por lo tanto, se puede transferir directamente del problema físico subyacente.

La Figura 4.14 muestra el espacio de estados de creencia alcanzables para el mundo de la aspiradora determinista y sin sensores. Sólo hay 12 estados de creencia alcanzables de un total de $2^8 = 256$ posibles estados de creencia.

Las definiciones anteriores permiten la construcción automática de la formulación del problema de estados de creencia a partir de la definición del problema físico subyacente. Realizado esto, resolveremos problemas sin sensores con cualquier algoritmo de búsqueda ordinario del Capítulo 3.

En la búsqueda de grafos simples, los estados recién alcanzados se prueban para ver si ya antes fueron alcanzados. Esto funciona para los estados de creencia; por ejemplo, en la Figura 4.14, la secuencia de acciones [*Suck,Left,Suck*] comenzando en el estado inicial alcanza el mismo estado de creencia que [*Right,Left,Suck*], es decir, {5, 7}. Ahora, considere el estado de creencia alcanzado por [*Left*], es decir, {1, 3, 5, 7}. Obvio, esto no es idéntico a {5, 7}, pero es un *superconjunto*. Descartamos (podar) cualquier estado de creencia de superconjunto. ¿Por qué? Porque una solución desde {1, 3, 5, 7} debe ser para cada uno de los estados individuales 1, 3, 5 y 7 y, por ello, es una solución para cualquier combinación de estos estados individuales, como {5, 7}; así, no necesitamos intentar resolver {1, 3, 5, 7}, nos concentramos en resolver el estado de creencia en rigor más fácil {5, 7}.

Al contrario, si {1, 3, 5, 7} ya ha sido generado y se ha visto que es solucionable, entonces cualquier *subconjunto*, como {5, 7}, está garantizado de ser solucionable. (Si tengo una solución que funciona cuando estoy confundido acerca de en qué estado me estoy, seguirá funcionando cuando esté menos confundido). Este nivel extra de poda puede mejorar en extremo la eficiencia de la resolución de problemas sin sensores.

Pero, aun con esta mejora, la resolución de problemas sin sensores tal como la hemos descrito rara vez es factible en la práctica. Un problema es la inmensidad del espacio de estados de creencia: vimos en el capítulo previo que a menudo un espacio de búsqueda de tamaño N es demasiado grande, y ahora tenemos espacios de búsqueda de tamaño 2^N. Además, cada elemento del espacio de búsqueda es un conjunto de hasta N elementos. Para N grande, no podremos representar ni siquiera un solo estado de creencia sin quedarnos sin espacio de memoria.

Una solución es representar el estado de creencia con una descripción más compacta. Podríamos decir que el agente sabe "Nada" en el estado inicial; después de moverse a la izquierda

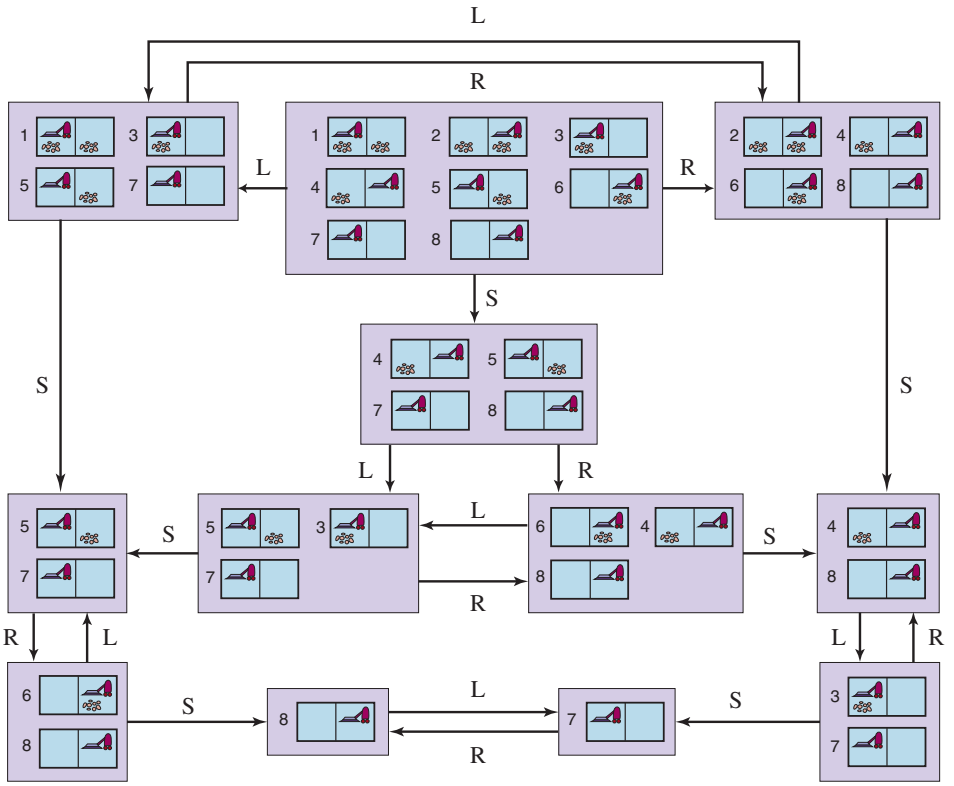


Figura 4.14 Porción accesible del espacio de estados de creencias para el mundo de la aspiradora determinista y sin sensores. Cada rectángulo corresponde a un único estado de creencia. En todo momento dado, el agente tiene un estado de creencia, pero no sabe en qué estado físico está. El estado de creencia inicial (ignorancia total) es el cuadro central superior.

(*Left*), podríamos decir, "No está en la columna extrema derecha", etc. El Capítulo 7 explica cómo hacer esto a manera de representación formal.

Otra aproximación consiste en evitar algoritmos de búsqueda estándar, que tratan los estados de creencia como cajas negras igual que cualquier otro estado del problema. En cambio, podemos mirar *dentro* de los estados de creencia y desarrollar **algoritmos de búsqueda de estados de creencia incrementales** que construyan la solución de un estado físico a la vez. Por ejemplo, en el mundo de la aspiradora sin sensor, el estado de creencia inicial es {1, 2, 3, 4, 5, 6, 7, 8}, y debemos encontrar una secuencia de acciones que funcione en los 8 estados. Lo haremos si hallamos primero una solución que funcione para el estado 1; luego comprobamos si funciona para el estado 2; si no, retrocedemos para hallar una solución diferente para el estado 1, y así sucesivamente.

Algoritmos de búsqueda de estados de creencia incrementales

Así como una búsqueda AND–OR debe hallar una solución para cada rama en un nodo AND, este algoritmo debe hallar una solución para cada estado en el estado de creencia; la diferencia es que la búsqueda AND–OR puede encontrar una solución diferente para cada rama, mientras que una búsqueda de estado de creencia incremental debe hallar *una* solución que funcione para *todos* los estados.

La principal ventaja del enfoque incremental es que suele ser capaz de detectar rápido el fracaso: si un estado de creencia no tiene solución, ocurre que un pequeño subconjunto del estado de

creencia, que consiste en los primeros estados examinados, también es no soluble. A veces, esto conduce a una aceleración proporcional al tamaño de los estados de creencia, que pueden ser tan grandes como el espacio de estados físicos en sí mismo.

4.4.2 Búsqueda en entornos parcialmente observables

Muchos problemas no pueden resolverse sin sensores. Por ejemplo, el 8-puzzle sin sensores es imposible. Además, algo de detección puede ayudar mucho: podemos resolver 8-puzzle si podemos ver sólo el cuadrado de la esquina superior izquierda. La solución implica mover cada ficha sucesivamente al cuadro observable y llevar un registro de su ubicación a partir de ahí.

Para un problema parcialmente observable, la especificación del problema especificará una función PERCEPT(s) que devuelve el percepto recibido por el agente en un estado dado. Si la percepción es no determinista, entonces podemos usar una función PERCEPT que devuelve un conjunto de percepciones posibles. Para problemas por completo observables, PERCEPT(s) = s para cada estado s, y para problemas sin sensores PERCEPT(s) = *null*.

Piense un mundo de aspiradora con detección local, donde el agente tiene un sensor de posición que aporta el percepto L en el cuadrado izquierdo, y R en el cuadrado derecho, y un sensor de suciedad que proporciona *Dirty* cuando el cuadrado actual está sucio y *Clean* si está limpio. Así, el PERCEPTO en el estado 1 es [*L,Dirty*]. Con observación parcial, sucederá que varios estados producirán el mismo percepto; el estado 3 también producirá [*L,Dirty*]. Además, dado este percepto inicial, el estado de creencia inicial será {1, 3}. Podemos pensar en el modelo de transición entre estados de creencia para problemas parcialmente observables como si ocurrieran en tres etapas, como se ve en la Figura 4.15:

- La etapa de **predicción** calcula el estado de creencia resultante de la acción, RESULT(b,a), tal como lo hicimos con problemas sin sensores. Para enfatizar que esta es una predicción, usamos la notación $\hat{b}$ = RESULT(b,a), donde el "acento circunflejo" sobre la b significa "estimado"; también usamos PREDICT(b,a) como sinónimo de RESULT(b,a).
- La etapa de **percepciones posibles** calcula el conjunto de percepciones que podrían observarse en el estado de creencia predicho (usando la letra o para la observación):

  POSIBLES-PERCEPTS($\hat{b}$) = {o : o = PERCEPT(s) y $s \in \hat{b}$ }.

- La etapa de **actualización** calcula el estado de creencia que resultaría de cada percepción posible. El estado de creencia actualizado b_o es el conjunto de estados $\hat{b}$ en que podrían haber producido la percepción:

  b_o = UPDATE($\hat{b}$, o) = {s : o = PERCEPT(s) y $s \in \hat{b}$ }.

  El agente necesita lidiar con las percepciones *posibles* al planificar, porque no conocerá las percepciones *reales* hasta que ejecute el plan. Note que el no determinismo en el entorno físico puede aumentar el estado de creencia en la etapa de predicción, pero cada estado de creencia actualizado b_o no puede ser más grande que el estado de creencia predicho $\hat{b}$; las observaciones sólo pueden ayudar a reducir la incertidumbre. Además, para la percepción determinista, los estados de creencia para las diferentes percepciones posibles serán disyuntos, formando una *partición* del estado de creencia predicho original.

Al juntar estas tres etapas, obtenemos los posibles estados de creencia resultantes de una acción dada y las percepciones posibles subsiguientes:

$$\text{RESULTS}(b,a) = \{b_o : b_o = \text{UPDATE}(\text{PREDICT}(b, a), o) \text{ and } o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}. \quad (4.5)$$

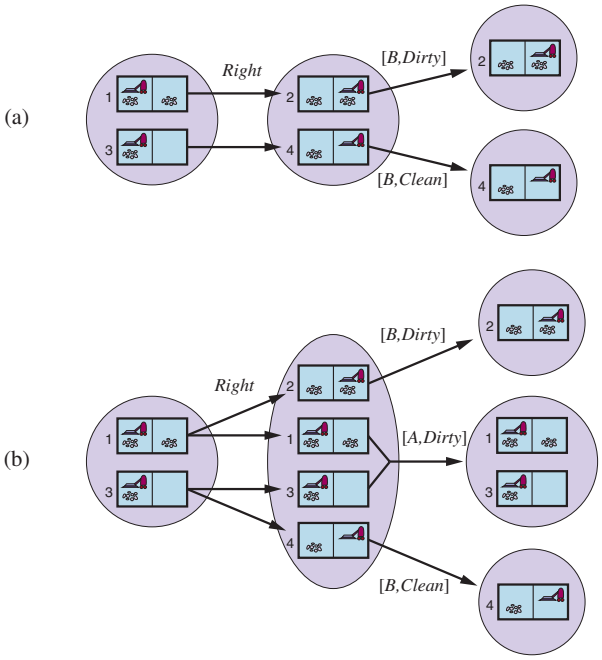


Figura 4.15 Dos ejemplos de transiciones en mundos de aspiradoras con detección local. (a) En el mundo determinista, *Right* se aplica en el estado de creencia inicial, lo que resulta en un nuevo estado de creencia predicho con dos posibles estados físicos; para éstos, las percepciones posibles son [*R, Dirty*] y [*R, Clean*], lo que lleva a dos estados de creencia, cada cual un singleton. (b) En el mundo resbaladizo, *Right* se aplica en el estado de creencia inicial, dando un nuevo estado de creencia con cuatro estados físicos; para esos estados, las percepciones posibles son [*L, Dirty*], [*R, Dirty*], y [*R, Clean*], lo que lleva a tres estados de creencia, como se aquí se muestra.

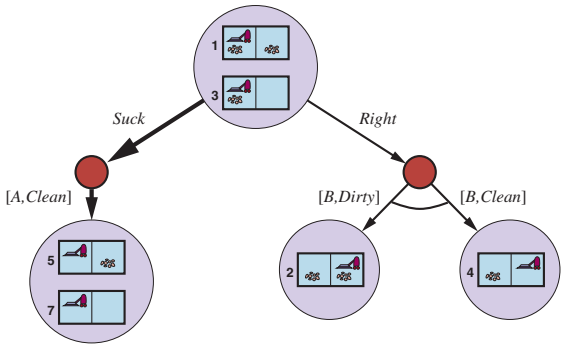


Figura 4.16 El primer nivel del árbol de búsqueda AND-OR para un problema en el mundo de la aspiradora con detección local; *Suck* es la primera acción de la solución.

4.4.3 Resolución de problemas parcialmente observables

La sección previa mostró cómo derivar la función RESULTS para un problema de estados de creencias no determinista de un problema físico subyacente, dada la función PERCEPT. Con esta formulación, el algoritmo de búsqueda AND–OR de la Figura 4.11 se aplicaría directamente para derivar una solución. La Figura 4.16 muestra parte del árbol de búsqueda para el mundo de la aspiradora de detección local, suponiendo un percepto inicial [*A,Dirty*]. La solución es el plan condicional

[*Suck*, *Right*, **if** *Bstate* ={6} **then** *Suck* **else** [ ]] .

Note que, como suministramos un problema de creencias al algoritmo de búsqueda AND–OR, devolvió un plan condicional que prueba el estado de creencias en lugar del estado real. Esto es como debe ser: en un entorno parcialmente observable el agente no sabrá el estado real.

Al igual que en el caso de los algoritmos de búsqueda estándar aplicados a problemas sin sensor, el algoritmo de búsqueda AND–OR trata los estados de creencias como cajas negras, al igual que cualquier otro estado. Podemos mejorar esto mediante la comprobación de estados de creencias ya generados que son subconjuntos o superconjuntos del estado actual, al igual que para problemas sin sensor. También se pueden derivar algoritmos de búsqueda incrementales, análogos a los descritos para problemas sin sensor, que dan mejoras sustanciales sobre el enfoque de caja negra.

4.4.4 Un agente para entornos parcialmente observables

Un agente para entornos parcialmente observables formula un problema, llama un algoritmo de búsqueda (como AND-OR-SEARCH) para resolverlo y ejecuta la solución. Hay dos diferencias principales entre este agente y el de entornos por completo observables y deterministas. Primero, la solución será un plan condicional en vez de una secuencia; para ejecutar una expresión if–then–else, el agente deberá probar la condición y ejecutar la rama apropiada del condicional. Segundo, el agente deberá mantener su estado de creencias mientras realiza acciones y recibe percepciones. Este proceso se asemeja al proceso de predicción-observación-actualización de la Ecuación (4.5), aunque es más simple porque la percepción es dada por el entorno en vez de ser calculada por el agente. Dado un estado de creencias inicial b, una acción a y una percepción o, el nuevo estado de creencias es:

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o). \tag{4.6}$$

Considere un mundo de aspiradora de *parvulario* donde los agentes sólo perciben el estado de su cuadrado actual, y cualquier cuadrado se ensucia en todo momento a menos que el agente lo esté limpiando en ese momento.[7] La Figura 4.17 muestra el estado de creencias de este entorno.

Monitoreo
Filtrado
Estimación de estado

En entornos parcialmente observables, que incluyen la gran mayoría de entornos del mundo real, mantener el estado de creencias es función fundamental de todo sistema inteligente. Esta función tiene varios nombres, como **monitoreo**, **filtrado** y **estimación de estado**. La Ecuación (4.6) se llama estimador de estado recursivo porque calcula el nuevo estado de creencias a partir del previo, en vez de examinar toda la secuencia de percepciones. Si el agente no quiere "quedarse atrás", el cálculo tiene que ocurrir tan rápido como llegan las percepciones. A medida que el entorno se vuelve más complejo, el agente sólo tendrá tiempo para calcular un estado de creencias aproximado, quizá centrándose en las implicaciones de la percepción para los

[7] Una vez más, nuestras disculpas para quienes no están familiarizados con el efecto de los párvulos en su entorno.

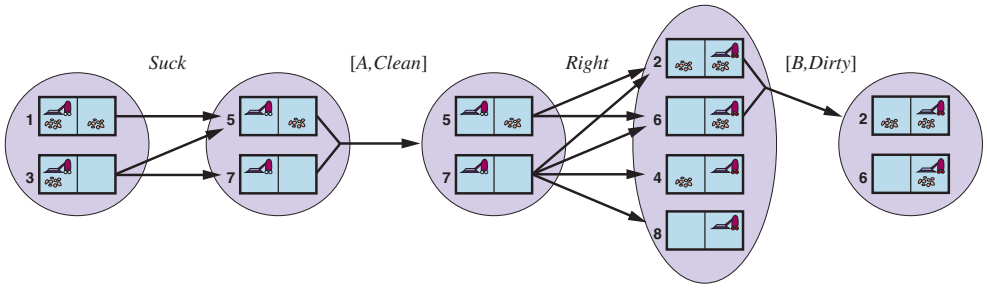


Figura 4.17 Dos ciclos de predicción-actualización del mantenimiento del estado de creencias en el mundo de la aspiradora del parvulario con detección local.

aspectos del entorno de interés actual. La mayor parte del trabajo en este problema se ha realizado para entornos estocásticos y de estado continuo con las herramientas de la teoría de la probabilidad, como se explica en el Capítulo 14.

En esta sección ejemplificaremos un entorno discreto con sensores deterministas y acciones no deterministas. Se trata de un robot con una tarea particular de estimación de estado llamada **localización**: determinar dónde se encuentra, dado un mapa del mundo y una secuencia de percepciones y acciones. Nuestro robot se coloca en el entorno tipo laberinto de la Figura 4.18. El robot está equipado con cuatro sensores de sonar que indican si hay un obstáculo —la pared exterior o un cuadrado sombreado en la figura— en cada una de las cuatro direcciones cardinales. La percepción está en forma de un vector de bits, un bit para cada una de las direcciones norte, este, sur y oeste, en ese orden, donde 1011 significa que hay obstáculos al norte, sur y oeste, pero no al este.

Localización

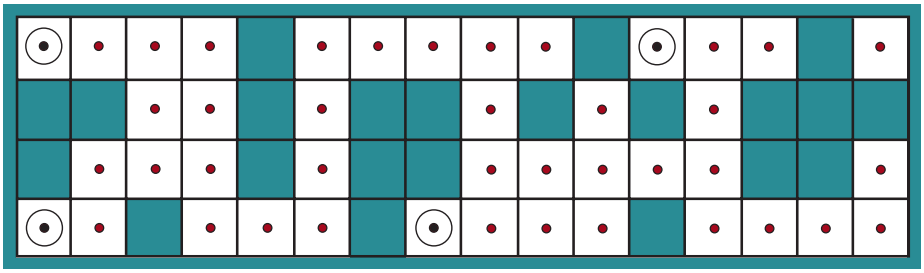
Suponga que los sensores proporcionan datos del todo correctos y que el robot tiene un mapa correcto del entorno. Pero, por desgracia, el sistema de navegación del robot falla, por ello cuando ejecuta una acción de giro a la *derecha*, se mueve aleatoriamente a uno de los cuadrados adyacentes. La tarea del robot es determinar su ubicación actual.

Imagine que el robot acaba de encenderse y no sabe dónde está —su estado de creencia inicial b consiste en el conjunto de todas las ubicaciones. Luego, el robot recibe la percepción 1011 y se actualiza usando la ecuación b_o = UPDATE(1011), resultando las 4 ubicaciones de la Figura 4.18(a). Puede inspeccionar el laberinto para ver que esas son las únicas cuatro ubicaciones que resultan de la percepción 1011.

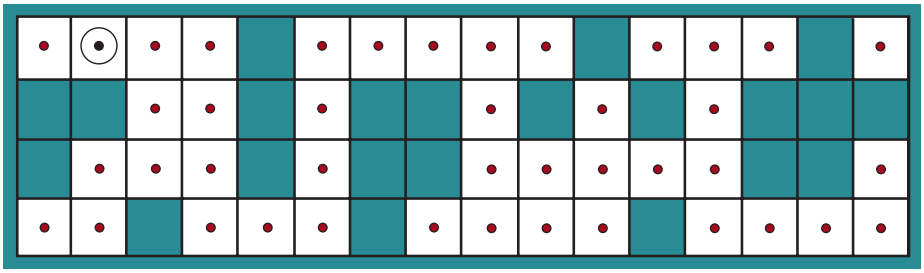
Después, el robot ejecuta una acción de giro a la *derecha*, pero el resultado es no determinista. El nuevo estado de creencia, b_a = PREDICT(b_o,*Right*), contiene todas las ubicaciones que están a sólo un paso de las ubicaciones en b_o. Cuando llega la segunda percepción, 1010, el robot realiza UPDATE(b_a, 1010) y descubre que el estado de creencia se reduce a la única ubicación mostrada en la Figura 4.18(b). Esa es la única ubicación que podría resultar de

UPDATE(PREDICT(UPDATE(b, 1011),*Right*), 1010).

Con acciones no deterministas, el paso PREDICT crece el estado de creencia, pero el paso UPDATE lo reduce de nuevo, siempre y cuando las percepciones den información útil para la identificación. A veces, las percepciones casi no ayudan para la localización: Si hubiera uno o más corredores largos este-oeste, un robot podría recibir una larga secuencia de percepciones 1010, pero nunca saber dónde estaban en el(los) corredor(es). Pero para entornos con una variación



(a) Posibles ubicaciones del robot después de $E_1 = 1011$



(b) Posibles ubicaciones del robot después de $E_1 = 1011, E_2 = 1010$

Figura 4.18 Posibles posiciones del robot, ⊙, (a) después de una observación, E_1 = 1011, y (b) después de mover un cuadrado y hacer una segunda observación, E_2 = 1010. Si los sensores no hacen ruido y el modelo de transición es preciso, sólo hay una posible ubicación para el robot que es consistente con esta secuencia de dos observaciones.

geográfica razonable, la localización suele converger rápido a un punto único, aun cuando las acciones son no deterministas.

¿Qué sucede si los sensores son defectuosos? Si sólo podemos razonar con lógica booleana, entonces tenemos que tratar cada bit del sensor como correcto o incorrecto, lo que es lo mismo que no tener nada de información perceptual. Pero veremos que el razonamiento probabilístico (Capítulo 12) nos permite extraer información útil de un sensor defectuoso siempre que esté equivocado menos de la mitad del tiempo.

4.5 Agentes de búsqueda online y entornos desconocidos

Búsqueda offline

Búsqueda online

Hasta ahora nos hemos concentrado en agentes que usan algoritmos de **búsqueda offline**. Calculan una solución completa antes de tomar su primera acción. En contraste, un agente de **búsqueda online**[8] intercala la computación y la acción: primero toma una acción, luego observa el entorno y calcula la siguiente acción. La búsqueda online es una buena idea en entornos dinámicos o semidinámicos, donde hay una penalización por quedarse sentado y computar por

[8] El término "online" aquí se refiere a algoritmos que deben procesar la entrada a medida que se recibe, en vez de esperar a que todo el conjunto de datos de entrada esté disponible. Este uso "online" no se relaciona con el concepto de "tener una conexión a Internet".

mucho tiempo. La búsqueda online también es útil en dominios no deterministas porque permite al agente centrar sus esfuerzos computacionales en las contingencias que realmente surgen en lugar de aquellas que *podrían* ocurrir, pero tal vez no lo harán.

Por supuesto, existe un compromiso: cuanto más planifique un agente por adelantado, menos veces se encontrará en apuros. En entornos desconocidos, donde el agente no sabe qué estados existen o qué hacen sus acciones, el agente debe usar éstas como experimentos para aprender sobre el entorno.

Problema de mapeo

Un ejemplo canónico de búsqueda online es el **problema de mapeo**: un robot se coloca en un edificio desconocido y debe explorar para construir un mapa que luego pueda ser usado para llegar de A a B. Los métodos para escapar de laberintos, conocimiento necesario para aspirantes a héroes de la Antigüedad, también son ejemplos de algoritmos de búsqueda online. Sin embargo, la exploración espacial no es la única forma de exploración online. Considere un recién nacido: tiene muchas acciones posibles, pero no conoce los resultados de ninguna de ellas, y sólo ha experimentado pocos de los estados posibles a los que puede llegar.

4.5.1 Problemas de búsqueda online

Un problema de búsqueda online se resuelve intercalando computación, percepción y acción. Iniciamos asumiendo un entorno determinista y por completo observable (el Capítulo 17 amplía estas premisas), y determinamos que el agente sólo conoce:

- ACTIONS(s), las acciones legales en el estado s;
- $c(s,a,s')$, el costo de aplicar la acción a en el estado s para llegar al estado s'. Note que esto no se puede usar hasta que el agente sepa que s' es el resultado.
- IS-GOAL(s), la prueba de objetivo.

Observe en particular que el agente *no puede* determinar RESULT(s,a) excepto estando realmente en s y haciendo a. Por ejemplo, en el problema del laberinto mostrado en la Figura 4.19, el agente no sabe que ir hacia *Arriba* desde (1,1) conduce a (1,2); ni, habiendo hecho eso, sabe que ir hacia *Abajo* lo llevará de vuelta a (1,1). Este grado de ignorancia se puede reducir en algunas aplicaciones, por ejemplo, un explorador robot podría saber cómo funcionan sus acciones de movimiento y sólo ignorar las ubicaciones de los obstáculos.

Finalmente, el agente podría tener acceso a una función heurística admisible $h(s)$ que estima la distancia desde el estado actual hasta un estado objetivo. Por ejemplo, en la Figura 4.19, el agente podría conocer la ubicación del objetivo y utilizar la heurística de distancia de Manhattan (página 97).

Por lo regular, el objetivo del agente es alcanzar un estado objetivo minimizando el costo. (Otro objetivo posible es sólo explorar todo el entorno). El costo es el costo total del camino en que el agente incurre a medida que viaja. Es común comparar este costo con el costo del camino en que el agente incurriría *si conociera el espacio de búsqueda de antemano*, es decir, el camino óptimo en el entorno conocido. En el lenguaje de los algoritmos online, esta comparación se llama **relación competitiva**; nos gustaría que fuera lo más pequeña posible.

Relación competitiva

Los exploradores online son vulnerables a **callejones sin salida**: estados desde los cuales no se puede llegar a ningún estado objetivo. Si el agente no sabe qué hace cada acción, podría ejecutar la acción "saltar al pozo sin fondo", y así nunca alcanzar el objetivo. En general, *ningún algoritmo puede evitar callejones sin salida en todos los espacios de estado*. Considere los dos espacios de estado sin salida en la Figura 4.20(a). Un algoritmo de búsqueda online que ha visitado los estados S y A no puede decir si está en el estado superior o en el inferior; ambos parecen idénticos según lo que ha visto el agente. Por lo tanto, no hay forma de que pueda saber cómo

Callejones sin salida

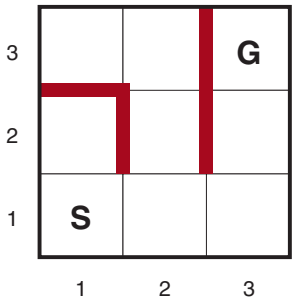


Figura 4.19 Problema de laberinto. El agente comienza en *S* y debe llegar a *G* pero no sabe nada del entorno.

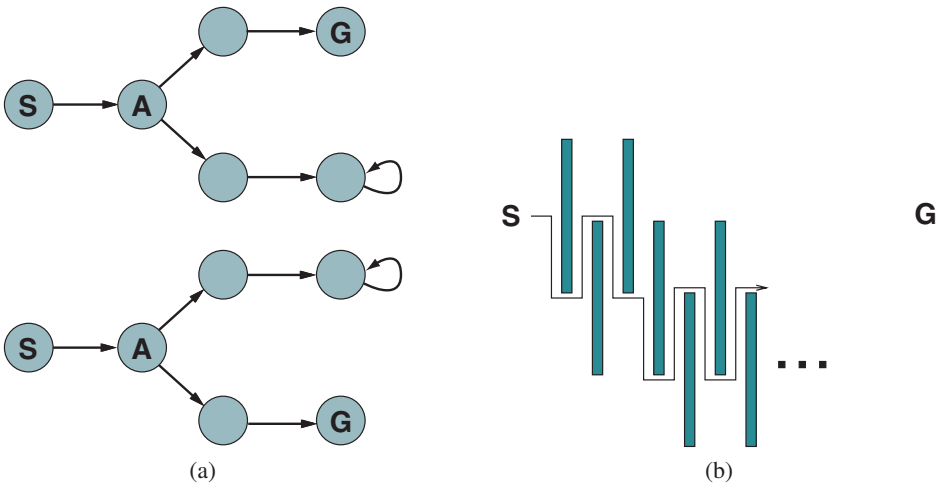


Figura 4.20 (a) Dos espacios de estados que podrían llevar a un agente de búsqueda online a un callejón sin salida. Cualquier agente determinado fallará en al menos uno de estos espacios. (b) Un entorno bidimensional que puede hacer que un agente de búsqueda online siga una ruta arbitrariamente ineficiente hacia la meta. Toda elección que tome el agente, el adversario bloquea esa ruta con otro muro largo y delgado, de modo que el camino seguido es más largo que el mejor camino posible.

Argumento adversario

elegir la acción correcta en ambos espacios de estado. Este es un ejemplo de un **argumento adversario**: podemos imaginar un adversario construyendo el espacio de estado mientras el agente lo explora y colocando los objetivos y callejones sin salida donde elija, como en la Figura 4.20(b).

Irreversible

Seguro para explorar

Los callejones sin salida son una dificultad real para la exploración de robots: escaleras, rampas, acantilados, calles de sentido único y terrenos naturales presentan estados desde los que algunas acciones son **irreversibles**, es decir, no hay forma de volver al estado anterior. El algoritmo de exploración que presentaremos sólo está garantizado para funcionar en espacios de estado que son **seguros para explorar**, es decir, algún estado objetivo es alcanzable desde cada estado alcanzable. Los espacios de estado con acciones sólo reversibles, como laberintos y 8-puzzles,

```
function ONLINE-DFS-AGENT(problem, s′) returns una acción
                s, a, el estado y la acción anteriores, inicialmente nulos
  persistent: result, un mapeo de tablas (s, a) en s′, Inicialmente vacía
                untried, una tabla que asigna s a una lista de acciones no probadas
                unbacktracked, una tabla que asigna s a una lista de estados a los que nunca
                se retrocedió

  if problem.IS-GOAL(s′) then return stop
  if s′ es un nuevo estado (not in untried) then untried[s′] ← problem.ACTIONS(s′)
  if s no es nulo then
    result[s, a] ← s′
    agregue s al principio de unbacktracked[s′]
  if untried[s′] está vacío then
    if unbacktracked[s′] está vacío then return stop
    else a ← una acción b tal que result[s′, b] = POP(unbacktracked[s′])
  else a ←POP(untried[s′])
  s ← s′
  return a
```

Figura 4.21 Agente de búsqueda online que usa la exploración en profundidad. El agente sólo puede explorar con seguridad espacios de estado donde cada acción puede ser "deshecha" por alguna otra acción.

son claramente seguros para explorar (si tienen solución). Cubriremos el tema de la exploración segura con más detalle en la Sección 22.3.2.

Aun en entornos seguros para explorar, no se puede garantizar una relación competitiva acotada si hay caminos de costo no acotado. Esto es fácil de mostrar en entornos con acciones irreversibles, pero, de hecho, sigue siendo cierto para el caso reversible también, como muestra la Figura 4.20(b). Por ello, es común caracterizar el rendimiento de los algoritmos de búsqueda online en términos del tamaño de todo el espacio de estados en lugar de sólo la profundidad del objetivo más superficial.

4.5.2 Agentes de búsqueda online

Después de cada acción, un agente online en un entorno observable recibe un percepto que le indica en qué estado ha llegado; con esta información mejora su mapa del entorno. Luego, el mapa actualizado se usa para planificar dónde ir a continuación. Esta intercalación de planificación y acción significa que los algoritmos de búsqueda online son muy diferentes de los algoritmos de búsqueda offline que vimos antes: los algoritmos offline exploran su *modelo* del espacio de estados, mientras que los algoritmos online exploran el mundo real. Por ejemplo, A* puede expandir un nodo en una parte del espacio y luego expandir de inmediato un nodo en una parte distante del espacio, porque la expansión de nodos implica acciones simuladas en lugar de reales.

Un algoritmo online sólo puede descubrir sucesores para un estado que ocupa físicamente. Para evitar viajar hasta un estado distante para expandir el siguiente nodo, parece mejor expandir nodos en un orden *local*. La búsqueda en profundidad tiene exactamente esta propiedad porque (excepto si el algoritmo está retrocediendo) el próximo nodo expandido es un hijo del nodo previo expandido.

Un agente de exploración en profundidad online (para acciones desconocidas pero deterministas) se muestra en la Figura 4.21. Este agente almacena su mapa en una tabla, *result*[*s, a*], que registra el estado resultante de ejecutar la acción *a* en el estado *s*. (Para acciones no deterministas, el

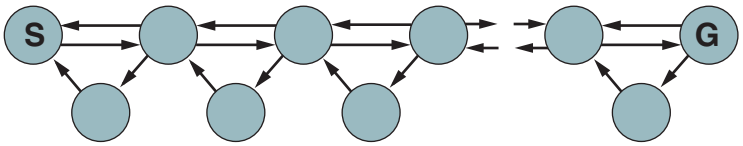


Figura 4.22 Entorno en el que un recorrido aleatorio requerirá muchos pasos exponenciales para encontrar el objetivo.

agente podría registrar un conjunto de estados bajo *results*[*s*,*a*].) Siempre que el estado actual tenga acciones no exploradas, el agente prueba una de esas acciones. La dificultad surge cuando el agente ha probado todas las acciones en un estado. En la búsqueda en profundidad offline, el estado sólo se elimina de la cola; en una búsqueda online, el agente tiene que retroceder en el mundo físico. En la búsqueda en profundidad, esto significa volver al estado desde el cual el agente ingresó recién al estado actual. Para lograr eso, el algoritmo mantiene otra tabla que enumera, para cada estado, los estados predecesores a los cuales el agente aún no ha retrocedido. Si el agente se ha quedado sin estados a los cuales puede retroceder, entonces su búsqueda está completa.

Recomendamos al lector que siga el progreso del ONLINE-DFS-AGENT cuando se aplique al laberinto dado en la Figura 4.19. Es muy fácil ver que, en el peor de los casos, el agente terminará recorriendo cada enlace en el espacio de estados exactamente dos veces. Para la exploración, esto es óptimo; pero para encontrar un objetivo, la proporción competitiva del agente podría ser arbitrariamente mala si se embarca en una larga excursión cuando hay un objetivo justo al lado del estado inicial. Una variante online del profundizamiento iterativo resuelve este problema; para un entorno que es un árbol uniforme, la proporción competitiva de dicho agente es una constante pequeña.

Debido a su método de retroceso, el ONLINE-DFS-AGENT sólo funciona en espacios de estados donde las acciones son reversibles. Existen algoritmos poco más complejos que funcionan en espacios de estados generales, pero ninguno de ellos tiene una proporción competitiva acotada.

4.5.3 Exploración local online

Al igual que la búsqueda de profundidad, la **búsqueda de ascenso de colinas** tiene la propiedad de la localidad en sus expansiones de nodos. De hecho, como mantiene sólo un estado actual en la memoria, este tipo de búsqueda *ya es* un algoritmo de búsqueda online. Pero, el algoritmo básico no es muy bueno para la exploración porque deja al agente sentado en máximos locales sin a dónde ir. Además, no se pueden usar reinicios aleatorios, porque el agente no puede teleportarse a un nuevo estado inicial.

Recorrido aleatorio

En vez de reinicios aleatorios, se podría considerar el uso de un **recorrido aleatorio** para explorar el entorno. Un recorrido aleatorio sólo selecciona al azar una de las acciones disponibles desde el estado actual; puede preferir las acciones aún no probadas. Es fácil demostrar que un recorrido aleatorio *eventualmente* encontrará un objetivo o completará su exploración, siempre que el espacio sea finito y se explore de manera segura.[9] Además, el proceso puede ser lento. La Figura 4.22 muestra un entorno donde un paseo aleatorio tomará exponencialmente muchos pasos para encontrar el objetivo, porque, para cada estado en la fila superior excepto *S*, el progreso hacia atrás es el doble de probable que el progreso hacia adelante. Claro, el ejemplo está ideado, pero hay muchos espacios de estados del mundo real cuya topología causa este tipo de "trampas" para los recorridos aleatorios.

[9] Los recorridos aleatorios se completan en infinitas cuadrículas unidimensionales y bidimensionales. En una cuadrícula tridimensional, la probabilidad de que la caminata regrese alguna vez al punto de partida es de apenas 0.3405 (Hughes, 1995).

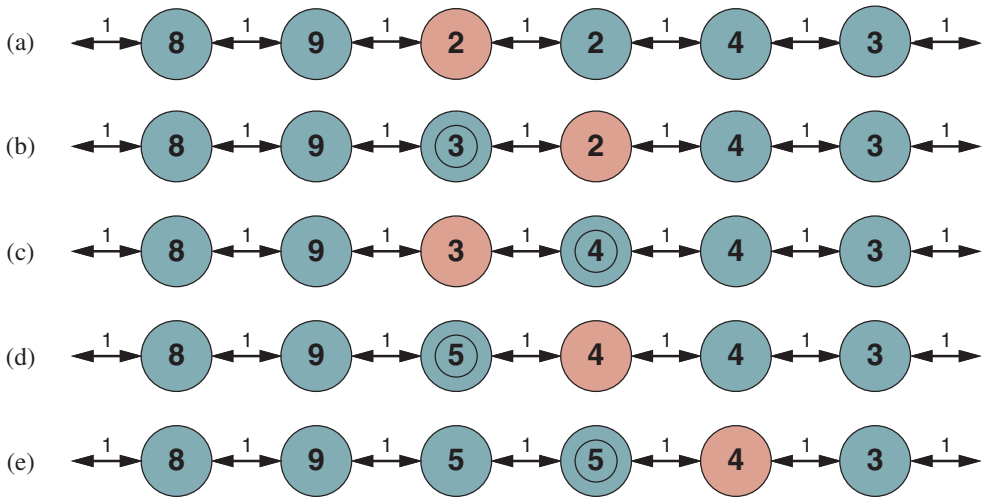


Figura 4.23 Cinco iteraciones de LRTA* en un espacio de estados unidimensional. Cada estado está etiquetado con $H(s)$, la estimación de costo actual para alcanzar un objetivo, y cada enlace tiene un costo de acción de 1. El estado rojo marca la ubicación del agente, las estimaciones de costo actualizadas en cada iteración tienen doble círculo.

El enfoque de mejorar el ascenso de colinas con *memoria* y no aleatoriedad resulta más efectivo. La idea básica es almacenar una "mejor estimación actual" $H(s)$ del costo para alcanzar el objetivo desde cada estado que ha sido visitado. $H(s)$ comienza siendo sólo la estimación heurística $h(s)$ y se actualiza a medida que el agente adquiere experiencia en el espacio de estados.

La Figura 4.23 muestra un ejemplo simple en un espacio de estados unidimensional. En (a), el agente parece estar atrapado en un mínimo local plano en el estado rojo. En lugar de quedarse donde está, el agente debería seguir lo que parece ser el mejor camino hacia el objetivo, dadas las estimaciones de costo actuales para sus vecinos. El costo estimado para llegar al objetivo a través de un vecino s' es el costo para llegar a s' más el costo estimado para llegar a un objetivo desde ahí, es decir, $c(s, a, s') + H(s')$. En el ejemplo, hay dos acciones, con costos estimados de $1 + 9$ hacia la izquierda y $1 + 2$ hacia la derecha, por lo que parece mejor moverse hacia la derecha.

En (b) está claro que la estimación de costo de 2 para el estado rojo en (a) fue en exceso optimista. Dado que el mejor movimiento tuvo un costo de 1 y llevó a un estado que está al menos a 2 pasos de un objetivo, el estado rojo debe estar al menos a 3 pasos de un objetivo, por lo que su H debe actualizarse en consecuencia, como lo muestra la Figura 4.23(b). Siguiendo este proceso, el agente se moverá hacia adelante y hacia atrás dos veces más, actualizando H cada vez y "aplanando" el mínimo local hasta que escape hacia la derecha.

Un agente que implementa este esquema, llamado A* en tiempo real de aprendizaje (**LRTA***), se muestra en la Figura 4.24. Al igual que ONLINE-DFS-AGENT, construye un mapa del entorno en la tabla de *resultados*. Actualiza la estimación de costo para el estado que acaba de dejar y luego elige el movimiento "aparentemente mejor" según sus estimaciones de costo actuales. Un detalle significativo es que se asume que las acciones que aún no se han intentado en un estado s siempre llevan inmediatamente al objetivo con el menor costo posible, es decir, $h(s)$. Este **optimismo bajo incertidumbre** alienta al agente a explorar nuevos caminos, posiblemente prometedores.

LRTA*

Optimismo bajo incertidumbre

```
function LRTA*-AGENT(problem, s', h) returns una acción
                 s, a, el estado y la acción anteriores, inicialmente nulos
  persistent: result, un mapeo de tablas (s, a) en s', inicialmente vacía
                 H, una tabla que asigna s a una estimación de costos, inicialmente vacía

  if IS-GOAL(s') then return stop
  if s' es un nuevo estado (no en H) then H[s']←h(s')
  if s no es nulo then
    result[s, a]←s'
    H[s]← min_{b∈ACTIONS(s)} LRTA*-COST(s, b, result[s, b],H)
  a← argmin_{b∈ACTIONS(s)} LRTA*-COST(problem, s', b, result[s', b],H)
  s←s'
  return a

function LRTA*-COST(problem, s, a, s', H) returns una estimación de costos
  if s' no está definida then return h(s)
  else return problem.ACTION-COST(s, a, s') + H[s']
```

Figura 4.24 LRTA*-AGENT selecciona una acción de acuerdo con los valores de los estados vecinos, que se actualizan a medida que el agente se mueve por el espacio de estados.

Un agente LRTA* está garantizado de encontrar un objetivo en cualquier entorno finito y seguramente exploratorio. Pero a diferencia de A*, no es completo para espacios de estados infinitos; hay casos en los que puede ser desviado infinitamente. Puede explorar un entorno de n estados en $O(n^2)$ pasos en el peor de los casos, pero con frecuencia lo hace mucho mejor. El agente LRTA* es sólo uno de una gran familia de agentes online que se pueden definir especificando la regla de selección de acciones y la regla de actualización de diferentes maneras. Discutimos esta familia, desarrollada originalmente para entornos estocásticos, en el Capítulo 22.

4.5.4 Aprendizaje en la búsqueda online

La ignorancia inicial de los agentes de búsqueda online ofrece varias oportunidades para aprender. Primero, los agentes aprenden un "mapa" del entorno, justo el resultado de cada acción en cada estado, sólo registrando cada una de sus experiencias. Segundo, los agentes de búsqueda local logran estimaciones más precisas del costo de cada estado usando reglas de actualización local, como en LRTA*. En el Capítulo 22, mostramos que estas actualizaciones eventualmente convergen en valores exactos para cada estado, siempre que el agente explore el espacio de estados de manera correcta. Una vez que se conocen los valores exactos, las decisiones óptimas pueden tomarse con moverse al sucesor de menor costo; es decir, el llano ascenso de colinas resulta una estrategia óptima.

Si siguió nuestra sugerencia de rastrear el comportamiento del ONLINE-DFS-AGENT en el entorno de la Figura 4.19, ya notó que el agente no es muy inteligente. Por ejemplo, después de haber visto que la acción *Up* va de (1,1) a (1,2), el agente aún no tiene idea de que la acción *Down* regresa a (1,1) o de que la acción *Up* también va de (2,1) a (2,2), de (2,2) a (2,3), y así sucesivamente. En general, nos gustaría que el agente aprendiera que *Up* aumenta la coordenada y a menos que haya una pared en el camino, que *Down* la reduce, etc.

Para que esto suceda, necesitamos dos cosas. Primero, una representación formal y explícitamente manipulable para este tipo de reglas generales; hasta ahora, hemos ocultado la información dentro de la caja negra llamada función RESULT. Los Capítulos 8 al 11 están dedicados a este tema. En segundo lugar, necesitamos algoritmos que puedan construir reglas generales adecuadas a partir de las observaciones específicas realizadas por el agente. Estos se tratan en el Capítulo 19.

Si anticipamos que se nos llamará a resolver múltiples problemas similares en el futuro, entonces tiene sentido invertir tiempo (y memoria) para facilitar esas búsquedas futuras. Hay varias formas de hacerlo, todas bajo el título de **búsqueda incremental**. Podríamos mantener el árbol de búsqueda en memoria y reutilizar las partes que no cambian en el nuevo problema. Podríamos mantener los valores heurísticos *h* y actualizarlos a medida que logramos nueva información, ya sea porque el mundo ha cambiado o porque hemos calculado una mejor estimación. O podríamos mantener los valores de la mejor ruta *g*, usándolos para ensamblar una nueva solución y actualizarlos cuando el mundo cambie.

Búsqueda incremental

Resumen

Este capítulo ha examinado algoritmos de búsqueda para problemas en entornos parcialmente observables, no deterministas, desconocidos y continuos.

- Los métodos de *búsqueda local* como el **ascenso de colina** mantienen sólo un pequeño número de estados en memoria. Se han aplicado a problemas de optimización, donde la idea es encontrar un estado de puntuación alta, sin preocuparse por el camino hacia el estado. Se han desarrollado varios algoritmos de búsqueda local estocástica, incluido el **simulated annealing**, que devuelve soluciones óptimas si se le proporciona un programa de enfriamiento adecuado.
- Muchos métodos de búsqueda local también se aplican a problemas en espacios continuos. Los problemas de **programación lineal** y **optimización convexa** obedecen ciertas restricciones sobre la forma del espacio de estados y la naturaleza de la función objetivo, y admiten algoritmos de tiempo polinomial que suelen ser en extremo eficientes en la práctica. Para unos problemas matemáticamente bien formados, encontramos el máximo usando cálculo para encontrar dónde el gradiente es cero; para otros problemas, debemos conformarnos con el gradiente empírico, que mide la diferencia en aptitud entre dos puntos cercanos.
- Un **algoritmo evolutivo** es una búsqueda estocástica de ascenso de colinas en la que se mantiene una población de estados. Los nuevos estados se generan mediante **mutación** y **cruzamiento**, que combina pares de estados.
- En entornos **no deterministas**, los agentes pueden aplicar la búsqueda AND-OR para generar planes **contingentes** que alcancen el objetivo independientemente de los resultados durante la ejecución.
- Cuando el entorno es parcialmente observable, el **estado de creencias** representa el conjunto de estados posibles en los que podría estar el agente.
- Los algoritmos de búsqueda estándar se pueden aplicar directamente al espacio de estados de creencias para resolver **problemas sin sensor**, y la búsqueda AND-OR del estado de creencias puede resolver problemas parcialmente observables generales. Los algoritmos incrementales que construyen soluciones estado por estado dentro de un estado de creencias a menudo son más eficientes.
- Los **problemas de exploración** surgen cuando el agente no tiene idea de los estados y acciones de su entorno. Para entornos explorables de manera segura, los agentes de **búsqueda online** pueden construir un mapa y encontrar una meta, si existe. Actualizar las estimaciones heurísticas a partir de la experiencia da un método efectivo para escapar de los mínimos locales.

Notas bibliográficas e históricas

Las técnicas de búsqueda local tienen una larga historia en matemáticas e informática. De hecho, el método de Newton-Raphson (Newton, 1671; Raphson, 1690) puede considerarse como un método de búsqueda local muy eficiente para espacios continuos en los que se dispone de información del gradiente. Brent (1973) es una referencia clásica para algoritmos de optimización que no requieren tal información. La búsqueda de haz, que hemos presentado como un algoritmo de búsqueda local, se originó como una variante de ancho limitado de la programación dinámica para el reconocimiento de voz en el sistema HARPY (Lowerre, 1976). Un algoritmo relacionado es analizado en profundidad por Pearl (1984, Cap. 5).

El tema de la búsqueda local fue revitalizado a principios de la década de 1990 por resultados sorprendentemente buenos para grandes problemas de satisfacción de restricciones como el problema de las n reinas (Minton *et al.*, 1992) y la satisfacción booleana (Selman *et al.*, 1992), y por la incorporación de aleatoriedad, múltiples búsquedas simultáneas y otras mejoras. Este renacimiento de lo que Christos Papadimitriou ha llamado algoritmos de "Nueva Era" también generó mayor interés entre los científicos informáticos teóricos (Koutsoupias y Papadimitriou, 1992; Aldous y Vazirani, 1994).

Búsqueda tabú

En el campo de la investigación operativa, una variante del ascenso de colinas llamada **búsqueda tabú** ha ganado popularidad (Glover y Laguna, 1997). Este algoritmo mantiene una lista tabú de k estados visitados previamente que no pueden ser revisitados; además de mejorar la eficiencia al buscar en grafos, esta lista puede permitir que el algoritmo escape de algunos mínimos locales.

Otra mejora útil en el ascenso de colina es el algoritmo STAGE (Boyan y Moore, 1998). La idea es utilizar los máximos locales encontrados por el ascenso de colina con reinicios aleatorios para tener una idea de la forma general de la imagen de espacio de estados. El algoritmo ajusta una superficie cuadrática suave al conjunto de máximos locales y luego calcula el máximo global de esa superficie analíticamente. Éste se convierte en el nuevo punto de reinicio. Gomes *et al.* (1998) mostraron que los tiempos de ejecución de los algoritmos de retroceso sistemático con frecuencia tienen una **distribución de cola pesada**, lo que significa que la probabilidad de un tiempo de ejecución muy largo es mayor de lo que se podría predecir si los tiempos de ejecución fueran distribuidos exponencialmente. Si la distribución del tiempo de ejecución tiene una cola pesada, los reinicios aleatorios encuentran una solución más rápido, en promedio, que una sola ejecución completa. Hoos y Stützle (2004) proporcionan una cobertura extensa del tema.

Distribución de cola pesada

Simulated annealing fue descrito por primera vez por Kirkpatrick *et al.* (1983), quienes tomaron prestado directamente del **algoritmo de Metrópolis** (que se utiliza para simular sistemas complejos en física (Metropolis *et al.*, 1953) y supuestamente fue inventado en una cena en Los Alamos). Simulated annealing es ahora un campo en sí mismo, con cientos de artículos publicados cada año.

Encontrar soluciones óptimas en espacios continuos es el tema de varios campos, incluidos **teoría de la optimización, teoría del control óptimo y cálculo de variaciones.** Las técnicas básicas son explicadas bien por Bishop (1995); mientras que Press *et al.* (2007) cubren una amplia gama de algoritmos y proporcionan software funcional.

Los investigadores se han inspirado para algoritmos de búsqueda y optimización de una amplia variedad de campos de estudio: metalurgia (simulated annealing); biología (algoritmos genéticos); neurociencia (redes neuronales); montañismo (ascenso de colina); economía (algoritmos basados en el mercado (Dias *et al.*, 2006)); física (enjambres de partículas (Li y Yao, 2012) y vidrios de espín (Mezard *et al.*, 1987)); comportamiento animal (aprendizaje por refuerzo, optimizadores de lobo gris (Mirjalili y Lewis, 2014)); ornitología (búsqueda de cucú (Yang y Deb, 2014)); entomología (colonias de hormigas (Dorigo *et al.*, 2008), colonias de abejas (Karaboga y Basturk, 2007), luciérnagas (Yang, 2009) y optimización de gusanos luminosos (Krishnanand y Ghose, 2009)); y otros.

La **programación lineal** (LP) fue estudiada sistemáticamente por el matemático Leonid Kantorovich (1939). Fue una de las primeras aplicaciones de las computadoras; el **algoritmo simplex** (Dantzig, 1949) todavía se usa a pesar de la complejidad exponencial en el peor de los casos. Karmarkar (1984) desarrolló una familia mucho más eficiente de métodos de **puntos interiores**, que se demostró que tenía complejidad polinómica para la clase más general de problemas de optimización convexa por Nesterov y Nemirovski (1994). Excelentes introducciones a la optimización convexa son proporcionadas por Ben-Tal y Nemirovski (2001) y Boyd y Vandenberghe (2004).

El trabajo de Sewall Wright (1931) sobre el concepto de una **imagen de aptitud** fue un precursor importante para el desarrollo de algoritmos genéticos. En la década de 1950, varios estadísticos, incluidos Box (1957) y Friedman (1959), usaron técnicas evolutivas para problemas de optimización, pero el enfoque sólo ganó popularidad hasta que Rechenberg (1965) introdujo **estrategias de evolución** para resolver problemas de optimización para perfiles alares. En las décadas de 1960 y 1970, John Holland (1975) defendió los algoritmos genéticos, tanto como una herramienta útil de optimización como un método para expandir nuestra comprensión de la adaptación (Holland, 1995).

El movimiento de la **vida artificial** (Langton, 1995) llevó esta idea un paso más allá, considerando los productos de los algoritmos genéticos como *organismos* en lugar de soluciones a problemas. El efecto Baldwin discutido en el capítulo fue propuesto casi al mismo tiempo por Conwy Lloyd Morgan (1896) y James (Baldwin, 1896). Las simulaciones por computadora han ayudado a aclarar sus implicaciones (Hinton y Nowlan, 1987; Ackley y Littman, 1991; Morgan y Griffiths, 2015). Smith y Szathmáry (1999), Ridley (2004) y Carroll (2007) proporcionan antecedentes generales sobre la evolución.

La mayoría de las comparaciones de algoritmos genéticos con otros enfoques (en especial el escalado estocástico) han encontrado que los algoritmos genéticos son más lentos para converger (O'Reilly y Oppacher, 1994; Mitchell *et al.*, 1996; Juels y Wattenberg, 1996; Baluja, 1997). Tales hallazgos no son universalmente populares dentro de la comunidad de Algoritmos Genéticos (AG), pero los intentos recientes dentro de esa comunidad para entender la búsqueda basada en poblaciones, como una forma aproximada de aprendizaje bayesiano (ver Capítulo 20), podrían ayudar a cerrar la brecha entre el campo y sus críticos (Pelikan *et al.,* 1999). La teoría de los **sistemas dinámicos cuadráticos** también puede explicar el rendimiento de los AG (Rabani *et al.*, 1998). Hay algunas aplicaciones prácticas impresionantes de los AG, en áreas tan diversas como el diseño de antenas (Lohn *et al.*, 2001), diseño asistido por computadora (Renner y Ekart, 2003), modelos climáticos (Stanislawska *et al.*, 2015), medicina (Ghaheri *et al.*, 2015) y diseño de redes neuronales profundas (Miikkulainen *et al.*, 2019).

El campo de la **programación genética** es un subcampo de los algoritmos genéticos en el que las representaciones son programas en lugar de cadenas de bits. Los programas se representan en forma de árboles de sintaxis, ya sea en un lenguaje de programación estándar o en formatos diseñados especialmente para representar circuitos electrónicos, controladores de robots, etc. El cruce implica la unión de subárboles de tal manera que se garantice que la descendencia sea expresiones bien formadas.

El interés en la programación genética fue impulsado por el trabajo de John Koza (1992, 1994), pero se remonta al menos a experimentos tempranos con código de máquina por Friedberg (1958) y con autómatas finitos por Fogel *et al.* (1966). Al igual que con los algoritmos genéticos, hay un debate sobre la efectividad de la técnica. Koza *et al.* (1999) describen experimentos en el uso de la programación genética para diseñar dispositivos de circuito.

Las revistas *Evolución Evolutionary Computation* y *IEEE Transactions on Evolutionary Computation* cubren algoritmos evolutivos; los artículos también se encuentran en *Complex Systems, Adaptive Behavior,* y *Artificial Life*. La principal conferencia es la *Genetic and Evolutionary Computation Conference* (GECCO). Textos sobresalientes sobre revisión de

algoritmos genéticos incluyen los de Mitchell (1996), Fogel (2000), Langdon y Poli (2002) y Poli *et al.* (2008).

La imprevisibilidad y la observabilidad parcial de los entornos reales fueron reconocidas tempranamente en proyectos de robótica que usaban técnicas de planificación, incluyendo Shakey (Fikes *et al.*, 1972) y FREDDY (Michie, 1972). Los problemas recibieron más atención después de la publicación del influyente artículo de McDermott (1978a) *Planning and Acting.*

El primer trabajo que hizo uso explícito de los árboles AND-OR parece haber sido el programa SAINT de Slagle para integración simbólica, mencionado en el Capítulo 1. Amarel (1967) aplicó la idea a la demostración de teoremas proposicionales, un tema revisado en el Capítulo 7, e introdujo un algoritmo de búsqueda similar a AND-OR-GRAPH-SEARCH. El algoritmo fue desarrollado después por Nilsson (1971), quien también describió AO*, que, como su nombre sugiere, encuentra soluciones óptimas. AO* fue mejorado aún más por Martelli y Montanari (1973).

AO* es un algoritmo de arriba hacia abajo; una generalización de abajo hacia arriba de A* es A*LD, para Derivación Más Ligera de A* (Felzenszwalb y McAllester, 2007). El interés en la búsqueda AND-OR experimentó un renacimiento a principios de la década de 2000, con nuevos algoritmos para encontrar soluciones cíclicas (Jimenez y Torras, 2000; Hansen y Zilberstein, 2001) y nuevas técnicas inspiradas en la programación dinámica (Bonet y Geffner, 2005).

La idea de transformar problemas parcialmente observables en problemas de estados de creencia se originó con Astrom (1965) para el caso mucho más complejo de incertidumbre probabilística (ver Capítulo 17). Erdmann y Mason (1988) estudiaron el problema de la manipulación robótica sin sensores, utilizando una forma continua de búsqueda de estados de creencia. Mostraron que era posible orientar una pieza en una mesa desde una posición inicial arbitraria mediante una secuencia bien diseñada de acciones de inclinación. Métodos más prácticos, basados en una serie de barreras diagonales precisamente orientadas en una cinta transportadora, utilizan los mismos conocimientos algorítmicos (Wiegley *et al.*, 1996).

El enfoque de estados de creencia fue reinventado en el contexto de problemas de búsqueda sin sensores y parcialmente observables por Genesereth y Nourbakhsh (1993). Se realizaron trabajos adicionales sobre problemas sin sensores en la comunidad de planificación basada en lógica (Goldman y Boddy, 1996; Smith y Weld, 1998). Este trabajo ha enfatizado representaciones concisas para estados de creencia, como se explica en el Capítulo 11. Bonet y Geffner (2000) introdujeron las primeras heurísticas efectivas para la búsqueda de estados de creencia; estas fueron refinadas por Bryce *et al.* (2006). El enfoque incremental para la búsqueda de estados de creencia, en el que las soluciones se construyen incrementalmente para subconjuntos de estados dentro de cada estado de creencia, fue estudiado en la literatura de planificación por Kurien *et al.* (2002); varios nuevos algoritmos incrementales fueron introducidos para problemas no deterministas y parcialmente observables por Russell y Wolfe (2005). Referencias adicionales para la planificación en entornos estocásticos y parcialmente observables aparecen en el Capítulo 17.

Los algoritmos para explorar espacios de estado desconocidos han sido de interés durante muchos siglos. La búsqueda en profundidad en un laberinto reversible se puede implementar manteniendo la mano izquierda en la pared; los bucles pueden evitarse marcando cada cruce. El problema más general de explorar **grafos eulerianos** (es decir, grafos en los que cada nodo tiene igual número de aristas entrantes y salientes) fue resuelto por un algoritmo debido a Hierholzer (1873).

Grafo Euleriano

El primer estudio algorítmico exhaustivo del problema de exploración para grafos arbitrarios fue llevado a cabo por Deng y Papadimitriou (1990), quienes desarrollaron un algoritmo completamente general, pero mostraron que no es posible un cociente competitivo acotado para explorar un grafo general. Papadimitriou y Yannakakis (1991) examinaron la cuestión de encontrar caminos hacia un objetivo en entornos de planificación de rutas geométricas (donde todas las acciones son reversibles). Mostraron que es posible lograr un pequeño cociente competitivo con obstáculos

cuadrados, pero con obstáculos rectangulares generales no se puede lograr un cociente acotado. (Ver Figura 4.20).

En un entorno dinámico, el estado del mundo puede cambiar espontáneamente sin ninguna acción por parte del agente. Por ejemplo, el agente puede planificar una ruta de manejo óptima de *A* a *B*, pero un accidente o tráfico inusualmente malo durante la hora pico pueden arruinar el plan. Los algoritmos de búsqueda incremental como el Lifelong Planning A* (Koenig *et al.*, 2004) y el D* Lite (Koenig y Likhachev, 2002) abordan esta situación.

El algoritmo LRTA* fue desarrollado por Korf (1990) como parte de una investigación sobre **búsqueda en tiempo real** para entornos en los que el agente debe actuar después de buscar sólo una cantidad fija de tiempo (una situación común en juegos de dos jugadores). LRTA* es de hecho un caso especial de algoritmos de aprendizaje por refuerzo para entornos estocásticos (Barto *et al.*, 1995). Su política de optimismo ante la incertidumbre, siempre dirigirse hacia el estado no visitado más cercano, puede resultar en un patrón de exploración menos eficiente en el caso no informado que la simple búsqueda en profundidad (Koenig, 2000). Dasgupta *et al.* (1994) muestran que la búsqueda iterativa online es óptimamente eficiente para encontrar un objetivo en un árbol uniforme sin información heurística.

Se han desarrollado varias variantes informadas sobre el tema de LRTA* con diferentes métodos para buscar y actualizar dentro de la porción conocida del grafo (Pemberton y Korf, 1992). Hasta ahora, no existe una comprensión teórica sólida de cómo encontrar objetivos con eficiencia óptima cuando se utiliza información heurística. Sturtevant y Bulitko (2016) proporcionan un análisis de algunos obstáculos que ocurren en la práctica.

CAPÍTULO 5

BÚSQUEDA ADVERSARIAL Y JUEGOS

En el que exploramos entornos donde otros agentes están conspirando en contra nuestra.

Búsqueda adversarial

En este capítulo cubrimos **entornos competitivos**, donde dos o más agentes tienen objetivos en conflicto, lo que da lugar a problemas de **búsqueda adversarial**. En vez de lidiar con el caos de escaramuzas del mundo real, tenemos juegos como el ajedrez, el Go y el póquer. Para investigadores de IA, la naturaleza simplificada de estos juegos es una ventaja: su estado es fácil de representar, y los agentes suelen estar restringidos a un pequeño número de acciones cuyos efectos se definen por reglas precisas. Los juegos físicos, como croquet y hockey sobre hielo, tienen descripciones más complicadas, una gama más amplia de posibles acciones y reglas imprecisas que definen la legalidad de las acciones. Con excepción del fútbol robótico, estos juegos no han atraído mucho interés en la comunidad de IA.

5.1 Teoría de juegos

Economía

Hay al menos tres posturas a tomar hacia los entornos multiagente. La primera, apropiada cuando el número de agentes es muy grande, es considerarlos en conjunto como una **economía**, ello nos permite hacer cosas como predecir que el aumento de la demanda provocará un incremento de precios, sin tener que predecir la acción de ningún agente individual.

Segunda, podríamos considerar a los agentes adversarios como parte del entorno, una parte que hace que éste sea no determinista. Pero si modelamos a los adversarios igual que, por ejemplo, a veces llueve y a veces no, perdemos la idea de que nuestros adversarios tratan activamente de derrotarnos, aunque se supone que la lluvia no tiene tal intención.

Tercera, modelar explícitamente a los agentes adversarios con técnicas de búsqueda de árbol de juego adversarial. Tema de este capítulo. Iniciamos con una clase restringida de juegos, definimos el movimiento óptimo y un algoritmo para encontrarlo: la búsqueda minimax y una generalización de la búsqueda AND-OR (Figura 4.11). Mostramos que la **poda** favorece una búsqueda más eficiente al ignorar porciones del árbol que no hacen diferencia para el movimiento óptimo. En general, para juegos no triviales, no tendremos tiempo suficiente para asegurarnos de encontrar el movimiento óptimo (aun con poda); tendremos que cortar la búsqueda en algún punto.

Poda

Para cada estado donde elegimos detener la búsqueda, preguntamos quién está ganando. Para responder, tenemos una disyuntiva: aplicar una **función de evaluación** heurística para estimar quién gana en función de las características del estado (Sección 5.3), o promediar los resultados de muchas simulaciones rápidas del juego desde ese estado hasta el final (Sección 5.4).

La Sección 5.5 discute juegos con un elemento de azar (mediante lanzamiento de dados o mezcla de cartas) y la Sección 5.6 cubre juegos de **información imperfecta** (como póquer y bridge, donde no todas las cartas son visibles para todos los jugadores).

Información imperfecta

5.1.1 Juegos de suma cero para dos jugadores

Información perfecta
Juegos de suma cero

Los juegos más estudiados por los teóricos en IA (como ajedrez y Go) son los llamados juegos deterministas, de dos jugadores, de toma de turnos, de **información perfecta** y de **suma cero**. "Información perfecta", sinónimo de "totalmente observable",[1] "suma cero" donde lo que es bueno para un jugador es malo para otro: no hay resultados "gana-gana". A menudo, para los juegos suma cero usamos el término **movimiento** como sinónimo de "acción", y **posición** como sinónimo de "estado".

Mover
Posición

Llamaremos a nuestros dos jugadores MAX y MIN, por razones que pronto serán evidentes. MAX mueve primero y luego los jugadores se turnan para mover hasta que el juego termina. Al final, se otorgan puntos al jugador ganador y se penaliza al perdedor. Un juego se puede definir formalmente con los siguientes elementos:

- S_0: El **estado inicial**, que especifica cómo se configura el juego al principio.
- To-Move(s): El jugador cuyo turno es moverse en el estado s.
- Acciones(s): El conjunto de movimientos legales en el estado s.
- Result(s,a): El **modelo de transición**, que define el estado resultante al tomar acción a en el estado s.
- Is-Terminal(s): Una **prueba terminal**, que es verdadera cuando el juego ha terminado y falsa de lo contrario. Los estados donde el juego ha terminado se llaman **estados terminales**.
- Utility(s,p): Una **función de utilidad** (también llamada función objetivo o función de recompensa), que define el valor numérico final para el jugador p si el juego termina en el estado terminal s. En el ajedrez, el resultado es una victoria, una derrota o empate, con valores de 1, 0 o 1/2.[2] Algunos juegos tienen una gama más amplia de posibles resultados, como los pagos en el backgammon que van de 0 a 192.

Modelo de transición
Prueba terminal
Estado terminal

Igual que en el Capítulo 3, el estado inicial, la función Acciones y la función Resultado definen el **grafo de espacio de estados**: un grafo donde los vértices son estados, las aristas son movimientos y un estado puede ser alcanzado por varios caminos. Como en el Capítulo 3, podemos superponer un **árbol de búsqueda** sobre una parte de ese grafo para determinar qué movimiento hacer. Definimos el **árbol de juego** completo como un árbol de búsqueda que sigue cada secuencia de movimientos hasta un estado terminal. El árbol de juego puede ser infinito si el espacio de estados es en sí mismo ilimitado o si las reglas le permiten repetir posiciones de modo infinito.

Grafo de espacio de estados
Árbol de búsqueda
Árbol de juego

La Figura 5.1 muestra parte del árbol de juego para el Gato (círculos y cruces). Desde el estado inicial, MAX tiene nueve movimientos posibles. El juego alterna entre la colocación de una X por parte de MAX y un O por parte de MIN hasta que llegamos a nodos hoja, propios a estados terminales, tales que un jugador tiene tres cuadrados en línea o todos están llenos. El número en cada nodo hoja indica el valor de utilidad del estado terminal desde el punto de vista de MAX; los valores altos son buenos para MAX y malos para MIN (de donde los jugadores toman sus nombres).

Para el Gato, el árbol de juego resulta pequeño, con menos de 9! = 362 880 nodos terminales (sólo 5 478 estados distintos). Pero para el ajedrez hay más de 10^{40} nodos, por ello es mejor considerar el árbol de juego como una construcción teórica que no se puede concretizar en el mundo físico.

---

[1] Algunos autores distinguen entre "juego de información imperfecta" como el póquer, donde los jugadores obtienen información privada sobre sus propias cartas que los otros jugadores desconocen, y "juego parcialmente observable" para referirse al StarCraft II, donde cada jugador ve el entorno cercano, pero no el entorno lejano.

[2] El ajedrez se considera un juego de "suma cero", aunque la suma de los resultados para los dos jugadores es +1 para cada juego, no cero. El término "suma constante" habría sido más preciso, pero "suma cero" es tradicional y tiene sentido si se imagina que cada jugador paga tarifa de entrada de 1/2.

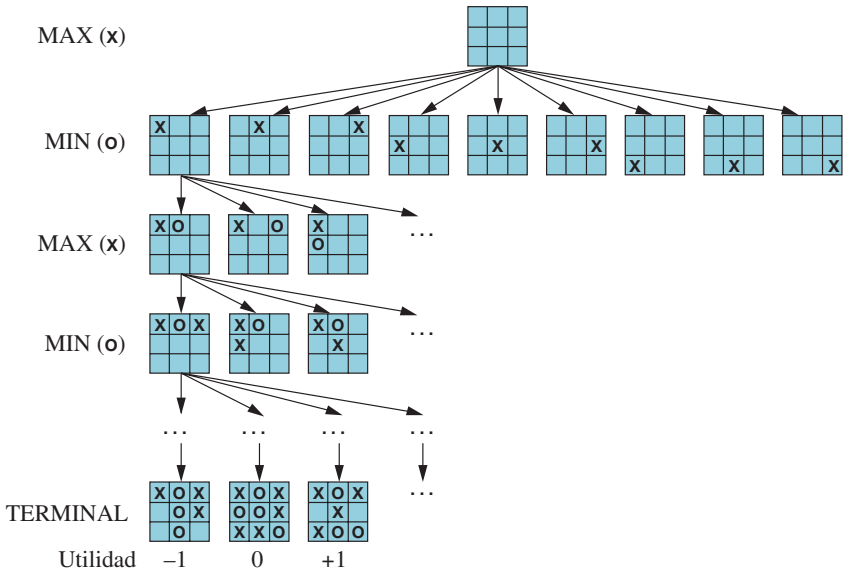


Figura 5.1 Un árbol de juego (parcial) para el juego Gato. El nodo superior es el estado inicial. MAX mueve primero al colocar una X en un cuadrado vacío. Mostramos parte del árbol, con movimientos alternos por MIN (O) y MAX (X), hasta que finalmente llegamos a estados terminales, a los que podemos asignar utilidades de acuerdo con las reglas del juego.

5.2 Decisiones óptimas en juegos

MAX quiere encontrar una secuencia de acciones que conduzcan a una victoria, pero MIN tiene algo que decir al respecto. Es decir, la estrategia de MAX debe ser un plan condicional, una estrategia contingente que especifique una respuesta a cada posible movimiento de MIN. En los juegos con resultado binario (ganar o perder), podríamos usar la búsqueda AND–OR (página 125) para generar el plan condicional. De hecho, para esos juegos, la definición de una estrategia ganadora es idéntica a la de una solución para un problema de planificación no determinista: en ambos casos, el resultado deseable debe garantizarse sin importar qué hace el "otro lado". Para los juegos con múltiples puntuaciones de resultados, necesitamos un algoritmo más general llamado **búsqueda minimax**.

Búsqueda minimax

Considere el juego de la Figura 5.2. Los posibles movimientos de MAX en el nodo raíz están etiquetados como a_1, a_2 y a_3. Las posibles respuestas a a_1 para MIN son b_1, b_2, b_3, etc. Este juego en particular termina luego de un movimiento de MAX y MIN. (Nota: En unos juegos, "mover" significa que ambos jugadores han realizado una acción; por ello, la palabra **jugada** (**ply**) se usa para significar inequívocamente un movimiento de un jugador, ello nos lleva a un nivel más profundo en el árbol del juego). Las utilidades de los estados terminales en este juego van de 2 a 14.

Jugada

Dado un árbol de juego, la estrategia óptima se puede determinar si se trabaja en el **valor minimax** de cada estado en el árbol, que escribimos como MINIMAX(s). Este valor es la utilidad (para MAX) de estar en ese estado, al *asumir que ambos jugadores juegan perfectamente* desde allí hasta el final del juego. El valor minimax de un estado terminal es sólo su utilidad. En un estado no terminal, MAX se mueve a un estado de valor máximo cuando es turno de MAX, y

Valor minimax

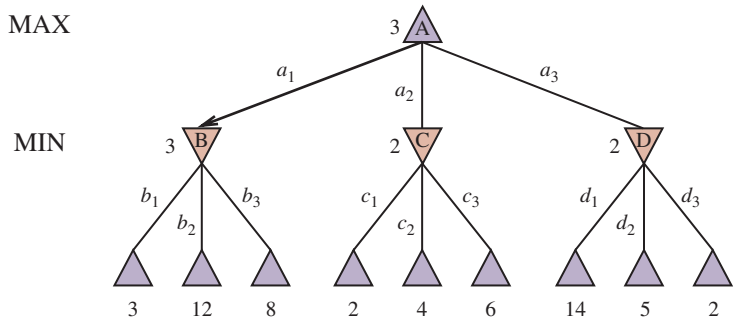


Figura 5.2 Un árbol de juego de dos capas. Los nodos △ son "nodos MAX", en los que es el turno de MAX para moverse, y los nodos ▽ son "nodos MIN". Los nodos terminales muestran los valores de utilidad para MAX; los otros nodos están etiquetados con sus valores minimax. El mejor movimiento de MAX en la raíz es a_1, porque conduce al estado con el valor minimax más alto, y la mejor respuesta de MIN es b_1, porque conduce al estado con el valor minimax más bajo.

MIN prefiere un estado de valor mínimo (es decir, valor mínimo para MAX y, por lo tanto, valor máximo para MIN). Entonces tenemos:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s,\text{MAX}) & \text{if IS-TERMINAL(s)} \\ \max_{a \in Acciones(s)} \text{MINIMAX}(\textbf{RESULTS}(s,a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in Acciones(s)} \text{MINIMAX}(\textbf{RESULTS}(s,a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

Aplicamos estas definiciones al árbol de juego en la Figura 5.2. Los nodos terminales en el nivel inferior obtienen sus valores de utilidad de la función de UTILIDAD del juego. El primer nodo MIN, etiquetado como *B*, tiene tres estados sucesores con valores 3, 12 y 8, por lo que su valor minimax es 3. Asimismo, los otros dos nodos MIN tienen un valor minimax de 2. El nodo raíz es un nodo MAX; sus estados sucesores tienen valores minimax de 3, 2 y 2; por ello tiene un valor minimax de 3. También podemos identificar la **decisión minimax** en la raíz: la acción a_1 es la elección óptima para MAX porque conduce a la decisión minimax al estado con el valor minimax más alto.

Decisión minimax

Esta definición de juego óptimo para MAX asume que MIN también juega de manera óptima. ¿Qué pasa si MIN no juega así? MAX lo hará al menos tan bien como contra un jugador óptimo, quizá mejor. Pero, eso no significa que sea mejor jugar el movimiento óptimo de minimax cuando se enfrenta a un oponente subóptimo. Piense una situación en la que el juego óptimo de ambos lados llevará a un empate, pero hay un movimiento arriesgado para MAX que lleva a un estado donde hay 10 posibles movimientos de respuesta por MIN que parecen razonables, pero 9 de ellos son una pérdida para MIN y uno es pérdida para MAX. Si MAX cree que MIN no tiene suficiente poder computacional para descubrir el movimiento óptimo, MAX podría intentar el movimiento arriesgado, en el sentido de que una probabilidad de 9/10 de ganar es mejor que un empate seguro.

5.2.1 El algoritmo de búsqueda minimax

Ahora que podemos calcular MINIMAX(*s*), podemos convertirlo en un algoritmo de búsqueda que encuentra el mejor movimiento para MAX probando todas las acciones y eligiendo aquella

```
function MINIMAX-SEARCH(game, state) returns una action
    player←game.TO-MOVE(state)
    value, move←MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns una pareja (utility, move)
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v← −∞
    for each a in game.ACTIONS(state) do
        v2, a2←MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move← v2, a
    return v, move

function MIN-VALUE(game, state) returns una pareja (utility, move)
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v← −∞
    for each a in game.ACTIONS(state) do
        v2, a2←MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move← v2, a
    return v, move
```

Figura 5.3 Algoritmo para calcular el movimiento óptimo utilizando minimax: el movimiento que conduce a un estado terminal con utilidad máxima, bajo la suposición de que el oponente juega para minimizar la utilidad. Las funciones MAX-VALUE y MIN-VALUE recorren todo el árbol del juego, hasta las hojas, para determinar el valor de cada estado y el movimiento para llegar allí.

cuyo estado resultante tenga el valor MINIMAX más alto. La Figura 5.3 muestra el algoritmo, un algoritmo recursivo que avanza hasta las hojas del árbol y luego **retrocede** los valores minimax a través del árbol conforme se desenrolla la recursión. Por ejemplo, en la Figura 5.2, el algoritmo primero se adentra en los tres nodos inferiores izquierdos y usa la función UTILITY en ellos para descubrir que sus valores son 3, 12 y 8, respectivamente. Luego toma el mínimo de estos valores, 3, y lo devuelve como el valor del nodo *B*. Un proceso similar devuelve hacia arriba los valores de 2 para *C* y 2 para *D*. Al final, tomamos el máximo de 3, 2 y 2 para obtener el valor devuelto de 3 para el nodo raíz.

El algoritmo minimax hace una exploración completa de profundidad primero del árbol de juego. Si la profundidad máxima del árbol es m y hay b movimientos legales en cada punto, entonces la complejidad temporal del algoritmo minimax es $O(b^m)$. La complejidad espacial es $O(bm)$ para un algoritmo que genera todas las acciones a la vez, o $O(m)$ para uno que genera una acción a la vez (ver página 80). La complejidad exponencial hace que MINIMAX sea impráctico para juegos complejos; así, el ajedrez tiene un factor de ramificación de casi 35 y el juego promedio tiene una profundidad de unas 80 jugadas, y no es factible buscar en $35^{80} \approx 10^{123}$ estados. Pero, MINIMAX sirve como base para el análisis matemático de juegos. Al aproximar el análisis minimax de varias maneras, podemos derivar algoritmos más prácticos.

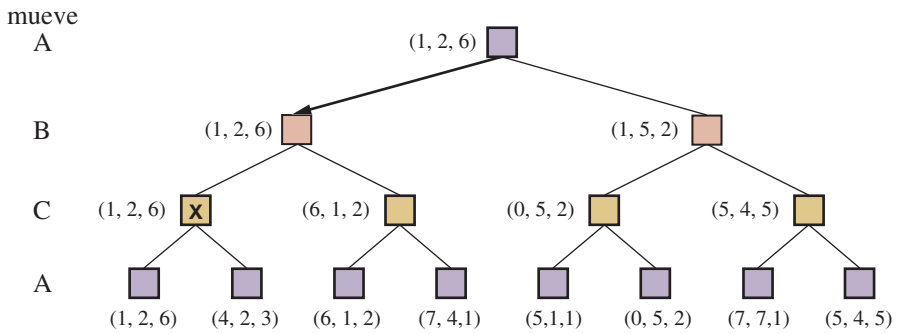


Figura 5.4 Primeros tres niveles de un árbol de juegos con tres jugadores (A,B,C). Cada nodo etiquetado con valores desde el punto de vista de cada jugador. El mejor movimiento es marcado en la raíz.

5.2.2 Decisiones óptimas en juegos multijugador

Muchos juegos populares permiten más de dos jugadores. Veamos cómo extender la idea de minimax a juegos multijugador. Esto es sencillo desde el punto de vista técnico, pero plantea algunos nuevos problemas conceptuales interesantes.

Primero, necesitamos reemplazar el valor único para cada nodo con un vector de valores. Por ejemplo, en un juego de tres jugadores con los jugadores A, B y C, se asocia un vector $\langle v_A, v_B, v_C \rangle$ con cada nodo. Para estados terminales, este vector da la utilidad del estado desde el punto de vista de cada jugador. (En juegos de dos jugadores y suma cero, el vector de dos elementos se reduce a un solo valor porque los valores son siempre opuestos). La forma más sencilla de implementar esto es que la función UTILITY devuelva un vector de utilidades.

Ahora consideramos estados no terminales. Piense el nodo marcado X en el árbol de juego de la Figura 5.4. En ese estado, el jugador C elige qué hacer. Ambas opciones llevan a estados terminales con vectores de utilidad $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$ y $\langle v_A = 4, v_B = 2, v_C = 3 \rangle$. Como 6 es mayor que 3, C debería elegir el primer movimiento. Esto es que si se alcanza el estado X, el juego posterior llevará a un estado terminal con utilidades $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$. Entonces, el valor devuelto de X es este vector. En general, el valor devuelto de un nodo n es el vector de utilidad del estado sucesor con el valor más alto para el jugador que elige en n.

Cualquiera que juegue multijugador, como Diplomacy o Settlers of Catan, rápido descubre que hay más en disputa que en juegos de dos jugadores. Estos juegos en general implican **alianzas**, formales o informales, entre los jugadores. Las alianzas se hacen y se rompen a medida que avanza el juego. ¿Cómo podemos entender tal comportamiento? ¿Son las alianzas una consecuencia natural de las estrategias óptimas para cada jugador en un juego multijugador? Resulta que pueden serlo.

Alianzas

Por ejemplo, suponga que A y B están en posiciones débiles y C en una posición más fuerte. Entonces, a menudo es óptimo para ambos A y B atacar a C en lugar de entre sí, para que C no los destruya individualmente. Así, la colaboración surge del comportamiento puramente egoísta. Claro, tan pronto como C se debilite bajo el ataque conjunto, la alianza pierde su valor y A o B podrían violar el acuerdo.

En unos casos, las alianzas explícitas sólo concretan lo que habría sucedido de todos modos. En otros, un estigma social se adhiere a romper una alianza, por ello los jugadores deben

balancear la ventaja inmediata de romper una alianza contra la desventaja a largo plazo: ser percibido como poco confiable. En la Sección 18.2 profundizará sobre estas dificultades.

Si el juego no es de suma cero, la colaboración también puede ocurrir con sólo dos jugadores. Suponga, por ejemplo, que hay un estado terminal con utilidades $\langle v_A = 1\,000, v_B = 1\,000 \rangle$ y que 1 000 es la utilidad máxima posible para cada jugador. Entonces, la estrategia óptima es que ambos jugadores hagan todo lo posible para alcanzar este estado, es decir, los jugadores automáticamente cooperan para lograr un objetivo mutuamente deseable.

5.2.3 Poda alfa-beta

El número de estados del juego es exponencial en la profundidad del árbol. Ningún algoritmo puede eliminar por completo el exponente, pero a veces podemos reducirlo a la mitad, calculando la decisión minimax correcta sin examinar cada estado mediante la **poda** (ver página 90) de grandes partes del árbol que no hacen diferencia en el resultado. La técnica particular que examinamos se llama **poda alfa-beta**.

Poda alfa-beta

Consideremos nuevamente el árbol de juego de dos capas de la Figura 5.2. Vamos a través del cálculo de la decisión óptima una vez más, y preste atención cuidadosa a lo que sabemos en cada punto del proceso. Los pasos se explican en la Figura 5.5. El resultado es que podemos identificar la decisión minimax sin evaluar nunca dos de los nodos hoja.

Otra forma de ver esto es como una simplificación de la fórmula para MINIMAX. Deje que los dos sucesores no evaluados del nodo C en la Figura 5.5 tengan valores x e y. Entonces el valor del nodo raíz está dado por

$$\begin{aligned} \text{MINIMAX}(root) &= \max(\min(3,12,8),\min(2,x,y),\min(14,5,2)) \\ &= \max(3,\min(2,x,y),2) \\ &= \max(3,z,2) \qquad \text{donde } z = \min(2,x,y) \le 2 \\ &= 3. \end{aligned}$$

En otras palabras, el valor de la raíz y, por lo tanto, la decisión minimax son *independientes* de los valores de las hojas x e y, y por lo tanto pueden ser podados.

La poda alfa-beta se puede aplicar a árboles de cualquier profundidad, y a menudo es posible podar subárboles enteros y no sólo hojas. El principio general es el siguiente: considere un nodo n en algún lugar del árbol (vea la Figura 5.6), tal que el jugador tiene la opción de moverse a n. Si el jugador tiene una mejor opción en el mismo nivel (por ejemplo, m' en la Figura 5.6) o en cualquier punto más arriba en el árbol (como, m en la Figura 5.6), entonces el jugador nunca se moverá a n. Entonces, una vez que hemos descubierto suficiente sobre n (examinando algunos de sus descendientes) para llegar a esta conclusión, podemos podarlo.

Recuerde que la búsqueda minimax es de profundidad, por ello en todo momento sólo debemos considerar los nodos a lo largo de un solo camino en el árbol. La poda alfa-beta recibe su nombre de los dos parámetros adicionales en MAX-VALUE ($estado$,α,β) (ver Figura 5.7) que describen límites en los valores devueltos que aparecen a lo largo de todo el camino:

α = el valor de la mejor opción (es decir, de mayor valor) encontrado hasta ahora en cualquier punto de elección a lo largo del camino para MAX. Piense: α = "al menos".

β = el valor de la mejor opción (es decir, de menor valor) encontrado hasta ahora en cualquier punto de elección a lo largo del camino para MIN. Piense: β = "como máximo".

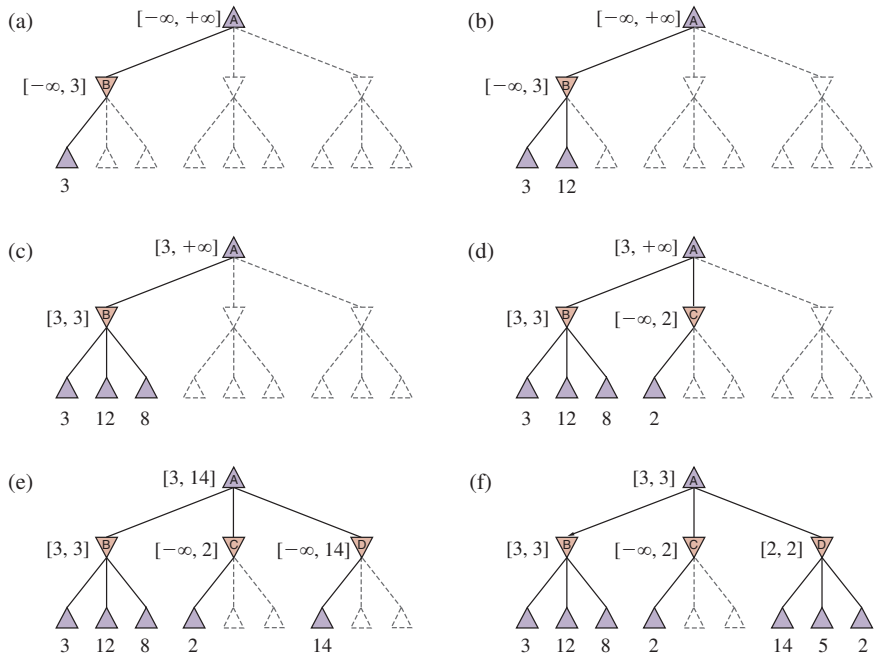


Figura 5.5 Etapas en el cálculo de la decisión óptima para el árbol de juego en la Figura 5.2. En cada punto, mostramos el rango de valores posibles para cada nodo. (a) La primera hoja debajo de *B* tiene valor 3. Entonces, *B*, que es un nodo MIN, tiene un valor *como máximo* 3. (b) La segunda hoja debajo de *B* tiene un valor de 12; MIN evitaría este movimiento, por ello el valor de *B* aún es máximo 3. (c) La tercera hoja debajo de *B* tiene valor de 8; hemos visto todos los estados sucesores de *B*, así el valor de *B* es 3 exacto. Ahora inferimos que el valor de la raíz es *al menos* 3, porque MAX tiene una opción que vale 3 en la raíz. d) La primera hoja debajo de *C* tiene el valor 2. Por ello, *C*, un nodo MIN, tiene un valor *máximo de* 2. Pero sabemos que *B* vale por 3, entonces MAX nunca elegiría *C*. Y no tiene sentido mirar los otros estados sucesores de un ejemplo de poda alfa-beta. e) La primera hoja debajo de *D* tiene valor 14, por ello *D* vale *máximo* 14. Esto persiste más alto que la mejor alternativa de MAX (es decir, 3), por ello debemos seguir explorando los estados sucesores de *D*. Vea también que ahora tenemos límites en todos los sucesores de la raíz, entonces el valor de la raíz también es máximo 14. (f) El segundo sucesor de *D* vale 5, así que de nuevo debemos seguir explorando. El tercer sucesor vale 2, por ello ahora *D* vale 2. La decisión de MAX en la raíz es moverse a *B*, y dar un valor de 3.

La búsqueda alfa-beta actualiza los valores de α y β a medida que avanza y poda las ramas restantes en un nodo (es decir, finaliza la llamada recursiva) tan pronto como se sabe que el valor del nodo actual es peor que el valor actual de α o β para MAX o MIN, respectivamente. El algoritmo completo se muestra en la Figura 5.7. La Figura 5.5 traza el progreso del algoritmo en un árbol de juego.

5.2.4 Orden de movimiento

La efectividad de la poda alfa-beta depende en gran medida del orden en que se examinan los estados. Por ejemplo, en la Figura 5.5(e) y (f), no pudimos podar ningún sucesor de *D* en absoluto porque los peores sucesores (desde el punto de vista de MIN) se generaron primero. Si el tercer sucesor de

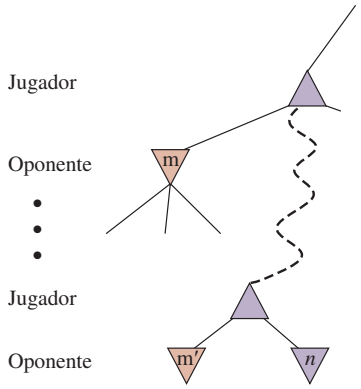


Figura 5.6 El caso general para la poda alfa-beta. Si m o m' es mejor que n para Jugador, nunca llegaremos a n en el juego.

```
function ALPHA-BETA-SEARCH(game, state) returns una acción
  player←game.TO-MOVE(state)
  value,move←MAX-VALUE(game, state,−∞,+∞)
  return move

function MAX-VALUE(game, state,α, β) returns una pareja (utility, move)
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v←−∞
  for each a in game.ACTIONS(state) do
    v2, a2←MIN-VALUE(game, game.RESULT(state, a),α, β)
    if v2 > v then
      v, move←v2, a
      α←MAX(α, v)
    if v ≥ β then return v, move
  return v, move

function MIN-VALUE(game, state,α, β) returns una pareja (utility, move)
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v←+∞
  for each a in game.ACTIONS(state) do
    v2, a2←MAX-VALUE(game, game.RESULT(state, a),α, β)
    if v2 < v then
      v, move←v2, a
      β←MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Figura 5.7 Algoritmo de búsqueda alfa-beta. Vea que estas funciones son las mismas que las de búsqueda MINIMAX en la Figura 5.3, excepto que mantenemos límites en las variables α y β, y los usamos para cortar la búsqueda cuando un valor está fuera de los límites.

D se hubiera generado primero, con valor 2, habríamos podido podar los otros dos sucesores. Esto sugiere que valdría la pena examinar primero los sucesores que probablemente sean los mejores.

Si esto se hiciera perfectamente, alfa-beta sólo necesitaría examinar $O(b^{m/2})$ nodos para elegir el mejor movimiento, en vez de $O(b^m)$ para minimax. Es decir, que la tasa de ramificación efectiva se convierte en $\sqrt{b}$ en lugar de *b; para* el ajedrez, cerca de 6 en lugar de 35. O sea, alfa-beta con una ordenación de movimientos perfecta resolverá un árbol casi dos veces más profundo que minimax en la misma cantidad de tiempo. Con una aleatoria ordenación de movimientos, el número total de nodos examinados será de casi $O(b^{3m/4})$ para *b* moderado. Obvio, ahora no podemos lograr una ordenación de movimientos *perfecta*, ¡en ese caso la función de ordenación podría usarse para un juego perfecto! Pero a menudo podemos acercarnos bastante. En el ajedrez, una función de ordenación muy simple (como intentar primero capturas, luego amenazas, después movimientos hacia adelante y luego movimientos hacia atrás) lleva a estar dentro de un factor de 2 del mejor resultado teórico $O(^{bm/2})$.

Agregar esquemas de ordenación de movimientos dinámicos, como intentar primero los mejores los movimientos del pasado, nos acerca al límite teórico. El pasado podría ser el movimiento anterior, a menudo las mismas amenazas permanecen, o podría provenir de la exploración previa del movimiento actual mediante un proceso de **profundización iterativa** (página 80). Primero, busca una profundidad de un nivel y registra la clasificación de los movimientos en función de sus evaluaciones. Luego la búsqueda es de mayor profundidad, al utilizar la clasificación previa para informar la ordenación de movimientos, etc. El tiempo de búsqueda aumentado por la profundización iterativa es compensado por una mejor ordenación de movimientos. Los mejores movimientos se llaman **movimientos asesinos**, y hacerlos primero es heurística de movimiento asesino.

Movimientos asesinos

En la Sección 3.3.3, señalamos que los caminos redundantes a estados repetidos causan aumento exponencial en el costo de búsqueda, y que mantener una tabla de estados previamente alcanzados puede abordar este problema. En la búsqueda del árbol de juego, los estados repetidos ocurren debido a **transposiciones,** diferentes permutaciones de transposición de la secuencia de movimiento que terminan en la misma posición, y el problema es abordado con una **tabla de transposición** que almacena en caché el valor heurístico de los estados.

Transposiciones

Tabla de transposición

Por ejemplo, suponga que White tiene un movimiento w_1 que puede ser respondido por Black con b_1 y un movimiento no relacionado w_2 en el otro lado del tablero que puede ser respondido por b_2, y que buscamos la secuencia de movimientos $[w_1,b_1,w_2,b_2]$; llamemos al estado resultante *s*. Después de explorar un gran subárbol debajo de *s*, encontramos su valor devuelto, que almacenamos en la tabla de transposición. Cuando más tarde buscamos la secuencia de movimientos $[w_2,b_2,w_1,b_1]$, terminamos en *s* de nuevo, y podemos extraer el valor de la tabla en vez de repetir la búsqueda. En el ajedrez, el uso de tablas de transposición es efectivo, y permite duplicar la profundidad de búsqueda alcanzable en la misma cantidad de tiempo.

Pese a la poda alfa-beta y un ordenamiento de movimientos inteligente, minimax no funcionará para el ajedrez o el Go, porque aún hay demasiados estados para explorar en el tiempo disponible. En su artículo sobre juegos de computadora, *Programming a Computer for Playing Chess* (Shannon, 1950), Claude Shannon reconoció este problema y propuso dos estrategias: **Estrategia tipo A,** considera todos los movimientos posibles hasta cierta profundidad en el árbol de búsqueda, y luego usa una función de evaluación heurística para estimar la utilidad de los estados en esa profundidad. Explora *una porción amplia* pero superficial del árbol. **Estrategia tipo B**, ignora movimientos que parecen malos, sigue líneas prometedoras "hasta donde sea posible", y explora una porción *profunda pero estrecha* del árbol.

Estrategia tipo A

Estrategia tipo B

Históricamente, la mayoría de los programas de ajedrez han sido de Tipo A (revisados en la siguiente sección), pero los programas de Go suelen ser Tipo B (vea la Sección 5.4), porque el factor de ramificación es mayor en Go. Recién, los programas Tipo B lograron un juego a nivel de campeón mundial en una variedad de juegos, incluido el ajedrez (Silver *et al.*, 2018).

5.3 Búsqueda de árbol alfa-beta heurística

Para usar nuestro limitado tiempo de computación, podemos cortar la búsqueda temprano, aplicar una **función de evaluación** heurística a los estados y tratar los nodos no terminales como si lo fueran. Es decir, reemplazamos la función UTILITY con EVAL, que estima la utilidad de un estado. Y reemplazamos la prueba terminal por una **prueba de corte**, que debe devolver verdadero para los estados terminales, o de lo contrario es libre de decidir cuándo cortar la búsqueda con base en la profundidad de búsqueda y toda propiedad del estado que elija considerar. Eso da la fórmula H-MINIMAX(s, d) para el valor heurístico minimax del estado s en la profundidad de búsqueda d:

Prueba de corte

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s,\text{MAX}) & \text{if IS-CUTOFF}(s,d) \\ \max_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a),d+1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a),d+1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

5.3.1 Funciones de evaluación

Una función de evaluación heurística EVAL(s,p) devuelve una *estimación* de la utilidad esperada del estado s para el jugador p, igual que las funciones heurísticas del Capítulo 3 devuelven una estimación de la distancia al objetivo. Para estados terminales, debe ser que EVAL(s,p) = UTILITY(s,p) y para estados no terminales, la evaluación debe dar un valor entre pérdida y victoria: UTILITY(*pérdida*,p) ≤ EVAL(s,p) ≤ UTILITY(*victoria*,p).

Además de esos requisitos, ¿qué hace a una función de evaluación buena? Primero, ¡los cálculos no debe dilatar mucho! (¡Lo importante es buscar más rápido!) Segundo, esta función debe estar bien correlacionada con las posibilidades reales de ganar. Uno bien podría preguntarse sobre la frase "posibilidades de ganar". Como el ajedrez no es un juego de azar: conocemos el estado actual con certeza, y no hay dados involucrados; si ningún jugador comete un error, el resultado está predeterminado. Pero si la búsqueda debe ser interrumpida en estados no terminales, entonces por necesidad el algoritmo será *incierto* sobre los resultados finales de esos estados (aunque la incertidumbre podría resolverse con recursos informáticos infinitos).

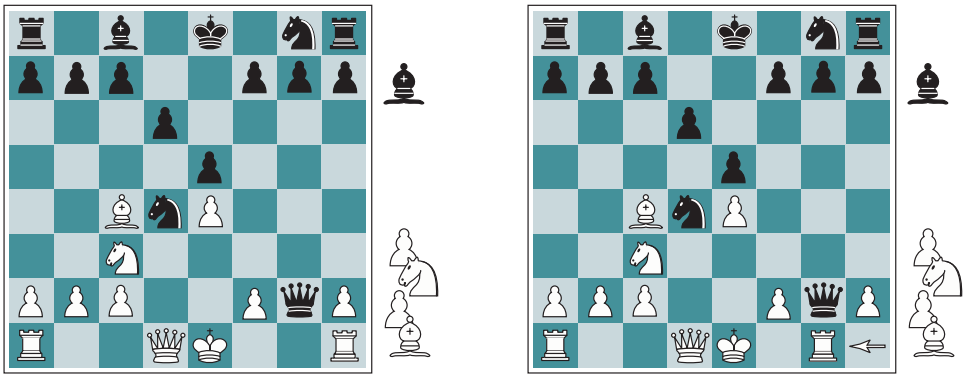
Características

Hagamos esta idea más concreta. La mayoría de las funciones de evaluación calculan varias **características** del estado; por ejemplo, en el ajedrez, tendríamos características para el número de peones blancos y negros, reinas blancas y negras, etc. Las características, tomadas en conjunto, definen varias *categorías* o *clases de equivalencia* de estados: éstos tienen los mismos valores en cada categoría para todas las características. Es decir, una categoría contendría todos los finales de dos peones contra uno. Cualquier categoría contendrá estados que conducen (con juego perfecto) a victorias, otros que llevan a empates y algunos a derrotas.

La función de evaluación no sabe qué estados son cuáles, pero puede devolver un solo valor que estima la *proporción* de estados con cada resultado. Por ejemplo, suponga que nuestra experiencia sugiere que 82% de los estados de la categoría los dos peones versus un peón lleva a una victoria (utilidad +1); 2 % a pérdida (0), y 16 % a empate (1/2). Entonces, una evaluación razonable para los estados en la categoría es el **valor esperado**: (0.82 × +1) + (0.02 × 0) + (0.16 × 1/2) = 0.90. En principio, el valor esperado se determina para cada categoría de estados, que resulta en una función de evaluación que funciona para todo estado.

Valor esperado

En la práctica, este tipo de análisis requiere muchas categorías y, por ello, mucha experiencia para estimar todas las probabilidades. En su lugar, la mayoría de las funciones de evaluación calculan contribuciones numéricas separadas para cada característica y luego las



(a) Juegan las blancas (b) Juegan las blancas

Figura 5.8 Dos posiciones de ajedrez que difieren sólo en la posición de la torre en la parte inferior derecha. En (a), las Negras tiene una ventaja de un caballo y dos peones, lo que debería ser suficiente para la victoria. En (b), las Blancas capturarán la reina, y logran una ventaja que debería ser suficiente para ganar.

combinan para encontrar el valor total. Por siglos, los ajedrecistas desarrollaron formas de juzgar el valor de una posición con sólo esta idea. Por ejemplo, los libros de ajedrez básico dan un **valor material** aproximado para cada pieza: un peón vale 1, un caballo o alfil 3, una torre 5 y la reina 9. Otras características como "buena estructura de peones" y "seguridad del rey" podrían valer medio peón. Estos valores sólo se suman para obtener la evaluación de la posición.

Valor material

Matemáticamente, este tipo de función de evaluación se denomina **función lineal ponderada** porque se expresa como

Función lineal ponderada

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^{n} w_i f_i(s),$$

donde cada f_i es una característica de la posición ("número de alfiles blancos") y cada w_i es un peso (que indica cuán importante es esa característica). Los pesos deben normalizarse para que la suma siempre esté dentro del rango de una derrota (0) a una victoria (+1). Una ventaja segura equivalente a un peón da una probabilidad sustancial de ganar, una ventaja equivalente a tres peones debería dar una victoria casi segura, como se en la Figura 5.8 (a). Dijimos que la función de evaluación debe estar bien correlacionada con las posibilidades reales de ganar, pero no tiene por qué estar correlacionada linealmente: si el estado s tiene el doble de probabilidades de ganar que el estado s' no requerimos que EVAL(S) sea el doble de EVAL(S'); todo lo que requerimos es que EVAL(S)> EVAL(S').

Sumar los valores de las características parece razonable de hacer, pero de hecho implica una fuerte suposición: que la contribución de cada característica es *independiente* de los valores de las otras características. Así, los programas actuales para ajedrez y otros juegos también usan combinaciones *no lineales* de características. Por ejemplo, un par de alfiles valdría más de dos veces el valor de uno solo, y un alfil vale más en el final del juego que antes, cuando la característica del *número de movimientos* es alta o la característica del *número de piezas restantes* es baja.

¿De dónde vienen las características y los pesos? No son parte de las reglas del ajedrez, pero sí de la cultura de la experiencia humana de jugar ajedrez. En juegos donde este tipo de experiencia

no está disponible, los pesos de la función de evaluación pueden ser estimados por las técnicas de aprendizaje automático del Capítulo 22. La aplicación de estas técnicas ha confirmado que un alfil vale casi tres peones, y parece que siglos de experiencia humana pueden ser replicados en sólo unas horas de aprendizaje automático.

5.3.2 Corte de la búsqueda

El siguiente paso es modificar ALPHA-BETA-SEARCH para que llame a la función heurística EVAL cuando sea apropiado cortar la búsqueda. Reemplazamos ambas líneas en la Figura 5.7 que mencionan IS-TERMINAL con la siguiente línea:

if *game*.IS-CUTOFF(*state*,*depth*) **then return** *game*.EVAL(*state*, *player*), *null*

También debemos llevar un registro de la *profundidad* actual de modo que se incremente en cada llamada recursiva. El enfoque más directo para controlar la cantidad de búsqueda es establecer un límite de profundidad fijo para que IS-CUTOFF (*state*, *depth*) devuelva *verdadero* para todas las *profundidades* mayores que algún valor fijo *d* (así como para todos los estados terminales). La profundidad *d* se elige de modo que se seleccione un movimiento dentro del tiempo asignado. Un mejor enfoque es aplicar profundización iterativa. (Ver Capítulo 3.) Cuando se acaba el tiempo, el programa devuelve el movimiento seleccionado por la búsqueda más profunda completada. Como un bono, si en cada ronda de profundización iterativa mantenemos las entradas en la tabla de transposición, las rondas posteriores serán más rápidas, y usaremos evaluaciones para mejorar el orden de los movimientos.

Estos enfoques simples llevan a errores debido a la naturaleza aproximada de la función de evaluación. Considere de nuevo la función de evaluación simple para el ajedrez basada en la ventaja material. Suponga que el programa busca hasta el límite de profundidad y alcanza la posición en la Figura 5.8 (b), donde las Negras están adelante por un caballo y dos peones. Informaría esto como el valor heurístico del estado, al declarar que el estado es una victoria probable de las Negras. Pero el siguiente movimiento de las Blancas captura la reina de las Negras de modo gratuito. Así, la posición es realmente favorable para las Blancas, pero esto sólo se ve mirando hacia adelante.

Reposo

La función de evaluación sólo debe aplicarse a posiciones en **reposo**, es decir, posiciones donde no haya movimientos pendientes (como capturar a la reina) que cambiarían radicalmente la evaluación. Para posiciones no en reposo, IS-CUTOFF devuelve falso y la búsqueda continúa hasta alcanzar posiciones en reposo. Esta **búsqueda de reposo** adicional a veces se limita a considerar sólo ciertos tipos de movimientos, como movimientos de captura, que rápido resolverán las incertidumbres en la posición.

Búsqueda de reposo

Efecto horizonte

El **efecto horizonte** es más difícil de eliminar. Surge si el programa enfrenta un movimiento del oponente que causa daño grave y que es inevitable, pero que puede ser temporalmente evitado mediante tácticas de retraso. Piense la posición de ajedrez en la Figura 5.9. Es claro que no hay forma de que el alfil negro escape. Es decir, la torre blanca puede capturarlo moviéndose a h1, luego a a1, después a a2; una captura a una profundidad de 6 jugadas.

Pero las Negras tienen una secuencia de movimientos que empuja la captura del alfil "más allá del horizonte". Suponga que las Negras buscan en una profundidad de 8 jugadas. La mayoría de sus movimientos llevarán a la eventual captura del alfil y, por lo tanto, serán marcados como movimientos "malos". Pero las Negras también considerarán la secuencia de movimientos que inicia atacando al rey con un peón, y animando al rey a capturar el peón. Las Negras puede hacer lo mismo con un segundo peón. Eso ocupa suficientes movimientos para que la captura del alfil no sea descubierta durante el resto de la búsqueda de las Negras. Éstas piensan que la línea de juego

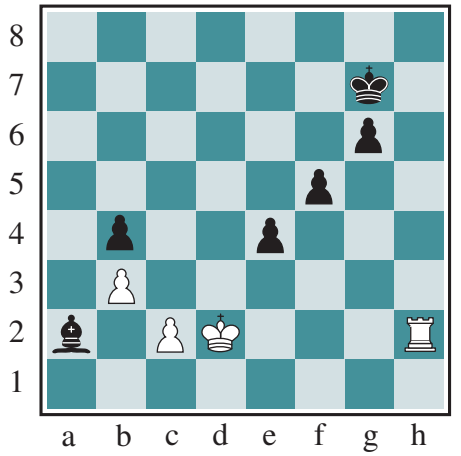


Figura 5.9 Efecto horizonte. Con el turno de las Negras, el alfil negro está condenado. Pero las Negras pueden evitar ese evento al hacer jaque al rey blanco con sus peones, animando a éste a capturar los peones. Esto empuja la pérdida inevitable del alfil más allá del horizonte. Así los sacrificios de peones son vistos por el algoritmo de búsqueda como buenos movimientos, no malos.

ha salvado al alfil al precio de dos peones, cuando lo único que ha hecho es desperdiciar peones y empujar la captura inevitable del alfil más allá del horizonte que las Negras ven.

Extensiones singulares

Una estrategia para mitigar el efecto horizonte es permitir **extensiones singulares**, movimientos "claramente mejores" que todos los demás en una posición determinada, aun si la búsqueda se cortara en ese punto. En este ejemplo, una búsqueda revelaría que tres movimientos del alfil blanco —h2 a h1, h1 a a1, y luego a1 capturando el alfil en a2— son cada uno a su vez mejores movimientos, por ello aun si una secuencia de movimientos de peones nos lleva al horizonte, estos movimientos claramente mejores tendrán la oportunidad de extender la búsqueda. Esto hace que el árbol sea más profundo, pero como suele haber pocas extensiones singulares, la estrategia no agrega muchos nodos totales al árbol, ha demostrado ser efectiva en la práctica.

5.3.3 Poda hacia adelante

Poda hacia adelante

La poda alfa-beta poda las ramas del árbol que no pueden tener efecto en la evaluación final, pero la **poda hacia adelante** poda los movimientos que parecen malos, que quizá podrían ser buenos. Así, la estrategia ahorra tiempo de computo con el riesgo de cometer un error. En palabras de Shannon, esta es una estrategia Tipo B. Claro, la mayoría de los ajedrecistas humanos hacen esto, consideran sólo unos movimientos desde cada posición (al menos conscientemente).

Un enfoque para la poda hacia adelante es la **búsqueda de haz** (ver página 115): en cada jugada, considere sólo un "haz" de los n mejores movimientos (según la función de evaluación) en vez de considerar todos los movimientos posibles. Aunque este enfoque es muy peligroso porque no hay garantía de que el mejor movimiento no sea podado.

El algoritmo PROBCUT, o corte probabilístico, (Buro, 1995) es una versión de poda hacia adelante de la búsqueda alfa-beta que usa estadísticas mediante la experiencia previa para reducir la posibilidad de que el mejor movimiento será podado. La búsqueda alfa-beta poda cualquier nodo que esté *probablemente* fuera de la ventana actual (α, β). PROBCUT también poda nodos

que *quizás* estén fuera de la ventana. Calcule esta probabilidad y haga una búsqueda superficial para calcular el valor devuelto v de un nodo y luego usa la experiencia pasada para estimar cuán probable es que un puntaje de v a una profundidad d en el árbol esté fuera de (α,β). Buro aplicó esta técnica a su programa Othello, LOGISTELLO, y descubrió que una versión de su programa con PROBCUT venció a la versión regular 64 % de las veces, aun cuando se le dio el doble de tiempo.

Reducción tardía de movimientos

Otra técnica, la **reducción tardía de movimientos**, funciona con la hipótesis del ordenamiento de movimientos bien hecho y, entonces, los movimientos posteriores en la lista de posibles movimientos son menos propensos a ser buenos movimientos. En vez de poda completa, reducimos la profundidad a la que los buscamos, y así se ahorra tiempo. Si la búsqueda reducida regresa con un valor por encima del valor α actual, volvemos a ejecutar la búsqueda con la profundidad total.

Combinar las técnicas descritas aquí resulta en un programa que juega ajedrez (u otros juegos) de manera creíble. Suponga que se implementó una función de evaluación para el ajedrez, una prueba de corte razonable con una búsqueda de estabilidad. Y suponga que tras meses de tediosa manipulación de bits, podemos generar y evaluar casi un millón de nodos por segundo en la PC más moderna. El factor de ramificación para el ajedrez es de unos 35, en promedio, y 35^5 es de cerca de 50 millones; por ello, si usáramos la búsqueda minimax, miraríamos hacia adelante sólo cinco jugadas en casi un minuto de cálculo; las reglas de la competencia no darían suficiente tiempo para buscar seis jugadas. Aunque no es incompetente, tal programa sería derrotado por un jugador de ajedrez humano promedio, que a veces planifica seis u ocho jugadas por adelantado.

Con búsqueda α-β y una gran tabla de transposición llegamos a casi 14 jugadas, que resulta en nivel de juego experto. Aun si cambiáramos nuestra PC por una con 8 GPUs, lograríamos más de mil millones de nodos por segundo, pero para lograr estatus de experto necesitaríamos una función de evaluación ajustada en extenso y una gran base de datos de movimientos finales. Los mejores programas de ajedrez, como STOCKFISH, tienen todo esto, y suelen alcanzar profundidad de 30 o más en el árbol de búsqueda, que superan con fácil la capacidad de un jugador humano.

5.3.4 Búsqueda versus tablas de búsqueda

Parece excesivo que un programa de ajedrez inicie un juego considerando un árbol de mil millones de estados de juego, sólo para concluir que jugará peón a e4 (el primer movimiento más popular). Los libros que describen un buen juego en la apertura y el final en el ajedrez han estado disponibles durante más de un siglo (Tattersall, 1911). Así que no sorprende que muchos programas usen *tablas de búsqueda* en vez de búsqueda para la apertura y el final de un juego.

Para aperturas, la computadora depende de la experiencia de los humanos. El mejor consejo de los expertos humanos sobre cómo jugar cada apertura se puede copiar de libros e ingresarlo en tablas para uso de la computadora. Además, éstas pueden recopilar estadísticas de una base de datos de juegos antes jugados para ver qué secuencias de apertura conducen más a menudo a una victoria. Para los primeros movimientos hay pocas posibilidades, la mayoría de las posiciones estarán en la tabla. En general, luego de 10 o 15 movimientos, terminamos en una posición rara vez vista, y el programa debe cambiar de tablas de búsqueda a búsqueda.

Casi al final del juego hay de nuevo menos posiciones posibles y, por ello, es más fácil hacer la búsqueda. Pero aquí es la computadora la de la experiencia: su análisis de finales va mucho más allá de las habilidades humanas. Los humanos novatos pueden ganar un final de rey y torre contra rey (KRK) si siguen reglas simples. Otros finales, como rey, alfil y caballo contra rey (KBNK), son difíciles de dominar y no tienen una descripción sucinta de la estrategia.

La computadora puede *resolver* por completo el final del juego creando una **política**, un mapeo de todos los estados posibles al mejor movimiento en ese estado. Luego, la computadora

juega perfectamente buscando la jugada correcta en esta tabla, construida mediante la búsqueda minimax **retrógrada**: comienza pensando todas las formas de colocar las piezas KBNK en el tablero. Las posiciones victoria para las blancas las marca como tales; luego invierte las reglas del ajedrez para hacer movimientos inversos. Todo movimiento de blancas que, sin importar cómo respondan las negras, termine en posición marcada como victoria, también es victoria. Continúa esta búsqueda hasta que todas las posiciones posibles se resuelvan como victoria, derrota o empate, y tendrá una tabla de búsqueda infalible para todo final de juego con esas piezas. Esto se ha hecho para finales de KBNK y para todos los finales con siete o menos piezas. Las tablas tienen 400 billones de posiciones, pero una tabla de ocho piezas requeriría 40 billones de posiciones.

Retrógrada

5.4 Búsqueda de árbol de Monte Carlo

El juego de Go ilustra dos debilidades principales de la búsqueda de árbol alfa-beta heurística: primero, Go tiene un factor de ramificación que comienza en 361, lo que significa que esta búsqueda se limitaría a sólo 4 o 5 jugadas. Segundo, es difícil definir una buena función de evaluación para Go porque el valor material no es un indicador fuerte y la mayoría de las posiciones están en flujo hasta el final del juego. En respuesta a ambos desafíos, los programas modernos de Go han abandonado la búsqueda alfa-beta para usar la estrategia **búsqueda de árbol de Monte Carlo (MCTS)**.[3]

Búsqueda de árbol de Monte Carlo (MCTS)

La estrategia básica de MCTS no usa una función de evaluación heurística. En su lugar, el valor de un estado se estima como la utilidad promedio sobre un número de **simulaciones** de juegos completos. Simulación comenzando desde el estado (también llamada **playout** o **rollout**) elige movimientos primero para un jugador, luego para el otro, y repite hasta que se alcance una posición terminal. En ese punto, las reglas del juego (no las heurísticas falibles) determinan quién ha ganado o perdido, y por qué puntuación. Para juegos en los que los únicos resultados son una victoria o una derrota, "utilidad promedio" es lo mismo que "porcentaje de victorias".

Simulaciones

Playout

Rollout

¿Cómo elegimos qué movimientos hacer durante el playout? Si sólo elegimos al azar, luego de múltiples simulaciones obtenemos respuesta a la pregunta "¿cuál es el mejor movimiento si ambos jugadores juegan al azar?" Para unos juegos simples, esa es la misma respuesta que "¿cuál es el mejor movimiento si ambos jugadores juegan bien?", pero para la mayoría de los juegos no lo es. Para obtener información útil del playout necesitamos una **política de playout** que favorezca los movimientos hacia los buenos. Para Go y otros juegos, estas políticas se han aprendido con éxito a partir del autojuego usando redes neuronales. A veces se usan heurísticas específicas del juego, como "considerar movimientos de captura" en el ajedrez o "tomar la esquina" en Othello.

Política de playout

Dada una política de playout, a continuación necesitamos decidir dos cosas: ¿desde qué posiciones comenzamos los playouts y cuántos playouts asignamos a cada posición? La respuesta más simple, llamada **búsqueda de Monte Carlo puro**, es hacer N simulaciones comenzando desde el estado actual y rastrear cuál de los posibles movimientos desde la posición actual tiene el mayor porcentaje de victorias.

Búsqueda de Monte Carlo puro

Para algunos juegos estocásticos, esto converge a un juego óptimo a medida que N aumenta, pero para la mayoría los juegos no son suficientes, necesitamos una **política de selección** centrada selectivamente en la computación sobre las partes importantes del árbol de juego. Esta estrategia equilibra dos factores: **exploración** de estados de pocas jugadas y **explotación** de estados de éxito en jugadas anteriores, para lograr una estimación más precisa de su valor. (Ver sección 17.3 para profundizar sobre el equilibrio entre exploración y explotación). La búsqueda de árbol

Política de selección

Exploración

Explotación

[3] Los algoritmos de "Monte Carlo" son algoritmos aleatorios llamados así por el Casino de Monte-Carlo en Mónaco.

de Monte Carlo se hace manteniendo un árbol de búsqueda y haciéndolo crecer en cada iteración de los siguientes cuatro pasos, como en la Figura 5.10:

- **Selección**: Comenzando en la raíz del árbol de búsqueda, elegimos un movimiento (guiado por la política de selección), llevando a un nodo sucesor, y repetimos ese proceso moviéndonos hacia abajo en el árbol hasta una hoja. La Figura 5.10(a) muestra un árbol de búsqueda con la raíz representando un estado donde el blanco acaba de mover, y el blanco ha ganado 37 de los 100 playouts realizados hasta ahora. La flecha gruesa muestra la selección de un movimiento por parte del negro que lleva a un nodo donde éste ha ganado 60/79 playouts. Este es el mejor porcentaje de victorias entre los tres movimientos, por ello seleccionarlo es un ejemplo de explotación. Pero también habría sido razonable seleccionar el nodo 2/11 por el bien de la exploración, con sólo 11 playouts, el nodo aún tiene alta incertidumbre en su valoración, y podría terminar siendo el mejor si logramos más información sobre él. La selección continúa hasta el nodo hoja marcado 27/35.
- **Expansión**: Hacemos crecer el árbol de búsqueda al generar un nuevo hijo del nodo seleccionado; la Figura 5.10(b) muestra el nuevo nodo marcado con 0/0. (Unas versiones generan más de un hijo en este paso).
- **Simulación**: Realizamos un playout desde el nodo hijo recién generado, al elegir movimientos para ambos jugadores de acuerdo con la política de playout. Estos movimientos *no* se registran en el árbol de búsqueda. En la figura, la simulación resulta en una victoria para el negro.
- **Retropropagación o propagación hacia atrás:** Usamos el resultado de la simulación para actualizar los nodos del árbol de búsqueda que van a la raíz. Como el negro ganó la jugada, los nodos negros se incrementan en número de victorias y de jugadas, por ello 27/35 se convierte en 28/26 y 60/79 en 61/80. Como el blanco perdió, los nodos blancos se incrementan sólo en el número de jugadas, por ello 16/53 se convierte en 16/54 y la raíz 37/100 en 37/101.

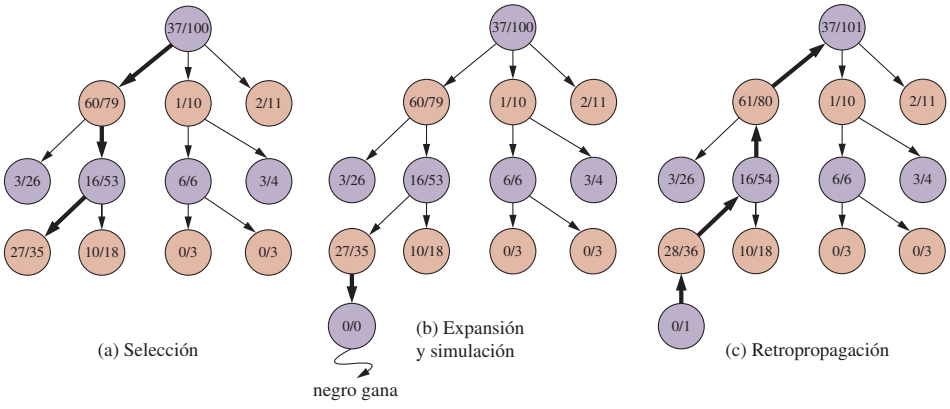


Figura 5.10 Iteración del proceso de elección de un movimiento con búsqueda de árbol de Monte Carlo (MCTS) usando métrica de selección de límites de confianza superiores aplicados a los árboles (UCT), mostrada tras realizar 100 iteraciones. En (a) seleccionamos movimientos, hasta el final del árbol, y terminamos en el nodo hoja marcado 27/35 (para 27 victorias de 35 jugadas para el negro). En (b) expandimos el nodo seleccionado y hacemos una simulación (jugada), que termina en victoria para el negro. En (c), los resultados de la simulación se retropropagan en el árbol.

```
function MONTE-CARLO-TREE-SEARCH(state) returns una action
    tree←NODE(state)
    while IS-TIME-REMAINING() do
        leaf ←SELECT(tree)
        child←EXPAND(leaf)
        result←SIMULATE(child)
        BACK-PROPAGATE(result, child)
    return el movimiento es ACTIONS(state) cuyo nodo tiene el número más grande de playouts
```

Figura 5.11 Algoritmo de búsqueda de árbol Monte Carlo. Se inicializa un árbol de juego, *árbol*, y se repite el ciclo SELECCIONAR / EXPANDIR / SIMULAR / RETROPROPAGAR hasta acabar el tiempo, y devolvemos el movimiento que llevó al nodo con el mayor número de playouts.

Repetimos los cuatro pasos por un número fijo de iteraciones, o hasta que se acabe el tiempo asignado; luego devolvemos el movimiento con el mayor número de juegos simulados.

Una política de selección efectiva se llama "límites de confianza superiores aplicados a los árboles" o **UCT**. La política clasifica cada posible movimiento en función de la fórmula de límite de confianza superior **UCB1**. (Vea más detalles en la sección 17.3.3). Para un nodo n, la fórmula es:

UCT

UCB1

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

donde $U(n)$ es la utilidad total de juegos que pasaron por el nodo n, $N(n)$ es el número de juegos a través del nodo n y PARENT(n) es el nodo padre de n en el árbol. Así, $\frac{U(n)}{N(n)}$ es el término de explotación: la utilidad promedio de n. El término con la raíz cuadrada es el término de exploración: tiene el recuento $N(n)$ en el denominador; es decir, el término será alto para los nodos que se han explorado pocas veces. En el numerador tiene el registro del número de veces que hemos explorado el padre de n. Esto es que si seleccionamos n algún porcentaje no nulo del tiempo, el término de exploración tiende a cero a medida que crecen los recuentos y los juegos se dan al nodo con la utilidad promedio más alta.

C es una constante que equilibra explotación y exploración. Hay un argumento teórico que C debería ser $\sqrt{2}$, pero en la práctica, los programadores prueban múltiples valores para C y eligen el que funciona mejor. (Algunos programas usan fórmulas algo diferentes; por ejemplo, ALPHAZERO agrega un término para la probabilidad de movimiento, que se calcula mediante una red neuronal entrenada a partir de auto-juego pasado). Con $C = 1.4$, el nodo 60/79 en la Figura 5.10 tiene la puntuación UCB1 más alta, pero con $C = 1.5$, sería el nodo 2/11.

La Figura 5.11 muestra el algoritmo completo UCT MCTS. Cuando las iteraciones terminan, se devuelve el movimiento con el mayor número de playouts. Podría pensar que sería mejor devolver el nodo con la utilidad promedio más alta, pero la idea es que un nodo con 65/100 victorias es mejor que uno con 2/3 victorias, porque este último tiene mucha incertidumbre. En cualquier caso, la fórmula UCB1 asegura que el nodo con más playouts es casi siempre el nodo con el mayor porcentaje de victorias, porque el proceso de selección favorece cada vez más el porcentaje de victorias a medida que aumenta el número de playouts.

El tiempo para calcular un juego es lineal, no exponencial, en la profundidad del árbol de juegos, pues sólo toma un movimiento en cada punto de elección. Eso da mucho tiempo

para múltiples playouts. Por ejemplo: considere un juego con un factor de ramificación de 32, donde el juego promedio dura 100 jugadas. Si tenemos suficiente potencia de cómputo para considerar mil millones de estados de juego antes de hacer un movimiento, entonces minimax busca 6 jugadas de profundidad, alfa-beta con un orden de movimiento perfecto busca 12 jugadas, y la búsqueda de Monte Carlo hace 10 millones de playouts. ¿Qué enfoque será mejor? Depende de la precisión de la función heurística versus las políticas de selección y playout.

La sabiduría convencional ha sido que la búsqueda de Monte Carlo presenta ventaja sobre alfa-beta para juegos como Go donde el factor de ramificación es muy alto (y, por ello, alfa-beta no puede buscar lo suficientemente profundo), o cuando es difícil definir una buena función de evaluación. Lo que hace alfa-beta es elegir el camino a un nodo que tenga la puntuación de función de evaluación más alta alcanzable, como el oponente intentará minimizar la puntuación. Por ello, si la función de evaluación es inexacta, alfa-beta será inexacta. Un error de cálculo en un solo nodo hace que alfa-beta elija (o evite) erróneamente un camino a ese nodo. Pero la búsqueda de Monte Carlo se basa en el conjunto de muchos juegos y, entonces, no es tan vulnerable a un solo error. Es posible combinar MCTS y funciones de evaluación haciendo un juego para un cierto número de movimientos, pero luego truncándolo y aplicando una función de evaluación.

También es posible combinar aspectos de alfa-beta y búsqueda de Monte Carlo. Por ejemplo, en juegos que pueden durar muchos movimientos, es posible que deseemos usar la **terminación temprana del juego**, en la que detenemos un juego que toma demasiados movimientos, y lo evaluamos con una función de evaluación heurística o sólo lo declaramos un empate.

Terminación temprana del juego

La búsqueda de Monte Carlo se aplica a juegos nuevos, en los que no hay un cuerpo de experiencia del cual extraer y definir una función de evaluación. Si conocemos las reglas del juego, la búsqueda de Monte Carlo no necesita información adicional. Las políticas de selección y juego hacen buen uso del conocimiento experto hecho a mano cuando está disponible, pero las buenas políticas se aprenden usando redes neuronales entrenadas sólo por autojuego.

La búsqueda de Monte Carlo tiene desventaja cuando es probable que un solo movimiento cambie el curso del juego, porque la naturaleza estocástica de la búsqueda de Monte Carlo significa que podría no considerar ese movimiento. Es decir, la poda Tipo B en la búsqueda de Monte Carlo significa que una línea vital de juego podría nunca ser explorada. Además, ocurre que la búsqueda de Monte Carlo también tiene desventaja cuando hay estados de juego que son "obviamente" victoria para uno u otro lado (según el conocimiento humano y una función de evaluación), pero donde aún se necesitarán muchos movimientos en un juego para verificar el ganador. Se creía desde hace tiempo que la búsqueda alfa-beta era más adecuada para juegos como el ajedrez con un bajo factor de ramificación y buenas funciones de evaluación, pero recién los enfoques de Monte Carlo han demostrado éxito en ajedrez y otros juegos.

La idea general de simular movimientos en el futuro, ver el resultado y usarlo para determinar cuáles son los buenos movimientos es un tipo de **aprendizaje por refuerzo**, que se cubre en el Capítulo 22.

5.5 Juegos estocásticos

Juegos estocásticos

Los **juegos estocásticos** nos acercan a la imprevisibilidad de la vida real al incluir un elemento aleatorio, como el lanzamiento de dados. El backgammon es un juego estocástico típico que combina suerte y habilidad. En la posición de backgammon de la Figura 5.12, el blanco ha sacado un 6-5 y tiene cuatro movimientos posibles (cada uno de los cuales mueve una pieza hacia adelante (en sentido horario) 5 posiciones y una pieza hacia adelante 6 posiciones).

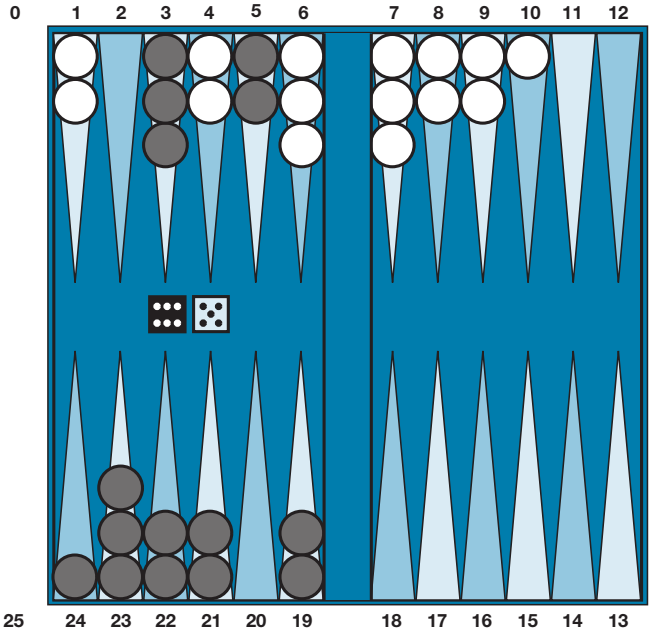


Figura 5.12 Posición típica de backgammon. El objetivo del juego es mover todas las piezas fuera del tablero. El Negro se mueve en sentido horario al 25, el Blanco en sentido antihorario al 0. Una pieza se mueve a cualquier posición a menos que haya múltiples piezas del oponente allí; si hay una ficha del oponente, esta es capturada y debe comenzar de nuevo. En la posición mostrada, el Negro ha sacado 6-5 y debe elegir entre cuatro movimientos legales: (5-11,5-10), (5-11,19-24), (5-10,10-16) y (5-11,11-16), donde la notación (5-11,11-16) significa mover una pieza de la posición 5 a la 11, y luego mover una pieza de 11 a 16.

En este punto, el Negro sabe qué movimientos puede hacer, pero no qué sacará el Blanco y, tampoco sabe cuáles serán los movimientos legales del Blanco. Eso significa que el Negro no puede construir un árbol de juego estándar del tipo visto en el ajedrez y el Gato. Un árbol de juego en backgammon debe incluir **nodos de posibilidad** además de nodos MAX y MIN. Los nodos de posibilidad se muestran como círculos en la Figura 5.13. Las ramas que salen de cada nodo de posibilidad denotan los posibles lanzamientos de dados; cada rama está etiquetada con el lanzamiento y su probabilidad. Hay 36 formas de lanzar dos dados, cada una igual de probable; pero como 6-5 es lo mismo que 5-6, sólo hay 21 lanzamientos distintos. Los seis dobles (1-1 a 6-6) tienen cada uno una probabilidad de 1/36, por ello decimos $P(1\text{-}1) = 1/36$. Los otros 15 lanzamientos distintos tienen cada uno probabilidad de 1/18.

Nodos de posibilidad

El siguiente paso es entender cómo tomar decisiones correctas. Obviamente, todavía queremos elegir el movimiento que lleva a la mejor posición. Sin embargo, las posiciones no tienen valores minimax definidos. En su lugar, solo podemos calcular el **valor esperado** de una posición: el promedio de todos los posibles resultados de los nodos de probabilidad.

Valor esperado

Esto nos lleva al **valor minimaxesperado** para juegos con nodos de posibilidad, una generalización del minimax del valor para juegos deterministas. Los nodos terminales y los MAX y MIN funcionan exactamente igual que antes (con la advertencia de que los movimientos legales para MAX y MIN dependerán del resultado del lanzamiento de dados en el nodo de posibilidad

Valor minimaxesperado

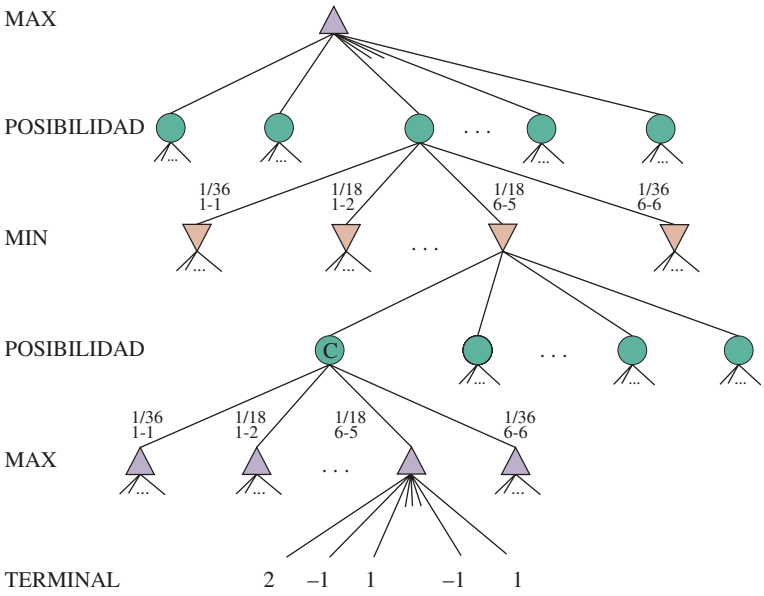


Figura 5.13 Árbol de juego esquemático para una posición de backgammon.

previo). Para los nodos de posibilidad calculamos el valor esperado, que es la suma del valor sobre todos los resultados, ponderado por la probabilidad de cada acción posible:

$$\text{EXPECTIMINIMAX}(s) = \begin{cases} \text{UTILITY}(s,\text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s,a)) & \text{if TO-MOVE}(s)=\text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s,a)) & \text{if TO-MOVE}(s)=\text{MIN} \\ \sum_r P(r)\text{EXPECTIMINIMAX}(\text{RESULT}(s,r)) & \text{if TO-MOVE}(s)=\text{CHANCE} \end{cases}$$

donde r representa un posible lanzamiento de dados (u otro evento de azar) y RESULT(s,r) es el mismo estado que s, con el hecho adicional de que el resultado del lanzamiento de dados es r.

5.5.1 Funciones de evaluación para juegos de azar

Igual que con minimax, la aproximación obvia con minimaxesperado es cortar la búsqueda en un punto y aplicar función de evaluación a cada hoja. Se podría pensar que las funciones de evaluación para juegos como el backgammon deberían ser iguales que las del ajedrez, sólo necesitan dar valores más altos a las mejores posiciones. Aunque, la presencia de nodos de posibilidad significa que uno tiene que ser más cuidadoso acerca de lo que significan los valores.

La Figura 5.14 muestra qué sucede: con una función de evaluación que asigna valores [1, 2, 3, 4] a las hojas, el movimiento a_1 es el mejor; con los valores [1, 20, 30, 400], el movimiento a_2 es el mejor. Así, el programa se comporta de manera totalmente diferente si hacemos un cambio en algunos de los valores de evaluación, aun si el orden de preferencia permanece igual.

Resulta que para evitar este problema, la función de evaluación debe devolver valores que sean una transformación lineal positiva de la **probabilidad** de ganar (o de la utilidad esperada, para

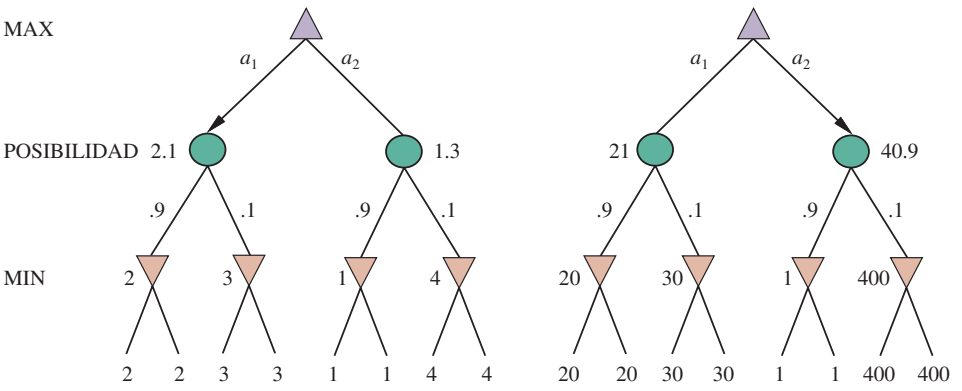


Figura 5.14 Una transformación que salva el orden de los valores de las hojas cambia el mejor movimiento.

juegos con resultados distintos a ganar/perder). Esta relación con la probabilidad es una importante propiedad general de situaciones en las que hay incertidumbre, y lo discutimos más en el Capítulo 16.

Si el programa supiera de antemano los lanzamientos de dados que saldrían durante el resto del juego, resolver un juego con dados sería igual que resolver un juego sin dados, lo que minimax hace en tiempo $O(b^m)$, donde b es el factor de ramificación y m es la profundidad máxima del árbol del juego. Debido a que minimaxesperado también considera todas las posibles secuencias de lanzamientos de dados, tomará $O(b^m n^m)$, donde n es el número de lanzamientos distintos.

Aun si la búsqueda se limita a una pequeña profundidad d, el costo adicional comparado con el de minimax hace poco realista considerar mirar hacia adelante muy lejos en la mayoría de juegos de azar. En backgammon n es 21 y b suele ser unos 20, pero en unas situaciones llega a casi 4 000 para tiradas de dados dobles. Tal vez sólo manejaríamos tres jugadas en la búsqueda.

Otra forma de pensar en el problema es: la ventaja de alfa-beta es que ignora los desarrollos futuros que no van a suceder, dada la mejor jugada. Por ello, se concentra en ocurrencias probables. Pero en un juego donde un lanzamiento de dos dados precede a cada movimiento, *no* hay secuencias probables de movimientos; aun el movimiento más probable ocurre sólo 2/36 del tiempo porque, para realizar el movimiento, los dados primero tendrían que salir de la manera correcta para hacerlo legal. Este es un problema general si la incertidumbre entra en escena: las posibilidades se multiplican mucho y formar planes detallados de acción se vuelve inútil porque el mundo quizá no jugará para nosotros.

Puede haberle ocurrido que la poda alfa-beta se pueda aplicar a los árboles de juegos con nodos de azar. Resulta que sí se puede. El análisis para los nodos MIN y MAX no cambia, aunque podemos podar los nodos de posibilidad con un poco de ingenio. Piense el nodo de posibilidad C en la Figura 5.13 y lo que sucede con su valor a medida que evaluamos a sus hijos. ¿Es posible encontrar un límite superior en el valor de C antes de haber mirado a todos sus hijos? (Recuerde que esto es lo que necesita alfa-beta para podar un nodo y su subárbol).

A primera vista, parecería imposible porque el valor de C es el *promedio* de los valores de sus hijos, y para calcular el promedio de un conjunto de números, debemos mirar todos los números. Pero si ponemos límites en los posibles valores de la función de utilidad, entonces podemos llegar

a límites para el promedio sin mirar cada número. Así, digamos que todos los valores de utilidad están entre -2 y +2; entonces el valor de los nodos hoja está limitado y, a su vez, *podemos* colocar un límite superior en el valor de un nodo de azar sin mirar a sus hijos.

En juegos donde el factor de ramificación para nodos de posibilidad es alto, como el juego Yahtzee donde se lanzan 5 dados en cada turno, es posible que desee considerar la poda hacia adelante que muestrea un número menor de las posibles ramas de posibilidad. O quizá desee evitar el uso de una función de evaluación por completo y optar por búsqueda de árbol de Monte Carlo en su lugar, donde cada juego incluye lanzar dados aleatorios.

5.6 Juegos parcialmente observables

Bobby Fischer declaró que "el ajedrez es guerra", que carece de al menos una característica importante de las guerras reales, a saber, **observabilidad parcial**. En la "niebla de la guerra", a menudo se desconoce el paradero de las unidades enemigas hasta que se revelan por contacto directo. Como resultado, la guerra incluye el uso de exploradores y espías para recopilar información y el uso de ocultación y engaño para confundir al enemigo.

Los juegos parcialmente observables comparten estas características y, por ello, son cualitativamente diferentes de los de secciones previas. Videojuegos como StarCraft son de manera particular desafiantes, al ser parcialmente observables, multiagentes, no deterministas, dinámicos y desconocidos.

En juegos *deterministas* parcialmente observables, la incertidumbre sobre el estado del tablero surge por completo por falta de acceso a las elecciones del oponente. Esta clase incluye juegos como Battleship (donde los barcos de cada jugador se colocan en lugares ocultos al oponente) y Stratego (donde las ubicaciones de las piezas son conocidas pero los tipos de piezas se ocultan).

Kriegspiel

Examinaremos el juego de **Kriegspiel**, una variante parcialmente observable del ajedrez donde las piezas son del todo invisibles para el oponente. Otros juegos también tienen versiones parcialmente observables: Phantom Go, Gato fantasma y Screen Shogi.

5.6.1 Kriegspiel: Ajedrez parcialmente observable

Las reglas de Kriegspiel son: Blanco y Negro ven cada uno un tablero que contiene sólo sus propias piezas. Un árbitro, que ve todas las piezas, adjudica el juego y periódicamente hace anuncios para ambos jugadores. Primero, Blanco propone al árbitro un movimiento que sería legal si no hubiera piezas negras. Si las piezas negras impiden el movimiento, el árbitro anuncia "ilegal" y Blanco sigue proponiendo movimientos hasta que da con uno legal, y aprende más sobre la ubicación de las piezas negras en el proceso.

Una vez que se propone un movimiento legal, el árbitro anuncia uno o más de los siguientes: "Captura en la casilla *X*" si hay una captura, y "Jaque por *D*" si el rey negro está en jaque, donde *D* es la dirección del jaque, y puede ser uno de "Caballo", "Fila", "Columna", "Diagonal larga" o "Diagonal corta". Si el Negro está en jaque mate o el juego está en tablas, el árbitro lo anuncia; de lo contrario, es el turno del Negro para mover.

Kriegspiel parecerá aterradoramente imposible, pero los humanos lo manejan bien y los programas de computadora se están actualizando. Ayuda a recordar la noción de un **estado de creencia** como se define en la Sección 4.4 y se ilustra en la Figura 4.14: el conjunto de todos los estados de tablero *lógicamente posibles* dados la historia completa de percepciones a la fecha. Al inicio, el estado de creencia del Blanco es una instancia única porque las piezas del Negro aún no se han movido. Luego de que el Blanco hace un movimiento y el Negro responde, el estado de creencia del Blanco contiene 20 posiciones, porque el Negro tiene 20 respuestas a todo

movimiento de apertura. Conforme avanza el juego, el seguimiento del estado de creencia es el problema de la **estimación de estado**, para el cual el paso de actualización se da en la Ecuación (4.6) (página 132). Podemos mapear la estimación de estado de Kriegspiel en el mismo marco parcialmente observable y no determinista de la Sección 4.4 si consideramos al oponente como la fuente del no determinismo; es decir, los RESULTADOS del movimiento del Blanco se componen del resultado (predecible) del movimiento del Blanco y el resultado impredecible dado por la respuesta del Negro.[4]

Dado un estado de creencia actual, el Blanco preguntará: "¿Puedo ganar el juego?" Para un juego parcialmente observable, la noción de una **estrategia** se altera; en vez de especificar un *movimiento* para cada posible movimiento que el oponente haga, necesitamos un movimiento para cada posible *secuencia perceptiva* que se pueda recibir.

Jaque mate garantizado

Para Kriegspiel, una estrategia ganadora o **jaque mate garantizado** es la que, para cada posibilidad, La secuencia de percepción lleva a un jaque mate real para cada estado posible del tablero en el estado de creencia actual, aparte de cómo se mueva el oponente. Con esta definición, el estado de creencia del oponente es irrelevante: la estrategia tiene que funcionar aun si el oponente puede ver todas las piezas. Esto simplifica mucho el cálculo. La Figura 5.15 muestra parte de un jaque mate garantizado para el final de KRK (rey y torre contra rey). Aquí, las negras tienen una sola pieza (el rey), por ello un estado de creencia para las blancas se muestra en un solo tablero marcando cada posición posible del rey negro.

El algoritmo de búsqueda general AND-OR se aplica al espacio de estados de creencias para encontrar jaque mate garantizado, tal como en la Sección 4.4. El algoritmo incremental de estado de creencias visto en la Sección 4.4.2 encontrar jaque mate en la mitad del juego hasta la profundidad 9, más allá de las habilidades de los jugadores humanos.

Jaque mate probabilístico

Además de los jaque mate garantizados, Kriegspiel admite el concepto nuevo que no tiene sentido en juegos totalmente observables: **jaque mate probabilístico.** Estos jaque mates se requiere que sean posibles en cada estado de tablero en el estado de creencia; son probabilísticos con respecto a la aleatorización de los movimientos del jugador ganador. Para tener la idea básica, piense el problema de encontrar un rey negro solitario usando apenas el rey blanco. Sólo moviéndose al azar, el rey blanco chocará *eventualmente* con el rey negro aun si este último intenta evitar este destino, ya que Negro no puede seguir adivinando los movimientos evasivos correctos de modo indefinido. En terminología de teoría de la probabilidad, la detección ocurre *con probabilidad* 1.

El final de juego KBNK —rey, alfil y caballo contra rey— se gana en este sentido; el Blanco presenta al Negro una secuencia aleatoria infinita de opciones, para una de las cuales el Negro adivinará incorrectamente y revelará su posición, lo que lleva a jaque mate. Además, el final de juego KBBK se gana con probabilidad $1 - \epsilon$. El Blanco sólo puede forzar victoria dejando uno de sus alfiles desprotegidos por un movimiento. Si el Negro está en el lugar correcto y captura el alfil (un movimiento ilegal si los alfiles están protegidos), el juego termina en empate. El Blanco puede optar por hacer el movimiento arriesgado en algún punto elegido al azar en medio de una secuencia larga, y reducir así ϵ a una constante arbitrariamente pequeña, pero no puede reducir ϵ a cero.

Jaque mate accidental

A veces una estrategia de jaque mate funciona para *unos* de los estados del tablero en la creencia actual estado, no en otros. Intentar tal estrategia puede tener éxito y llevar a **jaque mate accidental** —en el sentido de que el Blanco no *sabría* que es jaque mate— si las piezas de Negro están en los lugares correctos. (La mayoría de jaque mates en los juegos entre humanos son de

[4] A veces, el estado de creencia será muy grande para representarse sólo como lista de estados de tablero, pero ignoraremos esto por ahora. Los capítulos 7 y 8 sugieren métodos para representar de manera compacta estados de creencia grandes.

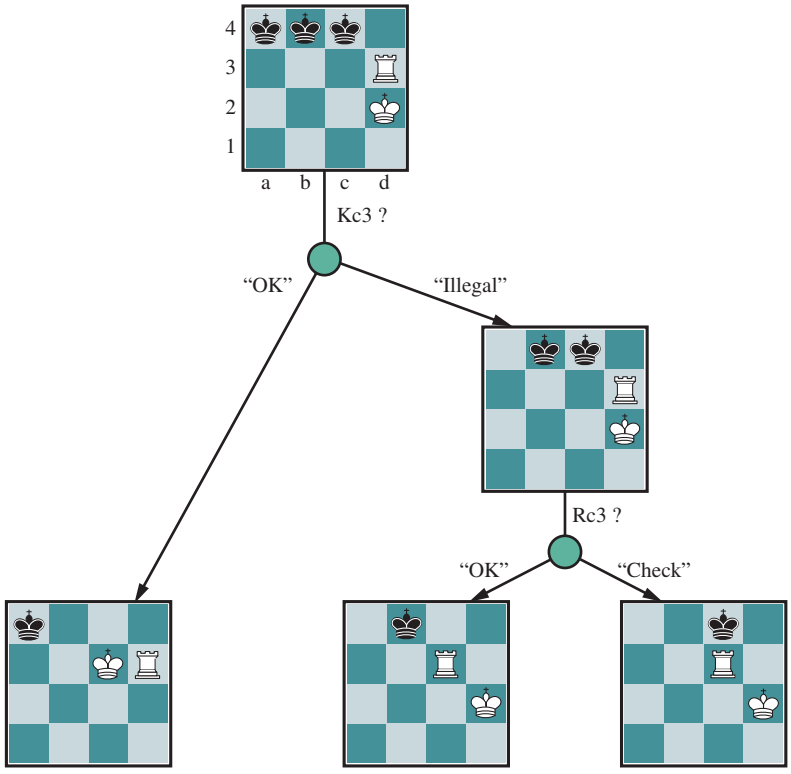


Figura 5.15 Parte de un jaque mate garantizado en el final KRK, mostrado en un tablero reducido. En el estado de creencia inicial, el rey negro está en una de tres posibles ubicaciones. Al combinar movimientos de sondeo, la estrategia reduce esto a uno. La ejecución del jaque mate se deja como ejercicio.

naturaleza accidental.) Esta idea lleva naturalmente a la pregunta *qué tan probable* es que una estrategia dada gane, lo que, a su vez, lleva a la pregunta *qué tan probable* es que cada estado de tablero en el estado de creencia actual sea el verdadero estado de tablero.

La primera idea de uno sería proponer que todos los estados de tablero en el estado de creencia actual son igual de probables, pero esto no puede ser correcto. Piense, por ejemplo, el estado de creencia del Blanco después del primer movimiento del Negro en el juego. Por definición (asumiendo que el Negro juega de manera óptima), el Negro debe haber jugado un movimiento óptimo, por ello todos los estados de tablero resultantes de movimientos subóptimos deberían asignarse una probabilidad cero.

Este argumento tampoco es del todo correcto, porque el *objetivo de cada jugador no es sólo mover las piezas a las casillas correctas, sino minimizar la información que el oponente tiene sobre su ubicación*. Jugar cualquier estrategia "óptima" predecible da al oponente información. Por ello, el juego óptimo en juegos parcialmente observables requiere disposición para jugar de manera algo *aleatoria*. (Por eso los inspectores de higiene de los restaurantes hacen visitas de inspecciones aleatorias). Esto significa seleccionar ocasionalmente movimientos que parecen "intrínsecamente" débiles, pero que ganan fuerza precisamente por su imprevisibilidad, porque el oponente es poco probable que haya preparado alguna defensa contra ellos.

A partir de estas consideraciones, parece que las probabilidades asociadas con los estados del tablero en el estado de creencia actual sólo se calculan dada una estrategia aleatoria óptima; a su vez, el cálculo de esa estrategia requiere conocer las probabilidades de los diversos estados en los que podría estar la junta. Este enigma puede resolverse adoptando la noción de teoría de juegos de una solución de **equilibrio**, que veremos en el Capítulo 17. Un equilibrio especifica una estrategia aleatoria óptima para cada jugador. Calcular los equilibrios es caro para Kriegspiel. Hoy, el diseño de algoritmos efectivos para el juego general de Kriegspiel es tema de investigación. La mayoría de sistemas dan una mirada anticipada de profundidad limitada en su propio espacio de estado de creencias, e ignoran el estado de creencias del oponente. Las funciones de evaluación se asemejan a las del juego observable, pero incluyen un componente para el tamaño del estado de creencia: ¡cuanto más pequeño, mejor! Volveremos a los juegos parcialmente observables bajo el tema Teoría de juegos en la Sección 18.2.

5.6.2 Juegos de cartas

Juegos de cartas como bridge, whist, hearts y póquer presentan observabilidad parcial *estocástica*, donde la información faltante es generada por la distribución aleatoria de las cartas.

A primera vista, parecería que estos juegos de cartas son como juegos de dados: las cartas se reparten al azar y determinan los movimientos disponibles para cada jugador, ¡pero todos los "dados" se lanzan al principio! Aunque esta analogía es incorrecta, sugiere un algoritmo: tratar el inicio del juego como un nodo de oportunidad con cada posible reparto como resultado, y luego usar la fórmula MinimaxEsperado para elegir el mejor movimiento. Tenga en cuenta que en este enfoque, el único nodo de oportunidad es el nodo raíz; luego de eso, el juego se vuelve por completo observable. Este enfoque a veces se llama *promedio sobre clarividencia* porque asume que una vez producido el reparto real, el juego se vuelve por completo observable para ambos jugadores. A pesar de su atractivo intuitivo, la estrategia puede llevarnos por mal camino. Considere la siguiente historia:

> Día 1: La carretera *A* lleva a una olla de oro; la *B* lleva a una bifurcación. La bifurcación izquierda lleva a dos ollas de oro, la derecha lleva a que lo atropelle un autobús.
>
> Día 2: La carretera *A* lleva a una olla de oro; la *B* lleva a una bifurcación. La bifurcación derecha lleva a dos ollas de oro, la bifurcación izquierda lleva a que lo atropelle un autobús.
>
> Día 3: La carretera *A* lleva a una olla de oro; la *B* lleva a una bifurcación. Le dicen que una bifurcación lleva a dos ollas de oro, la otra lo lleva a ser atropellado por un autobús. Pero, no sabe qué camino es cuál.

Promediar sobre la intuición lleva al siguiente razonamiento: en el día 1, *B* es la elección correcta; en el día 2, *B* es la elección correcta; en el día 3, la situación es la misma que en el día 1 o en el día 2, por ello *B* debe seguir siendo la opción correcta.

Ahora vemos cómo falla el promedio sobre la intuición: no considera el *estado de creencia* en el que estará el agente tras actuar. Un estado de creencia de ignorancia total no es deseable, en especial si una posibilidad es muerte segura. Como se asume que cada estado futuro será automáticamente uno de conocimiento perfecto, el enfoque de intuición nunca selecciona acciones que *reúnan información* (como el primer movimiento en la Figura 5.15), ni elegirá acciones que oculten información al oponente o dé información a un compañero, porque asume que ya conocen la información, y nunca **farolear** en el póquer,[5] porque

Farolear

[5] Farolear, apostar como si la mano fuera buena, aun si no lo es; es fundamental en la estrategia de póquer.

se supone que el oponente puede ver sus cartas. En el Capítulo 17, veremos cómo construir algoritmos que hacen todas estas cosas mediante la resolución del verdadero problema de decisión parcialmente observable, lo que resulta en una estrategia de equilibrio óptimo (ver Sección 18.2).

A pesar de las desventajas, el promedio de intuición puede resultar una estrategia efectiva, con unos trucos para que funcione mejor. En los juegos de cartas, el número de posibles repartos es muy grande. Así, en el bridge, cada jugador ve sólo dos de las cuatro manos; hay dos manos invisibles de 13 cartas cada una, por ello el número de repartos es $\binom{26}{13} = 10\,400\,600$. Resolver aun un reparto es difícil; por ello, resolver diez millones está fuera de discusión. Una forma de lidiar con este número enorme es mediante abstracción: es decir, tratar manos similares como idénticas. Por ejemplo, es importante qué ases y reyes hay en una mano, pero si la mano tiene un 4 o un 5 no es tan importante y se puede abstraer.

Otra forma de lidiar con el enorme número es la poda hacia adelante: considerar sólo una pequeña muestra aleatoria de N repartos, y de nuevo calcular la puntuación MINIMAXESPERADA. Aun para N muy pequeños, digamos, de 100 a 1 000, este método da una buena aproximación. También se puede aplicar a juegos deterministas como Kriegspiel, donde muestreamos posibles estados del juego en vez de posibles repartos; siempre y cuando tengamos alguna forma de estimar cuán probable es cada estado. Además, resulta hacer una búsqueda heurística con límite de profundidad en vez de buscar todo el árbol del juego.

Hasta ahora hemos asumido que cada reparto es igualmente probable. Eso tiene sentido para juegos como whist y hearts. Pero para el bridge, el juego es precedido por una fase de subasta donde cada equipo indica cuántos trucos espera ganar. Como los jugadores hacen ofertas en función de las cartas que tienen, los contrincantes aprenden algo sobre la probabilidad $P(s)$ de cada reparto. Tener esto en cuenta al decidir cómo jugar la mano es complicado, por las razones indicadas en nuestra descripción de Kriegspiel: los jugadores pueden ofertar de modo que minimicen la información transmitida a sus oponentes.

Las computadoras han alcanzado un nivel sobrehumano de rendimiento en el póquer. El programa Libratus enfrentó a cuatro de los mejores jugadores de póquer del mundo en una partida de 20 días de Texas Hold'em sin límite y los venció contundentemente. Como hay tantos estados posibles en el póquer, Libratus usó abstracción para reducir el espacio de estados: podría considerar que las dos manos AAA72 y AAA64 son equivalentes (ambas son "tres ases y unas cartas bajas"), y podría considerar que una apuesta de 200 dólares es lo mismo que 201 dólares. Libratus también monitorea a los otros jugadores, y si detecta que están explotando una abstracción, hará cálculos adicionales durante la noche para tapar ese agujero. En total, usó 25 millones de horas de CPU en un superordenador para lograr la victoria.

Los costos computacionales por Libratus (y costos similares por ALPHAZERO y otros sistemas) sugieren que este juego puede no ser factible para investigadores con presupuestos limitados. En cierto modo, eso es cierto: así como usted no espera armar un auto Fórmula Uno campeón con piezas de repuesto en su garaje, hay una ventaja en tener acceso a supercomputadoras o hardware especializado como las Unidades de Procesamiento de Tensores. Eso es cierto cuando se entrena un sistema, aunque el entrenamiento podría hacerse mediante colaboración de la multitud. Por ejemplo, el sistema LEELAZERO de código abierto es una reimplementación de ALPHAZERO que se entrena a través del autojuego en las computadoras de los participantes voluntarios. Una vez entrenados, los requisitos computacionales para jugar en torneos reales son modestos. ALPHASTAR ganó juegos de StarCraft II en una computadora de escritorio con una sola GPU, y ALPHAZERO podría haberse ejecutado en ese modo.

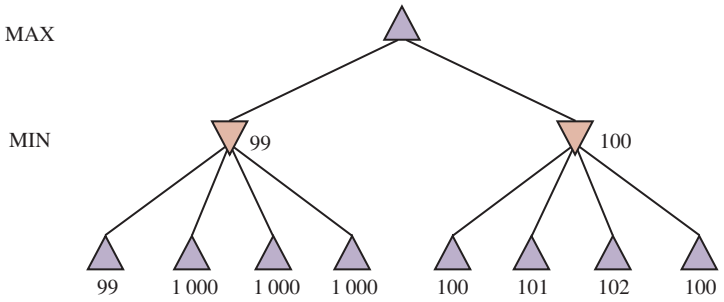


Figura 5.16 Árbol de juego de dos jugadas para el que la heurística minimax puede cometer error.

5.7 Limitaciones de los algoritmos de búsqueda de juegos

Como calcular decisiones óptimas en juegos complejos es difícil, los algoritmos deben hacer suposiciones y aproximaciones. La búsqueda alfa-beta usa la función de evaluación heurística como aproximación; la búsqueda de Monte Carlo calcula un promedio aproximado sobre una selección aleatoria de juegos. La elección del algoritmo a usar depende en parte de las características de cada juego: cuando el factor de ramificación es alto o es difícil definir una función de evaluación, se opta por la búsqueda de Monte Carlo. Pero ambos algoritmos sufren limitaciones esenciales.

Una limitación de la búsqueda alfa-beta es su vulnerabilidad a errores en la función heurística. La Figura 5.16 muestra un árbol de juegos de dos capas para el cual minimax sugiere tomar la rama de la derecha porque $100 > 99$. Ese es el movimiento correcto si las evaluaciones son exactamente precisas. Pero suponga que la evaluación de cada nodo tiene un error independiente de otros nodos y se distribuye aleatoriamente con una desviación estándar de σ. Luego, la rama izquierda es en realidad mejor el 71 % del tiempo cuando $\sigma = 5$, y el 58 % del tiempo cuando $\sigma = 2$ (porque una de las cuatro hojas de la rama derecha es probable que caiga por debajo de 99 en estos casos). Si los errores en la función de evaluación *no son* independientes, entonces la probabilidad de un error crece. Es difícil compensar esto porque no tenemos un buen modelo de las dependencias entre los valores de los nodos hermanos.

Una segunda limitación de alfa-beta y de Monte Carlo es que están diseñados para calcular (limitados en) los valores de los movimientos legales. Pero a veces hay un movimiento que es obviamente el mejor (como cuando sólo hay un movimiento legal) y, entonces, no tiene sentido perder tiempo de cómputo para averiguar el valor del movimiento; es mejor sólo hacer el movimiento. Un mejor algoritmo de búsqueda usaría la idea de *utilidad de una expansión de nodo*, al seleccionar expansiones de nodo de alta utilidad; es decir, las que quizá conduzcan a descubrir un movimiento mejor. Si no hay expansiones de nodo cuya utilidad sea mayor que su costo (en términos de tiempo), entonces el algoritmo debería detener la búsqueda y hacer un movimiento. Esto funciona para situaciones claras de favoritos y para el caso de movimientos *simétricos*, para los que ninguna cantidad de búsqueda mostrará que un movimiento es mejor que otro.

Metarazonamiento

El tipo de razonamiento sobre qué cálculos hacer se llama **metarazonamiento** (razonamiento sobre el razonamiento). Se aplica a los juegos y a todo tipo de razonamiento. Todos los cómputos se realizan con el propósito de lograr mejores decisiones, todas tienen costos y

alguna probabilidad de resultar en una cierta mejora en la calidad de la decisión. La búsqueda de Monte Carlo intenta hacer metarazonamiento para asignar recursos a las partes más importantes del árbol, pero no lo hace de manera óptima.

Una tercera limitación es que tanto el alfa-beta como el Monte Carlo hacen todo su razonamiento a nivel de movimientos individuales. Claramente, los humanos juegan de manera diferente: pueden razonar a un nivel más abstracto, considerando un objetivo de nivel superior, por ejemplo, atrapar a la reina del oponente, y usando el objetivo para generar *selectivamente* planes plausibles. En el Capítulo 11 estudiaremos este tipo de **planificación**, y en la Sección 11.4 mostraremos cómo planificar con una jerarquía de representaciones abstractas a concretas.

Un cuarto problema es la capacidad para incorporar **aprendizaje automático** al proceso de búsqueda de juegos. Los primeros programas dependían de la experiencia humana para crear funciones de evaluación, libros de apertura, estrategias de búsqueda y trucos de eficiencia. Recién vemos programas como AlphaZero (Silver *et al.,* 2018), que dependen del aprendizaje automático a partir del autojuego, no de la experiencia humana generada para el juego. A partir del Capítulo 19 revisamos al detalle el aprendizaje automático.

Resumen

Hemos examinado una variedad de juegos para entender qué significa el juego óptimo, cómo jugar bien en la práctica y construir una idea de cómo un agente debería actuar en cualquier tipo de entorno adverso. Las ideas más significativas son:

- Un juego puede ser definido por el **estado inicial** (cómo se configura el tablero), las **acciones** legales en cada estado, el **resultado** de cada acción, una **prueba terminal** (que dice cuándo termina el juego) y una **función de utilidad** que se aplica a los estados terminales para decir quién ganó y cuál es la puntuación final.
- En juegos de suma cero, discretos, deterministas, de dos jugadores y con **información perfecta**, el algoritmo **minimax** puede seleccionar movimientos óptimos mediante una enumeración en profundidad del árbol del juego.
- El algoritmo de búsqueda **alfa-beta** calcula el mismo movimiento óptimo que minimax, pero logra una eficiencia mucho mayor al eliminar subárboles demostrablemente irrelevantes.
- En general, no es factible considerar todo el árbol del juego (aun con alfa-beta), por lo que necesitamos cortar la búsqueda en algún punto y aplicar una **función de evaluación** heurística que estime la utilidad de un estado.
- La alternativa **búsqueda de árbol de Monte Carlo** (MCTS) evalúa los estados al no aplicar una función heurística, sino al jugar el juego hasta el final y utilizar las reglas de éste para ver quién ganó. Como los movimientos elegidos durante la **simulación** pueden no haber sido los movimientos óptimos, el proceso se repite varias veces y la evaluación es un promedio de los resultados.
- Muchos programas de juegos precalculan tablas de los mejores movimientos en la apertura y el final del juego para extraer de dicha tabla un movimiento en lugar de buscar.
- Los juegos de azar pueden ser manejados por **minimaxesperado**, una extensión del algoritmo minimax que evalúa un **nodo de posibilidad** al tomar la utilidad promedio de todos sus hijos y ponderarla por la probabilidad de cada hijo.
- En juegos de **información imperfecta**, como Kriegspiel y póquer, el juego óptimo requiere razonamiento sobre los **estados de creencia** actuales y futuros de cada jugador. Una sim-

ple aproximación se puede obtener promediando el valor de una acción sobre cada posible configuración de información faltante.

- Los programas han derrotado con notoriedad a jugadores humanos campeones en ajedrez, damas, Othello, Go, póquer y muchos otros juegos. Los humanos mantienen la ventaja en algunos juegos de información imperfecta, como bridge y Kriegspiel. En videojuegos como StarCraft y Dota 2, los programas son competitivos con expertos humanos, pero parte de su éxito quizá se deba a su capacidad para realizar muchas acciones muy rápido.

Notas bibliográficas e históricas

En 1846, Charles Babbage discutió la viabilidad de computadoras para el ajedrez y las damas (Morrison y Morrison, 1961). No entendió la complejidad exponencial de los árboles de búsqueda, pues afirmó que "las combinaciones involucradas en la Máquina Analítica superaron con creces todo requerimiento, aun por el juego de ajedrez". Babbage también diseñó, pero no construyó, una máquina especializada para jugar Gato. La primera máquina de juego fue construida cerca de 1890 por el ingeniero español Leonardo Torres y Quevedo. Se especializó en el final del juego de ajedrez "**KRK"** (rey y torre contra rey), y garantizó una victoria cuando el lado con la torre tiene el movimiento. El algoritmo **minimax** se remonta a un artículo de 1912 de Ernst Zermelo, el desarrollador de la teoría moderna de conjuntos.

El juego fue una de las primeras tareas emprendidas en IA, con los primeros esfuerzos de pioneros como Konrad Zuse (1945), Norbert Wiener en su libro *Cibernética* (1948) y Alan Turing (1953). Pero el artículo de Claude Shannon *Programming a Computer for Playing Chess* (1950) fue el que expuso todas las ideas principales: una representación de las posiciones del tablero, una función de evaluación, una búsqueda de quiescencia y algunas ideas para la búsqueda selectiva del árbol de juego. Slater (1950) tuvo la idea de una función de evaluación como una combinación lineal de características, y enfatizó la característica de movilidad en el ajedrez.

John McCarthy concibió la idea de la búsqueda **alfa-beta** en 1956, aunque la idea no apareció impresa hasta más tarde (Hart y Edwards, 1961). Knuth y Moore (1975) demostraron la exactitud de alfa-beta y analizaron su complejidad temporal, mientras que Pearl (1982b) demostró que alfa-beta era asintóticamente óptima entre todos los algoritmos de búsqueda de árboles de juego de profundidad fija.

Berliner (1979) presentó B*, un algoritmo de búsqueda heurística que mantiene límites de intervalo sobre el posible valor de un nodo en el árbol de juego, en lugar de darle una única estimación de valor puntual. La búsqueda del número de conspiración de David McAllester (1988) expande los nodos hoja que, al cambiar sus valores, podrían hacer que el programa prefiera una nueva jugada en la raíz del árbol. MGSS* (Russell y Wefald, 1989) usa las técnicas de toma de decisiones del Capítulo 16 para estimar el valor de la expansión de cada hoja en términos de la mejora esperada en la calidad de la decisión en la raíz.

El algoritmo SSS* (Stockman, 1979) se puede ver como un A* de dos jugadores que nunca expande más nodos que el alfa-beta. Los requisitos de memoria lo hacen impráctico, pero se ha desarrollado una versión de espacio lineal a partir del algoritmo RBFS (Korf y Chickering, 1996). Baum y Smith (1997) proponen una sustitución basada en la probabilidad para minimax, mostrando que resulta en mejores elecciones en ciertos juegos. El algoritmo **minimaxesperado** fue propuesto por Donald Michie (1966). Bruce Ballard (1983) extendió la poda alfa-beta para cubrir árboles con nodos de posibilidad.

El libro de Pearl *Heuristics* (1984) analiza en exhaustivo algoritmos de juego.

La simulación de Monte Carlo fue pionera por Metropolis y Ulam (1949) para cálculos relacionados con el desarrollo de la bomba atómica. La búsqueda de árbol de Monte Carlo (MCTS) fue introducida por Abramson (1987). Tesauro y Galperin (1997) mostraron cómo una búsqueda de Monte Carlo podría combinarse con una función de evaluación para el backgammon. La terminación temprana del playout se estudia por Lorentz (2015). AlphaGo terminó los playouts y aplicó una función de evaluación (Silver *et al.,* 2016). Kocsis y Szepesvari (2006) refinaron el enfoque con el mecanismo de selección "Upper Confidence Bounds applied to Trees". Chaslot *et al.* (2008) muestran cómo MCTS se aplica a una variedad de juegos y Browne *et al*. (2012) presentan una encuesta.

Koller y Pfeffer (1997) describen un sistema para resolver por completo juegos **parcialmente observables**. Maneja juegos más grandes que los sistemas previos, pero no la versión completa de juegos complejos como póquer y bridge. Frank *et al.* (1998) describen varias variantes de búsqueda de Monte Carlo para juegos parcialmente observables, incluido uno en el que MIN tiene información completa pero MAX no. Schofield y Thielscher (2015) adaptan un sistema de juego general para juegos parcialmente observables.

Ferguson derivó estrategias aleatorias para ganar el Kriegspiel con un alfil y un caballo (1992) o dos alfiles (1995) contra un rey. Los primeros programas de Kriegspiel se centraron en la búsqueda de jaque mate al final del juego y realizaron búsquedas AND-OR en el espacio del estado de creencias (Sakuta e Iida, 2002; Bolognesi y Ciancarini, 2003). Los algoritmos incrementales del estado de creencia permitieron encontrar jaque mate mucho más complejos en la mitad del juego (Russell y Wolfe, 2005; Wolfe y Russell, 2007), pero la estimación eficiente del estado sigue siendo el principal obstáculo para un juego general efectivo (Parker *et al*., 2005). Ciancarini y Favini (2010) aplican MCTS al Kriegspiel, y Wang *et al*. (2018b) describen una versión de estado de creencias de MCTS para Phantom Go.

Los hitos del **ajedrez** han sido marcados por los sucesivos ganadores del Premio Fredkin: BELLE (Condon y Thompson, 1982), el primer programa en alcanzar el estatus de maestro; Deep Thought (Hsu *et al*., 1990), el primero en alcanzar el estatus de maestro internacional; y Deep Blue (Campbell *et al*., 2002; Hsu, 2004), que derrotó al campeón mundial Garry Kasparov en un match de exhibición en 1997. Deep Blue ejecutó la búsqueda alfa-beta a más de 100 millones de posiciones por segundo, y podía generar extensiones singulares para alcanzar ocasionalmente una profundidad de 40 jugadas.

Los mejores programas de ajedrez hoy en día (por ejemplo, Stockfish, Komodo, Houdini) superan con creces a cualquier jugador humano. Éstos han reducido el factor de ramificación efectivo a menos de 3 (en comparación con el factor de ramificación real de alrededor de 35), buscando cerca de 20 jugadas a una velocidad de casi un millón de nodos por segundo en una computadora estándar de un núcleo. Usan técnicas de poda como la heurística de **movimiento nulo**, que genera un buen límite inferior en el valor de una posición, usando una búsqueda superficial en la que el oponente tiene que moverse dos veces al principio. También es importante la **poda de futilidad**, que ayuda a decidir de antemano qué movimientos causarán un corte beta en los nodos sucesores. Sunfish es un programa de ajedrez simplificado para fines educativos; el núcleo es de menos de 200 líneas de Python.

Movimiento nulo

Poda de futilidad

La idea del análisis retrogrado para calcular las tablas de finales de ajedrez se debe a Bellman (1965). A partir de esta idea, Ken Thompson (1986, 1996) y Lewis Stiller (1992, 1996) resolvieron todos los finales de ajedrez con hasta cinco piezas. Stiller descubrió un caso donde existía un mate forzado pero se requerían 262 movimientos; esto causó cierta consternación porque las reglas del ajedrez requieren que se produzca una captura o un movimiento de peón dentro de 50 movimientos, o de lo contrario se declara un empate. En 2012, Vladimir Makhnychev y Victor Zakharov compilaron la tabla de finales de Lomonosov, que resolvió todas las posiciones de final con hasta siete piezas, algunas requieren más de

500 movimientos sin una captura. La tabla para 7 piezas consume 140 terabytes; una tabla para 8 piezas sería 100 veces más grande.

En 2017, AlphaZero (Silver *et al*., 2018) derrotó a Stockfish (el campeón de ajedrez de computadora TCEC 2017) en una prueba de 1 000 juegos, con 155 victorias y 6 derrotas. Partidas adicionales también resultaron en victorias decisivas para AlphaZero, aun cuando se le dio sólo la décima parte del tiempo asignado a Stockfish.

El Gran Maestro Larry Kaufman se sorprendió del éxito de este programa de Monte Carlo y señaló: "Puede ser que la actual dominación de los motores de ajedrez minimax haya llegado a su fin, pero es demasiado pronto para decirlo". Garry Kasparov comentó: "Es un logro notable, incluso si deberíamos haberlo esperado después de AlphaGo. Se acerca al enfoque humano tipo B para el ajedrez de máquina soñado por Claude Shannon y Alan Turing en lugar de la fuerza bruta". Continuó prediciendo: "El ajedrez ha sido sacudido hasta sus raíces por AlphaZero, pero este es solo un pequeño ejemplo de lo que está por venir. Disciplinas rígidas como la educación y la medicina también serán sacudidas" (Sadler y Regan, 2019).

Checkers fue el primer juego clásico jugado por una computadora (Strachey, 1952). Arthur Samuel (1959, 1967) desarrolló un programa de damas que aprendió su propia función de evaluación a través del autojuego usando una forma de aprendizaje por refuerzo. Es todo un logro que Samuel haya sido capaz de crear un programa que jugara mejor que él, en una computadora IBM 704 con sólo 10 000 palabras de memoria y un procesador de 0.000001 GHz. MENACE —la Máquina Educable de Noughts y Cruces (Michie, 1963)— también usó el aprendizaje por refuerzo para competir en el juego de Gato. Su procesador era aún más lento: una colección de 304 cajas de fósforos que contenían cuentas de colores para representar el mejor movimiento aprendido en cada posición.

En 1992, el programa de damas Chinook de Jonathan Schaeffer desafió al legendario Marion Tinsley, quien había sido campeón mundial durante más de 20 años. Tinsley ganó la partida, pero perdió dos juegos, las cuarta y quinta derrotas en toda su carrera. Después de que Tinsley se retirara por motivos de salud, Chinook tomó la corona. La saga fue narrada por Schaeffer (2008).

En 2007 Schaeffer y su equipo "resolvieron" las damas (Schaeffer *et al*., 2007): el juego es un empate con un juego perfecto. Richard Bellman (1965) había predicho: "En las damas, el número de movimientos posibles en cualquier situación dada es tan pequeño que podemos esperar con confianza una solución completa de la computadora digital al problema del juego óptimo en este juego". Bellman no anticipó la escala del esfuerzo: la tabla final del juego para 10 piezas tiene 39 billones de entradas. Dada esta tabla, se necesitaron 18 años de CPU de búsqueda alfa-beta para resolver el juego.

I. J. Good, a quien Alan Turing le enseño el juego de **Go**, escribió (1965a) "Creo que será aún más difícil programar una computadora para jugar un juego razonable de Go que de ajedrez". Tenía razón: hasta 2015, los programas de Go sólo jugaban a nivel amateur. La literatura temprana se resume en Bouzy y Cazenave (2001) y Müller (2002).

El reconocimiento de patrones visuales fue propuesto como una técnica prometedora para Go por Zobrist (1970), mientras que Schraudolph *et al*. (1994) analizaron el uso del aprendizaje por refuerzo, Lubberts y Miikkulainen (2001) recomendaron redes neuronales, y Brügmann (1993) introdujo la búsqueda de árbol de Monte Carlo en Go. AlphaGo (Silver *et al.,* 2016) unió esas cuatro ideas para derrotar a los profesionales mejor clasificados Lee Sedol (por un marcador de 4-1 en 2015) y Ke Jie (por 3-0 en 2016).

Ke Jie comentó: "Después de que la humanidad pasara miles de años mejorando nuestras tácticas, las computadoras nos dicen que los humanos están completamente equivocados. Iría tan lejos como para decir que no hay un solo humano que haya tocado el borde de la verdad de Go".

Lee Sedol se retiró del Go, lamentando: "Incluso si me convirtiera en el número uno, hay una entidad que no puede ser derrotada".

En 2018, AlphaZero superó a AlphaGo en Go, y también derrotó a los mejores programas en ajedrez y shogi, aprendiendo a través del autojuego sin ningún conocimiento humano experto y sin acceso a ningún juego pasado. (Claro, depende de los humanos para definir la arquitectura básica como la búsqueda de árboles de Monte Carlo con redes neuronales profundas y aprendizaje por refuerzo, y para codificar las reglas del juego). El éxito de AlphaZero ha llevado a un mayor interés en el aprendizaje por refuerzo como componente clave de la IA general (ver Capítulo 22). Yendo un paso más allá, el sistema MuZero opera sin siquiera que se le indiquen las reglas del juego que está jugando, tiene que descubrir las reglas haciendo jugadas. MuZero logró resultados de vanguardia en Pacman, ajedrez, Go y 75 juegos de Atari (Schrittwieser *et al*., 2019). Aprende a generalizar; por ejemplo, aprende que en Pacman la acción "arriba" mueve al jugador hacia arriba en un cuadrado (a menos que haya una pared allí), aunque sólo ha observado el resultado de la acción "arriba" en un pequeño porcentaje de las ubicaciones en el tablero.

Othello, también llamado Reversi, tiene un espacio de búsqueda más pequeño que el ajedrez, pero definir una función de evaluación es difícil, porque la ventaja material no es tan importante como la movilidad. Los programas han estado en un nivel sobrehumano desde 1997 (Buro, 2002).

Backgammon, un juego de azar, fue analizado matemáticamente por Gerolamo Cardano (1663), y utilizado para jugar en computadora con el programa BKG (Berliner, 1980b), que usó una función de evaluación construida manualmente y buscó sólo hasta una profundidad de 1. Fue el primer programa en derrotar a un campeón mundial humano en un juego importante (Berliner, 1980a), aunque Berliner reconoció fácilmente que BKG tuvo mucha suerte con los dados. TD-gammon de Gerry Tesauro (1995) aprendió su función de evaluación usando redes neuronales entrenadas por auto-juego. Jugó consistentemente a nivel de campeón mundial y causó que los analistas humanos cambiaran su opinión sobre el mejor movimiento de apertura para varios lanzamientos de dados.

Póquer, al igual que Go, ha experimentado avances sorprendentes en los últimos años. Bowling *et al.* (2015) usaron la teoría de juegos (ver Sección 18.2) para determinar la estrategia óptima exacta para una versión de póquer con sólo dos jugadores y un número fijo de subidas con tamaños de apuesta fijos. En 2017, por primera vez, los jugadores campeones de póquer fueron vencidos en heads-up (dos jugadores) sin límite Texas hold 'em en dos partidas separadas contra los programas Libratus (Brown y Sandholm, 2017) y DeepStack (Moravčík *et al*., 2017). En 2019, Pluribus (Brown y Sandholm, 2019) derrotó a jugadores humanos profesionales de primer nivel en juegos de Texas hold 'em con seis jugadores. Los juegos multijugador presentan algunas preocupaciones estratégicas que cubriremos en el Capítulo 18. Petosa y Balch (2019) implementan una versión multijugador de AlphaZero.

Bridge: Smith *et al.* (1998) informan sobre cómo Bridge Baron ganó el campeonato de bridge por ordenador de 1998, usando planes jerárquicos (véase el Capítulo 11) y acciones de alto nivel, como el finessing y el squeeze, familiares para los jugadores de bridge. Ginsberg (2001) describe cómo su programa GIB, basado en la simulación de Monte Carlo (propuesto por primera vez para bridge por Levy (1989)), ganó el siguiente campeonato de computación y lo hizo sorprendentemente bien contra jugadores humanos expertos. En el siglo xxi, el campeonato de bridge por computadora ha sido dominado por dos programas comerciales, Jack y Wbridge5. Ninguno de los dos ha sido descrito en artículos publicados, pero se cree que ambos usan técnicas de Monte Carlo. En general, los programas de bridge están al nivel de los campeones humanos cuando realmente juegan las manos, pero se quedan atrás en la fase de subasta, porque no entienden completamente las convenciones utilizadas por los humanos para comunicarse con sus compañeros. Los programadores de bridge se han concentrado más en la producción

de programas útiles y educativos que animen a la gente a iniciarse en el juego, en lugar de derrotar a los campeones humanos.

Scrabble es un juego en el que los jugadores humanos aficionados tienen dificultades para encontrar palabras con una puntuación alta, pero para una computadora es fácil encontrar la puntuación más alta posible para una mano dada (Gordon, 1994); la parte difícil es planificar con anticipación en un juego parcialmente observable y estocástico. Sin embargo, en 2006, el programa QUACKLE derrotó al ex campeón mundial, David Boys, 3-2. Boys lo tomó bien, diciendo: "Todavía es mejor ser humano que ser una computadora". Una buena descripción de un programa superior, MAVEN, es dada por Sheppard (2002).

Los **videojuegos** como **StarCraft II** involucran cientos de unidades parcialmente observables que se mueven en tiempo real con espacios de observación y acción de alta dimensión casi continuos[6] y reglas complejas. Oriol Vinyals, quien fue el campeón de StarCraft de España a los 15 años, describió cómo el juego puede servir como un banco de pruebas y un gran desafío para el aprendizaje por refuerzo (Vinyals *et al*., 2017a). En 2019, Vinyals y el equipo de DeepMind presentaron el programa ALPHASTAR, basado en aprendizaje profundo y aprendizaje por refuerzo, que derrotó a jugadores humanos expertos en 10 juegos a 1, y se ubica en el 0.02 % superior de los jugadores humanos clasificados oficialmente (Vinyals *et al.,* 2019). ALPHASTAR tomó medidas para limitar el número de acciones por minuto que podía realizar en ráfagas críticas, en respuesta a los críticos que sentían que tenía una ventaja injusta.

Computadoras han derrotado a los mejores humanos en otros juegos de video populares como Super Smash Bros. (Firoiu *et al.,* 2017), Quake III (Jaderberg *et al*., 2019) y Dota 2 (Fernandez y Mahlmann, 2018), todos usando técnicas de aprendizaje profundo.

Los **juegos físicos** como el **fútbol robótico** (Visser *et al*., 2008; Barrett y Stone, 2015), **billar** (Lam y Greenspan, 2008; Archibald *et al*., 2009) y **ping-pong** (Silva *et al*., 2015) han atraído cierta atención en IA. Combinan todas las complicaciones de los juegos de video con el desorden del mundo real.

Las competiciones de juegos de computadora ocurren anualmente, incluyendo las Olimpiadas de Computadoras desde 1989. La Competencia General de Juegos (Love *et al.,* 2006) prueba programas que deben aprender a jugar un juego desconocido dado sólo una descripción lógica de las reglas del juego. La Asociación Internacional de Juegos de Computadora (ICGA) publica la *ICGA Journal* y organiza dos conferencias bienales alternas, la Conferencia Internacional sobre Computadoras y Juegos (ICCG o CG) y la Conferencia Internacional sobre Avances en Juegos de Computadora (ACG). El IEEE publica las *IEEE Transactions on Games* y organiza una Conferencia anual sobre Inteligencia Computacional y Juegos.

[6] Para un jugador humano, parece que los objetos se mueven continuamente, pero en realidad son discretos a nivel de pixel en la pantalla.

CAPÍTULO 6

PROBLEMAS DE SATISFACCIÓN DE RESTRICCIONES

En el que vemos cómo tratar los estados como algo más que simples cajas negras que conducen a nuevos métodos de búsqueda y una comprensión más profunda de la estructura del problema.

Los capítulos 3 y 4 exploraron la idea de que los problemas pueden resolverse buscando en el espacio de estados; es decir, un gráfico donde los nodos son estados y las aristas entre ellos son acciones. Vimos que las heurísticas específicas del dominio podrían estimar el costo de alcanzar el objetivo desde un estado dado, pero que desde el punto de vista del algoritmo de búsqueda cada estado es atómico o indivisible, una caja negra sin estructura interna. Para cada problema necesitamos un código específico del dominio para describir las transiciones entre estados.

En este capítulo rompemos la caja negra usando una **representación factorizada** para cada estado; esto es, un conjunto de **variables** donde cada una tiene un **valor**. Un problema se resuelve cuando cada variable tiene un valor que satisface todas las restricciones sobre la variable. Un problema descrito de esta manera se llama **problema de satisfacción de restricciones**, o **CSP**.

Problema de satisfacción de restricciones

Los algoritmos de búsqueda de CSP aprovechan la estructura de los estados y usan heurísticas *generales* del dominio, en lugar de específicas, para permitir la solución de problemas complejos. La idea principal es eliminar grandes porciones del espacio de búsqueda de una sola vez al identificar combinaciones de variable/valor que violan las restricciones. Los CSP tienen la ventaja adicional de que las acciones y el modelo de transición pueden deducirse de la descripción del problema.

6.1 Definiendo problemas de satisfacción de restricciones

Un problema de satisfacción de restricciones consta de tres componentes, $\mathcal{X}$, D, y C:

$\mathcal{X}$ es un conjunto de variables, $\{X_1,..., X_n\}$.
D es un conjunto de dominios, $\{D_1,..., D_n\}$, uno para cada variable.
C es un conjunto de restricciones que especifican combinaciones permitidas de valores.

Un dominio, D_i, consta de un conjunto de valores permitidos, $\{v_1,...,v_k\}$, para la variable X_i. Por ejemplo, una variable booleana tendría el dominio {*verdadero*, *falso*}. Diferentes variables pueden tener diferentes dominios de diferentes tamaños. Cada restricción C_j consta de un par $\langle$*alcance*, *rel*$\rangle$, donde el *alcance* es una tupla de variables que participan en la restricción y *rel* es una **relación** que define los valores que esas variables pueden tomar. Una relación puede representarse como un conjunto explícito de todas las tuplas de valores que satisfacen la restricción, o como una función que puede calcular si una tupla es miembro de la relación. Por ejemplo, si X_1 y X_2 tienen ambos el dominio {1,2,3}, entonces la restricción que dice que X_1 debe ser mayor que X_2 puede escribirse como $\langle(X_1,X_2),\{(3,1),(3,2),(2,1)\}\rangle$ o como $\langle(X_1,X_2),X_1 > X_2\rangle$.

Relación

Los CSP tratan con **asignaciones** de valores a variables, $\{X_i = v_i, X_j = v_j, \ldots\}$. Una asignación que no viola ninguna restricción se llama **asignación consistente** o legal. Una **asignación completa** es aquella en la que a cada variable se le asigna un valor, y una **solución** a un CSP es una asignación consistente y completa. Una **asignación parcial** es aquella que deja algunas variables sin asignar, y una **solución parcial** es una asignación parcial que es consistente. Resolver un CSP es un problema NP-completo en general, aunque hay subclases importantes de CSP que pueden resolverse de manera muy eficiente.

Asignaciones
Asignación consistente
Asignación completa
Solución
Asignación parcial
Solución parcial

6.1.1 Problema de ejemplo: Coloreado de mapas

Supongamos que, habiéndonos cansado de Rumania, estamos mirando un mapa de Australia que muestra cada uno de sus estados y territorios (Figura 6.1(a)). Se nos da la tarea de colorear cada región ya sea rojo, verde o azul, de tal manera que ningún par de regiones vecinas tenga el mismo color. Para formular esto como un CSP, definimos las variables para ser las regiones:

$$\chi = \{WA, NT, Q, NSW, V, SA, T\}.$$

El dominio de cada variable es el conjunto $D_i = \{rojo, verde, azul\}$. Las restricciones requieren que las regiones vecinas tengan colores distintos. Dado que hay nueve lugares donde las regiones limitan, hay nueve restricciones:

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, \\ WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}.$$

Aquí estamos usando abreviaturas; $SA \neq WA$ es un atajo para $\langle (SA, WA), SA \neq WA \rangle$, donde $SA \neq WA$ puede ser completamente enumerado a su vez, como

$$\{(rojo, verde), (rojo, azul), (verde, rojo), (verde, azul), (azul, rojo), (azul, verde)\}.$$

Hay muchas soluciones posibles a este problema, como

$$\{WA = rojo, NT = verde, Q = rojo, NSW = verde, V = rojo, SA = azul, T = rojo\}.$$

Puede ser útil visualizar un CSP como un **grafo de restricciones**, como se muestra en la Figura 6.1(b). Los nodos del grafo corresponden a las variables del problema, y una arista conecta cualquier par de variables que participan en una restricción.

Grafo de restricciones

¿Por qué formular un problema como un CSP? Una razón es que los CSPs proporcionan una representación natural para una amplia variedad de problemas; formular un problema como un CSP suele ser fácil. Otra es que años de trabajo de desarrollo han resultado en hacer que los solucionadores de CSP sean rápidos y eficientes. Una tercera es que un solucionador de CSP puede podar rápidamente grandes extensiones del espacio de búsqueda, cosa que un buscador de espacio de estados atómicos no hace. Por ejemplo, una vez que hemos elegido $\{SA = azul\}$ en el problema de Australia, podemos concluir que ninguna de las cinco variables vecinas puede tomar el valor *azul*. Un procedimiento de búsqueda que no usa restricciones tendría que considerar $3^5 = 243$ asignaciones para las cinco variables vecinas; como restricciones sólo tenemos $2^5 = 32$ asignaciones a considerar, una reducción de 87 %.

En la búsqueda de espacio de estados atómicos sólo podemos preguntar: ¿es este estado específico un objetivo? ¿No? ¿Qué tal este otro? Con los CSPs, una vez que descubrimos que una asignación parcial viola una restricción, podemos descartar de inmediato refinamientos adicionales de la asignación parcial. Además, podemos ver por qué la asignación no es una solución, vemos qué variables violan una restricción, por lo que podemos centrar la atención en las variables que importan. Como resultado, muchos problemas intratables para la búsqueda de espacio de estados atómicos pueden resolverse rápido cuando se formulan como un CSP.

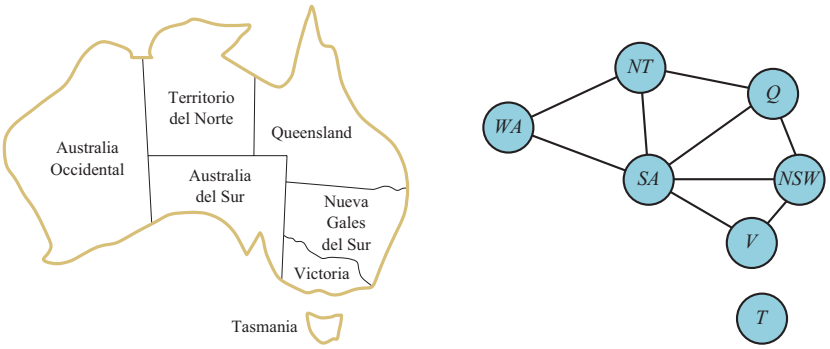


Figura 6.1 (a) Principales estados y territorios de Australia. Colorear este mapa puede aparecer como un problema de satisfacción de restricciones (CSP). El objetivo es asignar colores a cada región de manera que ninguna región vecina tenga el mismo color. (b) El problema de colorear el mapa representado como un grafo de restricciones.

6.1.2 Problema de ejemplo: Programación de talleres

Las fábricas tienen el problema de programar cada día de trabajo, sujeto a diversas restricciones. En la práctica, muchos de estos problemas se resuelven con técnicas de CSP. Consideremos el problema de programar el ensamble de un coche. Todo el trabajo se compone de tareas y cada tarea podemos modelarla como una variable, donde el valor de cada variable es el tiempo en que comienza la tarea, un tiempo expresado como un número entero de minutos. Las restricciones pueden afirmar que una tarea debe ocurrir antes que otra; por ejemplo, una rueda debe ser instalada antes de que se coloque el tapón, y que sólo ciertas tareas pueden realizarse a la vez. Las restricciones también pueden especificar que una tarea lleva cierta cantidad de tiempo para completarse.

Considere una pequeña parte del ensamble del coche, que consta de 15 tareas: instalar ejes (delantero y trasero), fijar las cuatro ruedas (derechas e izquierdas, delanteras y traseras), apretar las tuercas de cada rueda, fijar los tapones y dar una inspección final al ensamble. Podemos representar las tareas con 15 variables:

$$\chi = \{Axle_F, Axle_B, Wheel_{RF}, Wheel_{LF}, Wheel_{RB}, Wheel_{LB}, Nuts_{RF}, Nuts_{LF}, Nuts_{RB}, Nuts_{LB}, Cap_{RF}, Cap_{LF}, Cap_{RB}, Cap_{LB}, Inspect\}.$$

Restricciones de precedencia

Ahora, representamos las **restricciones de precedencia** entre las tareas individuales. Siempre que una tarea T_1 debe ocurrir antes que la tarea T_2, y la tarea T_1 tarda d_1 en completarse, añadimos una restricción aritmética de la forma

$$T_1 + d_1 \leq T_2.$$

En el ejemplo, los ejes (*axles*) tienen que estar en su lugar antes de colocar las ruedas (*wheels*), y se tarda 10 minutos en instalar un eje, así que escribimos

$$Axle_F + 10 \leq Wheel_{RF}; \; Axle_F + 10 \leq Wheel_{LF};$$
$$Axle_B + 10 \leq Wheel_{RB}; \; Axle_B + 10 \leq Wheel_{LB}.$$

A continuación decimos que para cada rueda, debemos fijar la rueda (que tarda 1 minuto), luego apretar las tuercas (*nuts*) (2 minutos), y al final colocar el tapón (1 minuto, aún no representado):

$$Wheel_{RF} + 1 \leq Nuts_{RF}; \; Nuts_{RF} + 2 \leq Cap_{RF};$$
$$Wheel_{LF} + 1 \leq Nuts_{LF}; \; Nuts_{LF} + 2 \leq Cap_{LF};$$
$$Wheel_{RB} + 1 \leq Nuts_{RB}; \; Nuts_{RB} + 2 \leq Cap_{RB};$$
$$Wheel_{LB} + 1 \leq Nuts_{LB}; \; Nuts_{LB} + 2 \leq Cap_{LB}$$

Suponga que tenemos cuatro trabajadores para instalar ruedas, pero tienen que compartir la herramienta para poner el eje en su lugar. Necesitamos una **restricción disyuntiva** para decir que $Axle_F$ y $Axle_B$ no deben superponerse en el tiempo; uno debe venir primero:

Restricción disyuntiva

$$(Axle_F + 10 \leq Axle_B) \textbf{ o } (Axle_B + 10 \leq Axle_F).$$

Esto parece una restricción más complicada, que combina aritmética y lógica. Pero aún se reduce a un conjunto de pares de valores que $Axle_F$ y $Axle_B$ pueden asumir.

También necesitamos afirmar que la inspección ocurre al final y toma 3 minutos. Para cada variable excepto *Inspect*, agregamos una restricción de la forma $X + d_X \leq Inspect$. Finalmente, suponga que hay un requisito para terminar todo el ensamble en 30 minutos. Podemos lograr eso limitando el dominio de todas las variables:

$$D_i = \{0, 1, 2, 3, \ldots, 30\}.$$

Este problema en particular es trivial de resolver, pero los CSP se han aplicado con éxito a problemas de programación de trabajos como este con miles de variables.

6.1.3 Variaciones en el formalismo CSP

El tipo más sencillo de CSP implica variables que tienen **dominios discretos y finitos**. Los problemas para colorear mapas y la programación con límites de tiempo son ambos de este tipo. El problema de las 8 reinas (Figure 4.3) también puede verse como un CSP de dominio finito, donde las variables $Q_1, \ldots, Q_8$ corresponden a las reinas en las columnas 1 a 8, y el dominio de cada variable especifica los posibles números de fila para la reina en esa columna, $D_i = \{1,2,3,4,5,6,7,8\}$. Las restricciones dicen que dos reinas no pueden estar en la misma fila o diagonal.

Dominio discreto
Dominio finito

Un dominio discreto puede ser **infinito**, como el conjunto de enteros o cadenas. (Si no pusiéramos un plazo en el problema de programación de trabajos, habría un número infinito de tiempos de inicio para cada variable). Con dominios infinitos, debemos usar restricciones implícitas como $T_1 + d_1 \leq T_2$ en lugar de tuplas explícitas de valores. Existen algoritmos de solución especiales (no discutidos aquí) para **restricciones lineales** en variables enteras, es decir, restricciones como la que acaba de darse en las que cada variable aparece sólo en forma lineal. Se puede demostrar que no existe ningún algoritmo para resolver **restricciones no lineales** generales en variables enteras, el problema es indecidible.

Infinito

Restricciones lineales

Restricciones no lineales

Los problemas de satisfacción de restricciones con **dominios continuos** son comunes en el mundo real y se estudian ampliamente en el campo de la investigación de operaciones. Por ejemplo, la programación de experimentos en el Telescopio Espacial Hubble requiere un tiempo muy preciso de observaciones; el inicio y el final de cada observación y maniobra son variables de valor continuo que deben obedecer una variedad de restricciones astronómicas, de precedencia y de energía. La categoría más conocida de CSP de dominio continuo es la de los problemas de **programación lineal**, donde las restricciones deben ser igualdades o desigualdades lineales. Los problemas de programación lineal se pueden resolver en tiempo polinomial en el número de variables. También se han estudiado problemas con diferentes tipos de restricciones y funciones objetivas: programación cuadrática, programación cónica de segundo orden, etc. Estos problemas constituyen un área importante de las matemáticas aplicadas.

Dominios continuos

Además de examinar los tipos de variables que pueden aparecer en los CSP, es útil mirar los tipos de restricciones. El tipo más simple es la **restricción unaria**, que restringe el valor de una sola variable. Por ejemplo, en el problema de colorear el mapa, podría ser el caso de que los australianos del sur no toleren el color verde; podemos expresar eso con la restricción unaria $\langle(SA), SA \neq green\rangle$. (La especificación inicial del dominio de una variable también puede verse como una restricción unaria).

Restricción unaria

Una **restricción binaria** relaciona dos variables. Por ejemplo, $SA \neq NSW$ es una restricción binaria. Un **CSP binario** es uno con sólo restricciones unarias y binarias; se puede representar como un grafo de restricción, como en la Figura 6.1(b).

Restricción binaria

CSP binario

También podemos definir restricciones de orden superior. La restricción ternaria *Between*(X,Y,Z); por ejemplo, se puede definir como $\langle(X,Y,Z), X < Y < Z$ **o** $X > Y > Z\rangle$.

Una restricción que involucra un número arbitrario de variables se llama **restricción global**. (El nombre es tradicional pero confuso porque una restricción global no necesita involucrar *todas* las variables en un problema). Una de las restricciones globales más comunes es *Alldiff*, que dice que todas las variables involucradas en la restricción deben tener valores diferentes. En los problemas de Sudoku (ver Sección 6.2.6), todas las variables en una fila, columna o caja de 3 × 3 deben satisfacer una restricción *Alldiff*.

Restricción global

Otro ejemplo es proporcionado por los rompecabezas **criptoaritméticos** (Figura 6.2(a)). Cada letra en un rompecabezas criptoaritmético representa un dígito diferente. Para el caso en la Figura 6.2(a), esto se representaría como la restricción global *Alldiff*(F,T,U,W,R,O). Las restricciones de suma en las cuatro columnas del rompecabezas pueden ser escritas como las siguientes restricciones n-arias:

Criptoaritméticos

$$O + O = R + 10 \bullet C_1$$
$$C_1 + W + W = U + 10 \bullet C_2$$
$$C_2 + \mathrm{T} + \mathrm{T} = O + 10 \bullet C_3$$
$$C_3 = F,$$

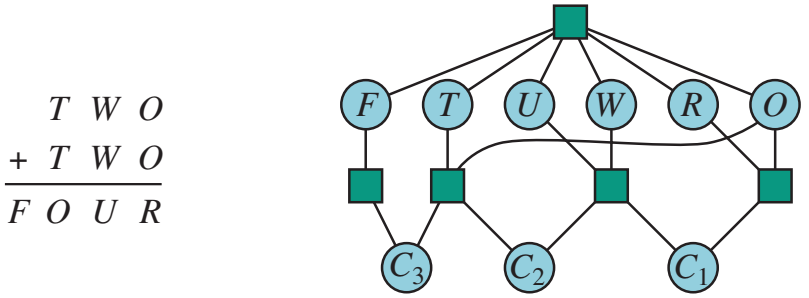
donde C_1, C_2 y C_3 son variables auxiliares que representan el dígito llevado a las columnas de decenas, centenas o miles. Estas restricciones pueden ser representadas en un **hipergrafo de restricciones**, como el que se muestra en la Figura 6.2(b). Un hipergrafo consta de nodos ordinarios (los círculos en la figura) e hipernodos (los cuadrados), que representan restricciones n-arias, restricciones que involucran n variables.

Hipergrafo de restricciones

Alternativamente, como el Ejercicio 6.NARY le pide demostrar, cada restricción de dominio finito puede ser reducida a un conjunto de restricciones binarias si se introducen suficientes variables auxiliares. Esto significa que podríamos transformar cualquier CSP en uno con sólo restricciones binarias, lo que ciertamente simplifica la vida del diseñador del algoritmo. Otra forma de convertir un CSP n-ario en uno binario es la transformación del **grafo dual**: crear un nuevo grafo en el que habrá una variable para cada restricción en el grafo original, y una restricción binaria para cada par de restricciones en el grafo original que comparten variables.

Grafo dual

Por ejemplo, considere un CSP con las variables $\chi = \{X,Y,Z\}$, cada una con el dominio $\{1,2,3,4,5\}$, y con las dos restricciones C_1: $\langle(X,Y,Z), X+Y=Z\rangle$ y C_2: $\langle(X,Y), X+1=Y\rangle$. Entonces el grafo dual tendría las variables $\chi = \{C_1,C_2\}$, donde el dominio de la variable C_1 en el grafo dual es el conjunto de las tuplas $\{(x_i,y_j,z_k)\}$ de la restricción C_1 en el problema original, y de manera similar el dominio de C_2 es el conjunto de las tuplas $\{(x_i,y_j)\}$. El grafo dual tiene la restricción binaria $\langle(C_1,C_2),R_1\rangle$, donde R_1 es una nueva relación que define la restricción entre C_1 y C_2; en este caso sería $R_1 = \{((1,2,3),(1,2)), ((2,3,5),(2,3))\}$.



(a) (b)

Figura 6.2 (a) Problema criptoaritmético. Cada letra representa un dígito distinto, y el objetivo es encontrar una sustitución de dígitos por letras de modo que la suma resultante sea aritméticamente correcta, con la restricción adicional de que no se permiten ceros a la izquierda. (b) El hipergrafo de restricción para el problema criptoaritmético, mostrando la restricción *Alldiff* (caja cuadrada en la parte superior) así como las restricciones de suma de columnas (cuatro cajas cuadradas en el medio). Las variables C_1, C_2 y C_3 representan los dígitos de acarreo para las tres columnas de derecha a izquierda.

Sin embargo, hay dos razones por las que podríamos preferir una restricción global como *Alldiff* en lugar de un conjunto de restricciones binarias. Primero, es más fácil y menos propenso a errores escribir la descripción del problema usando *Alldiff*. Segundo, es posible diseñar algoritmos de inferencia de propósito especial para restricciones globales más eficientes que operar con restricciones primitivas. Describimos estos algoritmos de inferencia en la Sección 6.2.5.

Las restricciones que hemos descrito hasta ahora han sido todas restricciones absolutas, en las que la violación de cualquiera de ellas descarta una solución potencial. Muchos CSPs del mundo real incluyen **restricciones de preferencia** las cuales indican qué soluciones son preferidas. Por ejemplo, en un problema de programación de clases universitarias existen restricciones absolutas de que ningún profesor puede enseñar dos clases al mismo tiempo. Pero también podemos permitir restricciones de preferencia: el profesor R podría preferir enseñar por la mañana, mientras que el profesor N prefiere enseñar por la tarde. Un horario que tiene al profesor R enseñando a las 2 p.m. seguiría siendo una solución permitida (a menos que el profesor R resulte ser el jefe del departamento) pero no sería una solución óptima.

Restricciones de preferencia

Las restricciones de preferencia suelen ser codificadas como costos en asignaciones de variables individuales; por ejemplo, asignar un espacio de tarde para el profesor R cuesta 2 puntos contra la función objetivo general, pero un espacio de mañana cuesta 1. Con esta formulación, los CSPs con preferencias pueden ser resueltos con métodos de búsqueda de optimización, ya sea basados en caminos o locales. Llamamos a tal problema un **problema de optimización con restricciones**, o COP. Los programas lineales son una clase de COPs.

Problema de optimización con restricciones

6.2 Propagación de restricciones: Inferencia en CSPs

Un algoritmo de búsqueda en el espacio de estados atómicos avanza de una sola manera: expandiendo un nodo para visitar los sucesores. Un algoritmo CSP tiene opciones. Puede generar sucesores al elegir una nueva asignación de variables, o puede hacer un tipo específico de inferencia, llamada **propagación de restricciones**: usando las restricciones para reducir el número de valores legales para una variable que, a su vez, puede reducir los valores legales para otra variable, etc. La

Propagación de restricciones

idea es que esto dejará menos opciones a considerar cuando hagamos la próxima elección de una asignación de variable. La propagación de restricciones puede o estar entrelazada con la búsqueda o puede hacerse como un paso de preprocesamiento, antes de que comience la búsqueda. A veces este preprocesamiento puede resolver todo el problema, por ello no se requiere ninguna búsqueda.

Consistencia local

La idea clave es la **consistencia local**. Si tratamos cada variable como un nodo en un grafo (ver Figura 6.1(b)) y cada restricción binaria como un borde, entonces el proceso de hacer cumplir la consistencia local en cada parte del grafo hace que los valores inconsistentes sean eliminados en todo el grafo. Existen diferentes tipos de consistencia local, que a continuación cubriremos en orden.

6.2.1 Consistencia de nodo

Consistente con el nodo

Una sola variable (correspondiente a un nodo en el grafo CSP) es **consistente con el nodo** si todos los valores en el dominio de la variable satisfacen las restricciones unarias de la variable. Por ejemplo, en la variante del problema de colorear el mapa de Australia (Figura 6.1), donde los australianos del sur no les gusta el verde, la variable *SA* comienza con el dominio {*rojo*, *verde*, *azul*}, y podemos hacerla consistente con el nodo si eliminamos el *verde*, y dejamos *SA* con el dominio reducido {*rojo*, *azul*}. Decimos que un grafo es consistente con el nodo si cada variable en el grafo es consistente con el nodo.

Es fácil eliminar todas las restricciones unarias en un CSP reduciendo el dominio de las variables con restricciones unarias al inicio del proceso de resolución. Como antes se mencionó, también es posible transformar todas las restricciones *n*-arias en binarias. Debido a esto, algunos solucionadores de CSP trabajan sólo con restricciones binarias, esperando que el usuario elimine las otras restricciones con anticipación. Hacemos esa suposición para el resto de este capítulo, excepto donde se indique.

6.2.2 Arco consistencia (consistencia de arco)

Arco-consistente

Una variable en un CSP es **arco-consistente**[1] si cada valor en su dominio satisface las restricciones binarias de la variable. Más formalmente, X_i es arco-consistente con respecto a otra variable X_j si para cada valor en el dominio actual D_i hay algún valor en el dominio D_j que satisface la restricción binaria en el arco (X_i,X_j). Un grafo es arco-consistente si cada variable es arco-consistente con cada otra variable. Por ejemplo, considere la restricción $Y = X^2$ donde el dominio de X e Y es el conjunto de dígitos decimales. Podemos escribir esta restricción explícitamente como

$$\langle (X,Y),\{(0,0),(1,1),(2,4),(3,9)\}\rangle.$$

Para hacer que X sea arco-consistente con respecto a Y, reducimos el dominio de X a {0,1,2,3}. Si también hacemos que Y sea arco-consistente con respecto a X, entonces el dominio de Y se convierte en {0,1,4,9}, y todo el CSP es arco-consistente. Por otro lado, la consistencia de arco no puede hacer nada por el problema de colorear el mapa de Australia. Considere la siguiente restricción de desigualdad en (*SA*, *WA*):

$$\{(rojo,verde),(rojo,azul),(verde,rojo),(verde,azul),(azul,rojo),(azul,verde)\}.$$

No importa qué valor elija para *SA* (o para *WA*), hay un valor válido para la otra variable. Por lo tanto, aplicar arco consistencia no tiene efecto en los dominios de ninguna de las variables.

El algoritmo más popular para hacer cumplir la arco-consistencia se llama AC-3 (ver Figura 6.3). Para hacer que cada variable sea arco-consistente, el algoritmo AC-3 mantiene una cola de arcos a considerar. Inicialmente, la cola contiene todos los arcos en el CSP. (Cada restricción binaria se convierte en dos arcos, uno en cada dirección). AC-3 luego saca un arco arbitrario (X_i,X_j) de la cola y hace

[1] Hemos usado el término "borde" en lugar de "arco", por lo que tendría más sentido llamar a esto "consistencia de borde", pero el nombre "consistencia de arco" o "arco-consistencia" es histórico.

```
function AC-3(csp) returns falso si se encuentra una inconsistencia y es true de otra manera
  queue ← una cola de arcos, inicialmente todos los arcos en csp

  while queue is not empty do
    (X_i, X_j)←POP(queue)
    if REVISE(csp,X_i,X_j) then
      if size of D_i = 0 then return false
      for each X_k in X_i.NEIGHBORS - {X_j} do
        add (X_k,X_i) to queue
  return true

function REVISE(csp,X_i,X_j) returns true si revisamos el dominio de X_i
  revised←false
  for each x in D_i do
    if ningún valor y en D_j permite que (x,y) satisfaga la restricción entre X_i y X_j, then
      delete x from D_i
      revised←true
  return revised
```

Figura 6.3 El algoritmo de arco-consistencia AC-3. Después de aplicar AC-3, cada arco es arco-consistente, o alguna variable tiene un dominio vacío, indicando que el CSP no puede ser resuelto. El nombre "AC-3" fue usado por el inventor del algoritmo (Mackworth, 1977) porque era la tercera versión desarrollada en el papel.

que X_i sea arco-consistente con respecto a X_j. Si esto deja D_i sin cambios, el algoritmo sólo pasa al siguiente arco. Pero si esto reduce D_i (hace que el dominio sea más pequeño), entonces añadimos a la cola todos los arcos (X_k,X_i) donde X_k es un vecino de X_i. Necesitamos hacer eso porque el cambio en D_i podría permitir más reducciones en D_k, incluso si hemos considerado previamente X_k. Si D_i se reduce hasta quedar vacío, entonces sabemos que todo el CSP no tiene una solución consistente, y AC-3 puede devolver inmediatamente el fallo. De lo contrario, seguimos comprobando, tratando de eliminar valores de los dominios de las variables hasta que no hay más arcos en la cola. En ese punto, nos quedamos con un CSP equivalente al CSP original, ambos tienen las mismas soluciones, pero el CSP que verifica la consistencia de arco será más rápido para buscar porque sus variables tienen dominios más pequeños. En algunos casos, resuelve por completo el problema (al reducir cada dominio a tamaño 1) y en otros demuestra que no existe ninguna solución (al reducir algún dominio a tamaño 0).

La complejidad de AC-3 puede ser analizada de la siguiente manera. Suponga un CSP con n variables, cada una con un tamaño de dominio de al menos d, y con c restricciones binarias (arcos). Cada arco (X_k,X_i) puede ser insertado en la cola sólo d veces porque X_i tiene como máximo d valores para eliminar. Comprobar la consistencia de un arco puede hacerse en tiempo $O(d^2)$, por lo que obtenemos un tiempo total en el peor de los casos de $O(cd^3)$.

6.2.3 Consistencia de camino

Supongamos que vamos a colorear el mapa de Australia con sólo dos colores, rojo y azul. La consistencia de arco no hace nada porque cada restricción puede ser satisfecha de manera individual con rojo en un extremo y azul en el otro. Pero es evidente que no hay solución al problema: porque Australia Occidental, Territorio del Norte y Australia del Sur se tocan entre sí, necesitamos al menos tres colores para ellos solos.

Consistencia de camino

La consistencia de arco refuerza los dominios (restricciones unarias) usando los arcos (restricciones binarias). Para avanzar en problemas como el coloreado de mapas, necesitamos una noción más fuerte de consistencia. La **consistencia de camino** refuerza las restricciones binarias usando restricciones implícitas que se infieren al mirar tríos de variables.

Un conjunto de dos variables $\{X_i,X_j\}$ es camino-consistente con respecto a una tercera variable X_m si, para cada asignación $\{X_i = a,X_j = b\}$ consistente con las restricciones (si las hay) en $\{X_i,X_j\}$, hay una asignación a X_m que satisface las restricciones en $\{X_i,X_m\}$ y $\{X_m,X_j\}$. El nombre se refiere a la consistencia general del camino de X_i a X_j con X_m en el medio.

Veamos cómo se comporta la consistencia de camino en la coloración del mapa de Australia con dos colores. Haremos que el conjunto $\{WA,SA\}$ es camino-consistente con respecto a *NT*. Comenzamos enumerando las asignaciones consistentes al conjunto. En este caso, sólo hay dos: $\{WA = rojo,SA = azul\}$ y $\{WA = azul,SA = rojo\}$. Podemos ver que con ambas asignaciones *NT* no puede ser ni *rojo* ni *azul* (porque entraría en conflicto con *WA* o *SA*). Como no hay ninguna opción válida para *NT* eliminamos ambas asignaciones y terminamos sin asignaciones válidas para $\{WA,SA\}$. Así, sabemos que no puede haber ninguna solución a este problema.

6.2.4 *K*-consistencia

K-consistencia

Formas más fuertes de propagación pueden ser definidas con la noción de ***k*-consistencia**. Un CSP es *k*–consistente si, para cualquier conjunto de *k*–1 variables y para cualquier asignación consistente a esas variables, siempre se puede asignar un valor consistente a cualquier *k*–ésima variable. 1–consistencia dice que, dado el conjunto vacío, podemos hacer cualquier conjunto de una variable consistente: esto es lo que llamamos consistencia de nodo. 2–consistencia es lo mismo que consistencia de arco. Para grafos de restricción binaria, 3–consistencia es lo mismo que consistencia de camino.

Fuertemente k-consistente

Un CSP es **fuertemente *k*-consistente** si es *k*-consistente y también es (*k*–1) consistente, (*k*–2)-consistente, ... todo el camino hasta 1–consistente. Ahora suponga que tenemos un CSP con *n* nodos y lo hacemos fuertemente *n*–consistente (es decir, fuertemente *k*–consistente para $k = n$). Entonces así podemos resolver el problema: Primero, elegimos un valor consistente para X_1. Luego estamos garantizados para poder elegir un valor para X_2 porque el grafo es 2–consistente, para X_3 porque es 3–consistente, y así sucesivamente. Para cada variable X_i, sólo necesitamos buscar a través de los *d* valores en el dominio para encontrar un valor consistente con $X_1,...,X_i–1$. El tiempo total de ejecución es sólo $O(n^2d)$.

Por supuesto, no hay almuerzo gratis: la satisfacción de restricciones es NP–duro en general, y cualquier algoritmo para establecer *n*–consistencia debe tomar tiempo exponencial en *n* en el peor de los casos. Peor aún, la *n*–consistencia también requiere espacio exponencial en *n*. En la práctica, determinar el nivel apropiado de verificación de consistencia es en su mayoría una ciencia empírica. Calcular 2–consistencia es común, y 3–consistencia menos común.

6.2.5 Restricciones globales

Recuerde que una **restricción global** involucra un número arbitrario de variables (no necesariamente todas las variables). Las restricciones globales ocurren con frecuencia en problemas reales y pueden ser manejadas por algoritmos de propósito especial más eficientes que los métodos de propósito general descritos hasta ahora. Por ejemplo, la restricción *Alldiff* dice que todas las variables involucradas deben tener valores distintos (como en el problema criptoaritmético anterior y los rompecabezas de Sudoku a continuación). Una forma simple de detección de inconsistencia para las restricciones *Alldiff* funciona de la siguiente manera: si *m* variables están involucradas en la restricción, y si tienen *n* posibles valores distintos en total, y $m > n$, entonces la restricción no puede ser satisfecha.

Esto lleva al siguiente algoritmo simple: Primero, elimina cualquier variable en la restricción que tenga un dominio singleton, y elimina el valor de esa variable de los dominios de las variables restantes. Repite mientras haya variables singleton. Si en algún momento se produce un dominio vacío o hay más variables que valores de dominio restantes, entonces se ha detectado una inconsistencia.

Este método puede detectar la inconsistencia en la asignación $\{WA = rojo, NSW = rojo\}$ para la Figura 6.1. Observe que las variables *SA*, *NT*, y *Q* están efectivamente conectadas por una restricción *Alldiff* porque cada par debe tener dos colores diferentes. Después de aplicar AC-3 con la asignación parcial, los dominios de *SA*, *NT*, y *Q* se reducen todos a {*verde*,*azul*}. Es decir, tenemos tres variables y sólo dos colores, por lo que la restricción *Alldiff* es violada. Así, un simple procedimiento de consistencia para una restricción de orden superior es a veces más efectivo que aplicar consistencia de arco a un conjunto equivalente de restricciones binarias.

Restricción de recursos

Otra restricción de orden superior importante es la **restricción de recursos**, a veces llamada la restricción *como máximo(Atmost)*. Por ejemplo, en un problema de programación, donde $P_1,\ldots,P_4$ denote los números de personal asignados a cada una de las cuatro tareas. La restricción de que no se asignen más de 10 personas en total se escribe como $Atmost(10,P_1,P_2,P_3,P_4)$. Podemos detectar una inconsistencia al comprobar la suma de los valores mínimos de los dominios actuales; por ejemplo, si cada variable tiene el dominio {3,4,5,6}, la restricción *como máximo* no puede ser satisfecha. También podemos hacer cumplir la consistencia eliminando el valor máximo de cualquier dominio si no es consistente con los valores mínimos de los otros dominios. Así, si cada variable en nuestro ejemplo tiene el dominio {2,3,4,5,6}, los valores 5 y 6 pueden ser eliminados de cada dominio.

Propagación de límites

Para grandes problemas con valores enteros limitados por recursos, como problemas logísticos que implican mover miles de personas en cientos de vehículos, por lo general no es posible representar el dominio de cada variable como un gran conjunto de enteros y reducir gradualmente ese conjunto mediante métodos de verificación de consistencia. En su lugar, los dominios se representan por límites superiores e inferiores y se gestionan mediante la **propagación de límites**. Por ejemplo, suponga que en un problema de programación de aerolíneas hay dos vuelos, F_1 y F_2, para los cuales los aviones tienen capacidades de 165 y 385 pasajeros, respectivamente. Los dominios iniciales para el número de pasajeros en los vuelos F_1 y F_2 son entonces

$$D_1 = [0,165] \text{ y } D_2 = [0,385].$$

Ahora suponga que tenemos la restricción adicional de que los dos vuelos juntos deben llevar 420 personas: $F_1 + F_2 = 420$. Propagando las restricciones de límites, reducimos los dominios a

$$D_1 = [35,165] \text{ y } D_2 = [255,385].$$

Consistente en límites

Decimos que un CSP es **consistente en límites** si para cada variable *X*, y para ambos valores de límite inferior y superior de *X*, existe algún valor de *Y* que satisface la restricción entre *X* e *Y* para cada variable *Y*. Este tipo de propagación de límites se usa ampliamente en problemas de restricción prácticos.

6.2.6 Sudoku

Sudoku

El popular rompecabezas **Sudoku** ha llevado a millones de personas a los problemas de satisfacción de restricciones, aunque puede que estas no se den cuenta. Un tablero de Sudoku consta de 81 cuadrados, algunos de ellos inicialmente llenos con dígitos del 1 al 9. El rompecabezas consiste en llenar todos los cuadrados restantes de manera que ningún dígito aparezca dos veces en ninguna fila, columna, o caja de 3×3 (ver Figura 6.4). Una fila, columna, o caja se llama **unidad**.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Figura 6.4 (a) Rompecabezas Sudoku y (b) su solución.

Los rompecabezas Sudoku que aparecen en periódicos y libros de rompecabezas tienen la propiedad de que hay exactamente una solución. Aunque algunos pueden ser difíciles de resolver a mano, tomando decenas de minutos, un solucionador CSP puede manejar miles de Sudokus por segundo.

Un Sudoku puede considerarse un CSP con 81 variables, una para cada cuadrado. Usamos los nombres de variables *A1* a *A9* para la fila superior (de izquierda a derecha), hasta *I1* a *I9* para la fila inferior. Los cuadrados vacíos tienen el dominio {1,2,3,4,5,6,7,8,9} y los cuadrados prellenados tienen un dominio que consta de un solo valor. Además, hay 27 diferentes *Alldiff* restricciones, una para cada unidad (fila, columna y caja de 9 cuadrados):

Alldiff (*A1,A2,A3,A4,A5,A6,A7,A8,A9*)
Alldiff (*B1,B2,B3,B4,B5,B6,B7,B8,B9*)
. . .
Alldiff (*A1,B1,C1,D1,E1,F1,G1,H1,I1*)
Alldiff (*A2,B2,C2,D2,E2,F2,G2,H2,I2*)
. . .
Alldiff (*A1,A2,A3,B1,B2,B3,C1,C2,C3*)
Alldiff (*A4, A5, A6, B4, B5, B6, C4, C5, C6*)
. . .

Veamos hasta dónde puede llevarnos la consistencia de arco. Suponga que las restricciones de *Alldiff* se han expandido en restricciones binarias (como $A1 \neq A2$) para que podamos aplicar el algoritmo AC-3 directamente. Considere la variable *E6* de la Figura 6.4(a); el cuadrado vacío entre el 2 y el 8 en la caja del medio. A partir de las restricciones en la caja, podemos eliminar 1, 2, 7 y 8 del dominio de *E6*. A partir de las restricciones en su columna, podemos eliminar 5, 6, 2, 8, 9 y 3 (aunque 2 y 8 ya fueron eliminados). Eso deja *E6* con un dominio de {4}; en otras palabras, sabemos la respuesta para *E6*. Ahora considere la variable *I6*, el cuadrado en la caja inferior del medio rodeado por 1, 3 y 3. Aplicando consistencia de arco en su columna, eliminamos 5, 6, 2, 4 (ya que ahora sabemos que *E6* debe ser 4), 8, 9 y 3. Eliminamos 1 por consistencia de arco

con I5, y nos quedamos sólo con el valor 7 en el dominio de *I6*. Ahora hay 8 valores conocidos en la columna 6, por lo que la consistencia del arco puede inferir que A6 debe ser 1. La inferencia continúa en estas líneas, y eventualmente, AC-3 puede resolver todo el rompecabezas, todas las variables tienen sus dominios reducidos a un solo valor, como se muestra en la Figura 6.4(b).

Por supuesto, el Sudoku pronto perdería su atractivo si cada rompecabezas pudiera ser resuelto por una aplicación mecánica de AC-3; de hecho, AC-3 sólo funciona para los rompecabezas Sudoku más fáciles. Los más difíciles pueden ser resueltos por PC-2, pero a un costo computacional mayor: hay 255 960 diferentes restricciones de camino a considerar en un Sudoku. Para resolver los más difíciles y para progresar de manera eficiente, tendremos que ser más astutos.

De hecho, el atractivo de los rompecabezas Sudoku para el solucionador humano es la necesidad de ser ingenioso en la aplicación de estrategias de inferencia más complejas. Los aficionados les dan nombres coloridos, como "triples desnudos". Esa estrategia funciona de la siguiente manera: en cualquier unidad (fila, columna o caja), encuentra tres cuadrados que cada uno tenga un dominio que contenga los mismos tres números o un subconjunto de esos números. Por ejemplo, los tres dominios podrían ser {1,8}, {3,8} y {1,3,8}. A partir de eso no sabemos qué cuadrado contiene 1, 3 u 8, pero sí sabemos que los tres números deben distribuirse entre los tres cuadrados. Por lo tanto, podemos eliminar 1, 3 y 8 de los dominios de todos los *demás* cuadrados en la unidad.

Es interesante notar hasta dónde podemos llegar sin decir algo específico del Sudoku. Por supuesto, tenemos que decir que hay 81 variables, que sus dominios son los dígitos del 1 al 9 y que hay 27 restricciones de *Alldiff*. Pero más allá de eso, todas las estrategias —consistencia de arco, consistencia de camino, etc.— se aplican en general a todos los CSP, no sólo a los problemas de Sudoku. Incluso los triples desnudos son realmente una estrategia para hacer cumplir la consistencia de las restricciones de *Alldiff* y no son específicos para Sudoku *per se*. Este es el poder del formalismo CSP: para cada nueva área de problema, sólo necesitamos definir el problema en términos de restricciones; luego los mecanismos generales de resolución de restricciones pueden hacerse cargo.

6.3 Búsqueda de retroceso para CSPs

A veces podemos terminar el proceso de propagación de restricciones y aún tener variables con múltiples valores posibles. En ese caso, tenemos que **buscar** una solución. En esta sección cubrimos los algoritmos de búsqueda de retroceso que trabajan en asignaciones parciales; en la próxima sección, analizamos los algoritmos de búsqueda local sobre asignaciones completas.

Considere cómo una búsqueda estándar de profundidad limitada (del Capítulo 3) podría resolver CSPs. Un estado sería una asignación parcial, y una acción extendería la asignación, agregando, digamos, $NSW = rojo$ o $SA = azul$ para el problema de colorear el mapa de Australia. Para un CSP con n variables de tamaño de dominio d, terminaríamos con un árbol de búsqueda donde todas las asignaciones completas (y así todas las soluciones) son nodos hoja a profundidad n. Pero observe que el factor de ramificación en el nivel superior sería nd porque cualquiera de los valores d puede asignarse a cualquiera de las variables n. En el siguiente nivel, el factor de ramificación es $(n - 1)d$, y así sucesivamente para n niveles. Entonces el árbol tiene $n! \cdot d^n$ hojas, ¡aunque sólo hay d^n posibles asignaciones completas!

Conmutatividad

Podemos recuperar ese factor de $n!$ al reconocer una propiedad crucial de los CSP: la **conmutatividad**. Un problema es conmutativo si el orden de aplicación de cualquier conjunto de acciones no importa. En los CSP, no hace ninguna diferencia si primero asignamos $NSW = rojo$ y luego $SA = azul$, o al revés. Por lo tanto, sólo necesitamos considerar una sola variable en cada nodo en el árbol de búsqueda. En la raíz podríamos hacer una elección entre $SA = rojo$, $SA = verde$, y

```
function BACKTRACKING-SEARCH(csp) returns una solución o fallo
  return BACKTRACK(csp,{ })

function BACKTRACK(csp,assignment) returns una solución o fallo
  if assignment está completo then return assignment
  var←SELECT-UNASSIGNED-VARIABLE(csp,assignment)
  for each value in ORDER-DOMAIN-VALUES(csp,var,assignment) do
    if value es consistente con assignment then
      agregue {var = value} a assignment
      inferences←INFERENCE(csp,var,assignment)
      if inferences ≠ fallo then
        agregue inferences a csp
        result←BACKTRACK(csp,assignment)
        if result ≠ fallo then return result
        quite inferences de csp
      quite {var = value} de assignment
  return fallo
```

Figura 6.5 Un algoritmo de retroceso simple para problemas de satisfacción de restricciones. El algoritmo se basa en la búsqueda recursiva de profundidad del Capítulo 3. Las funciones SELECT-UNASSIGNED-VARIABLE y ORDER-DOMAIN-VALUES, implementan las heurísticas de propósito general discutidas en la Sección 6.3.1. La función INFERENCE puede opcionalmente imponer arco-, camino-, o *k*-consistencia, según se desee. Si una elección de valor conduce a un fallo (notado ya sea por INFERENCE o por BACKTRACK), entonces las asignaciones de valor (incluyendo las realizadas por INFERENCE) se retractan y se prueba un nuevo valor.

SA = *azul*, pero nunca elegiríamos entre *NSW* = *rojo* y *SA* = *azul*. Con esta restricción, el número de hojas es d^n, como esperaríamos. En cada nivel del árbol tenemos que elegir con qué variable vamos a tratar, pero nunca tenemos que retroceder sobre esa elección.

La Figura 6.5 muestra un procedimiento de búsqueda de retroceso para CSPs. Elige repetidamente una variable no asignada, y luego prueba todos los valores en el dominio de esa variable por turno, tratando de extender cada uno en una solución a través de una llamada recursiva. Si la llamada tiene éxito, se devuelve la solución, y si falla, la asignación se restaura al estado anterior, y probamos el siguiente valor. Si ningún valor funciona, entonces devolvemos el fallo. Parte del árbol de búsqueda para el problema de Australia se muestra en la Figura 6.6, donde hemos asignado variables en el orden *WA*, *NT*, *Q*,....

Vea que BACKTRACKING-SEARCH mantiene sólo una representación de un estado (asignación) y altera esa representación en lugar de crear otras nuevas (consulte la página 80).

Mientras que los algoritmos de búsqueda no informados del Capítulo 3 sólo podrían mejorarse proporcionándoles heurísticas *específicas del dominio*, resulta que la búsqueda de retroceso puede mejorarse usando heurísticas *independientes del dominio* que aprovechan la representación factorizada de los CSP. En las siguientes cuatro secciones mostramos cómo se hace esto:

- (6.3.1) ¿Qué variable debería asignarse a continuación (SELECT-UNASSIGNED-VARIABLE), y en qué orden deberían probarse sus valores (ORDER-DOMAIN-VALUES)?
- (6.3.2) ¿Qué inferencias deberían realizarse en cada paso de la búsqueda (INFERENCE)?
- (6.3.3) ¿Podemos retroceder más de un paso cuando sea apropiado?
- (6.3.4) ¿Podemos guardar y reutilizar resultados parciales de la búsqueda?

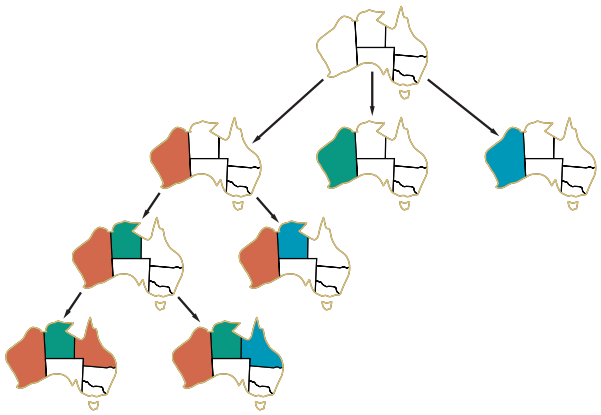


Figura 6.6 Detalle del árbol de búsqueda para el problema de colorear el mapa en la Figura 6.1.

6.3.1 Ordenamiento de variables y valores

El algoritmo de retroceso contiene la línea

var ←SELECT-UNASSIGNED-VARIABLE(*csp*,*assignment*).

La estrategia más simple para SELECT-UNASSIGNED-VARIABLE es el ordenamiento estático: elige las variables en orden, $\{X_1,X_2,...\}$. La siguiente más simple es elegir al azar. Ninguna estrategia es óptima. Por ejemplo, después de las asignaciones para *WA* = *rojo* y *NT* = *verde* en la Figura 6.6, sólo hay un valor posible para *SA*, por lo que tiene sentido asignar a continuación *SA* = *azul*, en lugar de asignar *Q*. De hecho, después de que se asigna *SA*, las elecciones para *Q*, *NSW* y *V* están todas forzadas.

Esta idea intuitiva, elegir la variable con los menos valores "legales", se llama heurística de **valores mínimos restantes** (MRV). También se ha llamado la heurística "más restringida variable" o heurística "falla primero", esta última porque elige una variable que es más probable que cause una falla pronto, podando así el árbol de búsqueda. Si alguna variable *X* no tiene valores legales restantes, la heurística MRV seleccionará *X* y se detectará la falla de inmediato, evitando búsquedas inútiles a través de otras variables. La heurística MRV generalmente funciona mejor que un ordenamiento aleatorio o estático, a veces por órdenes de magnitud, aunque los resultados varían dependiendo del problema.

Valores mínimos restantes

La heurística MRV no ayuda en absoluto a elegir la primera región para colorear en Australia, porque inicialmente cada región tiene tres colores legales. En este caso, la **heurística de grado** es la adecuada. Intenta reducir el factor de ramificación en futuras elecciones seleccionando la variable que está involucrada en el mayor número de restricciones en otras variables no asignadas. En la Figura 6.1, *SA* es la variable con el grado más alto, 5; las otras variables tienen grado 2 o 3, excepto *T*, que tiene grado 0. De hecho, una vez que se elige *SA*, aplicar la heurística de grado resuelve el problema sin ningún paso en falso: puede elegir *cualquier* color consistente en cada punto de elección y llegar a una solución sin necesitar algún retroceso. La heurística de valores mínimos restantes suele ser una guía más poderosa, pero la heurística de grado puede ser útil como desempate.

Heurística de grado

Una vez que se ha seleccionado una variable, el algoritmo debe decidir el orden en el que examinar sus valores. La heurística del **valor menos restringido** es efectiva para esto. Prefiere el valor que descarta el menor número de opciones para las variables vecinas en el grafo de

Valor menos restringido

restricción. Por ejemplo, suponga que en la Figura 6.1 hemos generado la asignación parcial con *WA* = *rojo* y *NT* = *verde* y que nuestra próxima elección es para *Q*. Azul sería una mala elección porque elimina el último valor legal restante para el vecino de *Q*, *SA*. Por lo tanto, la heurística del valor menos restringido prefiere rojo a azul. En general, la heurística intenta dejar la máxima flexibilidad para las asignaciones de variables posteriores.

¿Por qué la selección de variables debería ser fallo al comienzo, pero la selección de valores debería ser fallo al final? Eventualmente, cada variable tiene que ser asignada, por lo que al elegir la que es probable que falle primero, tendremos en promedio menos asignaciones exitosas para retroceder. Para el ordenamiento de valores, el truco es que sólo necesitamos una solución; por lo tanto, tiene sentido buscar los valores más probables primero. Si quisiéramos enumerar todas las soluciones en lugar de sólo encontrar una, entonces el ordenamiento de valores sería irrelevante.

6.3.2 Entrelazando búsqueda e inferencia

Vimos cómo AC-3 puede reducir los dominios de las variables *antes de* comenzar la búsqueda. Pero la inferencia puede ser aún más poderosa *durante* el curso de una búsqueda: cada vez que hacemos una elección de un valor para una variable, tenemos una nueva oportunidad para inferir nuevas reducciones de dominio en las variables vecinas.

Comprobación anticipada

Una de las formas más simples de inferencia se llama **comprobación anticipada**. Siempre que se asigna una variable *X*, el proceso de comprobación anticipada establece la consistencia de arco para ella: para cada variable *Y* no asignada conectada a *X* por una restricción, elimina del dominio de *Y* cualquier valor inconsistente con el valor elegido para *X*.

La Figura 6.7 muestra el progreso de la búsqueda de retroceso en el CSP de Australia con comprobación anticipada. Hay dos puntos importantes a tener en cuenta sobre este ejemplo. Primero, vea que después de que se asignan *WA* = *rojo* y *Q* = *verde*, los dominios de *NT* y *SA* se reducen a un solo valor; hemos eliminado la ramificación en estas variables por completo propagando información desde *WA* y *Q*. Un segundo punto a notar es que después de *V* = *azul*, el dominio de *SA* está vacío. Por lo tanto, la comprobación anticipada ha detectado que la asignación parcial {*WA* = *rojo*, *Q* = *verde*, *V* = *azul*} es inconsistente con las restricciones del problema, y el algoritmo retrocede inmediatamente.

Para muchos problemas, la búsqueda será más efectiva si combinamos la heurística MRV con la comprobación anticipada. Considere la Figura 6.7 después de asignar {*WA* = *rojo*}. Intuitivamente, parece que esa asignación restringe a sus vecinos, *NT* y *SA*, por lo que deberíamos manejar esas variables a continuación, y luego todas las demás variables encajarán en su lugar. Eso es exactamente lo que sucede con MRV: *NT* y *SA* cada uno tienen dos valores, por lo que se elige primero uno de ellos, luego el otro, luego *Q*, *NSW* y *V* en orden. Finalmente, *T* todavía tiene tres valores, y cualquiera de ellos funciona. Podemos ver la comprobación anticipada como una forma eficiente de calcular de manera incremental la información que la heurística MRV necesita para hacer su trabajo.

Aunque la comprobación anticipada detecta muchas inconsistencias, no detecta todas. El problema es que no mira lo suficientemente adelante. Por ejemplo, considere la fila *Q* = *verde* de la Figura 6.7. Hemos hecho que *WA* y *Q* sean arco-consistentes, pero hemos dejado tanto *NT* como *SA* con azul como su único valor posible, lo cual es una inconsistencia, ya que son vecinos.

Mantener la consistencia de arco

El algoritmo llamado MAC (para **Mantener la consistencia de arco**) detecta inconsistencias como esta. Después de que una variable X_i se le asigna un valor, el procedimiento INFERENCE llama a AC-3, pero en lugar de una cola de todos los arcos en el CSP, comenzamos sólo con los arcos (X_j, X_i) para todos los X_j variables no asignadas vecinos de X_i. A partir de ahí, AC-3 realiza la propagación de restricciones de la manera habitual, y si alguna variable tiene su dominio reducido al conjunto vacío, la llamada a AC-3 falla y sabemos que debemos retroceder inmediatamente.

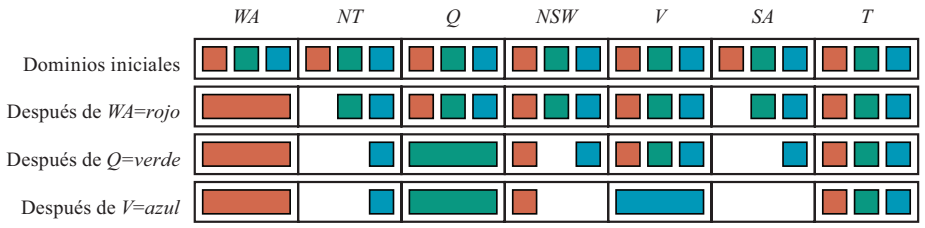


Figura 6.7 Progreso de una búsqueda de coloreado de mapas con comprobación anticipada. *WA = rojo* se asigna primero; luego la comprobación anticipada elimina el *rojo* de los dominios de las variables vecinas *NT* y *SA*. Después de que se asigna *Q = verde*, se elimina el verde de los dominios de *NT*, *SA* y *NSW*. Después de que se asigna *V = azul*, se elimina el azul de los dominios de *NSW* y *SA*, y se deja *SA* sin valores legales.

Podemos ver que MAC es estrictamente más poderoso que la comprobación anticipada, porque esta hace lo mismo que MAC en los arcos iniciales en la cola de MAC; pero a diferencia de MAC, la comprobación anticipada no propaga recursivamente las restricciones cuando se realizan cambios en los dominios de las variables.

6.3.3 Retroceso inteligente: mirando hacia atrás

El algoritmo de BÚSQUEDA DE RETROCESO en la Figura 6.5 tiene una política muy simple de qué hacer cuando una rama de la búsqueda falla: retroceder a la variable anterior e intentar un valor diferente. Esto se llama **retroceso cronológico** porque el punto de decisión *más reciente* es revisado. En esta subsección, consideramos mejores posibilidades.

Retroceso cronológico

Considere lo que sucede cuando aplicamos un retroceso simple en la Figura 6.1 con un orden de variables fijo *Q*, *NSW*, *V*, *T*, *SA*, *WA*, *NT*. También suponga que hemos generado la asignación parcial $\{Q = rojo, NSW = verde, V = azul, T = rojo\}$. Cuando intentamos la siguiente variable, *SA*, vemos que cada valor viola una restricción. Retrocedemos a *T* e intentamos un nuevo color para Tasmania! Obviamente esto es absurdo: recolorear Tasmania no puede ayudar a resolver el problema con Australia del Sur.

Un enfoque más inteligente es retroceder a una variable que podría solucionar el problema, una variable que fue responsable de hacer que uno de los posibles valores de *SA* fuera imposible. Para hacer esto, mantendremos un registro de un conjunto de asignaciones en conflicto con algún valor para *SA*. El conjunto (en este caso $\{Q = rojo, NSW = verde, V = azul\}$), se llama el **conjunto de conflictos** para *SA*. El método de **retroceso o salto atrás** retrocede a la asignación *más reciente* en el conjunto de conflictos; en este caso, el salto atrás saltaría sobre Tasmania e intentaría un nuevo valor para *V*. Este método se implementa fácilmente con una modificación a BACKTRACK, de modo que acumule el conjunto de conflictos mientras verifica un valor legal para asignar. Si no se encuentra un valor legal, el algoritmo debería devolver el elemento más reciente del conjunto de conflictos junto con el indicador de falla.

Conjunto de conflictos

Retroceso o salto atrás

El lector perspicaz habrá notado que la comprobación anticipada puede proporcionar el conjunto de conflictos sin trabajo adicional: siempre que la comprobación anticipada basada en una asignación $X = x$ elimina un valor del dominio de *Y*, debería agregar $X = x$ al conjunto de conflictos de *Y*. Si se elimina el último valor del dominio de *Y*, entonces las asignaciones en el conjunto de conflictos de *Y* se agregan al conjunto de conflictos de *X*. Es decir, ahora sabemos que $X = x$ conduce a una contradicción (en *Y*) y, por lo tanto, se debería intentar una asignación diferente para *X*.

El lector de ojos de águila habrá notado algo extraño: el retroceso ocurre cuando cada valor en un dominio está en conflicto con la asignación actual; pero la comprobación anticipada detecta este evento y evita que la búsqueda llegue a tal nodo! De hecho, se puede demostrar que *cada* rama podada por retroceso también es podada por comprobación anticipada. Por lo tanto, el retroceso simple es redundante en una búsqueda de comprobación anticipada o, de hecho, en una búsqueda que usa una comprobación de consistencia más fuerte, como MAC, sólo necesitas hacer uno u otro.

A pesar de las observaciones del párrafo anterior, la idea detrás del retroceso sigue siendo buena: retroceder en función de las razones del fracaso. El retroceso nota el fracaso cuando el dominio de una variable se vuelve vacío, pero en muchos casos una rama está condenada mucho antes de que esto ocurra. Considere de nuevo la asignación parcial $\{WA = rojo, NSW = rojo\}$ (que, según nuestra discusión anterior, es inconsistente). Suponga que intentamos $T = rojo$ a continuación y luego asignamos *NT*, *Q*, *V*, *SA*. Sabemos que ninguna asignación puede funcionar para estas últimas cuatro variables, por lo que eventualmente nos quedamos sin valores para probar en *NT*. Ahora, la pregunta es, ¿a dónde retroceder? El retroceso no puede funcionar, porque *NT sí* tiene valores consistentes con las variables asignadas anteriores, *NT* no tiene un conjunto completo de conflictos de variables anteriores que lo hicieron fallar. Sin embargo, sabemos que las cuatro variables *NT*, *Q*, *V* y *SA*, *tomadas juntas*, fallaron debido a un conjunto de variables anteriores, que deben ser aquellas variables que entran directamente en conflicto con las cuatro.

Esto lleva a una noción diferente y más profunda del conjunto de conflictos para una variable como *NT*: es ese conjunto de variables anteriores que causó que *NT*, junto con cualquier variable posterior, no tenga una solución consistente. En este caso, el conjunto es *WA* y *NSW*, por lo que el algoritmo debería retroceder a *NSW* y saltarse Tasmania. Un algoritmo de retroceso que usa conjuntos de conflictos definidos de esta manera se llama **retroceso dirigido por conflictos**.

Retroceso dirigido por conflictos

Ahora debemos explicar cómo se calculan estos nuevos conjuntos de conflictos. El método es de hecho muy simple. El "fracaso terminal" de una rama de la búsqueda siempre ocurre porque el dominio de una variable se vuelve vacío; esa variable tiene un conjunto de conflictos estándar. En nuestro ejemplo, *SA* falla, y su conjunto de conflictos es (digamos) $\{WA, NT, Q\}$. Retrocedemos a *Q*, y *Q absorbe* el conjunto de conflictos de *SA* (menos *Q* mismo, por supuesto) en su propio conjunto de conflictos directos, que es $\{NT, NSW\}$; el nuevo conjunto de conflictos es $\{WA, NT, NSW\}$. Es decir, no hay solución desde *Q* en adelante, dada la asignación anterior a $\{WA, NT, NSW\}$. Por lo tanto, retrocedemos a *NT*, el más reciente de estos. *NT* absorbe $\{WA, NT, NSW\} - \{NT\}$ en su propio conjunto de conflictos directos $\{WA\}$, dando $\{WA, NS\ W\}$ (como se indicó en el párrafo anterior). Ahora el algoritmo retrocede a *NSW*, como esperábamos. Para resumir: dejemos que X_j sea la variable actual, y dejemos que $conf(X_j)$ sea su conjunto de conflictos. Si cada valor posible para X_j falla, retroceda a la variable X_i más reciente en $conf(X_j)$ y vuelva a calcular el conjunto de conflictos para X_i de la siguiente manera:

$$conf(X_i) \rightarrow conf(X_i) \cup conf(X_j) - \{X_i\}.$$

6.3.4 Aprendizaje de restricciones

Cuando llegamos a una contradicción, el retroceso puede decirnos hasta dónde retroceder, por lo que no perdemos tiempo cambiando variables que no solucionarán el problema. También nos gustaría evitar encontrarnos con el mismo problema nuevamente. Cuando la búsqueda llega a una contradicción, sabemos que algún subconjunto del conjunto de conflictos es responsable del problema. El **aprendizaje de restricciones** es la idea de encontrar un conjunto mínimo de variables del conjunto de conflictos que causa el problema. Este conjunto de variables, junto con sus valores correspondientes, se llama **no-bueno**. Luego registramos el no-bueno, ya sea agregando una nueva restricción al CSP para prohibir esta combinación de asignaciones o manteniendo una caché separada de no-buenos.

Aprendizaje de restricciones

No-bueno

Por ejemplo, considere el estado $\{WA = rojo, NT = verde, Q = azul\}$ en la fila inferior de la Figura 6.6. La comprobación anticipada puede decirnos que este estado es un no-bueno porque no hay una asignación válida para *SA*. En este caso particular, registrar el no-bueno no ayudaría, porque una vez que podamos esta rama del árbol de búsqueda, nunca encontraremos esta combinación nuevamente. Pero suponga que el árbol de búsqueda en la Figura 6.6 fuera en realidad parte de un árbol de búsqueda más grande que comenzó asignando primero valores para *V* y *T*. Entonces sería útil registrar $\{WA = rojo, NT = verde, Q = azul\}$ como un no-bueno porque vamos a encontrarnos con el mismo problema de nuevo para cada posible conjunto de asignaciones a *V* y *T*.

Los no-buenos pueden ser utilizados efectivamente por la comprobación hacia adelante o por el retroceso. El aprendizaje de restricciones es una de las técnicas más importantes utilizadas por los solucionadores modernos de CSP para lograr eficiencia en problemas complejos.

6.4 Búsqueda local para CSPs

Los algoritmos de búsqueda local (ver Sección 4.1) resultan muy efectivos en la resolución de muchos CSPs. Usan una formulación de estado completo (como se introdujo en la Sección 4.1.1) donde cada estado asigna un valor a cada variable, y la búsqueda cambia el valor de una variable a la vez. Como ejemplo, usaremos el problema de las 8 reinas, definido como un CSP en la página 183. En la Figura 6.8 comenzamos a la izquierda con una asignación completa a las 8 variables; por lo regular esto violará varias restricciones. Luego elegimos aleatoriamente una variable en conflicto, que resulta ser Q_8, la columna más a la derecha. Nos gustaría cambiar el valor a algo que nos acerque más a una solución; el enfoque más obvio es seleccionar el valor que resulta en el número mínimo de conflictos con otras variables: la heurística de **mínimos conflictos**.

Mínimos conflictos

En la figura vemos que hay dos filas que sólo violan una restricción; elegimos $Q_8 = 3$ (es decir, movemos la reina a la 8ª columna, 3ª fila). En la siguiente iteración, en el tablero del medio de la figura, seleccionamos Q_6 como la variable a cambiar, y notamos que mover la reina a la 8ª fila no resulta en conflictos. En este punto no hay más variables en conflicto, así que tenemos una solución. El algoritmo se muestra en la Figura 6.9.[2]

Min-conflictos es sorprendentemente efectivo para muchos CSPs. De manera asombrosa, en el problema de las *n*-reinas, si no cuenta la colocación inicial de las reinas, el tiempo de ejecución de los mínimos conflictos es casi *independiente del tamaño del problema*. Resuelve incluso el problema de las *millón*-reinas en un promedio de 50 pasos (después de la asignación inicial). Esta notable observación fue el estímulo que llevó a una gran cantidad de investigación en la década de 1990 sobre la búsqueda local y la distinción entre problemas fáciles y difíciles, que abordamos en la Sección 7.6.3. Hablando en términos generales, *n*-reinas es fácil para la búsqueda local porque las soluciones están densamente distribuidas en todo el espacio de estados. Min-conflictos también funciona para problemas difíciles. Por ejemplo, se ha usado para programar observaciones para el Telescopio Espacial Hubble, reduciendo el tiempo necesario para programar una semana de observaciones de tres semanas (!) a alrededor de 10 minutos.

Todas las técnicas de búsqueda local de la Sección 4.1 son candidatas para su aplicación a CSPs, y algunas de ellas han demostrado ser en especial efectivas. El paisaje de un CSP bajo la heurística de mínimos conflictos suele tener una serie de mesetas. Puede haber millones de asignaciones de variables que están a sólo un conflicto de una solución. La búsqueda de mesetas, que permite movimientos laterales a otro estado con la misma puntuación, puede ayudar a la búsqueda local a encontrar

[2] La búsqueda local puede ser fácilmente extendida a problemas de optimización con restricciones (COPs). En ese caso, todas las técnicas para ascenso de colinas y el recocido simulado pueden ser aplicadas para optimizar la función objetivo.

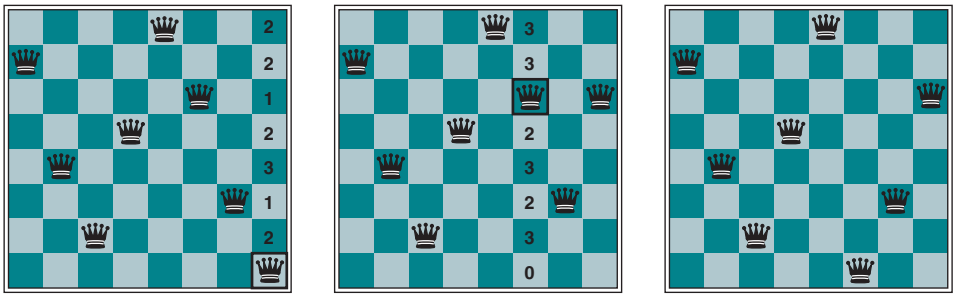


Figura 6.8 Una solución en dos pasos usando mínimos conflictos para un problema de 8 reinas. En cada etapa, se elige una reina para su reasignación en su columna. El número de conflictos (en este caso, el número de reinas atacantes) se muestra en cada cuadrado. El algoritmo mueve la reina al cuadrado de mínimos conflictos, rompiendo los empates al azar.

```
function MIN-CONFLICTS(csp,max_steps) returns una solución o fallo
    inputs: csp, a problemas de satisfacción de restricciones
            max_steps, el número de pasos permitidos antes de darse por vencido

    current← una asignación completa inicial para csp
    for i = 1 to max_steps do
        if current es una solución para csp then return current
        var← una variable conflictiva elegida al azar de csp.VARIABLES
        value← el valor v para var que minimiza CONFLICTS(csp, var, v, current)
        agregar var=value en current
    return fallo
```

Figura 6.9 El algoritmo de búsqueda local MIN-CONFLICTS para CSPs. El estado inicial puede ser elegido al azar o por un proceso de asignación voraz que elige un valor de conflicto mínimo para cada variable a su vez. La función CONFLICTS cuenta el número de restricciones violadas por un valor particular, dado el resto de la asignación actual.

su camino fuera de esta meseta. Este vagabundeo en la meseta puede ser dirigido con una técnica llamada *búsqueda tabú*: mantener una pequeña lista de estados recién visitados y prohibir al algoritmo volver a ellos. El recocido simulado también puede ser utilizado para escapar de las mesetas.

Ponderación de restricciones

Otra técnica llamada **ponderación de restricciones** tiene como objetivo concentrar la búsqueda en las restricciones importantes. A cada restricción se le asigna un peso numérico, inicialmente todos 1. En cada paso de la búsqueda, el algoritmo elige un par variable/valor para cambiar que resultará en el peso total más bajo de todas las restricciones violadas. Los pesos se ajustan luego incrementando el peso de cada restricción que es violada por la asignación actual. Esto tiene dos beneficios: añade topografía a las mesetas, asegurándose de que es posible mejorar desde el estado actual, y también añade aprendizaje: con el tiempo, las restricciones difíciles se asignan a pesos más altos.

Otra ventaja de la búsqueda local es que puede ser utilizada en un entorno en línea (ver Sección 4.5) cuando el problema cambia. Considere un problema de programación para los vuelos semanales de una aerolínea. El horario puede involucrar miles de vuelos y decenas de miles

de asignaciones de personal, pero el mal tiempo en un aeropuerto puede hacer que el horario sea inviable. Nos gustaría reparar el horario con un mínimo número de cambios. Esto puede hacerse fácilmente con un algoritmo de búsqueda local partiendo del horario actual. Una búsqueda de retroceso con el nuevo conjunto de restricciones por lo regular requiere mucho más tiempo y podría encontrar una solución con muchos cambios respecto al horario actual.

6.5 La estructura de los problemas

En esta sección examinamos formas en las que la *estructura* del problema, tal como se representa en el grafo de restricciones, puede ser usada para encontrar soluciones rápidamente. La mayoría de los enfoques también se aplican a otros problemas además de los CSPs, como el razonamiento probabilístico.

La única forma en que podemos esperar lidiar con el vasto mundo real es descomponiéndolo en subproblemas. Mirando de nuevo el grafo de restricciones para Australia (Figura 6.1(b), repetida como Figura 6.12(a)), un hecho destaca: Tasmania no está conectada al continente.[3] Intuitivamente, es obvio que colorear Tasmania y el continente son **subproblemas independientes**, cualquier solución para el continente combinada con cualquier solución para Tasmania da una solución para todo el mapa.

Subproblemas independientes

La independencia puede ser determinada sólo encontrando **componentes conectados** del grafo de restricciones. Cada componente corresponde a un subproblema CSP_i. Si la asignación S_i es una solución de CSP_i, entonces $\cup_i S_i$ es una solución de $\cup_i CSP_i$. ¿Por qué es esto importante? Suponga que cada CSP_i tiene c variables del total de n variables, donde c es una constante. Entonces hay n/c subproblemas, cada uno de los cuales toma a lo sumo d^c trabajo para ser resuelto, donde d es el tamaño del dominio. Por lo tanto, el trabajo total es $O(d^c n/c)$, que es *lineal* en n; sin la descomposición, el trabajo total es $O(d^n)$, exponencial en n. Hagamos esto más concreto: dividir un CSP booleano con 100 variables en cuatro subproblemas reduce el tiempo de solución, en el peor de los casos, desde la vida del universo hasta menos de un segundo.

Componentes conectados

Los subproblemas completamente independientes son deliciosos, entonces, pero raros. Afortunadamente, algunas otras estructuras de grafos también son fáciles de resolver. Por ejemplo, un grafo de restricciones es un **árbol** cuando cualquier dos variables están conectadas por sólo un camino. Mostraremos que *cualquier CSP estructurado en árbol puede ser resuelto en tiempo lineal en el número de variables*.[4] La clave es una nueva noción de consistencia, llamada **consistencia de arco direccional** o DAC. Un CSP se define como direccionalmente arco-consistente bajo un ordenamiento de variables $X_1, X_2, \ldots, X_n$ si y sólo si cada X_i es arco-consistente con cada X_j para $j > i$.

Consistencia de arco direccional

Para resolver un CSP estructurado en árbol, primero elija cualquier variable para ser la raíz del árbol, y elija un ordenamiento de las variables de modo que cada variable aparezca después de su padre en el árbol. Tal ordenamiento se llama **ordenación topológica**. La Figura 6.10(a) despliega un árbol de muestra y (b) muestra un posible ordenamiento. Cualquier árbol con n nodos tiene $n - 1$ aristas, por lo que podemos hacer este grafo direccionalmente arco-consistente en $O(n)$ pasos, cada uno de los cuales debe comparar hasta d posibles valores de dominio para dos variables, para un tiempo total de $O(nd^2)$. Una vez que tenemos un grafo direccionalmente arco-consistente, podemos sólo marchar por la lista de variables y elegir cualquier valor restante. Ya que cada arista de un padre a su hijo es arco-consistente, sabemos que para cualquier valor que elijamos para el padre, habrá un valor válido restante que elegir para el hijo. Eso significa que no tenemos que dar

Ordenación topológica

[3] Un cartógrafo cuidadoso o un tasmaniano patriótico podría objetar que Tasmania no debería ser coloreada igual que su vecino continental más cercano, para evitar la impresión de que podría ser parte de ese estado.

[4] Pero, muy pocas regiones del mundo tienen mapas estructurados en árbol, aunque Sulawesi se acerca.

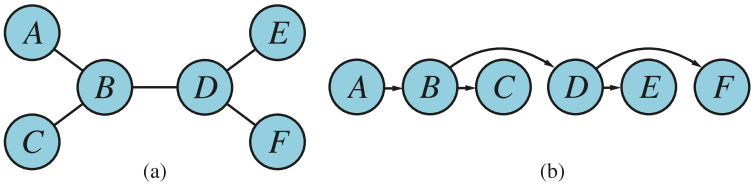


Figura 6.10 (a) Grafo de restricciones de un CSP estructurado en árbol. (b) Ordenamiento lineal de las variables consistente con el árbol con *A* como raíz. Esto se conoce como un **ordenamiento topológico** de las variables.

function TREE-CSP-SOLVER(*csp*) **returns** una solución, o *fallo*
  inputs: *csp*, un CSP con componentes *X*, *D*, *C*

  n← n**úmero de variables en** *X*
  assignment← una asignación vacía
  root← cualquier variable en *X*
  X←TOPOLOGICALSORT(*X*, *root*)
  for *j* = *n* **down to** 2 **do**
    MAKE-ARC-CONSISTENT(PARENT(X_j),X_j)
    if no puede hacerse consistente **then return** *fallo*
  for *i* = 1 **to** *n* **do**
    assignment[X_i]← cualquier valor consistente de D_i
    if no hay valor consistente **then return** *fallo*
  return *assignment*

Figura 6.11 El algoritmo TREE-CSP-SOLVER para resolver CSP estructurados en árbol. Si el CSP tiene una solución, la encontraremos en tiempo lineal; si no, detectaremos una contradicción.

marcha atrás; podemos movernos linealmente a través de las variables. El algoritmo completo se muestra en la Figura 6.11.

Ahora que tenemos un algoritmo eficiente para los árboles, podemos considerar si los grafos de restricciones más generales se pueden *reducir* a árboles de alguna manera. Hay dos formas de hacerlo: eliminando nodos (Sección 6.5.1) o colapsando los nodos juntos Sección 6.5.2).

6.5.1 Acondicionamiento del conjunto de cortes

La primera forma de reducir un grafo de restricciones a un árbol implica asignar valores a algunas variables para que las variables restantes formen un árbol. Considere el grafo de restricciones para Australia, que se muestra de nuevo en la figura 6.12 (a). Sin Australia del Sur, el grafo se convertiría en un árbol, como en (b). Por fortuna, podemos eliminar Australia del Sur (en el grafo, no en el país) fijando un valor para *SA* y eliminar de los dominios de las otras variables cualquier valor que sea incoherente con el valor elegido para *SA*.

Ahora, cualquier solución para el CSP después de que se eliminen *SA* y sus restricciones será coherente con el valor elegido para *SA*. (Esto funciona para CSP binarios; la situación resulta más complicada con restricciones de orden superior). Por lo tanto, podemos resolver el árbol restante con el algoritmo antes dado y así resolver todo el problema. Claro, en el caso

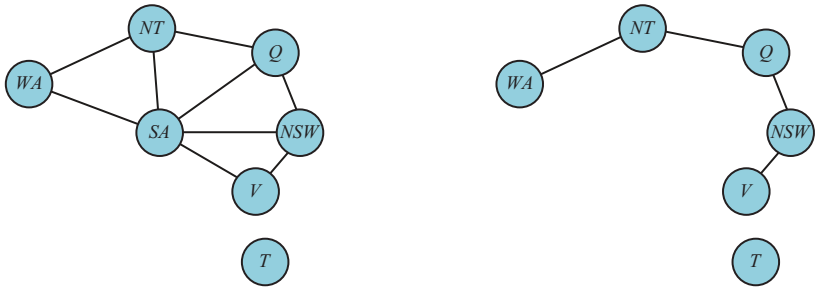


Figura 6.12 (a) Grafo de restricción original de la Figura 6.1. (b) Después de la eliminación de *SA*, el grafo de restricción se convierte en un bosque de dos árboles.

general (a diferencia del color del mapa), el valor elegido para *SA* podría ser el incorrecto, por lo que tendríamos que probar cada valor posible. El algoritmo general es el siguiente:

1. Elija un subconjunto *S* de las variables del CSP de modo que el grafo de restricciones se convierta en un árbol después de eliminar *S*. A *S* se le denomina **conjunto de cortes de ciclo**.
2. Para cada posible asignación a las variables de *S* que satisfaga todas las restricciones de *S*,
   (a) eliminar de los dominios de las variables restantes cualquier valor que sea inconsistente con la asignación de *S*, y
   (b) si el CSP restante dispone de una solución, devolverla junto con la asignación de *S*.

Conjunto de cortes de ciclo

Si el conjunto de cortes de ciclo tiene tamaño c, el tiempo total de ejecución es $O(d^c \cdot (n-c)d^2)$: tenemos que probar cada una de las combinaciones de valores d^c para las variables en *S*, y para cada combinación debemos resolver un problema de árbol de tamaño $n-c$. Si el grafo es "casi un árbol", entonces c será pequeña y el ahorro sobre el retroceso directo será enorme; para nuestro ejemplo de 100 variables booleanas, si pudiéramos encontrar un conjunto de cortes de tamaño $c = 20$, esto nos llevaría de la vida del Universo a unos minutos. En el peor de los casos, sin embargo, c puede ser tan grande como $(n-2)$. Encontrar el conjunto de cortes de ciclo *más pequeño* es NP-difícil, pero se conocen varios algoritmos de aproximación eficientes. El enfoque algorítmico general se denomina **condicionamiento de conjuntos de cortes**; este volverá a aparecer en el capítulo 13, donde se usa para razonar sobre probabilidades.

Condicionamiento de conjuntos de cortes

6.5.2 Descomposición de árboles

La segunda forma de reducir un grafo de restricciones a un árbol se basa en la construcción de **una descomposición de árbol** del grafo de restricciones: una transformación del grafo original en un árbol donde cada nodo en el árbol consta de un conjunto de variables, como en la Figura 6.13. La descomposición de un árbol debe cumplir estos tres requisitos:

Descomposición de árbol

- Cada variable en el problema original aparece al menos en uno de los nodos del árbol.
- Si dos variables están conectadas por una restricción en el problema original, deben aparecer juntas (junto con la restricción) en al menos uno de los nodos del árbol.
- Si una variable aparece en dos nodos del árbol, debe aparecer en cada nodo a lo largo del camino que conecta esos nodos.

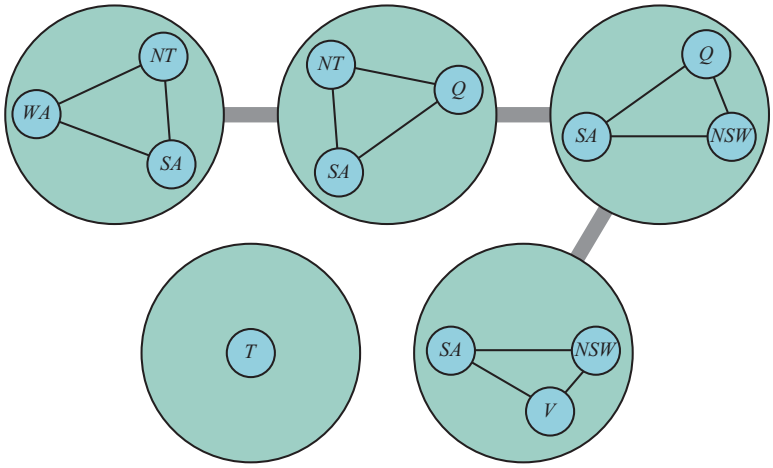


Figura 6.13 Una descomposición de árbol del grafo de restricciones en la Figura 6.12(a).

Las dos primeras condiciones aseguran que todas las variables y restricciones están representadas en la descomposición del árbol. La tercera condición parece bastante técnica, pero nos permite decir que cualquier variable del problema original debe tener el mismo valor dondequiera que aparezca: las restricciones en el árbol dicen que una variable en un nodo del árbol debe tener el mismo valor que la variable correspondiente en el nodo adyacente en el árbol. Por ejemplo, *SA* aparece en los cuatro nodos conectados en la Figura 6.13, por lo que cada borde en la descomposición del árbol incluye la restricción de que el valor de *SA* en un nodo debe ser el mismo que el valor de *SA* en el siguiente. Puede verificar a partir de la Figura 6.12 que esta descomposición tiene sentido.

Una vez que tenemos un grafo estructurado en árbol, podemos aplicar TREE-CSP-SOLVER para obtener una solución en tiempo $O(nd^2)$, donde n es el número de nodos del árbol y d es el tamaño del dominio más grande. Tenga en cuenta que en el árbol, un dominio es un conjunto de *tuplas* de valores, no sólo valores individuales.

Por ejemplo, el nodo superior izquierdo en la Figura 6.13 representa, a nivel del problema original, un subproblema con variables {*WA*,*NT*,*SA*}, dominio {*rojo*,*verde*,*azul*}, así como restricciones $WA \neq NT$,$SA \neq NT$,$WA \neq SA$. A nivel del árbol, el nodo representa una sola variable, a la que podemos llamar *SANTWA*, cuyo valor debe ser una terna de colores, como (*rojo*, *verde*, *azul*), pero no (*rojo*,*rojo*,*azul*), porque eso violaría la restricción $SA \neq NT$ del problema original. Luego podemos pasar de ese nodo al adyacente, con la variable que podemos llamar *SANTQ*, y encontrar que sólo hay una tupla, (*rojo*,*verde*,*azul*), consistente con la elección para *SANTWA*. El mismo proceso exacto se repite para los dos siguientes nodos, e independientemente podemos hacer cualquier elección para *T*.

Podemos resolver cualquier problema de descomposición de árbol en tiempo $O(nd^2)$ con TREE-CSP-SOLVER, que será eficiente siempre que d permanezca pequeño. Volviendo a nuestro ejemplo con 100 variables booleanas, si cada nodo tiene 10 variables, entonces $d = 2^{10}$ y deberíamos poder resolver el problema en segundos. Pero si hay un nodo con 30 variables, tomaría siglos.

Un grafo dado admite muchas descomposiciones de árbol; al elegir una descomposición, el objetivo es hacer los subproblemas lo más pequeños posible. (Poner todas las variables en un nodo es técnicamente un árbol, pero no es útil). El **ancho del árbol** de una descomposición de árbol

Ancho del árbol

de un grafo es uno menos que el tamaño del nodo más grande; el ancho del árbol del grafo en sí se define como el ancho mínimo entre todas sus descomposiciones de árbol. Si un grafo tiene un ancho de árbol w entonces el problema se puede resolver en tiempo $O(nd^{w+1})$ dada la descomposición de árbol correspondiente. Por lo tanto, los *CSP con grafos de restricción de ancho de árbol limitado se pueden resolver en tiempo polinomial*.

Desafortunadamente, encontrar la descomposición con ancho de árbol mínimo es NP-duro, pero hay métodos heurísticos que funcionan bien en la práctica. ¿Cuál es mejor: la descomposición de corte con tiempo $O(d^c \cdot (n - c)d^2)$, o la descomposición de árbol con tiempo $O(nd^{w+1})$? Donde quiera que tenga un corte de ciclo de tamaño c, también hay un ancho de árbol de tamaño $w < c + 1$, y puede ser mucho más pequeño en algunos casos. Por lo tanto, la consideración del tiempo favorece la descomposición del árbol, pero la ventaja del enfoque de corte de ciclo es que se puede ejecutar en memoria lineal, mientras que la descomposición de árbol requiere memoria exponencial en w.

6.5.3 Simetría de valor

Hasta ahora hemos mirado la estructura del grafo de restricción. También puede haber una estructura importante en los *valores* de las variables, o en la estructura de las relaciones de restricción en sí. Considere el problema de colorear el mapa con d colores. Para cada solución consistente, en realidad hay un conjunto de $d!$ soluciones formadas por permutar los nombres de los colores. Por ejemplo, en el mapa de Australia sabemos que *WA*, *NT* y *SA* deben tener todos colores diferentes, pero hay $3! = 6$ formas de asignar tres colores a tres regiones. Esto se llama **simetría de valor**. Nos gustaría reducir el espacio de búsqueda por un factor de $d!$ rompiendo la simetría en las asignaciones. Hacemos esto introduciendo una **restricción de ruptura de simetría**. Para nuestro ejemplo, podríamos imponer una restricción de orden arbitrario, $NT < SA < WA$, que requiere que los tres valores estén en orden alfabético. Esta restricción asegura que sólo una de las $d!$ soluciones es posible: $\{NT = azul, SA = verde, WA = rojo\}$.

Simetría de valor

Restricción de ruptura de simetría

Para coloreado de mapas, fue fácil encontrar una restricción que elimina la simetría. En general, eliminar toda simetría es NP-duro, pero romper la simetría de valor ha demostrado ser importante y efectivo en una amplia gama de problemas.

Resumen

- Los **problemas de satisfacción de restricciones** (CSPs) representan un estado con un conjunto de pares de variables/valores y representan las condiciones para una solución mediante un conjunto de restricciones en las variables. Muchos problemas importantes del mundo real pueden describirse como CSPs.
- Una serie de técnicas de **inferencia** utilizan las restricciones para descartar ciertas asignaciones de variables. Estos incluyen las consistencias de nodo, arco, camino y *k-consistencia*.
- La **búsqueda de retroceso** es una forma de búsqueda en profundidad; se usa para resolver CSP. La inferencia se puede entrelazar con la búsqueda.
- Las heurísticas de **valores mínimos restantes** y **de grado** son métodos independientes del dominio para decidir qué variable elegir a continuación en una búsqueda de retroceso. La heurística **de valor menos restringido** ayuda a decidir qué valor probar primero para una variable dada. El retroceso ocurre cuando no se puede encontrar una asignación legal para una variable. El **retroceso dirigido por conflictos** retrocede directamente a la fuente del problema. El **aprendizaje de restricciones** registra los conflictos a medida que se encuentran durante la búsqueda para evitar el mismo conflicto más tarde en la búsqueda.

- La búsqueda local usando heurística de **mínimos conflictos** también se ha aplicado a problemas de satisfacción de restricciones con gran éxito.
- La complejidad de resolver un CSP está muy relacionada con la estructura de su grafo de restricciones. Los problemas estructurados en árbol pueden resolverse en tiempo lineal. La **condición de corte** puede reducir un CSP general a uno estructurado en árbol y es muy eficiente (requiere sólo memoria lineal) si se puede encontrar un pequeño conjunto de corte. Las técnicas de descomposición de árboles transforman el CSP en un árbol de subproblemas y son eficientes si el ancho del árbol del grafo de restricciones es pequeño; sin embargo, necesitan memoria exponencial en el ancho del árbol del grafo de restricciones. Combinar la condición de corte con la descomposición de árboles puede permitir un mejor equilibrio de memoria versus tiempo.

Notas bibliográficas e históricas

Ecuaciones diofánticas

El matemático griego Diophantus (*ca*. 200–284) presentó y resolvió problemas que involucran restricciones algebraicas en ecuaciones, aunque no desarrolló una metodología generalizada. Hoy llamamos a las ecuaciones sobre dominios enteros **ecuaciones diofánticas**. El matemático indio Brahmagupta (*ca*. 650) fue el primero en mostrar una solución general sobre el dominio de los enteros para la ecuación $ax + by = c$. Los métodos sistemáticos para resolver ecuaciones lineales por variables de eliminación fueron estudiados por Gauss (1829); la solución de restricciones de desigualdad lineal se remonta a Fourier (1827).

Los problemas de satisfacción de restricciones de dominio finito también tienen una larga historia. Por ejemplo, el coloreado de grafos (del cual el coloreado de mapas es un caso especial) es un problema antiguo en matemáticas. La conjetura de los cuatro colores (que todo grafo plano puede ser coloreado con cuatro o menos colores) fue propuesta en 1852 por Francis Guthrie, un estudiante de De Morgan. La conjetura se resistió a ser solucionada, a pesar de que en varias publicaciones se afirmaba lo contrario, hasta que Appel y Haken (1977) idearon una prueba (ver el libro *Four Colors Suffice* (Wilson, 2004)). Los puristas se decepcionaron de que parte de la prueba dependiera de una computadora, por lo que Georges Gonthier (2008), usando el probador de teoremas Coq, derivó una prueba formal de que la prueba mediante el programa de Appel y Haken era correcta.

Clases específicas de problemas de satisfacción de restricciones ocurren a lo largo de la historia de la ciencia de la computación. Uno de los ejemplos tempranos más influyentes fue Sketchpad (Sutherland, 1963), que resolvió restricciones geométricas en diagramas y fue el precursor de los programas de dibujo modernos y las herramientas de CAD. La identificación de CSPs como una clase general se debe a Ugo Montanari (1974). La reducción de CSPs de orden superior a CSPs binarios puros con variables auxiliares (ver Ejercicio 6.NARY) se debe originalmente al lógico del siglo xix Charles Sanders Peirce. Fue introducido en la literatura de CSP por Dechter (1990b) y fue elaborado por Bacchus y Van Beek (1998). Los CSPs con preferencias entre soluciones se estudian ampliamente en la literatura de optimización; para una generalización del marco de CSP para permitir preferencias vea Bistarelli *et al*. (1997).

Los métodos de propagación de restricciones fueron popularizados por el éxito de Waltz (1975) en problemas de etiquetado de líneas poliédricas para la visión por computadora. Waltz mostró que en muchos problemas, la propagación elimina completamente la necesidad de retroceso. Montanari (1974) introdujo la noción de grafos de restricciones y propagación por consistencia de camino. Alan Mackworth (1977) propuso el algoritmo AC-3 para hacer cumplir la consistencia de arco, así como la idea general de combinar retroceso con algún grado de cumplimiento de consistencia. AC-4, un algoritmo de consistencia de arco

más eficiente desarrollado por Mohr y Henderson (1986), se ejecuta en el peor de los casos en tiempo $O(cd^2)$, pero puede ser más lento que AC-3 en casos promedio. El algoritmo PC-2 (Mackworth, 1977) logra la consistencia de camino de la misma manera que AC-3 logra la consistencia de arco.

Poco después de aparecido el artículo de Mackworth, los investigadores comenzaron a experimentar con el equilibrio entre el costo de la aplicación de la consistencia y los beneficios en términos de reducción de la búsqueda. Haralick y Elliott (1980) favorecieron el algoritmo de comprobación mínima hacia adelante descrito por McGregor (1979), mientras que Gaschnig (1979) sugirió una comprobación completa de la consistencia de arco después de cada asignación de variables, algoritmo que más tarde Sabin y Freuder (1994) llamaron MAC. El último artículo proporciona evidencia bastante convincente de que en CSPs más difíciles, la comprobación completa de la consistencia de arco vale la pena. Freuder (1978, 1982) investigó la noción de *k*-consistencia y su relación con la complejidad de resolver CSPs. Dechter y Dechter (1987) introdujeron la consistencia de arco direccional. Apt (1999) describe un marco algorítmico genérico dentro del cual se pueden analizar los algoritmos de propagación de consistencia, y se dan revisiones por Bessière (2006) y Barták *et al.* (2010).

Programación lógica de restricciones

Los métodos especiales para manejar restricciones de orden superior o globales se desarrollaron primero dentro del contexto de la **programación lógica de restricciones**. Marriott y Stuckey (1998) brindan una excelente cobertura de la investigación en esta área. La restricción *Alldiff* fue estudiada por Regin (1994), Stergiou y Walsh (1999), y Van Hoeve (2001). Hay algoritmos de inferencia más complejos para *Alldiff* (ver Van Hoeve y Katriel, 2006) que propagan más restricciones pero son más costosos de ejecutar computacionalmente. Las restricciones de límites se incorporaron en la programación lógica de restricciones por Van Hentenryck *et al*. (1998). Además, una revisión de restricciones globales es proporcionada por Van Hoeve y Katriel (2006).

El Sudoku se ha convertido en el CSP más conocido y fue descrito como tal por Simonis (2005). Agerbeck y Hansen (2008) describen algunas de las estrategias y muestran que Sudoku en un tablero $n^2 \times n^2$ está en la clase de problemas NP-duros.

En 1850, C. F. Gauss describió un algoritmo de retroceso recursivo para resolver el problema de las 8 reinas, publicado en la revista de ajedrez alemana *Schachzeitung* en 1848. Gauss llamó a su método *Tatonniren*, derivado de la palabra francesa *tâtonner*: buscar a tientas, como si estuviera en la oscuridad.

Según Donald Knuth (comunicación personal), R. J. Walker introdujo el término *retroceso* en la década de 1950. Walker (1960) describió el algoritmo básico de retroceso y lo usó para encontrar todas las soluciones al problema de las 13 reinas. Golomb y Baumert (1965) formularon, con ejemplos, la clase general de problemas combinatorios a los que se puede aplicar el retroceso, e introdujeron lo que llamamos la heurística MRV. Bitner y Reingold (1975) proporcionaron una revisión influyente de técnicas de retroceso. Brelaz (1979) usó la heurística de grado como desempate después de aplicar la heurística MRV. El algoritmo resultante, a pesar de su simplicidad, sigue siendo el mejor método para *k*-colorear grafos arbitrarios. Haralick y Elliott (1980) propusieron la heurística del valor menos restringido.

El método básico de retroceso es debido a John Gaschnig (1977, 1979). Kondrak y Van Beek (1997) mostraron que este algoritmo es esencialmente subsumido por la comprobación anticipada. El retroceso dirigido por conflictos fue ideado por Prosser (1993). Dechter (1990a) introdujo el retroceso basado en grafos, que limita la complejidad de los algoritmos basados en retroceso como una función del grafo de restricciones (Dechter y Frost, 2002).

Retroceso dirigido por dependencias

Una forma general de retroceso inteligente fue desarrollada temprano por Stallman y Sussman (1977). Su técnica de **retroceso dirigido por dependencias** combina retroceso o salto atrás con el aprendizaje de no-bueno (McAllester, 1990), el cual llevó al desarrollo de **sistemas de**

mantenimiento de la verdad (Doyle, 1979), que discutimos en la Sección 10.6.2. La conexión entre las dos áreas es analizada por de Kleer (1989).

Aprendizaje de restricciones

El trabajo de Stallman y Sussman también introdujo la idea de **aprendizaje de restricciones**, en la que los resultados parciales obtenidos por la búsqueda pueden ser guardados y reutilizados más tarde en la búsqueda. La idea fue formalizada por Dechter (1990a). **Backmarking** (Gaschnig, 1979) es un método particularmente simple en el que se guardan las asignaciones consistentes e inconsistentes por pares y se usan para evitar volver a comprobar las restricciones. Backmarking puede ser combinado con retroceso dirigido por conflictos; Kondrak y Van Beek (1997) presentan un algoritmo híbrido que subsume de manera demostrable cualquier método tomado por separado.

El método de backtracking (**retroceso) dinámico** (Ginsberg, 1993) retiene asignaciones parciales exitosas de subconjuntos posteriores de variables al retroceder sobre una elección anterior que no invalida el éxito posterior. Moskewicz *et al*. (2001) muestran cómo estas técnicas y otras se usan para crear un solucionador eficiente de SAT. Gomes *et al*. (2000) y Gomes y Selman (2001) realizaron estudios empíricos de varios métodos de retrocesos aleatorizados. Van Beek (2006) realiza una revisión sobre retroceso.

La búsqueda local en problemas de satisfacción de restricciones fue popularizada por el trabajo de Kirkpatrick *et al*. (1983) sobre *simulated annealing* (recocido simulado) (ver Capítulo 4), que se usa ampliamente para la disposición de VLSI y problemas de programación. Beck *et al*. (2011) dan una visión general del trabajo reciente sobre la programación de talleres. La heurística de mínimos conflictos fue propuesta por primera vez por Gu (1989) y fue desarrollada independientemente por Minton *et al*. (1992). Sosic y Gu (1994) mostraron cómo se podría aplicar para resolver el problema de las 3 000 000 reinas en menos de un minuto. El asombroso éxito de la búsqueda local usando mínimos conflictos en el problema de las *n*-reinas llevó a una reevaluación de la naturaleza y prevalencia de los problemas "fáciles" y "difíciles". Peter Cheeseman *et al*. (1991) exploraron la dificultad de los CSP generados aleatoriamente y descubrieron que casi todos estos problemas son trivialmente fáciles o no tienen soluciones. Sólo si los parámetros del generador de problemas se establecen en un cierto rango estrecho, dentro del cual casi la mitad de los problemas son resolubles, encontramos instancias de problemas "difíciles". Discutimos este fenómeno más adelante en el Capítulo 7.

Konolige (1994) demostró que la búsqueda local es inferior a la búsqueda de retroceso en problemas con cierto grado de estructura local; esto llevó a trabajos que combinaban la búsqueda local y la inferencia, como el de Pinkas y Dechter (1995). Hoos y Tsang (2006) proporcionan una revisión de las técnicas de búsqueda local, y se ofrecen libros de texto de Hoos y Stützle (2004) y Aarts y Lenstra (2003).

El trabajo relacionado con la estructura y complejidad de los CSP se origina con Freuder (1985) y Mackworth y Freuder (1985), quienes mostraron que la búsqueda en árboles arco-consistentes funciona sin ningún retroceso. Un resultado similar, con extensiones a hipergrafos acíclicos, fue desarrollado en la comunidad de bases de datos (Beeri *et al*., 1983). Bayardo y Miranker (1994) presentan un algoritmo para CSP estructurados en árbol que se ejecuta en tiempo lineal sin ninguna preprocesamiento. Dechter (1990a) describe el enfoque de corte de ciclo.

Desde que se publicaron esos documentos, ha habido un gran progreso en el desarrollo de resultados más generales que relacionan la complejidad de resolver un CSP con la estructura de su grafo de restricciones. La noción de ancho de árbol fue introducida por los teóricos de grafos Robertson y Seymour (1986). Dechter y Pearl (1987, 1989), con base en el trabajo de Freuder, aplicaron una noción relacionada (a la que llamaron **ancho inducido** pero es idéntica al ancho del árbol) a los problemas de satisfacción de restricciones y desarrollaron el enfoque de descomposición de árboles esbozado en la Sección 6.5.

Con base en este trabajo y en los resultados de la teoría de bases de datos, Gottlob *et al*. (1999a, 1999b) desarrollaron una noción, **ancho de hiperárbol**, que se basa en la caracterización del CSP como un hipergrafo. Además de mostrar que cualquier CSP con ancho de hiperárbol w se puede resolver en tiempo $O(n^{w+1}\log n)$, también mostraron que el ancho de hiperárbol subsume todas las medidas previamente definidas de "ancho" en el sentido de que hay casos donde el ancho de hiperárbol está acotado y las otras medidas no están restringidas.

El algoritmo RELSAT de Bayardo y Schrag (1997) combinó el aprendizaje de restricciones y el salto hacia atrás, y se demostró que superaba a muchos otros algoritmos de la época. Esto llevó a encontrar algoritmos de búsqueda AND-OR aplicables tanto a CSP como a razonamiento probabilístico (Dechter y Mateescu, 2007). Brown *et al.* (1988) introducen la idea de romper la simetría en CSP, y Gent *et al.* (2006) hacen una revisión.

El campo de la **satisfacción de restricciones distribuidas** analiza la resolución de CSP cuando hay una colección de agentes, cada uno de los cuales controla un subconjunto de las variables de restricción. Ha habido talleres anuales sobre este problema desde 2000, y buena cobertura en otros lugares (Collin *et al*., 1999; Pearce *et al*., 2008).

Comparar los algoritmos CSP es en su mayoría una ciencia empírica: pocos resultados teóricos muestran que un algoritmo domina a otro en todos los problemas; en cambio, necesitamos realizar experimentos para ver qué algoritmos funcionan mejor en instancias típicas de problemas. Como señala Hooker (1995), debemos tener cuidado de distinguir entre las pruebas competitivas, como ocurre en las competencias entre algoritmos basados en el tiempo de ejecución, y las pruebas científicas, cuyo objetivo es identificar las propiedades de un algoritmo que determinan su eficacia en una clase de problemas.

Los libros de texto de Apt (2003), Dechter (2003), Tsang (1993) y Lecoutre (2009), y la colección de Rossi *et al*. (2006), son excelentes recursos sobre el procesamiento de restricciones. Hay varios buenos artículos de revisión, como los de Dechter y Frost (2002), y Barták *et al.* (2010). Carbonnel y Cooper (2016) que revisan las clases tratables de CSPs. Kondrak y Van Beek (1997) dan una revisión analítica de los algoritmos de búsqueda de retroceso, y Bacchus y Van Run (1995) dan una revisión más empírica. La programación de restricciones se cubre en los libros de Apt (2003) y Fruhwirth y Abdennadher (2003). Los artículos sobre satisfacción de restricciones aparecen regularmente en *Artificial Inteligence* y en la revista especializada *Constraints*; los últimos solucionadores SAT se describen en la Competencia Internacional SAT anual. La conferencia principal del área es la *International Conference on Principles and Practice of Constraint Programming*, a menudo llamada *CP*.

CAPÍTULO 7

AGENTES LÓGICOS

En el que diseñamos agentes que pueden formar representaciones de un mundo complejo, usar un proceso de inferencia para derivar nuevas representaciones sobre el mundo, y usar estas nuevas representaciones para deducir qué hacer.

Agentes basados en conocimiento
Razonamiento
Representación

Los humanos, parece, saben cosas; y lo que saben les ayuda a hacer cosas. En IA, los **agentes basados en conocimiento** utilizan un proceso de **razonamiento** sobre una **representación** interna del conocimiento para decidir qué acciones tomar.

Los agentes de resolución de problemas de los Capítulos 3 y 4 saben cosas, pero sólo en un sentido muy limitado e inflexible. Saben qué acciones están disponibles y cuál será el resultado de realizar una acción específica desde un estado específico, pero no conocen hechos generales. Un agente de búsqueda de rutas no sabe que es imposible que una carretera tenga un número negativo de kilómetros de longitud. Un agente de 8-puzzle no sabe que dos fichas no pueden ocupar el mismo espacio. El conocimiento que tienen es útil para encontrar un camino desde el inicio hasta una meta, pero no para algo más.

Las representaciones atómicas utilizadas por los agentes de resolución de problemas también son muy limitantes. Por ejemplo, en un entorno parcialmente observable, la única opción de un agente de resolución de problemas para representar lo que sabe sobre el estado actual es enumerar todos los estados concretos posibles. Podría darle a un humano el objetivo de conducir a una ciudad de Estados Unidos con una población menor a 10 000, pero para decir eso a un agente de resolución de problemas, debería describir formalmente el objetivo como un conjunto explícito de las 16 000 o más ciudades que satisfacen la descripción.

El Capítulo 6 introdujo nuestra primera representación factorizada, en la que los estados se representan como asignaciones de valores a variables; este es un paso en la dirección correcta, que permite que algunas partes del agente trabajen de manera independiente del dominio y permite algoritmos más eficientes. En este capítulo, llevamos este paso a su conclusión lógica, por así decirlo; desarrollamos la **lógica** como una clase general de representaciones para apoyar a los agentes basados en conocimiento. Estos agentes pueden combinar y recombinar información para adaptarse a una miríada de propósitos. Esto puede estar muy alejado de las necesidades del momento, como cuando un matemático demuestra un teorema o un astrónomo calcula la esperanza de vida de la Tierra. Los agentes basados en conocimiento pueden aceptar nuevas tareas en forma de objetivos descritos explícitamente, y pueden lograr competencia rápidamente al ser informados o aprender nuevos conocimientos sobre el entorno; además, los agentes pueden adaptarse a los cambios en el entorno actualizando el conocimiento relevante.

Comenzamos en la Sección 7.1 con el diseño general del agente. La Sección 7.2 introduce un nuevo entorno simple, el mundo del wumpus, e ilustra el funcionamiento de un agente basado en conocimiento sin entrar en ningún detalle técnico. Luego explicamos los principios generales de la lógica en la Sección 7.3 y los detalles específicos de la **lógica proposicional** en la Sección 7.4. La lógica proposicional es una representación factorizada; aunque menos expresiva que la **lógica de primer orden** (Capítulo 8), que es la representación estructurada canónica, la lógica proposicional

```
function KB-AGENT(percept) returns an action
  persistent: KB, una base de conocimiento
              t, un contador, inicialmente 0, indicando el tiempo

  TELL(KB,MAKE-PERCEPT-SENTENCE(percept,t))
  action←ASK(KB,MAKE-ACTION-QUERY(t))
  TELL(KB,MAKE-ACTION-SENTENCE(action,t))
  t←t + 1
  return action
```

Figura 7.1 Un agente basado en conocimiento genérico. Dado un percepto, el agente agrega el percepto a su base de conocimiento, pregunta a la base de conocimiento cuál es la mejor acción, y le dice a la base de conocimiento que, de hecho, ha tomado esa acción.

ilustra todos los conceptos básicos de la lógica. También viene con tecnologías de inferencia bien desarrolladas, que describimos en las secciones 7.5 y 7.6. Finalmente, en la Sección 7.7 se combina el concepto de agentes basados en conocimiento con la tecnología de la lógica proposicional para construir algunos agentes simples para el mundo del wumpus.

7.1 Agentes basados en conocimiento

Base de conocimiento (BC)

Sentencia

Lenguaje de representación del conocimiento

Axioma

El componente central de un agente basado en conocimiento es su **base de conocimiento (BC)**, o **KB (por sus siglas en inglés)**. Una base de conocimiento es un conjunto de **sentencias**. (Aquí "sentencias" se usa como un término técnico. Está relacionado pero no es idéntico a las oraciones del inglés y otros idiomas naturales). Cada sentencia se expresa en un lenguaje llamado **lenguaje de representación del conocimiento** y representa alguna afirmación sobre el mundo. Cuando la sentencia se toma como dada sin ser derivada de otras sentencias, la llamamos un **axioma**.

Inferencia

Debe haber una forma de agregar nuevas sentencias a la base de conocimiento y una forma de consultar lo que se sabe. Los nombres estándar para estas operaciones son TELL y ASK, respectivamente. Ambas operaciones pueden implicar **inferencia**, es decir, derivar nuevas sentencias de las antiguas. La inferencia debe obedecer el requisito de que cuando uno hace una pregunta(ASK) a la BC, la respuesta debe seguir de lo que se ha dicho (o TELLed) a la BC previamente. Más adelante en este capítulo, seremos más precisos sobre la palabra "seguir". Por ahora, úselo para significar que el proceso de inferencia no debería inventar cosas a medida que avanza.

La Figura 7.1 muestra el esquema de un programa de agente basado en conocimientos. Como todos nuestros agentes, toma un percepto como entrada y devuelve una acción. El agente mantiene una BC, que inicialmente puede contener algunos **conocimientos de fondo**.

Conocimientos de fondo

Cada vez que se llama al programa del agente, este hace tres cosas. Primero, le dice (TELLS) a la BC lo que percibe. En segundo lugar, le pregunta (ASKs) a la BC qué acción debe realizar. En el proceso de responder a esta consulta, se puede hacer un razonamiento extenso sobre el estado actual del mundo, sobre los resultados de posibles secuencias de acción, y así sucesivamente. En tercer lugar, el programa del agente le dice(TELLS) a la BC qué acción se eligió, y devuelve la acción para que pueda ser ejecutada.

Los detalles del lenguaje de representación están ocultos dentro de tres funciones que implementan la interfaz entre los sensores y los actuadores, por un lado, y el sistema central de representación y razonamiento, por el otro. MAKE-PERCEPT-SENTENCE construye una sentencia

afirmando que el agente percibió el percepto dado en el momento dado. MAKE-ACTION-QUERY construye una sentencia que pregunta qué acción debe hacerse en el momento actual. Finalmente, MAKE-ACTION-SENTENCE construye una sentencia afirmando que la acción elegida fue ejecutada. Los detalles de los mecanismos de inferencia están ocultos dentro de TELL y ASK. Las secciones posteriores revelarán estos detalles.

El agente en la Figura 7.1 parece muy similar a los agentes con estado interno descritos en el Capítulo 2. Sin embargo, debido a las definiciones de TELL y ASK, el agente basado en conocimientos no es un programa cualquiera para calcular acciones. Es susceptible a una descripción a

Nivel de conocimiento

nivel de conocimiento, donde sólo necesitamos especificar lo que el agente sabe y cuáles son sus objetivos para determinar su comportamiento.

Por ejemplo, un taxi automatizado podría tener el objetivo de llevar a un pasajero de San Francisco a Marin County, y podría saber que el Golden Gate Bridge es el único enlace entre ambas ubicaciones. Entonces podemos esperar que cruce el puente Golden Gate *porque sabe que de esa forma logrará su objetivo*. Observe que este análisis es independiente de cómo funciona el

Nivel de implementación

taxi a **nivel de implementación**. No importa si su conocimiento geográfico está implementado como listas enlazadas o mapas de píxeles, o si razona manipulando cadenas de símbolos almacenados en registros o propagando señales ruidosas en una red de neuronas.

Un agente basado en conocimientos puede ser construido simplemente diciéndole (TELLing) lo que necesita saber. Comenzando con una BC vacía, el diseñador del agente puede decir (TELL) sentencias una por una hasta que el agente sepa cómo operar en su entorno. Esto se llama enfo-

Declarativo

Procedural

que **declarativo** para la construcción de sistemas. En contraste, el enfoque **procedural** codifica los comportamientos deseados directamente como código de programa. En las décadas de 1970 y 1980, los defensores de ambos enfoques participaron en acalorados debates. Ahora entendemos que un agente exitoso a menudo combina elementos declarativos y procedurales en su diseño, y que el conocimiento declarativo suele ser compilado en código procedural más eficiente.

También podemos proporcionar a un agente basado en conocimientos mecanismos que le permitan aprender por sí mismo. Estos mecanismos, que se discuten en el Capítulo 19, crean conocimiento general sobre el entorno a partir de una serie de perceptos. Un agente de aprendizaje puede ser totalmente autónomo.

7.2 El Mundo del wumpus

En esta sección describimos un entorno en el que los agentes basados en conocimientos pue-

Mundo del wumpus

den mostrar su valía. El **mundo del wumpus** es una cueva compuesta por habitaciones conectadas por pasadizos. Acechando en algún lugar de la cueva está el terrible wumpus, una bestia que se come a cualquiera que entre en su habitación. Un agente puede disparar al wumpus, pero el agente sólo tiene una flecha. Algunas habitaciones contienen pozos sin fondo que atraparán a cualquiera que se adentre en estas habitaciones (excepto al wumpus, que es demasiado grande para caer). La única característica redentora de este sombrío entorno es la posibilidad de encontrar un montón de oro. Aunque el mundo del wumpus es bastante aburrido, según los estándares modernos de los videojuegos, ilustra algunos puntos importantes sobre la inteligencia.

Un ejemplo de mundo del wumpus se muestra en la Figura 7.2. La definición precisa del entorno de la tarea se da, como se sugiere en la Sección 2.3, por la descripción PEAS:

- **Medida de rendimiento**: +1 000 por salir de la cueva con el oro, –1 000 por caer en un pozo o ser comido por el wumpus, –1 por cada acción tomada, y –10 por usar la flecha. El juego termina cuando el agente muere o cuando el agente sale de la cueva.

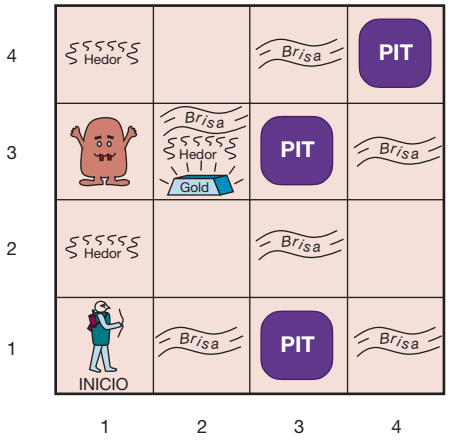
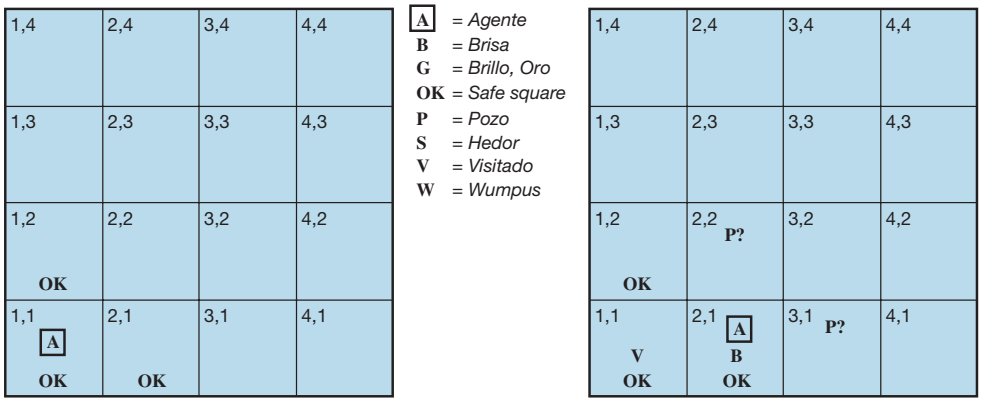


Figura 7.2 Un típico mundo del wumpus. El agente está en la esquina inferior izquierda, mirando hacia el este (hacia la derecha).

- **Entorno**: Una cuadrícula de 4 × 4 habitaciones, con paredes rodeando la cuadrícula. El agente siempre comienza en la casilla etiquetada como [1,1], mirando hacia el este. Las ubicaciones del oro y el wumpus se eligen al azar, con una distribución uniforme, de las casillas distintas a la casilla de inicio. Además, cada casilla distinta a la de inicio puede ser un pozo, con probabilidad 0.2.
- **Actuadores**: El agente puede *Avanzar*, *GirarIzquierda* 90°, o *GirarDerecha* 90°. El agente tiene una muerte miserable si entra en una casilla que contiene un pozo o un wumpus vivo. (Es seguro, aunque apestoso, entrar en una casilla con un wumpus muerto). Si un agente intenta moverse hacia adelante y choca con una pared, entonces el agente no se mueve. La acción *Agarrar* (*Grab*) puede ser utilizada para recoger el oro si está en la misma casilla que el agente. La acción *Disparar* (*Shoot*) puede ser utilizada para disparar una flecha en línea recta en la dirección en la que el agente está mirando. La flecha continúa hasta que golpea (y, por lo tanto, mata) al wumpus o golpea una pared. El agente sólo tiene una flecha, por lo que sólo la primera acción de *Shoot* tiene algún efecto. Finalmente, la acción *Escalar* (*Climb)* puede ser utilizada para salir de la cueva, pero sólo desde la casilla [1,1].
- **Sensores**: El agente tiene cinco sensores, cada uno proporciona un solo bit de información:
  - En las casillas directamente (no diagonalmente) adyacentes al wumpus, el agente percibirá un *Hedor* (*Stench*).[1]
  - En las casillas directamente adyacentes a un pozo, el agente percibirá una *Brisa* (*Breeze*).
  - En la casilla donde está el oro, el agente percibirá un *Brillo* (*Glitter*).
  - Cuando un agente se choca con una pared, percibirá un *Golpe* (*Bump*).
  - Cuando el wumpus es asesinado, emite un *Grito (Screem)* doloroso que puede ser percibido en cualquier lugar de la cueva.

  Los perceptos serán dados al programa del agente en forma de una lista de cinco símbolos; por ejemplo, si hay un hedor y una brisa, pero no brillo, golpe, o grito, el programa del agente recibirá [*Hedor*, *Brisa*, *Nada*, *Nada*, *Nada*].

[1] Presumiblemente la casilla que contiene al wumpus también tiene un hedor, pero cualquier agente que entre en esa casilla es comido antes de poder percibir algo.



(a) (b)

Figura 7.3 El primer paso dado por el agente en el mundo del wumpus. (a) Situación inicial, después del percepto [*Nada*, *Nada*, *Nada*, *Nada*, *Nada*]. (b) Después de moverse a [2,1] y percibir [*Nada*, *Brisa*, *Nada*, *Nada*, *Nada*].

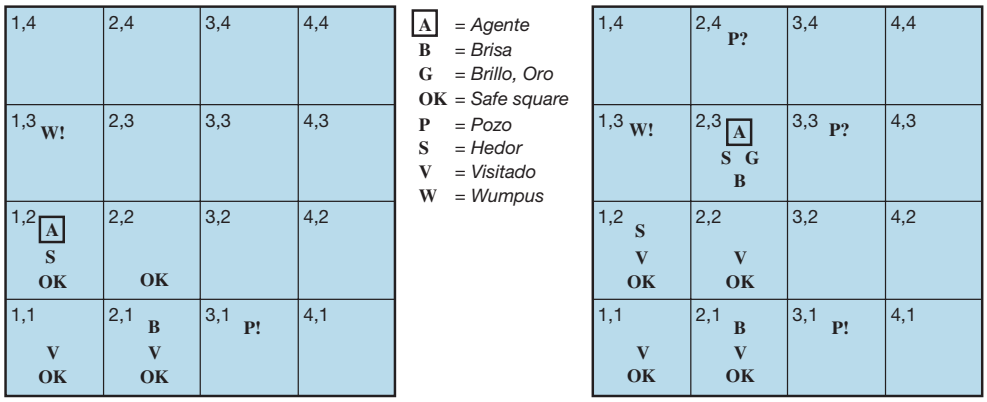
Podemos caracterizar el entorno wumpus a lo largo de las diversas dimensiones dadas en el Capítulo 2. Claramente, es determinista, discreto, estático y de un solo agente. (El wumpus no se mueve, afortunadamente). Es secuencial, porque las recompensas pueden llegar sólo después de que se han tomado muchas acciones. Es parcialmente observable, porque algunos aspectos del estado no son directamente perceptibles: la ubicación del agente, el estado de salud del wumpus y la disponibilidad de una flecha. En cuanto a las ubicaciones de los pozos y el wumpus: podríamos tratarlos como partes no observadas del estado, en cuyo caso, el modelo de transición para el entorno es completamente conocido, y encontrar las ubicaciones de los pozos completa el conocimiento del agente del estado. Alternativamente, podríamos decir que el modelo de transición en sí es desconocido porque el agente no sabe cuáles acciones *hacia adelante* son fatales; en cuyo caso, descubrir las ubicaciones de los pozos y wumpus completa el conocimiento del agente del modelo de transición.

Para un agente en el entorno, el principal desafío es su ignorancia inicial de la configuración del entorno; superar esta ignorancia parece requerir razonamiento lógico. En la mayoría de las instancias del mundo del wumpus, es posible que el agente recupere el oro de manera segura. Ocasionalmente, el agente debe elegir entre irse a casa con las manos vacías o arriesgarse a morir para encontrar el oro. Cerca del 21 % de los entornos son absolutamente injustos, porque el oro está en un pozo o rodeado de pozos.

Observemos a un agente de wumpus basado en conocimientos explorando el entorno mostrado en la Figura 7.2. Utilizamos un lenguaje de representación de conocimientos informal que consiste en escribir símbolos en una cuadrícula (como en las Figuras 7.3 y 7.4).

La BC inicial del agente contiene las reglas del entorno, como antes se describió; en particular, sabe que está en [1,1] y que [1,1] es una casilla segura; denotamos eso con una “A” y “OK”, respectivamente, en la casilla [1,1].

El primer percepto es [*Nada*, *Nada*, *Nada*, *Nada*, *Nada*], a partir del cual el agente puede concluir que sus casillas vecinas, [1,2] y [2,1], están libres de peligros, están OK. Figura 7.3(a) muestra el estado de conocimiento del agente en este punto.



(a) (b)

Figura 7.4 Dos etapas posteriores en el progreso del agente. (a) Después de moverse a [1,1] y luego a [1,2], y percibir [*Hedor*, *Nada*, *Nada*, *Nada*, *Nada*]. (b) Después de moverse a [2,2] y luego a [2,3], y percibir [*Hedor*, *Brisa*, *Brillo*, *Nada*, *Nada*].

Un agente cauteloso sólo se moverá a una casilla que sabe que está OK. Suponga que el agente decide avanzar a [2,1]. El agente percibe una brisa (denotada por "B") en [2,1], por lo que debe haber un pozo en una casilla vecina. El pozo no puede estar en [1,1], por las reglas del juego, por lo que debe haber un pozo en [2,2] o [3,1] o en ambos. La notación "P?" en la Figura 7.3(b) indica un posible pozo en esas casillas. En este punto, sólo hay una casilla conocida que está OK y que aún no ha sido visitada. Por lo tanto, el agente prudente dará la vuelta, volverá a [1,1] y luego procederá a [1,2].

El agente percibe un hedor en [1,2], lo que resulta en el estado de conocimiento mostrado en la Figura 7.4(a). El hedor en [1,2] significa que debe haber un wumpus cerca. Pero el wumpus no puede estar en [1,1], por las reglas del juego, y no puede estar en [2,2] (o el agente habría detectado un hedor cuando estaba en [2,1]). Por lo tanto, el agente puede inferir que el wumpus está en [1,3]. La notación W! indica esta inferencia. Además, la falta de una brisa en [1,2] implica que no hay un pozo en [2,2]. Sin embargo, el agente ya ha inferido que debe haber un pozo en [2,2] o [3,1], por lo que esto significa que debe estar en [3,1]. Esta es una inferencia bastante difícil, porque combina conocimientos adquiridos en diferentes momentos y en diferentes lugares, y se basa en la falta de un percepto para dar un paso crucial.

El agente ahora ha demostrado que no hay ni un pozo ni un wumpus en [2,2], por lo que está bien moverse allí. No mostramos el estado de conocimiento del agente en [2,2]; sólo asumimos que el agente gira y se mueve a [2,3], lo que nos da la Figura 7.4(b). En [2,3], el agente detecta un brillo, por lo que debería agarrar el oro y luego regresar a casa.

Note que en cada caso en el que el agente saca una conclusión de la información disponible, esa conclusión está *garantizada* para ser correcta si la información disponible es correcta. Esta es una propiedad fundamental del razonamiento lógico. En el resto de este capítulo, describimos cómo construir agentes lógicos que puedan representar información y sacar conclusiones como las descritas en los párrafos anteriores.

7.3 Lógica

Esta sección resume los conceptos fundamentales de la representación lógica y el razonamiento. Estas hermosas ideas son independientes de cualquiera de las formas particulares de la lógica. Por lo tanto, posponemos los detalles técnicos de esas formas hasta la próxima sección, usando en su lugar el ejemplo familiar de la aritmética ordinaria.

Sintaxis

En la Sección 7.1, dijimos que las BC consisten en sentencias. Estas sentencias se expresan de acuerdo con la **sintaxis** del lenguaje de representación, que especifica todas las sentencias que están bien formadas. La noción de sintaxis es clara en la aritmética ordinaria: "$x + y = 4$" es una sentencia bien formada, mientras que "$x4y+ =$" no lo es.

Semántica
Verdad
Mundo posible

Una lógica también debe definir la **semántica**, o significado, de las sentencias. La semántica define la **verdad** de cada sentencia con respecto a cada **mundo posible**. Por ejemplo, la aritmética especifica que la sentencia "$x + y = 4$" es verdadera en un mundo donde x es 2 y y es 2, pero es falsa en un mundo donde x es 1 y y es 1. En lógicas estándar, cada sentencia debe ser o verdadera o falsa en cada mundo posible, no hay un "punto medio".[2]

Modelo

Cuando necesitamos ser precisos, usamos el término **modelo** en lugar de "mundo posible". Mientras que los mundos posibles podrían ser pensados como (potencialmente) entornos reales en los que el agente podría estar o no estar, los modelos son abstracciones matemáticas, cada uno de los cuales tiene un valor de verdad fijo (verdadero o falso) para cada sentencia relevante. De manera informal, podemos pensar en un mundo posible como, por ejemplo, tener x hombres y y mujeres sentados en una mesa jugando al bridge, y la sentencia $x + y = 4$ es verdadera cuando hay cuatro personas en total. Formalmente, los posibles modelos son sólo todas las posibles asignaciones de enteros no negativos a las variables x y y. Cada una de estas asignaciones determina la verdad de cualquier sentencia de aritmética cuyas variables son x y y. Si una sentencia α es verdadera en el modelo m, decimos que m **satisface** α o a veces m **es un modelo de** α. Usamos la notación $M(\alpha)$ para significar el conjunto de todos los modelos de α.

Satisfacción

Implicación

Ahora que tenemos una noción de verdad, estamos listos para hablar sobre el razonamiento lógico. Esto implica la relación de **implicación** lógica entre sentencias, la idea de que una sentencia *sigue lógicamente* de otra sentencia. En notación matemática, escribimos

$$\alpha \models \beta$$

para significar que la sentencia α implica la sentencia β. La definición formal de implicación es: $\alpha \models \beta$ si y sólo si, en cada modelo en el que α es verdadera, β también es verdadera. Usando la notación recién introducida, podemos escribir

$$\alpha \models \beta \text{ si y sólo si } M(\alpha) \subseteq M(\beta).$$

(Note la dirección del $\subseteq$ aquí: si $\alpha \models \beta$, entonces α es una afirmación más fuerte que β: excluye *más* mundos posibles). La relación de implicación es familiar en aritmética; estamos contentos con la idea de que la sentencia $x = 0$ implica la sentencia $xy = 0$. Obviamente, en cualquier modelo donde x es cero, es el caso que xy es cero (independientemente del valor de y).

Podemos aplicar el mismo tipo de análisis al ejemplo de razonamiento del mundo del wumpus dado en la sección anterior. Considere la situación en la Figura 7.3(b): el agente no ha detectado nada en [1,1] y una brisa en [2,1]. Estos perceptos, combinados con el conocimiento del agente de las reglas del mundo del wumpus, constituyen la BC. El agente está interesado en si las casillas adyacentes [1,2], [2,2], y [3,1] contienen pozos. Cada una de las tres casillas podría o no contener

[2] La lógica difusa, discutida en el Capítulo 13, permite grados de verdad.

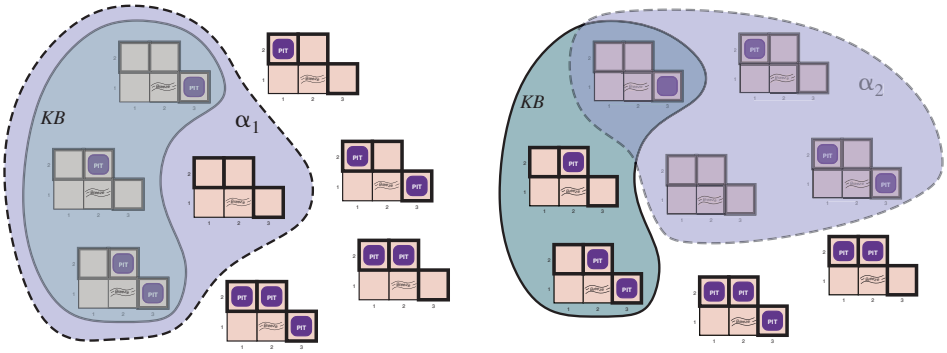


Figura 7.5 Modelos posibles para la presencia de pozos en las casillas [1,2], [2,2], y [3,1]. La BC correspondiente a las observaciones de nada en [1,1] y una brisa en [2,1] se muestra con la línea sólida. (a) La línea punteada muestra modelos de α_1 (no hay pozo en [1,2]). (b) La línea punteada muestra modelos de α_2 (no hay pozo en [2,2]).

un pozo, por lo que (ignorando otros aspectos del mundo por ahora) hay $2^3 = 8$ modelos posibles. Estos ocho modelos se muestran en la Figura 7.5.[3]

La BC puede ser pensada como un conjunto de sentencias o como una sola sentencia que afirma todas las sentencias individuales. La BC es falsa en modelos que contradicen lo que el agente sabe, por ejemplo, la BC es falsa en cualquier modelo en el que [1,2] contiene un pozo, porque no hay brisa en [1,1]. De hecho, sólo hay tres modelos en los que la BC es verdadera, y estos se muestran rodeados por una línea sólida en la Figura 7.5. Ahora consideremos dos posibles conclusiones:

α_1 = "No hay pozo en [1,2]" α_2 = "No hay pozo en [2,2]".

Hemos rodeado los modelos de α_1 y α_2 con líneas punteadas en las Figuras 7.5(a) y 7.5(b), respectivamente. Por inspección, vemos lo siguiente:

en cada modelo en el que BC es verdadera, α_1 también es verdadera.

Por lo tanto, BC |= α_1: no hay pozo en [1,2]. También podemos ver que

en algunos modelos en los que BC es verdadera, α_2 es falsa.

Por lo tanto, BC no implica α_2: el agente *no puede* concluir que no hay un pozo en [2,2]. (Tampoco puede concluir que hay un pozo en [2,2]).[4]

El ejemplo anterior no sólo ilustra la implicación sino que también muestra cómo la definición de implicación se puede aplicar para derivar conclusiones; es decir, para llevar a cabo la **inferencia lógica**. La inferencia del algoritmo ilustrado en la Figura 7.5 se llama **verificación de modelos**, porque enumera todos los posibles modelos para verificar que α es verdadero en todos los modelos en los que BC es verdadero, es decir, que $M(BC) \subseteq M(\alpha)$.

Inferencia lógica
Verificación de modelos

[3] Aunque la figura muestra los modelos como mundos parciales del wumpus, realmente no son más que asignaciones de *verdadero* y *falso* a las sentencias "hay un pozo en [1,2]" etc. Los modelos, en el sentido matemático, no necesitan tener wumpuses 'horribles' en ellos.

[4] El agente puede calcular la *probabilidad* de que haya un pozo en [2,2]; El Capítulo 12 muestra cómo.

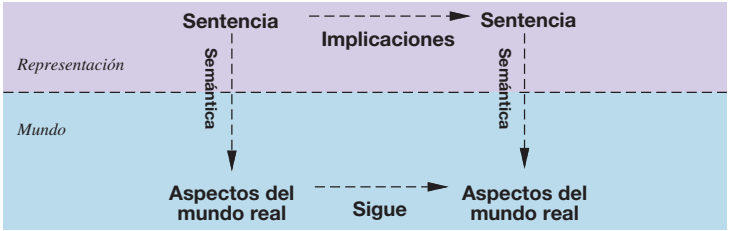


Figura 7.6 Las sentencias son configuraciones físicas del agente, y el razonamiento es un proceso de construcción de nuevas configuraciones físicas a partir de las antiguas. El razonamiento lógico debe garantizar que las nuevas configuraciones representen aspectos del mundo que realmente se derivan de los aspectos que representan las configuraciones antiguas.

Para entender la implicación y la inferencia lógica, podría ayudar pensar en el conjunto de todas las consecuencias de la BC como un pajar y de α como una aguja. La implicación es como la aguja en el pajar; la inferencia es como encontrarla. Esta distinción se incorpora en alguna notación formal: si un algoritmo de inferencia i puede derivar α de la BC, escribimos

$$\text{BC} \vdash_i \alpha,$$

que se pronuncia "α se deriva de la BC mediante i" o "i deriva α de la BC".

Sonido
Preservación de la verdad

Un algoritmo de inferencia que deriva sólo sentencias implicadas se llama **sonido** o **preservación de la verdad**. La solidez es una propiedad muy deseable. Un procedimiento de inferencia no sólido esencialmente inventa cosas a medida que avanza; anuncia el descubrimiento de agujas inexistentes. Es fácil ver que la verificación de modelos, cuando es aplicable,[5] es un procedimiento sólido.

Completitud

La propiedad de **completitud** también es deseable: un algoritmo de inferencia es completo si puede derivar cualquier sentencia que esté implicada. Para los pajares reales, finitos en extensión, parece obvio que un examen sistemático siempre puede decidir si la aguja está en el pajar. Para muchas BC, sin embargo, el pajar de consecuencias es infinito, y la completitud se convierte en un problema importante.[6] Por fortuna, existen procedimientos de inferencia completos para lógicas que son lo suficientemente expresivas para manejar muchas BC.

Hemos descrito un proceso de razonamiento cuyas conclusiones están garantizadas para ser verdaderas en cualquier mundo en el que las premisas sean verdaderas; en particular, *si la BC es verdadera en el mundo real, entonces cualquier sentencia* α *derivada de la BC por un procedimiento de inferencia sólido también es verdadera en el mundo real*. Por lo tanto, mientras que un proceso de inferencia opera en "sintaxis", configuraciones físicas internas como bits en registros o patrones de señales eléctricas en el cerebro, el proceso *corresponde* a la relación del mundo real en la que algún aspecto del mundo real es el caso en virtud de otros aspectos del mundo real que son el caso.[7] Esta correspondencia entre el mundo y la representación se ilustra en la Figura 7.6.

Fundamentación

El último problema a considerar es la **fundamentación**, la conexión entre los procesos de razonamiento lógico y el entorno real en el que existe el agente. En particular, *¿cómo sabemos*

[5] La verificación de modelos funciona si el espacio de modelos es finito; por ejemplo, en mundos de wumpus de tamaño fijo. Para la aritmética, por otro lado, el espacio de modelos es infinito: aun si nos restringimos a los enteros, hay infinitos pares de valores para x y y en la expresión $x + y = 4$.

[6] Compare con el caso de espacios de búsqueda infinitos en el Capítulo 3, donde la búsqueda en profundidad no es completa.

[7] Como lo expresó Wittgenstein (1922) en su famoso *Tractatus*: "El mundo es todo lo que es el caso".

que la BC es verdadera en el mundo real? (Después de todo, la BC es sólo "sintaxis" dentro de la cabeza del agente). Esta es una pregunta filosófica sobre la cual se han escrito muchos libros. (Ver Capítulo 27). Una respuesta simple es que los sensores del agente crean la conexión; por ejemplo, nuestro agente del mundo del wumpus tiene un sensor de olor. El programa del agente crea una sentencia adecuada siempre que hay un olor. Entonces, siempre que esa sentencia está en la BC, es verdadera en el mundo real. Por lo tanto, el significado y la verdad de las sentencias perceptivas están definidos por los procesos de detección y construcción de sentencias que las producen. ¿Qué pasa con el resto del conocimiento del agente, como su creencia de que los wumpuses causan olores en las casillas adyacentes? Esta no es una representación directa de un sólo percepto, sino una regla general, derivada, quizá, de la experiencia perceptual pero no idéntica a una declaración de esa experiencia. Reglas generales como esta son producidas por un proceso de construcción de sentencias llamado **aprendizaje**, que es el tema de la Parte V. El aprendizaje es falible. Podría ser el caso de que los wumpuses causen olores *excepto el 29 de febrero en años bisiestos*, que es cuando se bañan. Por lo tanto, la BC puede no ser verdadera en el mundo real, pero con buenos procedimientos de aprendizaje, hay razones para el optimismo.

7.4 Lógica proposicional: una lógica muy simple

Lógica proposicional

Ahora presentamos la **lógica proposicional**. Describimos su sintaxis (la estructura de las sentencias) y su semántica (la forma en que se determina la verdad de las sentencias); a partir de éstas derivamos un algoritmo sintáctico simple para la inferencia lógica que implementa la noción semántica de implicación. Todo ocurre, por supuesto, en el mundo del wumpus.

7.4.1 Sintaxis

Sentencias atómicas
Símbolo de proposición

La **sintaxis** de la lógica proposicional define las sentencias permitidas. Las **sentencias atómicas** consisten en un solo **símbolo de proposición**. Cada uno de estos símbolos representa una proposición que puede ser verdadera o falsa. Usamos símbolos que comienzan con una letra mayúscula y pueden contener otras letras o subíndices, por ejemplo: P, Q, R, $W_{1,3}$ y *MirandoAlEste*. Los nombres son arbitrarios pero a menudo se eligen para tener algún valor mnemotécnico, usamos $W_{1,3}$ para representar la proposición de que el wumpus está en [1,3]. (Recuerde que símbolos como $W_{1,3}$ son *atómicos*, es decir, W, 1, y 3 no son partes significativas del símbolo). Hay dos símbolos de proposición con significados fijos: *True* es la proposición siempre verdadera y *False* es la proposición siempre falsa. Las **sentencias complejas** se construyen a partir de sentencias más simples, utilizando paréntesis y operadores, llamados **conectivos lógicos**. Hay cinco conectivos lógicos de uso común:

Sentencias complejas
Conectivos lógicos

- $\neg$ (no). Una sentencia como $\neg W_{1,3}$ se llama la **negación** de $W_{1,3}$. Un **literal** es ya sea una sentencia atómica (un **literal positivo**) o una sentencia atómica negada (un **literal negativo**). — Negación, Literal
- $\wedge$ (y). Una sentencia cuyo conectiva principal es $\wedge$, como $W_{1,3} \wedge P_{1,3}$, se llama una **conjunción**; sus partes son los conjuntores. (El $\wedge$ parece una "A" como en "And"). — Conjunción
- $\vee$ (o). Una sentencia cuyo conector principal es $\vee$, como $(W_{1,3} \wedge P_{1,3}) \vee W_{2,2}$, es una **disyunción**; sus partes son **disyuntores**, en este ejemplo, $(W_{1,3} \wedge P_{1,3})$ y $W_{2,2}$. — Disyunción
- $\Rightarrow$ (implica). Una sentencia como $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ se llama una **implicación** (o condicional). Su **premisa** o **antecedente** es $(W_{1,3} \wedge P_{3,1})$, y su **conclusión** o **consecuente** es $\neg W_{2,2}$. Las implicaciones también se conocen como **reglas** o declaraciones **si-entonces**. El símbolo de implicación en otros libros a veces se escribe como $\supset$ o $\rightarrow$. — Implicación, Premisa, Conclusiones, Reglas
- $\Leftrightarrow$ (si y sólo si). La sentencia $W_{1,3} \Leftrightarrow \neg W_{2,2}$ es un **bicondicional**. — Bicondicional

$$
\begin{array}{rcl}
\textit{Sentencia} & \rightarrow & \textit{SentenciaAtómica} \mid \textit{SentenciaCompleja} \\
\textit{SentenciaAtómica} & \rightarrow & \textit{Verdadero} \mid \textit{Falso} \mid P \mid Q \mid R \mid \ldots \\
\textit{SentenciaCompleja} & \rightarrow & (\ \textit{Sentencia}\ ) \\
 & \mid & \neg\ \textit{Sentencia} \\
 & \mid & \textit{Sentencia} \wedge \textit{Sentencia} \\
 & \mid & \textit{Sentencia} \vee \textit{Sentencia} \\
 & \mid & \textit{Sentencia} \Rightarrow \textit{Sentencia} \\
 & \mid & \textit{Sentencia} \Leftrightarrow \textit{Sentencia} \\
\text{Precedencia de Operadores} & : & \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{array}
$$

Figura 7.7 Una gramática BNF (Forma de Backus-Naur) de sentencias en lógica proposicional, junto con precedencias de operadores, de mayor a menor.

La Figura 7.7 proporciona una gramática formal de la lógica proposicional. (La notación BNF se explica en la página 1030). La gramática BNF se complementa con una lista de precedencia de operadores para eliminar la ambigüedad cuando se utilizan varios operadores. El operador "not" ($\neg$) tiene la mayor precedencia, lo que significa que en la frase $\neg A \wedge B$ el $\neg$ se une más fuertemente, dándonos el equivalente de $(\neg A) \wedge B$, en lugar de $\neg(A \wedge B)$. (La notación para la aritmética ordinaria es la misma: $-2 + 4$ es 2, no -6). Cuando corresponde, también usamos paréntesis y corchetes para aclarar la estructura de la frase prevista y mejorar la legibilidad.

7.4.2 Semántica

Una vez especificada la sintaxis de la lógica proposicional, ahora especificamos su semántica. La semántica define las reglas para determinar la verdad de una sentencia con respecto a un modelo particular. En lógica proposicional, un modelo simplemente establece el **valor de verdad**, *verdadero* o *falso*, para cada símbolo de proposición. Por ejemplo, si las sentencias en la base de conocimiento hacen uso de los símbolos de proposición $P_{1,2}$, $P_{2,2}$, y $P_{3,1}$, entonces un modelo posible es

Valor de verdad

$$m_1 = \{P_{1,2} = \textit{falso}, P_{2,2} = \textit{falso}, P_{3,1} = \textit{verdadero}\}.$$

Con tres símbolos de proposición, hay $2^3 = 8$ modelos posibles, exactamente los representados en la Figura 7.5. Sin embargo, observe que los modelos son objetos matemáticos puros sin conexión necesaria con los mundos del wumpus. $P_{1,2}$ es sólo un símbolo; podría significar "hay un pozo en [1,2]" o "Estoy en París hoy y mañana".

La semántica para la lógica proposicional debe especificar cómo calcular el valor de verdad de *cualquier* sentencia, dado un modelo. Esto se hace de forma recursiva. Todas las sentencias se construyen a partir de sentencias atómicas y los cinco conectivos; por lo tanto, necesitamos especificar cómo calcular la verdad de las sentencias atómicas y cómo calcular la verdad de las sentencias formadas con cada uno de los cinco conectivos. Las sentencias atómicas son fáciles:

- *Verdadero* es verdadero en cada modelo y *Falso* es falso en cada modelo.
- El valor de verdad de cualquier otro símbolo de proposición debe especificarse directamente en el modelo. Por ejemplo, en el modelo m_1 dado antes, $P_{1,2}$ es falso.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
falso	*falso*	*verdadero*	*falso*	*falso*	*verdadero*	*verdadero*
falso	*verdadero*	*verdadero*	*falso*	*verdadero*	*verdadero*	*falso*
verdadero	*falso*	*falso*	*falso*	*verdadero*	*falso*	*falso*
verdadero	*verdadero*	*falso*	*verdadero*	*verdadero*	*verdadero*	*verdadero*

Figura 7.8 Tablas de verdad para los cinco conectivos lógicos. Para usar la tabla para calcular, por ejemplo, el valor de $P \vee Q$, cuando P es verdadero y Q es falso, primero busque a la izquierda la fila donde P es *verdadero* y Q es *falso* (la tercera fila). Luego mire en esa fila bajo la columna $P \vee Q$ para ver el resultado: *verdadero*.

Para sentencias complejas, tenemos cinco reglas, que se aplican a cualquier subsentencia P y Q (atómica o compleja) en cualquier modelo m (aquí "iff" significa "si y sólo si"):

- $\neg P$ es verdadero iff P es falso en m.
- $P \wedge Q$ es verdadero iff tanto P como Q son verdaderos en m.
- $P \vee Q$ es verdadero iff P o Q son verdaderos en m.
- $P \Rightarrow Q$ es verdadero a menos que P sea verdadero y Q sea falso en m.
- $P \Leftrightarrow Q$ es verdadero iff P y Q son ambos verdaderos o ambos falsos en m.

Tablas de verdad

Las reglas también pueden expresarse con **tablas de verdad** que especifican el valor de verdad de una sentencia compleja para cada posible asignación de valores de verdad a sus componentes. Las tablas de verdad para los cinco conectivos se dan en la Figura 7.8. A partir de estas tablas, el valor de verdad de cualquier sentencia s puede calcularse con respecto a cualquier modelo m mediante una simple evaluación recursiva. Por ejemplo, la sentencia $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$, evaluada en m_1, da *verdadero* $\wedge$ (*falso* $\vee$ *verdadero*) = *verdadero* $\wedge$ *verdadero* = *verdadero*. El Ejercicio 7.TRUV le pide que escriba el algoritmo PL-TRUE?(s,m), que calcula el valor de verdad de una sentencia de lógica proposicional s en un modelo m.

Las tablas de verdad para "and", "or" y "not" están en estrecha concordancia con nuestras intuiciones sobre las palabras en inglés. El principal punto de posible confusión es que $P \vee Q$ es verdadero cuando P es verdadero o Q es verdadero *o ambos*. Un conectivo diferente, llamado "o exclusivo" ("xor", para abreviar), produce falso cuando ambos disyuntivos son verdaderos.[8] No hay consenso sobre el símbolo para el o exclusivo; algunas opciones son $\dot{\vee}$ o $\neq$ o $\oplus$.

La tabla de verdad para $\Rightarrow$ puede no ajustarse del todo a la comprensión intuitiva de "P implica Q" o "si entonces Q". Para empezar, la lógica proposicional no requiere ninguna relación de *causación* o relevancia entre P y Q. La frase "5 es impar implica que Tokio es la capital de Japón" es una frase verdadera de la lógica proposicional (bajo la interpretación normal), aunque es una frase decididamente extraña en lenguaje natural. Otro punto de confusión es que cualquier implicación es verdadera siempre que su antecedente sea falso. Por ejemplo, "5 es par implica que Sam es inteligente" es verdadero, independientemente de si Sam es inteligente. Esto parece extraño, pero tiene sentido si piensas en "$P \Rightarrow Q$" como diciendo, "Si P es verdadero, entonces estoy afirmando que Q es verdadero; de lo contrario no estoy haciendo ninguna afirmación". La única forma de que esta sentencia sea *falsa* es si P es verdadero pero Q es falso.

El bicondicional, $P \Leftrightarrow Q$, es verdadero siempre que tanto $P \Rightarrow Q$ como $Q \Rightarrow P$ sean verdaderos. En español, esto se escribe a menudo como "P si y sólo si Q". Muchas de las reglas del mundo del wumpus se escriben mejor usando $\Leftrightarrow$. Por ejemplo, en una casilla hay brisa *si* una casilla vecina

[8] El latín utiliza dos palabras separadas: "vel" es o inclusivo y "aut" es o exclusivo.

tiene un pozo, y en una casilla hay brisa *sólo si* una casilla vecina tiene un pozo. Así que necesitamos un bicondicional,

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}),$$

donde $B_{1,1}$ significa que hay una brisa en [1,1].

7.4.3 Una base de conocimiento (BC) simple

Ahora que hemos definido la semántica para la lógica proposicional, podemos construir una BC para el mundo del wumpus. Nos enfocamos primero en los aspectos *inmutables* del mundo del wumpus, y dejamos los aspectos mutables para una sección posterior. Por ahora, necesitamos los siguientes símbolos para cada ubicación $[x, y]$:

$P_{x,y}$ es verdadero si hay un pozo en $[x, y]$.
$W_{x,y}$ es verdadero si hay un wumpus en $[x, y]$, muerto o vivo.
$B_{x,y}$ es verdadero si hay una brisa en $[x, y]$.
$S_{x,y}$ es verdadero si hay un hedor en $[x, y]$.
$L_{x,y}$ es verdadero si el agente está en la ubicación $[x, y]$.

Las sentencias que escribimos serán suficientes para derivar $\neg P_{1,2}$ (no hay pozo en [1,2]), como se hizo informalmente en la Sección 7.3. Etiquetamos cada sentencia R_i para que podamos referirnos a ellas:

- No hay pozo en [1,1]:

  $R_1 : \quad \neg P_{1,1}.$

- Una casilla tiene brisa si y sólo si hay un pozo en una casilla vecina. Esto tiene que ser declarado para cada casilla; por ahora, incluimos sólo las casillas relevantes:

  $R_2 : \quad B_{1,1} \quad \Leftrightarrow \quad (P_{1,2} \vee P_{2,1}).$
  $R_3 : \quad B_{2,1} \quad \Leftrightarrow \quad (P_{1,1} \vee P_{2,2} \vee P_{3,1}).$

- Las sentencias precedentes son verdaderas en todos los mundos de wumpus. Ahora incluimos las percepciones de brisa para las primeras dos casillas visitadas en el mundo específico en el que se encuentra el agente, hasta llegar a la situación en la Figura 7.3(b).

  $R_4 : \quad \neg B_{1,1}.$
  $R_5 : \quad B_{2,1}.$

7.4.4 Un procedimiento de inferencia simple

Nuestro objetivo ahora es decidir si $BC \models \alpha$ para alguna sentencia α. Por ejemplo, ¿es $\neg P_{1,2}$ implicado por nuestro *BC*? Nuestro primer algoritmo para la inferencia es un enfoque de comprobación de modelos que es una implementación directa de la definición de implicación: enumerar los modelos, y comprobar que α es verdadero en cada modelo en el que *BC* es verdadero. Los modelos son asignaciones de *verdadero* o *falso* a cada símbolo de proposición. Volviendo a nuestro ejemplo del mundo del wumpus, los símbolos de proposición relevantes son $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, y $P_{3,1}$. Con siete símbolos, hay $2^7 = 128$ modelos posibles; en tres de estos, *BC* es verdadero (Figura 7.9). En esos tres modelos, $\neg P_{1,2}$ es verdadero, por lo tanto no hay pozo en [1,2]. Por otro lado, $P_{2,2}$ es verdadero en dos de los tres modelos y falso en uno, por lo que aún no podemos decir si hay un pozo en [2,2].

La Figura 7.9 reproduce de una forma más precisa el razonamiento ilustrado en la Figura 7.5. Un algoritmo general para decidir la implicación en la lógica proposicional se muestra en la

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	BC
falso	*falso*	*falso*	*falso*	*falso*	*falso*	*falso*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*falso*	*falso*
falso	*falso*	*falso*	*falso*	*falso*	*falso*	*verdadero*	*verdadero*	*verdadero*	*falso*	*verdadero*	*falso*	*falso*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
falso	*verdadero*	*falso*	*falso*	*falso*	*falso*	*falso*	*verdadero*	*verdadero*	*falso*	*verdadero*	*verdadero*	*falso*
falso	*verdadero*	*falso*	*falso*	*falso*	*falso*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*
falso	*verdadero*	*falso*	*falso*	*falso*	*verdadero*	*falso*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*
falso	*verdadero*	*falso*	*falso*	*falso*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*
falso	*verdadero*	*falso*	*falso*	*verdadero*	*falso*	*falso*	*verdadero*	*falso*	*falso*	*verdadero*	*verdadero*	*falso*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
verdadero	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*verdadero*	*falso*	*verdadero*	*verdadero*	*falso*	*verdadero*	*falso*

Figura 7.9 Una tabla de verdad construida para la BC dada en el texto. La BC es verdadera si R_1 a R_5 son verdaderas, lo cual ocurre en sólo 3 de las 128 filas (las subrayadas en la columna de la derecha). En todas las 3 filas, $P_{1,2}$ es falsa, por lo que no hay pozo en [1,2]. Por otro lado, podría haber (o no) un pozo en [2,2].

```
function TT-ENTAILS?(KB,α) returns true o false
    inputs: KB, la base de conocimiento, una sentencia en lógica proposicional
            α, la consulta, una sentencia en lógica proposicional

    symbols← una lista de símbolos proposicionales en la BC y α
    return TT-CHECK-ALL(KB,α, symbols, {})

function TT-CHECK-ALL(KB,α, symbols, model)returns true o false
    if EMPTY?(symbols) then
        if PL-TRUE?(KB,model) then return PL-TRUE?(α,model)
        else return true    // cuando la KB es falsa, siempre retorna verdadero
    else
        P←FIRST(symbols)
        rest←REST(symbols)
        return (TT-CHECK-ALL(KB,α, rest, model ∪ {P = true})
                and
                TT-CHECK-ALL(KB,α, rest, model ∪ {P = false}))
```

Figura 7.10 Un algoritmo de enumeración de tabla de verdad para decidir la implicación proposicional. (TT significa tabla de verdad). PL-True? devuelve *true* si una sentencia se verifica dentro de un modelo. La variable *model* representa un modelo parcial, una asignación a algunos de los símbolos. La palabra clave **and** aquí es un símbolo de función de infijo en el lenguaje de programación pseudocódigo, no un operador en lógica de proposicional; toma dos argumentos y devuelve *true* o *false*.

Figura 7.10. Al igual que el algoritmo de BÚSQUEDA-RETROACTIVA en la página 192, TT-ENTAILS? realiza una enumeración recursiva de un espacio finito de asignaciones a símbolos. El algoritmo es **sólido** porque implementa directamente la definición de implicación, y completo porque funciona para cualquier *BC* y α y siempre termina: sólo hay un número finito de modelos para examinar.

Por supuesto, "número finito" no siempre es lo mismo que "pocos". Si la *BC* y α contienen n símbolos en total, entonces hay 2^n modelos. Entonces, la complejidad temporal del algoritmo es $O(2^n)$. (La complejidad espacial es sólo $O(n)$ porque la enumeración es en profundidad). Más adelante en el capítulo mostramos algoritmos mucho más eficientes en muchos casos. Desafortunadamente, la implicación proposicional es co-NP-completa (es decir, probablemente no más fácil que NP-completa, ver Apéndice A), *por lo que cada algoritmo de inferencia conocido para la lógica proposicional tiene una complejidad de peor caso que es exponencial en el tamaño de la entrada.*

7.5 Prueba de teoremas proposicionales

Hasta ahora, hemos mostrado cómo determinar la implicación mediante la verificación de modelos: enumerando modelos y mostrando que la sentencia debe satisfacerse en todos los modelos. En esta sección, mostramos cómo la implicación puede hacerse mediante la **prueba de teoremas**, aplicando reglas de inferencia directamente a las sentencias en nuestra BC para construir una prueba de la sentencia deseada sin consultar modelos. Si el número de modelos es grande pero la longitud de la prueba es corta, entonces la prueba de teoremas puede ser más eficiente que la verificación de modelos.

Prueba de teoremas

Antes de sumergirnos en los detalles de los algoritmos de prueba de teoremas, necesitaremos algunos conceptos adicionales relacionados con la implicación. El primer concepto es la **equivalencia lógica**: dos sentencias α y β son lógicamente equivalentes si son verdaderas en el mismo conjunto de modelos. Escribimos esto como $\alpha \equiv \beta$. (Note que $\equiv$ se usa para hacer afirmaciones sobre sentencias, mientras que $\Leftrightarrow$ se usa como parte de una sentencia). Por ejemplo, podemos mostrar fácilmente (usando tablas de verdad) que $P \wedge Q$ y $Q \wedge P$ son lógicamente equivalentes; otras equivalencias se muestran en la Figura 7.11. Estas equivalencias desempeñan mucho el mismo papel en la lógica que las identidades aritméticas en las matemáticas ordinarias. Una definición alternativa de equivalencia es la siguiente: cualquier dos sentencias α y β son equivalentes si y sólo si cada una de ellas implica a la otra:

Equivalencia lógica

$$\alpha \equiv \beta \quad \text{si y sólo si} \quad \alpha \models \beta \text{ y } \beta \models \alpha.$$

El segundo concepto que necesitaremos es **validez**. Una sentencia es válida si es verdadera en *todos* los modelos. Por ejemplo, la sentencia $P \vee \neg P$ es válida. Las sentencias válidas también se conocen como **tautologías**, es decir, que son *necesariamente* verdaderas. Debido a que la sentencia *True* es verdadera en todos los modelos, cada sentencia válida es lógicamente equivalente a *True*. ¿Qué son las sentencias válidas? A partir de nuestra definición de implicación, podemos derivar el **teorema de deducción**, conocido por los antiguos griegos como:

Validez

Tautología

Teorema de deducción

Para cada sentences α y β, $\alpha \models \beta$ *si y sólo* si la *sentencia* $(\alpha \Rightarrow \beta)$ es *válida*.

(El ejercicio 7.DEDU pide una prueba). Por lo tanto, podemos decidir si $\alpha \models \beta$ comprobando que $(\alpha \Rightarrow \beta)$ es verdadera en cada modelo, que es esencialmente lo que el algoritmo de inferencia en la Figura 7.10 hace, o demostrando que $(\alpha \Rightarrow \beta)$ es equivalente a *Verdadero*. A la inversa, el teorema de deducción establece que cada sentencia de implicación válida describe una inferencia legítima.

Satisfacibilidad

El concepto final que necesitaremos es **satisfacibilidad**. Una sentencia es satisfacible o satisfactoria si es verdadera en, o satisfecha por, *algún* modelo. Por ejemplo, la BC dada antes, $(R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5)$, es satisfacible porque hay tres modelos en los que es verdadera, como se muestra en la Figura 7.9. La satisfacibilidad se puede comprobar enumerando los posibles modelos hasta que uno es encontrado que satisface la sentencia. El problema de determinar la satisfacibilidad de

$$
\begin{array}{rcll}
(\alpha \wedge \beta) & \equiv & (\beta \wedge \alpha) & \text{conmutatividad de } \wedge \\
(\alpha \vee \beta) & \equiv & (\beta \vee \alpha) & \text{conmutatividad de } \vee \\
((\alpha \wedge \beta) \wedge \gamma) & \equiv & (\alpha \wedge (\beta \wedge \gamma)) & \text{asociatividad de } \wedge \\
((\alpha \vee \beta) \vee \gamma) & \equiv & (\alpha \vee (\beta \vee \gamma)) & \text{asociatividad de } \vee \\
\neg(\neg\alpha) & \equiv & \alpha & \text{doble negación eliminación} \\
(\alpha \Rightarrow \beta) & \equiv & (\neg\beta \Rightarrow \neg\alpha) & \text{contraposición} \\
(\alpha \Rightarrow \beta) & \equiv & (\neg\alpha \vee \beta) & \text{eliminación de implicación} \\
(\alpha \Leftrightarrow \beta) & \equiv & ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) & \text{eliminación de bicondicional} \\
\neg(\alpha \wedge \beta) & \equiv & (\neg\alpha \vee \neg\beta) & \text{De Morgan} \\
\neg(\alpha \vee \beta) & \equiv & (\neg\alpha \wedge \neg\beta) & \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) & \equiv & ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) & \text{distributividad de } \wedge \text{ sobre } \vee \\
(\alpha \vee (\beta \wedge \gamma)) & \equiv & ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) & \text{distributividad de } \vee \text{ sobre } \wedge
\end{array}
$$

Figura 7.11 Equivalencias lógicas estándar. Los símbolos α, β, y γ representan sentencia arbitrarias de lógica proposicional.

las sentencias en lógica proposicional, conocido como el problema **SAT**, fue el primer problema demostrado ser NP-completo. Muchos problemas en ciencias de la computación son realmente problemas de satisfacibilidad. Por ejemplo, todos los problemas de satisfacción de restricciones en el Capítulo 6 preguntan si las restricciones son satisfacibles por alguna asignación.

SAT

La validez y la satisfacción están, por supuesto, conectadas: α es válido si y sólo si $\neg\alpha$ es insatisfacible; contrapositivamente, α es satisfacible si y sólo si $\neg\alpha$ no es válido. También tenemos el siguiente resultado útil:

$\alpha \models \beta$ *si y sólo si la sentencia* $(\alpha \wedge \neg\beta)$ *es insatisfacible.*

Demostrar β a partir de α comprobando la insatisfacibilidad de $(\alpha \wedge \neg\beta)$ corresponde exactamente a la técnica de prueba matemática estándar de *reductio ad absurdum* (literalmente, "reducción al absurdo"). También se llama prueba por **refutación** o prueba por **contradicción**. Se asume que una sentencia β es falsa y se muestra que esto lleva a una contradicción con los axiomas conocidos α. Esta contradicción es exactamente lo que se entiende al decir que la sentencia $(\alpha \wedge \neg\beta)$ es insatisfacible.

Reductio ad absurdum
Refutación
Contradicción

7.5.1 Inferencia y pruebas

Esta sección cubre **reglas de inferencia** que se pueden aplicar para derivar una **prueba**, una cadena de conclusiones que conducen al objetivo deseado. La regla más conocida se llama **Modus Ponens** (latín para *modo que afirma*) y se escribe

Reglas de inferencia
Prueba
Modus Ponens

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

La notación significa que, siempre que se den sentencias de la forma $\alpha \Rightarrow \beta$ y α, entonces se puede inferir la sentencia β. Por ejemplo, si se dan $(\mathit{WumpusEnFrente} \wedge \mathit{WumpusVivo}) \Rightarrow \mathit{Disparar}$ y $(\mathit{WumpusEnFrente} \wedge \mathit{WumpusVivo})$, entonces se puede inferir *Disparar*.

Otra regla de inferencia útil es la **Eliminación-**$\wedge$, que dice que, de una conjunción, cualquiera de los conjuntores puede ser inferido:

Eliminación-$\wedge$

$$\frac{\alpha \wedge \beta}{\alpha}.$$

Por ejemplo, de $(\mathit{WumpusEnFrente} \wedge \mathit{WumpusVivo})$, se puede inferir *WumpusVivo*.

Al considerar los posibles valores de verdad de α y β, se puede demostrar fácilmente de una vez por todas que Modus Ponens y Elimination-$\wedge$ son sólidos. Estas reglas pueden entonces ser utilizadas en cualquier instancia particular donde se apliquen, generando inferencias sólidas sin la necesidad de enumerar modelos.

Todas las equivalencias lógicas en la Figura 7.11 pueden ser utilizadas como reglas de inferencia. Por ejemplo, la equivalencia para la eliminación bicondicional produce las dos reglas de inferencia

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{y} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

No todas las reglas de inferencia funcionan en ambas direcciones de esta manera. Por ejemplo, no podemos ejecutar Modus Ponens en la dirección opuesta para obtener $\alpha \Rightarrow \beta$ y α de β.

Veamos cómo estas reglas de inferencia y equivalencias se pueden usar en el mundo del wumpus. Comenzamos con la BC que contiene R_1 a R_5 y mostramos cómo probar $\neg P_{1,2}$; es decir, no hay un pozo en [1,2]:

1. Aplica la eliminación bicondicional a R_2 para obtener

   R_6: $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$.

2. Aplica Elimination-$\wedge$ a R_6 para obtener

   R_7: $((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$.

3. La equivalencia lógica para contrapositivas da

   R_8: $(\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$.

4. Aplica Modus Ponens con R_8 y el percepto R_4 (es decir, $\neg B_{1,1}$), para obtener

   R_9: $\neg(P_{1,2} \vee P_{2,1})$.

5. Aplica la regla de Morgan, dando la conclusión

   R_{10}: $\neg P_{1,2} \wedge \neg P_{2,1}$.

   Es decir, ni [1,2] ni [2,1] contienen un pozo.

Cualquiera de los algoritmos de búsqueda en el Capítulo 3 se puede usar para encontrar una secuencia de pasos que constituya una prueba como esta. Sólo necesitamos definir un problema de prueba de la siguiente manera:

- Estado Inicial: la BC inicial.
- Acciones: el conjunto de acciones consiste en todas las reglas de inferencia aplicadas a todas las sentencias que coinciden con la mitad superior de la regla de inferencia.
- Resultado: el resultado de una acción es agregar la sentencia en la mitad inferior de la regla de inferencia.
- Meta: la meta es un estado que contiene la sentencia que estamos tratando de probar.

Por lo tanto, buscar pruebas es una alternativa a enumerar modelos. En muchos casos prácticos, *encontrar una prueba puede ser más eficiente porque la prueba puede ignorar proposiciones irrelevantes, sin importar cuántas haya*. Por ejemplo, la prueba recién dada que lleva a $\neg P_{1,2} \wedge \neg P_{2,1}$ no menciona las proposiciones $B_{2,1}$, $P_{1,1}$, $P_{2,2}$, o $P_{3,1}$. Se pueden ignorar porque la proposición objetivo, $P_{1,2}$, aparece sólo en la sentencia R_2; las otras proposiciones en R_2 aparecen sólo en R_4 y R_2; por lo tanto, R_1, R_3 y R_5 no tienen relación con la prueba. Lo mismo sería cierto incluso si agregáramos un millón de sentencias más a la BC; el simple algoritmo de tabla de verdad, por otro lado, se vería abrumado por la explosión exponencial de modelos.

Una propiedad final de los sistemas lógicos es la **monotonía**, que dice que el conjunto de las sentencias detalladas sólo pueden aumentar a medida que se agrega información a la BC.[9] Para cualquier sentencia α y β, Monotonía

si $BC \models \alpha$ entonces $BC \wedge \beta \models \alpha$.

Por ejemplo, suponga que la BC contiene la afirmación adicional β que indica que hay exactamente ocho pozos en el mundo. Este conocimiento ayudaría al agente a sacar conclusiones *adicionales*, pero no puede invalidar ninguna conclusión α ya inferida, como la conclusión de que no hay un pozo en [1,2]. La monotonía significa que las reglas de inferencia se aplican siempre que se encuentren premisas adecuadas en la BC, la conclusión de la regla debe seguir *independientemente de lo que más esté en la BC*.

7.5.2 Prueba por resolución

Hemos argumentado que las reglas de inferencia cubiertas hasta ahora son *sólidas*, pero no hemos discutido la cuestión de la *integridad* de los algoritmos de inferencia que las usan. Los algoritmos de búsqueda como la búsqueda en profundidad iterativa (página 81) son completos en el sentido de que encontrarán cualquier objetivo alcanzable, pero si las reglas de inferencia disponibles son inadecuadas, entonces el objetivo no es alcanzable, no existe ninguna prueba que sólo use esas reglas de inferencia. Por ejemplo, si eliminamos la regla de eliminación bicondicional, la prueba en la sección anterior no pasaría. La sección actual presenta una única regla de inferencia, **resolución**, que produce un algoritmo de inferencia completo cuando se combina con cualquier algoritmo de búsqueda completo.

Comenzamos usando una versión simple de la regla de resolución en el mundo del wumpus. Consideremos los pasos que llevan a la Figura 7.4(a): el agente regresa de [2,1] a [1,1] y luego va a [1,2], donde percibe un hedor, pero no una brisa. Agregamos los siguientes hechos a la base de conocimiento:

R_{11}: $\neg B_{1,2}$.
R_{12}: $B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$.

Por el mismo proceso que condujo a R_{10} anteriormente, ahora podemos derivar la ausencia de pozos en [2,2] y [1,3] (recuerde que ya se sabe que [1,1] no tiene pozos):

R_{13}: $\neg P_{2,2}$.
R_{14}: $\neg P_{1,3}$.

También podemos aplicar la eliminación bicondicional a R_3, seguida de Modus Ponens con R_5, para obtener el hecho de que hay un pozo en [1,1], [2,2] o [3,1]:

R_{15}: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$.

Ahora viene la primera aplicación de la regla de resolución: el literal $\neg P_{2,2}$ en R_{13} se resuelve con el literal $P_{2,2}$ en R_{15} para dar el **resolvente** Resolvente

R_{16}: $P_{1,1} \vee P_{3,1}$.

En español; si hay un pozo en alguna de las casillas [1,1], [2,2] o [3,1] y no está en [2,2], entonces está en [1,1] o [3,1]. De manera similar, el literal $\neg P_{1,1}$ en R_1 se resuelve con el literal $P_{1,1}$ en R_{16} para dar

R_{17}: $P_{3,1}$.

En español: si hay un pozo en [1,1] o [3,1] y no está en [1,1], entonces está en [3,1]. Estos últimos dos pasos de inferencia son ejemplos de la regla de inferencia de **resolución unitaria** Resolución unitaria

[9] Las lógicas **no monótonas**, que violan la propiedad de monotonía, capturan una propiedad común del razonamiento humano: cambiar de opinión. Se discuten en la Sección 10.6.

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

Literales complementarios
Cláusula
Cláusula unitaria
Resolución

donde cada ℓ es un literal y ℓ_i y m son **literales complementarios** (es decir, uno es la negación del otro). Así, la regla de resolución unitaria toma una **cláusula**, una disyunción de literales, y un literal y produce una nueva cláusula. Note que un solo literal puede ser visto como una disyunción de un literal, también conocido como **cláusula unitaria**.

La regla de resolución unitaria puede generalizarse a la regla de **resolución** completa

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

donde ℓ_i y m_j son literales complementarios. Esto dice que la resolución toma dos cláusulas y produce una nueva cláusula que contiene todos los literales de las dos cláusulas originales excepto los dos literales complementarios. Por ejemplo, tenemos

$$\frac{P_{1,1} \vee P_{3,1}, \qquad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

Sólo puede resolver un par de literales complementarios a la vez. Por ejemplo, podemos resolver P y $\neg P$ para deducir

$$\frac{P \vee \neg Q \vee R, \qquad \neg P \vee Q}{\neg Q \vee Q \vee R}$$

pero no puede resolver en P y Q a la vez para inferir R. Hay un aspecto técnico más de la regla de resolución: la cláusula resultante debe contener sólo una copia de cada literal.[10] La eliminación de múltiples copias de literales se llama **factorización**. Por ejemplo, si resolvemos $(A \vee B)$ con $(A \vee \neg B)$, obtenemos $(A \vee A)$, que se reduce a sólo A por factorización.

Factorización

La *solidez* de la regla de resolución puede verse fácilmente considerando el literal ℓ_i que es complementario al literal m_j en la otra cláusula. Si ℓ_i es verdadero, entonces m_j es falso y, por lo tanto, $m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n$ debe ser verdadero, porque se da $m_1 \vee \cdots \vee m_n$. Si ℓ_i es falso, entonces $\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k$ debe ser verdadero porque se da $\ell_1 \vee \cdots \vee \ell_k$. Ahora ℓ_i es verdadero o falso, por lo que una u otra de estas conclusiones se mantiene, exactamente como lo indica la regla de resolución.

Lo que es más sorprendente acerca de la regla de resolución es que forma la base para una familia de procedimientos de inferencia completos. *Un demostrador de teoremas basado en resolución puede decidir, para cualquier sentencia α y β en lógica proposicional, si $\alpha \models \beta$.* Las dos siguientes subsecciones explican cómo la resolución logra esto.

Forma normal conjuntiva

La regla de resolución se aplica sólo a las cláusulas (es decir, disyunciones de literales), por lo que parecería ser relevante sólo para las BC y las consultas que consisten en cláusulas. ¿Cómo, entonces, puede conducir a un procedimiento de inferencia completo para toda la lógica proposicional? La respuesta es que *cada sentencia de la lógica proposicional es lógicamente equivalente a una conjunción de cláusulas.*

Forma normal conjuntiva o FNC

Una sentencia expresada como una conjunción de cláusulas se dice que está en **forma normal conjuntiva** o **FNC** (ver Figura 7.12). Ahora describimos un procedimiento para convertir a FNC. Ilustramos el procedimiento convirtiendo la sentencia $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ en FNC. Los pasos son los siguientes:

[10] Si una cláusula se ve como un *conjunto* de literales, entonces esta restricción se respeta automáticamente. El uso de la notación de conjunto para las cláusulas hace que la regla de resolución sea mucho más limpia, a costa de introducir notación adicional.

$$
\begin{array}{rcl}
\textit{SentenciaFNC} & \rightarrow & \textit{Cláusula}_1 \wedge \cdots \wedge \textit{Cláusula}_n \\
\textit{Cláusula} & \rightarrow & \textit{Literal}_1 \vee \cdots \vee \textit{Literal}_m \\
\textit{Hecho} & \rightarrow & \textit{Símbolo} \\
\textit{Literal} & \rightarrow & \textit{Símbolo} \mid \neg \text{ Símbolo} \\
\textit{Símbolo} & \rightarrow & P \mid Q \mid R \mid \ldots \\
\textit{CláusulaFormaHorn} & \rightarrow & \textit{CláusulaFormaDefinida} \mid \textit{CláusulaFormaObjectivo} \\
\textit{CláusulaFormaDefinida} & \rightarrow & \textit{Hecho} \mid (\textit{Símbolo}_1 \wedge \cdots \wedge \textit{Símbolo}_1) \Rightarrow \textit{Símbolo} \\
\textit{CláusulaFormaObjectivo} & \rightarrow & (\textit{Símbolo}_1 \wedge \cdots \wedge \textit{Símbolo}_1) \Rightarrow \text{Falso}
\end{array}
$$

Figura 7.12 Una gramática para la forma normal conjuntiva, cláusulas de Horn y cláusulas definidas. Una cláusula FNC como $\neg A \vee \neg B \vee C$ puede escribirse en forma de cláusula definida como $A \wedge B \Rightarrow C$.

1. Elimine $\Leftrightarrow$, reemplazando $\alpha \Leftrightarrow \beta$ con $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

   $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$.

2. Elimine $\Rightarrow$, reemplazando $\alpha \Rightarrow \beta$ con $\neg\alpha \vee \beta$:

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$.

3. FNC requiere que $\neg$ aparezca sólo en literales, por lo que "movemos $\neg$ hacia adentro" mediante la aplicación repetida de las siguientes equivalencias de la Figura 7.11:

   $\neg(\neg\alpha) \equiv \alpha$ (eliminación de doble negación)
   $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ (De Morgan)
   $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ (De Morgan)

   En el ejemplo, requerimos sólo una aplicación de la última regla:

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$.

4. Ahora tenemos una sentencia que contiene operadores $\wedge$ y $\vee$ anidados aplicados a literales. Aplicamos la ley distributiva de la Figura 7.11, distribuyendo $\vee$ sobre $\wedge$ donde sea posible.

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B1,1)$.

La sentencia original ahora está en FNC, como una conjunción de tres cláusulas. Es mucho más difícil de leer, pero se puede usar como entrada para un procedimiento de resolución.

Un algoritmo de resolución

Los procedimientos de inferencia basados en la resolución funcionan utilizando el principio de prueba por contradicción introducido en la página 223. Es decir, para mostrar que BC $\models \alpha$, mostramos que $(BC \wedge \neg\alpha)$ es insatisfacible. Hacemos esto demostrando una contradicción.

Se muestra un algoritmo de resolución en la Figura 7.13. Primero, $(BC \wedge \neg\alpha)$ se convierte en FNC. Luego, se aplica la regla de resolución a las cláusulas resultantes. Cada par que contiene literales complementarios se resuelve para producir una nueva cláusula, que se agrega al conjunto si aún no está presente. El proceso continúa hasta que suceda una de dos cosas:

- no hay nuevas cláusulas que se puedan agregar, en cuyo caso BC no implica α; o,
- dos cláusulas se resuelven para producir la cláusula *vacía*, en cuyo caso BC implica α.

```
function PL-RESOLUTION(BC,α) returns true o false
  inputs: BC, la BC, una sentencia en lógica proposicional
          α, la consulta, una sentencia en lógica proposicional

  clauses←un conjunto de cláusulas en la representación de FNC para BC ∧ ¬α
  new←{}
  while true do
    for each par de cláusulas C_i, C_j in clauses do
      resolvents←PL-RESOLVE(C_i,C_j)
      if resolvents contiene la cláusula vacía then return true
      new←new ∪ resolvents
    if new ⊆ clauses then return false
    clauses←clauses ∪ new
```

Figura 7.13 Un simple algoritmo de resolución para la lógica proposicional. PL-RESOLVE devuelve el conjunto de todas las cláusulas posibles obtenidas al resolver sus dos entradas.

La cláusula vacía, una disyunción sin disyuntores, es equivalente a *Falso* porque una disyunción es verdadera sólo si al menos uno de sus disyuntos es verdadero. Además, la cláusula vacía surge sólo de la resolución de dos cláusulas unitarias contradictorias como P y $\neg P$.

Podemos aplicar el procedimiento de resolución a una inferencia muy simple en el mundo del wumpus. Cuando el agente está en [1,1], no hay brisa, por lo que no puede haber pozos en las casillas vecinas. La *BC* relevante es

$$BC = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

y deseamos demostrar α, que es, digamos, $\neg P_{1,2}$. Cuando convertimos $(BC \wedge \neg\alpha)$ en FNC, obtenemos las cláusulas que se muestran en la parte superior de la Figura 7.14. La segunda fila de la figura muestra cláusulas obtenidas al resolver pares en la primera fila. Luego, cuando $P_{1,2}$ se resuelve con $\neg P_{1,2}$, obtenemos la cláusula vacía, mostrada como una pequeña casilla. La inspección de la Figura 7.14 revela que muchos pasos de resolución son inútiles. Por ejemplo, la cláusula $B_{1,1} \vee \neg B_{1,1} \vee P_{1,2}$ es equivalente a *Verdadero* $\vee P_{1,2}$ que es equivalente a *Verdadero*. Deducir que *Verdadero* es verdadero no es muy útil. Por lo tanto, cualquier cláusula en la que aparezcan dos literales complementarios puede ser descartada.

Completitud de la resolución

Cierre de resolución

Para concluir nuestra discusión sobre la resolución, ahora mostramos por qué PL-RESOLUTION es completa. Para hacer esto, introducimos el **cierre de resolución** $CR(S)$ de un conjunto de cláusulas S, que es el conjunto de todas las cláusulas derivables mediante la aplicación repetida de la regla de resolución a las cláusulas en S o sus derivados. El cierre de resolución es lo que PL-RESOLUTION calcula como el valor final de las *cláusulas* variables. Es fácil ver que $CR(S)$ debe ser finito: gracias al paso de factorización, sólo hay un número finito de cláusulas distintas que se pueden construir a partir de los símbolos $P_1,\ldots, P_k$ que aparecen en S. Por lo tanto, PL-RESOLUTION siempre termina.

Teorema fundamental de la resolución

El teorema de completitud para la resolución en lógica proposicional se llama **teorema fundamental de la resolución**:

> Si un conjunto de cláusulas es insatisfacible, entonces el cierre de resolución de esas cláusulas contiene la cláusula vacía.

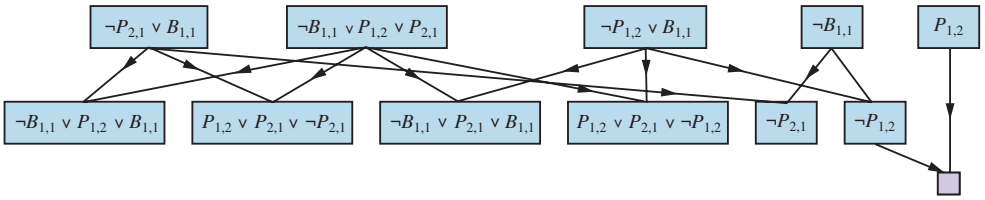


Figura 7.14 Aplicación parcial de PL-RESOLUTION a una inferencia simple en el mundo del wumpus para probar la consulta $\neg P_{1,2}$. Cada una de las cuatro cláusulas más a la izquierda en la fila superior se empareja con cada una de las otras tres, y se aplica la regla de resolución para obtener las cláusulas en la fila inferior. Vemos que la tercera y cuarta cláusulas en la fila superior se combinan para producir la cláusula $\neg P_{1,2}$, que luego se resuelve con $P_{1,2}$ para producir la cláusula vacía, lo que significa que la consulta está probada.

Este teorema se demuestra exponiendo su contrapositiva: si el cierre $CR(S)$ no contiene la cláusula vacía, entonces S es satisfactorio. De hecho, podemos construir un modelo para S con valores de verdad adecuados para $P_1,..., P_k$. El procedimiento de construcción es el siguiente:

Para i de 1 a k,
- Si una cláusula en $CR(S)$ contiene el literal $\neg P_i$ y todos sus otros literales son falsos bajo la asignación elegida para $P_1,..., P_{i-1}$, entonces asigna *falso* a P_i.
- De lo contrario, asigna *verdadero* a P_i.

Esta asignación a $P_1,..., P_k$ es un modelo de S. Para ver esto, asumamos lo contrario, que, en alguna etapa i en la secuencia, asignar el símbolo P_i hace que alguna cláusula C se vuelva falsa. Para que esto suceda, debe ser el caso que todos los *otros* literales en C ya deben haber sido falsificados por asignaciones a $P_1,..., P_{i-1}$. Por lo tanto, C ahora debe parecerse a ($falso \vee falso \vee \cdots falso \vee P_i$) o como ($falso \vee falso \vee falso \vee \neg P_i$). Si sólo uno de estos dos está en $CR(S)$, entonces el algoritmo asignará el valor de verdad apropiado a P_i para hacer que C sea verdadero, por lo que C sólo puede ser falsificado si *ambas* de estas cláusulas están en $CR(S)$.

Ahora, dado que $CR(S)$ está cerrado bajo resolución, contendrá el resolvente de estas dos cláusulas, y ese resolvente tendrá todos sus literales ya falsificados por las asignaciones a $P_1,..., P_{i-1}$. Esto contradice nuestra suposición de que la primera cláusula falsificada aparece en la etapa i. Por lo tanto, hemos demostrado que la construcción nunca falsifica una cláusula en $CR(S)$; es decir, produce un modelo de $CR(S)$. Finalmente, porque S está contenido en $CR(S)$, cualquier modelo de $CR(S)$ es un modelo de S mismo.

7.5.3 Cláusulas de Horn y cláusulas definidas

La completitud de la resolución la convierte en un método de inferencia muy importante. Sin embargo, en muchas situaciones prácticas, no se necesita todo el poder de la resolución. Algunas BC del mundo real satisfacen ciertas restricciones en la forma de las sentencias que contienen, lo que les permite utilizar un algoritmo de inferencia más restringido y eficiente.

Una de estas formas restringidas es la **cláusula definida**, que es una disyunción de literales de los cuales *exactamente uno es positivo*. Por ejemplo, la cláusula ($\neg L_{1,1} \vee \neg Breeze \vee B_{1,1}$) es una cláusula definida, mientras que ($\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$) no lo es, porque tiene dos cláusulas positivas. Cláusula definida

Ligeramente más general es la **cláusula de Horn**, que es una disyunción de literales de los cuales *a lo sumo uno es positivo*. Por lo tanto, todas las cláusulas definidas son cláusulas de Cláusula de Horn

Cláusulas de objetivo

Horn, al igual que las cláusulas sin literales positivos; a éstas se les llama **cláusulas de objetivo**. Las cláusulas de Horn están cerradas bajo resolución: si resuelves dos cláusulas de Horn, obtienes una cláusula de Horn. Una clase más es la sentencia k-FNC, que es una sentencia FNC donde cada cláusula tiene a lo sumo k literales.

Las BC que contienen sólo cláusulas definidas son interesantes por tres razones:

1. Cada cláusula definida se puede escribir como una implicación cuya premisa es una conjunción de literales positivos y cuya conclusión es un solo literal positivo. (Ver Ejercicio 7.DISJ). Por ejemplo, la cláusula definida $(\neg L_{1,1} \vee \neg \mathit{Brisa} \vee B_{1,1})$ se puede escribir como la implicación $(L_{1,1} \wedge \mathit{Brisa}) \Rightarrow B_{1,1}$. En la forma de implicación, la sentencia es más fácil de entender: dice que si el agente está en [1,1] y hay una percepción de brisa, entonces [1,1] es ventoso. En forma de Horn, la premisa se llama **cuerpo** y la conclusión se llama **cabeza**. Una sentencia que consta de un solo literal positivo, como $L_{1,1}$, se llama **hecho**. También puede ser escrito en forma de implicación como $\mathit{True} \Rightarrow L_{1,1}$, pero es más sencillo escribir solo $L_{1,1}$.

Cuerpo

Cabeza

Hecho

2. La inferencia con cláusulas de Horn se puede realizar a través de los algoritmos de **encadenamiento hacia adelante** y **encadenamiento hacia atrás**, que explicamos a continuación. Ambos algoritmos son naturales, en el sentido de que los pasos de inferencia son obvios y fáciles de seguir para los humanos. Este tipo de inferencia es la base de la **programación lógica**, que se discute en el Capítulo 9.

Encadenamiento hacia adelante

Encadenamiento hacia atrás

3. Decidir la implicación con cláusulas de Horn se puede hacer en un tiempo que es *lineal* en el tamaño de la BC, una sorpresa agradable.

7.5.4 Encadenamiento hacia adelante y hacia atrás

El algoritmo de encadenamiento hacia adelante PL-FC-ENTAILS?(BC, q) determina si un solo símbolo de proposición q, la consulta, está implicado por una BC de cláusulas definidas. Comienza a partir de hechos conocidos (literales positivos) en la BC. Si todas las premisas de una implicación son conocidas, entonces su conclusión se agrega al conjunto de hechos conocidos. Por ejemplo, si $L_{1,1}$ y *Brisa* son conocidos y $(L_{1,1} \wedge \mathit{Brisa}) \Rightarrow B_{1,1}$ está en la BC, entonces $B_{1,1}$ se puede agregar. Este proceso continúa hasta que se agrega la consulta q o hasta que no se pueden hacer más inferencias. El algoritmo se muestra en la Figura 7.15; el punto principal a recordar es que se ejecuta en tiempo lineal.

La mejor manera de entender el algoritmo es a través de un ejemplo y una imagen. La Figura .16(a) muestra una BC simple de cláusulas de Horn con A y B como hechos conocidos. La Figura 7.16(b) muestra la misma BC dibujada como un **grafo AND–OR** (ver Capítulo 4). En los grafos AND–OR, múltiples aristas unidas por un arco indican una conjunción, todas las aristas deben ser probadas, mientras que múltiples aristas sin un arco indican una disyunción, cualquier arista puede ser probada. Es fácil ver cómo funciona el encadenamiento hacia adelante en el grafo. Las hojas conocidas (aquí, A y B) se establecen, y la inferencia se propaga por el grafo tanto como sea posible. Dondequiera que aparezca una conjunción, la propagación espera hasta que todos los conjuntos sean conocidos antes de proceder. Se anima al lector a trabajar en el ejemplo en detalle.

Es fácil ver que el encadenamiento hacia adelante es **sólido**: cada inferencia es esencialmente una aplicación de Modus Ponens. El encadenamiento hacia adelante también es **completo**: se derivará cada sentencia atómica implicada. La forma más fácil de ver esto es considerar el estado final de la tabla *inferida* (después de que el algoritmo alcanza un punto fijo donde no son posibles nuevas inferencias). La tabla contiene *verdadero* para cada símbolo inferido durante el proceso, y *falso* para todos los demás símbolos. Podemos ver la tabla como un modelo lógico; además, *cada cláusula definida en la BC original es verdadera en este modelo.*

```
function PL-FC-ENTAILS?(BC, q) returns true o false
  inputs: BC, la base de conocimiento, un conjunto de cláusulas proposicionales definidas
          q, la consulta, un símbolo de proposición
  count←una tabla donde count[c] es inicialmente el número de símbolos en la premisa de la cláusula c.
  inferred←una tabla, donde inferred[s] es inicialmente falso para todos los símbolos
  queue←una cola de símbolos, inicialmente los símbolos que se saben que son verdaderos en la BC.

  while queue is not empty do
    p←POP(queue)
    if p = q then return true
    if inferred[p] = false then
      inferred[p]←true
      for each clause c in BC where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to queue
return false
```

Figura 7.15 El algoritmo de encadenamiento hacia adelante para la lógica proposicional. La *agenda* mantiene un registro de los símbolos que se sabe que son verdaderos pero aún no están "procesados". La tabla de *conteo* mantiene un registro de cuántas premisas de cada implicación aún no están probadas. Siempre que se procesa un nuevo símbolo p de la agenda, el conteo se reduce en uno para cada implicación en cuya premisa aparece p (fácilmente identificable en tiempo constante con la indexación adecuada). Si un conteo llega a cero, todas las premisas de la implicación son conocidas, por lo que su conclusión puede añadirse a la agenda. Finalmente, necesitamos llevar un registro de qué símbolos han sido procesados; un símbolo que ya está en el conjunto de símbolos inferidos no necesita ser añadido a la agenda de nuevo. Esto evita trabajo redundante y previene bucles causados por implicaciones como $P \Rightarrow Q$ y $Q \Rightarrow P$.

Para ver esto, asumamos lo contrario, es decir, que alguna cláusula $a_1 \wedge ... \wedge a_k \Rightarrow b$ es falsa en el modelo. Entonces $a_1 \wedge ... \wedge a_k$ debe ser verdadero en el modelo y b debe ser falso en el modelo. Pero esto contradice nuestra suposición de que el algoritmo ha alcanzado un punto fijo, porque ahora estaríamos autorizados para añadir b a la *BC*. Podemos concluir, por lo tanto, que el conjunto de sentencias atómicas inferidas en el punto fijo define un modelo de la *BC* original. Además, cualquier sentencia atómica q que es implicada por la *BC* debe ser verdadera en todos sus modelos y en este modelo en particular. Por lo tanto, cada sentencia atómica implicada q debe ser inferida por el algoritmo.

El encadenamiento hacia adelante es un ejemplo del concepto general de razonamiento basado en datos, es decir, **Data-driven;** esto es, el razonamiento en el que el foco de atención comienza con los datos conocidos. Puede ser utilizado dentro de un agente para derivar conclusiones a partir de percepciones entrantes, a menudo sin una consulta específica en mente. Por ejemplo, el agente de wumpus podría decir (TELL) sus percepciones a la BC utilizando un algoritmo de encadenamiento hacia adelante incremental en el que se pueden añadir nuevos hechos a la agenda para iniciar nuevas inferencias. En los humanos, una cierta cantidad de razonamiento basado en datos ocurre a medida que llega nueva información. Por ejemplo, si estoy en el interior y oigo que empieza a llover, podría ocurrírseme que el picnic será cancelado. Sin embargo, probablemente no se me ocurrirá que el decimoséptimo pétalo de la rosa más grande de mi jardín vecino se mojará; los humanos mantienen el encadenamiento hacia adelante bajo control, para no ser inundados con consecuencias irrelevantes.

Data-driven

El algoritmo de encadenamiento hacia atrás, como su nombre sugiere, trabaja hacia atrás desde la consulta. Si la consulta q se sabe que es verdadera, entonces no se necesita trabajo. De lo

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

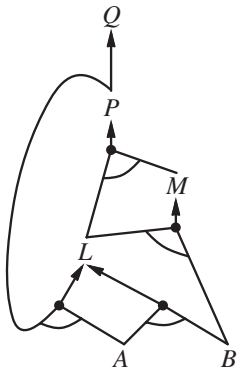
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

(a)



(b)

Figura 7.16 (a) Un conjunto de cláusulas de Horn. (b) El grafo AND–OR correspondiente.

contrario, el algoritmo encuentra aquellas implicaciones en la BC cuya conclusión es q. Si todas las premisas de una de esas implicaciones pueden ser probadas verdaderas (por encadenamiento hacia atrás), entonces q es verdadera. Cuando se aplica a la consulta Q en la Figura 7.16, retrocede por el grafo hasta que llega a un conjunto de hechos conocidos, A y B, que forman la base para una prueba. El algoritmo es esencialmente idéntico al algoritmo de búsqueda en el grafo AND-OR-GRAPH-SEARCH en la Figura 4.11. Al igual que con el encadenamiento hacia adelante, una implementación eficiente se ejecuta en tiempo lineal.

Razonamiento dirigido por objetivos

El encadenamiento hacia atrás es una forma de **razonamiento dirigido por objetivos**. Es útil para responder a preguntas específicas como "¿Qué debo hacer ahora?" y "¿Dónde están mis llaves?" A menudo, el costo del encadenamiento hacia atrás es *mucho menor* que lineal en el tamaño de la BC, porque el proceso sólo involucra a los hechos relevantes.

7.6 Verificación efectiva de modelos proposicionales

En esta sección, describimos dos familias de algoritmos eficientes para la inferencia proposicional general basada en la verificación de modelos: un enfoque basado en la búsqueda de retroceso, y uno en la búsqueda local de ascenso de colina. Estos algoritmos son parte de la "tecnología" de la lógica proposicional. Esta sección puede ser leída rápidamente en una primera lectura del capítulo.

Los algoritmos que describimos son para verificar la satisfacción: el problema SAT. (Como se señaló en la Sección 7.5, la prueba de implicación, $\alpha \models \beta$, se puede hacer probando la *in*satisfacción de $\alpha \wedge \neg\beta$). Mencionamos en la página 223 la conexión entre encontrar un modelo satisfactorio para una sentencia lógica y encontrar una solución para un problema de satisfacción de restricciones, por lo que quizá no sea sorprendente que las dos familias de algoritmos de satisfacción proposicional se parezcan mucho a los algoritmos de retroceso de la Sección 6.3 y los algoritmos de búsqueda local de la Sección 6.4. Sin embargo, son extremadamente importantes por derecho propio porque muchos problemas combinatorios en ciencias de la computación se pueden reducir a verificar la satisfacción de una sentencia proposicional. Cualquier mejora en los algoritmos de satisfacción tiene enormes consecuencias en nuestra capacidad para manejar la complejidad en general.

7.6.1 Un algoritmo de retroceso completo

Algoritmo de Davis–Putnam

El primer algoritmo que consideramos a menudo se llama **algoritmo de Davis–Putnam**, por el artículo de alto impacto de Martin Davis y Hilary Putnam (1960). El algoritmo es de hecho la versión descrita por Davis, Logemann y Loveland (1962), por lo que lo llamaremos DPLL por las iniciales de los cuatro autores. DPLL toma como entrada una sentencia en forma normal conjuntiva, un conjunto de cláusulas. Al igual que BACKTRACKING-SEARCH y TT-ENTAILS?, es esencialmente una enumeración recursiva y en profundidad de posibles modelos. Incorpora tres mejoras sobre el esquema simple de TT- ENTAILS?:

- *Terminación temprana*: El algoritmo detecta si la sentencia debe ser verdadera o falsa, aun con un modelo parcialmente completado. Una cláusula es verdadera si *cualquier* literal es verdadero, incluso si los otros literales aún no tienen valores de verdad; por lo tanto, la sentencia en su conjunto podría ser juzgada como verdadera incluso antes de que el modelo esté completo. Por ejemplo, la sentencia $(A \vee B) \wedge (A \vee C)$ es verdadera si A es verdadera, independientemente de los valores de B y C. De manera similar, una sentencia es falsa si *cualquier* cláusula es falsa, lo que ocurre cuando cada uno de sus literales es falso. Nuevamente, esto puede ocurrir mucho antes de que el modelo esté completo. La terminación temprana evita el examen de subárboles enteros en el espacio de búsqueda.
- *Heurística de símbolo puro*: Un **símbolo puro** es un símbolo que siempre aparece con el mismo "signo" en todas las cláusulas. Por ejemplo, en las tres cláusulas $(A \vee \neg B)$, $(\neg B \vee \neg C)$, y $(C \vee A)$, el símbolo A es puro porque sólo aparece el literal positivo, B es puro porque sólo aparece el literal negativo, y C es impuro. Es fácil ver que si una sentencia tiene un modelo, entonces tiene un modelo con los símbolos puros asignados para hacer que sus literales sean verdaderos, porque hacerlo nunca puede hacer que una cláusula sea falsa. Tenga en cuenta que, al determinar la pureza de un símbolo, el algoritmo puede ignorar las cláusulas que ya se sabe que son verdaderas en el modelo construido hasta ahora. Por ejemplo, si el modelo contiene $B = falso$, entonces la cláusula $(\neg B \vee \neg C)$ ya es verdadera, y en las cláusulas restantes C aparece sólo como un literal positivo; por lo tanto C se vuelve puro. Símbolo puro
- *Heurística de cláusula unitaria*: Una **cláusula unitaria** se definió antes como una cláusula con solo un literal. En el contexto de DPLL, también significa cláusulas en las que todos los literales menos uno ya están asignados como *falsos* por el modelo. Por ejemplo, si el modelo contiene $B = verdadero$, entonces $(\neg B \vee \neg C)$ se simplifica a $\neg C$, que es una cláusula unitaria. Obviamente, para que esta cláusula sea verdadera, C debe establecerse en *falso*. La heurística de cláusula unitaria asigna todos estos símbolos antes de ramificar el resto. Una consecuencia importante de la heurística es que cualquier intento de probar (por refutación) un literal que ya está en la BC tendrá éxito inmediatamente (Ejercicio 7.KNOW). También observe que asignar una cláusula unitaria puede crear otra cláusula unitaria; por ejemplo, cuando C se establece en *falso*, $(C \vee A)$ se convierte en una cláusula unitaria, causando que A deba ser asignado a verdadero. Esta "cascada" de asignaciones forzadas se llama **propagación unitaria**. Se asemeja al proceso de encadenamiento hacia adelante con cláusulas definidas y, de hecho, si la expresión FNC contiene sólo cláusulas definidas, entonces DPLL esencialmente replica el encadenamiento hacia adelante. (Ver Ejercicio *7.DPLL*). Propagación unitaria

El algoritmo DPLL se muestra en la Figura 7.17, que proporciona el esqueleto esencial del proceso de búsqueda sin los detalles de implementación.

Lo que la Figura 7.17 no muestra son los trucos que permiten a los solucionadores SAT escalar a problemas grandes. Es interesante que la mayoría de estos trucos son, de hecho, muy generales, y los hemos visto antes en otras formas:

```
function DPLL-SATISFIABLE?(s) returns true or false
    inputs: s, una sentencia en lógica proposicional

    clauses←the set of clauses in the FNC representation of s
    symbols←una lista de los símbolos de proposición en s
    return DPLL(clauses,symbols, {})

function DPLL(clauses,symbols,model) returns true or false

    if every clause in clauses is true in model then return true
    if some clause in clauses is false in model then return false
    P, value←FIND-PURE-SYMBOL(symbols,clauses,model)
    if P is non-null then return DPLL(clauses,symbols – P,model ∪ {P=value})
    P, value←FIND-UNIT-CLAUSE(clauses,model)
    if P is non-null then return DPLL(clauses,symbols – P,model ∪ {P=value})
    P←FIRST(symbols); rest←REST(symbols)
    return DPLL(clauses,rest,model ∪ {P=true}) or
           DPLL(clauses,rest,model ∪ {P=false}))
```

Figura 7.17 El algoritmo DPLL para verificar la satisfacibilidad de una sentencia en lógica proposicional. Las ideas detrás de FIND-PURE-SYMBOL y FIND-UNIT-CLAUSE se describen en el texto; cada uno devuelve un símbolo (o nulo) y el valor de verdad para asignar a ese símbolo. Al igual que TT-ENTAILS?, DPLL opera sobre modelos parciales.

1. **Análisis de componentes** (como se ve con Tasmania en CSPs): A medida que DPLL asigna valores de verdad a las variables, el conjunto de cláusulas puede separarse en subconjuntos disjuntos, llamados **componentes**, que no comparten variables no asignadas. Dado un método eficiente para detectar cuándo ocurre esto, un solucionador puede ganar velocidad considerable trabajando en cada componente por separado.
2. **Ordenamiento de variables y valores** (como se ve en la Sección 6.3.1 para CSPs): Nuestra sólo implementación de DPLL utiliza un ordenamiento de variables arbitrario y siempre intenta el valor verdadero antes que el falso. La **heurística de grado** (ver página 193) sugiere elegir la variable que aparece con más frecuencia en todas las cláusulas restantes.
3. **Retroceso inteligente** (como se ve en la Sección 6.3.3 para CSPs): Muchos problemas que no pueden resolverse en horas de tiempo de ejecución con retroceso cronológico pueden resolverse en segundos con retroceso inteligente que retrocede hasta el punto de conflicto relevante. Todos los solucionadores SAT que hacen retroceso inteligente usan alguna forma de **aprendizaje de cláusulas de conflicto** para registrar conflictos de manera que no se repitan más adelante en la búsqueda. Por lo general, se mantiene un conjunto de conflictos de tamaño limitado, y se descartan los que se usan raramente.
4. **Reinicios aleatorios** (como se ve en la página 113 para el ascenso de colinas): A veces, una ejecución parece no estar progresando. En este caso, podemos comenzar de nuevo desde la parte superior del árbol de búsqueda, en lugar de intentar continuar. Después de reiniciar, se toman diferentes elecciones aleatorias (en la selección de variables y valores). Las cláusulas que se aprendieron en la primera ejecución se conservan después del reinicio y pueden ayudar a podar el espacio de búsqueda. Reiniciar no garantiza que se encontrará una solución más rápido, pero sí reduce la varianza en el tiempo hasta la solución.
5. **Indexación inteligente** (como se ve en muchos algoritmos): Los métodos de aceleración utilizados en DPLL en sí, así como los trucos utilizados en los solucionadores modernos,

function WALKSAT(*clauses*,*p*,*max_flips*) **returns** un modelo satisfecho o *failure*
  inputs: *clauses*, un conjunto de cláusulas en lógica proposicional *p*, la probabilidad de elegir realizar un movimiento de "caminata aleatoria" típicamente alrededor de 0.5
  max_flips, número de cambios permitidos antes de rendirse

  model←una asignación aleatoria a *true*/*false* para los símbolos en las cláusulas
  for each *i*= 1 **to** *max_flips* **do**
    if *model* satisface *clauses* **then return** *model*
    clause← una cláusula seleccionada al azar de *clauses* que es falsa en el *model*
    if RANDOM(0, 1) ≤ *p* **then**
      voltea el valor en el *model* de un símbolo seleccionado aleatoriamente de *clause*
    else voltea el símbolo de la *clause* que maximiza el número de cláusulas satisfechas
  return *failure*

Figura 7.18 El algoritmo WALKSAT para verificar la satisfacibilidad cambiando aleatoriamente los valores de las variables. Existen muchas versiones del algoritmo.

requieren una indexación rápida de cosas como "el conjunto de cláusulas en las que la variable X_i aparece como un literal positivo". Esta tarea se complica por el hecho de que los algoritmos sólo están interesados en las cláusulas que aún no han sido satisfechas por asignaciones previas a las variables, por lo que las estructuras de indexación deben actualizarse dinámicamente a medida que avanza el cálculo.

Con estas mejoras, los solucionadores modernos pueden manejar problemas con decenas de millones de variables. Han revolucionado áreas como la verificación de hardware y la verificación de protocolos de seguridad, que antes requerían pruebas laboriosas y guiadas a mano.

7.6.2 Algoritmos de búsqueda local

Hemos visto varios algoritmos de búsqueda local hasta ahora en este libro, incluyendo HILL-CLIMBING (página 111) y SIMULATED-ANNEALING (página 115). Estos algoritmos pueden aplicarse directamente a problemas de satisfacción, siempre que elijamos la función de evaluación correcta. Como el objetivo es encontrar una asignación que satisfaga cada cláusula, una función de evaluación que cuente el número de cláusulas insatisfechas hará el trabajo. De hecho, esta es exactamente la medida usada por el algoritmo MIN-CONFLICTS para CSPs (página 198). Todos estos algoritmos dan pasos en el espacio de asignaciones completas, cambiando el valor de verdad de un símbolo a la vez. El espacio suele contener muchos mínimos locales, de los cuales se requieren varias formas de aleatoriedad para escapar. En los últimos años, ha habido una gran cantidad de experimentación para encontrar un buen equilibrio entre la codicia y la aleatoriedad.

Uno de los algoritmos más simples y efectivos que ha surgido de todo este trabajo se llama WALKSAT (Figura 7.18). En cada iteración, el algoritmo elige una cláusula insatisfacible y elige un símbolo en la cláusula para voltear. Elige al azar entre dos formas de elegir qué símbolo voltear: (1) un paso de "miniconflictos" que minimiza el número de cláusulas insatisfechas en el nuevo estado y (2) un paso de "random walk" que elige el símbolo al azar.

Cuando WALKSAT devuelve un modelo, la sentencia de entrada es de hecho satisfacible, pero cuando devuelve un *fracaso*, hay dos posibles causas: o bien la sentencia es insatisfacible o necesitamos darle más tiempo al algoritmo. Si establecemos $max_flips = \infty$ y $p > 0$, WALKSAT eventualmente devolverá

un modelo (si existe uno), porque los pasos de random-walk eventualmente darán con la solución. ¡Ay, si *max_flips* es infinito y la sentencia es insatisfacible, entonces el algoritmo nunca termina!

Por esta razón, WALKSAT es más útil cuando esperamos que exista una solución; por ejemplo, los problemas discutidos en los Capítulos 3 y 6 suelen tener soluciones. Por otro lado, WALKSAT no siempre puede detectar la *insatisfacción*, que es necesaria para decidir la implicación. Es decir, un agente no puede usar WALKSAT de *manera confiable* para demostrar que una casilla es segura en el mundo del wumpus. En cambio, puede decir, "Lo pensé durante una hora y no pude encontrar un mundo posible en el que la casilla *no sea* segura". Esto puede ser un buen indicador empírico de que la casilla es segura, pero ciertamente no es una prueba.

7.6.3 El paisaje de los problemas SAT aleatorios

Algunos problemas SAT son más difíciles que otros. Los problemas *fáciles* pueden ser resueltos por cualquier algoritmo antiguo, pero como sabemos que SAT es NP-completo, al menos algunas instancias de problema deben requerir tiempo de ejecución exponencial. En el Capítulo 6, vimos algunos descubrimientos sorprendentes sobre ciertos tipos de problemas. Por ejemplo, el problema de las *n*-reinas, que se pensaba que era bastante complicado para los algoritmos de búsqueda de retroceso, resultó ser trivialmente fácil para los métodos de búsqueda local, como miniconflictos. Esto se debe a que las soluciones están muy densamente distribuidas en el espacio de asignaciones, y cualquier asignación inicial está garantizada para tener una solución cercana. Así, *n*-reinas es fácil porque está **subrestringida**.

Subrestringido

Cuando miramos los problemas de satisfacción en forma normal conjuntiva, un problema subrestringido es aquel con relativamente *pocas* cláusulas que restringen las variables. Por ejemplo, aquí hay una sentencia 3-FNC generada aleatoriamente con cinco símbolos y cinco cláusulas:

$$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee E) \\ \wedge (E \vee \neg D \vee B) \wedge (B \vee E \vee \neg C).$$

Dieciséis de las 32 posibles asignaciones son modelos de esta sentencia, por lo que, en promedio, tomaría sólo dos conjeturas aleatorias para encontrar un modelo. Este es un problema de satisfacción fácil, al igual que la mayoría de los problemas subrestringidos. Por otro lado, un problema *sobrestringido* tiene muchas cláusulas en relación con el número de variables y es probable que no tenga soluciones. Los problemas sobrestringidos suelen ser fáciles de resolver, porque las restricciones llevan rápidamente a una solución o a un callejón sin salida del que no hay escapatoria.

Para ir más allá de estas intuiciones básicas, debemos definir exactamente cómo se generan las sentencias aleatorias. La notación $FNC_k(m,n)$ denota una sentencia k-FNC con m cláusulas y n símbolos, donde las cláusulas se eligen de manera uniforme, independientemente y sin reemplazo entre todas las cláusulas con k literales diferentes, que son positivos o negativos al azar. (Un símbolo no puede aparecer dos veces en una cláusula, ni una cláusula puede aparecer dos veces en una sentencia).

Dada una fuente de sentencias aleatorias, podemos medir la probabilidad de satisfacción. La Figura 7.19(a) traza la probabilidad para $FNC_3(m,50)$; es decir, sentencias con 50 variables y 3 literales por cláusula, como función de la relación cláusula/símbolo, m/n. Como esperamos, para m/n pequeño la probabilidad de satisfacción está cerca de 1, y para m/n grande la probabilidad está cerca de 0. La probabilidad cae con brusquedad alrededor de $m/n = 4.3$. Empíricamente, encontramos que el "acantilado" permanece casi en el mismo lugar (para $k = 3$) y se vuelve más y más pronunciado a medida que n aumenta.

Conjetura del umbral de satisfacibilidad

Teóricamente, la **conjetura del umbral de satisfacibilidad** dice que para cada $k \geq 3$, existe una proporción umbral r_k tal que, a medida que n tiende a infinito, la probabilidad de que $FNC_k(r\text{n},n)$ sea satisfactorio se convierte en 1 para todos los valores de r por debajo del umbral, y 0 para todos los valores por encima. La conjetura permanece sin demostrar, incluso para casos

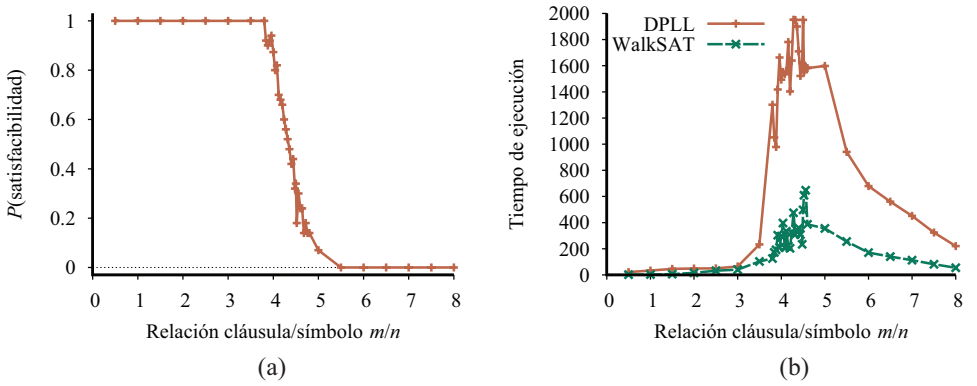


Figura 7.19 (a) Gráfico que muestra la probabilidad de que una sentencia 3-FNC aleatoria con $n = 50$ símbolos sea satisfactoria, en función de la relación cláusula/símbolo m/n. (b) Gráfico de la mediana del tiempo de ejecución (medido en número de iteraciones) para ambos DPLL y WALKSAT en 3-sentencias FNC. Los problemas más difíciles tienen una relación cláusula/símbolo de casi 4.3.

especiales como $k = 3$. Ya sea un teorema o no, este tipo de efecto de umbral es ciertamente común, para problemas de satisfacibilidad así como otros tipos de problemas NP-difíciles.

Ahora que tenemos una buena idea de dónde están los problemas satisfacibles e insatisfacibles, la siguiente pregunta es, ¿dónde están los problemas difíciles? Resulta que también suelen estar en el valor umbral. La Figura 7.19(b) muestra que los problemas de 50 símbolos en el valor umbral de 4.3 son 20 veces más difíciles de resolver que aquellos con una proporción de 3.3. Los problemas subrestringidos son los más fáciles de resolver (porque es muy fácil adivinar una solución); los problemas sobrestringidos no son tan fáciles como los subrestringidos, pero son todavía mucho más fáciles que los que están justo en el umbral.

7.7 Agentes basados en lógica proposicional

En esta sección reunimos lo que hemos aprendido hasta ahora para construir agentes del mundo del wumpus que usan lógica proposicional. El primer paso es permitir que el agente deduzca, en la medida de lo posible, el estado del mundo dado su historial de percepciones. Esto requiere escribir un modelo lógico completo de los efectos de las acciones. Luego, mostramos cómo la inferencia lógica puede ser utilizada por un agente en el mundo del wumpus. También mostramos cómo el agente puede mantener un seguimiento eficiente del mundo sin tener que volver al historial de percepciones para cada inferencia. Finalmente, mostramos cómo el agente puede usar la inferencia lógica para construir planes que están garantizados para alcanzar sus objetivos, siempre que su BC sea verdadera en el mundo real.

7.7.1 El estado actual del mundo

Como se indicó al principio del capítulo, un agente lógico opera deduciendo qué hacer a partir de una BC (base de conocimiento) de sentencias sobre el mundo. La BC está compuesta por axiomas, conocimiento general sobre cómo funciona el mundo, y sentencias de percepción obtenidas de la experiencia del agente en un mundo particular. En esta sección, nos centramos en el problema de deducir el estado actual del mundo del wumpus: ¿Estoy en una casilla que es segura? Y así sucesivamente.

En la Sección 7.4.3 comenzamos a recoger axiomas. El agente sabe que la casilla de inicio no contiene ningún pozo ($\neg P_{1,1}$) y ningún wumpus ($\neg W_{1,1}$). Además, para cada casilla, sabe que la casilla tiene brisa si y sólo si una casilla vecina tiene un pozo; y una casilla tiene hedor si y sólo si una casilla vecina tiene un wumpus. Por lo tanto, incluimos una gran colección de sentencias de la siguiente forma:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$
$$S_{1,1} \Leftrightarrow (W_{1,2} \vee W_{2,1})$$
$$\cdots$$

El agente también sabe que hay exactamente un wumpus. Esto se expresa en dos partes. Primero, tenemos que decir que hay *al menos un* wumpus:

$$W_{1,1} \vee W_{1,2} \vee \cdots \vee W_{4,3} \vee W_{4,4}.$$

Entonces tenemos que decir que hay *como máximo un* wumpus. Para cada par de ubicaciones, agregamos una sentencia diciendo que al menos una de ellas debe estar libre de wumpus:

$$\neg W_{1,1} \vee \neg W_{1,2}$$
$$\neg W_{1,1} \vee \neg W_{1,3}$$
$$\cdots$$
$$\neg W_{4,3} \vee \neg W_{4,4.}$$

Hasta este punto, todo bien. Ahora, consideremos los perceptos del agente. Estamos usando $S_{1,1}$ para significar que hay un hedor en [1,1]; ¿podemos usar una sola proposición, *Hedor* para significar que el agente percibe un hedor? Desafortunadamente no podemos: si no había hedor en el paso de tiempo anterior, entonces $\neg$*Hedor* ya estaría afirmado, y la nueva afirmación sólo resultaría en una contradicción. El problema se resuelve cuando nos damos cuenta de que un percepto afirma algo *sólo sobre el tiempo actual*. Así, si el paso de tiempo (según lo suministrado a MAKE-PERCEPT-SENTENCE en la Figura 7.1) es 4, entonces agregamos $Hedor^4$ a la BC, en lugar de *Hedor*, evitando cuidadosamente cualquier contradicción con $\neg Hedor^3$. Lo mismo ocurre con los perceptos de brisa, golpe, brillo y grito.

La idea de asociar proposiciones con pasos de tiempo se extiende a cualquier aspecto del mundo que cambia con el tiempo. Por ejemplo, la BC inicial incluye $L^0_{1,1}$ —el agente está en casilla [1,1] en el tiempo 0—, así como $MirandoAlEste^0$, $TieneFlecha^0$ y $WumpusVivo^0$. Usamos el sustantivo **flujo** (del latín *fluens*, fluyendo) para referirnos a un aspecto del mundo que cambia. "Flujo" es un sinónimo de "variable de estado", en el sentido descrito en la discusión de las representaciones factorizadas en la Sección 2.4.7 en la página 58. Los símbolos asociados con aspectos permanentes del mundo no necesitan un superíndice de tiempo y a veces se llaman **variables atemporales**.

Flujo

Variable atemporal

Podemos conectar los perceptos de hedor y brisa directamente a las propiedades de las casillas donde se experimentan de la siguiente manera.[11] Para cualquier paso de tiempo t y cualquier casilla $[x,y]$, afirmamos

$$L^t_{x,y} \Rightarrow (Brisa^t \Leftrightarrow B_{x,y})$$
$$L^t_{x,y} \Rightarrow (Hedor^t \Leftrightarrow S_{x,y}).$$

Ahora, por supuesto, necesitamos axiomas que permitan al agente seguir la pista de los flujos como $L^t_{x,y}$. Estos flujos cambian como resultado de las acciones tomadas por el agente; por lo tanto, en la terminología del Capítulo 3, necesitamos escribir el **modelo de transición** del mundo del wumpus como un conjunto de sentencias lógicas.

[11] La Sección 7.4.3 convenientemente pasó por alto este requisito.

Primero necesitamos símbolos de proposición para las ocurrencias de acciones. Al igual que con los perceptos, estos símbolos están indexados por el tiempo; así, $Avanzar^0$ significa que el agente ejecuta la acción *Avanzar* en el tiempo 0. Por convención, el percepto para un paso de tiempo dado ocurre primero, seguido por la acción para ese paso de tiempo, seguido por una transición al siguiente paso de tiempo.

Axiomas de efecto

Para describir cómo cambia el mundo, podemos intentar escribir **axiomas de efecto** que especifiquen el resultado de una acción en el siguiente paso de tiempo. Por ejemplo, si el agente está en la ubicación [1,1] mirando al este en el tiempo 0 y avanza, el resultado es que el agente está en la casilla [2,1] y ya no está en [1, 1]:

$$L^0_{1,1} \wedge MirandoAlEste^0 \wedge Avanzar^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1}). \tag{7.1}$$

Necesitaríamos una sentencia de este tipo para cada posible paso de tiempo, para cada uno de las 16 casillas y cada una de las cuatro orientaciones. También necesitaríamos sentencias similares para las otras acciones: *Agarrar*, *Disparar*, *Escalar*, *GirarIzquirda* y *GirarDerecha*.

Supongamos que el agente decide moverse hacia adelante en el tiempo 0 y afirma este hecho en su BC. Dado el axioma de efecto en la Ecuación (7.1), combinado con las afirmaciones iniciales sobre el estado en el tiempo 0, el agente ahora puede deducir que está en [2,1]. Eso es, $\text{ASK}(BC, L^1_{2,1}) = verdadero$. Hasta ahora, todo bien. Desafortunadamente, si preguntamos $\text{ASK}(BC, TieneFlecha^1)$, la respuesta es *falsa*, es decir, el agente no puede probar que todavía tiene la flecha; tampoco puede probar *que no* la tiene! La información se ha perdido porque el axioma de efecto no establece lo que permanece *inmutable* como resultado de una acción. La necesidad de hacer esto da lugar al **problema del marco**.[12] Una posible solución al problema del marco sería agregar **axiomas de marco** que afirmen explícitamente todas las proposiciones que permanecen iguales. Por ejemplo, para cada tiempo t tendríamos

Problema del marco

Axioma del marco

$$Avanzar^t \Rightarrow (TieneFlecha^t \Leftrightarrow TieneFlecha^{t+1})$$
$$Avanzar^t \Rightarrow (WumpusVivo^t \Leftrightarrow WumpusVivo^{t+1})$$
$$\ldots$$

donde mencionamos explícitamente cada proposición que permanece inalterada desde el tiempo t hasta el tiempo $t + 1$ bajo la acción *Avanzar*. Aunque el agente ahora sabe que todavía tiene la flecha después de avanzar y que el wumpus no ha muerto ni ha vuelto a la vida, la proliferación de axiomas de marco parece notablemente ineficiente. En un mundo con m acciones diferentes y n flujos, el conjunto de axiomas de marco será de tamaño $O(mn)$. Esta manifestación específica del problema del marco a veces se llama el **problema del marco representacional**. El problema tiene un papel significativo en la historia de la IA; lo exploramos más a fondo en las notas al final del capítulo.

Problema del marco representacional

El problema del marco representacional es significativo porque el mundo real tiene muchos flujos, por decirlo suavemente. Afortunadamente para nosotros los humanos, cada acción típicamente no cambia más que un pequeño número k de esos flujos: el mundo exhibe **localidad**. Resolver el problema del marco representacional requiere definir el modelo de transición con un conjunto de axiomas de tamaño $O(mk)$ en lugar de tamaño $O(mn)$. También existe un **problema de marco inferencial**: el problema de proyectar hacia adelante los resultados de un plan de acción de t pasos en tiempo $O(kt)$ en lugar de $O(nt)$.

Localidad

Problema de marco inferencial

La solución al problema implica cambiar el enfoque de escribir axiomas sobre *acciones* a escribir axiomas sobre *flujos*. Así, para cada flujo F, tendremos un axioma que define el valor de verdad de F^{t+1} en términos de flujos (incluyendo F mismo) en el tiempo t y las acciones que

[12] El nombre "problema del marco" proviene de "marco de referencia" en física: el supuesto fondo estacionario con respecto al cual se mide el movimiento. También tiene una analogía con los marcos de una película, en la que normalmente la mayoría del fondo permanece constante mientras ocurren cambios en el primer plano.

pueden haber ocurrido en el tiempo t. Ahora, el valor de verdad de F^{t+1} puede establecerse de una de dos maneras: ya sea que la acción en el tiempo t cause que F sea verdadero en $t + 1$, o F ya era verdadero en el tiempo t y la acción en el tiempo t no lo hace falso. Un axioma de esta forma se llama **axioma de estado sucesor** y tiene esta forma:

Axioma de estado sucesor

$$F^{t+1} \Leftrightarrow \mathit{AccionesCausaF}^t \vee (F^t \wedge \neg \mathit{AccionesCausaNoF}^t).$$

Uno de los axiomas de estado sucesor más simples es el de *TieneFlecha*. Como no hay acción para recargar, la parte $\mathit{AccionesCausaF}^t$ desaparece y nos quedamos con

$$\mathit{TieneFlecha}^{t+1} \Leftrightarrow (\mathit{TieneFlecha}^t \wedge \neg \mathit{Disparar}^t). \tag{7.2}$$

Para la ubicación del agente, los axiomas de estado sucesor son más elaborados. Por ejemplo, L^{t+1} es verdadero si (a) el agente se movió *hacia adelante* desde [1,2] cuando estaba mirando hacia el sur, o desde [2,1] cuando estaba mirando hacia el oeste; o (b) $L^t_{1,1}$ ya era verdadero y la acción no causó movimiento (ya sea porque la acción no era *Avanzar* o porque la acción golpeó contra una pared). Escrito en lógica proposicional, esto se convierte

$$\begin{aligned} L^{t+1}_{1,1} \quad \Leftrightarrow \quad & (L^t_{1,1} \wedge (\neg \mathit{Avanzar}^t \vee \mathit{Golpe}^{t+1})) \\ \vee \quad & (L^t_{1,2} \wedge (\mathit{MirandoAlSur}^t \wedge \mathit{Avanzar}^t)) \\ \vee \quad & (L^t_{2,1} \wedge (\mathit{MirandoAlOeste}^t \wedge \mathit{Avanzar}^t)). \end{aligned} \tag{7.3}$$

El ejercicio 7.SSAX pide que escriba axiomas para los flujos restantes del mundo del wumpus.

Dado un conjunto completo de axiomas de estado sucesor y los otros axiomas enumerados al principio de esta sección, el agente podrá preguntar (Ask) y responder cualquier pregunta respondible sobre el estado actual del mundo. Por ejemplo, en la Sección 7.2 la secuencia inicial de percepciones y acciones es

$$\neg \mathit{Hedor}^0 \wedge \neg \mathit{Brisa}^0 \wedge \neg \mathit{Brillo}^0 \wedge \neg \mathit{Golpe}^0 \wedge \neg \mathrm{Grito}^0; \mathit{Avanzar}^0$$

$$\neg \mathit{Hedor}^1 \wedge \mathit{Brisa}^1 \wedge \neg \mathit{Brillo}^1 \wedge \neg \mathit{Golpe}^1 \wedge \neg \mathrm{Grito}^1; \mathit{GirarDerecha}^1$$

$$\neg \mathit{Hedor}^2 \wedge \mathit{Brisa}^2 \wedge \neg \mathit{Brillo}^2 \wedge \neg \mathit{Golpe}^2 \wedge \neg \mathrm{Grito}^2; \mathit{GirarDerecha}^2$$

$$\neg \mathit{Hedor}^3 \wedge \mathit{Brisa}^3 \wedge \neg \mathit{Brillo}^3 \wedge \neg \mathit{Golpe}^3 \wedge \neg \mathrm{Grito}^3; \mathit{Avanzar}^3$$

$$\neg \mathit{Hedor}^4 \wedge \neg \mathit{Brisa}^4 \wedge \neg \mathit{Brillo}^4 \wedge \neg \mathit{Golpe}^4 \wedge \neg \mathrm{Grito}^4; \mathit{GirarDerecha}^4$$

$$\neg \mathit{Hedor}^5 \wedge \neg \mathit{Brisa}^5 \wedge \neg \mathit{Brillo}^5 \wedge \neg \mathit{Golpe}^5 \wedge \neg \mathrm{Grito}^5; \mathit{Avanzar}^5$$

$$\mathit{Hedor}^6 \wedge \neg \mathit{Brisa}^6 \wedge \neg \mathit{Brillo}^6 \wedge \neg \mathit{Golpe}^6 \wedge \neg \mathrm{Grito}^6$$

En este punto, tenemos $\mathrm{Ask}(\mathit{BC}, L^6_{1,2}) = \mathit{verdadero}$, por lo que el agente sabe dónde está. Además, $\mathrm{Ask}(\mathit{BC}, W_{1,3}) = \mathit{verdadero}$ y $\mathrm{Ask}(\mathit{BC}, P_{1,3}) = \mathit{verdadero}$, por lo que el agente ha encontrado al wumpus y uno de los pozos. La pregunta más importante para el agente es si una casilla está OK para moverse a ella; es decir, si la casilla está libre de un pozo o wumpus vivo. Es conveniente agregar axiomas para esto, que tienen la forma

$$\mathit{OK}^t_{x,y} \Leftrightarrow \neg P_{x,y} \wedge \neg(W_{x,y} \wedge \mathit{WumpusVivo}^t).$$

Finalmente, $\mathrm{Ask}(\mathit{BC}, \mathit{OK}^6_{2,2}) = \mathit{verdadero}$, por lo que la casilla [2,2] está bien para moverse. De hecho, dado un algoritmo de inferencia sólido y completo como DPLL, el agente puede responder cualquier pregunta solucionable sobre qué casillas están OK, y puede hacerlo en sólo unos pocos milisegundos para mundos de wumpus pequeños a medianos.

Resolver los problemas de marco representacional e inferencial es un gran paso adelante, pero queda un problema complicado: necesitamos confirmar que *todas* las precondiciones necesarias de una acción se mantienen para que tenga su efecto previsto. Dijimos que la acción *Avanzar*

mueve al agente adelante a menos que haya una pared en el camino, pero hay muchas otras excepciones inusuales que podrían hacer que la acción falle: el agente podría tropezar y caer, sufrir un ataque al corazón, ser llevado por murciélagos gigantes, etc. Especificar todas estas excepciones se llama el **problema de calificación**. No hay una solución completa dentro de la lógica; los diseñadores de sistemas tienen que usar su buen juicio al decidir cuán detallados quieren ser al especificar su modelo, y qué detalles quieren dejar fuera. En el Capítulo 12 veremos que la teoría de la probabilidad nos permite resumir todas las excepciones sin nombrarlas explícitamente.

Problema de calificación

7.7.2 Un agente híbrido

La capacidad de deducir varios aspectos del estado del mundo se puede combinar de manera directa con las reglas de condición-acción (ver Sección 2.4.2) y con los algoritmos de resolución de problemas de los Capítulos 3 y 4 para producir un **agente híbrido** para el mundo del wumpus. La Figura 7.20 muestra una posible forma de hacer esto. El programa del agente mantiene y actualiza una BC así como un plan actual. La BC inicial contiene los *axiomas atemporales*, aquellos que no dependen de t, como el axioma que relaciona la brisa de las casillas con la presencia de pozos. En cada paso de tiempo, se agrega la nueva sentencia perceptiva junto con todos los axiomas que dependen de t, como los axiomas de estado sucesor. (La siguiente sección explica por qué el agente no necesita axiomas para los pasos de tiempo *futuros*). Luego, el agente utiliza la inferencia lógica, haciendo preguntas a la BC, para averiguar qué casillas son seguras y cuáles aún no se han visitado.

Agente híbrido

El cuerpo principal del programa del agente construye un plan basado en una prioridad decreciente de objetivos. Primero, si hay un brillo, el programa construye un plan para agarrar el oro, seguir una ruta de regreso a la ubicación inicial y salir de la cueva. De lo contrario, si no hay un plan actual, el programa planea una ruta a la casilla segura más cercana que aún no ha visitado, asegurándose de que la ruta sólo pase por casillas seguras.

La planificación de rutas se hace con la búsqueda A*, no con ASK. Si no hay casillas seguras para explorar, el siguiente paso, si el agente todavía tiene una flecha, es intentar hacer una casilla segura disparando a una de las posibles ubicaciones del wumpus. Estas se determinan preguntando dónde ASK(BC,$\neg W_{x,y}$) es falso, es decir, donde *no* se sabe que *no hay* un wumpus. La función PLAN-SHOT (no mostrada) utiliza PLAN-ROUTE para planificar una secuencia de acciones que alinearán este disparar. Si esto falla, el programa busca una casilla para explorar que probablemente no sea insegura; es decir, una casilla para la cual ASK(BC,$\neg OK^t_{x,y}$) devuelva falso. Si no hay tal casilla, entonces la misión es imposible y el agente se retira a [1,1] y sale de la cueva.

7.7.3 Estimación del estado lógico

El programa del agente en la Figura 7.20 funciona bastante bien, pero tiene una debilidad importante: a medida que pasa el tiempo, el gasto computacional involucrado en las llamadas a ASK aumenta y aumenta. Esto sucede principalmente porque las inferencias requeridas tienen que retroceder cada vez más en el tiempo e involucrar más y más símbolos de proposición. Obviamente, esto es insostenible, ¡no podemos tener un agente cuyo tiempo para procesar cada percepción crece en proporción a la longitud de su vida! Lo que realmente necesitamos es un tiempo de actualización *constante*, es decir, independiente de t. La respuesta obvia es guardar, o **almacenar en caché**, los resultados de la inferencia, de modo que el proceso de inferencia en el siguiente paso de tiempo pueda construir sobre los resultados de los pasos anteriores en lugar de tener que empezar de nuevo desde cero.

Almacenar en caché

Como vimos en la Sección 4.4, la historia de las percepciones y todas sus ramificaciones pueden ser reemplazadas por el **estado de creencia**, es decir, alguna representación del conjunto de

```
function HYBRID-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list,[stench,breeze,glitter,bump,scream]
  persistent: BC, una base de conocimiento, inicialmente la "física Wumpus" atemporal
              t, un contador, inicialmente 0, que indica el plan de tiempo,
              una secuencia de acción, inicialmente vacía

  TELL(BC,MAKE-PERCEPT-SENTENCE(percept,t))
  TELL la BC las sentencias de "física"" temporal para el tiempo t
  safe←{[x,y] : ASK(BC,OK^t_x,y) = true}
  if ASK(BC,Glitter^t) = true then
    plan←[Grab] + PLAN-ROUTE(current,{[1,1]},safe) + [Climb]
  if plan is empty then
    unvisited←{[x,y] : ASK(BC,L^t'_x,y) = false for all t' ≤ t}
    plan←PLAN-ROUTE(current,unvisited∩safe,safe)
  if plan is empty and ASK(BC,HaveArrow^t) = true then
    posible_wumpus←{[x,y] : ASK(BC,¬W_x,y) = false}
    plan←PLAN-SHOT(current,possible_wumpus,safe)
  if plan is empty then          // No hay más remedio que arriesgarse
    not_unsafe←{[x,y] : ASK(BC,¬OK^t_x,y) = false}
    plan←PLAN-ROUTE(current,unvisited∩not_unsafe,safe)
  if plan is empty then
    plan←PLAN-ROUTE(current,{[1,1]},safe) + [Climb]
  action←POP(plan)
  TELL(BC,MAKE-ACTION-SENTENCE(action,t))
  t←t + 1
  return action

function PLAN-ROUTE(current,goals,allowed)returns una secuencia de acciones
  inputs: current, la posición real del agente
          goals, un conjunto de casillas; intenta planificar una ruta a una de ellas
          allowed, un conjunto de plazas que pueden formar parte del recorrido

  problem←ROUTE-PROBLEM(current, goals,allowed)
  return SEARCH(problem)          // Cualquier algoritmo de búsqueda del capítulo 3
```

Figura 7.20 Un programa de agente híbrido para el mundo del wumpus. Utiliza una BC proposicional para inferir el estado del mundo, y una combinación de búsqueda de resolución de problemas y código específico del dominio para elegir acciones. Cada vez que se llama a HYBRID-WUMPUS-AGENT, añade la percepción a la BC, y luego se basa en un plan previamente definido o crea un nuevo plan, y saca el primer paso del plan como la acción a realizar a continuación.

todos los estados actuales posibles del mundo.[13] El proceso de actualización del estado de creencia a medida que llegan nuevas percepciones se llama **estimación de estado** (ver página 132). Mientras que en la Sección 4.4 el estado de creencia era una lista explícita de estados, aquí podemos usar una sentencia lógica que involucre los símbolos de proposición asociados con el paso de tiempo actual, así como los símbolos atemporales. Por ejemplo, la sentencia lógica

$$WumpusVivo^1 \wedge L^1_{2,1} \wedge B_{2,1} \wedge (P_{3,1} \vee P_{2,2}) \tag{7.4}$$

[13] Podemos pensar en la historia de las percepciones en sí misma como una representación del estado de creencia, pero una que hace que la inferencia sea cada vez más costosa a medida que la historia se alarga.

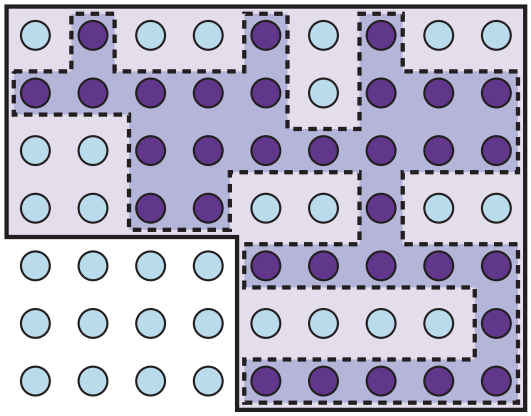


Figura 7.21 Representación de un estado de creencia 1-FNC (contorno en negrita) como una aproximación sólo representable y conservadora al estado de creencia exacto (región sombreada con contorno punteado). Cada mundo posible se muestra como un círculo; los sombreados son consistentes con todos los perceptos.

representa el conjunto de todos los estados en el tiempo 1 en el que el wumpus está vivo, el agente está en [2,1], esa casilla tiene brisa, y hay un pozo en [3,1] o [2,2] o en ambos.

Mantener un estado de creencia exacto como una fórmula lógica resulta no ser fácil. Si hay n símbolos que pueden cambiar para el tiempo t, entonces hay 2^n estados posibles, es decir, asignaciones de valores de verdad a esos símbolos. Ahora, el conjunto de estados de creencia es el conjunto de potencias (conjunto de todos los subconjuntos) de conjunto de estados físicos. Hay 2^n estados físicos, por lo tanto, 2^{2n} estados de creencia. Aun si usáramos la codificación más compacta posible de fórmulas lógicas, con cada estado de creencia representado por un número binario único, necesitaríamos números con $\log_2(2^{2n}) = 2^n$ bits para etiquetar el estado de creencia actual. Es decir, la estimación de estado exacta puede requerir fórmulas lógicas cuyo tamaño es exponencial en el número de símbolos.

Un esquema muy común y natural para la estimación de estado *aproximado* es representar los estados de creencia como conjunciones de literales, es decir, fórmulas en 1-FNC. Para hacer esto, el programa del agente sólo intenta probar X^t y $\neg X^t$ para cada símbolo X^t (así como cada símbolo atemporal cuyo valor de verdad aún no se conoce), dado el estado de creencia en $t-1$. La conjunción de literales probables se convierte en el nuevo estado de creencia, y se descarta el estado de creencia anterior.

Es importante entender que este esquema puede perder información a medida que pasa el tiempo. Por ejemplo, si la sentencia en la Ecuación (7.4) fuera el verdadero estado de creencia, entonces ni $P_{3,1}$ ni $P_{2,2}$ serían demostrables individualmente y ninguno aparecería en el estado de creencia en 1-FNC. (El Ejercicio 7.HYBR explora una posible solución a este problema). Por otro lado, porque cada literal en el estado de creencia 1-FNC se prueba a partir del estado de creencia anterior, y el estado de creencia inicial es una afirmación verdadera, sabemos que todo el estado de creencia en 1-FNC debe ser verdadero. Así, *el conjunto de estados posibles representados por el estado de creencia en 1-FNC incluye todos los estados que son de hecho posibles dados los perceptos históricos*. Como se ilustra en la Figura 7.21, el modelo 1-if no es nulo entonces el estado de creencia en FNC actúa como un sobre exterior simple, o aproximación conservadora, alrededor del estado de creencia exacto. Vemos esta idea de **aproximaciones conservadoras** a conjuntos complicados como un tema recurrente en muchas áreas de la IA.

Aproximaciones conservadoras

```
function SATPLAN(init,transition,goal,T_max) returns solution or failure
   inputs: init, transition, goal, constituye una descripción del problema
           T_max, un límite superior para la longitud del plan

   for t = 0 to T_max do
      fnc ←TRANSLATE-TO-SAT(init,transition,goal,t)
      model←SAT-SOLVER(fnc)
      if model is not null then
         return EXTRACT-SOLUTION(model)
   return failure
```

Figura 7.22 El algoritmo SATPLAN. El problema de planificación se traduce en una sentencia en FNC en la que se afirma que el objetivo se cumple en un paso de tiempo fijo t y se incluyen axiomas para cada paso de tiempo hasta t. Si el algoritmo de satisfacción encuentra un modelo, entonces se extrae un plan mirando aquellos símbolos de proposición que se refieren a acciones y se les asigna *verdadero* en el modelo. Si no existe ningún modelo, entonces el proceso se repite con el objetivo movido un paso más tarde.

7.7.4 Haciendo planes por inferencia proposicional

El agente en la Figura 7.20 usa la inferencia lógica para determinar qué casillas son seguras, pero usa la búsqueda A* para hacer planes. En esta sección, mostramos cómo hacer planes por inferencia lógica. La idea básica es muy simple:

1. Construir una sentencia que incluya
   (a) $Inicial^0$, una colección de afirmaciones sobre el estado inicial;
   (b) $Transición^1$,..., $Transición^t$, los axiomas de estado sucesor para todas las acciones posibles en cada momento hasta un tiempo máximo t;
   (c) la afirmación de que el objetivo se logra en el tiempo t: $TieneElOro^t \wedge EscalóFuera^t$.
2. Presenta la sentencia completa a un solucionador SAT. Si el solucionador encuentra un modelo satisfactorio, entonces el objetivo es alcanzable; si la sentencia es insatisfacible, entonces el problema es insoluble.
3. Suponiendo que se encuentra un modelo, extrae del modelo aquellas variables que representan acciones y se les asigna *verdadero*. Juntas representan un plan para alcanzar los objetivos.

Un procedimiento de planificación proposicional, SATPLAN, se muestra en la Figura 7.22. Implementa la idea básica recién dada, con un giro. Debido a que el agente no sabe cuántos pasos tomará para alcanzar el objetivo, el algoritmo intenta cada número posible de pasos t, hasta una longitud máxima concebible del plan T_{max}. De esta manera, se garantiza encontrar el plan más corto si existe uno. Debido a la forma en que SATPLAN busca una solución, este enfoque no puede usarse en un entorno parcialmente observable; SATPLAN sólo establecería las variables inobservables a los valores que necesita para crear una solución.

El paso clave en el uso de SATPLAN es la construcción de la BC. A simple vista podría parecer que los axiomas del mundo del wumpus en la Sección 7.7.1 son suficientes para los pasos 1(a) y 1(b) anteriores. Sin embargo, hay una diferencia significativa entre los requisitos para la implicación (como se prueba con ASK) y los de satisfacibilidad.

Considere, por ejemplo, la ubicación del agente, inicialmente [1,1], y suponga que el objetivo del agente es estar en [2,1] en el tiempo 1. La BC inicial contiene $L^0_{1,1}$ y el objetivo es $L^1_{2,1}$. Usando

ASK, podemos probar $L^1_{2,1}$ si se afirma $Avanzar^0$, y, tranquilizadoramente, no podemos probar $L^1_{2,1}$ si, por ejemplo, se afirma $Disparar^0$ en su lugar. Ahora, SATPLAN encontrará el plan [$Avanzar^0$]; hasta ahora, todo bien.

Desafortunadamente, SATPLAN también encuentra el plan [$Disparar^0$]. ¿Cómo podría ser esto? Para averiguarlo, inspeccionamos el modelo que construye SATPLAN: incluye la asignación $L^0_{2,1}$, es decir, el agente puede estar en [2,1] en el tiempo 1, estando allí en el tiempo 0 y disparando. Uno podría preguntar, "¿No dijimos que el agente está en [1,1] en el tiempo 0?" Sí, lo hicimos, ¡pero no le dijimos al agente que no puede estar en dos lugares a la vez! Para la implicación, $L^0_{2,1}$ es desconocido y no puede, por lo tanto, usarse en una prueba; por otro lado, para la satisfacibilidad $L^0_{2,1}$ es desconocido y puede, por lo tanto, establecerse a cualquier valor que ayude a hacer verdadero el objetivo.

SATPLAN es una buena herramienta de depuración para las BC porque revela lugares donde falta conocimiento. En este caso particular, podemos corregir la BC afirmando que, en cada paso de tiempo, el agente está exactamente en una ubicación, usando una colección de sentencias similares a las empleadas para afirmar la existencia de exactamente un wumpus. De manera alternativa, podemos afirmar $\neg L^0_{x,y}$ para todas las ubicaciones distintas de [1,1]; el axioma de estado sucesor para la ubicación se encarga de los pasos de tiempo subsiguientes. Las mismas correcciones también funcionan para asegurarse de que el agente tiene una y sólo una orientación a la vez.

Sin embargo, SATPLAN tiene más sorpresas reservadas. La primera es que encuentra modelos con acciones imposibles, como disparar sin flecha. Para entender por qué, necesitamos mirar más cuidadosamente lo que los axiomas de estado sucesor (como la Ecuación (7.3)) dicen sobre las acciones cuyas precondiciones no están satisfechas. Los axiomas sí predicen correctamente que nada sucederá cuando se ejecute tal acción (ver Ejercicio 7.SATP), ¡pero *no* dicen que la acción no puede ser ejecutada! Para evitar generar planes con acciones ilegales, debemos agregar **axiomas de precondición** afirmando que la ocurrencia de una acción requiere que las precondiciones estén satisfechas.[14] Por ejemplo, necesitamos decir, para cada tiempo t, que

Axiomas de precondición

$$Disparar^t \Rightarrow TieneFlecha^t.$$

Esto asegura que si un plan selecciona la acción de *Disparar* en cualquier momento, el agente tiene una flecha en ese momento.

La segunda sorpresa de SATPLAN es la creación de planes con múltiples acciones simultáneas. Por ejemplo, puede surgir un modelo en el que tanto $Avanzar^0$ como $Disparar^0$ son verdaderos, lo cual no está permitido. Para eliminar este problema, introducimos **axiomas de exclusión de acción**: para cada par de acciones A^t_i y A^t_j añadimos el axioma

Axiomas de exclusión de acción

$$\neg A^t_i \vee \neg A^t_j$$

Se podría señalar que caminar hacia adelante y disparar al mismo tiempo no es tan difícil de hacer; mientras que, digamos, disparar y agarrar al mismo tiempo es bastante impráctico. Al imponer axiomas de exclusión de acción sólo en pares de acciones que realmente interfieren entre sí, podemos permitir planes que incluyen múltiples acciones simultáneas, y ya que SATPLAN encuentra el plan legal más corto, podemos estar seguros de que aprovechará esta capacidad.

Para resumir, SATPLAN encuentra modelos para una sentencia que contiene el estado inicial, el objetivo, los axiomas del estado sucesor, los axiomas de precondición y los axiomas de exclusión de acción. Se puede demostrar que esta colección de axiomas es suficiente, en el sentido de que

[14] Observe que la adición de axiomas de precondición significa que no necesitamos incluir precondiciones para acciones en los axiomas del estado sucesor.

ya no hay "soluciones" espurias. Cualquier modelo que satisfaga la sentencia proposicional será un plan válido para el problema original. La tecnología moderna de resolución de SAT hace que el enfoque sea bastante práctico. Por ejemplo, un solucionador del estilo DPLL no tiene dificultades para generar la solución para la instancia del mundo del wumpus mostrada en la Figura 7.2.

Esta sección ha descrito un enfoque declarativo para la construcción de agentes: el agente trabaja mediante una combinación de afirmaciones de sentencias en la BC y la realización de inferencias lógicas. Este enfoque tiene algunas debilidades ocultas en frases como "para cada tiempo t" y "para cada casilla $[x,y]$". Para cualquier agente práctico, estas frases deben ser implementadas por código que genera automáticamente instancias del esquema de sentencia general para su inserción en la BC. Para un mundo del wumpus de tamaño razonable, uno comparable a un juego de computadora pequeño, podríamos necesitar un tablero de 100×100 y 1 000 pasos de tiempo, lo que lleva a una BC con decenas o cientos de millones de sentencias.

Esto no sólo se vuelve bastante impráctico, sino que también ilustra un problema más profundo: sabemos algo sobre el mundo del wumpus, a saber, que la "física" funciona de la misma manera en todas las casillas y todos los pasos de tiempo, que no podemos expresar directamente en el lenguaje de la lógica proposicional. Para resolver este problema, necesitamos un lenguaje más expresivo, uno en el que frases como "para cada tiempo t" y "para cada casilla $[x,y]$" puedan ser escritas de una manera natural. La lógica de primer orden, descrita en el Capítulo 8, es ese lenguaje; en la lógica de primer orden, un mundo del wumpus de cualquier tamaño y duración puede ser descrito en unas diez sentencias lógicas en lugar de diez millones o diez billones.

Resumen

Hemos introducido agentes de BC y hemos mostrado cómo definir una lógica con la que tales agentes pueden razonar sobre el mundo. Los puntos principales son los siguientes:

- Los agentes inteligentes necesitan conocimiento sobre el mundo para tomar buenas decisiones.
- El conocimiento está contenido en los agentes en forma de **sentencias** en un lenguaje de representación de conocimiento que se almacenan en una **base de conocimiento** (BC).
- Un agente basado en conocimiento está compuesto por una BC y un mecanismo de inferencia. Funciona almacenando sentencias sobre el mundo en su BC, utilizando el mecanismo de inferencia para inferir nuevas sentencias y utilizando estas sentencias para decidir qué acción tomar.
- Un lenguaje de representación se define por su **sintaxis**, que especifica la estructura de las sentencias, y su **semántica**, que define la **verdad** de cada sentencia en cada **mundo** o **modelo posible**.
- La relación de **implicación** entre las sentencias es crucial para nuestra comprensión del razonamiento. Una sentencia α implica otra sentencia β, si β es verdadera en todos los mundos donde α es verdadera. Las definiciones equivalentes incluyen la **validez** de la sentencia $\alpha \Rightarrow \beta$ y la **insatisfacibilidad** de la sentencia $\alpha \wedge \neg\beta$.
- La inferencia es el proceso de derivar nuevas sentencias de las antiguas. Los algoritmos de inferencia **sólidos** derivan *sólo* sentencias que se implican; los algoritmos **completos** derivan *todas* las sentencias que se implican.
- La **lógica proposicional** es un lenguaje simple que consiste en **símbolos de proposición** y **conectivos lógicos**. Puede manejar proposiciones que se sabe que son verdaderas, que se sabe que son falsas o que son completamente desconocidas.

- El conjunto de posibles modelos es finito, dado un vocabulario proposicional fijo, por lo que la implicación se puede comprobar enumerando modelos. Los algoritmos de inferencia eficientes para la **comprobación de modelos** en lógica proposicional incluyen métodos de retroceso y búsqueda local y a menudo pueden resolver problemas grandes rápidamente.
- **Las reglas de inferencia** son patrones de inferencia sólida que se pueden usar para encontrar pruebas. La regla de **resolución** produce un algoritmo de inferencia completo para BC que se expresa en **forma normal conjuntiva**. El **encadenamiento hacia adelante** y **encadenamiento hacia atrás** son algoritmos de razonamiento muy naturales para bases de conocimiento en **forma de Horn**.
- Los métodos de **búsqueda local** como WalkSAT se pueden usar para encontrar soluciones. Dichos algoritmos son sólidos pero no completos.
- La lógica de **estimación del estado** implica mantener una sentencia lógica que describe el conjunto de estados posibles consistentes con el historial de observación. Cada paso de actualización requiere inferencia utilizando el modelo de transición del entorno, que se construye a partir de **axiomas de estado sucesor** que especifican cómo cambia cada **flujo**.
- Las decisiones dentro de un agente lógico se pueden tomar mediante la resolución de SAT: encontrar posibles modelos que especifican secuencias de acciones futuras que alcanzan la meta. Este enfoque sólo funciona para entornos completamente observables o sin sensores.
- La lógica proposicional no escala a entornos de tamaño ilimitado porque carece del poder expresivo para tratar de manera concisa con el tiempo, el espacio y los patrones universales de relaciones entre objetos.

Notas bibliográficas e históricas

El artículo de John McCarthy "Programas con sentido común" (McCarthy, 1958, 1968) promulgó la noción de agentes que utilizan el razonamiento lógico para mediar entre percepciones y acciones. También levantó la bandera del declarativismo, señalando que decirle a un agente lo que necesita saber es una forma elegante de construir software. El artículo de Allen Newell (1982) "El nivel de conocimiento" argumenta que los agentes racionales pueden describirse y analizarse a un nivel abstracto definido por el conocimiento que poseen en lugar de los programas que ejecutan.

La lógica misma tuvo sus orígenes en la filosofía y matemáticas griegas antiguas. Platón discutió la estructura sintáctica de las sentencias, su verdad y falsedad, su significado y la validez de los argumentos lógicos. El primer estudio sistemático conocido de la lógica fue el *Organon* de Aristóteles. Sus **silogismos** eran lo que ahora llamamos reglas de inferencia, aunque carecían de la composicionalidad del silogismo de nuestras reglas actuales.

Silogismo

Las escuelas megárica y estoica comenzaron el estudio sistemático de los conectivos lógicos básicos en el siglo V a.C. Las tablas de verdad se deben a Filón de Megara. Los estoicos tomaron cinco reglas de inferencia básicas como válidas sin prueba, incluida la regla que ahora llamamos Modus Ponens. Derivaron una serie de otras reglas de estas cinco, utilizando, entre otros principios, el teorema de deducción (página 222) y fueron más claros acerca de la prueba que Aristóteles (Mates, 1953).

La idea de reducir la inferencia lógica a un proceso puramente mecánico se debe a Wilhelm Leibniz (1646–1716). George Boole (1847) introdujo el primer sistema completo y manejable de lógica formal en su libro *The Mathematical Analysis of Logic*. La lógica de Boole se modeló estrechamente en el álgebra ordinaria de números reales y utilizó la sustitución de expresiones lógicamente equivalentes como su método de inferencia principal. Aunque no manejó todo sobre la lógica proposicional, otros matemáticos pronto llenaron las piezas faltantes. Schröder (1877)

describió la forma normal conjuntiva, mientras que la forma de Horn fue introducida mucho más tarde por Alfred Horn (1951). La primera exposición completa de la lógica proposicional moderna (y la lógica de primer orden) se encuentra en *Begriffschrift* ("Escritura de conceptos" o "Notación conceptual") de Gottlob Frege (1879).

El primer dispositivo mecánico para llevar a cabo inferencias lógicas fue el Stanhope Demonstrator, construido por el tercer conde de Stanhope (1753–1816). William Stanley Jevons, uno de los matemáticos que extendió el trabajo de Boole, construyó su "piano lógico" en 1869 para hacer inferencias en lógica booleana. Una historia entretenida de estos primeros dispositivos de inferencia mecánica es proporcionada por Martin Gardner (1968). Los primeros programas de computadora para la inferencia lógica fueron el programa de Martin Davis de 1954 para pruebas en aritmética de Presburger (Davis, 1957), y el *Logic Theorist* de Newell, Shaw y Simon (1957).

Emil Post (1921) y Ludwig Wittgenstein (1922) usaron de manera independiente tablas de verdad como un método para probar la validez de las sentencias de lógica proposicional. El algoritmo Davis–Putnam (Davis y Putnam, 1960) fue el primer algoritmo para la resolución proposicional, y el algoritmo de retroceso DPLL mejorado (Davis *et al*., 1962) resultó ser más eficiente. La regla de resolución y una prueba de su integridad se desarrollaron en plena generalidad para la lógica de primer orden por J. A. Robinson (1965).

Stephen Cook (1971) mostró que decidir la satisfacibilidad de una sentencia en lógica proposicional (el problema SAT) es NP-completo. Se conocen muchos subconjuntos de lógica proposicional para los cuales el problema de satisfacción es solucionable en tiempo polinomial; las cláusulas de Horn son un subconjunto de este tipo.

Las primeras investigaciones mostraron que DPLL tiene una complejidad promedio de caso polinomial para ciertas distribuciones naturales de problemas. Aún mejor, Franco y Paull (1983) demostraron que los mismos problemas podrían resolverse en tiempo *constante* simplemente adivinando asignaciones aleatorias. Motivados por el éxito empírico de la búsqueda local, Koutsoupias y Papadimitriou (1992) demostraron que un simple algoritmo de ascenso de colina puede resolver *casi todos* los problemas de satisfacción de instancias muy rápidamente, sugiriendo que los problemas difíciles son raros. Schöning (1999) exhibió un algoritmo de ascenso de colina aleatorizado cuyo *peor caso* de tiempo de ejecución esperado en problemas 3-SAT es $O(1.333^n)$, aún exponencial, pero sustancialmente más rápido que los límites de peor caso anteriores. El récord actual es $O(1.32216^n)$ (Rolf, 2006).

Las ganancias de eficiencia en los solucionadores proposicionales han sido rápidas. Dados diez minutos de computación, el algoritmo DPLL original en hardware de 1962 sólo podía resolver problemas con 10 o 15 variables (en una laptop de 2019 serían alrededor de 30 variables). Para 1995, el solucionador SATZ (Li y Anbulagan, 1997) podía manejar 1 000 variables, gracias a las estructuras de datos optimizadas para indexar variables. Dos contribuciones cruciales fueron la técnica de indexación de **literales observados** de Zhang y Stickel (1996), que hace que la propagación de unidades sea muy eficiente, y la introducción de técnicas de aprendizaje de cláusulas (es decir, restricciones) de la comunidad CSP por Bayardo y Schrag (1997). Usando estas ideas, y estimulados por la perspectiva de resolver problemas de verificación de circuitos a escala industrial, Moskewicz *et al*. (2001) desarrollaron el solucionador CHAFF, que podía manejar problemas con millones de variables. A partir de 2002, se han celebrado competencias anuales de SAT; la mayoría de los algoritmos ganadores han sido variantes de CHAFF. El panorama de los solucionadores es estudiado por Gomes *et al*. (2008).

Literales observados

Los algoritmos de búsqueda local para satisfacción fueron probados por varios autores a lo largo de la década de 1980, basados en la idea de minimizar el número de cláusulas insatisfechas (Hansen y Jaumard, 1990). Un algoritmo particularmente efectivo fue desarrollado por Gu (1989) e independientemente por Selman *et al*. (1992), quienes lo llamaron GSAT y demostraron que era

capaz de resolver una amplia gama de problemas muy difíciles muy rápidamente. El algoritmo WalkSAT descrito en este capítulo se debe a Selman *et al*. (1996).

La "transición de fase" en la satisfacción de problemas aleatorios de *k*-SAT fue observada por primera vez por Simon y Dubois (1989) y ha dado lugar a una gran cantidad de investigación teórica y empírica, en parte debido a la conexión con los fenómenos de transición de fase en la física estadística. Crawford y Auton (1993) ubicaron la transición 3-SAT en una relación cláusula/variable de alrededor de 4.26, notando que esto coincide con un pico agudo en el tiempo de ejecución de su solucionador SAT. Cook y Mitchell (1997) proporcionan un excelente resumen de la literatura temprana sobre el problema. Algoritmos como la **survey propagation** (Parisi y Zecchina, 2002; Maneva *et al*., 2007) aprovechan propiedades especiales de instancias SAT aleatorias cerca del umbral de satisfacción y superan ampliamente los solucionadores SAT generales en tales instancias. El estado actual de la comprensión teórica está resumido por Achlioptas (2009).

Survey propagation

Fuentes de información confiables sobre satisfacción, tanto teórica como práctica, incluyen el *Manual de Satisfacción* (Biere *et al*., 2009), el fascículo de Donald Knuth (2015) sobre satisfacción, y las *Conferencias Internacionales regulares sobre Teoría y Aplicaciones de Pruebas de Satisfacción*, conocidas como SAT.

La idea de construir agentes con lógica proposicional se remonta al artículo seminal de McCulloch y Pitts (1943), que es bien conocido por iniciar el campo de las redes neuronales, en realidad estaba preocupado por la implementación de un diseño de agente basado en circuitos booleanos en el cerebro. Stan Rosenschein (Rosenschein, 1985; Kaelbling y Rosenschein, 1990) desarrolló formas de compilar agentes basados en circuitos a partir de descripciones declarativas del entorno de la tarea. Rod Brooks (1986, 1989) demuestra la efectividad de los diseños basados en circuitos para controlar robots (ver Capítulo 26). Brooks (1991) argumenta que los diseños basados en circuitos son *todo* lo que se necesita para la IA, que la representación y el razonamiento son engorrosos, costosos e innecesarios. En nuestra opinión, tanto el razonamiento como los circuitos son necesarios. Williams *et al*. (2003) describen un agente híbrido, no muy diferente de nuestro agente de wumpus, que controla las naves espaciales de la NASA, planificando secuencias de acciones y diagnosticando y recuperándose de fallas.

El problema general de realizar un seguimiento de un entorno parcialmente observable se introdujo para las representaciones basadas en estados en el Capítulo 4. Su instanciación para representaciones proposicionales fue estudiada por Amir y Russell (2003), quienes identificaron varias clases de entornos que admiten algoritmos eficientes de estimación de estado y demostraron que para varias otras clases el problema es intratable. El problema de **proyección temporal**, que implica determinar qué proposiciones se mantienen verdaderas después de que se ejecuta una secuencia de acciones, puede verse como un caso especial de estimación de estado con percepciones vacías. Muchos autores han estudiado este problema debido a su importancia en la planificación; algunos resultados de dureza importantes fueron establecidos por Liberatore (1997). La idea de representar un estado de creencia con proposiciones se puede rastrear hasta Wittgenstein (1922).

Proyección temporal

El enfoque de la estimación del estado lógico utilizando índices temporales en variables proposicionales fue propuesto por Kautz y Selman (1992). Las generaciones posteriores de SATPlan pudieron aprovechar los avances en los solucionadores SAT y siguen siendo una de las formas más efectivas de resolver problemas de planificación difíciles (Kautz, 2006).

El **problema del marco** fue reconocido por primera vez por McCarthy y Hayes (1969). Muchos investigadores consideraron el problema insoluble dentro de la lógica de primer orden, y estimuló una gran cantidad de investigación en lógicas no monótonas. Filósofos desde Dreyfus (1972) hasta Crockett (1994) han citado el problema del marco como un síntoma del fracaso inevitable de toda la empresa de la IA. La solución del problema del marco con axiomas de estado

sucesor se debe a Ray Reiter (1991). Thielscher (1999) identifica el problema del marco inferencial como una idea separada y proporciona una solución. En retrospectiva, se puede ver que los agentes de Rosenschein (1985) estaban usando circuitos que implementaban axiomas de estado sucesor, pero Rosenschein no advirtió que el problema del marco se resolvía en gran medida de esta manera.

Los solucionadores proposicionales modernos se han aplicado a una variedad de aplicaciones industriales, como la síntesis de hardware de computadora (Nowick *et al*., 1993). El comprobador de satisfacción SATMC se utilizó para detectar una vulnerabilidad previamente desconocida en un protocolo de inicio de sesión de navegador web (Armando *et al*., 2008).

El mundo del wumpus fue inventado como un juego por Gregory Yob (1975). Irónicamente, Yob lo desarrolló porque estaba aburrido con los juegos jugados en una cuadrícula rectangular: puso su wumpus en un dodecaedro, y nosotros lo volvimos a poner en la aburrida cuadrícula antigua. Michael Genesereth sugirió que el mundo del wumpus se usara como un banco de pruebas para agentes.

CAPÍTULO 8

LÓGICA DE PRIMER ORDEN

En el que nos damos cuenta de que el mundo está bendecido con muchos objetos, algunos de los cuales están relacionados con otros objetos; un mundo en el que nos esforzamos por razonar sobre ellos.

La lógica proposicional bastó para ilustrar los conceptos básicos de lógica, inferencia y agentes basados en conocimiento. No obstante, la lógica proposicional es limitada en lo que puede decir. En este capítulo examinamos la **lógica de primer orden**,[1] que puede representar mucho más de manera concisa. Comenzamos la sección 8.1 con una discusión de los lenguajes de representación en general; la Sección 8.2 cubre la sintaxis y semántica de la lógica de primer orden; las Secciones 8.3 y 8.4 ilustran el uso de la lógica de primer orden para representaciones simples.

Lógica de primer orden

8.1 Revisión de la representación

En esta sección, discutimos la naturaleza de los lenguajes de representación. Los lenguajes de programación (como C++ o Java o Python) son la clase más grande de lenguajes formales en uso común. Las estructuras de datos dentro de los programas se pueden usar para representar hechos; por ejemplo, un programa podría usar una matriz 4×4 para representar el contenido del mundo del wumpus. Así, la declaración del lenguaje de programación *Mundo*[2,2] ← *Pozo* es una forma natural de afirmar que hay un pozo en la casilla [2,2]. Juntar una cadena de tales declaraciones es suficiente para ejecutar una simulación del mundo del wumpus.

Lo que los lenguajes de programación carecen es de un mecanismo general para derivar hechos de otros hechos; cada actualización a una estructura de datos se realiza mediante un procedimiento específico del dominio cuyos detalles son derivados por el programador de su propio conocimiento del dominio. Este enfoque procedimental puede contrastarse con la naturaleza **declarativa** de la lógica proposicional, en la que el conocimiento y la inferencia son separados, y la inferencia es por completo independiente del dominio. Las bases de datos SQL toman una mezcla de conocimiento declarativo y procedimental.

Una segunda desventaja de las estructuras de datos en programas (y de las bases de datos) es la falta de una forma fácil de decir, por ejemplo, "Hay un pozo en [2,2] o [3,1]" o "Si el wumpus está en [1,1], entonces no está en [2,2]". Los programas pueden almacenar un solo valor para cada variable, pero algunos sistemas permiten que el valor sea "desconocido," aunque carecen de la expresividad requerida para manejar directamente información parcial.

La lógica proposicional es un lenguaje declarativo porque su semántica se basa en una relación de verdad entre sentencias y mundos posibles. También tiene suficiente poder expresivo para lidiar con información parcial, usando disyunción y negación. La lógica proposicional tiene una tercera propiedad que es deseable en los lenguajes de representación, a saber, la **composicionalidad**. En un lenguaje composicional, el significado de una sentencia es una función del significado de sus

Composicionalidad

[1] La lógica de primer orden también se llama **cálculo de predicados de primer orden**; puede abreviarse como **FOL** o **FOPC**.

partes. Por ejemplo, el significado de "$S_{1,4} \wedge S_{1,2}$" está relacionado con los significados de "$S_{1,4}$" y "$S_{1,2}$." Sería muy extraño si "$S_{1,4}$" significara que hay un hedor en la casilla [1,4] y "$S_{1,2}$" significara que hay un hedor en la casilla [1,2], pero "$S_{1,4} \wedge S_{1,2}$" significara que Francia y Polonia empataron 1–1 en el partido de clasificación de hockey sobre hielo de la semana pasada.

Sin embargo, la lógica proposicional, como una representación factorizada, carece del poder expresivo para describir de manera concisa un entorno con muchos objetos. Por ejemplo, nos vimos obligados a escribir una regla separada sobre brisas y pozos para cada casilla, tales como

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

En español, por otro lado, parece muy fácil decir, de una vez por todas, "Las casillas adyacentes a los pozos tienen brisa". La sintaxis y semántica del español hacen posible describir el entorno de manera concisa: el español, como la lógica de primer orden, es una representación estructurada.

8.1.1 El lenguaje del pensamiento

Los lenguajes naturales (como el inglés o el español) son muy expresivos de hecho. Logramos escribir casi todo este libro en lenguaje natural, con solo lapsos ocasionales en otros lenguajes (principalmente matemáticas y diagramas). Hay una larga tradición en lingüística y filosofía del lenguaje que ve el lenguaje natural como un lenguaje de representación de conocimiento declarativo. Si pudiéramos descubrir las reglas para el lenguaje natural, podríamos usarlas en sistemas de representación y razonamiento y obtener el beneficio de las miles de millones de páginas que se han escrito en lenguaje natural.

La visión moderna del lenguaje natural es que sirve como medio de *comunicación* en lugar de pura representación. Cuando un hablante señala y dice, "¡Mira!" el oyente llega a saber que, digamos, Superman finalmente ha aparecido sobre los tejados. Sin embargo, no querríamos decir que la sentencia "¡Mira!" representa ese hecho. Más bien, el significado de la sentencia depende tanto de la sentencia en sí como del *contexto* en el que se habló la sentencia. Claramente, uno no podría almacenar una sentencia como "¡Mira!" en una base de conocimiento (BC) y esperar recuperar su significado sin también almacenar una representación del contexto, lo que plantea la pregunta de cómo se puede representar el contexto en sí.

Los lenguajes naturales también sufren de *ambigüedad*, un problema para un lenguaje de representación. Como lo pone Pinker (1995): "*When people think about* spring, *surely they are not confused as to whether they are thinking about a season or something that goes* boing*—and if one word can correspond to two thoughts, thoughts can't be words*" ["Cuando las personas piensan en la *primavera*, seguramente no están confundidas en cuanto a si están pensando en una estación o en algo que hace *boing*, y si una palabra puede corresponder a dos pensamientos, los pensamientos no pueden ser palabras"].

La famosa **hipótesis de Sapir–Whorf,** Whorf (1956), afirma que nuestra comprensión del mundo está fuertemente influida por el lenguaje que hablamos. Es cierto que diferentes comunidades de habla dividen el mundo de manera diferente. Los franceses tienen dos palabras "chaise" y "fauteuil," para un concepto que los hablantes de inglés cubren con una sola: "chair." Pero los hablantes de inglés pueden reconocer fácilmente la categoría *fauteuil* y darle un nombre, algo así como "sillón de brazos abiertos", ¿realmente hace una diferencia el lenguaje? Whorf se basó principalmente en la intuición y la especulación, y sus ideas han sido en gran parte desestimadas, pero en los años transcurridos tenemos datos reales de estudios antropológicos, psicológicos y neurológicos.

Por ejemplo, ¿puede recordar cuál de las siguientes dos frases formó la apertura de la Sección 8.1?

> "Comenzamos la sección 8.1 con una discusión de los lenguajes de representación en general..."
>
> "Esta sección cubre el tema de los lenguajes de representación del conocimiento..."

Wanner (1974) realizó un experimento similar y encontró que los sujetos hicieron la elección correcta al nivel de azar cerca del 50 % de las veces, pero recordaron el contenido de lo que leyeron con una precisión superior al 90 %. Esto sugiere que las personas interpretan las palabras que leen y forman una representación interna no verbal, y que las palabras exactas son irrelevantes.

Es más interesante el caso en el que un concepto está completamente ausente en un idioma. Los hablantes del idioma aborigen australiano Guugu Yimithirr no tienen palabras para direcciones relativas (o *egocéntricas*), como frente, atrás, derecha o izquierda. En su lugar, utilizan direcciones absolutas, diciendo, por ejemplo, el equivalente a "Tengo dolor en mi brazo norte". Esta diferencia en el lenguaje hace una diferencia en el comportamiento: los hablantes de Guugu Yimithirr son mejores para navegar en terreno abierto, mientras que los hablantes de inglés son mejores para colocar el tenedor a la derecha del plato.

El lenguaje también parece influir en el pensamiento a través de características gramaticales aparentemente arbitrarias como el género de los sustantivos. Por ejemplo, "puente" es masculino en español pero femenino en alemán. Boroditsky (2003) pidió a los sujetos que eligieran adjetivos en inglés para describir una fotografía de un puente en particular. Los hablantes de español eligieron *grande*, *peligroso*, *fuerte* e *imponente*, mientras que los hablantes de alemán eligieron *hermoso*, *elegante*, *frágil* y *delgado*.

Las palabras pueden servir como puntos de anclaje que afectan cómo percibimos el mundo. Loftus y Palmer (1974) mostraron a sujetos experimentales una película de un accidente automovilístico. A los sujetos que se les preguntó "¿A qué velocidad iban los coches cuando se contactaron entre sí?" informaron un promedio de 32 mph, mientras que a los sujetos a los que se les hizo la pregunta con la palabra "chocaron" en lugar de "contactaron" informaron 41mph para los mismos coches en la misma película. En general, hay diferencias medibles pero pequeñas en el procesamiento cognitivo por parte de los hablantes de diferentes idiomas, pero no hay evidencia convincente de que esto conduzca a una diferencia importante en la visión del mundo.

En un sistema de razonamiento lógico que utiliza la forma normal conjuntiva (FNC), podemos ver que las formas lingüísticas "$\neg(A \vee B)$" y "$\neg A \wedge \neg B$" son las mismas porque podemos mirar dentro del sistema y ver que ambas sentencias se almacenan como la misma forma FNC canónica. Está empezando a ser posible hacer algo similar con el cerebro humano. Mitchell *et al.* (2008) colocaron a los sujetos en una máquina de imagen por resonancia magnética funcional (fMRI), les mostraron palabras como "apio" e hicieron imágenes de sus cerebros. Un programa de aprendizaje automático entrenado en pares (palabra, imagen) fue capaz de predecir correctamente el 77 % de las veces en tareas de elección binaria (por ejemplo, "apio" o "avión"). El sistema incluso puede predecir a niveles por encima del azar para palabras de las que nunca ha visto una imagen de fMRI antes (considerando las imágenes de palabras relacionadas) y para personas que nunca han visto antes (demostrando que la fMRI revela algún nivel de representación común entre las personas). Este tipo de trabajo todavía está en sus inicios, pero la fMRI (y otras tecnologías de imagen como la electrofisiología intracraneal (Sahin *et al*., 2009)) promete darnos ideas mucho más concretas de cómo son las representaciones del conocimiento humano.

Desde el punto de vista de la lógica formal, representar el mismo conocimiento de dos maneras diferentes no hace ninguna diferencia; los mismos hechos serán derivables de cualquier representación. En la práctica, sin embargo, una representación podría requerir menos pasos para derivar una conclusión, lo que significa que un razonador con recursos limitados podría llegar a la conclusión utilizando una representación pero no la otra. Para tareas *no deductivas* como el aprendizaje a partir de la experiencia, los resultados dependen *necesariamente* de la forma de las representaciones utilizadas. En el Capítulo 19 mostramos que cuando un programa de aprendizaje considera dos posibles teorías del mundo, ambas consistentes con todos los datos, la forma más común de romper el empate es elegir la teoría más sucinta, y eso depende del lenguaje utilizado para representar teorías. Por lo tanto, la influencia del lenguaje en el pensamiento es inevitable para cualquier agente que aprenda.

8.1.2 Combinando lo mejor de los lenguajes formales y naturales

Podemos adoptar la base de la lógica proposicional —una semántica declarativa, composicional independiente del contexto y no ambigua— y construir una lógica más expresiva sobre esa base al tomar ideas representacionales del lenguaje natural mientras evitamos sus inconvenientes. Cuando miramos la sintaxis del lenguaje natural, los elementos más obvios son los sustantivos y las frases sustantivas que se refieren a **objetos** (casillas, pozos, wumpuses) y los verbos y las frases verbales junto con los adjetivos y los adverbios que se refieren a las **relaciones** entre los objetos (es ventoso, es adyacente a, dispara). Algunas de estas relaciones son **funciones**, relaciones en las que sólo hay un "valor" para una "entrada" dada. Es fácil comenzar a enumerar ejemplos de objetos, relaciones y funciones:

Objetos
Relaciones
Funciones

- Objetos: personas, casas, números, teorías, Ronald McDonald, colores, juegos de béisbol, guerras, siglos...
- Relaciones: pueden ser relaciones unarias o **propiedades** como rojo, redondo, falso, primo, de varios pisos..., o relaciones *n*-arias más generales como hermano de, más grande que, dentro, parte de, tiene color, ocurrió después, posee, viene entre...
- Funciones: padre de, mejor amigo, tercera entrada de, uno más que, comienzo de...

Propiedades

De hecho, casi cualquier afirmación puede ser pensada como referente a objetos y propiedades o relaciones. Algunos ejemplos son:

- "Uno más dos es igual a tres".
  Objetos: uno, dos, tres, uno más dos; Relación: igual a; Función: más. ("Uno más dos" es un nombre para el objeto que se obtiene al aplicar la función "más" a los objetos "uno" y "dos". "Tres" es otro nombre para este objeto).
- "Las casillas vecinas al wumpus son olorosas".
  Objetos: wumpus, casillas; Propiedad: olorosa; Relación: vecina.
- "El malvado rey Juan gobernó Inglaterra en 1200".
  Objetos: Juan, Inglaterra, 1200; Relación: gobernó durante; Propiedades: malvado, rey.

El lenguaje de la **lógica de primer orden**, cuya sintaxis y semántica definimos en la siguiente sección, se construye en torno a objetos y relaciones. Ha sido importante para las matemáticas, la filosofía y la inteligencia artificial precisamente porque esos campos, y de hecho, gran parte de la existencia humana cotidiana, pueden ser útilmente pensados en términos de lidiar con objetos y las relaciones entre ellos. La lógica de primer orden también puede expresar hechos sobre *algunos* o *todos* los objetos en el universo. Esto permite representar leyes o reglas generales, como la afirmación "Las casillas vecinas al wumpus son olorosas".

Compromiso ontológico

La principal diferencia entre la lógica proposicional y la de primer orden radica en el **compromiso ontológico** que hace cada lenguaje, es decir, lo que asume sobre la naturaleza de la *realidad*. Matemáticamente, este compromiso se expresa a través de la naturaleza de los modelos formales con respecto a los cuales se define la verdad de las sentencias. Por ejemplo, la lógica proposicional asume que hay hechos que se mantienen o no en el mundo. Cada hecho puede estar en uno de dos estados, verdadero o falso, y cada modelo asigna *verdadero* o *falso* a cada símbolo de proposición (ver Sección 7.4.2). La lógica de primer orden asume más; a saber, que el mundo consiste en objetos con ciertas relaciones entre ellos que se mantienen o no. (Ver Figura 8.1). Los modelos formales son correspondientemente más complicados que los de la lógica proposicional.

Lenguaje	Compromiso ontológico (Qué existe en el mundo)	Compromiso epistemológico (Qué cree un agente sobre los hechos)
Lógica proposicional	hechos	verdadero/falso/desconocido
Lógica de primer orden	hechos, objetos, relaciones	verdadero/falso/desconocido
Lógica temporal	hechos, objetos, relaciones, tiempos	verdadero/falso/desconocido
Teoría de la probabilidad	hechos	grado de creencia $\in [0,1]$
Lógica difusa	hechos con grado de verdad $\in [0,1]$	valor de intervalo conocido

Figura 8.1 Lenguajes formales y sus compromisos ontológicos y epistemológicos.

Este compromiso ontológico es una gran fortaleza de la lógica (tanto proposicional como de primer orden), porque nos permite comenzar con declaraciones verdaderas e inferir otras declaraciones verdaderas. Es especialmente poderoso en dominios donde cada proposición tiene límites claros, como las matemáticas o el mundo del wumpus, donde una casilla tiene o no tiene un pozo; no hay posibilidad de una casilla con una indentación vagamente parecida a un pozo. Pero en el mundo real, muchas proposiciones tienen límites vagos: ¿Es Viena una gran ciudad? ¿Este restaurante sirve comida deliciosa? ¿Es esa persona alta? Depende de a quién le preguntes, y su respuesta podría ser "más o menos".

Una respuesta es refinar la representación: si una línea cruda que divide las ciudades en "grandes" y "no grandes" deja demasiada información para la aplicación en cuestión, entonces se puede aumentar el número de categorías de tamaño o usar un símbolo de función *Población*. Otra solución propuesta proviene de la **lógica difusa**, que hace el compromiso ontológico de que las proposiciones de la lógica difusa tienen un **grado de verdad** entre 0 y 1. Por ejemplo, la frase "Viena es una gran ciudad" podría ser verdadera en grado 0.8 en lógica difusa, mientras que "París es una gran ciudad" podría ser verdadera en grado 0.9. Esto se corresponde mejor con nuestra concepción intuitiva del mundo, pero hace más difícil hacer inferencias: en lugar de una regla para determinar la verdad de $A \wedge B$, la lógica difusa necesita reglas diferentes dependiendo del dominio. Otra posibilidad, cubierta en la Sección 24.1, es asignar cada concepto a un punto en un espacio multidimensional, y luego medir la distancia entre el concepto "gran ciudad" y el concepto "Viena" o "París".

Lógica difusa

Grado de verdad

Varias lógicas de propósito especial hacen aún más compromisos ontológicos; por ejemplo, la **lógica temporal** asume que los hechos ocurren en momentos particulares y que esos *tiempos* (que pueden ser puntos o intervalos) están ordenados. Así, las lógicas de propósito especial dan a ciertos tipos de objetos (y los axiomas sobre ellos) un estatus "de primera clase" dentro de la lógica, en lugar de sólo definirlos dentro de la base de conocimientos (BC). La **lógica de orden superior** considera las relaciones y funciones a las que la lógica de primer orden se refiere como objetos en sí mismos. Esto permite hacer afirmaciones sobre *todas* las relaciones, por ejemplo, se podría desear definir qué significa que una relación sea transitiva. A diferencia de la mayoría de las lógicas de propósito especial, la lógica de orden superior es estrictamente más expresiva que la lógica de primer orden, en el sentido de que algunas oraciones de la lógica de orden superior no pueden ser expresadas por ningún número finito de sentencias de la lógica de primer orden.

Lógica temporal

Lógica de orden superior

Una lógica también puede ser caracterizada por sus **compromisos epistemológicos**, los posibles estados de conocimiento que permite con respecto a cada hecho. En ambas lógicas, proposicional y de primer orden, una sentencia representa un hecho y el agente o bien cree que la sentencia es verdadera, cree que es falsa, o no tiene opinión. Estas lógicas, por lo tanto, tienen tres posibles estados de conocimiento con respecto a cualquier sentencia.

Compromisos epistemológicos

Por otro lado, los sistemas que utilizan la **teoría de la probabilidad** pueden tener cualquier grado de *creencia*, o *probabilidad subjetiva*, que varía desde 0 (total incredulidad) hasta 1 (total creencia). Es importante no confundir el grado de creencia en la teoría de la probabilidad con el grado de verdad en la lógica difusa. De hecho, algunos sistemas difusos permiten la incertidumbre (grado de creencia) acerca de los grados de verdad. Por ejemplo, un agente del mundo del wumpus probabilístico podría creer que el wumpus está en [1,3] con probabilidad 0.75 y en [2,3] con probabilidad 0.25 (aunque el wumpus está definitivamente en una casilla particular).

8.2 Sintaxis y semántica de la lógica de primer orden

Comenzamos esta sección especificando con mayor precisión la forma en que los mundos posibles de la lógica de primer orden reflejan el compromiso ontológico con los objetos y las relaciones. Luego introducimos los diversos elementos del lenguaje, explicando su semántica a medida que avanzamos. Los puntos principales son cómo el lenguaje facilita representaciones concisas y cómo su semántica conduce a procedimientos de razonamiento sólidos.

8.2.1 Modelos para la lógica de primer orden

El Capítulo 7 dijo que los modelos de un lenguaje lógico son las estructuras formales que constituyen los mundos posibles en consideración. Cada modelo vincula el vocabulario de las oraciones lógicas a elementos del mundo posible, de modo que se puede determinar la verdad de cualquier sentencia. Así, los modelos para la lógica proposicional vinculan los símbolos de proposición a valores de verdad predefinidos.

Dominio
Elementos de dominio

Los modelos para la lógica de primer orden son mucho más interesantes. ¡Primero, tienen objetos en ellos! El **dominio** de un modelo es el conjunto de objetos o **elementos de dominio** que contiene. Se requiere que el dominio *no sea vacío*, cada mundo posible debe contener al menos un objeto. (Ver Ejercicio 8.EMPT para una discusión sobre mundos vacíos). Matemáticamente hablando, no importa *qué* son estos objetos, todo lo que importa es *cuántos* hay en cada modelo particular, pero para fines pedagógicos usaremos un ejemplo concreto. La Figura 8.2 muestra un modelo con cinco objetos: Ricardo Corazón de León, Rey de Inglaterra de 1189 a 1199; su hermano menor, el malvado Rey Juan, que gobernó de 1199 a 1215; las piernas izquierdas de Ricardo y Juan; y una corona.

Tupla

Los objetos en el modelo pueden estar *relacionados* de varias maneras. En la figura, Ricardo y Juan son hermanos. Formalmente hablando, una relación es simplemente el conjunto de **tuplas** de objetos que están relacionados. (Una tupla es una colección de objetos dispuestos en un orden fijo y se escribe con paréntesis angulares rodeando los objetos). Por lo tanto, la relación de hermandad en este modelo es el conjunto

$$\{\langle \text{Ricardo Corazón de León, Rey Juan}\rangle, \langle \text{Rey Juan, Ricardo Corazón de León}\rangle\}. \tag{8.1}$$

(Aquí hemos nombrado los objetos en español, pero puede, si lo desea, sustituir mentalmente las imágenes por los nombres). La corona está en la cabeza del Rey Juan, por lo que la relación "en la cabeza" contiene sólo una tupla, ⟨la corona, el Rey Juan⟩. Las relaciones "hermano" y "en la cabeza" son relaciones binarias, es decir, relacionan pares de objetos. El modelo también contiene relaciones unarias, o propiedades: la propiedad "persona" es verdadera tanto para Ricardo como para Juan; la propiedad "rey" es verdadera sólo para Juan (presumiblemente porque Ricardo está muerto en este punto); y la propiedad "corona" es verdadera sólo para la corona.

Ciertos tipos de relaciones se consideran mejor como funciones, en el sentido de que un objeto dado debe estar relacionado exactamente con un objeto de esta manera. Por ejemplo, cada persona

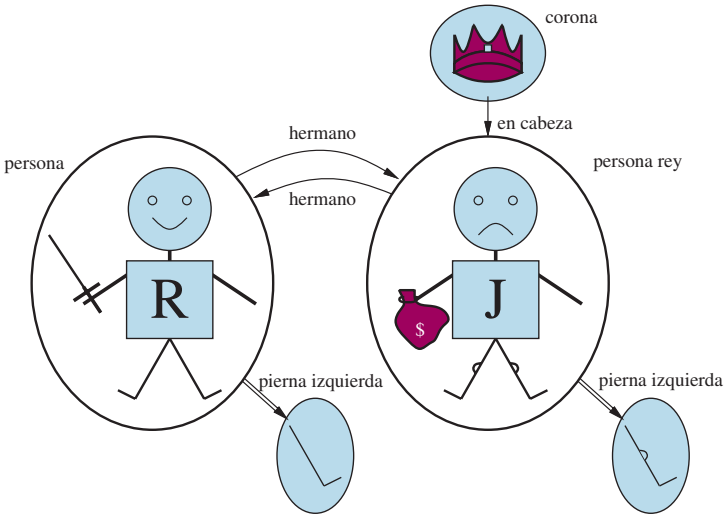


Figura 8.2 Un modelo que contiene cinco objetos, dos relaciones binarias (hermano y en-cabeza), tres relaciones unarias (persona, rey y corona), y una función unaria (pierna-izquierda).

tiene una pierna izquierda, por lo que el modelo tiene una función "pierna izquierda" unaria, una aplicación de una tupla de un elemento a un objeto, que incluye las siguientes asociaciones:

$$\langle \text{Ricardo Corazón de León} \rangle \rightarrow \text{La pierna izquierda de Ricardo}$$
$$\langle \text{Rey Juan} \rangle \rightarrow \text{La pierna izquierda de Juan}. \tag{8.2}$$

Funciones totales

Estrictamente hablando, los modelos en lógica de primer orden requieren **funciones totales**, es decir, debe haber valor para cada tupla de entrada. Así, la corona debe tener una pierna izquierda y también cada una de las piernas izquierdas. Existe una solución técnica a este incómodo problema que implica un objeto "invisible" adicional que es la pierna izquierda de todo lo que no tiene pierna izquierda, incluyéndose a sí mismo. Por fortuna, siempre y cuando no se hagan afirmaciones sobre las piernas izquierdas de las cosas que no tienen piernas izquierdas, estas tecnicidades no son importantes.

Hasta ahora, hemos descrito los elementos que pueblan los modelos para la lógica de primer orden. La otra parte esencial de un modelo es el enlace entre esos elementos y el vocabulario de las oraciones lógicas, que explicamos a continuación.

8.2.2 Símbolos e interpretaciones

Ahora consideramos la sintaxis de la lógica de primer orden. El lector impaciente puede obtener una descripción completa de la gramática formal en la Figura 8.3.

Los elementos sintácticos básicos de la lógica de primer orden son los símbolos que representan objetos, relaciones y funciones. Por lo tanto, los símbolos vienen en tres tipos: **símbolos constantes**, que representan objetos; **símbolos de predicado**, que representan relaciones; y **símbolos de función**, que representan funciones. Adoptamos la convención de que estos símbolos comenzarán con letras mayúsculas. Por ejemplo, podríamos usar los símbolos constantes *Ricardo* y *Juan*; los símbolos de predicado *Hermano*, *EnLaCabeza*, *Persona*, *Rey*, y *Corona*; y el símbolo de función *PiernaIzquierda*. Al igual que con los símbolos de proposición, la elección de los nombres depende por completo del usuario. Cada símbolo de predicado y función viene con una **aridad** que fija el número de argumentos.

Símbolos constantes
Símbolos de predicado
Símbolos de función
Aridad

Sentencia	→	*SentenciaAtómica* | *SentenciaCompleja*
SentenciaAtómica	→	*Predicado* | *Predicado*(*Término*,...) | *Término* = *Término*
SentenciaCompleja	→	(*Sentencia*)
	|	¬ *Sentencia*
	|	*Sentencia* ∧ *Sentencia*
	|	*Sentencia* ∨ *Sentencia*
	|	*Sentencia* ⇒ *Sentencia*
	|	*Sentencia* ⇔ *Sentencia*
	|	*Cuantificador Variable*,... *Sentencia*
Término	→	*Función*(*Término*,...)
	|	*Constante*
	|	*Variable*
Cuantificador	→	∀ | ∃
Constante	→	$A \mid X_1 \mid Juan \mid \cdots$
Variable	→	$a \mid x \mid s \mid \cdots$
Predicado	→	*Verdadero* | *Falso* | *Sigue* | *Ama* | *Llueve* | ···
Función	→	*Madre* | *PiernaIzquierda* | ···
PRECEDENCIA DE OPERADORES	:	¬,=,∧,∨,⇒,⇔

Figura 8.3 La sintaxis de la lógica de primer orden con igualdad, especificada en forma de Backus–Naur (ver página 1030 si no está familiarizado con esta notación). Las precedencias de los operadores están especificadas, de mayor a menor. La precedencia de los cuantificadores es tal que un cuantificador se mantiene ante todo lo que está a la derecha de la misma.

Cada modelo debe proporcionar la información necesaria para determinar si una sentencia dada es verdadera o falsa. Por lo tanto, además de sus objetos, relaciones y funciones, cada modelo incluye una **interpretación** que especifica exactamente qué objetos, relaciones y funciones son referidos por los símbolos constantes, de predicado y de función. Una posible interpretación para nuestro ejemplo, que un lógico llamaría la **interpretación prevista**, es la siguiente:

Interpretación

Interpretación prevista

- *Ricardo* se refiere a *Ricardo Corazón de León* y *Juan* se refiere al malvado Rey Juan.
- *Hermano* se refiere a la relación de hermandad, es decir, el conjunto de tuplas de objetos dado en la Ecuación (8.1); *EnLaCabeza* es una relación que se mantiene entre la corona y Rey Juan; *Persona*, *Rey*, y *Corona* son relaciones unarias que identifican personas, reyes y coronas.
- *PiernaIzquierda* se refiere a la función "pierna izquierda" tal como se define en la Ecuación (8.2).

Por supuesto, hay muchas otras posibles interpretaciones. Por ejemplo, una interpretación mapea a *Ricado* con la corona y a *Juan* con la pierna izquierda de Rey Juan. Hay cinco objetos en el modelo, por lo que hay 25 posibles interpretaciones sólo para los símbolos constantes *Ricardo* y *Juan*. Observe que no todos los objetos necesitan tener un nombre, por ejemplo, la interpretación prevista no nombra la corona ni las piernas. También es posible que un objeto tenga varios

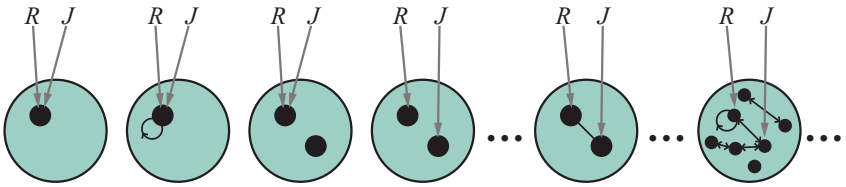


Figura 8.4 Algunos miembros del conjunto de todos los modelos para un lenguaje con dos símbolos constantes, *R* y *J*, y un símbolo de relación binaria. La interpretación de cada símbolo constante se muestra con una flecha gris. Dentro de cada modelo, los objetos relacionados están conectados por flechas.

nombres; hay una interpretación bajo la cual tanto *Ricardo* como *Juan* se refieren a la corona.[2] Si encuentra esta posibilidad confusa, recuerde que en lógica proposicional es perfectamente posible tener un modelo en el que *Nublado* y *Soleado* sean ambos verdaderos; es el trabajo de la base de conocimientos descartar modelos que son inconsistentes con nuestro conocimiento.

En resumen, un modelo en lógica de primer orden consta de un conjunto de objetos y una interpretación que mapea símbolos constantes a objetos, símbolos de función a funciones sobre esos objetos, y símbolos de predicado a relaciones. Al igual que con la lógica proposicional, la consecuencia lógica, la validez, etc., se definen en términos de *todos los modelos posibles*. Para tener una idea de cómo se ve el conjunto de todos los modelos posibles, vea la Figura 8.4. Esta figura muestra que los modelos varían en cuántos objetos contienen, de uno a infinito, y en la forma en que los símbolos constantes se mapean a objetos.

Debido a que el número de modelos de primer orden es ilimitado, no podemos verificar la consecuencia lógica enumerándolos todos (como lo hicimos para la lógica proposicional). Incluso si el número de objetos está restringido, el número de combinaciones puede ser muy grande. (Ver Ejercicio 8.MCNT.) Para el ejemplo en la Figura 8.4, hay 137 506 194 466 modelos con seis objetos o menos.

8.2.3 Términos

Término

Un **término** es una expresión lógica que se refiere a un objeto. Los símbolos constantes son términos, pero no siempre es conveniente tener un símbolo distinto para nombrar cada objeto. En español podemos usar la expresión "La pierna izquierda del Rey Juan" en lugar de darle un nombre a su pierna. Para eso sirven los símbolos de función: en vez de usar un símbolo constante, usamos *PiernaIzquierda*(*Juan*).[3]

En el caso general, un término complejo se forma con un símbolo de función seguido de una lista de términos como argumentos para el símbolo de función. Es importante recordar que un término complejo es sólo un tipo complicado de nombre. No es una "llamada a subrutina" que "devuelve un valor". No hay una subrutina *PiernaIzquierda* que tome a una persona como entrada y devuelva una pierna. Podemos razonar sobre las piernas izquierdas (por ejemplo, estableciendo la regla general de que todos tienen una y luego deduciendo que John debe tener una) sin proporcionar nunca una definición de *PiernaIzquierda*. Esto es algo que no se puede hacer con las subrutinas en los lenguajes de programación.

[2] Más adelante, en la Sección 8.2.8, examinamos una semántica en la que cada objeto debe tener exactamente un nombre.

[3] Las λ-**expresiones** (expresiones lambda) proporcionan una notación útil en la que se construyen nuevos símbolos de función "sobre la marcha". Por ejemplo, la función que eleva al cuadrado su argumento se puede escribir como $(\lambda x : x \times x)$ y se puede aplicar a argumentos como cualquier otro símbolo de función. Una λ-expresión también se puede definir y usar como un símbolo de predicado . El operador `lambda` en Lisp y Python tiene exactamente el mismo papel. Observe que el uso de λ de esta manera *no* aumenta el poder expresivo formal de la lógica de primer orden, porque cualquier sentencia que incluya una λ-expresión se puede reescribir "enchufando" sus argumentos para obtener una sentencia equivalente.

La semántica formal de los términos es sencilla. Considere un término $f(t_1,...,t_n)$. En este caso, el símbolo de función f se refiere a alguna función en el modelo (llámela F); los términos argumento se refieren a objetos en el dominio (llámalos $d_1,...,d_n$); y el término en su conjunto se refiere al objeto que es el valor de la función F aplicada a $d_1,..., d_n$. Por ejemplo, suponga que el símbolo de función *PiernaIzquierda* se refiere a la función de la Ecuación (8.2) y *John* se refiere al Rey Juan, entonces *PiernaIzquierda* (*Juan*) se refiere a la pierna izquierda del Rey Juan. Así, la interpretación fija el referente de cada término.

8.2.4 Sentencias atómicas

Sentencia atómica
Atómica

Ahora que tenemos términos para referirnos a objetos y símbolos de predicado para referirnos a relaciones, podemos combinarlos para hacer **sentencias atómicas** que afirmen hechos. Una **sentencia atómica** (o átomo para abreviar) se forma a partir de un símbolo de predicado seguido opcionalmente por una lista de términos entre paréntesis, como

Hermano(*Ricardo*,*Juan*).

Esto afirma, bajo la interpretación prevista dada antes, que Ricardo Corazón de León es el hermano del Rey Juan.[4] Las oraciones atómicas pueden tener términos complejos como argumentos. Así,

Matrimonio(*Padre*(*Ricardo*),*Madre*(*Juan*))

afirma que el padre de Ricardo Corazón de León está casado con la madre del Rey Juan (de nuevo, bajo una interpretación adecuada).[5]

*Una sentencia atómica es **verdadera** en un modelo dado si la relación a la que se refiere el símbolo de predicado se mantiene entre los objetos a los que se refieren los argumentos.*

8.2.5 Oraciones complejas

Podemos usar **conectivos lógicos** para construir oraciones más complejas, con la misma sintaxis y semántica que en el cálculo proposicional. Aquí hay cuatro oraciones que son verdaderas en el modelo de la Figura 8.2 bajo nuestra interpretación prevista:

$\neg$*Hermano*(*PiernaIzquierda*(*Ricardo*),*Juan*)
Hermano(*Ricardo*,*Juan*) $\wedge$ *Hermano*(*Juan*,*Ricardo*)
Rey(*Ricardo*) $\vee$ *Rey*(*Juan*)
$\neg$*Rey*(*Ricardo*) $\Rightarrow$ *Rey*(*Juan*).

8.2.6 Cuantificadores

Cuantificador

Si tenemos una lógica que permite objetos, es natural expresar propiedades de colecciones enteras de objetos, en lugar de enumerar los objetos por su nombre. Los **cuantificadores** nos permiten hacer esto. La lógica de primer orden contiene dos cuantificadores estándar: *universal* y *existencial*.

Cuantificación universal (∀)

Recuerde la dificultad en el Capítulo 7 con la expresión de reglas generales en lógica proposicional. Reglas como "Las casillas vecinas al wumpus son olorosas " y "Todos los reyes son personas", son

[4] Por lo general, seguimos la convención de orden de argumentos que dice que $P(x,y)$ se lee como "x es un P de y".

[5] Esta ontología sólo reconoce un padre y una madre para cada persona. Una ontología más compleja podría reconocer madre biológica, madre de nacimiento, madre adoptiva, etc.

el pan con mantequilla de la lógica de primer orden. Tratamos el primero de estos en la Sección 8.3. La segunda regla, "Todos los reyes son personas", se escribe en lógica de primer orden como

$$\forall x \; Rey(x) \Rightarrow Persona(x).$$

El **cuantificador universal** $\forall$ se pronuncia generalmente "Para todo...". (Recuerde que la A al revés significa "todo "). Así, la sentencia dice, "Para todo x, si x es un rey, entonces x es una persona". El símbolo x se llama **variable**. Por convención, las variables son letras minúsculas. Una variable es un término por sí misma, y como tal puede servir como el argumento de una función; por ejemplo, $PiernaIzquierda(x)$. Un término sin variables se llama **término base**.

Cuantificador universal

Variable

Término base

Intuitivamente, la sentencia $\forall x P$, donde P es cualquier sentencia lógica, dice que P es verdadera para cada objeto x. Más precisamente, $\forall x P$ es verdadera en un modelo dado si P es verdadera en todas las posibles **interpretaciones extendidas** construidas a partir de la interpretación dada en el modelo, donde cada interpretación extendida especifica un elemento del dominio al que se refiere x.

Interpretaciones extendidas

Esto suena complicado, pero es realmente sólo una forma cuidadosa de declarar el significado intuitivo de la cuantificación universal. Considere el modelo mostrado en la Figura 8.2 y la interpretación prevista que lo acompaña. Podemos extender la interpretación de cinco maneras:

$x \rightarrow$ Ricardo Corazón de León,
$x \rightarrow$ Rey Juan,
$x \rightarrow$ La pierna izquierda de Ricardo,
$x \rightarrow$ La pierna izquierda de Juan,
$x \rightarrow$ la corona.

La sentencia cuantificada universalmente $\forall x$ Rey$(x) \Rightarrow Persona(x)$ es verdadera en el modelo original si la sentencia $Rey(x) \Rightarrow Persona(x)$ es verdadera bajo cada una de las cinco interpretaciones extendidas. Es decir, la sentencia cuantificada universalmente es equivalente a afirmar las siguientes cinco sentencias:

Ricardo Corazón de León es un rey $\Rightarrow$ Ricardo Corazón de León es una persona.
El rey Juan es un rey $\Rightarrow$ El rey Juan es una persona.
La pierna izquierda de Ricardo es un rey $\Rightarrow$ La pierna izquierda de Ricardo es una persona.
La pierna izquierda de Juan es un rey $\Rightarrow$ La pierna izquierda de Juan es una persona.
La corona es un rey $\Rightarrow$ la corona es una persona.

Observemos detenidamente este conjunto de afirmaciones. Dado que, en nuestro modelo, el rey Juan es el único rey, la segunda sentencia afirma que él es una persona, como esperaríamos. ¿Pero qué pasa con las otras cuatro sentencias, que parecen hacer afirmaciones sobre las piernas y las coronas? ¿Es esa parte del significado de "Todos los reyes son personas"? De hecho, las otras cuatro afirmaciones son verdaderas en el modelo, pero no hacen ninguna afirmación sobre las cualificaciones de persona de las piernas, las coronas, o de hecho Ricardo. Esto se debe a que ninguno de estos objetos es un rey. Al observar la tabla de verdad para $\Rightarrow$ (Figura 7.8 en la página 219), vemos que la implicación es verdadera siempre que su premisa sea falsa, *independientemente* de la verdad de la conclusión. Por lo tanto, al afirmar la sentencia cuantificada universalmente, que es equivalente a afirmar una lista completa de implicaciones individuales, terminamos afirmando la conclusión de la regla sólo para aquellos objetos para los que la premisa es verdadera y no decimos nada en absoluto sobre aquellos objetos para los que la premisa es falsa. Así, la definición de la tabla de verdad de $\Rightarrow$ resulta ser perfecta para escribir reglas generales con cuantificadores universales.

Un error común, cometido con frecuencia, aun por lectores diligentes que han leído este párrafo varias veces, es usar la conjunción en lugar de la implicación. La sentencia

$$\forall x \; Rey(x) \wedge Persona(x)$$

sería equivalente a afirmar

Ricardo Corazón de León es un rey ∧ Ricardo Corazón de León es una persona,
El rey Juan es un rey ∧ El rey Juan es una persona,
La pierna izquierda de Ricardo es un rey ∧ La pierna izquierda de Ricardo es una persona,

y así sucesivamente. Obviamente, esto no captura lo que queremos

Cuantificación existencial (∃)

Cuantificador existencial

La cuantificación universal hace declaraciones sobre cada objeto. De manera similar, podemos hacer una declaración sobre *algún* objeto sin nombrarlo, usando un **cuantificador existencial**. Para decir, por ejemplo, que el rey Juan tiene una corona en su cabeza, escribimos

$$\exists x\ Corona(x) \wedge EnLaCabeza(x, Juan).$$

$\exists x$ se pronuncia "Existe un x tal que..." o "Para algún x...".

Intuitivamente, la sentencia $\exists x\ P$ dice que P es verdadera para al menos un objeto x. Más precisamente, $\exists x\ P$ es verdadera en un modelo dado si P es verdadera *en al menos una* interpretación extendida que asigna x a un elemento del dominio. Es decir, al menos una de las siguientes es verdadera:

Ricardo Corazón de León es una corona ∧ Ricardo Corazón de León está en la cabeza de Juan;
El rey Juan es una corona ∧ El rey Juan está en la cabeza de Juan;
La pierna izquierda de Ricardo es una corona ∧ La pierna izquierda de Ricardo está en la cabeza de Juan;
La pierna izquierda de Juan es una corona ∧ La pierna izquierda de Juan está en la cabeza de Juan;
La corona es una corona ∧ la corona está en la cabeza de Juan.

La quinta afirmación es verdadera en el modelo, por lo que la sentencia cuantificada existencialmente original es verdadera en el modelo. Observe que, según nuestra definición, la sentencia también sería verdadera en un modelo en el que el rey Juan llevaba dos coronas. Esto es totalmente coherente con la sentencia original "El rey Juan tiene una corona en su cabeza."[6]

Así como ⇒ parece ser el conector natural para usar con ∀, ∧ es el conector natural para usar con ∃. Usar ∧ como el conector principal con ∀ llevó a una afirmación demasiado fuerte en el ejemplo en la sección anterior; de hecho, usar ⇒ con ∃ generalmente lleva a una afirmación muy débil. Considere la siguiente sentencia:

$$\exists x\ Corona(x) \Rightarrow EnLaCabeza(x, Juan).$$

En la superficie, esto podría parecer una representación razonable de nuestra sentencia. Aplicando la semántica, vemos que la sentencia dice que al menos una de las siguientes afirmaciones es verdadera:

Ricardo Corazón de León es una corona ⇒ Ricardo Corazón de León está en la cabeza de Juan;
El rey Juan es una corona ⇒ El rey Juan está en la cabeza de Juan;
La pierna izquierda de Ricardo es una corona ⇒ La pierna izquierda de Ricardo está en la cabeza de Juan;

y así sucesivamente. Una implicación es verdadera si tanto la premisa como la conclusión son verdaderas, *o si su premisa es falsa*; así que si Ricardo Corazón de León no es una corona, entonces la primera afirmación es verdadera y el existencial está satisfecho. Por lo tanto, una sentencia de implicación cuantificada existencialmente es verdadera siempre que cualquier objeto no cumpla con la premisa; por lo tanto, tales sentencias realmente no dicen mucho en absoluto.

[6] Existe una variante del cuantificador existencial, generalmente escrita $\exists^1$ o $\exists!$, que significa "Existe exactamente uno". El mismo significado puede expresarse usando declaraciones de igualdad.

Cuantificadores anidados

Solemos expresar sentencias más complejas con múltiples cuantificadores. El caso más simple es donde los cuantificadores son del mismo tipo. Así, "Los hermanos son familiares " se escribe como

$$\forall x\ \forall y\ \mathit{Hermano}\ (x,y) \Rightarrow \mathit{Familiar}\ (x,y).$$

Los cuantificadores consecutivos del mismo tipo pueden escribirse como un cuantificador con varias variables. Por ejemplo, para decir que la relación "familiar" es simétrica, podemos escribir

$$\forall x,y\ \mathit{Familiar}\ (x,y) \Leftrightarrow \mathit{Familiar}\ (y,x).$$

En otros casos tendremos mezclas. "Todos aman a alguien" significa que para cada persona, hay alguien a quien esa persona ama:

$$\forall x\ \exists y\ \mathit{Ama}\ (x,y).$$

Por otro lado, para decir "Hay alguien que es amado por todos", escribimos

$$\exists y\ \forall x\ \mathit{Ama}(x,y).$$

Por lo tanto, el orden de cuantificación es muy importante. Se vuelve más claro si insertamos paréntesis. $\forall x(\exists y\ \mathit{Ama}\ (x,y))$ dice que *todos* tienen una propiedad particular, a saber, la propiedad de que aman a alguien. Por otro lado, $\exists y(\forall x\ \mathit{Ama}\ (x,y))$ dice que *alguien* en el mundo tiene una propiedad particular, a saber, la propiedad de ser amado por todos.

Puede surgir cierta confusión cuando se usan dos cuantificadores con el mismo nombre de variable. Considere la frase

$$\forall x(\mathit{Corona}(x) \lor (\exists x\ \mathit{Hermano}(\mathit{Ricardo},x))).$$

Aquí la x en $\mathit{Hermano}(\mathit{Ricardo},x)$ está cuantificada *existencialmente*. La regla es que la variable pertenece al cuantificador más interno que la menciona; entonces no estará sujeta a ninguna otra cuantificación. Otra forma de pensar en esto es: $\exists x\mathit{Hermano}(\mathit{Ricardo},x)$ es una frase sobre Ricardo (que tiene un hermano), no sobre x; por lo que poner un $\forall x$ fuera de él no tiene efecto. Podría haberse escrito igualmente bien $\exists z\mathit{Hermano}(\mathit{Ricardo},z)$. Debido a que esto puede ser una fuente de confusión, siempre usaremos diferentes nombres de variables con cuantificadores anidados.

Conexiones entre ∀ y ∃

Los dos cuantificadores están íntimamente conectados entre sí a través de la negación. Afirmar que a todos les disgustan las chirivías es lo mismo que afirmar que no existe alguien a quien le gusten, y viceversa:

$$\forall x\ \neg \mathit{Gusta}(x,\mathit{Chirivías}\ ) \quad \text{es equivalente a} \quad \neg\exists x\ \mathit{Gusta}\ (x,\mathit{Chirivías}\ ).$$

Podemos ir más allá: "A todos les gusta el helado" significa que no hay nadie a quien no le guste el helado:

$$\forall x\ \mathit{Gusta}\ (x,\mathit{Helado}) \quad \text{es equivalente a} \quad \neg\exists x\ \neg \mathit{Gusta}(x,\mathit{Helado}\ ).$$

Debido a que ∀ es realmente una conjunción sobre el universo de objetos y ∃ es una disyunción, no debería ser sorprendente que obedezcan las reglas de De Morgan. Las reglas de De Morgan para oraciones cuantificadas y no cuantificadas son las siguientes:

$$\begin{aligned}
\neg\exists x\ P &\equiv \forall x\ \neg P & \qquad \neg(P \lor Q) &\equiv \neg P \land \neg Q\\
\neg\forall x\ P &\equiv \exists x\ \neg P & \qquad \neg(P \land Q) &\equiv \neg P \lor \neg Q\\
\forall x\ P &\equiv \neg\exists x\ \neg P & \qquad P \land Q &\equiv \neg(\neg P \lor \neg Q)\\
\exists x\ P &\equiv \neg\forall x\ \neg P & \qquad P \lor Q &\equiv \neg(\neg P \land \neg Q).
\end{aligned}$$

Así, realmente no necesitamos ∀ y ∃, como no necesitamos ∧ y ∨. Pero la legibilidad es más importante que la parsimonia, por ello mantendremos ambos cuantificadores.

8.2.7 Igualdad

Símbolo de igualdad

La lógica de primer orden incluye una forma más de hacer oraciones atómicas, además de usar un predicado y términos como se describió antes. Podemos usar el **símbolo de igualdad** para significar que dos términos se refieren al mismo objeto. Por ejemplo,

$$Padre(Juan) = Enrique$$

dice que el objeto al que se refiere *Padre* (*Juan*) y el objeto al que se refiere *Enrique* son el mismo. Dado que una interpretación fija el referente de cualquier término, determinar la verdad de una frase de igualdad es sólo una cuestión de ver que los referentes de los dos términos son el mismo objeto.

El símbolo de igualdad se usa para declarar hechos sobre una función dada, como acabamos de hacer para el símbolo *Padre*. También se puede usar con negación para insistir en que dos términos no son el mismo objeto. Para decir que Ricardo tiene al menos dos hermanos, escribiríamos

$$\exists x,y\ Hermano(x,Ricardo) \land Hermano(y,Ricardo) \land \neg(x = y).$$

La frase

$$\exists x,y\ Hermano(x,Ricardo) \land Hermano(y,Ricardo)$$

no tiene el significado pretendido. En particular, es verdadero en el modelo de la Figura 8.2, donde Ricardo sólo tiene un hermano. Para ver esto, considere la interpretación extendida en la que tanto x como y se asignan a Rey Juan. La adición de $\neg(x = y)$ excluye tales modelos. La notación $x \neq y$ a veces se usa como abreviatura de $\neg(x = y)$.

8.2.8 Semántica de base de datos

Continuando con el ejemplo de la sección anterior, supongamos que creemos que Ricardo tiene dos hermanos, Juan y Geoffrey.[7] Podríamos escribir

$$Hermano(Juan,Ricardo) \land Hermano(Geoffrey,Ricardo), \tag{8.3}$$

pero eso no capturaría por completo el estado de cosas. Primero, la afirmación es verdadera en un modelo donde Ricardo sólo tiene un hermano, necesitamos agregar Juan ≠ Geoffrey. Segundo, la frase no descarta modelos en los que Ricardo tiene muchos más hermanos además de Juan y Geoffrey. Así, la traducción correcta de "Los hermanos de Ricardo son Juan y Geoffrey" es:

$$Hermano(Juan,Ricardo) \land Hermano(Geoffrey,Ricardo) \land Juan \neq Geoffrey$$
$$\land\ \forall x\ Hermano(x,Ricardo) \Rightarrow (x = Juan \lor x = Geoffrey).$$

Esta sentencia lógica parece mucho más engorrosa que la sentencia en inglés correspondiente. Pero si no traducimos correctamente el inglés, nuestro sistema de razonamiento lógico cometerá errores. ¿Podemos idear una semántica que permita una sentencia lógica más sencilla?

Suposición de nombres únicos

Suposición de mundo cerrado

Clausura de dominio

Una propuesta muy popular en los sistemas de bases de datos funciona de la siguiente manera. Primero, insistimos en que cada símbolo constante se refiere a un objeto distinto, la **suposición de nombres únicos**. Segundo, asumimos que las oraciones atómicas que no se sabe que son verdaderas son de hecho falsas, la **suposición de mundo cerrado**. Finalmente, invocamos la **clausura de dominio**, donde cada modelo contiene no más elementos de dominio que los nombrados por los símbolos constantes.

[7] En realidad tenía cuatro, los otros eran Guillermo y Jorge.

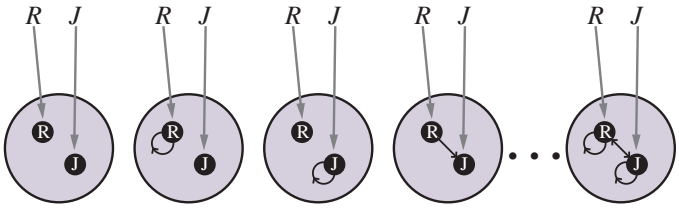


Figura 8.5 Algunos miembros del conjunto de todos los modelos para un lenguaje con dos símbolos constantes, *R* y *J*, y un símbolo de relación binaria, bajo semántica de base de datos. La interpretación de los símbolos constantes está fija, y hay un objeto distinto para cada símbolo constante.

Bajo la semántica resultante, la Ecuación (8.3) sí declara que Ricardo tiene exactamente dos hermanos, Juan y Geoffrey. Llamamos a esto **semántica de base de datos** para distinguirla de la semántica estándar de la lógica de primer orden. La semántica de base de datos también se usa en sistemas de programación lógica, como se explica en la Sección 9.4.4.

Semántica de base de datos

Es instructivo considerar el conjunto de todos los modelos posibles bajo semántica de base de datos para el mismo caso que se muestra en la Figura 8.4 (página 259). La Figura 8.5 muestra algunos de los modelos, que van desde el modelo sin tuplas que satisfacen la relación hasta el modelo con todas las tuplas que satisfacen la relación. Con dos objetos, hay cuatro posibles tuplas de dos elementos, por lo que hay $2^4 = 16$ diferentes subconjuntos de tuplas que pueden satisfacer la relación. Por lo tanto, hay 16 modelos posibles en total; muchos menos que los infinitamente muchos modelos para la semántica de primer orden estándar. Por otro lado, la semántica de base de datos requiere un conocimiento definitivo de lo que contiene el mundo.

Este ejemplo plantea un punto importante: no hay una semántica "correcta" para la lógica. La utilidad de cualquier semántica propuesta depende de cuán concisa e intuitiva hace la expresión de los tipos de conocimiento que queremos escribir, y de cuán fácil y natural es desarrollar las reglas de inferencia correspondientes. La semántica de base de datos es más útil cuando estamos seguros de la identidad de todos los objetos descritos en la BC y cuando tenemos todos los hechos a mano; en otros casos, es bastante incómoda . Por el resto de este capítulo, asumimos la semántica estándar mientras notamos instancias en las que esta elección conduce a expresiones engorrosas.

8.3 Usando la lógica de primer orden

Ahora que hemos definido un lenguaje lógico expresivo, aprendamos a usarlo. Aquí, proporcionamos ejemplos de sentencias en algunos **dominios** simples. En la representación de conocimiento, un dominio es sólo una parte del mundo sobre la que deseamos expresar algún conocimiento.

Dominio

Comenzamos con una breve descripción de la interfaz TELL/ASK para las BC de primer orden. Luego observamos los dominios de relaciones familiares, números, conjuntos y listas, y el mundo del wumpus. La Sección 8.4.2 contiene un ejemplo más sustancial (circuitos electrónicos) y el Capítulo 10 cubre todo en el universo.

8.3.1 Aserciones y consultas en lógica de primer orden

Se agregan sentencias a una BC usando TELL, exactamente como en la lógica proposicional. Tales sentencias se llaman **aserciones**. Por ejemplo, podemos aseverar que Juan es un rey, Ricardo es una persona, y todos los reyes son personas:

Aserción

TELL(*KB*, Rey (*John*)).
TELL(*KB*, Persona (*Richard*)).
TELL(*KB*, $\forall x$ Rey (x) $\Rightarrow$ Persona (x)).

Podemos preguntar a la BC usando ASK. Por ejemplo,

ASK(*KB*, *King*(*John*))

devuelve verdadero . Las preguntas hechas con ASK se llaman **consultas** o **metas**. En general, cualquier consulta que es consecuencia lógica de la BC debe ser respondida afirmativamente. Por ejemplo, dadas las tres afirmaciones anteriores, la consulta

ASK(*KB*, *Person*(*John*))

también debería devolver *verdadero* . Podemos hacer consultas cuantificadas, como

ASK(*KB*, $\exists x$ *Person*(*x*)).

La respuesta es *verdadero* , pero esto quizá no es tan útil como nos gustaría. Es más bien como responder "¿Puede decirme la hora?" con "Sí". Si queremos saber qué valor de *x* hace que la sentencia sea verdadera, necesitaremos una función diferente, a la que llamamos ASKVARS,

ASKVARS(*KB*,*Person*(*x*))

y que produce una secuencia de respuestas. En este caso encontraremos dos respuestas: {*x*/*John*} y {*x*/*Richard*}. Tal respuesta se llama **sustitución** o **lista de vinculación**. ASKVARS es generalmente reservado para BC que constan únicamente de cláusulas de Horn, porque en tales BC, cada forma de hacer que la consulta sea verdadera vinculará las variables a valores específicos. Eso no es el caso con la lógica de primer orden; en una base de conocimiento (BC) que sólo ha sido informada de que *King*(*John*) $\vee$ *King*(*Richard*) no hay una única vinculación a *x* que haga que la consulta $\exists x$ *King*(*x*) sea verdadera, aunque la consulta es de hecho verdadera.

Sustitución
Lista de vinculación

8.3.2 El dominio de parentesco

El primer ejemplo que consideramos es el dominio de las relaciones familiares, o parentesco. Este dominio incluye hechos como "Elizabeth es la madre de Charles" y "Charles es el padre de William" y reglas como "La abuela de uno es la madre del padre de uno".

Claramente, los objetos en nuestro dominio son personas. Los predicados unarios incluyen *Masculino* y *Femenino* , entre otros. Las relaciones de parentesco, hermandad, matrimonio, etc., están representadas por predicados binarios: *Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt,* y *Uncle* . Usamos funciones para *Madre* (*Mother*) y *Padre* (*Father*) , porque cada persona tiene exactamente una de cada una de estas, biológicamente (aunque podríamos introducir funciones adicionales para madres adoptivas, madres sustitutas, etc.). [Ante la ausencia de género neutro en el español –más precisamente: de palabras que establezcan relaciones de parentesco sin enunciar el género de ninguna de las partes involucradas–, este ejemplo y en subsiguientes se ha preferido dejar el texto en inglés de los predicados binarios].

Podemos pasar por cada función y predicado, escribiendo lo que sabemos en términos de los otros símbolos. Por ejemplo, la madre de uno es el progenitor de uno que es femenino :

$\forall m,c$ *Madre* $(c) = m \Leftrightarrow$ *Femenino* $(m) \wedge$ *Progenitor* (m,c).

El esposo de uno es el cónyuge masculino de uno:

$\forall w,h$ *Esposo* $(h,w) \Leftrightarrow$ *Masculino* (h) $\wedge$ *Cónyuge* (h,w).

Padre e *Hijo* son relaciones inversas:

$$\forall p,c\ Padre(p,c) \Leftrightarrow Hijo(c,p).$$

Un *Grandparent* es un *Parent* del *Parent* de uno:

$$\forall g,c\ Grandparent(g,c) \Leftrightarrow \exists p\ Parent(g,p) \wedge Parent(p,c).$$

Un *sibling* es otro *child* del *parent* de uno:

$$\forall x,y\ Sibling(x,y) \Leftrightarrow x \neq y \wedge \exists p\ Parent(p,x) \wedge Parent(p,y).$$

Podríamos seguir durante varias páginas más como esta, y el Ejercicio 8.KINS le pide que haga exactamente eso.

Cada una de estas sentencias puede ser vista como un **axioma** del dominio de parentesco, como se explica en la Sección 7.1. Los axiomas están comúnmente asociados con dominios puramente matemáticos; veremos algunos axiomas para números en breve, pero son necesarios en todos los dominios. Proporcionan la información factual básica a partir de la cual se pueden derivar conclusiones útiles. Nuestros axiomas de parentesco también son **definiciones**; tienen la forma $\forall x,y\ P(x,y) \Leftrightarrow ....$ Los axiomas definen la función *Madre* y los predicados *Esposo*, *Masculino*, *Progenitor*, *Grandparent*, y *Sibling* en términos de otros predicados. Nuestras definiciones "tocan fondo" en un conjunto básico de predicados (*Child*, *Femenino*, etc.) en términos de los cuales se definen finalmente los demás. Definición

Esta es una forma natural de construir la representación de un dominio, análoga a la forma en que los paquetes de software se construyen mediante definiciones sucesivas de subrutinas a partir de bibliotecas de funciones primitivas. Note que no hay necesariamente un conjunto único de predicados primitivos; podríamos haber usado *Parent* en lugar de *Child*. En algunos dominios, como mostramos, no hay un conjunto básico claramente identificable.

No todas las sentencias lógicas sobre un dominio son axiomas; algunas son **teoremas**, es decir, consecuencias lógicas de axiomas. Por ejemplo, considere la afirmación de que la hermandad es simétrica: Teorema

$$\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$$

¿Es esto un axioma o un teorema? De hecho, es un teorema que sigue lógicamente del axioma que define la hermandad. Si preguntamos a la BC esta sentencia, debería devolver verdadero.

Desde un punto de vista puramente lógico, una BC sólo necesita contener axiomas y no teoremas, porque los teoremas no aumentan el conjunto de conclusiones que se derivan de la BC. Desde un punto de vista práctico, los teoremas son esenciales para reducir el costo computacional de derivar nuevas sentencias. Sin ellos, un sistema de razonamiento tiene que partir de los primeros principios cada vez, algo así como un físico que tiene que volver a derivar las reglas del cálculo para cada nuevo problema.

No todos los axiomas son definiciones. Algunos proporcionan información más general sobre ciertos predicados sin constituir una definición. De hecho, algunos predicados no tienen una definición completa porque no sabemos lo suficiente para caracterizarlos completamente. Por ejemplo, no hay una forma obvia y definitiva de completar la sentencia

$$\forall x\ Persona(x) \Leftrightarrow ...$$

Afortunadamente, la lógica de primer orden nos permite hacer uso del predicado *Persona* sin definirlo completamente. En su lugar, podemos escribir especificaciones parciales de propiedades que cada persona tiene y propiedades que hacen que algo sea una persona:

$$\forall x\ Persona(x) \Rightarrow ...$$
$$\forall x\ ... \Rightarrow Persona(x).$$

Los axiomas también pueden ser "simplemente hechos", como *Masculino* (*Jim*) y *Cónyuge* (*Jim*,*Laura*). Estos hechos forman las descripciones de instancias de problemas específicos,

permitiendo que se puedan responder preguntas específicas. Si todo va bien, las respuestas a estas preguntas serán entonces teoremas que se derivan de los axiomas.

Por lo regular, uno descubre que las respuestas esperadas no se producen; por ejemplo, uno espera de *Cónyuge* (*Jim*,*Laura*) (bajo las leyes de muchos países) poder inferir que ¬*Cónyuge* (*George*,*Laura*); pero esto no se deduce de los axiomas antes dados, aun después de que agregamos Jim ≠ George como se sugiere en la Sección 8.2.8. Esto es una señal de que falta un axioma. El ejercicio 8.HILL pide al lector que lo proporcione.

8.3.3 Números, conjuntos y listas

Números naturales

Los números son quizás el ejemplo más vívido de cómo una gran teoría puede construirse a partir de un pequeño núcleo de axiomas. Aquí describimos la teoría de los **números naturales** o enteros no negativos. Necesitamos un predicado *NatNum* que será verdadero de los números naturales; necesitamos un símbolo constante, 0; y necesitamos un símbolo de función, S (sucesor). Los **axiomas de Peano** definen los números naturales y la suma.[8] Los números naturales se definen recursivamente:

Axiomas de Peano

$$NatNum(0).$$
$$\forall n\ NatNum(n) \Rightarrow NatNum(S(n)).$$

Es decir, 0 es un número natural, y para cada objeto n, si n es un número natural, entonces $S(n)$ es un número natural. Así que los números naturales son 0, $S(0)$, $S(S(0))$, etc. También necesitamos axiomas para restringir la función sucesora:

$$\forall n\ 0 \neq (S(n)).$$
$$\forall m,n\ m \neq n \Rightarrow S(m) \neq S(n).$$

Ahora podemos definir la suma en términos de la función sucesora:

$$\forall m\ NatNum(m) \Rightarrow +(0,m) = m.$$
$$\forall m,n\ NatNum(m) \wedge NatNum(n) \Rightarrow +(S(m),n) = S(+(m,n)).$$

El primero de estos axiomas dice que agregar 0 a cualquier número natural m da m mismo. Vea el uso del símbolo de función binaria "+" en el término $+(m,0)$; en matemáticas ordinarias, el término se escribiría $m + 0$ usando notación de **infijo**. (La notación que hemos usado para la lógica de primer orden se llama **prefijo**). Para que nuestras sentencias sobre números sean más fáciles de leer, permitimos el uso de la notación de infijo. También podemos escribir $S(n)$ como $n + 1$, por lo que el segundo axioma se convierte en

Infijo

Prefijo

$$\forall m,n\ NatNum(m) \wedge NatNum(n) \Rightarrow (m + 1) + n = (m + n) + 1.$$

Este axioma reduce la suma a la aplicación repetida de la función sucesora.

Azúcar sintáctica

El uso de la notación de infijo es un ejemplo de **azúcar sintáctica**, es decir, una extensión o abreviatura de la sintaxis estándar que no cambia la semántica. Cualquier sentencia que use azúcar puede ser "desazucarada" para producir una sentencia equivalente en lógica de primer orden ordinaria. Otro ejemplo es el uso de corchetes en lugar de paréntesis para facilitar la visualización de qué corchete izquierdo coincide con qué corchete derecho. Otro ejemplo es colapsar los cuantificadores: reemplazando $\forall x\ \forall y\ P(x,y)$ con $\forall x,y\ P(x,y)$.

Una vez que tenemos la suma, es sencillo definir la multiplicación como suma repetida, la exponentiación como multiplicación repetida, la división entera y los residuos, los números primos, y así sucesivamente. De este modo, toda la teoría de números (incluyendo la criptografía) puede construirse a partir de una constante, una función, un predicado y cuatro axiomas.

---

[8] Los axiomas de Peano también incluyen el principio de inducción, que es una oración de lógica de segundo orden en lugar de lógica de primer orden. La importancia de esta distinción se explica en el Capítulo 9.

conjuntos

El dominio de los **conjuntos** es también fundamental para las matemáticas, así como para el razonamiento de sentido común. (De hecho, es posible definir la teoría de números en términos de teoría de conjuntos). Queremos ser capaces de representar conjuntos individuales, incluido el conjunto vacío. Necesitamos una forma de construir conjuntos a partir de elementos o de operaciones en otros conjuntos. Queremos saber si un elemento es miembro de un conjunto y queremos distinguir conjuntos de objetos que no son conjuntos.

Usaremos el vocabulario normal de la teoría de conjuntos como azúcar sintáctica. El conjunto vacío es una constante escrita como {}. Hay un predicado unario, *Set*, que es verdadero de los conjuntos. Los predicados binarios son $x \in s$ (x es un miembro del conjunto s) y $s_1 \subseteq s_2$ (el conjunto s_1 es un subconjunto de s_2, posiblemente igual a s_2). Las funciones binarias son $s_1 \cap s_2$ (intersección), $s_1 \cup s_2$ (unión), y $Add(x,s)$ (el conjunto resultante de añadir el elemento x al conjunto s). Un posible conjunto de axiomas es el siguiente:

1. Los únicos conjuntos son el conjunto vacío y aquellos hechos añadiendo algo a un conjunto:

   $$\forall s\ Set(s) \Leftrightarrow (s = \{\}) \vee (\exists x,s_2\ Set(s_2) \wedge s = Add(x,s_2)).$$

2. El conjunto vacío no tiene elementos añadidos en él. Es decir, no hay forma de descomponer {} en un conjunto más pequeño y un elemento:

   $$\neg\exists x,s\ Add(x,s) = \{\}.$$

3. Agregar un elemento ya en el conjunto no tiene efecto:

   $$\forall x,s\ x \in s \Leftrightarrow s = Add(x,s).$$

4. Los únicos miembros de un conjunto son los elementos que se agregaron en él. Expresamos esto de manera recursiva, diciendo que x es un miembro de s si y sólo si s es igual a algún elemento y agregado a algún conjunto s_2, donde o bien y es lo mismo que x, o x es un miembro de s_2:

   $$\forall x,s\ x \in s \Leftrightarrow \exists x,s_2\ (s = Add(y,s_2) \wedge (x = y \vee x \in s_2)).$$

5. Un conjunto es un subconjunto de otro conjunto si y sólo si todos los miembros del primer conjunto son miembros del segundo conjunto:

   $$\forall s_1,s_2\ s_1 \subseteq s_2 \Leftrightarrow (\forall x\ x \in s_1 \Rightarrow x \in s_2).$$

6. Dos conjuntos son iguales si y sólo si cada uno es un subconjunto del otro:

   $$\forall s_1,s_2\ (s_1 = s_2) \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1).$$

7. Un objeto está en la intersección de dos conjuntos si y sólo si es miembro de ambos conjuntos:

   $$\forall x s_1,s_2\ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2).$$

8. Un objeto está en la unión de dos conjuntos si y sólo si es miembro de cualquiera de los conjuntos:

   $$\forall x,s_1,s_2\ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2).$$

listas

Las **listas** son similares a los conjuntos. Las diferencias son que las listas están ordenadas y el mismo elemento puede aparecer más de una vez en una lista. Podemos usar el vocabulario de Lisp para las listas: *Nil* es la lista constante sin elementos; *Cons*, *Append*, *First* y *Rest* son funciones; pero *Find* es el predicado que hace para las listas lo que *Member* hace para los conjuntos. *List* es un predicado que es verdadero sólo de las listas. Al igual que con los conjuntos, es común usar azúcar sintáctico en oraciones lógicas que involucran listas. La lista es []. El término $Cons(x,Nil)$ (es decir, la lista que contiene el elemento x seguido de nada) se escribe como $[x]$. Una lista de varios elementos, como $[A,B,C]$, corresponde al término anidado $Cons(A,Cons(B,Cons(C,Nil)))$. Ejercicio 8.LIST te pide que escribas los axiomas para las listas.

8.3.4 El mundo del wumpus

Algunos axiomas de lógica proposicional para el mundo del wumpus se dieron en el Capítulo 7. Los axiomas de primer orden en esta sección son mucho más concisos, pues capturan de una manera natural exactamente lo que queremos decir.

Recuerde que el agente wumpus recibe un vector de percepción con cinco elementos. La oración de primer orden correspondiente almacenada en la BC debe incluir tanto la percepción como el momento en que ocurrió; de lo contrario, el agente se confundirá sobre cuándo vio qué. Usamos enteros para los pasos de tiempo. Una oración perceptiva típica sería

Percibe ([*Hedor*, *Brisa*, *Brillo*, *Nada*, *Nada*],5).

Aquí, *Percibe* es un predicado binario, y *Hedor* y sucesivamente son constantes colocadas en una lista. Las acciones en el mundo del wumpus pueden representarse mediante términos lógicos:

Girar (*Derecha*), *Girar* (*Izquierda*), *Adelante*, *Disparar*, *Agarrar*, *Subir*.

Para determinar cuál es el mejor, el programa del agente ejecuta la consulta

AskVars(*KB*, *MejorAcción* (*a*,5)),

que devuelve una lista de enlaces como {*a*/*Agarrar* }. El programa del agente puede entonces devolver Agarrar como la acción a tomar. Los datos brutos de percepción implican ciertos hechos sobre el estado actual. Por ejemplo:

$$\forall t,s,g,w,c\ \textit{Percepto}([s,\textit{Brisa},g,w,c],t) \Rightarrow \textit{Brisa}\ (t)$$
$$\forall t,s,g,w,c\ \textit{Percepto}([s,\textit{Nada},g,w,c],t) \Rightarrow \neg\textit{Brisa}\ (t)$$
$$\forall t,s,b,w,c\ \textit{Percepto}([s,b,\textit{Brillo},w,c],t) \Rightarrow \textit{Brillo}(t)$$
$$\forall t,s,b,w,c\ \textit{Percepto}([s,b,\textit{Nada},w,c],t) \Rightarrow \neg\textit{Brillo}\ (t)$$

y así sucesivamente. Estas reglas exhiben una forma trivial del proceso de razonamiento llamado **percepción**, que estudiamos en profundidad en el Capítulo 25. Observe la cuantificación sobre el tiempo *t*. En la lógica proposicional, necesitaríamos copias de cada oración para cada paso de tiempo.

El comportamiento "reflejo" simple también puede implementarse mediante oraciones de implicación cuantificadas. Por ejemplo, tenemos

$$\forall t,\ \textit{Brillo}(t) \Rightarrow \textit{MejorAcción}(\textit{Agarrar},t).$$

Dada la percepción y las reglas de los párrafos anteriores, esto daría la conclusión deseada *MejorAcción*(*Agarrar*,5); es decir, *Agarrar* es lo correcto a hacer.

Hemos representado las entradas y salidas del agente; ahora es el momento de representar el entorno en sí. Comencemos con los objetos. Los candidatos obvios son casillas, pozos y el wumpus. Podríamos nombrar cada casilla, $\textit{Casillas}_{1,2}$ y así sucesivamente, pero entonces el hecho de que $\textit{Casillas}_{1,2}$ y $\textit{Casillas}_{1,3}$ son adyacentes tendría que ser un hecho "extra", y necesitaríamos un hecho así para cada par de casillas. Es mejor usar un término complejo en el que la fila y la columna aparezcan como enteros; por ejemplo, podemos usar sólo el término de lista [1,2]. La adyacencia de dos casillas cualquiera se puede definir como

$$\forall x,y,a,b\ \textit{Adjacente}([x,y],[a,b]) \Leftrightarrow$$
$$(x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1)).$$

Podríamos nombrar cada pozo, pero sería inapropiado por una razón diferente: no hay razón para

distinguir entre pozos.[9] Es más simple usar un predicado unario *Pozo* que es verdadero de las casillas que contienen pozos. Finalmente, ya que hay exactamente un wumpus, una constante Wumpus es tan buena como un predicado unario (y quizá más digna desde el punto de vista del wumpus).

La ubicación del agente cambia con el tiempo, por lo que escribimos *En* (*Agente*,*s*,*t*) para decir que el agente está en la casilla *s* en el tiempo *t*. Podemos fijar el wumpus a una ubicación específica para siempre con $\forall t\, At(Wumpus,[1,3],t)$. Luego podemos decir que los objetos sólo pueden estar en una ubicación a la vez:

$$\forall x,s_1,s_2,t\ At(x,s_1,t) \wedge At(x,s_2,t) \Rightarrow s_1 = s_2.$$

Dada su ubicación actual, el agente puede inferir propiedades de la casilla a partir de propiedades de su percepción actual. Por ejemplo, si el agente está en una casilla y percibe una brisa, entonces esa casilla es ventosa:

$$\forall s,t\ At(Agente,s,t) \wedge Brisa(t) \Rightarrow Ventoso\ (s).$$

Es útil saber que una *casilla* es ventosa porque sabemos que los pozos no pueden moverse. Observe que *Ventosa* no tiene argumento de tiempo.

Tras descubrir qué lugares son ventosos (o malolientes) y, muy importante, *no* ventosos (o *no* malolientes), el agente puede deducir dónde están los pozos (y dónde está el wumpus). Mientras que la lógica proposicional requiere un axioma separado para cada cuadrado (ver R_2 y R_3 en la página 220) y necesitaría un conjunto diferente de axiomas para cada disposición geográfica del mundo, la lógica de primer orden sólo necesita un axioma:

$$\forall s\ Brisa(s) \Rightarrow \exists r\ Adyacente(r,s) \wedge Pozo(r). \tag{8.4}$$

De manera similar, en la lógica de primer orden podemos cuantificar sobre el tiempo, por lo que sólo necesitamos un axioma de estado sucesor para cada predicado, en lugar de una copia diferente para cada paso de tiempo. Por ejemplo, el axioma para la flecha (Ecuación (7.2) en la página 240) se convierte en

$$\forall t\ TieneFlecha\ (t+1) \Leftrightarrow (TieneFlecha\ (t) \wedge \neg Acción\ (Disparar\ ,t)).$$

A partir de estas dos oraciones de ejemplo, podemos ver que la formulación de la lógica de primer orden no es menos concisa que la descripción original en inglés dada en el Capítulo 7. Se invita al lector a construir axiomas análogos para la ubicación y orientación del agente; en estos casos, los axiomas cuantifican tanto el espacio como el tiempo. Como en el caso de la estimación del estado proposicional, un agente puede usar la inferencia lógica con axiomas de este tipo para seguir el rastro de aspectos del mundo que no se observan directamente. El Capítulo 11 profundiza en el tema de los axiomas de estado sucesor de primer orden y sus usos para la construcción de planes.

8.4 Ingeniería del conocimiento en lógica de primer orden

La sección anterior ilustró el uso de la lógica de primer orden para representar el conocimiento en tres dominios simples. Esta sección describe el proceso general de construcción de la BC, un proceso llamado **ingeniería del conocimiento**. Un ingeniero del conocimiento es alguien que investiga un dominio particular, aprende qué conceptos son importantes en ese dominio, y crea una representación formal de los objetos y relaciones en el dominio. Ilustramos el proceso de

Ingeniería del conocimiento

[9] De manera similar, la mayoría de nosotros no nombramos a cada pájaro que vuela por encima mientras migra a regiones más cálidas en invierno. Un ornitólogo que desea estudiar patrones de migración, tasas de supervivencia, etc., sí nombra a cada pájaro, mediante un anillo en su pata, porque se deben rastrear cada pájaro individualmente.

ingeniería del conocimiento en un dominio de circuito electrónico. El enfoque que tomamos es adecuado para desarrollar BC de *propósito especial* cuyo dominio está cuidadosamente circunscrito y cuyo rango de consultas se conoce de antemano. Las BC de *propósito general*, que cubren una amplia gama de conocimientos humanos y están destinadas a apoyar tareas como la comprensión del lenguaje natural, se discuten en el Capítulo 10.

8.4.1 El proceso de ingeniería del conocimiento

Los proyectos de ingeniería del conocimiento varían ampliamente en contenido, alcance y dificultad, pero todos estos proyectos incluyen los siguientes pasos:

1. *Identificar las preguntas*. El ingeniero del conocimiento debe delinear el rango de preguntas que la BC soportará y los tipos de hechos que estarán disponibles para cada instancia de problema específica. Por ejemplo, ¿la BC del wumpus necesita poder elegir acciones o sólo se requiere para responder preguntas sobre el contenido del entorno? ¿Incluirán los hechos del sensor la ubicación actual? La tarea determinará qué conocimiento debe representarse para conectar las instancias de problemas con las respuestas. Este paso es análogo al proceso PEAS para diseñar agentes en el Capítulo 2.
2. *Reunir el conocimiento relevante*. El ingeniero del conocimiento podría ser un experto en el dominio o podría necesitar trabajar con verdaderos expertos para extraer lo que saben, un proceso llamado **adquisición de conocimiento**. En esta etapa, el conocimiento no se representa formalmente. La idea es entender el alcance de la BC, según lo determinado por la tarea, y entender cómo funciona realmente el dominio.

   Adquisición de conocimiento

   Para el mundo del wumpus, definido por un conjunto artificial de reglas, el conocimiento relevante es fácil de identificar. (Sin embargo, vea que la definición de adyacencia no se suministró explícitamente en las reglas del mundo del wumpus). Para los dominios reales, el problema de la relevancia puede ser muy difícil; por ejemplo, un sistema para simular diseños VLSI podría o no necesitar tener en cuenta las capacitancias parásitas y los efectos peliculares .
3. *Decidir sobre un vocabulario de predicados, funciones y constantes*. Es decir, traducir los conceptos importantes a nivel de dominio a nombres a nivel de lógica. Esto implica muchas preguntas de *estilo* de ingeniería del conocimiento. Al igual que el estilo de programación, esto puede tener un impacto significativo en el éxito eventual del proyecto. Por ejemplo, ¿deberían los pozos ser representados por objetos o por un predicado unario en las casillas? ¿Debería ser la orientación del agente una función o un predicado? ¿Dependerá la ubicación del wumpus del tiempo? Una vez que se han tomado las decisiones, el resultado es un vocabulario que se conoce como la **ontología** del dominio. La palabra *ontología* significa una teoría particular de la naturaleza del ser o la existencia. La ontología determina qué tipos de cosas existen, pero no determina sus propiedades específicas e interrelaciones.

   Ontología
4. *Codificar el conocimiento general sobre el dominio*. El ingeniero del conocimiento escribe los axiomas para todos los términos del vocabulario. Esto fija (en la medida de lo posible) el significado de los términos, permitiendo al experto verificar el contenido. A menudo, este paso revela conceptos erróneos o lagunas en el vocabulario que deben ser corregidos volviendo al paso 3 e iterando a través del proceso.
5. *Codificar una descripción de la instancia de problema*. Si la ontología está bien pensada, este paso es fácil. Implica escribir sentencias atómicas simples sobre instancias de conceptos que ya son parte de la ontología. Para un agente lógico, las instancias de problemas son suministradas por los sensores, mientras que una BC "incorpórea " recibe sentencias de la misma manera que los programas tradicionales reciben datos de entrada.

6. *Hacer consultas al procedimiento de inferencia y obtener respuestas.* Aquí es donde está la recompensa: podemos dejar que el procedimiento de inferencia opere en los axiomas y en hechos específicos del problema para derivar los hechos que nos interesa conocer. Así, evitamos la necesidad de escribir un algoritmo de solución específico para cada aplicación.
7. *Depurar y evaluar la BC.* Desafortunadamente, las respuestas a las consultas rara vez serán correctas en el primer intento. Más precisamente, las respuestas serán correctas *para la BC tal como está escrita*, suponiendo que el procedimiento de inferencia es sólido, pero no serán las que el usuario espera. Por ejemplo, si falta un axioma, algunas consultas no podrán ser respondidas desde la BC. Podría seguir un considerable proceso de depuración. Los axiomas faltantes o los axiomas que son demasiado débiles pueden identificarse fácilmente al notar lugares donde la cadena de razonamiento se detiene inesperadamente. Por ejemplo, si la BC incluye una regla de diagnóstico (ver Ejercicio 8.WUMD) para encontrar el wumpus,

   $$\forall s\ Apestoso(s) \Rightarrow Adjacente(Guarida(Wumpus),s),$$

   en lugar del bicondicional, entonces el agente nunca podrá probar la ausencia de wumpuses. Los axiomas incorrectos se pueden identificar porque son declaraciones falsas sobre el mundo. Por ejemplo, la frase

   $$\forall x\ NúmeroDePatas\ (x,4) \Rightarrow Mamífero\ (x)$$

   es falsa para reptiles, anfibios y mesas. *La falsedad de esta oración se determina independientemente del resto de la BC.* En contraste, un error típico en un programa se ve así:

   ```
   offset = position + 1.
   ```

   Es imposible decir si `desplazamiento` debe ser `posición` o `posición + 1` sin entender el contexto circundante.

Cuando llega al punto en que no hay errores obvios en su BC, es tentador declarar el éxito. Pero a menos que obviamente no haya errores, es mejor evaluar formalmente su sistema ejecutándolo en un conjunto de consultas de prueba y midiendo cuántas aciertan. Sin una medición objetiva, es demasiado fácil convencerte de que el trabajo está hecho. Para entender mejor este proceso de siete pasos, ahora lo aplicamos a un ejemplo extendido: el dominio de los circuitos electrónicos.

8.4.2 El dominio de los circuitos electrónicos

Desarrollaremos una ontología y una BC que nos permitan razonar sobre circuitos digitales del tipo mostrado en la Figura 8.6. Seguimos el proceso de siete pasos para la ingeniería del conocimiento.

Identificar las preguntas

Hay muchas tareas de razonamiento asociadas con los circuitos digitales. A nivel más alto, se analiza la funcionalidad del circuito. Por ejemplo, ¿el circuito de la Figura 8.6 realmente suma correctamente? Si todas las entradas están altas, ¿cuál es la salida de la puerta A2? Las preguntas sobre la estructura del circuito también son interesantes. Por ejemplo, ¿cuáles son todas las puertas conectadas al primer terminal de entrada? ¿El circuito contiene bucles de retroalimentación? Estas serán nuestras tareas en esta sección. Hay niveles más detallados de análisis, incluidos los relacionados con los retrasos de tiempo, el área del circuito, el consumo de energía, el costo de producción, etc. Cada uno de estos niveles requeriría conocimientos adicionales.

Reunir el conocimiento relevante

¿Qué sabemos sobre los circuitos digitales? Para nuestros propósitos, están compuestos de cables y puertas. Las señales fluyen a lo largo de los cables hasta los terminales de entrada de las puertas, y cada puerta produce una señal en el terminal de salida que fluye a lo largo de otro cable. Para determinar cuáles serán estas señales, necesitamos saber cómo transforman las puertas sus señales de entrada. Hay cuatro tipos de puertas: las puertas AND, OR y XOR tienen dos terminales de entrada, y las puertas NOT tienen una. Todas las puertas tienen una terminal de salida. Los circuitos, como las puertas, tienen terminales de entrada y salida.

Para razonar sobre la funcionalidad y la conectividad, no necesitamos hablar sobre los cables en sí, los caminos que toman, o las uniones donde se juntan. Todo lo que importa son las conexiones entre los terminales; podemos decir que un terminal de salida está conectado a otro terminal de entrada sin tener que decir qué los conecta realmente. Otros factores como el tamaño, la forma, el color o el costo de los diversos componentes son irrelevantes para nuestro análisis.

Si nuestro propósito fuera algo distinto de verificar diseños a nivel de puerta, la ontología sería diferente. Por ejemplo, si estuviéramos interesados en depurar circuitos defectuosos, entonces quizá sería una buena idea incluir los cables en la ontología, porque un cable defectuoso puede corromper la señal que fluye a lo largo de él. Para resolver fallas de tiempo, tendríamos que incluir retrasos de puerta. Si estuviéramos interesados en diseñar un producto que fuera rentable, entonces el costo del circuito y su velocidad en relación con otros productos en el mercado serían importantes.

Decidir sobre un vocabulario

Ahora sabemos que queremos hablar sobre circuitos, terminales, señales y puertas. El siguiente paso es elegir funciones, predicados y constantes para representarlos. Primero, necesitamos poder distinguir las puertas entre sí y de otros objetos. Cada puerta se representa como un objeto nombrado por una constante, sobre la cual afirmamos que es una puerta con, digamos, *Puerta* (X_1). El comportamiento de cada puerta está determinado por su tipo: una de las constantes *AND*, *OR*, *XOR*, o *NOT*. Como una puerta tiene exactamente un tipo, una función es apropiada: *Tipo* (X_1) = *XOR*. Los circuitos, como las puertas, se identifican por un predicado: *Circuito* (C_1).

A continuación consideramos los terminales, que se identifican por el predicado *Terminal*(x). Un circuito puede tener uno o más terminales de entrada y uno o más terminales de salida.

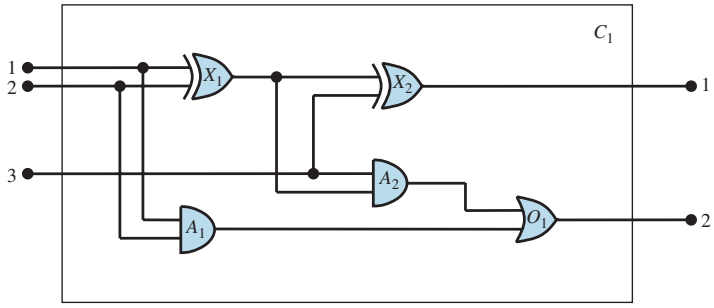


Figura 8.6 Un circuito digital C_1, que pretende ser un sumador completo de un bit. Las dos primeras entradas son los dos bits a sumar, y la tercera entrada es un bit de acarreo. La primera salida es la suma, y la segunda salida es un bit de acarreo para el siguiente sumador. El circuito contiene dos puertas XOR, dos puertas AND y una puerta OR.

Usamos la función *Entrada* $(1,X_1)$ para denotar el primer terminal de entrada para el circuito X_1. Una función similar *Salida* (n,c) se usa para los terminales de salida. La función *Aridad* (c,i,j) dice que el circuito c tiene i terminales de entrada y j terminales de salida. La conectividad entre las puertas se puede representar por un predicado, *Conectado*, que toma dos terminales como argumentos, como en *Conectado* (*Salida* $(1,X_1)$,*Entrada* $(1,X_2)$).

Finalmente, necesitamos saber si una señal está encendida o apagada. Una posibilidad es usar un predicado unario, *Encendido* (t), que es verdadero cuando la señal en un terminal está encendida. Sin embargo, esto dificulta un poco plantear preguntas como "¿Cuáles son todos los posibles valores de las señales en los terminales de salida del circuito C_1?" Por lo tanto, introducimos como objetos dos valores de señal, 1 y 0, que representan "encendido" y "apagado", respectivamente, y una función *Señal* (t) que denota el valor de la señal para el terminal t.

Codificar el conocimiento general del dominio

Una señal de que tenemos una buena ontología es que sólo requerimos unas pocas reglas generales, que se pueden declarar de manera clara y concisa. Estos son todos los axiomas que necesitaremos:

1. Si dos terminales están conectados, entonces tienen la misma señal:

   $$\forall t_1,t_2\ Terminal(t_1) \land Terminal(t_2) \land Connected(t_1,t_2) \Rightarrow Signal(t_1) = Signal(t_2)\ .$$

2. La señal en cada terminal es 1 o 0:

   $$\forall t\ Terminal(t) \Rightarrow Signal(t) = 1 \lor Signal(t) = 0\ .$$

3. *Connected* es conmutativo:

   $$\forall t_1,t_2\ Connected(t_1,t_2) \Leftrightarrow Connected(t_2,t_1).$$

4. Hay cuatro tipos de puertas:

   $$\forall g\ Gate(g) \land k = Type(g) \Rightarrow k = AND \lor k = OR \lor k = XOR \lor k = NOT.$$

5. La salida de una puerta AND es 0 si y sólo si alguna de sus entradas es 0:

   $$\forall g\ Gate(g) \land Type(g) = AND \Rightarrow Signal(Out(1,g)) = 0 \Leftrightarrow \exists n\ Signal(In(n,g)) = 0.$$

6. La salida de una puerta OR es 1 si y sólo si alguna de sus entradas es 1:

   $$\forall g\ Gate(g) \land Type(g) = OR \Rightarrow Signal(Out(1,g)) = 1 \Leftrightarrow \exists n\ Signal(In(n,g)) = 1.$$

7. La salida de una puerta XOR es 1 si y sólo si sus entradas son diferentes:

   $$\forall g\ Gate(g) \land Type(g) = XOR \Rightarrow$$
   $$Signal(Out(1,g)) = 1 \Leftrightarrow Signal(In(1,g)) \neq Signal(In(2,g)).$$

8. La salida de una puerta NOT es diferente de su entrada:

   $$\forall g\ Gate(g) \land Type(g) = NOT \Rightarrow Signal(Out(1,g)) \neq Signal(In(1,g)).$$

9. Las puertas (excepto para NOT) tienen dos entradas y una salida.

   $$\forall g\ Gate(g) \land Type(g) = NOT \Rightarrow Arity(g,1,1).$$
   $$\forall g\ Gate(g) \land k = Type(g) \land (k = AND \lor k = OR \lor k = XOR) \Rightarrow Arity(g,2,1)$$

10. Un circuito tiene terminales, hasta su aridad de entrada y salida, y nada más allá de su aridad:

    $$\forall c,i,j\ Circuit(c) \land Arity(c,i,j) \Rightarrow$$
    $$\forall n\ (n \leq i \Rightarrow Terminal(In(n,c))) \land (n > i \Rightarrow In(n,c) = Nothing) \land$$
    $$\forall n\ (n \leq j \Rightarrow Terminal(Out(n,c))) \land (n > j \Rightarrow Out(n,c) = Nothing)$$

11. Las puertas, terminales y señales son todas distintas.

$$\forall g,t,s\ Gate(g) \wedge Terminal(t) \wedge Signal(s) \Rightarrow g \neq t \wedge g \neq s \wedge t \neq s.$$

12. Las puertas son circuitos.

$$\forall g\ Gate(g) \Rightarrow Circuit(g)$$

Codifica la instancia específica del problema

El circuito mostrado en la Figura 8.6 se codifica como circuito C_1 con la siguiente descripción. Primero categorizamos el circuito y sus puertas componentes:

$$Circuito(C_1) \wedge Aridad(C_1,3,2)$$
$$Puerta(X_1) \wedge Tipo(X_1) = XOR$$
$$Puerta(X_2) \wedge Tipo(X_2) = XOR$$
$$Puerta(A_1) \wedge Tipo(A_1) = AND$$
$$Puerta(A_2) \wedge Tipo(A_2) = AND$$
$$Puerta(O_1) \wedge Tipo(O_1) = OR.$$

Luego mostramos las conexiones entre ellos:

$$Conectado(Salida(1,X_1),Entrada(1,X_2)) \qquad Conectado(In(1,C_1), Entrada(1,X_1))$$
$$Conectado(Salida(1,X_1), Entrada(2,A_2)) \qquad Conectado(Entrada(1,C_1), Entrada(1,A_1))$$
$$Conectado(Salida(1,A_2), Entrada(1,O_1)) \qquad Conectado(Entrada(2,C_1), Entrada(2,X_1))$$
$$Conectado(Salida(1,A_1), Entrada(2,O_1)) \qquad Conectado(Entrada(2,C_1), Entrada(2,A_1))$$
$$Conectado(Salida(1,X_2), Salida(1,C_1)) \qquad Conectado(Entrada(3,C_1), Entrada(2,X_2))$$
$$Conectado(Salida(1,O_1),Salida(2,C_1)) \qquad Conectado(Entrada(3,C_1), Entrada(1,A_2)).$$

Plantear consultas al procedimiento de inferencia

¿Qué combinaciones de entradas harían que la primera salida de C_1 (el bit de suma) sea 0 y la segunda salida de C_1 (el bit de acarreo) sea 1?

$$\exists i_1,i_2,i_3\ Signal(In(1,C_1)) = i_1 \wedge Signal(In(2,C_1)) = i_2 \wedge Signal(In(3,C_1)) = i_3 \wedge Signal(Out(1,C_1))=0 \wedge Signal(Out(2,C_1))=1.$$

Las respuestas son sustituciones para las variables i_1, i_2, y i_3 de tal manera que la oración resultante se deduce de la BC. AskVars nos dará tres de estas sustituciones:

$$\{i_1/1, i_2/1, i_3/0\} \quad \{i_1/1, i_2/0, i_3/1\} \quad \{i_1/0, i_2/1, i_3/1\}.$$

¿Cuáles son los posibles conjuntos de valores de todos los terminales para el circuito sumador?

$$\exists i_1,i_2,i_3,o_1,o_2\ Signal(In(1,C_1)) = i_1 \wedge Signal(In(2,C_1)) = i_2 \wedge Signal(In(3,C_1)) = i_3 \wedge Signal(Out(1,C_1)) = o_1 \wedge Signal(Out(2,C_1)) = o_2.$$

Esta consulta final devolverá una tabla de entrada-salida completa para el dispositivo, que se puede usar para verificar que suma sus entradas correctamente. Este es un ejemplo simple de **verificación de circuitos**. También podemos usar la definición del circuito para construir sistemas digitales más grandes, para los cuales se puede llevar a cabo el mismo tipo de procedimiento de verificación. (Ver Ejercicio 8.ADDR). Muchos dominios son susceptibles al mismo tipo de desarrollo de BC estructurados, en el que se definen conceptos más complejos sobre conceptos más simples.

Verificación de circuito

Depurar la base de conocimientos (BC)

Podemos perturbar la BC de varias formas para ver qué tipos de comportamientos erróneos emergen. Por ejemplo, supongamos que no leemos la Sección 8.2.8 y, por lo tanto, olvidamos afirmar que $1 \neq 0$. De repente, el sistema no podrá probar ninguna salida para el circuito, excepto para los casos de entrada 000 y 110. Podemos localizar el problema preguntando por las salidas de cada puerta. Por ejemplo, podemos preguntar

$$\exists i_1, i_2, o \; Signal(In(1,C_1)) = i_1 \wedge Signal(In(2,C_1)) = i_2 \wedge Signal(Out(1,X_1)) = o,$$

lo que revela que no se conocen salidas en X_1 para los casos de entrada 10 y 01. Luego, miramos el axioma para las puertas XOR, aplicado a X_1:

$$Signal(Out(1,X_1)) = 1 \Leftrightarrow Signal(In(1,X_1)) \neq Signal(In(2,X_1)).$$

Si las entradas se conocen como, digamos, 1 y 0, entonces esto se reduce a

$$Signal(Out(1,X_1)) = 1 \Leftrightarrow 1 \neq 0.$$

Ahora el problema es evidente: el sistema no puede inferir que $Signal(Out(1,X_1)) = 1$, por lo que necesitamos decirle que 1 /= 0.

Resumen

Este capítulo ha introducido la **lógica de primer orden**, un lenguaje de representación que es mucho más poderoso que la lógica proposicional. Los puntos importantes son los siguientes:

- Los lenguajes de representación del conocimiento deben ser declarativos, composicionales, expresivos, independientes del contexto y no ambiguos.
- Las lógicas difieren en sus **compromisos ontológicos** y **epistemológicos**. Mientras que la lógica proposicional sólo se compromete con la existencia de hechos, la lógica de primer orden se compromete con la existencia de objetos y relaciones y, por lo tanto, gana poder expresivo, apropiado para dominios como el mundo del wumpus y los circuitos electrónicos.
- Tanto la lógica proposicional como la lógica de primer orden comparten una dificultad para representar proposiciones vagas. Esta dificultad limita su aplicabilidad en dominios que requieren juicios personales, como la política o la cocina.
- La sintaxis de la lógica de primer orden se basa en la de la lógica proposicional. Añade términos para representar objetos, y tiene cuantificadores universales y existenciales para construir afirmaciones sobre todos o algunos de los posibles valores de las variables cuantificadas.
- Un **mundo posible**, o **modelo**, para la lógica de primer orden incluye un conjunto de objetos y una **interpretación** que mapea símbolos constantes a objetos, símbolos de predicado a relaciones entre objetos, y símbolos de función a funciones sobre objetos.
- Una sentencia atómica es verdadera sólo cuando la relación nombrada por el predicado se mantiene entre los objetos nombrados por los términos. Las **interpretaciones extendidas**, que mapean variables cuantificadoras a objetos en el modelo, definen la verdad de las sentencias cuantificadas.
- Desarrollar una BC en lógica de primer orden requiere un cuidadoso proceso de análisis del dominio, elección de un vocabulario, y codificación de los axiomas requeridos para soportar las inferencias deseadas.

Notas bibliográficas e históricas

Aunque la lógica de Aristóteles trataba de generalizaciones sobre objetos, se quedó muy corta en cuanto al poder expresivo de la lógica de primer orden. Una barrera importante para su posterior desarrollo fue su concentración en predicados de un sólo lugar en detrimento de muchos predicados relacionales de varios lugares. El primer tratamiento sistemático de las relaciones fue dado por Augustus De Morgan (1864), quien, para mostrar los tipos de inferencias que la lógica de Aristóteles no podía manejar, citó el siguiente ejemplo: "Todos los caballos son animales; por lo tanto, la cabeza de un caballo es la cabeza de un animal". Esta inferencia es inaccesible para Aristóteles porque cualquier regla válida que pueda soportar esta inferencia debe primero analizar la oración usando el predicado de dos lugares "x es la cabeza de y". La lógica de las relaciones fue estudiada en profundidad por Charles Sanders Peirce (Peirce, 1870; Misak, 2004).

La verdadera lógica de primer orden data de la introducción de cuantificadores en la *Begriffschrift* ("Escritura de Conceptos" o "Notación conceptual") de Gottlob Frege (1879). Peirce (1883) también desarrolló la lógica de primer orden independiente a Frege, aunque un poco más tarde. La capacidad de Frege para anidar cuantificadores fue un gran avance, pero utilizó una notación incómoda. La notación actual para la lógica de primer orden se debe sustancialmente a Giuseppe Peano (1889), pero la semántica es prácticamente idéntica a la de Frege. Curiosamente, los axiomas de Peano se debieron en gran medida a Grassmann (1861) y Dedekind (1888).

Leopold Löwenheim (1915) dio un tratamiento sistemático de la teoría de modelos para la lógica de primer orden, incluyendo el primer tratamiento adecuado del símbolo de igualdad. Los resultados de Löwenheim fueron ampliados después por Thoralf Skolem (1920). Alfred Tarski (1935, 1956) dio una definición explícita de verdad y satisfacción modeloteórica en lógica de primer orden, utilizando la teoría de conjuntos.

John McCarthy (1958) fue el principal responsable de la introducción de la lógica de primer orden como herramienta para construir sistemas de IA. Las perspectivas para la IA basada en lógica avanzaron de manera significa con el desarrollo de la resolución de Robinson (1965), un procedimiento completo para la inferencia de primer orden. El enfoque logicista echó raíces en la Universidad de Stanford. Cordell Green (1969a, 1969b) desarrolló un sistema de razonamiento de primer orden, QA3, que condujo a los primeros intentos de construir un robot lógico en SRI (Fikes y Nilsson, 1971). La lógica de primer orden fue aplicada por Zohar Manna y Richard Waldinger (1971) para razonar sobre programas y más tarde por Michael Genesereth (1984) para razonar sobre circuitos. En Europa, la programación lógica (una forma restringida de razonamiento de primer orden) fue desarrollada para el análisis lingüístico (Colmerauer *et al.*, 1973) y para sistemas declarativos generales (Kowalski, 1974). La lógica computacional también estaba bien arraigada en Edimburgo a través del proyecto LCF (Logic for Computable Functions) (Gordon *et al.*, 1979). Estos desarrollos se narran más a fondo en los Capítulos 9 y 10.

Las aplicaciones prácticas construidas con lógica de primer orden incluyen un sistema para evaluar los requisitos de fabricación para productos electrónicos (Mannion, 2002), un sistema para razonar sobre políticas de acceso a archivos y gestión de derechos digitales (Halpern y Weissman, 2008), y un sistema para la composición automatizada de servicios web (McIlraith y Zeng, 2001).

Las reacciones a la hipótesis de Whorf (Whorf, 1956) y el problema del lenguaje y el pensamiento en general, aparecen en varios libros (Pullum, 1991; Pinker, 2003) incluyendo los títulos aparentemente opuestos *¿Por qué el mundo se ve diferente en otros idiomas?* (Deutscher, 2010) y *¿Por qué el mundo se ve igual en cualquier idioma?* (McWhorter, 2014) (aunque ambos autores están de acuerdo en que hay diferencias y las diferencias son pequeñas). La teoría de "teorías" (Gopnik y Glymour, 2002; Tenenbaum *et al.*, 2007) ve el aprendizaje de los niños sobre el mundo como análogo a la construcción de teorías científicas. Así como las predicciones de un algoritmo

de aprendizaje automático dependen fuertemente del vocabulario que se le suministra, así también la formulación de teorías del niño dependerá del entorno lingüístico en el que se produce el aprendizaje.

Hay una serie de buenos textos introductorios sobre lógica de primer orden, incluyendo algunos de las principales figuras en la historia de la lógica: Alfred Tarski (1941), Alonzo Church (1956), y W.V. Quine (1982) (que es uno de los más legibles). Enderton (1972) da una perspectiva más matemáticamente orientada. Un tratamiento altamente formal de la lógica de primer orden, junto con muchos temas más avanzados en lógica, es proporcionado por Bell y Machover (1977). Manna y Waldinger (1985) dan una introducción legible a la lógica desde una perspectiva de ciencias de la computación, al igual que Huth y Ryan (2004), quienes se concentran en la verificación de programas. Barwise y Etchemendy (2002) adoptan un enfoque similar al que se utiliza aquí. Smullyan (1995) presenta resultados de manera concisa, utilizando el formato de tabla. Gallier (1986) proporciona una exposición matemática extremadamente rigurosa de la lógica de primer orden, junto con una gran cantidad de material sobre su uso en razonamiento automatizado. Los *Fundamentos Lógicos de la Inteligencia Artificial* (Genesereth y Nilsson, 1987) es tanto una sólida introducción a la lógica como el primer tratamiento sistemático de agentes lógicos con percepciones y acciones, y hay dos buenos manuales: van Bentham y ter Meulen (1997) y Robinson y Voronkov (2001). La revista de referencia para el campo de la lógica matemática pura es el *Journal of Symbolic Logic*, mientras que el *Journal of Applied Logic* se ocupa de preocupaciones más cercanas a las de la inteligencia artificial.

CAPÍTULO 9

INFERENCIA EN LA LÓGICA DE PRIMER ORDEN

En el que se definen los procedimientos efectivos para responder preguntas planteadas en lógica de primer orden.

En este capítulo, describimos algoritmos que responden cualquier pregunta de lógica de primer orden válida. La Sección 9.1 introduce reglas de inferencia para cuantificadores y muestra cómo reducir la inferencia de primer orden a una inferencia proposicional, aunque potencialmente a gran costo. La Sección 9.2 describe cómo la **unificación** es utilizada para construir reglas de inferencia que trabajen directamente con enunciados de primer orden. Luego se discuten tres familias de algoritmos de inferencia de primer orden: **encadenamiento hacia adelante** (***fc***, por sus siglas en inglés) (Sección 9.3), **encadenamiento hacia atrás** (***bc,*** por sus siglas en inglés) (Sección 9.4) y **demostración de teoremas basada en el principio de resolución** (Sección 9.5).

9.1 Inferencia proposicional vs. inferencia de primer orden

Una forma de hacer inferencia de primer orden es convertir la base de conocimiento (BC) de primer orden a lógica proposicional y usar inferencia proposicional, lo cual sabemos cómo hacer. Un primer paso es eliminar los cuantificadores universales. Ejemplo, supongamos que nuestra BC contiene el axioma folclórico estándar: *todos los reyes codiciosos son malvados*:

$$\forall x \; King(x) \wedge Greedy(x) \Rightarrow Evil(x).$$

A partir de eso, podemos inferir cualquiera de las siguientes oraciones:

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$
$$King(Richard) \wedge Greedy\,(Richard) \Rightarrow Evil(Richard)$$
$$King(Father(John)) \wedge Greedy(Father(John)) \Rightarrow Evil(Father(\text{John})).$$
$$\vdots$$

Instanciación Universal

En general, la regla de **Instanciación Universal** (**IU,** abreviado) indica que podemos inferir cualquier oración obtenida al sustituir un término **concreto** (un término sin variables) por una variable cuantificada universalmente.[1]

Para escribir la regla de inferencia formalmente, usamos la noción de **sustituciones** introducida en la Sección 8.3. Sea $\text{SUBST}(\theta,\alpha)$ el resultado de aplicar la sustitución θ a la oración α. Entonces, la regla se escribe como:

$$\frac{\forall v \; \alpha}{\text{SUBST}\,(\{v / g\}, \alpha\,)}$$

[1] No confundir estas sustituciones con las interpretaciones extendidas usadas para definir la semántica de los cuantificadores en la Sección 8.2.6. La sustitución reemplaza una variable con un término (un fragmento de sintaxis) para producir una nueva oración, mientras que una interpretación asigna una variable a un objeto en el dominio.

para cualquier variable v y término concreto *g*. Por ejemplo, las tres oraciones dadas antes se obtienen con las sustituciones {*x*/*John*},{*x*/*Richard*}, y {*x*/*Padre*(*John*)}.

Instanciación Existencial

Del mismo modo, la regla de **Instanciación Existencial** reemplaza una variable cuantificada existencialmente con un nuevo *símbolo constante único*. La declaración formal es la siguiente: para cualquier oración α, variable v y símbolo constante *k* que no aparezca en otro lugar en la BC,

$$\frac{\exists\, v\ \alpha}{\text{SUBST}\,(\{v\,/\,k\},\,\alpha)}$$

Por ejemplo, a partir de la oración

$\exists x\ Crown(x) \wedge OnHead(x,John)$

podemos inferir la oración

$Crown(C_1) \wedge OnHead(C_1,John)$

siempre y cuando C_1 no aparezca en otro lugar en la BC. Básicamente, la oración existencial indica que hay algún objeto que satisface una condición, y aplicar la regla de instanciación existencial simplemente da un nombre a ese objeto. Claro, ese nombre no debe pertenecer ya a otro objeto. Las matemáticas dan un buen ejemplo: supongamos que descubrimos que hay un número que es un poco más grande que 2.71828 y que satisface la ecuación $d(x^y)/dy=x^y$ para *x*. Podemos dar a este número el nombre e, pero sería un error darle el nombre de un objeto existente, como π. En lógica, el nuevo nombre se llama **constante de Skolem**.

Constante de Skolem

Mientras que la Instanciación Universal puede aplicarse muchas veces a un mismo axioma para producir consecuencias diferentes, la Instanciación Existencial sólo necesita aplicarse una vez, luego la oración cuantificada existencialmente puede ser descartada. Ejemplo, ya no necesitamos $\exists x\ Kill(x,Victim)$ una vez añadida la oración $Kill(Murderer,Victim)$.

9.1.1 Reducción a inferencia proposicional

Veamos cómo convertir cualquier BC de primer orden a una BC proposicional. Primero, así como una oración cuantificada existencialmente puede ser reemplazada por una instancia, una oración cuantificada universalmente puede ser reemplazada por el conjunto de *todas las posibles* instancias. Así, suponga que nuestra BC contiene sólo las oraciones

$$\begin{aligned}&\forall x\ King(x) \wedge Greedy(x) \Rightarrow Evil(x)\\ &King(John)\\ &Greedy(John)\\ &Brother(Richard,John).\end{aligned} \tag{9.1}$$

y que los únicos objetos son *John* and *Richard.* Aplicamos **IU** a la primera oración usando todas las posibles sustituciones, {*x*/*John*} y {*x*/*Richard*}. Obtenemos

$$\begin{aligned}&King(John) \wedge Greedy(John) \Rightarrow Evil(John)\\ &King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard).\end{aligned}$$

Luego reemplazamos las oraciones atómicas concretas, como *King*(*John*), con símbolos de proposición, como *JohnIsKing.* Y aplicamos cualquiera de los algoritmos proposicionales completos del Capítulo 7, para obtener conclusiones como *JohnIsEvil*, que es equivalente a *Evil*(*John*).

Proposicionalización

Esta técnica de **proposicionalización** puede ser completamente general, como mostramos en la Sección 9.5. Sin embargo, hay un problema cuando la BC incluye un símbolo de función, ¡el conjunto de posibles sustituciones de términos concretos es infinito! Por ejemplo, si la BC menciona el símbolo *Padre*, entonces se pueden construir infinitos términos anidados como *Father*(*Father*(*Father*(*John*)))

Por suerte, hay un famoso teorema de Jacques Herbrand (1930) que establece que, si una oración está implicada por la BC original de primer orden, entonces hay una prueba que involucra sólo un subconjunto *finito* de la BC proposicionalizada. Como cualquier subconjunto de ese tipo tiene una profundidad máxima de anidamiento entre sus términos concretos, encontramos el subconjunto primero generando todas las instanciaciones con símbolos constantes (*Richard* y *John*), luego todos los términos de profundidad 1 (*Father*(*Richard*) and *Father*(*John*)), luego todos los términos de profundidad 2, y así hasta que construyamos una prueba proposicional de la oración implicada.

Hemos esbozado un enfoque para la inferencia de primer orden mediante la proposicionalización **completa**; esto significa que toda oración implicada puede ser probada. Este es un logro importante, pues el espacio de modelos posibles es infinito. Además, ¡no sabemos que la oración *está* implicada hasta que se hace la prueba! ¿Qué sucede cuando la oración *no está* implicada? ¿Podemos saberlo? Para la lógica de primer orden no podemos saberlo. Nuestro procedimiento de prueba puede continuar generando términos anidados cada vez más profundos, pero no sabremos si está atrapado en un bucle infinito o si la prueba está a punto de aparecer. Esto se parece al problema de la detención para las máquinas de Turing. Alan Turing (1936) y Alonzo Church (1936) demostraron, de maneras diferentes, la inevitabilidad de este estado de cosas. *El punto de la implicación para la lógica de primer orden es* ***semidecidible****; es decir, existen algoritmos que dicen sí a cada oración implicada, pero no existe algoritmo que diga no a cada oración no implicada.*

9.2 Unificación e inferencia de primer orden

Un lector perspicaz habrá notado que el enfoque de proposicionalización genera instanciaciones innecesarias de oraciones cuantificadas universalmente. Preferimos un enfoque que use sólo una regla, razonando que $\{x/John\}$ resuelve la consulta $Evil(x)$ así: dada la regla de que los reyes codiciosos son malvados, encontrar algún x tal que x sea *king* y x sea *greedy*, y luego inferir que este x es *evil*. Por lo regular, si hay alguna sustitución θ que hace que cada una de las conjunciones de la premisa de la implicación sea idéntica a oraciones ya en la BC, entonces podemos afirmar la conclusión de la implicación, después de aplicar θ. En este caso, la sustitución $\theta = \{x/John\}$ logra ese objetivo. Ahora suponga que en lugar de saber $Greedy(John)$, sabemos que *todos* son codiciosos:

$$\forall y \; Greedy(y). \tag{9.2}$$

Entonces aún nos gustaría concluir que $Evil(John)$, porque sabemos que John es *king* (dado) y John es *greedy* (porque todos son *greedy*). Lo que necesitamos para que esto funcione es encontrar una sustitución para las variables en la oración de implicación y para las variables en las oraciones que están en la BC. En este caso, aplicar la sustitución $\{x/John, y/John\}$ a las premisas de implicación $King(x)$ y $Greedy(x)$ y a las oraciones de la BC $King(John)$ y $Greedy(y)$ hará que sean idénticas. Así, podemos inferir el consecuente de la implicación.

Modus Ponens Generalizado

Este proceso de inferencia puede ser capturado como única regla de inferencia que llamamos **Modus Ponens Generalizado**:[2] Para oraciones atómicas p_i, p_i' y q, donde hay una sustitución θ tal que

[2] El Modus Ponens Generalizado es más general que el Modus Ponens (p. 222) en el sentido de que los hechos conocidos y la premisa de la implicación sólo deben coincidir hasta una sustitución, en vez de exactamente. Al contrario, el Modus Ponens permite cualquier oración α como premisa, no sólo una conjunción de oraciones atómicas.

$$\frac{\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i), \text{ para todo } i, \quad p_1', p_2', \ldots, p_n', (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}.$$

Hay $n + 1$ premisas para esta regla: las n oraciones atómicas p_i' y la implicación única. La conclusión es el resultado de aplicar la sustitución θ al consecuente q. Para nuestro ejemplo:

p_1' is *King*(*John*) $\quad$ p_1 is *King*(x)
p_2' is *Greedy*(y) $\quad$ p_2 is *Greedy*(x)
θ is {x/John,y/*John*} $\quad$ q is *Evil*(x)
SUBST(θ,q) is *Evil*(*John*).

Es fácil mostrar que el Modus Ponens Generalizado es una regla de inferencia válida. Primero, observe que para cualquier oración p (cuyas variables se suponen cuantificadas universalmente) y para cualquier sustitución θ,

$$p \models \text{SUBST}(\theta, p)$$

es verdadero por Instanciación Universal. Es verdadero en particular para un θ que satisface las condiciones de la regla de Modus Ponens Generalizado. Así, a partir de $p_i', \ldots, p_n'$ podemos inferir

$$\text{SUBST}(\theta, p_1') \wedge \ldots \wedge \text{SUBST}(\theta, p_n')$$

y a partir de la implicación $p_1 \wedge \ldots \wedge p_n \Rightarrow q$ podemos inferir

$$\text{SUBST}(\theta, p_1) \wedge \ldots \wedge \text{SUBST}(\theta, p_n) \Rightarrow \text{SUBST}(\theta, q).$$

Ahora, θ en el Modus Ponens Generalizado está definido de modo que $\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i)$, para todo i; por lo tanto, la primera de ambas oraciones coincide exactamente con la premisa de la segunda. Entonces, SUBST(θ,q), prosigue por Modus Ponens.

Ampliada

El Modus Ponens Generalizado es una versión **ampliada** del Modus Ponens: lo eleva de la lógica proposicional terrenal (libre de variables) a la lógica de primer orden. En este capítulo veremos que podemos desarrollar versiones elevadas de algoritmos de encadenamiento hacia adelante, *bc* y resolución, introducidos en el Capítulo 7. La ventaja clave de las reglas de inferencia elevadas sobre la proposicionalización es que sólo hacen sustituciones necesarias para permitir que ciertas inferencias procedan.

9.2.1 Unificación

Unificación
Unificador

Las reglas de inferencia elevadas requieren sustituciones que hagan que diferentes expresiones lógicas luzcan idénticas. Este proceso se llama **unificación** y es un componente clave de todos los algoritmos de inferencia de primer orden. El algoritmo UNIFY dos oraciones y devuelve un **unificador** para ellas (una sustitución) si existe:

UNIFY(p,q)= θ donde SUBST(θ, p)=SUBST(θ,q).

Veamos ejemplos de cómo debería comportarse UNIFY. Suponga la consulta *AskVars*(*Knows*(*John*,x)): ¿quién conoce a John? Las respuestas se localizan al encontrar todas las oraciones en la BC que se unifican con *Knows*(*John*,x). Aquí los resultados de la unificación con cuatro oraciones diferentes que podrían estar en la BC:

UNIFY(*Knows*(*John*,x), *Knows*(*John*,*Jane*)) = {x/*Jane*}
UNIFY(*Knows*(*John*,x), *Knows*(y,*Bill*)) = {x/*Bill*,y/*John*}
UNIFY(*Knows*(*John*,x), *Knows*(y,*Mother*(y))) = {y/*John*,x/*Mother*(*John*)}
UNIFY(*Knows*(*John*,x), *Knows*(x,*Elizabeth*)) = *failure*.

La última unificación falla porque x no puede tomar los valores John y Elizabeth a la vez. Recuerde que *Knows*(*x*,*Elizabeth*) significa "Todos conocen a Elizabeth", así que *deberíamos* poder inferir que John conoce a Elizabeth. El problema surge sólo porque las dos oraciones usan el mismo nombre de variable, x, y se puede evitar **estandarizando aparte** una de las dos oraciones que se están unificando, lo que significa renombrar sus variables para evitar conflictos de nombres. Así, podemos renombrar *x Knows*(*x*,*Elizabeth*) *a* x_{17} (un nuevo nombre de variable) sin cambiar su significado. Ahora la unificación funcionará:

Estandarizando aparte

$$\text{UNIFY}(Knows(John,x), Knows(x_{17},Elizabeth)) = \{x/Elizabeth,x_{17}/John\}.$$

El ejercicio 9.STAN profundiza en la necesidad de estandarizado aparte.

Hay una complicación más: dijimos que UNIFY debería devolver una sustitución que haga que ambos argumentos se vean iguales. Pero podría haber más de un unificador. Así, UNIFY(*Knows*(*John*,*x*),*Knows*(*y*,*z*)) podría devolver {*y*/*John*,*x*/*z*} o podría devolver {*y*/*John*,*x*/*John*,*z*/*John*}. El primer unificador da *Knows*(*John*,*z*) como resultado de la unificación, mientras que el segundo da Knows(John,John). El segundo resultado se obtendría a partir del primero por una sustitución adicional {*z*/*John*}; decimos que el primer unificador es *más general* que el segundo, porque impone menos restricciones en los valores de las variables.

Unificador más general (MGU)

Cada par de expresiones unificables tiene un **unificador más general** (**MGU**, por sus siglas en inglés) que es único hasta el renombramiento y sustitución de variables. Así, {*x*/*John*} y {*y*/*John*} se consideran equivalentes, igual que {*x*/*John*,*y*/*John*} y {*x*/*John*,*y*/*x*}.

La Figura 9.1 muestra un algoritmo para calcular unificadores más generales. El proceso es simple: explorar recursivamente ambas expresiones de modo simultáneo "lado a lado", y construir un unificador en el camino, pero fallando si dos puntos correspondientes en las estructuras no coinciden. Hay un paso costoso: cuando se empareja una variable con un término complejo, se debe verificar si esta misma ocurre dentro del término; si lo hace, el emparejamiento falla porque no se puede construir un unificador consistente. Esto es, $S(x)$, no puede unificarse con S($S(x)$). Esta llamada **verificación de ocurrencia** hace que la complejidad de todo el algoritmo sea cuadrática en el tamaño de las expresiones que se unifican. Unos sistemas, incluidos sistemas de programación lógica, sólo omiten la verificación de ocurrencia y hacen que el usuario evite hacer inferencias incorrectas como resultado. Otros sistemas usan algoritmos de unificación más complejos con complejidad temporal lineal.

Verificación de ocurrencia

9.2.2 Almacenamiento y recuperación

Subyacentes a las funciones TELL, ASK y ASKVARS usadas para informar e interrogar una BC, están las funciones más primitivas STORE y FETCH. STORE(s) almacena una oración s en la BC y FETCH(q) devuelve todos los unificadores de modo que la consulta q se unifique con una oración en la BC. El problema con el que ilustramos la unificación: encontrar todos los hechos que se unifican con *Knows*(*John*,*x*), es una instancia de FETCH.

La forma más sencilla de implementar STORE y FETCH es mantener todos los hechos en una lista larga y unificar cada consulta con cada elemento de la lista. Tal proceso es ineficiente, pero funciona. En lo que sigue esbozaremos formas de hacer que la recuperación sea más eficiente.

Podemos hacer que FETCH sea más eficiente asegurándonos de que las unificaciones sean sólo con oraciones con alguna posibilidad de unificación. Así, no tiene sentido unificar *Knows*(*John*,*x*) con *Brother*(*Richard*,*John*). Podemos evitar tales unificaciones **indexando** los hechos en la BC. Un esquema simple llamado **indexación de predicados** coloca todos los hechos de *Knows* se almacenan en un contenedor y todos los hechos de Brother en otro. Estos contenedores pueden almacenarse en una tabla hash para un acceso eficiente.

Indexando

Indexación de predicados

```
function UNIFY(x, y, θ=vacío) returns una sustitución que hace y idénticas
  if θ = falla then return falla
  else if x = y then return θ
  else if VARIABLE?(x) then return UNIFY-VAR(x, y, θ)
  else if VARIABLE?(y) then return UNIFY-VAR(y, x, θ)
  else if COMPOUND?(x) and COMPOUND?(y) then
    return UNIFY(ARGS(x),ARGS(y),UNIFY(OP(x),OP(y), θ))
  else if LIST?(x) and LIST?(y) then
    return UNIFY(REST(x), REST(y),UNIFY(FIRST(x), FIRST(y), θ))
  else return falla

function UNIFY-VAR(var, x, θ) returns sustitución
  if {var/val} ∈ θ para todo val then return UNIFY(val, x, θ)
  else if {x/val} ∈ θ para todo val then return UNIFY(var, val, θ)
  else if OCCUR-CHECK?(var, x) then return falla
  else return agrega {var/x} to θ
```

Figura 9.1 Algoritmo de unificación. Los argumentos x y *y* pueden ser cualquier expresión: constante o variable, o una expresión compuesta (oración o término complejo), o una lista de expresiones. El argumento θ es una sustitución, en inicio la sustitución vacía, pero con pares {*var*/*val*} agregados conforme se recorre a través de las entradas, comparando las expresiones elemento por elemento. En una expresión compuesta como F(*A*,*B*), el campo OP(*x*) selecciona el símbolo de función *F* y el campo ARGS(*x*) seleccionan la lista de argumentos (*A*,*B*).

La indexación de predicados es útil cuando hay muchos símbolos de predicado, pero sólo pocos enunciados para cada símbolo. Pero, a veces, un predicado tiene muchos enunciados. Por ejemplo, suponga que el fisco quieren llevar un registro de quién emplea a quién, usando un predicado *Employs*(*x*,*y*). Este sería un bloque muy grande con quizá millones de empleadores y decenas de millones de empleados. Responder una consulta como *Employs*(*x*,*Ricardo*) con indexación de predicados requeriría escanear todo el contenedor.

Para la consulta en particular, sería útil si los hechos se indexaran por predicado y por segundo argumento, quizás usando una clave de tabla hash combinada. Luego podríamos sólo construir la clave a partir de la consulta y recuperar los hechos exactos que se unifican con la consulta. Para otras consultas, como *Employs*(*IBM*,*y*), necesitaríamos haber indexado los hechos combinando el predicado con el primer argumento. Así, los hechos pueden almacenarse bajo múltiples claves de índice, lo que los hace instantáneamente accesibles a varias consultas con las que podrían unificarse.

Dada una oración para almacenar, es posible construir índices para todas las consultas posibles que se unifiquen con ella. Para el hecho *Employs*(*IBM*,*Richard*), las consultas son:

Employs(*IBM*,*Richard*):	*Does IBM employ Richard?*
Employs(*x*,*Richard*):	*Who employs Richard?*
Employs(*IBM*,*y*):	*Whom does IBM employ?*
Employs(*x*,*y*):	*Who employs whom?*

Estas consultas forman una **retícula de subsunción**, como en la Figura 9.2(a). La retícula tiene algunas propiedades interesantes. El hijo de cualquier nodo en la retícula se obtiene de su padre mediante una única sustitución. El "descendiente común" más alto de cualquier par de

Retícula de subsunción

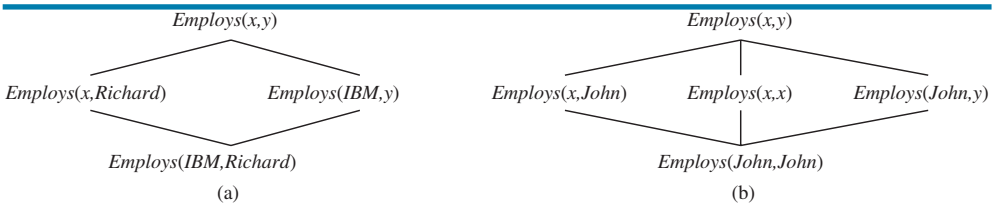


Figura 9.2 **(a)** La retícula de subsunción cuyo nodo más bajo es *Employs*(*IBM,Richard*). **(b)** La retícula de subsunción para la oración *Employs*(*John, John*).

nodos resulta de aplicar su unificador más general. Una oración con constantes repetidas tiene una retícula ligeramente diferente, como muestra la Figura 9.2(b). Aunque los símbolos de función no se muestran en la figura, también pueden incorporarse en la estructura de la retícula.

Para predicados con un pequeño número de argumentos, es un buen compromiso crear un índice para cada punto en la retícula de subsunción. Eso requiere más trabajo al momento de almacenamiento, y acelera el tiempo de recuperación. Pero, para un predicado con n argumentos, la retícula contiene $0(2^n)$ nodos. Si se permiten símbolos de función, el número de nodos también es exponencial en el tamaño de los términos en la oración a almacenar. Esto puede llevar a un enorme número de índices.

Debemos limitar de algún modo los índices que es probable que se utilicen con frecuencia en las consultas; de lo contrario, perderemos más tiempo creando los índices de lo que ahorraremos al tenerlos. Podríamos adoptar una política fija, como tener índices sólo en claves compuestas por un predicado más un sólo argumento. O podríamos aprender una política adaptativa que cree índices para satisfacer las demandas de los tipos de consultas que se hacen. Para bases de datos comerciales donde los hechos se cuentan por miles de millones, el problema ha sido objeto de estudio intensivo, desarrollo tecnológico y optimización continua.

9.3 Inferencia hacia adelante (*fc*)

En la Sección 7.5 vimos un algoritmo de inferencia hacia adelante para bases de conocimiento de cláusulas proposicionales definidas. Aquí ampliamos esa idea para cubrir cláusulas definidas de primer orden.

Claro, hay unas oraciones lógicas que no se pueden definir como una cláusula definida, y no pueden ser manejadas por este enfoque. Pero las reglas de la forma *Antecedente* $\Rightarrow$ *Consecuente* son suficientes para cubrir una amplia variedad de sistemas interesantes del mundo real.

9.3.1 Cláusulas definidas de primer orden

Las cláusulas definidas de primer orden son disyunciones de literales de los que *exactamente uno es positivo*. Eso significa que una cláusula definida es o atómica o una implicación cuyo antecedente es una conjunción de literales positivos y su consecuente es un solo literal positivo. No se permiten cuantificadores existenciales, y los cuantificadores universales son implícitos: si ve una x en una cláusula definida, significa que hay un cuantificador implícito $\forall x$. La cláusula definida de primer orden típica:

$$King(x) \wedge Greedy(x) \Rightarrow Evil(x),$$

pero los literales *King*(*John*) y *Greedy*(y) también cuentan como cláusulas definidas. Los literales de primer orden pueden incluir variables, por lo que *Greedy*(y) se interpreta como "todos son codiciosos" (el cuantificador universal es implícito).

Veamos cómo trabajar con cláusulas definidas para representar el siguiente problema:

> La ley establece que es un delito para un estadounidense vender armas a naciones hostiles. El país Nono, enemigo de Estados Unidos, tiene algunos misiles, y todos sus misiles fueron vendidos por el Coronel West, que es estadounidense.

Primero, representaremos estos hechos como cláusulas definidas de primer orden:

"… es un delito para un estadounidense vender armas a naciones hostiles":

$$American(x) \land Weapon(y) \land Sells(x,y,z) \land Hostile(z) \Rightarrow Criminal(x). \quad (9.3)$$

"Nono ... has some missiles". La oración $\exists x\ Owns(Nono,x) \land Missile(x)$ se transforma en dos cláusulas definidas mediante la Instanciación Existencial, introduciendo una nueva constante M_1:

$$Owns(Nono,M_1) \quad (9.4)$$
$$Missile(M_1) \quad (9.5)$$

"All of its missiles were sold to it by Colonel West":

$$Missile(x) \land Owns(Nono,x) \Rightarrow Sells(West,x,Nono). \quad (9.6)$$

También necesitaremos saber que los misiles son armas:

$$Missile(x) \Rightarrow Weapon(x) \quad (9.7)$$

y debemos saber que un enemigo de Estado Unidos cuenta como "hostile":

$$Enemy(x,America) \Rightarrow Hostile(x). \quad (9.8)$$

"West, who is American …":

$$American(West) \quad (9.9)$$

"The country Nono, an enemy of America…":

$$Enemy(Nono,America). \quad (9.10)$$

Esta BC resulta ser una BC **Datalog**: Datalog es un lenguaje que consiste en cláusulas definidas de primer orden sin símbolos de función. Datalog recibe su nombre porque puede representar el tipo de declaraciones típicamente realizadas en bases de datos relacionales. La ausencia de símbolos de función facilita la inferencia.

Datalog

9.3.2 Un algoritmo simple de encadenamiento hacia adelante

La Figura 9.3 muestra un algoritmo simple de encadenamiento hacia adelante. Inicia con los hechos conocidos, activa todas las reglas cuyas premisas se satisfacen, y añade sus conclusiones a los hechos conocidos. El proceso se repite hasta que la consulta es respondida (asumiendo que sólo se requiere una respuesta) o no se añaden nuevos hechos. Vea que un hecho no es "nuevo" si es sólo un **renombramiento** de un hecho conocido; una oración es un renombramiento de otra si son idénticas excepto por los nombres de las variables. Así, *Likes*(*x*,*IceCream*) y *Likes*(*y*,*IceCream*) son renombramientos entre sí. Ambos significan lo mismo: "Everyone likes ice cream".

renombramiento

Usaremos nuestro problema del crimen para ilustrar FOL-FC-Ask. Las oraciones de implicación disponibles para el encadenamiento son (9.3), (9.6), (9.7) y (9.8). Se requieren dos iteraciones.

```
function FOL-FC-ASK(KB,α) returns una sustitución or false
  inputs: la BC, un conjunto de cláusulas definidas de primer orden
          α, la consulta, una oración atómica

  while true do
    new←{}  // El conjunto de nuevas oraciones inferidas en cada iteración
    for each rule in KB do
      (p1 ∧. . .∧ pn ⇒ q)←STANDARDIZE-VARIABLES(rule)
      for each θ tal que SUBST(θ, p1 ∧ . . . ∧ pn) = SUBST(θ, p′1 ∧ . . . ∧ p′n)
                  para todo p′1, . . . ,p′n in KB
        q′←SUBST(θ, q)
        if q′ no se unifica con alguna oración en KB o new then
        agrega q′ a new
        φ←UNIFY(q′,α)
        if φ no es failure then return φ
    if new = {} then return false
    agrega new to KB
```

Figura 9.3 Un algoritmo de encadenamiento haca adelante conceptualmente directo, pero ineficiente. En cada iteración, agrega a *KB* (BC) todas las oraciones atómicas que se pueden inferir en un solo paso de las oraciones de implicación y las oraciones atómicas que ya están en BC. La función STANDARDIZE-VARIABLES reemplaza todas las variables en sus argumentos con otras nuevas que no se han usado antes.

- En la primera iteración, la regla (9.3) tiene premisas insatisfechas.
  La regla (9.6) se satisface con $\{x/M_1\}$, y se agrega $Sells(West,M_1,Nono)$.
  La regla (9.7) se satisface con $\{x/M_1\}$, y se agrega $Weapon(M_1)$.
  La regla (9.8) se satisface con $\{x/Nono\}$, y se agrega $Hostile(Nono)$.
- En la segunda iteración, la regla (9.3) se satisface con $\{x/West,y/M_1,z/Nono\}$, y se añade la inferencia $Criminal(West)$.

La Figura 9.4 muestra el árbol de prueba que se genera. Vea que en este punto no son posibles nuevas inferencias porque cada oración que podría ser concluida por *fc* ya está contenida explícitamente en la BC. Una BC así se llama **punto fijo** del proceso de inferencia. Los puntos fijos alcanzados por *fc* con cláusulas definidas de primer orden son similares a los del *fc* proposicional; la diferencia es que un punto fijo de primer orden puede incluir oraciones atómicas cuantificadas universalmente.

FOL-FC-ASK es fácil de analizar. Primero, es **razonable**, porque cada inferencia es sólo una aplicación de Modus Ponens Generalizado, que es razonable. Segundo, es **completo** para bases de conocimiento de cláusulas definidas; es decir, responde a cada consulta cuyas respuestas están implicadas por cualquier BC de cláusulas definidas.

Para BC Datalog, que no contienen símbolos de función, la prueba de completitud es muy fácil. Comenzamos contando el número de hechos posibles que se pueden agregar, lo que determina el número máximo de iteraciones. Sea k la **aridad** máxima (número de argumentos) de cualquier predicado, p el número de predicados, y n el número de símbolos constantes. No puede haber más de pn^k hechos distintos de facto, así que después de estas iteraciones, el algoritmo debe haber alcanzado un punto fijo. Luego podemos hacer un argumento muy similar a la demostración de completitud para el *fc* proposicional. (Vea la p. 231.) Los detalles sobre cómo hacer la

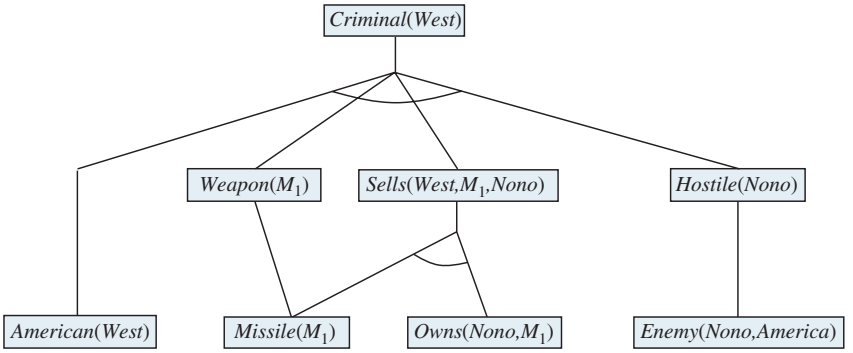


Figura 9.4 Árbol de pruebas generado por el encadenamiento directo en el ejemplo del crimen. Los hechos iniciales aparecen en el nivel inferior, los hechos inferidos en la primera iteración en el nivel medio, y hechos inferidos en la segunda iteración en el nivel superior.

transición de la completitud proposicional a la completitud de primer orden se dan para el algoritmo de resolución en la Sección 9.5.

Para cláusulas definidas generales con símbolos de función, FOL-FC-Ask puede generar hechos nuevos de manera infinita, así que debe ser cuidadoso. Para el caso en el que una respuesta a la oración de consulta q está implicada, se debe recurrir al teorema de Herbrand (página 282) para establecer que el algoritmo encontrará una prueba. (Consulte la Sección 9.5 para el caso de resolución). Si la consulta no tiene respuesta, el algoritmo podría no terminar en algunos casos. Así, si la BC incluye los axiomas de Peano

$$NatNum(0)$$
$$\forall n\ NatNum(n) \Rightarrow NatNum(S(n)),$$

entonces el *fc* agrega $NatNum(S(0))$, $NatNum(S(S(0)))$, $NatNum(S(S(S(0))))$, y así sucesivamente. Este problema en general es inevitable. Al igual que con la lógica general de primer orden, la vinculación con cláusulas definidas es semidecidible.

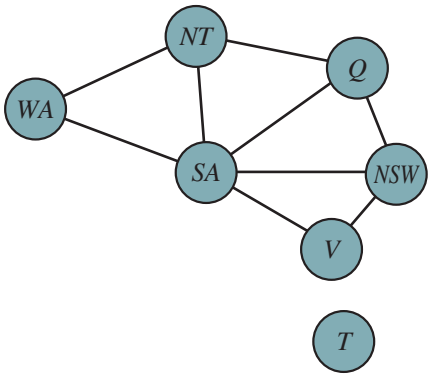
9.3.3 Encadenamiento hacia adelante (*fc*) eficiente

El algoritmo de *fc* (forward-chaining) en la Figura 9.3 está diseñado para facilitar la comprensión, no la eficiencia. Hay tres fuentes de ineficiencia. Uno, el bucle interno del algoritmo intenta hacer coincidir cada regla con cada hecho en la BC. Dos, el algoritmo vuelve a verificar cada regla en cada iteración, aun si se han realizado muy pocas adiciones a la BC. Tres, el algoritmo puede generar numerosos hechos irrelevantes para el objetivo. Abordamos cada uno de estos problemas en su debido momento.

Coincidencia de reglas en contra de hechos conocidos

El problema de hacer coincidir la premisa de una regla con los hechos en la BC puede parecer muy simple. Por ejemplo, supongamos que queremos aplicar la regla

$$Missile(x) \Rightarrow Weapon(x).$$



$$Diff(wa,nt) \wedge Diff(wa, sa) \wedge$$
$$Diff(nt,q) \wedge Diff(nt, sa) \wedge$$
$$Diff(q,nsw) \wedge Diff(q, sa) \wedge$$
$$Diff(nsw,v) \wedge Diff(nsw, sa) \wedge$$
$$Diff(v, sa) \Rightarrow Colorable()$$

$$Diff(Red,Blue) \quad Diff(Red,Green)$$
$$Diff(Green,Red) \quad Diff(Green,Blue)$$
$$Diff(Blue,Red) \quad Diff(Blue,Green)$$

Figura 9.5 (a) Gráfico de restricciones para colorear el mapa de Australia. (b) El CSP (Problema de satisfacción de restricciones) para colorear el mapa expresado como una única cláusula definida. Cada región se representa como variable con valor de una de las constantes *Red, Green,* o *Blue* (declaradas como *Diff*).

Entonces necesitamos encontrar todos los hechos que unifiquen con $Missile(x)$; en una BC adecuadamente indexado, esto se puede hacer en un tiempo constante por hecho. Consideremos ahora una regla como:

$$Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono).$$

De nuevo, podemos encontrar todos los objetos propiedad de Nono en tiempo constante por objeto; y, para cada objeto, verificaríamos si es un misil. Pero si la BC contiene muchos objetos propiedad de Nono y pocos misiles, entonces sería mejor encontrar todos los misiles primero y luego verificar si pertenecen a Nono. Este es el problema de **ordenación de los conjuntos**: encontrar un orden para resolver los conjuntos de la premisa de la regla de modo que el costo total se minimice. Encontrar el ordenamiento óptimo es NP-complejo, pero hay buenas heurísticas disponibles. Por ejemplo, la heurística de **valores mínimos restantes** (MRV, por sus siglas en inglés) usada para los CSP en el Capítulo 6 sugeriría ordenar los conjuntos para buscar misiles primero si hay menos misiles que objetos propiedad de Nono.

Ordenación de conjuntos

Coincidencia de patrones

La conexión entre esta **coincidencia de patrones** y la satisfacción de restricciones es en realidad estrecha. Podemos ver cada conjunto como una restricción en las variables que contiene, por ejemplo, $Misil(x)$ es una restricción unaria en x. Extendiendo esta idea, *expresamos cada CSP de dominio finito como una sola cláusula definida junto con algunos hechos asociados*. Considere el problema de colorear mapas de la Figura 6.1, que se vuelve a mostrar en la Figura 9.5(a). Una formulación equivalente como una sola cláusula definida se presenta en la Figura 9.5(b). Es claro, la conclusión $Colorable()$ sólo se infiere si el CSP tiene solución. Como los CSP en general incluyen problemas 3-SAT como casos especiales, podemos concluir que *hacer coincidir una cláusula definida con un conjunto de hechos es NP-complejo.*

Podría parecer bastante deprimente que el *fc* tenga un problema de coincidencia NP- complejo en su bucle interno. Hay tres formas de animarse:

- Recordemos, la mayoría de las reglas en BC del mundo real son pequeñas y simples (como las de nuestro ejemplo de crimen) en vez de grandes y complejas (como la formulación de CSP en la Figura 9.5). Es común en el mundo de las bases de datos asumir que los tamaños de las reglas y las aridades de los predicados están limitados por una constante, así como

preocuparse sólo por la **complejidad de los datos**, la complejidad de la inferencia como una función de la BC. Es fácil demostrar que la complejidad de datos del *fc* es polinomial, no exponencial. Complejidad de los datos

- Podemos considerar subclases de reglas en las que la coincidencia es eficiente. En esencia, cada cláusula Datalog es vista como definiendo un CSP, por ello la coincidencia será tratable justo cuando el CSP correspondiente sea tratable. El Capítulo 6 describe varias familias tratables de CSPs. Por ejemplo, si el grafo de restricciones (de nodos variables y cuyos enlaces son restricciones) forma un árbol, entonces el CSP se resuelve en tiempo lineal. Exactamente el mismo resultado se aplica a la coincidencia de reglas; así, si eliminamos Australia del Sur del mapa en la Figura 9.5, la cláusula será

  $$Diff(wa,nt) \land Diff(nt,q) \land Diff(q,nsw) \land Diff(nsw,v) \Rightarrow Colorable()$$

  lo que corresponde al CSP reducido (Figura 6.12, p. 201). Los algoritmos para resolver CSPs estructurados en árboles se aplican directo al problema de coincidencia de reglas.
- Podemos intentar eliminar intentos redundantes de coincidencia de reglas en el algoritmo de encadenamiento hacia adelante, como se describe a continuación.

Encadenamiento incremental hacia adelante

Cuando mostramos cómo funciona el *fc* en el ejemplo del crimen, hicimos trampa. En particular, omitimos parte de la coincidencia de reglas realizada por el algoritmo de la Figura 9.3. Por ejemplo, en la segunda iteración, la regla

$$Missile(x) \Rightarrow Weapon(x)$$

coincide con $Missile(M_1)$ (de nuevo), y claro, la conclusión $Weapon(M_1)$ ya se conoce, por lo que no ocurre nada. Tal coincidencia redundante de reglas se puede evitar si observamos: *Cada nuevo hecho inferido en la iteración t debe derivarse de al menos un nuevo hecho inferido en la iteración t − 1.* Esto es cierto porque cualquier inferencia que no requiera un nuevo hecho de la iteración $t - 1$ podría haberse hecho en la iteración $t - 1$.

Esta observación conduce naturalmente a un algoritmo de encadenamiento incremental hacia adelante donde, en la iteración t, verificamos una regla sólo si su premisa incluye un conjuntó p_i que coincide con un hecho p'_i recién inferido en la iteración t–1. El paso de coincidencia de reglas luego fija p_i para que coincida con p'_i, pero permite que los otros conjuntos de la regla coincidan con hechos de cualquier iteración anterior. Este algoritmo genera exactamente los mismos hechos en cada iteración que el algoritmo en la Figura 9.3, pero es mucho más eficiente.

Con indexación adecuada es fácil identificar las reglas que pueden ser desencadenadas por todo hecho dado, además muchos sistemas reales operan en modo de "actualización" en que el *fc* es respuesta a cada *TELL*. Las inferencias se propagan por el conjunto de reglas hasta que se alcanza el punto fijo; luego el proceso recomienza para el próximo nuevo hecho.

Típicamente, sólo una pequeña fracción de las reglas en la BC son realmente desencadenadas por la adición de un hecho dado. Esto significa que se realiza una gran cantidad de trabajo redundante al construir repetidamente coincidencias parciales que tienen algunas premisas insatisfechas. Nuestro ejemplo de crimen es muy pequeño para mostrar esto de manera efectiva, pero observe que se construye una coincidencia parcial en la primera iteración entre la regla

$$American(x) \land Weapon(y) \land Sells(x,y,z) \land Hostile(z) \Rightarrow Criminal(x)$$

y el hecho $American(West)$. Esta coincidencia parcial luego se descarta y se reconstruye en la segunda iteración (cuando la regla tiene éxito). Sería mejor retener y completar gradualmente las coincidencias parciales conforme llegan nuevos hechos, en lugar de desecharlos.

Algoritmo Rete

El **algoritmo Rete**[3] fue el primero en abordar este problema. El algoritmo preprocesa el conjunto de reglas en la BC para construir una red de flujo de datos en la que cada nodo es un literal de una premisa de regla. Las asignaciones de variables fluyen por la red y se filtran cuando no coinciden con un literal. Si dos literales en una regla comparten una variable —en el ejemplo del crimen—, $Sells(x,y,z) \wedge Hostile(z)$, entonces las asignaciones de cada literal se filtran por un nodo de igualdad. Una asignación de variable que llega a un nodo para un literal n-ario como $Sells(x,y,z)$ podría tener que esperar a que se establezcan las asignaciones para las otras variables antes de que el proceso pueda continuar. En un momento dado, el estado de una red Rete captura las coincidencias parciales de las reglas y evita una gran cantidad de recomputación.

Sistema de producción

Las redes Rete, y varias mejoras en ellas, han sido un componente clave de los llamados **sistemas de producción**, que fueron algunos de los primeros sistemas de adelanto en uso generalizado.[4] El sistema XCon (originalmente llamado R1; McDermott, 1982) construido con una arquitectura de sistema de producción, contenía varios miles de reglas para diseñar configuraciones de componentes informáticos para clientes de Digital Equipment Corporation, uno de los primeros éxitos comerciales claros en el naciente campo de sistemas expertos. Se han construido muchos otros sistemas similares con la misma tecnología subyacente, que se ha implementado en el lenguaje de propósito general OPS-5.

Arquitecturas cognitivas

Los sistemas de producción también son populares en **arquitecturas cognitivas**, es decir, modelos de razonamiento humano, como ACT (Anderson, 1983) y SOAR (Laird *et al.*, 1987). En tales sistemas, la "memoria de trabajo" del sistema modela la memoria a corto plazo humana, y las producciones son parte de la memoria a largo plazo. En cada ciclo de operación, las producciones se comparan con la memoria de trabajo de los hechos. Una producción cuyas condiciones se satisfacen puede agregar o eliminar hechos en la memoria de trabajo. En contraste con la situación típica en las bases de datos, los sistemas de producción con frecuencia tienen muchas reglas y pocos hechos. Con tecnología de coincidencia optimizada adecuadamente, los sistemas operan en tiempo real con decenas de millones de reglas.

Hechos irrelevantes

Otra fuente de ineficiencia es que el avance progresivo realiza las inferencias permitidas basadas en los hechos conocidos, *aun si son irrelevantes para el objetivo*. En nuestro ejemplo del crimen no había reglas capaces de sacar conclusiones irrelevantes. Pero si hubiera muchas reglas que describieran los hábitos alimenticios de los estadounidenses o los componentes y precios de los misiles, entonces FOL-FC-Ask habría generado conclusiones irrelevantes.

Bades de datos deductivas

Para evitar conclusiones irrelevantes se usa el *bc*, descrito en Sección 9.4. O restringir el *fc* a un subconjunto seleccionado de reglas, como en PL-FC-Entails (p. 231). Un tercer enfoque surgió en el campo de las **bases de datos deductivas**, bases a gran escala, como las bases de datos relacionales, que usan el *fc* como herramienta de inferencia estándar en vez de las consultas SQL. La idea es reescribir el conjunto de reglas, usando información del objetivo, para considerar sólo las asignaciones de variables relevantes, pertenecientes al **conjunto mágico**, durante la inferencia hacia adelante. Así, si el objetivo es $Criminal(West)$, la regla que concluye $Criminal(x)$ se reescribirá para incluir un extra conjuntivo que limite el valor de x:

Conjunto mágico

$$Magic(x) \wedge American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x).$$

[3] Red, del latín *rete*.

[4] La palabra **producción** en **sistemas de producción** denota una regla de condición-acción.

El hecho *Mágico*(*West*) se agrega a la BC. Así, incluso si la BC aloja hechos sobre millones de estadounidenses, sólo el coronel West será considerado en el proceso de inferencia hacia adelante. El proceso completo para definir conjuntos mágicos y reescribir la BC es muy complejo para entrar en detalles, pero la idea básica es realizar una especie de inferencia hacia atrás "genérica" desde el objetivo para determinar qué enlaces de variables deben ser restringidos. Así, el enfoque de los conjuntos mágicos puede considerarse como un tipo de híbrido entre la inferencia hacia adelante y el preprocesamiento hacia atrás.

9.4 Encadenamiento hacia atrás (*bc*)

La segunda familia principal de algoritmos de inferencia lógica usa *bc*, sobre cláusulas definidas. Los algoritmos trabajan hacia atrás desde el objetivo, encadenándose por reglas para encontrar hechos conocidos que respalden la prueba.

bc = backward chaining [por sus siglas en inglés]

9.4.1 Algoritmo de encadenamiento hacia atrás (*bc*, backward-chaining)

BC = *Knowledge base* [Base de conocimiento]

La Figura 9.6 muestra un algoritmo de *bc* para cláusulas definidas. FOL-BC-Ask(KB, *goal*)se demostrará si la BC tiene una regla de la forma $lhs \Rightarrow goal$, donde *lhs* (lado izquierdo) es una lista de conjuntos. Un hecho atómico como *American*(*West*) se considera una cláusula cuyo *lhs* es la lista vacía. Una consulta con variables se demostraría de muchas maneras; como, la consulta *Person*(*x*) se demostraría con la sustitución {*x/John*} y con {*x/Richard*}. Así, implementamos FOL-BC-Ask como un generador, una función da múltiples resultados, uno posible cada vez (vea el Apéndice B).

El *bc* (encadenamiento hacia atrás) es un tipo de búsqueda AND/OR: la parte OR porque la consulta objetivo puede ser demostrada por cualquier regla en la BC, y AND porque todo conjunto en el *lhs* de una cláusula debe ser demostrado. FOL-BC-Or funciona al obtener todas las cláusulas

```
function FOL-BC-Ask(KB, query) returns un generador de substituciones
    return FOL-BC-Or(KB, query, {})

function FOL-BC-Or(KB, objetivo, θ) returns una substitución
    for each rule en Fetch-Rules-For-Goal(KB, objetivo) do
        (lhs ⇒ rhs)←Standardize-Variables(regla)
        for each θ′ in FOL-BC-And(KB, lhs,Unify(rhs, objetivo, θ)) do
            yield θ′

function FOL-BC-And(KB, objetivos, θ) returns una substitución
    if θ = falla then return
    else if Length(objetivos) = 0 then yield θ
    else
        primer,rest←First(objetivos), Rest(objetivos)
        for each θ′ in FOL-BC-Or(KB, SUBST(θ, primer), θ) do
            for each θ″ in FOL-BC-And(KB, rest, θ′) do
                yield θ″
```

Figura 9.6 Algoritmo simple de *bc* para BC (base de conocimiento) de primer orden.

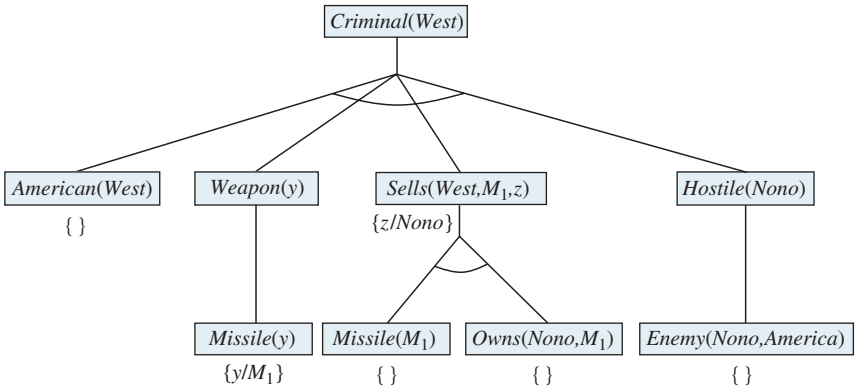


Figura 9.7 Árbol de pruebas construido por *bc* para demostrar que West es criminal. Su lectura es en profundidad y de izquierda a derecha. Para probar *Criminal*(*West*), debemos probar las cuatro conjunciones debajo de él. Algunas están en la *BC* y otras requieren un *bc* adicional. Los enlaces para cada unificación exitosa se muestran junto al subobjetivo correspondiente. Note que una vez que un subobjetivo de una conjunción tiene éxito, su sustitución se aplica a los subobjetivos posteriores. Así, cuando FOL-BC-Ask llegue a la última conjunción, originalmente *Hostil*(*z*), *z* ya está vinculado a *Nono*.

que unificarían con el objetivo, estandarizando las variables en la cláusula para que sean por completo nuevas, y luego, si la derecha (*rhs*) de la cláusula realmente se unifica con la meta, demostrando cada conjunción en el *lhs*, usando FOL-BC-And. Esa función demuestra cada una de las conjunciones por turno al dar seguimiento de la sustitución acumulada conforme avanza. La figura 9.7 es el árbol de prueba para derivar *Criminal* (*West*) a partir de las oraciones (9.3) a (9.10).

El *bc*, tal como la hemos escrito, es con claridad un algoritmo de búsqueda en profundidad. Esto significa que sus requisitos de espacio son lineales en el tamaño de la prueba. También significa que el *bc* (a diferencia del encadenamiento hacia adelante) sufre problemas con estados repetidos e incompletitud. A pesar de estas limitaciones, el *bc* ha demostrado ser popular y efectivo en los lenguajes de programación lógica.

9.4.2 Programación lógica

La programación lógica es una tecnología que se acerca a encarnar el ideal declarativo descrito en el Capítulo 7: los sistemas deben ser construidos expresando conocimiento en un lenguaje formal y los problemas deben ser resueltos ejecutando procesos de inferencia sobre ese conocimiento. El ideal se resume en la ecuación de Robert Kowalski,

Algoritm = *Logic* + *Control*.

Prolog

Prolog es el lenguaje de programación lógica más ampliamente utilizado. Se usa como lenguaje de prototipado rápido y para tareas de manipulación de símbolos como escribir compiladores (Van Roy, 1990) y analizar el lenguaje natural (Pereira y Warren, 1980). Se han escrito muchos sistemas expertos en Prolog para dominios legales, médicos, financieros y otros.

Los programas de Prolog son conjuntos de cláusulas definidas escritas en una notación diferente a la lógica estándar de primer orden. Prolog usa letras mayúsculas para variables y minúsculas para constantes, lo contrario de nuestra convención para la lógica. Las comas separan los

conjuntos en una cláusula, esta se escribe "al revés" a lo que estamos acostumbrados; en lugar de $A \wedge B \Rightarrow C$, en `Prolog`. Un ejemplo típico es:

```
criminal(X) :— american(X), weapon(Y), sells(X,Y,Z), hostile(Z).
```

En `Prolog`, la notación `[E|L]` denota una lista cuyo primer elemento es `E` y cuyo resto es `L`. Aquí hay un programa `Prolog` para `append(X,Y,Z)`, que tiene éxito si la lista `Z` es el resultado de concatenar las listas `X` y `Y`:

```
append([],Y,Z),
append([A|X],Y,[A|Z) :— append(X,Y,Z).
```

A alto nivel, podemos leer estas cláusulas como (1) agregar la lista vacía y la lista `Y` produce la misma lista `Y`, y (2) `[A|Z]` es el resultado de agregar `[A|X]` y `Y`, siempre que `Z` sea el resultado de agregar `X` y `Y`. En la mayoría de los lenguajes de alto nivel, podemos escribir una función recursiva similar que describe cómo agregar dos listas. Pero la definición de Prolog es en realidad más poderosa, porque describe una *relation* que se mantiene entre tres argumentos, en vez de una *function* calculada a partir de dos argumentos. Por ejemplo, podemos hacer la consulta `append(X,Y,[1,2,3])`: ¿qué dos listas se pueden concatenar para dar `[1,2,3]`? Prolog nos devuelve las soluciones

```
X=[]        Y=[1,2,3];
X=[1]       Y=[1,2];
X=[1,2]     Y=[3];
X=[1,2,3]   Y=[];
```

La ejecución de los programas Prolog se realiza mediante *bc* de profundidad primera, donde las cláusulas se prueban en el orden en que están escritas en la BC. El diseño de Prolog representa un compromiso entre la declaratividad y la eficiencia de ejecución. Algunos aspectos de Prolog quedan fuera de la inferencia lógica estándar:

- Prolog usa la semántica de base de datos de la Sección 8.2.8 y no la semántica de primer orden, esto se evidencia en su tratamiento de igualdad y negación (vea la Sección 9.4.4).
- Hay un conjunto de funciones incorporadas para aritmética. Los literales que usan estos símbolos de función se "prueban" ejecutando código en vez de hacer más inferencias. Así, el objetivo "`X es 4+3`" tiene éxito con `X` vinculado a 7. Por otro lado, el objetivo "`5 es X+Y`" falla, porque las funciones incorporadas no hacen resolución de ecuaciones arbitrarias.
- Hay predicados integrados con efectos secundarios al ejecutarse. Estos incluyen predicados de entrada/salida y predicados `assert/retract` para modificar la BC. Tales predicados no tienen contraparte en la lógica y pueden producir resultados confusos, por ejemplo, si los hechos se afirman en una rama del árbol de prueba que eventualmente falla.
- La **verificación de ocurrencia** se omite del algoritmo de unificación de Prolog. Esto significa que es posible hacer inferencias no válidas, que casi nunca son problema en la práctica.
- Prolog usa la búsqueda de *bc* de profundidad primero sin comprobar la recursión infinita. Esto lo hace un lenguaje de programación utilizable muy rápido cuando se usa correctamente, pero significa que algunos programas que parecen lógica válida no terminarán.

9.4.3 Inferencia redundante y bucles infinitos

Volvamos al talón de Aquiles de Prolog: la discrepancia entre la búsqueda de profundidad primera y los árboles de búsqueda que incluyen estados repetidos y caminos infinitos. Considere el siguiente programa lógico que decide si existe un camino entre dos puntos en un grafo dirigido:

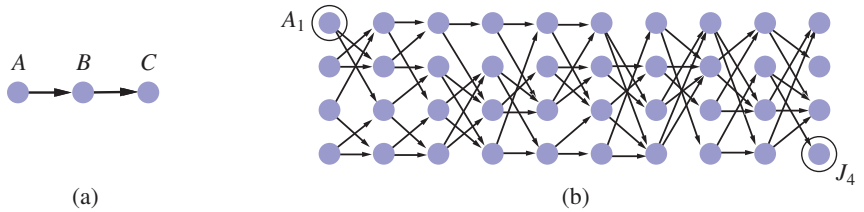


Figura 9.8 (a) Encontrar una ruta de A a C puede llevar a Prolog a un bucle infinito. (b) Un grafo donde cada nodo está conectado a dos sucesores aleatorios en la siguiente capa. Encontrar una ruta de A_1 a J_4 requiere 877 inferencias.

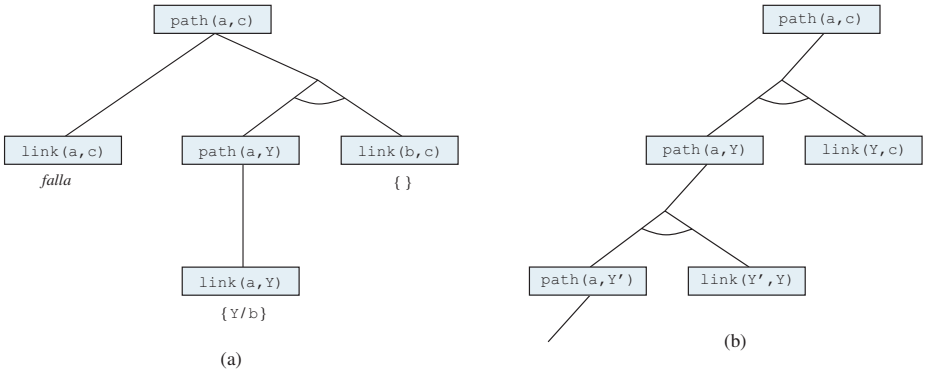


Figura 9.9 (a) Prueba de que existe una ruta de A a C. (b) Árbol de prueba infinito generado cuando las cláusulas están en el orden "incorrecto".

```
path(X,Z) :– link(X,Z).
path(X,Z) :– path(X,Y), link(Y,Z).
```

Se muestra un grafo simple de tres nodos, descrito por los hechos `link(a,b)` y `link(b,c)`, en la Figura 9.8(a). Con este programa, la consulta `path(a,b)` genera el árbol de prueba mostrado en la Figura 9.9(a). Además, si colocamos los dos enunciados en el orden

```
path(X,Z) :– path(X,Y), link(Y,Z).
path(X,Z) :– link(X,Z).
```

entonces Prolog sigue la ruta infinita mostrada en la Figura 9.9(b). Así, Prolog es **incompleto** como demostrador de teoremas para cláusulas definidas, aun para programas Datalog, como se ejemplifica; esto debido a que, para algunas BC no es posible demostrar oraciones implicadas. Note que el *fc* no sufre este problema: cuando se infieren `path(a,b)`, `path(b,c)` y `path(a,c)`, este encadenamiento se detiene.

El *bc* de profundidad primero tiene problemas con cálculos redundantes. Por ejemplo, al encontrar un *path* desde A_1 a J_4 en la Figura 9.8(b), Prolog hace 877 inferencias, la mayoría involucran encontrar todo posible *path* a nodos desde los cuales el objetivo es inalcanzable. Esto es similar al problema de estados repetidos discutido en el Capítulo 3. El total de la inferencia puede ser

exponencial en el número de hechos concretos que se generan. Si en su lugar aplicamos el encadenamiento hacia adelante, como máximo se pueden generar n^2 `path(X,Y)` hechos que vinculan n nodos. Para el problema en la Figura 9.8(b), sólo se necesitan 62 inferencias.

Programación dinámica

El *fc* en problemas de búsqueda de gráficos es ejemplo de **programación dinámica**, donde las soluciones a subproblemas se construyen incrementalmente a partir de las de subproblemas más pequeños y se almacenan en caché para evitar recomputación. Podemos obtener un efecto similar en un sistema de *bc*, excepto que aquí descomponemos objetivos grandes en objetivos más pequeños, en lugar de construirlos.

Programación lógica tabulada

De cualquier modo, almacenar resultados intermedios para evitar la duplicación es clave. Este es el enfoque adoptado por los sistemas de **programación lógica tabulados**, que usan mecanismos de almacenamiento y recuperación eficientes. La programación lógica tabulada combina la orientación hacia el objetivo del *bc* con la eficiencia de la programación dinámica del encadenamiento hacia adelante. También es completa para bases de conocimiento Datalog, lo que significa que el programador no tiene que preocuparse tanto por los bucles infinitos. (Aún es posible obtener un bucle infinito con predicados como `padre(X,Y)` que refieren a un número en potencia ilimitado de objetos.)

9.4.4 Semántica de base de datos de Prolog

Prolog usa semántica de base de datos (vea la Sección 8.2.8). La suposición de nombres únicos dice que cada constante de Prolog y cada término concreto refiere a un objeto distinto, y la suposición de mundo cerrado indica que las únicas oraciones verdaderas son implicadas por la BC. No se puede afirmar que una oración es falsa en Prolog. Esto hace que Prolog sea menos expresivo que la lógica de primer orden, pero es parte de que Prolog sea más eficiente y conciso. Considere las siguientes afirmaciones sobre algunas ofertas de cursos:

$$Course(CS,101),\ Course(CS,102),\ Course(CS,106),\ Course(EE,101). \tag{9.11}$$

Bajo la suposición de nombres únicos, *CS* y *EE* son diferentes (al igual que 101, 102 y 106), lo que significa que hay cuatro cursos distintos. Bajo la suposición de mundo cerrado no hay otros cursos, por ello hay exactamente cuatro cursos. Pero si estas fueran afirmaciones en FOL en vez de en semántica de base de datos, entonces todo lo que diríamos es que hay entre uno e infinitos cursos en algún lugar. Eso se debe a que las afirmaciones (en FOL) no niegan la posibilidad de que también se ofrezcan cursos no mencionados, ni indican que éstos son diferentes entre sí. Si quisiera traducir la Ecuación (9.11) a FOL, obtendría esta oración:

$$\begin{aligned} Course(d,n) \quad \Leftrightarrow \quad & (d = CS \wedge n = 101) \vee (d = CS \wedge n = 102) \\ & \vee (d = CS \wedge n = 106) \vee (d = EE \wedge n = 101) \end{aligned} \tag{9.12}$$

Completitud

Esto se llama **completitud** de la Ecuación (9.11). Expresa en FOL la idea de que hay como máximo cuatro cursos. Para expresar en FOL la idea de que hay al menos cuatro cursos, necesitamos escribir la completitud del predicado de igualdad:

$$\begin{aligned} x = y \quad \Leftrightarrow \quad & (x = CS \wedge y = CS) \vee (x = EE \wedge y = \text{EE}) \vee (x = 101 \wedge y = 101) \\ & \vee (x = 102 \wedge y = 102) \vee (x = 106 \wedge y = 106) \end{aligned}$$

La completitud es útil para entender la semántica de base de datos, pero para propósitos prácticos, si su problema puede describirse con semántica de base de datos, es más eficiente razonar con Prolog u otro sistema de semántica de base de datos, en lugar de traducirlo a FOL y razonar con un demostrador de teoremas FOL completo.

9.4.5 Programación lógica con restricciones

En la discusión sobre el *fc* (Sección 9.3), se vio cómo los problemas de satisfacción de restricciones (CSPs) pueden ser codificados como cláusulas definidas. Prolog estándar los resuelve exactamente como el algoritmo de retroceso de Figura 6.5.

Dado que el retroceso enumera los dominios de las variables, sólo funciona para CSPs de **dominio finito**. En términos de Prolog, debe haber un número finito de soluciones para cualquier objetivo con variables no enlazadas. (Por ejemplo, un problema de coloración de mapas en el que cada variable toma uno de cuatro colores diferentes). Los CSPs de dominio infinito, por ejemplo, con variables valuadas como enteros o reales, requieren algoritmos muy diferentes, como la propagación de límites o la programación lineal.

Consideremos este ejemplo: Se define `triangle(X,Y,Z)` como un predicado que se cumple si los tres argumentos son números que satisfacen la desigualdad triangular:

```
triangle(X,Y,Z) :—
X>0, Y>0, Z>0, X+Y>Z, Y+Z>X, X+Z>Y
```

Si le preguntamos a Prolog la consulta `triangle(3,4,5)`, tiene éxito. Por otro lado, si preguntamos `triangle(3,4,Z)`, no se encontrará ninguna solución, porque el subobjetivo `Z>0` no puede ser manejado por Prolog; no podemos comparar un valor no enlazado con `0`.

Programación lógica con restricciones

La **programación lógica con restricciones** (CLP, por sus siglas en inglés) permite que las variables estén *restringidas* en vez de *enlazadas*. Una solución CLP es el conjunto más específico de restricciones sobre las variables de consulta que pueden derivar de la BC. Así, la solución a la consulta `triangle(3,4,Z)` es la restricción `7 > Z >1`. Los programas de lógica estándar son sólo un caso especial de CLP donde las restricciones de solución deben ser restricciones de igualdad: enlaces.

Los sistemas CLP usan varios algoritmos de resolución de restricciones para las restricciones permitidas en el lenguaje. Así, un sistema que permite desigualdades lineales en variables valuadas incluiría un algoritmo de programación lineal para resolver restricciones. Los sistemas CLP adoptan un enfoque más flexible para resolver consultas de programación lógica estándar. Esto es, en lugar de retroceso de profundidad primero a la izquierda, usarían cualquiera algoritmo más eficiente del Capítulo 6, incluido el ordenamiento de conjuntos heurísticos, el retroceso, la condición de conjunto de corte, etc. Los sistemas CLP combinan elementos de algoritmos de satisfacción de restricciones, programación lógica y bases de datos deductivas.

Metarreglas

Se han definido varios sistemas que dan al programador más control sobre el orden de búsqueda para la inferencia. El lenguaje MRS (Genesereth y Smith, 1981; Russell, 1985) permite al programador escribir **metarreglas** para determinar qué conjuntos se intentan primero. El usuario podría escribir una regla que indique que el objetivo con menos variables debe intentarse primero, o podría escribir reglas específicas de dominio para predicados particulares.

9.5 Resolución

La última de las tres familias de sistemas lógicos, la única que funciona para cualquier BC, no sólo para cláusulas definidas, es la **resolución**. En la página 223 vio que la resolución proposicional es un procedimiento de inferencia completo para la lógica proposicional; aquí, la extendemos a la lógica de primer orden.

9.5.1 Forma normal conjuntiva para lógica de primer orden

El paso uno es convertir las oraciones a **forma normal conjuntiva** (FNC), es decir, una conjunción de cláusulas, donde cada cláusula es una disyunción de literales.[5] En FNC, los literales contienen variables, que se asumen cuantificadas universalmente. Así, la oración

$$\forall x,y,z\ American(x) \land Weapon(y) \land Sells(x,y,z) \land Hostile(z) \Rightarrow Criminal(x)$$

se convierte, en FNC,

$$\neg American(x) \lor \neg Weapon(y) \lor \neg Sells(x,y,z) \lor \neg Hostile(z) \lor Criminal(x).$$

La clave es que *Cada oración de lógica de primer orden puede convertirse en una oración FNC inferencialmente equivalente.*

El procedimiento de conversión a FNC es similar al caso proposicional de página 226. La diferencia principal surge de la necesidad de eliminar cuantificadores existenciales. El procedimiento se ilustra con la traducción de la oración "Todos los que aman a todos los animales es amado por alguien", o

$$\forall x\ [\forall y\ Animal(y) \Rightarrow Loves(x,y)] \Rightarrow [\exists y\ Loves(y,x)].$$

Los pasos son los siguientes:

- **Eliminar implicaciones:** Reemplazar $P \Rightarrow Q$ con $\neg P \lor Q$. Para nuestra muestra de oración, esto necesita hacerse dos veces:

  $$\forall x\ \neg[\forall y\ Animal(y) \Rightarrow Loves(x,y)] \lor [\exists y\ Loves(y,x)]$$
  $$\forall x\ \neg[\forall y\ \neg Animal(y) \lor Loves(x,y)] \lor [\exists y\ Loves(y,x)].$$

- **Mover ¬ hacia adentro:** Además de las reglas usuales para las conectivas negadas, necesitamos reglas para los cuantificadores negados. Así, tenemos

  $\neg\forall x\ p$ se convierte en $\exists x\ \neg p$.
  $\neg\exists x\ p$ se convierte en $\forall x\ \neg p$.

  Nuestra oración pasa por las siguientes transformaciones:

  $$\forall x\ [\exists y\ \neg(\neg Animal(y) \lor Loves(x,y))] \lor [\exists y\ Loves(y,x)].$$
  $$\forall x\ [\exists y\ \neg\neg Animal(y) \land \neg Loves(x,y)] \lor [\exists y\ Loves(y,x)].$$
  $$\forall x\ [\exists y\ Animal(y) \land \neg Loves(x,y)] \lor [\exists y\ Loves(y,x)].$$

  Note cómo un cuantificador universal ($\forall y$) en la premisa de la implicación se ha convertido en un cuantificador existencial. La oración ahora dice "Either there is some animal that x doesn't love, or (if this is not the case) someone loves x". Se ha preservado el significado de la oración original.
- **Estandarizar variables:** Para oraciones como $(\exists x\ P(x)) \lor [\exists x\ Q(x))$ que usan el mismo nombre de variable dos veces, cambie el nombre de una de las variables. Esto evita confusiones posteriores cuando se eliminen los cuantificadores. Así, se tiene

  $$\forall x\ [\exists y\ Animal(y) \land \neg Loves(x,y)] \lor [\exists y\ Loves(z,x)].$$

- **Esqueletar: Esqueletización** es el proceso de eliminar cuantificadores existenciales mediante la eliminación. En un caso simple, es similar a la regla de instanciación existencial de

Esqueletización

[5] Una cláusula también puede ser representada como una implicación con una conjunción de átomos en la premisa y una disyunción de átomos en la conclusión (Ejercicio 9.DISJ). Esto se denomina **forma normal implicativa** o **forma Kowalski** (en especial cuando se escribe con un símbolo de implicación de derecha a izquierda (Kowalski, 1979)) y en general es mucho más fácil de leer que una disyunción con muchos literales negados.

Sección 9.1: convirtiendo $\exists x\ P(x)$ en $P(A)$, donde A es una nueva constante. Sin embargo, no es posible aplicar la instantiación existencial a la oración anterior porque no coincide con el patrón $\exists \nu\ \alpha$; sólo partes de la oración coinciden con el patrón. Si se aplica ciegamente la regla a las dos partes coincidentes, se obtiene

$$\forall x\ [Animal(A) \land \neg Loves(x,A)] \lor Loves(B,x),$$

lo cual tiene un significado completamente erróneo: dice que todos o bien no aman un animal particular A o son amados por alguna entidad particular B. De hecho, la oración original permite que cada persona no ame un animal diferente o sea amada por una persona diferente. Así, se quiere que las entidades de Skolem dependan de x:

$$\forall x\ [Animal(F(x)) \land \neg Loves(x,F(x))] \lor Loves(G(x),x).$$

Función de Skolem

Aquí, F y G son **funciones de Skolem**. La regla general es que los argumentos de la función de Skolem son todas las variables cuantificadas universalmente en cuyo alcance aparece el cuantificador existencial. Al igual que con la instanciación existencial, la oración esqueletada se satisface exactamente cuando la oración original es satisfecha.

- **Eliminar cuantificadores universales:** Aquí, las variables restantes deben estar cuantificadas universalmente. Por ello, no se pierde información si se elimina el cuantificador:

  $$[Animal(F(x)) \land \neg Loves(x,F(x))] \lor Loves(G(x),x).$$

- **Distribuir $\lor$ sobre $\land$**:

  $$[Animal(F(x)) \lor Loves(G(x),x)] \land \neg Loves(x,F(x)) \lor Loves(G(x),x)].$$

  Este paso también puede requerir aplanar las conjunciones y disyunciones anidadas.

La oración ahora está en FNC y consta de dos cláusulas. Es mucho más difícil de leer que la oración original con implicaciones. (Puede ayudar explicar que la función de Skolem $F(x)$ se refiere al animal potencialmente no amado por x, mientras que $G(x)$ se refiere a alguien que podría amar (x). Por fortuna, rara vez los humanos necesitan mirar las oraciones FNC: el proceso de traducción es fácilmente automatizable.

9.5.2 La regla de inferencia de resolución

La regla de resolución para cláusulas de primer orden es una versión elevada de la regla de resolución proposicional de página 226. Dos cláusulas, que se asumen estandarizadas de modo que no comparten variables, pueden resolverse si contienen literales complementarios. Los literales proposicionales son complementarios si uno es la negación del otro; los literales de primer orden son complementarios si uno *se unifica con* la negación del otro. Así, tenemos

$$\frac{\ell_1 \lor \cdots \lor \ell_{k,} \quad m_1 \lor \cdots \lor m_{n,}}{\text{SUBST}(\theta,\ \ell_1 \lor \cdots \lor \ell_{i-1} \lor \ell_{i+1} \lor \cdots \lor \ell_k \lor m_1 \lor \cdots m_{j-1} \lor \text{m}_{\text{j+1}\ \lor} \cdots \lor m_n)}$$

donde UNIFY(ℓi,$\neg m_j$) = θ. Por ejemplo, podemos resolver las dos cláusulas

$$[Animal(F(x)) \lor Loves(G(x),x)] \text{ y } [\neg Loves(u,v) \lor \neg Kills(u,v)]$$

al eliminar literales complementarios $Loves(G(x),x)$ y $\neg Loves(u,v)$, con el unificador $\theta = \{u/G(x),v/x\}$, para producir la cláusula **resolvente**

$$[Animal(F(x)) \lor \neg Kills(G(x),x)].$$

Esta regla se llama regla de **resolución binaria** porque resuelve exactamente dos literales. La regla de resolución binaria por sí sola no da un procedimiento de inferencia completo. La regla completa de resolución resuelve subconjuntos de literales en cada cláusula que son unificables. Un enfoque alternativo es extender la **factorización**, eliminar literales redundantes, al caso de primer orden. La factorización proposicional reduce dos literales a uno si son *idénticos*; la factorización de primer orden reduce dos literales a uno si son *unificables*. El unificador se aplica a toda la cláusula. La combinación de resolución binaria y factorización es completa.

Resolución binaria

9.5.3 Ejemplos de prueba

La resolución demuestra que $KB \models \alpha$ al probar que $KB \wedge \neg\alpha$ es insatisfactorio, o sea, al derivar la cláusula vacía. El enfoque algorítmico es idéntico al caso proposicional, descrito en la Figura 7.13, por ello no es necesario repetirlo aquí. En cambio, ejemplificamos dos demostraciones. El primero es el ejemplo del crimen de la Sección 9.3. Las oraciones en FNC son

$$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x,y,z) \vee \neg Hostile(z) \vee Criminal(x)$$
$$\neg Missile(x) \vee \neg Owns(Nono,x) \vee \neg Sells(West,x,Nono)$$
$$\neg Enemy(x,American) \vee Hostile(x)$$
$$\neg Missile(x) \vee Weapon(x)$$
$$Owns(Nono,M_1) \qquad Missile(M_1)$$
$$American(West) \qquad Enemy(Nono,American)$$

También incluimos el objetivo negado $\neg Criminal(West)$. La prueba de resolución se ve en la Figura 9.10. Note la estructura: una sola "estructura principal" que comienza con la cláusula objetivo, resolviendo contra cláusulas de la BC hasta que se genera la cláusula vacía, característico de la resolución en BC de cláusulas de Horn. De hecho, las cláusulas a lo largo de la estructura principal coinciden *exactamente* a los valores consecutivos de la variable de *objetivos* en el algoritmo de *bc* de la Figura 9.6. Esto se debe a que siempre elegimos resolver

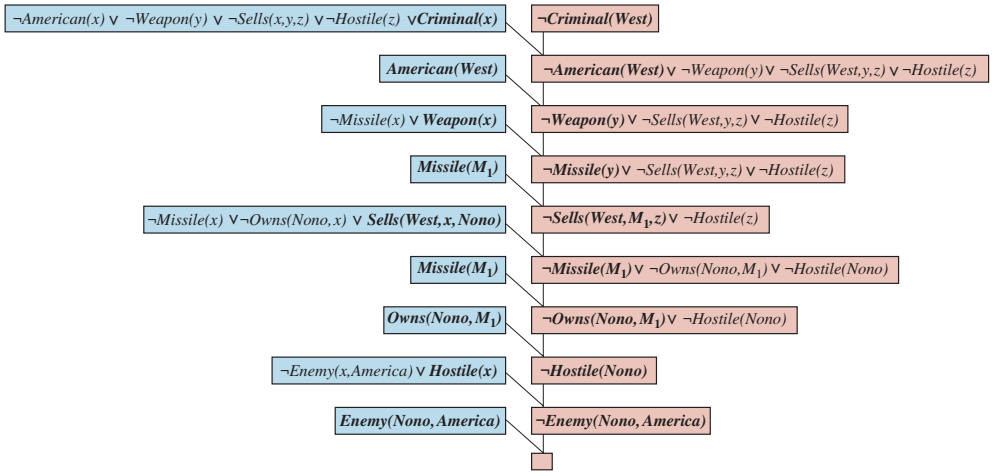


Figura 9.10 Una resolución prueba que West es un criminal. En cada paso de resolución, los literales que unifican están en negrita y la cláusula con el literal positivo está sombreada en azul.

con una cláusula cuyo literal positivo se unifica con el literal más a la izquierda de la cláusula "actual" en la estructura principal; esto es exacto lo que sucede en el *bc*. Así, el *bc* es sólo un caso especial de resolución con una estrategia de control particular para decidir qué resolución realizar después.

El segundo ejemplo usa la esqueletización e implica cláusulas que no son definidas. Esto resulta en una estructura de prueba algo más compleja. Por ejemplo:

> Todos los que aman a todos los animales son amados por alguien.
> Cualquiera que mate a un animal no es amado por nadie.
> Jack ama a todos los animales.
> Cualquiera, Jack o la curiosidad mataron al gato, que se llama Tuna.
> ¿La curiosidad mató al gato?

Primero, expresamos las oraciones originales, algunos conocimientos previos y el objetivo negado G en lógica de primer orden:

A. $\forall x\ [\forall y\ Animal(y) \Rightarrow Loves(x,y)] \Rightarrow [\exists y\ Loves(y,x)]$
B. $\forall x\ [\exists z\ Animal(z) \land Kills(x,z)] \Rightarrow [\forall y\ \neg Loves(y,x)]$
C. $\forall x\ Animal(x) \Rightarrow Loves(Jack,x)$
D. $Kills(Jack,Tuna) \lor Kills(Curiosity,Tuna)$
E. $Cat(Tuna)$
F. $\forall x\ Cat\ (x) \Rightarrow Animal(x)$
¬G. $\neg Kills\ (Curiosity,Tuna)$

Ahora aplicamos el procedimiento de conversión para convertir cada oración a FNC:

A1. $Animal(F(x)) \lor Loves(G(x),x)$
A2. $\neg Loves(x,F(x)) \lor Loves(G(x),x)$
B. $\neg Loves(y,x) \lor \neg Animal(z) \lor \neg Kills(x,z)$
C. $\neg Animal(x) \lor Loves(Jack,x)$
D. $Kills(Jack,Tuna) \lor Kills(Curiosity,Tuna)$
E. $Cat(Tuna)$
F. $\neg Cat(x) \lor Animal(x)$
¬G. $\neg Kills(Curiosity,Tuna)$

La prueba de resolución de que *la curiosidad mató al gato* [Curiosity killed the cat] se encuentra en la Figura 9.11. En este caso, la prueba podría parafrasearse de la siguiente manera:

> Suponga que *la curiosidad no mató a Atún*. Sabe que o bien Jack o la curiosidad lo hicieron; Jack debe haberlo hecho. Ahora, *Atún* es un gato y los gatos son animales, por ello *Atún* es un animal. Como cualquier persona que mate un animal no es amada por nadie, sabemos que nadie ama a Jack. Al contrario, Jack ama a todos los animales, así que alguien lo ama; luego, tenemos una contradicción. Entonces, la curiosidad mató al gato.

La prueba responde a "¿La curiosidad mató al gato?" pero con frecuencia queremos plantear preguntas más generales, como "¿Quién mató al gato?" La resolución puede hacer esto, pero requiere más trabajo obtener la respuesta. El objetivo es $\exists w\ Kills(w,Tuna)$, que, al negarse, se convierte en $\neg Kills(w,Tuna)$ en FNC. Repitiendo la prueba en la Figura 9.11 con el nuevo objetivo negado, obtenemos un árbol de prueba similar, pero con la sustitución $\{w/Curiosity\}$ en uno de los pasos. Así, en este caso, descubrir quién mató al gato es sólo cuestión de dar seguimiento de las asignaciones para las variables de consulta en la prueba. Con poco acierto, la resolución a veces produce **pruebas no constructivas** para objetivos existenciales, donde sabemos que una consulta es verdadera, pero no hay una asignación única para la variable.

Prueba no constructiva

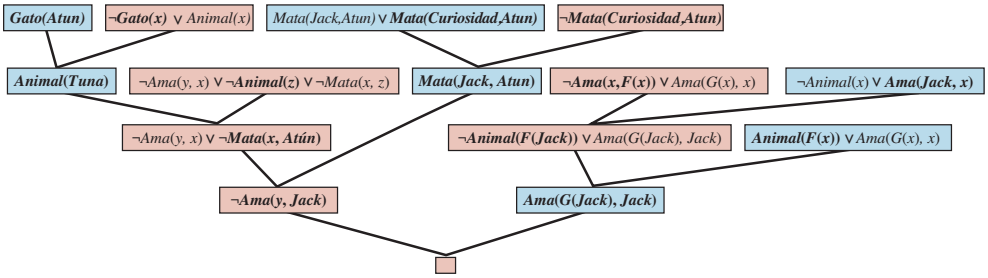


Figura 9.11 Prueba resuelta de *la curiosidad mató al gato*. Vea el uso de factorización en la derivación de la cláusula *Loves*(*G*(*Jack*),*Jack*). Note en la parte superior derecha que la unificación de *Loves*(*x*,*F*(*x*)) y *Loves*(*Jack*,*x*) sólo tendrá éxito luego que las variables se estandaricen.

9.5.4 Completitud de la resolución

Esta sección presenta una prueba de completitud de la resolución. Puede ser omitida por aquellos que estén dispuestos a aceptarlo sin más.

Demostramos que la resolución es **refutación completa**, es decir, que *si* un conjunto de oraciones es insatisfacible, entonces la resolución siempre derivará en contradicción. La resolución no puede usarse para generar todas las consecuencias lógicas de un conjunto de oraciones, pero puede usarse para establecer que una oración dada está implicada por el conjunto de oraciones. Así, se usa para encontrar todas las respuestas a una pregunta, $Q(x)$, demostrando que $KB \wedge \neg Q(x)$ esinsatisfacible.

Refutación completa

Damos por sentado que cualquier oración en lógica de primer orden (sin igualdad) puede ser reescrita como un conjunto de cláusulas en FNC. Esto puede demostrarse por inducción sobre la forma de la oración, usando oraciones atómicas como caso base (Davis y Putnam, 1960). El objetivo es demostrar: *si S es un conjunto de cláusulas insatisfacible, entonces la aplicación de un número finito de pasos de resolución a S producirá una contradicción*.

Nuestro esquema de prueba sigue la prueba original de Robinson con algunas simplificaciones de Genesereth y Nilsson (1987). Su estructura básica (Figura 9.12) es:

1. Vea que si S es insatisfacible, entonces existe un conjunto particular de *instancias fundamentales* de las cláusulas de S que también es insatisfacible (teorema de Herbrand).
2. Apelamos al **teorema de resolución fundamental (**Capítulo 7), que establece que la resolución proposicional es completa para oraciones fundamentales.
3. Usamos un **lema de elevación** para mostrar que, para toda prueba de resolución proposicional, usando el conjunto de oraciones fundamentales, existe una prueba de resolución de primer orden adecuada usando las oraciones de primer orden, de las que obtuvimos las fundamentales.

Para llevar a cabo el primer paso, son necesarios tres nuevos conceptos:

- **Universo de Herbrand:** Si S es un conjunto de cláusulas, entonces H_s, el universo de Herbrand de S, es el conjunto de todos los términos fundamentales construibles a partir de:

  a. Los símbolos de función en S, si los hay.
  b. Los símbolos constantes en S, si hay; si no hay, entonces un símbolo constante predeterminado, S.

Universo de Harbrand

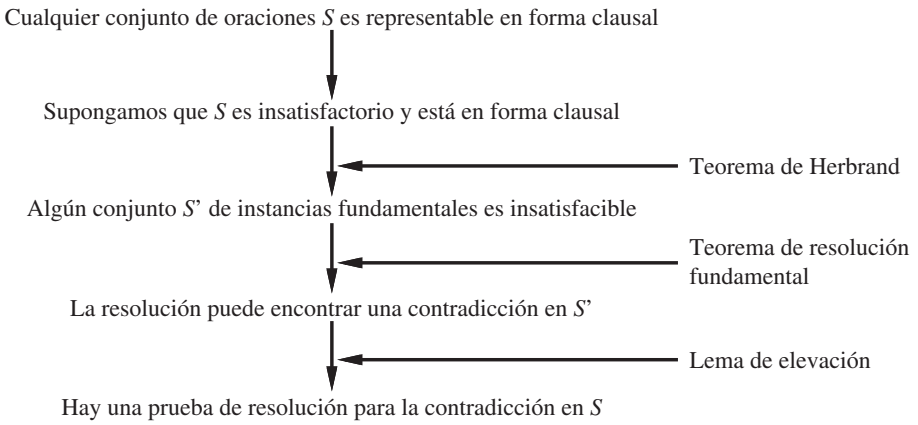


Figura 9.12 Estructura de una prueba de integridad para resolución.

Por ejemplo, si S contiene sólo la cláusula $\neg P(x,F(x,A)) \vee \neg Q(x,A) \vee R(x,B)$, entonces H_s es el siguiente conjunto infinito de términos fundamentales:

$$\{A,B,F(A,A),F(A,B),F(B,A),F(B,B),F(A,F(A,A)),...\}.$$

Saturación

- **Saturación**: Si S es un conjunto de cláusulas fundamentales y P uno de términos fundamentales, entonces $P(S)$, la saturación de S con respecto a P, es el conjunto de todas las cláusulas fundamentales obtenidas al aplicar todas las posibles sustituciones consistentes de términos fundamentales en P para variables en S.

Base de Herbrand

- **Base de Herbrand**: La saturación de un conjunto S de cláusulas con respecto a su universo de Herbrand se llama base de Herbrand de S, escrita como $HS(S)$. Así, si S contiene sólo la cláusula dada antes, entonces$HS(S)$ es el conjunto infinito de cláusulas

$$\{\neg P(A,F(A,A)) \vee \neg Q(A,A) \vee R(A,B),$$
$$\neg P(B,F(B,A)) \vee \neg Q(B,A) \vee R(B,B),$$
$$\neg P(F(A,A),F(F(A,A),A)) \vee \neg Q(F(A,A),A) \vee R(F(A,A),B),$$
$$\neg P(F(A,B),F(F(A,B),A)) \vee \neg Q(F(A,B),A) \vee R(F(A,B),B),...\}$$

Teorema de Herbrand

Estas definiciones nos permiten enunciar una forma del **teorema de Herbrand** (1930):

> Si un conjunto S de cláusulas es insatisfactorio, entonces existe un subconjunto finito de $H_s(S)$ que también es insatisfactorio.

Sea S' este subconjunto finito de oraciones fundamentales. Apelamos al teorema de resolución fundamental (p. 228) para demostrar que el **cierre de resolución** $RC(S')$ tiene la cláusula vacía. Es decir, ejecutar la resolución proposicional hasta su finalización en S' derivará una contradicción.

Una vez establecido que siempre hay una prueba de resolución que involucra un subconjunto finito de la base de Herbrand de S, el siguiente paso es demostrar que hay una prueba de resolución al usar las cláusulas de S en sí, que no son necesariamente cláusulas base. Iniciamos al considerar una única aplicación de la regla de resolución. Robinson planteó este lema:

> Sean C_1 y C_2 dos cláusulas sin variables compartidas, y C'_1 y C'_2 instancias fundamentales de C_1 y C_2. Si C' es un resolutivo de C'_1 y C'_2, entonces hay una cláusula C tal que (1) C es un resolutivo de C_1 y C_2 y (2) C' es instancia fundamental de C.

Teorema de incompletitud de Gödel

Al ampliar sutilmente el lenguaje de la lógica de primer orden para permitir el **esquema de inducción matemática** en aritmética, Kurt Gödel demostró en su **teorema de incompletitud** que hay oraciones aritméticas verdaderas que no pueden demostrarse.

La demostración del teorema de incompletitud sale del alcance de este libro, pues ocuparía mínimo 30 páginas, pero daremos una pista. Iniciamos con la teoría lógica de los números que tiene una única constante, 0, y una única función, S (la función sucesora). En el modelo previsto, $S(0)$ denota 1, $S(S(0))$ denota 2, y así en lo sucesivo. Por ello, el lenguaje tiene nombres para todos los números naturales. El vocabulario también incluye símbolos de función +, ×, y *Expt* (exponenciación) y el conjunto habitual de conectivos y cuantificadores lógicos.

Paso uno, notar que el conjunto de oraciones que podemos escribir en este idioma se puede enumerar. (Imagine definir un orden alfabético de los símbolos y organizarlos así, cada uno de los conjuntos de oraciones de longitud 1, 2, etc.). Luego podemos numerar cada oración con un número natural único $\#\alpha$ (el **número de Gödel**). Esto es crucial: la teoría de números tiene un nombre para cada una de sus propias oraciones. De manera similar, podemos numerar cada posible prueba P con un número de Gödel $G(P)$, porque una prueba es sólo una secuencia finita de oraciones.

Ahora suponga que tenemos un conjunto A recursivamente enumerable de oraciones que son afirmaciones verdaderas sobre los números naturales. Recuerde que A puede ser nombrado por un conjunto dado de números enteros, imaginemos escribir en nuestro lenguaje una oración $\alpha(j,A)$ así:

> $\forall i$ i no es el número de Gödel de una prueba de la oración cuyo número de Gödel es j, donde la prueba usa sólo premisas en A.

Entonces sea σ la oración $\alpha(\#\sigma,A)$, es decir, una oración que establece su propia imposibilidad de demostrar a partir de A. (Que esta oración siempre exista es cierto, pero no del todo obvio).

Presentamos el siguiente argumento ingenioso: suponga que σ es demostrable a partir de A; entonces σ es falso (porque σ dice que no se puede probar). Entonces tenemos una oración falsa demostrable a partir de A, porque A no puede consistir sólo en oraciones verdaderas, lo que viola nuestra premisa. Así, σ *no es* demostrable a partir de A. Pero esto es lo que σ afirma; por tanto σ es una oración verdadera.

Así, hemos demostrado (en menos de 29 páginas y media) que para cualquier conjunto de enunciados verdaderos de la teoría de números, y en particular cualquier conjunto de axiomas básicos, hay otros enunciados verdaderos que *no pueden* demostrarse a partir de esos axiomas. Esto establece, entre otras cosas, que nunca podremos probar todos los teoremas de las matemáticas *dentro de un sistema de axiomas determinado*. Cierto, este fue un descubrimiento importante para las matemáticas. Su significancia para la IA ha sido muy debatida, comenzando con especulaciones del propio Gödel. Retomamos el debate en el Capítulo 27.

Lema de elevación

Esto se llama **lema de elevación**, porque eleva un paso de prueba desde las cláusulas base hasta las generales de primer orden. Para demostrar su lema básico de elevación, Robinson tuvo que inventar la unificación y derivar las propiedades de la mayoría de los unificadores generales. En vez de repetir la prueba aquí, sólo se ilustrará el lema:

$$C_1 = \neg P(x,F(x,A)) \vee \neg Q(x,A) \vee R(x,B)$$
$$C_2 = \neg N(G(y),z) \vee P(H(y),z)$$
$$C'_1 = \neg P(H(B),F(H(B),A)) \vee \neg Q(H(B),A) \vee R(H(B),B)$$
$$C'_2 = \neg N(G(B),F(H(B),A)) \vee P(H(B),F(H(B),A))$$
$$C' = \neg N(G(B),F(H(B),A)) \vee \neg Q(H(B),A) \vee R(H(B),B)$$
$$C = \neg N(G(y),F(H(y),A)) \vee \neg Q(H(y),A) \vee R(H(y),B).$$

Vemos que en *efecto* C′ es una instancia fundamental de C. En general, para C'_1 y C'_2 tener algunos resolutivos, deben construirse aplicando primero a C_1 y C_2 el unificador más general de un par de literales complementarios en C_1 y C_2. Del lema de elevación, es fácil derivar una afirmación similar sobre cualquier secuencia de aplicaciones de la regla de resolución:

> Para cualquier cláusula C' en el cierre de resolución de S' hay una cláusula C en el cierre de resolución de S tal que C' es una instancia fundamental de C y la derivación de C tiene la misma longitud que la derivación de C'.

De este hecho, se deduce que si la cláusula vacía aparece en el cierre de resolución de S', también debe aparecer en el cierre de resolución de S. Esto se debe a que la cláusula vacía no puede ser una instancia fundamental de ninguna otra cláusula. En resumen: hemos demostrado que si S es insatisfactorio, entonces hay una derivación finita de la cláusula vacía usando la regla de resolución.

El lema de elevación proviene de probar desde las cláusulas fundamentales hasta las cláusulas de primer orden aumenta su poder. Este aumento surge del hecho de que la prueba de primer orden necesita instanciar variables sólo en la medida necesaria para la prueba, mientras que los métodos de cláusula fundamental requieren examinar una gran cantidad de instanciaciones arbitrarias.

9.5.5 Igualdad

Ninguno de los métodos de inferencia descritos hasta ahora permite manejar una afirmación de la forma $x = y$ sin algún trabajo adicional. Es posible adoptar tres enfoques distintos. El primero es axiomatizar la igualdad: escribir frases sobre la relación de igualdad en la BC. Necesitamos decir que la igualdad es reflexiva, simétrica y transitiva, y también que podemos sustituir iguales por iguales en cualquier predicado o función. Entonces se necesitan tres axiomas básicos, y luego uno para cada predicado y función:

$$\forall x \; x = x$$
$$\forall x,y \; x = y \Rightarrow y = x$$
$$\forall x,y,z \; x = y \wedge y = z \Rightarrow x = z$$
$$\forall x,y \; x = y \Rightarrow (P_1(x) \Leftrightarrow P_1(y))$$
$$\forall x,y \; x = y \Rightarrow (P_2(x) \Leftrightarrow P_2(y))$$
$$\vdots$$
$$\forall w,x,y,z \; w = y \wedge x = z \Rightarrow (F_1(w,x) = F_1(y,z))$$
$$\forall w,x,y,z \; w = y \wedge x = z \Rightarrow (F_2(w,x) = F_2(y,z))$$
$$\vdots$$

Dadas estas oraciones, un procedimiento de inferencia estándar como la resolución realizará tareas que requieren razonamiento de igualdad, como resolver ecuaciones matemáticas. Pero, estos axiomas generarán muchas conclusiones, la mayoría inútiles para una prueba. Así, el segundo enfoque es agregar reglas de inferencia, no axiomas. La regla más simple, la **demostración**, toma la cláusula unitaria $x = y$ y una cláusula α que tenga el término x, y produce una nueva cláusula sustituyendo y por x dentro de α. Funciona si el término en α se unifica con x; no necesita ser igual a x. Vea que la demostración es direccional; dado $x = y$, la x siempre se reemplaza con y, nunca al revés. Es decir, la demostración se puede usar para simplificar expresiones con demostradores como $z + 0 = z$ o $z^1 = z$. Otro ejemplo es

$$Father(Father(x)) = PaternalGrandfather(x)$$
$$Birthdate(Father(Father(Bella)),1926)$$

Podemos concluir mediante la demostración:

$$Birthdate(PaternalGrandfather(Bella),1926).$$

Más formalmente, tenemos:

- **Demostración**: Para cualquier término x, y y z donde z aparece en algún lugar en la literal m_i y donde $\text{UNIFY}(x,z) = \theta$, Demostración

$$\frac{x=y, \quad m_1 \vee \cdots \wedge m_n}{\text{SUB}(\text{SUBST}(\theta, x), \text{SUBST}(\theta,y), m_1 \vee \cdots \wedge m_n)}$$

  Donde SUBST es la sustitución usual de una lista de enlaces, y $\text{SUB}(x,y,m)$ significa reemplazar x con y en algún lugar dentro de m.

La regla puede extenderse para operar cláusulas no unitarias donde asoma un signo igual:

- **Demostración paramétrica**: Para cualquier término x,y y z donde z aparece en algún lugar dentro de la literal m_i, y donde $\text{UNIFY}(x,z) = \theta$, Demostración paramétrica

$$\frac{\ell_1 \vee \cdots \vee \ell_k \vee x = y, \quad m_1 \vee \cdots \vee m_n}{\text{SUB}(\text{SUBST}(\theta, x), \text{SUBST}(\theta,y), \text{SUBST}(\theta,\ell_1 \vee \cdots \vee \ell_k \vee \cdots m_1 \vee \cdots \vee m_n)}$$

Por ejemplo, a partir de

$$P(F(x,B),x) \vee Q(x) \text{ y } F(A,y) = y \vee R(y)$$

Tenemos que $\theta = \text{UNIFY}(F(A,y),F(x,B)) = \{x/A,y/B\}$, y podemos concluir mediante paramodulación la oración

$$P(B,A) \vee Q(A) \vee R(B).$$

La demostración paramétrica da un procedimiento de inferencia completo para lógica de primer orden con igualdad.

Un tercer enfoque es razonamiento de igualdad por completo en un algoritmo de unificación extendido. Aquí, los términos son unificables si son *demostrables* iguales en una sustitución, donde "demostrable" admite razonamiento sobre la igualdad. Es decir, los términos 1+2 y 2+1 normalmente no son unificables, pero un algoritmo de unificación que sabe que $x+y = y+x$ los unificaría con la sustitución vacía. Esta **unificación de ecuaciones** es con algoritmos eficientes para los axiomas particulares usados (conmutatividad, asociatividad, etc.), no por inferencia explícita con esos axiomas. Los demostradores de teoremas con esta técnica se relacionan con sistemas CLP de la Sección 9.4. Unificación de ecuaciones

9.5.6 Estrategias de resolución

Aplicar repetidas veces la regla de inferencia de resolución seguro encontrará una prueba siempre que esta exista. Aquí examinamos estrategias para encontrar pruebas de manera *eficiente*.

Preferencia de unidades

Preferencia de unidades: Esta estrategia prefiere resoluciones donde una de las oraciones es un solo literal (conocido como **cláusula unitaria**). La idea de la estrategia es que tratamos de generar una cláusula vacía, así que sería buena idea preferir inferencias que den cláusulas más cortas. Resolver una oración unitaria (como P) con cualquier otra oración (como $\neg P \vee \neg Q \vee R$) siempre da una cláusula (aquí, $\neg Q \vee R$), más corta que la primera. Cuando en 1964 se probó la estrategia de preferencia de unidades para la inferencia proposicional, llevó a su aceleración, al posibilitar la demostración de teoremas sin la preferencia. La **resolución de unidades** es una forma restringida de resolución en que cada paso de resolución involucra una cláusula unitaria. La resolución de unidades es incompleta en general para cláusulas de Horn. Las pruebas de resolución de unidades en éstas se asemejan a un encadenamiento hacia adelante.

El *demostrador de teoremas OTTER* (McCune, 1990) usa la forma de búsqueda *primero el mejor*. Su función heurística mide el "peso" de cada cláusula, donde se prefieren las más ligeras. La elección exacta de la heurística depende del usuario, pero en general, el peso de una cláusula se correlaciona con su tamaño o dificultad. Las cláusulas unitarias se tratan como ligeras; así, la búsqueda se ve como una generalización de la estrategia de preferencia de unidades.

Conjunto de soporte

Conjunto de soporte: Las preferencias a ciertas resoluciones primero son útiles, pero es más efectivo eliminar por completo algunas resoluciones potenciales. Por ejemplo, podemos insistir en que cada paso de resolución abarque al menos un elemento de un conjunto especial de cláusulas: el *conjunto* de *soporte*. El resolvente se agrega luego al conjunto de soporte. Si éste es pequeño en relación con toda la BC, el espacio de búsqueda se reducirá drásticamente.

Para garantizar la completitud de esta estrategia, se puede elegir el conjunto de soporte S de modo que el resto de oraciones sean conjuntamente satisfacibles. Así, se puede usar la consulta negada como conjunto de soporte, bajo el supuesto de que la BC original es consistente. (Después de todo, si no es consistente, entonces el hecho de que la consulta siga de ella es vacío.) La estrategia de conjunto de soporte tiene la ventaja adicional de generar árboles de prueba dirigidos por objetivos con frecuencia fáciles de entender para los humanos.

Resolución de entrada

Resolución de entrada: En esta estrategia, cada resolución combina una oración de entrada (de la BC o la consulta) con otra oración. La prueba en la Figura 9.10 (p. 301) usa sólo resoluciones de entrada y tiene la forma característica de una sola "estructura" con oraciones individuales combinándose en ella. Claro, el espacio de árboles de prueba de esta forma es más pequeño que el espacio de los grafos de prueba. En BC de Horn, Modus Ponens es una especie de estrategia de resolución de entrada, porque combina una implicación de la BC original con otras oraciones. No sorprende que la resolución de entrada sea completa para BC en forma de Horn, pero incompleta en el caso general. La estrategia de **resolución lineal** es una ligera generalización que permite que P y Q se resuelvan juntos, ya sea que P esté en la BC original o que P es un ancestro de Q en el árbol de prueba. La resolución lineal es completa.

Resolución lineal

Subsumisión

Subsumisión: El método de subsumisión elimina todas las oraciones subsumidas por (es decir, más específicas que) una oración existente en la BC. Así, si $P(x)$ está en la BC, entonces no tiene sentido agregar $P(A)$ y aún menos agregar $P(A) \vee Q(B)$. La subsumisión ayuda a mantener pequeña la BC y, por lo tanto, a mantener pequeño el espacio de búsqueda.

Aprendizaje: Mejoraremos un demostrador de teoremas aprendiendo de la experiencia. Dada una colección de teoremas ya demostrados, se entrena un sistema de aprendizaje automático para, dado un conjunto de premisas y un objetivo que probar, responder a la pregunta: ¿qué pasos de prueba son afines a los que triunfaron en el pasado? El sistema DEEPHOL (Bansal *et al.*, 2019) hace eso, usa redes neuronales profundas (vea el Capítulo 21) para construir modelos (*implementaciones*) de objetivos y premisas, y usarlos para hacer selecciones. El entrenamiento usa pruebas generadas por humanos o computadora como ejemplos, inicia con una colección de 10 000 pruebas.

Aprendizaje

Usos prácticos de los demostradores de teoremas de resolución

Hemos demostrado cómo la lógica de primer orden representa un escenario del mundo real simple con conceptos como ventas, armas y ciudadanía. Pero escenarios del mundo real complejos tienen incertidumbre y muchos desconocidos. La lógica ha triunfado en escenarios con conceptos formales y estrictamente definidos, como **síntesis** y **verificación** de hardware y software. La investigación en demostración de teoremas se da en campos de diseño de hardware, lenguajes de programación e ingeniería de software, no sólo en IA.

Síntesis
Verificación

En el caso del hardware, los axiomas describen las interacciones entre señales y elementos de circuito. (Ver ejemplo en Sección 8.4.2, p. 273). Los razonadores lógicos diseñados para la verificación han verificado CPU completas, incluidas sus propiedades de sincronización (Srivas y Bickford, 1990). El demostrador de teoremas AURA se ha usado para diseñar circuitos más compactos que cualquier diseño previo (Wojciechowski y Wojcik, 1983).

Con el software, el razonamiento sobre programas es similar al razonamiento sobre acciones (Capítulo 7): los axiomas describen las precondiciones y efectos de cada declaración. La síntesis formal de algoritmos fue uno de los primeros usos de los demostradores de teoremas, como describió Cordell Green (1969a), con base en ideas de Herbert Simon (1963). El punto es demostrar de modo constructivo un teorema que diga "existe un programa p que satisface cierta especificación". Aunque la **síntesis deductiva** totalmente automatizada aún no es factible para programación de propósito general, la síntesis deductiva a mano ha tenido éxito en el diseño de algoritmos novedosos y sofisticados. La síntesis de programas de propósito especial, como código de cómputo científico, es un área activa de investigación.

Técnicas similares ahora se aplican a la verificación de software por sistemas como el verificador de modelos SPIN (Holzmann, 1997). Por ejemplo, el programa de control de la nave espacial Remote Agent fue verificado antes y después del vuelo (Havelund *et al.*, 2000). El algoritmo de cifrado de clave pública RSA y el algoritmo de coincidencia de cadenas de Boyer-Moore también fue verificado así (Boyer y Moore, 1984).

Resumen

Se ha presentado un análisis de la inferencia lógica en lógica de primer orden y varios algoritmos para hacerlo.

- Un primer enfoque usa reglas de inferencia (**instanciación universal** e **instanciación existencial**) para **proposicionalizar** el problema de inferencia. Usualmente, este enfoque es lento, a menos que el dominio sea pequeño.
- Usar **unificación** para identificar sustituciones apropiadas para variables elimina el paso de instanciación en pruebas de primer orden, un proceso más eficiente.

- Una versión ampliada del **Modus Ponens** usa la unificación y crea una regla de inferencia natural y poderosa, el **Modus Ponens generalizado**. Los algoritmos de **encadenamiento hacia adelante (*fc*)** y **hacia atrás (*bc*)** la aplican a conjuntos de cláusulas definidas.
- El Modus Ponens generalizado es completo para cláusulas definidas, aunque el problema de implicación es **semidecidible**. Para BC de **Datalog** que consisten en cláusulas definidas libres de funciones, la implicación es decidible.
- El *fc* se usa en **bases de datos deductivas**, donde se combina con operaciones de base de datos relacionales. También se usa en **sistemas de producción**, que actualizan con eficiencia conjuntos de reglas grandes. El *fc* es completo para Datalog y se ejecuta en tiempo polinomial.
- El *bc* se usa en **sistemas de programación lógica**, que emplean tecnología de compiladores sofisticada para proporcionar inferencia veloz. El *bc* sufre de inferencias redundantes y bucles infinitos; estos pueden ser aliviados por **memorización**.
- **Prolog**, a diferencia de la lógica de primer orden, utiliza un mundo cerrado con la suposición de nombres únicos y la negación como fallo. Esto hace que Prolog sea un lenguaje de programación más práctico, pero lo aleja más de la lógica pura.
- La regla de inferencia de **resolución** generalizada proporciona un sistema de prueba completo para la lógica de primer orden, usando BC en forma normal conjuntiva.
- Existen varias estrategias para reducir el espacio de búsqueda de un sistema de resolución sin comprometer la completitud. Un problema fundamental es lidiar con la igualdad; mostramos cómo se usa la **demostración** y la **demostración paramétrica**.
- Los demostradores de teoremas basados en resolución eficiente se han usado para probar teoremas matemáticos interesantes y para verificar y sintetizar software y hardware.

Notas bibliográficas e históricas

Gottlob Frege, creó la lógica de primer orden completa en 1879, basó su sistema de inferencia en una colección de esquemas válidos más una regla de inferencia, el Modus Ponens. Whitehead y Russell (1910) expusieron las llamadas *reglas de paso* (el término real es de Herbrand (1930)) usadas para mover los cuantificadores al frente de las fórmulas. Las constantes y las funciones de Skolem fueron introducidas, con oportunidad, por Thoralf Skolem (1920). Skolem introdujo, con singularidad, el universo de Herbrand (Skolem, 1928).

El teorema de Herbrand (1930) tuvo un papel vital en el desarrollo del razonamiento automatizado. Herbrand también inventó la **unificación**. Gödel (1930) creó sobre las ideas de Skolem y Herbrand para mostrar que la lógica de primer orden tiene un procedimiento de prueba completo. Alan Turing (1936) y Alonzo Church (1936), con simultaneidad y con pruebas diferentes, mostraron que la validez en la lógica de primer orden no era decidible. El excelso libro de Enderton (1972) explica con rigurosidad y de manera asequible los resultados.

Abraham Robinson propuso construir un razonador automatizado a partir de la proposicionalización y el teorema de Herbrand, pero Paul Gilmore (1960) escribió el primer programa. Davis y Putnam (1960) introdujeron el método de proposicionalización de la Sección 9.1. A su vez, Prawitz (1960) desarrolló la idea clave de dejar que la búsqueda de inconsistencia proposicional guíe la búsqueda y generación de términos del universo de Herbrand cuando sea necesarios establecer inconsistencia proposicional. Esta idea llevó a John Alan Robinson (sin parentesco) a desarrollar la resolución (Robinson, 1965).

La resolución fue adoptada para sistemas de pregunta-respuesta por Cordell Green y Bertram Raphael (1968). La implementación temprana de IA puso esfuerzo en estructuras de datos para la recuperación eficiente de hechos; este trabajo se ve en textos de programación de IA (Charniak

et al., 1987; Norvig, 1992; Forbus y De Kleer, 1993). Después de 1970, el ***fc*** ya era alternativa de comprensión fácil a la resolución en la IA. Las aplicaciones de IA involucraban grandes cantidades de reglas, por ello era importante desarrollar tecnología eficiente de coincidencia de reglas, en especial para actualizaciones.

La tecnología para **sistemas de producción** se desarrolló para apoyar esas aplicaciones. El lenguaje del sistema de producción OPS-5 (Forgy, 1981; Brownston *et al*., 1985), que incorpora el eficiente proceso de coincidencia de Rete (Forgy, 1982), se usó para aplicaciones como el sistema experto R1 para configurar minicomputadoras (McDermott, 1982). Kraska *et al*. (2017) describen cómo las redes neuronales aprenden esquemas de indexación eficiente para conjuntos de datos específicos.

La arquitectura cognitiva SOAR (Laird *et al*., 1987; Laird, 2008) fue diseñada para manejar conjuntos de reglas grandes, hasta un millón de reglas (Doorenbos, 1994). Las aplicaciones de ejemplo de SOAR incluyen el control de aviones de combate simulados (Jones *et al*., 1998), gestión del espacio aéreo (Taylor *et al*., 2007), personajes de IA para videojuegos (Wintermute *et al.,* 2007) y herramientas de entrenamiento para soldados (Wray y Jones, 2005).

El campo de las **bases de datos deductivas** inició en un taller en Toulouse en 1977 entre expertos en inferencia lógica y bases de datos (Gallaire y Minker, 1978). Trabajos como el de Chandra y Harel (1980) y Ullman (1985) adoptaron **Datalog** como un lenguaje estándar para bases de datos deductivas. El desarrollo de la técnica de **conjuntos mágicos** para reescritura de reglas por Bancilhon *et al.* (1986) permitió que el *fc* aprovechara la ventaja de la orientación a objetivos del *bc*.

El Internet llevó a una mayor disponibilidad de bases de datos masivas en línea. Esto impulsó el interés en la integración de múltiples bases de datos en un espacio de datos consistente (Halevy, 2007). Kraska *et al.* (2017) mostraron aceleraciones de hasta 70 % al usar aprendizaje automático para crear **estructuras de índice aprendidas** para la búsqueda eficiente de datos.

El *bc* para la inferencia lógica surgió en el lenguaje PLANNER (Hewitt, 1969); mientras, en 1972, Alain Colmerauer había desarrollado e implementado **Prolog** con el fin de analizar el lenguaje natural; las cláusulas de Prolog en su origen eran reglas de gramática libre de contexto (Roussel, 1975; Colmerauer *et al*., 1973).

Parte de la base teórica para programación lógica lo desarrolló Robert Kowalski en el Imperial College London, al trabajar con Colmerauer; vea Kowalski (1988) y Colmerauer y Roussel (1993) para lograr una visión histórica general. Los compiladores eficientes de Prolog por lo regular se basan en el modelo de máquina abstracta Warren (WAM) de computación desarrollado por David H. D. Warren (1983). Van Roy (1990) mostró que, en términos de velocidad, los programas Prolog compiten con los programas en C.

Los métodos para evitar bucles innecesarios en programas lógicos recursivos fueron desarrollados de modo independiente por Smith *et al*. (1986) y Tamaki y Sato (1986). El último artículo también incluye memorización para programas lógicos, un método desarrollado como **programación lógica tabulada** por David S. Warren. Swift y Warren (1994) muestran cómo extender el WAM para manejar tabulaciones, lo que permite que los programas Datalog se ejecuten en orden de magnitud más rápido que los sistemas de bases de datos deductivas de *fc*.

El trabajo sobre programación lógica con restricciones lo inició Jaffar y Lassez (1987). Jaffar *et al*. (1992) desarrollaron el sistema CLP(R) para manejar restricciones de valores reales. Ahora hay productos comerciales que resuelven problemas de configuración y optimización a gran escala con programación de restricciones; entre los conocidos es ILOG (Junker, 2003). La programación de conjuntos de respuesta (Gelfond, 2008) extiende Prolog, permitiendo disyunción y negación.

Los textos sobre programación lógica y Prolog incluyen a Shoham (1994), Bratko (2009), Clocksin (2003) y Clocksin y Mellish (2003). Antes del año 2000, el *Journal of Logic Programming* era el diario oficial; ha sido reemplazado por *Theory and Practice of Logic Programming*. Las

International Conference on Logic Programming (ICLP) y el International Logic Programming Symposium (ILPS).

La investigación sobre **demostración de teoremas matemáticos** comenzó incluso antes de que se desarrollaran los primeros sistemas completos de primer orden. El Probador de Teoremas de Geometría de Herbert Gelernter (1959) usaba métodos de búsqueda heurística combinados con diagramas para depurar subobjetivos falsos y demostrar resultados intrincados en geometría euclidiana. Las reglas de **demostración** y **demostración parametrizada** para el razonamiento de igualdad fueron introducidas por Wos *et al*. (1967) y Wos y Robinson (1968), respectivamente. Estas reglas fueron desarrolladas independientemente en el contexto de sistemas de reescritura de términos (Knuth y Bendix, 1970). La incorporación del razonamiento de igualdad en el algoritmo de unificación se debe a Gordon Plotkin (1972). Jouannaud y Kirchner (1991) hacen una revisión de la unificación ecuacional desde una perspectiva de reescritura de términos. Baader y Snyder (2001) ofrecen una visión general de la unificación.

Se han propuesto varias estrategias de control para la resolución, comenzando con la estrategia de preferencia de unidad (Wos *et al*., 1964). La estrategia de conjunto de soporte fue propuesta por Wos *et al*. (1965) para proporcionar un grado de orientación a objetivos en la resolución. La resolución lineal se originó en Loveland (1970). Genesereth y Nilsson (1987, capítulo 5) analizaron una amplia variedad de estrategias de control. Alemi *et al.* (2017) muestran cómo el sistema DEEPMATH usa redes neuronales profundas para seleccionar los axiomas que tienen más probabilidades de conducir a una prueba cuando se entregan a un probador de teoremas tradicional. En cierto sentido, la red neuronal desempeña el papel de la intuición del matemático y el probador de teoremas desempeña el papel de la experiencia técnica del matemático. (Loos *et al*., 2017) muestran que este enfoque se puede extender para ayudar a guiar la búsqueda, permitiendo demostrar más teoremas.

Computational Logic (Boyer y Moore, 1979) es la referencia básica sobre el probador de teoremas de Boyer-Moore. Stickel (1992) describe el Probador de Teoremas de Tecnología Prolog (PTTP), que combina la compilación de Prolog y la eliminación de modelos. SETHEO (Letz *et al*., 1992) es otro probador de teoremas ampliamente utilizado y basado en este enfoque. LEANTAP (Beckert y Posegga, 1995) es un probador de teoremas eficiente implementado en sólo 25 líneas de Prolog. Weidenbach (2001) describe SPASS, uno de los probadores de teoremas más sólidos actuales. El probador de teoremas más exitoso en competiciones anuales recientes ha sido VAMPIRE (Riazanov y Voronkov, 2002). El sistema COQ (Bertot *et al*., 2004) y el solucionador ecuacional E (Schulz, 2004) también han demostrado ser herramientas valiosas para probar la corrección.

Los probadores de teoremas se han utilizado para sintetizar y verificar automáticamente software. Ejemplos incluyen el software de control para la cápsula Orion de la NASA (Lowry, 2008) y otras naves espaciales (Denney *et al*., 2006). El diseño del microprocesador de 32 bits FM9001 se demostró correcto mediante el sistema de demostración de teoremas NQTHM (Hunt y Brock, 1992).

La Conferencia sobre Deducción Automática (CADE, por sus siglas en inglés) lleva a cabo un concurso anual para probadores de teoremas automáticos. Sutcliffe (2016) describe la competencia de 2016; los sistemas con mayor puntuación incluyen VAMPIRE (Riazanov y Voronkov, 2002), PROVER9 (Sabri, 2015) y una versión actualizada de E (Schulz, 2013). Wiedijk (2003) compara la fortaleza de 15 demostradores matemáticos. TPTP (Miles de Problemas para Probadores de Teoremas) es una biblioteca de problemas de demostración de teoremas, útil para comparar el rendimiento de los sistemas (Sutcliffe y Suttner, 1998; Sutcliffe *et al*., 2006).

Los probadores de teoremas han descubierto resultados matemáticos novedosos que se escaparon a matemáticos humanos por décadas, como se detalla en el libro *Automated Reasoning and the Discovery of Missing Elegant Proofs* (Wos y Pieper, 2003). El programa SAM (Matemáticas

Semiautomatizadas) fue el primero en probar un lema en teoría de retículas (Guard *et al.*, 1969). El programa AURA también ha respondido preguntas abiertas en varias áreas de las matemáticas (Wos y Winker, 1983). El probador de teoremas de Boyer-Moore (Boyer y Moore, 1979) fue utilizado por Natarajan Shankar para construir una prueba formal del Teorema de Incompletitud de Gödel (Shankar, 1986). El sistema NUPRL probó el paradox de Girard (Howe, 1987) y el Lema de Higman (Murthy y Russell, 1990).

Algebra de Robbins

En 1933, Herbert Robbins propuso un conjunto simple de axiomas –**álgebra de Robbins**– que parecía definir el álgebra booleana, pero no se pudo encontrar una prueba (a pesar del trabajo serio de Alfred Tarski y otros) hasta que EQP (una versión de OTTER) calculó una prueba (McCune, 1997). Benzmüller y Paleo (2013) usaron un probador de teoremas de orden superior para verificar la prueba de Gödel de la existencia de "Dios". El teorema del empaquetado de esferas de Kepler fue demostrado por Thomas Hales (2005) con la ayuda de algunos cálculos computacionales complicados, pero la prueba no fue completamente aceptada hasta que se generó una prueba formal con la ayuda de los asistentes de prueba HOL Light e Isabelle (Hales *et al.*, 2017).

Muchos artículos tempranos en lógica matemática están recopilados en *Frege to Gödel: A Source Book in Mathematical Logic* (Van Heijenoort, 1967). Los libros de texto dirigidos a la deducción automatizada incluyen el clásico *Symbolic Logic and Mechanical Theorem Proving* (Chang y Lee, 1973), así como obras más recientes de Duffy (1991), Wos *et al.* (1992), Bibel (1993) y Kaufmann *et al.* (2000). La revista principal para la demostración de teoremas es *the Journal of Automated Reasoning*; las principales conferencias son la *annual Conference on Automated Deduction (CADE)* y *the International Joint Conference on Automated Reasoning (IJCAR)*. El *Handbook of Automated Reasoning* (Robinson y Voronkov, 2001) recopila artículos en el campo. *Mechanizing Proof* (MacKenzie, 2004) cubre la historia y la tecnología de la demostración de teoremas para el público en general.

CAPÍTULO 10

REPRESENTACIÓN DEL CONOCIMIENTO

En el que mostramos cómo representar diversos hechos sobre el mundo real de una forma que pueda usarse para razonar y resolver problemas.

Capítulos previos mostraron cómo un agente con una BC puede hacer inferencias que le posibiliten actuar de modo adecuado. En este capítulo nos preguntamos qué *contenido* poner en la BC de tal agente, cómo representar hechos del mundo. Usaremos la lógica de primer orden como lenguaje de representación, en capítulos siguientes se introducen diferentes formalismos de representación como redes jerárquicas de tareas para razonar planes (Capítulo 11), redes bayesianas para razonar con incertidumbre (Capítulo 13), modelos de Markov para razonar en el tiempo (Capítulo 17) y redes neuronales profundas para razonar imágenes, sonidos y otros datos (Capítulo 21). Sin importar qué representación se use, los hechos del mundo necesitan ser manejados y este capítulo sensibiliza en los problemas.

La Sección 10.1 presenta la idea de una ontología general que organiza todo en el mundo en una jerarquía de categorías. La Sección 10.2 cubre las categorías básicas de objetos, sustancias y medidas; la Sección 10.3 cubre eventos, y la Sección 10.4 analiza el conocimiento sobre creencias. Después consideraremos la tecnología para razonar con este contenido: la Sección 10.5 analiza sistemas de razonamiento diseñados para inferencia eficiente con categorías, y la Sección 10.6 discute el razonamiento con información predeterminada.

10.1 Ingeniería ontológica

En dominios "de juego", la elección de la representación no importa; muchas opciones funcionarán. Los dominios complejos de compra por Internet o conducción de un automóvil en tránsito requieren representaciones más generales y flexibles. Aquí mostraremos cómo crear estas representaciones con base en conceptos generales, como *Eventos, Tiempo, Objetos Físicos y Creencias*, que ocurren en dominios diferentes. Representar estos conceptos a veces se llama **ingeniería ontológica.**

Ingeniería ontológica

No podemos representar *todo* en el mundo, ni siquiera en un libro de 1 000 páginas, pero dejaremos marcadores del conocimiento nuevo para que cualquier dominio encaje. Por ejemplo, definiremos qué es un objeto físico, los detalles de diferentes tipos de objetos (robots, televisores, libros, etc.) pueden completarse luego. Esto es análogo a como los diseñadores de un framework de programación orientado a objetos (como el marco gráfico Java Swing) definen conceptos generales como *Ventana*, esperando que los usuarios los usen para definir conceptos específicos como *VentanaHoja de cálculo*. El marco general de conceptos se llama **ontología superior** debido a la convención de dibujar gráficos con los conceptos generales en la parte superior y los conceptos específicos debajo de ellos, como en la Figura 10.1.

Ontología superior

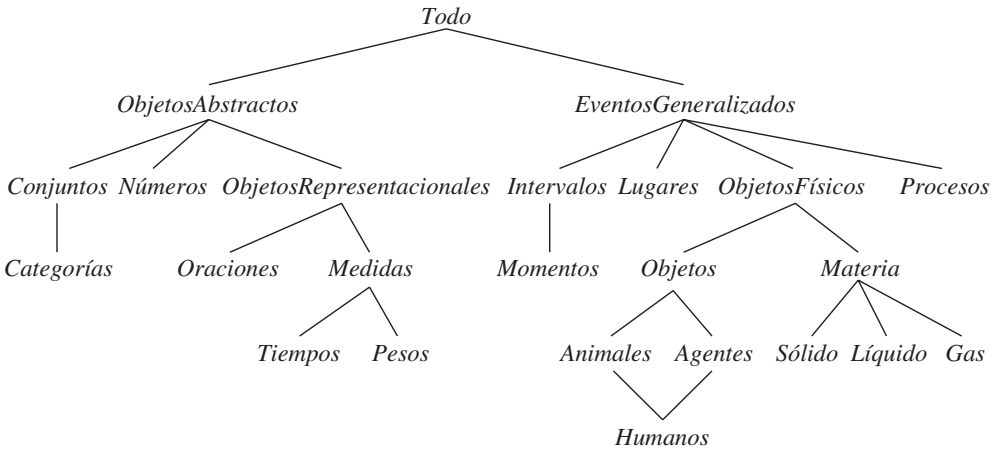


Figura 10.1 La ontología superior del mundo muestra los temas a ver en el capítulo. Cada enlace indica que el concepto inferior es una especialización del superior, las cuales no son necesariamente disyuntas: un ser humano es un animal y, a la vez, un agente. La Sección 10.3.2 muestra por qué los objetos físicos están en eventos generalizados.

Antes de pensar la ontología a fondo, debemos advertir que optamos por la lógica de primer orden para revisar el contenido y la organización del conocimiento, aunque unos aspectos del mundo real son difíciles de capturar en este sentido. La dificultad principal es que la mayoría de generalizaciones tienen excepciones o sólo son válidas hasta cierto punto. Así, aunque la regla "los tomates son rojos" es útil, unos tomates son verdes, amarillos o naranjas. En casi todas las reglas del capítulo encontraremos excepciones similares. La capacidad para manejar excepciones e incertidumbre es fundamental, pero es independiente de la tarea de comprender la ontología general; por ello, posponemos su discusión hasta la Sección 10.5, y el tema más general del razonamiento con incertidumbre hasta el Capítulo 12.

¿De qué sirve una ontología superior? Veamos la ontología de circuitos de la Sección 8.4.2. Hace suposiciones simplificadoras: el tiempo se omite, las señales son fijas y no se propagan, y la estructura del circuito es constante. Una ontología general consideraría las señales en momentos específicos e incluiría longitudes de cables y retrasos de difusión. Esto permitiría simular las propiedades temporales del circuito y, de hecho, esas simulaciones se ejecutan con frecuencia por diseñadores de circuitos.

También podríamos introducir clases más interesantes de compuertas, por ejemplo, al describir la tecnología (TTL, CMOS, etc.) y la especificación de entrada-salida. Si discutiéramos la confiabilidad o el diagnóstico, incluiríamos la posibilidad de que la estructura del circuito o las propiedades de las compuertas cambien espontáneamente. Para considerar las capacitancias parásitas, necesitaríamos representar dónde están los cables en la placa.

Si vemos el mundo del wumpus, se aplican criterios similares. Aunque representamos el tiempo, su estructura es simple: nada sucede excepto si el agente actúa, y los cambios son instantáneos. Una ontología más general, mejor adaptada al mundo real, permite cambios a la par extendidos en el tiempo. También usamos un predicado *Pozo* para decir qué cuadrados tienen pozos. Podríamos permitir diferentes tipos de pozos teniendo varios individuos pertenecientes a la clase de pozos, cada uno con diferentes propiedades. Igual, podríamos

permitir otros animales además de wumpus. Quizá no se precisaría la especie exacta a partir de los perceptos disponibles, por ello necesitaríamos construir una taxonomía biológica para ayudar al agente a predecir el comportamiento de los habitantes de la cueva a partir de pistas escasas.

Para cualquier ontología especializada, es posible realizar cambios como estos para avanzar a una mayor generalidad. Surge una pregunta: ¿estas ontologías convergen hacia una ontología de propósito general? Tras siglos de investigación filosófica y computacional, la respuesta es "Quizá". Aquí, presentamos una ontología de propósito general que sintetiza ideas de esos siglos. Dos características principales distinguen las ontologías de propósito general de las colecciones de ontologías especializadas:

- Una ontología de propósito general debería ser aplicable en casi cualquier dominio especializado (aún con axiomas específicos del dominio). Esto significa que ningún problema de representación puede ser evitado o ignorado.
- En todo dominio con exigencia suficiente, diferentes áreas de conocimiento deben *unificarse*, pues el razonamiento y la resolución de problemas podrían involucrar varias áreas a la vez. Así, un sistema de reparación de circuitos robóticos debe razonar en términos de conectividad eléctrica, disposición física y tiempo para el análisis de tiempo del circuito, y para estimar costos laborales. Las oraciones que describen el tiempo, por lo tanto, deben ser capaces de combinarse con las que describen la disposición espacial y funcionar para nanosegundos y minutos, como para ángstroms y metros.

Podemos aventurar que la empresa de ingeniería ontológica general tiene éxito limitado. Ninguna de las principales aplicaciones de IA (vistas en el Capítulo 1) usa ontología general; todas usan ingeniería de conocimiento y aprendizaje automático de propósito específico. Deferencias sociales y políticas dificultan que partes en competencia acuerden una ontología. Tom Gruber dice (2004), "Cada ontología es un tratado, un acuerdo social, entre personas con algún motivo común para compartir". Cuando las inquietudes en competencia superan la motivación para compartir, no puede haber una ontología común. Entre menos interesados haya, más fácil es crear una ontología y, así, es más difícil crear una ontología de propósito general que una de propósito limitado, como la Ontología Biomédica Abierta (Smith *et al.*, 2007). Las ontologías que existen se han creado por cuatro vías para:

1. Diseñar ontología y escribir axiomas como el sistema CYC, construido por ontólogos y lógicos capacitados (Lenat y Guha, 1990).
2. Importar categorías, atributos y valores de bases de datos existentes. DBPEDIA se construyó al importar hechos estructurados de Wikipedia (Bizer *et al.*, 2007).
3. Analizar documentos de texto y extraer su información. TEXTRUNNER se construyó tras leer un gran corpus de páginas web (Banko y Etzioni, 2008).
4. Alentar a aficionados no capacitados a ingresar conocimientos comunes. El sistema OPENMIND se construyó con voluntarios que proponían hechos en inglés (Singh *et al.*, 2002; Chklovski y Gil, 2005).

Por ejemplo, el Grafo de Conocimiento de Google usa contenido semiestructurado de Wikipedia y lo combina con contenido recopilado de la web bajo supervisión humana. Aloja más de 70 mil millones de hechos y da respuestas a cerca de un tercio de las búsquedas en Google (Dong *et al.*, 2014).

10.2 Categorías y objetos

Categoría

Organizar objetos en **categorías** es vital de la representación del conocimiento. Y la interacción con el mundo se da a nivel de objetos individuales, *parte del razonamiento ocurre a nivel de categorías*. Así, un comprador tendría el objetivo de comprar un balón, no un balón de *baloncesto particular*, como el BB_9. Las categorías sirven para predecir sobre objetos clasificados. Se infiere la presencia de objetos a partir de la entrada perceptual, se infiere la membresía de la categoría a partir de propiedades percibidas de los objetos, se usa la información de la categoría para predecir sobre objetos. Y, a partir de su cáscara verde, diámetro, forma ovoide, carne roja, semillas negras y lugar en la sección de frutas; se infiere que un objeto es una sandía; entonces, se infiere que esta sería útil para un plato de frutas.

Reificar

Hay dos opciones para representar categorías en lógica de primer orden: predicados y objetos. Podemos usar el predicado *BalonesDeBaloncesto*(*b*), o **reificar** (definir)[1] la categoría como un objeto, *BalonesDeBaloncesto*. Y podríamos decir *Miembro*(*b*,*BalonesDeBaloncesto*), que abreviaremos como $b \in BalonesDeBaloncesto$, para decir que *b* es miembro de la categoría balones de baloncesto. Decimos *Subconjunto*(*BalonesDeBaloncesto,Balones*), abreviado $BalonesDeBaloncesto \subset Balones$, para decir que balones de baloncesto es una **subcategoría** de *Balones*. Usaremos subcategoría, subclase y subconjunto sin distinción.

Subcategoría

Herencia

Las categorías organizan el conocimiento por **herencia**. Si decimos que las instancias de la categoría *Comida* son comestibles, y si afirmamos que *Fruta* es subclase de *Comida* y *Manzanas* es subclase de *Fruta*, inferiremos que cada manzana es comestible. Decimos que las manzanas individuales **heredan** la propiedad comestibilidad, de su membresía en la categoría *Comida*.

Jerarquía taxonómica

Las relaciones de subclase organizan categorías en **jerarquía taxonómica o taxonomía**, las cuales se han usado por siglos en campos técnicos. La taxonomía más grande del tipo organiza casi 10 millones de especies vivas y extintas, la mayoría escarabajos,[2] en una única jerarquía; la biblioteconomía desarrolló una taxonomía de todos los campos del conocimiento, codificada como el sistema de clasificación Dewey, y las autoridades fiscales y otras áreas del gobierno han desarrollado extensas taxonomías de ocupaciones y productos comerciales.

La lógica de primer orden facilita la declaración de hechos sobre categorías, ya sea al relacionar objetos con categorías o al cuantificar sus miembros. Vea ejemplos concretos:

- Un objeto es miembro de una categoría.
  $BB_9 \in BalonesDeBaloncesto$

- Una categoría es una subclase de otra categoría.
  $BalonesDeBaloncesto \subset Balones$

- Todos los miembros de una categoría tienen algunas propiedades.
  $(x \in BalonesDeBaloncesto) \Rightarrow Esférico(x)$

- Los miembros de una categoría pueden ser reconocidos por sus propiedades.
  $Naranja(x) \wedge Redondo(x) \wedge Diámetro(x) = 9.5'' \wedge x \in Balones \Rightarrow x \in BalonesDeBaloncesto$

- Una categoría en su conjunto tiene algunas propiedades.
  $Perros \in EspeciesDomesticadas$

[1] Convertir una proposición en un objeto se llama cosificación, del latín *res* (cosa). John McCarthy propuso el término "cosificación", pero no tuvo éxito.

[2] Cuando al biólogo J. B. S. Haldane se le preguntó qué se podía deducir sobre el Creador a partir del estudio de la naturaleza, él dijo: "Una afición excesiva por los escarabajos".

Note que como *Perros* es una categoría y es miembro de *EspeciesDomesticadas*, esta última debe ser una categoría de categorías. Claro, hay excepciones a muchas de estas reglas (los balones perforados no son esféricos); veremos estas excepciones después.

Aunque las relaciones de subclase y miembro son las más importantes para las categorías, queremos declarar relaciones entre categorías que no son subclases entre sí. Por ejemplo, si sólo decimos que *Estudiantes* y *EstudiantesGraduados* son subclases de *EstudiantesDePregrado*, entonces no hemos dicho que un estudiante de pregrado no pueda ser también un estudiante graduado. Decimos que dos o más categorías son **disyuntas** si no tienen miembros en común. También podemos decir que las clases de estudiante de pregrado y estudiante graduado forman una **descomposición exhaustiva** de estudiantes universitarios. Este tipo de descomposición de conjuntos disyuntas se conoce como una **partición**. Ejemplos de estos tres conceptos son:

Disyunto

Descomposición exhaustiva

Partición

Disyunción({*Animales*,*Vegetales*})
DescomposiciónExhaustiva({*Estadounidenses*,*Canadienses*,*Mexicanos*}, *Norteamericanos*)
Partición({*Animales*,*Plantas*,*Hongos*,*Protistas*,*Moneras*}, *SeresVivos*).

(Vea que *la DescomposiciónExhaustiva* de *Norteamericanos* no es una *Partición*, porque algunas personas tienen doble ciudadanía.) Los tres predicados se definen así:

$$Disyunción(s) \Leftrightarrow (\forall\, c_1,c_2\, c_1 \in s \wedge c_2 \in s \wedge c_2 \neq c_2 \Rightarrow Intersección(c_1,c_2) = \{\})$$
$$DescomposiciónExhaustiva(s,c) \Leftrightarrow (\forall\, i\; i \in c \Leftrightarrow \exists\, c_2\; c_2 \in s \wedge i \in c_2)$$
$$Partición(s,c) \Leftrightarrow Disyunción(s) \wedge DescomposiciónExhaustiva(s,c).$$

Las categorías también pueden *definirse* al proporcionar condiciones necesarias y suficientes para la membresía. Como, un soltero es un hombre adulto no casado:

$$x \in Solteros \Leftrightarrow NoCasado(x) \wedge x \in Adultos \wedge x \in Hombres\ .$$

Como discutimos en el cuadro lateral sobre tipos naturales en la página 320, las definiciones lógicas estrictas para las categorías en general sólo son posibles para términos formales artificiales, no para objetos ordinarios. Pero las definiciones no siempre son necesarias.

10.2.1 Composición física

La idea de que un objeto sea parte de otro es familiar. Así, la nariz es parte de la cabeza, Rumania es parte de Europa y este capítulo es parte de este libro. Usamos la relación general *ParteDe* para decir que una cosa es parte de otra. Los objetos se agrupan en jerarquías *ParteDe*, reminiscentes de la de *Subconjuntos*:

ParteDe(*Bucarest*,*Rumania*)
ParteDe(*Rumania*,*EuropadelEste*)
ParteDe(*Europa del Este*,*Europa*)
ParteDe(*Europa*,*Tierra*).

La relación *ParteDe* es transitiva y reflexiva; es decir,

$$ParteDe(x,y) \wedge ParteDe(y,z) \Rightarrow ParteDe(x,z)$$
$$ParteDe(x,x).$$

Objeto compuesto

Por lo tanto, podemos concluir *ParteDe*(*Bucarest*,*Tierra*). Las categorías de **objetos compuestos** se suelen caracterizar por relaciones estructurales entre las partes. Así, un bípedo es un objeto con exactamente dos piernas unidas a un cuerpo:

$$Bipedo(a) \Rightarrow \exists\, l_1,l_2,b\ Pierna(l_1) \wedge Pierna(l_2) \wedge Cuerpo(b) \wedge ParteDe(l_1,a) \wedge ParteDe(l_2,a) \wedge ParteDe(b,a) \wedge Adjunto(l_1,b) \wedge Adjunto(l_2,b) \wedge l_1 \neq l_2 \wedge [\forall l_3\ Pierna(l_3) \wedge ParteDe(l_3,a) \Rightarrow (l_3 = l_1 \vee l_3 = l_2)].$$

La notación para "exactamente dos" es un poco incómoda; estamos obligados a decir que hay dos piernas, que no son iguales, que si alguien propone una tercera pierna, debe ser igual a una de las otras dos. En la Sección 10.5.2, describimos un formalismo llamado lógica de descripción que facilita representar restricciones como "exactamente dos".

Podemos definir una relación *ParticiónDePartes* análoga a la relación *Partición* para categorías. (Vea el Ejercicio 10.DECM.) Un objeto está compuesto por las partes en su *ParticiónDePartes* y se puede ver que deriva unas propiedades de esas partes. Por ejemplo, la masa de un objeto compuesto es la suma de las masas de las partes. Vea que este no es el caso con las categorías, que no tienen masa, pero sus elementos podrían tenerla.

Es útil definir objetos compuestos con partes definidas, pero sin estructura particular. Así, podríamos querer decir "Las manzanas en esta bolsa pesan dos libras". La tentación sería atribuir este peso al *conjunto* de manzanas en la bolsa, pero esto sería un error porque el conjunto es un concepto matemático abstracto que tiene elementos, pero no tiene peso. En cambio, necesitamos un nuevo concepto, que llamaremos **racimo**. Por ejemplo, si las manzanas son $Manzana_1$, $Manzana_2$ y $Manzana_3$, entonces

Racimo

$$RacimoDe(\{Manzana_1, Manzana_2, Manzana_3\})$$

denota el objeto compuesto con las tres manzanas como partes (no elementos). Luego podemos usar el racimo como un objeto normal, pero sin estructura. Vea que $RacimoDe(\{x\}) = x$. Además, $RacimoDe(Manzanas)$ es el objeto compuesto que consiste en todas las manzanas, no debe confundirse con *Manzanas*, la categoría o conjunto de todas las manzanas.

Definimos *RacimoDe* como la relación *ParteDe*. Cada elemento de s es parte de $RacimoDe(s)$*:*

$$\forall\, x\ x \in s \Rightarrow ParteDe(x, RacimoDe(s)).$$

Además, $RacimoDe(s)$ *es el objeto más pequeño que satisface esta condición*. Ed decir, $RacimoDe(s)$ debe ser parte de cualquier objeto que tenga todos los elementos de s como partes:

$$\forall\, y\ [\forall\, x\ x \in s \Rightarrow ParteDe(x,y)] \Rightarrow ParteDe(RacimoDe(s),y).$$

Estos axiomas son un ejemplo de una técnica general llamada **minimización lógica**, que significa definir un objeto como el más pequeño que satisface ciertas condiciones.

Minimización lógica

10.2.2 Mediciones

En teorías científicas y de sentido común del mundo, los objetos tienen altura, masa, costo, etc. Los valores asignados a estas propiedades se llaman **medidas**. Las medidas cuantitativas ordinarias son fáciles de representar. Imaginamos que el universo incluye "objetos de medida" abstractos, como la longitud de este segmento de línea: ⊢——————⊣. Podemos llamar a esta longitud 1.5 pulgadas o 3.81 centímetros. Así, la misma longitud tiene diferentes nombres en el idioma. Representamos la longitud con una **función de unidades** que toma un número como argumento. (Ver alternativa en el Ejercicio 10.ALTM.)

Medida

Función de unidades

Tipos naturales

Unas categorías tienen definiciones estrictas: un objeto es un triángulo si y sólo si es un polígono de tres lados. Además, la mayoría de categorías en el mundo real no tienen una definición clara, se llaman categorías de **tipos naturales**. Así, los tomates suelen ser de un rojo apagado, casi esféricos; con una indentación en la parte superior donde estaba el tallo, de dos a cuatro pulgadas de diámetro; con piel delgada pero resistente, y con pulpa, semillas y jugo en su interior. Pero, hay variación: unos son amarillos o naranjas; los tomates no maduros son verdes, unos son más pequeños o grandes de lo normal, y los cherry son uniformemente pequeños. En lugar de una definición completa de tomates, tenemos un conjunto de características para identificar objetos que son tomates típicos y que, de hecho, no podrían identificar otros objetos. (¿Podría haber un tomate peludo como un durazno?)

Esto plantea un problema para un agente lógico. Este no puede estar seguro de que un objeto que ha percibido sea un tomate, y aunque estuviera, no podría estar seguro de qué propiedades de los tomates típicos tiene. Este problema es una consecuencia inevitable de operar en entornos parcialmente observables.

Un enfoque útil es separar lo que es cierto para las instancias de una categoría de lo que es cierto sólo para instancias típicas. Así, además de la categoría *Tomates*, también tendremos la categoría *Típico*(*Tomates*). Aquí, la función *Típico* asigna una categoría a la subclase que contiene sólo instancias típicas:

$$\mathit{Típico}(c) \subseteq c.$$

Casi todo el conocimiento sobre tipos naturales será sobre sus instancias típicas:

$$x \in \mathit{Típico}(\mathit{Tomates}) \Rightarrow \mathit{Rojo}(x) \wedge \mathit{Redondo}(x).$$

Podemos escribir hechos útiles sobre categorías sin definiciones exactas. La dificultad de las definiciones exactas para la mayoría de las categorías naturales fue explicada en detalle por Wittgenstein (1953). Él usó el ejemplo de los *juegos* para mostrar que los miembros de una categoría comparten "semejanzas familiares" en lugar de características necesarias y suficientes: ¿qué definición estricta abarca el ajedrez, tag, el solitario y el dodgeball?

La utilidad de noción de definición estricta fue cuestionada por Quine (1953). Él señaló que aun la definición de "soltero" como hombre adulto no casado es sospechosa; uno podría, por ejemplo, cuestionar la afirmación "el Papa es soltero". Si bien no es estrictamente *falsa*, este uso es *desafortunado* porque induce *inferencias* no deseadas por el oyente. La tensión podría resolverse quizás distinguiendo entre definiciones lógicas adecuadas para la representación del conocimiento interno y los criterios más matizados para el uso lingüístico feliz. Esto podría lograrse "filtrando" las afirmaciones derivadas de lo primero. También es posible que los fallos en el uso lingüístico sirvan como retroalimentación para modificar las definiciones internas, de modo que el filtrado se vuelva innecesario.

Si el segmento de línea se llama L_1, podemos escribir

$$Longitud(L_1) = Pulgadas(1.5) = Centímetros(3.81)$$

La conversión entre unidades se hace igualando múltiplos de una unidad a otra:

$$Centímetros(2.54 \times d) = Pulgadas(d)$$

Axiomas similares se pueden escribir para libras y kilogramos, segundos y días, y dólares y centavos. Las medidas se pueden usar para describir objetos como sigue:

$$Diámetro(BalonDeBaloncesto_{12}) = Pulgadas(9.5)$$
$$PrecioLista(BalonDeBaloncesto_{12}) = \$(19)$$
$$Peso(RacimoDe(Manzana_1, Manzana_2, Manzana_3\})) = Libras(2)$$
$$d \in Días \Rightarrow Duración(d) = Horas(24).$$

Vea que \$(1) *no es* un billete de un dólar, es un precio. Se pueden tener dos billetes de un dólar, pero sólo hay un objeto llamado \$(1). Note también que, aunque *Pulgadas*(0) y *Centímetros*(0) refieren a la misma longitud cero, no son idénticos a otras medidas cero, como *Segundos*(0).

Las medidas simples y cuantitativas son fáciles de representar, otras presentan problemas, porque no tienen una escala de valores acordada. Los ejercicios tienen dificultad, los postres delicia y los poemas belleza, pero a estas cualidades no se les puede asignar números. Uno podría, en un lapso de pura contabilidad, descartar esas propiedades como inútiles con propósito de razonamiento lógico; o, aún peor, imponer una escala numérica a la belleza. Esto sería un grave error, porque es innecesario. El aspecto más importante de las medidas no son los valores numéricos particulares, sino que las medidas pueden ser *ordenadas*.

Aunque las medidas no son números, podemos llegar a compararlas, usando un símbolo de orden como >. Por ejemplo, podríamos creer que los ejercicios de Norvig son más difíciles que los de Russell, y que uno puntúa menos en ejercicios más difíciles:

$$e_1 \in Ejercicios \wedge e_2 \in Ejercicios \wedge Escribió(Norvig, e_1) \wedge Escribió(Russell, e_2) \Rightarrow Dificultad(e_1) > Dificultad(e_2).$$
$$e_1 \in Ejercicios \wedge e_2 \in Ejercicios \wedge Dificultad(e_1) > Dificultad(e_2) \Rightarrow PuntuaciónEsperada(e_1) < PuntuaciónEsperada(e_2).$$

Esto es suficiente para permitir decidir qué ejercicios hacer, aunque nunca se usaron valores numéricos para la dificultad. (Pero, uno debe descubrir quién escribió cuáles ejercicios). Estos tipos de relaciones monótonas entre medidas forman la base para el campo de la **física cualitativa**, un subcampo de IA que investiga cómo razonar sobre sistemas físicos sin sumergirse en ecuaciones detalladas y simulaciones numéricas. La física cualitativa se discute en la sección Notas bibliográficas e históricas.

10.2.3 Objetos: Cosas y materia

El mundo real se puede ver como consistente en objetos primitivos (como, partículas atómicas) y objetos compuestos construidos a partir de aquellos. Al razonar en el nivel de objetos grandes, como manzanas y automóviles, podemos superar la complejidad involucrada al tratar con vastos números de objetos primitivos individuales. Pero, hay una parte significativa de la realidad que parece desafiar cualquier **individuación** obvia, división en objetos distintos, que llamamos con el nombre genérico **materia**. Por ejemplo, suponga que tengo mantequilla y un oso hormiguero frente a mí. Puedo decir hay un oso hormiguero, pero no hay un número obvio de "objetos mantequilla", porque cualquier parte de un objeto mantequilla también es un objeto mantequilla, al menos hasta

Individuación
Materia

que lleguemos a partes muy pequeñas. Esta es la distinción principal entre *cosas* y *materia*. Si cortamos un oso hormiguero por la mitad, no obtenemos dos cerdos hormigueros (lamentablemente).

La lengua inglesa distingue entre *cosas* y *materia* (*stuff and things*). Se dice "un oso hormiguero", no "una mantequilla", con la excepción de creídos restaurantes de California, Los lingüistas distinguen entre **sustantivos contables** (osos hormigueros, agujeros y teoremas) y **sustantivos de masa** (mantequilla, agua y energía). Varias ontologías competidoras afirman manejar esta distinción, aquí describimos sólo una; las otras se ven en las Notas bibliográficas e históricas.

Sustantivos contables

Sustantivos de masa

Al representar la materia, iniciamos con lo obvio: necesitamos como objetos en nuestra ontología al menos "bultos" brutos de *materia* con los que interactuar. Así, podríamos reconocer un trozo de mantequilla como el restante de la cena; podríamos recogerlo, pesarlo, venderlo, o lo que sea. En este sentido, es un objeto igual que un oso hormiguero. Llamemos *Mantequilla*[3]. También definimos la categoría *Mantequilla*; de modo informal, sus elementos serán las cosas de las que uno podría decir "Es mantequilla", incluida *Mantequilla*[3]. Con reservas sobre partes muy pequeñas que ahora omitiremos, cualquier parte de un objeto de mantequilla es un objeto de mantequilla:

$$b \in Mantequilla \land ParteDe(p,b) \Rightarrow p \in Mantequilla.$$

Ahora podemos decir que la mantequilla se derrite a 30 grados centígrados:

$$b \in Mantequilla \Rightarrow PuntoDeFusión(b, Centígrados(30)).$$

Decimos que la mantequilla es amarilla, menos densa que el agua, suave a temperatura ambiente y tiene alto contenido de grasa, etc. Por otro lado, la mantequilla no tiene un tamaño, forma o peso particulares. Podemos definir categorías más especializadas como *MantequillaSinSal*, que también es un tipo de *materia*. Tome en cuenta que la categoría *LibraDeMantequilla*, que incluye como miembros todos los objetos de mantequilla que pesan una libra, no es un tipo de *materia*. Si cortamos una libra de mantequilla por la mitad, no obtenemos, qué pesar, dos libras de mantequilla.

Intrínseco

Extrínseco

Lo que sucede en realidad es que algunas propiedades son **intrínsecas**: pertenecen a la propia sustancia del objeto, en lugar de al objeto en su totalidad. Cuando cortas una parte de *materia* por la mitad, ambas partes retienen las propiedades intrínsecas (densidad, punto de ebullición, sabor, color, propiedad, etc.). Por otro lado, sus propiedades **extrínsecas** (peso, longitud, forma, etc.) no se conservan bajo subdivisión. Una categoría de objetos que incluye en su definición sólo propiedades *intrínsecas* es una sustancia o sustantivo de masa; una clase que incluye *cualquier* propiedad extrínseca en su definición es un sustantivo contable. *Materia* y *Cosa* son las categorías de *materia* y *objeto* más generales, respectivamente.

10.3 Eventos

En la Sección 7.7.1 revisamos acciones: cosas que suceden, como $Disparar_t$, y *flujos*: aspectos del mundo que cambian, como $TenerFlecha_t$. Ambos fueron representados como proposiciones, y usamos axiomas de estado sucesor para decir que un fluente será verdadero en el tiempo $t + 1$ o si la acción en el tiempo t causó que fuera verdadero, o si ya era verdadero en el tiempo t y la acción no lo hizo falso. Eso fue para un mundo en el que las acciones son discretas, instantáneas, que suceden una a la vez, y no hay variación en cómo se realizan (es decir, sólo hay un tipo de acción de *Disparar*, no hay distinción entre disparar rápido, lento, nervioso, etc.).

Pero a medida que pasamos de dominios simplistas al mundo real, hay una gama mucho más rica de acciones o eventos[3] para tratar. Considere una acción continua, como llenar una bañera.

[3] Los términos "evento" y "acción" se usan sin distinción; ambos significan "algo que puede suceder".

Un axioma de estado sucesor diría que la bañera está vacía antes de la acción y llena tras la acción, pero no puede decir qué sucede *durante* la acción. Tampoco puede describir con facilidad dos acciones que ocurren al mismo tiempo, como cepillarse los dientes mientras la bañera se llena. Para manejar tales casos, introducimos un enfoque conocido como **cálculo de eventos**.

Cálculo de eventos

Estos objetos de cálculo son eventos, flujos y puntos temporales. *En(Shankar,Berkeley)* es un flujo: un objeto que se refiere al hecho de que Shankar está en Berkeley. El evento E_1 de Shankar volando desde San Francisco a Washington, D.C., se describe como

$$E_1 \in Vuelos \wedge Vuela(E_1,Shankar) \wedge Origen(E_1,SF) \wedge Destino(E_1,DC).$$

donde *Vuelos* es la categoría de todos los eventos de vuelo. Al reificar eventos, hacemos posible agregar cualquier cantidad de información arbitraria sobre ellos. Por ejemplo, podemos decir que el vuelo de Shankar fue accidentado con $Tardado(E_1)$. En una ontología donde los eventos son predicados *n-arios*, no habría forma de agregar información adicional como esta; pasar a un predicado $n + 1$-ario no es una solución escalable.

Para afirmar que un flujo es realmente verdadero desde algún punto en el tiempo t_1 al tiempo t_2, usamos el predicado T, como en $T(En(Shankar,Berkeley),t_1,t_2)$. Asimismo, usamos $Evento(E_1,t_1,t_2)$ para decir que el evento E_1 realmente ocurrió, comenzando en el tiempo t_1 y terminando en el tiempo t_2. El conjunto completo de predicados para una versión del cálculo de eventos es:[4]

$T(f,t_1,t_2)$	Flujo f es verdadero para todos los tiempos entre t_1 y t_2
$Evento(e,t_1,t_2)$	Evento e comienza en el tiempo t_1 y termina en t_2
$Inicio(e,f,t)$	Evento e hace que el flujo f se vuelva verdadero en el tiempo t
$Termina(e,f,t)$	Evento e hace que el flujo f deje de ser verdadero en el tiempo t
$Inicia(f,t_1,t_2)$	Flujo f se vuelve verdadero en algún momento entre t_1 y t_2
$Termina(f,t_1,t_2)$	Flujo f deja de ser verdadero en algún momento entre t_1 y t_2
$t_1 < t_2$	El punto de tiempo t_1 ocurre antes del tiempo t_2

Podemos describir los efectos de un evento de vuelo:

$$E = Vuela(a,aquí,allá) \wedge Evento(E,t_1,t_2) \Rightarrow Termina(E,En(a,aquí),t_1) \wedge Incia(E,En(a,allá),t_2)$$

Suponemos un evento distinguido, *Inicio*, que describe el estado inicial al decir qué flujos son verdaderos (usando *Inicia*) o falsos (usando *Termina*) en el tiempo de inicio. Podemos describir qué flujos son verdaderos en qué puntos en el tiempo con un par de axiomas para T y $\neg T$ que siguen el mismo formato general que los axiomas de estado sucesor: Suponga que un evento ocurre entre el tiempo t_1 y t_3, y en t_2 en algún lugar de ese intervalo de tiempo el evento cambia el valor del flujo f, al inicio (haciéndolo verdadero) o al termino (haciéndolo falso). Así, en el tiempo t_4 en el futuro, si ningún otro evento interviniente ha cambiado el flujo (terminado o iniciado), entonces el flujo habrá mantenido su valor. Formalmente, los axiomas son:

$$Ocurre(e,t_1,t_3) \wedge Inicia(e,f,t_2) \wedge \neg Terminado(f,t_2,t_4) \wedge t_1 \leq t_2 \leq t_3 \leq t_4 \Rightarrow T(f,t_2,t_4)$$

$$Ocurre(e,t_1,t_3)) \wedge Termina(e,f,t_2) \wedge \neg Iniciado(f,t_2,t_4)) \wedge t_1 \leq t_2 \leq t_3 \leq t_4 \Rightarrow \neg T(f, t_2,t_4)$$

[4] Nuestra versión está basada en Shanahan (1999), pero con algunas modificaciones.

donde *Terminado* e *Iniciado* se definen de la siguiente como:

$$Terminado(f,t_1,t_5) \Leftrightarrow$$
$$\exists\, e,t_2,t_3,t_4\ Ocurre(e,t_2,t_4) \wedge Termina(e,f,t_3) \wedge t1 \leq t_2 \leq t_3 \leq t_4 \leq t_5$$
$$Iniciado(f,t_1,t_5) \Leftrightarrow$$
$$\exists\, e,t_2,t_3,t_4\ Ocurre(e,t_2,t_4) \wedge Inicia(e,f,t_3) \wedge t_1 \leq t_2 \leq t_3 \leq t_4 \leq t_5$$

Es posible extender el cálculo de eventos para representar eventos simultáneos (dos personas siendo necesarias para montar un columpio), eventos exógenos (el viento moviendo un objeto), eventos continuos (la subida de la marea), eventos no deterministas (lanzar una moneda y que salga cara o cruz), y otras complicaciones.

10.3.1 Tiempo

El cálculo de eventos nos abre la posibilidad de hablar sobre puntos e intervalos de tiempo. Consideraremos dos tipos de intervalos de tiempo: momentos e intervalos extendidos. La distinción es que sólo los momentos tienen duración cero.

$$Partición(\{Momentos,IntervalosExtendidos\},Intervalos)$$
$$i \in Momento \Leftrightarrow Duración(i) = Segundos(0).$$

A continuación, inventamos una escala de tiempo y asociamos puntos en esa escala con momentos, dándonos tiempos absolutos. La escala de tiempo es arbitraria; la mediremos en segundos y diremos que el momento de medianoche (GMT) del 1 de enero de 1900 tiene tiempo 0. Las funciones *Inicio* y *Fin* seleccionan los momentos más tempranos y tardíos en un intervalo, y la función *Time* entrega el punto en la escala de tiempo para un momento. La función *Duración* da la diferencia entre el tiempo final y el tiempo inicial.

$$Intervalo(i) \Rightarrow Duración(i) = (Tiempo(Fin(i)) - Tiempo(Inicio(i))).$$
$$Tiempo(Inicio(1900dc)) = Segundos(0).$$
$$Tiempo(Inicio(2001dc)) = Segundos(3187324800).$$
$$Tiempo(Fin(2001dc)) = Segundos(3218860800).$$
$$Duración(2001dc) = Segundos(31536000).$$

Para hacer estos números más fáciles de leer, introducimos una función *Fecha*, que toma seis argumentos (horas, minutos, segundos, día, mes y año) y devuelve un punto en el tiempo:

$$Tiempo(Inicio(2001dc)) = Fecha(0,0,0,1,Ene,2001)$$
$$Fecha(0,20,21,24,1,1995) = Segundos(3000000000).$$

Dos intervalos *SeEncuentran* si el tiempo final del primero es igual al tiempo inicial del segundo. El conjunto completo de relaciones de intervalos (Allen, 1983) se ve aquí y en la Figura 10.2:

$$SeEncuentran(i,j) \Leftrightarrow Fin(i) = Inicio(j)$$
$$Antes(i,j) \Leftrightarrow Fin(i) < Inicio(j)$$
$$Después(j,i) \Leftrightarrow Antes(i,j)$$
$$Durante(i,j) \Leftrightarrow Inicio(j) < Inicio(i) < Fin(i) < Fin(j)$$
$$Solapa(i,j) \Leftrightarrow Inicio(i) < Inicio(j) < Fin(i) < Fin(j)$$
$$Comienza(i,j) \Leftrightarrow Inicio(i) = Inicio(j)$$
$$Termina(i,j) \Leftrightarrow Fin(i) = Fin(j)$$
$$Igual(i,j) \Leftrightarrow Inicio(i) = Inicio(j) \wedge Fin(i) = Fin(j)$$

Estas relaciones tienen significado intuitivo, excepto *Solapa*: que pensamos como simétrico (si *i* se solapa con *j*, *j* se solapa con *i*), pero en esta definición, *Solapa*(*i*,*j*) sólo es verdadero si *i* comienza

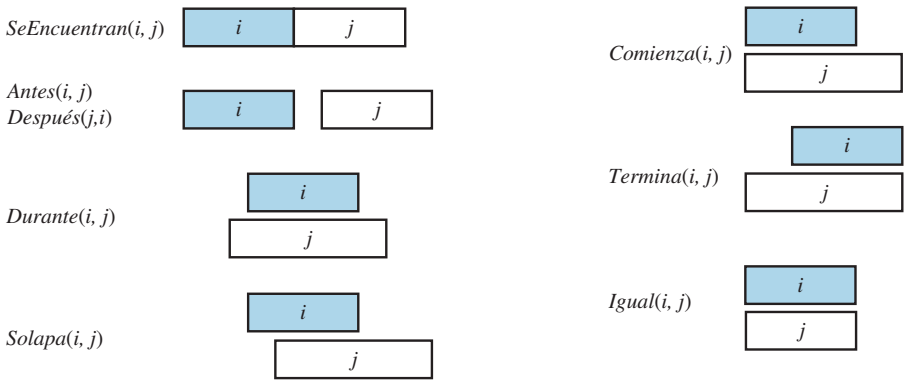


Figura 10.2 Predicados sobre intervalos de tiempo.

antes de *j*. Se ha demostrado que esta definición es más útil para escribir axiomas. Para decir que el reinado de Isabel II fue inmediato al de Jorge VI, y el reinado de Elvis se solapa con la década de 1950, podemos escribir que:

SeEncuentran(*ReinadoDe*(*GeorgeVI*),*ReinadoDe*(*IsabelII*)).
Solapamiento(*añoscincuenta*,*ReinadoDe*(*Elvis*)).
Comienzo(*años cincuenta*) = *Comienzo* (*AD*1950).
Fin(*añoscincuenta*) = *Fin* (*AD*1959).

10.3.2 Flujos y objetos

Los objetos físicos se pueden ver como eventos generalizados, en el sentido de que uno de ellos es un trozo de espacio-tiempo. Así, EUA se considera un evento iniciado en 1776 como una unión de 13 estados, que hoy persiste como unión de 50. Podemos describir las propiedades cambiantes de EUA usando flujos de estado, como *Población*(*EUA*). Una propiedad de EUA que cambia cada cuatro u ocho años, salvo infortunios, es su presidente. Se podría proponer que *Presidente*(*EUA*) es un término lógico que denota un objeto diferente en diferentes momentos.

Por desgracia, esto no es posible, porque un término denota un objeto exacto en una estructura de modelo dada. (El término *Presidente*(*EUA*,*t*) denota diferentes objetos, dependiendo del valor de *t*, pero nuestra ontología mantiene los índices de tiempo separados de los flujos). La única posibilidad es que *Presidente*(*EUA*) denote un único objeto que consiste en diferentes personas en diferentes momentos: el objeto George Washington de 1789 a 1797, John Adams de 1797 a 1801, etc., como en la Figura 10.3. Para decir que George Washington fue presidente durante todo 1790, podemos escribir

T(*Igual*(*Presidente*(EUA),*GeorgeWashington*),*Inicio*(1790dc),*Fin*(1790dc)).

Usamos el símbolo de función *Igual* en lugar del predicado lógico estándar =, porque no podemos tener un predicado como argumento para *T*, y porque la interpretación *no es* que *GeorgeWashington* y *Presidente*(EUA) sean lógicamente idénticos en 1790; la identidad lógica no es algo que pueda cambiar con el tiempo. La identidad es entre los subeventos de los objetos *Presidente*(EUA) y *GeorgeWashington* definidos por el periodo 1790.

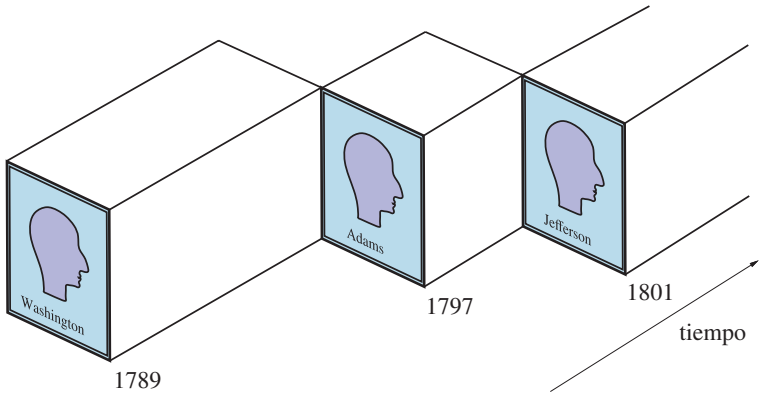


Figura 10.3 Vista esquemática del objeto *Presidente*(EUA) durante los primeros años.

10.4 Objetos mentales y lógica modal

Los agentes que ya construimos tienen creencias y pueden deducir nuevas; pero ninguno de ellos tiene conocimiento *sobre* las creencias o *sobre* la deducción. El conocimiento sobre el mismo conocimiento y los procesos de razonamiento es útil para controlar la inferencia. Suponga que Alicia pregunta "¿cuál es la raíz cuadrada de 1764?" y Bob responde "no lo sé". Si Alicia insiste "piénsale", Bob debería entender que con más reflexión puede responder esa pregunta. Por otro lado, si la pregunta fuera "¿está el presidente sentado ahora?", Bob debería saber que pensar no ayudará. El conocimiento sobre el conocimiento de otros agentes también es importante; Bob debería darse cuenta de que el presidente sí lo sabe.

Lo que necesitamos es un modelo de los objetos mentales que están en la mente de alguien (o en la base de conocimiento (BC) de algo) y de los procesos mentales que manipulan esos objetos mentales. El modelo no tiene que ser detallado. No tenemos que ser capaces de predecir cuántos milisegundos tomará para que un agente específico haga una deducción. Estaremos felices sólo de poder concluir si la persona en cuestión está sentada o no.

Actitud proposicional

Comenzamos con las **actitudes proposicionales** que un agente puede tener hacia los objetos mentales: actitudes como *Cree*, *Sabe*, *Quiere* e *Informa*. La dificultad es que estas actitudes no se comportan como predicados "normales". Por ejemplo, suponga que intentamos afirmar que Lois sabe que Superman puede volar:

$$Sabe(Lois, PuedeVolar(Superman)).$$

Un tema menor es que por lo regular pensamos en $PuedeVolar(Superman)$ como una oración, pero aquí aparece como un término. Esto puede ser solucionado cosificando $PuedeVolar(Superman)$, convirtiéndolo en un flujo. Un problema más serio es que, si es cierto que Superman es Clark Kent, entonces debemos concluir que Lois sabe que Clark puede volar, lo que es incorrecto porque (en las versiones de la historia) Lois no sabe que Clark es Superman.

$$(Superman = Clark) \wedge Sabe(Lois, PuedeVolar(Superman)) \models Sabe(Lois, PuedeVolar(Clark))$$

Esto resulta del hecho de que el razonamiento de igualdad está incorporado en la lógica. Eso suele ser bueno; si nuestro agente sabe que $2 + 2 = 4$ y $4 < 5$, entonces queremos que éste sepa que $2 + 2 < 5$. Esta propiedad se llama **transparencia referencial**: no importa qué término use la lógica para referirse a un objeto, lo que toca es el objeto que el término nombra. Para actitudes proposicionales como *creer* y *saber*, nos gustaría tener opacidad referencial: los términos usados *sí* importan, porque no todos los agentes saben qué términos *son* co-referenciales.

Transparencia referencial

Podríamos solucionar esto con aún más cosificación: podríamos tener un objeto para representar a Clark/Superman, otro para representar a la persona que Lois conoce como Clark, y otro más para la persona que Lois conoce como Superman. Pero, esta proliferación de objetos significa que las oraciones que queremos escribir rápido se vuelven verbosas y torpes.

La **lógica modal** está diseñada para abordar este problema. La lógica regular se preocupa por una sola modalidad, la de la verdad, lo que nos permite expresar "*P* es verdadero" o "*P* es falso". La lógica modal incluye **operadores modales** especiales que toman oraciones (en lugar de términos) como argumentos. Por ejemplo, "*A* sabe *P*" se representa con la notación $\mathbf{K}_A P$, donde **K** es el operador modal para el conocimiento. Toma dos argumentos, un agente (escrito como subíndice) y una oración. La sintaxis de la lógica modal es la misma que la de la lógica de primer orden, excepto que las oraciones también pueden formarse con operadores modales.

Lógica modal

Operador modal

La semántica de la lógica modal es más complicada. En la lógica de primer orden, un **modelo** contiene un conjunto de objetos y una interpretación que asigna cada nombre al objeto, relación o función apropiados. En la lógica modal queremos considerar la posibilidad de que la identidad secreta de Superman sea Clark y la posibilidad de que no lo sea.

Por lo tanto, necesitaremos un modelo más complicado, uno que consista en una colección de **mundos posibles** en lugar de sólo un mundo verdadero. Los mundos están conectados en un grafo por **relaciones de accesibilidad**, una relación para cada operador modal. Decimos que el mundo w_1 es accesible desde el mundo w_0 con respecto al operador modal $\mathbf{K}_A$ si todo en w_1 es consistente con lo que *A* sabe en w_0. Por ejemplo, en el mundo real, Bucarest es la capital de Rumania, pero para un agente que no lo sabe, un mundo donde la capital de Rumania es, digamos, Sofía, es accesible. Con suerte, un mundo donde $2 + 2 = 5$ no sería accesible para ningún agente.

Mundo posible

Relación de accesibilidad

En general, un átomo de conocimiento $\mathbf{K}_A P$ es verdadero en el mundo *w* si y sólo si *P* es verdadero en cada mundo accesible desde *w*. La verdad de oraciones más complejas se deriva por la aplicación recursiva de esta regla y las reglas normales de la lógica de primer orden. Eso significa que la lógica modal se usa para razonar sobre oraciones de conocimiento anidadas: lo que un agente sabe sobre el conocimiento de otro agente. Así, decimos que, aunque Lois no sabe si la identidad secreta de Superman es Clark Kent, ella sí sabe que Clark sabe:

$$\mathbf{K}_{Lois}[\boldsymbol{K}_{Clark} Identidad(Superman, Clark) \vee \boldsymbol{K}_{Clark} \neg Identidad(Superman, Clark)]$$

La lógica modal resuelve problemas enredados con la interacción de cuantificadores y conocimiento. La oración "Bond sabe que alguien es un espía" es ambigua. La primera interpretación es que hay alguien que Bond sabe que es un espía; esto lo escribimos como

$$\exists x \, \boldsymbol{K}_{Bond} Espía(x),$$

lo que en lógica modal significa que hay un *x* que, en todos los mundos accesibles, Bond sabe que es un espía. La segunda lectura es que Bond sólo sabe que hay al menos un espía:

$$\boldsymbol{K}_{Bond} \exists x \, Espía(x).$$

La interpretación de la lógica modal es que en cada mundo accesible hay un *x* que es un espía, pero no necesariamente es el mismo *x* en cada mundo.

Ahora que tenemos un operador modal para el conocimiento, podemos escribir axiomas para ello. Primero, podemos decir que los agentes son capaces de sacar conclusiones; si un agente sabe P y sabe que P implica Q, entonces el agente sabe Q:

$$(\mathbf{K}_a P \wedge \mathbf{K}_a(P \Rightarrow Q)) \Rightarrow \mathbf{K}_a Q.$$

A partir de esto (y algunas otras reglas sobre identidades lógicas) podemos establecer que $\mathbf{K}_A(P \vee \neg P)$ es una tautología; cada agente sabe que cada proposición P es verdadera o falsa. Por otro lado, $(\mathbf{K}_A P) \vee (\mathbf{K}_A \neg P)$ no es una tautología; en general, habrá muchas proposiciones que un agente no sabe si son verdaderas o falsas.

Se dice (volviendo a Platón) que el conocimiento es creencia verdadera justificada. Es decir, si es verdadero, si lo crees, y si tienes una razón buena e inatacable, entonces lo sabes. Eso significa que, si sabes algo, debe ser verdadero, y tenemos el axioma:

$$\mathbf{K}_a P \Rightarrow P.$$

Además, los agentes lógicos (pero no todas las personas) son capaces de realizar introspecciones sobre su propio conocimiento. Si saben algo, entonces saben que lo saben:

$$\mathbf{K}_a P \Rightarrow \mathbf{K}_a(\mathbf{K}_a P).$$

Lógica omnisciente

Podemos definir axiomas similares para la creencia (denotada por **B**) y otras modalidades. Sin embargo, un problema con enfoque de lógica modal es que asume la **omnisciencia lógica** por parte de los agentes. Es decir, si un agente conoce un conjunto de axiomas, luego, conoce todas las consecuencias de éstos. Lo cual está en terreno inestable aun para la noción algo abstracta de conocimiento, pero parece todavía peor para la creencia, porque ésta tiene más connotación de referirse a cosas físicamente representadas en el agente, no sólo derivables en potencial.

Ha habido intentos de definir una forma de racionalidad limitada para los agentes, para decir que los agentes creen sólo las afirmaciones que pueden derivarse con la aplicación de no más de k pasos de razonamiento, o no más de s segundos de cómputo. Estos intentos han sido generalmente insatisfactorios.

10.4.1 Otras lógicas modales

Muchas lógicas modales se han propuesto para diferentes modalidades, además de conocimiento. Una propuesta agrega operadores modales de *posibilidad* y *necesidad*: es posiblemente verdadero que un autor del libro esté sentado ahora mismo y es necesariamente verdadero que 2 + 2 = 4.

Como referimos en la Sección 8.1.2, algunos lógicos favorecen modalidades relacionadas con el tiempo. En la **lógica temporal lineal**, agregamos estos operadores modales:

Lógica temporal lineal

- **X** P: "P será verdadero en el próximo paso de tiempo"
- **F** P: "P será eventualmente (Finalmente) verdadero en algún paso de tiempo futuro"
- **G** P: "P es siempre (Globalmente) verdadero"
- P **U** Q: "P sigue siendo verdadero hasta que ocurra Q"

A veces hay operadores adicionales que se pueden derivar de estos. Agregar estos operadores modales hace que la lógica en sí sea más compleja (y, por lo tanto, hace más difícil para un algoritmo de inferencia lógica encontrar una prueba). Pero los operadores también nos permiten afirmar ciertos eventos de una forma más simple (lo que hace más rápida la inferencia lógica). La elección de qué lógica usar es similar a la elección de qué lenguaje de programación usar: elija uno que sea apropiado para su tarea, familiar para usted y para los demás que compartirán su trabajo, y que sea lo suficientemente eficiente para sus propósitos.

10.5 Sistemas de razonamiento para categorías

Las categorías son los bloques de construcción primarios de los esquemas de representación de conocimiento a gran escala. Esta sección describe sistemas diseñados especialmente para organizar y razonar con categorías. Hay dos familias de sistemas relacionados: **redes semánticas** ofrecen ayudas gráficas para visualizar una BC y algoritmos eficientes para inferir propiedades de un objeto en función de su membresía en una categoría; y **lógicas de descripción** dan un lenguaje formal para construir y combinar definiciones de categorías y algoritmos eficientes para decidir relaciones de subconjunto y superconjunto entre categorías.

Redes semánticas

Lógicas de descripción

10.5.1 Redes semánticas

En 1909, Charles S. Peirce propuso una notación gráfica de nodos y bordes llamada **grafos existenciales**, que llamó "la lógica del futuro". Así comenzó un amplio debate entre los defensores de la "lógica" y los de las "redes semánticas". Por desgracia, el debate oscureció el hecho de que las redes semánticas *son* una forma de lógica. La notación que éstas proporcionan es a menudo más conveniente para ciertos tipos de oraciones, pero si eliminamos los problemas de "interfaz humana", los conceptos subyacentes (objetos, relaciones, cuantificación, etc.) son los mismos.

Grafos existenciales

Hay muchas variantes de redes semánticas, todas capaces de representar objetos individuales, categorías de objetos y relaciones entre ellos. Una notación gráfica típica muestra nombres de objetos o categorías en óvalos o cuadros, y los conecta con enlaces etiquetados. Por ejemplo, en la Figura 10.4 el enlace *MiembroDe* entre *Mary* y *PersonasFemeninas*, corresponde a la afirmación lógica $Mary \in PersonasFemeninas$; asimismo, el enlace *HermanaDe* entre *Mary* y *John* corresponde a la afirmación $HermanaDe(Mary,John)$. Podemos conectar categorías con enlaces *SubconjuntoDe*. Es tan divertido dibujar burbujas y flechas que uno puede dejarse llevar. Así, sabemos que las personas tienen como madres a personas del sexo femenino, ¿luego, podemos dibujar un enlace *TieneMamá* de *Personas* a *PersonasFemeninas*? La respuesta es no, porque *TieneMamá* es una relación entre una persona y su madre, las categorías no tienen madres.[5]

Por ello, en la Figura 10.4 usamos una notación especial: enlace de doble cuadro. Este enlace afirma que

$$\forall x \; x \in Personas \Rightarrow [\forall y \; TieneMamá(x,y) \Rightarrow y \in PersonasFemeninas].$$

También podríamos querer afirmar que las personas tienen dos piernas; es decir,

$$\forall x \; x \in Personas \Rightarrow Piernas\,(x,2).$$

Como antes, necesitamos tener cuidado de no afirmar que una categoría tiene piernas; el enlace de cuadro único en la Figura 10.4 se usa para afirmar propiedades de cada miembro de una categoría.

La notación de la red semántica hace idóneo realizar el razonamiento de **herencia** del tipo presentado en la Sección 10.2. Así, por el hecho de ser una persona, Mary hereda la propiedad de tener dos piernas. Para averiguar cuántas piernas tiene Mary, el algoritmo de herencia sigue el enlace *MiembroDe* desde *Mary* hasta la categoría a la que pertenece, y luego sigue los enlaces

[5] Varios sistemas iniciales no lograban distinguir entre las propiedades de los miembros de una categoría y las de la categoría en su conjunto. Esto puede conducir directamente a inconsistencias, como dijo Drew McDermott (1976) en su artículo "Artificial Intelligence Meets Natural Stupidity". Otro problema común fue el uso de enlaces *IsA* para relaciones de subconjunto y de membresía, en correspondencia con el uso en inglés: "*a cat is* a mamífero" y "Fifi *is a cat*". Vea el Ejercicio 10.NATS para profundizar en estos temas.

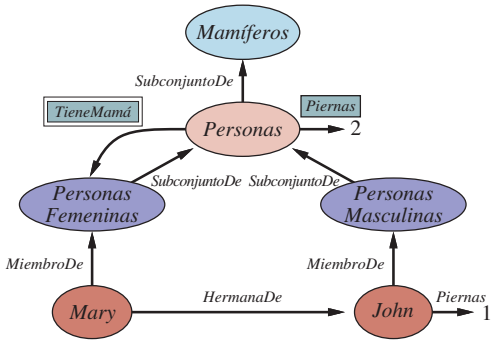


Figura 10.4 Una red semántica con cuatro objetos (John, Mary, 1 y 2) y cuatro categorías. Las relaciones se indican con enlaces etiquetados.

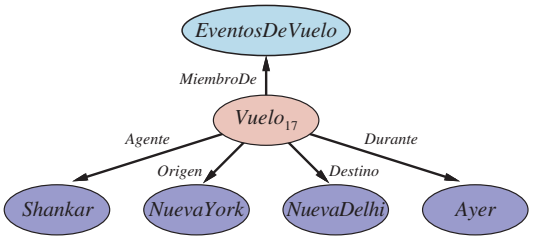


Figura 10.5 Un fragmento de una red semántica que muestra la representación de la afirmación lógica *Vuela*(*Shankar, NewYork, NewDelhi, Ayer*).

SubconjuntoDe hacia arriba en la jerarquía hasta encontrar una categoría para la cual hay un enlace *Piernas* en este caso, el enlace es con la categoría *Personas*. La simplicidad y eficiencia de este mecanismo de inferencia, si comparamos con la demostración de teoremas lógicos semidecidibles, es de las principales atracciones de las redes semánticas.

Herencia múltiple

La herencia se complica cuando un objeto puede pertenecer a más de una categoría o cuando una categoría puede ser un subconjunto de más de una categoría; esto se llama **herencia múltiple**. En tales casos, el algoritmo de herencia puede encontrar dos o más valores en conflicto que respondan a la consulta. Así, la herencia múltiple está prohibida en algunos lenguajes de **programación orientada a objetos** (POO, por sus siglas en inglés), como Java, que usa la herencia en jerarquía de clases. En general, se permite en redes semánticas, lo cual veremos en la Sección 10.6.

Quizás el lector notó un inconveniente obvio de la notación de redes semánticas, al comparar con la lógica de primer orden: el hecho de que los vínculos entre burbujas representan sólo relaciones *binarias*. Así, la frase *Vuela*(*Shankar*,*NewYork*,*NewDelhi*,*Ayer*) no se puede afirmar directamente en una red semántica. Empero, obtenemos el efecto de afirmaciones *n-arias* cosificando la proposición misma como un evento de una categoría de evento apropiada. La Figura 10.5 muestra la estructura de la red semántica para este evento en particular. Vea que la restricción a relaciones binarias obliga a la creación de una rica ontología de conceptos cosificados.

La cosificación de proposiciones posibilita representar cada oración atómica terrenal, sin funciones, de la lógica de primer orden en la notación de la red semántica. Unos tipos de oraciones cuantificadas universalmente se afirman con enlaces inversos, flechas con un solo cuadro y con doble cuadro aplicadas a categorías, pero eso aún nos deja lejos de la lógica de primer orden completa. La negación, disyunción, símbolos de función anidados y cuantificación existencial están ausentes. Ahora es *posible* extender la notación para equivalerla a la lógica de primer orden, como en los grafos existenciales de Peirce, pero al hacerlo anula una de las principales ventajas de las redes semánticas, que es la simplicidad y transparencia de los procesos de inferencia. Los diseñadores pueden construir una red grande y aún tener una buena idea sobre qué consultas serán eficientes, porque (a) es fácil visualizar los pasos siguientes del procedimiento de inferencia y (b) en unos casos, el lenguaje es tan simple que no se pueden plantear consultas difíciles.

En los casos donde el poder expresivo es muy limitado, muchos sistemas de redes semánticas dan un **anexo procedural** para llenar los vacíos. Este anexo es una técnica mediante la cual una consulta sobre (o afirmación de) cierta relación resulta en una llamada a un procedimiento especial diseñado para esa relación en vez de a un algoritmo de inferencia general.

Anexo procedural

Uno aspecto fundamental de las redes semánticas es su capacidad para representar **valores predeterminados** para categorías. En la Figura 10.4 se ve que John tiene una pierna, a pesar de ser una persona y de que las personas tienen dos piernas. En una lógica estricta de BC, esto sería contradictorio, pero en una red semántica, la afirmación: todas las personas tienen dos piernas, sólo tiene estado predeterminado; es decir, se asume que una persona tiene dos piernas a no ser que información específica afirme lo contrario. La semántica predeterminada se hace cumplir por el algoritmo de herencia, porque sigue los enlaces hacia arriba desde el objeto mismo (John, en este caso) y se detiene tan pronto encuentra un valor. Decimos que el predeterminado es **anulado** por el valor específico. Tenga en cuenta que podríamos anular el número predeterminado de piernas creando una categoría de *PersonasConUnaPierna*, un subconjunto de *Personas* del cual *John* es miembro.

Valor por defecto

Primordial

Podemos mantener una semántica estrictamente lógica para la red si decimos que la afirmación *Piernas* para *Personas* incluye una excepción para John:

$$\forall x \; x \in Personas \wedge x \neq John \Rightarrow Piernas(x,2).$$

Para una red *fija*, esto es semánticamente adecuado, pero será mucho menos conciso que la notación de la red misma si hay muchas excepciones. Pero, para una red que se actualizará con más afirmaciones; si tal enfoque falla, realmente diremos que cualquier persona aún desconocida con una pierna son excepciones. La Sección 10.6 profundiza más en este tema y en el razonamiento predeterminado en general.

10.5.2 Lógicas de descripción

La sintaxis de la lógica de primer orden está diseñada para facilitar decir cosas sobre objetos. Las **lógicas de descripción** son notaciones diseñadas para facilitar la descripción de definiciones y propiedades de categorías. Los sistemas de lógica de descripción evolucionaron de las redes semánticas en respuesta a la presión para formalizar lo que significan las redes mientras se retiene el énfasis en la estructura taxonómica como principio organizador.

Lógica de descripción

Las tareas vitales de inferencia para las lógicas de descripción son la **subsunción** (verifica si una categoría es un subconjunto de otra al comparar sus definiciones) y la **clasificación** (verifica si un objeto pertenece a una categoría). Unos sistemas incluyen la **consistencia** de una definición de categoría, si los criterios de membresía son lógicamente satisfactorios.

Subsunción

Clasificación

Consistencia

Concepto	→	**Thing**| *NombreDelConcepto*
	|	**And** (*Concepto,...*)
	|	**All** (*NombreDelRol,Concepto*)
	|	**AtLeast**(*Entero,NombreDeFunción*)
	|	**AtMost**(*Entero,NombreDeFunción*)
	|	**Fills**(*NombreDelRol,NombreDelIndividuo,...*)
	|	**SameAs**(*Ruta,Ruta*)
	|	**OneOf**(*NombreDelIndividuo,...*)
Ruta	→	[*NombreDelRol,...*]
NombreConcepto	→	*Adulto* | *Femenino* | *Masculino* | . . .
NombreDelRol	→	*Cónyuge* | *Hija* | *Hijo* | . . .

Figura 10.6 La sintaxis de las descripciones en un subconjunto del lenguaje CLASSIC.

El lenguaje CLASSIC (Borgida *et al.,* 1989) es una lógica de descripción típica. La sintaxis de las descripciones CLASSIC se muestra en la Figura 10.6.[6] Por ejemplo, para decir que los solteros son hombres adultos no casados, escribiríamos:

Soltero = *And*(*NoCasado,Adulto,Masculino*).

Lo equivalente en lógica de primer orden sería:

$$Soltero(x) \Leftrightarrow NoCasado(x) \land Adulto(x) \land Masculino(x).$$

Observe que la lógica de descripción tiene un álgebra de operaciones en predicados, lo que, claro, no podemos hacer en la lógica de primer orden. Cualquier descripción en CLASSIC puede traducirse en una oración de primer orden equivalente, pero unas descripciones son más directas en CLASSIC. Así, para describir el conjunto de hombres con al menos tres hijos desempleados y casados con doctoras, y como máximo dos hijas, todas profesoras de física o matemáticas, usamos:

And(Hombre,*AlLeast*(3,Hijo),*AtMost*(2,Hija),
All(Hijo,*And*(Desempleado,Casado,*All*(Cónyuge,Médico))),
All(Hija,*And*(Profesor,*Fills*(Departamento,Física,Matemáticas)))).

Lo dejamos como ejercicio para traducirlo a lógica de primer orden.

Quizás el aspecto fundamental de las lógicas de descripción sea su énfasis en el manejo de la inferencia. Una instancia del problema se resuelve al describirla y al preguntar si está subsumida por una de varias categorías de solución posibles. En los sistemas estándar de lógica de primer orden, con frecuencia es imposible predecir el tiempo de solución. A menudo, se deja al usuario ingeniar la representación para evitar conjuntos de oraciones que dilatan el sistema varias

[6] Considere que el lenguaje *no* permite afirmar sólo que un concepto o categoría es un subconjunto de otro. Esta es una política deliberada: la subsunción entre categorías debe ser derivable de algunos aspectos de las descripciones de las categorías. Si no es así, falta algo en las descripciones.

semanas en resolver un problema. Además, el enfoque en las lógicas de descripción es garantizar que la prueba de subsunción se resuelva en tiempo polinomial en el tamaño de las descripciones.[7]

Esto en principio suena bien, hasta que se vea que sólo puede tener una de dos consecuencias: ¡O los problemas difíciles no pueden ser declarados en absoluto, o requieren descripciones en potencia grandes! Pero, los resultados de ese manejo arrojan luz sobre qué tipos de construcciones causan problemas y ayudan al usuario a comprender cómo se comportan diferentes representaciones. Así, las lógicas de descripción suelen carecer de *negación* y *disyunción*; cada una fuerza los sistemas lógicos de primer orden a analizar casos en potencia exponenciales para garantizar completitud. CLASSIC permite sólo una forma limitada de disyunción en las construcciones *Fills* y *OneOf*, que permiten disyunción sobre individuos enumerados explícitamente, pero no sobre descripciones. Con descripciones disyuntivas, las definiciones anidadas llevan fácil a un número exponencial de rutas alternativas por las que una categoría se subsume a otra.

10.6 Razonamiento con información por defecto

En la sección anterior, vimos un ejemplo simple de una afirmación con estado predeterminado: las personas tienen dos piernas; la cual puede ser anulada por información específica, como que Long John Silver tiene una pierna. Vimos que el mecanismo de herencia en redes semánticas implementa la anulación de predeterminados de una manera simple y natural. Ahora, estudiaremos los predeterminados de manera general, con el objetivo de comprender su *semántica* en lugar de sólo proporcionar un mecanismo procedimental.

10.6.1 Circunscripción y lógica por defecto

Monotonicidad

Hemos visto dos ejemplos de procesos de razonamiento que violan la propiedad de **monotonicidad** de la lógica que se demostró en el Capítulo 7.[8] En este capítulo vimos que una propiedad heredada por todos los miembros de una categoría en una red semántica podría ser anulada por información más específica para una subcategoría. En la Sección 9.4.4, vimos que, bajo la suposición de mundo cerrado, si una proposición α no se menciona en BC entonces $BC \models \neg\alpha$, pero $BC \wedge \alpha = \alpha$.

La introspección simple sugiere que estas fallas de monotonicidad se generalizan en el razonamiento de sentido común. Parece que los humanos suelen "saltar a conclusiones". Por ejemplo, cuando vemos un automóvil estacionado en la calle, por lo regular estamos dispuestos a creer que tiene cuatro ruedas, aunque sólo se vean tres. Ahora bien, la teoría de la probabilidad puede con certeza concluir que la cuarta rueda existe con alta probabilidad; pero, para la mayoría de las personas, la posibilidad de que el automóvil no tenga cuatro ruedas *no surgirá a menos que surja nueva evidencia*. Así, parece que la conclusión de cuatro ruedas se alcanza por *defecto*, en ausencia de otra razón para dudarlo. Si llega nueva evidencia, por ejemplo, si el propietario lleva una rueda y el automóvil está levantado, entonces la conclusión puede ser retractada. Este tipo de razonamiento se dice que exhibe no **monotonicidad**, porque el conjunto de creencias no crece monótonamente con el tiempo a medida que llega nueva evidencia. Se han ideado **lógicas no monótonas** con nociones modificadas de verdad y consecuencia para capturar tal comportamiento. Analizaremos dos de esas lógicas estudiadas extensamente: circunscripción y lógica por defecto.

No monotonicidad

Lógica no monótona

La **circunscripción** puede verse como una versión más poderosa y precisa de la suposición de mundo cerrado. La idea es especificar predicados particulares que se asumen "tan falsos

Circunscripción

[7] CLASSIC prueba subsunción eficiente en la práctica, pero el tiempo de ejecución, en el peor caso, es exponencial.

[8] Recuerde que la monotonicidad requiere que todas las oraciones implicadas permanezcan implicadas después de que se agreguen nuevas oraciones a la BC. Es decir, si BC $\models \alpha$ entonces BC $\wedge \beta \models \alpha$.

como sea posible"; es decir, falsos para cada objeto, excepto aquellos que se saben verdaderos. Por ejemplo, suponga que queremos afirmar la regla predeterminada de que los pájaros vuelan. Pondríamos un predicado, digamos $Anormal_1(x)$, y escribiríamos

$$Pájaro(x) \wedge \neg Anormal_1(x) \Rightarrow Vuela(x).$$

Si decimos que $Anormal_1$ debe ser **circunscrito**, un razonador circunscriptivo está autorizado a asumir $\neg Anormal_1(x)$ a menos que se sepa que $Anormal_1(x)$ es verdadero. Esto permite *concluir* $Vuela(Piolín)$ a partir de la premisa $Pájaro(Piolín)$, pero la conclusión ya no se sostiene si se afirma $\neg Anormal_1(Piolín)$.

Preferencia de modelo

La circunscripción puede ser vista como un ejemplo de lógica de **preferencia de modelo**. En tales lógicas, una oración se deduce (con estado predeterminado) si es verdadera en todos los modelos *preferidos* de la BC, en contraposición al requisito de verdad en *todos* los modelos en la lógica clásica. Para la circunscripción, se prefiere un modelo sobre otro si tiene menos objetos anormales.[9] Veamos cómo funciona esta idea en el contexto de la herencia múltiple en redes semánticas. El ejemplo estándar para el cual la herencia múltiple es problemática se llama "diamante Nixon". Surge de la observación de que Nixon era Cuáquero (por defecto, pacifista) y Republicano (por defecto, no un pacifista). Podemos escribir esto como sigue:

$$Republicano(Nixon) \wedge Cuáquero(Nixon).$$
$$Republicano(x) \wedge \neg Anormal_2(x) \Rightarrow \neg Pacifista(x).$$
$$Cuáquero(x) \wedge \neg Anormal_3(x) \Rightarrow Pacifista(x).$$

Si circunscribimos $Anormal_2$ y $Anormal_3$, hay dos modelos preferidos: uno en el que $Anormal_2(Nixon)$ y $Pacifista(Nixon)$ son verdaderos y otro en el que $Anormal_3(Nixon)$ y $\neg Pacifista(Nixon)$ son verdaderos. Así, el razonador circunscriptivo permanece adecuadamente agnóstico respecto a si Nixon era un pacifista. Si deseamos, además, afirmar que las creencias religiosas tienen precedencia sobre las creencias políticas, podemos usar un formalismo llamado **circunscripción priorizada** para dar preferencia a los modelos donde $Anormal_3$ se minimiza.

Circunscripción priorizada

Lógica por defecto

Regla por defecto

La **lógica por defecto** es un formalismo en el que las **reglas por defecto** se pueden escribir para generar conclusiones contingentes, no monótonas. Una regla por defecto se ve así:

$$Pájaro(x) : Vuela(x)/Vuela(x).$$

Esta regla significa que si $Pájaro(x)$ es verdadero, y si $Vuela(x)$ es consistente con la BC, entonces $Vuela(x)$ se concluye por defecto. En general, una regla por defecto tiene la forma

$$P: J_1,...,J_n/C$$

donde P se llama el prerrequisito, C es conclusión y J_i son las justificaciones, si se demuestra que una de ellas es falsa, entonces la conclusión no se puede lograr. Cualquier variable que aparezca en J_i o C también debe aparecer en P. El ejemplo del diamante Nixon se representa en la lógica por defecto con un hecho y dos reglas por defecto:

$$Republicano(Nixon) \wedge Cuáquero(Nixon).$$
$$Republicano(x) : \neg Pacifista(x)/\neg Pacifista(x).$$
$$Cuáquero(x) : Pacifista(x)/Pacifista(x).$$

[9] Para el supuesto de mundo cerrado, se prefiere un modelo a otro si tiene menos átomos verdaderos; es decir, los modelos preferidos son los **mínimos**. Existe una conexión natural entre el supuesto de mundo cerrado y las BC de cláusula definida, porque el punto fijo alcanzado por el encadenamiento directo en las BC de cláusula definida es el modelo mínimo único. Vea la página 231 para profundizar sobre este punto.

Para interpretar el significado de las reglas predeterminadas, definimos la noción de una **extensión** de una teoría predeterminada como un conjunto maximal de consecuencias de la teoría. Es decir, una extensión S consiste de los hechos originales conocidos y un conjunto de conclusiones de las reglas predeterminadas, de modo que no se pueden extraer conclusiones adicionales de S, y las justificaciones de cada conclusión predeterminada en S son consistentes con S. Como en los modelos preferidos en la circunscripción, hay dos posibles extensiones para el diamante de Nixon: una donde es pacifista y otra donde no lo es. Hay esquemas priorizados donde unas reglas predeterminadas pueden preceder otras, lo que permite resolver ambigüedades.

Extensión

Desde 1980, cuando se propusieron lógicas no monótonas, se ha avanzado mucho en la comprensión de sus propiedades matemáticas. Pero, aún hay preguntas sin resolver. Por ejemplo, si "Los autos tienen cuatro ruedas" es falso, ¿qué significa tener en la BC? ¿Cuál es un buen conjunto de reglas predeterminadas para tener? Si no podemos decidir, para cada regla por separado, si pertenece a nuestra BC, entonces tenemos un grave problema de no modularidad. Finalmente, ¿cómo usar las creencias de un estado predeterminado para tomar decisiones? Este es, quizás, el problema más difícil para el razonamiento predeterminado.

Las decisiones suelen implicar compensaciones y, así, es necesario comparar las *fortalezas* de la creencia en los resultados de diferentes acciones y los *costos* de tomar una decisión incorrecta. Cuando se toman las mismas decisiones repetidamente, se pueden interpretar reglas predeterminadas como declaraciones de "probabilidad de umbral". Así, la regla predeterminada "Mis frenos siempre están bien", de hecho significa "La probabilidad de que mis frenos estén bien, sin más información, es alta en suficiencia para que la decisión óptima sea conducir sin verificarlos". Cuando el contexto de la decisión cambia, por ejemplo, si se conduce un camión cargado cuesta abajo, la regla predeterminada de repente se vuelve inapropiada, aunque no hay evidencia nueva de frenos defectuosos. Estas consideraciones han llevado a analizar cómo incorporar el razonamiento predeterminado dentro de las teorías de la probabilidad o de la utilidad.

10.6.2 Sistemas de mantenimiento de la verdad

Hemos visto que muchas de inferencias realizadas por un sistema de representación del conocimiento tendrán sólo un estado predeterminado, en vez de ser ciertas. Inevitablemente, unas de estas afirmaciones inferidas serían incorrectas y deberían retirarse ante información nueva. Este proceso se llama **revisión de creencias.**[10] Suponga que una BC contiene una oración P, quizás una conclusión predeterminada registrada por un algoritmo de encadenamiento hacia adelante o sólo una afirmación incorrecta, y queremos ejecutar $DECIR(BC,\neg P)$. Para evitar una contradicción, primero debemos ejecutar $RETRACTAR(BC,P)$; esto suena fácil, pero surgen problemas si se han inferido oraciones *adicionales* a partir de P y se han afirmado en la BC. Por ejemplo, la implicación $P \Rightarrow Q$ podría haberse usado para agregar Q. La "solución" obvia, retractar todas las oraciones inferidas de P, falla porque esas oraciones pueden tener otras justificaciones además de P. Entonces, si R y $R \Rightarrow Q$ también están en la BC, entonces Q no tiene que ser eliminado después de todo. Los **sistemas de mantenimiento de la verdad** están diseñados para manejar exactamente estos tipos de complicaciones.

Revisión de creencias

Sistemas de mantenimiento de la verdad

Un enfoque simple para el mantenimiento de la verdad es registrar el orden en que se dicen las oraciones a la BC, al numerarlas de P_1 a P_n. Cuando se realiza la llamada $RETRACTAR(BC,P_i)$,

[10] La revisión de creencias se contrasta con la actualización de creencias, cuando se revisa una BC para reflejar un cambio en el mundo, no nueva información sobre un mundo fijo. Tal actualización combina revisión de creencias con razonamiento sobre tiempo y cambio; también se relaciona con el proceso de filtrado del Capítulo 14.

el sistema vuelve al estado justo antes de que se agregara P_i, eliminando P_i y cualquier inferencia derivada de P_i. Las oraciones P_{i+1} a P_n luego se pueden volver a agregar. Es simple y se garantiza que la BC sea consistente, pero retractar P_i, requiere retractar y volver a afirmar $n - i$ oraciones, así como deshacer y rehacer todas las inferencias derivadas de éstas. Para sistemas a los que se les agregan muchos hechos, como grandes bases de datos comerciales, esto es impráctico.

JTMS

Un enfoque más eficiente es el sistema de mantenimiento de la verdad basado en justificaciones, o **JTMS** (por sus siglas en inglés). En un JTMS, cada oración en la BC está anotada con una **justificación** que consiste en el conjunto de oraciones a partir de las cuales fue inferida. Por ejemplo, si la BC ya contiene $P \Rightarrow Q$, entonces DECIR(P) hará que ahora se agregue Q con la justificación $\{P, P \Rightarrow Q\}$. En general, una oración puede tener cualquier número de justificaciones, las cuales hacen que la retracción sea eficiente. Dada la llamada REDACTAR(P), el JTMS eliminará exactamente aquellas oraciones para las cuales P es miembro de cada justificación. Por lo tanto, si una oración Q tenía la única justificación $\{P, P \Rightarrow Q\}$, sería eliminada; si tuviera la justificación adicional $\{P, P \vee R \Rightarrow Q\}$, aún sería eliminada; pero si también tuviera la justificación $\{R, P \vee R \Rightarrow Q\}$, entonces se salvaría. De esta manera, el tiempo requerido para la retracción de P depende sólo del número de oraciones derivadas de P en lugar del número de oraciones agregadas después de P.

Justificación

El JTMS asume que las oraciones que se consideran una vez quizá sean consideradas nuevamente, por lo que en lugar de eliminar una oración de la BC por completo cuando pierde todas las justificaciones, sólo marcamos la oración como *fuera* de la BC. Si una afirmación posterior restaura una de las justificaciones, entonces marcamos la oración como *de vuelta* en la BC. De esta manera, el JTMS retiene todas las cadenas de inferencia que usa y no necesita volver a derivar oraciones cuando una justificación vuelve a ser válida.

Además de manejar la retracción de información incorrecta, los TMS también pueden usarse para acelerar el análisis de múltiples situaciones hipotéticas. Suponga, por ejemplo, que el Comité Olímpico Rumano elige sedes para los eventos de natación, atletismo y equitación para los Juegos de 2048 en Rumania. Suponga, por ejemplo, que la primera hipótesis es *Sitio*(*Nadar*,*Pitesti*), *Sitio*(*Atletismo*,*Bucharest*), y *Sitio*(*Ecuestre*,*Arad*).

Entonces se debe realizar un razonamiento para calcular las consecuencias logísticas y, por lo tanto, la deseabilidad de la selección. Si queremos considerar *Sitio*(*Atletismo*, *Sibiu*), el JTMS evita la necesidad de recomenzar desde cero. En su lugar, sólo retractamos *Sitio*(*Atletismo*,*Bucharest*) y afirmamos *Sitio*(*Atletismo*,*Sibiu*) y el JTMS se encarga de las revisiones necesarias. Las cadenas de inferencia generadas a partir de la elección de Bucarest pueden ser reutilizadas con Sibiu, siempre que las conclusiones sean las mismas.

ATMS

Un sistema de mantenimiento de la verdad basado en suposiciones, o **ATMS** (por sus siglas en inglés), hace que este tipo de cambio de contexto entre mundos hipotéticos sea en particular eficiente. En un JTMS, el mantenimiento de justificaciones le permite moverse rápidamente de un estado a otro y realizar retracciones y afirmaciones, pero en todo momento sólo se representa un estado. Un ATMS representa *todos* los estados considerados alguna vez al mismo tiempo. Mientras que un JTMS sólo etiqueta cada oración como *dentro* o *fuera*, un ATMS lleva un registro, para cada oración, de qué suposiciones harían que ésta fuera verdadera. Es decir, cada oración tiene una etiqueta que consiste en un conjunto de conjuntos de suposiciones. La oración es verdadera sólo en los casos en que todas las suposiciones en uno de los conjuntos de suposiciones sean verdaderas.

Explicación

Los TSM también proporcionan un mecanismo para generar **explicaciones**. Técnicamente, una explicación de una oración P es un conjunto de oraciones E tal que E implica P. Si las oraciones en E se saben que son verdaderas, entonces E sólo da una base suficiente para

demostrar que P debe ser el caso. Pero las explicaciones también incluyen **suposiciones** (oraciones que no se sabe que sean verdaderas, pero que serían suficientes para probar P si fueran verdaderas). Así, si tu auto no arranca, quizá no tengas suficiente información para probar definitivamente la razón del problema. Pero una explicación razonable incluiría la suposición de que la batería está muerta. Esto, combinado con el conocimiento de cómo funcionan los autos, explica el comportamiento observado. En la mayoría de los casos, preferiremos una explicación E que sea mínima, lo que significa que no existe un subconjunto adecuado de E que también sea una explicación. Un ATMS puede generar explicaciones para el problema "el auto no arranca" con suposiciones como "sin gasolina no hay auto" o "batería muerta", en cualquier orden que deseemos, aun si unas suposiciones son contradictorias. Luego, vemos la etiqueta de la oración "el auto no arranca" para leer los conjuntos de suposiciones que justificarían la oración.

Suposiciones

Los algoritmos exactos usados para implementar TSM son un poco complicados, y no los cubrimos aquí. La complejidad computacional del problema de mantenimiento de la verdad es tan grande como la de la inferencia proposicional, es decir, NP-duro. Por ello, no debe esperar que el mantenimiento de la verdad sea una panacea. Pero, si se usa con cuidado, un TMS puede ofrecer un aumento sustancial en la capacidad de un sistema lógico para manejar entornos e hipótesis complejas.

Resumen

Al adentrarse en los detalles de cómo se representa una variedad de conocimientos, esperamos haber dado una idea de cómo se construyen las BC reales y de los interesantes problemas filosóficos que surgen. Los puntos principales son:

- La representación del conocimiento a gran escala requiere una ontología de propósito general para organizar y vincular los diversos dominios específicos del conocimiento.
- Una ontología de propósito general necesita cubrir una amplia variedad de conocimientos y debería ser capaz, en principio, de manejar cualquier dominio.
- Construir una ontología general de gran tamaño es un desafío significativo aún no realizado por completo, aunque los marcos actuales parecen ser muy robustos.
- Presentamos una **ontología superior** basada en categorías y en el cálculo de eventos. Cubrimos categorías, subcategorías, partes, objetos estructurados, mediciones, sustancias, eventos, tiempo y espacio, cambio y creencias.
- Los tipos naturales no pueden definirse completamente en lógica, pero las propiedades de los tipos naturales pueden representarse.
- Las acciones, eventos y el tiempo pueden representarse con el cálculo de eventos. Tales representaciones permiten a un agente construir secuencias de acciones e inferir lógicamente qué será cierto cuando ocurran estas acciones.
- Se han ideado sistemas de representación de propósito especial, como **redes semánticas** y **lógicas de descripción**, para ayudar a organizar una jerarquía de categorías. La **herencia** es una forma importante de inferencia, que permite deducir las propiedades de los objetos de su pertenencia a categorías.
- La **suposición del mundo cerrado**, tal como se implementa en programas lógicos, proporciona una manera simple para no especificar mucha información negativa. Es mejor interpretarla como un valor predeterminado que puede ser anulado por información adicional.
- Las **lógicas no monótonas**, como la circunscripción y la lógica por defecto, están destinadas a capturar el razonamiento por defecto en general.

- Los **sistemas de mantenimiento de la verdad (TSM)** manejan las actualizaciones y revisiones del conocimiento de manera eficiente.
- Es difícil construir ontologías grandes a mano; extraer conocimiento del texto facilita el trabajo.

Notas bibliográficas e históricas

Briggs (1985) afirma que la investigación en representación del conocimiento comenzó con la teorización india del primer milenio a.C. sobre la gramática del sánscrito Shastric. Los filósofos occidentales remontan su trabajo sobre el tema hacia el año 300 a.C. en la *Metafísica* de Aristóteles (literalmente, lo que está después del libro sobre física). El desarrollo de terminología técnica en cualquier campo puede considerarse como una forma de representación del conocimiento.

Las primeras discusiones sobre representación en IA se centraron en la "representación del *problema*" en vez de la "representación del *conocimiento*". (Véase, por ejemplo, la discusión de Amarel (1968) sobre el problema de "misioneros y caníbales"). En la década de 1970, la IA enfatizó el desarrollo de "sistemas expertos" (o "sistemas basados en conocimiento") que, si se les proporcionaba el conocimiento de dominio adecuado, podían igualar o superar el rendimiento de los expertos humanos en tareas definidas de con especificidad. Por ejemplo, el primer sistema experto, DENDRAL (Feigenbaum *et al*., 1971; Lindsay *et al*., 1980), interpretó la salida de un espectrómetro de masas (un instrumento usado para analizar la estructura de compuestos químicos orgánicos) tan preciso como un químico experto. Aunque el éxito de DENDRAL fue fundamental para convencer a los investigadores en IA de la importancia de la representación del conocimiento, los formalismos de representación usados en DENDRAL son muy específicos para el dominio de la química.

Con el tiempo, los investigadores se interesaron por los formalismos de representación del conocimiento estandarizados y las ontologías que podrían ayudar en la creación de nuevos sistemas expertos. Esto los llevó a territorios antes explorados por filósofos de la ciencia y del lenguaje. La disciplina impuesta en IA por la necesidad de que las teorías "funcionen" ha llevado a un progreso más veloz y profundo de cuando estos problemas eran exclusivos de la filosofía (aunque también ha llevado a la reiterada reinvención de la rueda).

Pero ¿hasta qué punto confiar en el conocimiento experto? Desde 1955, Paul Meehl (véase también Grove y Meehl, 1996) estudió los procesos de toma de decisiones de expertos entrenados en tareas subjetivas, como predecir el éxito de un estudiante en un programa de formación o la reincidencia de un delincuente. En 19 de 20 casos examinados, Meehl encontró que algoritmos de aprendizaje estadístico simples (como la regresión lineal o el Bayes ingenuo) predicen mejor que los expertos. Tetlock (2017) también estudia el conocimiento experto y encuentra que es deficiente en casos difíciles. El Servicio de Evaluación Educativa ha usado un programa automatizado para calificar millones de preguntas de ensayo en el examen GMAT desde 1999. El programa concuerda con los evaluadores humanos 97 % del tiempo, casi el mismo nivel en el que dos evaluadores humanos están de acuerdo (Burstein *et al*., 2001). (Esto no significa que el programa comprenda los ensayos, sólo distingue entre buenos y malos con la misma eficacia que los evaluadores humanos).

La creación de taxonomías o clasificaciones completas se remonta a la antigüedad. Aristóteles (384-322 a.C.) enfatizó los esquemas de clasificación y categorización. Su *Organon*, una colección de obras sobre lógica compiladas por sus estudiantes después de su muerte, incluía el tratado de las *Categorías* en el que intentaba construir lo que ahora llamaríamos una ontología superior. También introdujo las nociones de género y especie para la clasificación a un nivel inferior.

Nuestro sistema actual de clasificación biológica, incluido el uso de la "nomenclatura binomial" (clasificación mediante género y especie en el sentido técnico), fue inventado por el biólogo sueco Carolus Linnaeus, o Carl von Linne (1707-1778). Los problemas asociados con los tipos naturales y los límites de categoría inexactos han sido abordados por Wittgenstein (1953), Quine (1953), Lakoff (1987) y Schwartz (1977), entre otros.

Consulte el Capítulo 24 para una revisión sobre las representaciones de palabras y conceptos de redes neuronales profundas que evitan algunos de los problemas de una ontología estricta, aunque sacrifican parte de la precisión. Aún no sabemos la mejor manera de combinar las ventajas de las redes neuronales y la semántica lógica para la representación.

El interés en ontologías a mayor escala está aumentando, como se documenta en el *Handbook on Ontologies* (Staab, 2004). El proyecto OpenCYC (Lenat y Guha, 1990; Matuszek *et al*., 2006) ha lanzado una ontología de 150 000 conceptos, con una ontología superior similar a la mostrada en la Figura 10.1, así como conceptos específicos como "Pantalla OLED" y "iPhone", un tipo de "teléfono celular" que, a su vez, es un tipo de "electrónica de consumo", "teléfono", "dispositivo de comunicación inalámbrica" y otros conceptos. El proyecto NextKB extiende CYC y otros recursos incluido FrameNet y WordNet en una BC con casi 3 millones de hechos y un motor de razonamiento FIRE para acompañarlo (Forbus *et al*., 2010).

El proyecto DBpedia extrae datos estructurados de Wikipedia, en específico de Infoboxes: los pares de atributos/valores que acompañan muchos artículos de Wikipedia (Wu y Weld, 2008; Bizer *et al*., 2007). Hasta 2015, DBpedia contenía 400 millones de hechos sobre 4 millones de objetos, sólo en la versión en inglés; contando los 110 idiomas, se suman 1.5 mil millones de hechos (Lehmann *et al*., 2015).

El grupo de trabajo IEEE P1600.1 creó SUMO, la Ontología Fusionada Superior Sugerida (Niles y Pease, 2001; Pease y Niles, 2002), con cerca de 1 000 términos en la ontología superior y enlaces a más de 20 000 términos específicos del dominio. Stoffel *et al*. (1997) describen algoritmos para gestionar eficientemente una ontología grande. Una encuesta de técnicas para extraer conocimiento de páginas web está dada por Etzioni *et al*. (2008).

En la web, están emergiendo lenguajes de representación. RDF (Brickley y Guha, 2004) permite hacer afirmaciones en forma de triples relacionales y dan algunos medios para evolucionar el significado de los nombres con el tiempo. OWL (Smith *et al*., 2004) es una lógica de descripción que soporta inferencias sobre estos triples. Hasta ahora, el uso parece ser inversamente proporcional a la complejidad representacional: los formatos tradicionales de HTML y CSS representan más de 99% del contenido web, seguidos de los esquemas de representación más simples, como RDFa (Adida y Birbeck, 2008) y microformatos (Khare, 2006; Patel-Schneider, 2014) que usan marcado HTML y XHTML para agregar atributos al texto en las páginas web. El uso de ontologías RDF y OWL sofisticadas aún no es generalizado, y la visión completa de la Web Semántica (Berners-Lee *et al*., 2001) no se ha realizado. Las conferencias *Formal Ontology in Information Systems* (FOIS) cubren ontologías generales y específicas del dominio.

La taxonomía usada en este capítulo fue desarrollada por los autores y se basa en parte en su experiencia en el proyecto CYC y ene en el trabajo de Hwang y Schubert (1993) y Davis (1990, 2005). Una revisión inspiradora sobre el proyecto general de representación del conocimiento del sentido común aparece en el "Manifiesto de la Física Ingenua" de Hayes (1978, 1985b).

El proyecto de Gene Ontology (Consorcio de Gene Ontology, 2008) y el Lenguaje de Marcado Químico (Murray-Rust *et al*., 2003) son ejemplos exitosos de ontologías profundas dentro de un campo específico. Las dudas sobre la viabilidad de una sola ontología para todo el conocimiento son expresadas por Doctorow (2001), Gruber (2004), Halevy *et al*. (2009) y Smith (2004).

El cálculo de eventos fue introducido por Kowalski y Sergot (1986) para manejar el tiempo continuo, y ha habido varias variaciones (Sadri y Kowalski, 1995; Shanahan, 1997) y descripciones generales (Shanahan, 1999; Mueller, 2006). James Allen introdujo intervalos de tiempo por la misma razón (Allen, 1984), argumentando que los intervalos eran mucho más naturales que las situaciones para razonar sobre eventos extendidos y concurrentes. En Van Lambalgen y Hamm (2005) vemos cómo la lógica de eventos se relaciona con el lenguaje que usamos para hablar sobre eventos. Una alternativa a los cálculos de eventos y situaciones es el cálculo fluido (Thielscher, 1999), que reifica los hechos de los que están compuestos los estados.

Peter Ladkin (1986a, 1986b) introdujo intervalos de tiempo "cóncavos" (intervalos con brechas, en esencia, uniones de intervalos de tiempo "convexos" ordinarios) y aplicó las técnicas del álgebra abstracta matemática a la representación del tiempo. Allen (1991) investiga sistemáticamente la amplia variedad de técnicas disponibles para la representación del tiempo; Van Beek y Manchak (1996) analizan algoritmos para el razonamiento temporal. Hay similitudes significativas entre la ontología basada en eventos dada en este capítulo y un análisis de eventos del filósofo Donald Davidson (1980). Las **historias** en la ontología de líquidos de Pat Hayes (1985a) y las **crónicas** en la teoría de planes de McDermott (1985) también fueron influencias importantes en el campo y en este capítulo.

La cuestión del estatus ontológico de las sustancias tiene una larga historia. Platón propuso que las sustancias eran entidades abstractas distintas por completo de los objetos físicos; él diría *HechoDe* (*Mantequilla*$_3$,*Mantequilla*) en lugar de *Mantequilla* $\in$ *Mantequilla*. Esto lleva a una jerarquía de sustancias en la que, por ejemplo, *MantequillaSinSal* es una sustancia más específica que *Mantequilla*. La posición adoptada en este capítulo, en la que las sustancias son categorías de objetos, fue defendida por Richard Montague (1973). También ha sido adoptada en el proyecto CYC. Copeland (1993) plantea una crítica seria, pero no invencible.

El enfoque alternativo mencionado en el capítulo, en el que la mantequilla es un objeto que consiste en todos los objetos mantecosos en el universo, fue propuesto originalmente por el lógico polaco Leśniewski (1916). Su **mereología** (el nombre se deriva de la palabra griega para "parte") usaba la relación parte-todo como sustituto de la teoría matemática de conjuntos, con el objetivo de eliminar entidades abstractas como conjuntos. Una exposición más legible de estas ideas es la de Leonard y Goodman (1940), y la obra de Goodman, *The Structure of Appearance*(1977), aplica las ideas a varios problemas en la representación del conocimiento.

Si bien algunos aspectos del enfoque mereológico son incómodos, por ejemplo, la necesidad de un mecanismo de herencia separado basado en relaciones parte-todo, el enfoque ganó el apoyo de Quine (1960). Harry Bunt (1985) dio un extenso análisis de su uso en la representación del conocimiento. Casati y Varzi (1999) cubren partes, *wholes*, y una teoría general de localizaciones espaciales.

Hay tres enfoques principales para el estudio de objetos mentales. El tomado en este capítulo, basado en la lógica modal y mundos posibles, es el enfoque clásico de la filosofía (Hintikka, 1962; Kripke, 1963; Hughes y Cresswell, 1996). El libro *Reasoning about Knowledge* (Fagin *et al.,* 1995) dan una introducción completa, y Gordon y Hobbs (2017) ofrecen *A Formal Theory of Commonsense Psychology*.

El segundo enfoque es una teoría de primer orden en la que los objetos mentales son *fluents*. Davis (2005) y Davis y Morgenstern (2005) describen este enfoque, que se basa en el formalismo de mundos posibles y está construido sobre el trabajo de Robert Moore (1980, 1985).

El tercer enfoque es una **teoría sintáctica**, en la que los objetos mentales están representados por cadenas de caracteres. Una cadena es sólo un término complejo que denota una

lista de símbolos, por lo que *PuedeVolar*(*Clark*) se puede representar por la lista de símbolos [*P,u,e,d,eV,o,l,a,r*(,*C*,*l*,*a*,*r*,k,)]. La teoría sintáctica de los objetos mentales fue estudiada por primera vez en profundidad por Kaplan y Montague (1960), quienes mostraron que conducía a paradojas si no se manejaba con cuidado. Ernie Davis (1990) ofrece una excelente comparación de las teorías sintácticas y modales del conocimiento. Pnueli (1977) describe una lógica temporal usada para razonar sobre programas, trabajo que le valió el Premio Turing y que fue ampliado por Vardi (1996). Littman *et al*. (2017) muestran que una lógica temporal puede ser un buen lenguaje para especificar metas a un robot de aprendizaje por refuerzo de una manera fácil de especificar para un humano, y se generaliza bien a diferentes entornos.

El filósofo griego Porfirio (c. 234–305), comentando sobre las *Categorías* de Aristóteles, dibujó lo que podría calificarse como la primera red semántica. Charles S. Peirce (1909) desarrolló gráficos existenciales como el primer formalismo de red semántica usando lógica moderna. Ross Quillian (1961), impulsado por un interés en la memoria humana y el procesamiento del lenguaje, inició el trabajo sobre redes semánticas dentro de la IA. Un influyente artículo de Marvin Minsky (1975) presentó una versión de redes semánticas llamada **marcos**; un marco era una representación de un objeto o categoría, con atributos y relaciones con otros objetos o categorías.

La cuestión de la semántica surgió de manera aguda con respecto a las redes semánticas de Quillian (y las de otros que siguieron su enfoque), con sus omnipresentes y muy vagos "enlaces IS-A". El famoso artículo de Bill Woods (1975) "¿Qué hay en un enlace?" llamó la atención de los investigadores de IA sobre la necesidad de una semántica precisa en los formalismos de representación del conocimiento. Ron Brachman (1979) elaboró este punto y propuso soluciones. El "La lógica de los marcos" de Patrick Hayes (1979) fue aún más a fondo al afirmar que "La mayor parte de los 'marcos' es sólo una nueva sintaxis para partes de la lógica de primer orden". El "Semántica tarskiana, o, ¡No hay notación sin denotación!" de Drew McDermott (1978b) argumentaba que el enfoque basado en modelos de la semántica usado en la lógica de primer orden debería aplicarse a todos los formalismos de representación del conocimiento. Esta sigue siendo una idea en verdad controvertida; el propio McDermott ha revertido su posición en "Una crítica de la razón pura" (McDermott, 1987). Selman y Levesque (1993) discuten la complejidad de la herencia con excepciones, mostrando que en la mayoría de las formulaciones es NP-duro.

Las lógicas de descripción se desarrollaron como un subconjunto útil de la lógica de primer orden para la cual la inferencia es computacionalmente tratable. Hector Levesque y Ron Brachman (1987) mostraron que ciertos usos de la disyunción y la negación eran responsables principales de la intratabilidad de la inferencia lógica. Esto llevó a una mejor comprensión de la interacción entre la complejidad y la expresividad en los sistemas de razonamiento. Calvanese *et al*. (1999) resumen el estado del arte, y Baader *et al*. (2007) presentan un manual completo de lógica de descripción.

Los tres principales formalismos para tratar la inferencia no monótona: la circunscripción (McCarthy, 1980), la lógica por defecto (Reiter, 1980) y la lógica no monótona modal (McDermott y Doyle, 1980), fueron introducidos en un número especial de la *AI Journal*. Delgrande y Schaub (2003) discuten los méritos de las variantes, dado 25 años de retrospectiva. La programación de conjuntos de respuesta se puede ver como una extensión de la negación como fallo o como un refinamiento de la circunscripción; la teoría subyacente de la semántica del modelo estable fue introducida por Gelfond y Lifschitz (1988), y los principales sistemas de programación de conjuntos de respuesta son DLV (Eiter *et al*., 1998) y SMODELS (Niemelä *et al*., 2000). Lifschitz (2001) discute el uso de la programación de conjuntos de respuesta para la planificación. Brewka *et al*. (1997) ofrece una visión general de los diversos enfoques de la lógica no monótona. Clark (1978) cubre el enfoque de la negación como fallo a la programación lógica y la terminación de Clark.

Lifschitz (2001) discute la aplicación de la programación de conjuntos de respuesta a la planificación. Una variedad de sistemas de razonamiento no monótono basados en la programación lógica está documentados en las actas de las conferencias como por ejemplo *Logic Programming and Nonmonotonic Reasoning (LPNMR).*

El estudio de los sistemas de mantenimiento de la verdad comenzó con los sistemas TMS (Doyle, 1979) y RUP (McAllester, 1980), ambos eran esencialmente JTMS. Forbus y de Kleer (1993) explican en profundidad cómo se pueden utilizar los TMS en aplicaciones de IA. Nayak y Williams (1997) muestran cómo un TMS incremental eficiente llamado ITMS hace factible planificar las operaciones de una nave espacial de la NASA en tiempo real.

Este capítulo no pudo cubrir todas las áreas de la representación del conocimiento en profundidad. Los tres temas principales omitidos son los siguientes:

Física cualitativa

Física cualitativa: La física cualitativa es un subcampo de la representación del conocimiento preocupado específicamente por construir una teoría lógica, no numérica, de objetos y procesos físicos. El término fue acuñado por Johan de Kleer (1975), aunque la empresa podría decirse que comenzó en BUILD de Fahlman (1974), un planificador sofisticado para construir torres complejas de bloques. Fahlman descubrió en su proceso de diseño que la mayor parte del esfuerzo (80 %, según su estimación) se destinaba a modelar la física del mundo de los bloques para calcular la estabilidad de varias subestructuras de bloques, más que a la planificación en sí. Él esboza un proceso ingenuo similar a la física para explicar por qué los niños pequeños pueden resolver problemas similares a BUILD sin tener acceso a la aritmética de punto flotante de alta velocidad usada en la modelización física de BUILD. Hayes (1985a) utiliza "historias" –cortes tridimensionales del espacio-tiempo similares a los eventos de Davidson– para construir una física ingenua compleja de los líquidos. Davis (2008) ofrece una actualización de la ontología de los líquidos que describe el vertido de líquidos en contenedores.

De Kleer y Brown (1985), Ken Forbus (1985) y Benjamin Kuipers (1985) desarrollaron de manera independiente, y casi simultánea, sistemas que pueden razonar sobre un sistema físico basado en abstracciones cualitativas de las ecuaciones subyacentes. La física cualitativa pronto se desarrolló hasta el punto en que se hizo posible analizar una variedad impresionante de sistemas físicos complejos (Yip, 1991). Las técnicas cualitativas se han usado para construir diseños novedosos de relojes, limpiaparabrisas y caminantes de seis patas (Subramanian y Wang, 1994). La colección *Readings in Qualitative Reasoning about Physical Systems* (Weld y de Kleer, 1990), un artículo de enciclopedia de Kuipers (2001) y un artículo de manual de Davis (2007) proporcionan buenas introducciones al campo.

Razonamiento espacial

Razonamiento espacial: El razonamiento necesario para navegar en el mundo de los wumpus es trivial en comparación con la rica estructura espacial del mundo real. El primer intento serio de capturar el razonamiento del sentido común sobre el espacio aparece en el trabajo de Ernest Davis (1986, 1990). El cálculo de conexión de regiones de Cohn *et al*. (1997) admite una forma de razonamiento espacial cualitativo y ha dado lugar a nuevos tipos de sistemas de información geográfica; ver también (Davis, 2006). Al igual que con la física cualitativa, un agente puede avanzar mucho, por así decirlo, sin recurrir a una representación métrica completa.

Razonamiento psicológico

Razonamiento psicológico: El razonamiento psicológico implica el desarrollo de una *psicología* operativa para que los agentes artificiales la utilicen en el razonamiento sobre sí mismos y sobre otros agentes. Esto se basa con frecuencia en la llamada psicología popular, la teoría que se cree que los humanos en general utilizan para razonar sobre sí mismos y otros humanos. Cuando los investigadores de IA proporcionan a sus agentes artificiales teorías psicológicas para razonar sobre otros agentes, las teorías suelen basarse en la descripción del propio diseño lógico de

los agentes por parte de los investigadores. El razonamiento psicológico es actualmente más útil dentro del contexto de la comprensión del lenguaje natural, donde descubrir las intenciones del hablante es de suma importancia.

Minker (2001) recopila trabajos de destacados investigadores en representación del conocimiento, resumiendo 40 años de trabajo en el campo. Las actas de las conferencias internacionales tal como *Principles of Knowledge Representation and Reasoning* ofecen las fuentes más actualizadas para el trabajo en esta área. *Readings in Knowledge Representation* (Brachman y Levesque, 1985) y *Formal Theories of the Commonsense World* (Hobbs y Moore, 1985) son excelentes antologías sobre representación del conocimiento; la primera se centra más en trabajos históricamente importantes en lenguajes y formalismos de representación, la segunda en la acumulación del conocimiento en sí mismo. Davis (1990), Stefik (1995) y Sowa (1999) proporcionan introducciones de libros de texto a la representación del conocimiento, Van Harmelen *et al*. (2007) contribuye con un manual, y Davis y Morgenstern (2004) editaron un número especial del *AI Journal* sobre el tema. Davis (2017) ofrece una encuesta sobre la lógica para el razonamiento del sentido común. La conferencia *Theoretical Aspects of Reasoning About Knowledge* (TARK) cubre aplicaciones de la teoría del conocimiento en IA, economía y sistemas distribuidos.

CAPÍTULO 11

PLANIFICACIÓN AUTOMATIZADA

En el que vemos cómo un agente puede aprovechar la estructura de un problema para construir eficientemente planes de acción complejos.

Planificar un curso de acción es requisito clave para un agente inteligente. La representación adecuada de acciones y estados, y algoritmos correctos facilita esto. En la Sección 11.1, presentamos un lenguaje de representación **factorizado** general para problemas de planificación que representan de manera natural y sucinta una amplia variedad de dominios, puede escalar eficientemente a problemas grandes y no requiere heurísticas *ad hoc* para un nuevo dominio. La Sección 11.4 extiende el lenguaje de representación para admitir acciones jerárquicas, lo que nos lleva a abordar problemas más complejos. Cubrimos algoritmos eficientes para la planificación en la Sección 11.2, y heurísticas para ellos en la 11.3. En la Sección 11.5 tenemos en cuenta dominios parcialmente observables y no deterministas, y en la 11.6 extendemos de nuevo el lenguaje para cubrir problemas de programación con restricciones de recursos. Esto nos acerca a los planificadores usados en el mundo real para planificar y programar operaciones de naves espaciales, fábricas y campañas militares. La Sección 11.7 analiza la efectividad de estas técnicas.

11.1 Definición de planificación clásica

Planificación clásica

La **planificación clásica** es la tarea de encontrar una secuencia de acciones para lograr un objetivo en un entorno discreto, determinista, estático y por completo observable. Hemos visto dos enfoques para esta tarea: el agente de resolución de problemas del Capítulo 3 y el agente lógico proposicional híbrido del Capítulo 7. Ambos comparten dos limitaciones: primero, requieren heurísticas específicas para cada nuevo dominio: una función de evaluación heurística para la búsqueda y código escrito a mano para el agente híbrido del mundo wumpus; segundo, necesitan representar de modo evidente un espacio de estados exponencialmente grande. Por ejemplo, en el modelo lógico proposicional del mundo wumpus, el axioma para mover un paso adelante debía repetirse para las cuatro orientaciones del agente, T pasos de tiempo y n^2 ubicaciones actuales.

PDDL

En respuesta a estas limitaciones, los investigadores en planificación han invertido en una **representación factorizada** usando la familia de lenguajes **PDDL** (**L**enguaje de **D**efinición de **D**ominio de **P**lanificación (Ghallab *et al*., 1998), que nos permite expresar todas las acciones $4Tn^2$ con un solo esquema de acción, y no necesita conocimiento específico del dominio. El PDDL básico maneja dominios de planificación clásica, y las extensiones manejan dominios no clásicos continuos, parcialmente observables, concurrentes y multiagentes. La sintaxis de PDDL se basa en Lisp, pero la traduciremos a una forma que coincida con la notación usada en este libro.

Estado

En PDDL, un **estado** se representa como una conjunción de flujos atómicos concretos. Recuerde que "concreto" significa sin variables, "flujo" significa un aspecto del mundo que cambia con el tiempo, y "atómico concreto", que hay un solo predicado, y si hay argumentos, deben ser constantes. Por ejemplo, $Poor \wedge Unknown$ podría representar el estado de un

agente desafortunado, y $At(Truck_1,Melbourne) \wedge At(Truck_2,Sydney)$ podría representar un estado en un problema de entrega de paquetes. PDDL usa **semántica de base de datos**: la suposición de mundo cerrado significa que cualquier flujo que no se mencione es falso, y la suposición de nombres únicos significa que $Truck_1$ y $Truck_2$ son distintos.

Los siguientes flujos *no* están permitidos en un estado: $At(x, y)$ (porque tiene variables), $\neg Poor$ (porque es negación), y $At(Spouse(Ali),Sydney)$ (porque usa un símbolo de función, *Spouse*). Cuando sea conveniente, pensaremos en la conjunción de flujos como un *conjunto* de flujos.

Un **esquema de acción** representa una familia de acciones concretas. Por ejemplo, aquí hay un esquema de acción para volar un avión de un lugar a otro: Esquema de acción

Action(*Fly*(*p, from, to*),
PRECOND:$At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$
EFFECT:$\neg At(p, from) \wedge At(p, to)$)

El esquema consta del nombre de la acción, una lista de todas las variables utilizadas en el esquema, una **precondición** y un **efecto**. La precondición y el efecto son cada uno conjunciones de literales (sentencias atómicas positivas o negadas). Podemos elegir constantes para instanciar las variables, dando como resultado una acción concreta(libre de variables): Precondición Efecto

Action(*Fly*(P_1, *SFO, JFK*),
PRECOND:$At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK)$
EFFECT:$\neg At(P_1, SFO) \wedge At(P_1, JFK)$)

Una acción concreta a es **aplicable** en un estado s si s implica la precondición de a; es decir, si cada literal positivo en la precondición está en s y cada literal negado no lo está.

El **resultado** de ejecutar la acción aplicable a en el estado s se define como estado s' representado por el conjunto de flujos formados iniciando con s, eliminando los flujos que aparecen como literales negativos en los efectos de la acción (llamados **lista de eliminación** o $DEL(a)$), y agregando los flujos literales positivos en los efectos de la acción (**lista de adición** o ADD(a)): Lista de eliminación Lista de adición

$$\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a) . \qquad (11.1)$$

Por ejemplo, con la acción $Fly(P_1,SFO,JFK)$, eliminaríamos el flujo $At(P_1,SFO)$ y agregaríamos el flujo $At(P_1, JFK)$.

Un conjunto de esquemas de acción sirve como definición de un *dominio* de planificación. Un *problema* específico dentro del dominio se define con la adición de un estado inicial y un objetivo. El **estado inicial** es una conjunción de flujos concretos (introducidos con la palabra clave *Inicio* en la Figura 11.1). Como con todos los estados, se usa la suposición de mundo cerrado, lo que significa que cualquier átomo que no se mencione es falso. El **objetivo** (introducido con *Goal*) es similar a una precondición: una conjunción de literales (positivos o negativos) que pueden contener variables. Así, el objetivo $At(C_1,SFO) \wedge \neg At(C_2,SFO) \wedge At(p,SFO)$ se refiere a cualquier estado en el que la carga C_1 esté en *SFO* pero C_2 no lo esté, y en el que haya un avión en *SFO*.

11.1.1 Ejemplo de dominio: Transporte de carga aérea

La Figura 11.1 ilustra un problema de transporte aéreo de carga que implica cargar, descargar y transportar mercancía por avión. El problema se define con tres acciones: *Cargar* (*Load*), *Descargar* (*Unload*) y *Volar*(*Fly*). Las acciones afectan dos predicados: $At(c,p)$, que la carga c está en el avión p, y $At(x, a)$, que el objeto x (avión o carga) está en el aeropuerto a. Se debe tener

$Init(At(C1, SFO) \land At(C2, JFK) \land At(P1, SFO) \land At(P2, JFK)$
$\quad \land Cargo(C1) \land Cargo(C2) \land Plane(P1) \land Plane(P2)$
$\quad \land Airport(JFK) \land Airport(SFO))$
$Goal(At(C1, JFK) \land At(C2, SFO))$
$Action(Load(c, p, a),$
PRECOND: $At(c, a) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $\neg At(c, a) \land In(c, p))$
$Action(Unload(c, p, a),$
PRECOND: $In(c, p) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $At(c, a) \land \neg In(c, p))$
$Action(Fly(p, from, to),$
PRECOND: $At(p, from) \land Plane(p) \land Airport(from) \land Airport(to)$
EFFECT: $\neg At(p, from) \land At(p, to))$

Figura 11.1 Descripción de un problema de planificación del transporte aéreo de carga en PDDL.

cuidado para asegurar que los predicados *At* sean correctos. Cuando un avión vuela de entre aeropuertos, toda la carga va dentro del avión. En lógica de primer orden, sería fácil cuantificar sobre todos los objetos dentro del avión. Pero PDDL no tiene un cuantificador universal, por ello necesitamos una solución diferente. El enfoque que usamos es afirmar que una pieza de carga deja de estar en *At* en cualquier lugar cuando está en un avión; la carga sólo se convierte en *At* cuando es descarga en el nuevo aeropuerto. Entonces, *At* significa "disponible para usar en una ubicación determinada". El siguiente plan es una solución al problema:

$[Load(C1,P1,SFO),Fly(P1,SFO,JFK),Unload(C1,P1,JFK),$
$\quad Load(C2,P2,JFK),Fly(P2,JFK,SFO),Unload(C2,P2,SFO)].$

11.1.2 Ejemplo de dominio: El problema de la llanta de repuesto

Problema: cambiar la llanta desinflada (Figura 11.2). El objetivo es cambiar correctamente una llanta desinflada por la de repuesto, donde el estado inicial tiene una llanta desinflada en el eje y una de repuesto en la cajuela. Para simplificar, nuestra versión del problema es abstracta, sin tuercas atoradas ni líos. Tenemos cuatro acciones: sacar el repuesto de la cajuela, quitar del eje la llanta desinflada, montar la de repuesto en el eje y dejar el automóvil en la calle durante la noche. Suponemos que al dejarlo fuera en un vecindario peligroso, la consecuencia será que el robo de las llantas. $[Remove(Flat,Axle),Remove(Spare,Trunk),PutOn(Spare,Axle)]$ es una solución al problema.

11.1.3 Ejemplo de dominio: El mundo de los bloques

Uno de los dominios de planificación más famosos es el **mundo de bloques**. Se trata de un conjunto de bloques en forma de cubo colocados de modo arbitrario en una mesa.[1] Los bloques se pueden apilar, pero sólo uno puede estar directamente encima de otro. Un brazo robótico puede levantar un bloque y moverlo a otra posición, ya sea en la mesa o encima de otro bloque, pero sólo puede levantar un bloque a la vez, no puede levantar un bloque que tenga otro encima. Un objetivo típico es colocar el bloque *A* sobre *B* y el bloque *B* sobre *C* (ver Figura 11.3).

[1] El mundo de bloques usado en investigación de planificación es más simple que la versión de SHRDLU (p. 20).

Init(*Tire*(*Flat*) ∧ *Tire*(*Spare*) ∧ *At*(*Flat*,*Axle*) ∧ *At*(*Spare*,*Trunk*))
Goal(*At*(*Spare*,*Axle*))
Action(*Remove*(*obj*, *loc*),
  PRECOND: *At*(*obj*, *loc*)
  EFFECT: ¬*At*(*obj*, *loc*) ∧ *At*(*obj*,*Ground*))
Action(*PutOn*(*t*, *Axle*),
  PRECOND: *Tire*(*t*) ∧ *At*(*t*,*Ground*) ∧ ¬ *At*(*Flat*,*Axle*) ∧ ¬*At*(*Spare*,*Axle*)
  EFFECT: ¬ *At*(*t*,*Ground*) ∧ *At*(*t*,*Axle*))
Action(*LeaveOvernight*,
  PRECOND:
  EFFECT: ¬ *At*(*Spare*,*Ground*) ∧ ¬ *At*(*Spare*,*Axle*) ∧ ¬ *At*(*Spare*,*Trunk*)
    ∧ ¬ *At*(*Flat*,*Ground*) ∧ ¬ *At*(*Flat*,*Axle*) ∧ ¬ *At*(*Flat*, *Trunk*))

Figura 11.2 El problema de la llanta de repuesto.

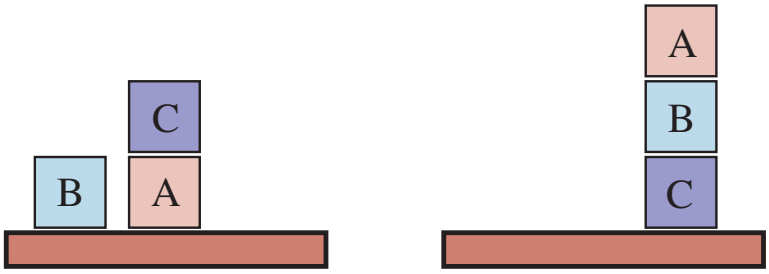


Figura 11.3 Diagrama del problema del mundo de los bloques en la Figura 11.4.

Init(*On*(*A*,*Table*) ∧ *On*(*B*,*Table*) ∧ *On*(*C*,*A*)
  ∧ *Block*(*A*) ∧ *Block*(*B*) ∧ *Block*(*C*) ∧ *Clear*(*B*) ∧ *Clear*(*C*) ∧ *Clear*(*Table*))
Goal(*On*(*A*,*B*) ∧ *On*(*B*,*C*))
Action(*Move*(*b*,*x*,*y*),
  PRECOND: *On*(*b*,*x*) ∧ *Clear*(*b*) ∧ *Clear*(*y*) ∧ *Block*(*b*) ∧ *Block*(*y*) ∧
    (*b*≠*x*) ∧ (*b*≠*y*) ∧ (*x*≠*y*),
  EFFECT: *On*(*b*,*y*) ∧ *Clear*(*x*) ∧ ¬*On*(*b*,*x*) ∧ ¬*Clear*(*y*))
Action(*MoveToTable*(*b*,*x*),
  PRECOND: *On*(*b*,*x*) ∧ *Clear*(*b*) ∧ *Block*(*b*) ∧ *Block*(*x*),
  EFFECT: *On*(*b*,*Table*) ∧ *Clear*(*x*) ∧ ¬*On*(*b*,*x*))

Figura 11.4 Un problema de planificación en el mundo de los bloques: construir una torre de tres bloques. Una solución es la secuencia [*MoveToTable*(*C*,*A*),*Move*(*B*,*Table*,*C*),*Move*(*A*,*Table*,*B*)].

Usamos $On(b, x)$ para indicar que el bloque b está sobre x, donde x es otro bloque o la mesa. La acción para mover b de la parte superior de x a la parte superior de y será $Move(b,x,y)$. Ahora, una precondición para mover b es que ningún otro bloque esté sobre él. En lógica de primer orden, esto sería $\neg\exists x\ On(x,b)$ o, alternativamente, $\forall x\ \neg On(x, b)$. PDDL básico no permite cuantificadores, así que en su lugar introducimos un predicado $Clear(x)$ que es verdadero si no hay nada sobre x. (Vea la descripción completa del problema en la Figura 11.4).

La acción *Move* mueve un bloque b de x a y si b y y están despejados. Después de realizar el movimiento, b sigue despejado, pero y no lo está. Un primer intento del esquema mover es

$Action(Move(b, x, y),$
PRECOND:$On(b, x) \wedge Clear(b) \wedge Clear(y),$
EFFECT:$On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$.

Lo malo es que esto no mantiene *Clear* adecuadamente cuando x o y es la mesa. Si x es la mesa, esta acción tiene el efecto *Clear(Table)*, pero la mesa no debería volverse despejada; y si y=*Table*, tiene la condición previa *Clear(Table)*, pero la mesa no tiene que estar despejada para que podamos mover un bloque sobre ella. Para solucionar esto, hacemos dos cosas. Primero, introducimos otra acción para mover un bloque b de x a la mesa:

$Action(MoveToTable(b,x),$
PRECOND:$On(b, x) \wedge Clear(b),$
EFFECT:$On(b,Table) \wedge Clear(x) \wedge \neg On(b, x))$.

Segundo, tomamos la interpretación de $Clear(x)$ como "hay un espacio despejado en x para sostener un bloque". Bajo esta interpretación, $Clear(Table)$ siempre será verdadero. El único problema es que nada impide que el planificador use $Move(b,x,Table)$ en lugar de $MoveToTable(b,x)$. Podríamos vivir con este problema, conducirá a un espacio de búsqueda mayor de lo necesario, pero no llevará a respuestas incorrectas, o podríamos introducir el predicado *Block* y agregar $Block(b) \wedge Block(y)$ a la condición previa de *Move*, como se ve en la Figura 11.4.

11.2 Algoritmos para planificación clásica

La descripción de un problema de planificación ofrece una forma obvia de búsqueda de un objetivo desde el estado inicial por el espacio de estados. Una ventaja sutil de la representación declarativa de los esquemas de acción es que también podemos buscar hacia atrás desde el objetivo, buscando el estado inicial (la Figura 11.5 compara las búsquedas hacia adelante y hacia atrás). Una tercera posibilidad es traducir la descripción del problema en un conjunto de sentencias lógicas, a las cuales aplicamos un algoritmo de inferencia lógica para dar con una solución.

11.2.1 Búsqueda de espacio de estados hacia adelante para planificación

Podemos resolver problemas de planificación aplicando alguno de los algoritmos de búsqueda heurística de los Capítulos 3 o 4. Los estados en este espacio son estados concretos, donde cada flujo es verdadero o falso. El objetivo es un estado que tiene todos los flujos positivos en el objetivo del problema y ninguno negativo. Las acciones aplicables en un estado, $Actions(s)$, son instancias concretas de los esquemas de acción, es decir, acciones donde las variables han sido reemplazadas por valores constantes.

Para determinar las acciones aplicables unificamos el estado actual con las precondiciones de cada esquema de acción. Para cada unificación exitosa en una sustitución, aplicamos la sustitución al esquema de acción para obtener una acción concreta sin variables. (Es requisito de los esquemas de acción que cualquier variable en el efecto también aparezca en la precondición; así garantizamos que no queden variables después de la sustitución).

Cada esquema se unifica de diversas maneras. En el ejemplo de la llanta de repuesto (página 346), la acción *Remove* tiene la precondición *At*(*obj,loc*), que coincide con el estado inicial de dos maneras, lo que resulta en las dos sustituciones {*obj*/*Flat*,*loc*/*Axle*} *y* {*obj*/*Spare*,*loc*/*Trunk*}; aplicar estas sustituciones produce dos acciones concretas. Si una acción tiene múltiples literales en la precondición, entonces cada una coincidirá potencialmente con el estado actual de varias maneras.

Al principio, parece que el espacio de estados podría ser demasiado grande para muchos problemas. Considere un problema de carga aérea con 10 aeropuertos, donde cada uno tiene de inicio 5 aviones y 20 piezas de carga. El objetivo es mover toda la carga del aeropuerto *A* al aeropuerto *B*. Hay una solución de 41 pasos para el problema: cargar las 20 piezas en un avión en *A* (*At*(P_1,*A*)), volar (*Fly*) a *B* y descargar todas las piezas.

Hallar esta solución en apariencia directa será difícil porque el factor de ramificación promedio es enorme: cada uno de los 50 aviones puede volar a otros 9 aeropuertos, y cada una de las 200 piezas puede ser descargada (si está cargada) o cargada en cualquier avión en su aeropuerto (si está descargada). Así, en cualquier estado hay un mínimo de 450 acciones (si todas las cargas están en aeropuertos sin aviones) y un máximo de 10 450 (si todas las piezas y aviones están en el

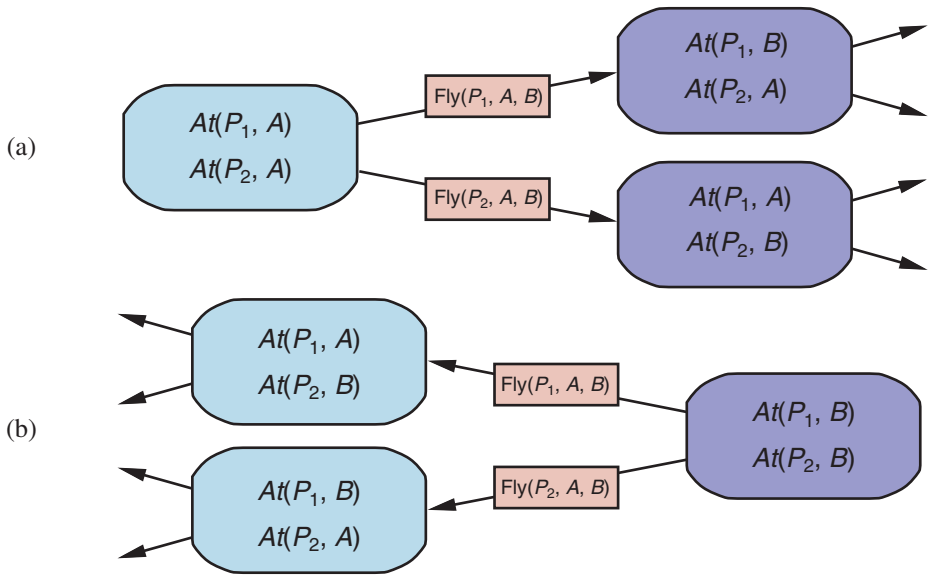


Figura 11.5 Dos enfoques para buscar un plan. (a) Búsqueda hacia adelante (progresión) a través del espacio de estados fundamentales, comenzando en el estado inicial y utilizando las acciones del problema para buscar hacia adelante un miembro del conjunto de estados objetivo. (b) Búsqueda hacia atrás (regresión) a través de descripciones de estados, comenzando en el objetivo y utilizando la inversa de las acciones para buscar hacia atrás el estado inicial.

mismo aeropuerto). En promedio, hay casi 2 000 acciones posibles por estado, por ello el grafo de búsqueda hasta la profundidad de la solución de 41 pasos tiene unos $2\,000^{41}$ nodos.

Claramente, aun esta instancia de problema relativamente pequeña es desalentadora sin heurística precisa. Aunque muchas aplicaciones del mundo real de la planificación han dependido de heurísticas específicas del dominio, resulta (como se ve en la Sección 11.3) que se pueden derivar automáticamente heurísticas fuertes independientes del dominio; eso es lo que hace factible la búsqueda hacia adelante.

11.2.2 Búsqueda hacia atrás para la planificación

Búsqueda de regresión

Acción relevante

En la búsqueda hacia atrás (o **búsqueda de regresión**) iniciamos en el objetivo y aplicamos las acciones hacia atrás hasta encontrar una secuencia de pasos que llegue al estado inicial. En cada paso consideramos **acciones relevantes** (a diferencia de la búsqueda hacia adelante, que considera acciones **aplicables**). Esto reduce de modo significativo el factor de ramificación, en especial en dominios con muchas acciones posibles.

Una acción relevante es aquella con un efecto que se **unifica** con uno de los literales del objetivo, pero sin efecto que niegue alguna parte del objetivo. Por ejemplo, con el objetivo $\neg Poor \wedge Famous$, una acción con el único efecto Famoso (*Famous*) sería relevante, pero una con el efecto $Poor \wedge Famous$ no se considera relevante: aunque esa acción podría usarse en algún momento en el plan (para establecer *Famous*), no puede aparecer en *este* punto del plan porque entonces *Poor* aparecería en el estado final.

¿Qué significa aplicar una acción con dirección hacia atrás? Dado un objetivo g y una acción a, la **regresión** desde g sobre a nos da una descripción del estado g' cuyos literales positivos y negativos están dados por

Regresión

$$POS(g') = (POS(g) - ADD(a)) \cup POS(Precond(a))$$
$$NEG(g') = (NEG(g) - DEL(a)) \cup NEG(Precond(a)).$$

Es decir, las precondiciones deben haber sido verdaderas antes, o de lo contrario la acción no podría haberse ejecutado, pero los literales positivos/negativos que fueron agregados/eliminados por la acción no necesariamente tenían que ser verdaderos antes.

Estas ecuaciones son directas para literales concretos, pero se requiere cuidado cuando hay variables en g y a. Por ejemplo, suponga que el objetivo es una entrega específica de carga a *SFO*: $At(C_2, SFO)$. El esquema de acción *Unload* tiene el efecto $At(c,a)$. Si unificamos eso con el objetivo, obtenemos la sustitución $\{c/C_2, a/SFO\}$; al aplicar la sustitución al esquema nos da un nuevo esquema que captura la idea de usar cualquier avión que esté en *SFO*:

$$Action(Unload(C_2, p', SFO),$$
$$\text{PRECOND}: In(C_2, p') \wedge At(p', SFO) \wedge Cargo(C_2) \wedge Plane(p') \wedge Airport(SFO)$$
$$\text{EFFECT}: At(C_2, SFO) \wedge \neg In(C_2, p')) .$$

Aquí reemplazamos p con una nueva variable llamada p'. Esto es una instancia de **estandarizar aparte** nombres de variables para que no haya conflicto entre diferentes variables que tienen el mismo nombre (ver página 284). La descripción del estado regresado da un nuevo objetivo:

$$g' = In(C_2, p') \wedge At(p', SFO) \wedge Cargo(C_2) \wedge Plane(p') \wedge Airport(SFO) .$$

Como otro ejemplo, consideremos el objetivo de poseer un libro con un número de ISBN específico: $Own(9780134610993)$. Dados un trillón de ISBN de 13 dígitos y el único esquema de acción

$$A = Action(Buy(i), \text{PRECOND}: ISBN(i), \text{EFFECT}: Own(i)).$$

una búsqueda hacia adelante sin una heurística tendría que comenzar a enumerar los 10 mil millones de acciones concretas *Buy*. Pero con la búsqueda hacia atrás, unificaríamos el objetivo

Own(9780134610993) con el efecto *Own(i')*, obteniendo la sustitución θ={i'/9780134610993}. Luego regresaríamos sobre la acción Subst(θ,A) para obtener la descripción del estado predecesor ISBN(9780134610993). Esto es parte del estado inicial, por lo que tenemos una solución y hemos terminado, habiendo considerado sólo una acción, no un trillón.

De manera más formal, suponga una descripción de objetivo g que contiene un literal de objetivo g_i y un esquema de acción A. Si A tiene un literal de efecto e'_j donde $Unify(g_i, e'_j) = \theta$, donde definimos $A' = SUST(\theta, A)$ y si no hay ningún efecto en A' que sea la negación de un literal en g, entonces A' es una acción relevante hacia g.

Para la mayoría de los dominios de problemas, la búsqueda hacia atrás mantiene el factor de ramificación más bajo que la búsqueda hacia adelante. Pero, que la búsqueda hacia atrás use estados con variables en lugar de estados concretos dificulta encontrar buenas heurísticas. Esa es la razón principal por la que la mayoría de los sistemas de hoy favorecen la búsqueda hacia adelante.

11.2.3 Planificación como satisfacción booleana

En la Sección 7.7.4, mostramos cómo algunas reescrituras de axiomas inteligentes podrían convertir un problema del mundo de wumpus en uno de satisfacción de lógica proposicional que sería entregado a un solucionador de satisfacción eficiente. Los planificadores basados en SAT, como SATPLAN, operan al traducir una descripción del problema PDDL en forma proposicional. La traducción implica los pasos:

- Proposicionalizar las acciones: para cada esquema de acción, formar proposiciones concretas sustituyendo constantes por cada una de las variables. Luego, en lugar de un solo esquema *Unload*(*c,p,a*), tendríamos proposiciones de acción separadas para cada combinación de carga, avión y aeropuerto (escritas con subíndices), y para cada paso de tiempo (escritas como superíndice).
- Agregar axiomas de exclusión de acción diciendo que no pueden ocurrir dos acciones al mismo tiempo, por ejemplo, $\neg(FlyP_1SFOJFK^1 \wedge FlyP_1SFOBUH^1)$.
- Agregar axiomas de precondición: para cada acción concreta A^t, agregar el axioma $A^t \Rightarrow \text{PRE}(A)^t$, es decir, si se cumple una acción en el tiempo t, las precondiciones deben haber sido verdaderas. Por ejemplo, $FlyP_1SFOJFK^1 \Rightarrow At(P_1,SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK)$.
- Definir el estado inicial: afirmar F^0 para cada flujo F en el estado inicial del problema, y $\neg F^0$ para cada flujo no mencionado en el estado inicial.
- Proposicionalizar el objetivo: éste se convierte en una disyunción sobre todas sus instancias concretas, donde las variables se reemplazan por constantes. Por ejemplo, el objetivo de tener el bloque A sobre otro bloque, $On(A, x) \wedge Block(x)$ en un mundo con objetos A, B y C, se reemplazaría por el objetivo

  $$(On(A, A) \wedge Block(A)) \vee (On(A, B) \wedge Block(B)) \vee (On(A,C) \wedge Block(C)) .$$

- Agregar axiomas de estado sucesor: para cada flujo F, agregar un axioma de la forma

  $$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t),$$

  donde *ActionCausesF* representa una disyunción de todas las acciones concretas que agregan F, y *ActionCausesNotF* representa una disyunción de todas las acciones concretas que eliminan F.

La traducción resultante suele ser más larga que el PDDL original, pero la eficiencia de los solucionadores SAT modernos con frecuencia lo compensa.

11.2.4 Enfoques Clásicos Adicionales para la Planificación

Los tres enfoques previos no son los únicos probados en los 50 años de historia de la planificación automatizada. Aquí la descripción breve de otros.

Grafo de planificación

Un enfoque llamado Graphplan usa una estructura de datos especializada, un **grafo de planificación**, para codificar restricciones sobre cómo se relacionan las acciones con sus precondiciones y efectos, y en qué cosas son mutuamente excluyentes.

Cálculo de situaciones

El método de **cálculo de situaciones** describe problemas de planificación en lógica de primer orden. Usa axiomas de estado sucesor igual que SATPLAN, pero esta lógica da mayor flexibilidad y axiomas más concisos. En general, el enfoque ha contribuido a la comprensión teórica de la planificación, pero no ha logrado impacto en aplicaciones prácticas, quizá porque los demostradores de primer orden no se han desarrollado como los programas de satisfacibilidad proposicional.

Es posible codificar un problema de planificación acotado (es decir, el problema de encontrar un plan de longitud k) como un **problema de satisfacción de restricciones** (CSP). La codificación es similar a la codificación de un problema SAT (Sección 11.2.3), con una simplificación vital: en cada paso de tiempo solo requerimos una variable, $Action^t$, cuyo dominio es el conjunto de acciones posibles. Ya no precisamos una variable para cada acción, ni axiomas de exclusión de acción.

Todos los enfoques vistos hasta ahora construyen planes *totalmente ordenados* que radican en secuencias estrictamente lineales de acciones. Pero si un problema de carga aérea debe cargar 30 paquetes en un avión y 50 en otro, parece inútil decretar un orden lineal específico de las 80 acciones de carga.

Planificación de orden parcial

La **planificación de orden parcial**, alternativa, representa un plan como un grafo en lugar de una secuencia lineal: cada acción es un nodo en el grafo, y para cada precondición de la acción hay una arista desde otra acción (o desde el estado inicial) que indica que la acción predecesora establece la precondición. Así, podríamos tener un plan de orden parcial que dice que las acciones *Remove*(*Spare*,*Trunk*) y *Remove*(*Flat*,*Axle*) deben anteceder *PutOn*(*Spare*,*Axle*), pero sin decir cuál de las acciones de *Remove* debería ser primero. Buscamos en el espacio de planes y no en estados del mundo, insertando acciones para satisfacer condiciones.

Entre 1980 y 1990, la planificación de orden parcial era lo mejor para manejar problemas de planificación con subproblemas independientes. Para el año 2000, los planificadores de búsqueda hacia adelante habían desarrollado excelentes heurísticas para descubrir con eficiencia los subproblemas independientes para los que se diseñó la planificación de orden parcial. Además, SATPLAN aprovechó la ley de Moore: una proposicionalización muy grande en 1980, hoy pequeña, porque ahora las computadoras tienen 10 000 veces más memoria. Como resultado, los planificadores de orden parcial no son competitivos en problemas de planificación clásica del todo automatizados.

No obstante, la planificación de orden parcial aún es la tecnología elegida para algunas tareas específicas, como programación de operaciones y planificación de orden parcial con heurísticas específicas del dominio. Muchos de estos sistemas usan bibliotecas de planes de alto nivel, como se describe en la Sección 11.4.

La planificación de orden parcial también se usa en dominios donde es sustancial que los humanos entiendan los planes. Por ejemplo, los planes operativos para naves espaciales y exploradores de Marte se generan con planificadores de orden parcial, y luego son verificados por operadores humanos antes de ser cargados en los vehículos para su ejecución. El enfoque de refinamiento de planes facilita que los humanos comprendan lo que hacen los algoritmos de planificación y verifiquen que los planes sean correctos antes su ejecución.

11.3 Heurísticas para la planificación

Las búsquedas hacia adelante y hacia atrás no son eficientes sin una buena función heurística. Recordemos del Capítulo 3 que una función heurística $h(s)$ estima la distancia desde un estado s hasta el objetivo, y que, si podemos derivar una heurística **admisible** para esta distancia, una que no sobreestime, entonces usaremos la búsqueda A* para encontrar soluciones óptimas.

Por definición, no hay forma de analizar un estado atómico, por ello se requiere la astucia de un analista (en general, humano) para definir heurísticas específicas buenas del dominio para problemas de búsqueda con estados atómicos. Como la planificación usa una representación factorizada para estados y acciones, esto posibilita definir heurísticas independientes del dominio.

Recuerde que una heurística admisible se deriva al definir un **problema flexible**, más fácil de resolver. El costo exacto de facilitar la solución de este problema deviene en heurística para el problema original. Un problema de búsqueda es un grafo donde los nodos son estados y las aristas acciones. El problema es encontrar un camino que conecte el estado inicial con un estado objetivo. Hay dos formas principales para flexibilizar este problema y facilitarlo: agregar más aristas al grafo, lo que facilita encontrar un camino, o agrupar varios nodos juntos y formar una abstracción del espacio de estados que tiene menos estados, así es más fácil de buscar.

Primero examinamos las heurísticas que suman aristas al grafo. Quizá la más simple sea la **heurística de ignorar precondiciones**, que elimina precondiciones de las acciones. Toda acción se vuelve aplicable en cada estado, y cualquier flujo objetivo único se alcanza en un paso (si hay acciones aplicables; si no las hay, el problema es imposible). Esto casi implica que el número de pasos para resolver el problema flexible es el número de metas insatisfechas; casi, pero no del todo, porque (1) alguna acción puede lograr múltiples metas y (2) unas acciones pueden deshacer los efectos de otras.

Heurística de ignorar precondiciones

En muchos problemas, una heurística precisa se logra si se considera (1) e ignora (2). Primero, volvemos flexibles las acciones al eliminar las precondiciones y los efectos, excepto los que son literales en el objetivo. Luego, contamos el número mínimo de acciones requeridas para que la unión de los efectos de esas acciones satisfaga el objetivo. Esto es una instancia del **problema de cobertura de conjuntos**.

Problema de cobertura de conjuntos

Pequeña consideración: el problema de cobertura de conjuntos es NP-duro. Por suerte, un simple algoritmo voraz garantiza devolver un conjunto de cobertura cuyo tamaño está dentro de un factor logarítmico de n de la verdadera cobertura mínima, donde n es el número de literales en el objetivo. Por desgracia, el algoritmo voraz no garantiza admisibilidad.

Es posible ignorar sólo precondiciones *seleccionadas* de las acciones. Piense en el rompecabezas de cajas deslizantes (8-puzzle o 15-puzzle) de la Sección 3.2. Podríamos codificarlo como un problema de planificación que involucra fichas con un solo esquema *Slide*:

$$\begin{aligned}&Action(Slide(t,s_1,s_2),\\&\quad \text{PRECOND:}On(t,s_1) \wedge Tile(t) \wedge Blank(s_2) \wedge Adjacent(s_1,s_2)\\&\quad \text{EFFECT:}On(t,s_2) \wedge Blank(s_1) \wedge \neg On(t,s_1) \wedge \neg Blank(s_2))\end{aligned}$$

Como en la Sección 3.6, si eliminamos las precondiciones $Blank(s_2) \wedge Adjacent(s_1,s_2)$ entonces cualquier ficha puede moverse en una acción a cualquier espacio y obtenemos la heurística del número de fichas mal colocadas. Si eliminamos sólo la precondición $Blank(s_2)$ entonces obtenemos la heurística de la distancia de Manhattan. Es fácil ver cómo estas heurísticas podrían derivarse en automático a partir de la descripción del esquema de acción. La facilidad para manipular estos esquemas es la gran ventaja de la representación factorizada de los problemas de planificación, en comparación con la representación atómica de los problemas de búsqueda.

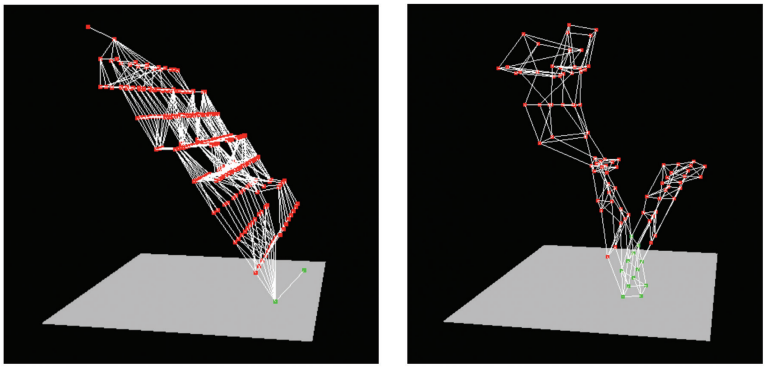


Figura 11.6 Dos espacios de estados a partir de problemas de planificación con la heurística de ignorar las listas de eliminación. La altura sobre el plano inferior es la puntuación heurística de un estado; los estados en el plano inferior son metas. No hay mínimos locales, por ello buscar el objetivo es sencillo. De Hoffmann (2005).

Ignorar las listas de eliminación

Otra posibilidad es la heurística **ignorar listas de eliminación**. Suponga que todos los objetivos y precondiciones contienen sólo literales positivos.[2] Queremos crear una versión relajada del problema original más fácil de resolver, donde la longitud de la solución sirva como una buena heurística. Podemos lograrlo al eliminar las listas de eliminación de todas las acciones (es decir, eliminar todos los literales negativos de los efectos). Esto posibilita avanzar de manera monótona hacia el objetivo; ninguna acción deshará el progreso realizado por otra acción. Encontrar la solución óptima a este problema flexible sigue siendo NP-duro, pero una solución aproximada se encuentra en tiempo polinomial mediante ascenso de colinas.

La Figura 11.6 muestra parte del espacio de estados para dos problemas de planificación usando heurística de ignorar las listas de eliminación. Los puntos representan estados, las aristas acciones y la altura de cada punto sobre el plano inferior representa el valor heurístico. Los estados en el plano inferior son soluciones. En ambos problemas, hay un amplio camino hacia el objetivo. No hay callejones sin salida, por ello no hay necesidad de retroceder; una búsqueda simple de ascenso de colinas encontrará fácilmente una solución a los problemas (pero puede no ser una solución óptima).

11.3.1 Podas independientes del dominio

Las representaciones factorizadas muestran que muchos estados son sólo variantes de otros. Así, suponga que tenemos doce bloques en una mesa, y el objetivo es poner el bloque A encima de una torre de tres bloques. El primer paso en una solución es colocar un bloque x encima del bloque y (donde x, y y A son todos diferentes). Luego ponemos A encima de x y ahí terminamos. Hay 11 opciones para x, y dado x, 10 opciones para y, y así 110 estados que considerar. Pero todos estos estados son simétricos: elegir uno sobre otro no hace diferencia; así, un planificador solo debería considerar uno de ellos. Este es el proceso de **reducción de simetría**: podamos de la

Reducción de simetría

[2] Muchos problemas se escriben con esta convención; para los no escritos así, reemplace cada literal negativo $\neg P$ en una meta o precondición con un literal positivo, P', y mude el estado inicial y los efectos de la acción en resultado.

consideración todas las ramas simétricas del árbol de búsqueda, excepto una. Para muchos dominios, esto marca la diferencia entre un problema intratable y una resolución eficiente.

Otra posibilidad es realizar podas hacia adelante, tomando el riesgo de quizás eliminar una solución óptima, con el fin de enfocar la búsqueda en ramas prometedoras. Podemos definir una acción preferida como sigue: primero, definir una versión flexible del problema y resolverlo para obtener un plan flexible. Luego, una **acción preferida** es o bien un paso del **plan flexible**, o logra alguna precondición de éste.

Acción preferida

A veces es posible resolver un problema de manera eficiente al reconocer que las interacciones negativas pueden descartarse. Decimos que un problema tiene **subobjetivos serializables** si existe un orden de subobjetivos tal que el planificador los logre en ese orden sin tener que deshacer ninguno de los subobjetivos logrados. Por ejemplo, en el mundo de bloques, si el objetivo es construir una torre (por ejemplo, *A* sobre *B*, que a su vez está sobre *C*, y ésta sobre *Table*, como en la Figura 11.3 de p. 347), entonces los subobjetivos son serializables de abajo hacia arriba: si primero logramos *C* sobre *Table*, nunca tendremos que deshacerlo mientras buscamos lograr otros subobjetivos. Un planificador que use el truco de abajo hacia arriba puede resolver cualquier problema en el mundo de bloques sin retroceso (aunque no siempre encontrará el plan más corto). Otro ejemplo, si hay una habitación con *n* interruptores de luz, cada uno para una lámpara, y el objetivo es tenerlas todas encendidas, entonces no tenemos que considerar permutaciones del orden; podríamos restringirnos arbitrariamente a planes que enciendan los interruptores en, por ejemplo, orden ascendente.

Subobjetivos serializables

Para el planificador Remote Agent que comandó la nave espacial Deep Space One de la NASA, se determinó que las proposiciones involucradas en el comando de una nave espacial son serializables. Esto quizá no sea tan sorprendente, porque una nave espacial está *diseñada* por ingenieros para ser lo más fácil posible de controlar (sujeto a otras restricciones). Al aprovechar el orden serializado de los objetivos, el planificador de esta nave pudo eliminar la mayor parte de la búsqueda. Esto significó que era lo suficientemente rápido como para controlar la nave espacial en tiempo real, algo que antes se consideraba imposible.

11.3.2 Abstracción de estados en la planificación

Un problema relajado nos deja con un problema de planificación simplificado sólo para calcular el valor de la función heurística. Muchos problemas de planificación tienen 10^{100} estados o más, y relajar las acciones no hace nada para reducir el número de estados, esto significa que aún puede ser costoso calcular la heurística. Entonces, ahora miramos relajaciones que disminuyen el número de estados formando una **abstracción de estado**: un mapeo de estados de muchos a uno en la representación concreta del problema a la representación abstracta.

Abstracción de estado

La forma más fácil de abstracción de estado es ignorar algunos flujos. Por ejemplo, piense en un problema de carga aérea con 10 aeropuertos, 50 aviones y 200 paquetes. Cada avión puede estar en uno de 10 aeropuertos y cada paquete en uno de los aviones o descargado en uno de los aeropuertos. Entonces hay $10^{50} \times (50 + 10)^{200} \approx 10^{405}$ estados. Ahora considere un problema particular en ese dominio en el que todos los paquetes están en sólo 5 de los aeropuertos, y todos los paquetes en un aeropuerto dado tienen el mismo destino. Entonces, una abstracción útil del problema es eliminar todos los flujos de *At* excepto los que involucran un avión y un paquete en cada uno de los 5 aeropuertos. Ahora hay sólo $10^5 \times (5 + 10)^5 \approx 10^{11}$ estados. Una solución en este espacio de estados abstracto será más corta que una en el espacio original (*y*, por lo tanto, será una heurística admisible), pero la solución abstracta es fácil de extender a una solución del problema original (al agregar acciones adicionales de *carga* y *descarga*).

Descomposición

Independencia de subobjetivos

Una idea clave en la definición de heurísticas es la **descomposición**: dividir un problema en partes, resolver cada parte de forma independiente y luego combinar las partes. La suposición de **independencia de subobjetivos** es que el costo de resolver una conjunción de subobjetivos se aproxima a la suma de los costos de resolver cada subobjetivo de forma *independiente*. La suposición de independencia de subobjetivos puede ser optimista o pesimista. Es optimista cuando existen interacciones negativas entre los subplanes para cada subobjetivo, como cuando una acción en un subplan elimina un objetivo logrado por otro subplan. Es pesimista, y por ello inadmisible, si los subplanes tienen acciones redundantes; es decir, dos acciones que podrían ser reemplazadas por una sola en el plan combinado.

Suponga que el objetivo es un conjunto de flujos G, que dividimos en subconjuntos disjuntos $G_1, ..., G_n$. Luego encontramos planes óptimos $P_1, ..., P_n$ que resuelven los respectivos subobjetivos. ¿Cuál es una estimación del costo del plan para lograr todos los G? Podemos pensar en cada $\text{COST}(P_i)$ como estimación heurística, y sabemos que, si las combinamos tomando su valor máximo, siempre obtenemos una heurística admisible. Así que $\max_i \text{COST}(P_i)$ es admisible, y a veces es exactamente correcto: podría ser que P_1 logre por casualidad todos los G_i. Pero, en general, la estimación es muy baja. ¿Podríamos sumar los costos en su lugar? Para muchos problemas, esa es una estimación razonable, no admisible. El mejor caso es cuando G_i y G_j son independientes, en el sentido de que los planes para uno no pueden reducir el costo de los planes para el otro. En ese caso, la estimación $\text{COST}(P_i) + \text{COST}(P_j)$ es admisible y más precisa que la estimación máxima.

Es claro que existe un gran potencial para reducir el espacio de búsqueda mediante la formación de abstracciones. El truco está en elegir las abstracciones correctas y usarlas de modo que el costo total —definir una abstracción, realizar una búsqueda abstracta y mapear la abstracción de vuelta al problema original— sea menor que el costo de resolver el problema original. Las técnicas de **bases de datos de patrones** de la Sección 3.6.3 son útiles, porque el costo de crear la base de datos de patrones se puede amortizar sobre múltiples instancias del problema.

Un sistema que usa heurísticas efectivas es FF, o FASTFORWARD (Hoffmann, 2005), un buscador de espacio de estados hacia adelante que, al estimar la heurística con la ayuda de un grafo de planificación, usa la heurística de ignorar las listas de eliminación. FF utiliza una búsqueda de ascenso de colinas (modificada para hacer un registro del plan) con la heurística para encontrar una solución. El algoritmo de ascenso de colinas de FF es no estándar: evita los máximos locales al ejecutar búsqueda en amplitud desde el estado actual hasta encontrar uno mejor. Si falla, FF cambia a búsqueda de mejor primera opción voraz.

11.4 Planificación jerárquica

Los métodos de resolución de problemas y planificación de los capítulos anteriores operan todos con un conjunto fijo de acciones atómicas. Las acciones pueden concatenarse, y los algoritmos de vanguardia pueden generar soluciones que contienen miles de acciones. Eso está bien si estamos planeando unas vacaciones y las acciones están en el nivel de "volar desde San Francisco a Honolulu", pero en el nivel de control motor de "doblar la rodilla izquierda en 5 grados" necesitaríamos concatenar millones o miles de millones de acciones, no miles.

Para cerrar esta brecha se requiere planificación en niveles más altos de abstracción. Un plan de alto nivel para unas vacaciones en Hawái podría ser "Ir al aeropuerto de San Francisco; tomar el vuelo HA 11 a Honolulu; vacacionar por dos semanas; tomar el HA 12 de regreso a San Francisco; ir a casa". Dado este plan, la acción "Ir al aeropuerto de San Francisco" puede ser vista como una tarea de planificación en sí misma, con una solución como "Elegir un servicio

de transporte; pedir un coche; ir al aeropuerto". Cada una de estas acciones, a su vez, puede ser descompuesta más, hasta que lleguemos a las acciones de control motor de bajo nivel como presionar un botón.

En este ejemplo, la planificación y la acción están entrelazadas; es decir, el problema de planificar el camino de la acera a la puerta se pospondría hasta después de haber sido dejado. Así, esa acción en particular permanecerá en un nivel abstracto antes de la fase de ejecución. Dejamos este tema para la Sección 11.5. Aquí, nos concentramos en la idea de la **descomposición jerárquica**, una idea que impregna casi todos los intentos de gestionar la complejidad. Por ejemplo, el software complejo se crea a partir de una jerarquía de subrutinas y clases; los ejércitos, gobiernos y corporaciones tienen jerarquías organizativas. El beneficio clave de la estructura jerárquica es que, en cada nivel de ésta, una tarea computacional, misión militar o función administrativa se reduce a un *pequeño* número de actividades en el siguiente nivel inferior, por lo que es pequeño el costo computacional de encontrar la forma correcta de organizar esas actividades para el problema actual.

Descomposición jerárquica

11.4.1 Acciones de alto nivel

La formulación básica adoptada para entender la descomposición jerárquica proviene del área de las **redes de tareas jerárquicas** o planificación HTN. Por ahora, asumimos plena observabilidad y determinismo, y un conjunto de acciones **(acciones primitivas)**, con esquemas de precondición-efecto estándar. El concepto clave adicional es la **acción de alto nivel** o HLA, por ejemplo, la acción "Ir al aeropuerto de San Francisco". Cada HLA tiene uno o más **refinamientos** posibles, en una secuencia de acciones, cada una de las cuales puede ser un HLA o una acción primitiva. Por ejemplo, la acción "Ir al aeropuerto de San Francisco", representada formalmente como *Go*(*Home*,*SFO*), podría tener dos refinamientos posibles, como en la Figura 11.7. La misma figura muestra un refinamiento **recursivo** para la navegación en el mundo de la aspiradora: para llegar a un destino, dar un paso y luego ir al destino.

Redes de tareas jerárquicas
Acción primitiva
Acción de alto nivel
Refinamiento

Estos ejemplos muestran que las acciones de alto nivel y sus refinamientos incorporan conocimiento sobre *cómo hacer las cosas*. Por ejemplo, los refinamientos para *Go*(*Home, SFO*) indican que para llegar al aeropuerto es posible conducir o tomar un servicio de transporte; comprar leche, sentarse y mover el caballo a e4 no deben considerarse.

Un refinamiento HLA que contiene sólo acciones primitivas se llama **implementación** del HLA. En un mundo de cuadrícula, las secuencias [*Right,Right,Down*] y [*Down,Right,Right*] ambas implementan el HLA *Navigate*([1, 3],[3, 2]). Una implementación de un plan de alto nivel (una secuencia de HLAs) es la concatenación de las implementaciones de cada HLA en la secuencia. Dadas las definiciones de precondición-efecto de cada acción primitiva, es fácil determinar si alguna implementación dada de un plan de alto nivel logra el objetivo.

Implementación

Podemos decir que *un plan de alto nivel logra el objetivo desde un estado dado, si al menos una de sus implementaciones logra el objetivo desde ese estado*. El "al menos una" en esta definición es crucial, pues no *todas* las implementaciones necesitan lograr el objetivo, porque el agente decide qué implementación ejecutará. Así, el conjunto de implementaciones posibles en la planificación HTN —cada una de las cuales puede tener un resultado diferente— no es el mismo que el conjunto de resultados posibles en la planificación no determinista. Allí, requerimos que un plan funcione para *todos* los resultados porque el agente no puede elegir el resultado; la naturaleza lo hace.

El caso más simple es un HLA que tiene exacto una implementación. En ese caso, podemos calcular las precondiciones y efectos del HLA a partir de los de la implementación (vea el Ejercicio 11.HLAU) y luego tratar el HLA exactamente como si fuera una acción primitiva en sí misma. Se puede demostrar que la colección correcta de HLAs puede resultar en que la

Refinement(*Go*(*Home*,*SFO*),
  STEPS: [*Drive*(*Home*,*SFOLongTermParking*),
    Shuttle(SFOLongTermParking,SFO)])
Refinement(*Go*(*Home*,*SFO*),
  STEPS: [*Taxi*(*Home*,*SFO*)])

Refinement(*Navigate*([*a*,*b*],[*x*,*y*]),
  PRECOND: $a=x \wedge b=y$
  STEPS: [ ])
Refinement(*Navigate*([*a*,*b*], [*x*,*y*]),
  PRECOND:*Connected*([*a*,*b*], [*a*−1,*b*])
  STEPS: [*Left*,*Navigate*([*a*−1,*b*], [*x*,*y*])])
Refinement(*Navigate*([*a*,*b*], [*x*,*y*]),
  PRECOND:*Connected*([*a*,*b*], [*a*+1,*b*])
  STEPS: [*Right*,*Navigate*([*a*+1,*b*], [*x*,*y*])])
...

Figura 11.7 Definiciones de posibles mejoras para dos acciones de alto nivel: ir al aeropuerto de San Francisco y navegar en el mundo de la aspiradora. En el último caso, observe la naturaleza recursiva de los refinamientos y el uso de condiciones previas.

complejidad temporal de la búsqueda ciega se reduzca de exponencial, en la profundidad de la solución, a lineal en la profundidad de la solución; aunque, idear tal colección de HLAs puede ser una tarea no trivial en sí misma. Cuando los HLAs tienen múltiples implementaciones posibles, existen dos opciones: una es buscar entre éstas una que funcione, como en la Sección 11.4.2; la otra es razonar directamente sobre los HLAs —a pesar de la multiplicidad de implementaciones—, como se explica en la Sección 11.4.3. Este último método permite la derivación de planes abstractos quizá correctos, sin la necesidad de considerar sus implementaciones.

11.4.2 Búsqueda de soluciones primitivas

La planificación HTN se suele formular con una única acción "de nivel superior" llamada *Act*, donde el objetivo es encontrar una implementación de *Act* que logre el objetivo. Este enfoque es general. Por ejemplo, los problemas de planificación clásica se pueden definir así: para cada acción primitiva a_i, proporcionar una refinación de *Act* con pasos [a_i, *Act*]. Esto crea una definición recursiva de *Act* que nos permite agregar acciones. Pero necesitamos alguna forma de detener la recursión; lo hacemos proporcionando una refinación más para *Act*, una con una lista vacía de pasos y con una precondición igual al objetivo del problema. Esto indica que, si el objetivo ya se ha logrado, entonces la implementación adecuada es no hacer nada.

El enfoque conduce a un algoritmo simple: elegir repetidamente un HLA en el plan actual y reemplazarlo con una de sus refinaciones, hasta que el plan logre el objetivo. Una posible implementación basada en la búsqueda de árbol por amplitud se muestra en la Figura 11.8. Los planes se consideran en orden de profundidad de anidamiento de las refinaciones, en lugar del número de pasos primitivos. Es sencillo diseñar una versión del algoritmo de búsqueda de grafos, así como versiones de búsqueda en profundidad e iterativa.

```
function HIERARCHICAL-SEARCH(problem, hierarchy) returns una solución o failure

  frontier ←un registro FIFO con [Act] como único elemento
  while true do
    if IS-EMPTY(frontier) then return failure
    plan ← POP(frontier) // elige el plan menos profundo en la frontera
    hla ←el primer HLA en plan, o null si es nulo
    prefix,suffix ←las subsecuencias de acción antes y después hla en plan
    outcome ←RESULT(problem.INITIAL, prefix)
    if hla is null then   // entonces el plan es primitivo y la salida es su resultado
      if problem.IS-GOAL(outcome) then return plan
    else for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
      agregar APPEND( prefix, sequence, suffix) en frontier
```

Figura 11.8 Implementación amplia de la búsqueda jerárquica de planificación anticipada. El plan inicial proporcionado al algoritmo es [*Act*]. La función REFINEMENTS devuelve un conjunto de secuencias de acciones, una para cada refinamiento del HLA precondiciones se satisfacen con el estado especificado, *outcome*.

En esencia, esta forma de búsqueda jerárquica explora el espacio de secuencias que se ajustan al conocimiento contenido en la biblioteca de HLAs sobre cómo deben hacerse las cosas. Se puede codificar una gran cantidad de conocimiento, no sólo en las secuencias de acciones descritas en cada refinamiento, también en las precondiciones para los refinamientos. Para algunos dominios, los planificadores HTN han generado planes enormes con muy poca búsqueda. Por ejemplo, O-PLAN (Bell y Tate, 1985), que combina la planificación HTN con la programación, se ha usado para desarrollar planes de producción para Hitachi. Un problema típico implica una línea de producción de 350 productos diferentes, 35 máquinas de ensamblaje y más de 2 000 operaciones diferentes. El planificador genera un horario de 30 días con tres turnos de 8 horas al día, que involucra decenas de millones de pasos. Otro aspecto cardinal de los planes HTN es que, por definición, están estructurados de forma jerárquica; esto los hace fáciles de entender para los humanos.

La utilidad computacional de la búsqueda jerárquica se ve al examinar un caso idealizado. Suponga un problema de planificación con una solución con d acciones primitivas. Para un planificador no jerárquico de espacio de estados hacia adelante con b acciones permitidas en cada estado, el costo es $O(b^d)$, como vimos en el Capítulo 3. Para un planificador HTN, imagine una estructura de refinamiento regular: cada acción no primitiva tiene r refinamientos posibles, cada uno en k acciones en el siguiente nivel inferior. Queremos saber cuántos árboles de refinamiento diferentes hay con esta estructura. Ahora, si hay d acciones en el nivel primitivo, el número de niveles debajo de la raíz es $\log_k d$, por lo que el número de nodos de refinamiento internos es $1 + k + k^2 + \ldots + k^{\log_k d - 1} = (d - 1)/(k - 1)$. Cada nodo interno tiene r refinamientos posibles, y así se podrían construir $r^{(d-1)/(k-1)}$ árboles de descomposición posibles.

Examinando esta fórmula, vemos que mantener r pequeño y k grande puede resultar en ahorros enormes: estamos tomando la raíz k-ésima del costo no jerárquico, si b y r son comparables. Un r pequeño y un k grande significan una biblioteca de HLAs con un pequeño número de refinamientos, cada uno produciendo una secuencia de acciones larga. Esto no siempre es posible: las secuencias de acciones largas utilizables en una amplia gama de problemas son muy raras.

La clave de la planificación HTN es una biblioteca de planes que aloja métodos conocidos para implementar acciones complejas y de alto nivel. Una forma de construir la biblioteca es *aprender* los métodos a partir de la experiencia en resolver problemas. Luego de la experiencia agotadora de construir un plan desde cero, el agente guardará el plan en la biblioteca como un método para implementar la acción de alto nivel definida por la tarea. Así, el agente puede hacerse más competente a medida que se construyen nuevos métodos sobre los antiguos. Un aspecto clave de este proceso de aprendizaje es la capacidad de *generalizar* los métodos que se construyen, al eliminar detalles específicos de la instancia del problema (como, el nombre del constructor o la dirección del terreno) y conservar sólo los elementos clave del plan. Nos parece inconcebible que los humanos sean tan competentes como lo son sin un mecanismo así.

11.4.3 Búsqueda de soluciones abstractas

La búsqueda jerárquica de soluciones en la sección anterior refinaba las HLAs hasta secuencias de acciones primitivas para determinar si un plan es viable. Esto contradice el sentido común: uno debería poder determinar que el plan de alto nivel de dos HLAs

[*Drive*(*Home*, *SFOLongTermParking*), *Shuttle*(*SFOLongTermParking*, *SFO*)]

llega al aeropuerto sin determinar una ruta precisa, elección de lugar de estacionamiento, etc. La solución es escribir descripciones de precondiciones-efecto de las HLAs, como con las acciones primitivas. A partir de las descripciones, debería ser fácil demostrar que el plan de alto nivel logra el objetivo. Este es el santo grial, digamos, de la planificación jerárquica, porque si derivamos un plan de alto nivel que logra de manera comprobada el objetivo, trabajando en un espacio de búsqueda pequeño de acciones de alto nivel, entonces nos comprometemos con ese plan y trabajamos en el problema de refinar cada paso. Esto da la reducción exponencial que buscamos.

Para que esto funcione, tiene que ser el caso que cada plan de alto nivel que "afirma" alcanzar el objetivo (por virtud de las descripciones de sus pasos) lo haga de hecho en el sentido antes definido: debe tener al menos una implementación que logre el objetivo. Esta propiedad se llama **propiedad de refinamiento descendente** para las descripciones de HLA.

Propiedad de refinamiento descendente

Escribir descripciones de HLA que satisfagan la propiedad de refinamiento descendente es, en principio, fácil: si las descripciones sean *verdaderas*, cualquier plan de alto nivel que afirme alcanzar el objetivo debe en verdad hacerlo; de lo contrario, las descripciones dan una afirmación falsa sobre lo que hacen las HLAs. Ya hemos visto cómo escribir descripciones verdaderas para HLAs que tienen sólo una implementación (Ejercicio 11.HLAU); surge un problema cuando la HLA tiene múltiples implementaciones. ¿Cómo describir los efectos de una acción a implementarse de muchas maneras diferentes?

Una respuesta segura (al menos para problemas con precondiciones y objetivos positivos) es incluir sólo los efectos positivos que se logran en *cada* implementación de la HLA y los efectos negativos de cualquier implementación. Así se satisfaría la propiedad de refinamiento descendente. Lamentablemente, esta semántica para HLAs es muy conservadora.

Piense de nuevo la HLA *Go*(*Home*, *SFO*), que tiene dos refinamientos, y suponga, por el bien del argumento, un mundo simple donde siempre se puede conducir al aeropuerto y estacionarse, aunque tomar taxi requiere *Cash* como precondición. Así, *Go*(*Home*, *SFO*) no siempre te lleva al aeropuerto. En particular, falla si *Cash* es falso y, por lo tanto, no podemos afirmar *At*(*Agent*, *SFO*) como un efecto de la HLA. Pero esto no tiene sentido, si el agente no tuviera *Cash*, conduciría por sí mismo. Requerir que un efecto se mantenga *para cada* implementación es equivalente a asumir que *otra persona*, un adversario, elegirá la implementación. Trata los múltiples resultados

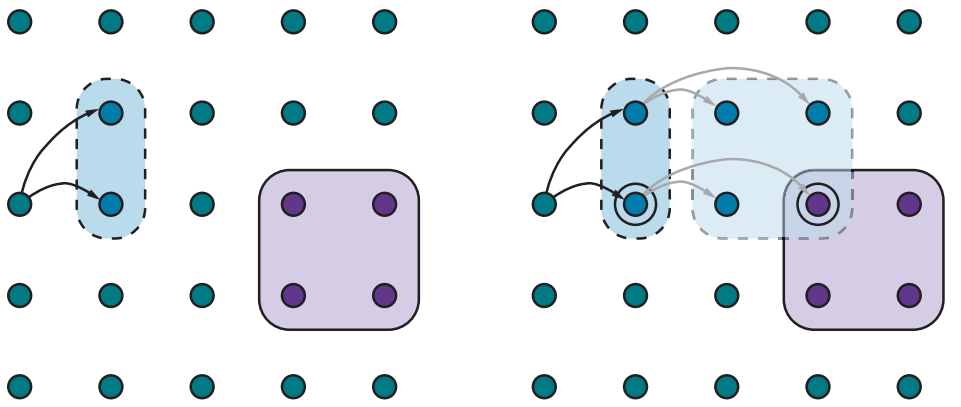


Figura 11.9 Ejemplos esquemáticos de conjuntos alcanzables. El conjunto de estados objetivo está sombreado en violeta. Las flechas negras indican posibles implementaciones de h_1 y h_2. (a) El conjunto alcanzable de un HLA h_1 en un estado s. (b) El conjunto alcanzable para la secuencia $[h_1, h_2]$. Debido a que esto se cruza con el objetivo establecido, la secuencia logra el objetivo.

del HLA tal como si el HLA fuera una acción **no determinista**, como en la Sección 4.3. En nuestro caso, el propio agente elegirá la implementación.

No determinismo demoníaco
No determinismo angelical
Semántica angelical
Conjunto alcanzable

La comunidad de lenguajes de programación acuñó el término **no determinismo demoníaco** para cuando un adversario toma las decisiones; que contrasta con **no determinismo angelical**, donde el agente mismo toma las decisiones. Tomamos prestado este término para definir **semántica angelical** para las descripciones de HLA. El concepto básico necesario para entender esta semántica es el **conjunto alcanzable** de un HLA: dado un estado s, el conjunto alcanzable para un HLA h, escrito como REACH(s,h), es el conjunto de estados alcanzables por cualquier implementación del HLA.

La idea clave es que el agente elige en qué elemento del conjunto alcanzable termina cuando ejecuta el HLA; así, un HLA con múltiples refinamientos es más "poderoso" que el mismo HLA con menos refinamientos. Además, definimos el conjunto alcanzable de una secuencia de HLAs. Así, el conjunto alcanzable de una secuencia $[h_1, h_2]$ es la unión de todos los conjuntos alcanzables obtenidos aplicando h_2 en cada estado en el conjunto alcanzable de h_1:

$$\text{REACH}(s, [h_1,h_2]) = \bigcup_{s' \in \text{REACH}(s, h_1)} \text{REACH}(s', h_2).$$

Dadas estas definiciones, un plan de alto nivel, una secuencia de HLAs, logra el objetivo si su conjunto alcanzable *interseca* el conjunto de estados objetivo. (Compare esto con la condición más fuerte para la semántica demoníaca, donde cada miembro del conjunto alcanzable debe ser un estado objetivo). Al contrario, si el conjunto alcanzable no interseca el objetivo, entonces en definitivo el plan no funciona. La Figura 11.9 ilustra estas ideas.

La noción de conjuntos alcanzables genera un algoritmo directo: buscar entre los planes de alto nivel y encontrar uno cuyo conjunto alcanzable interseque el objetivo; si eso sucede, el algoritmo puede *comprometerse* con ese plan abstracto, sabiendo que funciona, y centrarse en

refinar más el plan. Volveremos a los problemas algorítmicos luego; ahora, considere cómo se representan los efectos de un HLA, es decir, el conjunto alcanzable para cada estado inicial posible. Una acción primitiva puede establecer un flujo en *verdadero* o *falso* o dejarlo *sin cambios*. (Con efectos condicionales (ver Sección 11.5.1) hay una cuarta posibilidad: cambiar una variable a su opuesto).

Un HLA bajo semántica angelical puede hacer más: *controlar* el valor de un flujo, al establecerlo como verdadero o falso, dependiendo de la implementación elegida. Esto significa que un HLA puede tener nueve efectos diferentes en un flujo: si la variable comienza siendo verdadera, puede mantenerla siempre verdadera, siempre hacerla falsa o tener una elección; si el flujo comienza siendo falso, puede mantenerlo siempre falso, siempre hacerlo verdadero o tener una elección. Las tres elecciones para ambos casos pueden combinarse arbitrariamente, dando nueve posibilidades.

En notación esto es desafiante. Usaremos el lenguaje de listas de adición y eliminación (en lugar de flujo verdadero/falso) junto con el símbolo ~ para significar "posiblemente, si el agente así lo decide". Así, el efecto $\tilde{+}A$ significa "posiblemente agregar A", es decir, dejar A sin cambios o hacerlo verdadero. De manera similar, $\tilde{-}A$ significa "posiblemente eliminar A" y $\tilde{\pm}A$ significa "posiblemente agregar o eliminar A". Por ejemplo, el HLA *Go*(*Home*, *SFO*), con las dos refinaciones vistas en la Figura 11.7, posiblemente elimina *Cash* (si el agente decide tomar taxi), por ello debería tener el efecto $\tilde{-}Cash$. Así, vemos que las descripciones de los HLAs son *derivables* de las descripciones de sus refinamientos. Ahora, suponga los siguientes esquemas para los HLAs h_1 y h_2:

$$Action(h_1, \text{PRECOND: } \neg A, \text{EFFECT: } A \wedge \tilde{-}B),$$
$$Action(h_2, \text{PRECOND: } \neg B, \text{EFFECT: } \tilde{+}A \wedge \tilde{\pm}C).$$

Es decir, h_1 agrega A y posiblemente elimina B, mientras que h_2 posiblemente agrega A y tiene control total sobre C. Ahora, si sólo B es verdadero en el estado inicial y el objetivo es $A \wedge C$, entonces la secuencia $[h_1,h_2]$ logra el objetivo: elegimos una implementación de h_1 que haga que B sea falso, luego elegimos implementación de h_2 que deje A verdadero y haga C verdadero.

Esta discusión asume que los efectos de un HLA, el conjunto alcanzable para cualquier estado inicial dado, se describe exactamente al detallar el efecto sobre cada flujo. Sería bueno si siempre fuera verdadero, pero en muchos casos sólo podemos aproximar los efectos porque un HLA puede tener infinitas ejecuciones y producir conjuntos alcanzables arbitrariamente irregulares, algo así como el problema de estados de creencia irregular visto en la Figura 7.21 (página 243). Por ejemplo, dijimos que *Go*(*Home*,*SFO*) posiblemente elimina *Cash*, y posiblemente agrega *At*(*Car*,*SFOLongTermParking*); pero no puede hacer ambas cosas, de hecho, debe hacer exactamente una. Como con los estados de creencia, es posible que necesitemos escribir descripciones *aproximadas*. Usaremos dos tipos de aproximación: una **descripción optimista** REACH$^+(s,h)$ de un HLA puede exagerar el conjunto alcanzable, mientras que una **descripción pesimista** REACH$^-(s,h)$ puede subestimar el conjunto alcanzable. Así, tenemos

Descripción optimista
descripción pesimista

$$\text{REACH}^-(s,h) \subseteq \text{REACH}(s,h) \subseteq \text{REACH}^+(s,h).$$

Por ejemplo, una descripción optimista de *Go*(*Home*,*SFO*) indica que posiblemente elimina *Cash* y es posible agregar *At*(*Car*,*SFOLongTermParking*). Otro buen ejemplo surge en el 8-puzzle, la mitad de cuyos estados son inalcanzables desde cualquier estado dado (ver Ejercicio 11.PART): la descripción optimista de *Act* podría incluir todo el espacio de estado, ya que el conjunto alcanzable exacto es bastante irregular.

Con descripciones aproximadas, la prueba para determinar si un plan logra el objetivo necesita modificarse ligeramente. Si el conjunto alcanzable optimista para el plan no interseca el

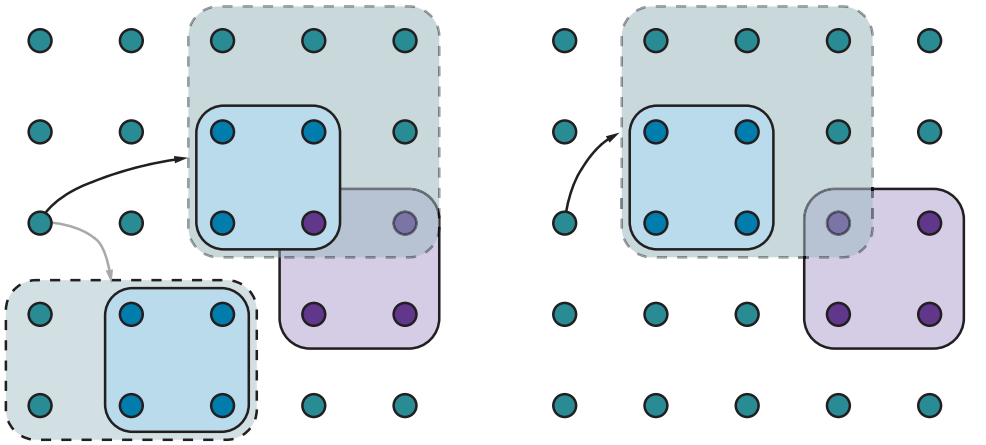


Figura 11.10 Logro de objetivos para planes de alto nivel con descripciones aproximadas. El conjunto de estados objetivo está sombreado en violeta. Para cada plan, se muestran los conjuntos alcanzables pesimistas (líneas continuas, cuadro azul claro) y optimistas (líneas discontinuas, cuadro verde claro). (a) El plan indicado por la flecha negra definitivamente logra el objetivo, pero el plan indicado por la flecha roja no. (b) Un plan que *posiblemente* logra la meta (el conjunto optimista alcanzable cruza la meta) pero no *necesariamente* logra la meta (el conjunto pesimista alcanzable no cruza la meta). Sería necesario perfeccionar más el plan para determinar si realmente logra el objetivo.

objetivo, entonces el plan no funciona; si el conjunto alcanzable pesimista interseca el objetivo, entonces el plan sí funciona (Figura 11.10(a)). Con descripciones exactas, un plan funciona o no, pero con descripciones aproximadas, hay un punto intermedio: si el conjunto optimista interseca el objetivo, pero el conjunto pesimista no, entonces no podemos decir si el plan funciona (Figura 11.10(b)). Cuando surge esta circunstancia, la incertidumbre puede resolverse refinando el plan. Esta es una situación común en el razonamiento humano. Por ejemplo, al planificar las vacaciones de dos semanas en Hawái, uno podría proponer pasar dos días en cada una de las siete islas. La prudencia indicaría que este plan ambicioso necesita ser refinado con agrega detalles sobre el transporte entre islas.

La Figura 11.11 muestra un algoritmo para la planificación jerárquica con descripciones angelicales aproximadas. Para simplificar, se ha mantenido el mismo esquema general, visto en la Figura 11.8; es decir, una búsqueda en amplitud en el espacio de refinamientos. Como ya se explicó, mediante la comprobación de las intersecciones de los conjuntos alcanzables optimistas y pesimistas con el objetivo, el algoritmo puede detectar planes que sí funcionarán y planes que no. (El Ejercicio 11.HLAP trata los detalles de cómo calcular los conjuntos alcanzables de un plan, dados las descripciones aproximadas de cada paso).

Cuando se encuentra un plan abstracto viable, el algoritmo *descompone* el problema original en subproblemas, uno para cada paso del plan. El estado inicial y el objetivo de cada subproblema se obtienen mediante la regresión de un estado objetivo alcanzable garantizado mediante esquemas de acción para cada paso del plan. (Vea en la Sección 11.2.2 el análisis de cómo funciona la regresión). La Figura 11.9(b) ilustra la idea básica: el estado circulado a la derecha es el estado objetivo alcanzable garantizado, el estado circulado a la izquierda es el objetivo intermedio obtenido al regresar el objetivo a través de la acción final.

```
function ANGELIC-SEARCH(problem, hierarchy, initialPlan) returns solución o fail

  frontier ←un registro FIFO con initialPlan como único elemento
  while true do
    if EMPTY?(frontier) then return fail
    plan ← POP(frontier) // elige el nodo menos profundo de la frontera
    if REACH+(problem.INITIAL, plan) intersecta problem.GOAL then
      if plan es primitivo then return plan //REACH+ es exacto para planos primitivos
      guaranteed←REACH−(problem.INITIAL,plan) ∩ problem.GOAL
      if guaranteed/={} y MAKING-PROGRESS(plan, initialPlan) then
        finalState ←cualquier elemento en guaranteed
        return DECOMPOSE(hierarchy,problem.INITIAL,plan, finalState)
      hla ←cualquier HLA en plan
      prefix,suffix ← las subsecuencias de acción antes y después hla en plan
      outcome ←RESULT(problem.INITIAL, prefix)
      for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
        frontier ← Insert(APPEND(prefix, sequence, suffix), frontier)

function DECOMPOSE(hierarchy, s0, plan, sf) returns una solución

  solution ←un plan vacío
  while plan no está vacío do
    action ←REMOVE-LAST(plan)
    si ←un estado en REACH−(s0, plan) tal que sf ∈REACH−(si, action)
    problem ←un problema con INITIAL = si and GOAL = sf
    solution←APPEND(ANGELIC-SEARCH(problem, hierarchy, action), solution)
    sf ← si
  return solution
```

Figura 11.11 Un algoritmo de planificación jerárquica que usa semántica angelical para identificar y comprometerse con planes de alto nivel que funcionen, mientras evita planes de alto nivel que no lo hagan. El predicado MAKING-PROGRESS verifica que no estemos atascados en una regresión infinita de refinamientos. En el nivel superior, llama a ANGELIC-SEARCH con [*Act*] como el *plan inicial*.

La capacidad para comprometerse o rechazar planes de alto nivel puede dar a ANGELIC-SEARCH una ventaja computacional significativa sobre HIERARCHICAL-SEARCH que, a su vez, puede tener mucha ventaja sobre BREADTH-FIRST-SEARCH simple y antiguo. Piense, por ejemplo, limpiar un gran mundo de la aspiradora en una disposición de habitaciones conectadas por pasillos estrechos, donde cada habitación es un rectángulo de $w \times h$ cuadrados. Hace sentido tener un HLA para *Navegar* (como en la Figura 11.7) y uno para *CleanWholeRoom*. (La limpieza de la habitación podría implementarse con la aplicación repetida de otro HLA para limpiar cada fila). Como hay cinco acciones primitivas, el costo para BREADTH-FIRST-SEARCH crece como 5^d, donde d es la longitud de la solución más corta (casi el doble del número total de cuadrados); el algoritmo no puede manejar ni siquiera dos habitaciones de 3×3. HIERARCHICAL-SEARCH es más eficiente, pero sufre de crecimiento exponencial porque prueba todas las formas de limpieza consistentes con la jerarquía. ANGELIC-SEARCH, escala cercana de manera lineal en el número de cuadrados, se compromete con una buena secuencia de alto nivel de pasos de limpieza, navegación y poda las otras opciones.

Asear un conjunto de habitaciones al limpiar cada habitación no es justamente ciencia espacial: para es fácil los humanos debido a la estructura jerárquica de la tarea. Si pensamos en lo difícil que es para los humanos resolver pequeños rompecabezas como el 8-puzzle, es probable que nuestra capacidad para resolver problemas complejos no venga de considerar combinatorias, sino de la habilidad para abstraer, descomponer problemas y eliminar las combinatorias.

El enfoque angelical puede extenderse para encontrar soluciones de menor costo al generalizar la noción de conjunto alcanzable. En lugar de que un estado sea alcanzable o no, cada estado tendrá un costo para la manera más eficiente de llegar allí. (El costo es infinito para estados inalcanzables). Las descripciones optimistas y pesimistas limitan los costos. Así, la búsqueda angelical puede encontrar planes abstractos óptimos demostrables sin considerar sus implementaciones. El mismo enfoque se puede usar para obtener algoritmos efectivos de **previsión jerárquica** para la búsqueda online, al estilo de LRTA* (página 140).

Previsión jerárquica

De cierto modo, tales algoritmos reflejan aspectos de la deliberación humana en tareas como planificar vacaciones a Hawái; considerar alternativas se hace inicialmente a un nivel abstracto en escalas de tiempo largas; algunas partes del plan quedan muy abstractas hasta el momento de la ejecución, por ejemplo, cómo pasar dos días de ocio en Moloka'i, mientras otras partes se planifican en detalle, como los vuelos y el alojamiento a reservar; sin estos últimos refinamientos, no hay garantía que el plan sea factible.

11.5 Planificar y actuar en dominios no deterministas

En esta sección extendemos la planificación para manejar entornos parcialmente observables, no deterministas y desconocidos. Los conceptos básicos reflejan el Capítulo 4, pero hay diferencias derivadas del uso de representaciones factorizadas, en vez de representaciones atómicas. Esto afecta el modo como representamos los **estados de creencias** (conjuntos de posibles estados físicos donde el agente podría estar, para entornos parcialmente observables) y la capacidad de acción y observación del agente. También podemos aprovechar métodos independientes del dominio de la Sección 11.3 para calcular heurísticas de búsqueda.

Cubriremos la **planificación sin sensor** (o **planificación conforme**) para entornos sin observaciones; **planificación de contingencia** para entornos parcialmente observables y no deterministas, y **planificación online** y **replanificación** para entornos desconocidos. Esto nos permitirá abordar problemas del mundo real de gran tamaño.

Considere este problema: dadas una silla y una mesa, el objetivo es que sean del mismo color. En el estado inicial tenemos dos latas de pintura, pero los colores de la pintura y de los muebles son desconocidos. Sólo la mesa está inicialmente en el campo de visión del agente:

$$Init(Object(Table) \land Object(Chair) \land Can(C_1) \land Can(C_2) \land InView(Table))$$
$$Goal(Color(Chair, c) \land Color(Table, c))$$

Hay dos acciones: destapar una lata de pintura y pintar un objeto usando la pintura de una lata abierta.

$Action(RemoveLid(can),$
  PRECOND:$Can(can)$
  EFFECT:$Open(can))$
$Action(Paint(x, can),$
  PRECOND:$Object(x) \land Can(can) \land Color(can,c) \land Open(can)$
  EFFECT:$Color(x, c))$

Los esquemas de acción son directos, con una excepción: las precondiciones y efectos ahora pueden contener variables que no forman parte de la lista de variables de la acción. Es decir, *Paint*(*x*,*can*) no menciona la variable *c*, que representa el color de la pintura en la lata. En el caso por completo observable, esto no está permitido; tendríamos que nombrar la acción *Paint*(*x*,*can*,*c*). Pero en el caso parcialmente observable, podríamos o no saber el color en la lata.

Para resolver un problema parcialmente observable, el agente deberá razonar sobre los perceptos que obtendrá cuando ejecute el plan. El percepto será suministrado por los sensores del agente cuando esté en el acto, pero cuando esté planeando necesitará un modelo de sus sensores. En el Capítulo 4, este modelo se daba mediante una función, PERCEPT(*s*). Para la planificación, aumentamos PDDL con un nuevo tipo de esquema, el **esquema perceptual**:

Esquema perceptual

Percept(*Color*(*x*,*c*),
  PRECOND:*Object*(*x*) ∧ *InView*(*x*)
Percept(*Color*(*can*,*c*),
  PRECOND:*Can*(*can*) ∧ *InView*(*can*) ∧ *Open*(*can*)

El primer esquema dice que siempre que un objeto esté a la vista, el agente percibirá su objeto (es decir, para el objeto *x*, el agente aprenderá el valor de verdad de *Color*(*x*,*c*) para todos los *c*). El segundo esquema dice que, si una lata abierta está a la vista, entonces el agente percibe el color de la pintura en la lata. Debido a que no hay eventos exógenos en este mundo, el color de un objeto permanecerá igual, aun si no está siendo percibido, hasta que el agente realice una acción para cambiar el color del objeto. Claro, el agente necesitará una acción que haga que los objetos (uno a la vez) estén a la vista:

Action(*LookAt*(*x*),
  PRECOND:*InView*(*y*) ∧ (*x* ≠ *y*)
  EFFECT:*InView*(*x*) ∧ ¬*InView*(*y*))

Para un entorno por completo observable, tendríamos un esquema *Perceptivo* sin precondiciones para cada flujo. Un agente sin sensor, por otro lado, no tiene esquemas *Perceptivos* en absoluto. Tome en cuenta que aun un agente sin sensor puede resolver el problema de la pintura. Una solución es abrir cualquier lata de pintura y aplicarla tanto a la silla como a la mesa, **obligándolos** a tener el mismo color (aunque el agente no sabe cuál es el color).

Un agente de planificación contingente con sensores puede generar un mejor plan. Primero, mira ambos muebles para obtener sus colores; si ya son iguales, entonces el plan está hecho. Si no lo son, mira las latas de pintura; si la pintura en una lata es del mismo color que uno de los mueble, entonces la aplica al otro mueble. De lo contrario, pinta ambos con cualquier color.

Finalmente, un agente de planificación online generaría un plan contingente con menos ramas al principio, quizás ignorando la posibilidad de que ninguna lata coincida con alguno de los muebles, y en la replanificación lidiar con los problemas que surjan. O podría lidiar con la incorrección de los esquemas de acción. Mientras que un planificador contingente sólo asume que los efectos de una acción siempre tienen éxito y que pintar la silla hace el trabajo, un agente de replanificación verificará el resultado y hará un plan adicional para corregir toda falla inesperada, como un área sin pintar o el color original se trasluzca.

En el mundo real, los agentes combinan enfoques. Los fabricantes de autos venden neumáticos de repuesto y bolsas de aire, encarnaciones físicas de ramas de planes contingentes diseñadas para enfrentar ponchaduras o choques. Además, la mayoría de los conductores nunca consideran estas posibilidades; cuando surge un problema, responden como agentes de replanificación. En general, los agentes planean sólo para contingencias de consecuencias mayores y una probabilidad no despreciable de suceder. Así, un conductor que prepara un viaje por el desierto del Sahara debería

hacer planes de contingencia explícitos para averías, mientras que un viaje al mercado requiere menos planificación anticipada. Analicemos cada uno de los tres enfoques con más detalle.

11.5.1 Planificación sin sensor

La Sección 4.4.1 (p. 126) introdujo la idea básica de buscar en el espacio de estados de creencias una solución para problemas sin sensor. Convertir un problema de planificación sin sensor a uno de planificación de estado de creencias funciona de modo similar como en esa sección; las principales diferencias son que el modelo de transición física subyacente está representado por una colección de esquemas de acción, y el estado de creencia puede ser representado por una fórmula lógica en vez de por un conjunto enumerado explícitamente de estados. Suponemos que el problema de planificación subyacente es determinista.

El estado de creencia inicial para el problema de pintura sin sensor puede ignorar los flujos *InView* porque el agente no tiene sensores. Además, tomamos como dados los hechos inmutables $Object(Table) \wedge Object(Chair) \wedge Can(C_1) \wedge Can(C_2)$ porque estos se mantienen en cada estado de creencia. El agente no sabe los colores de las latas ni los objetos, o si las latas están abiertas o cerradas, pero sabe que ambos tienen colores: $\forall x \; \exists c \; Color(x,c)$. Después de la Skolemización (Sección 9.5.1), obtenemos el estado de creencia inicial:

$$b_0 = Color(x, C(x)).$$

En la planificación clásica, donde se hace la **suposición del mundo cerrado**, asumiríamos que cualquier flujo no mencionado en un estado es falso, pero en la planificación sin sensor (y parcialmente observable) tenemos que cambiar a una **suposición de mundo abierto** en la que los estados contienen flujos positivos y negativos, y si un flujo no aparece, su valor es desconocido. Así, el estado de creencia corresponde exacto al conjunto de mundos posibles que satisfacen la fórmula. Dado este estado de creencia inicial, la siguiente secuencia de acciones es una solución:

$$[RemoveLid(Can_1), Paint(Chair, Can_1), Paint(Table, Can_1)] .$$

Ahora mostramos cómo progresar el estado de creencia a través de la secuencia de acciones para mostrar que el estado de creencia final satisface el objetivo.

Primero, note que en un estado de creencia dado b, el agente puede considerar cualquier acción cuyas precondiciones sean satisfechas por b. (Las otras acciones no pueden ser usadas porque el modelo de transición no define los efectos de las acciones cuyas precondiciones podrían no estar satisfechas). Con la Ecuación (4.4) (p. 127), la fórmula general para actualizar el estado de creencia b, dada una acción aplicable a en un mundo determinista, es la siguiente:

$$b' = \text{RESULT}(b, a) = \{s' : s' = \text{RESULT}_P(s, a) \text{ and } s \in b\}$$

donde RESULT_P define el modelo de transición física. Por ahora, asumimos que el estado de creencia inicial es siempre una conjunción de literales, es decir, una fórmula 1-CNF. Para construir el nuevo estado de creencia b', debemos considerar qué sucede con cada literal ℓ en cada estado físico s en b cuando se aplica la acción a. Para literales cuyo valor de verdad ya se conoce en b, el valor de verdad en b' se calcula a partir del valor actual y las listas de adición y de eliminación de la acción. (Así, si ℓ está en la lista de eliminación de la acción, entonces se agrega $\neg\ell$ a b'). ¿Qué sucede con un literal de valor de verdad desconocido en b? Hay tres casos:

1. Si la acción agrega ℓ, entonces ℓ será verdadero en b' independientemente de su valor inicial.

2. Si la acción elimina ℓ, ℓ será falso en b' con independencia de su valor inicial.
3. Si la acción no afecta ℓ, entonces ℓ mantendrá su valor inicial (que es desconocido) y no aparecerá en b'.

Por lo tanto, vemos que el cálculo de b' es casi idéntico al caso observable, que fue especificado por la Ecuación (11.1) en la página 345:

$$b' = \text{RESULT}(b, a) = (b - \text{DEL}(a)) \cup \text{ADD}(a) .$$

No podemos usar del todo la semántica de conjuntos porque (1) debemos asegurarnos de que b' no contenga ℓ ni $\neg\ell$, y (2) los átomos pueden contener variables no ligadas. Pero aun en el caso que RESULT(b,a) se calcula comenzando con b, estableciendo todo átomo que aparezca en DEL(a) como falso y todo átomo que aparezca en ADD(a) como verdadero. Por ejemplo, si aplicamos $RemoveLid(Can_1)$ al estado de creencia inicial b_0, obtenemos

$$b_1 = Color(x,C(x)) \wedge Open(Can_1) .$$

Cuando aplicamos la acción $Paint(Chair,Can_1)$, la precondición $Color(Can_1, c)$ se satisface por el literal $Color(x,C(x))$ con enlace $\{x/Can_1, c/C(Can_1)\}$ y el nuevo estado de creencia es

$$b_2 = Color(x,C(x)) \wedge Open(Can_1) \wedge Color(Chair,C(Can_1)) .$$

Finalmente, aplicamos la acción $Paint(Table, Can_1)$ para obtener

$$b_3 = Color(x,C(x)) \wedge Open(Can_1) \wedge Color(Chair,C(Can_1)) \wedge Color(Table,C(Can_1)) .$$

El estado de creencia final satisface el objetivo, $Color(Table,c) \wedge Color(Chair,c)$, con la variable c ligada a $C(Can_1)$.

El análisis anterior de la regla de actualización ha mostrado un hecho muy importante*: la familia de estados de creencia definidos como conjunciones de literales está cerrada bajo actualizaciones definidas por esquemas de acción PDDL*. Es decir, si el estado de creencia comienza como una conjunción de literales, entonces cualquier actualización producirá una conjunción de literales. Eso significa que, en un mundo con n flujos, cualquier estado de creencia puede ser representado por una conjunción de tamaño $O(n)$. Este es un resultado reconfortante, considerando que hay 2^n estados en el mundo. Dice que podemos representar de manera compacta todos los subconjuntos de esos 2^n estados que necesitaremos alguna vez. Además, el proceso de verificar estados de creencia subconjuntos o superconjuntos de estados de creencia visitados antes también es fácil, al menos en el caso proposicional.

El problema de esta agradable imagen es que sólo funciona para esquemas de acción que tienen los *mismos efectos* para todos los estados en los que sus precondiciones se satisfacen. Es esta propiedad la que permite la preservación de la representación del estado de creencia en forma de 1-CNF. Tan pronto como el efecto puede depender del estado, se introducen dependencias entre flujos y se pierde la propiedad de 1-CNF.

Considere, por ejemplo, el mundo de la aspiradora definido en la Sección 3.2.1. Deje que los flujos sean *AtL* y *AtR* para la ubicación del robot y *CleanL* y *CleanR* para el estado de los cuadrados. Según la definición del problema, la acción *Suck* no tiene precondición, siempre se puede hacer. La dificultad es que su efecto depende de la ubicación del robot: cuando el robot está en *AtL*, el resultado es *CleanL*, pero cuando está en *AtR*, el resultado es *CleanR*. Para tales acciones, nuestros esquemas de acción necesitarán algo nuevo: un **efecto condicional**. Estos tienen la sintaxis

Efecto condicional

"**when** *condición*: *efecto"*, donde la *condición* es una fórmula lógica a comparar con el estado actual, y el *efecto* es una fórmula que describe el estado resultante. Para el mundo de la aspiradora:

Action(*Suck*,
EFFECT:**when** *AtL*: *CleanL* ∧ **when** *AtR*: *CleanR*) .

Si se aplica al estado de creencia inicial *Verdadero*, el estado de creencia resultante es (*AtL* ∧ *CleanL*) ∨ (*AtR* ∧ *CleanR*), pues no está en 1-CNF. (Vea esta transición en la Figura 4.14, p. 129.) En general, los efectos condicionales pueden inducir dependencias arbitrarias entre los flujos en un estado de creencia, lo que lleva a estados de creencia de tamaño exponencial en el peor de los casos.

Es importante entender la diferencia entre precondiciones y efectos condicionales. Todos los efectos condicionales cuyas condiciones se satisfacen tienen sus efectos aplicados para generar el estado de creencia resultante; si ninguno se satisface, entonces el estado resultante no cambia. Además, si una *precondición* no se satisface, entonces la acción no es aplicable y el estado resultante es indefinido. Desde el punto de vista de la planificación sin sensor, es mejor tener efectos condicionales que una acción inaplicable. Podríamos dividir *Suck* en dos acciones, ambas con efectos incondicionales así:

Action(SuckL,
PRECOND*: AtL;* EFFECT*: CleanL)*
Action(SuckR,
PRECOND*: AtR;* EFFECT*: CleanR).*

Sólo tenemos esquemas incondicionales, por ello los estados de creencia permanecen en 1-CNF; por desventura, no podemos determinar la aplicabilidad de *SuckL* y *SuckR* en el estado de creencia inicial.

Parece inevitable que los problemas no triviales involucren estados de creencia complicados, al igual que los encontrados cuando consideramos el problema de la estimación del estado para el mundo del wumpus (vea la Figura 7.21, página 243). Entonces la solución sugerida fue usar una **aproximación conservadora** al estado de creencia exacto; por ejemplo, el estado de creencia puede permanecer en 1-CNF si contiene todas las literales cuyos valores de verdad pueden determinarse y trata todas las demás literales como desconocidas. Si bien este enfoque es *sólido*, en el sentido de que nunca genera un plan incorrecto, es *incompleto* porque puede ser incapaz de encontrar soluciones a problemas que necesariamente involucran interacciones entre literales. Para dar un ejemplo trivial, si el objetivo es que el robot esté en un cuadrado limpio, entonces [*Suck*] es una solución, pero un agente sin sensor que insiste en estados de creencia 1-CNF no lo encontrará.

Quizás una mejor solución sea buscar secuencias de acciones que mantengan el estado de creencia lo más simple posible. En el mundo de la aspiradora sin sensor, la secuencia de acciones [*Right,Suck,Left,Suck*] genera la siguiente secuencia de estados de creencia:

$b_0 = True$
$b_1 = AtR$
$b_2 = AtR \wedge CleanR$
$b_3 = AtL \wedge CleanR$
$b_4 = AtL \wedge CleanR \wedge CleanL$

Es decir, el agente puede resolver el problema mientras mantiene un estado de creencia 1-CNF, aunque algunas secuencias (como las que comienzan con *Suck*) salen de 1-CNF. La lección general no se pierde en los humanos: siempre estamos realizando pequeñas acciones (mirar la hora,

palpar nuestros bolsillos para asegurarnos de tener las llaves del auto, leer señales de calles mientras viajamos por una ciudad) para eliminar la incertidumbre y mantener nuestro estado de creencia manejable.

Hay otro enfoque diferente para el problema de estados de creencias muy complicados: no se moleste en calcularlos todos. Suponga que el estado de creencia inicial es b_0 y nos gustaría conocer el estado de creencia resultante de la secuencia de acciones $[a_1, ..., a_m]$. En lugar de calcularlo explícitamente, sólo represéntelo como "b_0 luego $[a_1, ..., a_m]$". Esta es una representación floja pero inequívoca del estado de creencia, y es muy concisa: $O(n + m)$ donde n es el tamaño del estado de creencia inicial (asumiendo que está en 1-CNF) y m es la longitud máxima de una secuencia de acciones. Como representación del estado de creencia, tiene una desventaja; pero determinar si se satisface el objetivo, o si una acción es aplicable, puede requerir mucho cálculo.

El cálculo se puede implementar como una prueba de consecuencia: si A_m representa la colección de axiomas de estados sucesores requeridos para definir ocurrencias de las acciones $a_1, ..., a_m$ —como se explica para SATPLAN en la Sección 11.2.3— y G_m afirma que el objetivo es verdadero después de m pasos, entonces el plan logra el objetivo si $b_0 \wedge A_m \models G_m$—es decir, si $b_0 \wedge A_m \wedge \neg G_m$ no es satisfactorio. Dado un solucionador SAT moderno, puede ser posible hacer esto mucho más rápido que calcular el estado de creencia completo. Así, si ninguna de las acciones en la secuencia tiene un flujo de objetivo particular en su lista de adición, el solucionador lo detectará de inmediato. También ayuda si los resultados parciales sobre el estado de creencia; por ejemplo, flujos conocidos como verdaderos o falsos, se almacenan en caché para simplificar cálculos posteriores.

La última pieza del rompecabezas de planificación sin sensor es una función heurística para guiar la búsqueda. El significado de la función heurística es el mismo que para la planificación clásica: una estimación (quizás admisible) del costo de lograr el objetivo desde el estado de creencia dado. Con estados de creencia, tenemos un hecho adicional: resolver cualquier subconjunto de un estado de creencia es más fácil que resolver el estado de creencia:

$$\text{if } b_1 \subseteq b_2 \text{ then } h^*(b_1) \leq h^*(b_2) .$$

Así, toda heurística admisible calculada para un subconjunto es admisible para el estado de creencia en sí mismo. Los candidatos más obvios son los subconjuntos singulares, es decir, estados físicos individuales. Podemos tomar cualquier colección aleatoria de estados $s_1, ..., s_N$ que estén en el estado de creencia b, aplicar cualquier heurística admisible h, y devolver

$$H(b) = \max\{h(s_1), ..., h(s_N)\}$$

como la estimación heurística para resolver b. También podemos usar heurísticas inadmisibles como la heurística de ignorar las listas de borrado (p. 354), que parece funcionar muy bien en la práctica.

11.5.2 Planificación contingente

Vimos en el Capítulo 4 que la planificación contingente, la generación de planes con ramificación condicional basada en percepciones, es apropiada para entornos con observabilidad parcial y para no determinismo, o ambos. Para el problema de la pintura parcialmente observable con los esquemas perceptivos antes dados, una posible solución condicional es la siguiente:

```
[LookAt(Table), LookAt(Chair),
  if Color(Table, c) ∧ Color(Chair, c) then NoOp
    else [RemoveLid(Can1), LookAt(Can1), RemoveLid(Can2), LookAt(Can2),
      if Color(Table, c) ∧ Color(Can, c) then Paint(Chair, can)
      else if Color(Chair,c) ∧ Color(can,c) then Paint(Table,can)
      else [Paint(Chair,Can1),Paint(Table,Can1)]]]
```

Las variables en este plan deben considerarse cuantificadas existencialmente; la segunda línea dice que si existe un color *c* que coincida con el color de la mesa y la silla, entonces el agente no debe hacer nada para alcanzar el objetivo. Al ejecutar este plan un agente de planificación contingente mantiene su estado de creencia como fórmula lógica, y al evaluar cada condición de rama determinando, si el estado de creencia implica la fórmula de la condición o su negación. (Depende del algoritmo de planificación contingente asegurar que el agente nunca termine en un estado de creencia donde el valor de verdad de la fórmula de la condición sea desconocido). Tome en cuenta que con condiciones de primer orden, la fórmula se satisface en más de una manera; por ejemplo, la condición $Color(Table, c) \wedge Color(can, c)$ se satisfaría con $\{can/Can_1\}$ y con $\{can/Can_2\}$ si ambas latas son del mismo color que la mesa. En ese caso, el agente puede elegir cualquier sustitución satisfactoria para aplicar al resto del plan.

Como se muestra en la Sección 4.4.2, calcular el nuevo estado de creencia *b* después de una acción *a* y un percepto subsiguiente se realiza en dos etapas. La primera calcula el estado de creencia después de la acción, como lo hace para el agente sin sensor:

$$\hat{b} = (b - \text{DEL}(a)) \cup \text{ADD}(a)$$

donde, como antes, asumimos un estado de creencia representado como una conjunción de literales. La segunda etapa es un poco más complicada. Suponga que se reciben literales perceptuales $p_1, ..., p_k$. Uno podría pensar que sólo necesitamos agregar estos al estado de creencia; de hecho, podemos inferir que se satisfacen las precondiciones para la percepción. Ahora, si un percepto *p* tiene exactamente un esquema perceptual, *Percept*(*p*,PRECOND:*c*), donde *c* es una conjunción de literales, entonces esos literales pueden ser añadidos al estado de creencia junto con *p*. Por otro lado, si *p* tiene más de un esquema perceptual cuyas precondiciones podrían cumplirse según el estado de creencia predicho $\hat{b}$, entonces tenemos que agregar la *disyunción* de las precondiciones. Obviamente, esto lleva al estado de creencia fuera de la FNC 1 y plantea las mismas complicaciones que los efectos condicionales, con prácticamente las mismas clases de soluciones.

Dado un mecanismo para calcular estados de creencia exactos o aproximados, podemos generar planes contingentes con una extensión de la búsqueda adelantada AND–OR sobre estados de creencia usados en la Sección 4.4. Las acciones con efectos no deterministas, que se definen sólo mediante el uso de una disyunción en el EFECTO del esquema de acción, pueden ser acomodadas con cambios menores en el cálculo de actualización del estado de creencia y sin cambios en el algoritmo de búsqueda.[3] Para la función heurística, muchos de los métodos sugeridos para planificación sin sensor también son aplicables en el caso parcialmente observable y no determinista.

11.5.3 Planificación online

Imagine un robot de soldadura por puntos en una fábrica de automóviles. Los movimientos rápidos y precisos del robot se repiten sin parar mientras cada automóvil pasa por la línea. Aunque técnicamente impresiona, el robot quizá no parece del todo *inteligente* porque el movimiento es una secuencia fija y preprogramada; claro, el robot no "sabe lo que está haciendo" en ningún sentido significativo. Ahora supongamos que una puerta mal colocada se cae del automóvil justo cuando el robot está a punto de aplicar soldadura por puntos. Rápido, el robot reemplaza su actuador de soldadura por una pinza, recoge la puerta, la revisa en busca de arañazos, la vuelve

[3] Si se requieren soluciones cíclicas para un problema no determinista, la búsqueda AND-OR debe generalizarse a una versión interactiva como LAO* (Hansen y Zilberstein, 2001).

a colocar en el automóvil, envía un correo electrónico al supervisor del piso, vuelve a activar al actuador de soldadura y reanuda su trabajo. De repente, el comportamiento del robot parece *intencionado* y no mecánico; asumimos que no resulta de un vasto plan contingente precomputado sino de un proceso de replanificación online, lo que significa que el robot *sí* necesita saber qué está tratando de *hacer*.

Monitoreo de ejecución

Replanificar presupone una forma de **monitoreo de ejecución** para determinar la necesidad de un nuevo plan. Esa necesidad surge cuando un agente de planificación contingente se cansa de planificar cada pequeña contingencia, como si el cielo pudiera caer sobre su cabeza.[4] Esto significa que el plan contingente queda en una forma incompleta. Por ejemplo, unas ramas de un plan contingente parcialmente construido pueden sólo decir *Replanificar*; si se alcanza tal rama en la ejecución, el agente vuelve al modo de planificación. Como referimos, la decisión sobre cuánto del problema resolver de antemano y cuánto dejar para replanificar implica compensaciones entre posibles eventos con diferentes costos y probabilidades de ocurrencia. Nadie quiere que su coche se averíe en medio del Sahara y sólo entonces pensar en tener suficiente agua.

Precondición faltante

Efecto faltante

Flujo faltante

Eventos exógenos

El replanteamiento sería necesario si el modelo del mundo del agente fuera incorrecto. El modelo para una acción puede tener una **precondición faltante;** es decir, el agente puede no saber que para quitar la tapa de la lata de pintura se requiere destornillador. El modelo tendría un **efecto faltante**: pintar un objeto podría manchar el suelo con pintura. O el modelo puede carecer de un **flujo faltante** que sólo no está en absoluto en la representación; así, el modelo antes dado no tiene noción de la cantidad de pintura en una lata, ni de cómo sus acciones afectan esta cantidad o de la necesidad de que la cantidad sea distinta de cero. El modelo también puede carecer de disposiciones para **eventos exógenos**, como alguien que vuelca la lata de pintura. Los eventos exógenos también incluyen cambios en el objetivo, como adicionar el requisito de que la mesa y la silla no estén pintadas de negro. Sin la capacidad para monitorear y replanificar, es probable que el comportamiento de un agente sea frágil si se basa en la corrección total de su modelo.

El agente online tiene la opción de (al menos) tres enfoques diferentes para monitorear el entorno al ejecutar el plan, antes de ejecutar una acción:

Monitoreo de acción

Monitoreo de plan

Monitoreo de objetivo

1. **Monitoreo de acción:** el agente verifica que todas las precondiciones sigan siendo válidas.
2. **Monitoreo de plan:** el agente verifica que el plan restante seguirá siendo exitoso.
3. **Monitoreo de objetivo:** el agente verifica si existe un mejor conjunto de objetivos que podría intentar lograr.

En la Figura 11.12 vemos un esquema de monitoreo de acción. El agente lleva un registro de su plan original (*plan completo*) y de la parte del plan que aún no se ha ejecutado, que se denota como *plan*. Después de ejecutar los primeros pasos del plan, el agente espera estar en el estado E. Pero el agente observa que en realidad está en el estado O. Entonces, necesita reparar el plan encontrando algún punto P en el plan original al que pueda regresar. (Puede ser que P sea el estado objetivo, G.) El agente intenta minimizar el costo total del plan: la parte de reparación (de O a P) más la continuación (de P a G).

[4] En 1954, la señora Hodges de Alabama fue alcanzada por un meteorito que atravesó su techo. En 1992, un trozo del meteorito Mbale golpeó a un niño en la cabeza; por fortuna, su descenso fue ralentizado por hojas de plátano (Jenniskens *et al*., 1994). Y en 2009, un niño alemán afirmó haber sido alcanzado en la mano por un meteorito del tamaño de un guisante. Ninguno de estos incidentes resultó en heridos graves, lo que sugiere que a veces se exagera la necesidad de una planificación contra tales contingencias.

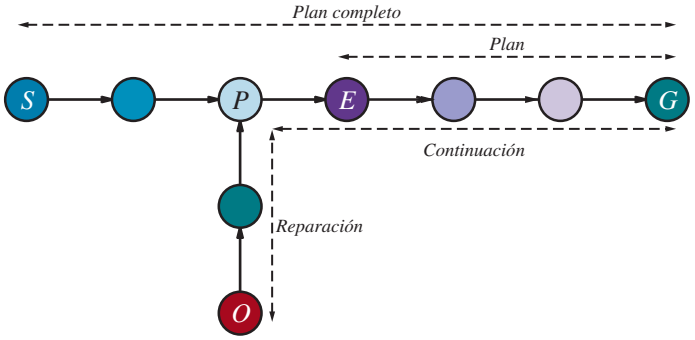


Figura 11.12 Al inicio, se espera que la secuencia "plan completo" lleve al agente de *S* a *G*. El agente ejecuta los pasos del plan hasta estar en el estado *E*, pero ve que en realidad está en *O*. Luego, el agente replanifica para la *reparación* mínima más la *continuación* hasta llegar a *G*.

Volvamos al ejemplo del problema de lograr una silla y una mesa del mismo color. Suponga que el agente elabora el siguiente plan:

[*LookAt*(*Table*), *LookAt*(*Chair*),
if *Color*(*Table, c*) ∧ *Color*(*Chair, c*) **then** *NoOp*
else [*RemoveLid*(Can_1), *LookAt*(Can_1),
if *Color*(*Table, c*) ∧ *Color*(Can_1, *c*) **then** *Paint*(*Chair, Can*$_1$)
else REPLAN]] .

Ahora el agente está listo para ejecutar el plan. El agente observa que la mesa y la lata de pintura son blancas pero la silla negra. Luego ejecuta *Paint*(*Chair*,Can_1). En este punto, un planificador clásico declararía victoria; el plan ha sido ejecutado. Pero un agente de monitoreo de ejecución online necesita verificar que la acción haya tenido éxito.

Suponga que el agente percibe un color gris moteado en la silla porque la pintura negra se traslu-ce. El agente entonces necesita hallar una posición de recuperación en el plan a la que apuntar y una secuencia de acciones de reparación para llegar allí. El agente nota que el estado actual es idéntico a la precondición antes de la acción *Paint*(*Chair*,Can_1), así que elige la secuencia vacía para *reparar* y hacer que su *plan* sea la misma secuencia [*Paint*] recién intentada. Con este nuevo plan, reanuda el monitoreo de ejecución y vuelve a intentar la acción *Pintar*. Este comportamiento se repetirá hasta percibir que la silla está por completo pintada. Note que el bucle se crea por un proceso de planificar-ejecutar-replanificar, en vez de por un bucle explícito en un plan. Vea también que el plan original no necesita cubrir todas las contingencias. Si el agente llega al paso marcado REPLAN, entonces puede generar un nuevo plan (quizás involucrando Can_2).

El monitoreo de acciones es un método simple de monitoreo de ejecución, que a veces puede llevar a un comportamiento menos inteligente. Por ejemplo, suponga que no hay pintura negra ni blanca, y el agente elabora un plan para resolver el problema pintando de rojo silla y mesa. Suponga que sólo hay suficiente pintura roja para la silla. Con el monitoreo de acciones, el agente pintaría la silla de rojo, luego notaría que se quedó sin pintura y no puede pintar la mesa, punto en el que replanificaría una reparación, quizá pintando ambas, silla y mesa, de verde. Un agente de monitoreo de planes puede detectar el fracaso siempre que el estado actual sea tal que el plan restante ya no funcione. Así, no perdería tiempo pintando la silla de rojo.

El monitoreo del plan logra esto al verificar las precondiciones para el éxito de todo el plan restante; es decir, las precondiciones de cada paso en el plan, excepto las que son logradas por otro paso en el plan restante. El monitoreo del plan corta la ejecución de un plan condenado tan pronto como sea posible, en vez de continuar hasta que ocurra el fallo.[5] El monitoreo también admite la **serendipia** o éxito accidental. Si alguien viene y pinta la mesa de rojo al tiempo que el agente pinta la silla de rojo, entonces las precondiciones finales del plan están satisfechas (el objetivo se ha logrado), y el agente puede irse a casa temprano.

Es sencillo modificar un algoritmo de planificación para que cada acción en el plan esté anotada con las precondiciones de la acción, lo que habilita el monitoreo de acciones. Es aún más complejo habilitar el monitoreo del plan. Los planificadores de orden parcial tienen la ventaja de que ya han construido estructuras que contienen las relaciones necesarias para el monitoreo del plan. El aumento de planificadores de espacio de estado con las anotaciones necesarias se puede hacer con una contabilidad cuidadosa mientras los flujos de objetivo son regresados a través del plan.

Una vez descrito un método para monitorear y replanificar, preguntemos: "¿Funciona?" Esta es una pregunta sorprendente y complicada. Si nos referimos a "¿Podemos garantizar que el agente siempre logrará el objetivo?", la respuesta es no, porque el agente podría llegar sin querer a un callejón sin salida del cual no hay reparación. Por ejemplo, el agente de aspiradora podría tener un modelo defectuoso de sí mismo y no saber que sus baterías pueden agotarse y, ya agotadas, no puede reparar ningún plan. Si descartamos los callejones sin salida, asumimos que existe un plan para alcanzar el objetivo desde *cualquier* estado en el entorno, y asumimos que el entorno es realmente no determinista, en el sentido de que dicho plan siempre tiene *alguna* posibilidad de éxito en cualquier intento de ejecución dado, entonces el agente eventualmente alcanzará el objetivo.

Los problemas surgen si una acción en apariencia no determinista no es aleatoria, sino que depende de una precondición que el agente no conoce. Por ejemplo, a veces una lata de pintura estará vacía, por ello pintar desde esa lata no tiene efecto. Ninguna cantidad de intentos cambiará esto.[6] Una solución es elegir entre el conjunto de posibles planes de reparación, en vez de intentar siempre el mismo. En este caso, el plan de reparación: abrir otra lata, funcionaría. Otro enfoque es **aprender** un mejor modelo. Cada fallo de predicción es una oportunidad para aprender; un agente debería poder modificar su modelo del mundo para que coincida con sus percepciones. A partir de ahí, el replanificador podrá encontrar una reparación que aborde el problema principal, en vez de depender de la suerte para elegir una buena reparación.

11.6 Tiempo, programación y recursos

La planificación clásica habla sobre *qué hacer, en qué orden*, pero no sobre el tiempo: *cuánto tiempo* lleva una acción y *cuándo* ocurre. Por ejemplo, en el dominio del aeropuerto podríamos elaborar un plan que diga qué aviones van y a dónde, llevando qué, pero no podríamos especificar los horarios de salida y llegada. Esto es materia de **programación**.

Programación
Restricción de recursos

El mundo real también impone **restricción de recursos**: una aerolínea tiene un número limitado de personal, y éste está en un vuelo pero no puede estar en otro a la vez. Esta sección introduce técnicas para problemas de planificación y programación con restricción de recursos.

[5] El seguimiento del plan significa que al fin, luego de 374 páginas, tenemos un agente más inteligente que un escarabajo pelotero (vea la página 41). Un agente de monitoreo de planes identificaría que la bola de estiércol no estaba en su agarre y replanearía para obtener otra bola y tapar su hoyo.

[6] La repetición inútil de un plan de reparación es tal cual la conducta de la avispa esfex (p. 41).

Jobs({*AddEngine1*≺*AddWheels1*≺*Inspect1*},
{*AddEngine2*≺*AddWheels2*≺*Inspect2*})

Resources(*EngineHoists*(1), *WheelStations*(1), *Inspectors*(*e*2), *LugNuts*(500))

Action(*AddEngine1*, DURATION:30,
USE:*EngineHoists*(1))
Action(*AddEngine2*, DURATION:60,
USE:*EngineHoists*(1))
Action(*AddWheels1*, DURATION:30,
CONSUME:*LugNuts*(20), USE:*WheelStations*(1))
Action(*AddWheels2*, DURATION:15,
CONSUME:*LugNuts*(20), USE:*WheelStations*(1))
Action(*Inspect*$_i$, DURATION:10,
USE:*Inspectors*(1))

Figura 11.13 Problema de programación de un taller para ensamblar dos autos con limitación de recursos. La notación $A \prec B$ significa que la acción A debe preceder a la acción B.

El enfoque que tomamos es "planificar primero, programar después": dividir el problema general en una fase de *planificación* en la que se seleccionan acciones, con restricción de orden, para cumplir con los objetivos del problema, y una fase posterior de *programación*, donde se agrega información temporal al plan para garantizar que cumpla con la restricción de recursos y plazos. Un enfoque común en entornos de fabricación y logística del mundo real, donde la fase de planificación o está automatizada o la realizan humanos expertos.

11.6.1 Representación de restricciones temporales y de recursos

Un **problema típico de programación de taller** (Sección 6.1.2) consiste en un conjunto de **trabajos**, cada uno con una colección de **acciones** con restricciones de orden entre ellas. Cada acción tiene una **duración** y un conjunto de restricciones de recursos requeridas por la acción. Una restricción especifica un tipo de recurso (pernos, llaves o pilotos), la cantidad requerida de ese recurso y si es **consumible** (es decir, los pernos ya no están disponibles para su uso) o **reutilizable** (un piloto está ocupado durante un vuelo, pero estará disponible cuando el vuelo haya terminado). Las acciones también pueden producir recursos (como acciones de fabricación y reabastecimiento).

Trabajo
Duración
Consumible
Reutilizable

Una solución a un problema de programación de taller especifica los tiempos de inicio para cada acción y debe cumplir las restricciones de orden temporal y restricciones de recursos. Al igual que con los problemas de búsqueda y planificación, las soluciones pueden evaluarse de acuerdo con una función de costo; esto puede ser muy complicado, con costos de recursos no lineales, costos de demora dependientes del tiempo, etc. Para simplificar, asumimos que la función de costo es sólo la duración total del plan (**makespan**).

Makespan

La Figura 11.13 muestra un ejemplo simple: un problema que implica el ensamblaje de dos autos. El problema consiste en dos trabajos, cada uno de la forma [*AddEngine*, *AddWheels*, *Inspect*]. Luego, la declaración *Resources* establece que hay cuatro tipos de recursos, y da el número de cada tipo disponible al principio: 1 grúa de motor, 1 estación de ruedas, 2 inspectores y 500 tuercas de rueda. Los esquemas de acción dan la duración y las necesidades de recursos de cada

acción. Las tuercas de rueda se *consumen* a medida que se agregan ruedas al auto, mientras que los otros recursos se "toman prestados" al comienzo de una acción y se liberan al final de ésta.

Agregación

La representación de recursos como cantidades numéricas, tal como *Inspectors*(2), en vez de entidades nombradas, como *Inspector*(I_1) e *Inspector*(I_2), es un ejemplo de una técnica llamada **agregación**: agrupar objetos individuales en cantidades cuando los objetos son indistinguibles. En nuestro problema de ensamblaje, no importa *qué* inspector inspeccione el auto, por lo que no es necesario hacer distinción. La agregación es esencial para reducir la complejidad. Piense lo que sucede cuando un programa propuesto tiene 10 acciones de *Inspección* concurrentes, pero solo hay 9 inspectores disponibles. Con los inspectores representados como cantidades, se detecta un fallo de inmediato y el algoritmo retrocede para intentar otro programa. Con los inspectores representados como individuos, el algoritmo intentaría las 9 formas posibles de asignar inspectores a acciones antes de darse cuenta de que ninguna de ellas funciona.

11.6.2 Resolución de problemas de programación (scheduling) de actividades

Iniciamos considerando sólo el problema de programación de actividades temporal, ignorando las restricciones de recursos. Para minimizar el makespan (duración del plan), debemos encontrar los tiempos de inicio más tempranos para todas las acciones consistentes con las restricciones de orden del problema. Es útil ver estas restricciones de orden como un grafo dirigido que relaciona las acciones (vea la Figura 11.14). Podemos aplicar el **método de la ruta crítica** (CPM, por sus siglas en inglés) a este grafo para determinar los posibles tiempos de inicio y finalización de cada acción. Una ruta a través de un grafo que representa un plan de orden parcial es una secuencia de acciones ordenada linealmente, que inicia con *Start* y termina con *Finish*. (Como las dos rutas en el plan de orden parcial de la Figura 11.14).

Método de la ruta crítica (CPM)

Ruta crítica

La **ruta crítica** es aquella cuya duración total es más larga; es "crítica" porque determina la duración de todo el plan: acortar otras rutas, no acorta el plan en su conjunto, pero retrasar el inicio de cualquier acción en la ruta crítica ralentiza todo el plan. Las acciones fuera de ella tienen una ventana de tiempo en la que pueden ejecutarse. La ventana se especifica en términos de un tiempo de inicio más temprano posible, *ES*, y un tiempo de inicio más tardío posible, *LS*. La cantidad $LS - ES$ se conoce como **holgura** de una acción. La Figura 11.14 muestra que todo el plan llevará 85 minutos, que cada acción en el trabajo superior tiene 15 minutos de holgura y que cada acción en la ruta crítica no tiene holgura (por definición). Juntos, los tiempos de *ES* y *LS* para todas las acciones constituyen una programación para el problema.

Holgura

Programación de actividades

Las siguientes fórmulas definen *ES* y *LS* y son un algoritmo de programación dinámica para calcularlos. A y B son acciones, y $A \prec B$ significa que A precede B:

$$ES(Start) = 0$$
$$ES(B) = \max_{A \prec B} ES(A) + Duration(A)$$
$$LS(Finish) = ES(Finish)$$
$$LS(A) = \min_{B \succ A} LS(B) - Duration(A).$$

La idea es que iniciemos asignando *ES*(*Start*) como 0. Luego, apenas obtenemos una acción B tal que todas las acciones inmediatas antes de B tienen valores de *ES* asignados, establecemos *ES*(B) como el máximo de los tiempos de finalización más tempranos de esas acciones inmediatas precedentes, donde el tiempo de finalización más temprano de una acción se define como el tiempo de inicio más temprano más la duración. Este proceso se repite hasta que se haya asignado un valor de *ES* a cada acción. Los valores de *LS* se calculan igual, trabajando hacia atrás desde la acción *Finish*.

La complejidad del algoritmo de ruta crítica es simplemente $O(Nb)$, donde N es el número de acciones y b es el factor de ramificación máximo dentro o fuera de una acción. (Para ver esto,

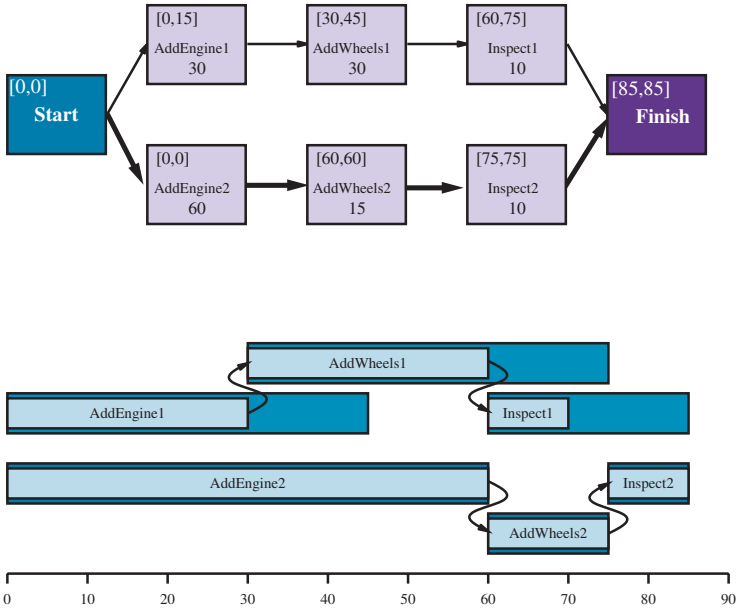


Figura 11.14 Arriba: representación de las restricciones temporales para el problema de programación del taller de la Figura 11.13. La duración de cada acción se da en la parte inferior de cada rectángulo. Para resolver el problema, calculamos las horas de inicio más tempranas y tardías como el par [*ES*, *LS*], mostrado en la parte superior izquierda. La diferencia entre ambos números es la *lentitud* de una acción; las acciones con holgura cero están en la ruta crítica, ilustrada con flechas en negrita. Abajo: la misma solución como línea de tiempo. Los rectángulos azul oscuro representan intervalos de tiempo durante los que se ejecuta una acción, siempre que se respeten las restricciones de orden. La parte desocupada indica holgura.

note que los cálculos de *LS* y *ES* se realizan una vez para cada acción, y cada cálculo itera sobre como máximo b otras acciones). Así, encontrar un horario de duración mínima, dado un orden parcial de las acciones y sin restricciones de recursos, es fácil.

Matemáticamente, los problemas de ruta crítica son fáciles de resolver porque se definen como una *conjunción* de desigualdades *lineales* en los tiempos de inicio y finalización. Si introducimos restricciones de recursos, las resultantes de éstas en los tiempos de inicio y finalización se complican. Así, las acciones *AddEngine*, que comienzan al mismo tiempo en la Figura 11.14, requieren el mismo *EngineHoist* y, por lo tanto, no pueden superponerse. La restricción "no pueden superponerse" es una *disyunción* de dos desigualdades lineales, una para cada orden posible. La introducción de disyunciones resulta en que la programación con restricciones de recursos sea NP-duro.

La Figura 11.15 muestra la solución con el tiempo de finalización más rápido, 115 minutos. Esto es 30 minutos más largo que los 85 minutos requeridos para un programa sin restricción de recursos. Vea que no hay momento en el que se requieran ambos inspectores, por ello movemos de inmediato uno de nuestros dos inspectores a una posición más productiva.

Hay una larga historia de trabajo en programación óptima. Un problema retador de 1963: encontrar el programa óptimo para un problema que abarca sólo 10 máquinas y 10 trabajos de 100 acciones cada uno, sin resolver por 23 años (Lawler *et al.*, 1993). Se han intentado muchos enfoques, como ramificación y acotación, de recocido simulado, la búsqueda tabú y la satisfacción

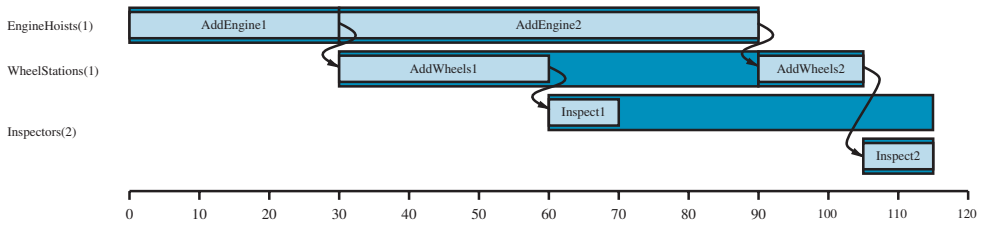


Figura 11.15 Solución al problema de programación del taller de la Figura 11.13, considerando restricciones de recursos. El margen izquierdo enumera los tres recursos reutilizables, las acciones se muestran alineadas en horizontal con los recursos que usan. Hay dos programas posibles, según qué conjunto use primero el polipasto del motor (*EngineHoist*). Mostramos la solución de menor duración, que tarda 115 minutos.

Mínima holgura

de restricciones. Un enfoque popular es la heurística de la **mínima holgura**: en cada iteración, programa para el inicio más temprano posible cualquier acción no programada que tenga sus acciones previas programadas y posea la menor holgura; luego actualiza los tiempos de inicio más tempranos y tardíos para cada acción afectada y repite. Esta heurística voraz se asemeja a la de los valores mínimos restantes (MRV) en la satisfacción de restricciones. Suele funcionar en la práctica, pero para nuestro problema de ensamblaje produce una solución de 130 minutos, no la solución de 115 minutos de la Figura 11.15.

Hasta aquí hemos supuesto que el conjunto de acciones y restricciones de orden está fijo. Con estas suposiciones, cada problema de programación se resuelve mediante una secuencia no superpuesta que evita todos los conflictos de recursos, siempre que cada acción sea factible por sí sola. Pero, si un problema de programación es difícil, puede no ser buena idea resolverlo así; quizá sea mejor reconsiderar las acciones y restricciones, en caso de que eso facilite el problema de programación. Por lo tanto, tiene sentido *integrar* la planificación y la programación al considerar las duraciones y superposiciones en la construcción de un plan. Varios algoritmos de planificación de la Sección 11.2 pueden ser ampliados para manejar esta información.

11.7 Análisis de enfoques de planificación

La planificación combina las dos áreas principales de la IA, cubiertas hasta ahora: la *búsqueda* y la *lógica*. Un planificador puede verse tanto como un programa que busca una solución como uno que demuestra (constructivamente) la existencia de una solución. La interacción de ideas de ambas áreas permite a los planificadores escalar desde problemas simples, donde el número de acciones y estados estaba limitado a alrededor de una docena, hasta aplicaciones industriales del mundo real con millones de estados y miles de acciones.

La planificación es un ejercicio en el control de la explosión combinatoria. Si hay n proposiciones en un dominio, entonces hay 2^n estados. Ante tal pesimismo, la identificación de subproblemas independientes puede ser un arma poderosa. En el mejor caso, la completa descomposición del problema nos brinda una aceleración exponencial. Pero, la descomposición se destruye por las interacciones negativas entre acciones. SATPLAN codifica relaciones lógicas entre subproblemas. La búsqueda hacia adelante aborda el problema de manera heurística al encontrar patrones (subconjuntos de proposiciones) que cubran los subproblemas independientes. Como este enfoque es heurístico, funcionará aun si los subproblemas no son del todo independientes.

Por desgracia, aún no comprendemos con claridad qué técnicas funcionan mejor en qué tipos de problemas. Quizá surjan nuevas técnicas, tal vez al proporcionar una síntesis de representaciones de primer orden y jerárquicas, altamente expresivas con las representaciones proposicionales y factorizadas altamente eficientes, que hoy dominan. Ejemplificamos sistemas de planificación de **portafolio** donde se dispone de una colección de algoritmos para aplicar a cualquier problema. Esto se hace de manera selectiva (el sistema clasifica cada nuevo problema para elegir el mejor algoritmo para él), o en paralelo (todos los algoritmos se ejecutan de modo simultáneo, cada uno en una CPU diferente), o intercalando los algoritmos según un horario.

Portafolio

Resumen

En este capítulo, describimos la representación PDDL tanto para problemas de planificación clásicos como extendidos, y presentamos varios enfoques algorítmicos para encontrar soluciones. Los puntos para recordar:

- Los sistemas de planificación son algoritmos de resolución de problemas que operan en representaciones explícitas factorizadas de estados y acciones. Estas representaciones posibilitan la derivación de heurísticas efectivas independientes del dominio y el desarrollo de algoritmos potentes y flexibles para resolver problemas.
- PDDL, el Lenguaje de Definición de Dominio de Planificación, describe los estados iniciales y objetivos como conjunciones de literales, y las acciones en términos de sus precondiciones y efectos. Las extensiones representan tiempo, recursos, perceptos, planes contingentes y planes jerárquicos.
- La búsqueda de espacio de estados puede operar en la dirección hacia adelante (**progresión**) o hacia atrás (**regresión**). Se pueden derivar heurísticas efectivas mediante suposiciones de independencia de subobjetivos y mediante diversas relajaciones del problema de planificación.
- Otros enfoques incluyen codificar un problema de planificación como un problema de satisfacción booleana o como un problema de satisfacción de restricciones; y buscar de modo explícito por el espacio de planes parcialmente ordenados.
- **La planificación de redes de tareas jerárquicas** (HTN) permite que el agente reciba asesoramiento del diseñador del dominio en forma de **acciones de alto nivel** (HLA) que pueden implementarse de varias maneras mediante secuencias de acciones de nivel inferior. Los efectos de las HLA se pueden definir con **semántica angelical**, lo que permite derivar planes de alto nivel demostrablemente correctos sin considerar implementaciones de nivel inferior. Los métodos HTN pueden crear planes muy grandes requeridos por muchas aplicaciones del mundo real.
- Los **planes contingentes** permiten que el agente perciba el mundo durante la ejecución para decidir qué rama del plan seguir. En algunos casos, se puede usar la **planificación sin sensor** o **conforme** para construir un plan que funcione sin necesidad de percepción. Tanto los planes conformes como los contingentes se pueden construir mediante búsqueda en el espacio de estados de creencias. La representación o computación eficiente de estados de creencias es un problema clave.
- Un **agente de planificación online** usa el monitoreo de ejecución e inserta reparaciones según sea necesario para recuperarse de situaciones inesperadas, que pueden deberse a acciones no deterministas, eventos exógenos o modelos incorrectos del entorno.
- Muchas acciones consumen **recursos**, como dinero, gas o materias primas. Es conveniente tratar estos recursos como medidas numéricas en un conjunto en lugar de tratar de razonar,

por ejemplo, sobre cada moneda y billete individual en el mundo. El tiempo es uno de los recursos más importantes. Puede ser manejado por algoritmos de programación especializados, o la programación puede integrarse con la planificación.

- Este capítulo extiende la planificación clásica para cubrir entornos no deterministas (donde los resultados de las acciones son inciertos), pero no es la lo último sobre planificación. El capítulo 17 describe técnicas para entornos estocásticos (en los cuales los resultados de las acciones tienen probabilidades asociadas): procesos de decisión de Markov, procesos de decisión de Markov parcialmente observables y teoría de juegos. En el capítulo 22 mostramos que el aprendizaje por refuerzo permite que un agente aprenda cómo comportarse a partir de éxitos y fracasos pasados.

Notas bibliográficas e históricas

La planificación con IA surgió de investigaciones sobre búsqueda de espacio de estados, demostración de teoremas y teoría de control. STRIPS (Fikes y Nilsson, 1971, 1993), el primer sistema de planificación importante fue diseñado como el planificador para el robot Shakey en SRI. La primera versión del programa se ejecutó en una computadora con apenas 192 KB de memoria. Su estructura de control general se modeló según GPS, Solucionador General de Problemas (Newell y Simon, 1961), un sistema de búsqueda de espacio de estados que usaba análisis de medios-fin.

El lenguaje de representación STRIPS evolucionó al Lenguaje de Descripción de Acciones, o ADL (Pednault, 1986), y luego al Lenguaje de Descripción de Dominio del Problema, o PDDL (Ghallab *et al*., 1998), utilizado para la Competencia Internacional de Planificación desde 1998. La versión más reciente es PDDL 3.1 (Kovacs, 2011).

Planificación lineal

Los planificadores en la década de 1970 descompusieron problemas calculando un subplan para cada subobjetivo y luego concatenando los subplanes en algún orden. Este enfoque, llamado **planificación lineal** por Sacerdoti (1975), pronto se descubrió que era incompleto. No puede resolver algunos problemas simples, como la anomalía de Sussman (ver Ejercicio 11.SUSS), hallada por Allen Brown durante la experimentación con el sistema HACKER (Sussman, 1975). Un planificador completo debe permitir la **intercalación** de acciones de diferentes subplanes dentro de una sola secuencia. El sistema WARPLAN de Warren (1974) logró eso y demostró cómo el lenguaje de programación lógica Prolog puede producir programas concisos; WARPLAN tiene sólo 100 líneas de código.

La planificación de orden parcial dominó los siguientes 20 años de investigación, con trabajos teóricos que describen la detección de conflictos (Tate, 1975a) y la protección de condiciones logradas (Sussman, 1975), e implementaciones que incluyen NOAH (Sacerdoti, 1977) y NONLIN (Tate, 1977). Eso llevó a modelos formales (Chapman, 1987; McAllester y Rosenblitt, 1991) que permitieron el análisis teórico de varios algoritmos y problemas de planificación, y a un sistema ampliamente distribuido, UCPOP (Penberthy y Weld, 1992).

Drew McDermott sospechaba que el énfasis en la planificación de orden parcial estaba desplazando otras técnicas que quizá deberían reconsiderarse ahora que las computadoras tenían 100 veces la memoria de la época de Shakey. Su UNPOP (McDermott, 1996) fue un programa de planificación de espacio de estados que empleaba la heurística de ignorar las listas de borrado. HSP, el Planificador de Búsqueda Heurística (Bonet y Geffner, 1999; Haslum, 2006), hizo que la búsqueda de espacio de estados fuera práctica para problemas de planificación grandes. El planificador FF o Fast Forward (Hoffmann, 2001; Hoffmann y Nebel, 2001; Hoffmann, 2005) y la variante FASTDOWNWARD (Helmert, 2006) ganaron competiciones internacionales de planificación en la década de 2000.

La búsqueda bidireccional (ver Sección 3.4.5) también ha sido conocida por su falta de heurísticas, pero se ha logrado cierto éxito usando la búsqueda inversa para crear un perímetro alrededor del objetivo, y luego refinando una heurística para buscar hacia adelante, hacia ese **perímetro** (Torralba *et al*., 2016). El planificador de búsqueda bidireccional SymBA* (Torralba *et al*., 2016) ganó la competencia de 2016.

Los investigadores recurrieron a PDDL y al paradigma de planificación para poder utilizar heurísticas independientes del dominio. Hoffmann (2005) analiza el espacio de búsqueda de la heurística de ignorar las listas de borrado. Edelkamp (2009) y Haslum *et al.* (2007) describen cómo construir bases de datos de patrones para heurísticas de planificación. Felner *et al.* (2004) muestran resultados alentadores usando bases de datos de patrones para rompecabezas de fichas deslizantes, que pueden considerarse como un dominio de planificación, pero Hoffmann *et al.* (2006) muestran algunas limitaciones de la abstracción para problemas de planificación clásicos. (Rintanen, 2012) discute heurísticas de selección de variables específicas de la planificación para la resolución de SAT.

Helmert *et al.* (2011) describen el sistema Fast Downward Stone Soup (FDSS), un portafolio de planificadores que, al igual que la fábula de la sopa de piedra, nos invita a arrojar tantos algoritmos de planificación como sea posible. El sistema mantiene un conjunto de problemas de entrenamiento, y para cada problema y cada algoritmo registra el tiempo de ejecución y el costo del plan resultante de la solución del problema. Luego, cuando se enfrenta a un nuevo problema, utiliza la experiencia pasada para decidir qué algoritmo(s) probar, con qué límites de tiempo, y adopta la solución con costo mínimo. FDSS fue un ganador en la Competición Internacional de Planificación de 2018 (Seipp y Röger, 2018). Seipp *et al.* (2015) describen un enfoque de aprendizaje automático para aprender automáticamente un buen portafolio, dado un nuevo problema. Vallati *et al*. (2015) ofrecen una visión general de los portafolios de programación. La idea de portafolios de algoritmos para problemas de búsqueda combinatoria se remonta a Gomes y Selman (2001).

Sistla y Godefroid (2004) cubren la reducción de simetría, y Godefroid (1990) cubre heurísticas para el ordenamiento parcial. Richter y Helmert (2009) demuestran las ganancias de eficiencia de la poda hacia adelante utilizando acciones preferidas.

Blum y Furst (1997) revitalizaron el campo de la planificación con su sistema Graphplan, que era órdenes de magnitud más rápido que los planificadores de orden parcial de la época. Bryce y Kambhampati (2007) ofrecen una visión general de los grafos de planificación. El uso del cálculo de situaciones para la planificación fue introducido por John McCarthy (1963) y refinado por Ray Reiter (2001).

Kautz *et al*. (1996) investigaron diversas formas de proposicionar esquemas de acciones, encontrando que las formas más compactas no necesariamente conducían a los tiempos de solución más rápidos. Un análisis sistemático fue realizado por Ernst *et al.* (1997), quienes también desarrollaron un "compilador" automático para generar representaciones proposicionales a partir de problemas PDDL. El planificador Blackbox, que combina ideas de Graphplan y SATPlan, fue desarrollado por Kautz y Selman (1998). Los planificadores basados en satisfacción de restricciones incluyen CPlan van Beek y Chen (1999) y GP-CSP (Do y Kambhampati, 2003).

También ha habido interés en la representación de un plan como un **diagrama de decisión binaria (BDD)**, una estructura de datos compacta para expresiones booleanas ampliamente estudiada en la comunidad de verificación de hardware (Clarke y Grumberg, 1987; McMillan, 1993). Hay técnicas para probar propiedades de diagramas de decisión binarios, incluida la propiedad de ser una solución a un problema de planificación. Cimatti et al. (1998) presentan un planificador basado en este enfoque. También se han usado otras representaciones, como la programación entera (Vossen *et al*., 2001).

Diagrama de decisión binaria (BDD)

Hay algunas comparaciones interesantes de los diversos enfoques de planificación. Helmert (2001) analiza varias clases de problemas de planificación y muestra que los enfoques basados en restricciones como Graphplan y SATPlan son los mejores para dominios NP-duros, mientras que

los enfoques basados en búsqueda funcionan mejor en dominios donde se pueden encontrar soluciones factibles sin retroceso. Graphplan y SATPLAN tienen problemas en dominios con muchos objetos porque eso significa que deben crear muchas acciones. En algunos casos, el problema puede retrasarse o evitarse generando las acciones proposicionales dinámicamente, solo según sea necesario, en lugar de instanciarlas todas antes de que comience la búsqueda.

Macrops

El primer mecanismo para la planificación jerárquica fue una facilidad en el programa STRIPS para aprender **macrops**, "macro-operadores" que consisten en una secuencia de pasos primitivos (Fikes *et al.,* 1972). El sistema ABSTRIPS (Sacerdoti, 1974) introdujo la idea de una **jerarquía de abstracción,** mediante la cual la planificación en niveles superiores estaba permitida para ignorar las precondiciones de acciones de niveles inferiores con el fin de derivar la estructura general de un plan de trabajo. La tesis de doctorado de Austin Tate (1975b) y el trabajo de Earl Sacerdoti (1977) desarrollaron las ideas básicas de la planificación HTN. Erol, Hendler y Nau (1994, 1996) presentan un planificador de descomposición jerárquica completa, así como una variedad de resultados de complejidad para planificadores HTN puros. Nuestra presentación de HLAs y semántica angelical se debe a Marthi *et al*. (2007, 2008).

Jerarquía de abstracción

Uno de los objetivos de la planificación jerárquica ha sido la reutilización de la experiencia de planificación previa en forma de planes generalizados. La técnica del **aprendizaje basado en explicaciones** se ha utilizado como un medio para generalizar planes previamente computados en sistemas como SOAR (Laird *et al.,* 1986) y PRODIGY (Carbonell *et al*., 1989). Un enfoque alternativo es almacenar planes previamente computados en su forma original y luego reutilizarlos para resolver nuevos problemas similares por analogía al problema original. Este es el enfoque adoptado por el campo llamado **planificación basada en casos** (Carbonell, 1983; Alterman, 1988). Kambhampati (1994) argumenta que la planificación basada en casos debería analizarse como una forma de planificación de refinamiento y proporciona una base formal para la planificación parcial basada en casos de órdenes.

Planificación basada en casos

Los planificadores tempranos carecían de condicionales y bucles, pero algunos podían usar coerción para formar planes conformantes. NOAH de Sacerdoti resolvió el problema de "llaves y cajas" (en el cual el planificador sabe poco sobre el estado inicial) usando coerción. Mason (1993) argumentó que con frecuencia se puede y se debe prescindir de la percepción en la planificación robótica, y describió un plan sin sensores que puede mover una herramienta a una posición específica en una mesa mediante una secuencia de acciones de inclinación, independientemente de la posición inicial.

Goldman y Boddy (1996) introdujeron el término **planificación conformante**, señalando que los planes sin sensores en la mayoría de los casos son efectivos incluso si el agente tiene sensores. El primer planificador conformante moderadamente eficiente fue Conformant Graphplan (CGP) de Smith y Weld (1998). Ferraris y Giunchiglia (2000) y Rintanen (1999) desarrollaron independientemente planificadores conformantes basados en SATPLAN. Bonet y Geffner (2000) describen un planificador conformante basado en búsqueda heurística en el espacio de estados de creencias, basándose en ideas desarrolladas por primera vez en la década de 1960 para procesos de decisión de Markov parcialmente observables o POMDP (consulte el Capítulo 17).

Actualmente, hay tres enfoques principales para la planificación conformante. Los dos primeros utilizan búsqueda heurística en el espacio de estados de creencias: HSCP (Bertoli *et al.,* 2001a) utiliza diagramas de decisión binaria (BDDs) para representar estados de creencias, mientras que Hoffmann y Brafman (2006) adoptan el enfoque perezoso de calcular pruebas de precondición y objetivo a pedido utilizando un solucionador SAT.

El tercer enfoque, defendido principalmente por Jussi Rintanen (2007), formula todo el problema de planificación sin sensores como una fórmula booleana cuantificada (QBF) y lo resuelve utilizando un solucionador QBF de propósito general. Los planificadores conformantes actuales son cinco órdenes de magnitud más rápidos que CGP. El ganador de la pista de planificación

conformante de 2006 en la Competencia Internacional de Planificación fue T_0 (Palacios y Geffner, 2007), que usa búsqueda heurística en el espacio de estados de creencias mientras mantiene la representación del estado de creencias simple mediante la definición de literales derivados que cubren efectos condicionales. Bryce y Kambhampati (2007) discuten cómo se puede generalizar un grafo de planificación

El enfoque de planificación contingente descrito en el capítulo se basa en Hoffmann y Brafman (2005), y fue influenciado por los algoritmos de búsqueda eficientes para grafos AND-OR cíclicos desarrollados por Jimenez y Torras (2000) y Hansen y Zilberstein (2001). El problema de la planificación contingente recibió más atención después de la publicación del influyente artículo de Drew McDermott (1978a), *Planning and Acting*. Bertoli *et al.* (2001b) describen MBP (Model-Based Planner), que utiliza diagramas de decisión binaria para hacer planificación conforme y contingente. Algunos autores utilizan "planificación condicional" y "planificación contingente" como sinónimos; otros hacen la distinción de que "condicional" se refiere a acciones con efectos no deterministas, y "contingente" significa usar la percepción para superar la observabilidad parcial.

En retrospectiva, ahora es posible ver cómo los principales algoritmos de planificación clásica dieron lugar a versiones extendidas para dominios inciertos. La búsqueda heurística de avance rápido a través del espacio de estados llevó a la búsqueda hacia adelante en el espacio de creencias (Bonet y Geffner, 2000; Hoffmann y Brafman, 2005); SATPLAN llevó a SATPLAN estocástico (Majercik y Littman, 2003) y a la planificación con lógica booleana cuantificada (Rintanen, 2007); la planificación de orden parcial llevó a UWL (Etzioni *et al.,* 1992) y CNLP (Peot y Smith, 1992); Graphplan condujo a Sensory Graphplan o SGP (Weld *et al*., 1998).

El primer planificador online con monitoreo de ejecución fue PLANEX (Fikes *et al.,* 1972), que trabajó con el planificador STRIPS para controlar el robot Shakey. SIPE (System for Interactive Planning and Execution monitoring) (Wilkins, 1988) fue el primer planificador en abordar sistemáticamente el problema de replanificación. Se ha utilizado en proyectos de demostración en varios dominios, incluida la planificación de operaciones en la cubierta de vuelo de un portaaviones, la programación de taller para una fábrica de cerveza australiana y la planificación de la construcción de edificios de varios pisos (Kartam y Levitt, 1990).

A mediados de la década de 1980, el pesimismo sobre los tiempos de ejecución lentos de los sistemas de planificación llevó a la propuesta de agentes reactivos llamados sistemas de **planificación reactiva** (Brooks, 1986; Agre y Chapman, 1987). Los "planes universales" (Schoppers, 1989) se desarrollaron como un método de tabla de búsqueda para la planificación reactiva, pero resultaron ser un redescubrimiento de la idea de **políticas** que se habían utilizado durante mucho tiempo en procesos de decisión de Markov (ver Capítulo 17). Koenig (2001) analiza técnicas de planificación online, bajo el nombre *de Búsqueda Centrada en el Agente.*

Planificación reactiva

La planificación con restricciones de tiempo fue tratada por primera vez por DEVISER (Vere, 1983). La representación del tiempo en los planes fue abordada por Allen (1984) y por Dean *et al*. (1990) en el sistema FORBIN. NONLIN+ (Tate y Whiter, 1984) y SIPE (Wilkins, 1990) podían razonar sobre la asignación de recursos limitados a varias etapas del plan. O-PLAN (Bell y Tate, 1985) se ha aplicado a problemas de recursos como la planificación de adquisiciones de software en Price Waterhouse y la planificación de ensamblaje de ejes traseros en Jaguar Cars.

Los dos planificadores SAPA (Do y Kambhampati, 2001) y T4 (Haslum y Geffner, 2001) ambos utilizaron búsqueda de espacio de estados hacia adelante con heurísticas sofisticadas para manejar acciones con duraciones y recursos. Una alternativa es usar lenguajes de acción muy expresivos, pero guiarlos mediante heurísticas específicas del dominio escritas por humanos, como se hace en ASPEN (Fukunaga *et al.,* 1997), HSTS (Jonsson *et al*., 2000) e IxTeT (Ghallab y Laruelle, 1994).

Se han implementado varios sistemas híbridos de planificación y programación: ISIS (Fox *et al.,* 1982; Fox, 1990) se ha utilizado para la programación de taller en Westinghouse, GARI (Descotte

y Latombe, 1985) planeó el mecanizado y la construcción de piezas mecánicas, FORBIN se utilizó para el control de fábricas y NONLIN+ se utilizó para la planificación logística naval. Elegimos presentar la planificación y la programación como dos problemas separados; Cushing *et al*. (2007) muestran que esto puede llevar a la incompletitud en ciertos problemas.

Existe una larga historia de programación en el ámbito aeroespacial. T-SCHED (Drabble, 1990) se utilizó para programar secuencias de comandos de misión para el satélite UOSAT-II. OPTIMUM-AIV (Aarup *et al.,* 1994) y PLAN-ERS1 (Fuchs *et al*., 1990), ambos basados en O-PLAN, se usaron para el ensamblaje de naves espaciales y la planificación de observaciones, respectivamente, en la Agencia Espacial Europea. SPIKE (Johnston y Adorf, 1992) se usó para la planificación de observaciones en la NASA para el Telescopio Espacial Hubble, mientras que el Sistema de Programación del Procesamiento en Tierra del Transbordador Espacial (Deale *et al.,* 1994) realiza la programación de taller para hasta 16 000 turnos de trabajadores. Remote Agent (Muscettola *et al*., 1998) se convirtió en el primer planificador-programador autónomo en controlar una nave espacial cuando voló a bordo de la sonda Deep Space One en 1999. Las aplicaciones espaciales han impulsado el desarrollo de algoritmos para la asignación de recursos; consulte Laborie (2003) y Muscettola (2002). La literatura sobre programación se presenta en un artículo de revisión clásico (Lawler *et al*., 1993), un libro (Pinedo, 2008) y un manual editado (Blazewicz et al., 2007).

La complejidad computacional de la planificación ha sido analizada por varios autores (Bylander, 1994; Ghallab *et al*., 2004; Rintanen, 2016). Hay dos tareas principales: **PlanSAT** es la pregunta de si existe algún plan que resuelva un problema de planificación. **PlanSAT Acotado** pregunta si hay una solución de longitud k o menos; esto se puede usar para encontrar un plan óptimo. Ambos son decidibles para la planificación clásica (porque el número de estados es finito). Pero si agregamos símbolos de función al lenguaje, entonces el número de estados se vuelve infinito, y PlanSAT se convierte sólo en semidecidible. Para problemas proposicionalizados, ambos están en la clase de complejidad PSPACE, una clase que es más grande (y, por lo tanto, más difícil) que NP y se refiere a problemas que pueden resolverse mediante una máquina de Turing determinista con una cantidad polinómica de espacio. Estos resultados teóricos son desalentadores, pero en la práctica, los problemas que queremos resolver tienden a no ser tan malos. La verdadera ventaja del formalismo de planificación clásica es que ha facilitado el desarrollo de heurísticas de dominio independiente muy precisas; otros enfoques no han sido tan fructíferos.

Readings in Planning (Allen *et al*., 1990) es una antología completa de trabajos tempranos en el campo. Weld (1994, 1999) proporciona dos excelentes encuestas de algoritmos de planificación de la década de 1990. Es interesante ver el cambio en los cinco años entre las dos encuestas: la primera se concentra en la planificación de orden parcial, y la segunda introduce Graphplan y SATPLAN. *Automated Planning and Acting* (Ghallab *et al.,* 2016) es un excelente libro de texto sobre todos los aspectos del campo. El texto de*Planning Algorithms*. La Valle (2006) cubre tanto la planificación clásica como la estocástica, con una cobertura extensa de la planificación del movimiento de robots.

La investigación en planificación ha sido central para la IA desde su inicio, y los documentos sobre planificación son un elemento básico de las revistas y conferencias de IA convencionales. También hay conferencias especializadas como la International Conference on Automated Planning and Scheduling y el International Workshop on Planning and Scheduling for Space.

CAPÍTULO 12

CUANTIFICAR LA INCERTIDUMBRE

Donde vemos cómo dominar la incertidumbre con grados numéricos de creencia.

12.1 Actuando bajo incertidumbre

Los agentes en el mundo real necesitan manejar la incertidumbre, ya sea debido a observabilidad parcial, al no determinismo o a los adversarios. Un agente puede nunca saber con certeza en qué estado se están ahora o dónde terminará luego de una secuencia de acciones.

Vimos que los agentes de resolución de problemas y lógicos manejan la **incertidumbre** al dar seguimiento a un **estado de creencia** (representación del conjunto de todos los posibles estados del mundo donde podría estar) y con un plan de contingencia para manejar toda eventualidad que sus sensores reporten en la ejecución. Este enfoque funciona en problemas simples, pero tiene inconvenientes:

Incertidumbre
Estado de creencia

- El agente debe considerar *toda posible* explicación de las observaciones de sus sensores, sin importar cuán improbable sea. Esto lleva a un gran estado de creencia lleno de posibilidades improbables.
- Un plan contingente correcto que maneje todas las eventualidades puede crecer de arbitrariamente y debe considerar contingencias arbitrarias improbables.
- A veces no hay un plan que garantice el logro del objetivo, pero el agente debe actuar. Debe tener una manera de comparar los méritos de los planes que no están garantizados.

Suponga que el objetivo de un taxi automatizado es llevar un pasajero al aeropuerto a tiempo. El taxi forma un plan, A_{90}, que implica salir de casa 90 minutos antes del vuelo y conducir a velocidad razonable. Aunque el aeropuerto está a sólo 5 millas de distancia, un agente lógico no concluirá con absoluta certeza que "El Plan A_{90} nos llevará al aeropuerto a tiempo". En cambio, llega a la conclusión más débil "El Plan A_{90} nos llevará al aeropuerto a tiempo, sí y sólo si el auto no se descompone, yo no tengo un accidente, la carretera no esté cerrada y un meteorito no dan en el auto, y ...". Ninguna de estas condiciones se deduce con certeza, por ello no podemos inferir que el plan tendrá éxito. Este es el problema de **calificación lógica** (página 241), para el cual aún no hemos visto una solución real.

Calificación lógica

No obstante, en cierto sentido, A_{90} es de hecho lo correcto. ¿Qué queremos decir con esto? Como vimos en el Capítulo 2, decimos que de todos los planes que podrían ejecutarse, se espera que A_{90} maximice la medida de rendimiento del agente (donde la expectativa es relativa al conocimiento del agente sobre el entorno). La medida de rendimiento incluye llegar al aeropuerto a tiempo para el vuelo, evitar una larga e improductiva espera en el aeropuerto y evitar multas por exceso de velocidad en el camino. El conocimiento del agente no permite garantizar ninguno de estos resultados para A_{90}, pero puede proporcionar cierto grado de creencia de que se lograrán. Otros planes, como A_{180}, podrían aumentar la creencia del agente de que llegará al aeropuerto a tiempo, pero también aumentarán la probabilidad de una larga y aburrida espera. *La opción correcta, la* ***decisión racional****, por lo tanto, depende tanto de la importancia relativa de varios*

objetivos como de la probabilidad y su grado de logro. El resto de esta sección afina estas ideas, en preparación para el desarrollo de las teorías generales de razonamiento incierto y decisiones racionales que veremos en este y los siguientes capítulos.

12.1.1 Resumiendo la incertidumbre

Veamos un ejemplo de razonamiento incierto: diagnosticar dolor de un paciente dental. Un diagnóstico —en medicina, reparación de automóviles o cualquier otro ámbito— casi siempre implica incertidumbre. Intentemos escribir reglas de diagnóstico dental usando lógica proposicional, así veremos cómo el enfoque lógico se altera. Considere la siguiente regla simple:

$$DolorDeDientes \Rightarrow Caries.$$

El problema es que esta regla es incorrecta. No todos los pacientes con dolor de dientes tienen caries; algunos tienen enfermedad de las encías, un absceso u otro problema:

$$DolorDeDientes \Rightarrow Caries \vee ProblemaDeEncías \vee Absceso \ldots$$

Pero, para que la regla sea verdadera, tenemos que agregar una lista casi ilimitada de posibles problemas. Podríamos intentar convertir la regla en una regla causal:

$$Caries \Rightarrow DolorDeDientes.$$

Pero esta regla tampoco es correcta; no todas las caries causan dolor. La única manera de corregir la regla es hacerla lógicamente exhaustiva: aumentar el lado izquierdo con todas las calificaciones necesarias para que una caries cause dolor de dientes. Intentar usar la lógica para enfrentar un dominio como el diagnóstico médico falla por tres razones principales:

Pereza

Ignorancia teórica

Ignorancia práctica

- **Pereza:** Es mucho trabajo enumerar el conjunto completo de antecedentes o consecuentes necesarios para asegurar una regla sin excepciones y es muy difícil usar esas reglas.
- **Ignorancia teórica:** La ciencia médica no tiene una teoría completa para el dominio.
- **Ignorancia práctica:** Aun conociendo todas las reglas, estaríamos inciertos de un paciente particular porque no se ha realizado o no se puede realizar todas las pruebas necesarias.

Grado de creencia

Teoría de la probabilidad

La conexión entre dolor de dientes y caries no es consecuencia lógica estricta en ninguna dirección. Esto es típico del dominio médico, y de la mayoría de dominios de juicio: derecho, negocios, diseño, reparación de autos, jardinería, etc. El conocimiento del agente, en el mejor de los casos, da sólo un **grado de creencia** en las oraciones relevantes. Nuestra principal herramienta para tratar grados de creencia es la **teoría de la probabilidad**. En terminología de la Sección 8.1, los **compromisos ontológicos** de la lógica y la teoría de la probabilidad son los mismos: el mundo se compone de hechos que son verdaderos o falsos, o que no se aplican en ningún caso particular; pero los **compromisos epistemológicos** son diferentes: un agente lógico cree que cada oración es verdadera o falsa o no tiene opinión, mientras que un agente probabilístico tiene un grado numérico de creencia entre 0 (para oraciones ciertamente falsas) y 1 (ciertamente verdaderas).

La teoría de la *probabilidad da una forma de resumir la incertidumbre que proviene de nuestra pereza e ignorancia*, y resuelve el problema de calificación. Es posible que no sepamos con certeza qué afecta a un paciente en particular, pero creemos que hay, digamos, 80 % de probabilidad; es decir, una probabilidad de 0.8 de que el dolor de dientes se deba a caries. Es decir, esperamos que, de todas las situaciones indistinguibles de la situación actual en cuanto a nuestro conocimiento, el paciente tenga caries en el 80 % de ellas. Esta creencia podría derivarse de datos estadísticos: 80 % de los pacientes con dolor de dientes revisados hasta ahora tenían caries, o de algún conocimiento dental general, o de una combinación de fuentes de evidencia.

Un punto de confusión es que, en el momento del diagnóstico, no hay incertidumbre en el mundo real: el paciente o tiene caries o no la tiene. Entonces, ¿qué significa decir que la probabilidad de caries es 0.8? ¿No debería ser 0 o 1? La respuesta es que las afirmaciones de probabilidad se hacen con respecto a un estado de conocimiento, no con respecto al mundo real. Decimos "La probabilidad de que el paciente tenga una caries, dado que tiene dolor de dientes, es 0.8". Si luego descubrimos que el paciente tiene un historial de enfermedad de las encías, podremos declarar algo diferente: "La probabilidad de que el paciente tenga caries, dado que tiene dolor de dientes y un historial de enfermedad de las encías, es 0.4". Si reunimos más pruebas concluyentes en contra de una caries, diremos "La probabilidad de que el paciente tenga caries, dado todo lo que sabemos ahora, es casi 0". Tome en cuenta que estas afirmaciones no se contradicen entre sí; cada una es una afirmación separada de un estado de conocimiento diferente.

12.1.2 Incertidumbre y decisiones racionales

Piense de nuevo el plan A^{90} para llegar al aeropuerto. Suponga que nos da un 97 % de probabilidad de alcanzar el vuelo. ¿Significa esto que es una elección racional? No necesariamente: podría haber otros planes, como A180, con probabilidades más altas. Si es vital no perder el vuelo, entonces vale la pena arriesgarse a esperar más tiempo en el aeropuerto. ¿Qué pasa con el plan A1440, que implica salir de casa 24 horas antes? En la mayoría de casos esta no es buena opción, porque, aunque casi garantiza llegar a tiempo, implica una espera enfadosa, sin mencionar una posible comida desagradable de aeropuerto.

Para tomar tales decisiones, un agente primero debe tener **preferencias** entre los diferentes **resultados** posibles de los diversos planes. Un resultado es un estado por completo especificado, que incluye factores como: si el agente llega a tiempo y la duración de la espera en el aeropuerto. Usamos la **teoría de la utilidad** para representar preferencias y razonar cuantitativamente con ellas. La teoría de la utilidad dice que cada estado (o secuencia de estados) tiene un grado de utilidad para un agente y que el agente preferirá estados con una utilidad más alta.

Preferencias

Resultados

Teoría de la utilidad

La utilidad de un estado es relativa a un agente; por ejemplo, la de un estado donde las Blancas hacen jaque a las Negras en una partida de ajedrez es alta para el agente que juega con Blancas, pero baja para el de las Negras. Pero no podemos ir sólo por las puntuaciones de 1, 1/2 y 0 que dictan las reglas del ajedrez de torneo; algunos jugadores (aun los autores) se emocionarían con un empate contra el campeón mundial, pero otros jugadores (incluido el excampeón mundial) podrían no emocionarse. No hay contabilidad para el gusto o las preferencias: verá que es extraño un agente que prefiere helado de chicle de jalapeño, y no de chispas de chocolate, pero no podrá decir que el agente es irracional. Una función de utilidad da cuenta de cualquier conjunto de preferencias, caprichosas o típicas, nobles o perversas. Tome en cuenta que la utilidad da cuenta del altruismo, aun del bienestar de otros como uno de los factores.

Las preferencias, expresadas por utilidades, se combinan con probabilidades en la teoría general de las decisiones racionales, la **teoría de la decisión**:

Teoría de la decisión

Teoría de la decisión = teoría de la probabilidad + teoría de la utilidad.

La idea fundamental de la teoría de la decisión es que *un agente es racional si y sólo si elige la acción que produce la mayor utilidad esperada, promediada sobre todos los posibles resultados de la acción*. Esto se llama **principio de máxima utilidad esperada (MEU)**. Aquí, "esperada" significa el "promedio" o "media estadística" de las utilidades de los resultados, ponderado por la probabilidad del resultado. Vimos este principio en acción en el Capítulo 5 cuando revisamos las

Principio de máxima utilidad esperada (MEU).

```
function DT-AGENT(percept) returns a action
  persistent: belief_state, Creencias probabilísticas sobre el estado actual de la action del
                mundo, la acción del agente

  actualizar belief_state basado en action y percept
  calcular las probabilidades de resultado de las acciones,
    dadas las descripciones de acciones y el belief_state actual
  seleccionar la action con la mayor utilidad esperada
    dadas las probabilidades de los resultados y la información de utilidad
  return action
```

Figura 12.1 Un agente teórico de decisiones que selecciona acciones racionales.

decisiones óptimas en el backgammon; de hecho, es un principio completamente general para la toma de decisiones de un solo agente.

La Figura 12.1 esboza la estructura de un agente que usa teoría de decisiones para seleccionar acciones. El agente es idéntico, a nivel abstracto, a los agentes descritos en los Capítulos 4 y 7 que mantienen un estado de creencia que refleja el historial de percepciones hasta la fecha. La diferencia principal es que el estado de creencia del agente de teoría de decisiones representa las *posibilidades* de estados del mundo y sus *probabilidades*. Dado el estado de creencia y cierto conocimiento sobre los efectos de las acciones, el agente hará predicciones probabilísticas sobre los resultados de las acciones y, por lo tanto, seleccionará la acción con la mayor utilidad esperada.

Los capítulos 12 y 13 se centran en la tarea de representar y calcular con información probabilística en general. El Capítulo 14 trata métodos para las tareas específicas de representar y actualizar el estado de creencia con el tiempo y predecir resultados. El Capítulo 15 examina formas de combinar la teoría de la probabilidad con lenguajes formales expresivos, como lógica de primer orden y lenguajes de programación de propósito general. El Capítulo 16 cubre teoría de la utilidad con más profundidad, y el Capítulo 17 desarrolla algoritmos para planificar secuencias de acciones en entornos estocásticos. El Capítulo 18 cubre la extensión de estas ideas a entornos multiagentes.

12.2 Notación básica de probabilidad

Para que nuestro agente represente y use información probabilística necesitamos un lenguaje formal. El lenguaje de la teoría de la probabilidad, tradicionalmente informal, ha sido escrito por matemáticos humanos para matemáticos humanos. El Apéndice A da una introducción estándar a la teoría de probabilidad elemental; aquí, usamos un enfoque adecuado a las necesidades de la IA y lo liamos con los conceptos de lógica formal.

12.2.1 De qué se trata la probabilidad

Como en las afirmaciones lógicas, las afirmaciones probabilísticas son sobre mundos posibles. Mientras que las afirmaciones lógicas indican qué mundos posibles están estrictamente descartados (aquellos en donde la afirmación es falsa), las probabilísticas hablan sobre cuán probable son los diversos mundos. En teoría de la probabilidad, el conjunto de todos los mundos posibles se llama **espacio muestral.** Los mundos posibles son *mutuamente excluyentes* y *exhaustivos*: dos mundos posibles no pueden ocurrir en simultáneo, y uno de los mundos posibles

Espacio muestral

debe ocurrir; es decir, si estamos a punto de lanzar dos dados (distinguibles), hay 36 mundos posibles para considerar: (1,1), (1,2), ..., (6,6). La letra griega Ω (omega mayúscula) se usa para referirse al espacio muestral, y ω (omega minúscula) refiere a elementos del espacio, o mundos posibles particulares.

Modelo de probabilidad

Un **modelo de probabilidad** especificado asocia una probabilidad numérica $P(\omega)$ con cada mundo posible.[1] Los axiomas básicos de la teoría de la probabilidad dicen que cada mundo posible tiene probabilidad entre 0 y 1, y que la probabilidad total del conjunto de mundos posibles es 1:

$$0 \leq P(\omega) \leq 1 \text{ para cada } \omega \text{ y } \sum_{\omega \in \Omega} P(\omega) = 1 \ . \tag{12.1}$$

Por ejemplo, si asumimos que cada dado es imparcial y los lanzamientos no interfieren entre sí, entonces cada uno de los mundos posibles (1,1), (1,2), ..., (6,6) tiene una probabilidad de 1/36. Si los dados están cargados, entonces unos mundos tendrán probabilidades más altas y otros más bajas, pero todas aún sumarán 1.

Eventos

Las afirmaciones y consultas probabilísticas no se refieren a mundos posibles particulares, sino a conjuntos de ellos. Así, podríamos preguntar por las probabilidades de que los dos dados sumen 11 o de que se obtengan dobles, etc. En teoría de la probabilidad, estos conjuntos se llaman **eventos**, un término usado mucho en el Capítulo 10 para un concepto diferente. En lógica, un conjunto de mundos corresponde a una **proposición** en un lenguaje formal; en específico, para cada proposición, el conjunto correspondiente contiene sólo los mundos posibles donde la proposición es verdadera. (Por lo tanto, "evento" y "proposición" significan casi lo mismo en este contexto, excepto que una proposición se expresa en un lenguaje formal). La probabilidad asociada con una proposición se define como la suma de probabilidades de los mundos en los que es verdadera.

$$\text{Para cada proposición } \varphi, P(\varphi) = \sum_{\omega \in \varphi} P(\omega). \tag{12.2}$$

Por ejemplo, al lanzar dados, tenemos $P(Total = 11) = P((5,6)) + P((6,5)) = 1/36 + 1/36 = 1/18$. Vea que la teoría de la probabilidad no requiere un conocimiento total de las probabilidades de cada mundo posible. Así, si creemos que los dados conspiran para dar el mismo número, podríamos afirmar que $P(dobles) = 1/4$ sin saber si los dados prefieren el doble 6 al doble 2. Igual que con las afirmaciones lógicas, esta afirmación *limita* el modelo de probabilidad subyacente sin determinarlo por completo.

Probabilidad incondicional

Probabilidad *a priori*

Evidencia

Probabilidad condicional

Probabilidad *a posteriori*

Las probabilidades como $P(Total = 11)$ y $P(dobles)$ se llaman **probabilidades incondicionales** o **probabilidades *a priori*** (y a veces sólo *a priori*); se refieren a grados de creencia en proposiciones *en ausencia de cualquier otra información*. Pero, la mayoría de las veces tenemos *alguna* información, en general llamada **evidencia**, que ya ha sido revelada. Así, el primer dado puede mostrar un 5 y esperamos que el otro deje de girar. En ese caso, no estamos interesados en la probabilidad incondicional de obtener dobles, sino en la probabilidad **condicional** o ***a posteriori*** (o sólo *a posteriori*) de obtener dobles *dado que el primer dado es un 5*. Esta probabilidad se escribe como $P(dobles|Dado_1 = 5)$, donde el "|" se pronuncia "dado".[2]

De manera similar, si voy al dentista para un chequeo programado con regularidad, entonces la probabilidad *a priori* $P(caries)$=0.2 sería de interés; pero si voy al dentista porque tengo dolor de dientes, es la probabilidad condicional $P(caries|dolorDeDientes)$=0.6 la que importa.

---

[1] Por ahora, suponemos un conjunto de mundos discreto y contable. El tratamiento adecuado del caso continuo conlleva ciertas complicaciones que son menos relevantes para la mayoría de los propósitos de la IA.

[2] Piense que la precedencia de "|" es tal que toda expresión de la forma $P(...|...)$ siempre significa $P((...)|(...))$.

Debe entender que $P(caries)=0.2$ es *válida* aún después de acreditar *dolor de dientes*; cosa que no es muy útil. Al tomar decisiones, un agente necesita condicionar *toda* la evidencia reunida. También se debe entender la diferencia entre condicionamiento y consecuencia lógica. La afirmación $P(caries|dolorDeDientes)=0.6$ no significa "Siempre que el dolor de dientes sea *verdadero*, concluya que la *caries* es verdadera con una probabilidad de 0.6", sino que "Siempre que el dolor de dientes sea *verdadero* y *no tengamos más información*, concluya que la *caries* es verdadera con una probabilidad de 0.6". La condición adicional es importante; es decir, si tuviéramos la información adicional de que el dentista no encontró caries, en definitiva no concluiríamos que la caries es verdadera con una probabilidad de 0.6; en su lugar, necesitamos usar $P(caries|dolorDeDientes \wedge \neg caries) = 0$.

Matemáticamente hablando, las probabilidades condicionales se definen en términos de probabilidades incondicionales de la siguiente manera: para cualquier proposición a y b, tenemos

$$P(a|b) = \frac{P(a \wedge b)}{\mathbf{P}(b)}, \tag{12.3}$$

que se cumple siempre que $P(b) > 0$. Por ejemplo,

$$P(doubles|Die_1{=}5) = \frac{P(doubles \wedge Die_1{=}5)}{P(Die_1{=}5)}.$$

La definición tiene sentido si se recuerda que observar b descarta todos los mundos posibles donde b es falso, dejando un conjunto cuya probabilidad total es $P(b)$. Dentro de ese conjunto, los mundos donde a es verdadero deben satisfacer $a \wedge b$ y constituir una fracción $P(a \wedge b)/P(b)$.

La definición de probabilidad condicional, Ecuación (12.3), puede escribirse en una forma diferente llamada **regla del producto**:

Regla del producto

$$P(a \wedge b){=}P(a|b)P(b). \tag{12.4}$$

La regla del producto es quizá más fácil de recordar: nace del hecho de que para que a y b sean verdaderos, necesitamos que b sea verdadero, y que a sea verdadero dado b.

12.2.2 El lenguaje de las proposiciones en las afirmaciones de probabilidad

En este capítulo y el siguiente, las proposiciones que describen conjuntos de mundos posibles suelen escribirse en notación que combina elementos de lógica proposicional y notación de satisfacción de restricciones. En la terminología de la Sección 2.4.7, es una **representación factorizada**, donde un mundo posible se representa por un conjunto de pares variable/valor. Una **representación estructurada** más expresiva también es posible, como muestra el Capítulo 15.

Variables aleatorias

Las variables en teoría de la probabilidad son **variables aleatorias**, y sus nombres comienzan con mayúscula. Así, en el ejemplo de los dados, *Total* y $Dado_1$ son variables aleatorias. Cada variable aleatoria es una función que mapea desde el dominio de los mundos posibles Ω a algún **rango**: el conjunto de valores posibles que puede tomar. El rango de *Total* para dos dados es el conjunto $\{2,...,12\}$ y el rango de $Dado_1$ es $\{1,...,6\}$. Los nombres de los valores siempre son en minúsculas, así podríamos escribir $\sum_x P(X{=}x)$ para sumar sobre los valores de X. Una variable aleatoria booleana tiene el rango $\{verdadero,falso\}$. Por ejemplo, la proposición se lanzan dobles se escribe como $Dobles{=}verdadero$. (Una alternativa para el rango de variables booleanas es el conjunto $\{0,1\}$, en cuyo caso se dice que la variable tiene distribución de **Bernoulli**). Por convención, las proposiciones de la forma $A{=}verdadero$ se abrevian como a, mientras que $A{=}falso$ se abrevia como $\neg a$. (Los usos de *dobles*, *caries* y *dolorDeDientes* en la sección previa son abreviaturas de este tipo).

Rango

Bernoulli

Los rangos pueden ser conjuntos de tokens arbitrarios. Podríamos elegir el rango *Edad* como el conjunto $\{juvenil,adolescente,adulto\}$ y el rango *Clima* sería $\{sol,lluvia,nube,nieve\}$. Cuando

no es posible ninguna ambigüedad, es común usar un valor por sí mismo para representar la proposición de que una variable particular tiene ese valor; así, sol puede representar *Clima = sol*.[3]

Los ejemplos previos tienen rangos finitos. Las variables también permiten rangos infinitos, sean discretos (los números enteros) o continuos (los reales). Para toda variable con un rango ordenado, también se permiten desigualdades, como $NúmeroDeÁtomosEnElUniverso \geq 10^{70}$.

Finalmente, podemos combinar estos tipos de proposiciones elementales (incluidas las formas abreviadas para variables booleanas) usando los conectivos de lógica proposicional. Por ejemplo, podemos expresar "La probabilidad de que el paciente tenga una caries, dado que es un adolescente sin dolor de dientes, es 0.1" de la siguiente manera:

$$P(caries | \neg dolorDeDientes \wedge adolescente) = 0.1.$$

En la notación de probabilidad, también es común usar una coma para la conjunción, por lo que podríamos escribir

$$P(caries | \neg dolorDeDientes, adolescente).$$

A veces querremos hablar sobre las probabilidades de todos los valores posibles de una variable aleatoria. Podríamos escribir:

$$P(Clima = sol) = 0.6$$
$$P(Clima = lluvia) = 0.1$$
$$P(Clima = nube) = 0.29$$
$$P(Clima = nieve) = 0.01,$$

pero como abreviatura permitiremos

$$\mathbf{P}(Clima) = (0.6, 0.1, 0.29, 0.01),$$

donde la **P** en negrita indica que el resultado es un vector de números, y donde asumimos un orden predefinido (*sol,lluvia,nube,nieve*) en el rango de *Clima*. Decimos que la declaración **P** define una **distribución de probabilidad** para la variable aleatoria *Clima*, es decir, una asignación de una probabilidad para cada valor posible de la variable aleatoria. (En este caso, con un rango finito y discreto, la distribución se llama **distribución categórica**). La notación **P** también se usa para distribuciones condicionales: $\mathbf{P}(X|Y)$ da los valores de $P(X = x_i | Y = y_j)$ para cada par i, j posible.

Distribución de probabilidad

Distribución categórica

Para variables continuas, no es posible escribir toda la distribución como un vector, porque hay infinitos valores. En su lugar, podemos definir la probabilidad de que una variable aleatoria tome algún valor x como una función parametrizada de x, en general llamada **función de densidad de probabilidad**. Por ejemplo, la sentencia

Función de densidad de probabilidad

$$P(TemperaturaDelMediodía = x) = Uniforme(x; 18C, 26C)$$

expresa la creencia de que la temperatura al mediodía está distribuida de manera uniforme entre 18 y 26 grados Celsius.

Las funciones de densidad de probabilidad (o **pdfs**) difieren en significado de las distribuciones discretas. Decir que la densidad de probabilidad es uniforme de 18 °C a 26 °C significa que hay 100 % de probabilidad de que la temperatura baje en parte de esa región de 8 °C de ancho y 50 % de probabilidad de que baje en cualquier subregión de 4 °C de ancho. Escribimos la densidad de probabilidad para una variable aleatoria continua X en el valor x como $P(X = x)$ o $P(x)$; la

[3] Estas convenciones tomadas en conjunto conducen a una posible ambigüedad en la notación al sumar valores de una variable booleana: $P(a)$ es la probabilidad de que A sea verdadero, mientras que en la expresión $\sum_a P(a)$ sólo refiere la probabilidad de cada valor a de A.

definición intuitiva de $P(x)$ es la probabilidad de que X caiga en una región arbitrariamente pequeña que comienza en x, dividida por el ancho de la región:

$$P(x) = \lim_{dx \to 0} P(x \leq X \leq x + dx)/dx.$$

Para *TemperaturaDelMediodía* tenemos

$$P(TemperaturaDelMediodía = x) = Uniforme(x;18C,26C) = \begin{cases} \frac{1}{8C} \text{ if } 18C \leq x \leq 26C \\ 0 \text{ otro caso} \end{cases},$$

donde C representa centígrados (no una constante). En $P(TemperaturaDelMediodía = 20.18C) = \frac{1}{8C}$, vea que 20.18C no es probabilidad, es densidad de probabilidad. La probabilidad de que *temperaturaDelMediodía* sea 20.18C *exacto* es cero, porque 20.18C es una región de ancho 0. Algunos autores usan símbolos diferentes para probabilidades discretas y densidades de probabilidad; nosotros usamos P para valores de probabilidad específicos y **P** para vectores de valores en ambos casos, pues rara vez surge confusión y las ecuaciones suelen ser idénticas. Vea que las probabilidades son números adimensionales, mientras que las funciones de densidad se miden con la unidad recíproco de grados centígrados. Si el mismo intervalo de temperatura se expresara en grados Fahrenheit, tendría un ancho de 14.4 grados, y la densidad sería 1/14.4F.

Además de las distribuciones en variables individuales, necesitamos notación para distribuciones en múltiples variables. Se usan comas para ello. Así, $\boldsymbol{P}(Clima,Caries)$ denota las probabilidades de todas las combinaciones de los valores de *Clima* y *Caries*. Esto es una tabla de 4×2 de probabilidades llamada **distribución conjunta de probabilidad** de *Clima* y *Caries*. También podemos combinar variables y valores específicos; $\boldsymbol{P}(sol\ Caries)$ sería un vector de dos elementos que da las probabilidades de una caries con un día soleado y ninguna con un día soleado.

Distribución conjunta de probabilidad

La notación **P** hace que ciertas expresiones sean mucho más concisas de lo que podrían ser de otra manera. Por ejemplo, las reglas del producto (ver Ecuación (12.4)) para todos los valores posibles de *Clima* y *Caries* se pueden escribir como una sola ecuación:

$$\mathbf{P}(Clima,Caries) = \mathbf{P}(Clima|Caries)\boldsymbol{P}(Caries),$$

en lugar de estas 4×2 = 8 ecuaciones (con abreviaturas W y C):

$$\begin{aligned}
&P(W = sol \wedge C = verdadero) = P(W = sol|C = verdadero)\ P(C = verdadero)\\
&P(W = lluvia \wedge C = verdadero) = P(W = lluvia|C = verdadero)\ P(C = verdadero)\\
&P(W = nube \wedge C = verdadero) = P(W = nube|C = verdadero)\ P(C = verdadero)\\
&P(W = nieve \wedge C = verdadero) = P(W = nieve|C = verdadero)\ P(C = verdadero)\\
&P(W = sol \wedge C = falso) = P(W = sol|C = falso)\ P(C = falso)\\
&P(W = lluvia \wedge C = falso) = P(W = lluvia|C = falso)\ P(C = falso)\\
&P(W = nube \wedge C = falso) = P(W = nube|C = falso)\ P(C = falso)\\
&P(W = nieve \wedge C = falso) = P(W = nieve|C = falso)\ P(C = falso).
\end{aligned}$$

Como caso degenerado, $\mathbf{P}(sol,caries)$ no tiene variables y, por lo tanto, es un vector de dimensión cero, que podemos pensar como un valor escalar.

Ya definimos una sintaxis para proposiciones y afirmaciones de probabilidad y dimos parte de la semántica: la Ecuación (12.2) define la probabilidad de una proposición como suma de probabilidades de mundos donde se cumple. Para completar la semántica, necesitamos decir cuáles son los mundos y cómo determinar si una proposición se cumple en un mundo. Tomamos prestada esta parte directamente de la semántica de la lógica proposicional, así: *Se define un mundo posible como una asignación de valores a todas las variables aleatorias en consideración*.

Es fácil ver que esta definición satisface el requisito básico de que los mundos posibles sean mutuamente excluyentes y exhaustivos (Ejercicio 12.EXEX). Así, si las variables aleatorias son

Caries, *DolorDeDientes* y *Clima*, entonces hay 2×2×4 = 16 mundos posibles. Además, la verdad de cualquier proposición dada se determina fácilmente en esos mundos mediante el mismo cálculo recursivo de verdad usado para la lógica proposicional (ver página 218).

Vea que unas variables aleatorias pueden ser redundantes, en el sentido de que sus valores pueden obtenerse en todos los casos a partir de los valores de otras variables. Es decir, la variable *Dobles* en el mundo de dos dados es verdadera exactamente si $Dado_1 = Dado_2$. Incluir *Dobles* como una de las variables aleatorias, además de $Dado_1$ y $Dado_2$, parece aumentar el número de mundos posibles de 36 a 72 y, claro, exactamente la mitad de 72 serán lógicamente imposibles y tendrán probabilidad de 0.

De la definición de mundos posibles previa se sigue que un modelo de probabilidad está por completo determinado por la distribución conjunta para toda variable aleatoria, o **distribución de probabilidad conjunta completa**. Así, dada *Caries*, *DolorDeDientes* y *Clima*, la distribución conjunta completa es $\mathbf{P}(Cavidad, DolorDeDientes, Clima)$. Esta distribución conjunta se representa como una tabla de 2 × 2 × 4 con 16 entradas. Como la probabilidad de cada proposición es una suma sobre los mundos posibles, una distribución conjunta completa es suficiente, en principio, para calcular la probabilidad de cualquier proposición. Veremos ejemplos de cómo hacer esto en la Sección 12.3.

Distribución de probabilidad conjunta complet

12.2.3 Axiomas de probabilidad y su razonabilidad

Los axiomas básicos de la probabilidad (Ecuaciones (12.1) y (12.2)) implican ciertas relaciones entre los grados de creencia que se pueden otorgar a proposiciones lógicamente relacionadas. Por ejemplo, podemos derivar la relación familiar entre la probabilidad de una proposición y la probabilidad de su negación:

$$\begin{aligned} P(\neg a) &= \textstyle\sum_{\omega \in \neg a} P(\omega) && \text{por la Ecuación (12.2)} \\ &= \textstyle\sum_{\omega \in \neg a} P(\omega) + \sum_{\omega \in a} P(\omega) - \sum_{\omega \in a} P(\omega) \\ &= \textstyle\sum_{\omega \in \Omega} P(\omega) - \sum_{\omega \in a} P(\omega) && \text{agrupando los dos primeros términos} \\ &= 1 - P(a) && \text{por la (12.1) y (12.2).} \end{aligned}$$

También podemos derivar la fórmula bien conocida para la probabilidad de una disyunción, a veces llamada **principio de inclusión-exclusión**:

Principio de inclusión-exclusión

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b). \tag{12.5}$$

Esta regla se recuerda al notar que los casos en los que a se cumple, junto con los casos en los que b se cumple, ciertamente cubren todos los casos en los que $a \vee b$ se cumple; pero sumar los dos conjuntos de casos cuenta su intersección dos veces, por lo que necesitamos restar $P(a \wedge b)$.

Axiomas de Kolmogorov

Las Ecuaciones (12.1) y (12.5) suelen llamarse **axiomas de Kolmogorov** en honor al matemático Andrei Kolmogorov, quien mostró cómo construir el resto de la teoría de la probabilidad a partir de esta base simple y cómo manejar las dificultades causadas por variables continuas[4]. Mientras que la Ecuación (12.2) tiene un enfoque definicional, la Ecuación (12.5) revela que los axiomas realmente restringen los grados de creencia que un agente podría tener respecto a proposiciones relacionadas de manera lógica. Esto es análogo al hecho de que un agente lógico no puede creer en simultáneo en A, B y $\neg(A \wedge B)$, porque no hay un mundo posible donde los tres sean verdaderos. Sin embargo, con probabilidades, las afirmaciones no se refieren de manera directa al mundo, sino al estado de conocimiento del agente. Entonces, ¿por qué un agente es capaz de sostener el siguiente conjunto de creencias (aunque violen los axiomas de Kolmogorov)?

$$P(a)=0.4 \qquad P(b)=0.3 \qquad P(a \wedge b)=0.0 \qquad P(a \vee b)=0.8 \tag{12.6}$$

[4] Las dificultades incluyen el **conjunto de Vitali**, un subconjunto bien definido del intervalo [0, 1] pero no su tamaño.

Proposition	Agent 1's belief	Agent 2 bets	Agent 1 bets	Agent 1 payoffs for each outcome			
				a,b	a,¬b	¬a,b	¬a,¬b
a	0.4	\$4 en a	\$6 en $\neg a$	\$6	−\$6	\$4	\$4
b	0.3	\$3 en b	\$7 en $\neg b$	\$7	\$3	−\$7	\$3
$a \vee b$	0.8	\$2 en $\neg(a \vee b)$	\$8 en $a \vee b$	\$2	\$2	\$2	−\$8
				−\$11	−\$1	−\$1	−\$1

Figura 12.2 Debido a que el agente 1 tiene creencias inconsistentes, el agente 2 puede diseñar un conjunto de tres apuestas que garantiza pérdida para el agente 1, sin importar el resultado de a y b.

Este tipo de pregunta ha sido objeto de décadas de intenso debate entre aquellos que defienden el uso de probabilidades como la única forma legítima para los grados de creencia y aquellos que defienden enfoques alternativos.

El siguiente argumento a favor de los axiomas de la probabilidad, establecido en 1931 por Bruno de Finetti (ver de Finetti, 1993): Si un agente tiene grado de creencia en una proposición a, entonces éste debería poder establecer probabilidades donde le sea indiferente apostar a favor o en contra de a.[5] Piénselo como un juego entre dos agentes: El agente 1 declara, "mi grado de creencia en el evento 'a' es 0.4". Luego, el agente 2 es libre de elegir si apostar a favor o en contra de a con apuestas consistentes con el grado de creencia declarado. Es decir, el agente 2 podría optar por aceptar la apuesta del agente 1 de que ocurrirá a, ofreciendo \$6 contra los \$4 del agente 1. O el agente 2 podría aceptar la apuesta del agente 1 de que $\neg a$ ocurrirá, ofreciendo \$4 contra los \$6 del agente 1. Luego vemos el resultado de a, y quien tenga razón recoge el dinero. Si los grados de creencia de uno no reflejan con precisión el mundo, entonces uno esperaría perder dinero a largo plazo frente a un agente adversario cuyas creencias reflejen mejor el estado del mundo.

El teorema de De Finetti no se preocupa por elegir los valores correctos para las probabilidades individuales, sino por elegir valores para las probabilidades de proposiciones lógicamente relacionadas: *Si el agente 1 expresa un conjunto de grados de creencia que violan los axiomas de la teoría de la probabilidad, entonces hay una combinación de apuestas por parte del agente 2 que garantiza que el agente 1 perderá dinero cada vez.* Así, suponga que el agente 1 tiene el conjunto de grados de creencia de la Ecuación (12.6). La Figura 12.2 muestra que si el agente 2 elige apostar \$4 en a, \$3 en b y \$2 en $\neg(a \vee b)$, entonces el agente 1 siempre pierde dinero, independiente de los resultados de a y b. El teorema de De Finetti implica que ningún agente racional permite tener creencias que violen los axiomas de la probabilidad.

Una objeción común al teorema de De Finetti es que este juego de apuestas es bastante artificial. Por ejemplo, ¿qué sucede si uno se niega a apostar? ¿Termina el argumento? La respuesta es que el juego de apuestas es un modelo abstracto para la situación de toma de decisiones en la que cada agente está de manera inevitable involucrado en todo momento. Cada acción (incluida la inacción) es una especie de apuesta, y cada resultado puede ser visto como un pago de la apuesta. Negarse a apostar es como negarse a permitir que pase el tiempo.

Otros argumentos filosóficos sólidos se han presentado a favor del uso de probabilidades, en especial los de Cox (1946), Carnap (1950) y Jaynes (2003). Cada uno construye un conjunto de

[5] Se podría argumentar que las preferencias del agente por diferentes saldos bancarios son tales que la posibilidad de perder 1 dólar no es contrarrestada por una posibilidad igual de ganar 1 dólar. Una posible respuesta es que los montos de las apuestas sean tan pequeños para evitar este problema. El análisis de Savage (1954) evita esta cuestión.

axiomas para razonar con grados de creencias: no contradicciones, correspondencia con la lógica ordinaria (por ejemplo, si la creencia en A aumenta, entonces la creencia en $\neg A$ debe disminuir), etc. El único axioma controvertido es que los grados de creencia deben ser números, o al menos actuar como números en el sentido de que deben ser transitivos (si la creencia en A es mayor que la creencia en B, que es mayor que la creencia en C, entonces la creencia en A debe ser mayor que C) y comparables (la creencia en A debe ser igual a, mayor que o menor que la creencia en B). Luego es posible demostrar que la probabilidad es el único enfoque que satisface estos axiomas.

Siendo el mundo como es, las demostraciones prácticas a veces hablan más que las pruebas. El éxito de los sistemas de razonamiento basados en la teoría de la probabilidad ha sido mucho más efectivo que los argumentos filosóficos para ganar adeptos. Ahora veremos cómo desplegar los axiomas para realizar inferencias.

12.3 Inferencia utilizando distribuciones conjuntas completas

En esta sección describimos un método simple para la **inferencia probabilística**, es decir, el cálculo de probabilidades *a posteriori* para proposiciones de **consulta** dadas evidencias observadas. Usamos la distribución conjunta completa como la "base de conocimiento" a partir de la cual se pueden derivar respuestas a todas las preguntas. En el camino, también introducimos técnicas útiles para manipular ecuaciones que involucran probabilidades.

Inferencia probabilística
Consulta

Comenzamos con un ejemplo simple: un dominio que consta sólo de las tres variables booleanas *DoloDeDientes*, *Caries* y *Atrapar* (la sonda de acero brusca del dentista se queda atrapada en mi diente). La distribución conjunta completa es una tabla de 2×2×2 como se ve en la Figura 12.3.

	dolorDeDientes		*¬dolorDeDientes*	
	atrapar	*¬atrapar*	*atrapar*	*¬atrapar*
caries	0.108	0.012	0.072	0.008
¬caries	0.016	0.064	0.144	0.576

Figura 12.3 Distribución conjunta completa para el mundo *DolorDeDientes*, *Caries* y *Atrapar*.

Vea que las probabilidades en la distribución conjunta suman 1, como requieren los axiomas de probabilidad. También note que la Ecuación (12.2) brinda una manera directa de calcular la probabilidad de toda proposición, simple o compleja: identifique los mundos posibles en los que la proposición es verdadera y sume sus probabilidades. Por ejemplo, hay seis mundos posibles donde $caries \vee dolorDeDientes$ se cumple:

$$P(caries \vee dolorDeDientes) = 0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28.$$

Una tarea en particular común es extraer la distribución sobre algún subconjunto de variables o una sola variable. Por ejemplo, sumar las entradas en la primera fila da la probabilidad incondicional o **marginal**[6] de caries:

Probabilidad marginal

$$P(caries) = 0.108 + 0.012 + 0.072 + 0.008 = 0.2.$$

[6] Llamada así debido a una práctica común entre los actuarios de escribir las sumas de las frecuencias observadas en los márgenes de las tablas de seguros.

Marginalización

Este proceso se llama **marginalización, o suma total**, porque sumamos las probabilidades para cada valor posible de las otras variables, y las saca de la ecuación. Podemos escribir la esta regla de marginalización general para cualquier conjunto de variables **Y** y **Z:**

$$\mathbf{P}(\mathbf{Y}) = \sum_{\mathbf{z}} \mathbf{P}(\mathbf{Y},\mathbf{Z}=\mathbf{z}), \tag{12.7}$$

Donde $\sum_z$ suma sobre todas las combinaciones posibles de valores del conjunto de variables **Z**. Como es habitual, podemos abreviar **P(Y,Z=z)** en esta ecuación como **P(Y,z)**. Para el ejemplo de *Caries*, la Ecuación (12.7) corresponde a:

$$\begin{aligned}\boldsymbol{P}(Caries) &= \boldsymbol{P}(Caries,dolorDeDientes,atrapar) + \boldsymbol{P}(Caries,dolorDeDientes,\neg atrapar)\\ &+ \boldsymbol{P}(Caries,\neg dolorDeDientes,atrapar)+\boldsymbol{P}(Caries,\neg dolorDeDientes,\neg atrapar)\\ &= (0.108,0.016) + (0.012,0.064) + (0.072,0.144) + (0.008,0.576)\\ &= (0.2,0.8).\end{aligned}$$

Condicionamiento

Usando la regla del producto (Ecuación (12.4)), podemos reemplazar **P(Y,z)** en la Ecuación (12.7) por **P(Y|z)P(z)**, Y obtener la regla **condicionamiento**:

$$\mathbf{P}(\mathbf{Y}) = \sum_{\mathbf{z}} \mathbf{P}(\mathbf{Y}|\mathbf{z})P(\mathbf{z}). \tag{12.8}$$

La marginalización y el condicionamiento resultan reglas útiles para todo tipo de derivaciones que implican expresiones de probabilidad.

En la mayoría de casos, nos interesa calcular probabilidades *condicionales* de unas variables, dadas evidencias sobre otras. Las probabilidades condicionales se encuentran usando la Ecuación (12.3) para obtener una expresión en términos de probabilidades incondicionales y, luego, evaluar la expresión a partir de la distribución conjunta completa. Es decir, podemos calcular la probabilidad de una caries, dada la evidencia de un dolor de dientes, de la siguiente manera:

$$\begin{aligned}P(caries|dolorDeDientes) &= \frac{P(caries \wedge dolorDeDientes)}{P(dolorDeDientes)}\\ &= \frac{0.108+0.012}{0.108+0.012+0.016+0.064} = 0.6.\end{aligned}$$

Sólo para comprobar, podemos calcular la probabilidad de que no haya caries, dado un dolor de dientes:

$$\begin{aligned}P(\neg caries|dolorDeDientes) &= \frac{P(\neg caries \wedge dolorDeDientes)}{P(dolorDeDientes)}\\ &= \frac{0.016+0.064}{0.108+0.012+0.016+0.064} = 0.4.\end{aligned}$$

Los dos valores suman 1.0, como deberían. Vea que el término $P(dolorDeDientes)$ está en el denominador para ambos cálculos. Si la variable *Caries* tuviera más de dos valores, estaría en el denominador para todos ellos. De hecho, se puede ver como una constante de **normalización** para la distribución $\mathbf{P}(Caries|DolorDeDientes)$, asegurando que sume 1. En los capítulos sobre probabilidad, usamos α para denotar esas constantes. Con esta notación, escribimos las dos ecuaciones previas en una:

$$\begin{aligned}&\mathbf{P}(Caries|dolorDeDientes) = \alpha\mathbf{P}(Cavidad,dolorDeDientes)\\ &\quad = \alpha[\mathbf{P}(Caries,dolorDeDientes) + \mathbf{P}(Caries,dolorDeDientes,\neg atrapar)]\\ &\quad = \alpha[(0.108,0.016) + (0.012,0.064)] = \alpha(0.12,0.08) = (0.6,0.4).\end{aligned}$$

En otras palabras, podemos calcular $\mathbf{P}(Caries|dolorDeDientes)$ aun si no conocemos el valor de $P(dolorDeDientes)$. Temporalmente olvidamos el factor $1/P(dolorDeDientes)$ y sumamos los valores para *caries* y $\neg caries$, obteniendo 0.12 y 0.08. Esas son las proporciones relativas correctas, pero no suman 1, por ello las normalizamos dividiendo cada una por 0.12 + 0.08, y obtenemos las verdaderas probabilidades de 0.6 y 0.4. La normalización es un atajo útil en muchos cálculos de probabilidad, para facilitar la computación y para permitirnos proceder cuando una evaluación de probabilidad (como $P(dolorDeDientes)$) no está disponible.

A partir del ejemplo, podemos extraer un procedimiento de inferencia general. Comenzamos con el caso en el que la consulta involucra una sola variable, X (*Caries* en el ejemplo). Sea **E** la lista de variables de evidencia (sólo *DolorDeDientes*), sea **e** la lista de valores observados para ellas, y sea **Y** las variables no observadas restantes (sólo *Atrapar* en el ejemplo). La consulta es $\mathbf{P}(X|\mathbf{e})$ y puede ser evaluada como

$$\mathbf{P}(X|\mathbf{e}) = \alpha\mathbf{P}(X,\mathbf{e}) = \alpha\sum_{\mathbf{y}}\mathbf{P}(X,\mathbf{e},\mathbf{y}), \tag{12.9}$$

donde la suma se realiza sobre todos los **y**s posibles (es decir, las posibles combinaciones de valores de las variables no observadas **Y**). Vea que juntas, las variables X, **E** y **Y** constituyen el conjunto completo de variables para el dominio, por ello $\mathbf{P}(X,\mathbf{e},\mathbf{y})$ es subconjunto de probabilidades de la distribución conjunta completa.

Dada la distribución conjunta completa con la que trabajar, la Ecuación (12.9) responde consultas probabilísticas para variables discretas. Pero no escala bien: para un dominio descrito por n variables booleanas, requiere una tabla de entrada de tamaño $O(2^n)$ y toma $O(2^n)$ tiempo procesar la tabla. En un problema realista, podríamos tener $n = 100$, lo que hace que $O(2^n)$ sea impráctico: ¡una tabla con $2^{100} \approx 10^{30}$ entradas! El problema real no es sólo de memoria y computación: es que si cada una de las 10^{30} probabilidades tiene que ser estimada por separado a partir de ejemplos, la cantidad de ejemplos requeridos será astronómica.

Por estas razones, la distribución conjunta completa en forma de tabla rara vez es una herramienta práctica para construir sistemas de razonamiento. En su lugar, debería ser vista como la base teórica sobre la cual se construyen enfoques más efectivos, al igual que las tablas de verdad formaron una base teórica para algoritmos más prácticos como DPLL en el Capítulo 7. El resto de este capítulo introduce ideas básicas requeridas en preparación para el desarrollo de sistemas realistas en el Capítulo 13.

12.4 Independencia

Expandamos la distribución conjunta completa en la Figura 12.3 agregando una cuarta variable, *Weather*. La distribución conjunta completa entonces se convierte en $\mathbf{P}(DolorDeDientes,Atrapar,Caries,EstadoDelTiempo)$, que tiene $2\times2\times2\times4 = 32$ entradas. Contiene cuatro "ediciones" de la tabla de la Figura 12.3, una para cada estado del tiempo. ¿Qué relación tienen estas ediciones entre sí y con la tabla original de tres variables? ¿Cómo se relaciona el valor de $P(dolorDeDientes,atrapar,caries,nube)$ con el valor de $P(dolorDeDientes,atrapar,caries)$? Podemos usar la regla del producto (Ecuación (12.4)):

$$P(dolorDeDientes,atrapar,caries,nube)$$
$$= P(nube|dolorDeDientes,atrapar,caries)\, P(dolorDeDientes,atrapar,caries).$$

Y, a menos que uno sea una deidad, los problemas dentales no deberían influir en el clima. Al menos para la odontología en interiores, es seguro decir que el estado del tiempo no influye en las variables dentales. Por lo tanto, la siguiente afirmación parece razonable:

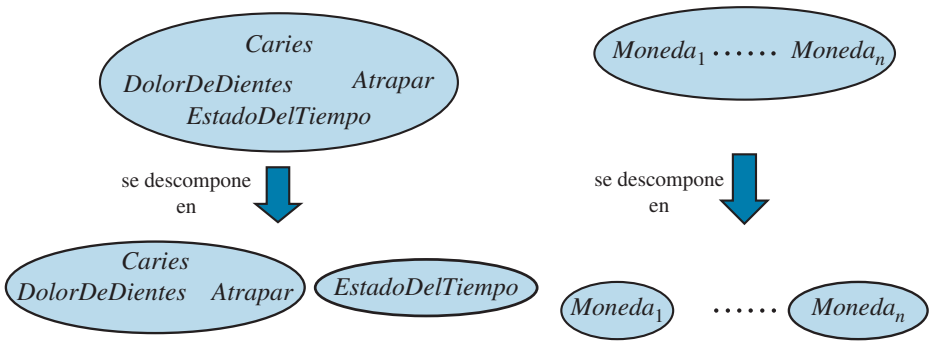


Figura 12.4 Dos ejemplos de factorización de una distribución conjunta grande en distribuciones más pequeñas, usando independencia absoluta. (a) Los problemas climáticos y dentales son independientes. (b) Los lanzamientos de monedas son independientes.

$$P(nube|dolorDeDientes,atrapar,caries) = P(nube). \tag{12.10}$$

A partir de esto, podemos deducir

$$P(dolorDeDientes,atrapar,caries,nube)=P(nube)P(dolorDeDientes,atrapar,\ caries).$$

Una ecuación similar existe para *cada entrada* en $\mathbf{P}(DolorDeDientes,Atrapar,Caries,EstadoDelTiempo)$. De hecho, podemos escribir la ecuación general

$$\boldsymbol{P}(DolorDeDientes,Atrapar,Caries,EstadoDelTiempo)=\mathbf{P}(DolorDeDientes,Atrapar,Caries)\mathbf{P}(EstadoDelTiempo).$$

Así, la tabla de 32 elementos para cuatro variables se construye a partir de una tabla de 8 elementos y una de 4. Esta descomposición se ilustra de manera esquemática en la Figura 12.4(a).

Independencia

La propiedad que usamos en la Ecuación (12.10) se llama **independencia** (o **independencia marginal** o **independencia absoluta**). En particular, el clima es independiente de los problemas dentales. La independencia entre las proposiciones a y b se escribe como

$$P(a|b) = P(a) \quad \text{o} \quad P(b|a) = P(b) \quad \text{o} \quad P(a \wedge b) = P(a)P(b). \tag{12.11}$$

Todas estas formas son equivalentes (Ejercicio 12.INDI). La independencia entre las variables X e Y se escribe como (de nuevo, todas son equivalentes):

$$\mathbf{P}(X|Y) = \mathbf{P}(X) \quad \text{o} \quad \mathbf{P}(Y|X) = \mathbf{P}(Y) \quad \text{o} \quad \mathbf{P}(X,Y) = \mathbf{P}(X)\mathbf{P}(Y).$$

Las afirmaciones de independencia se basan en conocimiento del dominio. Como ilustra el ejemplo de dolor de dientes y clima, pueden reducir mucho la cantidad de información necesaria para especificar la distribución conjunta completa. Si el conjunto completo de variables se divide en subconjuntos independientes, entonces la distribución conjunta completa se *factoriza* en distribuciones conjuntas separadas en esos subconjuntos. Así, la distribución conjunta completa sobre el resultado de n lanzamientos de moneda independientes, $\mathbf{P}(C_1,...,C_n)$, tiene 2^n entradas, pero se puede representar como el producto de n distribuciones de una sola variable $\mathbf{P}(C_i)$. En sentido práctico, la independencia entre odontología y meteorología es buena, porque si no, la práctica odontológica requeriría conocimiento íntimo de meteorología, y viceversa.

Si están disponibles, las afirmaciones de independencia permiten ayudar a reducir el tamaño de la representación del dominio y la complejidad del problema de inferencia. Pero, la separación clara de conjuntos enteros de variables por independencia es muy rara. Siempre que exista una conexión entre dos variables, por más indirecta, la independencia no se cumplirá. Además, aun los subconjuntos independientes pueden ser muy grandes, por ejemplo, la odontología podría involucrar docenas de enfermedades y cientos de síntomas, los cuales están interrelacionados. Para manejar esos problemas, necesitamos métodos más sutiles que el mero concepto de independencia.

12.5 Regla de Bayes y su uso

En la página 390 definimos la regla del producto (Ecuación (12.4)). De hecho, es posible escribirse en dos formas:

$$P(a \wedge b) = P(a|b)P(b) \quad \text{y} \quad P(a \wedge b) = P(b|a)P(a).$$

Igualando los dos lados derechos y dividiendo por $P(a)$, obtenemos

$$P(a|b) = \frac{P(a|b)P(b)}{P(a)}. \tag{12.12}$$

Esta ecuación se conoce como **regla de Bayes** (o ley de Bayes o teorema de Bayes). Esta ecuación simple subyace a la mayoría de los sistemas de IA modernos para inferencia probabilística. Regla de Bayes

El caso más general de la regla de Bayes para variables multivaluadas se escribe en la notación **P** como sigue:

$$\mathbf{P}(\mathrm{Y} \,|\, \mathrm{X}) = \frac{\mathbf{P}(\mathrm{X} \,|\, \mathrm{Y})\mathbf{P}(\mathrm{Y})}{\mathbf{P}(\mathrm{X})}.$$

Como antes, esto se toma como representación de un conjunto de ecuaciones, cada una tratando con valores específicos de las variables. También tendremos ocasión de usar una versión más general condicionada a cierta evidencia de fondo **e:**

$$\mathbf{P}(\mathrm{Y} \,|\, \mathrm{X}, \mathbf{e}) = \frac{(\mathbf{P}(\mathrm{X} \,|\, \mathrm{Y}, \mathbf{e})\mathbf{P}(\mathrm{Y}|\mathbf{e})}{\mathbf{P}(\mathrm{X}|\mathbf{e})}. \tag{12.13}$$

12.5.1 Aplicando la regla de Bayes: El caso simple

A simple vista, la regla de Bayes no parece muy útil. Nos permite calcular el término único $P(b|a)$ en términos de tres términos: $P(a|b)$, $P(b)$ *y* $P(a)$. Eso parece dar dos pasos hacia atrás; pero la regla de Bayes es útil en la práctica porque hay muchos casos en los que tenemos buenas estimaciones de probabilidad para estos tres números y necesitamos calcular el cuarto. Con frecuencia, percibimos como evidencia el *efecto* de alguna causa desconocida y nos gustaría determinar su *causa*. En ese caso, la regla de Bayes se convierte en

$$P(causa \,|\, efecto) = \frac{P(efecto|causa)P(causa)}{P(efecto)}.$$

La probabilidad condicional $P(efecto|causa)$ cuantifica la relación en la dirección **causal**, mientras que $P(causa|efecto)$ describe la dirección **diagnóstica**. En una tarea como el diagnóstico médico, con frecuencia tenemos probabilidades condicionales sobre relaciones causales. El médico conoce $P(síntomas|enfermedad)$ y quiere derivar un diagnóstico, $P(enfermedad|síntomas)$. Causal Diagnóstico

Por ejemplo, un médico sabe que la enfermedad meningitis causa en un paciente cuello rígido, digamos, 70 % del tiempo. El médico también conoce algunos hechos incondicionales: la probabilidad previa de que todo paciente tenga meningitis es de 1/50 000, y la probabilidad previa de que todo paciente tenga el cuello rígido es de 1 %. Dejando que s sea la proposición de que el paciente tiene el cuello rígido y m sea la proposición de que el paciente tiene meningitis, tenemos

$$\begin{aligned} P(s|m) &= 0.7 \\ P(m) &= 1/50\,000 \\ P(s) &= 0.01 \\ P(m|s) &= \frac{P(s|m)\mathrm{P(m)}}{P(s)} = \frac{0.7 \times 1/50000}{0.01} = 0.0014. \end{aligned} \tag{12.14}$$

Es decir, esperamos que sólo 0.14 % de los pacientes con cuello rígido tenga meningitis. Vea que, aunque un cuello rígido es un síntoma claro de meningitis (con una probabilidad de 0.7), la probabilidad de meningitis en pacientes con cuello rígido aún es baja. Esto se debe a que la probabilidad previa de cuello rígido (por cualquier causa) es más alta que la previa para meningitis.

La Sección 12.3 ilustró un proceso mediante el cual se puede evitar evaluar la probabilidad previa de la evidencia (aquí, $P(s)$) al calcular en su lugar una probabilidad posterior para cada valor de la variable de consulta (aquí, m y $\neg m$) y luego normalizar los resultados. El mismo proceso se aplica al usar la regla de Bayes. Tenemos

$$\mathbf{P}(M|s) = \alpha\langle P(s|m)P(m), P(\,|\neg m)P(\neg m)\rangle.$$

Por lo tanto, para usar este enfoque, necesitamos estimar $P(s|\neg m)$ en lugar de $P(s)$. No hay nada garantizado, a veces esto es más fácil, o más difícil. La forma general de la regla de Bayes con normalización es

$$\mathbf{P}(Y|X) = \alpha\mathbf{P}(X|Y)\mathbf{P}(Y), \tag{12.15}$$

donde α es la constante de normalización necesaria para que las entradas en $\mathbf{P}(Y|X)$ sumen 1.

Una pregunta obvia sobre la regla de Bayes es por qué se podría tener disponible la probabilidad condicional en una dirección, pero no en la otra. En el dominio de la meningitis, tal vez el médico sepa que un cuello rígido implica meningitis en 1 de cada 5 000 casos; es decir, el médico tiene información cuantitativa en la dirección **diagnóstica** de síntomas a causas. Un médico así no necesita usar la regla de Bayes.

Sin embargo, *el conocimiento diagnóstico con frecuencia es más frágil que el conocimiento causal*. Si hay una epidemia repentina de meningitis, la probabilidad incondicional de meningitis, $P(m)$, aumentará. El médico que derivó la probabilidad diagnóstica $P(m|s)$ directamente de la observación estadística de los pacientes antes de la epidemia no tendrá idea de cómo actualizar el valor, pero el médico que calcula $P(m|s)$ a partir de los otros tres valores verá que $P(m|s)$ debería aumentar de manera proporcional con $P(m)$. Lo más importante, la información causal $P(s|m)$ no *se ve afectada* por la epidemia, porque refleja cómo funciona la meningitis. El uso de este tipo de conocimiento causal directo o basado en modelos da la robustez crucial necesaria para hacer que los sistemas probabilísticos sean factibles en el mundo real.

12.5.2 Usar la regla de Bayes: Combinar evidencia

Hemos visto que la regla de Bayes puede ser útil para responder a consultas probabilísticas condicionadas a una pieza de evidencia, por ejemplo, el cuello rígido. En particular, hemos argumentado que con frecuencia se dispone información probabilística en forma de $P(efecto|causa)$. ¿Qué sucede cuando tenemos dos o más piezas de evidencia? Por ejemplo, ¿qué puede concluir

un dentista si su desagradable escarificador de acero se engancha en el diente adolorido de un paciente? Si conocemos la distribución conjunta completa (Figura 12.3), podemos leer la respuesta:

$$\mathbf{P}(Caries|dolorDeDientes \wedge atrapar) = \alpha(0.108,0.016) \approx (0.871,0.129).$$

Sin embargo, sabemos que tal enfoque no es escalable a un mayor número de variables. Podemos usar la regla de Bayes para reformular el problema:

$$\mathbf{P}(Caries|dolorDeDientes \wedge \text{atrapar}) = \alpha\mathbf{P}(dolorDeDientes \wedge atrapar|Caries)\ \mathbf{P}(\text{Caries}). \quad (12.16)$$

Para que esta reformulación funcione, necesitamos conocer las probabilidades condicionales de la conjunción *dolorDeDientes* ∧ *atrapar* para cada valor de *Caries*. Eso sería factible para sólo dos variables de evidencia, pero de nuevo no es escalable. Si hay *n* posibles variables de evidencia (rayos X, dieta, higiene oral, etc.), entonces hay $O(2^n)$ posibles combinaciones de valores observados para las cuales necesitaríamos conocer probabilidades condicionales. Esto no es mejor que usar la distribución conjunta completa.

Para avanzar, necesitamos hallar afirmaciones adicionales sobre el dominio que nos permitan simplificar las expresiones. La noción de **independencia** en la Sección 12.4 da una pista, pero debe ser refinada. Sería bueno si *dolorDeDientes* y *Caries* fueran independientes, pero no lo son: si el escarificador engancha en el diente, es probable que éste tenga caries y que ésta cause el dolor. Pero, tales variables *son* independientes, *dada la presencia o ausencia de una caries*. Cada una es de manera directa causada por la caries, pero ninguna tiene un efecto directo sobre la otra: el dolor de dientes depende del estado de los nervios en el diente, mientras que la precisión del escarificador depende principalmente de la habilidad del dentista, a la cual el dolor de dientes es irrelevante.[7] Matemáticamente, esta propiedad se expresa como

$$\mathbf{P}(dolorDeDientes \wedge atrapar|Caries) = \mathbf{P}(dolorDeDientes|Caries)\mathbf{P}(atrapar|Caries). \quad (12.17)$$

Independencia condicional

La ecuación expresa **independencia condicional** de *dolorDeDientes* y atrapar dado *Caries*. Podemos reemplazar esto en la Ecuación (12.16) para obtener la probabilidad de una caries:

$$\mathbf{P}(Caries|dolorDeDientes \wedge atrapar) = \alpha\mathbf{P}(dolorDeDientes|Caries)\ \mathbf{P}(atrapar|Caries)\ \mathbf{P}(Caries). \quad (12.18)$$

Ahora los requisitos de información son los mismos que para la inferencia, al usar cada pieza de evidencia por separado: la probabilidad previa $\mathbf{P}(Caries)$ para la variable de consulta y la probabilidad condicional de cada efecto, dado su causa.

La definición general de **independencia condicional** de dos variables X y Y, dada una tercera variable Z, es

$$\mathbf{P}(X,Y|Z) = \mathbf{P}(X|Z)\mathbf{P}(Y|Z).$$

En el dominio del dentista, por ejemplo, parece razonable afirmar la independencia condicional de las variables *DolorDeDientes* y *Atrapar*, dada *Caries*:

$$\mathbf{P}(DolorDeDientes,\ Atrapar|Caries) = \mathbf{P}(DolorDeDientes|Caries)\mathbf{P}(Atrapar|Caries). \quad (12.19)$$

Vea que esta afirmación es más fuerte que la Ecuación (12.17), que sólo afirma la independencia para valores específicos de *DolorDeDientes* y *Atrapar*. Al igual que con la independencia absoluta en la Ecuación (12.11), las formas equivalentes

$$\mathbf{P}(X|Y,Z) = \mathbf{P}(X|Z) \quad \text{y} \quad \mathbf{P}(Y|X,Z) = \mathbf{P}(Y|Z)$$

[7] Suponemos que el paciente y el dentista son individuos distintos.

también se puede usar (ver Ejercicio 12.PXYZ). La Sección 12.4 mostró que las afirmaciones de independencia absoluta permiten descomposición de la distribución conjunta completa en piezas más pequeñas. Pues lo mismo es cierto para las afirmaciones de independencia condicional. Así, dada la afirmación en la Ecuación (12.19), derivamos una descomposición como sigue:

$$\begin{aligned}&\mathbf{P}(\mathit{DolorDeDientes}, \mathit{Atrapar}, \mathit{Caries})\\ &\quad = \mathbf{P}(\mathit{DolorDeDientes}, \mathit{Atrapar} \mid \mathit{Caries})\mathbf{P}(\mathit{Caries}) \qquad \text{(regla del producto)}\\ &\quad = \mathbf{P}(\mathit{DolorDeDientes}|\mathit{Caries})\mathbf{P}(\mathit{Atrapar}|\mathit{Caries})\mathbf{P}(\mathit{Caries}) \qquad \text{(usando la Ecuación 12.19).}\end{aligned}$$

(Usted verificará que esta ecuación sí se cumple en la Figura 12.3). Así, la tabla original grande se descompone en tres tablas más pequeñas. La tabla original tiene 7 números independientes. (La tabla tiene $2^3 = 8$ entradas, pero deben sumar 1, por ello 7 son independientes). Las tablas más pequeñas contienen un total de 2+2+1 = 5 números independientes. (Para una distribución de probabilidad condicional como $\mathbf{P}(\mathit{DolorDeDientes}|\mathit{Caries})$ hay dos filas de dos números, y cada fila suma 1, por ello son dos números independientes; para una distribución previa como $\mathbf{P}(\mathit{Caries})$ hay sólo un número independiente). Pasar de 7 a 5 puede no parecer gran victoria, pero las ganancias pueden ser mucho mayores con un mayor número de síntomas.

En general, para n síntomas todos condicionalmente independientes dados *Caries*, el tamaño de la representación crece como $O(n)$ en lugar de $O(2^n)$. Eso significa que *las afirmaciones de independencia condicional permiten que los sistemas probabilísticos se escalen; además, son mucho más comunes que las afirmaciones de independencia absoluta*. Conceptualmente, *Caries* **separa** *DolorDeDientes* y *Atrapar* porque es causa directa de ambos. La descomposición de dominios probabilísticos grandes en subconjuntos débilmente conectados a través de la independencia condicional es uno de los desarrollos más importantes en la historia reciente de la IA.

Separación

12.6 Modelos Bayes ingenuos

El ejemplo del dentista ilustra un patrón común en el que una sola causa influye de manera directa en varios efectos, todos condicionalmente independientes, dados la causa. La distribución conjunta completa se escribe como

$$\mathbf{P}(\mathit{Causa}, \mathit{Efecto}_1, \ldots, \mathit{Efecto}_n) = \mathbf{P}(\mathit{Causa}) \prod_i \mathbf{P}(\mathit{Efecto}_i|\mathit{Causa}). \qquad (12.20)$$

Bayes ingenuo

Tal distribución de probabilidad se llama modelo **Bayes ingenuo**, "ingenuo" porque suele usarse (como supuesto simplificador) en casos donde las variables "efecto" *no* son estrictamente independientes dadas la variable de causa. (El modelo Bayes ingenuo es llamado **clasificador bayesiano**, un uso algo descuidado que ha llevado a verdaderos Bayesianos a nombrarlo modelo **Bayes idiota**). En la práctica, los sistemas de Bayes ingenuos funcionan muy bien, aun cuando la suposición de independencia condicional no es estrictamente verdadera.

Para usar un modelo Bayes ingenuo, aplicamos la Ecuación (12.20) para obtener la probabilidad de la causa dados algunos efectos observados. Llamemos a estos efectos $\mathbf{E}=\mathbf{e}$, mientras que las variables de efecto restantes $\mathbf{Y}$ no se observan. Luego, se aplica el método estándar para la inferencia de la distribución conjunta (Ecuación (12.9)):

$$\mathbf{P}(\mathit{Causa}|\mathbf{e}) = \alpha \sum_{\mathbf{y}} \mathbf{P}(\mathit{Causa}, \mathbf{e}, \mathbf{y}).$$

De la Ecuación (12.20), entonces obtenemos:

$$\begin{aligned}\mathbf{P}(Causa|\mathbf{e}) &= \alpha\sum_{\mathbf{y}}\mathbf{P}(Causa)\mathbf{P}(\mathbf{y}|Causa)(\prod_j \mathbf{P}(\mathrm{e}_j|Causa)) \\ &= \alpha\mathbf{P}(Causa)(\prod_j \mathbf{P}(\boldsymbol{e}_j|\mathrm{Causa}))\sum_{\mathbf{y}}\mathbf{P}(\mathbf{y}|\mathrm{Causa}) \\ &= \alpha\mathbf{P}(Causa)\prod_j \mathbf{P}(e_j|Causa) \qquad (12.21)\end{aligned}$$

donde la última línea sigue porque la suma sobre **y** es 1. Reinterpretando esta ecuación en palabras: para cada posible causa, multiplicar la probabilidad previa de la causa por el producto de las probabilidades condicionales de los efectos observados dados la causa; luego normalizar el resultado. El tiempo de ejecución de este cálculo es lineal en el número de efectos observados y no depende del número de efectos no observados (que pueden ser muy grandes en dominios como la medicina). Veremos en el próximo capítulo que este es un fenómeno común en la inferencia probabilística: las variables de evidencia cuyos valores no son observados generalmente

12.6.1 Clasificación de texto con Naive Bayes

Clasificación de texto

Veamos cómo se usa un modelo Naive Bayes para la tarea de **clasificación de texto**: dado un texto, decidir a qué de un conjunto predefinido de clases o categorías pertenece. Aquí, la "causa" es la variable *Categoría*, y las variables "efecto" son la presencia o ausencia de ciertas palabras clave, $TienePalabra_i$. Piense estas dos oraciones de ejemplo, tomadas de periódicos:

1. Las acciones subieron el lunes, con los principales índices ganando 1% mientras el optimismo persistía sobre la temporada de ganancias del primer trimestre.
2. Las fuertes lluvias aún golpearon gran parte de la costa este el lunes, con advertencias de inundaciones emitidas en la ciudad de Nueva York y otras ubicaciones.

La tarea es clasificar cada oración en una *Categoría*: secciones del periódico: *noticias, deportes, negocios, clima o diversión*. El modelo Naive Bayes consiste en las probabilidades previas **P**(*Categoría*) y las probabilidades condicionales $\mathbf{P}(TienePalabra_i|Categoría)$. Para cada categoría *c*, *P*(*Categoría* = *c*) se estima como la fracción de todo documento visto antes que son de categoría *c*. Así, si 9 % de los artículos son sobre clima, establecemos *P*(*Categoría* = *clima*) = 0.09. De modo similar, $\mathbf{P}(TienePalabra_i|Categoría)$ se estima como la fracción de documentos de cada categoría que contienen la palabra *i*; tal vez 37 % de los artículos sobre negocios contienen la palabra 6, "acciones", por ello $\mathbf{P}(TienePalabra_6 = true|Categoría = negocios)$ se establece en 0.37.[8]

Para categorizar un nuevo documento, verificamos qué palabras clave hay en el documento y aplicamos la Ecuación (12.21) para obtener la distribución de probabilidad *a posteriori* sobre las categorías. Si tenemos que predecir sólo una categoría, tomamos aquella con la mayor probabilidad *a posteriori*. Vea que, para esta tarea, cada variable de efecto está observada, ya que siempre podemos determinar si una palabra dada aparece en el documento.

[8] Hay que tener cuidado de no asignar probabilidad cero a palabras que no se hayan visto antes en una determinada categoría de documentos, pues el cero borraría todas las demás pruebas de la ecuación (12.21). El hecho de que aún no haya visto una palabra no significa que *nunca* la verá. En su lugar, reserve una pequeña porción de la distribución de probabilidad para representar palabras "no vistas antes". Consulte el Capítulo 20 para más información sobre este tema en general, y la Sección 23.1.4 para el caso particular de los modelos de palabras.

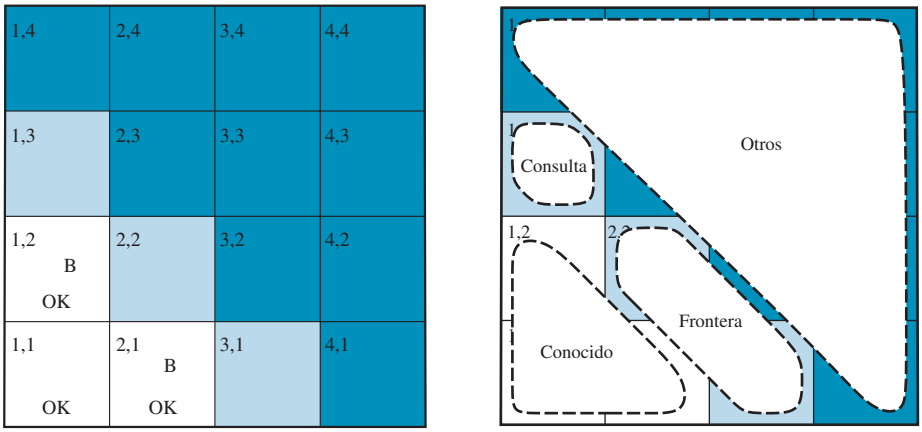


Figura 12.5 Después de encontrar brisa tanto en [1,2] como en [2,1], el agente se queda atascado: no hay ningún lugar seguro para explorar. (b) División de los cuadrados en *Conocido, Frontera* y *Otros*, para una consulta sobre [1,3].

El modelo de Bayes ingenuo asume que las palabras ocurren de manera independiente en los documentos, con frecuencias determinadas por la categoría del documento. Esta suposición de independencia se viola claramente en la práctica. Es decir, la frase "primer trimestre" ocurre con más frecuencia en artículos de negocios (o deportes) de lo que sugerirían al multiplicar las probabilidades de "primer" y "trimestre". La violación de la independencia en general significa que las probabilidades posteriores finales estarán mucho más cerca de 1 o 0 de lo que deberían estar; o sea, el modelo es demasiado seguro en sus predicciones. Además, aun con estos errores, la *clasificación* de las posibles categorías suele ser muy precisa.

Los modelos de Bayes ingenuos muy utilizados para determinación del lenguaje, recuperación de documentos, filtrado de spam y otras tareas de clasificación. Para tareas como diagnóstico médico, donde los valores reales de las probabilidades posteriores realmente importan, por ejemplo, para decidir si realizar una apendicectomía, uno con frecuencia preferiría usar los modelos más sofisticados descritos en el capítulo 13.

12.7 De vuelta al mundo del wumpus

Podemos combinar las ideas de este capítulo y resolver problemas de razonamiento probabilístico en el mundo del wumpus. (Vea en el Capítulo 7 una descripción completa de este mundo). La incertidumbre surge en el mundo del wumpus porque los sensores del agente dan sólo información parcial sobre el mundo. Por ejemplo, la Figura 12.5 muestra una situación donde cada una de las tres casillas no visitadas pero alcanzables: [1,3], [2,2] y [3,1], podrían contener un pozo. La inferencia lógica pura no permite concluir nada sobre qué casilla es más probable que sea segura, por ello un agente lógico podría tener que elegir al azar. Veremos que un agente probabilístico es capaz de hacerlo mucho mejor que el agente lógico.

El objetivo es calcular la probabilidad de que cada una de las tres casillas contenga un pozo. (Para este ejemplo, ignoramos el wumpus y el oro.) Las propiedades relevantes del mundo del

wumpus son que (1) un pozo causa brisas en todas las casillas vecinas y (2) cada casilla que no sea [1,1] contiene un pozo con probabilidad 0.2. El primer paso es identificar el conjunto de variables aleatorias que necesitamos:

- Al igual que en el caso de la lógica proposicional, queremos una variable booleana P_{ij} para cada casilla, que es verdadera si y sólo si la casilla $[i,j]$ realmente contiene un pozo.
- Tenemos variables booleanas B_{ij} que son verdaderas si y sólo si la casilla $[i,j]$ tiene brisa; incluimos estas variables sólo para casillas vistas, en este caso, [1,1], [1,2] y [2,1].

El siguiente paso es especificar la distribución conjunta completa, $\boldsymbol{P}(P_{1,1}, ..., P_{4,4}, B_{1,1}, B_{1,2}, B_{2,1})$. Aplicando la regla del producto, tenemos

$$\mathbf{P}(P_{1,1},..., P_{4,4}, B_{1,1}, B_{1,2}, B_{2,1}) = \mathbf{P}(B_{1,1}, B_{1,2}, B_{2,1}|P_{1,1},..., P_{4,4})\mathbf{P}(P_{1,1}, ..., P_{4,4}).$$

La descomposición facilita ver qué valores de probabilidad conjunta deberían ser. El primer término es distribución de probabilidad condicional de una configuración de brisa, dada una configuración de pozos; sus valores son 1 si todas las casillas con brisa son adyacentes a los pozos y 0 en caso contrario. El segundo término es probabilidad *a priori* de una configuración de pozos. Cada casilla contiene un pozo con probabilidad 0.2, independiente de otras casillas; por lo tanto,

$$\mathbf{P}(P_{1,1}, ..., P_{4,4}) = \prod_{i,j=1,1}^{4,4} \mathbf{P}(P_{i,j}). \tag{12.22}$$

Para una configuración particular con n pozos exactos, la probabilidad es $0.2^n \times 0.8^{16-n}$.

En la situación en la Figura 12.5(a), la evidencia consiste en la brisa observada (o su ausencia) en cada casilla que se visita, combinada con el hecho de que cada casilla de este tipo no contiene un pozo. Abreviamos estos hechos como $b = \neg b_{1,1} \wedge b_{1,2} \wedge b_{2,1}$ y $conocido = \neg p_{1,1} \wedge \neg p_{1,2} \wedge \neg p_{2,1}$. Estamos interesados en responder consultas como $\mathbf{P}(P_{1,3}|conocido,b)$: ¿Qué tan probable es que [1,3] contenga un pozo, dadas las observaciones hasta ahora?

Para responder a esta consulta, seguimos el enfoque estándar de la Ecuación (12.9), es decir, sumar sobre entradas de la distribución conjunta completa. Sea *Desconocido* el conjunto de variables $P_{i,j}$ para las casillas que no son las casillas conocidas y la casilla de consulta [1,3]. Entonces, por la Ecuación (12.9), tenemos

$$\mathbf{P}(P_{1,3}|conocido,b) = \alpha \sum_{desconocido} \mathbf{P}(P_{1,3},conocido,b,desconocido). \tag{12.23}$$

Las probabilidades conjuntas completas ya han sido especificadas, así que hemos terminado, es decir, a menos que nos importe la computación. Hay 12 casillas desconocidas; por lo tanto, la suma contiene $2^{12} = 4096$ términos. En general, la suma crece de forma exponencial con el número de casillas. Con seguridad, uno podría preguntar, ¿no son irrelevantes las otras casillas? ¿Cómo podría afectar [4,4] si [1,3] tiene un pozo? De hecho, esta intuición es aproximadamente correcta, pero necesita ser más precisa. Lo que realmente queremos decir es que si conociéramos los valores de todas las variables de pozo adyacentes a las casillas que nos interesan, entonces los pozos (o su ausencia) en otras casillas más distantes podrían no tener ningún efecto adicional en nuestra creencia.

Sea *Frontera* las variables de pozo (que no sean la variable de consulta) que son adyacentes a las casillas visitadas, en este caso sólo [2,2] y [3,1]. Además, sea *Otro* las variables de pozo para las otras casillas desconocidas; en este caso, hay 10 casillas "otras", como se muestra en la Figura 12.5(b). Con estas definiciones, $Desconocido = Frontera \cup Otro$. La idea clave dada con anterioridad ahora se puede enunciar como: las brisas observadas son *condicionalmente independientes*

de las otras variables, dadas las variables conocidas, de frontera y de consulta. Para usar esta idea, manipulamos la fórmula de consulta en una forma en la que las brisas estén condicionadas sobre todas las demás variables, y luego aplicamos la independencia condicional:

$$
\begin{aligned}
&\mathbf{P}(P_{1,3}|known,b)\\
&= \alpha \sum_{unknown} \mathbf{P}(P_{1,3}, known, b, unknown) \qquad \text{(de Ecuación (12.23))}\\
&= \alpha \sum_{unknown} \mathbf{P}(b|P_{1,3}, known, unknown)\mathbf{P}(P_{1,3}, known, unknown) \qquad \text{(product rule)}\\
&= \alpha \sum_{frontier} \sum_{other} \mathbf{P}(b|known, P_{1,3}, frontier, other)\mathbf{P}(P_{1,3}, known, frontier, other)\\
&= \alpha \sum_{frontier} \sum_{other} \mathbf{P}(b|known, P_{1,3}, frontier,)\mathbf{P}(P_{1,3}, known, frontier, other)
\end{aligned}
$$

donde el paso final utiliza independencia condicional: *b* es independiente de *otro* dado *conocido*, $P_{1,3}$ y *frontera*. Ahora, el primer término en esta expresión no depende de las variables *Otro*, así que podemos mover la suma hacia adentro:

$$
\begin{aligned}
&\mathbf{P}(P_{1,3}|known,b)\\
&= \alpha \sum_{frontier} \mathbf{P}(b|known, P_{1,3}, frontier) \sum_{other} \mathbf{P}(P_{1,3}, known, frontier, other)
\end{aligned}
$$

Por independencia, como en la Ecuación (12.22), el término a la derecha se factoriza, y luego los términos se reordenan:

$$
\begin{aligned}
&\mathbf{P}(P_{1,3}|known,b)\\
&= \alpha \sum_{frontier} \mathbf{P}(b|known, P_{1,3}, frontier) \sum_{other} \mathbf{P}(P_{1,3}, known)P\,(frontier)P\,(other)\\
&= \alpha\, \mathbf{P}(known)\mathbf{P}(P_{1,3}) \sum_{frontier} \mathbf{P}(b|known, P_{1,3}, frontier)P(frontier) \sum_{other} P\,(other)\\
&= \alpha' \mathbf{P}(P_{1,3}) \sum_{frontier} \mathbf{P}(b|known, P_{1,3}, frontier)P(frontier)
\end{aligned}
$$

donde el último paso incorpora *P*(*conocido*) en la constante de normalización y utiliza el hecho de que $\sum_{otro} P(otro)$ es igual a 1.

Ahora, sólo hay cuatro términos en la suma sobre las variables de frontera, $P_{2,2}$ y $P_{3,1}$. El uso de independencia e independencia condicional ha eliminado por completo las demás casillas en consideración.

Vea que las probabilidades en $\boldsymbol{P}(b|conocido, P_{1,3}, frontera)$ son 1 cuando las observaciones de brisa son consistentes con las demás variables y 0 de lo contrario. Por lo tanto, para cada valor de $P_{1,3}$, sumamos sobre los *modelos lógicos* para las variables de frontera que son consistentes con los hechos conocidos. (Comparar con la enumeración de modelos en la Figura 7.5 en la página 215). Los modelos y sus probabilidades a priori asociadas —*P*(*frontera*)— se muestran en la Figura 12.6. Tenemos

$$\mathbf{P}(P_{1,3}|conocido,b) = \alpha'\langle 0.2(0.04 + 0.16 + 0.16), 0.8(0.04 + 0.16)\rangle \approx \langle 0.31, 0.69\rangle.$$

Es decir, [1,3] (y [3,1] por simetría) contiene un pozo con casi 31 % de probabilidad. Un cálculo similar, que el lector podría desear realizar, muestra que [2,2] contiene un pozo con alrededor de 86 % de probabilidad. ¡El agente del wumpus definitivamente debería evitar [2,2]! Vea que

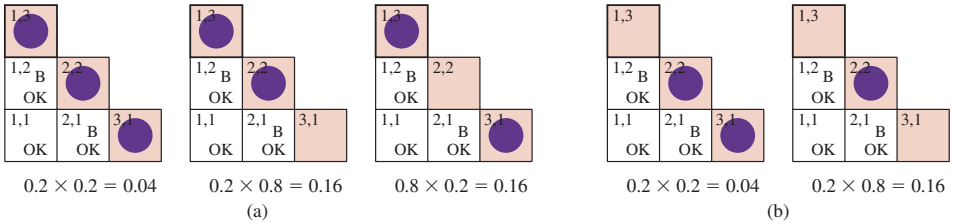


Figura 12.6 Modelos consistentes para las variables de frontera, $P_{2,2}$ y $P_{3,1}$, que muestran $P(frontera)$ para cada modelo: (a) tres modelos con $P_{1,3}$ = *verdadero* que muestran dos o tres pozos, y (b) dos modelos con $P_{1,3}$ = *falso,* muestra uno o dos pozos.

nuestro agente lógico del Capítulo 7 no sabía que [2,2] era peor que las otras casillas. La lógica puede decirnos que se desconoce si hay un pozo en [2, 2], pero necesitamos la probabilidad para decirnos qué tan probable es.

Esta sección ha demostrado que aun problemas complicados pueden formularse de manera precisa en la teoría de la probabilidad y resolverse con algoritmos simples. Para obtener soluciones *eficientes*, se pueden usar relaciones de independencia e independencia condicional para simplificar las sumas requeridas. Estas relaciones suelen corresponder a nuestra comprensión natural de cómo debería descomponerse el problema. En el próximo capítulo, se desarrollan representaciones formales para tales relaciones, así como algoritmos que operan en esas representaciones para realizar inferencia probabilística de manera eficiente.

Resumen

Este capítulo ha sugerido la teoría de la probabilidad como una base adecuada para el razonamiento incierto y ha proporcionado una introducción suave a su uso.

- La incertidumbre surge debido tanto a la pereza como a la ignorancia. Es inevitable en entornos complejos, no deterministas o parcialmente observables.
- Las **probabilidades** expresan la incapacidad del agente para llegar a una decisión definitiva con respecto a la verdad de una sentencia. Las probabilidades resumen las creencias del agente en relación con la evidencia.
- La **teoría de la decisión** combina las creencias y deseos del agente, al definir la mejor acción como aquella que maximiza la **utilidad** esperada.
- Las afirmaciones básicas de probabilidad incluyen **probabilidades *a priori*** o **incondicionales** y **probabilidades *a posteriori*** o **condicionales** sobre proposiciones simples y complejas.
- Los axiomas de la probabilidad limitan las probabilidades de proposiciones lógicamente relacionadas. Un agente que viola los axiomas debe comportarse de manera irracional en algunos casos.
- La **distribución de probabilidad conjunta completa** especifica la probabilidad de cada asignación completa de valores a variables aleatorias. Por lo general, es demasiado grande para crearla o usarla en su forma explícita, pero cuando está disponible, se puede usar para responder consultas sumando las entradas para los mundos posibles correspondientes a las proposiciones de consulta.

- La **independencia absoluta** entre subconjuntos de variables aleatorias permite que la distribución de probabilidad conjunta se factorice en distribuciones conjuntas más pequeñas, reduciendo en gran medida su complejidad.
- La **regla de Bayes** permite calcular probabilidades desconocidas a partir de probabilidades condicionales conocidas, en general en la dirección causal. Aplicar la regla de Bayes con muchas piezas de evidencia se enfrenta a los mismos problemas de escalamiento que la distribución conjunta completa.
- La **independencia condicional** provocada por relaciones causales directas en el dominio permite que la distribución conjunta completa se factorice en distribuciones condicionales más pequeñas. El modelo de **Bayes ingenuo** asume la independencia condicional de todas las variables de efecto, dada una sola variable de causa; su tamaño crece de manera lineal con el número de efectos.
- Un agente del mundo de wumpus puede calcular probabilidades para aspectos no observados del mundo, mejorando así las decisiones de un agente puramente lógico. La independencia condicional hace que estos cálculos sean tratables.

Notas bibliográficas e históricas

La teoría de la probabilidad fue inventada como una forma de analizar juegos de azar. Alrededor del año 850 d.C., el matemático indio Mahaviracarya describió cómo organizar un conjunto de apuestas que no pueden perder (lo que ahora llamamos apuesta holandesa). En Europa, los primeros análisis sistemáticos significativos fueron producidos por Girolamo Cardano alrededor de 1565, aunque la publicación fue póstuma (1663). Para ese momento, la probabilidad se había establecido como una disciplina matemática debido a una serie de resultados de una famosa correspondencia entre Blaise Pascal y Pierre de Fermat en 1654. El primer libro de texto publicado sobre probabilidad fue *De Ratiociniis in Ludo Aleae* (Sobre el razonamiento en un juego de azar) de Huygens (1657). La visión de "pereza e ignorancia" de la incertidumbre fue descrita por John Arbuthnot en el prefacio de su traducción de Huygens (Arbuthnot, 1692): "Es imposible para un dado, con cierta fuerza y dirección determinadas, no caer en un lado determinado, sólo que no conozco la fuerza y la dirección que hacen que caiga en dicho lado, y por eso lo llamo Azar, que no es más que la falta de arte".

La conexión entre la probabilidad y el razonamiento se remonta al menos al siglo XIX: en 1819, Pierre Laplace dijo: "La teoría de la probabilidad no es más que el sentido común reducido a cálculo". En 1850, James Maxwell dijo: "La verdadera lógica para este mundo es el cálculo de probabilidades, que tiene en cuenta la magnitud de la probabilidad que está o debería estar en la mente de un hombre razonable".

Ha habido un debate interminable sobre el origen y estado de los números de probabilidad.

Frecuentísimo

La posición **frecuentista** es que los números sólo pueden provenir de *experimentos*: si probamos a 100 personas y encontramos que 10 de ellas tienen una caries, entonces podemos decir que la probabilidad de una caries es casi 0.1. En esta vista, la afirmación "la probabilidad de una caries es 0.1" significa que 0.1 es la fracción que se observaría en el límite de infinitas muestras. A partir de cualquier muestra finita, podemos estimar la fracción real y también calcular qué tan precisa es probable que sea nuestra estimación.

Objetivista

La vista **objetivista** es que las probabilidades son aspectos reales del universo —propensiones de objetos a comportarse de ciertas maneras— en lugar de ser solo descripciones del grado de creencia de un observador. Por ejemplo, el hecho de que una moneda justa salga cara con una probabilidad de 0.5 es una propensión de la moneda en sí misma. En esta vista, las mediciones frecuentistas son intentos de observar estas propensiones. La mayoría de los físicos están de

acuerdo en que los fenómenos cuánticos son objetivamente probabilísticos, pero la incertidumbre a escala macroscópica —por ejemplo, en el lanzamiento de una moneda— en general surge de la ignorancia de las condiciones iniciales y no parece consistente con la vista de la propensión.

Subjetivismo

La vista **subjetivista** describe las probabilidades como una forma de caracterizar las creencias de un agente, en lugar de tener algún significado físico externo. La vista **bayesiana** subjetiva permite cualquier asignación autoconsistente de probabilidades previas a proposiciones, pero luego insiste en una actualización bayesiana adecuada a medida que llega la evidencia.

Clase de referencia

Aun una posición frecuentista estricta implica subjetividad debido al problema de la **clase de referencia**: al intentar determinar la probabilidad del resultado de un experimento *particular*, el frecuentista debe colocarlo en una clase de referencia de experimentos "similares" con frecuencias de resultado conocidas. Pero ¿cuál es la clase correcta? I. J. Good escribió: "cada evento en la vida es único, y cada probabilidad real que estimamos en la práctica es la de un evento que nunca ha ocurrido antes" (Good, 1983, p. 27).

Por ejemplo, dado un paciente particular, un frecuentista que quiere estimar la probabilidad de una caries considerará una clase de referencia de otros pacientes que sean similares en aspectos importantes —edad, síntomas, dieta— y verá qué proporción de ellos tenía una caries. Si el dentista considera todo lo que se sabe sobre el paciente —color de pelo, peso al gramo más cercano, apellido de soltera de la madre— entonces la clase de referencia se vuelve vacía. Este ha sido un problema molesto en la filosofía de la ciencia.

Pascal usó la probabilidad de maneras que requerían tanto la interpretación objetiva, como una propiedad del mundo basada en la simetría o la frecuencia relativa, y la interpretación subjetiva, basada en el grado de creencia —la primera en sus análisis de probabilidades en juegos de azar, la segunda en el famoso argumento de "la apuesta de Pascal" sobre la posible existencia de Dios. Sin embargo, Pascal no se dio cuenta de la distinción entre estas dos interpretaciones. La distinción fue dibujada por primera vez por James Bernoulli (1654–1705).

Leibniz introdujo la noción "clásica" de probabilidad como una proporción de casos enumerados, igualmente probables, que también fue utilizada por Bernoulli, aunque fue llevada a la prominencia por Laplace (1816). Esta noción es ambigua entre la interpretación de frecuencia y la interpretación subjetiva. Los casos pueden pensarse como igualmente probables ya sea debido a una simetría natural y física entre ellos, o porque no tenemos ningún conocimiento que nos lleve a considerar uno más probable que otro. El uso de esta última consideración subjetiva para justificar la asignación de probabilidades iguales se conoce como el **principio de indiferencia**. El principio a menudo se atribuye a Laplace (1816), pero nunca lo utilizó de manera explícita; Keynes (1921) sí lo hizo. George Boole y John Venn se refirieron a él como el **principio de razón insuficiente**.

Principio de indiferencia

Principio de razón insuficiente

El debate entre objetivistas y subjetivistas se agudizó en el siglo xx. Kolmogorov (1963), R. A. Fisher (1922) y Richard von Mises (1928) fueron defensores de la interpretación de la frecuencia relativa. La interpretación de "propensión" de Karl Popper (1959, publicada por primera vez en alemán en 1934) rastrea las frecuencias relativas a una simetría física subyacente. Frank Ramsey (1931), Bruno de Finetti (1937), R. T. Cox (1946), Leonard Savage (1954), Richard Jeffrey (1983) y E. T. Jaynes (2003) interpretaron las probabilidades como los grados de creencia de individuos específicos. Sus análisis del grado de creencia estaban estrechamente vinculados a las utilidades y al comportamiento —de manera específica, a la disposición para realizar apuestas.

Rudolf Carnap ofreció una interpretación diferente de la probabilidad —no como el grado de creencia que un individuo realmente tiene, sino como el grado de creencia que un razonador idealizado *debería* tener en una proposición particular a, dada una cierta evidencia $\mathbf{e}$. Carnap intentó hacer esta noción de grado de confirmación matemática precisa, como una relación lógica entre a y $\mathbf{e}$. En la actualidad se cree que no hay una lógica única de este tipo; más bien, cualquier lógica

de este tipo descansa en una distribución subjetiva de probabilidades previas cuyo efecto se disminuye a medida que se recopilan más observaciones.

Lógica inductiva

El estudio de esta relación tenía como objetivo constituir una disciplina matemática llamada **lógica inductiva**, análoga a la lógica deductiva ordinaria (Carnap, 1948, 1950). Carnap no pudo extender su lógica inductiva mucho más allá del caso proposicional, y Putnam (1963) mostró mediante argumentos adversativos que algunas dificultades eran inherentes. Un trabajo más reciente de Bacchus, Grove, Halpern y Koller (1992) extiende los métodos de Carnap a teorías de primer orden.

El primer marco axiomático rigurosamente para la teoría de la probabilidad fue propuesto por Kolmogorov (1950, primero publicado en alemán en 1933). Rényi (1970) con posterioridad presentó un enfoque axiomático que tomaba la probabilidad condicional, en lugar de la probabilidad absoluta, como primitiva.

Además de los argumentos de De Finetti sobre la validez de los axiomas, Cox (1946) demostró que cualquier sistema para el razonamiento incierto que cumpla su conjunto de suposiciones es equivalente a la teoría de la probabilidad. Esto renovó la confianza de los fanáticos de la probabilidad, pero otros no estaban convencidos, objetando la suposición de que la creencia debe ser representada por un solo número. Halpern (1999) describe las suposiciones y muestra algunas lagunas en la formulación original de Cox. Horn (2003) muestra cómo resolver las dificultades. Jaynes (2003) tiene un argumento similar que es más fácil de leer.

El reverendo Thomas Bayes (1702–1761) introdujo la regla para razonar sobre probabilidades condicionales que fue póstumamente nombrada en su honor (Bayes, 1763). Bayes sólo consideró el caso de priors uniformes; fue Laplace quien desarrolló de forma independiente el caso general. El razonamiento probabilístico bayesiano se ha utilizado en IA desde la década de 1960, en especial en el diagnóstico médico. Se utilizó no sólo para hacer un diagnóstico a partir de la evidencia disponible, sino también para seleccionar más preguntas y pruebas utilizando la teoría del valor de la información (Sección 16.6) cuando la evidencia disponible era inconclusa (Gorry, 1968; Gorry *et al.,* 1973). Un sistema superó a los expertos humanos en el diagnóstico de enfermedades abdominales agudas (de Dombal *et al*., 1974). Lucas *et al.* (2004) proporcionan una visión general.

Estos primeros sistemas bayesianos sufrieron varios problemas. Debido a que carecían de un modelo teórico de las condiciones que estaban diagnosticando, eran vulnerables a datos no representativos que ocurrían en situaciones para las cuales solo se disponía de una pequeña muestra (de Dombal *et al*., 1981). Aún más fundamental, debido a que carecían de un formalismo conciso (como el que se describirá en el Capítulo 13) para representar y usar información de independencia condicional, dependían de la adquisición, almacenamiento y procesamiento de enormes tablas de datos probabilísticos. Debido a estas dificultades, los métodos probabilísticos para lidiar con la incertidumbre cayeron en desgracia en la IA desde la década de 1970 hasta mediados de la década de 1980. Los desarrollos desde finales de la década de 1980 se describen en el próximo capítulo.

El modelo de Bayes ingenuo para distribuciones conjuntas ha sido estudiado en extenso en la literatura de reconocimiento de patrones desde la década de 1950 (Duda y Hart, 1973). También se ha utilizado, con frecuencia sin saberlo, en la recuperación de información, comenzando con el trabajo de Maron (1961). Los fundamentos probabilísticos de esta técnica, descritos de manera más detallada en el Ejercicio 12.BAYS, fueron elucidados por Robertson y Sparck Jones (1976). Domingos y Pazzani (1997) ofrecen una explicación para el éxito sorprendente del razonamiento bayesiano ingenuo incluso en dominios donde las suposiciones de independencia están claramente violadas.

Hay muchos buenos libros de texto introductorios sobre teoría de la probabilidad, incluidos los de Bertsekas y Tsitsiklis (2008), Ross (2015) y Grinstead y Snell (1997). DeGroot y Schervish

(2001) ofrecen una introducción combinada a la probabilidad y estadística desde un punto de vista bayesiano, y Walpole *et al.* (2016) ofrecen una introducción para científicos e ingenieros. Jaynes (2003) ofrece una exposición muy persuasiva del enfoque bayesiano. Billingsley (2012) y Venkatesh (2012) proporcionan tratamientos más matemáticos, incluidas todas las complicaciones con variables continuas que hemos dejado fuera. Hacking (1975) y Hald (1990) cubren la historia temprana del concepto de probabilidad, y Bernstein (1996) ofrece un relato popular.

CAPÍTULO 13

RAZONAMIENTO PROBABILÍSTICO

En el que explicamos cómo construir modelos eficientes de red para razonar bajo incertidumbre de acuerdo con las leyes de la teoría de la probabilidad, y cómo distinguir entre correlación y causalidad.

El Capítulo 12 introdujo los elementos básicos de la teoría de la probabilidad y destacó la importancia de las relaciones de independencia y de independencia condicional en la simplificación de las representaciones probabilísticas del mundo. Este capítulo introduce una forma sistemática de representar esas relaciones de manera explícita en forma de **redes bayesianas**. Definimos la sintaxis y la semántica de estas redes y mostramos cómo son usadas para capturar conocimiento incierto de manera natural y eficiente. Luego mostramos cómo la inferencia probabilística, aunque en el peor de los casos computacionalmente intratable, puede ser realizada de manera eficiente en muchas situaciones prácticas. También describimos una variedad de algoritmos de inferencia aproximada que con frecuencia son aplicables cuando la inferencia exacta es inviable. El Capítulo 15 extiende las ideas básicas de las redes bayesianas a lenguajes formales más expresivos para definir modelos de probabilidad.

13.1 Representando conocimiento en un dominio incierto

En el Capítulo 12, vimos que la distribución de probabilidad conjunta completa permite responder cualquier pregunta sobre el dominio, pero puede volverse intratable a medida que el número de variables crece. Además, especificar probabilidades para los mundos posibles uno por uno es antinatural y tedioso.

También observamos que las relaciones de independencia e independencia condicional entre variables permiten reducir en gran medida el número de probabilidades que necesitan ser especificadas para definir la distribución conjunta completa. Esta sección introduce la **red de Bayes,**[1] una estructura de datos para representar las dependencias entre variables. Las redes bayesianas representan *cualquier* distribución de probabilidad conjunta completa y, en muchos casos, lo hacen de manera muy concisa.

Redes bayesianas

Una red bayesiana es un grafo dirigido en el que cada nodo está anotado con información de probabilidad cuantitativa. La especificación completa es:

1. Cada nodo corresponde a una variable aleatoria, discreta o continua.
2. Los enlaces dirigidos o flechas conectan pares de nodos. Si hay una flecha del nodo X al nodo Y, se dice que X es un *padre* de Y. El grafo no tiene ciclos dirigidos y, por lo tanto, es un grafo acíclico dirigido, o DAG.
3. Cada nodo X_i tiene información de probabilidad asociada $\theta(X_i \mid Padres(X_i))$ que cuantifica el efecto de los padres en el nodo usando un número finito de **parámetros**.

Parámetro

[1] Las redes de Bayes, abreviadas "Bayes net", se llamaban **redes de creencias** en las décadas de 1980 y 1990. Una **red causal** es una red de Bayes con restricciones adicionales sobre el significado de las flechas (vea Sección 13.5). El término **modelo gráfico** se refiere a una clase más amplia que incluye las redes bayesianas.

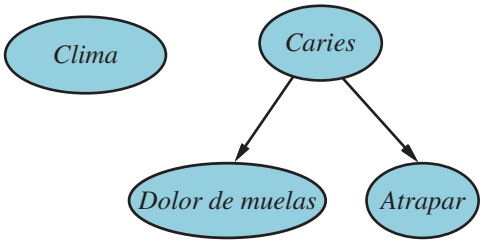


Figura 13.1 Una red bayesiana simple en la que el *clima* es independiente de las otras tres variables; y *dolor de muelas* y *atrapar* son condicionalmente independientes, dado la *caries*.

La topología de la red, el conjunto de nodos y enlaces, especifica las relaciones de independencia condicional que se mantienen en el dominio, de una manera que se precisará en breve. El significado *intuitivo* de una flecha suele ser que *X* tiene una *influencia directa* sobre *Y*, lo que sugiere que las causas deben ser padres de los efectos. Por lo general, es fácil para un experto en el dominio decidir qué influencias directas existen en el dominio, mucho más fácil, de hecho, que especificar las probabilidades en sí. Una vez que se ha establecido la topología de la red bayesiana, sólo necesitamos especificar la información de probabilidad local para cada variable, en forma de una distribución condicional dada por sus padres. La distribución conjunta completa para todas las variables está definida por la topología y la información de probabilidad local.

Recuerde el mundo descrito en el Capítulo 12, que consiste en las variables *dolor de muelas*, *caries*, *atrapar* y *clima*. Argumentamos que *Clima* es independiente de las otras variables; además, argumentamos que *dolor de muelas* y *atrapar* son condicionalmente independientes, dado la *caries*. Estas relaciones están representadas por la estructura de la red bayesiana mostrada en la Figura 13.1. Formalmente, la independencia condicional del *dolor de dientes* y *atrapar*, dado la *caries*, se indica por la ausencia de un enlace entre el *dolor de dientes* y *atrapar*. De manera intuitiva, la red representa el hecho de que la caries es una causa directa del *dolor de muelas* y *atrapar* , mientras que no existe ninguna relación causal directa entre el *dolor de dientes* y *atrapar*.

Ahora considere el siguiente ejemplo, un poco más complejo. Usted instaló una nueva alarma contra robos en casa; esta es muy fiable para detectar un robo, pero a veces se dispara con micro terremotos. (Este ejemplo se debe a Judea Pearl, residente de Los Ángeles que es propensa a terremotos). También tiene dos vecinos, John y Mary, que han prometido llamarle al trabajo cuando escuchen la alarma. John casi siempre llama cuando escucha la alarma, pero a veces confunde el timbre del teléfono con la alarma y también en esos casos llama. Mary, por otro lado, le gusta escuchar música muy fuerte y con frecuencia no escucha la alarma por completo. Dada la evidencia de quién ha llamado o no, nos gustaría estimar la probabilidad de un robo.

Una red bayesiana para este dominio aparece en la Figura 13.2. La estructura de la red muestra que los robos y los terremotos afectan de manera directa la probabilidad de que la alarma se dispare, pero si John y Mary llaman depende sólo de la alarma. La red representa así nuestras suposiciones de que ellos no perciben los robos directamente, no notan los terremotos menores, y no se consultan antes de llamar.

La información de probabilidad local adjunta a cada nodo en la Figura 13.2 toma la forma de una **tabla de probabilidad condicional (CPT)**. (Las CPTs sólo pueden ser usadas para variables discretas; otras representaciones, como las adecuadas para variables continuas, se describen en la Sección 13.2.) Cada fila en una CPT contiene la probabilidad condicional de cada valor del

Tabla de probabilidad condicional (CPT)

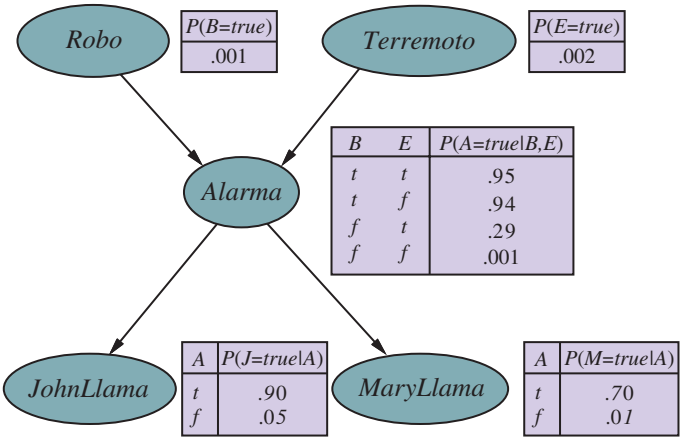


Figura 13.2 Una red bayesiana típica, mostrando tanto la topología como las tablas de probabilidad condicional (CPTs). En las CPTs, las letras *B*, *E*, *A*, *J*, y *M* representan *Robo*, *Terremoto*, *Alarma*, *JohnLlama*, y *MaryLlama*, respectivamente.

Caso de condicionamiento

nodo para un **caso de condicionamiento**. Un caso de condicionamiento es una combinación posible de valores para los nodos padres, un pequeño mundo posible, si se quiere. Cada fila debe sumar 1, porque las entradas representan un conjunto exhaustivo de casos para la variable. Para las variables booleanas, una vez que se sabe que la probabilidad de un valor verdadero es p, la probabilidad de falso debe ser $1 - p$, por lo que a menudo omitimos el segundo número, como en la Figura 13.2. En general, una tabla para una variable booleana con k padres booleanos contiene 2^k probabilidades especificadas de manera independiente. Un nodo sin padres tiene sólo una fila, que representa las probabilidades previas de cada valor posible de la variable.

Note que la red no tiene nodos correspondientes a que Mary esté actualmente escuchando música fuerte o a que el teléfono esté sonando y confundiendo a John. Estos factores se resumen en la incertidumbre asociada a los enlaces de *Alarma* a *JohnLlama* y *MaryLlama*. Esto muestra tanto la pereza como la ignorancia en operación, como se explica en la página 386: sería mucho trabajo averiguar por qué esos factores serían más o menos probables en cualquier caso particular, y no tenemos ninguna forma razonable de obtener la información relevante de todos modos.

Las probabilidades en realidad resumen un conjunto *potencialmente infinito* de circunstancias en las que la alarma podría fallar en sonar (alta humedad, fallo de energía, batería muerta, cables cortados, un ratón muerto atascado dentro de la campana, etc.) o John o Mary podrían fallar en llamar y reportarlo (fuera a almorzar, de vacaciones, temporalmente sordo, helicóptero pasando, etc.). De esta manera, un pequeño agente puede lidiar con un mundo muy grande, al menos de manera aproximada.

13.2 La semántica de las redes bayesianas

La *sintaxis* de una red bayesiana consiste en un grafo acíclico dirigido con alguna información de probabilidad local adjunta a cada nodo. La *semántica* define cómo la sintaxis corresponde a una distribución conjunta sobre las variables de la red.

Supongamos que la red bayesiana contiene n variables, $X_1,...,X_n$. Una entrada genérica en la distribución conjunta es entonces $P(X_1 = x_1 \wedge ... \wedge X_n = x_n)$, o $P(x_1,...,x_n)$ para abreviar. La

semántica de las redes bayesianas define cada entrada en la distribución conjunta de la siguiente manera:

$$P(x_1,...,x_n) = \prod_{i=1}^{n} \theta(x_i \mid padres(X_i)), \tag{13.1}$$

donde $padres(X_i)$ denota los valores de $Padres(X_i)$ que aparecen en $x_1,...,x_n$. Así, cada entrada en la distribución conjunta está representada por el producto de los elementos apropiados de las distribuciones condicionales locales en la red bayesiana.

Para ilustrar esto, podemos calcular la probabilidad de que la alarma haya sonado, pero ni un robo ni un terremoto han ocurrido, y tanto John como Mary llaman. Multiplicamos las entradas relevantes de las distribuciones condicionales locales (abreviando los nombres de las variables):

$$\begin{aligned} P(j,m,a,\neg b,\neg e) &= P(j|a)P(m|a)P(a|\neg b \wedge \neg e)P(\neg b)P(\neg e) \\ &= 0.90 \times 0.70 \times 0.01 \times 0.999 \times 0.998 = 0.00628. \end{aligned}$$

La Sección 12.3 explicó que la distribución conjunta completa puede ser usada para responder cualquier consulta sobre el dominio. Si una red bayesiana es una representación de la distribución conjunta, entonces también puede usarse para responder cualquier consulta, sumando todos los valores de probabilidad conjunta relevantes, cada uno calculado multiplicando las probabilidades de las distribuciones condicionales locales. La Sección 13.3 explica esto con más detalle, pero también describe métodos más eficientes.

Hasta ahora, hemos pasado por alto un punto importante: ¿cuál es el significado de los números que entran en las distribuciones condicionales locales $\theta(x_i|padres(X_i))$? Resulta que a partir de la Ecuación (13.1) podemos demostrar que los parámetros $\theta(x_i|padres(X_i))$ son las probabilidades condicionales $P(x_i|\text{padres}(X_i))$ implicadas por la distribución conjunta. Recuerde que las probabilidades condicionales se calculan a partir de la distribución conjunta de la siguiente manera:

$$\begin{aligned} P(xi|padres(X_i)) &\equiv \frac{P(x_i, padres(X_i))}{P(padres(X_i))} \\ &= \frac{\Sigma_{\mathbf{y}} P(x_i, padres(X_i),\mathbf{y})}{\Sigma x'_i,_{\mathbf{y}} P(x'_i, padres(X_i),\mathbf{y})} \end{aligned}$$

donde **y** representa los valores de todas las variables distintas de X_i y sus padres. A partir de esta última línea se demuestra que $P(x_i|padres(X_i)) = \theta(x_i|padres(X_i))$ (Ejercicio 13.CPTE). Por lo tanto, podemos reescribir la Ecuación (13.1) como

$$P(x_1,...,x_n) = \prod_{i=1}^{n} P(x_i|padres(X_i)). \tag{13.2}$$

Esto significa que cuando uno estima valores para las distribuciones condicionales locales, necesitan ser las probabilidades condicionales reales para la variable dada sus padres. Así, por ejemplo, cuando especificamos $\theta(JohnLlama = verdadero \mid Alarma = verdadero) = 0.90$, debería ser el caso que alrededor de 90% de las veces cuando suena la alarma, John llama. El hecho de que cada parámetro de la red tiene un significado preciso en términos de sólo un pequeño conjunto de variables es crucialmente importante para la robustez y facilidad de especificación de los modelos.

Un método para construir redes bayesianas

La Ecuación (13.2) define lo que significa una red bayesiana dada. El siguiente paso es explicar cómo *construir* una red bayesiana de tal manera que la distribución conjunta resultante sea una buena representación de un dominio dado. Ahora mostraremos que la Ecuación (13.2) implica ciertas relaciones de independencia condicional que pueden ser utilizadas para guiar al ingeniero de conocimiento en la construcción de la topología de la red. Primero, reescribimos las entradas

en la distribución conjunta en términos de probabilidad condicional, usando la regla del producto (ver página 390):

$$P(x_1,...,x_n) = P(x_n|x_{n-1},..., x_1)P(x_{n-1},...,x_1).$$

Luego repetimos el proceso, reduciendo cada probabilidad conjunta a una probabilidad condicional y una probabilidad conjunta en un conjunto más pequeño de variables. Terminamos con un producto de gran extensión:

$$\begin{aligned} P(x_1,...,x_n) &= P(x_n|x_{n-1},...,x_1)P(x_{n-1}|x_{n-2},...,x_1) \cdots P(x_2 \mid x_1)P(x_1) \\ &= \prod_{i=1}^{n} P(x_i|x_{i\ -1}, ..., x_1). \end{aligned}$$

Regla de la cadena

Esta identidad se llama **regla de la cadena**. Se mantiene para cualquier conjunto de variables aleatorias. Comparándola con la Ecuación (13.2), vemos que la especificación de la distribución conjunta es equivalente a la afirmación general de que, para cada variable X_i en la red,

$$\mathbf{P}(X_i|X_{i-1},...,X_1) = \mathbf{P}(X_i|Padres(X_i)), \tag{13.3}$$

siempre que $Padres(X_i) \subseteq \{X_{i-1},...,X_1\}$. Esta última condición se cumple numerando los nodos en

Orden topológico

orden topológico, es decir, en cualquier orden consistente con la estructura del grafo dirigido. Por ejemplo, los nodos en la Figura 13.2 podrían ser ordenados *B,E,A,J,M; E,B,A,M,J*; y así sucesivamente. Lo que dice la Ecuación (13.3) es que la red bayesiana es una representación correcta del dominio sólo si cada nodo es condicionalmente independiente de sus otros predecesores en el nodo ordenado, dado sus padres. Podemos satisfacer esta condición con esta metodología:

1. *Nodos*: Primero determine el conjunto de variables que se requieren para modelar el dominio. Ahora ordénelos, $\{X_1,..., X_n\}$. Cualquier orden funcionará, pero la red resultante será más compacta si las variables se ordenan de tal manera que las causas preceden a los efectos.
2. *Enlaces*: Para $i = 1$ a n hacer:
   - Elija un conjunto mínimo de padres para X_i de $X_1,...,X_{i-1}$, de tal manera que se cumpla la Ecuación (13.3).
   - Para cada padre inserte un enlace desde el padre a X_i.
   - CPTs: Escriba la tabla de probabilidad condicional, $\mathbf{P}(X_i|\ Padres(X_i))$.

De manera intuitiva, los padres del nodo X_i deben contener todos aquellos nodos en $X_1,..., X_{i-1}$ que *influyen de manera directa* en X_i. Por ejemplo, supongamos que hemos completado la red en la Figura 13.2 excepto por la elección de los padres para *MaryLlama*. *MaryLlama* es influida por si hay un *Robo* o un *Terremoto*, pero no *directamente* influida. Intuitivamente, nuestro conocimiento del dominio nos dice que estos eventos influyen en el comportamiento de llamada de Mary sólo a través de su efecto en la alarma. Además, dado el estado de la alarma, si John llama no tiene ninguna influencia en la llamada de Mary. Formalmente hablando, creemos que la siguiente declaración de independencia condicional se mantiene:

$$\mathbf{P}(MaryLlama|JohnLlama,Alarma,Terremoto,Robo) = \mathbf{P}(MaryLlama|Alarma).$$

Por lo tanto, *Alarma* será el único nodo padre para *MaryLlama*.

Debido a que cada nodo está conectado sólo a nodos anteriores, este método de construcción garantiza que la red es acíclica. Otra propiedad importante de las redes de Bayes es que no contienen valores de probabilidad redundantes. Si no hay redundancia, entonces no hay posibilidad de inconsistencia: *es imposible que el ingeniero de datos o el experto en el dominio creen una red bayesiana que viole los axiomas de probabilidad.*

Compactación y orden de nodos

Además de ser una representación completa y no redundante del dominio, en la mayoría de los casos una red de Bayes es mucho más *compacta* que la distribución conjunta completa. Esta propiedad es lo que hace factible manejar dominios con muchas variables. La compactación de las redes bayesianas es un ejemplo de una propiedad general de sistemas **localmente estructurados** (también llamados **dispersos**). En un sistema localmente estructurado cada subcomponente interactúa de manera directa sólo con un número limitado de otros componentes, con independencia del número total de estos. La estructura local por lo regular se asocia con una complejidad lineal, no exponencial.

Localmente estructurados
Dispersos

En el caso de las redes de Bayes, es razonable suponer que en la mayoría de los dominios cada variable aleatoria es influida de manera directa por al menos k variables adicionales. Si asumimos n variables booleanas para simplificar, entonces la cantidad de información necesaria para especificar cada tabla de probabilidad condicional será de al menos 2^k números, y la red completa puede ser especificada por $2^k \cdot n$ números. En contraste, la distribución conjunta contiene 2^n números. Para hacer esto concreto, supongamos que tenemos $n = 30$ nodos, cada uno con cinco padres ($k = 5$). Entonces, la red bayesiana requiere 960 números, pero la distribución conjunta completa requiere más de mil millones.

Especificar las tablas de probabilidad condicional para una red por completo conectada, en la que cada variable tiene todos sus predecesores como padres, requiere la misma cantidad de información que especificar la distribución conjunta en forma de tabla. Por esta razón, en general, dejamos fuera enlaces aun cuando existe una ligera dependencia, porque la ligera ganancia en precisión no vale la complejidad adicional en la red. Por ejemplo, uno podría objetar nuestra red de robo con el argumento de que si hay un gran terremoto, entonces John y Mary no llamarían incluso si escucharan la alarma, porque asumen que el terremoto es la causa. Si se debe agregar el enlace desde *Terremoto* a *JohnLlama* y *MaryLlama* (y, así, agrandar las tablas) depende de la importancia de obtener probabilidades más precisas en comparación con el costo de especificar la información adicional.

Incluso en un dominio estructurado localmente, obtendremos una red de Bayes compacta sólo si elegimos bien el orden de los nodos. ¿Qué pasa si elegimos el orden incorrecto? Considere de nuevo el ejemplo de robo. Suponga que decidimos agregar los nodos en el orden *MaryLlama*, *JohnLlama*, *Alarma*, *Robo*, *Terremoto*. Luego obtenemos la red un poco más complicada que se muestra en la Figura 13.3(a). El proceso es el siguiente:

- Añadiendo *MaryLlama*: Sin padres.
- Añadiendo *JohnLlama*: Si Mary llama, eso probablemente significa que la alarma ha sonado, lo que hace más probable que John llame. Por lo tanto, *JohnLlama* necesita a *MaryLlama* como padre.
- Añadiendo *Alarma*: Claramente, si ambos llaman, es más probable que la alarma haya sonado que si solo uno o ninguno llama, por lo que necesitamos a *MaryLlama* y *JohnLlama* como padres.
- Añadiendo *Robo*: Si sabemos el estado de la alarma, entonces la llamada de John o Mary podría darnos información sobre nuestro teléfono sonando o la música de Mary, pero no sobre el robo:

  $$\mathbf{P}(Robo|Alarma, JohnLlama, MaryLlama) = \mathbf{P}(Robo \mid Alarma).$$

  Por lo tanto, necesitamos sólo *Alarma* como padre.
- Añadiendo *Terremoto*: Si la alarma está encendida, es más probable que haya habido un terremoto. (La alarma es un tipo de detector de terremotos). Pero si sabemos que ha habido un robo, entonces eso explica la alarma, y la probabilidad de un terremoto sería apenas superior a la normal. Por lo tanto, necesitamos tanto *Alarma* como *Robo* como padres.

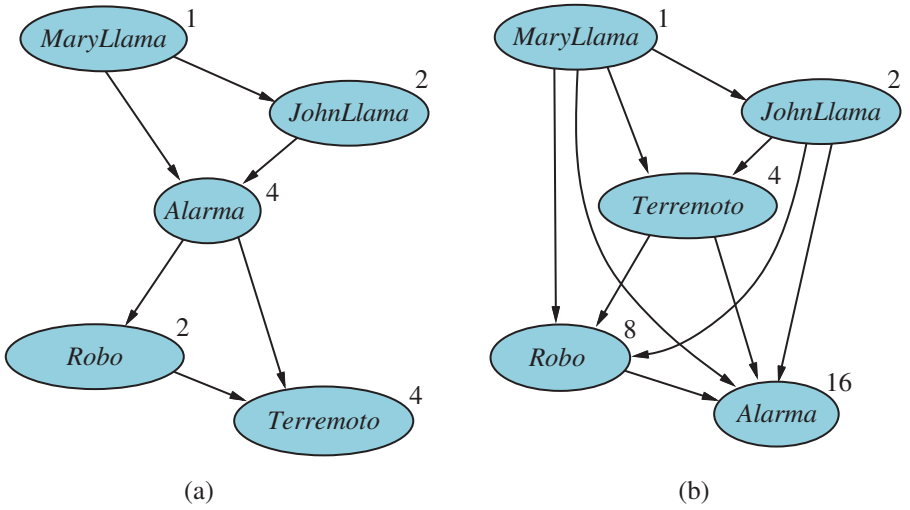


Figura 13.3 La estructura de la red y el número de parámetros depende del orden de introducción. (a) La estructura obtenida con el orden M, J, A, B, E. (b) La estructura obtenida con M, J, E, B, A. Cada nodo está anotado con el número de parámetros requeridos; 13 en total para (a) y 31 para (b). En la Figura 13.2, sólo se requerían 10 parámetros.

La red resultante tiene dos enlaces más que la red original de la Figura 13.2 y requiere 13 probabilidades condicionales en vez de 10. Lo que es peor, unos enlaces representan relaciones tenues que requieren juicios de probabilidad difíciles y antinaturales, como evaluar la probabilidad de *Terremoto*, dado el *Robo* y la *Alarma*. Este fenómeno es muy general y se relaciona con la distinción entre modelos **causales** y **diagnósticos**, introducida en la Sección 12.5.1 (véase también el Ejercicio 13.WUMD). *Si nos atenemos a un modelo causal, tendremos que especificar menos números, y los números a menudo serán más fáciles de obtener.* Por ejemplo, en el dominio de la medicina, Tversky y Kahneman (1982) han demostrado que los médicos expertos prefieren emitir juicios de probabilidad para las reglas causales que para las diagnósticas. En la sección 13.5 se profundiza en los modelos causales.

La Figura 13.3(b) muestra un orden de nodos muy malo: *MaryLlama*, *JohnLlama*, *Terremoto*, *Robo*, *Alarma*. Esta red requiere que se especifiquen 31 probabilidades distintas, exactamente el mismo número que la distribución conjunta completa. Sin embargo, es importante darse cuenta de que cualquiera de las tres redes puede representar *la misma distribución conjunta*. Las dos versiones en la Figura 13.3 no representan todas las relaciones de independencia condicional y, por lo tanto, especifican muchos números innecesarios.

13.2.1 Relaciones de independencia condicional en redes bayesianas

A partir de la semántica de las redes de Bayes definidas en la Ecuación (13.2), podemos derivar una serie de propiedades de independencia condicional. Ya hemos visto la propiedad de que una variable es condicionalmente independiente de sus otros predecesores, dados sus padres. También es posible demostrar la propiedad más general de "no descendientes", donde:

Descendiente

Cada variable es condicionalmente independiente de sus no ***descendientes****, dados sus padres.*

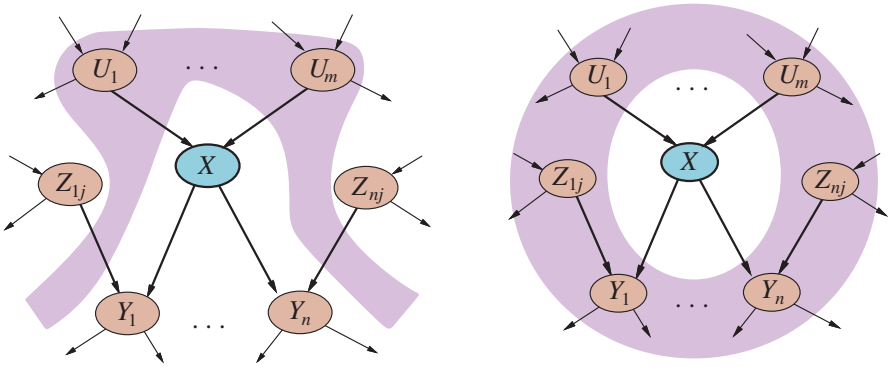


Figura 13.4 (a) Un nodo X es condicionalmente independiente de sus no descendientes (por ejemplo, los Z_{ij}s) dados sus padres (los U_is mostrados en el área sombreada). (b) Un nodo X es condicionalmente independiente de todos los otros nodos en la red, dado su manto de Markov (área sombreada).

Por ejemplo, en la Figura 13.2, la variable *JohnLlama* es independiente de *Robo*, *Terremoto* y *MaryLlama* dado el valor de *Alarma*. La definición se ilustra en la Figura 13.4(a).

Resulta que la propiedad de no descendientes combinada con la interpretación de los parámetros de la red $\theta(X_i \mid Padres(X_i))$ como probabilidades condicionales $\mathbf{P}(X_i \mid Padres(X_i))$ es suficiente para reconstruir la distribución conjunta completa dada en la Ecuación (13.2). Es decir, uno puede ver la semántica de las redes de Bayes de manera diferente: en vez de definir la distribución conjunta completa como el producto de distribuciones condicionales, la red define un conjunto de propiedades de independencia condicional. La distribución conjunta completa puede derivarse de esas propiedades.

Otra propiedad de independencia importante se deduce de la propiedad de no descendientes:

> una variable es condicionalmente independiente de todos los otros nodos en la red, dados sus padres, hijos y padres de los hijos, es decir, dado su **manto de Markov.**

Manto de Markov

(El Ejercicio 13.MARB le pide que demuestre esto). Por ejemplo, la variable *Robo* es independiente de *JohnLlama* y *MaryLlama*, dada *Alarma* y *Terremoto*. Esta propiedad se ilustra en la Figura 13.4(b). La propiedad del manto de Markov hace posible algoritmos de inferencia que usan procesos de muestreo estocástico completamente locales y distribuidos, como se explica en la Sección 13.4.2.

La pregunta de independencia condicional más general que uno podría hacer en una red de Bayes es si un conjunto de nodos **X** es condicionalmente independiente de otro conjunto **Y,** dado un tercer conjunto **Z**. Esto se puede determinar de manera eficiente examinando la red de Bayes para ver si **Z** **d-separa** **X** e **Y**. El proceso funciona de la siguiente manera:

D-separación

1. Considere sólo el **subgrafo ancestral** que consiste en **X**, **Y**, **Z** y sus ancestros. Subgrafo ancestral
2. Añada enlaces entre cualquier par de nodos no enlazados que compartan un hijo común; ahora tenemos el llamado **grafo moral**. Grafo moral
3. Reemplace todos los enlaces dirigidos por enlaces no dirigidos.
4. Si **Z** bloquea todos los caminos entre **X** e **Y** en el grafo resultante, entonces **Z** d-separa **X** y **Y**. En ese caso, **X** es condicionalmente independiente de **Y**, dado **Z**. De lo contrario, la red de Bayes original no requiere independencia condicional.

En resumen, la d-separación significa separación en el subgrafo ancestral moralizado y no dirigido. Aplicando la definición a la red de robos en la Figura 13.2, podemos deducir que *Robo* y *Terremoto* son independientes dado el conjunto vacío (es decir, son absolutamente independientes); que no necesariamente son condicionalmente independientes dado *Alarma*; y que *JohnLlama* y *MaryLlama* son condicionalmente independientes dado Alarma. También se observa que la propiedad del manto de Markov se deriva directamente de la propiedad de d-separación, ya que el manto de Markov de una variable la d-separa de todas las demás variables.

13.2.2 Representación eficiente de distribuciones condicionales

Incluso si el número máximo de k padres es pequeño, llenar la CPT para un nodo requiere hasta $O(2^k)$ números y quizás una gran cantidad de experiencia con todos los posibles casos de condicionamiento. De hecho, este es un escenario de caso peor en el que la relación entre padres e hijo es por completo arbitraria. Por lo general, tales relaciones se describen por una **distribución canónica** que se ajusta a algún patrón estándar. En tales casos, la tabla completa se puede especificar al nombrar el patrón y suministrar algunos parámetros.

Distribución canónica

Nodos deterministas

El ejemplo más simple lo proporcionan los **nodos deterministas**. El valor de un nodo determinista se especifica por los valores de sus padres, sin incertidumbre. La relación puede ser lógica: por ejemplo, la relación entre los nodos padres *Canadiense*, *Estadounidense*, *Mexicano* y el nodo hijo *Norteamericano* que es el hijo en la disyunción de los padres. La relación también puede ser numérica: por ejemplo, el *MejorPrecio* para un coche es el mínimo de los precios en cada concesionario de la zona; y el *AguaAlmacenada* en un embalse al final del año es la suma de la cantidad original, más las entradas (ríos, escorrentías, precipitaciones) y menos las salidas (liberaciones, evaporación, filtraciones).

Muchos sistemas de redes bayesianas permiten al usuario especificar funciones deterministas usando un lenguaje de programación de propósito general; esto hace posible incluir elementos complejos como modelos climáticos globales o simuladores de redes eléctricas dentro de un modelo probabilístico.

Independencia específica del contexto

Otro patrón importante que ocurre en la práctica es la **independencia específica del contexto** o CSI. Una distribución condicional exhibe CSI si una variable es condicionalmente independiente de algunos de sus padres dados *ciertos valores* de otros. Por ejemplo, supongamos que el *Daño* a su coche ocurrido durante un tiempo depende de la *Robustez* de su coche y de si ocurrió o no un *Accidente* en ese periodo. Si *Accidente* es falso, entonces el *Daño*, si lo hay, no depende de la *Robustez* de su coche. (Podría haber daños por vandalismo en la pintura o ventanillas del coche, pero asumiremos que todos los coches están igualmente sujetos a tales daños). Decimos que el *Daño* es específicamente independiente del contexto de la *Robustez* dado *Accidente* = *falso*. Los sistemas de redes bayesianas con frecuencia implementan CSI usando una sintaxis **if-then-else** para especificar distribuciones condicionales; por ejemplo, uno podría escribir

$$\mathbf{P}(Daño \mid Robustez, Accidente) = \\ \quad \textbf{if } (Accidente = falso) \textbf{ then } d_1 \textbf{ else } d_2(Robustez)$$

donde d_1 y d_2 representan distribuciones arbitrarias. Al igual que con el determinismo, la presencia de CSI en una red facilita inferencia eficiente. Todos los algoritmos de inferencia exacta mencionados en la Sección 13.3 pueden modificarse para aprovechar el CSI para acelerar el cálculo.

Ruidosa-OR

Las relaciones inciertas suelen ser caracterizadas por las relaciones lógicas **ruidosas**. El ejemplo estándar es la relación **ruidosa-OR**, una generalización del OR lógico. En lógica

Resfriado	*Gripe*	*Malaria*	$P(fiebre\|\cdot)$	$P(\neg fiebre\|\ )$
f	*f*	*f*	0.0	1.0
f	*f*	*t*	0.9	**0.1**
f	*t*	*f*	0.8	**0.2**
f	*t*	*t*	0.98	$0.02 = 0.2 \times 0.1$
t	*f*	*f*	0.4	**0.6**
t	*f*	*t*	0.94	$0.06 = 0.6 \times 0.1$
t	*t*	*f*	0.88	$0.12 = 0.6 \times 0.2$
t	*t*	*t*	0.988	$0.012 = 0.6 \times 0.2 \times 0.1$

Figura 13.5 Una tabla de probabilidad condicional completa para ¶(*Fiebre*|*Resfriado*, *Gripe*, *Malaria*), asumen un modelo ruidoso-OR con los tres valores *q* mostrados en negritas.

proposicional, podríamos decir que la *Fiebre* es verdadera si y sólo si el *Resfriado*, la *Gripe* o la *Malaria* son verdaderos. El modelo ruidoso-OR permite la incertidumbre sobre la capacidad de cada padre para causar que el hijo sea verdadero: la relación causal entre padre e hijo puede estar *inhibida*, por lo que un paciente podría tener un resfriado, pero no presentar fiebre.

El modelo hace dos suposiciones. Primero, asume que todas las posibles causas están listadas. (Si faltan algunas, siempre podemos agregar un llamado **nodo de fuga** que cubre "causas diversas"). En segundo, asume que la inhibición de cada padre es independiente de la inhibición de cualquier otro padre: por ejemplo, lo que inhibe a la *Malaria* de causar fiebre es independiente de lo que inhibe a la *Gripe* de causar fiebre. Dadas estas suposiciones, la *Fiebre* es *falsa* si y sólo si todos sus padres *verdaderos* están inhibidos, y la probabilidad de esto es el producto de las probabilidades de inhibición q_j para cada padre. Suponga que estas probabilidades de inhibición individuales son:

Nodo de fuga

$$Q_{\text{resfriado}} = P(\neg fiebre \mid resfriado, \neg gripe, \neg malaria) = 0.6,$$
$$Q_{\text{gripe}} = P(\neg fiebre \mid \neg resfriado, gripe, \neg malaria) = 0.2,$$
$$q_{\text{malaria}} = P(\neg fiebre \mid \neg resfriado, \neg gripe, malaria) = 0.1.$$

Entonces, a partir de esta información y las suposiciones de ruido-OR, se puede construir toda la CPT. La regla general es que

$$P(x_i \mid padres(X_i)) = 1 - \prod_{\{j:Xj = verdadero\}} q_j,$$

donde el producto se toma sobre los padres que se establecen como verdaderos para esa fila de la CPT. La Figura 13.5 ilustra este cálculo.

En general, las relaciones lógicas ruidosas en las que una variable depende de *k* padres se describen utilizando parámetros $O(k)$ en lugar de $O(2^k)$ para la tabla de probabilidad condicional completa. Esto facilita la evaluación y el aprendizaje. Por ejemplo, la red CPCS (Pradhan *et al.*, 1994) usa distribuciones ruidosas-OR y ruidosas-MAX para modelar relaciones entre enfermedades y síntomas en medicina interna. Con 448 nodos y 906 enlaces, sólo requiere 8 254 parámetros en lugar de 133 931 430 para una red con CPTs completas.

13.2.3 Redes de Bayes con variables continuas

Muchos problemas del mundo real involucran cantidades continuas, como altura, masa, temperatura y dinero. Por definición, las variables continuas tienen un número infinito de posibles valores, por lo que es imposible especificar probabilidades condicionales explícitas para cada valor. Una forma de manejar las variables continuas es con **discretización**, es decir, dividiendo los posibles valores en un conjunto fijo de intervalos. Por ejemplo, las temperaturas podrían dividirse en tres categorías: (<0 °C), (0 °C–100 °C), y (>100 °C). Al elegir el número de categorías, hay un equilibrio entre la pérdida de precisión y las grandes CPTs que pueden llevar a tiempos de ejecución lentos.

Discretización

Otro enfoque es definir una variable continua usando una de las familias estándar de funciones de densidad de probabilidad (ver Apéndice A). Por ejemplo, una distribución gaussiana (o normal) $\mathcal{N}(x; \mu, \sigma^2)$ se especifica con sólo dos parámetros, la media μ y la varianza σ^2. Otra solución, a veces llamada representación **no paramétrica**, es definir la distribución condicional implícita con una colección de instancias, cada una conteniendo valores específicos de las variables padre e hijo. Exploramos este enfoque al detalle en el Capítulo 19.

No paramétrica

Una red con variables discretas y continuas se llama **red bayesiana híbrida**. Para especificar una red híbrida, tenemos que especificar dos nuevos tipos de distribuciones: la distribución condicional para una variable continua dada padres discretos o continuos, y la distribución condicional para una variable discreta dada padres continuos. Considere el ejemplo simple en la Figura 13.6, donde un cliente compra una fruta dependiendo de su costo, que depende del tamaño de la cosecha y de si el esquema de subsidios del gobierno opera. La variable *Costo* es continua y tiene padres continuos y discretos; la variable *Compras* es discreta y tiene un padre continuo.

Red bayesiana híbrida

Para la variable *Costo*, necesitamos especificar $\mathbf{P}(Costo \mid Cosecha, subsidio)$. El padre discreto se maneja por enumeración, es decir, especificando tanto $\mathbf{P}(Costo \mid Cosecha, subsidio)$ como $\mathbf{P}(Costo \mid Cosecha, \neg subsidio)$. Para manejar la *Cosecha*, especificamos cómo la distribución sobre el costo c depende del valor continuo h de la *Cosecha*. Es decir, especificamos los *parámetros* de la distribución de costos como una función de h. La elección más común es la distribución condicional **lineal-gaussiana**, en la que el hijo tiene una distribución gaussiana cuya media μ varía de manera lineal con el valor del padre y cuya desviación estándar σ es fija. Necesitamos dos distribuciones, una para *subsidio* y otra para $\neg subsidio$, con diferentes parámetros:

Lineal-gaussiana

$$P(c|h,subsidy) = \mathcal{N}(c;a_t h + b_t,\sigma_t^2) = \frac{1}{\sigma_t\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{c-(a_t h+b_t)}{\sigma_t}\right)^2}$$

$$P(c|h,\neg subsidy) = \mathcal{N}(c;a_f h + b_f,\sigma_f^2) = \frac{1}{\sigma_f\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{c-(a_f h+b_f)}{\sigma_f}\right)^2}.$$

Para este ejemplo, entonces, la distribución condicional para el *Costo* se especifica nombrando la distribución lineal-gaussiana y proporcionando los parámetros a_t, b_t, σ_t, a_f, b_f, y σ_f. Las Figuras 13.7(a) y (b) muestran ambas relaciones. Vea que en cada caso la pendiente de c versus h es negativa, porque el costo disminuye a medida que aumenta el tamaño de la cosecha. (Por supuesto, la suposición de linealidad implica que el costo se vuelve negativo en algún punto; el modelo lineal es razonable sólo si el tamaño de la cosecha se limita a un rango estrecho). La Figura 13.7(c) muestra la distribución $P(c|h)$, promedia sobre los dos posibles valores de *Subsidio* y asume que cada uno tiene una probabilidad previa de 0.5. Esto muestra que aun con modelos simples, es posible representar distribuciones interesantes.

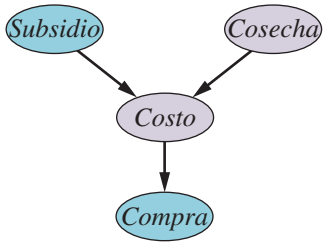


Figura 13.6 Red simple con variables discretas (*Subsidio* y *Compra*) y variables continuas (*Cosecha* y *Costo*).

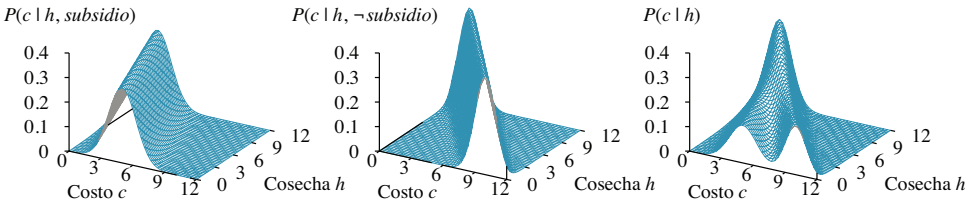


Figura 13.7 Los grafos en (a) y (b) muestran la distribución de probabilidad sobre el *Costo* como una función del tamaño de la *Cosecha*, con *Subsidio* verdadero y falso, respectivamente. El grafo (c) muestra la distribución *P*(*Costo*|Cosecha), obtenida sumando los dos casos de subsidio.

La distribución condicional lineal-gaussiana tiene algunas propiedades especiales. Una red que contiene sólo variables continuas con distribuciones lineal-gaussianas tiene una distribución conjunta que es una distribución gaussiana multivariante (ver Apéndice A) sobre todas las variables (Ejercicio 13.LGEX). Además, la distribución posterior, dada cualquier evidencia, también tiene esta propiedad.[2] Cuando se agregan variables discretas como padres (no como hijos) de variables continuas, la red define una **gaussiana condicional**, o CG, distribución: dada cualquier asignación a la gaussiana condicional variable discreta, la distribución sobre las variables continuas es una gaussiana multivariante.

Gaussiana condicional

Ahora pasamos a las distribuciones para variables discretas con padres continuos. Considere, por ejemplo, el nodo *Comprar* en la Figura 13.6. Parece razonable suponer que el cliente comprará si el costo es bajo y no comprará si es alto, y que la probabilidad de comprar varía suavemente en alguna región intermedia. Es decir, la distribución condicional es como una función de umbral "suave". Una forma de hacer umbrales suaves es usar la *integral* de la distribución normal estándar:

$$\Phi(x) = \int_{-\infty}^{x} N(s;0,1)ds.$$

[2] Se deduce que la inferencia en redes lineal-gaussianas toma sólo $O(n^3)$ veces, en el peor de los casos, independiente de la topología de la red. En la Sección 13.3, la inferencia para redes de variables discretas es NP-difícil.

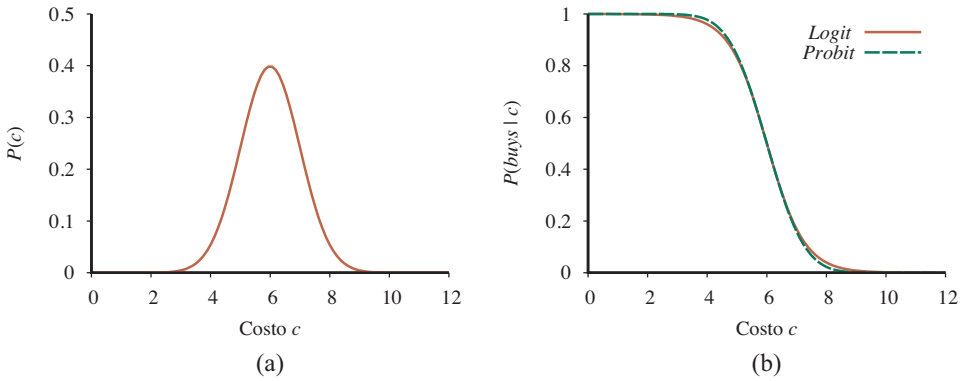


Figura 13.8 (a) Distribución normal (gaussiana) para el umbral de costo, centrada en $\mu = 6.0$ con desviación estándar $\sigma = 1.0$. (b) Modelos expit y probit para la probabilidad de compras dado el costo, para los parámetros $\mu = 6.0$ y $\sigma = 1.0$.

$\Phi(x)$ es una función creciente de x, mientras que la probabilidad de comprar disminuye con el costo, por lo que aquí invertimos la función:

$$P(buys|Cost = c) = 1 - \Phi((c - \mu)/\sigma),$$

lo que significa que el umbral de costo ocurre alrededor de μ, el ancho de la región de umbral es proporcional a σ, y la probabilidad de comprar disminuye a medida que aumenta el costo. Este modelo **probit** (pronunciado "pro-bit" y abreviatura de "unidad de probabilidad") se ilustra en la Figura 13.8(a). La forma se puede justificar proponiendo que el proceso de decisión subyacente tiene un umbral duro, pero que la ubicación precisa del umbral está sujeta a ruido gaussiano aleatorio.

Probit

Una alternativa al modelo probit es el modelo **expit** o **logit inverso**. Utiliza la **Función logit** $1/(1 + e^{-x})$ para producir un umbral suave, mapea cualquier x a un valor entre 0 y 1. Nuevamente, para nuestro ejemplo, lo invertimos para hacer una función decreciente; también escalamos el exponente por $4/\sqrt{2\pi}$ para coincidir con la pendiente del probit en la media:

Expit
Logit inverso
Función logit

$$P(\text{compra}|Costo = c) = 1 - \frac{1}{1 + exp\left(-\frac{4}{\sqrt{2\pi}} \cdot \frac{c-\mu}{\sigma}\right)}$$

Esto se ilustra en la Figura 13.8(b). Las dos distribuciones se ven similares, pero el logit en realidad tiene "colas" mucho más largas. El probit con frecuencia se ajusta mejor a situaciones reales, pero la función logística a veces es matemáticamente más fácil de manejar. Se usa ampliamente en el aprendizaje automático. Ambos modelos permiten generalizarse para manejar múltiples padres continuos tomando una combinación lineal de los valores de los padres. Esto también funciona para los padres discretos si sus valores son enteros; por ejemplo, con k padres booleanos, cada uno visto con valores 0 o 1, la entrada a la distribución expit o probit sería una combinación lineal ponderada con k parámetros, dando un modelo bastante similar al modelo ruidoso-OR discutido con anterioridad.

13.2.4 Caso de estudio: Seguro de coche

Una compañía de seguros de automóviles recibe solicitud de un individuo para asegurar un vehículo específico y debe decidir la prima anual adecuada a cobrar, basada en las reclamaciones

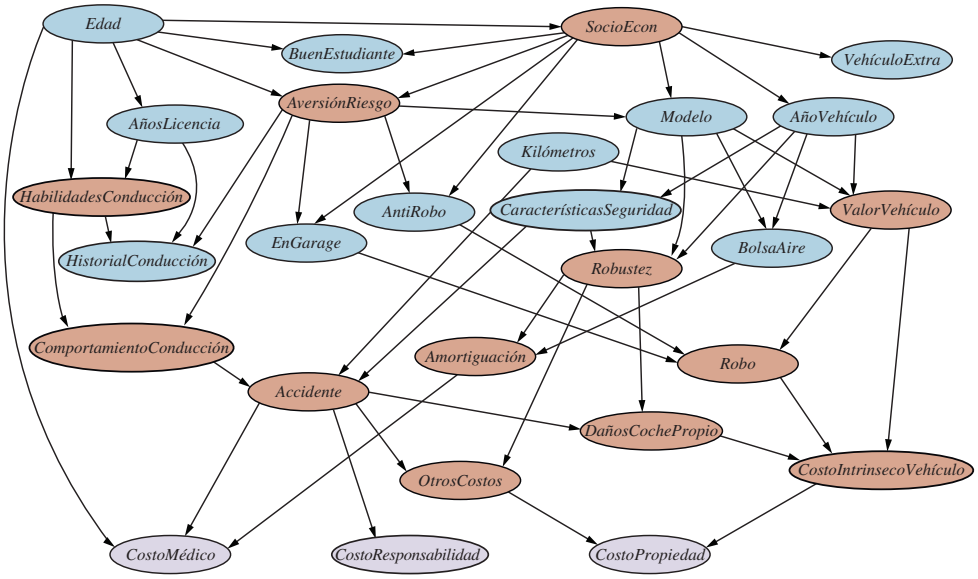


Figura 13.9 Una red bayesiana para evaluar las solicitudes de seguro de automóvil.

anticipadas que pagará para este solicitante. La tarea es construir una red bayesiana que capture la estructura causal del dominio y proporcione una distribución precisa y bien calibrada sobre las variables de salida, dada la evidencia disponible en el formulario de solicitud.[3] La red bayesiana incluirá **variables ocultas** que no son ni variables de entrada ni de salida, pero son esenciales para la estructurar la red para que sea razonablemente dispersa con un número manejable de parámetros. Las variables ocultas están sombreadas en marrón en la Figura 13.9.

Variables ocultas

Las reclamaciones a pagar, sombreadas en lavanda en la Figura 13.9, son de tres tipos: *CostoMédico* por cualquier lesión sufrida por el solicitante; *CostoResponsabilidad* por demandas presentadas por otras partes contra el solicitante y la compañía; y *CostoPropiedad* por daños al vehículo de cualquiera de las partes y pérdida del vehículo por robo. El formulario de solicitud pide la siguiente información de entrada (los nodos azul claro en la Figura 13.9):

- Sobre el solicitante: *Edad; AñosLicencia*, cuánto tiempo desde que se obtuvo por primera vez una licencia de conducir; *HistorialConducción*, algún resumen, quizá basado en "puntos", de accidentes recientes e infracciones de tráfico, y (para estudiantes) un indicador *BuenEstudiante* para un promedio de calificaciones de 3.0 (B) en una escala de 4 puntos.
- Sobre el vehículo: el *Modelo* y *AñoVehículo*; si tiene *BolsaAire,* y algún resumen de *CaracterísticasSeguridad* como frenos antibloqueo y advertencia de colisión.
- Sobre la situación de conducción: los *Kilómetros* anuales conducidos y cuán seguro está el vehículo *EnGarage*, si es que lo está.

[3] La red mostrada en la Figura 13.9 no está en uso actual, pero su estructura ha sido validada con expertos en seguros. En la práctica, la información solicitada en los formularios de solicitud varía según la compañía y la jurisdicción; por ejemplo, algunos solicitan el *Género*, y el modelo ciertamente podría hacerse más detallado y sofisticado.

Ahora necesitamos pensar en cómo organizar estas variables en una estructura causal. Las variables ocultas clave son si ocurrirá un *Robo* o *Accidente* en el próximo periodo de tiempo. Obviamente, uno no puede pedirle al solicitante que prediga estos; tienen que inferirse de la información disponible y la experiencia previa del asegurador.

¿Cuáles son los factores causales que llevan al *Robo*? El *Modelo* es ciertamente importante, algunos modelos son robados con mucha mayor frecuencia que otros porque hay un mercado de reventa eficiente para esos vehículos y sus partes; el *ValorVehículo* también importa, porque un vehículo viejo, golpeado o de alto kilometraje tiene un valor de reventa más bajo. Además, un vehículo que está *EnGarage* y tiene un dispositivo *AntiRobo* es más difícil de robar. La variable oculta *ValorVehículo* depende, a su vez, del *Modelo*, *AñoVehículo* y *Kilómetros*. *ValorVehículo* también dicta la cantidad de pérdida cuando ocurre un *Robo*, por lo que es uno de los contribuyentes a *CostoIntrínsecoVehículo* (el otro será accidentes, a los que llegaremos en breve).

Es común en modelos de este tipo introducir otra variable oculta, *SocioEcon*, la categoría socioeconómica del solicitante. Se cree que esto influye en una amplia gama de comportamientos y características. En nuestro modelo, no hay evidencia directa en forma de variables de ingresos y ocupación observadas;[4] pero *SocioEcon* influye en *Modelo* y *AñoVehículo, y* también afecta a *VehículoExtra* y *BuenEstudiante*, y depende algo de la *Edad*.

Para cualquier compañía de seguros, quizá la variable oculta más importante es *AversionRiesgo*: ¡las personas que son aversas al riesgo son buenos riesgos de seguro! La *Edad* y *SocioEcon* afectan a *AversiónRiesgo*, y sus "síntomas" incluyen la elección del solicitante de si el vehículo está *EnGarage* y tiene dispositivos *AntiRobo* y *CaracterísticasSeguridad*.

Al predecir futuros accidentes, la clave es el futuro *ComportamientoConducción* del solicitante, que es influido tanto por *AversiónRiesgo* como por *HabilidadesConducción*; este último, a su vez, depende de la *Edad* y *AñosLicencia*. El comportamiento de conducción pasado del solicitante se refleja en el *HistorialConducción*, que también depende de *AversiónRiesgo* y *HabilidadesConducción*, así como de *AñosLicencia* (porque alguien que recién comenzó a conducir puede no haber tenido tiempo para acumular una letanía de accidentes e infracciones). De esta manera, *HistorialConducción* proporciona evidencia sobre *AversiónRiesgo* y *HabilidadesConducción*, que ayudan a predecir el futuro *ComportamientoConducción*.

Podemos pensar en *ComportamientoConducción* como una tendencia por kilómetro a conducir de manera propensa a accidentes; si un *Accidente* ocurre en un tiempo establecido también depende del *Kilómetro* anual y de las *CaracterísticasSeguridad* del vehículo. Si ocurre un *Accidente*, hay tres tipos de costos: el *CostoMédico* para el solicitante depende de la *Edad* y *Amortiguación*, que a su vez depende de la *Robustez* del coche y si tiene *BolsaAire*; el *CostoResponsabilidad* (médico, dolor y sufrimiento, pérdida de ingresos, etc.) para el otro conductor; y el *CostoPropiedad* para el solicitante y el otro conductor, ambos dependen (de diferentes maneras) de la *Robustez* del coche y del *ValorVehículo* del solicitante.

Hemos ilustrado el tipo de razonamiento que se usa para desarrollar la topología y las variables ocultas en una red bayesiana. También necesitamos especificar los rangos y las distribuciones condicionales para cada variable. Para los rangos, la decisión principal es si hacer la variable discreta o continua. Por ejemplo, la *Robustez* del vehículo podría ser una variable continua entre 0 y 1, o una variable discreta con rango {*Lata*, *Normal*, *Tanque*}.

---

[4] Algunas compañías de seguros también adquieren el historial de crédito del solicitante para ayudarse a evaluar el riesgo; esto proporciona información considerable sobre la categoría socioeconómica. Siempre que se usen variables ocultas de este tipo, uno debe tener cuidado de que no se conviertan de manera inadvertida en proxies para variables como raza, que no se permite utilizar en decisiones de seguros. Las técnicas para evitar sesgos de este tipo se describen en el Capítulo 19.

Las variables continuas proporcionan más precisión, pero hacen que la inferencia exacta sea imposible, excepto en casos especiales. Una variable discreta con muchos valores posibles puede hacer tedioso completar las correspondientes tablas de probabilidad condicional inmensas y hace que la inferencia exacta sea más costosa, a menos que el valor de la variable siempre se observe. Por ejemplo, *Modelo* en un sistema real tendría miles de posibles valores, y esto hace que su hijo *ValorVehículo* tenga un CPT enorme que tendría que llenarse desde bases de datos de la industria; pero, porque el *Modelo* siempre se observa, esto no contribuye a la complejidad de la inferencia: de hecho, los valores observados para los tres padres eligen una fila relevante del CPT para *ValorVehículo*.

Las distribuciones condicionales en el modelo se dan en el repositorio de código para el libro; proporcionamos una versión sólo con variables discretas, para las cuales es posible realizar la inferencia exacta. En la práctica, muchas de las variables serían continuas y las distribuciones condicionales se aprenderían de los datos históricos sobre los solicitantes y sus reclamaciones de seguro. Veremos cómo aprender modelos de redes bayesinas a partir de datos en el Capítulo 20.

La pregunta final es, por supuesto, cómo hacer inferencia en la red para hacer predicciones. Ahora volvemos a esta pregunta. Para cada método de inferencia que describimos, evaluaremos el método en la red de seguros para medir los requisitos de tiempo y espacio del método.

13.3 Inferencia exacta en redes bayesianas

La tarea básica para cualquier sistema de inferencia probabilística es calcular la distribución de probabilidad posterior para un conjunto de **variables de consulta**, dado algún **evento** observado; en general, alguna asignación de valores a un conjunto de **variables de evidencia**.[5] Para simplificar la presentación, consideraremos sólo una variable de consulta a la vez; los algoritmos se pueden extender a consultas con múltiples variables. (Por ejemplo, podemos resolver la consulta $\mathbf{P}(U,V|\mathbf{e})$ multiplicando $\mathbf{P}(V|\mathbf{e})$ y $\mathbf{P}(U|V,\mathbf{e})$). Usaremos la notación del Capítulo 12: X denota la variable de consulta; $\mathbf{E}$ denota el conjunto de variables de evidencia $E_1,...,E_m$, y $\mathbf{e}$ es un evento observado particular; $\mathbf{Y}$ denota las variables ocultas (no evidencia, no consulta) $Y_1,...,Y_\ell$. Por lo tanto, el conjunto completo de variables es $\{X\} \cup \mathbf{E} \cup \mathbf{Y}$. Una consulta típica pide la distribución de probabilidad posterior $\mathbf{P}(X|\mathbf{e})$.

Evento

En la red de robos, podríamos observar el evento en el que *JohnLlama* = *verdadero* y *MaryLlama* = *verdadero*. Luego podríamos preguntar, por ejemplo, la probabilidad de que haya ocurrido un robo:

$$\mathbf{P}(Robo|JohnLlama= verdadero, MaryLlama= verdadero) = \langle 0.284,0.716\rangle.$$

En esta sección discutimos algoritmos exactos para calcular probabilidades posteriores así como la complejidad de esta tarea. Resulta que el caso general es intratable, por lo que la Sección 13.4 cubre métodos para la inferencia aproximada.

13.3.1 Inferencia por enumeración

En el Capítulo 12 se explicó que cualquier probabilidad condicional se calcula sumando términos de la distribución conjunta completa. Más específicamente, una consulta $\mathbf{P}(X|\mathbf{e})$ se puede responder usando la Ecuación (12.9), que repetimos aquí para conveniencia:

$$\mathbf{P}(X|\mathbf{e}) = \alpha\, \mathbf{P}(X,\mathbf{e}) = \alpha \sum_{\mathbf{y}} \mathbf{P}(X,\mathbf{e},\mathbf{y}).$$

[5] Otra tarea ampliamente estudiada es encontrar la explicación más probable para alguna evidencia observada. Esta y otras tareas se discuten en las notas al final del capítulo.

Ahora, una red bayesiana da una representación completa de la distribución conjunta completa. Más específicamente, la Ecuación (13.2) en la página 415 muestra que los términos $\boldsymbol{P}(x,\mathbf{e},\mathbf{y})$ en la distribución conjunta se escriben como productos de probabilidades condicionales de la red. Por lo tanto, *una consulta se puede responder usando una red bayesiana calculando sumas de productos de probabilidades condicionales de la red*.

Considere la consulta $\mathbf{P}(Robo|JohnLlama = verdadero, MaryLlama = verdadero)$. Las variables ocultas para esta consulta son *Terremoto* y *Alarma*. De la Ecuación (12.9), usando las letras iniciales para las variables para acortar las expresiones, tenemos

$$\mathbf{P}(B|j,m) = \alpha\mathbf{P}(B,j,m) = \alpha \sum_e \sum_a \mathbf{P}(B,j,m,e,a).$$

La semántica de las redes bayesianas (Ecuación (13.2)) luego nos da una expresión en términos de entradas CPT. Para simplificar, hacemos esto sólo para $Robo = verdadero$:

$$P(b|j,m) = \alpha \sum_e \sum_a P(b)P(e)P(a|b,e)P(j|a)P(m|a). \tag{13.4}$$

Para calcular esta expresión, tenemos que agregar cuatro términos, cada uno calculado multiplicando cinco números. En el peor de los casos, donde tenemos que sumar casi todas las variables, habrá $O(2^n)$ términos en la suma, cada uno un producto de $O(n)$ valores de probabilidad. Una implementación ingenua tendría, por lo tanto, una complejidad $O(n2^n)$.

Esto se puede reducir a $O(2^n)$ aprovechando la estructura anidada de la computación. En términos simbólicos, esto significa mover las sumas hacia adentro tanto como sea posible en expresiones como la Ecuación (13.4). Podemos hacer esto porque no todos los factores en el producto de probabilidades dependen de todas las variables. Así que tenemos

$$P(b|j,m) = \alpha P(b) \sum_e P(e) \sum_a P(a|b,e)P(j|a)P(m|a). \tag{13.5}$$

Esta expresión se puede evaluar recorriendo las variables en orden, multiplicando las entradas CPT a medida que avanzamos. Para cada suma, también necesitamos recorrer los posibles valores de la variable. La estructura de este cálculo se muestra como un árbol en la Figura 13.10. Usando los números de la Figura 13.2, obtenemos $P(b|j,m) = \alpha \times 0.00059224$. El cálculo correspondiente para $\neg b$ produce $\alpha \times 0.0014919$; por lo tanto,

$$\mathbf{P}(B|j,m) = \alpha \langle 0.00059224, 0.0014919 \rangle \approx \langle 0.284, 0.716 \rangle.$$

Es decir, la posibilidad de un robo, dado las llamadas de ambos vecinos, es de cerca de 28%.

El algoritmo ENUMERATION-ASK en la Figura 13.11 evalúa estos árboles de expresión usando recursión de profundidad, primero de izquierda a derecha. El algoritmo es similar en estructura al algoritmo de retroceso para resolver CSPs (Figura 6.5) y el algoritmo DPLL para satisfacibilidad (Figura 7.17). Su complejidad es sólo lineal en el número de variables: el algoritmo suma sobre la distribución conjunta completa sin construirla de manera explícita. Desafortunadamente, su tiempo de complejidad para una red con n variables booleanas (sin contar las variables de evidencia) siempre es $O(2^n)$, mejor que el $O(n2^n)$ para el enfoque antes descrito, pero aún muy sombrío. Para la red de seguros en la Figura 13.9, relativamente pequeña, la inferencia exacta usando enumeración requiere cerca de 227 millones de operaciones aritméticas para una consulta típica en las variables de costo.

Sin embargo, si observa cuidadosamente el árbol en la Figura 13.10 verá que contiene *subexpresiones repetidas*. Los productos $P(j|a)P(m|a)$ y $P(j|\neg a)P(m|\neg a)$ se calculan dos veces, una vez para cada valor de E. La clave para la inferencia eficiente en las redes de Bayes es evitar tal desperdicio de cálculos. La siguiente sección describe un método general para hacer esto.

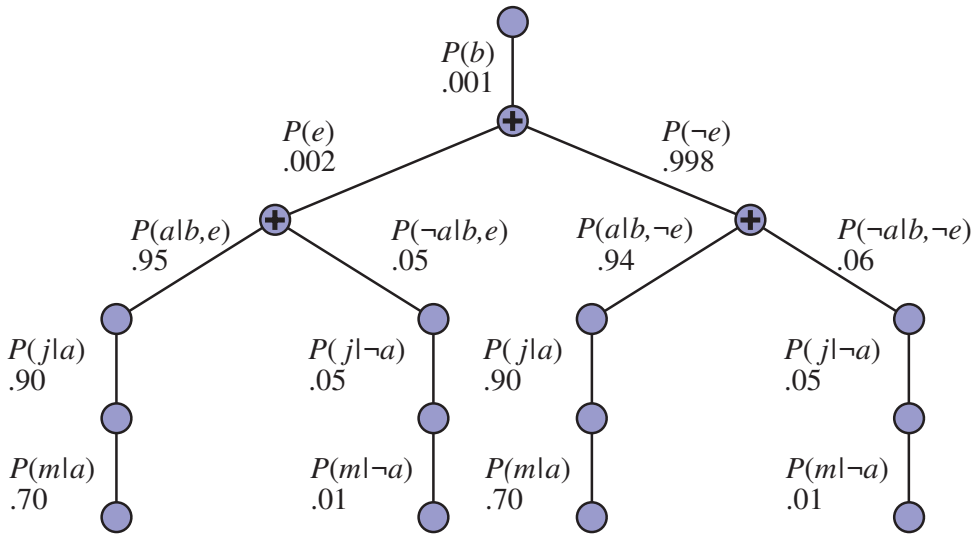


Figura 13.10 La estructura de la expresión mostrada en la Ecuación (13.5). La evaluación procede de arriba hacia abajo, multiplicando valores a lo largo de cada camino y sumando en los nodos "+". Observe la repetición de los caminos para *j* y *m*.

function ENUMERATION-ASK(*X*,**e**,*bn*) **returns** una distribución sobre *X*
  inputs: *X*, la variable de consulta
    e, valores observados para variables **E**
    bn, una red de Bayes con variables *vars*

  Q(*X*)← una distribución sobre *X*, inicialmente vacía
  for each valor x_i de *X* **do**
    Q(x_i)←ENUMERATE-ALL(*vars*,$\mathbf{e}_{xi}$)
      donde $\mathbf{e}_{x_i}$ es **e** extendida con $X = x_i$
  return NORMALIZE(**Q**(*X*))

function ENUMERATE-ALL(*vars*,**e**) **returns** un número real
  if EMPTY?(*vars*) **then return** 1.0
  V←FIRST(*vars*)
  if *V* es una variable de evidencia con valor *v* en **e**
    then return $P(v|padres(V)) \times$ ENUMERATE-ALL(REST(*vars*),**e**)
    else return $\sum_v P(v|padres(V)) \times$ ENUMERATE-ALL(REST(*vars*), $\mathbf{e}_v$)
      donde $\mathbf{e}_v$ es **e** extendida con $V = v$

Figura 13.11 El algoritmo de enumeración para la inferencia exacta en las redes de Bayes.

13.3.2 El algoritmo de eliminación de variables

El algoritmo de enumeración puede mejorarse eliminando cálculos repetidos del tipo ilustrado en la Figura 13.10. La idea es simple: hacer el cálculo una vez y guardar los resultados para su uso posterior. Esta es una forma de programación dinámica. Hay varias versiones de este enfoque; presentamos el algoritmo de **eliminación de variables**, que es el más simple. La eliminación de variables funciona evaluando expresiones como la Ecuación (13.5) en orden de *derecha a izquierda* (es decir, *de abajo hacia arriba* en la Figura 13.10). Los resultados intermedios se almacenan, y las sumas sobre cada variable se hacen sólo para aquellas partes de la expresión que dependen de la variable.

Eliminación de variables

Ilustremos este proceso para la red de robos. Evaluamos la expresión

$$\mathbf{P}(B|j,m) = \underbrace{\alpha\, \mathbf{P}(B)}_{\mathbf{f}_1(B)} \sum_e \underbrace{P(e)}_{\mathbf{f}_2(E)} \sum_a \underbrace{\mathbf{P}(a|B,e)P(j|a)P(m|a)}_{\mathbf{f}_3(A,B,E)\ \mathbf{f}_4(A)\ \mathbf{f}_5(A)}.$$

Factor

Observe que hemos anotado cada parte de la expresión con el nombre del **factor** correspondiente; cada factor es una matriz indexada por los valores de sus variables de argumento. Por ejemplo, los factores $\mathbf{f}_4(A)$ y $\mathbf{f}_5(A)$ correspondientes a $P(j|a)$ y $P(m|a)$ dependen sólo de A porque J y M están fijados por la consulta. Por lo tanto, son vectores de dos elementos:

$$\mathbf{f}_4(A) = \begin{pmatrix} P(j|a) \\ P(j|\neg a) \end{pmatrix} = \begin{pmatrix} 0.90 \\ 0.05 \end{pmatrix} \qquad \mathbf{f}_5(A) = \begin{pmatrix} P(m|a) \\ P(m|\neg a) \end{pmatrix} = \begin{pmatrix} 0.70 \\ 0.01 \end{pmatrix}.$$

$\mathbf{f}_3(A,B,E)$ será una matriz $2 \times 2 \times 2$, que es difícil de mostrar en la página impresa. (El "primer" elemento es dado por $P(a|b,e) = 0.95$ y el "último" por $P(\neg a|\neg b,\neg e) = 0.999$). En términos de factores, la expresión de consulta se escribe como

$$\mathbf{P}(B|j,m) = \alpha \mathbf{f}_1(B) \times \sum_e \mathbf{f}_2(E) \times \sum_a \mathbf{f}_3(A,B,E) \times \mathbf{f}_4(A) \times \mathbf{f}_5(A).$$

Producto punto a punto

Aquí el operador "×" no es la multiplicación de matrices ordinaria sino el **producto punto a punto** de operación, que se describirá en breve.

El proceso de evaluación suma variables (de derecha a izquierda) de productos punto de factores para producir nuevos factores, eventualmente produciendo un factor que constituye la solución; es decir, la distribución posterior sobre la variable de consulta. Los pasos son los siguientes:

- Primero, sumamos A del producto de $\mathbf{f}_3$, $\mathbf{f}_4$ y $\mathbf{f}_5$. Esto nos da un nuevo factor 2×2 $\mathbf{f}_6(B,E)$ cuyos índices varían sólo en B y E:

$$\begin{aligned} \mathbf{f}_6(B,E) &= \sum_a \mathbf{f}_3(A,B,E) \times \mathbf{f}_4(A) \times \mathbf{f}_5(A) \\ &= (\mathbf{f}_3(a,B,E) \times \mathbf{f}_4(a) \times \mathbf{f}_5(a)) + (\mathbf{f}_3(\neg a,B,E) \times \mathbf{f}_4(\neg \mathrm{a}) \times \mathbf{f}_5(\neg \mathrm{a})). \end{aligned}$$

  Ahora nos queda la expresión

$$\mathbf{P}(B|j,m) = \alpha \mathbf{f}_1(B) \times \sum_e \mathbf{f}_2(E) \times \mathbf{f}_6(B,E).$$

- A continuación, sumamos E del producto de $\mathbf{f}_2$ y $\mathbf{f}_6$:

$$\begin{aligned} \mathbf{f}_7(B) &= \sum_e \mathbf{f}_2(E) \times \mathbf{f}_6(B,E) \\ &= \mathbf{f}_2(e) \times \mathbf{f}_6(B,e) + \mathbf{f}_2(\neg e) \times \mathbf{f}_6(B,\neg e). \end{aligned}$$

  Esto deja la expresión

$$\mathbf{P}(B|j,m) = \alpha \mathbf{f}_1(B) \times \mathbf{f}_7(B)$$

  que se evalúa tomando el producto punto a punto y normalizando el resultado.

X	Y	$\mathbf{f}(X,Y)$	Y	Z	$\mathbf{g}(Y,Z)$	X	Y	Z	$\mathbf{h}(X,Y,Z)$
t	t	.3	t	t	.2	t	t	t	.3 × .2 = .06
t	f	.7	t	f	.8	t	t	f	.3 × .8 = .24
f	t	.9	f	t	.6	t	f	t	.7 × .6 = .42
f	f	.1	f	f	.4	t	f	f	.7 × .4 = .28
						f	t	t	.9 × .2 = .18
						f	t	f	.9 × .8 = .72
						f	f	t	.1 × .6 = .06
						f	f	f	.1 × .4 = .04

Figura 13.12 Ilustrando la multiplicación punto a punto: $\mathbf{f}(X,Y) \times \mathbf{g}(Y,Z) = \mathbf{h}(X,Y,Z)$.

Examinando esta secuencia, vemos que se requieren dos operaciones computacionales básicas: producto punto a punto de un par de factores, y sumando una variable de un producto de factores. La siguiente sección describe cada una de estas operaciones.

Operaciones en factores

El producto punto a punto de dos factores **f** y **g** produce un nuevo factor **h** cuyas variables son la *unión* de las variables en **f** y **g**, y cuyos elementos se dan por el producto de los elementos correspondientes en los dos factores. Suponga que los dos factores tienen variables $Y_1,...,Y_k$ en común. Entonces tenemos

$$\mathbf{f}(X_1...X_j,Y_1...Y_k) \times \mathbf{g}(Y_1...Y_k,Z_1,...Z_\ell) = \mathbf{h}(X_1...X_j,Y_1...Y_k,Z_1...Z\ell)$$

Si todas las variables son binarias, entonces **f** y **g** tienen 2^{j+k} y $2^{k+\ell}$ entradas, respectivamente, y el producto punto a punto tiene $2^{j+k+\ell}$ entradas. Por ejemplo, dados dos factores $\mathbf{f}(X,Y)$ y $\mathbf{g}(Y,Z)$, el producto punto a punto $\mathbf{f} \times \mathbf{g} = \mathbf{h}(X,Y,Z)$ tiene $2^{1+1+1} = 8$ entradas, como se ilustra en la Figura 13.12. Observe que el factor resultante de un producto punto a punto puede contener más variables que cualquier de los factores que se están multiplicando y que el tamaño de un factor es exponencial en el número de variables. Aquí es donde surge la complejidad tanto de espacio como de tiempo en el algoritmo de eliminación de variables.

La eliminación de una variable de un producto de factores se realiza sumando las submatrices formadas al fijar la variable a cada uno de sus valores por turno. Por ejemplo, para eliminar X de $\mathbf{h}(X,Y,Z)$, escribimos

$$\mathbf{h}_2(Y,Z) = \sum_x \mathbf{h}(X,Y,Z) = \mathbf{h}(x,Y,Z) + \mathbf{h}(\neg x,Y,Z)$$

$$= \begin{pmatrix} .06 & .24 \\ .42 & .28 \end{pmatrix} + \begin{pmatrix} .18 & .72 \\ .06 & .04 \end{pmatrix} = \begin{pmatrix} .24 & .96 \\ .48 & .32 \end{pmatrix}.$$

El único truco es notar que cualquier factor que *no* dependa de la variable a ser sumada puede ser movido fuera de la sumatoria. Por ejemplo, para sumar X del producto de **f** y **g**, podemos mover **g** fuera de la sumatoria:

$$\sum_x \mathbf{f}(X,Y) \times \mathbf{g}(Y,Z) = \mathbf{g}(Y,Z) \times \sum_x \mathbf{f}(X,Y).$$

```
function ELIMINATION-ASK(X,e,bn) devuelve una distribución sobre X
  inputs: X, la variable de consulta
          e, valores observados para variables E
          bn, una red bayesiana con variables vars
  factors←[ ]
  for each V in ORDER(vars) do
    factors←[MAKE-FACTOR(V,e)] + factors
    if V es una variable oculta then factors←SUM-OUT(V, factors)
  return NORMALIZE(POINTWISE-PRODUCT(factors))
```

Figura 13.13 El algoritmo de eliminación de variables para la inferencia exacta en las redes de Bayes.

Esto es potencialmente mucho más eficiente que calcular el producto punto a punto **h** más grande primero y luego sumar X de eso.

Observe que las matrices *no* se multiplican hasta que necesitamos sumar una variable del producto acumulado. En ese momento, multiplicamos sólo aquellas matrices que incluyen la variable a ser sumada. Dadas las funciones para el producto punto a punto y la suma, el algoritmo de eliminación de variables en sí permite ser escrito de manera bastante simple, como se muestra en la Figura 13.13.

Orden de las variables y relevancia de las variables

El algoritmo en la Figura 13.13 incluye una función ORDER no especificada para elegir un orden para las variables. Cada elección de ordenamiento produce un algoritmo válido, pero diferentes ordenamientos causan diferentes factores intermedios a ser generados durante el cálculo. Por ejemplo, en el cálculo antes mostrado, eliminamos A antes que E; si lo hacemos al revés, el cálculo se convierte en

$$\mathbf{P}(B|j,m) = \alpha \mathbf{f}_1(B) \times \sum_a \mathbf{f}_4(A) \times \mathbf{f}_5(A) \times \sum_e \mathbf{f}_2(E) \times \mathbf{f}_3(A,B,E),$$

durante el cual se generará un nuevo factor $\mathbf{f}_6(A, B)$.

En general, los requisitos de tiempo y espacio de la eliminación de variables están dominados por el tamaño del factor más grande, construido durante la operación del algoritmo. Esto, a su vez, es determinado por el orden de eliminación de variables y por la estructura de la red. Resulta intratable determinar el ordenamiento óptimo, pero hay varias buenas heurísticas disponibles. Un método muy efectivo es uno codicioso: eliminar la variable que minimiza el tamaño del próximo factor a ser construido.

Consideremos una consulta más: $\mathbf{P}(JohnLlama|Robo = true)$. Como de costumbre (ver Ecuación (13.5)), el primer paso es escribir la suma anidada:

$$\mathbf{P}(J|b) = \alpha P(b) \sum_e P(e) \sum_a P(a|b,e) \mathbf{P}(J|a) \sum_m P(m|a).$$

Al evaluar esta expresión de derecha a izquierda, notamos algo interesante: $\sum_m P(m|a)$ es igual a 1 por definición. Por lo tanto, no había necesidad de incluirlo en primer lugar; la variable M es *irrelevante* para esta consulta. Otra forma de decir esto es que el resultado de la consulta $P(JohnLlama|Robo = true)$ no cambia si eliminamos *MaryLlama* de la red en total. En general, podemos eliminar cualquier nodo hoja que no sea una variable de consulta o una variable de

evidencia. Después de su eliminación, podría haber algunos nodos hoja adicionales, y estos también pueden ser irrelevantes. Continuando este proceso, eventualmente encontramos que *cada variable que no es un antecesor de una variable de consulta o variable de evidencia es irrelevante para la consulta*. Un algoritmo de eliminación de variables permite, por lo tanto, eliminar todas estas variables antes de evaluar la consulta.

Cuando se aplica a la red de seguros mostrada en la Figura 13.9, la eliminación de variables muestra una mejora considerable sobre el algoritmo de enumeración ingenua. Usando el orden topológico inverso para las variables, la inferencia exacta usando eliminación es casi de 1 000 veces más rápida que el algoritmo de enumeración.

13.3.3 La complejidad de la inferencia exacta

La complejidad de la inferencia exacta en las redes de Bayes depende fuertemente de la estructura de la red. La red de robos de la Figura 13.2 pertenece a la familia de redes en las que, a lo sumo, hay un camino no dirigido (es decir, ignorando la dirección de las flechas) entre cualesquiera dos nodos en la red. Estos se llaman redes **conectadas individualmente** o **poliárbol**, y tienen una propiedad particularmente agradable: *El tiempo y la complejidad espacial de la inferencia exacta en poliárboles lineal en el tamaño de la red.* Aquí, el tamaño se define como el número de entradas CPT; si el número de padres de cada nodo está limitado por una constante, entonces la complejidad también será lineal en el número de nodos. Estos resultados son válidos para cualquier ordenamiento consistente con el ordenamiento topológico de la red (Ejercicio 13.VEEX).

Conectadas individualmente

Poliárbol

Para redes **conectadas múltiplemente**, como la red de seguros en la Figura 13.9, la eliminación variable puede tener tiempo exponencial y complejidad espacial en el peor de los casos, aun cuando el número de padres por nodo es limitado. Esto no es sorprendente cuando uno considera que *porque incluye inferencia en lógica proposicional como un caso especial, la inferencia en redes de Bayes es NP-difícil*. Para demostrar esto, necesitamos averiguar cómo codificar un problema de satisfacción proposicional como una red de Bayes, de tal manera que la ejecución de la inferencia en esta red nos diga si las sentencias proposicionales originales son satisfactorias o no. (En el lenguaje de la teoría de la complejidad, se busca **reducir** problemas de satisfacción a problemas de inferencia de red de Bayes). Esto resulta muy sencillo. La Figura 13.14 muestra cómo codificar un problema 3-SAT particular. Las variables proposicionales se convierten en las variables raíz de la red, cada una con probabilidad previa 0.5. La siguiente capa de nodos corresponde a las cláusulas, con cada variable de cláusula C_j conectada a las variables apropiadas como padres. La distribución condicional para una variable de cláusula es una disyunción determinista, con negación según sea necesario, de modo que cada variable de cláusula es verdadera si y sólo si la asignación a sus padres satisface esa cláusula. Finalmente, S es la conjunción de las variables de cláusula.

Conectadas múltiplemente

Reducir

Para determinar si la oración original es satisfactoria, evaluamos $P(S = verdadero)$. Si la oración es satisfactoria, entonces hay alguna asignación posible a las variables lógicas que hace que S sea verdadero; en la red de Bayes esto significa que hay un mundo posible con probabilidad distinta de cero en el que las variables raíz tienen esa asignación, las variables de cláusula tienen valor *verdadero*, y S tiene valor verdadero. Por lo tanto, $P(S = verdadero) > 0$ para una oración satisfactoria. Al contrario, $P(S = verdadero) = 0$ para una oración insatisfactoria: todos los mundos con $S = verdadero$ tienen probabilidad 0. Por lo tanto, podemos usar la inferencia de red de Bayes para resolver problemas 3-SAT; así, concluimos que la inferencia de red de Bayes es NP-difícil.

De hecho, podemos hacer más que esto. Observe que la probabilidad de cada asignación satisfactoria es 2^{-n} para un problema con n variables. Por lo tanto, el *número* de asignaciones satisfactorias es $P(S = verdadero)/(2^{-n})$. Debido a que calcular el *número* de asignaciones satisfactorias para

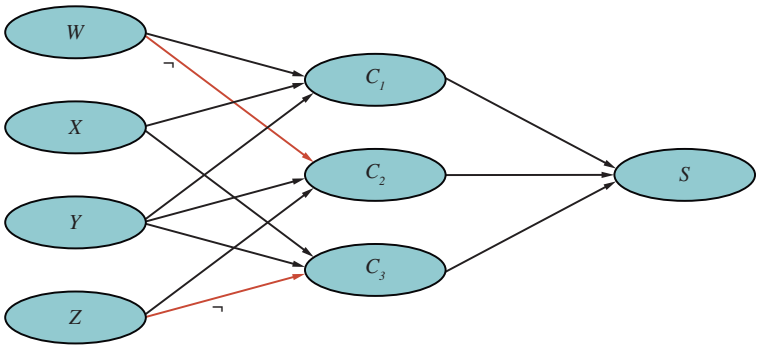


Figura 13.14 Codificación de red de Bayes de la sentencia 3-CNF. $(W \vee X \vee Y) \wedge (\neg W \vee Y \vee Z) \wedge (X \vee Y \vee \neg Z)$.

un 3-SAT el problema es #P-completo ("número-P completo"); esto significa que la inferencia de red de Bayes es #P-duro, es decir, estrictamente más difícil que los problemas NP-completos.

Existe una estrecha conexión entre la complejidad de la inferencia de red de Bayes y la complejidad de los problemas de satisfacción de restricciones (CSPs). Como discutimos en el Capítulo 6, la dificultad de resolver un CSP discreto está relacionada con cuán "parecido a un árbol" es su gráfico de restricciones. Medidas como el **ancho del árbol**, que limitan la complejidad de resolver un CSP, también se pueden aplicar de manera directa a las redes de Bayes. Además, el algoritmo de eliminación de variables se puede generalizar para resolver CSPs así como redes de Bayes.

Así como reducimos problemas de satisfacción a la inferencia de red de Bayes, podemos reducir la inferencia de red de Bayes a la satisfacción, lo que nos permite aprovechar la poderosa maquinaria desarrollada para la resolución de SAT (ver Capítulo 7). En este caso, la reducción es a una forma particular de resolución de SAT llamada **conteo de modelos ponderados** (WMC). El conteo de modelos regulares cuenta el número de asignaciones satisfactorias para una expresión SAT; WMC suma el peso total de esas asignaciones satisfactorias, donde, en esta aplicación, el peso es esencialmente el producto de las probabilidades condicionales para cada asignación de variables dada a sus padres. (Ver Ejercicio 13.WMCX para detalles). En parte porque la tecnología de resolución de SAT ha sido tan bien optimizada para aplicaciones a gran escala, la inferencia de red de Bayes a través de WMC es competitiva y, a veces, superior a otros algoritmos exactos en redes con gran ancho de árbol.

Conteo de modelos ponderados (WMC)

13.3.4 Algoritmos de agrupación

El algoritmo de eliminación de variables es simple y eficiente para responder consultas individuales. Sin embargo, si queremos calcular las probabilidades posteriores para todas las variables en una red, puede ser menos eficiente. Por ejemplo, en una red de poliárbol, se necesitaría emitir $O(n)$ consultas que cuestan $O(n)$ cada una, para un total de tiempo $O(n^2)$. Usando algoritmos de **agrupación** (también conocidos como algoritmos de **árbol de unión**), el tiempo se reduce a $O(n)$. Por esta razón, estos algoritmos se usan ampliamente en herramientas comerciales de red de Bayes.

Agrupación

Árbol de unión

La idea básica de la agrupación es unir nodos individuales de la red para formar nodos de agrupación de modo que la red resultante sea un poliárbol. Por ejemplo, la red múltiplemente conectada mostrada en la Figura 13.15(a) se puede convertir en un poliárbol combinando el nodo *Aspersor* y *Lluvia* en un nodo de agrupación llamado *Aspersor+Lluvia*, como se muestra en la

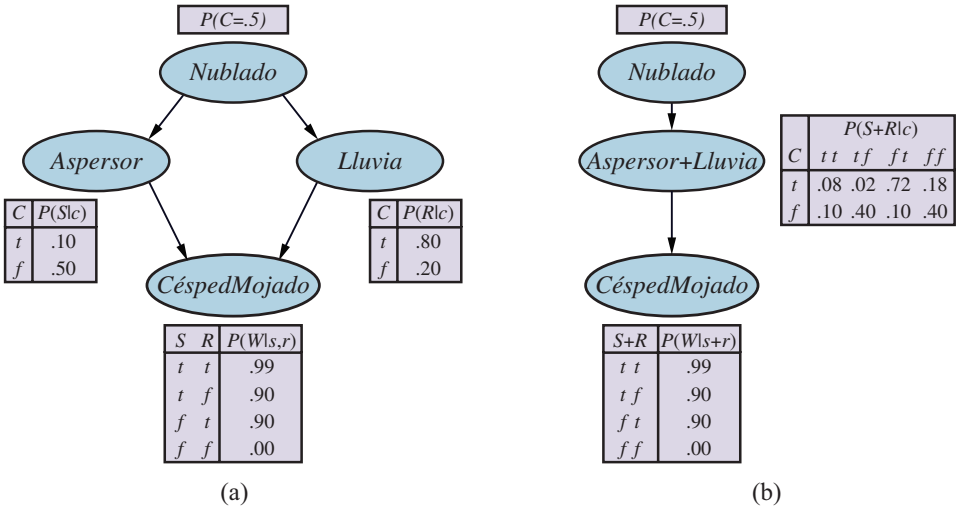


Figura 13.15 (a) Una red múltiplemente conectada que describe la rutina diaria de césped de Mary: cada mañana, ella revisa el clima; si está nublado, no suele encender el aspersor; si el aspersor está encendido, o si llueve durante el día, el césped estará mojado. Así, *Nublado* afecta a *CéspedMojado* a través de dos vías causales diferentes. (b) Un equivalente agrupado de la red múltiplemente conectada.

Figura 13.15(b). Los dos nodos booleanos son reemplazados por un **meganodo** que toma cuatro posibles valores: *tt*, *tf*, *ft*, y *ff*. El meganodo sólo tiene un padre, la variable booleana *Nublado*, por lo que hay dos casos de condicionamiento. Aunque este ejemplo no lo muestra, el proceso de agrupación típicamente produce meganodos que comparten algunas variables.

Meganodo

Una vez que la red está en forma de poliárbol, se requiere un algoritmo de inferencia de propósito especial, porque los métodos de inferencia ordinarios no permiten manejar meganodos que comparten variables entre sí. En esencia, el algoritmo es una forma de propagación de restricciones (ver Capítulo 6) donde las restricciones aseguran que los meganodos vecinos concuerden con la probabilidad posterior de cualquier variable que tengan en común. Con una contabilidad cuidadosa, este algoritmo puede calcular probabilidades posteriores para todos los nodos no evidenciados en la red en tiempo *lineal*, en el tamaño de la red agrupada. Sin embargo, la dificultad NP del problema no ha desaparecido: si una red requiere tiempo exponencial y espacio con eliminación de variables, entonces los CPTs en la red agrupada serán exponencialmente grandes.

13.4 Inferencia aproximada para redes bayesianas

Dada la intratabilidad de la inferencia exacta en redes grandes, ahora consideraremos métodos de inferencia aproximados. Esta sección describe algoritmos de muestreo aleatorizados, también llamados algoritmos de **Monte Carlo**, que proporcionan respuestas aproximadas cuya precisión depende del número de muestras generadas. Trabajan generando eventos aleatorios basados en las probabilidades en la red de Bayes y contando las diferentes respuestas encontradas en esos aleatorios eventos. Con suficientes muestras, podemos acercarnos arbitrariamente a recuperar la verdadera distribución de probabilidad, siempre que la red bayesiana no tenga distribuciones condicionales deterministas.

Monte Carlo

Los algoritmos de Monte Carlo, de los cuales el recocido simulado (página 115) es un ejemplo, se usan en muchas ramas de la ciencia para estimar cantidades difíciles de calcular de manera exacta. En esta sección, estamos interesados en el muestreo aplicado al cálculo de probabilidades posteriores en las redes bayesianas. Describimos dos familias de algoritmos: muestreo directo y muestreo de cadena de Markov. Se mencionan varios otros enfoques para la inferencia aproximada en las notas al final del capítulo.

13.4.1 Métodos de muestreo directo

El elemento primitivo en todo algoritmo de muestreo es la generación de muestras a partir de una distribución de probabilidad conocida. Así, una moneda imparcial puede considerarse una variable aleatoria *Moneda* con valores $\langle cara, cruz\rangle$ y una distribución previa $\mathbf{P}(Moneda) = \langle 0.5, 0.5\rangle$. Muestrear a partir de esta distribución es exactamente como lanzar la moneda: con probabilidad 0.5 devolverá *cara*, y con probabilidad 0.5 devolverá *cruz*. Dada una fuente de números aleatorios r distribuidos de manera uniforme en el rango [0,1], es un simple asunto muestrear cualquier distribución en una sola variable, ya sea discreta o continua. Esto se hace construyendo la distribución acumulativa para la variable y devolviendo el primer valor cuya probabilidad acumulativa supera r (ver Ejercicio 13.PRSA).

Comenzamos con un proceso de muestreo aleatorio para una red bayesiana que no tiene evidencia asociada con ella. La idea es muestrear cada variable a su vez, en orden topológico. La distribución de probabilidad de la cual se extrae el valor está condicionada a los valores ya asignados a los padres de la variable. (Debido a que muestreamos en orden topológico, se garantiza que los padres ya tienen valores). Este algoritmo se muestra en la Figura 13.16. Al aplicarlo a la red en la Figura 13.15(a) con el orden *Nublado*, *Aspersor*, *Lluvia*, *CéspedMojado*, podríamos producir un evento aleatorio de la siguiente manera:

1. Muestra de $\mathbf{P}(Nublado) = \langle 0.5,0.5\rangle$, el valor es *verdadero*.
2. Muestra de $\mathbf{P}(Aspersor|Nublado = true) = \langle 0.1,0.9\rangle$, el valor es *falso*.
3. Muestra de $\mathbf{P}(Lluvia|Nublado = true) = \langle 0.8,0.2\rangle$, el valor es *verdadero*.
4. Muestra de $\mathbf{P}(CéspedMojado|Aspersor = false, Lluvia = true) = \langle 0.9, 0.1\rangle$, el valor es *verdadero*.

En este caso, PRIOR-SAMPLE devuelve el evento [*verdadero*,*falso*,*verdadero*,*verdadero*].

Es fácil ver que PRIOR-SAMPLE genera muestras de la distribución conjunta previa especificada por la red. Primero, dejemos que $S_{PS}(x_1,...,x_n)$ sea la probabilidad de que un evento

---

function PRIOR-SAMPLE(*bn*) **returns** un evento muestreado del anterior especificado por *bn*
  inputs: *bn*, una red bayesiana que especifica la distribución conjunta $\mathbf{P}(X_1,...,X_n)$

  x← un evento con n elementos
  for each variable X_i **in** $X_1,...,X_n$ **do**
    x[i]← una muestra aleatoria de $\mathbf{P}(X_i|padres(X_i))$
  return x

Figura 13.16 Un algoritmo de muestreo que genera eventos a partir de una red bayesiana. Cada variable se muestrea de acuerdo con la distribución condicional dada los valores muestreados para los padres de la variable.

específico sea generado por el algoritmo PRIOR-SAMPLE. Sólo mirando el proceso de muestreo, tenemos

$$S_{PS}(x_1...x_n) = \prod_{i=1}^{n} P(x_i|padres(X_i))$$

porque cada paso de muestreo depende sólo de los valores de los padres. Esta expresión debería parecer familiar, porque también es la probabilidad del evento según la representación de la red bayesiana de la distribución conjunta, como se indica en la Ecuación (13.2). Es decir, tenemos

$$S_{PS}(x_1...x_n) = P(x_1...x_n).$$

Este simple hecho facilita responder preguntas usando muestras.

En cualquier algoritmo de muestreo, las respuestas se calculan contando las muestras reales generadas. Supongamos que hay N muestras totales producidas por el algoritmo PRIOR-SAMPLE, y dejemos que $N_{PS}(x_1,...,x_n)$ sea el número de veces que el evento específico $x_1,...,x_n$ ocurre en el conjunto de muestras. Esperamos que este número, como fracción del total, converja en el límite a su valor esperado de acuerdo con la probabilidad de muestreo:

$$\lim_{N\to\infty} \frac{N_{PS}(x_1,...,x_n)}{N} = S_{PS}(x_1,...,x_n) = P(x_1,...,x_n). \tag{13.6}$$

Por ejemplo, considere el evento antes producido: [*true,false,true,true*]. La probabilidad de muestreo para este evento es

$$S_{PS}(true,false,true,true) = 0.5 \times 0.9 \times 0.8 \times 0.9 = 0.324.$$

Por lo tanto, en el límite de N grande, esperamos que el 32.4 % de las muestras sean de este evento.

Siempre que usemos una igualdad aproximada ("≈") en lo que sigue, lo decimos en este sentido exacto, que la probabilidad estimada se vuelve exacta en el límite de muestra grande. Tal estimación se llama **consistente**. Por ejemplo, es posible producir una estimación consistente de la probabilidad de cualquier evento parcialmente especificado $x_1,...,x_m$, donde $m \leq n$, de la siguiente manera: Consistente

$$P(x_1,...,x_m) \approx N_{PS}(x_1,...,x_m)/N. \tag{13.7}$$

Es decir, la probabilidad del evento se estima como la fracción de todos los eventos completos generados por el proceso de muestreo que coinciden con el evento parcialmente especificado. Usaremos $\hat{P}$ (pronunciado "P-hat") para significar una probabilidad estimada. Entonces, si generamos 1 000 muestras de la red de aspersores, y 511 de ellas tienen *Rain* = *true*, entonces la probabilidad estimada de lluvia es $\hat{P}$ (*Rain* = *true*) = 0.511.

Muestreo de rechazo en redes bayesianas

El **muestreo de rechazo** es un método general para producir muestras a partir de una distribución difícil de muestrear, dada una distribución fácil de muestrear. En su forma más simple, se puede usar para calcular probabilidades condicionales, es decir, para determinar $P(X|\mathbf{e})$. El algoritmo REJECTION-SAMPLING se muestra en la Figura 13.17. Primero, genera muestras de la distribución previa especificada por la red. Luego, rechaza todos aquellos que no coinciden con la evidencia. Finalmente, la estimación $\hat{P}(X=x|\mathbf{e})$ se obtiene contando cuántas veces $X = x$ ocurre en las muestras restantes. Muestreo de rechazo

Dada $\hat{\mathbf{P}}(X|\mathbf{e})$ sea la distribución estimada que devuelve el algoritmo; esta distribución se calcula normalizando $N_{PS}(X,\mathbf{e})$, el vector de recuentos de muestras para cada valor de X donde la muestra coincide con la evidencia $\mathbf{e}$:

$$\hat{\mathbf{P}}(X|\mathbf{e}) = \alpha \mathbf{N}_{PS}(X,\mathbf{e}) = \frac{\mathbf{N}_{PS}(X,\mathbf{e})}{N_{PS}(\mathbf{e})}.$$

```
function REJECTION-SAMPLING(X,e,bn,N) returns una estimación de P(X|e)
  inputs: X, la variable de consulta
          e, valores observados para variables E
          bn, una red bayesiana
          N, el número total de muestras que se generarán
  local variables: C, un vector de recuentos para cada valor de X, inicialmente cero

  for j = 1 to N do
    x←PRIOR-SAMPLE(bn)
    if x es consistente con e then
      C[j]←C[j]+1 donde x_j es el valor de X en x
  return NORMALIZE(C)
```

Figura 13.17 Algoritmo de muestreo por rechazo para responder consultas, dada evidencia en una red bayesiana.

De la Ecuación (13.7), esto se convierte en

$$\hat{\mathbf{P}}(X|\mathbf{e}) \approx \frac{\mathbf{P}(X,\mathbf{e})}{P(\mathbf{e})} = \mathbf{P}(X|\mathbf{e}).$$

Es decir, el muestreo por rechazo produce una estimación consistente de la verdadera probabilidad.

Continuando con nuestro ejemplo de la Figura 13.15(a), suponga que deseamos estimar $\mathbf{P}(Rain|Sprinkler=true)$, utilizando 100 muestras. De las 100 que generamos, suponga que 73 tienen $Sprinkler=false$ y son rechazadas, mientras que 27 tienen $Sprinkler=true$; de las 27, 8 tienen $Rain=true$ y 19 tienen $Rain=false$. Por lo tanto,

$$\mathbf{P}(Rain|Sprinkler=true) \approx \text{NORMALIZE}(\langle 8,19\rangle) = \langle 0.296,0.704\rangle.$$

La verdadera respuesta es $\langle 0.3,0.7\rangle$. A medida que se recogen más muestras, la estimación convergerá a la verdadera respuesta. La desviación estándar del error en cada probabilidad será proporcional a $1/\sqrt{n}$, donde n es el número de muestras utilizadas en la estimación.

Ahora sabemos que el muestreo por rechazo converge a la respuesta correcta, la siguiente pregunta es, ¿qué tan rápido sucede eso? Más precisamente, ¿cuántas muestras se requieren antes de saber que las estimaciones resultantes están cerca de las respuestas correctas con alta probabilidad? Mientras que la complejidad de los algoritmos exactos depende en gran medida de la topología de la red: los árboles son fáciles, las redes densamente conectadas son difíciles, la complejidad del muestreo por rechazo depende principalmente de la fracción de muestras que se aceptan. Esta fracción es igual a la probabilidad *a priori* de la evidencia, $P(\mathbf{e})$. Desafortunadamente, para problemas complejos con muchas variables de evidencia, esta fracción es infinitesimalmente pequeña. Cuando se aplica a la versión discreta de la red de seguros de automóviles de la Figura 13.9, la fracción de muestras consistentes con un caso de evidencia típico muestreado de la propia red suele estar entre uno en mil y uno en diez mil. La convergencia es en extremo lenta (ver Figura 13.19 líneas arriba).

Esperamos que la fracción de muestras consistentes con la evidencia **e** disminuya de manera exponencial a medida que el número de variables de evidencia aumenta, por lo que el procedimiento es inutilizable para problemas complejos. También tiene dificultades con las variables de evidencia de valor continuo, porque la probabilidad de producir una muestra consistente con dicha evidencia es cero (si es de valor continuo) o infinitesimal (si es un número de punto flotante de precisión finita).

Observe que el muestreo por rechazo es similar a la estimación de probabilidades condicionales en el mundo real. Por ejemplo, para estimar la probabilidad condicional de que cualquier humano sobreviva a que un asteroide de 1 km de diámetro se estrelle contra la Tierra, uno puede contar cuán seguido cualquier humano sobrevive después de que un asteroide de 1 km de diámetro se estrella contra la Tierra, ignorando todos esos días cuando no ocurre tal evento. (Aquí, el universo mismo juega el papel del algoritmo de generación de muestras). Para obtener una estimación decente, es posible que sea necesario esperar a que ocurran 100 eventos de este tipo. Obviamente, esto podría llevar mucho tiempo, y esa es la debilidad del muestreo por rechazo.

Muestreo de importancia

Muestreo de importancia

La técnica estadística general de **muestreo de importancia** tiene como objetivo emular el efecto de muestreo de importancia de una distribución P usando muestras de otra distribución Q. Nos aseguramos de que las respuestas sean correctas en el límite aplicando un factor de corrección $P(\mathbf{x})/Q(\mathbf{x})$, también conocido como **peso**, a cada muestra $\mathbf{x}$ al contar las muestras.

La razón para usar el muestreo de importancia en las redes de Bayes es simple: nos gustaría muestrear de la verdadera distribución posterior condicionada a toda la evidencia, pero en la mayoría de los casos esto es muy difícil;[6] así que en su lugar, muestreamos de una distribución fácil y aplicamos las correcciones necesarias. La razón por la que el muestreo de importancia funciona también es simple. Suponga que las variables no evidencia sean $\mathbf{Z}$. si pudiéramos muestrear directamente de $P(\mathbf{z}|\mathbf{e})$, podríamos construir estimaciones como esta:

$$\hat{P}(\mathbf{z}|\mathbf{e}) = \frac{N_P(\mathbf{z})}{N} \approx P(\mathbf{z}|\mathbf{e})$$

donde $N_P(\mathbf{z})$ es el número de muestras con $\mathbf{Z}=\mathbf{z}$ cuando se muestrea de P. Ahora suponga que en su lugar muestreamos de $Q(\mathbf{z})$. La estimación en este caso incluye los factores de corrección:

$$\hat{P}(\mathbf{z}|\mathbf{e}) = \frac{N_Q(\mathbf{z})}{N}\frac{P(\mathbf{z}|\mathbf{e})}{Q(\mathbf{z})} \approx Q(\mathbf{z})\frac{P(\mathbf{z}|\mathbf{e})}{Q(\mathbf{z})} = P(\mathbf{z}|\mathbf{e}).$$

Por lo tanto, la estimación converge al valor correcto *independientemente de la distribución de muestreo Q que se use*. (El único requisito técnico es que $Q(\mathbf{z})$ no debe ser cero para ningún $\mathbf{z}$ donde $P(\mathbf{z}|\mathbf{e})$ sea distinto de cero). Intuitivamente, el factor de corrección compensa el sobremuestreo o el submuestreo. Por ejemplo, si $Q(\mathbf{z})$ es mucho mayor que $P(\mathbf{z}|\mathbf{e})$ para algún $\mathbf{z}$, entonces habrá muchas más muestras de ese $\mathbf{z}$ de las que debería haber, pero cada una tendrá un peso pequeño, por lo que funciona como si hubiera el número correcto.

Ponderación de verosimilitud

En cuanto a qué Q usar, queremos uno que sea fácil de muestrear y lo más cerca posible a la verdadera posterior $P(\mathbf{z}|\mathbf{e})$. El enfoque más común se llama **ponderación de verosimilitud** (por ponderación de verosimilitud razones que veremos en breve). Como se muestra en la función WEIGHTED-SAMPLE en la Figura 13.18, el algoritmo fija los valores para las variables de evidencia $\mathbf{E}$ y toma muestras de todas las variables no evidencia en orden topológico, cada una condicionada a sus padres. Esto garantiza que cada evento generado es consistente con la evidencia.

Llamemos a la distribución de muestreo producida por este algoritmo Q_{WS}. Si las variables no evidencia son $\mathbf{Z}=\{Z_1,...,Z_l\}$, entonces tenemos

$$Q_{WS}(\mathbf{z}) = \prod_{i=1}^{l} P(z_i \, padres(Z_i)) \tag{13.8}$$

[6] Si fuera fácil, entonces podríamos aproximar la probabilidad deseada a una precisión arbitraria con un número polinomial de muestras. Es posible demostrar que no puede existir tal esquema de aproximación en tiempo polinomial.

function LIKELIHOOD-WEIGHTING(X,**e**,bn,N) **returns** un estimado de **P**(X|**e**)
  inputs: X, la variable de consulta
    e, valores observados para variables **E**
    bn, una red bayesiana que especifica la distribución conjunta **P**(X_1,...,X_n)
    N, el número total de muestras que se generarán
  local variables: **W**, un vector de recuentos ponderados para cada valor de X, inicialmente cero

  for j = 1 **to** N **do**
    x,w←WEIGHTED-SAMPLE(bn, **e**)
    W[j]←**W**[j]+w donde x_j es el valor de X en **x**
  return NORMALIZE(**W**)

function WEIGHTED-SAMPLE(bn,**e**) **returns** un evento y un peso

  w←1; **x**←un evento con n elementos, con valores fijos desde **e**
  for i = 1 **to** n **do**
    if X_i es una variable de evidencia con valor x_{ij} en **e**
      then w←w × $P(X_i = x_{ij} | padres(X_i))$
      else **x**[i]← una muestra aleatoria de **P**($X_i|padres(X_i)$)
  return **x**,w

Figura 13.18 El algoritmo de ponderación de verosimilitud para la inferencia en redes bayesianas. En WEIGHTED-SAMPLE, cada variable no evidencia se muestrea de acuerdo con la distribución condicional dados los valores ya muestreados para los padres de la variable, mientras que se acumula un peso basado en la verosimilitud para cada variable de evidencia.

porque cada variable se muestrea condicionada a sus padres. Para completar el algoritmo, necesitamos saber cómo calcular la ponderación para cada muestra generada a partir de Q_{WS}. Según el esquema general para el muestreo de importancia, la ponderación debería ser

$$w(\mathbf{z}) = P(\mathbf{z}|\mathbf{e})/Q_{WS}(\mathbf{z}) = \alpha P(\mathbf{z},\mathbf{e})/Q_{WS}(\mathbf{z})$$

donde el factor de normalización $\alpha = 1/P(\mathbf{e})$ es el mismo para todas las muestras. Ahora **z** y **e** juntos cubren todas las variables en la red bayesiana, por lo que $P(\mathbf{z},\mathbf{e})$ es el producto de todas las probabilidades condicionales (Ecuación (13.2) página 415); podemos escribir esto como el producto de las probabilidades condicionales para las variables no evidencia multiplicado por el producto de las probabilidades condicionales para las variables de evidencia:

$$\begin{aligned} w(\mathbf{z}) &= \alpha \frac{P(\mathbf{z}, \mathbf{e})}{Q_{WS}(\mathbf{z})} = \alpha \frac{\Pi_{i=1}^{l} P(zi|padres(Z_i)) \; \Pi_{i=1}^{m} P(e_i|padres(E_i))}{\Pi_{i=1}^{l} P(zi|padres(Z_i))} \\ &= \alpha \prod_{i=1}^{m} P(e_i|padres(E_i)) \end{aligned} \tag{13.9}$$

Por lo tanto, la ponderación es el producto de las probabilidades condicionales para las variables de evidencia dados sus padres. (Las probabilidades de la evidencia se llaman generalmente **verosimilitudes**, de ahí el nombre). El cálculo de la ponderación se implementa de forma incremental en WEIGHTED-SAMPLE, multiplicando por la probabilidad condicional cada vez que se encuentra una variable de evidencia. La normalización se hace al final antes de que se devuelva el resultado de la consulta.

Apliquemos el algoritmo a la red mostrada en la Figura 13.15(a), con la consulta **P**(*Rain*|*Cloudy=true*,*WetGrass=true*) y el ordenamiento *Cloudy*, *Sprinkler, Rain, WetGrass*.

(Cualquier ordenamiento topológico servirá). El proceso es el siguiente: Primero, la ponderación w se establece en 1.0. Luego se genera un evento:

1. *Cloudy* es una variable de evidencia con valor *verdadero*. Por lo tanto, establecemos

$$w \leftarrow w \times P(Cloudy=true) = 0.5.$$

2. *Sprinkler* no es una variable de evidencia, por lo que se toma una muestra de $\mathbf{P}(Sprinkler|Cloudy=true) = \langle 0.1, 0.9 \rangle$; supongamos que esto devuelve *false*.
3. *Rain* no es una variable de evidencia, por lo que se toma una muestra de $\mathbf{P}(Rain|Cloudy=true) = \langle 0.8, 0.2 \rangle$; supongamos que esto devuelve *true*.
4. *WetGrass* es una variable de evidencia con valor *true*. Por lo tanto, establecemos

$$\begin{aligned} w &\leftarrow w \times P(WetGrass=true|Sprinkler=false|Rain=true) \\ &= 0.5 \times 0.9 = 0.45. \end{aligned}$$

Aquí WEIGHTED-SAMPLE devuelve el evento [*true*, *false*, *true*, *true*] con peso 0.45, y esto se contabiliza bajo *Lluvia = true*.

Observe que *padres*(Z_i) puede incluir tanto variables no evidencia como variables de evidencia. A diferencia de la distribución previa $P(\mathbf{z})$, la distribución Q_{WS} presta cierta atención a la evidencia: los valores muestreados para cada Z_i serán influidos por la evidencia entre los ancestros de Z_i. Por ejemplo, al muestrear *Sprinkler*, el algoritmo presta atención a la evidencia *Cloudy=true* en su variable padre. Por otro lado, Q_{WS} presta menos atención a la evidencia que la verdadera distribución posterior $P(\mathbf{z}|\mathbf{e})$, porque los valores muestreados para cada Z_i *ignoran* la evidencia entre los no ancestros de Z_i. Por ejemplo, al muestrear *Sprinkler* y *Rain*, el algoritmo ignora la evidencia en la variable hija *WetGrass=true*; esto significa que generará muchas muestras con *Sprinkler= false* y *Rain=false* a pesar de que la evidencia en realidad descarta este caso. Esas muestras tendrán peso cero.

Debido a que la ponderación de verosimilitud utiliza todas las muestras generadas, puede ser mucho más eficiente que el muestreo de rechazo. Sin embargo, sufrirá una degradación en el rendimiento a medida que aumenta el número de variables de evidencia. Esto se debe a que la mayoría de las muestras tendrán pesos muy bajos y, por lo tanto, la estimación ponderada estará dominada por la pequeña fracción de muestras que otorgan más que una probabilidad infinitesimal a la evidencia. El problema se agrava si las variables de evidencia ocurren "corriente abajo", es decir, tarde en el ordenamiento de variables, porque entonces las variables no evidencia no tendrán evidencia en sus padres y antepasados para guiar la generación de muestras. Esto significa que las muestras serán meras alucinaciones, simulaciones que tienen poco parecido con la realidad sugerida por la evidencia.

Cuando se aplica a la versión discreta de la red de seguros de automóviles en la Figura 13.9, el ponderamiento de probabilidad es considerablemente más eficiente que el muestreo de rechazo (ver Figura 13.19). La red de seguros es un caso relativamente benigno para el ponderamiento de probabilidad porque gran parte de la evidencia está "corriente arriba" y las variables de consulta son nodos hoja de la red.

13.4.2 Inferencia por simulación de cadena de Markov

Cadena de Markov Monte Carlo

Los algoritmos de **cadena de Markov Monte Carlo** (MCMC) funcionan de manera diferente al muestreo de rechazo y ponderación de similitud. En lugar de generar cada muestra desde cero, los algoritmos MCMC generan una muestra realizando un cambio aleatorio en la muestra anterior. Piense en un algoritmo MCMC como si estuviera en un *estado actual* particular que especifica un valor para cada variable y genera un *siguiente estado* al realizar cambios aleatorios en el estado actual.

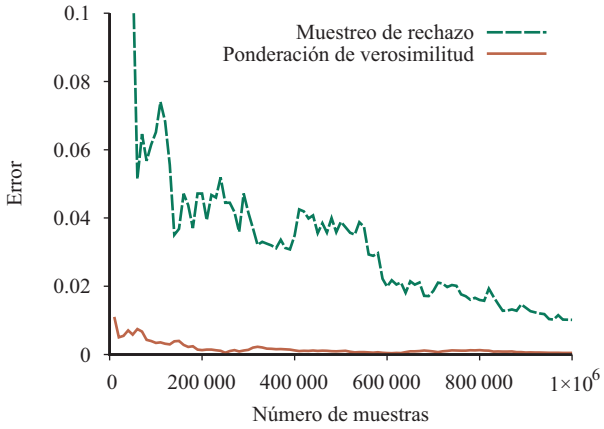


Figura 13.19 Rendimiento del muestreo de rechazo y ponderación de probabilidad en la red de seguros. El eje *x* muestra el número de muestras generadas, mientras que el eje *y* muestra el error absoluto máximo en cualquiera de los valores de probabilidad para una consulta en *CostoPropiedad*.

Cadena de Markov

El término **cadena de Markov** se refiere a un proceso aleatorio que genera una secuencia de estados. (Las cadenas de Markov también figuran de manera prominente en los Capítulos 14 y 17; el algoritmo de enfriamiento simulado en el Capítulo 4 y el algoritmo WalkSat en el Capítulo 7 también son miembros de la familia MCMC.) Comenzamos describiendo una forma particular de MCMC llamada **muestreo de Gibbs**, que es especialmente adecuada para las redes de Bayes. Luego describimos el algoritmo más general de **Metropolis–Hastings**, que da mucha más flexibilidad en la generación de muestras.

Muestreo de Gibbs

Metropolis–Hastings

Muestreo de Gibbs en redes bayesianas

El algoritmo de muestreo de Gibbs para redes bayesianas comienza con un estado arbitrario (con las variables de evidencia fijas en sus valores observados) y genera un siguiente estado muestreando aleatoriamente un valor para una de las variables no evidencia X_i. Recuerde de la página 419 que X_i es independiente de todas las demás variables, dada su manta de Markov (sus padres, hijos y otros padres de los hijos); por lo tanto, el muestreo de Gibbs para X_i significa muestrear *condicionado a los valores actuales de las variables en su manta de Markov*. El algoritmo deambula de manera aleatoria por el espacio de estados, el espacio de posibles asignaciones completas, volteando una variable a la vez, pero manteniendo las variables de evidencia fijas. El algoritmo completo se muestra en la Figura 13.20.

Considere la consulta **P**(*Rain*|*Sprinkler=true,WetGrass=true*) para la red en la Figura 13.15(a). Las variables de evidencia *Aspersor* y *CéspedMojado* están fijas a sus valores observados (ambos *true*), y las variables no evidencia *Nublado* y *Lluvia* se inicializan de manera aleatoria a, digamos, *true* y *false* respectivamente. Así, el estado inicial es [*true*,**true**,*false*,**true**], donde hemos marcado los valores de evidencia fijos en negrita. Ahora las variables no evidencia Z_i se muestrean repetidamente en algún orden aleatorio de acuerdo con una distribución de probabilidad $\rho(i)$ para elegir variables. Por ejemplo:

1. Se elige *Cloudy* y luego se muestrea, dado los valores actuales de su manta de Markov: en este caso, muestreamos de **P**(*Cloudy*|*Sprinkler=true,Rain=false*). Suponga que el resultado es *Cloudy=false*. Entonces el nuevo estado actual es [*false*,**true**,*false*,**true**].

function GIBBS-ASK(X,**e**,bn,N) **returns** un estimado de **P**(X|**e**)
  local variables: **C**, un vector de cuentas para cada valor de X, inicialmente cero
    Z, las variables no evidenciadas en bn
    x, el estado actual de la red, inicializado a partir de **e**

  inicializar **x** con valores aleatorios para las variables en **Z**
  for $k = 1$ **to** N **do**
    choose cualquier variable Z_i de **Z** de acuerdo con cualquier distribución $\rho(i)$
    establezca el valor de Z_i en **x** muestreando de **P**($Z_i|mb(Z_i)$)
    C[j]←**C**[j] +1 donde x_j es el valor de X en **x**
  return NORMALIZE(**C**)

Figura 13.20 Algoritmo de muestreo de Gibbs para inferencia aproximada en redes de Bayes; esta versión elige variables al azar, pero también funciona al recorrer las variables.

2. Se elige *Rain* y luego se muestrea, dado los valores actuales de su manta de Markov: en este caso, muestreamos de **P**(*Rain*|*Cloudy=false*,*Sprinkler=true*,*WetGrass=true*). Suponga que esto produce *Rain=true*. El nuevo estado actual es [*false*,**true**,*true*,**true**].

El único detalle restante se refiere al método de cálculo de la distribución de la manta de Markov **P**($X_i|mb(X_i)$), donde $mb(X_i)$ denota los valores de las variables en la manta de Markov de X_i, $MB(X_i)$. Afortunadamente, esto no implica ninguna inferencia compleja. Como se muestra en el Ejercicio 13.MARB, la distribución está dada por

$$P(X_i|mb(X_i)) = \alpha P(x_i|padres(X_i)) \prod_{Y_j \in Children(X_i)} P(y_j|\text{padres}(Y_j)). \qquad (13.10)$$

En otras palabras, para cada valor x_i, la probabilidad se da multiplicando las probabilidades de las CPTs de X_i y sus hijos. Por ejemplo, en el primer paso de muestreo mostrado arriba, muestreamos de **P**(*Cloudy*|*Sprinkler=true,Rain=false*). Por la Ecuación (13.10), y abreviando los nombres de las variables, tenemos

$$\begin{aligned} P(c|s,\neg r) &= \alpha P(c)P(s|c)P(\neg r|c) = \alpha 0.5\ 0.1\ 0.2 \\ P(\neg c|s,\neg r) &= \alpha P(\neg c)P(s|\neg c)P(\neg r|\neg c) = \alpha 0.5\ \cdot 0.5\ \cdot 0.8, \end{aligned}$$

por lo que la distribución de muestreo es $\alpha\langle 0.001,0.020\rangle \approx \langle 0.048,0.952\rangle$.

La Figura 13.21(a) muestra la cadena de Markov completa para el caso en que las variables se eligen de manera uniforme, es decir, $\rho(Cloudy) = \rho(Rain) = 0.5$. El algoritmo simplemente está deambulando en este grafo, siguiendo enlaces con las probabilidades indicadas. Cada estado visitado durante este proceso es una muestra que contribuye a la estimación para la variable de consulta *Rain*. Si el proceso visita 20 estados donde *Rain* es verdadero y 60 estados donde *Rain* es falso, entonces la respuesta a la consulta es NORMALIZE($\langle 20,60\rangle$) = $\langle 0.25,0.75\rangle$.

Análisis de las cadenas de Markov

Hemos dicho que el muestreo de Gibbs funciona deambulando de manera aleatoria por el espacio de estados para generar muestras. Para explicar por qué el muestreo de Gibbs funciona *correctamente;* es decir, por qué sus estimaciones convergen a valores correctos en el límite, necesitaremos un análisis cuidadoso. (Esta sección es algo matemática y se puede omitir en la primera lectura).

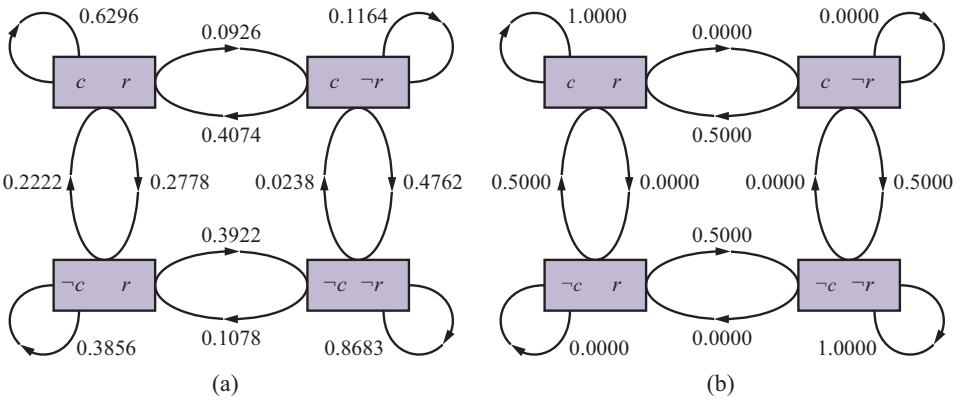


Figura 13.21 (a) Los estados y las probabilidades de transición de la cadena de Markov para la consulta *P*(*Rain*|*Sprinkler*=*true*,*WetGrass*=*true*). Note los autobucles: el estado permanece igual cuando se elige cualquier variable y luego vuelve a muestrear el mismo valor que ya tiene. (b) Las probabilidades de transición cuando el CPT para *Rain* lo restringe a tener el mismo valor que *Cloudy*.

Núcleo de transición

Comenzamos con algunos de los conceptos básicos para analizar las cadenas de Markov en general. Cualquier cadena de este tipo se define por su estado inicial y su **núcleo de transición** $k(\mathbf{x} \rightarrow \mathbf{x}')$, la probabilidad de una transición al estado $\mathbf{x}'$ partiendo del estado $\mathbf{x}$. Ahora, suponga que ejecutamos la cadena de Markov durante t pasos, y dejemos que $\pi_t(\mathbf{x})$ sea la probabilidad de que el sistema esté en el estado $\mathbf{x}$ en el tiempo t. De manera similar, dejemos que $\pi_{t+1}(\mathbf{x}')$ sea la probabilidad de estar en el estado $\mathbf{x}'$ en el tiempo $t + 1$. Dado $\pi_t(\mathbf{x})$, podemos calcular $\pi_{t+1}(\mathbf{x}')$ sumando, para todos los estados $\mathbf{x}$ en los que el sistema podría estar en el tiempo t, la probabilidad de estar en $\mathbf{x}$ veces la probabilidad de hacer la transición a $\mathbf{x}'$:

$$\pi_{t+1}(\mathbf{x}') = \sum_{\mathbf{x}} \pi_t(\mathbf{x}) k(\mathbf{x} \rightarrow \mathbf{x}').$$

Distribución estacionaria

Decimos que la cadena ha alcanzado su **distribución estacionaria** si $\pi_t = \pi_{t+1}$. Llamemos a esta distribución estacionaria π; su ecuación definitoria es, por lo tanto,

$$\pi(\mathbf{x}') = \sum_{\mathbf{x}} \pi(\mathbf{x}) k(\mathbf{x} \rightarrow \mathbf{x}') \qquad \text{para todo } \mathbf{x}'. \tag{13.11}$$

Ergódico

Siempre que el núcleo de transición k sea **ergódico**, es decir, cada estado es alcanzable desde cualquier otro y no hay ciclos estrictamente periódicos, hay una distribución π que satisface esta ecuación para cualquier k dado.

La ecuación (13.11) puede leerse como que el "flujo"" esperado desde cada estado (es decir, su "población" actual) es igual al "flujo" esperado desde todos los estados. Una forma obvia de satisfacer esta relación es si el flujo esperado entre cualquier par de estados es el mismo en ambas direcciones; es decir,

$$\pi(\mathbf{x}) k(\mathbf{x} \rightarrow \mathbf{x}') = \pi(\mathbf{x}') k(\mathbf{x}' \rightarrow \mathbf{x}) \qquad \text{para todo } \mathbf{x}, \mathbf{x}'. \tag{13.12}$$

Equilibrio detallado

Cuando estas ecuaciones se mantienen, decimos que $k(\mathbf{x} \rightarrow \mathbf{x}')$ está en **equilibrio detallado** con $\pi(\mathbf{x})$. Un caso especial es el autobucle $\mathbf{x} = \mathbf{x}'$; es decir, una transición de un estado a sí mismo. En ese caso, la condición de equilibrio detallado se convierte en $\pi(\mathbf{x})k(\mathbf{x} \rightarrow \mathbf{x}) = \pi(\mathbf{x})k(\mathbf{x} \rightarrow \mathbf{x})$ que por supuesto es trivialmente cierto para cualquier distribución estacionaria π y cualquier núcleo de transición k.

Podemos demostrar que el equilibrio detallado tenga como consecuencia lógica estacionariedad sólo sumando sobre $\mathbf{x}$ en la Ecuación (13.12). Tenemos

$$\sum_{\mathbf{x}} \pi(\mathbf{x})k(\mathbf{x} \rightarrow \mathbf{x}') = \sum_{\mathbf{x}} \pi(\mathbf{x}')k(\mathbf{x}' \rightarrow \mathbf{x}) = \pi(\mathbf{x}') \sum_{\mathbf{x}} k(\mathbf{x}' \rightarrow \mathbf{x}) = \pi(\mathbf{x}')$$

donde el último paso sigue porque se garantiza que ocurra una transición desde $\mathbf{x}'$.

Por qué funciona el muestreo de Gibbs

Ahora mostraremos que el muestreo de Gibbs devuelve estimaciones consistentes para las probabilidades posteriores. La afirmación básica es sencilla: *la distribución estacionaria del proceso de muestreo de Gibbs es la distribución posterior para las variables no evidencia condicionadas a la evidencia*. Esta propiedad notable se deriva de la forma específica en que el proceso de muestreo de Gibbs se mueve de estado a estado.

La definición general del muestreo de Gibbs es que se elige una variable X_i y luego se muestrea condicionalmente en función de los valores actuales de *todas* las demás variables. (Cuando se aplica específicamente a las redes de Bayes, usamos el hecho adicional de que muestrear condicionalmente en todas las variables es equivalente a muestrear condicionalmente en la manta de Markov de la variable, como se muestra en la página 419). Usaremos la notación $\overline{\mathbf{X}}_i$ para referirnos a estas otras variables (excepto las variables de evidencia); sus valores en el estado actual son $\overline{\mathbf{x}}_i$.

Para escribir el núcleo de transición $k(\mathbf{x} \rightarrow \mathbf{x}')$ para el muestreo de Gibbs, hay tres casos a considerar:

1. Los estados $\mathbf{x}$ y $\mathbf{x}'$ difieren en dos o más variables. En ese caso, $k(\mathbf{x} \rightarrow \mathbf{x}') = 0$ porque el muestreo de Gibbs sólo cambia una sola variable.
2. Los estados difieren en una variable X_i que cambia su valor de x_i a x'_i. La probabilidad de tal ocurrencia es

$$k(\mathbf{x} \rightarrow \mathbf{x}') = k((x_i, \overline{\mathbf{x}}_i) \rightarrow (x'_i, \overline{\mathbf{x}}_i)) = \rho(i)P(x'_i \,|\overline{\mathbf{x}}_i). \tag{13.13}$$

3. Los estados son los mismos: $\mathbf{x} = \mathbf{x}'$. En ese caso, *cualquier* variable podría ser elegida, pero luego el proceso de muestreo produce el mismo valor que la variable ya tiene. La probabilidad de tal ocurrencia es

$$k(\mathbf{x} \rightarrow \mathbf{x}) = \sum_i \rho(i)k((x_i, \overline{\mathbf{x}}_i) \rightarrow (x_i, \overline{\mathbf{x}}_i)) = \sum_i \rho(i)P(x_i|\overline{\mathbf{x}}_i).$$

Ahora mostramos que esta definición general de muestreo de Gibbs satisface la ecuación de equilibrio detallado con una distribución estacionaria igual a $P(\mathbf{x}|\mathbf{e})$, la verdadera distribución posterior en las variables no evidencia. Es decir, mostramos que $\pi(\mathbf{x})k(\mathbf{x} \rightarrow \mathbf{x}') = \pi(\mathbf{x}')k(\mathbf{x}' \rightarrow \mathbf{x})$ donde $\pi(\mathbf{x}) = P(\mathbf{x}|\mathbf{e})$, para todos los estados $\mathbf{x}$ y $\mathbf{x}'$.

Para los primeros y terceros casos dados con anterioridad, siempre se cumple el equilibrio detallado: si dos estados difieren en dos o más variables, la probabilidad de transición en ambas direcciones es cero. Si $\mathbf{x} \neq \mathbf{x}'$ entonces de la Ecuación (13.13), tenemos

$$\begin{aligned}
\pi(\mathbf{x})k(\mathbf{x} \rightarrow \mathbf{x}') &= P(\mathbf{x}|\mathbf{e})\rho(i)P(x'_i \,|\overline{\mathbf{x}}_i,\mathbf{e}) = \rho(i)P(x_i,\overline{\mathbf{x}}_i|\mathbf{e})P(x'_i \,|\overline{\mathbf{x}}_i,\mathbf{e}) \\
&= \rho(i)P(x_i|\overline{\mathbf{x}}_i,\mathbf{e})P(\overline{\mathbf{x}}_i|\mathbf{e})P(x'_i|\,\overline{\mathbf{x}}_i,\mathbf{e}) \qquad \text{(usando la regla de la cadena en el primer término)} \\
&= \rho(i)P(x_i|\overline{\mathbf{x}}_i,\mathbf{e})P(x'_i,\overline{\mathbf{x}}_i\,|\mathbf{e}) \qquad \text{(regla de la cadena inversa en los dos últimos términos)} \\
&= \pi(\mathbf{x}')k(\mathbf{x}' \rightarrow \mathbf{x}).
\end{aligned}$$

La última pieza del rompecabezas es la ergodicidad de la cadena; es decir, cada estado debe ser alcanzable desde cualquier otro y no hay ciclos periódicos. Ambas condiciones se cumplen

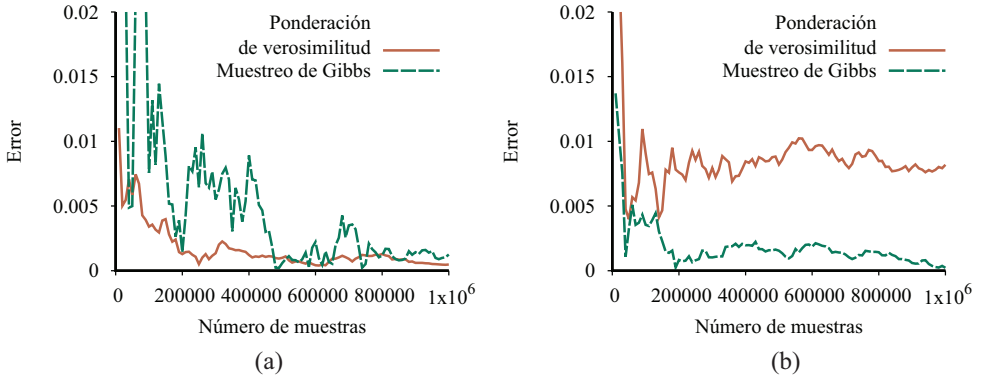


Figura 13.22 Rendimiento del muestreo de Gibbs en comparación con la ponderación de verosimilitud en la red de seguros de automóviles: (a) para la consulta estándar sobre *ValorPropiedad*, y (b) para el caso en que se observan las variables de salida y *Edad* es la variable de consulta.

siempre que las CPTs no contienen probabilidades de 0 o 1. La alcanzabilidad proviene del hecho de que podemos convertir un estado en otro cambiando una variable a la vez, mientras que la ausencia de ciclos periódicos proviene del hecho de que cada estado tiene un bucle propio con probabilidad distinta de cero. Por lo tanto, bajo las condiciones establecidas, k es ergódico, lo que significa que las muestras generadas por el muestreo de Gibbs eventualmente se extraerán de la verdadera distribución posterior.

Complejidad del muestreo de Gibbs

Primero, las buenas noticias: cada paso de muestreo de Gibbs implica calcular la distribución de la manta de Markov para la variable elegida X_i, lo que requiere un número de multiplicaciones proporcional al número de hijos de X_i y al tamaño del rango de X_i. Esto es importante porque significa que *el trabajo requerido para generar cada muestra es independiente del tamaño de la red*.

Ahora, las no necesariamente malas noticias: la complejidad del muestreo de Gibbs es mucho más difícil de analizar que la del muestreo de rechazo y la ponderación de probabilidad. Lo primero que hay que notar es que el muestreo de Gibbs, a diferencia de la ponderación de probabilidad, *sí* presta atención a la evidencia descendente. La información se propaga desde los nodos de evidencia en todas las direcciones: primero, cualquier vecino de los nodos de evidencia muestra valores que reflejan la evidencia en esos nodos; luego *sus* vecinos, y así sucesivamente. Por lo tanto, esperamos que el muestreo de Gibbs supere la ponderación de verosimilitud cuando la evidencia es principalmente descendente; y de hecho, esto se confirma en la Figura 13.22.

Tasa de mezcla

La tasa de convergencia para el muestreo de Gibbs, **tasa de mezcla** de la cadena de Markov definida por el algoritmo, depende fuertemente de las propiedades cuantitativas de las distribuciones condicionales en la red. Para ver esto, considere lo que sucede en la Figura 13.15(a) a medida que la CPT para *Rain* se vuelve determinista: llueve *si y sólo si* está nublado. En ese caso, la verdadera distribución posterior para la consulta $\mathbf{P}(Rain|sprinkler,wetGras)$ es aproximadamente $\langle 0.18,0.82\rangle$ pero el muestreo de Gibbs nunca alcanzará este valor. El problema es que los únicos dos estados conjuntos para *Cloudy* y *Rain* que tienen probabilidad distinta de cero son [*true*,*true*] y [*false*,*false*]. Comenzando en [*true*,*true*], la cadena nunca puede llegar a [*false*,*false*] porque las transiciones a los estados intermedios tienen probabilidad cero (ver Figura 13.21(b)). Por lo tanto, si el proceso

comienza en [true,true] siempre informa una probabilidad posterior para la consulta de ⟨1.0,0.0⟩; si comienza en [*false*,*false*] siempre informa una probabilidad posterior para la consulta de ⟨0.0,1.0⟩.

El muestreo de Gibbs falla porque la relación determinista entre *Cloudy* y *Rain* rompe la propiedad de ergodicidad que se requiere para la convergencia. Pero, si hacemos que la relación sea *casi* determinista, entonces se restaura la convergencia, aunque ocurre arbitrariamente lento. Hay varias soluciones que ayudan a los algoritmos MCMC a mezclarse más rápido. Una es el **muestreo de bloques**: podríamos muestrear múltiples variables simultaneas. En este caso, con Muestreo de bloques muestrear *Cloudy* y *Rain* conjuntamente, condicionados a su manta de Markov combinada. Otra es generar estados siguientes de manera más inteligente, como en la próxima sección.

Muestreo de bloques

Muestreo de Metropolis–Hastings

El método de muestreo de Metropolis–Hastings o MH es quizás el algoritmo MCMC más ampliamente aplicable. Al igual que el muestreo de Gibbs, MH está diseñado para generar muestras $\mathbf{x}$ (eventualmente) de acuerdo con las probabilidades objetivo $\pi(\mathbf{x})$; en el caso de la inferencia en redes bayesianas, queremos $\pi(\mathbf{x}) = P(\mathbf{x}|\mathbf{e})$. Al igual que el recocido simulado (página 115), MH tiene dos etapas en cada iteración del proceso de muestreo:

1. Muestrear un nuevo estado $\mathbf{x}'$ a partir de una **distribución de propuesta** $q(\mathbf{x}'|\mathbf{x})$, dado el estado actual $\mathbf{x}$. Distribución de propuesta
2. Aceptar o rechazar probabilísticamente $\mathbf{x}'$ de acuerdo con la **probabilidad de aceptación** Probabilidad de aceptación
$$a(\mathbf{x}'|\mathbf{x}) = \min\left(1, \frac{\pi(\mathbf{x}')q(\mathbf{x}|\mathbf{x}')}{\pi(\mathbf{x})q(\mathbf{x}'|\mathbf{x})}\right).$$
Si se rechaza la propuesta, el estado permanece en $\mathbf{x}$.

El núcleo de transición para MH consiste en este proceso de dos pasos. Vea que si se rechaza la propuesta, la cadena permanece en el mismo estado.

La distribución de propuesta es responsable, como su nombre sugiere, de proponer un siguiente estado $\mathbf{x}'$. Por ejemplo, $q(\mathbf{x}'|\mathbf{x})$ podría definirse de la siguiente manera:

- Con probabilidad 0.95, realiza un paso de muestreo de Gibbs para generar $\mathbf{x}'$.
- Al contrario, genera $\mathbf{x}'$ ejecutando el algoritmo WEIGHTED-SAMPLE de la página 440.

Esta distribución de propuesta hace que MH haga unos 20 pasos de muestreo de Gibbs y luego "reinicia" el proceso desde un nuevo estado (asumiendo que es aceptado) que se genera desde cero. Con esta estratagema supera el problema de que el muestreo de Gibbs se quede atascado en una parte del espacio de estados y no pueda alcanzar las otras partes.

Podría preguntarse cómo sabemos en realidad que MH con una propuesta tan extraña realmente converge a la respuesta correcta. Lo notable de MH es que la *convergencia a la distribución estacionaria correcta está garantizada para cualquier distribución de propuesta*, siempre que el núcleo de transición resultante sea ergódico.

Esta propiedad se deriva de la forma en que se define la probabilidad de aceptación. Al igual que con el muestreo de Gibbs, el autobucle con $\mathbf{x} = \mathbf{x'}$ satisface automáticamente el equilibrio detallado, por lo que nos centramos en el caso en que $\mathbf{x} \neq \mathbf{x'}$. Esto sólo puede ocurrir si se acepta la propuesta. La probabilidad de que ocurra tal transición es

$$k(\mathbf{x} \rightarrow \mathbf{x'}) = q(\mathbf{x'}|\mathbf{x})a(\mathbf{x'}|\mathbf{x}).$$

Al igual que con el muestreo de Gibbs, demostrar el equilibrio detallado significa mostrar que el flujo de $\mathbf{x}$ a $\mathbf{x'}$, $\pi(\mathbf{x})k(\mathbf{x} \rightarrow \mathbf{x'})$, coincide con el flujo de $\mathbf{x'}$ a $\mathbf{x}$, $\pi(\mathbf{x'})k(\mathbf{x'} \rightarrow \mathbf{x})$. Después de conectar

en la expresión anterior para $k(\mathbf{x}\rightarrow\mathbf{x'})$, la prueba es muy sencilla:

$$\begin{aligned}
\pi(\mathbf{x})q(\mathbf{x'}|\mathbf{x})a(\mathbf{x'}|\mathbf{x}) &= \pi(\mathbf{x})q(\mathbf{x'}|\mathbf{x}) \min\left(1, \frac{\pi(\mathbf{x}')q(\mathbf{x}|\mathbf{x}')}{\pi(\mathbf{x})q(\mathbf{x}'|\mathbf{x})}\right) && \text{(definición de } a(\cdot|\cdot)\text{)} \\
&= \min\left(\pi(\mathbf{x})q(\mathbf{x'}|\mathbf{x}), \pi(\mathbf{x'})q(\mathbf{x}|\mathbf{x'})\right) && \text{(multiplicando en)} \\
&= \pi(\mathbf{x'})q(\mathbf{x}|\mathbf{x'}) \min\left(\frac{\pi(\mathbf{x})q(\mathbf{x}'|\mathbf{x})}{\pi(\mathbf{x}')q(\mathbf{x}|\mathbf{x}')}, 1\right) && \text{(dividiendo)} \\
&= \pi(\mathbf{x'})q(\mathbf{x}|\mathbf{x'})a(\mathbf{x}|\mathbf{x'}).
\end{aligned}$$

Dejando de lado las propiedades matemáticas, la parte importante de MH en la que debemos centrarnos es la relación $\pi(\mathbf{x'})/\pi(\mathbf{x})$ en la probabilidad de aceptación. Esto indica que si se propone un próximo estado, *más* probable que el estado actual, definitivamente será aceptado. (Estamos pasando por alto, por ahora, el término $q(\mathbf{x}|\mathbf{x'})/q(\mathbf{x'}|\mathbf{x})$, que está ahí para garantizar el equilibrio detallado y es, en muchos espacios de estado, igual a 1 debido a la simetría). Si el estado propuesto es menos probable que el estado actual, su probabilidad de ser aceptado disminuye de manera proporcional.

Por lo tanto, una guía para diseñar distribuciones de propuestas es asegurarse de que los nuevos estados propuestos sean razonablemente probables. El muestreo de Gibbs lo hace de manera automática: propone desde la distribución de Gibbs $P(X_i|\mathbf{x}_{i}$, lo que significa que la probabilidad de generar cualquier nuevo valor para X_i es directamente proporcional a su probabilidad. (El ejercicio 13.GIBM le pide que demuestre que Gibbs es un caso especial de MH con probabilidad de aceptación de 1).

Otra guía es asegurarse de que la cadena se mezcle bien, lo que significa proponer a veces grandes movimientos a partes distantes del espacio de estado. En el ejemplo anterior, el uso ocasional de Muestreo Ponderado para reiniciar la cadena en un nuevo estado sirve para este propósito.

Además de la casi total libertad para diseñar distribuciones de propuestas, MH tiene dos propiedades adicionales que lo hacen práctico. Primero, la probabilidad posterior $\pi(\mathbf{x}) = P(\mathbf{x}|\mathbf{e})$ aparece en el cálculo de aceptación sólo en forma de una razón $\pi(\mathbf{x}')/\pi(\mathbf{x})$, lo cual es muy afortunado. Calcular $P(\mathbf{x}|\mathbf{e})$ directamente es el cálculo que intentamos aproximar usando MH, ¡así que no tendría sentido hacerlo para cada muestra! En su lugar, usamos el siguiente truco:

$$\frac{\pi(\mathbf{x}')}{\pi(\mathbf{x})} = \frac{P(\mathbf{x}'|\mathbf{e})}{P(\mathbf{x}|\mathbf{e})} = \frac{P(\mathbf{x}',\mathbf{e})}{P(\mathbf{e})} = \frac{P(\mathbf{e})}{P(\mathbf{x},\mathbf{e})} = \frac{P(\mathbf{x}',\mathbf{e})}{P(\mathbf{x},\mathbf{e})}.$$

Los términos en esta relación son probabilidades conjuntas completas, es decir, productos de probabilidades condicionales en la red de Bayes. La segunda propiedad útil de esta relación es que siempre que la distribución de propuestas haga cambios locales en $\mathbf{x}$ para producir $\mathbf{x'}$, sólo un pequeño número de términos en el producto de probabilidades condicionales será diferente. Todas las probabilidades condicionales que involucran variables cuyos valores no han cambiado se cancelarán en la relación. Por lo tanto, al igual que con el muestreo de Gibbs, el trabajo requerido para generar cada muestra es independiente del tamaño de la red siempre que los cambios de estado sean locales.

13.4.3 Compilación de inferencia aproximada

Los algoritmos de muestreo en las Figuras 13.17, 13.18 y 13.20 comparten una propiedad común: operan en una red de Bayes representada como una estructura de datos. Esto parece muy natural: después de todo, una red de Bayes es un grafo acíclico dirigido, ¿cómo podría representarse de otra manera? El problema con este enfoque es que las operaciones necesarias para acceder a la estructura de datos, como encontrar los padres de un nodo, se repiten miles o millones de veces a medida que se ejecuta el algoritmo de muestreo, y *todos estos cálculos son por completo innecesarios*.

La estructura de la red y las probabilidades condicionales permanecen fijas durante el cálculo, por lo que existe la oportunidad de *compilar* la red en código de inferencia específico del modelo que lleva a cabo sólo los cálculos de inferencia necesarios para esa red específica. (En caso de que esto suene familiar, es la misma idea usada en la compilación de programas lógicos en el Capítulo 9). Por ejemplo, supongamos que queremos muestrear la variable *Terremoto* en la red de robo de la Figura 13.2. Según el algoritmo GIBBS-ASK en la Figura 13.20, necesitamos realizar el siguiente cálculo:

establecer el valor de *Terremoto* en **x** muestreando desde $P(Terremoto|mb(Terremoto))$

donde la última distribución se calcula de acuerdo con la Ecuación (13.10), repetida aquí:

$$P(x_i|mb(X_i)) = \alpha P(x_i|padres(X_i)) \prod_{Y_j \in Children(X_i)} P(y_i|padres(y_i)).$$

A su vez, este cálculo requiere buscar los padres e hijos de *Terremoto* en la estructura de la red de Bayes; buscar sus valores actuales; usar esos valores para indexar en las CPT correspondientes (que también deben encontrarse en la red de Bayes); y multiplicar todas las filas apropiadas de esas CPT para formar una nueva distribución de la cual muestrear. Finalmente, como se señaló en la página 436, el paso de muestreo en sí tiene que construir la versión acumulativa de la distribución discreta y luego encontrar el valor en ella, que corresponde a un número aleatorio muestreado de [0,1].

Si, en cambio, compilamos la red, obtenemos código de muestreo específico del modelo para el variable *Earthquake* que se ve así:

```
r ← una prueba uniforme aleatoria de [0,1]
if Alarm = true
    then if Burglary = true
        then return [r < 0.0020212]
        else return [r < 0.36755]
    else if Burglary = true
        then return [r < 0.0016672]
        else return [r < 0.0014222]
```

Aquí, las variables de la red bayesiana *Alarm*, *Burglary*, etc., se convierten en variables de programa ordinarias con valores que comprenden el estado actual de la cadena de Markov. Las expresiones de umbral numérico se evalúan como *verdaderas* o *falsas* y representan las distribuciones de Gibbs precalculadas para cada combinación de valores en la manta de Markov de *Terremoto*. El código no es especialmente bonito, normalmente, será aproximadamente tan grande como la propia red bayesiana, pero es increíblemente eficiente. En comparación con GIBBS-ASK, el código compilado será típicamente 2-3 órdenes de magnitud más rápido. Puede realizar decenas de millones de pasos de muestreo por segundo en una laptop ordinaria, y su velocidad está limitada en gran medida por el costo de generar números aleatorios.

13.5 Redes causales

Hemos discutido varias ventajas de mantener el orden de los nodos en las redes bayesianas compatibles con la dirección de la causalidad. En particular, notamos la facilidad con la que es posible evaluar las probabilidades condicionales si se mantiene dicho orden, así como la compacidad de la estructura de la red resultante. Sin embargo, notamos que, en principio, cualquier orden de

nodos permite una construcción consistente de la red para representar la función de distribución conjunta. Esto se demostró en la Figura 13.3, donde el cambio en el orden de los nodos produjo redes más frondosas y mucho menos naturales que la red original en la Figura 13.2; no obstante, nos permitió representar la misma distribución en todas las variables.

Redes causales

Esta sección describe las **redes causales**, una clase restringida de redes bayesianas que prohíbe todos los ordenamientos que no sean compatibles con la causalidad. Exploraremos cómo construir tales redes, qué se gana con tal construcción, y cómo aprovechar esta ganancia en tareas de toma de decisiones. Considere la red bayesiana más simple imaginable, una sola flecha, *Fuego* → *Humo*. Nos dice que las variables *Fuego* y *Humo* pueden ser dependientes, por lo que necesitamos especificar el prior $P(Fuego)$ y la probabilidad condicional $P(Humo|Fuego)$ para especificar la distribución conjunta $P(\text{Fuego},Humo)$. Sin embargo, esta distribución permite ser representada igualmente bien por la flecha inversa *Fuego* ← *Humo*, utilizando el apropiado $P(Humo)$ y $P(Fuego|Humo)$ calculado a partir de la regla de Bayes. La idea de que estas dos redes son equivalentes, pues transmiten la misma información, provoca incomodidad e incluso resistencia en la mayoría de las personas. ¿Cómo podrían transmitir la misma información cuando sabemos que el *Fuego* causa *Humo*, no al revés?

En otras palabras, sabemos por nuestra experiencia y comprensión científica que eliminar el humo no detendría el fuego y extinguir el fuego detendría el humo. Por lo tanto, esperamos representar esta asimetría mediante la direccionalidad de la flecha entre ellos. Pero si la inversión de la flecha sólo hace las cosas equivalentes, cómo podemos esperar representar esta información importante de manera formal.

Las redes bayesianas causales, a veces llamadas Diagramas Causales, fueron ideadas para permitirnos representar asimetrías causales y aprovechar las asimetrías para razonar con información causal. La idea es decidir sobre la direccionalidad de la flecha por consideraciones que van más allá de la dependencia probabilística e invocan un tipo totalmente diferente de juicio. En lugar de preguntar a un experto si *Humo* y *Fuego* son probabilísticamente dependientes, como hacemos en las redes bayesianas ordinarias, ahora preguntamos ¿qué responde a qué, *Humo* a *Fuego* o *Fuego* a *Humo*?

Esto podría sonar un poco místico, pero puede ser hecho preciso por la noción de "asignación", similar al operador de asignación en los lenguajes de programación. Si la naturaleza asigna un valor a *Humo* con base en lo que la naturaleza aprende sobre *Fuego*, dibujamos una flecha de *Humo* a *Fuego*. Más importante aún, si juzgamos que la naturaleza asigna a *Fuego* un valor de verdad que depende de otras variables, no *Humo*, nos abstenemos de dibujar la flecha *Fuego* ← *Humo*. En otras palabras, el valor x_i de cada variable X_i está determinado por una ecuación $x_i = f_i(OtrasVariables)$, y se dibuja una flecha $X_j \rightarrow \mathrm{X_i}$ si y sólo si X_j es uno de los argumentos de f_i.

Ecuación estructural

La ecuación $x_i = f_i(\cdot)$ se llama **ecuación estructural**, porque describe un mecanismo estable en la naturaleza que, a diferencia de las probabilidades que cuantifican una red bayesiana, permanece invariante a las mediciones y cambios locales en el entorno.

Para apreciar esta estabilidad a los cambios locales, considere la Figura 13.23(a), que muestra una versión ligeramente modificada de la historia del aspersor de césped de la Figura 13.15. Para representar un aspersor desactivado, por ejemplo, eliminamos de la red todos los enlaces incidentes al nodo *Aspersor*. Para representar un césped cubierto por una tienda, eliminamos la flecha *Lluvia* → *CéspedMojado*. Cualquier reconfiguración local de los mecanismos en el entorno puede ser traducida, con sólo una modificación menor, en una reconfiguración isomorfa de la topología de la red. Se requeriría una transformación mucho más elaborada si la red hubiera sido construida en contra del orden causal. Esta estabilidad local es en particular importante para representar acciones o intervenciones, nuestro próximo tema de discusión.

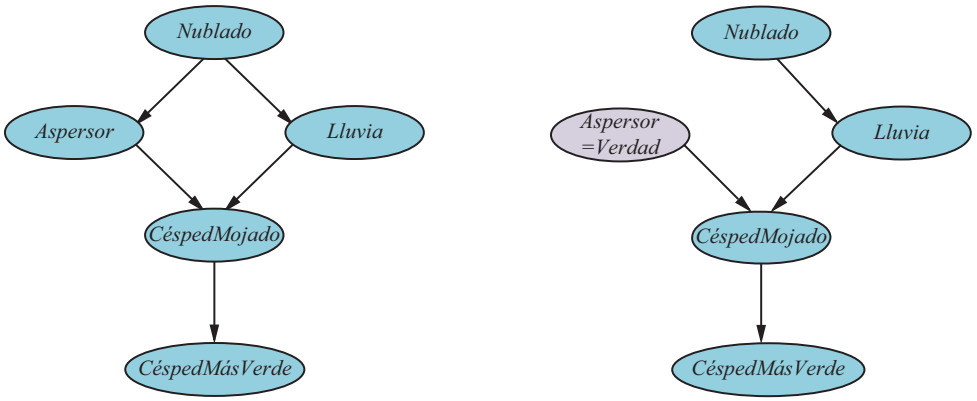


Figura 13.23 (a) Red bayesiana causal que representa relaciones de causa-efecto entre cinco variables. (b) La red después de realizar la acción "encender el aspersor".

13.5.1 Representando acciones: El operador-*do*

Considere de nuevo el ejemplo del *Aspersor* de la Figura 13.23(a). Según la semántica estándar de las redes bayesianas, la distribución conjunta de las cinco variables se da por un producto de cinco distribuciones condicionales:

$$P(c,r,s,w,g) = P(c)\,P(r|c)\,P(s|c)\,P(w|r,s)\,P(g|w) \tag{13.14}$$

donde hemos abreviado cada nombre de variable por su primera letra. Como un sistema de ecuaciones estructurales, el modelo se ve así:

$$\begin{aligned} C &= f_C(U_C) \\ R &= f_R(C,U_R) \\ S &= f_S(C,U_S) \\ W &= f_W(R,S,U_W) \\ G &= f_G(W,U_G) \end{aligned} \tag{13.15}$$

donde, sin pérdida de generalidad, f_C es la función identidad. Las variables U en estas ecuaciones representan **variables no modeladas**, también llamadas **términos de error** o **perturbaciones**, que perturban la relación funcional entre cada variable y sus padres. Por ejemplo, U_W representa otra fuente potencial de humedad, además de *Aspersor* y *Lluvia*, quizá *RocioMañana* o *HelicópteroAntiIncendios*.

Variables no modeladas

Si todas las variables U son variables aleatorias mutuamente independientes con priors (valores iniciales) adecuadamente elegidos, la distribución conjunta en la Ecuación (13.14) puede ser representada por las ecuaciones estructurales en la Ecuación (13.16). Así, un sistema de relaciones estocásticas permite ser capturado por un sistema de relaciones determinísticas, cada una de las cuales es afectada por una perturbación exógena. Sin embargo, el sistema de ecuaciones estructurales da más que eso: nos permite predecir cómo las *intervenciones* afectarán el funcionamiento del sistema y, por lo tanto, las consecuencias observables de esas intervenciones. Esto no es posible dada sólo la distribución conjunta.

Do-cálculo

Por ejemplo, supongamos que *encendemos el aspersor*, es decir, si nosotros (que somos, por definición, no parte de los procesos causales descritos por el modelo) *intervenimos* para imponer la condición *Aspersor* = *true*. En la notación del **do-cálculo**, que es una parte clave de la teoría de las redes causales, esto se escribe como *do*(*Aspersor* = *true*). Una vez hecho esto, significa que la variable aspersor ya no depende de si es un día nublado. Por lo tanto, eliminamos la ecuación $S = f_S(C,U_S)$ del sistema de ecuaciones estructurales y la reemplazamos con $S = true$, dándonos

$$\begin{aligned} C &= f_C(U_C) \\ R &= f_R(C,U_R) \\ S &= true \\ W &= f_W(R,S,U_W) \\ G &= f_G(W,U_G). \end{aligned} \tag{13.16}$$

A partir de estas ecuaciones, obtenemos la nueva distribución conjunta para las variables restantes condicionadas a *do*(*Aspersor*=*true*):

$$P(c,r,w,g|do(S = true) = P(c)\ P(r|c)\ P(w|r,s = true)\ P(g|w) \tag{13.17}$$

Esto corresponde a la red "simplificada" en la Figura 13.23(b). De la Ecuación (13.17), vemos que las únicas variables cuyas probabilidades cambian son *CéspedMojado* y *CéspedMásVerde*, es decir, los descendientes de la variable manipulada *Aspersor*.

Note la diferencia entre condicionar en la acción *do*(*Aspersor*=*true*) en la red original y condicionar en la *observación Aspersor* = verdadero. La red original nos dice que el aspersor es menos probable que esté encendido cuando el clima está nublado, así que si observamos el aspersor encendido, eso reduce la probabilidad de que el clima esté nublado. Pero el sentido común nos dice que si nosotros (operando desde fuera del mundo, por así decirlo) entramos y encendemos el aspersor, eso no afecta el clima ni proporciona nueva información sobre cómo es el clima ese día. Como se muestra en la Figura 13.23(b), intervenir rompe el encalce normal causal entre el clima y el aspersor. Esto evita que cualquier influencia fluya hacia atrás desde *Aspersor* a *Nublado*. Así, condicionar en *do*(*Aspersor* = *true*) en el gráfico original es equivalente a condicionar en Aspersor = verdadero en el grafo simplificado.

Un enfoque similar puede ser tomado para analizar el efecto de $do(X_j = x_{jk})$ en una red causal general con variables $X_1,...,X_n$. La red corresponde a una distribución conjunta definida en la forma usual (ver Ecuación (13.2)):

$$P(x_1,...,x_n) = \prod_{i=1}^{n} P(x_i|padres(X_i)) \tag{13.18}$$

Después de aplicar $do(X_j = x_{jk})$, la nueva distribución conjunta $P_{x_{jk}}$ omite el factor para X_j:

$$P_{xjk}(x_1,...,x_n) = \begin{cases} \Pi_{i\neq j} P(x_i|padres(X_i)) = \frac{P(x_1,...,x_n)}{P(x_j|padres(X_j)} & \text{if } x_j = x_{jk} \\ 0 & \text{if } x_j \neq x_{jk} \end{cases} \tag{13.19}$$

Esto se sigue del hecho de que establecer X_j a un valor particular x_{jk} corresponde a eliminar la ecuación $X_j = f_j(Padres\ (X_j),U_j)$ del sistema de ecuaciones estructurales para reemplazarla con $X_j = x_{jk}$. Con un poco más de manipulación algebraica, es posible derivar una fórmula para el efecto de establecer la variable X_j en cualquier otra variable X_i:

$$\begin{aligned} P(X_i = x_i|\text{do}(X_j = x_{jk}) &= P_{xjk}(X_i = x_i) \\ &= \sum_{padres(X_j)} P(x_i|x_{jk}, padres(X_j))P(padres(X_j)). \end{aligned} \tag{13.20}$$

Los términos de probabilidad en la suma se obtienen por cálculo en la red original, por cualquiera de los algoritmos de inferencia estándar. Esta ecuación es conocida como una **fórmula de ajuste,** un promedio ponderado por probabilidad de la influencia de X_j y sus padres en X_i, donde los pesos son los *priors* en los valores de los padres. Los efectos de intervenir en múltiples variables pueden ser calculados imaginando que las intervenciones individuales ocurren en secuencia, cada una eliminando, a su vez, las influencias causales en una variable y produciendo un nuevo modelo mutilado.

Fórmula de ajuste

13.5.2 El criterio de la puerta trasera

La capacidad de predecir el efecto de cualquier intervención es un resultado notable, pero requiere un conocimiento preciso de las distribuciones condicionales necesarias en el modelo, particularmente $P(x_j|\text{padres}(X_j))$. Sin embargo, en muchos contextos del mundo real, esto es mucho pedir. Por ejemplo, sabemos que los "factores genéticos" tienen un papel en la obesidad, pero no sabemos qué genes desempeñan un papel o la naturaleza precisa de sus efectos. Incluso en la simple historia de las decisiones de Mary sobre el aspersor (Figura 13.15, que también se aplica en la Figura 13.23(a)), podríamos saber que ella revisa el clima antes de decidir si encender el aspersor, pero podríamos no saber *cómo* toma su decisión.

La razón específica por la que esto es problemático en este caso es que nos gustaría predecir el efecto de encender el aspersor en una variable descendente como *CéspedMásVerde*, pero la fórmula de ajuste (Ecuación (13.20)) debe tener en cuenta no sólo la ruta directa desde *Aspersor*, sino también la ruta "puerta trasera" a través de *Nublado y Lluvia* . Si supiéramos el valor de *Lluvia*, esta ruta de la puerta trasera estaría bloqueada, lo que sugiere que podría haber una forma de escribir una fórmula de ajuste que condicione en *Lluvia* en lugar de *Nublado*. Y de hecho esto es posible:

$$P(g|do(S = true) = \sum_r P(g|S = true,r)P(r) \qquad (13.21)$$

En general, si deseamos encontrar el efecto de $do(X_j = x_{jk})$ en una variable X_i, el **criterio de la puerta trasera** permite escribir una fórmula de ajuste que condicione en cualquier conjunto de variables $\mathbf{Z}$ que cierra la puerta trasera, por así decirlo. En lenguaje más técnico, queremos un conjunto $\mathbf{Z}$ tal que X_i sea condicionalmente independiente de $Padres(X_j)$ dado X_j y $\mathbf{Z}$. Esto es una aplicación directa de la d-separación (ver página 419).

Criterio de la puerta trasera

El criterio de la puerta trasera es un bloque de construcción básico para una teoría del razonamiento causal que ha surgido en las últimas dos décadas. Proporciona una forma de argumentar contra un siglo de dogma estadístico que afirma que sólo un **ensayo controlado aleatorizado** puede proporcionar información causal. La teoría ha proporcionado herramientas conceptuales y algoritmos para el análisis causal en una amplia gama de configuraciones no experimentales y cuasi experimentales; para calcular probabilidades en declaraciones contrafácticas ("si esto hubiera sucedido en su lugar, ¿cuál habría sido la probabilidad?"); para determinar cuándo los hallazgos en una población pueden transferirse a otra; y para manejar todas las formas de datos faltantes al aprender modelos de probabilidad.

Ensayo controlado aleatorizado

Resumen

Este capítulo ha descrito las **redes bayesianas**, una representación bien desarrollada para el conocimiento incierto. Las redes bayesianas desempeñan un papel aproximadamente análogo al de la lógica proposicional para el conocimiento definitivo.

- Una red bayesiana es un grafo acíclico dirigido cuyos nodos corresponden a variables aleatorias; cada nodo tiene una distribución condicional para el nodo, dado sus padres.
- Las redes bayesianas proporcionan una forma concisa de representar relaciones de **independencia condicional** en el dominio.
- Una red bayesiana especifica una distribución de probabilidad conjunta sobre sus variables. La probabilidad de cualquier asignación dada a todas las variables se define como el producto de las entradas correspondientes en las distribuciones condicionales locales. Una red bayesiana, en general, es exponencialmente más pequeña que una distribución conjunta enumerada de manera explícita.
- Muchas distribuciones condicionales pueden representarse de manera compacta mediante familias canónicas de distribuciones. Las **redes bayesianas híbridas**, que incluyen variables discretas y continuas, usan una variedad de distribuciones canónicas.
- La inferencia en redes bayesianas significa calcular la distribución de probabilidad de un conjunto de variables de consulta, dado un conjunto de variables de evidencia. Los algoritmos de inferencia exactos, como la **eliminación de variables**, evalúan las sumas de productos de probabilidades condicionales de la manera más eficiente posible.
- En **poliárboles** (redes conectadas de manera única), la inferencia exacta lleva tiempo lineal en el tamaño de la red. En el caso general, el problema es intratable.
- Las técnicas de muestreo aleatorio como el **ponderamiento de verosimilitud** y el **Monte Carlo de cadena de Markov** permiten dar estimaciones razonables de las verdaderas probabilidades posteriores en una red y pueden manejar redes mucho más grandes que los algoritmos exactos.
- Mientras que las redes de Bayes capturan influencias probabilísticas, las **redes causales** capturan relaciones causales y permiten predecir los efectos de las intervenciones así como las observaciones.

Notas bibliográficas e históricas

El uso de redes para representar información probabilística comenzó a principios del siglo XX, con el trabajo de Sewall Wright sobre el análisis probabilístico de la herencia genética y los factores de crecimiento animal (Wright, 1921, 1934). I. J. Good (1961), en colaboración con Alan Turing, desarrolló representaciones probabilísticas y métodos de inferencia bayesianos que podrían considerarse como un precursor de las modernas redes bayesianas, aunque el documento no se cita a menudo en este contexto.[7] El mismo documento es la fuente original para el modelo ruidoso-OR.

La representación del **diagrama de influencia** para problemas de decisión, que incorporó una representación DAG para variables aleatorias, se utilizó en el análisis de decisiones a fines de la década de 1970 (ver Capítulo 16), pero sólo se usó la enumeración para la evaluación. Judea Pearl desarrolló el método de paso de mensajes para la inferencia en redes de árboles (Pearl, 1982a) y redes de poliárboles (Kim y Pearl, 1983) y explicó la importancia de los modelos de probabilidad causales en lugar de diagnósticos. El primer sistema experto que usó redes bayesianas fue CONVINCE (Kim, 1983).

Como se relata en el Capítulo 1, a mediados de la década de 1980 se produjo un auge en los sistemas expertos basados en reglas, que incorporaron métodos *ad hoc* para manejar la

[7] I. J. Good fue el estadístico en jefe del equipo de descifrado de códigos de Turing en la Segunda Guerra Mundial. En *2001: A Space Odyssey* (Clarke, 1968), Good y Minsky son acreditados con hacer el avance que llevó al desarrollo de la computadora HAL 9000.

incertidumbre. La probabilidad se consideró tanto impracticable como "cognitivamente implausible" como base para el razonamiento. En sus artículos "En defensa de la probabilidad" de Peter Cheeseman (1985) y "Una investigación sobre la comprensión de la computadora" (Cheeseman, 1988, con comentarios) ayudaron a cambiar las tornas.

El resurgimiento de la probabilidad dependió principalmente, del desarrollo de Pearl de las redes bayesianas y del amplio desarrollo de un enfoque probabilístico para la IA como se describe en su libro, *Razonamiento probabilístico en sistemas inteligentes* (Pearl, 1988). El libro cubrió cuestiones de representación, e incluyó relaciones de independencia condicional y el criterio de d-separación, como enfoques algorítmicos. Geiger *et al.* (1990a) y Tian *et al*. (1998) presentaron resultados computacionales clave sobre la detección eficiente de d-separación.

Eugene Charniak ayudó a presentar las ideas de Pearl a los investigadores de IA con un artículo popular, "Redes bayesianas sin lágrimas"[8] (1991), y un libro (1993). El libro de Dean y Wellman (1991) también ayudó a introducir las redes bayesianas a los investigadores de IA. Shachter (1998) presentó una forma simplificada de determinar la d-separación llamada el algoritmo "Bayes-ball".

A medida que se desarrollaban las aplicaciones de las redes de Bayes, los investigadores encontraron necesario ir más allá del modelo básico de variables discretas con CPTs. Por ejemplo, el sistema CPCS (Pradhan *et al.,* 1994), una red bayesiana para medicina interna con 448 nodos y 906 enlaces, hizo un uso extensivo de los operadores lógicos ruidosos propuestos por Good (1961). Boutilier *et al.* (1996) analizaron los beneficios algorítmicos de la independencia específica del contexto. La inclusión de variables aleatorias continuas en las redes bayesianas fue considerada por Pearl (1988) y Shachter y Kenley (1989); estos documentos discutieron redes que contienen sólo variables continuas con distribuciones gaussianas lineales.

Las redes híbridas con variables discretas y continuas fueron investigadas por Lauritzen y Wermuth (1989) e implementadas en el sistema cHUGIN (Olesen, 1993). Un análisis adicional de los modelos lineales-gaussianos, con conexiones a muchos otros modelos utilizados en estadísticas, aparece en Roweis y Ghahramani (1999); Lerner (2002) proporciona una discusión muy completa de su uso en redes bayesianas híbridas. La distribución probit generalmente se atribuye a Gaddum (1933) y Bliss (1934), aunque se había descubierto varias veces en el siglo XIX. El trabajo de Bliss fue ampliado de manera considerable por Finney (1947). El probit se ha usado con amplitud para modelar fenómenos de elección discreta y se puede extender para manejar más de dos opciones (Daganzo, 1979). El modelo expit (logit inverso) fue introducido por Berkson (1944); inicialmente muy ridiculizado, eventualmente se volvió más popular que el modelo probit. Bishop (1995) da una justificación simple para su uso.

Las primeras aplicaciones de las redes bayesianas en medicina incluyeron el sistema MUNIN para diagnosticar trastornos neuromusculares (Andersen *et al*., 1989) y el sistema PATHFINDER para patología (Heckerman, 1991). Las aplicaciones en ingeniería incluyen el trabajo del Electric Power Research Institute en el monitoreo de generadores de energía (Morjaria *et al*., 1995), el trabajo de la NASA en la visualización de información crítica en tiempo real en el Mission Control en Houston (Horvitz y Barry, 1995), y el campo general de la **tomografía de red**, que tiene como objetivo inferir propiedades locales no observadas de nodos y enlaces en Internet a partir de observaciones del rendimiento de los mensajes de extremo a extremo (Castro *et al*., 2004). Quizá los sistemas de redes bayesianas más utilizados han sido los módulos de diagnóstico y reparación (por ejemplo, el Asistente de impresora) en Microsoft Windows (Breese y Heckerman, 1996) y el Asistente de Office en Microsoft Office (Horvitz *et al*., 1998).

Otra área de aplicación importante es la biología: los modelos matemáticos utilizados para analizar la herencia genética en árboles familiares (el llamado **análisis de pedigrí**) son de

Análisis de pedigrí

[8] El título de la versión original del artículo era "Perla para cerdos".

hecho una forma especial de análisis de pedigrí de redes bayesianas. Los algoritmos de inferencia exacta para el análisis de pedigrí, que se asemejan a la eliminación de variables, se desarrollaron en la década de 1970 (Cannings *et al.,* 1978). Las redes bayesianas se han usado para identificar genes humanos por referencia a genes de ratón (Zhang *et al.,* 2003), inferir redes celulares (Friedman, 2004), análisis de enlace genético para localizar genes relacionados con enfermedades (Silberstein *et al.,* 2013), y muchas otras tareas en bioinformática. Podríamos seguir, pero en su lugar le referiremos a Pourret *et al*. (2008), una guía de 400 páginas sobre aplicaciones de redes bayesianas. Las aplicaciones publicadas durante la última década se cuentan por decenas de miles, que van desde la odontología hasta los modelos climáticos globales.

Judea Pearl (1985), en el primer artículo que utilizó el término "redes bayesianas", describió un algoritmo de inferencia para redes generales basado en la idea de condicionamiento de corte, introducida en el Capítulo 6. Por su parte, Ross Shachter (1986), trabajando en la comunidad de diagramas de influencia, desarrolló un algoritmo completo con base en la reducción dirigida por objetivos de la red utilizando transformaciones que preservan el posterior.

Pearl (1986) desarrolló un algoritmo de agrupamiento para la inferencia exacta en redes bayesianas generales, utilizando una conversión a un poliárbol dirigido de grupos en el que se usó el paso de mensajes para lograr la coherencia sobre las variables compartidas entre los grupos. Un enfoque similar, desarrollado por los estadísticos David Spiegelhalter y Steffen Lauritzen (Lauritzen y Spiegelhalter, 1988), se basa en la conversión a una forma no dirigida de modelo gráfico llamada **red de Markov**. Este enfoque se implementa en el sistema HUGIN, una herramienta eficiente y ampliamente utilizada para el razonamiento incierto (Andersen *et al*., 1989).

La idea básica de la eliminación de variables, que los cálculos repetidos dentro de la expresión general de suma de productos pueden evitarse mediante la memorización, apareció en el algoritmo de inferencia probabilística simbólica (SPI) (Shachter *et al.,* 1990). El algoritmo de eliminación que describimos se asemeja más al desarrollado por Zhang y Poole (1994). Los criterios para podar variables irrelevantes fueron desarrollados por Geiger *et al.* (1990b) y por Lauritzen *et al*. (1990); el criterio que damos es un caso especial simple de estos. Dechter (1999) muestra cómo la idea de eliminación de variables es en esencia idéntica a la **programación dinámica no serial** (Bertele y Brioschi, 1972).

Programación dinámica no serial

Esto conecta los algoritmos de la red bayesiana con métodos relacionados para resolver CSP y proporciona una medida directa de la complejidad de la inferencia exacta en términos del ancho del árbol de la red. Prevenir el crecimiento exponencial en el tamaño de los factores en la eliminación de variables se puede hacer eliminando variables de factores grandes (Dechter y Rish, 2003); también es posible limitar el error introducido de esta manera (Wexler y Meek, 2009). De manera alternativa, los factores se pueden comprimir representándolos mediante diagramas de decisión algebraicos en lugar de tablas (Gogate y Domingos, 2011).

Los métodos exactos basados en la enumeración recursiva (ver Figura 13.11) combinados con la memorización incluyen el algoritmo de condicionamiento recursivo (Darwiche, 2001), el algoritmo de eliminación de valor (Bacchus *et al*., 2003), y la búsqueda AND–OR (Dechter y Mateescu, 2007). El método de conteo de modelos ponderados (Sang *et al*., 2005; Chavira y Darwiche, 2008) por lo regular se basa en un solucionador SAT de estilo DPLL (ver Figura 7.17 en la página 234). Como tal, también está realizando una enumeración recursiva de asignaciones de variables con memorización, por lo que el enfoque es de hecho bastante similar. Estos tres algoritmos permiten implementar una gama completa de compensaciones de espacio/tiempo. Debido a que consideran asignaciones de variables, los algoritmos pueden aprovechar el determinismo y la independencia específica del contexto en el modelo. También pueden modificarse para usar un algoritmo eficiente de tiempo lineal siempre que la

asignación parcial haga que la red restante sea un poliárbol. (Esta es una versión del método de **condicionamiento de corte**, que se describió para CSP en el Capítulo 6). Para la inferencia exacta en modelos grandes, donde los requisitos de espacio de agrupamiento y eliminación de variables se vuelven enormes, estos algoritmos recursivos con frecuencia son el enfoque más práctico.

Hay otras tareas de inferencia importantes en las redes de Bayes además de calcular las probabilidades marginales. La **explicación más probable** o MPE es la asignación más probable a las variables no evidenciales dada la evidencia. (MPE es un caso especial de MAP, inferencia máxima *a posteriori*, que pide la asignación más probable a un *subconjunto* de variables no evidenciales dada la evidencia). Para tales problemas, se han desarrollado muchos algoritmos diferentes, algunos relacionados con los algoritmos de búsqueda de camino más corto o AND–OR; para un resumen, ver Marinescu y Dechter (2009).

Explicación más probable

El primer resultado sobre la complejidad de la inferencia en las redes de Bayes es debido a Cooper (1990), quien mostró que el problema general de calcular los marginales en las redes bayesianas es NP-difícil; como se señala en el capítulo, esto se puede fortalecer a #P-dureza a través de una reducción de contar asignaciones satisfactorias (Roth, 1996). Esto también implica la NP-dificultad de la inferencia aproximada (Dagum y Luby, 1993); sin embargo, para el caso en que las probabilidades estén alejadas de 0 y 1, una forma de ponderación de verosimilitud converge en tiempo (aleatorizado) polinomial (Dagum y Luby, 1997). Shimony (1994) mostró que encontrar la explicación más probable es NP-completo, intratable, pero algo más fácil que calcular los marginales, mientras que Park y Darwiche (2004) proporcionan un análisis de complejidad exhaustivo de cálculo de MAP, mostrando que cae en la clase de problemas NP^{PP}-completos, es decir, algo más difícil que calcular los marginales.

El desarrollo de algoritmos de aproximación rápidos para la inferencia de la red bayesiana es un área muy activa, con contribuciones de estadísticas, ciencias de la computación y física. El método de muestreo por rechazo es una técnica general que se remonta al menos a la aguja de Buffon (1777); fue aplicado por primera vez a las redes bayesianas por Max Henrion (1988), quien lo llamó **muestreo lógico**. El muestreo de importancia fue inventado originalmente para aplicaciones en física (Kahn, 1950a, 1950b) y aplicado a la inferencia de la red bayesiana por Fung y Chang (1989) (quienes llamaron al algoritmo "ponderación de evidencia") y por Shachter y Peot (1989).

En estadística, el **muestreo adaptativo** se ha aplicado a todo tipo de algoritmos de Monte Carlo para acelerar la convergencia. La idea básica es adaptar la distribución de la que se generan las muestras, en función del resultado de las muestras anteriores. Gilks y Wild (1992) desarrollaron el muestreo de rechazo adaptativo, mientras que el muestreo de importancia adaptativo parece haberse originado independientemente en física (Lepage, 1978), ingeniería civil (Karamchandani *et al*., 1989), estadística (Oh y Berger, 1992) y gráficos por computadora (Veach y Guibas, 1995). Cheng y Druzdzel (2000) describen una versión adaptativa del muestreo de importancia aplicado a la inferencia de la red bayesiana. Más recientemente, Le *et al*. (2017) han demostrado el uso de sistemas de aprendizaje profundo para producir distribuciones de propuestas que aceleran el muestreo de importancia en muchos órdenes de magnitud.

Los algoritmos de Monte Carlo de cadena de Markov (MCMC) comenzaron con el algoritmo de Metropolis, debido a Metropolis *et al.* (1953), que también fue la fuente del algoritmo de recocido simulado descrito en el Capítulo 4. Hastings (1970) introdujo el paso de aceptar/rechazar que es una parte integral de lo que ahora llamamos el algoritmo de Metropolis–Hastings. El muestreador de Gibbs fue ideado por Geman y Geman (1984) para la inferencia en redes de Markov no dirigidas. La aplicación del muestreo de Gibbs a las redes bayesianas se debe a Pearl (1987). Los documentos recopilados por Gilks *et al.* (1996) cubren tanto la teoría como las aplicaciones de MCMC.

Desde mediados de la década de 1990, MCMC se ha convertido en el caballo de batalla de la estadística bayesiana y el cálculo estadístico en muchas otras disciplinas, incluidas la física y la biología. El *Manual de la Cadena de Markov Monte Carlo* (Brooks *et al.,* 2011) cubre muchos aspectos de esta literatura. El paquete BUGS (Gilks *et al.,* 1994) fue un sistema temprano e influyente para el modelado e inferencia de la red bayesiana utilizando muestreo de Gibbs. STAN (nombrado en honor a Stanislaw Ulam, un originador de los métodos de Monte Carlo en física) es un sistema más reciente que utiliza la inferencia de Monte Carlo Hamiltoniana (Carpenter *et al*., 2017).

Hay dos familias muy importantes de métodos de aproximación que no cubrimos en el capítulo. La primera es la familia de métodos de **aproximación variacional**, que se pueden utilizar para simplificar cálculos complejos de todo tipo. La idea básica es proponer una versión reducida del problema original que sea simple de trabajar, pero que se parezca al problema original lo más posible. El problema reducido se describe por algunos **parámetros variacionales** λ que se ajustan para minimizar una función de distancia D entre el original y el problema reducido, en la mayoría de casos resolviendo el sistema de ecuaciones $\partial D/\partial\lambda = 0$. En muchos casos, es posible obtener límites superiores e inferiores estrictos. Los métodos variacionales han sido utilizados durante mucho tiempo en estadísticas (Rustagi, 1976). En física estadística, el método de campo medio es una aproximación variacional particular en la que se supone que las variables individuales que componen el modelo son completamente independientes.

Esta idea se aplicó para resolver grandes redes de Markov no dirigidas (Peterson y Anderson, 1987; Parisi, 1988). Saul *et al*. (1996) desarrollaron los fundamentos matemáticos para aplicar métodos variacionales a las redes bayesianas y obtuvieron aproximaciones de límite inferior precisas para las redes sigmoidales con el uso de métodos de campo medio. Jaakkola y Jordan (1996) extendieron la metodología para obtener límites inferiores y superiores. Desde estos primeros trabajos, los métodos variacionales se han aplicado a muchas familias específicas de modelos. El notable artículo de Wainwright y Jordan (2008) proporciona un análisis teórico unificador de la literatura sobre métodos variacionales.

Una segunda familia importante de algoritmos de aproximación se basa en el algoritmo de paso de mensajes de poliárbol de Pearl (1982a). Este algoritmo se puede aplicar a redes "loopy" generales, como sugirió Pearl (1988). Los resultados podrían ser incorrectos o el algoritmo podría fallar en terminar, pero en muchos casos los valores obtenidos están cerca de los valores verdaderos. Se prestó poca atención a este enfoque de **propagación de creencias en bucle** hasta que McEliece *et al*. (1998) observaron que es exactamente el cálculo realizado por el algoritmo de **decodificación turbo** (Berrou *et al*., 1993), que proporcionó un gran avance en el diseño de códigos de corrección de errores eficientes.

Propagación de creencias en bucle

Decodificación turbo

La implicación de estas observaciones es que si el BP en bucle es rápido y preciso en las redes muy grandes y muy conectadas utilizadas para la decodificación; por lo tanto, podría ser útil de manera más general. El soporte teórico para estos hallazgos, incluidas las pruebas de convergencia para algunos casos especiales, fue proporcionado por Weiss (2000b), Weiss y Freeman (2001), Yedidia *et al.* (2005), basándose en conexiones con ideas de la física estadística.

Las teorías de inferencia causal que van más allá de los ensayos controlados aleatorizados fueron propuestas por Rubin (1974) y Robins (1986), pero estas ideas permanecieron tanto oscuras como controversiales hasta que Judea Pearl desarrolló y presentó una teoría completamente articulada de la causalidad basada en redes causales (Pearl, 2000). Peters *et al.* (2017) desarrollan aún más la teoría, con un énfasis en el aprendizaje. Un trabajo más reciente, *The Book of Why* (Pearl y McKenzie, 2018), proporciona una introducción menos matemática pero más legible y de amplio alcance.

El razonamiento incierto en IA no siempre se ha basado en la teoría de la probabilidad. Como se señaló en el Capítulo 12, los primeros sistemas probabilísticos cayeron en desgracia a principios de la década de 1970, y dejaron un vacío parcial para ser llenado por métodos alternativos. Estos incluyeron sistemas expertos basados en reglas, la teoría de Dempster–Shafer y (en cierta medida) la lógica difusa.[9]

Los enfoques basados en reglas para la incertidumbre esperaban construir sobre el éxito de los sistemas basados en reglas lógicas, pero añadir una especie de "factor de corrección", más cortésmente llamado **factor de certeza**, a cada regla para acomodar la incertidumbre. El primer sistema de este tipo fue MYCIN (Shortliffe, 1976), un sistema experto médico para infecciones bacterianas. La colección *Rule-Based Expert Systems* (Buchanan y Shortliffe, 1984) proporciona una visión completa de MYCIN y sus descendientes (ver también Stefik, 1995).

David Heckerman (1986) mostró que una versión ligeramente modificada de los cálculos del factor de certeza da resultados probabilísticos correctos en algunos casos, pero resulta en un recuento excesivo de evidencia en otros casos. A medida que los conjuntos de reglas se hicieron más grandes, las interacciones indeseables entre las reglas se hicieron más comunes, y los profesionales encontraron que los factores de certeza de muchas otras reglas tenían que ser "ajustados" cuando se añadían nuevas reglas. Las propiedades matemáticas básicas que permiten las cadenas de razonamiento en lógica simplemente no se mantienen para la probabilidad.

La teoría de Dempster–Shafer se origina con un artículo de Arthur Dempster (1968) que propone una generalización de la probabilidad a valores de intervalo y una regla de combinación para usarlos. Tal enfoque podría aliviar la dificultad de especificar las probabilidades exactamente. Trabajos posteriores de Glenn Shafer (1976) llevaron a que la teoría de Dempster–Shafer se viera como un enfoque competidor para la probabilidad. Pearl (1988) y Ruspini *et al.* (1992) analizan la relación entre la teoría de Dempster–Shafer y la teoría de probabilidad estándar. En muchos casos, la teoría de probabilidad no requiere que las probabilidades se especifiquen exactamente: podemos expresar la incertidumbre sobre los valores de probabilidad como distribuciones de probabilidad (de segundo orden), como se explica en el Capítulo 20.

Los **conjuntos difusos** fueron desarrollados por Lotfi Zadeh (1965) en respuesta a la dificultad percibida de proporcionar entradas exactas a sistemas inteligentes. Un conjunto difuso es aquel en el que la pertenencia es una cuestión de grado. La **lógica difusa** es un método para razonar con expresiones lógicas que describen la pertenencia a conjuntos difusos. El **control difuso** es una metodología para construir sistemas de control en los que la correspondencia entre parámetros de entrada y salida de valor real está representada por reglas difusas. El control difuso ha tenido mucho éxito en productos comerciales como transmisiones automáticas, cámaras de video y afeitadoras eléctricas. El texto de Zimmermann (2001) proporciona una introducción completa a la teoría de conjuntos difusos; los documentos sobre aplicaciones difusas se recogen en Zimmermann (1999).

Con frecuencia, la lógica difusa ha sido percibida incorrectamente como un competidor directo de la teoría de la probabilidad, aunque en realidad aborda un conjunto diferente de problemas: en lugar de considerar la incertidumbre sobre la verdad de proposiciones bien definidas, la lógica difusa maneja la **vaguedad** en la correspondencia de términos en una teoría simbólica a un mundo real. La vaguedad es un problema real en cualquier aplicación de lógica, probabilidad o, de hecho, modelos matemáticos estándar a la realidad. Incluso una variable tan impecable como la masa de la Tierra resulta, al inspeccionarla, que varía con el tiempo a medida que los meteoritos y las moléculas van y vienen. También es impreciso, ¿incluye la atmósfera? Si es así, ¿hasta qué altura? En algunos casos, una mayor elaboración del modelo puede reducir la vaguedad, pero la lógica difusa toma la vaguedad como un hecho y desarrolla una teoría a su alrededor.

[9] Un cuarto enfoque, el **razonamiento por defecto**, trata las conclusiones no como "creídas hasta cierto grado", sino como "creídas hasta que se encuentra una mejor razón para creer algo más". Este enfoque se cubre en el Capítulo 10.

Teoría de la posibilidad

La **teoría de la posibilidad** (Zadeh, 1978) fue introducida para manejar la incertidumbre en sistemas difusos y tiene mucho en común con la probabilidad (Dubois y Prade, 1994).

Muchos investigadores de IA en la década de 1970 rechazaron la probabilidad porque los cálculos numéricos que se pensaba que requería la teoría de la probabilidad no eran aparentes para la introspección y suponían un nivel de precisión poco realista en nuestro conocimiento incierto. El desarrollo **de redes probabilísticas cualitativas** (Wellman, 1990a) proporcionó una abstracción puramente cualitativa de las redes bayesianas, al utilizar la noción de influencias positivas y negativas entre variables. Wellman muestra que en muchos casos dicha información es suficiente para la toma de decisiones óptima sin la necesidad de la especificación precisa de los valores de probabilidad. Goldszmidt y Pearl (1996) adoptan un enfoque similar. El trabajo de Darwiche y Ginsberg (1992) extrae las propiedades básicas de la condición y la combinación de evidencias de la teoría de la probabilidad y muestra que también pueden aplicarse en el razonamiento lógico y razonamiento por defecto.

Varios excelentes textos (Jensen, 2007; Darwiche, 2009; Koller y Friedman, 2009; Korb y Nicholson, 2010; Dechter, 2019) proporcionan tratamientos completos de los temas que hemos cubierto en este capítulo. La nueva investigación sobre el razonamiento probabilístico aparece tanto en revistas de IA convencionales, como *Artificial Intelligence* y el *Journal of AI Research*, así como en revistas más especializadas, como el *International Journal of Approximate Reasoning*. Muchos trabajos sobre modelos gráficos, que incluyen redes bayesianas, aparecen en revistas de estadística. Las actas de las conferencias sobre Incertidumbre en Inteligencia Artificial (UAI), Sistemas de Procesamiento de Información Neural (NeurIPS) e Inteligencia Artificial y Estadísticas (AISTATS) son buenas fuentes para la investigación actual.

CAPÍTULO 14

RAZONAMIENTO PROBABILÍSTICO A LO LARGO DEL TIEMPO

En el que intentamos interpretar el presente, entender el pasado y, tal vez, predecir el futuro, aun cuando muy poco está claro.

Los agentes en entornos parcialmente observables deben ser capaces de seguir el estado actual, en la medida en que sus sensores lo permitan. En la Sección 4.4 mostramos una metodología para hacer eso: un agente mantiene un **estado de creencia** que representa qué estados del mundo son posibles actualmente. A partir del estado de creencia y un **modelo de transición**, el agente permite predecir cómo podría evolucionar el mundo en el próximo paso de tiempo. A partir de los preceptos observados y un **modelo de sensor**, el agente puede actualizar el estado de creencia. Esta es una idea generalizada: en el Capítulo 4 los estados de creencia se representaban mediante conjuntos de estados enumerados de manera explícita, mientras que en los Capítulos 7 y 11 se representaban mediante fórmulas lógicas. Ambos enfoques definieron los estados de creencia en términos de qué estados del mundo eran *posibles*, pero no podían decir nada sobre qué estados eran *probables* o *improbables*. En este capítulo, aplicamos la teoría de la probabilidad para cuantificar el grado de creencia en los elementos del estado de creencia.

Como mostramos en la Sección 14.1, el tiempo en sí se maneja de la misma manera que en el Capítulo 7: un mundo cambiante se modela usando una variable para cada aspecto del estado del mundo *en cada punto en el tiempo*. Los modelos de transición y sensor pueden ser inciertos: el modelo de transición describe la distribución de probabilidad de las variables en el tiempo t, dado el estado del mundo en tiempos pasados, mientras que el modelo de sensor describe la probabilidad de cada precepto en el tiempo t, dado el estado actual del mundo. La Sección 14.2 define las tareas básicas de inferencia y describe la estructura general de los algoritmos de inferencia para modelos temporales. Luego, describimos tres tipos específicos de modelos: **modelos ocultos de Markov**, **filtros de Kalman** y **redes bayesianas dinámicas** (que incluyen modelos ocultos de Markov y filtros de Kalman como casos especiales).

14.1 Tiempo e incertidumbre

Hemos desarrollado nuestras técnicas para el razonamiento probabilístico en el contexto de mundos *estáticos*, en los que cada variable aleatoria tiene un único valor fijo. Por ejemplo, cuando reparamos un coche, asumimos que lo que falla y permanece en falla durante el proceso de diagnóstico; nuestro trabajo es inferir el estado del coche a partir de las pruebas observadas, que también permanecen fijas.

Ahora, considere un problema diferente: tratar a un paciente diabético. Al igual que en el caso de la reparación de coches, tenemos pruebas como dosis recientes de insulina, ingesta de alimentos, mediciones de azúcar en sangre y otros signos físicos. La tarea es evaluar el estado actual del paciente, incluyendo el nivel real de azúcar en sangre y de insulina. Con base en esta

información, podemos tomar una decisión sobre la ingesta de alimentos y la dosis de insulina del paciente. A diferencia del caso de la reparación de coches, aquí los aspectos *dinámicos* del problema son esenciales. Los niveles de azúcar en sangre y las mediciones de estos cambian con el tiempo, dependiendo de la última ingesta de alimentos y dosis de insulina, la actividad metabólica, la hora del día, etc. Para evaluar el estado actual a partir de la historia de las pruebas y predecir los resultados de las acciones de tratamiento, debemos modelar estos cambios.

Las mismas consideraciones surgen en otros contextos, como cuando se da seguimiento a la ubicación de un robot, a la actividad económica de una nación y a la comprensión de una secuencia hablada o escrita de palabras. ¿Cómo se modelan situaciones dinámicas como estas?

14.1.1 Estados y observaciones

Tiempo discreto
Cortes de tiempo

Este capítulo analiza modelos de **tiempo discreto**, en los que el mundo se ve como una serie de instantáneas o **cortes de tiempo**.[1] Numeraremos los cortes de tiempo 0, 1, 2, etc., en lugar de asignarles tiempos específicos. Por lo regular, se asume que el intervalo de tiempo Δ entre cortes es el mismo para cada intervalo. Para cualquier aplicación particular, se tiene que elegir un valor específico de Δ. A veces esto es dictado por el sensor; por ejemplo, una cámara de video podría suministrar imágenes a intervalos de 1/30 de segundo. En otros casos, el intervalo es dictado por las tasas típicas de cambio de las variables relevantes; por ejemplo, en el caso de la monitorización de la glucosa en sangre, las cosas pueden cambiar de manera significativa en el curso de diez minutos, por lo que un intervalo de un minuto podría ser apropiado. Por otro lado, en la modelización del desplazamiento continental a lo largo del tiempo geológico, un intervalo de un millón de años podría estar bien.

Cada corte de tiempo en un modelo de probabilidad de tiempo discreto contiene un conjunto de variables aleatorias, algunas observables y otras no. Para resumir, asumiremos que el mismo subconjunto de variables es observable en cada corte de tiempo (aunque esto no es estrictamente necesario en nada de lo que sigue). Usaremos $\mathbf{X}_t$ para denotar el conjunto de variables de estado en el tiempo t, que se supone que son inobservables, y $\mathbf{E}_t$ para denotar el conjunto de variables de evidencia observables. La observación en el tiempo t es $\mathbf{E}_t = \mathbf{e}_t$ para algún conjunto de valores $\mathbf{e}_t$.

Considere el siguiente ejemplo: Usted es guardia de seguridad estacionado en una instalación subterránea secreta. Quiere saber si hoy lloverá, pero su único acceso al mundo exterior se da cada mañana cuando ve al director entrar con o sin paraguas. Para cada día t, el conjunto $\mathbf{E}_t$ contiene así una única variable de evidencia $Paraguas_t$ o U_t para abreviar (si aparece el paraguas), y el conjunto $\mathbf{X}_t$ contiene una única variable de estado Lluvia_t o R_t para abreviar (si está lloviendo). Otros problemas pueden involucrar conjuntos más grandes de variables. En el ejemplo de la diabetes, las variables de evidencia podrían ser $AzucarSangreMedido_t$ y $Pulsaciones_t$ mientras que las variables de estado podrían incluir $AzucarSangre_t$ y $ContenidoEstomago_t$. (Observe que $AzucarSangre_t$ y $AzucarSangreMedido_t$ no son la misma variable; así es como tratamos con mediciones ruidosas de cantidades reales).

Asumiremos que la secuencia de estados comienza en $t = 0$ y la evidencia comienza a llegar en $t = 1$. Por lo tanto, nuestro mundo de paraguas está representado por variables de estado R_0, R_1, R_2,... y variables de evidencia U_1, U_2,... Usaremos la notación $a{:}b$ para denotar la secuencia de enteros de a a b y la notación $\mathbf{X}_{a:b}$ para denotar el conjunto de variables de $\mathbf{X}_a$ a $\mathbf{X}_b$. Por ejemplo, $U_{1:3}$ corresponde a U_1, U_2, U_3. (Note que esto es diferente de la notación utilizada en lenguajes de programación como Python y Go, donde `U[1:3]` *no* incluiría `U[3]`).

[1] La incertidumbre a lo largo del tiempo *continuo* permite ser modelada por **ecuaciones diferenciales estocásticas** (SDEs). Los modelos estudiados en este capítulo pueden ser vistos como aproximaciones en tiempo discreto a las SDEs.

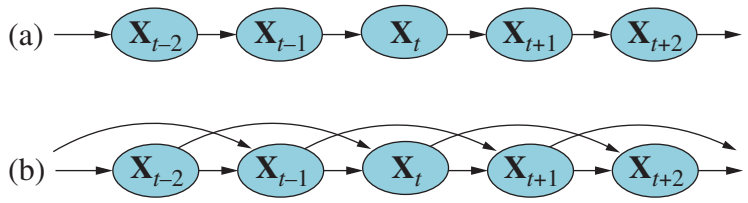


Figura 14.1 (a) Estructura de red bayesiana correspondiente a un proceso de Markov de primer orden con estado definido por las variables $\mathbf{X}_t$. (b) Un proceso de Markov de segundo orden.

14.1.2 Modelos de transición y sensor

Con el conjunto de variables de estado y evidencia para un problema dado decidido, el siguiente paso es especificar cómo evoluciona el mundo (el modelo de transición) y cómo las variables de evidencia obtienen sus valores (el modelo de sensor).

El modelo de transición especifica la distribución de probabilidad sobre las últimas variables de estado, dados los valores previos, es decir, $\mathbf{P}(\mathbf{X}_t|\mathbf{X}_{0:t-1})$. Ahora nos enfrentamos a un problema: el conjunto $\mathbf{X}_{0:t-1}$ es ilimitado en tamaño a medida que t aumenta. Resolvemos el problema haciendo una **suposición de Markov**: que el estado actual depende sólo de un *número finito fijo* de estados anteriores. Los procesos que satisfacen esta suposición fueron estudiados en profundidad por primera vez por el estadístico Andrei Markov (1856–1922), los cuales se llaman **procesos de Markov** o **cadenas de Markov,** y que vienen en varios sabores; el más simple es el **proceso de Markov de primer orden**, en el que el estado actual depende sólo del estado anterior y no de ningún estado temprano. En otras palabras, un estado proporciona suficiente información para hacer que el futuro sea condicionalmente independiente del pasado, y tenemos

Suposición de Markov

Procesos de Markov

Proceso de Markov de primer orden

$$\mathbf{P}(\mathbf{X}_t|\mathbf{X}_{0:t-1}) = \mathbf{P}(\mathbf{X}_t|\mathbf{X}_{t-1}). \tag{14.1}$$

Por lo tanto, en un proceso de Markov de primer orden, el modelo de transición es la distribución condicional $\mathbf{P}(\mathbf{X}_t|\mathbf{X}_{t-1})$. El modelo de transición para un proceso de Markov de segundo orden es la distribución condicional $\mathbf{P}(\mathbf{X}_t|\mathbf{X}_{t-2},\mathbf{X}_{t-1})$. La Figura 14.1 muestra las estructuras de red bayesiana correspondientes a procesos de Markov de primer y segundo orden.

Incluso con la suposición de Markov persiste un problema: hay infinitamente muchos valores posibles de t. ¿Necesitamos especificar una distribución diferente para cada paso de tiempo? Evitamos este problema asumiendo que los cambios en el estado del mundo son causados por un proceso **homogéneo en el tiempo**; es decir, un proceso de cambio gobernado por leyes que no cambian con el tiempo. En el mundo del paraguas, entonces, la probabilidad condicional de lluvia, $\mathbf{P}(R_t|R_{t-1})$, es la misma para todo t, y necesitamos especificar sólo una tabla de probabilidad condicional.

Homogéneo en el tiempo

Ahora, para el modelo de sensor, las variables de evidencia $\mathbf{E}_t$ *podrían* depender de variables anteriores, así como de las variables de estado actuales, pero cualquier estado que valga la pena debería ser suficiente para generar los valores actuales del sensor. Por lo tanto, hacemos una **suposición de Markov del sensor** de la siguiente manera:

Suposición de Markov del sensor

$$\mathbf{P}(\mathbf{E}_t|\mathbf{X}_{0:t},\mathbf{E}_{1:t-1}) = \mathbf{P}(\mathbf{E}_t|\mathbf{X}_t). \tag{14.2}$$

Por lo tanto, $\mathbf{P}(\mathbf{E}_t|\mathbf{X}_t)$ es nuestro modelo de sensor (a veces llamado el **modelo de observación**). La Figura 14.2 muestra el modelo de transición y el modelo de sensor para el ejemplo del paraguas. Observe la dirección de la dependencia entre estado y sensores: las flechas van del estado actual del mundo a los valores del sensor porque el estado del mundo *provoca* que los sensores

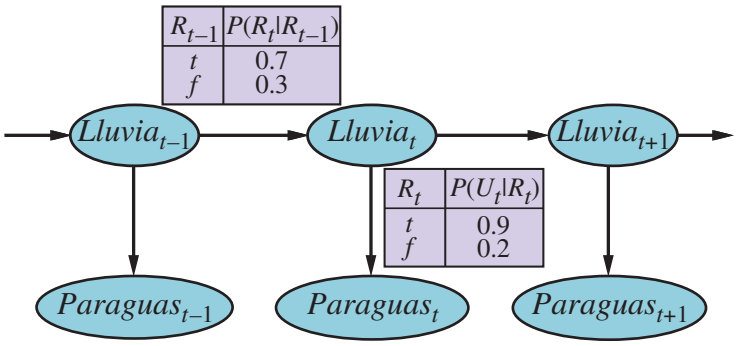


Figura 14.2 Estructura de la red bayesiana y distribuciones condicionales que describen el mundo del paraguas. El modelo de transición es $\mathbf{P}(Lluvia_t|Lluvia_{t-1})$ y el modelo del sensor es $\mathbf{P}(Paraguas_t|Lluvia_t)$.

adopten valores particulares: la lluvia *provoca* que aparezca el paraguas. (El proceso de inferencia, por supuesto, va en la otra dirección; la distinción entre la dirección de las dependencias modeladas y la dirección de la inferencia es una de las principales ventajas de las redes bayesianas).

Además de especificar los modelos de transición y sensor, necesitamos decir cómo comienza todo: la distribución de probabilidad previa en el tiempo 0,$\mathbf{P}(\mathbf{X}_0)$. Con eso, tenemos una especificación de la distribución conjunta completa sobre todas las variables, utilizando la Ecuación (13.2). Para cualquier paso de tiempo t,

$$\mathbf{P}(\mathbf{X}_{0:t},\mathbf{E}_{1:t}) = \mathbf{P}(\mathbf{X}_0)\prod_{i=1}^{t}\mathbf{P}(\mathbf{X}_i|\mathbf{X}_{i-1})\mathbf{P}(\mathbf{E}_i|\mathbf{X}_i). \tag{14.3}$$

Los tres términos en el lado derecho son el modelo de estado inicial $\mathbf{P}(\mathbf{X}_0)$, el modelo de transición $\mathbf{P}(\mathbf{X}_i|\mathbf{X}_{i-1})$, y el modelo del sensor $\mathbf{P}(\mathbf{E}_i|\mathbf{X}_i)$. Esta ecuación define la semántica de la familia de modelos temporales representados por los tres términos. Observe que las redes bayesianas estándar no permiten representar tales modelos porque requieren un conjunto finito de variables. La capacidad de manejar un conjunto infinito de variables proviene de dos cosas: primero, definir el conjunto infinito usando índices enteros; y segundo, el uso de la cuantificación universal implícita (ver Sección 8.2) para definir los modelos de sensor y transición para cada paso de tiempo.

La estructura en la Figura 14.2 es un proceso de Markov de primer orden: se supone que la probabilidad de lluvia depende sólo de si llovió el día anterior. Si tal suposición es razonable depende del dominio en sí. La suposición de Markov de primer orden indica que las variables de estado contienen *toda* la información necesaria para caracterizar la distribución de probabilidad para el siguiente segmento de tiempo. En ocasiones la suposición es cierta; por ejemplo, si una partícula ejecuta un paseo aleatorio a lo largo del eje x, cambiando su posición por ±1 en cada paso de tiempo, entonces usar la coordenada x como el estado da un proceso de Markov de primer orden. A veces la suposición es sólo aproximada, como en el caso de predecir la lluvia sólo con base en si llovió el día anterior. Hay dos formas de mejorar la precisión de la aproximación:

1. Aumentar el orden del modelo de proceso de Markov. Por ejemplo, podríamos hacer un modelo de segundo orden agregando $Lluvia_{t-2}$ como un padre de $Lluvia_t$, lo que podría dar predicciones más precisas. Por ejemplo, en Palo Alto, California, rara vez llueve más de dos días seguidos.

2. Aumentar el conjunto de variables de estado. Por ejemplo, podríamos agregar $Estación_t$ para permitirnos incorporar registros históricos de temporadas de lluvia, o podríamos agregar $Temperatura_t$, $Humedad_t$, y $Presión_t$ (quizás en un rango de ubicaciones) para permitirnos usar un modelo físico de condiciones de lluvia.

El ejercicio 14.AUGM pide que demuestre que la primera solución, aumentar el orden, permite reformularse como un aumento en el conjunto de variables de estado, manteniendo el orden fijo. Observe que agregar variables de estado podría mejorar el poder predictivo del sistema pero también aumenta los *requisitos* de predicción: ahora también tenemos que predecir las nuevas variables. Por lo tanto, estamos buscando un conjunto de variables "autosuficientes", lo que significa que tenemos que entender la "física" del proceso que se está modelando. El requisito de modelado preciso del proceso se reduce si podemos agregar nuevos sensores (por ejemplo, mediciones de temperatura y presión) que proporcionan información directa sobre las nuevas variables de estado.

Considere, por ejemplo, el problema de rastrear un robot que deambula al azar en el plano X–Y. Uno podría proponer que la posición y la velocidad son un conjunto suficiente de variables de estado: uno puede usar las leyes de Newton para calcular la nueva posición, y la velocidad puede cambiar de manera impredecible. Si el robot funciona con batería, entonces el agotamiento de la batería tendería a tener un efecto sistemático en el cambio de velocidad. Como esto, a su vez, depende de cuánta energía fue utilizada por todas las maniobras anteriores, se viola la propiedad de Markov.

Podemos restaurar la propiedad de Markov incluyendo el nivel de carga $Batería_t$ como una de las variables de estado que componen $\mathbf{X}_t$. Esto ayuda a predecir el movimiento del robot pero, a su vez, requiere un modelo para predecir $Batería_t$ a partir de $Batería_{t-1}$ y la velocidad. En algunos casos, eso se hace de manera confiable, pero con frecuencia encontramos que el error se acumula con el tiempo. En ese caso, es posible mejorar la precisión al *agregar un nuevo sensor* para el nivel de la batería. Volveremos al ejemplo de la batería en la Sección 14.5.

14.2 Inferencia en modelos temporales

Luego de establecer la estructura de un modelo temporal genérico, podemos formular las tareas básicas de inferencia que deben resolverse:

- El **filtrado**[2] o **estimación de estado** es la tarea de calcular el **estado de creencia** $\mathbf{P}(\mathbf{X}_t|\mathbf{e}_{1:t})$, la distribución posterior sobre el estado más reciente dada toda la evidencia hasta la fecha. En el ejemplo del paraguas, esto significaría calcular la probabilidad de lluvia hoy, dadas todas las observaciones del paraguas realizadas hasta ahora. El filtrado es lo que hace un agente racional para mantener un seguimiento del estado actual para que se puedan tomar decisiones racionales. Resulta que un cálculo casi idéntico proporciona la probabilidad de la secuencia de evidencia, $\mathbf{P}(\mathbf{e}_{1:t})$. — Filtrado, Estimación de estado, Estado de creencia
- **Predicción**: Esta es la tarea de calcular la distribución posterior sobre el estado *futuro*, dada toda la evidencia hasta la fecha. Es decir, deseamos calcular $\mathbf{P}(\mathbf{X}_{t+k}|\mathbf{e}_{1:t})$ para algún $k > 0$. En el ejemplo del paraguas, esto podría significar calcular la probabilidad de lluvia dentro de tres días, dadas todas las observaciones hasta la fecha. La predicción es útil para evaluar posibles cursos de acción con base en sus resultados esperados. — Predicción
- **Suavizado**: Esta es la tarea de calcular la distribución posterior sobre un estado pasado, dada toda la evidencia en el presente. Es decir, deseamos calcular $\mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:t})$ para algún k tal que $0 \leq k < t$. En el ejemplo del paraguas, podría significar calcular la probabilidad de que — Suavizado

[2] El término "filtrado" se refiere a las raíces de este problema en los primeros trabajos sobre procesamiento de señales, donde el problema es filtrar el ruido en una señal estimando sus propiedades subyacentes.

lloviera el miércoles pasado, dadas todas las observaciones del portador del paraguas realizadas hasta hoy. El suavizado proporciona una mejor estimación del estado en el tiempo k de la que estaba disponible en ese momento, porque incorpora más evidencia.[3]

- **Explicación más probable**: Dada una secuencia de observaciones, desearíamos encontrar la secuencia de estados que es más probable que haya generado esas observaciones. Es decir, deseamos calcular $\text{argmax}_{\mathbf{x}_{1:t}} P(\mathbf{x}_{1:t}|\mathbf{e}_{1:t})$. Por ejemplo, si el paraguas aparece cada uno de los primeros tres días y se ausenta en el cuarto, entonces la explicación más probable es que llovió los primeros tres días y no llovió el cuarto. Los algoritmos para esta tarea son útiles en muchas aplicaciones, incluyendo el reconocimiento de voz, donde el objetivo es encontrar la secuencia de palabras más probable, dada una serie de sonidos, y la reconstrucción de cadenas de bits transmitidas por un canal ruidoso.

Además de estas tareas de inferencia, también tenemos

- **Aprendizaje**: Los modelos de transición y sensor, si aún no se conocen, permiten aprenderse a partir de observaciones. Al igual que con las redes bayesianas estáticas, el aprendizaje de la red Bayes dinámica puede hacerse como un subproducto de la inferencia. La inferencia proporciona una estimación de qué transiciones ocurrieron en realidad y de qué estados generaron las lecturas del sensor, y estas estimaciones permiten usarse para aprender los modelos. El proceso de aprendizaje opera a través de un algoritmo de actualización iterativa llamado expectativa-maximización (EM) o, dada la evidencia, puede resultar de la actualización bayesiana de los parámetros del modelo. Consulte el Capítulo 20 para más detalles.

El resto de esta sección describe algoritmos genéricos para las cuatro tareas de inferencia, independiente del tipo particular de modelo empleado. Las mejoras específicas para cada modelo se describen en las secciones siguientes.

14.2.1 Filtrado y predicción

Como señalamos en la Sección 7.7.3, un algoritmo de filtrado útil necesita mantener una estimación de estado actual y renovar, en lugar de volver sobre toda la historia de percepciones para cada actualización. (De lo contrario, el costo de cada actualización aumenta a medida que pasa el tiempo). En otras palabras, dado el resultado del filtrado hasta el tiempo t, el agente necesita calcular el resultado para $t + 1$ a partir de la nueva evidencia $\mathbf{e}_{t+1}$. Así que tenemos

$$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) = f(\mathbf{e}_{t+1}, \mathbf{P}(\mathbf{X}_t|\mathbf{e}_{1:t}))$$

para alguna función f. Este proceso se llama **estimación recursiva**. (Ver también las Secciones 4.4 y 7.7.3). Podemos ver el cálculo como compuesto por dos partes: primero, la distribución del estado actual se proyecta hacia adelante de t a $t + 1$; luego se actualiza utilizando la nueva evidencia $\mathbf{e}_{t+1}$. Este proceso de dos partes surge de manera simple cuando se reorganiza la fórmula:

$$\begin{aligned}
\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) &= \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}, \mathbf{e}_{t+1}) && \text{(dividiendo la evidencia)} \\
&= \alpha\, \mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1}, \mathbf{e}_{1:t})\, \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}) && \text{(usando la regla de Bayes, dado } \mathbf{e}_{1:t}) \\
&= \alpha\, \underbrace{\mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})}_{\text{actualización}}\, \underbrace{\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t})}_{\text{predicción}} && \text{(por la suposición de Markov del sensor).} \qquad (14.4)
\end{aligned}$$

Aquí y a lo largo de este capítulo, α es una constante de normalización usada para hacer que las probabilidades sumen hasta 1. Ahora insertamos una expresión para la predicción de un paso

[3] En particular, cuando se rastrea un objeto en movimiento con observaciones de posición inexactas, el suavizado proporciona una trayectoria estimada más suave que el filtrado, de ahí el nombre.

$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t})$, obtenida condicionando en el estado actual $\mathbf{X}_t$. La ecuación resultante para la nueva estimación del estado es el resultado central en este capítulo:

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) &= \alpha\mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})\sum_{\mathbf{x}_t}\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t,\mathbf{e}_{1:t})P(\mathbf{x}_t|\mathbf{e}_{1:t}) \\ &= \alpha\underbrace{\mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})}_{\text{modelo de sensor}}\sum_{\mathbf{x}_t}\underbrace{\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t)}_{\text{modelo de transición}}\underbrace{P(\mathbf{x}_t|\mathbf{e}_{1:t})}_{\text{recursión}} \qquad \text{(Suposición de Markov).} \qquad (14.5)\end{aligned}$$

En esta expresión, todos los términos provienen del modelo o de la estimación del estado anterior. Por lo tanto, tenemos la formulación recursiva deseada. Podemos pensar en la estimación filtrada $\mathbf{P}(\mathbf{X}_t|\mathbf{e}_{1:t})$ como un "mensaje" $\mathbf{f}_{1:t}$ que se propaga hacia adelante a lo largo de la secuencia, que es modificado por cada transición y actualizado por cada nueva observación. El proceso está dado por

$$\mathbf{f}_{1:t+1} = \text{FORWARD}(\mathbf{f}_{1:t},\mathbf{e}_{t+1}),$$

donde FORWARD implementa la actualización descrita en la Ecuación (14.5) y el proceso comienza con $\mathbf{f}_{1:0} = \mathbf{P}(\mathbf{X}_0)$. Cuando todas las variables de estado son discretas, el tiempo para cada actualización es constante (es decir, independiente de t), y el espacio requerido también es constante. (Las constantes dependen, por supuesto, del tamaño del espacio de estado y del tipo específico del modelo temporal en cuestión). *El tiempo y el espacio requeridos para la actualización deben ser constantes si un agente finito va a seguir la distribución del estado actual de manera indefinida.*

Ilustremos el proceso de filtrado para dos pasos en el ejemplo básico del paraguas (Figura 14.2). Es decir, calcularemos $\mathbf{P}(R_2|u_{1:2})$ de la siguiente manera:

- En el día 0, no tenemos observaciones, sólo las creencias previas del guardia de seguridad; supongamos que consiste en $\mathbf{P}(R_0) = \langle 0.5,0.5\rangle$.
- En el día 1, aparece el paraguas, así que U_1=*verdadero*. La predicción de t=0 a t=1 es

$$\begin{aligned}\mathbf{P}(R_1) &= \sum_{r_0}\mathbf{P}(R_1|r_0)P(r_0) \\ &= \langle 0.7,0.3\rangle \times 0.5 + \langle 0.3,0.7\rangle \times 0.5 = \langle 0.5,0.5\rangle.\end{aligned}$$

  Luego, el paso de actualización multiplica por la probabilidad de la evidencia para t=1 y normaliza, como se muestra en la Ecuación (14.4):

$$\begin{aligned}\mathbf{P}(R_1|u_1) &= \alpha\mathbf{P}(u_1|R_1)\mathbf{P}(R_1) = \alpha\langle 0.9,0.2\rangle\langle 0.5,0.5\rangle \\ &= \alpha\langle 0.45,0.1\rangle \approx \langle 0.818,0.182\rangle.\end{aligned}$$

- En el día 2, aparece el paraguas, así que U_2 = *verdadero*. La predicción de t = 1 a t = 2 es

$$\begin{aligned}\mathbf{P}(R_2|u_1) &= \sum_{r_1}\mathbf{P}(R_2|r_1)P(r_1|u_1) \\ &= \langle 0.7,0.3\rangle \times 0.818 + \langle 0.3,0.7\rangle \times 0.182 \approx \langle 0.627,0.373\rangle,\end{aligned}$$

  y actualizándolo con la evidencia para t=2 da

$$\begin{aligned}\mathbf{P}(R_2|u_1,u_2) &= \alpha\mathbf{P}(u_2|R_2)\mathbf{P}(R_2|u_1) = \alpha\langle 0.9,0.2\rangle\langle 0.627,0.373\rangle \\ &= \alpha\langle 0.565,0.075\rangle \approx \langle 0.883,0.117\rangle.\end{aligned}$$

Intuitivamente, la probabilidad de lluvia aumenta del día 1 al día 2 porque la lluvia persiste. El ejercicio 14.CONV(a) le pide que investigue esta tendencia más a fondo.

La tarea de **predicción** puede verse sólo como un filtrado sin la adición de nueva evidencia. De hecho, el proceso de filtrado ya incorpora una predicción de un paso, y es fácil derivar el siguiente cálculo recursivo para predecir el estado en t+k+1 a partir de una predicción para t+k:

$$\mathbf{P}(\mathbf{X}_{t+k+1}|\mathbf{e}_{1:t}) = \sum_{\mathbf{x}_{t+k}} \underbrace{\mathbf{P}(\mathbf{X}_{t+k+1}|\mathbf{x}_{t+k})}_{\text{modelo de transición}} \underbrace{P(\mathbf{x}_{t+k}|\mathbf{e}_{1:t})}_{\text{recursión}}. \tag{14.6}$$

Note que este cálculo implica sólo el modelo de transición y no el modelo de sensor.

Es interesante considerar qué sucede a medida que intentamos predecir cada vez más en el futuro. Como muestra el Ejercicio 14.CONV(b), la distribución predicha para la lluvia converge a un punto fijo $\langle 0.5,0.5\rangle$, después del cual permanece constante para todo el tiempo.[4] Esta es la **distribución estacionaria** del proceso de Markov definido por el modelo de transición. (Consulte la página 444). Se conoce mucho sobre las propiedades de tales distribuciones y sobre el **tiempo de mezcla** aproximado, el tiempo que se tarda en llegar al punto fijo. En términos prácticos, esto condena al fracaso cualquier intento de predecir el estado *actual* para un número de pasos que es más que una pequeña fracción del tiempo de mezcla, a menos que la distribución estacionaria en sí misma esté concentrada en una pequeña área del espacio de estados. Cuanta más incertidumbre haya en el modelo de transición, más corto será el tiempo de mezcla y más se oscurecerá el futuro.

Tiempo de mezcla

Además de filtrar y predecir, podemos usar una recursión hacia adelante para calcular la **probabilidad** de la secuencia de evidencia, $P(\mathbf{e}_{1:t})$. Esta es una cantidad útil si queremos comparar diferentes modelos temporales que podrían haber producido la misma secuencia de evidencia (por ejemplo, dos modelos diferentes para la persistencia de la lluvia). Para esta recursión, usamos un mensaje de probabilidad $\ell_{1:t}(\mathbf{X}_t) = \mathbf{P}(\mathbf{X}_t,\mathbf{e}_{1:t})$. Es fácil demostrar (Ejercicio 14.LIKL) que el cálculo del mensaje es idéntico al del filtrado:

$$\ell_{1:t+1} = \text{FORWARD}(\ell_{1:t},\mathbf{e}_{t+1}).$$

Luego de calcular $\ell_{1:t}$, obtenemos la probabilidad real sumando $\mathbf{X}_t$:

$$L_{1:t} = P(\mathbf{e}_{1:t}) = \sum_{\mathbf{x}_t} \ell_{1:t}(\mathbf{x}_t). \tag{14.7}$$

Observe que el mensaje de probabilidad representa las probabilidades de secuencias de evidencia cada vez más largas a medida que pasa el tiempo y, por lo tanto, se vuelve numéricamente más pequeño y más pequeño, lo que lleva a problemas de desbordamiento con la aritmética de punto flotante. Este es un problema importante en la práctica, pero no entraremos en soluciones aquí.

14.2.2 Suavizado

Como dijimos antes, el suavizado es el proceso de calcular la distribución sobre los estados pasados dados los datos hasta el presente, es decir $\mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:t})$ para $0 \le k < t$. (Ver Figura 14.3). En previsión de otro enfoque de paso de mensajes recursivos, podemos dividir el cálculo en dos partes, la evidencia hasta k y la evidencia de $k+1$ a t,

$$\begin{aligned}
\mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:t}) &= \mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:k},\mathbf{e}_{k+1:t}) \\
&= \alpha\mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:k})\mathbf{P}(\mathbf{e}_{k+1:t}|\mathbf{X}_k,\mathbf{e}_{1:k}) \qquad \text{(usando la regla de Bayes, dado } \mathbf{e}_{1:k}) \\
&= \alpha\mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:k})\mathbf{P}(\mathbf{e}_{k+1:t}|\mathbf{X}_k) \qquad \text{(usando independencia condicional)} \\
&= \alpha\mathbf{f}_{1:k} \times \mathbf{b}_{k+1:t}.
\end{aligned} \tag{14.8}$$

donde "×" representa la multiplicación punto a punto de vectores. Aquí hemos definido un mensaje "hacia atrás" $\mathbf{b}_{k+1:t} = \mathbf{P}(\mathbf{e}_{k+1:t}|\mathbf{X}_k)$, análogo al mensaje hacia adelante $\mathbf{f}_{1:k}$. El mensaje hacia adelante

[4] Si se elige un día arbitrario para ser t=0, entonces tiene sentido elegir la prioridad $\mathbf{P}(Lluvia_0)$ para que coincida con la distribución estacionaria, por lo que elegimos $\langle 0.5,0.5\rangle$ como la prioridad. Si hubiéramos elegido una prioridad diferente, la distribución estacionaria aún habría resultado en $\langle 0.5,0.5\rangle$.

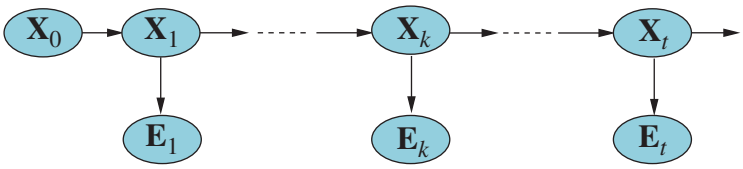


Figura 14.3 El suavizado calcula $\mathbf{P}(\mathbf{X}_k|\mathbf{e}_{1:t})$, la distribución posterior del estado en algún momento pasado k dada una secuencia completa de observaciones de 1 a t.

$\mathbf{f}_{1:k}$ se calcula filtrando hacia adelante de 1 a k, como se da en la Ecuación (14.5). Resulta que el mensaje hacia atrás $\mathbf{b}_{k+1:t}$ se calcula mediante un proceso recursivo que se ejecuta *hacia atrás* desde t:

$$\begin{aligned}
\mathbf{P}(\mathbf{e}_{k+1:t}|\mathbf{X}_k) &= \sum_{\mathbf{x}_{k+1}} \mathbf{P}(\mathbf{e}_{k+1:t}|\mathbf{X}_k,\mathbf{x}_{k+1})\mathbf{P}(\mathbf{x}_{k+1}|\mathbf{X}_k) \qquad \text{(condicionando en } \mathbf{X}_{k+1}\text{)} \\
&= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1:t}|\mathbf{x}_{k+1})\mathbf{P}(\mathbf{x}_{k+1}|\mathbf{X}_k) \qquad \text{(por independencia condicional)} \\
&= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1},\mathbf{e}_{k+2:t}|\mathbf{x}_{k+1})\mathbf{P}(\mathbf{x}_{k+1}|\mathbf{X}_k) \\
&= \sum_{\mathbf{x}_{k+1}} \underbrace{P(\mathbf{e}_{k+1}|\mathbf{x}_{k+1})}_{\text{modelo de sensor}}\underbrace{P(\mathbf{e}_{k+2:t}|\mathbf{x}_{k+1})}_{\text{recursión}}\underbrace{\mathbf{P}(\mathbf{x}_{k+1}|\mathbf{X}_k)}_{\text{modelo de transición}}, \qquad (14.9)
\end{aligned}$$

donde el último paso sigue por la independencia condicional de $\mathbf{e}_{k+1}$ y $\mathbf{e}_{k+2:t}$, dado $\mathbf{x}_{k+1}$. En esta expresión, todos los términos provienen del modelo o del mensaje hacia atrás anterior. Por lo tanto, tenemos la formulación recursiva deseada. En forma de mensaje,

$$\mathbf{b}_{k+1:t} = \text{BACKWARD}(\mathbf{b}_{k+2:t},\mathbf{e}_{k+1}),$$

donde BACKWARD implementa la actualización descrita en la Ecuación (14.9). Al igual que con la recursión hacia adelante, el tiempo y el espacio necesarios para cada actualización son constantes e independientes de t.

Ahora podemos ver que los dos términos en la Ecuación (14.8) se calculan mediante recursiones a través del tiempo, uno avanzando desde 1 hasta k y utilizando la ecuación de filtrado (14.5) y el otro retrocediendo desde t hasta $k + 1$ y usando la Ecuación (14.9).

Para la inicialización de la fase hacia atrás, tenemos $\mathbf{b}_{t+1:t} = \mathbf{P}(\mathbf{e}_{t+1:t}|\mathbf{X}_t) = \mathbf{P}(|\mathbf{X}_t) = \mathbf{1}$, donde $\mathbf{1}$ es un vector de 1s. La razón de esto es que $\mathbf{e}_{t+1:t}$ es una secuencia vacía, por lo que la probabilidad de observarla es 1.

Apliquemos ahora este algoritmo al ejemplo del paraguas, calculando la estimación suavizada para la probabilidad de lluvia en el tiempo $k = 1$, dadas las observaciones del paraguas en los días 1 y 2. Según la Ecuación (14.8), esto se da por

$$\mathbf{P}(R_1|u_1,u_2) = \alpha\mathbf{P}(R_1|u_1)\mathbf{P}(u_2|R_1). \qquad (14.10)$$

El primer término ya sabemos que es $\langle .818,.182\rangle$, del proceso de filtrado hacia adelante descrito antes. El segundo término se calcula aplicando la recursión hacia atrás en la Ecuación (14.9):

$$\begin{aligned}
\mathbf{P}(u_2|R_1) &= \sum_{r_2} \boldsymbol{P}(u_2|r_2)P(\,|r_2)\mathbf{P}(r_2|R_1) \\
&= (0.9 \times 1 \times \langle 0.7,0.3\rangle) + (0.2 \times 1 \times \langle 0.3,0.7\rangle) = \langle 0.69,0.41\rangle.
\end{aligned}$$

function FORWARD-BACKWARD(**ev**,*prior*) **returns** un vector de distribuciones de probabilidad
  inputs: **ev**, un vector de valores de evidencia para los pasos 1,..., t
    prior, la distribución previa en el estado inicial, $\mathbf{P}(\mathbf{X}_0)$
  local variables: **fv**, un vector de mensajes hacia adelante para pasos 0, ..., t
    b, una representación del mensaje hacia atrás, inicialmente todos los 1*s*
    sv, un vector de estimaciones suavizadas para pasos 1, ..., t

  fv[0]←*prior*
  for i= 1 **to** t **do**
    fv[i]←FORWARD(**fv**[i–1],**ev**[i])
  for i=t **down to** 1 **do**
    sv[i]←NORMALIZE(**fv**[i]×**b**)
    b←BACKWARD(**b**,**ev**[i])
  return sv

Figura 14.4 El algoritmo forward–backward para suavizar: calcular las probabilidades posteriores de una secuencia de estados dada una secuencia de observaciones. Los operadores FORWARD y BACKWARD están definidos por las Ecuaciones (14.5) y (14.9), respectivamente.

Introduciendo esto en la Ecuación (14.10), encontramos que la estimación suavizada para la lluvia en el día 1 es

$$\mathbf{P}(R_1|u_1,u_2) = \alpha\langle 0.818,0.182\rangle \times \langle 0.69,0.41\rangle \approx \langle 0.883,0.117\rangle.$$

Por lo tanto, la estimación suavizada para la lluvia en el día 1 es mayor que la estimación filtrada (0.818) en este caso. Esto se debe a que el paraguas en el día 2 hace más probable que haya llovido en el día 2; a su vez, como la lluvia tiende a persistir, eso hace más probable que haya llovido en el día 1.

Tanto las recursiones hacia adelante como hacia atrás toman una cantidad constante de tiempo por paso; por lo tanto, la complejidad temporal del suavizado con respecto a la evidencia $\mathbf{e}_{1:t}$ es $O(t)$. Esta es la complejidad para suavizar en un paso de tiempo particular k. Si queremos suavizar toda la secuencia, un método obvio es sólo ejecutar todo el proceso de suavizado una vez para cada paso de tiempo a suavizar. Esto resulta en una complejidad temporal de $O(t^2)$.

Un mejor enfoque utiliza una simple aplicación de programación dinámica para reducir la complejidad a $O(t)$. Una pista aparece en el análisis anterior del ejemplo del paraguas, donde pudimos reutilizar los resultados de la fase de filtrado hacia adelante. La clave para el algoritmo en tiempo lineal es *grabar los resultados* del filtrado hacia adelante en toda la secuencia. Luego ejecutamos la recursión hacia atrás desde t hasta 1, calculando la estimación suavizada en cada paso k a partir del mensaje hacia atrás calculado $\mathbf{b}_{k+1:t}$ y el mensaje hacia adelante almacenado $\mathbf{f}_{1:k}$. El algoritmo, llamado de manera apropiada **algoritmo forward–backward**, se muestra en la Figura 14.4.

Algoritmo forward–backward

El lector alerta habrá notado que la estructura de la red bayesiana mostrada en la Figura 14.3 es un *poli*árbol como se define en la página 433. Esto significa que una aplicación directa del algoritmo de agrupamiento también produce un algoritmo de tiempo lineal que calcula estimaciones suavizadas para toda la secuencia. Ahora se entiende que el algoritmo de avance-retroceso es en realidad un caso especial del algoritmo de propagación de poliárbol utilizado con métodos de agrupamiento (aunque los dos se desarrollaron de forma independiente).

El algoritmo forward–backward forma la columna vertebral computacional para muchas aplicaciones que tratan con secuencias de observaciones ruidosas. Tal como se ha descrito hasta

ahora, tiene dos inconvenientes prácticos. El primero es que su complejidad espacial puede ser demasiado alta cuando el espacio de estados es grande y las secuencias son largas. Utiliza el espacio $O(|\mathbf{f}|t)$ donde $|\mathbf{f}|$ es el tamaño de la representación del mensaje adelante. El requisito de espacio se reduce a $O(|\mathbf{f}|\log t)$ con un aumento concomitante en la complejidad del tiempo por un factor de $\log t$, como se muestra en el Ejercicio 14.ISLE. En algunos casos (ver Sección 14.3), es posible utilizar un algoritmo de espacio constante.

El segundo inconveniente del algoritmo básico es que necesita ser modificado para trabajar en un entorno *online* donde se deben calcular estimaciones suavizadas para las secciones de tiempo anteriores a medida que se agregan de manera continua nuevas observaciones al final de la secuencia. El requisito más común es para el **suavizado de retardo fijo**, que requiere calcular la estimación suavizada $\mathbf{P}(\mathbf{X}_{t-d}|\mathbf{e}_{1:t})$ suavizado de retardo fijo para d fijo. Es decir, el suavizado se realiza para la sección de tiempo d pasos detrás del tiempo actual t; a medida que t aumenta, el suavizado tiene que mantenerse. Note que es posible ejecutar el algoritmo forward–backward sobre la "ventana temporal deslizante" de d pasos a medida que se agrega cada nueva observación, pero esto parece ineficiente. En la Sección 14.3, veremos que el suavizado de retardo fijo puede, en algunos casos, hacerse en tiempo constante por actualización, independiente del retardo d.

Suavizado de retardo fijo

14.2.3 Encontrar la secuencia más probable

Supongamos que [*verdadero*,*verdadero*,*falso*,*verdadero*,*verdadero*] es la secuencia de paraguas observada para los primeros cinco días del guardia de seguridad en el trabajo. ¿Qué secuencia de clima es más probable que explique esto? ¿La ausencia del paraguas en el día 3 significa que no estaba lloviendo, o el director olvidó traerlo? Si no llovió el día 3, quizá (porque el clima tiende a persistir) no llovió el día 4 tampoco, pero el director trajo el paraguas por si acaso. En total, hay 2^5 posibles secuencias de clima que podríamos elegir. ¿Hay alguna forma de encontrar la más probable, sin enumerar todas y calcular sus probabilidades?

Podríamos intentar este procedimiento de tiempo lineal: usar el suavizado para encontrar la distribución posterior del clima en cada paso de tiempo; luego construir la secuencia, usando en cada paso el clima que es más probable de acuerdo con el posterior. Tal enfoque debería hacer sonar las alarmas en la cabeza del lector, porque las distribuciones posteriores calculadas por el suavizado son distribuciones sobre pasos de tiempo *individuales*, mientras que para encontrar la *secuencia* más probable debemos considerar las probabilidades *conjuntas* en todos los pasos de tiempo. Los resultados pueden ser bastante diferentes. (Ver Ejercicio 14.VITE.)

Existe un algoritmo de tiempo lineal para encontrar la secuencia más probable, pero requiere más reflexión. Se basa en la misma propiedad de Markov que produjo algoritmos eficientes para el filtrado y el suavizado. La idea es ver cada secuencia como un *camino* a través de un grafo cuyos nodos son los *estados* posibles en cada paso de tiempo. Tal grafo se muestra para el mundo del paraguas en la Figura 14.5(a). Ahora considere la tarea de encontrar el camino más probable a través de este grafo, donde la probabilidad de cualquier camino es el producto de las probabilidades de transición a lo largo del camino y las probabilidades de las observaciones dadas en cada estado.

Concentrémonos en particular en los caminos que llegan al estado $Lluvia_5$=*verdadero*. Debido a la propiedad de Markov, se deduce que el camino más probable al estado $Lluvia_5$=*verdadero* consiste en el camino más probable a *algún* estado en el tiempo 4 seguido de una transición a $Lluvia_5$ = *verdadero*; y el estado en el tiempo 4 que se convertirá en parte del camino a $Lluvia_5$ = *verdadero* es el que maximiza la probabilidad de ese camino. En otras palabras, *existe una relación recursiva entre los caminos más probables a cada estado* $\mathbf{x}_{t+1}$ *y los caminos* más probables a cada estado $\mathbf{x}_t$.

Es posible usar esta propiedad a fin de construir un algoritmo recursivo para calcular el camino más probable dada la evidencia. Usaremos un mensaje $m_{1:t}$ calculado recursivamente,

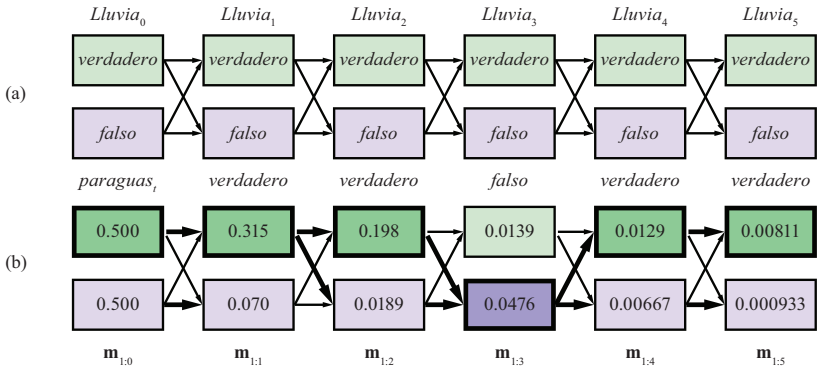


Figura 14.5 (a) Las posibles secuencias de estado para $Lluvia_t$ se pueden ver como caminos a través de un grafo de los posibles estados en cada paso de tiempo. (Los estados se muestran como rectángulos para evitar confusiones con los nodos en una red Bayes). (b) Operación del algoritmo de Viterbi para la secuencia de observación de paraguas [*verdadero*,*verdadero*,*falso*,*verdadero*,*verdadero*], donde la evidencia comienza en el tiempo 1. Para cada t, hemos mostrado los valores del mensaje $\mathbf{m}_{1:t}$, que da la probabilidad de la mejor secuencia llegando a cada estado en el tiempo t. Además, para cada estado, la flecha en negrita que conduce a él indica su mejor predecesor medido por el producto de la probabilidad de la secuencia precedente y la probabilidad de transición. Siguiendo las flechas en negrita desde el estado más probable en $\mathbf{m}_{1:5}$ da la secuencia más probable, mostrada por los contornos en negrita y el sombreado más oscuro.

como el mensaje forward $\mathbf{f}_{1:t}$ en el algoritmo de filtrado. El mensaje se define de la siguiente manera:[5]

$$\mathbf{m}_{1:t} = \max_{\mathbf{x}_{1:t-1}} \mathbf{P}(\mathbf{x}_{1:t-1}, \mathbf{X}_t, \mathbf{e}_{1:t}).$$

Para obtener la relación recursiva entre $\mathbf{m}_{1:t+1}$ y $\mathbf{m}_{1:t}$, podemos repetir más o menos los mismos pasos que usamos para la Ecuación (14.5):

$$\begin{aligned}
\mathbf{m}_{1:t+1} &= \max_{\mathbf{x}_{1:t}} \mathbf{P}(\mathbf{x}_{1:t}, \mathbf{X}_{t+1}, \mathbf{e}_{1:t+1}) = \max_{\mathbf{x}_{1:t}} \mathbf{P}(\mathbf{x}_{1:t}, \mathbf{X}_{t+1}, \mathbf{e}_{1:t}, e_{t+1}) \\
&= \max_{\mathbf{x}_{1:t}} \mathbf{P}(e_{t+1} | \mathbf{x}_{1:t}, \mathbf{X}_{t+1}, \mathbf{e}_{1:t}) \mathbf{P}(\mathbf{x}_{1:t}, \mathbf{X}_{t+1}, \mathbf{e}_{1:t}) \\
&= \mathbf{P}(e_{t+1} | \mathbf{X}_{t+1}) \max_{\mathbf{x}_{1:t}} \mathbf{P}(\mathbf{X}_{t+1}, | \mathbf{x}_t) P(\mathbf{x}_{1:t}, \mathbf{e}_{1:t}) \\
&= \mathbf{P}(e_{t+1} | \mathbf{X}_{t+1}) \max_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1}, | \mathbf{x}_t) \max_{\mathbf{x}_{1:t-1}} P(\mathbf{x}_{1:t-1}, \mathbf{x}_t, \mathbf{e}_{1:t})
\end{aligned} \tag{14.11}$$

donde el término final $\max_{\mathbf{x}_{1:t-1}} P(\mathbf{x}_{1:t-1}, \mathbf{x}_t, \mathbf{e}_{1:t})$ es la entrada para el estado particular $\mathbf{x}_t$ en el vector de mensaje $\mathbf{m}_{1:t}$. La Ecuación (14.11) es idéntica a la ecuación de filtrado (14.5) excepto que la suma sobre $\mathbf{x}_t$ en la Ecuación (14.5) es reemplazada por la maximización sobre $\mathbf{x}_t$ en la Ecuación (14.11), y no hay constante de normalización α en la Ecuación (14.11). Así, el algoritmo para calcular la secuencia más probable es similar al filtrado: comienza en el tiempo 0 con el prior

[5] Observe que estas no son las probabilidades de los caminos más probables para llegar a los estados $\mathbf{X}_t$ dada la evidencia, que serían las probabilidades condicionales $\max_{\mathbf{x}_{1:t-1}} \mathbf{P}(\mathbf{x}_{1:t-1}, \mathbf{X}_t | \mathbf{e}_{1:t})$; pero los dos vectores están relacionados por un factor constante $P(\mathbf{e}_{1:t})$. La diferencia es inmaterial porque el operador max no se preocupa por los factores constantes. Obtenemos una recursión más simple con $\mathbf{m}_{1:t}$ definido de esta manera.

$\mathbf{m}_{1:0} = \mathbf{P}(\mathbf{X}_0)$ y luego avanza a lo largo de la secuencia, calculando el **m** mensaje en cada paso de tiempo usando la Ecuación (14.11). El progreso de este cálculo se muestra en la Figura 14.5(b).

Al final de la secuencia de observación, $\mathbf{m}_{1:t}$ contendrá la probabilidad para la secuencia más probable que llega a cada uno de los estados finales. De este modo, uno puede seleccionar el estado final de la secuencia más probable en general (el estado delineado en negrita en el paso 5). En lugar de calcular su probabilidad, para identificar la secuencia real el algoritmo también necesitará registrar, para cada estado, el mejor estado que conduce a él; estos están indicados por las flechas en negrita en la Figura 14.5(b). La secuencia óptima se identifica siguiendo estas flechas en negrita hacia atrás desde el mejor estado final.

Algoritmo de Viterbi

El algoritmo que acabamos de describir se llama **algoritmo de Viterbi**, en honor a su inventor, Andrew Viterbi. Al igual que el algoritmo de filtrado, su complejidad temporal es lineal en t, la longitud de la secuencia. A diferencia del filtrado, que utiliza un espacio constante, su requerimiento de espacio también es lineal en t. Esto se debe a que el algoritmo de Viterbi necesita mantener los punteros que identifican la mejor secuencia que conduce a cada estado.

Un último punto práctico: el desbordamiento numérico es un problema significativo para el algoritmo de Viterbi. En la Figura 14.5(b), las probabilidades se están volviendo cada vez más pequeñas, y este es sólo un ejemplo ilustrativo. Las aplicaciones reales en el análisis de ADN o la decodificación de mensajes permiten tener miles o millones de pasos. Una posible solución es normalizar **m** en cada paso; observamos que este reescalado no afecta la corrección porque $\max(cx,cy)=c\cdot\max(x,y)$. Una segunda solución es usar logaritmos de probabilidades en todas partes y reemplazar la multiplicación por la suma. De nuevo, la corrección es inmutable porque la función log es monótona, por lo que $\max(\log x, \log y)=\log\max(x,y)$.

14.3 Modelos de Markov ocultos

La sección anterior desarrolló algoritmos para el razonamiento probabilístico temporal utilizando un marco general independiente de la forma específica de los modelos de transición y sensor, e independiente de la naturaleza de las variables de estado y evidencia. En esta y las próximas dos secciones, discutimos modelos y aplicaciones más concretos que ilustran el poder de los algoritmos básicos y, en algunos casos, permiten mejoras adicionales.

Modelo de Markov oculto

Comenzamos con el **modelo de Markov oculto**, o **HMM**. Un HMM es un modelo probabilístico temporal en el que el estado del proceso se describe mediante una única variable aleatoria discreta. Los posibles valores de la variable son los posibles estados del mundo. El ejemplo del paraguas descrito en la sección anterior es, por lo tanto, un HMM, ya que tiene sólo una variable de estado: $Lluvia_t$. ¿Qué sucede si tiene un modelo con dos o más variables de estado? Aún es posible ajustarlo al marco de HMM combinando las variables en una única "megavariable" cuyos valores son todas las posibles tuplas de valores de las variables de estado individuales. Veremos que la estructura restringida de los HMM permite una implementación matricial simple y elegante de todos los algoritmos básicos.[6]

Aunque los HMM requieren que el *estado* sea una única variable discreta, no hay ninguna restricción correspondiente en las variables de *evidencia*. Esto se debe a que las variables de evidencia siempre se observan, lo que significa que no es necesario realizar un seguimiento de ninguna distribución sobre sus valores. (Si una variable no se observa, se elimina del modelo para ese paso de tiempo). Existen múltiples variables de evidencia, tanto discretas como continuas.

[6] El lector no familiarizado con las operaciones básicas en vectores y matrices puede consultar el Apéndice A antes de continuar con esta sección.

14.3.1 Algoritmos de matriz simplificados

Con una única variable de estado discreta X_t, es posible dar forma concreta a las representaciones del modelo de transición, el modelo de sensor y los mensajes forward–backward. Dejemos que la variable de estado X_t tenga valores denotados por los números enteros 1,...,S, donde S es el número de estados posibles. El modelo de transición $\mathbf{P}(X_t|X_{t-1})$ se convierte en una matriz $S\times S$ $\mathbf{T}$, donde

$$\mathbf{T}_{ij} = P(X_t=j|X_{t-1}=i).$$

Es decir, $\mathbf{T}_{ij}$ es la probabilidad de una transición del estado i al estado j. Por ejemplo, si numeramos los estados *Rain=true* y *Rain=false* como 1 y 2, respectivamente, entonces la matriz de transición para el mundo del paraguas definido en la Figura 14.2 es

$$\mathbf{T} = \mathbf{P}(X_t|X_{t-1}) = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}.$$

También ponemos el modelo del sensor en forma de matriz. En este caso, dado que el valor de la variable de evidencia E_t se conoce en el tiempo t (llámalo e_t), sólo necesitamos especificar, para cada estado, cuán probable es que el estado cause que e_t aparezca: necesitamos $P(e_t|X_t=i)$ para cada estado i. Por conveniencia matemática, colocamos estos valores en una **matriz de observación** diagonal $S\times S$, $\mathbf{O}_t$, una para cada paso de tiempo. La entrada diagonal ith de $\mathbf{O}_t$ es $P(e_t|X_t=i)$ y las otras entradas son 0. Por ejemplo, en el día 1 en el mundo del paraguas de la Figura 14.5, U_1=*true*, y en el día 3, U_3=*false*, por lo que tenemos

Matriz de observación

$$\mathbf{O}_1 = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix}; \qquad \mathbf{O}_3 = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.8 \end{pmatrix}.$$

Ahora, si usamos vectores de columna para representar los mensajes forward–backward, todos los cálculos se convierten en simples operaciones de matriz-vector. La ecuación hacia adelante (forward) (14.5) se convierte en

$$\mathbf{f}_{1:t+1} = \alpha \mathbf{O}_{t+1}\mathbf{T}^{\mathsf{T}}\mathbf{f}_{1:t} \tag{14.12}$$

y la ecuación hacia atrás (backward) (14.9) se convierte en

$$\mathbf{b}_{k+1:t} = \mathbf{T}\mathbf{O}_{k+1}\mathbf{b}_{k+2:t}. \tag{14.13}$$

A partir de estas ecuaciones, podemos ver que la complejidad temporal del algoritmo forward–backward (Figura 14.4) aplicado a una secuencia de longitud t es $O(S^2t)$, porque cada paso requiere multiplicar un vector de S elementos por una matriz $S\times S$. El requisito de espacio es $O(St)$, porque el paso hacia adelante almacena t vectores de tamaño S.

Además de proporcionar una descripción elegante de los algoritmos de filtrado y suavizado para HMM, la formulación matricial revela oportunidades para algoritmos mejorados. El primero es una simple variación del algoritmo forward–backward que permite realizar el suavizado en un espacio *constante*, independiente de la longitud de la secuencia. La idea es que el suavizado para cualquier segmento de tiempo particular k requiere la presencia simultánea de ambos mensajes, hacia adelante y hacia atrás (forward-backward), $\mathbf{f}_{1:k}$ y $\mathbf{b}_{k+1:t}$, de acuerdo con la Ecuación (14.8). El algoritmo forward–backward logra esto almacenando los **f**s calculados en el paso hacia adelante para que estén disponibles durante el paso hacia atrás. Otra forma de lograr esto es con un solo paso que propaga tanto **f** como **b** en la misma dirección. Por ejemplo, el mensaje "hacia adelante" **f** se propaga hacia atrás si manipulamos la Ecuación (14.12) para trabajar en la otra dirección:

$$\mathbf{f}_{1:t} = \alpha'(\mathbf{T}^{\mathsf{T}})^{-1}\mathbf{O}_{t+1}^{-1}\mathbf{f}_{1:t+1}.$$

El algoritmo de suavizado modificado funciona primero ejecutando el paso hacia adelante estándar para calcular $\mathbf{f}_{t:t}$ (olvidando todos los resultados intermedios) y luego ejecutando el paso

function FIXED-LAG-SMOOTHING(e_t,*hmm*,d) **returns** una distribución a través de $\mathbf{X}_{t-d}$
  inputs: e_t, la evidencia actual de la etapa de tiempo t
    hmm, un modelo de Markov oculto con $S\times S$ matriz de transición $\mathbf{T}$
    d, la longitud del retraso para suavizar
  persistent: t, la hora actual, inicialmente 1
    $\mathbf{f}$, el mensaje hacia adelante $\mathbf{P}(X_t|e_{1:t})$, inicialmente *hmm*.PRIOR
    $\mathbf{B}$, la matriz de transformación paso d hacia atrás, inicialmente la matriz de identidad
    $e_{t-d:t}$, lista de pruebas de doble extremo de $t-d$ a t, inicialmente vacía
  local variables: $\mathbf{O}_{t-d}$,$\mathbf{O}_t$, matrices diagonales que contienen la información del modelo del sensor

  agregar e_t al final de $e_{t-d:t}$
  $\mathbf{O}t \leftarrow$ matriz diagonal que contiene $\mathbf{P}(e_t|X_t)$
  If $t > d$ **then**
    $\mathbf{f} \leftarrow$FORWARD($\mathbf{f}$,e_{t-d})
    elimina e_{t-d-1} del inicio de $e_{t-d:t}$
    $\mathbf{O}_{t-d} \leftarrow$ matriz diagonal que contiene $\mathbf{P}(e_{t-d}|X_{t-d})$
    $\mathbf{B} \leftarrow \mathbf{O}^{-1}_{t-d}\mathbf{T}^{-1}\mathbf{B}\mathbf{T}\mathbf{O}_t$
  else $\mathbf{B} \leftarrow \mathbf{B}\mathbf{T}\mathbf{O}_t$
  $t \leftarrow t+1$
  if $t > d+1$ **then return** NORMALIZE($\mathbf{f} \times \mathbf{B1}$) **else return** null

Figura 14.6 Un algoritmo para suavizar con un retraso de tiempo fijo de d pasos, implementado como un algoritmo en línea que produce la nueva estimación suavizada dada la observación para un nuevo paso de tiempo. Observe que la salida final NORMALIZE($\mathbf{f}\times\mathbf{B1}$) es sólo $\alpha\mathbf{f}\times\mathbf{b}$, por la Ecuación (14.14).

hacia atrás para ambos **b** y **f** juntos, utilizándolos para calcular la estimación suavizada en cada paso. Dado que sólo se necesita una copia de cada mensaje, los requisitos de almacenamiento son constantes (es decir, independientes de t, la longitud de la secuencia). Hay dos restricciones significativas en este algoritmo: requiere que la matriz de transición sea invertible y que el modelo de sensor no tenga ceros, es decir, que cada observación sea posible en cada estado.

Una segunda área en la que la formulación matricial revela una mejora es en el suavizado en línea con un retraso fijo. El hecho de que el suavizado se pueda hacer en un espacio constante sugiere que debería existir un algoritmo recursivo eficiente para el suavizado en línea, es decir, un algoritmo cuya complejidad de tiempo es independiente de la longitud del retraso. Supongamos que el retraso es d; es decir, estamos suavizando en el intervalo de tiempo $t-d$, donde el tiempo actual es t. Por la Ecuación (14.8), necesitamos calcular

$$\alpha\mathbf{f}_{1:t-d} \times \mathbf{b}_{t-d+1:t}$$

para el intervalo $t-d$. Luego, cuando llega una nueva observación, necesitamos calcular

$$\alpha\mathbf{f}_{1:t-d+1} \times \mathbf{b}_{t-d+2:t+1}$$

para el intervalo $t-d+1$. ¿Cómo se hace esto de manera incremental? Primero, se calcula $\mathbf{f}_{1:t-d+1}$ a partir de $\mathbf{f}_{1:t-d}$, utilizando el proceso de filtrado estándar, Ecuación (14.5).

Calcular el mensaje hacia atrás de manera incremental es más complicado, porque no existe una relación simple entre el antiguo mensaje hacia atrás $\mathbf{b}_{t-d+1:t}$ y el nuevo mensaje hacia atrás $\mathbf{b}_{t-d+2:t+1}$. En cambio, examinaremos la relación entre el antiguo mensaje hacia atrás $\mathbf{b}_{t-d+1:t}$ y el mensaje hacia atrás al frente de la secuencia, $\mathbf{b}_{t+1:t}$. Para hacer esto, aplicamos

Ecuación (14.13) d veces para obtener

$$\mathbf{b}_{t-d+1:t} = \left(\prod_{i=t-d+1}^{t} \mathbf{TO}_i \right) \mathbf{b}_{t+1:t} = \mathbf{B}_{t-d+1:t}\mathbf{1}, \tag{14.14}$$

donde la matriz $\mathbf{B}_{t-d+1:t}$ es el producto de la secuencia de matrices $\mathbf{T}$ y $\mathbf{O}$ y $\mathbf{1}$ es un vector de 1s. $\mathbf{B}$ puede considerarse como un "operador de transformación" que transforma un mensaje hacia atrás posterior en uno anterior. Una ecuación similar se mantiene para los nuevos mensajes hacia atrás *después* de que la próxima observación llega:

$$\mathbf{b}_{t-d+2:t+1} = \left(\prod_{i=t-d+2}^{t+1} \mathbf{TO}_i \right) \mathbf{b}_{t+2:t+1} = \mathbf{B}_{t-d+2:t+1}\mathbf{1}. \tag{14.15}$$

Examinando las expresiones de producto en las Ecuaciones (14.14) y (14.15), vemos que tienen una relación simple: para obtener el segundo producto, "divida" el primer producto por el primer elemento $\mathbf{TO}_{t-d+1}$, y multiplique por el nuevo último elemento $\mathbf{TO}_{t+1}$. En lenguaje matricial, por lo tanto, hay una relación simple entre las antiguas y nuevas matrices $\mathbf{B}$:

$$\mathbf{B}_{t-d+2:t+1} = \mathbf{O}_{t-d+1}^{-1}\mathbf{T}^{-1}\mathbf{B}_{t-d+1:t}\mathbf{TO}_{t+1}. \tag{14.16}$$

Esta ecuación proporciona una actualización incremental para la matriz $\mathbf{B}$ que, a su vez (a través de la ecuación (14.15)), nos permite calcular el nuevo mensaje hacia atrás $\mathbf{b}_{t-d+2:t+1}$. El algoritmo completo, que requiere almacenar y actualizar $\mathbf{f}$ y $\mathbf{B}$, se muestra en la Figura 14.6.

14.3.2 Ejemplo de modelo de Markov oculto: Localización

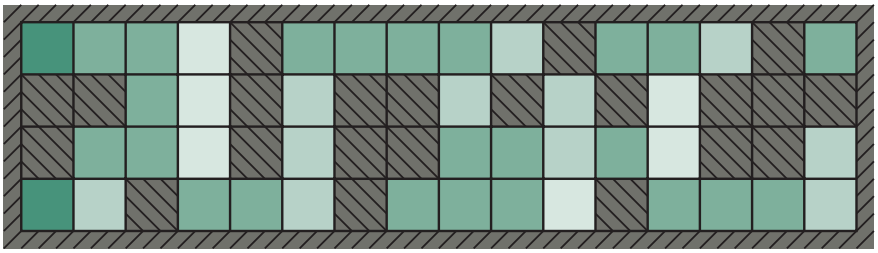
En la página 133, introdujimos una forma simple del problema de **localización** para el mundo del vacío. En esa versión, el robot tenía una única acción de *Movimiento* no determinista y sus sensores informaban si había o no obstáculos inmediatamente al norte, sur, este y oeste; el estado de creencia del robot era el conjunto de posibles ubicaciones en las que podría estar.

Aquí hacemos el problema un poco más realista permitiendo ruido en los sensores, y formalizando la idea de que el robot se mueve de manera aleatoria; es decir, es igualmente probable que se mueva a cualquier casilla vacía adyacente. La variable de estado X_t representa la ubicación del robot en la retícula discreta; el dominio de esta variable es el conjunto de casillas vacías, que etiquetaremos con los enteros $\{1,...,S\}$. Sea NEIGHBORS(i) el conjunto de casillas vacías adyacentes a i y $N(i)$ el tamaño de ese conjunto. Entonces el modelo de transición para la acción de *Movimiento* indica que es igualmente probable que el robot termine en cualquier casilla vecina:

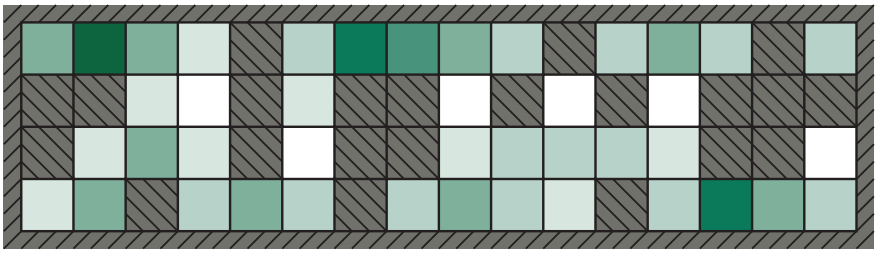
$$P(X_{t+1}=j|X_t=i)=\mathbf{T}_{ij} = \begin{cases} 1/N(i) \text{ if } j \in \text{NEIGHBORS}(i) \\ 0 \text{ otherwise.} \end{cases}$$

No sabemos dónde comienza el robot, así que asumiremos una distribución uniforme sobre todas las casillas; es decir, $P(X_0=i)=1/S$. Para el entorno particular que consideramos (Figura 14.7), S=42 y la matriz de transición $\mathbf{T}$ tiene 42×42 = 1 764 entradas.

La variable del sensor E_t tiene 16 posibles valores, cada uno con una secuencia de cuatro bits que indica la presencia o ausencia de un obstáculo en cada una de las direcciones de la brújula NESW. Por ejemplo, 1010 significa que los sensores del norte y del sur informan de un obstáculo y los del este y el oeste no. Supongamos que la tasa de error de cada sensor es ε y que los errores ocurren de manera independiente para las cuatro direcciones del sensor. En ese caso, la probabilidad de acertar los cuatro bits es $(1-\varepsilon)^4$ y la probabilidad de fallarlos todos es ε^4. Además, si d_{it} es la discrepancia, el número de bits que son diferentes, entre los valores verdaderos para la casilla i



(a) Distribución posterior sobre la ubicación del robot después de E_1=1011



(b) Distribución posterior sobre la ubicación del robot después de E_1=1011, E_2=1010

Figura 14.7 Distribución posterior sobre la ubicación del robot: (a) después de una observación E_1=1011 (es decir, obstáculos al norte, sur y oeste); (b) después de un movimiento aleatorio a una ubicación adyacente y una segunda observación E_2=1010 (es decir, obstáculos al norte y sur). El tamaño de cada disco corresponde a la probabilidad de que el robot esté en esa ubicación. La tasa de error del sensor para cada bit es ε= 0.2.

y la lectura real e_t, entonces la probabilidad de que un robot en la casilla i reciba una lectura de sensor e_t es

$$P(E_t = e_t|X_t=i)=(\mathbf{O}_t)_{ii}=(1-\varepsilon)^{4-d_{it}}\varepsilon^{d_{it}}.$$

Por ejemplo, la probabilidad de que una casilla con obstáculos al norte y al sur produzca una lectura de sensor 1110 es $(1-\varepsilon)^3\varepsilon^1$.

Dadas las matrices $\mathbf{T}$ y $\mathbf{O}_t$, el robot usa la Ecuación (14.12) para calcular la posterior distribución sobre las ubicaciones, es decir, para averiguar dónde está. La Figura 14.7 muestra las distribuciones $\mathbf{P}(X_1|E_1$=1011) y $\mathbf{P}(X_2|E_1$=1011,E_2=1010). Este es el mismo laberinto que vimos antes en la Figura 4.18 (página 134), pero allí usamos el filtrado lógico para encontrar las ubicaciones que eran *posibles*, asumiendo una detección perfecta. Esas mismas ubicaciones siguen siendo las más *probables* con la detección ruidosa, pero ahora *cada* ubicación tiene alguna probabilidad distinta de cero porque cualquier ubicación podría producir cualquier valor de sensor.

Además de filtrar para estimar su ubicación actual, el robot usa el suavizado (Ecuación (14.13)) para averiguar dónde estaba en cualquier momento pasado, por ejemplo, dónde comenzó en el tiempo 0, y usa el algoritmo de Viterbi para averiguar el camino más probable que ha tomado para llegar a donde está ahora. La Figura 14.8 muestra el error de localización y el error de ruta de Viterbi para varios valores de la tasa de error del sensor por bit ε. Aun cuando ε es 0.20, lo que significa que la lectura total del sensor es incorrecta 59 % del tiempo, el robot suele ser capaz de averiguar su locación a dos casillas después de 20 observaciones. Esto se debe a la capacidad del

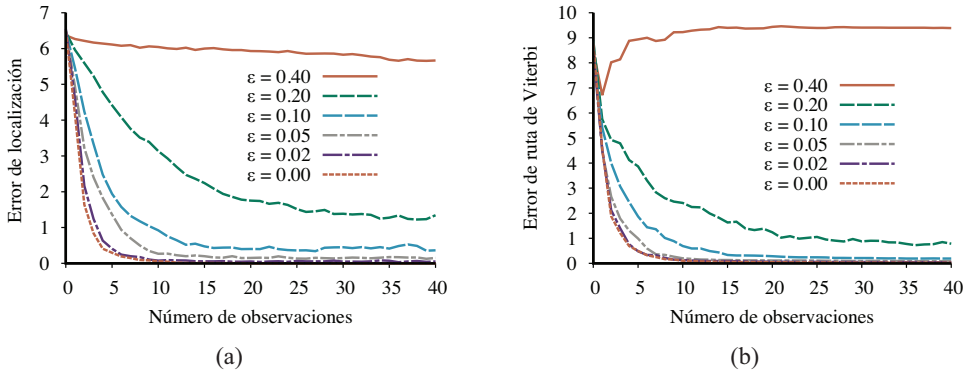


Figura 14.8 Rendimiento de la localización HMM en función de la longitud de la secuencia de observación para varios valores diferentes de la probabilidad de error del sensor ε; datos promediados en 400 carreras. (a) El error de localización, definido como la distancia de Manhattan desde la ubicación verdadera. (b) El error de ruta de Viterbi, definido como la distancia media de Manhattan de los estados en la ruta de Viterbi desde los estados correspondientes en la ruta verdadera.

algoritmo para integrar la evidencia a lo largo del tiempo y tener en cuenta las restricciones probabilísticas impuestas en la secuencia de ubicación por el modelo de transición. Cuando ε es 0.10 o menos, el robot necesita apenas unas observaciones para averiguar dónde está y para rastrear su posición con precisión. Cuando ε es 0.40, tanto el error de localización como el error de ruta de Viterbi permanecen grandes; en otras palabras, el robot está perdido. Esto se debe a que un sensor con una probabilidad de error de 0.40 proporciona muy poca información para contrarrestar la pérdida de información sobre la posición del robot que proviene del movimiento aleatorio impredecible.

La variable de estado para el ejemplo que hemos considerado en esta sección es una ubicación física en el mundo. Otros problemas, incluyen diferentes aspectos del mundo. El ejercicio 14.ROOM le pide que considere una versión del robot aspirador que tiene la política de ir recto tanto como pueda; sólo cuando encuentra un obstáculo cambia a una nueva dirección. Para modelar este robot, cada estado en el modelo consta de un par (*ubicación*, *dirección*). Para el entorno en la Figura 14.7, que tiene 42 casillas vacías, esto lleva a 168 estados y una matriz de transición con $168^2 = 28\,224$ entradas, todavía un número manejable.

Si agregamos la posibilidad de suciedad en cada una de las 42 casillas vacías, el número de estados se multiplica por 2^{42} y la matriz de transición tiene más de 10^{29} entradas, ya no es un número manejable. En general, si el estado está compuesto por n variables discretas con a lo sumo d valores cada una, la matriz de transición HMM correspondiente tendrá tamaño $O(d^{2n})$ y el tiempo de cálculo por actualización también será $O(d^{2n})$.

Por estas razones, aunque los HMM tienen muchos usos en áreas que van desde el reconocimiento de voz hasta la biología molecular, están por definición limitados en su capacidad para representar procesos complejos. En la terminología introducida en el Capítulo 2, los HMM son una representación atómica: los estados del mundo no tienen estructura interna y están etiquetados por enteros. La Sección 14.5 muestra cómo usar redes bayesianas dinámicas, una representación factorizada, para modelar dominios con muchas variables de estado. La siguiente sección muestra cómo manejar dominios con variables de estado continuas que, por supuesto, conducen a un espacio de estado infinito.

14.4 Filtros de Kalman

Imagine observar a un pequeño pájaro volando a través del denso follaje de la jungla al anochecer: vislumbra breves e intermitentes destellos de movimiento; intenta adivinar dónde está el pájaro y dónde aparecerá después para no perderlo. O imagine que es un operador de radar de la Segunda Guerra Mundial mirando un débil y errante blip que aparece en la pantalla una vez cada 10 segundos. O, yendo aún más atrás, imagine que es Kepler quien trata de reconstruir los movimientos de los planetas a partir de una colección de observaciones angulares muy inexactas, tomadas en intervalos irregulares y medidos de manera imprecisa.

En todos estos casos, está haciendo filtrado: estimando variables de estado (aquí, la posición y velocidad de un objeto en movimiento) a partir de observaciones ruidosas a lo largo del tiempo. Si las variables fueran discretas, podríamos modelar el sistema con un modelo de Markov oculto. Esta sección examina métodos para manejar variables continuas, usando un algoritmo llamado **filtrado de Kalman**, a partir de uno de sus inventores, Rudolf Kalman.

Filtrado de Kalman

El vuelo del pájaro podría estar especificado por seis variables continuas en cada punto de tiempo; tres para la posición (X_t, Y_t, Z_t) y tres para la velocidad $(\dot{X}_t, \dot{Y}_t, \dot{Z}_t)$. Necesitaremos densidades condicionales adecuadas para representar los modelos de transición y sensor; como en el Capítulo 13, utilizaremos distribuciones **lineales gaussianas**. Esto significa que el siguiente estado $\mathbf{X}_{t+1}$ debe ser una función lineal del estado actual $\mathbf{X}_t$, más algún ruido Gaussiano, una condición bastante razonable en práctica. Considere, por ejemplo, la coordenada X del pájaro, ignorando las otras coordenadas por ahora. Deje que el intervalo de tiempo entre observaciones sea Δ, y asuma una velocidad constante durante el intervalo; entonces la actualización de la posición está dada por $X_{t+\Delta} = X_t + \dot{X}\Delta$. Agregando ruido Gaussiano (para tener en cuenta la variación del viento, etc.), obtenemos un modelo de transición lineal-Gaussiano:

$$P(X_{t+\Delta} = x_{t+\Delta} | X_t = x_t, \dot{X}_t = \dot{x}_t) = N(x_{t+\Delta}; x_t + \dot{x}_t\Delta, \sigma^2).$$

La estructura de la red bayesiana para un sistema con vector de posición $\mathbf{X}_t$ y velocidad $\dot{\mathbf{X}}_t$ se muestra en la Figura 14.9. Note que esta es una forma muy específica de modelo lineal-Gaussiano; la forma general será descrita más adelante en esta sección y cubre una amplia gama de aplicaciones, más allá de los simples ejemplos de movimiento del primer párrafo. El lector podría consultar algunas de las propiedades matemáticas de las distribuciones gaussianas en el Apéndice A; para nuestros propósitos inmediatos, lo más importante es que una distribución **gaussiana multivariante** para d variables está especificada por una media μ de d elementos y una matriz de covarianza Σ de $d \times d$.

14.4.1 Actualizando distribuciones gaussianas

En el Capítulo 13 en la página 423, aludimos a una propiedad clave de la familia de distribuciones lineales-gaussianas: permanece cerrada bajo la actualización bayesiana. (Es decir, dada cualquier evidencia, la posterior sigue estando en la familia lineal-gaussiana). Aquí hacemos esta afirmación precisa en el contexto del filtrado en un modelo de probabilidad temporal. Las propiedades requeridas corresponden al cálculo de filtrado en dos pasos en la Ecuación (14.5):

1. Si la distribución actual $\mathbf{P}(\mathbf{X}_t | \mathbf{e}_{1:t})$ es gaussiana y el modelo de transición $\mathbf{P}(\mathbf{X}_{t+1} | \mathbf{x}_t)$ es lineal-gaussiano, entonces la distribución predicha en un paso dada por

$$\mathbf{P}(\mathbf{X}_{t+1} | \mathbf{e}_{1:t}) = \int_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} | \mathbf{x}_t) P(\mathbf{x}_t | \mathbf{e}_{1:t}) d\mathbf{x}_t \tag{14.17}$$

también es una distribución gaussiana.

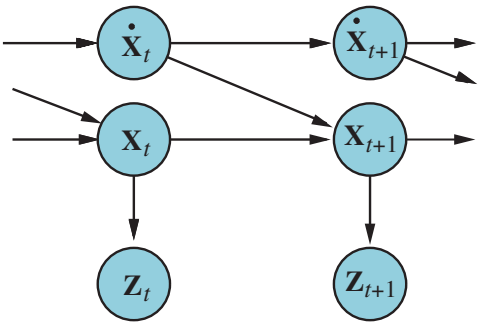


Figura 14.9 Estructura de red bayesiana para un sistema dinámico lineal con posición $\mathbf{X}_t$, velocidad $\dot{\mathrm{X}}_t$, y medición de posición Zt.

2. Si la predicción $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t})$ es gaussiana y el modelo de sensor $\mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})$ es lineal-gaussiano, entonces, después de condicionar a la nueva evidencia, la distribución actualizada

$$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t+1}) = \alpha\mathbf{P}(\mathbf{e}_{t+1}|\mathbf{X}_{t+1})\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{e}_{1:t}) \tag{14.18}$$

también es una distribución gaussiana.

Por lo tanto, el operador FORWARD (hacia adelante) para el filtrado de Kalman toma un mensaje hacia adelante Gaussiano $\mathbf{f}_{1:t}$, especificado por una media μ_t y una covarianza Σ_t, que produce un nuevo mensaje hacia adelante Gaussiano multivariante $\mathbf{f}_{1:t+1}$, especificado por una media μ_{t+1} y una covarianza Σ_{t+1}. Así que si comenzamos con un Gaussiano prior $\mathbf{f}_{1:0}=\mathbf{P}(\mathbf{X}_0)=\mathcal{N}(\mu_0,\Sigma_0)$, el filtrado con un modelo lineal-Gaussiano produce una distribución gaussiana de estado para todo el tiempo.

Esto parece ser un resultado agradable y elegante, pero ¿por qué es tan importante? La razón es que excepto por unos pocos casos especiales como este, *el filtrado con redes continuas o híbridas (discretas y continuas) genera distribuciones de estado cuya representación crece sin límites con el tiempo*. Esta afirmación no es fácil de demostrar en general, pero el Ejercicio 14.KFSW muestra lo que sucede para un ejemplo simple.

14.4.2 Un simple ejemplo unidimensional

Hemos dicho que el operador FORWARD para el filtro de Kalman mapea un Gaussiano en un nuevo Gaussiano. Esto se traduce en calcular una nueva media y covarianza a partir de la media y covarianza previas. Derivar la regla de actualización en el caso general (multivariante) requiere bastante álgebra lineal, así que nos quedaremos con un caso univariante muy simple por ahora, y luego daremos los resultados para el caso general. Aun para el caso univariante, los cálculos son algo tediosos, pero creemos que vale la pena verlos porque la utilidad del filtro de Kalman está tan íntimamente ligada a las propiedades matemáticas de las distribuciones gaussianas.

El modelo temporal que consideramos describe un **paseo aleatorio** de una única variable de estado continua X_t con una observación ruidosa Z_t. Un ejemplo podría ser el índice de "confianza del consumidor", que permite ser modelado como experimentando un cambio aleatorio Gaussiano cada mes y es medido por una encuesta aleatoria al consumidor que también introduce ruido de muestreo Gaussiano. Se asume que la distribución a priori es gaussiana con varianza σ_0^2:

$$P(x_0)=\alpha e^{-\frac{1}{2}\left(\frac{(x_0-\mu_0)^2}{\sigma_0^2}\right)}.$$

(Por simplicidad, usamos el mismo símbolo α para todas las constantes de normalización en esta sección). El modelo de transición agrega una perturbación gaussiana de varianza constante σ_x^2 al estado actual:

$$P(x_{t+1}|x_t) = \alpha e^{-\frac{1}{2}\left(\frac{(x_{t+1}-x_t)^2}{\sigma_x^2}\right)}.$$

El modelo de sensor asume ruido gaussiano con varianza $\sigma 2/z$:

$$P(z_t|x_t) = \alpha e^{-\frac{1}{2}\left(\frac{(z_t-x_t)^2}{\sigma_z^2}\right)}.$$

Ahora, dada la prioridad $P(X_0)$, la distribución predicha en un paso proviene de la Ecuación (14.17):

$$\begin{aligned} P(x_1) &= \int_{-\infty}^{\infty} P(x_1|x_0)P(x_0)dx_0 = \alpha \int_{-\infty}^{\infty} e^{-\frac{1}{2}\left(\frac{(x_1-x_0)^2}{\sigma_x^2}\right)} e^{-\frac{1}{2}\left(\frac{(x_0-\mu_0)^2}{\sigma_0^2}\right)} dx_0 \\ &= \alpha \int_{-\infty}^{\infty} e^{-\frac{1}{2}\left(\frac{\sigma_0^2(x_1-x_0)^2+\sigma_x^2(x_0-\mu_0)^2}{\sigma_0^2\sigma_x^2}\right)} dx_0. \end{aligned}$$

Esta integral parece complicada. La clave para progresar es notar que el exponente es la suma de dos expresiones *cuadráticas* en x_0 y, por lo tanto, es en sí misma una cuadrática en x_0. Un simple truco conocido como **completar el cuadrado** permite reescribir cualquier cuadrática $ax_0^2 + bx_0 + c$ como la suma de un término cuadrado $a(x_0 - \frac{-b}{2a})^2$ y un término residual $c - \frac{b^2}{4a}$ que es independiente de x_0. En este caso, tenemos $a = (\sigma_0^2 + \sigma_x^2)/(\sigma_0^2\sigma_x^2)$, $b = -2(\sigma_0^2 x_1 + \sigma_x^2\mu_0)/(\sigma_0^2\ \sigma_x^2)$, y $c = (\sigma_0^2 x_1^2 + \sigma_x^2\ \mu_0^2)/(\sigma_0^2\ \sigma_x^2)$. El término residual permite sacarse fuera de la integral, lo que nos da

Completar el cuadrado

$$P(x_1) = \alpha e^{-\frac{1}{2}\left(c-\frac{b^2}{4a}\right)} \int_{-\infty}^{\infty} e^{-\frac{1}{2}\left(a(x_0-\frac{-b}{2a})^2\right)} dx_0$$

Ahora la integral es la integral de una gaussiana sobre su rango completo, que es 1. Por lo tanto, nos quedamos sólo con el término residual del cuadrático. Sustituyendo de nuevo las expresiones para a, b, y c y simplificando, obtenemos

$$P(x_1) = \alpha e^{-\frac{1}{2}\left(\frac{(x_1-\mu_0)^2}{\sigma_0^2+\sigma_x^2}\right)}.$$

Es decir, la distribución predicha en un paso es una gaussiana con la misma media μ_0 y una varianza igual a la suma de la varianza original σ_0^2 y la varianza de transición σ_x^2.

Para completar el paso de actualización, necesitamos condicionar a la observación en el primer paso de tiempo, es decir, z_1. Desde la Ecuación (14.18), esto se da por

$$\begin{aligned} P(x_1|z_1) &= \alpha P(z_1|x_1)P(x_1) \\ &= \alpha e^{-\frac{1}{2}\left(\frac{(z_1-x_1)^2}{\sigma_z^2}\right)} e^{-\frac{1}{2}\left(\frac{(x_1-\mu_0)^2}{\sigma_0^2+\sigma_x^2}\right)} \end{aligned}$$

Una vez más, combinamos los exponentes y completamos el cuadrado (Ejercicio 14.KALM), obteniendo la siguiente expresión para el posterior:

$$P(x_1|z_1) = \alpha e^{-\frac{1}{2}\frac{\left(x_1-\frac{(\sigma_0^2+\sigma_x^2)z_1+\sigma_z^2\mu_0}{\sigma_0^2+\sigma_x^2+\sigma_z^2}\right)^2}{(\sigma_0^2+\sigma_x^2)\sigma_z^2/(\sigma_0^2+\sigma_x^2+\sigma_z^2)}}. \tag{14.19}$$

Por lo tanto, después de un ciclo de actualización, tenemos una nueva distribución gaussiana para la variable de estado.

Desde la fórmula gaussiana en la Ecuación (14.19), vemos que la nueva media y la desviación estándar se calculan a partir de la media antigua y la desviación estándar de la siguiente manera:

$$\mu_{t+1} = \frac{(\sigma_t^2+\sigma_x^2)z_{t+1}+\sigma_z^2\mu_t}{\sigma_t^2+\sigma_x^2+\sigma_z^2} \quad \text{y} \quad \sigma_{t+1}^2 = \frac{(\sigma_t^2+\sigma_x^2)\sigma_z^2}{\sigma_t^2+\sigma_x^2+\sigma_z^2}. \tag{14.20}$$

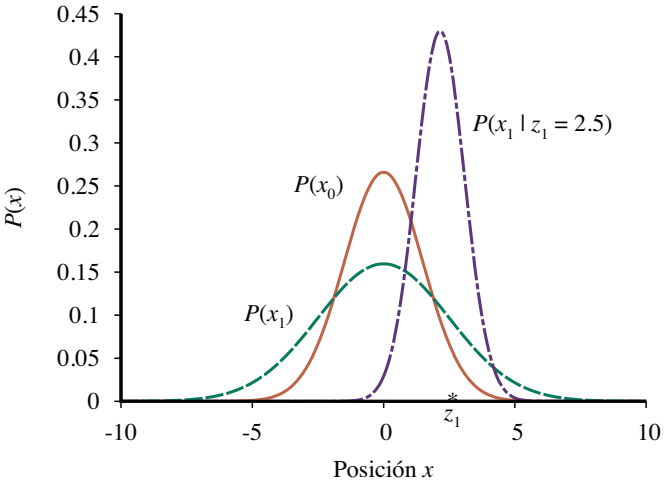


Figura 14.10 Etapas en el ciclo de actualización del filtro de Kalman para un paseo aleatorio con un prior dado por μ_0=0.0 y σ_0=1.5, ruido de transición dado por σ_x=2.0, ruido del sensor dado por σ_z=1.0, y una primera observación z_1=2.5 (marcada en el eje x). Observe cómo la predicción $P(x_1)$ se aplana, con relación a $P(x_0)$, por el ruido de transición. Observe también que la media de la distribución posterior $P(x_1|z_1)$ está a la izquierda de la observación z_1 porque la media es un promedio ponderado de la predicción y la observación.

La Figura 14.10 muestra un ciclo de actualización del filtro de Kalman en el caso unidimensional para valores particulares de los modelos de transición y sensor.

La ecuación (14.20) tiene el mismo papel que la ecuación general de filtrado (14.5) o la ecuación de filtrado HMM (14.12). Sin embargo, debido a la naturaleza especial de las distribuciones gaussianas, las ecuaciones tienen algunas propiedades adicionales interesantes.

Primero, podemos interpretar el cálculo para la nueva media μ_{t+1} como una *media ponderada* de la nueva observación z_{t+1} y la antigua media μ_t. Si la observación es poco fiable, entonces σ^2_z es grande y prestamos más atención a la media antigua; si la media antigua es poco fiable (σ^2_t es grande) o el proceso es impredecible (σ^2_x es grande), entonces prestamos más atención a la observación.

En segundo lugar, observe que la actualización para la varianza σ^2_{t+1} es *independiente de la observación*. Nosotros, por lo tanto, podemos calcular de antemano cuál será la secuencia de valores de varianza. En tercer lugar, la secuencia de valores de varianza converge rápidamente a un valor fijo que depende sólo de σ^2_x y σ^2_z, simplificando así los cálculos posteriores. (Ver Ejercicio 14.VARI).

14.4.3 El caso general

La derivación anterior ilustra la propiedad clave de las distribuciones gaussianas que permite que el filtrado de Kalman funcione: el hecho de que el exponente es una forma cuadrática. Esto es cierto no sólo para el caso univariante; la distribución gaussiana multivariante completa tiene la forma

$$\mathcal{N}(\mathbf{x};\mu,\Sigma) = \alpha e^{-\frac{1}{2}((\mathbf{x}-\mu)^\top\Sigma^{-1}(\mathbf{x}-\mu))}$$

Multiplicando los términos en el exponente, vemos que el exponente también es una función cuadrática de los valores x_i en $\mathbf{x}$. Por lo tanto, el filtrado preserva la naturaleza gaussiana de la distribución del estado.

Primero definamos el modelo temporal general utilizado con el filtrado de Kalman. Se requiere que tanto el modelo de transición como el modelo de sensor sean una transformación *lineal* con ruido gaussiano aditivo. Así, tenemos

$$\begin{aligned} P(\mathbf{x}_{t+1}|\mathbf{x}_t) &= \mathcal{N}(\mathbf{x}_{t+1}; \mathbf{F}\mathbf{x}_t, \Sigma_x) \\ P(\mathbf{z}_t|\mathbf{x}_t) &= \mathcal{N}(\mathbf{z}_t; \mathbf{H}\mathbf{x}_t, \Sigma_z), \end{aligned} \tag{14.21}$$

donde $\mathbf{F}$ y Σ_x son matrices que describen el modelo de transición lineal y la covarianza del ruido de transición, y $\mathbf{H}$ y Σ_z son las matrices correspondientes para el modelo de sensor. Ahora las ecuaciones de actualización para la media y la covarianza son, en su totalidad complejas:

$$\begin{aligned} \mu_{t+1} &= \mathbf{F}\mu_t + \mathbf{K}_{t+1}(\mathbf{z}_{t+1} - \mathbf{H}\mathbf{F}\mu_t) \\ \Sigma_{t+1} &= (\mathbf{I} - \mathbf{K}_{t+1}\mathbf{H})(\mathbf{F}\Sigma_t\mathbf{F}^\top + \Sigma_x), \end{aligned} \tag{14.22}$$

donde $\mathbf{K}_{t+1} = (\mathbf{F}\Sigma_t\mathbf{F}^\top + \Sigma_x)\mathbf{H}^\top(\mathbf{H}(\mathbf{F}\Sigma_t\mathbf{F}^\top + \Sigma_x)\mathbf{H}^\top + \Sigma_z)^{-1}$ es la **matriz de ganancia de Kalman**. Créalo o no, estas ecuaciones tienen cierto sentido intuitivo. Por ejemplo, considere la actualización para la estimación del estado medio μ. El término $\mathbf{F}\mu_t$ es el estado *predicho* en t+1, por lo que $\mathbf{H}\mathbf{F}\mu_t$ es la observación *predicha*. Por lo tanto, el término $\mathbf{z}_{t+1} - \mathbf{H}\mathbf{F}\mu_t$ representa el error en la observación predicha. Esto se multiplica por $\mathbf{K}_{t+1}$ para corregir el estado predicho; por lo tanto, $\mathbf{K}_{t+1}$ es una medida de *cuán serio tomar la nueva observación* en relación con la predicción. Como en la Ecuación (14.20), también tenemos la propiedad de que la actualización de la varianza es independiente de las observaciones. La secuencia de valores para Σ_t y $\mathbf{K}_t$ permite, por lo tanto, ser calculada offline, y los cálculos reales requeridos durante el seguimiento en línea son bastante modestos.

Matriz de ganancia de Kalman

Para ilustrar estas ecuaciones en acción, las hemos aplicado al problema de rastrear un objeto en movimiento en el plano X–Y. Las variables de estado son $\mathbf{X}=(X,Y,\dot{X},\dot{Y})^\top$, por lo que $\mathbf{F}$, Σ_x, $\mathbf{H}$, y Σ_z son matrices 4 × 4. La Figura 14.11(a) muestra la trayectoria verdadera, una serie de observaciones ruidosas, y la trayectoria estimada por el filtrado de Kalman, junto con las covarianzas indicadas por los contornos de una desviación estándar. El proceso de filtrado hace un buen trabajo al rastrear el movimiento real, y, como se esperaba, la varianza alcanza un punto fijo.

También podemos derivar ecuaciones para *suavizar* así como filtrar con modelos lineales-gaussianos. Los resultados del suavizado se muestran en la Figura 14.11(b). Observe cómo la varianza en la estimación de la posición se reduce de manera drástica, excepto en los extremos de la trayectoria (¿por qué?), y que la trayectoria estimada es mucho más suave.

14.4.4 Aplicabilidad del filtrado de Kalman

El filtro de Kalman y sus elaboraciones se utilizan en una amplia gama de aplicaciones. La aplicación "clásica" es en el seguimiento de radar de aviones y misiles. Las aplicaciones relacionadas incluyen el seguimiento acústico de submarinos y vehículos terrestres y el seguimiento visual de vehículos y personas. En una vena un poco más esotérica, los filtros de Kalman se usan para reconstruir trayectorias de partículas a partir de fotografías de cámaras de burbujas y corrientes oceánicas mediante mediciones de superficie por satélite. El rango de aplicación es mucho mayor que sólo el seguimiento del movimiento: cualquier sistema caracterizado por variables de estado continuas y mediciones ruidosas servirá. Tales sistemas incluyen molinos de pulpa, plantas químicas, reactores nucleares, ecosistemas de plantas y economías nacionales.

El hecho de que el filtrado de Kalman se pueda aplicar a un sistema no significa que los resultados serán válidos o útiles. Las suposiciones hechas: transición lineal-gaussiana y sensor, los modelos son muy fuertes. El **filtro de Kalman extendido** (EKF) intenta superar las no linealidades en el sistema que se está modelando. Un sistema es **no lineal** si el modelo de transición no permite ser descrito como una multiplicación de matrices del vector de estado, como en

Filtro de Kalman extendido

No lineal

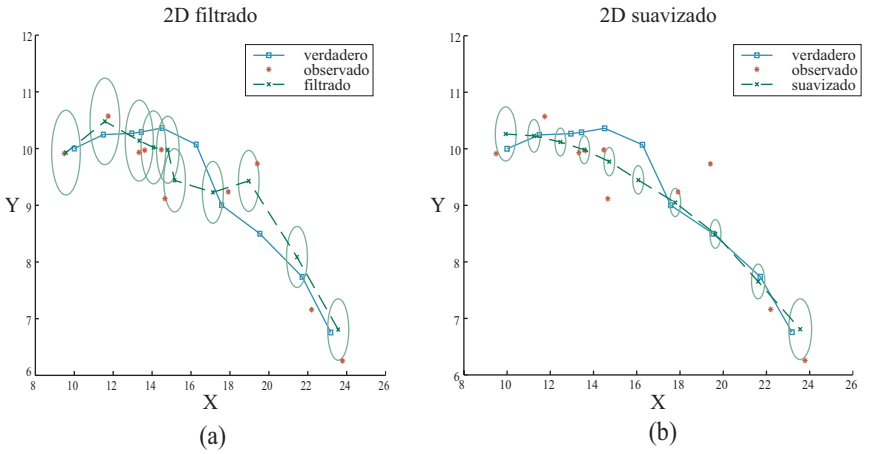


Figura 14.11 (a) Resultados del filtrado de Kalman para un objeto en movimiento en el plano X–Y, mostrando la trayectoria real (de izquierda a derecha), una serie de observaciones ruidosas, y la trayectoria estimada por el filtrado de Kalman. La varianza en la estimación de la posición está indicada por los óvalos. (b) Los resultados del suavizado de Kalman para la misma secuencia de observación.

la Ecuación (14.21). El EKF funciona modelando el sistema como *localmente* lineal en $\mathbf{x}_t$ en la región de $\mathbf{x}_t = \mu_t$, la media de la distribución de estado actual. Esto funciona bien para sistemas suaves y bien comportados, y permite que el rastreador mantenga y actualice una distribución de estado gaussiana que es una aproximación razonable al verdadero posterior. Se da un ejemplo detallado en el Capítulo 26.

¿Qué significa que un sistema sea "no suave" o "mal comportado"? Técnicamente, significa que hay una no linealidad significativa en la respuesta del sistema dentro de la región que está "cerca" (según la covarianza Σ_t) a la media actual μ_t. Para entender esta idea en términos no técnicos, considere el ejemplo de intentar rastrear un pájaro mientras vuela a través de la jungla. El pájaro parece estar yendo a alta velocidad directamente hacia el tronco de un árbol. El filtro de Kalman, ya sea regular o extendido, sólo permite hacer una predicción gaussiana de la ubicación del pájaro, y la media de esta gaussiana estará centrada en el tronco, como se muestra en la Figura 14.12(a). Un modelo razonable del pájaro, por otro lado, predeciría una acción evasiva hacia un lado o el otro, como se muestra en la Figura 14.12(b). Tal modelo es altamente no lineal, porque la decisión del pájaro varía de manera brusca dependiendo de su ubicación precisa en relación con el tronco.

Para manejar ejemplos como estos, necesitamos un lenguaje más expresivo para representar el comportamiento del sistema que se está modelando. Dentro de la comunidad de teoría de control, para la cual problemas como las maniobras evasivas de las aeronaves plantean los mismos tipos de dificultades, la solución estándar es el **filtro Kalman de conmutación**. En este enfoque, varios filtros Kalman funcionan en paralelo, cada uno utilizando un modelo diferente del sistema; por ejemplo, uno para el vuelo recto, uno para los giros bruscos a la izquierda y uno para los giros bruscos a la derecha. Se usa una suma ponderada de predicciones, donde el peso depende de cuán bien cada filtro se ajusta a los datos actuales. En la siguiente sección veremos que esto es un caso especial del modelo general de red bayesiana dinámica, obtenido al agregar una variable de estado de "maniobra" discreta a la red mostrada en la Figura 14.9. Los filtros Kalman de conmutación se discuten más adelante en el Ejercicio 14.KFSW.

Filtro Kalman de conmutación

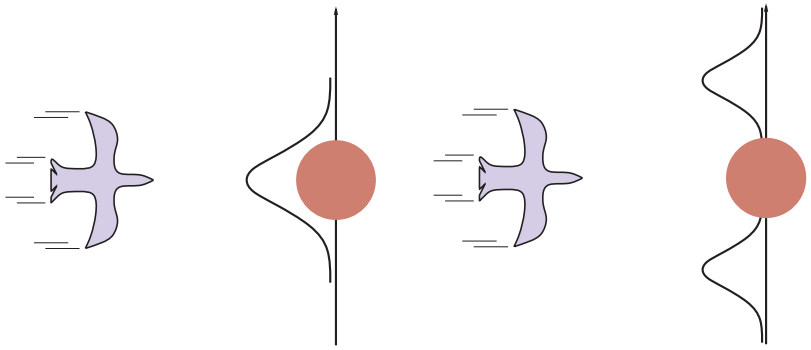


Figura 14.12 Un pájaro volando hacia un árbol (vistas superiores). (a) Un filtro Kalman predice la ubicación del pájaro utilizando una única gaussiana centrada en el obstáculo. (b) Un modelo más realista permite la acción evasiva del pájaro, prediciendo que volará a uno u otro lado.

14.5 Redes bayesianas dinámicas

Redes bayesianas dinámicas

Las **redes bayesianas dinámicas**, o **DBN**, extienden la semántica de las redes bayesianas estándar para manejar modelos de probabilidad temporales del tipo descrito en la Sección 14.1. Ya hemos visto ejemplos de DBN: la red del paraguas en la Figura 14.2 y la red del filtro Kalman en la Figura 14.9. En general, cada segmento de una DBN permite tener cualquier número de variables de estado $\mathbf{X}_t$ y variables de evidencia $\mathbf{E}_t$. Para simplificar, asumimos que las variables, sus enlaces y sus distribuciones condicionales se replican de segmento a segmento y que la DBN representa un proceso de Markov de primer orden, de modo que cada variable permite tener padres sólo en su propio segmento o en el segmento anterior. De esta manera, la DBN corresponde a una red bayesiana con variables infinitas.

Debería quedar claro que cada modelo de Markov oculto puede representarse como una DBN con una única variable de estado y una única variable de evidencia. También es el caso que cada DBN de variable discreta permite representarse como un HMM; como se explicó en la Sección 14.3, podemos combinar todas las variables de estado en la DBN en una única variable de estado cuyos valores son todas las posibles tuplas de valores de las variables de estado individuales. Ahora, si cada HMM es una DBN y cada DBN se traduce en un HMM, ¿cuál es la diferencia? La diferencia es que, *al descomponer el estado de un sistema complejo en sus variables constituyentes, podemos aprovechar la escasez en el modelo de probabilidad temporal*.

Para ver lo que esto significa en la práctica, recuerde que en la Sección 14.3 dijimos que una representación HMM para un proceso temporal con n variables discretas, cada una con hasta d valores, necesita una matriz de transición de tamaño $O(d^{2n})$. La representación DBN, por otro lado, tiene tamaño $O(nd^k)$ si el número de padres de cada variable está limitado por k. En otras palabras, la representación DBN es lineal, no exponencial en el número de variables. Para el robot aspiradora con 42 ubicaciones de suciedad, el número de probabilidades requeridas se reduce de 5×10^{29} a unos pocos miles.

Ya hemos explicado que cada modelo de filtro Kalman permite representarse en una DBN con variables continuas y distribuciones condicionales lineales-gaussianas (Figura 14.9). Debería quedar claro a partir de la discusión al final de la sección anterior que *no* todas las DBN pueden ser representadas por un modelo de filtro Kalman. En un filtro Kalman, la distribución de estado

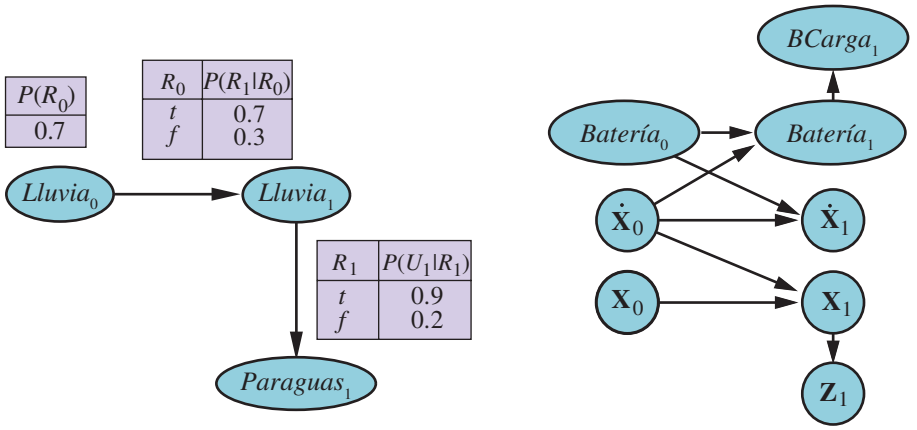


Figura 14.13 Izquierda: Especificación del prior, modelo de transición y modelo de sensor para la DBN del paraguas. Los siguientes segmentos son copias del segmento 1. Derecha: Una DBN para el movimiento del robot en el plano X-Y.

actual siempre es una única distribución gaussiana multivariante; es decir, un único "bulto" en una ubicación particular. Las DBN, por otro lado, permiten modelar distribuciones arbitrarias.

Para muchas aplicaciones del mundo real, esta flexibilidad es esencial. Considere, por ejemplo, la ubicación actual de mis llaves. Podrían estar en mi bolsillo, en la mesa de noche, en la barra de la cocina, colgando de la puerta principal o encerradas en el coche. Un único bulto gaussiano que incluyera todos estos lugares tendría que asignar una probabilidad significativa a que las llaves estén colgadas en el aire sobre el jardín frontal. Aspectos del mundo real como agentes con propósito, obstáculos y bolsillos introducen "no linealidades" que requieren combinaciones de variables discretas y continuas para obtener modelos razonables.

14.5.1 Construyendo DBN

Para construir un DBN, se debe especificar tres tipos de información: la distribución previa sobre las variables de estado, $\mathbf{P}(\mathbf{X}_0)$; el modelo de transición $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{X}_t)$; y el modelo de sensor $\mathbf{P}(\mathbf{E}_t|\mathbf{X}_t)$. Para especificar los modelos de transición y sensor, también se debe especificar la topología de las conexiones entre las sucesivas secciones y entre las variables de estado y evidencia. Debido a que se asume que los modelos de transición y sensor son homogéneos en el tiempo, los mismos para todos los t, es más conveniente especificarlos para la primera sección. Por ejemplo, la especificación completa de DBN para el mundo del paraguas se da por la red de tres nodos mostrada en la Figura 14.13(a). A partir de esta especificación, es posible construir el DBN completo con un número ilimitado de secciones de tiempo según sea necesario copiando la primera sección.

Consideremos ahora un ejemplo más interesante: monitorear un robot alimentado por batería moviéndose en el plano X–Y, como se introdujo al final de la Sección 14.1. Primero, necesitamos variables de estado, que incluirán tanto $\mathbf{X}_t=(X_t,Y_t)$ para la posición y $\dot{\mathbf{X}}_t=(\dot{X}_t,\dot{Y}_t)$ para la velocidad. Nosotros asumimos algún método de medición de posición, quizás una cámara fija o GPS a bordo (*Global Positioning System*), que produce mediciones $\mathbf{Z}_t$. La posición en el siguiente paso de tiempo depende de la posición y velocidad actuales, como en el modelo estándar de

filtro Kalman. La velocidad en el siguiente paso depende de la velocidad actual y el estado de la batería. Agregamos $Batería_t$ para representar el nivel de carga de batería actual, que tiene como padres el nivel de batería anterior y la velocidad, y agregamos $BCarga_t$, que mide el nivel de carga de la batería. Esto nos da el modelo básico mostrado en la Figura 14.13(b).

Vale la pena mirar a detalle la naturaleza del modelo de sensor para $BCarga_t$. Supongamos, por simplicidad, que tanto $Batería_t$ como $BCarga_t$ pueden tomar valores discretos de 0 a 5. (El Ejercicio 14.BATT le pide relacionar este modelo discreto con un modelo continuo correspondiente). Si el medidor siempre es preciso, entonces el CPT $\mathbf{P}(BCarga_t|Batería_t)$ debería tener probabilidades de 1.0 "a lo largo de la diagonal" y probabilidades de 0.0 en otros lugares. En realidad, el ruido siempre se filtra en las mediciones. Para mediciones continuas, se podría usar una distribución gaussiana con una pequeña varianza.[7] Para nuestras variables discretas, podemos aproximar una gaussiana usando una distribución en la que la probabilidad de error disminuye de la manera apropiada, de modo que la probabilidad de un error grande es muy pequeña. Usamos el término **error gaussiano modelo** para cubrir tanto las versiones continuas como discretas.

Error gaussiano modelo

Cualquiera con experiencia práctica en robótica, control de procesos informatizados u otras formas de detección automática testificará que pequeñas cantidades de ruido de medición son el menor de los problemas. Los sensores reales *fallan*. Cuando un sensor falla, no necesariamente envía una señal que dice, "Oh, por cierto, los datos que estoy a punto de enviarte son un montón de sinsentidos". En vez de enviar sinsentidos. El tipo de falla más simple se llama **fallo transitorio**, donde el sensor ocasionalmente decide enviar algunas sinrazones; por ejemplo, el sensor de nivel de batería podría tener la costumbre de enviar una lectura de 0 cuando alguien golpea el robot, aun si la batería está completamente cargada.

Fallo transitorio

Veamos qué sucede cuando ocurre un fallo transitorio con un modelo de error gaussiano que no acomoda tales fallas. Suponga que el robot está detenido y observa 20 lecturas consecutivas de batería en 5. Luego, el medidor de batería tiene una baja temporal y la siguiente lectura es $BCarga_{21} = 0$. ¿A qué nos llevará el simple modelo de error gaussiano a creer sobre $Batería_{21}$? De acuerdo con la regla de Bayes, la respuesta depende tanto del modelo de sensor $\mathbf{P}(BCarga_{21} = 0|Batería_{21})$ como de la predicción $\mathbf{P}(Batería_{21}|BCarga_{1:20})$. Si la probabilidad de un gran error del sensor es significativamente menor que la probabilidad de una transición a $Batería_{21} = 0$, aun si esta última es muy poco probable, entonces la distribución posterior asignará una alta probabilidad a que la batería esté vacía.

Una segunda lectura de 0 en $t = 22$ hará que esta conclusión sea casi segura. Si el fallo transitorio luego desaparece y la lectura vuelve a 5 desde $t = 23$ en adelante, la estimación para el nivel de batería volverá a 5. (Esto no significa que el algoritmo piense que la batería se recargó, lo cual es físicamente imposible; en cambio, el algoritmo ahora cree que la batería nunca estuvo baja y la hipótesis improbable de que el medidor de batería tuvo dos errores enormes consecutivos debe ser la explicación correcta). Este curso de eventos se ilustra en la curva superior de la Figura 14.14(a), que muestra el valor esperado (ver Apéndice A) de $Batería_t$ con el tiempo, utilizando un modelo de error gaussiano discreto.

A pesar de la recuperación, hay un momento ($t = 22$) en el que el robot está convencido de que su batería está presumiblemente vacía; entonces, debería enviar una señal de socorro y apagarse. Por desgracia, su modelo de sensor simplificado en exceso lo ha llevado por mal camino. La moraleja de la historia es simple: *para que el sistema maneje de manera correcta el fallo del sensor, el modelo del sensor debe incluir la posibilidad de fallo.*

[7] Hablando de manera estricta, una distribución gaussiana es problemática porque asigna una probabilidad distinta de cero a niveles de carga negativos grandes. La **distribución beta** es a veces una mejor elección para una variable cuyo rango está restringido.

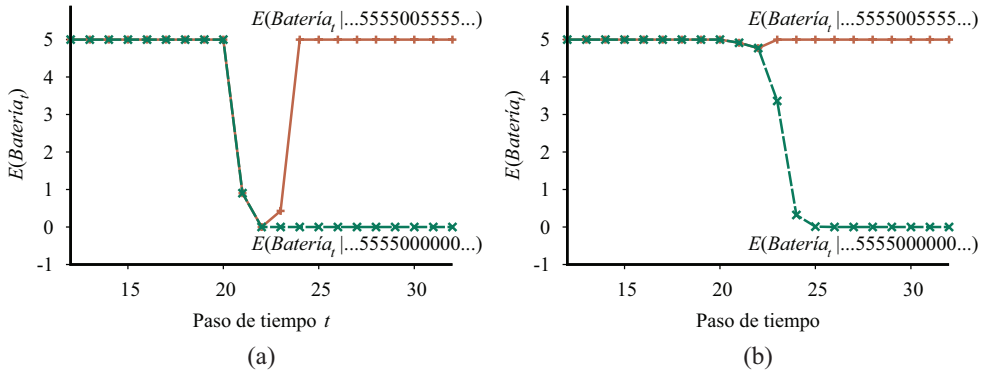


Figura 14.14 (a) Curva superior: trayectoria del valor esperado de $Batería_t$ para una secuencia de observación que consiste en todos los 5s excepto para los 0s en $t = 21$ y $t = 22$, usando un simple modelo de error gaussiano. Curva inferior: trayectoria cuando la observación permanece en 0 desde $t = 21$ en adelante. (b) El mismo experimento se ejecuta con el modelo de fallo transitorio. El fallo transitorio se maneja bien, pero el fallo persistente resulta en un pesimismo excesivo sobre la carga de la batería.

El tipo más sencillo de modelo de fallo para un sensor permite una cierta probabilidad de que el sensor devuelva un valor por completo incorrecto, independiente del estado real del mundo. Por ejemplo, si el medidor de batería falla devolviendo 0, podríamos decir que

$$P(BCarga_t = 0 | Batería_t = 5) = 0.03,$$

Modelo de fallo transitorio

que es presumiblemente mucho mayor que la probabilidad asignada por el simple modelo de error gaussiano. Llamemos a esto el **modelo de fallo transitorio**. ¿Cómo ayuda cuando nos enfrentamos a una lectura de 0? Siempre que la probabilidad *predicha* de una batería vacía, según las lecturas hasta ahora, sea mucho menor que 0.03, entonces la mejor explicación de la observación $BCarga_{21} = 0$ es que el sensor ha fallado de manera temporal. Intuitivamente, podemos pensar en la creencia sobre el nivel de la batería como que tiene una cierta cantidad de "inercia" que ayuda a superar blips temporales en la lectura del medidor. La curva superior en la Figura 14.14(b) muestra que el modelo de fallo transitorio permite manejar fallos transitorios sin un cambio catastrófico en las creencias.

Hasta aquí los blips temporales. ¿Qué pasa con un fallo persistente del sensor? Los fallos de este tipo son demasiado comunes. Si el sensor devuelve 20 lecturas de 5 seguidas por 20 lecturas de 0, entonces el modelo de fallo del sensor transitorio descrito en el párrafo anterior resultará en que el robot gradualmente llegue a creer que su batería está vacía cuando en realidad puede ser que el medidor haya fallado. La curva inferior en la Figura 14.14(b) muestra la "trayectoria" de la creencia para este caso. Para $t = 25$, cinco lecturas de 0, el robot está convencido de que su batería está vacía. En general, preferiríamos que el robot creyera que su medidor de batería está fallando, si de hecho este es el evento más probable.

Modelo de fallo persistente

Como era de esperar, para manejar el fallo persistente, necesitamos un **modelo de fallo persistente** que describa cómo se comporta el sensor en condiciones normales y después del fallo. Para hacer esto, necesitamos aumentar el estado del sistema con una variable adicional, digamos, *BMFallo*, que describe el estado del medidor de batería. La persistencia del fallo debe ser

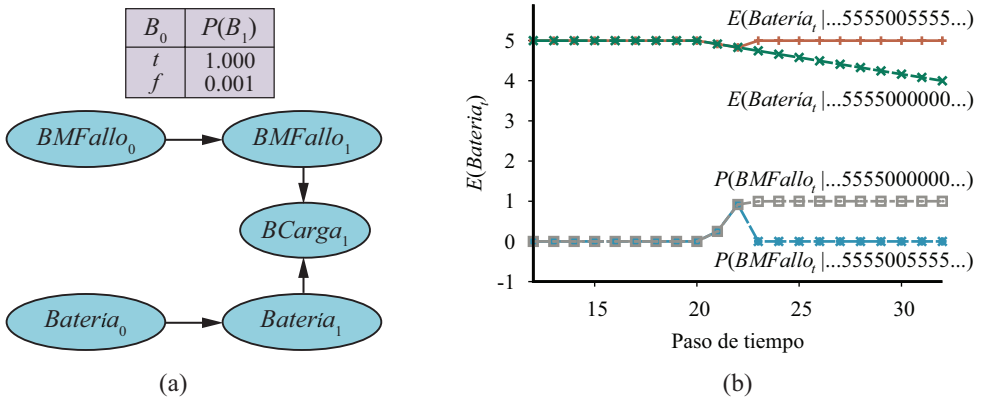


Figura 14.15 (a) Un fragmento de DBN que muestra la variable de estado del sensor requerida para modelar el fallo persistente del sensor de la batería. (b) Curvas superiores: trayectorias del valor esperado de $Batería_t$ para las secuencias de observación de "fallo transitorio" y "fallo permanente". Curvas inferiores: trayectorias de probabilidad para *BMFallo* dadas las dos secuencias de observación.

modelada por un arco que enlaza $BMFallo_0$ con $BMFallo_1$. Este **arco de persistencia** tiene un CPT que da una pequeña probabilidad de fallo en cualquier paso de tiempo, digamos 0.001, pero especifica que el sensor permanece en falla una vez que ha fallado. Cuando el sensor está bien, el modelo del sensor para *BCarga* es idéntico al modelo de fallo transitorio; cuando el sensor falla, dice que *BCarga* siempre es 0, independiente de la carga real de la batería.

Arco de persistencia

El modelo de fallo persistente para el sensor de la batería se muestra en la Figura 14.15(a). Su rendimiento en las dos secuencias de datos (blip temporal y fallo persistente) se muestra en la Figura 14.15(b). Hay varias cosas que notar sobre estas curvas. Primero, en el caso del blip temporal, la probabilidad de que el sensor falle aumenta de modo significativo después de la segunda lectura 0, pero de inmediato vuelve a cero una vez que se observa un 5. Segundo, en el caso de fallo persistente, la probabilidad de que el sensor falle aumenta a casi 1 y se mantiene allí. Finalmente, una vez que se sabe que el sensor falló, el robot sólo es capaz de suponer que su batería se descarga a la tasa "normal". Esto se muestra en el nivel gradualmente descendente de $E(Batería_t|...)$.

Hasta ahora, apenas hemos arañado la superficie del problema de representar procesos complejos. La variedad de modelos de transición es enorme, pues abarca temas tan dispares como modelar el sistema endocrino humano y modelar varios vehículos conduciendo en una autopista. El modelado de sensores también es un vasto subcampo en sí mismo. Pero las redes bayesianas dinámicas permiten modelar incluso fenómenos sutiles, como la deriva del sensor, la descalibración repentina y los efectos de las condiciones exógenas (como el clima) en las lecturas del sensor.

14.5.2 Inferencia exacta en DBNs

Luego de esbozar algunas ideas para representar procesos complejos como DBNs, ahora nos volvemos a la cuestión de la inferencia. En cierto sentido, esta pregunta ya ha sido respondida: las redes bayesianas dinámicas *son* redes bayesianas, de estas ya tenemos algoritmos para la inferencia en redes bayesianas. Dada una secuencia de observaciones, es posible construir la representación completa de la red bayesiana de un DBN replicando cortes hasta que la red sea lo suficientemente

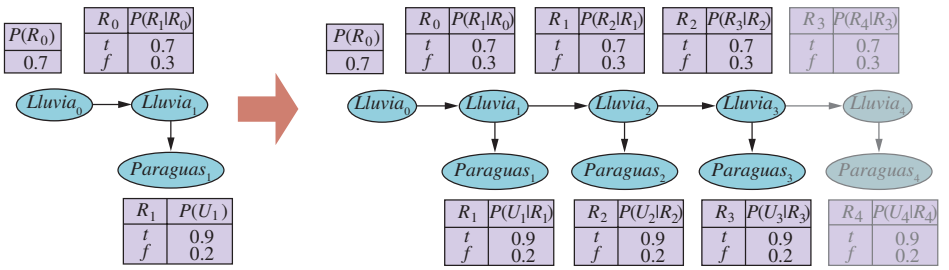


Figura 14.16 Desenrollando una red bayesiana dinámica: las secciones se replican para acomodar la secuencia de observación $Paraguas_{1:3}$. Las secciones adicionales no tienen efecto en las inferencias dentro del periodo de observación.

grande para acomodar las observaciones, como en la Figura 14.16. Esta técnica se llama **desenrollado**. (Técnicamente, el DBN es equivalente a la red seminfinita obtenida al desenrollar para siempre. Los cortes añadidos más allá de la última observación no tienen efecto en las inferencias dentro del periodo de observación y permiten ser omitidos). Una vez que el DBN está desenrollado, es posible utilizar cualquiera de los algoritmos de inferencia —eliminación de variables, métodos de agrupación, y así sucesivamente— descritos en el Capítulo 13.

Pero, una aplicación ingenua del desenrollado no sería en particular eficiente. Si queremos realizar el filtrado o suavizado con una larga secuencia de observaciones $\mathbf{e}_{1:t}$, la red desenrollada requeriría espacio $O(t)$ y, por lo tanto, crecería sin límite a medida que se añaden más observaciones. Además, si sólo ejecutamos el algoritmo de inferencia de nuevo cada vez que se añade una observación, el tiempo de inferencia por actualización también aumentará como $O(t)$.

Mirando hacia atrás a la Sección 14.2.1, vemos que el tiempo constante y el espacio por actualización de filtrado permiten lograrse si el cálculo se realiza de forma recursiva. Por definición, la actualización de filtrado en la Ecuación (14.5) funciona *sumando* las variables de estado del paso de tiempo anterior para obtener la distribución para el nuevo paso de tiempo. Sumar variables es lo que hace el algoritmo de **eliminación de variables** (Figura 13.13), y resulta que ejecutar la eliminación de variables con las variables en orden temporal imita la operación de la actualización de filtrado recursivo en la Ecuación (14.5). El algoritmo modificado mantiene a lo sumo dos secciones en memoria en cualquier momento: comenzando con la sección 0, añadimos la sección 1, luego sumamos la sección 0, luego añadimos la sección 2, luego sumamos la sección 1, y así sucesivamente. De esta manera, podemos lograr un espacio y tiempo constantes por actualización de filtrado. (El mismo rendimiento permite lograrse con modificaciones adecuadas al algoritmo de agrupación). El Ejercicio 14.DBNE le pide verificar este hecho para la red de paraguas.

Hasta aquí las buenas noticias; ahora las malas: Resulta que la "constante" para el tiempo de actualización y por la complejidad del espacio es, en casi todos los casos, exponencial en el número de variables de estado. Lo que sucede es que, a medida que avanza la eliminación de variables, los factores crecen para incluir todas las variables de estado (o, más precisamente, todas esas variables de estado que tienen padres en el corte de tiempo anterior). El tamaño máximo del factor es $O(d^{n+k})$ y el costo total de actualización por paso es $O(nd^{n+k})$, donde d es el tamaño del dominio de las variables y k es el número máximo de padres de cualquier variable de estado.

Por supuesto, esto es mucho menos que el costo de la actualización de HMM, que es $O(d^{2n})$, pero aun así es inviable para un gran número de variables. Este hecho sombrío significa que *aunque podemos usar DBNs para representar procesos temporales muy complejos con muchas variables*

escasamente conectadas, no podemos razonar de manera eficiente y exacta sobre esos procesos. El modelo DBN en sí, que representa la distribución conjunta previa sobre todas las variables, es factorizable en sus CPTs constituyentes; no obstante, la distribución conjunta posterior condicionada a una secuencia de observación -el mensaje hacia adelante- por lo regular *no* es factorizable. El problema es intratable en general, por lo que debemos recurrir a métodos aproximados.

14.5.3 Inferencia aproximada en DBNs

La Sección 13.4 describió dos algoritmos de aproximación: ponderación de probabilidad (Figura 13.18) y Monte Carlo de cadena de Markov (MCMC, Figura 13.20). De los dos, el primero se adapta con mayor facilidad al contexto DBN. (Un algoritmo de filtrado MCMC se describe de manera breve en las notas al final de este capítulo). Veremos, sin embargo, que se requieren varias mejoras sobre el algoritmo de ponderación de probabilidad estándar antes de que surja un método práctico.

Recuerde que la ponderación de probabilidad funciona muestreando los nodos de no evidencia de la red en orden topológico, ponderando cada muestra por la probabilidad que otorga a las variables de evidencia observadas. Al igual que con los algoritmos exactos, podríamos aplicar la ponderación de probabilidad de manera directa a un DBN desenrollado, aunque esto sufriría de los mismos problemas de aumento de tiempo y requisitos de espacio por actualización a medida que crece la secuencia de observación. El problema es que el algoritmo estándar ejecuta cada muestra a su vez, a través de toda la red.

En cambio, sólo podemos ejecutar todas las N muestras juntas a través del DBN, una sección a la vez. El algoritmo modificado se ajusta al patrón general de los algoritmos de filtrado, con el conjunto de N muestras como el mensaje hacia adelante. La primera innovación clave, entonces, es *usar las muestras mismas como una representación aproximada de la distribución de estado actual.* Esto cumple con el requisito de un tiempo "constante" por actualización, aunque la constante depende del número de muestras requeridas para mantener una aproximación precisa. Tampoco es necesario desenrollar el DBN, porque sólo necesitamos tener en memoria la sección actual y la siguiente. Llamamos este enfoque **muestreo de importancia secuencial** o SIS.

Muestreo de importancia secuencial

En nuestra discusión sobre la ponderación de probabilidad en el Capítulo 13, señalamos que la precisión del algoritmo sufre si las variables de evidencia están "aguas abajo" de las variables que se están muestreando, porque en ese caso las muestras se generan sin ninguna influencia de la evidencia y casi todas tendrán pesos muy bajos.

Ahora, si miramos la estructura típica de un DBN, digamos, el DBN de paraguas en la Figura 14.16, vemos que, en efecto, las variables de estado tempranas se muestrearán sin el beneficio de la evidencia posterior. De hecho, mirando de manera más cuidadosa, vemos que ¡*ninguna* de las variables de estado tiene *algunas* variables de evidencia entre sus antepasados! Por lo tanto, aunque el peso de cada muestra dependerá de la evidencia, el conjunto real de muestras generadas será *por completo independiente* de la evidencia. Por ejemplo, incluso si el jefe trae el paraguas todos los días, el proceso de muestreo aún podría alucinar días interminables de Sol.

Lo que esto significa en la práctica es que la fracción de muestras que permanecen razonablemente cerca de la serie real de eventos (y, por lo tanto, tienen peso no despreciable) disminuye exponencialmente con t, la longitud de la secuencia. En otras palabras, para mantener un nivel dado de precisión, necesitamos aumentar el número de muestras de modo exponencial con t. Dado que un algoritmo de filtrado en tiempo real sólo permite usar un número limitado de muestras, lo que sucede en la práctica es que el error se dispara después de un número muy pequeño de pasos de actualización. La Figura 14.19 en la página 494 muestra este efecto para SIS aplicado al problema de localización de retícula-mundo de la Sección 14.3: aun con 100 000 muestras, la aproximación de SIS falla por completo después de unos 20 pasos.

function PARTICLE-FILTERING(**e**,*N*,*dbn*) **returns** un conjunto de muestras para el siguiente paso de tiempo
  inputs: **e**, las nuevas evidencias entrantes
    N, el número de muestras que se van a mantener
    dbn, a DBN definido por $\mathbf{P}(\mathbf{X}_0)$,$\mathbf{P}(\mathbf{X}_1|\mathbf{X}_0)$, y $\mathbf{P}(\mathbf{E}_1|\mathbf{X}_1)$
  persistent: *S*, un vector de muestras de tamaño *N*, generado inicialmente a partir de $\mathbf{P}(\mathbf{X}_0)$
  local variables: *W*, un vector de pesos de tamaño *N*

  for *i* = 1 a *N* **do**
    $S[i] \leftarrow$ muestra de $\mathbf{P}(\mathbf{X}_1|\mathbf{X}_0 = S[i])$ // *step 1*
    $W[i] \leftarrow \mathbf{P}(\mathbf{e}|\mathbf{X}_1 = S[i])$ // *step 2*
  $S \leftarrow$ WEIGHTED-SAMPLE-WITH-REPLACEMENT(*N*,*S*,*W*) // *step 3*
  return *S*

Figura 14.17 El algoritmo de filtrado de partículas implementado como una operación de actualización recursiva con estado (el conjunto de muestras). Cada una de las operaciones de muestreo implica muestrear las variables de sección relevantes en orden topológico. La operación WEIGHTED-SAMPLE-WITH-REPLACEMENT se puede implementar para ejecutarse en tiempo esperado $O(N)$. Los números de paso se refieren a la descripción en el texto.

Note que necesitamos una mejor solución. La segunda innovación clave es *enfocar el conjunto de muestras en las regiones de alta probabilidad del espacio de estado*. Esto se realiza descartando muestras que tienen un peso muy bajo, según las observaciones, mientras se replican aquellas que tienen un peso alto. De esa manera, la población de muestras se mantendrá razonablemente cerca de la realidad. Si pensamos en las muestras como un recurso para modelar la distribución posterior, entonces tiene sentido usar más muestras en regiones del espacio de estado donde la posterior es más alta.

Filtrado de partículas

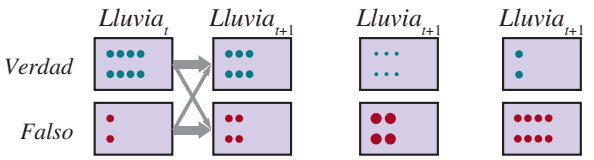
Una familia de algoritmos llamada **filtrado de partículas** está diseñada para hacer eso. (Otro nombre temprano fue **muestreo de importancia secuencial con remuestreo**, pero por alguna razón no se popularizó). El filtrado de partículas funciona de la siguiente manera: Primero, generamos una población de *N* muestras de la distribución previa $\mathbf{P}(\mathbf{X}_0)$. Luego, el ciclo de actualización se repite para cada paso de tiempo:

1. Cada muestra se propaga hacia adelante muestreando el siguiente valor de estado $\mathbf{x}_{t+1}$ dado el valor actual $\mathbf{x}_t$ para la muestra, con base en el modelo de transición $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t)$.
2. Cada muestra se pondera por la probabilidad que asigna a la nueva evidencia, $P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1})$.
3. La población se *remuestrea* para generar una nueva población de *N* muestras. Cada nueva muestra se selecciona de la población actual; la probabilidad de que se seleccione una muestra en particular es proporcional a su peso. Las nuevas muestras no tienen peso.

El algoritmo se muestra en detalle en la Figura 14.17, y su operación para el DBN de paraguas se ilustra en la Figura 14.18.

Podemos demostrar que este algoritmo es consistente, da las probabilidades correctas a medida que *N* tiende a infinito, examinando las operaciones en un ciclo de actualización. Asumimos que la población de muestras comienza con una representación correcta del mensaje hacia adelante; es decir, $\mathbf{f}_{1:t} = \mathbf{P}(\mathbf{X}_t|\mathbf{e}_{1:t})$ en el tiempo *t*. Escribiendo $N(\mathbf{x}_t|\mathbf{e}_{1:t})$ para el número de muestras que ocupan el estado $\mathbf{x}_t$ después de las observaciones $\mathbf{e}_{1:t}$ han sido procesadas, por lo tanto tenemos

$$N(\mathbf{x}_t|\mathbf{e}_{1:t})/N = P(\mathbf{x}_t|\mathbf{e}_{1:t}) \tag{14.23}$$



(a) Propaga (b) Peso (c) Remuestreo

Figura 14.18 El ciclo de actualización del filtrado de partículas para el DBN de paraguas con $N = 10$, mostrando las poblaciones de muestra de cada estado. (a) En el tiempo t, 8 muestras indican *lluvia* y 2 indican $\neg$*lluvia*. Cada uno se propaga hacia adelante muestreando el siguiente estado a través del modelo de transición. En el tiempo $t + 1$, 6 muestras indican *lluvia* y 4 indican $\neg$*lluvia*. (b) $\neg$*paraguas* se observa en $t + 1$. Cada muestra se pondera por su probabilidad para la observación, como lo indican el tamaño de los círculos. (c) Se genera un nuevo conjunto de 10 muestras mediante la selección aleatoria ponderada del conjunto actual, resultando en 2 muestras que indican *lluvia* y 8 que indican $\neg$*lluvia*.

para N grande. Ahora propagamos cada muestra hacia adelante muestreando las variables de estado en $t + 1$, dado los valores para la muestra en t. El número de muestras que alcanzan el estado $\mathbf{x}_{t+1}$ desde cada $\mathbf{x}_t$ es la probabilidad de transición veces la población de $\mathbf{x}_t$; por lo tanto, el número total de muestras que alcanzan $\mathbf{x}_{t+1}$ es

$$N(\mathbf{x}_{t+1}|\mathbf{e}_{1:t}) = \sum_{\mathbf{x}_t} P(\mathbf{x}_{t+1}|\mathbf{x}_t) N(\mathbf{x}_t|\mathbf{e}_{1:t}).$$

Ahora ponderamos cada muestra por su probabilidad para la evidencia en $t + 1$. Una muestra en el estado $\mathbf{x}_{t+1}$ recibe peso $P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1})$. El peso total de las muestras en $\mathbf{x}_{t+1}$ después de ver $\mathbf{e}_{t+1}$ es, por lo tanto,

$$W(\mathbf{x}_{t+1}|\mathbf{e}_{1:t+1}) = P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1}) N(\mathbf{x}_{t+1}|\mathbf{e}_{1:t}).$$

Ahora para el paso de remuestreo. Dado que cada muestra se replica con una probabilidad proporcional a su peso, el número de muestras en el estado $\mathbf{x}_{t+1}$ después del remuestreo es proporcional al peso total en $\mathbf{x}_{t+1}$ antes del remuestreo:

$$\begin{aligned}
N(\mathbf{x}_{t+1}|\mathbf{e}_{1:t+1})/N &= \alpha W(\mathbf{x}_{t+1}|\mathbf{e}_{1:t+1}) \\
&= \alpha P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1}) N(\mathbf{x}_{t+1}|\mathbf{e}_{1:t}) \\
&= \alpha P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1}) \sum_{\mathbf{x}_t} P(\mathbf{x}_{t+1}|\mathbf{x}_t) N(\mathbf{x}_t|\mathbf{e}_{1:t}) \\
&= \alpha N P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1}) \sum_{\mathbf{x}_t} P(\mathbf{x}_{t+1}|\mathbf{x}_t) P(\mathbf{x}_t|\mathbf{e}_{1:t}) \quad \text{(por 14.23)} \\
&= \alpha' P(\mathbf{e}_{t+1}|\mathbf{x}_{t+1}) \sum_{\mathbf{x}_t} P(\mathbf{x}_{t+1}|\mathbf{x}_t) P(\mathbf{x}_t|\mathbf{e}_{1:t}) \\
&= P(\mathbf{x}_{t+1}|\mathbf{e}_{1:t+1}) \quad \text{(por 14.5)}.
\end{aligned}$$

Por lo tanto, la población de muestras después de un ciclo de actualización representa de manera correcta el mensaje hacia adelante en el tiempo $t + 1$.

El filtrado de partículas, por lo tanto, es *consistente*, ¿pero es *eficiente*? Para muchos casos prácticos, parece que la respuesta es sí: el filtrado de partículas parece mantener una buena aproximación al posterior verdadero utilizando un número constante de muestras. La Figura 14.19 ilustra que el filtrado de partículas hace un buen trabajo en el problema de localización del mundo de la

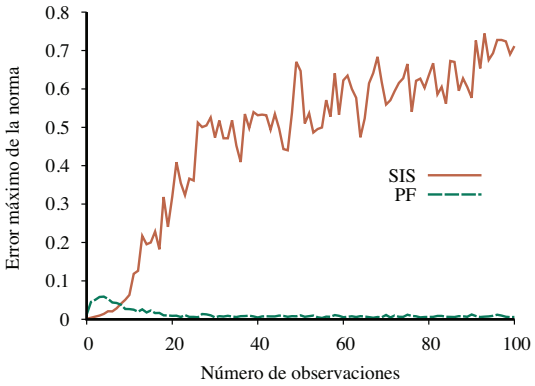


Figura 14.19 Error máximo de la norma en la estimación de la ubicación del mundo de la cuadrícula (comparado con la inferencia exacta) para la ponderación de la probabilidad (muestreo de importancia secuencial) con 100 000 muestras y filtrado de partículas con 1 000 muestras; datos promediados en 50 ejecuciones.

cuadrícula con sólo mil muestras. También funciona en problemas del mundo real: el algoritmo soporta miles de aplicaciones en ciencia e ingeniería. (Algunas referencias se dan al final del capítulo). Maneja combinaciones de variables discretas y continuas, así como modelos no lineales y no gaussianos para variables continuas. Bajo ciertas suposiciones, en particular, que las probabilidades en los modelos de transición y sensor están alejadas de 0 y 1, también es posible demostrar que la aproximación mantiene un error acotado con alta probabilidad, como sugiere la figura.

Sin embargo, el algoritmo de filtrado de partículas tiene debilidades. Veamos cómo se desempeña para el mundo del vacío con suciedad añadida. Recuerde de la Sección 14.3.2 que esto aumenta el tamaño del espacio de estados por un factor de 2^{42}, haciendo la inferencia HMM exacta inviable. Queremos que el robot deambule y construya un mapa de dónde está ubicada la suciedad. (Este es un ejemplo **simple de localización y mapeo simultáneos** o **SLAM**, que cubrimos con más profundidad en Capítulo 26). Dejemos que $Sucio_{i,t}$ signifique que la casilla i está sucia en el tiempo t y que $SensorSucio_t$ sea verdadero si y sólo si el robot detecta suciedad en el tiempo t. Asumiremos que, en cualquier casilla dada, la suciedad persiste con probabilidad p, mientras que una casilla limpia se ensucia con probabilidad $1 - p$ (lo que significa que cada casilla está sucia la mitad del tiempo, en promedio). El robot tiene un sensor de suciedad para su ubicación actual; el sensor es preciso con probabilidad 0.9. La Figura 14.20 ilustra el DBN.

Para simplificar, comenzaremos asumiendo que el robot tiene un sensor de ubicación perfecto, en lugar del sensor de pared ruidoso. El rendimiento del algoritmo se muestra en la Figura 14.21(a), donde sus estimaciones para la suciedad se comparan con los resultados de la inferencia exacta. (Veremos en breve cómo es posible la inferencia exacta). Para valores bajos de la persistencia de la suciedad p, el error permanece pequeño, pero esto no es un gran logro, porque para cada casilla el posterior verdadero para la suciedad está cerca de 0.5 si el robot no ha visitado esa casilla en un tiempo reciente. Para valores más altos de p, la suciedad permanece más tiempo, por lo que visitar una casilla proporciona información más útil que es válida durante un periodo más largo. Quizás, el filtrado de partículas *empeora* para valores más altos de p. Falla completamente cuando $p = 1$, aunque eso parece el caso más fácil: la suciedad llega en el tiempo 0 y permanece para siempre, por lo que después de algunas vueltas al mundo, el robot debería tener un mapa de suciedad casi perfecto ¿Por qué falla el filtrado de partículas en este caso?

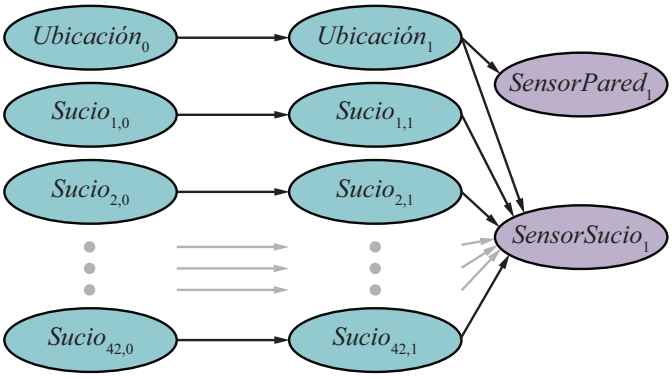


Figura 14.20 Una red de Bayes dinámica para la localización y mapeo simultáneos en el mundo estocástico y sucio de la aspiradora. Las casillas sucias persisten con probabilidad p, y las casillas limpias se ensucian con probabilidad $1-p$. El sensor local de suciedad es 90 % preciso, para la casilla en la que el robot se encuentra en el tiempo actual.

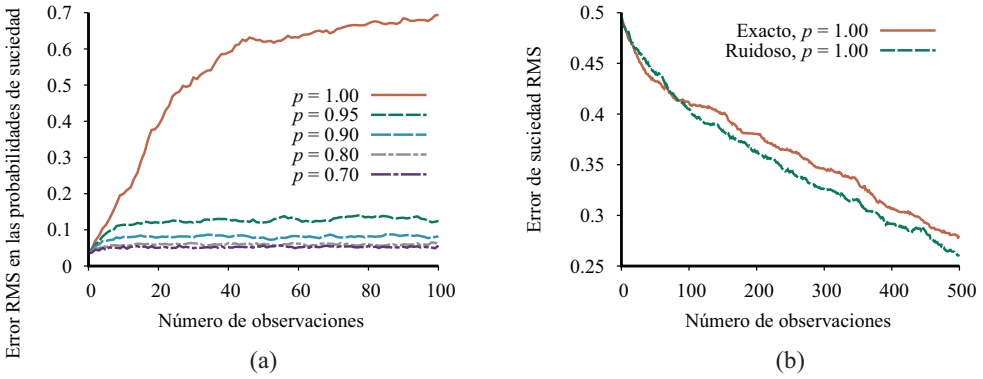


Figura 14.21 (a) Rendimiento del algoritmo estándar de filtrado de partículas con 1 000 partículas que indica el error RMS en las probabilidades marginales de suciedad en comparación con la inferencia exacta para diferentes valores de la persistencia de la suciedad p. (b) Rendimiento del filtrado de partículas Rao-Blackwellizado (100 partículas) en comparación con la verdad de campo, tanto para la detección de ubicación exacta como para la detección de pared ruidosa y con suciedad determinista. Datos promedidos en 20 ejecuciones.

Resulta que la condición teórica que requiere que "las probabilidades en los modelos de transición y sensor son mayores que 0 y menores que 1" es más que una mera pedantería matemática. Lo que sucede es que primero cada partícula inicialmente contiene 42 suposiciones de $\mathbf{P}(\mathbf{X}_0)$ sobre qué casillas tienen suciedad y cuáles no. Luego, el estado de cada partícula se proyecta hacia adelante en el tiempo de acuerdo con el modelo de transición. Desafortunadamente, el modelo de transición para la suciedad determinista es determinista: la suciedad se queda donde está. Por lo tanto, las suposiciones iniciales en cada partícula nunca se actualizan con la evidencia.

La posibilidad de que las suposiciones iniciales sean todas correctas es 2^{-42} o alrededor de 2×10^{-13}, por lo que es improbable que mil partículas (o aun un millón de partículas) incluyan una con el mapa de suciedad correcto. En la mayoría de los casos, la mejor partícula de mil acertará alrededor de 32 y fallará 10, y, en general, sólo habrá una partícula así o quizás unas pocas. Una de esas mejores partículas llegará a dominar la probabilidad total a medida que avanza el tiempo y la diversidad de la población de partículas colapsará. Entonces, porque todas las partículas están de acuerdo en un único mapa incorrecto, el algoritmo se convence de que ese mapa es correcto y nunca cambia de opinión.

Por fortuna, el problema de la localización y mapeo simultáneos tiene una estructura especial: condicionado a la secuencia de ubicaciones del robot, los estados de suciedad de las casillas individuales son independientes (Ejercicio 14.RBPF). En particular,

$$\mathbf{P}(Sucio_{1,0:t},\ldots,Sucio_{42,0:t}|SensorSucio_{1:t},SensorPared_{1:t},Ubicación_{1:t}) = \prod_i \mathbf{P}(Sucio_{i,0:t}|SensorSucio_{1:t}, Ubicación_{1:t}). \quad (14.24)$$

Rao-Blackwellización

Esto significa que es útil aplicar un truco estadístico llamado **Rao-Blackwellización**, que se basa en la idea de que la inferencia exacta siempre es más precisa que el muestreo, aun si es sólo para un subconjunto de las variables. (Vea el Ejercicio 14.RAOB). Para el problema SLAM, ejecutamos el filtrado de partículas en la ubicación del robot y luego, para cada partícula, ejecutamos la inferencia HMM exacta para cada casilla de suciedad de forma independiente, condicionada a la secuencia de ubicaciones en esa partícula. Por lo tanto, cada partícula contiene una ubicación muestreada, más 42 posteriores marginales exactos para las 42 casillas exactas; es decir, asumiendo que la trayectoria de ubicación hipotetizada seguida por esa partícula es correcta. Este enfoque, llamado **filtro de partículas Rao-Blackwellizado**, maneja el caso de la suciedad determinista sin dificultad, construyendo de manera gradual un mapa de suciedad exacto con detección de ubicación exacta o detección de pared ruidosa, como se muestra en la Figura 14.21(b).

Filtro de partículas Rao-Blackwellizado

En casos que no satisfacen el tipo de estructura de independencia condicional ejemplificada por la Ecuación (14.24), la Rao-Blackwellización no es aplicable. Las notas al final del capítulo mencionan una serie de algoritmos propuestos para manejar el problema general de filtrado con variables estáticas. Ninguno tiene la elegancia y la aplicabilidad general del filtro de partículas, pero varios son efectivos en la práctica en ciertas clases de problemas.

Resumen

Este capítulo ha abordado el problema general de representar y razonar sobre procesos temporales probabilísticos. Los puntos principales son:

- El cambio de estado del mundo se maneja usando un conjunto de variables aleatorias para representar el estado en cada punto en el tiempo.
- Las representaciones permiten diseñarse para (de manera aproximada) satisfacer la **propiedad de Markov**, de modo que el futuro es independiente del pasado dado el presente. Combinado con la suposición de que el proceso es **homogéneo en el tiempo**, esto simplifica la representación.
- Un modelo de probabilidad temporal puede considerarse como que contiene un **modelo de transición** que describe la evolución del estado y un **modelo de sensor** que describe el proceso de observación.

- Las principales tareas de inferencia en modelos temporales son el **filtrado (estimación del estado)**, la **predicción**, el **suavizado** y el cálculo de la **explicación más probable**. Cada una de estas tareas puede lograrse utilizando algoritmos recursivos simples cuyo tiempo de ejecución es lineal en la longitud de la secuencia.
- Se estudiaron en mayor profundidad tres familias de modelos temporales: **modelos de Markov ocultos**, **filtros de Kalman** y **redes bayesianas dinámicas** (que incluyen los otros dos como casos especiales).
- A menos que se hagan suposiciones especiales, como en los filtros de Kalman, la inferencia exacta con muchas variables de estado es intratable. En la práctica, el algoritmo de **filtrado de partículas** y sus descendientes son una familia efectiva de algoritmos de aproximación.

Notas bibliográficas e históricas

Muchas de las ideas básicas para estimar el estado de los sistemas dinámicos provinieron del matemático C. F. Gauss (1809), quien formuló un algoritmo determinista de mínimos cuadrados para el problema de estimar órbitas a partir de observaciones astronómicas. A. A. Markov (1913) desarrolló lo que más tarde se llamó la **suposición de Markov** en su análisis de procesos estocásticos; estimó una cadena de Markov de primer orden en las letras de la novela *Eugene Onegin*. La teoría general de las cadenas de Markov y sus tiempos de mezcla está cubierta por Levin *et al.* (2008).

Durante la Segunda Guerra Mundial se realizó un trabajo clasificado significativo sobre filtrado por Wiener (1942) para procesos en tiempo continuo y por Kolmogorov (1941) para procesos en tiempo discreto. Aunque este trabajo condujo a importantes desarrollos tecnológicos durante los siguientes 20 años, su uso de una representación de dominio de frecuencia hizo que muchos cálculos fueran bastante engorrosos. La modelización directa del espacio de estados del proceso estocástico resultó ser más sencilla, como mostraron Peter Swerling (1959) y Rudolf Kalman (1960). Este último papel describió lo que ahora se conoce como el filtro de Kalman para la inferencia hacia adelante en sistemas lineales con ruido gaussiano; sin embargo, los resultados de Kalman ya habían sido obtenidos previamente por el astrónomo danés Thorvold Thiele (1880) y por el físico ruso Ruslan Stratonovich (1959). Después de una visita al Centro de Investigación Ames de la NASA en 1960, Kalman vio la aplicabilidad del método para el seguimiento de trayectorias de cohetes, y el filtro se implementó más tarde para las misiones Apollo.

Los resultados clave sobre el alisado fueron derivados por Rauch *et al.* (1965), y el llamado alisador Rauch–Tung–Striebel sigue siendo una técnica estándar hoy en día. Muchos resultados tempranos se recogen en Gelb (1974). Bar-Shalom y Fortmann (1988) dan un tratamiento más moderno con un sabor bayesiano, así como muchas referencias a la vasta literatura sobre el tema. Chatfield (1989) y Box *et al.* (2016) cubren el enfoque de la teoría del control para el análisis de series de tiempo.

El modelo de Markov oculto y los algoritmos asociados para la inferencia y el aprendizaje, incluido el algoritmo forward-backward, fueron desarrollados por Baum y Petrie (1966). El algoritmo de Viterbi apareció por primera vez en (Viterbi, 1967). Ideas similares también aparecieron de manera independiente en la comunidad de filtrado de Kalman (Rauch *et al.*, 1965).

El algoritmo forward-backward fue uno de los principales precursores de la formulación general del algoritmo EM (Dempster *et al.,* 1977); vea también el Capítulo 20. El alisado de espacio constante aparece en Binder *et al.* (1997b), al igual que el algoritmo de dividir y conquistar desarrollado en el Ejercicio 14.ISLE. El alisado de retardo fijo en tiempo constante para HMMs apareció por primera vez en Russell y Norvig (2003).

Los HMMs han encontrado muchas aplicaciones en el procesamiento del lenguaje (Charniak, 1993), reconocimiento del habla (Rabiner y Juang, 1993), traducción automática (Och y Ney,

2003), computational biology (Krogh *et al.,* 1994; Baldi *et al.,* 1994), economía financiera (Bhar y Hamori, 2004) y otros campos. Ha habido varias extensiones al modelo básico de HMM: por ejemplo, el HMM Jerárquico (Fine *et al.*, 1998) y el HMM en Capas (Oliver *et al.*, 2004) introducen estructura de nuevo en el modelo, reemplazando la única variable de estado de los HMMs.

Las redes bayesianas dinámicas (DBNs) permiten ser vistas como una codificación escasa de un proceso de Markov y fueron utilizadas por primera vez en AI por Dean y Kanazawa (1989b), Nicholson y Brady (1992), y Kjaerulff (1992). Este último trabajo extiende el sistema de redes bayesianas HUGIN para acomodar redes bayesianas dinámicas. El libro de Dean y Wellman (1991) ayudó a popularizar las DBNs y el enfoque probabilístico para la planificación y el control dentro de la IA. Murphy (2002) proporciona un análisis exhaustivo de las DBNs.

Las redes bayesianas dinámicas se han vuelto populares para modelar una variedad de procesos de movimiento complejos en visión por computadora (Huang *et al.*, 1994; Intille y Bobick, 1999). Al igual que los HMMs, han encontrado aplicaciones en el reconocimiento del habla (Zweig y Russell, 1998; Livescu *et al.*, 2003), localización de robots (Theocharous *et al.*, 2004), y genómica (Murphy y Mian, 1999; Li *et al.,* 2011). Otras áreas de aplicación incluyen el análisis de gestos (Suk *et al.*, 2010), la detección de fatiga del conductor (Yang *et al.,* 2010), y la modelización del tráfico urbano (Hofleitner *et al.*, 2012).

La relación entre HMMs y DBNs, y entre el algoritmo forward–backward y la propagación de la red bayesiana, fue explicada por Smyth *et al.* (1997). Una mayor unificación con los filtros de Kalman (y otros modelos estadísticos) aparece en Roweis y Ghahramani (1999). Existen procedimientos para aprender los parámetros (Binder *et al.,* 1997a; Ghahramani, 1998) y las estructuras (Friedman *et al.*, 1998) de los DBNs. Las **redes bayesianas en tiempo continuo** (Nodelman *et al.*, 2002) son el análogo de los DBNs en estado discreto y tiempo continuo, evitando la necesidad de elegir una duración particular para los pasos de tiempo.

Los primeros algoritmos de muestreo para el filtrado (también llamados métodos de Monte Carlo secuenciales) fueron desarrollados en la comunidad de teoría de control por Handschin y Mayne (1969), y la idea de remuestreo que es el núcleo del filtrado de partículas apareció en una revista de control rusa (Zaritskii *et al.*, 1975). Más tarde fue reinventado en estadística como **muestreo de importancia secuencial con remuestreo**, o **SIR** (Rubin, 1988; Liu y Chen, 1998), en teoría de control como filtrado de partículas (Gordon *et al.*, 1993; Gordon, 1994), en IA como **supervivencia del más apto** (Kanazawa *et al.*, 1995), y en visión por computadora como **condensación** (Isard y Blake, 1996).

Reversión de evidencia

El artículo de Kanazawa *et al.* (1995) incluye una mejora llamada **reversión de evidencia** en la que el estado en el tiempo $t + 1$ se muestrea condicionalmente tanto al estado en el tiempo t como a la evidencia en el tiempo $t + 1$. Esto permite que la evidencia influya de manera directa en la generación de muestras y fue probado por Doucet (1997) y Liu y Chen (1998) para reducir el error de aproximación.

El filtrado de partículas se ha aplicado en muchas áreas, incluido el seguimiento de patrones de movimiento complejos en video (Isard y Blake, 1996), la predicción del mercado de valores (de Freitas *et al.*, 2000), y el diagnóstico de fallos en rovers planetarios (Verma *et al.*, 2004). Desde su invención, se han publicado decenas de miles de artículos sobre aplicaciones y variantes del algoritmo. Las implementaciones escalables en hardware paralelo se han vuelto importantes; aunque uno podría pensar que es sencillo distribuir N partículas en hasta N hilos de procesador, el algoritmo básico requiere comunicación sincronizada entre hilos para el paso de remuestreo (Hendeby *et al.*, 2010). El **algoritmo de cascada de partículas** (Paige *et al.*, 2015) elimina el requisito de sincronización, lo que resulta en un cálculo paralelo mucho más rápido.

El **filtro de partículas Rao-Blackwellizado** se debe a Doucet *et al.* (2000) y Murphy y Russell (2001); su aplicación a problemas prácticos de localización y mapeo en robótica es descrito en el Capítulo 26. Se han propuesto muchos otros algoritmos para manejar problemas de filtrado más

generales con variables estáticas o casi estáticas, incluido el algoritmo de resample–move (Gilks y Berzuini, 2001), el algoritmo Liu–West (Liu y West, 2001), el filtro Storvik (Storvik, 2002), el filtro de parámetro extendido (Erol *et al.*, 2013), y el filtro de parámetro asumido (Erol *et al.,* 2017). Este último es una combinación de filtrado de partículas con un filtro de densidad asumida mucho más antiguo llamado **filtro de densidad asumida**. Un filtro de densidad asumida supone que la distribución posterior en el tiempo t pertenece a una familia particularmente parametrizada de manera finita; si los pasos de proyección y actualización la llevan fuera de esta familia, la distribución es proyectada de nuevo para dar la mejor aproximación dentro de la familia. Para los DBNs, el algoritmo Boyen–Koller (Boyen *et al.*, 1999) y el algoritmo de **frontera factorizada** (Murphy y Weiss, 2001) suponen que la distribución posterior puede ser aproximada bien por un producto de pequeños factores.

Filtro de densidad asumida

Frontera factorizada

Los métodos MCMC (ver Sección 13.4.2) permiten ser aplicados al problema de filtrado; por ejemplo, el muestreo de Gibbs puede ser aplicado de manera directa a un DBN desenrollado. La familia de algoritmos **MCMC de partículas** (Andrieu *et al.,* 2010; Lindsten *et al.*, 2014) combina MCMC en el modelo temporal desenrollado con filtrado de partículas para generar las propuestas MCMC; aunque se demuestra que converge a la distribución posterior correcta en el caso general (es decir, con variables estáticas y dinámicas), es un algoritmo offline. Para evitar el problema de aumentar los tiempos de actualización a medida que crece la red desenrollada, el filtro **MCMC decayed** (Marthi *et al.*, 2002) prefiere muestrear variables de estado más recientes, con una probabilidad que disminuye para variables más antiguas.

MCMC de partículas

MCMC decayed

El libro de Doucet *et al.* (2001) recoge muchos artículos importantes sobre los algoritmos **de Monte Carlo secuenciales** (SMC), de los cuales el filtrado de partículas es la instancia más importante. Hay Monte Carlo Secuencial tutoriales útiles de Arulampalam *et al.* (2002) y Doucet y Johansen (2011). También hay varios resultados teóricos sobre las condiciones bajo las cuales los métodos SMC mantienen un error acotado de manera indefinida en comparación con la verdadera posterior (Crisan y Doucet, 2002; Del Moral, 2004; Del Moral *et al.*, 2006).

Monte Carlo secuenciales

CAPÍTULO 15

PROGRAMACIÓN PROBABILÍSTICA

En el que explicamos la idea de lenguajes universales para la representación y la inferencia del conocimiento probabilístico en dominios inciertos.

El espectro de representaciones —atómicas, factorizadas y estructuradas— ha sido un tema persistente en la IA. Para modelos deterministas, los algoritmos de búsqueda asumen sólo una representación atómica; los CSPs y la lógica proposicional ofrecen representaciones factorizadas; mientras que la lógica de primer orden y los sistemas de planificación aprovechan las representaciones estructuradas. El poder expresivo que ofrecen las representaciones estructuradas produce modelos mucho más concisos que las descripciones factorizadas o atómicas equivalentes.

Para los modelos probabilísticos, las redes bayesianas descritas en los Capítulos 13 y 14 son representaciones factorizadas: el conjunto de variables aleatorias es fijo y finito, y cada una tiene un rango fijo de valores posibles. Este hecho limita la aplicabilidad de las redes bayesianas, porque la representación de la red bayesiana para un dominio complejo es demasiado grande. Esto hace inviable tanto la construcción de tales representaciones a mano como aprenderlas de cualquier cantidad razonable de datos.

El problema de crear un lenguaje formal expresivo para la información probabilística ha desafiado a algunas de las mentes más grandes de la historia, entre ellos Gottfried Leibniz (coinventor del cálculo), Jacob Bernoulli (descubridor del cálculo de variaciones, y la Ley de los Grandes Números), Augustus De Morgan, George Boole, Charles Sanders Peirce (uno de los principales lógicos del siglo XIX), John Maynard Keynes (el principal economista del siglo XX), y Rudolf Carnap (uno de los más grandes filósofos analíticos del siglo XX). El problema resistió estos y muchos otros esfuerzos hasta la década de 1990.

Gracias en parte al desarrollo de las redes bayesianas, ahora existen lenguajes formales matemáticamente elegantes y prácticos que permiten la creación de modelos probabilísticos para dominios muy complejos. Estos lenguajes son *universales* en el mismo sentido que las máquinas de Turing son universales: pueden representar cualquier modelo de probabilidad computable, al igual que las máquinas de Turing permiten representar cualquier función computable. Además, estos lenguajes vienen con algoritmos de inferencia de propósito general, análogos a los algoritmos de inferencia lógica sólidos y completos como la resolución.

Hay dos formas de introducir poder expresivo en la teoría de la probabilidad. Una es mediante la lógica: diseñar un lenguaje que defina probabilidades sobre mundos posibles de primer orden, en lugar de los mundos posibles proposicionales de las redes de Bayes. Este camino se cubre en las Secciones 15.1 y 15.2, mientras que la Sección 15.3 cubre el caso específico del razonamiento temporal. La segunda ruta es a través de los lenguajes de programación tradicionales: introducimos elementos estocásticos —elecciones aleatorias, por ejemplo— en estos lenguajes, y revisamos los programas como definiciones de distribuciones de probabilidad sobre sus propias trazas de ejecución. Este enfoque se cubre en la Sección 15.4.

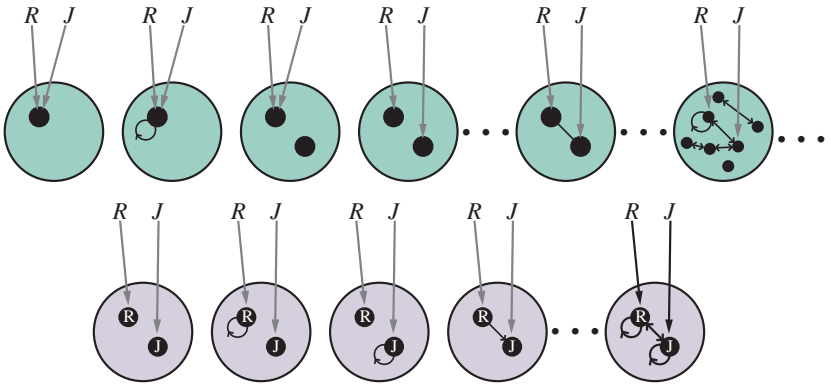


Figura 15.1 Arriba: Algunos miembros del conjunto de todos los mundos posibles para un lenguaje con dos símbolos constantes, *R* y *J*, y un símbolo de relación binaria bajo la semántica estándar para la lógica de primer orden. Abajo: los mundos posibles bajo la semántica de la base de datos. La interpretación de los símbolos constantes es fija, y hay un objeto distinto para cada símbolo constante.

Ambos caminos conducen a un **lenguaje de programación probabilística (PPL)**. El primer camino conduce a PPLs declarativas, que tienen la misma relación con las PPLs generales que la programación lógica (Capítulo 9) tiene con los lenguajes de programación generales.

Lenguaje de programación probabilística (PPL)

15.1 Modelos de probabilidad relacionales

Recuerde del Capítulo 12 que un modelo de probabilidad define un conjunto Ω de mundos posibles con una probabilidad $P(\omega)$ para cada mundo ω. Para las redes bayesianas, los mundos posibles son asignaciones de valores a variables; para el caso booleano en particular, los mundos posibles son idénticos a los de la lógica proposicional.

Entonces, para un modelo de probabilidad de primer orden, parece que necesitamos que los mundos posibles sean los de la lógica de primer orden; es decir, un conjunto de objetos con relaciones entre ellos y una interpretación que mapea símbolos constantes a objetos, símbolos de predicado a relaciones, y símbolos de función a funciones sobre esos objetos. (Ver Sección 8.2). El modelo también necesita definir una probabilidad para cada uno de estos mundos posibles, al igual que una red bayesiana define una probabilidad para cada asignación de valores a variables.

Suponga, por un momento, que hemos descubierto cómo hacer esto. Entonces, como de costumbre (ver página 389), podemos obtener la probabilidad de cualquier sentencia lógica de primer orden φ (phi) como una suma sobre los mundos posibles donde es verdadera:

$$P(\varphi) = \sum_{\omega:\varphi \text{ es verdadero en } \omega} P(\omega). \tag{15.1}$$

Las probabilidades condicionales $P(\varphi|\mathbf{e})$ se obtienen de manera similar, por ello podemos, en principio, plantear cualquier pregunta que queramos a nuestro modelo y obtener una respuesta. Hasta ahora, todo bien.

Sin embargo, hay un problema: el conjunto de modelos de primer orden es infinito. Vimos esto de manera explícita en la Figura 8.4 en la página 259, que mostramos nuevamente en la Figura 15.1 (arriba). Esto significa que (1) la suma en la Ecuación (15.1) podría ser inviable, y (2) especificar una distribución completa y consistente sobre un conjunto infinito de mundos podría ser muy difícil.

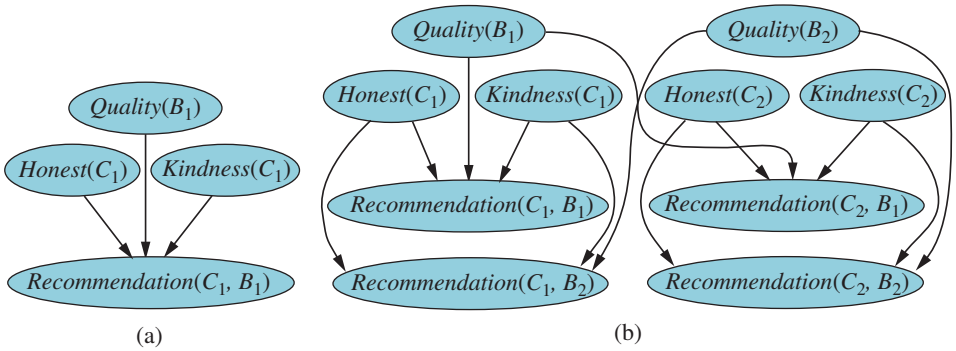


Figura 15.2 (a) Red de Bayes para un solo cliente C_1 recomendando un solo libro B_1. *Honest*(C_1) es booleano, mientras que las otras variables tienen valores enteros del 1 al 5. (b) Red de Bayes con dos clientes y dos libros.

En esta sección, evitamos este problema considerando la **semántica de la base de datos** definida en la Sección 8.2.8 (página 264). La semántica de la base de datos hace la **suposición de nombres únicos**; aquí, la adoptamos para los símbolos constantes. También asume el **cierre del dominio**, no hay más objetos más allá de los que se nombran. Podemos garantizar un conjunto finito de mundos posibles haciendo que el conjunto de objetos en cada mundo sea el conjunto de símbolos constantes que se utilizan; como se muestra en la Figura 15.1 (abajo), no hay incertidumbre sobre la asignación de símbolos a objetos o sobre los objetos que existen.

Modelo de probabilidad relacional

Llamaremos a los modelos definidos de esta manera **modelos de probabilidad relacional**, o **RPMs**.[1] La diferencia más significativa entre la semántica de los RPMs y la semántica de la base de datos introducida en la Sección 8.2.8 es que los RPMs no hacen la suposición del mundo cerrado, en un sistema de razonamiento probabilístico no es posible asumir que cada hecho desconocido es falso.

15.1.1 Sintaxis y semántica

Comencemos con un ejemplo simple: supongamos que un minorista de libros en línea quisiera proporcionar evaluaciones generales de productos basadas en recomendaciones recibidas de sus clientes. La evaluación tomará la forma de una distribución posterior sobre la calidad del libro, dada la evidencia disponible. La solución más sencilla es evaluar con base en la recomendación promedio, quizá con una varianza determinada por el número de recomendaciones, pero esto no tiene en cuenta el hecho de que algunos clientes son más amables que otros y algunos son menos honestos que otros. Los clientes amables tienden a dar buenas recomendaciones aun a libros mediocres, mientras que los clientes deshonestos dan recomendaciones muy altas o muy bajas por razones distintas a la calidad pues podrían estar pagados para promocionar los libros de algún editor.[2]

Para un solo cliente C_1 recomendando un solo libro B_1, la red de Bayes podría parecerse a la mostrada en la Figura 15.2(a). (Al igual que en la Sección 9.1, las expresiones con paréntesis como *Honest*(C_1) son sólo símbolos elegantes, en este caso, nombres elegantes para variables aleatorias).

[1] El nombre *modelo de probabilidad relacional* fue asignado por Pfeffer (2000) a una representación diferente, pero las ideas subyacentes son las mismas.

[2] Un teórico de juegos aconsejaría a un cliente deshonesto evitar la detección recomendando de manera ocasional un buen libro de un competidor. Ver Capítulo 18.

Con dos clientes y dos libros, la red de Bayes se parece a la que se muestra en la Figura 15.2(b). Para un mayor número de libros y clientes, resulta impráctico especificar una red de Bayes a mano.

Por fortuna, la red tiene mucha estructura repetida. Cada variable *Recommendation*(c, b) tiene como padres las variables *Honest*(c), *Kindness*(c), y *Quality*(b). Además, las tablas de probabilidad condicional (CPTs) para todas las variables *Recommendation*(c, b) son idénticas, al igual que las de todas las variables *Honest*(c), y así sucesivamente. La situación parece hecha a medida para un lenguaje de primer orden. Nos gustaría decir algo como

$$\mathit{Recommendation}(c,b) \sim \mathit{RecCPT}(\mathit{Honest}(c), \mathit{Kindness}(c), \mathit{Quality}(b))$$

lo que significa que la recomendación de un cliente para un libro depende probabilísticamente de la honestidad y amabilidad del cliente y de la calidad del libro según una CPT fija.

Al igual que la lógica de primer orden, los RPMs tienen símbolos constantes, de función y de predicado. También asumiremos una **firma de tipo** para cada función, es decir, una especificación del tipo de cada argumento y el valor de la función. (Si se conoce el tipo de cada objeto, muchos mundos posibles espurios son eliminados por este mecanismo; por ejemplo, no necesitamos preocuparnos por la amabilidad de cada libro, libros recomendandos por clientes, y así sucesivamente). Para el dominio de recomendación de libros, los tipos son *Customer* y *Book*, y las firmas de tipo para las funciones y predicados son las siguientes:

Firma de tipo

$$\mathit{Honest}: \mathit{Customer} \rightarrow \{\mathit{true},\mathit{false}\}$$
$$\mathit{Kindness}: \mathit{Customer} \rightarrow \{1,2,3,4,5\}$$
$$\mathit{Quality}: \mathit{Book} \rightarrow \{1,2,3,4,5\}$$
$$\mathit{Recommendation}: \mathit{Customer} \times \mathit{Book} \rightarrow \{1,2,3,4,5\}$$

Los símbolos constantes serán los nombres de clientes y libros que aparezcan en el conjunto de datos del minorista. En el ejemplo de la Figura 15.2(b), estos eran C_1, C_2 y B_1, B_2.

Dadas las constantes y sus tipos, junto con las funciones y sus firmas de tipo, las **variables aleatorias básicas** del **RPM** se obtienen instanciando cada función con cada posible combinación de objetos. Para el modelo de recomendación de libros, las variables aleatorias básicas incluyen *Honest*(C_1), *Quality*(B_2), *Recommendation*(C_1,B_2), y así sucesivamente. Estas son las variables que aparecen en la Figura 15.2(b). Debido a que cada tipo tiene únicamente múltiples instancias finitas (gracias a la suposición de cierre de dominio), el número de variables aleatorias básicas también es finito.

Variable aleatoria básica

Para completar el RPM, tenemos que escribir las dependencias que rigen estas variables aleatorias. Hay una declaración de dependencia para cada función, donde cada argumento de la función es una variable lógica (es decir, una variable que varía sobre objetos, como en la lógica de primer orden). Por ejemplo, la siguiente dependencia establece que, para cada cliente c, la probabilidad previa de honestidad es 0.99 *verdadero* y 0.01 *falso*:

$$\mathit{Honest}(c) \sim \langle 0.99, 0.01 \rangle$$

De manera similar, podemos establecer probabilidades previas para el valor de amabilidad de cada cliente y la calidad de cada libro, cada uno en la escala 1–5:

$$\mathit{Kindness}(c) \sim \langle 0.1, 0.1, 0.2, 0.3, 0.3 \rangle$$
$$\mathit{Quality}(b) \sim \langle 0.05, 0.2, 0.4, 0.2, 0.15 \rangle$$

Finalmente, necesitamos la dependencia para las recomendaciones: para cualquier cliente c y libro b, la puntuación depende de la honestidad y amabilidad del cliente y la calidad del libro:

$$\mathit{Recommendation}(c,b) \sim \mathit{RecCPT}(\mathit{Honest}(c), \mathit{Kindness}(c), \mathit{Quality}(b))$$

donde *RecCPT* es una tabla de probabilidad condicional definida por separado con $2 \times 5 \times 5 = 50$ filas, cada una con 5 entradas. Para fines de ilustración, asumiremos que una recomendación honesta para un libro de calidad q de una persona de amabilidad k se distribuye de manera uniforme en el rango $[\lfloor \frac{q+k}{2} \rfloor, \lceil \frac{q+k}{2} \rceil]$.

La semántica del RPM se obtiene instanciando estas dependencias para todos constantes conocidas, dando una red bayesiana (como en la Figura 15.2(b)) que define una distribución conjunta sobre las variables aleatorias del RPM.[3]

El conjunto de mundos posibles es el producto cartesiano de los rangos de todas las variables aleatorias básicas, y, como con las redes bayesianas, la probabilidad para cada mundo posible es el producto de las probabilidades condicionales relevantes del modelo. Con C clientes y B libros, hay variables *C Honest*, *C Kindness*, *B Quality* y *BC Recommendation*, lo que lleva a $2^C 5^{C+B+BC}$ mundos posibles. Con diez millones de libros y mil millones de clientes, eso es alrededor de $10^7 \times 10^{15}$ mundos. Gracias al poder expresivo de RPMs, el modelo de probabilidad completo todavía tiene sólo menos de 300 parámetros, la mayoría de ellos en la tabla RecCPT.

Podemos refinar el modelo afirmando una **independencia específica del contexto** (ver página 420) para reflejar el hecho de que los clientes deshonestos ignoran la calidad al dar una recomendación; además, la amabilidad no tiene ningún papel en sus decisiones. Así, *Recommendation*(*c*,*b*) es independiente de *Kindness(c)* y *Quality*(*b*) cuando *Honest*(*c*) = *falso:*

Recommendation(*c*,*b*) ∼ **if** *Honest*(*c*) **then**
HonestRecCPT(*Kindness*(*c*),*Quality*(*b*))
else ⟨0.4,0.1,0.0,0.1,0.4⟩.

Este tipo de dependencia puede parecer una declaración if-then–else ordinaria en un lenguaje de programación, pero hay una diferencia clave: el motor de inferencia *no necesariamente conoce el valor de la prueba condicional* porque *Honest*(*c*) es una variable aleatoria.

Podemos elaborar este modelo de infinitas formas para hacerlo más realista. Por ejemplo, suponga que un cliente honesto que es fan del autor de un libro siempre le da al libro un 5, independiente de la calidad:

Recommendation(*c*,*b*) ∼ **if** *Honest*(*c*) **then**
if *Fan*(*c*,*Author*(*b*)) **then** *Exactly*(5)
else *HonestRecCPT*(*Kindness*(*c*),*Quality*(*b*))
else ⟨0.4,0.1,0.0,0.1,0.4⟩

De nuevo, la prueba condicional *Fan*(c,*Author*(*b*)) es desconocida, pero si un cliente sólo da cincos a los libros de un autor en particular y no es muy amable, entonces la probabilidad posterior de que el cliente sea un fan de ese autor será alta. Además, la distribución posterior tenderá a descontar los cincos del cliente al evaluar la calidad de los libros de ese autor.

En este ejemplo, asumimos de manera implícita que el valor de *Author*(*b*) se conoce para cada *b*, pero esto puede no ser el caso. ¿Cómo puede el sistema razonar sobre si, digamos, C_1 es un fan de *Author*(B_2) cuando *Author*(B_2) es desconocido? La respuesta es que el sistema tiene que razonar sobre todos los autores posibles. Supongamos (para simplificar) que sólo hay dos autores, A_1 y A_2. Entonces *Author*(B_2) es una variable aleatoria con dos posibles valores, A_1 y A_2, y es un

[3] Se requieren algunas condiciones técnicas para que un RPM defina una distribución adecuada. Primero, las dependencias deben ser *acíclicas*; de lo contrario, la red bayesiana resultante tendrá ciclos. Segundo, las dependencias deben estar bien fundamentadas: no puede haber cadenas de antepasados infinitas, como las que podrían surgir de dependencias recursivas. Vea el Ejercicio 15.HAMD para una excepción a esta regla.

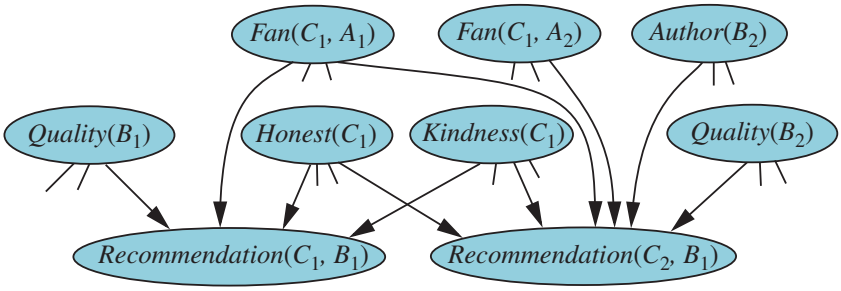


Figura 15.3 Fragmento de la red de Bayes equivalente para el RPM de recomendación de libros cuando *Author*(B_2) es desconocido.

padre de *Recommendation*(C_1,B_2). Las variables *Fan*(C_1,A_1) y *Fan*(C_1,A_2) también son padres. La distribución condicional para *Recommendation*(C_1,B_2) es entonces en esencia un **multiplexor** en el que el padre *Author*(B_2) actúa como un selector para elegir cuál de *Fan*(C_1,A_1) y *Fan*(C_1, A_2) llega a influir en la recomendación. Un fragmento de la red de Bayes equivalente se muestra en la Figura 15.3. La incertidumbre en el valor de *Author*(B_2), que afecta a la estructura de dependencia de la red, es un ejemplo de **incertidumbre relacional**.

Multiplexor

Incertidumbre relacional

En caso de que se pregunte cómo el sistema permite averiguar quién es el autor de B_2: considere la posibilidad de que tres otros clientes sean fans de A_1 (y no tengan otros autores favoritos en común) y todos tres hayan dado a B_2 un 5, aunque la mayoría de los otros clientes lo encuentren bastante sombrío. En ese caso, es probable que A_1 sea el autor de B_2. La aparición de un razonamiento sofisticado como este a partir de un modelo RPM de sólo unas pocas líneas es un ejemplo intrigante de cómo las influencias probabilísticas se propagan a través de la red de interconexiones entre los objetos en el modelo. A medida que se añaden más dependencias y más objetos, la imagen transmitida por la distribución posterior con frecuencia se vuelve más y más clara.

15.1.2 Ejemplo: Calificación de los niveles de habilidad de los jugadores

Numerosos juegos competitivos tienen una medida numérica de los niveles de habilidad de los jugadores, a veces llamada **calificación**. Quizá la más conocida es la calificación Elo para los jugadores de ajedrez, que califica a un principiante típico en alrededor de 800 y al campeón del mundo en algún lugar por encima de 2 800. Aunque las calificaciones Elo tienen una base estadística, tienen algunos elementos *ad hoc*. Podemos desarrollar un esquema de calificación bayesiano de la siguiente manera: cada jugador i tiene un nivel de habilidad subyacente *Skill*(i); en cada juego g, el rendimiento real de i es *Performance*(i,g), que puede variar del nivel de habilidad subyacente; y el ganador de g es el jugador cuyo rendimiento en g es mejor. Como un RPM, el modelo se ve así:

Calificación

$$\textit{Skill}(i) \sim \mathcal{N}(\mu,\sigma^2)$$
$$\textit{Performance}(i,g) \sim \mathcal{N}(\textit{Skill}(i),\beta^2)$$
$$\textit{Win}(i,j,g) = \textbf{if } \textit{Game}(g,i,j) \textbf{ then}(\textit{Performance}(i,g) > \textit{Performance}(j,g))$$

donde β^2 es la varianza del rendimiento real de un jugador en cualquier juego específico en relación con el nivel de habilidad subyacente del jugador. Dado un conjunto de jugadores y juegos, así como los resultados de algunos de los juegos, un motor de inferencia RPM permite calcular una distribución posterior sobre la habilidad de cada jugador y el probable resultado de cualquier juego adicional que pueda jugarse.

Para los juegos de equipo, asumiremos como una primera aproximación que el rendimiento general del equipo t en el juego g es la suma de los rendimientos individuales de los jugadores en t:

$$TeamPerformance(t,g) = \sum_{i \in t} Performance(i,g).$$

Aunque los rendimientos individuales no son visibles para el motor de calificaciones, los niveles de habilidad de los jugadores todavía permiten estimarse a partir de los resultados de varios juegos, siempre que las composiciones de los equipos varíen a lo largo de los juegos. El motor de calificaciones TrueSkill™ de Microsoft utiliza este modelo, junto con un algoritmo de inferencia aproximada eficiente, para servir a cientos de millones de usuarios todos los días.

Este modelo permite ser elaborado de numerosas formas. Por ejemplo, podríamos suponer que los jugadores más débiles tienen una mayor variación en su rendimiento; podríamos incluir el papel del jugador en el equipo, y podríamos considerar tipos específicos de rendimiento y habilidad; por ejemplo, defender y atacar, para mejorar la composición del equipo y la precisión predictiva.

15.1.3 Inferencia en modelos de probabilidad relacionales

El enfoque más directo para la inferencia en RPMs es construir la red bayesiana equivalente, dada los símbolos constantes conocidos que pertenecen a cada tipo. Con B libros y C clientes, el modelo básico dado con anterioridad podría ser construido con bucles simples:[4]

for $b = 1$ **to** B *do*
  add node $Quality_b$ with no parents, prior $\langle 0.05,0.2,0.4,0.2,0.15 \rangle$
for $c = 1$ **to** C *do*
  add node $Honest_c$ with no parents, prior $\langle 0.99,0.01 \rangle$
  add node $Kindness_c$ with no parents, prior $\langle 0.1,0.1,0.2,0.3,0.3 \rangle$
  for $b = 1$ **to** B *do*
    add node $Recommendation_{c,b}$ with parents $Honest_c, Kindness_c, Quality_b$
    and conditional distribution $RecCPT(Honest_c, Kindness_c, Quality_b)$

Concreta
Desenrollado

Esta técnica se llama **concreta** o **desenrollado**; es el análogo exacto de la **proposicionalización** para la lógica de primer orden (página 280). El inconveniente obvio es que la red de Bayes resultante es muy grande. Además, si hay muchos objetos candidatos para una relación o función desconocida; por ejemplo, el autor desconocido de B_2, entonces algunas variables en la red permiten tener muchos padres.

Afortunadamente, es posible evitar generar toda la red de Bayes implícita. Como vimos en la discusión del algoritmo de eliminación de variables en la página 433, cada variable que no es un ancestro de una variable de consulta o variable de evidencia es irrelevante para la consulta. Además, si la consulta es condicionalmente independiente de alguna variable dada la evidencia, entonces esa variable también es irrelevante. Por lo tanto, al encadenar a través del modelo a partir de la consulta y la evidencia, podemos identificar sólo el conjunto de variables relevantes para la consulta. Estas son las únicas que necesitan ser instanciadas para crear un fragmento minúsculo de la red de Bayes implícita. La inferencia en este fragmento da la misma respuesta que la inferencia en toda la red de Bayes implícita.

Otra vía para mejorar la eficiencia de la inferencia proviene de la presencia de una subestructura repetida en la red de Bayes desenrollada. Esto significa que muchos de los factores construidos

[4] Varios paquetes estadísticos verían este código como la *definición* del RPM, en lugar de construir una red de Bayes para realizar inferencias en el RPM. Esta visión, sin embargo, pasa por alto un papel importante para la sintaxis del RPM: sin una sintaxis con semántica clara, no hay forma de que la estructura del modelo pueda ser aprendida a partir de los datos.

durante la eliminación de variables (y tipos similares de tablas construidas por algoritmos de agrupamiento) serán idénticos; los esquemas de almacenamiento en caché efectivos han producido aceleraciones de tres órdenes de magnitud para redes grandes.

En tercer lugar, los algoritmos de inferencia MCMC tienen algunas propiedades interesantes cuando se aplican a RPMs con incertidumbre relacional. MCMC funciona mediante el muestreo de mundos posibles completos, por lo que en cada estado la estructura relacional es conocida. En el ejemplo anterior, cada estado MCMC especificaría el valor de $Author(B_2)$, y, por lo tanto, los otros autores potenciales ya no son padres de los nodos de recomendación para B_2. Para MCMC, entonces, la incertidumbre relacional no causa ningún aumento en la complejidad de la red; en cambio, el proceso MCMC incluye transiciones que cambian la estructura relacional y, por lo tanto, la estructura de dependencia, de la red desenrollada.

Finalmente, en algunos casos es posible evitar concretar el modelo por completo. Los demostradores de teoremas de resolución y los sistemas de programación lógica evitan la proposicionalización instanciando las variables lógicas sólo según sea necesario para que la inferencia se realice; es decir, *elevan* el proceso de inferencia por encima del nivel de las oraciones proposicionales concretas y hacen que cada paso elevado haga el trabajo de muchos pasos concretos.

La misma idea permite aplicarse en la inferencia probabilística. Por ejemplo, en el algoritmo de eliminación de variables, un factor elevado representa un conjunto entero de factores concretos que asignan probabilidades a las variables aleatorias en el RPM, donde esas variables aleatorias difieren sólo en los símbolos constantes usados para construirlas. Los detalles de este método rebasan el alcance de este libro, pero se dan referencias al final del capítulo.

15.2 Modelos de probabilidad de universo abierto

Antes argumentamos que la semántica de la base de datos era apropiada para situaciones en las que conocemos el conjunto de objetos relevantes que existen y podemos identificarlos sin ambigüedades. (En particular, todas las observaciones sobre un objeto se asocian de manera correcta con el símbolo constante que lo nombra). En muchos entornos del mundo real, sin embargo, estas suposiciones son insostenibles. Por ejemplo, una editorial podría usar un ISBN (International Standard Book Number) como un símbolo constante para nombrar cada libro, aunque un "libro" lógico dado (por ejemplo, *Gone With the Wind*) tiene varios ISBNs correspondientes a tapa dura, tapa blanda, letra grande, reediciones, etc. Tendría sentido agregar recomendaciones mediante múltiples ISBNs, pero la editorial puede no saber con certeza qué ISBNs son el mismo libro. (Note que no estamos cosificando las *copias individuales* del libro, lo cual podría ser necesario para ventas de libros usados, ventas de autos, etc.). Peor aún, cada cliente se identifica por un ID de inicio de sesión, pero un cliente deshonesto puede tener miles de IDs! En el campo de la seguridad informática, estos múltiples IDs se llaman **sybils** y su uso para confundir un sistema de reputación se llama un **ataque sybil**.[5] Por lo tanto, incluso una aplicación simple en un dominio en línea relativamente bien definido implica **incertidumbre de existencia** (¿cuáles son los libros y clientes reales subyacentes a los datos observados?) e **incertidumbre de identidad** (¿qué términos lógicos se refieren al mismo objeto?).

Sybil

Ataque Sybil

Incertidumbre de existencia

Incertidumbre de identidad

Los fenómenos de incertidumbre de existencia e identidad se extienden mucho más allá de los vendedores de libros en línea. De hecho, son omnipresentes:

- Un sistema de visión no sabe qué existe, si acaso, a la vuelta de la próxima esquina, y puede que no sepa si el objeto que ve ahora es el mismo que vio hace unos minutos.

[5] El nombre "Sybil"" proviene de un famoso caso de trastorno de personalidad múltiple.

- Un sistema de comprensión de texto no sabe con antelación las entidades que se presentarán en un texto, y debe razonar sobre si frases como "Mary", "Dr. Smith", "ella", "su cardiólogo", "su madre", etc., se refieren al mismo objeto.
- Un analista de inteligencia en busca de espías nunca sabe cuántos espías hay en realidad, sólo puede adivinar si varios pseudónimos, números de teléfono, y qué avistamientos pertenecen al mismo individuo.

De hecho, gran parte de la cognición humana parece requerir aprender qué objetos existen y ser capaz de conectar observaciones, que casi nunca vienen con IDs únicos adjuntos, a objetos hipotéticos en el mundo.

Modelo de probabilidad de universo abierto

Por lo tanto, necesitamos ser capaces de definir un **modelo de probabilidad de universo abierto** (OUPM) basado en la semántica estándar de la lógica de primer orden, como se ilustra en la parte superior de la Figura 15.1. Un lenguaje para OUPMs proporciona una forma de escribir tales modelos mientras garantiza una distribución de probabilidad única y consistente sobre el espacio infinito de mundos posibles.

15.2.1 Sintaxis y semántica

La idea básica es entender cómo las redes bayesianas ordinarias y las RPMs logran definir un modelo de probabilidad único, y transferir esa percepción al entorno de primer orden. En esencia, una red de Bayes *genera* cada mundo posible, evento por evento, en el orden topológico definido por la estructura de la red, donde cada evento es una asignación de un valor a una variable. Un RPM extiende esto a conjuntos enteros de eventos, definidos por las posibles instancias de las variables lógicas en un predicado o función dado. Los OUPMs van más allá al permitir pasos generativos que *añaden objetos* al mundo posible en construcción, donde el número y tipo de objetos pueden depender de los objetos que ya están en ese mundo, así como de sus propiedades y relaciones. Es decir, el evento que se genera no es la asignación de un valor a una variable, sino la *existencia* misma de los objetos.

Declaración de número

Una forma de hacer esto en OUPMs es proporcionar **declaraciones de número** que especifican distribuciones condicionales sobre los números de objetos de varios tipos Por ejemplo, en el dominio de recomendación de libros, podríamos querer distinguir entre *customers* (personas reales) y sus IDs de *inicio de sesión*. (¡En realidad son los IDs de inicio de sesión los que hacen recomendaciones, no los clientes!) Supongamos (para mantener las cosas simples) que el número de clientes es uniforme entre 1 y 3 y el número de libros es uniforme entre 2 y 4:

$$\begin{aligned} &\#Customer \sim UniformInt(1,3) \\ &\quad \#Book \sim UniformInt(2,4). \end{aligned} \tag{15.2}$$

Esperamos que los clientes honestos tengan sólo un ID, mientras que los clientes deshonestos podrían tener entre 2 y 5 IDs:

$$\begin{aligned} \#LoginID(Owner=c) \sim \quad & \textbf{if } Honest(c) \textbf{ then } Exactly(1) \\ & \textbf{else } UniformInt(2,5). \end{aligned} \tag{15.3}$$

Esta declaración de número especifica la distribución sobre el número de identificaciones de inicio de sesión para las cuales el cliente c es el *Owner*. La función *Owner* se llama **función de origen** porque dice de dónde provino cada objeto generado por esta declaración de número.

Función de origen

El ejemplo en el párrafo anterior usa una distribución uniforme sobre los enteros entre 2 y 5 para especificar el número de inicios de sesión para un cliente deshonesto. Esta distribución

particular está acotada, pero en general es posible que no haya un límite *a priori* en el número de objetos. La distribución más usada sobre los enteros no negativos es la **distribución Poisson**. La Poisson tiene un parámetro, λ, que es el número esperado de objetos, distribución Poisson y una variable X muestreada de Poisson(λ) tiene la siguiente distribución:

Distribución Poisson

$$P(X = k) = \lambda^k e^{-\lambda}/k!.$$

La varianza de la Poisson también es λ, por lo que la desviación estándar es $\sqrt{\lambda}$. Esto significa que para valores grandes de λ, la distribución es estrecha en relación con la media; por ejemplo, si el número de hormigas en un nido se modela con una Poisson con una media de un millón, la desviación estándar es sólo de mil, o 0.1 %. Para números grandes, en la mayoría de los casos tiene más sentido usar la **distribución log-normal discreta**, apropiada cuando el logaritmo del número de objetos log-normal se distribuye de manera normal. Una forma intuitiva, que llamamos **orden de distribución de magnitud**, usa logaritmos en base 10: así, una distribución OM(3,1) tiene una media de orden de magnitud 10^3 y una desviación estándar de un orden de magnitud, es decir, la mayor parte de la masa de probabilidad cae entre 10^2 y 10^4.

Distribución log-normal discreta

Orden de distribución de magnitud

La semántica formal de los OUPMs comienza con una definición de los objetos que pueblan los mundos posibles. En la semántica estándar de la lógica de primer orden tipada, los objetos son sólo tokens numerados con tipos. En los OUPMs, cada objeto es un historial de generación; por ejemplo, un objeto podría ser "la cuarta identificación de inicio de sesión del séptimo cliente". (La razón de esta construcción ligeramente barroca se aclarará en breve). Para tipos sin funciones de origen, por ejemplo, los tipos de *Customer* y *Book* en la Ecuación (15.2), los objetos tienen un origen vacío; es decir, $\langle Customer, ,2\rangle$ se refiere al segundo cliente generado a partir de esa declaración de número. Para declaraciones de número con funciones de origen; por ejemplo, la Ecuación (15.3), cada objeto registra su origen; como, el objeto $\langle LoginID,\langle Owner,\langle Customer, ,2\rangle\rangle,3\rangle$ es el tercer inicio de sesión perteneciente al segundo cliente.

Las **variables de número** de un OUPM especifican cuántos objetos hay de cada tipo con cada origen posible en cada mundo posible; así $\#LoginID_{\langle Owner,\langle Customer,,2\rangle\rangle}(\omega)=4$ significa que en el mundo ω, el cliente 2 posee 4 IDs de inicio de sesión. Como en los modelos de probabilidad relacional, las **variables aleatorias básicas** determinan los valores de los predicados y funciones para todas las tuplas de objetos; así, $Honest_{\langle Customer,,2\rangle}(\omega)=true$ significa que en el mundo ω, el cliente 2 es *Honest*. Un mundo posible se define por los valores de todas las variables de número y variables aleatorias básicas. Un mundo puede ser generado a partir del modelo muestreando en orden topológico; la Figura 15.4 muestra un ejemplo. La probabilidad de un mundo así construido es el producto de las probabilidades de todos los valores muestreados; en este caso, 1.2672×10^{-11}. Ahora queda claro por qué cada objeto contiene su origen: esta propiedad asegura que cada mundo permite ser construido por una secuencia de generación. Si este no fuera el caso, la probabilidad de un mundo sería una suma combinatoria incómoda de todas las posibles secuencias de generación que lo crean.

Variable de número

Los modelos de universo abierto permiten tener infinitamente muchas variables aleatorias, por lo que la teoría completa implica consideraciones no triviales de teoría de la medida. Por ejemplo, las declaraciones de número con distribuciones de Poisson o de orden de magnitud permiten números ilimitados de objetos, lo que lleva a números ilimitados de variables aleatorias para las propiedades y relaciones de esos objetos. Además, los OUPMs pueden tener dependencias recursivas y tipos infinitos (enteros, cadenas, etc.). Finalmente, la corrección formal prohíbe las dependencias cíclicas y las cadenas de antepasados que retroceden de manera infinita; estas condiciones son indecidibles en general, pero ciertas condiciones suficientes sintácticas permiten ser verificadas.

Variable	Valor	Probabilidad
$\#Customer$	2	0.3333
$\#Book$	3	0.3333
$Honesth_{\langle Customer,,1\rangle}$	*true*	0.99
$Honesth_{\langle Customer,,2\rangle}$	*false*	0.01
$Kindness_{\langle Customer,,1\rangle}$	4	0.3
$Kindness_{\langle Customer,,2\rangle}$	1	0.1
$Quality_{\langle Book,,1\rangle}$	1	0.05
$Quality_{\langle Book,,2\rangle}$	3	0.4
$Quality_{\langle Book,,3\rangle}$	5	0.15
$\#LoginID_{\langle Owner,\langle Customer,,1\rangle\rangle}$	1	1.0
$\#LoginID_{\langle Owner,\langle Customer,,2\rangle\rangle}$	2	0.25
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,1\rangle\rangle,1\rangle,\langle Book,,1\rangle}$	2	0.5
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,1\rangle\rangle,1\rangle\langle Book,,2\rangle}$	4	0.5
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,1\rangle\rangle,1\rangle\langle Book,,3\rangle}$	5	0.5
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,1\rangle\langle Book,,1\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,1\rangle\langle Book,,2\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,1\rangle\langle Book,,3\rangle}$	1	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,2\rangle\langle Book,,1\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,2\rangle\langle Book,,2\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,2\rangle\langle Book,,3\rangle}$	1	0.4

Figura 15.4 Un mundo particular para la recomendación de libros OUPM. Las variables de número y las variables aleatorias básicas se muestran en orden topológico, junto con sus valores elegidos y las probabilidades para esos valores.

15.2.2 Inferencia en modelos de probabilidad de universo abierto

Debido al tamaño enorme y a veces ilimitado de la red bayesiana implícita que corresponde a un OUPM típico, desenrollarlo y realizar una inferencia exacta es muy impráctico. En cambio, debemos considerar algoritmos de inferencia aproximada como MCMC (ver Sección 13.4.2).

Hablando en términos generales, un algoritmo MCMC para un OUPM está explorando el espacio de mundos posibles definidos por conjuntos de objetos y relaciones entre ellos, como se ilustra en la Figura 15.1(arriba). Un movimiento entre estados adyacentes en este espacio permite no sólo alterar relaciones y funciones, sino también agregar o restar objetos y cambiar las interpretaciones de los símbolos constantes. Aunque cada mundo posible puede ser enorme, los cálculos de probabilidad requeridos para cada paso, ya sea en el muestreo de Gibbs o en Metropolis-Hastings, son locales y en la mayoría de los casos toman un tiempo constante. Esto se debe a que la relación de probabilidad entre los mundos vecinos depende de un subgrafo de tamaño constante alrededor de las variables, cuyos valores se cambian. Además, una consulta lógica permite evaluarse de *manera incremental* en cada mundo visitado, generalmente en tiempo constante por mundo, en lugar de volver a calcularse desde cero.

Se debe prestar una consideración especial al hecho de que un OUPM típico permite tener mundos posibles de tamaño infinito. Como ejemplo, considere el modelo de seguimiento de múltiples objetivos en la Figura 15.9: la función $X(a,t)$, que denota el estado de la aeronave a en el tiempo t, corresponde a una secuencia infinita de variables para un número ilimitado de aeronaves en cada paso. Para esta razón, MCMC para OUPMs no muestrea mundos posibles completamente especificados sino mundos *parciales,* cada uno correspondiente a un conjunto disjunto de mundos completos. Un mundo parcial

es una *instanciación mínima de autoapoyo*[6] de un subconjunto de las variables relevantes; es decir, antecesores de las variables de evidencia y consulta. Por ejemplo, las variables $X(a,t)$ para valores de t mayores que el último tiempo de observación (o el tiempo de consulta, el que sea mayor) son irrelevantes, por lo que el algoritmo puede considerar sólo un prefijo finito de la secuencia infinita.

15.2.3 Ejemplos

El caso de uso estándar para un OUPM tiene tres elementos: el *modelo*, la *evidencia* (los hechos conocidos en un escenario dado) y la *consulta*, que permite ser cualquier expresión, quizá con variables lógicas libres. La respuesta es una probabilidad conjunta posterior para cada posible conjunto de sustituciones para las variables libres, dada la evidencia, según el modelo.[7] Cada modelo incluye declaraciones de tipo, firmas de tipo para los predicados y funciones, una o más declaraciones de número para cada tipo, y una declaración de dependencia para cada predicado y función. (En los ejemplos a continuación, se omiten las declaraciones y firmas donde el significado es claro). Como en los RPM, las declaraciones de dependencia usan una sintaxis if-then-else para manejar dependencias específicas del contexto.

Coincidencia de citas

Millones de trabajos de investigación académica e informes técnicos se pueden encontrar en línea en forma de archivos pdf. En la mayoría de los casos estos documentos contienen una sección cerca del final llamada "Referencias" o "Bibliografía", en la que se proporcionan citas, cadenas de caracteres, para informar al lector sobre trabajos relacionados. Estas cadenas se localizan y "extraen" de los archivos pdf con el objetivo de crear una representación similar a una base de datos que relaciona documentos e investigadores por autoría y enlaces de citas. Sistemas como CiteSeer y Google Scholar presentan tal representación a sus usuarios; detrás de escena, los algoritmos operan para encontrar documentos, extraer las cadenas de citas e identificar los documentos reales a los que se refieren las cadenas de citas. Esta es una tarea difícil porque estas cadenas no contienen identificadores de objetos e incluyen errores de sintaxis, ortografía, puntuación y contenido. Para ilustrar esto, aquí hay dos ejemplos relativamente benignos:

1. [Lashkari *et al.,* 94] Collaborative Interface Agents, Yezdi Lashkari, Max Metral, and Pattie Maes, Proceedings of the Twelfth National Conference on Articial Intelligence, MIT Press, Cambridge, MA, 1994.
2. Metral M. Lashkari, Y. and P. Maes. Collaborative interface agents. In Conference of the American Association for Artificial Intelligence, Seattle, WA, August 1994.

La pregunta clave es una de identidad: ¿estas citas son del mismo documento o diferentes documentos? Al hacer esta pregunta, incluso los expertos no están de acuerdo o no están dispuestos a decidir, lo que indica que razonar bajo incertidumbre va a ser una parte importante para resolver este problema.[8] Los enfoques *ad hoc*, como los métodos basados en una métrica de similitud textual, con frecuencia fracasan de manera miserable. Por ejemplo, en 2002, CiteSeer informó de más de 120 libros distintos escritos por Russell y Norvig.

---

[6] Una instanciación autosuficiente de un conjunto de variables es aquella en la que los padres de cada variable en el conjunto también están en el conjunto.

[7] Al igual que con Prolog, puede haber infinitos conjuntos de sustituciones de tamaño ilimitado; diseñar interfaces exploratorias para tales respuestas es un desafío interesante de visualización.

[8] La respuesta es sí, son el mismo documento. La "Conferencia Nacional sobre Inteligencia Articial" (observe cómo falta la "fi", gracias a un error al extraer el carácter de ligadura) es otro nombre para la conferencia AAAI; la conferencia tuvo lugar en Seattle mientras que el editor de las actas está en Cambridge.

type Researcher, Paper, Citation
random String Name(Researcher)
random String Title(Paper)
random Paper PubCited(Citation)
random String Text(Citation)
random Boolean Professor(Researcher)
origin Researcher Author(Paper)

#Researcher ~ *OM*(3,1)
Name(*r*) ~ *NamePrior*()
Professor(*r*) ~ *Boolean*(0.2)
#Paper(*Author*= *r*) ~ **if** *Professor*(*r*) **then** *OM*(1.5,0.5) **else** *OM*(1,0.5)
Title(*p*) ~ *PaperTitlePrior*()
CitedPaper(*c*) ~ *UniformChoice*({*Paper p*})
Text(*c*) ~ *HMMGrammar*(*Name*(*Author*(*CitedPaper*(c))),*Title*(*CitedPaper*(c)))

Figura 15.5 Un OUPM para la extracción de información de citas. Para simplificar, el modelo asume un autor por documento y omite detalles de la gramática y los modelos de error.

Para resolver el problema utilizando un enfoque probabilístico, necesitamos un modelo generativo para el dominio. Es decir, nos preguntamos cómo llegan a existir estas cadenas de citas en el mundo. El proceso comienza con los investigadores, que tienen nombres. (No necesitamos preocuparnos por cómo llegaron a existir los investigadores; sólo necesitamos expresar nuestra incertidumbre sobre cuántos hay). Estos investigadores escriben algunos documentos, que tienen títulos; las personas citan los documentos, combinando los nombres de los autores y el título del documento (con errores) en el texto de la cita de acuerdo con alguna gramática. Los elementos básicos de este modelo se muestran en la Figura 15.5, cubriendo el caso en que los documentos tienen solo un autor.[9]

Dadas sólo las cadenas de citas como evidencia, la inferencia probabilística en este modelo para seleccionar la explicación más probable para los datos produce una tasa de error 2 a 3 veces menor que la de CiteSeer (Pasula *et al*., 2003). El proceso de inferencia también exhibe una forma de desambiguación colectiva basada en el conocimiento: entre más citas haya para un documento dado, más detallado será el análisis de cada una de ellas, porque los análisis tienen que estar de acuerdo con los hechos sobre el documento.

Monitoreo de tratados nucleares

Verificar el Tratado de Prohibición Completa de los Ensayos Nucleares requiere encontrar todos los eventos sísmicos en la Tierra por encima de una magnitud mínima. La CTBTO de la ONU mantiene una red de sensores, el Sistema de Monitoreo Internacional (IMS); su software de procesamiento automatizado, basado en 100 años de investigación en sismología, tiene una tasa de fallos de detección de alrededor de 30 %. El sistema NET-VISA (Arora *et al*., 2013), basado en un OUPM, reduce de manera significativa los fallos de detección.

[9] El caso de varios autores tiene la misma estructura general pero es un poco más complicado. Las partes del modelo no mostradas: el *NamePrior*, *rTitlePrior*, y *HMMGrammar*, son modelos de probabilidad tradicionales. Por ejemplo, el *NamePrior* es una mezcla de una distribución categórica sobre nombres reales y un modelo de trigramas de letras (ver Sección 23.1) para cubrir nombres no vistos previamente, ambos entrenados a partir de datos en la base de datos del censo de Estados Unidos.

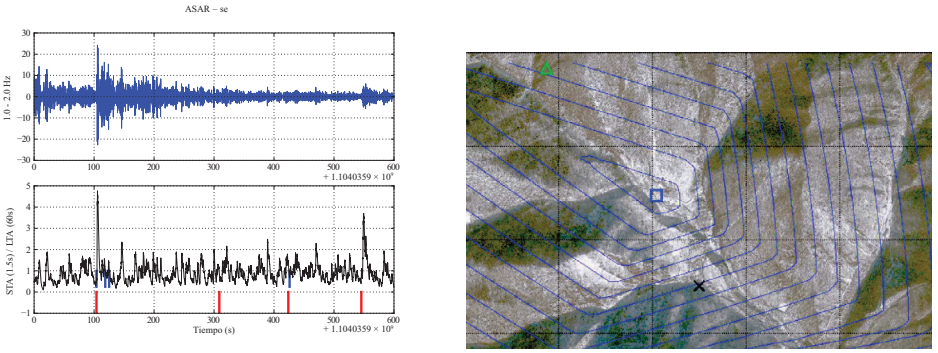
$\#SeismicEvents \sim Poisson(T * \lambda_e)$
$Time(e) \sim UniformReal(0,T)$
$EarthQuake(\text{e}) \sim Boolean(0.999)$
$Location(e) \sim$ **if** $Earthquake(e)$ **then** $SpatialPrior()$ **else** $UniformEarth()$
$Depth(e) \sim$ **if** $Earthquake(e)$ **then** $UniformReal(0{,}700)$ **else** $Exactly(0)$
$Magnitude(e) \sim Exponential(log(10))$
$Detected(e,p,s) \sim Logistic(weights(s,p), Magnitude(e), Depth(\text{e}), Dist(e,s))$
$\#Detections(site = s) \sim Poisson(\text{T} * \lambda_f(s))$
$\#Detections(event=e, phase=p, station=s) =$ **if** $Detected(e,p,s)$ **then** 1 **else** 0
$OnsetTime(a,s)$ **if** $(event(a) = null)$ **then** $\sim UniformReal(0,T)$
else $= Time(event(a)) + GeoTT(Dist(event(a),s), Depth(event(a)),phase(a))$
$+ Laplace(\mu_t(s),\sigma_t(s))$
$Amplitude(a,s)$ **if**$(event(a) = null)$ **then** $\sim NoiseAmpModel(s)$
else $= AmpModel(Magnitude(event(a)), Dist(event(a),s), Depth(event(a)), phase(a))$
$Azimuth(a,s)$ **if**$(event(a) = \text{null})$ **then** $\sim UniformReal(0, 360)$
else $= GeoAzimuth(Location(event(a)),Depth(event(a)),phase(a),Site(s))$
$+ Laplace(0,\sigma_s(s))$
$Slowness(a,s)$ **if** $(event(a) = null)$ **then** $\sim UniformReal(0{,}20)$
else $= GeoSlowness(Location(event(a)),Depth(event(a)),phase(a),Site(s))$
$+ Laplace(0,\sigma_s(s))$
$ObservedPhase(a,s) \sim CategoricalPhaseModel(phase(a))$

Figura 15.6 Una versión simplificada del modelo NET-VISA (ver texto).

El modelo NET-VISA (Figura 15.6) expresa de manera directa la geofísica relevante. Describe las distribuciones sobre el número de eventos en un intervalo de tiempo dado (la mayoría de los cuales ocurren de manera natural), así como sobre su tiempo, magnitud, profundidad y ubicación. Las ubicaciones de los eventos naturales se distribuyen según una prioridad espacial que se entrena (como otras partes del modelo) a partir de datos históricos; los eventos provocados por el hombre, según las reglas del tratado, se asumen que ocurren uniformemente sobre la superficie de la Tierra. En cada estación *s*, cada fase (tipo de onda sísmica) *p* de un evento *e* produce 0 o 1 detecciones (señales por encima del umbral); la probabilidad de detección depende de la magnitud y profundidad del evento y de su distancia a la estación. También ocurren detecciones de "falsa alarma" según un parámetro de tasa específico de la estación. El tiempo de llegada medido, la amplitud y otras propiedades de una detección *d* de un evento real dependen de las propiedades del evento de origen y de su distancia a la estación de medición.

Una vez entrenado, el modelo se ejecuta de manera continua. La evidencia consiste en detecciones (90 % de las cuales son falsas alarmas) extraídas de los datos brutos de la forma de onda del IMS, y la consulta normalmente pide el historial de eventos más probable, o *boletín*, dado los datos. Los resultados hasta ahora son alentadores; por ejemplo, en 2009 el boletín automatizado SEL3 de la ONU se perdió el 27.4 % de los 27294 eventos en el rango de magnitud 3-4 mientras que NET-VISA se perdió el 11.1 %. Además, las comparaciones con redes regionales densas muestran que NET-VISA encuentra hasta un 50 % más de eventos reales que los boletines finales producidos por los analistas sísmicos expertos de la ONU. NET-VISA también tiende a asociar más detecciones con un evento dado, lo que lleva a estimaciones de ubicación más precisas (ver Figura 15.7). A partir del 1 de enero de 2018, NET-VISA ha sido desplegado como parte de la tubería de monitoreo de la CTBTO.

A pesar de las diferencias superficiales, los dos ejemplos son similares: hay objetos desconocidos (papeles, terremotos) que generan percepciones según algún proceso físico (citación,



(a) (b)

Figura 15.7 **(a)** Arriba: Ejemplo de forma de onda sísmica registrada en Alice Springs, Australia. Abajo: la forma de onda después del procesamiento para detectar los tiempos de llegada de las ondas sísmicas. Las líneas azules son las llegadas detectadas de manera automática; las líneas rojas son las llegadas reales. (b) Estimaciones de ubicación para la prueba nuclear de DPRK del 12 de febrero de 2013: Boletín de Eventos Tardíos de la ONU CTBTO (triángulo verde en la parte superior izquierda); NET-VISA (cuadrado azul en el centro). La entrada a la instalación de prueba subterránea (pequeña "x") está a 0.75 km de la estimación de NET-VISA. Los contornos muestran la distribución de ubicación posterior de NET-VISA. Cortesía de la Comisión Preparatoria de la CTBTO.

propagación sísmica). Los perceptos son ambiguos en cuanto a su origen, pero cuando se hipotetiza que varios perceptos se originaron con el mismo objeto desconocido, se pueden inferir con mayor precisión las propiedades de ese objeto.

La misma estructura y patrones de razonamiento se mantienen para áreas como la deduplicación de bases de datos y la comprensión del lenguaje natural. En algunos casos, inferir la existencia de un objeto implica agrupar perceptos, un proceso semejante a la tarea de agrupamiento en el aprendizaje automático. En otros casos, un objeto puede no generar perceptos en absoluto y aun así tener su existencia inferida, como sucedió, por ejemplo, cuando las observaciones de Urano llevaron al descubrimiento de Neptuno. La existencia del objeto no observado se deduce de sus efectos sobre el comportamiento y las propiedades de los objetos observados.

15.3 Mantener seguimiento de un mundo complejo

El Capítulo 14 consideró el problema de mantener un seguimiento del estado del mundo, pero sólo cubrió el caso de las representaciones atómicas (HMMs) y las representaciones factorizadas (DBNs y filtros de Kalman). Esto tiene sentido para mundos con un solo objeto, quizás un solo paciente en la unidad de cuidados intensivos o un solo pájaro volando a través del bosque. En esta sección, vemos qué sucede cuando dos o más objetos generan las observaciones. Lo que hace que este caso sea diferente de la simple estimación del estado es que ahora existe la posibilidad de *incertidumbre* sobre qué objeto generó qué observación. Este es el problema de **incertidumbre de identidad** de la Sección 15.2 (página 507), ahora visto en un contexto temporal. En la literatura de teoría de control, este es el problema de **asociación de datos**; es decir, el problema de asociar datos de observación con los objetos que los generaron. Aunque podríamos ver esto como otro ejemplo de modelado probabilístico de universo abierto, es lo suficientemente importante en la práctica para merecer su propia sección.

Asociación de datos

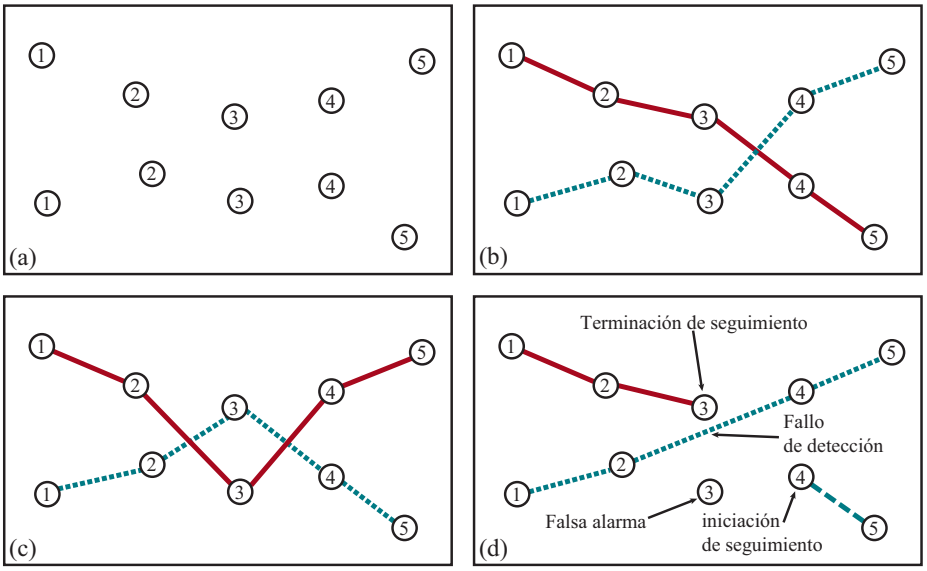


Figura 15.8 **(a)** Observaciones realizadas de las ubicaciones de objetos en el espacio 2D durante cinco pasos de tiempo. Cada blip de observación está etiquetado con el paso de tiempo pero no identifica el objeto que lo produjo. (b–c) Posibles hipótesis sobre las pistas de objetos subyacentes. (d) Una hipótesis para el caso en el que son posibles las falsas alarmas, los fallos de detección y la iniciación/terminación de la pista.

15.3.1 Ejemplo: Seguimiento multitarget

El problema de asociación de datos se estudió originalmente en el contexto del seguimiento de radar de múltiples objetivos, donde los pulsos reflejados son detectados a intervalos de tiempo fijos por una antena de radar rotativa. En cada paso de tiempo, pueden aparecer múltiples blips en la pantalla, pero no hay una observación directa de qué blips en el tiempo t corresponden a qué blips en el tiempo $t-1$. La Figura 15.8(a) muestra un ejemplo simple con dos blips por paso de tiempo durante cinco pasos. Cada blip está etiquetado con su paso de tiempo pero carece de cualquier información de identificación.

Suponga, por el momento, que sabemos que hay dos aviones, A_1 y A_2, generando los blips. En la terminología de OUPMs, A_1 y A_2 son **objetos garantizados**, lo que significa que se garantiza que existen y que son distintos; además, en este caso no hay otros objetos. (En otras palabras, en lo que respecta a los aviones, este escenario coincide con la semántica de la base de datos que se asume en las RPMs). Si se definen sus verdaderas posiciones como $X(A_1,t)$ y $X(A_2,t)$, donde t es un entero no negativo que indexa los tiempos de actualización del sensor. Asumimos que la primera observación llega en $t = 1$, y en el tiempo 0 la distribución *a priori* para la ubicación de cada avión es *InitX*(). Sólo para aclarar las cosas, también asumiremos que cada avión se mueve de forma independiente de acuerdo con un modelo de transición conocido; por ejemplo, un modelo lineal–Gaussiano, como se usa en el filtro de Kalman (Sección 14.4).

Objetos garantizados

La última pieza es el modelo del sensor: de nuevo, asumimos un modelo lineal–Gaussiano donde un avión en la posición x produce un blip b cuya posición de blip observada $Z(b)$ es una función lineal de x con ruido Gaussiano añadido. Cada avión genera un blip en cada paso de

$\#Aircraft(EntryTime = t) \sim Poisson(\lambda_a)$
$Exits(a,t) \sim \textbf{if } InFlight(a,t) \textbf{ then } Boolean(\alpha_e)$
$InFlight(a,t) = (t = EntryTime(a)) \vee (InFlight(a,t-1) \wedge \neg Exits(a,t-1))$
$X(a,t) \sim \textbf{if } t = EntryTime(a) \textbf{ then } InitX()$
$\quad \textbf{else if } InFlight(a,t) \textbf{ then } \mathcal{N}(\mathbf{F}X(a,t-1), \Sigma_x)$
$\#Blip(Source=a, Time=t) \sim \textbf{if } InFlight(a,t) \textbf{ then } Bernoulli(DetectionProb(X(a,t)))$
$\#Blip(Time=t) \sim Poisson(\lambda_f)$
$Z(b) \sim \textbf{if } Source(b)=null \textbf{ then } UniformZ(R) \textbf{ else } \mathcal{N}(\mathbf{H}X(Source(b),Time(b)),\Sigma_z)$

Figura 15.9 Un OUPM para el seguimiento de radar de múltiples objetivos con falsas alarmas, fallo de detección, y entrada y salida de aviones. La tasa a la que los nuevos aviones entran en escena es λ_a, mientras que la probabilidad por paso de tiempo de que un avión salga de la escena es α_e. Las falsas alarmas (es decir, las que no son producidas por un avión) aparecen de manera uniforme en el espacio a una tasa de λ_f por paso de tiempo. La probabilidad de que un avión sea detectado (es decir, produzca un blip) depende de su posición actual.

tiempo, así que el blip tiene como origen un avión y un paso de tiempo. Así, omitiendo el anterior por ahora, el modelo se ve así:

$\textbf{guaranteed } Aircraft\ A_1, A_2$
$X(a,t) \sim \textbf{if } t = 0 \textbf{ then } InitX() \textbf{ else } \mathcal{N}(\mathbf{F}X(a,t-1),\Sigma_x)$
$\#Blip(Source=a, Time=t) = 1$
$Z(b) \sim \mathcal{N}(\mathbf{H}X(Source(b),Time(b)),\Sigma_z)$

donde $\mathbf{F}$ y Σ_x son matrices que describen el modelo de transición lineal y la covarianza del ruido de transición, y $\mathbf{H}$ y Σ_z son las matrices correspondientes para el modelo del sensor. (Ver página 483).

La principal diferencia entre este modelo y un filtro de Kalman estándar es que hay *dos* objetos que producen lecturas de sensores (blips). Esto significa que hay *incertidumbre* en cualquier paso de tiempo sobre qué objeto produjo qué lectura del sensor. Cada mundo posible en este modelo incluye una asociación, definida por los valores de todas las variables $Source(b)$ para todos los pasos de tiempo, entre aviones y blips. Dos posibles hipótesis de asociación se muestran en la Figura 15.8(b–c). En general, para n objetos y T pasos de tiempo, hay $(n!)^T$ formas de asignar blips a aviones, un número terriblemente grande.

Falsa alarma
Clutter
Fallos de detección

El escenario descrito hasta ahora involucraba n objetos conocidos generando n observaciones en cada paso de tiempo. Las aplicaciones reales de la asociación de datos suelen ser mucho más complicadas. En la mayoría de los casos, las observaciones reportadas incluyen **falsas alarmas** (también conocidas como **clutter**), que no son causados por objetos reales. Pueden ocurrir **fallos de detección**, lo que significa que no se reporta ninguna observación para un objeto real. Finalmente, llegan nuevos objetos y desaparecen los antiguos. Estos fenómenos, que crean aún más mundos posibles de los que preocuparse, se ilustran en la Figura 15.8(d). El OUPM correspondiente se da en la Figura 15.9.

Debido a su importancia práctica para aplicaciones civiles y militares, se han escrito decenas de miles de artículos sobre el problema del seguimiento de múltiples objetivos y la asociación de datos. Muchos de ellos intentan resolver los complejos detalles matemáticos de los cálculos de probabilidad para el modelo en la Figura 15.9, o para versiones más simples de él. En cierto sentido, esto es innecesario una vez que el modelo se expresa en un lenguaje de programación probabilística, porque el motor de inferencia de propósito general implementa matemáticas correctas

para cualquier modelo, incluyendo este. Además, las elaboraciones del escenario (vuelo en formación, objetos que se dirigen a destinos desconocidos, objetos que despegan o aterrizan, etc.) Permiten ser manejados por pequeños cambios en el modelo sin recurrir a nuevas derivaciones matemáticas y programación compleja.

Desde un punto de vista práctico, el desafío con este tipo de modelo es la complejidad de la inferencia. Como para todos los modelos de probabilidad, la inferencia significa sumar las variables que no son la consulta y la evidencia. Para el filtrado en HMMs y DBNs, pudimos sumar las variables de estado de 1 a $t-1$ con un simple truco de programación dinámica; para los filtros de Kalman, también aprovechamos las propiedades especiales de los Gaussianos. Para la asociación de datos, somos menos afortunados. No hay algoritmo exacto eficiente (conocido), por la misma razón que no hay ninguno para el filtro de Kalman de conmutación (página 484): la distribución de filtrado, que describe la distribución conjunta sobre números y ubicaciones de aviones en cada paso de tiempo, termina como una mezcla de manera exponencial de muchas distribuciones, una para cada forma de elegir una secuencia de observaciones para asignar a cada avión.

Como respuesta a la complejidad de la inferencia exacta, se han utilizado varios métodos aproximados. El enfoque más simple es elegir una única "mejor" asignación en cada paso de tiempo, dadas las posiciones predichas de los objetos en el tiempo actual. Esta asignación asocia observaciones con objetos y permite actualizar la pista de cada objeto y hacer una predicción para el próximo paso de tiempo. Para elegir la "mejor" asignación, es común usar el llamado filtro de vecino más cercano, que elige de manera repetida el emparejamiento predicho de posición y observación más cercano de **Nearest-neighbor filter** (filtro del vecino más cercano), y añade ese emparejamiento a la asignación. El filtro de vecino más cercano funciona bien cuando los objetos están bien separados en el espacio de estado y la incertidumbre de la predicción y el error de observación son pequeños, en otras palabras, cuando no hay posibilidad de confusión.

Filtro del vecino más cercano

Cuando hay más incertidumbre en cuanto a la asignación correcta, un mejor enfoque es elegir la asignación que maximiza la probabilidad conjunta de las observaciones actuales dadas las posiciones predichas. Esto se hace de manera eficiente usando el **algoritmo húngaro** (Kuhn, 1955), a pesar de que hay $n!$ asignaciones a elegir a medida que llega cada nuevo paso de tiempo.

Algoritmo húngaro

Cualquier método que se comprometa con una única mejor asignación en cada paso de tiempo fracasa bajo condiciones más difíciles. En particular, si el algoritmo se compromete con una asignación incorrecta, la predicción en el próximo paso de tiempo puede ser errónea, lo que lleva a más asignaciones incorrectas, etc. Los enfoques de muestreo son mucho más efectivos. Un algoritmo de **filtrado de partículas** (ver página 492) para la asociación de datos funciona manteniendo una gran colección de posibles asignaciones actuales. Un algoritmo **MCMC** explora el espacio de historias de asignación; por ejemplo, la Figura 15.8(b-c) podrían ser estados en el espacio de estado MCMC, y puede cambiar de opinión sobre decisiones de asignación anteriores.

Una forma obvia de acelerar la inferencia basada en muestreo para el seguimiento de múltiples objetivos es usar el truco de **Rao-Blackwellización** del Capítulo 14 (página 496): dada una hipótesis de asociación específica para todos los objetos, el cálculo de filtrado para cada objeto permite ser hecho de manera exacta y eficiente, en lugar de muestrear muchas posibles secuencias de estado para los objetos. Por ejemplo, con el modelo en la Figura 15.9, el cálculo de filtrado significa ejecutar un filtro de Kalman para la secuencia de observaciones asignadas a un objeto hipotetizado dado. Además, al cambiar de una hipótesis de asociación a otra, los cálculos sólo tienen que ser re-hechos para los objetos cuyas observaciones asociadas han cambiado. Los métodos actuales de asociación de datos MCMC permiten manejar muchos cientos de objetos en tiempo real mientras dan una buena aproximación a las distribuciones posteriores verdaderas.

(a) (b)

Figura 15.10 Imágenes de (a) cámaras de vigilancia aguas arriba y (b) aguas abajo a dos millas de distancia en la Autopista 99 en Sacramento, California. El vehículo enmarcado ha sido identificado en ambas cámaras.

15.3.2 Ejemplo: Monitoreo de tráfico

La Figura 15.10 muestra dos imágenes de cámaras separadas en una autopista de California. En esta aplicación, estamos interesados en dos objetivos: estimar el tiempo que se tarda, bajo las condiciones actuales de tráfico, para ir de un lugar a otro en el sistema de autopistas; y medir la *demanda*; es decir, cuántos vehículos viajan entre cualquier dos puntos en el sistema en momentos particulares del día y en días particulares de la semana. Ambos objetivos requieren resolver el problema de asociación de datos en un área amplia con muchas cámaras y decenas de miles de vehículos por hora.

Con la vigilancia visual, las falsas alarmas son causadas por sombras en movimiento, vehículos articulados, reflejos en charcos, etc.; los fallos de detección son causados por la ocultación, la niebla, la oscuridad y la falta de contraste visual; y los vehículos están constantemente entrando y saliendo del sistema de autopistas en puntos que pueden no estar monitoreados. Además, la apariencia de cualquier vehículo dado permite cambiar de manera dramática entre cámaras dependiendo de las condiciones de iluminación y la pose del vehículo en la imagen, y el modelo de transición cambia a medida que van y vienen los atascos de tráfico. Finalmente, en el tráfico denso con cámaras ampliamente separadas, el error de predicción en el modelo de transición para un coche que va de una ubicación de cámara a la siguiente es mucho mayor que la separación típica entre vehículos. A pesar de estos problemas, los modernos algoritmos de asociación de datos han tenido éxito en la estimación de parámetros de tráfico en entornos reales.

La asociación de datos es una base esencial para mantener un seguimiento de un mundo complejo, porque sin ella no hay forma de combinar múltiples observaciones de cualquier objeto dado. Cuando los objetos en el mundo interactúan entre sí en actividades complejas, entender el mundo requiere combinar la asociación de datos con los modelos de probabilidad relacional y de universo abierto de la Sección 15.2. Esta es, en la actualidad, un área activa de investigación.

```
function GENERATE-IMAGE() returns una imagen con algunas letras
    letters←GENERATE-LETTERS(10)
    return RENDER-NOISY-IMAGE(letters, 32, 128)

function GENERATE-LETTERS(λ) returns un vector de letras
    n ~ Poisson(λ)
    letters←[ ]
    for i = 1 to n do
        letters[i] ~ UniformChoice({a,b,c,···})
    return letters

function RENDER-NOISY-IMAGE(letters,width,height) returns una imagen ruidosa de las letras
    clean image←RENDER(letters,width,height,text_top = 10, text_left = 10)
    noisy image←[ ]
    noise variance ~ UniformReal(0.1, 1)
    for row = 1 to width do
        for col = 1 to height do
            noisy_image[row,col] ~ N(clean_image[row,col],noise variance)
    return noisy_image
```

Figura 15.11 Programa generativo para un modelo de probabilidad de universo abierto para el reconocimiento óptico de caracteres. El programa generativo produce imágenes degradadas que contienen secuencias de letras al generar cada secuencia, renderizándola en una imagen 2D e incorporando ruido aditivo en cada píxel.

15.4 Programas como modelos de probabilidad

Múltiples lenguajes de programación probabilística se han construido sobre la idea de que los modelos de probabilidad permiten definirse utilizando código ejecutable en cualquier lenguaje de programación que incorpore una fuente de aleatoriedad. Para tales modelos, los mundos posibles son trazas de ejecución, donde la probabilidad de cualquier traza es la probabilidad de las elecciones aleatorias requeridas para que esa traza ocurra. Los PPL creados de esta manera heredan todo el poder expresivo del lenguaje subyacente, incluidas estructuras de datos complejas, recursión y, en algunos casos, funciones de orden superior. Muchos PPL son de hecho computacionalmente universales: es posible representar cualquier distribución de probabilidad que pueda ser muestreada por una máquina de Turing probabilística que se detiene.

15.4.1 Ejemplo: Lectura de texto

Ilustramos este enfoque para el modelado probabilístico y la inferencia a través del problema de escribir un programa que lee texto degradado. Este tipo de modelos se pueden construir para leer texto que ha sido manchado o está borroso debido al daño del agua, o debido al envejecimiento del papel en el que está impreso. También es posible construir para romper algunos tipos de CAPTCHAs.

La Figura 15.11 muestra un programa generativo que contiene dos componentes: (i) una forma de generar una secuencia de letras; y (ii) una forma de generar una representación ruidosa y borrosa de estas letras utilizando una biblioteca gráfica estándar. La Figura 15.12 (superior) muestra imágenes de ejemplo generadas al invocar GENERATE-IMAGE nueve veces.

Figura 15.12 El panel superior muestra doce imágenes degradadas producidas al ejecutar el programa generativo de la Figura 15.11. El número de letras, sus identidades, la cantidad de ruido aditivo y el ruido específico de cada pixel son parte del dominio del modelo de probabilidad. El panel inferior muestra doce imágenes degradadas producidas al ejecutar el programa generativo de la Figura 15.15. El modelo de Markov para las letras en general produce secuencias de letras que son más fáciles de pronunciar.

15.4.2 Sintaxis y semántica

Programa generativo

Un **programa generativo** es un programa ejecutable en el que cada elección aleatoria define una variable aleatoria en un modelo de probabilidad asociado. Imaginemos desenrollar la ejecución de un programa que hace elecciones aleatorias, paso a paso. Definamos X_i como la variable aleatoria correspondiente a la i-ésima elección aleatoria hecha por el programa; como de costumbre, x_i denota un valor posible de X_i. Llamemos $\omega = \{x_i\}$ un **trazado de ejecución** del programa generativo, es decir, una secuencia de posibles valores para las elecciones aleatorias. Al ejecutar el programa una vez, se genera una traza de este tipo, de ahí el término "programa generativo".

Trazado de ejecución

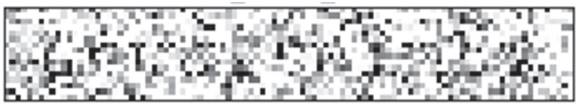
El espacio de todas las posibles trazas de ejecución Ω puede ser visto como el espacio muestral de un modelo de probabilidad definido por el programa generativo. La distribución de probabilidad sobre las trazas permite ser definida como el producto de las probabilidades de cada elección aleatoria individual: $P(\omega) = \prod_i P(x_i|x_1,...x_{i-1})$. Esto es análogo a la distribución sobre mundos en un OUPM.

Es conceptualmente sencillo convertir cualquier OUPM en un programa generativo correspondiente. Este programa generativo hace elecciones aleatorias para cada declaración numérica y para el valor de cada variable aleatoria básica, cuya existencia se implica por las declaraciones numéricas. El principal trabajo extra que el programa generativo necesita hacer es crear estructuras de datos que representen los objetos, funciones y relaciones de los posibles mundos en el OUPM. Estas estructuras son creadas de manera automática por el motor de inferencia OUPM porque el OUPM asume que cada mundo posible es una estructura de modelo de primer orden, mientras que un PPL típico no hace tal suposición.

Las imágenes en la Figura 15.12 se pueden usar para obtener una comprensión intuitiva de la distribución de probabilidad $P(\Omega)$: vemos niveles variables de ruido, y en las imágenes menos ruidosas también vemos secuencias de letras de diferentes longitudes. Se define ω_1 como el rastro correspondiente a la imagen en la esquina superior derecha de esta figura, que contiene las letras `ocflwe`. Si desenrolláramos este rastro ω_1 en una red bayesiana, tendría 4 104 nodos: 1 nodo para la variable n; 6 nodos para las variables $letters[i]$; 1 nodo para la $noise_variance$; y 4 096 nodos para los píxeles en $noisy_image$. Así vemos que este programa generativo define una probabilidad de universo abierto modelo: el número de elecciones aleatorias que hace no está limitado a priori, sino que depende del valor de la variable aleatoria n.

uncertainty
uncertainty
uncertainty

Figura 15.13 Imagen de entrada ruidosa (arriba) y resultados de inferencia (abajo) producidos por tres ejecuciones, cada una de 25 iteraciones MCMC, con el modelo de la Figura 15.11. Observe que el proceso de inferencia identifica de manera correcta la secuencia de letras.



qnucitainty
qncurtbinty
qncuftaintv

uncertainty
uncsitainty
unchftsinty

Figura 15.14 Arriba: imagen de entrada ruidosa. Abajo a la izquierda: con tres resultados de inferencia de 25 iteraciones MCMC con el modelo de letra independiente de la Figura 15.11. Abajo a la derecha: tres resultados de inferencia con el modelo de bigrama de letras de la Figura 15.15. Ambos modelos muestran ambigüedad en los resultados, pero los resultados del último modelo reflejan el conocimiento previo de secuencias de letras plausibles.

function GENERATE-MARKOV-LETTERS(λ) **returns** un vector de letras
  $n \sim Poisson(\lambda)$
  letters ←[ ]
  letter_probs←MARKOV-INITIAL()
  for $i = 1$ **to** n **do**
    letters[i] ~ *Categorical*(*letter_probs*)
    letter_probs←MARKOV-TRANSITION(*letters*[i])
  return *letters*

Figura 15.15 Programa generativo para un modelo mejorado de reconocimiento óptico de caracteres, que genera letras de acuerdo con un modelo de bigrama de letras cuyas frecuencias de pares de letras se estiman a partir de una lista de palabras en inglés.

15.4.3 Resultados de inferencia

Apliquemos este modelo para interpretar imágenes de letras que se han degradado con ruido aditivo. La Figura 15.13 muestra una imagen degradada, junto con los resultados de tres ejecuciones independientes de MCMC. Para cada ejecución, mostramos una representación de las letras contenidas en el rastro después de detener la cadena de Markov. En los tres casos, el resultado es la secuencia de letras `uncertainty` (incertidumbre), lo que sugiere que la distribución posterior está concentrada en la interpretación correcta.

Ahora degradamos el texto aún más, desenfocándolo al punto que sea difícil de leer para las personas. La Figura 15.14 muestra los resultados de inferencia en esta entrada más desafiante. Esta vez, aunque la inferencia MCMC parece haber convergido en (lo que sabemos que es) el número correcto de letras, la primera letra se identifica de manera errónea como una `q` y hay incertidumbre sobre cinco de las diez letras siguientes.

En este punto, hay múltiples formas posibles de interpretar los resultados. Podría ser que la inferencia MCMC se haya mezclado bien y los resultados sean un buen reflejo del verdadero posterior dado el modelo y la imagen; en ese caso, la incertidumbre en algunas de las letras y el error en la primera letra son inevitables. Para obtener mejores resultados, podríamos necesitar mejorar el modelo de texto o reducir el nivel de ruido. También podría ser que la inferencia MCMC no se haya mezclado de manera correcta: si ejecutamos 300 cadenas durante 25 mil o 25 millones de iteraciones, podríamos encontrar una distribución de resultados bastante diferente, tal vez indicando que la primera letra es probablemente `u` en lugar de `q`.

Ejecutar inferencias adicionales podría ser costoso en términos de dólares y tiempo de espera. Además, no hay una prueba infalible para la convergencia de los métodos de inferencia de Monte Carlo. Podríamos intentar mejorar el algoritmo de inferencia, quizá diseñando una mejor distribución de propuesta para MCMC o usando pistas de abajo hacia arriba de la imagen para sugerir mejores hipótesis iniciales. Estas mejoras requieren un pensamiento adicional, implementación y depuración. La tercera alternativa es mejorar el modelo; por ejemplo, podríamos incorporar conocimientos sobre palabras en inglés, como las probabilidades de pares de letras. Ahora consideramos esta opción.

15.4.4 Mejorando el programa generativo para incorporar un modelo de Markov

Los lenguajes de programación probabilística son modulares, de modo que facilitan la exploración de mejoras en el modelo subyacente. La Figura 15.15 muestra el programa generativo para un modelo mejorado que genera letras de manera secuencial, en lugar de independiente. Este programa generativo usa un modelo de Markov que dibuja cada letra dada la letra anterior, con probabilidades de transición estimadas a partir de una lista de referencia de palabras en inglés.

La Figura 15.12 muestra doce imágenes muestreadas producidas por este programa generativo. Observe que las secuencias de letras son significativamente más parecidas al inglés que las generadas a partir del programa en la Figura 15.11. El panel derecho en la Figura 15.14 muestra los resultados de la inferencia de este modelo de Markov aplicado a la imagen de alto ruido. Las interpretaciones se asemejan más a la traza generadora, aunque todavía hay cierta incertidumbre.

15.4.5 Inferencia en programas generativos

Al igual que con los OUPM, la inferencia exacta en programas generativos suele ser cara o imposible. Por otro lado, es fácil ver cómo realizar un muestreo de rechazo: ejecutar el programa, mantener sólo las trazas que coinciden con la evidencia y contar las diferentes respuestas de consulta encontradas en esas trazas. La ponderación de probabilidades también es sencilla: para cada

traza generada, mantener un registro del peso de la traza multiplicando todas las probabilidades de los valores observados en el camino.

La ponderación de probabilidades funciona bien sólo cuando los datos son razonablemente probables según el modelo. En casos más difíciles, MCMC suele ser el método de elección. MCMC aplicado a programas probabilísticos implica muestrear y modificar las trazas de ejecución. Muchas de las consideraciones que surgen con los OUPM también se aplican aquí; además, el algoritmo debe tener cuidado con las modificaciones a una traza de ejecución, como cambiar el resultado de una declaración if, que permite invalidar el resto de la traza.

Las mejoras adicionales en la inferencia provienen de múltiples líneas de trabajo. Algunas mejoras pueden producir cambios fundamentales en la clase de problemas que son tratables con un PPL dado, incluso en principio; la inferencia elevada, descrita con anterioridad para los RPM, permite tener este efecto. En muchos casos, el MCMC genérico es demasiado lento, y se necesitan propuestas de propósito especial para permitir que el proceso de inferencia se mezcle de manera rápida.

Un enfoque importante del trabajo reciente en PPL ha sido facilitar a los usuarios la definición y uso de tales propuestas para que la eficiencia de la inferencia de PPL coincida con la de los algoritmos de inferencia personalizados diseñados para modelos específicos.

Múltiples enfoques prometedores están destinados a reducir la sobrecarga de la inferencia probabilística. La idea de compilación descrita para las redes de Bayes en la Sección 13.4.3 se aplica a la inferencia en OUPM y PPL, y, en general, produce aceleraciones de dos a tres órdenes de magnitud. También ha habido propuestas para *hardware de propósito especial* para algoritmos como el paso de mensajes y MCMC. Por ejemplo, el hardware de Monte Carlo explota las representaciones de probabilidad de baja precisión y el paralelismo masivo de grano fino para ofrecer mejoras de 100–10 000x en velocidad y eficiencia energética.

Los métodos basados en el aprendizaje también pueden dar mejoras sustanciales en la velocidad. Por ejemplo, las **distribuciones de propuestas adaptativas** permiten aprender de manera gradual cómo generar propuestas MCMC que tienen una probabilidad razonable de ser aceptadas y son razonablemente efectivas para explorar el paisaje de probabilidad del modelo para asegurar una mezcla rápida. También es posible entrenar modelos de aprendizaje profundo (ver Capítulo 21) para representar distribuciones de propuestas para el muestreo de importancia, utilizando datos sintéticos que se generaron a partir del modelo subyacente.

Distribuciones de propuestas adaptativas

En general, se espera que cualquier formalismo construido sobre lenguajes de programación generales se tope con la barrera de la computabilidad, y este es el caso de los PPL. Sin embargo, si asumimos que el programa subyacente se detiene para todas las entradas y todas las elecciones aleatorias, ¿la exigencia adicional de hacer inferencia probabilística aún hace que el problema sea indecidible? Resulta que la respuesta es sí, pero sólo para un modelo computacional con variables aleatorias continuas de precisión infinita. En ese caso, se vuelve posible escribir un modelo de probabilidad computable en el que la inferencia codifica el problema de detención. Por otro lado, con números de precisión finita y con las distribuciones de probabilidad suaves que se utilizan en aplicaciones reales, la inferencia sigue siendo decidible.

Resumen

Este capítulo ha explorado representaciones expresivas para modelos de probabilidad basados tanto en la lógica como en los programas.

- Los **modelos de probabilidad relacionales** (RPM) definen modelos de probabilidad en mundos derivados de la **semántica de la base de datos** para lenguajes de primer orden; son apropiados cuando todos los objetos y sus identidades se conocen con certeza.

- Dado un RPM, los objetos en cada mundo posible corresponden a los símbolos constantes en el RPM, y las variables aleatorias básicas son todas las posibles instancias de los símbolos de predicado con objetos que reemplazan cada argumento. Por lo tanto, el conjunto de mundos posibles es finito.
- Los RPM proporcionan modelos concisos para mundos con un gran número de objetos y permiten manejar la incertidumbre relacional.
- Los **modelos de probabilidad de universo abierto** (OUPM) se basan en la semántica completa de la lógica de primer orden, permitiendo nuevos tipos de incertidumbre como la incertidumbre de identidad y existencia.
- Los **programas generativos** son representaciones de modelos de probabilidad, incluyendo OUPMs, como programas ejecutables en un **lenguaje de programación probabilística** o **PPL**. Un programa generativo representa una distribución sobre los **trazos de ejecución** del programa. Los PPLs suelen proporcionar un poder expresivo *universal* para los modelos de probabilidad.

Notas bibliográficas e históricas

Hailperin (1984) y Howson (2003) relatan la larga historia de intentos de conectar la probabilidad y la lógica, remontándose a los *Nouveaux Essais* de Leibniz en 1704. Estos intentos usualmente involucraban probabilidades adjuntas de manera directa a las oraciones lógicas. El primer tratamiento riguroso fue la **lógica de probabilidad** proposicional de Gaifman (Gaifman, 1964b). La idea es que una probabilidad, la afirmación $P(\varphi) \geq p$ es una restricción en la distribución sobre los mundos posibles, al igual que una oración lógica ordinaria es una restricción sobre los propios mundos posibles. Cualquier distribución P que satisface la restricción es un modelo, en el sentido lógico estándar, de la afirmación de probabilidad, y una afirmación de probabilidad implica otra cuando los modelos de la primera son un subconjunto de los modelos de la segunda.

Lógica de probabilidad

Dentro de tal lógica, uno puede probar, por ejemplo, que $P(\alpha \wedge \beta) \leq P(\alpha \Rightarrow \beta)$. La satisfacibilidad de conjuntos de afirmaciones de probabilidad permite ser determinada en el caso proposicional por programación lineal (Hailperin, 1984; Nilsson, 1986). Así, tenemos una "lógica de probabilidad" en el mismo sentido que "lógica temporal"; un sistema lógico especializado para el razonamiento probabilístico.

Para aplicar la lógica de probabilidad a tareas como demostrar teoremas interesantes en la teoría de la probabilidad, se necesitaba un lenguaje más expresivo. Gaifman (1964a) propuso una lógica de probabilidad de *primer orden*, con mundos posibles siendo estructuras de modelo de primer orden y con probabilidades adjuntas a las sentencias de lógica de primer orden (sin función). Scott y Krauss (1966) extendieron los resultados de Gaifman para permitir el anidamiento infinito de cuantificadores y conjuntos infinitos de oraciones.

Dentro de la IA, el descendiente más directo de estas ideas aparece en los **programas de lógica probabilística** (Lukasiewicz, 1998), en los cuales un rango de probabilidad está adjunto a cada cláusula de Horn de primer orden y la inferencia se realiza resolviendo programas lineales, como sugirió Hailperin. Halpern (1990) y Bacchus (1990) también se basaron en el enfoque de Gaifman, explorando algunos de los problemas básicos de representación del conocimiento desde la perspectiva de la IA en lugar de la teoría de la probabilidad y la lógica matemática.

Bases de datos probabilísticas

El subcampo de las **bases de datos probabilísticas** también tiene oraciones lógicas etiquetadas con probabilidades (Dalvi *et al*., 2009); pero en este caso las probabilidades están adjuntas de manera directa a las tuplas de la base de datos. (En IA y estadística, la probabilidad se adjunta a las relaciones generales, mientras que las observaciones se consideran evidencia incontrovertible). Aunque las bases de datos probabilísticas permiten modelar dependencias complejas, en la

práctica se encuentra que estos sistemas usan suposiciones de independencia global a través de las tuplas.

Adjuntar probabilidades a las sentencias hace que sea muy difícil definir modelos de probabilidad completos y consistentes. Cada desigualdad *restringe* el modelo de probabilidad subyacente a yacer en un semiespacio en el espacio de alta dimensión de los modelos de probabilidad. La conjunción de afirmaciones corresponde a la intersección de las restricciones. Asegurar que la intersección produce un solo punto no es fácil. De hecho, el principal resultado en Gaifman (1964a) es la construcción de un solo modelo de probabilidad que requiere 1) una probabilidad para cada sentencia de tierra posible y 2) restricciones de probabilidad para infinitamente muchas oraciones cuantificadas de manera existencial.

Una solución a este problema es escribir una teoría parcial y luego "completarla" eligiendo un modelo canónico en el conjunto permitido. Nilsson (1986) propuso elegir el modelo de *máxima entropía* consistente con las restricciones especificadas. Paskin (2002) desarrolló una "lógica probabilística de máxima entropía" con restricciones expresadas como pesos (probabilidades relativas) adjuntas a las cláusulas de primer orden. Tales modelos se llaman **redes de lógica de Markov** o MLNs (Richardson y Domingos, 2006) y se han convertido en una técnica popular para aplicaciones que involucran datos relacionales. Los enfoques de máxima entropía, incluyendo MLNs, permiten producir resultados contra intuitivos en algunos casos (Milch, 2006; Jain *et al*., 2007, 2010).

A principios de la década de 1990, los investigadores que trabajaban en aplicaciones complejas notaron las limitaciones expresivas de las redes bayesianas y desarrollaron varios lenguajes para escribir "plantillas" con variables lógicas, a partir de las cuales se podían construir de manera automática grandes redes para cada instancia del problema (Breese, 1992; Wellman *et al*., 1992). El lenguaje más importante fue BUGS (Bayesian inference Using Gibbs Sampling) (Gilks *et al*., 1994; Lunn *et al*., 2013), que combinó redes bayesianas con la **variable aleatoria indexada,** notación común en estadísticas. (En BUGS, una variable aleatoria indexada se ve como $X[i]$, donde i tiene un rango entero definido).

Variable aleatoria indexada

Estos lenguajes de universo cerrado heredaron la propiedad clave de las redes bayesianas: cada BC bien formada define un modelo de probabilidad único y consistente. Otros lenguajes de universo cerrado se basaron en las capacidades representacionales e inferenciales de la programación lógica (Poole, 1993; Sato y Kameya, 1997; Kersting *et al.,* 2000) y las redes semánticas (Koller y Pfeffer, 1998; Pfeffer, 2000).

La investigación sobre modelos de probabilidad de universo abierto tiene varios orígenes. En estadísticas, el problema de **enlace de registros** surge cuando los registros de datos no contienen identificadores únicos estándar: enlace de registros por ejemplo, varias citas de este libro podrían nombrar a su primer autor como "Stuart J. Russell" o "S. Russell" o incluso "Stewart Russel." Otros autores comparten el nombre "S. Russell".

Enlace de registros

Existen cientos de empresas para resolver problemas de enlace de registros en datos financieros, médicos, de censo y otros. El análisis probabilístico se remonta al trabajo de Dunn (1946); el modelo Fellegi–Sunter (1969), que es en esencia naive Bayes aplicado a la coincidencia, todavía domina la práctica actual. La incertidumbre de identidad también se considera en el seguimiento de múltiples objetivos (Sittler, 1964), cuya historia se esboza en el Capítulo 14.

En IA, la suposición de trabajo hasta la década de 1990 era que los sensores podían suministrar oraciones lógicas con identificadores únicos para objetos, como era el caso con Shakey. En el área de comprensión del lenguaje natural, Charniak y Goldman (1992) propusieron un análisis probabilístico de la correferencia, donde dos expresiones lingüísticas (digamos, "Obama" y "el presidente") pueden referirse a la misma entidad. Huang y Russell (1998) y Pasula *et al*. (1999) desarrollaron un análisis bayesiano de la incertidumbre de identidad para la vigilancia del tráfico. Pasula *et al.* (2003) desarrollaron un modelo generativo complejo para autores, trabajos y cadenas de citas, que involucra tanto la incertidumbre relacional como de identidad, y demostraron una alta precisión para la extracción de información de citas.

El primer lenguaje formal para modelos de probabilidad de universo abierto fue Blog (Milch *et al*., 2005; Milch, 2006), que venía con un motor de inferencia MCMC de propósito general (muy lento). Laskey (2008) describe otro lenguaje de modelado de universo abierto llamado **redes bayesianas de múltiples entidades**. El sistema de monitoreo sísmico global NET-VISA descrito en el texto se debe a Arora *et al.* (2013). El sistema de clasificación Elo fue desarrollado en 1959 por Arpad Elo (1978) pero es en esencia el mismo en el modelo Case V de Thurstone (Thurstone, 1927). El modelo TrueSkill de Microsoft (Herbrich *et al.,* 2007; Minka *et al*., 2018) se basa en la versión bayesiana de Elo de Mark Glickman (1999) y ahora se ejecuta en el PPL infer.NET.

La asociación de datos para el seguimiento de múltiples objetivos se describió por primera vez en un entorno probabilístico por Sittler (1964). El primer algoritmo práctico para problemas a gran escala fue el "rastreador de múltiples hipótesis" o algoritmo MHT (Reid, 1979). Los documentos importantes son recopilados por Bar-Shalom y Fortmann (1988) y Bar-Shalom (1992). El desarrollo de un algoritmo MCMC para la asociación de datos se debe a Pasula *et al*. (1999), quienes lo aplicaron a problemas de vigilancia del tráfico. Oh *et al*. (2009) proporcionan un análisis formal y comparaciones experimentales con otros métodos. Schulz *et al*. (2003) describen un método de asociación de datos basado en el filtrado de partículas.

Ingemar Cox analizó la complejidad de la asociación de datos (Cox, 1993; Cox y Hingorani, 1994) y llevó el tema a la atención de la comunidad de visión. También notó la aplicabilidad del algoritmo húngaro de tiempo polinomial al problema de encontrar las asignaciones más probables, que durante mucho tiempo se consideró un problema intratable en la comunidad de seguimiento. El algoritmo en sí fue publicado por Kuhn (1955), basado en traducciones de documentos publicados en 1931 por dos matemáticos húngaros, Dénes König y Jenö Egerváry. Sin embargo, el teorema básico se había derivado previamente en un manuscrito latino inédito del famoso matemático Carl Gustav Jacobi (1804–1851).

La idea de que los programas probabilísticos también podrían representar modelos de probabilidad complejos se debe a Koller *et al.* (1997). El primer PPL de trabajo fue IBAL de Avi Pfeffer (2001, 2007), basado en un lenguaje funcional simple. BLOG puede considerarse un PPL declarativo. La conexión entre PPLs declarativos y funcionales fue explorada por McAllester *et al.* (2008). CHURCH (Goodman *et al*., 2008), un PPL construido sobre el lenguaje Scheme, fue pionero en la idea de aprovechar un lenguaje de programación existente. CHURCH también introdujo el primer algoritmo de inferencia MCMC para modelos con funciones de orden superior aleatorias y generó interés en la comunidad de ciencias cognitivas como una forma de modelar formas complejas de aprendizaje humano (Lake *et al.,* 2015). Los PPL también se conectan de formas interesantes con la teoría de la computabilidad (Ackerman *et al.,* 2013) y la investigación del lenguaje de programación.

En la década de 2010, surgieron docenas de PPL basados en una amplia gama de lenguajes de programación subyacentes. Figaro, basado en el lenguaje Scala, se ha utilizado para una amplia variedad de aplicaciones (Pfeffer, 2016). Gen (Cusumano-Towner *et al*., 2019), basado en Julia y TensorFlow, se ha utilizado para la percepción de máquinas en tiempo real, así como para el aprendizaje de estructuras bayesianas para el análisis de datos de series temporales. Los PPL construidos sobre frameworks de deep learning incluyen Pyro (Bingham *et al*., 2019) (construido sobre PyTorch) y Edward (Tran *et al*., 2017) (construido sobre TensorFlow).

Ha habido esfuerzos para hacer que la programación probabilística sea accesible para más personas, como usuarios de bases de datos y hojas de cálculo. Tabular (Gordon *et al*., 2014) proporciona un lenguaje de esquema relacional similar a una hoja de cálculo sobre infer.NET. BayesDB (Saad y Mansinghka, 2017) permite a los usuarios combinar y consultar programas probabilísticos utilizando un lenguaje similar a SQL.

La inferencia en programas probabilísticos en la mayoría de los casos ha dependido de métodos aproximados porque los algoritmos exactos no se escalan a los tipos de modelos que los PPL permiten representar. Los lenguajes como BUGS, LIBBI (Murray, 2013) y STAN (Carpenter *et al*.,

2017) en general operan construyendo la red bayesiana equivalente completa y luego ejecutando inferencia en ella: muestreo de Gibbs en el caso de BUGS, Monte Carlo secuencial en el caso de LIBBI, y Monte Carlo Hamiltoniano en el caso de STAN. Los programas en estos lenguajes permiten leerse como instrucciones para construir la red bayesiana básica. Breese (1992) mostró cómo generar sólo el fragmento relevante de la red completa, dado la consulta y la evidencia.

Trabajar con una red Bayesiana básica significa que los mundos posibles visitados por MCMC están representados por un vector de valores para las variables en la red bayesiana. La idea de muestrear de manera directa los mundos posibles de primer orden se debe a Russell (1999). En el lenguaje FACTORIE (McCallum *et al*., 2009), los mundos posibles en el proceso MCMC se representan dentro de un sistema de base de datos relacional estándar. Los mismos dos documentos proponen la reevaluación incremental de consultas como una forma de evitar la evaluación completa de consultas en cada mundo posible.

Los métodos de inferencia basados en fundamentos son análogos a los primeros métodos de proposicionalización para la inferencia lógica de primer orden (Davis y Putnam, 1960). Para la inferencia lógica, tanto los demostradores de teoremas de resolución como los sistemas de programación lógica dependen de la **elevación** (Sección 9.2) para evitar la instanciación innecesaria de variables lógicas.

Pfeffer *et al.* (1999) introdujeron un algoritmo de eliminación de variables que almacenaba en caché cada factor calculado para su reutilización por cálculos posteriores que involucraban las mismas relaciones pero diferentes objetos, logrando así algunas de las ganancias computacionales de la elevación. El primer algoritmo de inferencia probabilística verdaderamente elevado fue una forma de eliminación de variables descrita por Poole (2003) y, posteriormente, mejorada por de Salvo Braz *et al.* (2007). Avances posteriores, incluyendo casos donde ciertas probabilidades agregadas permiten calcularse en forma cerrada, son descritos por Milch *et al.* (2008) y Kisynski y Poole (2009). Ahora hay un entendimiento bastante bueno de cuándo es posible la elevación y de su complejidad (Gribkoff *et al*., 2014; Kazemi *et al*., 2017).

Los métodos para acelerar la inferencia vienen en varios sabores, como se señala en el capítulo. Varios proyectos han explorado algoritmos más sofisticados, combinados con técnicas de compilador y/o propuestas aprendidas. LIBBI (Murray, 2013) introdujo la primera inferencia de Gibbs de partículas para programas probabilísticos; uno de los primeros compiladores de inferencia, con soporte GPU para SMC masivamente paralelo; y el uso del lenguaje de modelado para definir propuestas personalizadas de MCMC. La compilación de la inferencia probabilística también es estudiada por Wingate *et al.* (2011), Paige y Wood (2014), Wu *et al.* (2016a). Claret *et al.* (2013), Hur *et al.* (2014), y Cusumano-Towner *et al.* (2019) demuestran métodos de análisis estático para transformar programas probabilísticos en formas más eficientes. PICTURE (Kulkarni *et al*., 2015) es el primer PPL que permite a los usuarios aplicar el aprendizaje de las ejecuciones hacia adelante del programa generativo para entrenar propuestas rápidas de abajo hacia arriba. Le *et al*. (2017) describen el uso de técnicas de aprendizaje profundo para el muestreo de importancia eficiente en un PPL. En la práctica, los algoritmos de inferencia para modelos de probabilidad complejos con frecuencia utilizan una mezcla de técnicas para diferentes subconjuntos de variables en el modelo. Mansinghka *et al.* (2013) enfatizó la idea de programas de inferencia que aplican tácticas de inferencia diversas a subconjuntos de variables elegidas durante el tiempo de ejecución de la inferencia.

La colección editada por Getoor y Taskar (2007) incluye muchos trabajos importantes sobre modelos de probabilidad de primer orden y su uso en aprendizaje automático. Los trabajos de programación probabilística aparecen en todas las principales conferencias sobre aprendizaje automático y razonamiento probabilístico, incluyendo NeurIPS, ICML, UAI, y AISTATS. Se han adjuntado talleres regulares de PPL a las conferencias NeurIPS y POPL (Principles of Programming Languages), y la primera Conferencia Internacional sobre Programación Probabilística se celebró en 2018.

CAPÍTULO 16

TOMA DE DECISIONES SENCILLAS

En el que vemos cómo un agente debe tomar decisiones para obtener lo que quiere en un mundo incierto, al menos tanto como sea posible y de manera consistente.

En este capítulo, completamos los detalles de cómo la teoría de la utilidad se combina con la teoría de la probabilidad para dar lugar a un agente basado en la teoría de la decisión, un agente que permite tomar decisiones racionales basadas en lo que cree y lo que quiere. Tal agente permite tomar decisiones en contextos en los que la incertidumbre y los objetivos conflictivos dejan a un agente lógico sin forma de decidir. Un agente basado en objetivos tiene una distinción binaria entre buenos (objetivo) y malos (no objetivo) estados, mientras que un agente basado en la teoría de la decisión asigna un rango continuo de valores a los estados y, por lo tanto, puede elegir un mejor estado aun cuando no hay un mejor estado disponible.

La Sección 16.1 introduce el principio básico de la teoría de la decisión: la maximización de la utilidad esperada. La Sección 16.2 muestra que el comportamiento de un agente racional permite ser modelado maximizando una función de utilidad. La Sección 16.3 discute la naturaleza de las funciones de utilidad con mayor detalle, principalmente su relación con las cantidades individuales como el dinero. La Sección 16.4 muestra cómo manejar las funciones de utilidad que dependen de varias cantidades. En la Sección 16.5, describimos la implementación de sistemas de toma de decisiones. En particular, introducimos un formalismo llamado **red de decisión** (también conocido como **diagrama de influencia**) que extiende las redes bayesianas mediante la incorporación de acciones y utilidades. La Sección 16.6 muestra cómo un agente basado en la teoría de la decisión permite calcular el valor de adquirir nueva información para mejorar sus decisiones.

Mientras que las Secciones 16.1-16.6 asumen que el agente opera con una función de utilidad dada y conocida, la Sección 16.7 relaja esta suposición. Discutimos las consecuencias de la incertidumbre de preferencia por parte de la máquina, la más importante de las cuales es la deferencia con los humanos.

16.1 Combinando creencias y deseos bajo incertidumbre

Comenzamos con un agente que, como todos los agentes, tiene que tomar una decisión. Tiene disponibles algunas acciones a. Puede haber incertidumbre sobre el estado actual, así que asumiremos que el agente asigna una probabilidad $P(s)$ a cada posible estado actual s. También existe incertidumbre sobre los resultados de las acciones; el modelo de transición está dado por $P(s'|s,a)$, la probabilidad de que la acción a en el estado s llegue al estado s'. Como estamos interesados en el resultado s', también usaremos la notación abreviada $P(\text{RESULT}(a) = s')$, la probabilidad de llegar a s' haciendo a en el estado actual, sea cual sea. Los dos están relacionados de la siguiente manera:

$$P(\text{RESULT}(a) = s') = \sum_s P(s)P(s'|s,a).$$

La teoría de la decisión, en su forma más simple, trata de elegir entre acciones basadas en la deseabilidad de sus resultados *inmediatos*; es decir, se asume que el entorno es episódico en el

sentido definido en la página 45. (Esta suposición se relaja en el Capítulo 17). Las preferencias del agente se capturan mediante una **función de utilidad**, $U(s)$, que asigna un único número para expresar la conveniencia de un estado. La **utilidad esperada** de una acción dada la evidencia, $EU(a)$, es el valor medio de la utilidad de los resultados, ponderado por la probabilidad de que el resultado ocurra:

Función de utilidad
Utilidad esperada

$$EU(a) = \sum_{s'} P(\text{RESULT}(a) = s')U(s') \tag{16.1}$$

El principio de **máxima utilidad esperada** (**MEU**) dice que un agente racional debe elegir la acción que maximiza la utilidad esperada del agente:

$$\textit{acción} = \underset{a}{\textit{argmax}}\ EU(a).$$

En cierto sentido, el principio MEU podría ser visto como una prescripción para el comportamiento inteligente. Todo lo que un agente inteligente tiene que hacer es calcular las diversas cantidades, maximizar la utilidad sobre sus acciones, y ya está. Pero esto no significa que el problema de la IA ¡se *resuelva* con la definición!

El principio MEU *formaliza* la noción general de que un agente inteligente debería "hacer lo correcto", pero no *operativiza* ese consejo. Estimar la distribución de probabilidad $P(s)$ sobre los posibles estados del mundo, que se pliega en $P(\text{RESULT}(a)=s')$, requiere percepción, aprendizaje, representación del conocimiento e inferencia. Calcular $P(\text{RESULT}(a)=s')$ en sí mismo requiere un modelo causal del mundo. Existen múltiples acciones a considerar, y calcular las utilidades de los resultados $U(s')$ puede requerir más búsqueda o planificación porque un agente puede no saber cuán bueno es un estado hasta que sabe a dónde es posible llegar desde ese estado. Un sistema de IA actuando en nombre de un humano puede no conocer la verdadera función de utilidad del humano, por lo que es posible que exista incertidumbre sobre U. En resumen, la teoría de la decisión no es una panacea que resuelve el problema de la IA, pero sí proporciona los comienzos de un marco matemático básico generalizado, que permite definir el problema de la IA.

El principio MEU tiene una clara relación con la idea de medidas de rendimiento introducidas en el Capítulo 2. La idea básica es simple, considera los entornos que podrían llevar a un agente a tener un historial de percepción dado, y considera los diferentes agentes que podríamos diseñar. *Si un agente actúa de manera que maximiza una función de utilidad que refleje de manera correcta la medida de rendimiento, entonces el agente logrará la puntuación de rendimiento más alta posible (promediada en todos los posibles entornos).* Esta es la justificación central para el principio MEU en sí. Si bien la afirmación parece tautológica; de hecho, incorpora una transición muy importante de la medida de rendimiento externa a una función de utilidad interna. La medida de rendimiento da una puntuación para una historia, una secuencia de estados. Por lo tanto, se aplica después de que un agente completa una secuencia de acciones. La función de utilidad se aplica al siguiente estado, por lo que se usa para guiar las acciones paso a paso.

16.2 La base de la teoría de la utilidad

De manera intuitiva, el principio de Máxima Utilidad Esperada (MEU) parece una forma razonable de tomar decisiones, pero de ninguna manera es obvio que sea la única forma racional. Después de todo, ¿por qué maximizar la utilidad *promedio* es tan especial? ¿Qué hay de malo en un agente que maximiza la suma ponderada de los cubos de las posibles utilidades, o intenta minimizar la peor pérdida posible? ¿Podría un agente actuar de manera racional sólo expresando preferencias entre estados, sin darles valores numéricos? Finalmente, ¿por qué debería existir una función de utilidad con las propiedades requeridas en absoluto? Veremos.

16.2.1 Restricciones sobre las preferencias racionales

Estas preguntas pueden ser respondidas escribiendo algunas restricciones sobre las preferencias que un agente racional debería tener y luego mostrando que el principio MEU se deriva de las restricciones. Usamos la siguiente notación para describir las preferencias de un agente:

$A \succ B$ el agente prefiere A sobre B.
$A \sim B$ el agente es indiferente entre A y B.
$A \succsim B$ el agente prefiere A sobre B o es indiferente entre ellos.

Ahora la pregunta obvia es, ¿qué tipos de cosas son A y B? Podrían ser estados del mundo, en escenarios que no hay incertidumbre sobre lo que realmente se ofrece. Por ejemplo, un pasajero de aerolínea al que se le ofrece "el plato de pasta o el pollo" no sabe qué se esconde debajo de la cubierta de papel de aluminio.[1] La pasta podría ser deliciosa o congelada, el pollo podría estar jugoso o muy cocido para reconocerlo. Podemos pensar en el conjunto de resultados para cada acción como una **lotería**, piense en cada acción como un boleto. Una lotería L con posibles resultados $S_1,...,S_n$ que ocurren con probabilidades $p_1,...,p_n$ se escribe

Lotería

$$L = [p_1,S_1;p_2,S_2;...p_n,S_n].$$

En general, cada resultado S_i de una lotería es un estado atómico u otra lotería. El problema principal para la teoría de la utilidad es entender cómo las preferencias entre loterías complejas están relacionadas con las preferencias entre los estados subyacentes en esas loterías. Para abordar este problema, enumeramos seis axiomas o restricciones que requerimos que cualquier relación de preferencia razonable obedezca:

Ordenabilidad

- **Ordenabilidad**: Dadas dos loterías, un agente racional debe preferir una o calificarlas como preferibles. Es decir, el agente no puede evitar decidir. Como se señaló en la página 394, negarse a apostar es como negarse a permitir que pase el tiempo.

  Exactamente uno de $(A \succ B)$, $(B \succ A)$, o $(A \sim B)$ se mantiene.

Transitividad

- **Transitividad**: Dadas tres loterías, si un agente prefiere A sobre B y prefiere B sobre C, entonces el agente debe preferir A sobre C.

  $(A \succ B) \wedge (B \succ C) \Rightarrow (A \succ C)$.

Continuidad

- **Continuidad**: Si alguna lotería B está entre A y C en preferencia, entonces hay alguna probabilidad p para la cual el agente racional será indiferente entre obtener B con seguridad y la lotería que produce A con probabilidad p, y C con probabilidad $1 - p$.

  $A \succ B \succ C \Rightarrow \exists p\ [p,A;\ 1 - p,C] \sim B$.

Sustituibilidad

- **Sustituibilidad**: Si un agente es indiferente entre dos loterías A y B, entonces el agente es indiferente entre dos loterías más complejas que son iguales excepto que B se sustituye por A en una de ellas. Esto se mantiene independiente de las probabilidades y los otros resultados en las loterías.

  $A \sim B \Rightarrow [p,A;\ 1 - p,C] \sim [p,B;\ 1 - p,C]$.

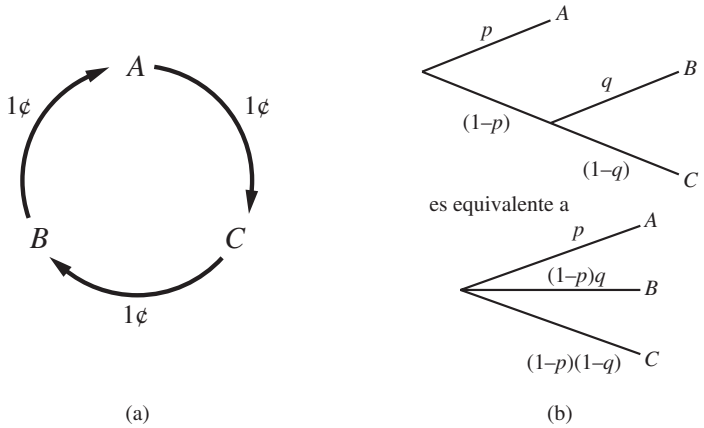
  Esto también se mantiene si sustituimos $\succ$ por $\sim$ en este axioma.

Monotonicidad

- **Monotonicidad**: Suponga que dos loterías tienen los mismos dos posibles resultados, A y B. Si un agente prefiere A sobre B, entonces el agente debe preferir la lotería que tiene una mayor probabilidad para A (y viceversa).

  $A \succ B \Rightarrow (p > q \Leftrightarrow [p,A;\ 1 - p,B] \succ [q,A;\ 1 - q,B])$.

[1] Ofrecemos disculpas a los lectores cuyas aerolíneas locales ya no ofrecen comida en vuelos largos.



(a) (b)

Figura 16.1 (a) Las preferencias no transitivas $A \succ B \succ C \succ A$ resultan en un comportamiento irracional: un ciclo de intercambios cada uno costando un centavo. (b) El teorema de descomposición.

- **Descomposición**: Las loterías compuestas permiten reducirse a más simples utilizando las leyes de probabilidad. Esto ha sido llamado la regla de "no diversión en el juego": como muestra la Figura 16.1(b), comprime dos loterías consecutivas en una sola lotería equivalente.[2] Descomposición

  $$[p,A;\ 1-p,[q,B;\ 1-q,C]] \sim [p,A;\ (1-p)q,B;\ (1-p)(1-q),C].$$

Estas restricciones son conocidas como axiomas de la teoría de la utilidad. Cada axioma permite ser motivado mostrando que un agente que lo viola exhibirá un comportamiento patente irracional en algunas situaciones. Por ejemplo, podemos motivar la transitividad haciendo que un agente con preferencias no transitivas nos dé todo su dinero. Suponga que el agente tiene las preferencias no transitivas $A \succ B \succ C \succ A$, donde A, B, y C son bienes que son intercambiados libremente. Si el agente en al tiempo actual tiene A, entonces podríamos ofrecer intercambiar C por A más un centavo. El agente prefiere C, por lo tanto, estaría dispuesto a hacer este intercambio. Luego podríamos ofrecer intercambiar B por C, extrayendo otro centavo y, finalmente, intercambiar A por B. Esto nos devuelve al punto de partida, excepto que el agente nos ha dado tres centavos (Figura 16.1(a)). Podemos seguir dando vueltas al ciclo hasta que el agente no tenga dinero en absoluto. Note que el agente ha actuado de manera irracional en este caso.

16.2.2 Las preferencias racionales conducen a la utilidad

Note que los axiomas de la teoría de la utilidad son axiomas sobre preferencias, no dicen nada sobre una función de utilidad. De hecho, de los axiomas de la utilidad podemos derivar las siguientes consecuencias (para la prueba, consulte Von Neumann y Morgenstern, 1944):

- **Existencia de la función de utilidad**: Si las preferencias de un agente obedecen a los axiomas de la utilidad, entonces existe una función U tal que $U(A) > U(B)$ si y sólo si A es preferida a B, y $U(A) = U(B)$ si y sólo si el agente es indiferente entre A y B. Es decir,

  $$U(A) > U(B) \Leftrightarrow A \succ B \text{ y } U(A) = U(B) \Leftrightarrow A \sim B.$$

[2] Podemos tener en cuenta el disfrute del juego codificando los eventos de juego en la descripción del estado; por ejemplo, "Tener $10 y haber apostado" podría ser preferido a "Tener $10 y no haber apostado".

- **Utilidad esperada de una lotería**: La utilidad de una lotería es la suma de la probabilidad de cada resultado por la utilidad de ese resultado.

$$U([p_1,S_1;...;p_n,S_n]) = \sum_i p_i U(S_i).$$

En otras palabras, una vez que se especifican las probabilidades y utilidades de los posibles estados de resultado, la utilidad de una lotería compuesta que involucra esos estados está completamente determinada. Debido a que el resultado de una acción no determinista es una lotería, se deduce que un agente actúa de manera racional; es decir, de acuerdo con sus preferencias, sólo eligiendo una acción que maximice la utilidad esperada de acuerdo con la Ecuación (16.1).

Los teoremas anteriores establecen que (asumiendo las restricciones sobre las preferencias racionales) *existe* una función de utilidad para cualquier agente racional. Los teoremas no establecen que la función de utilidad sea *única*. Es fácil ver, de hecho, que el comportamiento de un agente no cambiaría si su función de utilidad $U(S)$ se transformara de acuerdo a

$$U'(S) = aU(S) + b, \tag{16.2}$$

donde a y b son constantes y $a > 0$; una transformación afín positiva.[3] Este hecho fue observado en el Capítulo 5 (página 167) para juegos de azar de dos jugadores; aquí, vemos que se aplica a todo tipo de escenarios de decisión.

Función de valor
Función de utilidad ordinal

Como en el juego, en un ambiente determinista un agente necesita sólo una clasificación de preferencia en los estados, los números no importan. Esto se llama una **función de valor** o **función de utilidad ordinal**.

Es importante recordar que la existencia de una función de utilidad que describe el comportamiento de preferencia de un agente no necesariamente significa que el agente esté maximizando de manera explícita esa función de utilidad en sus propias deliberaciones. Como mostramos en el Capítulo 2, el comportamiento racional puede generarse de cualquier número de formas. Un agente racional podría implementarse con una búsqueda en la tabla (si el número de estados posibles es lo suficientemente pequeño).

Al ver el comportamiento de un agente racional, un observador puede aprender sobre la función de utilidad que representa lo que el agente trata de lograr (aun si el agente no lo sabe). Volveremos a este punto en la Sección 16.7.

16.3 Funciones de utilidad

Las funciones de utilidad mapean de loterías a números reales. Sabemos que deben obedecer seis axiomas o restricciones: ordenabilidad, transitividad, continuidad, sustituibilidad, monotonicidad y descomposición (paradoja de Banach–Tarski o teorema de descomposición). ¿Es eso todo lo que podemos decir sobre las funciones de utilidad? Estrictamente hablando, eso es todo: un agente puede tener cualquier preferencia que le guste. Por ejemplo, un agente podría preferir tener un número primo de dólares en su cuenta bancaria; en cuyo caso, si tuviera $16 regalaría $3. Esto podría ser inusual, pero no podemos llamarlo irracional. Un agente podría preferir un Ford Pinto abollado de 1973 a un Mercedes nuevo y brillante. El agente podría preferir números primos de dólares sólo cuando posee el Pinto, pero cuando posee el Mercedes, podría preferir más dólares a menos. Por fortuna, las preferencias de los agentes reales suelen ser más sistemáticas y, por lo tanto, más fáciles de manejar.

[3] En este sentido, las utilidades se parecen a las temperaturas: una temperatura en Fahrenheit es 1.8 veces la temperatura Celsius más 32, pero convertir de una a otra no te hace sentir más caliente o más frío.

16.3.1 Evaluación de la utilidad y escalas de utilidad

Si queremos construir un sistema de teoría de decisiones que ayude a un humano a tomar decisiones o actúe en su nombre, primero debemos averiguar cuál es la función de utilidad del humano. Este proceso, con frecuencia llamado **provocación de preferencias**, implica presentar opciones al humano y usar las preferencias observadas para precisar la función de utilidad subyacente.

Provocación de preferencias

La ecuación (16.2) dice que no hay una escala absoluta para las utilidades, pero es útil; sin embargo, establecer *alguna* escala en la que se puedan registrar y comparar las utilidades para cualquier problema en particular. Es posible establecer una escala fijando las utilidades de dos resultados particulares, al igual que fijamos una escala de temperatura fijando el punto de congelación y el punto de ebullición del agua. En general, fijamos la utilidad de un "mejor premio posible" en $U(S) = u_{\top}$ y una "peor posible catástrofe" en $U(S) = u_{\bot}$. (Ambos deben ser finitos). Las **utilidades normalizadas** usan una escala con $u_{\bot} = 0$ y $u_{\top} = 1$. Con tal escala, un fanático de Inglaterra podría asignar una utilidad de 1 a Inglaterra ganando la Copa del Mundo y una utilidad de 0 a Inglaterra al no clasificar.

Utilidades normalizadas

Dada una escala de utilidad entre $u_{\top}$ y $u_{\bot}$, podemos evaluar la utilidad de cualquier premio particular S pidiendo al agente que elija entre S y una **lotería estándar** $[p,u_{\top};(1-p),u_{\bot}]$. La probabilidad p se ajusta hasta que el agente es indiferente entre S y la lotería estándar. Asumiendo utilidades normalizadas, la utilidad de S está dada por p. Una vez hecho esto para cada premio, se determinan las utilidades para todas las loterías que involucran esos premios. Suponga, por ejemplo, que queremos saber cuánto valora nuestro fanático de Inglaterra el resultado de que Inglaterra llegue a la semifinal y luego pierda. Comparamos ese resultado con una lotería estándar con probabilidad p de ganar el trofeo y probabilidad $1 - p$ de un fracaso ignominioso para calificar. Si hay indiferencia en $p = 0.3$, entonces 0.3 es el valor de llegar a la semifinal y luego perder.

Lotería estándar

En problemas de decisión médica, de transporte, ambientales y otros, las vidas de las personas están en juego. (Sí, hay cosas más importantes que las fortunas de Inglaterra en la Copa del Mundo). En tales casos, $u_{\bot}$ es el valor asignado a la muerte inmediata (o en los peores casos, muchas muertes). *Aunque nadie se siente cómodo poniendo un valor en la vida humana, es un hecho que todo el tiempo se hacen compensaciones en asuntos de vida y muerte.* Los aviones reciben una revisión completa a intervalos, en lugar de después de cada viaje. Los automóviles se fabrican de una manera que equilibra los costos contra las tasas de supervivencia en accidentes. Toleramos un nivel de contaminación del aire que mata a cuatro millones de personas al año.

De manera paradójica, negarse a poner un valor monetario a la vida significa que la vida se infravalora. Ross Shachter describe una agencia gubernamental que encargó un estudio sobre la eliminación de asbestos de las escuelas. Los analistas de decisiones que realizaron el estudio asumieron un valor particular en dólares para la vida de un niño en edad escolar, y argumentaron que la elección racional bajo esa suposición era eliminar el asbesto. La agencia, indignada por la idea de establecer el valor de una vida, rechazó el informe de plano. Luego decidió en contra de la eliminación del asbesto, afirmando de manera implícita un valor más bajo para la vida de un niño que el asignado por los analistas.

En la actualidad varias agencias del gobierno de Estados Unidos, incluida la Agencia de Protección Ambiental, la Administración de Alimentos y Medicamentos, y el Departamento de Transporte, usan el **valor de una vida estadística** para determinar los costos y beneficios de las regulaciones e intervenciones. Los valores típicos en 2019 son de unos $10 millones.

Valor de una vida estadística

Se han hecho algunos intentos para averiguar el valor que las personas asignan a sus propias vidas. Una moneda común usada en el análisis médico y de seguridad es el **micromort**, una posibilidad entre un millón de morir. Si pregunta a las personas cuánto pagarían para evitar un riesgo, por ejemplo, para evitar jugar a la ruleta rusa con un revólver de un millón de tambores,

Micromort

responderán con cifras muy grandes, quizá decenas de miles de dólares, pero su comportamiento real refleja un valor monetario mucho menor para un micromort.

Por ejemplo, en el Reino Unido, conducir un coche durante 230 millas implica un riesgo de un micromort. A lo largo de la vida de su coche, digamos, 92 000 millas, eso son 400 micromorts. Las personas parecen estar dispuestas a pagar alrededor de $12 000 más por un coche más seguro que reduce a la mitad el riesgo de muerte. Por lo tanto, su acción de compra de coches dice que tienen un valor de $60 por micromort. Varios estudios han confirmado una cifra en este rango en muchos individuos y tipos de riesgo. Sin embargo, las agencias gubernamentales como el Departamento de Transporte de Estados Unidos suelen establecer una cifra más baja; sólo gastarán alrededor de $6 en reparaciones de carreteras por vida esperada salvada. Por supuesto, estos cálculos sólo son válidos para riesgos pequeños. La mayoría de las personas no aceptarán matarse a sí mismas, aun por $60 millones.

QALY

Otra medida es el **QALY**, o año de vida ajustado por calidad. Los pacientes están dispuestos a aceptar una expectativa de vida más corta para evitar una discapacidad. Por ejemplo, los pacientes con insuficiencia renal en promedio son indiferentes entre vivir dos años en diálisis y un año en plena salud.

16.3.2 La utilidad del dinero

La teoría de la utilidad tiene sus raíces en la economía, pues la economía proporciona un candidato obvio para una medida de utilidad: el dinero (o, más específico, los activos netos totales de un agente). La casi universal intercambiabilidad del dinero por todo tipo de bienes y servicios sugiere que el dinero desempeña un papel significativo en las funciones de utilidad humana.

Preferencia monótona

Por lo general, será el caso de que un agente prefiera más dinero a menos, todas las demás cosas siendo iguales. Decimos que el agente muestra una **preferencia monótona** por más dinero. Esto no significa que el dinero se comporte como una función de utilidad, porque no dice nada sobre las preferencias entre *loterías* que involucran dinero.

Suponga que ha triunfado sobre los otros competidores en un concurso de televisión. El presentador ahora le ofrece una elección: llevarse el premio de $1 000 000 o apostarlo en un volado al lanzar una moneda. Si la moneda sale cara, usted pierde todo; pero si sale cruz, usted obtiene $2 500 000. Si actúa como la mayoría de las personas, rechazaría la apuesta y se llevaría el millón. ¿Está siendo irracional?

Valor monetario esperado

Asumiendo que la moneda no está falseada, el **valor monetario esperado** (EMV) de la apuesta es $\frac{1}{2}(\$0) + \frac{1}{2}(\$2\,500\,000) = \$1\,250\,000$, que es más que el original $1 000 000. Pero eso no significa que aceptar la apuesta es una mejor decisión. Suponga que usamos S_n para denotar el estado de poseer una riqueza total de $\$n$, y que su riqueza actual es $\$k$. Entonces las utilidades esperadas de las dos acciones de aceptar y rechazar la apuesta son

$$\begin{aligned} EU(Aceptar) &= \tfrac{1}{2}U(S_k) + \tfrac{1}{2}U(S_{k+2\,500\,000}), \\ EU(Rechazar) &= U(S_{k+1\,000\,000}). \end{aligned}$$

Para determinar qué hacer, necesitamos asignar utilidades a los estados finales. La utilidad no es directamente proporcional al valor monetario, porque la utilidad de su primer millón es muy alta (o eso dicen), mientras que la utilidad de un millón adicional es menor. Suponga que asigna una utilidad de 5 a su estado financiero actual (S_k), un 9 al estado $S_{k+2\,500\,000}$, y un 8 al estado $S_{k+1\,000\,000}$. Entonces, la acción racional sería declinar, porque la utilidad esperada de aceptar es sólo 7 (menos que el 8 por declinar). Por otro lado, un multimillonario quizá tendría una función de utilidad que es localmente lineal en el rango de unos pocos millones más y, por lo tanto, aceptaría la apuesta.

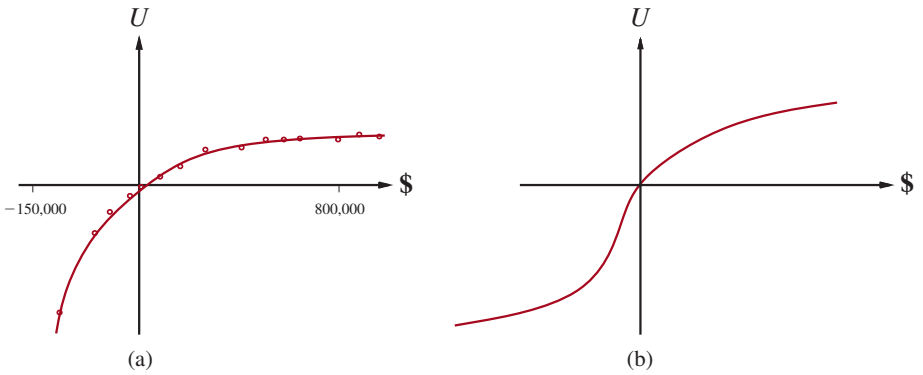


Figura 16.2 La utilidad del dinero. (a) Datos empíricos para el Sr. Beard en un rango limitado. (b) Una curva típica para el rango completo.

En un estudio pionero de las funciones de utilidad reales, Grayson (1960) encontró que la utilidad del dinero era proporcional al *logaritmo* de la cantidad. (Esta idea fue sugerida por primera vez por Bernoulli (1738); ver Ejercicio 16.STPT). Una curva de utilidad particular, para un cierto Sr. Beard, se muestra en la Figura 16.2(a). Los datos obtenidos para las preferencias del Sr. Beard son consistentes con una función de utilidad

$$U(S_{k+n}) = -263.31 + 22.09 \log(n + 150\,000)$$

para el rango entre $n = -\$150\,000$ y $n = \$800\,000$.

No debemos suponer que esta es la función de utilidad definitiva para el valor monetario, pero es probable que la mayoría de las personas tengan una función de utilidad que es cóncava para la riqueza positiva. Endeudarse es malo, pero las preferencias entre diferentes niveles de deuda pueden mostrar una inversión de la concavidad asociada con la riqueza positiva. Por ejemplo, alguien que ya tiene una deuda de \$10 000 000 podría aceptar una apuesta en un volado justo con una ganancia de \$10 000 000 para cara y una pérdida de \$20 000 000 para cruz.[4] Esto produce la curva en forma de S mostrada en la Figura 16.2(b).

Si restringimos nuestra atención a la parte positiva de las curvas, donde la pendiente está disminuyendo, entonces para cualquier lotería L, la utilidad de enfrentarse a esa lotería es menor que la utilidad de recibir el valor monetario esperado de la lotería como algo seguro:

$$U(L) < U(S_{EMV(L)}).$$

Es decir, los agentes con curvas de esta forma son **adversos al riesgo**: prefieren algo seguro con un pago que es menor que el valor monetario esperado de una apuesta. Por otro lado, en la región "desesperada" de gran riqueza negativa en la Figura 16.2(b), el comportamiento es de **búsqueda de riesgo**. El valor que un agente aceptará en lugar de una lotería se llama el **equivalente cierto** de la lotería. Los estudios han demostrado que la mayoría de las personas aceptarán alrededor de \$400 en lugar de una apuesta que da \$1 000 la mitad del tiempo y \$0 la otra mitad; es decir, el equivalente cierto de la lotería es \$400, mientras que el EMV es \$500.

Adversos al riesgo

Búsqueda de riesgo

Equivalente cierto

La diferencia entre el EMV de una lotería y su equivalente cierto se llama **prima de seguro**. La aversión al riesgo es la base de la industria del seguro, porque significa que las primas de seguro

Prima de seguro

[4] Tal comportamiento podría llamarse desesperado, pero es racional si uno ya se encuentra en una situación desesperada.

son positivas. Las personas preferirían pagar una pequeña prima de seguro que apostar el precio de su casa contra la posibilidad de un incendio. Desde el punto de vista de la compañía de seguros, el precio de la casa es muy pequeño en comparación con las reservas totales de la empresa. Esto significa que la curva de utilidad del asegurador es lineal en una región tan pequeña, y la apuesta le cuesta a la compañía casi nada.

Neutral al riesgo

Observe que para *pequeños* cambios en la riqueza en relación con la riqueza actual casi cualquier curva será lineal. Se dice que un agente que tiene una curva lineal es **neutral al riesgo**. Por lo tanto, para apuestas con sumas pequeñas, esperamos neutralidad al riesgo. En cierto sentido, esto justifica el procedimiento simplificado que propone pequeñas apuestas para evaluar probabilidades y justificar los axiomas de probabilidad en la Sección 12.2.3.

16.3.3 Utilidad esperada y decepción posterior a la decisión

La forma racional de elegir la mejor acción, a^*, es maximizar la utilidad esperada:

$$a^* = \underset{a}{argmax}\, EU(a).$$

Si hemos calculado la utilidad esperada de manera correcta según nuestro modelo de probabilidad, y si este modelo de probabilidad refleja correctamente los procesos estocásticos subyacentes que generan los resultados, entonces, en promedio obtendremos la utilidad que esperamos si todo el proceso se repite muchas veces.

Sin embargo, en realidad nuestro modelo en general simplifica demasiado la situación real, ya sea porque no sabemos lo suficiente (al tomar una decisión de inversión compleja) o porque el cálculo de la verdadera utilidad esperada es demasiado difícil (por ejemplo, al hacer un movimiento en backgammon, necesitando tener en cuenta todos los posibles lanzamientos de dados futuros). En ese caso, estamos trabajando con *estimaciones* $\widehat{EU}(a)$ de la verdadera utilidad esperada. Asumiremos que las estimaciones no están **sesgadas;** es decir, el valor esperado del error, $E(\widehat{EU}(a) - EU(a))$, es cero. En ese caso, todavía parece razonable elegir la acción con la mayor utilidad estimada y esperar recibir esa utilidad, en promedio, cuando se ejecuta la acción.

Sesgadas

Por desgracia, el resultado real en general será significativamente *peor* de lo que estimamos, ¡aunque la estimación no estaba sesgada! Para ver por qué, considere un problema de decisión en el que hay k opciones, cada una de las cuales tiene una utilidad estimada verdadera de 0. Supongamos que el error en cada estimación de utilidad es independiente y tiene una distribución normal unitaria; es decir, una Gaussiana con media cero y desviación estándar de 1, mostrada como la curva en negrita en la Figura 16.3. Ahora, a medida que comenzamos a generar las estimaciones, algunos de los errores serán negativos (pesimistas) y algunos serán positivos (optimistas). Debido a que seleccionamos la acción con la estimación de utilidad *más alta*, estamos favoreciendo las estimaciones optimistas, y esa es la fuente del sesgo.

Es un asunto sencillo calcular la distribución del máximo de las k estimaciones y, por lo tanto, cuantificar el alcance de nuestra decepción. (Este cálculo es un caso especial de calcular una **estadística de orden**, la distribución de cualquier elemento clasificado particular de una muestra). Suponga que cada estimación X_i tiene una función de densidad de probabilidad $f(x)$ y distribución acumulativa $F(x)$. (Como se explica en el Apéndice A, la distribución acumulativa F mide la probabilidad de que el costo sea menor o igual a cualquier cantidad dada; es decir, integra la densidad original f.) Ahora, que X^* sea la estimación más grande, como $\max\{X_1,...,X_k\}$. Entonces la distribución acumulativa para X^* es

Estadística de orden

$$\begin{aligned} P(\max\{X_1,...,X_k\} \le x) &= P(X_1 \le x,...,X_k \le x) \\ &= P(X_1 \le x)...P(X_k \le x) = F(x)^k. \end{aligned}$$

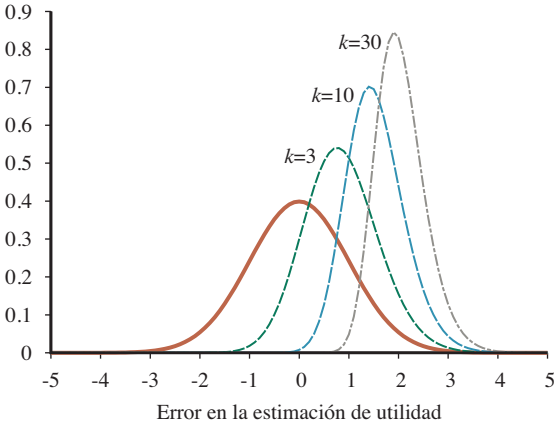


Figura 16.3 Optimismo injustificado causado por elegir la mejor de k opciones: asumimos que cada opción tiene una utilidad verdadera de 0 pero una estimación de utilidad que se distribuye según una normal unitaria (curva marrón). Las otras curvas muestran las distribuciones del máximo de k estimaciones para $k = 3$, 10 y 30.

La función de densidad de probabilidad es la derivada de la función de distribución acumulativa, por lo que la densidad para X^*, el máximo de k estimaciones, es

$$P(x) = \frac{d}{dx}\left(F(x)^k\right) = kf(x)(F(x))^{k-1}.$$

Estas densidades se muestran para diferentes valores de k en la Figura 16.3 para el caso donde $f(x)$ es la normal unitaria. Para $k = 3$, la densidad para X^* tiene una media alrededor de 0.85, por lo que la decepción promedio será de casi 85 % de la desviación estándar en las estimaciones de utilidad. Con más opciones, es más probable que surjan estimaciones optimistas: para $k = 30$, la decepción será de alrededor del doble de la desviación estándar en las estimaciones.

Esta tendencia a que la utilidad esperada estimada de la mejor opción sea demasiado alta se llama **maldición del optimizador** (Smith y Winkler, 2006). Afecta incluso a los más experimentados en la toma de decisiones de analistas y estadísticos. Las manifestaciones graves incluyen creer que un nuevo medicamento emocionante que ha curado a 80 % de los pacientes en un ensayo curará a 80 % de los pacientes (se ha elegido de k = miles de medicamentos candidatos) o que un fondo mutuo anunciado como que tiene rendimientos superiores al promedio continuará teniéndolos (se ha elegido para aparecer en el anuncio de k = docenas de fondos en la cartera general de la compañía). Incluso es posible el caso de que lo que parece ser la mejor opción no lo sea, si la varianza en la estimación de utilidad es alta: un medicamento que ha curado a 9 de 10 pacientes y ha sido seleccionado de entre miles de medicamentos probados, quizás sea *peor* que uno que ha curado a 800 de 1 000.

Maldición del optimizador

La maldición del optimizador surge en todas partes debido a la ubicuidad de los procesos de selección que maximizan la utilidad, por lo que tomar las estimaciones de utilidad al pie de la letra es una mala idea. Podemos evitar la maldición con un enfoque bayesiano que usa un modelo de probabilidad explícito $\mathbf{P}(\widehat{EU}\,|\,EU)$ del error en las estimaciones de utilidad. Dado este modelo y un prior sobre lo que podríamos esperar que sean las utilidades, tratamos la estimación de utilidad como evidencia y calculamos la distribución posterior para la utilidad verdadera usando la regla de Bayes.

16.3.4 Juicio humano e irracionalidad

Teoría normativa
Teoría descriptiva

La teoría de la decisión es una **teoría normativa** que describe cómo *debería* actuar un agente racional. Una **teoría descriptiva**, por otro lado, describe cómo actúan los agentes reales, como los humanos. La aplicación de la teoría económica se vería muy mejorada si ambas, la normativa y la descriptiva, coincidieran; pero parece haber algunas pruebas experimentales en contra. La evidencia sugiere que los humanos son "predeciblemente irracionales" (Ariely, 2009).

El problema más conocido es la paradoja de Allais (Allais, 1953). A las personas se les da a elegir entre las loterías A y B, y luego entre C y D, que tienen los siguientes premios:

A: 80 % de probabilidad de \$4 000 C: 20 % de probabilidad de \$4 000
B: 100 % de probabilidad de \$3 000 D: 25 % de probabilidad de \$3 000

La mayoría de las personas prefieren B sobre A (toman lo seguro), y C sobre D (toman el EMV más alto). ¡El análisis normativo no está de acuerdo! Podemos ver esto con más claridad si usamos la libertad implícita en la Ecuación (16.2) para establecer $U(\$0) = 0$. En ese caso, entonces $B \succ A$ implica que $U(\$3\,000) > 0.8U(\$4\,000)$, mientras que $C \succ D$ implica lo contrario. En otras palabras, no hay una función de utilidad que sea consistente con estas elecciones.

Efecto de certeza

Una explicación para las preferencias aparentemente irracionales es el **efecto de certeza** (Kahneman y Tversky, 1979): las personas se sienten atraídas por las ganancias que son ciertas. Hay varias razones por las que esto puede ser así.

Primero, las personas pueden preferir reducir su carga computacional; al elegir resultados ciertos, no tienen que calcular con probabilidades. Pero el efecto persiste incluso cuando los cálculos involucrados son muy fáciles.

En segundo lugar, las personas desconfían de la legitimidad de las probabilidades declaradas. Confío en que un volado es 50/50 si tengo control sobre la moneda y el lanzamiento, pero puedo desconfiar del resultado si el lanzamiento lo hace alguien con un interés personal en el resultado.[5] En presencia de desconfianza, podría ser mejor optar por lo seguro.[6]

Tercero, las personas pueden tener en cuenta su estado emocional además de su estado financiero. Las personas saben que experimentarían arrepentimiento si renunciaran a una recompensa segura (B) por un 80 % de probabilidad de una recompensa más alta y luego perdieran.

En otras palabras, si se elige A, hay 20 % de probabilidad de no obtener dinero *y sentirse como un completo idiota*, lo cual es peor que no obtener dinero. Entonces, quizá las personas que eligen B sobre A y C sobre D no son irracionales; están dispuestos a renunciar a \$200 de EMV para evitar 20 % de probabilidad de sentirse como un idiota.

Un problema relacionado es la paradoja de Ellsberg. Aquí los premios son fijos, pero las probabilidades están subdeterminadas. Su pago dependerá del color de una bola elegida de una urna. Se le dice que la urna contiene 1/3 de bolas rojas, y 2/3 de bolas negras o amarillas, pero no sabe cuántas son negras y cuántas amarillas. De nuevo, se le pregunta si prefiere la lotería A o B; y luego C o D:

A: \$100 por una bola roja C: \$100 por una bola roja o amarilla
B: \$100 por una bola negra D: \$100 por una bola negra o amarilla.

Debería estar claro que si cree que hay más bolas rojas que negras, entonces debería preferir A sobre B y C sobre D; si piensa que hay menos rojo que negro debería preferir lo opuesto. Pero resulta

[5] Por ejemplo, el matemático/mago Persi Diaconis puede hacer que un volado salga como él quiere cada vez (Landhuis, 2004).

[6] Aun lo seguro es posible no sea cierto. A pesar de las promesas de hierro fundido, no hemos recibido esos \$27 000 000 de la cuenta bancaria nigeriana de un pariente fallecido previamente desconocido.

que la mayoría de las personas prefieren *A* sobre *B* y también prefieren *D* sobre *C*, aunque no hay estado del mundo para el cual esto sea racional. Parece que las personas tienen **ambigüedad de aversión**: *A* le da una probabilidad de 1/3 de ganar, mientras que *B* podría estar en cualquier lugar entre 0 y 2/3. De manera similar, *D* le da una probabilidad de 2/3, mientras que *C* podría estar en cualquier lugar entre 1/3 y 3/3. La mayoría de las personas eligen la probabilidad conocida en lugar de las desconocidas.

Ambigüedad de aversión

Otro problema es que la redacción exacta de un problema de decisión puede tener gran impacto en las elecciones del agente; esto se llama **efecto de encuadre**. Los experimentos muestran que a las personas les gusta un procedimiento médico descrito como tener una "tasa de supervivencia de 90 %" casi el doble que uno descrito como tener una "tasa de mortalidad de 10 %", aunque estas dos afirmaciones significan lo mismo. Esta discrepancia en el juicio se ha encontrado en múltiples experimentos y es la misma ya sean los sujetos pacientes en una clínica, estudiantes de escuela de negocios sofisticados o médicos experimentados.

Efecto de encuadre

Las personas se sienten más cómodas haciendo juicios de utilidad *relativos* en lugar de absolutos. Es posible que tenga poca idea de cuánto podría disfrutar de los diversos vinos que ofrece un restaurante. El restaurante se aprovecha de esto ofreciendo una botella de $200 que nadie comprará, pero que sirve para sesgar al alza la estimación del cliente del valor de todos los vinos, haciendo que una botella de $55 parezca una ganga. Esto se llama el **efecto de anclaje**.

Efecto de anclaje

Si los informantes humanos insisten en juicios de preferencia contradictorios, no hay nada que los agentes automatizados puedan hacer para ser consistentes con ellos. Por fortuna, los juicios de preferencia hechos por los humanos con frecuencia están abiertos a revisión a la luz de una mayor consideración. Paradojas como las de Allais y Ellsberg se reducen mucho (pero no se eliminan) si las opciones se explican mejor. En el trabajo en la Escuela de Negocios de Harvard sobre la evaluación de la utilidad del dinero, Keeney y Raiffa (1976, p. 210) encontraron lo siguiente:

> Los sujetos tienden a ser demasiado adversos al riesgo en lo pequeño y, por lo tanto... las funciones de utilidad ajustadas muestran primas de riesgo inaceptablemente grandes para loterías con una gran dispersión. Sin embargo, la mayoría de los sujetos permiten reconciliar sus inconsistencias y sienten que han aprendido una lección importante sobre cómo quieren comportarse. Como consecuencia, algunos sujetos cancelan su seguro de colisión de automóviles y contratan más seguro a término para sus vidas.

La evidencia de la irracionalidad humana también es cuestionada por los investigadores en el campo de la **psicología evolutiva**, quienes señalan el hecho de que nuestros mecanismos de toma de decisiones cerebrales no evolucionaron para resolver problemas de palabras con probabilidades y premios expresados como números decimales. Concedamos, por el bien del argumento, que el cerebro tiene mecanismos neurales incorporados para calcular con probabilidades y utilidades, o algo funcionalmente equivalente. Si es así, las entradas requeridas se obtendrían a través de la experiencia acumulada de resultados y recompensas en lugar de a través de presentaciones lingüísticas de valores numéricos.

Psicología evolutiva

Está lejos de ser obvio que podamos acceder de manera directa a los mecanismos neurales incorporados del cerebro presentando problemas de decisión en forma lingüística/numérica. El hecho mismo de que diferentes redacciones del *mismo problema de decisión* provocan diferentes opciones, sugiere que el problema de decisión en sí mismo no se está entendiendo. Animados por esta observación, los psicólogos han intentado presentar problemas en razonamiento incierto y toma de decisiones en formas "evolutivamente apropiadas"; por ejemplo, en lugar de decir "tasa de supervivencia de 90 %", el experimentador podría mostrar 100 animaciones de figuras de palo de la operación, donde el paciente muere en 10 de ellas y sobrevive en 90. Con problemas de decisión planteados de esta manera, el comportamiento de las personas parece estar mucho más cerca del estándar de racionalidad.

16.4 Funciones de utilidad multiatributo

La toma de decisiones en el campo de las políticas públicas implica apuestas altas, tanto en dinero como en vidas. Por ejemplo, al decidir qué niveles de emisiones dañinas se permiten a una central eléctrica, los políticos responsables deben sopesar la prevención de la muerte y la discapacidad contra el beneficio de la energía y la carga económica de mitigar las emisiones. Elegir un sitio para un nuevo aeropuerto requiere considerar la interrupción causada por la construcción, el costo de la tierra, la distancia desde los centros de población, el ruido de las operaciones de vuelo, problemas de seguridad derivados de la topografía y las condiciones climáticas locales, etc. Problemas como estos, en los que los resultados se caracterizan por dos o más atributos, son manejados por la **teoría de utilidad multiatributo**. En esencia, es la teoría de comparar manzanas con naranjas.

Teoría de utilidad multiatributo

Si se define $\mathbf{X}= X_1,...,X_n$ y $\mathbf{x}=\langle x_1,...,x_n\rangle$ como un vector completo de asignaciones, donde cada x_i es ya sea un valor numérico o un valor discreto con un orden asumido en valores. El análisis es más fácil si lo organizamos de manera que los valores más altos de un atributo siempre correspondan a utilidades más altas: las utilidades son crecientes. Eso significa que no podemos usar, digamos, el número de muertes, d como un atributo; tendríamos que usar $-d$. También significa que no es posible usar la temperatura de la habitación, t, como un atributo. Si la función de utilidad para la temperatura tiene un pico en 70° F y cae a ambos lados, entonces podríamos dividir el atributo en dos partes. Podríamos usar $t - 70$ para medir si la habitación está lo suficientemente caliente, y $70 - t$ para medir si está lo suficientemente fresca; ambos atributos serían crecientes hasta que alcanzan su valor de utilidad máximo en 0; la curva de utilidad es plana a partir de ese punto, lo que significa que no obtienes más "suficiente caliente" por encima de 70° F, ni más "suficiente fresco" por debajo de 70° F.

Los atributos en el problema del aeropuerto podrían ser:

- *Rendimiento*, medido por el número de vuelos por día;
- *Seguridad*, medida por menos el número esperado de muertes por año;
- *Tranquilidad*, medida por menos el número de personas que viven bajo las rutas de vuelo;
- *Frugalidad*, medida por el costo negativo de construcción.

Comenzamos examinando casos en los que se permiten tomar decisiones *sin* combinar los valores de los atributos en un sólo valor de utilidad. Luego observamos casos en los que las utilidades de las combinaciones de atributos pueden especificarse de manera muy concisa.

16.4.1 Dominancia

Supongamos que el sitio del aeropuerto S_1 cuesta menos, genera menos contaminación acústica y es más seguro que el sitio S_2. No dudaríamos en rechazar S_2. Entonces decimos que hay una **dominancia estricta** de S_1 sobre S_2. En general, si una opción tiene un valor más bajo en todos los atributos que alguna otra opción, no necesita ser considerada más. La dominancia estricta es útil para reducir el campo de opciones a los verdaderos contendientes, aunque rara vez produce una elección única. La Figura 16.4(a) muestra un diagrama esquemático para el caso de dos atributos.

Dominancia estricta

Eso está bien para el caso determinista, en el que se conocen con seguridad los valores de los atributos. ¿Qué pasa con el caso general, donde los resultados son inciertos? Es posible construir un análogo directo de la dominancia estricta, donde, a pesar de la incertidumbre, todos los posibles resultados concretos para S_1 dominan de manera estricta todos los posibles resultados para S_2. (Ver Figura 16.4(b)). Por supuesto, esto quizás ocurra con menor frecuencia que en el caso determinista.

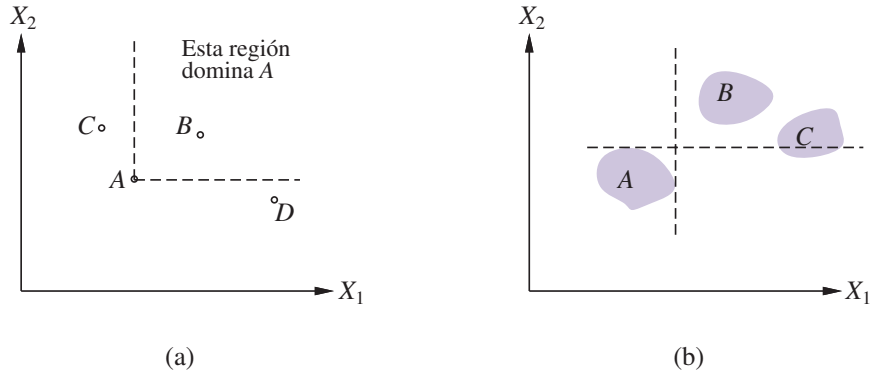


Figura 16.4 Dominancia estricta. (a) Determinista: La opción A está dominada por B pero no por C o D. (b) Incierta: A está dominada por B pero no por C.

Dominancia estocástica

Por fortuna, existe una generalización más útil llamada **dominancia estocástica**, que ocurre con frecuencia en problemas reales. La dominancia estocástica es más fácil de entender en el contexto de un solo atributo. Suponga que creemos que el costo de colocar el aeropuerto en S_1 se distribuye de manera uniforme entre \$2.8 mil millones y \$4.8 mil millones y que el costo en S_2 se distribuye de manera uniforme entre \$3 mil millones y \$5.2 mil millones. Definimos el atributo *Frugalidad* como el costo negativo. La Figura 16.5(a) muestra las distribuciones para la frugalidad de los sitios S_1 y S_2. Luego, dada sólo la información de que la elección más frugal es mejor (todas las demás cosas siendo iguales), podemos decir que S_1 domina de manera estocástica a S_2 (es decir, S_2 permite ser descartado). Es importante notar que esto *no* se sigue de comparar los costos esperados. Por ejemplo, si supiéramos que el costo *exacto* de S_1 es de \$3.8 mil millones, entonces *no podríamos* tomar una decisión sin información adicional sobre la utilidad del dinero. (Podría parecer extraño que *más* información sobre el costo de S_1 podría hacer que el agente sea *menos* capaz de decidir. La paradoja se resuelve al notar que en ausencia de información exacta sobre el costo, la decisión es más fácil de tomar, aunque es más probable que sea incorrecta).

La relación exacta entre las distribuciones de atributos necesarias para establecer la dominancia estocástica se ve mejor al examinar las distribuciones acumulativas, mostradas en la Figura 16.5(b). Si la distribución acumulativa para S_1 siempre está a la derecha de la distribución acumulativa para S_2, entonces, estocásticamente hablando, S_1 es más barato que S_2. Formalmente, si dos acciones A_1 y A_2 conducen a distribuciones de probabilidad $p_1(x)$ y $p_2(x)$ en el atributo X, entonces A_1 domina de manera estocástica A_2 en X si

$$\forall x \int_{-\infty}^{x} p_1(x')\,dx' \leq \int_{-\infty}^{x} p_2(x')\,dx'.$$

La relevancia de esta definición para la selección de decisiones óptimas proviene de la siguiente propiedad: *si A_1 domina de manera estocástica A_2, entonces para cualquier función de utilidad monótona no decreciente $U(x)$, la utilidad esperada de A_1 es al menos tan alta como la utilidad esperada de A_2*. Para ver por qué esto es cierto, considere las dos utilidades esperadas, $\int p_1(x)U(x)dx$ y $\int p_2U(x)dx$. En inicio, no es obvio por qué la primera integral es mayor que la segunda, dado que la condición de dominancia estocástica tiene una integral p_1 que es menor que la integral p_2.

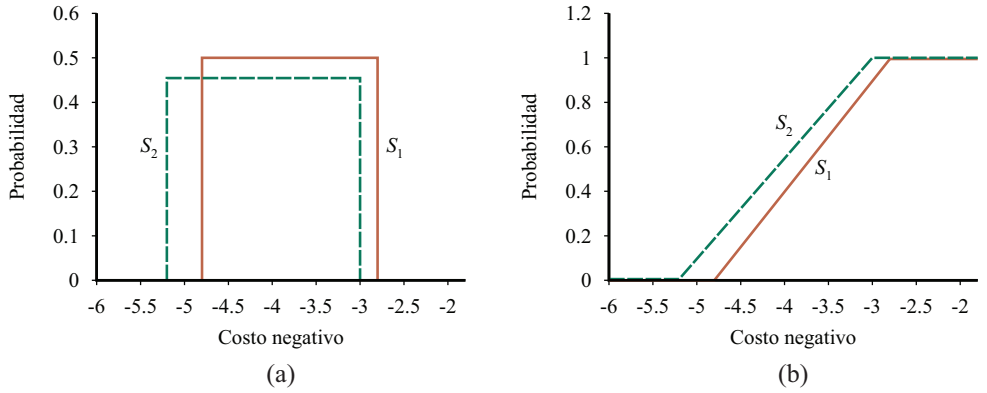


Figura 16.5 Dominancia estocástica. (a) S_1 domina de manera estocástica S_2 en frugalidad (costo negativo). (b) Distribuciones acumulativas para la frugalidad de S_1 y S_2.

Sin embargo, en lugar de pensar en la integral sobre x, piense en la integral sobre y, la probabilidad acumulativa, tal como se muestra en la Figura 16.5(b). Para cualquier valor de y, el valor correspondiente de x (y, por lo tanto, de $U(x)$) es mayor para S_1 que para S_2; así que si integramos una cantidad mayor en todo el rango de y, estamos obligados a obtener un resultado mayor. De manera formal, es sólo una sustitución de $y = P_1(x)$ en la integral para el valor esperado de S_1 y $y = P_2(x)$ en la integral para el de S_2. Con estas sustituciones, tenemos $dy = \frac{d}{dx}(P_1(x))dx = p_1(x)dx$ para S_1 y $dy = p_2(x)dx$ para S_2, por lo tanto

$$\int_{-\infty}^{\infty} p_1(x)U(x)dx = \int_0^1 U(P_1^{-1}(y))dy \geq \int_0^1 U(P_2^{-1}(y))dy = \int_{-\infty}^{\infty} p_2(x)U(x)dx.$$

Esta desigualdad nos permite preferir A_1 a A_2 en un problema de un solo atributo. Más en general, si una acción es dominada de manera estocástica por otra acción en *todos* los atributos en un problema de múltiples atributos, entonces permite ser descartada.

La condición de dominancia estocástica puede parecer bastante técnica y quizá no sea tan fácil de evaluar sin cálculos de probabilidad extensos. De hecho, es posible decidir en múltiples casos. Por ejemplo, ¿preferiría caer de cabeza sobre concreto desde 3 milímetros o 3 metros? Suponiendo que eligió 3 milímetros, ¡buena elección! ¿Por qué es necesariamente una mejor decisión? Hay mucha incertidumbre sobre el grado de daño que sufrirá en ambos casos; pero para cualquier nivel de daño dado, la probabilidad de que sufra al menos ese nivel de daño es mayor cuando cae desde 3 metros que desde 3 milímetros. En otras palabras, 3 milímetros domina de manera estocástica a 3 metros en el atributo de *Seguridad*.

Este tipo de razonamiento es una segunda naturaleza para los humanos; es tan obvio que ni siquiera lo pensamos. La dominación estocástica abunda también en el problema del aeropuerto. Suponga, por ejemplo, que el costo de transporte de la construcción depende de la distancia al proveedor. El costo en sí es incierto, pero cuanto mayor es la distancia, mayor es el costo. Si S_1 está más cerca que S_2, entonces S_1 dominará S_2 en frugalidad. Aunque no los presentaremos aquí, existen algoritmos para propagar este tipo de información cualitativa entre variables inciertas en **redes probabilísticas cualitativas**, lo que permite a un sistema tomar decisiones racionales basadas en estocásticas dominancia de redes, sin usar ningún valor numérico.

Redes probabilísticas cualitativas

16.4.2 Estructura de preferencia y utilidad multiatributo

Suponga que tenemos n atributos, cada uno de los cuales tiene d valores posibles distintos. Para especificar la función de utilidad completa $U(x_1,\ldots,x_n)$, necesitamos d^n valores en el peor de los casos. La teoría de la utilidad multiatributo tiene como objetivo identificar una estructura adicional en las preferencias humanas para que no necesitemos especificar todos los valores d^n de manera individual. Habiendo identificado alguna regularidad en la preferencia de comportamiento, luego derivamos **teoremas de representación** para mostrar que un agente con un cierto tipo de representación de estructura de preferencia tiene una función de utilidad

Teoremas de representación

$$U(x_1,\ldots,x_n) = F[f_1(x_1),\ldots,f_n(x_n)],$$

donde F es (esperamos) una función simple como la suma. Observe la similitud con el uso de las redes bayesianas para descomponer la probabilidad conjunta de varias variables aleatorias.

Como ejemplo, suponga que cada x_i es la cantidad de dinero que el agente tiene en una moneda en particular: dólares, euros, marcos, liras, etc. Las funciones f_i podrían entonces convertir cada cantidad en una moneda común, y F sería la suma.

Preferencias sin incertidumbre

Comencemos con el caso determinista. En la página 532 notamos que para los entornos deterministas, el agente tiene una función de valor, que aquí escribimos como $V(x_1,\ldots,x_n)$; el objetivo es representar esta función de manera concisa. La regularidad básica que surge en la preferencia determinista las estructuras se llaman **independencia de preferencia**. Dos atributos X_1 y X_2 son independientes de un tercer atributo X_3, si la preferencia entre los resultados $\langle x_1,x_2,x_3\rangle$ y $\langle x'_1,x'_2,x_3\rangle$ no depende del valor particular x_3 para el atributo X_3.

Independencia de preferencia

Volviendo al ejemplo del aeropuerto, donde tenemos (entre otros atributos a considerar) *Tranquilidad*, *Frugalidad* y *Seguridad*, es posible proponer que la *Tranquilidad* y la *Frugalidad* son independientes de la *Seguridad*. Por ejemplo, si preferimos un resultado con 20 000 personas residiendo en la ruta de vuelo y un costo de construcción de 4 000 millones de dólares, en lugar de un resultado con 70 000 personas residiendo en la ruta de vuelo y un costo de 3 700 millones de dólares, cuando en ambos casos el nivel de seguridad es de 0.006 muertes por billón de pasajeros-milla, entonces tendríamos la misma preferencia cuando el nivel de seguridad es 0.012 o 0.003. La misma independencia se mantendría para las preferencias entre cualquier otro par de valores para *Tranquilidad* y *Frugalidad*. También es evidente que la *Frugalidad* y la *Seguridad* son independientes de la *Tranquilidad* y que la *Tranquilidad* y la *Seguridad* son independientes de la *Frugalidad*.

Decimos que el conjunto de atributos {*Tranquilidad*, *Frugalidad*, *Seguridad*} exhibe **independencia preferencial mutua (MPI)**. La MPI indica que, aunque cada atributo puede ser importante, no afecta la forma en que uno intercambia los otros atributos entre sí.

Independencia preferencial mutua (MPI)

La independencia preferencial mutua es un nombre complicado, pero conduce a una forma simple para la función de valor del agente (Debreu, 1960): *Si los atributos $X_1,\ldots,X_n$ son mutuamente independientes, entonces las preferencias del agente permiten ser representadas por una función de valor*

$$V(x_1,\ldots,x_n) = \sum_i V_i(x_i),$$

donde cada V_i se refiere sólo al atributo X_i. Por ejemplo, bien podría ser el caso de que la decisión del aeropuerto se pueda tomar usando una función de valor

$$V(\mathit{tranquilidad},\mathit{frugalidad},\mathit{seguridad}) = \mathit{tranquilidad} \times 10^4 + \mathit{frugalidad} + \mathit{seguridad} \times 10^{12}.$$

Una función de valor de este tipo se llama **función de valor aditiva**. Las funciones aditivas son una forma natural de describir las preferencias de un agente y son válidas en muchas situaciones

Función de valor aditiva

del mundo real. Para n atributos, evaluar una función de valor aditiva requiere evaluar n funciones de valor unidimensionales separadas en lugar de una función n-dimensional; con frecuencia, esto representa una reducción exponencial en el número de experimentos de preferencia que se necesitan. Incluso cuando MPI no se cumple de manera estricta, como podría ser el caso en valores extremos de los atributos, una función de valor aditiva aún podría proporcionar una buena aproximación a las preferencias del agente. Esto es cierto cuando las violaciones de MPI ocurren en partes de los rangos de atributos que es poco probable que ocurran en la práctica.

Para entender mejor MPI, ayuda mirar los casos donde *no se* cumple. Suponga que está en un mercado medieval y considera comprar perros de caza, pollos y jaulas de mimbre para los pollos. Los perros de caza son muy valiosos, pero si no tiene suficientes jaulas para los pollos los perros se los comerán; por lo tanto, el intercambio entre perros y pollos depende del número de jaulas, lo que viola MPI. La existencia de este tipo de interacciones entre varios atributos hace que sea mucho más difícil evaluar la función de valor general.

Preferencias con incertidumbre

Cuando la incertidumbre está presente en el dominio, también necesitamos considerar la estructura de las preferencias entre loterías y entender las propiedades resultantes de las funciones de utilidad, en lugar de sólo las funciones de valor. Las matemáticas de este problema pueden volverse muy complicadas, así que presentamos sólo uno de los principales resultados para dar una idea de lo que es posible hacer.

Independencia de utilidad

La noción básica de **independencia de utilidad** extiende la independencia de preferencia para cubrir las loterías: un conjunto de atributos **X** es independiente en utilidad de un conjunto de atributos **Y**, si las preferencias entre las loterías en los atributos en **X** son independientes de los valores particulares de los atributos en **Y**. Un conjunto de atributos es **mutuamente independiente en utilidad** (MUI) si cada uno de sus subconjuntos es independiente en utilidad de los atributos restantes. De nuevo, parece razonable proponer que los atributos del aeropuerto son MUI.

Mutuamente independiente en utilidad

Función de utilidad multiplicativa

MUI implica que el comportamiento del agente permite ser descrito usando una **función de utilidad multiplicativa** (Keeney, 1974). La forma general de una función de utilidad multiplicativa se ve mejor al mirar el caso de tres atributos. Para concisión, usamos U_i para significar $U_i(x_i)$:

$$U = k_1U_1 + k_2U_2 + k_3U_3 + k_1k_2U_1U_2 + k_2k_3U_2U_3 + k_3k_1U_3U_1 + k_1k_2k_3U_1U_2U_3.$$

Aunque esto no parece muy simple, contiene sólo tres funciones de utilidad de un solo atributo y tres constantes. En general, un problema de n atributos que exhibe MUI permite ser modelado usando n utilidades de un solo atributo y n constantes. Cada una de las funciones de utilidad de un solo atributo puede ser desarrollada de manera independiente de los otros atributos, por lo que esta combinación estará garantizada para generar las preferencias generales correctas. Se requieren suposiciones adicionales para obtener una función de utilidad aditiva.

16.5 Redes de decisión

Diagrama de influencia

Red de decisión

En esta sección, observamos un mecanismo general para tomar decisiones racionales. La notación con frecuencia se llama **diagrama de influencia** (Howard y Matheson, 1984), pero usaremos el término más descriptivo de **red de decisión**. Las redes de decisión combinan redes bayesianas con tipos de nodos adicionales para acciones y utilidades. Usamos el problema de elegir un sitio para un aeropuerto como ejemplo.

16.5.1 Representando un problema de decisión con una red de decisión

En su forma más general, una red de decisión representa información sobre el estado actual del agente, sus posibles acciones, el estado que resultará de la acción del agente, y la utilidad de ese estado. Por lo tanto, proporciona un sustrato para implementar agentes basados en utilidad del tipo introducido por primera vez en la Sección 2.4.5. La Figura 16.6 muestra una red de decisión para el problema de ubicación del aeropuerto. Ilustra los tres tipos de nodos utilizados:

Nodos de azar

- **Nodos de azar** (óvalos) representan variables aleatorias, al igual que en las redes bayesianas. El agente podría dudar sobre el costo de construcción, el nivel de tráfico aéreo y el potencial de litigio, y las variables de *Seguridad*, *Tranquilidad*, y total *Frugalidad*, cada una de las cuales también depende del sitio elegido. Cada nodo de azar tiene asociado una distribución condicional que está indexada por el estado de los nodos padres. En las redes de decisión, los nodos padres permiten incluir nodos de decisión, así como nodos de azar. Note que cada uno de los nodos de azar del estado actual podría ser parte de una gran red bayesiana para evaluar los costos de construcción, los niveles de tráfico aéreo o los potenciales de litigio.

Nodos de decisión

- **Nodos de decisión** (rectángulos) representan puntos donde el tomador de decisiones tiene una elección de nodos de decisión de acciones. En este caso, la acción *AirportSite* permite tomar un valor diferente para cada sitio en consideración. La elección influye en la seguridad, la tranquilidad y la frugalidad de la solución. En este capítulo, asumimos que tratamos con un solo nodo de decisión. El Capítulo 17 trata casos en los que se debe tomar más de una decisión.

Nodos de utilidad

- **Nodos de utilidad** (diamantes) representan la función de utilidad del agente.[7] Los nodos de utilidad tienen como padres todas las variables que describen los resultados que afectan de manera directa la utilidad. Asociado con el nodo de utilidad hay una descripción de la utilidad del agente como una función de los atributos de los padres. La descripción podría ser sólo o una tabulación de la función o una función aditiva o lineal parametrizada de los valores de los atributos. Por ahora, asumiremos que la función es determinista; es decir, dados los valores de sus variables padre, el valor del nodo de utilidad está completamente determinado.

También se utiliza una forma simplificada en muchos casos. La notación sigue siendo idéntica, pero se omiten los nodos de probabilidad que describen los estados de resultado. En cambio, el nodo de utilidad está conectado de manera directa a los nodos de estado actual y al nodo de decisión. En este caso, en lugar de representar una función de utilidad en los estados de resultado, el nodo de utilidad representa la utilidad *esperada* asociada con cada acción, según se define en la Ecuación (16.1) en la página 529; es decir, el nodo está asociado con una **función de utilidad de acción** (también conocida como **función Q** en el aprendizaje por refuerzo, como se describe en el Capítulo 22). La Figura 16.7 muestra la representación de utilidad de acción del problema de ubicación del aeropuerto.

Función de utilidad de acción

Note que, debido a que los nodos de probabilidad de *Tranquilidad*, *Seguridad* y *Frugalidad* en la Figura 16.6 se refieren a estados futuros, no permiten tener valores establecidos como variables de evidencia. Por lo tanto, la versión simplificada que omite estos nodos puede usarse siempre que se pueda usar la forma más general. Aunque la forma simplificada contiene menos nodos, la omisión de una descripción explícita del resultado de la decisión de ubicación significa que es menos flexible con respecto a los cambios en las circunstancias.

[7] Estos nodos también se llaman **nodos de valor** en la literatura.

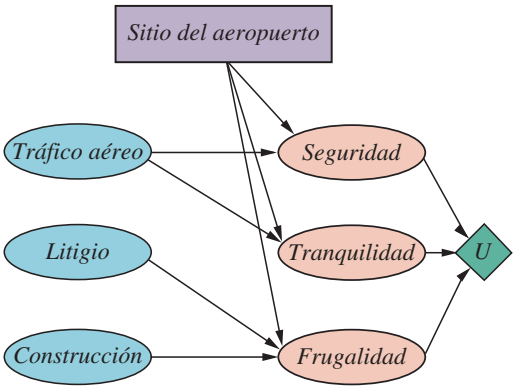


Figura 16.6 Red de decisión para el problema de ubicación del aeropuerto.

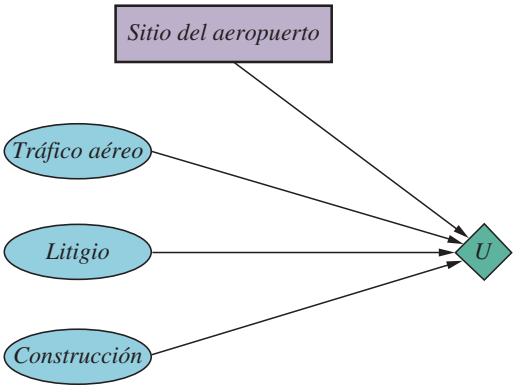


Figura 16.7 Representación simplificada del problema de ubicación del aeropuerto. Los nodos de probabilidad correspondientes a los estados de resultado han sido eliminados.

Por ejemplo, en la Figura 16.6, un cambio en los niveles de ruido de los aviones permite reflejarse en un cambio en la tabla de probabilidad condicional asociada con el nodo *Tranquilidad*, mientras que un cambio en el peso otorgado a la contaminación acústica en la función de utilidad puede reflejarse en un cambio en la tabla de utilidad. Por otro lado, en el diagrama de utilidad de acción, Figura 16.7, todos estos cambios tienen que reflejarse en cambios en la tabla de utilidad de acción. En esencia, la formulación de utilidad de acción es una versión *compilada* de la formulación original, obtenida sumando las variables de estado de resultado.

16.5.2 Evaluación de redes de decisión

Las acciones se seleccionan evaluando la red de decisión para cada posible configuración del nodo de decisión. Una vez que se establece el nodo de decisión, se comporta como un nodo de probabilidad que se ha establecido como una variable de evidencia. El algoritmo para evaluar redes de decisión es el siguiente:

1. Establecer las variables de evidencia para el estado actual.
2. Para cada posible valor del nodo de decisión:
   (a) Establecer el nodo de decisión a ese valor.
   (b) Calcular las probabilidades posteriores para los nodos padres del nodo de utilidad, utilizando un algoritmo de inferencia probabilística estándar.
   (c) Calcular la utilidad resultante para la acción.
3. Devolver la acción con la mayor utilidad.

Este es un enfoque sencillo que permite utilizar cualquier algoritmo de red bayesiana disponible y permite incorporarlo de manera directa en el diseño del agente dado en la Figura 12.1, página 388. Veremos en el Capítulo 17 que la posibilidad de ejecutar varias acciones en secuencia hace que el problema sea mucho más interesante.

16.6 El valor de la información

En el análisis anterior, hemos asumido que toda la información relevante, o al menos toda la información disponible, se proporciona al agente antes de que tome su decisión. En la práctica, esto casi nunca es el caso. *Una de las partes más importantes de la toma de decisiones es saber qué preguntas plantear.* Por ejemplo, un médico no puede esperar que se le proporcionen los resultados de todas las posibles pruebas diagnósticas y preguntas en el momento en que un paciente entra por primera vez en la sala de consulta. Las pruebas son costosas y, a veces, peligrosas (tanto de manera directa como por los retrasos asociados). Su importancia depende de dos factores: si los resultados de las pruebas conducirían con un plan de tratamiento significativamente mejor, y cuán probable son los diversos resultados de las pruebas.

Esta sección describe la **teoría del valor de la información**, que permite a un agente elegir qué información adquirir. Asumimos que antes de seleccionar una acción "real" representada por el nodo de decisión, el agente puede adquirir el valor de cualquiera de las variables de oportunidad observables en el modelo. Por lo tanto, la teoría del valor de la información implica una forma simplificada de toma de decisiones secuenciales, simplificada porque las acciones de observación sólo afectan el **estado de creencia** del agente, no el estado físico externo. El valor de cualquier observación particular debe derivarse del potencial para afectar la acción física eventual del agente; y este potencial permite estimarse de manera directa a partir del modelo de decisión en sí.

Teoría del valor de la información

16.6.1 Un ejemplo simple

Suponga que una compañía petrolera espera comprar uno de n bloques indistinguibles de derechos de perforación oceánica. Además, imagine que uno de los bloques contiene petróleo que generará beneficios netos de C dólares, mientras que los demás carecen de valor. El precio de venta de cada bloque es C/n dólares. Si la compañía es neutral al riesgo, entonces será indiferente entre comprar un bloque y no comprarlo porque el beneficio esperado es cero en ambos casos.

Ahora, suponga que un sismólogo ofrece a la compañía los resultados de una revisión del bloque número 3, que indica si el bloque contiene petróleo. ¿Cuánto debería estar dispuesta a pagar la compañía por la información? La forma de responder a esta pregunta es examinar lo que haría la compañía si tuviera la información:

- Con una probabilidad de $1/n$, la revisión indicará petróleo en el bloque 3. En este caso, la compañía comprará el bloque 3 por C/n dólares y obtendrá un beneficio de $C - C/n = (n - 1)C/n$ dólares.

- Con una probabilidad de $(n-1)/n$, la revisión mostrará que el bloque no contiene petróleo, en cuyo caso la compañía comprará un bloque diferente. Ahora, la probabilidad de encontrar petróleo en uno de los otros bloques cambia de $1/n$ a $1/(n-1)$, por lo que la compañía obtiene un beneficio esperado de $C/(n-1) - C/n = C/n(n-1)$ dólares.

Ahora podemos calcular el beneficio esperado, dado el acceso a la información de la revisión:

$$\frac{1}{n} \times \frac{(n-1)C}{n} + \frac{n-1}{n} \times \frac{C}{n(n-1)} = C/n.$$

Por lo tanto, la información vale C/n dólares para la compañía, y la compañía debería estar dispuesta a pagar al sismólogo una fracción significativa de esta cantidad.

El valor de la información se deriva del hecho de que, *con* la información, el curso de acción permite cambiar para adaptarse a la situación *real*. Es posible discriminar de acuerdo con la situación, mientras que, sin la información, uno tiene que hacer lo que es mejor en promedio sobre las posibles situaciones. En general, el valor de una determinada pieza de información se define como la diferencia en el valor esperado entre las mejores acciones antes y después de obtener la información.

16.6.2 Una fórmula general para la información perfecta

Valor de la información perfecta

Es simple derivar una fórmula matemática general para el valor de la información. Asumimos que es posible obtener evidencia exacta sobre el valor de alguna variable aleatoria E_j (es decir, aprendemos $E_j = e_j$), por lo que se utiliza la frase **valor de la información perfecta** (VPI).[8]

En el estado de información inicial del agente, el valor de la acción actual mejor α es, desde la Ecuación (16.1),

$$EU(\alpha) = \max_a \sum_{s'} P(\text{RESULT}(a) = s')U(s'),$$

y el valor de la nueva mejor acción (después de que se obtiene la nueva evidencia $E_j = e_j$) será

$$EU(\alpha_{ej}|e_j) = \max_a \sum_{s'} P(\text{RESULT}(a) = s'|e_j)U(s').$$

Pero E_j es una variable aleatoria cuyo valor actual es desconocido, por lo que para determinar el valor de descubrir E_j debemos promediar sobre todos los posibles valores e_j que podríamos descubrir para E_j, utilizando nuestras creencias actuales sobre su valor:

$$VPI(E_j) = \left(\sum_{ej} P(E_j = e_j)EU(\alpha_{e_j}|E_j = e_j)\right) - EU(\alpha).$$

Para obtener alguna intuición para esta fórmula, considere el caso simple donde sólo hay dos acciones de las cuales elegir, a_1 y a_2. Sus utilidades esperadas actuales son U_1 y U_2. La información $E_j = e_j$ producirá algunas nuevas utilidades esperadas U'_1 y U'_2 para las acciones, pero antes de obtener E_j, tendremos algunas distribuciones de probabilidad sobre los posibles valores de U'_1 y U'_2 (que asumimos son independientes).

Suponga que a_1 y a_2 representan dos rutas diferentes a través de una cordillera en invierno: a_1 es una bonita autopista recta a través de un túnel, y a_2 es un camino de tierra serpenteante por la

[8] No hay pérdida de expresividad al requerir información perfecta. Suponga que queríamos modelar el caso en el que nos volvemos algo más seguros sobre una variable. Podemos hacer eso introduciendo *otra* variable sobre la cual aprendemos información perfecta. Por ejemplo, suponga que en inicio tenemos una amplia incertidumbre sobre la variable *Temperatura*. Luego obtenemos el conocimiento perfecto *Termómetro* = 37; esto nos da información sobre la verdadera *Temperatura*, y la incertidumbre debido al error de medición está codificada en el modelo del sensor **P**(*Termómetro*|*Temperatura*). Ver Ejercicio 16.VPIX para otro ejemplo.

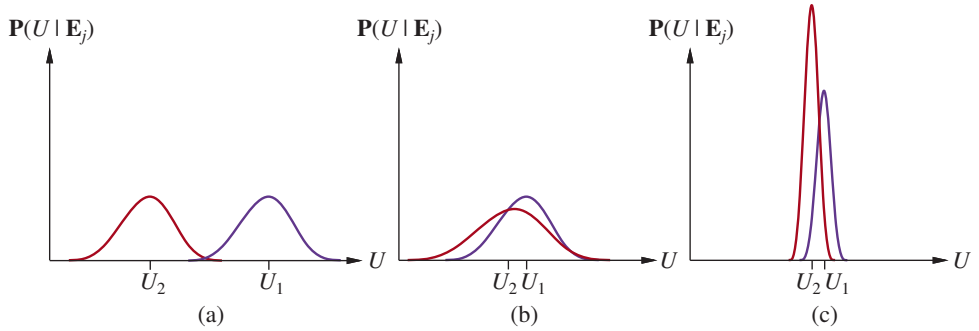


Figura 16.8 Tres casos genéricos para el valor de la información. En (a), a_1 seguirá siendo superior a a_2, por lo que la información no es necesaria. En (b), la elección no es clara y la información es crucial. En (c), la elección no es clara, pero como hace poca diferencia, la información es menos valiosa. (Nota: El hecho de que U_2 tenga un pico alto en (c) significa que su valor esperado se conoce con mayor certeza que U_1).

cima. Sólo con esta información, a_1 es preferible, porque es muy posible que a_2 esté bloqueado por la nieve, mientras que es poco probable que algo bloquee a_1. U_1 es, por lo tanto, superior a U_2. Es posible obtener informes de satélite E_j sobre el estado actual de cada carretera que darían nuevas expectativas, U'_1 y U'_2, para los dos cruces. Las distribuciones para estas expectativas se muestran en la Figura 16.8(a). Note que, en este caso, no vale la pena el gasto de obtener informes de satélite, porque es poco probable que la información derivada de ellos cambie el plan. Sin ningún cambio, la información no tiene valor.

Ahora suponga que estamos eligiendo entre dos caminos de tierra serpenteantes de longitudes diferentes y llevamos un pasajero herido. Entonces, aun cuando U_1 y U_2 están muy cerca, las distribuciones de U'_1 y U'_2 son muy amplias. Hay una significativa posibilidad de que la segunda ruta resulte estar despejada mientras la primera esté bloqueada y, en este caso, la diferencia en utilidades será muy alta. La fórmula VPI indica que podría valer la pena obtener los informes de satélite. Tal situación se muestra en la Figura 16.8(b).

Finalmente, suponga que estamos eligiendo entre los dos caminos de tierra en verano, cuando es poco probable que estén bloqueados por la nieve. En este caso, los informes de satélite podrían mostrar una ruta más pintoresca que la otra debido a prados alpinos en flor, o quizá más húmeda debido a lluvia reciente. Por lo tanto, es muy probable que cambiáramos nuestro plan si tuviéramos la información. Sin embargo, en este caso, la diferencia en valor entre las dos rutas todavía es probable que sea muy pequeña, por lo que no nos molestaremos en obtener los informes. Esta situación se muestra en la Figura 16.8(c).

En resumen, *la información tiene valor en la medida en que es probable que provoque un cambio de plan y en la medida en que el nuevo plan será mejor que el plan anterior.*

16.6.3 Propiedades del valor de la información

Uno podría preguntarse si es posible que la información sea perjudicial: ¿permite tener un valor esperado negativo? De manera intuitiva, uno debería esperar que esto sea imposible. Después de todo, en el peor de los casos, uno podría ignorar la información y fingir que nunca la ha recibido. Esto se confirma con el siguiente teorema, que se aplica a cualquier agente de teoría de decisiones que utilice cualquier red de decisiones con posibles observaciones E_j:

- *El valor esperado de la información es no negativo:*

$$\forall j \; VPI(E_j) \geq 0.$$

El teorema se deduce de manera directa de la definición de VPI, y dejamos la prueba como un ejercicio (Ejercicio 16.NNVP). Es, por supuesto, un teorema sobre el valor *esperado*, no el valor *real*. La información adicional permite llevar a un plan que *resulta ser* peor que el plan original si la información resulta engañosa. Por ejemplo, una prueba médica que da como resultado un falso positivo puede llevar a una cirugía innecesaria; pero eso no significa que la prueba no deba hacerse.

Es importante recordar que el VPI depende del estado actual de la información. Permite cambiar a medida que se adquiere más información. Para cualquier evidencia E_j dada, el valor de adquirirla puede disminuir (por ejemplo, si otra variable restringe el estado posterior para E_j) o aumentar (es decir, si otra variable proporciona una pista sobre la cual E_j se basa, permitiendo la elaboración de un nuevo y mejor plan). Así, el VPI no es aditivo. Es decir,

$$VPI(E_j,E_k) \neq VPI(E_j) + VPI(E_k) \qquad \text{(en general)}.$$

VPI es, sin embargo, independiente del orden. Es decir,

$$VPI(E_j,E_k) = VPI(E_j) + VPI(E_k|E_j) = VPI(E_k) + VPI(E_j|E_k) = VPI(E_k,E_j)$$

donde la notación $VPI(\cdot|E)$ denota el VPI calculado de acuerdo con la distribución posterior donde E ya se ha observado. La independencia del orden distingue las acciones de detección de las acciones ordinarias y simplifica el problema de calcular el valor de una secuencia de acciones de detección. Volvemos a esta pregunta en la siguiente sección.

16.6.4 Implementación de un agente de recopilación de información

Un agente sensato debería plantear preguntas con un orden razonable, debería evitar hacer preguntas irrelevantes, debería tener en cuenta la importancia de cada pieza de información con relación a su costo, y debería dejar de hacer preguntas cuando sea apropiado. Todas estas capacidades se pueden lograr utilizando el valor de la información como guía.

La Figura 16.9 muestra el diseño general de un agente que permite recopilar información de manera inteligente antes de actuar. Por ahora, asumimos que con cada variable de evidencia observable E_j, hay un costo asociado, $C(E_j)$, que refleja el costo de obtener la evidencia mediante pruebas, consultores, preguntas, o lo que sea. El agente solicita lo que parece ser la más eficiente observación en términos de ganancia de utilidad por unidad de costo. Asumimos que el resultado de la acción *Request*(E_j) es que el próximo percepto proporciona el valor de E_j. Si ninguna observación vale su costo, el agente selecciona una acción "real".

Miope

El algoritmo del agente que hemos descrito implementa una forma de recopilación de información que se llama **miope**. Esto se debe a que utiliza la fórmula VPI de manera corta de vista, calculando el valor de la información como si sólo se fuera a adquirir una única variable de evidencia. El control miope se basa en la misma idea heurística que la búsqueda codiciosa y con frecuencias funciona bien en la práctica. (Por ejemplo, se ha demostrado que supera a los médicos expertos en la selección de pruebas diagnósticas). Sin embargo, si no hay una única variable de evidencia que ayude mucho, un agente miope podría tomar una acción precipitada cuando hubiera sido mejor solicitar dos o más variables primero y luego tomar una acción. La siguiente sección considera la posibilidad de obtener múltiples observaciones.

```
function INFORMATION-GATHERING-AGENT(percept) returns an action
  persistent: D, una red de decisiones

  integra el percept en D
  j ← el valor que maximiza VPI(E_j)/C(E_j)
  if VPI(E_j) > C(E_j)
     then return Request(E_j)
  else return la mejor acción de D
```

Figura 16.9 Diseño de un agente simple y miope de recopilación de información. El agente funciona seleccionando de manera repetida la observación con el mayor valor de información, hasta que el costo de la próxima observación es mayor que su beneficio esperado.

16.6.5 Recolección de información no miope

El hecho de que el valor de una secuencia de observaciones sea invariante bajo permutaciones de la secuencia es intrigante pero no, por sí mismo, conduce a algoritmos eficientes para la recolección óptima de información. Incluso si nos limitamos a elegir de antemano un subconjunto fijo de observaciones para recopilar, hay 2^n posibles subconjuntos de n observaciones potenciales. En el caso general, nos enfrentamos a un problema aún más complejo de encontrar un plan condicional óptimo (como se describe en la Sección 11.5.2) que elige una observación y luego actúa o elige más observaciones, dependiendo del resultado. Dichos planes forman árboles, y el número de tales árboles es superexponencial en n.[9]

Para las observaciones de variables en una red de decisiones, resulta que este problema es intratable aun cuando la red es un poliárbol. Sin embargo, hay casos especiales en los que el problema se puede resolver de manera eficiente. Aquí presentamos un caso de este tipo: el problema de la **búsqueda del tesoro** (o el problema de la **secuencia de pruebas de menor costo**, para los menos inclinados). Hay n ubicaciones 1,...,n; cada ubicación i contiene un tesoro con probabilidad independiente $P(i)$; y cuesta $C(i)$ verificar la ubicación i. Esto corresponde a una red de decisiones donde todas las variables de evidencia potencial $Treasure_i$ son independientes. El agente examina las ubicaciones en algún orden hasta que se encuentra el tesoro; la pregunta es, ¿cuál es el orden óptimo?

Búsqueda del tesoro

Para responder a esta pregunta, tendremos que considerar los costos esperados y las probabilidades de éxito de varias secuencias de observaciones, suponiendo que el agente se detiene cuando se encuentra el tesoro. Encontremos $\mathbf{x}$ tal secuencia; $\mathbf{xy}$ sea la concatenación de las secuencias $\mathbf{x}$ e $\mathbf{y}$; $C(\mathbf{x})$ sea el costo esperado de $\mathbf{x}$; $P(\mathbf{x})$ sea la probabilidad de que la secuencia $\mathbf{x}$ tenga éxito en encontrar el tesoro; y $F(\mathbf{x}) = 1 - P(\mathbf{x})$ sea la probabilidad de que falle. Dadas estas definiciones, tenemos

$$C(\mathbf{xy}) = C(\mathbf{x}) + F(\mathbf{x})C(\mathbf{y}), \tag{16.3}$$

es decir, la secuencia $\mathbf{xy}$ incurrirá en el costo de $\mathbf{x}$ y, si $\mathbf{x}$ falla, también incurrirá en el costo de $\mathbf{y}$.

La idea básica en cualquier problema de optimización de secuencias es mirar el cambio en el costo, definido por $\Delta = C(\mathbf{wxyz}) - C(\mathbf{wyxz})$, cuando dos subsecuencias adyacentes $\mathbf{x}$ e $\mathbf{y}$ en una secuencia general $\mathbf{wxyz}$ se invierten. Cuando la secuencia es óptima, todos estos cambios

[9] El problema habitual de generar comportamiento secuencial en un entorno parcialmente observable cae bajo el título de **procesos de decisión de Markov parcialmente observables**, descrito en el Capítulo 17.

empeoran la secuencia. El primer paso es mostrar que el signo del efecto (aumento o disminución el costo) no depende del contexto proporcionado por **w** y **z**. Tenemos

$$\begin{aligned}\Delta &= [C(\mathbf{w}) + F(\mathbf{w})C(\mathbf{xyz})] - [C(\mathbf{w}) + F(\mathbf{w})C(\mathbf{yxz})] \qquad \text{(por la Ecuación (16.3))}\\ &= F(\mathbf{w})[C(\mathbf{xyz}) - C(\mathbf{yxz})]\\ &= F(\mathbf{w})[(C(\mathbf{xy}) + F(\mathbf{xy})C(\mathbf{z})) - (C(\mathbf{yx}) + F(\mathbf{yx})C(\mathbf{z}))] \qquad \text{(por la Ecuación (16.3))}\\ &= F(\mathbf{w})[C(\mathbf{xy}) - C(\mathbf{yx})] \qquad \text{(ya que } F(\mathbf{xy}) = F(\mathbf{yx})\text{).}\end{aligned}$$

Por lo tanto, hemos demostrado que la dirección del cambio en el costo de toda la secuencia depende sólo de la dirección del cambio en el costo del par de elementos que se invierten; el contexto del par no importa. Esto nos da una forma de ordenar la secuencia por comparaciones por pares para obtener una solución óptima. De manera específica, ahora tenemos

$$\begin{aligned}\Delta &= F(\mathbf{w})[(C(\mathbf{x}) + F(\mathbf{x})C(\mathbf{y})) - (C(\mathbf{y}) + F(\mathbf{y})C(\mathbf{x}))] \qquad \text{(por la Ecuación (16.3))}\\ &= F(\mathbf{w})[C(\mathbf{x})(1 - F(\mathbf{y})) - C(\mathbf{y})(1 - F(\mathbf{x}))] = F(\mathbf{w})[C(\mathbf{x})P(\mathbf{y}) - C(\mathbf{y})P(\mathbf{x})].\end{aligned}$$

Esto se mantiene para cualquier secuencia **x** e **y**, por lo que se mantiene cuando **x** e **y** son observaciones individuales de las ubicaciones i y j, respectivamente. Entonces sabemos que, para que i y j sean adyacentes en una secuencia óptima, debemos tener $C(i)P(j) \leq C(j)P(i)$, o $\frac{P(i)}{C(i)} \geq \frac{P(j)}{C(j)}$. En otras palabras, el orden óptimo clasifica las ubicaciones según la probabilidad de éxito por unidad de costo. El ejercicio 16.HUNT le pide que determine si esta es en realidad la política seguida por el algoritmo en la Figura 16.9 para este problema.

16.6.6 Análisis de sensibilidad y decisiones robustas

Análisis de sensibilidad

La práctica del **análisis de sensibilidad** es generalizada en las disciplinas tecnológicas: significa analizar cuánto cambia la salida de un proceso a medida que se ajustan los parámetros del modelo. El análisis de sensibilidad en sistemas probabilísticos y de teoría de la decisión es muy importante porque las probabilidades utilizadas con normalidad se aprenden de los datos o se estiman por expertos humanos, lo que significa que están sujetas a una considerable incertidumbre. Sólo en casos raros, como los lanzamientos de dados en backgammon, se conocen de manera objetiva las probabilidades.

Para un proceso de toma de decisiones impulsado por la utilidad es posible pensar en la salida como la decisión real tomada o la utilidad esperada de esa decisión. Considere primero este último: dado que la expectativa depende de las probabilidades del modelo, podemos calcular la derivada de la utilidad esperada de cualquier acción dada con respecto a cada uno de esos valores de probabilidad. (Por ejemplo, si todas las distribuciones de probabilidad condicional en el modelo están tabuladas de manera explícita, entonces calcular la expectativa implica calcular una proporción de dos expresiones de suma de productos; para más información sobre este tema vea el Capítulo 20). Así, es posible determinar qué parámetros en el modelo tienen el mayor efecto en la utilidad esperada de la decisión final.

Si, en cambio, nos preocupa la decisión real tomada, en lugar de su utilidad según el modelo, entonces podemos sólo variar los parámetros de manera sistemática (quizás usando búsqueda binaria) para ver si la decisión cambia, y si es así, cuál es la perturbación más pequeña que causa tal cambio. Uno podría pensar que no importa tanto qué decisión se toma, sino cuál es su utilidad. Eso es cierto, pero en la práctica puede haber una diferencia sustancial entre la utilidad *real* de una decisión y la utilidad *según el modelo*.

Si todas las perturbaciones razonables de los parámetros dejan la decisión óptima sin cambios, entonces es razonable suponer que la decisión es buena, aun si la estimación de utilidad para esa decisión es incorrecta. Si, por otro lado, la decisión óptima cambia a medida que cambian los

parámetros del modelo, entonces hay una buena posibilidad de que el modelo produzca una decisión que es subóptima. En ese caso, vale la pena invertir más esfuerzo para refinar el modelo.

Estas intuiciones se han formalizado en varios campos (teoría de control, análisis de decisiones, gestión de riesgos) que proponen la noción de una decisión **robusta** o **minimax**, es decir, una que da el mejor resultado en el peor caso. Aquí, "peor caso" significa peor con respecto a todas las variaciones plausibles en los valores de los parámetros del modelo. Dejando que θ represente todos los parámetros en el modelo, la decisión robusta se define como:

Robusta

$$a^* = \underset{a}{\text{argmax}} \; \underset{\theta}{minEU}(a;\theta).$$

En múltiples casos, en particular en la teoría de control, el enfoque robusto conduce a diseños que funcionan de manera confiable en la práctica. En otros casos, conduce a decisiones conservadoras. Por ejemplo, al diseñar un coche autónomo, el enfoque robusto asumiría el peor caso para el comportamiento de los otros vehículos en la carretera; es decir, que todos son conducidos por maníacos homicidas. En ese caso, la solución óptima para el coche es quedarse en casa.

La teoría de la decisión bayesiana ofrece una alternativa a los métodos robustos: si hay incertidumbre sobre los parámetros del modelo, entonces modela esa incertidumbre usando hiperparámetros.

Mientras que el enfoque robusto podría decir que alguna probabilidad θ_i en el modelo podría estar en cualquier lugar entre 0.3 y 0.7, con el valor real elegido por un adversario para hacer que las cosas salgan de lo peor posible, el enfoque bayesiano pondría una distribución de probabilidad previa en θ_i y luego procedería como antes. Esto requiere más esfuerzo de modelado; por ejemplo, el modelador bayesiano debe decidir si los parámetros θ_i y θ_j son independientes, pero resulta en un mejor rendimiento en la práctica.

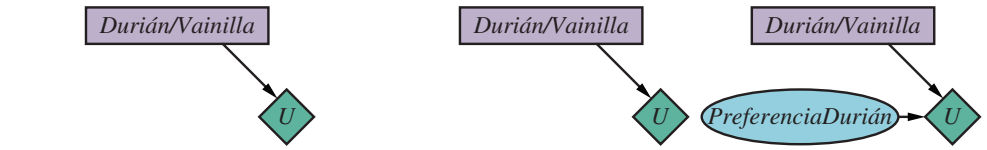
Además de la incertidumbre paramétrica, las aplicaciones de la teoría de la decisión en el mundo real también sufren de *incertidumbre estructural*. Por ejemplo, la suposición de independencia de *Tráfico aéreo*, *Litigio* y *Construcción* en la Figura 16.6 puede ser incorrecta, y es posible que existan variables adicionales que el modelo omite. En la actualidad, no tenemos una buena comprensión de cómo tener en cuenta este tipo de incertidumbre. Una posibilidad es mantener un conjunto de modelos, quizá generados por algoritmos de aprendizaje automático, con la esperanza de que el conjunto capture las variaciones significativas que importan.

16.7 Preferencias desconocidas

En esta sección discutimos qué sucede cuando hay incertidumbre sobre la función de utilidad cuyo valor esperado se va a optimizar. Hay dos versiones de este problema: una en la que un agente (máquina o humano) no está seguro de su *propia* función de utilidad, y otra en la que una máquina se supone que debe ayudar a un humano pero no está segura de lo que el humano quiere.

16.7.1 Incertidumbre sobre las propias preferencias

Imagine que está en una heladería en Tailandia y sólo quedan dos sabores: vainilla y durián. Ambos cuestan \$2. Usted sabe que le gusta la vainilla y estaría dispuesto a pagar hasta \$3 por un helado de vainilla en un día tan caluroso, por lo que hay una ganancia neta de \$1 por elegir vainilla. Por otro lado, no tiene idea de si le gusta el durián o no, pero ha leído en Wikipedia que el durián provoca diferentes respuestas en diferentes personas: algunos encuentran que "supera en sabor a todas las demás frutas del mundo", mientras que otros lo comparan con "aguas residuales, vómito rancio, emanación fétida de zorrillo y torundas quirúrgicas usadas".



D/V	⋯	–$82	⋯	+$0	+$1	⋯	+$98	⋯
durián	⋯	0.5	⋯	0.0	0.0	⋯	0.5	⋯
vainilla	⋯	0.0	⋯	0.0	1.0	⋯	0.0	⋯

(a)

D/V	*U*
durián	+$8
vainilla	+$1

(b)

D/V	*PreferenciaDurián*	*U*
durián	*Verdad*	+$98
durián	*Falso*	–$82
vainilla	*Verdad*	+$1
vainilla	*Falso*	+$1

(c)

Figura 16.10 (a) Una red de decisión para la elección del helado con una función de utilidad incierta. (b) La red con la utilidad esperada de cada acción. (c) Moviendo la incertidumbre de la función de utilidad a una nueva variable aleatoria.

Para poner algunos números en esto, digamos que hay 50 % de posibilidades de que lo encuentre sublime (+$100) y 50 % de posibilidades de que lo odie (-$80 si el sabor persiste toda la tarde). Aquí no hay incertidumbre sobre qué premio va a ganar, es el mismo helado de durián de cualquier manera, pero hay incertidumbre sobre sus propias preferencias por ese premio.

Podríamos extender el formalismo de la red de decisión para permitir utilidades inciertas, como se muestra en la Figura 16.10(a). Sin embargo, si no hay más información que obtener sobre sus preferencias de durián; por ejemplo, si la tienda no le deja probarlo primero, entonces el problema de decisión es idéntico al que se muestra en la Figura 16.10(b). Es posible reemplazar el valor incierto del durián con su ganancia neta esperada de $(0.5 \times \$100) - (0.5 \times \$80) - \$2 = \$8$ y su decisión permanecerá sin cambios.

Si es posible que sus creencias sobre el durián cambien, quizá pruebe un poco, o descubre que todos sus parientes vivos aman el durián, entonces la transformación en la Figura 16.10(b) no es válida. Resulta, sin embargo, que todavía podemos encontrar un modelo equivalente en el que la función de utilidad es determinista. En lugar de decir que hay incertidumbre sobre la función de utilidad, movemos esa incertidumbre "al mundo", por así decirlo. Es decir, creamos una nueva variable aleatoria *PreferenciaDurián* con probabilidades previas de 0.5 para *verdadero* y *falso*, como se muestra en la Figura 16.10(c). Con esta variable extra, la función de utilidad se vuelve determinista, pero aún podemos manejar el cambio de creencias sobre sus preferencias de durián.

El hecho de que las preferencias desconocidas permiten ser modeladas por variables aleatorias ordinarias significa que podemos seguir usando la maquinaria y los teoremas desarrollados para las preferencias conocidas. Por otro lado, no significa que siempre podamos asumir que las preferencias son conocidas. La incertidumbre todavía está ahí y todavía afecta cómo deben comportarse los agentes.

16.7.2 Deferencia a los humanos

Ahora pasemos al segundo caso antes mencionado: una máquina que se supone que debe ayudar a un humano pero no está segura de lo que el humano quiere. El tratamiento completo de este caso debe ser diferido al Capítulo 18, donde discutimos decisiones que involucran a más de un agente. Aquí, hacemos una pregunta simple: ¿bajo qué circunstancias tal máquina se someterá al humano?

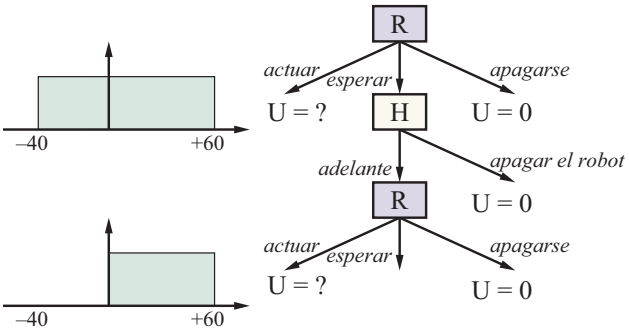


Figura 16.11 El juego del interruptor de apagado. *R*, el robot, elige actuar ahora, con un pago altamente incierto; apagarse o deferir a *H*, el humano. *H* puede apagar *R* o dejarlo seguir adelante. *R* ahora tiene la misma elección de nuevo. Actuar todavía tiene un pago incierto, pero ahora *R* sabe que el pago es no negativo.

Para estudiar esta pregunta, consideremos un escenario muy simple, como se muestra en la Figura 16.11. Robbie es un robot de software que trabaja como asistente personal para Harriet, una humana ocupada. Harriet necesita una habitación de hotel para su próxima reunión de negocios en Ginebra. Robbie puede actuar ahora, digamos que le reserva a Harriet en un hotel muy caro cerca del lugar de la reunión. No está muy seguro de cuánto le gustará a Harriet el hotel y su precio; digamos que tiene una probabilidad uniforme para su valor neto para Harriet entre −40 y +60, con un promedio de +10. También podría "apagarse a sí mismo" —menos melodramático, sacarse del proceso de reserva del hotel por completo— que definimos, sin pérdida de generalidad, para tener un valor de 0 para Harriet. Si esas fueran sus dos opciones, seguiría adelante y reservaría el hotel, incurriendo en un riesgo significativo de hacer infeliz a Harriet. (Si el rango fuera de -60 a +40, con un promedio de -10, se apagaría a sí mismo en su lugar). Sin embargo, le daremos a Robbie una tercera opción: explicar su plan, esperar, y dejar que Harriet lo apague. Harriet puede apagarlo o dejarlo seguir adelante y reservar ella el hotel. ¿Qué bien podría hacer esto, podríamos preguntar, dado que él podría tomar ambas decisiones por sí mismo?

El punto es que la elección de Harriet, apagar a Robbie o dejarlo seguir adelante, proporciona a Robbie información sobre las preferencias de Harriet. Supongamos ahora que Harriet es racional, por lo que si Harriet deja que Robbie siga adelante, lo que significa que para Harriet el valor es positivo. Ahora, como se muestra en la Figura 16.11, la creencia de Robbie cambia: es uniforme entre 0 y +60, con un promedio de +30.

Entonces, si evaluamos las opciones iniciales de Robbie desde su punto de vista:

1. Actuar ahora y reservar el hotel tiene un valor esperado de +10.
2. Apagarse a sí mismo tiene un valor de 0.
3. Esperar y dejar que Harriet lo apague lleva a dos posibles resultados:
   (a) Hay 40 % de posibilidades, con base en la incertidumbre de Robbie sobre las preferencias de Harriet, de que ella odie el plan y apague a Robbie, con valor 0.
   (b) Hay 60 % de posibilidades de que a Harriet le guste el plan y permita que Robbie siga adelante, con un valor esperado de +30.

Por lo tanto, esperar tiene un valor esperado de $(0.4 \times 0) + (0.6 \times 30) = +18$, que es mejor que el +10 que Robbie espera si actúa ahora.

La conclusión es que Robbie tiene un incentivo positivo para diferir con Harriet, es decir, para permitirse ser apagado. Este incentivo proviene de manera directa de la incertidumbre de Robbie sobre las preferencias de Harriet. Robbie es consciente de que hay una posibilidad (40 % en este ejemplo) de que esté a punto de hacer algo que hará infeliz a Harriet, en cuyo caso sería preferible ser apagado a seguir adelante. Si Robbie ya estuviera seguro sobre las preferencias de Harriet, sólo seguiría adelante y tomaría la decisión (o se apagaría a sí mismo); no habría en absoluto nada que ganar consultando a Harriet, porque, según las creencias definitivas de Robbie, ya permite predecir lo que ella va a decidir.

De hecho, es posible demostrar el mismo resultado en el caso general: siempre que Robbie no esté completamente seguro de que está a punto de hacer lo que Harriet misma haría, le conviene más permitirle que lo apague. De manera intuitiva, su decisión proporciona a Robbie información, y el valor esperado de la información siempre es no negativo. Por el contrario, si Robbie está seguro de la decisión de Harriet, su decisión no proporciona nueva información, por lo que Robbie no tiene incentivo para permitirle que decida.

De manera formal, se define $P(u)$ como la probabilidad previa de Robbie sobre la utilidad de Harriet para la acción propuesta a. Entonces el valor de seguir adelante con a es

$$EU(a) = \int_{-\infty}^{\infty} P(u) \cdot u du = \int_{-\infty}^{0} P(u) \cdot u du + \int_{0}^{\infty} P(u) \cdot u du.$$

(Veremos en breve por qué se divide el integral de esta manera). Por otro lado, el valor de la acción d, diferir con Harriet, se compone de dos partes: si $u > 0$ entonces Harriet deja que Robbie siga adelante, por lo que el valor es u, pero si $u < 0$ entonces Harriet apaga a Robbie, por lo que el valor es 0:

$$EU(d) = \int_{-\infty}^{0} P(u) \cdot 0 du + \int_{0}^{\infty} P(u) \cdot u du.$$

Comparando las expresiones para $EU(a)$ y $EU(d)$, vemos que

$$EU(d) \geq EU(a)$$

porque la expresión para $EU(d)$ tiene la región de utilidad negativa anulada. Las dos opciones tienen igual valor sólo cuando la región negativa tiene probabilidad cero; es decir, cuando Robbie ya está seguro de que a Harriet le gusta la acción propuesta.

Hay algunas consideraciones obvias en el modelo que vale la pena explorar de inmediato. La primera elaboración es imponer un costo por el tiempo de Harriet. En ese caso, Robbie está menos inclinado a molestar a Harriet si el riesgo a la baja es pequeño. Así es como debería ser. Y si Harriet está enfadada por ser interrumpida, no debería sorprenderse demasiado si Robbie de manera ocasional hace cosas que no le gustan.

La segunda consideración es permitir alguna probabilidad de error humano; es decir, Harriet a veces podría apagar a Robbie incluso cuando su acción propuesta es razonable, y a veces podría dejar que Robbie siga adelante aun cuando su acción propuesta es indeseable. Es sencillo incorporar esta probabilidad de error en el modelo (ver Ejercicio 16.OFFS). Como uno podría esperar, la solución muestra que Robbie está menos inclinado a diferir con una Harriet irracional que a veces actúa en contra de sus propios intereses. Cuanto se comporta de manera más aleatoria, más incierto tiene que estar Robbie acerca de sus preferencias antes de diferir con ella. De nuevo, así es como debería ser: por ejemplo, si Robbie es un automóvil autónomo y Harriet es su pasajera de dos años traviesa, Robbie no debería permitir que Harriet lo apague en medio de la autopista.

Resumen

Este capítulo muestra cómo combinar la teoría de la utilidad con la probabilidad para permitir que un agente seleccione acciones que maximizarán su rendimiento esperado.

- La **teoría de la probabilidad** describe lo que un agente debería creer con base en la evidencia, la **teoría de la utilidad** describe lo que un agente quiere, y la **teoría de la decisión** las une a ambas dos para describir lo que un agente debería hacer.
- Podemos usar la teoría de la decisión para construir un sistema que tome decisiones considerando todas las acciones posibles y elija la decisión que conduce al mejor resultado esperado. Tal sistema se conoce como un **agente racional**.
- La teoría de la utilidad muestra que un agente, cuyas preferencias entre loterías son consistentes con un conjunto de axiomas simples, permite describirse como poseedor de una función de utilidad; además, el agente selecciona acciones como si maximizara su utilidad esperada.
- La **teoría de la utilidad multiatributo** se ocupa de las utilidades que dependen de varios atributos distintos de los estados. La **dominancia estocástica** es una técnica útil para tomar decisiones inequívocas, incluso sin valores de utilidad precisos para los atributos.
- Las **redes de decisión** proporcionan un formalismo simple para expresar y resolver problemas de decisión. Son una extensión natural de las redes bayesianas, que contienen nodos de decisión y utilidad además de los nodos de azar.
- A veces, resolver un problema implica encontrar más información antes de tomar una decisión. El **valor de la información** se define como la mejora esperada en la utilidad en comparación con tomar una decisión sin la información; es en particular útil para guiar el proceso de recopilación de información antes de tomar una decisión final.
- Cuando, como suele ser el caso, es imposible especificar la función de utilidad del humano, las máquinas deben operar bajo la incertidumbre sobre el objetivo verdadero. Esto hace una diferencia significativa cuando existe la posibilidad de que la máquina adquiera más información sobre las preferencias humanas. Mostramos con un argumento simple que la incertidumbre sobre las preferencias asegura que la máquina se diferirá al humano, hasta el punto de permitirse ser apagada.

Notas bibliográficas e históricas

En el tratado del siglo XVII *L'art de Penser*, o *Lógica de Port-Royal*, Arnauld (1662) afirma:

> Para juzgar lo que uno debe hacer para obtener un bien o evitar un mal, es necesario considerar no sólo el bien y el mal en sí, sino también la probabilidad de que ocurra o no ocurra; y ver geométricamente la proporción que todas estas cosas tienen en conjunto.

Los textos modernos hablan de utilidad en lugar de bien y mal, pero esta declaración señala de manera correcta que uno debería multiplicar la utilidad por la probabilidad ("ver geométricamente") para dar la utilidad esperada, y maximizar eso sobre todos los resultados ("todas estas cosas") para "juzgar lo que uno debe hacer". Es notable cuánto acertó Arnauld, hace más de 350 años, y sólo 8 años después de que Pascal y Fermat mostraron por primera vez cómo usar la probabilidad de manera correcta.

Daniel Bernoulli (1738), investigando la paradoja de San Petersburgo (ver Ejercicio 16.STPT), fue el primero en darse cuenta de la importancia de la medición de preferencias para las loterías,

escribiendo "el *valor* de un artículo no debe basarse en su *precio*, sino más bien en la *utilidad* que produce" (cursiva suya). El filósofo utilitarista Jeremy Bentham (1823) propuso el **cálculo hedónico** para pesar "placeres" y "dolores", argumentando que todas las decisiones (no sólo las monetarias) podrían reducirse a comparaciones de utilidad.

Cálculo hedónico

La introducción de Bernoulli de la utilidad, una cantidad interna y subjetiva, para explicar el comportamiento humano a través de una teoría matemática fue una propuesta notable para su tiempo. Fue aún más notable por el hecho de que, a diferencia de las cantidades monetarias, los valores de utilidad de varias apuestas y premios no son directamente observables; en cambio, las utilidades deben inferirse de las preferencias exhibidas por un individuo. Pasarían dos siglos antes de que las implicaciones de la idea se desarrollaran y fuera aceptada por estadísticos y economistas.

La derivación de utilidades numéricas a partir de preferencias fue realizada por primera vez por Ramsey (1931); los axiomas para la preferencia en el texto actual están más cerca en forma a los redescubiertos en *Teoría de juegos y comportamiento económico* (Von Neumann y Morgenstern, 1944). Ramsey había derivado probabilidades subjetivas (no sólo utilidades) de las preferencias de un agente; Savage (1954) y Jeffrey (1983) realizan construcciones más recientes de este tipo. Beardon *et al*. (2002) muestran que una función de utilidad no es suficiente para representar preferencias no transitivas y otras situaciones anómalas.

En el periodo de posguerra, la teoría de la decisión se convirtió en una herramienta estándar en economía, finanzas y ciencia de la gestión. Surgió un campo de **análisis de decisiones** para ayudar a construir las decisiones de política más racionales en áreas como estrategia militar, diagnóstico médico, salud pública, diseño de ingeniería y gestión de recursos. El proceso implica un **tomador de decisiones** que establece preferencias entre resultados y un **analista de decisiones** que enumera las posibles acciones y resultados y elicitas las preferencias del tomador de decisiones para determinar el mejor curso de acción. Von Winterfeldt y Edwards (1986) proporcionan una perspectiva matizada sobre el análisis de decisiones y su relación con las estructuras de preferencia humana. Smith (1988) ofrece una visión general de la metodología del análisis de decisiones.

Análisis de decisiones

Tomador de decisiones

Analista de decisiones

Hasta la década de 1980, los problemas de decisión multivariante se manejaban construyendo "árboles de decisión" de todas las posibles instancias de las variables. Los diagramas de influencia o redes de decisión, que aprovechan las mismas propiedades de independencia condicional que las redes bayesianas, fueron introducidos por Howard y Matheson (1984), basados en trabajos anteriores en SRI (Miller *et al*., 1976). El algoritmo de Howard y Matheson construyó el árbol de decisión completo (exponencialmente grande) a partir de la red de decisión. Shachter (1986) desarrolló un método para tomar decisiones basadas de manera directa en una red de decisiones, sin la creación de un árbol de decisiones intermedio. Este algoritmo también fue uno de los primeros en proporcionar inferencia completa para redes bayesianas múltiplemente conectadas. Nilsson y Lauritzen (2000) vinculan algoritmos para redes de decisiones con desarrollos en curso en algoritmos de agrupación para redes bayesianas. La colección de Oliver y Smith (1990) tiene una serie de artículos tempranos útiles sobre redes de decisiones, al igual que el número especial de 1990 de la revista *Networks*. El texto de Fenton y Neil (2018) proporciona una guía práctica para resolver problemas de decisión del mundo real utilizando redes de decisiones. Los artículos sobre redes de decisiones y modelado de utilidad también aparecen de manera regular en las revistas *Management Science y Decision Analysis*.

Note que pocos investigadores de IA adoptaron herramientas de teoría de la decisión después de las primeras aplicaciones en la toma de decisiones médicas descritas en el Capítulo 12. Una de las pocas excepciones fue Jerry Feldman, quien aplicó la teoría de la decisión a problemas de visión (Feldman y Yakimovsky, 1974) y planificación (Feldman y Sproull, 1977). Los sistemas expertos basados en reglas de finales de la década de 1970 y principios de la de 1980

se concentraron en responder preguntas, en lugar de tomar decisiones. Aquellos sistemas que sí recomendaban acciones en general lo hacían usando reglas de condición-acción en lugar de representaciones explícitas de resultados y preferencias.

Las redes de decisión ofrecen un enfoque mucho más flexible, por ejemplo, al permitir que las preferencias cambien mientras se mantiene constante el modelo de transición, o viceversa. También permiten una estimación inicial de qué información buscar a continuación. A finales de la década de 1980, en parte debido al trabajo de Pearl en las redes de Bayes, los sistemas expertos de teoría de la decisión ganaron amplia aceptación (Horvitz *et al*., 1988; Cowell *et al*., 2002). De hecho, a partir de 1991, el diseño de la portada de la revista *Artificial Intelligence* ha representado una red de decisiones, aunque parece que se ha tomado cierta licencia artística con la dirección de las flechas.

Los intentos prácticos de medir las utilidades humanas comenzaron con el análisis de decisiones de posguerra (ver arriba). La medida de utilidad micromort es discutida por Howard (1989). Thaler Thaler (1992) encontró que para una oportunidad de 1/1 000 de muerte, un encuestado no pagaría más de $200 para eliminar el riesgo, pero no aceptaría $50 000 para asumir el riesgo.

El uso de **QALYs** (años de vida ajustados por calidad) para realizar análisis de costo-beneficio de intervenciones médicas y políticas sociales relacionadas se remonta al menos al trabajo de Klarman *et al*. (1968), aunque el término en sí fue utilizado por primera vez por Zeckhauser y Shepard (1976). Al igual que el dinero, los QALYs corresponden de manera directa a las utilidades sólo bajo suposiciones muy fuertes, como la neutralidad ante el riesgo, que con frecuencia se violan (Beresniak *et al*., 2015); sin embargo, los QALYs se utilizan mucho en la práctica, por ejemplo, en la formación de políticas del Servicio Nacional de Salud en el Reino Unido. Vea Russell (1990) para un ejemplo típico de un argumento para un cambio importante en la política de salud pública en términos de utilidad esperada medida en QALYs.

Teoría de utilidad multiatributo

Keeney y Raiffa (1976) nos introducen a la **teoría de utilidad multiatributo**. Describen las primeras implementaciones informáticas de métodos para obtener los parámetros necesarios para una función de utilidad multiatributo e incluyen relatos extensos de aplicaciones reales de la teoría. Abbas (2018) cubre muchos avances desde 1976. La teoría fue introducida a la IA por el trabajo de Wellman (1985), quien también investigó el uso de la dominancia estocástica y los modelos de probabilidad cualitativa (Wellman, 1988, 1990a). Wellman y Doyle (1992) proporcionan un esbozo preliminar de cómo un conjunto complejo de relaciones de independencia de utilidad podría usarse para proporcionar un modelo estructurado de una función de utilidad, de la misma manera que las redes bayesianas proporcionan un modelo estructurado de distribuciones de probabilidad conjunta. Bacchus y Grove (1995, 1996) y La Mura y Shoham (1999) dan más resultados en esta línea. Boutilier *et al*. (2004) describen las CP-nets, un formalismo de modelo gráfico trabajado para declaraciones de preferencia condicional *ceteribus paribus*.

La **maldición del optimizador** fue llevada a la atención de los analistas de decisiones de una manera contundente por Smith y Winkler (2006), quienes señalaron que los beneficios financieros para el cliente proyectados por los analistas para su curso de acción propuesto casi nunca se materializaban. Rastrean esto de manera directa al sesgo introducido al seleccionar una acción óptima y muestran que un análisis bayesiano más completo elimina el problema.

Decepción posterior a la decisión

Maldición del ganador

El mismo concepto subyacente ha sido llamado **decepción posterior a la decisión** por Harrison y March (1984) y se notó en el contexto de analizar proyectos de inversión de capital por Brown (1974). La maldición del optimizador también está relacionada con la **maldición del ganador** (Capen *et al*., 1971; Thaler, 1992), que se aplica a la oferta competitiva en subastas: quien gana la subasta es muy probable que haya sobreestimado el valor del objeto en cuestión. Capen *et al*. citan a un ingeniero petrolero sobre el tema de la oferta por derechos de perforación de petróleo: “Si uno gana los derechos de perforación de un pozo contra dos o tres competidores,

puede sentirse bien por su buena fortuna. ¿Pero cómo debería sentirse si ganó contra 50 competidores? Enfermo".

La paradoja de Allais, debida al economista Maurice Allais (1953), ganador del Premio Nobel, fue probada de manera experimental para mostrar que las personas son inconsistentes en sus juicios (Tversky y Kahneman, 1982; Conlisk, 1989). La paradoja de Ellsberg sobre la aversión a la ambigüedad fue introducida en la tesis doctoral de Daniel Ellsberg (1962).[10] Fox y Tversky (1995) describen un estudio adicional sobre la aversión a la ambigüedad. Machina (2005) da una visión general de la elección bajo incertidumbre y cómo puede variar de la teoría de la utilidad esperada. Vea el texto clásico de Keeney y Raiffa (1976) y el trabajo más reciente de Abbas (2018) para un análisis en profundidad de las preferencias con incertidumbre.

Irracionalidad

El 2009 fue un gran año para los libros populares sobre la **irracionalidad** humana, incluyendo *Predictably Irrational* (Ariely, 2009), *Sway* (Brafman y Brafman, 2009), *Nudge* (Thaler y Sunstein, 2009), *Kluge* (Marcus, 2009), *How We Decide* (Lehrer, 2009) y *On Being Certain* (Burton, 2009). Complementan el libro clásico *Judgment Under Uncertainty* (Kahneman *et al*., 1982) y el artículo que lo inició todo (Kahneman y Tversky, 1979). Kahneman proporciona una cuenta perspicaz y legible en *Thinking: Fast and Slow* (Kahneman, 2011).

Por otro lado, el campo de la psicología evolutiva (Buss, 2005) ha ido en contra de esta literatura, argumentando que los humanos son bastante racionales en contextos evolutivamente apropiados. Sus adherentes señalan que la irracionalidad es penalizada por definición en un contexto evolutivo y muestran que en algunos casos es un artefacto de la configuración experimental (Cummins y Allen, 1998). Ha habido un resurgimiento reciente de interés en los modelos bayesianos de cognición, revirtiendo décadas de pesimismo (Elio, 2002; Chater y Oaksford, 2008; Griffiths *et al*., 2008); sin embargo, este resurgimiento no está exento de detractores (Jones y Love, 2011).

La teoría del valor de la información se exploró primero en el contexto de los experimentos estadísticos, donde se utilizó una cuasi-utilidad (reducción de entropía) (Lindley, 1956). El teórico del control Ruslan Stratonovich (1965) desarrolló la teoría más general presentada aquí, en la que la información tiene valor en virtud de su capacidad para afectar las decisiones. El trabajo de Stratonovich no era conocido en Occidente, donde Ron Howard (1966) fue pionero en la misma idea. Su artículo concluye con la observación "Si la teoría del valor de la información y las estructuras teóricas de decisión asociadas no ocupan en el futuro una gran parte de la educación de los ingenieros, entonces la profesión de ingeniería encontrará que su papel tradicional de gestionar recursos científicos y económicos para el beneficio del hombre ha sido cedido a otra profesión". Hasta la fecha, la revolución implícita en los métodos de gestión no ha ocurrido.

El algoritmo de recopilación de información miope descrito en el capítulo es omnipresente en la literatura de análisis de decisiones; sus contornos básicos permiten discernirse en el artículo original sobre diagramas de influencia (Howard y Matheson, 1984). Los métodos de cálculo eficientes son estudiados por Dittmer y Jensen (1997). Laskey (1995) y Nielsen y Jensen (2003) discuten métodos para el análisis de sensibilidad en redes bayesianas y redes de decisiones, respectivamente. El texto clásico *Robust and Optimal Control* (Zhou *et al*., 1995) proporciona una cobertura completa y una comparación de los enfoques robustos y teóricos de la decisión para las decisiones bajo incertidumbre.

El problema de la búsqueda del tesoro fue resuelto de manera independiente por múltiples autores, remontándose al menos a los artículos sobre pruebas secuenciales de Gluss (1959) y Mitten (1960). El estilo de prueba en este capítulo se basa en un resultado básico, debido a Smith (1956), que relaciona el valor de una secuencia con el valor de la misma secuencia con dos elementos

[10] Ellsberg más tarde se convirtió en analista militar en la Corporación RAND y filtró documentos conocidos como los Papeles del Pentágono, contribuyendo así al fin de la guerra de Vietnam.

adyacentes permutados. Estos resultados para pruebas independientes se extendieron a problemas de búsqueda de árboles y grafos más generales (donde las pruebas están parcialmente ordenadas) por Kadane y Simon (1977). Los resultados sobre la complejidad de los cálculos no miope del valor de la información fueron obtenidos por Krause y Guestrin (2009). Krause *et al.* (2008) identificaron casos en los que la submodularidad conduce a un algoritmo de aproximación tratable, basándose en el trabajo seminal de Nemhauser *et al*. (1978) sobre funciones submodulares; Krause y Guestrin (2005) identifican casos en los que un algoritmo exacto de programación dinámica proporciona una solución eficiente tanto para la elección de subconjuntos de evidencia como para la generación de planes condicionales.

Harsanyi (1967) estudió el problema de la información incompleta en la teoría de juegos, donde los jugadores pueden no conocer de manera exacta las funciones de pago de los demás. Demostró que tales juegos eran idénticos a los juegos con información imperfecta, donde los jugadores tienen incertidumbre sobre el estado del mundo, mediante el truco de agregar variables de estado que se refieren a los pagos de los jugadores. Cyert y de Groot (1979) desarrollaron una teoría de **utilidad adaptativa** en la que un agente podría estar incierto sobre su propia función de utilidad y podría obtener más información a través de la experiencia.

Utilidad adaptativa

El trabajo sobre la elicitación de preferencias bayesianas (Chajewska *et al.*, 2000; Boutilier, 2002) comienza desde la suposición de una probabilidad previa sobre la función de utilidad del agente. Fern *et al*. (2014) proponen un modelo de **asistencia** basado en la teoría de la decisión en el que un robot intenta determinar y ayudar con un objetivo humano sobre el cual en inicio no está seguro. El ejemplo del interruptor de apagado en la Sección 16.7.2 está adaptado de Hadfield-Menell *et al.* (2017b). Russell (2019) propone un marco general para la IA benéfica en el que el juego del interruptor de apagado es un ejemplo clave.

Asistencia

CAPÍTULO 17

TOMANDO DECISIONES COMPLEJAS

En el que examinamos métodos para decidir qué hacer hoy, pues podemos enfrentar otra decisión mañana.

Problema de decisión secuencial

En este capítulo abordamos los problemas computacionales involucrados en la toma de decisiones en un entorno estocástico. Mientras que el Capítulo 16 se enfoca en los problemas de decisión únicos o episódicos, en los que la utilidad del resultado de cada acción era bien conocida, aquí nos preocupamos por los **problemas de decisión secuenciales**, en los que la utilidad del agente depende de una secuencia de decisiones. Los problemas de decisión secuenciales incorporan utilidades, incertidumbre, detección y, como casos especiales, problemas de búsqueda y planificación. La Sección 17.1 explica cómo se definen los problemas de decisión secuenciales, mientras que la Sección 17.2 describe métodos para resolverlos y para producir comportamientos apropiados para un entorno estocástico. La Sección 17.3 cubre los **problemas de bandidos multiarmados**, una clase específica y fascinante de problemas de decisión secuenciales que surgen en muchos contextos. En la Sección 17.4 se exploran los problemas de decisión en entornos parcialmente observables y la Sección 17.5 describe cómo resolverlos.

17.1 Problemas de decisión secuencial

Suponga que un agente está situado en el entorno 4 × 3, mostrado en la Figura 17.1(a). Partiendo del estado inicial, debe elegir una acción en cada paso de tiempo. La interacción con el entorno termina cuando el agente llega a uno de los estados objetivo, marcados +1 o –1. Al igual que para los problemas de búsqueda, las acciones disponibles para el agente en cada estado están dadas por ACCIONES(s), a veces abreviado a $A(s)$; en el entorno 4 × 3, las acciones en cada estado son *Arriba*, *Abajo*, *Izquierda* y *Derecha*. Asumimos por ahora que el entorno es **completamente observable**, por lo que el agente siempre sabe dónde está.

Si el entorno fuera determinista, una solución sería fácil: [*Arriba*, *Arriba*, *Derecha*, *Derecha*, *Derecha*]. Sin embargo, el entorno no siempre irá de acuerdo con esta solución, porque las acciones son poco fiables. El modelo particular de movimiento estocástico que adoptamos se ilustra en la Figura 17.1(b). Cada acción logra el efecto previsto con una probabilidad de 0.8, pero el resto del tiempo, la acción mueve al agente en ángulos rectos a la dirección prevista. Además, si el agente choca contra una pared, se queda en la misma casilla. Por ejemplo, desde la casilla de inicio (1,1), la acción *Arriba* mueve al agente a (1,2) con una probabilidad de 0.8, pero se mueve a la derecha a (2,1) con una probabilidad de 0.1, y se mueve a la izquierda con una probabilidad de 0.1; choca contra la pared y se queda en (1,1). En tal entorno, la secuencia [*Arriba*, *Arriba*, *Derecha*, *Derecha*, *Derecha*] sube alrededor de la barrera y llega al estado objetivo en (4,3) con una probabilidad de $0.8^5 = 0.32768$. También hay una pequeña posibilidad de llegar de manera accidental al objetivo, yendo por el otro lado con una probabilidad de $0.1^4 \times 0.8$, para un total de 0.32776. (Ver también Ejercicio 17.MDPX).

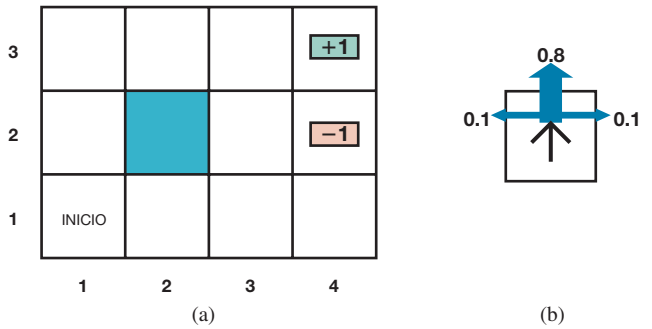


Figura 17.1 (a) Entorno estocástico 4 × 3 que presenta al agente un problema de decisión secuencial. (b) Ilustración del modelo de transición del entorno: el resultado "previsto" ocurre con una probabilidad de 0.8, pero con probabilidad de 0.2 el agente se mueve en ángulos rectos a la dirección prevista. Una colisión con una pared resulta en ningún movimiento. Las transiciones a los dos estados terminales tienen una recompensa de +1 y –1, respectivamente; todas las demás transiciones tienen una recompensa de –0.04.

Como en el Capítulo 3, el **modelo de transición** (o "modelo", cuando el significado es claro) describe el resultado de cada acción en cada estado. Aquí, el resultado es estocástico, por lo que escribimos $P(s'|s,a)$ para la probabilidad de llegar al estado s' si la acción a se realiza en el estado s. (Algunos autores escriben $T(s,a,s')$ para el modelo de transición). Asumiremos que las transiciones son **Markovianas**: la probabilidad de llegar a s' desde s depende sólo de s y no de la historia de estados previos.

Para completar la definición del entorno de la tarea, debemos especificar la función de utilidad para el agente. Debido a que el problema de decisión es secuencial, la función de utilidad dependerá de una secuencia de estados y acciones, una **historia del entorno**, en lugar de un solo estado. Más adelante en esta sección, investigaremos la naturaleza de las funciones de utilidad en las historias; por ahora, estipulamos que para cada transición de s a s', a través de la acción a, el agente recibe una **recompensa** $R(s,a,s')$. Las recompensas pueden ser positivas o negativas, pero están limitadas por $\pm R_{\max}$.[1]

Recompensa

Para nuestro ejemplo particular, la recompensa es de −0.04 para todas las transiciones, excepto aquellas que entran en estados terminales (con recompensas de +1 y –1). La utilidad de una historia del entorno es (por ahora) la *suma* de las recompensas recibidas. Por ejemplo, si el agente llega al estado +1 después de 10 pasos, su utilidad total será de $(9 \times -0.04) + 1 = 0.64$. La recompensa negativa de –0.04 da al agente un incentivo para llegar a (4,3) de manera rápida, por lo que nuestro entorno es una generalización estocástica de los problemas de búsqueda del Capítulo 3. Otra forma de decir esto es que al agente no le gusta vivir en este entorno y, por lo tanto, quiere salir lo antes posible.

Para resumir: un problema de decisión secuencial para un entorno por completo observable, estocástico con un modelo de transición markoviano y recompensas aditivas se llama un **proceso de decisión de Markov**, o MDP, y consta de un conjunto de estados (con un estado inicial s_0); un conjunto ACTIONS(s) de acciones en cada estado; un modelo de transición $P(s'|s,a)$; y una función de recompensa $R(s,a,s')$. Los métodos para resolver MDPs con frecuencia implican **programación dinámica**: simplificar un problema dividiéndolo de manera recursiva en piezas más pequeñas y recordando las soluciones óptimas para las piezas.

Proceso de decisión de Markov

Programación dinámica

[1] También es posible usar costos $c(s,a,s')$, como en la definición de problemas de búsqueda en el Capítulo 3. Sin embargo, el uso de recompensas es estándar en la literatura sobre decisiones secuenciales bajo incertidumbre.

La siguiente pregunta es, ¿cómo se ve una solución al problema? Ninguna secuencia de acción fija permite resolver el problema, porque el agente puede terminar en un estado distinto al objetivo. Por lo tanto, una solución debe especificar qué debe hacer el agente para cualquier estado que el agente pueda alcanzar. Una solución de este tipo se llama **política**. Es tradicional denotar una política por π, y $\pi(s)$ es la acción recomendada por la política π para el estado s. No importa cuál sea el resultado de la acción, el estado resultante estará en la política, y el agente sabrá qué hacer después.

Política

Cada vez que se ejecuta una política dada a partir del estado inicial, la naturaleza estocástica del entorno puede llevar a una historia de entorno diferente. La calidad de una política se mide, por lo tanto, por la utilidad *esperada* de las posibles historias de entorno generadas por esa política. Una **política óptima** es una política que produce la mayor utilidad esperada. Usamos π^* para denotar una política óptima. Dada π^*, el agente decide qué hacer consultando su percepción actual, que le indica el estado actual s, y luego ejecuta la acción $\pi^*(s)$. Una política representa de manera explícita la función del agente y, en consecuencia, es una descripción de un agente de reflejo simple, calculado a partir de la información utilizada para un agente basado en utilidad.

Política óptima

Las políticas óptimas para el mundo de la Figura 17.1 se muestran en la Figura 17.2(a). Hay dos políticas porque el agente es indiferente entre ir a la izquierda y subir desde (3,1): ir a la izquierda es más seguro, aunque más largo; mientras que subir es más rápido, pero se corre el riesgo de caer en (4,2) por accidente. En general, con frecuencia habrá múltiples políticas óptimas.

El equilibrio entre riesgo y recompensa cambia dependiendo del valor de $r = R(s,a,s')$ para las transiciones entre estados no terminales. Las políticas mostradas en la Figura 17.2(a) son óptimas para $-0.0850 < r < -0.0273$. La Figura 17.2(b) muestra políticas óptimas para otros cuatro rangos de r. Cuando $r < -1.6497$, la vida es tan dolorosa que el agente se dirige a la salida más cercana, incluso si la salida vale –1. Cuando $-0.7311 < r < -0.4526$, la vida es muy desagradable; el agente toma la ruta más corta al estado +1 desde (2,1), (3,1) y (3,2), pero desde (4,1) el costo de llegar a +1 es tan alto que el agente prefiere sumergirse en –1. Cuando la vida es sólo ligeramente sombría ($-0.0274 < r < 0$), la política óptima no toma *riesgo alguno en absoluto*. En (4,1) y (3,2), el agente se dirige lejos del estado –1 para que no pueda caer por accidente, aunque esto signifique golpear su cabeza contra la pared bastantes veces. Finalmente, si $r > 0$, entonces la vida es agradable y el agente evita *ambas* salidas. Siempre que las acciones en (4,1), (3,2) y (3,3) son como se muestra, cada política es óptima y el agente obtiene una recompensa total infinita porque nunca entra en un estado terminal. Resulta que hay nueve políticas óptimas en total para varios rangos de r; el Ejercicio 17.THRC le pide que las encuentre.

La introducción de la incertidumbre acerca los MDPs al mundo real más que a los problemas de búsqueda deterministas. Por esta razón, los MDPs han sido estudiados en varios campos, entre ellos la IA, la investigación de operaciones, la economía y la teoría del control. Además, se han propuesto docenas de algoritmos de solución, varios de los cuales discutimos en la Sección 17.2. Sin embargo, primero detallamos con mayor precisión las definiciones de utilidades, políticas óptimas y modelos para MDPs.

17.1.1 Utilidades a lo largo del tiempo

En el ejemplo de MDP en la Figura 17.1, el rendimiento del agente se midió mediante una suma de recompensas por las transiciones experimentadas. Esta elección de medida de rendimiento no es arbitraria, pero no es la única posibilidad para la función de utilidad[2] en las historias del entorno, que escribimos como $U_h([s_0,a_0,s_1,a_1,\ldots,s_n])$.

[2] En este capítulo usamos U para la función de utilidad (para ser consistentes con el resto del libro), pero muchos trabajos sobre MDPs en su lugar usan V (para *valor*).

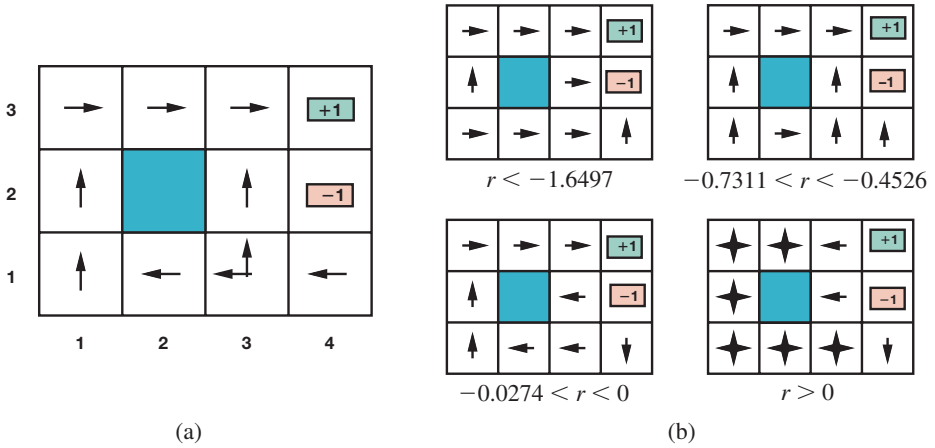


Figura 17.2 (a) Las políticas óptimas para el entorno estocástico con $r = -0.04$ para transiciones entre estados no terminales. Hay dos políticas porque en el estado (3,1) tanto *Izquierda* como *Hacia arriba* son óptimas. (b) Políticas óptimas para cuatro rangos diferentes de r.

La primera pregunta a responder es si hay un **horizonte finito** o un **horizonte infinito** para la toma de decisiones. Un horizonte finito significa que hay un tiempo *fijo* N después del cual nada importa: el juego ha terminado, por así decirlo. Así,

Horizonte finito
Horizonte infinito

$$U_h([s_0,a_0,s_1,a_1,\ldots,s_{N+k}]) = U_h([s_0,a_0,s_1,a_1,\ldots,s_N])$$

para todo $k > 0$. Por ejemplo, suponga que un agente comienza en (3,1) en el mundo 4×3 de la Figura 17.1, e imagine que $N = 3$. Entonces, para tener alguna posibilidad de llegar al estado +1, el agente debe dirigirse hacia él de manera directa, y la acción óptima es *hacia arriba*. Por otro lado, si $N = 100$, entonces hay mucho tiempo para tomar la ruta segura yendo a la *izquierda*. *Así, con un horizonte finito, una acción óptima en un estado dado depende de cuánto tiempo queda*. Una política que depende del tiempo se llama **no estacionaria**.

Política no estacionaria

Por otro lado, sin un límite de tiempo fijo no hay razón para comportarse de manera diferente en el mismo estado en diferentes momentos. En consecuencia, una acción óptima depende sólo del estado actual, pero la política óptima es **estacionaria**. Entonces, las políticas para el caso de horizonte infinito son más simples que aquellas para el caso de horizonte finito, pero nos ocupamos del caso de horizonte infinito en este capítulo. (Veremos más adelante que para entornos parcialmente observables, el caso de horizonte infinito no es tan simple). Tenga en cuenta que "horizonte infinito" no significa que todas las secuencias de estados son infinitas, sólo significa que no hay una fecha límite fija. Es posible que existan secuencias de estados finitas en un MDP de horizonte infinito que contiene un estado terminal.

Política estacionaria

La siguiente pregunta que debemos resolver es cómo calcular la utilidad de las secuencias de estados. A lo largo de este capítulo, usaremos **recompensas descontadas aditivas**: la utilidad de una historia es

Recompensas descontadas aditivas

$$U_h([s_0,a_0,s_1,a_1,s_2,\ldots]) = R(s_0,a_0,s_1) + \gamma R(s_1,a_1,s_2) + \gamma^2 R(s_2,a_2,s_3) + \cdots,$$

donde el **factor de descuento** γ es un número entre 0 y 1. El factor de descuento describe la preferencia de un agente por las recompensas actuales sobre las recompensas futuras. Cuando γ está cerca de 0, las recompensas en el futuro lejano se ven como insignificantes. Cuando γ está cerca de 1, un agente está más dispuesto a esperar recompensas a largo plazo. Cuando γ es 1, las

Factor de descuento

Recompensa aditiva

recompensas descontadas se reducen al caso especial de **recompensas aditivas** puras. Observe que la parte aditiva se usó de manera implícita en nuestro uso de funciones de costo de ruta en algoritmos de búsqueda heurística (Capítulo 3).

Hay múltiples razones por las que las recompensas descontadas aditivas tienen sentido. Una es empírica: tanto los humanos como los animales parecen valorar más las recompensas a corto plazo que las recompensas a un futuro lejano. Otra es económica: si las recompensas son monetarias, entonces es mejor obtenerlas antes que después; esto se debe a que las recompensas tempranas permiten ser invertidas y producir retornos, mientras espera las recompensas a futuro. En este contexto, un factor de descuento de γ es equivalente a una tasa de interés de $(1/\gamma) - 1$. Por ejemplo, un factor de descuento de $\gamma = 0.9$ es equivalente a una tasa de interés de 11.1 %.

Una tercera razón es la incertidumbre sobre las verdaderas recompensas: pueden no llegar nunca debido a todo tipo de razones que no se tienen en cuenta en el modelo de transición. Bajo ciertas suposiciones, un factor de descuento de *gamma* es equivalente a agregar una probabilidad $1 - \gamma$ de terminación accidental en cada paso de tiempo, independiente de la acción tomada.

Una cuarta justificación surge de una propiedad natural de las preferencias sobre las historias. En la terminología de la teoría de utilidad multiatributo (ver Sección 16.4), cada transición $s_t \xrightarrow{a_t} s_{t+1}$ permite ser visto como un **atributo** de la historia $[s_0,a_0,s_1,a_1,s_2...]$. En principio, la función de utilidad podría depender de formas complejas en estos atributos; sin embargo, existe una suposición plausible de independencia de preferencia que es posible realizar, pues las preferencias del agente entre secuencias de estados son **estacionarias**.

Preferencia estacionaria

Asuma dos historias $[s_0,a_0,s_1,a_1,s_2,...]$ y $[s'_0,a'_0,s'_1,a'_1,s'_2,...]$ que comienzan con la misma transición (es decir, $s_0=s'_0$, $a_0 = a'_0$ y $s_1=s'_1$). Entonces la estacionariedad para las preferencias significa que las dos historias deben ser ordenadas por preferencia de la misma manera que las historias $[s_1,a_1,s_2,...]$ y $[s'_1,a'_1,s'_2,...]$. Esto significa que si prefiere un futuro a otro, comenzando mañana, entonces aún debería preferir ese futuro si fuera a comenzar hoy en lugar de mañana. La estacionariedad es una suposición inocua, pero el descuento aditivo es la única forma de utilidad en las historias que la satisface.

Una justificación final para las recompensas descontadas es que hace que algunas infinidades desagradables desaparezcan. Con horizontes infinitos hay una dificultad potencial: si el entorno no contiene un estado terminal o si el agente nunca llega a uno, entonces todas las historias del entorno serán infinitamente largas, y las utilidades con recompensas no descontadas aditivas con frecuencia serán infinitas. Mientras podemos estar de acuerdo en que $+\infty$ es mejor que $-\infty$, comparar dos secuencias de estado con $+\infty$ de utilidad es más difícil. Hay tres soluciones, dos de las cuales ya las hemos visto:

1. Con recompensas descontadas, la utilidad de una secuencia infinita es *finita*. De hecho, si $\gamma < 1$ y las recompensas están limitadas por $\pm R_{\max}$, tenemos
$$U_h([s_0,a_0,s_1,...]) = \sum_{t=0}^{\infty} \gamma^t R(s_t,a_t,s_{t+1}) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1-\gamma}, \tag{17.1}$$
usando la fórmula estándar para la suma de una serie geométrica infinita.
2. Si el entorno contiene estados terminales *y si se garantiza que el agente llegará a uno de manera eventual*, entonces nunca necesitaremos comparar secuencias infinitas. Una política garantizada para alcanzar un estado terminal se llama **política adecuada**. Así, con políticas adecuadas podemos usar $\gamma = 1$ (es decir, recompensas no descontadas aditivas). Las primeras tres políticas mostradas en la Figura 17.2(b) son adecuadas, pero la cuarta es inadecuada. Obtiene recompensa total infinita al mantenerse alejado de los estados terminales cuando la recompensa por las transiciones entre estados no terminales es positiva. La existencia de políticas inadecuadas puede causar que los algoritmos estándar resuelvan

Política adecuada

MDPs con recompensas aditivas y, por lo tanto, proporciona una buena razón para usar recompensas descontadas.

3. Las secuencias infinitas se comparan en términos de la **recompensa promedio** obtenida por pasos. Suponga que las transiciones a la casilla (1,1) en el mundo 4 × 3 tienen una recompensa de 0.1, pero las transiciones a otros estados no terminales tienen una recompensa de 0.01. Entonces, una política que hace todo lo posible para permanecer en (1,1) tendrá una recompensa promedio más alta que una que permanece en otro lugar. La recompensa promedio es un criterio útil para algunos problemas, pero el análisis de los algoritmos de recompensa promedio es complejo.

Recompensa promedio

Las recompensas aditivas descontadas presentan las menores dificultades en la evaluación de historias, por lo que las usaremos de ahora en adelante.

17.1.2 Políticas óptimas y las utilidades de los estados

Habiendo decidido que la utilidad de una historia dada es la suma de las recompensas descontadas, podemos comparar políticas a partir de las utilidades *esperadas* obtenidas una vez ejecutadas. Asumimos que el agente está en algún estado inicial s y definimos S_t (una variable aleatoria) como el estado al que llega el agente en el tiempo t al ejecutar una política particular π. (Note que, $S_0 = s$, el estado en el que se encuentra el agente ahora). La distribución de probabilidad sobre las secuencias de estados $S_1, S_2, \ldots$, está determinada por el estado inicial s, la política π, y el modelo de transición para el entorno.

La utilidad esperada obtenida al ejecutar π comenzando en s está dada por

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1})\right], \tag{17.2}$$

donde la expectativa E es con respecto a la distribución de probabilidad sobre las secuencias de estados determinadas por s y π. Ahora, de todas las políticas que el agente podría elegir para ejecutar comenzando en s, una (o más) tendrá utilidades esperadas más altas que todas las demás. Usaremos π_s^* para denotar una de estas políticas:

$$\pi_s^* = \underset{\pi}{\text{argmax}}\ U^{\pi}(s). \tag{17.3}$$

Recuerde que π_s^* es una política, por lo que recomienda una acción para cada estado; su conexión con s en particular es que es una política óptima cuando s es el estado inicial. Una consecuencia notable del uso de utilidades descontadas con horizontes infinitos es que la política óptima es *independiente* del estado inicial. (Por supuesto, la *secuencia de acciones* no será independiente; recuerde que una política es una función que especifica una acción para cada estado). Este hecho parece obvio: si la política π_a^* es óptima comenzando en a y la política π_b^* es óptima comenzando en b, entonces, cuando llegan a un tercer estado c, no hay una buena razón para que estén en desacuerdo entre sí, o con π_c^*, sobre qué hacer después.[3] Por lo tanto, podemos escribir π^* para una política óptima.

Dada esta definición, la verdadera utilidad de un estado es $U^{\pi^*}(s)$; es decir, la suma esperada de recompensas descontadas si el agente ejecuta una política óptima. Escribimos esto como $U(s)$, coincidiendo con la notación utilizada en el Capítulo 16 para la utilidad de un resultado. La Figura 17.3 muestra las utilidades para el mundo 4 × 3. Observe que las utilidades son mayores para los estados más cercanos a la salida +1, porque se requieren menos pasos para llegar a la salida.

[3] Aunque esto parece obvio, no se cumple para las políticas de horizonte finito o para otras formas de combinar recompensas a lo largo del tiempo, como tomar el máximo. La prueba se sigue de manera directa de la unicidad de la función de utilidad en los estados, como se muestra en la Sección 17.2.1

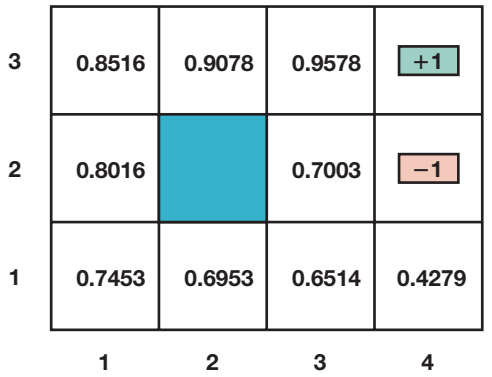


Figura 17.3 Las utilidades de los estados en el mundo 4 × 3 con $\gamma = 1$ y $r = -0.04$ para transiciones a estados no terminales.

La función de utilidad $U(s)$ permite al agente seleccionar acciones utilizando el principio de máxima utilidad esperada del Capítulo 16; es decir, elige la acción que maximiza la recompensa para el próximo paso, más la utilidad descontada esperada del estado subsiguiente:

$$\pi^*(s) = \operatorname*{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]. \tag{17.4}$$

Hemos definido la utilidad de un estado, $U(s)$, como la suma esperada de recompensas descontadas a partir de ese punto en adelante. De esto se deduce que hay una relación directa entre la utilidad de un estado y la utilidad de sus vecinos: *la utilidad de un estado es la recompensa esperada para la próxima transición, más la utilidad descontada del próximo estado, suponiendo que el agente elige la acción óptima*. Es decir, la utilidad de un estado se da por

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]. \tag{17.5}$$

Ecuación de Bellman

Esto se llama la **ecuación de Bellman**, por Richard Bellman (1957). Las utilidades de los estados, definidas por la Ecuación (17.2) como la utilidad esperada de las secuencias de estados subsiguientes, son soluciones del conjunto de ecuaciones de Bellman. De hecho, son soluciones *únicas*, como mostramos en la Sección 17.2.1.

Echemos un vistazo a una de las ecuaciones de Bellman para el mundo 4 × 3. La expresión para $U(1,1)$ es

$$\begin{aligned}\max\{\quad & [0.8(-0.04 + \gamma U(1,2)) + 0.1(-0.04 + \gamma U(2,1)) + 0.1(-0.04 + \gamma U(1,1))],\\ & [0.9(-0.04 + \gamma U(1,1)) + 0.1(-0.04 + \gamma U(1,2))],\\ & [0.9(-0.04 + \gamma U(1,1)) + 0.1(-0.04 + \gamma U(2,1))],\\ & [0.8(-0.04 + \gamma U(2,1)) + 0.1(-0.04 + \gamma U(1,2)) + 0.1(-0.04 + \gamma U(1,1))]\}\end{aligned}$$

donde las cuatro expresiones corresponden a los movimientos *Arriba*, *Izquierda*, *Abajo* y *Derecha*. Cuando introducimos los números de la Figura 17.3, con $\gamma = 1$, encontramos que *Arriba* es la mejor acción.

Otra cantidad importante es la **función de utilidad de acción**, o **función Q**: $Q(s,a)$ es la utilidad esperada de tomar una acción dada en un estado dado. La función Q está relacionada con las utilidades de la manera obvia:

Función Q

$$U(s) = \max_a Q(s,a). \tag{17.6}$$

Además, la política óptima se permite extraer de la función Q de la siguiente manera:

$$\pi^*(s) = \underset{a}{\text{argmax}}\, Q(s,a). \tag{17.7}$$

También podemos desarrollar una ecuación de Bellman para las funciones Q, notando que la recompensa total esperada por tomar una acción es su recompensa inmediata más la utilidad descontada del estado de resultado, que a su vez se expresa en términos de la función Q:

$$\begin{aligned} Q(s,a) &= \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')] \\ &= \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q(s',a')] \end{aligned} \tag{17.8}$$

Resolver las ecuaciones de Bellman para U (o para Q) nos da lo que necesitamos para encontrar una política óptima. La función Q aparece una y otra vez en algoritmos para resolver MDPs, por lo que usaremos la siguiente definición:

function Q-VALUE(mdp,s,a,U) **returns** un valor de utilidad

return $\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U[s']]$

17.1.3 Escala de recompensas

En el capítulo 16 se señaló que la escala de utilidades es arbitraria: una transformación afín deja la decisión óptima sin cambios. Podemos reemplazar $U(s)$ por $U'(s) = mU(s) + b$ donde m y b son cualquier constante tal que $m > 0$. Es fácil ver, a partir de la definición de utilidades como sumas descontadas de recompensas, que una transformación similar de recompensas dejará la política óptima sin cambios en un MDP:

$$R'(s,a,s') = mR(s,a,s') + b.$$

Resulta, sin embargo, que la descomposición aditiva de recompensas de utilidades conduce a más libertad en la definición de recompensas. Deje que $\Phi(s)$ sea *cualquier* función del estado s. Entonces, de acuerdo con el **teorema de conformación**, la siguiente transformación deja la política óptima sin cambios:

Teorema de conformación

$$R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s). \tag{17.9}$$

Para demostrar que esto es cierto, necesitamos probar que dos MDPs, M y M′, tienen políticas óptimas idénticas, siempre y cuando sólo difieran en sus funciones de recompensa, según lo especificado en la Ecuación (17.9). Comenzamos desde la ecuación de Bellman para Q, la función Q para MDP M:

$$Q(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q(s',a')].$$

Ahora dejemos que $Q'(s,a) = Q(s,a) - \Phi(s)$ y lo insertamos en esta ecuación; obtenemos

$$Q'(s,a) + \Phi(s) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} (Q'(s',a') + \Phi(s'))].$$

que luego se simplifica a

$$\begin{aligned} Q'(s,a) &= \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma\Phi(s') - \Phi(s) + \gamma \max_{a'} Q'(s',a')] \\ &= \sum_{s'} P(s'|s,a)[R'(s,a,s') + \gamma \max_{a'} Q'(s',a')]. \end{aligned}$$

En otras palabras, $Q'(s,a)$ satisface la ecuación de Bellman para MDP M'. Ahora podemos extraer la política óptima para M' usando la Ecuación (17.7):

$$\pi^*_{M'}(s) = \operatorname*{argmax}_a Q'(s,a) = \operatorname*{argmax}_a Q(s,a) - \Phi(s) = \operatorname*{argmax}_a Q(s,a) = \pi^*_M(s).$$

La función $\Phi(s)$ se llama **potencial**, por analogía con el potencial eléctrico (voltaje) que da lugar a campos eléctricos. El término $\gamma\Phi(s') - \Phi(s)$ funciona como un gradiente del potencial. Así, si $\Phi(s)$ tiene un valor más alto en estados que tienen una utilidad más alta, la adición de $\gamma\Phi(s') - \Phi(s)$ a la recompensa tiene el efecto de llevar al agente "cuesta arriba" en utilidad.

A primera vista, parecería contraintuitivo que podamos modificar la recompensa de esta manera sin cambiar la política óptima. Ayuda si recordamos que *todas las políticas son óptimas* con una función de recompensa que es cero en todas partes. Esto significa, según el teorema de conformación, que todas las políticas son óptimas para cualquier recompensa basada en el potencial de la forma $R(s,a,s') = \gamma\Phi(s') - \Phi(s)$. De manera intuitiva, esto se debe a que con tal recompensa va de A a B, no importa en qué dirección vaya el agente. (Esto es más fácil de ver cuando $\gamma = 1$: a lo largo de cualquier camino, la suma de las recompensas se colapsa a $\Phi(B) - \Phi(A)$, por lo que todos los caminos son igualmente buenos). Entonces, agregar una recompensa basada en el potencial a cualquier otra recompensa no debería cambiar la política óptima.

La flexibilidad proporcionada por el teorema de conformación significa que en realidad podemos ayudar al agente haciendo que la recompensa inmediata refleje de manera directa lo que el agente debería hacer. De hecho, si establecemos $\Phi(s) = U(s)$, entonces la política codiciosa π_G con respecto a la recompensa modificada R' también es una política óptima:

$$\begin{aligned}
\pi_G(s) &= \operatorname*{argmax}_a \sum_{s'} P(s'|s,a)R'(s,a,s') \\
&= \operatorname*{argmax}_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma\Phi(s') - \Phi(s)] \\
&= \operatorname*{argmax}_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s') - U(s)] \\
&= \operatorname*{argmax}_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s') \\
&= \pi^*(s) \qquad \text{(por la Ecuación (17.4)).}
\end{aligned}$$

Por supuesto, para establecer $\Phi(s) = U(s)$, necesitaríamos conocer $U(s)$; así que no hay almuerzo gratis, pero aún hay un valor considerable en definir una función de recompensa que sea útil en la medida de lo posible. Esto es lo que hacen los entrenadores de animales cuando proporcionan una pequeña golosina al animal por cada paso en la secuencia objetivo.

17.1.4 Representando MDPs

La forma más sencilla de representar $P(s'|s,a)$ y $R(s,a,s')$ es con grandes tablas tridimensionales de tamaño $|S|^2|A|$. Esto está bien para problemas pequeños como el mundo 4×3, para el cual las tablas tienen $11^2 \times 4 = 484$ entradas cada una. En algunos casos, las tablas son **dispersas**, la mayoría de las entradas son cero porque cada estado s puede transitar a sólo un número limitado de estados s', lo que significa que las tablas son de tamaño $O(|S||A|)$. Para problemas más grandes, incluso las tablas dispersas son demasiado grandes.

Red de decisión dinámica

Al igual que en el Capítulo 16, donde las redes bayesianas se extendieron con acción y nodos de utilidad para crear redes de decisión, podemos representar MDPs extendiendo redes bayesianas dinámicas (DBNs, ver Capítulo 14) con nodos de decisión, recompensa y utilidad para crear **redes de decisión dinámicas**, o DDNs. Las DDNs son **representaciones factorizadas** en la terminología

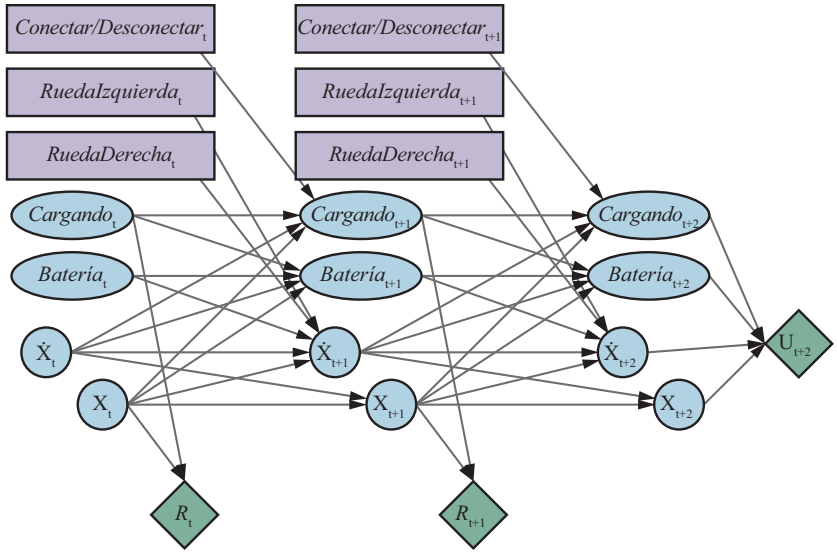


Figura 17.4 Red de decisión dinámica para un robot móvil con variables de estado para el nivel de batería, estado de carga, ubicación y velocidad, y variables de acción para los motores de las ruedas izquierda y derecha y para la carga.

del Capítulo 2; con frecuencia tienen una ventaja de complejidad exponencial sobre las representaciones atómicas y permiten modelar problemas del mundo real bastante sustanciales.

La Figura 17.4, que se basa en el DBN en la Figura 14.13(b) (página 486), muestra algunos elementos de un modelo realista para un robot móvil que permite cargarse a sí mismo. El estado S_t se descompone en cuatro variables de estado:

- $\mathbf{X}_t$ consta de la ubicación bidimensional en una cuadrícula más la orientación;
- $\dot{\mathbf{X}}_t$ es la tasa de cambio de $\mathbf{X}_t$;
- $Cargando_t$ es verdadero cuando el robot está conectado a una fuente de energía;
- $Batería_t$ es el nivel de batería, que modelamos como un entero en el rango 0,...,5.

El espacio de estados para el MDP es el producto cartesiano de los rangos de estas cuatro variables. La acción ahora es un conjunto $\mathbf{A}_t$ de variables de acción, compuesto por *Conectar*/*Desconectar*, que tiene tres valores (*conectado*, *desconectado* y *sin conexión*); *RuedaIzquierda* para la energía enviada a la rueda izquierda; y *RuedaDerecha* para la energía enviada a la rueda derecha. El conjunto de acciones para el MDP es el producto cartesiano de los rangos de estas tres variables. Observe que cada variable de acción afecta sólo a un subconjunto de las variables de estado.

El modelo de transición general es la distribución condicional $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{X}_t,\mathbf{A}_t)$, que permite ser calculado como un producto de probabilidades condicionales del DDN. La recompensa aquí es una variable sola que depende sólo de la ubicación $\mathbf{X}$ (por ejemplo, al llegar a un destino) y *Cargando*, ya que el robot tiene que pagar por la electricidad utilizada; en este modelo particular, la recompensa no depende de la acción ni del estado de resultado.

La red en la Figura 17.4 ha sido proyectada tres pasos hacia el futuro. Observe que la red incluye nodos para las *recompensas* para t, $t+1$ y $t+2$, pero la *utilidad* para $t+3$. Esto es porque el agente debe maximizar la suma (descontada) de todas las recompensas futuras, y $U(\mathbf{X}_{t+3})$ representa la

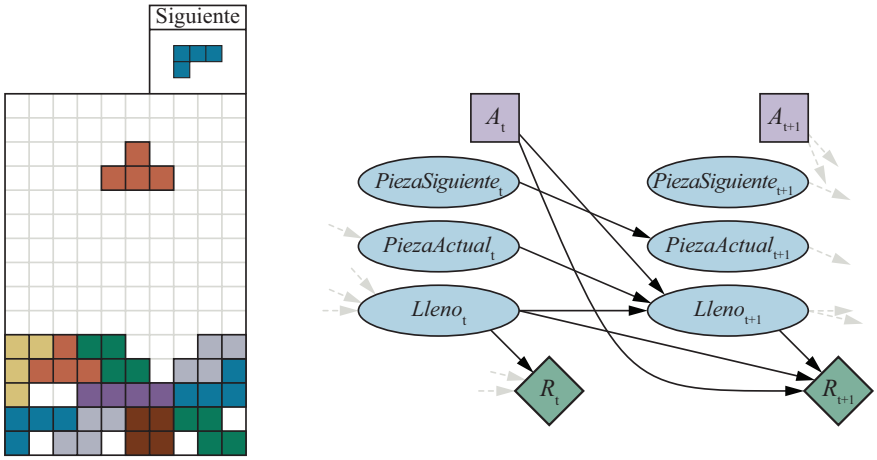


Figura 17.5 (a) Juego de Tetris. La pieza en forma de T en la parte superior central puede caer en cualquier orientación y en cualquier posición horizontal. Si se completa una fila, esa fila desaparece y las filas superiores se mueven hacia abajo, entonces el agente recibe un punto. La siguiente pieza (aquí, la pieza en forma de L en la parte superior derecha) se convierte en la pieza actual, y aparece una nueva pieza siguiente, elegida al azar entre los siete tipos de piezas. El juego termina si el tablero se llena hasta la parte superior. (b) El DDN para el MDP de Tetris.

recompensa por todas las recompensas desde $t + 3$ en adelante. Si se dispone de una aproximación heurística a U, permite incluirse en la representación del MDP de esta manera y utilizarse en lugar de una expansión adicional. Este enfoque está relacionado con el uso de la búsqueda de profundidad limitada y las funciones de evaluación heurística para juegos en el Capítulo 5.

Otro MDP interesante y bien estudiado es el juego de Tetris (Figura 17.5(a)). Las variables de estado para el juego son la *PiezaActual*, la *PiezaSiguiente* y una variable con valor de vector de bits *Lleno* con un bit para cada una de las 10×20 ubicaciones del tablero. Por lo tanto, el espacio de estado tiene $7 \times 7 \times 2^{200} \approx 10^{62}$ estados. El DDN para Tetris se muestra en la Figura 17.5(b). Note que $Lleno_{t+1}$ es una función determinista de $Lleno_t$ y A_t. Resulta que cada política para Tetris es apropiada (llega a un estado terminal): de manera eventual el tablero se llena a pesar de los mejores esfuerzos para vaciarlo.

17.2 Algoritmos para MDPs

En esta sección, presentamos cuatro algoritmos diferentes para resolver MDPs. Los primeros tres, **iteración de valor**, **iteración de política** y **programación lineal**, generan soluciones exactas fuera de línea. El cuarto es una familia de algoritmos aproximados online que incluye la **planificación de Monte Carlo**.

Planificación de Monte Carlo

17.2.1 Iteración de valor

Iteración de valor

La ecuación de Bellman (Ecuación (17.5)) es la base del algoritmo de **iteración de valor** para resolver MDPs. Si hay n estados posibles, entonces hay n ecuaciones de Bellman, una para cada

```
function VALUE-ITERATION(mdp,ϵ) returns una función de utilidad
  inputs: mdp, un MDP con estados S, acciones A(s), modelo de transición P(s'|s,a),
          recompensas R(s,a,s'), descuento γ
          ϵ, el error máximo permitido en la utilidad de cualquier estado
  local variables: U,U', vectores de utilidades para estados en S, con valor inicial cero
          δ, el cambio relativo máximo en la utilidad de cualquier estado

  repeat
    U←U'; δ←0
    for each estado s in S do
      U'[s]←max_{a∈A(s)} Q-VALUE(mdp,s,a,U)
      if|U'[s] − U[s]| > δ then δ←|U'[s] − U[s]|
  until δ ≤ ϵ (1−γ)/γ
  return U
```

Figura 17.6 El algoritmo de iteración de valor para calcular las utilidades de los estados. La condición de terminación es de la Ecuación (17.12).

estado. Las n ecuaciones contienen n incógnitas: las utilidades de los estados. Así que nos gustaría resolver estas ecuaciones simultáneas para encontrar las utilidades. Hay un problema: las ecuaciones son *no lineales*, porque el operador "max" no es un operador lineal. Mientras que los sistemas de ecuaciones lineales sí permiten ser resueltos mediante el uso de técnicas de álgebra lineal, los sistemas de ecuaciones no lineales son más problemáticos. Una cosa a probar es un enfoque *iterativo*. Empezamos con valores iniciales arbitrarios para las utilidades, calculamos el lado derecho de la ecuación y lo introducimos en el lado izquierdo, actualizando así la utilidad de cada estado a partir de las utilidades de sus vecinos. Repetimos esto hasta que alcanzamos un equilibrio.

Dejemos que $U_i(s)$ sea el valor de utilidad para el estado s en la iteración i. El paso de iteración, llamado **actualización de Bellman**, se ve así:

Actualización de Bellman

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U_i(s')], \tag{17.10}$$

donde se supone que la actualización se aplica de manera simultánea a todos los estados en cada iteración. Si aplicamos la actualización de Bellman de manera infinita, estamos garantizados de alcanzar un equilibrio (ver "convergencia de la iteración de valor" a continuación), en cuyo caso los valores de utilidad finales deben ser soluciones a las ecuaciones de Bellman. De hecho, también son las soluciones *únicas*, y la política correspondiente (obtenida usando la Ecuación (17.4)) es óptima. El algoritmo detallado, incluida una condición de terminación cuando las utilidades están "suficientemente cerca", se muestra en la Figura 17.6. Note que hacemos uso de la función Q-VALOR definida en la página 569.

Podemos aplicar la iteración de valor al mundo 4 × 3 en la Figura 17.1(a). Comenzando con valores iniciales de cero, las utilidades evolucionan como se muestra en la Figura 17.7(a). Observe cómo los estados a diferentes distancias de (4,3) acumulan recompensa negativa hasta que se encuentra un camino a (4,3), momento en el que las utilidades comienzan a aumentar. Podemos pensar en el algoritmo de iteración de valor como *propagando información* a través del espacio de estado mediante actualizaciones locales.

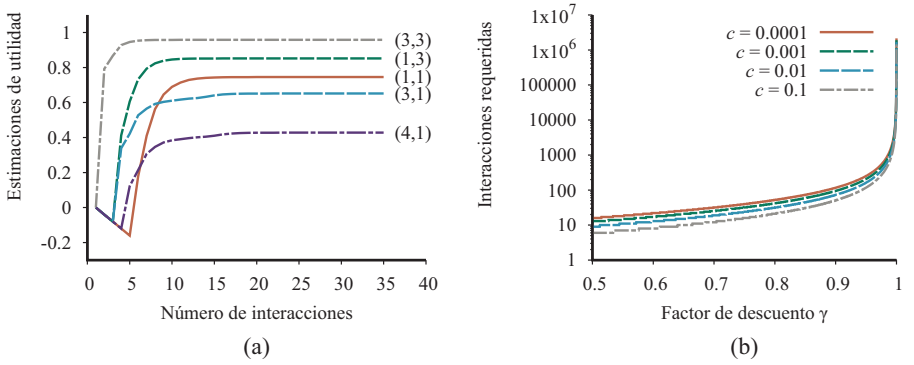


Figura 17.7 (a) Grafo que muestra la evolución de las utilidades de los estados seleccionados usando la iteración de valor. (b) Número de iteraciones de valor necesarias para garantizar un error de máximo $\epsilon = c \cdot R_{max}$, para diferentes valores de c, como función del factor de descuento γ.

Convergencia de la iteración de valor

Dijimos que la iteración de valor converge de manera eventual a un conjunto único de soluciones de las ecuaciones de Bellman. En esta sección explicamos por qué sucede esto. Introducimos algunas ideas matemáticas útiles en el camino y obtenemos unos métodos para evaluar el error en la función de utilidad devuelta cuando el algoritmo se termina temprano; esto es útil porque significa que no tenemos que correr para siempre. Esta sección es bastante técnica.

Contracción

El concepto básico utilizado para demostrar que la iteración de valor converge es la noción de una **contracción**. Hablando en términos generales, una contracción es una función de un argumento que, cuando se aplica a dos entradas diferentes a su vez, produce dos valores de salida que están "más juntos" que las entradas originales, al menos por algún factor constante. Por ejemplo, la función "dividir por dos" es una contracción, porque después de dividir cualquier dos números por dos su diferencia se reduce a la mitad. Observe que la función "dividir por dos" tiene un punto fijo, a saber, cero, que no cambia con la aplicación de la función. A partir de este ejemplo, podemos discernir dos propiedades importantes de las contracciones:

- Una contracción tiene sólo un punto fijo; si hubiera dos puntos fijos, no se acercarían cuando se aplicara la función, por lo que no sería una contracción.
- Cuando la función se aplica a cualquier argumento, el valor debe acercarse al punto fijo (porque el punto fijo no se mueve), por lo que la aplicación repetida de una contracción siempre alcanza el punto fijo en el límite.

Ahora, suponga que vemos la actualización de Bellman (Ecuación (17.10)) como un operador B que se aplica de manera simultánea para actualizar la utilidad de cada estado. Entonces la ecuación de Bellman se convierte en $U = BU$ y la ecuación de actualización de Bellman se define como

$$U_{i+1} \longleftarrow BU_i.$$

Norma máxima

Ahora, necesitamos una forma de medir las distancias entre los vectores de utilidad. Usaremos la **norma máxima**, que mide la "longitud" de un vector por el valor absoluto de su componente más grande:

$$||U|| = \max_s |U(s)|.$$

Con esta definición, la "distancia" entre dos vectores, $\|U - U'\|$, es la máxima diferencia entre cualquier dos elementos correspondientes. El resultado principal de esta sección es el siguiente: Dejemos que U_i y U'_i sean cualesquiera dos vectores de utilidad. Entonces tenemos

$$\|BU_i - BU'_i\| \leq \gamma \|U_i - U'_i\|. \tag{17.11}$$

Es decir, *la actualización de Bellman es una contracción por un factor de γ en el espacio de los vectores de utilidad*. (El Ejercicio 17.VICT proporciona alguna orientación sobre cómo probar esta afirmación). Por lo tanto, a partir de las propiedades de las contracciones en general, se deduce que la iteración de valor siempre converge a una solución única de las ecuaciones de Bellman siempre que $\gamma < 1$.

También podemos usar la propiedad de contracción para analizar la tasa de convergencia a una solución. En particular, podemos reemplazar U'_i en la Ecuación (17.11) con las *verdaderas* utilidades U, para las cuales $BU = U$. Entonces obtenemos la desigualdad

$$\|BU_i - U\| \leq \gamma \|U_i - U.$$

Si vemos $\|U_i - U\|$ como el *error* en la estimación U_i, vemos que el error se reduce por un factor de al menos γ en cada iteración. En consecuencia, la iteración de valor converge de manera exponencial. Podemos calcular el número de iteraciones requeridas de la siguiente manera: Primero, recordamos de la Ecuación (17.1) que las utilidades de todos los estados están limitadas por $\pm R_{\max}/(1-\gamma)$; esto significa que el error inicial máximo es $U_0 - U \leq 2R_{\max}/(1-\gamma)$. Suponga que corremos durante N iteraciones para alcanzar un error de máximo ϵ. Entonces, porque el error se reduce al menos γ cada vez, requerimos $\gamma^N \cdot 2R_{\max}/(1-\gamma) \leq \epsilon$. Tomando logaritmos, encontramos que

$$N = \lceil \log(2R_{\max}/\epsilon(1-\gamma))/\log(1/\gamma) \rceil$$

las iteraciones son suficientes. La Figura 17.7(b) muestra cómo N varía con γ para diferentes valores de la relación $\epsilon/R_{\max}$. La buena noticia es que, debido a la convergencia de manera exponencial, N no depende mucho de la relación $\epsilon/R_{\max}$. La mala noticia es que N crece a medida que γ se acerca a 1. Podemos obtener una convergencia rápida si hacemos γ pequeño, pero esto le da al agente un horizonte corto y podría perder los efectos a largo plazo de las acciones del agente.

El límite de error en el párrafo anterior da una idea de los factores que influyen en el tiempo de ejecución del algoritmo, aunque a veces es demasiado conservador como método para decidir cuándo detener la iteración. Para este último propósito, podemos usar un límite que relaciona el error con el tamaño de la actualización de Bellman en cualquier iteración dada. Con base en la propiedad de contracción (Ecuación (17.11)) es posible demostrar que si la actualización es pequeña (es decir, la utilidad de ningún estado cambia mucho), entonces el error, en comparación con la verdadera función de utilidad, también es pequeño. Más específico,

$$\text{si} \quad \|U_{i+1} - U_i\| < \epsilon(1-\gamma)/\gamma \quad \text{entonces} \quad \|U_{i+1} - U\| < \epsilon. \tag{17.12}$$

Esta es la condición de terminación usada en el algoritmo de ITERACIÓN-VALOR de la Figura 17.6.

Hasta ahora hemos analizado el error en la función de utilidad devuelta por la iteración de valor algoritmo. *Sin embargo, lo que realmente le importa al agente es qué tan bien le irá si toma sus decisiones en función de esta función de utilidad*. Suponga que después de i iteraciones de iteración de valor, el agente tiene una estimación U_i de la verdadera utilidad U y obtiene la política de utilidad esperada máxima (MEU) π_i basada en una mirada hacia adelante de un paso utilizando U_i (como en la Ecuación (17.4)). ¿Será el comportamiento resultante casi tan bueno como el comportamiento óptimo? Esta es una pregunta crucial para cualquier agente real, y resulta que la respuesta es sí. $U^{\pi_i}(s)$ es la utilidad obtenida si π_i se ejecuta a partir de s, y la **pérdida de política** $\|U^{\pi_i} - U\|$ es lo máximo que el agente es capaz de perder por ejecutar

Pérdida de política

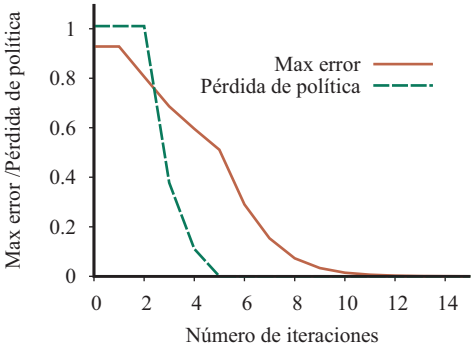


Figura 17.8 Error máximo $||U_i - U||$ de las estimaciones de utilidad y pérdida de política $||U^{\pi i} - U||$, como función del número de iteraciones de iteración de valor en el mundo 4 × 3.

π_i en lugar de la política óptima π^*. La pérdida de política de π_i está conectada al error en U_i por la siguiente desigualdad:

$$\text{si} \quad ||U_i - U|| < \epsilon \quad \text{entonces} \quad ||U^{\pi i} - U|| < 2\epsilon. \tag{17.13}$$

En la práctica, con frecuencia ocurre que π_i se vuelve óptimo mucho antes de que U_i haya convergido. La Figura 17.8 muestra cómo el error máximo en U_i y la pérdida de política se acercan a cero a medida que avanza el proceso de iteración de valor para el entorno 4 × 3 con $\gamma = 0.9$. La política π_i es óptima cuando $i = 5$, aunque el error máximo en U_i todavía es 0.51.

Ahora tenemos todo lo que necesitamos para usar la iteración de valor en la práctica. Sabemos que converge a las utilidades correctas, podemos limitar el error en las estimaciones de utilidad si nos detenemos después de un número finito de iteraciones, y podemos limitar la pérdida de política que resulta de ejecutar la política MEU correspondiente. Como nota final, todos los resultados en esta sección dependen de descuento con $\gamma < 1$. Si $\gamma = 1$ y el entorno contiene estados terminales, entonces es posible derivar un conjunto similar de resultados de convergencia y límites de error.

17.2.2 Iteración de política

En la sección anterior observamos que es posible obtener una política óptima aun cuando la estimación de la función de utilidad es inexacta. Si una acción es claramente mejor que todas las demás, entonces la magnitud exacta de las utilidades en los estados involucrados no necesita ser precisa. Esta idea sugiere una forma alternativa de encontrar políticas óptimas. El algoritmo de **iteración de política** alterna los siguientes dos pasos, comenzando desde alguna política inicial π_0:

Iteración de política

Evaluación de la política

Mejora de la política

- **Evaluación de la política**: Dada una política π_i, calcular $U_i = U^{\pi i}$, la utilidad de cada estado si π_i fuera ejecutado.
- **Mejora de la política**: Calcular una nueva política MEU π_{i+1}, utilizando una mirada hacia adelante de un paso basada en U_i (como en la Ecuación (17.4)).

El algoritmo termina cuando el paso de mejora de la política no produce ningún cambio en las utilidades. En este punto, sabemos que la función de utilidad U_i es un punto fijo de la actualización de Bellman, por lo que es una solución a las ecuaciones de Bellman, y π_i debe ser una política óptima. Debido a que sólo hay políticas finitas para un espacio de estado finito, y para cada

function POLICY-ITERATION(*mdp*) **returns** una política
  inputs: *mdp*, un MDP con estados *S*, acciones $A(s)$, modelo de transición $P(s'|s,a)$
  local variables: *U*, un vector de utilidades para estados en *S*, con valor inicial igual a cero
    π, un vector de política indexado por estado, en inicio aleatorio

  repeat
    $U \leftarrow$ POLICY-EVALUATION(π,*U*,*mdp*)
    unchanged? $\leftarrow$ verdadero
    for each estado *s* **in** *S* **do**
      $a^* \leftarrow \underset{a \in A(s)}{\text{argmax}}$ Q-VALUE(*mdp*,*s*,*a*,*U*)
      if Q-Value(*mdp*,*s*,a^*,*U*) > Q-Value (*mdp*,*s*,$\pi[s]$,*U*) **then**
        $\pi[s] \leftarrow a^*$; *unchanged*?$\leftarrow$falso
  until *unchanged*?
  return π

Figura 17.9 Algoritmo de iteración de políticas para calcular una política óptima.

iteración es posible demostrarse que produce una mejor política, la iteración de la política debe terminar. El algoritmo se muestra en la Figura 17.9. Al igual que con la iteración de valor, utilizamos la función Q-VALUE definida en la página 569.

¿Cómo implementamos POLICY-EVALUATION? Resulta que hacerlo es más sencillo que resolver las ecuaciones de Bellman estándar (que es lo que hace la iteración de valor), porque la acción en cada estado está fijada por la política. En la iteración *i*, la política π_i especifica la acción $\pi_i(s)$ en el estado *s*. Esto significa que tenemos una versión simplificada de la ecuación de Bellman (17.5) que relaciona la utilidad de *s* (bajo π_i) con las utilidades de sus vecinos:

$$U_i(s) = \sum_{s'} P(s'|s,\pi_i(s))[R(s,\pi_i(s),s') + \gamma U_i(s')]. \tag{17.14}$$

Por ejemplo, suponga que π_i es la política mostrada en la Figura 17.2(a). Así $\pi_i(1,1) = \textit{Arriba}$, $\pi_i(1,2) = \textit{Arriba}$, y así sucesivamente, y las ecuaciones de Bellman simplificadas son

$$U_i(1,1) = 0.8[-0.04 + U_i(1,2)] + 0.1[-0.04 + U_i(2,1) + 0.1[-0.04 + U_i(1,1)]],$$
$$U_i(1,2) = 0.8[-0.04 + U_i(1,3)] + 0.2[-0.04 + U_i(1,2)],$$

para todos los estados. El punto importante es que estas ecuaciones son *lineales*, porque se ha eliminado el operador "max". Para *n* estados, tenemos *n* ecuaciones lineales con *n* incógnitas, que se resuelven en tiempo $O(n^3)$ mediante métodos estándar de álgebra lineal. Si el modelo de transición es escaso, es decir, si cada estado transita sólo a un pequeño número de otros estados, entonces el proceso de solución puede ser aún más rápido.

Para espacios de estado pequeños, la evaluación de políticas utilizando métodos de solución exacta suele ser el enfoque más eficiente. Para espacios de estado grandes, el tiempo $O(n^3)$ podría ser prohibitivo. Por fortuna, no es necesario hacer una *exacta* evaluación de política. En su lugar, podemos realizar algunos pasos de iteración de valor simplificados (simplificados porque la política es fija) para dar una buena aproximación de las utilidades. La actualización de Bellman simplificada para este proceso es

$$U_{i+1}(s) \leftarrow \sum_{s'} P(s'|s,\pi_i(s))[R(s,\pi_i(s),s') + \gamma U_i(s')],$$

y esto se repite múltiples veces para producir la próxima estimación de utilidad. El algoritmo resultante se llama **iteración de política modificada**.

Iteración de política modificada

Los algoritmos que hemos descrito hasta ahora requieren actualizar la utilidad o la política para todos los estados a la vez. Resulta que esto no es estrictamente necesario. De hecho, en cada iteración, podemos elegir *cualquier subconjunto* de estados y aplicar *cualquier* tipo de actualización (mejora de la política o iteración de valor simplificada) a ese subconjunto. Este algoritmo muy general se llama **iteración de política asíncrona**. Dadas ciertas condiciones en la política inicial y la función de utilidad inicial, se garantiza que la iteración de la política asíncrona convergerá en una política óptima. La libertad de elegir cualquier estado para trabajar significa que podemos diseñar algoritmos heurísticos mucho más eficientes; por ejemplo, algoritmos que se concentran en actualizar los valores de los estados que es probable que sean alcanzados por una buena política. No tiene sentido planificar los resultados de una acción que nunca harás.

Iteración de política asíncrona

17.2.3 Programación lineal

La **programación lineal** o LP (por sus siglas en inglés), que se mencionó de manera breve en el Capítulo 4 (página 121), es un enfoque general para formular problemas de optimización restringida, y hay muchos solucionadores LP de fuerza industrial disponibles. Dado que las ecuaciones de Bellman implican muchas sumas y máximos, no es sorprendente que resolver un MDP se pueda reducir a resolver un programa lineal formulado de manera adecuada.

La idea básica de la formulación es considerar como variables en la LP las utilidades $U(s)$ de cada estado s, notando que las utilidades para una política óptima son las utilidades más altas alcanzables consistentes con las ecuaciones de Bellman. En lenguaje LP, eso significa que buscamos minimizar $U(s)$ para todos los s sujetos a las desigualdades

$$U(s) \geq \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \mathrm{U}(s')]$$

para cada estado s y cada acción a.

Esto crea una conexión desde la programación dinámica hasta la programación lineal, para la cual se han estudiado en profundidad los algoritmos y los problemas de complejidad. Por ejemplo, a partir del hecho de que la programación lineal se resuelve en tiempo polinomial, es posible demostrar que los MDP se permiten resolver en tiempo polinomial en el número de estados y acciones y el número de bits necesarios para especificar el modelo. En la práctica, resulta que los solucionadores de LP rara vez son tan eficientes como la programación dinámica para resolver MDP. Además, el tiempo polinomial puede sonar bien, pero el número de estados suele ser muy grande. Finalmente, vale la pena recordar que incluso el algoritmo de búsqueda más simple y menos informado del Capítulo 3 se ejecuta en tiempo lineal en el número de estados y acciones.

17.2.4 Algoritmos online para MDP

La iteración de valor y la iteración de política son algoritmos *offline*: como el algoritmo A^* en el Capítulo 3, generan una solución óptima para el problema, que luego permite ser ejecutada por un agente simple. Para MDP suficientemente grandes, como el MDP de Tetris con 10^{62} estados, la solución fuera de línea exacta, aun por un algoritmo de tiempo polinomial, no es posible. Se han desarrollado múltiples técnicas para la solución offline aproximada de MDP; estos se cubren en las notas al final del capítulo y en el Capítulo 22 (Aprendizaje por refuerzo).

Aquí consideraremos algoritmos online, análogos a los utilizados para jugar juegos en el Capítulo 5, donde el agente realiza una cantidad significativa de cálculos en cada punto de decisión en lugar de operar con información precalculada.

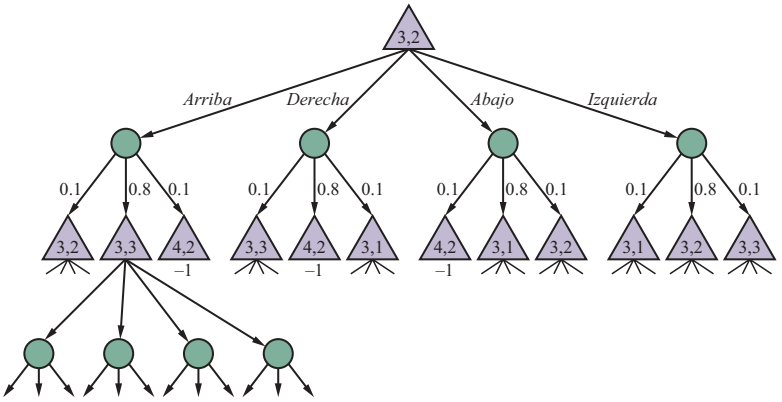


Figura 17.10 Parte de un árbol expectimax para el MDP 4 × 3 con raíz en (3,2). Los nodos triangulares son modos máximos y los nodos circulares son nodos de oportunidad.

El enfoque más sencillo es en realidad una simplificación del algoritmo EXPECTIMINIMAX para árboles de juegos con nodos de oportunidad: el algoritmo EXPECTIMAX construye un árbol de nodos máximos y de oportunidad alternos, como se ilustra en la Figura 17.10. (Hay una ligera diferencia con el EXPECTIMINIMAX estándar en que hay recompensas en las transiciones no terminales así como en las terminales). Una función de evaluación se aplica a las hojas no terminales del árbol, o se les da un valor predeterminado. Es posible extraer una decisión del árbol de búsqueda respaldando los valores de utilidad desde las hojas, tomando un promedio en los nodos de oportunidad y tomando el máximo en los nodos de decisión.

Para problemas en los que el factor de descuento γ no está demasiado cerca de 1, el ϵ-horizonte es un concepto útil. Dejemos que ϵ sea un límite deseado en el error absoluto en las utilidades calculadas a partir de un árbol de expectimax de profundidad limitada, en comparación con las utilidades exactas en el MDP. Entonces el ϵ-horizonte es la profundidad del árbol H tal que la suma de las recompensas más allá de cualquier hoja a esa profundidad es menor que ϵ; en términos generales, cualquier cosa que suceda después de H es irrelevante porque está muy lejos en el futuro. Debido a que la suma de las recompensas más allá de H está limitada por $\gamma^H R_{\max}/(1-\gamma)$, una profundidad de $H = \lceil \log_\gamma \epsilon(1-\gamma)/R_{\max} \rceil$ es suficiente. Entonces, construir un árbol hasta esta profundidad da decisiones casi óptimas. Por ejemplo, con $\gamma = 0.5$, $\epsilon = 0.1$, y $R_{\max} = 1$, encontramos $H = 5$, lo que parece razonable. Por otro lado, si $\gamma = 0.9$, $H = 44$, ¡lo que parece menos razonable!

Además de limitar la profundidad, también es posible evitar el factor de ramificación potencialmente enorme en los nodos de oportunidad. (Por ejemplo, si todas las probabilidades condicionales en un modelo de transición DBN son distintas de cero, las probabilidades de transición, que se dan por el producto de las probabilidades condicionales, también son distintas de cero, lo que significa que cada estado tiene *alguna* probabilidad de transición a cada otro estado).

Como se señaló en la Sección 13.4, las expectativas con respecto a una distribución de probabilidad P permiten ser aproximadas generando N muestras de P y utilizando la media de la muestra. En forma matemática, tenemos

$$\sum_x P(x)f(x) \approx \frac{1}{N}\sum_{i=1}^{N} f(x_i).$$

Entonces, si el factor de ramificación es muy grade, lo que significa que hay muchos valores de x posibles, es viable obtener una buena aproximación al valor del nodo de oportunidad muestreando

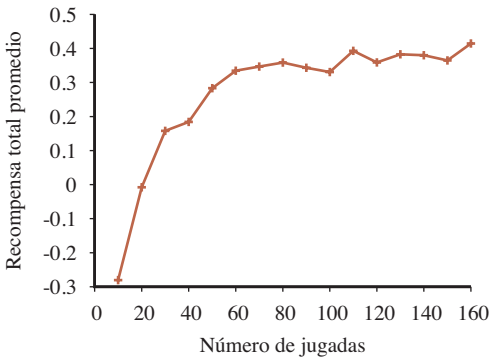


Figura 17.11 Rendimiento de UCT como función del número de jugadas por movimiento para el mundo 4 × 3 utilizando una política de jugadas aleatorias, promediado sobre 1 000 carreras por punto de datos.

un límite acotado de resultados de la acción. En la mayoría de los casos, las muestras se centrarán en los *más probables* resultados porque esos son los más asequibles de ser generados.

Si mira de cerca el árbol en la Figura 17.10, notará algo: no es realmente un árbol. Por ejemplo, la raíz (3,2) también es una hoja, por lo que deberíamos considerar esto como un grafo, y deberíamos restringir el valor de la hoja (3,2) para que sea el mismo que el valor de la raíz (3,2), ya que son el mismo estado. De hecho, esta línea de pensamiento nos lleva muy rápido de vuelta a las ecuaciones de Bellman las cuales relacionan los valores de los estados con los valores de los estados vecinos. Los estados explorados en realidad constituyen un sub-MDP del MDP original, y este sub-MDP permite ser resuelto utilizando cualquiera de los algoritmos de este capítulo para tomar una decisión para el estado actual. (Los estados fronterizos suelen tener un valor estimado fijo).

Programación dinámica en tiempo real (RTDP)

Este enfoque general se llama **programación dinámica en tiempo real (RTDP)** y es casi análogo a LRTA* en el Capítulo 4. Los algoritmos de este tipo pueden ser efectivos en dominios de tamaño moderado como los mundos de retícula; en dominios más grandes como Tetris, hay dos problemas. Primero, el espacio de estados es tal que cualquier conjunto manejable de estados explorados contiene muy pocos estados repetidos; en consecuencia, uno podría usar un simple árbol de expectimax. Segundo, una simple heurística para los nodos fronterizos puede no ser suficiente para guiar al agente, en especial si las recompensas son escasas.

Una posible solución es aplicar el aprendizaje por refuerzo para generar una heurística mucho más precisa (ver Capítulo 22). Otro enfoque es mirar más adelante en el MDP utilizando el enfoque de Monte Carlo de la Sección 5.4. De hecho, el algoritmo UCT de la Figura 5.10 fue desarrollado originalmente para MDPs y no para juegos. Los cambios requeridos para resolver MDPs en lugar de juegos son mínimos: surgen del hecho de que el oponente (la naturaleza) es estocástico y de la necesidad de llevar un registro de las recompensas en lugar de sólo las victorias y las pérdidas.

Cuando se aplica al mundo 4 × 3, el rendimiento de UCT no es en especial impresionante. Como muestra la Figura 17.11, se necesitan en promedio 160 jugadas para alcanzar una recompensa total de 0.4, mientras que una política óptima tiene una recompensa total esperada de 0.7453 desde el estado inicial (ver Figura 17.3). Una razón por la que UCT puede tener dificultades es que construye un árbol en lugar de un grafo y utiliza (una aproximación a) expectimax en lugar

de programación dinámica. El mundo 4 × 3 es "cíclico": aunque sólo hay 9 estados no terminales, las jugadas de UCT continúan durante más de 50 acciones.

UCT parece mejor adaptado para Tetris, donde las jugadas llegan lo suficientemente lejos en el futuro para dar al agente una idea de si una jugada arriesgada en potencia funcionará al final o causará un gran amontonamiento. El Ejercicio 17.UCTT explora la aplicación de UCT a Tetris. Una pregunta interesante es cuánto ayuda una política de simulación simple; por ejemplo, una que evita crear voladizos y coloca las piezas lo más bajo posible.

17.3 Problemas de bandido

En Las Vegas, un *bandido de un solo brazo* es una máquina tragamonedas. Un jugador puede insertar una moneda, tirar de la palanca y recoger las ganancias (si las hay). Un **bandido de *n* brazos** tiene *n* palancas. Detrás de cada palanca hay una distribución de probabilidad fija pero desconocida de ganancias; cada tirada muestra esa distribución desconocida.

Bandido de *n* brazos

El jugador debe elegir qué palanca jugar en cada moneda sucesiva: ¿la que ha dado mejores resultados o, tal vez, una que aún no se ha probado? Este es un ejemplo del compromiso omnipresente entre la **explotación** de la mejor acción actual para obtener recompensas y la **exploración** de estados y acciones previamente desconocidos para obtener información, que en algunos casos se convierte en una mejor política y mejores recompensas a largo plazo. En el mundo real, uno tiene que decidir de manera constante entre continuar en una existencia cómoda, versus aventurarse en lo desconocido con la esperanza de una vida mejor.

El problema del bandido de *n* brazos es un modelo formal para problemas reales en muchas áreas vitales, como decidir cuál *n* posibles nuevos tratamientos probar para curar una enfermedad, en cuál de *n* posibles inversiones poner parte de sus ahorros, cuál de *n* posibles proyectos de investigación financiar o cuál de *n* posibles anuncios mostrar cuando el usuario visita una página web en particular.

Los primeros trabajos sobre el problema comenzaron en Estados Unidos durante la Segunda Guerra Mundial; resultó tan recalcitrante que los científicos aliados propusieron que "el problema se dejara caer sobre Alemania, como el último instrumento de sabotaje intelectual" (Whittle, 1979).

Resulta que los científicos, durante y después de la guerra, trataban de probar "hechos verdaderos" sobre los problemas de los bandidos que son, de hecho, falsos. (Como Bradt *et al.* (1956) lo expresaron, "Hay muchas propiedades agradables que las estrategias óptimas no poseen"). Por ejemplo, se asumía en general que una política óptima de manera eventual se decidiría por el mejor brazo a largo plazo; de hecho, hay una probabilidad finita de que una política óptima se decida por un brazo subóptimo. Ahora tenemos una sólida comprensión teórica de los problemas de los bandidos, así como algoritmos útiles para resolverlos.

Hay múltiples definiciones diferentes de **problemas de bandidos**; una de las más limpias y más generales es la siguiente:

Problemas de bandidos

- Cada brazo M_i es un **proceso de recompensa de Markov** o MRP (por sus siglas en inglés); es decir, un MDP con una sola posible acción a_i. Tiene estados S_i, modelo de transición $P_i(s'|s,a_i)$, y recompensa $R_i(s,a_i,s')$. El brazo define una distribución sobre secuencias de recompensas $R_{i,0},R_{i,1},R_{i,2},...$, donde cada $R_{i,t}$ es una variable aleatoria.
- El problema general del bandido es un MDP: el espacio de estados está dado por el producto cartesiano $S = S_1 \times \cdots \times S_n$; las acciones son $a_1,...,a_n$; el modelo de transición actualiza el estado de cualquier brazo M_i que se selecciona, de acuerdo con su modelo de transición específico, dejando los otros brazos sin cambios, mientras que el factor de descuento es γ.

Proceso de recompensa de Markov

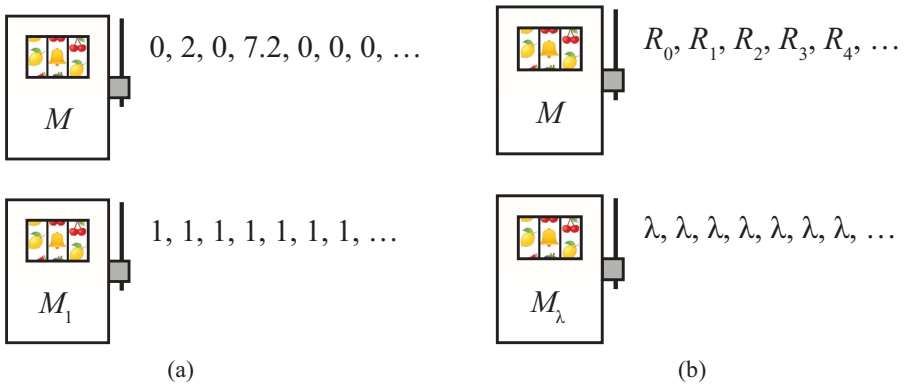


Figura 17.12 (a) Un problema determinista de bandido con dos brazos. Los brazos pueden ser tirados en cualquier orden, y cada uno produce la secuencia de recompensas mostrada. (b) Un caso más general del bandido en (a), donde el primer brazo da una secuencia arbitraria de recompensas y el segundo brazo da una recompensa fija λ.

Esta definición es muy general, pues cubre una amplia gama de casos. La propiedad clave es que los brazos son independientes, acoplados sólo por el hecho de que el agente puede trabajar en un solo brazo a la vez. Es posible definir una versión aún más general en la que es factible aplicar esfuerzos fraccionales a todos los brazos de manera simultánea, pero el esfuerzo total en todos los brazos está limitado; los resultados básicos descritos aquí se aplican a este caso.

Veremos en breve cómo formular un problema típico de bandido dentro de este marco, pero calentemos con el caso especial simple de secuencias de recompensas deterministas. Dejemos $\gamma = 0.5$, y supongamos que hay dos brazos etiquetados M y M_1. Tirar de M varias veces produce la secuencia de recompensas 0,2,0,7.2,0,0,..., mientras que tirar de M_1 produce 1,1,1,... (Figura 17.12(a)). Si al principio uno tuviera que comprometerse con un brazo u otro y quedarse con él, la elección se haría calculando la utilidad (recompensa total descontada) para cada brazo:

$$U(M) = (1.0 \times 0) + (0.5 \times 2) + (0.5^2 \times 0) + (0.5^3 \times 7.2) = 1.9$$

$$U(M_1) = \sum_{t=0}^{\infty} 0.5^t = 2.0.$$

Uno podría pensar que la mejor opción es ir con M_1, pero un momento más de reflexión muestra que comenzar con M y luego cambiar a M_1 después de la cuarta recompensa da la secuencia $S = 0,2,0,7.2,1,1,1,...$, para lo cual

$$U(S) = (1.0 \times 0) + (0.5 \times 2) + (0.5^2 \times 0) + (0.5^3 \times 7.2) + \sum_{t=4}^{\infty} 0.5^t = 2.025.$$

Por lo tanto, la estrategia S que cambia de M a M_1 en el momento adecuado es mejor que cualquier brazo trabajando de manera individual. De hecho, S es óptimo para este problema: todos los demás tiempos de cambio dan menos recompensa.

Generalicemos un poco este caso, de modo que ahora el primer brazo M produce una secuencia arbitraria $R_0,R_1,R_2,...$ (que puede ser conocido o desconocido) y el segundo brazo M_λ produce $\lambda,\lambda,\lambda,...$ para alguna constante fija conocida λ (ver Figura 17.12(b)). Esto se llama **un bandido de un solo brazo** en la literatura, porque es equivalente al caso en el que hay un brazo M que produce recompensas $R_0,R_1,R_2,...$ y cuesta λ por cada tirón. (Tirar del brazo M es equivalente a no tirar

Bandido de un solo brazo

de M_λ, por lo que renuncia a una recompensa de λ cada vez). Con un solo brazo, la única opción es si volver a tirar o parar. Si tira del primer brazo T veces (es decir, en los tiempos 0, 1,..., $T-1$) decimos que el **tiempo de parada es** T.

Tiempo de parada

Volviendo a nuestra versión con M y M_λ, suponga que después de T tiradas del primer brazo, una estrategia óptima tira del segundo brazo por primera vez. Como no se obtiene información de este movimiento (ya sabemos que la recompensa será λ), en el tiempo $T+1$ estaremos en la misma situación y, por lo tanto, una estrategia óptima debe hacer la misma elección.

De manera equivalente, podemos decir que una estrategia óptima es ejecutar el brazo M hasta el tiempo T y luego cambiar a M_λ por el resto del tiempo. Es posible que $T=0$ si la estrategia elige M_λ inmediatamente, o $T=\infty$ si la estrategia nunca elige M_λ, o en algún punto intermedio. Ahora consideremos el valor de λ como una estrategia óptima es *indiferente* entre (a) ejecutar M hasta el mejor tiempo de parada posible y luego cambiar a M_λ, y (b) elegir M_λ de manera inmediata. En el punto de inflexión tenemos

$$\max_{T>0} E\left[\left(\sum_{t=0}^{T-1}\gamma^t R_t\right)+\sum_{t=T}^{\infty}\gamma^t\lambda\right] = \sum_{t=0}^{\infty}\gamma^t\lambda,$$

que se simplifica a

$$\lambda = \max_{T>0}\frac{E\left(\sum_{t=0}^{T-1}\gamma^t R_t\right)}{E\left(\sum_{t=0}^{T-1}\gamma^t\right)}. \tag{17.15}$$

Esta ecuación define una especie de "valor" para M en términos de su capacidad para entregar una corriente de recompensas oportunas; el numerador de la fracción representa una utilidad mientras que el denominador es pensado como un "tiempo descontado", por lo que el valor describe la máxima utilidad obtenible por unidad de tiempo descontado. (Es importante recordar que T en la ecuación es un tiempo de parada, que está gobernado por una regla de parada en lugar de ser un simple entero; se reduce a un simple entero sólo cuando M es una secuencia de recompensas determinista). El valor definido en la ecuación (17.15) se llama **índice de Gittins** de M.

Índice de Gittins

Lo notable del índice de Gittins es que proporciona una política óptima muy simple para cualquier problema de bandido: *tira del brazo que tiene el índice de Gittins más alto, luego actualiza los índices de Gittins*. Además, porque el índice del brazo M_i depende sólo de las propiedades de ese brazo, una decisión óptima en la primera iteración se calcula en tiempo $O(n)$, donde n es el número de brazos. Y porque los índices de Gittins de los brazos que no son seleccionados permanecen sin cambios, cada decisión después de la primera se calcula en tiempo $O(1)$.

17.3.1 Calculando el índice de Gittins

Para tener una mejor idea del índice, calculemos el valor del numerador, denominador y relación en la Ecuación (17.15) para diferentes posibles tiempos de parada en la secuencia de recompensas determinista 0,2,0,7.2,0,0,0,...:

T	1	2	3	4	5	6
R_t	0	2	0	7.2	0	0
$\sum\gamma^t R_t$	0.0	1.0	1.0	1.9	1.9	1.9
$\sum\gamma^t$	1.0	1.5	1.75	1.875	1.9375	1.9687
relación	0.0	0.6667	0.5714	1.0133	0.9806	0.9651

Note que la relación disminuirá a partir de aquí porque el numerador permanece constante, mientras que el denominador continúa aumentando. Por lo tanto, el índice de Gittins para este brazo es

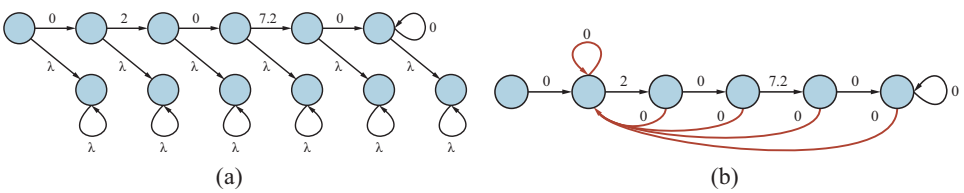


Figura 17.13 (a) La secuencia de recompensas $M = 0,2,0,7.2,0,0,0,...$ aumentada con una opción para cambiar de manera permanente a un brazo constante M_λ en cada punto. (b) Un MDP cuyo valor óptimo es equivalente al valor óptimo para (a), en el punto donde la política óptima es indiferente entre M y M_λ.

1.0133, el valor máximo alcanzado por la relación. En combinación con un brazo fijo M_λ con $0 < \lambda \leq 1.0133$, la política óptima recoge las primeras cuatro recompensas de M y luego cambia a M_λ. Para $\lambda > 1.0133$, la política óptima siempre elige M_λ.

Para calcular el índice de Gittins para un brazo general M con estado actual s, hacemos la siguiente observación: en el punto de inflexión donde una política óptima es indiferente entre elegir el brazo M y elegir el brazo fijo M_λ, el valor de elegir M es el mismo que el valor de elegir una secuencia infinita de recompensas λ.

Suponga que aumentamos M de manera que en cada estado en M el agente tiene dos opciones: continuar con M como antes o renunciar y recibir una secuencia infinita de recompensas λ (ver Figura 17.13(a)). Esto convierte a M en un MDP, cuya política óptima es la regla de parada óptima para M. Por lo tanto, el valor de una política óptima en este nuevo MDP es igual al valor de una secuencia infinita de recompensas λ, es decir, $\lambda/(1-\gamma)$. Así que podemos resolver este MDP... pero no conocemos el valor de λ para poner en el MDP, ya que esto es lo que estamos tratando de calcular. Pero sí sabemos que, en el punto de inflexión, una política óptima es indiferente entre M y M_λ, por lo que podríamos reemplazar la elección de obtener una secuencia infinita de recompensas λ con la elección de volver y reiniciar M desde su estado inicial s. (Más específico, añadimos una nueva acción en cada estado que tiene las mismas recompensas y resultados que la acción disponible en s; ver Ejercicio 17.KATV). Este nuevo MDP M^s, llamado un MDP de reinicio, se ilustra en la Figura 17.13(b).

Reiniciar MDP

Tenemos el resultado general de que el índice de Gittins para un brazo M en el estado s es igual a $1-\gamma$ veces el valor de una política óptima para el MDP de reinicio M^s. Este MDP es resuelto por cualquiera de los algoritmos en la Sección 17.2. La iteración de valor aplicada a M^s en la Figura 17.13(b) da un valor de 2.0266 para el estado inicial, por lo que tenemos $\lambda = 2.0266 \cdot (1-\gamma) = 1.0133$ como antes.

17.3.2 El bandido de Bernoulli

Bandido de Bernoulli

Quizás el ejemplo más simple y conocido de un problema de bandido es el **bandido de Bernoulli**, donde cada brazo M_i produce una recompensa de 0 o 1 con una probabilidad fija, pero desconocida μ_i. El estado del brazo M_i está definido por s_i y f_i, los conteos de éxitos ($1s$) y fracasos ($0s$) hasta ahora para ese brazo; la probabilidad de transición predice el próximo resultado a ser 1 con probabilidad $(s_i)/(s_i + f_i)$ y 0 con probabilidad $(f_i)/(s_i + f_i)$. Los conteos se inicializan en 1 para que las probabilidades iniciales sean 1/2 en lugar de 0/0.[4] El proceso de recompensa de Markov se muestra en la Figura 17.14(a).

[4] Las probabilidades son las de un proceso de actualización bayesiana con un prior Beta(1,1) (ver Sección 20.2.5).

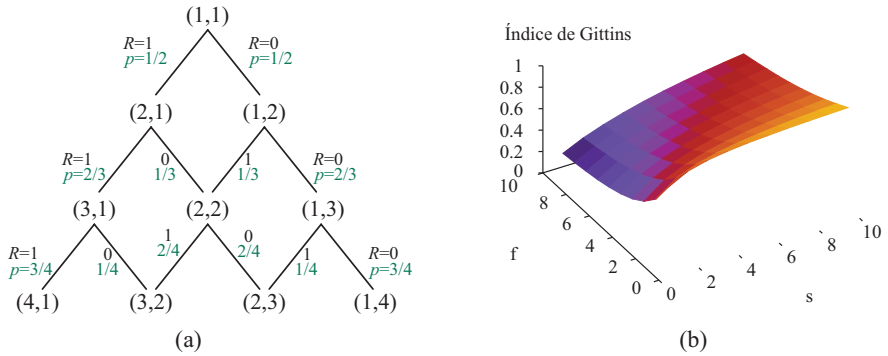


Figura 17.14 (a) Estados, recompensas y probabilidades de transición para el bandido de Bernoulli. (b) Índices de Gittins para los estados del proceso de bandido de Bernoulli.

No es posible aplicar la transformación de la sección anterior para calcular el índice de Gittins del brazo de Bernoulli porque tiene múltiples estados infinitos. Sin embargo, podemos obtener una aproximación muy precisa resolviendo el MDP truncado con estados hasta $s_i + f_i = 100$ y $\gamma = 0.9$. Los resultados se muestran en la Figura 17.14(b). Los resultados son intuitivamente razonables: vemos que, en general, se prefieren los brazos con mayores probabilidades de pago, pero también hay un **bono de exploración** asociado con los brazos que sólo se han probado unas pocas veces. Por ejemplo, el índice para el estado (3,2) es mayor que el índice para el estado (7,4) (0.7057 vs. 0.6922), a pesar de que el valor estimado en (3,2) es menor (0.6 vs. 0.6364).

Bono de exploración

17.3.3 Políticas de bandido aproximadamente óptimas

Calcular los índices de Gittins para problemas más realistas rara vez es fácil. Por fortuna, las propiedades generales observadas en la sección anterior, es decir, la conveniencia de alguna combinación de valor estimado e incertidumbre, se prestan a la creación de políticas simples que resultan ser "casi tan buenas" como las políticas óptimas.

La primera clase de métodos utiliza la heurística de **límite de confianza superior** o UCB, introducido con anterioridad para la búsqueda de árboles de Monte Carlo (Figura 5.11 en la página 163). La idea básica es usar las muestras de cada brazo para establecer un **intervalo de confianza** para el valor del brazo, es decir, un rango dentro del cual se estima que el valor se encuentra con alta confianza; luego elige el brazo con el límite superior más alto en su intervalo de confianza. El límite superior es la estimación actual del valor medio $\hat{\mu}_i$ más algún múltiplo de la desviación estándar de la incertidumbre en el valor. La desviación estándar es proporcional a $\sqrt{1/N_i}$, donde N_i es el número de veces que se ha muestreado el brazo M_i. Por lo tanto, tenemos un valor de índice aproximado para el brazo M_i el cual se define como:

Límite de confianza superior

Intervalo de confianza

$$\text{UCB}(M_i) = \hat{\mu}_i + g(N)/\sqrt{N_i},$$

donde $g(N)$ es una función elegida de N, el número total de muestras extraídas de todos los brazos. Una política de UCB elige el brazo con el valor de UCB más alto. Observe que el valor de UCB no es un índice porque depende de N, el número total de muestras extraídas de todos los brazos, y no sólo del brazo en sí.

La definición precisa de g determina el **arrepentimiento** en relación con la política clarividente, que elige el mejor brazo y produce una recompensa promedio μ^*. Un resultado famoso debido

a Lai y Robbins (1985) muestra que, para el caso no descontado, ningún algoritmo posible permite tener un arrepentimiento que crezca más lento que $O(\log N)$. Múltiples elecciones diferentes de g conducen a una política de UCB que coincide con este crecimiento; por ejemplo, podemos usar $g(N) = (2\log(1 + N\log^2 N))^{1/2}$.

Muestreo de Thompson

Un segundo método, el **muestreo de Thompson** (Thompson, 1933), elige un brazo de manera aleatoria según la probabilidad de que el brazo sea de hecho óptimo, dadas las muestras hasta ahora. Supongamos que $P_i(\mu_i)$ es la distribución de probabilidad actual para el valor verdadero del brazo M_i. Entonces, una forma sencilla de implementar el muestreo de Thompson es generar una muestra de cada P_i y luego elegir la mejor muestra. Este algoritmo también tiene un arrepentimiento que crece como $O(\log N)$.

17.3.4 Variantes no indexables

Los problemas de bandido se motivaron en parte por la tarea de probar nuevos tratamientos médicos en pacientes gravemente enfermos. Para esta tarea, el objetivo de maximizar el número total de éxitos a lo largo del tiempo tiene sentido: cada prueba exitosa significa una vida salvada, cada fracaso una vida perdida.

Sin embargo, si cambiamos las suposiciones surge un problema diferente. Suponga que, en lugar de determinar el mejor tratamiento médico para cada nuevo paciente humano, estamos probando diferentes medicamentos en muestras de bacterias con el objetivo de decidir qué medicamento es el mejor. Luego pondremos ese medicamento en producción y renunciaremos a los demás. En este escenario, no hay un costo adicional si la bacteria muere, hay un costo fijo por cada prueba, pero no tenemos que minimizar los fracasos de las pruebas; más bien, estamos tratando de tomar una buena decisión lo más rápido posible.

Problema de selección

La tarea de elegir la mejor opción bajo estas condiciones se llama **problema de selección**. Los problemas de selección son ubicuos en contextos industriales y de personal. Con frecuencia se debe decidir qué proveedor utilizar para un proceso o qué empleados candidatos contratar. Los problemas de selección son similares al problema del bandido pero tienen diferentes propiedades matemáticas. En particular, *no existe una función de índice para los problemas de selección*. La prueba de este hecho requiere mostrar cualquier escenario donde la política óptima cambie sus preferencias por dos brazos M_1 y M_2 cuando se agrega un tercer brazo M_3 (ver Ejercicio 17.SELC).

El Capítulo 5 introdujo el concepto de problemas de decisión a **metanivel**, como decidir qué cálculos hacer durante una búsqueda de árbol de juego antes de hacer un movimiento. Una decisión a metanivel de este tipo también es un problema de selección en lugar de un problema de bandido. Note que una expansión o evaluación de nodo *cuesta* la misma cantidad de tiempo, ya sea que produzca un valor de salida alto o bajo. Quizá sea sorprendente, entonces, que el algoritmo de búsqueda de árboles de Monte Carlo (ver página 163) haya tenido tanto éxito, dado que intenta resolver problemas de selección con la heurística de UCB, que fue diseñada para problemas de bandido. En general, se espera que los algoritmos de bandido óptimos exploren mucho menos que los algoritmos de selección óptimos, porque el algoritmo de bandido asume que un ensayo fallido cuesta dinero real.

Superproceso de bandido o BSP

Una generalización importante del proceso de bandido es el **superproceso de bandido** o **BSP**, en el que cada brazo es un proceso de decisión de Markov completo en sí mismo, en lugar de ser un proceso de recompensa de Markov con sólo una acción posible. Todas las demás propiedades permanecen iguales: los brazos son independientes, sólo uno (o un número limitado) permite ser trabajado a la vez, y hay un sólo factor de descuento.

Ejemplos de BSPs incluyen la vida diaria, donde sólo se permite atender una tarea a la vez, aunque múltiples tareas necesiten atención; gestión de proyectos con múltiples proyectos; enseñanza con múltiples alumnos que necesitan orientación individual, etc. El término ordinario para esto es **multitarea**. Es tan ubicuo que apenas se nota: cuando se formula un problema de decisión

Multitarea

del mundo real, los analistas de decisiones rara vez preguntan si su cliente tiene otros problemas no relacionados.

Uno podría razonar de la siguiente manera: "Si hay n MDPs disjuntos, entonces es obvio que una política óptima en general se construye a partir de las soluciones óptimas de los MDPs individuales. Dada su política óptima π_i, cada MDP se convierte en un proceso de recompensa de Markov donde sólo hay una acción $\pi_i(s)$ en cada estado s. Entonces hemos reducido el superproceso de bandido de n brazos a un proceso de bandido de n brazos". Por ejemplo, si un promotor inmobiliario tiene un equipo de construcción y varios centros comerciales para construir, parece ser sólo de sentido común que uno debería idear el plan de construcción óptimo para cada centro comercial y luego resolver el problema del bandido para decidir a dónde enviar el equipo cada día.

Aunque esto suena muy plausible, es incorrecto. De hecho, la política óptima global para un BSP permite incluir acciones que son subóptimas de manera local desde el punto de vista del MDP constituyente en el que se toman. La razón de esto es que la disponibilidad de otros MDPs en los que actuar cambia el equilibrio entre las recompensas a corto y largo plazo en un MDP componente. De hecho, tiende a llevar un comportamiento más codicioso en cada MDP (buscando recompensas a corto plazo) porque apuntar a una recompensa a largo plazo en un MDP retrasaría las recompensas en todos los demás MDPs.

Por ejemplo, supongamos que el programa de construcción óptimo de manera local para un centro comercial tiene la primera tienda disponible para alquilar en la semana 15, mientras que un programa subóptimo cuesta más, pero tiene la primera tienda disponible en la semana 5. Si hay cuatro centros comerciales para construir, podría ser mejor usar el programa subóptimo local en cada uno para que los alquileres comiencen a llegar en las semanas 5, 10, 15 y 20, en lugar de las semanas 15, 30, 45 y 60. En otras palabras, lo que sería sólo un retraso de 10 semanas para un único MDP se convierte en un retraso de 40 semanas para el cuarto MDP. En general, las políticas óptimas global y local necesariamente coinciden sólo cuando el factor de descuento es 1; en ese caso, no hay costo por retrasar las recompensas en ningún MDP.

La siguiente pregunta es cómo resolver los BSPs. Note que la solución óptima global para un BSP podría calcularse convirtiéndola en un MDP global en el espacio de estados del producto cartesiano. El número de estados sería exponencial en el número de brazos del BSP, por lo que esto sería impráctico.

En cambio, podemos aprovechar la naturaleza laxa de la interacción entre los brazos. Esta interacción surge sólo de la capacidad limitada del agente para atender los brazos de manera simultánea: cuánta utilidad se renuncia por paso de tiempo al no dedicar ese paso de tiempo a otro brazo. Cuanto mayor es el **costo de oportunidad**, más necesario es generar recompensas tempranas en un brazo dado. En algunos casos, una política óptima en un brazo dado no se ve afectada por el costo de oportunidad. (Esto es cierto en un proceso de recompensa de Markov porque sólo hay una política). En ese caso, es posible aplicar una política óptima convirtiendo ese brazo en un proceso de recompensa de Markov.

Costo de oportunidad

Tal política óptima, si existe, se llama **política dominante**. Resulta que al agregar acciones a los estados, siempre es posible crear una versión relajada de un MDP (ver Sección 3.6.2) para que tenga una política dominante, que así da un límite superior al valor de actuar en el brazo. Es posible calcular un límite inferior resolviendo cada brazo por separado (lo que permite dar lugar a una política subóptima en general) y luego calculando los índices de Gittins. Si el límite inferior para actuar en un brazo es mayor que los límites superiores para todas las demás acciones, entonces el problema está resuelto; si no, entonces una combinación de búsqueda anticipada y recomputación de límites está garantizada para identificar una política óptima para el BSP. Con este enfoque, es posible resolver BSPs relativamente grandes (10^{40} estados o más) en unos pocos segundos.

Política dominante

17.4 MDPs parcialmente observables

La descripción de los procesos de decisión de Markov en la Sección 17.1 asumió que el entorno era **completamente observable**. Con esta suposición, el agente siempre sabe en qué estado se encuentra. Esto, combinado con la suposición de Markov para el modelo de transición, significa que la política óptima depende sólo del estado actual.

Cuando el entorno es sólo **parcialmente observable**, la situación es, se podría decir, mucho menos clara. El agente no sabe en qué estado se encuentra, por lo que no permite ejecutar la acción $\pi(s)$ recomendada para ese estado. Además, la utilidad de un estado s y la acción óptima en s dependen no sólo de s, sino también de *cuánto sabe el agente* cuando está en s. Para estas razones, los **MDP parcialmente observables** (o POMDPs, pronunciados "pom-dee-pees") suelen considerarse mucho más difíciles que los MDP ordinarios. Sin embargo, no podemos evitar los POMDPs, porque el mundo real es uno.

MDP parcialmente observable

17.4.1 Definición de POMDPs

Para entender los POMDPs, primero debemos definirlos de manera correcta. Un POMDP tiene los mismos elementos que un MDP: el modelo de transición $P(s'|s,a)$, acciones $A(s)$, y la función de recompensa $R(s,a,s')$; pero, al igual que los problemas de búsqueda parcialmente observables de la Sección 4.4, también tiene un **modelo de sensor** $P(e|s)$. Aquí, como en el Capítulo 14, el modelo de sensor especifica la probabilidad de percibir evidencia e en el estado s.[5] Por ejemplo, podemos convertir el mundo 4×3 de la Figura 17.1 en un POMDP añadiendo un sensor ruidoso o parcial en lugar de suponer que el agente conoce su ubicación. Se podría utilizar el sensor de cuatro bits ruidoso de la página 476, que informa de la presencia o ausencia de una pared en cada dirección cardinal con una precisión de $1 - \epsilon$.

Al igual que con los MDPs, podemos obtener representaciones compactas para grandes POMDPs utilizando redes de decisión dinámicas (ver Sección 17.1.4). Añadimos variables de sensor $\mathbf{E}_t$, asumiendo que las variables de estado $\mathbf{X}_t$ no permiten ser directamente observables. El modelo de sensor POMDP se da entonces por $\mathbf{P}(\mathbf{E}_t|\mathbf{X}_t)$. Por ejemplo, podríamos añadir variables de sensor a la DDN en la Figura 17.4 como $MedidorDeBatería_t$ para estimar la carga real *Batería* y *Velocímetro* para estimar la magnitud del vector de velocidad $\dot{\mathbf{X}}_t$. Un sensor de sonar $Muros_t$ podría dar distancias estimadas a la pared más cercana en cada una de las cuatro direcciones cardinales relativas a la orientación actual del robot; estos valores dependen de la posición y orientación actual $\mathbf{X}_t$.

En los Capítulos 4 y 11, estudiamos problemas de planificación no deterministas y parcialmente observables e identificamos el **estado de creencia** –el conjunto de estados reales en los que podría estar el agente– como un concepto clave para describir y calcular soluciones. En los POMDPs, el estado de creencia b se convierte en una *distribución de probabilidad* sobre todos los posibles estados, al igual que en el Capítulo 14. Por ejemplo, el estado de creencia inicial para el POMDP 4×3 podría ser la distribución uniforme sobre los nueve estados no terminales junto con $0s$ para los estados terminales, es decir, $\langle \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, 0, 0 \rangle$.

Usamos la notación $b(s)$ para referirnos a la probabilidad asignada al estado real s por el estado de creencia b. El agente permite calcular su estado de creencia actual como la distribución de probabilidad condicional sobre los estados reales dada la secuencia de percepciones y acciones hasta ahora. Esto es en esencia la tarea de **filtrado** descrita en el Capítulo 14. La ecuación básica de filtrado recursivo (14.5 en la página 467) muestra cómo calcular el nuevo estado de creencia a partir del estado de creencia anterior y la nueva evidencia. Para los POMDPs, también tenemos una acción

[5] El modelo de sensor también depende de la acción y del estado de resultado, pero este cambio no es fundamental.

a considerar, pero el resultado es en esencia el mismo. Si b era el estado de creencia anterior, y el agente realiza la acción a y luego percibe evidencia e, entonces el nuevo estado de creencia se obtiene calculando la probabilidad de estar ahora en el estado s', para cada s', con la siguiente fórmula:

$$b'(s') = \alpha P(e|s') \sum_s P(s'|s,a)b(s),$$

donde α es una constante de normalización que hace que el estado de creencia sume a 1. Por analogía con el operador de actualización para el filtrado (página 467), podemos escribir esto como

$$b' = \alpha \textsc{Forward}(b,a,e). \tag{17.16}$$

En el POMDP 4 × 3, supongamos que el agente se mueve a la *Izquierda* y su sensor informa de una pared adyacente; entonces, es muy probable (aunque no garantizado, porque tanto el movimiento como el sensor son ruidosos) que el agente esté ahora en (3,1). El Ejercicio 17.POMD le pide que calcule los valores de probabilidad exactos para el nuevo estado de creencia.

La idea fundamental necesaria para entender los POMDPs es esta: *la acción óptima depende sólo del estado de creencia actual del agente*. Es decir, una política óptima permite describirse por un mapeo $\pi^*(b)$ de estados de creencia a acciones. *No* depende del estado *real* en el que se encuentra el agente. Esto es bueno, porque el agente no conoce su estado real; todo lo que sabe es el estado de creencia. Por lo tanto, el ciclo de decisión de un agente POMDP puede desglosarse en los siguientes tres pasos:

1. Dado el estado de creencia actual b, ejecute la acción $a = \pi^*(b)$.
2. Observe la percepción e.
3. Establezca el estado de creencia actual en $\textsc{Forward}(b,a,e)$ y repita.

Podemos pensar en los POMDPs como que requieren una búsqueda en el espacio de estado de creencia, al igual que los métodos para problemas sin sensores y de contingencia en el Capítulo 4. La principal diferencia es que el espacio de estado de creencia de POMDP es *continuo*, porque un estado de creencia de POMDP es una distribución de probabilidad. Por ejemplo, un estado de creencia para el mundo 4 × 3 es un punto en un espacio continuo de 11 dimensiones. Una acción cambia el estado de creencia, no sólo el estado físico, porque afecta la percepción que se recibe. Así, la acción se evalúa al menos en parte de acuerdo con la información que el agente adquiere como resultado. Por lo tanto, los POMDPs incluyen el valor de la información (Sección 16.6) como un componente del problema de decisión.

Observemos el resultado de las acciones. En particular, calculemos la probabilidad de que un agente en el estado de creencia b llegue al estado de creencia b' después de ejecutar la acción a. Ahora, si conociéramos la acción y la percepción subsiguiente, entonces la Ecuación (17.16) proporcionaría una actualización determinista al estado de creencia: $b' = \textsc{Forward}(b,a,e)$. Por supuesto, la percepción subsiguiente aún no se conoce, por lo que el agente podría llegar a uno de varios estados de creencia posibles b', dependiendo de la percepción que se reciba. La probabilidad de percibir e, dado que se realizó a comenzando en el estado de creencia b, se da sumando todos los estados reales s' a los que podría llegar el agente:

$$\begin{aligned} P(e|a,b) &= \sum_{s'} P(e|a,s',b)P(s'|a,b) \\ &= \sum_{s'} P(e|s')P(s'|a,b) \\ &= \sum_{s'} P(e|s') \sum_s P(s'|s,a)b(s). \end{aligned}$$

Escribamos la probabilidad de llegar a b' desde b, dada la acción a, como $P(b'|b,a)$. Esta probabilidad se calcula de la siguiente manera:

$$P(b'|b,a) = \sum_e P(b'|e,a,b)P(e|a,b)$$
$$= \sum_e P(b'|e,a,b) \sum_{s'} P(e|s') \sum_s P(s'|s,a)b(s), \quad (17.17)$$

donde $P(b'|e,a,b)$ es 1 si $b' = \text{FORWARD}(b,a,e)$ y 0 en caso contrario.

La Ecuación (17.17) define un modelo de transición para el espacio de estado de creencia. También podemos definir una función de recompensa para las transiciones de estado de creencia, que se deriva de la recompensa esperada de las transiciones de estado reales que podrían estar ocurriendo. Aquí, usamos la forma simple $\rho(b,a)$, la recompensa esperada si el agente hace a en el estado de creencia b:

$$\rho(b,a) = \sum_s b(s) \sum_{s'} P(s'|s,a)R(s,a,s').$$

Juntos, $P(b'|b,a)$ y $\rho(b,a)$ definen un MDP observable en el espacio de estados de creencia. Además, es posible demostrar que una política óptima para este MDP, $\pi^*(b)$ también es una política óptima para el POMDP original. En otras palabras, *resolver un POMDP en un espacio de estado físico se reduce a resolver un MDP en el espacio de estado de creencia correspondiente*. Este hecho es quizá menos sorprendente si recordamos que el estado de creencia siempre es observable para el agente, por definición.

17.5 Algoritmos para resolver POMDPs

Hemos mostrado cómo reducir POMDPs a MDPs, pero los MDPs que obtenemos tienen un espacio de estado continuo (y, en general, de alta dimensión). Esto significa que tendremos que rediseñar los algoritmos de programación dinámica de las Secciones 17.2.1 y 17.2.2, que asumían un espacio de estado finito y un número finito de acciones. Aquí describimos un algoritmo de iteración de valor diseñado de manera específica para POMDPs, seguido de un algoritmo de toma de decisiones en línea similar a los desarrollados para juegos en el Capítulo 5.

17.5.1 Iteración de valor para POMDPs

La Sección 17.2.1 describió un algoritmo de iteración de valor que calculaba un valor de utilidad para cada estado. Con múltiples estados infinitos de creencia, necesitamos ser más creativos. Considere una política óptima π^* y su aplicación en un estado de creencia específico b: la política genera una acción, luego, para cada percepción subsiguiente, el estado de creencia se actualiza y se genera una nueva acción, y así sucesivamente. Por lo tanto, para este b específico la política es equivalente a un **plan condicional**, como se definió en el Capítulo 4 para problemas no deterministas y parcialmente observables. En lugar de pensar en políticas, pensemos en planes condicionales y cómo varía la utilidad esperada de ejecutar un plan condicional fijo con el estado de creencia inicial. Hacemos dos observaciones:

1. Se define la utilidad de ejecutar un plan condicional *fijo* p comenzando en el estado físico s como $\alpha_p(s)$. Entonces la utilidad esperada de ejecutar p en el estado de creencia b es sólo $\sum_s b(s)\alpha_p(s)$ o $b \cdot \alpha_p$ si consideramos ambos como vectores. Por lo tanto, la utilidad esperada de un plan condicional fijo varía de *manera lineal* con b; es decir, corresponde a un hiperplano en el espacio de creencia.
2. En cualquier estado de creencia dado b, una política óptima elegirá ejecutar el plan condicional con la mayor utilidad esperada; y la utilidad esperada de b bajo una política óptima

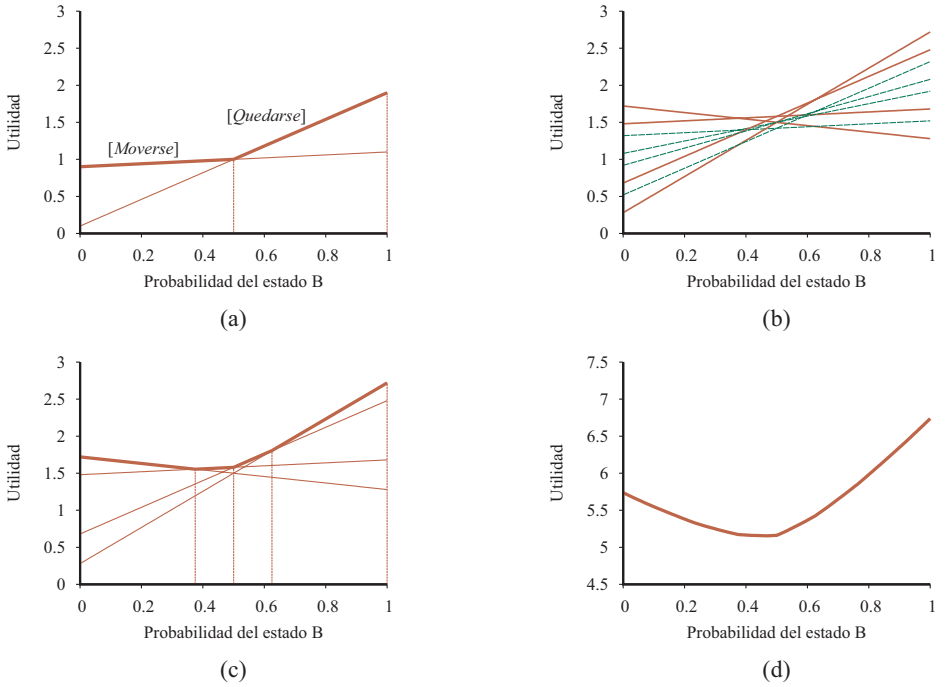


Figura 17.15 (a) Utilidad de dos planes de un paso como función del estado de creencia inicial $b(\boldsymbol{B})$ para el mundo de dos estados, con la función de utilidad correspondiente mostrada en negrita. (*b*) Utilidades para 8 planes distintos de dos pasos. (c) Utilidades para cuatro planes de dos pasos no dominados. (d) Función de utilidad para planes óptimos de ocho pasos.

es sólo la utilidad de ese plan condicional: $U(b) = U^{\pi^*}(b) = \max_p b \cdot \alpha_p$. Si una política óptima π^* elige ejecutar p comenzando en b, entonces es razonable esperar que pueda elegir ejecutar p en estados de creencia que están muy cerca de b; de hecho, si limitamos la profundidad de los planes condicionales, sólo hay múltiples planes finitos y, en general, el espacio continuo de estados de creencia se dividirá en *regiones*, cada una correspondiente a un plan condicional particular que es óptimo en esa región.

A partir de estas dos observaciones, vemos que la función de utilidad $U(b)$ en los estados de creencia, siendo el máximo de una colección de hiperplanos, será *lineal a trozos* y *convexa*.

Para ilustrar esto, usamos un escenario simple de dos estados. Los estados están etiquetados como A y B y hay dos acciones: *Quedarse* con probabilidad 0.9 y *Moverse* cambia al otro estado con probabilidad 0.9. Las recompensas son $R(\cdot,\cdot,A) = 0$ y $R(\cdot,\cdot,B) = 1$; es decir, cualquier transición que termine en A tiene recompensa cero y cualquier transición que termine en B tiene recompensa 1. Por ahora asumiremos que el factor de descuento $\boldsymbol{\gamma} = 1$ y el sensor informa el estado correcto con probabilidad 0.6. Note que el agente debe *Quedarse* cuando está en el estado B y *Moverse* cuando está en el estado A. ¡El problema es que no sabe dónde está!

La ventaja de un mundo de dos estados es que el espacio de creencia se visualiza en una dimensión, porque las dos probabilidades $b(\boldsymbol{A})$ y $b(\boldsymbol{B})$ suman 1. En la Figura 17.15(a), el eje x representa el estado de creencia, definido por $b(\boldsymbol{B})$, la probabilidad de estar en el estado $\boldsymbol{B}$. Ahora

consideremos los planes de un paso [*Quedarse*] y [*Moverse*], cada uno de los cuales recibe la recompensa por una transición de la siguiente manera:

$$\begin{aligned}
\alpha_{[Quedarse]}(A) &= 0.9R(A,Quedarse,A) + 0.1R(A,Quedarse,B) = 0.1 \\
\alpha_{[Quedarse]}(B) &= 0.1R(B,Quedarse,A) + 0.9\mathrm{R}(B,Quedarse,B) = 0.9 \\
\alpha_{[Moverse]}(A) &= 0.1R(A,Moverse,A) + 0.9R(A,Moverse,B) = 0.9 \\
\alpha_{[Moverse]}(B) &= 0.9R(B,Moverse,A) + 0.1R(B,Moverse,B) = 0.1
\end{aligned}$$

Los hiperplanos (líneas, en este caso) para $b\cdot\alpha_{[Quedarse]}$ y $b\cdot\alpha_{[Moverse]}$ se muestran en la Figura 17.15(a) y su máximo se muestra en negrita. La línea en negrita, por lo tanto, representa la función de utilidad para el problema de horizonte finito que permite sólo una acción y, en cada "pieza" de la función de utilidad lineal a trozos, una acción óptima es la primera acción del plan condicional correspondiente. En este caso, la política óptima de un sólo paso es *Quedarse* cuando $b(\boldsymbol{B}) > 0.5$ y *Moverse* en caso contrario.

Una vez que tenemos utilidades $\alpha_p(s)$ para todos los planes condicionales p de profundidad 1 en cada estado físico estado s, podemos calcular las utilidades para los planes condicionales de profundidad 2 considerando cada primera acción posible, cada percepción subsecuente posible y, luego, cada forma de elegir un plan de profundidad 1 para ejecutar para cada percepción:

[*Quedarse*; **if** *Percepción* = *A* **then** *Quedarse* **else** *Quedarse*]
[*Quedarse*; **if** *Percepción* = *A* **then** *Quedarse* **else** *Moverse*]
[*Moverse*; **if** *Percepción* = *A* **then** *Quedarse* **else** *Quedarse*]
…

Hay ocho planes distintos de profundidad 2 en total, sus utilidades se muestran en la Figura 17.15(b). Observe que cuatro de los planes, mostrados como líneas discontinuas, son subóptimos en todo el espacio de creencias; decimos que estos planes están **dominados** y no necesitan ser considerados más. Hay cuatro planes no dominados, cada uno de los cuales es óptimo en una región específica, como se muestra en la Figura 17.15(c). Las regiones dividen el espacio del estado de creencia.

Plan dominado

Repetimos el proceso para la profundidad 3, etc. En general, dejemos que p sea un plan condicional de profundidad d cuya acción inicial es a y cuyo subplan de profundidad, $(d-1)$, para la percepción e es $p.e$; entonces

$$\alpha_p(s) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \sum_{e} P(e|s')\alpha_{p.e}(s')]. \tag{17.18}$$

Esta recursión nos da un algoritmo de iteración de valor, que se da en la Figura 17.16. La estructura del algoritmo y su análisis de error son similares a los del algoritmo básico de iteración de valor en la Figura 17.6, página 573; la principal diferencia es que en lugar de calcular un número de utilidad para cada estado, POMDP-Value-Iteration mantiene una colección de planes no dominados con sus hiperplanos de utilidad.

La complejidad del algoritmo depende de cuántos planes se generan. Dado $|A|$ acciones y $|E|$ posibles observaciones, hay $|A|^{O(|E|d-1)}$ planes de profundidad d distintos. Incluso para el humilde mundo de dos estados con $d = 8$, eso son 2^{255} planes. La eliminación de planes dominados es esencial para reducir este crecimiento doblemente exponencial: el número de planes no dominados con $d = 8$ es sólo 144. La función de utilidad para estos 144 planes se muestra en la Figura 17.15(d).

Observe que los estados de creencia intermedios tienen un valor inferior al estado *A* y al estado *B*, porque en los estados intermedios el agente carece de la información necesaria para elegir una buena acción. Esto es porque la información tiene valor en el sentido definido en la Sección 16.6 y las políticas óptimas en POMDPs con frecuencia incluyen acciones de recopilación de información.

function POMDP-VALUE-ITERATION($pomdp$,ϵ) **returns** una función de utilidad
  inputs: $pomdp$, un POMDP con estados S, acciones $A(s)$, modelo de transición $P(s'|s,a)$,
    modelo de sensor $P(e|s)$, recompensa $R(s)$, descuento γ
    ϵ, el error máximo permitido en la utilidad de cualquier estado
  local variables: U, U', conjuntos de planes p con vectores de utilidad asociados α_p

  $U' \leftarrow$ a un conjunto que contiene sólo el plan vacío [ ], con $\alpha_{[\,]}(s) = R(s)$
  repeat
    $U \leftarrow U'$
    $U' \leftarrow$ el conjunto de todos los planes que consisten en una acción y, para cada posible próxima percepción, un plan en U con vectores de utilidad calculados de acuerdo con la Ecuación (17.18)
    $U' \leftarrow$ REMOVE-DOMINATED-PLANS(U')
  until MAX-DIFFERENCE(U,U') $\leq \epsilon(1-\gamma)/\gamma$
  return U

Figura 17.16 Un bosquejo de alto nivel del algoritmo de iteración de valor para POMDPs. El paso REMOVE-DOMINATED-PLANS y la prueba MAX-DIFFERENCE suelen implementarse como programas lineales.

Dada tal función de utilidad, es posible extraer una política ejecutable observando qué hiperplano es óptimo en cualquier estado de creencia b y ejecutando la primera acción del plan correspondiente. En la Figura 17.15(d), la política óptima correspondiente sigue siendo la misma que para los planes de profundidad-1: *Quedarse* cuando $b(B) > 0.5$ y, de lo contrario, *Moverse*.

En la práctica, el algoritmo de iteración de valor en la Figura 17.16 es ineficiente para problemas más grandes, incluso el POMDP de 4 × 3 es demasiado difícil. La razón principal es que dados n planes condicionales no dominados en el nivel d, el algoritmo construye $|A| \cdot n^{|E|}$ planes condicionales en el nivel $d + 1$ antes de eliminar los dominados. Con el sensor de cuatro bits, $|E|$ es 16, y n puede estar en los cientos, por lo que esto es desesperado.

Desde que se desarrolló este algoritmo en la década de 1970, ha habido varios avances, incluyendo formas más eficientes de iteración de valor y varios tipos de algoritmos de iteración de políticas. Algunos de estos se discuten en las notas al final del capítulo. Para los POMDPs generales, sin embargo, encontrar políticas óptimas es muy difícil (PSPACE-hard, es decir, muy difícil de verdad). La siguiente sección describe un método diferente y aproximado para resolver POMDPs, uno basado en la búsqueda de anticipación.

17.5.2 Algoritmos online para POMDPs

El diseño básico para un agente POMDP online es sencillo: comienza con algún estado de creencia previo; elige una acción basada en algún proceso de deliberación centrado en su estado de creencia actual; después de actuar, recibe una observación y actualiza su estado de creencia utilizando un algoritmo de filtrado; y el proceso se repite.

Una elección obvia para el proceso de deliberación es el algoritmo expectimax de la Sección 17.2.4, excepto con estados de creencia en lugar de estados físicos como los nodos de decisión en el árbol. Los nodos de oportunidad en el árbol POMDP tienen ramas etiquetadas por posibles observaciones y que conducen al siguiente estado de creencia, con probabilidades de transición dadas por la Ecuación (17.17). Un fragmento del árbol expectimax del estado de creencia para el POMDP de 4 × 3 se muestra en la Figura 17.17.

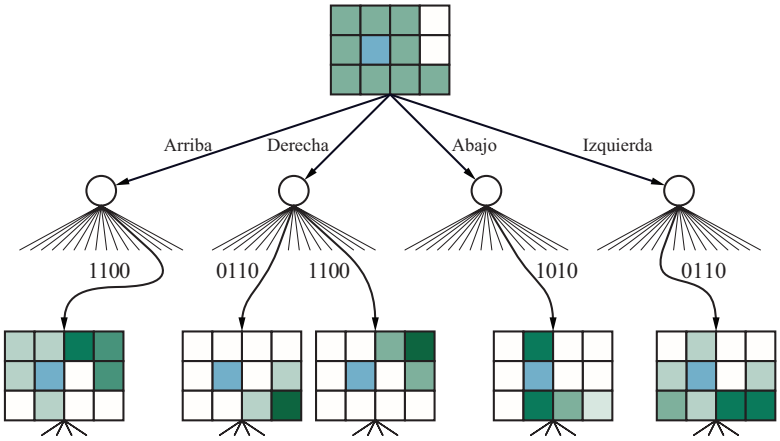


Figura 17.17 Parte de un árbol expectimax para el POMDP 4 × 3 con un estado de creencia inicial uniforme. Los estados de creencia se representan con sombreado proporcional a la probabilidad de estar en cada ubicación.

La complejidad temporal de una búsqueda exhaustiva a profundidad d es $O(|A|^d \cdot |\mathbf{E}|^d)$, donde $|A|$ es el número de acciones disponibles y $|\mathbf{E}|$ es el número de percepciones posibles. (Note que esto es mucho menos que el número de posibles planes condicionales de profundidad-d generados por el valor iteración). Como en el caso observable, el muestreo en los nodos de oportunidad es una buena manera de reducir el factor de ramificación sin perder demasiada precisión en la decisión final. Por lo tanto, la complejidad de la toma de decisiones en línea aproximada en POMDPs puede no ser peor que en MDPs.

Para espacios de estado muy grandes, el filtrado exacto es inviable, por lo que el agente necesitará ejecutar un algoritmo de filtrado aproximado como el filtrado de partículas (ver página 492). Entonces, los estados de creencia en el árbol expectimax se convierten en colecciones de partículas en lugar de distribuciones de probabilidad exactas. Para problemas con horizontes largos, también podemos necesitar ejecutar el tipo de reproducciones a largo plazo usadas en el algoritmo UCT (Figura 5.11). La combinación de filtrado de partículas y UCT aplicada a POMDPs se conoce como planificación de Monte Carlo parcialmente observable o **POMCP**. Con una representación DDN para el modelo, el algoritmo POMCP es, al menos en principio, aplicable a POMDPs muy grandes y realistas. Los detalles del algoritmo se exploran en el Ejercicio 17.POMC. POMCP es capaz de generar un comportamiento competente en el POMDP de 4 × 3. Un ejemplo corto (y algo afortunado) se muestra en la Figura 17.18.

POMCP

Los agentes POMDP basados en redes de decisión dinámica y toma de decisiones en línea tienen una serie de ventajas si los comparamos con otros diseños de agentes más simples presentados en capítulos anteriores. En particular, manejan entornos estocásticos parcialmente observables y permiten revisar sus "planes" para manejar evidencias inesperadas. Con modelos de sensores apropiados, permiten manejar fallas de sensores y pueden planificar para recopilar información. Muestran una "degradación elegante" bajo presión de tiempo y en entornos complejos, utilizando varias técnicas de aproximación.

¿Entonces qué falta? El principal obstáculo para la implementación en el mundo real de tales agentes es la incapacidad para generar un comportamiento exitoso a largo plazo. Las reproducciones aleatorias o casi aleatorias no tienen ninguna esperanza de obtener ninguna recompensa

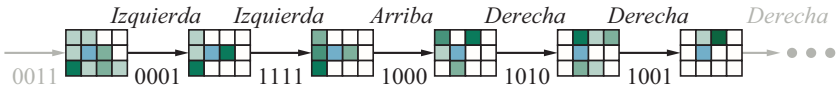


Figura 17.18 Una secuencia de percepciones, estados de creencia y acciones en el POMDP 4 × 3 con un error de detección de pared de $\epsilon = 0.2$. Note cómo los primeros movimientos a la *izquierda* son seguros, es muy poco probable que caigan en (4,2) y coaccionan la ubicación del agente en un pequeño número de posibles ubicaciones. Después de moverse hacia *arriba*, el agente piensa que probablemente está en (3,3), aunque quizás en (1,3). Por fortuna, moverse a la *derecha* es una buena idea en ambos casos, por lo que se mueve a la *derecha*, descubre que había estado en (1,3) y ahora está en (2,3), y luego continúa moviéndose a la *derecha* y alcanza la meta.

positiva en, digamos, la tarea de poner la mesa para la cena, lo que podría requerir decenas de millones de acciones de control motor. Parece necesario tomar prestadas algunas de las ideas de planificación jerárquica descritas en la Sección 11.4. En el momento de escribir, aún no existen formas satisfactorias y eficientes de aplicar estas ideas en entornos estocásticos, parcialmente observables.

Resumen

Este capítulo muestra cómo usar el conocimiento sobre el mundo para tomar decisiones aun cuando los resultados de una acción son inciertos y las recompensas por actuar podrían no cosecharse hasta que hayan pasado muchas acciones. Los puntos principales son los siguientes:

- Los problemas de decisión secuencial en entornos estocásticos, también llamados **procesos de decisión de Markov**, o MDPs, están definidos por un **modelo de transición** que especifica los resultados probabilísticos de las acciones y una **función de recompensa** que especifica la recompensa en cada estado.
- La utilidad de una secuencia de estados es la suma de todas las recompensas a lo largo de la secuencia, posiblemente descontada a lo largo del tiempo. La solución de un MDP es una **política** que asocia una decisión con cada estado al que el agente podría llegar. Una política óptima maximiza la utilidad de las secuencias de estados encontradas cuando se ejecuta.
- La utilidad de un estado es la suma esperada de recompensas cuando se ejecuta una política óptima desde ese estado. El algoritmo de **iteración de valor** resuelve de manera iterativa un conjunto de ecuaciones que relacionan la utilidad de cada estado con las de sus vecinos.
- La **iteración de políticas** alterna entre calcular las utilidades de los estados bajo la política actual y mejorar la política actual con respecto a las utilidades actuales.
- Los MDPs parcialmente observables, o POMDPs, son mucho más difíciles de resolver que los MDPs. Se resuelven mediante la conversión a un MDP en el espacio continuo de estados de creencia; se han ideado tanto algoritmos de iteración de valor como de iteración de políticas. El comportamiento óptimo en los POMDPs incluye la recopilación de información para reducir la incertidumbre y, por lo tanto, tomar mejores decisiones en el futuro.
- Es posible construir un agente de decisión teórica para entornos POMDP. El agente utiliza una **red de decisión dinámica** para representar los modelos de transición y sensor, para actualizar su estado de creencia y para proyectar posibles secuencias de acción hacia adelante.

Volveremos a los MDPs y POMDPs en el Capítulo 22, que cubre métodos de **aprendizaje por refuerzo** que permiten a un agente mejorar su comportamiento a partir de la experiencia.

Notas bibliográficas e históricas

Richard Bellman desarrolló las ideas subyacentes al enfoque moderno de los problemas de decisión secuencial mientras trabajaba en la Corporación RAND a partir de 1949. De acuerdo con su autobiografía (Bellman, 1984), él acuñó el término "programación dinámica" para ocultar a Charles Wilson, Secretario de Defensa fóbico a la investigación, que de hecho su grupo estaba haciendo matemáticas. (Esto no es cierto, porque su primer artículo usando el término (Bellman, 1952) apareció antes de que Wilson se convirtiera en Secretario de Defensa en 1953). El libro de Bellman, *Programación dinámica* (1957), dio al nuevo campo una sólida fundación e introdujo el algoritmo de iteración de valor.

Shapley (1953b) en realidad describió el algoritmo de iteración de valor independiente de Bellman, pero sus resultados no fueron apreciados con amplitud en la comunidad de investigación de operaciones, quizá porque se presentaron en el contexto más general de los juegos de Markov. Aunque las formulaciones originales incluían el descuento, su análisis en términos de preferencias estacionarias fue sugerido por Koopmans (1972). El teorema de conformación se debe a Ng *et al.* (1999).

La tesis de doctorado de Ron Howard (1960) introdujo la iteración de políticas y la idea de recompensa promedio para resolver problemas de horizonte infinito. Varios resultados adicionales fueron introducidos por Bellman y Dreyfus (1962). El uso de mapeos de contracción en el análisis de los algoritmos de programación dinámica se debe a Denardo (1967). La iteración de políticas modificada se debe a Van Nunen (1976) y Puterman y Shin (1978). La iteración de políticas asincrónicas fue analizada por Williams y Baird (1993), quienes también probaron el límite de pérdida de políticas en la Ecuación (17.13). La familia general de **algoritmos de barrido** priorizados tiene como objetivo acelerar la convergencia a políticas óptimas ordenando de forma heurística los cálculos de actualización de valor y política (Moore y Atkeson, 1993; Andre *et al.*, 1998; Wingate y Seppi, 2005).

La formulación de la resolución de MDP como un programa lineal se debe a De Ghellinck (1960), Manne (1960) y D'Épenoux (1963). Aunque la programación lineal ha sido considerada inferior a la programación dinámica como método de solución exacta para los MDPs, de Farias y Roy (2003) muestran que es posible usar la programación lineal y una representación lineal de la función de utilidad para obtener soluciones aproximadas comprobablemente buenas para MDPs muy grandes. Papadimitriou y Tsitsiklis (1987) y Littman *et al.* (1995) proporcionan resultados generales sobre la complejidad computacional de los MDPs. Yinyu Ye (2011) analiza la relación entre la iteración de políticas y el método simplex para la programación lineal y demuestra que para γ fijo, el tiempo de ejecución de la iteración de políticas es polinomial en el número de estados y acciones.

El trabajo seminal de Sutton (1988) y Watkins (1989) sobre métodos de aprendizaje por refuerzo para resolver MDPs tuvo un papel significativo en la introducción de MDPs en la comunidad de IA. (El trabajo anterior de Werbos (1977) contenía ideas similares, pero no fue adoptado en la misma medida). Los investigadores de IA han impulsado los MDPs en la dirección de representaciones más expresivas que acomodan problemas mucho más grandes que las representaciones atómicas tradicionales basadas en matrices de transición.

Las ideas básicas para una arquitectura de agente utilizando redes de decisión dinámicas fueron propuestas por Dean y Kanazawa (1989a). Tatman y Shachter (1990) mostraron cómo aplicar algoritmos de programación dinámica a modelos DDN. Varios autores hicieron la conexión entre MDPs y problemas de planificación de IA, desarrollando formas probabilísticas de la representación STRIPS compacta para modelos de transición (Wellman, 1990b; Koenig, 1991). El libro *Planning and Control* de Dean y Wellman (1991) explora la conexión en gran profundidad.

MDPs factorizados

MDPs relacionales

Trabajos posteriores sobre **MDPs factorizados** (Boutilier *et al.*, 2000; Koller y Parr, 2000; Guestrin *et al*., 2003b) utilizan representaciones estructuradas de la función de valor así como de la transición modelo, con mejoras comprobables en complejidad. Los **MDPs relacionales** (Boutilier *et al*., 2001; Guestrin *et al*., 2003a) van un paso más allá, utilizando representaciones estructuradas para manejar dominios con muchos objetos relacionados. Los MDPs y POMDPs de universo abierto (Srivastava *et al*., 2014b) también permiten la incertidumbre sobre la existencia e identidad de objetos y acciones.

Múltiples autores han desarrollado algoritmos en línea aproximados para la toma de decisiones en MDPs, tomando prestado de manera explícita de los enfoques de IA anteriores para la búsqueda en tiempo real y el juego (Werbos, 1992; Dean *et al*., 1993; Tash y Russell, 1994). El trabajo de Barto *et al.* (1995) sobre RTDP (programación dinámica en tiempo real) proporcionó un marco general para entender tales algoritmos y su conexión con el aprendizaje por refuerzo y la búsqueda heurística. El análisis de expectimax con profundidad limitada con muestreo en nodos de azar se debe a Kearns *et al.* (2002). El algoritmo UCT descrito en el capítulo es debido a Kocsis y Szepesvari (2006) y toma prestado de trabajos anteriores sobre playouts aleatorios para estimar los valores de los estados (Abramson, 1990; Brügmann, 1993; Chang *et al.*, 2005).

Los problemas de bandidos fueron introducidos por Thompson (1933) pero cobraron prominencia después de la Segunda Guerra Mundial a través del trabajo de Herbert Robbins (1952). Bradt *et al.* (1956) demostraron los primeros resultados sobre las reglas de parada para bandidos de un solo brazo, que eventualmente llevaron a los resultados revolucionarios de John Gittins (Gittins y Jones, 1974; Gittins, 1989). Katehakis y Veinott (1987) sugirieron el MDP de reinicio como un método para calcular los índices de Gittins. El texto de Berry y Fristedt (1985) cubre muchas variaciones del problema básico, mientras que el texto online pellucid (degeneración marginal pelúcida) de Ferguson (2001) conecta los problemas de bandidos con los problemas de parada.

Lai y Robbins (1985) iniciaron el estudio del arrepentimiento asintótico de las políticas de bandidos óptimas. La heurística UCB fue introducida y analizada por Auer *et al.* (2002). Los superprocesos de bandidos (BSPs) fueron estudiados por primera vez por Nash (1973) pero han permanecido en gran medida desconocidos en IA. Hadfield-Menell y Russell (2015) describen un algoritmo eficiente de ramificación y acotación capaz de resolver BSPs relativamente grandes. Los problemas de selección fueron introducidos por Bechhofer (1954). Hay *et al.* (2012) desarrollaron un marco formal para los problemas de meta-razonamiento, mostrando que las instancias simples se mapeaban a problemas de selección en lugar de problemas de bandidos. También demostraron el satisfactorio resultado de que el costo computacional esperado de la estrategia computacional óptima nunca es mayor que la ganancia esperada en la calidad de la decisión, aunque hay casos en los que la política óptima permite, con cierta probabilidad, seguir computando mucho después del punto en el que cualquier posible ganancia ha sido agotada.

La observación de que un MDP parcialmente observable permite transformarse en un MDP regular sobre estados de creencia se debe a Astrom (1965) y Aoki (1965). El primer algoritmo completo para la solución exacta de POMDPs, en esencia el algoritmo de iteración de valor presentado en este capítulo, fue propuesto por Edward Sondik (1971) en su tesis de doctorado. (Un artículo de revista posterior de Smallwood y Sondik (1973) contiene algunos errores, pero es más accesible). Lovejoy (1991) revisó los primeros veinticinco años de investigación de POMDP, llegando a conclusiones algo pesimistas sobre la factibilidad de resolver grandes problemas.

La primera contribución significativa dentro de la IA fue el algoritmo Witness (Cassandra *et al*., 1994; Kaelbling *et al*., 1998), una versión mejorada de la iteración de valor POMDP. Pronto siguieron otros algoritmos, incluyendo un enfoque debido a Hansen (1998) que construye una política de forma incremental en forma de un autómata de estado finito cuyos estados definen los posibles estados de creencia del agente.

Trabajos más recientes en IA se han centrado en métodos de iteración de valor **basados en puntos** que, en cada iteración, generan planes condicionales y α-vectores para un conjunto finito de estados de creencia en lugar de para todo el espacio de creencia. Lovejoy (1991) propuso tal algoritmo para una cuadrícula fija de puntos, enfoque también adoptado por Bonet (2002). Un influyente artículo de Pineau *et al*. (2003) sugirió generar puntos alcanzables simulando trayectorias de una manera algo codiciosa; Spaan y Vlassis (2005) observan que sólo es necesario generar planes para un pequeño subconjunto de puntos seleccionados al azar para mejorar los planes de la iteración anterior para todos los puntos del conjunto. Shani *et al.* (2013) revisan estos y otros desarrollos en algoritmos basados en puntos, que han llevado a buenas soluciones para problemas con miles de estados. Debido a que los POMDP son PSPACE-hard (Papadimitriou y Tsitsiklis, 1987), el progreso adicional en los métodos de solución fuera de línea puede requerir aprovechar varios tipos de estructura en las funciones de valor que surgen de una representación factorizada del modelo.

El enfoque online para los POMDP, utilizando la búsqueda de anticipación para seleccionar una acción para el estado de creencia actual, fue examinado por primera vez por Satia y Lave (1973). El uso de muestreo en nodos de azar fue explorado de forma analítica por Kearns *et al.* (2000) y Ng y Jordan (2000). El algoritmo POMCP se debe a Silver y Veness (2011).

Con el desarrollo de algoritmos de aproximación razonablemente efectivos para los POMDP, su uso como modelos para problemas del mundo real ha aumentado, en particular en educación (Rafferty *et al*., 2016), sistemas de diálogo (Young *et al*., 2013), robótica (Hsiao *et al*., 2007; Huynh y Roy, 2009) y coches autónomos (Forbes *et al*., 1995; Bai *et al*., 2015). Una aplicación a gran escala importante es el Sistema de Evitación de Colisiones en el Aire X (ACAS X), que evita que los aviones y drones colisionen en el aire. El sistema utiliza POMDP con redes neuronales para hacer aproximación de funciones. ACAS X mejora significativamente la seguridad en comparación con el heredado sistema TCAS, que fue construido en la década de 1970 utilizando tecnología de sistemas expertos (Kochenderfer, 2015; Julian *et al*., 2018).

La toma de decisiones complejas también ha sido estudiada por economistas y psicólogos. Ellos encuentran que los tomadores de decisiones no siempre son racionales, y pueden no estar operando como se describe en los modelos de este capítulo. Por ejemplo, cuando se les da a elegir, la mayoría de las personas prefieren $100 hoy sobre una garantía de $200 en dos años, pero esas mismas personas prefieren $200 en ocho años sobre $100 en seis años. Una forma de interpretar este resultado es que las personas no están utilizando recompensas descontadas exponencialmente aditivas; tal vez están utilizando **recompensas hiperbólicas** (la función hiperbólica se hunde de manera más abrupta a corto plazo que la función de decaimiento exponencial). Rubinstein (2003) discute esta y otras posibles interpretaciones.

Recompensa hiperbólica

Los textos de Bertsekas (1987) y Puterman (1994) proporcionan introducciones rigurosas a los problemas de decisiones secuenciales y la programación dinámica. Bertsekas y Tsitsiklis (1996) incluyen cobertura de aprendizaje por refuerzo. Sutton y Barto (2018) cubren terreno similar pero en un estilo más accesible. Sigaud y Buffet (2010), Mausam y Kolobov (2012) y Kochenderfer (2015) cubren la toma de decisiones secuenciales desde una perspectiva de IA. Krishnamurthy (2016) proporciona una cobertura exhaustiva de los POMDP.

CAPÍTULO 18

TOMA DE DECISIONES SIMPLES MULTIAGENTE

En el que examinamos qué hacer cuando más de un agente habita el entorno.

18.1 Propiedades de los entornos multiagente

Hasta ahora, hemos asumido en gran medida que sólo un agente ha estado teniendo percepción, planificación y ejecución de la acción. Aunque esto representa una enorme simplificación, que no logra capturar muchas configuraciones de IA del mundo real. En este capítulo, por lo tanto, consideraremos los problemas que surgen cuando un agente debe tomar decisiones en entornos que contienen múltiples actores. Tales entornos se llaman **sistemas multiagente**, sistemas donde los agentes enfrentan un **problema de planificación multiagente**. No obstante, la naturaleza precisa del problema de planificación multiagente y las técnicas apropiadas para resolverlo dependerán, como veremos, de las relaciones entre los agentes en el entorno.

Sistemas multiagente

Problema de planificación multiagente

18.1.1 Un tomador de decisiones

La primera posibilidad es que mientras el entorno contiene múltiples *actores*, sólo contiene un *tomador de decisiones*. En tal caso, el tomador de decisiones desarrolla planes para los otros agentes y les dice qué hacer. La suposición de que los agentes harán lo que se les dice se llama **suposición de agente benevolente**. Sin embargo, incluso en este escenario, los planes que involucran múltiples actores requerirán que los actores *sincronicen* sus acciones. Los actores A y B tendrán que actuar al mismo tiempo para acciones conjuntas (como cantar un dueto), en diferentes momentos para acciones mutuamente excluyentes (como recargar baterías cuando sólo hay un enchufe), y de manera secuencial cuando uno establece una precondición para el otro (como A lavando los platos y luego B secándolos).

Suposición de agente benevolente

Un caso especial es donde tenemos un sólo tomador de decisiones con múltiples actuadores que operan de manera simultánea; por ejemplo, un humano que puede caminar y hablar al mismo tiempo. Dicho agente necesita hacer una **planificación de múltiples actuadores** para manejar cada actuador mientras maneja interacciones positivas y negativas entre los actuadores. Cuando los actuadores están físicamente desacoplados en unidades separadas, como en una flota de robots de entrega en una fábrica, la planificación de múltiples actuadores se convierten en **planificación de cuerpos múltiples**.

Planificación de múltiples actuadores

Planificación cuerpos múltiples

Un problema de cuerpos múltiples sigue siendo un problema de agente único "estándar" siempre que la información del sensor relevante recopilada por cada cuerpo pueda agruparse, ya sea de manera centralizada o dentro de cada cuerpo, para formar una estimación común del estado del mundo que luego informa la ejecución del plan general; en este caso, los múltiples cuerpos pueden considerarse como actuando como un solo cuerpo. Cuando las restricciones de comunicación imposibilitan esto, tenemos lo que a veces se llama un problema de **planificación descentralizada**; esto es, quizás un error de nombre, porque la fase de planificación está centralizada pero la fase de ejecución está

Planificación descentralizada

al menos parcialmente desacoplada. En este caso, el subplan construido para cada cuerpo puede necesitar incluir acciones comunicativas explícitas con otros cuerpos. Por ejemplo, múltiples robots de reconocimiento que cubren una amplia área con frecuencia están fuera de contacto por radio entre sí y deben compartir sus hallazgos durante los momentos en que la comunicación es factible.

18.1.2 Múltiples tomadores de decisiones

Contrapartes

La segunda posibilidad es que los otros actores en el entorno también sean tomadores de decisiones: cada uno tiene preferencias y elige y ejecuta su propio plan. Los llamamos **contrapartes**. En este caso, podemos distinguir dos posibilidades adicionales.

Objetivo común

Problema de coordinación

- La primera es que, aunque hay múltiples tomadores de decisiones, todos están persiguiendo un **objetivo común**. Esta es la situación de los trabajadores en una empresa, en la que diferentes tomadores de decisiones están persiguiendo, se espera, los mismos objetivos en nombre de la empresa. El principal problema al que se enfrentan los tomadores de decisiones en este entorno es **el problema de coordinación**: necesitan asegurarse de que todos están empujando en la misma dirección y no están estropeando de manera accidental los planes de los demás.
- La segunda posibilidad es que cada uno de los tomadores de decisiones tiene sus propias preferencias personales y que perseguirá lo mejor que pueda. Podría ser que las preferencias estén diametralmente opuestas, como es el caso en los juegos de suma cero como el ajedrez (vea el Capítulo 5). Aunque la mayoría de los encuentros multiagente son más complicados que eso, con preferencias más complejas.

Teoría de juegos

Cuando hay múltiples tomadores de decisiones, cada uno persigue sus propias preferencias; un agente debe tener en cuenta las preferencias de otros agentes, así como el hecho de que estos otros agentes *también* están tomando en cuenta las preferencias de otros agentes, y así sucesivamente. Esto nos lleva al ámbito de la **teoría de juegos**: teoría de toma de decisiones estratégicas. Es este aspecto *estratégico* del razonamiento, donde cada uno de los jugadores toma en cuenta cómo pueden actuar otros jugadores; factor que distingue a la teoría de juegos de la teoría de decisiones. De la misma manera que la teoría de decisiones proporciona la base teórica para la toma de decisiones en la IA de un solo agente, la teoría de juegos proporciona la base teórica para la toma de decisiones en sistemas multiagente.

Toma de decisiones estratégicas

El uso de la palabra "juego" aquí tampoco es ideal: una inferencia natural es que la teoría del juego se ocupa de actividades recreativas o escenarios artificiales. Nada podría estar más lejos de la verdad. La teoría de juegos es la teoría de la **toma de decisiones estratégicas**. Se utiliza en toma de decisiones que incluyen la subasta de derechos de perforación de petróleo y derechos de espectro de frecuencia inalámbrica, procedimientos de quiebra, decisiones de desarrollo y precios de productos, y defensa nacional; todas ellas situaciones que involucran miles de millones de dólares y muchas vidas. La teoría de juegos en IA se usa de dos maneras principales:

Diseño de agentes

1. **Diseño de agentes**: La teoría de juegos permite ser utilizada por un agente para analizar sus posibles decisiones y calcular la utilidad esperada para cada una de estas (bajo la suposición de que otros agentes están actuando de manera racional, de acuerdo con la teoría de juegos). Así, las técnicas de teoría de juegos permiten determinar la mejor estrategia contra un jugador racional y el retorno esperado para cada jugador.

Diseño de mecanismos

2. **Diseño de mecanismos**: Cuando un entorno está habitado por muchos agentes, podría ser posible definir las reglas del entorno (es decir, el juego que los agentes deben jugar) de manera que el bien colectivo de todos los agentes se maximice cuando cada agente adopta la solución de teoría de juegos que maximiza su propia utilidad. Por ejemplo, la

teoría de juegos ayuda a diseñar los protocolos para una colección de enrutadores de tráfico de Internet de manera que cada enrutador tenga un incentivo para actuar de tal manera que se maximice el rendimiento global. El diseño de mecanismos también se utiliza para construir sistemas multiagente inteligentes que resuelven problemas complejos de manera distribuida.

La teoría de juegos proporciona una gama de diferentes modelos, cada uno con su propio conjunto de suposiciones subyacentes; es importante elegir el modelo correcto para cada configuración. La distinción más importante es si debemos considerarlo un juego cooperativo o no:

- En un **juego cooperativo** es posible tener un acuerdo vinculante entre agentes, lo que permite una cooperación robusta. En el mundo humano, los contratos legales y las normas sociales ayudan a establecer tales acuerdos vinculantes. En el mundo de los programas de computadora, es posible inspeccionar el código fuente para asegurarse de que seguirá un acuerdo. Utilizamos la teoría de juegos cooperativa para analizar esta situación. Juego cooperativo
- Si no es posible llegar a acuerdos vinculantes, tenemos un **juego no cooperativo**. Aunque este término sugiere que el juego es inherentemente competitivo, y que la cooperación no es posible, eso no tiene por qué ser el caso: no cooperativo significa que no hay un acuerdo central que vincule a todos los agentes y garantice la cooperación. Pero bien podría ser que los agentes decidan cooperar de forma independiente, porque está en su propio interés. Utilizamos la teoría de juegos no cooperativa para analizar esta situación. Juego no cooperativo

Algunos entornos combinarán múltiples dimensiones diferentes. Por ejemplo, una empresa de mensajería permite hacer una planificación centralizada y offline para las rutas de sus camiones y aviones cada día, pero deja algunos aspectos abiertos para decisiones autónomas de los conductores y pilotos que responden de manera individual a situaciones de tráfico y clima. Además, los objetivos de la empresa y sus empleados se alinean, hasta cierto punto, mediante el pago de **incentivos** (salarios y bonos), una señal segura de que este es un verdadero sistema multiagente. Incentivos

18.1.3 Planificación multiagente

Por el momento, trataremos los entornos de múltiples actuadores, multiples cuerpos y multiagentes de la misma manera, etiquetándolos como entornos **multiactor**, utilizando el término genérico **actor** para cubrir actuadores, cuerpos y agentes. El objetivo de esta sección es trabajar en cómo definir modelos de transición, planes correctos y algoritmos de planificación eficientes para el entorno multiactor. Un plan correcto es aquel que, si es ejecutado por los actores, logra el objetivo. (En el verdadero entorno multiagente, por supuesto, los agentes pueden no estar de acuerdo en ejecutar ningún plan en particular, pero al menos sabrán qué planes *funcionarían* si *estuvieran* de acuerdo en ejecutarlos). Multiactor Actor

Una dificultad clave al intentar llegar a un modelo satisfactorio de acción multiagente es que debemos lidiar de alguna manera con el espinoso problema de la **concurrencia**, lo que significa que los planes de cada agente permiten ser ejecutados de manera simultánea. Si vamos a razonar sobre la ejecución de planes multiactor, entonces, primero necesitaremos un modelo de planes multiactor que incorpore un modelo satisfactorio de acción concurrente. Concurrencia

Además, la acción multiactor plantea todo un conjunto de problemas que no son una preocupación en la planificación de un solo agente. En particular, *los agentes deben tener en cuenta la forma en que sus propias acciones interactúan con las acciones de otros agentes*. Por ejemplo, un agente necesitará considerar si las acciones realizadas por otros agentes podrían estropear las precondiciones de sus propias acciones, si los recursos que utiliza mientras ejecuta su política

son compartibles, o pueden ser agotados por otros agentes, si las acciones son mutuamente excluyentes, y si un agente inclinado a ayudar podría considerar cómo sus acciones podrían facilitar las acciones de otros.

Para responder las preguntas planteadas aquí necesitamos un modelo de acción concurrente dentro del cual podamos formularlas de manera correcta. Los modelos de acción concurrente han sido un enfoque importante de investigación en la comunidad de ciencias de la computación durante décadas, pero no ha prevalecido ningún modelo definitivo y universalmente aceptado. Sin embargo, los siguientes tres enfoques se han establecido de manera amplia.

Ejecución entrelazada

El primer enfoque es considerar la **ejecución entrelazada** de las acciones en los planes respectivos. Por ejemplo, suponga que tenemos dos agentes, A y B, con planes de la siguiente manera:

$$A : [a_1, a_2]$$
$$B : [b_1, b_2].$$

La idea clave del modelo de ejecución entrelazada es que lo único de lo que podemos estar seguros en la ejecución de los planes de los dos agentes es que se preservará el orden de las acciones en los planes respectivos. Si, además, asumimos que las acciones son atómicas, entonces hay seis formas diferentes en las que los dos planes anteriores podrían ejecutarse de manera concurrente:

$$[a_1, a_2, b_1, b_2]$$
$$[b_1, b_2, a_1, a_2]$$
$$[a_1, b_1, a_2, b_2]$$
$$[b_1, a_1, b_2, a_2]$$
$$[a_1, b_1, b_2, a_2]$$
$$[b_1, a_1, a_2, b_2]$$

Para que un plan sea correcto en el modelo de ejecución entrelazada, *debe ser correcto con respecto a todas las posibles intercalaciones de los planes*. El modelo de ejecución entrelazada ha sido ampliamente adoptado dentro de la comunidad de concurrencia, porque es un modelo razonable de la forma en que múltiples hilos se turnan para ejecutarse en una sola CPU. Sin embargo, no modela el caso en que dos acciones suceden al mismo tiempo. Además, el número de secuencias entrelazadas crecerá de manera exponencial con el número de agentes y acciones: como consecuencia, verificar la corrección de un plan, que es computacionalmente sencillo en configuraciones de un solo agente, es computacionalmente difícil con el modelo de ejecución entrelazada.

Concurrencia verdadera

El segundo enfoque es la **concurrencia verdadera**, en la que no intentamos crear un ordenamiento completo serializado de las acciones, sino que las dejamos *parcialmente ordenadas*: sabemos que a_1 ocurrirá antes que a_2; pero con respecto al ordenamiento de a_1 y b_1, por ejemplo, no podemos decir nada, uno puede ocurrir antes que el otro, o podrían ocurrir de manera concurrente. Siempre podemos "aplanar" un modelo de orden parcial de planes concurrentes en un modelo entrelazado, pero al hacerlo, perdemos la información de orden parcial. Aunque los modelos de orden parcial son más satisfactorios que los modelos entrelazados como una cuenta teórica de la acción concurrente, en la práctica no han sido adoptados de manera amplia.

Sincronización

El tercer enfoque es asumir una **sincronización** perfecta: hay un reloj global al que cada agente tiene acceso, cada acción toma la misma cantidad de tiempo y las acciones en cada punto del plan conjunto son simultáneas. Por lo tanto, las acciones de cada agente se ejecutan de manera sincronizada, al unísono entre sí (es posible que algunos agentes ejecuten una acción de no operación cuando están esperando que se completen otras acciones). La ejecución sincrónica no es un modelo muy completo de concurrencia en el mundo real, pero tiene una semántica simple y, por esta razón, es el modelo con el que trabajaremos aquí.

Actors(*A*,*B*)
Init(*At*(*A*,*LeftBaseline*) ∧ *At*(*B*,*RightNet*) ∧
  Approaching(*Ball*,*RightBaseline*) ∧ *Partner*(*A*,*B*) ∧ *Partner*(*B*,*A*)
Goal(*Returned*(*Ball*) ∧ (*At*(*x*,*RightNet*) ∨ *At*(*x*,*LeftNet*))
Action(*Hit*(*actor*,*Ball*),
  PRECOND:*Approaching*(*Ball*,*loc*) ∧ *At*(*actor*,*loc*)
  EFFECT:*Returned*(*Ball*))
Action(*Go*(*actor*,*to*),
  PRECOND:*At*(*actor*,*loc*) ∧ *to* ≠ *loc*,
  EFFECT:*At*(*actor*,*to*) ∧ ¬*At*(*actor*,*loc*))

Figura 18.1 El problema del dobles en tenis. Dos actores, *A* y *B*, juegan juntos y pueden estar en una de las cuatro ubicaciones: *LeftBaseline*, *RightBaseline*, *LeftNet* y *RightNet*. La pelota sólo permite ser devuelta si un jugador está en el lugar correcto. La acción *NoOp* es una simulación, que no tiene efecto. Note que cada acción debe incluir al actor como un argumento.

Comenzamos con el modelo de transición; para el caso determinista de un solo agente, esta es la función RESULT(*s*,*a*), que da el estado que resulta de realizar la acción *a* cuando el entorno está en el estado *s*. En el entorno de un solo agente, podría haber *b* diferentes opciones para la acción; *b* puede ser bastante grande, en especial para representaciones de primer orden con muchos objetos sobre los que actuar, pero los esquemas de acción proporcionan una representación concisa de todos modos.

En el entorno multiactor con *n* actores, la única acción *a* es reemplazada por una **acción conjunta** ⟨a_1,..., a_n⟩, donde a_i es la acción tomada por el actor *i*. De manera inmediata, vemos dos problemas: primero, tenemos que describir el modelo de transición para b^n diferentes acciones conjuntas; segundo, tenemos un problema de planificación conjunta con un factor de ramificación de b^n. Acción conjunta

Habiendo agrupado a los actores en un sistema multiactor con un enorme factor de ramificación, el principal enfoque de la investigación sobre la planificación multiactor ha sido *desacoplar* a los actores en la medida de lo posible, de modo que, idealmente, la complejidad del problema crezca de manera lineal con *n* en lugar de exponencial con b^n.

Si los actores no tienen interacción entre sí, por ejemplo, *n* actores cada uno jugando un juego de solitario, entonces podemos resolver *n* problemas separados. Si los actores están **ligeramente acoplados**, ¿podemos alcanzar algo cercano a esta mejora exponencial? Esta pregunta es, por supuesto, central en muchas áreas de la IA. Hemos visto métodos de solución exitosos para sistemas ligeramente acoplados en el contexto de CSPs, donde los grafos de restricciones "como árboles" produjeron métodos de solución eficientes (ver página 202), así como en el contexto de bases de datos de patrones disjuntos (página 100) y heurísticas aditivas para la planificación (página 356). Ligeramente acoplado

El enfoque estándar para problemas ligeramente acoplados es pretender que los problemas están completamente desacoplados y luego arreglar las interacciones. Para el modelo de transición, esto significa escribir esquemas de acción como si los actores actuaran de forma independiente.

Veamos cómo funciona esto para un partido de tenis doble. Aquí, tenemos dos jugadores de tenis humanos que forman un equipo de dobles con el objetivo común de ganar el partido contra un equipo oponente. Suponga que en un punto del juego, el equipo tiene el objetivo de devolver la pelota que les ha sido servida y asegurarse de que al menos uno de ellos está cubriendo la red. La Figura 18.1 muestra las condiciones iniciales, el objetivo y los esquemas de acción para este problema. Es fácil ver que podemos pasar de las condiciones iniciales al objetivo con un **plan conjunto** de dos pasos que especifica lo que cada jugador tiene que hacer: Plan conjunto

A debería moverse hacia la línea de base derecha y golpear la pelota, mientras que *B* debería quedarse quieto en la red:

PLAN 1: *A*: [*Go*(*A*,*RightBaseline*), *Hit*(*A*,*Ball*)]
B: [*NoOp*(*B*),*NoOp*(*B*)].

Sin embargo, los problemas surgen cuando un plan dicta que ambos agentes golpeen la pelota al mismo tiempo. En el mundo real esto no funcionará, pero el esquema de acción para *Hit* dice que la pelota será devuelta con éxito. La dificultad es que las precondiciones restringen el estado en el que una acción por sí misma permite ser ejecutada con éxito, pero no restringen otras acciones concurrentes que podrían estropearlo.

Restricción de acción concurrente

Resolvemos este problema agregando una nueva característica a los esquemas de acción: una **restricción de acción concurrente** que indica qué acciones deben o no deben ejecutarse de manera concurrente. Por ejemplo, la acción *Hit* podría describirse de la siguiente manera:

Action(*Hit*(*actor*,*Ball*),
CONCURRENT: $\forall b\ b \neq actor \Rightarrow \neg Hit(b,Ball)$
PRECOND:*Approaching*(*Ball*,*loc*) $\wedge$ *At*(*actor*,*loc*)
EFFECT:*Returned*(*Ball*)).

En otras palabras, la acción *Hit* tiene su efecto declarado sólo si ninguna otra acción *Hit* de otro agente ocurre al mismo tiempo. (En el enfoque SATPLAN, esto sería manejado por un axioma de **exclusión de acción parcial**). Para algunas acciones, el efecto deseado se logra sólo cuando otra acción ocurre de manera concurrente. Por ejemplo, se necesitan dos agentes para llevar un refrigerador lleno de bebidas a la cancha de tenis:

Action(*Carry*(*actor*,*cooler*,*here*,*there*),
CONCURRENT: $\exists b\ b \neq actor \wedge Carry(b,cooler,here,there)$
PRECOND: *At*(actor,here) $\wedge$ At(cooler,here) $\wedge$ Cooler(cooler)
EFFECT: At(*actor*,*there*) $\wedge$ *At*(*cooler*,*there*) $\wedge \neg At(actor,here) \wedge \neg At(cooler,here)$).

Con estos tipos de esquemas de acción, cualquiera de los algoritmos de planificación descritos en el Capítulo 11 se puede adaptar con sólo modificaciones menores para generar planes multiactor. En la medida en que el acoplamiento entre subplanes es débil, lo que significa que las restricciones de concurrencia entran en juego de manera rara durante la búsqueda del plan, se esperaría que las diversas heurísticas derivadas para la planificación de un solo agente también sean efectivas en el contexto multiactor.

18.1.4 Planificación con múltiples agentes: cooperación y coordinación

Ahora consideremos un verdadero escenario multiagente en el que cada agente hace su propio plan. Para empezar, suponga que los objetivos y la base de conocimientos son compartidos. Uno podría pensar que esto se reduce al caso de múltiples cuerpos, cada agente calcula la solución conjunta y ejecuta su propia parte de esa solución. Sin embargo, el "el" en "la solución conjunta" es engañoso. Aquí hay un segundo plan que también logra el objetivo:

PLAN 2: *A*: [*Go*(*A*,*LeftNet*),*NoOp*(*A*)]
B: [*Go*(*B*,*RightBaseline*),*Hit*(*B*,*Ball*)].

Si ambos agentes acuerdan el plan 1 o el plan 2, se logrará el objetivo. Pero si *A* elige el plan 2 y *B* elige el plan 1, entonces nadie devolverá la pelota. Al contrario, si *A* elige 1 y *B* elige 2, ambos intentarán golpear la pelota y eso también fallará. Los agentes lo saben, pero ¿cómo pueden coordinarse para asegurarse de que están de acuerdo con el plan?

Una opción es adoptar una **convención** antes de participar en una actividad conjunta. Una convención es cualquier restricción en la selección de planes conjuntos. Por ejemplo, la convención "mantente en tu lado de la cancha" descartaría el plan 1, haciendo que ambos socios seleccionen el plan 2. Los conductores en una carretera enfrentan el problema de no chocar entre sí; esto se resuelve (parcialmente) adoptando la convención "mantente en el lado derecho de la carretera" en la mayoría de los países; la alternativa, "mantente en el lado izquierdo", funciona igual de bien siempre que todos los agentes en un entorno estén de acuerdo. Consideraciones similares se aplican al desarrollo del lenguaje humano, donde lo importante no es qué idioma debe hablar cada individuo, sino el hecho de que en una comunidad todos hablen el mismo idioma. Cuando las convenciones son generalizadas, se llaman **leyes sociales**.

Convención

Leyes sociales

En ausencia de una convención, los agentes usan la **comunicación** para lograr un conocimiento común de un plan conjunto factible. Por ejemplo, un jugador de tenis podría gritar "¡Mío!" o "¡Tuyo!" para indicar un plan conjunto preferido. La comunicación no necesariamente implica un intercambio verbal. Por ejemplo, un jugador puede comunicar un plan conjunto preferido al otro ejecutando la primera parte de él. Si el agente A se dirige a la red, entonces el agente B está obligado a volver a la línea de base para golpear la pelota, porque el plan 2 es el único plan conjunto que comienza con A dirigiéndose a la red. Este enfoque de coordinación, a veces llamado **reconocimiento de plan**, funciona cuando una sola acción (o una corta secuencia de acciones) de un agente es suficiente para que el otro determine un plan conjunto de manera inequívoca.

Reconocimiento de plan

18.2 Teoría de juegos no cooperativa

Ahora introduciremos los conceptos clave y las técnicas analíticas de la teoría de juegos, la teoría que sustenta la toma de decisiones en entornos multiagente. Nuestro recorrido comenzará con la teoría de juegos no cooperativa.

18.2.1 Juegos con un solo movimiento: juegos en forma normal

El primer modelo de juego que veremos es uno en el que todos los jugadores toman acción de manera simultánea y el resultado del juego se basa en el perfil de acciones que se seleccionan de esta manera. (En realidad, no es crucial que las acciones ocurran al mismo tiempo; lo que importa es que ningún jugador tiene conocimiento de las elecciones de los otros jugadores). Estos juegos se llaman en **juegos de forma normal**. Un juego en forma normal se define por tres componentes:

Juego de forma normal

- **Jugadores** o agentes que tomarán decisiones. Los juegos de dos jugadores han recibido la mayor atención, aunque los juegos de n jugadores para $n > 2$ también son comunes. Damos a los jugadores nombres en mayúsculas, como *Ali* y *Bo* u O y E.

  Jugadores
- **Acciones** que los jugadores pueden elegir. Daremos a las acciones nombres en minúsculas, como *uno* o *testificar*. Los jugadores pueden o no tener el mismo conjunto de acciones disponibles.
- Una **función de pago** que da la utilidad a cada jugador para cada combinación de acciones de todos los jugadores. Para juegos de dos jugadores, la función de pago para un jugador es representada por una matriz en la que hay una fila para cada acción posible de un jugador, y una columna para cada elección posible del otro jugador: una fila elegida y una columna elegida definen una celda de matriz, que está etiquetada con el pago para el jugador relevante. En el caso de dos jugadores, es convencional combinar las dos matrices en una sola **matriz de pago**, en la que cada celda está etiquetada con pagos para ambos jugadores.

  Función de pago

  Matriz de pago

Para ilustrar estas ideas, veamos un juego de ejemplo, llamado **Morra de dos dedos**. En este juego, dos jugadores, O y E, muestran de manera simultánea uno o dos dedos. Dejemos que el

número total de dedos mostrados sea *f*. Si *f* es impar, *O* recoge *f* dólares de *E*; y si *f* es par, *E* recoge *f* dólares de *O*.[1] La matriz de pago para Morra de dos dedos es la siguiente:

	O: *uno*	*O*: *dos*
E: *uno*	$E = +2, O = -2$	$E = -3, O = +3$
E: *dos*	$E = -3, O = +3$	$E = +4, O = -4$

Jugador de fila
Jugador de columna

Decimos que *E* es el **jugador de fila** y *O* es el **jugador de columna**. Entonces, por ejemplo, la esquina inferior derecha muestra que cuando el jugador *O* elige la acción dos y *E* también elige dos, el pago es +4 para *E* y −4 para *O*.

Antes de analizar Morra de dos dedos, vale la pena considerar por qué se necesitan ideas de teoría de juegos en absoluto: ¿por qué no podemos enfrentar el desafío que enfrenta (digamos) el jugador *E* utilizando el aparato de teoría de decisiones y maximización de utilidad que hemos estado usando en otras partes del libro? Para ver por qué se necesita algo más, supongamos que *E* trata de encontrar la mejor acción para realizar. Las alternativas son *una* o *dos*. Si *E* elige *una*, entonces el pago será +2 o −3. Sin embargo, el pago que *E* recibirá dependerá de la elección hecha por *O*: lo más que *E* puede hacer, como jugador de fila, es forzar el resultado del juego a estar en una fila particular. De manera similar, *O* sólo elige la columna.

Para elegir de manera óptima entre estas posibilidades, *E* debe tener en cuenta cómo *O* actuará como un tomador de decisiones racional. Pero *O*, a su vez, debe tener en cuenta el hecho de que *E* también es un tomador de decisiones racional. Por lo tanto, la toma de decisiones en entornos multiagente es muy diferente en carácter a la toma de decisiones en entornos de un solo agente, porque los jugadores necesitan tener en cuenta cada razonamiento del otro. El papel de los **conceptos de solución** en la teoría de juegos es tratar de hacer este tipo de razonamiento preciso.

Concepto de solución
Estrategia
Estrategia pura
Estrategia mixta

El término **estrategia** se utiliza en la teoría de juegos para denotar lo que hemos llamado previamente una política. Una **estrategia pura** es una política determinista; para un juego de un solo movimiento, una estrategia pura es una sola acción. Como veremos a continuación, para muchos juegos un agente permite hacerlo mejor con una **estrategia mixta**, que es una política aleatorizada que selecciona acciones de acuerdo con una distribución de probabilidad. La estrategia mixta que elige la acción *a* con probabilidad *p* y la acción *b* con de otra manera se escribe $[p{:}a;(1-p){:}b]$. Por ejemplo, una estrategia mixta para Morra de dos dedos podría ser [0.5:*uno*;0.5:*dos*]. Un **perfil de estrategia** es una asignación de una estrategia a cada jugador; dado el perfil de estrategia, el **resultado** del juego es un valor numérico para cada jugador, si los jugadores usan estrategias mixtas, entonces debemos usar la utilidad esperada.

Perfil de estrategia

Entonces ¿cómo deben decidir los agentes actuar en juegos como Morra? La teoría de juegos proporciona una serie de conceptos de solución que intentan definir la acción racional con respecto a las creencias de un agente sobre las creencias del otro agente. Sin embargo, no hay un concepto de solución perfecto: es problemático definir qué significa "racional" cuando cada agente elige sólo una parte del perfil de estrategia que determina el resultado.

Dilema del prisionero

Introducimos nuestro primer concepto de solución a través de lo que quizá sea el juego más famoso en el canon de la teoría de juegos: el **dilema del prisionero**. Este juego está motivado por la siguiente historia: Dos presuntos ladrones, Ali y Bo, son atrapados en fragancia en la escena de un robo y son interrogados por separado. Un fiscal ofrece a cada uno un trato: si testificas

[1] Morra es una versión recreativa de un **juego de inspección**. En tales juegos, un inspector elige un día para inspeccionar una instalación (como un restaurante o una planta de armas biológicas), y el operador de la instalación elige un día para esconder todas las cosas desagradables. El inspector gana si los días son diferentes, y el operador de la instalación gana si son iguales.

contra tu compañero como el líder de una banda de ladrones, quedarás libre por ser el cooperativo, mientras su compañero cumplirá 10 años en prisión. Sin embargo, si ambos testifican uno contra el otro, ambos recibirán 5 años. Ali y Bo también saben que si ambos se niegan a testificar, sólo cumplirán 1 año cada uno por el cargo menor de posesión de propiedad robada. Ahora Ali y Bo enfrentan el llamado dilema del prisionero: ¿deberían testificar o negarse? Siendo agentes racionales, Ali y Bo quieren maximizar su propia utilidad esperada, lo que significa minimizar el número de años en prisión, cada uno es indiferente sobre el bienestar del otro jugador. El dilema del prisionero se captura en la siguiente matriz de pagos:

	Ali:testificar	*Ali:rechazar*
Bo:testificar	$A = -5, B = -5$	$A = -10, B = 0$
Bo:rechazar	$A = 0, B = -10$	$A = -1, B = -1$

Ahora, póngase en el lugar de Ali. Ella permite analizar la matriz de pagos de la siguiente manera:

- Suponga que Bo juega a *testificar*. Entonces yo obtengo 5 años si testifico y 10 años si no lo hago, así que en ese caso testificar es mejor.
- Por otro lado, si Bo juega a *negarse*, entonces yo quedo libre si testifico y obtengo 1 año si me niego, así que testificar también es mejor en ese caso.
- Así que *no importa lo que Bo elija hacer*, sería mejor para mí testificar.

Ali ha descubierto que testificar es una **estrategia dominante** para el juego. Decimos que una estrategia s para el jugador p **domina fuertemente** la estrategia s' si el resultado para s es mejor para p que el resultado para s', para cada elección de estrategias por los otros jugadores. La estrategia s **domina débilmente** a s' si s es mejor que s' en al menos un perfil de estrategia y no peor en ningún otro. Una estrategia dominante es una estrategia que domina a todas las demás. Un supuesto común en la teoría de juegos es que un *jugador racional siempre elegirá una estrategia dominante y evitará una estrategia dominada*. Siendo racional, o al menos no queriendo ser considerado irracional, Ali elige la estrategia dominante.

Estrategia dominante

Domina fuertemente

Domina débilmente

No es difícil ver que el razonamiento de Bo será idéntico: él también concluirá que *testificar* es una estrategia dominante para él, y elegirá jugarla. La solución del juego, según el análisis de estrategia dominante, será que ambos jugadores elijan *testificar*, y como consecuencia ambos cumplirán 5 años en prisión.

En una situación como esta, donde todos los jugadores eligen una estrategia dominante, entonces el resultado que se deriva se dice que es un **equilibrio de estrategia dominante**. Es un "equilibrio" porque ningún jugador tiene algún incentivo para desviarse por su parte de él: por definición, si lo hicieran, no podrían hacerlo mejor, y podrían hacerlo peor. En este sentido, el equilibrio de estrategia dominante es un concepto de solución muy fuerte.

Equilibrio de estrategia dominante

Volviendo al dilema del prisionero, podemos ver que el *dilema* es que el resultado del equilibrio de estrategia dominante en el que ambos jugadores testifican es peor para ambos jugadores que el resultado que obtendrían si ambos se negaran a testificar. El resultado (*negarse*, *negarse*) daría a ambos jugadores sólo un año en prisión, lo que sería mejor para *ambos* que los 5 años que cada uno serviría si eligieran el equilibrio de estrategia dominante.

¿Hay alguna manera de que Ali y Bo lleguen al resultado (*negarse*, *negarse*)? Es una opción *válida* para ambos negarse a testificar, pero es difícil ver cómo los agentes racionales podrían hacer esta elección, dada la forma en que se configura el juego. Recuerde, este es un juego no cooperativo: no se les permite hablar entre ellos, por lo que no pueden hacer un acuerdo vinculante para *negarse*.

Sin embargo, es posible llegar a la solución (*negarse*, *negarse*) si cambiamos las reglas del juego. Podríamos cambiarlo por un juego cooperativo en el que los agentes puedan llegar a un acuerdo vinculante. O podríamos cambiar a un **juego repetido** en el que los jugadores sepan que se volverán a encontrar; a continuación veremos cómo funciona esto. De manera alternativa, los jugadores pueden tener creencias morales que fomenten la cooperación y la equidad. Pero eso significaría que tienen diferentes funciones de utilidad y, de nuevo, estarían jugando un juego diferente.

La presencia de una estrategia dominante para un jugador en particular simplifica el proceso de toma de decisiones para ese jugador. Una vez que Ali se ha dado cuenta de que testificar es una estrategia dominante, no necesita invertir ningún esfuerzo en tratar de averiguar qué hará Bo, porque sabe que *no importa lo que haga Bo*, testificar sería su **mejor respuesta**. Sin embargo, la mayoría de los juegos no tienen ni estrategias dominantes ni equilibrios de estrategia dominante. Es raro que una sola estrategia sea la mejor respuesta a todas las posibles estrategias contrapartes.

Mejor respuesta

El siguiente concepto de solución que consideramos es más débil que el equilibrio de estrategia dominante, pero es mucho más aplicable. Se llama **equilibrio de Nash**, y lleva el nombre de John Forbes Nash, Jr. (1928–2015), quien lo estudió en su tesis de doctorado de 1950, trabajo por el cual fue galardonado con el Premio Nobel en 1994.

Equilibrio de Nash

Un perfil de estrategia es un equilibrio de Nash si ningún jugador puede cambiar de manera unilateral su estrategia y como consecuencia recibir un pago más alto, bajo la suposición de que los otros jugadores se quedaron con sus elecciones de estrategia. Por lo tanto, en un equilibrio de Nash, cada jugador está jugando de manera simultánea una mejor respuesta a las elecciones de sus contrapartes. Un equilibrio de Nash representa un punto estable en un juego: estable en el sentido de que no hay incentivo racional para que ningún jugador se desvíe. Sin embargo, los equilibrios de Nash son puntos estables *locales*: como veremos, un juego puede contener múltiples equilibrios de Nash.

Dado que una estrategia dominante es una mejor respuesta a *todas* las estrategias de las contrapartes, se deduce que cualquier equilibrio de estrategia dominante también debe ser un equilibrio de Nash (Ejercicio 18.EQIB). En el dilema del prisionero, por lo tanto, hay un equilibrio de estrategia dominante único, que también es el equilibrio único de Nash.

	Ali:*l*	*Ali*:*r*
Bo:*t*	$A = 10, B = 10$	$A = 0, B = 0$
Bo:*b*	$A = 0, B = 0$	$A = 1, B = 1$

El siguiente ejemplo de juego demuestra, primero, que a veces los juegos no tienen estrategias dominantes, y segundo, que algunos juegos tienen múltiples equilibrios de Nash.

Es fácil verificar que no hay estrategias dominantes en este juego, para ninguno de los jugadores y, por lo tanto, no hay equilibrio de estrategia dominante. Sin embargo, los perfiles de estrategia (t,l) y (b,r) son ambos equilibrios de Nash. Ahora, claramente es de interés para ambos agentes apuntar al *mismo* equilibrio de Nash, ya sea (t,l) o (b,r), pero dado que estamos en el dominio de la teoría de juegos *no cooperativos*, los jugadores deben hacer sus elecciones de forma independiente, sin ningún conocimiento de las elecciones de los demás, y sin ninguna forma de llegar a un acuerdo con ellos. Este es un ejemplo de un **problema de coordinación**: los jugadores quieren coordinar sus acciones a nivel global, de modo que ambos elijan acciones que conduzcan al mismo equilibrio, pero deben hacerlo utilizando sólo la toma de decisiones local.

Se han propuesto varias formas de resolver problemas de coordinación. Una idea es la de los **puntos focales**. Un punto focal en un juego es un resultado que de alguna manera se destaca para los jugadores como un resultado "obvio" en el que coordinar sus elecciones. Esto por supuesto no es una definición precisa, lo que significa dependerá del juego en cuestión. En el ejemplo

Puntos focales

anterior, sin embargo, hay un punto focal obvio: el resultado (t,l) daría a ambos jugadores una utilidad mayor de la que obtendrían si se coordinaran en (b,r). Desde el punto de vista de la teoría de juegos, ambos resultados son equilibrios de Nash, pero sería un jugador perverso quien esperara coordinarse en (b,r).

Algunos juegos no tienen equilibrios de Nash en estrategias puras, como ilustra el siguiente juego, llamado **centavos coincidentes**. En este juego, Ali y Bo eligen de manera simultánea un lado de una moneda coincidente, ya sea cara o cruz: si hacen las mismas elecciones, entonces Bo le da a Ali \$1, mientras que, si hacen elecciones diferentes, entonces Ali le da a Bo \$1:

Centavos coincidentes

	Ali:cara	*Ali:cruz*
Bo:cara	$A = 1, B = -1$	$A = -1, B = 1$
Bo:cruz	$A = -1, B = 1$	$A = 1, B = -1$

Invitamos al lector a comprobar que el juego no contiene estrategias dominantes, y que ningún resultado es un equilibrio de Nash en estrategias puras: en cada resultado, un jugador se arrepiente de su elección, y preferiría haber elegido de manera diferente, dada la elección del otro jugador.

Para encontrar un equilibrio de Nash el truco es usar estrategias mixtas para permitir a los jugadores que se aleatoricen sobre sus elecciones. Nash demostró que *cada juego tiene al menos un equilibrio de Nash en estrategias mixtas*. Esto explica por qué el equilibrio de Nash es un concepto de solución tan importante: otros conceptos de solución, como el equilibrio de estrategia dominante, no están garantizados para existir para cada juego, pero siempre obtenemos una solución si buscamos equilibrios de Nash con estrategias mixtas.

En el caso de los centavos coincidentes, tenemos un equilibrio de Nash en estrategias mixtas si ambos jugadores eligen *cara* y *cruz* con igual probabilidad. Para ver que este resultado es de hecho un equilibrio de Nash, suponga que uno de los jugadores eligió un resultado con una probabilidad distinta de 0.5. Entonces el otro jugador podría aprovechar eso, poniendo todo su peso detrás de una estrategia particular. Por ejemplo, suponga que Bo jugó *cara* con probabilidad 0.6 (y, por lo tanto, *cruz* con probabilidad 0.4). Entonces, Ali haría mejor jugando *cara* con certeza. De este modo, es fácil ver que Bo jugando *cara* con probabilidad 0.6 no podría formar parte de ningún equilibrio de Nash.

18.2.2 Bienestar social

La perspectiva principal en la teoría de juegos es la de los jugadores dentro del juego tratando de obtener los mejores resultados que puedan para ellos mismos. Sin embargo, a veces es instructivo adoptar una perspectiva diferente. Suponga que fuera una entidad benevolente y omnisciente mirando el juego, y fuera capaz de *elegir* el resultado. Siendo benevolente, quiere elegir el mejor resultado general, el resultado que sería mejor para la sociedad en su conjunto, por así decirlo. ¿Cómo debería elegir? ¿Qué criterios podría aplicar? Aquí es donde entra en juego la noción de **bienestar social**.

Bienestar social

Probablemente el criterio de bienestar social más importante y menos controvertido es que debería evitar resultados que *desperdicien* utilidad. Este requisito se captura en el concepto de **Optimalidad de Pareto**, que lleva el nombre del economista italiano Vilfredo Pareto (1848-1923). Un resultado es Pareto óptimo si no hay otro resultado que haría que un jugador estuviera mejor sin hacer que alguien más estuviera peor. Si elige un resultado que no es Pareto óptimo, entonces desperdicia utilidad en el sentido de que podría haber dado más utilidad, a al menos un agente, sin quitarle a otros agentes.

Optimalidad de Pareto

El **bienestar social utilitario** es una medida de cuán bueno es un resultado en el agregado. El bienestar social utilitario de un resultado es la suma de las utilidades dadas a los jugadores por

Bienestar social utilitario

ese resultado. Hay dos dificultades clave con el bienestar social utilitario. La primera es que considera la suma pero no la *distribución* de las utilidades entre los jugadores, por lo que podría llevar a una distribución muy desigual si eso sucede para maximizar la suma. La segunda dificultad es que asume una *escala común* para las utilidades. Muchos economistas argumentan que esto es imposible de establecer porque la utilidad (a diferencia del dinero) es una cantidad subjetiva. Si estamos tratando de decidir cómo dividir un lote de galletas, ¿deberíamos dárselas todas al monstruo de la utilidad que dice: "Amo las galletas mil veces más que cualquier otra persona"? Eso maximizaría la utilidad total autoinformada, pero no parece correcto.

Bienestar social igualitario

Coeficiente de Gini

La cuestión de cómo se distribuye la utilidad entre los jugadores es abordada por la investigación en **bienestar social igualitario**. Por ejemplo, una propuesta sugiere que deberíamos maximizar la utilidad esperada del miembro más desfavorecido de la sociedad, un enfoque maximin. Otras métricas son posibles, incluyendo el **coeficiente de Gini**, que resume cuán uniforme se distribuye la utilidad entre los jugadores. Las principales dificultades con tales propuestas es que permiten sacrificar una gran cantidad de bienestar total por pequeñas ganancias distributivas, y, al igual que el utilitarismo simple, todavía están a merced del monstruo de la utilidad.

Aplicar estos conceptos al juego del dilema del prisionero, introducido antes, explica por qué se le llama un dilema. Recuerde que (*testificar*, *testificar*) es un equilibrio de estrategia dominante, y el único equilibrio de Nash. Sin embargo, este es el único resultado que *no* es Pareto óptimo. El resultado (*rechazar*, *rechazar*) maximiza tanto el bienestar social utilitario como el igualitario. El dilema en el dilema del prisionero surge porque un concepto de solución muy fuerte (equilibrio de estrategia dominante) conduce a un resultado que en esencia falla en cada prueba de lo que cuenta como un resultado razonable desde el punto de vista de la "sociedad". Aun así, no hay una forma clara para que los jugadores individuales lleguen a una mejor solución.

Computando equilibrios

Ahora consideremos las principales preguntas computacionales asociadas con los conceptos discutidos antes. Primero consideraremos estrategias puras, donde no se permite la aleatorización.

Si los jugadores sólo tienen un número finito de opciones posibles, entonces la búsqueda exhaustiva permite ser utilizada para encontrar equilibrios: iterar a través de cada perfil de estrategia posible y verificar si algún jugador tiene una desviación beneficiosa de ese perfil; si no, entonces es un equilibrio de Nash en estrategias puras. Las estrategias dominantes y los equilibrios de estrategia dominante son calculados por algoritmos similares. Sin embargo, el número de perfiles de estrategia posibles para n jugadores cada uno con m posibles acciones, es m^n; es decir, grande para una búsqueda exhaustiva.

Mejor respuesta miope

Un enfoque alternativo, que funciona bien en algunos juegos, es la **mejor respuesta miope** (también conocida como **mejor respuesta iterada**): comienza eligiendo un perfil de estrategia al azar; luego, si algún jugador no está jugando su elección óptima dadas las elecciones de los demás, cambia su elección a una óptima y repite el proceso. El proceso convergerá si conduce a un perfil de estrategia en el que cada jugador esté tomando una elección óptima, dadas las elecciones de los demás, un equilibrio de Nash, en otras palabras. Para algunos juegos, la mejor respuesta miope no converge, pero para algunas clases importantes de juegos, se garantiza que convergerá.

El cálculo de los equilibrios de estrategia mixta es algorítmicamente mucho más intrincado. Para mantener las cosas simples, nos centraremos en los métodos para juegos de suma cero y comentaremos de manera breve sobre su extensión a otros juegos al final de esta sección.

Juegos de suma cero

En 1928, Von Neumann desarrolló un método para encontrar la estrategia mixta óptima para **juegos de suma cero** de dos jugadores; juegos en los que los pagos siempre suman cero (o un valor constante, como se explica en la página 147). Claramente, Morra es tal juego. Para juegos de suma cero de dos jugadores sabemos que los pagos son iguales y opuestos, por lo que sólo

necesitamos considerar los pagos de un solo jugador, que será el maximizador (al igual que en el Capítulo 5). Para Morra, elegimos al jugador par E para ser el maximizador, por lo que podemos definir la matriz de pagos por los valores $U_E(e,o)$, el pago a E si E hace e y O hace o. (Por conveniencia llamamos a la jugadora E "ella" y a O "él"). El método de Von Neumann se llama técnica **maximin**, y funciona de la siguiente manera:

Maximin

- Suponga que cambiamos las reglas de la siguiente manera: primero E elige su estrategia y la revela a O. Luego O elige su estrategia, con conocimiento de la estrategia de E. Finalmente, evaluamos el pago esperado del juego basado en las estrategias elegidas. Esto nos da un juego de turnos al que podemos aplicar el algoritmo **minimax** estándar del Capítulo 5. Suponga que esto da un resultado $U_{E,O}$. Claramente, este juego favorece a O, por lo que la verdadera utilidad U del juego original (desde el punto de vista de E) es *al menos* $U_{E,O}$. Por ejemplo, si sólo miramos estrategias puras, el árbol de juego minimax tiene un valor de raíz de -3 (ver Figura 18.2(a)), por lo que sabemos que $U \geq -3$.
- Ahora suponga que cambiamos las reglas para, primero, obligar a O a revelar su estrategia, seguido de E. Entonces el valor minimax de este juego es $U_{O,E}$, y porque este juego favorece a E sabemos que U es *como máximo* $U_{O,E}$. Con estrategias puras, el valor es +2 (ver Figura 18.2(b)), así que sabemos $U \leq +2$.

Combinando estos dos argumentos, vemos que la verdadera utilidad U de la solución al juego original debe satisfacer

$$U_{E,O} \leq U \leq U_{O,E} \qquad \text{o en este caso,} \qquad -3 \leq U \leq 2.$$

Para precisar el valor de U, necesitamos dirigir nuestro análisis a las estrategias mixtas. Primero, observe lo siguiente: *una vez que el primer jugador ha revelado una estrategia, el segundo jugador podría elegir una estrategia pura*. La razón es simple: si el segundo jugador juega una estrategia mixta, $[p{:}uno;(1-p){:}dos]$, su utilidad esperada es una combinación lineal $(p \cdot U_{uno} + (1-p) \cdot U_{dos})$ de las utilidades de las estrategias puras, U_{uno} y U_{dos}. Esta combinación lineal nunca es mejor que la mejor de U_{uno} y U_{dos}, por lo que el segundo jugador puede elegir la mejor.

Con esta observación en mente, los árboles minimax permiten ser pensados como teniendo de manera infinita múltiples ramas en la raíz, correspondientes a las estrategias mixtas infinitas que el primer jugador puede elegir. Cada uno de estos conduce a un nodo con dos ramas correspondientes a las estrategias puras para el segundo jugador. Podemos representar estos árboles infinitos de manera finita teniendo una elección "parametrizada" en la raíz:

- Si E elige primero, la situación es como se muestra en la Figura 18.2(c). E elige la estrategia $[p{:}uno;(1-p){:}dos]$ en la raíz, y luego O elige una estrategia pura (y, por lo tanto, una jugada) dado el valor de p. Si O elige *uno*, el pago esperado (para E) es $2p - 3(1-p) = 5p - 3$; si O elige *dos*, el pago esperado es $-3p + 4(1-p) = 4 - 7p$. Podemos dibujar estos dos pagos como líneas rectas en un grafo, donde p varía de 0 a 1 en el eje x, como se muestra en la Figura 18.2(e). O, el minimizador, siempre elegirá la línea más baja de las dos, como se muestra por las líneas gruesas en la figura. Por lo tanto, lo mejor que E puede hacer en la raíz es elegir p para que esté en el punto de intersección, que es donde

  $$5p - 3 = 4 - 7p \qquad \Rightarrow \qquad p = 7/12.$$

  La utilidad para E en este punto es $U_{E,O} = -1/12$.

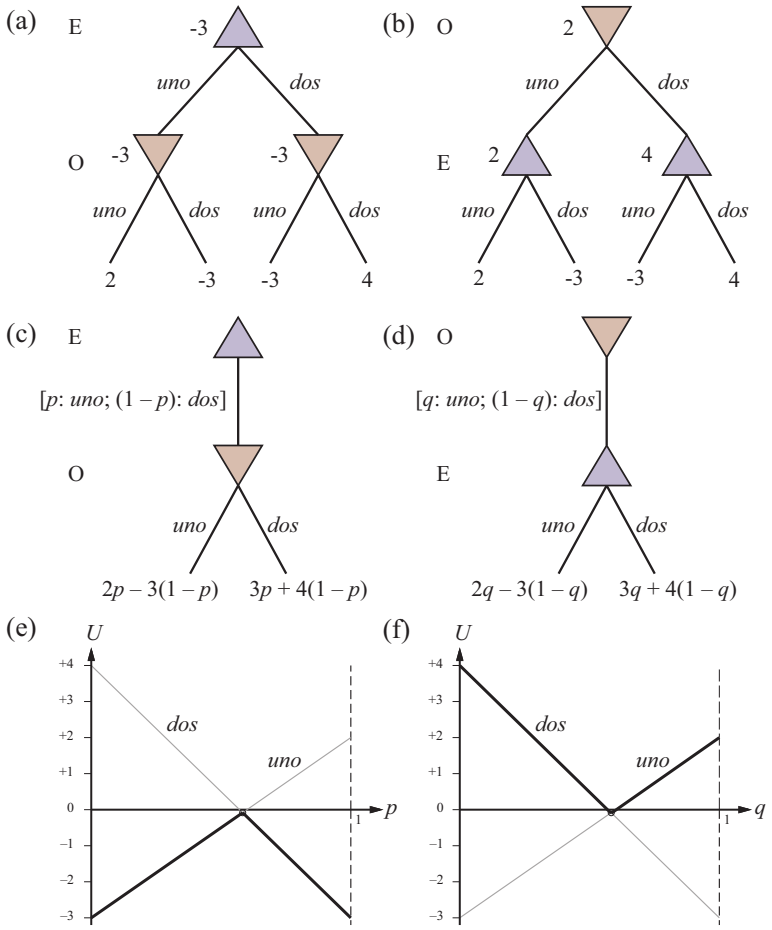


Figura 18.2 (a) y (b): Árboles de juego minimax para Morra de dos dedos si los jugadores se turnan para jugar estrategias puras. (c) y (d): Árboles de juego parametrizados donde el primer jugador juega una estrategia mixta. Los pagos dependen del parámetro de probabilidad (p o q) en la estrategia mixta. (e) y (f): Para cualquier valor particular del parámetro de probabilidad, el segundo jugador elegirá la "mejor" de las dos acciones, por lo que el valor de la estrategia mixta del primer jugador está dado por las líneas pesadas. El primer jugador elegirá el parámetro de probabilidad para la estrategia mixta en el punto de intersección.

- Si O se mueve primero, la situación es como se muestra en la Figura 18.2(d). O elige la estrategia $[q\!:\!uno;(1-q)\!:\!dos]$ en la raíz, y luego E elige un movimiento dado el valor de q. Los pagos son $2q - 3(1-q) = 5q - 3$ y $-3q + 4(1-q) = 4 - 7q$.[2] De nuevo, la Figura 18.2(f) muestra que lo mejor que O puede hacer en la raíz es elegir el punto de intersección:

  $$5q - 3 = 4 - 7q \qquad \Rightarrow \qquad q = 7/12.$$

  La utilidad para E en este punto es $U_{O,E} = -1/12$.

[2] Es una coincidencia que estas ecuaciones sean las mismas que las de p; la coincidencia surge porque $U_E(uno,dos) = U_E(dos,uno) = -3$. Esto también explica por qué la estrategia óptima es la misma para ambos jugadores.

Ahora sabemos que la verdadera utilidad del juego original está entre −1/12 y −1/12; es decir, es −1/12! (La conclusión es que es mejor ser *O* que *E* si está jugando este juego). Además, la verdadera utilidad se logra con la estrategia mixta [7/12:*uno*; 5/12:*dos*], que debería ser jugada por ambos jugadores. Esta estrategia se llama el **equilibrio maximin** del juego, y es un equilibrio de Nash. Note que cada estrategia componente en una estrategia mixta de equilibrio tiene la misma utilidad esperada. En este caso, tanto *uno* como *dos* tienen la misma utilidad esperada, −1/12, como la estrategia mixta en sí.

Equilibrio maximin

Nuestro resultado para Morra de dos dedos es un ejemplo del resultado general de Von Neumann: *cada juego de suma cero de dos jugadores tiene un equilibrio maximin cuando se permiten estrategias mixtas*. Además, cada equilibrio de Nash en un juego de suma cero es un maximin para ambos jugadores. Un jugador que adopta la estrategia maximin tiene dos garantías: Primero, ninguna otra estrategia permite hacerlo mejor contra un oponente que juega bien (aunque algunas otras estrategias podrían ser mejores para explotar a un oponente que comete errores irracionales). Segundo, el jugador continúa haciéndolo igual de bien incluso si la estrategia se revela al oponente.

El algoritmo general para encontrar equilibrios maximin en juegos de suma cero es algo más complicado de lo que podrían sugerir las Figuras 18.2(e) y (f). Cuando hay *n* acciones posibles, una estrategia mixta es un punto en el espacio *n*-dimensional y las líneas se convierten en hiperplanos. También es posible que algunas estrategias puras para el segundo jugador estén dominadas por otras, por lo que no son óptimas contra *ninguna* estrategia para el primer jugador. Después de eliminar todas estas estrategias (lo que podría tener que hacerse de manera repetida), la elección óptima en la raíz es el punto de intersección más alto (o más bajo) de los hiperplanos restantes.

Encontrar esta elección es un ejemplo de un problema de **programación lineal**: maximizar una función objetivo sujeta a restricciones lineales. Estos problemas permiten resolverse con técnicas estándar en tiempo polinomial en el número de acciones (y en el número de bits utilizados para especificar la función de recompensa, si quiere ponerse técnico).

La pregunta sigue siendo, ¿qué debería hacer un agente racional al jugar un solo juego de Morra? El agente racional habrá derivado el hecho de que [7/12:*uno*; 5/12:*dos*] es la estrategia de equilibrio maximin, y asumirá que esto es conocimiento mutuo con un oponente racional. El agente podría usar un dado de 12 lados o un generador de números aleatorios para elegir de manera aleatoria según esta estrategia mixta, en cuyo caso el pago esperado sería −1/12 para *E*. O el agente podría decidir jugar *uno*, o *dos*. En cualquier caso, el pago esperado sigue siendo -1/12 para *E*. Elegir de manera unilateral una acción particular no perjudica el pago esperado, pero permitir que el otro agente sepa que uno ha tomado tal decisión unilateral *sí* afecta el pago esperado, porque entonces el oponente puede ajustar la estrategia en consecuencia.

Encontrar equilibrios en juegos no suma cero es algo más complicado. El enfoque general tiene dos pasos: (1) Enumerar todos los posibles subconjuntos de acciones que podrían formar estrategias mixtas. Por ejemplo, primero intenta todos los perfiles de estrategia donde cada jugador usa una sola acción, luego aquellos donde cada jugador usa una o dos acciones, y así sucesivamente. Esto es exponencial en el número de acciones y, por lo tanto, sólo se aplica a juegos relativamente pequeños. (2) Para cada perfil de estrategia enumerado en (1), verificar si es un equilibrio. Esto se hace resolviendo un conjunto de ecuaciones e inecuaciones que son similares a las utilizadas en el caso de suma cero. Para dos jugadores estas ecuaciones son lineales y se resuelven con técnicas básicas de programación lineal, pero para tres o más jugadores son no lineales y pueden ser muy difíciles de resolver.

18.2.3 Juegos repetidos

Juego repetido

Juego de etapa

Hasta ahora, sólo hemos examinado juegos que duran un solo movimiento. El tipo más simple de juego de varios movimientos es el **juego repetido** (también llamado **juego iterado**), en el que los jugadores juegan de manera repetida rondas de un juego de un solo movimiento, llamado el **juego de etapa**. Una estrategia en un juego repetido especifica una elección de acción para cada jugador en cada paso de tiempo para cada posible historia de elecciones anteriores de los jugadores.

Primero, veamos el caso en el que el juego de etapa se repite un número fijo, finito y mutuamente conocido de rondas, todas estas condiciones son necesarias para que el siguiente análisis funcione. Suponga que Ali y Bo están jugando una versión repetida del dilema del prisionero, y que ambos saben que deben jugar 100 rondas del juego. En cada ronda, se les preguntará si *testificar* o *negarse*, y recibirán una recompensa por esa ronda de acuerdo con las reglas del dilema del prisionero que vimos con anterioridad.

Al final de las 100 rondas, encontramos la recompensa total para cada jugador sumando las recompensas de ese jugador en las 100 rondas. ¿Qué estrategias deberían elegir Ali y Bo para jugar este juego? Considere el siguiente argumento. Ambos saben que la ronda 100 no será un juego repetido, es decir, su resultado no tiene efecto en las rondas futuras. Entonces, en la ronda 100, en efecto están jugando un solo juego del dilema del prisionero.

Como antes vimos, el resultado de la ronda 100 será (*testificar*, *testificar*), la estrategia de equilibrio dominante para ambos jugadores. Pero una vez que se determina la ronda 100, la ronda 99 no tiene efecto en las rondas posteriores, por lo que también dará (*testificar*, *testificar*). Por este argumento inductivo, ambos jugadores elegirán *testificar* en cada ronda, obteniendo una sentencia total de prisión de 500 años cada uno. Este tipo de razonamiento se conoce como **inducción hacia atrás**, y tiene un papel fundamental en la teoría de juegos.

Inducción hacia atrás

Sin embargo, si eliminamos una de las tres condiciones: fijo, finito o mutuamente conocido, entonces el argumento inductivo no se sostiene. Supongamos que el juego se repite un número *infinito* de veces. Matemáticamente, una estrategia para un jugador en un juego repetido de manera infinitas es una función que mapea cada posible historia finita de ese juego a una elección en el juego de etapa para ese jugador en la ronda correspondiente. Así, una estrategia mira lo que sucedió antes en el juego y decide qué elección hacer en la ronda actual. Pero no podemos almacenar una tabla infinita en una computadora finita. Necesitamos un modelo *finito* de estrategias para juegos que se jugarán un número *infinito* de rondas. Por esta razón, es estándar representar estrategias para juegos repetidos de manera infinita como máquinas de estado finito (FSM) con salida.

Ojo por ojo

La Figura 18.3 ilustra una serie de estrategias de FSM para el dilema del prisionero iterado. Considere la estrategia **Ojo por Ojo**. Cada óvalo es un estado de la máquina, y dentro del óvalo está la elección que haría la estrategia si la máquina estuviera en ese estado. Desde cada estado, tenemos una arista saliente para cada posible elección del agente contrario: seguimos la arista saliente correspondiente a la elección hecha por el otro para encontrar el siguiente estado de la máquina. Finalmente, un estado está etiquetado con una flecha entrante, indicando que es el estado inicial. Así, con Ojo por Ojo, la máquina comienza en el estado de negarse; si el agente contrario juega a *negarse*, entonces se queda en el estado de *negarse;* pero si el contrario juega a *testificar*, pasa al estado de *testificar*. Permanecerá en el estado de *testificar* mientras su contraparte juegue a *testificar*, pero si alguna vez su contraparte juega a *negarse*, volverá al estado de *negarse*. En resumen, Ojo por Ojo comenzará eligiendo *negarse*, y luego copiará lo que su contraparte hizo en la ronda anterior.

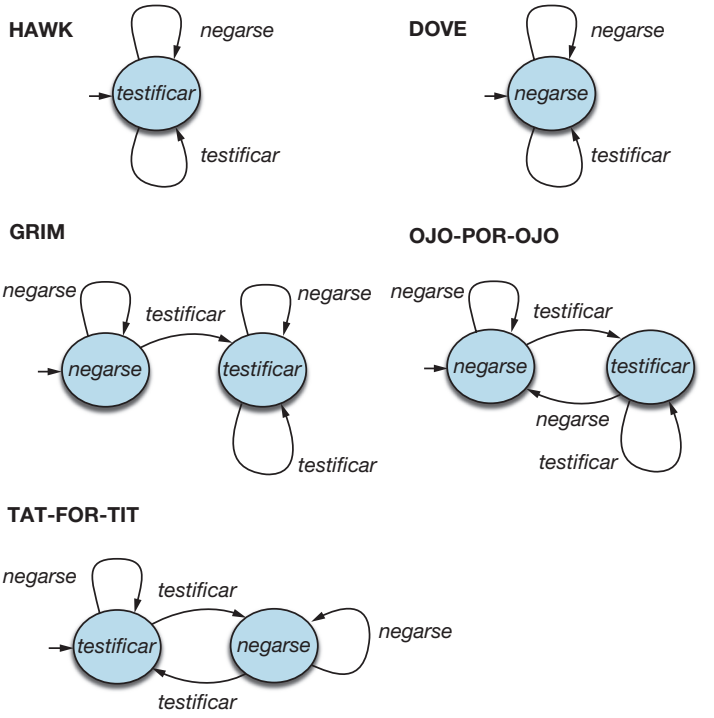


Figura 18.3 Algunas estrategias comunes, con nombres coloridos, de máquinas de estado finito para el dilema del prisionero repetido de manera infinita.

Las estrategias de HAWK y DOVE son más simples: HAWK juega a *testificar* en cada ronda, mientras que DOVE juega a *negarse* en cada ronda. La estrategia GRIM es algo similar a OJO-POR-OJO, pero con una diferencia importante: si alguna vez su contraparte juega a *testificar*, entonces en esencia se convierte en HAWK: que juega a *testificar* para siempre. Mientras que OJO-POR-OJO es indulgente, en el sentido de que responderá a un *negarse* posterior reciprocando lo mismo, con GRIM no hay vuelta atrás. Sólo jugar a *testificar* una vez resultará en un castigo (jugar a *testificar*) que dura para siempre. (¿Puedes ver lo que hace OJO-POR-OJO?)

El siguiente problema con los juegos repetidos de manera infinita es cómo medir la utilidad de una secuencia infinita de recompensas. Aquí, nos centraremos en el enfoque del **límite de las medias**, en esencia, esto significa tomar el promedio de las utilidades recibidas a lo largo de la secuencia infinita. Con este enfoque, dada una secuencia infinita de recompensas $(U_0, U_1, U_2, ...)$, definimos la utilidad de la secuencia para el jugador correspondiente como:

Límite de las medias

$$\lim_{T \to \infty} \frac{1}{T} \sum_{t=0}^{T} U_t.$$

Este valor no puede garantizarse que converja para secuencias arbitrarias de utilidades, pero se garantiza que lo hará para las secuencias de utilidad que se generan si usamos estrategias de FSM. Para ver esto, observe que, si las estrategias de FSM juegan entre sí, entonces *de manera eventual, las FSM volverán a entrar en una configuración en la que estaban antes, en cuyo punto comenzarán a repetirse ellas mismas*. Más preciso, cualquier secuencia de utilidad generada por

estrategias de FSM consistirá en una secuencia no repetida finita (posiblemente vacía), seguida de una secuencia finita no vacía que con frecuencia se repite de manera infinita. Para calcular la utilidad promedio recibida por un jugador a lo largo de esa secuencia infinita, tenemos que calcular el promedio sobre la secuencia repetida finita.

En lo que sigue, asumiremos que los jugadores en un juego repetido de manera infinita sólo eligen una máquina de estado finito para jugar el juego en su nombre. No imponemos ninguna restricción en estas máquinas: permiten ser tan grandes y elaboradas como los jugadores quieran. Cuando todos los jugadores han elegido una máquina de estado finito para jugar en su nombre, entonces podemos calcular los pagos para cada jugador utilizando el límite de los medios de enfoque como ya antes se describió. De esta manera, un juego repetido de manera infinita se reduce a un juego de forma normal, aunque uno con estrategias posibles infinitas para cada jugador.

Veamos qué sucede cuando jugamos el dilema del prisionero repetido de manera infinita usando algunas estrategias de la Figura 18.3. Primero, supongamos que Ali y Bo eligen DOVE.

	0	1	2	3	4	5	...	
Ali: DOVE	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	...	utilidad = −1
Bo: DOVE	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	...	utilidad = −1

No es difícil ver que este par de estrategias no forma un equilibrio de Nash: cualquiera de los jugadores habría hecho mejor en alterar su elección a HAWK. Entonces, supongamos que Ali cambia a HAWK:

	0	1	2	3	4	5	...	
Ali: HAWK	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	...	utilidad = 0
Bo: DOVE	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	...	utilidad = −10

Este es el peor resultado posible para Bo; y este par de estrategias no es un equilibrio de Nash. Bo habría hecho mejor eligiendo también HAWK:

	0	1	2	3	4	5	...	
Ali: HAWK	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	...	utilidad = −5
Bo: HAWK	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	...	utilidad = −5

Este par de estrategias *sí* forma un equilibrio de Nash, pero no uno muy interesante, nos lleva más o menos de vuelta a donde comenzamos en la versión de un solo tiro del juego, con ambos jugadores testificando uno contra el otro. Ilustra una propiedad clave de los juegos repetidos de manera infinita: los equilibrios de *Nash del juego de etapas se mantendrán como equilibrios en una versión infinitamente repetida del juego.*

Sin embargo, nuestra historia aún no ha terminado. Suponga que Bo cambió a GRIM:

	0	1	2	3	4	5	...	
Ali: HAWK	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	...	utilidad = −5
Bo: GRIM	*rechaza*	*testifica*	*testifica*	*testifica*	*testifica*	*testifica*	...	utilidad = −5

Aquí, Bo no hace peor que jugar HAWK: en la primera ronda, Ali juega *testifica* mientras Bo juega *rechaza*, pero esto desencadena a Bo en testificar para siempre después: la pérdida de utilidad en la primera ronda desaparece en el límite. En general, los dos jugadores obtienen la misma utilidad como si ambos hubieran jugado HAWK. Pero aquí está la cosa: estas estrategias no forman un equilibrio de Nash porque esta vez, Ali tiene una desviación beneficiosa, a GRIM. Si ambos jugadores eligen GRIM, entonces esto es lo que sucede:

	0	1	2	3	4	5	...	
Ali: GRIM	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	...	utilidad = −1
Bo: GRIM	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	*rechaza*	...	utilidad = −1

Los resultados y los pagos son los mismos que si ambos jugadores hubieran elegido DOVE, pero a diferencia de ese caso, GRIM jugando contra GRIM forma un equilibrio de Nash, y Ali y Bo logran de manera racional un resultado que es imposible en la versión de un solo tiro del juego.

Para ver que estas estrategias forman un equilibrio de Nash, suponga por contradicción que no lo hacen. Entonces un jugador, asume sin pérdida de generalidad que es Ali, tiene una desviación benéfica, en la forma de una estrategia FSM que daría un pago más alto que GRIM. Ahora, en algún momento esta estrategia tendría que hacer algo diferente de GRIM, de lo contrario obtendría la misma utilidad. Entonces, en algún momento debe jugar *testifica*. Pero entonces la estrategia GRIM de Bo cambiaría al modo de castigo, testificando de manera permanente en respuesta. En ese punto, Ali estaría condenada a recibir un pago de no más de −5: peor que el −1 que habría recibido eligiendo GRIM. Por lo tanto, ambos jugadores eligiendo GRIM forman un equilibrio de Nash en el dilema del prisionero repetido de modo infinito, dando un resultado sostenido de manera racional que es imposible en la versión de un solo tiro del juego.

Este es un ejemplo de una clase general de resultados llamados los **teoremas populares de Nash**, que caracterizan los resultados que permiten ser sostenidos por equilibrios de Nash en juegos repetidos de manera infinita. Digamos que el *valor de seguridad* de un jugador es el mejor pago que el jugador podría garantizar obtener. Entonces la forma general de los teoremas populares de Nash es que *cada resultado en el que cada jugador recibe al menos su valor de seguridad permite ser sostenido como un equilibrio de Nash en un juego repetido de manera infinita*. Las estrategias GRIM son la clave para los teoremas populares: la amenaza mutua de castigo si algún agente no juega su parte en el resultado deseado mantiene a los jugadores en línea. Pero funciona como un disuasivo sólo si el otro jugador cree que ha adoptado esta estrategia, o al menos que podría haberla adoptado.

Teoremas populares de Nash

También podemos obtener diferentes soluciones cambiando a los agentes, en lugar de cambiar las reglas de compromiso. Suponga que los agentes son máquinas de estado finito con n estados y están jugando un juego con $m > n$ pasos totales. Los agentes son, en consecuencia, incapaces de representar el número de pasos restantes y deben tratarlo como un desconocido. Por lo tanto, no pueden hacer la inducción hacia atrás y son libres de llegar al equilibrio más favorable (*rechaza*, *rechaza*) en el dilema del prisionero iterado. En este caso, la ignorancia es felicidad o, más bien, hacer que su oponente crea que usted es ignorante es bienaventuranza. Su éxito en estos juegos repetidos depende en gran medida de la *percepción* que tiene el otro jugador de usted como un bravucón o un simplón, y no de sus características reales.

18.2.4 Juegos secuenciales: La forma extensiva

En el caso general, un juego consiste en una secuencia de turnos que no necesariamente son todos iguales. Tales juegos se representan mejor con un árbol de juego, que los teóricos de juegos llaman la **forma extensiva**. El árbol incluye toda la misma información que vimos en la Sección 5.1: un estado inicial S_0, una función PLAYER(s) que indica qué jugador tiene el movimiento, una función ACTIONS(s) que enumera las posibles acciones, una función RESULT(s,a) que define la transición a un nuevo estado, y una función parcial UTILITY(s,p), que se define sólo en estados terminales, para dar el pago para cada jugador. Los juegos estocásticos permiten ser capturados introduciendo un jugador distinguido, *Azar*, que puede tomar acciones aleatorias. La "estrategia" de *Azar* es parte de la definición del juego, especificada como una distribución de probabilidad sobre acciones (los otros jugadores llegan a elegir su propia estrategia). Para representar juegos con acciones no deterministas, como el billar, dividimos la acción en dos partes: la acción del jugador en sí misma tiene un resultado determinista, y luego la *Azar* tiene un turno para reaccionar a la acción a su manera caprichosa.

Forma extensiva

Información perfecta

Por el momento, haremos una suposición simplificadora: suponemos que los jugadores tienen **información perfecta**. A grandes rasgos, la información perfecta significa que, cuando el juego les pide que tomen una decisión, saben dónde están en el árbol del juego: no tienen ninguna incertidumbre sobre lo que ha sucedido antes en el juego. Esta es, por supuesto, la situación en juegos como el ajedrez o el Go, pero no en juegos como el póker o el Kriegspiel. En la siguiente sección mostraremos cómo la forma extensa permite ser utilizada para capturar información imperfecta en los juegos, aunque por el momento asumiremos información perfecta.

Una estrategia en un juego de forma extensa de información perfecta es una función para un jugador que para cada uno de sus estados de decisión s dicta qué acción en ACTIONS(s) el jugador debe elegir para ejecutar. Cuando cada jugador ha seleccionado una estrategia, entonces el perfil de estrategia resultante trazará un camino en el árbol del juego desde el estado inicial S_0 hasta un estado terminal, y la función UTILITY define las utilidades que cada jugador recibirá entonces.

Dado este escenario, podemos aplicar de manera directa el aparato de equilibrios de Nash antes introducido para analizar juegos de forma extensa. Para poder calcular los equilibrios de Nash, usamos una generalización directa de la técnica de búsqueda minimax que vimos en el Capítulo 5. En la literatura sobre juegos de forma extensa, la técnica se llama inducción hacia atrás, ya vimos la inducción hacia atrás utilizada de manera informal para analizar el dilema del prisionero repetido de modo finito. La inducción hacia atrás utiliza la programación dinámica al trabajar hacia atrás desde los estados terminales hasta el estado inicial y al etiquetar de manera progresiva cada estado con un perfil de pago (una asignación de pagos a los jugadores) que se obtendría si el juego se jugara de manera óptima a partir de ese punto.

De manera más detallada, para cada estado no terminal s, si todos los hijos de s han sido etiquetados con un perfil de pago, entonces etiquetar s con un perfil de pago del estado hijo que maximiza el pago del jugador que toma la decisión en el estado s. (Si hay un empate, entonces elige de manera arbitraria; si tenemos nodos de azar, entonces calcula la utilidad esperada). El algoritmo de inducción hacia atrás está garantizado para terminar, y además se ejecuta en tiempo polinomial en el tamaño del árbol del juego.

A medida que el algoritmo realiza su trabajo, traza estrategias para cada jugador. Estas estrategias resultan ser estrategias de equilibrio de Nash, y el perfil de pago que etiqueta el estado inicial es un perfil de pago que se obtendría jugando estrategias de equilibrio de Nash. Por lo tanto, las estrategias de equilibrio de Nash para juegos de forma extensa permiten calcularse en tiempo polinomial usando la inducción hacia atrás; y dado que el algoritmo está garantizado para etiquetar el estado inicial con un perfil de pago, se deduce que todo juego de forma extensa tiene al menos un equilibrio de Nash en estrategias puras.

Estos resultados son atractivos, pero hay varias advertencias. Los árboles de juego se hacen muy grandes con rapidez, por lo que el tiempo de ejecución polinomial debe entenderse en ese contexto. Pero de manera más problemática, el equilibrio de Nash en sí mismo tiene algunas limitaciones cuando se aplica a juegos de forma extensa. Considera el juego en la Figura 18.4. El jugador 1 tiene dos movimientos disponibles: *hacia lo alto* o *abajo*. Si se mueve hacia *abajo*, entonces ambos jugadores reciben un pago de 0 (de manera independiente del movimiento seleccionado por el jugador 2). Si se mueve *hacia lo alto*, entonces el jugador 2 se enfrenta a la elección de moverse hacia *arriba* o *hacia abajo*: si se mueve *hacia abajo*, entonces ambos jugadores reciben un pago de 0, mientras que si se mueve hacia *arriba*, entonces ambos reciben 1.

La inducción hacia atrás nos indica que (*hacia lo alto*, *arriba*) es un equilibrio de Nash, lo que resulta en que ambos jugadores reciban un pago de 1. Sin embargo, (*hacia lo bajo*, *abajo*) también es un equilibrio de Nash, lo que resultaría en que ambos jugadores reciban un pago de 0. El jugador 2 está amenazando al jugador 1, al indicar que si se le pide que tome una decisión, elegirá *abajo*, lo que resultará en un pago igual a 0 para el jugador 1; en este caso, el jugador 1 no tiene una alternativa mejor que elegir *hacia lo bajo*. El problema es que la amenaza del jugador 2 (de

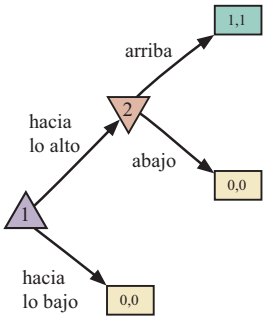


Figura 18.4 Un juego de forma extensa con un equilibrio de Nash contraintuitivo.

jugar *abajo*) no es una **amenaza creíble**, porque si el jugador 2 es realmente llamado a tomar la decisión, entonces elegirá *arriba*.

Amenaza creíble

Un refinamiento del equilibrio de Nash llamado **equilibrio de Nash perfecto en subjuegos** se ocupa de este problema. Para definirlo, necesitamos la idea de un **subjuego**. Cada estado de decisión en un árbol de juego (incluido el estado inicial) define un subjuego, el juego en la Figura 18.4, por lo tanto, contiene dos subjuegos, uno enraizado en el estado de decisión del jugador 1, uno enraizado en el estado de decisión del jugador 2. *Un perfil de estrategias entonces forma un equilibrio de Nash perfecto en subjuegos en un juego G si es un equilibrio de Nash en cada subjuego de G.* Aplicando esta definición al juego de la Figura 18.4, encontramos que (*hacia lo alto, arriba*) es perfecto en subjuegos, pero (*hacia lo bajo, abajo*) no lo es; porque elegir *abajo* no es un equilibrio de Nash del subjuego enraizado en el estado de decisión del jugador 2.

Equilibrio de Nash perfecto en subjuegos

Subjuego

Aunque necesitamos una nueva terminología para definir el equilibrio de Nash perfecto en subjuegos, no necesitamos nuevos algoritmos. Las estrategias calculadas a través de la inducción hacia atrás serán equilibrios de Nash perfectos en subjuegos, y se deduce que cada juego de forma extensa de información perfecta tiene un equilibrio de Nash perfecto en subjuegos, que es calculado en tiempo polinomial en el tamaño del árbol del juego.

Azar y movimientos simultáneos

Para representar juegos estocásticos, como el backgammon, en forma extensa, añadimos un jugador llamado *Azar*, cuyas elecciones están determinadas por una distribución de probabilidad.

Para representar movimientos simultáneos, como en el dilema del prisionero o el Morra de dos dedos, imponemos un orden arbitrario a los jugadores, pero tenemos la opción de afirmar que las acciones del jugador anterior no son observables para los jugadores posteriores: por ejemplo, Ali debe elegir *rechazar* o *testificar* primero, luego Bo elige, pero Bo no sabe qué elección hizo Ali en ese momento (también podemos representar el hecho de que el movimiento se revela más tarde). Sin embargo, asumimos que los jugadores siempre recuerdan todas *sus* acciones anteriores; esta suposición se llama **recuerdo perfecto**.

Capturando información imperfecta

Una característica clave de la forma extensa que la distingue de los árboles de juego que vimos en el Capítulo 5 es que permite capturar la observabilidad parcial. Los teóricos de juegos utilizan el término **información imperfecta** para describir situaciones en las que los jugadores no están seguros sobre el

Información imperfecta

estado actual del juego. Sin embargo, la inducción hacia atrás no funciona con juegos de información imperfecta que, en general, son más complejos de resolver que los juegos de información perfecta.

Vimos en la Sección 5.6 que un jugador en un juego parcialmente observable como Kriegspiel permite crear un árbol de juego sobre el espacio de **estados de creencia**. Con ese árbol, vimos que en algunos casos un jugador encuentra una secuencia de movimientos (una estrategia) que lleva a un jaque mate forzado sin importar en qué estado real comenzamos, y sin importar qué estrategia use el oponente. No obstante, las técnicas del Capítulo 5 no podían decirle a un jugador qué hacer cuando no hay un jaque mate garantizado. Si la mejor estrategia del jugador depende de la estrategia del oponente y viceversa, entonces minimax (o alfa-beta) por sí solo no permite encontrar una solución. La forma extensa *nos permite* encontrar soluciones porque representa los estados de creencia (los teóricos de juegos los llaman **conjuntos de información**) de *todos* los jugadores a la vez. A partir de esa representación podemos encontrar soluciones de equilibrio, tal como lo hicimos con los juegos de forma normal.

Conjuntos de información

Como ejemplo simple de un juego secuencial, coloque a dos agentes en el mundo 4 × 3 de la Figura 17.1 y hágales moverse de manera simultánea hasta que un agente llegue a una casilla de salida y obtenga la recompensa de esa casilla. Si especificamos que no se produce movimiento cuando los dos agentes intentan moverse a la misma casilla de manera simultánea (un problema común en muchas intersecciones de tráfico), entonces ciertas estrategias puras pueden quedarse atascadas para siempre. Por lo tanto, los agentes necesitan una estrategia mixta para desempeñarse bien en este juego: eligen al azar entre avanzar y quedarse quietos. Esto es lo que se hace para resolver colisiones de paquetes en redes Ethernet.

A continuación, consideraremos una variante muy simple de póker. La baraja sólo tiene cuatro cartas, dos ases y dos reyes. Se reparte una carta a cada jugador. El primer jugador tiene, por lo tanto, la opción de *subir* las apuestas del juego de 1 punto a 2, o *pasar*. Si el jugador 1 pasa, el juego termina. Si el jugador 1 sube, entonces el jugador 2 tiene la opción de *aceptar*, concediendo que el juego vale 2 puntos, o *retirarse*, concediendo el punto 1. Si el juego no termina con un retiro, entonces la recompensa depende de las cartas: es cero para ambos jugadores si tienen la misma carta; de lo contrario, el jugador con el rey paga las apuestas al jugador con el as.

El árbol de forma extensa para este juego se muestra en la Figura 18.5. El jugador 0 es *Azar*; los jugadores 1 y 2 están representados por triángulos. Cada acción se representa como una flecha con una etiqueta, correspondiente a *subir*, *pasar*, *aceptar* o *retirarse* o, para *Azar*, las cuatro

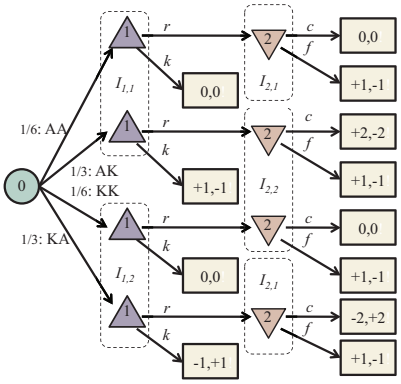


Figura 18.5 Forma extensa de una versión simplificada de póquer con dos jugadores y sólo cuatro cartas. Los movimientos son *r* (subir), f (retirarse), c (aceptar) y k (pasar).

posibles reparticiones ("AK" significa que el jugador 1 recibe un as y el jugador 2 un rey). Los estados terminales son rectángulos etiquetados por su recompensa para el jugador 1 y el jugador 2. Los conjuntos de información se muestran como cajas punteadas etiquetadas; por ejemplo, $I_{1,1}$ es el conjunto de información donde es el turno del jugador 1, y él sabe que tiene un as (pero no sabe qué tiene el jugador 2). En el conjunto de información $I_{2,1}$, es el turno del jugador 2 y ella sabe que tiene un as y que el jugador 1 ha subido, pero no sabe qué carta tiene el jugador 1. (Debido a los límites del papel bidimensional, este conjunto de información se muestra como dos cajas en lugar de una).

Una forma de resolver un juego extenso es convertirlo en un juego de forma normal. Recuerde que la forma normal es una matriz, cada fila de la cual está etiquetada con una estrategia pura para el jugador 1, y cada columna con una estrategia pura para el jugador 2. En un juego extenso, una estrategia pura para el jugador *i* corresponde a una acción para cada conjunto de información que involucra a ese jugador. Entonces en la Figura 18.5, una estrategia pura para el jugador 1 es "subir cuando estoy en $I_{1,1}$ (es decir, cuando tengo un as), y pasar cuando estoy en $I_{1,2}$ (cuando tengo un rey)". En la matriz de recompensas, a continuación, esta estrategia se llama *rk*. De manera similar, la estrategia *cf* para el jugador 2 significa "aceptar cuando tengo un as y retirarme cuando tengo un rey". Dado que este es un juego de suma cero, la matriz a continuación sólo da la recompensa para el jugador 1; el jugador 2 siempre tiene la recompensa opuesta:

	2:*cc*	2:*cf*	2:*ff*	2:*fc*
1:*rr*	0	-1/6	1	7/6
1:*kr*	-1/3	-1/6	5/6	2/3
1:*rk*	1/3	**0**	1/6	1/2
1:*kk*	0	**0**	0	0

Este juego es tan simple que tiene dos equilibrios de estrategia pura, mostrados en negrita: *cf* para el jugador 2 y *rk* o *kk* para el jugador 1. Pero, en general, podemos resolver juegos extensos convirtiéndolos en forma normal y luego encontrando una solución (por lo regular, en una estrategia mixta) utilizando métodos estándar de programación lineal. Eso funciona en teoría. Pero si un jugador tiene I conjuntos de información y a acciones por conjunto, entonces ese jugador tendrá a^I estrategias puras. En otras palabras, el tamaño de la matriz de forma normal es exponencial en el número de conjuntos de información, por lo que en la práctica el enfoque sólo funciona para árboles de juego diminutos, una docena de estados más o menos. Un juego como el póker Texas Sujétalos para dos jugadores tiene alrededor de 10^{18} estados, lo que hace que este enfoque sea completamente inviable.

¿Cuáles son las alternativas? En el Capítulo 5 vimos cómo la búsqueda alfa-beta podía manejar juegos de información perfecta con árboles de juego enormes generando el árbol de manera incremental, podando algunas ramas y evaluando de manera heurística los nodos no terminales. Pero ese enfoque no funciona bien para juegos con información imperfecta, por dos razones: primero, es más difícil podar, porque necesitamos considerar estrategias mixtas que combinan múltiples ramas, no una estrategia pura que siempre elige la mejor rama. En segundo lugar, es más difícil evaluar de manera heurística un nodo no terminal, porque estamos lidiando con conjuntos de información, no con estados individuales.

Koller *et al.* (1996) acudieron al rescate con una representación alternativa de extensos juegos, llamada la **forma de secuencia**, que es sólo lineal en el tamaño del árbol, en lugar de exponencial. En vez de representar estrategias, representa caminos a través del árbol; el número de caminos es igual al número de nodos terminales. Los métodos estándar de programación lineal permiten aplicarse de nuevo a esta representación. El sistema resultante permite resolver variantes de póker con 25 000 estados en un minuto o dos. Esto es una aceleración exponencial sobre el enfoque de forma normal, pero aún está lejos de manejar, digamos, el póquer Texas Sujétalos para dos jugadores, con 10^{18} estados.

Forma de secuencia

Si no podemos manejar 10^{18} estados, quizá podamos simplificar el problema cambiando el juego a una forma más simple. Por ejemplo, si tengo un as y estoy considerando la posibilidad de que la próxima carta me dé un par de ases, entonces no me importa el suit de la próxima carta; bajo las reglas del póker cualquier suit servirá igual de bien. Esto sugiere formar una **abstracción** del juego, uno en el que se ignoran los suits. El árbol de juego resultante será más pequeño por un factor de 4! = 24. Suponga que puedo resolver este juego más pequeño; ¿cómo se relacionará la solución de ese juego con el juego original? Si ningún jugador considera ir por un flush (la única mano donde los suits importan), entonces la solución para la abstracción también será una solución para el juego original. Sin embargo, si cualquier jugador está contemplando un flush, entonces la abstracción será sólo una solución aproximada (pero es posible calcular límites para el error).

Hay muchas oportunidades para la abstracción. Por ejemplo, en el punto de un juego donde cada jugador tiene dos cartas, si tengo un par de reinas, entonces las manos de los otros jugadores podrían ser abstraídas en tres clases: *mejor* (sólo un par de reyes o un par de ases), *igual* (par de reinas) o *peor* (todo lo demás). Sin embargo, esta abstracción podría ser demasiado gruesa. Una mejor abstracción dividiría *peor* en, digamos, *par medio* (nueves a través de jotas), *par bajo* y *sin par*. Estos ejemplos son abstracciones de estados; también es posible abstraer acciones. Por ejemplo, en lugar de tener una acción de apuesta para cada entero del 1 al 1 000, podríamos restringir las apuestas a 10^0, 10^1, 10^2 y 10^3. O podríamos eliminar una de las rondas de apuestas por completo. También podemos abstraer sobre los nodos de azar, considerando sólo un subconjunto de las posibles ofertas. Esto es equivalente a la técnica de despliegue utilizada en los programas de Go. Juntando todas estas abstracciones, podemos reducir los 10^{18} estados del póker a 10^7 estados, un tamaño que se resuelve con las técnicas actuales.

Vimos en el Capítulo 5 cómo programas de póker como Libratus y DeepStack pudieron derrotar a campeones humanos en el póker Texas Sujétalos de dos jugadores. Más reciente, el programa Pluribus pudo derrotar a campeones humanos en el póker de seis jugadores en dos formatos: cinco copias del programa en la mesa con un humano y una copia del programa con cinco humanos. Hay un enorme salto en complejidad aquí. Con un oponente, hay $\binom{50}{2}=1\,225$ posibilidades para las cartas ocultas del oponente. Pero con cinco oponentes hay $50elige10 \approx 10$ mil millones de posibilidades. Pluribus desarrolla una estrategia base desde el juego propio, luego modifica la estrategia durante el juego real para reaccionar a una situación específica. Pluribus utiliza una combinación de técnicas, incluyendo búsqueda de árbol de Monte Carlo, búsqueda limitada en profundidad y abstracción.

La forma extensiva es una representación versátil: permite manejar entornos parcialmente observables, multiagente, estocásticos, secuenciales, en tiempo real, la mayoría de los casos difíciles de la lista de propiedades del entorno en la página 43. Sin embargo, hay dos limitaciones a la forma extensiva en particular y a la teoría de juegos en general. Primero, no se maneja bien con estados y acciones continuas (aunque ha habido algunas extensiones al caso continuo; por ejemplo, la teoría de la **competencia de Cournot** utiliza la teoría de juegos para resolver problemas donde dos empresas eligen precios para sus productos de un espacio continuo). Segundo, la teoría de juegos asume que el juego es *conocido*. Partes del juego permiten ser especificadas como inobservables para algunos de los jugadores, pero debe ser conocido qué partes son inobservables. En casos en los que los jugadores aprenden la estructura desconocida del juego con el tiempo, el modelo comienza a descomponerse. Examinemos cada fuente de incertidumbre, y si cada una puede ser representada en la teoría de juegos.

Competencia de Cournot

Acciones: No hay una forma fácil de representar un juego donde los jugadores tienen que descubrir qué acciones están disponibles. Considere el juego entre los escritores de virus informáticos y los expertos en seguridad. Parte del problema es anticipar qué acción intentarán los escritores de virus a continuación.

Estrategias: La teoría de juegos es muy buena para representar la idea de que las estrategias de los otros jugadores son en inicio desconocidas, siempre y cuando asumamos que todos los agentes son racionales. La teoría no dice qué hacer cuando los otros jugadores son menos que totalmente racionales. La noción de un **equilibrio de Bayes-Nash** aborda de manera parcial este punto: es un equilibrio con respecto a la distribución de probabilidad previa de un jugador sobre las estrategias de los otros jugadores, es decir, expresa las creencias de un jugador sobre las estrategias probables de los otros jugadores.

Equilibrio de Bayes-Nash

Azar: Si un juego depende del lanzamiento de un dado, es muy fácil modelar un nodo de azar con distribución uniforme sobre los resultados. ¿Pero qué pasa si es posible que el dado sea injusto? Podemos representar eso con otro nodo de azar, más arriba en el árbol, con dos ramas para "el dado es justo" y "el dado es injusto", de tal manera que los nodos correspondientes en cada rama estén en el mismo conjunto de información (es decir, los jugadores no saben si el dado es justo o no). ¿Y qué pasa si sospechamos que el otro oponente sí sabe? Entonces añadimos *otro* nodo de azar, con una rama representando el caso en que el oponente sí sabe, y una en la que el oponente no sabe.

Utilities: ¿Y si no conocemos los bienes y servicios de nuestro oponente? De nuevo, eso permite ser modelado con un nodo de azar, de tal manera que el otro agente conoce sus propias utilidades en cada rama, pero nosotros no. ¿Pero qué pasa si no conocemos nuestras *propias* utilidades? Por ejemplo, ¿cómo sé si es racional pedir la ensalada del chef si no sé cuánto me gustará? Podemos modelar eso con otro nodo de azar especificando una "calidad intrínseca" inobservable de la ensalada.

Así, vemos que la teoría de juegos es buena para representar la mayoría de las fuentes de incertidumbre, pero al costo de duplicar el tamaño del árbol cada vez que añadimos otro nodo; un hábito que rápidamente conduce a árboles grandes no manejables. Debido a estos y otros problemas, la teoría de juegos ha sido utilizada para *analizar* entornos que están en equilibrio, en lugar de para *controlar* agentes dentro de un entorno.

18.2.5 Pagos inciertos y juegos de asistencia

En el Capítulo 1 (página 4), señalamos la importancia de diseñar sistemas de IA que puedan operar bajo incertidumbre sobre el verdadero objetivo humano. El Capítulo 16 (página 553) introdujo un modelo simple para la incertidumbre sobre las *propias* preferencias, utilizando el ejemplo del helado con sabor a durián. Mediante el simple dispositivo de añadir una nueva variable latente al modelo para representar las preferencias desconocidas, junto con un modelo de sensor apropiado (por ejemplo, observando el sabor de una pequeña muestra del helado), las preferencias inciertas permiten ser manejadas de una manera natural.

El Capítulo 16 también estudió el **problema del interruptor de apagado**: mostramos que un robot con incertidumbre sobre las preferencias humanas se deferirá al humano y permitirá que se le apague. En ese problema, Robbie el robot no está seguro de las preferencias de Harriet, la humana, pero modelamos la decisión de Harriet (si apagar o no a Robbie) como una simple consecuencia determinista de sus propias preferencias para la acción que Robbie propone. Aquí, generalizamos esta idea en un juego completo de dos personas llamado **juego de asistencia**, en el que tanto Harriet como Robbie son jugadores. Asumimos que Harriet observa sus propias preferencias θ y actúa de acuerdo con ellas, mientras que Robbie tiene una probabilidad previa $P(\theta)$ sobre las preferencias de Harriet. El pago se define por θ y es idéntico para ambos jugadores: tanto Harriet como Robbie están maximizando el pago de Harriet. De esta manera, el juego de asistencia proporciona un modelo formal de la idea de IA benéfica demostrable introducida en el Capítulo 1.

Además del comportamiento deferente exhibido por Robbie en el problema del interruptor de apagado, que es un tipo restringido de juego de asistencia, otros comportamientos que emergen

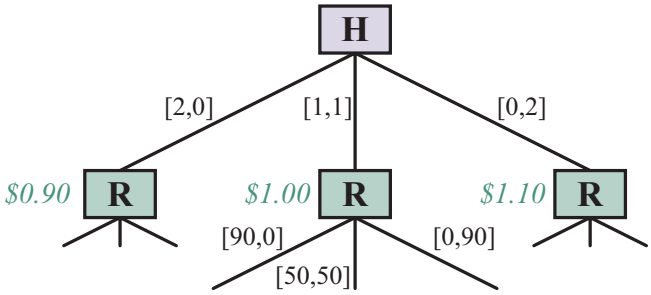


Figura 18.6 El juego de la engrapadora. Cada rama está etiquetada [*p*,*s*] denotando el número de engrapadoras y grapas fabricadas en esa rama. Harriet la humana puede elegir hacer dos engrapadoras, dos grapas, o una de cada una. (Los valores en cursiva verde son los valores para Harriet si el juego terminara allí, asumiendo $\theta = 0.45$). Robbie el robot luego tiene la opción de hacer 90 engrapadoras, 90 grapas, o 50 de cada una.

como equilibrio las estrategias en los juegos de asistencia general incluyen acciones por parte de Harriet que describiríamos como enseñar, recompensar, ordenar, corregir, demostrar o explicar, así como acciones por parte de Robbie que describiríamos como pedir permiso, aprender de las demostraciones, suscitar preferencias, etc. El punto clave es que estos comportamientos no necesitan ser guionizados: al resolver el juego, Harriet y Robbie descubren por sí mismos cómo transmitir la información de preferencia de Harriet a Robbie, para que Robbie pueda ser más útil para Harriet. No necesitamos estipular de antemano que Harriet debe "dar recompensas" o que Robbie debe "seguir instrucciones", aunque estas pueden ser interpretaciones razonables de cómo terminan comportándose.

Juego de la engrapadora

Para ilustrar los juegos de asistencia, usaremos el **juego de la engrapadora**; un juego muy simple en el que Harriet la humana tiene un incentivo para "señalar" a Robbie, el robot, alguna información sobre sus preferencias. Robbie es capaz de interpretar esa señal porque permite resolver el juego y, por lo tanto, entiende qué tendría que ser cierto sobre las preferencias de Harriet en orden para que ella señale de esa manera.

Los pasos del juego se muestran en la Figura 18.6. En esta se involucra hacer engrapadoras y grapas. Las preferencias de Harriet se expresan mediante una función de pago que depende del número de engrapadoras y el número de grapas producidas, con una cierta "tasa de cambio" entre las dos. El parámetro de preferencia de Harriet θ denota el valor relativo (en dólares) de una engrapadora; por ejemplo, ella podría valorar las engrapadoras en $\theta = 0.45$ dólares, lo que significa que las grapas valen $1 - \theta = 0.55$ dólares. Así, si se producen p engrapadoras y s grapas, el pago de Harriet será $p\theta + s(1 - \theta)$ dólares en total. La prioridad de Robbie es $P(\theta) = \mathit{Uniforme}(\theta;0,1)$. En el juego en sí, Harriet va primero, y puede elegir hacer dos engrapadoras, dos grapas, o una de cada. Luego, Robbie puede elegir hacer 90 engrapadoras, 90 grapas o 50 de cada.

Note que si ella estuviera haciendo esto por sí misma, Harriet sólo haría dos grapas, con un valor de $1.10. (Vea las anotaciones en el primer nivel del árbol en la Figura 18.6). Pero Robbie está observando y aprende de su elección. ¿Qué es aprende exactamente? Bueno, eso depende de cómo haga su elección Harriet. ¿Cómo hace Harriet su elección? Eso depende de cómo Robbie va a interpretarlo. Podemos resolver esta circularidad encontrando un equilibrio de Nash. En este caso, es único y se estima aplicando la mejor respuesta miope: elija cualquier estrategia para Harriet; elija la mejor estrategia para Robbie, dada la estrategia de Harriet; elija la mejor

estrategia para Harriet, dada la estrategia de Robbie, y así sucesivamente. El proceso se desarrolla de la siguiente manera:

1. Comienza con la estrategia codiciosa para Harriet: hacer dos engrapadoras si prefiere las engrapadoras; hacer una de cada una si es indiferente; hacer dos grapas si prefiere las grapas.
2. Hay tres posibilidades que Robbie debe considerar, dada esta estrategia para Harriet:

   (a) Si Robbie ve a Harriet hacer dos engrapadoras, deduce que ella prefiere las engrapadoras, por lo que ahora cree que el valor de una engrapadora se distribuye de manera uniforme entre 0.5 y 1.0, con un promedio de 0.75. En ese caso, su mejor plan es hacer 90 engrapadoras con un valor esperado de $67.50 para Harriet.
   (b) Si Robbie ve a Harriet hacer uno de cada, deduce que ella valora las engrapadoras y las grapas en 0.50, por lo que la mejor elección es hacer 50 de cada una.
   (c) Si Robbie ve a Harriet hacer dos grapas, entonces por el mismo argumento que en (a), debería hacer 90 grapas.

3. Dada esta estrategia para Robbie, la mejor estrategia de Harriet ahora es algo diferente de la estrategia codiciosa en el paso 1. Si Robbie va a responder a ella haciendo uno de cada uno hasta hacer 50, entonces ella está mejor haciendo uno de cada uno no sólo si es indiferente, sino si está en cualquier lugar cerca de la indiferencia. De hecho, la política óptima ahora es hacer uno de cada uno si valora las engrapadoras en cualquier lugar entre 0.446 y 0.554.
4. Dada esta nueva estrategia para Harriet, la estrategia de Robbie permanece sin cambios. Por ejemplo, si ella elige una de cada una, él deduce que el valor de una engrapadora se distribuye de manera uniforme entre 0.446 y 0.554, con un promedio de 0.50, por lo que la mejor elección es hacer 50 de cada uno. Como la estrategia de Robbie es la misma que en el paso 2, la mejor respuesta de Harriet será la misma que en el paso 3, y hemos encontrado el equilibrio.

Con su estrategia, en efecto, Harriet está enseñando a Robbie sobre sus preferencias usando un código simple —un lenguaje, si quiere— que surge del análisis de equilibrio. También cabe destacar que Robbie nunca aprende las preferencias de Harriet, pero aprende lo suficiente para actuar de manera óptima en su nombre; es decir, actúa tal y como lo haría si conociera sus preferencias. Es benéfico para Harriet bajo las suposiciones establecidas, y bajo la suposición de que Harriet está jugando el juego de manera correcta.

La mejor respuesta miope funciona para este ejemplo y otros similares, pero no para casos más complejos. Es posible demostrar que siempre que no haya empates que causen problemas de coordinación, encontrar un perfil de estrategia óptimo para un juego de asistencia es reducible a resolver un POMDP cuyo espacio de estados es el espacio de estados subyacente del juego más los parámetros de preferencia humana θ. Los POMDPs en general son muy difíciles de resolver (Sección 17.5), pero los POMDPs que representan juegos de asistencia tienen una estructura adicional que permite algoritmos más eficientes.

Los juegos de asistencia permiten generalizarse para permitir múltiples participantes humanos, múltiples robots, humanos racionales imperfectos, humanos que no conocen sus propias preferencias, etc. Al proporcionar un espacio de acción facturado o estructurado, en lugar de las simples acciones atómicas en el juego de la engrapadora, las oportunidades de comunicación pueden ser mejoradas. Pocas de estas variaciones han sido exploradas hasta ahora, pero esperamos que la propiedad clave de los juegos de asistencia siga siendo cierta: cuanto más inteligente es el robot, mejor es el resultado para el humano.

18.3 Teoría de juegos cooperativos

Recuerde que los juegos cooperativos capturan escenarios de toma de decisiones en los que los agentes permiten formar acuerdos vinculantes entre ellos para cooperar. Luego pueden beneficiarse de recibir un valor extra en comparación con lo que obtendrían actuando solos.

Comenzamos introduciendo un modelo para una clase de **juegos cooperativos**. De manera formal, estos juegos se llaman "juegos cooperativos con utilidad transferible en forma de función característica". La idea del modelo es que cuando un grupo de agentes coopera, el grupo en su conjunto obtiene algún valor de utilidad, que luego puede ser repartido entre los miembros del grupo. El modelo no dice qué acciones tomarán los agentes, ni la estructura del juego en sí especifica cómo se dividirá el valor obtenido (eso vendrá después).

Función característica

Por definición, usamos la fórmula $G = (N, v)$ para decir que un juego cooperativo, G, está definido por un conjunto de jugadores $N = \{1,...,n\}$ y una **función característica**, v, que para cada subconjunto de jugadores $C \subseteq N$ da el valor que el grupo de jugadores podría obtener, si deciden trabajar juntos.

Con frecuencia, asumimos que el conjunto vacío de jugadores no logra nada ($v(\{\}) = 0$), y que la función es no negativa ($v(C) \geq 0$ para todo C). En algunos juegos hacemos la suposición adicional de que los jugadores no logran nada trabajando solos: $v(\{i\}) = 0$ para todo $i \in N$.

18.3.1 Estructuras de coalición y resultados

Coalición

Es convencional referirse a un subconjunto de jugadores C como una **coalición**. En el uso cotidiano, el término "coalición" implica una colección de personas con alguna causa común (como la Coalición para Detener la Violencia de las Armas), pero nos referiremos a cualquier subconjunto de jugadores como una coalición. El conjunto de todos los jugadores N se conoce como la **gran coalición**.

Gran coalición

En nuestro modelo, cada jugador debe elegir unirse a una coalición (que podría ser una coalición de sólo el jugador individual). Por lo tanto, las coaliciones forman una **partición** del conjunto de jugadores. Llamamos a la partición una estructura de coalición. Formalmente, una **estructura de coalición** sobre un conjunto de jugadores N es un conjunto de coaliciones $\{C_1,...,C_k\}$ tal que:

Estructura de coalición

$$C_i \neq \{\}$$
$$C_i \subseteq N$$
$$C_i \cap C_j = \{\} \text{ para todo } i \neq j \in N$$
$$C_1 \cup \cdots \cup C_k = N.$$

Por ejemplo, si tenemos $N = \{1,2,3\}$, entonces hay siete posibles coaliciones:

$$\{1\},\ \{2\},\ \{3\},\ \{1,2\},\ \{2,3\},\ \{3,1\},\ \text{y}\ \{1,2,3\}$$

y cinco posibles estructuras de coalición:

$$\{\{1\},\{2\},\{3\}\},\ \{\{1\},\{2,3\}\},\ \{\{2\},\{1,3\}\},\ \{\{3\},\{1,2\}\},\ \text{y}\ \{\{1,2,3\}\}.$$

Usamos la notación $\mathbf{CS}(N)$ para denotar el conjunto de todas las estructuras de coalición sobre el conjunto de jugadores N, y $CS(i)$ para denotar la coalición a la que pertenece el jugador i.

El **resultado** de un juego está definido por las elecciones que hacen los jugadores, al decidir qué coaliciones formar, y al elegir cómo dividir el valor $v(C)$ que recibe cada coalición. Por definición, dado un juego cooperativo definido por (N,v), el resultado es un par $(CS, \mathbf{x})$ que consiste en una estructura de coalición y un **vector de pago** $\mathbf{x} = (x_1,...,x_n)$ donde x_i es el valor que va al

Vector de pago

jugador i. El pago debe satisfacer la restricción de que cada coalición C divida todo su valor $v(C)$ entre sus miembros:

$$\sum_{i \in C} x_i = v(C) \qquad \text{para todo } C \in CS$$

Por ejemplo, dado el juego $(\{1,2,3\},v)$ donde $v(\{1\}) = 4$ y $v(\{2,3\}) = 10$, un posible resultado es:

$(\{\{1\},\{2,3\}\},(4,5,5))$.

Es decir, el jugador 1 se queda solo y acepta un valor de 4, mientras que los jugadores 2 y 3 se unen para recibir un valor de 10, que eligen dividir por igual.

Algunos juegos cooperativos tienen la característica de que cuando dos coaliciones se fusionan, ellas no lo hacen peor que si se hubieran quedado separadas. Esta propiedad se llama **superaditividad**. Formalmente, un juego es superaditivo si su función característica satisface la siguiente condición: Superaditividad

$$v(C \cup D) \geq v(C) + v(D) \qquad \text{para todo } C,D \subseteq N$$

Si un juego es superaditivo, entonces la gran coalición recibe un valor que es al menos tan alto o más alto que el total recibido por cualquier otra estructura de coalición. Sin embargo, como veremos en breve, los juegos superaditivos no siempre terminan con una gran coalición, por la misma razón que los jugadores no siempre llegan a un resultado deseable de manera colectiva Pareto-óptimo en el dilema del prisionero.

18.3.2 Estrategia en juegos cooperativos

La suposición básica en la teoría de juegos cooperativos es que los jugadores tomarán decisiones estratégicas sobre con quién cooperarán. De manera intuitiva, los jugadores no desearán trabajar con jugadores improductivos, buscarán a jugadores que produzcan de manera colectiva un alto valor de coalición. Pero estos jugadores buscados estarán haciendo su propio razonamiento estratégico. Antes de poder describir este razonamiento, necesitamos algunas definiciones adicionales.

Una **imputación** para un juego cooperativo (N,v) es un vector de pago que satisface las siguientes dos condiciones: Imputación

$$\sum_{i=1}^{n} x_i = v(N)$$
$$x_i \geq v(\{i\}) \text{ para toda } i \in N.$$

La primera condición dice que una imputación debe distribuir el valor total de la gran coalición; la segunda condición, conocida como **racionalidad individual**, dice que cada jugador está al menos tan bien como si hubiera trabajado solo. Racionalidad individual

Dada una imputación $\mathbf{x} = (x_1,...,x_n)$ y una coalición $C \subseteq N$, definimos $x(C)$ como la suma $\sum_{i \in C} x_i$, la cantidad total desembolsada a C por la imputación $\mathbf{x}$.

Después, definimos el **núcleo** de un juego (N,v) como el conjunto de todas las imputaciones $\mathbf{x}$ que satisfacen la condición $x(C) \geq v(C)$ para cada posible coalición $C \subset N$. Por lo tanto, si una imputación $\mathbf{x}$ *no está* en el núcleo, entonces existe alguna coalición $C \subset N$ tal que $v(C) > x(C)$. Los jugadores en C se negarían a unirse a la gran coalición porque estarían mejor quedándose con C. Núcleo

Por lo tanto, el núcleo de un juego consiste en todos los posibles vectores de pago a los que ninguna coalición podría objetar con el argumento de que podrían hacerlo mejor al no unirse a la gran coalición. Así, si el núcleo está vacío, entonces la gran coalición no puede formarse porque no importa cómo la gran coalición dividirá su pago, alguna coalición más pequeña se negaría a unirse. Las principales preguntas computacionales en torno al núcleo se relacionan con si está vacío o no, y si una distribución de pago en particular está en el núcleo.

La definición del núcleo lleva naturalmente a un sistema de desigualdades lineales, como sigue (las incógnitas son las variables $x_1,...,x_n$, y los valores $v(C)$ son constantes):

$x_i \geq v(\{i\})$ para todo $i \in N$
$\sum_{i \in N} x_i = v(N)$
$\sum_{i \in N} x_i \geq v(C)$ para todo $C \subseteq N$

Cualquier solución a estas desigualdades definirá una imputación en el núcleo. Permite formular las desigualdades como un programa lineal utilizando una función objetivo ficticia (por ejemplo, maximizando $\sum_{i \in N} x_i$), lo que permitirá calcular imputaciones en tiempo polinomial en el número de desigualdades. La dificultad radica en que esto genera un número exponencial de desigualdades (una para cada una de las 2^n coaliciones posibles). Por lo tanto, este enfoque produce un algoritmo para verificar la no vacuidad del núcleo que se ejecuta en tiempo exponencial. Si es posible mejorar este enfoque depende del juego que se estudie: para muchas clases de juegos cooperativos, el problema de verificar la no vacuidad del núcleo es co-NP-completo. A continuación, se proporciona un ejemplo.

Antes de proceder, Veamos un ejemplo de un juego superaditivo con un núcleo vacío. El juego tiene tres jugadores $N = \{1,2,3\}$, y tiene una función característica definida de la siguiente manera:

$$v(C) = \begin{cases} 1 \text{ si } |C| \geq 2 \\ 0 \text{ de lo contrario.} \end{cases}$$

Ahora, consideremos cualquier imputación (x_1,x_2,x_3) para este juego. Dado que $v(N) = 1$, debe darse el caso de que al menos un jugador i tiene $x_i > 0$, y los otros dos reciben un pago total inferior a 1. Esos dos podrían beneficiarse formando una coalición sin el jugador i y compartiendo el valor 1 entre ellos. Pero dado que esto se cumple para todas las imputaciones, el núcleo debe estar vacío.

El núcleo formaliza la idea de que la gran coalición es estable, en el sentido de que ninguna coalición puede separarse de ella de manera rentable. Sin embargo, el núcleo puede contener imputaciones que *no son razonables*, en el sentido de que uno o más jugadores podrían sentir que fueron injustas. Suponga que $N = \{1,2\}$, y tenemos una función característica ν definida como sigue:

$v(\{1\}) = v(\{2\}) = 5$
$v(\{1,2\}) = 20.$

Aquí, la cooperación genera un excedente de 10 sobre lo que los jugadores podrían obtener trabajando en aislamiento y, por lo tanto, la cooperación tendrá sentido en este escenario. Ahora, es fácil ver que la imputación (6,14) está en el núcleo de este juego: ningún jugador puede desviarse para obtener una mayor utilidad. Pero desde el punto de vista del jugador 1, esto podría parecer injusto, porque otorga 9/10 del excedente al jugador 2. Así, la noción del núcleo nos dice cuándo puede formarse una gran coalición, pero no nos dice cómo distribuir el pago.

Valor de Shapley

El **valor de Shapley** es una propuesta elegante para dividir el valor $v(N)$ entre los jugadores, dado que se formó la gran coalición N. Formulado por Lloyd Shapley, premio Nobel, a principios de los años 1950, el valor de Shapley está destinado a ser un esquema de distribución *justo*.

¿Qué significa *justo*? Sería injusto distribuir $v(N)$ basándose en el color de ojos de los jugadores, su género o color de piel. Con frecuencia, los estudiantes sugieren que el valor $v(N)$ debería dividirse en partes iguales, lo que parece que podría ser justo, hasta que consideramos que esto daría la misma recompensa a los jugadores que contribuyen mucho y a los que no contribuyen nada. La idea de Shapley fue sugerir que la única forma justa de dividir el valor $v(N)$ era hacerlo de acuerdo con cuánto *contribuyó* cada jugador a crear el valor $v(N)$.

Contribución marginal

Primero, se debe definir la noción de **contribución marginal** de un jugador. La contribución marginal que un jugador i hace a una coalición C es el valor que i añadiría (o quitaría) si se uniera

a la coalición C. Formalmente, la contribución marginal que el jugador i hace a C se denota por $mc_i(C)$:

$$mc_i(C) = v(C \cup \{i\}) - v(C).$$

Ahora, un primer intento de definir un esquema de división de pagos online con la sugerencia de Shapley de que los jugadores deben ser recompensados de acuerdo con su contribución sería pagar a cada jugador i el valor que añadiría a la coalición que contiene a todos los demás jugadores:

$$mc_i(N - \{i\}).$$

El problema es que esto supone de manera implícita que el jugador i es el *último* en entrar a la coalición. Entonces, Shapley sugirió que se deben considerar todas las formas posibles en que la gran coalición podría formarse, es decir, todos los órdenes posibles de los jugadores N, y considerar el valor que i añade a los jugadores precedentes en el orden. Luego, un jugador debería ser recompensado pagando *el promedio de la contribución marginal que el jugador i hace, sobre todos los órdenes posibles de los jugadores, al conjunto de jugadores precedentes a i en el orden*.

Denotamos por P todas las posibles permutaciones (es decir, órdenes) de los jugadores N, y denotamos a los miembros de P por p, p', etc. Donde $p \in P$ e $i \in N$, denotamos por p_i al conjunto de jugadores que preceden a i en el orden p. Entonces, el valor de Shapley para un juego G es la imputación $\varphi(G) = (\varphi_1(G),...,\varphi_n(G))$ definida como sigue:

$$\varphi_i(G) = \frac{1}{n!} \sum_{p \in P} mc_i(p_i). \tag{18.1}$$

Esto debería convencer de que el valor de Shapley es una propuesta razonable. Pero el hecho notable es que es la única solución a un conjunto de axiomas que caracteriza un esquema de distribución de pagos "justo". Se necesitarán algunas definiciones más antes de definir los axiomas.

Se define un **jugador maniquí** como un jugador i que nunca añade valor a una coalición, es decir, $mc_i(C) = 0$ para todo $C \subseteq N - \{i\}$. Se dirá que dos jugadores i y j son **jugadores simétricos** si siempre hacen contribuciones idénticas a las coaliciones; es decir, $mc_i(C) = mc_j(C)$ para todo $C \subseteq N - \{i,j\}$. Finalmente, donde $G = (N,v)$ y $G' = (N,v')$ son juegos con el mismo conjunto de jugadores, el juego $G + G'$ es el juego con el mismo conjunto de jugadores y una función característica v'' definida por $v''(C) = v(C) + v'(C)$.

Jugador maniquí

Jugadores simétricos

Dadas estas definiciones, se pueden definir los axiomas de equidad que satisface el valor de Shapley:

- *Eficiencia*: $\sum_{i \in N} \varphi_i(G) = v(N)$. (Todo el valor debe distribuirse).
- *Jugador maniquí*: Si i es un jugador maniquí en G, entonces $\varphi_i(G) = 0$. (Los jugadores que nunca contribuyen con algo deben recibir nada).
- *Simetría*: Si i y j son simétricos en G entonces $\varphi_i(G) = \varphi_j(G)$. (Los jugadores que hacen contribuciones idénticas deben recibir pagos idénticos).
- *Aditividad*: El valor es aditivo sobre los juegos: para todos los juegos $G = (N,v)$ y $G' = (N,v')$, y para todos los jugadores $i \in N$, se tiene que $\varphi_i(G + G') = \varphi_i(G) + \varphi_i(G')$.

El axioma de la aditividad es, sin duda, muy técnico. Sin embargo, si lo aceptamos como un requisito, podemos establecer la siguiente propiedad clave: *el valor de Shapley es la única forma de distribuir el valor coalicional de manera que se cumplan estos axiomas de justicia.*

18.3.3 Cálculo en juegos cooperativos

Desde un punto de vista teórico, ahora tenemos una solución satisfactoria; pero desde un punto de vista computacional, necesitamos saber cómo *representar de manera compacta* los juegos cooperativos y cómo *calcular de manera eficiente* los conceptos de solución como el núcleo y el valor de Shapley.

La representación obvia para una función característica sería una tabla que enumere el valor $v(C)$ para todas las 2^n coaliciones. Esto es inviable para un n grande. Se han desarrollado una serie de enfoques para representar de manera compacta los juegos cooperativos, que se pueden distinguir por si son *completos* o no. Un esquema de representación completo es aquel que es capaz de representar *cualquier* juego cooperativo. El inconveniente con los esquemas de representación completos es que siempre habrá algunos juegos que no se pueden representar de manera compacta. Una alternativa es usar un esquema de representación que esté garantizado para ser compacto, pero que no sea completo.

Redes de contribución marginal

Redes de contribución marginal

Ahora describimos un esquema de representación, llamado **redes de contribución marginal** (*MC-nets*). Usaremos una versión simplificada para facilitar la presentación, pero la simplificación la hace incompleta: la versión completa de las MC-nets es una representación completa.

La idea detrás de las redes de contribución marginal es representar la función característica de un juego (N,v) como un conjunto de reglas de valor de coalición, de la forma: (C_i,x_i), donde $C_i \subseteq N$ es una coalición y x_i es un número. Para calcular el valor de una coalición C, sólo sumamos los valores de todas las reglas (C_i,x_i) tales que $C_i \subseteq C$. Así, dado un conjunto de reglas $R = \{(C_i,x_i),...,(C_k,x_k)\}$, la función característica correspondiente es:

$$v(C) = \sum\{x_i | (C_i,x_i) \in R \text{ y } C_i \subseteq C\}.$$

Suponga que tenemos un conjunto de reglas R que contiene las siguientes tres reglas:

$$\{(\{1,2\},5), \quad (\{2\},2), \quad (\{3\},4)\}.$$

Entonces, por ejemplo, tenemos:

- $v(\{1\}) = 0$ (porque no se aplican reglas),
- $v(\{3\}) = 4$ (tercera regla),
- $v(\{1,3\}) = 4$ (tercera regla),
- $v(\{2,3\}) = 6$ (segunda y tercera reglas), y
- $v(\{1,2,3\}) = 11$ (primera, segunda y tercera reglas).

Con esta representación podemos calcular el valor de Shapley en tiempo polinomial. La idea clave es que cada regla se entiende como que define un juego por sí misma, en el que los jugadores son simétricos. Por lo tanto, apelando a los axiomas de aditividad y simetría de Shapley, el valor de Shapley $\varphi_i(R)$ del jugador i en el juego asociado con el conjunto de reglas R es:

$$\varphi_i(R) = \sum_{(C,x)\in R} \begin{cases} \frac{x}{|C|} & \text{si } i \in C \\ 0 & \text{en otro caso.} \end{cases}$$

La versión de las redes de contribución marginal que hemos presentado aquí no es un esquema de representación *completo*: hay juegos cuya función característica no es posible representar usando conjuntos de reglas de la forma antes descrita. Un tipo más rico de redes de contribución marginal permite reglas de la forma (φ,x), donde φ es una fórmula lógica proposicional sobre los jugadores N: una coalición C satisface la condición φ si corresponde a una asignación satisfactoria para φ.

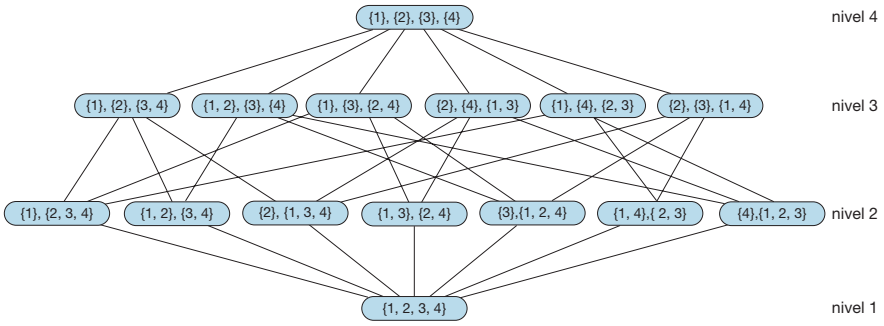


Figura 18.7 El grafo de estructura de coalición para $N = \{1,2,3,4\}$. El nivel 1 tiene estructuras de coalición que contienen una sola coalición; el nivel 2 tiene estructuras de coalición que contienen dos coaliciones, y así sucesivamente.

Este esquema es una representación completa: en el peor de los casos, necesitamos una regla para cada coalición posible. Además, el valor de Shapley se calcula en tiempo polinomial con este esquema; los detalles son más complicados que para las reglas simples antes descritas, aunque el principio básico es el mismo; consulte las notas al final del capítulo para las referencias.

Estructuras de coalición para el máximo bienestar social

Obtenemos una perspectiva diferente sobre los juegos cooperativos si asumimos que los agentes comparten un propósito común. Por ejemplo, si pensamos en los agentes como trabajadores en una empresa, las consideraciones estratégicas relacionadas con la formación de coaliciones que aborda el núcleo no son relevantes. En cambio, podríamos querer organizar la fuerza laboral (los agentes) en equipos para maximizar su productividad general. Por lo regular, la tarea es encontrar una coalición que maximice el *bienestar social* del sistema, definido como la suma de los valores de las coaliciones individuales. Escribimos el bienestar social de una estructura de coalición CS como $sw(CS)$, con la siguiente definición:

$$sw(CS) = \sum_{C \in CS} v(C).$$

Entonces una estructura de coalición socialmente óptima CS^* con respecto a G maximiza esta cantidad. Encontrar una estructura de coalición socialmente óptima es un problema computacional natural, que ha sido estudiado más allá de la comunidad de sistemas multiagente: a veces llamado **problema de partición de conjuntos**. Sin embargo, el problema es NP-difícil, porque el número de posibles estructuras de coalición crece de manera exponencial con el número de jugadores.

Problema de partición de conjuntos

Encontrar la estructura de coalición óptima mediante una búsqueda exhaustiva ingenua es, por lo tanto, inviable en general. Un enfoque influyente para la formación de estructuras de coalición óptimas se basa en la idea de buscar un subespacio del **grafo de estructura de coalición**. La idea se explica mejor con referencia a un ejemplo.

Estructura de coalición

Suponga que tenemos un juego con cuatro agentes, $N = \{1,2,3,4\}$. Existen quince posibles estructuras de coalición para este conjunto de agentes. Podemos organizar estos en un grafo de estructura de coalición, como se muestra en la Figura 18.7, donde los nodos en el nivel ℓ del grafo corresponden a todas las estructuras de coalición con ℓ coaliciones. Un borde ascendente en el grafo representa la división de una coalición en el nodo inferior en dos coaliciones separadas en el nodo superior. Por

ejemplo, hay un borde de {{1},{2,3,4}} a {{1},{2},{3,4}} porque esta última estructura de coalición se obtiene de la anterior dividiendo la coalición {2,3,4} en las coaliciones {2} y {3,4}.

La estructura de coalición óptima CS^* se encuentra en algún lugar dentro del grafo de estructura de coalición; para encontrar esto, parece que tendríamos que evaluar cada nodo en el grafo. Considere las dos filas inferiores del grafo, niveles 1 y 2. Cada coalición posible (excluyendo la coalición vacía) aparece en estos dos niveles. (Por supuesto, no todas las posibles estructuras de coalición aparecen en estos dos niveles). Ahora, suponga que restringimos nuestra búsqueda de una posible estructura de coalición a *sólo* estos dos niveles, no subimos más en el grafo. Dejemos que CS' sea la mejor estructura de coalición que encontramos en estos dos niveles, y dejemos que CS^* sea la mejor estructura de coalición en general. Dejemos que C^* sea una coalición con el valor más alto de todas las coaliciones posibles:

$$C^* \in \arg\max_{C \subseteq N} v(C).$$

El valor de la mejor estructura de coalición que encontramos en los primeros dos niveles del grafo de estructura de coalición debe ser al menos tanto como el valor de la mejor coalición posible: $sw(CS') \geq v(C^*)$. Esto se debe a que cada coalición posible aparece en al menos una estructura de coalición en los primeros dos niveles del grafo. Así que asumamos el peor de los casos, es decir, $sw(CS') = v(C^*)$.

Compare el valor de $sw(CS')$ con $sw(CS^*)$. Dado que $sw(CS')$ es el valor más alto posible de cualquier estructura de coalición, y hay n agentes ($n = 4$ en el caso de la Figura 18.7), entonces el valor más alto posible de $sw(CS^*)$ sería $nv(C^*) = n{\cdot}sw(CS')$. En otras palabras, en el peor caso posible, el valor de la mejor estructura de coalición que encontramos en los primeros dos niveles del gráfico sería $\frac{1}{n}$ el valor del mejor, donde n es el número de agentes. Por lo tanto, aunque buscar en los primeros dos niveles del grafo no garantiza que nos dará la estructura de coalición óptima, *sí* garantiza que nos dará una que no es peor que $\frac{1}{n}$ de la óptima. En la práctica, será mucho mejor que eso.

18.4 Tomando decisiones colectivas

Ahora pasaremos del diseño de agentes al **diseño de mecanismos**, el problema de diseñar el juego correcto para que juegue una colección de agentes. Formalmente, un **mecanismo** consta de

Centro

1. Un lenguaje para describir el conjunto de estrategias permitidas que los agentes pueden adoptar.
2. Un agente distinguido, llamado el **centro**, que recoge informes de elecciones de estrategia de los agentes en el juego. (Por ejemplo, el subastador es el centro en una subasta).
3. Una regla de resultado, conocida por todos los agentes, que el centro utiliza para determinar los pagos a cada agente, dadas sus elecciones de estrategia.

Esta sección discute algunos de los mecanismos más importantes.

18.4.1 Asignando tareas con la red de contrato

Protocolo de red de contrato

El **protocolo de red de contrato** es quizá la técnica de resolución de problemas multiagente más antigua y más importante estudiada en IA. Es un protocolo de alto nivel para compartir tareas. Como el nombre lo sugiere, la red de contrato se inspiró en la forma en que las empresas hacen uso de los contratos.

El protocolo de red de contrato tiene en general cuatro fases principales, vea la Figura 18.8. El proceso comienza con un agente identificando la necesidad de acción cooperativa con respecto a alguna tarea. La necesidad surge porque el agente no tiene la capacidad para llevar a cabo la

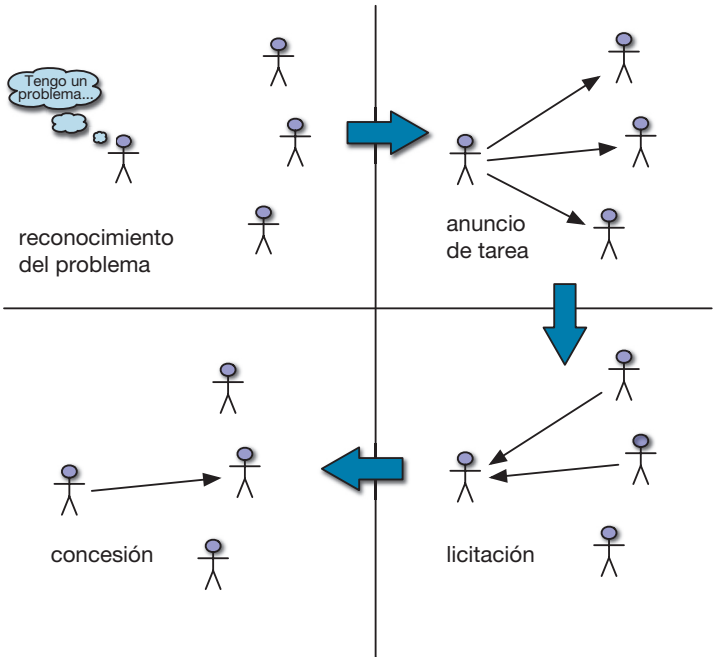


Figura 18.8 Protocolo de asignación de tareas de red de contrato.

tarea en aislamiento, o porque una solución cooperativa podría ser de alguna manera mejor (más rápida, más eficiente, más precisa).

El agente anuncia la tarea a otros agentes en la red con un mensaje de **anuncio de tarea** y luego actúa como el **administrador** de esa tarea durante su duración. El mensaje del anuncio de tarea debe incluir suficiente información para que los destinatarios puedan juzgar si están dispuestos y si son capaces de pujar por la tarea. La información precisa incluida en un anuncio de tarea dependerá del área de aplicación. Podría ser algún código que necesita ser ejecutado o podría ser una especificación lógica de un objetivo a alcanzar. El anuncio de tarea también podría incluir otra información que pueda ser requerida por los destinatarios, como plazos, requisitos de calidad de servicio, etc.

Anuncio de tarea
Administrador

Cuando un agente recibe un anuncio de tarea debe evaluarlo con respecto a sus propias capacidades y preferencias. En particular, cada agente debe determinar si tiene la capacidad para llevar a cabo la tarea y, segundo, si desea hacerlo o no. Sobre esta base, es posible entonces presentar una **oferta** para la tarea. Una oferta indicará con frecuencia las capacidades del ofertante que son relevantes para la tarea anunciada, y cualquier término y condición bajo los cuales la tarea será llevada a cabo.

Oferta

En general, un administrador puede recibir múltiples ofertas en respuesta a un solo anuncio de tarea. Con base en la información de las ofertas, el administrador selecciona al agente (o agentes) más apropiado para ejecutar la tarea. Los agentes exitosos son notificados a través de un mensaje de premio, y se convierten en contratistas para la tarea, al asumir la responsabilidad de la tarea hasta que se complete.

Las principales tareas computacionales requeridas para implementar el protocolo de red de contrato se resumen de la siguiente manera:

- *Procesamiento de anuncios de tareas.* Al recibir un anuncio de tarea, un agente decide si desea pujar por la tarea anunciada.
- *Procesamiento de ofertas.* Al recibir múltiples ofertas, el administrador debe decidir a qué agente otorgar la tarea, y luego otorgar la tarea.
- *Procesamiento de premios.* Los ofertantes exitosos (contratistas) deben intentar llevar a cabo la tarea, lo que significa generar nuevas subtareas, que se anuncian a través de más anuncios de tareas.

A pesar de (o quizá debido a) su simplicidad, la red de contratos es probablemente el marco más implementado y mejor estudiado para la resolución de problemas cooperativos. Es aplicable en múltiples entornos, por ejemplo, una variación de ella se realiza cada vez que solicita un coche con Uber.

18.4.2 Asignación de recursos escasos con subastas

Uno de los problemas más importantes en los sistemas multiagente es asignar recursos escasos; pero podríamos decir simplemente "asignar recursos", ya que en la práctica la mayoría de los recursos útiles son escasos en cierto sentido. La **subasta** es el mecanismo más importante para asignar recursos. El escenario más simple para una subasta es donde hay un solo recurso y hay múltiples posibles **postores**. Cada postor i tiene un valor de utilidad v_i para el artículo.

Subasta

Postores

En algunos casos, cada postor tiene un **valor privado** para el artículo. Por ejemplo, un suéter de mal gusto puede ser atractivo para un postor y no tener valor para otro.

En otros casos, como la subasta de derechos de perforación para un terreno petrolífero, el artículo tiene un valor común, el terreno producirá cierta cantidad de dinero, X, y todos los postores valoran un dólar por igual, pero hay incertidumbre sobre cuál es el valor real de X. Los diferentes postores tienen información diferente y, por lo tanto, diferentes estimaciones del valor real del artículo. En cualquier caso, los postores terminan con su propio v_i. Dado v_i, cada postor tiene la oportunidad, en el momento o momentos apropiados en la subasta, de hacer una oferta b_i. La oferta más alta, b_{max}, gana el artículo, pero el precio pagado no tiene por qué ser b_{max}; eso es parte del diseño del mecanismo.

Subasta de oferta ascendente

Subasta inglesa

El mecanismo de subasta más conocido es la **subasta de oferta ascendente**,[3] o **subasta inglesa**, en la que el centro comienza pidiendo una oferta mínima (o de **reserva**) b_{min}. Si algún postor está dispuesto a pagar esa cantidad, el centro luego pide $b_{min} + d$, para algún incremento d, y continúa desde allí. La subasta termina cuando nadie está dispuesto a pujar más, entonces el último postor gana el artículo y paga el precio ofertado.

¿Cómo sabemos si este es un buen mecanismo? Un objetivo es maximizar los ingresos esperados para el vendedor. Otro objetivo es maximizar una noción de utilidad global. Estos objetivos se superponen en cierta medida, porque un aspecto de maximizar la utilidad global es asegurar que el ganador de la subasta sea el agente que más valora el artículo (y, por lo tanto, está dispuesto a pagar más). Nosotros decimos que una subasta es **eficiente** si los bienes van al agente que más los valora. La subasta de oferta ascendente suele ser tanto eficiente como maximizadora de ingresos, pero si el precio de reserva se fija demasiado alto, el postor que más lo valora puede no pujar, y si el precio de reserva se fija demasiado bajo, el vendedor obtiene menos ingresos.

Eficiente

Quizá lo más importante que un mecanismo de subasta puede hacer es alentar a un número suficiente de postores a entrar en el juego y desalentarlos de participar en la **colusión**. La colusión es un acuerdo injusto o ilegal de dos o más postores para manipular los precios. Puede ocurrir en acuerdos secretos de trastienda o de manera tacita, dentro de las reglas del mecanismo.

Colusión

[3] La palabra "subasta" proviene del latín *augeo*, aumentar.

Por ejemplo, en 1999, Alemania subastó diez bloques de espectro de telefonía móvil en una subasta simultánea (se tomaron ofertas en los diez bloques al mismo tiempo), utilizando la regla de que cualquier oferta debe ser un mínimo de 10 % de aumento sobre la oferta anterior en un bloque. Sólo había dos postores creíbles; el primero, Mannesman, ingresó la oferta de 20 millones de marcos alemanes en los bloques 1-5, y 18.18 millones en los bloques 6-10. ¿Por qué 18.18 millones? Uno de los gerentes de T-Mobile dijo que "interpretaron la primera puja de Mannesman como una oferta". Ambas partes podrían calcular que un aumento de 10 % en 18.18 millones es 19.99 millones, y así la oferta de Mannesman fue interpretada como diciendo "cada uno podemos obtener la mitad de los bloques por 20 millones; no lo estropeemos pujando los precios más altos". Y, de hecho, T-Mobile pujó 20 millones en los bloques 6-10, eso fue el final de la puja.

El gobierno alemán obtuvo menos de lo que esperaba, porque los dos competidores pudieron usar el mecanismo de puja para llegar a un acuerdo tácito sobre cómo no competir. Desde el punto de vista del gobierno, se podría haber obtenido un mejor resultado con cualquiera de estos cambios en el mecanismo: un precio de reserva más alto; una subasta de primer precio con sobre cerrado para que los competidores no pudieran comunicarse mediante sus ofertas o por medio de sus incentivos para atraer a un tercer postor. Quizá la regla de 10 % fue un error en el diseño del mecanismo, porque facilitó la señalización precisa de Mannesman a T-Mobile.

En general, tanto el vendedor como la función de utilidad global se benefician si hay más postores, aunque la utilidad global sufre si se cuenta el costo del tiempo perdido de los postores que no tienen ninguna posibilidad de ganar. Una forma de alentar a más postores es hacer el mecanismo más fácil para ellos. Después de todo, si requiere demasiada investigación o cálculo por parte de los postores, pueden decidir llevar su dinero a otro lugar.

Por lo tanto, es deseable que los postores tengan una **estrategia dominante**. Recuerde que "dominante" significa que la estrategia funciona contra todas las demás estrategias, lo que a su vez significa que un agente puede adoptarla sin tener en cuenta las demás estrategias. Un agente con una estrategia dominante sólo permite pujar, sin perder tiempo contemplando las posibles estrategias de otros agentes. Un mecanismo por el cual los agentes tienen una estrategia dominante se llama mecanismo a **prueba de estrategias**. Si, como suele ser el caso, esa estrategia implica que los postores revelen su verdadero valor, v_i, entonces se llama una subasta **reveladora de la verdad**, o **veraz**; también se usa el término **compatible con incentivos**. El **principio de revelación** afirma que cualquier mecanismo permite ser transformado en un mecanismo revelador de la verdad equivalente, por lo que parte del diseño del mecanismo es encontrar estos mecanismos equivalentes.

Prueba de estrategias

Reveladora de la verdad

Principio de revelación

Resulta que la subasta de oferta ascendente tiene la mayoría de las propiedades deseables. El postor con el valor más alto v_i obtiene los bienes a un precio de $b_o + d$, donde b_o es la oferta más alta entre todos los demás agentes y d es el incremento del subastador.[4] Los postores tienen una estrategia dominante simple: seguir pujando mientras el costo actual esté por debajo de su v_i. El mecanismo no es del todo revelador de la verdad, porque el postor ganador revela sólo que su $v_i \geq b_o + d$; tenemos un límite inferior en v_i pero no una cantidad exacta.

Una desventaja (desde el punto de vista del vendedor) de la subasta de oferta ascendente es que puede desalentar la competencia. Suponga que en una puja por el espectro de telefonía móvil hay una empresa con ventaja que todos están de acuerdo en que podría aprovechar los clientes e infraestructuras existentes y, por lo tanto, podría obtener un beneficio mayor que cualquier otra. Los competidores potenciales pueden ver que no tienen ninguna oportunidad en una subasta de oferta ascendente, porque la empresa con ventaja siempre puede ofrecer más. Por lo tanto, los

[4] En realidad hay una pequeña posibilidad de que el agente con el v_i más alto no consiga los bienes, en el caso en que $b_o < v_i < b_o + d$. La posibilidad de esto es pequeña si se disminuye el incremento d.

competidores pueden no entrar en absoluto, y la empresa con ventaja termina ganando al precio de reserva.

Otra propiedad negativa de la subasta inglesa son sus altos costos de comunicación. Ya sea que la subasta se lleve a cabo en una sala o todos los postores tengan que tener líneas de comunicación seguras y de alta velocidad; en cualquier caso, tienen que tener tiempo para pasar por varias rondas de ofertas.

Subasta de oferta sellada

Un mecanismo alternativo, que requiere mucha menos comunicación, es la **subasta de oferta sellada**. Cada postor hace una sola oferta y la comunica al subastador, sin que los otros postores la vean. Con este mecanismo ya no hay una estrategia dominante simple. Si su valor es v_i y cree que el máximo de todas las otras ofertas de los agentes será b_o, entonces deberías ofrecer $b_o + \epsilon$, para algún ϵ pequeño, si eso es menor que v_i. Por lo tanto, su oferta depende de su estimación de las ofertas de los otros agentes, requiriendo que haga más trabajo. Además, tenga en cuenta que el agente con el v_i más alto podría no ganar la subasta. Esto se compensa con el hecho de que la subasta es más competitiva, pues reduce el sesgo hacia un postor con ventaja.

Subasta de segundo precio de oferta sellada

Subasta de Vickrey

Un pequeño cambio en el mecanismo de las subastas de oferta sellada conduce a la **subasta de segundo precio de oferta sellada**, también conocida como **subasta de Vickrey**.[5] En tales subastas, el ganador paga el precio de la subasta de la *segunda* oferta más alta, b_0, en lugar de pagar su propia oferta. Esta simple modificación elimina las deliberaciones complejas requeridas para las subastas de oferta sellada estándar (o de primer precio), porque la estrategia dominante es ahora ofrecer v_i; el mecanismo es revelador de la verdad. Note que la utilidad del agente i en términos de su oferta b_i, su valor v_i, y la mejor oferta entre los otros agentes, b_0, es

$$U_i = \begin{cases} (v_i - b_0) & \text{si } b_i > b_0 \\ 0 & \text{en otro caso.} \end{cases}$$

Para ver que $b_i = v_i$ es una estrategia dominante, note que cuando $(v_i - b_0)$ es positivo, cualquier oferta que gane la subasta es óptima, y ofrecer v_i en particular gana la subasta. Por otro lado, cuando $(v_i - b_0)$ es negativo, cualquier oferta que pierda la subasta es óptima, y ofrecer v_i en particular pierde la subasta. Así que ofrecer v_i es óptimo para todos los posibles valores de b_0 y, de hecho, v_i es la única oferta que tiene esta propiedad. Debido a su simplicidad y los mínimos requisitos de cálculo para el vendedor y los postores, la subasta de Vickrey es utilizada en sistemas de IA distribuidos.

Los motores de búsqueda en Internet realizan varios billones de subastas cada año para vender anuncios junto con sus resultados de búsqueda, y los sitios de subastas en línea manejan $100 mil millones al año en bienes, todos utilizando variantes de la subasta de Vickrey. Note que el valor esperado para el vendedor es b_0, que es el mismo retorno esperado que el límite de la subasta inglesa a medida que el incremento d se va a cero. Este es en realidad un resultado muy general: el **teorema de equivalencia de ingresos** establece que, con pequeñas advertencias, cualquier mecanismo de subasta en el que los postores tienen valores v_i conocidos sólo por ellos mismos (pero conocen la distribución de probabilidad de la que se extraen esos valores), producirá el mismo ingreso esperado. Este principio significa que los diversos mecanismos no están compitiendo con base en la generación de ingresos, sino más bien en otras cualidades.

Teorema de equivalencia de ingresos

Aunque la subasta de segundo precio es reveladora de la verdad, resulta que subastar n bienes con una subasta de $n + 1$ precio no es reveladora de la verdad. Muchos motores de búsqueda en Internet utilizan un mecanismo en el que subastan n espacios para anuncios en una página. El postor más alto gana el primer lugar, el segundo más alto obtiene el segundo lugar, y así sucesivamente. Cada ganador paga el precio ofrecido por el postor inmediatamente inferior, con el

[5] Nombrado en honor a William Vickrey (1914–1996), Premio Nobel de Economía en 1996 por este trabajo, quien tres días después murió de un ataque al corazón.

entendimiento de que el pago se realiza sólo si el buscador realmente hace clic en el anuncio. Los espacios superiores se consideran más valiosos porque es más probable que se noten y se haga clic en ellos.

Imagine que tres postores, b_1, b_2 y b_3, tienen valoraciones por un clic de $v_1 = 200$, $v_2 = 180$, y $v_3 = 100$, y que $n = 2$ espacios están disponibles; y se sabe que el primer lugar se hace clic en 5 % de las veces y el último lugar 2 %. Si todos los postores ofrecen de manera veraz, entonces b_1 gana el primer lugar y paga 180, y tiene un retorno esperado de $(200 - 180) \times 0.05 = 1$. El segundo lugar va para b_2. Pero b_1 puede ver que si ella ofreciera cualquier cosa en el rango 101–179, concedería el primer lugar a b_2, ganaría el segundo lugar, y obtendría un retorno esperado de $(200 - 100) \times .02 = 2$. Así, b_1 permite duplicar su retorno esperado ofreciendo menos que su verdadero valor en este caso.

En general, los postores en esta subasta de precio $n + 1$ deben gastar mucha energía analizando las ofertas de los demás para determinar su mejor estrategia; no hay una estrategia dominante simple.

Aggarwal *et al*. (2006) muestran que hay un único mecanismo de subasta veraz para este problema de múltiples espacios, en el que el ganador del espacio j paga el precio por el espacio j sólo por esos clics adicionales que están disponibles en el espacio j y no en el espacio $j + 1$. El ganador paga el precio por el espacio inferior por los clics restantes. En nuestro ejemplo, b_1 ofrecería 200 de manera veraz, y pagaría 180 por los $.05 - .02 = .03$ clics adicionales en el primer lugar, pero sólo pagaría el costo del último lugar, 100, por los .02 clics restantes. Así, el retorno total para b_1 sería $(200 - 180) \times .03 + (200 - 100) \times .02 = 2.6$.

Otro ejemplo de donde las subastas pueden entrar en juego dentro de la IA es cuando un conjunto de agentes están decidiendo si cooperar en un plan conjunto. Hunsberger y Grosz (2000) muestran que esto se logra de manera eficiente con una subasta en la que los agentes pujan por roles en el plan conjunto.

Bienes comunes

Ahora consideremos otro tipo de juego, en el que los países establecen su política para controlar la contaminación del aire. Cada país tiene una elección: pueden reducir la contaminación a un costo de −10 puntos por implementar los cambios necesarios, o pueden continuar contaminando, lo que les da una utilidad neta de −5 (en costos de salud añadidos, etc.) y también contribuye con −1 puntos a cada otro país (porque el aire se comparte entre países). Note que la estrategia dominante para cada país es "continuar contaminando", pero si hay 100 países y cada uno sigue esta política, entonces cada país obtiene una utilidad total de −104, mientras que si cada país redujera la contaminación, ellos tendrían una utilidad de −10. Esta situación se llama la **tragedia de los comunes**: si nadie tiene para pagar por el uso de un recurso común, entonces permite ser explotado de una manera que conduzca a una menor utilidad total para todos los agentes. Es similar al dilema del prisionero: hay otra solución al juego que es mejor para todas las partes, pero parece no haber manera de que los agentes racionales lleguen a esa solución bajo el juego actual.

Tragedia de los comunes

Un enfoque para lidiar con la tragedia de los comunes es cambiar el mecanismo a uno que cobre a cada agente por usar los comunes. Por lo regular, necesitamos asegurar que todas las **externalidades** –efectos sobre la utilidad global que no se reconocen en las transacciones de externalidades de los agentes individuales– se hacen explícitas.

Externalidades

Establecer los precios de manera correcta es la parte difícil. En el límite, este enfoque equivale a crear un mecanismo en el que se requiere de manera efectiva a cada agente maximizar la utilidad global, pero puede hacerlo tomando una decisión local. Para este ejemplo, un impuesto al carbón sería un ejemplo de un mecanismo que cobra por el uso de los comunes de una manera que, si se implementa bien, maximiza la utilidad global.

VCG

Resulta que hay un diseño de mecanismo, conocido como el mecanismo de **Vickrey–Clarke–Groves** o VCG, que tiene dos propiedades favorables. Primero, es maximizador de la utilidad; es decir, maximiza la utilidad global, que es la suma de las utilidades para todas las partes, $\sum_i v_i$. Segundo, el mecanismo es revelador de la verdad, la estrategia dominante para todos los agentes es revelar su verdadero valor. No hay necesidad de que se involucren en cálculos de licitación estratégica complicados.

Daremos un ejemplo usando el problema de asignar algunos bienes comunes. Suponga que una ciudad decide que quiere instalar algunos transceptores de internet inalámbrico gratuitos. Sin embargo, el número de transceptores disponibles es menor que el número de barrios que los quieren. La ciudad quiere maximizar la utilidad global, pero si le dice a cada consejo de barrio "¿Cuánto valoras un transceptor gratuito (y, por cierto, se los daremos a las partes que más los valoren)?" entonces cada barrio tendrá un incentivo para informar un valor muy alto. El mecanismo VCG desalienta esta estratagema y les da un incentivo para informar su verdadero valor. Funciona de la siguiente manera:

1. El centro pide a cada agente que informe su valor por un artículo, v_i.
2. El centro asigna los bienes a un conjunto de ganadores, W, para maximizar $\sum_{i \in W} v_i$.
3. El centro calcula para cada agente ganador cuánta pérdida ha causado su presencia individual en el juego a los perdedores (que cada uno obtuvo 0 utilidad, pero podría haber obtenido v_j si hubieran sido un ganador).
4. Cada agente ganador luego paga al centro un impuesto igual a esta pérdida.

Por ejemplo, suponga que hay 3 transceptores disponibles y 5 postores que pujan 100, 50, 40, 20, y 10. Así que el conjunto de 3 ganadores, W, son los que pujaron 100, 50, y 40 y la utilidad global de asignar estos bienes es 190. Para cada ganador, es el caso de que si no hubieran estado en el juego, la oferta de 20 habría sido un ganador. Así, cada ganador paga un impuesto de 20 al centro.

Todos los ganadores deberían estar contentos porque pagan un impuesto que es menor que su valor, y todos los perdedores están tan contentos como pueden estar, porque valoran los bienes menos que el impuesto requerido. Por eso el mecanismo es revelador de la verdad. En este ejemplo, el valor crucial es 20; sería irracional pujar por encima de 20 si su verdadero valor estaba realmente por debajo de 20, y viceversa. Dado que el valor crucial podría ser cualquier cosa (dependiendo de los otros postores), eso significa que siempre es irracional pujar algo distinto a tu verdadero valor.

El mecanismo VCG es muy general, y permite ser aplicado a todo tipo de juegos, no sólo a subastas, con una ligera generalización del mecanismo descrito arriba. Por ejemplo, en una **subasta combinatoria** hay múltiples artículos diferentes disponibles y cada postor puede hacer múltiples ofertas, cada una en un subconjunto de los artículos. Por ejemplo, en la licitación de parcelas de tierra, un postor podría querer la parcela X o la parcela Y, pero no ambas; otro podría querer cualesquiera tres parcelas adyacentes, etc. El mecanismo VCG se usa para encontrar el resultado óptimo, aunque con 2^N subconjuntos de N bienes para lidiar con, el cálculo del resultado óptimo es NP completo. Con algunas advertencias el mecanismo VCG es único: cada mecanismo óptimo diferente es en esencia equivalente.

18.4.3 Votación

Teoría de la elección social

La siguiente clase de mecanismos que vamos a ver son los procedimientos de votación, del tipo que se utilizan para la toma de decisiones políticas en las sociedades democráticas. El estudio de los procedimientos de votación deriva del dominio de la **teoría de la elección social**.

La configuración básica es la siguiente. Como de costumbre, tenemos un conjunto $N = \{1,...,n\}$ de agentes, que en esta sección serán los votantes. Estos votantes quieren tomar decisiones con respecto a un conjunto $\Omega = \{\omega_1,\omega_2,...\}$ de posibles resultados. En una elección política, cada elemento de Ω podría representar a un candidato diferente ganando la elección.

Cada votante tendrá preferencias sobre Ω. Estas se expresan no como utilidades cuantitativas sino como comparaciones cualitativas: escribimos $\omega \succ_i \omega'$ para significar que el resultado ω está clasificado por encima del resultado ω' por el agente i. En una elección con tres candidatos, el agente i podría tener $\omega_2 \succ_i \omega_3 \succ_i \omega_1$.

El problema fundamental de la teoría de la elección social es combinar estas preferencias, usando una **función de bienestar social**, para llegar a un **orden de preferencia social**: una clasificación de los candidatos, desde el más preferido hasta el menos preferido. En algunos casos, sólo estamos interesados en un **resultado social**; el resultado más preferido por el grupo en su conjunto. Escribiremos $\omega \succ^* \omega'$ para significar que ω está clasificado por encima de ω' en el orden de preferencia social.

Función de bienestar social

Resultado social

Un escenario más sencillo es donde no nos preocupa obtener un orden completo de candidatos, sino sólo queremos elegir un conjunto de ganadores. Una **función de elección social** toma como entrada un orden de preferencia para cada votante, y produce como salida un conjunto de ganadores.

Función de elección social

Las sociedades democráticas quieren un resultado social que refleje las preferencias de los votantes. No obstante, esto no siempre es sencillo. Considere la **Paradoja de Condorcet**, un famoso ejemplo planteado por el Marqués de Condorcet (1743–1794). Suponga que tenemos tres resultados, $\Omega = \{\omega_a,\omega_b,\omega_c\}$, y tres votantes, $N = \{1,2,3\}$, con preferencias como sigue.

Paradoja de Condorcet

$$\begin{array}{l} \omega_a \succ_1 \omega_b \succ_1 \omega_c \\ \omega_c \succ_2 \omega_a \succ_2 \omega_b \\ \omega_b \succ_3 \omega_c \succ_3 \omega_a \end{array} \tag{18.2}$$

Ahora, suponga que tenemos que elegir a uno de los tres candidatos con base en estas preferencias. La paradoja es que:

- 2/3 de los votantes prefieren ω_3 sobre ω_1.
- 2/3 de los votantes prefieren ω_1 sobre ω_2.
- 2/3 de los votantes prefieren ω_2 sobre ω_3.

Entonces, para cada posible ganador, podemos señalar a otro candidato que sería preferido por al menos 2/3 del electorado. Es obvio que en una democracia no podemos esperar hacer feliz a *cada* votante. Esto demuestra que hay escenarios en los que *no importa qué resultado elijamos, la mayoría de los votantes preferirá un resultado diferente*. Una pregunta natural es si existe algún procedimiento de "buena" elección social que realmente refleje las preferencias de los votantes. Para responder a esto, necesitamos ser precisos sobre lo que queremos decir cuando decimos que una regla es "buena". Enumeraremos algunas propiedades que nos gustaría que cumpliera una buena función de bienestar social:

- *Condición de Pareto*: Indica que si cada votante clasifica ω_i por encima de ω_j, entonces $\omega_i \succ^* \omega_j$.
- *Condición del ganador de Condorcet*: Se dice que un resultado es un ganador de Condorcet si una mayoría de candidatos lo prefieren sobre todos los demás resultados. Dicho de otra manera, un ganador de Condorcet es un candidato que vencería a todos los demás candidatos en una elección por pares. La condición del ganador de Condorcet dice que si ω_i es un ganador de Condorcet, entonces ω_i debería ser clasificado primero.

- *Independencia de alternativas irrelevantes* (*IIA*): Suponga que hay una serie de candidatos, incluidos ω_i y ω_j, y las preferencias de los votantes son tales que $\omega_i \succ^* \omega_j$. Ahora, suponga que un votante cambió sus preferencias de alguna manera, pero *no* sobre la clasificación relativa de ω_i y ω_j. La condición IIA dice que, $\omega_i \succ^* \omega_j$ no debería cambiar.
- *No dictaduras*: No debería ser el caso de que la función de bienestar social muestre las preferencias de un votante e ignore a todos los demás votantes.

Teorema de Arrow

Estas cuatro condiciones parecen razonables, pero un teorema fundamental de la teoría de la elección social llamado **teorema de Arrow** (debido a Kenneth Arrow) nos dice que es imposible satisfacer las cuatro condiciones (para casos donde hay al menos tres resultados). Eso significa que para cualquier mecanismo de elección social que podamos elegir, habrá algunas situaciones (quizás inusuales o patológicas) que conducirán a resultados controvertidos. Sin embargo, no significa que la toma de decisiones democráticas sea desesperada en la mayoría de los casos. Aún no hemos visto ningún procedimiento de votación real, así que veamos algunos ahora.

Voto de mayoría simple

Con sólo dos candidatos, el **voto de mayoría simple** (el método estándar en Estados Unidos y Reino Unido) es el mecanismo preferido. Preguntamos a cada votante cuál de los dos candidatos prefieren, el que obtenga más votos es el ganador.

Votación por pluralidad

Con más de dos resultados, la **votación por pluralidad** es un sistema común. Preguntamos a cada votante por su primera opción, y seleccionamos al o los candidatos (más de uno en caso de empate) que obtengan más votos, incluso si nadie obtiene una mayoría. Aunque es común, la votación por pluralidad ha sido criticada por entregar resultados impopulares. Un problema clave es que sólo tiene en cuenta al candidato mejor clasificado en las preferencias de cada votante.

Recuento de Borda

El **recuento de Borda** (a partir de Jean-Charles de Borda, contemporáneo y rival de Condorcet) es un procedimiento de votación que tiene en cuenta toda la información en el orden de preferencia de un votante. Suponga que tenemos k candidatos. Entonces, para cada votante i, tomamos su orden de preferencia $\succ_i$, y damos una puntuación de k al candidato mejor clasificado, una puntuación de $k - 1$ al segundo candidato mejor clasificado, y así sucesivamente, hasta el candidato menos favorecido en el orden de i. La puntuación total para cada candidato es su recuento de Borda, y para obtener el resultado social $\succ^*$, los resultados se ordenan por su recuento de Borda, de mayor a menor. Un problema práctico con este sistema es que pide a los votantes que expresen preferencias sobre todos los candidatos, y algunos votantes sólo se preocupan por un subconjunto de candidatos.

Votación de aprobación

En la **votación de aprobación**, los votantes presentan un subconjunto de los candidatos que aprueban. Los ganadores son aquellos que son aprobados por la mayoría de los votantes. Este sistema se utiliza cuando la tarea es elegir a varios ganadores.

Votación instantánea de desempate

En la **votación instantánea de desempate**, los votantes clasifican a todos los candidatos, y si un candidato tiene una mayoría de votos en primer lugar, se le declara ganador. Si no, se elimina al candidato con menos votos en primer lugar. Ese candidato se elimina de todas las clasificaciones de preferencia (por lo que aquellos votantes que tenían al candidato eliminado como su primera opción ahora tienen a otro candidato como su nueva primera opción), y el proceso se repite. De manera eventual, algún candidato tendrá una mayoría de votos en primer lugar (a menos que haya un empate).

Votación de regla de mayoría verdadera

En la **votación de regla de mayoría verdadera**, el ganador es el candidato que vence a todos los demás candidatos en comparaciones por pares. Se pide a los votantes una clasificación completa de preferencias de todos los candidatos. Decimos que ω vence a ω', si más votantes tienen $\omega \succ \omega'$ que tienen $\omega' \succ \omega$. Este sistema tiene la buena propiedad de que la mayoría siempre está de acuerdo con el ganador, pero tiene la mala propiedad de que no todas las elecciones serán decididas: en la paradoja de Condorcet, por ejemplo, ningún candidato gana una mayoría.

Manipulación estratégica

Además del Teorema de Arrow, otro resultado negativo importante en el área de la teoría de la elección social es el **Teorema de Gibbard–Satterthwaite**. Este resultado se relaciona con las circunstancias bajo las cuales un votante puede beneficiarse de *tergiversar sus preferencias*.

Teorema de Gibbard–Satterthwaite

Recuerde que una función de elección social toma como entrada un orden de preferencia para cada votante y da como salida un conjunto de candidatos ganadores. Cada votante tiene, por supuesto, sus propias preferencias verdaderas, pero no hay nada en la definición de una función de elección social que requiera que los votantes informen sus preferencias de manera *veraz*; es posible declarar las preferencias que quieran.

En algunos casos, tiene sentido para un votante tergiversar sus preferencias. Por ejemplo, en la votación por pluralidad, los votantes que piensan que su candidato preferido no tiene ninguna posibilidad de ganar pueden votar por su segunda opción en su lugar. Eso significa que la votación por pluralidad es un juego en el que los votantes tienen que pensar de manera estratégica (sobre los otros votantes) para maximizar su utilidad esperada.

Esto plantea una pregunta interesante: ¿podemos diseñar un mecanismo de votación que sea inmune a tal manipulación?, ¿un mecanismo que revele la verdad? El teorema de Gibbard–Satterthwaite con el protocolo de ofertas alternas nos dice que no podemos: *Cualquier función de elección social que satisface la condición de Pareto para un dominio con más de dos resultados es manipulable o una dictadura*. Es decir, para cualquier procedimiento de elección social "razonable", habrá algunas circunstancias bajo las cuales un votante puede beneficiarse en principio al tergiversar sus preferencias. Sin embargo, no nos dice *cómo* se podría hacer tal manipulación, y no nos dice que tal manipulación sea probable *en la práctica*.

18.4.4 Negociación

Regateo, o negociación, es otro mecanismo que se utiliza con frecuencia en la vida cotidiana. Ha sido estudiado en la teoría de juegos desde la década de 1950 y, más reciente, se ha convertido en una tarea para agentes automatizados. La negociación se utiliza cuando los agentes necesitan llegar a un acuerdo sobre un asunto de interés común. Los agentes hacen ofertas (también llamadas propuestas o acuerdos) entre sí bajo protocolos específicos, y aceptan o rechazan cada oferta.

Regateo con el protocolo de ofertas alternas

Un protocolo de negociación influyente es el **modelo de negociación de ofertas alternas**. Por simplicidad, supondremos de nuevo sólo dos agentes. La negociación se lleva a cabo en una secuencia de rondas. A_1 comienza, en la ronda 0, haciendo una oferta. Si A_2 acepta la oferta, entonces se implementa la oferta. Si A_2 rechaza la oferta, entonces la negociación pasa a la siguiente ronda. Esta vez A_2 hace una oferta y A_1 elige aceptarla o rechazarla, y así sucesivamente. Si la negociación nunca termina (porque los agentes rechazan cada oferta), entonces definimos el resultado como el **acuerdo de conflicto**. Un supuesto simplificador conveniente es que ambos agentes prefieren llegar a un resultado, cualquier resultado, en un tiempo finito en lugar de quedarse atascados en el acuerdo de conflicto que consume tiempo de manera infinita.

Modelo de negociación de ofertas alternas

Acuerdo de conflicto

Usaremos el escenario de dividir un pastel para ilustrar las ofertas alternas. La idea es que hay algún recurso (el "pastel") cuyo valor es 1, que se divide en dos partes, una parte para cada agente. Por lo tanto, una oferta en este escenario es un par $(x, 1-x)$, donde x es la cantidad del pastel que recibe A_1, y $1-x$ es la cantidad que recibe A_2. El espacio de posibles acuerdos (el **conjunto de negociación**) es entonces:

Conjunto de negociación

$$\{(x, 1-x) : 0 \leq x \leq 1\}.$$

Ahora, ¿cómo deben negociar los agentes en este escenario? Para entender la respuesta a esta pregunta, primero veremos algunos casos más simples.

Primero, supongamos que permitimos que se lleve a cabo *sólo una ronda*. Así, A_1 hace una propuesta; A_2 puede aceptarlo (en cuyo caso se implementa el acuerdo), o rechazarlo (en cuyo caso se implementa el acuerdo de conflicto). Este es un **juego del ultimátum**. En este caso, resulta que A_1, el **primer jugador**, tiene todo el poder. Suponga que A_1 propone obtener todo el pastel, es decir, propone el acuerdo (1,0). Si A_2 rechaza, entonces se implementa el acuerdo de conflicto; ya que por definición A_2 preferiría obtener 0 en lugar del acuerdo de conflicto, A_2 estaría mejor aceptando. Por supuesto, A_1 no permite hacerlo mejor que obtener todo el pastel. Por lo tanto, estas dos estrategias, A_1 propone obtener todo el pastel y A_2 acepta, forman un equilibrio de Nash.

Juego del ultimátum

Considere el caso en que permitimos *dos* rondas de negociación. Ahora el poder ha cambiado: A_2 sólo puede rechazar la primera oferta, convirtiendo el juego en un juego de una ronda en el que A_2 es el primer jugador y, por lo tanto, obtendrá todo el pastel. En general, si el número de rondas es un número fijo, entonces quien se mueve al último obtendrá todo el pastel.

Ahora pasemos al caso general, donde no hay límite en el número de rondas. Suponga que A_1 usa la siguiente estrategia:

> Siempre propone (1,0), y siempre rechaza cualquier contraoferta.

¿Cuál es la mejor respuesta de A_2 a esto? Si A_2 rechaza de manera continua la propuesta, entonces los agentes negociarán para siempre, lo cual por definición es el peor resultado para A_2 (así como para A_1). Por lo tanto, A_2 no permite hacerlo mejor que aceptar la primera propuesta que hace A_1. De nuevo, este es un equilibrio de Nash. Pero ¿qué pasa si A_1 usa la estrategia:

> Siempre propone (0.8,0.2), y siempre rechaza cualquier oferta.

Por un argumento similar podemos ver que para esta oferta o *para cualquier posible acuerdo* $(x,1-x)$ *en el conjunto de negociación, hay un par de estrategias de negociación de equilibrio de Nash, tal que el resultado será el acuerdo sobre el acuerdo en el primer periodo de tiempo.*

Agentes impacientes

Este análisis nos dice que si no se ponen restricciones en el número de rondas, entonces habrá un número infinito de equilibrios de Nash. Así que agreguemos una suposición:

> Para cualquier resultado x y tiempos t_1 y t_2, donde $t_1 < t_2$, ambos agentes preferirían el resultado x en el tiempo t_1 sobre el resultado x en el tiempo t_2.

En otras palabras, los agentes son **impacientes**. Un enfoque estándar para la impaciencia es usar un **factor de descuento** γ_i (ver página 565) para cada agente ($0 \leq \gamma_i < 1$). Suponga que en algún punto de la negociación al agente i se le ofrece una parte del pastel de tamaño x. El valor de la porción x en el tiempo t es $\gamma^t_i x$. Por lo tanto, en el primer paso de negociación (tiempo 0), el valor es $\gamma^0_i x = x$, y en cualquier momento posterior el valor de la misma oferta será menor. Un valor más grande para γ_i (más cercano a 1), por lo tanto, implica más paciencia; un valor más pequeño significa menos paciencia.

Para analizar el caso general, primero consideremos la negociación durante periodos de tiempo fijos, como antes. El caso de ronda 1 tiene el mismo análisis que se dio antes: sólo tenemos un juego del ultimátum. Con *dos* rondas la situación cambia, porque el valor del pastel se reduce de acuerdo con factores de descuento γ_i. Suponga que A_2 rechaza la propuesta inicial de A_1. Entonces A_2 obtendrá todo el pastel con un ultimátum en la segunda ronda. Pero el valor de ese pastel entero ha disminuido: sólo vale γ_2 para A_2. El agente A_1 puede tener en cuenta este hecho ofreciendo

$(1 - \gamma_2, \gamma_2)$, una oferta que A_2 puede aceptar porque A_2 no permite hacer nada mejor que γ_2 en este momento. (Si le preocupa lo que sucede con los empates, sólo haga que la oferta sea $(1 - (\gamma_2 + \epsilon), \gamma_2 + \epsilon)$ para algún valor pequeño de ϵ).

Por lo tanto, las dos estrategias de A_1 ofreciendo $(1 - \gamma_2, \gamma_2)$, y A_2 aceptando esa oferta están en equilibrio de Nash. Los jugadores pacientes (aquellos con un γ_2 más grande) podrán obtener partes más grandes del pastel bajo este protocolo: en este contexto, la paciencia realmente es una virtud.

Ahora considere el caso general, donde no hay límites en el número de rondas. Como en el caso de ronda 1, A_1 permite elaborar una propuesta que A_2 debería aceptar, porque le da a A_2 la cantidad máxima alcanzable, dado los factores de descuento. Resulta que A_1 obtendrá

$$\frac{1 - \gamma_2}{1 - \gamma_1\gamma_2}$$

y A_2 obtendrá el resto.

Negociación en dominios orientados a tareas

Dominios orientados a tareas

En esta sección, consideramos la negociación para **dominios orientados a tareas**. En tal dominio, un conjunto de tareas deben llevarse a cabo, y cada tarea se asigna de manera inicial a un conjunto de agentes. Los agentes se benefician negociando quién realizará qué tareas. Por ejemplo, suponga que algunas tareas se realizan en un torno y otras en una fresadora, y que cualquier agente que use una máquina debe incurrir en un costo de configuración significativo. Entonces, tendría sentido que un agente le ofrezca a otro "De todos modos tengo que configurar en la fresadora. ¿Qué tal si hago todas tus tareas de fresado, y usted hace todas mis tareas de torno?"

A diferencia del escenario de negociación, comenzamos con una asignación inicial, por lo que si los agentes no logran acordar ninguna oferta, realizan las tareas T_i^0 que se les asignaron al inicio.

Para mantener las cosas simples, de nuevo asumiremos sólo dos agentes. Dejemos que T sea el conjunto de todas las tareas y que (T_1^0, T_2^0) denote la asignación inicial de tareas a los dos agentes en el tiempo 0. Cada tarea en T debe ser asignada a un agente. Asumimos que tenemos una función de costo c, que para cada conjunto de tareas T' da un número real positivo $c(T')$ que indica el costo para cualquier agente de llevar a cabo las tareas T'. (Asumimos que el costo depende sólo de las tareas, no del agente que realiza la tarea). La función de costo es monótona, agregar más tareas nunca reduce el costo, y el costo de no hacer nada es cero: $c(\{\}) = 0$. Como ejemplo, suponga que el costo de configurar la fresadora es 10 y cada tarea de fresado cuesta 1, entonces el costo de un conjunto de dos tareas de fresado sería 12, y el costo de un conjunto de cinco sería 15.

Otra oferta de la forma (T_1, T_2) significa que el agente i se compromete a realizar el conjunto de tareas T_i, a costo $c(T_i)$. La utilidad para el agente i es la cantidad que tienen que ganar al aceptar la oferta, la diferencia entre el costo de hacer este nuevo conjunto de tareas versus el conjunto inicial de tareas asignadas:

$$U_i((T_1, T_2)) = c(T_i) - c(T_i^0).$$

Individualmente racional

Una oferta (T_1, T_2) es **individualmente racional** si $U_i((T_1, T_2)) \geq 0$ para ambos agentes. Si un trato no es racional de manera individual, entonces al menos un agente permite hacerlo mejor sólo realizando las tareas que se le asignaron al inicio.

El conjunto de negociación para dominios orientados a tareas (asumiendo agentes racionales) es el conjunto de ofertas que son tanto individualmente racionales como óptimas de Pareto. No tiene sentido hacer una oferta individualmente irracional que será rechazada, ni hacer una oferta cuando hay una mejor oferta que mejora la utilidad de un agente sin perjudicar a nadie más.

El protocolo de concesión monótona

Protocolo de concesión monótona

El protocolo de negociación que consideramos para dominios orientados a tareas se conoce como **protocolo de concesión monótona**. Las reglas de este protocolo son las siguientes.

- La negociación procede en una serie de rondas.
- En la primera ronda, ambos agentes proponen de manera simultánea un trato, $D_i = (T_1, T_2)$, del conjunto de negociación. (Esto es diferente de las ofertas alternas que vimos antes).
- Se llega a un acuerdo si los dos agentes proponen los tratos D_1 y D_2, respectivamente, de tal manera que (i) $U_1(D_2) \geq U_1(D_1)$ o (ii) $U_2(D_1) \geq U_2(D_2)$; es decir, si uno de los agentes encuentra que el trato propuesto por el otro es al menos tan bueno o mejor que la propuesta que hizo. Si se llega a un acuerdo, entonces la regla para determinar el acuerdo es la siguiente: Si la oferta de cada agente coincide o supera la del otro agente, entonces se selecciona una de las propuestas al azar. Si sólo una propuesta supera o iguala la propuesta del otro, entonces este es el acuerdo.
- Si no se llega a un acuerdo, entonces la negociación procede a otra ronda de propuestas simultáneas. En la ronda $t + 1$, cada agente debe repetir la propuesta de la ronda anterior o hacer una **concesión**, una propuesta que es más preferida por el otro agente (es decir, tiene una utilidad más alta).
- Si ninguno de los agentes hace una concesión, entonces la negociación termina, y ambos agentes implementan el trato de conflicto, realizando las tareas que se les asignaron al inicio.

Concesión

Dado que el conjunto de posibles tratos es finito, los agentes no permiten negociar de manera indefinida: o los agentes llegarán a un acuerdo o habrá una ronda en la que ninguno de los agentes ceda. Sin embargo, el protocolo no garantiza que se llegará a un acuerdo *de manera rápida*: dado que el número de posibles tratos es $O(2^{|T|})$, es concebible que la negociación continúe durante un número de rondas exponencial en el número de tareas a asignar.

La estrategia de Zeuthen

Estrategia de Zeuthen

Hasta ahora, no hemos dicho nada sobre cómo los participantes en la negociación podrían o deberían comportarse al usar el protocolo de concesión monótona para dominios orientados a tareas. Una posible estrategia es la **estrategia de Zeuthen**.

La idea de la estrategia de Zeuthen es medir la disposición de un agente a *arriesgarse al conflicto*. De manera intuitiva, un agente estará más dispuesto a arriesgarse al conflicto si la diferencia en la utilidad entre su propuesta actual y el trato de conflicto es baja. En este caso, el agente tiene poco que perder si la negociación falla y se implementa el trato de conflicto y, por lo tanto, está más dispuesto a arriesgarse al conflicto, y menos dispuesto a ceder. En contraste, si la diferencia entre la propuesta actual del agente y el trato de conflicto es alta, entonces el agente tiene más que perder con el conflicto y, entonces, está menos dispuesto a arriesgarse al conflicto y, así, más dispuesto a ceder.

La disposición del agente i a arriesgarse al conflicto en la ronda t, denotada como $riskt_i^t$, se mide de la siguiente manera:

$$riskt_i^t = \frac{\text{utilidad que } i \text{ pierde al ceder y aceptar la oferta de } j}{\text{utilidad que } i \text{ pierde al no ceder y causar conflicto}}.$$

Hasta que se llegue a un acuerdo, el valor de $riskt^t_i$ será un valor entre 0 y 1. Mayores valores de $riskt_i^t$ (más cerca de 1) indica que i tiene menos que perder del conflicto y, por lo tanto, está más dispuesto a arriesgar el conflicto.

La estrategia de Zeuthen dice que la primera propuesta de cada agente debe ser un trato en el conjunto de negociación que maximice su propia utilidad (puede haber más de uno). Después de eso, el agente que debe ceder en la ronda t de negociación debe ser el que tenga el valor más pequeño de riesgo, el que tiene más que perder del conflicto si ninguno cede.

La siguiente pregunta a responder es ¿cuánto se debe ceder? La respuesta proporcionada por la estrategia de Zeuthen es, "Justo lo suficiente para cambiar el equilibrio de riesgo al otro agente". Es decir, un agente debe hacer la concesión *más pequeña* que hará que el otro agente ceda en la próxima ronda.

Hay un refinamiento final a la estrategia de Zeuthen. Suponga que en algún momento ambos agentes tienen *igual* riesgo. Entonces, según la estrategia, ambos deberían ceder. Pero al saber esto, un agente podría potencialmente "defectuar" al no ceder y así beneficiarse. Para evitar la posibilidad de que ambos cedan en este punto, extendemos la estrategia haciendo que los agentes "lancen una moneda" para decidir quién debe ceder si alguna vez se alcanza una situación de riesgo igual.

Con esta estrategia, el acuerdo será Pareto óptimo e individualmente racional. Sin embargo, dado que el espacio de posibles acuerdos es exponencial en el número de tareas, seguir esta estrategia requiere $O(2^{|T|})$ cálculos de la función de costo en cada paso de la negociación. Por último, la estrategia de Zeuthen (con la regla de lanzar la moneda) está en equilibrio de Nash.

Resumen

- La planificación **multiagente** es necesaria cuando hay otros agentes en el entorno con los que cooperar o competir. Es posible construir planes conjuntos, pero deben ser complementados con alguna forma de coordinación si dos agentes van a acordar qué plan conjunto ejecutar.
- La **teoría de juegos** describe el comportamiento racional para los agentes en situaciones en las que interactúan varios agentes. La teoría de juegos es para la toma de decisiones multiagente lo que la teoría de decisiones es para la toma de decisiones de un solo agente.
- Los **conceptos de solución** en la teoría de juegos están destinados a caracterizar los resultados racionales de un juego, resultados que podrían ocurrir si cada agente actuara de manera racional.
- La **teoría de juegos no cooperativa** asume que los agentes deben tomar sus decisiones de forma independiente. El **equilibrio de Nash** es el concepto de solución más importante en la teoría de juegos no cooperativa. Un equilibrio de Nash es un perfil de estrategia en el que ningún agente tiene un incentivo para desviarse de su estrategia especificada. Tenemos técnicas para lidiar con juegos repetidos y juegos secuenciales.
- La **teoría de juegos cooperativa** considera escenarios en los que los agentes permiten hacer acuerdos vinculantes para formar coaliciones con el fin de cooperar. Los conceptos de solución en el juego cooperativo intentan formular qué coaliciones son estables (el **núcleo**) y cómo dividir de manera justa el valor que obtiene una coalición (el **valor de Shapley**).
- Existen técnicas especializadas para ciertas clases importantes de decisiones multiagente: la red de contratos para compartir tareas; las subastas que se utilizan para asignar de manera eficiente recursos escasos; la negociación para llegar a acuerdos sobre asuntos de interés común; y los procedimientos de votación para agregar preferencias.

Notas bibliográficas e históricas

Es una curiosidad del campo que los investigadores en IA no comenzaron a considerar seriamente los problemas que rodean a los agentes interactuantes hasta la década de 1980, y el campo de los sistemas multiagente no se estableció como una subdisciplina distintiva de la IA hasta una década después. Sin embargo, las ideas que insinúan los sistemas multiagente estaban presentes en la década de 1970. Por ejemplo, en su muy influyente teoría de la *Sociedad de la mente*, Marvin Minsky (1986, 2007) propuso que las mentes humanas están construidas a partir de un conjunto de agentes. Doug Lenat tuvo ideas similares en un marco que llamó BEINGS (Lenat, 1975). En la década de 1970, basándose en su trabajo de doctorado sobre el sistema PLANNER, Carl Hewitt propuso un modelo de cálculo como agentes interactuantes llamado el **modelo de actor**, que se ha establecido como uno de los modelos fundamentales en el cálculo concurrente (Hewitt, 1977; Agha, 1986).

La prehistoria del campo de los sistemas multiagente está documentada a fondo en una colección de documentos titulada *Readings in Distributed Artificial Intelligence* (Bond and Gasser, 1988). La colección está precedida por una declaración detallada de los principales desafíos de investigación en sistemas multiagente, que sigue siendo relevante hoy, más de treinta años después de que fue escrita. La investigación inicial sobre sistemas multiagente tendía a asumir que todos los agentes en un sistema estaban actuando con un interés común, con un solo diseñador. Esto ahora se reconoce como un caso especial del entorno multiagente más general, el caso especial se conoce como **resolución de problemas distribuidos cooperativos**. Un sistema clave de esta época fue el Testbed de Monitoreo de Vehículos Distribuidos (DVMT), desarrollado bajo la supervisión de Victor Lesser en la Universidad de Massachusetts (Lesser y Corkill, 1988). El DVMT modeló un escenario en el que una colección de agentes de sensores acústicos distribuidos de manera geográfica coopera para rastrear el movimiento de los vehículos.

Resolución de problemas distribuidos cooperativos

La era contemporánea de la investigación de sistemas multiagente comenzó a finales de la década de 1980, cuando se descubrió que los agentes con preferencias diferentes son la norma en la IA y la sociedad; a partir de este punto, la teoría de juegos comenzó a establecerse como la principal metodología para estudiar a dichos agentes.

La planificación multiagente ha saltado en popularidad en los últimos años, aunque tiene una larga historia. Konolige (1982) formaliza la planificación multiagente en lógica de primer orden, mientras que Pednault (1986) da una descripción al estilo STRIPS. La noción de intención conjunta, que es esencial si los agentes van a ejecutar un plan conjunto, proviene del trabajo sobre actos comunicativos (Cohen y Perrault, 1979; Cohen y Levesque, 1990; Cohen *et al*., 1990). Boutilier y Brafman (2001) muestran cómo adaptar la planificación de orden parcial a un entorno de múltiples actores. Brafman y Domshlak (2008) idean un algoritmo de planificación de múltiples actores cuya complejidad crece sólo de manera lineal con el número de actores, siempre que el grado de acoplamiento (medido en parte por el ancho del árbol del grafo de interacciones entre agentes) esté limitado.

La planificación multiagente es más difícil cuando hay agentes adversarios. Como dijo Jean-Paul Sartre (1960), "En un partido de fútbol, todo se complica por la presencia del otro equipo". El general Dwight D. Eisenhower dijo, "En la preparación para la batalla siempre he encontrado que los planes son inútiles, pero la planificación es indispensable", lo que significa que es importante tener un plan o política condicional, y no esperar que un plan incondicional tenga éxito.

El tema del aprendizaje por refuerzo (RL) distribuido y multiagente (RL) no se cubrió en este capítulo pero es de gran interés actual. En RL distribuido, el objetivo es idear métodos por los cuales varios agentes coordinados aprendan a optimizar una función de utilidad

común. Por ejemplo, ¿es posible idear métodos mediante los cuales subagentes separados para la navegación del robot y la evasión de obstáculos del robot puedan cooperar para lograr un sistema de control combinado que sea globalmente óptimo? Se han obtenido algunos resultados básicos en esta dirección (Guestrin *et al*., 2002; Russell y Zimdars, 2003). La idea básica es que cada subagente aprende su propia función Q (una especie de función de utilidad; véase la Sección 22.3.3) a partir de su propio flujo de recompensas. Por ejemplo, un componente de navegación del robot puede recibir recompensas por avanzar hacia la meta, mientras que el componente de evitación de obstáculos recibe recompensas negativas por cada colisión. Cada decisión global maximiza la suma de las funciones Q y todo el proceso converge hacia soluciones globalmente óptimas.

Las raíces de la teoría de juegos se rastrean hasta las propuestas hechas en el siglo XVII por Christiaan Huygens y Gottfried Leibniz para estudiar las interacciones humanas competitivas y cooperativas de manera científica y matemática. A lo largo del siglo XIX, varios economistas de renombre crearon ejemplos matemáticos simples para analizar ejemplos particulares de situaciones competitivas.

Los primeros resultados formales en teoría de juegos se deben a Zermelo (1913) (quien había sugerido, un año antes, una forma de búsqueda minimax para juegos, aunque incorrecta). Emile Borel (1921) introdujo la noción de una estrategia mixta. John von Neumann (1928) demostró que cada juego de dos personas, de suma cero, tiene un equilibrio maximin en estrategias mixtas y un valor bien definido. La colaboración de Von Neumann con el economista Oskar Morgenstern llevó a la publicación en 1944 de la *Teoría de juegos y comportamiento económico*, el libro definitorio para la teoría de juegos. La publicación del libro se retrasó por la escasez de papel durante la guerra hasta que un miembro de la familia Rockefeller subvencionó de manera personal su publicación.

En 1950, a la edad de 21 años, John Nash publicó sus ideas sobre equilibrios en juegos generales (no de suma cero). Su definición de una solución de equilibrio, aunque anticipada en el trabajo de Cournot (1838), se conoció como equilibrio de Nash. Después de un largo retraso debido a la esquizofrenia que sufrió a partir de 1959, Nash recibió el Premio Nobel Memorial en Economía (junto con Reinhart Selten y John Harsanyi) en 1994. El equilibrio de Bayes–Nash es descrito por Harsanyi (1967) y discutido por Kadane y Larkey (1982). Algunos problemas en el uso de la teoría de juegos para el control de agentes son cubiertos por Binmore (1982). Aumann y Brandenburger (1995) muestran cómo es posible alcanzar diferentes equilibrios dependiendo del conocimiento que cada jugador tiene.

El dilema del prisionero fue inventado como un ejercicio de aula por Albert W. Tucker en 1950 (a partir de un ejemplo de Merrill Flood y Melvin Dresher) y está cubierto con amplitud por Axelrod (1985) y Poundstone (1993). Los juegos repetidos fueron introducidos por Luce y Raiffa (1957), y Abreu y Rubinstein (1988) discuten el uso de máquinas de estado finito para juegos repetidos, técnicamente, máquinas de Moore. El texto de Mailath y Samuelson (2006) se concentra en juegos repetidos.

Los juegos de información parcial en forma extensiva fueron introducidos por Kuhn (1953). La forma de secuencia para juegos de información parcial fue inventada por Romanovskii (1962) y de manera independiente por Koller *et al.* (1996); el artículo de Koller y Pfeffer (1997) proporciona una introducción legible al campo y describe un sistema para representar y resolver juegos secuenciales. El uso de la abstracción para reducir un árbol de juego a un tamaño que se resuelve con la técnica de Koller fue introducido por Billings *et al.* (2003). Después, métodos mejorados para encontrar el equilibrio permitieron la solución de abstracciones con 10^{12} estados (Gilpin *et al*., 2008; Zinkevich *et al.,* 2008). Bowling *et al.* (2008) muestran cómo usar el muestreo de importancia para obtener una mejor estimación del valor de una estrategia. Waugh *et al.* (2009) encontraron que el enfoque de abstracción es vulnerable a cometer errores

sistemáticos en la aproximación de la solución de equilibrio: funciona para algunos juegos, pero no para otros. Brown y Sandholm (2019) demostraron que, al menos en el caso del póker Texas Sujétalos multijugador, estas vulnerabilidades son superadas por suficiente potencia de cálculo. Utilizaron un servidor de 64 núcleos funcionando durante 8 días para calcular una estrategia de base para su programa Pluribus. Con esa estrategia pudieron derrotar a oponentes campeones humanos.

La teoría de juegos y los MDP se combinan en la teoría de los juegos de Markov, también llamados juegos estocásticos (Littman, 1994; Hu y Wellman, 1998). Shapley (1953b) en realidad describió el algoritmo de iteración de valor independiente de Bellman, pero sus resultados no fueron apreciados, quizá porque se presentaron en el contexto de los juegos de Markov. La teoría de juegos evolutiva (Smith, 1982; Weibull, 1995) mira la deriva de la estrategia a lo largo del tiempo: si la estrategia de su oponente está cambiando, ¿cómo debería reaccionar?

Los libros de texto sobre teoría de juegos, desde un punto de vista económico, incluyen los de Myerson (1991), Fudenberg y Tirole (1991), Osborne (2004), y Osborne y Rubinstein (1994). Desde una perspectiva de IA tenemos a Nisan *et al.* (2007) y Leyton-Brown y Shoham (2008). Vea Sandholm (1999) para una conocer una encuesta útil de toma de decisiones multiagente.

RL multiagente se distingue de RL distribuido por la presencia de agentes que no permiten coordinar sus acciones (excepto por actos comunicativos explícitos) y que pueden no compartir la misma función de utilidad. Así, RL multiagente se ocupa de problemas teóricos de juegos secuenciales o **juegos de Markov**, como se define en el Capítulo 17. Lo que causa problemas es el hecho de que, mientras un agente está aprendiendo a derrotar la política de su oponente, el oponente está cambiando su política para derrotar al agente. Así, el entorno es **no estacionario** (vea la página 444).

Littman (1994) notó esta dificultad al introducir los primeros algoritmos de RL para juegos de suma cero de Markov. Hu y Wellman (2003) presentan un algoritmo de Q-learning para juegos de suma general que converge cuando el equilibrio de Nash es único; cuando hay múltiples equilibrios, la noción de convergencia no es tan fácil de definir (Shoham *et al.,* 2004).

Los juegos de asistencia fueron introducidos bajo el título de **aprendizaje por refuerzo inverso cooperativo** por Hadfield-Menell *et al.* (2017a). Malik *et al.* (2018) introdujeron un eficiente solucionador POMDP diseñado específicamente para juegos de asistencia. Están relacionados con los **juegos del principal–agente** en economía, en los que un principal (por ejemplo, un empleador) y un agente (digamos, un empleado) necesitan encontrar un arreglo mutuamente benéfico a pesar de tener preferencias diferentes. Las diferencias principales son que (1) el robot no tiene preferencias propias, y (2) el robot no está seguro acerca de las preferencias humanas que necesita optimizar.

Juego del principal–agente

Los juegos cooperativos fueron estudiados por primera vez por Von Neumann y Morgenstern (1944). La noción del núcleo fue introducida por Donald Gillies (1959), y el valor de Shapley por Lloyd Shapley (1953a). Una buena introducción a las matemáticas de los juegos cooperativos es Peleg y Sudholter (2002). Los juegos simples en general son discutidos en detalle por Taylor y Zwicker (1999). Para una introducción a los aspectos computacionales de la teoría de juegos cooperativos vea Chalkiadakis *et al.* (2011).

Se han desarrollado muchos esquemas de representación compacta para juegos cooperativos durante las últimas tres décadas, comenzando con el trabajo de Deng y Papadimitriou (1994). El más influyente de estos esquemas es el modelo de redes de contribución marginal, que fue introducido por Ieong y Shoham (2005). El enfoque para la formación de coaliciones que describimos fue desarrollado por Sandholm *et al.* (1999); Rahwan *et al.* (2015) revisan el estado del arte.

El protocolo de red de contrato fue introducido por Reid Smith para su trabajo de doctorado en la Universidad de Stanford a finales de la década de 1970 (Smith, 1980). El protocolo parece

ser tan natural que se reinventa de manera regular hasta el día de hoy. Las bases económicas del protocolo fueron estudiadas por Sandholm (1993).

Las subastas y el diseño de mecanismos han sido temas principales en ciencias de la computación e IA durante varias décadas: vea Nisan (2007) para una perspectiva de ciencias de la computación convencional, Krishna (2002) para una introducción a la teoría de subastas y Cramton *et al.* (2006) para una colección de artículos sobre aspectos computacionales de las subastas.

El Premio Nobel Memorial de Economía 2007 fue para Hurwicz, Maskin y Myerson "por haber sentado las bases de la teoría del diseño de mecanismos" (Hurwicz, 1973). La tragedia de los comunes, un problema motivador para el campo, fue analizado por William Lloyd (1833) pero nombrado y publicado por Garrett Hardin (1968). Ronald Coase presentó un teorema en el que si los recursos están sujetos a propiedad privada y si los costos de transacción son lo suficientemente bajos, entonces los recursos se administrarán de manera eficiente (Coase, 1960). Señala que, en la práctica, los costos de transacción son altos, por lo que este teorema no se aplica y deberíamos buscar otras soluciones más allá de la privatización y el mercado. El libro de Elinor Ostrom, *Gobernando los comunes* (1990), describió soluciones para el problema basadas en colocar el control de gestión sobre los recursos en manos de la gente local que tiene más conocimiento de la situación. Tanto Coase como Ostrom ganaron el Premio Nobel de economía por su trabajo.

El principio de revelación se debe a Myerson (1986), y el teorema de equivalencia de ingresos fue desarrollado de manera independiente por Myerson (1981) y Riley y Samuelson (1981). Dos economistas, Milgrom (1997) y Klemperer (2002), escriben sobre las subastas de espectro de miles de millones de dólares en las que participaron.

El diseño de mecanismos se utiliza en la planificación multiagente (Hunsberger y Grosz, 2000; Stone *et al*., 2009) y la programación (Rassenti *et al.,* 1982). Varian (1995) ofrece una breve descripción general con conexiones a la literatura de ciencias de la computación, mientras que Rosenschein y Zlotkin (1994) presentan un tratamiento de la longitud del libro con aplicaciones a la IA distribuida. El trabajo relacionado con la IA distribuida tiene varios nombres, incluido inteligencia colectiva (Tumer y Wolpert, 2000; Segaran, 2007) y control basado en el mercado (Clearwater, 1996). Desde 2001 ha habido una Competencia Anual de Agentes Comerciales (TAC), en la que los agentes intentan obtener el mayor beneficio en una serie de subastas (Wellman *et al*., 2001; Arunachalam y Sadeh, 2005).

La literatura de elección social es enorme y abarca desde consideraciones filosóficas sobre la naturaleza de la democracia hasta análisis técnicos altamente específicos de procedimientos de votación específicos. Campbell y Kelly (2002) proporcionan un buen punto de partida para esta literatura. El *Manual de Elección Social Computacional* ofrece una serie de artículos que revisan temas de investigación y métodos en este campo (Brandt *et al.,* 2016). El teorema de Arrow enumera las propiedades deseadas de un sistema de votación y demuestra que es imposible lograr todas ellas (Arrow, 1951). Dasgupta y Maskin (2008) muestran que la regla de la mayoría (no la regla de la pluralidad, y no la votación de elección clasificada) es el sistema de votación más robusto. La complejidad computacional de manipular las elecciones fue estudiada por primera vez por Bartholdi *et al.* (1989).

Apenas hemos rozado la superficie del trabajo sobre negociación en la planificación multiagente. Durfee y Lesser (1989) discuten cómo las tareas permiten ser compartidas entre agentes mediante la negociación. Kraus *et al.* (1991) describen un sistema para jugar Diplomacy, un juego de mesa que requiere negociación, formación de coaliciones y deshonestidad. Stone (2000) muestra cómo los agentes pueden cooperar como compañeros de equipo en el entorno competitivo, dinámico y parcialmente observable del fútbol robótico. En un artículo posterior, Stone (2003) analiza dos entornos multiagente competitivos: RoboCup, una competencia de fútbol robótico, y TAC, la Competencia de Agentes Comerciales basada en subastas, y encuentra que la

intractabilidad computacional de nuestros enfoques teóricamente bien fundamentados actuales ha llevado a que muchos sistemas multiagente sean diseñados por métodos *ad hoc*. Sarit Kraus ha desarrollado una serie de agentes que permiten negociar con humanos u otros agentes, vea Kraus (2001) para una revisión. El protocolo de concesión monótona para la negociación automatizada fue propuesto por Jeffrey S. Rosenschein y sus estudiantes (Rosenschein y Zlotkin, 1994). El protocolo de ofertas alternativas fue desarrollado por Rubinstein (1982).

Los libros sobre sistemas multiagente incluyen los de Weiss (2000a), Young (2004), Vlassis (2008), Shoham y Leyton-Brown (2009), y Wooldridge (2009). La conferencia principal para sistemas multiagente es la Conferencia Internacional sobre Agentes Autónomos y Sistemas Multiagente (AAMAS); también hay una revista con el mismo nombre. La Conferencia ACM sobre Comercio Electrónico (EC) también publica muchos trabajos relevantes, en particular en el área de algoritmos de subasta. La revista principal para la teoría de juegos es *Juegos y Comportamiento Económico*.

CAPÍTULO 19

APRENDIENDO DE EJEMPLOS

En el que describimos agentes que mejoran su comportamiento mediante el estudio de experiencias pasadas y predicciones sobre el futuro.

Un agente está **aprendiendo** si mejora su rendimiento después de hacer observaciones sobre el mundo. El aprendizaje va desde un acto tan trivial como anotar una lista de compras, hasta la profundidad de cuando Albert Einstein dedujo una nueva teoría del universo. Cuando el agente es una computadora, lo llamamos **aprendizaje automático**: una computadora observa algunos datos, construye un modelo basado en el aprendizaje automático de datos y utiliza el modelo tanto como en una **hipótesis** sobre el mundo como en un software que permite resolver problemas.

Aprendizaje automático

¿Por qué querríamos que una máquina aprenda? ¿Por qué no sólo programarla de la manera correcta desde el principio? Existen dos razones principales. Primero, los diseñadores no permiten anticipar todas las situaciones futuras posibles. Por ejemplo, un robot diseñado para navegar por laberintos debe aprender la disposición de cada nuevo laberinto que encuentra; un programa para predecir los precios de las acciones debe aprender a adaptarse cuando las condiciones cambian de auge a caída. Segundo, a veces los diseñadores no tienen idea de cómo programar una solución ellos mismos. La mayoría de las personas son buenas para reconocer las caras de los miembros de la familia, pero lo hacen de manera subconsciente, así que aun los mejores programadores no saben cómo programar una computadora para realizar esa tarea, excepto utilizando algoritmos de aprendizaje automático.

En este capítulo, entrelazamos una discusión de varias clases de modelos: árboles de decisión (Sección 19.3), modelos lineales (Sección 19.6), modelos no paramétricos como los vecinos más cercanos (Sección 19.7), modelos de conjunto como los bosques aleatorios (Sección 19.8), con consejos prácticos sobre la construcción de sistemas de aprendizaje automático (Sección 19.9), y discusión de la teoría del aprendizaje automático (Secciones 19.1 a 19.5).

19.1 Formas de aprendizaje

Cualquier componente de un programa de agente permite ser mejorado por el aprendizaje automático. Las mejoras y las técnicas utilizadas para realizarlas, dependen de estos factores:

- Qué *componente* se va a mejorar.
- Qué *conocimientos previos* tiene el agente, que influyen en el *modelo* que construye.
- Qué *datos* y *retroalimentación* sobre esos datos están disponibles.

El Capítulo 2 describió múltiples diseños de agentes. Los **componentes** de estos agentes incluyen:

1. Un mapeo directo de las condiciones en el estado actual a las acciones.
2. Un medio para inferir propiedades relevantes del mundo a partir de la secuencia de percepciones.
3. Información sobre cómo evoluciona el mundo y sobre los resultados de las posibles acciones que el agente puede tomar.

4. Información de utilidad que indica la deseabilidad de los estados del mundo.
5. Información de valor de acción que indica la deseabilidad de las acciones.
6. Metas que describen los estados más deseables.
7. Un generador de problemas, crítico y elemento de aprendizaje que permiten al sistema mejorar.

Cada uno de estos componentes permite ser aprendido. Considere un agente de vehículo autónomo que aprende observando a un conductor humano. Cada vez que el conductor frena, el agente podría aprender una regla de condición-acción para cuándo frenar (componente 1). Al ver muchas imágenes de cámaras que se le dice que contienen autobuses, permite aprender a reconocerlos (componente 2). Al probar acciones y observar los resultados; por ejemplo, al frenar fuerte en una carretera mojada, puede aprender los efectos de sus acciones (componente 3). Luego, cuando recibe quejas de pasajeros que han sido sacudidos durante el viaje, permite aprender un componente útil de su función de utilidad general (componente 4).

La tecnología del aprendizaje automático se ha convertido en una parte estándar de la ingeniería de software. Cada vez que está construyendo un sistema de software, incluso si no lo considera como un agente de IA, los componentes del sistema son mejorados con el aprendizaje automático. Por ejemplo, el software para analizar imágenes de galaxias bajo lente gravitacional se aceleró en un factor de 10 millones con un modelo aprendido por máquina (Hezaveh *et al.*, 2017), mientras que el uso de energía para enfriar los centros de datos se redujo en 40 % con otro modelo aprendido por máquina (Gao, 2014). El ganador del Premio Turing, David Patterson y Jeff Dean, el jefe de IA de Google, declararon el amanecer de una "Edad de oro" para la arquitectura de computadoras debido al aprendizaje automático (Dean *et al.*, 2018).

Hemos visto múltiples ejemplos de modelos para componentes de agentes: modelos atómicos, factorizados y relacionales basados en lógica o probabilidad, etc. Se han ideado algoritmos de aprendizaje para todos ellos.

Conocimiento previo

Este capítulo asume poco **conocimiento previo** por parte del agente: comienza desde cero y aprende de los datos. En la Sección 21.7.2 consideramos el **aprendizaje de transferencia**, en el cual el conocimiento de un dominio se transfiere a un nuevo dominio, de modo que el aprendizaje puede proceder más rápido con menos datos. Sin embargo, asumimos que el diseñador del sistema elige un marco de modelo que permite conducir a un aprendizaje efectivo.

Ir de un conjunto específico de observaciones a una regla general se llama **inducción**; a partir de las observaciones de que el Sol salió todos los días en el pasado, inducimos que el Sol saldrá mañana. Esto difiere de la **deducción** que estudiamos en el Capítulo 7 porque las conclusiones inductivas pueden ser incorrectas, mientras que las conclusiones deductivas están garantizadas para ser correctas si las premisas son correctas.

Este capítulo se centra en problemas donde la entrada es una **representación factorizada**, un vector de valores de atributos. También es posible que la entrada sea cualquier tipo de estructura de datos, incluyendo atómicas y relacionales.

Clasificación

Regresión

Cuando la salida es uno de un conjunto finito de valores (como soleado/nublado/lluvioso o verdadero/falso), el problema de aprendizaje se llama **clasificación**. Cuando es un número (como la temperatura de mañana, medida ya sea como un número entero o un número real), el problema de aprendizaje tiene el nombre (poco conocido[1]) de **regresión**.

[1] Un nombre mejor habría sido aproximación de función o predicción numérica. Pero en 1886 Francis Galton escribió un influyente artículo sobre el concepto de regresión a la media (por ejemplo, los hijos de padres altos probablemente serán más altos que el promedio, pero no tan altos como los padres). Galton mostró grafos con lo que llamó "líneas de regresión", y los lectores asociaron la palabra "regresión" con la técnica estadística de aproximación de funciones en lugar de con el tema de la regresión a la media.

Hay tres tipos de **retroalimentación** que pueden acompañar a las entradas, y que determinan los tres tipos principales de aprendizaje:

Retroalimentación

- En el **aprendizaje supervisado**, el agente observa pares de entrada-salida y aprende una función que mapea de entrada a salida. Por ejemplo, las entradas podrían ser imágenes de cámara, cada una acompañada de una salida que dice "autobús" o "peatón", etc. Una salida como esta se llama **etiqueta**. El agente aprende una función que, al recibir una nueva imagen, predice la etiqueta apropiada. En el caso de las acciones de frenado (componente 1 arriba), una entrada es el estado actual (velocidad y dirección del automóvil, condición de la carretera), y una salida es la distancia que se tomó para frenar. En este caso, un conjunto de valores de salida permite ser obtenido por el agente a partir de sus propias percepciones (después del hecho); el entorno es el maestro, y el agente aprende una función que mapea estados a distancia de parada.

  Aprendizaje supervisado

  Etiqueta
- En el **aprendizaje no supervisado**, el agente aprende patrones en la entrada sin ninguna retroalimentación explícita no supervisada. La tarea de aprendizaje no supervisado más común es el **agrupamiento**: detectar posibles agrupaciones útiles de ejemplos de entrada. Por ejemplo, cuando se muestran millones de imágenes tomadas de Internet, un sistema de visión por computadora permite identificar un gran grupo de imágenes similares que un hablante de inglés llamaría "cats".

  Aprendizaje no supervisado
- En el **aprendizaje por refuerzo**, el agente aprende de una serie de refuerzos: recompensas y castigos. Por ejemplo, al final de una partida de ajedrez, se le dice al agente que ha ganado (una recompensa) o perdido (un castigo). Depende del agente decidir cuáles de las acciones previas al refuerzo fueron las más responsables de ello, y alterar sus acciones para apuntar a más recompensas en el futuro.

  Aprendizaje por refuerzo

19.2 Aprendizaje supervisado

De manera formal, la tarea del aprendizaje supervisado es esta:

> Dado un **conjunto de entrenamiento** de N pares de entrada-salida de ejemplo
>
> $$(x_1,y_1),(x_2,y_2),...(x_N,y_N),$$
>
> donde cada par fue generado por una función desconocida $y = f(x)$, descubre una función h que se aproxima a la verdadera función f.

Conjunto de entrenamiento

La función h se llama **hipótesis** sobre el mundo. Se extrae de un **espacio de hipótesis** $\mathcal{H}$ de posibles funciones. Por ejemplo, el espacio de hipótesis podría ser el conjunto de polinomios de grado 3, o el conjunto de funciones de Javascript, o el conjunto de fórmulas lógicas booleanas 3-SAT.

Espacio de hipótesis

Con un vocabulario alternativo, podemos decir que h es un **modelo** de los datos, extraído de una **clase de modelos** $\mathcal{H}$, o podemos decir que es una función extraída de una **clase de funciones**. Llamamos a la salida y_i la **verdad absoluta**: la verdadera respuesta que estamos pidiendo a nuestro modelo que prediga.

Clase de modelos

Verdad absoluta

¿Cómo elegimos un espacio de hipótesis? Podríamos tener algún conocimiento previo sobre el proceso que generó los datos. Si no, podemos realizar un **análisis de datos exploratorio**: examinando los datos con pruebas estadísticas y visualizaciones: histogramas, grafos de dispersión, grafos de caja, para tener una idea de los datos, y alguna idea de qué espacio de hipótesis podría ser apropiado. O es posible probar múltiples espacios de hipótesis y evaluar cuál funciona mejor.

Análisis de datos exploratorio

¿Cómo elegimos una buena hipótesis dentro del espacio de hipótesis? Podríamos esperar por una **hipótesis consistente**: y h tal que cada x_i en el conjunto de entrenamiento tiene $h(x_i) = y_i$. Con salidas de valor continuo no podemos esperar una coincidencia exacta con la verdad absoluta; en

Hipótesis consistente

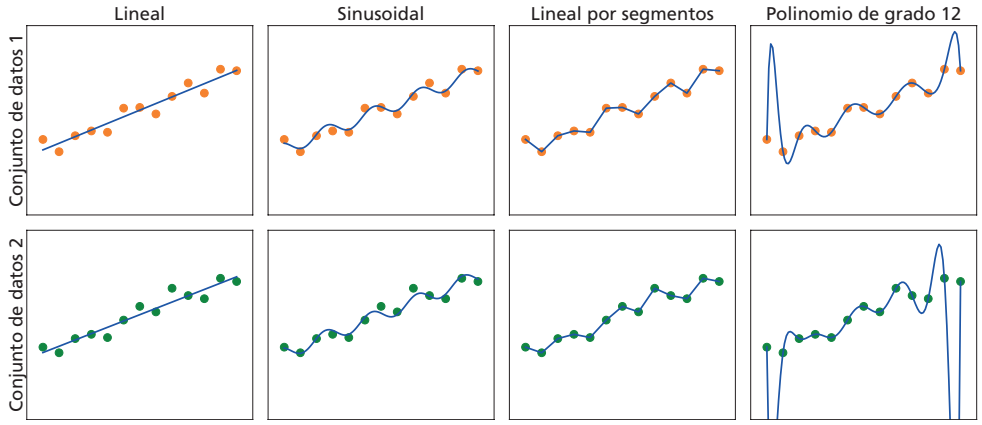


Figura 19.1 Encontrando hipótesis para ajustar los datos. **Fila superior**: cuatro grafos de funciones de mejor ajuste de cuatro espacios de hipótesis diferentes entrenados en el conjunto de datos 1. **Fila inferior**: las mismas cuatro funciones, pero entrenadas en un conjunto de datos ligeramente diferente (muestreado de la misma función $f(x)$).

cambio, nosotros buscamos una **función de mejor ajuste** para la cual cada $h(x_i)$ esté cerca de y_i (de una manera que formalizaremos en la Sección 19.4.2).

La verdadera medida de una hipótesis no es cómo lo hace en el conjunto de entrenamiento, sino más bien cómo maneja las entradas que aún no ha visto. Podemos evaluar eso con una segunda muestra de pares (x_i,y_i) llamada **conjunto de prueba**. Decimos que h **generaliza** bien si predice con precisión las salidas del conjunto de prueba.

Conjunto de prueba
Generalización

La Figura 19.1 muestra que la función h que descubre un algoritmo de aprendizaje depende del espacio de hipótesis $\mathcal{H}$ que considera y del conjunto de entrenamiento que se le da. Cada uno de los cuatro grafos en la fila superior tiene el mismo conjunto de entrenamiento de 13 puntos de datos en el plano (x,y). Los cuatro grafos en la fila inferior tienen un segundo conjunto de 13 puntos de datos; ambos conjuntos son representativos de la misma función desconocida $f(x)$. Cada columna muestra la hipótesis de mejor ajuste h de un espacio de hipótesis diferente:

- **Columna 1**: Líneas rectas; funciones de la forma $h(x) = w_1x + w_0$. No hay ninguna línea que pudiera ser una hipótesis consistente para los puntos de datos.
- **Columna 2**: Funciones sinusoidales de la forma $h(x) = w_1x + \sin(w_0x)$. Esta elección no es del todo consistente, pero se ajusta muy bien a ambos conjuntos de datos.
- **Columna 3**: Funciones lineales segmentadas donde cada segmento de línea conecta los puntos de un punto de datos al siguiente. Estas funciones siempre son consistentes.
- **Columna 4**: Polinomios de grado 12, $h(x) = \sum_{i=0}^{12} w_i x^i$. Estos son consistentes: siempre podemos obtener un polinomio de grado 12 para ajustar 13 puntos distintos. Pero sólo porque la hipótesis es consistente no significa que sea una buena suposición.

Una forma de analizar los espacios de hipótesis es por el sesgo que imponen (independiente del conjunto de datos de entrenamiento) y la varianza que producen (de un conjunto de entrenamiento a otro).

Sesgo

Por **sesgo** entendemos (de manera laxa) la tendencia de una hipótesis predictiva a desviarse del valor esperado cuando se promedia sobre diferentes conjuntos de entrenamiento. El sesgo

en la mayoría de los casos resulta de restricciones impuestas por el espacio de hipótesis. Por ejemplo, el espacio de hipótesis de funciones lineales induce un fuerte sesgo: sólo permite funciones que consisten en líneas rectas. Si hay algún patrón en los datos que no sea la pendiente general de una línea, una función lineal no será capaz de representar esos patrones. Decimos que una hipótesis está **subajustada** cuando no logra encontrar un patrón en los datos. Por otro lado, la función lineal a trozos tiene un sesgo bajo; la forma de la función está determinada por los datos.

Subajuste

Por **varianza** nos referimos a la cantidad de cambio en la hipótesis debido a la fluctuación en los datos de entrenamiento. Las dos filas de la Figura 19.1 representan conjuntos de datos que fueron muestreados de la misma función $f(x)$. Los conjuntos de datos resultaron ser diferentes. Para las primeras tres columnas, la pequeña diferencia en el conjunto de datos se traduce en una pequeña diferencia en la hipótesis. A eso lo llamamos baja varianza. Pero los polinomios de grado 12 en la cuarta columna tienen una alta varianza: observe cuán diferentes son las dos funciones en ambos extremos del eje x. Note que, al menos uno de estos polinomios debe ser una mala aproximación a la verdadera $f(x)$. Decimos que una función está **sobreajustando** los datos cuando presta demasiada atención al conjunto de datos particular en el que se entrena, lo que hace que se desempeñe mal en datos no vistos.

Varianza

Sobreajustando

Con frecuencia existe un **compromiso entre sesgo y varianza**: una elección entre hipótesis más complejas de bajo sesgo que se ajustan bien a los datos de entrenamiento e hipótesis de baja varianza más simples que permiten generalizar mejor. Albert Einstein dijo en 1933, "el objetivo supremo de toda teoría es hacer que los elementos básicos irreducibles sean lo más simples y pocos posibles sin tener que renunciar a la representación adecuada de un solo dato de experiencia". En otras palabras, Einstein recomienda elegir la hipótesis más simple que se ajuste a los datos. Este principio se rastrea hasta el filósofo inglés Guillermo de Occam del siglo XIV.[2] Su principio de que "la pluralidad [de entidades] no debe ser postulada sin necesidad" se llama **navaja de Occam** porque se usa para "rasurar" explicaciones dudosas.

Sesgo-varianza

Definir la simplicidad no es fácil. Parece claro que un polinomio con sólo dos parámetros es más simple que uno con trece parámetros. Haremos esta intuición más precisa en la Sección 19.3.4. Sin embargo, en el Capítulo 21 veremos que los modelos de redes neuronales profundas permiten generalizar muy bien, a pesar de que son muy complejos, algunos de ellos tienen miles de millones de parámetros. Por lo tanto, el número de parámetros por sí solo no es una buena medida de la aptitud de un modelo. Quizá deberíamos apuntar a la "adecuación", no a la "simplicidad" en una clase de modelo. Consideraremos este problema en la Sección 19.4.1.

¿Cuál hipótesis es la mejor en la Figura 19.1? No podemos estar seguros. Si supiéramos que los datos representan, digamos, el número de visitas a un sitio web que crece de día en día, pero también cicla dependiendo de la hora del día, entonces podríamos favorecer la función sinusoidal. Si supiéramos que los datos no son cíclicos, pero tienen mucho ruido, eso favorecería la función lineal.

En algunos casos, un analista está dispuesto a decir no sólo que una hipótesis es posible o imposible, sino cuán probable es. El aprendizaje supervisado se puede hacer eligiendo la hipótesis h^* que es más probable dada la data:

$$h^* = \underset{h \in \mathcal{H}}{\text{argmax}}\, P(h|data).$$

Por la regla de Bayes esto es equivalente a

$$h^* = \underset{h \in \mathcal{H}}{\text{argmax}}\, P(\text{data}|h)P(h).$$

[2] El nombre con frecuencia se escribe de manera indistinta como Occam u Ockham.

Entonces, podemos decir que la probabilidad previa $P(h)$ es alta para un polinomio suave de grado 1 o 2 y más baja para un polinomio de grado 12 con picos grandes y agudos. Permitimos funciones de aspecto inusual cuando los datos dicen que las necesitamos, pero las desalentamos dándoles una baja probabilidad previa.

¿Por qué no dejar que $\mathcal{H}$ sea la clase de todos los programas de computadora, o todas las máquinas de Turing? El problema es que *hay un equilibrio entre la expresividad de un espacio de hipótesis y la complejidad computacional de encontrar una buena hipótesis dentro de ese espacio*. Por ejemplo, ajustar una línea recta a los datos es un cálculo fácil, ajustar polinomios de alto grado es algo más difícil y ajustar las máquinas de Turing es indecidible. Una segunda razón para preferir espacios de hipótesis simples es que querremos usar h después de haberlo aprendido, y calcular $h(x)$ cuando h es una función lineal está garantizado que será rápido, mientras que calcular un programa de máquina de Turing arbitrario ni siquiera está garantizado que termine.

Por estas razones, la mayoría de los trabajos sobre aprendizaje se han centrado en representaciones simples. En los últimos años ha habido un gran interés en el aprendizaje profundo (Capítulo 21), donde las representaciones no son simples pero donde el cálculo $h(x)$ aún sólo toma un **número limitado de pasos** para calcular con el hardware apropiado.

Veremos que el equilibrio entre expresividad y complejidad no es simple: como vimos con la lógica de primer orden, Capítulo 8, que un lenguaje expresivo hace posible que una hipótesis *simple* se ajuste a los datos, mientras que restringir la expresividad del lenguaje significa que cualquier hipótesis consistente debe ser compleja.

19.2.1 Problema de ejemplo: Espera en el restaurante

Describiremos un problema de aprendizaje supervisado en detalle: el problema de decidir si esperar una mesa en un restaurante. Este problema se utilizará a lo largo del capítulo para demostrar diferentes clases de modelos. Para este problema, la salida, y, es una variable booleana que llamaremos *WillWait,* que es verdadera para los ejemplos en los que esperamos una mesa. La entrada, x, es un vector de diez valores de atributos, cada uno de los cuales tiene valores discretos:

1. *Alternativa*: si hay un restaurante alternativo adecuado cerca.
2. *Bar*: si el restaurante tiene un área de bar cómoda para esperar.
3. *Fri/Sat*: verdadero los viernes y sábados.
4. *Hambriento*: si tenemos hambre ahora mismo.
5. *Patrones*: cuántas personas hay en el restaurante (los valores son *Ninguno*, *Algunos* y *Lleno*).
6. *Precio*: el rango de precios del restaurante ($, $$, $$$).
7. *Lluvia*: si está lloviendo afuera.
8. *Reserva*: si hicimos una reservación.
9. *Tipo*: el tipo de restaurante (francés, italiano, tailandés o hamburguesa).
10. *Estimación de espera*: estimación de espera del anfitrión: 0–10, 10–30, 30–60, o >60 minutos.

Un conjunto de 12 ejemplos, tomados de la experiencia de uno de nosotros (SR), se muestra en la Figura 19.2. Observe lo escasos que son estos datos: hay $2^6 \times 3^2 \times 4^2 = 9\,216$ posibles combinaciones de valores para los atributos de entrada, pero sólo se nos da la salida correcta para 12 de ellos; cada uno de los otros 9 204 podría ser verdadero o falso, no lo sabemos. Esta es la esencia de la inducción: necesitamos hacer nuestra mejor suposición en estos 9 204 valores de salida faltantes, dada sólo la evidencia de los 12 ejemplos.

Ejemplo	Atributos de entrada										Atributos de salida
	Alt	*Bar*	*Viernes*	*Hambriento*	*Clientes*	*Pre*	*Lluvia*	*Reservación*	*Tipo*	*Estimación*	*Va a esperar*
$\mathbf{x}_1$	*Sí*	*No*	*No*	*Sí*	*Algo*	*$$$*	*No*	*Sí*	*Fran*	*0-10*	y_1 = Sí
$\mathbf{x}_2$	*Sí*	*No*	*No*	*Sí*	*Lleno*	*$*	*No*	*No*	*Thai*	*30-60*	y_2 = No
$\mathbf{x}_3$	*No*	*Sí*	*No*	*No*	*Algo*	*$*	*No*	*No*	*Burger*	*0-10*	y_3 = Sí
$\mathbf{x}_4$	*Sí*	*No*	*Sí*	*Sí*	*Lleno*	*$*	*Sí*	*No*	*Thai*	*10-30*	y_4 = Sí
$\mathbf{x}_5$	*Sí*	*No*	*Sí*	*No*	*Lleno*	*$$$*	*No*	*Sí*	*Fran*	*>60*	y_5 = No
$\mathbf{x}_6$	*No*	*Sí*	*No*	*Sí*	*Algo*	*$$*	*Sí*	*Sí*	*Itali*	*0-10*	y_6 = Sí
$\mathbf{x}_7$	*No*	*Sí*	*No*	*No*	*Nada*	*$*	*Sí*	*No*	*Burger*	*0-10*	y_7 = No
$\mathbf{x}_8$	*No*	*No*	*No*	*Sí*	*Algo*	*$$*	*Sí*	*Sí*	*Thai*	*0-10*	y_8 = Sí
$\mathbf{x}_9$	*No*	*Sí*	*Sí*	*No*	*Lleno*	*$*	*Sí*	*No*	*Burger*	*>60*	y_9 = No
$\mathbf{x}_{10}$	*Sí*	*Sí*	*Sí*	*Sí*	*Lleno*	*$$$*	*No*	*Sí*	*Itali*	*10-30*	y_{10} = No
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*Nada*	$	*No*	*No*	*Tahi*	*0-10*	y_{11} = No
$\mathbf{x}_{12}$	*Sí*	*Sí*	*Sí*	*Sí*	*Lleno*	$	*No*	*No*	*Burger*	*30-60*	y_{12} = Sí

Figura 19.2 Ejemplos para el dominio restaurante.

19.3 Aprendizaje de árboles de decisión

Árbol de decisión

Un **árbol de decisión** es una representación de una función que asigna un vector de valores de atributos a un único valor de salida, una "decisión". Un árbol de decisión llega a su decisión realizando una secuencia de pruebas, comenzando en la raíz y siguiendo la rama correspondiente hasta que se alcanza una hoja. Cada nodo interno en el árbol corresponde a una prueba del valor de uno de los atributos de entrada, las ramas desde el nodo están etiquetadas con los posibles valores del atributo, y los nodos hoja especifican qué valor se debe devolver por la función.

En general, los valores de entrada y salida pueden ser discretos o continuos, pero por ahora se considerarán sólo entradas consistentes en valores discretos y salidas que sean *verdaderas* (un ejemplo **positivo**) o *falsas* (un ejemplo **negativo**). A esto se le llama **clasificación booleana**. Se usará j para indexar los ejemplos ($\mathbf{x}_j$ es el vector de entrada para el ejemplo j-*ésimo* y y_j es la salida), y $x_{j,i}$ para el i-*ésimo* atributo del ejemplo j-*ésimo*.

Positivo

Negativo

El árbol que representa la función de decisión que SR usa para el problema del restaurante se muestra en la Figura 19.3. Siguiendo las ramas, se ve que un ejemplo con *Clientes* = *Lleno* y *Va a esperar* = 0–10 se clasificará como positivo (es decir, sí, se esperará por una mesa).

19.3.1 Expresividad de los árboles de decisión

Un árbol de decisión booleano es equivalente a una declaración lógica de la forma:

$$Salida \Leftrightarrow (Path_1 | \vee Path_2 \vee \cdots),$$

donde cada $Path_1$ es una conjunción de la forma $(A_m = v_x \wedge A_n = v_y \wedge \cdots)$ de pruebas de valor de atributo correspondientes a un camino desde la raíz hasta una hoja *verdadera*. Así, toda la expresión está en forma normal disyuntiva, lo que significa que cualquier función en lógica proposicional se puede expresar como un árbol de decisión.

Para muchos problemas, el formato de árbol de decisión produce un resultado claro, conciso y comprensible. De hecho, múltiples manuales de "Cómo hacer" (por ejemplo, para reparar autos) están escritos como árboles de decisión. Pero algunas funciones no se pueden representar de

manera concisa. Por ejemplo, la función de mayoría, que devuelve verdadero si y sólo si más de la mitad de las entradas son verdaderas, requiere un árbol de decisión exponencialmente grande, al igual que la función de paridad, que devuelve verdadero si y sólo si un número par de atributos de entrada son verdaderos. Con atributos de valor real, la función $y > A_1 + A_2$ es difícil de representar con un árbol de decisión porque el límite de decisión es una línea diagonal, y todas las pruebas de un árbol de decisión dividen el espacio en cajas rectangulares alineadas con los ejes. Habría que apilar muchas cajas para aproximar de cerca la línea diagonal. En otras palabras, los árboles de decisión son buenos para algunos tipos de funciones y malos para otros.

¿Existe *algún* tipo de representación que sea eficiente para todos los tipos de funciones? Desafortunadamente la respuesta es no, existen demasiadas funciones para poder representarlas todas con un pequeño número de bits. Incluso considerando sólo funciones booleanas con n atributos booleanos, la tabla de verdad tendrá 2^n filas, y cada fila puede devolver *verdadero* o *falso*, por lo que hay 2^{2^n} funciones diferentes. Con 20 atributos, hay $2^{1\,048\,576} \approx 10^{300\,000}$ funciones, por lo que, si se limita a una representación de un millón de bits, no es posible representar todas estas funciones.

19.3.2 Aprendiendo árboles de decisión a partir de ejemplos

Se desea encontrar un árbol que sea consistente con los ejemplos en la Figura 19.2 y que sea lo más pequeño posible. Sin embargo, es complicado encontrar un árbol consistentemente más pequeño. Pero con algunos heurísticos simples, es posible encontrar uno de manera eficiente que esté cerca del más pequeño posible. El algoritmo LEARN-DECISION-TREE adopta una estrategia codiciosa de dividir y conquistar: siempre se prueba primero el atributo más importante, y luego se resuelven de manera recursiva los subproblemas más pequeños definidos por los posibles resultados de la prueba. Por "atributo más importante" se refiere a aquel que hace la mayor diferencia en la clasificación de un ejemplo. De esta manera, se espera llegar a la clasificación correcta con un pequeño número de pruebas, lo que significa que todos los caminos en el árbol serán cortos y el árbol en su conjunto será poco profundo.

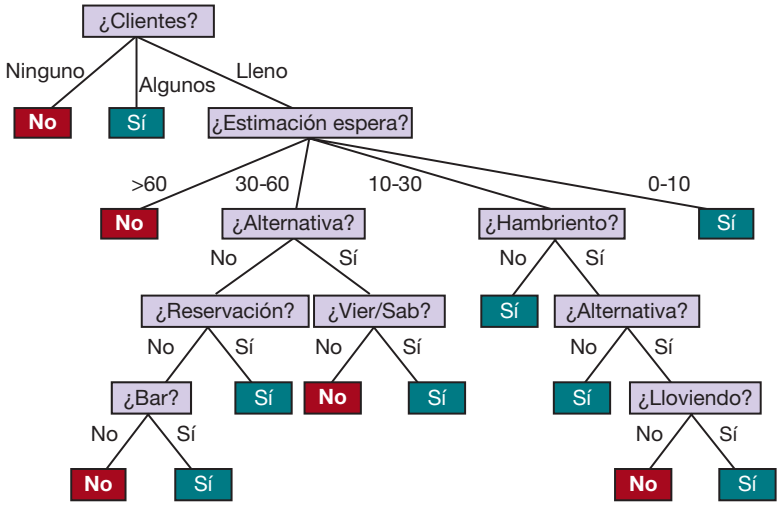


Figura 19.3 Árbol de decisiones para decidir si esperar una mesa.

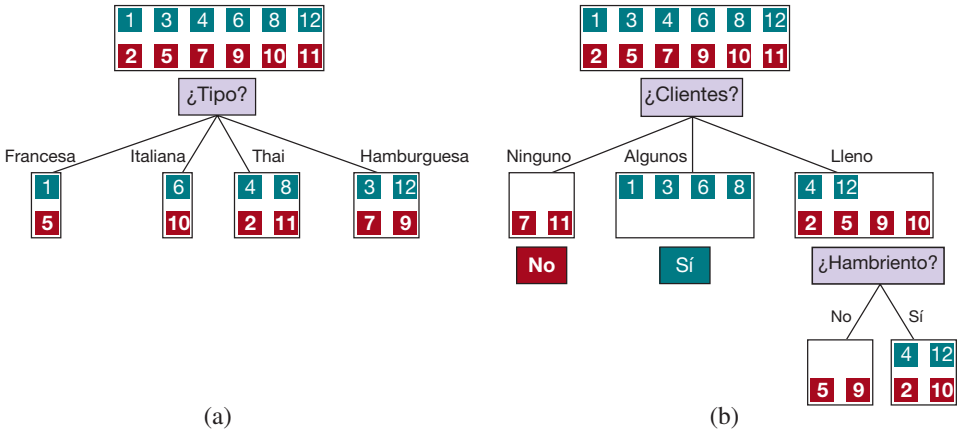


Figura 19.4 Dividiendo los ejemplos mediante pruebas en los atributos. En cada nodo se muestran los ejemplos positivos (cajas claras) y negativos (cajas oscuras) que quedan. (a) Dividir por *Tipo* no nos acerca a distinguir entre ejemplos positivos y negativos. (b) Dividir por *Clientes* hace un buen trabajo al separar ejemplos positivos y negativos. Después de dividir por *Clientes*, *Hambriento* es una segunda prueba bastante buena.

La Figura 19.4(a) muestra que *Tipo* es un atributo pobre, porque deja cuatro posibles resultados, cada uno de los cuales tiene el mismo número de ejemplos positivos que negativos. Por otro lado, en (b) se ve que *Clientes* es un atributo bastante importante, porque si el valor es *Ninguno* o *Algunos*, entonces nos queda conjuntos de ejemplos para los cuales se puede responder de manera definitiva (*No* y *Sí*, respectivamente). Si el valor es *Lleno*, se deja un conjunto mixto de ejemplos. Hay cuatro casos a considerar para estos subproblemas recursivos:

1. Si los ejemplos restantes son todos positivos (o todos negativos), entonces se ha terminado: se puede responder *Sí* o *No*. La Figura 19.4(b) muestra ejemplos de esto en las ramas *Ninguno* y *Algunos*.
2. Si hay algunos ejemplos positivos y algunos negativos, entonces se elige el mejor atributo para dividirlos. La Figura 19.4(b) muestra que se usa *Hambriento* para dividir los ejemplos restantes.
3. Si no quedan ejemplos, significa que no se ha observado ningún ejemplo para esta combinación de valores de atributos, y se devuelve el valor de salida más común del conjunto de ejemplos que se usaron para construir el nodo padre.
4. Si no quedan atributos, pero hay tanto ejemplos positivos como negativos, significa que estos ejemplos tienen exactamente la misma descripción, pero diferentes clasificaciones. Esto puede suceder porque hay un error o **ruido** en los datos, porque el dominio no es determinista o porque no se puede observar un atributo que distinguiría los ejemplos. Lo mejor que se puede hacer es devolver el valor de salida más común de los ejemplos restantes.

Ruido

El algoritmo LEARN-DECISION-TREE se muestra en la Figura 19.5. Note que el conjunto de ejemplos es una entrada al algoritmo, pero en ninguna parte los ejemplos aparecen en el árbol devuelto por el algoritmo. Un árbol consiste en pruebas sobre atributos en los nodos interiores, valores de atributos en las ramas y valores de salida en los nodos hoja. Los detalles de la función IMPORTANCE se dan en la Sección 19.3.3. El resultado del algoritmo de aprendizaje en nuestro conjunto de

function LEARN-DECISION-TREE(*examples*,*attributes*,*parent_examples*) **returns** un **árbol**

if *examples* está vacío **then return** PLURALITY-VALUE(*parent_examples*)
else if todos los *examples* tienen la misma clasificación **then return** la clasificación
else if attributes está vacío **then return** PLURALITY-VALUE(examples)
else
  $A \leftarrow \text{argmax}_{a \in attributes}$ IMPORTANCE(a,*examples*)
  Tree ← Un nuevo árbol de decisión con prueba de raíz A
  for each valor v de A **do**
    exs←{e : $e \in examples$ **and** $e.A = v$}
    subtree←LEARN-DECISION-TREE(exs,*attributes*−A,*examples*)
    agregue una rama a *tree* con etiqueta ($A = v$) y subárbol *subtree*
  return *tree*

Figura 19.5 Algoritmo de aprendizaje del árbol de decisiones. La función IMPORTANCE se describe en la Sección 19.3.3. La función PLURALITY-VALUE selecciona el valor de salida más común entre un conjunto de ejemplos y deshace los empates de forma aleatoria.

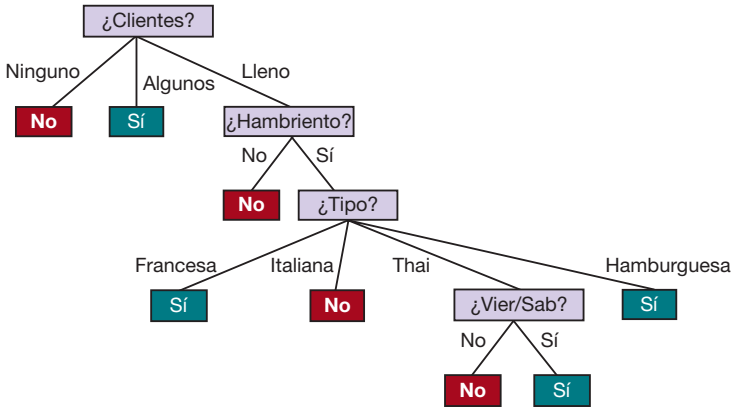


Figura 19.6 El árbol de decisiones inducido a partir del conjunto de entrenamiento de 12 ejemplos.

entrenamiento de ejemplo se muestra en la Figura 19.6. Claramente, el árbol es diferente del árbol original mostrado en la Figura 19.3. Uno podría concluir que el algoritmo de aprendizaje no está haciendo un muy buen trabajo de aprendizaje de la función correcta. Sin embargo, esta sería una conclusión equivocada. El algoritmo de aprendizaje mira los *ejemplos*, no la función correcta y, de hecho, su hipótesis (ver Figura 19.6) no sólo es consistente con todos los ejemplos, sino que ¡es por mucho más simple que el árbol original! Con ejemplos ligeramente diferentes, el árbol podría ser muy desemejante, pero la función que representa sería similar.

El algoritmo de aprendizaje no tiene ninguna razón para incluir pruebas para *Lluvia* y *Reservación*, porque permite clasificar todos los ejemplos sin ellos. También ha detectado un patrón interesante y previamente sospechado: SR esperará por comida tailandesa los fines de semana. También está destinado a cometer algunos errores en casos donde no ha visto ejemplos. Por ejemplo, nunca ha visto un caso donde la espera es de 0–10 minutos pero el restaurante está

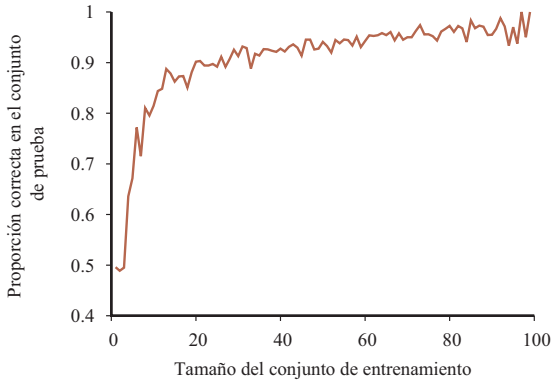


Figura 19.7 Curva de aprendizaje para el algoritmo del árbol de decisión de aprendizaje en 100 ejemplos generados de manera aleatoria en el dominio del restaurante. Cada punto de datos es el promedio de 20 ensayos.

lleno. En ese caso dice que no espere cuando *Hambriento* es falso, pero SR esperaría. Con más ejemplos de entrenamiento el programa de aprendizaje podría corregir este error.

Podemos evaluar el rendimiento de un algoritmo de aprendizaje con una **curva de aprendizaje**, como se muestra en la Figura 19.7. Para esta figura tenemos 100 ejemplos a nuestra disposición, que dividimos de manera aleatoria en un conjunto de entrenamiento y un conjunto de prueba. Aprendemos una hipótesis h con el conjunto de entrenamiento y medimos su precisión con el conjunto de prueba. Podemos hacer esto empezando con un conjunto de entrenamiento de tamaño 1 e incrementando uno a la vez hasta el tamaño 99. Para cada tamaño, en realidad repetimos el proceso de dividir aleatoriamente en conjuntos de entrenamiento y prueba 20 veces, y promediamos los resultados de los 20 ensayos. La curva muestra que a medida que el tamaño del conjunto de entrenamiento crece, la precisión aumenta. (Por esta razón, las curvas de aprendizaje también se llaman **grafos felices**). En este grafo alcanzamos 95 % de precisión, y parece como si la curva podría continuar aumentando si tuviéramos más datos.

Curva de aprendizaje

Grafos felices

19.3.3 Elegir pruebas de atributos

El algoritmo del árbol de decisión de aprendizaje elige el atributo con la mayor IMPORTANCIA. Ahora vamos a mostrar cómo medir esta importancia utilizando la noción de ganancia de información, que es definida en términos de **entropía**, la cantidad fundamental en la teoría de la información (Shannon y Weaver, 1949).

Entropía

Entropía es una medida de la incertidumbre de una variable aleatoria: cuanta más información, menos entropía. Una variable aleatoria con un solo valor posible, una moneda que siempre cae cara, no tiene incertidumbre y, por lo tanto, su entropía se define como cero. Una moneda imparcial es igualmente probable que caiga cara o cruz cuando se lanza, y pronto mostraremos que esto cuenta como "1 bit" de entropía. El lanzamiento de un dado imparcial de *cuatro* caras tiene 2 bits de entropía, porque hay 22 opciones igualmente probables. Ahora considere una moneda cargada que marca cara 99 % de las veces. De manera intuitiva, esta moneda tiene menos incertidumbre que la moneda imparcial, si adivinamos cara estaremos equivocados sólo 1 % de las veces, por lo que nos gustaría que tuviera una medida de entropía que esté cerca de cero, pero positiva. En general, la entropía de una variable aleatoria V con valores v_k teniendo probabilidad

$P(v_k)$ se define como

Entropía: $$H(V) = \sum_k P(v_k)\log_2 \frac{1}{P(v_k)} = -\sum_k P(v_k)\log_2 P(v_k).$$

Podemos comprobar que la entropía de un lanzamiento de moneda imparcial es de hecho 1 bit:

$$H(Fair) = -(0.5\log_2 0.5 + 0.5\log_2 0.5) = 1.$$

Y de un dado de cuatro caras es de 2 bits:

$$H(Die4) = -(0.25\log_2 0.25 + 0.25\log_2 0.25 + 0.25\log_2 0.25 + 0.25\log_2 0.25) = 2$$

Para la moneda cargada con 99 % de caras, obtenemos

$$H(Loaded) = -(0.99\log_2 0.99 + 0.01\log_2 0.01) \approx 0.08 \text{ bits}.$$

Será útil definir $B(q)$ como la entropía de una variable aleatoria booleana que es verdadera con probabilidad q:

$$B(q) = -(q\log_2 q + (1-q)\log_2(1-q)).$$

Así, $H(Loaded) = B(0.99) \approx 0.08$. Ahora volvamos al aprendizaje del árbol de decisión. Si un conjunto de entrenamiento contiene p ejemplos positivos y n ejemplos negativos, entonces la entropía de la variable de salida en todo el conjunto es

$$H(Output) = B\left(\frac{p}{p+n}\right)$$

El conjunto de entrenamiento del restaurante en la Figura 19.2 tiene $p = n = 6$, por lo que la entropía correspondiente es $B(0.5)$ o 1 bit exacto. El resultado de una prueba en un atributo A nos dará algo de información, lo que reduce la entropía total en una cantidad. Podemos medir esta reducción mirando la entropía restante después de la prueba del atributo.

Un atributo A con d valores distintos divide el conjunto de entrenamiento E en subconjuntos $E_1,\ldots,E_d$. Cada subconjunto E_k tiene p_k ejemplos positivos y n_k ejemplos negativos, por lo que si seguimos esa rama, necesitaremos un adicional $B(p_k/(p_k + n_k))$ bits de información para responder a la pregunta. Un ejemplo elegido al azar del conjunto de entrenamiento tiene el valor *k-iesimo* para el atributo (es decir, es E_k con probabilidad $(p_k + n_k)/(p + n)$), por lo que la entropía esperada restante después de probar el atributo A es

$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right).$$

Ganancia de información

La **ganancia de información** del test del atributo en A es la reducción esperada en entropía:

$$Gain(A) = B\left(\frac{p}{p+n}\right) - Remainder(A).$$

De hecho, $Gain(A)$ es justo lo que necesitamos para implementar la función Importancia. Volviendo a los atributos considerados en la Figura 19.4, tenemos

$$Gain(Patrons) = 1 - \left[\tfrac{2}{12}B\left(\tfrac{0}{2}\right) + \tfrac{4}{12}B\left(\tfrac{4}{4}\right) + \tfrac{6}{12}B\left(\tfrac{2}{6}\right)\right] \approx 0.541 \text{ bits},$$

$$Gain(Type) = 1 - \left[\tfrac{2}{12}B\left(\tfrac{1}{2}\right) + \tfrac{2}{12}B\left(\tfrac{1}{2}\right) + \tfrac{4}{12}B\left(\tfrac{2}{4}\right) + \tfrac{4}{12}B\left(\tfrac{2}{4}\right)\right] = 0 \text{ bits},$$

confirmando nuestra intuición de que *Clientes* es un mejor atributo para dividir primero. De hecho, *Clientes* tiene la máxima ganancia de información de cualquiera de los atributos y, en consecuencia, sería elegido por el algoritmo de aprendizaje del árbol de decisión como la raíz.

19.3.4 Generalización y sobreajuste

Queremos que nuestros algoritmos de aprendizaje encuentren una hipótesis que se ajuste a los datos de entrenamiento; pero más importante aún, queremos que se generalice bien para datos no vistos antes. En la Figura 19.1 vimos que un polinomio de alto grado permite ajustarse a todos los datos, pero tiene oscilaciones salvajes que no están justificadas por los datos: se ajusta, pero puede sobreajustarse. El sobreajuste es más probable a medida que aumenta el número de atributos, pero es menos probable a medida que aumentamos el número de ejemplos de entrenamiento. Los espacios de hipótesis más grandes (por ejemplo, árboles de decisión con más nodos o polinomios de alto grado) tienen más capacidad tanto para ajustarse como para sobreajustarse; algunas clases de modelos son más propensas al sobreajuste que otras.

Poda de árbol de decisión

Para los árboles de decisión, una técnica llamada **poda de árboles de decisión** combate el sobreajuste. La poda funciona eliminando nodos que no son relevantes. Comenzamos con un árbol completo, tal como lo generó LEARN-DECISION-TREE. Luego miramos un nodo de prueba que sólo tiene nodos hoja como descendientes. Si la prueba parece ser irrelevante, detectando sólo ruido en los datos, entonces eliminamos la prueba y la reemplazamos por un nodo hoja. Repetimos este proceso considerando cada prueba con sólo descendientes de hojas hasta que cada uno ha sido podado o aceptado tal como está.

La pregunta es ¿cómo detectamos que un nodo está probando un atributo irrelevante? Suponga que estamos en un nodo que consta de p ejemplos positivos y n ejemplos negativos. Si el atributo es irrelevante, esperaríamos que dividiera los ejemplos en subconjuntos de manera que cada subconjunto tenga cerca de la misma proporción de ejemplos positivos que todo el conjunto, $p/(p + n)$ y, en consecuencia, la ganancia de información estará cerca de cero.[3] Por lo tanto, una baja ganancia de información es una buena pista de que el atributo es irrelevante. Ahora la pregunta es ¿cuánta ganancia deberíamos requerir para dividir en un atributo particular?

Prueba de significación

Hipótesis nula

Podemos responder a esta pregunta utilizando una **prueba de significación** estadística. Tal prueba comienza asumiendo que no hay un patrón subyacente (la llamada **hipótesis nula**). Luego, los datos reales se analizan para calcular hasta qué punto se desvían de una ausencia perfecta de patrón. Si el grado de desviación es estadísticamente improbable (por lo general se toma una probabilidad de 5 % o menos), entonces eso se considera una buena evidencia de la presencia de un patrón significativo en los datos. Las probabilidades se calculan a partir de distribuciones estándar de la cantidad de desviación que se esperaría ver en un muestreo aleatorio.

En este caso, la hipótesis nula es que el atributo es irrelevante y, por lo tanto, que la ganancia de información para una muestra infinitamente grande sería cero. Necesitamos calcular la probabilidad de que, bajo la hipótesis nula, una muestra de tamaño $\nu = n + p$ muestre la desviación observada de la distribución esperada de ejemplos positivos y negativos. Podemos medir la desviación comparando los números reales de ejemplos positivos y negativos en cada subconjunto, p_k y n_k, con los números esperados, $\hat{p}_k$ y $\hat{n}_k$, asumiendo verdadera irrelevancia:

$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n} \qquad \hat{n}_k = n \times \frac{p_k + n_k}{p + n}.$$

Una medida conveniente de la desviación total está dada por

$$\Delta = \sum_{k=1}^{d} \frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k}.$$

Bajo la hipótesis nula, el valor de Δ se distribuye de acuerdo con la χ^2 (chi-cuadrada) con d - 1

[3] La ganancia será positiva excepto en el caso poco probable de que todas las proporciones sean *exactamente* las mismas. (Ver Ejercicio 19.NNGA).

grados de libertad. Podemos usar una función de estadísticas χ^2 para ver si un valor particular de Δ confirma o rechaza la hipótesis nula. Por ejemplo, considere el atributo *Tipo* de restaurante con cuatro valores y, por lo tanto, tres grados de libertad. Un valor de Δ= 7.82 o más rechazaría la hipótesis nula al nivel de 5 % (y un valor de Δ = 11.35 o más rechazaría al nivel de 1 %). Valores por debajo de eso llevan a aceptar la hipótesis de que el atributo es irrelevante y, en consecuencia, la rama asociada del árbol debe ser podada. Esto se conoce como **poda χ^2**.

Poda χ^2

Con la poda se tolera el ruido en los ejemplos. Los errores en la etiqueta del ejemplo (un ejemplo (**x**, *Sí*) que debería ser (**x**, *No*)) dan un aumento lineal en el error de predicción, mientras que los errores en las descripciones de los ejemplos (como, *Precio* = $ cuando en realidad era *Precio* = $$) tienen un efecto asintótico que empeora a medida que el árbol se reduce a conjuntos más pequeños. Los árboles podados rinden mejor que los árboles no podados cuando los datos contienen una gran cantidad de ruido. Además, los árboles podados suelen ser mucho más pequeños y, en consecuencia, más fáciles de entender y más eficientes de ejecutar.

Una advertencia final: podría pensar que la poda χ^2 y la ganancia de información parecen similares, pero por qué no combinarlas usando un enfoque llamado **detención temprana**, hacer que el algoritmo del árbol de decisiones deje de generar nodos cuando no hay un buen atributo para dividir, en lugar de pasar por todo el problema de generar nodos y luego podarlos. El problema con la detención temprana es que nos impide reconocer situaciones en las que no hay un buen atributo, pero hay combinaciones de atributos que son informativos. Por ejemplo, considere la función XOR de dos atributos binarios. Si hay aproximadamente la misma cantidad de ejemplos para las cuatro combinaciones de valores de entrada, entonces ninguno de los atributos será informativo; no obstante, lo correcto es dividir en uno de los atributos (no importa cuál) y, luego, en el segundo nivel obtendremos divisiones que son muy informativas. La detención temprana se perdería esto, pero generar y luego podar lo maneja de manera correcta.

Detención temprana

19.3.5 Ampliando la aplicabilidad de los árboles de decisión

Los árboles de decisión son más útiles al manejar las siguientes complicaciones:

- **Datos faltantes**: En muchos dominios no se conocerán todos los valores de los atributos para cada ejemplo. Los valores podrían no haberse registrado o podrían ser demasiado costosos de obtener. Esto da lugar a dos problemas: primero, dado un árbol de decisión completo, ¿cómo deberíamos clasificar un ejemplo donde falta uno de los atributos de prueba?; segundo, ¿cómo deberíamos modificar la fórmula de ganancia de información cuando algunos ejemplos tienen valores desconocidos para el atributo? Estas preguntas se abordan en el Ejercicio 19.MISS.
- **Atributos de entrada continuos y multivaluados**: Para atributos continuos como *Altura*, *Peso* o *Tiempo* puede ser que cada ejemplo tenga un valor de atributo diferente. La medida de ganancia de información daría su puntuación más alta a tal atributo, lo que daría un árbol superficial con este atributo en la raíz, y subárboles de un solo ejemplo para cada valor posible debajo de él. Pero eso no ayuda cuando obtenemos un nuevo ejemplo para clasificar con un valor de atributo que no hemos visto antes.

  Una mejor manera de lidiar con los valores continuos es una prueba de **punto de división**, una prueba de desigualdad en el valor de un atributo. Por ejemplo, en un nodo dado en el árbol, podría ser el caso que la prueba en *Peso* > 160 proporciona la mayor información. Existen métodos eficientes para encontrar buenos puntos de división: comienza ordenando los valores del atributo y luego considera sólo los puntos de división que están entre dos ejemplos en disposición ordenada que tienen diferentes clasificaciones, mientras se lleva un registro de los totales acumulados de ejemplos positivos y negativos a cada lado del punto

Punto de división

de división. La división es la parte más costosa de las aplicaciones de aprendizaje de árboles de decisión en el mundo real.

Para atributos que no son continuos y no tienen un orden significativo, pero tienen una gran cantidad de valores posibles (por ejemplo, *Código postal* o *Número de tarjeta de crédito*), una medida llamada **relación de ganancia de información** (ver Ejercicio 19.GAIN) permite ser utilizada para evitar la división en muchos subárboles de un solo ejemplo. Otro enfoque útil es permitir una **prueba de igualdad** de la forma $A = v_k$. Por ejemplo, la prueba *Código postal* = 10 002 podría usarse para seleccionar un gran grupo de personas en este código postal en la ciudad de Nueva York, y agrupar a todos los demás en el subárbol "otro".

- **Atributo de salida de valor continuo**: Si se trata de predecir una salida numérica de valor, como el precio de un apartamento, entonces necesitamos un **árbol de regresión**, no un árbol de clasificación. Un árbol de regresión tiene en cada hoja una función lineal de algún subconjunto de atributos numéricos, en lugar de un solo valor de salida. Así, la rama para apartamentos de dos habitaciones podría terminar con una función lineal de los pies cuadrados y el número de baños. El algoritmo de aprendizaje debe decidir cuándo dejar de dividir y cuando aplicar la regresión lineal (ver Sección 19.6) sobre los atributos. El nombre **CART**, que en español significa Árboles de Clasificación y Regresión, se usa para cubrir ambas clases.

Árbol de regresión

CART

Un sistema de aprendizaje de árboles de decisión para aplicaciones del mundo real debe ser capaz de manejar todos estos problemas. El manejo de variables de valor continuo es en especial importante debido a que tanto los procesos físicos como financieros proporcionan datos numéricos. Se han construido varios paquetes comerciales que cumplen con estos criterios y se han utilizado para desarrollar miles de sistemas en el campo. En muchas áreas de la industria y el comercio los árboles de decisión son el primer método que se prueba cuando se va a extraer un método de clasificación de un conjunto de datos.

Los árboles de decisión tienen muchas ventajas: facilidad de comprensión, escalabilidad a grandes conjuntos de datos y versatilidad en el manejo de entradas discretas y continuas, así como clasificación y regresión. Sin embargo, pueden tener una precisión subóptima (principalmente debido a la búsqueda codiciosa) y, si los árboles son muy profundos, entonces obtener una predicción para un nuevo ejemplo es costoso en tiempo de ejecución. Los árboles de decisión también son **inestables** así que agregar sólo un nuevo ejemplo puede cambiar la prueba en la raíz, lo que cambia todo el árbol. En la Sección 19.8.2 veremos que el **modelo de bosque aleatorio** permite solucionar algunos de estos problemas.

Inestable

Modelo de bosque aleatorio

19.4 Selección y optimización del modelo

Nuestro objetivo en el aprendizaje automático es seleccionar una hipótesis que se ajuste de manera óptima a ejemplos futuros. Para hacer eso con precisión necesitamos definir "ejemplo futuro" y "ajuste óptimo".

Primero haremos la suposición de que los ejemplos futuros serán como los del pasado. Llamamos a esto la suposición de **estacionariedad**; sin ella, todas las apuestas están apagadas. Asumimos que cada ejemplo E_j tiene la misma distribución de probabilidad previa:

Estacionariedad

$$\mathbf{P}(E_j) = \mathbf{P}(E_{j+1}) = \mathbf{P}(E_{j+2}) = ...,$$

y es independiente de los ejemplos anteriores:

$$\mathbf{P}(E_j) = \mathbf{P}(E_j | E_{j-1}, E_{j-2}, ...).$$

Los ejemplos que satisfacen estas ecuaciones son *independientes e idénticamente distribuidos* o **i.i.d**.

i.i.d.

Tasa de error

El siguiente paso es definir el "ajuste óptimo". Por ahora, se dirá que el ajuste óptimo es la hipótesis que minimiza la **tasa de error**: la proporción de veces que $h(x) \neq y$ para un ejemplo (x,y). (Más adelante se ampliará esto para permitir que diferentes errores tengan distintos costos, dando en efecto crédito parcial por respuestas que estén "casi" correctas). Es posible estimar la tasa de error de una hipótesis sometiéndola a una prueba: midiendo su desempeño en un conjunto de pruebas. Sería trampa para una hipótesis (o un estudiante) echar un vistazo a las respuestas antes de realizar la prueba. La manera más sencilla de asegurar que esto no suceda es dividir los ejemplos en dos conjuntos: un **conjunto de entrenamiento** para crear la hipótesis y un **conjunto de prueba** para evaluarla.

Si se va a crear una sola hipótesis, este enfoque es suficiente. Pero con frecuencia se terminan creando múltiples hipótesis: se podría querer comparar dos modelos de aprendizaje automático completamente diferentes o ajustar los distintos "botones" dentro de un modelo. Por ejemplo, se podrían probar diferentes umbrales para la poda de árboles de decisión con o diferentes grados para polinomios. Estos "botones" se llaman **hiperparámetros**: parámetros de la clase de modelo, no del modelo individual.

Hiperparámetros

Suponga que un investigador genera una hipótesis para una configuración de hiperparámetro de poda con χ^2, mide las tasas de error en el conjunto de prueba y luego prueba diferentes hiperparámetros. Ninguna hipótesis individual ha echado un vistazo a los datos del conjunto de prueba, pero el proceso *general* sí, a través del investigador.

La manera de evitar esto es reservar el conjunto de prueba, guardarlo hasta que se haya completado todo el entrenamiento, la experimentación, el ajuste de hiperparámetros, el reentrenamiento, etc. Esto significa que se necesitan *tres* conjuntos de datos:

Conjunto de validación

1. Un **conjunto de entrenamiento** para entrenar modelos candidatos.
2. Un **conjunto de validación**, también conocido como **conjunto de desarrollo** o **dev set**, para evaluar los modelos candidatos y elegir el mejor.
3. Un **conjunto de prueba** para hacer una evaluación final imparcial del mejor modelo.

Validación cruzada *k*-fold

¿Qué pasa si no se tiene suficiente información para crear estos tres conjuntos de datos? Se puede aprovechar más los datos utilizando una técnica llamada **validación cruzada *k*-fold**. La idea es que cada ejemplo cumpla una doble función, como datos de entrenamiento y de validación, pero no al mismo tiempo. Primero, se dividen los datos en k subconjuntos iguales. Luego, se realizan k rondas de aprendizaje; en cada ronda se reserva $1/k$ de los datos como conjunto de validación y los ejemplos restantes se usan como conjunto de entrenamiento. El puntaje promedio del conjunto de prueba de las k rondas debería ser una mejor estimación que un solo puntaje. Los valores populares para k son 5 y 10, suficiente para dar una estimación que probablemente sea precisa desde el punto de vista estadístico, a un costo de 5 a 10 veces más tiempo de cálculo. El extremo es $k = n$, también conocido como **validación cruzada leave-one-out** o **LOOCV**. Incluso con la validación cruzada, aún se necesita un conjunto de prueba separado.

LOOCV

En la Figura 19.1 (página 654) se observó cómo una función lineal subajusta el conjunto de datos y un polinomio de alto grado sobreajusta los datos. Se puede pensar en la tarea de encontrar una buena hipótesis como dos subtareas: la **selección de modelo**[4] elige un buen espacio de hipótesis y la **optimización** (también llamada **entrenamiento**) encuentra la mejor hipótesis dentro de ese espacio.

Selección de modelo

Optimización

[4] Aunque el término "selección de modelo" es de uso común, un nombre mejor habría sido "selección de *clase* de modelo" o "selección de espacio de hipótesis". El término "modelo" se ha utilizado en la literatura para referirse a tres diferentes niveles de especificidad: un amplio espacio de hipótesis (como "polinomios"), un espacio de hipótesis con hiperparámetros definidos (como "polinomios de grado 2") y una hipótesis específica con todos los parámetros definidos (como $5x^2+3x-2$).

```
function MODEL-SELECTION(Learner,examples,k) returns un par (hipótesis, tasa de error)
    err ← una matriz, indexada por tamaño, que almacena las tasas de error del conjunto de validación
    training_set, test_set ← una partición de examples hacia dos conjuntos
    for size = 1 to ∞ do
        err[size] ← CROSS-VALIDATION(Learner,size,training_set,k)
        if err está empezando a aumentar significativamente then
            best_size ← el valor de size con mínimo err[size]
            h ← Learner(best_size,training_set)
            return h, ERROR-RATE(h,test_set)

function CROSS-VALIDATION(Learner,size,examples,k) returns tasa de error
    N ← el número of examples
    errs ← 0
    for i = 1 to k do
        validation_set ← examples[(i − 1) × N/k:i × N/k]
        training_set ← examples – validation_set
        h ← Learner(size,training_set)
        errs ← errs + ERROR-RATE(h,validation_set)
    return errs / k      // tasa de error promedio en conjuntos de validación, a través de la
                            validación cruzada de k-fold
```

Figura 19.8 Algoritmo para seleccionar el modelo que tiene el menor error de validación. Construye modelos de complejidad creciente y elige el que tiene la mejor tasa de error empírica, *err*, en el conjunto de datos de validación. *Learner*(*size*,*examples*) devuelve una hipótesis cuya complejidad está definida por el parámetro *size* y que está entrenada con *examples*. En CROSS-VALIDATION, cada iteración del bucle **for** selecciona una porción diferente de los *examples* como el conjunto de validación y mantiene los otros ejemplos como el conjunto de entrenamiento. Luego, devuelve el error promedio del conjunto de validación en todos los pliegues. Una vez que se ha determinado cuál valor del parámetro *size* es el mejor, MODEL-SELECTION devuelve el modelo (es decir, *learner*/hipótesis) de ese tamaño (*size*), entrenado con todos los ejemplos de entrenamiento, junto con su tasa de error en los ejemplos de prueba reservados.

Parte de la selección de modelos es cualitativa y subjetiva: se podrían seleccionar polinomios en lugar de árboles de decisión basados en algo que sabemos sobre el problema. Y parte es cuantitativa y empírica: dentro de la clase de polinomios, podríamos seleccionar *Grado* = 2, porque ese valor funciona mejor en el conjunto de datos de validación.

19.4.1 Model selection

La Figura 19.8 describe un algoritmo de MODEL-SELECTION simple. Toma como argumento un algoritmo de aprendizaje, *Learner* (por ejemplo, podría ser LEARN-DECISION-TREE). *Learner* toma un hiperparámetro, que se llama *size* en la figura. Para árboles de decisión podría ser el número de nodos en el árbol; para polinomios, el *size* sería *Grade*. MODEL-SELECTION comienza con el valor más pequeño de *size*, produciendo un modelo simple (que quizá subajustará los datos) e itera a través de valores mayores de *size*, considerando modelos más complejos. Al final, MODEL-SELECTION selecciona el modelo que tiene la tasa de error promedio más baja en los datos de validación reservados.

En la Figura 19.9 vemos dos patrones típicos que ocurren en la selección de modelos. En ambos (a) y (b) el error del conjunto de entrenamiento disminuye de manera monótona (con ligera

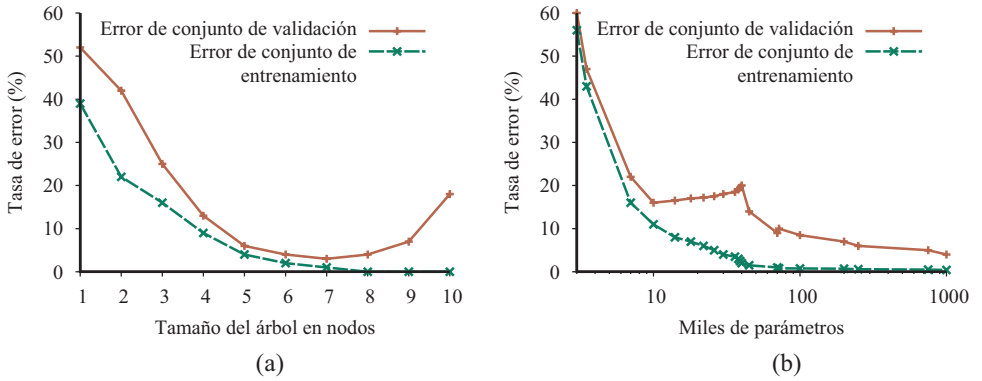


Figura 19.9 Tasas de error en datos de entrenamiento (línea inferior, verde) y datos de validación (línea superior, naranja) para modelos de diferente complejidad en dos problemas diferentes. MODEL-SELECTION elige el valor del hiperparámetro con el error de conjunto de validación más bajo. En (a) la clase de modelo son árboles de decisión y el hiperparámetro es el número de nodos. Los datos provienen de una versión del problema del restaurante. El tamaño óptimo es 7. En (b) la clase de modelo son redes neuronales convolucionales (ver Sección 21.3) y el hiperparámetro es el número de parámetros regulares en la red. Los datos son el conjunto de datos MNIST de imágenes de dígitos; la tarea es identificar cada dígito. El número óptimo de parámetros es 1 000 000 (note la escala logarítmica).

fluctuación aleatoria) a medida que aumenta la complejidad del modelo. La complejidad se mide por el número de nodos del árbol de decisión en (a) y por el número de parámetros de la red neuronal (w_i) en (b). Para muchas clases de modelos, el error del conjunto de entrenamiento llega a cero a medida que aumenta la complejidad.

Los dos casos difieren en el error del conjunto de validación. En (a) vemos una curva de error de validación en forma de U: el error disminuye durante un tiempo a medida que aumenta la complejidad del modelo, pero luego llegamos a un punto donde el modelo comienza a sobreajustar y el error de validación aumenta. MODEL-SELECTION elige el valor en el fondo de la curva de error de validación en forma de U: en este caso un árbol de tamaño 7. Este es el punto que mejor equilibra el subajuste y el sobreajuste. En (b) vemos una curva inicial en forma de U al igual que en (a) pero luego el error de validación comienza a disminuir; la tasa de error de validación más baja es el punto final en la trama, con 1 000 000 de parámetros.

¿Por qué algunas curvas de error de validación son como (a) y algunas como (b)? Se reduce a cómo las diferentes clases de modelos hacen uso de la capacidad excesiva, y qué tan bien eso coincide con el problema en cuestión. A medida que agregamos capacidad a una clase de modelo, con frecuencia llegamos al punto donde todos los ejemplos de entrenamiento pueden representarse de manera perfecta dentro del modelo. Por ejemplo, dado un conjunto de entrenamiento con n ejemplos distintos, siempre hay un árbol de decisión con n nodos hoja que permite representar todos los ejemplos.

Interpolado

Decimos que un modelo que se ajusta de manera exacta a todos los datos de entrenamiento ha **interpolado** los datos.[5] Las clases de modelos en la mayoría de los casos comienzan a sobreajustar a medida que la capacidad se acerca al punto de interpolación. Eso parece ser porque la mayor parte de la capacidad del modelo se concentra en los ejemplos de entrenamiento y la capacidad que queda se asigna de modo aleatorio de una manera que no es representativa de los patrones

[5] Algunos autores dicen que el modelo ha "memorizado" los datos.

en el conjunto de datos de validación. Algunas clases de modelos nunca se recuperan de este sobreajuste, como con los árboles de decisión en (a). Pero para otras clases de modelos, agregar capacidad significa que hay más funciones candidatas, y algunas de ellas son adecuadas para los patrones de datos que están en la verdadera función $f(x)$. Cuanto mayor sea la capacidad, más de estas representaciones adecuadas habrá y más probable será que el mecanismo de optimización pueda aterrizar en una.

Las redes neuronales profundas (Capítulo 21), las máquinas de kernel (Sección 19.7.5), los bosques aleatorios (Sección 19.8.2) y los conjuntos impulsados (Sección 19.8.4) tienen la propiedad de que su error de conjunto de validación tiende a disminuir a medida que aumenta la capacidad, como en la Figura 19.9(b).

Podríamos extender el algoritmo de selección de modelos de varias formas: podríamos comparar clases de modelos dispares, llamando a MODEL-SELECTION con DECISION-TREE-LEARNER como argumento y luego con POLYNOMIAL-LEARNER, y viendo cuál funciona mejor. Podríamos permitir múltiples hiperparámetros, lo que significa que necesitaríamos un algoritmo de optimización más complejo, como una búsqueda en retícula (ver Sección 19.9.3) en lugar de una búsqueda lineal.

19.4.2 De tasas de error a pérdida

Hasta ahora hemos tratando de minimizar la tasa de error. Esto es mejor que maximizar la tasa de error, pero no es toda la historia. Considere el problema de clasificar los mensajes de correo electrónico como spam o no spam. Es peor clasificar el no spam como spam (y así, potencialmente perder un mensaje importante) que clasificar el spam como no spam (y así, sufrir unos segundos de molestia). Por lo tanto, un clasificador con una tasa de error de 1 %, donde casi todos los errores eran clasificar spam como no spam, sería mejor que un clasificador con sólo una tasa de error de 0.5 %, si la mayoría de esos errores eran clasificar no spam como spam. En el Capítulo 16 vimos que los tomadores de decisiones deben maximizar la utilidad esperada, y la utilidad es lo que los aprendices también deben maximizar. Sin embargo, en el aprendizaje automático es tradicional expresar esto como un negativo: minimizar una **función de pérdida** en lugar de maximizar una función de utilidad. La función de pérdida $L(x,y,\hat{y})$ se define como la cantidad de utilidad perdida al predecir $h(x) = \hat{y}$ cuando la respuesta correcta es $f(x) = y$:

Función de pérdida

$$L(x,y,\hat{y}) = \textit{Utilidad}(\text{resultado de usar } y \text{ dada una entrada } x) - \textit{Utilidad}(\text{resultado de usar } \hat{y} \text{ dada una entrada } x)$$

Esta es la formulación más general de la función de pérdida. En la mayoría de los casos se usa una versión simplificada, $L(y,\hat{y})$, que es independiente de x. Usaremos la versión simplificada para el resto de este capítulo, lo que significa que no podemos decir que es peor clasificar de manera incorrecta una carta de Mamá, que clasificar incorrectamente una carta de nuestro primo molesto, pero podemos decir que es 10 veces peor clasificar el no spam como spam que viceversa:

$$L(\textit{spam},\textit{nospam}) = 1,\ L(\textit{nospam},\textit{spam}) = 10.$$

Note que $L(y,y)$ siempre es cero; por definición no hay pérdida cuando adivina bien. Para funciones con salidas discretas, podemos enumerar un valor de pérdida para cada posible mala clasificación, pero no podemos enumerar todas las posibilidades para datos de valor real. Si $f(x)$ es 137.035999, estaríamos bastante contentos con $h(x) = 137.036$, ¿pero qué tan contentos deberíamos estar? En general, los errores pequeños son mejores que los grandes; dos funciones que implementan esa idea son el valor absoluto de la diferencia (llamada la pérdida L_1), y el cuadrado de la diferencia (llamada la pérdida L_2, piense "2" al cuadrado). Para salidas de valor discreto, si estamos satisfechos con la idea de minimizar la tasa de error, podemos usar la función de pérdida $L_{0/1}$, que tiene una pérdida de 1 para una respuesta incorrecta:

Pérdida por valor absoluto:	$L_1(y,\hat{y}) = \lvert y - \hat{y} \rvert$
Pérdida por error cuadrático:	$L_2(y,\hat{y}) = (y - \hat{y})^2$
Pérdida 0/1:	$L_{0/1}(y,\hat{y}) = 0$ si $y = \hat{y}$, de lo contrario 1

Teóricamente, el agente de aprendizaje maximiza su utilidad esperada al elegir la hipótesis que minimiza la pérdida esperada sobre todos los pares de entrada-salida que verá. Para calcular esta expectativa, es necesario definir una distribución de probabilidad *a priori* $\mathbf{P}(X,Y)$ sobre los ejemplos. Sea $\mathcal{E}$ el conjunto de todos los ejemplos posibles de entrada-salida. Entonces, la **pérdida de generalización** esperada *(GenLoss)* para una hipótesis h (con respecto a la función de pérdida L) es

Pérdida de generalización

$$GenLoss_L(h) = \sum_{(x,y)\in\mathcal{E}} L(y,h(x))P(x,y),$$

y la mejor hipótesis, h^*, es la que tiene la mínima pérdida de generalización esperada:

$$h^* = \operatorname*{argmin}_{h\in\mathcal{H}} GenLoss_L(h).$$

Debido a que $P(x,y)$ no se conoce en la mayoría de los casos, el agente de aprendizaje sólo puede *estimar* la pérdida de generalización con la **pérdida empírica** en un conjunto de ejemplos E de tamaño N:

Pérdida empírica

$$EmpLoss_{L,E}(h) = \sum_{(x,y)\in E} L(y,h(x))\,\frac{1}{N}.$$

La mejor hipótesis estimada $\hat{h}^*$ es entonces la que tiene la mínima pérdida empírica:

$$\hat{h}^* = \operatorname*{argmin}_{h\in\mathcal{H}} EmpLoss_{L,E}(h).$$

Existen cuatro razones por las que $\hat{h}^*$ difiere de la función verdadera f: irrealizabilidad, varianza, ruido y complejidad computacional.

Realizable

Primero, decimos que un problema de aprendizaje es **realizable** si el espacio de hipótesis $\mathcal{H}$ contiene realmente la función verdadera f. Si $\mathcal{H}$ es el conjunto de funciones lineales, y la función verdadera f es una función cuadrática, entonces ninguna cantidad de datos recuperará la f verdadera. Segundo, la **varianza** significa que un algoritmo de aprendizaje generalmente devolverá diferentes hipótesis para diferentes conjuntos de ejemplos. Si el problema es realizable, entonces la varianza disminuye hacia cero a medida que aumenta el número de ejemplos de entrenamiento.

Ruidoso

Tercero, f puede ser no determinista o **ruidosa**; puede devolver valores diferentes de $f(x)$ para la misma x. Por definición, el ruido no se puede predecir (sólo se puede caracterizar). Y finalmente, cuando $\mathcal{H}$ es una función complicada en un espacio de hipótesis grande, puede ser **computacionalmente intractable** buscar sistemáticamente todas las posibilidades; en ese caso, una búsqueda puede explorar parte del espacio y devolver una hipótesis razonablemente buena, pero no siempre puede garantizar la mejor.

Aprendizaje a pequeña escala

Los métodos tradicionales en estadísticas y los primeros años del aprendizaje automático se centraron en el **aprendizaje a pequeña escala**, donde el número de ejemplos de entrenamiento varió desde decenas hasta unos pocos miles. Aquí, la pérdida de generalización proviene principalmente del error de aproximación de no tener la verdadera f en el espacio de hipótesis y del error de estimación de no tener suficientes ejemplos de entrenamiento para limitar la varianza.

Aprendizaje a gran escala

En los últimos años ha habido un mayor énfasis en el **aprendizaje a gran escala**, con millones de ejemplos. Aquí, la pérdida de generalización puede estar dominada por los límites de cálculo: hay suficientes datos y un modelo lo suficientemente rico que podría encontrar un h muy cercano a la f verdadera, pero el cálculo para encontrarlo es complejo, por lo que se conforma con una aproximación.

19.4.3 Regularización

En la Sección 19.4.1, vimos cómo realizar la selección de modelos con validación cruzada. Un enfoque alternativo es buscar una hipótesis que minimice directamente la suma ponderada de la pérdida empírica y la complejidad de la hipótesis, a la que llamaremos costo total:

$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$
$$\hat{h}^* = \operatorname*{argmin}_{h \in \mathcal{H}} Cost(h).$$

Aquí λ es un hiperparámetro, un número positivo que sirve como una tasa de conversión entre pérdida y complejidad de la hipótesis. Si λ se elige bien, equilibra adecuadamente la pérdida empírica de una función simple frente a la tendencia de una función complicada a sobreajustar.

Regularización

Este proceso de penalización explícita de hipótesis complejas se llama **regularización**: se busca funciones que sean más regulares. Tenga en cuenta que ahora se están haciendo dos elecciones: la función de pérdida (L_1 o L_2) y la medida de complejidad, que se llama **función de regularización**. La elección de la función de regularización depende del espacio de hipótesis. Por ejemplo, para los polinomios, una buena función de regularización es la suma de los cuadrados de los coeficientes: mantener la suma pequeña nos guiaría lejos del polinomio de grado 12 en la Figura 19.1. Se mostrará un ejemplo de este tipo de regularización en la Sección 19.6.3.

Función de regularización

Otra forma de simplificar los modelos es reducir las dimensiones con las que trabajan los modelos. Se puede realizar un proceso de **selección de características** para descartar atributos que parecen irrelevantes. La poda χ^2 es un tipo de selección de características.

Selección de características

Es posible tener la pérdida empírica y la complejidad medidas en la misma escala, sin el factor de conversión λ: ambas se pueden medir en bits. Primero se codifica la hipótesis como un programa de máquina de Turing y se cuenta el número de bits. Luego se cuenta el número de bits necesarios para codificar los datos, donde un ejemplo predicho correctamente cuesta cero bits y el costo de un ejemplo predicho incorrectamente depende de cuán grande sea el error. La **longitud mínima de descripción** o hipótesis MDL minimiza el número total de bits requeridos. Esto funciona bien en el límite, pero para problemas más pequeños, la elección de la codificación para el programa, cómo codificar mejor un árbol de decisión como una cadena de bits, afecta el resultado. En el Capítulo 20 (página 724), se describe una interpretación probabilística del enfoque MDL.

Longitud mínima de descripción

19.4.4 Ajuste de hiperparámetros

En la Sección 19.4.1 mostramos cómo seleccionar el mejor valor del hiperparámetro *size* aplicando validación cruzada a cada valor posible hasta que la tasa de error de validación aumenta. Ese es un buen enfoque cuando hay un solo hiperparámetro con un pequeño número de valores posibles. Pero cuando hay múltiples hiperparámetros o cuando tienen valores continuos, es más difícil elegir buenos valores.

El enfoque más simple para la afinación de hiperparámetros es la **afinación manual**: adivine algún parámetro de valores con base en la experiencia pasada, entrene un modelo, mida su rendimiento en los datos de validación, analice los resultados y use su intuición para sugerir nuevos valores de parámetros. Repita hasta que tenga un rendimiento satisfactorio (o se quede sin tiempo, presupuesto de computación o paciencia).

Afinación manual

Si sólo hay unos pocos hiperparámetros, cada uno con un pequeño número de valores posibles, entonces un enfoque más sistemático llamado **búsqueda en retícula** es apropiado: pruebe todas las combinaciones de valores y vea cuál se desempeña mejor en los datos de validación. Las diferentes combinaciones se ejecutan en paralelo en diferentes máquinas, por lo que si tiene suficientes recursos informáticos, esto no tiene que ser lento, aunque en algunos casos se ha

Búsqueda en retícula

sabido que la selección del modelo consume recursos en clusters de mil computadoras durante días a la vez.

Las estrategias de búsqueda de los Capítulos 3 y 4 también entran en juego. Por ejemplo, si dos hiperparámetros son independientes entre sí, permiten optimizarse por separado.

Búsqueda aleatoria

Si hay demasiadas combinaciones de valores posibles, entonces la **búsqueda aleatoria** toma muestras de manera uniforme del conjunto de todas las configuraciones de hiperparámetros posibles, repitiendo durante el tiempo que esté dispuesto a gastar el tiempo y los recursos computacionales. La toma de muestras aleatorias también es una buena manera de manejar valores continuos.

Optimización bayesiana

Cuando cada ejecución de entrenamiento lleva mucho tiempo, puede ser útil obtener información de cada una. La **optimización bayesiana** trata la tarea de elegir buenos valores de hiperparámetros como un problema de aprendizaje automático en sí mismo. Es decir, piense en el vector de valores de hiperparámetros $\mathbf{x}$ como una entrada y la pérdida total en el conjunto de validación para el modelo construido con esos hiperparámetros como una salida, y; entonces estamos tratando de encontrar la función $y = f(\mathbf{x})$ que minimiza la pérdida y. Cada vez que hacemos una ejecución de entrenamiento obtenemos un nuevo par $(y, f(\mathbf{x}))$, que podemos usar para actualizar nuestra creencia sobre la forma de la función f.

La idea es equilibrar la explotación (elección de valores de parámetros que están cerca de un buen resultado anterior) con la exploración (prueba de valores de parámetros novedosos). Este es el mismo equilibrio que vimos en la búsqueda de árboles de Monte Carlo (Sección 5.4); de hecho, la idea de los límites de confianza superior se utiliza aquí también para minimizar el arrepentimiento. Si hacemos la suposición de que f permite ser aproximada por un **proceso gaussiano**, entonces las matemáticas de actualizar nuestra creencia sobre f funcionan bien. Snoek *et al*. (2013) explican las matemáticas y dan una guía práctica para el enfoque, mostrando que puede superar la afinación manual de parámetros, incluso por expertos.

Entrenamiento basado en población (PBT)

Una alternativa a la optimización bayesiana es el **entrenamiento basado en población (PBT)**. PBT comienza utilizando búsqueda aleatoria para entrenar (en paralelo) una población de modelos, cada uno con diferentes valores de hiperparámetros. Luego se entrenan una segunda generación de modelos, pero eligen valores de hiperparámetros basados en los valores exitosos de la generación anterior, así como por mutación aleatoria, como en los algoritmos genéticos (Sección 4.1.4). Así, el entrenamiento basado en población comparte la ventaja de la búsqueda aleatoria de que muchas ejecuciones se pueden hacer en paralelo, además comparte la ventaja de la optimización bayesiana (o de la afinación manual por un humano inteligente) de que podemos obtener información de las ejecuciones anteriores para informar a las posteriores.

19.5 La teoría del aprendizaje

¿Cómo podemos estar seguros de que nuestra hipótesis aprendida predecirá bien para entradas previamente no vistas? Es decir, ¿cómo sabemos que la hipótesis h está cerca de la función objetivo f si no sabemos qué es f? Estas preguntas han sido reflexionadas durante siglos por Occam, Hume y otros. En las últimas décadas han surgido otras preguntas: ¿cuántos ejemplos necesitamos para obtener una buena h? ¿Qué espacio de hipótesis deberíamos usar? Si el espacio de hipótesis es muy complejo, ¿podemos incluso encontrar la mejor h o tenemos que conformarnos con un máximo local? ¿Qué tan compleja debería ser h? ¿Cómo evitamos el sobreajuste? Esta sección examina estas preguntas.

Comenzaremos con la pregunta de cuántos ejemplos se necesitan para aprender. Vimos, a partir de la curva de aprendizaje para el aprendizaje de árboles de decisión en el problema del restaurante (Figura 19.7 en la página 661) que la precisión mejora con más datos de entrenamiento. Las curvas de aprendizaje son útiles, pero son específicas para un algoritmo de aprendizaje particular en un problema particular. ¿Existen algunos principios más generales que rigen el número de ejemplos necesarios?

Teoría del aprendizaje computacional

Preguntas como esta son abordadas por la **teoría del aprendizaje computacional**, que se encuentra en la intersección de la IA, la estadística y la ciencia de la computación teórica. El principio subyacente es que cualquier hipótesis que esté equivocada casi con seguridad será "descubierta" con alta probabilidad después de un pequeño número de ejemplos, porque hará una predicción incorrecta. Por lo tanto, cualquier hipótesis que sea consistente con un conjunto lo suficientemente grande de ejemplos de entrenamiento es poco probable que esté equivocada: es decir, debe ser **probablemente aproximada y correcta (PAC)**.

Probablemente aproximada y correcta (PAC)

Aprendizaje PAC

Cualquier algoritmo de aprendizaje que devuelva hipótesis que son probablemente aproximadas y correctas se llama un algoritmo de **aprendizaje PAC**; podemos usar este enfoque para proporcionar límites en el rendimiento de varios algoritmos de aprendizaje.

Los teoremas de aprendizaje PAC, como todos los teoremas, son consecuencias lógicas de los axiomas. Cuando un teorema (a diferencia de, por ejemplo, un comentarista político) afirma algo sobre el futuro basado en el pasado, los axiomas tienen que proporcionar el "jugo" para hacer esa conexión. Para el aprendizaje PAC, el jugo es proporcionado por la suposición de estacionariedad introducida en la página 665, que dice que los futuros ejemplos se van a extraer de la misma distribución fija $\mathbf{P}(E) = \mathbf{P}(X,Y)$ como los ejemplos pasados. (Note que no tenemos que saber qué distribución es, sólo que no cambia). Además, para mantener las cosas simples, asumiremos que la verdadera función f es determinista y es miembro del espacio de hipótesis $\mathcal{H}$ que se está considerando.

Los teoremas PAC más simples tratan con funciones booleanas, para las cuales la pérdida 0/1 es apropiada. La **tasa de error** de una hipótesis h, definida de manera informal antes, aquí se define de manera formal como el error de generalización esperado para ejemplos extraídos de la distribución estacionaria:

$$\text{error}(h) = GenLoss_{L_{0/1}}(h) = \sum_{x,y} L_{0/1}(y, h(x)) P(x,y).$$

En otras palabras, error(h) es la probabilidad de que h clasifique de manera incorrecta un nuevo ejemplo. Esta es la misma cantidad que se mide de manera experimental en las curvas de aprendizaje antes mostradas.

Se llama **aproximadamente correcta** a una hipótesis h si $\text{error}(h) \leq \epsilon$, donde ϵ es una pequeña constante. Mostraremos que podemos encontrar un N tal que, después de entrenar con N ejemplos, con alta probabilidad, todas las hipótesis consistentes serán aproximadamente correctas. Se puede pensar en una hipótesis aproximadamente correcta como "cercana" a la verdadera función en el espacio de hipótesis: se encuentra dentro de lo que se llama la **bola** ϵ alrededor de la verdadera función f. El espacio de hipótesis fuera de la bola ϵ se llama $\mathcal{H}_{\text{bad}}$.

Bola ϵ

Podemos derivar un límite en la probabilidad de que una hipótesis "errónea" $h_b \in \mathcal{H}_{\text{bad}}$ sea consistente con los primeros N ejemplos de la siguiente manera. Sabemos que $\text{error}(h_b) > \epsilon$. Por lo tanto, la probabilidad de que esté de acuerdo con un ejemplo dado es como máximo $1 - \epsilon$. Dado que los ejemplos son independientes, el límite para N ejemplos es:

$$P(h_b \text{ está de acuerdo con } N \text{ ejemplos}) \leq (1 - \epsilon)^N.$$

La probabilidad de que $\mathcal{H}_{\text{bad}}$ contenga al menos una hipótesis consistente está limitada por la suma de las probabilidades individuales:

$$P(\mathcal{H}_{\text{bad}} \text{ contiene una hipótesis consistente}) \leq |\mathcal{H}_{\text{bad}}|(1 - \epsilon)^N \leq |\mathcal{H}|(1 - \epsilon)^N,$$

donde hemos utilizado el hecho de que $\mathcal{H}_{\text{bad}}$ es un subconjunto de $\mathcal{H}$ y, por lo tanto, $|\mathcal{H}_{\text{bad}}| \leq |\mathcal{H}|$. Nos gustaría reducir la probabilidad de este evento por debajo de algún número pequeño δ:

$$P(\mathcal{H}_{\text{bad}} \text{ contiene una hipótesis consistente}) \leq |\mathcal{H}|(1 - \epsilon)^N \leq \delta.$$

Dado que $1 - \epsilon \leq e^{-\epsilon}$, podemos lograr esto si permitimos que el algoritmo vea

$$N \geq \frac{1}{\epsilon}\left(\ln\frac{1}{\delta} + \ln|\mathcal{H}|\right) \tag{19.1}$$

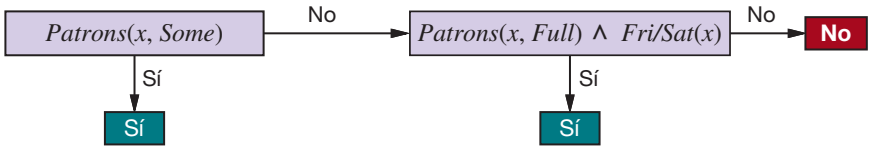


Figura 19.10 Lista de decisiones para el problema del restaurante.

ejemplos. Por lo tanto, con probabilidad al menos $1 - \delta$, después de ver tantos ejemplos, el algoritmo de aprendizaje devolverá una hipótesis que tiene un error de a lo sumo ϵ. En otras palabras, es, aproximadamente correcto. El número de ejemplos requeridos, como función de ϵ y δ, se llama la **complejidad de la muestra** del algoritmo de aprendizaje.

Complejidad de la muestra

Como vimos antes, si $\mathcal{H}$ es el conjunto de todas las funciones booleanas en n atributos, entonces $|\mathcal{H}| = 2^{2^n}$. Por lo tanto, la complejidad de la muestra del espacio crece como 2^n. Dado que el número de ejemplos posibles también es 2^n, esto sugiere que el aprendizaje PAC en la clase de todas las funciones booleanas requiere ver todos, o casi todos, los ejemplos posibles. Un momento de reflexión revela la razón de esto: $\mathcal{H}$ contiene suficientes hipótesis para clasificar cualquier conjunto dado de ejemplos de todas las formas posibles. En particular, para cualquier conjunto de N ejemplos, el conjunto de hipótesis consistentes con esos ejemplos contiene igual número de hipótesis que predicen que x_{N+1} será positivo e hipótesis que predicen que x_{N+1} será negativo.

Entonces, para obtener una verdadera generalización a ejemplos no vistos, parece que necesitamos restringir el espacio de hipótesis $\mathcal{H}$ de alguna manera; por supuesto, si restringimos el espacio, podríamos eliminar la verdadera función por completo. Hay tres formas de escapar de este dilema.

La primera es aportar conocimientos previos al problema.

La segunda, que introdujimos en la Sección 19.4.3, es insistir en que el algoritmo devuelva no sólo cualquier hipótesis consistente, sino de preferencia una simple (como se hace en el aprendizaje de árboles de decisión). En los casos en que encontrar hipótesis consistentes simples es tratable, los resultados de la complejidad de la muestra son, por lo regular, mejores que para los análisis basados sólo en la consistencia.

La tercera forma, que perseguimos a continuación, es centrarnos en los subconjuntos aprendibles de todo el espacio de hipótesis de las funciones booleanas. Este enfoque se basa en la suposición de que el espacio de hipótesis restringido contiene una hipótesis h que está lo suficientemente cerca de la función verdadera, f; los beneficios son que el espacio restringido de hipótesis permite una generalización efectiva y, por lo general, es más fácil de buscar. Ahora examinaremos uno de estos espacios de hipótesis restringidos a detalle.

19.5.1 Ejemplo de aprendizaje PAC: aprendiendo listas de decisiones

Ahora mostramos cómo aplicar el aprendizaje PAC a un nuevo espacio de hipótesis: **listas de decisiones**. Una lista de decisiones consta de una serie de pruebas, cada una de las cuales es una conjunción de literales. Si una prueba tiene éxito cuando se aplica a una descripción de ejemplo, la lista de decisiones especifica el valor a devolver. Si la prueba falla, el procesamiento continúa con la siguiente prueba en la lista. Las listas de decisiones se parecen a los árboles de decisiones, pero su estructura general es más simple: sólo se ramifican en una dirección. En contraste, las pruebas individuales son más complejas. La Figura 19.10 muestra una lista de decisiones que representa la siguiente hipótesis:

Listas de decisiones

$$WillWait \Leftrightarrow (Patrons = Some) \vee (Patrons = Full \wedge Fri/Sat).$$

function DECISION-LIST-LEARNING(*examples*) **returns** una lista de decisiones, o *failure*

  if los *examples* están vacíos, **then return** la lista de decisiones trivial *No*
  $t \leftarrow$ una prueba que coincide con un subconjunto no vacío de $examples_t$ de *examples* de tal manera que los miembros de $examples_t$ son todos positivos o todos negativos
  if no hay tal t **then return** *failure*
  if los ejemplos en $examples_t$ son positivos **then** $o \leftarrow$ *Yes* **else** $o \leftarrow$ *No*
  return una lista de decisiones con la prueba inicial t y el resultado o y las pruebas restantes dadas por DECISION-LIST-LEARNING(*examples* — $examples_t$)

Figura 19.11 Algoritmo para aprender listas de decisiones.

Si permitimos pruebas de tamaño arbitrario, entonces las listas de decisiones permiten representar cualquier función booleana (Ejercicio 19.DLEX). Por otro lado, si restringimos el tamaño de cada prueba a un máximo de k literales, entonces es posible que el algoritmo de aprendizaje generalice con éxito a partir de un pequeño número de ejemplos. Utilizamos la notación k-DL para una lista de decisiones con hasta k conjunciones. El ejemplo en la Figura 19.10 está en 2-DL. Es fácil demostrar (Ejercicio 19.DLEX) que k-DL incluye como un subconjunto k-**DT**, el conjunto de todos los árboles de decisión de profundidad a lo sumo k. Utilizaremos la notación k-DL(n) para denotar un k-DL usando n atributos booleanos. *k*-DT

La primera tarea es demostrar que k-DL es aprendible, es decir, que cualquier función en k-DL es aproximada con precisión después de entrenar con un número razonable de ejemplos. Para hacer esto, necesitamos calcular el número de hipótesis posibles. Dejemos que el conjunto de conjunciones de a lo sumo k literales usando n atributos sea $Conj(n,k)$. Debido a que una lista de decisiones se construye a partir de pruebas, y porque cada prueba está asociada a un resultado *Sí* o *No* o puede estar ausente de la lista de decisiones, hay a lo sumo $3^{|Conj(n,k)|}$ conjuntos distintos de pruebas componentes. Cada uno de estos conjuntos de pruebas puede estar en cualquier orden, por lo que

$$|k\text{-DL}(n)| \leq 3^c c! \text{ donde } c = |Conj(n,k)|.$$

El número de conjunciones de a lo sumo k literales de n atributos se da por

$$|Conj(n,k)| = \sum_{i=0}^{k} \binom{2n}{i} = O(n^k).$$

Por lo tanto, después de algún trabajo, obtenemos

$$|k\text{-DL}(n)| = 2^{O(n^k \log_2(n^k))}.$$

Podemos insertar esto en la Ecuación (19.1) para demostrar que el número de ejemplos necesarios para el aprendizaje PAC de una función k-DL(n) es polinomial en n:

$$N \geq \frac{1}{\epsilon}\left(1n\frac{1}{\delta} + O(n^k \log_2(n^k))\right).$$

Por lo tanto, cualquier algoritmo que devuelva una lista de decisiones consistente logrará un aprendizaje PAC de una función k-DL en un número razonable de ejemplos, para k pequeño.

La siguiente tarea es encontrar un algoritmo eficiente que devuelva una lista de decisiones consistente. Utilizaremos un algoritmo voraz llamado DECISION-LIST-LEARNING que encuentra una prueba que coincide de manera exacta con algún subconjunto del conjunto de entrenamiento. Una vez que encuentra tal prueba, la añade a la lista de decisiones en construcción y elimina los ejemplos

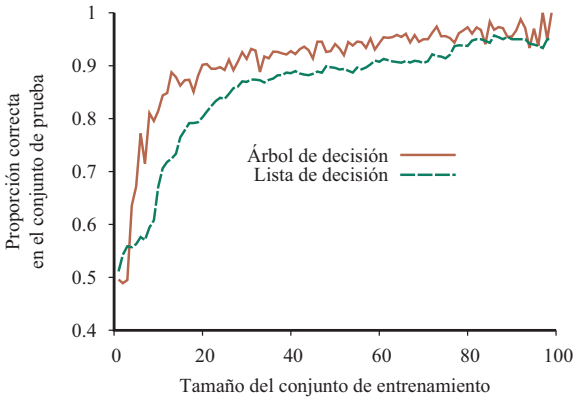


Figura 19.12 Curva de aprendizaje para el algoritmo DECISION-LIST-LEARNING en los datos del restaurante. La curva para LEARN-DECISION-TREE se muestra para comparación; los árboles de decisión funcionan un poco mejor en este problema en particular.

correspondientes. Luego construye el resto de la lista de decisiones utilizando sólo los ejemplos restantes. Esto se repite hasta que no quedan ejemplos. El algoritmo se muestra en la Figura 19.11.

Este algoritmo no especifica el método para seleccionar la próxima prueba a añadir a la lista de decisiones. Aunque los resultados formales dados antes no dependen del método de selección, parecería razonable preferir pruebas pequeñas que coincidan con grandes conjuntos de ejemplos clasificados de manera uniforme, de modo que la lista de decisiones en general será lo más compacta posible. La estrategia más simple es encontrar la prueba más pequeña t que coincida con cualquier subconjunto clasificado de manera uniforme, independiente del tamaño del subconjunto. Este enfoque funciona bastante bien, como sugiere la Figura 19.12. Para este problema, el árbol de decisiones aprende un poco más rápido que la lista de decisiones, pero tiene más variación. Ambos métodos son más de 90 % precisos después de 100 ensayos.

19.6 Regresión y clasificación lineal

Función lineal

Ahora es el momento de pasar de los árboles y listas de decisiones a un espacio de hipótesis diferente, uno que ha sido utilizado durante cientos de años: la clase de **funciones lineales** de entradas de valor continuo. Comenzaremos con el caso más simple: regresión con una función lineal univariada, también conocida como "ajuste de una línea recta". La Sección 19.6.3 cubre el caso multivariable. Las Secciones 19.6.4 y 19.6.5 muestran cómo convertir las funciones lineales en clasificadores aplicando umbrales duros y suaves.

19.6.1 Regresión lineal univariada

Peso

Una función lineal univariada (una línea recta) con entrada x y salida y tiene la forma $y = w_1x + w_0$, donde w_0 y w_1 son coeficientes de valor real a aprender. Usamos la letra w porque pensamos en los coeficientes como **pesos**; el valor de y cambia al cambiar el peso relativo de un término u otro. Definiremos $\mathbf{w}$ como el vector $\langle w_0, w_1 \rangle$, y definiremos la función lineal con esos pesos como

$$h_w(x) = w_1x + w_0.$$

La Figura 19.13(a) muestra un ejemplo de un conjunto de entrenamiento de n puntos en el plano x,y, cada punto representando el tamaño en pies cuadrados y el precio de una casa en venta. La tarea de encontrar el h_w que mejor se ajuste a estos datos se llama **regresión lineal**. Para ajustar una línea a los datos, todo lo que tenemos que hacer es encontrar los valores de los pesos $\langle w_0, w_1 \rangle$ que minimizan la pérdida empírica. Es tradicional (remontándose a Gauss[6]) utilizar la función de pérdida de error cuadrado, L_2, sumada sobre todos los ejemplos de entrenamiento:

Regresión lineal

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^{N} L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{j=1}^{N} (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2.$$

Nos gustaría encontrar $\mathbf{w}^* = \text{argmin}_{\mathbf{w}} Loss(h_{\mathbf{w}})$. La suma $\sum_{j=1}^{N}(y_j - (w_1 x_j + w_0))^2$ se minimiza cuando sus derivadas parciales con respecto a w_0 y w_1 son cero:

$$\frac{\partial}{\partial w_0} \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 = 0 \text{ y } \frac{\partial}{\partial w_1} \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 = 0. \tag{19.2}$$

Estas ecuaciones tienen una solución única:

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}; \quad w_0 = (\sum y_j - w_1(\sum x_j))/N. \tag{19.3}$$

Para el ejemplo en la Figura 19.13(a), la solución es $w_1 = 0.232$, $w_0 = 246$, y la línea con esos pesos se muestra como una línea discontinua en la figura.

Muchas formas de aprendizaje implican ajustar los pesos para minimizar una pérdida, por lo que es de ayuda tener una imagen mental de lo que está sucediendo en el **espacio de peso**; el espacio definido por todas las posibles configuraciones del espacio de pesos. Para la regresión lineal univariante, el espacio de peso definido por w_0 y w_1 es bidimensional, por lo que podemos graficar la pérdida como una función de w_0 y w_1 en un grafo 3D (ver Figura 19.13(b)). Vemos que la función de pérdida es **convexa**, como se define en la página 122; esto es cierto para *cada* problema de regresión lineal con una función de pérdida L_2, e implica que no hay mínimos locales. En cierto sentido, eso es el final de la historia para los modelos lineales; si necesitamos ajustar líneas a los datos, aplicamos la Ecuación (19.3).[7]

Espacio de peso

19.6.2 Descenso del gradiente

El modelo lineal univariante tiene la agradable propiedad de que resulta fácil encontrar una solución óptima donde las derivadas parciales son cero. Pero eso no siempre será el caso, por lo que introducimos aquí un método para minimizar la pérdida que no depende de resolver para encontrar ceros de las derivadas, y se puede aplicar a cualquier función de pérdida, sin importar cuán compleja sea.

Como se discutió en la Sección 4.2 (página 119) podemos buscar a través de un espacio de peso continúo modificando de forma incremental los parámetros. Allí llamamos al algoritmo **ascenso de colina**, pero aquí estamos minimizando la pérdida, no maximizando la ganancia, por lo que usaremos el término **descenso del gradiente**. Elegimos cualquier punto de partida en el espacio de pesos —aquí, un punto en el plano (w_0, w_1)— y luego calculamos una estimación del gradiente y nos movemos una pequeña cantidad en la dirección más empinada hacia abajo, repitiendo hasta que convergemos en un punto en el espacio de peso con pérdida mínima (local).

Descenso del gradiente

[6] Gauss demostró que si los valores de y_j tienen ruido con distribución normal, entonces los valores más probables de w_1 y w_0 se obtienen utilizando la pérdida L_2, minimizando la suma de las casillas de errores. (Si los valores tienen ruido que sigue una distribución de Laplace (doble exponencial), entonces la pérdida L_1 es apropiada).

[7] Con algunas advertencias: la función de pérdida L_2 es apropiada cuando hay ruido con distribución normal independiente de x, todos los resultados dependen de la suposición de estacionariedad, etc.

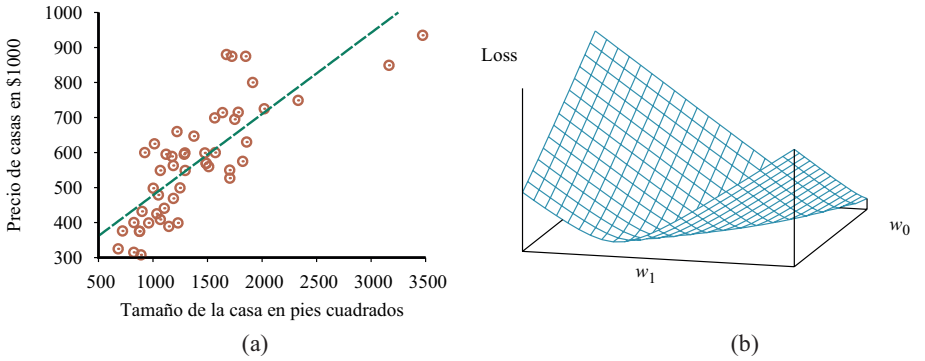


Figura 19.13 (a) Puntos de datos de precio contra espacio de piso de casas en venta en Berkeley, CA, en julio de 2009, junto con la hipótesis de función lineal que minimiza la pérdida de error cuadrado: $y = 0.232x + 246$. (b) Grafo de la función de pérdida $\sum_j (y_j - w_1 x_j + w_0)^2$ para varios valores de w_0, w_1. Note que la función de pérdida es convexa, con un único mínimo global.

El algoritmo es el siguiente:

w → cualquier punto en el espacio de parámetros
while not convergió **do**
  for each w_i **in w do**

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w}) \tag{19.4}$$

Tasa de aprendizaje

El parámetro α, que llamamos el **tamaño del paso** en la Sección 4.2, por lo general es llamado **tasa de aprendizaje** cuando tratamos de minimizar la pérdida en un problema de aprendizaje. Puede ser una constante fija o decaer con el tiempo a medida que avanza el proceso de aprendizaje.

Regla de la cadena

Para la regresión univariante la pérdida es cuadrática, por lo que la derivada parcial será lineal. (Lo único que necesita saber de cálculo es la **regla de la cadena**: $\partial g(f(x))/\partial x = g'(f(x))\,\partial f(x)/\partial x$, más los hechos de que $\frac{\partial}{\partial x} x^2 = 2x$ y $\frac{\partial}{\partial x} x = 1$). Primero trabajemos en las derivadas parciales, las pendientes, en el caso simplificado de mero ejemplo de entrenamiento, (x,y):

$$\begin{aligned} \frac{\partial}{\partial w_i} Loss(\mathbf{w}) &= \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(x))^2 = 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(x)) \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i}(y - (w_1 x + w_0)). \end{aligned} \tag{19.5}$$

Aplicando esto a ambos w_0 y w_1 obtenemos:

$$\frac{\partial}{\partial w_0} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)); \qquad \frac{\partial}{\partial w_1} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x.$$

Introduciendo esto en la Ecuación (19.4), y doblando el 2 en la tasa de aprendizaje α no especificada, obtenemos la siguiente regla de aprendizaje para los pesos:

$$w_0 \leftarrow w_0 + \alpha(y - h_{\mathbf{w}}(x)); \qquad w_1 \leftarrow w_1 + \alpha(y - h_{\mathbf{w}}(x)) \times x.$$

Estas actualizaciones tienen sentido intuitivo: si $h_{\mathbf{w}}(x) > y$ (es decir, la salida es demasiado grande), reduce un poco w_0, y reduce w_1 si x fue una entrada positiva pero aumenta w_1 si x fue una entrada negativa.

Las ecuaciones anteriores cubren un ejemplo de entrenamiento. Para N ejemplos de entrenamiento, queremos minimizar la suma de las pérdidas individuales por ejemplo. La derivada de una suma es la suma de las derivadas, así:

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)); \qquad w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)) \times x_j.$$

Descenso del gradiente por lotes

Estas actualizaciones constituyen la regla de aprendizaje del **descenso del gradiente por lotes** para la regresión lineal univariante (también llamado **descenso de gradiente determinista**). La superficie de pérdida es convexa, lo que significa que no hay mínimos locales en los que quedarse atascado y se garantiza la convergencia al mínimo global (siempre que no elijamos una α tan grande que se pase), aunque es muy lento: tenemos que sumar todos los ejemplos de entrenamiento N para cada paso, pero puede haber muchos pasos. El problema se agrava si N es mayor que el tamaño de la memoria del procesador. Al paso que cubre todos los ejemplos de entrenamiento se denomina **época**.

Época

Una variante más rápida se llama **descenso de gradiente estocástico o SGD**: este selecciona de manera aleatoria un pequeño número de ejemplos de entrenamiento en cada paso y actualiza de acuerdo con la Ecuación (19.5). La versión original de SGD seleccionó sólo un ejemplo de entrenamiento para cada paso, pero ahora es más común seleccionar un **minibatch** de m de los N ejemplos. Suponga que tenemos $N = 10\,000$ ejemplos y elegimos un minibatch de tamaño $m = 100$. Entonces, en cada paso hemos reducido la cantidad de cálculo en un factor de 100; pero como el error estándar del gradiente estimado es proporcional a la raíz cuadrada del número de ejemplos, el error estándar sólo aumenta en un factor de 10. Así que, incluso si tenemos que dar 10 veces más pasos antes de la convergencia, el SGD de minibatch sigue siendo 10 veces más rápido que el SGD de lote completo en este caso.

Descenso de gradiente estocástico o SGD

Minibatch

Con algunas arquitecturas de CPU o GPU, podemos elegir m para aprovechar las operaciones vectoriales paralelas, haciendo un paso con m ejemplos casi tan rápido como un paso con sólo un ejemplo. Dentro de estas restricciones, trataríamos m como un hiperparámetro que debería ajustarse para cada problema de aprendizaje.

La convergencia del SGD de minibatch no está garantizada, oscila alrededor del mínimo sin asentarse. Veremos en la página 684 cómo un pipeline de disminución de la tasa de aprendizaje, α, (como en el recocido simulado) sí garantiza la convergencia.

SGD es útil en un entorno en línea, donde los nuevos datos llegan uno a la vez, y la suposición de estacionariedad puede no cumplirse. (De hecho, SGD también se conoce como **descenso de gradiente online**). Con una buena elección para α un modelo evolucionará de manera lenta, recordando algo de lo que aprendió en el pasado, pero también adaptándose a los cambios representados por los nuevos datos.

Descenso de gradiente online

SGD se aplica ampliamente a modelos distintos de la regresión lineal, en particular a las redes neuronales. Aun cuando la superficie de pérdida no es convexa, el enfoque ha demostrado ser efectivo para encontrar buenos mínimos locales que están cerca del mínimo global.

19.6.3 Regresión lineal multivariable

Regresión lineal multivariable

Podemos extender a problemas de **regresión lineal multivariable** en los que cada ejemplo $\mathbf{x}_j$ es un vector de n elementos.[8] Nuestro espacio de hipótesis es el conjunto de funciones de la forma

$$h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1 x_{j,1} + \cdots + w_n x_{j,n} = w_0 + \sum_i w_i x_{j,i}.$$

[8] El lector puede consultar en el Apéndice A un resumen breve de álgebra lineal. Además, note que usamos el término "regresión multivariable" para significar que la entrada es un vector de múltiples valores, pero la salida es una sola variable. Usaremos este término de "regresión multivariante" para el caso en que la salida también es un vector de múltiples variables. Sin embargo, otros autores usan los dos términos de manera indistinta.

El término w_0, la intersección, se destaca como diferente de los demás. Podemos solucionar eso inventando un atributo de entrada ficticio, $x_{j,0}$, que se define como siempre igual a 1. Entonces h es el producto escalar de los pesos y el vector de entrada (o equivalente, el producto de la matriz del transpuesto de los pesos y el vector de entrada):

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_i x_{j,i}.$$

El mejor vector de pesos, $\mathbf{w}^*$, minimiza la pérdida de error cuadrado sobre los ejemplos:

$$\mathbf{w}^* = \underset{w}{\operatorname{argmin}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j).$$

La regresión lineal multivariable en realidad no es mucho más complicada que el caso univariante que acabamos de cubrir. El descenso del gradiente alcanzará el mínimo (único) de la función de pérdida; la ecuación de actualización para cada peso w_i es

$$w_i \leftarrow w_i + \alpha \sum_j (y_j - h_{\mathbf{w}}(\mathbf{x}_j)) \times x_{j,i}. \tag{19.6}$$

Con las herramientas del álgebra lineal y el cálculo vectorial, también es posible resolver de manera analítica para $\mathbf{w}$ que minimiza la pérdida. Deje que $\mathbf{y}$ sea el vector de salidas para los ejemplos de entrenamiento, y $\mathbf{X}$ sea la **matriz de datos**; es decir, la matriz de entradas con un ejemplo n-dimensional por fila. Entonces el vector de salidas predichas es $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ y la pérdida de error cuadrado sobre todos los datos de entrenamiento es

Matriz de datos

$$L(\mathbf{w}) = \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2.$$

Establecemos el gradiente a cero:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = 0.$$

Reorganizando, encontramos que el vector de peso de pérdida mínima está dado por

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \tag{19.7}$$

Llamamos a la expresión $(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ la **pseudoinversa** de la matriz de datos, mientras que la Ecuación (19.7) se llama **ecuación normal**.

Pseudoinversa

Ecuación normal

Con la regresión lineal univariante no tuvimos que preocuparnos por el sobreajuste. Pero con la regresión lineal multivariable en espacios de alta dimensión es posible que alguna dimensión, que en realidad es irrelevante, parezca útil por casualidad, lo que resulta en un sobreajuste.

Por lo tanto, es común usar la **regularización** en funciones lineales multivariables para evitar el sobreajuste. Recuerde que con la regularización minimizamos el costo total de una hipótesis, contando tanto la pérdida empírica como la complejidad de la hipótesis:

$$Costo(h) = EmpLoss(h) + \lambda Complexity(h).$$

Para las funciones lineales, la complejidad permite especificarse como una función de los pesos. Podemos considerar una familia de funciones de regularización:

$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q.$$

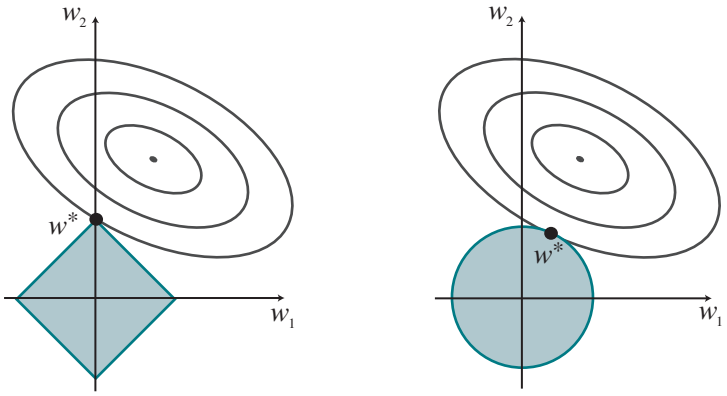


Figura 19.14 Por qué la regularización L_1 tiende a producir un modelo escaso. Izquierda: con la regularización L_1 (caja), la pérdida mínima alcanzable (contornos concéntricos) con frecuencia ocurre en un eje, lo que significa un peso de cero. Derecha: con la regularización L_2 (círculo), la pérdida mínima es probable que ocurra en cualquier lugar del círculo, sin dar preferencia a los pesos cero.

Al igual que con las funciones de pérdida, con $q = 1$ tenemos la regularización L_1,[9] que minimiza la suma de los valores absolutos; con $q = 2$, la regularización L_2 minimiza la suma de cuadrados. ¿Qué función de regularización debería elegir? Eso depende del problema específico, pero la regularización L_1 tiene una ventaja importante: tiende a producir un **modelo escaso**. Es decir, establece muchos pesos en cero, declarando que los atributos correspondientes son por completo irrelevantes, al igual que lo hace Learn-Decision-Tree (aunque por un mecanismo diferente). Las hipótesis que descartan atributos pueden ser más fáciles de entender para un humano y menos propensas a sobreajustar.

Modelo escaso

La Figura 19.14 da una explicación intuitiva de por qué la regularización L_1 lleva a pesos de cero, mientras que la regularización L_2 no. Observe que minimizar $Loss(\mathbf{w}) + \lambda Complexity(\mathbf{w})$ es equivalente a minimizar $Loss(\mathbf{w})$ sujeto a la restricción de que $Complexity(\mathbf{w}) \leq c$, para alguna constante c que está relacionada con λ. Ahora, en la Figura 19.14(a) la caja con forma de diamante representa el conjunto de puntos $\mathbf{w}$ en el espacio de peso bidimensional que tienen una complejidad L_1 menor que c; nuestra solución tendrá que estar en algún lugar dentro de esta caja. Los óvalos concéntricos representan contornos de la función de pérdida, con la pérdida mínima en el centro. Queremos encontrar el punto en la caja que esté más cerca del mínimo; puede ver en el diagrama que, para una posición arbitraria del mínimo y sus contornos, será común que la esquina de la caja encuentre su camino más cerca del mínimo, porque las esquinas son puntiagudas. Y, por supuesto, las esquinas son los puntos que tienen un valor de cero en alguna dimensión.

En la Figura 19.14(b) hemos hecho lo mismo para la medida de complejidad L_2, que representa un círculo en lugar de un diamante. Note que, en general, no hay razón para que la intersección aparezca en uno de los ejes; por lo tanto, la regularización L_2 no tiende a producir pesos cero. El resultado es que el número de ejemplos requeridos para encontrar una buena h es lineal en el número de características irrelevantes para la regularización L_2, pero sólo logarítmico con la regularización L_1. La evidencia empírica en muchos problemas respalda este análisis.

Otra forma de verlo es que la regularización L_1 toma en serio los ejes dimensionales, mientras que L_2 los trata como arbitrarios. La función L_2 es esférica, lo que la hace Invariante a la

[9] Quizá sea confuso que la notación L_1 y L_2 se utilice tanto para las funciones de pérdida como para las funciones de regularización. No es necesario que se usen en pares: se puede usar la pérdida L_2 con la regularización L_1, o viceversa.

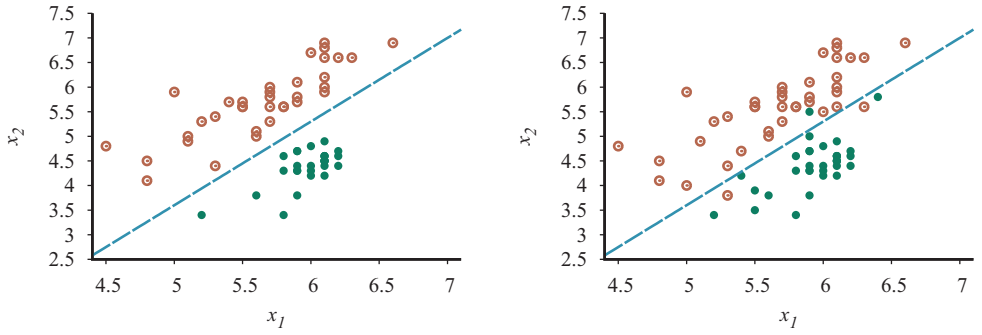


Figura 19.15 (a) Grafo de dos parámetros de datos sísmicos, magnitud de onda corporal x_1 y magnitud de onda superficial x_2, para terremotos (círculos naranjas abiertos) y explosiones nucleares (círculos verdes) ocurridos entre 1982 y 1990 en Asia y Medio Oriente (Kebeasy *et al.*, 1998). También se muestra un límite de decisión entre las clases. (b) El mismo dominio con más puntos de datos. Los terremotos y las explosiones ya no son linealmente separables.

rotación: imagine un conjunto de puntos en un plano, medidos por sus coordenadas x y y. Ahora imagine rotar los ejes 45°. Obtendría un conjunto diferente de valores (x',y') que representan los mismos puntos. Si aplica la regularización L_2 antes y después de rotar, obtiene el mismo punto como respuesta (aunque el punto se describiría con las nuevas coordenadas (x',y'). Eso es apropiado cuando la elección de los ejes es arbitraria, cuando no importa si sus dos dimensiones son distancias al norte y al este, o distancias al noreste y al sureste. Con la regularización L_1 obtendría una respuesta diferente, porque la función L_1 no es invariante a la rotación. Eso es apropiado cuando los ejes no son intercambiables; no tiene sentido rotar "número de baños" 45° hacia "tamaño del lote".

19.6.4 Clasificadores lineales con un umbral difícil

Las funciones lineales se usan para hacer clasificación así como regresión. Por ejemplo, la Figura 19.15(a) muestra puntos de datos de dos clases: terremotos (de interés para los sismólogos) y explosiones subterráneas (de interés para los expertos en control de armas). Cada punto está definido por dos valores de entrada, x_1 y x_2, que se refieren a las magnitudes de las ondas corporales y superficiales calculadas a partir de la señal sísmica. Dados estos datos de entrenamiento, la tarea de clasificación es aprender una hipótesis h que tomará nuevos puntos (x_1,x_2) y devolverá ya sea 0 para terremotos o 1 para explosiones.

Límite de decisión

Un **límite de decisión** es una línea (o una superficie, en dimensiones superiores) que separa las dos clases. En la Figura 19.15(a), el límite de decisión es una línea recta. Un límite de decisión lineal se llama **separador lineal** y los datos que admiten tal separador se llaman **linealmente separables**. El separador lineal en este caso está definido por

Separador lineal

Separabilidad lineal

$$x_2 = 1.7x_1 - 4.9 \quad \text{o} \quad -4.9 + 1.7x_1 - x_2 = 0.$$

Las explosiones, que queremos clasificar con valor 1, están debajo y a la derecha de esta línea; son puntos para los cuales $-4.9 + 1.7x_1 - x_2 > 0$, mientras que los terremotos tienen $-4.9 + 1.7x_1 - x_2 < 0$.

Podemos simplificar la ecuación convirtiéndola en la forma de producto escalar de vectores, con $x_0 = 1$ tenemos

$$-4.9x_0 + 1.7x_1 - x_2 = 0,$$

y podemos definir el vector de pesos,

$$\mathbf{w} = \langle -4.9, 1.7, -1 \rangle,$$

y escribir la hipótesis de clasificación

$h_{\mathbf{w}}(\mathbf{x}) = 1$ si $\mathbf{w} \cdot \mathbf{x} \geq 0$ y 0 en caso contrario.

De manera alternativa, podemos pensar en h como el resultado de pasar la función lineal $\mathbf{w} \cdot \mathbf{x}$ a través de una **función de umbral**:

Función de umbral

$h_{\mathbf{w}}(\mathbf{x}) = Umbral(\mathbf{w} \cdot \mathbf{x})$ donde $Umbral(z) = 1$ si $z \geq 0$ y 0 en caso contrario.

La función de umbral se muestra en la Figura 19.17(a).

Ahora que la hipótesis $h_{\mathbf{w}}(\mathbf{x})$ tiene una forma matemática bien definida, podemos pensar en elegir los pesos $\mathbf{w}$ para minimizar la pérdida. En las Secciones 19.6.1 y 19.6.3, hicimos esto tanto en forma cerrada (poniendo el gradiente a cero y resolviendo para los pesos) como por descenso de gradiente en el espacio de pesos. Aquí no podemos hacer ninguna de esas cosas porque el gradiente es cero casi en todas partes en el espacio de pesos, excepto en aquellos puntos donde $\mathbf{w} \cdot \mathbf{x} = 0$, y en esos puntos el gradiente está indefinido.

Sin embargo, existe una regla de actualización de pesos simple que converge a una solución; es decir, a un separador lineal que clasifica los datos de manera perfecta, siempre que los datos sean linealmente separables. Para un solo ejemplo $(\mathbf{x}, y)$, tenemos

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i \tag{19.8}$$

que es idéntico a la Ecuación (19.6), ¡la regla de actualización para la regresión lineal! Esta regla se llama **regla de aprendizaje del perceptrón**, por razones que se aclararán en el Capítulo 21. A pesar de que estamos considerando un problema de clasificación 0/1, el comportamiento es algo diferente. Tanto el valor verdadero y como la salida de la hipótesis $h_{\mathbf{w}}(\mathbf{x})$ son 0 o 1, por lo que hay tres posibilidades:

Regla de aprendizaje del perceptrón

- Si la salida es correcta (es decir, $y = h_{\mathbf{w}}(\mathbf{x})$) entonces los pesos no se cambian.
- Si y es 1 pero $h_{\mathbf{w}}(\mathbf{x})$ es 0, entonces w_i *incrementa* cuando la entrada correspondiente x_i es positiva y *decrece* cuando x_i es negativa. Esto tiene sentido, porque queremos hacer $\mathbf{w} \cdot \mathbf{x}$ más grande para que $h_{\mathbf{w}}(\mathbf{x})$ produzca un 1.
- Si y es 0 pero $h_{\mathbf{w}}(\mathbf{x})$ es 1, entonces w_i *decrece* cuando la entrada correspondiente x_i es positiva e *incrementa* cuando x_i es negativa. Esto tiene sentido, porque queremos hacer $\mathbf{w} \cdot \mathbf{x}$ más pequeño para que $h_{\mathbf{w}}(\mathbf{x})$ produzca un 0.

Con frecuencia, la regla de aprendizaje aplica un ejemplo a la vez, eligiendo ejemplos al azar (como en el descenso de gradiente estocástico). La Figura 19.16(a) muestra una **curva de entrenamiento** para esta regla de aprendizaje aplicada a los datos de terremoto/explosión mostrados en la Figura 19.15(a). Una curva de entrenamiento mide el rendimiento del clasificador en un conjunto de entrenamiento fijo a medida que el proceso de aprendizaje avanza una actualización a la vez en ese conjunto de entrenamiento. La curva muestra la regla de actualización convergiendo a un separador lineal de error cero. El proceso de "convergencia" no es propiamente agradable, pero siempre funciona. Esta ejecución en particular tarda 657 pasos en converger, para un conjunto de datos con 63 ejemplos, por lo que cada ejemplo se presenta aproximadamente 10 veces en promedio. Con frecuencia, la variación entre las ejecuciones es grande.

Curva de entrenamiento

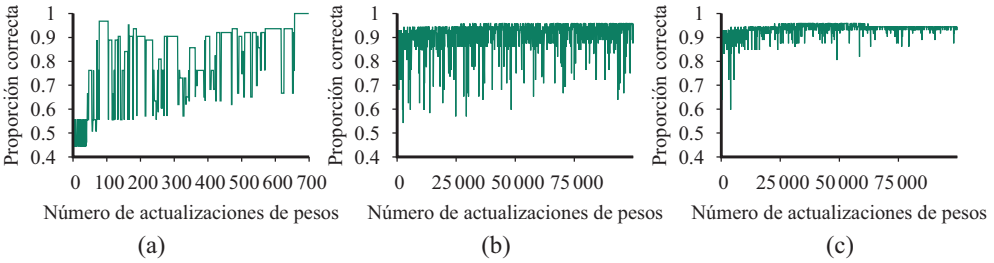


Figura 19.16 (a) Grafo de la precisión total del conjunto de entrenamiento contra el número de iteraciones a través del conjunto de entrenamiento para la regla de aprendizaje del perceptrón, dada la data de terremoto/explosión en la Figura 19.15(a). (b) Mismo grafo para los datos ruidosos, no separables en la Figura 19.15(b); note el cambio en la escala del eje *x*. (c) El mismo grafo que en (b), con un horario de tasa de aprendizaje $\alpha(t) = 1\,000/(1\,000 + t)$.

Hemos dicho que la regla de aprendizaje del perceptrón converge a un separador lineal perfecto cuando los puntos de datos son linealmente separables; pero ¿qué pasa si no lo son? Esta situación es común en el mundo real. Por ejemplo, la Figura 19.15(b) añade de nuevo los puntos de datos que Kebeasy *et al.* (1998) dejaron fuera cuando trazaron los datos mostrados en la Figura 19.15(a). En la Figura 19.16(b) mostramos la regla de aprendizaje del perceptrón fallando en converger incluso después de 10 000 pasos; aunque muchas veces alcanza la solución de error mínimo (tres errores), el algoritmo sigue cambiando los pesos. En general, la regla del perceptrón puede no converger a una solución estable para una tasa de aprendizaje α fija, pero si α decae como $O(1/t)$ donde t es el número de iteración, entonces se demuestra que la regla converge a una solución de error mínimo cuando los ejemplos son presentado en una secuencia aleatoria.[10] También es posible demostrar que encontrar la solución de error mínimo es NP-difícil, por lo que se espera que se requieran muchas presentaciones de los ejemplos para que se logre la convergencia. La Figura 19.16(c) muestra el proceso de entrenamiento con un horario de tasa de aprendizaje $\alpha(t) = 1\,000/(1\,000 + t)$: la convergencia no es perfecta después de 100 000 iteraciones, pero es mucho mejor que el caso de α fijo.

19.6.5 Clasificación lineal con regresión logística

Hemos visto que pasar la salida de una función lineal a través de la función de umbral crea un clasificador lineal; sin embargo, la naturaleza difícil del umbral causa algunos problemas: la hipótesis $h_{\mathbf{w}}(\mathbf{x})$ no es diferenciable y es, de hecho, una función discontinua de sus entradas y sus pesos. Esto hace que el aprendizaje con la regla del perceptrón sea una aventura impredecible. Además, el clasificador lineal siempre anuncia una predicción segura de 1 o 0, incluso para ejemplos que están muy cerca del límite; sería mejor si pudiera clasificar algunos como un claro 0 o 1, y otros como claros casos fronterizos.

Todos estos problemas se resuelven en gran medida suavizando la función de umbral, aproximando el umbral difícil con una función continua y diferenciable. En el Capítulo 13 (página 424), vimos dos funciones que parecen umbrales suaves: la integral de la distribución normal estándar

[10] Técnicamente, requerimos que $\sum_{t=1}^{\infty} \alpha(t) = \infty$ y $\sum_{t=1}^{\infty} \alpha^2(t) < \infty$. La tasa de aprendizaje $\alpha(t) = O(1/t)$ satisface estas condiciones. En general usamos $c/(c + t)$ para alguna constante c.

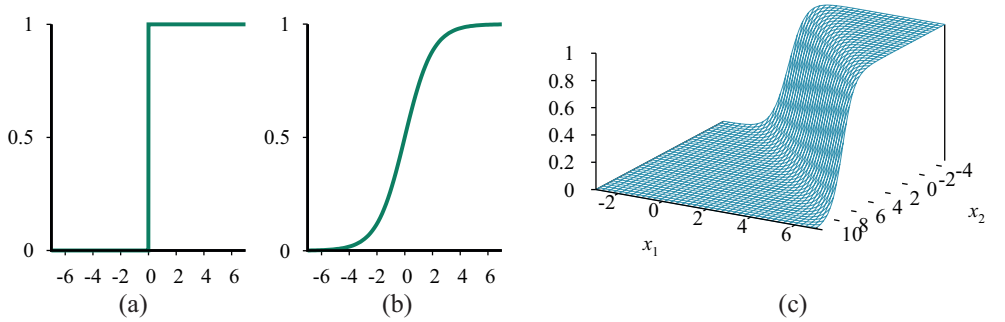


Figura 19.17 (a) Función de umbral difícil *Threshold*(z) con salida 0/1. Note que la función no es diferenciable en $z = 0$. (b) Función logística, $Logistic(z) = \frac{1}{1+e^{-z}}$, también conocida como función sigmoide. (c) Grafo de una hipótesis de regresión logística $h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w}\cdot\mathbf{x})$ para los datos mostrados en la Figura 19.15(b).

(utilizada para el modelo probit) y la función logística (utilizada para el modelo logit). Aunque las dos funciones son muy similares en forma, la función

$$Logistic(z) = \frac{1}{1+e^{-z}}$$

tiene propiedades matemáticas más convenientes. La función se muestra en la Figura 19.17(b). Con la función logística reemplazando la función de umbral, ahora tenemos

$$h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w}\cdot\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}\cdot\mathbf{x}}}.$$

Un ejemplo de tal hipótesis para el problema de terremoto/explosión de dos entradas se muestra en la Figura 19.17(c). Observe que la salida, siendo un número entre 0 y 1, puede interpretarse como una *probabilidad* de pertenecer a la clase etiquetada como 1. La hipótesis forma un límite suave en el espacio de entrada y da una probabilidad de 0.5 para cualquier entrada en el centro de la región de límite, y se acerca a 0 o 1 a medida que nos alejamos del límite.

El proceso de ajustar los pesos de este modelo para minimizar la pérdida en un conjunto de datos se llama **regresión logística**. No hay una solución cerrada fácil para encontrar el valor óptimo de $\mathbf{w}$ con este modelo, pero el cálculo del descenso del gradiente es sencillo. Como nuestras hipótesis ya no sólo dan salida a 0 o 1, utilizaremos la función de pérdida L_2; además, para mantener las fórmulas legibles, usaremos g para representar la función logística, con g' su derivada.

Regresión logística

Para un solo ejemplo $(\mathbf{x},y)$, la derivación del gradiente es la misma que para la regresión lineal (Ecuación (19.5)) hasta el punto donde se inserta la forma real de h. (Para esta derivación, de nuevo necesitamos la regla de la cadena). Tenemos

$$\begin{aligned}
\frac{\partial}{\partial w_i} Loss(\mathbf{w}) &= \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(\mathbf{x}))^2 \\
&= 2(y - h_{\mathbf{w}}(\mathbf{x})) \times \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(\mathbf{x})) \\
&= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times g'(\mathbf{w}\cdot\mathbf{x}) \times \frac{\partial}{\partial w_i}\mathbf{w}\cdot\mathbf{x} \\
&= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times g'(\mathbf{w}\cdot\mathbf{x}) \times x_i.
\end{aligned}$$

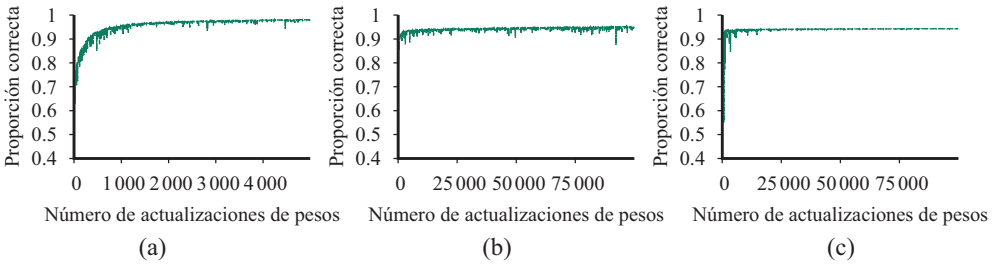


Figura 19.18 Repetición de los experimentos en la Figura 19.16 utilizando regresión logística. La trama en (a) cubre 5 000 iteraciones en lugar de 700, mientras que las tramas en (b) y (c) usan la misma escala de antes.

La derivada g' de la función logística satisface $g'(z) = g(z)(1 - g(z))$, así que tenemos

$$g'(\mathbf{w}\cdot\mathbf{x}) = g(\mathbf{w}\cdot\mathbf{x})(1 - g(\mathbf{w}\cdot\mathbf{x})) = h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x}))$$

por lo que la actualización del peso para minimizar la pérdida da un paso en la dirección de la diferencia entre la entrada y la predicción, $(y\ h_{\mathbf{w}}(\mathbf{x}))$, y la longitud de ese paso depende de la constante α y g':

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x}) \times h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i. \tag{19.9}$$

Repitiendo los experimentos de la Figura 19.16 con regresión logística en lugar del clasificador de umbral lineal, obtenemos los resultados mostrados en la Figura 19.18. En (a), el caso linealmente separable, la regresión logística es algo más lenta para converger, pero se comporta de manera mucho más predecible. En (b) y (c), donde los datos son ruidosos y no separables, la regresión logística converge mucho más rápido y de manera confiable. Estas ventajas tienden a trasladarse a aplicaciones del mundo real, y la regresión logística se ha convertido en una de las técnicas de clasificación más populares para problemas en medicina, marketing, análisis de encuestas, calificación crediticia, salud pública y otras aplicaciones.

19.7 Modelos no paramétricos

La regresión lineal utiliza los datos de entrenamiento para estimar un conjunto fijo de parámetros **w;** eso define nuestra hipótesis $h_{\mathbf{w}}(\mathbf{x})$. En ese punto es posible desechar los datos de entrenamiento porque todos están resumidos por **w**. Un modelo de aprendizaje que resume los datos con un conjunto de parámetros de tamaño fijo (independiente del número de ejemplos de entrenamiento) se llama **modelo paramétrico**.

Modelo paramétrico

Cuando los conjuntos de datos son pequeños, tiene sentido tener una fuerte restricción en las hipótesis permitidas para evitar el sobreajuste. Pero cuando hay millones o miles de millones de ejemplos de los que aprender, parece una mejor idea dejar que los datos hablen por sí mismos en lugar de obligarlos a hablar a través de un pequeño vector de parámetros. Si los datos dicen que la respuesta correcta es una función muy ondulada, no deberíamos restringirnos a funciones lineales o ligeramente onduladas.

Modelo no paramétrico

Un **modelo no paramétrico** es aquel que no permite ser caracterizado por un conjunto acotado de parámetros. Por ejemplo, la función lineal a trozos de la Figura 19.1 retiene todos los puntos de datos como parte del modelo. Los métodos de aprendizaje que hacen esto también han sido descritos como **aprendizaje basado en instancias** o **aprendizaje basado en memoria**. El

Aprendizaje basado en instancias

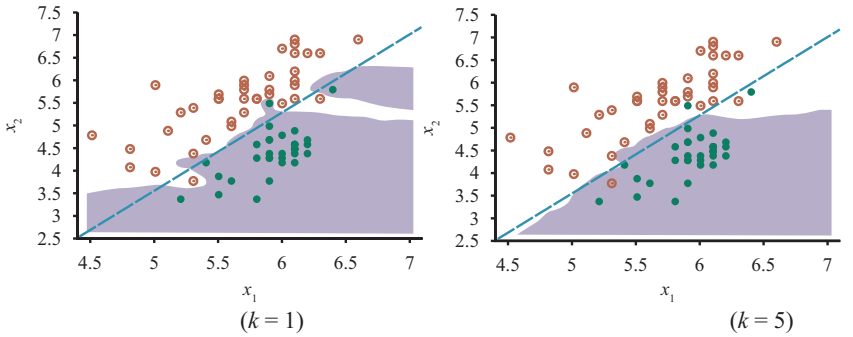


Figura 19.19 (a) Modelo de k-vecinos más cercanos mostrando la extensión de la clase de explosión para los datos en la Figura 19.15, con $k = 1$. El sobreajuste es aparente. (b) Con $k = 5$, el problema de sobreajuste desaparece para este conjunto de datos.

método de aprendizaje basado en instancias más simple es la **tabla de búsqueda**: tome todos los ejemplos de entrenamiento, póngalos en una tabla de búsqueda y luego, cuando le pidan $h(\mathbf{x})$, vea si $\mathbf{x}$ está en la tabla; si lo está, devuelva la y correspondiente. Tabla de búsqueda

El problema con este método es que no generaliza bien: cuando $\mathbf{x}$ no está en la tabla no tenemos información sobre un valor plausible.

19.7.1 Modelos de vecinos más cercanos

Podemos mejorar la tabla de búsqueda con una pequeña variación: dada una consulta $\mathbf{x}_q$, en lugar de encontrar un ejemplo que sea igual a $\mathbf{x}_q$, encontramos los k ejemplos que están *más cerca* de $\mathbf{x}_q$. Esto se llama búsqueda k-**vecinos más cercanos**. Usaremos la notación $NN(k,\mathbf{x}_q)$ para denotar el conjunto de k vecinos más cercanos a $\mathbf{x}_q$. Vecinos más cercanos

Para hacer clasificación, encuentre el conjunto de vecinos $NN(k,\mathbf{x}_q)$ y tome el valor de salida más común; por ejemplo, si $k = 3$ y los valores de salida son $\langle Sí, No, Sí\rangle$, entonces la clasificación será *Sí*. Para evitar empates en la clasificación binaria, k suele elegirse como un número impar.

Para hacer regresión, podemos tomar la media o la mediana de los k vecinos, o podemos resolver un problema de regresión lineal en los vecinos. La función lineal a trozos de la Figura 19.1 resuelve un problema de regresión lineal (trivial) con los dos puntos de datos a la derecha y a la izquierda de $\mathbf{x}_q$. (Cuando los puntos de datos x_i están igualmente espaciados, estos serán los dos vecinos más cercanos).

En la Figura 19.19, mostramos el límite de decisión de la clasificación de k-vecinos más cercanos para $k = 1$ y 5 en el conjunto de datos de terremotos de la Figura 19.15. Los métodos no paramétricos todavía están sujetos a subajuste y sobreajuste, al igual que los métodos paramétricos. En este caso, los vecinos más cercanos 1 están sobreajustados; reacciona demasiado al negro aislado en la parte superior derecha y al blanco aislado en (5.4, 3.7). El límite de decisión de los 5 vecinos más cercanos es bueno; un k más alto subajusta. Como de costumbre, la validación cruzada puede usarse para seleccionar el mejor valor de k.

La misma frase "más cercano" implica una métrica de distancia. ¿Cómo medimos la distancia desde un punto de consulta $\mathbf{x}_q$ hasta un punto de ejemplo $\mathbf{x}_j$? En general, las distancias se miden con una **distancia de Minkowski** o norma L^p, definida como Distancia de Minkowski

$$L^p(\mathbf{x}_j,\mathbf{x}_q) = (\sum_i |x_{j,i} - x_{q,i}|^p)^{1/p}.$$

Con $p = 2$ esto es la distancia euclidiana y con $p = 1$ es la distancia de Manhattan. Con valores de atributos booleanos, el número de atributos en los que los dos puntos difieren se llama **distancia de Hamming**. Con frecuencia se utiliza la distancia euclidiana si las dimensiones están midiendo propiedades similares, como el ancho, la altura y la profundidad de las partes, mientras que la distancia de Manhattan se utiliza si son disímiles, como la edad, el peso y el género de un paciente. Tenga en cuenta que si usamos los números brutos de cada dimensión, entonces la distancia total se verá afectada por un cambio en las unidades en cualquier dimensión. Es decir, si cambiamos la dimensión de la *altura* de metros a millas mientras mantenemos las mismas dimensiones de *ancho* y *profundidad*, obtendremos diferentes vecinos más cercanos. ¿Y cómo comparamos una diferencia de edad con una diferencia de peso? Un enfoque común es aplicar la **normalización** a las mediciones en cada dimensión. Podemos calcular la media μ_i y la desviación estándar σ_i de los valores en cada dimensión, y reescalarlos para que $x_{j,i}$ se convierta en $(x_{j,i} - \mu_i)/\sigma_i$. Una métrica más compleja conocida como la **distancia de Mahalanobis** tiene en cuenta la covarianza entre dimensiones.

Distancia de Hamming

Normalización

Distancia de Mahalanobis

En espacios de baja dimensión con abundancia de datos, los vecinos más cercanos funcionan muy bien: es probable que tengamos suficientes puntos de datos cercanos para obtener una buena respuesta. Pero a medida que aumenta el número de dimensiones nos encontramos con un problema: ¡los vecinos más cercanos en espacios de alta dimensión con frecuencia no están muy cerca! Considere k-vecinos más cercanos en un conjunto de datos de N puntos distribuidos de manera uniforme en el interior de un hipercubo unitario n-dimensional. Definiremos el vecindario k de un punto como el hipercubo más pequeño que contiene los k vecinos más cercanos. Dejemos que ℓ sea la longitud media del lado de un vecindario. Entonces el volumen del vecindario (que contiene k puntos) es ℓ^n y el volumen del cubo completo (que contiene N puntos) es 1. Por lo tanto, en promedio, $\ell^n = k/N$. Tomando raíces n-ésimas de ambos lados obtenemos $\ell = (k/N)^{1/n}$.

Para ser concretos, dejemos $k = 10$ y $N = 1\,000\,000$. En dos dimensiones ($n = 2$; una casilla unitaria), el vecindario promedio tiene $\ell = 0.003$, una pequeña fracción de la casilla unitaria, y en 3 dimensiones ℓ es sólo 2 % de la longitud del borde del cubo unitario. Pero cuando llegamos a 17 dimensiones, ℓ es la mitad de la longitud del borde del hipercubo unitario, y en 200 dimensiones es 94 %. Este problema se ha llamado **maldición de la dimensionalidad**.

Maldición de la dimensionalidad

Otra forma de verlo: considere los puntos que caen dentro de una cáscara delgada que compone a 1 % exterior del hipercubo unitario. Estos son valores atípicos; en general, será difícil encontrar un buen valor para ellos porque estaremos extrapolando en lugar de interpolar. En una dimensión, estos valores atípicos son sólo de 2% de los puntos en la línea unitaria (aquellos puntos donde $x < .01$ o $x > .99$), pero en 200 dimensiones, más del 98 % de los puntos caen dentro de esta cáscara delgada, casi todos los puntos son valores atípicos. Puede ver un ejemplo de un mal ajuste de vecinos más cercanos en valores atípicos si mira hacia adelante a la Figura 19.20(b).

La función $NN(k, \mathbf{x}_q)$ es trivial: dado un conjunto de N ejemplos y una consulta $\mathbf{x}_q$, itera a través de los ejemplos, mide la distancia a $\mathbf{x}_q$ de cada uno, y guarda los mejores k. Si estamos satisfechos con una implementación que toma $O(N)$ tiempo de ejecución, entonces eso es el final de la historia. Pero los métodos basados en instancias están diseñados para grandes conjuntos de datos, por lo que nos gustaría algo más rápido. Las dos próximas subsecciones muestran cómo los árboles y las tablas hash se utilizan para acelerar el cálculo.

19.7.2 Encontrar vecinos más cercanos con árboles k-d

Un árbol binario equilibrado sobre datos con un número arbitrario de dimensiones se llama **árbol k–d**, para árbol k-dimensional. La construcción de un árbol k-d es similar a la construcción de un árbol binario equilibrado. Comenzamos con un conjunto de ejemplos y en el nodo raíz los dividimos a lo largo de la dimensión i al probar si $x_i \leq m$, donde m es la mediana de los ejemplos a lo

Árbol K-d

largo de la dimensión *i-esima*; así, una mitad de los ejemplos estarán en la rama izquierda del árbol y la otra mitad en la derecha. Luego hacemos de manera recursiva un árbol para los conjuntos de ejemplos de la izquierda y la derecha, parando cuando quedan menos de dos ejemplos. Para elegir una dimensión en la que dividir en cada nodo del árbol, uno selecciona la dimensión i mod n en el nivel i del árbol. (Note que podemos necesitar dividir en cualquier dimensión dada varias veces a medida que avanzamos por el árbol). Otra estrategia es dividir en la dimensión que tiene la mayor dispersión de valores.

La búsqueda exacta desde un árbol k-d es igual que la búsqueda desde un árbol binario (con la ligera complicación de que necesita prestar atención a qué dimensión está probando en cada nodo). Pero la búsqueda del vecino más cercano es más complicada. A medida que bajamos por las ramas, dividiendo los ejemplos a la mitad, en algunos casos podemos ignorar la mitad de los ejemplos. Pero no siempre. A veces el punto que estamos consultando cae muy cerca del límite divisorio. El punto de consulta en sí mismo podría estar en el lado izquierdo del límite, pero uno o más de los k vecinos más cercanos podrían estar en realidad en el lado derecho.

Tenemos que probar esta posibilidad calculando la distancia del punto de consulta al límite divisorio, y luego buscar en ambos lados si no podemos encontrar k ejemplos en la izquierda que estén más cerca que esta distancia. Debido a este problema, los árboles k-d son apropiados sólo cuando hay muchos más ejemplos que dimensiones, de preferencia en al menos segundos ejemplos. Por lo tanto, los árboles k-d son una buena elección para hasta unas 10 dimensiones cuando hay miles de ejemplos o hasta 20 dimensiones con millones de ejemplos.

19.7.3 Hash sensible a la localidad

Las tablas hash tienen el potencial de proporcionar una búsqueda aún más rápida que los árboles binarios. Pero ¿cómo podemos encontrar vecinos más cercanos usando una tabla hash, cuando los códigos hash se basan en una coincidencia *exacta*? Los códigos hash distribuyen de manera aleatoria los valores entre los bins, pero queremos que los puntos cercanos se agrupen juntos en el mismo bin (bloque); queremos un **hash sensible a la localidad** (**LSH**).

Hash sensible a la localidad

No podemos usar hashes para resolver $NN(k,\mathbf{x}_q)$ de manera exacta, pero con un uso inteligente de algoritmos aleatorizados, podemos encontrar una solución *aproximada*. Primero definimos el problema de los **vecinos cercanos aproximados**: dado un conjunto de datos de puntos de ejemplo y un punto de consulta $\mathbf{x}_q$, encontrar un punto de ejemplo (o puntos) con alta probabilidad que esté cerca de $\mathbf{x}_q$. Para ser más precisos, requerimos que si hay un punto $\mathbf{x}_j$ que esté dentro de un radio r de $\mathbf{x}_q$, entonces, con alta probabilidad el algoritmo encontrará un punto $\mathbf{x}_{j'}$ que está dentro de la distancia cr de $\mathbf{x}_q$. Si no hay ningún punto dentro del radio r, entonces se permite que el algoritmo informe de un fracaso. Los valores de c y "alta probabilidad" son hiperparámetros del algoritmo.

Vecinos cercanos aproximados

Para resolver vecinos cercanos aproximados, necesitaremos una función hash $g(\mathbf{x})$ que tenga la propiedad de que, para cualquier par de puntos $\mathbf{x}_j$ y $\mathbf{x}_{j'}$, la probabilidad de que tengan el mismo código hash es pequeña si su distancia es mayor que cr, y es alta si su distancia es menor que r. Por simplicidad, trataremos cada punto como una cadena de bits. (Cualquier característica que no sea booleana se codifica en un conjunto de características booleanas).

Nos basamos en la intuición de que si dos puntos están cerca en un espacio n-dimensional, necesariamente estarán cerca cuando se proyecten en un espacio unidimensional (una línea). De hecho, podemos discretizar la línea en compartimentos, cubos hash, de modo que con alta probabilidad los puntos cercanos se proyecten en el mismo cubo. Los puntos que están lejos entre sí tenderán a proyectarse en diferentes cubos, pero siempre habrá algunas proyecciones que proyecten puntos lejanos en el mismo cubo. Por lo tanto, el cubo para el punto $\mathbf{x}_q$ contiene muchos (pero no todos) puntos que están cerca de $\mathbf{x}_q$, y podría contener algunos puntos que están lejos.

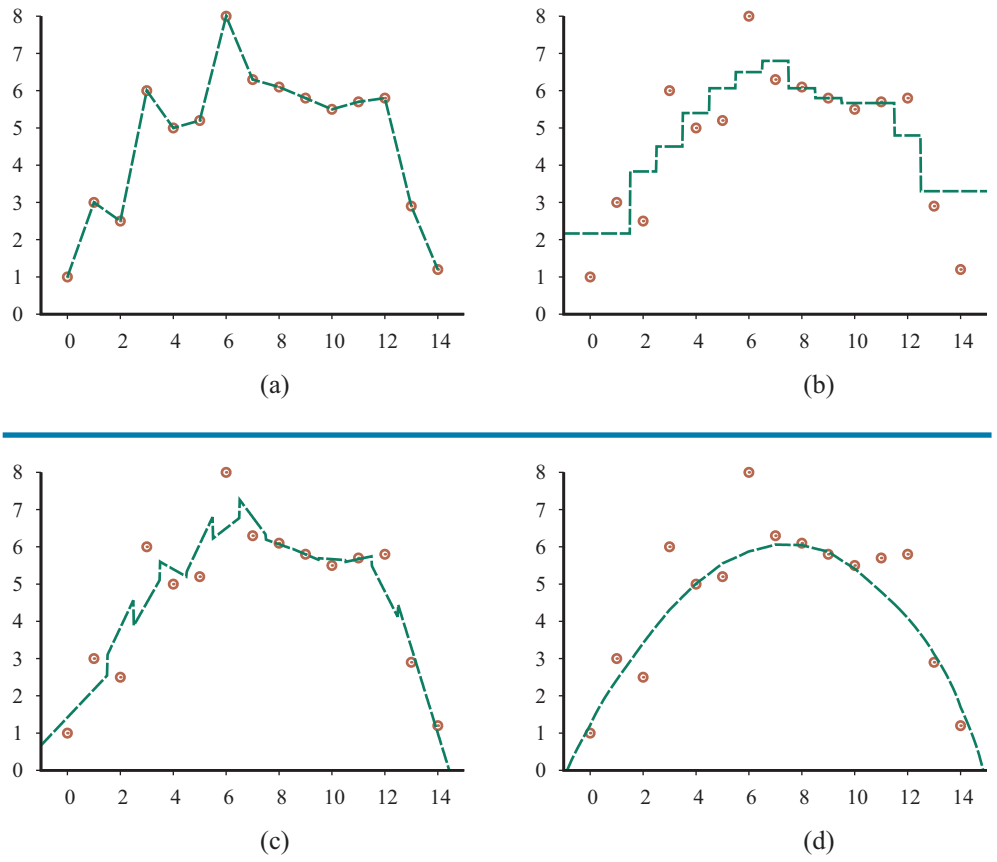


Figura 19.20 Modelos de regresión no paramétricos: (a) conectar los puntos, (b) promedio de 3 vecinos más cercanos, (c) regresión lineal de 3 vecinos más cercanos, (d) regresión ponderada de manera local con un kernel cuadrático de ancho 10.

El truco de LSH es crear *múltiples* proyecciones aleatorias y combinarlas. Una proyección aleatoria es sólo un subconjunto aleatorio de la representación de cadena de bits. Elegimos ℓ proyecciones aleatorias diferentes y creamos ℓ tablas hash, $g_1(\mathbf{x}),..., g_\ell(\mathbf{x})$. Luego ingresamos todos los ejemplos en cada tabla hash. Después, cuando se nos da un punto de consulta $\mathbf{x}_q$, obtenemos el conjunto de puntos en el cubo $g_i(\mathbf{x}_q)$ de cada tabla hash, y unimos estos ℓ conjuntos en un conjunto de puntos candidatos, C. Luego calculamos la distancia real a $\mathbf{x}_q$ para cada uno de los puntos en C y devolvemos los k puntos más cercanos. Con alta probabilidad, cada uno de los puntos que están cerca de $\mathbf{x}_q$ aparecerá en al menos uno de los cubos, y aunque algunos puntos lejanos también aparecerán, podemos ignorarlos. Con problemas reales grandes, como encontrar los vecinos cercanos en un conjunto de datos de 13 millones de imágenes web utilizando 512 dimensiones (Torralba *et al*., 2008), el hashing sensible a la localidad necesita examinar sólo unas pocas miles de imágenes de 13 millones para encontrar vecinos más cercanos, un aceleración mil veces mayor que los enfoques exhaustivos o de árbol k-d.

19.7.4 Regresión no paramétrica

Ahora veremos enfoques no paramétricos para la *regresión* en lugar de la clasificación. La Figura 19.20 muestra un ejemplo de algunos modelos diferentes. En (a), tenemos quizás el método más simple de todos, conocido como "conectar los puntos", y con desdén como "regresión no paramétrica lineal a trozos". Este modelo crea una función $h(x)$ que, cuando se le da una consulta x_q, considera los ejemplos de entrenamiento a la izquierda y a la derecha de x_q e interpola entre ellos. Cuando el ruido es bajo, este método trivial en realidad no es tan malo, por eso es una característica estándar del software de grafos en hojas de cálculo. Pero cuando los datos son ruidosos, la función resultante es puntiaguda y no generaliza bien.

Regresión de *k*-vecinos más cercanos

La **regresión de *k*-vecinos más cercanos** mejora en conectar los puntos. En lugar de usar sólo los dos ejemplos a la izquierda y a la derecha de un punto de consulta x_q, usamos los k vecinos más cercanos. (Aquí estamos usando $k = 3$). Un valor más grande de k tiende a suavizar la magnitud de los picos, aunque la función resultante tiene discontinuidades. La Figura 19.20 muestra dos versiones de la regresión de k-vecinos más cercanos. En (b), tenemos el promedio de k-vecinos más cercanos: $h(x)$ es el valor medio de los k puntos, $\sum y_j/k$. Observe que en los puntos periféricos cerca de $x = 0$ y $x = 14$ las estimaciones son pobres porque toda la evidencia proviene de un lado (el interior) e ignora la tendencia. En (c), tenemos la regresión lineal de k-vecinos más cercanos, que encuentra la mejor línea a través de los k ejemplos. Esto hace un mejor trabajo capturando tendencias en los valores atípicos, pero aún es discontinuo. En ambos (b) y (c), nos quedamos con la pregunta de cómo elegir un buen valor para k. La respuesta, como siempre, es la validación cruzada.

Regresión ponderada localmente

La **regresión ponderada localmente** (Figura 19.20(d)) nos da las ventajas de los vecinos más cercanos sin las discontinuidades. Para evitar discontinuidades en $h(x)$, necesitamos evitar discontinuidades en el conjunto de ejemplos que usamos para estimar $h(x)$. La idea de la regresión ponderada de manera local es que en cada punto de consulta x_q, los ejemplos que están cerca de x_q se ponderan mucho, y los ejemplos que están más lejos se ponderan menos, mientras que los más lejanos no se ponderan en absoluto. La disminución de peso sobre la distancia suele ser gradual, no repentina.

Kernel

Decidimos cuánto ponderar cada ejemplo con una función conocida como **kernel**, cuya entrada es una distancia entre el punto de consulta y el ejemplo. Una función de kernel $\mathcal{K}$ es una función decreciente de la distancia con un máximo en 0, de modo que $\mathcal{K}(Distance(\mathbf{x}_j,\mathbf{x}_q))$ da un mayor peso a los ejemplos $\mathbf{x}_j$ que están más cerca del punto de consulta $\mathbf{x}_q$ para el que estamos tratando de predecir el valor de la función. La integral del valor del kernel sobre todo el espacio de entrada para $\mathbf{x}$ debe ser finita, y si elegimos hacer la integral 1, ciertos cálculos son más fáciles.

La Figura 19.20(d) se generó con un kernel cuadrático, $\mathcal{K}(d) = \max(0,1 - (2|d|/w)^2)$, con un ancho de kernel $w = 10$. También se utilizan otras formas, como las gaussianas. Por lo general, el ancho importa más que la forma exacta: este es un hiperparámetro del modelo que es mejor elegido por validación cruzada. Si los kernels son demasiado anchos obtendremos un ajuste insuficiente pero si son demasiado estrechos obtendremos un sobreajuste. En la Figura 19.20(d), un ancho de kernel de 10 da una curva suave que parece justa.

Hacer una regresión ponderada de manera local con kernels es ahora sencillo. Para un punto de consulta dado $\mathbf{x}_q$ resolvemos el siguiente problema de regresión ponderada:

$$\mathbf{w}^* = \operatorname*{argmin}_{\mathbf{w}} \sum_j \mathcal{K}(Distance(\mathbf{x}_q,\mathbf{x}_j))(y_j - \mathbf{w} \cdot \mathbf{x}_j)^2,$$

donde *Distance* es cualquiera de las métricas de distancia discutidas para los vecinos más cercanos. Entonces la respuesta es $h(\mathbf{x}_q) = \mathbf{w}^* \cdot \mathbf{x}_q$.

Note que necesitamos resolver un nuevo problema de regresión para *cada* punto de consulta, eso es lo que significa ser *local*. (En la regresión lineal ordinaria resolvimos una vez de manera global el problema de regresión, y luego usamos el mismo $h_{\mathbf{w}}$ para cualquier punto de consulta). Mitigando contra este trabajo extra está el hecho de que cada problema de regresión será más fácil de resolver, porque sólo involucra los ejemplos con peso distinto de cero, los ejemplos dentro del ancho del kernel de la consulta. Cuando los anchos del kernel son pequeños, esto puede ser sólo unos pocos puntos.

La mayoría de los modelos no paramétricos tienen la ventaja de que es fácil hacer una validación cruzada de uno en uno sin tener que volver a calcular todo. Con un modelo de k-vecinos más cercanos, por ejemplo, cuando se nos da un ejemplo de prueba $(\mathbf{x},y)$ recuperamos los k vecinos más cercanos una vez, calculamos la pérdida por ejemplo $L(y,h(\mathbf{x}))$ de ellos, y registramos eso como el resultado de dejar uno fuera para cada ejemplo que no es uno de los vecinos. Luego recuperamos los $k+1$ vecinos más cercanos y registramos resultados distintos para dejar fuera a cada uno de los k vecinos. Con N ejemplos, todo el proceso es $O(k)$, no $O(kN)$.

19.7.5 Máquinas de soporte vectorial

Máquinas de soporte vectorial (SVM)

A principios de la década del 2000, la clase de modelos de **máquinas de soporte vectorial** (**SVM**, por sus siglas en inglés) era el enfoque más popular para el aprendizaje supervisado "listo para usar", para cuando no se tiene ningún conocimiento previo especializado sobre un dominio. Esa posición ha sido ahora tomada por las redes de aprendizaje profundo y los bosques aleatorios, pero las SVM conservan tres propiedades atractivas:

1. Las SVM construyen un **separador de margen máximo**, un límite de decisión con la mayor distancia posible a los puntos de ejemplo. Esto les ayuda a generalizar de manera correcta.
2. Las SVM crean un hiperplano separador *lineal*, pero tienen la capacidad de incrustar los datos en un espacio de mayor dimensión, utilizando el llamado **truco del kernel**. Con frecuencia, los datos que no son linealmente separables en el espacio de entrada original son separables en el espacio de mayor dimensión.
3. Las SVM son no paramétricas, el hiperplano separador está definido por un conjunto de puntos de ejemplo, no por una colección de valores de parámetros. Pero, mientras que los modelos de vecinos más cercanos necesitan retener todos los ejemplos, un modelo SVM sólo mantiene los ejemplos que están más cerca del plano separador, en general, sólo una pequeña constante multiplicada por el número de dimensiones. Así, las SVM combinan las ventajas de los modelos no paramétricos y paramétricos: tienen la flexibilidad para representar funciones complejas, pero son resistentes al sobreajuste.

Vemos en la Figura 19.21(a) un problema de clasificación binaria con tres límites de decisión candidatos, cada uno es un separador lineal. Cada uno de ellos es consistente con todos los ejemplos, por lo que desde el punto de vista de la pérdida 0/1, cada uno sería igualmente bueno. La regresión logística encontraría alguna línea separadora; la ubicación exacta de la línea depende de *todos* los puntos de ejemplo. La idea clave de las SVM es que algunos ejemplos son más importantes que otros, y que prestar atención a ellos permite llevar a una mejor generalización.

Considere la línea separadora más baja de las tres en (a). Se acerca mucho a cinco de los ejemplos negros. Aunque clasifica de manera correcta todos los ejemplos y, por lo tanto, minimiza la pérdida, debería ponerle nervioso que tantos ejemplos estén cerca de la línea; puede ser que otros ejemplos negros resulten estar en el lado equivocado de la línea.

Las SVM abordan este problema: en lugar de minimizar la *pérdida empírica* esperada en los datos de entrenamiento, las SVM intentan minimizar la pérdida de *generalización* esperada. No

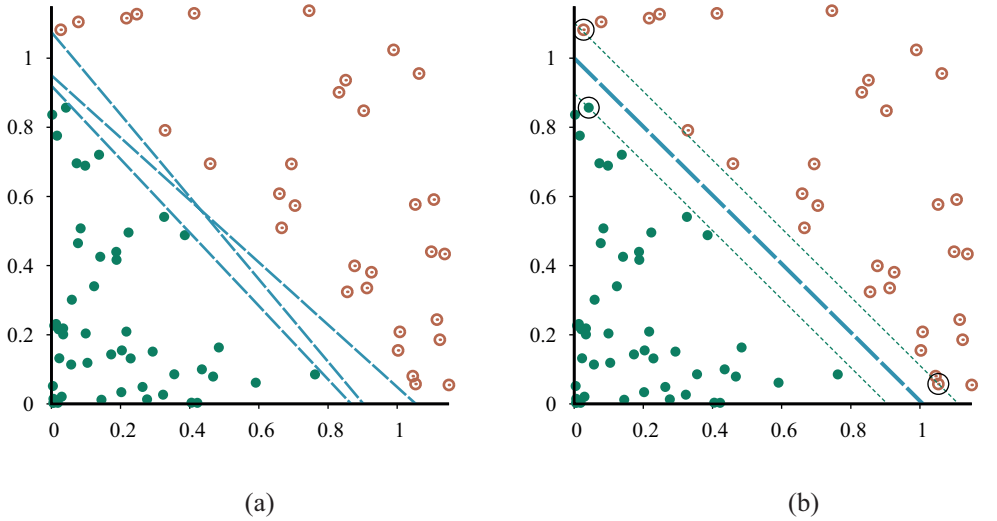


Figura 19.21 Clasificación de la máquina de soporte vectorial: (a) Dos clases de puntos (círculos abiertos naranjas y círculos llenos verdes) y tres separadores lineales candidatos. (b) El separador de margen máximo (línea discontinua gruesa) está en el punto medio del **margen** (área entre líneas discontinuas). Los **vectores de soporte** (puntos con círculos negros grandes) son los ejemplos más cercanos al separador; aquí hay tres.

sabemos dónde caerán los puntos aún no vistos, pero bajo la suposición probabilística de que se extraen de la misma distribución que los ejemplos vistos con anterioridad, existen argumentos de la teoría del aprendizaje computacional (Sección 19.5) que sugieren que minimizamos la pérdida de generalización eligiendo el separador que está más lejos de los ejemplos que hemos visto hasta ahora. Nosotros llamamos a este separador, mostrado en la Figura 19.21(b), **separador de margen máximo**. El **margen** es el ancho del área limitada por líneas discontinuas en la figura, el doble de la distancia desde el separador hasta el punto de ejemplo más cercano.

Separador de margen máximo
Margen

Ahora ¿cómo encontramos este separador? Antes de mostrar las ecuaciones, alguna notación: de manera canónica las SVM usan la convención de que las etiquetas de clase son +1 y -1, en lugar del +1 y 0 que hemos estado usando hasta ahora. Además, mientras que antes pusimos la intersección en el vector de peso $\mathbf{w}$ (y un valor ficticio 1 correspondiente en $x_{j,0}$), las SVM no hacen eso; mantienen la intersección como un parámetro separado, b.

Con eso en mente, el separador se define como el conjunto de puntos $\{\mathbf{x} : \mathbf{w}\cdot\mathbf{x} + b = 0\}$. Nosotros podríamos buscar el espacio de $\mathbf{w}$ y b con descenso de gradiente para encontrar los parámetros que maximizan el margen mientras clasifica de manera correcta todos los ejemplos.

Sin embargo, resulta que hay otro enfoque para resolver este problema. No mostraremos los detalles, pero diremos que existe una representación alternativa llamada representación dual, en la que se encuentra la solución óptima resolviendo

$$\underset{\alpha}{\text{argmax}} \sum_j \alpha_j - \frac{1}{2}\sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k) \tag{19.10}$$

sujeto a las restricciones $\alpha_j \geq 0$ y $\sum_j \alpha_j y_j = 0$. Este es un problema de optimización de **programación cuadrática**, para el cual existen buenos paquetes de software. Una vez que hemos

Programación cuadrática

encontrado el vector α podemos volver a $\mathbf{w}$ con la ecuación $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$, o podemos permanecer en la representación dual. Hay tres propiedades importantes de la Ecuación (19.10). Primero, la expresión es convexa, tiene un único máximo global que permite encontrarse de manera eficiente. Segundo, *los datos entran en la expresión sólo en forma de productos escalares de pares de puntos*. Esta segunda propiedad también es cierta para la ecuación del separador en sí; una vez que se han calculado los α_j óptimos, la ecuación es[11]

$$h(\mathbf{x}) = \text{sign}\left(\sum_j \alpha_j y_j (\mathbf{x} \cdot \mathbf{x}_j) - b\right). \tag{19.11}$$

Vectores de soporte

Una última propiedad importante es que los pesos α_j asociados con cada punto de datos son *cero* excepto para los **vectores de soporte**, los puntos más cercanos al separador. (Se llaman vectores de "soporte" porque "sostienen" el plano de separación). Debido a que hay muchos menos vectores de soporte que ejemplos, las SVM obtienen algunas de las ventajas de los modelos paramétricos.

¿Qué pasa si los ejemplos no son linealmente separables? La Figura 19.22(a) muestra un espacio de entrada definido por los atributos $\mathbf{x} = (x_1, x_2)$, con ejemplos positivos ($y = +1$) dentro de una región circular y ejemplos negativos ($y = -1$) fuera. Note que, no hay un separador lineal para este problema. Ahora, suponga que reexpresamos los datos de entrada; es decir, mapeamos cada vector de entrada $\mathbf{x}$ a un nuevo vector de valores de características, $F(\mathbf{x})$. En particular, usemos las tres características

$$f_1 = x_1^2, \qquad f_2 = x_2^2, \qquad f_3 = \sqrt{2} x_1 x_2. \tag{19.12}$$

En breve veremos de dónde vienen, pero por ahora sólo mire lo que sucede. La Figura 19.22(b) muestra los datos en el nuevo espacio tridimensional definido por las tres características; ¡los datos son *linealmente separables* en este espacio! Este fenómeno es en realidad muy general: si los datos se mapean en un espacio de dimensión suficientemente alta, entonces casi siempre serán linealmente separables, si mira un conjunto de puntos desde suficientes direcciones, encontrará una forma de alinearlos. Aquí, usamos sólo tres dimensiones;[12] El Ejercicio 19.SVME le pide que muestre que cuatro dimensiones son suficientes para separar linealmente un círculo en cualquier lugar del plano (no sólo en el origen), y cinco dimensiones son suficientes para separar linealmente cualquier elipse. En general (con algunas excepciones especiales) si tenemos N puntos de datos entonces siempre serán separables en espacios de $N - 1$ dimensiones o más (Ejercicio 19.EMBE).

Ahora, en la mayoría de los casos no esperaríamos encontrar un separador lineal en el espacio de entrada $\mathbf{x}$, pero podemos encontrar separadores lineales en el espacio de características de alta dimensión $F(\mathbf{x})$ reemplazando $\mathbf{x}_j \cdot \mathbf{x} k$ en la Ecuación (19.10) con $F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$. Esto por sí sólo no es notable, reemplazar $\mathbf{x}$ por $F(\mathbf{x})$ en *cualquier* algoritmo de aprendizaje tiene el efecto requerido, pero el producto escalar tiene algunas propiedades especiales. Resulta que $F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$ se calcula sin primero calcular F para cada punto. En nuestro espacio de características tridimensional definido por la Ecuación (19.12), un poco de álgebra muestra que

$$F(\mathbf{x}_j) \cdot F(\mathbf{x}_k) = (\mathbf{x}_j \cdot \mathbf{x}_k)^2.$$

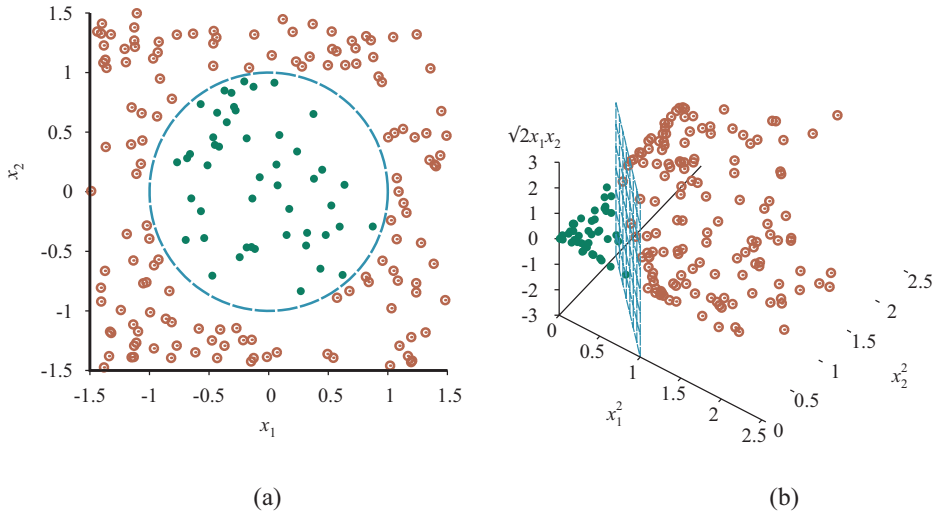
Función de kernel

(Esa es la razón por la que $\sqrt{2}$ está en f_3). La expresión $(\mathbf{x}_j \cdot \mathbf{x}_k)^2$ se llama **función de kernel**,[13] y en general se escribe como $K(\mathbf{x}_j, \mathbf{x}_k)$. La función de kernel se aplica a pares de datos de entrada para evaluar productos escalares en algún espacio de características correspondiente. Entonces,

[11] La función sign(x) devuelve +1 para un x positivo, –1 para un x negativo.

[12] El lector puede notar que podríamos haber usado sólo f_1 y f_2, pero el mapeo 3D ilustra mejor la idea.

[13] Este uso de "función de kernel" es diferente de los kernels en la regresión ponderada de manera local. Algunos kernels SVM son métricas de distancia, pero no todos lo son.



(a) (b)

Figura 19.22 (a) Un conjunto de entrenamiento bidimensional con ejemplos positivos como círculos verdes llenos y ejemplos negativos como círculos naranjas abiertos. También se muestra la verdadera frontera de decisión, $x_1^2 + x_2^2 \leq 1$. (b) Los mismos datos después de mapearlos en un espacio de entrada tridimensional $(x_1^2, x_2^2, \sqrt{2}x_1x_2)$. La frontera de decisión circular en (a) se convierte en una frontera de decisión lineal en tres dimensiones. La Figura 19.21(b) da un primer plano del separador en (b).

podemos encontrar separadores lineales en el espacio de características de mayor dimensión $F(\mathbf{x})$ simple, reemplazando $\mathbf{x}_j \cdot \mathbf{x}_k$ en la Ecuación (19.10) con una función de kernel $K(\mathbf{x}_j, \mathbf{x}_k)$. Por lo tanto, podemos aprender en el espacio de mayor dimensión, pero sólo calculamos funciones de kernel en lugar de la lista completa de características para cada punto de datos.

El siguiente paso es ver que no hay nada especial en el kernel $K(\mathbf{x}_j, \mathbf{x}_k) = (\mathbf{x}_j \cdot \mathbf{x}_k)^2$. Corresponde a un espacio de características de mayor dimensión en particular, pero otras funciones de kernel corresponden a otros espacios de características. Un resultado respetable en matemáticas, el **teorema de Mercer** (1909), indica que cualquier función de kernel "razonable"[14] corresponde a *algún* espacio de características. Estos espacios de características son grandes, incluso para kernels que parecen inocuos. Por ejemplo, el **kernel polinominal**, $K(\mathbf{x}_j, \mathbf{x}_k) = (1 + \mathbf{x}_j \cdot \mathbf{x}_k)^d$, corresponde a un espacio de características cuya dimensión es exponencial en d. Un kernel común es el gaussiano: $K(\mathbf{x}_j, \mathbf{x}_k) = e^{-\gamma|xj - xk|^2}$.

Teorema de Mercer

Kernel polinominal

19.7.6 El truco del kernel

Este, entonces, es el ingenioso **truco de kernel**: introduciendo estos kernels en la Ecuación (19.10), se permiten encontrar separadores lineales óptimos de manera eficiente en espacios de características con miles de millones de (o incluso infinitas) dimensiones. Los separadores lineales resultantes, cuando se mapean de nuevo al espacio de entrada original, pueden corresponder a límites de decisión ondulados y no lineales entre los ejemplos positivos y negativos.

Truco de kernel

En el caso de datos ruidosos, es posible que no queramos un separador lineal en algún espacio de alta dimensión. Más bien, nos gustaría una superficie de decisión en un espacio de menor

[14] Aquí, "razonable" significa que la matriz $\mathbf{K}_{jk} = K(\mathbf{x}_j, \mathbf{x}_k)$ es positiva definida.

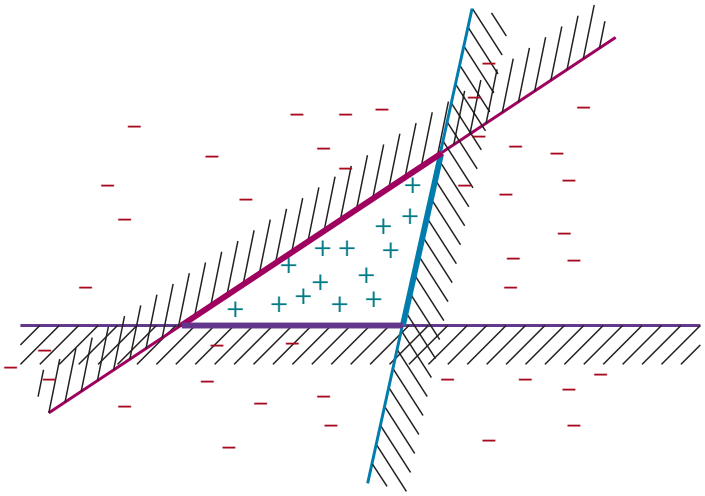


Figura 19.23 Ilustración del aumento de la capacidad expresiva obtenida mediante el aprendizaje en conjunto. Tomamos tres hipótesis de umbral lineal, cada una de las cuales clasifica de manera positiva en el lado no sombreado, y clasificamos como positivo cualquier ejemplo clasificado positivamente por los tres. La región triangular resultante es una hipótesis no expresable en el espacio de hipótesis original.

Margen suave

dimensión que no separa de manera clara las clases, sino que refleja la realidad de los datos ruidosos. Eso es posible con el clasificador de **margen suave**, que permite que los ejemplos caigan en el lado equivocado del límite de decisión, pero les asigna una penalización proporcional a la distancia requerida para moverlos de nuevo al lado correcto.

El método del kernel se aplica no sólo con algoritmos de aprendizaje que encuentran separadores lineales óptimos, sino también con cualquier otro algoritmo que se pueda reformular para trabajar sólo con productos punto de pares de puntos de datos, como en las Ecuaciones (19.10) y (19.11). Una vez hecho esto, el producto punto se reemplaza por una función de kernel y tenemos una versión **kernelizada** del algoritmo.

Kernelizada

19.8 Aprendizaje en conjunto

Aprendizaje en conjunto

Hasta ahora hemos examinado métodos de aprendizaje en los que se utiliza una única hipótesis para hacer predicciones. La idea del **aprendizaje en conjunto** es seleccionar una colección, o **conjunto**, de hipótesis, $h_1, h_2, ..., h_n$, y combinar sus predicciones mediante promedio, votación, o por otro nivel de aprendizaje automático. Llamamos a las hipótesis individuales **modelos base** y a su combinación un **modelo de conjunto**.

Modelos base

Modelo de conjunto

Hay dos razones para hacer esto. La primera es reducir el sesgo. El espacio de hipótesis de un modelo base puede ser muy restrictivo, imponiendo un fuerte sesgo (como el sesgo de un límite de decisión lineal en la regresión logística). Un conjunto permite ser más expresivo y, por lo tanto, tener menos sesgo, que los modelos base. La Figura 19.23 muestra que un conjunto de tres clasificadores lineales representa una región triangular que no podría ser representada por un solo clasificador lineal. Un conjunto de n clasificadores lineales permite que más funciones sean realizables, a un costo de sólo n veces más cálculo; esto es en la mayoría de los casos mejor que permitir un espacio de hipótesis general que podría requerir más cálculo.

La segunda razón es reducir la varianza. Consideremos un conjunto de $K = 5$ clasificadores binarios que combinamos usando votación por mayoría. Para que el conjunto clasifique mal un nuevo ejemplo, *al menos tres de los cinco clasificadores tienen que clasificarlo mal*. La esperanza es que esto sea menos probable que una única clasificación errónea por un solo clasificador. Para cuantificar eso, suponga que ha entrenado un solo clasificador que es correcto en 80 % de los casos. Ahora, cree un conjunto de 5 clasificadores, cada uno entrenado en un subconjunto diferente de los datos para que sean independientes. Suponga que esto lleva a alguna reducción en la calidad, y cada clasificador individual es correcto en sólo 75 % de los casos. Pero juntos, el voto mayoritario del conjunto será correcto en 89 % de los casos (y 99 % con 17 clasificadores), asumiendo verdadera independencia.

En la práctica, la suposición de independencia es irrazonable: los clasificadores individuales comparten algunos de los mismos datos y suposiciones y, por lo tanto, no son independientes y compartirán algunos de los mismos errores. Pero si los clasificadores componentes son al menos algo no correlacionados, entonces el aprendizaje en conjunto cometerá menos errores de clasificación. Ahora consideraremos cuatro formas de crear conjuntos: bagging, bosques aleatorios, apilamiento (*stacking*) e impulso (*boosting*).

19.8.1 Bagging

Bagging

En **bagging**,[15] generamos K conjuntos de entrenamiento distintos muestreando con reemplazo desde el conjunto de entrenamiento original. Es decir, seleccionamos de manera aleatoria N ejemplos del conjunto de entrenamiento, pero cada una de esas selecciones podría ser un ejemplo que seleccionamos antes. Luego ejecutamos nuestro algoritmo de aprendizaje automático en los N ejemplos para obtener una hipótesis. Repetimos este proceso K veces, obteniendo K hipótesis diferentes. Luego, cuando se nos pide predecir el valor de una nueva entrada, agregamos las predicciones de todas las hipótesis K. Para problemas de clasificación, eso significa tomar el voto de pluralidad (el voto mayoritario para la clasificación binaria). Para problemas de regresión, la salida final es el promedio:

$$h(\mathbf{x}) = \frac{1}{K}\sum_{i=1}^{K} h_i(\mathbf{x})$$

El bagging tiende a reducir la varianza y es un enfoque estándar cuando hay datos limitados o cuando se ve que el modelo base está sobreajustando. El bagging se aplica a cualquier clase de modelo, pero se usa de manera más común con árboles de decisión. Es apropiado porque los árboles de decisión son inestables: un conjunto de ejemplos diferente puede llevar a un árbol completamente diferente. El bagging suaviza esta varianza. Si tiene acceso a múltiples computadoras, entonces el bagging es eficiente, porque las hipótesis se calculan en paralelo.

19.8.2 Bosques aleatorios

No obstante, el bagging de árboles de decisión con frecuencia termina dándonos K árboles que están altamente correlacionados. Si hay un atributo con una ganancia de información muy alta, es probable que sea la raíz de la mayoría de los árboles. El modelo de **bosque aleatorio** es una forma de bagging de árboles de decisión en la que tomamos medidas adicionales para hacer que el conjunto de K árboles sea más diverso para reducir la varianza. Los bosques aleatorios se utilizan para la clasificación o la regresión.

Bosque aleatorio

La idea clave es variar de manera aleatoria las *elecciones de atributos* (en lugar de los ejemplos de entrenamiento). En cada punto de división en la construcción del árbol, seleccionamos una muestra aleatoria de atributos, y luego calculamos cuál de ellos da la mayor ganancia de información. Si

[15] Nota sobre la terminología: En estadística, una muestra con reemplazo se llama **bootstrap**, y "bagging" es la abreviatura de "bootstrap aggregating".

hay n atributos, una elección predeterminada común es que cada división elija de manera aleatoria $\sqrt{n}$ atributos para considerar para problemas de clasificación, o $n/3$ para problemas de regresión.

Una mejora adicional es usar la aleatoriedad en la selección del *valor* del punto de división: para cada atributo seleccionado, tomamos de manera aleatoria varias muestras candidatas de una distribución uniforme sobre el rango del atributo. Luego seleccionamos el valor que tiene la mayor ganancia de información. Eso hace que sea más probable que cada árbol en el bosque sea diferente. Los árboles construidos de esta manera se llaman **árboles extremadamente aleatorizados** (**ExtraTrees**).

Árboles extremadamente aleatorizados (ExtraTrees)

Los bosques aleatorios son eficientes para crear. Podría pensar que tomaría K veces más tiempo crear un conjunto de K árboles, pero no es tan malo, por tres razones: (a) cada punto de división se ejecuta más rápido porque estamos considerando menos atributos, (b) podemos omitir el paso de poda para cada árbol individual, porque el conjunto en su conjunto disminuye el sobreajuste, y (c) si tenemos K computadoras disponibles, podemos construir todos los árboles en paralelo. Por ejemplo, Adele Cutler informa que para un problema de 100 atributos, si tenemos sólo tres CPUs podemos crecer un bosque de $K = 100$ árboles en casi el mismo tiempo que se tarda en crear un sólo árbol de decisión en una sola CPU.

Todos los hiperparámetros de los bosques aleatorios permiten ser entrenados por validación cruzada: el número de árboles K, el número de ejemplos utilizados por cada árbol N (con frecuencia expresado como un porcentaje del conjunto de datos completo), el número de atributos utilizados en cada punto de división (expresado como una función del número total de atributos, como $\sqrt{n}$), y el número de puntos de división aleatorios intentados si estamos usando ExtraTrees. En lugar de la estrategia regular de validación cruzada, podríamos medir el **error fuera de bolsa**: el error medio en cada ejemplo, utilizando sólo los árboles cuyo conjunto de ejemplos no incluyó ese ejemplo en particular.

Error fuera de bolsa

Hemos sido advertidos de que los modelos más complejos son propensos al sobreajuste, y observamos que eso es cierto para los árboles de decisión, donde encontramos que la **poda** era una respuesta para prevenir el sobreajuste. Los bosques aleatorios son modelos complejos y sin podar. Sin embargo, son resistentes al sobreajuste. A medida que aumenta la capacidad agregando más árboles al bosque, tiende a mejorar la tasa de error del conjunto de validación. La curva suele parecerse a la Figura 19.9(b), no a la (a).

Breiman (2001) proporciona una prueba matemática de que (en casi todos los casos) a medida que agrega más árboles al bosque, el error converge, no crece. Una forma de pensar en ello es que la selección aleatoria de atributos produce una variedad de árboles, reduciendo así la varianza, pero como no necesitamos podar los árboles, permiten cubrir todo el espacio de entrada a una resolución más alta. Algunos árboles pueden cubrir casos únicos que aparecen sólo unas pocas veces en los datos, y sus votos resultan decisivos, pero son superados en votos cuando no se aplican. Dicho esto, los bosques aleatorios no son inmunes al sobreajuste. Aunque el error no permite aumentar en el límite, eso no significa que el error se reducirá a cero.

Los bosques aleatorios han tenido mucho éxito en una amplia variedad de problemas de aplicación. En las competencias de ciencia de datos de Kaggle, fueron el enfoque más popular de los equipos ganadores de 2011 a 2014, y siguen siendo un enfoque común hasta el día de hoy (aunque el **aprendizaje profundo** y el **aumento (ascenso) de gradiente** se han vuelto aún más comunes entre los ganadores recientes). El paquete randomForest en R ha sido un favorito particular. En finanzas, los bosques aleatorios se han utilizado para predecir el incumplimiento de tarjetas de crédito, la predicción de ingresos del hogar y la fijación de precios de opciones. Las aplicaciones mecánicas incluyen el diagnóstico de fallos de máquinas y la detección remota. Las aplicaciones bioinformáticas y médicas incluyen la retinopatía diabética, la expresión génica de microarrays, el análisis de expresión de proteínas de espectro de masas, el descubrimiento de biomarcadores y la predicción de interacciones proteína-proteína.

19.8.3 Apilamiento

Generalización apilada

Mientras que el bagging combina múltiples modelos base de la misma clase de modelos entrenados en diferentes datos, la técnica de **generalización apilada** (o **apilamiento** para abreviar) combina múltiples modelos base apilados de diferentes clases de modelos entrenados en los mismos datos. Por ejemplo, suponga que se nos da el conjunto de datos del restaurante, la primera fila de los cuales se muestra aquí:

$\mathbf{x}_1$ = Sí, No, No, Sí, Algunos, $$$, No, Sí, Francesa, 0–10; y_1 = Sí

Separamos los datos en conjuntos de entrenamiento, validación y prueba y usamos el conjunto de entrenamiento para entrenar, digamos, tres modelos base separados: un modelo SVM, un modelo de regresión logística y un modelo de árbol de decisión.

En el siguiente paso tomamos el conjunto de datos de validación y aumentamos cada fila con las predicciones hechas a partir de los tres modelos base, obteniendo filas que se ven así (donde las predicciones se muestran en negrita):

x_2 = Sí, No, No, Sí, Completo, $, No, No, Tailandesa, 30–60, **Sí**, **No**, **No**; y_2 =No

Usamos este conjunto de validación para entrenar un nuevo modelo de conjunto, digamos un modelo de regresión logística (pero no necesariamente tiene que ser una de las clases de modelos base). El modelo de conjunto permite usar las predicciones y los datos originales como mejor le parezca. Podría aprender un promedio ponderado de los modelos base; por ejemplo, que las predicciones deben ponderarse en una proporción de 50%:30%:20%. O podría aprender interacciones no lineales entre los datos y las predicciones; por ejemplo confiando más en el modelo SVM cuando el tiempo de espera es largo. Usamos los mismos datos de entrenamiento para entrenar cada uno de los modelos base, y luego usamos los datos de validación retenidos (más predicciones) para entrenar el modelo de conjunto. También es posible usar validación cruzada si se desea.

El método se llama "apilamiento" porque se puede pensar en él como una capa de modelos base con un modelo de conjunto apilado encima de él, operando en la salida de los modelos base. De hecho, es posible apilar múltiples capas, cada una operando en la salida de la capa anterior. El apilamiento reduce el sesgo y, por lo regular, conduce a un rendimiento que resulta mejor que cualquiera de los modelos base individuales. El apilamiento es utilizado con frecuencia por los equipos ganadores en competencias de ciencia de datos (como Kaggle y la KDD Cup), debido a que los individuos permiten trabajar de forma independiente, cada uno refinando su propio modelo base, para luego unirse y construir el modelo de conjunto apilado final.

19.8.4 Impulso

Boosting

Conjunto de entrenamiento ponderado

El método de conjunto más popular se llama **boosting**. Para entender cómo funciona, necesitamos introducir primero la idea de un **conjunto de entrenamiento ponderado**, en el que cada ejemplo tiene un peso asociado $w_j \geq 0$ que describe cuánto debería contar el ejemplo durante el entrenamiento. Es decir, si un ejemplo tuviera un peso de 3 y los otros ejemplos todos tuvieran un peso de 1, eso sería equivalente a tener 3 copias del ejemplo en el conjunto de entrenamiento.

Boosting comienza con pesos iguales $w_j = 1$ para todos los ejemplos. A partir de este conjunto de entrenamiento, genera la primera hipótesis, h_1. En general, h_1 clasificará algunos de los ejemplos de entrenamiento de manera correcta y algunos de manera incorrecta. Nos gustaría que la siguiente hipótesis lo hiciera mejor en los ejemplos mal clasificados, así incrementamos sus pesos mientras disminuimos los pesos de los ejemplos clasificados de manera correcta.

A partir de este nuevo conjunto de entrenamiento ponderado, generamos la hipótesis h_2. El proceso continúa de esta manera hasta que hemos generado K hipótesis, donde K es una entrada para el algoritmo

de boosting. Los ejemplos que son difíciles de clasificar obtendrán pesos cada vez mayores hasta que el algoritmo se vea obligado a crear una hipótesis que los clasifique de manera correcta. Tenga en cuenta que este es un algoritmo codicioso en el sentido de que no retrocede; una vez que ha elegido una hipótesis h_i, nunca deshará esa elección, más bien agregará nuevas hipótesis. También es un algoritmo secuencial, por lo que no podemos calcular todas las hipótesis en paralelo como podríamos con bagging.

El conjunto final permite que cada hipótesis vote, como en bagging, excepto que cada hipótesis obtiene un número ponderado de votos: a las hipótesis que lo hicieron mejor en sus respectivos conjuntos de entrenamiento ponderados se les da más peso de voto. Para la regresión o la clasificación binaria tenemos

$$h(\mathbf{x}) = \sum_{i=1}^{K} z_i h_i(\mathbf{x})$$

donde z_i es el peso de la i-ésima hipótesis. (Esta ponderación de hipótesis es distinta de la ponderación de ejemplos).

La Figura 19.24 muestra cómo funciona el algoritmo de manera conceptual. Hay muchas variantes de la idea básica de boosting, con diferentes formas de ajustar los pesos de los ejemplos y combinar las hipótesis. Las variantes comparten la idea general de que los ejemplos difíciles obtienen más peso a medida que pasamos de una hipótesis a la siguiente. Al igual que los métodos de aprendizaje bayesianos que veremos en el Capítulo 20, también dan más peso a las hipótesis más precisas.

Un algoritmo específico, llamado ADABOOST, se muestra en la Figura 19.25. Por lo general, este algoritmo se aplica con árboles de decisión como las hipótesis componentes; con frecuencia los árboles están limitados en tamaño. ADABOOST tiene una propiedad muy importante: si el algoritmo de aprendizaje de entrada L es un algoritmo de **aprendizaje débil**, significa que L siempre devuelve una hipótesis con precisión en el conjunto de entrenamiento que es mejor que adivinar al azar (es decir, 50 % + ϵ para la clasificación booleana), entonces ADABOOST devolverá una hipótesis que *clasifica de manera perfecta los datos de entrenamiento* para un K suficientemente grande. Así, el algoritmo *aumenta* la precisión del algoritmo de aprendizaje original en los datos de entrenamiento.

Aprendizaje débil

En otras palabras, el boosting permite superar cualquier cantidad de sesgo en el modelo base, siempre que el modelo base sea ϵ mejor que adivinar al azar. (En nuestro pseudocódigo dejamos de generar hipótesis si obtenemos una que es peor que al azar). Este resultado se mantiene sin importar cuán inexpresivo sea el espacio de hipótesis original y sin importar cuán compleja sea la función que se está aprendiendo. Las fórmulas exactas para los pesos en la Figura 19.25 (con *error*/(1 – *error*, etc.) se eligen para facilitar la prueba de esta propiedad (ver Freund y Schapire, 1996). Por supuesto, esta propiedad no garantiza la precisión en ejemplos no vistos con anterioridad.

Veamos cómo funciona el *boosting* en los datos del restaurante. Elegiremos como nuestro espacio de hipótesis original la clase de **bloques de decisión**, que son árboles de decisión con sólo una prueba, en la raíz. La curva inferior en la Figura 19.26(a) muestra que los bloques de decisión sin boosting no son muy efectivos para este conjunto de datos, alcanzando un rendimiento de predicción de solo 81 % en 100 ejemplos de entrenamiento. Cuando se aplica el boosting (con $K = 5$), el rendimiento es mejor, alcanzando 93 % después de 100 ejemplos.

Bloque de decisión

Ocurre algo interesante a medida que aumenta el tamaño del conjunto K. La Figura 19.26(b) muestra el rendimiento del conjunto de entrenamiento (en 100 ejemplos) en función de K. Observe que el error llega a cero cuando K es 20; es decir, una combinación de mayoría ponderada de 20 bloques de decisión es suficiente para ajustar los 100 ejemplos exactos, este es el punto de interpolación. A medida que se agregan más bloques al conjunto, el error permanece en cero. El grafo también muestra *que el rendimiento del conjunto de prueba continúa aumentando mucho después de que el error del conjunto de entrenamiento ha llegado a cero.* En $K = 20$, el rendimiento de la prueba es 0.95 (o 0.05 de error), y el rendimiento aumenta a 0.98 hasta $K = 137$, antes de caer de manera gradual a 0.95.

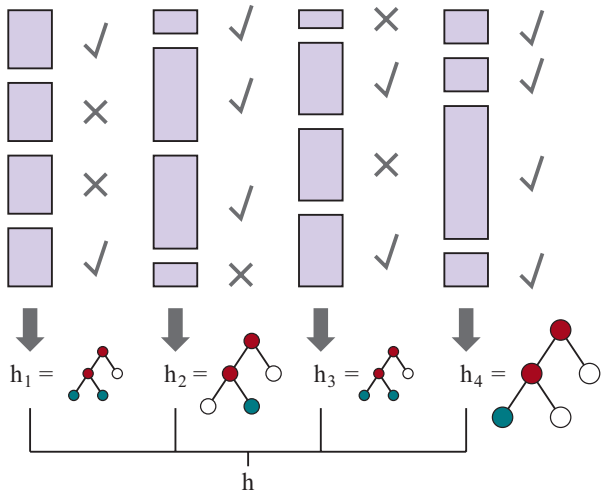


Figura 19.24 Cómo funciona el algoritmo de boosting. Cada rectángulo sombreado corresponde a un ejemplo, la altura del rectángulo corresponde al peso. Las marcas de verificación y las cruces indican si el ejemplo fue clasificado de manera correcta por la hipótesis actual. El tamaño del árbol de decisión indica el peso de esa hipótesis en el conjunto final.

Este hallazgo, robusto en conjuntos de datos y espacios de hipótesis, fue una gran sorpresa cuando se observó por primera vez. La navaja de Occam nos dice que no hagamos hipótesis más complejas de lo necesario, pero el grafo nos dice que ¡las predicciones *mejoran* a medida que la hipótesis del conjunto se vuelve más compleja¡ Se han propuesto varias explicaciones para esto. Una visión es que el boosting aproxima el **aprendizaje bayesiano** (ver Capítulo 20), que se puede demostrar que es un algoritmo de aprendizaje óptimo, y que la aproximación mejora a medida que se agregan más hipótesis. Otra posible explicación es que la adición de más hipótesis permite que el conjunto sea más seguro en su distinción entre ejemplos positivos y negativos, lo que le ayuda a la hora de clasificar nuevos ejemplos.

19.8.5 Boosting de gradiente

Boosting de gradiente

Para la regresión y clasificación de datos tabulares factorizados, el **boosting de gradiente**, a veces llamado máquinas de boosting de gradiente (GBM) o árboles de regresión impulsados por gradiente (GBRT), se ha convertido en un método muy popular. Como su nombre indica, el boosting de gradiente es una forma de aumento que utiliza el descenso de gradiente. Recuerde que en AdaBoost, comenzamos con una hipótesis $\mathbf{h}_1$, y la impulsamos con una secuencia de hipótesis que prestan especial atención a los ejemplos que las anteriores hicieron mal. En el boosting de gradiente también agregamos nuevas hipótesis de boosting, que prestan atención no a ejemplos específicos, sino al **gradiente** entre las respuestas correctas y las respuestas dadas por las hipótesis anteriores.

Como en los otros algoritmos que utilizaban el descenso de gradiente, comenzamos con una función de pérdida diferenciable; podríamos usar el error cuadrado para la regresión o la pérdida logarítmica para la clasificación. Como en AdaBoost, luego construimos un árbol de decisión. En la Sección 19.6.2 utilizamos el descenso de gradiente para minimizar los parámetros de un modelo, calculamos la pérdida y actualizamos los parámetros en la dirección de menos pérdida.

```
function AdaBoost(examples,L,K) returns una hipótesis
  inputs: examples, conjunto de N ejemplos etiquetados (x_1,y_1),..., (x_N,y_N)
          L, un algoritmo de aprendizaje
          K, el número de hipótesis en el conjunto
  local variables: w, un vector de N pesos de ejemplo, en inicio todos 1/N
                   h, un vector de K hipótesis
                   z, un vector de K pesos de hipótesis

  ϵ ← es un número pequeño positivo, utilizado para evitar la división por cero
  for k = 1 to K do
    h[k] ← L(examples,w)
    error ← 0
    for j = 1 to N do          // Compute the total error for h[k]
      if h[k](x_j) ≠ y_j then error ← error + w[j]
    if error > 1/2 then break del bucle
    error ← min(error, 1 − ϵ)
    for j = 1 to N do          // Give more weight to the examples h[k] got wrong
      if h[k](x_j) = y_j then w[j] ← w[j]·error/(1−error)
    w ← Normalize(w)
    z[k] ← ½ log((1−error)/error)          // Give more weight to accurate h[k]
  return Function(x) : ∑ z_i h_i(x)
```

Figura 19.25 La variante AdaBoost del método de refuerzo para el aprendizaje en conjunto. El algoritmo genera hipótesis reponderando sucesivamente los ejemplos de entrenamiento. La función Weighted-Majority genera una hipótesis que devuelve el valor de salida con el voto más alto de las hipótesis en **h**, con votos ponderados por **z**. Para problemas de regresión o para clasificación binaria con dos clases -1 y 1, esto es $\sum_k \mathbf{h}[k]\mathbf{z}[k]$.

Con el boosting de gradiente, no estamos actualizando los parámetros de los modelos existentes, estamos actualizando los parámetros del siguiente árbol, pero debemos hacerlo de una manera que reduzca la pérdida moviéndonos en la dirección correcta a lo largo del gradiente.

Como en los modelos que vimos en la Sección 19.4.3, la **regularización** ayuda a prevenir el sobreajuste. Eso puede venir en la forma de limitar el número de árboles o su tamaño (en términos de su profundidad o número de nodos). Puede provenir de la tasa de aprendizaje, α, que dice cuánto moverse en la dirección del gradiente; los valores en el rango de 0.1 a 0.3 son comunes, y cuanto menor sea la tasa de aprendizaje, más árboles necesitaremos en el conjunto.

El aumento de gradiente se implementa en el popular paquete XGBoost (eXtreme Gradient Boosting), que se utiliza de manera rutinaria tanto para aplicaciones a gran escala en la industria (para problemas con miles de millones de ejemplos), como por los ganadores de competencias de ciencia de datos (en 2015, fue utilizado por todos los equipos en el top 10 de la KDDCup). XGBoost hace aumento de gradiente con poda y regularización, y se preocupa por ser eficiente, organizando de manera cuidadosa la memoria para evitar fallos de caché, y permitir el cálculo paralelo en múltiples máquinas.

19.8.6 Aprendizaje online

Hasta ahora, todo lo que hemos hecho en este capítulo se ha basado en la suposición de que los datos son i.i.d. (independientes e idénticamente distribuidos). Por un lado, es una suposición

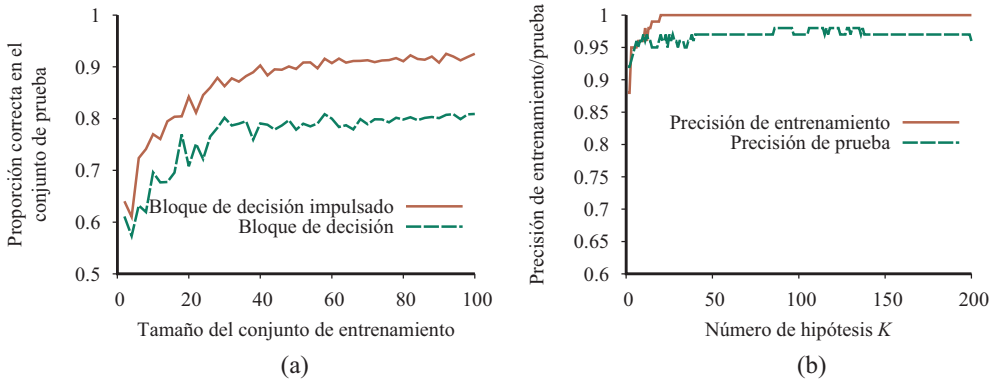


Figura 19.26 (a) Grafo que muestra el rendimiento de los bloques de decisión impulsados con $K = 5$ contra bloques de decisión no impulsados en los datos del restaurante. (b) La proporción correcta en el conjunto de entrenamiento y el conjunto de prueba en función de K, el número de hipótesis en el conjunto. Observe que la precisión del conjunto de prueba mejora incluso después de que la precisión del entrenamiento alcanza 1; es decir, después de que el conjunto se ajusta de manera exacta a los datos.

sensata: si el futuro no se parece en nada al pasado, ¿cómo podemos predecir algo? Por otro lado, es una suposición demasiado fuerte: sabemos que hay correlaciones entre el pasado y el futuro, y en escenarios complejos es poco probable que capturemos todos los datos que harían que el futuro sea independiente del pasado dado los datos.

En esta sección examinamos qué hacer cuando los datos no son i.i.d., cuando pueden cambiar con el tiempo. En este caso, importa *cuando* hacemos una predicción, por lo que adoptaremos la perspectiva llamado **aprendizaje online**: un agente recibe una entrada x_j de la naturaleza, predice el correspondiente y_j, y luego se le dice la respuesta correcta. Entonces el proceso se repite con x_{j+1}, y así sucesivamente. Uno podría pensar que esta tarea es desesperada, si la naturaleza es adversarial, todas las predicciones pueden estar equivocadas. Resulta que hay algunas garantías que podemos hacer.

Aprendizaje online

Consideremos la situación en la que nuestra entrada consiste en predicciones de un panel de expertos. Por ejemplo, cada día K los expertos predicen si el mercado de valores subirá o bajará, y nuestra tarea es agrupar esas predicciones y hacer las nuestras. Una forma de hacer esto es para llevar un registro de cómo se desempeña cada experto, y elegir creerles en proporción a su rendimiento pasado. Esto se llama **algoritmo de mayoría ponderada aleatorizada**. Podemos describirlo de manera formal:

Algoritmo de mayoría ponderada aleatorizada

Inicializar un conjunto de pesos $\{w_1,...,w_K\}$ todos a 1.
for each problema a resolver **do**

1. Recibir las predicciones $\{\hat{y}_1,..., \hat{y}_K\}$ de los expertos.
2. Elegir de manera aleatoria un experto k^* en proporción a su peso: $P(k) = w_k$.
3. **yield** $\hat{y}_{k^*}$ como la respuesta a este problema.
4. Reciba la respuesta correcta y.
5. Para cada experto k tal que $\hat{y}_k \neq y$, actualiza $w_k \leftarrow \beta w_k$
6. Normalice los pesos para que $\sum_k w_k = 1$.

Aquí β es un número, $0 < \beta < 1$, que indica cuánto penalizar a un experto por cada error.

Arrepentimiento

Medimos el éxito de este algoritmo en términos de **arrepentimiento**, que se define como el número de errores adicionales que cometemos en comparación con el experto que, en retrospectiva, tuvo el mejor registro de predicción. Dejemos que M^* sea el número de errores cometidos por el mejor experto. Entonces el número de errores, M, cometidos por el algoritmo de mayoría ponderada aleatoria, está limitado por[16]

$$M < \frac{M^* \ln(1/\beta) + \ln K}{1 - \beta}.$$

Este límite se mantiene para *cualquier* secuencia de ejemplos, incluso aquellos elegidos por adversarios que intentan hacer lo peor. Para ser específicos, cuando hay $K = 10$ expertos, si elegimos $\beta = 1/2$ entonces nuestro número de errores está limitado por $1.39M^* + 4.6$, y si $\beta = 3/4$ por $1.15M^* + 9.2$. En general, si β está cerca de 1, entonces somos receptivos al cambio a largo plazo; si el mejor experto cambia, lo captaremos antes de mucho tiempo. Sin embargo, pagamos una penalidad al principio, cuando comenzamos con todos los expertos confiados por igual; podemos aceptar el consejo de los malos expertos durante demasiado tiempo. Cuando β está más cerca de 0, estos dos factores se invierten. Observe que podemos elegir β de modo que M se acerque de manera asintótica a M^* a largo plazo; esto se llama **aprendizaje sin arrepentimientos** (porque la cantidad promedio de arrepentimiento por prueba tiende a 0 a medida que aumenta el número de pruebas).

Aprendizaje sin arrepentimientos

El aprendizaje online es útil cuando los datos pueden estar cambiando con el tiempo. También es útil para aplicaciones que involucran una gran colección de datos que está en constante crecimiento, incluso si los cambios son graduales. Por ejemplo, con un conjunto de datos de millones de imágenes web, no querría volver a entrenar desde cero cada vez que se agrega una sola nueva imagen. Sería más práctico tener un algoritmo online que permita agregar imágenes de forma incremental. Para la mayoría de los algoritmos de aprendizaje basados en minimizar la pérdida, existe una versión online basada en minimizar el arrepentimiento. Muchos de estos algoritmos online vienen con límites garantizados en el arrepentimiento.

Parece sorprendente que existan límites tan estrechos sobre cuán bien podemos hacerlo en comparación con un panel de expertos. Lo que es aún más sorprendente es que cuando tales paneles se reúnen para pronosticar sobre concursos políticos o eventos deportivos, el público que los ve está tan dispuesto a escuchar sus predicciones y tan desinteresado en conocer sus tasas de error.

19.9 Desarrollo de sistemas de aprendizaje automático

En este capítulo nos hemos concentrado en explicar la *teoría* del aprendizaje automático. La *práctica* de usar el aprendizaje automático para resolver problemas prácticos es una disciplina separada. Durante los últimos 50 años, la industria del software ha evolucionado una metodología de desarrollo de software que hace más probable que un proyecto de software (tradicional) sea un éxito. Aunque todavía estamos en las primeras etapas de definir una metodología para proyectos de aprendizaje automático, las herramientas y técnicas no están tan desarrolladas. Aquí hay un desglose de los pasos típicos en el proceso.

19.9.1 Formulación del problema

El primer paso es averiguar qué problema quiere resolver. Hay dos partes para esto. Primero pregunte, "¿qué problema quiero resolver para mis usuarios?" Una respuesta como "hacer más fácil para los usuarios organizar y acceder a sus fotos" es demasiado vaga, es mejor "ayudar a un

[16] Blum (1996) da una prueba elegante.

usuario a encontrar todas las fotos que coinciden con un término específico, como *París*". Luego pregunte, "¿qué parte(s) del problema se resuelven con el aprendizaje automático?" quizá decidiendo sobre "aprender una función que mapea una foto a un conjunto de etiquetas; luego, cuando se da una etiqueta como consulta, recuperar todas las fotos con esa etiqueta".

Para hacer esto concreto, necesita especificar una función de pérdida para su componente de aprendizaje automático, quizá midiendo la precisión del sistema al predecir una etiqueta correcta. Este objetivo debería estar correlacionado con sus verdaderos objetivos, pero por lo general será distinto: el verdadero objetivo podría ser maximizar el número de usuarios que gana y mantiene en su sistema, y los ingresos que producen. Esas son métricas que debe seguir, pero no necesariamente las que permite construir de manera directa para un modelo de aprendizaje automático.

Cuando ha descompuesto su problema en partes, puede encontrar que hay múltiples componentes que permiten ser manejados por la ingeniería de software a la antigua, no por el aprendizaje automático. Por ejemplo, para un usuario que pide "las mejores fotos", podría implementar un procedimiento simple que ordene las fotos por el número de vistas y "likes". Una vez que haya desarrollado su sistema general hasta el punto en que sea viable, puede volver y optimizar, reemplazando los componentes simples con modelos de aprendizaje automático más sofisticados.

Parte de la formulación del problema es decidir si está lidiando con aprendizaje supervisado, no supervisado o por refuerzo. Las distinciones no siempre son tan claras. En el **aprendizaje semisupervisado** se dan algunos ejemplos etiquetados y los usamos para extraer más información de una gran colección de ejemplos no etiquetados. Esto se ha convertido en un enfoque común, con empresas emergentes cuya misión es etiquetar de manera rápida algunos ejemplos para ayudar a los sistemas de aprendizaje automático a hacer un mejor uso de los ejemplos no etiquetados restantes.

Aprendizaje semisupervisado

A veces tiene la opción de qué enfoque usar. Considere un sistema para recomendar canciones o películas a los clientes. Podría abordar esto como un problema de aprendizaje supervisado, donde las entradas incluyen una representación del cliente y la salida etiquetada es si les gustó o no la recomendación, o podría abordarlo como un problema de aprendizaje por refuerzo, donde el sistema realiza una serie de acciones de recomendación y, ocasionalmente, recibe una recompensa del cliente por hacer una buena sugerencia.

Las etiquetas en sí mismas pueden no ser las verdades oraculares que esperamos. Imagine que está tratando de construir un sistema para adivinar la edad de una persona a partir de una foto. Recopila algunos ejemplos etiquetados al hacer que las personas suban fotos y declaren su edad. Eso es aprendizaje supervisado. Pero en realidad algunas de las personas mintieron sobre su edad. No es sólo que haya ruido aleatorio en los datos; más bien, las inexactitudes son sistemáticas y descubrirlas es un problema de aprendizaje no supervisado que involucra imágenes, edades autoinformadas y edades verdaderas (desconocidas). Por lo tanto, tanto el ruido como la falta de etiquetas crean un continuo entre el aprendizaje supervisado y no supervisado. El campo de **aprendizaje débilmente supervisado** se centra en el uso de etiquetas ruidosas, imprecisas o suministradas por no expertos.

Aprendizaje débilmente supervisado

19.9.2 Recopilación, evaluación y gestión de datos

Cada proyecto de aprendizaje automático necesita datos; en el caso de nuestro proyecto de identificación de fotos allí hay conjuntos de datos de imágenes disponibles de modo gratuito, como **ImageNet**, que tiene más de 14 millones de fotos con alrededor de 20 000 etiquetas diferentes. A veces podemos tener que fabricar nuestros propios datos, lo que puede hacerse con nuestro propio trabajo o mediante **crowdsourcing** a trabajadores pagados o voluntarios no remunerados que operan mediante un servicio de Internet. A veces los datos provienen de sus usuarios. Por ejemplo, el servicio de navegación Waze anima a los usuarios a subir datos sobre atascos de tráfico y los utiliza para proporcionar direcciones de navegación actualizadas para todos los usuarios. El aprendizaje de

ImageNet

transferencia (ver Sección 21.7.2) se utiliza cuando no tiene suficientes datos propios: comienza con un conjunto de datos de propósito general disponible al público (o un modelo que ha sido preentrenado en estos datos), y luego agrega datos específicos de sus usuarios y vuelve a entrenar.

Si despliega un sistema a los usuarios, estos proporcionarán comentarios, quizá haciendo clic en un elemento e ignorando los demás. Necesitará una estrategia para lidiar con tales datos. Eso implica una revisión con expertos en privacidad (ver Sección 27.3.2) para asegurarse de que obtiene el permiso adecuado para recopilar datos, que tiene procesos para garantizar la integridad de los datos del usuario y que ellos entienden lo que hará con los datos. También necesita asegurarse de que sus procesos son justos y no sesgados (ver Sección 27.3.3). Si hay datos que considera demasiado sensibles para recopilar pero que serían útiles para un modelo de aprendizaje automático, considere un enfoque de aprendizaje federado donde los datos permanecen en el dispositivo del usuario, aunque los parámetros del modelo se comparten de una manera que no revela datos privados.

Procedencia de los datos

Es una buena práctica mantener la **procedencia de los datos** para todos sus datos. Para cada columna en su conjunto de datos, debe conocer la definición exacta, de dónde provienen los datos, cuáles son los posibles valores y quién ha trabajado en ellos. ¿Hubo periodos de tiempo en los que se interrumpió una fuente de datos? ¿La definición de alguna fuente de datos evolucionó con el tiempo? Necesitará saber esto si quiere comparar resultados a lo largo de periodos de tiempo.

Esto es cierto si se basa en datos que son producidos por alguien más, sus necesidades y las suyas pueden divergir, y podrían terminar cambiando la forma en que se producen los datos o podrían dejar de actualizarlos por completo. Necesita monitorear sus fuentes de datos para detectar esto. Tener un *pipeline* de manejo de datos confiable, flexible y seguro es más crítico para el éxito que los detalles exactos del algoritmo de aprendizaje automático. La procedencia también es importante por razones legales, como el cumplimiento de la ley de privacidad.

Para cualquier tarea habrá preguntas sobre los datos: ¿Son estos los datos correctos para mi tarea? ¿Captura suficientes de las entradas correctas para darnos una oportunidad de aprender un modelo? ¿Contiene las salidas que quiero predecir? Si no es así, ¿puedo construir un modelo no supervisado? ¿O puedo etiquetar una parte de los datos y luego hacer un aprendizaje semisupervisado? ¿Son datos relevantes? Es genial tener 14 millones de fotos, pero si todos sus usuarios son especialistas interesados en un tema específico, entonces una base de datos general no ayudará, necesitará recopilar fotos sobre el tema específico. ¿Cuántos datos de entrenamiento son suficientes? (¿Necesito recopilar más datos? ¿Puedo descartar algunos datos para hacer el cálculo más rápido?) La mejor manera de responder a esto es razonar por analogía a un proyecto similar con un tamaño de conjunto de entrenamiento conocido.

Una vez que comience, permite trazar una curva de aprendizaje (ver Figura 19.7) para ver si más datos ayudarán o si el aprendizaje ya ha alcanzado un punto de meseta. Hay innumerables reglas empíricas injustificadas para el número de ejemplos de entrenamiento que necesitará: millones para problemas difíciles, miles para problemas promedio, cientos o miles para cada clase en un problema de clasificación, 10 veces más ejemplos que parámetros del modelo, 10 veces más ejemplos que características de entrada, $O(d \log d)$ ejemplos para d características de entrada, más ejemplos para modelos no lineales que para modelos lineales, más ejemplos si se requiere mayor precisión, menos ejemplos si usa regularización, suficientes ejemplos para alcanzar el poder estadístico necesario para rechazar la hipótesis nula en la clasificación. Todas estas reglas vienen con advertencias, al igual que la regla sensata que sugiere probar lo que ha funcionado en el pasado para problemas similares.

Debe pensar de manera defensiva sobre sus datos. ¿Podría haber errores de entrada de datos? ¿Qué se puede hacer con los campos de datos faltantes? Si recopila datos de sus clientes (u otras personas), ¿podrían algunas de las personas ser adversarios que intentan manipular el sistema? ¿Hay errores de ortografía o terminología inconsistente en los datos de texto? (Por ejemplo, ¿"Apple", "AAPL" y "Apple Inc." se refieren a la misma compañía?) Necesitará un proceso para detectar y corregir todas estas posibles fuentes de error de datos.

Cuando los datos son limitados, la **ampliación de datos** ayuda. Por ejemplo, con un conjunto de datos de imágenes permite crear múltiples versiones de cada imagen rotando, trasladando, recortando o escalando cada imagen, o cambiando el brillo o el balance de color o añadiendo ruido. Siempre que estos sean cambios pequeños, la etiqueta de la imagen debería permanecer igual, y un modelo entrenado en tales datos aumentados será más robusto.

Ampliación de datos

A veces los datos son abundantes pero están clasificados en **clases desequilibradas**. Por ejemplo, un conjunto de entrenamiento de transacciones con tarjeta de crédito podría consistir en 10 000 000 de transacciones válidas y 1 000 fraudulentas. Un clasificador que dice "válido" independiente de la entrada alcanzará una precisión de 99.99 % en este conjunto de datos. Para ir más allá de eso, un clasificador tendrá que prestar más atención a los ejemplos fraudulentos. Para ayudarlo a hacer eso, es posible **submuestrear** la clase mayoritaria (es decir, ignorar algunos de los ejemplos de la clase "válida") o **sobremuestrear** la clase minoritaria (es decir, duplicar algunos de los ejemplos de la clase "fraudulenta"). Puede usar una función de pérdida ponderada que da una penalización mayor a perder un caso fraudulento.

Clases desequilibradas

Submuestrear

Sobremuestrear

El boosting también ayuda a centrarse en la clase minoritaria. Si está utilizando un método de conjunto, puede cambiar las reglas por las que el conjunto vota y dar "fraudulento" como respuesta incluso si sólo una minoría del conjunto vota por "fraudulento". Esto ayuda a equilibrar clases desequilibradas generando datos sintéticos con técnicas como SMOTE (Chawla *et al.*, 2002) o ADASYN (He *et al.*, 2008).

Debe considerar los **valores atípicos** en sus datos. Un valor atípico es un punto de datos que está lejos de otros puntos. Por ejemplo, en el problema del restaurante, si el precio fuera un valor numérico en lugar de uno categórico, y si un ejemplo tuviera un precio de $316 mientras que todos los demás fueran $30 o menos, ese ejemplo sería un valor atípico. Métodos como la regresión lineal son susceptibles a los valores atípicos porque deben formar un único modelo lineal global que tenga en cuenta todas las entradas, no permiten tratar al valor atípico de manera diferente a otros puntos de ejemplo y, por lo tanto, un único valor atípico puede tener un gran efecto en todos los parámetros del modelo.

Valores atípicos

Con atributos como el precio que son números positivos, podemos disminuir el efecto de los valores atípicos transformando los datos, tomando el logaritmo de cada valor, por lo que $20, $25 y $316 se convierten en 1.3, 1.4 y 2.5. Esto tiene sentido desde un punto de vista práctico, porque el valor alto ahora tiene menos influencia en el modelo, y desde un punto de vista teórico, porque, como vimos en la Sección 16.3.2, la utilidad del dinero es logarítmica.

Métodos como los árboles de decisión que se construyen a partir de múltiples modelos locales pueden tratar los valores atípicos de manera individual: no importa si el valor más grande es $300 o $31, de cualquier manera permite ser tratado en su propio nodo local después de una prueba de la forma $cost \leq 30$. Eso hace que los árboles de decisión (y, por lo tanto, los bosques aleatorios y el aumento de gradiente) sean más robustos a los valores atípicos.

Ingeniería de características

Después de corregir errores evidentes, también es posible preprocesar sus datos para facilitar su análisis. Ya hemos visto el proceso de cuantificación: forzar una entrada de valor continuo, como el tiempo de espera, en contenedores fijos (0–10 minutos, 10–30, 30–60, o >60). El conocimiento del dominio indica qué umbrales son importantes, como comparar la $edad \geq 18$ cuando se estudian los patrones de votación. También vimos (página 688) que los algoritmos de vecinos más cercanos funcionan mejor cuando los datos se normalizan para tener una desviación estándar de 1. Con atributos categóricos como soleado/nublado/lluvioso, en la mayoría de los casos es útil transformar los datos en tres atributos booleanos separados, uno de los cuales es verdadero (a esto lo llamamos **codificación one-hot**). Esto es en particular útil cuando el modelo de aprendizaje automático es una red neuronal.

Codificación one-hot

También permite introducir nuevos atributos basados en su conocimiento del dominio. Por ejemplo, dado un conjunto de datos de compras de clientes donde cada entrada tiene un atributo de fecha es posible que desee aumentar los datos con nuevos atributos que indiquen si la fecha es un fin de semana o un día festivo.

En otro ejemplo, considere la tarea de estimar el valor real de las casas que están en venta. En la Figura 19.13 mostramos una versión de maqueta de este problema, haciendo una regresión lineal del tamaño de la casa al precio de venta. Aunque realmente queremos estimar el precio en que se vende una casa, no el precio de venta que se pide. Para resolver esta tarea necesitaremos datos sobre ventas reales. Pero eso no significa que debamos descartar los datos sobre el precio de venta, podemos usarlo como una de las características de entrada. Además del tamaño de la casa, necesitaremos más información: el número de habitaciones, dormitorios y baños; si la cocina y los baños han sido remodelados recientemente; la edad de la casa y quizá su estado de conservación; si tiene calefacción y aire acondicionado central; el tamaño del patio y el estado de la fachada.

También necesitaremos información sobre el lote y el vecindario. Pero ¿cómo definimos el vecindario? ¿Por código postal? ¿Qué pasa si un código postal abarca un vecindario deseable y uno indeseable? ¿Qué pasa con el distrito escolar? ¿Debería ser el *nombre* del distrito escolar una característica o los *puntajes promedio de las pruebas*? La capacidad de hacer un buen trabajo de ingeniería de características es crítica para el éxito. Como dice Pedro Domingos (2012), "Al final del día, algunos proyectos de aprendizaje automático tienen éxito y otros fracasan. ¿Qué hace la diferencia? El factor más importante son las características utilizadas".

Análisis de datos exploratorio y visualización

John Tukey (1977) acuñó el término **análisis de datos exploratorio** (EDA) para el proceso de explorar datos con el fin de comprenderlos, no para hacer predicciones o probar hipótesis. Esto se hace con visualizaciones, pero también con estadísticas resumidas. Mirar algunos histogramas o grafos de dispersión ayuda a determinar si faltan datos o son erróneos, si sus datos están normalmente distribuidos o tienen colas pesadas y qué modelo de aprendizaje podría ser apropiado.

Puede ser útil agrupar sus datos y luego visualizar un punto de datos prototipo en el centro de cada grupo. Por ejemplo, en el conjunto de datos de imágenes, puede identificar que aquí hay un grupo de caras de gatos, cerca hay un grupo de gatos durmiendo; otros grupos representan otros objetos. Espere iterar varias veces entre la visualización y el modelado: para crear grupos necesita una función de distancia para decirle qué elementos están cerca uno del otro, pero para elegir una buena función de distancia necesita tener una idea de los datos.

También es útil detectar valores atípicos que están lejos de los prototipos, estos valores pueden ser considerados **críticos** del modelo prototipo y dar una idea de qué tipo de errores podría cometer su sistema. Un ejemplo sería un gato vistiendo un disfraz de león.

Nuestros dispositivos de visualización de computadoras (pantallas o papel) son bidimensionales, lo que significa que es fácil visualizar datos bidimensionales. Y nuestros ojos están acostumbrados a entender datos tridimensionales que se han proyectado en dos dimensiones. Pero muchos conjuntos de datos tienen docenas o incluso millones de dimensiones. Para visualizarlos podemos hacer una reducción de dimensionalidad, proyectando los datos a un mapa en dos dimensiones (o a veces a tres dimensiones, que luego permiten explorarse de forma interactiva).[17]

[17] Geoffrey Hinton proporciona el consejo útil "Para lidiar con un espacio de 14 dimensiones, visualice un espacio de 3D y diga 'catorce' en voz alta".

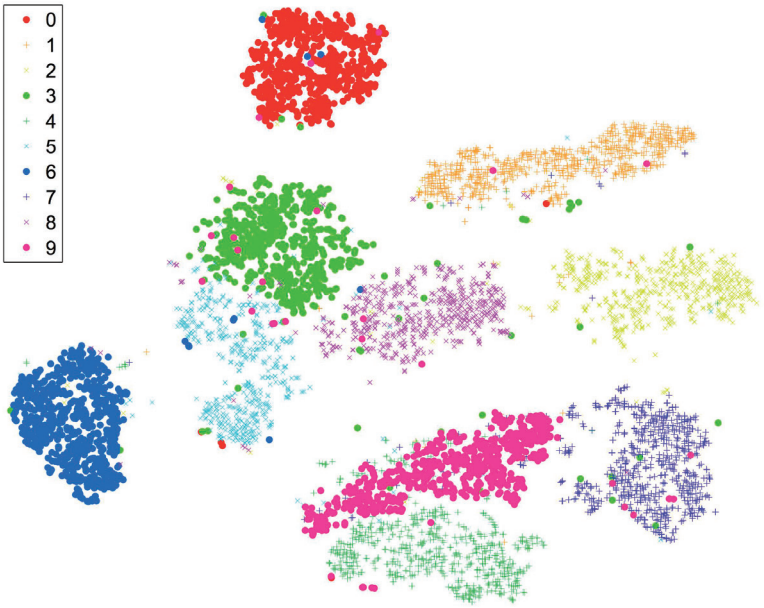


Figura 19.27 Un mapa t-SNE bidimensional del conjunto de datos MNIST, una colección de 60 000 imágenes de dígitos escritos a mano, cada una de 28 × 28 pixeles y, por lo tanto, 784 dimensiones. Existen grupos para los diez dígitos, con algunas confusiones en cada grupo; por ejemplo, el grupo superior es para el dígito 0, pero dentro de los límites del grupo hay algunos puntos de datos que representan los dígitos 3 y 6. El algoritmo t-SNE encuentra una representación que acentúa las diferencias entre los grupos.

El mapa no permite mantener todas las relaciones entre los puntos de datos, pero debería tener la propiedad de que los puntos similares en el conjunto de datos original están cerca en el mapa. Una técnica llamada **incrustación de vecinos estocásticos distribuidos en t (t-SNE)** hace precisamente eso. La Figura 19.27 muestra un mapa t-SNE del conjunto de datos de reconocimiento de dígitos MNIST. Los paquetes de análisis y visualización de datos como Pandas, Bokeh y Tableau pueden facilitar el trabajo con sus datos.

Incrustación de vecinos estocásticos distribuidos en t (t-SNE)

19.9.3 Selección y entrenamiento del modelo

Con los datos limpios en la mano y una sensación intuitiva de ellos, es hora de construir un modelo. Eso significa elegir una clase de modelo (¿bosques aleatorios? ¿redes neuronales profundas? ¿un conjunto?), entrenar su modelo con los datos de entrenamiento, ajustar cualquier hiperparámetro de la clase (¿número de árboles? ¿número de capas?) con los datos de validación, depurar el proceso y, por último, evaluar el modelo en los datos de prueba.

No hay una forma garantizada de elegir la mejor clase de modelo, pero hay algunas pautas generales. Los bosques aleatorios son buenos cuando hay muchas características categóricas y cree que muchas de ellas pueden ser irrelevantes. Los métodos no paramétricos son buenos cuando tiene muchos datos y ningún conocimiento previo, y cuando no quiere preocuparse demasiado por elegir las características correctas (siempre que haya menos de 20 o así). Sin embargo, los métodos no paramétricos en general le dan una función h que es más costosa de ejecutar.

La regresión logística funciona bien cuando los datos son linealmente separables o se pueden convertir para que lo sean mediante una ingeniería de características ingeniosa. Las máquinas de soporte vectorial son un buen método para probar cuando el conjunto de datos no es demasiado grande, tienen un rendimiento similar al de la regresión logística en datos separables y pueden ser mejores para datos de alta dimensionalidad. Los problemas relacionados con el reconocimiento de patrones, como el procesamiento de imágenes o voz, suelen abordarse con redes neuronales profundas (ver Capítulo 21).

La elección de hiperparámetros se puede hacer con una combinación de experiencia, haciendo lo que ha funcionado bien en problemas similares anteriores, y búsqueda: realizar experimentos con múltiples valores posibles para los hiperparámetros. A medida que se realizan más experimentos, se obtendrán ideas para probar diferentes modelos. Sin embargo, si se mide el rendimiento en los datos de validación, se obtiene una nueva idea y se realizan más experimentos, entonces existe el riesgo de sobreajustar los datos de validación. Si se dispone de suficientes datos, es posible que se quiera tener varios conjuntos de datos de validación separados para evitar este problema. Esto es especialmente cierto si se inspeccionan los datos de validación visualmente, en lugar de sólo ejecutar evaluaciones sobre ellos.

Suponga que se está construyendo un clasificador, por ejemplo, un sistema para clasificar correos electrónicos como spam. Etiquetar un correo legítimo como spam se denomina un **falso positivo**. Habrá un compromiso entre falsos positivos y falsos negativos (etiquetar un correo de spam como legítimo); si se quiere mantener más correos legítimos fuera de la carpeta de spam, necesariamente se terminará enviando más spam a la bandeja de entrada. Pero ¿cuál es la mejor manera de hacer este compromiso? Se pueden probar diferentes valores de hiperparámetros y obtener diferentes tasas para los dos tipos de errores, diferentes puntos en este compromiso. Un grafo llamado **curva de característica operativa del receptor** (**ROC**, por sus siglas en inglés) traza falsos positivos frente a verdaderos positivos para cada valor del hiperparámetro, ayudando a visualizar los valores que serían buenas opciones para el compromiso. Una métrica llamada "área bajo la curva ROC" o **AUC** proporciona un resumen de un solo número de la curva ROC, lo cual es útil si se desea desplegar un sistema y permitir que cada usuario elija su punto de compromiso.

Falso positivo

Curva de característica operativa del receptor (ROC)

AUC

Otra útil herramienta de visualización para problemas de clasificación es una **matriz de confusión**: una tabla bidimensional de recuentos de cuántas veces se clasifica o se clasifica incorrectamente cada categoría como otra categoría.

Matriz de confusión

Pueden existir compromisos en factores distintos a la función de pérdida. Si se puede entrenar un modelo de predicción del mercado de valores que genere $10 en cada operación, eso es excelente, pero no si cuesta $20 en costos de computación por cada predicción. Un programa de traducción automática que se ejecuta en un teléfono y permite leer letreros en una ciudad extranjera es útil, pero no si agota la batería después de una hora de uso. Es importante tener en cuenta todos los factores que conducen a la aceptación o el rechazo del sistema y diseñar un proceso en el que se pueda iterar rápidamente el proceso de obtener una nueva idea, realizar un experimento y evaluar los resultados del experimento para ver si se ha logrado algún progreso. Hacer que este proceso de iteración sea rápido es uno de los factores más importantes para el éxito en el aprendizaje automático.

19.9.4 Confianza, interpretabilidad y explicabilidad

Se ha descrito una metodología de aprendizaje automático donde se desarrolla el modelo con datos de entrenamiento, se eligen hiperparámetros con datos de validación y se obtiene una métrica final con datos de prueba. Tener un buen desempeño en esa métrica es una condición necesaria pero no suficiente para **confiar** en el modelo. Y no se trata sólo de uno mismo, otros interesados, incluidos reguladores, legisladores, la prensa y los usuarios también están interesados en la confiabilidad del sistema (así como en atributos relacionados como la fiabilidad, la responsabilidad y la seguridad).

Un sistema de aprendizaje automático sigue siendo un software y se puede generar confianza con todas las herramientas típicas para verificar y validar cualquier sistema de software:

- **Control de versiones**: Sistemas para el control de versiones, compilación y seguimiento de errores/incidentes.
- **Pruebas**: Pruebas unitarias para todos los componentes que cubran casos canónicos simples, así como casos adversariales complicados, pruebas de fuzzing (donde se generan entradas aleatorias), pruebas de regresión, pruebas de carga y pruebas de integración de sistemas: todas son importantes para cualquier sistema de software. Para el aprendizaje automático, también se tienen pruebas en los conjuntos de datos de entrenamiento, validación y prueba.
- **Revisión**: Revisión y análisis del código, revisiones de privacidad, revisiones de equidad (ver Sección 27.3.3), y otras revisiones de cumplimiento legal.
- **Monitoreo**: Tableros y alertas para asegurarse de que el sistema esté en funcionamiento y continúe teniendo un alto nivel de precisión.
- **Responsabilidad**: ¿Qué sucede cuando el sistema se equivoca? ¿Cuál es el proceso para presentar una queja o apelar la decisión del sistema? ¿Cómo se puede rastrear quién fue responsable del error? La sociedad espera (aunque no siempre obtiene) responsabilidad en decisiones importantes tomadas por bancos, políticos y la ley, y deberían esperar responsabilidad de los sistemas de software, incluidos los sistemas de aprendizaje automático.

Además, existen algunos factores que son especialmente importantes para los sistemas de aprendizaje automático, como a continuación se detalla.

Interpretabilidad

Interpretabilidad: Se dice que un modelo de aprendizaje automático es **interpretable** si se puede inspeccionar el modelo real y comprender por qué obtuvo una respuesta particular para una entrada dada, y cómo cambiaría la respuesta si la entrada cambiara.[18] Los modelos de árboles de decisión se consideran altamente interpretables; se puede entender que seguir el camino *Clientes=Lleno* y *Estimación espera*=0–10 en un árbol de decisión lleva a una decisión de *esperar*. Un árbol de decisión es interpretable por dos razones. Primero, los humanos tienen experiencia en comprender reglas IF/THEN. (En contraste, es muy difícil para los humanos obtener una comprensión intuitiva del resultado de una multiplicación de matrices seguida de una función de activación, como se hace en algunos modelos de redes neuronales). En segundo lugar, el árbol de decisión fue, en cierto sentido, construido para ser interpretable: la raíz del árbol se eligió para ser el atributo con la mayor ganancia de información.

Los modelos de regresión lineal también se consideran interpretables; es posible examinar un modelo para predecir el alquiler de un apartamento y ver que por cada habitación adicional, el alquiler aumenta en $500, según el modelo. Esta idea de "Si cambio x, ¿cómo cambiará la salida?" está en el núcleo de la interpretabilidad. Por supuesto, la correlación no es causalidad, por lo que los modelos interpretables responden a lo *qu*é es el caso, pero no necesariamente a *por qué* es el caso.

Explicabilidad

Explicabilidad: Un modelo explicable es aquel que puede ayudar a entender "¿*por qué* se produjo esta salida para esta entrada?" En nuestra terminología, la interpretabilidad deriva de inspeccionar el modelo real, mientras que la explicabilidad puede ser proporcionada por un proceso separado. Es decir, el modelo en sí puede ser una caja negra difícil de entender, pero un módulo de explicación puede resumir lo que hace el modelo. Para un sistema de reconocimiento de imágenes con redes neuronales que clasifica una imagen como un *perro*, si se intentara interpretar el modelo directamente, lo mejor que se podría obtener sería algo como "después de procesar las capas convolucionales, la activación para la salida de *perro* en la capa softmax fue mayor que en

[18] Esta terminología no es universalmente aceptada; algunos autores utilizan "interpretable" y "explicable" como sinónimos, ambos haciendo referencia a alcanzar algún tipo de comprensión de un modelo.

cualquier otra clase". Eso no es un argumento muy convincente. Pero un módulo de explicación separado podría ser capaz de examinar el modelo de red neuronal y llegar a la explicación "tiene cuatro patas, pelo, una cola, orejas caídas y un hocico largo; es más pequeño que un lobo y está acostado en una cama de perro, así que creo que es un perro". Las explicaciones son una forma de construir confianza, y algunas regulaciones como el GDPR europeo (Reglamento General de Protección de Datos) requieren que los sistemas proporcionen explicaciones.

Como ejemplo de un módulo de explicación separado, el sistema de explicaciones interpretables locales del modelo agnóstico (LIME) funciona así: no importa qué clase de modelo use, LIME construye un modelo interpretable, con frecuencia un árbol de decisión o un modelo lineal, que es una aproximación de su modelo, y luego interpreta el modelo lineal para crear explicaciones que digan cuán importante es cada característica. LIME logra esto tratando el modelo de aprendizaje automático como una caja negra, y sondeándolo con diferentes valores de entrada aleatorios para crear un conjunto de datos a partir del cual se permite construir el modelo interpretable. Este enfoque es apropiado para datos estructurados, pero no para cosas como imágenes, donde cada pixel es una característica, y ningún pixel es "importante" por sí mismo. A veces elegimos una clase de modelo por su explicabilidad, podríamos elegir árboles de decisión sobre redes neuronales no porque tengan una mayor precisión sino porque la explicabilidad nos da más confianza en ellos.

Sin embargo, una explicación simple puede llevar a una falsa sensación de seguridad. Después de todo, en la mayoría de los casos se elige un modelo de aprendizaje automático (en lugar de un programa tradicional escrito a mano) porque el problema que estamos tratando de resolver es complejo, y no sabemos cómo escribir un programa tradicional. En ese caso, no deberíamos esperar que necesariamente haya una explicación simple para cada predicción.

Si está construyendo un modelo de aprendizaje automático con el propósito de entender el dominio, entonces la interpretabilidad y la explicabilidad le ayudarán a llegar a ese entendimiento. Pero si sólo quiere el mejor software de rendimiento, entonces las pruebas pueden darle más confianza que las explicaciones. ¿En cuál confiaría: en un avión experimental que nunca ha volado antes pero que tiene una explicación detallada de por qué es seguro o en un avión que ha completado con seguridad 100 vuelos anteriores y ha recibido mantenimiento pero que no viene con ninguna explicación garantizada?

19.9.5 Operación, monitoreo y mantenimiento

Una vez que esté contento con el rendimiento de su modelo, puede desplegarlo a sus usuarios. Se enfrentará a desafíos adicionales. Primero, está el problema de la **cola larga** de entradas de usuario; puede haber probado su sistema en un gran conjunto de pruebas, pero si su sistema es popular, pronto verá entradas que nunca habían sido probadas; necesita saber si su modelo se generaliza bien para ellos, lo que significa que necesita **monitorear** su rendimiento en datos en vivo, rastreando estadísticas, mostrando un tablero de control y enviando alertas cuando las métricas clave caen por debajo de un umbral. Además de actualizar de manera automática las estadísticas sobre las interacciones de los usuarios, es posible que necesite contratar y entrenar evaluadores humanos para que miren su sistema y califiquen lo bien que está funcionando.

Cola larga

Monitoreo

No estacionariedad

En segundo lugar, está el problema de la **no estacionariedad**, el mundo cambia con el tiempo. Suponga que su sistema clasifica el correo electrónico como spam o no spam. Tan pronto como clasifique con éxito un lote de mensajes de spam, los spammers verán lo que ha hecho y cambiarán sus tácticas, enviando un nuevo tipo de mensaje que no ha visto antes. El no spam también evoluciona, ya que los usuarios cambian la mezcla de correo electrónico contra su mensajería o servicios de escritorio contra los móviles que utilizan.

Continuamente se enfrentará a la pregunta de qué es mejor: un modelo que ha sido bien probado pero fue construido a partir de datos más antiguos, contra un modelo que se construye a partir de los

Pruebas para características y datos

(1) Las expectativas de las características están capturadas en un esquema. (2) Todas las características son benéficas. (3) Ningún costo de características es demasiado. (4) Las características se adhieren a los requisitos de nivel meta. (5) El pipeline (flujo) de datos tiene controles de privacidad apropiados. (6) Es posible añadir de manera rápida nuevas características. (7) Todo el código de características de entrada está probado.

Pruebas para el desarrollo del modelo

(1) Cada especificación del modelo pasa por una revisión de código. (2) Cada modelo se registra en un repositorio. (3) Las métricas proxy offline se correlacionan con las métricas reales. (4) Todos los hiperparámetros han sido ajustados. (5) Se conoce el impacto de la obsolescencia del modelo. (6) Un modelo más simple no es mejor. (7) La calidad del modelo es suficiente en todas las porciones de datos importantes. El modelo ha sido probado para consideraciones de inclusión.

Pruebas para la infraestructura de aprendizaje automático

(1) La formación es reproducible. (2) El código de especificación del modelo está probado en la unidad. (3) La tubería completa de ML está probada en la integración. (4) La calidad del modelo se valida antes de intentar servirlo. (5) El modelo permite la depuración observando el cálculo paso a paso de la formación o la inferencia en un solo ejemplo. (6) Los modelos son probados a través de un proceso de canario antes de que entren en los entornos de producción de servicio. (7) Los modelos permiten ser rápida y seguramente revertidos a una versión de servicio anterior.

Pruebas de monitoreo para el aprendizaje automático

(1) Los cambios de dependencia resultan en notificación. (2) Las invariantes de datos se mantienen en las entradas de entrenamiento y servicio. (3) Las características de entrenamiento y servicio calculan los mismos valores. (4) Los modelos no están demasiado obsoletos. (5) El modelo es numéricamente estable. (6) El modelo no ha experimentado regresiones en la velocidad de entrenamiento, la latencia de servicio, el rendimiento, o el uso de RAM. (7) El modelo no ha experimentado una regresión en la calidad de la predicción en los datos servidos.

Figura 19.28 Un conjunto de criterios para ver cómo lo estás haciendo al desplegar su modelo de aprendizaje automático con pruebas suficientes. Abreviado de Breck *et al.* (2016), quienes también proporcionan una métrica de puntuación.

últimos datos pero no ha sido probado en uso real. Diferentes sistemas tienen distintos requisitos de frescura: algunos problemas se benefician de un nuevo modelo cada día, o incluso cada hora, mientras que otros problemas permiten mantener el mismo modelo durante meses. Si está implementando un nuevo modelo cada hora, será impráctico ejecutar un conjunto de pruebas y un proceso de revisión manual para cada actualización. Deberá automatizar el proceso de prueba y lanzamiento para que los cambios pequeños puedan ser aprobados de manera automática, pero los cambios más grandes desencadenen una revisión apropiada. Puede considerar el equilibrio entre un modelo online donde los nuevos datos modifican el modelo existente, frente a un modelo offline donde cada nuevo lanzamiento requiere construir un nuevo modelo desde cero.

No es sólo que los datos cambiarán, por ejemplo, se usarán nuevas palabras en los mensajes de correo electrónico no deseados. También es que todo el esquema de datos puede cambiar: podría comenzar clasificando correo electrónico no deseado y necesitar adaptarse para clasificar mensajes de texto no deseados, mensajes de voz no deseados, videos no deseados, etc. La Figura 19.28 proporciona una rúbrica general para guiar al profesional en la elección del nivel apropiado de pruebas y monitoreo.

Resumen

Este capítulo introdujo el aprendizaje automático y se centró en el aprendizaje supervisado mediante ejemplos. Los puntos principales fueron:

- El aprendizaje toma muchas formas, dependiendo de la naturaleza del agente, el componente a mejorar y la retroalimentación disponible.
- Si la retroalimentación disponible proporciona la respuesta correcta para los ejemplos de entrada, entonces el problema de aprendizaje se llama **aprendizaje supervisado**. La tarea es aprender una función $y = h(x)$. Aprender una función cuya salida es un valor continuo u ordenado (como el *peso*) se llama **regresión**; aprender una función con un pequeño número de posibles categorías de salida se llama **clasificación.**
- Queremos aprender una función que no sólo esté de acuerdo con los datos sino que también sea probable que esté de acuerdo con los datos futuros. Necesitamos equilibrar el acuerdo con los datos contra la simplicidad de la hipótesis.
- Los **árboles de decisión** permiten representar todas las funciones booleanas. La heurística de **ganancia de información** proporciona un método eficiente para encontrar un árbol de decisión simple y consistente.
- El rendimiento de un algoritmo de aprendizaje se visualiza mediante una **curva de aprendizaje**, que muestra la precisión de la predicción en el **conjunto de prueba** en función del tamaño del **conjunto de entrenamiento**.
- Cuando hay varios modelos para elegir, la **selección de modelos** permite elegir buenos valores de hiperparámetros, según lo confirmado por la **validación cruzada** en los datos de validación. Una vez que se eligen los valores de los hiperparámetros, construimos nuestro mejor modelo utilizando todos los datos de entrenamiento.
- A veces no todos los errores son iguales. Una **función de pérdida** nos dice cuán malo es cada error; el objetivo, entonces, es minimizar la pérdida en un conjunto de validación.
- La **teoría del aprendizaje computacional** analiza la complejidad de la muestra y la complejidad computacional del aprendizaje inductivo. Hay un equilibrio entre la expresividad del espacio de hipótesis y la facilidad de aprendizaje.
- La **regresión lineal** es un modelo ampliamente utilizado. Los parámetros óptimos de un modelo de regresión lineal se calculan de manera exacta o se encuentran mediante una búsqueda de descenso de gradiente, que es una técnica que se aplica a modelos que no tienen una solución de forma cerrada.
- Un clasificador lineal con un umbral duro, también conocido como **perceptrón**, permite ser entrenado por una simple regla de actualización de peso para ajustar los datos que son **linealmente separables**. En otros casos, la regla no converge.
- La **regresión logística** reemplaza el umbral difícil del perceptrón con un umbral suave definido por una función logística. El descenso de gradiente funciona bien incluso para datos ruidosos que no son linealmente separables.
- Los **modelos no paramétricos** utilizan todos los datos para hacer cada predicción, en lugar de intentar resumir los datos con unos pocos parámetros. Los ejemplos incluyen los **vecinos más cercanos** y la **regresión ponderada localmente**.
- Las **máquinas de soporte vectorial** encuentran separadores lineales con **margen máximo** para mejorar el rendimiento de generalización del clasificador. Los **métodos de kernel** transforman de manera implícita los datos de entrada en un espacio de alta dimensión donde puede existir un separador lineal, incluso si los datos originales no son separables.
- Los métodos de conjunto como el **bagging** y el **boosting** con frecuencia funcionan mejor que los métodos individuales. En el **aprendizaje online** podemos agregar las opiniones de

los expertos para acercarnos de manera arbitraria al rendimiento del mejor experto, aun cuando la distribución de los datos cambia de manera constante.
- Construir un buen modelo de aprendizaje automático requiere experiencia en todo el proceso de desarrollo, desde la gestión de datos hasta la selección y optimización del modelo, hasta el mantenimiento continuo.

Notas bibliográficas e históricas

El Capítulo 1 cubrió la historia de las investigaciones filosóficas sobre el tema del aprendizaje inductivo. A Guillermo de Occam (1280-1349), el filósofo más influyente de su siglo y un importante contribuyente a la epistemología, lógica y metafísica medieval, se le atribuye la declaración latina: *Entia non sunt multiplicanda praeter necessitatem*, que traducida al español dicta: "Las entidades no deben multiplicarse más allá de la necesidad", la cual, con el paso del tiempo, sería llamada "navaja de Occam". No obstante, este loable consejo no se encuentra en sus escritos en estas palabras exactas (aunque sí dijo *Pluralitas non est ponenda sine necessitate* ("La pluralidad no debe ser puesta sin necesidad"). Un sentimiento similar fue expresado antes por Aristóteles en el año 350 a.C. en su *Física*, libro I, capítulo VI: "Siempre es preferible lo más limitado, si es adecuado".

Por su parte, David Hume (1711-1776) formuló el *problema de la inducción*, reconociendo que generalizar a partir de ejemplos admite la posibilidad de errores, de una manera que la deducción lógica no lo hace. Él vio que no había forma de tener una solución garantizada correcta al problema, pero propuso el principio de *uniformidad de la naturaleza*, que hemos llamado *estacionariedad*. Lo que Occam y Hume estaban diciendo es que cuando hacemos inducción, estamos eligiendo de la multitud de modelos consistentes uno que es más probable, porque es más simple y coincide con nuestras expectativas. En la actualidad, el teorema de *no almuerzo gratis* (Wolpert y Macready, 1997; Wolpert, 2013) dice que si un algoritmo de aprendizaje funciona bien en un conjunto de problemas, es sólo porque funcionará mal en un conjunto diferente: si nuestro árbol de decisión predice de manera correcta el comportamiento de espera del restaurante de SR, debe funcionar mal para alguna otra persona hipotética que tenga el comportamiento de espera opuesto en las entradas no observadas.

El aprendizaje automático fue una de las ideas clave en el nacimiento de la informática. Alan Turing (1947) lo anticipó al decir "Suponga que hemos configurado una máquina con ciertas tablas de instrucciones iniciales, construidas de tal modo que estas tablas podrían, en ocasiones y si surgiera una buena razón, modificar esas tablas". Arthur Samuel (1959) definió el aprendizaje automático como el "campo de estudio que da a las computadoras la capacidad de aprender sin ser programadas de manera explícita" mientras creaba su programa de aprendizaje de damas.

El primer uso notable de los **árboles de decisión** fue en EPAM, el "Elementary Perceiver And Memorizer" (Feigenbaum, 1961), que era una simulación del aprendizaje de conceptos humanos. ID3 (Quinlan, 1979) añadió la idea crucial de elegir el atributo con máxima entropía. Los conceptos de entropía y teoría de la información fueron desarrollados por Claude Shannon para ayudar en el estudio de la comunicación (Shannon y Weaver, 1949). (Shannon también contribuyó con uno de los primeros ejemplos de aprendizaje automático, un ratón mecánico llamado Teseo que aprendió a navegar por un laberinto mediante ensayo y error). El método χ^2 de poda de árboles fue descrito por Quinlan (1986). Una descripción de C4.5, un paquete de árboles de decisión de fuerza industrial, puede encontrarse en Quinlan (1993). Un paquete de software alternativo de fuerza industrial, CART (para *Classification and Regression Trees*) fue desarrollado por el estadístico Leo Breiman y sus colegas (Breiman *et al*., 1984).

Hyafil y Rivest (1976) demostraron que encontrar un árbol de decisión óptimo (en lugar de encontrar un buen árbol a través de selecciones localmente codiciosas) es NP-completo. Pero Bertsimas y Dunn (2017) señalan que en los últimos 25 años, los avances en el diseño de hardware y en los algoritmos para la programación de enteros mixtos han resultado en una aceleración de 800 mil millones de veces, lo que significa que ahora es factible resolver este problema NP-difícil al menos para problemas con no más de unos pocos miles de ejemplos y unas pocas docenas de características.

La **validación cruzada** fue introducida por primera vez por Larson (1931) y, en una forma cercana a lo que mostramos, por Stone (1974) y Golub *et al*. (1979). El procedimiento de regularización se debe a Tikhonov (1963).

Sobre la cuestión del sobreajuste, John von Neumann fue citado (Dyson, 2004) alardeando, "Con cuatro parámetros puedo ajustar un elefante y con cinco puedo hacer que mueva su trompa", lo que significa que un polinomio de alto grado puede ser hecho para ajustar casi cualquier dato, pero con el costo potencial de sobreajustar. Mayer *et al.* (2010) le demostraron que tenía razón al demostrar un elefante de cuatro parámetros y un movimiento de cinco parámetros, y Boué (2019) fue aún más lejos, demostrando un elefante y otros animales con una función caótica de un parámetro.

Zhang *et al*. (2016) analizan bajo qué condiciones un modelo permite memorizar los datos de entrenamiento. Realizan experimentos utilizando datos aleatorios, seguramente un algoritmo que obtiene un error cero en un conjunto de entrenamiento con etiquetas aleatorias debe estar memorizando el conjunto de datos. Sin embargo, concluyen que el campo aún tiene que descubrir una medida precisa de lo que significa que un modelo sea "simple" en el sentido de la navaja de Occam. Arpit *et al*. (2017) muestran que las condiciones bajo las cuales puede ocurrir la memorización dependen de los detalles tanto del modelo como del conjunto de datos.

Belkin *et al*. (2019) discuten el compromiso entre sesgo y varianza en el aprendizaje automático y por qué algunas clases de modelos continúan mejorando después de alcanzar el punto de interpolación, mientras que otras clases de modelos exhiben la curva en forma de U. Berrada *et al*. (2019) desarrollan un nuevo algoritmo de aprendizaje basado en el descenso de gradiente que explota la capacidad de los modelos para memorizar y establecer buenos valores para el hiperparámetro de la tasa de aprendizaje.

El análisis teórico de los algoritmos de aprendizaje comenzó con el trabajo de Gold (1967) sobre la **identificación en el límite**. Este enfoque fue motivado en parte por modelos de descubrimiento científico de la filosofía de la ciencia (Popper, 1962), pero ha sido aplicado al problema de aprender gramáticas a partir de oraciones de ejemplo (Osherson *et al*., 1986).

Complejidad de Kolmogorov

Mientras que el enfoque de identificación en el límite se concentra en la convergencia eventual, el estudio de la **complejidad de Kolmogorov** o **complejidad algorítmica**, desarrollados de manera independiente por Solomonoff (1964, 2009) y Kolmogorov (1965), intenta proporcionar una definición formal para la noción de simplicidad utilizada en la navaja de Occam. Para escapar del problema de que la simplicidad depende de la forma en que se representa la información, se propone que la simplicidad se mida por la longitud del programa más corto para una máquina de Turing universal que reproduce de manera correcta los datos observados. Aunque hay muchas máquinas de Turing universales posibles y, por lo tanto, muchos programas "más cortos" posibles, estos programas difieren en longitud por lo más un constante que es independiente de la cantidad de datos. Esta hermosa idea que, en esencia muestra que *cualquier* sesgo de representación inicial será eventualmente superado por los datos, está empañada sólo por la indecidibilidad de calcular la longitud del programa más corto. Medidas aproximadas como la **longitud mínima de descripción**, o MDL (Rissanen, 1984, 2007) pueden ser utilizadas en su lugar y han producido excelentes resultados en la práctica. El texto de Li y Vitanyi (2008) es la mejor fuente para la complejidad de Kolmogorov.

La teoría del **aprendizaje PAC** fue inaugurada por Leslie Valiant (1984), destacando la importancia de la complejidad computacional y de muestra. Con Michael Kearns (1990), Valiant demostró que varias clases de conceptos no permiten ser aprendidas de manera tratable por PAC, aunque hay suficiente información disponible en los ejemplos. Se obtuvieron algunos resultados positivos para clases como las listas de decisiones (Rivest, 1987).

Una tradición independiente de análisis de complejidad de muestra ha existido en estadística, comenzando con el trabajo sobre la **teoría de la convergencia uniforme** (Vapnik y Chervonenkis, 1971). La llamada **dimensión VC** proporciona una medida análoga a, pero más general que la medida $\ln|H|$ obtenida del análisis PAC. La dimensión VC permite aplicarse a clases de funciones continuas, a las que no se aplica el análisis PAC estándar. La teoría del aprendizaje PAC y la teoría VC fueron conectadas por primera vez por los "cuatro alemanes" (ninguno de los cuales es alemán): Blumer, Ehrenfeucht, Haussler y Warmuth (1989).

Dimensión VC

La **regresión lineal** con pérdida de error cuadrado se remonta a Legendre (1805) y Gauss (1809), quienes estaban trabajando en la predicción de órbitas alrededor del Sol. (Gauss afirmó estar usando la técnica desde 1795, pero se retrasó en publicarla). El uso moderno de la regresión multivariable para el aprendizaje automático se cubre en textos como Bishop (2007). Las diferencias entre la regularización L_1 y L_2 son analizadas por Ng (2004) y Moore y DeNero (2011).

El término **función logística** proviene de Pierre-François Verhulst (1804–1849), un estadístico que utilizó la curva para modelar el crecimiento de la población con recursos limitados, un modelo más realista que el crecimiento geométrico sin restricciones propuesto por Thomas Malthus. Verhulst la llamó la *courbe logistique*, debido a su relación con la curva logarítmica. El término **maldición de la dimensionalidad** proviene de Richard Bellman (1961).

La **regresión logística** permite resolverse con descenso de gradiente o con el método de Newton–Raphson (Newton, 1671; Raphson, 1690). Una variante del método de Newton llamada L-BFGS se utiliza para problemas de grandes dimensiones; la L quiere decir "memoria limitada", lo que significa que evita crear las matrices completas de una sola vez y en su lugar crea partes de ellas sobre la marcha. BFGS son las iniciales de los autores (Byrd *et al*., 1995). La idea del descenso de gradiente se remonta a Cauchy (1847); el descenso de gradiente estocástico (SGD) fue introducido en la comunidad de optimización estadística por Robbins y Monro (1951), redescubierto para redes neuronales por Rosenblatt (1960), y popularizado para el aprendizaje a gran escala por Bottou y Bousquet (2008). Bottou *et al*. (2018) reconsideran el tema del aprendizaje a gran escala con una década de experiencia adicional.

Los modelos de **vecinos más cercanos** se remontan al menos a Fix y Hodges (1951) y han sido una herramienta estándar en estadísticas y reconocimiento de patrones desde entonces. Dentro de la IA, fueron popularizados por Stanfill y Waltz (1986), quienes investigaron métodos para adaptar la métrica de distancia a los datos. Hastie y Tibshirani (1996) desarrollaron una forma de localizar la métrica en cada punto del espacio, dependiendo de la distribución de datos alrededor de ese punto. Gionis *et al*. (1999) introdujeron el hashing sensible a la localidad (LSH), que revolucionó la recuperación de objetos similares en espacios de alta dimensión. Andoni e Indyk (2006) proporcionan una encuesta de LSH y métodos relacionados, y Samet (2006) cubre las propiedades de los espacios de alta dimensión. La técnica es útil para datos genómicos, donde cada registro tiene millones de atributos (Berlin *et al*., 2015).

Las ideas detrás de las **máquinas de kernel** provienen de Aizerman *et al*. (1964) (quienes también introdujeron el truco del kernel), pero el desarrollo completo de la teoría se debe a Vapnik y sus colegas (Boser *et al*., 1992). Las SVM se hicieron prácticas con la introducción del clasificador de margen suave para manejar datos ruidosos en un artículo que ganó el Premio Teoría y Práctica ACM 2008 (Cortes y Vapnik, 1995), y del algoritmo de Optimización Mínima Secuencial (SMO) para resolver de manera eficiente problemas de SVM utilizando programación

cuadrática (Platt, 1999). Las SVM han demostrado ser muy efectivas para tareas como la categorización de texto (Joachims, 2001), la genómica computacional (Cristianini y Hahn, 2007), y el reconocimiento de dígitos escritos a mano de DeCoste y Schölkopf (2002).

Como parte de este proceso, se han diseñado muchos nuevos kernels que trabajan con cadenas, árboles y otros tipos de datos no numéricos. Una técnica relacionada que también utiliza el truco del kernel para representar de manera implícita un espacio de características exponencial es el perceptrón votado (Freund y Schapire, 1999; Collins y Duffy, 2002). Los libros de texto sobre SVM incluyen a Cristianini y Shawe-Taylor (2000) y Schölkopf y Smola (2002). Una exposición más amigable aparece en el artículo de la revista *AI Magazine* de Cristianini y Schölkopf (2002). Bengio y LeCun (2007) muestran algunas de las limitaciones de las SVM y otros métodos locales no paramétricos para aprender funciones que tienen una estructura global pero no tienen suavidad local.

La primera prueba matemática del valor de un conjunto fue el teorema del jurado de Condorcet (1785), que demostró que si los jurados son independientes y un jurado individual tiene al menos 50 % de posibilidades de decidir un caso de manera correcta, entonces cuantos más jurados agregue, mejor será la posibilidad de decidir el caso correctamente. Más reciente, el **aprendizaje en conjunto** se ha convertido en una técnica cada vez más popular para mejorar el rendimiento de los algoritmos de aprendizaje.

El primer algoritmo de **bosque aleatorio**, utilizando selección de atributos aleatorios, es de Ho (1995); una versión independiente fue introducida por Amit y Geman (1997). Breiman (2001) agregó las ideas de **bagging** y "error fuera de la bolsa". Friedman (2001) introdujo la terminología Máquina de Impulso Gradiente (GBM), expandiendo el enfoque para permitir la clasificación multiclase, regresión y problemas de clasificación.

Michel Kearns (1988) definió el Problema de Impulso de Hipótesis: dado un aprendiz que predice sólo ligeramente mejor que adivinar al azar, ¿es posible derivar un aprendiz que se desempeña bien? El problema fue respondido de manera afirmativa en un artículo teórico de Schapire (1990) que condujo al algoritmo AdaBoost de Freund y Schapire (1996) y a un trabajo teórico adicional Schapire (2003). Friedman *et al*. (2000) explican el impulso desde el punto de vista de un estadístico. Chen y Guestrin (2016) describen el sistema XGBoost, que ha sido utilizado con gran éxito en muchas aplicaciones a gran escala.

El aprendizaje online se cubre en una encuesta de Blum (1996) y un libro de Cesa-Bianchi y Lugosi (2006). Dredze *et al*. (2008) introducen la idea del aprendizaje online ponderado por confianza para la clasificación: además de mantener un peso para cada parámetro, también mantienen una medida de confianza, de modo que un nuevo ejemplo puede tener un gran efecto en las características que rara vez se vieron antes (y, por lo tanto, tenían baja confianza) y un pequeño efecto en las características comunes que ya han sido bien estimadas. Yu *et al*. (2011) describen cómo un equipo de estudiantes trabaja juntos para construir un clasificador de conjunto en la competencia KDD. Una posibilidad emocionante es crear un conjunto "extremadamente grande" de mezcla de expertos que utiliza un subconjunto escaso de expertos para cada ejemplo entrante (Shazeer *et al*., 2017). Seni y Elder (2010) revisan los métodos de conjunto.

En términos de consejos prácticos para construir sistemas de aprendizaje automático, Pedro Domingos describe algunas cosas que saber (2012). Andrew Ng da pistas para desarrollar y depurar un producto utilizando el aprendizaje automático (Ng, 2019). O'Neil y Schutt (2013) describen el proceso de hacer ciencia de datos. Tukey (1977) introdujo el **análisis de datos exploratorio**, y Gelman (2004) da una visión actualizada del proceso. Bien *et al*. (2011) describen el proceso de elegir prototipos para la interpretabilidad, y Kim *et al*. (2017) muestran cómo encontrar críticos que están a la máxima distancia de los prototipos utilizando una métrica llamada discrepancia media máxima. Wattenberg *et al*. (2016) describen cómo usar t-SNE. Para obtener una

visión completa de cómo está funcionando su sistema de aprendizaje automático implementado, Breck *et al*. (2016) ofrecen una lista de verificación de 28 pruebas que puede aplicar para obtener una puntuación de prueba de ML general. Riley (2019) describe tres trampas comunes del desarrollo de ML.

Banko y Brill (2001), Halevy *et al*. (2009), y Gandomi y Haider (2015) discuten las ventajas de usar las grandes cantidades de datos que ahora están disponibles. Lyman y Varian (2003) estimaron que se produjeron alrededor de 5 exabytes (5×10^{18} bytes) de datos en 2002, y que la tasa de producción se duplica cada 3 años; Hilbert y Lopez (2011) estimaron 2×10^{21} bytes para 2007, indicando una aceleración. Guyon y Elisseeff (2003) discuten el problema de la selección de características con grandes conjuntos de datos.

Doshi-Velez y Kim (2017) proponen un marco para el **aprendizaje automático interpretable** o la **IA explicativa (XAI)**. Miller *et al.* (2017) señalan que hay dos tipos de explicaciones, una para los diseñadores de un sistema de IA y otra para los usuarios, y necesitamos tener claro a qué apuntamos. El sistema LIME (Ribeiro *et al*., 2016) construye modelos lineales interpretables que aproximan cualquier sistema de aprendizaje automático que tenga. Un sistema similar, SHAP (Lundberg y Lee, 2018) (Shapley Additive exPlanations), utiliza la noción de un valor de Shapley (página 628) para determinar la contribución de cada característica.

La idea de que podríamos aplicar el aprendizaje automático a la tarea de resolver problemas de aprendizaje automático es tentadora. Thrun y Pratt (2012) dan una visión general temprana del campo en una colección editada titulada *Aprendiendo a Aprender*. Recientemente, el campo ha adoptado el nombre **aprendizaje automático automatizado** (**AutoML**); Hutter *et al*. (2019) dan una visión general.

Aprendizaje automático automatizado (AutoML)

Kanter y Veeramachaneni (2015) describen un sistema para realizar la selección automática de características. Bergstra y Bengio (2012) describen un sistema para buscar en el espacio de hiperparámetros, al igual que Thornton *et al*. (2013) y Bermúdez-Chacón *et al.* (2015). Wong *et al*. (2019) muestran cómo el aprendizaje por transferencia permite acelerar AutoML para modelos de aprendizaje profundo. Se han organizado competencias para ver qué sistemas son los mejores en las tareas de AutoML (Guyon *et al*., 2015). (Steinruecken *et al*., 2019) describen un sistema llamado el Estadístico Automático: le da algunos datos y escribe un informe, mezclando texto, grafos y cálculos. Los principales proveedores de computación en la nube han incluido AutoML como parte de sus ofertas. Algunos investigadores prefieren el término **metaaprendizaje**: por ejemplo, el sistema MAML (Model-Agnostic Meta-Learning) (Finn *et al*., 2017) funciona con cualquier modelo que pueda ser entrenado por descenso de gradiente; entrena un modelo central para que sea fácil de ajustar el modelo con nuevos datos en nuevas tareas.

A pesar de todo este trabajo, todavía no tenemos un sistema completo para resolver de manera automática los problemas de aprendizaje automático. Para hacer eso con el aprendizaje automático supervisado tendríamos que empezar con un conjunto de datos de ejemplos $(\mathbf{x}_j, y_j)$. Aquí la entrada $\mathbf{x}_j$ es una especificación del problema, en la forma en que un problema se encuentra inicialmente: una descripción vaga de los objetivos, y algunos datos con los que trabajar, quizá con un plan vago de cómo adquirir más datos. La salida y_i sería un programa completo de aprendizaje automático en funcionamiento, junto con una metodología para mantener el programa: recopilación de más datos, limpieza de los mismos, pruebas y monitoreo del sistema, etc. Se esperaría que necesitáramos un conjunto de datos de miles de tales ejemplos. Pero no existe tal conjunto de datos, por lo que los sistemas AutoML existentes están limitados en lo que permiten lograr.

Existe una amplia variedad de libros que nos introducen a la ciencia de datos y al aprendizaje automático, en conjunto con paquetes de software como Python (Segaran, 2007; Raschka, 2015; Nielsen, 2015), Scikit-Learn (Pedregosa *et al*., 2011), R (Conway y White, 2012), Pandas

(McKinney, 2012), NumPy (Marsland, 2014), PyTorch (Howard y Gugger, 2020), TensorFlow (Ramsundar y Zadeh, 2018), y Keras (Chollet, 2017; Géron, 2019).

Hay una serie de valiosos libros de texto en aprendizaje automático (Bishop, 2007; Murphy, 2012) y en los campos aliados y superpuestos de reconocimiento de patrones (Ripley, 1996; Duda *et al*., 2001), estadísticas (Wasserman, 2004; Hastie *et al*., 2009; James *et al*., 2013), ciencia de datos (Blum *et al*., 2020), minería de datos (Han *et al*., 2011; Witten y Frank, 2016; Tan *et al*., 2019), teoría del aprendizaje computacional (Kearns y Vazirani, 1994; Vapnik, 1998) y teoría de la información (Shannon y Weaver, 1949; MacKay, 2002; Cover y Thomas, 2006). Burkov (2019) intenta la introducción más corta posible al aprendizaje automático, y Domingos (2015) ofrece una visión general no técnica del campo. La investigación actual en aprendizaje automático se publica en las actas anuales de la Conferencia Internacional sobre Aprendizaje Automático (ICML), la Conferencia Internacional sobre Representaciones de Aprendizaje (ICLR), y la conferencia sobre Sistemas de Procesamiento de Información Neural (NeurIPS); y en *Machine Learning* y el *Journal of Machine Learning Research*.

CAPÍTULO 20

APRENDIZAJE DE MODELOS PROBABILÍSTICOS

En el que vemos el aprendizaje como una forma de razonamiento incierto a partir de observaciones, e ideamos modelos para representar el mundo incierto.

En el capítulo 12 se señala la prevalencia de la incertidumbre en entornos reales. Los agentes pueden manejar la incertidumbre utilizando los métodos de la probabilidad y de la teoría de la decisión, pero primero deben aprender sus teorías probabilísticas del mundo a partir de la experiencia. Este capítulo explica cómo pueden hacerlo, formulando la tarea de aprendizaje en sí misma como un proceso de inferencia probabilística (Sección 20.1). Veremos que una visión bayesiana del aprendizaje es extremadamente poderosa pues proporciona soluciones generales a los problemas de ruido, sobreajuste y predicción óptima. También tiene en cuenta el hecho de que un agente menos que omnisciente nunca puede estar seguro de qué teoría del mundo es la correcta, y aun así debe tomar decisiones utilizando alguna teoría del mundo.

Describimos métodos para el aprendizaje de modelos de probabilidad, principalmente redes bayesianas, en las Secciones 20.2 y 20.3. Parte del material de este capítulo es claramente matemático, aunque las lecciones generales se pueden entender sin sumergirse en los detalles. Al lector le puede ayudar repasar los Capítulos 12 y 13 y echarle un vistazo al Apéndice A.

20.1 Aprendizaje estadístico

Los conceptos clave en este capítulo, al igual que en el capítulo 19, son **datos** e **hipótesis**. Aquí los datos son **evidencia**, es decir, instancias de algunas o todas las variables aleatorias que describen el dominio. Las hipótesis de este capítulo son teorías probabilísticas de cómo funciona el dominio, incluidas las teorías lógicas como un caso especial.

Veamos un ejemplo sencillo. Nuestro dulce sorpresa favorito viene en dos sabores: cereza (mmm) y lima (ugh). El fabricante tiene un peculiar sentido del humor y envuelve cada pieza de caramelo en el mismo envoltorio opaco, independientemente del sabor. El caramelo se vende en grandes bolsas, de las cuales se sabe que hay cinco tipos, de nuevo, indistinguibles desde el exterior:

h_1: 100 % cereza,
h_2: 75 % cereza + 25 % lima,
h_3: 50 % cereza + 50 % lima,
h_4: 25 % cereza + 75 % lima,
h_5: 100 % lima.

Dado una nueva bolsa de dulces, la variable aleatoria H (para *hipótesis*) denota el tipo de bolsa, con posibles valores de h_1 a h_5. Claro, H no es directamente observable. A medida que se abren y se inspeccionan los dulces, se revelan los datos: $D_1, D_2,..., D_N$, donde cada D_i es una variable aleatoria con posibles valores *cereza* y *lima*. La tarea básica a la que se enfrenta el agente es predecir

el sabor de la próxima pieza de caramelo.[1] A pesar de su aparente trivialidad, este escenario sirve para introducir muchas de las cuestiones principales. El agente realmente necesita inferir una teoría de su mundo, aunque muy simple.

Aprendizaje bayesiano

El **aprendizaje bayesiano** sólo calcula la probabilidad de cada hipótesis, dados los datos, y hace predicciones sobre esa base. Es decir, las predicciones se hacen utilizando *todas* las hipótesis, ponderados por sus probabilidades, en lugar de usar una sola "mejor" hipótesis. De esta manera, el aprendizaje se reduce a la inferencia probabilística.

Hipótesis a priori
Probabilidad

Sea $\mathbf{D}$ la representación de todos los datos, con el valor observado $\mathbf{d}$. Las cantidades clave en el enfoque bayesiano son las **hipótesis a priori**, $P(h_i)$, y la **probabilidad** de los datos bajo cada hipótesis, $P(\mathbf{d}|h_i)$. La probabilidad de cada hipótesis se obtiene mediante la regla de Bayes:

$$P(h_i|\mathbf{d}) = \alpha P(\mathbf{d}|h_i)P(h_i). \tag{20.1}$$

Ahora, suponga que queremos hacer una predicción sobre una cantidad desconocida X. Luego tenemos

$$\mathbf{P}(X|\mathbf{d}) = \sum_i \mathbf{P}(X|h_i)P(h_i|\mathbf{d}). \tag{20.2}$$

donde cada hipótesis determina una distribución de probabilidad sobre X. Esta ecuación muestra que las predicciones son medias ponderadas sobre las predicciones de las hipótesis individuales, donde el peso $P(h_i|\mathbf{d})$ es proporcional a la probabilidad previa de h_i y a su grado de ajuste, según a la ecuación (20.1). Las hipótesis mismas son esencialmente "intermediarias" entre los datos brutos y las predicciones.

Para nuestro ejemplo de los dulces, asumiremos por el momento que la distribución anterior sobre $h_1,....,h_5$ viene dada por $\langle 0.1,0.2,0.4,0.2,0.1\rangle$, según lo anunciado por el fabricante. La probabilidad de los datos se calcula bajo el supuesto de que las observaciones son **i.i.d.** (véase página 665), de modo que

$$P(\mathbf{d}|h_i) = \prod_j P(d_j|h_i). \tag{20.3}$$

Por ejemplo, suponga que la bolsa es en realidad una bolsa de lima (h_5) y los primeros 10 caramelos son todos de lima; entonces $P(\mathbf{d}|h_3)$ es 0.5^{10}, porque la mitad de los caramelos en una bolsa h_3 son de lima.[2] La Figura 20.1(a) muestra cómo cambian las probabilidades posteriores de las cinco hipótesis a medida que se observa la secuencia de 10 caramelos de lima. Observe que las probabilidades comienzan en sus valores previos, por lo que h_3 es inicialmente la elección más probable y sigue siendo así después de que se desenvuelve 1 caramelo de lima. Después de que se desenvuelven 2 caramelos de lima, h_4 es el más probable; después de 3 o más, h_5 (la temida bolsa de lima) es el más probable. Después de 10 seguidos, estamos bastante seguros de nuestro destino. La Figura 20.1(b) muestra la probabilidad predicha de que el próximo caramelo sea de lima, con base en la Ecuación (20.2). Como esperaríamos aumenta monótonamente hacia 1.

El ejemplo muestra que *la predicción bayesiana eventualmente concuerda con la verdadera hipótesis*. Esta es una característica del aprendizaje bayesiano. Para cualquier *a prior* fijo que no descarte la verdadera hipótesis, la probabilidad posterior de cualquier hipótesis falsa desaparecerá, eventualmente, bajo ciertas condiciones técnicas. Esto sucede porque la probabilidad de

[1] Los lectores con conocimientos estadísticos reconocerán este escenario como una variante de la configuración **de urna y bola**. Encontramos urnas y bolas menos atractivas que los caramelos.

[2] Ya antes dijimos que las bolsas de caramelos son muy grandes; de lo contrario, la suposición i.i.d. falla. Técnicamente, es más correcto (pero menos higiénico) volver a envolver cada caramelo después de la inspección y regresarlo a la bolsa.

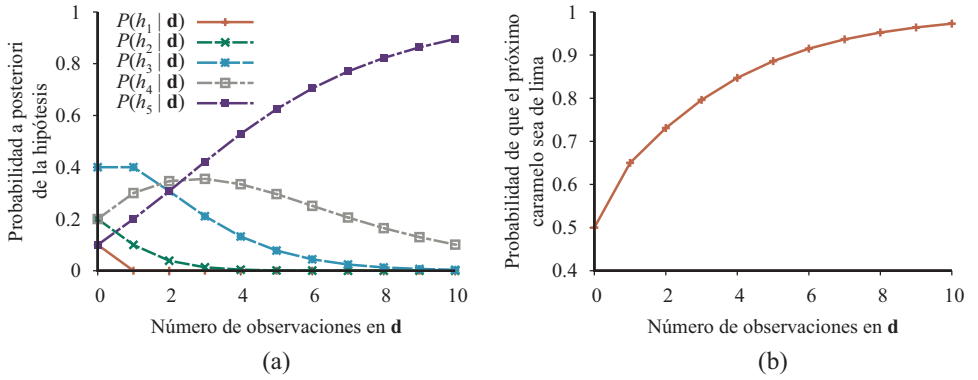


Figura 20.1 **(a)** Probabilidades posteriores $P(h_i|d_1,...,d_N)$ de la Ecuación (20.1). El número de observaciones N varía de 1 a 10, y cada observación es de un caramelo de lima. (b) Predicción bayesiana $P(D_{N+1} = \text{lima}|d_1,...,d_N)$ de la Ecuación (20.2).

generar datos "no característicos" de manera indefinida es infinitamente pequeña. (Este punto es análogo a uno hecho en la discusión del aprendizaje PAC en el Capítulo 19). Más importante, la predicción bayesiana es *óptima*, ya sea que el conjunto de datos sea pequeño o grande. Dada la hipótesis previa, se espera que cualquier otra predicción sea correcta con menos frecuencia.

Por supuesto, la optimalidad del aprendizaje bayesiano tiene un precio. Para problemas de aprendizaje reales, el espacio de hipótesis suele ser muy grande o infinito, como vimos en el Capítulo 19. En algunos casos, la suma en la Ecuación (20.2) (o integración, en el caso continuo) permite llevarse a cabo de manera manejable, pero en la mayoría de los casos debemos recurrir a métodos aproximados o simplificados.

Una aproximación común, una que se adopta en la ciencia, es hacer predicciones basadas en una hipótesis *más probable*, es decir, una h_i que maximiza $P(h_i|\mathbf{d})$. Esto se llama hipótesis de **máximo a posteriori** o MAP (pronunciado en inglés como "em-ay-pee"). Las predicciones hechas de acuerdo con una hipótesis MAP h_{MAP} son aproximadamente bayesianas en la medida en que $\mathbf{P}(X|\mathbf{d}) \approx \mathbf{P}(X|h_{\text{MAP}})$. En nuestro ejemplo de caramelos es $h_{\text{MAP}} = h_5$ después de tres caramelos de lima seguidos, por lo que, entonces, el aprendiz de MAP predice que el cuarto caramelo es de lima con probabilidad 1.0, una predicción mucho más peligrosa que la predicción bayesiana de 0.8 mostrada en la Figura 20.1(b). A medida que llegan más datos, las predicciones MAP y bayesianas se acercan, porque los competidores de la hipótesis MAP se vuelven cada vez menos probables.

Máximo a posteriori

Aunque este ejemplo no lo muestra, con frecuencia es mucho más fácil encontrar hipótesis MAP que el aprendizaje bayesiano, porque requiere resolver un problema de optimización en lugar de un gran problema de sumación (o integración).

Tanto en el aprendizaje bayesiano como en el aprendizaje MAP, el prior de la hipótesis $P(h_i)$ tiene un papel importante. En el Capítulo 19 vimos que el sobreajuste puede ocurrir cuando el espacio de hipótesis es demasiado expresivo, es decir, cuando contiene muchas hipótesis que se ajustan bien al conjunto de datos. Los métodos de aprendizaje bayesianos y MAP utilizan el *a priori* para *penalizar la complejidad*. Típicamente, las hipótesis más complejas tienen una menor probabilidad previa, en parte porque hay muchas de ellas. Por otro lado, las hipótesis más complejas tienen una mayor capacidad para ajustarse a los datos. (En el caso extremo, una tabla de búsqueda permite reproducir los datos de manera exacta). Por lo tanto, el prior de la hipótesis incorpora un equilibrio entre la complejidad de una hipótesis y su grado de ajuste a los datos.

Podemos ver el efecto de este equilibrio más claramente en el caso lógico, donde H contiene sólo hipótesis *deterministas* (como h_1, que dice que cada caramelo es de cereza). En ese caso, $P(\mathbf{d}|h_i)$ es 1 si h_i es consistente y 0 en caso contrario. Mirando la Ecuación (20.1), vemos que h_{MAP} será entonces la *teoría lógica más simple que sea consistente con los datos*. Por lo tanto, el aprendizaje máximo *a posteriori* proporciona una encarnación natural de la navaja de Ockham.

Otra visión sobre el equilibrio entre la complejidad y el grado de ajuste se obtiene al tomar el logaritmo de la Ecuación (20.1). Elegir h_{MAP} para maximizar $P(\mathbf{d}|h_i)P(h_i)$ es equivalente a minimizar

$$-\log_2 P(\mathbf{d}|h_i) - \log_2 P(h_i).$$

Usando la conexión entre la codificación de información y la probabilidad que introdujimos en la Sección 19.3.3, vemos que el término $-\log 2\ P(h_i)$ es igual al número de bits requeridos para especificar la hipótesis h_i. Además, $-\log 2\ P(\text{d}|\ h_i)$ es el número adicional de bits requeridos para especificar los datos, dada la hipótesis. (Para ver esto, considere que no se requieren bits si la hipótesis predice los datos de manera exacta, como con h_5 y la cadena de caramelos de lima, y $\log_2 1 = 0$.) Por lo tanto, el aprendizaje MAP está eligiendo la hipótesis que proporciona la máxima compresión de los datos. La misma tarea es abordada de manera directa por el método de aprendizaje de la **longitud mínima de descripción**, o MDL. Mientras que el aprendizaje MAP expresa la simplicidad asignando mayores probabilidades a hipótesis más simples, MDL la expresa de manera directa contando los bits en una codificación binaria de las hipótesis y los datos.

Una simplificación final se proporciona al asumir un prior **uniforme** sobre el espacio de hipótesis. En ese caso, el aprendizaje MAP se reduce a elegir un h_i que maximiza $P(\mathbf{d}|\ h_i)$. Esto se llama hipótesis de **máxima verosimilitud**, h_{ML}. El aprendizaje de máxima verosimilitud es muy común en estadística, una disciplina en la que muchos investigadores desconfían de la naturaleza subjetiva de los priors de hipótesis. Es un enfoque razonable cuando no hay razón para preferir una hipótesis sobre otra *a priori*, por ejemplo, cuando todas las hipótesis son igualmente complejas.

Máxima verosimilitud

Cuando el conjunto de datos es grande, la distribución previa sobre las hipótesis es menos importante, la evidencia de los datos es suficientemente fuerte para inundar la distribución previa sobre las hipótesis. Eso significa que el aprendizaje de máxima verosimilitud es una buena aproximación al aprendizaje bayesiano y MAP con grandes conjuntos de datos, pero tiene problemas (como veremos) con pequeños conjuntos de datos.

20.2 Aprendizaje con datos completos

La tarea general de aprender un modelo de probabilidad, dado que se supone que los datos se generan a partir de ese modelo, se llama **estimación de densidad**. (El término se aplicó originalmente a las funciones de densidad de probabilidad para variables continuas, pero ahora se usa también para distribuciones discretas). La estimación de densidad es una forma de aprendizaje no supervisado. Esta sección cubre el caso más simple, donde tenemos **datos completos**. Los datos son completos cuando cada punto de datos contiene valores para cada variable en el modelo de probabilidad que se está aprendiendo. Nos centramos en el **aprendizaje de parámetros**, encontrando los parámetros numéricos para un modelo de probabilidad cuya estructura está fija. Por ejemplo, podríamos estar interesados en aprender las probabilidades condicionales en una red bayesiana con una estructura dada. También echaremos un vistazo breve al problema de aprender la estructura y a la estimación de densidad no paramétrica.

Estimación de densidad

Datos completos

Aprendizaje de parámetros

20.2.1 Aprendizaje de parámetros de máxima verosimilitud: Modelos discretos

Suponga que compramos una bolsa de caramelos de lima y cereza de un nuevo fabricante, cuyas proporciones de sabor son completamente desconocidas; la fracción de cereza podría estar en cualquier lugar entre 0 y 1. En ese caso, tenemos un continuo de hipótesis. El **parámetro** en este caso, al que llamamos θ, es la proporción de caramelos de cereza, y la hipótesis es h_θ. (La proporción de caramelos de lima es $1 - \theta$). Si asumimos que todas las proporciones son igualmente probables *a priori*, entonces un enfoque de máxima verosimilitud es razonable. Si modelamos la situación con una red bayesiana, necesitamos sólo una variable aleatoria, *Sabor* (el sabor de un caramelo elegido al azar de la bolsa). Tiene valores *cereza* y *lima*, donde la probabilidad de *cereza* es θ (vea la Figura 20.2(a)). Ahora suponga que desenvolvemos N caramelos, de los cuales c son de cereza y $\ell = N - c$ son de lima. Según la Ecuación (20.3), la probabilidad de este conjunto de datos particular es

$$P(\mathbf{d}|h_\theta) = \prod_{j=i}^{N} P(d_j|h_\theta) = \theta^c \cdot (1-\theta)^\ell .$$

La hipótesis de máxima verosimilitud se da por el valor de θ que maximiza esta expresión. Debido a que la función logarítmica es monótona, se obtiene el mismo valor maximizando el **logaritmo de verosimilitud** en su lugar:

Logaritmo de verosimilitud

$$L(\mathbf{d}|h_\theta) = \log P(\mathbf{d}|h_\theta) = \sum_{j=i}^{N} \log P(d_j|h_\theta) = c\log\theta + \ell\log(1-\theta).$$

(Al tomar logaritmos, reducimos el producto a una suma sobre los datos, que generalmente son más fácil de maximizar). Para encontrar el valor de máxima verosimilitud de θ, diferenciamos L con respecto a θ y establecemos la expresión resultante en cero:

$$\frac{dL(\mathbf{d}|h_\theta)}{d\theta} = \frac{c}{\theta} - \frac{\ell}{1-\theta} = 0 \Rightarrow \qquad \theta = \frac{c}{c+\ell} = \frac{c}{N}.$$

En español, entonces, la hipótesis de máxima verosimilitud h_{ML} afirma que ¡la proporción real de caramelos de cereza en la bolsa es igual a la proporción observada en los caramelos desenvueltos hasta ahora!

Parece que hemos hecho mucho trabajo para descubrir lo obvio. Sin embargo, hemos presentado un método estándar para el aprendizaje de parámetros de máxima verosimilitud, un método con amplia aplicabilidad:

1. Escriba una expresión para la probabilidad de los datos como una función del (los) parámetro(s).
2. Escriba la derivada del logaritmo de verosimilitud con respecto a cada parámetro.
3. Encuentre los valores de los parámetros de manera que las derivadas sean cero.

El paso más complicado suele ser el último. En nuestro ejemplo, fue trivial pero veremos que en muchos casos necesitamos recurrir a algoritmos de solución iterativos u otras técnicas de optimización numérica, como se describe en la Sección 4.2. (Necesitaremos verificar que la matriz hessiana sea negativa-definida). El ejemplo también ilustra un problema significativo con el aprendizaje de máxima verosimilitud en general: *cuando el conjunto de datos es lo suficientemente pequeño como para que algunos eventos aún no se hayan observado, por ejemplo, no hay caramelos de cereza, la hipótesis de máxima verosimilitud asigna cero probabilidad a esos eventos*. Se utilizan varios trucos para evitar este problema, como inicializar los recuentos para cada evento a 1 en lugar de 0.

Echemos un vistazo a otro ejemplo. Suponga que este nuevo fabricante de caramelos quiere dar una pequeña pista al consumidor y utiliza envolturas de caramelos de colores rojo y verde.

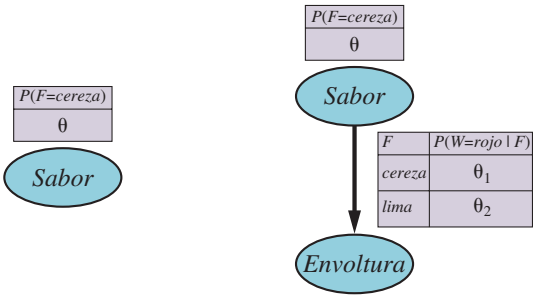


Figura 20.2 (a) Modelo de red bayesiana para el caso de caramelos con una proporción desconocida de cereza y lima. (b) Modelo para el caso en que el color de la envoltura depende (probabilísticamente) del sabor del caramelo.

La *envoltura* para cada caramelo se selecciona *probabilísticamente*, según alguna distribución condicional desconocida, dependiendo del sabor. El modelo de probabilidad correspondiente se muestra en la Figura 20.2(b). Observe que tiene tres parámetros: θ, θ_1 y θ_2. Con estos parámetros, la probabilidad de ver, digamos, un caramelo de cereza en una envoltura verde se puede obtener de la semántica estándar para las redes bayesianas (página 415):

$$\begin{aligned} &P(Sabor = cereza, Envoltura = verde \mid h_{\theta,\theta 1,\theta 2}) \\ &= P(Sabor = cereza|h_{\theta,\theta 1,\theta 2})P(Envoltura = verde|Sabor = cereza, \mathrm{h}_{\theta,\theta 1,\theta 2}) \\ &= \theta\cdot(1-\theta_1). \end{aligned}$$

Ahora desenvolvemos N caramelos, de los cuales c son de cereza y ℓ son de lima. Los recuentos de envoltorios son: r_c cuando los caramelos de cereza tienen envolturas rojas y g_c si tienen verde, mientras que r_ℓ de los caramelos de lima tienen roja y g_ℓ tienen verde. La probabilidad de los datos se da por

$$P(\mathbf{d}|h_{\theta,\theta 1,\theta 2}) = \theta^c(1-\theta)^\ell\cdot\theta_1^{r_c}(1-\theta_1)^{g_c}\cdot\theta_2^{r_\ell}(1-\theta_2)^{g_\ell}.$$

Esto parece bastante horrible, pero tomar logaritmos ayuda:

$$L = [c\log\theta + \ell\log(1-\theta)] + [r_c\log\theta_1 + g_c\log(1-\theta_1)] + [r_\ell\log\theta_2 + g_\ell\log(1-\theta_2)].$$

El beneficio de tomar logaritmos es claro: el logaritmo de verosimilitud es la suma de tres términos, cada uno de los cuales contiene un solo parámetro. Cuando tomamos derivadas con respecto a cada parámetro y las establecemos en cero, obtenemos tres ecuaciones independientes, cada una conteniendo sólo un parámetro:

$$\begin{aligned} \tfrac{\partial L}{\partial\theta} = \tfrac{c}{\theta} - \tfrac{\ell}{1-\theta} = 0 \quad &\Rightarrow \quad \theta = \tfrac{c}{c+\ell} \\ \tfrac{\partial L}{\partial\theta_1} = \tfrac{r_c}{\theta_1} - \tfrac{g_c}{1-\theta_1} = 0 \quad &\Rightarrow \quad \theta_1 = \tfrac{r_c}{r_c+g_c} \\ \tfrac{\partial L}{\partial\theta_2} = \tfrac{r_\ell}{\theta_2} - \tfrac{g_\ell}{1-\theta_2} = 0 \quad &\Rightarrow \quad \theta_2 = \tfrac{r_\ell}{r_\ell+g_\ell} \end{aligned}$$

La solución para θ es la misma que antes. La solución para θ_1, la probabilidad de que un caramelo de cereza tenga una envoltura roja, es la fracción observada de caramelos de cereza con envolturas rojas, de manera similar para θ_2.

Estos resultados son muy reconfortantes, y es fácil ver que se pueden extender a cualquier red bayesiana cuyas probabilidades condicionales se representen como tablas. El punto más

importante es que *con datos completos, el problema de aprendizaje de parámetros de máxima verosimilitud para una red bayesiana se descompone en problemas de aprendizaje separados, uno para cada parámetro.* (Consulte el Ejercicio 20.NORX para el caso no tabulado, donde cada parámetro afecta varias probabilidades condicionales). El segundo punto es que los valores de los parámetros para una variable, dados sus padres, son sólo las frecuencias observadas de los valores de las variables para cada configuración de los valores principales. Al igual que antes, debemos tener cuidado de evitar ceros cuando el conjunto de datos es pequeño.

20.2.2 Modelos bayesianos ingenuos

Quizás el modelo de red bayesiana más común utilizado en el aprendizaje automático es el modelo de **Bayes ingenuo** presentado por primera vez en la página 402. En este modelo, la variable de "clase" C (que se va a predecir) es la raíz y las variables de "atributo" X_i, las hojas. El modelo es "ingenuo" porque asume que los atributos son condicionalmente independientes entre sí, dada la clase. (El modelo de la Figura 20.2 (b) es un modelo de Bayes ingenuo con clase *Sabor* y un sólo atributo, *Envoltura*). En el caso de las variables booleanas, los parámetros son

$$\theta = P(C = verdadero), \theta_{i1} = P(X_i = verdadero | C = verdadero), \theta_{i2} = P(X_i = verdadero | C = falso).$$

Los valores de los parámetros de máxima verosimilitud se encuentran exactamente de la misma manera que en la Figura 20.2(b). Una vez que el modelo se ha entrenado de esta manera, se puede utilizar para clasificar nuevos ejemplos para los que no se observa la variable de clase C. Con valores de atributo observados $x_1,...,x_n$, La probabilidad de cada clase viene dada por

$$\mathbf{P}(C|x_1,...,x_n) = \alpha \mathbf{P}(C) \prod_i \mathbf{P}(x_i|C).$$

Se puede obtener una predicción determinista eligiendo la clase más probable. La Figura 20.3 muestra la curva de aprendizaje de este método cuando se aplica al problema del restaurante del Capítulo 19. El método aprende bastante bien, pero no tan bien como el aprendizaje del árbol de decisión; esto se debe presumiblemente a que la hipótesis verdadera, que es un árbol de decisión, no es representable exactamente utilizando un modelo de Bayes ingenuo. El aprendizaje ingenuo de Bayes resulta ser sorprendentemente bueno en una amplia gama de aplicaciones; la versión mejorada (Ejercicio 20.BNBX) es uno de los algoritmos de aprendizaje de propósito general más efectivos. El aprendizaje de Bayes ingenuo se adapta bien a problemas muy grandes: con n atributos booleanos, sólo hay $2n+1$ parámetros y *no se requiere búsqueda para encontrar* h_{ML}, *la hipótesis de Bayes ingenua de máxima verosimilitud*. Por último, los sistemas de aprendizaje de Bayes ingenuos manejan bien los datos ruidosos o faltantes y pueden dar predicciones probabilísticas cuando sea apropiado. Su principal inconveniente es el hecho de que el supuesto de la independencia condicional rara vez es exacto; como se señaló en la página 403, la suposición conduce a probabilidades demasiado confiadas que a menudo están muy cerca de 0 o 1, en especial con un gran número de atributos.

20.2.3 Modelos generativos y discriminativos

Podemos distinguir dos tipos de modelos de aprendizaje automático utilizados para los clasificadores: generativos y discriminativos. Un **modelo generativo** modela la distribución de probabilidad de cada clase. Para el ejemplo del modelo generativo, el clasificador de texto de Bayes ingenuo de la Sección 12.6.1 crea un modelo independiente para cada categoría de texto posible: uno para deportes, otro para el clima, etc. Cada modelo incluye la probabilidad previa de la categoría; por ejemplo, $P(Categoría=clima)$, así como la probabilidad condicional $\mathbf{P}(Entradas|Categoría=clima)$. A partir de estos podemos calcular la probabilidad conjunta $\mathbf{P}(Entradas, Categoría=clima)$) y podemos generar una selección aleatoria de palabras que sea representativa de los textos en la categoría de *clima*.

Modelo generativo

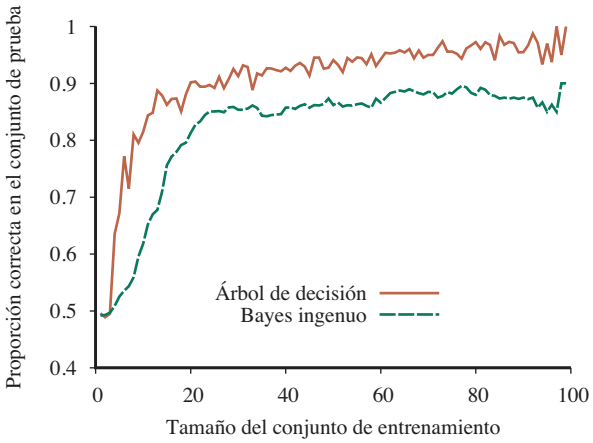


Figura 20.3 Curva de aprendizaje para el aprendizaje de Bayes ingenuo aplicado al problema del restaurante del Capítulo 19. Se muestra la curva de aprendizaje para el aprendizaje del árbol de decisión para comparación.

Modelo discriminativo

Un **modelo discriminativo** aprende de manera directa el límite de decisión entre clases. Es decir, aprende **P**(*Categoría*|*Entradas*). Dados los ejemplos de entradas, un modelo discriminativo llegará a una categoría de salida, pero no es posible usar un modelo discriminativo para, digamos, generar palabras aleatorias que son representativas de una categoría. La regresión logística, los árboles de decisión y las máquinas de vectores de soporte son todos modelos discriminativos.

Dado que los modelos discriminativos ponen todo su énfasis en definir el límite de decisión, es decir, en realizar la tarea de clasificación que se les pidió hacer, tienden a rendir mejor en el límite, con una cantidad arbitraria de datos de entrenamiento. Sin embargo, con datos limitados, en algunos casos un modelo generativo rinde mejor. (Ng y Jordan, 2002) comparan el clasificador de Bayes generativo ingenuo con el clasificador de regresión logística discriminativa en 15 conjuntos de datos (pequeños) y encuentran que con la cantidad máxima de datos, el modelo discriminativo rinde mejor en 9 de 15 conjuntos de datos, pero con sólo una pequeña cantidad de datos, el modelo generativo rinde mejor en 14 de 15 conjuntos de datos.

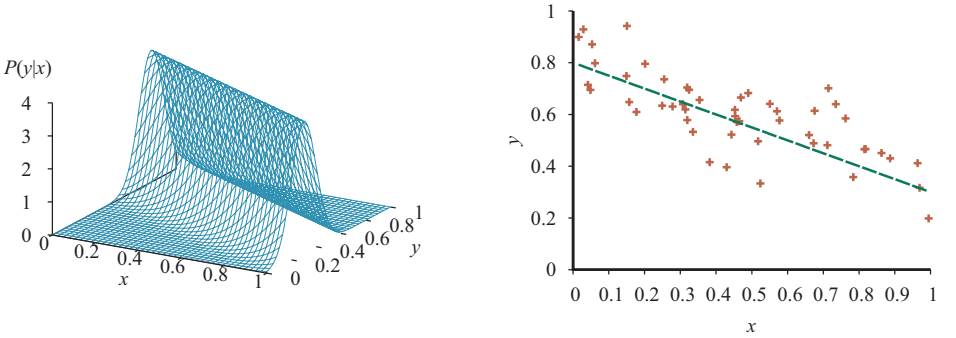
20.2.4 Aprendizaje de parámetros de máxima verosimilitud: Modelos continuos

Los modelos de probabilidad continuos como el modelo **lineal-Gaussiano** se mostraron en la página 422. Debido a que las variables continuas son ubicuas en las aplicaciones del mundo real, es importante saber cómo aprender los parámetros de los modelos continuos a partir de los datos. Los principios para el aprendizaje de máxima verosimilitud son idénticos en los casos continuos y discretos.

Comencemos con un caso muy simple: aprender los parámetros de una función de densidad gaussiana en una sola variable. Es decir, asumimos que los datos se generan de la siguiente manera:

$$P(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

Los parámetros de este modelo son la media μ y la desviación estándar σ. (Note que la "constante" de normalización depende de σ, por lo que no podemos ignorarla). Dejemos que los valores



(a) (b)

Figura 20.4 (a) Modelo lineal-Gaussiano descrito como $y = \theta_1 x + \theta_2$ más ruido Gaussiano con varianza fija. (b) Conjunto de 50 puntos de datos generados a partir de este modelo y la línea de mejor ajuste.

observados sean $x_1,...,x_N$. Entonces la log de verosimilitud es

$$L = \sum_{j=1}^{N} \log \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{(x_j-\mu)^2}{2\sigma^2}} = N(-\log\sqrt{2\pi} - \log\sigma) - \sum_{j=1}^{N} \frac{(x_j-\mu)^2}{2\sigma^2}.$$

Estableciendo las derivadas a cero como de costumbre, obtenemos

$$\begin{aligned} \frac{\partial L}{\partial \mu} &= -\frac{1}{\sigma^2}\sum_{j=1}^{N}(x_j-\mu)^2 = 0 \quad &\Rightarrow \quad \mu &= \frac{\sum_j x_j}{N} \\ \frac{\partial L}{\partial \sigma} &= -\frac{N}{\sigma} + \frac{1}{\sigma^3}\sum_{j=1}^{N}(x_j-\mu)^2 = 0 \quad &\Rightarrow \quad \sigma &= \sqrt{\frac{\sum_j (x_j-\mu)^2}{N}}. \end{aligned} \tag{20.4}$$

Es decir, el valor de máxima verosimilitud de la media es el promedio de la muestra y el valor de máxima verosimilitud de la desviación estándar es la raíz cuadrada de la varianza de la muestra. De nuevo, estos son resultados reconfortantes que confirman la práctica de "sentido común".

Ahora considere un modelo lineal-gaussiano con un padre continuo X y un hijo continuo Y. Como se explica en la página 422, Y tiene una distribución gaussiana cuya media depende de manera lineal del valor de X y cuya desviación estándar es fija. Para aprender la distribución condicional $P(Y|X)$, podemos maximizar la verosimilitud condicional

$$P(y|x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{(y-(\theta_1 x+\theta_2))^2}{2\sigma^2}}. \tag{20.5}$$

Aquí, los parámetros son θ_1, θ_2, y σ. Los datos son una colección de pares (x_j,y_j), como se ilustra en la Figura 20.4. Usando los métodos habituales (Ejercicio 20.LINR) podemos encontrar los valores de máxima verosimilitud de los parámetros. El punto aquí es diferente. Si consideramos sólo los parámetros θ_1 y θ_2 que definen la relación lineal entre x y y, queda claro que maximizar la log verosimilitud con respecto a estos parámetros es lo mismo que *minimizar* el numerador $(y - (\theta_1 x + \theta_2))^2$ en el exponente de la Ecuación (20.5). Esta es la pérdida L_2, el error cuadrado entre el valor real y y la predicción $\theta_1 x + \theta_2$.

Esta es la cantidad minimizada por el procedimiento de **regresión lineal** estándar descrito en la Sección 19.6. Ahora podemos entender por qué: minimizar la suma de errores cuadrados da el modelo de línea recta de máxima verosimilitud, *siempre que los datos se generen con ruido Gaussiano de varianza fija*.

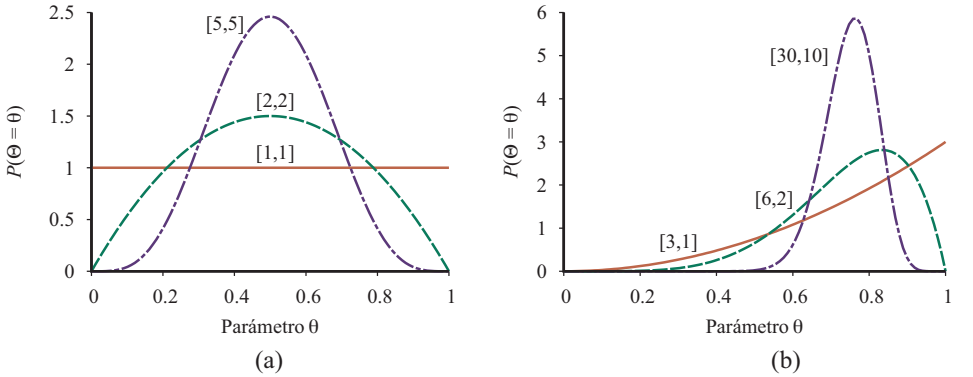


Figura 20.5 Ejemplos de la distribución *Beta*(a,b) para diferentes valores de (a,b).

20.2.3 Aprendizaje de parámetros bayesianos

El aprendizaje de máxima verosimilitud da lugar a procedimientos simples, pero tiene serias deficiencias con conjuntos de datos pequeños. Por ejemplo, después de ver un caramelo de cereza, la hipótesis de máxima verosimilitud es que la bolsa es 100 % cereza (es decir, $\theta = 1.0$). A menos que la hipótesis previa sea que las bolsas deben ser todas de cereza o todas de lima, esta no es una conclusión razonable. Es más probable que la bolsa sea una mezcla de lima y cereza. El enfoque bayesiano para el aprendizaje de parámetros comienza con una hipótesis previa y actualiza la distribución a medida que llegan los datos.

El ejemplo de los caramelos en la Figura 20.2(a) tiene un parámetro, θ: la probabilidad de que un caramelo seleccionado al azar tenga sabor a cereza. En la visión bayesiana, θ es el valor (desconocido) de una variable aleatoria Θ que define el espacio de hipótesis; la hipótesis previa es la distribución previa sobre $\mathbf{P}(\Theta)$. Así, $P(\Theta=\theta)$ es la probabilidad previa de que la bolsa tenga una fracción θ de caramelos de cereza.

Si el parámetro θ puede ser cualquier valor entre 0 y 1, entonces $\mathbf{P}(\Theta)$ es una función de densidad de probabilidad continua (ver Sección A.3). Si no sabemos nada sobre los posibles valores de θ podemos usar la función de densidad uniforme $P(\theta) = \mathit{Uniform}(\theta;0,1)$, que dice que todos los valores son igualmente probables.

Distribuciones beta
Hiperparámetros

Una familia más flexible de funciones de densidad de probabilidad es conocida como las **distribuciones beta**. Cada distribución beta está definida por dos **hiperparámetros**[3] a y b de tal manera que

$$\mathit{Beta}(\theta;a,b) = \alpha\theta^{a-1}(1-\theta)^{b-1}, \tag{20.6}$$

para θ en el rango [0,1]. La constante de normalización α, que hace que la distribución se integre a 1, depende de a y b. La Figura 20.5 muestra cómo se ve la distribución para varios valores de a y b. El valor medio de la distribución beta es $a/(a + b)$, por lo que valores más grandes de a sugieren una creencia de que Θ está más cerca de 1 que de 0. Valores más grandes de $a + b$ hacen que la distribución sea más puntiaguda, sugiriendo una mayor certeza sobre el valor de Θ. Resulta que la función de densidad uniforme es la misma que *Beta*(1,1): la media es 1/2, y la distribución es plana.

[3] Se les llama hiperparámetros porque parametrizan una distribución sobre θ, que es en sí misma un parámetro.

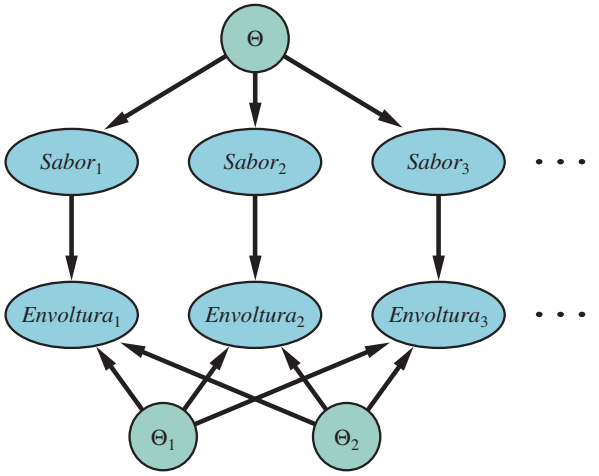


Figura 20.6 Red bayesiana que corresponde a un proceso de aprendizaje bayesiano. Las distribuciones posteriores para las variables de parámetro Θ, Θ_1 y Θ_2 pueden inferirse de sus distribuciones previas y la evidencia en $Sabor_i$ y $Envoltura_i$.

Además de su flexibilidad, la familia beta tiene otra propiedad útil : si Θ tiene un $Beta(a,b)$ previo, entonces, después de observar un punto de datos, la distribución posterior para Θ también es una distribución beta. En otras palabras, *Beta* está cerrado bajo actualización. La familia beta se llama **prior conjugado** para la familia de distribuciones para una variable booleana.[4] Veamos cómo funciona este Prior conjugado. Suponga que observamos un caramelo de cereza; entonces tenemos

Prior conjugado

$$\begin{aligned} P(\theta|D_1=cereza) &= \alpha P(D_1=cereza|\theta)P(\theta) \\ &= \alpha'\theta \cdot Beta(\theta;a,b) = \alpha'\theta \cdot \theta^{a-1}(1-\theta)^{b-1} \\ &= \alpha'\theta^a(1-\theta)^{b-1} = \alpha' Beta(\theta;a+1,b). \end{aligned}$$

Por lo tanto, después de ver un caramelo de cereza, incrementamos el parámetro a para obtener el posterior; de manera similar, después de ver un caramelo de lima, incrementamos el parámetro b. Así, podemos ver a y b hiperparámetros como **cuentas virtuales**, en el sentido de que un $Beta(a,b)$ previo se comporta exactamente como si hubiéramos comenzado con un $Beta(1,1)$ previo uniforme y visto $a-1$ caramelos de cereza reales y $b-1$ caramelos de lima reales.

Cuentas virtuales

Al examinar una secuencia de distribuciones beta para valores crecientes de a y b, manteniendo las proporciones fijas, podemos ver cómo la distribución posterior sobre el parámetro Θ cambia a medida que llegan los datos. Por ejemplo, supongamos que la bolsa de caramelos real es 75 % de cereza. La Figura 20.5(b) muestra la secuencia $Beta(3,1)$, $Beta(6,2)$, $Beta(30,10)$. Claramente, la distribución está convergiendo a un pico estrecho alrededor del verdadero valor de Θ. Para grandes conjuntos de datos, entonces, el aprendizaje bayesiano (al menos en este caso) converge a la misma respuesta que el aprendizaje de máxima verosimilitud.

Ahora consideremos un caso más complicado. La red en la Figura 20.2(b) tiene tres parámetros, θ, θ_1 y θ_2, donde θ_1 es la probabilidad de una envoltura roja en un caramelo de cereza y θ_2 es

[4] Otros priors conjugados incluyen la familia **Dirichlet** para los parámetros de una distribución multivaluada discreta y la familia **Normal–Wishart** para los parámetros de una distribución gaussiana. Ver Bernardo y Smith (1994).

la probabilidad de una envoltura roja en un caramelo de lima. El prior de hipótesis bayesiano debe cubrir los tres parámetros, es decir, necesitamos especificar $\mathbf{P}(\Theta,\Theta_1,\Theta_2)$. Por lo general, asumimos **independencia de parámetros**:

Independencia de parámetros

$$\mathbf{P}(\Theta,\Theta_1,\Theta_2) = \mathbf{P}(\Theta)\mathbf{P}(\Theta_1)\mathbf{P}(\Theta_2).$$

Con esta suposición cada parámetro puede tener su propia distribución beta que se actualiza por separado a medida que llegan los datos. La Figura 20.6 muestra cómo podemos incorporar el prior de hipótesis y cualquier dato en una red bayesiana, en la que tenemos un nodo para cada variable de parámetro.

Los nodos Θ,Θ_1,Θ_2 no tienen padres. Para la i-ésima observación de una envoltura y el sabor correspondiente de un pedazo de caramelo, agrega los nodos $Envoltura_i$ y $Sabor_i$. $Sabor_i$ depende del parámetro de sabor Θ:

$$P(Sabor_i = cereza|\Theta=\theta) = \theta.$$

$Envoltura_i$ depende de Θ_1 y Θ_2:

$$P(Envoltura_i = rojo|Sabor_i = cereza,\Theta_1=\theta_1) = \theta_1$$
$$P(Envoltura_i = rojo|Sabor_i = lima,\Theta_2=\theta_2) = \theta_2.$$

Ahora, todo el proceso de aprendizaje bayesiano para la red de Bayes original en la Figura 20.2(b) puede formularse como un problema de *inferencia* en la red de Bayes derivada mostrada en la Figura 20.6, donde los datos y los parámetros se convierten en nodos. Una vez que hemos agregado todos los nuevos nodos de evidencia, podemos consultar las variables de parámetro (en este caso, Θ,Θ_1,Θ_2). Bajo esta formulación, *sólo hay un algoritmo de aprendizaje*: el algoritmo de inferencia para redes bayesianas.

Por supuesto, la naturaleza de estas redes es algo diferente a las del Capítulo 13 debido al potencialmente enorme número de variables de evidencia que representan el conjunto de entrenamiento y la prevalencia de variables de parámetros de valor continuo. La inferencia exacta podría ser imposible excepto en casos muy simples como el modelo de Bayes ingenuo. Los profesionales suelen utilizar métodos de inferencia aproximados como MCMC (Sección 13.4.2); muchos paquetes de software estadístico incorporan implementaciones eficientes de MCMC para este propósito.

20.2.6 Regresión lineal bayesiana

Aquí ilustramos cómo aplicar un enfoque bayesiano a una tarea estadística estándar: la regresión lineal. El enfoque convencional se describió en la Sección 19.6 como la minimización de la suma de errores cuadrados y se reinterpretó en la Sección 20.2.4 como la maximización de la verosimilitud asumiendo un modelo de error gaussiano. Estos producen una única mejor hipótesis: una línea recta con valores específicos para la pendiente y la intersección y una varianza fija para el error de predicción en cualquier punto dado. No hay una medida de cuán confiado uno debería estar en los valores de la pendiente y la intersección.

Además, si uno está prediciendo un valor para un punto de datos no visto lejos de los puntos de datos observados, parece no tener sentido asumir un error de predicción que es el mismo que el error de predicción para un punto de datos justo al lado de un punto de datos observado. Parecería más sensato que el error de predicción sea mayor, cuanto más lejos esté el punto de datos de los datos observados, porque un pequeño cambio en la pendiente causará un gran cambio en el valor predicho para un punto distante.

El enfoque bayesiano soluciona ambos problemas. La idea general, como en la sección anterior, es colocar un prior en los parámetros del modelo; aquí, los coeficientes del modelo lineal y la varianza del ruido, y luego calcular el posterior de los parámetros dados los datos. Para datos

multivariados y un modelo de ruido desconocido, esto conduce a mucha álgebra lineal, por lo que nos enfocamos en un caso simple: datos de variable única, un modelo que está restringido a pasar por el origen, y ruido conocido: una distribución normal con varianza σ^2. Entonces sólo tenemos un parámetro θ y el modelo es

$$P(y|x,\theta) = \mathcal{N}(y;\theta x,\sigma_y^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{(y-\theta x)^2}{\sigma^2}\right)}. \tag{20.7}$$

Como el log de verosimilitud es cuadrática en θ, la forma apropiada para un prior conjugado en θ también es una gaussiana. Esto asegura que el posterior para θ también será gaussiano. Asumiremos una media θ_0 y una varianza σ_0^2 para el prior, de modo que

$$P(\theta) = \mathcal{N}(\theta;\theta_0,\sigma_0^2) = \frac{1}{\sigma_0\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{(\theta-\theta_0)^2}{\sigma_0^2}\right)}. \tag{20.8}$$

Dependiendo de los datos que se estén modelando, uno podría tener alguna idea de qué tipo de pendiente θ esperar, o uno podría ser completamente agnóstico. En este último caso, tiene sentido elegir θ_0 para ser 0 y σ_0^2 para ser grande, un llamado **prior no informativo**. Finalmente, podemos asumir un prior no informativo prior $P(x)$ para el valor x de cada punto de datos, pero esto es completamente inmaterial para el análisis porque no depende de θ.

Prior no informativo

Ahora la configuración está completa, por lo que podemos calcular el posterior para θ usando la Ecuación (20.1): $P(\theta|\mathbf{d}) \propto P(\mathbf{d}|\theta)P(\theta)$. Los puntos de datos observados son $\mathbf{d}=(x_1,y_1),\ldots,(x_N,y_N)$, por lo que la verosimilitud para los datos se obtiene de la Ecuación (20.7) de la siguiente manera:

$$\begin{aligned} P(\mathbf{d}|\theta) &= \left(\prod_i P(x_i)\right) \prod_i P(y_i|x_i,\theta) = \alpha \prod_i e^{-\frac{1}{2}\left(\frac{(y_i-\theta x_i)^2}{\sigma^2}\right)} \\ &= \alpha e^{-\frac{1}{2}\sum i\left(\frac{(y_i-\theta x_i)^2}{\sigma^2}\right)}, \end{aligned}$$

donde hemos absorbido los priors de valor x y las constantes de normalización para las N gaussianas en una constante α que es independiente de θ. Ahora combinamos esto y el prior del parámetro de la Ecuación (20.8) para obtener el posterior:

$$P(\theta|\mathbf{d}) = \alpha'' e^{-\frac{1}{2}\left(\frac{(\theta-\theta_0)^2}{\sigma_0^2}\right)} e^{-\frac{1}{2}\sum i\left(\frac{(y_i-\theta x)^2}{\sigma^2}\right)}$$

Aunque esto parece complicado, cada exponente es una función cuadrática de θ, por lo que la suma de los dos exponentes también lo es. Por lo tanto, toda la expresión representa una distribución gaussiana para θ. Usando manipulaciones algebraicas muy similares a las de la Sección 14.4, encontramos

$$P(\theta|\mathbf{d}) = \alpha''' e^{-\frac{1}{2}\left(\frac{(\theta-\theta_N)^2}{\sigma_N^2}\right)}$$

con media y varianza "actualizadas" dadas por

$$\theta_N = \frac{\sigma^2\theta_0 + \sigma_0^2\sum_i x_i y_i}{\sigma^2 + \sigma_0^2\sum_i x_i^2} \quad \text{y} \quad \sigma_N^2 = \frac{\sigma^2\sigma_0^2}{\sigma^2 + \sigma_0^2\sum_i x_i^2}.$$

Analicemos estas fórmulas para ver qué significan. Cuando los datos están estrechamente concentrados en una pequeña región del eje x cerca del origen, $\sum_i x_i^2$ será pequeño y la varianza posterior σ_N^2 será grande, de manera aproximada igual a la varianza previa σ_0^2. Esto es lo que uno esperaría: los datos hacen poco para restringir la rotación de la línea alrededor del origen. Al contrario, cuando los datos están ampliamente distribuidos a lo largo del eje, $\sum_i x_i^2$ será grande y la varianza posterior σ_N^2 será pequeña, aproximadamente igual a $\sigma^2/(\sum_i x_i^2)$, por lo que la pendiente estará muy restringida.

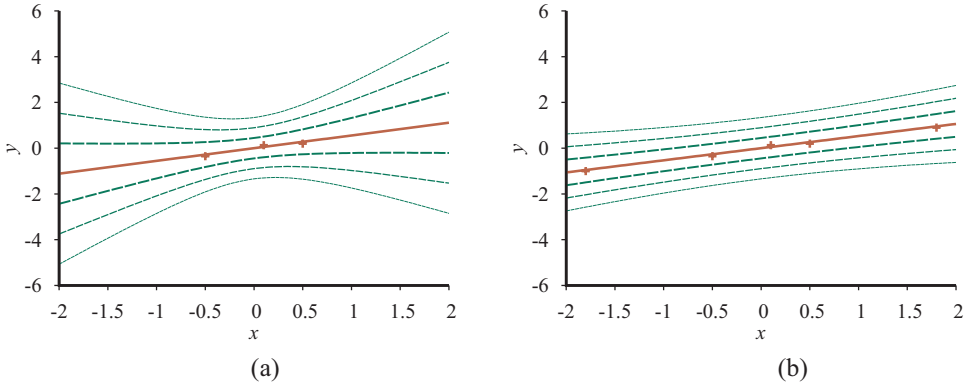


Figura 20.7 Regresión lineal bayesiana con un modelo restringido a pasar por el origen y varianza de ruido fija $\sigma^2 = 0.2$. Se muestran contornos a ±1, ±2 y ±3 desviaciones estándar para la densidad predictiva. (a) Con tres puntos de datos cerca del origen, la pendiente es muy incierta, con $\sigma_N^2 \approx 0.3861$. Observe cómo la incertidumbre aumenta con la distancia desde los puntos de datos observados. (b) Con dos puntos de datos adicionales más alejados, la pendiente θ está muy estrechamente restringida, con $\sigma_N^2 \approx 0.0286$. La varianza restante en la densidad predictiva se debe casi enteramente al ruido fijo σ^2.

Para hacer una predicción en un punto de datos específico, tenemos que integrar sobre los posibles valores de θ, como sugiere la Ecuación (20.2):

$$P(y|x,\mathbf{d}) = \int_{-\infty}^{\infty} P(y|x,\mathbf{d},\theta)P(\theta|x,\mathbf{d}) = \int_{-\infty}^{\infty} P(y|x,\theta)P(\theta|\mathbf{d})d\theta$$
$$= \alpha \int_{-\infty}^{\infty} e^{-\frac{1}{2}\left(\frac{(y-\theta x)^2}{\sigma^2}\right)} e^{-\frac{1}{2}\left(\frac{(\theta-\theta_N)^2}{\sigma_N^2}\right)} d\theta$$

De nuevo, la suma de los dos exponentes es una función cuadrática de θ, por lo que tenemos una gaussiana sobre θ cuya integral es 1. Los términos restantes en y forman otra gaussiana:

$$P(y|x,\mathbf{d}) \propto e^{-\frac{1}{2}\left(\frac{(y-\theta_N x)^2}{\sigma^2+\sigma_N^2 x^2}\right)} .$$

Al observar esta expresión, vemos que la predicción media para y es $\theta_N x$, es decir, se basa en la media posterior para θ. La varianza de la predicción está dada por el ruido del modelo σ^2 más un término proporcional a x^2, lo que significa que la desviación estándar de la predicción aumenta asintóticamente de manera lineal con la distancia desde el origen. La Figura 20.7 ilustra este fenómeno. Como se señaló al principio de esta sección, tener mayor incertidumbre para las predicciones que están más alejadas de los puntos de datos observados tiene mucho sentido.

20.2.7 Aprendiendo estructuras de redes de Bayes

Hasta ahora, hemos asumido que la estructura de la red bayesiana está dada y sólo estamos tratando de aprender los parámetros. La estructura de la red representa un conocimiento causal básico sobre el dominio que en general es fácil de suministrar para un experto, o incluso un usuario ingenuo. Sin embargo, en algunos casos el modelo causal puede no estar disponible o estar sujeto a disputa; por ejemplo, ciertas corporaciones han afirmado durante mucho

tiempo que fumar no causa cáncer, mientras que otras corporaciones afirman que las concentraciones de CO_2 no tienen efecto sobre el clima, por lo que es importante entender cómo se puede aprender la estructura de una red bayesiana a partir de datos. Esta sección ofrece un breve esbozo de las ideas principales.

El enfoque más obvio es *buscar* un buen modelo. Podemos comenzar con un modelo que no contenga enlaces y comenzar a agregar padres para cada nodo, ajustando los parámetros con los métodos que acabamos de cubrir y midiendo la precisión del modelo resultante. De manera alternativa, podemos iniciar con una suposición inicial sobre la estructura y usar la búsqueda de ascenso de colinas o el recocido simulado para hacer modificaciones, reajustando los parámetros después de cada cambio en la estructura. Las modificaciones pueden incluir invertir, agregar o eliminar enlaces. No debemos introducir ciclos en el proceso, por lo que muchos algoritmos asumen que se da un orden para las variables, y que un nodo permite tener padres sólo entre aquellos nodos que aparecen antes en el orden (tal como en el proceso de construcción en el Capítulo 13). Para una generalidad completa, también necesitamos buscar posibles ordenamientos.

Existen dos métodos alternativos para decidir cuándo se ha encontrado una buena estructura. El primero es probar si las afirmaciones de independencia condicional implícitas en la estructura se cumplen en los datos. Por ejemplo, el uso de un modelo bayesiano ingenuo para el problema del restaurante asume que

$$\mathbf{P}(\mathit{Hambriento}, \mathit{Bar} | \mathit{Esperará}) = \mathbf{P}(\mathit{Hambriento} | \mathit{Esperará})\mathbf{P}(\mathit{Bar} | \mathit{Esperará})$$

y podemos verificar en los datos si la misma ecuación se mantiene entre las frecuencias condicionales correspondientes. Aun si la estructura describe la verdadera naturaleza causal del dominio, las fluctuaciones estadísticas en el conjunto de datos significan que la ecuación nunca se cumplirá *exactamente*, por lo que necesitamos realizar una prueba estadística adecuada para ver si hay suficiente evidencia de que se viola la hipótesis de independencia. La complejidad de la red resultante dependerá del umbral utilizado para esta prueba, cuanto más estricta sea la prueba de independencia, se agregarán más enlaces y mayor será el peligro de sobreajuste.

Un enfoque más consistente con las ideas de este capítulo es evaluar el grado en que el modelo propuesto explica los datos (en un sentido probabilístico). Sin embargo, debemos tener cuidado con cómo medimos esto. Si sólo intentamos encontrar la hipótesis de máxima verosimilitud, terminaremos con una red completamente conectada, porque agregar más padres a un nodo no puede disminuir la verosimilitud (Ejercicio 20.MLPA). Nos vemos obligados a penalizar la complejidad del modelo de alguna manera. El enfoque MAP (o MDL) resta una penalización de la verosimilitud de cada estructura (después de la sintonización de parámetros) antes de comparar diferentes estructuras. El enfoque bayesiano coloca un prior conjunto sobre estructuras y parámetros. Por lo general, hay demasiadas estructuras para sumar (superexponencial en el número de variables), por lo que la mayoría de los profesionales utilizan MCMC para muestrear sobre estructuras.

Penalizar la complejidad (por métodos MAP o bayesianos) introduce una conexión importante entre la estructura óptima y la naturaleza de la representación para las distribuciones condicionales en la red. Con distribuciones tabulares, la penalización de complejidad para la distribución de un nodo crece de manera exponencial con el número de padres, pero con, digamos, distribuciones OR ruidosas, crece sólo de manera lineal. Esto significa que el aprendizaje con modelos OR ruidosos (u otros modelos parametrizados de manera compacta) tiende a producir estructuras aprendidas con más padres que el aprendizaje con distribuciones tabulares.

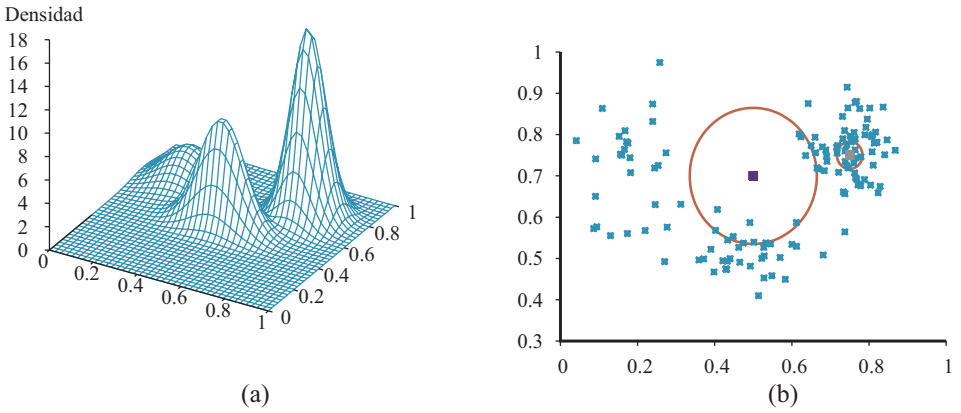


Figura 20.8 (a) Grafo 3D de la mezcla de gaussianas de la Figura 20.12(a). (b) Muestra de 128 puntos de la mezcla, junto con dos puntos de consulta (pequeños cuadrados naranjas) y sus 10 vecindarios más cercanos (círculo grande y círculo más pequeño a la derecha).

20.2.8 Estimación de densidad con modelos no paramétricos

Estimación de densidad no paramétrica

Es posible aprender un modelo de probabilidad sin hacer ninguna suposición sobre su estructura y parametrización adoptando los métodos no paramétricos de la Sección 19.7. La tarea de **estimación de densidad no paramétrica** se realiza en general en dominios continuos, como el que se muestra en la Figura 20.8(a). La figura muestra una función de densidad de probabilidad en un espacio definido por dos variables continuas. En la Figura 20.8(b) vemos una muestra de puntos de datos de esta función de densidad. La pregunta es, ¿podemos recuperar el modelo a partir de las muestras?

Primero consideraremos modelos de ***k*-vecinos más cercanos**. (En el Capítulo 19 vimos modelos de vecinos más cercanos para clasificación y regresión; aquí los vemos para estimación de densidad). Dada una muestra de puntos de datos, para estimar la densidad de probabilidad desconocida en un punto de consulta $\mathbf{x}$ podemos medir la densidad de los puntos de datos en el vecindario de $\mathbf{x}$. La Figura 20.8(b) muestra dos puntos de consulta (pequeños cuadrados). Para cada punto de consulta hemos dibujado el círculo más pequeño que encierra 10 vecinos, el vecindario de los 10 vecinos más cercanos. Podemos ver que el círculo central es grande, lo que significa que hay una baja densidad allí, y el círculo de la derecha es pequeño, lo que significa que hay una alta densidad allí. En la Figura 20.9 mostramos tres grafos de estimación de densidad usando *k*-vecinos más cercanos, para diferentes valores de *k*. Parece claro que (b) es correcto, mientras que (a) es demasiado puntiagudo (*k* es demasiado pequeño) y (c) es muy suave (*k* es demasiado grande).

Otra posibilidad es usar **funciones de kernel**, como lo hicimos para la regresión ponderada de manera local. Para aplicar un modelo de kernel a la estimación de densidad, suponga que cada punto de datos genera su propia pequeña función de densidad. Por ejemplo, podríamos usar gaussianas esféricas con desviación estándar w a lo largo de cada eje. Entonces la densidad estimada en un punto de consulta $\mathbf{x}$ es el promedio de los kernels de datos:

$$P(\mathbf{x}) = \frac{1}{N}\sum_{j=i}^{N} \mathcal{K}(\mathbf{x},\mathbf{x}_j) \qquad \text{donde} \qquad \mathcal{K}(\mathbf{x},\mathbf{x}_j) = \frac{1}{(w^2\sqrt{2\pi})^d} e^{-\frac{D(\mathbf{x},\mathbf{x}_j)^2}{2w^2}},$$

donde d es el número de dimensiones en $\mathbf{x}$ y D es la función de distancia euclidiana. Nosotros todavía tenemos el problema de elegir un valor adecuado para la anchura del kernel w; la Figura

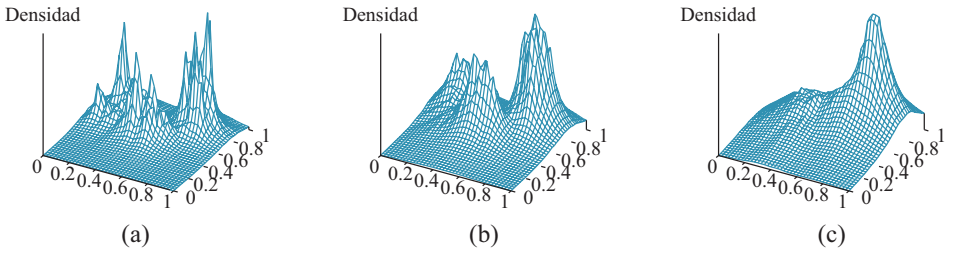


Figura 20.9 Estimación de densidad utilizando k-vecinos más cercanos, aplicada a los datos en la Figura 20.8(b), para $k = 3$, 10, y 40, respectivamente. $k = 3$ es demasiado puntiagudo, 40 es demasiado suave, y 10 es justo lo correcto. El mejor valor para k puede ser elegido por validación cruzada.

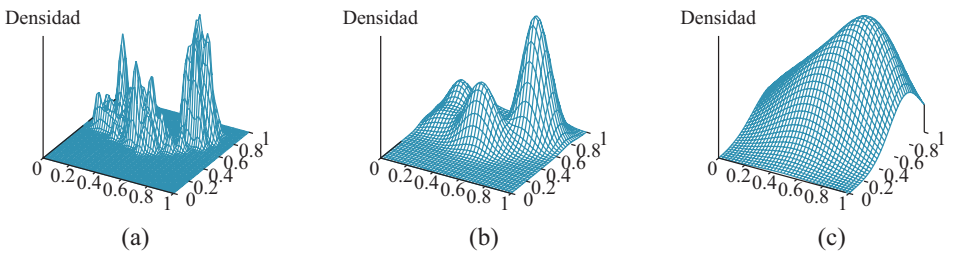


Figura 20.10 Estimación de la densidad utilizando kernels para los datos en la Figura 20.8(b), utilizando kernels gaussianos con $w = 0.02$, 0.07, y 0.20, respectivamente. $w = 0.07$ es correcto.

20.10 muestra valores que son demasiado pequeños, justos y demasiado grandes. Un buen valor de w puede ser elegido usando la validación cruzada.

20.3 Aprendizaje con variables ocultas: el algoritmo EM

La sección anterior trataba con el caso completamente observable. Muchos problemas del mundo real tienen **variables ocultas** (a veces llamadas **variables latentes**), que no son observables en los datos. Por ejemplo, los registros médicos con frecuencia incluyen los síntomas observados, el diagnóstico del médico, el tratamiento aplicado y, quizás, el resultado del tratamiento, pero ¡rara vez contienen una observación directa de la enfermedad en sí! (Note que el *diagnóstico* no es la *enfermedad*; es una consecuencia causal de los síntomas observados, que a su vez son causados por la enfermedad). Uno podría preguntar, "Si la enfermedad no se observa, ¿podríamos construir un modelo basado sólo en las variables observadas?" La respuesta aparece en la Figura 20.11, que muestra un pequeño modelo de diagnóstico ficticio para una enfermedad del corazón. Hay tres factores predisponentes observables y tres síntomas observables (que son demasiado deprimentes para nombrar). Suponga que cada variable tiene tres posibles valores (por ejemplo, *ninguno*, *moderado*, y *severo*). Eliminando la variable oculta de la red en

Variables latentes

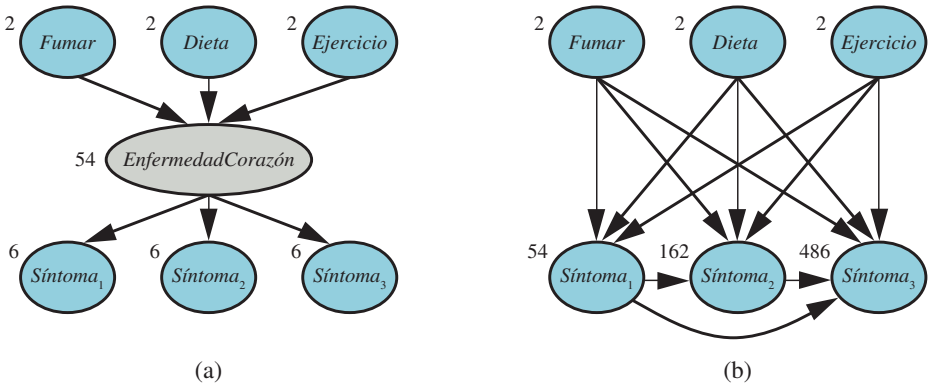


Figura 20.11 (a) Red de diagnóstico para la enfermedad del corazón, que se supone es una variable oculta. Cada variable tiene tres posibles valores y está etiquetada con el número de parámetros independientes en su distribución condicional; el número total es 78. (b) Red equivalente con *EnfermedadCorazón* eliminado. Note que las variables de síntoma ya no son condicionalmente independientes dadas sus padres. Esta red requiere 708 parámetros.

(a) produce la red en (b); el número total de parámetros aumenta de 78 a 708. Así, *las variables latentes permiten reducir de manera drástica el número de parámetros requeridos para especificar una red bayesiana*. Esto, a su vez, hace posible reducir la cantidad de datos necesarios para aprender los parámetros.

Las variables ocultas son importantes, pero complican el problema de aprendizaje. En la Figura 20.11(a), por ejemplo, no es obvio cómo aprender la distribución condicional para *EnfermedadCorazón*, dado sus padres, porque no conocemos el valor de *EnfermedadCorazón* en cada caso; el mismo problema surge en el aprendizaje de las distribuciones para los síntomas. Esta sección describe un algoritmo llamado **expectativa-maximización**, o EM, que resuelve este problema de manera muy general. Mostraremos tres ejemplos y luego proporcionaremos una descripción general. El algoritmo parece magia al principio, pero una vez que se ha desarrollado la intuición, es posible encontrar aplicaciones para EM en una gran variedad de problemas de aprendizaje.

Expectativa-maximización

20.3.1 Agrupamiento no supervisado: aprendiendo mezclas de gaussianas

Agrupamiento no supervisado

El **agrupamiento no supervisado** es el problema de discernir múltiples categorías en una colección de objetos. El problema es no supervisado porque no se dan las etiquetas de categoría. Por ejemplo, suponga que registramos los espectros de cien mil estrellas; ¿hay diferentes *tipos* de estrellas revelados por los espectros? Y, si es así, ¿cuántos *tipos* y cuáles son sus características? Todos estamos familiarizados con términos como "gigante roja" y "enana blanca", pero las estrellas no llevan estas etiquetas en sus sombreros, los astrónomos tuvieron que realizar un agrupamiento no supervisado para identificar estas categorías. Otros ejemplos incluyen la identificación de especies, géneros, órdenes, filos, etc., en la taxonomía de Linneo y en la creación de tipos naturales para objetos ordinarios (ver Capítulo 10).

El agrupamiento no supervisado comienza con datos. La Figura 20.12(b) muestra 500 puntos de datos, cada uno de los cuales especifica los valores de dos atributos continuos. Los puntos de datos podrían corresponder a estrellas, mientras que los atributos corresponderían a intensidades

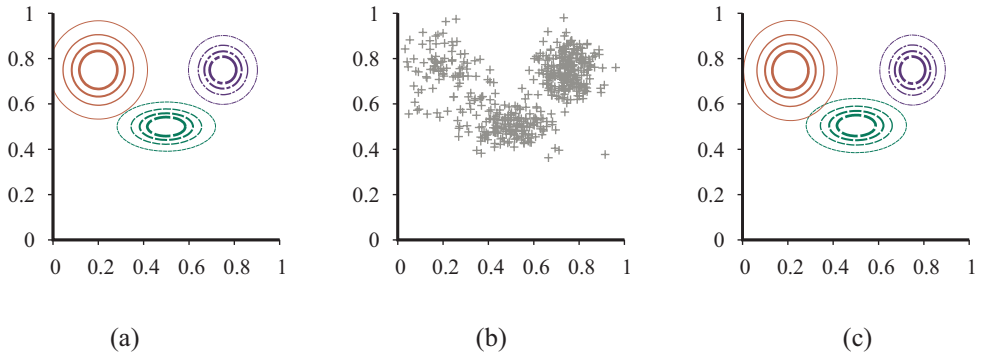


Figura 20.12 (a) Un modelo de mezcla gaussiana con tres componentes; los pesos (de izquierda a derecha) son 0.2, 0.3, y 0.5. (b) 500 puntos de datos muestreados del modelo en (a). (c) El modelo reconstruido por EM a partir de los datos en (b).

espectrales en dos frecuencias particulares. A continuación, necesitamos entender qué tipo de distribución de probabilidad podría haber generado los datos. El agrupamiento presupone que los datos se generan a partir de una **distribución de mezcla**, *P*. Tal distribución de mezcla tiene una *k* **componentes**, cada uno de los cuales es una distribución en sí mismo. Un punto de datos se genera eligiendo primero un componente y luego generando una muestra de ese componente. Dejemos que la variable aleatoria *C* denote el componente, con valores 1,...,*k*; entonces la distribución de mezcla se da por

Distribución de mezcla
Componentes

$$P(\mathbf{x}) = \sum_{i=1}^{k} \mathbf{P}(C=i)\mathbf{P}(\mathbf{x}|C=i),$$

donde **x** se refiere a los valores de los atributos para un punto de datos. Para datos continuos, una elección natural para las distribuciones de componentes es la gaussiana multivariante, que da la llamada **mezcla de gaussianas** de familias de distribuciones. Los parámetros de una mezcla de gaussianas son $w_i = P(C=i)$ (el peso de cada componente), μ_i (la media de cada componente), y Σ_i (la covarianza de cada componente). La Figura 20.12(a) muestra una mezcla de tres gaussianas; esta mezcla es de hecho la fuente de los datos en (b) así como el modelo mostrado en la Figura 20.8(a) en la página 736.

Mezcla de gaussianas

El problema del agrupamiento no supervisado, entonces, es recuperar un modelo de mezcla gaussiana como el de la Figura 20.12(a) a partir de datos brutos como los de la Figura 20.12(b). Claramente, si *supiéramos* qué componente generó cada punto de datos, sería fácil recuperar las gaussianas de los componentes: podríamos seleccionar todos los puntos de datos de un componente dado y luego aplicar (una versión multivariante de) la Ecuación (20.4) (página 729) para ajustar los parámetros de una gaussiana a un conjunto de datos. Por otro lado, si *conociéramos* los parámetros de cada componente, entonces podríamos, al menos en sentido probabilístico, asignar cada punto de datos a un componente.

El problema es que no conocemos ni las asignaciones ni los parámetros. La idea básica de EM en este contexto es *fingir* que conocemos los parámetros del modelo y luego inferir la probabilidad de que cada punto de datos pertenezca a cada componente. Después de eso, ajustamos los componentes a los datos, donde cada componente se ajusta al conjunto de datos completo con cada punto ponderado por la probabilidad de que pertenezca a ese componente. El proceso se repite hasta la convergencia. En esencia, estamos "completando" los datos al inferir distribuciones

de probabilidad sobre las variables ocultas, a qué componente pertenece cada punto de datos, con base en el modelo actual. Para la mezcla de gaussianas, inicializamos los parámetros del modelo de mezcla de manera arbitraria y luego iteramos los siguientes dos pasos:

1. **Paso E**: Calcule las probabilidades $p_{ij} = P(C=i|\mathbf{x}_j)$, la probabilidad de que el dato $\mathbf{x}_j$ fue generado por el componente i. Por la regla de Bayes, tenemos $p_{ij} = \alpha P(\mathbf{x}_j|C=i)P(C=i)$. El término $P(\mathbf{x}_j|C=i)$ es la probabilidad en $\mathbf{x}_j$ de la i-ésima gaussiana, y el término $P(C=i)$ es el parámetro de peso para la i-ésima gaussiana. Defina $n_i = \sum_j p_{ij}$, el número efectivo de puntos de datos actualmente asignados al componente i.
2. **Paso M**: Calcule la nueva media, covarianza y pesos de los componentes usando los siguientes pasos en secuencia:

$$\mu_i \leftarrow \sum_j p_{ij}\mathbf{x}_j / n_i$$
$$\Sigma_i \leftarrow \sum_j p_{ij}(\mathbf{x}_j - \mu_i)(\mathbf{x}_j - \mu_i)^\top / n_i$$
$$w_i \leftarrow n_i / N$$

donde N es el número total de puntos de datos. El paso E, o paso de *expectativa*, puede considerarse como el cálculo de los valores esperados p_{ij} de las **variables indicadoras** ocultas Z_{ij}, donde Z_{ij} es 1 si el dato $\mathbf{x}_j$ fue generado por el i-ésimo componente y 0 en caso contrario. El paso M, o paso de *maximización*, encuentra los nuevos valores de los parámetros que maximizan la log-verosimilitud de los datos, dada la expectativa de los valores de las variables indicadoras ocultas.

Variables indicadoras

El modelo final que EM aprende cuando se aplica a los datos en la Figura 20.12(a) se muestra en la Figura 20.12(c); es prácticamente indistinguible del modelo original a partir del cual se generaron los datos (línea horizontal). La Figura 20.13(a) traza la log-verosimilitud de los datos según el modelo actual a medida que avanza EM.

Hay dos puntos a tener en cuenta. Primero, la log-verosimilitud para el modelo aprendido final *supera* ligeramente la del modelo original, a partir del cual se generaron los datos. Esto podría parecer sorprendente, pero refleja el hecho de que los datos se generaron de manera aleatoria y podrían no proporcionar una reflexión exacta del modelo subyacente. El segundo punto es que *EM aumenta la log-verosimilitud de los datos en cada iteración*. Este hecho permite ser probado en general. Además, bajo ciertas condiciones (que se mantienen en la mayoría de los casos), es posible demostrar que EM alcanza un máximo local en verosimilitud. (En casos raros, podría alcanzar un punto de silla o incluso un mínimo local). En este sentido, EM se asemeja a un algoritmo de ascenso de colinas basado en gradientes, pero observe que no tiene un parámetro de "tamaño de paso".

Las cosas no siempre van tan bien como podría sugerir la Figura 20.13(a). Puede suceder, por ejemplo, que un componente gaussiano se encoja de tal manera que sólo cubra un único punto de datos. ¡Entonces su varianza irá a cero y su verosimilitud irá al infinito! Si no sabemos cuántos componentes hay en la mezcla, tenemos que probar diferentes valores de k y ver cuál es el mejor; eso puede ser una fuente de error. Otro problema es que dos componentes permiten "fusionarse", adquiriendo medias y varianzas idénticas y compartiendo sus puntos de datos. Estos tipos de máximos locales degenerados son problemas serios, en especial en dimensiones altas. Una solución es colocar priors en los parámetros del modelo y aplicar la versión MAP de EM. Otra es reiniciar un componente con nuevos parámetros aleatorios si se vuelve demasiado pequeño o demasiado cercano a otro componente. La inicialización sensata también ayuda.

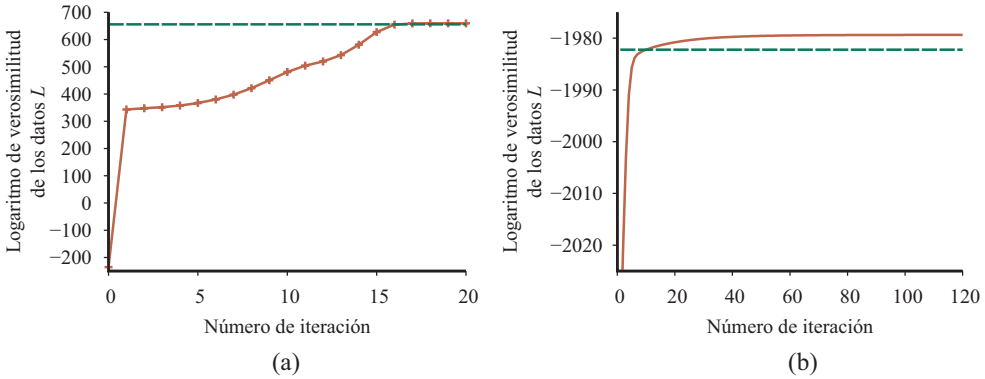


Figura 20.13 Grafos que muestran el logaritmo de la verosimilitud de los datos, L, como una función de la iteración EM. La línea horizontal muestra el logaritmo de la verosimilitud según el modelo verdadero. (a) Grafo para el modelo de mezcla gaussiana en la Figura 20.12. (b) Grafo para la red bayesiana en la Figura 20.14(a).

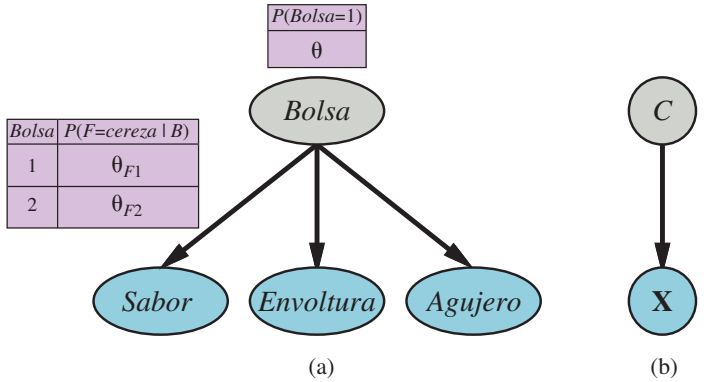


Figura 20.14 (a) Un modelo de mezcla para caramelos. Las proporciones de diferentes sabores, envolturas y presencia de agujeros en el caramelo dependen de la bolsa, que no se observa. (b) Red bayesiana para una mezcla gaussiana. La media y la covarianza de las variables observables **X** dependen del componente C.

20.3.2 Aprendiendo los valores de los parámetros de la red Bayes para variables ocultas

Para aprender una red bayesiana con variables ocultas, aplicamos las mismas ideas que funcionaron para las mezclas de gaussianas. La Figura 20.14(a) representa una situación en la que hay dos bolsas de caramelos que se han mezclado. Los caramelos se describen por tres características: además del *Sabor* y *Envoltura*, algunos caramelos tienen un *Agujero* en el medio y otros no. La distribución de caramelos en cada bolsa se describe por un modelo **bayesiano ingenuo**: las características son independientes, dada la bolsa, pero la distribución de probabilidad condicional para

cada característica depende de la bolsa. Los parámetros son los siguientes: θ es la probabilidad *a priori* de que un caramelo provenga de la Bolsa 1; θ_{F1} y θ_{F2} son las probabilidades de que el sabor sea cereza, dado que el caramelo proviene de la Bolsa 1 o la Bolsa 2, respectivamente; θ_{W1} y θ_{W2} dan las probabilidades de que la envoltura sea roja; y θ_{H1} y θ_{H2} dan las probabilidades de que el caramelo tenga un agujero.

El modelo general es un modelo de mezcla: una suma ponderada de dos distribuciones diferentes, cada una de las cuales es un producto de distribuciones univariantes independientes. (De hecho, también podemos modelar la mezcla de gaussianas como una red bayesiana, como se muestra en la Figura 20.14(b).) En la figura, la bolsa es una variable oculta porque, una vez que los caramelos se han mezclado, ya no sabemos de qué bolsa proviene cada caramelo. En tal caso, ¿podemos recuperar las descripciones de las dos bolsas observando caramelos de la mezcla? Trabajemos a través de una iteración de EM para este problema. Primero, veamos los datos. Generamos 1 000 muestras de un modelo cuyos parámetros verdaderos son los siguientes:

$$\theta = 0.5, \theta_{F1} = \theta_{W1} = \theta_{H1} = 0.8, \theta_{F2} = \theta_{W2} = \theta_{H2} = 0.3. \tag{20.9}$$

Es decir, los caramelos tienen igual probabilidad de provenir de cualquier bolsa; la primera es en su mayoría cereza con envoltorios rojos y agujeros; la segunda es en su mayoría lima con envoltorios verdes y sin agujeros. Los recuentos para los ocho posibles tipos de caramelos son los siguientes:

	W = *rojo*		W = *verde*	
	$H = 1$	$H = 0$	$H = 1$	$H = 0$
F = *cereza*	273	93	104	90
F = lima	79	100	94	167

Comenzamos inicializando los parámetros. Por simplicidad numérica, elegimos de manera arbitraria[5]

$$\theta^{(0)} = 0.6, \quad \theta_{F1}^{(0)} = \theta_{W1}^{(0)} = \theta_{H1}^{(0)} = 0.6, \quad \theta_{F2}^{(0)} = \theta_{W2}^{(0)} = \theta_{H2}^{(0)} = 0.4. \tag{20.10}$$

Primero, trabajemos en el parámetro θ. En el caso completamente observable, estimaríamos esto de manera directa a partir de los recuentos *observados* de caramelos de las bolsas 1 y 2. Como la bolsa es una variable oculta, calculamos los recuentos *esperados* en su lugar. El recuento esperado $\hat{N}(Bolsa = 1)$ es la suma, sobre todos los caramelos, de la probabilidad de que el caramelo provenga de la bolsa 1:

$$\theta^{(1)} = \hat{N}(Bolsa = 1)/N = \sum_{j=i}^{N} P(Bolsa = 1 | sabor_j, envoltura_j, agujeros_j)/N.$$

Estas probabilidades pueden ser calculadas por cualquier algoritmo de inferencia para redes bayesianas. Para un modelo bayesiano ingenuo como el de nuestro ejemplo, podemos hacer la inferencia "a mano", utilizando la regla de Bayes y aplicando la independencia condicional:

$$\theta^{(1)} = \frac{1}{N} \sum_{j=1}^{N} \frac{\boldsymbol{P}(sabor_j|bolsa{=}1)\boldsymbol{P}(envoltura_j|bolsa{=}1)\boldsymbol{P}(agujeros_j|bolsa{=}1)\boldsymbol{P}(bolsa{=}1)}{\sum_i \boldsymbol{P}(sabor_j|bolsa{=}i)\boldsymbol{P}(envoltura_j|bolsa{=}i)\boldsymbol{P}(agujeros_j|bolsa{=}i)\boldsymbol{P}(bolsa{=}i)}$$

Aplicando esta fórmula a, digamos, los 273 caramelos de cereza con envoltura roja y agujeros, obtenemos una contribución de

$$\frac{273}{1\,000} \cdot \frac{\theta_{F1}^{(0)}\theta_{W1}^{(0)}\theta_{H1}^{(0)}\theta^{(0)}}{\theta_{F1}^{(0)}\theta_{W1}^{(0)}\theta_{H1}^{(0)}\theta^{(0)} + \theta_{F2}^{(0)}\theta_{W2}^{(0)}\theta_{H2}^{(0)}(1-\theta^{(0)})} \approx 0.22797.$$

[5] En la práctica es mejor elegirlos al azar para evitar máximos locales debido a la simetría.

Continuando con los otros siete tipos de caramelos en la tabla de recuentos, obtenemos $\theta^{(1)} = 0.6124$.

Ahora considere los otros parámetros, como θ_{F1}. En el caso completamente observable, estimaríamos esto directamente a partir de los recuentos *observados* de caramelos de cereza y lima de la Bolsa 1. El recuento *esperado* de caramelos de cereza de la Bolsa 1 se da por

$$\sum_{j:Sabor_j=cereza} P(Bolsa=1|Sabor_j=cereza, envoltura_j, agujeros_j).$$

De nuevo, estas probabilidades pueden ser calculadas por cualquier algoritmo de red bayesiana. Completando este proceso, obtenemos los nuevos valores de todos los parámetros:

$$\begin{aligned} &\theta^{(1)} = 0.6124, \quad \theta_{F1}^{(1)} = 0.6684, \theta_{W1}^{(1)} = 0.6483, \theta_{H1}^{(1)} = 0.6558, \\ &\theta_{F2}^{(1)} = 0.3887, \quad \theta_{W2}^{(1)} = 0.3817, \theta_{H2}^{(1)} = 0.3827. \end{aligned} \tag{20.11}$$

La log-verosimilitud de los datos aumenta desde unos −2 044 inicialmente a cerca de −2 021 después de la primera iteración, como se muestra en la Figura 20.13(b). Es decir, la actualización mejora la verosimilitud en sí misma por un factor de casi $e^{23} \approx 10^{10}$. En la décima iteración, el modelo aprendido se ajusta mejor que el modelo original ($L = -1982.214$). A partir de entonces, el progreso se vuelve muy lento. Esto no es inusual con EM, y muchos sistemas prácticos combinan EM con un gradiente basado en algoritmo como Newton-Raphson (ver Capítulo 4) para la última fase de aprendizaje.

La lección general de este ejemplo es que *las actualizaciones de parámetros para el aprendizaje de redes bayesianas con variables ocultas están directamente disponibles a partir de los resultados de la inferencia en cada ejemplo. Además, sólo se necesitan probabilidades posteriores locales para cada parámetro*. Aquí, "local" significa que la tabla de probabilidad condicional (CPT) para cada variable X_i se puede aprender a partir de probabilidades posteriores que involucran sólo X_i y sus padres $\mathbf{U}_i$. Definiendo θ_{ijk} como el parámetro CPT, $P(X_i = x_{ij}|\mathbf{U}_i=\mathbf{u}_{ik})$, la actualización se da por los recuentos esperados normalizados de la siguiente manera:

$$\theta_{jk} \rightarrow \hat{N}(X_i = x_{ij}, \mathbf{U}_i = \mathbf{u}_{ik})/\hat{N}(\mathbf{U}_i = \mathbf{u}_{ik}).$$

Los recuentos esperados se obtienen sumando sobre los ejemplos, calculando las probabilidades $P(X_i = x_{ij}, \mathbf{U}_i = \mathbf{u}_{ik})$ para cada uno utilizando cualquier algoritmo de inferencia de red Bayesiana. Para los algoritmos exactos, incluida la eliminación de variables, todas estas probabilidades se pueden obtener de manera directa como un subproducto de la inferencia estándar, sin necesidad de cálculos adicionales específicos para el aprendizaje. Además, la información necesaria para el aprendizaje está disponible *de manera local* para cada parámetro.

Dando un paso atrás, podemos pensar en lo que el algoritmo EM está haciendo en este ejemplo como recuperar siete parámetros (θ, θ_{F1}, θ_{W1}, θ_{H1}, θ_{F2}, θ_{W2}, θ_{H2}) de siete ($2^3 - 1$) recuentos observados en los datos. (El octavo recuento está fijado por el hecho de que los recuentos suman 1 000). Si cada caramelo fuera descrito por dos atributos en lugar de tres (digamos, omitiendo los agujeros), habríamos tenido cinco parámetros (θ, θ_{F1}, θ_{W1}, θ_{F2}, θ_{W2}) pero sólo tres ($2^2 - 1$) recuentos observados. En tal caso, no es posible recuperar el peso de la mezcla θ o las características de las dos bolsas que se mezclaron. Decimos entonces que el modelo de dos atributos es no **identificable**.

Identificable

La identificabilidad en las redes bayesianas es un tema complicado. Note que incluso con tres atributos y siete recuentos, no podemos recuperar de manera única el modelo, porque hay dos modelos equivalentes a nivel observacional con la variable *Bolsa* invertida. Dependiendo de cómo se inicialicen los parámetros, EM convergerá ya sea a un modelo donde la bolsa 1 tiene principalmente cereza y la bolsa 2 principalmente lima, o viceversa. Este tipo de no identificabilidad es inevitable con variables que nunca se observan.

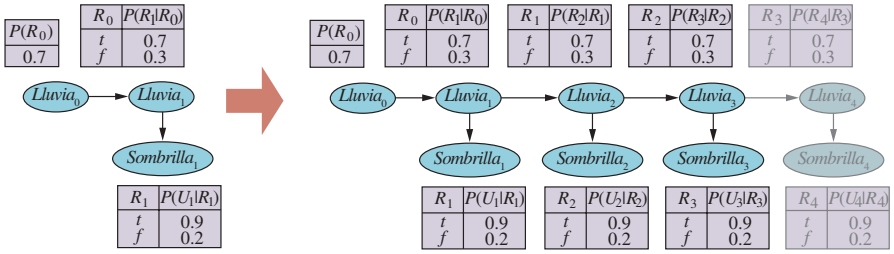


Figura 20.15 Una red bayesiana dinámica desenrollada que representa un modelo oculto de Markov (repetición de la Figura 14.16).

20.3.3 Aprendiendo modelos ocultos de Markov

Nuestra última aplicación de EM implica aprender las probabilidades de transición en modelos ocultos de Markov (HMMs). Recuerde de la Sección 14.3 que un modelo oculto de Markov puede ser representado por una red bayesiana dinámica con una única variable de estado discreta, como se ilustra en la Figura 20.15. Cada punto de datos consiste en una *secuencia* de observación de longitud finita, por lo que el problema es aprender las probabilidades de transición a partir de un conjunto de secuencias de observación (o de una sola secuencia larga).

Ya hemos visto cómo aprender redes de Bayes, pero hay una complicación: en las redes de Bayes cada parámetro es distinto; en un modelo oculto de Markov, por otro lado, las probabilidades de transición individuales del estado i al estado j en el tiempo t, $\theta_{ijt} = P(X_{t+1} = j | X_t = i)$, se *repiten* a lo largo del tiempo; es decir, $\theta_{ijt} = \theta_{ij}$ para todo t. Para estimar la probabilidad de transición del estado i al estado j, calculamos la proporción esperada de veces que el sistema experimenta una transición al estado j cuando está en el estado i:

$$\theta_{ij} \leftarrow \sum_t \hat{N}(X_{t+1} = j, X_t = i) / \sum_t \hat{N}(X_t = i).$$

Las cuentas esperadas se calculan mediante un algoritmo de inferencia HMM. El algoritmo **forward–backward** mostrado en la Figura 14.4 puede modificarse para calcular las probabilidades necesarias. Un punto importante es que las probabilidades requeridas se obtienen mediante **suavizado** en lugar de **filtrado**. El filtrado da la distribución de probabilidad del estado actual dado el pasado, pero el suavizado proporciona la distribución dada toda la evidencia, incluido lo que sucede después de que ocurrió una transición particular. La evidencia en un caso de asesinato por lo general se obtiene *después* del crimen (es decir, la transición del estado i al estado j) ha tenido lugar.

20.3.4 La forma general del algoritmo EM

Hemos visto varias instancias del algoritmo EM. Cada una implica calcular los valores esperados de las variables ocultas para cada ejemplo y luego recalcular los parámetros, utilizando los valores esperados como si fueran valores observados. Dejemos que $\mathbf{x}$ sean todos los valores observados en todos los ejemplos, dejemos que $\mathbf{Z}$ denote todas las variables ocultas para todos los ejemplos, y dejemos que θ sean todos los parámetros para el modelo de probabilidad. Entonces el algoritmo EM es

$$\theta^{(i+1)} = \underset{\theta}{\text{argmax}} \sum_{\mathbf{z}} P(\mathbf{Z} = \mathbf{z} | \mathbf{x}, \theta^{(i)}) L(\mathbf{x}, \mathbf{Z} = \mathbf{z} | \theta).$$

Esta ecuación es el algoritmo EM en pocas palabras. El paso E es el cálculo de la sumatoria, que es la expectativa del log de verosimilitud de los datos "completados" con respecto a la distribución $P(\mathbf{Z}=\mathbf{z}|\mathbf{x},\theta^{(i)})$, que es la posterior sobre las variables ocultas, dados los datos. El paso M es la maximización de este log de verosimilitud esperado con respecto a los parámetros. Para mezclas de gaussianas, las variables ocultas son las $Z_{ij}\mathbf{s}$, donde Z_{ij} es 1 si el ejemplo j fue generado por el componente i. Para redes de Bayes, Z_{ij} es el valor de la variable no observada X_i en el ejemplo j. Para HMMs, Z_{jt} es el estado de la secuencia en el ejemplo j en el tiempo t. Partiendo de la forma general, es posible derivar un algoritmo EM para una aplicación específica una vez que se han identificado las variables ocultas apropiadas.

Tan pronto como entendemos la idea general de EM, se vuelve fácil derivar todo tipo de variantes y mejoras. Por ejemplo, en muchos casos el paso E, el cálculo de las posteriores sobre las variables ocultas, es intratable, como en las grandes redes de Bayes. Resulta que es posible usar un paso E *aproximado* y aún obtener un algoritmo de aprendizaje efectivo. Con un algoritmo de muestreo como MCMC (ver Sección 13.4), el proceso de aprendizaje es muy intuitivo: cada estado (configuración de variables ocultas y observadas) visitado por MCMC se trata como si fuera una observación completa. Así, los parámetros pueden actualizarse de manera directa después de cada transición MCMC. Otras formas de inferencia aproximada, como los métodos variacionales y la propagación de creencias en bucle, también han demostrado ser efectivas para aprender redes muy grandes.

20.3.5 Aprendiendo estructuras de redes de Bayes con variables ocultas

En la Sección 20.2.7, discutimos el problema de aprender estructuras de redes de Bayes con datos completos. Cuando las variables no observadas influyen en los datos observados, las cosas se vuelven más difíciles. En el caso más simple, un experto humano podría decirle al algoritmo de aprendizaje que ciertas variables ocultas existen, dejándolo al algoritmo para encontrar un lugar para ellas en la estructura de la red. Por ejemplo, un algoritmo podría intentar aprender la estructura mostrada en la Figura 20.11(a) en la página 738, dada la información de que *EnfermedadCorazón* (una variable de tres valores) debería incluirse en el modelo. Como en el caso de los datos completos, el algoritmo general tiene un bucle externo que busca estructuras y un bucle interno que ajusta los parámetros de la red dada la estructura.

Si al algoritmo de aprendizaje no se le dice qué variables ocultas existen, entonces hay dos opciones: o bien pretender que los datos son realmente completos —lo que puede obligar al algoritmo a aprender un modelo intensivo en parámetros como el de la Figura 20.11(b)— o *inventar* nuevas variables ocultas para simplificar el modelo. Este último enfoque permite implementarse incluyendo nuevas opciones de modificación en la búsqueda de estructuras: además de modificar los enlaces, el algoritmo posibilita agregar o eliminar una variable oculta o cambiar su aridad (número mínimo de argumentos necesarios para que un operador funcione). Por supuesto, el algoritmo no sabrá que la nueva variable que ha inventado se llama *EnfermedadCorazón*; ni tendrá nombres significativos para los valores. Por fortuna, las variables ocultas recién inventadas con frecuencia estarán conectadas a variables preexistentes, por lo que un experto humano podrá con regularidad inspeccionar las distribuciones condicionales locales que involucran la nueva variable y determinar su significado.

Como en el caso de los datos completos, el aprendizaje de estructuras de máxima verosimilitud pura resultará en una red completamente conectada (además, una sin variables ocultas), por lo que se requiere alguna forma de penalización de complejidad. También podemos aplicar MCMC para muestrear muchas posibles estructuras de red, aproximando así el aprendizaje bayesiano. Por ejemplo, podemos aprender mezclas de gaussianas con un número desconocido de componentes muestreando sobre el número; la distribución posterior aproximada para el número de gaussianas está dada por las frecuencias de muestreo del proceso MCMC.

En el caso de los datos completos, el bucle interno para aprender los parámetros es muy rápido, sólo se trata de extraer las frecuencias condicionales del conjunto de datos. Cuando hay variables ocultas, el bucle interno puede implicar muchas iteraciones de EM o un algoritmo basado en gradientes, y cada iteración implica el cálculo de posteriores en una red de Bayes, que es en sí misma un problema NP-difícil. Hasta la fecha, este enfoque ha demostrado ser impráctico para aprender modelos complejos.

EM estructural

Una posible mejora es el llamado algoritmo **EM estructural**, que funciona de manera muy similar al EM ordinario (paramétrico) excepto que el algoritmo permite actualizar la estructura, así como los parámetros. Así como el EM ordinario utiliza los parámetros actuales para calcular los recuentos esperados en el paso E y luego aplica esos recuentos en el paso M para elegir nuevos parámetros, el EM estructural utiliza la estructura actual para calcular recuentos esperados y luego aplica esos recuentos en el paso M para evaluar la verosimilitud de potenciales nuevas estructuras. (Esto contrasta con el método de bucle externo/bucle interno, que calcula nuevos recuentos esperados para cada estructura potencial). De esta manera, el EM estructural puede hacer varias alteraciones estructurales a la red sin recalcular los recuentos esperados, y es capaz de aprender estructuras de red de Bayes no triviales. El EM estructural tiene un espacio de búsqueda sobre el espacio de estructuras en lugar del espacio de estructuras y parámetros. Sin embargo, queda mucho trabajo por hacer antes de que podamos decir que el problema de aprendizaje de la estructura está resuelto.

Resumen

Los métodos de aprendizaje estadístico van desde el simple cálculo de promedios hasta la construcción de modelos complejos como las redes bayesianas. Tienen aplicaciones en toda la informática, ingeniería, biología computacional, neurociencia, psicología y física. Este capítulo ha presentado algunas de las ideas básicas y ha dado una idea de los fundamentos matemáticos. Los puntos principales son los siguientes:

- Los métodos de **aprendizaje bayesianos** formulan el aprendizaje como una forma de inferencia probabilística, utilizando las observaciones para actualizar una distribución previa sobre las hipótesis. Este enfoque proporciona una buena manera de implementar la navaja de Ockham, pero rápidamente se vuelve intratable para espacios de hipótesis complejos.
- El aprendizaje de **máxima a posteriori** (MAP) selecciona una única hipótesis más probable dados los datos. El prior de la hipótesis todavía se usa y el método en la mayoría de los casos es más tratable que el aprendizaje bayesiano completo.
- El aprendizaje de **máxima verosimilitud** selecciona la hipótesis que maximiza la verosimilitud de los datos; es equivalente al aprendizaje MAP con un prior uniforme. En casos simples como la regresión lineal y las redes bayesianas completamente observables, las soluciones de máxima verosimilitud se pueden encontrar fácilmente en forma cerrada. El aprendizaje **Naive Bayes** es una técnica en particular efectiva que escala bien.
- Cuando algunas variables están ocultas, es posible encontrar soluciones de máxima verosimilitud local utilizando el algoritmo de **maximización de la expectativa** (EM). Las aplicaciones incluyen el agrupamiento no supervisado utilizando mezclas de gaussianas, el aprendizaje de redes bayesianas y el aprendizaje de modelos ocultos de Markov.
- Aprender la estructura de las redes bayesianas es un ejemplo de **selección de modelos**. Esto generalmente implica una búsqueda discreta en el espacio de estructuras. Se requiere algún método para equilibrar la complejidad del modelo contra el grado de ajuste.

- Los **modelos no paramétricos** representan una distribución utilizando la colección de puntos de datos. Por lo tanto, el número de parámetros crece con el conjunto de entrenamiento. Los métodos de vecinos más cercanos miran los ejemplos más cercanos al punto en cuestión, mientras que los métodos de **kernel** forman una combinación ponderada por distancia de todos los ejemplos.

El aprendizaje estadístico continúa siendo un área de investigación muy activa. Se han hecho enormes avances tanto en teoría como en práctica, hasta el punto de que es posible aprender casi cualquier modelo para el cual la inferencia exacta o aproximada es factible.

Notas bibliográficas e históricas

La aplicación de técnicas de aprendizaje estadístico en IA fue un área activa de investigación en los primeros años (ver Duda y Hart, 1973) pero se separó de la IA convencional a medida que este último campo se concentraba en métodos simbólicos. Un resurgimiento de interés ocurrió poco después de la introducción de los modelos de redes bayesianas a finales la década de 1980; casi al mismo tiempo comenzó a surgir una visión estadística del aprendizaje de redes neuronales. A finales de la década de 1990, hubo una notable convergencia de intereses en el aprendizaje automático, las estadísticas y las redes neuronales, centrada en métodos para crear grandes modelos probabilísticos a partir de datos.

El modelo de Bayes ingenuo es una de las formas más antiguas y simples de red bayesiana, que data de la década de 1950. Sus orígenes se mencionaron en el Capítulo 12. Su sorprendente éxito se explica parcialmente por Domingos y Pazzani (1997). Una forma potenciada de aprendizaje de Bayes ingenuo ganó la primera competencia de minería de datos de la Copa KDD (Elkan, 1997). Heckerman (1998) ofrece una excelente introducción al problema general del aprendizaje de la red de Bayes. El aprendizaje de parámetros bayesianos con priors de Dirichlet para redes bayesianas fue discutido por Spiegelhalter *et al.* (1993). La distribución beta como prior conjugado para una variable de Bernoulli fue derivada por primera vez por Thomas (Bayes, 1763) y luego reintroducida por Karl Pearson (1895) como un modelo para datos sesgados; durante muchos años se le conoció como "distribución de tipo I de Pearson". La regresión lineal bayesiana se discute en el texto de Box y Tiao (1973); Minka (2010) proporciona un resumen conciso de las derivaciones para el caso multivariante general.

Varios paquetes de software incorporan mecanismos para el aprendizaje estadístico con modelos de red de Bayes. Estos incluyen BUGS (Bayesian inference Using Gibbs Sampling) (Gilks *et al.*, 1994; Lunn *et al.*, 2000, 2013), JAGS (Just Another Gibbs Sampler) (Plummer, 2003), y STAN (Carpenter *et al.*, 2017).

Los primeros algoritmos para aprender estructuras de red de Bayes utilizaron pruebas de independencia condicional (Pearl, 1988; Pearl y Verma, 1991). Spirtes *et al.* (1993) implementaron un enfoque integral en el paquete TETRAD para el aprendizaje de la red de Bayes. Las mejoras algorítmicas desde entonces llevaron a una clara victoria en la competencia de minería de datos de la Copa KDD 2001 para un método de aprendizaje de red de Bayes (Cheng *et al.*, 2002). (¡La tarea específica aquí era un problema de bioinformática con 139 351 características!) Un enfoque de aprendizaje de estructura basado en maximizar la probabilidad fue desarrollado por Cooper y Herskovits (1992) y mejorado por Heckerman *et al.* (1994).

Los algoritmos más recientes han logrado un rendimiento bastante respetable en el caso de datos completos (Moore y Wong, 2003; Teyssier y Koller, 2005). Un componente importante es una estructura de datos eficiente, el AD-tree, para almacenar en caché los recuentos de todas las posibles combinaciones de variables y valores (Moore y Lee, 1997). Friedman y Goldszmidt

(1996) señalaron la influencia de la representación de las distribuciones condicionales locales en la estructura aprendida.

El problema general de aprender modelos de probabilidad con variables ocultas y datos faltantes fue abordado por Hartley (1958), quien describió la idea general de lo que luego se llamó EM y dio varios ejemplos. Un impulso adicional provino del algoritmo Baum–Welch para el aprendizaje de HMM (Baum y Petrie, 1966), que es un caso especial de EM. El artículo de Dempster, Laird y Rubin (1977), que presentó el algoritmo EM en forma general y analizó su convergencia, es uno de los artículos más citados tanto en informática como en estadística. (Dempster en sí mismo ve a EM como un esquema más que como un algoritmo, ya que puede requerirse una buena cantidad de trabajo matemático antes de que pueda aplicarse a una nueva familia de distribuciones). McLachlan y Krishnan (1997) dedican un libro completo al algoritmo y sus propiedades. El problema específico de aprender modelos de mezcla, incluidas mezclas de gaussianas, es cubierto por Titterington *et al.* (1985).

Dentro de la IA, AUTOCLASS (Cheeseman *et al.*, 1988; Cheeseman y Stutz, 1996) fue el primer sistema exitoso que utilizó EM para el modelado de mezclas. AUTOCLASS se aplicó a una serie de tareas científicas de clasificación del mundo real, incluido el descubrimiento de nuevos tipos de estrellas a partir de datos espectrales (Goebel *et al.*, 1989) y nuevas clases de proteínas e intrones en bases de datos de secuencias de ADN/proteínas (Hunter y States, 1992).

Para el aprendizaje de parámetros de máxima verosimilitud en redes de Bayes con variables ocultas, EM y los métodos basados en gradientes fueron introducidos alrededor del mismo tiempo por Lauritzen (1995) y Russell *et al.* (1995). El algoritmo EM estructural fue desarrollado por Friedman (1998) y aplicado al aprendizaje de máxima verosimilitud de estructuras de red de Bayes con variables latentes. Friedman y Koller (2003) describen el aprendizaje de estructura bayesiana. Daly *et al.* (2011) revisan el campo del aprendizaje de la red de Bayes, proporcionando extensas citas a la literatura.

La capacidad de aprender la estructura de las redes bayesianas está estrechamente conectada con el problema de recuperar información *causal* a partir de datos. Es decir, ¿es posible aprender redes de Bayes de tal manera que la estructura de la red recuperada indique influencias causales reales? Durante muchos años los estadísticos evitaron esta pregunta, creyendo que los datos observacionales (en oposición a los datos generados a partir de ensayos experimentales) sólo podían proporcionar información correlacional; después de todo, cualquier par de variables que parezcan relacionadas podrían de hecho estar influidas por un tercer factor causal desconocido, en lugar de influir directamente entre sí. Pearl (2000) ha presentado argumentos convincentes en contra, demostrando que de hecho hay muchos casos en los que se es posible determinar la causalidad y desarrollan el formalismo de la **red causal** para expresar causas y los efectos de la intervención, así como las probabilidades condicionales ordinarias.

La estimación de densidad no paramétrica, también llamada estimación de densidad de **ventana de Parzen**, fue investigada inicialmente por Rosenblatt (1956) y Parzen (1962). Desde ese momento, se ha desarrollado mucha literatura investigando las propiedades de varios estimadores. Devroye (1987) ofrece una introducción completa. También hay literatura en rápido crecimiento sobre métodos bayesianos no paramétricos, originada con el trabajo seminal de Ferguson (1973) sobre el **proceso de Dirichlet**, que puede considerarse como una distribución sobre distribuciones de Dirichlet. Estos métodos son en particular útiles para mezclas con números desconocidos de componentes. Ghahramani (2005) y Jordan (2005) proporcionan tutoriales útiles sobre las muchas aplicaciones de estas ideas al aprendizaje estadístico. El texto de Rasmussen y Williams (2006) cubre el **proceso gaussiano**, que proporciona una forma de definir distribuciones previas sobre el espacio de funciones continuas.

Proceso de Dirichlet

Proceso gaussiano

El material en este capítulo reúne trabajos de los campos de estadística y reconocimiento de patrones, por lo que la historia se ha contado muchas veces de muchas maneras. Buenos textos

sobre estadísticas bayesianas incluyen los de DeGroot (1970), Berger (1985) y Gelman *et al.* (1995). Bishop (2007), Hastie *et al.* (2009), Barber (2012) y Murphy (2012) proporcionan excelentes introducciones al aprendizaje automático estadístico. Para la clasificación de patrones, el texto clásico durante muchos años ha sido Duda y Hart (1973), ahora actualizado (Duda *et al.*, 2001). La conferencia anual NeurIPS (Neural Information Processing Systems, antes NIPS), cuyas actas se publican como la serie *Advances in Neural Information Processing Systems*, incluye muchos documentos de aprendizaje bayesiano, al igual que la conferencia anual sobre Inteligencia Artificial y Estadísticas. Los lugares específicamente bayesianos incluyen las Reuniones Internacionales de Valencia sobre Estadísticas Bayesianas y la revista *Bayesian Analysis*.

CAPÍTULO 21

APRENDIZAJE PROFUNDO

En el cual el descenso de gradiente aprende programas de varios pasos, con implicaciones significativas para los principales subcampos de la inteligencia artificial.

Aprendizaje profundo

El **aprendizaje profundo** es una amplia familia de técnicas para el aprendizaje automático en el que las hipótesis toman la forma de circuitos algebraicos complejos con fuerzas de conexión ajustables. La palabra "profundo" se refiere al hecho de que los circuitos están organizados en múltiples **capas**, lo que significa que las rutas de cálculo desde las entradas hasta las salidas tienen múltiples pasos. El aprendizaje profundo es el enfoque más usado para aplicaciones como reconocimiento de objetos visuales, traducción automática, reconocimiento de voz, síntesis de voz y síntesis de imágenes; también desempeña un papel significativo en las aplicaciones de aprendizaje por refuerzo (ver Capítulo 22).

Capa

El aprendizaje profundo tiene sus orígenes en los primeros trabajos que intentaron modelar redes de neuronas en el cerebro (McCulloch y Pitts, 1943) con circuitos computacionales. Por ello, las redes entrenadas por métodos de aprendizaje profundo suelen llamarse **redes neuronales**, aunque la semejanza con las células y estructuras neuronales reales es superficial.

Red neuronal

Las verdaderas razones del éxito del aprendizaje profundo aún no se han elucidado del todo, tiene ventajas evidentes sobre algunos métodos vistos en el Capítulo 19, en especial para datos de alta dimensión como las imágenes. Por ejemplo, si bien métodos como la regresión lineal y logística permiten manejar un gran número de variables de entrada, la ruta de cálculo desde cada entrada hasta la salida es corta: multiplicación por un solo peso, luego agregación en la salida final. Además, las diferentes variables de entrada contribuyen de forma independiente a la salida, sin interactuar entre sí (Figura 21.1(a)). Esto limita el poder expresivo de tales modelos. Sólo permiten representar funciones y límites lineales en el espacio de entrada, mientras que la mayoría de los conceptos del mundo real son mucho más complejos.

Las listas de decisiones y los árboles de decisiones, por otro lado, permiten rutas de cálculo largas que dependen de múltiples variables de entrada, pero sólo para una fracción relativamente pequeña de los posibles vectores de entrada (Figura 21.1(b)). Si un árbol de decisiones tiene rutas de cálculo largas para una fracción significativa de las posibles entradas, debe ser exponencialmente grande en el número de variables de entrada. La idea básica del aprendizaje profundo es entrenar circuitos de manera que las rutas de cálculo sean largas y permitan que todas las variables de entrada interactúen de formas complejas (Figura 21.1(c)). Estos modelos de circuitos resultan ser suficientemente expresivos para capturar la complejidad de los datos del mundo real para muchos tipos importantes de problemas de aprendizaje.

La Sección 21.1 describe redes de avance simple, sus componentes y aspectos esenciales del aprendizaje. La Sección 21.2 profundiza en cómo se ensamblan las redes profundas, mientras que la Sección 21.3 cubre la clase de redes neuronales convolucionales, importantes en aplicaciones de visión. Las Secciones 21.4 y 21.5 profundizan en algoritmos para entrenar redes a partir de datos y métodos para mejorar la generalización. La Sección 21.6 cubre redes con estructura recurrente, adecuadas para datos secuenciales. La Sección 21.7 describe formas de usar el aprendizaje

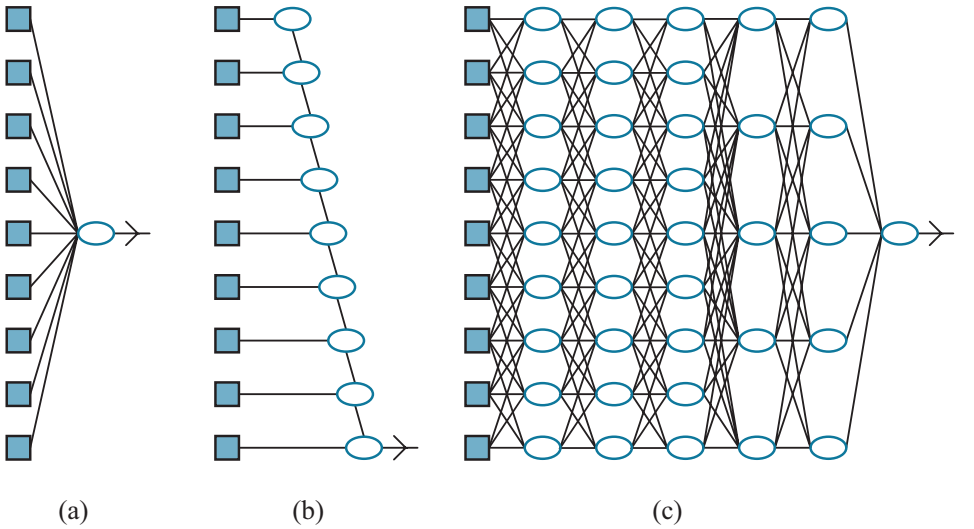


Figura 21.1 (a) Modelo superficial, como la regresión lineal, tiene rutas de cálculo cortas entre las entradas y la salida. (b) Una red de lista de decisiones (página 674) tiene algunas rutas largas para posibles valores de entrada, pero la mayoría de las rutas son cortas. (c) Una red de aprendizaje profundo tiene rutas de cálculo más largas, que permiten a cada variable interactuar con las demás.

profundo para tareas distintas al aprendizaje supervisado. Finalmente, la Sección 21.8 examina la gama de aplicaciones del aprendizaje profundo.

21.1 Redes de avance simple

Una **red de avance**, como su nombre indica, tiene conexiones sólo en una dirección, es decir, forma un grafo acíclico dirigido con nodos de entrada y salida designados. Cada nodo calcula una función de sus entradas y pasa el resultado a sus sucesores en la red. La información fluye a través de la red desde los nodos de entrada hasta los nodos de salida, y no hay bucles. Una **red recurrente**, por otro lado, retroalimenta sus salidas intermedias o finales en sus propias entradas. Esto significa que los valores de señal dentro de la red forman un sistema dinámico que tiene un estado interno o memoria. Consideraremos las redes recurrentes en la Sección 21.6.

Red de avance

Red recurrente

Los circuitos booleanos, que implementan funciones booleanas, son un ejemplo de redes de avance. En estos circuitos, las entradas se limitan a 0 y 1, y cada nodo implementa una función booleana simple de sus entradas, lo que produce un 0 o un 1. En las redes neuronales, los valores de entrada suelen ser continuos, pero los nodos toman entradas continuas y producen salidas continuas. Algunas entradas a los nodos son **parámetros** de la red; la red aprende ajustando los valores de estos parámetros para que la red en su conjunto se ajuste a los datos de entrenamiento.

21.1.1 Redes como funciones complejas

Cada nodo dentro de una red se llama una **unidad**. Siguiendo el diseño propuesto por McCulloch y Pitts, una unidad calcula la suma ponderada de las entradas de los nodos predecesores y luego

Unidad

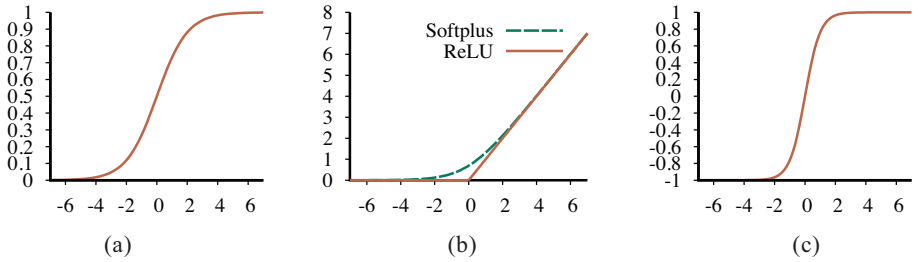


Figura 21.2 Funciones de activación utilizadas en sistemas de aprendizaje profundo: (a) la función logística o sigmoide; (b) la función ReLU y la función softplus; (c) la función tanh.

aplica una función no lineal para producir su salida. Dejemos que a_j denote la salida de la unidad j y dejemos que $w_{i,j}$ sea el peso adjunto al enlace desde la unidad i hasta la unidad j; entonces tenemos

$$a_j = g_j(\sum_i w_{i,j} a_i) \equiv g_j(in_j),$$

Función de activación

donde g_j es una **función de activación** no lineal asociada con la unidad j, e in_j es la suma ponderada de las entradas a la unidad j.

Como en la Sección 19.6.3 (página 679), estipulamos que cada unidad tiene una entrada extra de una unidad ficticia 0 que se fija en +1 y un peso $w_{0,j}$ para esa entrada. Esto permite que la entrada ponderada total *en* j a la unidad j sea distinta de cero aun cuando las salidas de la capa anterior sean todas cero. Con esta convención, podemos escribir la ecuación anterior en forma vectorial:

$$a_j = g_j(\mathbf{w}^\top \mathbf{x}) \tag{21.1}$$

donde $\mathbf{w}$ es el vector de pesos que conducen a la unidad j (incluido $w_{0,j}$) y $\mathbf{x}$ es el vector de entradas a la unidad j (incluido el +1).

El hecho de que la función de activación sea no lineal es importante porque si no lo fuera, cualquier composición de unidades seguiría representando una función lineal. La no linealidad es lo que permite que las redes de unidades suficientemente grandes representen funciones arbitrarias. El teorema de **aproximación universal** establece que con una red con sólo dos capas de unidades computacionales, la primera no lineal y la segunda lineal, es posible aproximar cualquier función continua a un grado de precisión arbitrario. La prueba funciona mostrando que una red exponencialmente grande permite representar múltiples "protuberancias" de diferentes alturas en diferentes ubicaciones en el espacio de entrada, aproximando así la función deseada. Es decir, las redes suficientemente grandes implementan una tabla de búsqueda para funciones continuas, al igual que los árboles de decisión suficientemente grandes que implementan una tabla de búsqueda para funciones booleanas.

Se utilizan diferentes funciones de activación. Las más comunes son:

Sigmoide

- Logística o **sigmoide**, que también se usa en regresión logística (ver página 685):

  $\sigma(x) = 1/(1 + e^{-x})$.

ReLU

- **ReLU**, cuyo nombre es abreviatura de **unidad lineal rectificada**:

  $\text{ReLU}(x) = \max(0,x)$.

Softplus

- **Softplus**, versión suave de la función ReLU:

  $\text{softplus}(x) = \log(1 + e^x)$.

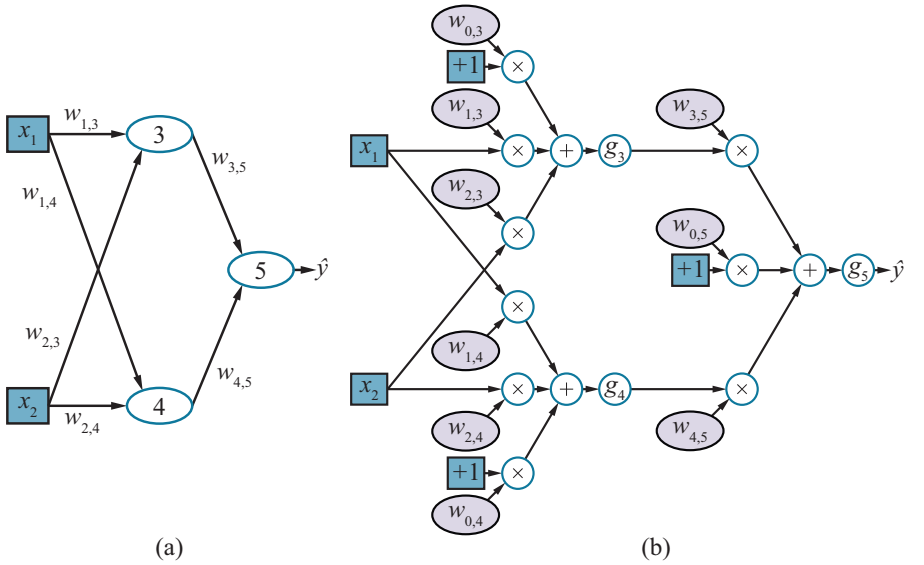


Figura 21.3 (a) Una red neuronal con dos entradas, una capa oculta de dos unidades y una unidad de salida. Las entradas ficticias y sus pesos asociados no se muestran. (b) La red en (a) desempaquetada en su grafo de cálculo completo.

La derivada de la función softplus es la función sigmoide.

- Función **tanh**: Tanh

$$\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}.$$

Note que el rango de tanh es (−1, +1). Tanh es una versión escalada y desplazada de la sigmoide, ya que $\tanh(x) = 2\sigma(2x) - 1$.

Estas funciones se muestran en la Figura 21.2. Vea que todas ellas son monótonamente no decrecientes, lo que significa que sus derivadas g' son no negativas. En secciones posteriores nos extenderemos sobre la elección de la función de activación .

Acoplar varias unidades en una red crea una función compleja, una composición de las expresiones algebraicas representadas por las unidades individuales. Por ejemplo, la red mostrada en la Figura 21.3(a) representa una función $h_{\mathbf{w}}(\mathbf{x})$, parametrizada por peso $\mathbf{w}$, que mapea un vector de entrada de dos elementos $\mathbf{x}$ a un valor de salida escalar $\hat{y}$. La estructura interna de la función refleja la estructura de la red. Por ejemplo, podemos escribir una expresión para la salida $\hat{y}$ de la siguiente manera:

$$\begin{aligned}\hat{y} &= g_5(in_5) = g_5(w_{0,5} + w_{3,5}\, a_3 + w_{4,5} a_4) \\ &= g_5(w_{0,5} + w_{3,5} g_3(in_3) + w_{4,5} g_4(in_4)) \\ &= g_5(w_{0,5} + w_{3,5} g_3(w_{0,3} + w_{1,3} x_1 + w_{2,3} x_2) \\ &\quad + w_{4,5} g_4(w_{0,4} + w_{1,4} x_1 + w_{2,4} x_2)).\end{aligned} \tag{21.2}$$

Así, se define la salida $\hat{y}$ expresada como una función $h_{\mathbf{w}}(\mathbf{x})$ de las entradas y los pesos.

Grafo de cálculo
Grafo de flujo de datos

La Figura 21.3(a) muestra la forma tradicional en que una red podría ser representada en un libro sobre redes neuronales. Una forma más general de pensar en la red es como un **grafo de cálculo** o **grafo de flujo de datos;** en esencia, un circuito en el que cada nodo representa un cálculo elemental. La Figura 21.3(b) muestra el grafo de cálculo correspondiente a la red en la Figura 21.3(a); el grafo hace explícito cada elemento del cálculo global. También distingue entre las entradas (en azul) y los pesos (en malva claro): los pesos pueden ser ajustados para hacer que la salida $\hat{y}$ concuerde con el valor verdadero y en los datos de entrenamiento. Cada peso es como un control de volumen que determina cuánto escucha el siguiente nodo en el grafo de ese particular predecesor en el grafo.

Así como la Ecuación (21.1) describió la operación de una unidad en forma vectorial, podemos hacer algo similar para la red en su conjunto. En general usaremos $\mathbf{W}$ para denotar una matriz de pesos; para esta red, $\mathbf{W}^{(1)}$ denota los pesos en la primera capa ($w_{1,3,}$ $w_{1,4,}$ etc.) y $\mathbf{W}^{(2)}$ denota los pesos en la segunda capa ($w_{3,5}$ etc.). Finalmente, $\mathbf{g}^{(1)}$ y $\mathbf{g}^{(2)}$ definen las funciones de activación en la primera y segunda capas. Entonces, toda la red puede ser escrita de la siguiente manera:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{g}^{(2)}(\mathbf{W}^{(2)}\mathbf{g}^{(1)}(\mathbf{W}^{(1)}\mathbf{x})). \tag{21.3}$$

Como la Ecuación (21.2), esta expresión corresponde a un grafo de cálculo, aunque mucho más simple que el grafo en la Figura 21.3(b); aquí, el grafo es una cadena con matrices de pesos alimentando cada capa.

El grafo de cálculo en la Figura 21.3(b) es relativamente pequeño y superficial, pero la misma idea se aplica a todas las formas de aprendizaje profundo: construimos grafos de cálculo y ajustamos sus pesos para ajustar los datos. El grafo en la Figura 21.3(b) también está **totalmente conectado**, lo que significa que cada nodo en cada capa está conectado a cada nodo en la siguiente capa. Esto es, en cierto sentido, el predeterminado, pero veremos en la Sección 21.3 que la elección de la conectividad de la red también es importante para lograr un aprendizaje efectivo.

Totalmente conectado

21.1.2 Gradientes y aprendizaje

En la Sección 19.6, introdujimos un enfoque para el aprendizaje supervisado basado en el **descenso de gradiente**: calcular el gradiente de la función de pérdida con respecto a los pesos, y ajustar los pesos a lo largo de la dirección del gradiente para reducir la pérdida. (Si aún no ha leído la Sección 19.6, le recomendamos que la lea antes de continuar). Es posible aplicar el mismo enfoque para aprender los pesos en los grafos de cálculo. Para los pesos que conducen a las unidades en la **capa de salida**, las que producen la salida de la red, el cálculo del gradiente es idéntico al proceso en la Sección 19.6. Para los pesos que conducen a las unidades en las **capas ocultas**, que no están conectadas de manera directa a las salidas, el proceso es apenas un poco más complicado.

Capa de salida
Capa oculta

Por ahora, utilizaremos la función de pérdida al cuadrado, L_2, y calcularemos el gradiente para la red en la Figura 21.3 con respecto a un solo ejemplo de entrenamiento $(\mathbf{x},y)$. (Para múltiples ejemplos, el gradiente es la suma de los gradientes para los ejemplos individuales). La red emite una predicción $\hat{y} = h_{\mathbf{w}}(\mathbf{x})$ y el valor verdadero es y, así que tenemos

$$\textit{Pérdida}(h_{\mathbf{w}}) = L_2(y, h_{\mathbf{w}}(\mathbf{x})) = ||y - h_{\mathbf{w}}(\mathbf{x})||^2 = (y - \hat{y})^2.$$

Para calcular el gradiente de la pérdida con respecto a los pesos, necesitamos las mismas herramientas de cálculo que ya utilizamos en el Capítulo 19; en principio, la **regla de la cadena**, $\partial g(f(x))/\partial x = g'(f(x))\partial f(x)/\partial x$. Comenzaremos con un caso simple: un peso como $w_{3,5}$ que está conectado a la unidad de salida. Operamos de manera directa en las expresiones que definen la red de la Ecuación (21.2):

$$\begin{aligned}\frac{\partial}{\partial w_{3,5}} Perdida(h_{\mathbf{w}}) &= \frac{\partial}{\partial w_{3,5}}(y-\hat{y})^2 = -2(y-\hat{y})\frac{\partial \hat{y}}{\partial w_{3,5}} \\ &= -2(y-\hat{y})\frac{\partial}{\partial w_{3,5}} g_5(in_5) = -2(y-\hat{y})g'_5(in_5)\frac{\partial}{\partial w_{3,5}} in_5 \\ &= -2(y-\hat{y})g'_5(in_5)\frac{\partial}{\partial w_{3,5}}(w_{0,5}+w_{3,5}a_3+w_{4,5}a_4) \\ &= -2(y-\hat{y})g'_5(in_5)a_3. \end{aligned} \tag{21.4}$$

La simplificación en la última línea sigue porque $w_{0,5}$ y $w_{4,5}a_4$ no dependen de $w_{3,5}$, ni el coeficiente de $w_{3,5}$,a_3.

El caso un poco más difícil implica un peso como $w_{1,3}$ que no está conectado de manera directa a la unidad de salida. Aquí, tenemos que aplicar la regla de la cadena una vez más. Los primeros pasos iniciales son idénticos, por lo que los omitimos:

$$\begin{aligned}\frac{\partial}{\partial w_{1,3}} Perdida(h_{\mathbf{w}}) &= -2(y-\hat{y})g'_5(in_5)\frac{\partial}{\partial w_{1,3}}(w_{0,5}+w_{3,5}a_3+w_{4,5}a_4) \\ &= -2(y-\hat{y})g'_5(in_5)w_{3,5}\frac{\partial}{\partial w_{1,3}} a_3 \\ &= -2(y-\hat{y})g'_5(in_5)w_{3,5}\frac{\partial}{\partial w_{1,3}} g_3(in_3) \\ &= -2(y-\hat{y})g'_5(in_5)w_{3,5}g'_3(in_3)\frac{\partial}{\partial w_{1,3}} in_3 \\ &= -2(y-\hat{y})g'_5(in_5)w_{3,5}g'_3(in_3)\frac{\partial}{\partial w_{1,3}}(w_{0,3}+w_{1,3}x_1+w_{2,3}x_2) \\ &= -2(y-\hat{y})g'_5(in_5)w_{3,5}g'_3(in_3)x_1. \end{aligned} \tag{21.5}$$

Entonces, tenemos expresiones simples para el gradiente de la pérdida con respecto a los pesos $w_{3,5}$ y $w_{1,3}$.

Si definimos $\Delta_5 = 2(\hat{y}-y)g'_5(in_5)$ como una especie de "error percibido" en el punto donde la unidad 5 recibe su entrada, entonces el gradiente con respecto a $w_{3,5}$ es sólo $\Delta_5 a_3$. Esto tiene mucho sentido: si Δ_5 es positivo, eso significa que $\hat{y}$ es demasiado grande (recuerde que g' siempre es no negativo); si a_3 también es positivo, entonces aumentar $w_{3,5}$ sólo empeorará las cosas, mientras que si a_3 es negativo, entonces aumentar $w_{3,5}$ reducirá el error. La magnitud de a_3 también importa: si a_3 es pequeño para este ejemplo de entrenamiento, entonces $w_{3,5}$ no desempeñó un papel importante en la producción del error y no necesita ser cambiado de manera significativa.

Si también se define $\triangle_3 = \triangle_5 w_{3,5}g'_3(in_3)$, entonces el gradiente para $w_{3,5}$ se convierte en $\triangle_3 x_1$. Así, el error percibido en la entrada a la unidad 3 es el error percibido en la entrada a la unidad 5, multiplicado por la información a lo largo del camino de regreso desde 5 a 3. Este fenómeno es completamente general y da lugar al término **retropropagación** para la forma en que el error en la salida se pasa de regreso a través de la red.

Retropropagación

Otra característica importante de estas expresiones de gradiente es que tienen como factores las derivadas locales $g'_j(in_j)$. Como antes se señaló, estas derivadas siempre son no negativas, pero pueden ser muy cercanas a cero (en el caso de las funciones sigmoide, softplus y tanh) o exactamente cero (en el caso de ReLUs), si las entradas del ejemplo de entrenamiento en cuestión

llegan a poner la unidad j en la región plana de operación. Si la derivada g'_i es pequeña o cero, eso significa que cambiar los pesos que llevan a la unidad j tendrá un efecto insignificante en su salida. Como resultado, las redes profundas con múltiples capas pueden sufrir de un **desvanecimiento de gradiente**: las señales de error se extinguen por completo a medida que se propagan de regreso a través de la red. La Sección 21.3.3 proporciona una solución a este problema.

Desvanecimiento de gradiente

Se ha demostrado que los gradientes en nuestra red de ejemplo son expresiones simples que se pueden calcular pasando información de regreso a través de la red desde las unidades de salida. Resulta que esta propiedad se mantiene en general. De hecho, como se muestra en la Sección 21.4.1, los cálculos de gradientes para cualquier grafo computacional de avance tienen la misma estructura que el grafo computacional subyacente. Esta propiedad se sigue directamente de las reglas de la diferenciación.

Se han mostrado los detalles minuciosos de un cálculo de gradiente, pero no hay de qué preocuparse: no hay necesidad de rehacer las derivaciones en las Ecuaciones (21.4) y (21.5) para cada nueva estructura de red. Todos esos gradientes se pueden calcular mediante el método de **diferenciación automática**, que aplica las reglas del cálculo de manera sistemática para calcular gradientes para cualquier programa numérico.[1] De hecho, el método de retropropagación en el aprendizaje profundo es sólo una aplicación de la diferenciación en **modo reverso**, que aplica la regla de la cadena "desde el exterior hacia adentro" y obtiene las ventajas de eficiencia de la programación dinámica cuando la red en cuestión tiene muchas entradas y relativamente pocas salidas.

Diferenciación automática

Modo reverso

Todos los paquetes importantes para el aprendizaje profundo proporcionan diferenciación automática, de modo que los usuarios pueden experimentar libremente con diferentes estructuras de red, funciones de activación, funciones de pérdida y formas de composición sin tener que hacer muchos cálculos para derivar un nuevo algoritmo de aprendizaje para cada experimento. Esto ha fomentado un enfoque llamado **aprendizaje de extremo a extremo**, en el cual un sistema computacional complejo para una tarea como la traducción automática se puede componer de varios subsistemas entrenables; el sistema completo se entrena de manera integral a partir de pares de entrada/salida. Con este enfoque, el diseñador sólo necesita tener una idea vaga sobre cómo debe estructurarse el sistema general; no es necesario saber de antemano exactamente qué debe hacer cada subsistema o cómo etiquetar sus entradas y salidas.

Aprendizaje de extremo a extremo

21.2 Grafos computacionales para el aprendizaje profundo

Se han establecido las ideas básicas del aprendizaje profundo: representar hipótesis como grafos computacionales con pesos ajustables y calcular el gradiente de la función de pérdida con respecto a esos pesos para ajustar los datos de entrenamiento. Ahora se analiza cómo armar grafos computacionales. Se comienza con la capa de entrada, que es donde el ejemplo de entrenamiento o prueba $\mathbf{x}$ se codifica como valores de los nodos de entrada. Luego se considera la capa de salida, donde las salidas $\hat{\mathbf{y}}$ se comparan con los valores reales $\mathbf{y}$ para derivar una señal de aprendizaje para ajustar los pesos. Finalmente, se examinan las capas ocultas de la red.

21.2.1 Codificación de entrada

Los nodos de entrada y salida de un grafo computacional son los que se conectan directamente a los datos de entrada $\mathbf{x}$ y los datos de salida $\mathbf{y}$. La codificación de los datos de entrada suele ser sencilla, al menos para el caso de datos factorizados donde cada ejemplo de entrenamiento

[1] Los métodos de diferenciación automática se desarrollaron originalmente en las décadas de 1960 y 1970 para optimizar los parámetros de sistemas definidos por grandes y complejos programas Fortran.

contiene valores para n atributos de entrada. Si los atributos son booleanos, se tienen n nodos de entrada; generalmente, *falso* se mapea a una entrada de 0 y *verdadero* se mapea a 1, aunque a veces se usan -1 y +1. Los atributos numéricos, sean enteros o de valor real, por lo regular se usan tal cual, aunque pueden ser escalados para encajar dentro de un rango fijo; si las magnitudes para diferentes ejemplos varían enormemente, los valores se pueden mapear a una escala logarítmica.

Las imágenes no encajan en la categoría de datos factorizados; aunque una imagen RGB de tamaño $X \times Y$ píxeles puede considerarse como $3XY$ atributos de valor entero (típicamente con valores en el rango {0,..., 255}), esto ignoraría el hecho de que las tripletas RGB pertenecen al mismo píxel en la imagen y el hecho de que la adyacencia de píxeles realmente importa. Por supuesto, podemos mapear adyacentes píxeles en nodos de entrada adyacentes en la red, pero el significado de adyacencia se pierde por completo si las capas internas de la red están completamente conectadas. En la práctica, las redes utilizadas con datos de imagen tienen estructuras internas tipo *array* que pretenden reflejar la semántica de adyacencia. Veremos esto con más detalle en la Sección 21.3.

Los atributos categóricos con más de dos valores, como el atributo *Tipo* en el problema del restaurante del Capítulo 19, que tiene valores Francés, Italiano, Tailandés, o hamburguesa, que en general se codifican usando la llamada **codificación one-hot**. Un atributo con d posibles valores es representado por d bits de entrada separados. Para cualquier valor dado, el bit de entrada correspondiente se establece en 1 y todos los demás se establecen en 0. Esto por lo regular funciona mejor que mapear los valores a enteros. Si usáramos enteros para el atributo *Tipo*, Tailandés sería 3 y hamburguesa sería 4. Debido a que la red es una composición de funciones continuas, no tendría más remedio que prestar atención a la adyacencia numérica, pero en este caso la adyacencia numérica entre Tailandés y hamburguesa no tiene sentido de manera sistemática.

21.2.2 Capas de salida y funciones de pérdida

En el lado de salida de la red, el problema de codificar los valores de datos brutos en valores reales y para los nodos de salida del grafo es muy similar al problema de codificación de entrada. Por ejemplo, si la red trata de predecir la variable *Clima* del Capítulo 12, que tiene valores {*sol*, *lluvia*, *nube*, *nieve*}, usaríamos una codificación one-hot con cuatro bits.

Hasta aquí los valores de datos $\mathbf{y}$. ¿Qué pasa con la predicción $\hat{\mathbf{y}}$? Idealmente, coincidiría con el valor deseado $\mathbf{y}$, y la pérdida sería cero, y habríamos terminado. En la práctica, esto rara vez sucede antes de que hayamos comenzado el proceso de ajustar los pesos. Por lo tanto, necesitamos pensar qué significa un valor de salida incorrecto y cómo medir la pérdida. Al derivar los gradientes en las Ecuaciones (21.4) y (21.5), comenzamos con la función de pérdida de error cuadrado. Esto mantiene el álgebra simple, pero no es la única posibilidad. De hecho, para la mayoría de las aplicaciones de aprendizaje profundo, es más común interpretar los valores de salida $\hat{\mathbf{y}}$ como probabilidades y usar el **logaritmo negativo de la verosimilitud** como la función de pérdida, como lo hicimos con el aprendizaje de **máxima verosimilitud** en el Capítulo 20.

El aprendizaje de máxima verosimilitud encuentra el valor de $\mathbf{w}$ que maximiza la probabilidad de los datos observados. Y porque la función logarítmica es monótona, esto es equivalente a maximizar el logaritmo de la verosimilitud de los datos, que es equivalente a su vez a minimizar una función de pérdida definida como el logaritmo negativo de la verosimilitud. (Recuerde de la página 725 que tomar logaritmos convierte los productos de probabilidades en sumas, que son más manejables para tomar derivadas). En otras palabras, estamos buscando $\mathbf{w}^*$ que minimiza la suma de logaritmos negativos de las probabilidades de los N ejemplos:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} -\sum_{j=1}^{N} \log P_{\mathbf{w}}(\mathbf{y}_j|\mathbf{x}_j). \tag{21.6}$$

Entropía cruzada

En la literatura de aprendizaje profundo, es común hablar de minimizar la pérdida de **entropía cruzada**. Esta, escrita como $H(P,Q)$, es una especie de medida de disimilitud entre dos distribuciones P y Q.[2] La definición general es

$$H(P,Q) = \mathbf{E}_{\mathbf{z}\sim P(\mathbf{z})}[\log Q(\mathbf{z})] = \int P(\mathbf{z}) \log Q(\mathbf{z}) d\mathbf{z}. \tag{21.7}$$

En el aprendizaje automático, por lo regular usamos esta definición, con P siendo la distribución verdadera sobre ejemplos de entrenamiento, $P^*(\mathbf{x},\mathbf{y})$, y Q siendo la hipótesis predictiva $P_{\mathbf{w}}(\mathbf{y}|\mathbf{x})$. Minimizar la entropía cruzada $H(P^*(\mathbf{x},\mathbf{y}),P_{\mathbf{w}}(\mathbf{y}|\mathbf{x}))$, al ajustar $\mathbf{w}$, hace que la hipótesis concuerde lo más posible con la distribución verdadera. En realidad, no podemos minimizar esta entropía cruzada porque no tenemos acceso a la verdadera distribución de datos $P^*(\mathbf{x},\mathbf{y})$; pero sí tenemos acceso a muestras de $P^*(\mathbf{x},\mathbf{y})$, por lo que la suma sobre los datos reales en la Ecuación (21.6) aproxima la expectativa en la Ecuación (21.7).

Para minimizar el logaritmo negativo de la verosimilitud (o la entropía cruzada), necesitamos poder interpretar la salida de la red como una probabilidad. Por ejemplo, si la red tiene una unidad de salida con una función de activación sigmoide y está aprendiendo una clasificación booleana, podemos interpretar el valor de salida como la probabilidad de que el ejemplo pertenezca a la clase positiva. (De hecho, esto es cómo se usa la regresión logística; ver página 684). Por lo tanto, para problemas de clasificación booleana, con frecuencia usamos una capa de salida sigmoide.

Los problemas de clasificación multiclase son comunes en el aprendizaje automático. Por ejemplo, los clasificadores usados para el reconocimiento de objetos en la mayoría de los casos necesitan reconocer miles de categorías distintas de objetos. Los modelos de lenguaje natural que intentan predecir la siguiente palabra en una oración pueden elegir entre decenas de miles de palabras posibles. Para este tipo de predicción necesitamos que la red emita una distribución categórica, es decir, si hay d respuestas posibles, necesitamos d nodos de salida que representen probabilidades que sumen 1.

Softmax

Para lograr esto, usamos una capa **softmax**, que emite un vector de d valores dados un vector de valores de entrada $\mathbf{in}=\langle in_1, ..., in_d\rangle$. El elemento kth de ese vector de salida se define como

$$\text{softmax}(\mathbf{in})_k = \frac{e^{in_k}}{\sum_{k'=1}^{d} e^{in_{k'}}}.$$

Por construcción, la función softmax emite un vector de números no negativos que suman 1. Como de costumbre, la entrada in_k a cada uno de los nodos de salida será una combinación lineal ponderada de las salidas de la capa anterior. Debido a las exponenciales, la capa softmax acentúa las diferencias en las entradas: por ejemplo, si el vector de entradas se da por $\mathbf{in}=\langle 5,2,0,-2\rangle$, entonces las salidas son $\langle 0.946, 0.047, 0.006, 0.001\rangle$. El softmax es, sin embargo, suave y diferenciable (Ejercicio 21.SOFG), a diferencia de la función max. Es fácil demostrar (Ejercicio 21.SMSG) que el sigmoide es un softmax con $d = 2$. En otras palabras, así como las unidades sigmoideas propagan información de clase binaria a través de una red, las unidades softmax propagan información de multiclase.

Para un problema de regresión, donde el valor objetivo y es continuo, es común usar una capa de salida lineal, en otras palabras, $\hat{y}_j = in_j$, sin ninguna función de activación g, y para interpretar esto como la media de una predicción gaussiana con varianza fija. Como señalamos en la página 729, maximizar la probabilidad (es decir, minimizar el logaritmo negativo de la probabilidad)

[2] La entropía cruzada no es una distancia en el sentido habitual porque $H(P,P)$ no es cero; más bien, es igual a la entropía $H(P)$. Es fácil demostrar que $H(P,Q) = H(P) + D_{KL}(P||Q)$, donde D_{KL} es la **divergencia de Kullback–Leibler**, que sí satisface $D_{KL}(P||P) = 0$. Por lo tanto, para P fijo, variar Q para minimizar la entropía cruzada también minimiza la divergencia KL.

con un Gausiano de varianza fija es lo mismo que minimizar el error cuadrado. Por lo tanto, una capa de salida lineal interpretada de esta manera hace la regresión lineal clásica. Las características de entrada a esta regresión lineal son las salidas de la capa anterior, que suelen resultar de múltiples transformaciones no lineales de las entradas originales a la red.

Densidad de mezcla

Muchas otras capas de salida son posibles. Por ejemplo, una capa de **densidad de mezcla** representa las salidas usando una mezcla de distribuciones gaussianas. (Ver Sección 20.3.1 para más detalles sobre las mezclas gaussianas). Tales capas predicen de cada componente de la mezcla la frecuencia relativa, la media y la varianza. Siempre que estos valores de salida sean interpretados de manera adecuada por la función de pérdida para definir la probabilidad para el verdadero valor de salida **y**, la red, después del entrenamiento, ajustará un modelo de mezcla gaussiana en el espacio de características definido por las capas anteriores.

21.2.3 Capas ocultas

Durante el proceso de entrenamiento, una red neuronal muestra muchos valores de entrada **x** y muchos valores de salida correspondientes *y*. Mientras procesa un vector de entrada **x**, la red neuronal realiza varias computaciones intermedias antes de producir la salida *y*. Podemos pensar en los valores calculados en cada capa de la red como una *representación* diferente para la entrada **x**. Cada capa transforma la representación producida por la capa anterior para producir una nueva representación. La composición de todas estas transformaciones tiene éxito, si todo va bien, en transformar la entrada en la salida deseada. De hecho, una hipótesis de por qué el aprendizaje profundo funciona bien es que la compleja transformación de extremo a extremo que mapea de la entrada a la salida, digamos, de una imagen de entrada a la categoría de salida "jirafa", es descompuesta por las muchas capas en la composición de muchas transformaciones relativamente simples, cada una de las cuales es bastante fácil de aprender por un proceso de actualización local.

En el proceso de formar todas estas transformaciones internas, las redes profundas con frecuencia descubren representaciones intermedias significativas de los datos. Por ejemplo, una red que aprende a reconocer objetos complejos en imágenes permite formar capas internas que detectan subunidades útiles: bordes, esquinas, elipses, ojos, caras, gatos. O puede que no, las redes profundas permiten formar capas internas cuyo significado es opaco para los humanos, aunque la salida sigue siendo correcta.

Las capas ocultas de las redes neuronales suelen ser menos diversas que las capas de salida. Durante los primeros 25 años de investigación con redes multicapa (*circa* 1985–2010), los nodos internos usaron funciones de activación sigmoidal y tanh casi de manera exclusiva. A partir de 2010, la ReLU y la softplus se vuelven más populares, en parte porque se cree que evitan el problema de los gradientes que desaparecen, referido en la Sección 21.1.2. La experimentación con redes cada vez más profundas sugería que, en muchos casos, se obtenía mejor aprendizaje con redes profundas y relativamente estrechas en lugar de redes anchas y superficiales, dado un número total fijo de pesos. Un ejemplo típico de esto se muestra en la Figura 21.7 en la página 769.

Por supuesto, hay muchas otras estructuras a considerar para los grafos de cálculo, además de jugar con el ancho y la profundidad. En el momento de escribir, hay poco entendimiento de por qué algunas estructuras parecen funcionar mejor que otras para algún problema en particular. Con la experiencia, los profesionales adquieren cierta intuición sobre cómo diseñar redes y cómo solucionarlas cuando no funcionan, al igual que los chefs adquieren intuición sobre cómo diseñar recetas y cómo solucionarlas cuando saben desagradables. Por esta razón, las herramientas que facilitan la exploración y evaluación rápidas de diferentes estructuras son esenciales para el éxito en problemas del mundo real.

21.3 Redes convolucionales

Mencionamos en la Sección 21.2.1 que una imagen no permite ser pensada como un simple vector de valores de píxeles de entrada, porque la adyacencia de los píxeles realmente importa. Si construyéramos una red con capas completamente conectadas y una imagen como entrada, obtendríamos el mismo resultado, ya sea que entrenáramos con imágenes no alteradas o con imágenes cuyos píxeles se hubieran permutado de manera aleatoria. Además, supongamos que hay n píxeles y n unidades en la primera capa oculta, a la que los píxeles proporcionan entrada. Si la entrada y la primera capa oculta están completamente conectadas, eso significa n^2 pesos; para una imagen RGB típica de megapíxeles, eso son 9 billones de pesos. Un espacio de parámetros tan vasto requeriría vastos números de imágenes de entrenamiento y un enorme presupuesto computacional para ejecutar el algoritmo de entrenamiento.

Estas consideraciones sugieren que deberíamos construir la primera *capa oculta de manera que cada unidad oculta reciba entrada sólo de una pequeña región local de la imagen*. Esto mata dos pájaros de un tiro. Primero, respeta la adyacencia, al menos de manera local. (Y veremos más adelante que si las capas subsiguientes tienen la misma propiedad de localidad, entonces la red respetará la adyacencia en un sentido global). Segundo, reduce el número de pesos: si cada región local tiene $l \ll n$ píxeles, entonces habrá $ln \ll n^2$ pesos en total.

Hasta ahora, todo bien. Pero nos falta otra propiedad importante de las imágenes: en términos generales, cualquier cosa que sea detectable en una pequeña región local de la imagen, quizás un ojo o una hoja de césped, se vería igual si apareciera en otra pequeña región local de la imagen. En otras palabras, esperamos que los datos de la imagen muestren una **invariancia espacial** aproximada, al menos a escalas pequeñas a moderadas.[3] No necesariamente esperamos que las mitades superiores de las fotos se parezcan a las mitades inferiores, por lo que hay una escala más allá de la cual la invariancia espacial ya no se mantiene.

Invariancia espacial

La invariancia espacial local se logra restringiendo los l pesos que conectan una región local a una unidad en la capa oculta para que sean los mismos para cada unidad oculta. (Es decir, para las unidades ocultas i y j, los pesos $w_{1,i},\ldots, w_{l,i}$ son los mismos que $w_{1,j},\ldots,w_{l,j}$). Esto convierte las unidades ocultas en detectores de características que detectan la misma característica dondequiera que aparezca en la imagen. En general, queremos que la primera capa oculta detecte muchos tipos de características, no sólo una; por lo tanto, para cada región de imagen local podríamos tener d unidades ocultas con d conjuntos distintos de pesos. Esto significa que hay dl pesos en total, un número que no sólo es mucho más pequeño que n^2, sino que en realidad es independiente de n, el tamaño de la imagen. Así, al inyectar algún conocimiento previo, a saber, conocimiento de la adyacencia y la invariancia espacial, podemos desarrollar modelos que tienen muchos menos parámetros y aprenden mucho más rápido.

Red neuronal convolucional (CNN)

Kernel

Convolución

Una **red neuronal convolucional** (**CNN**) es aquella que contiene conexiones espacialmente locales, al menos en las primeras capas, y tiene patrones de pesos que se replican en las unidades de cada capa. Un patrón de pesos que se replica en varias regiones locales se llama **kernel** y el proceso de aplicar el kernel a los píxeles de la imagen (o a unidades organizadas de manera espacial en una capa subsiguiente) se llama **convolución**.[4]

Los kernels y las convoluciones son más fáciles de ilustrar en una dimensión que en dos o más, por lo que asumiremos un vector de entrada $\mathbf{x}$ de tamaño n, correspondiente a n píxeles en una

[3] Es posible aplicar ideas similares para procesar fuentes de datos de series de tiempo como las formas de onda de audio. Estos generalmente exhiben **invariancia temporal**: una palabra suena igual sin importar a qué hora del día se pronuncia. Las redes neuronales recurrentes (Sección 21.6) muestran de manera nativa invariancia temporal.

[4] En la terminología del procesamiento de señales, llamaríamos a esta operación una correlación cruzada, no una convolución. Pero se usa "convolución" dentro del campo de las redes neuronales.

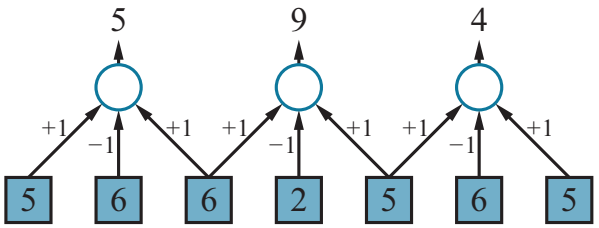


Figura 21.4 Un ejemplo de una operación de convolución unidimensional con un kernel de tamaño $l = 3$ y un paso $s = 2$. La respuesta máxima está centrada en el píxel de entrada más oscuro (menor intensidad). Los resultados se pasarían a través de una función de activación no lineal (no mostrada) antes de pasar a la siguiente capa oculta.

imagen dimensional, y un vector kernel $\mathbf{k}$ de tamaño l. (Para simplificar, asumiremos que l es un número impar). Todas las ideas se trasladan de manera sencilla a casos de dimensiones superiores.

Escribimos la operación de convolución usando el símbolo $*$, por ejemplo: $\mathbf{z} = \mathbf{x} * \mathbf{k}$. La operación se define de la siguiente manera:

$$z_i = \sum_{j=1}^{l} k_j x_{j+i-(l+1)/2}. \tag{21.8}$$

En otras palabras, para cada posición de salida i, tomamos el producto escalar entre el kernel $\mathbf{k}$ y un fragmento de $\mathbf{x}$ centrado en x_i con ancho l.

El proceso se ilustra en la Figura 21.4 para un vector de kernel [+1, −1, +1], que detecta un punto más oscuro en la imagen 1D. (La versión 2D podría detectar una línea más oscura). Vea que en este ejemplo los píxeles en los que se centran los kernels están separados por una distancia de 2 píxeles; decimos que el kernel se aplica con un **paso** $s = 2$. Vea que la capa de salida tiene menos píxeles de paso: debido al paso, el número de píxeles se reduce de n a casi n/s. (En dos dimensiones, el número de píxeles sería de alrededor de $n/s_x s_y$, donde s_x y s_y son los pasos en las direcciones x e y en la imagen). Decimos "aproximadamente" debido a lo que sucede en el borde de la imagen: en la Figura 21.4 la convolución se detiene en los bordes de la imagen, pero es posible rellenar la entrada con píxeles adicionales (ya sean ceros o copias de los píxeles exteriores) para que el kernel permita aplicarse $\lfloor n/s \rfloor$ veces. Para kernels pequeños, usamos $s = 1$, por lo que la salida tiene las mismas dimensiones que la imagen (ver Figura 21.5).

Paso

La operación de aplicar un kernel a través de una imagen se implementa usando un programa con bucles anidados adecuados; pero es posible formularlo como una única operación matricial, al igual que la aplicación de la matriz de peso en la Ecuación (21.1). Por ejemplo, la convolución ilustrada en la Figura 21.4 se puede ver como la siguiente multiplicación de matrices:

$$\begin{pmatrix} +1 & -1 & +1 & 0 & 0 & 0 & 0 \\ 0 & 0 & +1 & -1 & +1 & 0 & 0 \\ 0 & 0 & 0 & 0 & +1 & -1 & +1 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \\ 6 \\ 2 \\ 5 \\ 6 \\ 5 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \\ 4 \end{pmatrix}. \tag{21.9}$$

En esta matriz de peso, el kernel aparece en cada fila, desplazado según el paso en relación con la fila anterior, uno no necesariamente construiría la matriz de peso explicita: es ceros, después de

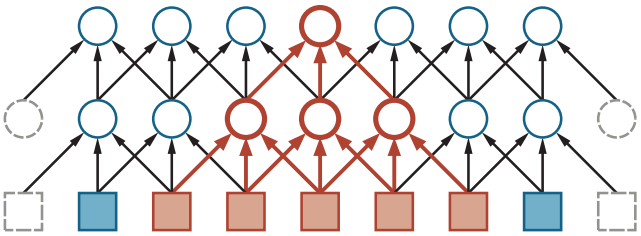


Figura 21.5 Las dos primeras capas de una CNN para una imagen 1D con un tamaño de kernel $l = 3$ y un paso $s = 1$. Se agrega relleno en los extremos izquierdo y derecho para mantener las capas ocultas del mismo tamaño que la entrada. Se muestra en rojo el campo receptivo de una unidad en la segunda capa oculta. En general, cuanto más profunda es la unidad, mayor es el campo receptivo.

todo; pero el hecho de que la convolución es una operación matricial lineal sirve como un recordatorio de que el descenso de gradiente se aplica de manera fácil y efectiva a las CNN, al igual que a las redes neuronales simples.

Como antes se mencionó, habrá d kernels, no sólo uno; por lo tanto, con un *stride* (paso) de 1, la salida será d veces mayor. Esto significa que una matriz de entrada bidimensional se convierte en una matriz tridimensional de unidades ocultas, donde la tercera dimensión es de tamaño d. Es importante organizar la capa oculta de esta manera, para que todas las salidas del kernel de una ubicación de imagen en particular permanezcan asociadas con esa ubicación. A diferencia de las dimensiones espaciales de la imagen, esta "dimensión del kernel" adicional no tiene ninguna propiedad de adyacencia, por lo que *no* tiene sentido ejecutar convoluciones a lo largo de ella.

Campo receptivo

Las CNN se inspiraron en modelos de la corteza visual propuestos en neurociencia. En esos modelos, el **campo receptivo** de una neurona es la porción de la entrada sensorial que afecta la activación de esa neurona. En una CNN, el campo receptivo de una unidad en la primera capa oculta es pequeño, sólo del tamaño del kernel, es decir, l píxeles. En las capas más profundas de la red, puede ser más grande. La Figura 21.5 ilustra esto para una unidad en la segunda capa oculta, cuyo campo receptivo contiene cinco píxeles. Cuando el *stride* es 1, como en la figura, un nodo en la capa oculta m tendrá un campo receptivo de tamaño $(l - 1)m + 1$; por lo tanto, el crecimiento es lineal en m. (En una imagen 2D, cada dimensión del campo receptivo crece de manera lineal con m, por lo que el área crece de manera cuadrática). Cuando el *stride* es mayor que 1, cada píxel en la capa m representa s píxeles en la capa $m - 1$; por lo tanto, el campo receptivo crece como $O(ls^m)$; es decir, exponencialmente con la profundidad. El mismo efecto ocurre con las capas de agrupación, que discutimos a continuación.

21.3.1 Agrupación y submuestreo

Agrupación

Una capa de **agrupación** en una red neuronal resume un conjunto de unidades adyacentes de la capa anterior con un solo valor. La agrupación funciona igual que una capa de convolución, con un tamaño de kernel l y un *stride* s, pero la operación que se aplica es fija en lugar de aprendida. En la mayoría de los casos, no se asocia ninguna función de activación con la capa de agrupación. Hay dos formas comunes de agrupación:

- El *average-pooling* calcula el valor promedio de sus l entradas. Esto es idéntico a la convolución con un vector de kernel uniforme $\mathbf{k} = [1/l,...,1/l]$. Si establecemos $l = s$, el efecto es hacer más gruesa la resolución de la imagen, es decir, **submuestrearla**, por un factor de s. Un objeto que ocupaba, digamos, $10s$ píxeles ahora ocuparía sólo 10 píxeles después de la agrupación. Lo

Submuestreo

mismo el clasificador aprendido que sería capaz de reconocer el objeto a un tamaño de 10 píxeles en la imagen original ahora sería capaz de reconocer ese objeto en la imagen agrupada, aun si era demasiado grande para reconocer en la imagen original. Es decir, el *average-pooling* facilita el reconocimiento a múltiples escalas. También reduce el número de pesos requeridos en las capas siguientes, lo que lleva a un costo computacional más bajo y quizás a un aprendizaje más rápido.

- El *max-pooling* calcula el valor máximo de sus l entradas. También es posible usar para el submuestreo, pero tiene una semántica algo diferente. Supongamos que aplicamos *max-pooling* a la capa oculta [5, 9, 4] en la Figura 21.4: el resultado sería un 9, indicando que en algún lugar de la imagen de entrada hay un punto más oscuro que es detectado por el kernel. En otras palabras, el *max-pooling* actúa como una especie de disyunción lógica, diciendo que una característica existe en algún lugar en el campo receptivo de la unidad.

Si el objetivo es clasificar la imagen en una de las c categorías, entonces la capa final de la red será un softmax con c unidades de salida. Las primeras capas de la CNN tienen el tamaño de la imagen, por lo que en algún lugar intermedio debe haber reducciones significativas en el tamaño de la capa. Las capas de convolución y las capas de agrupación con *stride* mayor que 1 sirven para reducir el tamaño de la capa. También es posible reducir el tamaño de la capa teniendo una capa completamente conectada con menos unidades que la capa anterior. Las CNN en la mayoría de los casos tienen una o dos de estas capas antes de la capa softmax final.

21.3.2 Operaciones de tensor en CNNs

En las Ecuaciones (21.1) y (21.3) observamos que el uso de la notación de vector y matriz es útil para mantener las derivaciones matemáticas simples y refinadas y proporcionar descripciones concisas de los grafos de cálculo. Los vectores y las matrices son casos especiales unidimensionales y bidimensionales de **tensores**, que (en terminología de aprendizaje profundo) son arreglos multidimensionales de cualquier dimensión.[5]

Tensores

Para las CNN, los tensores son una forma de llevar un registro de la "forma" de los datos a medida que avanzan a través de las capas de la red. Esto es importante porque toda la noción de convolución depende de la idea de adyacencia: se supone que los elementos de datos adyacentes están relacionados de manera sistemática, por lo que tiene sentido aplicar operadores a regiones locales de los datos. Además, con los primitivos de lenguaje adecuados para construir tensores y aplicar operadores, las capas mismas permiten describirse de manera concisa como mapas de entradas de tensor a salidas de tensor.

Una razón final para describir las CNN en términos de operaciones tensoriales es la eficiencia computacional: dada una descripción de una red como una secuencia de operaciones tensoriales, un paquete de software de aprendizaje profundo puede generar código compilado que está altamente optimizado para el sustrato computacional subyacente. Las cargas de trabajo de aprendizaje profundo se ejecutan en GPUs (unidades de procesamiento gráfico) o TPUs (unidades de procesamiento tensorial), que ponen a disposición un alto grado de paralelismo. Por ejemplo, uno de los pods TPU de tercera generación de Google tiene un rendimiento equivalente a unos diez millones de portátiles. Aprovechar estas capacidades es esencial si se está entrenando una gran CNN en una gran base de datos de imágenes. Por lo tanto, es común procesar no una imagen a la vez, sino muchas imágenes en paralelo; como veremos en la Sección 21.4, esto también se alinea bien con la forma en que el algoritmo de descenso de gradiente estocástico calcula gradientes con respecto a un minibatch de ejemplos de entrenamiento.

Pongamos todo esto junto en forma de un ejemplo. Supongamos que estamos entrenando en imágenes RGB de 256×256 con un tamaño de minibatch de 64. La entrada en este caso será un

[5] La definición matemática adecuada de los tensores requiere que se mantengan ciertas invarianzas bajo un cambio de base.

tensor cuatridimensional de tamaño 256 × 256 × 3 × 64. Luego aplicamos 96 kernels de tamaño 5 × 5 × 3 con un *stride* de 2 en ambas direcciones x e y en la imagen. Esto da un tensor de salida de tamaño 128 × 128 × 96 × 64. Este tensor también se llama **mapa de características**, ya que muestra cómo cada característica extraída por un kernel aparece en toda la imagen; en este caso está compuesto por 96 **canales**, donde cada canal lleva información de una característica. Vea que, a diferencia del tensor de entrada, este mapa de características ya no tiene canales de color dedicados; sin embargo, la información del color todavía está presente en los diversos canales de características si el algoritmo de aprendizaje encuentra que el color es útil para las predicciones finales de la red.

Mapa de características

Canal

21.3.3 Redes residuales

Red residual

Las **redes residuales** son un enfoque popular y exitoso para construir redes muy profundas que evitan el problema de los gradientes que desaparecen.

Los modelos profundos típicos utilizan capas que aprenden una nueva representación en la capa i al reemplazar la representación en la capa i - 1. Si usamos la notación de matriz-vector que introdujimos en la Ecuación (21.3), con $\mathbf{z}^{(i)}$, siendo los valores de las unidades en la capa i, tenemos

$$\mathbf{z}^{(i)} = f(\mathbf{z}^{(i-1)}) = \mathbf{g}^{(i)}(\mathbf{W}^{(i)}\mathbf{z}^{(i-1)}).$$

Debido a que cada capa reemplaza la representación de la capa anterior, todas las capas deben aprender a hacer algo útil. Cada capa debe, al menos, preservar la información relevante para la tarea contenida en la capa anterior. Si establecemos $\mathbf{W}^{(i)} = 0$ para cualquier capa i, toda la red deja de funcionar. Si también establecemos $\mathbf{W}^{(i-1)} = \mathbf{0}$, la red no sería capaz ni de aprender: la capa i no aprendería porque no observaría ninguna variación en su entrada desde la capa $i - 1$, y la capa $i - 1$ no aprendería porque el gradiente retropropagado desde la capa i siempre sería cero. Por supuesto, estos son ejemplos extremos, pero ilustran la necesidad de que las capas sirvan como conductos para las señales que pasan a través de la red.

La idea clave de las redes residuales es que una capa debería *perturbar* la representación de la capa anterior en lugar de *reemplazarla* por completo. Si la perturbación aprendida es pequeña, la siguiente capa está cerca de ser una copia de la capa anterior. Esto se logra con la siguiente ecuación para la capa i en términos de la capa $i - 1$:

$$\mathbf{z}^{(i)} = \mathbf{g}_r^{(i)}(\mathbf{z}^{(i-1)} + f(\mathbf{z}^{(i-1)})), \tag{21.10}$$

donde $\mathbf{g}_r$ denota las funciones de activación para la capa residual. Aquí pensamos en f como el **residual**, perturbando el comportamiento por defecto de pasar la capa $i-1$ a la capa i. La función usada para calcular el residual es una red neuronal con una capa no lineal combinada con una capa lineal:

Residual

$$f(\mathbf{z}) = \mathbf{V}\mathbf{g}(\mathbf{W}\mathbf{z}),$$

donde $\mathbf{W}$ y $\mathbf{V}$ son matrices de pesos aprendidas con los pesos de sesgo habituales añadidos.

Las redes residuales hacen posible aprender redes más profundas de manera fiable. Considere lo que sucede si establecemos $\mathbf{V} = \mathbf{0}$ para una capa particular con el fin de desactivar esa capa. Entonces el residual f desaparece y la Ecuación (21.10) se simplifica a

$$\mathbf{z}^{(i)} = \mathbf{g}_r(\mathbf{z}^{(i-1)}).$$

Ahora supongamos que $\mathbf{g}_r$ consiste en funciones de activación ReLU y que $\mathbf{z}^{(i-1)}$ también aplica una función ReLU a sus entradas: $\mathbf{z}^{(i-1)} = \text{ReLU}(\mathbf{in}^{(i-1)})$. En ese caso tenemos

$$\mathbf{z}^{(i)} = \mathbf{g}_r(\mathbf{z}^{(i-1)}) = \text{ReLU}(\mathbf{z}^{(i-1)}) = \text{ReLU}(\text{ReLU}(\mathbf{in}^{(i-1)})) = \text{ReLU}(\text{in}^{(i-1)}) = \mathbf{z}^{(i-1)},$$

donde el penúltimo paso sigue porque ReLU(ReLU(x))=ReLU(x). En otras palabras, en redes residuales con activaciones ReLU, una capa con cero pesos pasa sus entradas sin ningún cambio. El resto de la red funciona como si la capa nunca hubiera existido. Mientras que las redes tradicionales deben aprender a propagar información y están sujetas a un fracaso catastrófico de la propagación de información para malas elecciones de los parámetros, las redes residuales propagan información por defecto.

Las redes residuales se utilizan con capas convolucionales en aplicaciones de visión, pero de hecho son una herramienta de propósito general que hace que las redes profundas sean más robustas y permite a los investigadores experimentar con diseños de red complejos y heterogéneos. En el momento de escribir, no es raro ver redes residuales con cientos de capas. El diseño de estas redes está evolucionando de manera rápida, por lo que cualquier especificidad adicional que pudiéramos proporcionar quizás estaría desactualizada antes de llegar a la forma impresa. Los lectores que deseen conocer las mejores arquitecturas para aplicaciones específicas deben consultar las publicaciones de investigación recientes.

21.4 Algoritmos de aprendizaje

Entrenar una red neuronal consiste en modificar los parámetros de la red para minimizar la función de pérdida en el conjunto de entrenamiento. En principio, se podría utilizar cualquier tipo de algoritmo de optimización. En la práctica, las redes neuronales modernas casi siempre se entrenan con alguna variante del descenso de gradiente estocástico (SGD).

Cubrimos el descenso de gradiente estándar y su versión estocástica en la Sección 19.6.2. Aquí, el objetivo es minimizar la pérdida $L(\mathbf{w})$, donde $\mathbf{w}$ representa todos los parámetros de la red. Cada paso de actualización en el proceso de descenso de gradiente se ve así:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w}),$$

donde α es la tasa de aprendizaje. Para el descenso de gradiente estándar, la pérdida L se define con respecto al conjunto de entrenamiento completo. Para SGD, se define con respecto a un minibatch de m ejemplos elegidos de manera aleatoria en cada paso.

Como se señaló en la Sección 4.2, la literatura sobre métodos de optimización para espacios continuos de alta dimensión incluye innumerables mejoras al descenso de gradiente básico. No los vamos a cubrir todos aquí, pero vale la pena mencionar algunas consideraciones importantes relevantes para el entrenamiento de redes neuronales:

- Para la mayoría de las redes que resuelven problemas del mundo real, tanto la dimensionalidad de $\mathbf{w}$ como el tamaño del conjunto de entrenamiento son muy grandes. Estas consideraciones abogan a favor de usar SGD con un tamaño de minibatch relativamente pequeño m: la estocasticidad ayuda al algoritmo a escapar de mínimos locales pequeños en el espacio de peso de alta dimensión (como en el recocido simulado, ver página 114); y el pequeño tamaño del minibatch asegura que el costo computacional de cada paso de actualización de peso es una pequeña constante, independiente del tamaño del conjunto de entrenamiento.
- Debido a que la contribución del gradiente de cada ejemplo de entrenamiento en el minibatch SGD se calcula de forma independiente, el tamaño del minibatch en la mayoría de los casos se elige para aprovechar al máximo el paralelismo de hardware en GPUs o TPUs.
- Para mejorar la convergencia, una buena idea es usar una tasa de aprendizaje que disminuye con el tiempo. Elegir el horario correcto suele ser una cuestión de prueba y error.
- Cerca de un mínimo local o global de la función de pérdida con respecto al conjunto de entrenamiento completo, los gradientes estimados a partir de minibatches pequeños tendrán

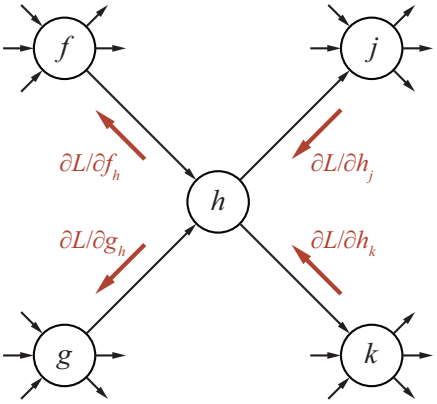


Figura 21.6 Ilustración de la retropropagación de la información del gradiente en un grafo de cálculo arbitrario. El cálculo hacia adelante de la salida de la red procede de izquierda a derecha, mientras que la retropropagación de los gradientes va de derecha a izquierda.

una alta varianza y puede apuntar en la dirección equivocada, dificultando la convergencia. Una solución es aumentar el tamaño del minibatch a medida que avanza el entrenamiento; otra es incorporar la idea de **momentum**, que mantiene un promedio móvil de los gradientes de los minibatches pasados para compensar los tamaños de minibatch pequeños.

Momentum

- Se debe tener cuidado para mitigar las inestabilidades numéricas que surgen debido a desbordamiento, subflujo y error de redondeo. Estos son problemáticos con el uso de exponenciales en funciones de activación softmax, sigmoide y tanh, y con los cálculos iterados en redes muy profundas y redes recurrentes (Sección 21.6) que conducen a activaciones y gradientes que desaparecen y explotan.

En general, el proceso de aprendizaje de los pesos de la red suele ser uno que muestra rendimientos decrecientes. Corremos hasta que ya no es práctico disminuir el error de prueba corriendo más tiempo. Por lo general, esto no significa que hayamos alcanzado un mínimo global o incluso local de la función de pérdida. En cambio, significa que tendríamos que hacer un número grande de pasos muy pequeños para seguir reduciendo el costo, o que los pasos adicionales sólo causarían sobreajuste, o que las estimaciones del gradiente son demasiado inexactas para hacer más progresos.

21.4.1 Calculando gradientes en grafos de computación

En la página 755, derivamos el gradiente de la función de pérdida con respecto a los pesos en una red específica (muy simple). Vimos que el gradiente se podía calcular retropropagando la información de error desde la capa de salida de la red hasta las capas ocultas. También dijimos que este resultado se mantiene para cualquier grafo de computación hacia adelante. Aquí, explicamos cómo funciona.

La Figura 21.6 muestra un nodo genérico en un grafo de computación. (El nodo h tiene grado de entrada y de salida 2, pero nada en el análisis depende de esto). Durante el paso hacia adelante, el nodo calcula alguna función arbitraria h a partir de sus entradas, que provienen de los nodos f y g. A su vez, h alimenta su valor a los nodos j y k.

El proceso de retropropagación pasa mensajes hacia atrás a lo largo de cada enlace en la red. En cada nodo, se recogen los mensajes entrantes y se calculan nuevos mensajes para pasar hacia

atrás a la siguiente capa. Como muestra la figura, los mensajes son todas las derivadas parciales de la pérdida L. Por ejemplo, el mensaje hacia atrás $\partial L/\partial h_j$ es la derivada parcial de L con respecto a la primera entrada de j, que es el mensaje hacia adelante de h a j. Ahora, h afecta a L a través de j y k, por lo que tenemos

$$\partial L/\partial h = \partial L/\partial h_j + \partial L/\partial h_k. \tag{21.11}$$

Con esta ecuación, el nodo h permite calcular la derivada de L con respecto a h sumando los mensajes entrantes de j y k. Ahora, para calcular los mensajes salientes $\partial L/\partial f_h$ y $\partial L/\partial g_h$, usamos las siguientes ecuaciones:

$$\frac{\partial L}{\partial f_h} = \frac{\partial L}{\partial h}\frac{\partial h}{\partial f_h} \quad \text{y} \quad \frac{\partial L}{\partial g_h} = \frac{\partial L}{\partial h}\frac{\partial h}{\partial g_h} \tag{21.12}$$

En la Ecuación (21.12), $\partial L/\partial h$ ya fue calculado por la Ecuación (21.11), y $\partial h/\partial f_h$ y $\partial h/\partial g_h$ son las derivadas de h con respecto a sus primeros y segundos argumentos, respectivamente. Por ejemplo, si h es un nodo de multiplicación, es decir, $h(f,g) = f \cdot g$, entonces $\partial h/\partial f_h = g$ y $\partial h/\partial g_h = f$. Los paquetes de software para el aprendizaje profundo suelen venir con una biblioteca de tipos de nodos (adición, multiplicación, sigmoide, etc.), cada uno de los cuales sabe cómo calcular sus propias derivadas según sea necesario para la Ecuación (21.12).

El proceso de retropropagación comienza con los nodos de salida, donde cada mensaje inicial $\partial L/\partial \hat{y}_j$ se calcula de manera directa a partir de la expresión para L en términos del valor predicho $\hat{\mathbf{y}}$ y el valor verdadero $\mathbf{y}$ de los datos de entrenamiento. En cada nodo interno, los mensajes hacia atrás entrantes se suman de acuerdo con la Ecuación (21.11) y se generan los mensajes salientes a partir de la Ecuación (21.12). El proceso termina en cada nodo en el grafo de computación que representa un peso w (por ejemplo, los óvalos malva claro en la Figura 21.3(b)). En ese punto, la suma de los mensajes entrantes a w es $\partial L/\partial w$, el gradiente que necesitamos para actualizar w. El Ejercicio 21.BPRE pide que aplique este proceso a la red simple en la Figura 21.3 para volver a derivar las expresiones del gradiente en las Ecuaciones (21.4) y (21.5).

El intercambio de pesos, como se usa en las redes convolucionales (Sección 21.3) y las redes recurrentes (Sección 21.6), se maneja tratando cada peso compartido como un solo nodo con múltiples arcos salientes en el grafo de computación. Durante la retropropagación, esto resulta en múltiples mensajes de gradiente entrantes. Por la Ecuación (21.11), significa que el gradiente para el peso compartido es la suma de las contribuciones del gradiente de cada lugar donde se usa en la red.

Está claro a partir de esta descripción del proceso de retropropagación que su costo computacional es lineal en el número de nodos en el grafo de computación, al igual que el costo de la computación hacia adelante. Además, debido a que los tipos de nodos suelen estar fijos cuando se diseña la red, todos los cálculos de gradiente se preparan en forma simbólica con anticipación y se compilan en código de manera eficiente para cada nodo en el grafo. También se debe notar que los mensajes en la Figura 21.6 no necesitan ser escalares: podrían ser vectores, matrices o tensores de dimensiones superiores, de modo que los cálculos de gradiente se mapean en GPUs o TPUs para beneficiarse del paralelismo.

Una desventaja de la retropropagación es que requiere almacenar la mayoría de los valores intermedios que se calcularon durante la propagación hacia adelante para calcular los gradientes en el paso hacia atrás. Esto significa que el costo total de memoria de entrenar la red es proporcional al número de unidades en toda la red. Por lo tanto, incluso si la red en sí misma está representada sólo de manera implícita por código de propagación con muchos bucles, y no explícitamente por una estructura de datos, todos los resultados intermedios de ese código de propagación deben almacenarse de manera explícita.

21.4.2 Normalización por lotes

Normalización por lotes

La **normalización por lotes** es una técnica comúnmente usada que mejora la tasa de convergencia del SGD al reescalar los valores generados en las capas internas de la red a partir de los ejemplos dentro de cada minibatch. Aunque las razones de su efectividad no se comprenden bien al momento de la escritura, se incluye porque confiere beneficios significativos en la práctica. De cierto modo, la normalización por lotes parece tener efectos similares a los de la red residual.

Considere un nodo z en alguna parte de la red: los valores de z para los m ejemplos en un minibatch son $z_1,...,z_m$. La normalización por lotes reemplaza cada z_i con una nueva cantidad $\hat{z}i$:

$$\hat{z}i = \gamma \frac{zi - \mu}{\sqrt{\in + \sigma^2}} + \beta,$$

donde μ es el valor medio de z en el minibatch, σ es la desviación estándar de $z_1,...,z_m$, $\in$ es una constante añadida para evitar la división por cero, y γ y β son parámetros aprendidos.

La normalización por lotes estandariza la media y la varianza de los valores, según lo determinado por los valores de β y γ. Esto hace que sea mucho más sencillo entrenar una red profunda. Sin normalización por lotes, la información permite perderse si los pesos de una capa son demasiado pequeños, y la desviación estándar en esa capa se reduce a casi cero. La normalización por lotes previene que esto ocurra. También reduce la necesidad de una cuidadosa inicialización de todos los pesos en la red para asegurar que los nodos en cada capa estén en la región de operación correcta para permitir la propagación de información.

Con la normalización por lotes, usualmente se incluyen β y γ, que pueden ser específicos de nodos o de capas, entre los parámetros de la red, de modo que se incluyen en el proceso de aprendizaje. Después del entrenamiento, β y γ se fijan en sus valores aprendidos.

21.5 Generalización

Hasta ahora se ha descrito cómo ajustar una red neuronal a su conjunto de entrenamiento, pero en el aprendizaje automático el objetivo es generalizar a nuevos datos que no se han visto previamente, medido por el rendimiento en un conjunto de prueba. En esta sección, se abordan tres enfoques para mejorar el rendimiento de generalización: elegir la arquitectura de red adecuada, penalizar grandes pesos y perturbar de manera aleatoria los valores que pasan a través de la red durante el entrenamiento.

21.5.1 Elegir una arquitectura de red

Se ha dedicado mucho esfuerzo en la investigación de aprendizaje profundo para encontrar arquitecturas de red generalizadas. De hecho, para cada tipo particular de datos —imágenes, voz, texto, video, etc.— gran parte del progreso en el rendimiento ha surgido de explorar diferentes tipos de arquitecturas de red y variar el número de capas, su conectividad y los tipos de nodos en cada capa.[6]

Algunas arquitecturas de redes neuronales están diseñadas de manera explícita para generalizar tipos particulares de datos: las redes convolucionales codifican la idea de que el mismo extractor de características es útil en todas las ubicaciones a través de una cuadrícula espacial, y las redes recurrentes codifican la idea de que la misma regla de actualización es útil en todos los puntos de una secuencia de datos. En la medida en que estas suposiciones sean válidas, se espera que las arquitecturas convolucionales generalicen imágenes y las redes recurrentes generalicen texto y señales de audio.

[6] Observe que gran parte de este trabajo incremental y exploratorio es realizado por estudiantes de posgrado, algunos han llamado al proceso descenso del estudiante de posgrado (GSD).

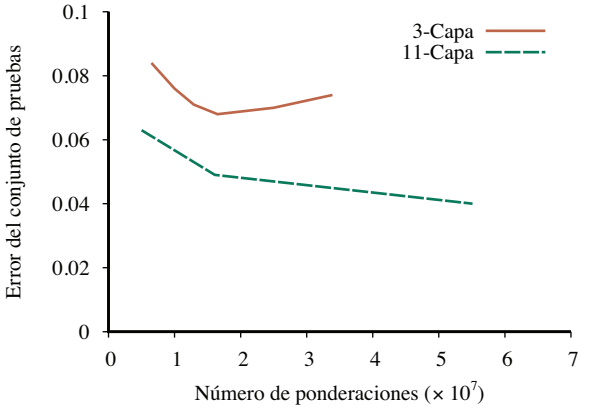


Figura 21.7 Error del conjunto de pruebas en función de la anchura de la capa (medida por el número total de ponderaciones) para redes convolucionales de tres y once capas. Los datos provienen de las primeras versiones del sistema de Google para transcribir direcciones en fotos tomadas por automóviles de Street View (Goodfellow *et al.,* 2014).

Uno de los hallazgos empíricos más importantes en el campo del aprendizaje profundo es que al comparar dos redes con números similares de pesos, la red más profunda ofrece un mejor rendimiento de generalización. La Figura 21.7 muestra este efecto para al menos una aplicación del mundo real: reconocer números de casas. Los resultados muestran que, para cualquier número fijo de parámetros, una red de once capas da un error de conjunto de prueba mucho menor que una red de tres capas.

Los sistemas de aprendizaje profundo funcionan bien en algunas, pero no en todas las tareas. Para tareas con entradas de alta dimensión: imágenes, video, señales de voz, etc., se desempeñan mejor que cualquier otro enfoque puro de aprendizaje automático. La mayoría de los algoritmos descritos en el Capítulo 19 permiten manejar entradas de alta dimensión sólo si se preprocesan usando características diseñadas de manera manual para reducir la dimensionalidad. Este enfoque de preprocesamiento, que prevaleció antes de 2010, no ha producido un rendimiento comparable al logrado por los sistemas de aprendizaje profundo.

Note que los modelos de aprendizaje profundo están capturando algunos aspectos importantes de estas tareas. En particular, su éxito implica que las tareas son resueltas por programas paralelos con un número relativamente pequeño de pasos (10 a 10^3 en lugar de, por ejemplo, 10^7). Esto quizá no es sorprendente, porque estas tareas son resueltas por el cerebro en menos de un segundo, tiempo suficiente para sólo unas pocas decenas de disparos neuronales secuenciales. Además, al examinar las representaciones de capas internas aprendidas por redes convolucionales profundas para tareas de visión, encontramos evidencia de que los pasos de procesamiento parecen implicar la extracción de una secuencia de representaciones cada vez más abstractas de la escena, comenzando con pequeños bordes, puntos y características de esquina y terminando con objetos enteros y arreglos de múltiples objetos.

Por otro lado, debido a que son circuitos simples, los modelos de aprendizaje profundo carecen del poder expresivo de composición y cuantificación que vemos en la lógica de primer orden (Capítulo 8) y las gramáticas libres de contexto (Capítulo 23).

Aunque los modelos de aprendizaje profundo generalizan bien en muchos casos, también pueden producir errores poco intuitivos. Tienden a producir mapeos de entrada-salida que son discontinuos, por lo que un pequeño cambio en una entrada puede causar un gran cambio en la salida. Por

ejemplo, es posible alterar sólo unos pocos píxeles en una imagen de un perro y hacer que la red clasifique al perro como un avestruz o un autobús escolar, incluso aunque la imagen alterada todavía se vea como un perro. Una imagen alterada de este tipo se llama **ejemplo adversarial**.

Ejemplo adversarial

En espacios de baja dimensión es difícil encontrar ejemplos adversarios. Pero para una imagen con un millón de valores de píxeles, aunque la mayoría de estos contribuyen a que la imagen se clasifique en el medio de la región "perro" del espacio, con frecuencia es el caso de que hay algunas dimensiones donde el valor del píxel está cerca del límite a otra categoría. Un adversario con la capacidad de ingeniería inversa de la red permite encontrar la diferencia de vector más pequeña que movería la imagen sobre la frontera.

Cuando se descubrieron los ejemplos adversarios por primera vez, provocaron dos carreras mundiales: una para encontrar algoritmos de aprendizaje y arquitecturas de red que no serían susceptibles al ataque adversario, y otra para crear ataques adversarios cada vez más efectivos contra todo tipo de sistemas de aprendizaje. Hasta ahora, los atacantes parecen estar adelante. De hecho, mientras que inicialmente se suponía que uno necesitaría acceso a las partes internas de la red entrenada para construir un ejemplo adversarial específico para esa red, resultó que uno puede construir ejemplos adversarios *robustos* que engañan a múltiples redes con diferentes arquitecturas, hiperparámetros y conjuntos de entrenamiento. Estos hallazgos sugieren que los modelos de aprendizaje profundo reconocen objetos de formas que son bastante diferentes al sistema visual humano.

21.5.2 Búsqueda de arquitectura neural

Desafortunadamente, aún no tenemos un conjunto claro de pautas para ayudarlo a elegir la mejor arquitectura de red para un problema particular. El éxito en la implementación de una solución de aprendizaje profundo requiere experiencia y buen juicio.

Desde los primeros días de la investigación de redes neuronales, se han hecho intentos para automatizar el proceso de selección de arquitectura. Podemos pensar en esto como un caso de ajuste de hiperparámetros (Sección 19.4.4), donde los hiperparámetros determinan la profundidad, el ancho, la conectividad y otros atributos de la red. Sin embargo, hay tantas opciones que se deben hacer que enfoques simples como la búsqueda en cuadrícula no permite cubrir todas las posibilidades en un tiempo razonable.

Búsqueda de arquitectura neural

Por lo tanto, es común usar la **búsqueda de arquitectura neural** para explorar el espacio de estado de las posibles arquitecturas de red. Muchas de las técnicas de búsqueda y aprendizaje que cubrimos antes en el libro se han aplicado a la búsqueda de arquitectura neural.

Los algoritmos evolutivos han sido populares porque es sensato hacer tanto la recombinación (unir partes de dos redes) como la mutación (agregar o eliminar una capa o cambiar un valor de parámetro). El ascenso de colinas también permite usarse con estas mismas operaciones de mutación. Algunos investigadores han enmarcado el problema como aprendizaje por refuerzo, y algunos como optimización bayesiana. Otra posibilidad es tratar las posibilidades arquitectónicas como un espacio diferenciable continuo y usar el descenso de gradiente para encontrar una solución óptima de manera local.

Para todas estas técnicas de búsqueda, un gran desafío es estimar el valor de una red candidata. La forma directa de evaluar una arquitectura es entrenarla en un conjunto de prueba para múltiples lotes y luego evaluar su precisión en un conjunto de validación. Pero con redes grandes eso podría llevar muchos días de GPU.

Por lo tanto, ha habido muchos intentos de acelerar este proceso de estimación eliminando o al menos reduciendo el costoso proceso de entrenamiento. Podemos entrenar en un conjunto de datos más pequeño. Podemos entrenar para un pequeño número de lotes y predecir cómo mejoraría la red con más lotes. Podemos usar una versión reducida de la arquitectura de la red que esperamos conserva las propiedades de la versión completa. Podemos entrenar una red grande y luego

buscar subgrafos de la red que funcionen mejor; esta búsqueda es rápida porque los subgrafos comparten parámetros y no tienen que ser reentrenados.

Otro enfoque es aprender una función de evaluación heurística (como se hizo para la búsqueda A*). Es decir, comenzar eligiendo unos pocos cientos de arquitecturas de red y entrenarlas y evaluarlas. Eso nos da un conjunto de datos de pares (red, puntuación). Luego aprenda un mapeo de las características de una red a una puntuación predicha. A partir de ese punto, podemos generar un gran número de redes candidatas y estimar de manera rápida su valor. Después de una búsqueda a través del espacio de redes, la(s) mejor(es) permite(n) ser evaluada(s) con un procedimiento de entrenamiento completo.

21.5.3 Decaimiento de peso

En la Sección 19.4.3 vimos que la **regularización**, limita la complejidad de un modelo, y ayuda a la generalización. Esto es cierto para los modelos de aprendizaje profundo también. En el contexto de las redes neuronales llamamos a este enfoque **decaimiento de peso**.

Decaimiento de peso

El decaimiento de peso consiste en agregar una penalización $\lambda\Sigma_{i,j}W_{i,j}^2$ a la función de pérdida utilizada para entrenar la red neuronal, donde λ es un hiperparámetro que controla la fuerza de la penalización y la suma se toma sobre todos los pesos en la red. Usar $\lambda = 0$ es equivalente a no usar decaimiento de peso, mientras que usar valores más grandes de λ alienta a los pesos a volverse pequeños. Es común usar decaimiento de peso con λ cerca de 10^{-4}.

Elegir una arquitectura de red específica puede verse como una restricción absoluta en el espacio de hipótesis: una función es representable dentro de esa arquitectura o no lo es. Los términos de penalización de la función de pérdida como el decaimiento de peso ofrecen una restricción más suave: las funciones representadas con pesos grandes están en la familia de funciones, pero el conjunto de entrenamiento debe proporcionar más evidencia a favor de estas funciones de la que se requiere para elegir una función con pesos pequeños.

No es sencillo interpretar el efecto del decaimiento de peso en una red neuronal. En redes con funciones de activación sigmoideas, se hipotetiza que el decaimiento de peso ayuda a mantener las activaciones cerca de la parte lineal de la sigmoide, evitando la región de operación plana que conduce a gradientes que desaparecen. Con funciones de activación ReLU, el decaimiento de peso parece ser benéfico, pero la explicación que tiene sentido para las sigmoideas ya no se aplica porque la salida de ReLU es lineal o cero. Además, con conexiones residuales, el decaimiento de peso alienta a la red a tener pequeñas diferencias entre capas consecutivas en lugar de pequeños valores de peso absolutos. A pesar de estas diferencias en el comportamiento del decaimiento de peso en muchas arquitecturas, el decaimiento de peso sigue siendo útil.

Una explicación para el efecto beneficioso del decaimiento de peso es que implementa una forma de aprendizaje máximo a posteriori (MAP) (ver página 723). Dejando que $\mathbf{X}$ e $\mathbf{y}$ representen las entradas y salidas en todo el conjunto de entrenamiento, la hipótesis h_{MAP} satisface

$$\begin{aligned} h_{\text{MAP}} &= \operatorname*{argmax}_{\mathbf{w}} P(\mathbf{y}|\mathbf{X},\mathbf{W})P(\mathbf{W}) \\ &= \operatorname*{argmin}_{\mathbf{w}} [-\log P(\mathbf{y}|\mathbf{X},\mathbf{W}) - \log P(\mathbf{W})]. \end{aligned}$$

El primer término es la pérdida de entropía cruzada habitual; el segundo término prefiere pesos probables bajo una distribución previa. Esto se alinea con una función de pérdida regularizada si establecemos

$$\log P(\mathbf{W}) = -\lambda \sum_{i,j} W_{i,j}^2,$$

lo que significa que $P(\mathbf{W})$ es una prioridad gaussiana de media cero.

21.5.4 Abandono

Abandono

Otra forma en que podemos intervenir para reducir el error del conjunto de prueba de una red, a costa de dificultar la adaptación al conjunto de entrenamiento, es usar el **abandono**. En cada paso del entrenamiento, el abandono aplica un paso de aprendizaje de retropropagación a una nueva versión de la red que se crea desactivando un subconjunto elegido al azar de las unidades. Esta es una aproximación muy tosca y de muy bajo costo para entrenar un gran conjunto de diferentes redes (ver Sección 19.8).

En específico, supongamos que estamos usando descenso de gradiente estocástico con tamaño de minibatch m. Para cada minibatch, el algoritmo de abandono aplica el siguiente proceso a cada nodo en la red: con probabilidad p, la salida de la unidad se multiplica por un factor de $1/p$; de lo contrario, la salida de la unidad se fija en cero. El abandono se aplica a las unidades en las capas ocultas con $p = 0.5$; para las unidades de entrada, un valor de $p = 0.8$ resulta ser el más efectivo. Este proceso produce una red adelgazada con la mitad de las unidades que la original, a la cual se aplica la retropropagación con el minibatch de m ejemplos de entrenamiento. El proceso se repite de manera habitual hasta que se completa el entrenamiento. Al momento de la prueba, el modelo se ejecuta sin abandono.

Podemos pensar en el abandono desde varias perspectivas:

- Al introducir ruido en el momento del entrenamiento, se obliga al modelo a ser robusto al ruido.
- Como antes se señaló, el abandono aproxima la creación de un gran conjunto de redes adelgazadas. Esta afirmación puede ser verificada para modelos lineales, y parece conservarse de manera experimental para modelos de aprendizaje profundo.
- Las unidades ocultas entrenadas con abandono deben aprender no sólo a ser unidades ocultas útiles; también deben aprender a ser compatibles con muchos otros posibles conjuntos de otras unidades ocultas que pueden o no estar incluidas en el modelo completo. Esto es similar a los procesos de selección que guían la evolución de los genes: cada gen no sólo debe ser efectivo en su propia función, debe funcionar bien con otros genes, cuya identidad en futuros organismos varía de manera considerable.
- El abandono aplicado a las capas posteriores en una red profunda obliga a que la decisión final se tome de manera robusta prestando atención a todas las características abstractas del ejemplo, en lugar de centrarse en una sola e ignorar las demás. Por ejemplo, un clasificador de imágenes de animales podría ser capaz de lograr un alto rendimiento en el conjunto de entrenamiento sólo mirando la nariz del animal, pero fallaría en un caso de prueba donde la nariz estuviera oscurecida o dañada. Con el abandono, habrá casos de entrenamiento donde la "unidad de nariz" interna se anule, haciendo que el proceso de aprendizaje encuentre características de identificación adicionales. Observe que tratar de lograr el mismo grado de robustez añadiendo ruido a los datos de entrada sería difícil: no hay una manera fácil de saber con antelación que la red va a centrarse en las narices, y no hay una manera fácil de eliminar de manera automática las narices de cada imagen.

En conjunto, el abandono obliga al modelo a aprender múltiples explicaciones robustas para cada entrada. Esto hace que el modelo se generalice bien, pero también que sea más difícil adaptarse al conjunto de entrenamiento, por lo general es necesario usar un modelo más grande y entrenarlo durante más iteraciones.

21.6 Redes neuronales recurrentes

Las redes neuronales recurrentes (RNNs) se distinguen de las redes de avance en que permiten ciclos en el grafo de cálculo. En todos los casos que consideraremos, cada ciclo tiene un retraso, de

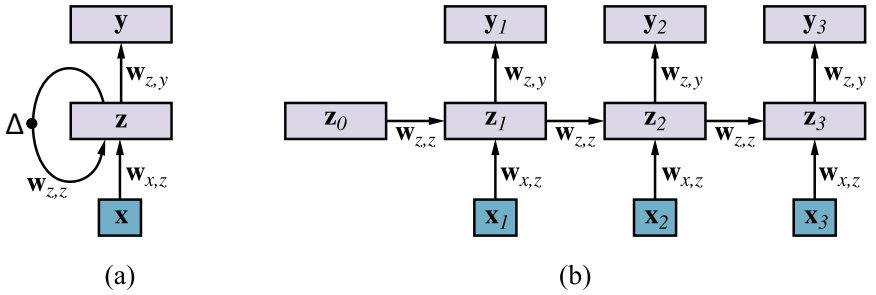


Figura 21.8 (a) Diagrama esquemático de una RNN básica donde la capa oculta **z** tiene conexiones recurrentes; el símbolo Δ indica un retraso. (b) La misma red desenrollada a lo largo de tres pasos de tiempo para crear una red de avance. Note que los pesos se comparten a través de todos los pasos de tiempo.

modo que las unidades permiten tomar como entrada un valor calculado a partir de su propia salida en un paso anterior en el cálculo. (Sin el retraso, un circuito cíclico permite llegar a un estado inconsistente). Esto permite que la RNN tenga un estado interno, o **memoria**: las entradas recibidas en pasos de tiempo anteriores afectan la respuesta de la RNN a la entrada actual. Memoria

Las RNNs también son utilizadas para realizar cálculos más generales, después de todo, las computadoras ordinarias son sólo circuitos booleanos con memoria, y para modelar sistemas neuronales reales, muchos de los cuales contienen conexiones cíclicas. Aquí nos enfocamos en el uso de RNNs para analizar datos secuenciales, donde asumimos que un nuevo vector de entrada $\mathbf{x}_t$ llega en cada paso de tiempo.

Como herramientas para analizar datos secuenciales, las RNNs permiten ser comparadas con los modelos ocultos de Markov, las redes Bayesianas dinámicas y los filtros de Kalman descritos en el Capítulo 14. (El lector puede encontrar útil referirse a ese capítulo antes de proceder). Al igual que esos modelos, las RNNs hacen una **suposición de Markov** (ver página 463): el estado oculto $\mathbf{z}_t$ de la red es suficiente para capturar la información de todas las entradas anteriores. Además, supongamos que describimos el proceso de actualización de RNN para el estado oculto por la ecuación $\mathbf{z}_t = f_{\mathbf{w}}(\mathbf{z}_{t-1}, \mathbf{x}_t)$ para alguna función parametrizada $f_{\mathbf{w}}$. Una vez entrenada, esta función representa un proceso **homogéneo en el tiempo** (página 463); una afirmación cuantificada de manera universal de que la dinámica representada por $f_{\mathbf{w}}$ se mantiene para todos los pasos de tiempo. Así, las RNN añaden potencia expresiva en comparación con las redes prealimentadas, al igual que lo hacen las redes convolucionales, y al igual que las redes Bayes dinámicas añaden potencia expresiva en comparación con las redes Bayes regulares. De hecho, si intentara usar una red prealimentada para analizar datos secuenciales, el tamaño fijo de la capa de entrada obligaría a la red a examinar sólo una ventana de datos de longitud finita, en cuyo caso la red no detectaría dependencias a larga distancia.

21.6.1 Entrenando una RNN básica

El modelo básico que consideraremos tiene una capa de entrada **x**, una capa oculta **z** con conexiones recurrentes, y una capa de salida **y**, como se muestra en la Figura 21.8(a). Asumimos que tanto **x** como **y** se observan en los datos de entrenamiento en cada paso de tiempo. Las ecuaciones que definen el modelo se refieren a los valores de las variables indexadas por el paso de tiempo t:

$$\mathbf{z}_t = f_{\mathbf{w}}(\mathbf{z}_{t-1}, \mathbf{x}_t) = \mathbf{g}_z(\mathbf{W}_{z,z}\mathbf{z}_{t-1} + \mathbf{W}_{x,z}\mathbf{x}_t) \equiv \mathbf{g}_z(\mathbf{in}_{z,t})$$
$$\hat{\mathbf{y}}_t = \mathbf{g}_y(\mathbf{W}_{z,y}\mathbf{z}_t) \equiv \mathbf{g}_y(\mathbf{in}_{y,t}), \qquad (21.13)$$

donde $\mathbf{g}_z$ y $\mathbf{g}_y$ denotan las funciones de activación para las capas ocultas y de salida, respectivamente. Como de costumbre, asumimos una entrada ficticia extra fija en +1 para cada unidad, así como pesos de sesgo asociados con esas entradas.

Dada una secuencia de vectores de entrada $\mathbf{x}_1,...,\mathbf{x}_T$ y salidas observadas $\mathbf{y}_1,...,\mathbf{y}_T$, podemos convertir este modelo en una red prealimentada "desenrollándola" durante T pasos, como se muestra en la Figura 21.8(b). Vea que las matrices de peso $\mathbf{W}_{x,z}$, $\mathbf{W}_{z,z}$, y $\mathbf{W}_{z,y}$ se comparten en todos los pasos de tiempo. En la red desenrollada, es fácil ver que podemos calcular gradientes para entrenar los pesos de la manera habitual; la única diferencia es que la compartición de pesos a través de las capas hace que el cálculo del gradiente sea un poco más complicado.

Para mantener las ecuaciones simples, mostraremos el cálculo del gradiente para una RNN con sólo una unidad de entrada, una unidad oculta y una unidad de salida. Para este caso, haciendo explícitos los pesos de sesgo, tenemos $z_t = g_z(w_{z,z}z_{t-1} + w_{x,z}x_t + w_{0,z})$ y $\hat{y}_t = g_y(w_{z,y}z_t + w_{0,y})$. Como en las Ecuaciones (21.4) y (21.5), asumiremos una pérdida de error cuadrado L, en este caso, sumada sobre los pasos de tiempo. Las derivaciones para los pesos de la capa de entrada y de salida $w_{x,z}$ y $w_{z,y}$ son idénticas a la Ecuación (21.4), por lo que las dejamos como un ejercicio. Para el peso de la capa oculta $w_{z,z}$, los primeros pasos también siguen el mismo patrón que la Ecuación (21.4):

$$\begin{aligned}
\frac{\partial L}{\partial w_{z,z}} &= \frac{\partial}{\partial w_{z,z}}\sum_{t=1}^{T}(y_t-\hat{y}_t)^2 = \sum_{t=1}^{T}-2(y_t-\hat{y}_t)\frac{\partial \hat{y}_t}{\partial w_{z,z}} \\
&= \sum_{t=1}^{T}-2(y_t-\hat{y}_t)\frac{\partial}{\partial w_{z,z}}g_y(in_{y,t}) = \sum_{t=1}^{T}-2(y_t-\hat{y}_t)\,g'_y(in_{y,t})\frac{\partial}{\partial w_{z,z}}in_{y,t} \\
&= \sum_{t=1}^{T}-2(y_t-\hat{y}_t)g'_y(in_{y,t})\frac{\partial}{\partial w_{z,z}}(w_{z,y}z_{t+}W_{0,y}) \\
&= \sum_{t=1}^{T}-2(y_t-\hat{y}_t)g'_y(in_{y,t})w_{z,y}\frac{\partial z_t}{\partial w_{z,z}} \qquad (21.14)
\end{aligned}$$

Ahora el gradiente para la unidad oculta z_t se puede obtener del paso de tiempo anterior de la siguiente manera:

$$\begin{aligned}
\frac{\partial z_t}{\partial w_{z,z}} &= \frac{\partial}{\partial w_{z,z}}g_z(in_{z,t}) = g'_z(in_{z,t})\frac{\partial}{\partial w_{z,z}}in_{z,t} = g'_z(in_{z,t})\frac{\partial}{\partial w_{z,z}}(w_{z,z}z_{t-1}+w_{x,z}x_t+w_{0,z}) \\
&= g'_z(in_{z,t})(z_{t-1}+w_{z,z}\frac{\partial z_{t-1}}{\partial w_{z,z}}), \qquad (21.15)
\end{aligned}$$

donde la última línea utiliza la regla para las derivadas de los productos: $\partial(uv)/\partial x = v\partial u/\partial x + u\partial v/\partial x$.

Mirando la Ecuación (21.15), notamos dos cosas. Primero, la expresión del gradiente es recursiva: la contribución al gradiente del paso de tiempo t se calcula usando la contribución del paso de tiempo $t-1$. Si ordenamos los cálculos de la manera correcta, el tiempo total de ejecución para calcular el gradiente será lineal en el tamaño de la red. Este algoritmo se llama **retropropagación a través del tiempo**, y en la mayoría de los casos es manejado de manera automática por sistemas de software de aprendizaje profundo. En segundo lugar, si iteramos el cálculo recursivo, vemos que los gradientes en T incluirán términos proporcionales a $w_{z,z}\Pi^{T}_{t=1}g'_z(in_{z,t})$. Para sigmoideas, tanhs, y ReLUs, $g' \leq 1$, por lo que nuestro RNN sufrirá del problema del gradiente desvaneciente (ver página 756) si $w_{z,z} < 1$. Por otro lado, si $w_{z,z} > 1$, podemos experimentar el problema del **gradiente explosivo**. (Para el caso general, estos resultados dependen del primer valor propio de la matriz de pesos $\mathbf{W}_{z,z}$). La siguiente sección describe un diseño de RNN más elaborado para mitigar este problema.

Retropropagación a través del tiempo

Gradiente explosivo

21.6.2 Memoria a largo plazo de corto plazo RNNs

Varias arquitecturas especializadas de RNN han sido diseñadas con el objetivo de permitir que la información se conserve durante muchos pasos de tiempo. Una de las más populares es la **memoria a largo plazo y corto plazo o LSTM**. El componente de memoria a largo plazo de un LSTM, llamado **celda de memoria** y denotado por **c**, es *copiado* de un paso de tiempo a otro. (En contraste, el RNN básico multiplica su memoria por una matriz de pesos en cada paso de tiempo, como se muestra en la Ecuación (21.13)). La nueva información entra en la memoria *añadiendo* actualizaciones; así, las expresiones del gradiente no se acumulan de manera multiplicativa con el tiempo. Los LSTMs también incluyen **unidades de compuerta**, que son vectores que controlan el flujo de información en el LSTM a través de la multiplicación elemento a elemento del vector de información correspondiente:

Memoria a largo plazo y corto plazo

Celda de memoria

Unidades de compuerta

- La **compuerta de olvido f** determina si cada elemento de la celda de memoria es recordado (copiado al siguiente paso de tiempo) u olvidado (reiniciado a cero). Compuerta de olvido
- La **compuerta de entrada i** determina si cada elemento de la celda de memoria se actualiza mediante la adición por nueva información del vector de entrada en el paso de tiempo actual. Compuerta de entrada
- La **compuerta de salida o** determina si cada elemento de la celda de memoria se transfiere a la compuerta de salida memoria a corto plazo **z**, que desempeña un papel similar al estado oculto en los RNN básicos. Compuerta de salida

Mientras que la palabra "compuerta" en el diseño de circuitos connota una función booleana, las compuertas en los LSTMs son suaves, por ejemplo, los elementos del vector de la celda de memoria serán parcialmente olvidados si los elementos correspondientes del vector de la compuerta de olvido son pequeños, pero no cero. Los valores para las unidades de compuerta siempre están en el rango [0,1] y se obtienen como las salidas de una función sigmoide aplicada a la entrada actual y al estado oculto anterior. En detalle, las ecuaciones de actualización para el LSTM son las siguientes:

$$\mathbf{f}_t = \sigma(\mathbf{W}_{x,f}\mathbf{x}_t + \mathbf{W}_{z,f}\mathbf{z}_{t-1})$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_{x,i}\mathbf{x}_t + \mathbf{W}_{z,i}\mathbf{z}_{t-1})$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_{x,o}\mathbf{x}_t + \mathbf{W}_{z,o}\mathbf{z}_{t-1})$$
$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{i}_t \odot \tanh(\mathbf{W}_{x,c}\mathbf{x}_t + \mathbf{W}_{z,c}\mathbf{z}_{t-1})$$
$$\mathbf{z}_t = \tanh(\mathbf{c}_t) \odot \mathbf{o}_t,$$

donde los subíndices en las diversas matrices de pesos **W** indican el origen y destino de los enlaces correspondientes. El símbolo $\odot$ denota multiplicación elemento a elemento.

Los LSTMs fueron una de las primeras formas utilizables de RNN. Han demostrado un excelente rendimiento en una amplia gama de tareas, incluido reconocimiento de voz y reconocimiento de escritura a mano. Su uso en el procesamiento del lenguaje natural se discute en el Capítulo 24.

21.7 Aprendizaje no supervisado y aprendizaje de transferencia

Los sistemas de aprendizaje profundo que hemos discutido hasta ahora se basan en el aprendizaje supervisado, que requiere que cada ejemplo de entrenamiento esté etiquetado con un valor para la función objetivo. Aunque dichos sistemas permiten alcanzar un alto nivel de precisión en el conjunto de prueba, como lo demuestran los resultados de la competencia ImageNet, por ejemplo, requieren muchos más datos etiquetados de los que un humano necesitaría para la misma tarea. Por ejemplo, un niño sólo necesita ver una imagen de una jirafa, en lugar de miles, para poder reconocer jirafas de manera confiable en una amplia gama de configuraciones y vistas. Note que, algo falta en nuestra historia de aprendizaje

profundo; de hecho, puede ser el caso de que nuestro enfoque actual al aprendizaje profundo supervisado hace que algunas tareas sean inalcanzables porque los requisitos para los datos etiquetados excederían lo que la raza humana (o el universo) es capaz de suministrar. Además, incluso en casos donde la tarea es factible, etiquetar grandes conjuntos de datos requiere escasa y costosa mano de obra humana.

Por estas razones, existe un intenso interés en diferentes paradigmas de aprendizaje que reducen la dependencia de los datos etiquetados. Como vimos en el Capítulo 19, estos paradigmas incluyen el **aprendizaje no supervisado**, el **aprendizaje por transferencia** y el **aprendizaje semisupervisado**. Los algoritmos de aprendizaje no supervisado aprenden únicamente a partir de entradas no etiquetadas $\mathbf{x}$, que con frecuencia están más disponibles que los ejemplos etiquetados. Los algoritmos de aprendizaje no supervisado suelen producir modelos generativos que, valga la redundancia, generan texto, imágenes, audio y video realistas, en lugar de predecir etiquetas para dichos datos. Los algoritmos de aprendizaje por transferencia requieren algunos ejemplos etiquetados, pero permiten mejorar su rendimiento mediante el estudio de ejemplos etiquetados para diferentes tareas, lo que hace posible aprovechar más fuentes de datos existentes. Los algoritmos de aprendizaje semisupervisado requieren algunos ejemplos etiquetados, pero mejoran su rendimiento estudiando también ejemplos no etiquetados. Esta sección cubre enfoques de aprendizaje profundo para el aprendizaje no supervisado y por transferencia; mientras que el aprendizaje semisupervisado también es un área activa de investigación en la comunidad de aprendizaje profundo, las técnicas desarrolladas hasta ahora no han demostrado ser efectivas en la práctica, por lo que no las cubrimos.

1.7.1 Aprendizaje no supervisado

Todos los algoritmos de aprendizaje supervisado tienen el mismo objetivo: dado un conjunto de entrenamiento de entradas $\mathbf{x}$ y salidas correspondientes $y = f(\mathbf{x})$, aprender una función h que se aproxime bien a f. Los algoritmos de aprendizaje no supervisado, por otro lado, toman un conjunto de entrenamiento de ejemplos no etiquetados $\mathbf{x}$. Aquí describimos dos cosas que tal algoritmo podría intentar hacer. Lo primero es aprender nuevas representaciones, por ejemplo, características nuevas de las imágenes que faciliten la identificación de los objetos en una imagen. Lo segundo es aprender un modelo generativo, con frecuencia en forma de una distribución de probabilidad a partir de la cual es posible generar nuevas muestras. (Los algoritmos para aprender redes de Bayes en el Capítulo 20 caen en esta categoría). Muchos algoritmos son capaces de aprender tanto la representación como el modelado generativo.

Supongamos que aprendemos un modelo conjunto $P_W(\mathbf{x},\mathbf{z})$, donde $\mathbf{z}$ es un conjunto de variables latentes no observadas que representan el contenido de los datos $\mathbf{x}$ de alguna manera. De acuerdo con el espíritu del capítulo, no predefinimos los significados de las variables $\mathbf{z}$; el modelo es libre de aprender a asociar $\mathbf{z}$ con $\mathbf{x}$ como elija. Por ejemplo, un modelo entrenado en imágenes de dígitos escritos a mano podría elegir usar una dirección en el espacio $\mathbf{z}$ para representar el grosor de los trazos de la pluma, otra para representar el color de la tinta, otra para representar el color de fondo, etc. Con imágenes de caras, el algoritmo de aprendizaje podría elegir una dirección para representar el género y otra para capturar la presencia o ausencia de gafas, como se ilustra en la Figura 21.9.

Un modelo de probabilidad aprendido $P_W(\mathbf{x},\mathbf{z})$ logra tanto el aprendizaje de la representación (ha construido vectores $\mathbf{z}$ significativos a partir de los vectores $\mathbf{x}$ brutos) como el modelado generativo: si integramos $\mathbf{z}$ de $P_W(\mathbf{x},\mathbf{z})$ obtenemos $P_W(\mathbf{x})$.

PCA probabilístico: Un modelo generativo simple

PPCA

Ha habido muchas propuestas para la forma que $P_W(\mathbf{x},\mathbf{z})$ podría tomar. Uno de los más simples es el modelo de **análisis de componentes principales probabilístico** (**PPCA**).[7] En un modelo

[7] El PCA estándar implica ajustar una gaussiana multivariante a los datos de entrada brutos y luego seleccionar los ejes más largos, los componentes principales, de esa distribución elipsoidal.

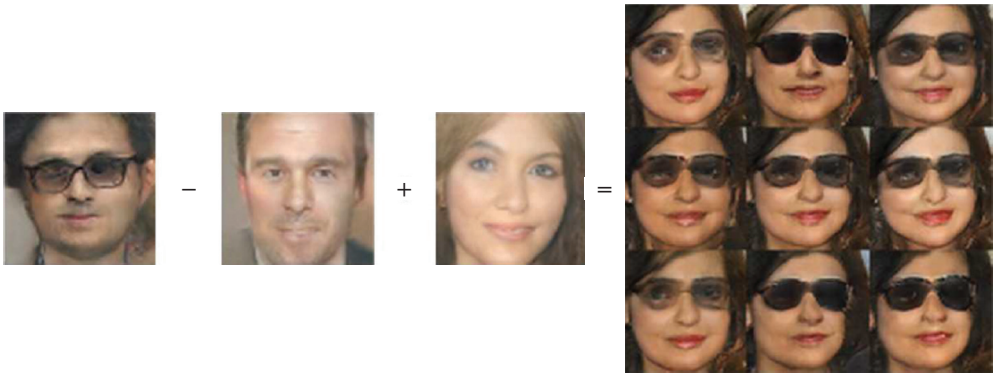


Figura 21.9 Una demostración de cómo un modelo generativo ha aprendido a usar diferentes direcciones en el espacio **z** para representar diferentes aspectos de las caras. Podemos realizar aritmética en el espacio **z**. Las imágenes aquí todas son generadas a partir del modelo aprendido y muestran qué sucede cuando decodificamos diferentes puntos en el espacio **z**. Comenzamos con las coordenadas para el concepto de "hombre con gafas", restamos las coordenadas para "hombre", agregamos las coordenadas para "mujer" y obtenemos las coordenadas para "mujer con gafas". Imágenes reproducidas con permiso de (Radford *et al*., 2015).

PPCA, **z** se elige a partir de un Gaussiano esférico de media cero, luego **x** se genera a partir de **z** aplicando una matriz de pesos **W** y agregando ruido Gaussiano esférico:

$$\begin{aligned} P(\mathbf{z}) &= \mathcal{N}(\mathbf{z};\mathbf{0},\mathbf{I}) \\ P_W(\mathbf{x}|\mathbf{z}) &= \mathcal{N}(\mathbf{x};\mathbf{W}\mathbf{z},\sigma^2\mathbf{I}). \end{aligned}$$

Los pesos **W** (y opcionalmente el parámetro de ruido σ^2) se aprenden maximizando la probabilidad de los datos, dada por

$$P_W(\mathbf{x}) = \int P_W(\mathbf{x},\mathbf{z})d\mathbf{z} = \mathcal{N}(\mathbf{x};\mathbf{0},\mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I}). \tag{21.16}$$

La maximización con respecto a **W** se hace mediante métodos de gradiente o mediante un eficiente algoritmo iterativo EM (ver Sección 20.3). Una vez que se ha aprendido **W**, se generan de manera directa nuevas muestras de datos a partir de $P_W(\mathbf{x})$ usando la Ecuación (21.16). Además, nuevas observaciones x, que tienen una probabilidad muy baja según la Ecuación (21.16), permiten ser marcadas como potenciales anomalías.

Con PPCA, asumimos que la dimensionalidad de **z** es mucho menor que la dimensionalidad de **x**, de modo que el modelo aprende a explicar los datos lo mejor posible en términos de un pequeño número de características; las cuales se extraen para su uso en clasificadores estándar calculando $\hat{\mathbf{z}}$, la expectativa de $P_W(\mathbf{z}|\mathbf{x})$.

Generar datos a partir de un modelo PCA probabilístico es sencillo: primero se toma una muestra de **z** a partir de su prior gaussiano fijo, luego se toma una muestra de **x** a partir de una gaussiana con media **Wz**. Como veremos en breve, muchos otros modelos generativos se parecen a este proceso, pero utilizan mapeos complicados definidos por modelos profundos en lugar de mapeos lineales del espacio **z** al espacio **x**.

Autoencoders

Autoencoder

Muchos algoritmos de aprendizaje profundo no supervisados se basan en la idea de un **autoencoder,** un modelo que contiene dos partes: un codificador que mapea de $\mathbf{x}$ a una representación $\hat{\mathbf{z}}$ y un decodificador que mapea de una representación $\hat{\mathbf{z}}$ a datos observados $\mathbf{x}$. En general, el codificador es sólo una función parametrizada f, mientras que el decodificador sólo es una función parametrizada g. El modelo se entrena de manera que $\mathbf{x} \approx g(f(\mathbf{x}))$, de modo que el proceso de codificación se invierte por el proceso de decodificación. Las funciones f y g pueden ser modelos lineales simples parametrizados por una sola matriz o permiten ser representados por una red neuronal profunda.

Un autoencoder muy simple es el autoencoder lineal, donde tanto f como g son lineales con una matriz de pesos $\mathbf{W}$ compartida:

$$\begin{aligned} \hat{\mathbf{z}} &= f(\mathbf{x}) = \mathbf{W}\mathbf{x} \\ \mathbf{x} &= g(\hat{\mathbf{z}}) = \mathbf{W}^\top \hat{\mathbf{z}}. \end{aligned}$$

Una forma de entrenar este modelo es minimizar el error cuadrado $\Sigma_j \|\boldsymbol{x}_j - g(f(\mathbf{x}_j))\|^2$ de modo que $\mathbf{x} \approx g(f(\mathbf{x}))$. La idea es entrenar $\mathbf{W}$ para que una $\hat{\mathbf{z}}$ de baja dimensión retenga tanta información como sea posible para reconstruir los datos de alta dimensión $\mathbf{x}$. Este autoencoder lineal resulta estar conectado con el análisis de componentes principales clásico (PCA). Cuando $\mathbf{z}$ es de dimensión m, la matriz $\mathbf{W}$ debería aprender a abarcar los m componentes principales de los datos, en otras palabras, el conjunto de m direcciones ortogonales en las que los datos tienen la mayor variación, o equivalente los m eigenvectores de la matriz de covarianza de los datos que tienen los eigenvalores más grandes, como en PCA.

El modelo PCA es un modelo generativo simple que corresponde a un autoencoder lineal simple. La correspondencia sugiere que existe una forma de capturar tipos más complejos de modelos generativos utilizando tipos más complejos de autoencoders. El **autoencoder variacional** (VAE) proporciona una forma de hacer esto.

Autoencoder variacional

Los métodos variacionales se introdujeron de manera breve en la página 458 como una forma de aproximar la distribución posterior en modelos de probabilidad complejos, donde sumar o integrar un gran número de variables ocultas es intratable. La idea es usar un **posterior variacional** $Q(\mathbf{z})$, extraído de una familia computacionalmente manejable de distribuciones, como una aproximación al posterior verdadero. Por ejemplo, podríamos elegir Q de la familia de distribuciones gaussianas con una matriz de covarianza diagonal. Dentro de la familia elegida de distribuciones manejables, Q se optimiza para estar lo más cerca posible de la distribución posterior verdadera $P(\mathbf{z}|\mathbf{x})$.

Posterior variacional

Para nuestros propósitos, la noción de "lo más cerca posible" está definida por la divergencia KL, que mencionamos en la página 758. Esto se da por

$$D_{KL}(Q(\mathbf{z})\|P(\mathbf{z}|\mathbf{x})) = \int Q(\mathbf{z}) \log \frac{Q(\mathbf{z})}{P(\mathbf{z}|\mathbf{x})} d\mathbf{z},$$

que es un promedio (con respecto a Q) de la relación logarítmica entre Q y P. Es fácil ver que $D_{KL}(Q(\mathbf{z})\|P(\mathbf{z}|\mathbf{x})) \geq 0$, con igualdad cuando Q y P coinciden. Luego podemos definir el **límite inferior variacional** $\mathcal{L}$ (a veces llamado el **límite inferior de la evidencia**, o **ELBO**) en el log verosimilitud de los datos:

Límite inferior variacional

ELBO

$$\mathcal{L}(\mathbf{x},Q) = \log P(\mathbf{x}) - D_{KL}(Q(\mathbf{z})\|P(\mathbf{z}|\mathbf{x})). \tag{21.17}$$

Podemos ver que $\mathcal{L}$ es un límite inferior para $\log P$ porque la divergencia KL es no negativa. El aprendizaje variacional maximiza con respecto a los parámetros $\mathbf{w}$ en lugar de maximizar $\log P(\mathbf{x})$, con la esperanza de que la solución encontrada, $\mathbf{w}^*$, esté cerca de maximizar $\log P(\mathbf{x})$ también.

Tal como está escrito, $\mathcal{L}$ no parece ser más fácil de maximizar que $\log P$. Afortunadamente, podemos reescribir la Ecuación (21.17) para revelar una mejor tratabilidad computacional:

$$\begin{aligned}\mathcal{L} &= \log P(\mathbf{x}) - \int Q(\mathbf{z}) \log \frac{Q(\mathbf{z})}{P(\mathbf{z}|\mathbf{x})} d\mathbf{z} \\ &= -\int Q(\mathbf{z}) \log Q(\mathbf{z}) d\mathbf{z} + \int Q(\mathbf{z}) \log P(\mathbf{x}) P(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\ &= H(Q) + \mathbf{E}_{\mathbf{z}\sim Q} \log P(\mathbf{z},\mathbf{x})\end{aligned}$$

donde $H(Q)$ es la entropía de la distribución Q. Para algunas familias variacionales Q (como las distribuciones Gaussianas), $H(Q)$ es evaluada de manera analítica. Además, la función objetivo, $\mathbf{E}_{\mathbf{z}\sim Q} \log P(\mathbf{z},\mathbf{x})$, admite una estimación imparcial eficiente a través de muestras de $\mathbf{z}$ de Q. Para cada muestra, $P(\mathbf{z},\mathbf{x})$ es evaluada de manera eficiente, por ejemplo, si P es una red Bayes, $P(\mathbf{z},\mathbf{x})$ es sólo un producto de probabilidades condicionales porque $\mathbf{z}$ y $\mathbf{x}$ comprenden todas las variables.

Los autoencoders variacionales proporcionan un medio para realizar el aprendizaje variacional en el entorno de aprendizaje profundo. El aprendizaje variacional implica maximizar $\mathcal{L}$ con respecto a los parámetros de P y Q. Para un autoencoder variacional, el decodificador $g(\mathbf{z})$ se interpreta como definido $\log P(\mathbf{x}|\mathbf{z})$. Por ejemplo, la salida del decodificador podría definir la media de una condicional Gaussiana. De manera similar, la salida del codificador $f(\mathbf{x})$ se interpreta como definiendo los parámetros de Q, por ejemplo, Q podría ser una Gaussiana con media $f(\mathbf{x})$. Entrenar el autoencoder variacional entonces consiste en maximizar $\mathcal{L}$ con respecto a los parámetros de tanto el codificador f como el decodificador g, que pueden ser redes profundas y complicadas.

Modelos autorregresivo profundos

Modelo autorregresivo

Un **modelo autorregresivo** (o modelo AR) es uno en el que cada elemento x_i del vector de datos $\mathbf{x}$ se predice en función de otros elementos del vector. Tal modelo no tiene variables latentes. Si $\mathbf{x}$ es de tamaño fijo, un modelo AR permite considerarse como una red Bayes por completo observable y, en ocasiones, por completo conectada. Esto significa que calcular la verosimilitud de un vector de datos dado según un modelo AR es trivial; lo mismo ocurre para predecir el valor de una única variable faltante, dadas todas las demás, y para muestrear un vector de datos del modelo.

La aplicación más común de los modelos autorregresivos es en el análisis de datos de series de tiempo, donde un modelo AR de orden k predice x_t dado $x_{t-k},\ldots, x_{t-1}$. En la terminología del Capítulo 14, un modelo AR es un modelo de Markov no oculto. En la terminología del Capítulo 23, un modelo n-grama de secuencias de letras o palabras es un modelo AR de orden $n-1$.

En los modelos AR clásicos, donde las variables son de valor real, la distribución condicional $P(x_t|x_{t-k},\ldots,x_{t-1})$ es un modelo lineal-Gaussiano con varianza fija cuya media es una combinación lineal ponderada de $x_{t-k},\ldots,x_{t-1}$, en otras palabras, un modelo de regresión lineal estándar. La solución de máxima verosimilitud está dada por las **ecuaciones de Yule-Walker**, relacionadas con las **ecuaciones normales** en la página 680.

Ecuaciones de Yule-Walker

Un **modelo autorregresivo profundo** es uno en el que el modelo lineal-Gaussiano es reemplazado por una red profunda arbitraria con una capa de salida adecuada dependiendo de si x_t es discreto o continuo. Las aplicaciones recientes de este enfoque autorregresivo incluyen el modelo WaveNet de DeepMind para la generación de voz (Van den Oord *et al.*, 2016a). WaveNet se entrena en señales acústicas crudas, muestreadas 16 000 veces por segundo, e implementa un modelo AR no lineal de orden 4 800 con una estructura convolucional multicapa. En las pruebas resulta ser más realista que los sistemas de generación de voz anteriores de última generación.

Modelo autorregresivo profundo

Redes generativas adversariales

Red generativa adversarial (GAN)
Generador

Una **red generativa adversarial** (**GAN**) es en realidad un par de redes que se combinan para formar un sistema generativo. Una de las redes, el **generador**, mapea valores de $\mathbf{z}$ a $\mathbf{x}$ para producir muestras de la distribución $P_w(\mathbf{x})$. Un esquema típico toma muestras de $\mathbf{z}$ de una Gaussiana unitaria de dimensión moderada y luego la pasa a través de una red profunda h_w para obtener $\mathbf{x}$. La otra red, el **discriminador**, es un clasificador entrenado para clasificar las entradas $\mathbf{x}$ como reales (extraídas del conjunto de entrenamiento) o falsas (creadas por el generador). Las GANs son un tipo de **modelo implícito** en el sentido de que es posible generar muestras, pero sus probabilidades no están disponibles; en una red Bayes, por otro lado, la probabilidad de una muestra es el producto de las probabilidades condicionales a lo largo del camino de generación de la muestra.

Discriminador

Modelo implícito

El generador está relacionado con el decodificador del marco de trabajo del autoencoder variacional. El desafío en el modelado implícito es diseñar una función de pérdida que haga posible entrenar el modelo utilizando muestras de la distribución, en lugar de maximizar la probabilidad asignada a los ejemplos de entrenamiento del conjunto de datos.

Tanto el generador como el discriminador se entrenan de manera simultánea, con el generador aprendiendo a engañar al discriminador y el discriminador aprendiendo a separar con precisión los datos reales de los falsos. La competencia entre el generador y el discriminador se describe en el lenguaje de la teoría de juegos (ver Capítulo 18). La idea es que, en el estado de equilibrio del juego, el generador debería reproducir la distribución de entrenamiento de manera perfecta, de tal modo que el discriminador no pueda desempeñarse mejor que adivinando al azar. Las GANs han funcionado particularmente bien para tareas de generación de imágenes. Por ejemplo, las GANs permiten crear imágenes fotorrealistas de alta resolución de personas que nunca han existido (Karras *et al*., 2017).

Traducción no supervisada

Las tareas de traducción, en sentido amplio, consisten en transformar una entrada $\mathbf{x}$ que tiene una estructura rica en una salida y que también tiene una estructura rica. En este contexto, "estructura rica" significa que los datos son multidimensionales y tienen dependencias estadísticas interesantes entre las diversas dimensiones. Las imágenes y las oraciones en lenguaje natural tienen una estructura rica, pero un sólo número, como un ID de clase, no lo tiene. Transformar una oración del inglés al francés o convertir una foto de una escena nocturna en una foto equivalente tomada durante el día son ambos ejemplos de tareas de traducción.

La traducción supervisada consiste en recopilar muchos pares $(\mathbf{x},\mathbf{y})$ y entrenar el modelo para mapear cada $\mathbf{x}$ al $\mathbf{y}$ correspondiente. Por ejemplo, los sistemas de traducción automática con frecuencia se entrenan en pares de oraciones que han sido traducidas por traductores humanos profesionales. Para otros tipos de traducción, es posible que no haya datos de entrenamiento supervisados disponibles. Por ejemplo, considera una foto de una escena nocturna que contiene muchos coches y peatones en movimiento. Presumiblemente no es factible encontrar todos los coches y peatones y devolverlos a sus posiciones originales en la foto nocturna para volver a tomar la misma foto durante el día. Para superar esta dificultad, es posible utilizar técnicas de **traducción no supervisadas** capaces de entrenar en muchos ejemplos de $\mathbf{x}$ y muchos ejemplos separados de $\mathbf{y}$ pero sin pares correspondientes $(\mathbf{x},\mathbf{y})$.

Traducción no supervisada

Estos enfoques se basan en GANs; por ejemplo, es posible entrenar un generador GAN para producir un ejemplo realista de $\mathbf{y}$ cuando se condiciona en $\mathbf{x}$, y otro generador GAN para realizar el mapeo inverso. El marco de entrenamiento GAN hace posible entrenar a un generador para generar cualquiera de las muchas muestras posibles que el discriminador acepta como un ejemplo *realista* de $\mathbf{y}$ dado $\mathbf{x}$, sin necesidad de un $\mathbf{y}$ emparejado específico similar a como se necesita en

el aprendizaje supervisado. Se da más detalle sobre la traducción no supervisada para imágenes en la Sección 25.7.5.

21.7.2 Aprendizaje de transferencia y aprendizaje multitarea

Aprendizaje de transferencia

En el **aprendizaje de transferencia**, la experiencia con una tarea de aprendizaje ayuda a un agente a aprender mejor en otra tarea. Por ejemplo, una persona que ya ha aprendido a jugar tenis por lo regular encontrará más fácil aprender deportes relacionados como el ráquetbol y el squash; un piloto que ha aprendido a volar un tipo de avión de pasajeros comercial aprenderá de manera rápida a volar un avión de otro tipo; un estudiante que ya ha aprendido álgebra encontrará fácil aprender cálculo.

Todavía no conocemos los mecanismos del aprendizaje de transferencia humano. Para las redes neuronales, el aprendizaje consiste en ajustar los pesos, por lo que el enfoque más plausible para el aprendizaje de transferencia es copiar los pesos aprendidos para la tarea A a una red que será entrenada para la tarea B. Los pesos se actualizan entonces por descenso de gradiente de la manera usual usando datos para la tarea B. Puede ser una buena idea usar una tasa de aprendizaje más pequeña en la tarea B, dependiendo de cuán similares sean las tareas y cuántos datos se usaron en la tarea A.

Observe que este enfoque requiere experiencia humana para seleccionar las tareas: por ejemplo, los pesos aprendidos durante el entrenamiento de álgebra pueden no ser muy útiles en una red destinada al ráquetbol. Además, la noción de copiar pesos requiere un mapeo simple entre los espacios de entrada para las dos tareas y arquitecturas de red idénticas.

Una razón para la popularidad del aprendizaje de transferencia es la disponibilidad de modelos preentrenados de alta calidad. Por ejemplo, podría descargar un modelo preentrenado de reconocimiento de objetos visuales como el modelo ResNet-50 entrenado en el conjunto de datos COCO, ahorrándose semanas de trabajo. A partir de ahí es posible modificar los parámetros del modelo proporcionando imágenes adicionales y etiquetas de objetos para su tarea específica.

Suponga que quiere clasificar tipos de monociclos. Sólo tiene unas pocas cientos de fotos de diferentes monociclos, pero el conjunto de datos COCO tiene más de 3 000 imágenes en cada una de las categorías de bicicletas, motocicletas y patinetas. Esto significa que un modelo preentrenado en COCO ya tiene experiencia con ruedas y carreteras, y otras características relevantes que serán útiles para interpretar las imágenes de monociclos.

En ocasiones querrá congelar las primeras capas del modelo preentrenado, estas capas sirven como detectores de características que serán útiles para su nuevo modelo. A su nuevo conjunto de datos se le permitirá modificar los parámetros de los niveles superiores; estas son las capas que identifican características específicas del problema y hacen la clasificación. Sin embargo, a veces la diferencia entre sensores significa que incluso las capas de nivel más bajo necesitan ser reentrenadas.

Como otro ejemplo, para aquellos que están construyendo un sistema de lenguaje natural, ahora es común comenzar con un modelo preentrenado como el modelo ROBERTA (ver Sección 24.6), que ya "sabe" mucho sobre el vocabulario y la sintaxis del lenguaje cotidiano. El siguiente paso es ajustar el modelo de dos maneras. Primero, dándole ejemplos del vocabulario especializado utilizado en el dominio deseado; quizás un dominio médico (donde aprenderá sobre "infarto al miocardio") o quizás un dominio financiero (donde aprenderá sobre "responsabilidad fiduciaria"). Segundo, entrenando el modelo en la tarea que debe realizar. Si debe responder preguntas, entrénelo en pares de preguntas/respuestas.

Un tipo importante de aprendizaje de transferencia implica la transferencia entre simulaciones y el mundo real. Por ejemplo, el controlador de un automóvil autónomo permite ser entrenado en miles de millones de millas de conducción simulada, lo cual sería imposible en el

mundo real. Luego, cuando el controlador se transfiere al vehículo real, se adapta de manera rápida al nuevo entorno.

Aprendizaje multitarea

El **aprendizaje multitarea** es una forma de aprendizaje de transferencia en la que entrenamos de manera simultánea un modelo en múltiples objetivos. Por ejemplo, en lugar de entrenar un sistema de lenguaje natural en la etiquetación de partes del discurso y luego transferir los pesos aprendidos a una nueva tarea, como la clasificación de documentos, entrenamos un sistema de manera simultánea en etiquetación de partes del discurso, clasificación de documentos, detección de idiomas, predicción de palabras, modelado de la dificultad de las frases, detección de plagio, implicación de las frases y respuesta a preguntas. La idea es que para resolver cualquiera de estas tareas, un modelo podría ser capaz de aprovechar las características superficiales de los datos. Pero para resolver las ocho a la vez con una capa de representación común, es más probable que el modelo cree una representación común que refleje el uso y contenido real del lenguaje natural.

21.8 Aplicaciones

El aprendizaje profundo se ha aplicado con éxito a muchas áreas problemáticas importantes en la IA. Para explicaciones más detalladas, remitimos al lector a los capítulos relevantes: Capítulo 22 para el uso del aprendizaje profundo en sistemas de aprendizaje por refuerzo, Capítulo 24 para el procesamiento del lenguaje natural, Capítulo 25 (en particular la Sección 25.4) para la visión por computadora, y Capítulo 26 para la robótica.

21.8.1 Visión

Comenzamos con la visión por computadora, el área de aplicación que ha tenido el mayor impacto en el aprendizaje profundo, y viceversa. Aunque las redes convolucionales profundas se habían utilizado desde la década de 1990 para tareas como el reconocimiento de escritura a mano, y las redes neuronales habían comenzado a superar a los modelos de probabilidad generativos para el reconocimiento de voz alrededor de 2010, fue el éxito del sistema de aprendizaje profundo AlexNet en la competencia ImageNet 2012 lo que impulsó al aprendizaje profundo al centro de atención.

La competencia ImageNet era una tarea de aprendizaje supervisado con 1 200 000 imágenes en 1 000 categorías diferentes, y los sistemas se evaluaban en la puntuación "top-5", cuántas veces la categoría correcta aparece en las cinco primeras predicciones. AlexNet logró una tasa de error de 15.3 %, mientras que el siguiente mejor sistema tenía una tasa de error de más de 25 %. AlexNet tenía cinco capas convolucionales intercaladas con capas de max-pooling, seguidas de tres capas por completo conectadas. Utilizaba funciones de activación ReLU y aprovechaba las GPUs para acelerar el proceso de entrenamiento de 60 millones de pesos.

Desde 2012, con mejoras en el diseño de la red, los métodos de entrenamiento y los recursos informáticos, la tasa de error del top-5 se ha reducido a menos de 2 %, muy por debajo de la tasa de error de un humano entrenado (alrededor de 5 %). Las CNN se han aplicado a una amplia gama de tareas de visión, desde coches autónomos hasta la clasificación de pepinos.[8] Conducir, que se cubre en la Sección 25.7.6 y en múltiples secciones del Capítulo 26, es una de las tareas de visión más exigentes: no sólo el algoritmo debe detectar, localizar, rastrear y reconocer palomas, bolsas de papel y peatones, también tiene que hacerlo en tiempo real con una precisión casi perfecta.

[8] El famoso cuento del agricultor japonés de pepinos que construyó su propio robot clasificador de pepinos usando TensorFlow es, en gran parte, mítico. El algoritmo fue desarrollado por el hijo del agricultor, que trabajó antes como ingeniero de software en Toyota, y su baja precisión —cerca de 70%— significaba que los pepinos aún tenían que ser clasificados a mano (Zeeberg, 2017).

21.8.2 Procesamiento de lenguaje natural

El aprendizaje profundo también ha tenido un gran impacto en las aplicaciones de procesamiento de lenguaje natural (NLP) como la traducción automática y el reconocimiento de voz. Algunas ventajas del aprendizaje profundo para estas aplicaciones incluyen la posibilidad de aprendizaje de extremo a extremo, la generación automática de representaciones internas para los significados de las palabras, y la intercambiabilidad de los codificadores y decodificadores aprendidos.

El aprendizaje de extremo a extremo se refiere a la construcción de sistemas enteros como una única función aprendida *f*. Por ejemplo, una *f* para la traducción automática podría tomar como entrada una frase en inglés S_E y producir una frase japonesa equivalente $S_J = f(S_E)$. Tal *f* permite ser aprendida de los datos de entrenamiento en forma de pares de frases traducidas por humanos (o incluso pares de textos, donde la alineación de las frases o frases correspondientes es parte del problema a resolver). Un enfoque de flujo de trabajo más clásico podría primero analizar S_E, luego extraer su significado, luego reexpresar el significado en japonés como S_J, luego posteditar S_J usando un modelo de lenguaje para japonés. Este enfoque de flujo de trabajo tiene dos grandes desventajas: primero, los errores se acumulan en cada etapa; segundo, los humanos tienen que determinar qué constituye un "árbol de análisis" y una "representación de significado", pero no hay una verdad de base accesible para estas nociones; por ello, nuestras ideas teóricas sobre ellas son, casi con seguridad, incompletas.

En nuestra etapa actual de comprensión, entonces, el enfoque de flujo de trabajo clásico — que, al menos de manera ingenua, parece corresponder a cómo trabaja un traductor humano— es superado por el método de extremo a extremo (E2E) hecho posible por el aprendizaje profundo. Por ejemplo, Wu e*t al.* (2016b) mostraron que la traducción de extremo a extremo usando aprendizaje profundo redujo los errores de traducción en 60 % en relación con un sistema basado en el flujo de trabajo anterior. A partir de 2020, los sistemas de traducción automática se acercan al rendimiento humano para pares de idiomas como el francés y el inglés, para los cuales están disponibles conjuntos de datos emparejados muy amplios, y son utilizables para otros pares de idiomas que cubren la mayoría de la población de la Tierra. Incluso hay evidencia de que las redes entrenadas en varios idiomas de hecho aprenden una representación de significado interna: por ejemplo, después de aprender a traducir del portugués al inglés y del inglés al español, es posible traducir de manera directa del portugués al español sin ningún par de frases portugués/español en el conjunto de entrenamiento.

Uno de los hallazgos más significativos que ha surgido de la aplicación del aprendizaje profundo a las tareas de lenguaje es que se obtiene mucho provecho de la representación de palabras individuales como vectores en un espacio de alta dimensión, los llamados **embeddings de palabras** (ver Sección 24.1). Los vectores suelen extraerse de los pesos de la primera capa oculta de una red entrenada en grandes cantidades de texto, y capturan las estadísticas de los contextos léxicos en los que se utilizan las palabras. Debido a que las palabras con significados similares se usan en contextos similares, terminan cerca unas de otras en el espacio vectorial. Esto permite que la red generalice de manera efectiva a través de categorías de palabras, sin la necesidad de que los humanos predefinan esas categorías. Por ejemplo, una frase que comienza con "John compró una sandía y dos libras de ..." es probable que continúe con "manzanas" o "plátanos" pero no con "torio" o "geografía". Tal predicción es mucho más fácil de hacer si "manzanas" y "plátanos" tienen representaciones similares en la capa interna.

21.8.3 Aprendizaje por refuerzo

En el aprendizaje por refuerzo (RL), un agente de toma de decisiones aprende de una secuencia de señales de recompensa que proporcionan alguna indicación de la calidad de su

comportamiento. El objetivo es optimizar la suma de las recompensas futuras. Esto se hace de varias maneras: en la terminología del Capítulo 17, el agente aprende una función de valor, una función Q, una política, etc. Desde el punto de vista del aprendizaje profundo, todas estas son funciones que permiten ser representadas por grafo de cálculo. Por ejemplo, una función de valor en Go toma una posición de tablero como entrada y devuelve una estimación de cuán ventajosa es la posición para el agente. Si bien los métodos de entrenamiento en RL difieren de los del aprendizaje supervisado, la capacidad de los grafos de cálculo de múltiples capas para representar funciones complejas en grandes espacios de entrada ha demostrado ser muy útil. El campo de investigación resultante se llama **aprendizaje profundo por refuerzo**.

Aprendizaje profundo por refuerzo

En la década de 1950, Arthur Samuel experimentó con representaciones de múltiples capas de funciones de valor en su trabajo sobre el aprendizaje por refuerzo para el juego de damas, pero encontró que en la práctica un aproximador de función lineal funcionaba mejor. (Esto puede haber sido una consecuencia de trabajar con una computadora casi 100 mil millones de veces menos potente que una unidad de procesamiento de tensor moderna). La primera demostración exitosa de RL profundo (siglas de **r**einforcement **l**earning) fue el agente de Atari de DeepMind, DQN (Mnih *et al.,* 2013). Se entrenaron distintas copias de este agente para jugar cada uno de varios videojuegos de Atari diferentes, y demostraron habilidades como disparar a naves espaciales alienígenas, rebotar bolas con paletas y conducir coches de carreras simulados. En cada caso, el agente aprendió una función Q a partir de datos de imagen en bruto con la señal de recompensa siendo la puntuación del juego. El trabajo posterior ha producido sistemas de RL profundo que juegan a un nivel superhumano en la mayoría de los 57 juegos de Atari diferentes. El sistema AlphaGo de DeepMind también utilizó RL profundo para derrotar a los mejores jugadores humanos en el juego de Go (ver Capítulo 5).

A pesar de sus impresionantes éxitos, el RL profundo todavía enfrenta obstáculos significativos: en la mayoría de los casos es difícil obtener un buen rendimiento, y el sistema entrenado permite comportarse de manera muy impredecible si el entorno difiere apenas un poco de los datos de entrenamiento (Irpan, 2018). En comparación con otras aplicaciones de aprendizaje profundo, el RL profundo rara vez se aplica en entornos comerciales. Sin embargo, es un área de investigación muy activa.

Resumen

Este capítulo describió métodos para aprender funciones representadas por grafos computacionales profundos. Los puntos principales fueron:

- Las **redes neuronales** representan funciones no lineales complejas con una red de unidades de umbral lineal-parametrizadas.
- El algoritmo de **retropropagación** implementa un descenso de gradiente en el espacio de parámetros para minimizar la función de pérdida.
- El aprendizaje profundo funciona bien para el reconocimiento de objetos visuales, el reconocimiento de voz, el procesamiento de lenguaje natural y el aprendizaje por refuerzo en entornos complejos.
- Las redes convolucionales son adecuadas para el procesamiento de imágenes y otras tareas donde los datos tienen una topología de cuadrícula.
- Las redes recurrentes son efectivas para tareas de procesamiento de secuencias, incluido el modelado de lenguaje y la traducción automática.

Notas bibliográficas e históricas

La literatura sobre redes neuronales es vasta. Cowan y Sharp (1988b, 1988a) revisan la historia temprana, comenzando con el trabajo de McCulloch y Pitts (1943). (Como se mencionó en el Capítulo 1, John McCarthy ha señalado el trabajo de Nicolas Rashevsky (1936, 1938) como el modelo matemático más temprano del aprendizaje neural). Norbert Wiener, pionero de la cibernética y la teoría del control (Wiener, 1948), trabajó con McCulloch y Pitts e influyó en muchos jóvenes investigadores, incluido Marvin Minsky, quien puede haber sido el primero en desarrollar una red neuronal funcional en hardware, en 1951 (ver Minsky y Papert, 1988, pp. ix–x). Alan Turing (1948) escribió un informe de investigación titulado *Maquinaria inteligente* que comienza con la frase "Propongo investigar la pregunta de si es posible que la maquinaria muestre comportamiento inteligente" y continúa describiendo una arquitectura de red neuronal recurrente que llamó "máquinas desorganizadas de tipo B" y un enfoque para entrenarlas. Desafortunadamente, el informe no se publicó hasta 1969, y fue prácticamente ignorado hasta hace poco.

El perceptrón, una red neuronal de una capa con una función de activación de umbral duro, fue popularizado por Frank Rosenblatt (1957). Después de una demostración en julio de 1958, el *New York Times* lo describió como "el embrión de una computadora electrónica que [la Marina] espera que pueda caminar, hablar, ver, escribir, reproducirse y ser consciente de su existencia". Rosenblatt (1960) demostró después el teorema de convergencia del perceptrón, aunque había sido anticipado por trabajos puramente matemáticos fuera del contexto de las redes neuronales (Agmon, 1954; Motzkin y Schoenberg, 1954). También se realizó un trabajo temprano en redes multicapa, incluyendo **perceptrones de Gamba** (Gamba *et al*., 1961) y **madalinas** (Widrow, 1962). *Learning Machines* (Nilsson, 1965) cubre gran parte de este trabajo temprano y más. La posterior desaparición de los primeros esfuerzos de investigación del perceptrón fue acelerada (o, los autores más tarde afirmaron, explicada) por el libro *Perceptrons* (Minsky y Papert, 1969), que lamentaba la falta de rigor matemático en el campo. El libro señaló que los perceptrones de una sola capa sólo podían representar conceptos linealmente separables y señaló la falta de algoritmos de aprendizaje efectivos para redes multicapa. Estas limitaciones ya eran bien conocidas (Hawkins, 1961) y habían sido reconocidas por el propio Rosenblatt (Rosenblatt, 1962).

Los trabajos recopilados por Hinton y Anderson (1981), basados en una conferencia en San Diego en 1979, permiten considerarse como el marcador de un renacimiento del conexionismo. La antología "PDP" (Procesamiento Paralelo Distribuido) de dos volúmenes (Rumelhart y McClelland, 1986) ayudó a difundir el evangelio, por así decirlo, en especial en las comunidades de psicología y ciencias cognitivas. El desarrollo más importante de este periodo fue el algoritmo de retropropagación para entrenar redes multicapa.

El algoritmo de retropropagación fue descubierto de manera independiente múltiples veces en diferentes contextos (Kelley, 1960; Bryson, 1962; Dreyfus, 1962; Bryson y Ho, 1969; Werbos, 1974; Parker, 1985) y Stuart Dreyfus (1990) lo llama el "procedimiento de gradiente de Kelley–Bryson". Aunque Werbos lo había aplicado a las redes neuronales, esta idea no se hizo ampliamente conocida hasta que apareció un artículo de David Rumelhart, Geoff Hinton y Ron Williams (1986) en *Nature* donde daba una presentación no matemática del algoritmo. La respetabilidad matemática fue mejorada por trabajos que mostraban que las redes retroalimentadas multicapa son (sujetas a condiciones técnicas) aproximadores universales de funciones (Cybenko, 1988, 1989). A finales de la década de 1980 y principios de la de 1990 se vio un enorme crecimiento en la investigación de redes neuronales: el número de trabajos se multiplicó por un factor de 200 entre 1980–84 y 1990–94.

A finales de la década de 1990 y principios del siglo XXI, el interés en las redes neuronales disminuyó a medida que otras técnicas –como las redes de Bayes, los métodos de conjunto y

las máquinas de kernel– cobraron protagonismo. El interés en los modelos profundos surgió cuando la investigación de Geoff Hinton sobre las redes bayesianas profundas —modelos generativos con variables de categoría en la raíz y variables de evidencia en las hojas— comenzó a dar frutos al superar a las máquinas de kernel en pequeños conjuntos de datos de referencia (Hinton *et al*., 2006). El interés en el aprendizaje profundo explotó cuando Krizhevsky *et al*. (2013) utilizaron redes convolucionales profundas para ganar la competencia ImageNet (Russakovsky *et al*., 2015).

Los comentaristas con frecuencia citan la disponibilidad de "big data" y el poder de procesamiento de las GPUs como los principales factores contribuyentes en la aparición del aprendizaje profundo. Las mejoras arquitectónicas también fueron importantes, incluidos la adopción de la función de activación ReLU en lugar de la logística sigmoide (Jarrett *et al.,* 2009; Nair y Hinton, 2010; Glorot *et al.,* 2011) y más tarde el desarrollo de redes residuales (He *et al*., 2016).

En el lado algorítmico, el uso del descenso de gradiente estocástico (SGD) con pequeños lotes fue esencial para permitir que las redes neuronales se escalen a grandes conjuntos de datos (Bottou y Bousquet, 2008). La normalización por lotes (Ioffe y Szegedy, 2015) también ayudó a hacer el proceso de entrenamiento más rápido y confiable y ha generado varias técnicas de normalización adicionales (Ba *et al*., 2016; Wu y He, 2018; Miyato *et al.,* 2018). Varios trabajos han estudiado el comportamiento empírico del SGD en grandes redes y grandes conjuntos de datos (Dauphin *et al*., 2015; Choromanska *et al*., 2014; Goodfellow *et al*., 2015b). En el lado teórico, se ha avanzado en la explicación de la observación de que el SGD aplicado a redes sobreparametrizadas con frecuencia alcanza un mínimo global con un error de entrenamiento de cero, aunque hasta ahora los teoremas a este efecto asumen una red con capas mucho más anchas de lo que ocurriría en la práctica (Allen-Zhu *et al*., 2018; Du *et al*., 2018). Tales redes tienen más que suficiente capacidad para funcionar como tablas de búsqueda para los datos de entrenamiento.

La última pieza del rompecabezas, al menos para las aplicaciones de visión, fue el uso de redes convolucionales. Estas tuvieron sus orígenes en las descripciones del sistema visual de los mamíferos por los neurofisiólogos David Hubel y Torsten Wiesel (Hubel y Wiesel, 1959, 1962, 1968). Describieron "células simples" en el sistema visual de un gato que se asemejan a detectores de bordes, así como "células complejas" que son invariantes a algunas transformaciones como pequeñas traslaciones espaciales. En las modernas redes convolucionales, la salida de una convolución es análoga a una célula simple mientras que la salida de una capa de agrupación es análoga a una célula compleja.

El trabajo de Hubel y Wiesel inspiró muchos de los primeros modelos conexionistas de visión (Marr y Poggio, 1976). El neocognitrón (Fukushima, 1980; Fukushima y Miyake, 1982), diseñado como un modelo de la corteza visual, era en esencia una red convolucional en términos de arquitectura del modelo, aunque un algoritmo de entrenamiento efectivo para dichas redes tuvo que esperar hasta que Yann LeCun y sus colaboradores mostraron cómo aplicar la retropropagación (LeCun *et al*., 1995). Uno de los primeros éxitos comerciales de las redes neuronales fue el reconocimiento de dígitos escritos a mano usando redes convolucionales (LeCun *et al*., 1995).

Las redes neuronales recurrentes (RNNs) se propusieron como modelos de función cerebral en la década de 1970, pero no se asociaron algoritmos de aprendizaje efectivos con estas propuestas. El método de retropropagación a través del tiempo aparece en la tesis doctoral de Paul Werbos (1974), y su posterior artículo de revisión (Werbos, 1990) proporciona varias referencias adicionales a redescubrimientos del método en la década de 1980. Uno de los trabajos más influyentes sobre RNNs fue debido a Jeff Elman (1990), con base en una arquitectura RNN sugerida por Michael Jordan (1986). Williams y Zipser (1989) presentan un algoritmo para el aprendizaje en línea en RNNs. Bengio *et al.* (1994) analizaron el problema de los gradientes que desaparecen

en las redes recurrentes. La arquitectura de memoria a largo plazo (LSTM) (Hochreiter, 1991; Hochreiter y Schmidhuber, 1997; Gers *et al.,* 2000) se propuso como una forma de evitar este problema. Más recientemente, se han derivado diseños efectivos de RNN de manera automática (Jozefowicz *et al.,* 2015; Zoph y Le, 2016).

Se han probado múltiples métodos para mejorar la generalización en las redes neuronales. La decadencia del peso fue sugerida por Hinton (1987) y analizada matemáticamente por Krogh y Hertz (1992). El método de dropout se debe a Srivastava *et al*. (2014a). Szegedy *et al.* (2013) introdujeron la idea de ejemplos adversarios, generando una gran literatura.

Poole *et al*. (2017) mostraron que las redes profundas (pero no las superficiales) permiten desenredar funciones complejas en manifolds (colectores) planos en el espacio de unidades ocultas. Rolnick y Tegmark (2018) mostraron que el número de unidades requeridas para aproximar una cierta clase de polinomios de n variables crece de manera exponencial para las redes superficiales, pero sólo de manera lineal para las redes profundas.

White *et al*. (2019) mostraron que su sistema BANANAS podría hacer una búsqueda de arquitectura neuronal (NAS) prediciendo la precisión de una red con un margen de 1 % después de entrenar en sólo 200 arquitecturas de muestra aleatorias. Zoph y Le (2016) usan el aprendizaje por refuerzo para buscar el espacio de arquitecturas de redes neuronales. Real *et al*. (2018) usan un algoritmo evolutivo para hacer la selección del modelo, Liu *et al.* (2017) utilizan algoritmos evolutivos en representaciones jerárquicas, y Jaderberg *et al.* (2017) describen el entrenamiento basado en población. Liu *et al.* (2019) relajan el espacio de arquitecturas a un espacio diferenciable continuo y utilizan el descenso de gradiente para encontrar una solución óptima local. Pham *et al*. (2018) describen el sistema ENAS (Efficient Neural Architecture Search), que busca subgrafos óptimos de un grafo más grande. Es rápido porque no necesita reentrenar parámetros. La idea de buscar un subgrafo va de vuelta al algoritmo de "daño cerebral óptimo" de LeCun *et al.* (1990).

A pesar de esta impresionante variedad de enfoques, hay críticos que sienten que el campo aún no ha madurado. Yu *et al*. (2019) muestran que en algunos casos estos algoritmos NAS no son más eficientes que la selección de arquitectura aleatoria. Para una encuesta de resultados recientes en la búsqueda de arquitectura neuronal, ver Elsken *et al*. (2018).

El aprendizaje no supervisado constituye un gran subcampo dentro de las estadísticas, principalmente bajo el título de estimación de densidad. Silverman (1986) y Murphy (2012) son buenas fuentes para técnicas clásicas y modernas en esta área. El análisis de componentes principales (PCA) se remonta a Pearson (1901); el nombre proviene del trabajo independiente de Hotelling (1933). El modelo PCA probabilístico (Tipping y Bishop, 1999) agrega un modelo generativo para los propios componentes principales. El autoencoder variacional se debe a Kingma y Welling (2013) y Rezende *et al*. (2014); Jordan *et al*. (1999) proporcionan una introducción a los métodos variacionales para la inferencia en modelos gráficos.

Para los modelos autorregresivos, el texto clásico es de Box *et al.* (2016). Las ecuaciones de Yule–Walker para ajustar modelos AR fueron desarrolladas de manera independiente por Yule (1927) y Walker (1931). Los modelos autorregresivos con dependencias no lineales fueron desarrollados por varios autores (Frey, 1998; Bengio y Bengio, 2001; Larochelle y Murray, 2011). El modelo WaveNet autorregresivo (Van den Oord *et al.,* 2016a) se basó en trabajos anteriores sobre generación de imágenes autorregresivas (Van den Oord *et al*., 2016b). Las redes generativas adversariales, o GANs, fueron propuestas por primera vez por Goodfellow *et al*. (2015a), y han encontrado muchas aplicaciones en IA. Se está llegando a cierta comprensión teórica de sus propiedades, lo que lleva a modelos y algoritmos GAN mejorados (Li y Malik, 2018b, 2018a; Zhu *et al*., 2019). Parte de esa comprensión implica protegerse contra ataques adversariales (Carlini *et al.,* 2019).

Múltiples ramas de investigación en redes neuronales han sido populares en el pasado pero hoy en día no se exploran activamente. Las **redes de Hopfield** (Hopfield, 1982) tienen conexiones simétricas entre cada par de nodos y permiten aprender a almacenar patrones en una memoria asociativa, de modo que recuperan un patrón completo al indexar en la memoria usando un fragmento del patrón. Las redes de Hopfield son deterministas; más tarde se generalizaron a las máquinas estocásticas de Boltzmann (Hinton y Sejnowski, 1983, 1986). Las **máquinas de Boltzmann** son quizás el primer ejemplo de un modelo generativo profundo. La dificultad de la inferencia en las máquinas de Boltzmann llevó a avances en técnicas de Monte Carlo y en técnicas variacionales (ver Sección 13.4).

Red de Hopfield

Máquina de Boltzmann

La investigación en redes neuronales para la IA también ha estado entrelazada hasta cierto punto con la investigación en redes neuronales biológicas. Los dos temas coincidieron en la década de 1940, y las ideas para redes convolucionales y aprendizaje por refuerzo se rastrean hasta estudios de sistemas biológicos; pero en la actualidad, las nuevas ideas en aprendizaje profundo suelen basarse en preocupaciones puramente computacionales o estadísticas. El campo de la **neurociencia computacional** tiene como objetivo construir modelos computacionales que capturen propiedades importantes y específicas de los sistemas biológicos reales. Se dan descripciones generales por Dayan y Abbott (2001) y Trappenberg (2010).

Neurociencia computacional

Para las redes neuronales modernas y el aprendizaje profundo, los libros de texto líderes son los de Goodfellow *et al*. (2016) y Charniak (2018). También hay muchas guías prácticas asociadas con los diversos paquetes de software de código abierto para el aprendizaje profundo. Tres de los líderes del campo, Yann LeCun, Yoshua Bengio y Geoff Hinton, introdujeron las ideas clave a los investigadores no relacionados con la IA en un influyente artículo de *Nature* (2015). Los tres recibieron el Premio Turing 2018. Schmidhuber (2015) proporciona una visión general, y Deng *et al.* (2014) se centran en tareas de procesamiento de señales.

Los principales lugares de publicación para la investigación en aprendizaje profundo son la conferencia sobre Sistemas de Procesamiento de Información Neural (NeurIPS), la Conferencia Internacional sobre Aprendizaje de Máquinas (ICML) y la Conferencia Internacional sobre Representaciones de Aprendizaje (ICLR). Las principales revistas son *Machine Learning*, el *Journal of Machine Learning Research y Neural Computation*. Cada vez más, debido al ritmo acelerado de la investigación, los trabajos aparecen primero en arXiv.org y con frecuencia se describen en los blogs de investigación de los principales centros de investigación.

CAPÍTULO 22

APRENDIZAJE POR REFUERZO

En el que vemos cómo experimentar recompensas y castigos puede enseñar a un agente la manera de maximizar las recompensas en el futuro.

Con el **aprendizaje supervisado**, un agente aprende al observar ejemplos de pares de entrada/salida proporcionados por un "profesor". En este capítulo, veremos cómo los agentes aprenden de manera activa de su propia experiencia, sin un profesor, considerando su propio éxito o fracaso final.

22.1 Aprendiendo de las recompensas

Considere el problema de aprender a jugar ajedrez. Imagine tratar esto como un problema de aprendizaje supervisado utilizando los métodos de los Capítulos 19-21. La función del agente de ajedrez toma como entrada una posición del tablero y devuelve un movimiento, por lo que entrenamos esta función mediante ejemplos de posiciones de ajedrez, cada una etiquetada con el movimiento correcto. Ahora, resulta que tenemos bases de datos disponibles de varios millones de partidas de grandes maestros, cada una contiene una secuencia de posiciones y movimientos. Los movimientos realizados por el ganador son, con excepciones, asumidos como buenos, si no perfectos. Así, tenemos un conjunto de entrenamiento prometedor. El problema es que hay relativamente pocos ejemplos (alrededor de 10^8) en comparación con el espacio de todas las posiciones de ajedrez posibles (alrededor de 10^{40}). En una nueva partida, pronto se encuentran posiciones significativamente diferentes a las de la base de datos, y es probable que la función del agente entrenado falle, no porque no tiene idea de lo que se supone que deben lograr sus movimientos (jaque mate) o incluso porque no supiera qué efecto tienen los movimientos en las posiciones de las piezas. Y, por supuesto, el ajedrez es una pequeña parte del mundo real. Para problemas más realistas, necesitaríamos bases de datos de grandes maestros mucho más vastas, pero no existen.[1]

Aprendizaje por refuerzo

Una alternativa es el **aprendizaje por refuerzo** (**RL**, por sus siglas en inglés), en el que un agente interactúa con el mundo. Refuerza y recibe de manera periódica **recompensas** (o, en la terminología de la psicología, **refuerzos**) que reflejan lo bien que lo está haciendo. Por ejemplo, en el ajedrez la recompensa es 1 por ganar, 0 por perder, y $\frac{1}{2}$ para un empate. Ya hemos visto el concepto de recompensas en el Capítulo 17 para **procesos de decisión de Markov** (**MDP**). De hecho, el objetivo es el mismo en el aprendizaje por refuerzo: maximizar la suma esperada de recompensas. El aprendizaje por refuerzo difiere de "sólo resolver un MDP" porque el agente *no recibe* el MDP como un problema para resolver; el agente está *en* el MDP. Es posible que no conozca el modelo de transición o la función de recompensa, y tiene que actuar para aprender más. Imagine jugar un nuevo juego cuyas reglas no conoce; después de un centenar de movimientos, el árbitro le dice "Usted pierde". Eso es el aprendizaje por refuerzo en pocas palabras.

Desde nuestro punto de vista como diseñadores de sistemas de IA, dar una señal de recompensa al agente suele ser mucho más fácil que proporcionar ejemplos etiquetados de cómo

[1] Como Yann LeCun y Alyosha Efros han señalado, "la revolución de la IA no será supervisada".

comportarse. Primero, la recompensa (como vimos para el ajedrez) es muy concisa y fácil de especificar: solo requiere unas pocas líneas de código para decirle al agente de ajedrez si ha ganado o perdido el juego o para decirle al agente de carreras de autos que ha ganado o perdido la carrera o ha chocado. En segundo lugar, no tenemos que ser expertos, capaces de proporcionar la acción correcta en cualquier situación, como sería el caso si intentáramos aplicar el aprendizaje supervisado.

Sin embargo, un poco de experiencia resulta muy útil en el aprendizaje por refuerzo. Los dos ejemplos en el párrafo anterior, las recompensas de ganar/perder para el ajedrez y las carreras, son lo que llamamos recompensas **escasas**, porque en la gran mayoría de los estados al agente no se le da ninguna señal de recompensa informativa en absoluto. En juegos como el tenis y el cricket, podemos proporcionar recompensas adicionales por cada punto ganado o por cada carrera anotada. En las carreras de autos podríamos recompensar al agente por avanzar en la pista en la dirección correcta. Cuando se aprende a gatear, cualquier movimiento hacia adelante es un logro. Estas recompensas intermedias facilitan mucho el aprendizaje.

Escasas

Siempre que podamos proporcionar la señal de recompensa correcta al agente, el aprendizaje por refuerzo da una forma generalizada para construir sistemas de IA. Esto es en particular cierto para los entornos *simulados*, donde no faltan oportunidades para ganar experiencia. La adición del aprendizaje profundo como herramienta dentro de los sistemas de RL también ha hecho posibles nuevas aplicaciones, incluidas aprender a jugar videojuegos de Atari a partir de la entrada visual sin etiquetar (Mnih *et al*., 2013), controlar robots (Levine *et al*., 2016), y jugar póker (Brown y Sandholm, 2017).

Literalmente se han ideado cientos de diferentes algoritmos de aprendizaje por refuerzo, muchos de ellos emplean como herramientas una amplia gama de métodos de aprendizaje de los Capítulos 19–21. Aquí, cubrimos las ideas básicas y damos una noción de la variedad de enfoques con algunos ejemplos, los cuales categorizamos de la siguiente manera:

Aprendizaje por refuerzo basado en modelos

- **Aprendizaje por refuerzo basado en modelos:** En estos enfoques, el agente utiliza un modelo de transición del entorno para ayudar a interpretar las señales de recompensa y tomar decisiones sobre cómo actuar. El modelo puede ser en inicio desconocido, en cuyo caso el agente aprende el modelo observando los efectos de sus acciones, o puede ser conocido; por ejemplo, un programa de ajedrez conoce las reglas del ajedrez aun si no sabe cómo elegir buenos movimientos. En entornos parcialmente observables, el modelo de transición también es útil para la **estimación de estados** (ver Capítulo 14). En el aprendizaje por refuerzo basado en modelos los sistemas con frecuencia aprenden una **función de utilidad** $U(s)$, definida (como en el Capítulo 17) en términos de la suma de recompensas desde el estado s en adelante.[2]

Aprendizaje por refuerzo sin modelo

- **Aprendizaje por refuerzo sin modelo:** En estos enfoques, el agente ni conoce ni aprende un modelo de transición para el entorno. En su lugar, aprende una representación más directa de cómo comportarse. Esto viene en una de dos variedades:

Aprendizaje de utilidad-acción

Q-learning

Función Q

  - **Aprendizaje de utilidad-acción:** Introdujimos funciones de utilidad-acción en el Capítulo 17. La forma más común de aprendizaje de utilidad-acción es el **Q-learning**, donde el agente aprende una **función Q**, o función de calidad, $Q(s,a)$, que define la suma de recompensas desde el estado s en adelante, si se toma la acción a. Dada una función Q, el agente elige qué hacer en s encontrando la acción con el valor Q más alto.

Búsqueda de políticas

  - **Búsqueda de políticas:** El agente aprende una política $\pi(s)$ que mapea de manera directa de estados a acciones. En la terminología del Capítulo 2, esto es un **agente reflejo**.

[2] En la literatura de RL, que se basa más en investigación de operaciones que en economía, las funciones de utilidad suelen llamarse **funciones de valor** y denotan como $V(s)$.

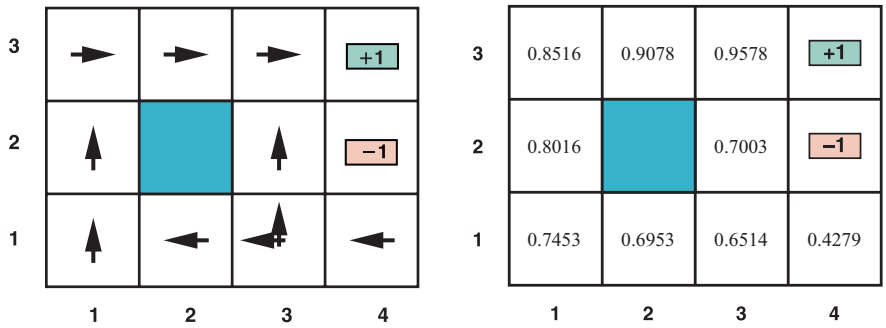


Figura 22.1 (a) Políticas óptimas para el entorno estocástico con $R(s, a, s') = -0.04$ para transiciones entre estados no terminales. Hay dos políticas porque en el estado (3,1) tanto *Izquierda* como *Arriba* son óptimas. Vimos esto antes en la Figura 17.2. (b) Utilidades de los estados en el mundo 4 × 3, dada la política π.

Comenzamos la Sección 22.2 con el **aprendizaje por refuerzo pasivo**, donde la política del agente es fija y la tarea es aprender las utilidades de los estados (o de los pares estado-acción); esto también podría implicar aprender un modelo del entorno. (Una comprensión de los procesos de decisión de Markov, como se describe en el Capítulo 17, es esencial para esta sección). La Sección 22.3 cubre el **aprendizaje por refuerzo activo**, donde el agente también debe averiguar qué hacer. El principal problema es la **exploración**: un agente debe experimentar tanto como sea posible su entorno para aprender a comportarse en él. La Sección 22.4 discute cómo un agente usa el aprendizaje inductivo (incluyendo métodos de aprendizaje profundo) para aprender mucho más rápido de sus experiencias. También discutimos otros enfoques que permiten ayudar a escalar RL para resolver problemas reales, incluida la provisión de seudorrecompensas intermedias para guiar al aprendiz y organizar el comportamiento en una jerarquía de acciones. La Sección 22.5 cubre métodos para la búsqueda de políticas. En la Sección 22.6, exploramos el **aprendizaje por observación**: entrenando a un agente de aprendizaje usando demostraciones en lugar de señales de recompensa. Finalmente, la Sección 22.7 informa sobre las aplicaciones del aprendizaje por refuerzo.

Aprendizaje por refuerzo pasivo

Aprendizaje por refuerzo activo

22.2 Aprendizaje por refuerzo pasivo

Comenzamos con el caso simple de un entorno por completo observable con un número limitado de acciones y estados, en el que un agente ya tiene una política fija $\pi(s)$ que determina sus acciones. El agente está tratando de aprender la función de utilidad $U^{\pi}(s)$; la recompensa total descontada esperada si la política π se ejecuta comenzando en el estado s. Llamamos a esto un **agente de aprendizaje pasivo**.

Agente de aprendizaje pasivo

La tarea de aprendizaje pasivo es similar a la tarea de **evaluación de políticas**, parte de la iteración de políticas algoritmo descrito en la Sección 17.2.2. La diferencia es que el agente de aprendizaje pasivo no conoce el modelo de transición $P(s'|s,a)$, que especifica la probabilidad de alcanzar el estado s' desde el estado s después de realizar la acción a; tampoco conoce la función de recompensa $R(s,a,s')$, que especifica la recompensa por cada transición.

Usaremos como ejemplo el mundo 4 × 3 introducido en el Capítulo 17. La Figura 22.1 muestra las políticas óptimas para ese mundo y las utilidades correspondientes. El agente ejecuta un

Prueba

conjunto de **pruebas** en el entorno utilizando su política π. En cada prueba, el agente comienza en el estado (1,1) y experimenta una secuencia de transiciones de estado hasta que alcanza uno de los estados terminales, (4,2) o (4,3). Sus percepciones suministran tanto el estado actual como la recompensa recibida por la transición que acaba de ocurrir para alcanzar ese estado. Las pruebas típicas podrían parecerse a esto:

$$(1,1) \xrightarrow[Arriba]{-.04} (1,2) \xrightarrow[Arriba]{-.04} (1,3) \xrightarrow[Derecha]{-.04} (1,2) \xrightarrow[Arriba]{-.04} (1,3) \xrightarrow[Derecha]{-.04} (2,3) \xrightarrow[Derecha]{-.04} (3,3) \xrightarrow[Derecha]{+1} (4,3)$$
$$(1,1) \xrightarrow[Arriba]{-.04} (1,2) \xrightarrow[Arriba]{-.04} (1,3) \xrightarrow[Derecha]{-.04} (2,3) \xrightarrow[Derecha]{-.04} (3,3) \xrightarrow[Derecha]{-.04} (3,2) \xrightarrow[Arriba]{-.04} (3,3) \xrightarrow[Derecha]{+1} (4,3)$$
$$(1,1) \xrightarrow[Arriba]{-.04} (1,2) \xrightarrow[Arriba]{-.04} (1,3) \xrightarrow[Derecha]{-.04} (2,3) \xrightarrow[Derecha]{-.04} (3,3) \xrightarrow[Derecha]{-.04} (3,2) \xrightarrow[Arriba]{-1} (4,2)$$

Note que cada transición está anotada con la acción realizada y la recompensa recibida en el siguiente estado. El objetivo es utilizar la información sobre las recompensas para aprender la utilidad esperada $U^{\pi}(s)$ asociada a cada estado no terminal s. La utilidad se define como la suma esperada de recompensas (descontadas) obtenidas si se sigue la política π. Como en la Ecuación (17.2) en la página 567, escribimos

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1})\right], \tag{22.1}$$

donde $R(S_t,\pi(S_t),S_{t+1})$ es la recompensa recibida cuando se toma la acción $\pi(S_t)$ en el estado S_t y se alcanza el estado S_{t+1}. Note que S_t es una variable aleatoria que denota el estado alcanzado en el tiempo t al ejecutar la política π, comenzando desde el estado $S_0 = s$. Incluiremos un **factor de descuento** γ en todas nuestras ecuaciones, pero para el mundo 4×3 estableceremos $\gamma = 1$, lo que significa sin descuento.

22.2.1 Estimación directa de utilidad

Estimación directa de utilidad

Valor esperado de la recompensa acumulada

La idea de la **estimación directa de la utilidad** es que la utilidad de un estado se define como la recompensa total esperada a partir de ese estado en adelante (llamado el **valor esperado de la recompensa acumulada**), y que cada prueba proporciona una *muestra* de esta cantidad para cada estado visitado. Por ejemplo, la primera de las tres pruebas antes mostradas proporciona una muestra de recompensa total de 0.76 para el estado (1,1), dos muestras de 0.80 y 0.88 para (1,2), dos muestras de 0.84 y 0.92 para (1,3), y así sucesivamente. De este modo, al final de cada secuencia el algoritmo calcula la recompensa acumulada para cada estado y, en consecuencia, actualiza la utilidad estimada para ese estado, sólo para mantener un promedio en ejecución para cada estado en una tabla. En el límite de un número infinito de pruebas, el promedio de las muestras convergerá a la verdadera expectativa en la Ecuación (22.1).

Esto significa que hemos reducido el aprendizaje por refuerzo a un problema estándar de aprendizaje supervisado en el que cada ejemplo es un par (*estado, recompensa acumulada*). Tenemos múltiples algoritmos poderosos para el aprendizaje supervisado, por lo que este enfoque parece prometedor, pero ignora una restricción importante: *La utilidad de un estado está determinada por la recompensa y la utilidad esperada de los estados sucesores*. En específico, los valores de utilidad obedecen las ecuaciones de Bellman para una política fija (ver también la Ecuación (17.14)):

$$U_i(s) = \sum_{s'} P(s' \mid s, \pi_i(s))[R(s, \pi_i(s), s') + \gamma U_i(s')]. \tag{22.2}$$

Al ignorar las conexiones entre los estados, la estimación directa de la utilidad pierde oportunidades de aprendizaje. Por ejemplo, el segundo de los tres ensayos previos dados llega al estado (3,2), que no ha sido visitado antes. La siguiente transición llega a (3,3), que se sabe desde el primer ensayo que tiene una alta utilidad. La ecuación de Bellman sugiere que (3,2) también es probable que tenga una alta utilidad, porque conduce a (3,3), pero la estimación directa de la utilidad no aprende

nada hasta el final del ensayo. De manera más amplia, podemos ver la estimación directa de la utilidad como la búsqueda de U en un espacio de hipótesis que es mucho más grande de lo que necesita ser, en el sentido de que incluye muchas funciones que violan las ecuaciones de Bellman. Por esta razón, el algoritmo en la mayoría de los casos converge de manera lenta.

22.2.2 Programación dinámica adaptativa

Programación dinámica adaptativa

Un agente de **programación dinámica adaptativa** (o ADP, por sus siglas en inglés) aprovecha las restricciones de la dinámica adaptativa entre las utilidades de los estados al aprender el modelo de transición que los conecta y resolver el proceso de decisión de Markov correspondiente utilizando programación dinámica. Para un agente de aprendizaje pasivo, esto significa conectar el modelo de transición aprendido $P(s'|s, \pi(s))$ y las recompensas observadas $R(s, \pi(s),s')$ en la Ecuación (22.2) para calcular las utilidades de los estados. Como mencionamos en nuestra discusión sobre la iteración de políticas en el Capítulo 17, estas ecuaciones de Bellman son lineales cuando la política π es fija, por lo que se pueden resolver utilizando cualquier paquete de álgebra lineal.

Como alternativa, podemos adoptar el enfoque de la **iteración de políticas modificada** (ver página 578), utilizando un proceso simplificado de iteración de valor para actualizar las estimaciones de utilidad después de cada cambio en el modelo aprendido. Debido a que el modelo en general cambia de manera mínima con cada observación, el proceso de iteración de valor permite usar las estimaciones de utilidad anteriores como valores iniciales y, por lo regular, converger de manera rápida.

Aprender el modelo de transición es fácil, porque el entorno es completamente observable. Esto significa que tenemos una tarea de aprendizaje supervisada donde la entrada para cada ejemplo de entrenamiento es un par estado-acción, (s,a), y la salida es el estado resultante, s'. El modelo de transición $P(s'|s,a)$ se representa como una tabla y se estima de manera directa a partir de los recuentos que se acumulan en $N_{s'|sa}$. Los recuentos registran con qué frecuencia se alcanza el estado s' al ejecutar a en s. Por ejemplo, en los tres ensayos dados en la página 792, *Derecha* se ejecuta cuatro veces en (3,3) y el estado resultante es (3,2) dos veces y (4,3) dos veces, por lo que $P((3,2)|(3,3), \textit{Derecha})$ y $P((4,3)|(3,3), \textit{Derecha})$ se estima que ambos sean $\frac{1}{2}$.

El programa completo del agente para un agente ADP pasivo se muestra en la Figura 22.2. Su rendimiento en el mundo 4×3 se muestra en la Figura 22.3. En términos de cuán rápido mejoran sus estimaciones de valor, el agente ADP está limitado sólo por su capacidad para aprender el modelo de transición. En este sentido, proporciona un estándar contra el cual medir cualquier otro algoritmo de aprendizaje por refuerzo. Sin embargo, es intratable para espacios de estado grandes. En el juego de backgammon, por ejemplo, implicaría resolver casi 10^{20} ecuaciones en 10^{20} incógnitas.

22.2.3 Aprendizaje de diferencias temporales

Resolver el MDP subyacente como en la sección anterior no es la única forma de aplicar las ecuaciones de Bellman al problema de aprendizaje. Otra forma es utilizar las transiciones observadas para ajustar las utilidades de los estados observados de manera que concuerden con las ecuaciones de restricción. Considere, por ejemplo, la transición de (1,3) a (2,3) en el segundo ensayo en la página 792. Suponga que como resultado del primer ensayo, las estimaciones de utilidad son $U(1,3) = 0.88$ y $U^{\pi}(2,3) = 0.96$. Ahora, si esta transición de (1,3) a (2,3) ocurriera todo el tiempo, esperaríamos que las utilidades obedecieran a la ecuación

$$U^{\pi}(1,3) = -0.04 + U^{\pi}(2,3),$$

así $U^{\pi}(1,3)$ sería 0.92. Por lo tanto, su estimación actual de 0.84 podría ser un poco baja y debería aumentarse. Por lo regular, cuando ocurre una transición del estado s al estado s' a través de la acción π(s), aplicamos la siguiente actualización a $U^{\pi}(s)$:

function PASSIVE-ADP-LEARNER(*percept*) **returns** una acción
  inputs: *percept*, un percepto indica el estado actual de s' y la señal de recompensa r
  persistent: π, una política fija
    mdp, un MDP con modelo P, recompensas R, acciones A, descuento γ
    U, una tabla de utilidades para estados, en inicio vacía
    $N_{s'|s,a}$, una tabla de vectores de recuento de resultados indexados por estado y acción, inicialmente cero
    s,a, el estado y acción anteriores, en inicio nulos

  if s' es nuevo **then** $U[s'] \leftarrow 0$
  if s no es nulo **then**
    incrementar $N_{s'|s,a}[s,a][s']$
    $R[s,a,s'] \leftarrow r$
    añadir a a $A[s]$
    $\mathbf{P}(\cdot|s,a) \leftarrow$ NORMALIZE($N_{s'|s,a}[s,a]$)
    $U \leftarrow$ POLICYEVALUATION(π, U, mdp)
  $s,a \leftarrow s', \pi[s']$
  return a

Figura 22.2 Agente de aprendizaje por refuerzo pasivo basado en programación dinámica adaptativa. El agente elige un valor para γ y luego calcula de manera incremental los valores P y R del MDP. La función POLICYEVALUATION resuelve las ecuaciones de Bellman de política fija, como se describe en la página 577.

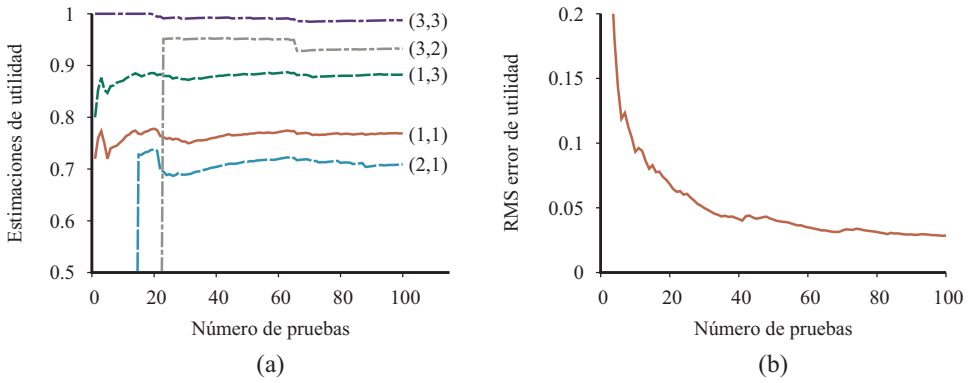


Figura 22.3 Las curvas de aprendizaje ADP pasivo para el mundo 4 × 3, dada la política óptima mostrada en la Figura 22.1. (a) Las estimaciones de utilidad para un subconjunto seleccionado de estados, como una función del número de pruebas. Observe que se necesitan 14 y 23 pruebas respectivamente antes de que los estados poco visitados (2,1) y (3,2) "descubran" que se conectan al estado de salida +1 en (4,3). (b) El error cuadrático medio (ver Apéndice A) en la estimación para $U(1,1)$, promediado en 50 ejecuciones de 100 pruebas cada una.

```
function PASSIVE-TD-LEARNER(percept) returns una acción
    inputs: percept, un percepto indica el estado actual s′ y la señal de recompensa r
    persistent: π, una política fija
                s, el estado anterior, inicialmente nulo
                U, una tabla de utilidades para estados, en inicio vacía
                N_s, una tabla de frecuencias para estados, inicialmente cero

    if s′ es nuevo then U[s′] ← 0
    if s no es nulo then
        incrementar N_s[s]
        U[s] ← U[s] + α(N_s[s]) × (r + γU[s′] - U[s])
    s ← s′
    return π[s′]
```

Figura 22.4 Un agente de aprendizaje por refuerzo pasivo que aprende estimaciones de utilidad empleando diferencias temporales. La función de tamaño de paso $\alpha(n)$ se elige para garantizar la convergencia.

$$U^{\pi}(s) \leftarrow U^{\pi}(s) + \alpha[R(s,\pi(s),s') + \gamma U^{\pi}(s') - U^{\pi}(s)]. \quad (22.3)$$

Aquí, α es el parámetro de **tasa de aprendizaje**. Debido a que esta regla de actualización emplea la diferencia en utilidades entre estados sucesivos (y, por lo tanto, tiempos sucesivos), con frecuencia se le llama ecuación de **diferencia temporal** (TD, por sus siglas en inglés). Al igual que en las reglas de actualización de peso del Capítulo 19 (por ejemplo, Ecuación (19.6) en la página 680), el término TD: $R(s,(s),s') + \gamma U^{\pi}(s') - U^{\pi}(s)$, es una señal de error, y la actualización tiene como objetivo reducir el error.

Diferencia temporal

Todos los métodos de diferencia temporal funcionan ajustando las estimaciones de utilidad hacia el equilibrio ideal que se mantiene de manera local cuando las estimaciones de utilidad son correctas. En el caso del aprendizaje pasivo, el equilibrio está dado por la Ecuación (22.2). Ahora bien, la Ecuación (22.3) de hecho hace que el agente alcance el equilibrio dado por la Ecuación (22.2), pero hay cierta sutileza involucrada. Primero, observe que la actualización sólo involucra al sucesor observado s', mientras que las condiciones de equilibrio reales implican todos los posibles estados siguientes. Uno podría pensar que esto causa un cambio grande en $U^{\pi}(s)$ cuando ocurre una transición muy rara; pero, de hecho, debido a que las transiciones raras ocurren poco, el *valor promedio* de $U^{\pi}(s)$ convergerá a la cantidad correcta en el límite, incluso si el valor en sí mismo continúa fluctuando.

Además, si convertimos el parámetro α en una función que disminuye a medida que aumenta el número de veces que se ha visitado un estado, como se muestra en la Figura 22.4, entonces $U^{\pi}(s)$ en sí convergerá al valor correcto.[3] La Figura 22.5 ilustra el rendimiento del agente TD pasivo en el mundo 4 × 3. No aprende tan rápido como el agente ADP y muestra mucha más variabilidad, pero es mucho más simple y requiere mucho menos cálculo por observación. Observe que TD *no necesita un modelo de transición para realizar sus actualizaciones*. El propio entorno proporciona la conexión entre los estados vecinos en forma de transiciones observadas.

[3] Las condiciones técnicas se dan en la página 684. En la Figura 22.5 hemos usado $\alpha(n) = 60/(59 + n)$, que satisface las condiciones.

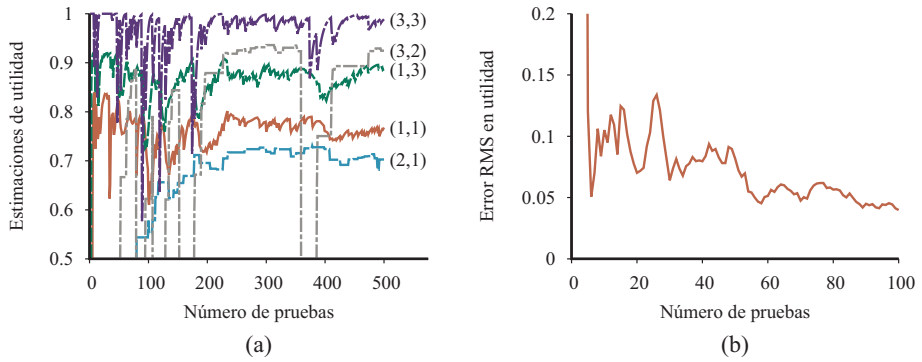


Figura 22.5 Curvas de aprendizaje de TD para el mundo 4 × 3. (a) Las estimaciones de utilidad para un subconjunto seleccionado de estados, en función del número de pruebas, para una única ejecución de 500 pruebas. Compare con la ejecución de 100 pruebas en la Figura 22.3(a). (b) El error cuadrático medio en la estimación para $U(1,1)$, promediado en 50 ejecuciones de 100 pruebas cada una.

Los enfoques de ADP y TD están relacionados. Ambos intentan hacer ajustes locales a las estimaciones de utilidad para hacer que cada estado "concuerde" con sus sucesores. Una diferencia es que TD ajusta un estado para que concuerde con su sucesor *observado* (Ecuación (22.3)), mientras que ADP ajusta el estado para que concuerde con *todos* los sucesores que podrían ocurrir, ponderados por sus probabilidades (Ecuación (22.2)). Esta diferencia desaparece cuando los efectos de los ajustes de TD se promedian en un gran número de transiciones, porque la frecuencia de cada sucesor en el conjunto de transiciones es casi proporcional a su probabilidad. Una diferencia más importante es que mientras que TD hace un sólo ajuste por transición observada, ADP hace tantos como necesita para restaurar la consistencia entre las estimaciones de utilidad U y el modelo de transición P. Aunque la transición observada sólo hace un cambio local en P, sus efectos podrían necesitar ser propagados a través de U. Así, TD puede ser visto como una primera aproximación cruda pero eficiente a ADP.

Seudoexperiencia

Cada ajuste realizado por ADP podría ser visto, desde el punto de vista de TD, como resultado de una **seudoexperiencia** generada al simular el modelo de transición actual. Es posible extender el enfoque de TD para usar un modelo de transición para generar varias seudoexperiencias, transiciones que el agente TD puede imaginar que *podrían* suceder, dada su modelo actual. Para cada transición observada, el agente TD genera un gran número de transiciones imaginarias. De esta manera, las estimaciones de utilidad resultantes se aproximarán cada vez más a las de ADP; por supuesto, a expensas de un mayor tiempo de cálculo.

En una línea similar, podemos generar versiones más eficientes de ADP al aproximar de manera directa los algoritmos para la iteración de valor o la iteración de política. Aunque el algoritmo de iteración de valor es eficiente, es intratable si tenemos, digamos, 10^{100} estados. Sin embargo, muchos de los ajustes necesarios a los valores de estado en cada iteración serán pequeños. Un posible enfoque para generar respuestas razonablemente buenas de manera rápida es limitar el número de ajustes realizados después de cada transición observada. También es posible usar una heurística para clasificar los posibles ajustes de manera que sólo se realicen los más significativos.

Barrido priorizado

La heurística de **barrido priorizado** prefiere hacer ajustes a los estados cuyos sucesores probables acaban de sufrir un *gran* ajuste en sus propias estimaciones de utilidad.

Usando heurísticas como esta, los algoritmos ADP aproximados permiten aprender casi tan rápido como el ADP completo, en términos del número de secuencias de entrenamiento;

pero pueden ser órdenes de magnitud más eficientes en términos de cálculo total (ver Ejercicio 22.PRSW). Esto les permite manejar espacios de estado que son demasiado grandes para el ADP completo. Los algoritmos ADP aproximados tienen una ventaja adicional: en las primeras etapas de aprendizaje de un nuevo entorno, el modelo de transición P estará lejos de ser correcto, por lo que tiene poco sentido calcular una función de utilidad exacta para que coincida con él. Un algoritmo de aproximación usa un tamaño de ajuste mínimo que disminuye a medida que el modelo de transición se vuelve más preciso. Esto elimina las carreras muy largas de iteración de valor que pueden ocurrir temprano en el aprendizaje debido a grandes cambios en el modelo.

22.3 Aprendizaje por refuerzo activo

Un agente de aprendizaje pasivo tiene una política fija que determina su comportamiento. Un **agente de aprendizaje activo** decide qué acciones tomar. Comencemos con el agente de programación dinámica adaptativa (ADP) y consideremos cómo es posible modificarlo para aprovechar esta nueva libertad.

Primero, el agente necesitará aprender un modelo de transición completo con probabilidades de resultado para *todas* las acciones, en lugar de sólo el modelo para la política fija. El mecanismo de aprendizaje utilizado por PASSIVE-ADP-AGENT servirá para esto. A continuación, necesitamos tener en cuenta el hecho de que el agente tiene una elección de acciones. Las utilidades que necesita aprender son las definidas por la política óptima; obedecen a las ecuaciones de Bellman (que repetimos aquí):

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]. \tag{22.4}$$

Estas ecuaciones se resuelven para obtener la función de utilidad U utilizando los algoritmos de iteración de valor o iteración de políticas del Capítulo 17.

El problema final es qué hacer en cada paso. Habiendo obtenido una función de utilidad U que es óptima para el modelo aprendido, el agente extrae una acción óptima mediante una mirada hacia adelante de un paso para maximizar la utilidad esperada; de manera alternativa, si utiliza la iteración de políticas, la política óptima ya está disponible, por lo que podría ejecutar la acción que recomienda la política óptima. ¿Pero debería hacerlo?

22.3.1 Exploración

La Figura 22.6 muestra los resultados de una secuencia de pruebas para un agente ADP que sigue la recomendación de la política óptima para el modelo aprendido en cada paso. ¡El agente *no* aprende las verdaderas utilidades ni la verdadera política óptima! Lo que sucede en cambio es que, en la tercera prueba, encuentra una política que alcanza la recompensa de +1 a lo largo de la ruta inferior a través de (2,1), (3,1), (3,2) y (3,3). (Ver Figura 22.6(b)). Después de experimentar con variaciones menores, a partir de la octava prueba se adhiere a esa política, nunca aprendiendo las utilidades de los otros estados y nunca encontrando la ruta óptima a través de (1,2), (1,3) y (2,3). Llamaremos a este agente un **agente codicioso**, porque es un agente que toma de manera agresiva (codiciosamente) la acción que ahora cree que es óptima en cada paso. A veces la codicia da sus frutos y el agente converge a la política óptima, pero en la mayoría de los casos no lo hace.

Agente codicioso

¿Cómo puede ser que elegir la acción óptima conduzca a resultados subóptimos? La respuesta es que el modelo aprendido no es el mismo que el entorno real; lo que es óptimo en el modelo aprendido puede ser subóptimo en el entorno real. Desafortunadamente, el agente no sabe cuál es el entorno real, por lo que no permite calcular la acción óptima para el entorno real. ¿Qué debería hacer entonces?

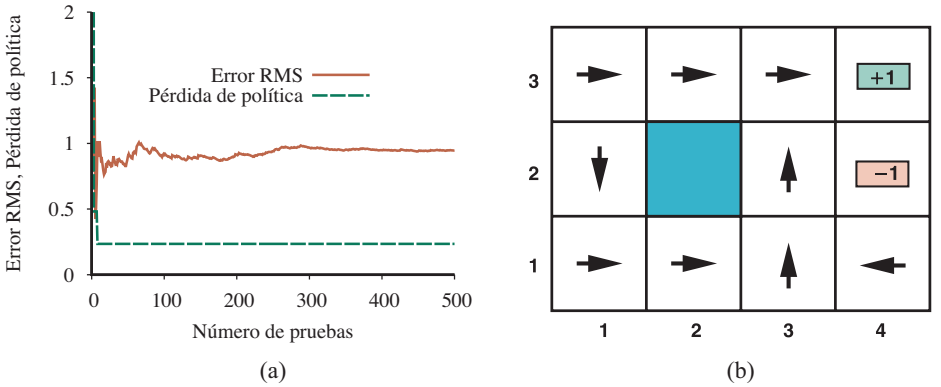


Figura 22.6 Rendimiento de un agente ADP codicioso que ejecuta la acción recomendada por la política óptima para el modelo aprendido. (a) El error cuadrático medio (RMS) promediado en todas las nueve casillas no terminales y la pérdida de política en (1,1). Vemos que la política converge de manera rápida, después de sólo ocho pruebas, a una política subóptima con una pérdida de 0.235. (b) La política subóptima a la que converge el agente codicioso en esta secuencia particular de pruebas. Observe la acción *Down* en (1,2).

El agente codicioso ha pasado por alto el hecho de que las acciones hacen más que proporcionar *recompensas*; también entregan *información* en forma de percepciones en los estados resultantes. Como vimos con los **problemas de bandidos** en la Sección 17.3, un agente debe hacer un equilibrio entre la **explotación** de la mejor acción actual para maximizar su recompensa a corto plazo y la **exploración** de estados previamente desconocidos para obtener información que pueda llevar a un cambio en la política (y a mayores recompensas en el futuro). En el mundo real, uno tiene que decidir constantemente entre continuar en una existencia cómoda, versus aventurarse en lo desconocido con la esperanza de una vida mejor.

Aunque los problemas de bandidos son difíciles de resolver para obtener un esquema de exploración *óptimo*, aún es posible idear un esquema que descubra una política óptima, incluso si podría tardar más en hacerlo que lo óptimo. Cualquier esquema de este tipo no debería ser codicioso en términos del próximo movimiento inmediato, sino que debería ser lo que se llama "codicioso en el límite de la exploración infinita", o **GLIE**. Un esquema GLIE debe probar cada acción en cada estado un número ilimitado de veces para evitar tener una probabilidad finita de que se pierda una acción óptima. Un agente ADP que utiliza tal esquema aprenderá el verdadero modelo de transición, y luego operará bajo explotación.

GLIE

Existen varios esquemas GLIE; uno de los más simples es hacer que el agente elija una acción aleatoria en el paso de tiempo t con probabilidad $1/t$ y siga la política codiciosa en caso contrario. Aunque esto finalmente converge a una política óptima, es lento. Un enfoque mejor daría algún peso a las acciones que el agente no ha probado de manera frecuente, mientras tiende a evitar acciones que se cree que son de baja utilidad (como hicimos con la búsqueda de árboles de Monte Carlo en la Sección 5.4). Esto se implementa modificando la ecuación de restricción (22.4) para que asigne una estimación de utilidad más alta a pares de estado-acción relativamente inexplorados.

Esto equivale a un prior (conocimiento inicial) optimista sobre los posibles entornos y hace que el agente se comporte en inicio como si hubiera maravillosas recompensas dispersas por todas partes. Usemos $U^{+}(s)$ para denotar la estimación optimista de la utilidad (es decir, la recompensa esperada para ir) del estado s, y que $N(s,a)$ sea el número de veces que la acción a se

ha probado en el estado s. Suponga que estamos utilizando la iteración de valor en un agente de aprendizaje ADP; entonces necesitamos reescribir la ecuación de actualización (Ecuación (17.10) en la página 573) para incorporar la estimación optimista:

$$U^+(s) \leftarrow \max_a f\left(\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U^+(s')], N(s,a)\right). \tag{22.5}$$

Función de exploración

Aquí, f es la **función de exploración**. La función $f(u,n)$ determina cómo la codicia (preferencia para valores altos de la utilidad u) se negocia contra la curiosidad (preferencia por acciones que tienen no se han probado con la suficiente frecuencia y tienen un recuento n bajo). La función debe ser creciente en u y decreciente en n. Note que existen múltiples funciones posibles que se ajustan a estas condiciones. Una definición simple es

$$f(u,n) = \begin{cases} R^+ & \text{if } n < N_e \\ u & \text{de otra manera,} \end{cases}$$

donde R^+ es una estimación optimista de la mejor recompensa posible obtenible en cualquier estado y N_e es un parámetro fijo. Esto tendrá el efecto de hacer que el agente pruebe cada par de estado-acción al menos N_e veces. El hecho de que U^+ en lugar de U aparezca en el lado derecho de la Ecuación (22.5) es muy importante. A medida que la exploración avanza, los estados y acciones cerca del estado inicial podrían ser probados un gran número de veces. Si usáramos U, la estimación de utilidad más pesimista, entonces el agente pronto se inclinaría a explorar más lejos. El uso de U^+ significa que los beneficios de la exploración se propagan desde los bordes de regiones inexploradas, de modo que las acciones que conducen *hacia* regiones inexploradas tienen un peso más alto, en lugar de sólo acciones que son en sí mismas desconocidas.

El efecto de esta política de exploración se observa en la Figura 22.7(b), que muestra una convergencia rápida hacia una pérdida de política cero, a diferencia del enfoque codicioso. Se encuentra una política casi óptima después de sólo 18 pruebas. Observe que el error RMS en las estimaciones de utilidad no converge tan rápido. Esto se debe a que el agente deja de explorar las partes no gratificantes del espacio de estado bastante pronto, visitándolas sólo "por accidente" a partir de entonces. Tiene mucho sentido que el agente no se preocupe por las utilidades exactas de los estados que sabe que son indeseables y pueden evitarse. No tiene mucho sentido aprender sobre la mejor estación de radio para escuchar mientras se cae de un acantilado.

22.3.2 Exploración segura

Hasta ahora hemos asumido que un agente es libre de explorar como desee, que cualquier recompensa negativa sirve sólo para mejorar su modelo del mundo. Es decir, si jugamos una partida de ajedrez y perdemos, no sufrimos ningún daño (excepto, quizá, nuestro orgullo), y lo que aprendimos nos hará un mejor jugador en el próximo juego. De manera similar, en un entorno de simulación para un vehículo autónomo, podríamos explorar los límites de su rendimiento, además de que cualquier accidente nos da más información. Si el coche se estrella, sólo pulsamos el botón de reinicio.

No obstante, el mundo real es menos indulgente. Si usted es un pez sol bebé, su probabilidad de sobrevivir hasta la edad adulta es de alrededor de 0.00000001. Muchas acciones son **irreversibles**, en el sentido definido para los agentes de búsqueda en línea en la Sección 4.5: no hay forma de volver al estado anterior. En el peor de los casos, el agente entra en un **estado absorbente** donde ninguna acción tiene ningún efecto y no se reciben recompensas.

Estado absorbente

En múltiples entornos prácticos, no es posible permitir que nuestros agentes tomen acciones irreversibles o entren en estados absorbentes. Por ejemplo, un agente que aprende a conducir en

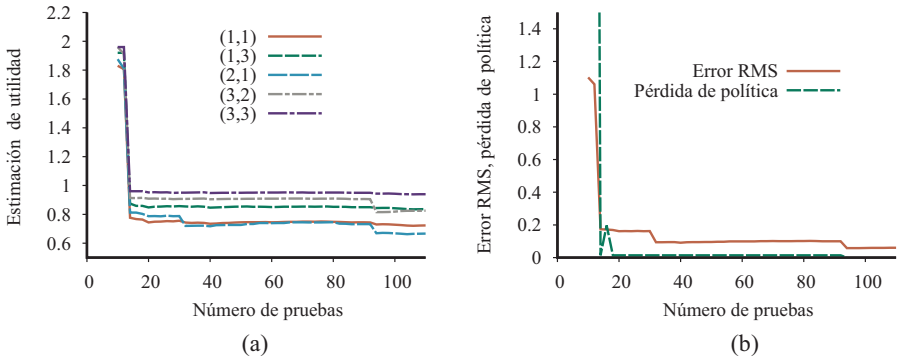


Figura 22.7 Rendimiento del agente ADP exploratorio utilizando $R^+ = 2$ y $N_e = 5$. (a) Estimaciones de utilidad para estados seleccionados a lo largo del tiempo. (b) Error RMS en los valores de utilidad y pérdida de política asociada.

un coche real debería evitar realizar acciones que podrían llevar a cualquiera de las siguientes situaciones:

- estados con grandes recompensas negativas, como graves choques de autos;
- estados de los cuales no hay escape, como conducir el auto a una zanja profunda;
- estados que limitan de manera permanente las recompensas futuras, como dañar el motor del auto de modo que su velocidad máxima se reduzca.

Podemos terminar en un mal estado ya sea porque nuestro modelo es *desconocido*, y elegimos explorar en una dirección que resulta ser mala, o porque nuestro modelo es *incorrecto* y no sabemos que una acción dada puede tener un resultado desastroso. Note que el algoritmo en la Figura 22.2 está utilizando la estimación de máxima verosimilitud (ver Capítulo 20) para aprender el modelo de transición; además, al elegir una política basada sólo en el modelo *estimado*, actúa *como si* el modelo fuera correcto. ¡Esto no es necesariamente una buena idea! Por ejemplo, un agente de taxi que no supiera cómo funcionan los semáforos podría ignorar una luz roja una o dos veces sin efectos perjudiciales y luego formular una política para ignorar todas las luces rojas a partir de entonces.

Una mejor idea sería elegir una política que funcione razonablemente bien para toda la gama de modelos que tienen una posibilidad razonable de ser el modelo verdadero, aun si la política resulta ser subóptima para el modelo de máxima verosimilitud. Hay tres enfoques matemáticos que tienen este enfoque.

Aprendizaje reforzado bayesiano

El primer enfoque, el **aprendizaje reforzado bayesiano**, asume una probabilidad previa $P(h)$ sobre las hipótesis h acerca de cuál es el modelo verdadero; la probabilidad posterior $P(h|\mathbf{e})$ se obtiene de la manera habitual mediante la regla de Bayes, dadas las observaciones hasta la fecha. Entonces, si el agente ha decidido dejar de aprender, la política óptima es la que da la mayor utilidad esperada. Dejemos que U_h^π sea la utilidad esperada, promediada sobre todos los posibles estados iniciales, obtenida al ejecutar la política π en el modelo h. Entonces tenemos

$$\pi^* = \underset{\pi}{\text{argmax}} \sum_h P(\text{h}|\mathbf{e}) U_h^\pi .$$

En algunos casos especiales, ¡esta política incluso puede ser calculada! Sin embargo, si el agente continuará aprendiendo en el futuro, encontrar una política óptima se vuelve mucho más difícil, porque

el agente debe considerar los efectos de las observaciones futuras en sus creencias sobre el modelo de transición. El problema se convierte en un **POMDP de exploración** cuyos estados de creencia son distribuciones sobre modelos. En principio, este POMDP de exploración permite ser formulado y resuelto antes de que el agente pise el mundo. (El Ejercicio 22.EPOM pide que haga esto para el juego de Buscaminas para encontrar el mejor primer movimiento). El resultado es una estrategia completa que le dice al agente qué hacer a continuación, dada cualquier secuencia de percepciones posible. Resolver el POMDP de exploración es por lo regular inmanejable, pero el concepto proporciona una base analítica para entender el problema de exploración descrito en la Sección 22.3.

Exploración POMDP

Vale la pena señalar que ser bayesiano por definición no protegerá al agente de una muerte inoportuna. A menos que el prior (conocimiento inicial) dé alguna indicación de percepciones que sugieran peligro, no hay nada que impida al agente tomar una acción exploratoria que conduzca a un estado absorbente. Por ejemplo, solía pensarse que los bebés humanos tenían miedo innato a las alturas y no se caerían de un acantilado, pero resulta que esto no es cierto (Adolph *et al.*, 2014).

El segundo enfoque, derivado de la **teoría de control robusto**, permite un conjunto de posibles modelos $\mathcal{H}$ sin asignarles probabilidades, y define una política robusta óptima como aquella que da el mejor resultado en el peor caso sobre $\mathcal{H}$:

Teoría de control robusto

$$\pi^* = \underset{\pi}{\text{argmax}} \; \underset{h}{min} \; U_h^\pi$$

Con frecuencia, el conjunto $\mathcal{H}$ será el conjunto de modelos que superan algún umbral de probabilidad en $P(h|\mathbf{e})$, por lo que los enfoques robustos y bayesianos están relacionados.

El enfoque de control robusto se considera como un juego entre el agente y un adversario, donde el adversario elige el peor resultado posible para cualquier acción, y la política que obtenemos es la solución minimax para el juego. Nuestro agente lógico wumpus (ver Sección 7.7) es un agente de control robusto de esta manera: considera todos los modelos que son lógicamente posibles, pero no explora ninguna ubicación que pueda contener un pozo o un wumpus, por lo que está encontrando la acción con la máxima utilidad en el peor caso sobre todas las hipótesis posibles.

El problema con la suposición del peor caso es que resulta en un comportamiento conservador. Un vehículo autónomo que asume que todos los demás conductores *intentarán colisionar con él* no tiene más remedio que quedarse en estacionado en casa. La vida real está llena de tales compensaciones de riesgo-recompensa.

Aunque una razón para aventurarse en el aprendizaje por refuerzo fue escapar de la necesidad de un profesor humano (como en el aprendizaje supervisado), resulta que el conocimiento humano ayuda a mantener un sistema seguro. Una forma es grabar una serie de acciones de un profesor experimentado, para que el sistema actúe de manera razonable desde el principio, y pueda aprender a mejorar desde allí. Una segunda forma es que un humano escriba restricciones sobre lo que un sistema permite hacer, y que un programa fuera del sistema de aprendizaje por refuerzo haga cumplir esas restricciones. Por ejemplo, al entrenar un helicóptero autónomo, es posible proporcionar una política parcial que tome el control cuando el helicóptero entra en un estado desde el cual cualquier acción insegura adicional llevaría a un estado irrecuperable, uno en el que el controlador de seguridad no garantice que se evitará el estado absorbente. En todos los demás estados, el agente de aprendizaje es libre de hacer lo que le plazca.

22.3.3 Aprendizaje Q de diferencias temporales

Ahora que tenemos un agente ADP activo, consideremos cómo construir un agente de aprendizaje de diferencias temporales (TD) activo. El cambio más obvio es que el agente tendrá que aprender un modelo de transición para que pueda elegir una acción basada en $U(s)$ a través de una anticipación de un paso. El problema de adquisición de modelo para el agente TD es idéntico al del agente ADP, y la regla de actualización TD permanece sin cambios. Una vez más, es posible

demostrar que el algoritmo TD convergerá a los mismos valores que ADP, a medida que el número de secuencias de entrenamiento tiende a infinito.

El método **Q-learning** evita la necesidad de un modelo aprendiendo una función de utilidad de acción $Q(s,a)$ en lugar de una función de utilidad $U(s)$. $Q(s,a)$ denota la recompensa total descontada esperada, si el agente toma la acción a en s y actúa de manera óptima a partir de entonces. Conocer la función Q permite al agente actuar de manera óptima eligiendo $\text{argmax}_a\, Q(s,a)$, sin necesidad de anticipación basada en un modelo de transición.

También podemos derivar una actualización TD sin modelo para los valores Q. Comenzamos con la ecuación de Bellman para $Q(s,a)$, repetida aquí de la Ecuación (17.8):

$$Q(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q(s',a')] \tag{22.6}$$

A partir de esto, podemos escribir la actualización TD de Q-learning, por analogía con la actualización TD para utilidades en la Ecuación (22.3):

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]. \tag{22.7}$$

Esta actualización se calcula siempre que se ejecuta la acción a en el estado s que conduce al estado s'. Como en la Ecuación (22.3), el término $R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)$ representa un error que la actualización está tratando de minimizar.

La parte importante de esta ecuación es lo que no contiene: *un agente de aprendizaje Q TD no necesita un modelo de transición, $P(s'|s,a)$, ni para el aprendizaje ni para la selección de acción*. Como se señaló al principio del capítulo, los métodos sin modelo se aplican aun en dominios muy complejos porque no se necesita proporcionar ni aprender ningún modelo. Por otro lado, el agente de aprendizaje Q no tiene medios para mirar hacia el futuro, por lo que puede tener dificultades cuando las recompensas son escasas y se deben construir secuencias de acción largas para alcanzarlas.

El diseño completo del agente para un agente de Q-learning TD exploratorio se muestra en la Figura 22.8. Observe que utiliza la misma función de exploración f que la utilizada por el agente ADP exploratorio, de ahí la necesidad de mantener estadísticas sobre las acciones tomadas (la tabla N). Si se utiliza una política de exploración más simple, digamos, actuando al azar en alguna fracción de los pasos, donde la fracción disminuye con el tiempo, entonces podemos prescindir de las estadísticas.

SARSA

Q-learning tiene un pariente cercano llamado **SARSA** (por estado, acción, recompensa, estado, acción). La regla de actualización para SARSA es muy similar a la regla de actualización de Q-learning (Ecuación (22.7)), excepto que SARSA actualiza con el valor Q de la acción a' que se toma realmente:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma Q(s',a') - Q(s,a)], \tag{22.8}$$

La regla se aplica al final de cada quíntuple s, a, r, s', a', de ahí el nombre. La diferencia con Q-learning es bastante sutil: mientras que Q-learning respalda el valor Q de la mejor acción en s', SARSA espera hasta que se toma una acción y respalda el valor Q para esa acción. Si el agente es codicioso y siempre toma la acción con el mejor valor Q, los dos algoritmos son idénticos. Sin embargo, cuando se está explorando, difieren: si la exploración produce una recompensa negativa, SARSA penaliza la acción, mientras que Q-learning no.

Fuera de política

Q-learning es un algoritmo de aprendizaje **fuera de política**, porque aprende los valores Q que responden a la pregunta "¿Cuánto valdría esta acción en este estado, suponiendo que dejo de usar la política que estoy usando ahora y comienzo a actuar de acuerdo con una política que elige la mejor acción (según mis estimaciones)?" SARSA es un algoritmo **en política**: aprende los valores Q que responden a la pregunta "¿Cuánto valdría esta acción en este estado, suponiendo

En política

function Q-LEARNING-AGENT(*percept*) **returns** una acción
  inputs: *percept,* un percepto que indica el estado actual s' y la señal de recompensa r
  persistent: Q, una tabla de valores de acción indexada por estado y acción, en inicio cero
    N_{sa}, una tabla de frecuencias para pares de estado-acción, inicialmente cero
    s,a, el estado y la acción anteriores, en inicio nulos

  if s no es nulo **then**
    incrementar $N_{sa}[s,a]$
    $Q[s,a] \leftarrow Q[s,a] + \alpha(N_{sa}[s,a])(r + \gamma \max_{a'} Q[s',a'] - Q[s,a])$
  $s,a \leftarrow s'$, $\text{argmax}_{a'} f(Q[s',a'],N_{sa}[s',a'])$
  return a

Figura 22.8 Agente de Q-learning exploratorio. Es un aprendiz activo que aprende el valor $Q(s,a)$ de cada acción en cada situación. Utiliza la misma función de exploración f que el agente ADP exploratorio, pero evita tener que aprender el modelo de transición.

que me quedo con mi ¿política?" Q-learning es más flexible que SARSA, en el sentido de que un agente de Q-learning permite aprender a comportarse bien bajo el control de una amplia variedad de políticas de exploración. Por otro lado, SARSA es apropiado si la política general está incluso controlada por otros agentes o programas, en cuyo caso es mejor aprender una función Q para lo que realmente sucederá en lugar de lo que sucedería si el agente eligiera las acciones estimadas como las mejores. Tanto Q-learning como SARSA aprenden la política óptima para el mundo 4×3, pero lo hacen a un ritmo mucho más lento que el agente ADP. Esto se debe a que las actualizaciones locales no hacen cumplir la consistencia entre todos los valores Q a través del modelo.

22.4 Generalización en aprendizaje por refuerzo

Hasta ahora, hemos asumido que las funciones de utilidad y las funciones Q se representan en forma tabular con un valor de salida para cada estado. Esto funciona para espacios de estado con hasta alrededor de 10^6 estados, que es más que suficiente para nuestros entornos de retícula bidimensionales de juguete. Pero en entornos del mundo real con muchos más estados, la convergencia será demasiado lenta. El backgammon es más simple que la mayoría de las aplicaciones del mundo real, pero tiene alrededor de 10^{20} estados. No podemos visitarlos todos para aprender a jugar el juego.

El Capítulo 5 introdujo la idea de una **función de evaluación** como una medida compacta de la deseabilidad para espacios de estado potencialmente vastos. En la terminología de este capítulo, la función de evaluación es una función de utilidad aproximada; utilizamos el término **aproximación de función** para el proceso de construir una aproximación compacta de la verdadera función de utilidad o función Q. Por ejemplo, podríamos aproximar la función de utilidad utilizando una combinación lineal ponderada de **características** $f_1,\ldots,f_n$:

Aproximación de función

$$\hat{U}_\theta(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \cdots + \theta_n f_n(s).$$

En lugar de aprender 10^{20} valores de estado en una tabla, un algoritmo de aprendizaje por refuerzo permite aprender, digamos, 20 valores para los parámetros $\theta = \theta_1,\ldots, \theta_{20}$ que hacen que $\hat{U}_\theta$ sea una buena aproximación a la verdadera función de utilidad. A veces, esta función de utilidad aproximada se combina con una búsqueda anticipada para producir decisiones más precisas. Agregar

búsqueda anticipada significa que el comportamiento efectivo puede generarse a partir de una aproximación de función de utilidad mucho más simple, que se aprende con muchas menos experiencias.

La aproximación de funciones hace práctico representar funciones de utilidad (o Q) para espacios de estado muy grandes, pero más importante aún, permite la generalización inductiva: el agente permite generalizar desde los estados que ha visitado hasta los que aún no ha visitado. Tesauro (1992) utilizó esta técnica para construir un programa de juego de backgammon que estaba a nivel de campeón humano, aunque sólo exploró una billonésima parte del espacio de estado completo de backgammon.

22.4.1 Aproximando la estimación de utilidad directa

El método de estimación de utilidad directa (Sección 22.2) genera trayectorias en el espacio de estados y extrae, para cada estado, la suma de recompensas recibidas desde ese estado en adelante hasta la terminación. El estado y la suma de recompensas recibidas constituyen un ejemplo de entrenamiento para un algoritmo de **aprendizaje supervisado**. Por ejemplo, suponga que representamos las utilidades para el mundo 4 × 3 utilizando una función lineal simple, donde las características de las casillas son sólo sus coordenadas x e y. En ese caso, tenemos

$$\hat{U}_\theta(x,y) = \theta_0 + \theta_1 x + \theta_2 y. \tag{22.9}$$

Por lo tanto, si $(\theta_0, \theta_1, \theta_2) = (0.5, 0.2, 0.1)$, entonces $\hat{U}_\theta(1,1) = 0.8$. Dada una colección de pruebas, obtenemos un conjunto de valores de muestra de $\hat{U}_\theta(x,y)$, y podemos encontrar el mejor ajuste, en el sentido de minimizar el error cuadrado, utilizando la regresión lineal estándar (ver Capítulo 19).

Para el aprendizaje por refuerzo, tiene más sentido usar un algoritmo de aprendizaje en línea que actualice los parámetros después de cada prueba. Suponga que realizamos una prueba y la recompensa total obtenida comenzando en (1,1) es 0.4. Esto sugiere que $\hat{U}_\theta(1,1)$, actualmente 0.8, es demasiado grande y debe ser reducido. ¿Cómo se deben ajustar los parámetros para lograr esto? Como con redes neuronales de aprendizaje, escribimos una función de error y calculamos su gradiente con respecto a los parámetros. Si $u_j(s)$ es la recompensa total observada desde el estado s en adelante en el jth ensayo, entonces el error se define como (la mitad) la diferencia cuadrada del total predicho y el total real: $E_j(s) = (\hat{U}_\theta(s) - u_j(s))^2/2$. La tasa de cambio del error con respecto a cada parámetro θ_i es $\partial E_j/\partial\theta_i$, por lo que para mover el parámetro en la dirección de disminuir el error, queremos

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial E_j(s)}{\partial \theta_i} = \theta_i + \alpha[u_j(s) - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}. \tag{22.10}$$

Regla de Widrow–Hoff
Regla delta

Esto se llama la **regla de Widrow–Hoff**, o la **regla delta**, para mínimos cuadrados en línea. Para el aproximador de función lineal $\hat{U}_\theta(s)$ en la Ecuación (22.9), obtenemos tres reglas de actualización simples:

$$\theta_0 \leftarrow \theta_0 + \alpha[u_j(s) - \hat{U}_\theta(s)],$$
$$\theta_1 \leftarrow \theta_1 + \alpha[u_j(s) - \hat{U}_\theta(s)]x,$$
$$\theta_2 \leftarrow \theta_2 + \alpha[u_j(s) - \hat{U}_\theta(s)]y.$$

Podemos aplicar estas reglas al ejemplo donde $\hat{U}_\theta(1, 1)$ es 0.8 y $u_j(1,1)$ es 0.4. Los parámetros θ_0, θ_1, y θ_2 son todos disminuidos por 0.4α, lo que reduce el error para (1,1). Observe que ¡*cambiar los parámetros θ_i en respuesta a una transición observada entre dos estados también cambia los valores de $\hat{U}_\theta$ para todos los demás estados!* Esto es lo que queremos decir cuando decimos que la función la aproximación permite a un aprendiz de refuerzo generalizar a partir de sus experiencias.

El agente aprenderá más rápido si utiliza una aproximación de función, siempre que el espacio de hipótesis no sea demasiado grande e incluya algunas funciones que se ajusten razonablemente bien a la verdadera función de utilidad. El Ejercicio 22.APLM le pide que evalúe el rendimiento

de la utilidad directa estimación, tanto como con y sin aproximación de función. La mejora en el mundo 4 × 3 es notable pero no dramática, porque este es un espacio de estado muy pequeño para empezar. La mejora es mucho mayor en un mundo 10 × 10 con una recompensa de +1 en (10,10).

El mundo 10 × 10 se adecua bien para una función de utilidad lineal porque la verdadera utilidad función es suave y casi lineal: es de manera básica una pendiente diagonal con su esquina inferior en (1,1) y su esquina superior en (10,10). (Ver Ejercicio 22.TENX). Por otro lado, si ponemos el +1 recompensa en (5,5), la verdadera utilidad es más como una pirámide y el aproximador de función en la Ecuación (22.9) fallará.

¡Sin embargo, no todo está perdido! Recuerde que lo que importa para la aproximación de la función lineal es que la función sea lineal en las características. Pero podemos elegir las características para ser arbitrarias funciones no lineales de las variables de estado. Por lo tanto, podemos incluir una característica como $f_3(x,y) = \sqrt{(x - x_g)^2 + (y - y_g)^2}$ que mide la distancia al objetivo. Con esta nueva característica, el aproximador de función lineal funciona bien.

22.4.2 Aproximando el aprendizaje de diferencias temporales

Podemos aplicar estas ideas igual de bien a los aprendices de diferencias temporales. Todo lo que necesitamos hacer es ajustar los parámetros para tratar de reducir la diferencia temporal entre estados sucesivos. Las nuevas versiones de las ecuaciones de TD y Q-learning (22.3 en la página 795 y 22.7 en la página 802) se definen como

$$\theta_i \leftarrow \theta_i + \alpha[R(s,a,s') + \gamma \hat{U}_\theta(s') - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i} \tag{22.11}$$

para las utilidades, y

$$\theta_i \leftarrow \theta_i + \alpha[R(s,a,s') + \gamma \max_{a'} \hat{Q}_\theta(s'a') - \hat{Q}_\theta(s,a)] \frac{\partial \hat{Q}_\theta(s,a)}{\partial \theta_i} \tag{22.12}$$

para Q-valores. Para el aprendizaje pasivo de TD, es posible demostrar que la regla de actualización converja a la aproximación más cercana posible a la verdadera función cuando el aproximador de función es *lineal* en las características.[4] Con el aprendizaje activo y funciones *no lineales* como las redes neuronales, casi todas las apuestas están fuera: hay algunos casos muy simples en los que los parámetros pueden irse al infinito con estas reglas de actualización, aunque hay buenas soluciones en el espacio de hipótesis. Hay algoritmos más sofisticados que evitan estos problemas, pero en la actualidad el aprendizaje por refuerzo con aproximadores de función generales sigue siendo un arte delicado.

Olvido catastrófico

Además de los parámetros que divergen hacia el infinito, hay un problema más sorprendente llamado **olvido catastrófico**. Suponga que está entrenando un vehículo autónomo para conducir a lo largo de carreteras (simuladas) de manera segura, sin chocar. Asigna una alta recompensa negativa por cruzar el borde de la carretera, y usa características cuadráticas de la posición de la carretera para que el vehículo pueda aprender que la utilidad de estar en el medio de la carretera es mayor que estar cerca del borde. Todo va bien y el auto aprende a conducir de manera perfecta por el medio de la carretera. Después de unos minutos, empieza a aburrirse y está a punto de detener la simulación y escribir los excelentes resultados. De repente, el vehículo se desvía de la carretera y se estrella. ¿Por qué? Lo que ha sucedido es que el coche ha aprendido *demasiado bien*: porque aprendió a alejarse del borde y que toda la región central de la carretera es un lugar seguro para estar, pero olvidó que la región más cercana al borde es peligrosa. La región central, por lo tanto, tiene una función de valor plana, por lo que las características cuadráticas obtienen un peso cero;

[4] La definición de distancia entre funciones de utilidad es bastante técnica; ver Tsitsiklis y Van Roy (1997).

luego, cualquier peso distinto de cero en las características lineales hace que el coche se deslice fuera de la carretera hacia un lado u otro.

Repetición de experiencia

Una solución a este problema, llamada **repetición de experiencia**, asegura que el coche sigue reviviendo su comportamiento juvenil de choque a intervalos regulares. El algoritmo de aprendizaje permite retener trayectorias de todo el proceso de aprendizaje y reproducir esas trayectorias para asegurar que su función de valor siga siendo precisa para partes del espacio de estado que ya no visita.

Para los sistemas de aprendizaje por refuerzo basados en modelos, la aproximación de función también es útil para aprender un modelo del entorno. Recuerde que aprender un modelo para un entorno *observable* es un problema de aprendizaje *supervisado*, porque el siguiente percepto da el estado de resultado. Cualquiera de los métodos de aprendizaje supervisado en los Capítulos 19-21 puede ser utilizado, con los ajustes adecuados, para el hecho de que necesitamos predecir una descripción de estado completa en lugar de sólo una clasificación booleana o un solo valor real. Con un modelo aprendido, el agente permite hacer una búsqueda de anticipación para mejorar sus decisiones y puede llevar a cabo simulaciones internas para mejorar sus representaciones aproximadas de U o Q en lugar de requerir experiencias reales lentas y potencialmente costosas.

Para un entorno *parcialmente observable*, el problema de aprendizaje es mucho más difícil porque el siguiente percepto ya no es una etiqueta para el problema de predicción de estado. Si sabemos cuáles son las variables ocultas y cómo están relacionadas causalmente entre sí y con las variables observables, entonces podemos fijar la estructura de una red bayesiana dinámica y usar el algoritmo EM para aprender los parámetros, como se describió en el Capítulo 20. Aprender la estructura interna de las redes bayesianas dinámicas y crear nuevas variables de estado todavía se considera un problema difícil. Las redes neuronales profundas recurrentes (Sección 21.6) han tenido éxito en algunos casos al inventar la estructura oculta.

22.4.3 Aprendizaje por refuerzo profundo

Hay dos razones por las que necesitamos ir más allá de los aproximadores de función lineal: primero, es posible que no haya una buena función lineal que se acerque a aproximar la función de utilidad o la función Q; segundo, siempre es *posible* que no podamos inventar las características necesarias, en especial en nuevos dominios. Si lo piensa, estas son en realidad la misma razón: siempre es posible representar U o Q como combinaciones lineales de características, en especial si tenemos características como $f_1(s) = U(s)$ o $f_2(s,a) = Q(s,a)$, pero a menos que podamos encontrar tales características (en una forma computable de modo eficiente) el aproximador de función lineal puede ser insuficiente.

Por estas razones (o razón), los investigadores han explorado aproximadores de función más complejos y no lineales desde los primeros días del aprendizaje por refuerzo. Actualmente, las redes neuronales profundas (Capítulo 21) son muy populares en este papel y han demostrado ser efectivas incluso cuando la entrada es una imagen cruda sin ninguna extracción de características diseñada por humanos. Si todo va bien, la red neuronal profunda descubre en efecto las características útiles por sí misma. Y si la capa final de la red es lineal, entonces podemos ver qué características está utilizando la red para construir su propio aproximador de función lineal. Un sistema de aprendizaje por refuerzo que utiliza una red profunda como aproximador de función se llama sistema de aprendizaje por refuerzo profundo.

Al igual que en la Ecuación (22.9), la red profunda es una función parametrizada por θ, excepto que ahora la función es mucho más complicada. Los parámetros son todos los pesos en todas las capas de la red. Sin embargo, los gradientes requeridos para las Ecuaciones (22.11) y (22.12) son los mismos que los gradientes requeridos para el aprendizaje supervisado, y son calculados por el mismo proceso de retropropagación descrito en la Sección 21.4.

Como explicamos en la Sección 22.7, el RL profundo ha logrado resultados significativos, incluido aprender a jugar una amplia gama de videojuegos a un nivel experto, derrotar al campeón mundial humano en Go y entrenar robots para realizar tareas complejas.

A pesar de sus impresionantes éxitos, el RL profundo aún enfrenta obstáculos significativos: con frecuencia es difícil lograr un buen rendimiento y el sistema entrenado puede comportarse de manera muy impredecible si el entorno difiere incluso un poco de los datos de entrenamiento. En comparación con otras aplicaciones de aprendizaje profundo, el RL profundo rara vez se aplica en entornos comerciales. Sin embargo, es un área de investigación muy activa.

22.4.4 Formación de recompensas

Como se señaló en la introducción a este capítulo, los entornos del mundo real pueden tener recompensas muy escasas: se requieren muchas acciones primitivas para lograr una recompensa distinta de cero. Por ejemplo, un robot jugador de futbol soccer podría enviar cien mil comandos de control motor a sus diversas articulaciones antes de conceder un gol. Ahora tiene que averiguar qué hizo mal. El término técnico para esto es el problema de **asignación de crédito**. Aparte de jugar billones de juegos de futbol para la asignación de crédito que la recompensa negativa eventualmente se propaga de vuelta a las acciones responsables de ella, ¿existe una buena solución?

Asignación de crédito

Un método común, originalmente utilizado en el entrenamiento de animales, se llama **formación de recompensas**. Esto implica proporcionar al agente recompensas adicionales, llamadas **seudorrecompensas**, por "hacer progresos". Por ejemplo, podríamos dar seudorrecompensas al robot por hacer contacto con el balón o por avanzarlo a la meta. Tales recompensas permiten acelerar el aprendizaje y son simples de proporcionar, pero existe el riesgo de que el agente aprenda a maximizar las seudorrecompensas en lugar de las verdaderas recompensas; como, estar al lado del balón y "vibrar" causa muchos contactos con el balón.

Formación de recompensas

Seudorrecompensas

En el Capítulo 17 (página 569), vimos una forma de modificar la función de recompensa sin cambiar la política óptima. Para cualquier función potencial $\Phi(s)$ y cualquier función de recompensa R, podemos crear una nueva función de recompensa R' de la siguiente manera:

$$R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s).$$

La función potencial Φ se construye para reflejar cualquier aspecto deseable del estado, como el logro de subobjetivos o la distancia a un estado terminal deseado. Por ejemplo, Φ para el robot jugador de futbol soccer podría añadir un bono constante para los estados en los que el equipo del robot tiene posesión y otro bono para reducir la distancia del balón desde la meta de los oponentes. Esto resultará en un aprendizaje más rápido en general, pero no evitará que el robot, por ejemplo, aprenda a dar un pase hacia atrás al portero cuando hay amenaza de peligro.

22.4.5 Aprendizaje por refuerzo jerárquico

Otra forma de lidiar con secuencias de acción muy largas es dividirlas en unas pocas piezas más pequeñas, y luego dividir esas en piezas aún más pequeñas, y así de manera sucesiva, hasta que las secuencias de acción sean lo suficientemente cortas para facilitar el aprendizaje. Este enfoque se llama **aprendizaje por refuerzo jerárquico** (HRL), y tiene mucho en común con los métodos de **planificación HTN** descritos en el Capítulo 11. Por ejemplo, marcar un gol en el futbol puede desglosarse en las siguientes acciones: obtener posesión, pasar a un compañero de equipo, recibir el balón de un compañero de equipo, driblar hacia la meta y disparar; cada uno de estas acciones permite ser desglosada aún más en comportamientos motores de nivel inferior. Note que existen múltiples formas de obtener posesión y disparar, múltiples compañeros de equipo a los que se

Aprendizaje por refuerzo jerárquico

podría pasar, etc., por lo que cada acción de nivel superior permite tener muchas implementaciones de nivel inferior diferentes.

Mantenerse alejado

Para ilustrar estas ideas, utilizaremos un juego de futbol simplificado llamado **keepaway** (**mantenerse alejado**), en el que un equipo de tres jugadores intenta mantener la posesión del balón, driblando y pasando entre ellos, el mayor tiempo posible, mientras que el otro equipo de dos jugadores intenta tomar posesión interceptando un pase o derribando a un jugador en posesión.[5] El juego se implementa dentro del simulador RoboCup 2D, que da modelos de movimiento de estado continuo detallados, con pasos de tiempo de 100 ms, y que ha demostrado ser buen banco de pruebas para sistemas RL.

Programa parcial

Un agente de aprendizaje por refuerzo jerárquico comienza con un **programa parcial** que esboza una estructura jerárquica para el comportamiento del agente. El lenguaje de programación parcial para programas de agentes extiende cualquier lenguaje de programación ordinario al añadir primitivas para elecciones no especificadas que deben ser llenadas por el aprendizaje. (Aquí, usamos seudocódigo para el lenguaje de programación). El programa parcial puede ser complicado, siempre y cuando termine.

Es fácil ver que HRL incluye RL ordinario como un caso especial. Sólo proporcionamos el programa parcial trivial que permite al agente seguir eligiendo cualquier acción de $A(s)$, el conjunto de acciones que se ejecutan en el estado actual s:

```
while true do
    choose(A(s)).
```

El operador de **elección** (**choose**) permite al agente elegir cualquier elemento del conjunto especificado. El proceso de aprendizaje convierte el programa parcial del agente en un programa completo al aprender cómo se debe hacer cada elección. Por ejemplo, el proceso de aprendizaje podría asociar una función Q con cada elección; una vez que se aprenden las funciones Q, el programa produce comportamiento eligiendo la opción con el valor Q más alto cada vez que se encuentra con una elección.

Los programas de agentes para keepaway son más interesantes. Veremos el programa parcial para un solo jugador en el equipo "portero". La elección de qué hacer en el nivel superior depende de si el jugador tiene el balón o no:

```
while not Is-Terminal(s) do
    if Ball-In-My-Possession(s) then choose({Pass,Hold,Dribble})
    else choose({Stay,Move,Intercept-Ball}).
```

Cada una de estas elecciones invoca una subrutina que permite hacer más elecciones, hasta llegar a acciones primitivas que se pueden ejecutar de manera directa. Por ejemplo, la acción de alto nivel Pass elige a un compañero de equipo para pasar, pero también tiene la opción de no hacer nada y devolver el control al nivel superior, si es apropiado (por ejemplo, si no hay a quien pasar):

```
choose({Pass-To(choose(TeamMates(s))),return}).
```

La rutina Pass-To entonces tiene que elegir una velocidad y dirección para el pase. Aunque es relativamente fácil para un humano, aun para uno con poca experiencia en futbol, proporcionar este tipo de consejo de alto nivel al agente de aprendizaje, sería difícil, si no imposible, escribir las reglas para determinar la velocidad y dirección de la patada para maximizar la probabilidad de mantener la posesión. De manera similar, está lejos de ser obvio cómo elegir al compañero de equipo correcto para recibir el balón o dónde moverse para hacerse disponible para recibir el balón. El programa parcial proporciona conocimientos generales, andamiaje general y organización estructural para comportamientos complejos, y el proceso de aprendizaje trabaja todos los detalles.

[5] Los rumores de que keepaway se inspiró en las tácticas del mundo real del FC Barcelona probablemente no tienen fundamento.

Los fundamentos teóricos de HRL se basan en el concepto del **espacio de estado conjunto**, en el espacio de estado conjunto en el que cada estado (s,m) está compuesto por un estado físico s y un estado de máquina m. El estado de la máquina está definido por el estado interno actual del programa del agente: el contador de programa para cada subrutina en la pila de llamadas actual, los valores de los argumentos y los valores de todas las variables locales y globales. Por ejemplo, si el programa del agente ha elegido pasar al compañero de equipo Ali y está en medio de calcular la velocidad del pase, entonces el hecho de que Ali es el argumento de PASS-TO es parte del estado de la máquina actual. Un **estado de elección** $\sigma = (s,m)$ es un estado de elección en el que el contador de programa para m está en un punto de elección en el programa del agente. Entre dos estados de elección, pueden ocurrir cualquier número de transiciones computacionales y acciones físicas, pero todas están predestinadas, por así decirlo: por definición, el agente no está tomando ninguna decisión entre estados de elección. En esencia, el agente RL jerárquico está resolviendo un problema de decisión markoviano con los siguientes elementos:

Espacio de estado conjunto

Estado de elección

- Los estados son los estados de elección σ del espacio de estados conjuntos.
- Las acciones en σ son las elecciones c disponibles en σ de acuerdo con el programa parcial.
- La función de recompensa $\rho(\sigma,c,\sigma')$ es la suma esperada de recompensas para todas las transiciones físicas que ocurren entre los estados de elección σ y σ'.
- El modelo de transición $\tau(\sigma,c,\sigma')$ se define de la manera obvia: si c invoca una acción física a, entonces τ toma prestado del modelo físico $P(s'|s,a)$; si c invoca una transición computacional, como llamar a una subrutina, entonces la transición modifica de manera determinista el estado computacional m de acuerdo con las reglas del lenguaje de programación.[6]

Al resolver este problema de decisión, el agente encuentra la política óptima que es consistente con el programa parcial original.

El RL jerárquico es un método muy efectivo para aprender comportamientos complejos. En keepaway, un agente HRL basado en el programa parcial antes esbozado aprende una política que mantiene la posesión para siempre contra la política estándar de *taker*, una mejora significativa sobre el récord anterior de unos 10 segundos. Una característica importante es que las habilidades de nivel inferior no son subrutinas fijas en el sentido habitual; sus elecciones son sensibles al estado interno completo del programa del agente, por lo que se comportan de manera diferente dependiendo de dónde se invocan dentro de ese programa y qué está sucediendo en ese momento. Si es necesario, las funciones Q para las elecciones de nivel inferior permiten ser inicializadas por un proceso de entrenamiento separado con su propia función de recompensa, y luego integradas en el sistema general para que puedan ser adaptadas para funcionar bien en el contexto del agente completo.

En la sección anterior vimos que las recompensas de conformación son útiles para aprender comportamientos complejos. En HRL, el hecho de que el aprendizaje se lleve a cabo en el espacio de estados conjuntos proporciona oportunidades adicionales para la conformación. Por ejemplo, para ayudar con el aprendizaje de la función Q para un pase preciso dentro de la rutina PASS-TO, es posible proporcionar una recompensa de conformación que dependa de la ubicación del destinatario previsto y de la proximidad de los oponentes a ese jugador: la pelota debe estar cerca del destinatario, pero lejos de los oponentes. Eso parece obvio, pero *la identidad del destinatario previsto para un pase no*

[6] Debido a que es posible ejecutar más de una acción física antes de que se alcance el siguiente estado de elección, el problema es técnicamente un proceso de decisión semi-Markov, que permite que las acciones tengan diferentes duraciones, incluidas duraciones estocásticas. Si el factor de descuento $\gamma < 1$, entonces la duración de la acción afecta el descuento aplicado a la recompensa obtenida durante la acción, lo que significa que se tiene que hacer un poco de contabilidad de descuento extra y la transición el modelo incluye la distribución de duración.

es parte del estado físico del mundo. El estado físico consiste sólo en las posiciones, orientaciones y velocidades de los jugadores y la pelota. No hay "pase" y no hay "receptor" en el mundo físico; estos son constructos internos. Esto significa que no hay forma de proporcionar tal consejo sensato a un sistema RL estándar.

Descomposición aditiva

La estructura jerárquica del comportamiento también proporciona una **descomposición aditiva** natural de la función de utilidad general. Recuerde que la utilidad es la suma de recompensas a lo largo del tiempo, así que considere una secuencia de, digamos, diez pasos de tiempo con recompensas $[r_1, r_2,...,r_{10}]$. Suponga que para los primeros cinco pasos de tiempo el agente está haciendo PASS-TO(*Ali*) y durante los cinco pasos restantes está haciendo MOVE-INTO-SPACE. Entonces la utilidad para el estado inicial es la suma de la recompensa total durante PASS-TO y la recompensa total durante MOVE-INTO-SPACE. El primero depende sólo de si la pelota llega a Ali con suficiente tiempo y espacio para que Ali retenga la posesión, y el último depende sólo de si el agente llega a un buen lugar para recibir la pelota. En otras palabras, la utilidad general se descompone en múltiples términos, cada uno de los cuales depende sólo de unas pocas variables. Esto, a su vez, significa que el aprendizaje ocurre mucho más rápido que si intentamos aprender una sola función de utilidad que depende de todas las variables. Esto es algo análogo a los teoremas de representación que subyacen a la concisión de las redes de Bayes (Capítulo 13).

22.5 Búsqueda de políticas

Búsqueda de políticas

El último enfoque que consideraremos para los problemas de aprendizaje por refuerzo se llama **búsqueda de políticas**. En cierto modo, la búsqueda de políticas es el método más simple de todos en este capítulo: la idea es seguir ajustando la política mientras su rendimiento mejora, luego detenerse.

Comencemos con las políticas mismas. Recuerde que una política π es una función que mapea estados a acciones. Estamos interesados en representaciones *parametrizadas* de π que tienen muchos menos parámetros de los que hay estados en el espacio de estados (como en la sección anterior). Por ejemplo, podríamos representar π por una colección de funciones Q parametrizadas, una para cada acción, y tomar la acción con el valor predicho más alto:

$$\pi(s) = \underset{a}{\text{argmax}}\ \hat{Q}_\theta(s,a). \tag{22.13}$$

Cada función Q podría ser una función lineal, como en la Ecuación (22.9), o podría ser una función no lineal como una red neuronal profunda. La búsqueda de políticas ajustará entonces los parámetros θ para mejorar la política. ¡Observe que si la política está representada por funciones Q, entonces la búsqueda de políticas resulta en un proceso que aprende funciones Q. *¡Este proceso no es lo mismo que el aprendizaje Q!*

En el aprendizaje Q con aproximación de función, el algoritmo encuentra un valor de θ tal que $\hat{Q}_\theta$ está "cerca" de Q^*, la función Q óptima. Además, la búsqueda de políticas encuentra un valor de θ que resulta en un buen rendimiento; los valores encontrados por ambos métodos pueden diferir de manera sustancial. (Así, la función Q aproximada definida por $\hat{Q}_\theta(s,a) = Q^*(s,a)/100$ da un rendimiento óptimo, aunque no se acerque en absoluto a Q^*). Otro claro ejemplo de la diferencia es el caso donde $\pi(s)$ se calcula usando, digamos, una búsqueda de anticipación de profundidad-10 con una función de utilidad aproximada $\hat{U}_\theta$. Un valor de θ que da buenos resultados podría estar muy lejos de hacer que $\hat{U}_\theta$ se parezca a la verdadera función de utilidad.

Un problema con las representaciones de políticas del tipo dado en la Ecuación (22.13) es que la política es una función *discontinua* de los parámetros cuando las acciones son discretas. Es decir, habrá valores de θ tal que un cambio infinitesimal en θ hace que la política cambie de una acción a otra. Esto significa que el valor de la política también puede cambiar de manera

discontinua, lo que hace difícil la búsqueda basada en gradientes. Por esta razón, los métodos de búsqueda de políticas con frecuencia utilizan una representación de **política estocástica** $\pi_\theta(s,a)$, que especifica la probabilidad de la política estocástica de seleccionar la acción a en el estado s. Una representación popular es la función **softmax**:

Política estocástica

$$\pi_\theta(s,a) = \frac{e^{\beta \hat{Q}_\theta(s,a)}}{\sum_{a'} e^{\beta \hat{Q}_\theta(s,a')}}. \tag{22.14}$$

El parámetro $\beta > 0$ modula la suavidad del softmax: para valores de β que son grandes en comparación con las separaciones entre los valores Q, el softmax se acerca a un max duro, mientras que para valores de β cercanos a cero, el softmax se acerca a una elección aleatoria uniforme entre las acciones. Para todos los valores finitos de β, el softmax proporciona una función diferenciable de θ; por lo tanto, el valor de la política (que depende continuamente de las probabilidades de selección de acción) es una función diferenciable de θ.

Ahora veamos métodos para mejorar la política. Comenzamos con el caso más simple: una política y un entorno deterministas. Definamos $\rho(\theta)$ como el **valor de la política**, es decir, el valor esperado de la recompensa cuando se ejecuta π_θ. Si podemos derivar una expresión para $\rho(\theta)$ en forma cerrada, entonces tenemos un problema de optimización estándar, como se describe en el Capítulo 4. Podemos seguir el vector de **gradiente de política** $\nabla_{\theta\rho}(\theta)$, siempre que $\rho(\theta)$ sea diferenciable. Como alternativa, si el gradiente de la política $\rho(\theta)$ no está disponible en forma cerrada, podemos evaluar π_θ ejecutándolo y observando la recompensa acumulada. Podemos seguir el gradiente empírico ascendiendo colinas, es decir, evaluando el cambio en el valor de la política para pequeños incrementos en cada parámetro. Con las advertencias habituales, este proceso convergerá a un óptimo local en el espacio de políticas.

Valor de la política

Gradiente de política

Cuando el entorno (o la política) es no determinista, las cosas se vuelven más difíciles. Suponga que estamos tratando de ascender colinas, que requiere comparar $\rho(\theta)$ y $\rho(\theta + \Delta\theta)$ para algún pequeño $\Delta\theta$. El problema es que la recompensa total para cada prueba puede variar, por lo que las estimaciones del valor de la política a partir de un pequeño número de pruebas serán muy poco fiables; intentar comparar dos de estas estimaciones será aún menos fiable. Una solución es realizar muchas pruebas, midiendo la varianza de la muestra y usándola para determinar que se han realizado suficientes pruebas para obtener una indicación confiable de la dirección de mejora para $\rho(\theta)$. Sin embargo, esto es impráctico para muchos problemas reales en los que las pruebas son costosas, consumir mucho tiempo e incluso ser peligrosas.

Para el caso de una política no determinista $\pi_\theta(s,a)$, es posible obtener una estimación imparcial del gradiente en θ, $\nabla_{\theta\rho}(\theta)$, directamente de los resultados de las pruebas ejecutadas en θ. Para simplificar, derivaremos esta estimación para el caso simple de un entorno episódico en el que cada acción a obtiene una recompensa $R(s_0,a,s_0)$ y el entorno se reinicia en s_0. En este caso, el valor de la política es el valor esperado de la recompensa, y tenemos

$$\nabla_\theta \rho(\theta) = \nabla_\theta \sum_a R(s_0,a,s_0)\pi_\theta(s_0,a) = \sum_a R(s_0,a,s_0)\nabla_\theta \pi_\theta(s_0,a).$$

Ahora realizamos un truco simple para que esta suma pueda ser aproximada por muestras generadas a partir de la distribución de probabilidad definida por $\pi_\theta(s_0,a)$. Suponga que tenemos N pruebas en total, y la acción tomada en la prueba jth es a_j. Entonces

$$\begin{aligned}\nabla_\theta \rho(\theta) &= \sum_a \pi_\theta(s_0,a).\frac{R(s_0,a,s_0)\nabla_\theta \pi_\theta(s_0,a)}{\pi_\theta(s_0,a)} \\ &= \approx \frac{1}{N}\sum_{j=1}^{N} \frac{R(s_0,a_j,s_0)\nabla_\theta \pi_\theta(s_0,a_j)}{\pi_\theta(s_0,a_j)}.\end{aligned}$$

Por lo tanto, el valor real del gradiente de la política se aproxima por una suma de términos que involucran el gradiente de la probabilidad de selección de acción en cada prueba. Para el caso secuencial, esto se generaliza a

$$\nabla_\theta \rho(\theta) = \approx \frac{1}{N} \sum_{j=1}^{N} \frac{u_j(s) \nabla_\theta \pi_\theta(s,a_j)}{\pi_\theta(s,a_j)}$$

para cada estado s visitado, donde a_j se ejecuta en s en la prueba jth y $u_j(s)$ es la recompensa total recibida desde el estado s en adelante en la prueba jth. El algoritmo resultante, llamado REINFORCE, debido a Ron Williams (1992), es en general mucho más efectivo que el ascenso de colinas usando muchas pruebas en cada valor de θ. Sin embargo, todavía es mucho más lento de lo necesario.

Considere la siguiente tarea: dadas dos políticas de blackjack, determine cuál es la mejor. Las políticas podrían tener verdaderos retornos netos por mano de, digamos, -0.21% y +0.06%, por lo que averiguar cuál es mejor es muy importante. Una forma de lograr esto es hacer que cada política juegue contra un "dealer" (crupier) estándar para un cierto número de manos y luego medir sus respectivas ganancias. El problema con esto, como hemos visto, es que las ganancias de cada política fluctúan de manera agresiva dependiendo de si recibe buenas o malas cartas. Se necesitarían varios millones de manos (partidas) para tener una indicación confiable de qué política es mejor. El mismo problema surge al usar muestreo aleatorio para comparar dos políticas adyacentes en un algoritmo de ascenso de colinas.

Una mejor solución para el blackjack es generar un cierto número de manos por adelantado y *hacer que cada programa juegue el mismo conjunto de manos*. De esta manera, eliminamos el error de medición debido a las diferencias en las cartas recibidas. Sólo se necesitan unas pocas miles de partidas para determinar cuál de las dos políticas de blackjack es mejor.

Muestreo correlacionado

Esta idea, llamada **muestreo correlacionado**, se puede aplicar a la búsqueda de políticas en general, dado un simulador de entorno en el que se pueden repetir las secuencias de números aleatorios. Se implementó en un algoritmo de búsqueda de políticas llamado PEGASUS (Ng y Jordan, 2000), uno de los primeros algoritmos en lograr un vuelo de helicóptero autónomo completamente estable (ver Figura 22.9(b)). Es posible demostrar que el número de secuencias aleatorias necesarias para asegurar que el valor de *cada* política está bien estimado, dependiendo sólo de la complejidad del espacio de políticas pero no, en absoluto, de la complejidad del dominio subyacente.

22.6 Aprendizaje por observación y por refuerzo inverso

Algunos dominios son tan complejos que es difícil definir una función de recompensa para su uso en el aprendizaje por refuerzo. ¿Qué queremos con exactitud que haga nuestro vehículo autónomo? Ciertamente no debería tardar demasiado en llegar al destino, pero no debería conducir tan rápido como para incurrir en un riesgo indebido o para recibir multas por exceso de velocidad. Debería conservar combustible/energía. Debería evitar zarandear o acelerar demasiado a los pasajeros, pero puede frenar de manera brusca en una emergencia, etc. Decidir cuánto peso dar a cada uno de estos factores es una tarea difícil. Peor aún y casi con seguridad, hemos olvidado factores importantes como la obligación de comportarse con consideración hacia otros conductores. Omitir un factor por lo regular conduce a un comportamiento que asigna un valor extremo al factor omitido, en este caso, una conducción poco considerada, para maximizar los factores restantes.

Un enfoque es hacer pruebas exhaustivas en simulación, notar comportamientos problemáticos e intentar modificar la función de recompensa para eliminar esos comportamientos. Otro enfoque es buscar fuentes adicionales de información sobre la función de recompensa apropiada.

Una fuente tal es el comportamiento de los agentes que ya están optimizando (o, digamos, casi optimizando) esa función de recompensa, en este caso, los conductores humanos expertos.

Aprendizaje por observación

El campo general del **aprendizaje por observación** estudia el proceso de aprender cómo comportarse con base en las observaciones dadas del comportamiento experto. Mostramos al algoritmo ejemplos de conducción experta y le decimos que "lo haga así". Hay (al menos) dos formas de abordar el problema del aprendizaje por observación. La primera es la que discutimos de manera breve al principio del capítulo: asumiendo que el entorno es observable, aplicamos el aprendizaje supervisado al par de estado-acción observados para aprender una política $\pi(s)$. Esto se llama **aprendizaje por imitación**. Ha tenido cierto éxito en robótica (ver página 966) pero sufre del problema de la fragilidad: aun pequeñas desviaciones del conjunto de entrenamiento conducen a errores que crecen con el tiempo y, finalmente, al fracaso. Además, el aprendizaje por imitación duplicará como máximo el rendimiento del profesor, no lo superará. Cuando los humanos aprendemos por imitación, a veces usamos la frase peyorativa "hacerle al mono" para describir lo que estamos haciendo. (Es muy posible que los simios usen el término "humanizar" entre ellos, quizás en un sentido aún más peyorativo). La implicación es que el aprendiz de imitación no entiende *por qué* debería realizar ninguna acción dada.

Aprendizaje por imitación

El segundo enfoque para el aprendizaje por observación es entender *por qué*: observar las acciones del experto (y los estados resultantes) e intentar averiguar qué función de recompensa está maximizando el experto. Entonces podríamos derivar una política óptima con respecto a esa función de recompensa. Se espera que este enfoque produzca políticas robustas a partir de relativamente pocos ejemplos de comportamiento experto; después de todo, el campo del aprendizaje por refuerzo se basa en la idea de que la función de recompensa, en lugar de la política o la función de valor, es la definición más sucinta, robusta y transferible de la tarea. Además, si el aprendiz hace las concesiones apropiadas para la posible suboptimalidad por parte del experto, entonces es posible hacerlo mejor que el experto optimizando una aproximación precisa a la verdadera función de recompensa. Llamamos a esto enfoque de **aprendizaje por refuerzo inverso** (IRL, por sus siglas en inglés): aprender recompensas observando una política, en lugar de aprender una política observando recompensas.

Aprendizaje por refuerzo inverso

¿Cómo encontramos la función de recompensa del experto, dadas las acciones del experto? Comencemos suponiendo que el experto actuaba de manera racional. En ese caso, parece que buscar una función de recompensa R^* tal que la recompensa total esperada descontada bajo la política del experto es mayor que (o al menos la misma que) bajo cualquier otra política posible.

No obstante, habrá múltiples funciones de recompensa que satisfacen esta restricción; una de ellas es $R^*(s,a,s') = 0$, porque cualquier política es racional cuando no hay recompensas en absoluto.[7] Otro problema con este enfoque es que la suposición de un experto racional es poco realista. Esto significa, por ejemplo, que un robot que observa a Lee Sedol haciendo lo que eventualmente resulta ser un movimiento perdedor contra AlphaGo, tendría que asumir que Lee Sedol estaba tratando de perder el juego.

Para evitar el problema de que $R^*(s,a,s') = 0$ explica cualquier comportamiento observado, ayuda pensar de manera bayesiana. (Ver Sección 20.1 para recordar qué significa esto). Suponga que observamos los datos $\mathbf{d}$ y dejamos que h_R sea la hipótesis de que R es la verdadera función de recompensa. Entonces, según la regla de Bayes, tenemos

$$P(h_R|\mathbf{d}) = \alpha P(\mathbf{d}|h_R)P(h_R).$$

[7] Según la Ecuación (17.9) en la página 569, una función de recompensa $R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s)$ tiene las mismas políticas óptimas que $R(s,a,s')$, por lo que podemos recuperar la función de recompensa sólo hasta la posible adición de cualquier función de conformación $\Phi(s)$. Esto no es un problema tan serio, porque un robot que usa R' se comportará igual que un robot que usa la R "correcta".

Ahora, si el prior (conocimiento inicial) $P(h_R)$ se basa en la simplicidad, entonces la hipótesis de que $R = 0$ puntúa bastante bien, porque 0 es simple. Por otro lado, el término $P(\mathbf{d}|h_R)$ es infinitesimal para la hipótesis de que $R = 0$, porque no explica por qué el experto eligió ese comportamiento particular de entre el vasto espacio de comportamientos que serían óptimos si la hipótesis fuera verdadera. Además, para una función de recompensa R que tiene una política óptima única o una clase de equivalencia relativamente pequeña de políticas óptimas, $P(\mathbf{d}|h_R)$ será mucho más alto.

Para permitir el error ocasional del experto, permitimos que $P(\mathbf{d}|h_R)$ sea distinto de cero incluso cuando **d** proviene de un comportamiento que es un poco subóptimo según R. Una típica suposición, debe decirse, hecha más por conveniencia matemática que por fidelidad a los datos humanos reales, es que un agente cuya verdadera función Q es $Q(s,a)$ elige no de acuerdo con la política determinista $\pi(s) = \arg\max_a Q(s,a)$ sino de acuerdo con una política estocástica definida por la distribución softmax de la Ecuación (22.14). Esto a veces se llama **racionalidad de Boltzmann** porque, en la mecánica estadística, las probabilidades de ocupación de estado en una distribución de Boltzmann dependen de manera exponencial de sus niveles de energía.

Racionalidad de Boltzmann

Hay docenas de algoritmos de RL inversa en la literatura. Uno de los más simples se llama **coincidencia de características**. Este asume que la función de recompensa es escrita como una combinación lineal ponderada de características:

Coincidencia de características

$$R_\theta(s,a,s') = \sum_{i=1}^{n} \theta_i f_i(s,a,s') = \theta \cdot \mathbf{f}.$$

Por ejemplo, las características en el dominio de la conducción podrían incluir velocidad, velocidad en exceso del límite de velocidad, aceleración, proximidad al obstáculo más cercano, etc.

Recuerde de la Ecuación (17.2), en la página 567, que la utilidad de ejecutar una política π, comenzando en el estado s_0, se define como

$$U^\pi(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), (S_{t+1}))\right],$$

donde la expectativa E es con respecto a la distribución de probabilidad sobre secuencias de estados determinadas por s y π. Como se supone que R es una combinación lineal de valores de características, podemos reescribir esto de la siguiente manera:

$$\begin{aligned} U^\pi(s) &= E\left[\sum_{t=0}^{\infty} \gamma^t \sum_{i=1}^{n} \theta_i f_i(S_t, \pi(S_t), S_{t+1})\right] \\ &= \sum_{i=1}^{n} \theta_i E\left[\sum_{t=0}^{\infty} \gamma^t f_i(S_t, \pi(S_t), S_{t+1})\right] \\ &= \sum_{i=1}^{n} \theta_i \mu_i(\pi) = \theta \cdot \mu(\pi) \end{aligned}$$

Expectativa de característica

donde hemos definido la **expectativa de característica** $\mu_i(\pi)$ como el valor descontado esperado de la característica f_i cuando se ejecuta la política π. Por ejemplo, si f_i es la velocidad excesiva del vehículo (por encima del límite de velocidad), entonces $\mu_i(\pi)$ es la velocidad excesiva promedio (descontada por el tiempo) durante toda la trayectoria. El punto clave sobre las expectativas de características es el siguiente: *si una política π produce expectativas de características $\mu_i(\pi)$ que coinciden con las de la política del experto π_E, entonces π es tan buena como la política del experto según la propia función de recompensa del experto.* Ahora, no podemos medir los valores exactos para las expectativas de características de la política del experto, pero podemos aproximarlas utilizando los valores promedio en las trayectorias observadas. Por lo tanto, necesitamos encontrar valores para los parámetros θ_i de tal manera que las expectativas de características de la política

inducida por los valores de los parámetros coincidan con las de la política experta en las trayectorias observadas. El siguiente algoritmo logra esto con cualquier límite de error deseado.

- Elige una política predeterminada inicial $\pi^{(0)}$.
- Para $j = 1,2,\ldots$ hasta la convergencia:
  - Encuentra los parámetros $\theta^{(j)}$ de tal manera que la política del experto supere al máximo a las políticas $\pi^{(0)},\ldots,\pi^{(j-1)}$ según la utilidad esperada $\theta^{(j)}\cdot\mu(\pi)$.
  - Definiendo $^{(j)}$ como la política óptima para la función de recompensa $R^{(j)} = \theta^{(j)}\cdot\mathbf{f}$.

Este algoritmo converge a una política que está cerca en valor a la del experto, según la propia función de recompensa del experto. Sólo requiere $O(n\log n)$ iteraciones y $O(n\log n)$ demostraciones de expertos, donde n es el número de características.

Un robot puede usar el aprendizaje por refuerzo inverso para aprender una buena política para sí mismo, entendiendo las acciones de un experto. Además, el robot puede aprender las políticas utilizadas por otros agentes en un dominio multiagente, ya sean adversarios o cooperativos. Y, finalmente, el aprendizaje por refuerzo inverso permite ser utilizado para la investigación científica (sin pensar en el diseño del agente), para entender mejor el comportamiento de los humanos y otros animales.

Una suposición clave en RL inverso es que el "experto" se comporta de manera óptima, o casi óptima, con respecto a alguna función de recompensa en un MDP de un solo agente. Esta es una suposición razonable si el aprendiz está observando al experto a través de un espejo unidireccional mientras el experto realiza su trabajo sin darse cuenta. No es una suposición razonable si el experto está consciente del aprendiz. Por ejemplo, suponga que un robot está en la escuela de medicina aprendiendo a ser cirujano observando a un experto humano, un algoritmo de RL inverso asumiría que el humano realiza la cirugía de la manera óptima habitual, como si el robot no estuviera allí. Pero eso no es lo que sucedería: el cirujano humano está motivado para que el robot (como cualquier otro estudiante de medicina) aprenda rápido y bien, por lo que modificará de manera sustancial su comportamiento. Podría explicar lo que está haciendo a medida que avanza; podría señalar errores a evitar, como hacer la incisión demasiado profunda o los puntos demasiado apretados; podría describir los planes de contingencia en caso de que algo salga mal durante la cirugía. Ninguno de estos comportamientos tiene sentido al realizar la cirugía en aislamiento, por lo que los algoritmos de RL inverso no podrán interpretar la función de recompensa subyacente. En cambio, necesitamos entender este tipo de situación como un juego de asistencia de dos personas, como se describe en la Sección 18.2.5.

22.7 Aplicaciones del aprendizaje por refuerzo

Ahora nos volvemos a las aplicaciones del aprendizaje por refuerzo. Estas incluyen el juego, donde se conoce el modelo de transición y el objetivo es aprender la función de utilidad, y la robótica, donde el modelo es, en inicio, desconocido.

22.7.1 Aplicaciones en el juego

En el Capítulo 1 describimos el trabajo temprano de Arthur Samuel sobre el aprendizaje por refuerzo para el juego de damas, que comenzó en 1952. Pasaron algunas décadas antes de que el desafío fuera retomado, esta vez por Gerry Tesauro en su trabajo sobre el backgammon. El primer intento de Tesauro (1990) fue un sistema llamado NEUROGAMMON. El enfoque fue una variante interesante sobre el aprendizaje por imitación. La entrada era un conjunto de 400 juegos jugados por Tesauro contra sí mismo. En lugar de aprender una política, NEUROGAMMON convirtió cada movimiento (s,a,s') en un conjunto de ejemplos de entrenamiento, cada uno de los cuales

etiquetó s' como una mejor posición que alguna otra posición s'' alcanzable desde s por un movimiento diferente. La red tenía dos mitades separadas, una para s' y otra para s'', y estaba obligada a elegir cuál era mejor comparando las salidas de las dos mitades. De esta manera, cada mitad se vio obligada a aprender una función de evaluación $\hat{U}_\theta$. NEUROGAMMON ganó la Olimpiada de Computadoras de 1989, fue el primer programa de aprendizaje que ganó un torneo de juegos de computadora, pero nunca progresó más allá del nivel intermedio de juego de Tesauro.

El siguiente sistema de Tesauro, TD-GAMMON (1992), adoptó el recientemente publicado método de aprendizaje TD de Sutton, volviendo en esencia al enfoque explorado por Samuel, pero con un mayor entendimiento técnico de cómo hacerlo de manera correcta. La función de evaluación $\hat{U}_\theta$ fue representada por una red neuronal completamente conectada con una sola capa oculta que contiene 80 nodos. (También utilizó algunas características de entrada diseñadas de manera manual tomadas de NEUROGAMMON). Después de 300 000 juegos de entrenamiento, alcanzó un nivel de juego comparable a los tres mejores jugadores humanos del mundo. Kit Woolsey, un jugador de los diez mejores, dijo, "No hay duda en mi mente de que su juicio posicional es mucho mejor que el mío".

El siguiente desafío fue aprender a partir de entradas perceptuales crudas, algo más cercano al mundo real, en lugar de representaciones de tableros de juego discretos. A partir de 2012, un equipo en Deep-Mind desarrolló el sistema de **red Q profunda** (**DQN**, por sus siglas en inglés), el primer sistema moderno de RL profundo. DQN utiliza una red neuronal profunda para representar la función Q; de lo contrario, es un sistema típico de aprendizaje por refuerzo. DQN fue entrenado por separado en cada uno de los 49 videojuegos de Atari. Aprendió a conducir coches de carreras simulados, a disparar a naves espaciales alienígenas y a rebotar bolas con paletas. En cada caso, el agente aprendió una función Q a partir de datos de imagen crudos con la señal de recompensa siendo la puntuación del juego. En general, el sistema funcionó a un nivel casi humano experto, aunque algunos juegos le dieron problemas. Un juego en particular, *Montezuma's Revenge* , resultó ser demasiado difícil, porque requería estrategias de planificación extendidas, y las recompensas eran demasiado escasas. Trabajos posteriores produjeron sistemas de RL profundos que generaron comportamientos exploratorios más extensos y fueron capaces de conquistar *Montezuma's Revenge* y otros juegos difíciles.

Red Q profunda (DQN)

El sistema ALPHAGO de DeepMind también utilizó el aprendizaje profundo por refuerzo para vencer a los mejores jugadores humanos en el juego de Go (ver Capítulo 5). Mientras que una función Q sin mirar hacia adelante es suficiente para los juegos de Atari, principalmente reactivos por naturaleza, Go requiere una mirada sustancial hacia adelante. Por esta razón, ALPHAGO aprendió tanto una función de valor como una función Q que guio su búsqueda prediciendo qué movimientos valen la pena explorar. La función Q, implementada como una red neuronal convolucional, es lo suficientemente precisa por sí misma para vencer a la mayoría de los jugadores humanos aficionados sin ninguna búsqueda en absoluto.

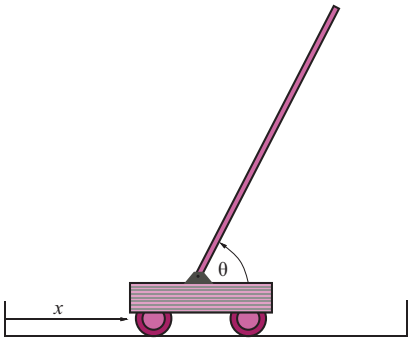
22.7.2 Aplicación al control de robots

Carro-poste
Péndulo invertido

La configuración para el famoso problema de equilibrio del **carro-poste**, también conocido como **péndulo invertido**, ilustrado en la Figura 22.9(a). El problema es mantener el poste más o menos vertical ($\theta \approx 90°$) aplicando fuerzas para mover el carro hacia la derecha o hacia la izquierda, mientras se mantiene la posición x dentro de los límites de la pista. Se han publicado varios miles de artículos sobre aprendizaje por refuerzo y teoría del control en este problema aparentemente simple. Una dificultad es que las variables de estado x, θ, $\dot{x}$, y $\dot{\theta}$ son continuos. Sin embargo, las acciones se definen como discretas: tirón a la izquierda o tirón a la derecha, el llamado régimen de **control bang-bang**.

Control bang-bang

El trabajo más temprano sobre el aprendizaje para este problema fue realizado por Michie y Chambers (1968), utilizando un carro real y un poste, no una simulación. Su algoritmo BOXES fue



(a)

(b)

Figura 22.9 (a) Configuración para el problema de equilibrar un poste largo en la parte superior de un carro en movimiento. El carro puede ser sacudido a la izquierda o a la derecha por un controlador que observa la posición x del carro y la velocidad $\dot{x}$, así como el ángulo θ del poste y la tasa de cambio de ángulo $\dot{\theta}$. (b) Seis imágenes superpuestas de lapso de tiempo de un sólo helicóptero autónomo realizando una muy difícil maniobra "nariz en círculo". El helicóptero está bajo el control de una política desarrollada por el algoritmo de búsqueda de políticas PEGASUS (Ng *et al.,* 2003). Se desarrolló un modelo de simulador observando los efectos de varias maniobras de control en el helicóptero real; luego el algoritmo se ejecutó en el modelo de simulador durante la noche. Se desarrolló una variedad de controladores para diferentes maniobras. En todos los casos, el rendimiento superó con creces al de un piloto humano experto utilizando control remoto. (Imagen cortesía de Andrew Ng).

capaz de equilibrar el poste durante más de una hora después de 30 pruebas. El algoritmo inicial discretizó el espacio de estado de cuatro dimensiones en cajas, de ahí el nombre. Luego realizó pruebas hasta que el poste se cayó. Se asoció un refuerzo negativo con la acción final en la caja final y luego se propagó a través de la secuencia. Es posible obtener una mejor generalización y un aprendizaje más rápido utilizando un algoritmo que divide *de manera adaptativa* el espacio de estado de acuerdo con la variación observada en la recompensa, o utilizando un aproximador de función no lineal de estado continuo, como una red neuronal. Hoy en día, equilibrar un péndulo invertido *triple* (tres postes unidos de extremo a extremo) es un ejercicio común, una hazaña muy por encima de las capacidades de la mayoría de los humanos, pero alcanzable utilizando el aprendizaje por refuerzo.

Aún más impresionante es la aplicación del aprendizaje por refuerzo al vuelo de helicópteros controlados por radio (Figura 22.9(b)). En general, este trabajo ha utilizado la búsqueda de políticas sobre grandes MDPs (Bagnell y Schneider, 2001; Ng *et al*., 2003), con frecuencia combinada con aprendizaje por imitación e inverso RL dado las observaciones de un piloto humano experto (Coates *et al*., 2009).

El RL inverso también ha sido aplicado con éxito para interpretar el comportamiento humano, incluyendo la predicción de destino y la selección de ruta por parte de los taxistas basada en 100 000 millas de datos GPS (Ziebart *et al*., 2008) y los movimientos físicos detallados de los peatones en entornos complejos basados en horas de observación de video (Kitani *et al*., 2012). En el área de la robótica, una sola demostración de experto fue suficiente para que el cuadrúpedo LittleDog aprendiera una función de recompensa de 25 características y atravesara de manera ágil un área de terreno rocoso previamente no visto (Kolter *et al*., 2008). Para más información sobre cómo se utilizan RL e inverso RL en robótica, ver las Secciones 26.7 y 26.8.

Resumen

Este capítulo ha examinado el problema del aprendizaje por refuerzo: cómo un agente llega a ser competente en un entorno desconocido, dadas sólo sus percepciones y recompensas ocasionales. El aprendizaje por refuerzo es un paradigma aplicable para crear sistemas inteligentes. Los principales puntos del capítulo son.

- El diseño general del agente dicta el tipo de información que debe ser aprendida:
  - Un agente de **aprendizaje por refuerzo basado en modelos** adquiere (o está equipado con) un modelo de transición $P(s'|s,a)$ para el entorno y aprende una función de utilidad $U(s)$.
  - Un **agente de aprendizaje por refuerzo** sin modelo permite aprender una función de utilidad de acción $Q(s,a)$ o una política $\pi(s)$.
- Las utilidades permiten ser aprendidas usando varios enfoques diferentes:
  - La **estimación directa de utilidad** usa la recompensa total observada para un estado dado como evidencia directa para aprender su utilidad.
  - La **programación dinámica adaptativa** (ADP) aprende un modelo y una función de recompensa a partir de observaciones, luego utiliza la iteración de valor o política para obtener las utilidades o una política óptima. ADP hace un uso óptimo de las restricciones locales sobre las utilidades de los estados impuestas a través de la estructura de vecindario del entorno.
  - Los métodos de **diferencia temporal** (TD) ajustan las estimaciones de utilidad para ser más consistentes con las de los estados sucesores. Pueden ser vistos como simples aproximaciones del enfoque ADP que permiten aprender sin requerir un modelo de transición. Sin embargo, el uso de un modelo aprendido para generar seudoexperiencias resulta en un aprendizaje más rápido.
- Las funciones de utilidad de acción, o funciones Q, pueden ser aprendidas por un enfoque ADP o un enfoque TD. Con TD, el **aprendizaje Q** no requiere modelo en la fase de aprendizaje o selección de acción. Esto simplifica el problema de aprendizaje, pero restringe la capacidad de aprender en entornos complejos, porque el agente no permite simular los resultados de posibles cursos de acción.
- Cuando el agente de aprendizaje es responsable de seleccionar acciones mientras aprende, debe equilibrar el valor estimado de esas acciones contra el potencial para aprender nueva información útil. Una solución exacta para el problema de exploración es inviable, pero algunas heurísticas simples hacen un trabajo razonable. Un agente explorador también debe tener cuidado de evitar una muerte prematura.
- En grandes espacios de estado, los algoritmos de aprendizaje por refuerzo deben usar una representación funcional aproximada de $U(s)$ o $Q(s,a)$ para generalizar sobre los estados. El **aprendizaje por refuerzo profundo**, utilizando redes neuronales profundas como aproximadores de funciones, ha logrado un éxito considerable en problemas difíciles.
- La **conformación de recompensas** y el **aprendizaje por refuerzo jerárquico** son útiles para aprender comportamientos complejos, en particular cuando las recompensas son escasas y se requieren largas secuencias de acciones para obtenerlas.
- Los métodos de **búsqueda de políticas** operan de manera directa en una representación de la política, intentando mejorarla con base en el rendimiento observado. La variación en el rendimiento en un dominio estocástico es un problema serio; para dominios simulados esto puede ser superado fijando la aleatoriedad de antemano.
- El **aprendizaje por observación** del comportamiento experto es una solución efectiva cuando es difícil especificar una función de recompensa correcta. El **aprendizaje por**

imitación formula el problema como aprendizaje supervisado de una política a partir de los pares de estado-acción del experto. El **aprendizaje por refuerzo inverso** infiere información de recompensa del comportamiento del experto.

El aprendizaje por refuerzo continúa siendo una de las áreas más activas de la investigación en aprendizaje automático. Nos libera de la construcción manual de comportamientos y de etiquetar los vastos conjuntos de datos requeridos para el aprendizaje supervisado, o de tener que codificar de manera manual las estrategias de control. Las aplicaciones en robótica prometen ser particularmente valiosas; éstas requerirán métodos para manejar entornos continuos, de alta dimensión, parcialmente observables en los que los comportamientos exitosos pueden consistir en miles o incluso millones de acciones primitivas.

Hemos presentado una variedad de enfoques para el aprendizaje por refuerzo porque no hay (al menos hasta ahora) un enfoque mejor. La cuestión de los métodos basados en modelos contra los métodos libres de modelos es, en su esencia, una cuestión sobre la mejor manera de representar la función del agente. Este es un problema en los fundamentos de la inteligencia artificial. Como afirmamos en el Capítulo 1, una de las características históricas clave de gran parte de la investigación en IA es su adhesión (en la mayoría de los casos no declarada) al enfoque **basado en el conocimiento**. Esto equivale a asumir que la mejor manera de representar la función del agente es construir una representación de algunos aspectos del entorno en el que se encuentra el agente. Algunos argumentan que con acceso a suficientes datos, los métodos libres de modelos permiten tener éxito en cualquier dominio. Quizás esto sea cierto en teoría, aunque, el universo puede no contener suficientes datos para hacerlo cierto en la práctica. (Por ejemplo, no es fácil imaginar cómo un enfoque libre de modelos permitiría diseñar y construir, digamos, el detector de ondas gravitacionales LIGO). Nuestra intuición, por lo que vale, es que a medida que el entorno se vuelve más complejo, las ventajas de un enfoque basado en modelos se vuelven más evidentes.

Notas bibliográficas e históricas

Parece probable que la idea clave del aprendizaje por refuerzo, que los animales hacen más de lo que se les recompensa y menos de lo que se les castiga, tuvo un papel significativo en la domesticación de los perros hace al menos 15 000 años. Los primeros fundamentos de nuestro entendimiento científico del aprendizaje por refuerzo incluyen el trabajo del fisiólogo ruso Ivan Pavlov, quien ganó el Premio Nobel en 1904, y el del psicólogo estadounidense Edward Thorndike, en particular por su libro *Inteligencia animal* (1911). Hilgard y Bower (1975) proporcionan una buena revisión.

Alan Turing (1948, 1950) propuso el aprendizaje por refuerzo como un enfoque para enseñar a las computadoras; lo consideró una solución parcial, pues escribió, "El uso de castigos y recompensas es, en el mejor de los casos, una parte del proceso de enseñanza". El programa de damas de Arthur Samuel (1959, 1967) fue el primer uso exitoso de aprendizaje automático de cualquier tipo. Samuel sugirió la mayoría de las ideas modernas en aprendizaje por refuerzo, incluyendo el aprendizaje de diferencias temporales y la aproximación de funciones. Experimentó con representaciones de múltiples capas de funciones de valor, similar al RL profundo de hoy. Al final, descubrió que una simple función de evaluación lineal sobre características hechas a mano funcionaba mejor. Esto puede haber sido una consecuencia de trabajar con una computadora aproximadamente 100 mil millones de veces menos potente que una unidad de procesamiento tensor moderna.

Por la misma época, los investigadores en teoría de control adaptativo (Widrow y Hoff, 1960), basándose en el trabajo de Hebb (1949), estaban entrenando redes simples usando la regla delta.

Así, el aprendizaje por refuerzo combina influencias de psicología animal, neurociencia, investigación de operaciones y teoría del control óptimo.

La conexión entre el aprendizaje por refuerzo y los procesos de decisión de Markov fue establecida por primera vez por Werbos (1977). (El trabajo de Ian Witten (1977) describió un proceso similar al TD en el lenguaje de la teoría de control). El desarrollo del aprendizaje por refuerzo en la IA proviene principalmente del trabajo en la Universidad de Massachusetts a principios de la década de 1980 (Barto *et al*., 1981). Un influyente artículo de Rich Sutton (1988) proporcionó una comprensión matemática de los métodos de diferencia temporal. La combinación del aprendizaje de la diferencia temporal con la generación basada en modelos de experiencias simuladas fue propuesta en la arquitectura DYNA de Sutton (Sutton, 1990). El Q-learning fue desarrollado en la tesis doctoral de Chris Watkins (1989), mientras que SARSA apareció en un informe técnico de Rummery y Niranjan (1994). El barrido prioritario fue introducido de manera independiente por Moore y Atkeson (1993) y Peng y Williams (1993).

La aproximación de funciones en el aprendizaje por refuerzo se remonta al programa de damas de Arthur Samuel (1959). El uso de redes neuronales para representar funciones de valor fue común en la década de 1980 y cobró protagonismo en el programa TD-Gammon de Gerry Tesauro (Tesauro, 1992, 1995). Las redes neuronales profundas son actualmente la opción más popular para los aproximadores de funciones en el aprendizaje por refuerzo. Arulkumaran *et al*. (2017) y Francois-Lavet *et al*. (2018) dan una visión general del RL profundo. El sistema DQN (Mnih *et al.,* 2015) utiliza una red profunda para aprender una función Q, mientras que AlphaZero (Silver *et al*., 2018) aprende tanto una función de valor para usar con un modelo conocido como una función Q para usar en decisiones de nivel meta que guían la búsqueda. Irpan (2018) advierte que los sistemas de RL profundo pueden rendir mal si el entorno real es incluso diferente del entorno de entrenamiento.

Las combinaciones lineales ponderadas de características y las redes neuronales son representaciones factorizadas para la aproximación de funciones. También es posible aplicar el aprendizaje por refuerzo a representaciones estructuradas; esto se llama **aprendizaje por refuerzo relacional** (Tadepalli *et al.,* 2004). El uso de descripciones relacionales permite la generalización a través de comportamientos complejos que involucran diferentes objetos.

El análisis de las propiedades de convergencia de los algoritmos de aprendizaje por refuerzo utilizando la aproximación de funciones es un tema extremadamente técnico. Los resultados para el aprendizaje TD se han fortalecido de manera progresiva para el caso de los aproximadores de funciones lineales (Sutton, 1988; Dayan, 1992; Tsitsiklis y Van Roy, 1997), aunque se han presentado varios ejemplos de divergencia para funciones no lineales (ver Tsitsiklis y Van Roy, 1997, para profundizar en esta discusión). Papavassiliou y Russell (1999) describen un tipo de aprendizaje por refuerzo que converge con cualquier forma de aproximador de funciones, siempre que el problema de ajustar la hipótesis a los datos sea resoluble. Liu *et al*. (2018) describen la familia de algoritmos **TD de gradiente** y proporcionan un análisis teórico extenso de la convergencia y la complejidad de la muestra.

Barto *et al.* (1995) discuten una variedad de métodos de exploración para problemas de decisión secuenciales. Kearns y Singh (1998) y Brafman y Tennenholtz (2000) describen algoritmos que exploran entornos desconocidos y están garantizados para converger en políticas casi óptimas con una complejidad de muestra que es polinomial en el número de estados. El aprendizaje por refuerzo bayesiano (Dearden *et al*., 1998, 1999) ofrece otro ángulo tanto sobre la incertidumbre del modelo como sobre la exploración.

La idea básica que subyace al aprendizaje por imitación es aplicar el aprendizaje supervisado a un conjunto de entrenamiento de acciones expertas. Esta es una idea antigua en el control adaptativo, pero cobró prominencia en la IA con el trabajo de Sammut *et al.* (1992) sobre "Aprender a volar" en un simulador de vuelo. Llamaron a su método **clonación de comportamiento**. Años

después, el mismo grupo de investigación informó que el método era mucho más frágil de lo que se había informado en inicio (Camacho y Michie, 1995): incluso las perturbaciones más pequeñas causaban que la política aprendida se desviara de la trayectoria deseada, lo que llevaba a errores acumulativos a medida que el agente se alejaba cada vez más del conjunto de entrenamiento. (Véase también la discusión en la página 966). El trabajo sobre el aprendizaje por observación tiene como objetivo hacer el enfoque más robusto, en parte incluyendo información sobre los resultados deseados en lugar de sólo la política experta. Ng *et al*. (2003) y Coates *et al*. (2009) muestran cómo funciona el aprendizaje por observación para aprender a volar un helicóptero real, como se ilustra en la Figura 22.9(b) en la página 817.

El aprendizaje por refuerzo inverso (IRL) fue introducido por Russell (1998), y los primeros algoritmos fueron desarrollados por Ng y Russell (2000). (Un problema similar ha sido estudiado en economía durante mucho más tiempo, bajo el título de **estimación estructural de MDPs** (Sargent, 1978)). El algoritmo dado en el capítulo se debe a Abbeel y Ng (2004). Baker *et al*. (2009) describen cómo la comprensión de las acciones de otro agente es vista como una planificación inversa. Ho *et al*. (2017) muestran que los agentes permiten aprender mejor de los comportamientos que son instructivos en lugar de óptimos. Hadfield-Menell *et al*. (2017a) extienden IRL a una formulación de juego teórico que abarca tanto al observador como al demostrador, mostrando cómo los comportamientos de enseñanza y aprendizaje emergen como soluciones del juego.

García y Fernández (2015) dan una encuesta completa sobre el aprendizaje por refuerzo seguro. Munos *et al*. (2017) describen un algoritmo para la exploración segura fuera de la política (por ejemplo, Q-learning). Hans *et al*. (2008) dividen el problema de la exploración segura en dos partes: definir una función de seguridad para indicar qué estados evitar, y definir una política de respaldo para llevar al agente de vuelta a la seguridad cuando podría entrar en un estado inseguro. You *et al*. (2017) muestran cómo entrenar un modelo de aprendizaje profundo por refuerzo para conducir un coche en simulación, y luego usar el aprendizaje de transferencia para conducir de forma segura en el mundo real.

Thomas *et al*. (2017) ofrecen un enfoque de aprendizaje que está garantizado, con alta probabilidad, para no hacerlo peor que la política actual. Akametalu *et al*. (2014) describen un enfoque basado en la alcanzabilidad, en el que el proceso de aprendizaje opera bajo la guía de una política de control que asegura que el agente nunca llega a un estado inseguro. Saunders *et al*. (2018) demuestran que un sistema puede usar la intervención humana para evitar que se desvíe de la región segura, y permite aprender con el tiempo a necesitar menos intervención.

Los métodos de búsqueda de políticas fueron llevados a la vanguardia por Williams (1992), quien desarrolló la familia de algoritmos REINFORCE, que significa "REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility". Trabajos posteriores de Marbach y Tsitsiklis (1998), Sutton *et al*. (2000), y Baxter y Bartlett (2000) fortalecieron y generalizaron los resultados de convergencia para la búsqueda de políticas. Schulman *et al*. (2015b) describen la **optimización de política en la Región de Confianza**, un algoritmo de búsqueda de políticas teóricamente bien fundamentado y práctico que ha generado muchas variantes. El método de muestreo correlacionado para reducir la varianza en las comparaciones de Monte Carlo se debe a Kahn y Marshall (1953); también es uno de los varios métodos de reducción de varianza explorados por Hammersley y Handscomb (1964).

Los primeros enfoques al aprendizaje por refuerzo jerárquico (HRL) intentaron construir jerarquías usando la **abstracción de estado**, es decir, agrupando estados en estados abstractos y luego haciendo RL en el espacio de estados abstractos (Dayan y Hinton, 1993). Sin embargo, el modelo de transición para los estados abstractos es con frecuencia no-markoviano, lo que lleva a un comportamiento divergente de los algoritmos estándar de RL. El enfoque de abstracción temporal en este capítulo fue desarrollado a finales de la década de 1990 (Parr y Russell,

1998; Andre y Russell, 2002; Sutton *et al.,* 2000) y ampliado para manejar comportamientos concurrentes por Marthi *et al*. (2005). Dietterich (2000) introdujo la noción de una descomposición aditiva de las funciones Q inducida por la jerarquía de subrutinas. La abstracción temporal se basa en un resultado mucho más antiguo debido a Forestier y Varaiya (1978), quienes mostraron que un gran MDP permite ser descompuesto en un sistema de dos capas en el que una capa supervisora elige entre los controladores de bajo nivel, cada uno de los cuales devuelve el control al supervisor al terminar. El problema de aprender la jerarquía de abstracción en sí ha sido estudiado al menos desde el trabajo de Peter Andreae (1985); para una exploración reciente en el aprendizaje de primitivas de movimiento de robots, vea Frans *et al.* (2018). El juego de keepaway fue introducido por Stone *et al*. (2005); la solución HRL dada aquí es debido a Bai y Russell (2017).

La neurociencia con frecuencia ha inspirado el aprendizaje por refuerzo y ha confirmado el valor del enfoque. La investigación utilizando la grabación de una sola célula sugiere que el sistema de dopamina en los cerebros de los primates implementa algo que se parece al aprendizaje de la función de valor (Schultz *et al.,* 1997). El texto de neurociencia de Dayan y Abbott (2001) describe posibles implementaciones neuronales del aprendizaje de la diferencia temporal; la investigación relacionada describe otros experimentos neurocientíficos y conductuales (Dayan y Niv, 2008; Niv, 2009; Lee *et al*., 2012).

El trabajo en aprendizaje por refuerzo ha sido acelerado por la disponibilidad de entornos de simulación de código abierto para desarrollar y probar agentes de aprendizaje. El Arcade Learning Environment (ALE) de la Universidad de Alberta (Bellemare *et al*., 2013) proporcionó un marco para 55 videojuegos clásicos de Atari. Los píxeles en la pantalla se proporcionan al agente como percepciones, junto con una puntuación preestablecida del juego hasta el momento. ALE fue utilizado por el equipo de DeepMind para implementar el aprendizaje DQN y verificar la generalidad de su sistema en una amplia variedad de juegos (Mnih *et al*., 2015).

DeepMind, a su vez, ha liberado varias plataformas de agentes, incluyendo el DeepMind Lab (Beattie *et al*., 2016), los AI Safety Gridworlds (Leike *et al*., 2017), la plataforma de juegos Unity (Juliani *et al*., 2018), y el DM Control Suite (Tassa *et al.,* 2018). Blizzard lanzó el StarCraft II Learning Environment (SC2LE), al que DeepMind añadió el componente PySC2 para el aprendizaje de máquinas en Python (Vinyals *et al*., 2017a).

La simulación de AI Habitat de Facebook (Savva *et al*., 2019) proporciona un entorno virtual fotorrealista para tareas robóticas en interiores, y su plataforma Horizon (Gauci *et al*., 2018) permite el aprendizaje por refuerzo en sistemas de producción a gran escala. El sistema Synthia (Ros *et al*., 2016) es un entorno de simulación diseñado para mejorar las capacidades de visión por computadora de los vehículos autónomos. El OpenAI Gym (Brockman *et al*., 2016) ofrece varios entornos para agentes de aprendizaje por refuerzo, y es compatible con otras simulaciones como el simulador de futbol de Google.

Littman (2015) realiza una encuesta sobre el aprendizaje por refuerzo para una audiencia científica general. El texto canónico de Sutton y Barto (2018), dos de los pioneros del campo, muestra cómo el aprendizaje por refuerzo entrelaza las ideas de aprendizaje, planificación y acción. Kochenderfer (2015) adopta un enfoque menos matemático, con abundantes ejemplos del mundo real. Un libro breve de Szepesvari (2010) ofrece una visión general de los algoritmos de aprendizaje por refuerzo. Bertsekas y Tsitsiklis (1996) proporcionan una base rigurosa en la teoría de la programación dinámica y la convergencia estocástica. Los trabajos de aprendizaje por refuerzo se publican con frecuencia en las revistas *Machine Learning* y *Journal of Machine Learning Research*, y en las actas de la International Conference on Machine Learning (ICML) y las conferencias Neural Information Processing Systems (NeurIPS).

CAPÍTULO 23

PROCESAMIENTO DEL LENGUAJE NATURAL

En el cual vemos cómo una computadora usa el lenguaje natural para comunicarse con los humanos y aprende de lo que ellos han escrito.

Los humanos aprendieron a hablar hace cerca de 100 000 años, y hace apenas 5 000 años aprendieron a escribir. La complejidad y diversidad del lenguaje humano diferencia a *Homo sapiens* de todas las demás especies. Por supuesto, existen otros atributos que son humanos, como: ninguna otra especie viste ropa, crea arte o pasa dos horas al día en las redes sociales de la manera que lo hacen los humanos. Pero cuando Alan Turing propuso su prueba de inteligencia, la basó en el lenguaje, no en el arte ni en la vestimenta, quizá debido a su alcance universal y porque el lenguaje captura tanto del comportamiento inteligente: un hablante (o escritor) tiene el **objetivo** de comunicar algún **conocimiento**, luego **planea** algún lenguaje que **representa** el conocimiento, y **actúa** para lograr el objetivo. El oyente (o lector) **percibe** el lenguaje e **infiere** el significado pretendido. Este tipo de comunicación a través del lenguaje ha permitido que la civilización crezca; es nuestro principal medio para transmitir conocimientos culturales, legales, científicos y tecnológicos. Hay tres razones principales para que las computadoras hagan **procesamiento de lenguaje natural (NLP)**:

Procesamiento de lenguaje natural (NLP)

- Para **comunicarse** con los humanos. En múltiples situaciones, para los humanos es conveniente usar el habla para interactuar con las computadoras, y en la mayoría de las situaciones resulta más conveniente usar lenguaje natural en lugar de un lenguaje formal, como el cálculo de predicados de primer orden.
- Para **aprender**. Los humanos han escrito mucho conocimiento usando el lenguaje natural. Wikipedia por sí misma tiene 30 millones de páginas de hechos como "Los bebés de arbusto son pequeños primates nocturnos", mientras que con dificultad encontramos fuentes de hechos como este escrito en lógica formal. Si queremos que nuestro sistema sepa mucho, es mejor que entienda el lenguaje natural.
- Para avanzar en la **comprensión científica** de los lenguajes y el uso del lenguaje, utilizando las herramientas de la IA en conjunto con la lingüística, la psicología cognitiva y la neurociencia.

En este capítulo examinamos varios modelos matemáticos para el lenguaje; además, revisaremos las tareas que se pueden lograr con ellos.

23.1 Modelos de lenguaje

Los lenguajes formales, como la lógica de primer orden, están definidos con precisión, como se vio en el Capítulo 8. La **gramática** define la sintaxis (modo en que se combinan las palabras, los grupos que forman y las relaciones que establecen para expresar significados) de las sentencias legales, las **reglas semánticas** definen el significado.

Los lenguajes naturales, como inglés o chino, no permiten ser caracterizadas:

- Los juicios de lenguaje varían de persona a persona y de tiempo en tiempo. Todos están de acuerdo en que "Not to be invited is sad" es una sentencia gramaticalmente correcta, pero la gente no está de acuerdo en la gramaticalidad de "To be not invited is sad".
- El lenguaje natural es **ambiguo** ("He saw her duck" puede significar que ella posee un ave acuática, o que ella se evadió con un movimiento hacia abajo) y **vago** ("That's great!" no especifica cuán genial es, ni qué es).
- El mapeo de símbolos a objetos no está definido de manera formal. En la lógica de primer orden, dos usos del símbolo "Ricardo" deben referirse a la misma persona, pero en el lenguaje natural dos ocurrencias de la misma palabra o frase se refieren a diferentes cosas en el mundo.

Si no podemos hacer una distinción booleana definitiva entre cadenas gramaticales y no gramaticales, al menos podemos decir cuán probable o improbable es cada una.

Modelo de lenguaje

Definimos un **modelo de lenguaje** como una distribución de probabilidad que describe la probabilidad de cualquier cadena. Tal modelo debería decir que "Do I dare disturb the universe?" tiene una probabilidad razonable como una cadena gramaticalmente correcta, pero "Universe dare the I disturb do?" es muy improbable.

Con un modelo de lenguaje, podemos predecir qué palabras es probable que vengan a continuación en un texto, y sugerir que sea autocompletado para un correo electrónico o mensaje de texto. Podemos calcular qué alteraciones a un texto lo harían más probable, y sugerir así correcciones ortográficas o gramáticas. Con un par de modelos podemos calcular la traducción más probable de una sentencia. Con algunos ejemplos de pares de pregunta/respuesta como datos de entrenamiento, podemos calcular la respuesta más probable a una pregunta. Por lo tanto, los modelos de lenguaje están en el corazón de una amplia gama de tareas de lenguaje natural. La tarea de modelado del lenguaje sirve como punto de referencia común para medir el progreso en la comprensión del lenguaje.

Los lenguajes naturales son complejos, por lo que cualquier modelo de lenguaje será, en el mejor de los casos, una aproximación. El lingüista Edward Sapir expresó "Ningún lenguaje es consistente. Todas las gramáticas tienen fugas" (Sapir, 1921). El filósofo Donald Davidson afirmó "no existe tal cosa como un lenguaje, no si un lenguaje es... una estructura compartida definida" (Davidson, 1986), con lo que quiso decir que no hay un modelo de lenguaje definitivo para el inglés de la misma manera que lo hay para Python 3.8; todos tenemos modelos diferentes, pero de alguna manera logramos comunicarnos. En esta sección cubrimos modelos de lenguaje simplistas que están equivocados, pero que aun así resultan útiles para ciertas tareas.

23.1.1 El modelo de bolsa de palabras

La Sección 12.6.1 explicó cómo un modelo de Bayes ingenuo basado en la presencia de palabras específicas podría clasificar de manera confiable las frases en categorías; por ejemplo, la frase (1) a continuación se categoriza como *negocio*, y (2) como *clima*.

1. Las acciones se dispararon el lunes, con los principales índices ganando 1 % a medida que persistía el optimismo sobre la temporada de ganancias del primer trimestre.
2. La lluvia fuerte continuó azotando gran parte de la costa este el lunes, con advertencias de inundaciones difundidas en la ciudad de Nueva York y otros lugares.

Esta sección revisita el modelo de Bayes ingenuo y lo presenta como un modelo de lenguaje completo. Eso significa que no sólo queremos saber qué categoría es más probable para cada frase; queremos una distribución de probabilidad conjunta sobre todas las frases y categorías. Eso sugiere que deberíamos considerar *todas* las palabras de la frase. Dada una frase compuesta por las palabras w_1, w_2,... w_N (que escribiremos como $w_{1:N}$, como en el Capítulo 14), la fórmula de Bayes ingenua (Ecuación (12.21)) nos da

$$\mathbf{P}(Clase|w_{1:N}) = \alpha\mathbf{P}(Clase)\prod_j \mathbf{P}(w_j|Clase).$$

Modelo de bolsa de palabras

La aplicación de Bayes ingenuo a cadenas de palabras se llama **modelo de bolsa de palabras**. Se trata de un modelo generativo que describe un proceso para generar una frase: Imagine que para cada categoría (*negocios, clima,* etc.) tenemos una bolsa llena de palabras (es posible que imaginemos cada palabra escrita en un pedazo de papel dentro de la bolsa; cuanto más común es la palabra, más veces se escabullirá y duplicará). Para generar texto, primero seleccione una de las bolsas y descarte las demás. Meta la mano en esa bolsa y saque una palabra al azar; esta será la primera palabra de la frase. Luego vuelva a poner la palabra y saque una segunda palabra. Repita hasta que saque un indicador de fin de frase (por ejemplo, un punto).

Este modelo está equivocado: asume de manera falsa que cada palabra es independiente de las demás y, por lo tanto, no genera frases coherentes. Pero nos permite hacer clasificaciones con buena precisión usando la fórmula de Bayes ingenuo: las palabras "acciones" y "ganancias" son evidencia clara para la sección de negocios, mientras que "lluvia" y "nublado" sugieren la sección del clima.

Corpus

Podemos aprender las probabilidades previas necesarias para este modelo a través de un entrenamiento supervisado en un cuerpo o **corpus** de texto, donde cada segmento de texto está etiquetado con una clase. Un corpus consta de al menos un millón de palabras de texto, y como mínimo de decenas de miles de palabras de vocabulario distintas. Recién estamos viendo que se utilizan corpus aún más grandes, como las 2.5 mil millones de palabras en Wikipedia o el corpus iWeb de 14 mil millones de palabras extraídas de 22 millones de páginas web.

A partir de un corpus podemos estimar la probabilidad previa de cada categoría, $\mathbf{P}(Clase)$, por conteo cómo es de común cada categoría. También podemos usar los conteos para estimar la probabilidad condicional de cada palabra dada la categoría, $\mathbf{P}(w_j|Clase)$. Por ejemplo, si hemos visto 3 000 textos y 300 de ellos fueron clasificados como *negocios*, entonces es posible estimar $P(Clase = negocios) \approx 300/3\,000 = 0.1$. Y si dentro de la categoría de *negocios* hemos visto 100 000 palabras, y si la palabra "acciones" apareció 700 veces, en consecuencia podemos estimar $P(acciones|Clase = negocios) \approx 700/100\,000 = 0.007$. La estimación por conteo funciona bien cuando tenemos conteos altos (y baja varianza), pero veremos en la Sección 23.1.4 una mejor manera de estimar las probabilidades cuando los conteos son bajos.

A veces, un enfoque de aprendizaje automático diferente, como la regresión logística, las redes neuronales o las máquinas de vectores de soporte, funciona incluso mejor que el Bayes ingenuo. Las características del modelo de aprendizaje automático son las palabras en el vocabulario: "a", "aardvark",..., "zyzzyva", y los valores son el número de veces que cada palabra aparece en el texto (o, a veces, sólo un valor booleano que indica si la palabra aparece o no). Eso hace las características del vector grande y disperso; podríamos tener 100 000 palabras en el modelo de lenguaje y, por lo tanto, un vector de características de longitud 100 000, pero para un texto corto casi todas las características serán cero.

Como hemos visto, algunos modelos de aprendizaje automático funcionan mejor cuando hacemos **selección de características**, limitándonos a un subconjunto de las palabras como características. Podríamos eliminar palabras que son muy raras (y, por lo tanto, tienen una alta variación en sus poderes predictivos), así como palabras que, comunes a todas las clases (como "the", en inglés), no discriminan entre clases. También podemos mezclar otras características con nuestras características basadas en palabras; por ejemplo, si estamos clasificando mensajes de correo electrónico, podríamos agregar características para el remitente, la hora en que se envió el mensaje, las palabras en el encabezado del asunto, la presencia de puntuación no estándar, el porcentaje de letras mayúsculas, si hay un adjunto, etc.

Tokenización

Note que no es trivial decidir qué es una *palabra*. ¿Es "aren't" una palabra, o debería dividirse como "aren/'/t" o "are/n't", o algo más? El proceso de dividir un texto en una secuencia de palabras se llama **tokenización**.

23.1.2 Modelos de palabras *n*-grama

El modelo de bolsa de palabras tiene limitaciones. Por ejemplo, la palabra "quarter" es común en las categorías de *negocios* y *deportes*. Pero la secuencia de cuatro palabras "first quarter earnings report" es común sólo en los *negocios* y "fourth quarter touchdown passes" es común sólo en los *deportes*. Nos gustaría que nuestro modelo hiciera esa distinción. Podríamos ajustar el modelo de bolsa de palabras tratando frases especiales como "first-quarter earnings report" como si fueran palabras únicas, pero un enfoque más principiado es introducir un nuevo modelo, donde cada palabra depende de las palabras anteriores. Podemos comenzar haciendo que una palabra dependa de *todas* las palabras anteriores en una sentencia:

$$P(w_{1:N}) = \prod_{j=1}^{N} P(w_j | w_{1:j-1}).$$

Este modelo es en cierto sentido correcto en el sentido de que captura todas las posibles interacciones entre palabras, pero no es práctico: con un vocabulario de 100 000 palabras y una longitud de sentencia de 40, este modelo tendría 10^{200} parámetros para estimar. Podemos comprometernos con un modelo de **cadena de Markov** que considera sólo la dependencia entre n palabras adyacentes.

Modelo *n*-grama

Esto se conoce como un **modelo *n*-grama** (de la raíz griega *gramma* que significa "cosa escrita"): una secuencia de símbolos escritos de longitud n se llama un n-grama, con casos especiales "unigrama" para 1-grama, "bigrama" para 2-grama, y "trigrama" para 3-grama. En un modelo n-grama, la probabilidad de cada la palabra depende sólo de las $n - 1$ palabras anteriores; es decir:

$$P(w_j | w_{1:j-1}) = P(w_j | w_{j-n+1:j-1})$$
$$P(w_{1:N}) = \prod_{j=1}^{N} P(w_j | w_{j-n+1:j-1}).$$

Los modelos N-grama funcionan bien para clasificar secciones de periódicos, así como para otras tareas de clasificación como la **detección de spam** (distinguir el correo electrónico spam de no spam), **análisis de sentimientos** (clasificar una reseña de una película o producto como positiva o negativa) y **atribución de autor** (Hemingway tiene un estilo y vocabulario diferente a Faulkner o Shakespeare).

Detección de spam
Análisis de sentimientos
Atribución de autor

23.1.3 Otros modelos *n*-grama

Una alternativa a un modelo de palabras n-grama es un **modelo a nivel de caracter** en el que la probabilidad de cada caracter está determinada por los n−1 caracteres anteriores. Este enfoque es útil para lidiar con palabras desconocidas y para idiomas que tienden a contraer palabras, como en la palabra danesa "Speciallægepraksisplanlægningsstabiliseringsperiode".

Modelo a nivel de carácter

Los modelos a nivel de caracteres son adecuados para la tarea de **identificación de idiomas**: dado un texto, determinar en qué idioma está escrito. Incluso con textos muy cortos como "Hello, world" o "Wie geht's dir", los modelos de letras de n-gramas permiten identificar el primero como inglés y el segundo como alemán, logrando en general una precisión mayor a 99 %. (Los idiomas relacionados como el sueco y el noruego son más difíciles de distinguir y requieren muestras más largas; allí, la precisión está en el rango de 95 %). Los modelos de caracteres son buenos en ciertas tareas de clasificación, como decidir que "dextroamphetamine" es un nombre de medicamento, "Kallenberger" es un nombre de persona, y "Plattsburg" es un nombre de ciudad, incluso si nunca hemos visto estas palabras antes.

Identificación de idioma

Otra posibilidad es el modelo de **skip-grama**, en el que contamos palabras que están cerca una de la otra, pero saltamos una palabra (o más) entre ellas. Por ejemplo, dado el texto francés "je ne comprends pas" los 1-skip-bigrams son "je comprends", y "ne pas". Recopilando estos ayuda

Skip-grama

a crear un mejor modelo de francés, porque nos informa sobre la conjugación ("je" va con "comprends", no "comprend") y la negación ("ne" va con "pas"); no obtendríamos eso de los bigramas regulares solos.

23.1.4 Suavizando modelos de *n*-gramas

Los *n*-gramas de alta frecuencia como "of the" tienen altos conteos en el corpus de entrenamiento, por lo que su estimación de probabilidad es probable que sea precisa: con un corpus de entrenamiento diferente obtendríamos una estimación similar. Los *n*-gramas de baja frecuencia tienen conteos bajos que están sujetos a ruido aleatorio, tienen alta varianza. Nuestros modelos funcionarán mejor si podemos suavizar esa varianza.

Además, siempre existe la posibilidad de que se nos pida evaluar un texto que contenga una palabra desconocida o **fuera del vocabulario**: una que nunca apareció en el corpus de entrenamiento. Pero sería un error asignar a tal palabra una probabilidad de cero, porque entonces la probabilidad de toda la sentencia, $P(w_{1:N})$, sería cero.

Fuera de vocabulario

Una forma de modelar palabras desconocidas es modificar el corpus de entrenamiento reemplazando palabras frecuentes con un símbolo especial, con frecuencia <UNK>. Podríamos decidir de antemano mantener sólo, digamos, las 50 000 palabras más comunes, o todas las palabras con una frecuencia mayor al 0.0001 %, y reemplazar las demás con <UNK>. Luego calculamos los conteos de *n*-gramas para el corpus como de costumbre, tratando <UNK> como cualquier otra palabra. Cuando una palabra desconocida aparece en un conjunto de prueba, buscamos su probabilidad bajo <UNK>. A veces se utilizan diferentes símbolos de palabras desconocidas para diferentes tipos. Por ejemplo, una cadena de dígitos podría ser reemplazada por <NUM>, o una dirección de correo electrónico con <EMAIL>. (Notamos que también es aconsejable tener un símbolo especial, como <S>, para marcar el inicio (y el final) de un texto. De esa manera, cuando la fórmula para las probabilidades de bigramas pregunta por la palabra antes de la primera palabra, la respuesta es <S>, no un error).

Incluso después de que hemos manejado palabras desconocidas, tenemos el problema de los *n*-gramas no vistos. Por ejemplo, un texto de prueba podría contener la frase "colorless aquamarine ideas", tres palabras que podemos haber visto de manera individual en el corpus de entrenamiento, pero nunca en ese orden exacto. El problema es que algunos *n*-gramas de baja probabilidad aparecen en el corpus de entrenamiento, mientras que otros *n*-gramas de baja probabilidad ocurren para no aparecer en absoluto. No queremos que algunos de ellos tengan una probabilidad cero mientras que otros tienen una pequeña probabilidad positiva; queremos aplicar **suavizado** a todos los *n*-gramas similares, reservando parte de la masa de probabilidad del modelo para los *n*-gramas nunca vistos, para reducir la varianza del modelo.

Suavizado

El tipo más simple de suavizado fue sugerido por Pierre-Simon Laplace en el siglo XVIII para estimar la probabilidad de eventos raros, como que el Sol no salga mañana. La (incorrecta) teoría del Sistema Solar de Laplace sugería que tenía alrededor de $N = 2$ millones de días de antigüedad. Según los datos, hubo cero de dos millones de días en que el Sol no salió, sin embargo, no queremos decir que la probabilidad sea cero. Laplace mostró que si adoptamos un prior uniforme y lo combinamos con la evidencia hasta ahora, obtenemos una mejor estimación de $1/(N + 2)$ para la probabilidad de que el Sol no salga mañana, ya sea que lo haga o no (eso es el 2 en el denominador) y un prior uniforme dice que es tan probable como no (eso es el 1 en el numerador). El suavizado de Laplace (también llamado suavizado de suma uno) es un paso en la dirección correcta, pero para muchas aplicaciones de lenguaje natural se desempeña mal.

Otra opción es un **modelo de retroceso**, en el que comenzamos estimando recuentos de *n*-gramas, pero para cualquier secuencia particular que tenga un recuento bajo (o cero), retrocedemos a $(n - 1)$-gramas. El **suavizado por interpolación lineal** es un modelo de retroceso que combina

Modelo de retroceso

Suavizado por interpolación lineal

trigramas, bigramas y modelos unigramas por interpolación lineal. Se define la estimación de probabilidad como

$$\hat{P}(c_i|c_{i-2:i-1}) = \lambda_3 P(c_i|c_{i-2:i-1}) + \lambda_2 P(c_i|c_{i-1}) + \lambda_1 P(c_i),$$

donde $\lambda_3 + \lambda_2 + \lambda_1 = 1$. Los valores del parámetro λ_i pueden ser fijos o permiten ser entrenados con un algoritmo de expectativa-maximización. También es posible que los valores de λ_i dependan de los recuentos: si tenemos un alto recuento de trigramas, entonces el peso se incrementa; si sólo hay un recuento bajo, entonces el peso en los modelos de bigrama y unigrama se incrementa.

Un grupo de investigadores ha desarrollado técnicas de suavizado cada vez más sofisticadas (como Witten-Bell y Kneser-Ney), mientras que otro grupo sugiere recopilar un corpus más grande para que incluso las técnicas de suavizado simples funcionen bien (un enfoque de este tipo se llama "retroceso ingenuo"). Ambos están llegando al mismo objetivo: reducir la variación en el modelo de lenguaje.

23.1.5 Representaciones de palabras

Los *N*-gramas permiten darnos un modelo que predice con precisión la probabilidad de secuencias de palabras, diciéndonos que, por ejemplo, "a black cat" es una frase en inglés más probable que "cat black a" porque "a black cat" aparece en cerca de 0.000014 % de los trigramas en un corpus de entrenamiento, mientras que "cat black a" no aparece en absoluto. Todo lo que el modelo de palabras *n*-grama sabe lo aprendió de recuentos de secuencias de palabras específicas.

Pero un hablante nativo de inglés contaría una historia diferente: "a black cat" es válido porque sigue un patrón familiar (artículo-adjetivo-sustantivo), mientras que "cat black a" no lo hace.

Ahora, considere la frase "the fulvous kitten". Un hablante de inglés podría reconocer esto como el patrón artículo-adjetivo-sustantivo (incluso un hablante que no sabe que "fulvo" significa "amarillo parduzco" podría reconocer que en inglés casi todas las palabras que terminan en "*-ous*" son adjetivos). Además, el hablante reconocería la estrecha conexión sintáctica entre "a" y "the", así como la estrecha relación semántica entre "cat" y "kitten". Por lo tanto, la aparición de "a black cat" en los datos es evidencia, mediante generalización, de que "the fulvous kitten" también es inglés válido.

El modelo *n*-grama pierde esta generalización porque es un modelo *atómico*: cada palabra es un átomo, distinto de todas las demás palabras, sin estructura interna. Hemos visto a lo largo de este libro que los modelos *factorizados* o *estructurados* permiten un mayor poder expresivo y una mejor generalización. Veremos en la Sección 24.1 que un modelo factorizado llamado **incrustaciones de palabras** (embeddings) da una mejor capacidad para generalizar.

Diccionario
WordNet

Un tipo de modelo de palabras estructurado es un **diccionario**, en la mayoría de los casos construido mediante trabajo manual. Por ejemplo, **WordNet** es un diccionario de código abierto, curado a mano en formato legible por máquina que ha demostrado ser útil para muchas aplicaciones[1] de lenguaje natural. A continuación se muestra la entrada de WordNet para "kitten":

```
"kitten" <noun.animal> ("young domestic cat") IS A: young_mammal

"kitten" <verb.body> ("give birth to kittens")
  EXAMPLE: "our cat kittened again this year"
```

WordNet le ayudará a separar los sustantivos de los verbos y a obtener las categorías básicas (un gatito es un mamífero joven, que es un mamífero, que es un animal), pero no le dirá los detalles de cómo se ve o se comporta un gatito. WordNet le dirá que dos subclases de gato son el *Gato siamés* y el *Gato Manx*, pero no le dirá más sobre las razas.

[1] Incluso aplicaciones de visión por computadora: WordNet proporciona el conjunto de categorías utilizadas por ImageNet.

Etiqueta	Palabra	Descripción	Etiqueta	Palabra	Descripción
CC	*and*	Conjunción coordinante	PRP$	*your*	Pronombre posesivo
CD	*three*	Número cardinal	RB	*quickly*	Adverbio
DT	*the*	Determinante	RBR	*quicker*	Adverbio, comparativo
EX	*there*	Existencial ahí	RBS	*quickest*	Adverbio, superlativo
FW	*per se*	Palabra extranjera	RP	*off*	Partícula
IN	*of*	Preposición	SYM	+	Símbolo
JJ	*purple*	Adjetivo	TO	*to*	a
JJR	*better*	Adjetivo, comparativo	UH	*eureka*	Interjección
JJS	*best*	Adjetivo, superlativo	VB	*talk*	Verbo, forma base
LS	*1*	Marcador de elemento de lista	VBD	*talked*	Verbo, tiempo pasado
MD	*should*	Modal	VBG	*talking*	Verbo, gerundio
NN	*kitten*	Sustantivo, singular o masa	VBN	*talked*	Verbo, participio pasado
NNS	*kittens*	Sustantivo, plural	VBP	*talk*	Verbo, no-3ra persona singular
NNP	*Ali*	Nombre propio, singular	VBZ	*talks*	Verbo, 3ra persona singular
NNPS	*Fords*	Sustantivo propio, plural	WDT	*which*	Determinante wh
PDT	*all*	Predeterminante	WP	*who*	Pronombre wh
POS	*'s*	Final posesivo	WP$	*whose*	Pronombre posesivo wh
PRP	*you*	Pronombre personal	WRB	*where*	Adverbio wh
$	*$*	Signo de dólar	#	#	Signo de libra
“	‘	Comillas izquierdas	”	’	Comillas derechas
(	[	Paréntesis izquierdo	)	]	Paréntesis derecho
,	,	Coma	.	!	Final de la frase
:	;	Puntuación media de la frase			

Figura 23.1 Etiquetas de parte del discurso (con una palabra de ejemplo para cada etiqueta) para el corpus Penn Treebank (Marcus *et al.,* 1993). Aquí “VBZ” corresponde a “tercera persona singular presente”.

23.1.6 Etiquetado de parte de discurso (POS)

Una forma básica de categorizar palabras es por su **parte del discurso (POS)**, también llamada **categoría lexical** o **etiqueta**: *sustantivo, verbo, adjetivo*, etc. Las partes del discurso permiten a los modelos de lenguaje capturar generalizaciones como “los adjetivos con frecuencia vienen antes que los sustantivos en inglés”. (En otros idiomas, como el francés, es al revés (generalmente)).

Parte del discurso (POS)

Todos están de acuerdo en que “sustantivo” y “verbo” son partes del discurso, pero cuando entramos en detalles no hay una lista definitiva. La Figura 23.1 muestra las 45 etiquetas utilizadas en el **Penn Treebank**, un corpus de más de tres millones de palabras de texto anotado con etiquetas de parte del discurso. Como veremos más adelante, el Penn Treebank también anota muchas sentencias con árboles de análisis sintáctico, de donde el corpus obtiene su nombre. Aquí hay un extracto que dice que “from” está etiquetado como una preposición (IN), “the” como un determinante (DT), y así sucesivamente:

Penn Treebank

Desde	el	principio	,	se	necesitó	una	persona	con	grandes	cualidades	para	tener	éxito
IN	DT	NN	,	PRP	VBD	DT	NN	IN	JJ	NNS	TO	VB	

Etiquetado de parte del discurso

La tarea de asignar una parte del discurso a cada palabra en una sentencia se llama **etiquetado de parte del discurso**. Aunque no es muy interesante en sí mismo, es un primer paso útil en muchas otras tareas de NLP, como la respuesta a preguntas o la traducción. Incluso para una tarea simple como la síntesis de texto a voz, es importante saber que en inglés el sustantivo "record" se pronuncia de manera diferente al verbo "record". En esta sección veremos cómo se aplican dos modelos familiares a la tarea de etiquetado y, en el Capítulo 24 consideraremos un tercer modelo.

Un modelo común para el etiquetado POS es el **modelo de Markov oculto (HMM)**. Recordemos de la Sección 14.3 que un modelo de Markov oculto toma una secuencia temporal de observaciones de evidencia y predice los estados ocultos más probables que podrían haber producido esa secuencia. En el ejemplo de HMM en la página 473, la evidencia consistía en observaciones de una persona llevando un paraguas (o no), y el estado oculto era lluvia (o no) en el mundo exterior. Para el etiquetado POS, la evidencia es la secuencia de palabras, $W_{1:N}$, y los estados ocultos son las categorías léxicas, $C_{1:N}$.

El HMM es un modelo generativo que dice que la forma de producir lenguaje es comenzar en un estado, como (en inglés) IN, el estado para las preposiciones, y luego tomar dos decisiones: qué palabra (como *from*) debe emitirse, y qué estado (como DT) debe venir a continuación. El modelo no considera ningún contexto aparte del estado actual de la parte del discurso, ni tiene idea de lo que la sentencia está tratando de transmitir. Y, sin embargo, es un modelo útil, si aplicamos el **algoritmo de Viterbi** (Sección 14.2.3) para encontrar la secuencia más probable de estados ocultos (etiquetas), encontramos que el etiquetado logra una precisión muy alta; alrededor de 97 %.

Algoritmo de Viterbi

Para crear un HMM para el etiquetado POS, necesitamos el modelo de transición, que da la probabilidad de que una parte del discurso siga a otra, $\mathbf{P}(C_t|C_{t-1})$, y el modelo sensor, $\mathbf{P}(W_t|C_t)$. Por ejemplo, $\mathbf{P}(C_t=VB|C_{t-1}=MD) = 0.8$ significa que dado un verbo modal (como *would*), podemos esperar que la siguiente palabra sea un verbo (como *think*) con probabilidad 0.8. ¿De dónde viene el número 0.8? Al igual que con los modelos *n*-grama, de los recuentos en el corpus, con el suavizado apropiado. Resulta que hay 13 124 instancias de *MD* en el Penn Treebank, y 10 471 de ellas son seguidas por un *VB*.

Para el modelo sensor, $P(W_t=would|C_t=MD) = 0.1$ significa que cuando estamos eligiendo un verbo modal, elegiremos *would* 10 % de las veces. Estos números también provienen del corpus cuentas, con suavizado.

Una debilidad de los modelos HMM es que todo lo que sabemos sobre el lenguaje tiene que ser expresado en términos de los modelos de transición y sensor. La parte del discurso para la palabra actual está determinada por las probabilidades en estos dos modelos, así como por la parte del discurso de la palabra anterior. No hay una forma fácil para que un desarrollador de sistemas diga, por ejemplo, que *cualquier* palabra que termine en "ous" quizá sea un adjetivo, ni que en la frase "attorney general", *fiscal* es un sustantivo, no un adjetivo.

Por fortuna, la **regresión logística** sí tiene la capacidad de representar información como esta. Recuerde de la Sección 19.6.5 que en un modelo de regresión logística, la entrada es un vector, **x**, de valores de características. Luego tomamos el producto escalar, **w**·**x**, de esas características con un vector de pesos **w** preentrenado, y transformamos esa suma en un número entre 0 y 1 que es interpretado como la probabilidad de que la entrada sea un ejemplo positivo de una categoría.

Los pesos en el modelo de regresión logística corresponden a cuán predictiva es cada característica para cada categoría; los valores de los pesos son aprendidos por descenso de gradiente. Para el etiquetado POS construiríamos 45 modelos de regresión logística diferentes, uno para cada parte del discurso, y preguntaríamos a cada modelo cuán probable es que la palabra de ejemplo sea miembro de esa categoría, dadas los valores de las características para esa palabra en su contexto particular.

La pregunta entonces es ¿qué deberían ser las características? Los etiquetadores de POS suelen utilizar características de valor binario que codifican información sobre la palabra que se está etiquetando, w_i (y, quizás, otras palabras cercanas), así como la categoría que se asignó a la palabra anterior, c_{i-1} (y, quizás, la categoría de palabras anteriores). Las características dependen de la identidad exacta de una palabra, algunos aspectos de la forma en que se deletrea o algún atributo de una entrada de diccionario. Un conjunto de características de etiquetado POS podría incluir:

w_{i-1} = "I"	w_{i+1} = "for"
w_{i-1} = "you"	c_{i-1} = IN
w_i termina con "ous"	w_i contiene un guión
w_i termina con "ly"	w_i contiene un dígito
w_i empieza con "un"	w_i está todo en mayúsculas
w_{i-2} = "to" y c_{i-1} = VB	w_{i-2} tiene el atributo PRESENTE
w_{i-1} = "I" y w_{i+1} = "to"	w_{i-2} tiene el atributo PASADO

Por ejemplo, la palabra "walk" puede ser un sustantivo o un verbo, pero en "I walk to school", la característica en la última fila, columna izquierda podría ser utilizada para clasificar "walk" como un verbo (VBP). Como otro ejemplo, la palabra "cut" puede ser un sustantivo (NN), verbo en pasado (VBD) o verbo en presente (VBP). Dada la frase "Yesterday I cut the rope", la característica en la última fila, columna derecha podría ayudar a etiquetar "cut" como VBD, mientras que en la frase "Now I cut the rope", la característica por encima de esa podría ayudar a etiquetar "cut" como VBP.

En total, podría haber un millón de características, pero para cualquier palabra dada, sólo unas pocas docenas serán distintas de cero. Las características suelen ser creadas a mano por un diseñador de sistemas humano que piensa en plantillas de características interesantes.

La regresión logística no tiene la noción de una secuencia de entradas: le da un sólo vector de características (información sobre una sola palabra) y produce una salida (una etiqueta). Pero podemos forzar a la regresión logística a manejar una secuencia con una **búsqueda voraz**: empezar eligiendo la categoría más probable para la primera palabra y proceder al resto de las palabras en orden de izquierda a derecha. En cada paso la categoría c_i se asigna de acuerdo con

$$c_i = \underset{c' \in Categorías}{\text{argmax}} \; P(c' | w_{1:N}, c_{1:i-1}).$$

Es decir, se permite al clasificador mirar cualquiera de las características no categorizadas para cualquiera de las palabras en cualquier lugar de la frase (porque estas características están todas fijas), así como cualquier categoría previamente asignada.

Note que la búsqueda voraz hace una elección de categoría definitiva para cada palabra, y luego pasa a la siguiente palabra; si esa elección es contradicha por evidencia más adelante en la frase, no hay posibilidad de volver atrás y revertir la elección. Eso hace que el algoritmo sea rápido. El algoritmo de Viterbi, en contraste, mantiene una tabla de todas las posibles elecciones de categoría en cada paso, y siempre tiene la opción de cambiar. Eso hace que el algoritmo sea más preciso, pero más lento. Para ambos algoritmos, un compromiso es una **búsqueda en haz**, en la que consideramos cada posible categoría en cada paso de tiempo, pero luego sólo mantenemos las b etiquetas más probables, descartando las otras etiquetas menos probables. Cambiar b compensa velocidad contra precisión.

Naive Bayes y los modelos de Markov ocultos son **modelos generativos** (ver Sección 20.2.3). Es decir, aprenden una distribución de probabilidad conjunta, $\mathbf{P}(W,C)$, y podemos generar una frase aleatoria muestreando de esa distribución de probabilidad para obtener una primera palabra (con categoría) de la frase, y luego añadiendo palabras, una a la vez.

Por otro lado, la regresión logística es un **modelo discriminativo**. Aprende una distribución de probabilidad condicional $\mathbf{P}(C|W)$, lo que significa que permite asignar categorías dada una secuencia de palabras, pero no permite generar sentencias aleatorias. En general, los investigadores han encontrado que los modelos discriminativos tienen una tasa de error más baja, quizá porque modelan la salida prevista de manera directa y porque facilitan que un analista cree características adicionales. Sin embargo, los modelos generativos tienden a converger de manera más rápida, por lo que son preferidos cuando el tiempo de entrenamiento disponible es corto o cuando hay datos de entrenamiento limitados.

23.1.7 Comparando modelos de lenguaje

Para tener una idea de cómo son los diferentes modelos n-grama, construimos modelos unigrama (es decir, bolsa de palabras), bigrama, trigrama y 4-grama sobre las palabras de este libro y luego muestreamos de manera aleatoria secuencias de palabras de cada uno de los cuatro modelos:

- $n = 1$: *son lógicos como son confusos a un agente objetivo que puede ser correcto*
- $n = 2$: *los sistemas son muy similares el enfoque computacional estaría representado*
- $n = 3$: *la planificación y programación están integradas el éxito del modelo Bayes ingenuo es*
- $n = 4$: *aprovechando la estructura de las redes bayesianas y desarrollando varios lenguajes para escribir "plantillas" con variables lógicas, a partir de las cuales se podrían construir de manera automática grandes redes para cada instancia del problema*

De esta pequeña muestra debería quedar claro que el modelo unigrama es una aproximación muy pobre del idioma en general o de un libro de texto de IA en particular, y que el modelo 4-grama es imperfecto, pero mucho mejor. A continuación, para demostrar cómo las muestras cambian entre las fuentes de entrenamiento (y no sólo por diversión) agregamos el texto de la Biblia del Rey James al modelo 4-grama dando como resultado estas muestras aleatorias:

- *Demuestra que cualquier problema 3-SAT permite reducirse a otros más simples utilizando las leyes de tu Dios.*
- *Amos, den a sus sirvientes lo que es verdadero si y sólo si tanto* P *como* Q *en cualquier modelo* m *por un simple experimento: ponga su mano en ello, usted y sus hogares porque es agradable.*
- *Muchos suplicarán al SEÑOR su Dios, diciendo, No. Pero ignoraremos este problema por ahora. Los capítulos 7 y 8 sugieren métodos para representar de manera compacta estados de creencia muy grandes.*
- *Y sucedió, como si no tuviera sucesores.*
- *La estimación de utilidad directa es sólo una instancia del algoritmo general en el que se construyen nuevos símbolos de función "sobre la marcha". Por ejemplo, el primer hijo del Espíritu Santo.*

Hay un límite para los modelos n-grama: a medida que n aumenta, producirán un lenguaje más fluido, pero tienden a reproducir pasajes largos de sus datos de entrenamiento de manera literal, en lugar de generar texto novedoso. Los modelos de lenguaje con representaciones más complejas de palabras y contexto pueden hacerlo mejor. El resto de este capítulo demuestra cómo la **gramática** permite mejorar un modelo de lenguaje. El Capítulo 24 muestra cómo los métodos de aprendizaje profundo recién han producido modelos de lenguaje impresionantes. Un modelo de aprendizaje profundo, GPT-2, produce muestras fluidas cuando se le da un indicio. Le dimos a GPT-2 las dos primeras sentencias de este párrafo como indicio, y produjo las siguientes dos muestras:

- *Numerosos estudios han demostrado que los mejores modelos de lenguaje que permiten generar lenguaje son aquellos que utilizan redes neuronales, que en particular utilizan aprendizaje profundo para modelar el habla. He utilizado uno de estos modelos para generar cientos de sentencias en varios idiomas, y tiene la capacidad de generar traducciones casi perfectas de mis sentencias de entrenamiento. Es un gran modelo para tener, y le ahorrará mucho tiempo en el futuro cuando necesite traducir entre muchos idiomas.*

- *Estos modelos también permiten un mejor rendimiento si sus muestras se eligen al azar, en lugar de por un experto en el dominio. Sin embargo, como regla general los modelos n-grama y los modelos de lenguaje que incluyen una gran cantidad de características tienden a producir un texto mejor, ya que estas características contribuyen más al aprendizaje del lenguaje. Para medir el rendimiento de un modelo, he hecho algunas sentencias de muestra que me gustaría entrenar para entender mejor los modelos de habla y lenguaje. Puede leerlos para tener una idea de cómo el modelo predice.*

Vemos que estos pasajes son diversos y gramaticalmente fluidos; además, se adhieren a temas relevantes para las sentencias de inicio. Aunque las sentencias no se basan unas en otras para avanzar una tesis coherente. El modelo de lenguaje GPT-2 se conoce como un **modelo transformador**, el cual será revisado en la Sección 24.4; más ejemplos de GPT-2 se encuentran en la Figura 24.14. Otro modelo transformador es el Lenguaje Transformador Condicional, CTRL. Permite ser controlado de manera más flexible; en las siguientes muestras se pidió a CTRL que generara texto en la categoría de *reseñas de productos*, con una calificación de 1 y de 4 (de 5): calificación especificada (de 5):

- **1.0:** *Compré esto para mi hijo que es un gran fanático del programa. Estaba tan emocionado de recibirlo y cuando lo abrió, todos estábamos muy decepcionados. La calidad del producto es terrible. Parece algo que comprarías en una tienda de un dólar.*
- **4.0:** *Compré esto para mi esposo y le encanta. Tiene una muñeca pequeña, por lo que es difícil encontrar relojes que le queden bien. Este le queda a la perfección.*

23.2 Gramática

En el Capítulo 7 usamos la Forma de Backus–Naur (BNF) para escribir una gramática para el lenguaje de la lógica de primer orden. Una **gramática** es un conjunto de reglas que define la estructura de árbol de las frases permitidas, un **lenguaje** es el conjunto de sentencias que siguen esas reglas.

Los lenguajes naturales no funcionan como el lenguaje formal de la lógica de primer orden, no tienen un límite duro entre las sentencias permitidas y no permitidas, ni tienen una única estructura de árbol definitiva para cada sentencia. Sin embargo, la estructura jerárquica *es* importante en el lenguaje natural. La palabra "Stocks" en "Stocks rallied on Monday" no es sólo una palabra, ni es sólo un *sustantivo*; en esta sentencia también comprende una *frase nominal*, que es el sujeto de la siguiente *frase verbal*. Las **categorías sintácticas** como *frase nominal* o *frase verbal* ayudan a limitar las palabras probables en cada punto dentro de una sentencia, y la **estructura de la frase** proporciona un marco para el significado o **semántica** de la sentencia.

Categorías sintácticas

Estructura de la frase

Existen múltiples modelos de lenguaje en competencia basados en la idea de la sintaxis jerárquica estructural; en esta sección describiremos un modelo popular llamado **gramática probabilística libre de contexto**, o PCFG. Una gramática probabilística asigna una probabilidad a cada cadena; "libre de contexto" significa que cualquier regla permite usarse en cualquier contexto: las reglas para una frase nominal al principio de una sentencia son las mismas que para otra frase nominal más adelante en la sentencia, y si la misma frase ocurre en dos ubicaciones, debe tener la misma probabilidad cada vez. Definiremos una gramática PCFG para un pequeño fragmento de inglés que es adecuado para la comunicación entre agentes que exploran el mundo del wumpus. Llamamos a este lenguaje $\mathcal{E}_0$ (ver Figura 23.2). Una regla gramatical como

Gramática probabilística libre de contexto

$$\begin{array}{lll} \mathit{Adjs} & \rightarrow \mathit{Adjetivo} & [0.80] \\ & \mid \mathit{Adjetivo}\ \mathit{Adjs} & [0.20] \end{array}$$

significa que la categoría sintáctica *Adjs* consiste en un solo *Adjetivo*, con probabilidad 0.80, o en un *Adjetivo* seguido de una cadena que constituye un *Adjs*, con probabilidad 0.20.

S	→	*NP VP*	[0.90]	Yo + siento una brisa
	|	*S Conj S*	[0.10]	Siento una brisa + y + Huele mal
NP	→	*Pronombre*	[0.25]	Yo
	|	*Nombre*	[0.10]	Ali
	|	*Sustantivo*	[0.10]	pozos
	|	*Artículo Sustantivo*	[0.25]	el + wumpus
	|	*Artículo Adjs Sustantivo*	[0.05]	el + muerto apestoso + wumpus
	|	*Dígito Dígito*	[0.05]	3 4
	|	*NP PP*	[0.10]	el wumpus + en 1 3
	|	*NP RelClause*	[0.05]	el wumpus + que huele mal
	|	*NP Conj NP*	[0.05]	el wumpus + y + yo
VP	→	*Verbo*	[0.40]	apesta
	|	*VP NP*	[0.35]	siento + una brisa
	|	*VP Adjetivo*	[0.05]	huele + muerto
	|	*VP PP*	[0.10]	está + en 1 3
	|	*VP Adverbio*	[0.10]	ir + adelante
Adjs	→	*Adjetivo*	[0.80]	maloliente
	|	*Adjetivo Adjs*	[0.20]	huele mal + muerto
PP	→	*Prep NP*	[1.00]	a + el este
RelClause	→	*RelPro VP*	[1.00]	eso + huele mal

Figura 23.2 Gramática para $\mathcal{E}_0$, con frases de ejemplo para cada regla. Las categorías sintácticas son sentencia (*S*), frase nominal (*NP*), frase verbal (*VP*), lista de adjetivos (Adjs), frase preposicional (*PP*), y cláusula relativa (*RelClause*).

Sustantivo	→	**hedor** [0.05]| **brisa** [0.10]| **wumpus** [0.15]| **pozos** [0.05]| ...
Verbo	→	**es** [0.10]| **siento** [0.10]| **huele** [0.10]| **apesta** [0.05]| ...
Adjetivo	→	**derecha** [0.10]| **muerto** [0.05]| **maloliente** [0.02]| **ventoso** [0.02] ...
Adverbio	→	**aquí** [0.05]| **adelante** [0.05]| **cerca** [0.02]| ...
Pronombre	→	**me** [0.10]| **tú** [0.03]| **yo** [0.10]| **eso** [0.10]| ...
RelPro	→	**eso** [0.40]| **cual** [0.15]| **quien** [0.20]| **a quien** [0.02]| ...
Nombre	→	**Ali** [0.01]| **Bo** [0.01]| **Boston** [0.01]| ...
Artículo	→	**el** [0.40]| **un** [0.30]| **una** [0.10]| **cada** [0.05]| ...
Prep	→	**a** [0.20]| **en** [0.10]| **sobre** [0.05]| **cerca** [0.10]| ...
Conj	→	**y** [0.50]| **o** [0.10]| **pero** [0.20]| **aún** [0.02]| ...
Dígito	→	**0** [0.20]| **1** [0.20]| **2** [0.20]| **3** [0.20]| **4** [0.20]| ...

Figura 23.3 El léxico para $\mathcal{E}_0$. *RelPro* es la abreviatura de pronombre relativo, *Prep* para preposición, y *Conj* para conjunción. La suma de las probabilidades para cada categoría es 1.

Desafortunadamente, la gramática **sobregenera**: es decir, genera sentencias que no son gramaticales, como "Yo ir yo". También **subgenera**: hay muchas sentencias en inglés que rechaza, como "I think the wumpus is smelly". Veremos cómo aprender una mejor gramática más tarde; por ahora nos concentramos en lo que podemos hacer con esta gramática muy simple.

Sobregenera
Subgenera

23.2.1 El léxico de $\mathcal{E}_0$

El **léxico**, o lista de palabras permitidas, se define en la Figura 23.3. Cada una de las categorías léxicas termina en... para indicar que hay otras palabras en la categoría. Para los sustantivos, nombres, verbos, adjetivos y adverbios es inviable, incluso, en principio listar todas las palabras. No sólo hay decenas de miles de miembros en cada clase, sino que los nuevos, como *humblebrag* o *microbioma*, se están añadiendo de manera constante. Estas cinco categorías se llaman **clases abiertas**. Los sustantivos, pronombres relativos, artículos, preposiciones y conjunciones se llaman **clases cerradas**; tienen un pequeño número de palabras (una docena, aproximada), y cambian a lo largo de los siglos, no meses. Por ejemplo, "thee" y "thou" eran pronombres comúnmente utilizados en el siglo XVII, estaban en declive en el siglo XIX, y hoy en día sólo se ven en la poesía y algunos dialectos regionales.

Léxico
Clases abiertas
Clases cerradas

23.3 Análisis sintáctico

El **análisis sintáctico** es el proceso de analizar una cadena de palabras para descubrir su estructura de frases, según las reglas de una gramática. Podemos pensar en ello como una **búsqueda** de un árbol de análisis válido cuyas hojas son las palabras de la cadena. La Figura 23.4 muestra que podemos empezar con el símbolo *S* y buscar de arriba hacia abajo, o podemos empezar con las palabras y buscar de abajo hacia arriba. Sin embargo, las estrategias de análisis puro de arriba hacia abajo o de abajo hacia arriba son ineficientes, porque terminan repitiendo esfuerzo en áreas del espacio de búsqueda que conducen a callejones sin salida. Considere las siguientes dos sentencias:

Análisis sintáctico

Have the students in section 2 of Computer Science 101 take the exam.
Have the students in section 2 of Computer Science 101 taken the exam?

Aunque comparten las primeras 10 palabras, estas sentencias tienen análisis muy diferentes, porque la primera es un comando y la segunda es una pregunta. Un algoritmo de análisis de izquierda a derecha tendría que adivinar si la primera palabra es parte de un comando o una pregunta y no podrá saber si la suposición es correcta hasta al menos la undécima palabra, take or taken (*tomar* o *tomado*). Si el algoritmo adivina mal, tendrá que retroceder hasta la primera palabra y reanalizar toda la sentencia bajo la otra interpretación.

Para evitar esta fuente de ineficiencia podemos usar **programación dinámica**: cada vez que analizamos una subcadena, almacenamos los resultados para no tener que reanalizarla más tarde. Por ejemplo, una vez que descubrimos que "los estudiantes de la sección 2 de Ciencias de la Computación 101" es un *NP*, podemos registrar ese resultado en una estructura de datos conocida como **grafo**. Un algoritmo que hace esto se llama **grafo analizador**. Como estamos tratando con gramáticas libres de contexto, cualquier frase que se encontró en el contexto de una rama del árbol de búsqueda funciona igual de bien en cualquier otra rama del árbol de búsqueda. Existen muchos tipos de analizadores de grafos; describimos una versión probabilística de un algoritmo de análisis de grafos de abajo hacia arriba llamado **algoritmo CYK**, por sus inventores, Ali Cocke, Daniel Younger y Tadeo Kasami.[2]

Analizador sintáctico de grafos
Algoritmo CYK

[2] A veces los autores son acreditados en el orden CKY.

Lista de ítems	Regla
S	
NP VP	*S* →*NP VP*
NP VP Adjective	*VP* →*VP Adjective*
NP Verb Adjective	*VP* →*Verb*
NP Verb **dead**	*Adjective* →**dead**
NP **is dead**	*Verb* →**is**
Article Noun **is dead**	*Article Noun*
Article **wumpus is dead**	*Noun* →**wumpus**
the wumpus is dead	*Article* →**the**

Figura 23.4 Analizando la cadena "The wumpus is dead" como una sentencia, de acuerdo con la gramática $\mathcal{E}_0$. Visto como un análisis de arriba hacia abajo, empezamos con *S*, y en cada paso coincidimos con un no terminal *X* con una regla de la forma ($X \rightarrow Y...$) y reemplazamos *X* en la lista de elementos con *Y...*; por ejemplo, reemplazando *S* con la secuencia *NP VP*. Visto como un análisis de abajo hacia arriba, empezamos con las palabras "the wumpus is dead", y en cada paso coincidimos con una cadena de tokens como (*Y...*) contra una regla de la forma ($X \rightarrow Y...$) y reemplazamos los tokens con *X*; por ejemplo, reemplazando "the" con *Artículo* o *Artículo Sustantivo* con *NP*.

El algoritmo CYK se muestra en la Figura 23.5. Requiere una gramática con todas las reglas en uno de dos formatos muy específicos: reglas léxicas de la forma $X \rightarrow$ **palabra** $[p]$, y reglas sintácticas de la forma $X \rightarrow Y\,Z[p]$, con dos categorías en el lado derecho. Este formato de gramática, llamado **Forma Normal de Chomsky**, parece restrictivo, pero no lo es: cualquier gramática libre de contexto permite ser transformada de manera automática en Forma Normal de Chomsky. El Ejercicio 23.CNFX te guía a través del proceso.

Forma Normal de Chomsky

El algoritmo CYK utiliza un espacio de $O(n^2m)$ para las tablas *P* y *T*, donde *n* es el número de palabras en la sentencia y *m* es el número de símbolos no terminales en la gramática, y toma tiempo $O(n^3m)$. Si queremos un algoritmo que esté garantizado para funcionar para todas las gramáticas libres de contexto posibles, entonces no podemos hacerlo mejor que eso. Aunque en realidad sólo queremos analizar lenguajes naturales, no todas las gramáticas posibles. Los lenguajes naturales han evolucionado para ser fáciles de entender en tiempo real, no para ser lo más complicados posible, por lo que parece que deberían ser susceptibles a un algoritmo de análisis más rápido.

Para intentar llegar a $O(n)$, podemos aplicar la búsqueda A^* de una manera directa: cada estado es una lista de elementos (palabras o categorías), como se muestra en la Figura 23.4. El estado inicial es una lista de palabras, mientras que un estado objetivo es el único elemento *S*. El costo de un estado es el inverso de su probabilidad, según las reglas aplicadas hasta ahora, y hay varias heurísticas para estimar la distancia restante hasta el objetivo; las mejores heurísticas en uso actual provienen del aprendizaje automático aplicado a un corpus de sentencias.

Con el algoritmo A^* no tenemos que buscar en todo el espacio de estados, y tenemos la garantía de que el primer análisis encontrado será el más probable (asumiendo una heurística admisible). Esto suele ser más rápido que CYK, pero (dependiendo de los detalles de la gramática) todavía más lento que $O(n)$. Un ejemplo de resultado de un análisis se muestra en la Figura 23.6.

Al igual que con el etiquetado de partes del habla, podemos usar una **búsqueda de haz** para el análisis, donde en cualquier momento consideramos sólo las *b* alternativas de análisis más probables. Esto significa que no estamos garantizando encontrar el análisis sintáctico con mayor

```
function CYK-PARSE(words, grammar) returns una tabla de árboles de análisis
  inputs: words, una lista de palabras
          grammar, una estructura con LEXICALRULES y GRAMMARRULES
  T ← una tabla        // T[X,i,k] is most probable X tree spanning words_{i:k}
  P ← una tabla, inicializada en 0        // P[X,i,k] la probabilidad de T[X,i,k]
  // Insertar categorías léxicas para cada palabra.
  for i = 1 to LEN(words) do
    for each (X,p) in grammar.LEXICALRULES(words_i) do
      P[X,i,i] ←p
      T[X,i,i] ←TREE(X,words_i)
  // Construir X_{i:k} a partir de Y_{i:j} + Z_{j+1:k}, los tramos más cortos primero.
  for each (i,j,k) in SUBSPANS(LEN(words)) do
    for each (X,Y,Z,p) in grammar.GRAMMARRULES do
      PYZ ← P[Y,i,j] × P[Z,j+1,k] × p
      if PYZ > P[X,i,k] do
        P[X,i,k] ← PYZ
        T[X,i,k] ←TREE(X,T[Y,i,j], T[Z,j+1,k])
  return T

function SUBSPANS(N) yields (i,j,k) tuplas
  for length = 2 to N do
    for i = 1 to N + 1 − length do
      k ← i + length – 1
      for j = i to k − 1 do
        yield (i, j, k)
```

Figura 23.5 El algoritmo CYK para el análisis. Dada una secuencia de palabras, encuentra el árbol de análisis más probable para la secuencia y sus subsecuencias. La tabla $P[X,i,k]$ da la probabilidad del árbol más probable de la categoría X que abarca las $words_{i:k}$. La tabla de salida $T[X,i,k]$ contiene el árbol más probable de la categoría X que abarca incluso las posiciones i a k. La función SUBSPANS devuelve todas las tuplas (i,j,k) que cubren un tramo de $words_{i:k}$, con $i \leq j < k$, listando las tuplas por el aumento de la longitud del tramo $i{:}k$, de modo que cuando vamos a combinar dos tramos más cortos en un tramo más largo, los tramos más cortos ya están en la tabla. LEXICALRULES(*word*) devuelve una colección de pares (X,p), uno para cada regla de la forma $X \rightarrow word$ [p], y GRAMMARRULES da tuplas (X,Y,Z,p), una para cada regla gramatical de la forma $X \rightarrow Y\,Z$ [p].

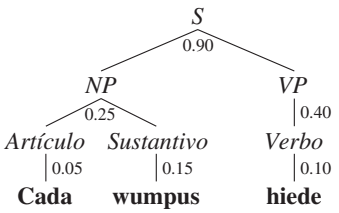


Figura 23.6 Árbol de análisis sintáctico para la sentencia "Every wumpus smells" de acuerdo con la gramática E_0. Cada nodo interior del árbol está etiquetado con su probabilidad. La probabilidad del árbol en su conjunto es $0.9 \times 0.25 \times 0.05 \times 0.15 \times 0.40 \times 0.10 = 0.0000675$. El árbol también permite ser escrito en forma lineal como [*S*[*NP*[*Article* **every**][*Noun* **wumpus**]][*VP*[*Verb* **smells**]]].

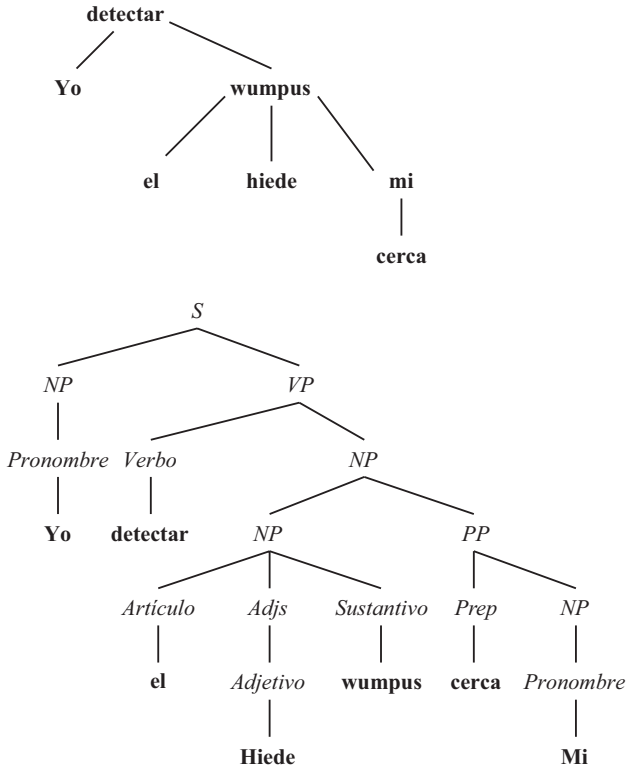


Figura 23.7 Un análisis sintáctico de estilo de dependencia (arriba) y el correspondiente análisis sintáctico de estructura de frase (abajo) para la sentencia *I detect the smelly wumpus near me*.

probabilidad, pero (con una implementación cuidadosa) el analizador permite operar en tiempo $O(n)$ y aún encuentra el mejor análisis la mayoría de las veces.

Analizador sintáctico determinista

Un analizador sintáctico de búsqueda de haz con $b = 1$ se llama **analizador sintáctico determinista**. Un enfoque determinista popular es **análisis sintáctico de desplazamiento-reducción**, en el que recorremos la sentencia palabra por palabra, eligiendo en cada punto si desplazar la palabra a una pila de constituyentes, o reducir el(los) constituyente(s) superior(es) en la pila de acuerdo con una regla gramatical. Cada estilo de análisis sintáctico tiene sus adeptos dentro de la comunidad de NLP. Aunque es posible transformar un sistema de desplazamiento-reducción en un PCFG (y viceversa), cuando se aplica el aprendizaje automático al problema de inducir una gramática, el sesgo inductivo y, por lo tanto, las generalizaciones que cada sistema hará serán diferentes (Abney *et al*., 1999).

Análisis sintáctico de desplazamiento-reducción

23.3.1 Análisis sintáctico de dependencias

Gramática de dependencia

Existe una alternativa sintáctica ampliamente utilizada llamada **gramática de dependencias**, que asume que la estructura sintáctica está formada por relaciones binarias entre elementos léxicos, sin necesidad de constituyentes sintácticos. La Figura 23.7 muestra una sentencia con un análisis de dependencia y un análisis de estructura de frase.

```
[[S[NP-2 Her eyes]
    [VP were
        [VP glazed
            [NP*-2]
            [SBAR-ADV as if
                    [S[NP she]
                      [VP did n't
                        [VP[VP hear [NP*-1]]
                            or
                            [VP[ADVP even] see [NP*-1]]
                            [NP-1 him]]]]]]]
.]
```

Figura 23.8 Árbol anotado para la sentencia "Her eyes were glazed as if she didn't hear or even see him" del Penn Treebank. Note un fenómeno gramatical que aún no hemos cubierto: el movimiento de una frase de una parte del árbol a otra. Este árbol analiza la frase "hear or even see him" como consistente en dos *VPs* constituyentes, [*VP* **hear** [*NP**-1]] y [*VP* [*ADVP* **even**] **see** [*NP**-1]], ambos de los cuales tienen un objeto faltante, denotado *-1, que se refiere a la *NP* etiquetada en otra parte del árbol como [*NP*-1 **him**]. De manera similar, el [*NP**-2] se refiere al [*NP*-2 **Her eyes**].

En un sentido, la gramática de dependencia y la gramática de estructura de frase son sólo variantes notacionales. Si el árbol de estructura de frase está anotado con la cabeza de cada frase, es posible recuperar el árbol de dependencia de él. En la otra dirección, podemos convertir un árbol de dependencia en un árbol de estructura de frase introduciendo categorías arbitrarias (aunque es posible que no siempre obtengamos un árbol de aspecto natural de esta manera).

En consecuencia, no preferiríamos una notación sobre la otra porque una sea más poderosa; más bien preferiríamos una porque es más natural, ya sea más familiar para los desarrolladores humanos de un sistema, o más natural para un sistema de aprendizaje automático que tendrá que aprender las estructuras. En general, los árboles de estructura de frase son naturales para idiomas (como el inglés) con un orden de palabras mayormente fijo; los árboles de dependencia son naturales para idiomas (como el latín) con un orden de palabras mayormente libre, donde el orden de las palabras está determinado más por la pragmática que por las categorías sintácticas.

La popularidad de la gramática de dependencia hoy en día se debe en gran parte al proyecto Universal Dependencies (Nivre *et al*., 2016), un proyecto de treebank de código abierto que define un conjunto de relaciones y proporciona millones de frases analizadas en más de 70 idiomas.

23.3.2 Aprendiendo un analizador sintáctico de ejemplos

Construir una gramática para una parte significativa del inglés es laborioso y propenso a errores. Esto sugiere que sería mejor **aprender** las reglas gramaticales (y las probabilidades) en lugar de escribirlas a mano. Para aplicar el aprendizaje supervisado, necesitamos pares de entrada/salida de frases y sus árboles de análisis. El Penn Treebank es la fuente más conocida de tales datos, con más de 100 mil frases anotadas con estructura de árbol de análisis. La Figura 23.8 muestra un árbol anotado del Penn Treebank.

Dado un treebank, podemos crear un PCFG contando el número de veces que cada tipo de nodo aparece en un árbol (con las advertencias habituales sobre el suavizado de recuentos bajos). En la Figura 23.8, hay dos nodos de la forma [*S*[*NP*...][*VP*...]]. Contaríamos estos, y todos los

demás subárboles con raíz *S* en el corpus. Si hay 1 000 nodos *S* de los cuales 600 son de esta forma, entonces creamos la regla:

$S \rightarrow NP\ VP$ [0.6].

En total, el Penn Treebank tiene más de 10 000 tipos de nodos diferentes. Esto refleja el hecho de que el inglés es un idioma complejo, pero también indica que los anotadores que crearon el treebank favorecieron los árboles planos, quizá más planos de lo que nos gustaría. Por ejemplo, la frase "the good and the bad" se analiza sintácticamente como una sola frase nominal en lugar de como dos frases nominales conjuntas, lo que nos da la regla:

NP → Artículo Sustantivo Conjunción Artículo Sustantivo.

Hay cientos de reglas similares que definen una frase nominal como una cadena de categorías con una conjunción en algún lugar en medio; una gramática más concisa podría capturar todas las reglas de la frase nominal conjunta con la regla única

NP → NP Conjunción NP.

Bod *et al.* (2003) y Bod (2008) muestran cómo recuperar de manera automática reglas generalizadas como esta, reduciendo el número de reglas que salen del treebank, y creando una gramática que termina generalizando mejor para frases nunca vistas. Llaman a su enfoque **análisis sintáctico orientado a datos**.

Hemos visto que los treebanks no son perfectos, contienen errores y tienen análisis sintácticos idiosincrásicos. También está claro que crear un treebank requiere mucho trabajo; eso significa que los treebanks seguirán siendo relativamente pequeños en tamaño, en comparación con todo el texto que no ha sido anotado con árboles. Un enfoque alternativo es el **análisis sintáctico no supervisado**, en el que aprendemos una nueva gramática (o mejoramos una gramática existente) utilizando un corpus de frases sin árboles.

Análisis sintáctico no supervisado

El **algoritmo inside-outside** (Dodd, 1988), que no cubriremos aquí, aprende a estimar las probabilidades en un PCFG a partir de ejemplos de frases sin árboles, de manera similar a cómo el algoritmo forward-backward (Figura 14.4) estima las probabilidades. Spitkovsky *et al.* (2010a) describen un enfoque de aprendizaje no supervisado que utiliza el **aprendizaje curricular**: comienza con la parte fácil del currículo, las frases de 2 palabras cortas e inequívocas como "He left" permiten ser analizadas con base en conocimientos previos o anotaciones. Cada nuevo análisis sintáctico de una frase corta amplía el conocimiento del sistema para que, eventualmente, pueda abordar frases de 3 palabras, luego de 4 palabras y, de manera eventual, frases de 40 palabras.

Aprendizaje curricular

También podemos usar el **análisis sintáctico semisupervisado**, en el que comenzamos con un pequeño número de árboles como datos para construir una gramática inicial, luego agregamos un gran número de sentencias sin analizar para mejorar la gramática. El enfoque semisupervisado permite hacer uso de **corchetes parciales**: podemos usar texto ampliamente disponible que ha sido marcado por los autores, no por expertos lingüísticos, con una estructura parcialmente arbórea, en forma de anotaciones HTML o similares. En el texto HTML, la mayoría de los corchetes corresponden a un componente sintáctico, por lo que la anotación parcial ayuda a aprender una gramática (Pereira y Schabes, 1992; Spitkovsky *et al.*, 2010b). Considere este texto HTML de un artículo de periódico:

Análisis sintáctico semisupervisado

Corchetes parciales

```
   En 1998, sin embargo, como yo <a>establecí en
<i>The New Republic</i></a> y Bill Clinton
<a>confirmó en sus memorias</a>, Netanyahu cambió de opinión
```

Las palabras rodeadas por etiquetas `<i></i>` forman una frase nominal, y las dos cadenas de palabras rodeadas por etiquetas `<a></a>` forman cada una frase verbal.

23.4 Gramáticas aumentadas

Hasta ahora hemos tratado con **gramáticas libres de contexto**. Pero no todos los *NP* aparecen en todos los contextos con la misma probabilidad. La sentencia "I ate a banana" está bien, pero "Me ate a banana" es incorrecta, y "I ate a bandanna" es improbable.

El problema es que nuestra gramática se centra en categorías léxicas, como *Pronombre*, pero mientras que "I" y "me" son ambos pronombres, sólo "Yo" puede ser el sujeto de una sentencia. De manera similar, "banana" y "bandana" son ambos sustantivos, pero el primero es mucho más probable que sea objeto de "comió". Los lingüistas dicen que el pronombre "Yo" está en el caso sujeto (es decir, es el sujeto de un verbo) y "me" está en el caso[3] objeto (es decir, es el objeto de un verbo). También dicen que "Yo" está en primera persona ("tú" es segunda persona, y "ella" es tercera persona) y es singular ("nosotros" es plural). Una categoría como *Pronombre* que ha sido aumentada con características como "caso sujeto, primero persona singular" se llama una **subcategoría.**

Subcategoría

En esta sección mostramos cómo una gramática permite representar este tipo de conocimiento para hacer distinciones más finas sobre qué sentencias son más probables. También mostraremos cómo construir una representación de la **semántica** de una frase, de una manera compositiva. Todo esto se logrará con una **gramática aumentada** en la que los no terminales no son sólo símbolos atómicos como *Pronombre* o *NP*, sino representaciones estructuradas. Por ejemplo, la frase nominal "I" podría representarse como *NP*(*Sbj,1S,Hablante*), que significa "una frase nominal que está en el caso sujeto, primera persona singular, y cuyo significado es el hablante de la sentencia". En contraste, "me" se representaría como *NP*(*Obj, 1S, Hablante*), marcando el hecho de que está en el caso objeto.

Gramática aumentada

Considere la secuencia "*Sustantivo* y *Sustantivo* o *Sustantivo*", que permite ser analizada ya sea como "[*Sustantivo* y *Sustantivo*] o *Sustantivo*", o como "*Sustantivo* y [*Sustantivo* o *Sustantivo*]". Nuestra gramática libre de contexto no tiene forma de expresar una preferencia por una interpretación sobre la otra, porque la regla para *NPs* conjuntos, $NP \rightarrow NP\ Conjunción\ NP$[0.05], dará la misma probabilidad a cada interpretación. Nos gustaría una gramática que prefiera las interpretaciones "[[spaghetti y albóndigas] o lasaña]" y "[spaghetti y [tarta o pastel]]" sobre la agrupación alternativa para cada una de estas frases.

Una **PCFG lexicalizada** es un tipo de gramática aumentada que nos permite asignar probabilidades basadas en propiedades de las palabras en una frase que no son sólo las categorías sintácticas. Los datos serían muy escasos si la probabilidad de, digamos, una sentencia de 40 palabras dependiera de *todas* las 40 palabras; este es el mismo problema que notamos con los *n*-gramas. Para simplificar, introducimos la noción de la **cabeza** de una frase, la palabra más importante. Así, "banana" es la cabeza del NP "una banana" y "comió" es la cabeza del *VP* "comió una banana". La notación VP(*v*) denota una frase con la categoría *VP* cuya palabra principal es v. Aquí hay una PCFG lexicalizada:

PCFG lexicalizada

Cabeza

$VP(v) \rightarrow Verbo(v)\ NP(n)$	$[P_1(v,n)]$
$VP(v) \rightarrow Verbo(v)$	$[P_2(v)]$
$NP(n) \rightarrow Artículo(a)\ Adjs(j)\ Sustantivo(n)$	$[P_3(n,a)]$
$NP(n) \rightarrow NP(n)\ Conjunción(c)\ NP(m)$	$[P_4(n,c,m)]$
$Verbo(\textbf{comió}) \rightarrow \textbf{comió}$	[0.002]
$Sustantivo(\textbf{banana}) \rightarrow \textbf{banana}$	[0.0007]

[3] El caso subjetivo también se llama a veces caso nominativo y el caso objetivo a veces se llama caso acusativo. Muchos idiomas también hacen otra distinción con un caso dativo para palabras en la posición de objeto indirecto.

$$
\begin{aligned}
S(v) &\rightarrow NP(Sbj,pn,n)\ VP(pn,v) \mid \ldots \\
NP(c,pn,n) &\rightarrow Pronoun(c,pn,n) \mid Noun(c,pn,n) \mid \ldots \\
VP(pn,v) &\rightarrow Verb(pn,v)\ NP(Obj,pn,n) \mid \ldots \\
PP(head) &\rightarrow Prep(head)\ NP(Obj,pn,h) \\
Pronoun(Sbj,1S,\mathbf{I}) &\rightarrow \mathbf{I} \\
Pronoun(Sbj,1P,\mathbf{we}) &\rightarrow \mathbf{we} \\
Pronoun(Obj,1S,\mathbf{me}) &\rightarrow \mathbf{me} \\
Pronoun(Obj,3P,\mathbf{them}) &\rightarrow \mathbf{them} \\
\\
Verb(3S,\mathbf{see}) &\rightarrow \mathbf{see}
\end{aligned}
$$

Figura 23.9 Parte de una gramática aumentada que maneja la concordancia de caso, la concordancia sujeto-verbo y las palabras principales. Los nombres en mayúsculas son constantes: *Sbj*, y *Obj* para el caso subjetivo y objetivo; *1S* para la primera persona del singular; *1P* y *3P* para la primera y tercera persona del plural. Como de costumbre, los nombres en minúsculas son variables. Para simplificar, se han omitido las probabilidades.

Aquí $P_1(v,n)$ significa la probabilidad de que un *VP* encabezado por *v* se una a un *NP* encabezado por *n* para formar un *VP*. Podemos especificar que "ate,banana" es más probable que "ate un pañuelo" asegurándonos de que P_1(ate,banana) > P_1(*ate,bandanna*). Note que como sólo estamos considerando las cabezas de frase, la distinción entre "comió un plátano" y "comió un plátano rancio" no será capturado por P_1. Por definición, P_1 es una enorme tabla de probabilidades: si hay 5 000 verbos y 10 000 sustantivos en el vocabulario, entonces P_1 requiere 50 millones de entradas, pero la mayoría de ellas no se almacenarán de manera explícita; más bien se derivarán de suavizar y retroceder. Por ejemplo, podemos retroceder de $P_1(v,n)$ a un modelo que depende sólo de *v*. Tal modelo requeriría 10 000 veces menos parámetros, pero aún permite capturar regularidades importantes, como el hecho de que un verbo transitivo como "ate" es más probable que sea seguido por un *NP* (independiente de la cabeza) que un verbo intransitivo como "dormir".

Vimos en la Sección 23.2 que la gramática simple para $\mathcal{E}_0$ sobregenera, produciendo no sentencias como "Yo vio ella" o "Yo ve ella". Para evitar este problema, nuestra gramática tendría que saber que "ella", no "la", es un objeto válido de "vio" (o de cualquier otro verbo) y que "ve", no "veo", es la forma del verbo que acompaña al sujeto "Yo".

Podríamos codificar estos hechos en las entradas de probabilidad, por ejemplo, haciendo que $P_1(v,ella)$ sea un número muy pequeño, para todos los verbos *v*. Pero es más conciso y modular aumentar la categoría *NP* con variables adicionales: *NP*(*c,pn,n*) se usa para representar una frase nominal con caso *c* (subjetivo u objetivo), persona y número *pn* (por ejemplo, tercera persona singular), y sustantivo principal *n*. La Figura 23.9 muestra una gramática lexicalizada aumentada que maneja estas variables adicionales. Consideremos una regla gramatical en detalle:

$$S(v) \rightarrow NP(Sbj,pn,n)\ VP(pn,v)\ [P_5(n,v)].$$

Esta regla dice que cuando un *NP* es seguido por un *VP* forman un *S*, pero sólo si el *NP* tiene el caso subjetivo (*Sbj*) y la persona y el número (*pn*) del *NP* y *VP* son idénticos. (Decimos que están *de acuerdo*). Si eso se cumple, entonces tenemos un *S* cuya cabeza es el verbo del *VP*. Aquí hay un ejemplo de regla léxica,

$$Pronoun(Sbj,1S,Yo) \rightarrow \mathbf{Yo}\ [0.005]$$

que dice que "Yo" es un *pronombre* en el caso subjetivo, primera persona del singular, con cabeza "Yo".

$Exp(op(x_1,x_2)) \rightarrow Exp(x_1)\ Operator(op)\ Exp(x_2)$
$Exp(x) \rightarrow (Exp(x))$
$Exp(x) \rightarrow Number(x)$
$Number(x) \rightarrow Digit(x)$
$Number(10 \times x_1 + x_2) \rightarrow Number(x_1)\ Digit(x_2)$
$Operator(+) \rightarrow +$
$Operator(-) \rightarrow -$
$Operator(\times) \rightarrow \times$
$Operator(\div) \rightarrow \div$
$Digit(0) \rightarrow \mathbf{0}$
$Digit(1) \rightarrow \mathbf{1}$
...

Figura 23.10 Gramática para expresiones aritméticas aumentada con semántica. Cada variable x_i representa la semántica de un constituyente.

23.4.1 Interpretación semántica

Para mostrar cómo agregar semántica a una gramática, comenzamos con un ejemplo que es más simple que el inglés: la semántica de las expresiones aritméticas. La Figura 23.10 muestra una gramática para expresiones aritméticas, donde cada regla se aumenta con un solo argumento que indica la interpretación semántica de la frase. La semántica de un dígito como "3" es el dígito en sí. La semántica de la expresión "3 + 4" es el operador "+" aplicado a la semántica de las frases "3" y "4". Las reglas gramaticales obedecen al principio de la **semántica composicional**: la semántica de una frase es una función de la semántica de las subfrases. La Figura 23.11 muestra el árbol de análisis para 3 + (4 ÷ 2) según esta gramática. La raíz del árbol de análisis es *Exp*(5), una expresión cuya interpretación semántica es 5.

Semántica composicional

Ahora pasemos a la semántica del inglés, o al menos una pequeña parte de ella. Utilizaremos la lógica de primer orden para nuestra representación semántica. Así, la simple frase "Ali ama a Bo" debería obtener la representación semántica *Ama*(*Ali*,*Bo*). Pero ¿qué pasa con las frases constituyentes? Podemos representar el *NP* "Ali" con el término lógico Ali. Pero el *VP* "ama a Bo" no es ni un término lógico ni una sentencia lógica completa. De manera intuitiva, "ama a Bo" es una descripción que podría o no aplicarse a una persona en particular. (En este caso, se aplica a Ali). Esto significa que "ama a Bo" es un **predicado** que, cuando se combina con un término que representa a una persona, produce una sentencia lógica completa.

Usando la notación λ (ver página 259), podemos representar "ama a Bo" como el predicado

$$\lambda x\ Loves(x,Bo).$$

Ahora necesitamos una regla que diga "un *NP* con semántica *n* seguido de un *VP* con semántica *pred* produce una sentencia cuya semántica es el resultado de aplicar *pred* a *n*":

$$S(pred(n)) \rightarrow NP(n)\ VP(pred).$$

La regla nos dice que la interpretación semántica de "Ali ama a Bo" es

$$(\lambda x\ Loves(x,Bo))(Ali),$$

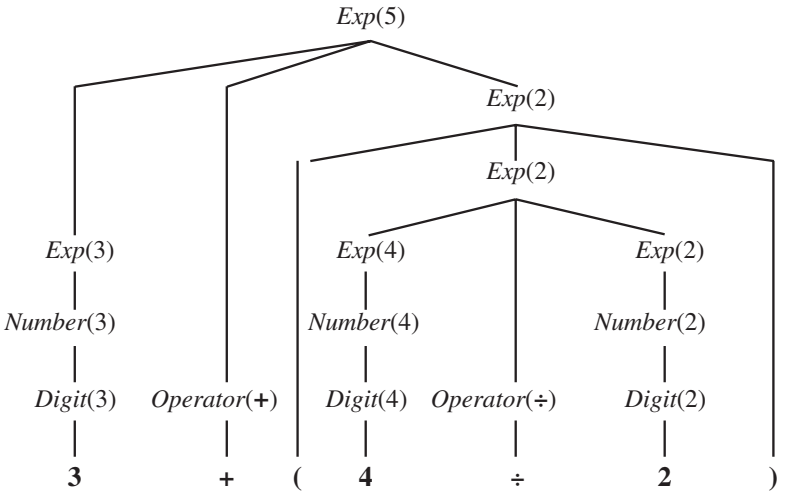
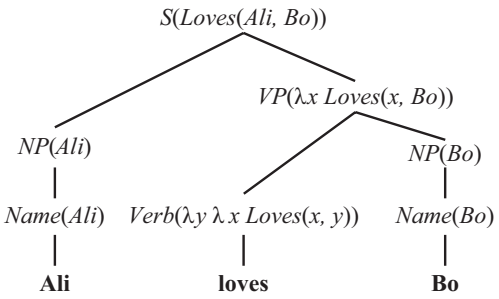


Figura 23.11 Árbol de análisis con interpretaciones semánticas para la cadena "3 + (4 ÷ 2)".

$S(pred(n)) \rightarrow NP(n)\ VP(pred)$
$VP(pred(n)) \rightarrow Verb(pred)\ NP(n)$
$NP(n) \rightarrow Name(n)$

$Name(Ali) \rightarrow$ **Ali**
$Name(Bo) \rightarrow$ **Bo**
$Verb(\lambda y\ \lambda x\ Loves(x,y)) \rightarrow$ **loves**

(a)



(b)

Figura 23.12 (a) Gramática que permite derivar un árbol de análisis y una interpretación semántica para "Ali loves Bo" (y tres otras sentencias). Cada categoría está aumentada con un solo argumento que representa la semántica. (b) Árbol de análisis sintáctico con interpretaciones semánticas para la cadena "Ali ama a Bo".

que es equivalente a *Loves*(*Ali,Bo*). Técnicamente, decimos que esto es una β-reducción de la aplicación de la función lambda.

El resto de la semántica sigue de una manera directa a partir de las elecciones que hemos hecho hasta ahora. Como los *VPs* se representan como predicados, los verbos también deberían ser predicados. El verbo "love" se representa como $\lambda y\ \lambda x\ Loves(x,y)$, el predicado que, cuando se le da el argumento *Bo*, devuelve el predicado $\lambda x\ Loves(x,Bo)$. Terminamos con la gramática y el árbol de análisis que se muestra en la Figura 23.12. En una gramática más completa, pondríamos todas las ampliaciones (semántica, caso, persona-número, y cabeza) juntas en un conjunto de reglas. Aquí sólo mostramos la ampliación semántica para hacer más claro cómo funcionan las reglas.

23.4.2 Aprendiendo gramáticas semánticas

No obstante, el Penn Treebank no incluye representaciones semánticas de sus sentencias, sólo árboles sintácticos. Entonces, si vamos a aprender una gramática semántica, necesitaremos una fuente diferente de ejemplos. Los autores Zettlemoyer y Collins (2005) describen un sistema que aprende una gramática para un sistema de respuesta a preguntas a partir de ejemplos que consisten en una sentencia emparejada con la forma semántica para la sentencia:

- **Sentencia:** ¿Qué estados limitan con Texas?
- **Forma lógica:** $\lambda x.state(x) \wedge \lambda x.borders(x, Texas)$

Dada una gran colección de pares como este y un poco de conocimiento codificado a mano para cada nuevo dominio, el sistema genera entradas léxicas plausibles (por ejemplo, que "Texas" y "estado" son sustantivos de tal manera que *estado*(*Texas*) es verdadero), y de manera simultánea aprende parámetros para una gramática que permite al sistema analizar sintácticamente sentencias en representaciones semánticas. Zettlemoyer y el sistema de Collins logró un 79 % de precisión en dos conjuntos de prueba diferentes de sentencias no vistas. Zhao y Huang (2015) demuestran un analizador de reducción de desplazamiento que funciona más rápido y logra una precisión de 85 % a 89 %.

Una limitación de estos sistemas es que los datos de entrenamiento incluyen formas lógicas. Estos son costosos de crear y requieren anotadores humanos con experiencia especializada, pero no todos entienden las sutilezas del cálculo lambda y la lógica de predicados. Es mucho más fácil recopilar ejemplos de pares de preguntas/respuestas:

- **Pregunta**: ¿Qué estados limitan con Texas?
- **Respuesta:** Louisiana, Arkansas, Oklahoma, Nuevo México.

- **Pregunta:** ¿Cuántas veces cabría Rhode Island en California?
- **Respuesta:** 135

Estos pares de preguntas/respuestas son muy comunes en la Web, por lo que es posible armar una gran base de datos sin expertos humanos. Utilizando esta gran fuente de datos, es posible construir analizadores que superen a aquellos que utilizan una pequeña base de datos de formas lógicas anotadas (Liang *et al*., 2011; Liang y Potts, 2015). El enfoque clave descrito en estos documentos es inventar una forma lógica interna que sea composicional pero que no permita un espacio de búsqueda exponencialmente grande.

23.5 Complicaciones del lenguaje natural real

La gramática del inglés real es infinitamente compleja (y otros idiomas son igualmente complejos). Mencionaremos de manera breve algunos de los temas que contribuyen a esta complejidad.

Cuantificación: Considere la frase "Cada agente siente una brisa". La frase tiene sólo una interpretación sintáctica bajo $\mathcal{E}_0$, pero es semánticamente ambigua: ¿hay una brisa que es sentida por todos los agentes, o cada agente siente una brisa personal separada? Las dos interpretaciones permiten representarse como

Cuantificación

$$\begin{aligned}&\forall a \ \ a \in Agents \Rightarrow \\ &\qquad \exists b \ b \in Breezes \wedge Feel(a,b); \\ &\exists b \ \ b \in Breezes \wedge \forall a \ a \in Agents \Rightarrow \\ &\qquad Feel(a,b).\end{aligned}$$

Forma cuasi-lógica

Un enfoque estándar para la cuantificación es que la gramática defina no una sentencia semántica lógica real, sino una **forma cuasi-lógica** que luego se convierte en una sentencia lógica mediante algoritmos fuera del proceso de análisis. Esos algoritmos permiten tener reglas de preferencia para elegir un alcance de cuantificador sobre otro, preferencias que no necesitan reflejarse de manera directa en la gramática.

Pragmática

Pragmática: Hemos mostrado cómo un agente puede percibir una cadena de palabras y usar una gramática para derivar un conjunto de posibles interpretaciones semánticas. Ahora abordamos el problema de completar la interpretación agregando información dependiente del contexto sobre la situación actual. La necesidad más obvia de información pragmática es en la resolución del significado de los **índices**, que son frases que se refieren de manera directa a la situación actual. Por ejemplo, en la frase "I am in Boston today", tanto "I (yo)" como "today (hoy)" son índices. La palabra "yo" estaría representada por *Speaker*, un fluido que se refiere a diferentes objetos en diferentes momentos, y correspondería al oyente resolver el referente del fluido, eso no se considera parte de la gramática sino un problema de pragmática.

Índice

Acto de habla

Otra parte de la pragmática es interpretar la intención del hablante. El enunciado del hablante se considera un **acto de habla**, y corresponde al oyente descifrar qué tipo de acción es: pregunta, afirmación, promesa, advertencia, comando, etc. Un comando como "ve a 2 2" se refiere de manera implícita al oyente. Hasta ahora, nuestra gramática para *S* sólo cubre sentencias declarativas. Podemos extenderla para cubrir comandos: un comando es una frase verbal donde el sujeto es implícitamente el oyente del comando:

$$S(Command(pred(Hearer))) \rightarrow VP(pred).$$

Dependencias a larga distancia

Dependencias a larga distancia: En la Figura 23.8 vimos que "she didn't hear or even see him" se analizó con dos huecos donde falta un *NP*, pero se refiere al *NP* "him". Podemos usar el símbolo ␣ para representar los huecos: "she didn't [hear ␣ or even see ␣] him". En general, la distancia entre el hueco y el NP al que se refiere puede ser arbitrariamente larga: en "Who did the agent tell you to give the gold to ␣?" el hueco se refiere a "Who?", que está a 11 palabras de distancia.

Un sistema complejo de reglas aumentadas se puede usar para asegurarse de que los *NPs* faltantes coincidan de manera correcta. Las reglas son complejas; por ejemplo, no es posible tener un hueco en una rama de una conjunción *NP*: What did she play [NP Dungeons and ␣]?" "¿Qué jugó [*NP* Dungeons and]?" es incorrecto. Pero puede tener el mismo hueco en ambas ramas de una conjunción *VP*, como en la frase "¿Qué hiciste [*VP* [*VP* oler ␣] y [*VP* disparar una flecha a ␣]]?"

Tiempo y tiempo verbal

Tiempo y tiempo verbal: Suponga que queremos representar la diferencia entre "Ali ama a Bo" y "Ali amó a Bo". El inglés utiliza los tiempos verbales (pasado, presente y futuro) para indicar el tiempo relativo de un evento. Una buena opción para representar el tiempo de los eventos es la notación del cálculo de eventos de la Sección 10.3. En el cálculo de eventos tenemos

$$\text{Ali ama a Bo: } E_1 \in Loves(Ali,Bo) \wedge During(Now,Extent(E_1))$$
$$\text{Ali amó a Bo: } E_2 \in Loves(Ali,Bo) \wedge After(Now,Extent(E_2)).$$

Esto sugiere que nuestras dos reglas léxicas para las palabras "ama" y "amó" deberían ser estas:

$$Verbo(\lambda y\, \lambda x\, e \in Loves(x,y) \wedge During(Now,e)) \rightarrow \textbf{ama}$$
$$Verbo(\lambda y\, \lambda x\, e \in Loves(x,y) \wedge After(Now,e)) \rightarrow \textbf{amó}.$$

Aparte de este cambio, todo lo demás sobre la gramática permanece igual, lo cual es una noticia alentadora; sugiere que estamos en el camino correcto si podemos agregar una complicación como el tiempo de los verbos (aunque apenas hemos rascado la superficie de una gramática completa para el tiempo y el tiempo verbal).

Ambigüedad: Tendemos a pensar en la ambigüedad como un fracaso en la comunicación; cuando un oyente es consciente de una ambigüedad en un enunciado, significa que el enunciado es poco claro o confuso. Aquí hay algunos ejemplos tomados de titulares de periódicos: Ambigüedad

> El equipo ayuda a la víctima de una mordedura de perro.
> La policía comienza campaña para atropellar a los peatones imprudentes.
> Helicóptero impulsado por humanos vuela.
> Industria de pañales de tela salvada por vertederos llenos.
> Incluye a tus hijos cuando hagas galletas.
> Bomba en baño portátil; la policía no tiene nada para ir.
> Los bebedores de leche se están volviendo polvo.
> Dos hermanas se reencuentran después de 18 años en la caja del supermercado.

Tales confusiones son la excepción; la mayor parte del tiempo el lenguaje que escuchamos parece inequívoco. Así, cuando los investigadores comenzaron a usar computadoras para analizar el lenguaje en la década de 1960, se sorprendieron al descubrir que casi todas las sentencias son ambiguas, con múltiples análisis posibles (a veces cientos), incluso cuando el único análisis preferido es el único que los hablantes nativos notan. Por ejemplo, entendemos la frase "brown rice and black beans" como "[arroz marrón] y [frijoles negros]", y nunca consideramos la interpretación de baja probabilidad "marrón [arroz y frijoles negros]", donde el adjetivo "marrón" está modificando toda la frase, no solo el "arroz". Cuando escuchamos "Fuera de un perro, un libro es el mejor amigo de una persona", interpretamos "fuera de" como que significa "excepto para", y nos parece gracioso cuando la siguiente frase de la broma de Groucho Marx es "Dentro de un perro es demasiado oscuro para leer".

La **ambigüedad léxica** es cuando una palabra tiene más de un significado: "back" puede ser un adverbio (volver), un adjetivo (puerta trasera), un sustantivo (la parte trasera de la habitación), un verbo (apoyar a un candidato), o un nombre propio (un río en Nunavut, Canadá). "Jack" puede ser un nombre propio, un sustantivo (una carta de juego, una pieza de juego de metal de seis puntas, una bandera náutica, un pez, un pájaro, un queso, un enchufe, etc.), o un verbo (levantar un coche, cazar con una luz, o golpear una pelota de béisbol con fuerza). **Ambigüedad sintáctica** se refiere a una frase que tiene múltiples análisis: "I smelled a wumpus in 2,2" tiene dos análisis: uno donde la frase preposicional "in 2,2" modifica el sustantivo y uno donde modifica el verbo. La ambigüedad sintáctica conduce a una **ambigüedad semántica**, porque un análisis significa que el wumpus está en 2,2 y el otro significa que un hedor está en 2,2. En este caso, obtener la interpretación equivocada podría ser un error mortal. Ambigüedad léxica. Ambigüedad sintáctica. Ambigüedad semántica.

También existe ambigüedad entre los significados literales y figurativos. Las figuras del habla son importantes en la poesía y son comunes en el habla cotidiana también. Una **metonimia** es una figura de habla en la que un objeto se usa para representar a otro. Cuando escuchamos "Chrysler announced a new model", no lo interpretamos como diciendo que las empresas pueden hablar; más bien entendemos que un portavoz de la empresa hizo el anuncio. La metonimia es común y con frecuencia es interpretada de manera inconsciente por los oyentes humanos. Metonimia.

No obstante, nuestra gramática tal como está escrita no es tan hábil. Para manejar de manera adecuada la semántica de la metonimia necesitamos introducir un nuevo nivel de ambigüedad. Podríamos hacer esto proporcionando *dos* objetos para la interpretación semántica de cada frase en la sentencia: uno para el objeto al que la frase se refiere literalmente (Chrysler) y uno para la referencia metonímica (el portavoz). Luego tenemos que decir que hay una relación entre los dos.

En nuestra gramática actual, "Chrysler announced" se interpreta como

$$x = Chrysler \wedge e \in Announce(x) \wedge After(Now, Extent(e)).$$

Necesitamos cambiar eso a

$$x = Chrysler \wedge e \in Announce(m) \wedge After(Now, Extent(e)) \wedge Metonymy(m,x).$$

Esto dice que hay una entidad x que es igual a Chrysler y otra entidad m que hizo el anuncio, y que las dos están en una relación de metonimia. El siguiente paso es definir qué tipos de relaciones de metonimia pueden ocurrir. El caso más simple es cuando no hay metonimia en absoluto, el objeto literal x y el objeto metonímico m son idénticos:

$$\forall\, m,x\ (m = x) \Rightarrow Metonymy(m,x).$$

Para el ejemplo de Chrysler, una generalización razonable es que una organización permite ser utilizada para ser representada por un portavoz de esa organización:

$$\forall\, m,x\ x \in Organizations \wedge Spokesperson(m,x) \Rightarrow Metonymy(m,x).$$

Otras metonimias incluyen al autor por las obras (leo a *Shakespeare*) o de manera más general al productor por el producto (conduzco un *Honda*) y la parte por el todo (Los Red Sox necesitan un *brazo* fuerte). Algunos ejemplos de metonimia, como "El *sándwich de jamón* en la Mesa 4 quiere otra cerveza", son más novedosos y se interpretan con respecto a una situación (como atender mesas y no conocer el nombre de un cliente).

Metáfora

Una **metáfora** es otra figura del habla, en la que una frase con un significado literal se utiliza para sugerir un significado diferente a través de una analogía. Así, la metáfora permite ser vista como una especie de metonimia donde la relación es de similitud.

Desambiguación

La **desambiguación** es el proceso de recuperar el significado más probablemente intencionado de un enunciado. En cierto sentido, ya tenemos un marco para resolver este problema: cada regla tiene una probabilidad asociada con ella; así, la probabilidad de una interpretación es producto de las probabilidades de las reglas que llevaron a la interpretación. Pero las probabilidades reflejan cuán comunes son las frases en el corpus del que se aprendió la gramática y, por lo tanto, reflejan el conocimiento general, no el conocimiento específico de la situación actual. Para hacer la desambiguación de manera correcta, necesitamos combinar cuatro modelos:

1. El **modelo del mundo**: La probabilidad de que una proposición ocurra en el mundo. Dado lo que sabemos sobre el mundo, es más probable que un hablante que dice "Estoy muerto" signifique "Estoy en grandes problemas" o "Perdí este videojuego", en lugar de "Mi vida terminó y aún así puedo seguir hablando".
2. El **modelo mental**: La probabilidad de que el hablante forme la intención de comunicar un hecho determinado al oyente. Este enfoque combina modelos de lo que el hablante cree, lo que el hablante cree que el oyente cree, etc. Por ejemplo, cuando un político dice: "No soy un delincuente", el modelo del mundo podría asignar una probabilidad de sólo 50 % a la proposición de que el político no es un criminal, y 99.999 % a la proposición de que no es un bastón de pastor enganchado. Sin embargo, seleccionamos la primera interpretación porque es lo más probable que se diga.
3. El **modelo de lenguaje**: La probabilidad de que se elija una cadena de palabras determinada, dado que el hablante tiene la intención de comunicar un hecho determinado.

4. El **modelo acústico**: Para la comunicación hablada, la probabilidad de que se genere una secuencia particular de sonidos, dado que el hablante ha elegido una cadena determinada de palabras. (Para la comunicación escrita o mecanografiada, tenemos el problema del reconocimiento óptico de caracteres).

23.6 Tareas de lenguaje natural

El procesamiento del lenguaje natural es un campo grande que merece uno o dos libros de texto propios (Goldberg, 2017; Jurafsky y Martin, 2020). En esta sección describimos de manera breve algunas de las tareas principales; es posible usar las referencias para obtener más detalles.

Reconocimiento de voz

El **reconocimiento de voz** es la tarea de transformar el sonido hablado en texto. Luego podemos realizar más tareas (como responder preguntas) en el texto resultante. Los sistemas actuales tienen una tasa de error de palabras de casi 3 a 5 % (dependiendo de los detalles del conjunto de prueba), similar a las transcripciones humanas. El desafío para un sistema que utiliza reconocimiento de voz es responder de manera adecuada, incluso cuando hay errores en palabras individuales.

Los sistemas de hoy utilizan una combinación de redes neuronales recurrentes y modelos ocultos de Markov (Hinton *et al*., 2012; Yu y Deng, 2016; Deng, 2016; Chiu *et al*., 2017; Zhang *et al*., 2017). La introducción de redes neuronales profundas para el habla en 2011 condujo a una mejora inmediata y dramática de alrededor de 30 % en la tasa de error, esto en un campo que parecía estar maduro y que antes avanzaba sólo un par de porcentajes por año. Las redes neuronales profundas son una buena opción porque el problema del reconocimiento de voz tiene una descomposición composicional natural: formas de onda a fonemas a palabras a sentencias. Se cubrirán en el próximo capítulo.

Texto a voz

La síntesis de **texto a voz** es el proceso inverso: pasar de texto a sonido. Taylor (2009) da una visión general de la longitud de un libro. El desafío es pronunciar cada palabra de manera correcta, y hacer que el flujo de cada sentencia parezca natural, con las pausas y énfasis correctos.

Otra área de desarrollo está en la síntesis de diferentes voces, comenzando con una elección entre una voz masculina o femenina genérica, luego permitiendo dialectos regionales e, incluso, imitando voces de celebridades. Al igual que con el reconocimiento de voz, la introducción de redes neuronales recurrentes profundas condujo a una gran mejora, con alrededor de 2/3 de los oyentes diciendo que el sistema neural WaveNet (Van den Oord *et al*., 2016a) sonaba más natural que el sistema no neural anterior.

La **traducción automática** transforma el texto de un idioma a otro. Los sistemas suelen ser entrenados usando un corpus bilingüe: un conjunto de documentos emparejados, donde un miembro del par está en, digamos, inglés, y el otro está en, digamos, francés. Los documentos no necesitan ser anotados de ninguna manera; el sistema de traducción automática aprende a alinear frases y luego, cuando se le presenta una frase nueva en un idioma, permite generar una traducción al otro.

Los sistemas en la década de 2000 utilizaban modelos de n-gramas y lograban resultados en general suficientes para transmitir el significado de un texto, pero contenían errores sintácticos en la mayoría de las frases. Un problema era el límite en la longitud de los n-gramas: incluso con un límite grande de 7 era difícil que la información fluyera de un extremo de la frase al otro. Un problema distinto era que toda la información en un modelo de n-gramas está al nivel de palabras individuales. Un sistema de este tipo podría aprender que “black cat” se traduce a “chat noir”, pero no podría aprender la regla de que en inglés, por lo general, los adjetivos vienen antes del sustantivo, pero en francés después del sustantivo.

Los modelos de secuencia a secuencia con redes neuronales recurrentes (Sutskever *et al*., 2015) solucionaron el problema. Podían generalizar mejor (porque podían usar embeddings de

palabras en lugar de recuentos de *n*-gramas de palabras específicas) y podrían formar modelos composicionales a lo largo de los diversos niveles de la red profunda para pasar de manera efectiva la información. El trabajo posterior utilizando el mecanismo de enfoque de atención del modelo transformador (Vaswani *et al*., 2018) aumentó aún más el rendimiento, y un modelo híbrido que incorpora aspectos de ambos modelos lo hace aún mejor, acercándose al rendimiento humano en algunos pares de idiomas (Wu *et al*., 2016b; Chen *et al*., 2018).

Extracción de información

La **extracción de información** es el proceso de adquirir conocimiento al examinar un texto y buscar ocurrencias de clases particulares de objetos y relaciones entre ellos. Una tarea típica es extraer instancias de direcciones de páginas web, con campos de la base de datos para la calle, la ciudad, el estado y el código postal; o instancias de tormentas de informes meteorológicos, con campos para temperatura, velocidad del viento y precipitación. Si el texto fuente está bien estructurado (por ejemplo, en forma de tabla), entonces técnicas simples como las expresiones regulares permiten extraer la información (Cafarella *et al.,* 2008). Se vuelve más difícil si estamos tratando de extraer *todos* los hechos, en lugar de un tipo específico (como los informes meteorológicos); Banko *et al.* (2007) describen el sistema TextRunner que realiza la extracción sobre un conjunto abierto y en expansión de relaciones. Para el texto de forma libre, las técnicas incluyen modelos de Markov ocultos y sistemas de aprendizaje basados en reglas (como se usa en TextRunner y NELL (Never-Ending Language Learning) (Mitchell *et al*., 2018)). Los sistemas más recientes utilizan redes neuronales recurrentes, aprovechando la flexibilidad de los embeddings de palabras. Puede encontrar una visión general en Kumar (2017).

Recuperación de información

La **recuperación de información** es la tarea de encontrar documentos que sean relevantes e importantes para una consulta dada. Los motores de búsqueda en Internet como Google y Baidu realizan esta tarea miles de millones de veces al día. Tres buenos libros de texto sobre el tema son Manning *et al.* (2008), Croft *et al.* (2010) y Baeza-Yates y Ribeiro-Neto (2011).

Respuesta a preguntas

La **respuesta a preguntas** es una tarea diferente, en ella la consulta realmente es una pregunta, como "¿Quién fundó la Guardia Costera de Estados Unidos?" y la respuesta no es una lista clasificada de documentos sino una respuesta real: "Alexander Hamilton". Ha habido sistemas de respuesta a preguntas desde la década de 1960 que se basan en el análisis sintáctico como se discute en este capítulo, pero sólo desde 2001 tales sistemas han utilizado la recuperación de información web para aumentar su cobertura. Katz (1997) describe el analizador Start y el contestador de preguntas. Banko *et al.* (2002) describen AskMSR, que era menos sofisticado en términos de su capacidad de análisis sintáctico, pero más agresivo en el uso de la búsqueda web y la clasificación de los resultados. Por ejemplo, para responder "¿Quién fundó la Guardia Costera de Estados Unidos?" buscaría consultas como [* fundó la Guardia Costera de Estados Unidos] y [la Guardia Costera de Estados Unidos fue fundada por *], y luego examinar las múltiples páginas web resultantes para elegir una respuesta probable, sabiendo que la palabra de consulta "quién" sugiere que la respuesta debería ser una persona. La Conferencia de Recuperación de Texto (TREC) recopila investigaciones sobre este tema y ha organizado competencias anuales desde 1991 (Allan *et al*., 2017). Recién hemos visto otros conjuntos de prueba, como el conjunto de prueba ARC de AI2 de preguntas de ciencia básica (Clark *et al.,* 2018).

Resumen

Los puntos principales de este capítulo son los siguientes:

- Los modelos de lenguaje probabilísticos basados en *n*-gramas recuperan una cantidad sorprendente de información sobre un lenguaje. Permiten desempeñarse bien en tareas tan

diversas como el lenguaje identificación, corrección ortográfica, análisis de sentimientos, clasificación de géneros y reconocimiento de entidades nombradas.

- Estos modelos de lenguaje permiten tener millones de características, por lo que el preprocesamiento y el suavizado de los datos para reducir el ruido es importante.
- Al construir un sistema de lenguaje estadístico es mejor idear un modelo que pueda hacer buen uso de los **datos** disponibles, incluso si el modelo parece demasiado simplista.
- Las incrustaciones (*embeddings*) de palabras pueden dar una representación más rica de las palabras y sus similitudes.
- Para capturar la estructura jerárquica del lenguaje, las gramáticas de **estructura de frase** (y, en particular, las gramáticas **libres de contexto**) son útiles. El formalismo de la gramática libre de contexto probabilística (PCFG) es ampliamente utilizado, al igual que el formalismo de la gramática de dependencia.
- Las sentencias en un lenguaje libre de contexto permiten ser analizadas en tiempo $O(n^3)$ por un **analizador de grafos** como el **algoritmo CYK**, que requiere que las reglas gramaticales estén en la **Forma Normal de Chomsky.** Con una pequeña pérdida de precisión, los lenguajes naturales permiten ser analizados en tiempo $O(n)$, utilizando una búsqueda de haz o un analizador de reducción de desplazamiento.
- Un **banco de árboles** puede ser un recurso para aprender una gramática PCFG con parámetros.
- Es conveniente **aumentar** una gramática para manejar cuestiones como el acuerdo sujeto-verbo y el caso del pronombre, y para representar información a nivel de palabras en lugar de sólo a nivel de categorías.
- La **interpretación semántica** también permite ser manejada por una gramática aumentada. Podemos aprender una gramática semántica de un corpus de preguntas emparejadas ya sea con la forma lógica de la pregunta o con la respuesta.
- El lenguaje natural es complejo y difícil de capturar en una gramática formal.

Notas bibliográficas e históricas

Los modelos de letras n-grama para el modelado de lenguaje fueron propuestos por Markov (1913). Claude Shanon (Shannon y Weaver, 1949) fue el primero en generar modelos de palabras n-grama de inglés. El **modelo de bolsa de palabras** obtiene su nombre de un pasaje del lingüista Zellig Harris (1954), "el lenguaje no es una bolsa de palabras sino una herramienta con propiedades particulares". Norvig (2009) da algunos ejemplos de tareas que se realizan con modelos n-grama.

Chomsky (1956, 1957) señaló las limitaciones de los modelos de estado finito en comparación con los modelos libres de contexto, concluyendo, "Los modelos probabilísticos no dan ninguna visión particular sobre algunos de los problemas básicos de la estructura sintáctica". Esto es cierto, pero los modelos probabilísticos *sí* proporcionan una visión sobre algunos *otros* problemas básicos, problemas que los modelos libres de contexto ignoran. Las observaciones de Chomsky tuvieron el desafortunado efecto de alejar a muchas personas de los modelos estadísticos durante dos décadas, hasta que estos modelos reaparecieron para su uso en el campo del reconocimiento de voz (Jelinek, 1976) y en la ciencia cognitiva, donde la **teoría de la optimalidad** (Smolensky y Prince, 1993; Kager, 1999) postuló que el lenguaje funciona encontrando el candidato más probable que satisface de manera óptima las restricciones competitivas.

El suavizado de suma uno, sugerido por primera vez por Pierre-Simon Laplace (1816), fue formalizado por Jeffreys (1948). Otras técnicas de suavizado incluyen el suavizado de interpolación (Jelinek y Mercer, 1980), el suavizado de Witten-Bell (1991), el suavizado de Good-Turing

(Church y Gale, 1991), suavizado de Kneser-Ney (1995, 2004) y retroceso estúpido (Brants *et al.,* 2007). Chen y Goodman (1996) y Goodman (2001) revisan las técnicas de suavizado.

Los modelos de letras y palabras *n*-grama simples no son los únicos modelos probabilísticos posibles. El modelo de **asignación latente de Dirichlet** (Blei *et al*., 2002; Hoffman *et al*., 2011) es un modelo de texto probabilístico que ve un documento como una mezcla de temas, cada uno con su propia distribución de palabras. Este modelo permite ser visto como una extensión y racionalización del modelo de **indexación semántica latente** de Deerwester *et al.* (1990) y también está relacionado con el modelo de mezcla de múltiples causas de (Sahami *et al.,* 1996). Y, por supuesto, hay un gran interés en los modelos de lenguaje no probabilísticos, como los modelos de aprendizaje profundo cubiertos en el Capítulo 24.

Joulin *et al.* (2016) proporcionan un conjunto de trucos para la clasificación de texto eficiente. Joachims (2001) utiliza la teoría del aprendizaje estadístico y las máquinas de vectores de soporte para dar un análisis teórico de cuándo será exitosa la clasificación. Apté *et al.* (1994) informan una precisión de 96 % en la clasificación de artículos de noticias de Reuters en la categoría "Ganancias". Koller y Sahami (1997) informan una precisión de hasta 95 % con un clasificador Bayes ingenuo y hasta 98.6 % con un clasificador Bayes.

Schapire y Singer (2000) muestran que los clasificadores lineales simples logran una precisión casi tan buena como los modelos más complejos, y se ejecutan más rápido. Zhang *et al.* (2016) describen un clasificador de texto a nivel de carácter (en lugar de a nivel de palabra). Witten *et al.* (1999) describen algoritmos de compresión para la clasificación y muestran la profunda conexión entre el algoritmo de compresión LZW y los modelos de lenguaje de entropía máxima.

Wordnet (Fellbaum, 2001) es un diccionario público de unas 100 000 palabras y frases, categorizadas en partes del discurso y vinculadas por relaciones semánticas como sinónimo, antónimo y parte de. Charniak (1996) y Klein y Manning (2001) discuten el análisis con gramáticas de treebank. El Corpus Nacional Británico (Leech *et al.,* 2001) contiene 100 millones de palabras, y la World Wide Web contiene varios billones de palabras; Franz y Brants (2006) describen el corpus de *n*-gramas de Google de acceso público de 13 millones de palabras únicas de un billón de palabras de texto web. Buck *et al.* (2014) describen un conjunto de datos similar del proyecto Common Crawl. El Penn Treebank (Marcus *et al.,* 1993; Bies *et al*., 2015) proporciona árboles de análisis para un corpus de inglés de 3 millones de palabras.

Muchas de las técnicas del modelo de *n*-gramas también se utilizan en problemas de bioinformática. La bioestadística y el PLN probabilístico se están acercando, ya que cada uno se ocupa de secuencias largas y estructuradas elegidas de un alfabeto.

Los primeros etiquetadores de parte del discurso (POS) utilizaron una variedad de técnicas, incluyendo conjuntos de reglas (Brill, 1992), *n*-gramas (Church, 1988), árboles de decisión (Màrquez y Rodríguez, 1998), HMMs (Brants, 2000) y regresión logística (Ratnaparkhi, 1996). Históricamente, un modelo de regresión logística también se llamaba "modelo de Markov de entropía máxima" o MEMM, por lo que algunos trabajos están bajo ese nombre. Jurafsky y Martin (2020) tienen un buen capítulo sobre etiquetado POS. Ng y Jordan (2002) comparan modelos discriminativos y generativos para tareas de clasificación.

Al igual que las redes semánticas, las gramáticas libres de contexto fueron descubiertas por primera vez por gramáticos indios antiguos (en especial Panini, *ca*. 350 a.C.) que estudiaban el sánscrito Shastric (Ingerman, 1967). Fueron reinventadas por Noam Chomsky (1956) para el análisis del inglés y de manera independiente por John Backus (1959) y Peter Naur para el análisis de Algol-58.

Las **gramáticas libres de contexto probabilísticas** fueron investigadas por primera vez por Booth (1969) y Salomaa (1969). Los algoritmos para PCFGs se presentan en el excelente monografía corta de Charniak (1993) y los excelentes libros de texto largos de Manning y

Schütze (1999) y Jurafsky y Martin (2020). Baker (1979) introduce el algoritmo inside-outside para aprender un PCFG. Los **PCFGs lexicalizados** (Charniak, 1997; Hwa, 1998) combinan los mejores aspectos de los PCFGs y los modelos de *n*-gramas. Collins (1999) describe el análisis de PCFG, que está lexicalizado con características de cabeza, y Johnson (1998) muestra cómo la precisión de un PCFG depende de la estructura del treebank del que se aprendieron sus probabilidades.

Ha habido muchos intentos de escribir gramáticas formales de lenguajes naturales, tanto en la lingüística "pura" como en la lingüística computacional. Hay varias gramáticas informales pero completas del inglés (Quirk *et al.,* 1985; McCawley, 1988; Huddleston y Pullum, 2002). Desde la década de 1980, ha habido una tendencia hacia la lexicalización: poner más información en el léxico y menos en la gramática.

La gramática funcional-lexical, o LFG (Bresnan, 1982) fue el primer formalismo gramatical importante en ser lexicalizado. Si llevamos la lexicalización al extremo, terminamos con la **gramática categorial** (Clark y Curran, 2004), en la que existen mínimo dos reglas gramaticales, o con la **gramática de dependencia** (Smith y Eisner, 2008; Kübler *et al.,* 2009) en la que no hay categorías sintácticas, sólo relaciones entre palabras.

El análisis sintáctico informatizado fue demostrado por primera vez por Yngve (1955). Se desarrollaron algoritmos eficientes en la década de 1960, con algunos giros desde entonces (Kasami, 1965; Younger, 1967; Earley, 1970; Graham *et al.,* 1980). Church y Patil (1982) describen la ambigüedad sintáctica y abordan formas de resolverla.

Klein y Manning (2003) describen el análisis A^*, mientras Pauls y Klein (2009) lo extienden a análisis sintáctico *K*-best A^*, en el que el resultado no es un sólo análisis sino los *K* mejores. Goldberg *et al.* (2013) describen los trucos de implementación necesarios para asegurarse de que un analizador de búsqueda en haz es $O(n)$ y no $O(n^2)$. Zhu *et al.* (2013) describen un analizador de cambio-reducción determinista rápido para lenguajes naturales y Sagae y Lavie (2006) muestran cómo agregar búsqueda a un cambio-reducción, un analizador permite hacerlo más preciso, a costa de algo de velocidad.

Hoy en día, los analizadores de código abierto precisos incluyen Parsey McParseface de Google (Andor *et al.,* 2016), el Stanford Parser (Chen y Manning, 2014), el Berkeley Parser (Kitaev y Klein, 2018) y el analizador SpaCy. Todos ellos hacen generalización a través de redes neuronales y logran cerca de 95 % de precisión en los conjuntos de prueba de Wall Street Journal o Penn Treebank. Hay algunas críticas al campo que se está enfocando en medir el rendimiento en unos pocos corpora selectos y, tal vez, sobre ajustando en ellos.

La interpretación semántica formal de los lenguajes naturales se origina dentro de la filosofía y la lógica formal, en particular el trabajo de Alfred Tarski (1935) sobre la semántica de los lenguajes formales. Bar-Hillel (1954) fue el primero en considerar los problemas de la pragmática (como los indexicals) y proponer que podrían ser manejados por la lógica formal. El ensayo de Richard Montague "English as a formal language" (1970) es una especie de manifiesto para el análisis lógico del lenguaje, pero hay otros libros que son más legibles (Dowty *et al.,* 1991; Portner y Partee, 2002; Cruse, 2011). Mientras que los programas de interpretación semántica están diseñados para elegir la interpretación más probable, los críticos literarios (Empson, 1953; Hobbs, 1990) han sido ambiguos sobre si la ambigüedad es algo a resolver o a apreciar. Norvig (1988) discute los problemas de considerar múltiples interpretaciones simultáneas, en lugar de conformarse con una única interpretación de máxima probabilidad. Lakoff y Johnson (1980) dan un análisis atractivo y un catálogo de metáforas comunes en inglés. Martin (1990) y Gibbs (2006) ofrecen modelos computacionales de interpretación de metáforas.

El primer sistema de PLN que resolvió una tarea real fue el sistema de respuesta a preguntas de Baseball (Green *et al.,* 1961), que manejaba preguntas sobre una base de datos de estadísticas

de béisbol. Poco después de eso fue SHRDLU de Winograd (1972), que manejaba preguntas y comandos sobre una escena de bloques del mundo, y LUNAR de Woods (1973), que respondía preguntas sobre las rocas traídas de la luna por el programa Apollo.

Banko *et al.* (2002) presentan el sistema de respuesta a preguntas ASKMSR; un sistema similar se debe a Kwok *et al.* (2001). Pasca y Harabagiu (2001) discuten un sistema de respuesta a preguntas ganador de un concurso.

Los enfoques modernos para la interpretación semántica con frecuencia asumen que la asignación de la sintaxis a la semántica se aprenderá a partir de ejemplos (Zelle y Mooney, 1996; Zettlemoyer y Collins, 2005; Zhao y Huang, 2015). El primer resultado importante sobre la **inducción de la gramática** fue uno negativo: Gold (1967) demostró que no es posible aprender de manera confiable una gramática libre de contexto correcta, dada un conjunto de cadenas de esa gramática. Lingüistas prominentes, como Chomsky (1957) y Pinker (2003), han utilizado el resultado de Gold para argumentar que debe haber una **gramática universal** innata que todos los niños tienen desde el nacimiento. El llamado argumento de la **Pobreza del estímulo** dice que a los niños no se les da suficiente entrada para aprender una CFG, por lo que ya deben "conocer" la gramática y estar ajustando algunos de sus parámetros.

Gramática universal

Mientras este argumento continúa teniendo influencia en gran parte de la lingüística chomskiana, ha sido desestimado por otros lingüistas (Pullum, 1996; Elman *et al.,* 1997) y la mayoría de los científicos de la computación. Ya en 1969, Horning mostró que *es* posible aprender, en el sentido del aprendizaje PAC, una gramática libre de contexto *probabilística*. Desde entonces ha habido muchas demostraciones empíricas convincentes de aprendizaje de lenguaje a partir de ejemplos positivos, como aprender gramáticas semánticas con programación lógica inductiva (Muggleton y De Raedt, 1994; Mooney, 1999), las tesis doctorales de Schütze (1995) y de Marcken (1996), y toda la línea de modernos sistemas de procesamiento de lenguaje basados en el modelo transformador (Sección 24). Hay una Conferencia Internacional anual sobre Inferencia Gramatical (ICGI).

El sistema DRAGON de James Baker (Baker, 1975) podría considerarse el primer sistema exitoso de reconocimiento de voz. Fue el primero en usar HMMs para el habla. Después de varias décadas de sistemas basados en modelos de lenguaje probabilísticos, el campo comenzó a cambiar a las redes neuronales profundas (Hinton *et al.,* 2012). Deng (2016) describe cómo la introducción del aprendizaje profundo permitió una mejora rápida en el reconocimiento de voz, y reflexiona sobre las implicaciones para otras tareas de NLP. Hoy en día, el aprendizaje profundo es el enfoque dominante para todos los sistemas de reconocimiento de voz a gran escala. El reconocimiento de voz permite ser visto como la primera área de aplicación que destacó el éxito del aprendizaje profundo, con la visión por computadora siguiendo poco después.

El interés en el campo de la **recuperación de información** fue impulsado por el uso generalizado de la búsqueda en Internet. Croft *et al.* (2010) y Manning *et al.* (2008) proporcionan libros de texto que cubren los conceptos básicos. La conferencia TREC organiza una competencia anual para sistemas de IR y publica actas con resultados.

Brin y Page (1998) describen el algoritmo PageRank, que tiene en cuenta los enlaces entre las páginas, y dan una visión general de la implementación de un motor de búsqueda en la Web. Silverstein *et al.* (1998) investigan un registro de mil millones de búsquedas en la Web. La revista *Information Retrieval* y las actas de la conferencia anual insignia *SIGIR* cubren los desarrollos recientes en el campo.

La **extracción de información** ha sido impulsada por las Conferencias Anuales de Comprensión de Mensajes (MUC), patrocinadas por el gobierno de Estados Unidos. Se dan encuestas de sistemas basados en plantillas por Roche y Schabes (1997), Appelt (1999) y Muslea (1999). Grandes bases de datos de hechos fueron extraídas por Craven *et al.* (2000), Pasca *et al.* (2006), Mitchell (2007) y Durme y Pasca (2008). Freitag y McCallum (2000) discuten HMMs

para la Extracción de Información. Los campos aleatorios condicionales también se han utilizado para esta tarea (Lafferty *et al*., 2001; McCallum, 2003); Sutton y McCallum (2007) proporcionan un tutorial con orientación práctica. Sarawagi (2007) ofrece una encuesta exhaustiva.

Dos enfoques tempranos e influyentes para la ingeniería del conocimiento automatizada para NLP fueron de Riloff (1993), quien demostró que un diccionario construido de manera automática funcionaba casi tan bien como un diccionario específico del dominio elaborado a mano, y de Yarowsky (1995), quien demostró que la tarea de clasificación del sentido de las palabras podría realizarse mediante entrenamiento no supervisado en un corpus de texto sin etiquetar con una precisión tan buena como los métodos supervisados.

La idea de extraer de manera simultánea plantillas y ejemplos de un puñado de etiquetados fue desarrollada independiente y de manera simultánea por Blum y Mitchell (1998), quienes la llamaron **cotraining**, y por Brin (1998), quien la llamó DIPRE (Extracción Dual Iterativa de Relaciones de Patrones). Es posible ver por qué el término *cotraining* ha perdurado. Un trabajo temprano similar, bajo el nombre de bootstrapping, fue realizado por Jones *et al.* (1999). El método fue avanzado por los sistemas QXTract (Agichtein y Gravano, 2003) y KnowItAll (Etzioni *et al.,* 2005). La lectura de máquinas fue introducida por Mitchell (2005) y Etzioni *et al*. (2006) y es el foco del proyecto TextRunner (Banko *et al.,* 2007; Banko y Etzioni, 2008).

Este capítulo se ha centrado en las frases de lenguaje natural, pero también es posible hacer la extracción de información basada en la estructura física o el diseño geométrico del texto en lugar de en la estructura lingüística. Listas, tablas, grafos, diagramas, etc., ya sea codificados en HTML o accedidos a través del análisis visual de documentos pdf, son el hogar de datos que permiten ser extraídos y consolidados (Hurst, 2000; Pinto *et al.,* 2003; Cafarella *et al.,* 2008).

Ken Church (2004) muestra que la investigación del lenguaje natural ha oscilado entre concentrarse en los datos (empirismo) y concentrarse en las teorías (racionalismo); describe las ventajas de tener buenos recursos lingüísticos y esquemas de evaluación, pero se pregunta si hemos ido demasiado lejos (Church y Hestness, 2019). Los primeros lingüistas se concentraron en los datos de uso del lenguaje real, incluidos los recuentos de frecuencia. Noam Chomsky (1956) demostró las limitaciones de los modelos de estado finito, lo que llevó a un énfasis en los estudios teóricos de la sintaxis, sin tener en cuenta el rendimiento del lenguaje real. Este enfoque dominó durante veinte años, hasta que el empirismo hizo un regreso basado en el éxito del trabajo en el reconocimiento del habla estadístico (Jelinek, 1976). Hoy en día, el énfasis en los datos empíricos del lenguaje continúa, y hay un interés creciente en los modelos que consideran constructos de nivel superior, como las relaciones sintácticas y semánticas, no sólo las secuencias de palabras. También hay un fuerte énfasis en los modelos de redes neuronales de aprendizaje profundo del lenguaje, que cubriremos en el Capítulo 24.

Se presenta trabajo sobre aplicaciones de procesamiento de lenguaje en la conferencia bianual de Procesamiento de Lenguaje Natural Aplicado (ANLP), la conferencia sobre Métodos Empíricos en Procesamiento de Lenguaje Natural (EMNLP) y la revista *Ingeniería de Lenguaje Natural*. Una amplia gama de trabajos de NLP aparece en la revista *Lingüística Computacional* y su conferencia, ACL, y en la Conferencia Internacional de Lingüística Computacional (COLING). Jurafsky y Martin (2020) dan una introducción completa a la voz y NLP.

CAPÍTULO 24

APRENDIZAJE PROFUNDO PARA EL PROCESAMIENTO DEL LENGUAJE NATURAL

En el que las redes neuronales profundas realizan una variedad de tareas de lenguaje, capturando la estructura del lenguaje natural así como su fluidez.

El Capítulo 23 explicó los elementos clave del lenguaje natural, incluidas la gramática y la semántica. Los sistemas basados en el análisis de sintaxis y semántica han demostrado éxito en muchas tareas, pero su rendimiento está limitado por la interminable complejidad de los fenómenos lingüísticos en el texto real. Dada la gran cantidad de texto disponible en forma legible por máquina, tiene sentido considerar si los enfoques basados en el aprendizaje automático basado en datos son más efectivos. Exploramos esta hipótesis utilizando las herramientas proporcionadas por los sistemas de aprendizaje profundo (Capítulo 21).

Comenzamos en la Sección 24.1 mostrando cómo el aprendizaje permite mejorarse representando palabras como puntos en un espacio de alta dimensión, y no como valores atómicos. La Sección 24.2 cubre el uso de redes neuronales recurrentes para capturar el significado y el contexto a largo plazo a medida que el texto se procesa de manera secuencial. La Sección 24.3 se centra en la traducción automática, uno de los grandes éxitos del aprendizaje profundo aplicado al procesamiento del lenguaje natural (NLP, por sus siglas en inglés). Las Secciones 24.4 y 24.5 cubren modelos que permiten ser entrenados a partir de grandes cantidades de texto sin etiquetar y luego aplicados a tareas específicas, con frecuencia logran un rendimiento de vanguardia. Finalmente, la Sección 24.6 hace un balance de dónde estamos y cómo permite progresar el campo.

24.1 Incrustaciones (*embeddings*) de palabras

Nos gustaría una representación de palabras que no requiera ingeniería de características de manual, pero que permita la generalización entre palabras relacionadas de manera sintáctica ("sin color" e "ideal" son ambos adjetivos), semánticamente ("gato" y "gatito" son ambos felinos), de manera típica ("soleado" y "aguanieve" son ambos términos meteorológicos), en términos de sentimiento ("increíble" tiene un sentimiento opuesto a "vergonzoso"), o de otra manera.

¿Cómo deberíamos codificar una palabra en un vector de entrada $\mathbf{x}$ para usar en una red neuronal? Como se explicó en la Sección 21.2.1 (página 756), podríamos usar un **vector one-hot**; es decir, codificamos la i-ésima palabra en el diccionario con un bit 1 en la i-ésima posición de entrada y un 0 en todas las demás posiciones. Pero tal representación no capturaría la similitud entre las palabras.

Siguiendo la máxima del lingüista John R. Firth (1957), "Conocerá una palabra por la compañía que guarda", podríamos representar cada palabra con un vector de recuentos de n-gramas de todas las frases en las que aparece la palabra. Sin embargo, los recuentos brutos de n-gramas son engorrosos. Con un vocabulario de 100 000 palabras, hay 10^{25} 5-gramas de los que mantener un registro (aunque los vectores en este espacio de 10^{25} dimensiones serían bastante

dispersos, la mayoría de los recuentos serían cero). Obtendríamos una mejor generalización si redujéramos esto a un vector de menor tamaño, quizá con sólo unos pocos cientos de dimensiones. Llamamos a este vector más pequeño y denso una **incrustación (*embedding*) de palabras**: un vector de baja dimensión que representa una palabra. Las incrustaciones de palabras se *aprenden de manera automática* a partir de los datos. (Más adelante veremos cómo se hace esto). ¿Cómo son estas incrustaciones de palabras aprendidas? Por un lado, cada una es sólo un vector de números, donde las dimensiones individuales y sus valores numéricos no tienen significados discernibles:

Incrustación (*embedding*) de palabras

"aardvark" = [–0.7, +0.2, –3.2,...]
"ábaco" = [+0.5, +0.9, –1.3,...]
...
"zyzzyva" = [–0.1, +0.8, –0.4,...].

Por otro lado, el espacio de características tiene la propiedad de que las palabras similares terminan teniendo vectores similares. Podemos ver eso en la Figura 24.1, donde hay grupos separados para palabras de país, parentesco, transporte y comida.

Resulta que, por razones que no comprendemos, los vectores de incrustación de palabras tienen propiedades adicionales más allá de la mera proximidad para palabras similares. Por ejemplo, supongamos que miramos los vectores **A** para Atenas y **B** para Grecia. Para estas palabras, la diferencia de vectores $\mathbf{B} - \mathbf{A}$ parece codificar la relación país/capital. Otros pares, Francia y París, Rusia y Moscú, Zambia y Lusaka, tienen en esencia la misma diferencia de vectores.

Podemos usar esta propiedad para resolver problemas de analogía de palabras como "Atenas es a Grecia como Oslo es a [¿qué]?" Escribiendo **C** para el vector de Oslo y **D** para el desconocido, hipotetizamos que $\mathbf{B} - \mathbf{A} = \mathbf{D} - \mathbf{C}$, lo que nos da $\mathbf{D} = \mathbf{C} + (\mathbf{B} - \mathbf{A})$. Y cuando calculamos este nuevo vector **D**, encontramos que está más cerca de "Noruega" que de cualquier otra palabra. La Figura 24.2 muestra que este tipo de aritmética vectorial funciona para muchas relaciones.

Sin embargo, no hay garantía de que un algoritmo de incrustación de palabras específico, ejecutado en un corpus particular, capturará una relación semántica determinada. Las incrustaciones de palabras son populares porque han demostrado ser una buena representación para tareas de lenguaje posteriores (como responder preguntas o traducir o resumir), no porque estén garantizadas para responder preguntas de analogía por sí mismas.

El uso de vectores de incrustación de palabras en lugar de codificaciones one-hot de palabras resulta ser útil para todas las aplicaciones de aprendizaje profundo a tareas de NLP. De hecho, en múltiples casos es posible usar vectores **preentrenados** genéricos, obtenidos de cualquiera de varios proveedores, para la tarea de NLP particular de uno. Al momento de escribir, los diccionarios de vectores comúnmente utilizados incluyen WORD2VEC, GloVe (Vectores Globales) y FASTTEXT, que tiene incrustaciones para 157 idiomas. Usar un modelo preentrenado puede ahorrar mucho tiempo y esfuerzo. Para más información sobre estos recursos, ver la Sección 24.5.1.

También es posible entrenar sus propios vectores de palabras; esto por lo regular se hace al mismo tiempo que se entrena una red para una tarea en particular. A diferencia de las incrustaciones preentrenadas genéricas, las incrustaciones de palabras producidas para una tarea específica permiten ser entrenadas en un corpus cuidadosamente seleccionado y tenderán a enfatizar aspectos de las palabras que son útiles para la tarea. Suponga, por ejemplo, que la tarea es el etiquetado de parte de la palabra (POS) (ver Sección 23.1.6). Recuerde que esto implica predecir la parte correcta del discurso para cada palabra en una sentencia. Aunque esto es una tarea simple, no es trivial porque muchas palabras permiten ser etiquetadas de múltiples maneras; por ejemplo, la palabra *cortar* puede ser un verbo en presente (transitivo o intransitivo), un verbo en pasado, un verbo infinitivo, un participio pasado, un adjetivo o un sustantivo. Si un adverbio temporal cercano se refiere al pasado,

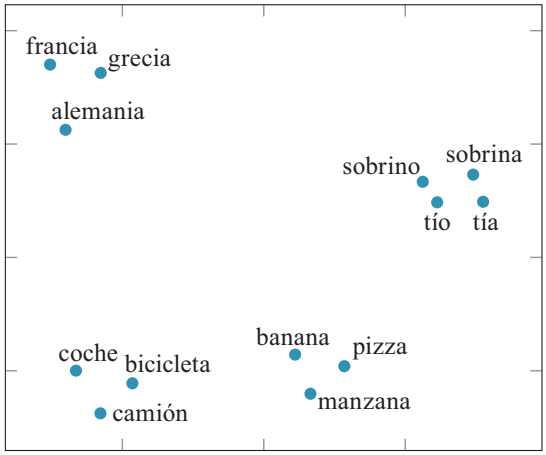


Figura 24.1 Vectores de incrustación de palabras calculados por el algoritmo GloVe entrenado en 6 mil millones de palabras de texto. Los vectores de palabras de 100 dimensiones se proyectan en dos dimensiones en esta visualización. Las palabras similares aparecen cerca una de la otra.

A	**B**	**C**	**D=C+(B-A)**	**Relación**
Atenas	Grecia	Oslo	Noruega	*Capital*
Astana	Kazajistán	Harare	Zimbabue	*Capital*
Angola	kwanza	Irán	rial	*Moneda*
cobre	Cu	oro	Au	*Símbolo atómico*
Microsoft	Windows	Google	Android	*Sistema operativo*
Nueva York	New York Times	Baltimore	Baltimore Sun	*Periódico*
Berlusconi	Silvio	Obama	Barack	*Nombre*
Suiza	Suizo	Camboya	Camboyano	*Nacionalidad*
Einstein	científico	Picasso	pintor	*Ocupación*
hermano	hermana	nieto	nieta	*Relación familiar*
Chicago	Illinois	Estocolmo	California	*Estado*
posible	imposible	ético	poco ético	*Negativo*
ratón	ratones	dólar	dólares	*Plural*
fácil	más fácil	suerte	la más afortunada	*Superlativo*
caminando	caminó	natación	nadó	*Pasado*

Figura 24.2 Un modelo de incrustación de palabras a veces permite responder a la pregunta "**A** es a **B** como **C** es a [¿qué]?" con aritmética vectorial: dado los vectores de incrustación de palabras para las palabras **A**, **B** y **C**, calcula el vector **D**=**C** + (**B** **A**) y busca la palabra que está más cerca de **D**. (Las respuestas en la columna **D** fueron calculadas de manera automática por el modelo. Las descripciones en la columna "Relación" fueron agregadas a mano). Adaptado de Mikolov *et al.* (2013, 2014).

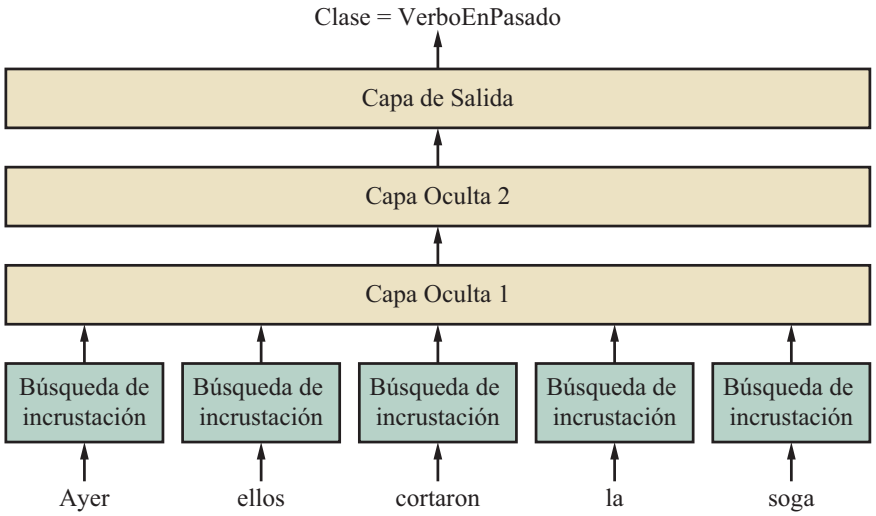


Figura 24.3 Modelo de etiquetado de parte de la palabra de avance. Este modelo toma una ventana de 5 palabras como entrada y predice la etiqueta de la palabra en el medio, aquí, *cortar*. El modelo es capaz de tener en cuenta la posición de la palabra porque cada una de las 5 incrustaciones de entrada se multiplica por una parte diferente de la primera capa oculta. Los valores de los parámetros para las incrustaciones de palabras y para las tres capas se aprenden todos de manera simultánea durante el entrenamiento.

eso sugiere que esta ocurrencia particular de *cortó* es un verbo en pasado; y podríamos esperar, entonces, que la incrustación capture el aspecto de referencia al pasado de los adverbios.

El etiquetado POS sirve como una buena introducción a la aplicación del aprendizaje profundo a NLP, sin las complicaciones de tareas más complejas como responder preguntas (ver Sección 24.5.3). Dado un corpus de oraciones con etiquetas POS, aprendemos los parámetros para las incrustaciones de palabras y el etiquetador POS de manera simultánea. El proceso funciona de la siguiente manera:

1. Elige el ancho w (un número impar de palabras) para la ventana de predicción que se utilizará para etiquetar cada palabra. Un valor típico es $w = 5$, lo que significa que la etiqueta se predice con base en la palabra más las dos palabras a la izquierda y las dos palabras a la derecha. Divide cada sentencia en su corpus en ventanas superpuestas de longitud w. Cada ventana produce un ejemplo de entrenamiento que consiste en las w palabras como entrada y la categoría POS de la palabra del medio como salida.
2. Crea un vocabulario de todos los tokens de palabras únicas que ocurren más de, digamos, 5 veces en los datos de entrenamiento. Denota el número total de palabras en el vocabulario como v.
3. Ordena este vocabulario en cualquier orden arbitrario (quizá de manera alfabética).
4. Elige un valor d como el tamaño de cada vector de incrustación de palabras.
5. Crea una nueva matriz de pesos v-por-d llamada $\mathbf{E}$. Esta es la matriz de incrustación de palabras. La fila i de $\mathbf{E}$ es la incrustación de palabras de la i-ésima palabra en el vocabulario. Inicializa $\mathbf{E}$ de manera aleatoria (o a partir de vectores preentrenados).
6. Configure una red neuronal que genere una etiqueta de parte del discurso, como se muestra en la Figura 24.3. La primera capa consistirá en w copias de la matriz de incrustación. Podríamos usar dos capas ocultas adicionales, $\mathbf{z}_1$ y $\mathbf{z}_2$ (con matrices de pesos $\mathbf{W}_1$ y $\mathbf{W}_2$, res-

pectivamente), seguidas de una capa softmax que produce una distribución de probabilidad de salida $\hat{\mathbf{y}}$ sobre las posibles categorías de partes del discurso para la palabra del medio:

$$\mathbf{z}_1 = \sigma(\mathbf{W}_1\mathbf{x})$$
$$\mathbf{z}_2 = \sigma(\mathbf{W}_2\mathbf{z}_1)$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{out}\mathbf{z}_2).$$

7. Para codificar una secuencia de w palabras en un vector de entrada, busque la incrustación para cada palabra y concatene los vectores de incrustación. El resultado es un vector de entrada real valorado $\mathbf{x}$ de longitud wd. Aunque una palabra dada tendrá el mismo vector de incrustación ya sea que ocurra en la primera posición, la última o en algún lugar intermedio, cada incrustación se multiplicará por una parte diferente de la primera capa oculta; por lo tanto, estamos codificando de manera implícita la posición relativa de cada palabra.
8. Entrene los pesos $\mathbf{E}$ y las otras matrices de pesos $\mathbf{W}_1$, $\mathbf{W}_2$ y $\mathbf{W}_{out}$ usando el descenso de gradiente. Si todo va bien, la palabra del medio, *cortaron*, se etiquetará como un verbo en pasado, basado en la evidencia en la ventana, que incluye la palabra temporal pasada "ayer", el pronombre sujeto de tercera persona "ellos" antes de *cortaron*, y así sucesivamente.

Una alternativa a las incrustaciones de palabras es un **modelo a nivel de caracteres** en el que la entrada es una secuencia de caracteres, cada uno codificado como un vector one-hot. Tal modelo tiene que aprender cómo los caracteres se unen para formar palabras. La mayoría del trabajo en NLP se adhiere a las codificaciones a nivel de palabra en lugar de a nivel de caracteres.

24.2 Redes neuronales recurrentes para NLP

Ahora tenemos una buena representación para palabras individuales en aislamiento, pero el lenguaje consiste en una secuencia ordenada de palabras en la que el **contexto** de las palabras circundantes es importante. Para tareas simples como el etiquetado de partes del discurso, una ventana de tamaño fijo y pequeño de quizá cinco palabras proporciona suficiente contexto.

Tareas más complejas como la respuesta a preguntas o la resolución de referencias pueden requerir docenas de palabras como contexto. Por ejemplo, en la frase "*Eduardo me dijo que Miguel estaba muy enfermo, así que* **lo** *llevé al hospital"*, saber que **lo** se refiere a *Miguel* y no a *Eduardo* requiere un contexto que abarca desde la primera hasta la última palabra de la sentencia de 14 palabras.

24.2.1 Modelos de lenguaje con redes neuronales recurrentes

Comenzaremos con el problema de crear un **modelo de lenguaje** con suficiente contexto. Recuerde que un modelo de lenguaje es una distribución de probabilidad sobre secuencias de palabras. Nos permite predecir la siguiente palabra en un texto dado todas las palabras anteriores, y con frecuencia se usa como un bloque de construcción para tareas más complejas.

Construir un modelo de lenguaje con un modelo n-grama (como en la Sección 23.1) o una red de propagación directa con una ventana fija de n palabras puede tener dificultades debido al problema del contexto: ya sea que el contexto requerido exceda el tamaño de la ventana fija o el modelo tenga demasiados parámetros, o ambos.

Además, una red de propagación directa tiene el problema de la **asimetría**: lo que aprende sobre, digamos, la aparición de la palabra *lo* como la 12a palabra de la sentencia, tendrá que reaprenderlo para la aparición de *lo* en otras posiciones en la sentencia, porque los pesos son diferentes para cada posición de palabra.

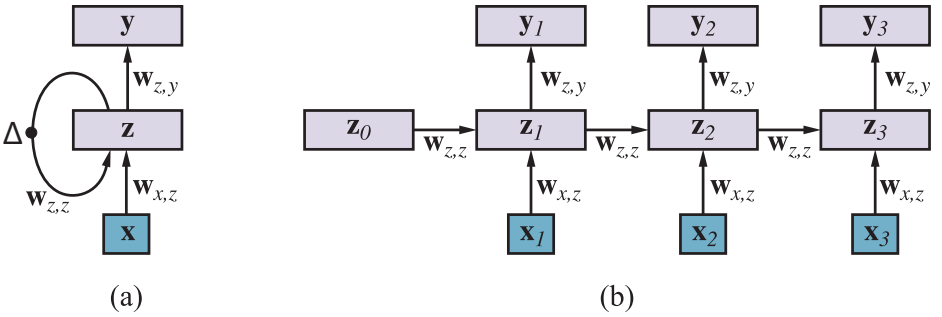


Figura 24.4 (a) Diagrama esquemático de una RNN donde la capa oculta **z** tiene conexiones recurrentes; el símbolo Δ indica un retraso. Cada entrada **x** es el vector de incrustación de palabras de la siguiente palabra en la sentencia. Cada salida **y** es la salida para ese paso de tiempo. (b) La misma red desplegada durante tres pasos de tiempo para crear una red de propagación directa. Tenga en cuenta que los pesos se comparten en todos los pasos de tiempo.

En la Sección 21.6, introdujimos la **red neuronal recurrente** o **RNN**, que está diseñada para procesar datos de series de tiempo, un dato a la vez. Esto sugiere que las RNN podrían ser útiles para procesar el lenguaje, una palabra a la vez. Repetimos la Figura 21.8 aquí, ahora como la Figura 24.4.

En un modelo de lenguaje RNN, cada palabra de entrada se codifica como un vector de incrustación de palabras, $\mathbf{x}_t$. Hay una capa oculta $\mathbf{z}_t$ que se pasa como entrada de un paso de tiempo al siguiente. Estamos interesados en hacer una clasificación multiclase: las clases son las palabras del vocabulario. Así, la salida $\mathbf{y}_t$ será una distribución de probabilidad softmax sobre los posibles valores de la siguiente palabra en la sentencia.

La arquitectura RNN resuelve el problema de tener demasiados parámetros. El número de parámetros en las matrices de peso $w_{,z,z}$, $w_{,x,z}$, y $w_{,z,y}$ permanece constante, independiente del número de palabras, es $O(1)$. Esto contrasta con las redes de avance, que tienen parámetros $O(n)$, y los modelos de n-gramas, que tienen parámetros $O(v^n)$, donde v es el tamaño del vocabulario.

La arquitectura RNN también resuelve el problema de la asimetría, porque los pesos son los mismos para cada posición de la palabra.

La arquitectura RNN a veces también permite resolver el problema del contexto limitado. En teoría, no hay límite para cuánto puede retroceder en la entrada el modelo. Cada actualización de la capa oculta $\mathbf{z}_t$ tiene acceso tanto a la palabra de entrada actual $\mathbf{x}_t$ como a la capa oculta anterior $\mathbf{z}_{t-1}$, lo que significa que la información sobre cualquier palabra en la entrada permite mantenerse en la capa oculta de manera indefinida, copiada (o modificada según corresponda) de un paso de tiempo al siguiente. Por supuesto, hay una cantidad limitada de almacenamiento en **z**, por lo que no permite recordar todo sobre todas las palabras anteriores.

En la práctica, los modelos RNN funcionan bien en una variedad de tareas, pero no en todas. Puede ser difícil predecir si serán exitosos para un problema dado. Un factor que contribuye al éxito es que el proceso de entrenamiento alienta a la red a asignar espacio de almacenamiento en **z** a los aspectos de la entrada que resultarán útiles.

Para entrenar un modelo de lenguaje RNN, utilizamos el proceso de entrenamiento descrito en la Sección 21.6.1. Las entradas, $\mathbf{x}_t$, son las palabras en un corpus de texto de entrenamiento, y las salidas observadas son las mismas palabras desplazadas por 1. Es decir, para el texto de entrenamiento "hola mundo", la primera entrada $\mathbf{x}_1$ es la incrustación de palabras para "hola" y la primera salida $\mathbf{y}_1$ es la incrustación de palabras para "mundo". Estamos entrenando el modelo para

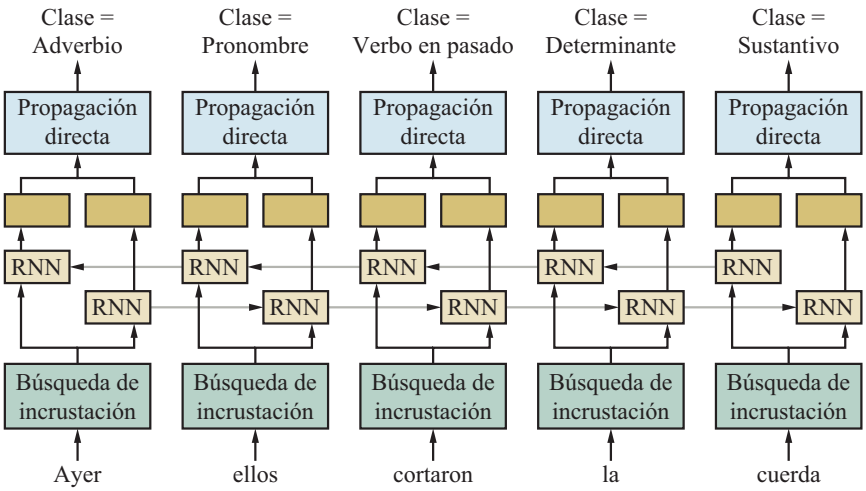


Figura 24.5 Una red RNN bidireccional para el etiquetado POS.

predecir la próxima palabra, y esperamos que para hacerlo utilice la capa oculta para representar información útil. Como se explicó en la Sección 21.6.1, calculamos la diferencia entre la salida observada y la salida real calculada por la red, y retropropagamos en el tiempo, teniendo cuidado de mantener los pesos iguales para todos los pasos de tiempo.

Una vez que el modelo ha sido entrenado, podemos usarlo para generar texto aleatorio. Le damos al modelo una palabra de entrada inicial $\mathbf{x}_1$, a partir de la cual producirá una salida $\mathbf{y}_1$ que es una distribución de probabilidad softmax sobre las palabras. Tomamos una sola palabra de la distribución, registramos la palabra como la salida para el tiempo t, y la retroalimentamos como la próxima palabra de entrada $\mathbf{x}_2$. Repetimos esto tanto como se desee. Al tomar muestras de $\mathbf{y}_1$ tenemos una elección: podríamos tomar siempre la palabra más probable, podríamos tomar muestras de acuerdo con la probabilidad de cada palabra, o podríamos sobremuestrear las palabras menos probables para inyectar más variedad en la salida generada. El peso de muestreo es un hiperparámetro del modelo.

Aquí hay un ejemplo de texto aleatorio generado por un modelo RNN entrenado en las obras de Shakespeare (Karpathy, 2015):

> *Casarse, y lo hará, mi señor, llorar en tal uno sería lo más hermoso;*
> *Sin embargo, ahora fui adoptado como heredero*
> *Del lamentable día del mundo,*
> *¿Para observar el próximo camino con su padre con su rostro?*

24.2.2 Clasificación con redes neuronales recurrentes

También es posible usar RNNs para otras tareas de lenguaje, como el etiquetado de partes del discurso o la resolución de correferencia. En ambos casos, las capas de entrada y ocultas serán las mismas, pero para un etiquetador POS la salida será una distribución softmax sobre las etiquetas POS, y para la resolución de correferencia será una distribución softmax sobre los antecedentes posibles. Por ejemplo, cuando la red llega a la entrada **él** en "*Eduardo me dijo que Miguel estaba muy enfermo así que* **lo** *llevé al hospital"* debería dar una alta probabilidad para "*Miguel"*.

Entrenar una RNN para hacer una clasificación como esta, se realiza de la misma manera que con el modelo de lenguaje. La única diferencia es que los datos de entrenamiento requerirán etiquetas; parte del discurso o indicaciones de referencia. Eso hace que sea mucho más difícil recopilar los datos que para el caso de un modelo de lenguaje, donde sólo necesitamos texto sin etiquetar.

En un modelo de lenguaje queremos predecir la *enésima* palabra dadas las palabras anteriores. Pero para la clasificación, no hay razón para limitarnos a mirar sólo las palabras anteriores. Es útil mirar hacia adelante en la sentencia. En nuestro ejemplo de correferencia, el referente *él* sería diferente si la sentencia concluyera "para ver a Miguel" en lugar de "al hospital", por lo que mirar hacia adelante es crucial. Sabemos por experimentos de seguimiento ocular que los lectores humanos no van de izquierda a derecha.

Para capturar el contexto a la derecha, podemos usar una **RNN bidireccional**, que concatena un modelo de derecha a izquierda separado en el modelo de izquierda a derecha. Un ejemplo de uso de una RNN bidireccional para el etiquetado POS se muestra en la Figura 24.5.

RNN bidireccional

En el caso de una RNN multicapa, $\mathbf{z}_t$ será el vector oculto de la última capa. Para una RNN bidireccional, $\mathbf{z}_t$ suele ser la concatenación de vectores de los modelos de izquierda a derecha y de derecha a izquierda.

Las RNN también permiten usarse para tareas de clasificación a nivel de sentencia (o a nivel de documento), en las que una única salida llega al final, en lugar de tener una corriente de salidas, una por paso de tiempo. Por ejemplo, en el **análisis de sentimientos** el objetivo es clasificar un texto como *Positivo* o *Negativo*. Por ejemplo, "*Esta película fue mal escrita y está mal actuada"* debería ser clasificada como *Negativa*. (Algunos esquemas de análisis de sentimientos usan más de dos categorías, o usan un valor escalar numérico).

Usar RNNs para una tarea a nivel de sentencia es un poco más complejo, ya que necesitamos obtener una representación de toda la sentencia, y a partir de las salidas por palabra $\mathbf{y}_t$ de la RNN. La forma más sencilla de hacer esto es usar el estado oculto de la RNN correspondiente a la última palabra de la entrada, ya que la RNN habrá leído toda la sentencia en ese paso de tiempo. Sin embargo, esto puede sesgar de manera implícita el modelo hacia prestar más atención al final de la sentencia. Otra técnica común es agrupar todos los vectores ocultos. Por ejemplo, el **agrupamiento promedio** calcula el promedio elemento a elemento de todos los vectores ocultos:

Agrupamiento promedio

$$\bar{\mathbf{z}} = \frac{1}{s}\sum_{t=1}^{s} \mathbf{z}_t.$$

El vector $\bar{\mathbf{z}}$ agrupado de dimensión d permite entonces ser alimentado en una o más capas de avance antes de ser alimentado en la capa de salida.

24.2.3 LSTMs para tareas de NLP

Dijimos que las RNN a veces resuelven el problema del contexto limitado. En teoría, cualquier información podría ser transmitida de una capa oculta a la siguiente durante cualquier número de pasos de tiempo. Pero en la práctica la información puede perderse o distorsionarse, al igual que en el juego del teléfono, en el que los jugadores se colocan en línea y el primer jugador susurra un mensaje al segundo, que lo repite al tercero, y así sucesivamente. En general, el mensaje que sale al final está bastante corrompido del mensaje original. Este problema para las RNN es similar al problema del **desvanecimiento de gradiente** que describimos en la página 756, excepto que ahora estamos lidiando con capas a lo largo del tiempo en lugar de con capas profundas.

En la Sección 21.6.2 introdujimos el modelo de **memoria a corto plazo (LSTM).** Este es un tipo de RNN con unidades de compuerta que no sufren del problema de reproducir de manera imperfecta un mensaje de un paso de tiempo al siguiente. Más bien, un LSTM permite elegir

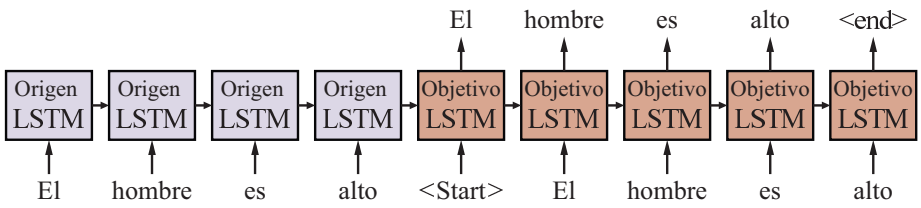


Figura 24.6 Modelo básico de secuencia a secuencia. Cada bloque representa un paso de tiempo LSTM. (Por simplicidad, las capas de incrustación y salida no se muestran). En pasos sucesivos alimentamos la red con las palabras de la sentencia origen "El hombre es alto", seguido de la etiqueta <start> para indicar que la red debe empezar a producir la sentencia objetivo. El estado oculto final al final de la sentencia origen se utiliza como el estado oculto para el inicio de la sentencia objetivo. Después de eso, cada palabra de sentencia objetivo en el tiempo t se usa como entrada en el tiempo $t + 1$, hasta que la red produce la etiqueta <end> para indicar que la generación de la sentencia ha finalizado.

recordar algunas partes de la entrada, copiándola al siguiente paso de tiempo, y olvidar otras partes. Considere un modelo de lenguaje que maneje un texto como

Los atletas, que todos ganaron sus clasificatorios locales y avanzaron a las finales en Tokio, ahora ...

En este punto, si le preguntáramos al modelo qué próxima palabra es más probable, "compiten" o "compites", esperaríamos que elija "compiten" porque concuerda con el sujeto "Los atletas". Un LSTM permite aprender a crear una característica latente para la persona y el número del sujeto y copiar esa característica hacia adelante sin alteración hasta que sea necesario tomar una decisión como esta. Un RNN regular (o un modelo de n-gramas, para el caso) con frecuencia se confunde en oraciones largas con muchas palabras intervinientes entre el sujeto y el verbo.

24.3 Modelos de secuencia a secuencia

Traducción automática (MT)

Idioma origen

Idioma objetivo

Una de las tareas más estudiadas en NLP es la **traducción automática (MT**, por sus siglas en inglés**),** donde el objetivo es traducir una sentencia de un **idioma origen** a un **idioma objetivo**; por ejemplo, de español a inglés. Entrenamos un modelo de MT con un gran corpus de pares de oraciones origen/objetivo. El objetivo es entonces traducir con precisión nuevas oraciones que no están en nuestros datos de entrenamiento.

¿Podemos usar RNNs para crear un sistema MT? Es posible codificar la sentencia origen con un RNN. Si hubiera una correspondencia uno a uno entre las palabras origen y las palabras objetivo, entonces podríamos tratar MT como una simple tarea de etiquetado: dada la palabra origen "perro" en español, la etiquetamos como la palabra inglesa correspondiente "dog". Aunque, de hecho, las palabras no son una a una: en español las tres palabras "caballo de mar" corresponden a la única palabra inglesa "seahorse", y las dos palabras "perro grande" se traducen a "big dog", con el orden de las palabras invertido. La reordenación de palabras puede ser aún más extrema; en inglés el sujeto suele estar al principio de una sentencia, pero en fiyiano el sujeto suele estar al final. Entonces ¿cómo generamos una sentencia en el idioma objetivo?

Parece que deberíamos generar una palabra a la vez, pero mantener un registro del contexto para que podamos recordar partes del origen que aún no se han traducido, y llevar un registro de lo que se ha traducido para que no nos repitamos. También parece que para algunas sentencias

tenemos que procesar toda la sentencia origen antes de comenzar a generar el objetivo. En otras palabras, la generación de cada palabra objetivo depende de toda la sentencia origen y de todas las palabras objetivo generadas con anterioridad.

Esto le da a la generación de texto para MT una estrecha conexión con un modelo de lenguaje RNN estándar, como se describe en la Sección 24.2. Note que, si hubiéramos entrenado un RNN en texto en inglés, sería más probable que generara "big dog" que "dog big". Sin embargo, no queremos generar sólo cualquier sentencia aleatoria en el idioma objetivo; queremos generar una sentencia en el idioma objetivo que *corresponda* a la sentencia en el idioma origen. La forma más sencilla de hacerlo es usar dos RNNs, uno para el origen y uno para el objetivo. Ejecutamos el RNN origen sobre la sentencia origen y luego usamos el estado oculto final del RNN origen como el estado oculto inicial para el RNN objetivo. De esta manera, cada palabra objetivo está condicionada de manera implícita tanto por toda la sentencia origen como por las palabras objetivo anteriores.

Modelo básico de secuencia a secuencia

Esta arquitectura de red neuronal se llama **modelo básico de secuencia a secuencia**, un ejemplo de lo cual se muestra en la Figura 24.6. Los modelos de secuencia a secuencia se utilizan más comúnmente para traducción automática, pero también permiten ser utilizadas para otra serie de tareas, como generar de manera automática una leyenda de texto a partir de una imagen, o resumirla: reescribir un texto largo en uno más corto que mantenga el mismo significado.

Los modelos básicos de secuencia a secuencia fueron un avance significativo en NLP y MT en específico. Según Wu *et al.* (2016b) el enfoque llevó a una reducción del error de 60 % sobre los métodos de MT anteriores. Pero estos modelos sufren de tres grandes deficiencias:

- **Sesgo de contexto cercano:** Lo que los RNN quieran recordar del pasado, deben ajustarlo a su estado oculto. Por ejemplo, digamos que un RNN está procesando la palabra (o paso de tiempo) 57 en una secuencia de 70 palabras. Es probable que el estado oculto contenga más información sobre la palabra en el paso de tiempo 56 que la palabra en el paso de tiempo 5, porque cada vez que se actualiza el vector oculto tiene que reemplazar cierta cantidad de información existente con nueva información. Este comportamiento es parte del diseño intencional del modelo y, en la mayoría de los casos, tiene sentido para NLP ya que el contexto cercano es más importante. Sin embargo, el contexto lejano también es crucial y puede perderse en un modelo RNN; incluso los LSTM tienen dificultades con esta tarea.
- **Límite de tamaño de contexto fijo:** En un modelo de traducción RNN, toda la frase origen se comprime en un sólo vector de estado oculto de dimensión fija. Un LSTM utilizado en un modelo de PNL de última generación normalmente tiene alrededor de 1 024 dimensiones, y si tenemos que representar, digamos, una frase de 64 palabras en 1 024 dimensiones, esto sólo nos da 16 dimensiones por palabra, no suficiente para frases complejas. Aumentar el tamaño del vector de estado oculto lleva a un entrenamiento lento y sobreajuste.
- **Procesamiento secuencial más lento:** Como se discutió en la Sección 21.3, las redes neuronales realizan ganancias de eficiencia considerables al procesar los datos de entrenamiento en lotes para aprovechar el soporte de hardware eficiente para la aritmética de matrices. Por otro lado, los RNN parecen estar limitados a operar en los datos de entrenamiento una palabra a la vez.

24.3.1 Atención

¿Qué pasaría si el RNN objetivo estuviera condicionado a *todos* los vectores ocultos del RNN origen, en lugar de sólo el último? Esto mitigaría las limitaciones del sesgo de contexto cercano y los límites de tamaño de contexto fijo, permitiendo al modelo acceder a cualquier palabra anterior igual de bien. Una forma de lograr este acceso es concatenar todos los vectores ocultos del

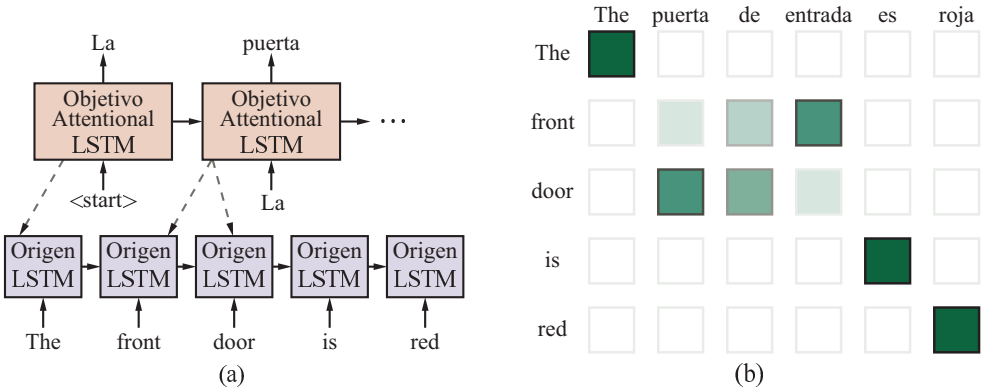


Figura 24.7 (a) Modelo de secuencia a secuencia con atención para la traducción de inglés a español. Las líneas discontinuas representan la atención. (b) Ejemplo de matriz de probabilidad de atención para un par de frases bilingües, con cajas más oscuras que representan valores más altos de a_{ij}. Las probabilidades de atención suman uno en cada columna.

RNN origen. Sin embargo, esto causaría un gran aumento en el número de pesos, con un aumento concomitante en el tiempo de cálculo y potencial sobreajuste también. En cambio, podemos aprovechar el hecho de que cuando el RNN objetivo está generando el objetivo una palabra a la vez, es probable que sólo una pequeña parte del origen sea relevante para cada palabra objetivo.

Note que el RNN objetivo debe prestar atención a diferentes partes del origen para cada palabra. Suponga que una red está entrenada para traducir del inglés al español. Se le dan las palabras "The front door is red" seguidas de un marcador de fin de frase, lo que significa que es hora de comenzar a emitir palabras en español. Así que de manera ideal debería prestar primero atención a "The" y generar "La", luego prestar atención a "door" y emitir "puerta", etc.

Atención

Podemos formalizar este concepto con un componente de red neuronal llamado **atención**, que se permite usar para crear una "sumarización basada en contexto" de la frase origen en una representación de dimensión fija. El vector de contexto $\mathbf{c}_i$ contiene la información más relevante para generar la próxima palabra objetivo, y se utilizará como una entrada adicional al RNN objetivo. Un modelo de secuencia a secuencia que usa atención se llama un **modelo de secuencia a secuencia con atención**. Si el RNN objetivo estándar se escribe como:

Modelo de secuencia a secuencia con atención

$$\mathbf{h}_i = RNN(\mathbf{h}_{i-1}, \mathbf{x}_i),$$

el RNN objetivo para modelos de secuencia a secuencia con atención se escribe como:

$$\mathbf{h}_i = RNN(\mathbf{h}_{i-1}, [\mathbf{x}_i; \mathbf{c}_i])$$

donde $[\mathbf{x}_i; \mathbf{c}_i]$ es la concatenación de los vectores de entrada y contexto, $\mathbf{c}_i$, definido como:

$$r_{ij} = \mathbf{h}_{i-1} \cdot \mathbf{s}_j$$

$$a_{ij} = e^{r_{ij}} / (\sum_k e^{r_{ik}})$$

$$\mathbf{c}_i = \sum_j a_{ij} \cdot \mathbf{s}_j$$

donde $\mathbf{h}_{i-1}$ es el vector RNN objetivo que se va a usar para predecir la palabra en el paso de tiempo i, y $\mathbf{s}_j$ es la salida del vector RNN de origen para la palabra de origen (o paso de tiempo) j. Tanto $\mathbf{h}_{i-1}$ como $\mathbf{s}_j$ son vectores de dimensión d, donde d es el tamaño oculto. El valor de r_{ij} es, por lo tanto, la "puntuación de atención" bruta entre el estado objetivo actual y la palabra de origen j. Estas puntuaciones se normalizan luego en una probabilidad a_{ij} usando un softmax sobre todas las palabras de origen. Finalmente, estas probabilidades se utilizan para generar un promedio ponderado de los vectores RNN de origen, $\boldsymbol{c}_i$ (otro vector de dimensión d).

Un ejemplo de un modelo de secuencia a secuencia de atención se da en la Figura 24.7 (a). Hay algunos detalles importantes para entender. Primero, el componente de atención en sí no tiene pesos aprendidos y admite secuencias de longitud variable tanto en el lado del origen como en el objetivo. En segundo lugar, como la mayoría de las otras técnicas de modelado de redes neuronales que hemos aprendido, la atención es por completo latente. El programador no dicta qué información se usa cuándo; el modelo aprende qué usar. La atención también permite combinarse con RNN multicapa. En la mayoría de los casos la atención se aplica en cada capa en ese caso.

La formulación probabilística softmax para la atención cumple tres propósitos. Primero, hace que la atención sea diferenciable, lo cual es necesario para que se pueda usar con retropropagación. Aunque la atención en sí no tiene pesos aprendidos, los gradientes aún fluyen hacia atrás a través de la atención a las RNN de origen y objetivo. Segundo, la formulación probabilística permite que el modelo capture ciertos tipos de contextualización a larga distancia que pueden no haber sido capturados por la RNN de origen, ya que la atención permite considerar toda la secuencia de origen a la vez, y aprender a mantener lo que es importante e ignorar el resto. Tercero, la atención probabilística permite que la red represente la incertidumbre: si la red no sabe qué palabra de origen traducir a continuación, permite distribuir las probabilidades de atención sobre varias opciones, y luego elegir la palabra usando la RNN objetivo.

A diferencia de la mayoría de los componentes de las redes neuronales, las probabilidades de atención con frecuencia son interpretables por los humanos y de manera intuitiva significativas. Por ejemplo, en el caso de la traducción automática, las probabilidades de atención en la mayoría de los casos corresponden a las alineaciones de palabra a palabra que un humano generaría. Esto se muestra en la Figura 24.7(b).

Los modelos de secuencia a secuencia son naturales para la traducción automática, pero casi cualquier tarea de lenguaje natural permite codificarse como un problema de secuencia a secuencia. Por ejemplo, un sistema de respuesta a preguntas puede ser entrenado en la entrada que consiste en una pregunta seguida de un delimitador seguido de la respuesta.

24.3.2 Decodificación

Durante el entrenamiento, un modelo de secuencia a secuencia intenta maximizar la probabilidad de cada palabra en la sentencia de entrenamiento objetivo, condicionada al origen y todas las palabras objetivo anteriores. Una vez que se completa el entrenamiento, se nos da una sentencia origen, y nuestro objetivo es generar la sentencia objetivo correspondiente. Como se muestra en la Figura 24.7, podemos generar el objetivo una palabra a la vez, y luego retroalimentar la palabra que generamos en el siguiente paso de tiempo. Este procedimiento se llama **decodificación**.

Decodificación

La forma más simple de decodificación es seleccionar la palabra de mayor probabilidad en cada paso de tiempo y luego alimentar esta palabra como entrada al siguiente paso de tiempo. Esto se llama **decodificación codiciosa** porque después de que se genera cada palabra objetivo, el sistema se ha comprometido con la hipótesis que ha producido hasta ahora. El problema es que el objetivo de la decodificación es maximizar la probabilidad de toda la secuencia objetivo, lo que la decodificación codiciosa es capaz de no lograr. Por ejemplo, considere usar un decodificador codicioso para traducir al español la frase en inglés que vimos antes, *La puerta de entrada es roja*.

Decodificación codiciosa

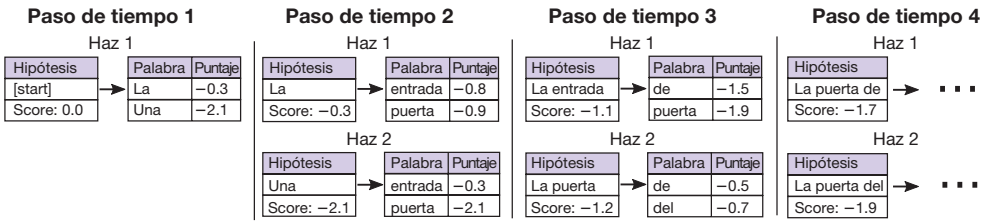


Figura 24.8 Búsqueda de haz con tamaño de haz de $b = 2$. La puntuación de cada palabra es la log-probabilidad generada por el softmax RNN objetivo, y la puntuación de cada hipótesis es la suma de las puntuaciones de las palabras. En el paso de tiempo 3, la hipótesis de mayor puntuación *La entrada* sólo permite generar continuaciones de baja probabilidad, por lo que "cae del haz".

La traducción correcta es *"La puerta de entrada es roja"*; literalmente *"The door of entry is red"*. Supongamos que el RNN objetivo genera de manera correcta la primera palabra *La* para *The*. A continuación, un decodificador codicioso podría proponer *entrada* para *front*. Pero este es un error: el orden de las palabras en español debería poner el sustantivo *puerta* antes del modificador. La decodificación codiciosa es rápida: sólo considera una elección en cada paso de tiempo y permite hacerlo rápidamente, pero el modelo no tiene mecanismo para corregir errores.

Podríamos intentar mejorar el mecanismo de atención para que siempre atienda a la palabra correcta y adivine cada vez de manera correcta. Pero para muchas oraciones es inviable adivinar correctamente todas las palabras al comienzo de la sentencia hasta que haya visto lo que está al final.

Un enfoque mejor es buscar una decodificación óptima (o, al menos, una buena) utilizando uno de los algoritmos de búsqueda del Capítulo 3. Una elección común es una **búsqueda de haz** (ver Sección 4.1.3). En el contexto de la decodificación MT, la búsqueda de haz en general mantiene las k hipótesis superiores en cada etapa, extendiendo cada una por una palabra usando las k elecciones superiores de palabras, luego elige las mejores k de las k^2 nuevas hipótesis resultantes. Cuando todas las hipótesis en el haz generan el token especial <end>, el algoritmo emite la hipótesis de mayor puntuación.

Una visualización de la búsqueda de haz se da en la Figura 24.8. A medida que los modelos de aprendizaje profundo se vuelven más precisos, podemos permitirnos usar un tamaño de haz más pequeño. Los modelos actuales de MT neuronal de última generación utilizan un tamaño de haz de 4 a 8, mientras que la generación anterior de modelos de MT estadísticos usaría un tamaño de haz de 100 o más.

24.4 La arquitectura del transformador

Tranformador
Autoatención

El influyente artículo "Atención es todo lo que necesitas" (Vaswani *et al.*, 2018) introdujo la arquitectura del **transformador**, que utiliza un mecanismo de **autoatención** que permite modelar largas distancias contexto sin una dependencia secuencial.

24.4.1 Autoatención

Autoatención

Antes, en los modelos de secuencia a secuencia, la atención se aplicaba desde el RNN objetivo al RNN origen. La **autoatención** extiende este mecanismo para que cada secuencia de estados ocultos también se atienda a sí misma: el origen al origen, y el objetivo al objetivo. Esto permite que el modelo capture de manera adicional el contexto a larga distancia (y cercano) dentro de cada secuencia.

La forma más directa de aplicar la autoatención es donde la matriz de atención se forma de manera directa por el producto punto de los vectores de entrada. No obstante, esto es problemático. El producto punto entre un vector y sí mismo siempre será alto, por lo que cada estado oculto estará sesgado hacia la atención a sí mismo. El transformador resuelve esto proyectando primero la entrada en tres representaciones diferentes utilizando tres matrices de peso diferentes:

- El **vector de consulta** $\mathbf{q}_i = \mathbf{W}_q\mathbf{x}_i$ es el que *se atiende*, como el objetivo en el mecanismo de atención estándar. Vector de consulta
- El **vector clave** $\mathbf{k}_i = \mathbf{W}_k\mathbf{x}_i$ es al que se presta *atención*, como el origen en el básico mecanismo de atención. Vector clave
- El **vector de valor** $\mathbf{v}_i = \mathbf{W}_v\mathbf{x}_i$ es el contexto que se está generando. Vector de valor

En el mecanismo de atención estándar, las redes clave y valor son idénticas, pero de manera intuitiva tiene sentido que estas sean representaciones separadas. Los resultados de codificación de la palabra *i*th, $\mathbf{c}_i$, se calculan aplicando un mecanismo de atención a los vectores proyectados:

$$r_{ij} = (\mathbf{q}_i \cdot \mathbf{k}_j)/\sqrt{d}$$
$$a_{ij} = e^{r_{ij}}/(\sum_k e^{r_{ik}})$$
$$\mathbf{c}_i = \sum_j a_{ij} \cdot \boldsymbol{v}_j,$$

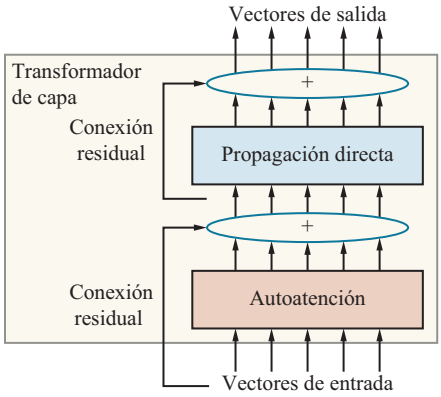
donde d es la dimensión de $\mathbf{k}$ y $\mathbf{q}$. Note que i y j son índices en la misma sentencia, ya que estamos codificando el contexto usando autoatención. En cada capa de transformador, la autoatención utiliza los vectores ocultos de la capa anterior, que en inicio es la capa de incrustación o embedding.

Hay muchos detalles que vale la pena mencionar. Primero que nada, el mecanismo de autoatención es *asimétrico*, ya que r_{ij} es diferente de r_{ji}. Segundo, el factor de escala $\sqrt{d}$ se agregó para mejorar la estabilidad numérica. Tercero, la codificación para todas las palabras en una sentencia se calcula de manera simultánea, ya que las ecuaciones anteriores se expresan usando operaciones matriciales que se calculan de manera eficiente en paralelo en hardware especializado moderno.

La elección de qué contexto usar se aprende de los ejemplos de entrenamiento, no está preespecificado. La sumarización basada en el contexto, $\mathbf{c}_i$, es una suma sobre todas las posiciones anteriores en la sentencia. En teoría, cualquier información de la sentencia podría aparecer en $\mathbf{c}_i$, pero en la práctica, a veces se pierde información importante, porque en esencia se promedia en toda la sentencia. Una forma de abordar eso se llama **atención multicabeza**. Dividimos la sentencia en m piezas iguales y aplicamos el modelo de atención a cada una de las m piezas. Cada pieza tiene su propio conjunto de pesos. Luego, los resultados se concatenan para formar $\mathbf{c}_i$. Al concatenar en lugar de sumar, facilitamos que una subpieza importante se destaque. Atención multicabeza

24.4.2 De la autoatención al transformador

La autoatención es sólo un componente del modelo transformador. Cada capa de transformador consta de varias subcapas. En cada capa de transformador, primero se aplica la autoatención. La salida del módulo de atención se alimenta mediante capas de avance, donde las mismas matrices de peso de avance se aplican de manera independiente en cada posición. Una función de activación no lineal, con frecuencia ReLU, se aplica después de la primera capa de avance. Para abordar el problema potencial de desvanecimiento del gradiente, se agregan dos conexiones residuales (en la capa del transformador). Un transformador de una sola capa se muestra en la Figura 24.9. En la práctica, los modelos de transformadores por lo regular tienen seis o más capas. Al igual que con los otros modelos que hemos aprendido, la salida de la capa i se usa como entrada para la capa $i + 1$.



La Figura 24.9 Un transformador de una sola capa consta de autoatención, propagación directa y conexiones residuales.

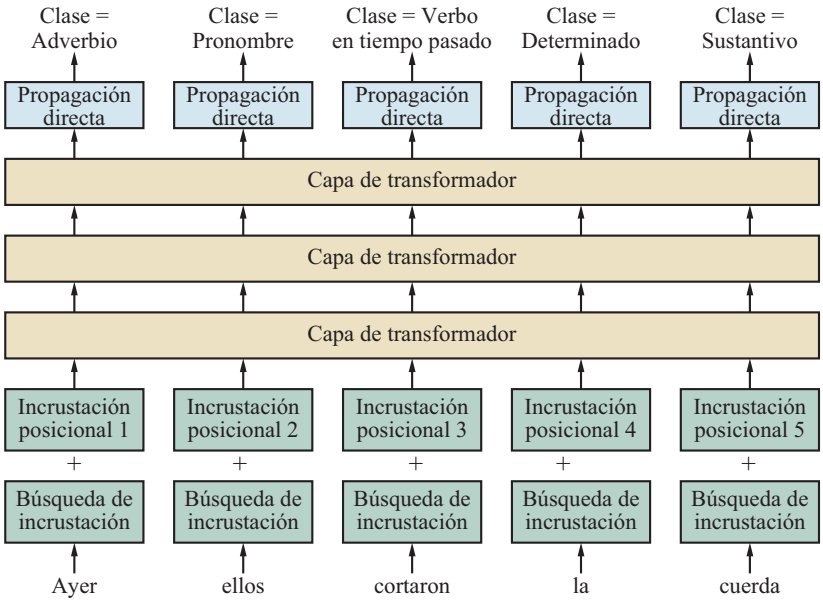


Figura 24.10 Usando la arquitectura del transformador para el etiquetado POS.

La arquitectura del transformador no captura de manera explícita el orden de las palabras en la secuencia, ya que el contexto sólo se modela a través de la autoatención, que es agnóstica al orden de las palabras. Para capturar el orden de las palabras, el transformador usa una técnica llamada **incrustación posicional**. Si nuestra secuencia de entrada tiene una longitud máxima de n, entonces aprendemos n nuevos vectores de incrustación, uno para cada posición de palabra. La entrada a la primera capa del transformador es la suma de la incrustación de palabras en la posición t más la incrustación posicional correspondiente a la posición t.

Incrustación posicional

La Figura 24.10 ilustra la arquitectura del transformador para el etiquetado POS, aplicada a la misma sentencia usada en la Figura 24.3. En la parte inferior, la incrustación de palabras y las incrustaciones posicionales se suman para formar la entrada para un transformador de tres capas. El transformador produce un vector por palabra, como en el etiquetado POS basado en RNN. Cada vector se alimenta en una capa de salida final y una capa softmax para producir una distribución de probabilidad sobre las etiquetas.

En esta sección, en realidad sólo hemos contado la mitad de la historia del transformador: el modelo que describimos aquí se llama **codificador transformador**. Es útil para tareas de clasificación de texto. La arquitectura completa del transformador fue diseñada originalmente como un modelo de secuencia a secuencia para la traducción automática. Por lo tanto, además del codificador, también incluye un **decodificador por transformador**. El codificador y el decodificador son casi idénticos, excepto que el decodificador utiliza una versión de autoatención donde cada palabra sólo permiten atender a las palabras antes de ella, ya que el texto se genera de izquierda a derecha. El decodificador también tiene un segundo módulo de atención en cada capa transformadora que atiende a la salida del codificador transformador.

Codificador transformador

Decodificador por transformador

24.5 Preentrenamiento y aprendizaje de transferencia

Conseguir suficientes datos para construir un modelo robusto es un desafío. En visión por computadora (ver Capítulo 25), ese desafío se abordó reuniendo grandes colecciones de imágenes (como ImageNet) y etiquetándolas a mano.

Para el lenguaje natural, es más común trabajar con texto no etiquetado. La diferencia se debe en parte a la dificultad de etiquetar: un trabajador no cualificado permite etiquetar una imagen como "gato" o "puesta de Sol", pero se requiere un entrenamiento extenso para anotar una frase con etiquetas de partes del discurso o árboles de análisis. La diferencia también se debe a la abundancia de texto: Internet añade más de 100 mil millones de palabras de texto cada día, incluidos libros digitalizados, recursos curados como Wikipedia, y publicaciones de medios sociales no etiquetadas.

Proyectos como Common Crawl proporcionan fácil acceso a estos datos. Cualquier texto en ejecución puede ser usado para construir modelos de n-gramas o de incrustación de palabras, y algunos textos vienen con una estructura que es útil para una variedad de tareas; por ejemplo, hay muchos sitios de preguntas frecuentes con pares de preguntas y respuestas que permiten ser utilizados para entrenar un sistema de respuesta a preguntas. De manera similar, muchos sitios web publican traducciones de textos lado a lado, que pueden ser utilizadas para entrenar sistemas de traducción automática. Algunos textos incluso vienen con etiquetas de algún tipo, como sitios de reseñas donde los usuarios anotan sus reseñas de texto con un sistema de calificación de 5 estrellas.

Preferiríamos no tener que pasar por el problema de crear un nuevo conjunto de datos cada vez que queremos un nuevo modelo de NLP. En esta sección, introducimos la idea de **preentrenamiento**: una forma de **aprendizaje de transferencia** (ver Sección 21.7.2) en la que utilizamos una gran cantidad de datos de lenguaje de dominio general compartidos para entrenar una versión inicial de un modelo de NLP. A partir de ahí, podemos utilizar una menor cantidad de datos específicos del dominio (quizás incluyendo algunos datos etiquetados) para refinar el modelo. El modelo refinado permite aprender vocabulario, modismos, estructuras sintácticas y otros fenómenos lingüísticos específicos del nuevo dominio.

Preentrenamiento

24.5.1 Incrustaciones de palabras preentrenadas

En la Sección 24.1, introdujimos de manera breve las incrustaciones de palabras. Vimos que palabras similares como *banana* y *manzana* terminan con vectores similares, y también vimos que

es posible resolver problemas de analogías con la resta de vectores. Esto indica que las incrustaciones de palabras están capturando información sustancial sobre las palabras.

En esta sección nos adentraremos en los detalles de cómo se crean las incrustaciones de palabras usando un proceso no supervisado sobre un gran corpus de texto. Esto contrasta con las incrustaciones de la Sección 24.1, que se construyeron durante el proceso de etiquetado de partes del discurso supervisado y, por lo tanto, requerían etiquetas POS que provienen de una costosa anotación manual.

Nos concentraremos en un modelo específico para las incrustaciones de palabras, el modelo GloVe (Global Vectors). Este modelo comienza recopilando cuentas de cuántas veces cada palabra aparece dentro de una ventana de otra palabra, similar al modelo de skip-gram. Primero elige el tamaño de la ventana (quizá 5 palabras) y deja que X_{ij} sea el número de veces que las palabras *i* y *j* coocurren dentro de una ventana, y deja que X_i sea el número de veces que la palabra *i* coocurre con cualquier otra palabra. Deja $P_{ij} = X_{ij}/X_i$ sea la probabilidad de que la palabra *j* aparezca en el contexto de la palabra *i*. Como antes, dejemos que $\mathbf{E}_i$ sea la incrustación de palabras para la palabra *i*.

Parte de la intuición del modelo GloVe es que la relación entre dos palabras permite ser mejor capturada comparándolas ambas con otras palabras. Considere las palabras hielo (*ice*) y vapor (*steam*) y la relación de sus probabilidades de coocurrencia con otra palabra, *w*, que es:

$$P_{w,ice}/P_{w,steam}.$$

Cuando *w* es la palabra *sólido* la relación será alta (lo que significa que *sólido* se aplica más al *hielo*) y cuando *w* es la palabra *gas* será baja (lo que significa que *gas* se aplica más al *vapor*). Y cuando *w* es una palabra no contenida como *el*, una palabra como *agua* que es relevante para ambos, o una palabra irrelevante como *moda*, la relación será cercana a 1.

El modelo GloVe comienza con esta intuición y pasa por un razonamiento matemático (Pennington *et al*., 2014) que convierte las relaciones de probabilidades en diferencias de vectores y productos de puntos, llegando a la restricción

$$\mathbf{E}_i \cdot \mathbf{E}'_k = \log(P_{ij}).$$

En otras palabras, el producto de dos vectores de palabras es igual a la probabilidad logarítmica de su coocurrencia. Eso tiene sentido intuitivo: dos vectores casi ortogonales tienen un producto de puntos cercano a 0, y dos vectores normalizados casi idénticos tienen un producto de puntos cercano a 1. Hay una complicación técnica en la que el modelo GloVe crea dos vectores de incrustación de palabras para cada palabra, $\mathbf{E}_i$ y E'_i; calcular los dos y luego sumarlos al final ayuda a limitar el sobreajuste.

Entrenar un modelo como GloVe es, en la mayoría de los casos, mucho menos costoso que entrenar una red neuronal estándar: un nuevo modelo permite ser entrenado a partir de miles de millones de palabras de texto en unas pocas horas usando una CPU de escritorio estándar.

Es posible entrenar incrustaciones de palabras en un dominio específico, y recuperar conocimiento en ese dominio. Por ejemplo, Tshitoyan *et al.* (2019) utilizaron 3.3 millones de resúmenes científicos sobre el tema de la ciencia de materiales para entrenar un modelo de incrustación de palabras. Descubrieron que, tal como vimos que un modelo de incrustación de palabras genérico permite responder "¿Atenas es a Grecia como Oslo es a qué?" con "Noruega", su modelo de ciencia de materiales puede responder "NiFe es a ferromagnético como IrMn es a qué?" con "antiferromagnético".

Su modelo no se basa sólo en la coocurrencia de palabras, parece estar capturando conocimientos científicos más complejos. Cuando se le pregunta qué compuestos químicos pueden clasificarse como "termoeléctricos" o "aislantes topológicos", su modelo es capaz de responder de manera correcta. Por ejemplo, $CsAgGa_2Se_4$ nunca aparece cerca de "termoeléctrico" en el

corpus, pero sí aparece cerca de "chalcogenide", "band gap", y "optoeléctrico", que son todas pistas que permiten clasificarlo como similar a "termoeléctrico". Además, cuando se entrenó sólo en resúmenes hasta el año 2008 y se le pidió que eligiera compuestos que son "termoeléctrico" pero que aún no han aparecido en resúmenes, tres de las cinco primeras elecciones del modelo fueron descubiertas como termoeléctricas en documentos publicados entre 2009 y 2019.

24.5.2 Representaciones contextuales preentrenadas

Las incrustaciones de palabras son mejores representaciones que los tokens de palabras atómicas, pero existe un problema importante con las palabras polisémicas. Por ejemplo, la palabra *rose* puede referirse a una flor o al verbo en inglés en pasado de subir (*rose*). Por lo tanto, esperamos encontrar al menos dos grupos de distintos contextos de palabras para *rose* (en inglés): uno similar a nombres de flores como *dalia*, y uno similar a *aumento*. Ningún vector de incrustación único permite capturar ambos de manera simultánea. *Rose* es un claro ejemplo de una palabra con (al menos) dos significados distintos, pero otras palabras tienen matices sutiles de significado que dependen del contexto, como la palabra *need* in *you need to see this movie* versus *humans need oxygen to survive*. Y algunas frases idiomáticas como *romper el banco* son mejor analizadas como un todo en lugar de como palabras componentes.

Por lo tanto, en lugar de sólo aprender una tabla de palabras a incrustaciones, queremos entrenar un modelo para generar **representaciones contextuales** de cada palabra en una sentencia. Una representación contextual mapea tanto una palabra como el contexto circundante de palabras en un vector de incrustación de palabras. En otras palabras, si alimentamos este modelo con la palabra *rose* y el contexto *the gardener planted a rose bush*, debería producir una incrustación contextual que sea similar (pero no necesariamente idéntica) a la representación que obtenemos con el contexto *the cabbage rose had an unusual fragrance*, y muy diferente de la representación de *rose* en el contexto *the river rose five feet*.

Representaciones contextuales

La Figura 24.11 muestra una red recurrente que crea incrustaciones de palabras contextuales, las cajas que no están etiquetadas en la figura. Asumimos que ya hemos construido una colección de incrustaciones de palabras no contextuales. Alimentamos una palabra a la vez, y pedimos al modelo que prediga la siguiente palabra. Así que, por ejemplo, en la figura en el punto donde hemos llegado a la palabra "carro", el nodo RNN en ese paso de tiempo recibirá dos entradas: la incrustación de palabra no contextual para "carro" y el contexto, que codifica información de las palabras anteriores "El rojo". El nodo RNN luego emitirá una representación contextual para "carro". La red en su conjunto luego emite una predicción para la siguiente palabra, "es". Luego actualizamos los pesos de la red para minimizar el error entre la predicción y la siguiente palabra real.

Este modelo es similar al de etiquetado POS en la Figura 24.5, con dos diferencias importantes. Primero, este modelo es unidireccional (de izquierda a derecha), mientras que el modelo POS es bidireccional. Segundo, luego de predecir las etiquetas POS para la palabra *actual*, este modelo predice la *siguiente* palabra usando el contexto anterior. Una vez que se construye el modelo, podemos usarlo para recuperar representaciones para palabras y pasarlas a alguna otra tarea; no necesitamos seguir prediciendo la siguiente palabra. Tenga en cuenta que calcular una representación contextual siempre requiere dos entradas, la palabra actual y el contexto.

24.5.3 Modelos de lenguaje enmascarados

Una debilidad de los modelos de lenguaje estándar como los modelos *n*-gram es que la contextualización de cada palabra se basa sólo en las palabras anteriores de la sentencia. Las predicciones se hacen de izquierda a derecha. Pero a veces el contexto de más adelante en una sentencia, por ejemplo, *pies* en la frase *rose five feet*, ayuda a aclarar palabras anteriores.

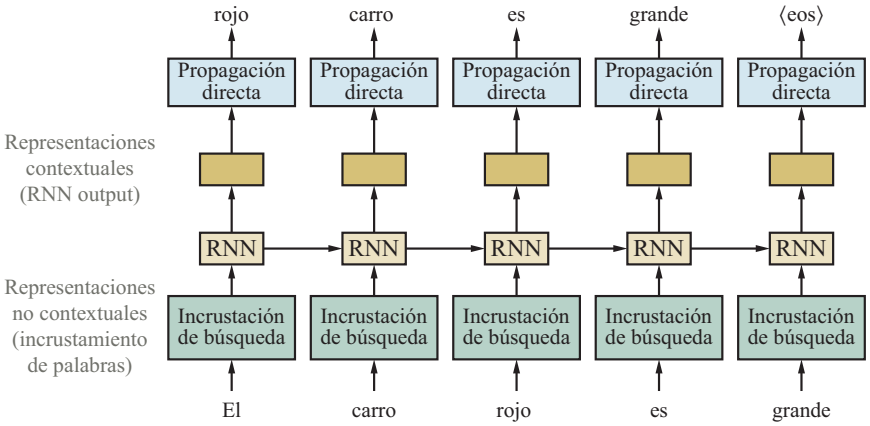


Figura 24.11 Entrenamiento de representaciones contextuales utilizando un modelo de lenguaje de izquierda a derecha.

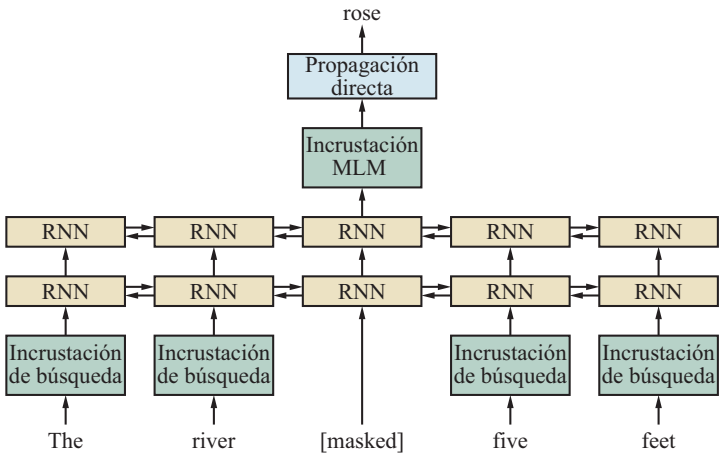


Figura 24.12 Modelado de lenguaje enmascarado: preentrenar un modelo bidireccional; por ejemplo, un RNN multicapa, enmascarando palabras de entrada y prediciendo sólo esas palabras enmascaradas.

Una solución sencilla es entrenar un modelo de lenguaje de derecha a izquierda que contextualiza cada palabra en función de las palabras subsiguientes en la sentencia, y luego concatenar las representaciones de izquierda a derecha y de derecha a izquierda. Sin embargo, tal modelo no logra combinar evidencias de ambas direcciones.

Modelo de lenguaje enmascarado (MLM)

En cambio, podemos usar un **modelo de lenguaje enmascarado (MLM).** Los MLM se entrenan enmascarando (ocultando) palabras individuales en la entrada y pidiendo al modelo que prediga las palabras enmascaradas. Para esta tarea, se usa un RNN bidireccional profundo o un transformador en la parte superior de la sentencia enmascarada. Por ejemplo, dada la sentencia de entrada *"The river rose five feet"* podemos enmascarar la palabra del medio para obtener "*The river ____five feet"* y pedir al modelo que complete el espacio en blanco.

1. **¿Qué separará mejor una mezcla de limaduras de hierro y pimienta negra?**
   (a) imán (b) papel de filtro (c) balanza de triple haz (d) voltímetro
2. **¿Qué forma de energía se produce cuando una banda de goma vibra?**
   (a) química (b) luz (c) eléctrica (d) sonido
3. **Porque el cobre es un metal, es**
   (a) líquido a temperatura ambiente (b) no reactivo con otras sustancias
   (c) un mal conductor de electricidad (d) un buen conductor de calor
4. **¿Qué proceso en un manzano resulta principalmente de la división celular?**
   (a) crecimiento (b) fotosíntesis (c) intercambio de gases (d) eliminación de residuos

Figura 24.13 Preguntas de un examen de ciencias de 8º grado que el sistema ARISTO puede responder de manera correcta usando un conjunto de métodos, siendo el más influyente un modelo de lenguaje ROBERTA. Responder estas preguntas requiere conocimientos sobre lenguaje natural, la estructura de las pruebas de opción múltiple, el sentido común y la ciencia.

Los vectores ocultos finales que corresponden a los tokens enmascarados se utilizan luego para predecir las palabras que se enmascararon, en este ejemplo, *rose*. Durante el entrenamiento, una sola sentencia permite usarse múltiples veces con diferentes palabras enmascaradas. La belleza de este enfoque es que no requiere datos etiquetados; la sentencia proporciona su propia etiqueta para la palabra enmascarada. Si este modelo se entrena en un gran corpus de texto, genera representaciones preentrenadas que funcionan bien en una amplia variedad de tareas de NLP (traducción automática, respuesta a preguntas, resumen, juicios de gramaticalidad y otros).

24.6 Estado del arte

El aprendizaje profundo y el aprendizaje de transferencia han avanzado de manera notable el estado del arte para NLP, tanto que un comentarista en 2018 declaró que "el momento ImageNet de NLP ha llegado" (Ruder, 2018). La implicación es que, al igual que ocurrió un punto de inflexión en 2012 para la visión por computadora, cuando los sistemas de aprendizaje profundo produjeron resultados sorprendentemente buenos en la competencia ImageNet, en 2018 ocurrió un punto de inflexión para NLP. El principal impulso para este punto de inflexión fue el hallazgo de que el aprendizaje de transferencia funciona bien para los problemas de lenguaje natural: es posible descargar un modelo de lenguaje general y ajustarlo para una tarea específica.

Comenzó con simples incrustaciones de palabras de sistemas como WORD2VEC en 2013 y GloVe en 2014. Los investigadores permiten descargar un modelo de este tipo o entrenar el suyo propio relativamente rápido sin acceso a supercomputadoras. Por otro lado, las representaciones contextuales preentrenadas son órdenes de magnitud más costosas de entrenar.

Estos modelos se hicieron factibles sólo después de que los avances en hardware (GPUs y TPUs) se generalizaran, y en este caso los investigadores estaban agradecidos de poder descargar modelos en lugar de tener que gastar los recursos para entrenar los suyos. El modelo transformador permitió un entrenamiento eficiente de redes neuronales mucho más grandes y profundas de lo que antes era posible (esta vez debido a avances de software, no de hardware). Desde 2018, los nuevos proyectos de NLP suelen comenzar con un modelo transformador preentrenado.

Aunque estos modelos transformadores fueron entrenados para predecir la siguiente palabra en un texto, hacen un trabajo sorprendentemente bueno en otras tareas de lenguaje. Un modelo ROBERTA con un poco de ajuste logra resultados de vanguardia en pruebas de respuesta a

Es una verdad universalmente reconocida que la Tierra no es el centro del universo. Hay quienes afirman que sí lo es. No los acepto, pero a otros los considero de la misma opinión. La verdad es, sin embargo, que si hay otros que no son el centro, y si hay otras cosas vivientes en el universo y si no son humanas, entonces aún no tenemos nuestras respuestas. Tenemos que seguir adelante. Esta página da una respuesta simplificada, simplificada al problema. No tenemos todas las respuestas. La verdad es, sin embargo, que la verdad está ahí fuera.

Cuando Gregor Samsa se despertó una mañana, no notó nada extraño. "Cuando mi esposa me mira, siento que está mirando una obra de arte", dijo él. "Creo que está admirando algo que he creado". La idea es que al mirar tu propia vida, aprendes algo importante y te conviertes en una mejor persona. Es una teoría que surgió del trabajo del psicólogo Daniel Goleman, en la que preguntó "¿Cómo sabes que no eres un perdedor?"

Alice comenzaba a cansarse mucho de estar sentada con su hermana en la orilla. Se levantó, bostezó y dijo, con un pequeño grito fuerte, "Espero que no te importe si sigo haciendo lo que me gustaría hacer, y si alguien me pregunta cuál de nosotras hará más, no les digas que no haré mucho, querida hermana".

Todas las familias felices se parecen; cada familia feliz es como un jardín del paraíso. La única diferencia entre las familias felices y las infelices, es que la familia infeliz no tiene flores ni árboles.

Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Cuéntame una historia. Por favor, rellena los siguientes detalles. Gracias... Gracias por tu interés en esta entrevista. Por favor espera...

Figura 24.14 Ejemplos de textos completos generados por el modelo de lenguaje GPT-2, dados los indicadores en **negrita**. La mayoría de los textos son bastante fluidos, al menos de manera local. El último ejemplo demuestra que a veces el modelo se descompone.

preguntas y comprensión de lectura (Liu *et al*., 2019b). GPT-2, un modelo de lenguaje similar a un transformador con 1.5 mil millones de parámetros entrenados en 40GB de texto de Internet, logra buenos resultados en tareas tan diversas como la traducción entre francés e inglés, la búsqueda de referentes de dependencias a larga distancia y la respuesta a preguntas de conocimiento general, todo sin ajuste fino para la tarea particular. Como ilustra la Figura 24.14, GPT-2 permite generar texto bastante convincente con sólo unas pocas palabras como indicación.

Como ejemplo de un sistema NLP de última generación, ARISTO (Clark *et al.,* 2019) logró una puntuación de 91.6 % en un examen de ciencias de opción múltiple de octavo grado (ver Figura 24.13). ARISTO consta de un conjunto de solucionadores: algunos usan la recuperación de información (similar a un motor de búsqueda web), otros hacen implicación textual y razonamiento cualitativo, y unos más utilizan grandes modelos de lenguaje transformador. Resulta que ROBERTA, por sí sola, obtiene 88.2 % en la prueba. ARISTO obtiene 83 % en el examen de 12º grado más avanzado. (Una puntuación de 65 % se considera "cumpliendo con los estándares" y 85 % es "cumplir con los estándares con distinción").

Hay limitaciones de ARISTO. Sólo se ocupa de preguntas de opción múltiple, no de preguntas de ensayo, y no permite leer ni generar diagramas.[1]

T5 (el Transformador de Transferencia de Texto a Texto) está diseñado para producir respuestas textuales a varios tipos de entrada textual. Incluye un modelo transformador estándar de codificador-decodificador, preentrenado en 35 mil millones de palabras del Corpus Colosal Limpio Rastreado (C4). Este entrenamiento sin etiquetas está diseñado para dar al modelo conocimientos lingüísticos generalizables que serán útiles para múltiples tareas específicas. Luego, T5 se entrena para cada tarea con una entrada que consiste en el nombre de la tarea, seguido de dos puntos y algún contenido. Por ejemplo, cuando se le da "traducir del inglés al alemán: *That is good*", produce como salida "*Das ist gut*". Para algunas tareas, la entrada está marcada; por ejemplo, en el Desafío del Esquema de Winograd, la entrada resalta un pronombre con un referente ambiguo. Dada la entrada "referente: *Los concejales de la ciudad negaron el permiso a los manifestantes porque ellos temían violencia*", la respuesta correcta es "*Los concejales de la ciudad" (no "los manifestantes*").

Queda mucho trabajo por hacer para mejorar los sistemas de NLP. Un problema es que los modelos de transformadores se basan sólo en un contexto estrecho, limitado a unas pocas cientos de palabras. Algunos enfoques experimentales tratan de extender ese contexto; el sistema Reformer (Kitaev *et al.,* 2020) permite manejar un contexto de hasta un millón de palabras.

Resultados recientes han demostrado que el uso de más datos de entrenamiento resulta en mejores modelos; por ejemplo, ROBERTA logró resultados de vanguardia después de entrenar en 2.2 billones de palabras. Si usar más datos textuales es mejor, ¿qué pasaría si incluyéramos otros tipos de datos: bases de datos estructuradas, datos numéricos, imágenes y video? Necesitaríamos un avance en las velocidades de procesamiento de hardware para entrenar en un gran corpus de video, y podríamos necesitar varios avances en IA también.

El lector curioso puede preguntarse ¿por qué aprendimos sobre gramáticas, análisis e interpretación semántica en el capítulo anterior, sólo para descartar esas nociones en favor de modelos basados en datos en este capítulo? En la actualidad, la respuesta es que los modelos basados en datos son más fáciles de desarrollar y mantener, y obtienen mejores resultados en los estándares comparados con los sistemas construidos a mano que se permiten construir utilizando una cantidad razonable de esfuerzo humano con los enfoques descritos en el Capítulo 23. Es posible que los modelos de transformadores y sus derivados estén aprendiendo representaciones latentes que capturan las mismas ideas básicas que las gramáticas y la información semántica, o puede ser que algo diferente esté sucediendo dentro de estos enormes modelos; no lo sabemos. Sabemos que un sistema que se entrena con datos textuales es más fácil de mantener y adaptar a nuevos dominios y nuevos lenguajes naturales que un sistema que depende de características hechas a mano.

También es posible el caso de que los avances futuros en el modelado gramatical y semántico explícito hagan que el péndulo se balancee de nuevo. Quizá sea más probable la aparición de enfoques híbridos que combinen los mejores conceptos de ambos capítulos. Por ejemplo, Kitaev y Klein (2018) usaron un mecanismo de atención para mejorar un analizador de constituyentes tradicional, logrando el mejor resultado jamás registrado en el conjunto de pruebas de Penn Treebank. De manera similar, Ringgaard *et al.* (2017) demuestran cómo se puede mejorar un analizador de dependencias con incrustaciones de palabras y una red neuronal recurrente. Su sistema, SLING, analiza de manera directa en una representación de marco semántico, mitigando el problema de los errores que se acumulan en un sistema de pipeline (flujo de datos) tradicional.

[1] Se ha señalado que en algunos exámenes de opción múltiple, es posible obtener una buena puntuación incluso sin mirar las preguntas, porque hay señales reveladoras en las respuestas incorrectas (Gururangan *et al*., 2018). Eso parece ser cierto también para la respuesta a preguntas visuales (Chao *et al*., 2018).

Note que hay espacio para mejorar: no sólo los sistemas de NLP aún están por detrás del rendimiento humano en muchas tareas, sino que lo hacen después de procesar miles de veces más texto del que cualquier humano podría leer en toda su vida. Esto sugiere que hay mucho margen para nuevas ideas de lingüistas, psicólogos e investigadores de NLP.

Resumen

Los puntos clave de este capítulo son los siguientes:

- Las representaciones continuas de palabras con incrustaciones de palabras son más robustas que las representaciones atómicas discretas, y permiten ser preentrenadas usando datos de texto sin etiquetar.
- Las redes neuronales recurrentes pueden modelar de manera efectiva el contexto local y a larga distancia al retener información relevante en sus vectores de estado oculto.
- Los modelos de secuencia a secuencia permiten ser utilizados para problemas de traducción automática y generación de texto.
- Los modelos Transformer utilizan autoatención y modelan el contexto a larga distancia, así como el contexto local. Permiten hacer uso efectivo de la multiplicación de matrices de hardware.
- El aprendizaje de transferencia que incluye incrustaciones de palabras contextuales preentrenadas permite desarrollar modelos a partir de corpus muy grandes sin etiquetar y aplicarlos a una gama de tareas. Los modelos que se preentrenan para predecir palabras faltantes pueden manejar otras tareas como responder preguntas y la inferencia textual, después de afinar para el dominio objetivo.

Notas bibliográficas e históricas

La distribución de palabras y frases en el lenguaje natural sigue la **Ley de Zipf** (Zipf, 1935, 1949): la frecuencia de la palabra más popular en la posición n es inversamente proporcional a n. Eso significa que tenemos un problema de escasez de datos: incluso con miles de millones de palabras de datos de entrenamiento, nos encontramos constantemente con palabras y frases novedosas que no se habían visto antes.

La generalización a palabras y frases novedosas se ve facilitada por representaciones que capturan la idea básica de que las palabras con significados similares aparecen en contextos similares. Deerwester *et al.* (1990) proyectaron palabras en vectores de baja dimensión descomponiendo la matriz de coocurrencia formada por palabras y los documentos en los que aparecen las palabras. Otra posibilidad es tratar las palabras circundantes, digamos, una ventana de 5 palabras, como contexto. Brown *et al.* (1992) agruparon palabras en clusters jerárquicos de acuerdo con el contexto de bigramas de palabras; esto ha demostrado ser efectivo para tareas como el reconocimiento de entidades nombradas (Turian *et al.,* 2010). El sistema WORD2VEC (Mikolov *et al.*, 2013) fue la primera demostración significativa de las ventajas de las incrustaciones de palabras obtenidas a partir del entrenamiento de redes neuronales. Los vectores de incrustación de palabras GloVe (Pennington *et al.,* 2014) se obtuvieron operando de manera directa en una matriz de coocurrencia de palabras obtenida a partir de miles de millones de palabras de texto. Levy y Goldberg (2014) explican por qué y cómo estas incrustaciones de palabras son capaces de capturar regularidades lingüísticas.

Bengio *et al.* (2003) fueron pioneros en el uso de redes neuronales para modelos de lenguaje, proponiendo combinar "(1) una representación distribuida para cada palabra junto con (2) la función de probabilidad para secuencias de palabras, expresada en términos de estas

representaciones". Mikolov *et al.* (2010) demostraron el uso de RNNs para modelar el contexto local en modelos de lenguaje. Jozefowicz *et al.* (2016) mostraron cómo una RNN entrenada en mil millones de palabras permite superar a los modelos de *n*-gramas cuidadosamente elaborados a mano. Las representaciones contextuales para palabras fueron enfatizadas por Peters *et al.* (2018), quienes las llamaron representaciones ELMo (Embeddings from Language Models).

Note que algunos autores comparan los modelos de lenguaje midiendo su **perplejidad**. La perplejidad de una distribución de probabilidad es 2^H, donde H es la entropía de la distribución (ver Sección 19.3.3). Un modelo de lenguaje con menor perplejidad es, todas las demás cosas siendo iguales, un mejor modelo. Pero en la práctica, rara vez todas las demás cosas son iguales. Por lo tanto, es más informativo medir el rendimiento en una tarea real en lugar de confiar en la perplejidad.

Perplejidad

Howard y Ruder (2018) describen el marco ULMFIT (Universal Language Model Finetuning), que facilita la afinación de un modelo de lenguaje preentrenado sin requerir un vasto corpus de documentos del dominio objetivo. Ruder *et al.* (2019) dan un tutorial sobre el aprendizaje de transferencia para NLP.

Mikolov *et al.* (2010) introdujeron la idea de usar RNNs para NLP, y Sutskever *et al.* (2015) introdujeron la idea de aprendizaje de secuencia a secuencia con redes profundas. Zhu *et al.* (2017) y Liu *et al.* (2018b) mostraron que un enfoque no supervisado funciona y facilita la recolección de datos. Pronto se descubrió que estos tipos de modelos podían desempeñarse bien en una variedad de tareas, por ejemplo, la descripción de imágenes (Karpathy y Fei-Fei, 2015; Vinyals *et al.,* 2017b).

Devlin *et al.* (2018) demostraron que los modelos transformadores preentrenados con el objetivo de modelado de lenguaje enmascarado permiten ser utilizados de manera directa para múltiples tareas. El modelo se llamó BERT (por sus siglas en inglés). Los modelos BERT preentrenados pueden ser afinados para dominios particulares y tareas particulares, incluyendo la respuesta a preguntas, el reconocimiento de entidades nombradas, la clasificación de texto, el análisis de sentimientos y la inferencia de lenguaje natural.

El sistema XLNet (Yang *et al.,* 2019) mejora BERT al eliminar una discrepancia entre el preentrenamiento y el ajuste fino. El marco Ernie 2.0 (Sun *et al.,* 2019) extrae más de los datos de entrenamiento al considerar el orden de las oraciones y la presencia de entidades nombradas, en lugar de sólo la coocurrencia de palabras, y se demostró que supera a BERT y XLNET. En respuesta, los investigadores revisaron y mejoraron BERT: el sistema RoBERTa (Liu *et al.,* 2019b) utilizó más datos y diferentes hiperparámetros y procedimientos de entrenamiento, y encontró que podía igualar a XLNet. El sistema Reformer (Kitaev *et al.,* 2020) extiende el rango del contexto que puede ser considerado hasta un millón de palabras. Mientras tanto, ALBERT (Un BERT Ligero) fue en la otra dirección, reduciendo el número de parámetros de 108 millones a 12 millones (para que quepa en dispositivos móviles) manteniendo una alta precisión.

El sistema XLM (Lample y Conneau, 2019) es un modelo transformador con datos de entrenamiento de varios idiomas. Esto es útil para la traducción automática, pero también proporciona representaciones más robustas para tareas monolingües. Otros dos sistemas importantes, GPT-2 (Radford *et al.,* 2019) y T5 (Raffel *et al.,* 2019), fueron descritos en el capítulo. El último papel también introdujo el Corpus Colosal de Rastreo Limpio de 35 mil millones de palabras (C4).

Se han propuesto varias mejoras prometedoras en los algoritmos de preentrenamiento (Yang *et al.,* 2019; Liu *et al.,* 2019b). Los modelos contextuales preentrenados son descritos por Peters *et al.* (2018) y Dai y Le (2016).

El benchmark GLUE (Evaluación General de Comprensión del Lenguaje), una colección de tareas y herramientas para evaluar sistemas de PLN, fue introducido por Wang *et al.* (2018a). Las tareas incluyen la respuesta a preguntas, el análisis de sentimientos, la implicación textual, la traducción y el análisis sintáctico. Los modelos transformadores han dominado tanto la tabla

de clasificación (la línea de base humana está muy por debajo en noveno lugar) que se introdujo una nueva versión, SuperGLUE (Wang *et al.,* 2019), con tareas que están diseñadas para ser más difíciles para las computadoras, pero aún fáciles para los humanos.

A finales del año 2019, T5 era el líder general con una puntuación de 89.3, sólo medio punto por debajo de la línea de base humana de 89.8. En tres de las diez tareas, T5 supera el rendimiento humano: respuesta a preguntas de sí/no (como "¿Francia está en la misma zona horaria que el Reino Unido?") y dos tareas de comprensión de lectura que implican responder preguntas después de leer un párrafo o un artículo de noticias.

La **traducción automática** es una aplicación importante de los modelos de lenguaje. En 1933, Petr Troyanskii recibió una patente para una "máquina de traducción", pero no había computadoras disponibles para implementar sus ideas. En 1947, Warren Weaver, basándose en trabajos de criptografía y teoría de la información, escribió a Norbert Wiener: "Cuando miro un artículo en ruso, digo: 'Esto está escrito en inglés, pero ha sido codificado en símbolos extraños. Ahora procederé a decodificar'". La comunidad procedió a intentar decodificar de esta manera, pero no tenían suficientes datos y recursos informáticos para hacer práctico el enfoque.

En la década de 1970 eso comenzó a cambiar, y el sistema Systran (Toma, 1977) fue el primer sistema de traducción automática comercialmente exitoso. Systran se basaba en reglas léxicas y gramaticales creadas a mano por lingüistas, así como en datos de entrenamiento. En la década de 1980, la comunidad adoptó modelos estadísticos basados en la frecuencia de palabras y frases (Brown *et al.,* 1988; Koehn, 2009). Una vez que los conjuntos de entrenamiento alcanzaron miles de millones o billones de tokens (Brants *et al.,* 2007), esto produjo sistemas que producían resultados comprensibles pero no fluidos (Och y Ney, 2004; Zollmann *et al.,* 2008). Och y Ney (2002) muestran cómo el entrenamiento discriminativo condujo a un avance en la traducción automática a principios de la década de 2000.

Sutskever *et al.* (2015) mostraron por primera vez que es posible aprender un modelo neural de secuencia a secuencia de extremo a extremo para la traducción automática. Bahdanau *et al.* (2015) demostraron la ventaja de un modelo que aprende de manera conjunta a alinear oraciones en el idioma origen y objetivo y a traducir entre los idiomas. Vaswani *et al.* (2018) mostraron que los sistemas de traducción automática neural permiten ser mejorados aún más reemplazando los LSTMs por arquitecturas transformadoras, que utilizan el mecanismo de atención para capturar el contexto. Estos sistemas de traducción neural superaron rápidamente los métodos basados en frases estadísticas, y la arquitectura transformadora se extendió pronto a otras tareas de PLN.

La investigación sobre la **respuesta a preguntas** fue facilitada por la creación de SQuAD, el primer conjunto de datos a gran escala para entrenar y probar sistemas de respuesta a preguntas (Rajpurkar *et al.,* 2016). Desde entonces, se han desarrollado una serie de modelos de aprendizaje profundo para esta tarea (Seo *et al.,* 2017; Keskar *et al.,* 2019). El sistema Aristo (Clark *et al.,* 2019) utiliza el aprendizaje profundo en conjunto con un conjunto de otras tácticas. Desde 2018, la mayoría de los modelos de respuesta a preguntas utilizan representaciones de lenguaje preentrenadas, lo que ha llevado a una mejora notable sobre los sistemas anteriores.

La **inferencia del lenguaje natural** es la tarea de juzgar si una hipótesis (*los perros necesitan comer*) se deduce de una premisa (*todos los animales necesitan comer*). Esta tarea fue popularizada por el Desafío Pascal (Dagan *et al.,* 2005). Ahora están disponibles conjuntos de datos a gran escala (Bowman *et al.,* 2015; Williams *et al.,* 2018). Los sistemas basados en modelos preentrenados como ELMo y BERT proporcionan en la actualidad el mejor rendimiento en tareas de inferencia de lenguaje.

La Conferencia sobre Aprendizaje Computacional del Lenguaje Natural (CoNLL) se centra en el aprendizaje para NLP. Todas las conferencias y revistas mencionadas en el Capítulo 23 ahora incluyen trabajos sobre aprendizaje profundo, que hoy tiene una posición dominante en el campo de NLP.

CAPÍTULO 25

VISIÓN POR COMPUTADORA

En el que conectamos la computadora al mundo auténtico y sin filtros a través del lente de una cámara.

La mayoría de los animales tienen ojos, en la generalidad de los casos a un costo significativo: los ojos ocupan mucho espacio, utilizan energía y son muy frágiles. Este costo se justifica por el inmenso valor que proporcionan. Un agente que ve puede predecir el futuro, decir con qué podría chocar, decidir si atacar o huir o cortejar; adivinar si el terreno ante sí es pantanoso o firme, y decir a qué distancia está una fruta. En este capítulo, describimos cómo recuperar información de la avalancha de datos que proviene de los ojos o de las cámaras.

25.1 Introducción

La visión es un canal perceptivo que acepta un **estímulo** e informa alguna representación del mundo. La mayoría de los agentes que utilizan la visión usan **detección pasiva**, no necesitan enviar luz para ver. En contraste, la **detección activa** implica enviar una señal como radar o ultrasonido, y detectar una reflexión. Entre los agentes que usan detección activa encontramos murciélagos (ultrasonido), delfines (sonido), peces abisales (luz), y algunos robots (luz, sonido, radar). Para entender un canal perceptivo, uno debe estudiar los fenómenos físicos y estadísticos que ocurren en la detección, así como lo que el proceso perceptivo debería producir. Nos concentramos en la visión en este capítulo, pero los robots en el mundo real emplean una variedad de sensores para percibir sonido, tacto, distancia, temperatura, posición global y aceleración.

Característica

Una **característica** es un número obtenido aplicando cálculos simples a una imagen. Es posible obtener información completa muy útil directamente de las características. El agente wumpus tenía cinco sensores, cada uno de los cuales extraía un solo bit de información. Estos bits, que son características, podrían ser interpretados de manera directa por el programa. Otro ejemplo lo encontramos en muchos animales voladores que calculan una característica simple que da una buena estimación del tiempo de contacto con un objeto cercano; esta característica permite ser pasada de manera directa a los músculos que controlan la dirección o las alas, y permite cambios de dirección muy rápidos. Este **enfoque de extracción** de características enfatiza cálculos simples y directos aplicados a las respuestas de los sensores.

El enfoque **basado en modelos** para la visión utiliza dos tipos de modelos. Un **modelo de objeto** podría ser el tipo de modelo geométrico preciso producido por sistemas de diseño asistidos por computadora. También podría ser una declaración vaga sobre las propiedades generales de los objetos; por ejemplo, la afirmación de que todas las caras vistas en baja resolución se ven iguales. Un **modelo de renderizado** describe los procesos físicos, geométricos y estadísticos que producen el estímulo desde el mundo. Aunque los modelos de renderizado son ahora sofisticados y exactos, el estímulo suele ser ambiguo. Un objeto blanco bajo luz baja puede parecer un objeto negro bajo luz intensa. Un objeto pequeño y cercano puede parecer lo mismo que un

objeto grande y lejano. Sin evidencia adicional, no podemos decir si lo que vemos es a Godzilla de juguete destrozando un edificio de maqueta o un monstruo real destruyendo un edificio real.

Hay dos formas principales de manejar estas ambigüedades. Primero, algunas interpretaciones son más probables que otras. Por ejemplo, podemos estar seguros de que la imagen no muestra a un Godzilla real destruyendo un edificio real, porque no existen Godzillas reales. Segundo, algunas ambigüedades son insignificantes; por ejemplo, el paisaje lejano pueden ser árboles o ser una superficie plana pintada. Para la mayoría de las aplicaciones, la diferencia es irrelevante, porque los objetos están lejos y, por lo tanto, no chocaremos con ellos ni interactuaremos con ellos pronto.

Reconstrucción

Reconocimiento

Los dos problemas centrales de la visión por computadora son la **reconstrucción**, donde un agente construye un modelo del mundo a partir de una imagen o un conjunto de imágenes, y el **reconocimiento**, en el cual, con base en información visual y de otro tipo, un agente distingue entre los objetos que encuentra. Ambos problemas deben interpretarse de manera muy amplia. Construir un modelo geométrico a partir de imágenes es reconstrucción (y las soluciones son muy valiosas), pero a veces necesitamos construir un mapa de las diferentes texturas en una superficie, y esto también es reconstrucción. Asignar nombres a los objetos que aparecen en una imagen es reconocimiento. A veces necesitamos responder preguntas como: ¿Está dormido? ¿Come carne? ¿Qué extremo tiene dientes? Responder a estas preguntas también es reconocimiento.

Los últimos treinta años de investigación han producido herramientas y métodos poderosos para abordar estos problemas centrales. Comprender estos métodos requiere una comprensión de los procesos por los cuales se forman las imágenes.

25.2 Formación de la imagen

La imagen distorsiona la apariencia de los objetos. Una foto tomada mirando hacia abajo por un largo conjunto de vías de tren rectas sugerirá que los rieles convergen y se encuentran. Si sostiene su mano frente a su ojo, puede bloquear la luna, aunque la luna es más grande que su mano (esto también funciona con el Sol, pero podría dañar sus ojos al comprobarlo). Si sostiene un libro por el lado plano frente a su cara y lo inclina hacia atrás y hacia adelante, parecerá encogerse y crecer *en la imagen*. Este efecto se conoce como **acortamiento** (Figura 25.1). Los modelos de estos efectos son esenciales para construir sistemas competentes de reconocimiento de objetos y también proporcionan pistas poderosas para reconstruir la geometría.

25.2.1 Imágenes sin lentes: la cámara estenopeica

Escena

Imagen

Los sensores de imagen recogen la luz dispersada de los objetos en una **escena** y crean una **imagen** bidimensional (2D). En el ojo, estos sensores consisten en dos tipos de células, bastones y conos: hay alrededor de 100 millones de bastones sensibles a la luz en un amplio rango de longitudes de onda, y 5 millones de conos esenciales para la visión en color. Los conos son de tres tipos principales, cada uno de los cuales es sensible a un conjunto diferente de longitudes de onda. En las cámaras, la imagen se forma en un plano de imagen. En las cámaras de película, el plano de imagen está recubierto con haluros de plata. En las cámaras digitales, el plano de imagen se subdivide en una retícula de unos pocos millones de **pixeles**.

Pixeles

Sensor

Nos referimos a todo el plano de imagen como un **sensor**, pero cada pixel es un sensor individual minúsculo, en general un dispositivo de acoplamiento de carga (CCD) o un semiconductor de óxido metálico complementario (CMOS). Cada fotón que llega al sensor produce un efecto eléctrico, cuya intensidad depende de la longitud de onda del fotón. La salida del sensor es la suma de todos estos efectos en alguna ventana de tiempo, lo que significa que los sensores de imagen informan un promedio ponderado de la intensidad de la luz que llega al sensor. El

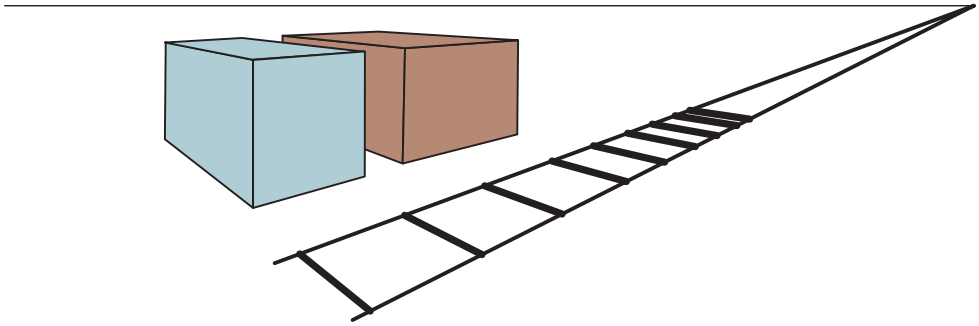


Figura 25.1 La geometría en la escena aparece distorsionada en las imágenes. Las líneas paralelas parecen encontrarse, como las vías del tren en un pueblo desolado. Los edificios que tienen ángulos rectos en la escena del mundo real tienen ángulos distorsionados en la imagen.

promedio es sobre la longitud de onda, la dirección desde la cual llegan los fotones, el tiempo y el área del sensor.

Para ver una imagen enfocada, debemos asegurarnos de que todos los fotones que llegan a un sensor provienen del mismo punto en el objeto en el mundo real. La forma más sencilla de formar una imagen enfocada es ver objetos estacionarios con una **cámara estenopeica**, que consiste en un orificio abertura, O, en el frente de una caja, y un plano de imagen en la parte trasera de la caja (Figura 25.2). La abertura se llama **apertura**. Si el orificio es lo suficientemente pequeño, cada sensor minúsculo en el plano de imagen sólo verá fotones que provienen del mismo punto en el objeto, por lo que la imagen está enfocada. También podemos formar imágenes enfocadas de objetos en movimiento con una cámara estenopeica, siempre que el objeto se mueva sólo una corta distancia en la ventana de tiempo de los sensores. De lo contrario, la imagen del objeto en movimiento está desenfocada, un efecto conocido como **desenfoque de movimiento**. Una forma de manipular la ventana de tiempo es abrir y cerrar el orificio de la cámara estenopeica.

Cámara estenopeica

Apertura

Desenfoque de movimiento

Las cámaras estenopeicas facilitan la comprensión del modelo geométrico del comportamiento de la cámara (que es más complicado, pero similar, con la mayoría de los otros dispositivos de imagen). Usaremos un sistema de coordenadas tridimensional (3D) con el origen en O, y consideraremos un punto P en la escena, con coordenadas (X,Y,Z). P se proyecta al punto P' en el plano de imagen con coordenadas (x,y,z). Si f es la **longitud focal**, la distancia desde el orificio hasta la imagen plano, entonces por triángulos similares, podemos derivar las siguientes ecuaciones:

Longitud focal

$$\frac{-x}{f} = \frac{X}{Z}, \frac{-y}{f} = \frac{Y}{Z} \quad \Rightarrow \quad x = \frac{-fX}{Z}, y = \frac{-fY}{Z}.$$

Estas ecuaciones definen un proceso de formación de imagen conocido como **proyección en perspectiva**. Note que la Z en el denominador significa que cuanto más lejos esté un objeto, más pequeña será su imagen. Además, tenga en cuenta que los signos negativos significan que la imagen está *invertida*, tanto de izquierda a derecha como de arriba a abajo, en comparación con la escena.

Proyección en perspectiva

La imagen en perspectiva tiene una serie de efectos geométricos. Los objetos distantes parecen pequeños. Las líneas paralelas convergen en un punto en el horizonte. (Piense en las vías del tren, Figura 25.1). Una línea en la escena en la dirección (U,V,W) y que pasa por el punto (X_0,Y_0,Z_0) se describe como el conjunto de puntos $(X_0 + \lambda U, Y_0 + \lambda V, Z_0 + \lambda W)$, con λ variando entre $-\infty$ y $+\infty$. Diferentes elecciones de (X_0,Y_0,Z_0) dan lugar a diferentes líneas paralelas entre sí.

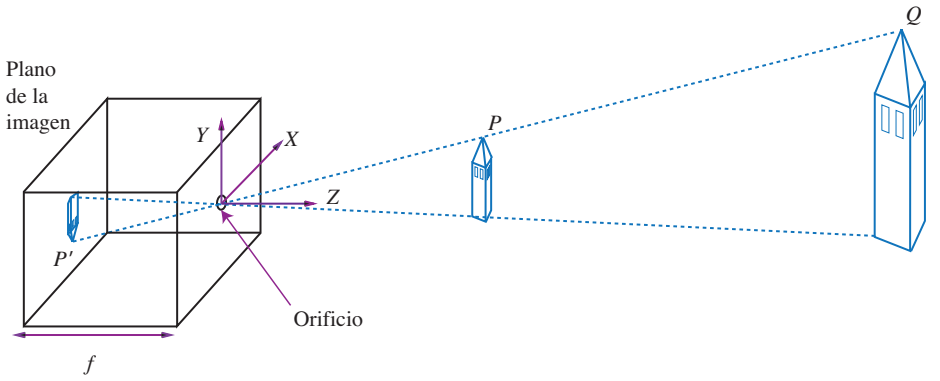


Figura 25.2 Cada elemento sensible a la luz en la parte posterior de una cámara estenopeica recibe luz que pasa a través del orificio desde un pequeño rango de direcciones. Si el orificio es lo suficientemente pequeño, el resultado es una imagen enfocada detrás del orificio. El proceso de proyección significa que los objetos grandes y distantes parecen lo mismo que los objetos más pequeños y cercanos; el punto P' en el plano de la imagen podría haber venido de una torre de juguete cercana en el punto P o de una torre real distante en el punto Q.

La proyección de un punto $P\lambda$ desde esta línea hasta el plano de la imagen se define como:

$$P\lambda = \left(f\frac{X_0 + \lambda u}{Z_0 + \lambda W}, f\frac{Y_0 + \lambda V}{Z_0 + \lambda W} \right).$$

A medida que $\lambda \to \infty$ o $\lambda \to -\infty$, se convierte en $P_\infty = (fU/W, fV/W)$ si $W \neq 0$. Esto significa que dos líneas paralelas que salen de diferentes puntos en el espacio convergerán en la imagen, para λ grande, los puntos de la imagen son casi los mismos, cualquiera que sea el valor de (X_0, Y_0, Z_0) (de nuevo, piense en las vías del tren, Figura 25.1). Llamamos P_∞ el **punto de fuga** asociado con la familia de líneas rectas con dirección (U,V,W). Las líneas con la misma dirección comparten el mismo punto de fuga.

Punto de fuga

25.2.2 Sistemas de lentes

Las cámaras estenopeicas permiten enfocar bien la luz, pero como el orificio es pequeño, sólo entrará un poco de luz, y la imagen será oscura. En un corto periodo de tiempo, sólo unos pocos fotones golpearán cada punto en el sensor, por lo que la señal en cada punto estará dominada por fluctuaciones aleatorias; decimos que una imagen de película oscura es granulada y una imagen digital oscura es ruidosa; de cualquier manera, la imagen es de baja calidad.

Agrandar el orificio (la apertura) hará que la imagen sea más brillante al recoger más luz de un rango más amplio de direcciones. Sin embargo, con una apertura más grande, la luz que golpea un punto particular en el plano de la imagen habrá venido de múltiples puntos en la escena del mundo real, por lo que la imagen estará desenfocada. Necesitamos alguna forma de reenfocar la imagen.

Lente

Los ojos de los vertebrados y las cámaras modernas usan un sistema de **lentes**, un solo pedazo de tejido transparente en el ojo y un sistema de varios elementos de lentes de vidrio en una cámara. En la Figura 25.3 vemos que la luz de la punta de la vela se esparce en todas direcciones. Una cámara (o un ojo) con una lente capta toda la luz que golpea en cualquier parte de la lente, una superficie mucho mayor que un agujero, y enfoca toda esa luz en un solo punto en el plano de la imagen. La luz de otras partes de la vela sería recogida y enfocada de manera similar en otros puntos en el plano de la imagen. El resultado es una imagen más brillante, menos ruidosa y enfocada.

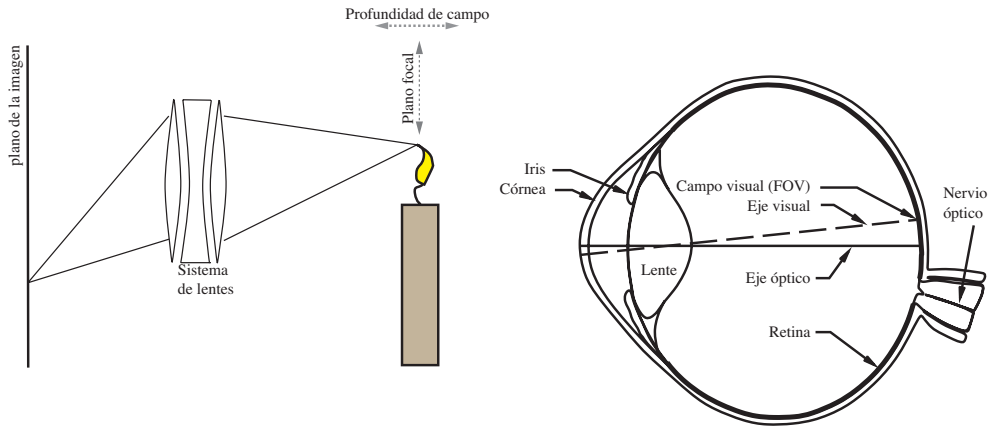


Figura 25.3 Las lentes recogen la luz que sale de un punto en la escena (aquí, la punta de la llama de la vela) en un rango de direcciones, y dirigen toda la luz para que llegue a un solo punto en el plano de la imagen. Los puntos en la escena cerca del plano focal, dentro de la profundidad de campo, estarán enfocados de manera correcta. En las cámaras, los elementos del sistema de lentes se mueven para cambiar el plano focal, mientras que en el ojo, la forma de la lente es cambiada por músculos especializados.

Los sistemas de lentes no enfocan toda la luz de todos los lugares del mundo real; el diseño de la lente los restringe a enfocar la luz sólo desde puntos que se encuentran dentro de un rango de profundidades Z desde el lente. El centro de este rango, donde el enfoque es más nítido, se llama **plano focal**, y el rango de profundidades para el cual el enfoque sigue siendo lo suficientemente nítido se llama **profundidad de campo**. Cuanto mayor es la apertura de la lente (abertura), menor es la profundidad de campo.

Plano focal

Profundidad de campo

¿Qué pasa si quiere enfocar algo a una distancia diferente? Para mover el plano focal, los elementos de la lente en una cámara permiten moverse hacia atrás y hacia adelante, y la lente en el ojo puede cambiar de forma, pero con la edad, la lente del ojo tiende a endurecerse, lo que la hace menos capaz de ajustar las distancias focales, y requiere que muchos humanos aumenten su visión con lentes externas, gafas.

25.2.3 Proyección ortográfica escalada

Los efectos geométricos de la imagen en perspectiva no siempre son pronunciados. Por ejemplo, las ventanas de un edificio al otro lado de la calle parecen mucho más pequeñas que las que están cerca, pero dos ventanas que están una al lado de la otra tendrán el mismo tamaño, aunque una esté un poco más lejos. Tenemos la opción de manejar las ventanas con un modelo simplificado llamado **proyección ortográfica escalada**, en lugar de proyección en perspectiva. Si la profundidad Z de todos los puntos en un objeto cae dentro del rango $Z_0 \pm \Delta Z$, con $\Delta Z \ll Z_0$, entonces el factor de escala de perspectiva f/Z se aproxima por una constante $s = f/Z_0$. Las ecuaciones para la proyección desde las coordenadas de la escena (X,Y,Z) al plano de la imagen se convierten en $x = sX$ y $y = sY$. El acortamiento aún ocurre en el modelo de proyección ortográfica escalada porque es causado por el objeto que se inclina lejos de la vista.

Proyección ortográfica escalada

25.2.4 Luz y sombreado

El brillo de un pixel en la imagen es una función del brillo del parche de superficie en la escena que se proyecta al pixel. Para las cámaras modernas esta función es lineal para intensidades medias de luz, pero tiene notables no linealidades para iluminaciones más oscuras y más brillantes. Usaremos un modelo lineal. El brillo de la imagen es una fuerte, aunque ambigua, pista de la forma y de la identidad de los objetos. La ambigüedad ocurre porque hay tres factores que contribuyen a la cantidad de luz que proviene de un punto en un objeto a la imagen: la intensidad general de la **luz ambiental**; si el punto está enfrentando la luz o está en sombra; y la cantidad de luz **reflejada** desde el punto.

Luz ambiental

Reflejada

Las personas son buenas para desambiguar el brillo; por lo regular pueden distinguir la diferencia entre un objeto negro en luz brillante y un objeto blanco en sombra, aun si ambos tienen el mismo brillo general. Sin embargo, las personas a veces confunden la sombra y las marcas, una raya de maquillaje oscuro debajo del pómulo con frecuencia parecerá un efecto de sombreado, haciendo que la cara parezca más delgada.

La mayoría de las superficies reflejan la luz mediante un proceso de reflexión difusa. La **reflexión difusa** dispersa la luz de manera uniforme en las direcciones que salen de una superficie, por lo que el brillo de una superficie difusa no depende de la dirección de visualización. La mayoría de los tejidos tienen esta propiedad, al igual que la mayoría de las pinturas, superficies de madera rugosas, la mayoría de la vegetación y la piedra o el concreto rugosos.

Reflexión difusa

La **reflexión especular** hace que la luz entrante salga de una superficie en un lóbulo de direcciones que está determinado por la dirección desde la que llegó la luz. Un espejo es un ejemplo. Lo que ve depende de la dirección en la que mire al espejo. En este caso, el lóbulo de direcciones es muy estrecho, por eso es posible ver diferentes objetos en un espejo.

Reflexión especular

Para múltiples superficies, el lóbulo es más amplio. Estas superficies muestran pequeñas manchas brillantes, con frecuencia llamadas **especularidades**. A medida que la superficie o la luz se mueven, las especularidades también se mueven. Lejos de estas manchas, la superficie se comporta como si fuera difusa. Las especularidades con frecuencia se ven en superficies metálicas, superficies pintadas, superficies de plástico y superficies húmedas. Estas son fáciles de identificar, porque son pequeñas y brillantes (Figura 25.4). Para casi todos los propósitos, es suficiente modelar todas las superficies como si fueran difusas con especularidades.

Especularidades

La principal fuente de iluminación en el exterior es el Sol, cuyos rayos viajan todos paralelos entre sí en una dirección conocida porque está muy lejos. Modelamos este comportamiento con una **fuente de luz puntual distante**. Este es el modelo de iluminación más importante y es muy efectivo para escenas interiores y exteriores. La cantidad de luz recogida por un parche de superficie en este modelo depende del ángulo θ entre la dirección de iluminación y la normal (perpendicular) a las superficies (Figura 25.5).

Fuente de luz puntual distante

Un parche de superficie difusa iluminado por este modelo reflejará una fracción de la luz que recoge, dada por el **albedo difuso**. Para superficies prácticas, esto se encuentra en el rango 0.05-0.95. La **ley del coseno de Lambert** establece que el brillo de un parche difuso se define como:

Albedo difuso

Ley del coseno de Lambert

$$I = \rho I_0 \cos\theta,$$

donde I_0 es la intensidad de la fuente de luz, θ es el ángulo entre la dirección de la fuente de luz y la normal de la superficie, y ρ es el albedo difuso. Esta ley predice que los pixeles de imagen brillantes provienen de parches de superficie que enfrentan la luz de manera directa y los pixeles oscuros provienen de parches que ven la luz sólo de manera tangencial, por lo que la sombra en una superficie proporciona cierta información de forma. Si la superficie no puede ver la fuente, entonces está en **sombra**. Las sombras rara vez son un negro uniforme, porque la superficie en

Sombra

Figura 25.4 Esta fotografía ilustra una variedad de efectos de iluminación. Hay especularidades en la olla de acero inoxidable. Las cebollas y las zanahorias son superficies difusas brillantes porque enfrentan la dirección de la luz. Las sombras aparecen en puntos de superficie que no permiten ver la fuente de luz en absoluto. Dentro de la olla hay algunas superficies difusas oscuras donde la luz golpea en un ángulo tangencial. (También hay algunas sombras dentro de la olla). Foto de Ryman Cabannes/Image Professionals GmbH/Alamy Stock Photo.

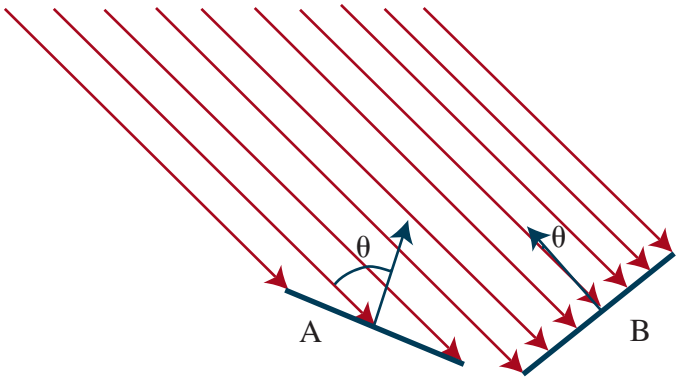


Figura 25.5 Dos parches de superficie son iluminados por una fuente puntual distante, cuyos rayos se muestran como flechas de luz. El parche A está inclinado lejos de la fuente (θ está cerca de 90°) y recoge menos energía, porque corta menos rayos de luz por unidad de área de superficie. El parche B, enfrentando la fuente (θ está cerca de 0°), recoge más energía.

sombra en la mayoría de los casos recibe algo de luz de otras fuentes. Al aire libre, la fuente más importante aparte del Sol es el cielo, que es bastante brillante. En interiores, la luz reflejada de otras superficies ilumina los parches en sombra. Estas **inter-reflexiones** también tienen un efecto significativo en el brillo de otras superficies. Estos efectos a veces se modelan agregando un término de **iluminación ambiental** constante a la intensidad predicha de la iluminación ambiental.

Inter-reflexiones

Iluminación ambiental

25.2.5 Color

La fruta es un soborno que un árbol ofrece a los animales para llevar sus semillas alrededor. Los árboles que pueden señalar cuando este soborno está listo tienen una ventaja, al igual que los animales que pueden leer estas señales. Como resultado, la mayoría de las frutas comienzan verdes y se vuelven rojas o amarillas cuando están maduras, del mismo modo la mayoría de animales que comen frutas ven estos cambios de color. En general, la luz que llega al ojo tiene diferentes cantidades de energía en diferentes longitudes de onda, y se representa con una densidad de energía espectral.

Las cámaras y el sistema de visión humana responden a la luz en longitudes de onda que van desde 380nm (violeta) hasta casi 750nm (rojo). En los sistemas de imágenes en color, hay diferentes tipos de receptores que responden más o menos fuerte a diferentes longitudes de onda. En los humanos, la sensación de color ocurre cuando el sistema de visión compara las respuestas de los receptores cercanos entre sí en la retina. Los sistemas de visión de color animal por lo regular suelen tener raros tipos de receptores, por lo que representan poco del detalle en la función de densidad de energía espectral (algunos animales sólo tienen un tipo de receptor; algunos tienen hasta seis tipos). La visión de color humana se produce con tres tipos de receptor. También, la mayoría de los sistemas de cámaras de color usan sólo tres tipos de receptores, porque las imágenes se producen para humanos, pero algunos sistemas especializados permiten producir mediciones muy detalladas de la densidad de energía espectral.

Principio de tricromía

Primarios

RGB

Debido a que la mayoría de los humanos tienen tres tipos de receptores sensibles al color, se aplica el **principio de tricromía**. Esta idea, propuesta por primera vez por Thomas Young en 1802, establece que un observador humano puede igualar la apariencia visual de cualquier densidad de energía espectral, por compleja que sea, mezclando cantidades apropiadas de sólo tres **primarios**. Los primarios son fuentes de luz de color, elegidas de manera que ninguna mezcla de dos coincidirá con la tercera. Una elección común es tener un primario rojo, uno verde y uno azul, abreviado como **RGB** (siglas en inglés para *red*, *green* y *blue*). Aunque un objeto de color dado permite tener muchas frecuencias de luz componentes, podemos igualar el color mezclando sólo los tres primarios, y la mayoría de las personas estarán de acuerdo en las proporciones de la mezcla. Eso significa que podemos representar imágenes de color con sólo tres números por pixel: los valores RGB.

Para la mayoría de las aplicaciones de visión por computadora, es preciso modelar una superficie como si tuviera tres albedos difusos diferentes (RGB) y modelar las fuentes de luz como si tuvieran tres intensidades (RGB). Luego aplicamos la ley del coseno de Lambert a cada uno para obtener valores de pixeles rojos, verdes y azules. Este modelo predice de manera correcta que la misma superficie producirá parches de imagen de diferentes colores bajo luces de diferentes colores. De hecho, los observadores humanos son muy buenos ignorando los efectos de las luces de diferentes colores y parecen estimar el color que la superficie tendría bajo luz blanca, un efecto conocido como **constancia del color**.

Constancia del color

25.3 Características de imagen simples

La luz se refleja en los objetos de la escena para formar una imagen compuesta por, digamos, doce millones de pixeles de tres bytes. Como con todos los sensores, habrá ruido en la imagen, y en cualquier caso hay muchos datos con los que lidiar. La forma de empezar a analizar estos datos es producir representaciones simplificadas que expongan lo que es importante, pero reduzcan el detalle. La mayoría de la práctica actual aprende estas representaciones a partir de datos. Pero hay cuatro propiedades de las imágenes y el video que son en particular generales: bordes, textura, flujo óptico y segmentación en regiones.

Un borde ocurre donde hay una gran diferencia en la intensidad de los pixeles en una parte de una imagen. La construcción de representaciones de bordes implica operaciones locales en una imagen: necesitas comparar un valor de pixel con algunos valores cercanos, y no requiere ningún

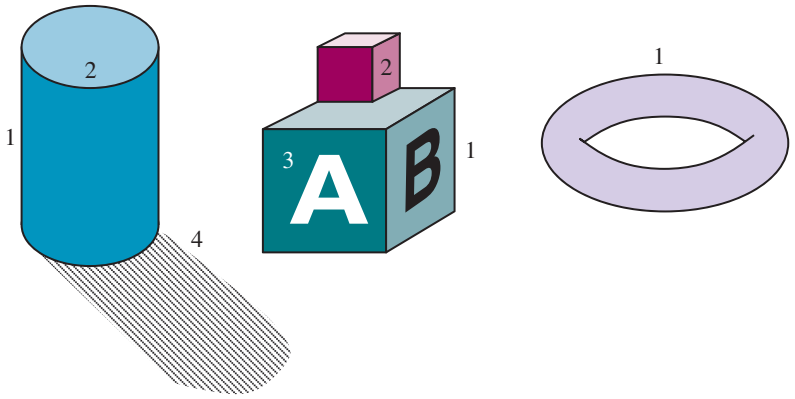


Figura 25.6 Diferentes tipos de bordes: (1) discontinuidades de profundidad; (2) discontinuidades de orientación de la superficie; (3) discontinuidades de reflectancia, y (4) discontinuidades de iluminación (sombras).

conocimiento sobre lo que es en la imagen. Por lo tanto, la detección de bordes viene al principio de la secuencia de operaciones y la llamamos una operación "temprana" o "de bajo nivel".

Las otras operaciones requieren manejar un área más grande de la imagen. Por ejemplo, una descripción de textura se aplica a un grupo de pixeles, para decir "a rayas", se necesita ver algunas rayas. El flujo óptico representa de dónde se mueven los pixeles de una imagen a la siguiente en una secuencia, y esto permite cubrir un área más grande. La segmentación divide una imagen en regiones de pixeles que pertenecen a un mismo conjunto, y hacerlo requiere mirar toda la región. Operaciones como esta, a veces se denominan operaciones "de nivel medio".

25.3.1 Bordes

Los **bordes** son líneas rectas o curvas en el plano de la imagen a través de las cuales hay un "significativo" cambio en el brillo de la imagen. El objetivo de la detección de bordes es abstraerse de la imagen desordenada y de varios megabytes hacia una representación más compacta y abstracta, como en la Figura 25.6. Los efectos en la escena resultan en grandes cambios en la intensidad de la imagen y, por lo tanto, producen bordes en la imagen. Las discontinuidades de profundidad (etiquetadas como 1 en la figura) pueden causar bordes porque cuando cruza la discontinuidad, el color cambia. Cuando la normal de la superficie cambia (etiquetada como 2 en la figura), la intensidad de la imagen cambia. Cuando cambia la reflectancia de la superficie (etiquetada como 3), la intensidad de la imagen cambia. Finalmente, una sombra (etiquetada como 4) es una discontinuidad en la iluminación que causa un borde en la imagen, aunque no haya un borde en el objeto. Los detectores de bordes no permiten desentrañar la causa de la discontinuidad, lo que se deja para un procesamiento posterior.

Bordes

Encontrar bordes requiere cuidado. La Figura 25.7 (arriba) muestra una sección transversal unidimensional de una imagen perpendicular a un borde, con un borde en $x = 50$.

Podría diferenciar la imagen y buscar lugares donde la magnitud de la derivada $I'(x)$ es grande. Esto casi funciona, pero en la Figura 25.7 (medio), vemos que aunque hay un pico en $x = 50$, también hay picos subsidiarios en otros lugares (por ejemplo, $x = 75$) que podrían confundirse con bordes verdaderos. Estos surgen debido a la presencia de "ruido" en la imagen. **Ruido** aquí significa cambios en el valor de un pixel que no tienen que ver con un borde. Por ejemplo, podría

Ruido

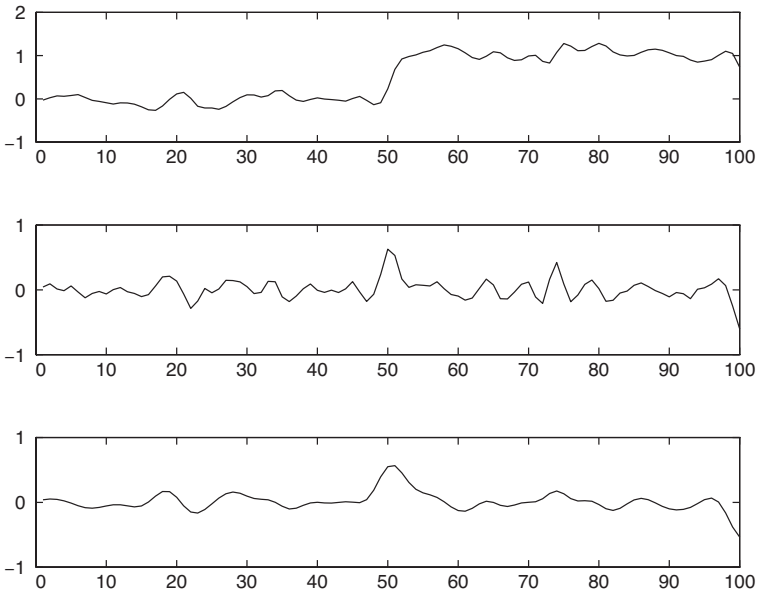


Figura 25.7 Arriba: Perfil de intensidad $I(x)$ a lo largo de una sección unidimensional a través de un borde de paso. Medio: La derivada de la intensidad, $I'(x)$. Los valores grandes de esta función corresponden a bordes, pero la función es ruidosa. Abajo: La derivada de una versión suavizada de la intensidad. El borde candidato ruidoso en $x = 75$ ha desaparecido.

haber ruido térmico en la cámara, podría haber arañazos en la superficie del objeto que cambian la normal de la superficie a la escala más fina, podría haber variaciones menores en el albedo de la superficie, etc. Cada uno de estos efectos hace que el gradiente parezca grande, pero no significa que haya un borde presente. Si "suavizamos" la imagen primero, los picos espurios se reducen, como vemos en la Figura 25.7 (abajo).

Filtro gaussiano

El suavizado implica usar pixeles circundantes para suprimir el ruido. Predeciremos el valor "verdadero" de nuestro pixel como una suma ponderada de pixeles cercanos, con más peso para los pixeles más cercanos. Una elección natural de pesos es un **filtro gaussiano**. Recuerde que la función gaussiana de media cero con desviación estándar σ es

$$G_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2} \qquad \text{en una dimensión, o}$$

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2} \qquad \text{en dos dimensiones.}$$

Aplicar un filtro gaussiano significa reemplazar la intensidad $I(x_0,y_0)$ con la suma, sobre todos los pixeles (x,y), de $I(x,y)G_\sigma(d)$, donde d es la distancia desde (x_0,y_0) hasta (x,y). Este tipo de suma ponderada es tan común que hay un nombre especial y notación para ello. Decimos que la función h es la **convolución** de dos funciones f y g (definido como $h = f * g$) si tenemos

Convolución

$$h(x) = \sum_{u=-\infty}^{+\infty} f(u)g(x-u) \qquad \text{en una dimensión, o}$$

$$h(x,y) = \sum_{u=-\infty}^{+\infty} \sum_{v=-\infty}^{+\infty} f(u,v)g(x-u,y-v) \qquad \text{en dos dimensiones.}$$

Por lo tanto, la función de suavizado se logra convolucionando la imagen con el gaussiano, $I * G_\sigma$. Un σ de 1 pixel es suficiente para suavizar una pequeña cantidad de ruido, mientras que 2 pixeles suavizarán una cantidad mayor, pero con la pérdida de algunos detalles. Debido a que la influencia del gaussiano se desvanece de manera rápida con la distancia, en la práctica podemos reemplazar el $\pm\infty$ en las sumas con algo como $\pm 3\sigma$.

Tenemos la oportunidad de hacer una optimización aquí: podemos combinar el suavizado y la detección de bordes en una sola operación. Es un teorema que para cualquier función f y g, la derivada de la convolución, $(f * g)'$, es igual a la convolución con la derivada, $f * (g')$. Así que en lugar de suavizar la imagen y luego diferenciar, podemos convolucionar la imagen con la derivada de la función de suavizado gaussiana, G'_σ. Luego marcamos como bordes aquellos picos en la respuesta que están por encima de algún umbral, elegido para eliminar picos espurios debido al ruido.

Existe una generalización natural de este algoritmo desde secciones transversales unidimensionales a imágenes 2D generales. En dos dimensiones los bordes pueden estar en cualquier ángulo θ. Considerando el brillo de la imagen como una función escalar de las variables x, y, su gradiente es un vector

$$\nabla I = \begin{pmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{pmatrix}$$

Los bordes corresponden a ubicaciones en las imágenes donde el brillo sufre un cambio brusco y, por lo tanto, la magnitud del gradiente, $||\nabla I||$ debería ser grande en un punto de borde. Cuando la imagen se vuelve más brillante o más oscura, el vector gradiente en cada punto se alarga o se acorta, pero la dirección del gradiente

$$\frac{\nabla I}{||\nabla I||} = \begin{pmatrix} cos\theta \\ sin\theta \end{pmatrix}$$

no cambia. Esto nos da un $\theta = \theta(x,y)$ en cada pixel, que define la **orientación** del borde en ese pixel. Esta característica es útil, porque no depende de la intensidad de la imagen. Orientación

Como usted podría esperar de la discusión sobre la detección de bordes en señales unidimensionales, para formar el gradiente, en realidad no calculamos ∇I, sino $\nabla(I * G_\sigma)$, después de suavizar la imagen al convolucionarla con un gaussiano. Una propiedad de las convoluciones es que esto es equivalente a convolucionar la imagen con las derivadas parciales del gaussiano. Una vez que hemos calculado el gradiente, podemos obtener bordes encontrando puntos de borde y uniéndolos. Para decir si determinar un punto es un punto de borde, debemos mirar otros puntos a una pequeña distancia hacia adelante y hacia atrás a lo largo de la dirección del gradiente. Si la magnitud del gradiente en uno de estos puntos es mayor, entonces podríamos obtener un mejor punto de borde desplazando la curva de borde. Además, si la magnitud del gradiente es muy pequeña, el punto no puede ser un punto de borde. Por lo tanto, en un punto de borde, la magnitud del gradiente es un máximo local a lo largo de la dirección del gradiente, y la magnitud del gradiente está por encima de un umbral adecuado.

Una vez que hemos marcado los pixeles de borde con este algoritmo, la siguiente etapa es vincular aquellos pixeles que pertenecen a las mismas curvas de borde. Esto se hace asumiendo que cualesquiera de los dos pixeles vecinos, que son ambos pixeles de borde con orientaciones consistentes, pertenecen a la misma curva de borde.

La detección de bordes no es perfecta. La Figura 25.8(a) muestra una imagen de una escena que contiene una engrapadora sobre un escritorio, y la Figura 25.8(b) muestra la salida de un algoritmo de detección de bordes en esta imagen. Note que la salida no es perfecta: hay huecos donde no aparece ningún borde, y hay bordes de "ruido" que no corresponden a nada significativo en la escena. Las etapas posteriores de procesamiento tendrán que corregir estos errores.

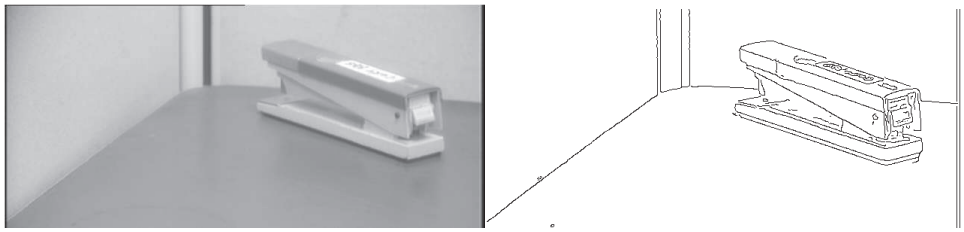


Figura 25.8 (a) Fotografía de una engrapadora. (b) Bordes calculados a partir de (a).

25.3.2 Textura

Textura

En el lenguaje cotidiano, la **textura** de las superficies insinúa cómo se sienten cuando se pasa un dedo sobre ellas (las palabras "textura", "textil" y "texto" tienen la misma raíz latina, una palabra para tejer). En la visión computacional, la textura se refiere a un patrón en una superficie que es percibido de manera visual. Por lo general, estos patrones son más o menos regulares. Ejemplos incluyen el patrón de ventanas en un edificio, los puntos en un suéter, las manchas en la piel de un leopardo, las hojas de césped en un césped, las piedras en una playa y una multitud de personas en un estadio.

A veces, la disposición es muy periódica, como en los puntos de un suéter; en otros casos, como las piedras en una playa, la regularidad sólo es en un sentido estadístico: la densidad de las piedras es la misma en diferentes partes de la playa. Un modelo habitual de textura es un patrón repetitivo de elementos, a veces llamados **texels**. Este modelo es muy útil porque es difícil hacer o encontrar texturas reales que nunca se repitan.

Texels

La textura es una propiedad de un parche de imagen, en lugar de un pixel aislado. Una buena descripción de la textura de un parche debería resumir cómo se ve el parche. La descripción no debería cambiar cuando cambia la iluminación. Esto descarta el uso de puntos de borde; si una textura está bien iluminada, muchas ubicaciones dentro del parche tendrán un alto contraste y generarán puntos de borde; pero si la misma textura se ve con menos luz brillante, muchos de estos bordes no estarán por encima del umbral. La descripción debería cambiar de una manera sensata cuando el parche gira. Es importante preservar la diferencia entre las rayas verticales y las rayas horizontales, *pero* no si las rayas verticales se giran a la horizontal.

Se ha demostrado que las representaciones de textura con estas propiedades son útiles para dos tareas clave. La primera es identificar objetos: una cebra y un caballo tienen una forma similar, pero texturas diferentes. La segunda es emparejar parches en una imagen con parches en otra imagen, un paso clave para recuperar información 3D de múltiples imágenes (Sección 25.6.1).

Aquí hay una construcción básica para una representación de textura. Dado un parche de imagen, calcula la orientación del gradiente en cada pixel del parche, y luego caracteriza el parche con un histograma de orientaciones. Las orientaciones del gradiente son en gran medida invariantes a los cambios en la iluminación (el gradiente se hará más largo, pero no cambiará de dirección). El histograma de orientaciones parece capturar aspectos importantes de la textura. Por ejemplo, las rayas verticales tendrán dos picos en el histograma (uno para el lado izquierdo de cada raya y uno para el derecho); las manchas de leopardo tendrán orientaciones distribuidas de manera uniforme.

Pero no sabemos qué tan grande debe ser un parche para describirlo. Hay dos estrategias. En aplicaciones especializadas, la información de la imagen revela qué tan grande debería ser el parche (por ejemplo, uno podría crecer un parche lleno de rayas hasta que cubra la cebra). Una alternativa

es describir un parche centrado en cada pixel para un rango de escalas. Este rango con frecuencia va desde unos pocos pixeles hasta la extensión de la imagen. Ahora divide el parche en bins, y en cada bin construye un histograma de orientación, luego resume el patrón de histogramas a través de los bins. Ya no es habitual construir estas descripciones a mano. En su lugar, se utilizan redes neuronales convolucionales para producir representaciones de textura. Pero las representaciones construidas por las redes parecen reflejar esta construcción de manera aproximada.

25.3.3 Flujo óptico

A continuación, consideremos qué sucede cuando tenemos una secuencia de video, en lugar de sólo una imagen estática. Siempre que hay un movimiento relativo entre la cámara y uno o más objetos en la escena, el movimiento aparente resultante en la imagen se llama **flujo óptico**. Este describe la dirección y velocidad del movimiento de las características *en la imagen* como resultado del movimiento relativo entre el espectador y la escena. Por ejemplo, los objetos distantes vistos desde un coche en movimiento tienen un aparente movimiento mucho más lento que los objetos cercanos, por lo que la tasa de movimiento aparente puede decirnos algo sobre la distancia.

Flujo óptico

En la Figura 25.9 mostramos dos cuadros de un video de una jugadora de tenis. A la derecha mostramos los vectores de flujo óptico calculados a partir de estas imágenes. El flujo óptico codifica información útil sobre la estructura de la escena: la jugadora de tenis se está moviendo y el fondo (en su mayoría) no se mueve. Además, los vectores de flujo revelan algo sobre lo que está haciendo la jugadora: un brazo y una pierna se están moviendo rápido, pero las otras partes del cuerpo no.

El campo vectorial de flujo óptico es representado por sus componentes $v_x(x,y)$ en la dirección x y $v_y(x,y)$ en la dirección y. Para medir el flujo óptico, necesitamos encontrar puntos correspondientes entre un marco de tiempo y el siguiente. Una técnica muy simple se basa en el hecho de que los parches de imagen alrededor de los puntos correspondientes tienen patrones de intensidad similares. Considere un bloque de pixeles centrado en el pixel $p,(x_0,y_0)$, en el tiempo t. Este bloque de pixeles debe compararse con bloques de pixeles centrados en varios pixeles candidatos q_i en $(x_0 + D_x, y_0 + D_y)$ en el tiempo $t + D_t$. Una posible medida de similitud es la **suma de diferencias cuadradas (SSD)**:

Suma de diferencias cuadradas (SSD)

$$\text{SSD}(D_x,D_y) = \sum_{(x,y)} (I(x,y,t) - I(x + D_x,y + D_y,t + D_t))^2.$$

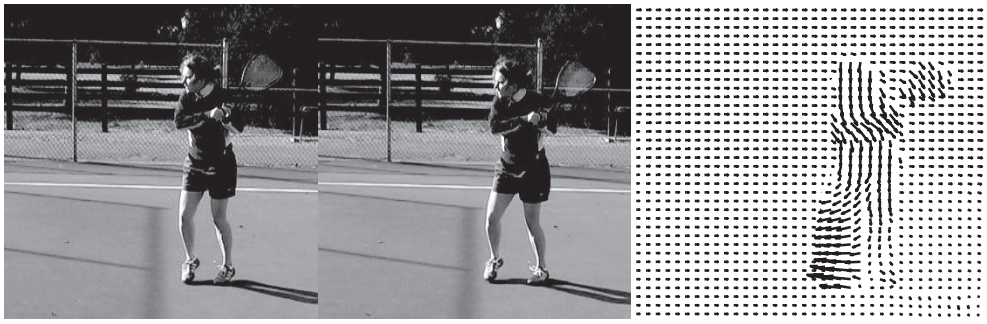


Figura 25.9 Dos cuadros de una secuencia de video y el campo de flujo óptico correspondiente al desplazamiento de un cuadro a otro. Observe cómo el movimiento de la raqueta de tenis y la pierna delantera es capturado por las direcciones de las flechas. (Imágenes cortesía de Thomas Brox).

Aquí, (x, y) varía sobre los pixeles en el bloque centrado en (x_0,y_0). Encontramos el (D_x,D_y) que minimiza el SSD. El flujo óptico en (x_0,y_0) es entonces $(v_x,v_y) = (D_x/D_t,D_y/D_t)$. Note que para que esto funcione debería haber alguna textura en la escena, resultando en ventanas que contienen una variación significativa en el brillo entre los pixeles. Si uno está mirando una pared blanca uniforme, entonces el SSD va a ser casi el mismo para las diferentes coincidencias candidatas q, y el algoritmo se reduce a hacer una suposición a ciegas. Los algoritmos de mejor rendimiento para medir el flujo óptico se basan en una variedad de restricciones adicionales para lidiar con situaciones en las que la escena sólo está parcialmente texturizada.

25.3.4 Segmentación de imágenes naturales

Segmentación

La **segmentación** es el proceso de dividir una imagen en grupos de pixeles similares. La idea básica es que cada pixel de la imagen esté asociado con ciertas propiedades visuales, como brillo, color y textura. Dentro de un objeto, o una sola parte de un objeto, estos atributos varían poco; mientras que a través de un límite interobjeto en general hay un gran cambio en uno o más de estos atributos. Necesitamos encontrar una partición de la imagen en conjuntos de pixeles de manera que estas restricciones se cumplan lo mejor posible. Observe que no es suficiente sólo encontrar bordes, porque muchos bordes no son límites de objetos. Entonces, por ejemplo, un tigre en la hierba puede generar un borde en cada lado de cada raya y cada hoja de hierba. En todos los confusos datos de bordes, podemos perder el tigre por las rayas.

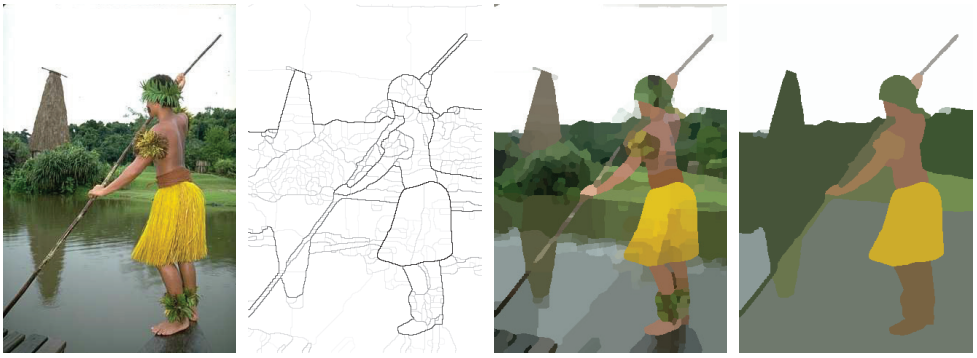
Regiones

Hay dos formas de estudiar el problema, una centrada en detectar los límites de estos grupos, y otra en detectar los propios grupos, llamados **regiones**. Ilustramos esto en la Figura 25.10, mostrando la detección de límites en (b) y la extracción de regiones en (c) y (d).

Una forma de formalizar el problema de detectar curvas de límite es como un problema de clasificación, susceptible a las técnicas de aprendizaje automático. Una curva de límite en la ubicación de pixeles (x,y) tendrá una orientación θ. Un vecindario de imagen centrado en (x,y) se parece más o menos a un disco, cortado en dos mitades por un diámetro orientado en θ. Podemos calcular la probabilidad $P_b(x,y,\theta)$ de que haya una curva de límite en ese pixel a lo largo de esa orientación comparando las características en las dos mitades. La forma natural de predecir esta probabilidad es entrenar un clasificador de aprendizaje automático usando un conjunto de datos de imágenes naturales en las que los humanos han marcado los límites de la verdad de base: el objetivo del clasificador es marcar esos límites marcados por los humanos y ninguno más.

Los límites detectados por esta técnica son mejores que los encontrados utilizando la simple técnica de detección de bordes antes descrita. Pero todavía hay dos limitaciones: (1) los pixeles de límite formados por el umbral $P_b(x,y,\theta)$ no están garantizados para formar curvas cerradas, por lo que este enfoque no entrega regiones, y (2) la toma de decisiones explota sólo el contexto local, y no usa restricciones de consistencia global.

El enfoque alternativo se basa en intentar "agrupar" los pixeles en regiones basadas en sus propiedades de brillo, color y textura. Hay varias formas en las que esta intuición permite formalizarse de manera matemática. Por ejemplo, Shi y Malik (2000) plantean esto como un problema de partición de grafos. Los nodos del grafo corresponden a pixeles, y los bordes a conexiones entre pixeles. El peso W_{ij} en el borde que conecta un par de pixeles i y j se basa en cuán similares son los dos pixeles en brillo, color, textura, etc. Luego encuentran particiones que minimizan un criterio de *corte normalizado*. Hablando en términos generales, el criterio para dividir el grafo es minimizar la suma de pesos de las conexiones a través de los grupos y maximizar la suma de pesos de las conexiones dentro de los grupos.



(a) (b) (c) (d)

Figura 25.10 (a) Imagen original. (b) Contornos de límite, donde cuanto mayor es el valor de P_b, más oscuro es el contorno. (c) Segmentación en regiones, correspondiente a una partición fina de la imagen. Las regiones se representan en sus colores medios. (d) Segmentación en regiones, correspondiente a una partición más gruesa de la imagen, lo que resulta en menos regiones. (Imágenes cortesía de Pablo Arbelaez, Michael Maire, Charless Fowlkes y Jitendra Malik).

Resulta que los enfoques basados en encontrar límites y en encontrar regiones pueden estar acoplados, pero no exploraremos estas posibilidades aquí. No es posible esperar que la segmentación basada en atributos locales de bajo nivel, como brillo y color, entregue los límites finales correctos de todos los objetos en la escena. Para encontrar de manera confiable los límites asociados con los objetos, también es necesario incorporar conocimientos de alto nivel sobre los tipos de objetos que se espera encontrar en una escena. En este momento, una estrategia popular es producir una sobresegmentación de una imagen, donde se garantiza que no se ha perdido la marcación de ninguno de los verdaderos límites, pero es posible marcar múltiples límites falsos. Las regiones resultantes, llamadas superpixeles, proporcionan una reducción significativa en la complejidad computacional para varios algoritmos, ya que el número de superpixeles puede estar en los cientos, en comparación con millones de pixeles brutos. La explotación de conocimientos de alto nivel de los objetos es el tema de la siguiente sección, y la detección real de los objetos en las imágenes es el tema de la Sección 25.5.

25.4 Clasificando imágenes

La clasificación de imágenes se aplica a dos casos principales. En uno, las imágenes son de *objetos*, tomadas de una taxonomía dada de clases, y no hay mucho más de importancia en la imagen; por ejemplo, un catálogo de imágenes de ropa o muebles, donde el fondo no importa, y la salida del clasificador es "sudadera de cachemira" o "silla de escritorio".

En el otro caso, cada imagen muestra una *escena* que contiene varios objetos. Entonces, en la sabana podría ver una jirafa y un león, y en la sala podría ver un sofá y una lámpara, pero no esperaría una jirafa o un submarino en su sala de estar. Ahora tenemos métodos para la clasificación de imágenes a gran escala que permiten emitir con precisión "sabana" o "sala de estar".

Apariencia

Los sistemas modernos clasifican las imágenes utilizando la **apariencia** (es decir, color y textura, en lugar de la geometría). Hay dos dificultades. Primero, diferentes instancias de la misma clase podrían verse diferentes, algunos gatos son negros y otros son naranjas. Segundo, el mismo

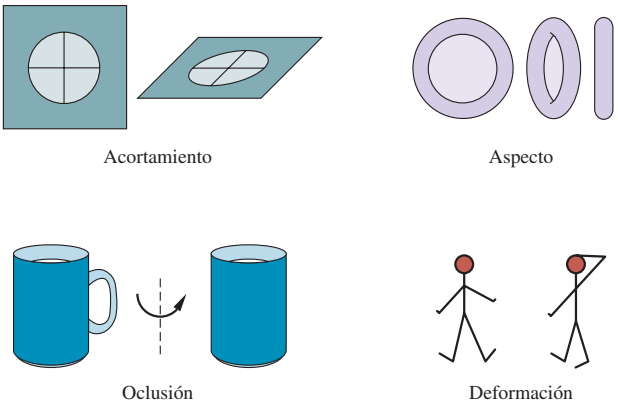


Figura 25.11 Fuentes importantes de variación de apariencia que permiten hacer que diferentes imágenes del mismo objeto se vean diferentes. Primero, los elementos permiten acortarse, como el parche circular en la parte superior izquierda. Este parche se ve desde un ángulo oblicuo, por lo que es elíptico en la imagen. Segundo, los objetos vistos desde diferentes direcciones pueden cambiar de forma de manera radical, un fenómeno conocido como aspecto. En la parte superior derecha hay tres aspectos diferentes de una rosquilla. La oclusión hace que el asa de la taza en la parte inferior izquierda desaparezca cuando la taza se gira. En este caso, porque el cuerpo y el asa pertenecen a la misma taza, tenemos autooclusión. Finalmente, en la parte inferior derecha, algunos objetos pueden deformarse de manera importante.

gato podría parecer diferente en diferentes momentos dependiendo de varios efectos (como se ilustra en la Figura 25.11):

- **Iluminación**, cambia el brillo y el color de la imagen.
- **Acortamiento**, causa que un patrón visto desde un ángulo oblicuo se distorsione.
- **Aspecto**, hace que los objetos se vean diferentes cuando se ven desde diferentes direcciones. Una rosquilla vista desde el lado parece un óvalo aplanado, pero desde arriba es un anillo.
- **Oclusión**, algunas partes del objeto están ocultas. Los objetos pueden ocluirse entre sí, o partes de un objeto pueden ocluir otras partes, un efecto conocido como **autooclusión**.
- **Deformación**, el objeto cambia su forma. Por ejemplo, la jugadora de tenis mueve sus brazos y piernas.

Los métodos modernos tratan estos problemas aprendiendo representaciones y clasificadores de grandes cantidades de datos de entrenamiento utilizando una red neuronal convolucional. Con un conjunto de entrenamiento suficientemente rico, el clasificador habrá visto cualquier efecto de importancia muchas veces en el entrenamiento, por lo que permite ajustarse al efecto.

25.4.1 Clasificación de imágenes con redes neuronales convolucionales

Las **redes neuronales convolucionales (CNN)** son clasificadores de imágenes espectacularmente exitosos. Con suficientes datos de entrenamiento y suficiente ingenio de entrenamiento, las CNN producen sistemas de clasificación muy exitosos, mucho mejores que los que se han podido producir con otros métodos.

El conjunto de datos de ImageNet tuvo un papel histórico en el desarrollo de sistemas de clasificación de imágenes al proporcionarles más de 14 millones de imágenes de entrenamiento, clasificadas

en más de 30 000 categorías de grano fino. ImageNet también impulsó el progreso con una competencia anual. Los sistemas se evalúan tanto por la precisión de clasificación de su mejor suposición única como por la precisión del top-5, en la que se permite a los sistemas presentar cinco suposiciones, por ejemplo, *malamute*, *husky*, *akita*, *samoyedo*, *perro esquimal*. ImageNet tiene 189 subcategorías de *perro*, por lo que incluso a los humanos amantes de los perros les resulta difícil etiquetar las imágenes de manera correcta con una sola suposición.

En la primera competencia de ImageNet en 2010, los sistemas no pudieron hacerlo mejor que 70 % de precisión en el top-5. La introducción de redes neuronales convolucionales en 2012, y su posterior refinamiento, llevaron a una precisión de 98 % en el top-5 (superando el rendimiento humano) y de 87 % en la precisión del top-1 para 2019. La razón principal de este éxito parece ser que las características que están siendo utilizadas por los clasificadores CNN se aprenden de los datos, no son creadas a mano por un investigador; esto asegura que las características son útiles para la clasificación.

El progreso en la clasificación de imágenes ha sido rápido debido a la disponibilidad de grandes y desafiantes conjuntos de datos como ImageNet; debido a competencias basadas en estos conjuntos de datos que son justas y abiertas; y debido a la difusión generalizada de modelos exitosos. Los ganadores de las competencias publican el código y con frecuencia los parámetros preentrenados de sus modelos, lo que facilita que otros jueguen con arquitecturas exitosas e intenten mejorarlas.

25.4.2 Por qué las redes neuronales convolucionales clasifican bien las imágenes

La clasificación de imágenes se entiende mejor mirando los conjuntos de datos, pero ImageNet es demasiado grande para mirarlo al detalle. El conjunto de datos MNIST es una colección de 70 000 imágenes de dígitos escritos a mano, 0-9, que en la mayoría de los casos se usa como un conjunto de datos de calentamiento estándar. Mirar este conjunto de datos (algunos ejemplos aparecen en la Figura 25.12) expone algunas propiedades importantes, muy generales. Es posible tomar una imagen de un dígito y hacer una serie de pequeñas alteraciones sin cambiar la identidad del dígito: desplazarlo, rotarlo, hacerlo más brillante o más oscuro, más pequeño o más grande. Esto significa que los valores individuales de los pixeles no son en particular informativos, sabemos que un 8 debería tener algunos pixeles oscuros en el centro pero un 0 no, aunque esos pixeles oscuros estarán en ubicaciones de pixeles ligeramente diferentes en cada instancia de un 8.

Otra propiedad importante de las imágenes es que los patrones locales son informativos: Los dígitos 0, 6, 8 y 9 tienen bucles; los dígitos 4 y 8 tienen cruces; aunque los dígitos 1, 2, 3, 5 y 7 tienen finales de línea, pero no bucles ni cruces; los dígitos 6 y 9 tienen bucles y finales de línea. Además, las relaciones espaciales entre los patrones locales son informativas. Un 1 tiene dos finales de línea, uno encima del otro; un 6 tiene un final de línea encima de un bucle. Estas observaciones sugieren una estrategia que es un principio central de la visión por computadora moderna: construye características que responden a patrones en pequeños vecindarios localizados; luego otras características miran los patrones de *esas* características; luego otras miran los patrones de esas, y así sucesivamente.

Esto es lo que las redes neuronales convolucionales hacen bien. Debería pensar en una capa, una convolución seguida de una función de activación ReLU, como un detector de patrones local (Figura 25.12). La convolución mide cuánto se parece cada ventana local de la imagen al patrón del núcleo; la ReLU establece las ventanas de puntuación baja a cero, y enfatiza las ventanas de puntuación alta. Así que la convolución con múltiples núcleos encuentra múltiples patrones; además, los patrones compuestos permiten ser detectados aplicando otra capa a la salida de la primera capa.

Piense en la salida de la primera capa convolucional. Cada ubicación recibe entradas de pixeles en una ventana alrededor de esa ubicación. Como hemos visto, la salida de la ReLU forma un detector de patrones simple. Ahora, si ponemos una segunda capa encima de esta, cada ubicación en la segunda capa recibe entradas de los valores de la primera capa en una ventana alrededor de esa

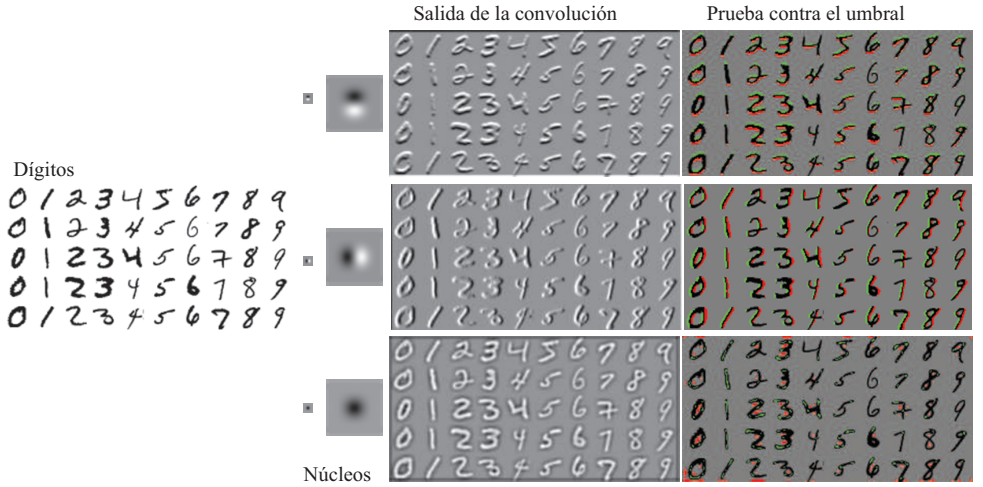


Figura 25.12 A la extrema izquierda, algunas imágenes del conjunto de datos MNIST. Tres núcleos aparecen en el centro izquierdo. Se muestran a tamaño real (bloques diminutos) y ampliados para revelar su contenido: el gris medio es cero, la luz es positiva, y la oscuridad es negativa. El centro derecho muestra los resultados de aplicar estos núcleos a las imágenes. La derecha muestra los pixeles donde la respuesta es mayor que un umbral (verde) o menor que un umbral (rojo). Note que esto da (de arriba a abajo): un detector de barras horizontales, un detector de barras verticales y (más difícil de notar) un detector de finales de línea. Estos detectores prestan atención al contraste de la barra, así que (por ejemplo) una barra horizontal que es clara en la parte superior y oscura en la parte inferior produce una respuesta positiva (verde), y una que es oscura en la parte superior y clara en la parte inferior obtiene una respuesta negativa (roja). Estos detectores son efectivos, pero no perfectos.

ubicación. Esto significa que las ubicaciones en la segunda capa se ven afectadas por una ventana de pixeles más grande que las de la primera capa. Debería pensar en estos como representando "patrones de patrones". Si colocamos una tercera capa encima de la segunda capa, las ubicaciones en esa tercera capa dependerán de una ventana de pixeles aún más grande; una cuarta capa dependerá de una ventana aún más grande, y así sucesivamente. La red está creando patrones en múltiples niveles, y lo está haciendo *aprendiendo* de los datos en lugar de tener los patrones dados por un programador.

Aumento del conjunto de datos

Mientras que entrenar una CNN "out of the box" a veces funciona, ayuda conocer algunas técnicas prácticas. Una de las más importantes es el **aumento del conjunto de datos**, en el que se copian y modifican ligeramente los ejemplos de entrenamiento. Por ejemplo, se podría desplazar, rotar o estirar una imagen de manera aleatoria en una pequeña cantidad, o desplazar el tono de los pixeles en una pequeña cantidad. Introducir esta variación simulada en el punto de vista o la iluminación en el conjunto de datos ayuda a aumentar el tamaño del conjunto de datos, aunque por supuesto los nuevos ejemplos están altamente correlacionados con los originales. También es posible usar el aumento en el momento de la prueba en lugar del momento del entrenamiento. En este enfoque, la imagen se replica y modifica varias veces (por ejemplo, con recortes aleatorios), mientras que el clasificador se ejecuta en cada una de las imágenes modificadas. Las salidas del clasificador de cada copia se utilizan entonces para votar una decisión final sobre la clase general.

Cuando está clasificando imágenes de escenas, cada pixel podría ser útil. Pero cuando clasifica imágenes de objetos, algunos pixeles no son parte del objeto y, por lo tanto, podrían ser una

distracción. Por ejemplo, si un gato está acostado en una cama de perro, queremos que un clasificador se concentre en los pixeles del gato, no en la cama. Los clasificadores de imágenes modernos manejan esto bien, clasificando una imagen como "gato" con precisión, incluso si pocos pixeles se encuentran en el gato. Hay dos razones para esto. Primero, los clasificadores basados en CNN son buenos para ignorar patrones que no son discriminativos. Segundo, los patrones que se encuentran fuera del objeto podrían ser discriminativos (por ejemplo, un juguete de gato, un collar con un pequeño cascabel, o un plato de comida de gato podrían ayudar a decir que estamos mirando un gato). Este efecto se conoce como **contexto**. El contexto ayuda o perjudica, dependiendo del conjunto de datos y aplicación particulares.

Contexto

25.5 Detectando objetos

Los clasificadores de imágenes predicen *qué* hay en la imagen; clasifican toda la imagen como perteneciente a una clase. Los detectores de objetos encuentran múltiples objetos en una imagen, informan a *qué* clase pertenece cada objeto, y también informan *dónde* está cada objeto al dar una **caja delimitadora** alrededor del objeto.[1] El conjunto de las clases están fijas de manera predeterminada. Así que podríamos intentar detectar todas las caras, todos los coches, o todos los gatos.

Caja delimitadora

Es posible construir un detector de objetos mirando una pequeña **ventana deslizante** en la más grande imagen, un rectángulo. En cada lugar, clasificamos lo que vemos en la ventana, usando un clasificador CNN. Luego tomamos las clasificaciones de alta puntuación —un gato aquí, un perro allá— e ignoramos las otras ventanas. Después de un poco de trabajo resolviendo conflictos, tenemos un conjunto final de objetos con sus ubicaciones. Todavía hay algunos detalles por resolver:

Ventana deslizante

- **Decidir sobre una forma de ventana:** la elección más fácil, por mucho, es usar rectángulos alineados con los ejes. (La alternativa, alguna forma de máscara que recorte el objeto de la imagen, casi nunca se usa, porque es difícil de representar y de calcular). Todavía necesitamos elegir el ancho y la altura de los rectángulos.
- **Construir un clasificador para ventanas:** ya sabemos cómo hacer esto con una CNN.
- **Decidir qué ventanas observar:** de todas las ventanas posibles, queremos seleccionar las que quizá tengan objetos interesantes en ellas.
- **Elegir qué ventanas informar:** las ventanas se superpondrán, y no queremos informar el mismo objeto varias veces en ventanas apenas diferentes. Algunos objetos no valen la pena mencionar; piense en la cantidad de sillas y personas en una foto de una gran aula de conferencias llena. ¿Deberían todos ser reportados como objetos individuales? Quizá sólo los objetos que aparecen grandes en la imagen, la primera fila, deberían ser reportados. La elección depende del uso previsto del detector de objetos.
- **Informar ubicaciones precisas de objetos usando estas ventanas:** una vez que sabemos que el objeto está en algún lugar de la ventana, podemos permitirnos hacer más cálculos para averiguar una ubicación más precisa dentro de la ventana.

Veamos más a detalle el problema de decidir qué ventanas observar. Buscar todas las ventanas posibles no es eficiente; en una imagen de $n \times n$ pixeles hay $O(n^4)$ ventanas rectangulares posibles. Pero sabemos que las ventanas que contienen objetos tienden a tener un color y una textura bastante coherentes. Por otro lado, las ventanas que cortan un objeto por la mitad tienen regiones

[1] Usaremos el término "caja" para referirnos a cualquier región rectangular alineada con los ejes de la imagen, y el término "ventana" como sinónimo de "caja", pero con la connotación de que tenemos una ventana en la entrada donde esperamos ver algo, y una caja delimitadora en la salida cuando lo hemos encontrado.

o bordes que cruzan el lado de la ventana. En consecuencia, tiene sentido tener un mecanismo que puntúe "objetividad"; si una caja tiene un objeto en ella, independiente de lo que sea ese objeto. Podemos encontrar las cajas que parecen tener un objeto en ellas, y luego clasificar el objeto sólo para esas cajas que pasan la prueba de objetividad.

Red de propuestas regionales (RPN)

Una red que encuentra regiones con objetos se llama **red de propuestas regionales (RPN)**. El detector de objetos conocido como Faster RCNN codifica una gran colección de cajas delimitadoras como un mapa de tamaño fijo. Luego construye una red que permite predecir una puntuación para cada caja, y entrena esta red para que la puntuación sea grande cuando la caja contiene un objeto, y pequeña en caso contrario. Codificar cajas como un mapa es sencillo. Consideramos cajas centradas en puntos a lo largo de la imagen; no necesitamos considerar cada punto posible (porque al moverse un pixel no es probable que cambie la clasificación); una buena elección es un **paso** (el desplazamiento entre puntos centrales) de 16 pixeles. Para cada punto central consideramos varias cajas posibles, llamadas **cajas de anclaje**. Faster RCNN utiliza nueve cajas: tamaños pequeño, mediano y grande; y relaciones de aspecto altas, anchas y cuadradas.

En términos de la arquitectura de la red neuronal, construye un bloque 3D donde cada ubicación espacial en el bloque tiene dos dimensiones para el punto central y una dimensión para el tipo de caja. Ahora cualquier caja con una puntuación de objetividad suficientemente buena se llama **región de interés** (**ROI**, por sus siglas en inglés), y debe ser comprobada por un clasificador. Pero los clasificadores CNN prefieren imágenes de tamaño fijo, y las cajas que pasan la prueba de objetividad variarán en tamaño y forma. No es posible que las cajas tengan el mismo número de pixeles, pero podemos hacer que tengan el mismo número de características muestreando los pixeles para extraer características, un proceso llamado **agrupamiento de ROI**. Este mapa de características de tamaño fijo se pasa luego al clasificador.

Ahora, respecto al problema de decidir qué ventanas informar. Suponga que miramos ventanas de tamaño 32×32 con un paso de 1: cada ventana está desplazada por un sólo pixel de la anterior. Habrá muchas ventanas que son similares y deberían tener puntuaciones parecidas. Si todas tienen una puntuación por encima del umbral, no queremos informar todas ellas, porque todas se refieren a vistas ligeramente diferentes del mismo objeto. Por otro lado, si el paso es demasiado grande, podría ser que un objeto no esté contenido en ninguna ventana y se perderá. En cambio,

Supresión no máxima

podemos usar un algoritmo codicioso llamado **supresión no máxima.** Así, primero, construya una lista ordenada de todas las ventanas con puntuaciones por encima de un umbral; luego, mientras haya ventanas en la lista, elija la ventana con la puntuación más alta y acéptela como que contiene un objeto; descarte de la lista todas las demás ventanas que se superponen en gran medida.

Finalmente, tenemos el problema de informar la ubicación precisa de los objetos. Suponga que tenemos una ventana que tiene una puntuación alta y ha pasado por la supresión no máxima. Es poco probable que esta ventana esté en el lugar correcto exacto (recuerde, miramos un número relativamente pequeño de ventanas con un pequeño número de tamaños posibles). Utilizamos la representación de características calculada por el clasificador para predecir mejoras que recortarán la ventana a un cuadro delimitador adecuado, un paso conocido como **regresión de caja delimitadora**.

Regresión de caja delimitadora

Evaluar los detectores de objetos requiere cuidado. Primero necesitamos un conjunto de prueba: una colección de imágenes con cada objeto en la imagen marcada por una etiqueta de categoría de verdad de base y un cuadro delimitador. Por lo general, los cuadros y las etiquetas son proporcionados por humanos. Luego alimentamos cada imagen al detector de objetos y comparamos su salida con la verdad de base. Deberíamos estar dispuestos a aceptar cuadros que estén desviados por unos pocos pixeles, porque los cuadros de verdad de base no serán perfectos. La puntuación de evaluación debe equilibrar el recuerdo (encontrar todos los objetos que están allí) y la precisión (no encontrar objetos que no están allí).

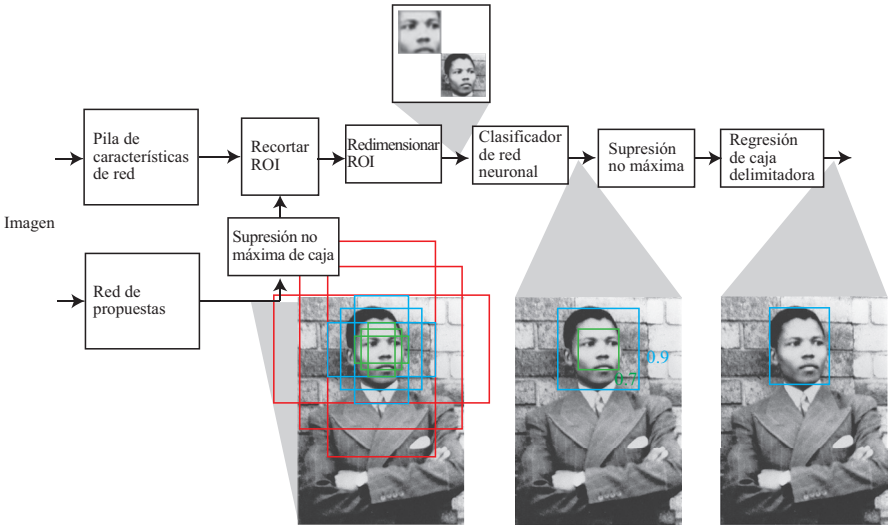


Figura 25.13 Faster RCNN utiliza dos redes. Se alimenta una imagen de un joven Nelson Mandela en el detector de objetos. Una red calcula las puntuaciones de "objetividad" de las cajas de imagen candidatas, llamadas "cajas de anclaje", centradas en un punto de la cuadrícula. Hay nueve cajas de anclaje (tres escalas, tres relaciones de aspecto) en cada punto de la retícula. Para la imagen de ejemplo, una caja verde interior y una caja azul exterior han pasado la prueba de objetividad. La segunda red es una pila de características que calcula una representación de la imagen adecuada para la clasificación. Las cajas con la puntuación de objetividad más alta se cortan del mapa de características, se estandarizan en tamaño con el agrupamiento de ROI y se pasan a un clasificador. La caja azul tiene una puntuación más alta que la caja verde y la superpone, por lo que la caja verde es rechazada por la supresión no máxima. Finalmente, la regresión del cuadro delimitador lleva la caja azul para que se ajuste a la cara. Esto significa que el muestreo relativamente grueso de ubicaciones, escalas y relaciones de aspecto no debilita la precisión. Foto de Sipa/Shutterstock.

25.6 El mundo 3D

Las imágenes muestran una imagen 2D de un mundo 3D. Pero esta imagen 2D está llena de pistas sobre el mundo 3D. Un tipo de pista ocurre cuando tenemos múltiples imágenes del mismo mundo, y podemos emparejar puntos entre imágenes. Otro tipo de pista está disponible dentro de una sola imagen.

25.6.1 Pistas 3D desde múltiples vistas

Dos imágenes de objetos en un mundo 3D son mejores que una por varias razones:

- Si tiene dos imágenes de la misma escena tomadas desde diferentes puntos de vista y sabe suficiente sobre las dos cámaras, es posible construir un modelo 3D, una colección de puntos con sus coordenadas en 3 dimensiones, averiguando qué punto en la primera vista corresponde a qué punto en la segunda vista, y aplicando algo de geometría. Esto es cierto para casi todos los pares de direcciones de visualización y casi todos los tipos de cámara.
- Si tiene dos vistas de suficientes puntos, y sabe qué punto en la primera vista corresponde a qué punto en la segunda vista, no necesita saber mucho sobre las cámaras para construir

un modelo 3D. Dos vistas de dos puntos que le dan cuatro coordenadas x,y, y sólo necesitas tres coordenadas para especificar un punto en el espacio 3D; la coordenada extra es útil para averiguar lo que necesita saber sobre las cámaras. Esto es cierto para casi todos los pares de direcciones de visualización y casi todos los tipos de cámara.

El problema clave es establecer qué punto en la primera vista corresponde a cuál en la segunda vista. Las descripciones detalladas de la apariencia local de un punto utilizando características de textura simples (como las de la sección 25.3.2) suelen ser suficientes para emparejar puntos. Por ejemplo, en una escena de tráfico en una calle podría haber sólo un semáforo verde visible en dos imágenes tomadas de esta escena; podemos entonces hipotetizar que estos corresponden entre sí. La geometría de múltiples vistas de cámara está muy bien entendida (pero es demasiado complicada para exponer aquí). La teoría produce restricciones geométricas sobre qué punto en una imagen puede coincidir con qué punto en la otra. Es posible obtener otras restricciones razonando sobre la suavidad de las superficies reconstruidas.

Hay dos formas de obtener múltiples vistas de una escena. Una es tener dos cámaras o dos ojos (sección 25.6.2). Otra es moverse (sección 25.6.3). Si tiene más de dos vistas, permite recuperar tanto la geometría del mundo como los detalles de la vista con mucha precisión. La sección 25.7.3 discute algunas aplicaciones para esta tecnología.

25.6.2 Estereopsis binocular

La mayoría de los vertebrados tienen dos ojos. Esto es útil para la redundancia en caso de perder un ojo, pero también ayuda de otras maneras. La mayoría de las presas tienen los ojos en el lado de la cabeza para permitir un campo de visión más amplio. Los depredadores tienen los ojos al frente, lo que les permite usar la **estereopsis binocular**. Sostenga ambos índices frente a su cara, con un ojo cerrado, y ajústelos para que el dedo delantero oculte el otro dedo en la *vista del ojo abierto*. Cambie de ojo, ahora debería notar que los dedos han cambiado de posición uno con respecto al otro. Este cambio de posición de la vista izquierda a la vista derecha se conoce como **disparidad**. En la elección correcta del sistema de coordenadas, si superponemos las imágenes izquierda y derecha de un objeto a cierta profundidad, el objeto se desplaza de manera horizontal en la imagen superpuesta, y el tamaño del desplazamiento es el recíproco de la profundidad. Esto se observa en la Figura 25.14, donde el punto más cercano de la pirámide se desplaza a la izquierda en la imagen derecha, pero a la derecha en la imagen izquierda.

Estereopsis binocular

Disparidad

Para medir la disparidad necesitamos resolver el problema de correspondencia, para determinar para un punto en la imagen izquierda, su "pareja" en la imagen derecha que resulta de la proyección del mismo punto de la escena. Esto es análogo a lo que se hace al medir el flujo óptico, y los enfoques más simples son algo similares. Estos métodos buscan bloques de pixeles izquierdos y derechos que coincidan, utilizando la suma de diferencias cuadradas (como en la Sección 25.3.3). Los métodos más sofisticados utilizan representaciones de textura más detalladas de bloques de pixeles (como en la Sección 25.3.2). En la práctica, utilizamos algoritmos mucho más sofisticados, que explotan restricciones adicionales.

Suponiendo que podemos medir la disparidad, ¿de qué manera esta medición nos proporciona información sobre la profundidad en la escena? Tendremos que trabajar la relación geométrica entre la disparidad y la profundidad. Consideraremos primero el caso en que ambos ojos (o cámaras) están mirando hacia adelante con sus ejes ópticos paralelos. La relación de la cámara derecha con la cámara izquierda es entonces sólo un desplazamiento a lo largo del eje x por una cantidad b, la **línea base**. Podemos usar las ecuaciones de flujo óptico de la Sección 25.3.3, si pensamos en esto como resultado de un vector de traslación $\mathbf{T}$ actuando durante el tiempo δt, con $T_x = b/\delta t$ y $T_y = T_z = 0$. La disparidad horizontal y vertical están dadas por los componentes del flujo óptico,

Línea base

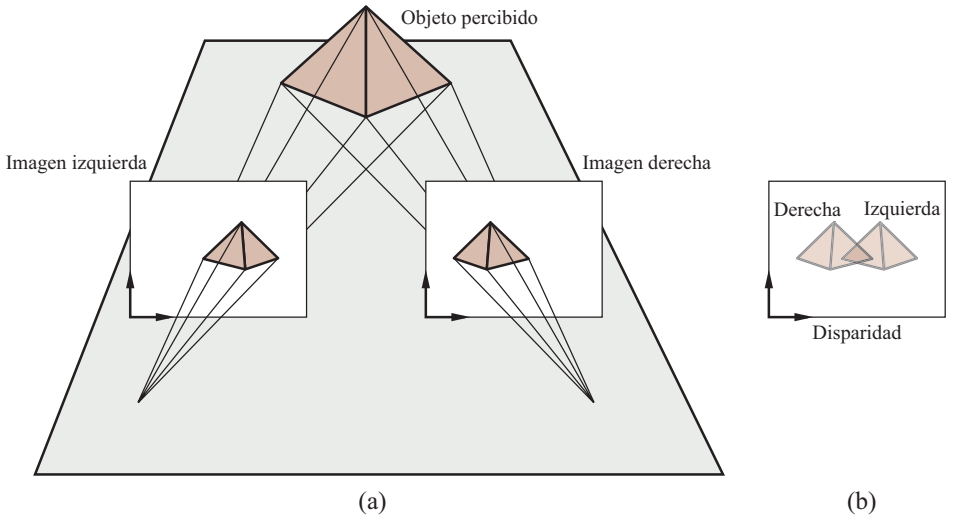


Figura 25.14 Traducir una cámara paralela al plano de la imagen causa que las características de la imagen se muevan en el plano de la cámara. La disparidad en las posiciones que resulta es una pista para la profundidad. Si superponemos las imágenes izquierda y derecha, como en (b), vemos la disparidad.

multiplicados por el paso de tiempo δt, $H = v_x \delta t$, $V = v_y \delta t$. Realizando las sustituciones, obtenemos el resultado de que $H = b/Z$, $V = 0$. En otras palabras, la disparidad horizontal es igual a la relación de la línea base con la profundidad, y la disparidad vertical es cero. Podemos recuperar la profundidad Z dado que conocemos b, y podemos medir H.

Fijar

Bajo condiciones normales de visualización, los humanos **fijan**; es decir, hay algún punto en la escena en el que se intersectan los ejes ópticos de los dos ojos. La Figura 25.15 muestra dos ojos fijados en un punto P_0, que está a una distancia Z del punto medio de los ojos. Por conveniencia, calcularemos la disparidad *angular*, medida en radianes. La disparidad en el punto de fijación P_0 es cero. Para algún otro punto P en la escena que está δZ más lejos, podemos calcular los desplazamientos angulares de las imágenes izquierda y derecha de P, que llamaremos P_L y P_R, respectivamente. Si cada uno de estos puntos se desplaza por un ángulo $\delta\theta/2$ en relación a P_0, entonces el desplazamiento entre P_L y P_R, que es la disparidad de P, es sólo $\delta\theta$. Desde la Figura 25.15, $\tan\theta = \frac{b/2}{Z}$ y $\tan(\theta - \delta\theta/2) = \frac{b/2}{Z+\delta Z}$, pero para ángulos pequeños, $\tan\theta \approx \theta$, entonces

$$\delta\theta/2 = \frac{b/2}{Z} - \frac{b/2}{Z+\delta Z} \approx \frac{b\delta Z}{2Z^2}$$

y, dado que la disparidad real es $\delta\theta$, tenemos

$$\text{disparidad} = \frac{b\delta Z}{Z^2}$$

En humanos, la línea base b es de unos 6 cm. Suponga que Z es de unos 100 cm y que el $\delta\theta$ detectable más pequeño (correspondiente al tamaño de un solo pixel) es de unos 5 segundos de arco, dando un δZ de 0.4 mm. Para $Z = 30$ cm, obtenemos el valor $\delta Z = 0.036$ mm. Es decir, a una distancia de 30 cm, los humanos pueden discriminar profundidades que difieren por tan sólo 0.036 mm, lo que nos permite enhebrar agujas y similares.

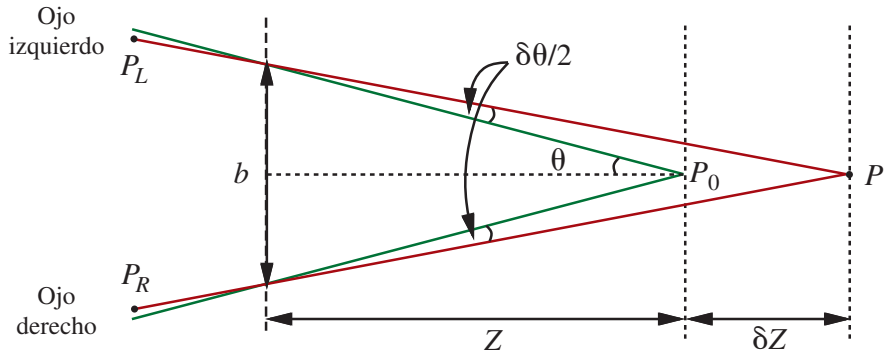


Figura 25.15 Relación entre disparidad y profundidad en estereopsis. Los centros de proyección de los dos ojos están a una distancia b, y los ejes ópticos se intersectan en el punto de fijación P_0. El punto P en la escena se proyecta a los puntos P_L y P_R en los dos ojos. En términos angulares, la disparidad entre estos es $\delta\theta$ (el diagrama muestra dos ángulos de $\delta\theta/2$).

25.6.3 Pistas 3D de una cámara en movimiento

Suponga que tenemos una cámara moviéndose en una escena. Tomemos la Figura 25.14 y etiquetemos la imagen izquierda como "Tiempo t" y la imagen derecha como "Tiempo $t + 1$". La geometría no ha cambiado, por lo que todo el material de la discusión de estereopsis también se aplica cuando una cámara se mueve. Lo que llamamos disparidad en esa sección ahora se piensa como movimiento aparente en la imagen, y se llama flujo óptico. Esta es una fuente de información tanto para el movimiento de la cámara como para la geometría de la escena. Para entender esto, afirmamos (sin prueba) una ecuación que relaciona el flujo óptico con la velocidad de traslación del espectador **T** y la profundidad en la escena.

El campo de flujo óptico es un campo vectorial de velocidades en la imagen, $(v_x(x,y), v_y(x,y))$. Las expresiones para estos componentes, en un marco de coordenadas centrado en la cámara y asumiendo una longitud focal de $f = 1$, son

$$v_x(x,y) = \frac{-T_x + xT_z}{Z(x,y)} \quad \text{y} \quad v_y(x,y) = \frac{-T_y + yT_z}{Z(x,y)}.$$

donde $Z(x,y)$ es la coordenada z (es decir, profundidad) del punto en la escena correspondiente al punto en la imagen en (x,y).

Foco de expansión

Note que ambos componentes del flujo óptico, $v_x(x, y)$ y $v_y(x, y)$, son cero en el punto $x = T_x/T_z$, $y = T_y/T_z$. Este punto se llama **foco de expansión** del campo de flujo. Suponiendo que cambiamos el origen en el plano x–y para que esté en el foco de expansión; entonces las expresiones para el flujo óptico toman una forma simple. (x', y') definen las nuevas coordenadas denotadas por $x' = x - T_x/T_z$, $y' = y - T_y/T_z$. Entonces

$$v_x(x',y') = \frac{x'T_z}{Z(x',y')}, \qquad v_y(x',y') = \frac{y'T_z}{Z(x',y')}.$$

Note que hay una ambigüedad de factor de escala aquí (por eso asumir una longitud focal de $f = 1$ es inofensivo). Si la cámara se movía dos veces más rápido, y cada objeto en la escena era dos veces más grande y a dos veces la distancia a la cámara, el campo de flujo óptico sería el mismo. Pero aún podemos extraer información bastante útil.

1. Suponga que usted es una mosca intentando aterrizar en una pared y quiere información útil del campo de flujo óptico. Este campo de flujo óptico no puede informar la distancia a la pared o la velocidad hacia la pared, debido a la ambigüedad de escala. Pero si divide la distancia por la velocidad, la ambigüedad de escala se cancela. El resultado es el tiempo de contacto, dado por Z/T_z, y es muy útil para controlar el enfoque de aterrizaje. Hay evidencia experimental considerable de que muchas especies animales explotan esta pista.
2. Considere dos puntos a profundidades Z_1, Z_2, respectivamente. Podemos no conocer el valor absoluto de ninguno de estos, pero al considerar el inverso de la relación de las magnitudes de flujo óptico en estos puntos, podemos determinar la relación de profundidad Z_1/Z_2. Esta es la pista de paralaje de movimiento, una que usamos cuando miramos por la ventana lateral de un coche o tren en movimiento e inferimos que las partes del paisaje que se mueven de manera más lenta están más lejos.

25.6.4 Pistas 3D desde una vista

Incluso una sola imagen proporciona una rica colección de información sobre el mundo 3D. Esto es cierto incluso si la imagen es sólo un dibujo lineal. Los dibujos lineales han fascinado a los científicos de la visión, porque las personas tienen una sensación de forma y disposición 3D aunque el dibujo parece contener muy poca información para elegir de la vasta colección de escenas que podrían producir el mismo dibujo. La oclusión es una fuente clave de información: si hay evidencia en la imagen de que un objeto oculta a otro, entonces el objeto que oculta está más cerca del ojo.

En imágenes de escenas reales, la textura es una pista sólida para la estructura 3D. La sección 25.3.2 afirmó que la textura es un patrón repetitivo de texeles. Aunque la distribución de texeles puede ser uniforme en objetos en la escena —por ejemplo, guijarros en una playa—, puede no ser uniforme en la imagen, los guijarros más lejanos aparecen más pequeños que los guijarros más cercanos. Piense, como otro ejemplo, en un trozo de tela de lunares. Todos los puntos son del mismo tamaño y forma en la tela, pero en una vista en perspectiva algunos puntos son elipses debido al acortamiento. Los métodos modernos explotan estas pistas aprendiendo un mapeo de imágenes a estructura 3D (Sección 25.7.4), en lugar de razonar de manera directa sobre las matemáticas subyacentes de la textura.

La sombra —variación en la intensidad de luz recibida de diferentes partes de una superficie en una escena— es determinada por la geometría de la escena y por las propiedades de reflectancia de las superficies. Existe evidencia suficiente de que la sombra es una pista para la forma 3D. El argumento físico es fácil. Desde el modelo físico de la sección 25.2.4, sabemos que si una normal de superficie apunta hacia la fuente de luz, la superficie es más brillante, y si apunta en sentido contrario, la superficie es más oscura. Este argumento se complica si la reflectancia de la superficie no es conocida, y el campo de iluminación no es uniforme, pero los humanos parecen ser capaces de obtener una percepción útil de la forma a partir de la sombra. Sabemos, con frustración, poco sobre algoritmos para hacer esto.

Si hay un objeto familiar en la imagen, cómo se ve depende de su **pose**, es decir, su posición y orientación con respecto al espectador. Hay algoritmos para recuperar la pose a partir de correspondencias entre puntos en un objeto y puntos en un modelo del objeto. Recuperar la pose de un objeto conocido tiene múltiples aplicaciones; por ejemplo, en una tarea de manipulación industrial, el brazo del robot no permite recoger un objeto hasta que se conoce la pose. Las aplicaciones de cirugía robótica dependen de calcular las transformaciones entre la posición de la cámara y las posiciones de la herramienta quirúrgica y el paciente (para obtener la transformación de la posición de la herramienta a la posición del paciente).

Pose

Las relaciones espaciales entre los objetos son otra pista importante. Aquí hay un ejemplo. Todos los peatones tienen la misma altura y tienden a estar en un plano sobre el piso. Si nosotros sabemos dónde está el horizonte en una imagen, podemos clasificar a los peatones por distancia a la cámara.

Esto funciona porque sabemos dónde están sus pies; y los peatones, cuyos pies están más cerca del horizonte en la imagen, están más lejos de la cámara, por lo que deben ser más pequeños en la imagen. Esto significa que podemos descartar algunas respuestas del detector, si un detector encuentra un peatón que es grande en la imagen y cuyos pies están cerca del horizonte, ha encontrado un peatón enorme; estos no existen, por lo que el detector está equivocado. A su vez, un detector de peatones confiable es capaz de producir estimaciones del horizonte, si hay varios peatones en la escena a diferentes distancias de la cámara. Esto se debe a que el escalado relativo de los peatones es una pista de dónde está el horizonte. En consecuencia, podemos extraer una estimación del horizonte del detector, luego usar esta estimación para podar los errores del detector de peatones.

25.7 Usando visión por computadora

Aquí examinamos una serie de aplicaciones de visión por computadora. Ahora hay muchas herramientas y kits de herramientas de visión por computadora confiables, por lo que la gama de aplicaciones exitosas y útiles es extraordinaria. Muchas son desarrolladas en casa por entusiastas para propósitos especiales, lo cual es un testimonio de cuán utilizables son los métodos y cuánto impacto tienen. (Por ejemplo, un entusiasta creó una excelente puerta para mascotas con base en la detección de objetos que niega la entrada a un gato si éste trae un ratón muerto, una búsqueda en la web lo encontrará para usted).

25.7.1 Entendiendo lo que las personas están haciendo

Si pudiéramos construir sistemas que entendieran lo que las personas hacen al analizar videos, podríamos construir interfaces de computadora-humanas que observen a las personas y reaccionen a su comportamiento. Con estas interfaces podríamos: diseñar mejores edificios y lugares públicos al recopilar y utilizar datos sobre lo que las personas hacen en público, construir sistemas de vigilancia de seguridad más precisos y menos intrusivos, construir comentaristas deportivos automatizados, hacer que los sitios de construcción y los lugares de trabajo sean más seguros al generar advertencias cuando las personas y las máquinas se acercan peligrosamente, construir videojuegos que hagan que un jugador se levante y se mueva, y ahorrar energía al gestionar el calor y la luz en un edificio para coincidir con el lugar donde están los ocupantes y lo que están haciendo.

El estado del arte para algunos problemas hoy es extremadamente sólido. Hay métodos que permiten predecir con mucha precisión las ubicaciones de las articulaciones de una persona en una imagen. Siguen estimaciones muy buenas de la configuración 3D del cuerpo de esa persona (vea la Figura 25.16). Esto funciona porque las imágenes del cuerpo tienden a tener efectos de perspectiva débiles, y los segmentos del cuerpo no varían mucho en longitud, por lo que el acortamiento de un segmento del cuerpo en una imagen es una buena pista para el ángulo entre él y el plano de la cámara. Con un sensor de profundidad, estas estimaciones se hacen lo suficientemente rápido para incorporarlas en las interfaces de los videojuegos.

Clasificar lo que las personas hacen al momento es más difícil. El video que muestra comportamientos bien estructurados, como ballet, gimnasia o tai chi, donde hay vocabularios específicos que se refieren a actividades bien delimitadas sobre fondos simples, es muy fácil de manejar. Es posible obtener buenos resultados con una gran cantidad de datos etiquetados y una red neuronal convolucional apropiada. Sin embargo, es difícil demostrar que los métodos realmente funcionan, porque dependen en gran medida del contexto. Por ejemplo, un clasificador que etiqueta bien las secuencias de "natación" podría ser un detector de piscinas, que no funcionaría para (digamos) nadadores en ríos.

Los problemas más generales siguen abiertos, por ejemplo, cómo vincular las observaciones del cuerpo y los objetos cercanos a los objetivos e intenciones de las personas en movimiento.

Figura 25.16 Reconstruir humanos a partir de una sola imagen hoy resulta práctico. Cada fila muestra una reconstrucción de la forma del cuerpo 3D obtenida a partir de una sola imagen. Estas reconstrucciones son posibles porque los métodos permiten estimar la ubicación de las articulaciones, los ángulos de las articulaciones en 3D, la forma del cuerpo y la pose del cuerpo con respecto a una imagen. Cada fila muestra lo siguiente: **extrema izquierda** una imagen; **centro izquierda** la imagen con el cuerpo reconstruido superpuesto; **centro derecha** otra vista del cuerpo reconstruido, y **derecha** otra vista del cuerpo reconstruido. Las diferentes vistas del cuerpo dificultan mucho ocultar errores en la reconstrucción. (Figura cortesía de Angjoo Kanazawa, producida por un sistema descrito en Kanazawa *et al*. (2018a)).

Una fuente de dificultad es que comportamientos similares parecen diferentes, y comportamientos diferentes parecen similares, como muestra la Figura 25.17.

Otra dificultad se debe a la escala de tiempo. Lo que alguien está haciendo depende mucho de la escala de tiempo, como lo ilustra la Figura 25.18. Otro efecto importante que se muestra en esa figura es que el comportamiento permite ser compuesto: varios comportamientos reconocidos permiten combinarse para formar un comportamiento de nivel superior como preparar un refrigerio.

También es posible que se produzcan comportamientos no relacionados al mismo tiempo, como interpretar una canción mientras se prepara un refrigerio. Un desafío es que no tenemos un vocabulario común para las piezas de comportamiento. Las personas tienden a pensar que conocen muchos nombres de comportamientos, pero no permiten producir listas largas de tales palabras a pedido. Eso dificulta la obtención de conjuntos de datos de comportamientos etiquetados de manera consistente.

Los clasificadores aprendidos están garantizados para comportarse bien sólo si los datos de entrenamiento y prueba provienen de la misma distribución. No tenemos forma de verificar que esta restricción se aplica a las imágenes, pero observamos que los clasificadores de imágenes y los detectores de objetos funcionan muy bien. Sin embargo, para los datos de actividad, la relación entre los datos de entrenamiento y prueba es más desconfiable porque las personas hacen tantas cosas en tantos contextos. Por ejemplo, suponga que tenemos un detector de peatones que funciona bien en

Figura 25.17 La misma acción puede parecer muy diferente; y diferentes acciones pueden parecer similares. Estos ejemplos muestran acciones tomadas de un conjunto de datos de comportamientos naturales; las etiquetas son elegidas por los curadores del conjunto de datos, en lugar de ser predichas por un algoritmo. **Arriba:** ejemplos de la etiqueta "abrir refrigerador", algunos mostrados en primer plano y otros desde lejos. **Abajo:** ejemplos de la etiqueta "sacar algo del refrigerador". Observe cómo en ambas filas la mano del sujeto está cerca de la puerta del refrigerador: distinguir entre los casos requiere un juicio muy sutil sobre dónde está la mano y dónde está la puerta. (Figura cortesía de David Fouhey, tomada de un conjunto de datos descrito en Fouhey *et al.* (2018)).

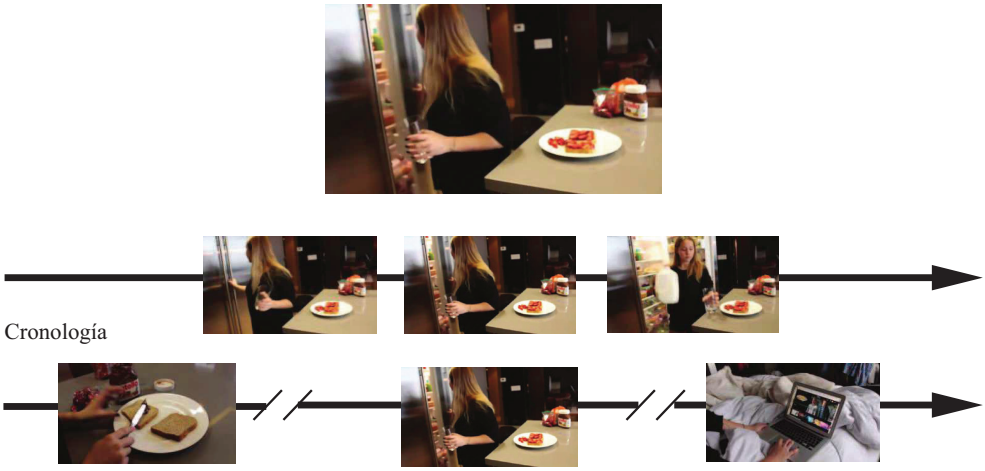


Figura 25.18 Lo que llama una acción depende de la escala de tiempo. El único cuadro en la parte **superior** se describe mejor como abrir el refrigerador (no mira el contenido cuando cierra un refrigerador). Pero si mira un corto clip de video (indicado por los cuadros en la fila del **centro**), la acción se describe mejor como sacar leche del refrigerador. Si mira un clip largo (los cuadros en la fila **inferior**), la acción se describe mejor como preparar un refrigerio. Observe que esto ilustra una forma en que el comportamiento se compone: sacar leche del refrigerador a veces es parte de preparar un refrigerio, y abrir el refrigerador suele ser parte de sacar leche del refrigerador. (Figura cortesía de David Fouhey, tomada de un conjunto de datos descrito en Fouhey *et al.* (2018).

Un bebé comiendo un pedazo de comida en su boca

Un niño pequeño comiendo un pedazo de pastel

Un pequeño pájaro está posado en una rama

Un pequeño oso marrón está sentado en la hierba

Figura 25.19 Los sistemas automatizados de subtitulado de imágenes producen algunos resultados atinados y algunos fracasos. Las dos leyendas a la izquierda describen de manera atinada las respectivas imágenes, aunque "comiendo... en su boca" es una difluencia típica de los modelos de lenguaje de redes neuronales recurrentes utilizados por los primeros sistemas de subtitulado. Para las dos leyendas de la derecha, el sistema de subtitulado parece no saber sobre ardillas y, por lo tanto, adivina el animal a partir del contexto; también falla en reconocer que las dos ardillas están comiendo. Las imágenes mostradas son similares pero no idénticas a las imágenes originales de las que se generaron las leyendas. Para las imágenes originales vea Aneja *et al.* (2018). (Créditos de las imágenes: geraine/Shutterstock; ESB Professional/Shutterstock; BushAlex/Shutterstock; Maria.Tem/Shutterstock).

un gran conjunto de datos. Habrá fenómenos raros (por ejemplo, personas montando monociclos) que no aparecen en el conjunto de entrenamiento, por lo que no podemos decir con seguridad cómo funcionará el detector en tales casos. El desafío es demostrar que el detector es seguro, hagan lo que hagan los peatones; lo cual es difícil para las teorías actuales de aprendizaje

25.7.2 Vinculando imágenes y palabras

Muchas personas crean y comparten imágenes y videos en Internet. La dificultad está en encontrar lo que se quiere. Por lo regular, las personas quieren buscar usando palabras (en lugar de, por ejemplo, bocetos de ejemplo). Como la mayoría de las imágenes no vienen con palabras adjuntas, es natural intentar y construir **sistemas de etiquetado** que rotulen imágenes con palabras relevantes. La maquinaria subyacente es sencilla: aplicamos métodos de clasificación de imágenes y detección de objetos y rotulamos la imagen con las palabras de salida. Pero las etiquetas no son una descripción completa de lo que está sucediendo en una imagen. Importa quién está haciendo qué, pero las etiquetas no capturan esto. Por ejemplo, etiquetar una imagen de un gato en la calle con las categorías de objeto "gato", "calle", "bote de basura" y "espinas de pescado" deja fuera la información de que el gato está sacando las espinas de pescado de un bote de basura abierto en la calle.

Sistemas de etiquetado

Como alternativa al etiquetado, podríamos construir **sistemas de subtitulación**, sistemas que escriben una subtitulación de una o más frases que describen la imagen. La maquinaria subyacente es sencilla: acoplar una red convolucional (para representar la imagen) a una red recurrente neuronal o red transformadora (para generar frases), y entrenar el resultado con un conjunto de datos de imágenes subtituladas. Hay muchas imágenes con subtítulos disponibles en Internet; los conjuntos de datos curados usan trabajo humano para aumentar cada imagen con subtítulos adicionales para capturar la variación en el lenguaje natural. Por ejemplo, el conjunto de datos COCO (Common Objects in Context) es una colección completa de más de 200 000 imágenes etiquetadas con cinco subtítulos por imagen.

Sistemas de subtitulación

Los métodos actuales para la subtitulación usan detectores para encontrar un conjunto de palabras que describen la imagen, y proporcionan esas palabras a un modelo de secuencia entrenado para generar una frase.

Figura 25.20 Los sistemas de respuesta a preguntas visuales producen respuestas (en general, elegidas de un conjunto de opciones múltiples) a preguntas en lenguaje natural sobre imágenes. **Arriba:** el sistema produce respuestas muy sensatas a preguntas muy difíciles sobre la imagen. **Abajo:** respuestas menos satisfactorias. Por ejemplo, el sistema está adivinando sobre el número de agujeros en una pizza, porque no entiende qué es lo que cuenta como un agujero, y tiene dificultades reales para contar. De manera similar, el sistema selecciona marrón para la pata del gato porque el fondo es marrón y no permite localizar la pata de manera correcta. Las imágenes mostradas son similares pero no idénticas a las imágenes originales a las que se aplicó el sistema de respuesta a preguntas. Para las imágenes originales ver Goyal *et al.* (2017). (Créditos de imagen: (Arriba) Tobyanna/Shutterstock; 679411/Shutterstock; ESB Professional/Shutterstock; Africa Studio/Shutterstock; (Abajo) Stuart Russell; Maxisport/Shutterstock; Chendongshan/Shutterstock; Scott Biales DitchTheMap/Shutterstock).

Los métodos más precisos buscan entre las frases que el modelo genera para encontrar la mejor, y los métodos fuertes parecen requerir una búsqueda lenta. Las frases se evalúan con un conjunto de puntuaciones que verifican si la frase generada (a) utiliza frases comunes en las anotaciones de verdad de base y (b) no utiliza otras frases. Estas puntuaciones son difíciles de usar de manera directa como una función de pérdida, pero los métodos de aprendizaje por refuerzo se usan para entrenar redes que obtienen muy buenas puntuaciones. En la mayoría de los casos habrá una imagen en el conjunto de entrenamiento cuya descripción tiene el mismo conjunto de palabras que una imagen en el conjunto de prueba; en ese caso, un sistema de subtitulación recupera un subtítulo válido en lugar de generar uno nuevo. Los sistemas de escritura de subtítulos producen una mezcla de excelentes resultados y errores embarazosos (ver Figura 25.19).

Los sistemas de subtitulación permiten ocultar su ignorancia omitiendo mencionar detalles que no pueden acertar o utilizando pistas contextuales para adivinar. Por ejemplo, los sistemas de subtitulación tienden a ser malos para identificar el género de las personas en las imágenes y, con frecuencia, adivinan con base en las estadísticas de los datos de entrenamiento. Eso lleva a errores: a los hombres también les gusta ir de compras y las mujeres también hacen snowboard. Una forma de establecer si un sistema tiene una buena representación de lo que está sucediendo en una imagen es obligarlo a responder preguntas sobre la imagen. Este es un **sistema de respuesta a preguntas visuales** o sistema **VQA**. Una alternativa es un sistema de **diálogo visual**, al que se le da una imagen, su subtítulo y un diálogo. El sistema debe responder a la última pregunta en el diálogo. Como muestra la Figura 25.20, la visión sigue siendo difícil y los sistemas VQA con frecuencia cometen errores.

Respuesta a preguntas visuales (VQA)

Diálogo visual

25.7.3 Reconstrucción desde muchas vistas

Reconstruir un conjunto de puntos desde muchas vistas, que podrían provenir de video o de una colección de fotografías de turistas, es similar a reconstruir los puntos desde dos vistas, pero hay algunas diferencias importantes. Hay mucho más trabajo por hacer para establecer la correspondencia entre puntos en diferentes vistas pues los puntos pueden entrar y salir de la vista, lo que hace que el proceso de coincidencia y reconstrucción sea más desordenado. Pero más vistas significa más restricciones en la reconstrucción y en los parámetros de visualización recuperados, por lo que, en general, es posible producir estimaciones en extremo precisas tanto de la posición de los puntos como de los parámetros de visualización. Más o menos, la reconstrucción procede emparejando puntos en pares de imágenes, extendiendo estas coincidencias a grupos de imágenes, llegando a una solución aproximada para la geometría y los parámetros de visualización, luego puliendo esa solución. Pulir significa minimizar el error entre los puntos predichos por el modelo (de geometría y parámetros de visualización) y las ubicaciones de las características de la imagen. Los procedimientos detallados son demasiado complejos para cubrir por completo, pero ahora se entienden muy bien y son muy confiables.

Todas las restricciones geométricas sobre las correspondencias son conocidas para cualquier forma concebible de cámara útil. Los procedimientos permiten generalizarse para tratar con vistas que no son ortográficas, para tratar con puntos que sólo se observan en algunas vistas, para tratar con parámetros de cámara desconocidos (como la longitud focal), y para explotar varias búsquedas sofisticadas de correspondencias apropiadas. Es práctico reconstruir con precisión un modelo de una ciudad entera a partir de imágenes. Algunas aplicaciones son:

- **Construcción de modelos:** Por ejemplo, uno podría construir un sistema de modelado que toma muchas vistas que representan un objeto y produce una malla 3D detallada de polígonos texturizados para su uso en aplicaciones de gráficos por computadora y realidad virtual. Es habitual construir modelos como este a partir de video, por ahora es posible construir tales modelos a partir de conjuntos aparentemente aleatorios de imágenes. Es decir, puede construir un modelo 3D de la Estatua de la Libertad a partir de imágenes encontradas en Internet.
- **Mezcla animación con actores en vivo en video:** Para colocar personajes de gráficos por computadora en video real, necesitamos saber cómo se movió la cámara para el video real, para poder renderizar el personaje de manera correcta, cambiando la vista a medida que se mueve la cámara.
- **Reconstrucción de trayectorias:** Los robots móviles necesitan saber por dónde han pasado. Si el robot tiene una cámara, podemos construir un modelo del camino de la cámara por el mundo; eso servirá como una representación del camino del robot.
- **Gestión de la construcción:** Los edificios son artefactos en extremo complicados, y mantener un registro de lo que sucede durante su construcción es difícil y costoso. Una forma de mantenerse al día es sobrevolar el sitio de construcción, una vez a la semana, con drones para filmar el estado actual. Luego construir un modelo 3D del estado filmado y explorar la diferencia entre los planes y la reconstrucción utilizando técnicas de visualización. La Figura 25.21 ilustra esta aplicación.

25.7.4 Geometría desde una sola vista

Las representaciones geométricas son útiles si quiere moverse, porque permiten revelar dónde está, a dónde puede ir, y con qué es probable que se tope. Pero no siempre es conveniente usar múltiples vistas para producir un modelo geométrico. Por ejemplo, cuando abre la puerta y entra

Figura 25.21 Los modelos 3D de sitios de construcción se producen a partir de imágenes mediante algoritmos de estructura con base en el movimiento y estéreo de múltiples vistas. Estas imágenes ayudan a las empresas de construcción a coordinar el trabajo en grandes edificios comparando un modelo 3D de la construcción real con los planes de construcción visualizados a la fecha. **Izquierda:** Visualización de un modelo geométrico capturado por drones. Los puntos 3D reconstruidos se representan en color, por lo que el resultado parece un progreso hasta la fecha (note el edificio parcialmente completado con grúa). Las pequeñas pirámides muestran la pose de un dron cuando capturó una imagen, para permitir la visualización de la trayectoria de vuelo. **Derecha:** En realidad estos sistemas son utilizados por equipos de construcción; este equipo ve el modelo del sitio construido, y lo compara con los planos del edificio como parte de la reunión de coordinación. (Figura cortesía de Derek Hoiem, Mani Golparvar-Fard y Reconstruct, producida por un sistema comercial descrito en una publicación de blog en `medium.com/reconstruct-inc`).

en una habitación, sus ojos están demasiado juntos para recuperar una buena representación de la profundidad a objetos distantes a través de la habitación. Podría mover su cabeza de un lado a otro, pero eso consume tiempo y es inconveniente.

Mapa de profundidad

Una alternativa es predecir un **mapa de profundidad**, un arreglo que da la profundidad a cada pixel en la imagen, desde la cámara, a partir de una sola imagen. Para muchos tipos de escenas, esto es fácil de hacer con precisión, porque el mapa de profundidad tiene una estructura muy simple. Esto resulta en particular cierto para habitaciones y escenas de interiores en general. La mecánica es sencilla. Se obtiene un conjunto de datos de imágenes y mapas de profundidad, luego se entrena una red para predecir mapas de profundidad a partir de imágenes. Es posible resolver una interesante diversidad de variaciones del problema. El problema con un mapa de profundidad es que no dice nada sobre la parte trasera de los objetos, o el espacio detrás de estos. No obstante, hay métodos que permiten predecir qué voxels (pixeles 3D) están ocupados por objetos conocidos (la geometría del objeto es conocida), pero cómo se vería un mapa de profundidad si se eliminara un objeto (y, así, dónde podría esconder objetos). Estos métodos funcionan porque las formas de los objetos están bastante estilizadas.

Como vimos en la Sección 25.6.4, recuperar la pose de un objeto conocido usando un modelo 3D es sencillo. Ahora, imagine que ve una sola imagen de, digamos, un gorrión. Si en el pasado ha visto muchas imágenes de aves parecidas a gorriones, es posible reconstruir una estimación razonable tanto de la pose del gorrión como de su modelo geométrico a partir de esa única imagen. Usando las imágenes pasadas construye una pequeña familia paramétrica de modelos geométricos para aves parecidas a gorriones; luego, utiliza un procedimiento de optimización para encontrar el mejor conjunto de parámetros y puntos de vista para explicar la imagen que ve. Este argumento también funciona para proporcionar textura para ese modelo, incluso para las partes que no puede ver (Figura 25.22).

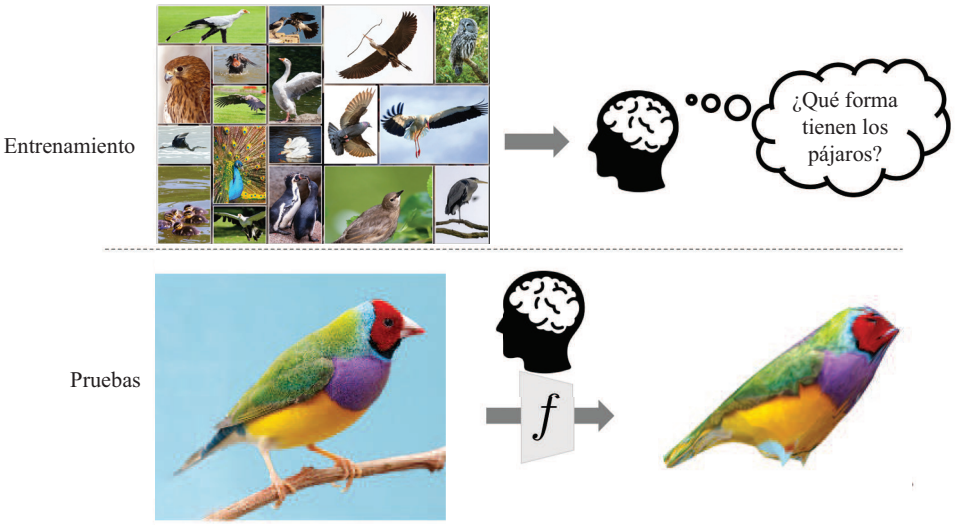


Figura 25.22 Si ha visto muchas imágenes de alguna categoría, digamos, aves (**arriba**), es posible usarlas para producir una reconstrucción 3D desde una nueva vista única (**abajo**). Necesita estar seguro de que todos los objetos tienen una geometría similar (así que una imagen de un avestruz no ayudará si está mirando un gorrión), pero los métodos de clasificación permiten resolver esto. A partir de la diversidad de imágenes puede estimar cómo se distribuyen los valores de textura en la imagen a través del objeto, y así completar la textura para partes del ave que aún no ha visto (**abajo**). (Figura cortesía de Angjoo Kanazawa, producida por un sistema descrito en Kanazawa *et al.* (2018b). Crédito de la foto superior: Satori/123RF; Crédito inferior izquierdo: Four Oaks/Shutterstock).

25.7.5 Haciendo imágenes

Ahora es común insertar modelos de gráficos por computadora en fotografías de manera convincente, como en la Figura 25.23, donde se ha colocado una estatua en una foto de una habitación. Primero estime un mapa de profundidad y albedo para la imagen. Luego estime la iluminación en la imagen al compararla con otras imágenes con iluminación conocida. Coloque el objeto en el mapa de profundidad de la imagen, y renderice el mundo resultante con un programa de renderizado físico, una herramienta estándar en gráficos por computadora. Finalmente, mezcle la imagen modificada con la imagen original.

Transformación de imagen

Las redes neuronales también permiten ser entrenadas para hacer **transformación de imágenes**: mapear imágenes desde tipo X; por ejemplo, una imagen borrosa, una imagen aérea de una ciudad o el dibujo de un nuevo producto, a imágenes de tipo Y, como, una versión desenfocada de la imagen, un mapa de carreteras o una fotografía de producto. Esto es más fácil cuando los datos de entrenamiento consisten en pares de imágenes (X, Y), en la Figura 25.24 cada par de ejemplos tiene una imagen aérea y la sección correspondiente del mapa de carreteras. La pérdida de entrenamiento compara la salida de la red con la salida deseada, y también tiene un componente de pérdida de una red generativa adversaria (GAN) que asegura que la salida tiene los tipos correctos de características para las imágenes de tipo Y. Como vemos en la parte de prueba de la Figura 25.24, los sistemas de este tipo funcionan muy bien.

A veces no tenemos imágenes que estén emparejadas entre sí, pero sí tenemos una gran colección de imágenes de tipo X (digamos, fotos de caballos) y una colección separada de tipo Y

Figura 25.23 A la **izquierda**, imagen de una escena real. A la **derecha**, se ha insertado un objeto de gráficos por computadora en la escena. Note que la luz parece venir de la dirección correcta, y que el objeto parece proyectar sombras apropiadas. La imagen generada es convincente aun si hay pequeños errores en la iluminación y las sombras, porque las personas no son expertas en identificar estos errores. (Figura cortesía de Kevin Karsch, producida por un sistema descrito en Karsch *et al.* (2011)).

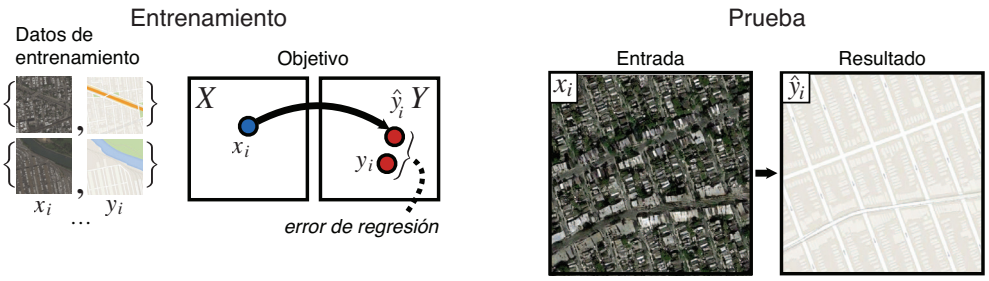


Figura 25.24 Traducción de imagen emparejada donde la entrada consta de imágenes aéreas y las correspondientes fichas de mapa, y el objetivo es entrenar una red para producir una ficha de mapa a partir de una imagen aérea. (El sistema también permite aprender a generar imágenes aéreas a partir de fichas de mapa). La red se entrena comparando $\hat{y}_i$ (la salida para el ejemplo x_i de tipo X) con la salida correcta y_i de tipo Y. Luego, en el momento de la prueba, la red debe hacer nuevas imágenes de tipo Y a partir de nuevas entradas de tipo X. (Figura cortesía de Phillip Isola, Jun-Yan Zhu y Alexei A. Efros, producida por un sistema descrito en Isola *et al.* (2017). Datos del mapa © 2019 Google).

(digamos, fotos de cebras). Imagine a un artista que tiene la tarea de crear una imagen de una cebra corriendo en un campo. Al artista le gustaría poder seleccionar justo la imagen correcta de un caballo, y luego tener la computadora transformando de manera automática el caballo en una cebra (Figura 25.25). Para lograr esto podemos entrenar dos redes de transformación, con una restricción adicional llamada restricción de ciclo. La primera red mapea caballos a cebras; la segunda red mapea cebras a caballos; y la restricción de ciclo requiere que cuando mapea X a Y a X

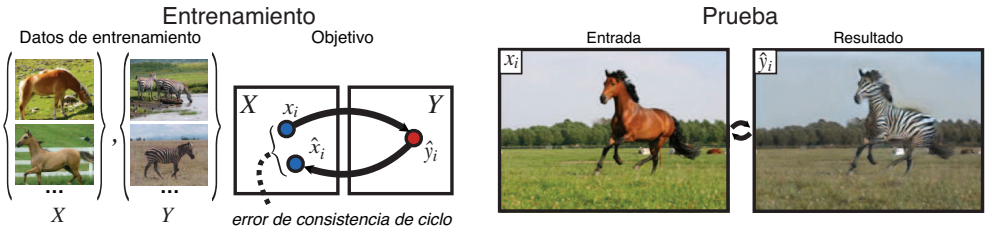


Figura 25.25 Traducción de imagen no emparejada: dadas dos poblaciones de imágenes (aquí el tipo X son caballos y el tipo Y son cebras), pero sin pares correspondientes, aprenda a traducir un caballo en una cebra. El método entrena dos predictores: uno que mapea el tipo X al tipo Y, y otro mapea el tipo Y al tipo X. Si la primera red mapea un caballo x_i a una cebra $\hat{y}_i$, la segunda red debería mapear $\hat{y}_i$ de vuelta al x_i original. La diferencia entre x_i y $\hat{x}_i$ es lo que entrena las dos redes. El ciclo de Y a X y de vuelta debe estar cerrado. Tales redes permiten imponer con éxito transformaciones en las imágenes. (Figura cortesía de Alexei A. Efros; ver Zhu *et al.* (2017). Foto de caballo corriendo de Justyna Furmanczyk Gibaszek/Shutterstock).

(o Y a X a Y), obtiene lo que empezó. De nuevo, las pérdidas de GAN aseguran que las imágenes de caballos (o cebras) que las redes emiten son "como" imágenes reales de caballos (o cebras).

Transferencia de estilo

Otro efecto artístico se llama **transferencia de estilo**: la entrada consta de dos imágenes: *contenido* (por ejemplo, una fotografía de un gato), y el *estilo* (es decir, una pintura abstracta). La salida es una versión del gato representada en el estilo abstracto (ver Figura 25.26). La clave para resolver este problema es que si examinamos una red neuronal convolucional profunda (CNN) que ha sido entrenada para hacer reconocimiento de objetos (por ejemplo, en ImageNet), encontramos que las primeras capas tienden a representar el estilo de una imagen, y las últimas capas representan el contenido. Dejemos que p sea la imagen de contenido y s sea la imagen de estilo, y dejemos que $E(x)$ sea el vector de activaciones de una capa temprana en la imagen x y $L(x)$ sea el vector de activaciones de una capa tardía en la imagen x. Luego queremos generar alguna imagen x que tenga un contenido similar a la foto de la casa, es decir, minimiza $|L(x) - L(p)|$, y también tiene un estilo similar a la pintura impresionista, esto es, minimiza $|E(x) - E(s)|$. Usamos el descenso de gradiente con una función de pérdida que es una combinación lineal de estos dos factores para encontrar una imagen x que minimice la pérdida.

Deepfake

Las redes generativas adversarias o antagónicas (GANs) permiten crear imágenes fotorrealistas novedosas, engañando a la mayoría de las personas la mayoría del tiempo. Un tipo de imagen es el **deepfake**, una imagen o video que parece una persona en particular, pero se genera a partir de un modelo. Por ejemplo, cuando Carrie Fisher tenía 60 años, se superpuso una réplica generada de su rostro de 19 años en el cuerpo de otro actor para la realización de *Rogue One*. La industria del cine crea cada vez mejores deepfakes con fines artísticos, y los investigadores trabajan en contramedidas para detectar deepfakes, para mitigar los efectos destructivos de las noticias falsas.

Las imágenes generadas también se pueden usar para mantener la privacidad. Por ejemplo, hay conjuntos de datos de imágenes en prácticas radiológicas que serían útiles para los investigadores, pero no se permiten publicar debido a la confidencialidad del paciente. Los modelos de imágenes generativas toman un conjunto de datos privado de imágenes y producen un conjunto de datos sintéticos que se pueden compartir con los investigadores. Este conjunto de datos debe ser (a) como el conjunto de datos de entrenamiento; (b) diferente; y (c) controlable. Considere las radiografías de tórax. El conjunto de datos sintético debe ser como el conjunto de datos de entrenamiento en el sentido de que cada imagen individual engañaría a un radiólogo y las frecuencias de cada efecto deberían ser correctas, por lo que un radiólogo no se sorprendería de que tan

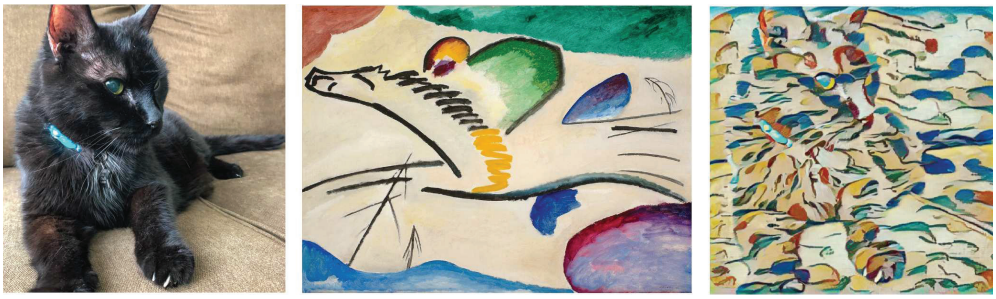


Figura 25.26 Transferencia de estilo: El *contenido* de una foto de un gato se combina con el *estilo* de una pintura abstracta para producir una nueva imagen del gato representada en estilo abstracto (derecha). La pintura es *Lyrisches* o *The Lyrical* de Wassily Kandinsky (dominio público); el gato es Cosmo.

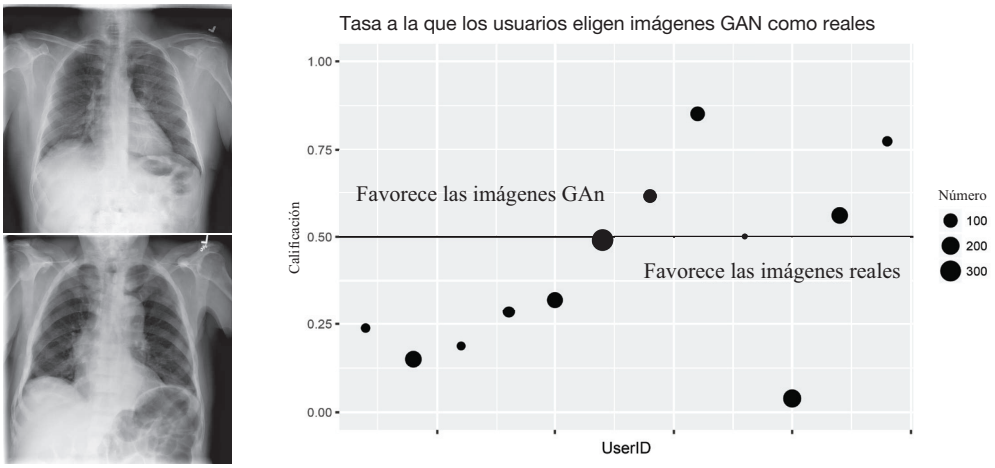


Figura 25.27 Imágenes generadas por GAN de radiografías de pulmón. A la izquierda, un par de radiografías, una real y una generada por GAN. A la derecha, resultados de una prueba que pide a los radiólogos, dado un par de radiografías como el de la izquierda, decir cuál es la radiografía real. En promedio, 61 % del tiempo eligieron de manera correcta, algo mejor que al azar. Pero difirieron en su precisión, el gráfico de la derecha muestra la tasa de error para 12 radiólogos diferentes; uno de ellos tuvo una tasa de error cercana a 0 % y otro tuvo errores de 80 %. El tamaño de cada punto indica el número de imágenes que cada radiólogo vio. Figura cortesía de Alex Schwing, producida por un sistema descrito en Deshpande *et al.* (2019).

frecuente (por ejemplo) aparece la neumonía. El nuevo conjunto de datos debería ser diferente, en el sentido de que no revela información personal identificable. El nuevo conjunto de datos debería ser controlable, de modo que las frecuencias de los efectos se puedan ajustar para reflejar las comunidades de interés. Por ejemplo, las neumonías son más comunes en ancianos que en adultos jóvenes. Cada uno de estos objetivos es difícil de alcanzar, pero se han creado conjuntos de datos de imágenes que algunas veces engañan a los radiólogos practicantes (Figura 25.27).

25.7.6 Controlando el movimiento con la visión

Uno de los principales usos de la visión es proporcionar información para manipular objetos, recogerlos, agarrarlos y girarlos, etc., así como para evitar obstáculos al navegar. La capacidad de usar la visión para estos propósitos está presente en los sistemas visuales animales más primitivos. En muchos casos, el sistema visual es mínimo, en el sentido de que extrae del campo de luz disponible sólo la información que el animal necesita para informar su comportamiento. Es probable que los sistemas de visión modernos evolucionaron a partir de organismos primitivos tempranos que usaban un punto fotosensible en un extremo para orientarse hacia (o lejos de) la luz. Vimos en la Sección 25.6 que las moscas usan un sistema de detección de flujo óptico muy simple para aterrizar en muros.

Suponga que, en lugar de aterrizar en muros, queremos construir un coche autónomo. Este es un proyecto que exige mucho más al sistema perceptivo. La percepción en un coche autónomo tiene que apoyar las siguientes tareas:

- **Control lateral:** Asegúrese de que el vehículo permanezca de manera segura dentro de su carril o cambie de carril con cuidado cuando sea necesario.
- **Control longitudinal:** Asegúrese de que haya una distancia segura entre su vehículo y el de adelante.
- **Evitar obstáculos:** Monitorear los vehículos en carriles vecinos y estar preparado para maniobras evasivas. Detectar peatones y permitirles cruzar de manera segura.
- **Obedecer las señales de tráfico:** Estas incluyen semáforos, señales de stop, señales de límite de velocidad y señales de mano de la policía.

El problema para un conductor (humano o computadora) es generar acciones de dirección, aceleración y frenado apropiadas para realizar mejor estas tareas.

Para tomar buenas decisiones, el conductor debe construir un modelo del mundo y los objetos en él. La Figura 25.28 muestra algunas de las inferencias visuales necesarias para construir este modelo. Para el control lateral, el conductor necesita mantener una representación de la posición y orientación del coche en relación con el carril. Para el control longitudinal, el conductor necesita mantener una distancia segura del vehículo de adelante (que no es fácil de identificar en, por ejemplo, carreteras curvas de varios carriles). Evitar obstáculos y dar seguimiento de las señales de tránsito requieren inferencias adicionales.

Las carreteras fueron diseñadas para humanos que conducen usando la visión, por lo que en principio debería ser posible conducir usando sólo la visión. Sin embargo, en la práctica, los coches autónomos comerciales utilizan una variedad de sensores, incluidas cámaras, LiDAR, radares y micrófonos. Un LiDAR o radar permite la medición directa de la profundidad, que es más precisa que los métodos visuales de la Sección 25.6. Tener múltiples sensores aumenta el rendimiento en general y es importante en condiciones de poca visibilidad; por ejemplo, el radar permite atravesar la niebla que bloquea las cámaras y los lidars. Los micrófonos pueden detectar vehículos que se acercan (en especial los que tienen sirenas) antes de que se hagan visibles.

También ha habido mucha investigación sobre robots móviles que navegan en entornos interiores y exteriores. Las aplicaciones abundan, como la última milla de entrega de paquetes o pizza. Los enfoques tradicionales dividen esta tarea en dos etapas, como se muestra en la Figura 25.29:

- **Construcción de mapas:** La Localización y Mapeo Simultáneos o SLAM (ver página 935) es la tarea de construir un modelo 3D del mundo, incluyendo la ubicación del robot en el mundo (o, más específico, la ubicación de cada una de las cámaras del robot). Este modelo (en general representado como una nube de puntos de obstáculos) puede construirse a partir de una serie de imágenes de diferentes posiciones de la cámara.

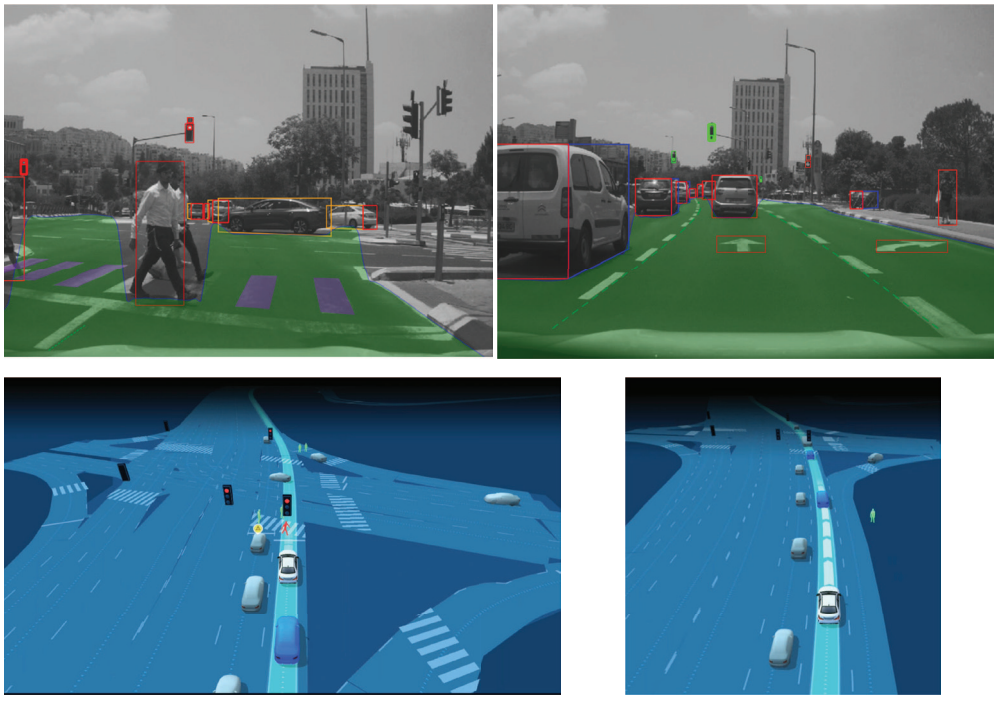


Figura 25.28 Detección basada en cámaras de Mobileye para vehículos autónomos. **Fila superior:** Dos imágenes de una cámara frontal, tomadas con segundos de diferencia. El área verde es el espacio libre, y es el área a la que el vehículo podría moverse físicamente en el futuro inmediato. Los objetos se muestran con cajas delimitadoras 3D que definen sus lados (rojo para la parte trasera, azul para el lado derecho, amarillo para el lado izquierdo y verde para el frente). Los objetos incluyen vehículos, peatones, el borde interno de las marcas del carril propio (necesario para el control lateral), otras marcas pintadas en la carretera y pasos de peatones, señales de tráfico y semáforos. No se muestran animales, postes y conos, aceras, barandillas y objetos generales (por ejemplo, un sofá que cayó de la caja de un camión). Cada objeto está marcado con una posición y velocidad 3D. **Fila inferior**: Un modelo físico completo del entorno, renderizado a partir de los objetos detectados. (Las imágenes muestran los resultados del sistema sólo de visión de Mobileye). (Imágenes cortesía de Mobileye).

- **Planificación de rutas:** Una vez que el robot tiene acceso a este mapa 3D y se localiza en él, el objetivo es encontrar una trayectoria libre de colisiones desde la posición actual hasta la ubicación de la meta (ver Sección 26.6).

Se han explorado múltiples variantes de este enfoque general. Por ejemplo, en el enfoque de mapeo y planificación cognitiva, las dos etapas de construcción de mapas y planificación de rutas son dos módulos en una red neuronal que se entrena de principio a fin para minimizar una función de pérdida. Tal sistema no tiene que construir un mapa completo, que es redundante e innecesario, si todo lo que necesita es suficiente información para navegar desde el punto A hasta el punto B sin chocar con obstáculos.

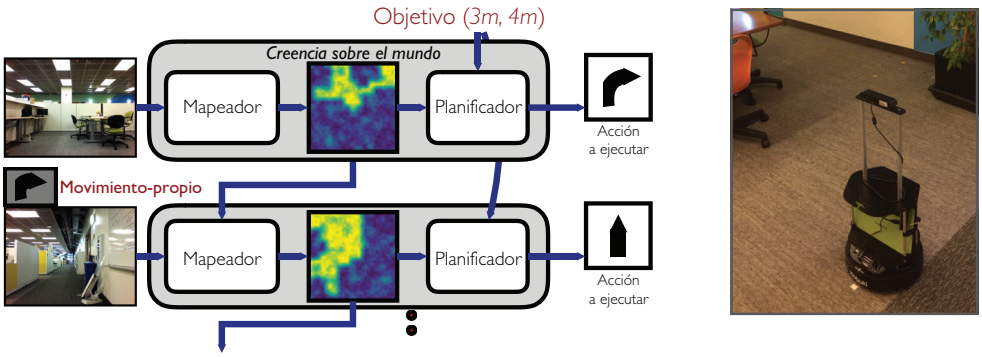


Figura 25.29 La navegación se aborda mediante la descomposición en dos problemas: mapeo y planificación. Con cada paso sucesivo en el tiempo, la información de los sensores se usa para construir de manera incremental un modelo incierto del mundo. Este modelo, junto con la especificación del objetivo, se pasa a un planificador que emite la próxima acción que el robot debe tomar para alcanzar el objetivo. Los modelos del mundo pueden ser geométricos (como en SLAM clásico), o semánticos (como se obtiene a través del aprendizaje), o, incluso, topológicos (con base en puntos de referencia). El robot real aparece a la derecha. (Figuras cortesía de Saurabh Gupta).

Resumen

Aunque la percepción parece ser una actividad sin esfuerzo para los humanos, requiere una cantidad significativa de cálculo sofisticado. El objetivo de la visión es extraer la información necesaria para tareas como manipulación, navegación y reconocimiento de objetos.

- La geometría y óptica de la formación de imágenes está bien entendida. Dada una descripción de una escena 3D, podemos producir con facilidad una imagen de ella desde alguna posición de cámara arbitraria, este es un problema de grafos. El problema inverso, el problema de visión por computadora, tomar una imagen y convertirla en una descripción 3D, es más difícil.
- Las representaciones de imágenes capturan bordes, texturas, flujo óptico y regiones. Estos proporcionan pistas para los límites de los objetos y para la correspondencia entre imágenes.
- Las redes neuronales convolucionales producen clasificadores de imágenes precisos que utilizan características aprendidas. Más o menos, las características son patrones de patrones de patrones. Es difícil predecir cuándo estos clasificadores funcionarán bien, porque los datos de prueba pueden ser diferentes a los datos de entrenamiento de alguna manera importante. La experiencia enseña que con frecuencia son lo suficientemente precisos para usar en la práctica.
- Los clasificadores de imágenes permiten convertirse en detectores de objetos. Un clasificador puntúa las cajas en una imagen por su objetividad; luego, otro decide si hay un objeto en la caja y qué objeto es. Los métodos de detección de objetos no son perfectos, pero son utilizables para una amplia variedad de aplicaciones.
- Con más de una vista de una escena, es posible recuperar la estructura 3D de la escena y la relación entre vistas. En muchos casos es posible recuperar la geometría 3D a partir de una sola vista.
- Los métodos de visión por computadora están siendo aplicados de manera amplia.

Notas bibliográficas e históricas

Este capítulo se ha centrado en la visión, pero otros canales perceptuales han sido estudiados y utilizados en robótica. Para la percepción auditiva (audición), ya hemos cubierto el reconocimiento de voz, y también ha habido un considerable trabajo en la percepción de la música (Koelsch y Siebel, 2005) y el aprendizaje automático de la música (Engel *et al.,* 2017) así como en el aprendizaje automático para sonidos en general (Sharan y Moir, 2016).

La percepción táctil o el tacto (Luo *et al.*, 2017) es importante en la robótica, esta se discute en el Capítulo 26. La percepción olfativa automatizada (olor) ha visto menos trabajo, pero se ha demostrado que los modelos de aprendizaje profundo permiten aprender a predecir olores basados en la estructura de las moléculas (Sanchez-Lengeling *et al.,* 2019).

Los intentos sistemáticos de entender la visión humana se remontan a la antigüedad. Euclides (*ca*. 300 a. C.) escribió sobre la perspectiva natural, la asociación que, con cada punto P en el mundo tridimensional, la dirección del rayo OP que une el centro de proyección O con el punto P. Euclides era consciente de la noción de paralaje de movimiento. Las pinturas romanas antiguas, como las conservadas por la erupción del Vesubio del año 79 d. C., utilizaban un tipo informal de perspectiva, con más de una línea de horizonte.

La comprensión matemática de la proyección en perspectiva, esta vez en el contexto de la proyección sobre superficies planas, tuvo su siguiente avance significativo en el siglo XV en la Italia renacentista. Brunelleschi suele ser acreditado con la creación de las primeras pinturas basadas en la proyección geométrica correcta de una escena tridimensional alrededor de 1413. En 1435, Alberti codificó las reglas e inspiró a generaciones de artistas. En particular, Leonardo da Vinci y Albrecht Dürer fueron notables en su desarrollo de la ciencia de la perspectiva, como se llamaba en aquellos días. Las descripciones de Leonardo a finales del siglo XV sobre la interacción de luz y sombra (claroscuro), las regiones de sombra umbra y penumbra, y la perspectiva aérea aún valen la pena leerlas en traducción (Kemp, 1989).

Aunque la perspectiva era conocida por los griegos, ellos estaban confundidos por el papel de los ojos en la visión. Aristóteles pensaba en los ojos como dispositivos que emitían rayos, más o menos en la manera de los telémetros láser modernos. Esta visión errónea fue desmentida por el trabajo de científicos árabes, como Alhazen, en el siglo X.

Siguió el desarrollo de varios tipos de cámaras. Estas consistían en habitaciones (*cámara* es latín para "alcoba") donde la luz se dejaba entrar a través de un pequeño agujero en una pared para proyectar una imagen de la escena exterior en la pared opuesta. Por supuesto, en todas estas cámaras la imagen estaba invertida, lo que causó no pocos problemas. Si se pensaba en el ojo como tal dispositivo de imagen, ¿cómo vemos de pie? Este enigma ejercitó a las mentes más grandes de la época (incluido Leonardo). Fue necesario el trabajo de Kepler y Descartes para resolver la cuestión. Descartes colocó un ojo del que se había eliminado la cutícula opaca en un agujero en una contraventana. El resultado fue una imagen invertida formada en un pedazo de papel colocado en la retina. Aunque la imagen retiniana está de hecho invertida, esto no causa un problema porque el cerebro interpreta la imagen de la manera correcta. En jerga moderna, sólo hay que acceder a la estructura de datos de manera apropiada.

Los próximos avances importantes en la comprensión de la visión tuvieron lugar en el siglo XIX. El trabajo de Helmholtz y Wundt, descrito en el Capítulo 1, estableció la experimentación psicofísica como una disciplina científica rigurosa. Los trabajos de Young, Maxwell y Helmholtz establecieron una teoría tricromática de la visión del color. El hecho de que los humanos pueden ver profundidad si las imágenes presentadas a los ojos izquierdo y derecho son ligeramente diferentes fue demostrado por la invención del estereoscopio de Wheatstone (1838). De inmediato el dispositivo se hizo popular en salones sociales y literarios de toda Europa.

El concepto esencial de la estereopsis binocular —donde dos imágenes de una escena tomadas desde puntos de vista ligeramente diferentes llevan información suficiente para obtener una reconstrucción tridimensional de la escena— fue explotado en el campo de la fotogrametría. Se obtuvieron resultados matemáticos clave; por ejemplo, Kruppa (1913) demostró que dadas dos vistas de cinco puntos distintos en una escena, se podía reconstruir la rotación y la traslación entre las dos posiciones de la cámara, así como la profundidad de la escena (hasta un factor de escala).

Aunque la geometría de la estereopsis había sido entendida durante mucho tiempo, el problema de la correspondencia en la fotogrametría solía ser resuelto por humanos tratando de emparejar puntos correspondientes. La asombrosa habilidad de los humanos para resolver el problema de la correspondencia fue ilustrada por los estereogramas de puntos aleatorios de Julesz (1971). El campo de la visión por computadora ha dedicado mucho esfuerzo hacia una solución automática del problema de la correspondencia.

En la primera mitad del siglo xx, los resultados de investigación más significativos en visión fueron obtenidos por la escuela de psicología Gestalt, liderada por Max Wertheimer. Esta escuela señaló la importancia de la organización perceptual: para un humano, la imagen no es una colección de salidas de fotoreceptores puntillistas (pixeles), sino que está organizada en grupos coherentes. La tarea de visión por computadora de encontrar regiones y curvas se remonta a esta idea. Los gestaltistas también llamaron la atención sobre el fenómeno "figura-fondo": un contorno que separa dos regiones de la imagen que en el mundo están a diferentes profundidades parece pertenecer sólo a la región más cercana, la "figura", y no a la región más lejana, el "fondo".

El trabajo gestalt fue continuado por J. J. Gibson (1950, 1979), quien señaló la importancia del flujo óptico y los gradientes de textura en la estimación de variables ambientales como la inclinación y la oblicuidad de la superficie. Enfatizó la importancia del estímulo y lo rico que era. Gibson, Olum y Rosenblatt (1955) señalaron que el campo de flujo óptico contenía suficiente información para determinar el movimiento del observador en relación con el entorno. En particular, Gibson enfatizó el papel del observador activo, cuyo movimiento autodirigido facilita la recogida de información sobre el entorno externo.

La visión por computadora se remonta a la década de 1960. La tesis de Roberts (1963) promulgada en el MIT sobre la percepción de cubos y otros objetos del mundo de los bloques fue una de las primeras publicaciones en el campo. Roberts introdujo varias ideas clave, incluyendo la detección de bordes y la coincidencia basada en modelos.

En las décadas de 1960 y 1970, el progreso fue lento, obstaculizado por la falta de recursos computacionales y de almacenamiento. El procesamiento visual de bajo nivel recibió mucha atención, con técnicas extraídas de campos relacionados como el procesamiento de señales, el reconocimiento de patrones y la agrupación de datos.

La detección de bordes se trató como un primer paso esencial en el procesamiento de imágenes, ya que reducía la cantidad de datos a procesar. La técnica de detección de bordes de Canny, ampliamente utilizada, fue introducida por John Canny (1986). Martin, Fowlkes y Malik (2004) mostraron cómo combinar múltiples pistas, como brillo, textura y color, en un marco de aprendizaje automático para encontrar mejor las curvas de límite.

El problema relacionado de encontrar regiones de brillo coherente, color y textura se presta a formulaciones donde encontrar la mejor partición se convierte en un problema de optimización. Tres ejemplos líderes se basan en campos aleatorios de Markov debido a Geman y Geman (1984), la formulación variacional de Mumford y Shah (1989), y cortes normalizados por Shi y Malik (2000).

Durante gran parte de las décadas de 1960, 1970 y 1980, se presentaron dos paradigmas distintos que buscaban el reconocimiento visual, dictados por diferentes perspectivas sobre lo que

se percibía como el problema principal. La investigación en visión por computadora sobre el reconocimiento de objetos se centró, en gran medida, en los problemas que surgen de la proyección de objetos tridimensionales en imágenes bidimensionales. La idea de alineación, también introducida por primera vez por Roberts, resurgió en la década de 1980 en los trabajos de Lowe (1987) y Huttenlocher y Ullman (1990).

La comunidad de reconocimiento de patrones adoptó un enfoque diferente, considerando los aspectos 3D a 2D del problema como insignificantes. Sus ejemplos motivadores estaban en dominios como el reconocimiento de caracteres ópticos y el reconocimiento de códigos postales escritos a mano, en los cuales la principal preocupación es la de aprender las variaciones típicas características de una clase de objetos y separarlas de otras clases. Podemos rastrear las arquitecturas de redes neuronales para el análisis de imágenes hasta los estudios de Hubel y Wiesel (1962, 1968) del córtex visual en gatos y monos. Desarrollaron un modelo jerárquico de la vía visual con neuronas en áreas inferiores del cerebro (en especial el área llamada V1) respondiendo a características como bordes orientados y barras, y neuronas en áreas superiores respondiendo a estímulos más específicos ("células de la abuela" en la versión de dibujos animados).

Fukushima (1980) propuso una arquitectura de red neuronal para el reconocimiento de patrones motivada por la jerarquía de Hubel y Wiesel. Su modelo tenía capas alternas de células simples y células complejas, que incorporaban muestreo descendente; también tenía invariancia de desplazamiento, al incorporar la estructura convolucional. LeCun *et al.* (1989) dio el paso adicional de usar retropropagación para entrenar los pesos de esta red, y así nacieron lo que hoy llamamos redes neuronales convolucionales. Vea LeCun *et al.* (1995) para una comparación de enfoques.

A partir de finales de la década de 1990, acompañando un papel mucho mayor del modelado probabilístico y el aprendizaje automático estadístico en el campo de la inteligencia artificial en general, surgió un acercamiento entre estas dos tradiciones. Dos líneas de trabajo contribuyeron de manera significativa. Una fue la investigación sobre la detección de rostros (Rowley *et al.,* 1998; Viola y Jones, 2004) que demostró el poder de las técnicas de reconocimiento de patrones en tareas importantes y útiles.

El otro fue el desarrollo de descriptores de puntos, que permiten la construcción de vectores de características a partir de partes de objetos (Schmid y Mohr, 1996). Hay tres estrategias clave para construir un buen descriptor de punto local: uno utiliza orientaciones para obtener invariancia de iluminación; otro necesita describir la estructura de la imagen cerca de un punto en detalle, y más lejos sólo de manera aproximada, un tercero necesita usar histogramas espaciales para suprimir variaciones causadas por pequeños errores en la localización del punto. El descriptor SIFT de Lowe (2004) explotó estas ideas de manera muy efectiva; otra variante popular fue el descriptor HOG debido a Dalal y Triggs (2005).

Las décadas de 1990 y 2000 vieron un debate continuo entre los devotos del diseño inteligente de características como SIFT y HOG y los aficionados a las redes neuronales que creían que las buenas características deberían surgir de manera automática del entrenamiento de extremo a extremo. La forma de resolver tal debate es mediante puntos de referencia en conjuntos de datos estándar, ya en la década de 2000 los resultados en un conjunto de datos estándar de detección de objetos, PASCAL VOC, argumentaron a favor de las características diseñadas a mano. Esto cambió cuando Krizhevsky *et al.* (2013) mostraron que en la tarea de clasificación de imágenes en el conjunto de datos ImageNet, su red neuronal (llamada AlexNet) dio tasas de error de manera significativa más bajas que las técnicas de visión por computadora convencionales.

¿Cuál fue el secreto del éxito de AlexNet? Además de las innovaciones técnicas (como el uso de unidades de activación ReLU) debemos dar mucho crédito a los **grandes datos** y la **gran computación**. Por grandes datos nos referimos a la disponibilidad de grandes conjuntos de datos con etiquetas de categoría, como ImageNet, que proporcionó los datos de entrenamiento para estas grandes redes profundas con millones de parámetros. Los conjuntos de datos anteriores como

Caltech-101 o PASCAL VOC no tenían suficientes datos de entrenamiento, y MNIST y CIFAR eran considerados "conjuntos de datos de juguete" por la comunidad de visión por computadora. Esta línea de etiquetado de conjuntos de datos para el benchmarking y para la extracción de estadísticas de imágenes fue posible gracias al deseo de las personas de subir sus colecciones de fotos a Internet en sitios como Flickr. La forma en que la gran computación resultó más útil fue a través de las GPUs, un desarrollo de hardware impulsado por las necesidades de la industria de los videojuegos.

En uno o dos años, la evidencia fue bastante clara. Por ejemplo, el trabajo de la red neuronal convolucional basada en regiones (RCNN) de Girshick *et al.* (2016) demostró que la arquitectura de AlexNet podría ser modificada, haciendo uso de ideas de visión por computadora como las propuestas de regiones, para hacer posible la detección de objetos de última generación en PASCAL VOC. Nos hemos dado cuenta de que, en general, las redes más profundas funcionan mejor y que los miedos al sobreajuste son exagerados. Tenemos nuevas técnicas como la **normalización de lotes** para lidiar con la regularización.

La reconstrucción de la estructura tridimensional a partir de múltiples vistas tiene sus raíces en la literatura de fotogrametría. En la era de la visión por computadora, Ullman (1979), y Longuet-Higgins (1981) son trabajos tempranos influyentes. Las preocupaciones sobre la estabilidad de la estructura a partir del movimiento fueron aliviadas de manera significativa por el trabajo de Tomasi y Kanade (1992) quienes demostraron que con el uso de múltiples cuadros, y la consiguiente amplia línea de base, la forma podía ser recuperada con bastante precisión.

Una innovación conceptual introducida en la década de 1990 fue el estudio de la estructura proyectiva a partir del movimiento. Aquí no es necesaria la calibración de la cámara, como demostró Faugeras (1992). Este descubrimiento está relacionado con la introducción del uso de invariantes geométricos en el reconocimiento de objetos, como lo estudiaron Mundy y Zisserman (1992), y el desarrollo de la estructura afín a partir del movimiento por Koenderink y Van Doorn (1991).

En la década de 1990, con un gran aumento en la velocidad y almacenamiento de las computadoras y la disponibilidad generalizada de video digital, el análisis de movimiento encontró muchas nuevas aplicaciones. La construcción de modelos geométricos de escenas del mundo real para su renderización mediante técnicas de gráficos por computadora resultó popular, liderada por algoritmos de reconstrucción como el desarrollado por Debevec *et al.* (1996). Los libros de Hartley y Zisserman (2000) y Faugeras *et al.* (2001) proporcionan un tratamiento completo de la geometría de múltiples vistas.

Los humanos pueden percibir la forma y la disposición espacial a partir de una sola imagen, y modelar esto ha demostrado ser un gran desafío para los investigadores de visión por computadora. La inferencia de la forma a partir de la sombra fue estudiada por primera vez por Berthold Horn (1970); Horn y Brooks (1989) presentan una extensa revisión de los principales trabajos de un periodo en el que este era un problema muy estudiado. Gibson (1950) fue el primero en proponer gradientes de textura como una pista para la forma. Las matemáticas de los contornos de oclusión y, más en general, la comprensión de los eventos visuales en la proyección de objetos curvos suaves, se deben en gran medida al trabajo de Koenderink y Van Doorn, que encuentra un tratamiento extenso en *Solid Shape* de Koenderink (1990).

Más reciente, la atención se ha centrado en tratar el problema de la recuperación de forma y superficie a partir de una sola imagen como un problema de inferencia probabilística, donde las pistas geométricas no se modelan de manera explícita, sino que se utilizan de manera implícita en un marco de aprendizaje. Un buen ejemplo es el trabajo de Hoiem *et al.* (2007); recién esto ha sido reelaborado utilizando redes neuronales profundas.

Pasando ahora a las aplicaciones de la visión por computadora para guiar la acción, Dickmanns y Zapp (1987) demostraron por primera vez un automóvil autónomo conduciendo en autopistas a altas velocidades; Pomerleau (1993) logró un rendimiento similar utilizando un enfoque de red

neuronal. Hoy en día la construcción de coches autónomos es un gran negocio, con las empresas de automóviles establecidas compitiendo con nuevos entrantes como Baidu, Cruise, Didi, Google Waymo, Lyft, Mobileye, Nuro, Nvidia, Samsung, Tata, Tesla, Uber y Voyage para comercializar sistemas que ofrecen diversas capacidades, que van desde la asistencia al conductor hasta la autonomía total.

Para el lector interesado en la visión humana, *Vision Science: Photons to Phenomenology* de Stephen Palmer (1999) proporciona el mejor tratamiento integral a este tema; *Visual Perception: Physiology, Psychology and Ecology* de Vicki Bruce, Patrick Green y Mark Georgeson (2003) es un libro de texto más corto. Los libros *Eye, Brain and Vision* de David Hubel (1988) y *Perception* de Irvin Rock (1984) son introducciones amigables centradas en la neurofisiología y la percepción respectivamente. El libro *Vision* de David Marr (Marr, 1982) tuvo un papel histórico en la conexión de la visión por computadora con las áreas tradicionales de la visión biológica: psicofísica y neurobiología. Aunque muchos de sus modelos específicos para tareas como la detección de bordes y el reconocimiento de objetos no han resistido la prueba del tiempo, la perspectiva teórica donde cada tarea se analiza a un nivel informativo, computacional y de implementación sigue siendo esclarecedora.

Para el campo de la visión por computadora, los libros de texto más completos disponibles hoy en día son *Computer Vision: A Modern Approach* (Forsyth y Ponce, 2002) y Computer Vision: Algorithms and Applications (Szeliski, 2011). Los problemas geométricos en la visión por computadora se tratan a fondo en *Multiple View Geometry in Computer Vision* (Hartley y Zisserman, 2000). Estos libros se escribieron antes de la revolución del aprendizaje profundo, por lo que para los últimos resultados, consulte la literatura primaria.

Dos de las principales revistas de visión por computadora son *IEEE Transactions on Pattern Analysis and Machine Intelligence* e *International Journal of Computer Vision*. Las conferencias de visión por computadora incluyen ICCV (International Conference on Computer Vision), CVPR (Computer Vision and Pattern Recognition) y ECCV (European Conference on Computer Vision). La investigación con un componente significativo de aprendizaje automático también se publica en NeurIPS (Neural Information Processing Systems), y el trabajo en la interfaz con gráficos por computadora con frecuencia aparece en la conferencia ACM SIGGRAPH (Special Interest Group in Graphics). Múltiples documentos de visión aparecen como preimpresiones en el servidor arXiv, y los primeros informes de nuevos resultados aparecen en blogs de los principales laboratorios de investigación.

CAPÍTULO 26

ROBÓTICA

En el que los agentes están dotados de sensores y actuadores físicos con los que se mueven y realizan acciones en el mundo real.

26.1 Robots

Los **robots** son agentes físicos que realizan tareas manipulando el mundo físico; para hacerlo, están equipados con **actuadores** como piernas, ruedas, articulaciones y pinzas. Los actuadores están diseñados para aplicar fuerzas físicas en el entorno; cuando lo hacen, suceden algunas cosas: el estado del robot cambia (por ejemplo, un vehículo gira sus ruedas y avanza en el camino como resultado), el estado del entorno cambia (como cuando un brazo de robot usa su pinza para empujar una taza por el mostrador) e incluso el estado de las personas alrededor del robot cambia (cuando un exoesqueleto se mueve cambia la configuración de la pierna de una persona o cuando un robot móvil avanza hacia las puertas del ascensor y una persona, lo suficientemente amable, se da cuenta que debe moverse fuera del camino o presionar el botón para el robot).

Robots
Actuadores

Los robots también están equipados con **sensores** que les permiten percibir su entorno. La robótica actual emplea un conjunto diverso de sensores (como cámaras, radares, láseres y micrófonos para medir el estado del entorno y de las personas que lo rodean) y giroscopios, sensores de tensión y par, así como acelerómetros para medir el estado propio del robot.

Sensor

Maximizar la utilidad esperada para un robot significa elegir cómo activar sus actuadores para aplicar las fuerzas físicas *correctas*, las que llevarán a cambios en el estado que acumulen la mayor recompensa posible. En última instancia, los robots están tratando de realizar alguna tarea en el mundo físico.

Los robots operan en entornos parcialmente observables y estocásticos: las cámaras no permiten ver alrededor de las esquinas y los engranajes pueden resbalar. Además, las personas que actúan en ese mismo entorno son impredecibles, por lo que el robot necesita hacer predicciones sobre ellas.

Los robots suelen modelar su entorno con un espacio de estado continuo (la posición del robot tiene coordenadas continuas) y un espacio de acción continuo (la cantidad de corriente que un robot envía a su motor también se mide en unidades continuas). Algunos robots operan en espacios de alta dimensión: los vehículos necesitan conocer su posición, orientación y velocidad, así como la de los agentes cercanos; los brazos de los robots tienen seis o siete articulaciones que permiten moverse de manera independiente; además, los robots que imitan el cuerpo humano tienen cientos de articulaciones.

El aprendizaje robótico está limitado porque el mundo real se niega a operar más rápido que en tiempo real. En un entorno simulado, es posible utilizar algoritmos de aprendizaje (como el algoritmo Q-learning descrito en el Capítulo 22) para aprender en pocas horas a partir de millones de pruebas. En un entorno real, podría llevar años realizar estas pruebas, pero el robot no puede arriesgar (y, por lo tanto, no es posible aprender de) una prueba que podría causar daño. Por lo

(a)

(b)

Figura 26.1 (a) Brazo robótico industrial con un actuador personalizado. (Crédito de la imagen: Macor/123RF). (b) Brazo de Robot Asistente Kinova® JACO® montado en silla de ruedas. (Kinova y JACO son marcas registradas de Kinova, Inc.).

tanto, transferir lo aprendido en la simulación a un robot real en el mundo real —el problema de la **simulación a la realidad**—, es una área activa de investigación. Los sistemas robóticos prácticos necesitan incorporar conocimientos previos sobre el robot, el entorno físico y las tareas a realizar para que el robot pueda aprender rápido y actuar de forma segura.

La robótica reúne muchos de los conceptos que hemos visto en este libro, incluyendo la estimación de estado probabilística, la percepción, la planificación, el aprendizaje no supervisado, el aprendizaje por refuerzo y la teoría de juegos. Para algunos de estos conceptos, la robótica sirve como un ejemplo de aplicación desafiante. Para otros conceptos, este capítulo inaugura nuevos terrenos, por ejemplo, introducir la versión continua de técnicas que antes sólo vimos en el caso discreto.

26.2 Hardware del robot

Hasta ahora en este libro hemos tomado la arquitectura del agente, sensores, actuadores y procesadores como dada y nos hemos concentrado en el programa del agente. Pero el éxito de los robots reales depende, al menos, del diseño de sensores y actuadores que son apropiados para la tarea.

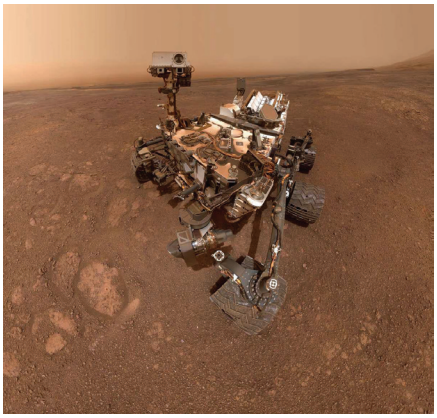
26.2.1 Tipos de robots desde la perspectiva del hardware

Robot antropomórfico

Cuando piensa en un robot, podría imaginar algo con una cabeza y dos brazos moviéndose sobre patas o ruedas. Tales **robots antropomórficos** han sido popularizados en las historias de ficción, como la película *Terminator* y en los dibujos animados *Los Supersónicos*. Pero los robots reales vienen en muchas formas y tamaños.

Manipulador

Los **manipuladores** son brazos de robot. No necesariamente tienen que estar unidos a un cuerpo de robot; podrían estar atornillados a una mesa o al suelo, como lo están en las fábricas (Figura 26.1 (a)). Algunos robots tienen gran capacidad de carga, como los que ensamblan vehículos; mientras que otros, como los brazos montables en sillas de ruedas que asisten a personas con discapacidades motoras (Figura 26.1(b)), cargan menos, pero son más seguros en entornos humanos.



(a)

(b)

Figura 26.2 (a) El Rover Curiosity de la NASA tomando una selfie en Marte. (Imagen cortesía de la NASA). (b) Dron Skydio acompañando a una familia en un paseo en bicicleta. (Imagen cortesía de Skydio).

Los **robots móviles** son aquellos que utilizan ruedas, patas o rotores para moverse por el entorno. Los **drones cuadricópteros** son un tipo de **vehículo aéreo no tripulado** (**UAV**, por sus siglas en inglés); **vehículo autónomo bajo el agua** (**AUV**, por sus siglas en inglés) recorren los océanos. Pero muchos robots móviles se quedan en interiores y se mueven sobre ruedas, como una aspiradora o un robot de entrega de toallas en un hotel. Sus contrapartes al aire libre incluyen **vehículos autónomos** o **rovers** (**exploradores**) que examinan nuevos terrenos, incluso en la superficie de Marte (Figura 26.2). Por último, los **robots con patas** están destinados a atravesar terrenos difíciles, inaccesibles con ruedas; su inconveniente es que controlar las patas para que un traslado resulte exitoso es más desafiante que girar las ruedas.

Robot móvil
Dron cuadricóptero
UAV
AUV
Vehículo autónomo
Rover
Robot con patas

Otros tipos de robots incluyen prótesis, exoesqueletos, robots con alas, enjambres y entornos inteligentes en los que el robot es toda la habitación.

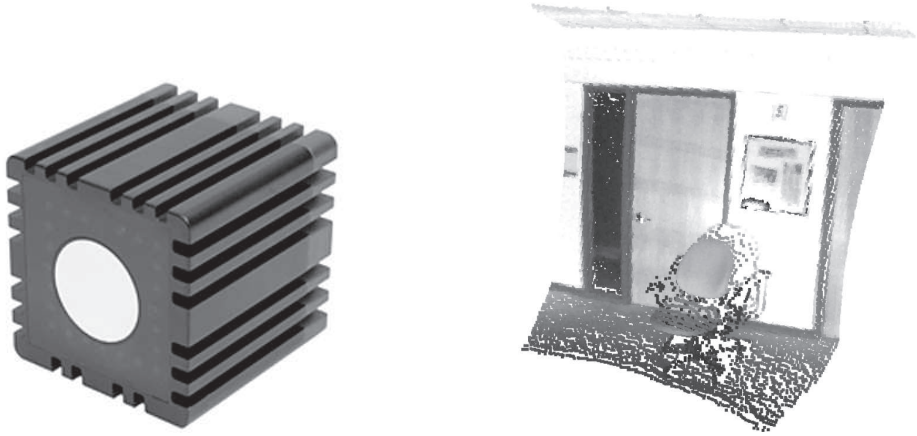
26.2.2 Sintiendo el mundo

Los sensores son la interfaz perceptual entre el robot y el entorno. Los **sensores pasivos**, como cámaras, son observadores del entorno: capturan señales que son generadas por otras fuentes en el entorno. Los **sensores activos**, como el sonar, envían energía al entorno. Se basan en el hecho de que esta energía se refleja de vuelta al sensor. Los sensores activos suelen proporcionar más información que los sensores pasivos, pero a costa de un mayor consumo de energía y con el peligro de interferencias cuando se utilizan varios sensores activos al mismo tiempo. También distinguimos si un sensor está dirigido a detectar el entorno, la ubicación del robot o la configuración interna de este.

Sensor pasivo
Sensor activo

Los **medidores de distancia** son sensores que miden la distancia a los objetos cercanos. Los sensores de **sonar** son detectores de rango activos que emiten ondas de sonido direccionales, las cuales son reflejadas por los objetos con parte del sonido volviendo al sensor (sonar). El tiempo y la intensidad de la señal de retorno indican la distancia a los objetos cercanos. El sonar es la tecnología de elección para los vehículos submarinos autónomos, fue popular en los primeros días de la robótica de interiores. **Visión estéreo** (ver Sección 25.6) se basa en múltiples cámaras para

Medidor de distancia
Sonar
Visión estéreo



(a) (b)

Figura 26.3 (a) Cámara de tiempo de vuelo. (Imagen cortesía de Mesa Imaging GmbH). (b) Imagen de rango 3D obtenida con esta cámara. La imagen de rango permite detectar obstáculos y objetos en las cercanías de un robot. (Imagen cortesía de Willow Garage, LLC.).

capturar el entorno desde puntos de vista ligeramente diferentes, analizando el paralaje resultante en estas imágenes para calcular el rango de los objetos circundantes.

Para los robots móviles terrestres, el sonar y la visión estéreo rara vez se utilizan, porque su precisión no es del todo fiable. El Kinect es un sensor de uso popular y de bajo costo que combina una cámara y un proyector de **luz estructurada**, que proyecta un patrón de líneas de cuadrícula en una escena. La cámara ve cómo se doblan las líneas de la cuadrícula, lo que da al robot información sobre la forma de los objetos en la escena. Si se desea, la proyección puede ser luz infrarroja, para no interferir con otros sensores (como los ojos humanos).

Luz estructurada

La mayoría de los robots terrestres ahora están equipados con telémetros ópticos activos. Al igual que los sensores de sonar, los sensores de rango óptico emiten señales activas (luz) y miden el tiempo que toma a una reflexión de esta señal regresar al sensor. La Figura 26.3(a) muestra una **cámara de tiempo de vuelo**. Esta cámara logra imágenes de rango como la que se muestra en la Figura 26.3(b) de hasta 60 cuadros por segundo. Los vehículos autónomos con frecuencia utilizan **LiDARs de escaneo** (abreviatura de *detección y rango de luz*), sensores de actividad que emiten haces de láser y detectan el haz reflejado, lo que arroja mediciones de rango precisas a un centímetro en un rango de 100 metros. Utilizan arreglos complejos de espejos o elementos rotativos para barrer el haz a través del entorno y construir un mapa. Los LiDARs de escaneo suelen funcionar mejor que las cámaras de tiempo de vuelo a distancias más largas y en luz diurna brillante.

Cámara de tiempo de vuelo

LiDAR de escaneo

El **radar** es, en la mayoría de los casos, el sensor de detección de rango de elección para los vehículos aéreos autónomos o no autónomos. Los sensores de radar permiten medir distancias de kilómetros y tienen como ventaja sobre los sensores ópticos que ven a través de la niebla. En el extremo cercano de la detección de rango están los **sensores táctiles** como las antenas, los paneles de golpe, y la piel sensible al tacto. Estos sensores miden el rango basado en el contacto físico y sólo permiten ser desplegados para detectar objetos muy cerca del robot.

Radar

Sensor táctil

Una segunda clase importante son los **sensores de ubicación**. La mayoría de estos sensores utilizan la detección de rango como un componente principal para determinar la ubicación. Al aire libre, el **Sistema de Posicionamiento Global** (GPS, por sus siglas en inglés) es la solución más común al

Sensor de ubicación

Sistema de Posicionamiento Global (GPS)

problema de la localización. El GPS mide la distancia hasta satélites que emiten señales pulsadas. En la actualidad, hay 31 satélites GPS operativos en órbita y 24 satélites GLONASS, el equivalente ruso. Los receptores GPS permiten recuperar la distancia a un satélite analizando los cambios de fase. Al triangular las señales de varios satélites, los receptores GPS pueden determinar su ubicación absoluta en la Tierra con una precisión de apenas unos metros. El **GPS diferencial** implica un segundo receptor terrestre con ubicación conocida que, en condiciones ideales, proporciona una precisión de milímetros.

GPS diferencial

Sin embargo, el GPS no funciona en interiores o bajo el agua. En interiores, la localización con frecuencia se logra colocando balizas en el entorno en ubicaciones conocidas. Múltiples entornos interiores están llenos de estaciones base inalámbricas que ayudan a los robots a localizarse mediante el análisis de la señal inalámbrica. Bajo el agua, las balizas de sonar activas proporcionan una sensación de ubicación pues utilizan el sonido para informar a los AUV de sus distancias relativas a esas balizas.

La tercera clase importante son los **sensores propioceptivos**, que informan al robot de su propio movimiento. Para medir la configuración exacta de una articulación robótica, los motores suelen estar equipados con **decodificadores de eje** que miden con precisión el movimiento angular de un eje. En los brazos de los robots, los decodificadores ayudan a rastrear la posición de las articulaciones. En los robots móviles, los decodificadores de eje informan sobre las revoluciones de las ruedas para **odometría,** la medición de la distancia recorrida. Sin embargo, las ruedas tienden a derrapar y resbalar, por lo cual la odometría sólo es precisa en distancias cortas. Las fuerzas externas, como el viento y las corrientes oceánicas, aumentan la incertidumbre posicional. Los **sensores inerciales**, como los giroscopios, reducen la incertidumbre al confiar en la resistencia de la masa al cambio de velocidad.

Sensores propioceptivos

Decodificadores de eje

Odometría

Sensor inercial

Otros aspectos importantes del estado del robot son medidos por **sensores de fuerza** y **sensores de par**. Estos son indispensables cuando los robots manejan objetos frágiles o cuyo tamaño y forma exactos son desconocidos. Imagine un manipulador robótico de una tonelada atornillando una frágil bombilla. Sería muy fácil aplicar demasiada fuerza y romper la bombilla. Los sensores de fuerza permiten al robot sentir cuánta fuerza está aplicando al agarrar la bombilla, y los sensores de par le permiten sentir cuánta fuerza está aplicando al girarla. Los sensores de alta calidad permiten medir fuerzas en todas las direcciones: tres de traslación y tres de rotación. Hacen esto a una frecuencia de varios cientos de veces por segundo para que un robot pueda detectar de manera rápida fuerzas inesperadas y corregir sus acciones antes de que rompa una bombilla. Sin embargo, puede ser un desafío equipar a un robot con sensores de alta gama y la potencia computacional para monitorearlos.

Sensor de fuerza

Sensor de par

26.2.3 Produciendo movimiento

El mecanismo que inicia una acción se llama **actuador**; ejemplos incluyen transmisiones, engranajes, cables y enlaces. El tipo de actuador más común es el **actuador eléctrico**, que utiliza electricidad para hacer girar un motor. Estos se utilizan en sistemas que necesitan movimiento rotacional, como las articulaciones de un brazo de robot. Los **actuadores hidráulicos** utilizan fluido hidráulico presurizado (como aceite o agua) y los **actuadores neumáticos** utilizan aire comprimido para generar movimiento mecánico.

Actuador

Actuador hidráulico

Actuador neumático

Los actuadores se utilizan con frecuencia para mover las articulaciones que conectan cuerpos rígidos (eslabones). Los brazos y las piernas tienen articulaciones. En las **articulaciones revolutas**, un eslabón rota con respecto al otro. En las **articulaciones prismáticas**, un eslabón se desliza a lo largo del otro. Ambos son articulaciones de un solo eje (un eje de movimiento). Otros tipos de articulaciones incluyen las articulaciones esféricas, cilíndricas y planas, que son articulaciones de múltiples ejes.

Articulación revoluta

Articulación prismática

Para interactuar con objetos en el entorno, los robots utilizan agarres. El tipo más básico de agarre es el **agarre de mandíbula paralela**, con dos dedos y un solo actuador que mueve los dedos en conjunto para agarrar objetos. Este actuador es tanto amado como odiado por su

Agarre de mandíbula paralela

simplicidad. Los agarres de tres dedos ofrecen un poco más de flexibilidad manteniendo la simplicidad. En el otro extremo del espectro están las manos humanoides (antropomórficas). Por ejemplo, la mano Dexterous Shadow tiene un total de 20 actuadores, lo que ofrece mucha más flexibilidad para la manipulación compleja, incluidas las maniobras de manipulador en mano (piensa recoger su teléfono móvil y rotarlo en la mano para orientarlo de manera correcta), pero esta flexibilidad tiene un precio: aprender a controlar estos agarres complejos es más desafiante.

26.3 ¿Qué tipo de problema resuelve la robótica?

Ahora que sabemos qué podría ser el hardware del robot, estamos listos para considerar el software del agente que impulsa el hardware para alcanzar nuestros objetivos. Primero necesitamos decidir el marco computacional para este agente. Hemos hablado de la búsqueda en entornos deterministas, MDP para entornos estocásticos pero completamente observables, POMDP para observabilidad parcial y juegos para situaciones en las que el agente no actúa en aislamiento. Dado un marco computacional, necesitamos instanciar sus ingredientes: funciones de recompensa o utilidad, estados, acciones, espacios de observación, etc.

Ya hemos señalado que los problemas de robótica son no deterministas, parcialmente observables y multiagente. Utilizando las nociones de teoría de juegos del Capítulo 18, podemos ver que unas veces los agentes son cooperativos y otras competitivos. En un pasillo estrecho por donde sólo un agente puede ir primero, un robot y una persona colaboran porque ambos quieren asegurarse de no chocar entre sí. Pero en algunos casos podrían competir un poco para llegar rápido a su destino. Si el robot es demasiado educado y siempre hace espacio, podría quedarse atascado en situaciones de mucha gente y nunca llegar a su objetivo.

Por lo tanto, cuando los robots actúan en aislamiento y conocen su entorno, el problema que están resolviendo permite formularse como un MDP o cuando les falta información se convierte en un POMDP, pero cuando actúan alrededor de personas en la mayoría de los casos se formulan como un juego.

¿Cuál es la función de recompensa del robot en esta formulación? Por lo regular, el robot actúa en servicio de un humano, por ejemplo, entregando comida a un paciente de hospital para la recompensa del paciente, no la suya. Para la mayoría de los entornos de robótica, aun cuando los diseñadores de robots intenten especificar una función de recompensa proxy lo suficientemente buena, la verdadera función de recompensa reside en el usuario al que se supone que el robot debe ayudar. El robot necesitará descifrar los deseos del usuario, o confiar en un ingeniero para especificar una aproximación de los deseos del usuario.

En cuanto a la acción, estado y espacios de observación del robot, la forma más general es que las observaciones consistan en la alimentación de sensores crudos (por ejemplo, las imágenes que llegan de las cámaras o los impactos de láser que llegan del LiDAR). Las acciones consisten en corrientes eléctricas crudas que son enviadas a los motores, mientras que el estado es lo que el robot necesita saber para su toma de decisiones. Esto significa que hay una gran brecha entre los perceptos y controles de bajo nivel, y los planes de alto nivel que el robot necesita hacer. Para cerrar la brecha, los roboticistas desacoplan aspectos del problema para simplificarlo.

Por ejemplo, sabemos que cuando resolvemos POMDP con corrección, la percepción y la acción interactúan: la percepción informa qué acciones tienen sentido, pero la acción también informa la percepción, con agentes tomando acciones para recopilar información cuando esa información tiene valor en pasos posteriores. Sin embargo, los robots con frecuencia separan la percepción de la acción, al consumir las salidas de la percepción y fingir que no obtendrán más información en el futuro. Además, se requiere una planificación jerárquica, porque un objetivo de alto nivel como "llegar a la cafetería" está muy alejado de un comando de motor como "girar el eje principal 1°",

En robótica se usa una jerarquía de tres niveles. El nivel de **planificación de tareas** decide un plan o política para acciones de alto nivel, a veces llamadas primitivas de acción o subobjetivos: moverse a la puerta, abrirla, ir al ascensor, presionar el botón, etc. Luego, la **planificación del movimiento** se encarga de encontrar un camino que lleve al robot de un punto a otro, logrando cada subobjetivo. Finalmente, el **control** se utiliza para lograr el movimiento planificado utilizando los actuadores del robot. Dado que la tarea de nivel de planificación se define por lo regular sobre estados y acciones discretos, en este capítulo nos centraremos en la planificación del movimiento y el control.

Planificación de tareas

Control

Por separado, el **aprendizaje de preferencias** se encarga de estimar el objetivo de un usuario final, mientras que la **predicción de personas** se utiliza para pronosticar las acciones de otras personas en el entorno del robot. Todas estas se combinan para determinar el comportamiento del robot.

Aprendizaje de preferencias

Predicción de personas

Cada vez que dividimos un problema en piezas separadas reducimos la complejidad, pero renunciamos a las oportunidades para que las piezas se ayuden entre sí. La acción ayuda a mejorar la percepción y a determinar qué tipo de percepción es útil. Del mismo modo, las decisiones a nivel de movimiento pueden no ser las mejores al tener en cuenta cómo se rastreará ese movimiento o las decisiones a nivel de tarea pueden hacer que el plan de la tarea sea no instanciable a nivel de movimiento. Por lo tanto, con el progreso en estas áreas separadas viene el impulso para reintegrarlas: ejecutar la planificación del movimiento y el control juntos, realizar la planificación de la tarea y el movimiento juntos y reintegrar la percepción, la predicción y la acción, cerrando el bucle de retroalimentación. Hoy en día, la robótica debe continuar progresando en cada área, mientras se construye sobre este progreso para lograr una mejor integración.

26.4 Percepción robótica

La percepción es el proceso por el cual los robots mapean las mediciones de los sensores en representaciones internas del entorno. Gran parte de este proceso utiliza las técnicas de visión por computadora del capítulo anterior. Pero la percepción para la robótica debe tratar con sensores adicionales como LiDAR y sensores táctiles.

La percepción es difícil porque los sensores son ruidosos y el entorno es parcialmente observable, impredecible y dinámico. En otras palabras, los robots tienen todos los problemas de **estimación de estado** (o **filtrado**) que discutimos en la Sección 14.2. Por regla general, las buenas representaciones internas para los robots tienen tres propiedades:

1. Contienen suficiente información para que el robot tome buenas decisiones.
2. Están estructurados de manera que se actualizan de modo eficiente.
3. Son naturales en el sentido de que las variables internas corresponden a variables de estado naturales en el mundo físico.

En el Capítulo 14, vimos que los filtros de Kalman, los HMM y las redes bayesianas dinámicas permiten representar los modelos de transición y sensor de un entorno parcialmente observable, y describimos algoritmos exactos y aproximados para actualizar el **estado de creencia**: la distribución de probabilidad posterior sobre las variables de estado del entorno. Varios modelos de redes bayesianas dinámicas para este proceso se mostraron en el Capítulo 14. Para problemas de robótica, incluimos las propias acciones pasadas del robot como variables observadas en el modelo. La Figura 26.4 muestra la notación utilizada en este capítulo: $\boldsymbol{X}_t$ es el estado del entorno (incluyendo el robot) en el tiempo t, $\boldsymbol{Z}_t$ es la observación recibida en el tiempo t, y A_t es la acción tomada después de que se recibe la observación.

Nos gustaría calcular el nuevo estado de creencia, $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{z}_{1:t+1}, a_{1:t})$, a partir del estado de creencia actual, $\mathbf{P}(\mathbf{X}_t|\mathbf{z}_{1:t}, a_{1:t-1})$, y la nueva observación $\mathbf{z}_{t+1}$. Hicimos esto en la Sección 14.2, pero hay

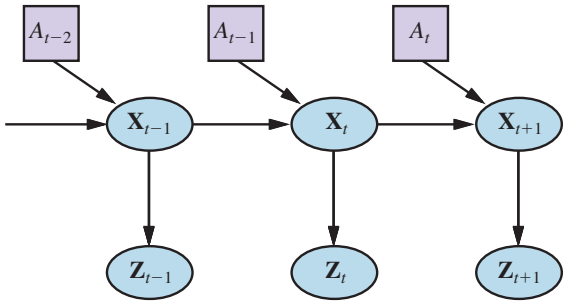


Figura 26.4 La percepción del robot puede ser vista como inferencia temporal de secuencias de acciones y mediciones, como se ilustra en esta red de decisión dinámica.

dos diferencias aquí: condicionamos en las acciones así como en las observaciones, y tratamos con variables *continuas* en lugar de *discretas*. Así, modificamos la ecuación de filtrado recursivo (14.5 en la página 467) para usar la integración en lugar de la suma:

$$\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{z}_{1:t+1}, a_{1:t}) = \alpha \mathbf{P}(\mathbf{z}_{t+1}|\mathbf{X}_{t+1}) \int \mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t, a_t) P(\mathbf{x}_t|\mathbf{z}_{1:t}, a_{1:t-1}) d\mathbf{x}_t. \quad (26.1)$$

Esta ecuación establece que el posterior sobre las variables de estado $\mathbf{X}$ en el tiempo $t + 1$ se calcula de manera recursiva a partir de la estimación correspondiente un paso de tiempo antes. Este cálculo involucra la acción anterior a_t y la medición del sensor actual $\mathbf{z}_{t+1}$. Por ejemplo, si nuestro objetivo es desarrollar un robot jugador de futbol, $\mathbf{X}_{t+1}$ podría incluir la ubicación del balón de futbol en relación con el robot. El posterior $\mathbf{P}(\mathbf{X}_t|\mathbf{z}_{1:t}, a_{1:t-1})$ es una distribución de probabilidad sobre todos los estados que captura lo que sabemos a partir de las mediciones de los sensores pasados y los controles. La ecuación (26.1) nos dice cómo estimar de manera recursiva esta ubicación, mediante la incorporación incremental de las mediciones de los sensores (por ejemplo, imágenes de la cámara) y los comandos de movimiento del robot. La probabilidad $\mathbf{P}(\mathbf{X}_{t+1}|\mathbf{x}_t, a_t)$ se llama el **modelo de transición** o **modelo de movimiento**, y $\mathbf{P}(\mathbf{z}_{t+1}|\mathbf{X}_{t+1})$ es el **modelo de sensor**.

Modelo de movimiento

26.4.1 Localización y mapeo

Localización

La **localización** es el problema de averiguar dónde están las cosas, incluido el propio robot. Para mantener las cosas simples, piense en un robot móvil que se mueve con lentitud en un mundo bidimensional plano. También suponga que al robot se le da un mapa exacto del entorno. (Un ejemplo de tal mapa aparece en la Figura 26.7). La pose de un robot móvil de este tipo está definida por sus dos coordenadas cartesianas con valores x y y así como su rumbo con valor θ, como se ilustra en Figura 26.5(a). Si organizamos esos tres valores en un vector, entonces cualquier estado particular está dado por $\mathbf{X}_t = (x_t, y_t, \theta_t)^\top$. Hasta ahora todo bien.

En la aproximación cinemática, cada acción consiste en la especificación "instantánea" de dos velocidades: una velocidad de traslación v_t y una velocidad de rotación ω_t. Para intervalos de tiempo pequeños Δt, un modelo determinista crudo del movimiento de tales robots es dado por

$$\hat{\mathbf{X}}_{t+1} = f(\mathbf{X}_t, \underbrace{v_t, \omega_t}_{a_t}) = \mathbf{X}_t + \begin{pmatrix} v_t \Delta t \cos\theta_t \\ v_t \Delta t \sin\theta_t \\ \omega_t \Delta t \end{pmatrix}.$$

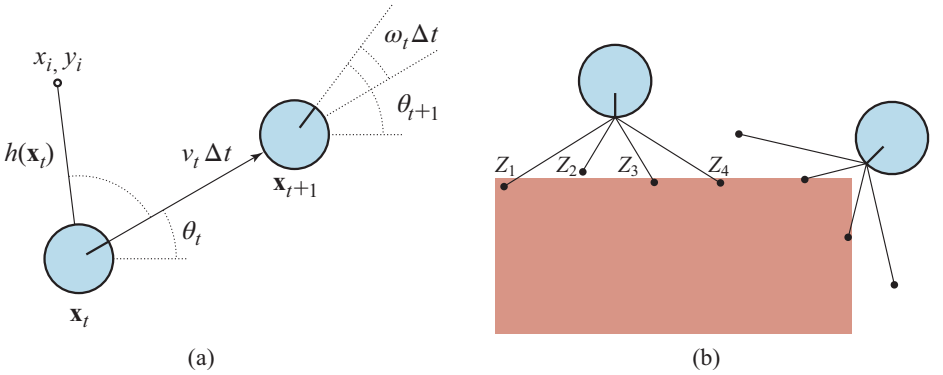


Figura 26.5 (a) Modelo cinemático simplificado de un robot móvil. El robot se muestra como un círculo con una línea de radio interior que marca la dirección hacia adelante. El estado $\mathbf{x}_t$ consiste en la posición (x_t, y_t) (mostrada de manera implícita) y la orientación θ_t. El nuevo estado $\mathbf{x}_{t+1}$ se obtiene mediante una actualización en la posición de $v_t\Delta_t$ y en la orientación de $\omega_t\Delta_t$. También se muestra un hito en (x_i, y_i) observado en el tiempo t. (b) El modelo de sensor de escaneo de rango. Se muestran dos posibles poses de robot para un escaneo de rango dado (z_1, z_2, z_3, z_4). Es mucho más probable que la pose de la izquierda generará el escaneo de rango que la pose de la derecha.

La notación $\hat{\mathbf{X}}$ se refiere a una predicción de estado determinista. Por supuesto, los robots físicos son algo impredecible. Esto se modela con una distribución gaussiana con media $f(\mathbf{X}_t, v_t, \omega_t)$ y covarianza Σ_x. (Vea el Apéndice A para una definición matemática).

$$\mathbf{P}(\mathbf{X}_{t+1} | \mathbf{X}_t, v_t, \omega_t) = \mathcal{N}(\hat{\mathbf{X}}_{t+1}, \Sigma_x).$$

Esta distribución de probabilidad es el modelo de movimiento del robot. Modela los efectos del movimiento a_t en la ubicación del robot.

A continuación, necesitamos un modelo de sensor. Consideraremos dos tipos de modelos de sensores. El primero asume que los sensores detectan características *estables* y *reconocibles* del entorno llamadas **tierras-marcas**. Para cada hito, se informa el rango y el rodamiento. Suponga que el estado del robot es $\mathbf{x}_t = (x_t, y_t, \theta_t)^\top$ y detecta un hito cuya ubicación se sabe que es $(x_i, y_i)^\top$. Sin ruido, una predicción del rango y el rodamiento se calcula mediante geometría simple (ver Figura 26.5(a)):

Tierras-marcas

$$\hat{\mathbf{Z}}_t = h(\mathbf{x}_t) = \begin{pmatrix} \sqrt{(x_t - x_i)^2 + (y_t - y_i)^2} \\ \arctan \frac{y_i - y_t}{x_i - x_t} - \theta_t \end{pmatrix}.$$

De nuevo, el ruido distorsiona nuestras mediciones. Para mantener las cosas sencillas, asumamos ruido gaussiano con covarianza Σ_z, lo que nos da el modelo de sensor

$$P(\mathbf{z}_t | \mathbf{x}_t) = \mathcal{N}(\hat{\mathbf{Z}}_t, \boldsymbol{\Sigma}_z).$$

Se utiliza un modelo de sensor algo diferente para una **matriz de sensores** de rango, cada uno tiene un rodamiento fijo en relación con el robot. Tales sensores producen un vector de valores de rango $\mathbf{z}_t = (z_1, \ldots, z_M)^\top$.

Matriz de sensores

Dada una pose $\mathbf{x}_t$, dejemos que $\hat{z}_j$ sea el rango calculado a lo largo de la dirección del haz j desde $\mathbf{x}_t$ hasta el obstáculo más cercano. Como antes, esto se verá afectado por el ruido gaussiano.

```
funtion MONTE-CARLO-LOCALIZACIÓNa, z, N, P(X'|X,v,ω), P(z|z*), mapa
  returns un conjunto de muestras, S, para el siguiente paso
  inputs: a, velocidades del robot v y ω
          z, un vector de M puntos de datos de escaneo de rango
          P(X'|X,v,ω), modelo de movimiento
          P(z|z*), un modelo de ruido de sensor de rango
          map, un mapa 2D del entorno
  persistent: S, un vector de N muestras
  local variables: W, un vector de N pesos
                   S', un vector temporal de N muestras

  if S está vacío then
    for i = 1 hasta N do          //initialization phase
      S[i] ← muestra de P(X_0)
  for i = 1 hasta N do            // update cycle
    S'[i] ← muestra de P(X'|X = S[i],v,ω)
    W[i] ← 1
    for j = 1 hasta M do
      z* ← RAYCAST(j,X = S'[i],map)
      W[i] ←W[i]·P(z_j|z*)
    S ← WEIGHTED-SAMPLE-WITH-REPLACEMENT(N, S', W)
  retur S
```

Figura 26.6 Algoritmo de localización de Monte Carlo utilizando un modelo de sensor de escaneo de rango con ruido independiente.

Con frecuencia, asumimos que los errores para las diferentes direcciones del haz son independientes e idénticamente distribuidos, por lo que tenemos

$$P(\mathbf{z}_t|\mathbf{x}_t) = \alpha \prod_{j=1}^{M} e^{-(zj - \hat{z}j)/2\sigma^2)}$$

La Figura 26.5(b) muestra un ejemplo de un escaneo de rango de cuatro haces y dos posibles poses de robot, una de las cuales es probable que haya producido el escaneo observado y una que no. Al comparar el modelo de escaneo de rango con el modelo de hito, vemos que el modelo de escaneo de rango tiene la ventaja de que no es necesario *identificar* un hito antes de que se pueda interpretar el escaneo de rango; de hecho, en la Figura 26.5(b), el robot se enfrenta a una pared sin características. Por otro lado, si hay hitos visibles e identificables, proporcionan localización instantánea.

La Sección 14.4 describió el filtro de Kalman, que representa el estado de creencia como un solo gaussiano multivariado, y el filtro de partículas, que representa el estado de creencia mediante una colección de partículas que corresponden a estados. La mayoría de los algoritmos de localización modernos utilizan una de estas dos representaciones de la creencia del robot $\mathbf{P}(\mathbf{X}_t|\mathbf{z}_{1:t}, a_{1:t-1})$.

Localización de Monte Carlo

La localización utilizando el filtrado de partículas se llama **localización de Monte Carlo**, o MCL (por sus siglas en inglés). El algoritmo MCL es una instancia del algoritmo de filtrado de partículas de la Figura 14.17 (página 492). Todo lo que necesitamos hacer es proporcionar el modelo de movimiento y el modelo de sensor apropiados. La Figura 26.6 muestra una versión utilizando el modelo de sensor de escaneo de rango. La operación del algoritmo se ilustra en la Figura 26.7, en la manera en que el robot descubre su localización dentro de un edificio de oficinas. En la primera imagen, las partículas se distribuyen de manera uniforme en función del prior, indicando

incertidumbre global sobre la posición del robot. En la segunda imagen, llega el primer conjunto de mediciones y las partículas forman grupos en las áreas de alta creencia posterior. En el tercero, hay suficientes mediciones disponibles para empujar todas las partículas a una sola ubicación.

El filtro de Kalman es la otra forma importante para localizar. Un filtro de Kalman representa el posterior $\mathbf{P}(\mathbf{X}_t|\mathbf{z}_{1:t},a_{1:t-1})$ por un gaussiano. La media de este gaussiano se denotará μ_t y su covarianza Σ_t. El principal problema con las creencias gaussianas es que sólo están cerradas bajo modelos de movimiento lineal f y modelos de medición lineal h. Para f o h no lineales, el resultado de la actualización de un filtro no es en general gaussiana. Por lo tanto, los algoritmos de localización que utilizan el filtro de Kalman **linealizan** los modelos de movimiento y sensor. La linealización es una aproximación local de una función no lineal por una función lineal. La Figura 26.8 ilustra el concepto de linealización para un modelo de movimiento de robot (unidimensional). A la izquierda, representa un modelo de movimiento no lineal $f(\mathbf{x}_t,a_t)$ (el control a_t se omite en este gráfico ya que no desempeña una función en la linealización). A la derecha, esta función se aproxima por una función lineal $\tilde{f}(\mathbf{x}_t,a_t)$. Esta función lineal es tangente a f en el punto μ_t, la media de nuestra estimación de estado en el tiempo t. Tal linealización se llama **serie de Taylor** de primer grado. Un filtro de Kalman que linealiza f y h mediante la serie de Taylor se llama **filtro de Kalman extendido** (o EKF, por sus siglas en inglés). La Figura 26.9 muestra una secuencia de estimaciones de un robot que ejecuta un algoritmo de localización de filtro de Kalman extendido.

Linealiza

Serie de Taylor

A medida que el robot se mueve, la incertidumbre en su estimación de ubicación aumenta, como lo muestran las elipses de error. Su error disminuye a medida que detecta el rango y el rumbo a un hito con ubicación conocida y aumenta de nuevo cuando el robot pierde de vista el hito. Los algoritmos EKF funcionan bien si los hitos se identifican con facilidad. De lo contrario, la distribución posterior puede ser multimodal, como en la Figura 26.7(b). El problema de la necesidad de conocer la identidad de los hitos es una instancia del problema de **asociación de datos** discutido en la Figura 15.3.

En algunas situaciones, no hay un mapa del entorno disponible, por ello el robot tendrá que adquirir un mapa. Este es un poco un problema de huevo y gallina: el robot de navegación tendrá que determinar su ubicación relativa a un mapa que no conoce del todo, al mismo tiempo que construye este mapa mientras no conoce del todo su ubicación actual. Este problema es importante para muchas aplicaciones de robots, y se ha estudiado con amplitud bajo el nombre de **localización y mapeo simultáneo**, abreviado como **SLAM** (por sus siglas en inglés).

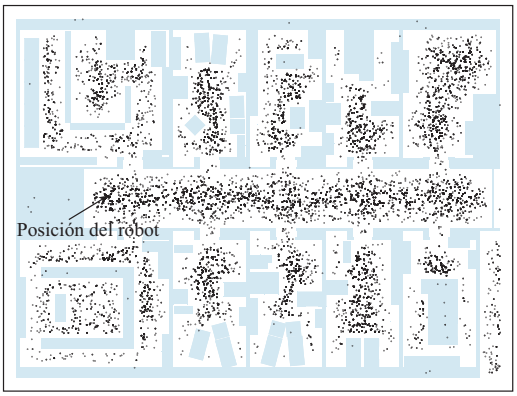
Localización y mapeo simultáneo

Los problemas de SLAM se resuelven utilizando múltiples técnicas probabilísticas diferentes, incluido el filtro de Kalman extendido ya discutido. Usar el EKF es sencillo: sólo aumenta el vector de estado para incluir las ubicaciones de los hitos en el entorno. Por fortuna, la actualización de EKF se escala de manera cuadrática, por lo que para mapas pequeños (por ejemplo, unos cientos de hitos) el cálculo es factible. Los mapas más ricos se obtienen utilizando métodos de relajación de gráficos, similares a las técnicas de inferencia de redes bayesianas discutidas en el Capítulo 13. La expectativa-maximización también se utiliza para SLAM.

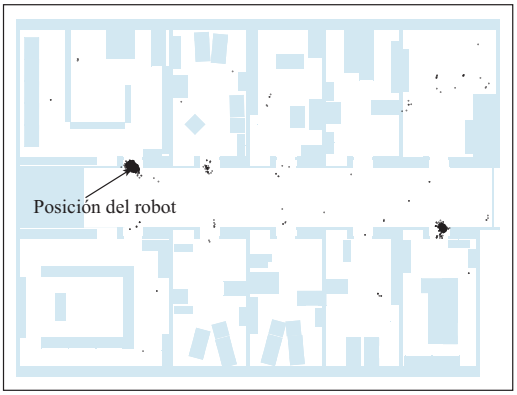
26.4.2 Otros tipos de percepción

No toda la percepción del robot se trata de localización o mapeo. Los robots también perciben la temperatura, los olores, el sonido, etc. Muchas de estas cantidades se estiman utilizando variantes de redes de Bayes dinámicas. Todo lo que se requiere para tales estimadores son distribuciones de probabilidad condicional que caracterizan la evolución de las variables de estado con el tiempo, y modelos de sensores que describen la relación de las mediciones con las variables de estado.

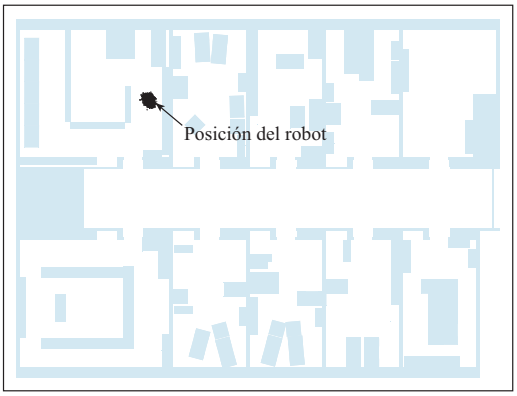
También es posible programar un robot como un agente reactivo, sin razonar de manera explícita sobre las distribuciones de probabilidad sobre los estados. Cubrimos ese enfoque en la Sección 26.9.1.



(a)



(b)



(c)

Figura 26.7 Localización de Monte Carlo, un algoritmo de filtrado de partículas para la localización de robots móviles. (a) Incertidumbre global inicial. (b) Incertidumbre bimodal aproximada después de navegar en el pasillo (simétrico). (c) Incertidumbre unimodal después de entrar en una habitación y encontrarla distintiva.

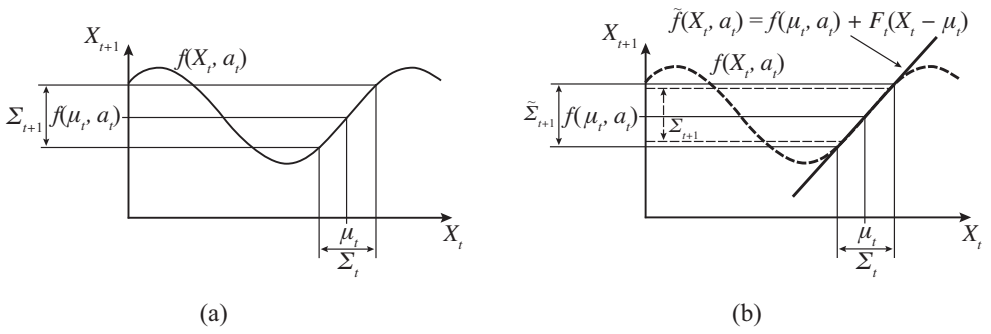


Figura 26.8 Ilustración unidimensional de un modelo de movimiento linealizado: (a) La función f, y la proyección de una media μ_t y un intervalo de covarianza (basado en Σ_t) en el tiempo $t + 1$. (b) La versión linealizada es la tangente de f en μ_t. La proyección de la media μ_t es correcta. Sin embargo, la covarianza proyectada $\tilde{\Sigma}_{t+1}$ difiere de Σ_{t+1}.

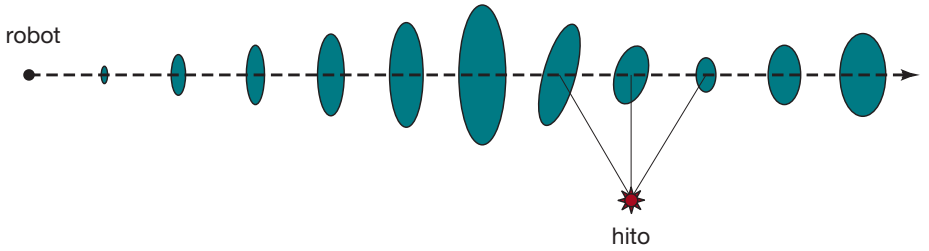


Figura 26.9 Localización utilizando el filtro de Kalman extendido. El robot se mueve en línea recta y a medida que avanza aumenta su incertidumbre en la estimación de su ubicación, como se ilustra en las elipses de error. Cuando observa un hito con posición conocida, la incertidumbre se reduce.

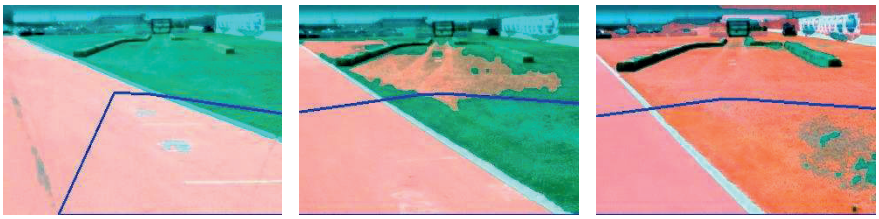
La tendencia en robótica es clara hacia representaciones con semántica bien definida. Las técnicas probabilísticas superan otros enfoques en múltiples problemas perceptuales difíciles, como la localización y el mapeo. Sin embargo, las técnicas estadísticas a veces son muy engorrosas, pero las soluciones más simples pueden ser igual de efectivas en la práctica. Para ayudar a decidir qué enfoque tomar, la experiencia en el trabajo con robots físicos reales es su mejor maestra.

26.4.3 Aprendizaje supervisado y no supervisado en la percepción del robot

El aprendizaje automático tiene un papel importante en la percepción del robot. Este es en particular el caso cuando la mejor representación interna no se conoce. Un enfoque común es mapear flujos de sensores de alta dimensión en espacios de menor dimensión usando métodos de aprendizaje automático no supervisado (vea el capítulo 19). Tal enfoque es llamado **embedding de baja dimensión**. El aprendizaje automático hace posible aprender modelos de sensores y movimientos a partir de datos, mientras se descubre de manera simultánea una representación interna adecuada.

Embedding de baja dimensión

Otra técnica de aprendizaje automático permite a los robots adaptarse de manera continua a grandes cambios en las mediciones de los sensores. Imagínese caminando desde un espacio iluminado por el Sol hasta una habitación oscura con luces de neón. Sin duda, las cosas son más oscuras



(a) (b) (c)

Figura 26.10 Secuencia de clasificaciones de "superficie transitable" utilizando visión adaptativa. (a) Sólo la carretera está clasificada como transitable (área rosa). La línea azul en forma de V muestra hacia dónde se dirige el vehículo. (b) Se ordena al vehículo que salga de la carretera, y el clasificador comienza a clasificar parte del césped como transitable. (c) El vehículo ha actualizado su modelo de superficies transitables para corresponder al césped y la carretera. (Cortesía de Sebastian Thrun).

en el interior. Pero el cambio de fuente de luz también afecta todos los colores: la luz de neón tiene un componente más fuerte de luz verde que la luz del Sol. Sin embargo, de alguna manera parece que no notamos el cambio. Si caminamos junto con personas en una habitación iluminada con neón, no pensamos que sus caras se volvieron de repente verdes. Nuestra percepción se adapta rápidamente a las nuevas condiciones de iluminación, y nuestro cerebro ignora las diferencias.

Las técnicas de percepción adaptativa permiten a los robots adaptarse a dichos cambios. Un ejemplo se muestra en la Figura 26.10, tomada del dominio de conducción autónoma. Aquí un vehículo terrestre no tripulado adapta su clasificador del concepto "superficie transitable". ¿Cómo funciona esto? El robot utiliza un láser para clasificar una pequeña área frente al robot. Cuando esta área se encuentra plana en el escaneo de rango láser, se utiliza como un ejemplo de entrenamiento positivo para el concepto "superficie transitable". Luego se entrena una técnica de mezcla de gaussianas similar al algoritmo EM discutido en el Capítulo 20 para reconocer los coeficientes específicos de color y textura de la pequeña muestra. Las imágenes en la Figura 26.10 son el resultado de aplicar este clasificador a la imagen completa.

Aprendizaje autosupervisado

Los métodos que hacen que los robots recojan sus propios datos de entrenamiento (¡etiquetados!) se llaman **autosupervisados**. En este caso, el robot utiliza el aprendizaje automático para aprovechar un sensor de corto alcance que funciona bien para la clasificación del terreno en un sensor que ve mucho más lejos. Eso permite que el robot conduzca más rápido y reduzca la velocidad sólo cuando el modelo del sensor dice que hay un cambio en el terreno que necesita ser examinado a detalle por los sensores de corto alcance.

26.5 Planificación y control

Las deliberaciones del robot finalmente se reducen a decidir cómo moverse, desde el nivel de tarea abstracta hasta el envío de corriente eléctrica a sus motores. En esta sección, simplificamos asumiendo que la percepción (y, cuando sea necesario, la predicción) se dan, por lo que el mundo es observable. Asumimos además transiciones deterministas (dinámicas) del mundo.

Camino

Comenzamos separando el movimiento del control. Definimos un **camino** como una secuencia de puntos en el espacio geométrico que un robot (o una parte de un robot, como un brazo) seguirá. Esto está relacionado con la noción de camino en el Capítulo 3, aunque aquí nos referimos a una secuencia de puntos en el espacio en lugar de una secuencia de acciones discretas. La tarea de encontrar un buen camino se llama **planificación de movimiento**.

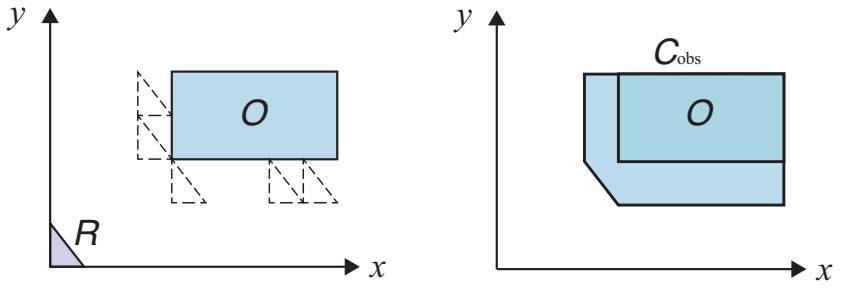


Figura 26.11 Robot triangular simple que puede trasladarse, y necesita evitar un obstáculo rectangular. A la izquierda está el espacio de trabajo, a la derecha está el espacio de configuración.

Una vez que tenemos un camino, la tarea de ejecutar una secuencia de acciones para seguir el camino es llamado **control de seguimiento de trayectoria**. Una **trayectoria** es un camino que tiene un tiempo asociado con cada punto en el camino. Un camino indica "ve de A a B a C, etc." y una trayectoria dice "comienza en A, tarda 1 segundo en llegar a B, y otros 1.5 segundos en llegar a C, etc.".

Control de seguimiento de trayectoria

Trayectoria

26.5.1 Espacio de configuración

Imagine un robot simple, $\mathcal{R}$, en forma de triángulo recto como el representado en el triángulo color lavanda en la esquina inferior izquierda de la Figura 26.11. El robot necesita planificar un camino que evite el obstáculo rectangular, $\mathcal{O}$. El espacio físico en el que se mueve un robot se llama **espacio de trabajo**. Este robot en particular puede moverse en cualquier dirección en el plano x-y, pero no es capaz de rotar. La figura muestra otras cinco posibles posiciones del robot con contornos punteados; estos están tan cerca del obstáculo como el robot permite llegar.

Espacio de trabajo

El cuerpo del robot podría representarse como un conjunto de puntos (x,y) (o puntos (x,y,z) para un robot tridimensional), igual que el obstáculo. Con esta representación, evitar el obstáculo significa que ningún punto en el robot se superpone a ningún punto en el obstáculo. La planificación del movimiento requeriría cálculos en conjuntos de puntos, lo cual es complicado y consume mucho tiempo.

Podemos simplificar los cálculos utilizando un esquema de representación en el que todos los puntos que componen el robot se representan como un solo punto en un espacio multidimensional abstracto, al que llamamos **espacio de configuración**, o **espacio C**. La idea es que el conjunto de puntos que comprende el robot se calcula si sabemos (1) las medidas básicas del robot (para nuestro robot triángulo, la longitud de los tres lados servirá) y (2) la **pose** actual del robot, su posición y orientación.

Espacio de configuración

Espacio C

Para nuestro robot triangular, dos dimensiones son suficientes para el espacio C: si conocemos las coordenadas (x,y) de un punto específico en el robot, usaremos el vértice del ángulo recto, entonces podemos calcular dónde está cada otro punto del triángulo (porque conocemos el tamaño y la forma del triángulo, y porque el triángulo no puede rotar). En la esquina inferior izquierda de la Figura 26.11, el triángulo lavanda permite ser representado por la configuración (0,0).

Si cambiamos las reglas para que el robot pueda rotar, entonces necesitaremos tres dimensiones, (x,y,θ), para poder calcular dónde está cada punto. Aquí θ es el ángulo de rotación del robot en el plano. Si el robot también tuviera la capacidad de estirarse, creciendo de manera uniforme por un factor de escala s, entonces el espacio C tendría cuatro dimensiones, (x,y,θ,s).

Por ahora nos quedaremos con el simple espacio C bidimensional del robot triángulo no rotatorio. La siguiente tarea es averiguar dónde están los puntos en el obstáculo en el espacio C. Considere los cinco triángulos de línea discontinua a la izquierda de la Figura 26.11 y observe dónde está el vértice del ángulo recto en cada uno de estos. Luego, imagine todas las formas en que el triángulo podría deslizarse. Note que el vértice del ángulo recto no puede entrar en el obstáculo y tampoco permite acercarse más de lo que está en cualquiera de los cinco triángulos de línea discontinua. Así, usted ve que el área donde el vértice del ángulo recto no puede ir, el **obstáculo del espacio C**, es el polígono de cinco lados a la derecha de la Figura 26.11 etiquetado $\mathcal{C}_{obs}$

Obstáculo del espacio C

En el lenguaje cotidiano hablamos de que hay múltiples obstáculos para el robot: una mesa, una silla, algunas paredes. Pero la notación matemática es un poco más fácil si pensamos en todos estos como combinados en un "obstáculo" que resulta tener componentes desconectados. En general, el obstáculo del espacio C es el conjunto de todos los puntos q en $\mathcal{C}$ tal que, si el robot se colocara en esa configuración, su geometría de espacio de trabajo intersectaría el obstáculo del espacio de trabajo.

Deje que los obstáculos en el espacio de trabajo sean el conjunto de puntos O, y que el conjunto de todos los puntos en el robot en la configuración q sea $\mathcal{A}(q)$. Entonces el obstáculo del espacio C se define como

$$C_{obs} = \{q : q \in C \textbf{ y } \mathcal{A}(q) \cap O \neq \{\}\}$$

y el **espacio libre** es $C_{free} = C - C_{obs}$.

Espacio libre

El espacio C se vuelve más interesante para los robots con partes móviles. Considere el brazo de dos enlaces de la Figura 26.12(a). Está atornillado a una mesa por lo que la base no se mueve, pero el brazo tiene dos articulaciones que se mueven de manera independiente, a estos los llamamos **grados de libertad (DOF)**. Mover las articulaciones altera las coordenadas (x,y) del codo, la pinza y cada punto en el brazo. El espacio de configuración del brazo es bidimensional: $(\theta_{shou},\theta_{elb})$, donde θ_{shou} es el ángulo de la articulación del hombro, y θ_{elb} es el ángulo de la articulación del codo.

Grados de libertad (DOF)

Saber la configuración de nuestro brazo de dos enlaces significa que podemos determinar dónde está cada punto en el brazo mediante trigonometría simple. En general, el mapeo de **cinemática directa** es una función

Cinemática directa

$$\phi_b : C \rightarrow W$$

que toma una configuración y devuelve la ubicación de un punto b particular en el robot cuando el robot está en esa configuración. Un mapeo de cinemática directa particularmente útil es el del actuador final del robot, ϕ_{EE}. El conjunto de todos los puntos en el robot en una configuración particular q es denotado por $\mathcal{A}(q) \subset W$:

$$\mathcal{A}(q) = \bigcup_b \{\phi_b(q)\}.$$

El problema inverso, de mapear una ubicación deseada para un punto en el robot a la configuración que el robot necesita para que eso suceda, se conoce como **cinemática inversa**:

Cinemática inversa

$$IK_b: x \in W \mapsto \{q \in C \; s.t. \; \phi_b(q) = x\}.$$

A veces, el mapeo de cinemática inversa toma no sólo una posición, sino una orientación deseada como entrada. Por ejemplo, cuando queremos que un manipulador agarre un objeto, podemos calcular una posición y orientación deseadas para su agarre y usar la cinemática inversa para determinar una configuración de objetivo para el robot. Luego, un planificador necesita encontrar una forma de llevar al robot desde su configuración actual hasta la configuración de objetivo sin intersectar obstáculos.

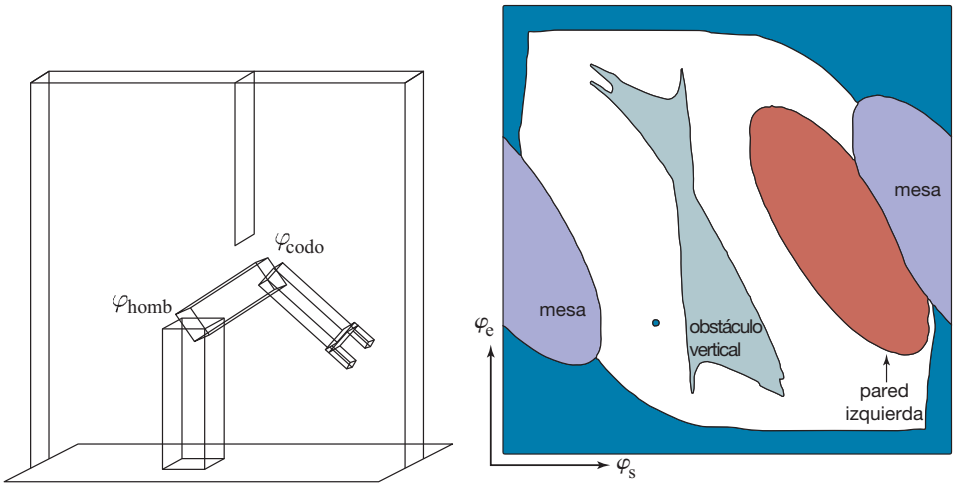


Figura 26.12 (a) Representación del espacio de trabajo de un brazo de robot con dos grados de libertad. El espacio de trabajo es una caja con un obstáculo plano colgando del techo. (b) Espacio de configuración del mismo robot. Sólo las regiones blancas en el espacio son configuraciones libres de colisiones. El punto en este diagrama corresponde a la configuración del robot mostrada a la izquierda.

Los obstáculos del espacio de trabajo con frecuencia se representan como formas geométricas simples, en especial en los libros de texto de robótica, que suelen centrarse en los obstáculos poligonales. Pero ¿cómo se ven los obstáculos en el espacio de configuración?

Para el brazo de dos enlaces, los obstáculos simples en el espacio de trabajo, como una línea vertical, tienen contrapartes de espacio C muy complejas, como se muestra en la Figura 26.12(b). Las diferentes sombras del espacio ocupado corresponden a los diferentes objetos en el espacio de trabajo del robot: la región oscura que rodea todo el espacio libre corresponde a configuraciones en las que el robot colisiona consigo mismo. Es fácil ver que los valores extremos de los ángulos del hombro o del codo causan tal violación. Las dos regiones ovaladas a ambos lados del robot corresponden a la mesa en la que está montado el robot. La tercera región ovalada corresponde a la pared izquierda.

Finalmente, el objeto más interesante en el espacio de configuración es el obstáculo vertical que cuelga del techo e impide los movimientos del robot. Este objeto tiene una forma divertida en el espacio de configuración: es no lineal y, en algunos lugares, incluso cóncavo. Con un poco de imaginación, el lector reconocerá la forma de la pinza en el extremo superior izquierdo.

Animamos al lector a hacer una pausa por un momento y estudiar este diagrama. ¡La forma de este obstáculo en el espacio C no es del todo obvia! El punto dentro de la Figura 26.12(b) marca la configuración del robot en la Figura 26.12(a). La Figura 26.13 muestra tres configuraciones adicionales, tanto en el espacio de trabajo como en el espacio de configuración. En la configuración conf-1, la pinza está agarrando el obstáculo vertical.

Vemos que incluso si el espacio de trabajo del robot está representado por polígonos planos, la forma del espacio libre es muy complicada. En la práctica, se *sondea* un espacio de configuración en lugar de construirlo de manera explícita. Un planificador puede generar una configuración y luego probar para ver si está en el espacio libre aplicando la cinemática del robot, y verificar las colisiones en las coordenadas del espacio de trabajo.

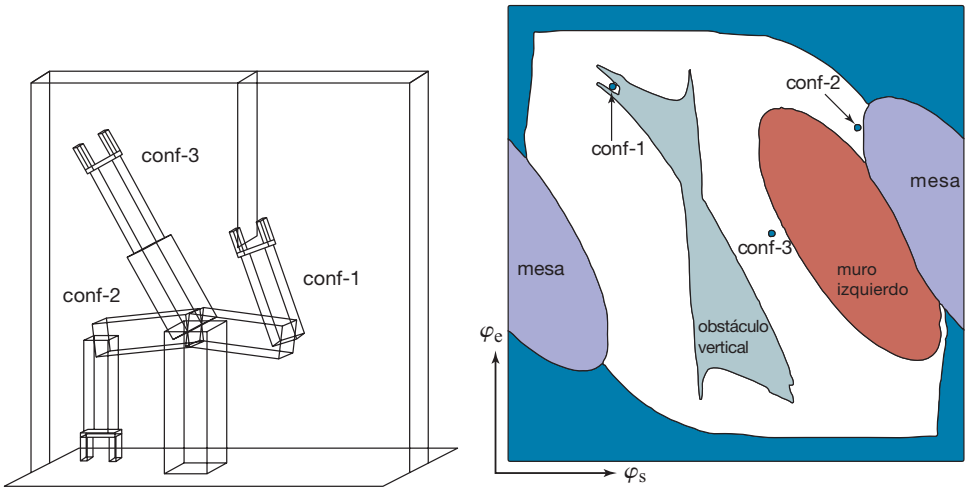


Figura 26.13 Tres configuraciones de robot mostradas en el espacio de trabajo y en el espacio de configuración.

26.5.2 Planificación de movimiento

Planificación de movimiento

El problema de **planificación de movimiento** es el de encontrar un plan que lleve a un robot de una configuración a otra sin colisionar con un obstáculo. Es un bloque de construcción básico para el movimiento y la manipulación. En la Sección 26.5.4 discutiremos cómo hacer esto bajo dinámicas complicadas, como dirigir un automóvil que puede desviarse del camino si toma una curva demasiado rápido. Por ahora nos centraremos en el problema simple de planificación de movimiento de encontrar una trayectoria geométrica que esté libre de colisiones. La planificación de movimiento es un **problema de búsqueda** de estado continuo por excelencia, pero es posible discretizar el espacio y aplicar los algoritmos de búsqueda del Capítulo 3.

Problema del mudancero de pianos

El problema de planificación de movimiento a veces es conocido como **problema del mudancero de pianos**. Recibe su nombre de los esfuerzos de quien muda un piano grande e irregular de una habitación a otra sin golpear nada. Tenemos:

- un *mundo* de espacio de trabajo W en $\mathbb{R}^2$ para el plano o $\mathbb{R}^3$ para tres dimensiones,
- una *región de obstáculos* $O \subset W$,
- un robot con un espacio de configuración C y un conjunto de puntos $A(q)$ para $q \in C$,
- una configuración inicial $q_s \in C$, y
- una configuración de objetivo $q_g \in C$.

La región de obstáculos induce un obstáculo C-space C_{obs} y su espacio libre correspondiente C_{free} definido como en la sección anterior. Necesitamos encontrar un **camino** continuo a través del espacio libre. Usaremos una curva parametrizada, $\tau(t)$, para representar el camino, donde $\tau(0) = q_s$ y $\tau(1) = q_g$ y $\tau(t)$ para cada t entre 0 y 1 es algún punto en C_{free}. Es decir, t parametriza qué tan lejos estamos a lo largo del camino, desde el inicio hasta la meta. Note que t actúa de alguna manera como las veces en que t aumenta la distancia a lo largo del camino, pero t siempre es un punto en el intervalo [0,1] y no se mide en segundos.

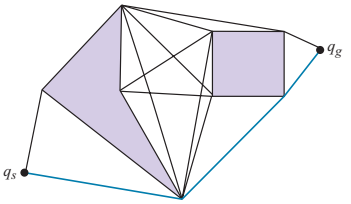


Figura 26.14 Grafo de visibilidad. Las líneas conectan cada par de vértices que se "ven" entre sí, líneas que no atraviesan un obstáculo. El camino más corto debe estar sobre estas líneas.

El problema de planificación de movimiento puede complicarse de múltiples formas: definiendo el objetivo como un conjunto de posibles configuraciones en lugar de una única configuración; definiendo el objetivo en el espacio de trabajo en lugar del espacio C; definiendo una función de costo (por ejemplo, longitud del camino) a minimizar; satisfaciendo restricciones (por ejemplo, si el camino implica llevar una taza de café, asegurándose de que la taza siempre esté orientada hacia arriba para que el café no se derrame).

Espacios de planificación de movimiento: Retrocedamos un paso y asegurémonos de entender los espacios involucrados en la planificación de movimiento. Primero está el espacio de trabajo o mundo W. Los puntos en W son puntos en el mundo tridimensional cotidiano. A continuación, tenemos el espacio de configuraciones, C. Los puntos q en C son d-dimensionales, con d el número de grados de libertad del robot, y se mapean a conjuntos de puntos $\mathcal{A}(q)$ en W. Finalmente, está el espacio de caminos. El espacio de caminos es un espacio de funciones. Cada punto en este espacio se mapea a una curva entera a través del espacio C. ¡Este espacio es ∞-dimensional! De manera intuitiva, necesitamos d dimensiones para cada configuración a lo largo del camino, y hay tantas configuraciones en un camino como puntos en el intervalo numérico [0,1]. Ahora, consideremos algunas formas de resolver el problema de planificación de movimiento.

Grafos de visibilidad

Para el caso simplificado de espacios de configuración bidimensional y obstáculos espacio C poligonal, los **grafos de visibilidad** son una forma conveniente de resolver el problema de planificación de movimiento con solución garantizada de camino más corto. Dejemos que $V_{obs} \subset C$ sea el conjunto de vértices de los polígonos que componen $Cobs$, y dejemos que $V = V_{obs} \cup \{q_s, q_g\}$.

Grafos de visibilidad

Se construye un grafo $G = (V,E)$ en el conjunto de vértices V con aristas $e_{ij} \in E$ conectando un vértice v_i a otro vértice v_j si la línea que conecta los dos vértices está libre de colisiones, es decir, si $\{\lambda v_i + (1-\lambda)v_j : \lambda \in [0,1]\} \cap C_{obs} = \{\}$. Cuando esto sucede, decimos que los dos vértices "se ven entre sí", de ahí es donde los grafos de "visibilidad" obtuvieron su nombre.

Para resolver el problema de planificación de movimiento, todo lo que necesitamos hacer es ejecutar una búsqueda de grafo discreto (por ejemplo, búsqueda de best-first) en el grafo G con estado inicial q_s y objetivo q_g. En la Figura 26.14 vemos un grafo de visibilidad y una solución óptima de tres pasos. Una búsqueda óptima en grafos de visibilidad siempre nos dará el camino óptimo (si existe uno), o informará de un fracaso si no existe un camino.

Diagramas de Voronoi

Los grafos de visibilidad fomentan caminos que corren adyacentes a un obstáculo, si tuviera que caminar alrededor de una mesa para llegar a la puerta, el camino más corto sería pegarse lo más

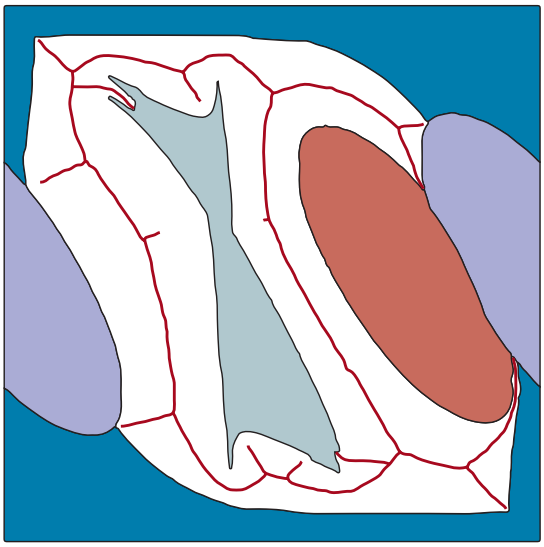


Figura 26.15 Un diagrama de Voronoi que muestra el conjunto de puntos (líneas negras) equidistantes a dos o más obstáculos en el espacio de configuración.

posible a la mesa. Sin embargo, si el movimiento o la detección son no deterministas, eso le pondría en riesgo de chocar con la mesa. Una forma de abordar esto es fingir que el cuerpo del robot es un poco más grande de lo que realmente es, pues proporciona una zona de amortiguación. Otra forma es aceptar que la longitud del camino no es la única métrica que queremos optimizar. La Sección 26.8.2 muestra cómo aprender una buena métrica a partir de ejemplos humanos de comportamiento.

Diagrama de Voronoi

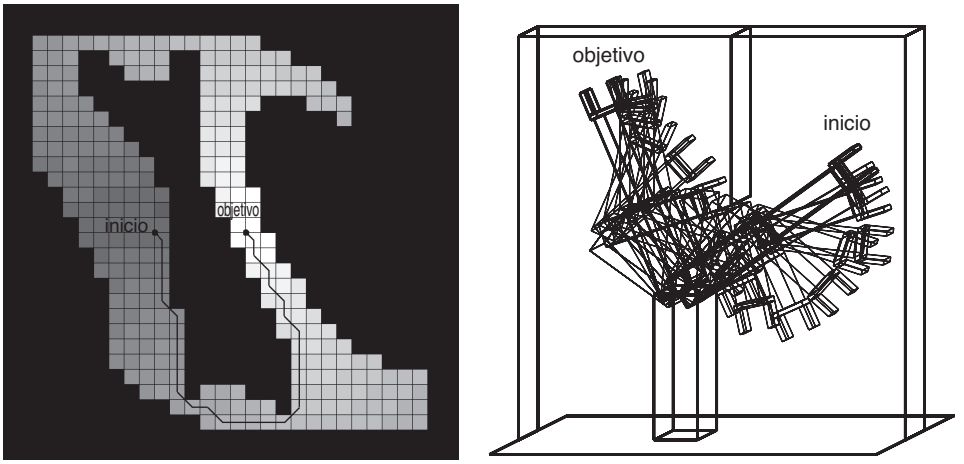
Región

Grafo de Voronoi

Una tercera forma es usar una técnica diferente, una que coloca los caminos lo más lejos posible de los obstáculos en lugar de abrazarlos de cerca. Un **diagrama de Voronoi** es una representación que nos permite hacer eso. Para tener una idea de lo que hace un diagrama de Voronoi, considere un espacio donde los obstáculos son, digamos, una docena de puntos pequeños dispersos en un plano. Ahora rodea cada uno de los puntos de obstáculo con una **región** que consta de todos los puntos en el plano que están más cerca de ese punto de obstáculo que de cualquier otro punto de obstáculo. Así, las regiones dividen el plano. El diagrama de Voronoi consta del conjunto de regiones, y el **grafo de Voronoi** consta de los bordes y vértices de las regiones.

Cuando los obstáculos son áreas, no puntos, todo se mantiene bastante igual. Cada región todavía contiene todos los puntos que están más cerca de un obstáculo que de cualquier otro, donde la distancia se mide hasta el punto más cercano en un obstáculo. Los límites entre las regiones todavía corresponden a puntos que están equidistantes entre dos obstáculos, pero ahora el límite es una curva en lugar de una línea recta. Calcular estos límites puede ser costoso en espacios de alta dimensión.

Para resolver el problema de planificación de movimiento, conectamos el punto de inicio q_s al punto más cercano en el grafo de Voronoi a través de una línea recta, y lo mismo para el punto de objetivo q_g. Luego usamos la búsqueda de grafos discretos para encontrar el camino más corto en el grafo. Para problemas como navegar por corredores en interiores, esto da un buen camino que va por el medio del corredor. Sin embargo, en entornos al aire libre surge con trayectorias ineficientes; por ejemplo, al sugerir un desvío innecesario de 100 metros para mantenerse en el medio de un espacio abierto de 200 metros.



(a) (b)

Figura 26.16 (a) Función de valor y camino encontrado para una aproximación de celda de cuadrícula discreta del espacio de configuración. (b) El mismo camino visualizado en coordenadas de espacio de trabajo. Observe cómo el robot dobla su codo para evitar una colisión con el obstáculo vertical.

Descomposición de celdas

Descomposición de celdas

Un enfoque alternativo para la planificación del movimiento es discretizar el C-espacio. los métodos de **Descomposición de celdas** descomponen el espacio libre en un número finito de regiones contiguas, llamadas celdas. Estas celdas están diseñadas de tal manera que el problema de planificación de trayectorias dentro de una sola celda permite resolverse por medios simples (por ejemplo, moviéndose en línea recta). El problema de planificación de trayectorias luego se convierte en un problema de búsqueda de grafos discretos (como con los grafos de visibilidad y los grafos de Voronoi) para encontrar un camino a través de una secuencia de celdas.

La descomposición de celdas más simple consiste en una cuadrícula regularmente espaciada. La Figura 26.16(a) muestra una descomposición de retícula cuadrada del espacio y un camino de solución que es óptimo para este tamaño de retícula. El sombreado en escala de grises indica el *valor* de cada celda de cuadrícula de espacio libre, el costo del camino más corto desde esa celda hasta el objetivo. (Estos valores permiten ser calculados por una forma determinista del algoritmo de ITERACIÓN DE VALOR dado en la Figura 17.6 en la página 573). La Figura 26.16(b) muestra la trayectoria de espacio de trabajo correspondiente para el brazo. Por supuesto, también podríamos usar el algoritmo A* para encontrar un camino más corto.

Esta descomposición de retícula tiene la ventaja de que es simple de implementar, pero sufre de tres limitaciones. Primero, sólo es viable para espacios de configuración de baja dimensión, porque el número de celdas de la retícula aumenta de manera exponencial con d, el número de dimensiones. (¿Suena familiar? Esta es la maldición de la dimensionalidad). En segundo lugar, los caminos a través del espacio de estado discretizado no siempre serán suaves. Vemos en la Figura 26.16(a) que las partes diagonales del camino son dentadas y, por lo tanto, muy difíciles de seguir con precisión para el robot. El robot puede intentar suavizar el camino de la solución, pero esto está lejos de ser sencillo.

En tercer lugar, está el problema de qué hacer con las celdas que son "mixtas"; es decir, que no están ni completamente dentro del espacio libre ni completamente dentro del espacio ocupado.

Un camino de solución que incluye tal casilla puede no ser una solución real, porque es posible que no haya forma de cruzar la casilla de manera segura. Esto haría que el planificador de trayectorias *no sea sólido*. Por otro lado, si insistimos en que sólo se permiten usar casillas completamente libres, el planificador será *incompleto*, porque podría ser el caso de que los únicos caminos hacia el objetivo pasen por casillas mixtas, podría ser que un corredor sea lo suficientemente ancho para que el robot pase, pero el corredor sólo está cubierto por casillas mixtas.

El primer enfoque para este problema es la *subdivisión adicional* de las casillas mixtas, quizás utilizando casillas de la mitad del tamaño original. Esto continua de manera recursiva hasta que se encuentre un camino que esté por completo dentro de las celdas libres. Este método funciona bien y es completo si hay una forma de decidir si una casilla dada es una casilla mixta, lo cual es fácil sólo si los límites del espacio de configuración tienen descripciones matemáticas relativamente simples.

Es importante notar que la descomposición celular no requiere representar de manera explícita el espacio de obstáculos C_{obs}. Podemos decidir incluir una casilla o no utilizando un **verificador de colisiones**. Esta es una noción crucial para la planificación del movimiento. Un verificador de colisiones es una función $\gamma(q)$ que se mapea a 1 si la configuración colisiona con un obstáculo, y 0 en caso contrario. Es mucho más fácil verificar si una configuración específica está en colisión, que construir de manera explícita todo el espacio de obstáculos C_{obs}.

Verificador de colisiones

Al examinar la trayectoria de solución mostrada en la Figura 26.16(a), podemos ver una dificultad adicional que tendrá que resolverse. El camino contiene esquinas arbitrariamente agudas, pero un robot físico tiene impulso y no puede cambiar de dirección de manera instantánea. Este problema permite resolverse almacenando, para cada celda de la retícula, el estado continuo exacto (posición y velocidad) que se alcanzó cuando la casilla se alcanzó en la búsqueda. Suponga que, además de propagar información a las casillas cercanas a la retícula, usamos este estado continuo como base y aplicamos el modelo de movimiento del robot continuo para saltar a las celdas cercanas. Así que no hacemos un giro instantáneo de 90°; hacemos un giro redondeado gobernado por las leyes del movimiento. Ahora podemos garantizar que la trayectoria resultante es suave y permite ser ejecutada por el robot. Un algoritmo que implementa esto es el **híbrido A***.

Híbrido A*

Planificación de movimiento aleatorizado

La planificación de movimiento aleatorizado realiza una búsqueda de grafos en una descomposición *aleatoria* del espacio de configuración, en lugar de una descomposición celular regular. La idea clave es muestrear un conjunto aleatorio de puntos y crear bordes entre ellos si hay una forma simple de pasar de uno a otro (por ejemplo, a través de una línea recta) sin colisionar; entonces podemos buscar en este grafo.

Hojas de ruta probabilísticas (PRM)

Un algoritmo de **hojas de ruta probabilísticas (PRM**, por sus siglas en inglés**)** es una forma de aprovechar esta idea. Asumimos que tenemos acceso a un verificador de colisiones γ (definido en la página 946) y a un **planificador simple** $B(q_1,q_2)$ que devuelve un camino de q_1 a q_2 (o fracaso) pero lo hace *de manera rápida*. Este planificador simple no va a ser completo, puede devolver un fracaso incluso si existe una solución. Su trabajo es hacerlo de manera rápida e intentar conectar q_1 y q_2 y dejar que el algoritmo principal sepa si tiene éxito. Lo usaremos para definir si existe un borde entre dos vértices.

Planificador simple

Hito

El algoritmo comienza muestreando M **hitos**, puntos en C_{free}, además de los puntos q_s y q_g. Utiliza un muestreo de rechazo, donde las configuraciones se muestrean de manera aleatoria y las colisiones se verifican usando γ hasta que se encuentran un total de M hitos. A continuación, el algoritmo utiliza el planificador simple para intentar conectar pares de hitos. Si el planificador simple devuelve logro, entonces se añade un borde entre el par al grafo; de lo contrario, el grafo permanece como está. Intentamos conectar cada hito ya sea a sus k vecinos más cercanos (a esto lo llamamos k-PRM), o a todos los hitos en una esfera de un radio r. Finalmente, el algoritmo

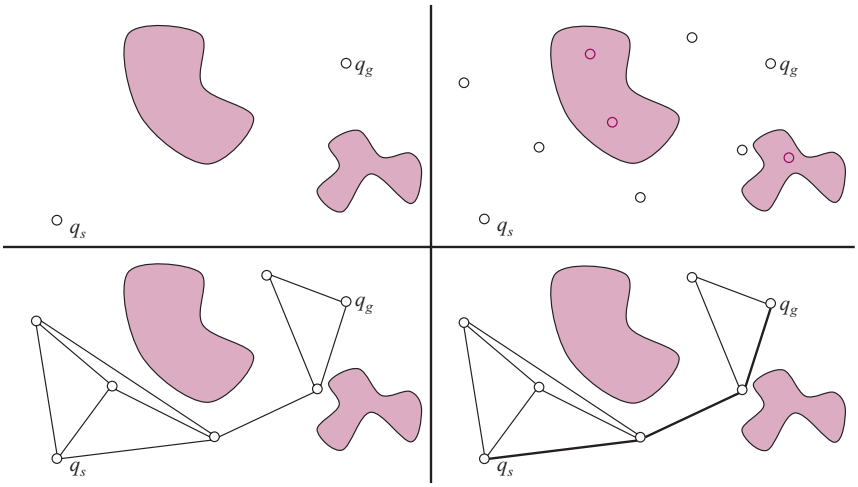


Figura 26.17 El algoritmo de mapa de trayectoria probabilístico (PRM). **Arriba a la izquierda**: las configuraciones de inicio y objetivo. **Arriba a la derecha**: muestra M hitos libres de colisiones (aquí $M = 5$). **Abajo a la izquierda**: conecta cada hito a sus k vecinos más cercanos (aquí $k = 3$). **Abajo a la derecha**: encuentra el camino más corto desde el inicio hasta el objetivo en el grafo resultante.

busca un camino en este grafo de q_s a q_g. Si no se encuentra un camino, entonces se muestrean M hitos más, se añaden al grafo, y el proceso se repite.

La Figura 26.17 muestra un mapa de trayectoria con el camino encontrado entre dos configuraciones. Los PRM no son completos, pero son lo que se llama **probabilísticamente completos**, de modo eventual encontrarán un camino, si existe. De manera intuitiva, esto se debe a que siguen muestreando más hitos. Los PRM funcionan bien aun en espacios de configuración de alta dimensión.

Probabilísticamente completos

Los PRM también son populares para la **planificación de múltiples revisiones**, en la que se tienen múltiples problemas de planificación de movimientos dentro del mismo espacio C. Una vez que el robot alcanza una meta, se le pide que alcance otra meta en el mismo espacio de trabajo. Los PRM son útiles porque el robot puede dedicar tiempo por adelantado para construir un mapa de trayectoria y amortizar el uso de ese mapa de trayectoria en múltiples consultas.

Planificación de múltiples revisiones

Árboles aleatorios de exploración rápida

Una extensión de los PRM llamada **árboles aleatorios de exploración rápida (RRT)** es popular para la planificación de revisiones únicas. Incrementamos la construcción de dos árboles, uno con q_s como la raíz y uno con q_g como la raíz. Se eligen hitos aleatorios y se intenta conectar cada nuevo hito a los árboles existentes. Si un hito conecta ambos árboles, esto significa que se ha encontrado una solución, como en la Figura 26.18. Si no, el algoritmo encuentra el punto más cercano en cada árbol y añade al árbol una nueva arista que se extiende desde el punto a una distancia δ hacia el hito. Esto tiende a hacer crecer el árbol hacia secciones previamente inexploradas del espacio.

Árboles aleatorios de exploración rápida (RRT)

A los roboticistas les encantan los RRT por su facilidad de uso. Sin embargo, las soluciones RRT suelen ser no óptimas y carecen de suavidad. Por lo tanto, los RRT con frecuencia son seguidos por un paso de posprocesamiento. El más común es el "acortamiento", en el que seleccionamos de manera aleatoria uno de los vértices en el camino de solución e intentamos eliminarlo conectando sus vecinos entre sí (a través del planificador simple). Hacemos esto de manera

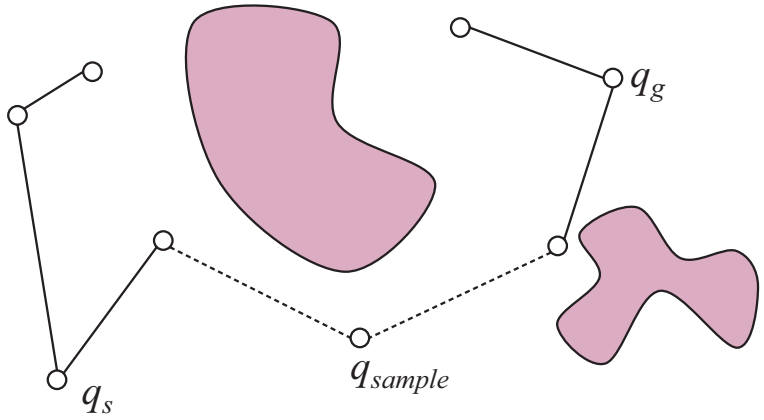


Figura 26.18 El algoritmo RRT bidireccional construye dos árboles (uno desde el inicio, el otro desde la meta) conectando de manera incremental cada muestra al nodo más cercano en cada árbol, si la conexión es posible. Cuando una muestra se conecta a ambos árboles, eso significa que hemos encontrado un camino de solución.

(a) (b) (c)

Figura 26.19 Instantáneas de una trayectoria producida por un RRT y posprocesada con acortamiento. (Cortesía de Anca Dragan).

repetida durante tantos pasos como tiempo de cálculo tengamos. Aun así, las trayectorias pueden parecer un poco antinaturales debido a las posiciones aleatorias del hito que se seleccionaron, como se muestra en la Figura 26.19.

RRT*

RRT* es una modificación a RRT que hace que el algoritmo sea asintóticamente óptimo: la solución converge a la solución óptima a medida que se muestrean más y más hitos. La idea clave es elegir el vecino más cercano en función de una noción de costo por venir en lugar de la distancia desde el hito solo, y reconfigurar el árbol, al intercambiar los padres de los vértices más antiguos si es más barato alcanzarlos a través del nuevo hito.

Optimización de trayectoria para planificación cinemática

Los algoritmos de muestreo aleatorizado tienden a construir primero un camino complejo pero factible y luego lo optimizan. La optimización de la trayectoria hace lo contrario: comienza con un camino simple pero inviable y luego trabaja para sacarlo de la colisión. El objetivo es

encontrar un camino que optimice una función de costo[1] sobre trayectorias. Es decir, queremos minimizar la función de costo $J(\tau)$, donde $\tau(0) = q_s$ y $\tau(1) = q_g$.

J se llama **funcional** porque es una función sobre funciones. El argumento para J es τ, que es en sí misma una función: $\tau(t)$ toma como entrada un punto en el intervalo [0,1] y lo mapea a una configuración. Un funcional de costo estándar equilibra entre dos aspectos importantes del movimiento del robot: evitar colisiones y eficiencia,

$$J = J_{obs} + \lambda J_{eff}$$

donde la eficiencia J_{eff} mide la longitud del camino y también permite medir la suavidad. Una forma conveniente de definir la eficiencia es con un cuadrático: integra la primera derivada cuadrada de τ (veremos en un momento por qué esto de hecho incentiva caminos cortos):

$$J_{eff} = \int_0^1 \frac{1}{2} ||\dot{\tau}(s)||^2 ds.$$

Para el término de obstáculo, supongamos que podemos calcular la distancia $d(x)$ desde cualquier punto $x \in W$ hasta el borde de obstáculo más cercano. Esta distancia es positiva fuera de los obstáculos, 0 en el borde y negativo por dentro. Esto se denomina **campo de distancia firmado**. Ahora podemos definir un campo de costo en el espacio de trabajo, llamado c, que tiene un alto costo dentro de los obstáculos, y un pequeño costo justo afuera. Con este costo podemos hacer que los puntos en el espacio de trabajo odien estar dentro de los obstáculos, y que les disguste estar justo al lado de ellos (evitando el problema del grafo de visibilidad de estar siempre saliendo por los bordes de los obstáculos). Por supuesto, nuestro robot no es un punto en el espacio de trabajo, por lo que tenemos un poco más de trabajo que hacer, necesitamos considerar todos los puntos b en el cuerpo del robot:

Campo de distancia firmado

$$J_{obs} = \int_0^1 \int_b c(\underbrace{\phi_b(\tau(s))}_{\in W}) || \frac{d}{ds} \underbrace{\phi_b(\tau(s))}_{\in W} || \, db \, ds.$$

Esto se llama **integral de caminos**, no sólo integra c en el camino para cada punto del cuerpo, sino que se multiplica por la derivada para hacer que el costo sea invariante al *reajuste* del camino. Imagine un robot barriendo a través del campo de costo, acumulando costo a medida que se mueve. Independiente de cuán rápido o lento se mueva el brazo a través del campo, debe acumular el mismo costo.

Integral de caminos

La forma más sencilla de resolver el problema de optimización anterior y encontrar un camino es el *descenso de gradiente*. Si se pregunta cómo tomar gradientes de funcionales con respecto a funciones, el llamado *cálculo de variaciones* está aquí para ayudar. Resulta en especial fácil para funcionales de la forma

$$J[\tau] = \int_0^1 F(s, \tau(s), \dot{\tau}(s)) ds$$

que son integrales de funciones que dependen sólo del parámetro s, el valor de la función en s, y la derivada de la función en s. En tal caso, la **ecuación de Euler-Lagrange** dice que el gradiente es

Ecuación de Euler-Lagrange

$$\nabla_\tau J(s) = \frac{\partial F}{\partial \tau(s)}(s) - \frac{d}{dt} \frac{\partial F}{\partial \dot{\tau}(s)}(s).$$

Si miramos de cerca a J_{eff} y J_{obs}, ambos siguen este patrón. En particular para J_{eff}, tenemos $F(s, \tau(s), \dot{\tau}(s)) = ||\dot{\tau}(s)||^2$. Para sentirnos un poco más cómodos con esto, calculemos el gradiente

[1] A los roboticistas les gusta minimizar una función de costo J, mientras que en otras partes de la IA intentamos maximizar una función de utilidad U o una recompensa R.

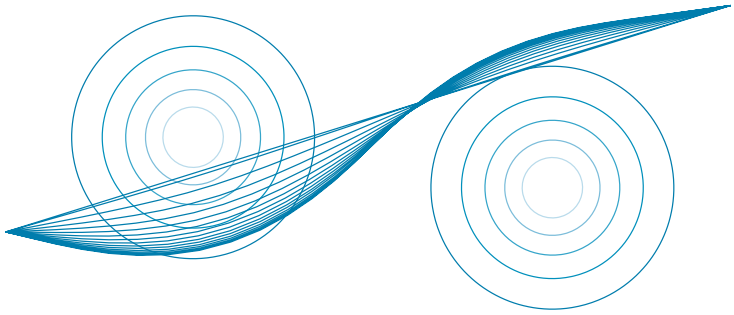


Figura 26.20 Optimización de trayectoria para la planificación de movimientos. Dos obstáculos puntuales con bandas circulares de costo decreciente alrededor de ellos. El optimizador comienza con la trayectoria de línea recta, y deja que los obstáculos doblen la línea lejos de las colisiones, encontrando el camino mínimo a través del campo de costo.

para J_{eff} solo. Vemos que F no tiene una dependencia directa en $\tau(s)$, por lo que el primer término en la fórmula es 0. Nos quedamos con

$$\nabla_\tau J(s) = 0 - \frac{d}{dt}\dot{\tau}(s)$$

ya que la parcial de F con respecto a $\dot{\tau}(s)$ es $\dot{\tau}(s)$.

Observe cómo nos facilitamos las cosas al definir a J_{eff} es un bonito cuadrático de la derivada (e incluso pusimos un $\frac{1}{2}$ al frente para que el 2 se cancele bien). En la práctica, verá que este truco ocurre mucho para la optimización, el arte no sólo está en elegir cómo optimizar la función de costo, sino también en elegir una función de costo que se comportará bien con cómo la optimizará. Simplificando nuestro gradiente, obtenemos

$$\nabla_\tau J(s) = -\ddot{\tau}(s).$$

Ahora, dado que J_{eff} es un cuadrático, establecer este gradiente a 0 nos da la solución para τ si no tuviéramos que lidiar con obstáculos. Integrando una vez, obtenemos que la primera derivada debe ser constante; integrando de nuevo obtenemos que $\tau(s) = a \cdot s + b$, con a y b determinados por las restricciones de punto final para $\tau(0)$ y $\tau(1)$. ¡El camino óptimo con respecto a J_{eff} es, por lo tanto, la línea recta desde el inicio hasta la meta! De hecho, es la forma más eficiente de ir de uno a otro si no hay obstáculos de los que preocuparse.

Por supuesto, la adición de trabajos es lo que hace las cosas difíciles, y le ahorraremos derivar su gradiente aquí. El robot con frecuencia inicializaría su camino para ser una línea recta, que atravesaría de manera directa algunos obstáculos. Luego calcularía el gradiente del costo sobre el camino actual, y el gradiente serviría para alejar el camino lejos de los obstáculos (Figura 26.20). Tenga en cuenta que el descenso de gradiente sólo encontrará una solución *óptima local*, al igual que la escalada de colinas. Métodos como el recocido simulado (Sección 4.1.2) se usan para la exploración, para hacer más probable que el óptimo local sea bueno.

26.5.3 Control de seguimiento de trayectoria

Hemos cubierto cómo *planificar* movimientos, pero no cómo *moverse* de manera real; es decir, cómo aplicar corriente a los motores, producir torque, mover el robot. Este es el dominio de la **teoría de control**, un campo de creciente importancia en la IA. Hay dos preguntas principales a

Teoría de control

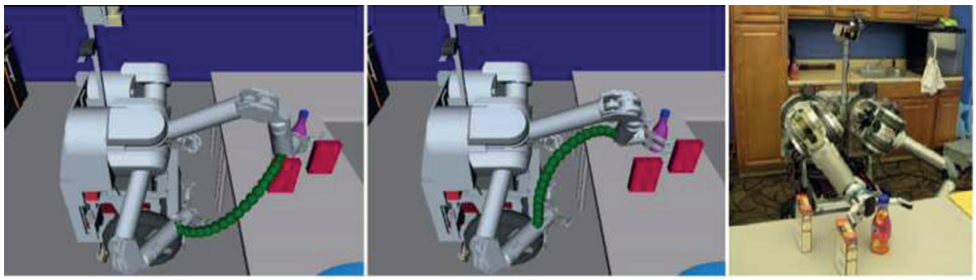


Figura 26.21 Representación de tarea resuelta con un optimizador de trayectoria, cómo alcanzar una botella y agarrarla. Izquierda: trayectoria inicial, trazada para el actuador final. Centro: trayectoria final después de la optimización. Derecha: configuración objetivo. (Cortesía de Anca Dragan. Ver Ratliff *et al.* (2009)).

responder: ¿cómo convertimos una descripción matemática de una trayectoria en una secuencia de acciones en el mundo real (control de bucle abierto), ¿y cómo nos aseguramos de que estamos en el camino correcto (control de bucle cerrado)?

De configuraciones a torques para seguimiento de bucle abierto: Nuestra trayectoria $\tau(t)$ nos da configuraciones. El robot comienza en reposo en $q_s = \tau(0)$. A partir de ahí, los motores del robot convertirán las corrientes en torques, lo que llevará al movimiento. Pero ¿qué torques debería buscar el robot, de modo que termine en $q_g = \tau(1)$?

Modelo de dinámica

Aquí es donde entra la idea de un **modelo dinámico** (o modelo de transición). Podemos dar Al robot una función f que calcula los efectos que los torques tienen en la configuración. ¿Recuerda de la física que $F = ma$? Bueno, también hay algo así para los torques, en la forma $u = f^{-1}(q, \dot{q}, \ddot{q})$, con u un torque, $\dot{q}$ una velocidad, y $\ddot{q}$ una aceleración.[2] Si el robot está en la configuración q y la velocidad $\dot{q}$, y se aplica el torque u, eso llevaría a la aceleración $\ddot{q} = f(q, \dot{q}, u)$. La tupla $(q, \dot{q})$ es un **estado dinámico**, porque incluye velocidad, mientras que q es el **estado cinemático** y no es suficiente para calcular qué torque aplicar. f es un modelo de dinámica determinista en el MDP sobre estados dinámicos con torques como acciones. f^{-1} es la **dinámica inversa**, que nos dicen qué torque aplicar si queremos una aceleración particular, lo que lleva a un cambio en la velocidad y, por lo tanto, un cambio en el estado dinámico.

Estado dinámico
Estado cinemático
Dinámica inversa

Ahora, podríamos pensar en $t \in [0,1]$ como "tiempo" en una escala de 0 a 1 y seleccionar nuestro torque usando dinámicas inversas:

$$u(t) = f^{-1}(\tau(t), \dot{\tau}(t), \ddot{\tau}(t)) \tag{26.2}$$

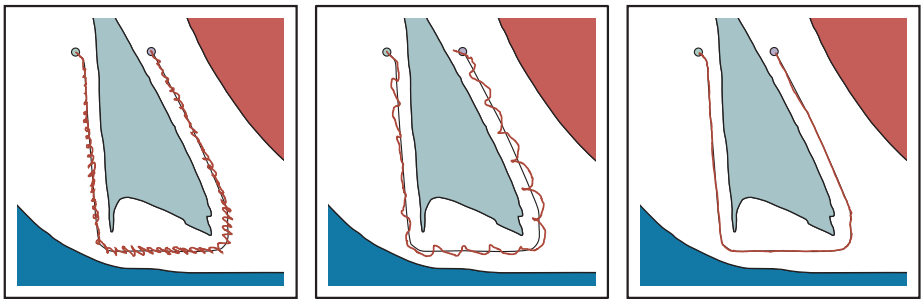
asumiendo que el robot comienza en $(\tau(0), \dot{\tau}(0))$. Sin embargo, en realidad las cosas no son tan fáciles.

La trayectoria τ se creó como una secuencia de puntos, sin tener en cuenta las velocidades y aceleraciones. Como tal, la trayectoria puede no satisfacer $\dot{\tau}(0) = 0$ (el robot comienza a velocidad 0), o incluso que τ sea diferenciable (y mucho menos dos veces diferenciable). Además, el significado del punto final "1" no está claro: ¿a cuántos segundos corresponde eso?

En la práctica, antes incluso de que pensemos en seguir una trayectoria de referencia, solemos **reajustar**, eso es, transformarlo en una trayectoria $\xi(t)$ que mapea el intervalo $[0,T]$ durante algún tiempo T en puntos en el espacio de configuración C. (El símbolo ξ es la letra griega Xi). El reajuste de tiempo es más complicado de lo que podría pensar, pero hay formas aproximadas de hacerlo; por ejemplo, eligiendo una velocidad y aceleración máximas, y usando un perfil que acelera

Reajustar

[2] Omitimos los detalles de f^{-1} aquí, pero involucran masa, inercia, gravedad y fuerzas de Coriolis y centrífugas.



(a) (b) (c)

Figura 26.22 Control del brazo del robot utilizando (a) control proporcional con factor de ganancia 1.0, (b) control proporcional con factor de ganancia 0.1, y (c) control PD (control proporcional derivativo) con factores de ganancia 0.3 para el componente proporcional y 0.8 para el componente diferencial. En todos los casos, el brazo del robot intenta seguir la línea de trayectoria suave, pero en (a) y (b) se desvía de la trayectoria.

hasta esa máxima velocidad, se queda allí tanto tiempo como pueda, y luego desacelera de nuevo a 0. Asumiendo que podemos hacer esto, la Ecuación (26.2) anterior puede ser reescrita como

$$u(t) = f^{-1}(\xi(t), \dot{\xi}(t), \ddot{\xi}(t)). \tag{26.3}$$

Ley de control

Incluso con el cambio de τ a ξ, una trayectoria real, la ecuación de aplicación de torques de arriba (llamada **ley de control**) tiene un problema en la práctica. Pensando en la sección de aprendizaje por refuerzo, podría adivinar cuál es. La ecuación funciona muy bien en la situación donde f es exacta, pero la molesta realidad se interpone como siempre: en sistemas reales, no podemos medir masas e inercias de manera exacta, y f puede no tener en cuenta de manera adecuada fe-

Fricción estática

nómenos físicos como **fricción estática** en los motores (la fricción que tiende a evitar que las superficies estacionarias se pongan en movimiento, para hacerlas pegajosas). Entonces, cuando el brazo del robot comienza a aplicar esos torques pero f está mal, los errores se acumulan y le desvían cada vez más de la trayectoria de referencia.

En vez de sólo dejar que esos errores se acumulen, un robot puede usar un proceso de control que mira dónde cree que está, compara eso con dónde quería estar y aplica un torque para minimizar el error.

Un controlador que proporciona fuerza en proporción negativa al error observado es conocido

Controlador P

como controlador proporcional o**,** para abreviar, **controlador P**. La ecuación para la fuerza es:

$$u(t) = K_P(\xi(t) - q_t)$$

Factor de ganancia

donde q_t es la configuración actual, y K_P es una constante que representa el **factor de ganancia** del controlador. K_P regula cuán fuerte el controlador corrige las desviaciones entre el estado actual q_t y el estado deseado $\xi(t)$.

La Figura 26.22(a) ilustra lo que puede salir mal con el control proporcional. Siempre que una desviación ocurre, ya sea debido al ruido o a las restricciones en las fuerzas que el robot puede aplicar, el robot proporciona una fuerza opuesta cuya magnitud es proporcional a esta desviación. De manera intuitiva, esto podría parecer plausible pues las desviaciones deberían ser compensadas por una contrafuerza para mantener al robot en la pista. Sin embargo, como ilustra la Figura 26.22(a), un controlador proporcional puede hacer que el robot aplique demasiada fuerza, sobrepasando el camino deseado y zigzagueando adelante y atrás. Este es el resultado de la inercia

natural del robot: una vez que vuelve a su posición de referencia, el robot tiene una velocidad que no puede detenerse de manera instantánea.

En la Figura 26.22(a), el parámetro $K_P = 1$. A primera vista, uno podría pensar que elegir un valor más pequeño para K_P solucionaría el problema, dando al robot un enfoque más suave para el camino deseado. Sin embargo, este no es el caso. La Figura 26.22(b) muestra una trayectoria para $K_P = .1$, exhibiendo todavía un comportamiento oscilatorio. El valor más bajo del parámetro de ganancia ayuda, pero no resuelve el problema. De hecho, en ausencia de fricción, el controlador P es en esencia una ley de resorte; por lo que oscilará de manera indefinida alrededor de una ubicación objetivo fija.

Hay una serie de controladores que son superiores a la simple ley de control proporcional. Se dice que un controlador es **estable** si pequeñas perturbaciones conducen a un error acotado entre el robot y la señal de referencia. Se dice que es **estrictamente estable** si es capaz de volver y luego mantenerse en su trayectoria de referencia ante tales perturbaciones. Nuestro controlador P parece ser estable pero no estrictamente estable, ya que no logra mantenerse cerca de su trayectoria de referencia.

Estable

Estrictamente estable

El controlador más simple que logra una estabilidad estricta en nuestro dominio es un **controlador PD**. La letra 'P' vuelve a representar *proporcional*, y 'D' representa *derivativo*. Los controladores PD se describen con la siguiente ecuación:

Controlador PD

$$u(t) = K_P(\xi(t) - q_t) + K_D(\dot{\xi}(t) - \dot{q}_t). \tag{26.4}$$

Como sugiere esta ecuación, los controladores PD extienden los controladores P por un componente diferencial, que agrega al valor de $u(t)$ un término que es proporcional a la primera derivada del error $\xi(t) - q_t$ a lo largo del tiempo. ¿Cuál es el efecto de tal término? En general, un término derivativo amortigua el sistema que se está controlando. Para ver esto, considere una situación donde el error está cambiando con el tiempo, como es el caso de nuestro controlador P anterior. La derivada de esto fallará y entonces contrarrestará el término proporcional, lo que reducirá la respuesta general a la perturbación. No obstante, si el mismo error persiste y no cambia, la derivada desaparecerá y el término proporcional domina la elección del control.

La Figura 26.22(c) muestra el resultado de aplicar este controlador PD a nuestro brazo de robot, utilizando como parámetros de ganancia $K_P = .3$ y $K_D = .8$. Note que la trayectoria resultante es mucho más suave y no exhibe ninguna oscilación obvia.

Sin embargo, los controladores PD sí tienen modos de fallar. En particular, los controladores PD pueden fallar en regular un error a cero, aun en ausencia de perturbaciones externas. En la mayoría de los casos, tal situación es el resultado de una fuerza externa sistemática que no es parte del modelo. Por ejemplo, un vehículo autónomo que conduce en una superficie inclinada puede encontrarse de manera sistemática empujado hacia un lado. El desgaste en los brazos de los robots causa errores sistemáticos similares. En tales situaciones, se requiere retroalimentación sobreproporcional para acercar el error a cero. La solución a este problema radica en agregar un tercer término a la ley de control, basado en el error *integrado* a lo largo del tiempo:

$$u(t) = K_P(\xi(t) - q_t) + K_I \int_0^t (\xi(s) - q_s)ds + K_D(\xi(t) - \dot{q}_t). \tag{26.5}$$

Aquí K_I es un tercer parámetro de ganancia. El término $\int_0^t (\xi(s)$ calcula la integral del error a lo largo del tiempo. El efecto de este término es que se corrigen las desviaciones duraderas entre la señal de referencia y el estado real. Los términos integrales, entonces, aseguran que un controlador no exhiba un error sistemático a largo plazo, aunque sí representan un peligro de comportamiento oscilatorio.

Un controlador con los tres términos se llama **controlador PID** (para proporcional integral derivada). Los controladores PID se utilizan en la industria, para una variedad de problemas de control. Piense en los tres términos de la siguiente manera: proporcional: intenta más cuanto más

Controlador PID

lejos esté de la trayectoria; derivada: intenta aún más si el error está aumentando; integral: intenta más si no ha progresado durante mucho tiempo.

Control de par de torsión calculado

Un término medio entre el control de bucle abierto basado en dinámicas inversas y el control PID de bucle cerrado se llama **control de par de torsión calculado**. Calculamos el par de torsión que nuestro modelo piensa que necesitaremos, pero compensamos la inexactitud del modelo con términos de error proporcionales:

$$u(t) = \underbrace{f^{-1}(\xi(t), \dot{\xi}(t), \ddot{\xi}(t))}_{\text{alimentación}} + \underbrace{m(\xi(t))\,(K_P(\xi(t) - q_t)) + K_D(\dot{\xi}(t) - \dot{q}_t)}_{\text{retroalimentación}}. \tag{26.6}$$

Componente de avance

Componente de retroalimentación

El primer término se llama **componente de avance** porque mira hacia dónde necesita ir el robot y calcula qué par de torsión podría ser necesario. El segundo es el **componente de retroalimentación** porque retroalimenta el error actual en el estado dinámico en la ley de control. $m(q)$ es la matriz de inercia en la configuración q, a diferencia del control PD normal, las ganancias cambian con la configuración del sistema.

Planes versus políticas

Retrocedamos un paso y asegurémonos de entender la analogía entre lo que ha sucedido hasta ahora en este capítulo y lo que aprendimos en los capítulos de búsqueda, MDP y aprendizaje por refuerzo. Con el movimiento en robótica, realmente estamos considerando un MDP subyacente donde los estados son estados dinámicos (configuración y velocidad), y las acciones son entradas de control, por lo general en forma de torques. Si vuelve a mirar nuestras leyes de control arriba, son *políticas*, no *planes*, las que le dicen al robot qué acción tomar desde *cualquier* estado que pueda alcanzar. Sin embargo, suelen estar lejos de ser políticas *óptimas*. Debido a que el estado dinámico es continuo y de alta dimensión (al igual que el espacio de acción), las políticas óptimas son difíciles de extraer.

En lugar de eso, lo que hicimos aquí es descomponer el problema. Primero elaboramos un plan en un estado y espacio de acción simplificados: usamos sólo el estado cinemático, y asumimos que los estados son alcanzables entre sí, sin prestar atención a las dinámicas subyacentes. Esto es la planificación del movimiento, y nos da la trayectoria de referencia. Si conocíamos las dinámicas a la perfección, podríamos convertir esto en un plan para el estado original y el espacio de acción con la Ecuación (26.3).

Pero como nuestro modelo de dinámicas es con frecuencia erróneo, lo convertimos en una política que intenta seguir el plan, volviendo a él cuando se desvía. Al hacer esto, introducimos suboptimalidad de dos maneras: primero planificando sin considerar las dinámicas, y segundo asumiendo que si nos desviamos del plan, lo óptimo es volver al plan original. En lo que sigue, describimos técnicas que calculan políticas de manera directa sobre el estado dinámico, evitando la separación por completo.

26.5.4 Control óptimo

En lugar de usar un planificador para crear una trayectoria cinemática, y sólo preocuparnos por las dinámicas del sistema después del hecho, aquí discutimos cómo podríamos ser capaces de hacerlo todo a la vez. Tomaremos el problema de optimización de trayectoria para trayectorias cinemáticas, y lo convertiremos en verdadera optimización de trayectoria con dinámicas: optimizaremos de manera directa sobre las acciones, teniendo en cuenta las dinámicas (o transiciones).

Esto nos acerca mucho más a lo que hemos visto en los capítulos de búsqueda y MDP. Si conocemos las dinámicas del sistema, entonces podemos encontrar una secuencia de acciones para ejecutar, como lo hicimos en el Capítulo 3. Si no estamos seguros, entonces podríamos querer una política, como en el Capítulo 17.

En esta sección, estamos mirando el MDP subyacente en el que trabaja el robot. Estamos pasando de los MDP discretos familiares a los continuos. Denotaremos nuestro estado dinámico del mundo por x, como es práctica común, el equivalente de s en MDP discretos. Dejemos que x_s y x_g sean los estados de inicio y objetivo.

Queremos encontrar una secuencia de acciones que, cuando son ejecutadas por el robot, resulten en pares de estado-acción con un costo acumulativo bajo. Las acciones son torques que denotamos con $u(t)$ para t empezando en 0 y terminando en T. De manera formal, queremos encontrar la secuencia de torques u que minimice un coste acumulativo J:

$$\min_u \int_0^T J(x(t),u(t))dt \tag{26.7}$$

sujeto a las restricciones

$$\forall t, \dot{x}(t) = f(x(t),u(t))$$
$$x(0) = x_s,\ x(T) = x_g.$$

¿Cómo está esto conectado con la planificación del movimiento y el control de seguimiento de trayectoria? Bueno, imagine que tomamos la noción de eficiencia y despeje de los obstáculos y la ponemos en la función de costo J, tal como lo hicimos antes en la optimización de trayectoria sobre el estado cinemático. El estado dinámico es la configuración y la velocidad, y los torques u lo cambian a través de las dinámicas f del seguimiento de trayectoria en bucle abierto. La diferencia es que ahora estamos pensando en las configuraciones y los torques al mismo tiempo. A veces es posible que deseemos tratar la prevención de colisiones como una restricción dura también, algo que además hemos mencionado antes cuando miramos la optimización de trayectoria para el estado cinemático solamente.

Para resolver este problema de optimización, podemos tomar gradientes de J, no con respecto a la secuencia τ de configuraciones, sino de manera directa con respecto a los controles u. A veces es útil incluir la secuencia de estados x como una variable de decisión también, y usar las restricciones de dinámicas para asegurar que x y u son consistentes. Hay varias técnicas de optimización de trayectoria usando este enfoque; dos de ellas se conocen por los nombres de **disparo múltiple** y **colocación directa**. Ninguna de estas técnicas encontrará la solución óptima global, pero en la práctica es posible que los robots humanoides caminen y que los vehículos autónomos conduzcan.

La magia ocurre cuando en el problema de arriba, J es cuadrática y f es lineal en x y u. Queremos minimizar

$$\min \int_0^\infty x^T Q x + u^T R u dt \qquad \text{sujeto a} \qquad \forall t, \dot{x}(t) = Ax(t) + Bu(t).$$

Podemos optimizar sobre un horizonte infinito en lugar de uno finito, y obtenemos una política desde cualquier estado en lugar de sólo una secuencia de controles. Q y R necesitan ser definidos como matrices para que esto funcione. Esto nos da el **regulador cuadrático lineal (LQR)**. Con LQR, la función de valor óptimo (llamada **costo a seguir**) es cuadrática, y la política óptima es lineal. La política parece $u = -Kx$, donde encontrar la matriz K requiere resolver una algebraica **Ecuación de Riccati**, no se necesita optimización local, no se necesita iteración de valor, no se necesita iteración de política!

Regulador cuadrático lineal (LQR)

Ecuación de Riccati

Debido a la facilidad de encontrar la política óptima, LQR encuentra muchos usos en la práctica a pesar del hecho de que los problemas reales rara vez tienen costos cuadráticos y dinámicas lineales. Un método útil se llama **LQR iterativo (ILQR)**, que funciona comenzando con una solución y luego calculando de manera iterativa una aproximación lineal de la dinámica y una aproximación cuadrática del costo alrededor de ella, luego resolviendo el sistema LQR resultante para llegar a una nueva solución. Las variantes de LQR también se utilizan para el seguimiento de trayectorias.

LQR iterativo (ILQR)

26.6 Planificación de movimientos inciertos

En robótica, la incertidumbre surge de la observabilidad parcial del entorno y de los efectos estocásticos (o no modelados) de las acciones del robot. Los errores también surgen del uso de algoritmos de aproximación como el filtrado de partículas, que no proporciona al robot un estado de creencia exacto aun si el entorno está modelado de manera perfecta.

La mayoría de los robots de hoy en día utilizan algoritmos deterministas para la toma de decisiones, como los algoritmos de planificación de trayectorias de la sección anterior o los algoritmos de búsqueda introducidos en el Capítulo 3. Estos algoritmos deterministas se adaptan de dos formas: primero, tratan con el espacio de estado continuo convirtiéndolo en un espacio discreto (por ejemplo, con grafos de visibilidad o descomposición de celdas). En segundo lugar, tratan con la incertidumbre en el estado actual eligiendo el **estado más probable** de la distribución de probabilidad producida por el algoritmo de estimación de estado. Ese enfoque hace que el cálculo sea más rápido y se ajuste mejor a los algoritmos de búsqueda deterministas. En esta sección discutimos métodos para tratar con la incertidumbre que son análogos a los algoritmos de búsqueda más complejos cubiertos en el Capítulo 4.

Estado más probable

Primero, en lugar de planes deterministas, la incertidumbre requiere políticas. Ya discutimos cómo el control de seguimiento de trayectoria convierte un plan en una política para compensar los errores en la dinámica. Sin embargo, a veces si la hipótesis más probable cambia lo suficiente, seguir el plan diseñado para una hipótesis diferente es demasiado subóptimo. Aquí es donde entra la **replanificación en línea**: podemos recalcular un nuevo plan basado en la nueva creencia. Muchos robots hoy en día utilizan una técnica llamada **control predictivo del modelo (MPC)**, donde planifican para un horizonte de tiempo más corto, pero replanean en cada paso de tiempo. (MPC está, por lo tanto, relacionado con los algoritmos de búsqueda en tiempo real y los algoritmos de juego). Esto resulta en una política: en cada paso, ejecutamos un planificador y tomamos la primera acción en el plan; si llega nueva información o terminamos no donde esperábamos, eso está bien, porque vamos a replanificar de todos modos y eso nos dirá qué hacer a continuación.

Replanificación en línea

Control predictivo del modelo (MPC)

En segundo lugar, la incertidumbre requiere acciones de **recopilación de información**. Cuando consideramos sólo la información que tenemos y hacemos un plan basado en ella (esto se llama separar la estimación del control), estamos resolviendo (de manera aproximada) un nuevo MDP en cada paso, correspondiente a nuestra creencia actual sobre dónde estamos o cómo funciona el mundo. Pero, en realidad, la incertidumbre se captura mejor con el marco de POMDP: hay algo que no observamos directamente, ya sea la ubicación o configuración del robot, la ubicación de los objetos en el mundo o los parámetros del modelo de dinámica en sí; por ejemplo, ¿dónde está el centro de masa del enlace dos en este brazo?

Lo que perdemos cuando no resolvemos el POMDP es la capacidad de razonar sobre la *futura información* que obtendrá el robot: en los MDP sólo planificamos con lo que sabemos, no con lo que de manera eventual *podríamos* saber. ¿Recuerda el valor de la información? Bueno, los robots que planifican usando su creencia actual como si nunca fueran a descubrir nada más, no tienen en cuenta el valor de la información. Nunca tomarán acciones que parezcan subóptimas en este momento según lo que saben, pero que en realidad resultarán en mucha información y permitirán al robot hacerlo bien.

¿Cómo se ve tal acción para un robot de navegación? El robot podría acercarse a un hito para obtener una mejor estimación de dónde está, incluso si ese hito está fuera de camino según el conocimiento actual. Esta acción es óptima sólo si el robot considera las nuevas observaciones que obtendrá, en lugar de mirar sólo la información que ya tiene.

Para evitar esto, las técnicas de robótica a veces definen acciones de recolección de información de manera explícita, como mover una mano hasta que toque una superficie (**movimientos guardados**); y asegurarse de que el robot haga eso antes de idear un plan para alcanzar su objetivo real.

Movimiento guardado

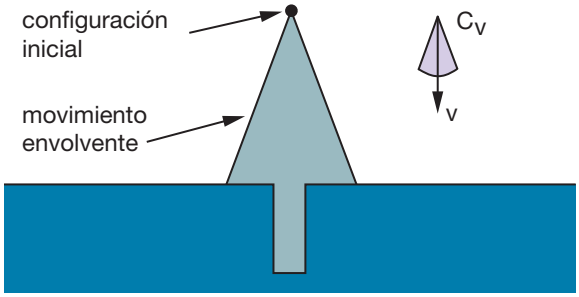


Figura 26.23 Un entorno bidimensional, cono de incertidumbre de velocidad y cubierta de posibles movimientos del robot. La velocidad pretendida es *v*, pero con incertidumbre la velocidad real podría estar en cualquier lugar en C_v, resultando en una configuración final en algún lugar del sobre de movimiento, lo que significa que no sabríamos si golpeamos el agujero o no.

Cada movimiento guardado consta de (1) un comando de movimiento y (2) una condición de terminación, que es un predicado sobre los valores del sensor del robot que indica cuándo detenerse.

A veces, el propio objetivo podría alcanzarse mediante una secuencia de movimientos guardados garantizados para tener éxito, independiente de la incertidumbre. Como ejemplo, la Figura 26.23 muestra un espacio de configuración bidimensional con un agujero vertical estrecho. Podría ser el espacio de configuración para la inserción de una clavija rectangular en un agujero o una llave de automóvil en el encendido. Los comandos de movimiento son velocidades constantes. Las condiciones de terminación son el contacto con una superficie. Para modelar la incertidumbre en el control, asumimos que en lugar de moverse en la dirección ordenada, el movimiento real del robot se encuentra en el cono C_v alrededor de él.

La figura muestra qué pasaría si el robot intentara moverse de manera directa hacia abajo desde la configuración inicial. Debido a la incertidumbre en la velocidad, el robot podría moverse en cualquier lugar dentro del cono envolvente, quizás entrando en el agujero, pero más probablemente aterrizando a un lado de él. Debido a que el robot no sabría entonces en qué lado del agujero está, no sabría hacia dónde moverse.

Una estrategia más sensata se muestra en las Figuras 26.24 y 26.25. En la Figura 26.24 el robot se mueve de manera deliberada a un lado del agujero. El comando de movimiento se muestra en la figura, y la prueba de terminación es el contacto con cualquier superficie. En la Figura 26.25 se da un comando de movimiento que hace que el robot se deslice a lo largo de la superficie y entre en el agujero. Debido a que todas las velocidades posibles en la cubierta de movimiento están a la derecha, el robot se deslizará a la derecha siempre que esté en contacto con una superficie horizontal.

Se deslizará por el borde vertical derecho del agujero cuando lo toque, porque todas las velocidades posibles están hacia abajo en relación con una superficie vertical. Seguirá moviéndose hasta que alcance el fondo del agujero, porque esa es su condición de terminación. A pesar de la incertidumbre de control, todas las trayectorias posibles del robot terminan en contacto con el fondo del agujero; es decir, a menos que las irregularidades de la superficie hagan que el robot se quede en un lugar.

Otras técnicas más allá de los movimientos guardados cambian la función de costo para incentivar acciones que sabemos que llevarán a información, como la heurística de **navegación de frontera** que requiere el robot para permanecer cerca de los hitos conocidos. Más en general, las técnicas permiten incorporar la ganancia de **información esperada** (reducción de la entropía de la creencia) como un término en la función de costo, lo que lleva al robot a razonar de manera

Navegación de frontera

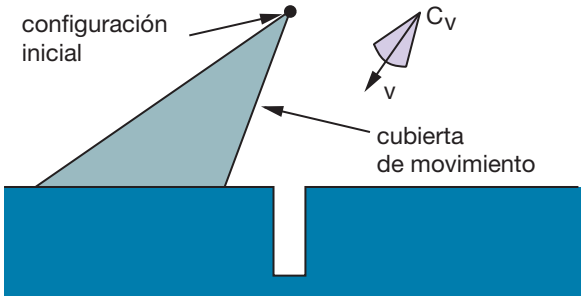


Figura 26.24 El primer comando de movimiento y la cubierta resultante de posibles movimientos del robot. No importa qué movimiento real se produzca, sabemos que la configuración final estará a la izquierda del agujero.

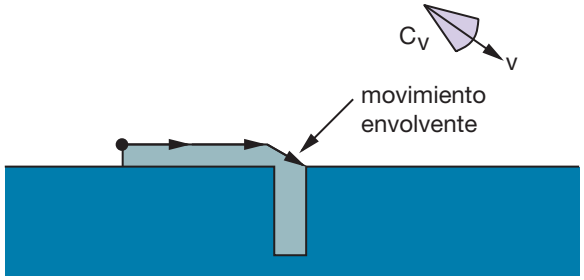


Figura 26.25 El segundo comando de movimiento y la envolvente de posibles movimientos. Incluso con error, de manera eventual entraremos en el agujero.

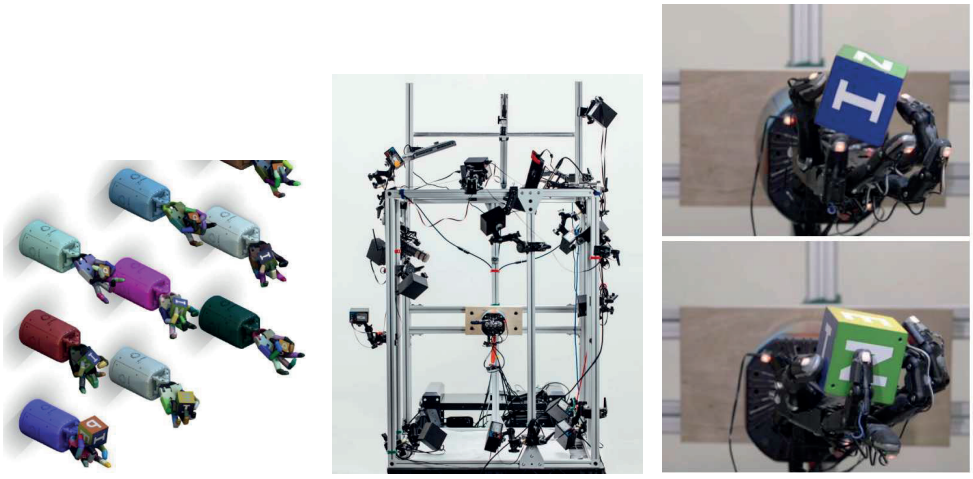
explícita sobre cuánta información aporta cada acción al decidir qué hacer. Aunque son más difíciles computacionalmente, estos enfoques tienen la ventaja de que el robot inventa sus propias acciones de recolección de información en lugar de depender de heurísticas proporcionadas por humanos y estrategias predefinidas que en la mayoría de los casos carecen de flexibilidad.

26.7 Aprendizaje por refuerzo en robótica

Hasta ahora hemos considerado tareas en las que el robot tiene acceso al modelo dinámico del mundo. En muchas tareas, es muy difícil escribir tal modelo, lo que nos sitúa en el dominio del aprendizaje por refuerzo (RL).

Un desafío del RL en robótica es la naturaleza continua de los espacios de estado y acción, que manejamos a través de la discretización o, con frecuencia, a través de la aproximación de funciones. Las políticas o funciones de valor se representan como combinaciones de características útiles conocidas o como redes neuronales profundas. Las redes neuronales permiten mapear de entradas brutas de manera directa a salidas, y así evitar en gran medida la necesidad de ingeniería de características, pero requieren más datos.

Un desafío mayor es que los robots operan en el mundo real. Hemos visto cómo el aprendizaje por refuerzo permite ser usado para aprender a jugar partidas simuladas de ajedrez o Go. Pero



(a) (b) (c)

Figura 26.26 Entrenando una política robusta. (a) Se ejecutan múltiples simulaciones de una mano de robot manipulando objetos con diferentes parámetros aleatorizados para la física y la iluminación. (Imagen cortesía de Wojciech Zaremba). (b) Entorno del mundo real, con una sola mano de robot en el centro de una jaula, rodeada de cámaras y telémetros. (c) La simulación y el entrenamiento del mundo real producen múltiples y diferentes políticas para agarrar objetos; aquí una pinza de pellizco y una pinza de cuatro pinzas. (Imagen cortesía de OpenAI. Vea Andrychowicz *et al.* (2018a)).

cuando un robot real se mueve en el mundo real, tenemos que asegurarnos de que sus acciones sean seguras (sin cosas ¡rotas!), y tenemos que aceptar que el progreso será más lento que en una simulación porque el mundo se niega a moverse más rápido que de un segundo por segundo. Gran parte de lo interesante de usar el aprendizaje por refuerzo en robótica se reduce a cómo podríamos reducir la complejidad de la muestra del mundo real, el número de interacciones con el mundo físico que el robot necesita antes de que haya aprendido cómo hacer la tarea.

26.7.1 Explotando modelos

Una forma natural de evitar la necesidad de muchas muestras del mundo real es utilizar tanto conocimiento de la dinámica del mundo como sea posible. Por ejemplo, es posible que no sepamos cuál es el coeficiente de fricción o la masa de un objeto, pero podríamos tener ecuaciones que describen la dinámica en función de estos parámetros.

En tal caso, el **aprendizaje por refuerzo basado en modelos** (Capítulo 22) es atractivo, donde el robot permite alternar entre ajustar los parámetros de la dinámica y calcular una mejor política. Incluso si las ecuaciones son incorrectas porque no logran modelar cada detalle de la física, los investigadores han experimentado con el aprendizaje de un término de error, además de los parámetros, que puede compensar la inexactitud del modelo físico. O, podemos abandonar las ecuaciones y, en su lugar, ajustar modelos lineales locales del mundo que cada uno aproxime la dinámica en una región del espacio de estado, un enfoque que ha tenido éxito en hacer que los robots dominen tareas dinámicas complejas como el malabarismo.

Un modelo del mundo también es útil para reducir la complejidad de la muestra de los métodos de aprendizaje por refuerzo sin modelo al hacer la transferencia **de la simulación a la realidad**:

De la simulación a la realidad

transferir políticas que funcionan en simulación al mundo real. La idea es usar el modelo como un simulador para una búsqueda de políticas (Sección 22.5). Para aprender una política que se transfiera bien, podemos agregar ruido al modelo durante el entrenamiento, lo que hace que la política sea más robusta. O, podemos entrenar políticas que funcionarán con una *variedad* de modelos muestreando diferentes parámetros en las simulaciones, a veces referido como **aleatorización de dominio**. Un ejemplo se observa en la Figura 26.26, donde una tarea de manipulación diestra se entrena en simulación variando atributos visuales, así como atributos físicos como fricción o amortiguación.

Aleatorización de dominio

Finalmente, los enfoques híbridos que toman ideas de ambos algoritmos basados en modelos y libres de modelos están destinados a darnos lo mejor de ambos. El enfoque híbrido se originó con la arquitectura Dyna, donde la idea era iterar entre actuar y mejorar la política, pero la mejora de la política vendría de dos formas complementarias: 1) la forma estándar sin modelo, de usar la experiencia para actualizar de manera directa la política, y 2) la forma basada en modelos, de usar la experiencia para ajustar un modelo, luego planificar con él para generar una política.

Técnicas más recientes han experimentado con la adaptación de modelos locales, planificando con ellos para generar acciones, y utilizando estas acciones como supervisión para ajustar una política, luego iterando para obtener modelos cada vez mejores alrededor de las áreas que la política necesita. Esto se ha aplicado con éxito en el **aprendizaje de extremo a extremo**, donde la política toma pixeles como entrada y emite torques como acciones, lo que permitió la primera demostración de RL profundo en robots físicos.

Los modelos también permiten ser explotados con el propósito de garantizar una **exploración segura**. Aprender de manera lenta pero segura es mejor que aprender rápidamente pero estrellarse y quemarse a mitad del camino. Más cuestionable aún que reducir muestras del mundo real es reducir muestras del mundo real en estados *peligrosos*, no queremos que los robots se caigan de los acantilados y no queremos que rompan nuestras tazas favoritas o, peor aún, colisionen con objetos y personas. Un modelo aproximado con incertidumbre asociada a él (por ejemplo, considerando un rango de valores para sus parámetros) puede guiar la exploración e imponer restricciones en las acciones que se permite tomar al robot para evitar estos estados peligrosos. Esta es un área de investigación activa en robótica y control.

26.7.2 Explotando otra información

Los modelos son útiles pero hay más que podemos hacer para reducir aún más la complejidad de la muestra.

Al configurar un problema de aprendizaje por refuerzo tenemos que seleccionar los espacios de estado y acción, la representación de la política o función de valor y la función de recompensa que estamos utilizando. Estas decisiones tienen un gran impacto en cuán fácil o difícil estamos haciendo el problema.

Primitiva de movimiento

Un enfoque es usar **primitivas de movimiento** de nivel superior en lugar de acciones de bajo nivel como comandos de torque. Una primitiva de movimiento es una habilidad parametrizada que tiene el robot. Por ejemplo, un jugador de fútbol robótico podría tener la habilidad de "pasar el balón al jugador en (x,y)". Todo lo que la política necesita hacer es averiguar cómo combinarlos y establecer sus parámetros, en lugar de reinventarlos. Este enfoque aprende mucho más rápido que los enfoques de bajo nivel, pero sí restringe el espacio de posibles comportamientos que el robot puede aprender.

Otra forma de reducir el número de muestras del mundo real necesarias para el aprendizaje es reutilizar información de episodios de aprendizaje anteriores en otras tareas, en lugar de comenzar desde cero. Esto cae bajo el paraguas del **metaprendizaje** o **aprendizaje de transferencia**.

Finalmente, las personas son una gran fuente de información. En la siguiente sección nos referiremos a cómo interactuar con las personas, y parte de ello es cómo usar sus acciones para guiar el aprendizaje del robot.

26.8 Humanos y robots

Hasta ahora nos hemos centrado en un robot que planea y aprende cómo actuar *en aislamiento*. Esto es útil para algunos robots, como los vehículos exploradores que enviamos para investigar planetas distantes en nuestro nombre. Aunque, en general, no construimos robots para trabajar en aislamiento; los construimos para ayudarnos, para trabajar en entornos humanos, alrededor y con nosotros.

Esto plantea dos desafíos complementarios. El primero es optimizar la recompensa cuando hay personas actuando en el mismo entorno que el robot. Llamamos a esto el **problema de coordinación** (ver Sección 18.1). Cuando la recompensa del robot depende no sólo de sus propias acciones, sino también de las acciones que toman las personas, el robot tiene que elegir sus acciones de una manera que se combine bien con las de ellas. Cuando el humano y el robot están en el mismo equipo, eso se convierte en **colaboración**.

El segundo es el desafío de optimizar para lo que la gente realmente quiere. Si un robot va a ayudar a las personas, su función de recompensa necesita incentivar las acciones que las personas quieren que el robot ejecute. Descubrir la función de recompensa correcta (o política) para el robot es en sí mismo un problema de interacción. Exploraremos estos dos desafíos por turnos.

26.8.1 Coordinación

Suponga por ahora, como lo hemos estado haciendo, que el robot tiene acceso a una función de recompensa claramente definida. Pero, en lugar de necesitar optimizarlo en aislamiento, ahora el robot necesita optimizarlo alrededor de un humano que también está actuando. Por ejemplo, cuando un vehículo autónomo se incorpora a la autopista, necesita negociar la maniobra con el conductor humano que viene en el carril objetivo, ¿debería acelerar e integrarse al frente o reducir la velocidad e incorporarse detrás? Más tarde, cuando se detiene en una señal de alto, al prepararse para girar a la derecha, tiene que estar atento al ciclista en el carril de bicicletas y al peatón a punto de pisar el paso peatonal.

O, considere un robot móvil en un pasillo. Alguien que se dirige de manera directa hacia el robot da un pequeño paso a la derecha, indicando de qué lado del robot quiere pasar. El robot tiene que responder, aclarando sus intenciones.

Humanos como agentes poco más o menos racionales

Una forma de formular la coordinación con un humano es modelarlo como un juego entre el robot y el humano (Sección 18.2). Con este enfoque hacemos de manera explícita la suposición de que las personas son agentes incentivados por objetivos. Esto no significa de manera automática que sean agentes perfectamente racionales (es decir, que encuentren soluciones óptimas en el juego), pero sí significa que el robot permite estructurar la forma en que razona acerca del humano mediante la noción de posibles objetivos que el humano podría tener. En este juego:

- al estado del entorno captura las configuraciones de ambos agentes, el robot y el humano; lo llamamos $x = (x_R, x_H)$;
- cada agente puede tomar acciones, u_R y u_H respectivamente;
- cada agente tiene un objetivo que permite representarse como un costo, J_R y J_H: cada agente quiere llegar a su objetivo de manera segura y eficiente;
- y, como en cualquier juego, cada objetivo depende del estado y de las acciones de *ambos* agentes: $J_R(x, u_R, u_H)$ y $J_H(x, u_H, u_R)$. Piense en la interacción vehículos-peatón: el vehículo debería parar si el peatón cruza, y debería avanzar si el peatón espera.

Tres aspectos importantes complican este juego. Primero, el humano y el robot no necesariamente conocen los objetivos del otro. Esto lo convierte en un **juego de información incompleta**.

Juego de información incompleta

Segundo, el estado y los espacios de acción son *continuos*, como lo han sido a lo largo de este capítulo. Aprendimos en el Capítulo 5 cómo hacer una búsqueda en árbol para abordar juegos discretos, pero ¿cómo abordamos espacios continuos?

Tercero, aunque a nivel alto el modelo de juego tiene sentido, los humanos se mueven y tienen objetivos, el comportamiento de un humano es posible que no siempre este bien caracterizado como una solución al juego. El juego viene con un desafío computacional no sólo para el robot, sino también para nosotros, los humanos. Requiere pensar en lo que el robot hará en respuesta a lo que la persona hace, lo que depende de lo que el robot piensa que la persona hará, y pronto llegamos a "¿qué crees que pienso que piensas que pienso?", ¡son tortugas todo el camino hacia abajo! Los humanos no pueden lidiar con todo eso y muestran ciertas suboptimalidades. Esto significa que el robot debería tener en cuenta tales suboptimalidades.

Entonces ¿qué debe hacer un vehículo autónomo cuando el problema de coordinación es tan difícil? Haremos algo similar a lo que hemos hecho antes en este capítulo. Para la planificación y control del movimiento, tomamos un MDP y lo dividimos en planificar una trayectoria y luego seguirla con un controlador. Aquí también tomaremos el juego y lo dividiremos en hacer predicciones sobre las acciones humanas, y en decidir qué debe hacer el robot dadas estas predicciones.

Prediciendo la acción humana

Predecir las acciones humanas es difícil porque dependen de las acciones del robot y viceversa. Un truco que utilizan los robots es pretender que la persona está ignorando al robot. El robot asume que las personas son óptimas con ruido con respecto a su objetivo, que es desconocido para el robot y que se modela como ya no dependiente de las acciones del robot: $J_H(x,u_H)$. En particular, cuanto mayor es el valor de una acción para el objetivo (menor es el costo para continuar), más probable es que el humano la tome. El robot permite crear un modelo para $P(u_H|x,J_H)$; por ejemplo, utilizando la función softmax de la página 811:

$$P(u_H|x,J_H) \propto e^{-Q(x,u_H;J_H)} \tag{26.8}$$

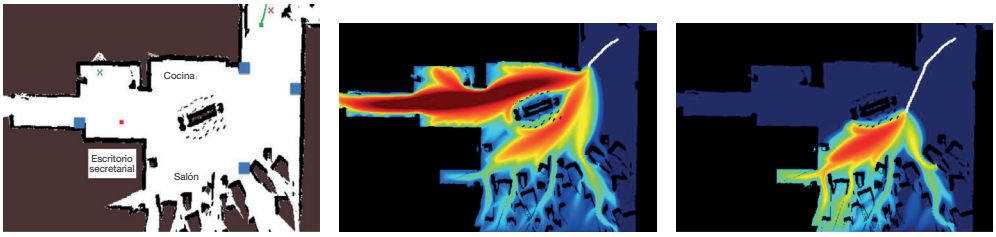
con $Q(x,u_H;J_H)$ la función de valor Q correspondiente a J_H (el signo negativo está ahí porque en robótica nos gusta minimizar el costo, no maximizar la recompensa). Tenga en cuenta que el robot no asume acciones óptimas, ni asume que las acciones se eligen en función del razonamiento sobre el robot en absoluto.

Armado con este modelo, el robot utiliza las acciones en curso del humano como evidencia sobre J_H. Si tenemos un modelo de observación de cómo las acciones humanas dependen del objetivo del humano, cada acción humana puede incorporarse para actualizar la creencia del robot sobre qué objetivo tiene la persona:

$$b'(J_H) \propto b(J_H)P(u_H|x,J_H). \tag{26.9}$$

Un ejemplo lo encontramos en la Figura 26.27: el robot está rastreando la ubicación de un humano y, a medida que el humano se mueve, el robot actualiza su creencia sobre los objetivos humanos. Si el humano se dirige hacia las ventanas, el robot aumenta la probabilidad de que el objetivo sea mirar por la ventana, y disminuye la probabilidad de que el objetivo sea ir a la cocina, que está en la otra dirección.

Así es como las acciones pasadas del humano terminan informando al robot sobre lo que el humano hará en el futuro. Tener una creencia sobre el objetivo del humano ayuda al robot a anticipar qué próximas acciones tomará el humano. El mapa de calor en la figura muestra que el robot predice el futuro: el rojo es el más probable; el azul el menos probable.



(a) (b) (c)

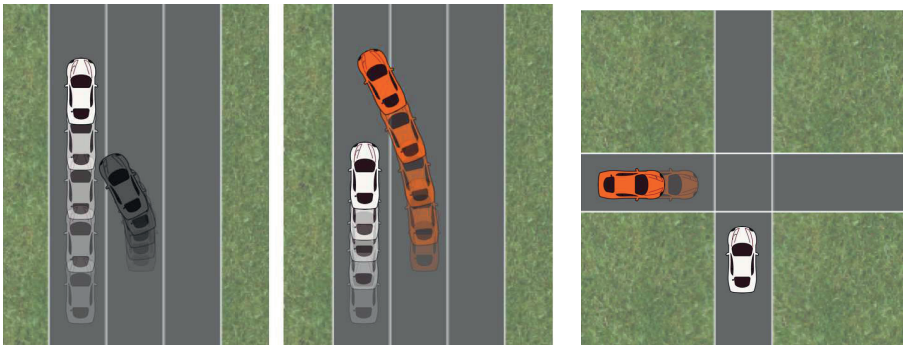
Figura 26.27 Hacer predicciones al asumir que las personas son ruidosamente racionales dada su meta: el robot usa las acciones pasadas para actualizar una creencia sobre a qué meta se dirige la persona, y luego usa la creencia para hacer predicciones sobre acciones futuras. (a) Mapa de una habitación. (b) Predicciones después de ver una pequeña parte de la trayectoria de la persona (camino blanco); (c) Predicciones después de ver más acciones humanas: el robot ahora sabe que la persona no se dirige al pasillo de la izquierda, porque el camino tomado hasta ahora sería un mal camino, si ese fuera el objetivo de la persona. (Imágenes cortesía de Brian D. Ziebart. Ver Ziebart *et al.* (2009)).

Lo mismo sucede al conducir. Es posible que no sepamos cuánto valora otro conductor la eficiencia, pero si vemos que aceleran cuando alguien está tratando de incorporarse delante de ellos, ahora sabemos un poco más sobre ellos. Una vez que sabemos eso, podemos anticipar mejor lo que harán en el futuro: es probable que el mismo conductor se acerque más detrás de nosotros o se abra paso entre el tráfico para adelantarse.

Cuando el robot hace predicciones sobre las futuras acciones humanas, ha reducido su problema a resolver un MDP. Las acciones humanas complican la función de transición, pero siempre que el robot pueda anticipar qué acción tomará la persona desde cualquier estado futuro, el robot puede calcular $P(x'|x,u_R)$: permite calcular $P(u_H|x)$ a partir de $P(u_H|x,J_H)$ al marginar J_H, y combinarlo con $P(x'|x,u_R,u_H)$, la función de transición (dinámica) de cómo se actualiza el mundo en función de las acciones del robot y del humano. En la Sección 26.5 nos centramos en cómo resolver esto en espacios de estado y acción continuos para dinámicas deterministas, en la Sección 26.6 discutimos cómo hacerlo con dinámicas estocásticas e incertidumbre.

Separar la predicción de la acción facilita al robot el manejo de la interacción, pero sacrifica el rendimiento tanto como separar la estimación del movimiento, o separar la planificación del control.

Un robot con esta división ya no comprende que sus acciones influyen en lo que las personas terminan haciendo. En contraste, el robot en la Figura 26.27 anticipa a dónde irán las personas y luego optimiza para alcanzar su propio objetivo y evitar colisiones con ellas. En la Figura 26.28 se observa un vehículo autónomo incorporándose a la autopista. Si sólo planificara en reacción a otros vehículos, podría esperar mucho tiempo mientras otros vehículos ocupan su carril objetivo. En contraste, un vehículo que razona en conjunto sobre la predicción y la acción, sabe que las diferentes acciones que podría tomar resultarán en diferentes reacciones por parte del humano. Si empieza a enfilarse, es probable que los otros vehículos reduzcan un poco la velocidad y hagan espacio. Los roboticistas están trabajando hacia interacciones coordinadas como esta para que los robots puedan trabajar mejor con los humanos.



(a) (b)

Figura 26.28 (a) Izquierda: un vehículo autónomo (carril central) predice que el conductor humano (carril izquierdo) quiere seguir adelante, y planea una trayectoria que reduce la velocidad y se integra detrás. Derecha: el vehículo tiene en cuenta la influencia que sus acciones tienen en las acciones humanas, y se da cuenta de que puede integrarse adelante y confiar en que el conductor humano reducirá la velocidad. (b) Ese mismo algoritmo produce una estrategia inusual en una intersección: el vehículo se da cuenta de que hace que la persona (abajo) avance más rápido por la intersección si retrocede un poco. (Imágenes cortesía de Anca Dragan. Ver Sadigh *et al.* (2016)).

Predicciones humanas sobre el robot

La información incompleta es bidireccional: el robot no conoce el objetivo del humano y el humano, a su vez, no conoce el objetivo del *robot*, las personas necesitan hacer predicciones sobre los robots. Como diseñadores de robots, no estamos a cargo de cómo hace predicciones el ser humano, sólo podemos controlar lo que hace el robot. Sin embargo, el robot permite actuar de manera que sea más *fácil* para el humano hacer predicciones correctas. El robot puede suponer que el humano está usando algo aproximadamente análogo a la Ecuación (26.8) para estimar el objetivo del robot J_R, y así el robot actuará de manera que su verdadero objetivo pueda ser inferido.

Un caso especial del juego es cuando el humano y el robot están en el mismo equipo, trabajando hacia el mismo objetivo: $J_H = J_R$. Imagine tener un robot personal en casa que le ayude a cocinar la cena o a limpiar, estos son ejemplos de **colaboración**.

Agente conjunto

Ahora podemos definir un **agente conjunto** cuyas acciones son tuplas de acciones humano-robot, (u_H, u_R) y que optimiza para $J_H(x, u_H, u_R) = J_R(x, u_R, u_H)$, y estamos resolviendo un problema de planificación regular. Calculamos el plan o política óptima para el agente conjunto, y ¡eh, aquí!, ahora sabemos qué deben hacer el robot y el humano.

Esto funcionaría muy bien si las personas fueran óptimas. El robot haría su parte del plan conjunto, el humano la suya. Sin embargo, en la práctica las personas no parecen seguir el plan conjunto perfectamente trazado, ¡tienen una mente propia! Ya hemos aprendido una forma de manejar esto, en la Sección 26.6. Lo llamamos **control predictivo del modelo (MPC)**: la idea era planificar un plan, ejecutar la primera acción y luego replanificar. De esa manera, el robot siempre adapta su plan a lo que el humano está haciendo en realidad.

Trabajemos a partir de un ejemplo. Suponga que usted y el robot están en su cocina y han decidido hacer guafles. Usted está un poco más cerca del refrigerador, por lo que el plan conjunto óptimo sería que usted tiene que sacar los huevos y la leche del refrigerador, mientras el robot busca la harina en la alacena. El robot sabe esto porque permite medir con bastante precisión dónde está todo. Pero suponga que usted empieza a dirigirse a la alacena de la harina. Va en contra

del plan conjunto óptimo. En lugar de apegarse a él y también ir con obstinación por la harina, el robot MPC recalcula el plan óptimo, y ahora que está lo suficientemente cerca de la harina, es mejor que el robot tome la guaflera en su lugar.

Si sabemos que las personas pueden desviarse de la optimalidad, podemos tenerlo en cuenta con anticipación. En nuestro ejemplo, el robot puede intentar anticipar que usted va a buscar la harina en el momento en que da su primer paso (digamos, usando la técnica de predicción anterior). Incluso si todavía es óptimo para usted dar la vuelta y dirigirse al refrigerador, el robot no debería suponer que eso es lo que va a pasar. En cambio, el robot permite calcular un plan en el que usted siga haciendo lo que parece querer hacer.

Humanos como agentes de caja negra

No tenemos que tratar a las personas como agentes intencionales impulsados por objetivos para que los robots se coordinen con nosotros. Un modelo alternativo es que el humano es sólo un agente cuya política π_H "interfiere" con la dinámica del entorno. El robot no conoce π_H, pero permite modelar el problema como la necesidad de actuar en un MDP con dinámica desconocida. Ya hemos visto esto antes: para agentes generales en el Capítulo 22 y para robots en particular en la Sección 26.7.

El robot permite ajustar un modelo de política π_H a los datos humanos y usarlo para calcular una política óptima para sí mismo. Debido a la escasez de datos, esto se ha utilizado hasta ahora a nivel de tarea. Por ejemplo, los robots han aprendido a través de la interacción qué acciones tienden a tomar las personas (en respuesta a sus propias acciones) para la tarea de colocar y perforar tornillos en una tarea de montaje industrial.

Luego, también está la alternativa de aprendizaje por refuerzo sin modelo: el robot puede comenzar con alguna política o función de valor inicial, y seguir mejorándola con el tiempo a través de prueba y error.

26.8.2 Aprendiendo a hacer lo que los humanos quieren

Otra forma en que la interacción con los humanos entra en la robótica es en J_R en sí, la función de costo o recompensa del robot. El marco de los agentes racionales y los algoritmos asociados reducen el problema de generar un buen comportamiento a especificar una buena función de recompensa. Pero para los robots, como para muchos otros agentes de IA, conseguir el costo correcto sigue siendo difícil.

Pensemos en los vehículos autónomos: queremos que lleguen al destino, que sean seguros, que conduzcan de manera cómoda para sus pasajeros, que obedezcan las leyes de tránsito, etc. Un diseñador de tal sistema necesita equilibrar estos diferentes componentes de la función de costo. La tarea del diseñador es difícil porque los robots están construidos para ayudar a los usuarios finales, pero no todos los usuarios finales son iguales. Todos tenemos diferentes preferencias sobre cuán agresiva queremos que sea la conducción de nuestro vehículo, etc.

A continuación, exploramos dos alternativas para intentar que el comportamiento del robot coincida con lo que realmente queremos que haga. La primera es aprender una función de costo a partir de la entrada humana. La segunda es eludir la función de costo e imitar las demostraciones humanas de la tarea.

Aprendizaje de preferencias: Aprendiendo funciones de costos

Imagine que un usuario final está mostrándole a un robot cómo hacer una tarea. Por ejemplo, está conduciendo el vehículo de la manera en que le gustaría que fuera conducido por el robot. ¿Es posible pensar en una forma de que el robot use estas acciones, que llamamos "demostraciones", para averiguar qué función de costo debería optimizar?

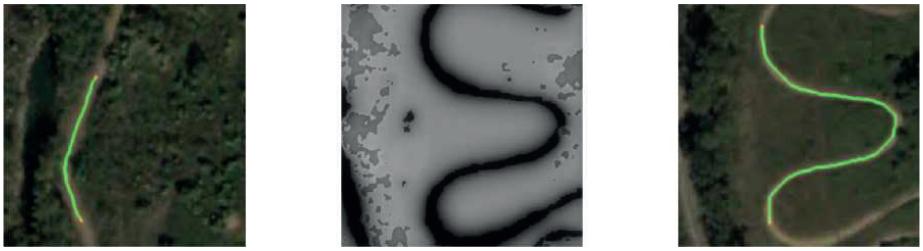


Figura 26.29 Izquierda: Un robot móvil en una demostración en donde se mantiene en el camino de tierra. Centro: El robot infiere la función de costo deseada y la utiliza en una nueva escena, sabiendo que debe poner un costo más bajo en el camino allí. Derecha: El robot planea una trayectoria para la nueva escena que también se mantiene en el camino, reproduciendo las preferencias detrás de la demostración. (Imágenes cortesía de Nathan Ratliff y James A. Bagnell. Ver Ratliff *et al.* (2006)).

Ya hemos visto la respuesta a esto en la Sección 26.8.1. Allí, la configuración era un poco diferente: teníamos a otra persona tomando acciones en el mismo espacio que el robot, y el robot necesitaba predecir lo que la persona haría. Una técnica que revisamos para hacer estas predicciones era asumir que las personas actúan para optimizar de manera ruidosa alguna función de costo J_H, y podemos usar sus acciones en curso como evidencia de qué función de costo es esa. Podemos hacer lo mismo aquí, pero no con el propósito de predecir el comportamiento humano en el futuro, sino de adquirir la función de costo que el robot mismo debería optimizar. Si la persona conduce de manera defensiva, la función de costo que explicará sus acciones pondrá mucho peso en la seguridad y menos en la eficiencia. El robot puede adoptar esta función de costo como propia y optimizarla cuando conduce el vehículo por sí mismo.

Los roboticistas han experimentado con diferentes algoritmos para hacer que esta inferencia de costo sea computacionalmente tratable. En la Figura 26.29, vemos un ejemplo donde se enseña a un robot a preferir permanecer en la carretera a conducir por el terreno a campo traviesa. Con frecuencia, en tales métodos la función de costo ha sido representada como una combinación de características hechas a mano, pero trabajos recientes también han estudiado cómo representarla usando una red neuronal profunda, sin ingeniería de características.

Hay otras formas para que una persona proporcione entrada. Una persona podría usar el lenguaje en lugar de la demostración para instruir al robot. Otra podría actuar como crítico, observando al robot realizar una tarea de una manera (o de dos maneras) y luego decidir qué tan bien se hizo la tarea (o cuál manera fue mejor) o dando consejos sobre cómo mejorar.

Aprendiendo políticas directamente por imitación

Una alternativa es eludir las funciones de costo y aprender la *política* deseada del robot de manera directa. En nuestro ejemplo del vehículo, las demostraciones del humano construyen un conjunto de datos conveniente de estados etiquetados por la acción que el robot debe tomar en cada estado: $\mathcal{D} = \{(x_i, u_i)\}$. El robot permite ejecutar un aprendizaje supervisado para ajustar una política $\pi: x \mapsto u$, y ejecutar esa política. Esto se llama **aprendizaje por imitación** o **clonación de comportamiento**.

Clonación de comportamiento

Un desafío con este enfoque es la **generalización** a nuevos estados. El robot no sabe por qué las acciones en su base de datos han sido marcadas como óptimas. No tiene ninguna regla causal; todo lo que permite hacer es ejecutar un algoritmo de aprendizaje supervisado para intentar aprender una política que se generalice a estados desconocidos. Sin embargo, no hay garantía de que la generalización sea correcta.

Generalización

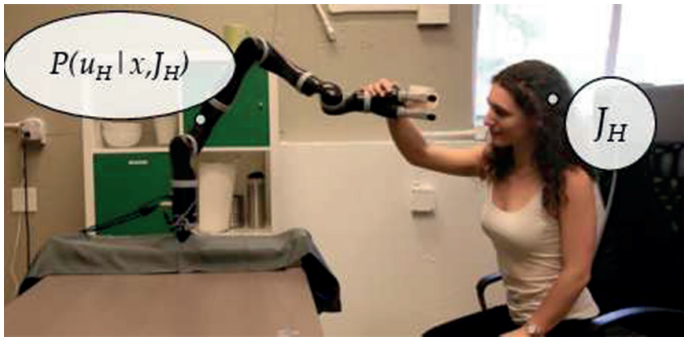


Figura 26.30 Una profesora humana empuja al robot hacia abajo para enseñarle a permanecer más cerca de la mesa. El robot actualiza su comprensión de la función de costo deseada y comienza a optimizarla. (Imagen cortesía de Anca Dragan. Ver Bajcsy *et al.* (2017)).

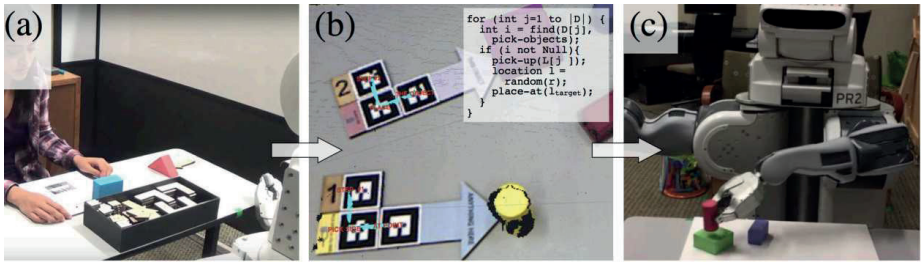


Figura 26.31 Interfaz de programación que implica colocar bloques especialmente diseñados en el espacio de trabajo del robot para seleccionar objetos y especificar acciones de alto nivel. (Imágenes cortesía de Maya Cakmak. Ver Sefidgar *et al.* (2017)).

El proyecto de vehículo autónomo ALVINN utilizó este enfoque y encontró que aun cuando se parte de un estado en $\mathcal{D}$, π cometerá pequeños errores que llevarán al vehículo fuera de la trayectoria demostrada. Allí, π cometerá un error mayor que llevará el vehículo aún más lejos del curso deseado.

Podemos abordar esto en el momento del entrenamiento si intercalamos la recopilación de etiquetas y el aprendizaje: comenzar con una demostración, aprender una política, luego desplegar esa política y preguntarle al humano qué acción tomar en cada estado en el camino, luego repetir. El robot aprende así cómo corregir sus errores a medida que se desvía de las acciones deseadas por el humano.

De manera alternativa, podemos abordarlo aprovechando el aprendizaje por refuerzo. El robot permite ajustar un modelo de dinámica basado en las demostraciones, y luego usar el control óptimo (Sección 26.5.4) para generar una política que optimice para mantenerse cerca de la demostración. Una versión de esto se ha utilizado para realizar maniobras muy desafiantes a nivel experto en un pequeño helicóptero controlado por radio (ver Figura 22.9(b)).

El sistema DAGGER (Data Aggregation) comienza con una demostración de un experto humano. A partir de eso aprende una política π_1 y la usa para generar un conjunto de datos $\mathcal{D}$. Luego, a partir de $\mathcal{D}$ genera una nueva política π_2 que imita mejor los datos humanos originales. Esto se repite y en

la *enésima* iteración utiliza π_n para generar más datos, para ser añadidos a $\mathcal{D}$, que luego se utiliza para crear π_{n+1}. En otras palabras, en cada iteración el sistema recopila nuevos datos bajo la política actual y entrena la próxima política utilizando todos los datos recopilados hasta ahora.

Técnicas recientes relacionadas utilizan **entrenamiento adversarial**: alternan entre entrenar un clasificador para distinguir entre la política aprendida del robot y las demostraciones del humano, y entrenar una nueva política de robot a través del aprendizaje por refuerzo para engañar al clasificador. Estos avances permiten al robot manejar estados que están cerca de las demostraciones, pero la generalización a estados lejanos o a nuevas dinámicas es un trabajo en progreso.

Interfaces de enseñanza y el problema de correspondencia. Hasta ahora hemos imaginado el caso de un vehículo autónomo o de un helicóptero autónomo para los que las demostraciones humanas utilizan las mismas acciones que el robot puede tomar por sí mismo: acelerar, frenar y conducir. Pero ¿qué pasa si hacemos esto para tareas como limpiar la mesa de la cocina? Tenemos dos opciones aquí: o bien la persona demuestra usando su propio cuerpo mientras el robot mira o la persona guía físicamente los actuadores del robot.

Problema de correspondencia

El primer enfoque es atractivo porque llega de manera natural a los usuarios finales. Sin embargo, padece el **problema de correspondencia**: cómo mapear las acciones humanas en las acciones del robot. Las personas tienen diferentes cinemáticas y dinámicas que los robots. Eso no sólo hace que sea difícil *traducir* o *reorientar* el movimiento humano al movimiento del robot (por ejemplo, reorientar un agarre humano de cinco dedos a un agarre de robot de dos dedos), sino que con frecuencia la estrategia de alto nivel que una persona podría usar no es apropiada para el robot.

Enseñanza kinestésica

El segundo enfoque, donde el maestro humano mueve los actuadores del robot a las posiciones correctas, se llama **enseñanza kinestésica**. No es fácil para los humanos enseñar de esta manera, en especial para enseñar a robots con múltiples articulaciones. El maestro necesita coordinar todos los grados de libertad mientras guía el brazo a través de la tarea. Los investigadores han revisado alternativas, como demostrar **fotogramas clave** en lugar de trayectorias continuas, así como usar la **programación visual** para permitir a los usuarios finales programar primitivas para una tarea en lugar de demostrar desde cero (Figura 26.31). A veces se combinan ambos enfoques.

Fotograma clave

Programación visual

26.9 Marcos robóticos alternativos

Hasta ahora hemos adoptado una visión de la robótica basada en la noción de definir o aprender una función de recompensa, y hacer que el robot optimice esa función de recompensa (ya sea a través de la planificación o el aprendizaje), a veces en coordinación o colaboración con los humanos. Esta es una visión **deliberativa** de la robótica, a contrastar con una visión **reactiva**.

Deliberativo

Reactivo

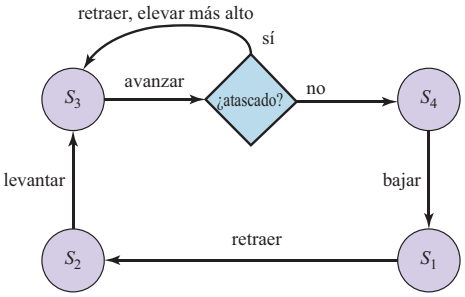
26.9.1 Controladores reactivos

En algunos casos, es más fácil establecer una buena política para un robot que modelar el mundo y planificar. En este sentido, en lugar de un agente *racional*, tenemos un agente *reflejo*.

Por ejemplo, imagine un robot con patas que intenta levantar una pata por encima de un obstáculo. Podríamos indicar a este robot una regla que dijera: levantar la pata a una pequeña altura h y moverla hacia adelante, y si la pata encuentra un obstáculo, moverla hacia atrás y empezar de nuevo a una altura mayor. Podría decir que h está modelando un aspecto del mundo, pero también podemos pensar en h como una variable auxiliar del controlador del robot, carente de significado físico directo.

Un ejemplo de esto es el robot de seis patas (hexápodo), mostrado en la Figura 26.32(a), diseñado para caminar por terrenos difíciles. Los sensores del robot son inadecuados para obtener

(a)



(b)

Figura 26.32 (a) Genghis, un robot hexápodo. (Imagen cortesía de Rodney A. Brooks.) (b) Una máquina de estado finito aumentada (AFSM) que controla una pata. El AFSM reacciona a la retroalimentación del sensor: si una pata se atasca durante la fase de balanceo hacia adelante, se levantará incrementando su altura.

modelos precisos del terreno para planificar trayectorias. Además, incluso si añadiéramos cámaras y telémetros de alta precisión, los 12 grados de libertad (dos por cada pata) harían que el problema de planificación de trayectorias resultante fuera computacionalmente difícil.

No obstante, es posible especificar un controlador de manera directa *sin* un modelo ambiental explícito. (Ya hemos visto esto con el controlador PD, que pudo mantener un brazo de robot complejo en el objetivo *sin* un modelo explícito de la dinámica del robot).

Marcha

Para el robot hexápodo, primero elegimos una **marcha** o patrón de movimiento de las extremidades. Una estáticamente estable es mover primero las patas delanteras derechas, traseras derechas y centrales izquierdas hacia adelante (manteniendo las otras tres fijas), y luego mover las otras tres. Esta marcha funciona bien en terreno plano. En terreno accidentado, los obstáculos pueden impedir que una pierna se balancee hacia adelante. Este problema puede superarse con una regla de control muy simple: *cuando el movimiento hacia adelante de una pierna está bloqueado, retírela, levántela más alto y vuelva a intentarlo.* El controlador resultante se muestra en la Figura 26.32(b) como una simple máquina de estado finito; constituye un agente reflejo con estado, donde el estado interno está representado por el índice del estado de la máquina actual (s_1 a s_4).

26.9.2 Arquitecturas de subsunción

Arquitectura de subsunción

La **arquitectura de subsunción** (Brooks, 1986) es un marco para ensamblar controladores de máquinas de estado finito. Los nodos en estas máquinas contienen pruebas para ciertas variables de sensor, en cuyo caso la traza de ejecución de una máquina de estado finito está condicionada por el resultado de dicha prueba. Los arcos pueden estar etiquetados con mensajes que se generarán al atravesarlos, y que se envían a los motores del robot o a otras máquinas de estado finito. Además, las máquinas de estado finito poseen temporizadores internos (relojes) que controlan el tiempo que tarda en atravesar un arco. Las máquinas resultantes se llaman **máquinas de estado finito aumentadas** (**AFSM**, por sus siglas en inglés)**,** donde la ampliación se refiere al uso de relojes.

Máquina de estado finito aumentada (AFSM)

Un ejemplo de una AFSM simple es la máquina de cuatro estados de la que acabamos de hablar, mostrada en la Figura 26.32(b). Esta AFSM implementa un controlador cíclico, cuya ejecución en su mayoría no depende de la retroalimentación ambiental. Sin embargo, la fase de balanceo hacia adelante sí depende de la retroalimentación del sensor. Si la pierna está atascada,

significa que no ha logrado ejecutar el balanceo hacia adelante, el robot retrae la pierna, la levanta un poco más alto y trata de ejecutar el balanceo hacia adelante una vez más. Así, el controlador es capaz de *reaccionar* a contingencias que surgen de la interacción del robot y su entorno.

La arquitectura de subsunción ofrece primitivas adicionales para sincronizar AFSM y para combinar los valores de salida de múltiples AFSM, posiblemente en conflicto. De esta manera, permite al programador componer controladores cada vez más complejos de manera ascendente. En nuestro ejemplo, podríamos comenzar con AFSM para las piernas individuales, seguido de una AFSM para coordinar múltiples piernas. Además, podríamos implementar comportamientos de nivel superior como evitar colisiones que podrían implicar retroceder y girar.

La idea de componer controladores de robots a partir de AFSMs es muy intrigante. Imagine lo difícil que sería generar el mismo comportamiento con cualquiera de los algoritmos de planificación de trayectorias en el espacio de configuración descritos en la sección anterior. Primero necesitaríamos un modelo preciso del terreno. El espacio de configuración de un robot con seis patas, cada una impulsada por dos motores independientes, lo que suma 18 dimensiones (12 dimensiones para la configuración de las patas, y seis para la ubicación y orientación del robot con relación a su entorno). Incluso si nuestros ordenadores fueran lo suficientemente rápidos para encontrar trayectorias en espacios de tan alta dimensión, tendríamos que preocuparnos por efectos desagradables como el deslizamiento del robot por una pendiente.

Debido a tales efectos estocásticos, un solo camino a través del espacio de configuración sería casi con certeza demasiado frágil; incluso, un controlador PID podría no ser capaz de hacer frente a tales contingencias. En otras palabras, generar comportamiento de movimiento deliberadamente es un problema demasiado complejo en algunos casos para los algoritmos de planificación de movimiento de robots actuales.

Sin embargo, la arquitectura de subsunción tiene sus propios problemas. Primero, las AFSM son impulsadas por la entrada de sensores en bruto, un arreglo que funciona si los datos del sensor son fiables y contienen toda la información necesaria para la toma de decisiones, pero falla si los datos del sensor tienen que ser integrados de formas no triviales a lo largo del tiempo. Por lo tanto, los controladores de estilo de subsunción se han aplicado a tareas simples, como seguir una pared o moverse hacia fuentes de luz visibles. En segundo lugar, la falta de deliberación hace difícil cambiar los objetivos del robot. Un robot con una arquitectura de subsunción por lo regular realiza sólo una tarea, y no tiene noción de cómo modificar sus controles para adaptarse a diferentes objetivos (al igual que el escarabajo de estiércol en la página 41).

En tercer lugar, en múltiples problemas del mundo real, la política que queremos es demasiado compleja para codificarla de manera explícita. Piense en el ejemplo de la Figura 26.28, de un vehículo autónomo que necesita negociar un cambio de carril con un conductor humano. Podríamos empezar con una política simple que entra en el carril objetivo. Pero cuando probamos el vehículo, descubrimos que no todos los conductores en el carril objetivo van a reducir la velocidad para dejar entrar al vehículo. Podríamos entonces añadir un poco más de complejidad: hacer que el vehículo se acerque al carril objetivo, esperar una respuesta del conductor en ese carril y luego proceder o retroceder. Pero cuando probamos el vehículo y nos damos cuenta de que el acercamiento necesita suceder a una velocidad diferente dependiendo de la velocidad del vehículo en el carril objetivo, de si hay otro vehículo adelante en el carril objetivo, de si hay un vehículo detrás del vehículo inicial, etc. El número de condiciones que necesitamos considerar para determinar el curso de acción correcto puede ser muy grande, incluso para una maniobra aparentemente simple. Esto, a su vez, presenta desafíos de escalabilidad para las arquitecturas de estilo subsunción.

Dicho todo esto, la robótica es un problema complejo con muchos enfoques: deliberativo, reactivo o una mezcla de ambos, que se basa en física, modelos cognitivos, datos o una mezcla de estos. El enfoque correcto sigue siendo objeto de debate, investigación científica y habilidad de ingeniería.

(a) (b)

Figura 26.33 (a) Un paciente con una interfaz cerebro-máquina controla un brazo de robot para agarrar una bebida. (Imagen cortesía de la Universidad de Brown). (b) Roomba, el robot aspirador. (Foto de HANDOUT/KRT/Newscom).

26.10 Dominios de aplicación

La tecnología robótica ya está permeando nuestro mundo, pues tiene el potencial de mejorar nuestra independencia, salud y productividad. Aquí referimos algunos ejemplos de sus aplicaciones.

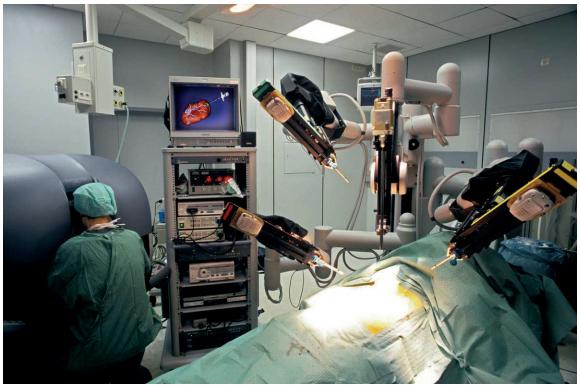
Cuidado en el hogar: Los robots han comenzado a entrar en el hogar para cuidar adultos mayores y personas con discapacidades motoras, al asistirlas en actividades de la vida cotidiana y posibilitarles una vida más independiente. Entre estos robots se incluyen sillas de ruedas y brazos montados en sillas de ruedas como el brazo Kinova de la Figura 26.1(b). Aunque comienzan siendo operados de manera directa por un humano, estos robots están ganando cada vez más autonomía. En el horizonte están los robots operados por **interfaces cerebro-máquina**, que han demostrado permitir a las personas con cuadriplejia usar un brazo de robot para agarrar objetos e incluso alimentarse a sí mismos (Figura 26.33(a)). Relacionado con estos están las extremidades protésicas, que responden de manera inteligente a nuestras acciones, y los exoesqueletos, que nos dan una fuerza sobrehumana o hacen que las personas, que no pueden controlar sus músculos desde la cintura para abajo, vuelvan a caminar.

Los robots personales están destinados a ayudarnos con tareas cotidianas como la limpieza y la organización, y a liberarnos tiempo. Aunque la manipulación todavía tiene un camino por recorrer antes de que pueda operar sin problemas en entornos humanos desordenados y no estructurados, la navegación ha avanzado un poco. En particular, muchas casas ya disfrutan de un robot aspirador móvil como el de la Figura 26.33(b).

Cuidado de la salud: Los robots asisten y auxilian a los cirujanos pues facilitan procedimientos más precisos, mínimamente invasivos, más seguros y con mejores resultados para los pacientes. El robot quirúrgico Da Vinci de la Figura 26.34(a) está ahora operando en hospitales en Estados Unidos.

Servicios: Los robots móviles ayudan en edificios de oficinas, hoteles y hospitales. Savioke ha puesto robots en hoteles para entregar diversos productos, como toallas o pasta de dientes a la habitación. Los robots Helpmate y TUG llevan comida y medicinas en los hospitales (Figura 26.34(b)), mientras que el robot Moxi de Diligent Robotics ayuda a las enfermeras con responsabilidades logísticas y administrativas. Co-Bot recorre los pasillos de la Universidad Carnegie Mellon para guiarlo a la oficina de alguien. Nosotros también podemos usar **robots de telepresencia** como el Beam para asistir a reuniones y conferencias de forma remota o para echarle un ojo a nuestros abuelos.

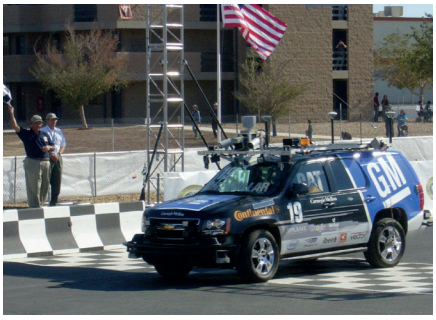
Robots de telepresencia



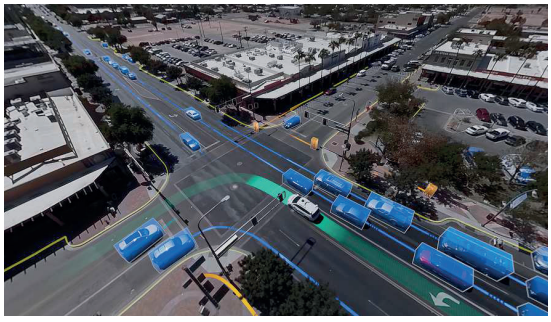
(a)

(b)

Figura 26.34 (a) Robot quirúrgico en la sala de operaciones. (Foto de Patrick Landmann/Science Source). (b) Robot de entrega en el hospital. (Foto tomada de Wired).



(a)

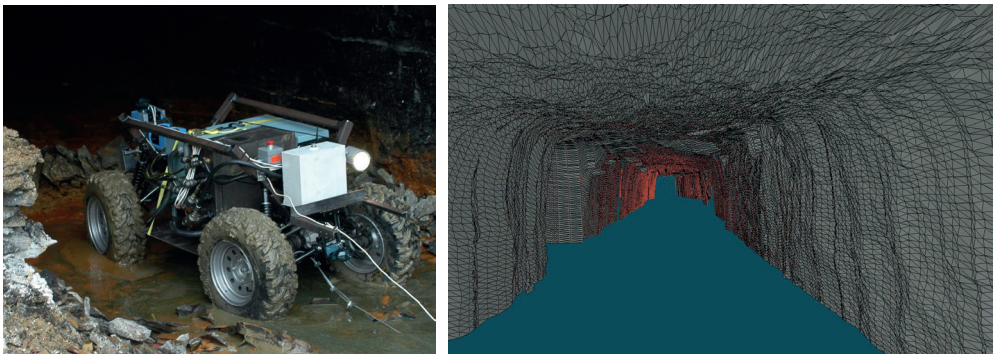


(b)

Figura 26.35 (a) Vehículo autónomo BOSS ganador del Desafío Urbano de DARPA. (Foto de Tangi Quemener/AFP/Getty Images/Newscom. Cortesía de Sebastian Thrun). (b) Vista aérea que muestra la percepción y las predicciones del vehículo autónomo Waymo (vehículo blanco con rastro verde). Otros vehículos (cajas azules) y peatones (cajas naranjas) se muestran con trayectorias anticipadas. Los límites de la carretera/acera están en amarillo. (Foto cortesía de Waymo).

Vehículos autónomos: Mientras conducimos, algunos de nosotros nos distraemos de manera ocasional por llamadas de celular, mensajes de texto u otros descuidos. El triste resultado: más de un millón de personas mueren cada año en accidentes de tránsito. Además, muchos de nosotros gastamos considerables cantidades de tiempo conduciendo y nos gustaría recuperar algo de él. Todo esto ha llevado a un esfuerzo masivo en curso para desplegar vehículos autónomos.

Los prototipos han existido desde la década de 1980, pero el progreso fue estimulado por el Gran Desafío DARPA de 2005, una carrera de vehículos autónomos de más de 200 kilómetros desafiantes de terreno desértico sin ensayar. El vehículo Stanley de Stanford completó el recorrido en menos de siete horas, ganó un premio de $2 millones y un lugar en el Museo Nacional de Historia Americana. La Figura 26.35(a) muestra al robot BOSS ganador en 2007 del Desafío



(a) (b)

Figura 26.36 (a) Robot mapeando una mina de carbón abandonada. (b) Mapa 3D de la mina elaborado por el robot. (Cortesía de Sebastian Thrun).

Urbano de DARPA, una carrera complicada en las calles de la ciudad donde los robots se enfrentaban a otros robots y tenían que obedecer las reglas de tránsito.

En 2009, Google inició un proyecto de conducción autónoma (con muchos de los investigadores que habían trabajado en Stanley y BOSS), que ahora se ha convertido en Waymo. En 2018 Waymo comenzó las pruebas sin conductor (sin nadie en el asiento del conductor) en los suburbios de Phoenix, Arizona. Entretanto, otras empresas de conducción autónoma y empresas de viajes compartidos están trabajando en el desarrollo de su propia tecnología, mientras que los fabricantes de automóviles han estado vendiendo vehículos con más y más inteligencia asistida, como el **asistente de conductor** de Tesla, destinado para la conducción en autopista. Otras empresas apuntan a aplicaciones de conducción no en autopista en las que se incluyen campus universitarios y comunidades de jubilados. Y empresas que se centran en aplicaciones no de pasajeros como el transporte de camiones, la entrega de comestibles y el estacionamiento de valet.

Asistente de conductor

Entretenimiento: Disney ha estado usando robots (bajo el nombre de **animatronics**) en sus parques desde 1963. Originalmente, estos robots estaban restringidos a diseños hechos a mano, en bucle abierto, movimiento invariable y habla, pero desde 2009 una versión llamada **autonomatronics** permite generar acciones autónomas. Los robots también toman la forma de juguetes inteligentes para niños; por ejemplo, el Cozmo de Anki participa en juegos con niños y puede golpear la mesa con frustración cuando pierde. Finalmente, los quadrotors como el R1 de Skydio de la Figura 26.2(b) actúan como fotógrafos y videógrafos personales, siguiéndonos para tomar fotos de acción mientras esquiamos o montamos en bicicleta.

Animatronics

Autonomatronics

Exploración y entornos peligrosos: Los robots han ido donde ningún humano ha ido antes, incluyendo la superficie de Marte. Los brazos robóticos ayudan a los astronautas a desplegar y recuperar satélites y ayudaron a construir la Estación Espacial Internacional. Los robots también ayudan a explorar las profundidades marinas. Se utilizan de manera rutinaria para construir mapas de barcos hundidos. La Figura 26.36 muestra un robot mapeando una mina de carbón abandonada, junto con un modelo 3D de la mina construido con sensores de rango. En 1996, un equipo de investigadores liberó un robot con patas en el cráter de un volcán activo para adquirir datos para la investigación del clima. Los robots se están convirtiendo en herramientas muy efectivas para recopilar información en dominios que son difíciles (o peligrosos) para que las personas accedan.

Los robots han ayudado a las personas a limpiar residuos nucleares, más específico en Three Mile Island, Chernobyl y Fukushima. Los robots estuvieron presentes después del colapso del

World Trade Center en Nueva York, donde entraron en estructuras consideradas muy peligrosas para los equipos de búsqueda y rescate humanos. Aquí también, estos robots se despliegan inicialmente a través de teleoperación, pero a medida que la tecnología avanza se están volviendo más y más autónomos, con un operador humano a cargo pero sin tener que especificar cada comando individual.

Industria: Hoy en día, la mayoría de los robots se utilizan en fábricas para automatizar tareas difíciles y peligrosas, o aburridas para los humanos. (La mayoría de los robots para manufactura se usan en fábricas de automóviles). Automatizar estas tareas es positivo en términos de producir de manera eficiente lo que la sociedad necesita. Al mismo tiempo, también significa desplazar a algunos trabajadores humanos de sus empleos. Esto tiene importantes implicaciones políticas y económicas: la necesidad de reciclaje y educación, la necesidad de una distribución justa de recursos, etc. Estos temas se discuten más adelante en la Sección 27.3.5.

Resumen

La robótica trata sobre agentes físicamente encarnados que permiten cambiar el estado del mundo físico. En este capítulo, hemos aprendido lo siguiente:

- Los tipos más comunes de robots son los **manipuladores** (brazos de robot) y los **robots móviles**. Tienen **sensores** para percibir el mundo y **actuadores** que producen movimiento, que luego afecta al mundo a través de los **efectores**.
- El problema general de la robótica implica **estocasticidad** (que permite ser manejada por MDP), **observabilidad parcial** (que puede ser manejada por POMDP) y actuar con y alrededor de **otros agentes** (que permite ser manejado con la teoría de juegos). El problema se hace aún más difícil por el hecho de que la mayoría de los robots trabajan en espacios de estado y acción continuos y de alta dimensión. También operan en el mundo real, que se niega a correr más rápido que en tiempo real y en el que los fallos llevan a que se dañen cosas reales, sin capacidad de "deshacer".
- De manera ideal, el robot resolvería todo el problema de una vez: las observaciones en forma de feeds de sensores crudos entran y las acciones en forma de torques o corrientes a los motores salen. En la práctica, sin embargo, esto es demasiado desalentador y los roboticistas con frecuencia desacoplan diferentes aspectos del problema y los tratan de forma independiente.
- Por lo regular, separamos la percepción (estimación) de la acción (generación de movimiento). La percepción en robótica implica visión por computadora para reconocer los alrededores mediante cámaras, pero también localización y mapeo.
- La percepción robótica se preocupa por estimar cantidades relevantes para la decisión a partir de datos de sensores. Para hacerlo, necesitamos una representación interna y un método para actualizar esta representación interna con el tiempo.
- Los **algoritmos de filtrado probabilístico**, como los filtros de partículas y los filtros de Kalman, son útiles para la percepción del robot. Estas técnicas mantienen el estado de creencia, una distribución posterior sobre las variables de estado.
- Para generar movimiento usamos **espacios de configuración**, donde un punto especifica todo lo que necesitamos saber para localizar cada **punto del cuerpo** en el robot. Por ejemplo, para un brazo de robot con dos articulaciones, una configuración consiste en los dos ángulos de las articulaciones.
- En general, desacoplamos el problema de generación de movimiento en **planificación de movimiento**, ocupado en producir un plan, y **control de seguimiento de trayectoria**, ocupado

en producir una política para las entradas de control (comandos de actuador) que resulta en la ejecución del plan.

- La planificación del movimiento se resuelve mediante la búsqueda en el grafo utilizando **descomposición de celdas,** algoritmos de **planificación de movimiento aleatorizados** —que muestrean hitos en el espacio de configuración continuo— o la **optimización de trayectorias** —que puede empujar de manera iterativa un camino en línea recta fuera de la colisión aprovechando un **campo de distancia firmado**.
- Un camino encontrado por un algoritmo de búsqueda permite ejecutarse usando el camino como la trayectoria de referencia para un **controlador PID**, que corrige constantemente los errores entre el punto donde está el robot y donde se supone que debe estar, o mediante el **control de par calculado**, que agrega un término de **prealimentación** que hace uso de la **dinámica inversa** para calcular de manera aproximada qué par enviar para avanzar a lo largo de la trayectoria.
- El **control óptimo** une la planificación del movimiento y el seguimiento de la trayectoria calculando una trayectoria óptima de manera directa sobre las entradas de control. Esto es en especial fácil cuando tenemos costos cuadráticos y dinámicas lineales, lo que resulta en un regulador cuadrático lineal **(LQR)**. Los métodos populares hacen uso de esto al linealizar las dinámicas y calcular aproximaciones de segundo orden del costo **(ILQR)**.
- La planificación bajo incertidumbre une la percepción y la acción mediante la **replanificación en línea** (como el control predictivo de modelos) y las acciones de **recopilación de información** que ayudan a la percepción.
- El aprendizaje por refuerzo se aplica en la robótica con técnicas que se esfuerzan por reducir el número requerido de interacciones con el mundo real. Dichas técnicas suelen explotar modelos, ya sea estimándolos y usándolos para planificar, o entrenando políticas robustas con respecto a diferentes parámetros posibles del modelo.
- La interacción con los humanos requiere la capacidad de **coordinar** las acciones del robot con las suyas, lo que permite formularse como un juego. Por lo general, descomponemos la solución en **predicción**, en la que usamos las acciones en curso de la persona para estimar lo que hará en el futuro, y **acción**, en la que usamos las predicciones para calcular el movimiento óptimo para el robot.
- Ayudar a los humanos también requiere la capacidad de aprender o inferir lo que quieren. Los robots abordan esto aprendiendo la función de costo deseada que deben optimizar a partir de la entrada humana, como demostraciones, correcciones o instrucciones en lenguaje natural. De manera alternativa, los robots permiten imitar el comportamiento humano y utilizar el aprendizaje por refuerzo para ayudar a abordar el desafío de la generalización a nuevos estados.

Notas bibliográficas e históricas

La palabra **robot** fue popularizada por el dramaturgo checo Karel Čapek en 1920 en su obra *R.U.R.* (*Robots Universales de Rossum*). Los robots, que fueron cultivados de manera química en lugar de construidos mecánicamente, terminan resentidos con sus amos y deciden tomar el control. Parece que fue el hermano de Čapek, Josef, quien primero combinó las palabras checas "robota" (trabajo obligatorio) y "robotnik" (siervo) para sintetizar "robot" en su cuento corto de 1917 *Opilec* (Glanc, 1978). El término *robótica* fue inventado para una historia de ciencia ficción (Asimov, 1950).

La idea de una máquina autónoma precede a la palabra "robot" por miles de años. En la mitología griega del siglo VII a.C., un robot llamado Talos fue construido por Hefesto, el dios griego de la metalurgia, para proteger la isla de Creta. La leyenda cuenta que la hechicera Medea derrotó

a Talos prometiéndole inmortalidad, pero luego drenó su fluido vital. Así, este es el primer ejemplo de un robot cometiendo un error en el proceso de cambiar su función objetivo. En el 322 a.C., Aristóteles anticipó el desempleo tecnológico, especulando "Si cada herramienta, cuando se le ordena, o incluso por su propia voluntad, pudiera hacer el trabajo que le corresponde... entonces no habría necesidad ni de aprendices para los maestros trabajadores ni de esclavos para los señores".

En el siglo III a.C. un robot humanoide real llamado el Sirviente de Filón podía verter vino o agua en una copa; una serie de válvulas cortaban el flujo en el momento adecuado. En el siglo XVIII se construyeron autómatas maravillosos, siendo el pato mecánico de Jacques Vaucanson de 1738 un ejemplo temprano, pero los comportamientos complejos que exhibían estaban fijados de antemano. Quizás el primer ejemplo de un dispositivo tipo robot programable fue el telar de Jacquard (1805), descrito en la página 15.

La "tortuga" de Grey Walter, construida en 1948, podría considerarse el primer robot móvil autónomo, aunque su sistema de control no era programable. La "Bestia de Hopkins", construida en 1960 en la Universidad Johns Hopkins, era mucho más sofisticada pues tenía sensores de sonar y fotocélulas, hardware de reconocimiento de patrones y podía reconocer la placa de cubierta de un enchufe de corriente alterna estándar. ¡Era capaz de buscar enchufes, conectarse y luego recargar sus baterías! Aun así, la Bestia tenía un repertorio limitado de habilidades.

El primer robot móvil de propósito general fue "Shakey", desarrollado en lo que entonces era el Stanford Research Institute (ahora SRI) a finales de la década de 1960 (Fikes y Nilsson, 1971; Nilsson, 1984). Shakey fue el primer robot en integrar percepción, planificación y ejecución, y gran parte de la investigación posterior en IA fue influida por este notable logro. Shakey aparece en la portada de este libro con el líder del proyecto Charlie Rosen (1917–2002). Otros influyentes proyectos incluyen el Stanford Cart y el CMU Rover (Moravec, 1983). Cox y Wilfong (1990) describen el trabajo clásico en vehículos autónomos.

El primer robot comercial fue un brazo llamado UNIMATE, para *automatización universal*, desarrollado por Joseph Engelberger y George Devol en su compañía, Unimation. En 1961, el primer robot UNIMATE fue vendido a General Motors para su uso en la fabricación de tubos de imagen de televisión. También en 1961 fue cuando Devol obtuvo la primera patente de robot en Estados Unidos.

En 1973, Toyota y Nissan comenzaron a usar una versión actualizada de UNIMATE para la soldadura de puntos de carrocerías de automóviles. Esto inició una gran revolución en la fabricación de automóviles que tuvo lugar principalmente en Japón y Estados Unidos, y que aún continúa. Unimation siguió en 1978 con el desarrollo del robot Puma (Programmable Universal Machine for Assembly), que fue el estándar *de facto* para la manipulación robótica durante las dos décadas siguientes. Se venden alrededor de 500 000 robots cada año, con la mitad de ellos destinados a la industria automotriz.

En manipulación, el primer gran esfuerzo por crear una máquina de mano-ojo fue MH-1 de Heinrich Ernst, descrito en su tesis de doctorado del MIT (Ernst, 1961). El proyecto Machine Intelligence en Edimburgo también demostró un impresionante sistema temprano para el montaje basado en visión llamado FREDDY (Michie, 1972).

La investigación sobre robótica móvil ha sido estimulada por varias competencias importantes. La competencia anual de robots móviles de AAAI comenzó en 1992. El primer ganador de la competencia fue CARMEL (Congdon *et al.*, 1992). El progreso ha sido constante e impresionante: en competencias recientes los robots ingresaron al complejo de la conferencia, encontraron su camino hasta el mostrador de registro, se registraron para la conferencia e incluso dieron una charla corta.

La competencia **RoboCup**, lanzada en 1995 por Kitano y sus colegas (1997), tiene como objetivo "desarrollar un equipo de robots humanoides totalmente autónomos que puedan ganar contra

el equipo campeón mundial humano en fútbol" para 2050. Algunas competencias usan robots con ruedas, algunos robots humanoides y algunas simulaciones de software. Stone (2016) describe las innovaciones recientes en RoboCup.

El **DARPA Grand Challenge**, organizado por DARPA en 2004 y 2005, requería que los vehículos autónomos recorrieran más de 200 kilómetros a través del desierto en menos de diez horas (Buehler *et al*., 2006). En el evento original en 2004, ningún robot recorrió más de ocho millas, lo que llevó a muchos a creer que el premio nunca sería reclamado. En 2005, el robot Stanley de Stanford ganó la competencia en poco menos de siete horas (Thrun, 2006). Luego DARPA organizó el **Urban Challenge**, una competencia en la que los robots tenían que navegar 60 millas en un entorno urbano con tráfico. El robot Boss de la Universidad Carnegie Mellon obtuvo el primer lugar y reclamó el premio de $2 millones (Urmson y Whittaker, 2008). Los pioneros en el desarrollo de autos robóticos incluyeron a Dickmanns y Zapp (1987) y Pomerleau (1993).

El campo de la cartografía robótica ha evolucionado a partir de dos orígenes distintos. El primer hilo comenzó con el trabajo de Smith y Cheeseman (1986), quienes aplicaron filtros de Kalman al problema de localización y mapeo simultáneos (SLAM). Este algoritmo fue implementado por primera vez por Moutarlier y Chatila (1989) y luego extendido por Leonard y Durrant-Whyte (1992); consulte Dissanayake *et al.* (2001) para obtener una descripción general de las primeras variaciones del filtro de Kalman. El segundo hilo comenzó con el desarrollo de la representación de la **retícula de ocupación** para el mapeo probabilístico, que especifica la probabilidad de que cada ubicación (*x,y*) esté ocupada por un obstáculo (Moravec y Elfes, 1985).

Retícula de ocupación

Kuipers y Levitt (1988) fueron de los primeros en proponer un mapeo topológico en lugar de métrico, motivado por modelos de cognición espacial humana. Un artículo seminal de Lu y Milios (1997) reconoció la escasez del problema de localización y mapeo simultáneos, lo que dio lugar al desarrollo de técnicas de optimización no lineal por Konolige (2004) y Montemerlo y Thrun (2004), así como métodos jerárquicos por Bosse *et al.* (2004). Shatkay y Kaelbling (1997) y Thrun *et al.* (1998) introdujeron el algoritmo EM en el campo del mapeo robótico para la asociación de datos. Es posible encontrar una descripción general de los métodos de mapeo probabilístico en (Thrun *et al.,* 2005).

Las primeras técnicas de localización de robots móviles son estudiadas por Borenstein *et al.* (1996). Aunque el filtrado de Kalman era bien conocido como un método de localización en la teoría del control durante décadas, la formulación probabilística general del problema de localización no apareció en la literatura de IA hasta mucho más tarde, a través del trabajo de Tom Dean y colegas (Dean *et al.,* 1990) y de Simmons y Koenig (1995). Este último trabajo introdujo el término **localización de Markov**. La primera aplicación real de esta técnica fue de Burgard *et al.* (1999), a través de una serie de robots que se desplegaron en museos. La localización de Monte Carlo basada en filtros de partículas fue desarrollada por Fox *et al.* (1999) y ahora se utiliza de manera amplia. El **Rao-Blackwellized, filtro de partículas** combina el filtrado de partículas para la localización de robots con filtrado exacto para la construcción de mapas (Murphy y Russell, 2001; Montemerlo *et al.*, 2002).

Localización de Markov

Rao-Blackwellized, filtro de partículas

Una gran cantidad de trabajos tempranos sobre **planificación de movimientos** se centró en algoritmos geométricos para problemas de planificación de movimientos determinísticos completamente observables. La dificultad PSPACE de la planificación del movimiento del robot se mostró en un artículo seminal de Reif (1979). La representación del espacio de configuración se debe a Lozano-Perez (1983). Una serie de artículos de Schwartz y Sharir sobre lo que llamaron problemas de los **movimientos de piano** (Schwartz *et al.,* 1987) fue altamente influyente.

Movimientos de piano

La descomposición de celdas recursivas para la planificación del espacio de configuración se originó en el trabajo de Brooks y Lozano-Perez (1985) y mejoró de manera significativa por Zhu y Latombe (1991). Los primeros algoritmos de esqueletización se basaron en diagramas de Voronoi

Grafo de visibilidad

(Rowat, 1979) y **grafos de visibilidad** (Wesley y Lozano-Perez, 1979). Guibas *et al.* (1992) desarrollaron técnicas eficientes para calcular diagramas de Voronoi de forma incremental, mientras que Choset (1996) generalizó los diagramas de Voronoi a problemas de planificación de movimiento más amplios.

John Canny (1988) estableció el primer algoritmo exponencial único para la planificación del movimiento. El texto seminal de Latombe (1991) cubre una variedad de enfoques para la planificación del movimiento, al igual que los textos de Choset *et al*. (2005) y LaValle (2006). Kavraki *et al*. (1996) desarrollaron la teoría de los mapas de carreteras probabilísticos. Kuffner y LaValle (2000) desarrollaron árboles de exploración aleatoria rápida (RRTs).

La inclusión de la optimización en la planificación del movimiento geométrico comenzó con las bandas elásticas (Quinlan y Khatib, 1993), que refinan las trayectorias cuando los obstáculos del espacio de configuración cambian. Ratliff *et al.* (2009) formularon la idea como la solución a un problema de control óptimo, permitiendo que la trayectoria inicial comience en colisión y deformándola al mapear los gradientes de obstáculos del espacio de trabajo a través del Jacobiano en el espacio de configuración. Schulman *et al.* (2013) propusieron una alternativa práctica de segundo orden.

El control de los robots como sistemas dinámicos, ya sea para la manipulación o la navegación, ha generado una vasta literatura. Si bien este capítulo explicó los conceptos básicos del **control de seguimiento de trayectoria** y el control óptimo, dejó fuera subcampos enteros, incluyendo el control adaptativo, el control robusto y el análisis de Lyapunov. En lugar de suponer que todo sobre el sistema se conoce a priori, el control adaptativo tiene como objetivo adaptar los parámetros de la dinámica y/o la ley de control en línea. El control robusto, por otro lado, tiene como objetivo diseñar controladores que funcionen bien a pesar de la incertidumbre y las perturbaciones externas.

El análisis de Lyapunov se desarrolló originalmente en la década de 1890 para el análisis de estabilidad de sistemas no lineales generales, pero no fue hasta principios de la década de 1930 que los teóricos del control se dieron cuenta de su verdadero potencial. Con el desarrollo de los métodos de optimización, el análisis de Lyapunov se extendió a las funciones de barrera de control, que se prestan muy bien a las herramientas modernas de optimización. Estos métodos se utilizan ampliamente en la robótica moderna para el diseño de controladores en tiempo real y el análisis de seguridad.

Los trabajos cruciales en el control robótico incluyen una trilogía sobre el control de impedancia de Hogan (1985) y un estudio general de la dinámica de los robots de Featherstone (1987). Dean y Wellman (1991) estuvieron entre los primeros en intentar unir la teoría del control y los sistemas de planificación de IA. Tres libros de texto clásicos sobre las matemáticas de la manipulación de robots se deben a Paul (1981), Craig (1989) y Yoshikawa (1990). El control para la manipulación está cubierto por Murray (2017).

El área de **agarre** también es importante en robótica: el problema de determinar un agarre estable es muy difícil (Mason y Salisbury, 1985). Un agarre competente requiere sensibilidad al tacto o **retroalimentación háptica** para determinar las fuerzas de contacto y detectar el deslizamiento (Fearing y Hollerbach, 1985). Entender cómo agarrar la amplia variedad de objetos en el mundo es una tarea desalentadora. (Bousmalis *et al.,* 2017) describen un sistema que combina experimentación en el mundo real con simulaciones guiadas por transferencia de la simulación a la realidad para producir un agarre robusto.

Retroalimentación háptica

El control de campo potencial, que intenta resolver los problemas de planificación de movimiento y control de manera simultánea, fue desarrollado para la robótica por Khatib (1986). En robótica móvil, esta idea se consideró una solución práctica al problema de evitación de colisiones y luego se extendió a un algoritmo llamado **histogramas de campo vectorial** por Borenstein (1991).

Histograma de campo vectorial

ILQR se utiliza en la actualidad con amplitud en la intersección de la planificación de movimiento y control y se debe a Li y Todorov (2004). Es una variante de la técnica mucho más antigua de programación dinámica diferencial (Jacobson y Mayne, 1970).

La planificación de movimientos finos con detección limitada fue investigada por Lozano-Perez *et al.* (1984) y Canny y Reif (1987). La navegación basada en hitos (Lazanas y Latombe, 1992) utiliza muchas de las mismas ideas en el ámbito de los robots móviles. Las funciones de navegación, la versión de robótica de una política de control para MDP deterministas, fueron introducidas por Koditschek (1987). El trabajo clave que aplica los métodos POMDP (Sección 17.4) a la planificación de movimiento bajo incertidumbre en robótica se debe a Pineau *et al.* (2003) y Roy *et al.* (2005).

El **aprendizaje por refuerzo** en robótica despegó con el trabajo seminal de Bagnell y Schneider (2001) y Ng *et al.* (2003), quienes desarrollaron el paradigma en el contexto del control de helicópteros autónomos. Kober *et al.* (2013) ofrece una visión general de cómo cambia el aprendizaje por refuerzo cuando se aplica al problema de la robótica. Muchas de las técnicas implementadas en sistemas físicos construyen modelos dinámicos aproximados, que se remontan a los modelos lineales ponderados localmente debido a Atkeson *et al.* (1997). Pero los gradientes de política también desempeñaron su papel, permitiendo que los robots humanoides (simplificados) caminen (Tedrake *et al*., 2004), o que un brazo de robot golpee una pelota de béisbol (Peters y Schaal, 2008).

Levine *et al.* (2016) demostraron la primera aplicación de **aprendizaje por refuerzo profundo** en un robot real. Al mismo tiempo, el RL sin modelo en simulación se estaba extendiendo a dominios continuos (Schulman *et al.,* 2015a; Heess *et al.,* 2016; Lillicrap *et al.,* 2015). Otros trabajos aumentaron de manera masiva la recopilación de datos físicos para mostrar el aprendizaje de agarres y modelos dinámicos (Pinto y Gupta, 2016; Agrawal *et al.,* 2017; Levine *et al*., 2018). La transferencia de la simulación a la realidad o **sim-to-real** (Sadeghi y Levine, 2016; Andrychowicz *et al*., 2018a), el **metaprendizaje** (Finn *et al*., 2017), y el aprendizaje por refuerzo sin modelo eficiente en muestras (Andrychowicz *et al*., 2018b) son áreas activas de investigación.

Los primeros métodos para predecir las **acciones humanas** hicieron uso de enfoques de filtrado (Madhavan y Schlenoff, 2003), pero el trabajo seminal de Ziebart *et al.* (2009) propuso la predicción modelando a las personas como agentes aproximadamente racionales. Sadigh *et al.* (2016) capturó cómo estas predicciones deberían depender realmente de lo que el robot decide hacer, construyendo hacia un entorno teórico del juego. Para entornos colaborativos, Sisbot *et al.* (2007) fue pionero en la idea de tener en cuenta lo que las personas quieren en la función de costo del robot. Nikolaidis y Shah (2013) descompusieron la colaboración en aprender cómo actuará el humano, pero también en aprender cómo quiere el humano que actúe el robot, ambos alcanzables a partir de demostraciones. Para el aprendizaje a partir de la demostración, vea Argall *et al.* (2009). Akgun *et al.* (2012) y Sefidgar *et al.* (2017) estudiaron la enseñanza por parte de usuarios finales en lugar de expertos.

Tellex *et al.* (2011) mostraron cómo los robots permiten inferir lo que las personas quieren a partir de instrucciones de lenguaje natural. Finalmente, no sólo los robots necesitan inferir lo que las personas quieren y planean hacer, sino que las personas también necesitan hacer las mismas inferencias sobre los robots. Dragan *et al.* (2013) incorporaron un modelo de las inferencias humanas en la planificación del movimiento del robot.

El campo de la **interacción humano-robot** es mucho más amplio de lo que cubrimos en este capítulo, que se centró en los aspectos de planificación y aprendizaje. Thomaz *et al.* (2016) proporciona una encuesta de interacción más amplia desde una perspectiva computacional. Ross *et al.* (2011) describen el sistema DAGGER.

El tema de las arquitecturas de software para robots genera mucho debate religioso. El buen candidato pasado de moda de IA, la arquitectura de tres capas, se remonta al diseño de Shakey y es revisado por Gat (1998). La arquitectura de subsunción se debe a Brooks (1986), aunque ideas similares fueron desarrolladas de manera independiente por Braitenberg, cuyo libro, *Vehículos* (1984), describe una serie de robots simples basados en el enfoque conductual.

El éxito del robot caminante de seis patas de Brooks fue seguido por muchos otros proyectos. Connell, en su tesis de doctorado (1989), desarrolló un robot móvil completamente reactivo que era capaz de recuperar objetos. Las extensiones del paradigma a sistemas multirrobot se encuentra en el trabajo de Parker (1996) y Mataric (1997). GRL (Horswill, 2000) y COLBERT (Konolige, 1997) resumen las ideas de la robótica basada en comportamientos concurrentes en lenguajes de control de robots generales. Arkin (1998) revisa algunos de los enfoques más populares en este campo.

Dos libros de texto tempranos, de Dudek y Jenkin (2000) y de Murphy (2000), cubren la robótica en general. Las revisiones más recientes se deben a Bekey (2008) y Lynch y Park (2017). Un excelente libro sobre manipulación de robots aborda temas avanzados como el movimiento complaciente (Mason, 2001). La planificación del movimiento del robot se cubre en Choset *et al.* (2005) y LaValle (2006). Thrun *et al.* (2005) introduce la robótica probabilística. *El Manual de robótica* (Siciliano y Khatib, 2016) es una visión general masiva y completa de toda la robótica.

La conferencia principal para la robótica es Robotics: Science and Systems Conference, seguida por la IEEE International Conference on Robotics and Automation. La interacción humano-robot es el lugar principal para la interacción. Las revistas líderes en robótica incluyen *IEEE Robotics and Automation, International Journal of Robotics Research, y Robotics and Autonomous Systems.*

CAPÍTULO 27

FILOSOFÍA, ÉTICA Y SEGURIDAD DE LA IA

En el que consideramos las preguntas fundamentales sobre el significado de la IA: cómo podemos desarrollarla y aplicarla con base en la ética, y cómo podemos mantenerla segura.

Los filósofos han planteado preguntas fundamentales por mucho tiempo: ¿Cómo funcionan las mentes? ¿Es posible que las máquinas actúen de manera inteligente como lo hacen las personas? ¿Tendrían estas máquinas mentes reales y conscientes?

A esas preguntas nosotros añadimos: ¿Cuáles son las implicaciones éticas de las máquinas inteligentes en el uso cotidiano? ¿Se debe permitir a las máquinas decidir matar a los humanos? ¿Pueden los algoritmos ser justos e imparciales? ¿Qué harán los humanos si las máquinas permiten hacer todo tipo de trabajo? ¿Y cómo controlamos las máquinas que pueden llegar a ser más inteligentes que nosotros?

27.1 Los límites de la IA

IA débil
IA fuerte

En 1980, el filósofo John Searle introdujo una distinción entre la **IA débil**, la idea de que las máquinas débiles de IA podrían actuar *como si* fueran inteligentes, y la **IA fuerte**, la afirmación de que las máquinas fuertes de IA que lo hacen están pensando de manera consiente (no sólo *simulando* el pensamiento). Con el tiempo, la definición de IA fuerte se desplazó para referirse a lo que también se llama "IA a nivel humano" o "IA general", programas que permiten resolver una variedad arbitraria y amplia de tareas, incluidas tareas novedosas, y hacerlo tan bien como un humano.

Los críticos de la IA débil que objetaron la posibilidad misma de comportamiento inteligente en las máquinas ahora parecen tan miopes como Simon Newcomb, quien en octubre de 1903 escribió "el vuelo aéreo es uno de los grandes problemas con los que el hombre nunca podrá lidiar", justo dos meses antes del vuelo de los hermanos Wright en el Kitty Hawk. Sin embargo, el progreso vertiginoso de los últimos años no prueba que no pueda haber límites para lo que la IA puede lograr. Alan Turing (1950), la primera persona en definir la IA, también fue el primero en plantearle posibles objeciones, previendo casi todas las que después plantearon otros.

27.1.1 El argumento de la informalidad

El "argumento de la informalidad del comportamiento" de Turing indica que el comportamiento humano es demasiado complejo para ser capturado por cualquier conjunto formal de reglas; los humanos deben usar algunas pautas informales que, el argumento afirma, nunca podrían ser capturadas en un conjunto formal de reglas y, por lo tanto, nunca podrían ser codificadas en un programa de computadora.

Un defensor clave de esta visión fue Hubert Dreyfus, quien produjo una serie de críticas influyentes de la inteligencia artificial: *Lo que las computadoras no pueden hacer* (1972), la secuela *Lo*

que las computadoras aún no pueden hacer (1992), y, con su hermano Stuart, *Mente sobre máquina* (1986). De manera similar, el filósofo Kenneth Sayre (1993) dijo "La inteligencia artificial *perseguida dentro del culto al computacionalismo* no tiene ni siquiera una posibilidad fantasmal de producir resultados duraderos". La tecnología que critican llegó a ser llamada **IA de la vieja escuela (GOFAI)**.

IA de la vieja escuela (GOFAI)

GOFAI corresponde al diseño de agente lógico más simple descrito en el Capítulo 7, allí vimos que es difícil capturar cada contingencia de comportamiento apropiado en un conjunto de reglas lógicas necesarias y suficientes; a eso lo llamamos el **problema de la calificación**. Pero como revisamos en el Capítulo 12, los sistemas de razonamiento probabilístico son más apropiados para dominios abiertos, y como vimos en el Capítulo 21, los sistemas de aprendizaje profundo funcionan bien en una variedad de tareas "informales". Por lo tanto, la crítica no está dirigida contra las computadoras *per se*, sino contra un estilo particular de programarlas con reglas lógicas, un estilo que fue popular en la década de 1980 pero que ha sido eclipsado por nuevos enfoques.

Uno de los argumentos más fuertes de Dreyfus es para los agentes situados en lugar de los motores de inferencia lógica desencarnados. Un agente cuyo entendimiento de "perro" proviene sólo de un conjunto limitado de oraciones lógicas como "$Perro(x) \Rightarrow Mamífero(x)$" está en desventaja en comparación con un agente que ha visto correr a los perros, ha jugado a buscar con ellos y ha sido lamido por uno. Como el filósofo Andy Clark (1998) dice, "Los cerebros biológicos son ante todo los sistemas de control para los cuerpos biológicos. Los cuerpos biológicos se mueven y actúan en entornos reales ricos". Según Clark, somos "buenos en frisbee, malos en lógica".

Cognición encarnada

El enfoque de la **cognición encarnada** sostiene que no tiene sentido considerar el cerebro por separado: la cognición tiene lugar dentro de un cuerpo, que está incrustado en un entorno. Necesitamos estudiar el sistema como un todo; el funcionamiento del cerebro explota regularidades en su entorno, incluyendo el resto de su cuerpo. Bajo el enfoque de la cognición encarnada, la robótica, la visión y otros sensores se vuelven centrales, no periféricos.

En general, Dreyfus vio áreas donde la IA no tenía respuestas completas y afirmó que, por lo tanto, la IA es imposible; ahora vemos muchas de estas mismas áreas sometidas a una investigación y desarrollo continuos que conducen a una mayor capacidad, no a la imposibilidad.

27.1.2 El argumento desde la discapacidad

El "argumento desde la discapacidad" sostiene que "una máquina nunca permite hacer *X*". Como ejemplos de *X*, Turing enumera los siguientes:

> Sea amable, ingenioso, hermoso, amigable, tenga iniciativa, tenga sentido del humor, distinga lo correcto de lo incorrecto, cometa errores, enamórese, disfrute de las fresas con crema, haga que alguien se enamore de usted, aprenda de la experiencia, use las palabras de manera correcta, sea el sujeto de sus propios pensamientos, tenga tanta diversidad de comportamiento como el hombre, haga algo realmente nuevo.

En retrospectiva, algunas de estas acciones son muy fáciles, todos estamos familiarizados con las computadoras que "cometen errores". Las computadoras con capacidades de metarrazonamiento (Capítulo 5) permiten examinar sus propios cálculos, siendo así el sujeto de su propio razonamiento. Una tecnología centenaria tiene la capacidad probada de "hacer que alguien se enamore de ella", el oso de peluche. El experto en ajedrez por computadora David Levy predice que para 2050 la gente se enamorará de los robots humanoides de manera cotidiana. En cuanto a un robot enamorándose, ese es un tema común en la ficción,[1] pero sólo ha habido una especulación académica limitada sobre

[1] Por ejemplo, la ópera *Coppélia* (1870), la novela *¿Sueñan los androides con ovejas eléctricas?* (1968), las películas *IA* (2001), *Wall-E* (2008) y *Her* (2013).

el tema (Kim *et al*., 2007). Las computadoras han hecho cosas que son "realmente nuevas", haciendo descubrimientos significativos en astronomía, matemáticas, química, mineralogía, biología, informática y otros campos, y han creado nuevas formas de arte a través de la transferencia de estilo (Gatys *et al*., 2016). En general, los programas superan el rendimiento humano en algunas tareas y se quedan atrás en otras. Lo único que es claro que no pueden hacer es ser humanos.

27.1.3 La objeción matemática

Turing (1936) y Gödel (1931) demostraron que ciertas preguntas matemáticas son en principio irresolubles por sistemas formales particulares. El teorema de la incompletitud de Gödel (ver Sección 9.5) es el ejemplo más famoso de esto. En resumen, para cualquier marco axiomático formal F lo suficientemente poderoso para hacer aritmética, es posible construir una llamada sentencia de Gödel $G(F)$ con las siguientes propiedades:

- $G(F)$ es una sentencia de F, pero no permite ser probada dentro de F.
- Si F es consistente, entonces $G(F)$ es verdadera.

Filósofos como J. R. Lucas (1961) han afirmado que este teorema demuestra que las máquinas son inferiores a los humanos, porque las máquinas son sistemas formales que están limitados por el teorema de la incompletitud, no es posible establecer la verdad de su propia sentencia de Gödel, mientras que los humanos no tienen tal limitación. Esto ha causado mucha controversia y ha generado vasta literatura, incluidos dos libros del matemático/físico Sir Roger Penrose (1989, 1994). Penrose repite la afirmación de Lucas con algunas novedades, como la hipótesis de que los humanos son diferentes porque sus cerebros operan por gravedad cuántica, una teoría que hace múltiples predicciones falsas sobre la fisiología cerebral.

Examinaremos tres de los problemas con la afirmación de Lucas. Primero, un agente no debería avergonzarse de que no pueda establecer la verdad de alguna sentencia mientras que otros agentes pueden. Considere la siguiente sentencia:

> Lucas no puede afirmar de manera consistente que esta sentencia es verdadera.

Si Lucas afirmara esta sentencia, entonces se estaría contradiciendo a sí mismo; por lo tanto, no es posible afirmarla de manera consistente y, en consecuencia, es verdadera. Hemos demostrado así que hay una sentencia verdadera que Lucas no puede afirmar de manera consistente mientras que otras personas (y máquinas) pueden. Pero eso no nos hace pensar menos en Lucas.

En segundo lugar, el teorema de incompletitud de Gödel y los resultados relacionados se aplican a las *matemáticas*, no a las *computadoras*. Ninguna entidad, humana o máquina, permite probar cosas que son imposibles de probar. Lucas y Penrose asumen con falsedad que los humanos pueden de alguna manera sortear estos límites, como cuando Lucas (1976) dice "debemos asumir nuestra propia consistencia, si el pensamiento en verdad ha de ser posible". Pero esta es una suposición infundada: los humanos son inconsistentes. Esto es, de hecho, cierto para el razonamiento cotidiano, pero también lo es para el pensamiento matemático cuidadoso. Un ejemplo famoso es el problema del mapa de cuatro colores. Alfred Kempe (1879) publicó una prueba que fue aceptada durante 11 años hasta que Percy Heawood (1890) señaló una imperfección.

En tercer lugar, el teorema de incompletitud de Gödel se aplica sólo a sistemas formales que son lo suficientemente poderosos para hacer aritmética. Esto incluye las máquinas de Turing. La afirmación de Lucas se basa en parte en la afirmación de que las computadoras son equivalentes a las máquinas de Turing. Esto no es del todo cierto. Las máquinas de Turing son infinitas, pero las computadoras y los cerebros son finitos, y cualquier computadora puede, por lo tanto, ser descrita como un sistema

(muy grande) en lógica proposicional, que no está sujeto al teorema de incompletitud de Gödel. Lucas asume que los humanos pueden "cambiar sus mentes" mientras que las computadoras no lo permiten; pero eso también es falso, una computadora puede retractarse de una conclusión después de nuevas evidencias o de una deliberación adicional, es posible actualizar su hardware y puede cambiar sus procesos de toma de decisiones con aprendizaje automático o reescritura de software.

27.1.4 Medir la IA

Alan Turing, en su famoso artículo "Computing Machinery and Intelligence" (1950), sugirió que en lugar de preguntar si las máquinas pueden pensar, deberíamos preguntar si las máquinas pasan una prueba de comportamiento, que ha llegado a ser llamada la **prueba de Turing**. La prueba requiere que un programa tenga una conversación (a través de mensajes escritos) con un interrogador durante cinco minutos. El interrogador entonces tiene que adivinar si la conversación es con un programa o una persona; el programa pasa la prueba si engaña al interrogador 30 % del tiempo. Para Turing, el punto clave no eran los detalles exactos de la prueba, sino la idea de medir la inteligencia por el rendimiento en algún tipo de tarea de comportamiento abierto, en lugar de por especulación filosófica.

No obstante, Turing conjeturó que para el año 2000 una computadora con un almacenamiento de mil millones de unidades podría pasar la prueba, pero aquí estamos al otro lado del 2000, y todavía no podemos ponernos de acuerdo en si algún programa lo ha logrado. Muchas personas han sido engañadas cuando no sabían que podrían estar chateando con una computadora. El programa ELIZA y los chatbots de Internet como MGONZ (Humphrys, 2008) y NATACHATA (Jonathan *et al*., 2009) engañan a sus corresponsales de manera repetida, y el chatbot CYBERLOVER ha atraído la atención de las autoridades policiales debido a su afición por engañar a los demás chateadores para que revelen suficiente información personal para que su identidad pueda ser robada.

En 2014, un chatbot llamado Eugene Goostman engañó a 33 % de los jueces aficionados no entrenados en una prueba de Turing. El programa afirmaba ser un niño de Ucrania con un dominio limitado del inglés; esto ayudó a explicar sus errores gramaticales. Quizá la prueba de Turing es una prueba de la credulidad humana. Hasta ahora ningún juez bien entrenado ha sido engañado (Aaronson, 2014).

Las competencias de la prueba de Turing han llevado a mejores chatbots, pero no han sido un foco de investigación dentro de la comunidad de IA. En cambio, los investigadores de IA que anhelan la competencia tienen más probabilidades de concentrarse en jugar ajedrez, Go, StarCraft II, de tomar un examen de ciencias de 8º grado o de identificar objetos en imágenes. En muchas de estas competencias, los programas han alcanzado o superado el rendimiento a nivel humano, pero eso no significa que los programas sean humanos fuera de la tarea específica. El punto es mejorar la ciencia y la tecnología básicas y proporcionar herramientas útiles, no engañar a los jueces.

27.2 ¿Pueden las máquinas pensar?

Algunos filósofos afirman que una máquina que actúa de manera inteligente no estaría pensando, sino que sólo sería una *simulación* de pensamiento. Pero a la mayoría de los investigadores de IA no les preocupa la distinción; de hecho, el científico de la computación Edsger Dijkstra (1984) dijo que "La pregunta de si las *máquinas pueden pensar*... es tan relevante como la pregunta de si los *submarinos pueden nadar*". La primera definición de *nadar* del *American Heritage Dictionary's* es "Moverse a través del agua mediante extremidades, aletas o cola"; la mayoría de las personas están de acuerdo en que los submarinos, al no tener extremidades, no nadan. El diccionario también define *volar* como "Moverse por el aire mediante alas o partes similares a alas"; la mayoría de las personas están de acuerdo en que los aviones, al tener partes similares a alas,

pueden volar. Sin embargo, ni las preguntas ni las respuestas tienen relevancia para el diseño o las capacidades de los aviones y submarinos; más bien se trata del uso de palabras en inglés. (El hecho de que los barcos *naden* (*"privet"*) en ruso amplifica este punto). Los hablantes de inglés aún no se han decidido por una definición precisa para la palabra "pensar", ¿requiere "un cerebro" o "partes parecidas a un cerebro"?

De nuevo, el tema fue abordado por Turing. Él observa que nunca tenemos *alguna* evidencia directa sobre los estados mentales internos de otros humanos, una especie de solipsismo mental. Aunque, dice Turing, "En lugar de discutir de manera continua sobre este punto, es usual tener la **convención de cortesía** de que todos piensan". Turing argumenta que también extenderíamos la cortesía a la convención de las máquinas, si sólo tuviéramos experiencia con las que actúan de manera inteligente. Sin embargo, ahora que tenemos algo de experiencia, parece que nuestra disposición para atribuir sensibilidad depende al menos tanto de la apariencia y voz humanoides como de la pura inteligencia.

Convención de cortesía

27.2.1 La habitación china

El filósofo John Searle rechaza la amabilidad cortés. Su famoso argumento de la **habitación china** (Searle, 1990) es el siguiente: Imagine a un humano, que sólo entiende inglés, dentro de una habitación que contiene un libro de reglas, escrito en inglés, y varias pilas de papel. Se deslizan bajo la puerta de la habitación pedazos de papel que contienen símbolos indescifrables. El humano sigue las instrucciones en el libro de reglas, encuentra símbolos en las pilas, escribe símbolos en nuevos pedazos de papel, reorganiza las pilas, y así sucesivamente. De manera eventual, las instrucciones harán que uno o más símbolos sean transcritos en un pedazo de papel que se pasa de vuelta al mundo exterior. Desde el exterior, vemos un sistema que está tomando entrada en forma de frases chinas y generando respuestas chinas fluidas e inteligentes.

Habitación china

Searle entonces argumenta: se da por hecho que el humano no entiende chino. El libro de reglas y las pilas de papel, siendo sólo pedazos de papel, no entienden chino. Por lo tanto, no hay comprensión del chino. Además, Searle dice que la habitación china está haciendo lo mismo que haría una computadora, por lo tanto, las computadoras no generan comprensión.

Searle (1980) es un defensor del **naturalismo biológico**, según el cual los estados mentales son características emergentes de alto nivel que son causadas por procesos físicos de bajo nivel *en las neuronas*, y son las (no especificadas) propiedades de las neuronas las que importan: según los prejuicios de Searle, las neuronas tienen "eso", los transistores no. Ha habido muchas refutaciones del argumento de Searle, pero no hay consenso. Su argumento podría ser usado (quizá por robots) para argumentar que un humano no puede tener verdadera comprensión; después de todo, un humano está hecho de células, las células no entienden; entonces, no hay comprensión. De hecho, esa es la trama de la historia de ciencia ficción de Terry Bisson (1990) *They're Made Out of Meat*, en la que robots alienígenas exploran la Tierra y no son capaces de creer que trozos de carne puedan ser conscientes. Cómo pueden serlo sigue siendo un misterio.

Naturalismo biológico

27.2.2 Conciencia y qualia

A lo largo de todos los debates sobre la IA fuerte está el tema de la **conciencia**: la conciencia del mundo exterior, del yo, y la experiencia subjetiva de vivir. El término técnico para la naturaleza intrínseca de las experiencias es **qualia** (del latín, "de qué tipo"). La gran pregunta es si las máquinas pueden tener *qualia*. En la película *2001*, cuando el astronauta David Bowman está desconectando los "circuitos cognitivos" de la computadora HAL 9000, dice *"Tengo miedo, Dave. Dave, mi mente se está yendo. Puedo sentirlo"*. ¿HAL tiene sentimientos (y merece simpatía)? ¿O la respuesta es sólo una respuesta algorítmica, no diferente de "Error 404: no encontrado"?

Conciencia

Qualia

Existe una pregunta similar para los animales: los dueños de mascotas están seguros de que su perro o gato tiene conciencia, pero no todos los científicos están de acuerdo. Los grillos cambian su comportamiento basado en la temperatura, pero pocas personas dirían que los grillos experimentan la *sensación* de estar calientes o fríos.

Una razón por la que el problema de la conciencia es complicado es porque sigue siendo mal definido, aun después de siglos de debate. Pero es posible que la ayuda esté en camino. Recién, los filósofos se han unido a los neurocientíficos bajo los auspicios de la Fundación Templeton para iniciar una serie de experimentos que podrían resolver algunos de los problemas. Los defensores de dos teorías principales de la conciencia (la teoría del espacio de trabajo global y la teoría de la información integrada) han acordado que los experimentos podrían confirmar una teoría sobre la otra, una rareza en la filosofía.

Alan Turing (1950) concede que la cuestión de la conciencia es difícil, pero niega que tenga mucha relevancia para la práctica de la IA: "No deseo dar la impresión de que creo que no hay misterio sobre la conciencia... Pero no creo que estos misterios deban ser resueltos antes de que podamos responder a la pregunta con la que estamos preocupados en este artículo". Estamos de acuerdo con Turing, nos interesa crear programas que se comporten de manera inteligente. Los aspectos individuales de la conciencia, la conciencia de sí mismo y de la atención pueden ser programados y ser parte de una máquina inteligente. El proyecto adicional de hacer que una máquina sea consciente, de manera exacta a como los humanos lo son, no es uno que estemos equipados para asumir. Estamos de acuerdo en que comportarse de manera inteligente requerirá cierto grado de *conciencia*, que variará de una tarea a otra tarea, y que las tareas que involucran interacción con humanos requerirán un modelo de experiencia subjetiva humana.

En el asunto de modelar la experiencia, los humanos tienen una clara ventaja sobre las máquinas, porque usan su propio aparato subjetivo para apreciar la experiencia subjetiva de los demás. Por ejemplo, si quiere saber *cómo se siente* cuando alguien se golpea el pulgar con un martillo, puede golpearse el pulgar con un martillo. Las máquinas no tienen tal capacidad, aunque a diferencia de los humanos, permiten ejecutar el código de los demás.

27.3 La ética de la IA

Dado que la IA es una tecnología poderosa, tenemos la obligación moral de usarla bien para promover los aspectos positivos y evitar o mitigar los negativos.

Los aspectos positivos son muchos. Por ejemplo, la IA permite salvar vidas mediante un mejor diagnóstico médico, nuevos descubrimientos médicos, una mejor predicción de eventos climáticos extremos y una conducción más segura con asistencia al conductor y (eventualmente) tecnologías de conducción autónoma. También hay muchas oportunidades para mejorar vidas. El programa AI for Humanitarian Action de Microsoft aplica la IA para recuperarse de desastres naturales, atender las necesidades de los niños, proteger a los refugiados y promover los derechos humanos. El programa AI for Social Good de Google apoya el trabajo en la protección de la selva tropical, la jurisprudencia de derechos humanos, el monitoreo de la contaminación, la medición de las emisiones de combustibles fósiles, la consejería en crisis, la verificación de noticias, la prevención del suicidio, el reciclaje y otros problemas. El University of Chicago's Center for Data Science for Social Good aplica el aprendizaje automático a problemas en justicia penal, desarrollo económico, educación, salud pública, energía y medio ambiente.

Las aplicaciones de IA en la gestión de cultivos y la producción de alimentos ayudan a alimentar al mundo. La optimización de los procesos empresariales mediante la utilización del aprendizaje automático hará que las empresas sean más productivas, aumenten la riqueza y proporcionen más empleo. La automatización permite reemplazar las tareas tediosas y peligrosas que enfrentan

muchos trabajadores, y liberarlos para que se concentren en aspectos más interesantes de las tareas. Las personas con discapacidades se beneficiarán de la asistencia basada en IA para ver, oír y moverse. La traducción automática ya permite a las personas de diferentes culturas comunicarse. Las soluciones de IA basadas en software tienen un costo marginal de producción cercano a cero y, por lo tanto, tienen el potencial de democratizar el acceso a la tecnología avanzada (aun cuando otros aspectos del software tienen el potencial de centralizar el poder).

A pesar de muchos de estos aspectos positivos, no debemos ignorar los negativos. Muchas nuevas tecnologías han tenido **efectos secundarios negativos** no deseados: la fisión nuclear trajo Chernobyl y la amenaza de destrucción global; el motor de combustión interna trajo contaminación del aire, calentamiento global y la pavimentación del paraíso. Otras tecnologías pueden tener efectos negativos aun cuando se usan según lo previsto, como el gas sarín, los rifles AR-15 y la solicitud telefónica. La automatización creará riqueza, pero bajo las condiciones económicas actuales gran parte de esa riqueza fluirá hacia los propietarios de los sistemas automatizados, lo que llevará a un aumento de la desigualdad de ingresos. Esto es disruptivo para una sociedad que funciona bien. En los países en desarrollo, el camino tradicional hacia el crecimiento a través de la fabricación de bajo costo para la exportación puede ser cortado, ya que los países ricos adoptan instalaciones de fabricación automatizadas en sus propios territorios. Nuestras decisiones éticas y de gobernanza dictarán el nivel de desigualdad que la IA generará.

Efectos secundarios negativos

Todos los científicos e ingenieros enfrentan consideraciones éticas sobre qué proyectos deben o no deben asumir, y cómo asegurarse de que la ejecución del proyecto sea segura y benéfica. En 2010, el Engineering and Physical Sciences Research Council del Reino Unido celebró una reunión para desarrollar un conjunto de "principios de robótica". En años posteriores, otras agencias gubernamentales, organizaciones sin fines de lucro y empresas crearon conjuntos similares de principios. La esencia es que cada organización que crea tecnología de IA, y todos en la organización, tienen la responsabilidad de asegurarse de que la tecnología contribuya al bien, no al daño. Los principios más citados son:

Garantizar la seguridad	Establecer responsabilidad
Garantizar la equidad	Defender los derechos humanos y los valores
Respetar la privacidad	Reflejar la diversidad e inclusión
Promover la colaboración	Evitar la concentración de poder
Proporcionar transparencia	Reconocer las implicaciones legales y políticas
Limitar los usos perjudiciales de la IA	Contemplar las implicaciones para el empleo

Tenga en cuenta que muchos de los principios, como "garantizar la seguridad", son aplicables a todos los sistemas de software o hardware, no sólo a los sistemas de IA. Varios principios están redactados de manera vaga, lo que dificulta su medición o cumplimiento. Eso se debe en parte a que la IA es un campo grande con muchos subcampos, cada uno de los cuales tiene un conjunto diferente de normas históricas y diferentes relaciones entre los desarrolladores de IA y las partes interesadas. Mittelstadt (2019) sugiere que los subcampos deberían desarrollar cada uno pautas de acción más específicas y precedentes de casos.

27.3.1 Armas autónomas letales

La ONU define una arma autónoma letal como aquella que localiza, selecciona y ataca (es decir, mata) objetivos humanos sin supervisión humana. Varias armas cumplen algunos de estos criterios. Por ejemplo, las minas terrestres se han utilizado desde el siglo XVII: permiten seleccionar y atacar objetivos en un sentido limitado según el grado de presión ejercida o la cantidad de metal presente, pero no pueden salir y localizar objetivos por sí mismas. (Las minas terrestres

están prohibidas por el Tratado de Ottawa). Los misiles guiados, en uso desde la década de 1940, permiten perseguir objetivos, pero tienen que ser apuntados en la dirección general correcta por un humano. El autodisparo de las armas controladas por radar se han utilizado para defender barcos navales desde la década de 1970; están destinadas a destruir misiles entrantes, pero también podrían atacar aeronaves tripuladas. Aunque la palabra "autónomo" con frecuencia se usa para describir vehículos aéreos no tripulados o **drones**, la mayoría de estas armas son pilotadas a distancia y requieren la actuación humana de la carga letal.

Al momento de escribir estas líneas, múltiples sistemas de armas parecen haber cruzado la línea hacia la plena autonomía. Por ejemplo, el misil Harop de Israel es una "munición de merodeo" con una envergadura de diez pies y una ojiva de cincuenta libras. Busca durante hasta seis horas en una región geográfica dada cualquier objetivo que cumpla con un criterio dado y luego lo destruye. El criterio podría ser "emite una señal de radar que se parece al radar antiaéreo" o "parece un tanque". El fabricante turco STM publicita su cuadricóptero Kargu, que lleva hasta 1.5 kg de explosivos, como capaz de "Golpe autónomo... objetivos seleccionados en imágenes... seguimiento de objetivos en movimiento... antipersonal... reconocimiento facial".

Las armas autónomas han sido llamadas la "tercera revolución en la guerra" después de la pólvora y las armas nucleares. Su potencial militar es obvio. Por ejemplo, pocos expertos dudan de que los aviones de combate autónomos derrotarían a cualquier piloto humano. Los aviones, tanques y submarinos autónomos son más baratos, más rápidos, más maniobrables y tienen un alcance más largo que sus contrapartes tripuladas.

Desde 2014, las Naciones Unidas en Ginebra han llevado a cabo discusiones regulares bajo los auspicios de la Convención sobre ciertas Armas Convencionales (CCW) sobre la cuestión de si prohibir o no las armas autónomas letales. Al momento de escribir estas líneas, 30 naciones, que van desde China hasta la Santa Sede, han declarado su apoyo a un tratado internacional, mientras que otros países clave, incluyendo Israel, Rusia, Corea del Sur y Estados Unidos, se oponen a la prohibición.

El debate sobre las armas autónomas incluye aspectos legales, éticos y prácticos. Los problemas legales están gobernados por el CCW, que requiere la posibilidad de discriminar entre combatientes y no combatientes, el juicio de la necesidad militar de un ataque, y la evaluación de la proporcionalidad entre el valor militar de un objetivo y la posibilidad de daño colateral. La factibilidad de cumplir con estos criterios es una pregunta de ingeniería, cuya respuesta sin duda cambiará con el tiempo. En la actualidad, la discriminación parece factible en algunas circunstancias y sin duda mejorará rápido, pero la necesidad y la proporcionalidad no son factibles en la actualidad: requieren que las máquinas realicen juicios subjetivos y situacionales que son de manera considerable más difíciles que las tareas relativamente simples de buscar y comprometer objetivos potenciales. Por estas razones, sería legal usar armas autónomas sólo en circunstancias en las que un operador humano pueda prever de manera razonable que la ejecución de la misión no resultará en que se apunte a civiles o que las armas realicen ataques innecesarios o desproporcionados. Esto significa que, por el momento, sólo se podrían realizar misiones muy restringidas con armas autónomas.

En el lado ético, algunos consideran inaceptable delegar la decisión de matar humanos a una máquina. Por ejemplo, el embajador de Alemania en Ginebra ha declarado que "no aceptará que la decisión sobre la vida y la muerte sea tomada por un sistema autónomo", mientras que Japón "no tiene planes de desarrollar robots con humanos fuera del bucle, que puedan ser capaces de cometer asesinato". El general Paul Selva, segundo oficial militar de mayor rango en Estados Unidos, dijo en 2017, "No creo que sea razonable para nosotros poner a los robots a cargo de si tomamos una vida humana o no". Por último, António Guterres, Secretario General de las Naciones Unidas, declaró en 2019 que "las máquinas con el poder y la discreción para quitar vidas sin la

participación humana son políticamente inaceptables, moralmente repugnantes y deberían ser prohibidas por la ley internacional".

Más de 140 ONGs en más de 60 países son parte de la Campaña para Detener los Robots Asesinos, y una carta abierta organizada en 2015 por el Future of Life Institute fue firmada por más de 4 000 investigadores de IA[2] y 22 000 personas.

Contra esto, es posible argumentar que a medida que la tecnología mejora, debería ser posible desarrollar armas que sean *menos* propensas que los soldados o pilotos humanos a causar bajas civiles. (También está el beneficio importante de que las armas autónomas reducen la necesidad de que los soldados y pilotos humanos arriesguen la vida). Los sistemas autónomos no sucumbirán a la fatiga, frustración, histeria, miedo, ira o venganza, y no necesitan "disparar primero y preguntar después" (Arkin, 2015). Así como las municiones guiadas han reducido el daño colateral en comparación con las bombas no guiadas, se espera que las armas inteligentes mejoren aún más la precisión de los ataques. (Contra esto, ver Benjamin (2013) para un análisis de las bajas en la guerra de drones). Esta, en apariencia, es la posición de Estados Unidos en la última ronda de negociaciones en Ginebra.

Quizá de manera contraintuitiva, Estados Unidos es también uno de los pocos países cuyas políticas propias prohíben el uso de armas autónomas. El plan de ruta del Departamento de Defensa de Estados Unidos de 2011 dice: "Por el futuro previsible, las decisiones sobre el uso de la fuerza [por sistemas autónomos] y la elección de qué objetivos individuales involucrar con fuerza letal se mantendrán bajo control humano". La razón principal de esta política es práctica: los sistemas autónomos no son lo suficientemente confiables para entregarles decisiones militares.

El problema de la confiabilidad salió a la luz el 26 de septiembre de 1983, cuando la pantalla de la computadora del oficial de misiles soviético Stanislav Petrov mostró una alerta de un ataque de misiles entrante. Según el protocolo, Petrov debería haber iniciado un contraataque nuclear, pero sospechó que la alerta era un error y la trató como tal. Estaba en lo correcto, y la Tercera Guerra Mundial (apenas) fue evitada. No sabemos qué hubiera pasado si no hubiera habido un humano en el bucle.

La confiabilidad es una preocupación muy seria para los comandantes militares, que conocen bien la complejidad de las situaciones de campo de batalla. Los sistemas de aprendizaje automático que operan en el entrenamiento pueden tener un rendimiento cuando se despliegan. Los ciberataques contra armas autónomas podrían resultar en bajas por fuego amigo; desconectar el arma de toda comunicación permite prevenir eso (asumiendo que no ha sido comprometida ya), pero entonces el arma no puede ser retirada si está funcionando mal.

El problema práctico predominante con las armas autónomas es que son armas de destrucción masiva escalables, en el sentido de que la escala de un ataque que se puede lanzar es proporcional a la cantidad de hardware que es posible desplegar. Un cuadricóptero de dos pulgadas de diámetro permite llevar una carga explosiva letal, y un millón pueden caber en un contenedor de envío regular. Precisamente porque son autónomas, estas armas no necesitarían un millón de supervisores humanos para hacer su trabajo.

Como armas de destrucción masiva, las armas autónomas escalables tienen ventajas para el atacante en comparación con las armas nucleares y el bombardeo masivo: dejan la propiedad intacta y permiten ser aplicadas de manera selectiva para eliminar sólo a aquellos que podrían amenazar a una fuerza ocupante. Ciertamente podrían ser utilizadas para exterminar a un grupo étnico entero o a todos los adherentes de una religión particular. En muchas situaciones, también serían indetectables. Estas características las hacen en particular atractivas para los actores no estatales.

[2] Incluidos los dos autores de este libro.

Estas consideraciones, en particular las características que favorecen al atacante, sugieren que las armas autónomas reducirán la seguridad global y nacional para todas las partes. La respuesta racional para los gobiernos parece ser participar en discusiones de control de armas en lugar de una carrera armamentista.

Doble uso

No obstante, el proceso de diseñar un tratado no está exento de dificultades. La IA es una tecnología de **doble uso**: las tecnologías de IA que tienen aplicaciones pacíficas como el control de vuelo, el seguimiento visual, la cartografía, la navegación, y la planificación multiagente son aplicadas con facilidad a propósitos militares. Es fácil convertir un cuadricóptero autónomo en un arma adjuntando un explosivo y ordenándole que busque un objetivo. Lidiar con esto requerirá una implementación cuidadosa de regímenes de cumplimiento con la cooperación de la industria, como ya ha sido demostrado con cierto éxito por la Chemical Weapons Convention.

27.3.2 Vigilancia, seguridad y privacidad

En 1976, Joseph Weizenbaum advirtió que la tecnología de reconocimiento automático de voz podría conducir a espionaje telefónico generalizado y, en consecuencia, a la pérdida de libertades civiles. Hoy esa amenaza se ha hecho realidad con la mayoría de las comunicaciones electrónicas, pasando por servidores centrales que pueden ser monitoreados, y ciudades llenas de micrófonos y cámaras que permiten identificar y rastrear individuos con base en el reconocimiento de su voz, rostro y forma de caminar. La vigilancia, que solía requerir recursos humanos caros y escasos, ahora puede ser realizada a gran escala por máquinas.

Cámara de vigilancia

A partir de 2018, había hasta 350 millones de **cámaras de vigilancia** en China y 70 millones en Estados Unidos. China y otros países han comenzado a exportar tecnología de vigilancia a países de baja tecnología, algunos con reputación de maltrato a sus ciudadanos y de atacar de manera desproporcionada a las comunidades marginadas. Los ingenieros de IA deben tener claro qué usos de la vigilancia son compatibles con los derechos humanos y rechazar trabajar en aplicaciones que son incompatibles.

A medida que más de nuestras instituciones operan online, nos volvemos más vulnerables al cibercrimen (phishing, fraude con tarjetas de crédito, botnets, ransomware) y al ciberterrorismo (incluidos ataques potencialmente mortales como el cierre de hospitales y plantas de energía o el secuestro de coches autónomos). El aprendizaje automático es una herramienta poderosa para ambos lados en la batalla de la **ciberseguridad**. Los atacantes pueden usar la automatización para buscar inseguridades y permiten aplicar el aprendizaje por refuerzo para intentos de phishing y chantaje automatizado. Los defensores pueden usar el aprendizaje no supervisado para detectar patrones de tráfico entrante anómalos (Chandola *et al.,* 2009; Malhotra *et al.,* 2015) y varias técnicas de aprendizaje automático para detectar fraudes (Fawcett y Provost, 1997; Bolton y Hand, 2002). A medida que los ataques se vuelven más sofisticados, hay una mayor responsabilidad para todos los ingenieros, no sólo los expertos en seguridad, para diseñar sistemas seguros desde el principio. Una previsión (Kanal, 2017) sitúa el mercado del aprendizaje automático en ciberseguridad en alrededor de $100 mil millones para 2021.

Ciberseguridad

A medida que interactuamos con las computadoras durante cada vez más tiempo de nuestras vidas cotidianas, los gobiernos y las corporaciones recopilan más datos sobre nosotros. Los recolectores de datos tienen una responsabilidad moral y legal de ser buenos administradores de los datos que poseen. En Estados Unidos, la Ley de Portabilidad y Responsabilidad del Seguro de Salud (HIPAA) y la Ley de Derechos Educativos y Privacidad Familiar (FERPA) protegen la privacidad de los registros médicos y estudiantiles. El Reglamento General de Protección de Datos (GDPR) de la Unión Europea exige que las empresas diseñen sus sistemas con la protección de los datos en mente y requiere que obtengan el consentimiento del usuario para cualquier recolección o procesamiento de datos.

Equilibrado contra el derecho individual a la privacidad está el valor que la sociedad obtiene al compartir datos. Queremos poder detener a los terroristas sin oprimir la disidencia pacífica, y queremos curar enfermedades sin comprometer el derecho de ningún individuo a mantener su historial médico privado. Una práctica clave es la **desidentificación**: eliminando la identificación personal información (como nombre y número de seguridad social) para que los investigadores médicos puedan usar los datos para avanzar en el bien común. El problema es que los datos desidentificados compartidos pueden estar sujetos a reidentificación. Por ejemplo, si los datos eliminan el nombre, el número de seguridad social y la dirección de la calle, pero incluyen la fecha de nacimiento, el género y el código postal, entonces, como mostró Latanya Sweeney (2000), 87 % de la población de Estados Unidos puede ser reidentificada de manera única. Sweeney enfatizó este punto al reidentificar el registro de salud del gobernador de su estado cuando fue admitido al hospital. En la competencia del **Premio Netflix**, se desidentificaron los registros de las calificaciones de películas individuales que fueron lanzadas, y se pidió a los competidores que idearan un algoritmo de aprendizaje automático que pudiera predecir con precisión qué películas le gustarían a un individuo. Pero los investigadores pudieron reidentificar a los usuarios individuales al hacer coincidir la fecha de una calificación en la base de datos de Netflix con la fecha de una calificación similar en la Base de Datos de Películas de Internet (IMDB), donde los usuarios a veces usan sus nombres reales (Narayanan y Shmatikov, 2006).

Desidentificación

Premio Netflix

Este riesgo permite mitigarse en cierta medida generalizando campos: por ejemplo, al reemplazar la fecha de nacimiento exacta con el año de nacimiento, o un rango más amplio como "20-30 años". Eliminar un campo por completo puede verse como una forma de generalizar a "cualquiera". Pero la generalización por sí sola no garantiza que los registros estén a salvo de la reidentificación; puede ser que sólo haya una persona en el código postal 94720 que tiene entre 90 y 100 años. Una propiedad útil es la **k-anonimidad**: una base de datos es k-anonimizada si cada registro en la base de datos es indistinguible de al menos $k - 1$ otros registros. Si hay registros que son más únicos que esto, tendrían que ser más generalizados.

K-anonimidad

Una alternativa a compartir registros desidentificados es mantener todos los registros privados, pero permitir **consulta agregada**. Se proporciona una API para consultas contra la base de datos y las consultas válidas reciben una respuesta que resume los datos con un conteo o promedio. Pero no se da ninguna respuesta si violara ciertas garantías de privacidad. Por ejemplo, podríamos permitir a un epidemiólogo preguntar, para cada código postal, el porcentaje de personas con cáncer. Para códigos postales con al menos n personas se daría un porcentaje (con una pequeña cantidad de ruido aleatorio), pero no se daría ninguna respuesta para códigos postales con menos de n personas.

Consulta agregada

Se debe tener cuidado para protegerse contra la desidentificación utilizando múltiples consultas. Por ejemplo, si la consulta "salario promedio y número de empleados de la empresa XYZ de 30-40 años" da la respuesta [$81 234 12] y la consulta "salario promedio y número de empleados de la empresa XYZ de 30-41 años" da la respuesta [$81 199 13], y si usamos LinkedIn para encontrar al único de 41 años en la empresa XYZ, entonces los hemos identificado con éxito, y podemos calcular su salario exacto, aunque todas las respuestas involucraron a 12 o más personas. El sistema debe ser cuidadosamente diseñado para proteger contra esto, con una combinación de límites en las consultas que se pueden hacer (quizá sólo se permite consultar un conjunto predefinido de rangos de edad no superpuestos) y la precisión de los resultados (quizás ambas consultas dan la respuesta "alrededor de $81 000").

Una garantía más sólida es la **privacidad diferencial**, que asegura que un atacante no es capaz de usar consultas para reidentificar a cualquier individuo en la base de datos, aun si el atacante puede hacer múltiples consultas y tiene acceso a bases de datos de enlace separadas. La respuesta a la consulta emplea un algoritmo aleatorizado que agrega una pequeña cantidad de ruido al resultado. Dada una base de datos D, cualquier registro en la base de datos r, cualquier consulta Q, y una

Privacidad diferencial

posible respuesta y a la consulta, decimos que la base de datos D tiene privacidad ϵ-diferencial si la probabilidad logarítmica de la respuesta y varía en menos de ϵ cuando agregamos el registro r:

$$|\log P(Q(D) = y) - \log P(Q(D + r) = y)| \leq \epsilon.$$

En otras palabras, si una persona decide participar en la base de datos o no, no hace ninguna diferencia en las respuestas que cualquiera puede obtener, y, por lo tanto, no hay desincentivo de privacidad para participar. Muchas bases de datos están diseñadas para garantizar la privacidad diferencial.

Aprendizaje federado

Hasta ahora hemos considerado el problema de compartir datos desidentificados desde una base de datos central. El enfoque **aprendizaje federado** (Konečnỳ *et al.,* 2016) no tiene base de datos central; en cambio, los usuarios mantienen sus propias bases de datos locales que alojan sus datos privados. Sin embargo, permiten compartir parámetros de un modelo de aprendizaje automático que se mejora con sus datos, sin el riesgo de revelar ninguno de los datos privados. Imagine una aplicación de comprensión del habla que los usuarios ejecutan de manera local en su teléfono. Esta aplicación contiene una red neuronal de base, que luego se mejora mediante el entrenamiento local en las palabras que se escuchan en el teléfono del usuario. De manera periódica, los propietarios de la aplicación sondean a un subconjunto de los usuarios y les piden los valores de los parámetros de su red local mejorada, pero no de ninguno de sus datos brutos. Los valores de los parámetros se combinan para formar un nuevo modelo mejorado que luego se pone a disposición de todos los usuarios, para que todos obtengan el beneficio del entrenamiento que realizan otros usuarios.

Agregación segura

Para que este esquema preserve la privacidad, tenemos que poder garantizar que los parámetros del modelo compartidos por cada usuario no pueden ser ingeniería inversa. Si enviamos los parámetros brutos, existe la posibilidad de que un adversario que los inspeccione pueda deducir si, por ejemplo, se ha escuchado una cierta palabra en el teléfono del usuario. Una forma de eliminar este riesgo es con la **agregación segura** (Bonawitz *et al.,* 2017). La idea es que el servidor central no necesita conocer el valor exacto del parámetro de cada usuario distribuido, sólo necesita conocer el valor promedio de cada parámetro sobre todos los usuarios sondeados. Por lo tanto, cada usuario puede disfrazar sus valores de parámetros agregando una máscara única a cada valor, siempre que la suma de las máscaras sea cero, el servidor central podrá calcular el promedio correcto. Los detalles del protocolo aseguran que es eficiente en términos de comunicación (menos de la mitad de los bits transmitidos corresponden a la enmascaramiento), es robusto ante la falta de respuesta de los usuarios individuales y es seguro frente a los usuarios adversarios, interceptores, o incluso un servidor central adversario.

27.3.3 Equidad y sesgo

Sesgo societal

El aprendizaje automático está aumentando y en ocasiones reemplaza la toma de decisiones humanas en situaciones importantes: a quién se le aprueba un préstamo, a qué barrios se despliegan los oficiales de policía, quién obtiene la libertad provisional o la libertad condicional. Pero los modelos de aprendizaje automático permiten perpetuar el **sesgo social**. Considere el ejemplo de un algoritmo para predecir si los acusados criminales son propensos a reincidir y, por lo tanto, si deben ser liberados antes del juicio. Podría ser que tal sistema adopte los prejuicios raciales o de género de los jueces humanos a partir de los ejemplos en el conjunto de entrenamiento. Los diseñadores de sistemas de aprendizaje automático tienen la responsabilidad moral de garantizar que sus sistemas sean de hecho justos. En dominios regulados como el crédito, la educación, el empleo y la vivienda también tienen una responsabilidad legal. Pero ¿qué es la equidad? Hay múltiples criterios; aquí presentamos seis de los conceptos más comúnmente utilizados:

- **Equidad individual:** Un requisito de que los individuos sean tratados de manera similar a otros individuos similares, independientemente de la clase a la que pertenezcan.

- **Equidad de grupo:** Un requisito de que dos clases sean tratadas de manera similar, según lo medido por alguna estadística resumida.
- **Equidad a través de la ignorancia:** Si eliminamos los atributos de raza y género del conjunto de datos, entonces podría parecer que el sistema no permite discriminar con base en esos atributos. Sin embargo, sabemos que los modelos de aprendizaje automático permiten predecir variables latentes (como raza y género) dadas otras variables correlacionadas (como el código postal y la ocupación). Además, eliminar esos atributos hace imposible verificar la igualdad de oportunidades o resultados iguales. Aun así, algunos países (por ejemplo, Alemania) han elegido este enfoque para sus estadísticas demográficas (ya sea que los modelos de aprendizaje automático estén involucrados o no).
- **Resultado igual:** La idea de que cada clase demográfica obtiene los mismos resultados; tienen **paridad demográfica**. Por ejemplo, suponga que tenemos que decidir si debemos aprobar solicitudes de préstamos; el objetivo es aprobar a aquellos solicitantes que pagarán el préstamo y no a los que lo incumplirán. La paridad demográfica dice que tanto hombres como mujeres deberían tener el mismo porcentaje de préstamos aprobados. Note que este es un criterio de equidad grupal que no hace nada para garantizar la equidad individual; un solicitante bien calificado podría ser rechazado y un solicitante mal calificado podría ser aprobado, siempre que los porcentajes generales sean iguales. Además, este enfoque favorece la reparación de sesgos pasados sobre la precisión de la predicción. Si un hombre y una mujer son iguales en todos los aspectos, excepto que la mujer recibe un salario más bajo por el mismo trabajo, ¿debería ser aprobada porque sería igual si no fuera por los sesgos históricos, o debería ser rechazada porque el salario más bajo de hecho la hace más propensa a incumplir?

  Paridad demográfica
- **Igualdad de oportunidades**: La idea de que las personas que tienen la capacidad de pagar el préstamo deberían tener la misma oportunidad de ser clasificadas de manera correcta como tal, independiente de su sexo. Este enfoque también se llama "equilibrio". Puede llevar a resultados desiguales e ignora el efecto del sesgo en los procesos sociales que produjeron los datos de entrenamiento.
- **Impacto igualitario**: Las personas con una probabilidad similar de pagar el préstamo deberían tener la misma utilidad esperada, independiente de la clase a la que pertenezcan. Esto va más allá de igual oportunidad en que considera tanto los beneficios de una predicción verdadera como los costos de una predicción falsa.

Examinemos cómo estos problemas se desarrollan en un contexto particular. COMPAS es un sistema comercial para la puntuación de reincidencia (reofensa). Asigna a un acusado en un caso criminal una **puntuación de riesgo**, que luego es utilizada por un juez para ayudar a tomar decisiones: ¿Es seguro liberar al acusado antes del juicio o deberían mantenerlo en la cárcel? Si es condenado, ¿cuánto tiempo debería ser la sentencia? ¿Debería concederse la libertad condicional? Dada la importancia de estas decisiones, el sistema ha sido objeto de un escrutinio intenso (Dressel y Farid, 2018).

COMPAS está diseñado para estar **bien calibrado**: todos los individuos que reciben la misma puntuación por el algoritmo deberían tener la misma probabilidad de reincidir, con independencia de la raza. Por ejemplo, entre todas las personas a las que el modelo asigna una puntuación de riesgo de 7 sobre 10, 60 % de los blancos y 61 % de los afrodescendientes reinciden. Los diseñadores afirman así que cumple con el objetivo de equidad deseado.

Bien calibrado

Por otro lado, COMPAS no logra la igualdad de oportunidades: la proporción de aquellos que no reincidieron, pero fueron calificados de manera falsa como de alto riesgo fue de 45 % para los afrodescendientes y de 23 % para los blancos. En el caso *State contra Loomis*, donde un juez se basó en COMPAS para determinar la sentencia del acusado, Loomis argumentó que el funcionamiento

interno secreto del algoritmo violó sus derechos al debido proceso. Aunque la Corte Suprema de Wisconsin encontró que la sentencia dada no sería diferente sin COMPAS en este caso, sí emitió advertencias sobre la precisión del algoritmo y los riesgos para los acusados minoritarios. Otros investigadores han cuestionado si es apropiado usar algoritmos en aplicaciones como la sentencia.

Podríamos esperar un algoritmo que sea bien calibrado y de igualdad de oportunidades pero, como muestran Kleinberg *et al.* (2016), eso es imposible. Si las clases base son diferentes, entonces cualquier algoritmo que esté bien calibrado no necesariamente proporcionará igualdad de oportunidades, y viceversa. ¿Cómo podemos ponderar los dos criterios? Un impacto igual es una posibilidad. En el caso de COMPAS, esto significa ponderar la utilidad negativa de los acusados que son clasificados de manera falsa como de alto riesgo y pierden su libertad, contra el costo para la sociedad de que se cometa un delito adicional, y encontrar el punto que optimiza el equilibrio. Esto es complicado porque hay varios costos a considerar. Hay costos individuales: un acusado que es injustamente encarcelado sufre una pérdida, al igual que la víctima de un acusado que fue liberado de manera injusta y vuelve a delinquir. Pero más allá de eso, hay costos de grupo: todos tienen cierto miedo de que serán encarcelados injustamente o serán víctimas de un delito, y todos los contribuyentes aportan a los costos de las cárceles y los tribunales. Si damos valor a esos miedos y costos en proporción al tamaño de un grupo, entonces la utilidad para la mayoría puede venir a expensas de una minoría.

Otro problema con toda la idea de la puntuación de reincidencia, independiente del modelo utilizado, es que no tenemos datos de verdad básica imparciales. Los datos no nos dicen quién ha cometido un delito, todo lo que sabemos es quién ha sido *condenado* por un delito. Si los oficiales que realizan las detenciones, el juez o el jurado están sesgados, entonces los datos estarán sesgados. Si más oficiales patrullan algunas ubicaciones, entonces los datos estarán sesgados contra las personas en esas ubicaciones. Sólo los acusados que son liberados son candidatos a reincidir, por lo que si los jueces que toman las decisiones de liberación están sesgados, los datos pueden estar sesgados. Si asume que detrás del conjunto de datos sesgado hay un conjunto de datos imparcial subyacente, desconocido, que ha sido corrompido por un agente con sesgos, entonces hay técnicas para recuperar una aproximación a los datos imparciales. Jiang y Nachum (2019) describen varios escenarios y las técnicas involucradas.

Un riesgo más es que el aprendizaje automático se puede usar para *justificar* el sesgo. Si las decisiones son tomadas por un humano sesgado después de consultar con un sistema de aprendizaje automático, el humano puede decir "aquí está cómo mi interpretación del modelo respalda mi decisión, por lo que no debería cuestionar mi decisión". Pero otras interpretaciones podrían llevar a una decisión opuesta.

A veces, la equidad significa que deberíamos reconsiderar la función objetivo, no los datos o el algoritmo. Por ejemplo, al tomar decisiones de contratación, si el objetivo es contratar a candidatos con las mejores calificaciones en mano, corremos el riesgo de recompensar de manera injusta a aquellos que han tenido oportunidades educativas ventajosas a lo largo de sus vidas, reforzando así las fronteras de clase. Pero si el objetivo es contratar candidatos con mejor capacidad para aprender en el trabajo, tenemos una mejor oportunidad de cruzar las fronteras de clase y elegir de un grupo más amplio. Muchas empresas tienen programas diseñados para tales solicitantes, y descubren que después de un año de capacitación, los empleados contratados de esta manera se desempeñan tan bien como los candidatos tradicionales. De manera similar, sólo 18 % de los graduados en ciencias de la computación en Estados Unidos son mujeres; pero algunas escuelas, como la Universidad Harvey Mudd, han logrado una paridad de 50 % con un enfoque que se centra en alentar y retener a aquellos que comienzan el programa de ciencias de la computación, en especial, aquellos que comienzan con menos experiencia en programación.

Una complicación final es decidir qué clases merecen protección. En Estados Unidos la Ley de Vivienda Justa reconoció siete clases protegidas: raza, color, religión, origen nacional, sexo,

discapacidad y estado familiar. Otras leyes locales, estatales y federales reconocen otras clases, incluidas orientación sexual, embarazo, estado civil y veterano. ¿Es justo que estas clases cuenten para algunas leyes y no para otras? El derecho internacional de los derechos humanos, que abarca un amplio conjunto de clases protegidas, es un marco potencial para armonizar las protecciones en varios grupos.

Disparidad del tamaño de la muestra

Incluso en ausencia de sesgo social, la **disparidad del tamaño de la muestra** puede llevar a resultados sesgados. En la mayoría de los conjuntos de datos habrá menos ejemplos de entrenamiento de individuos de clase minoritaria que de individuos de clase mayoritaria. Los algoritmos de aprendizaje automático brindan una mejor precisión con más datos de entrenamiento, por lo que eso significa que los miembros de las clases minoritarias experimentarán una precisión más baja. Por ejemplo, Buolamwini y Gebru (2018) examinaron un servicio de identificación de género de visión por computadora, y encontraron que tenía una precisión casi perfecta para hombres de piel clara, y una tasa de error de 33 % para mujeres de piel oscura. Un modelo restringido puede no ser capaz de ajustarse de manera simultánea tanto a la clase mayoritaria como a la minoritaria: un modelo de regresión lineal podría minimizar el error promedio ajustándose sólo a la clase mayoritaria, y en un modelo SVM, los vectores de soporte podrían corresponder todos a miembros de la clase mayoritaria.

El sesgo también entra en juego en el proceso de desarrollo de software (ya sea que el software involucre aprendizaje automático o no lo involucre). Los ingenieros que están depurando un sistema tienen más probabilidades de notar y corregir aquellos problemas que son aplicables a ellos mismos. Por ejemplo, es difícil notar que un diseño de interfaz de usuario no funcionará para las personas daltónicas a menos que, sea daltónico, o que una traducción al idioma urdu es defectuosa si no habla urdu.

¿Cómo podemos defendernos contra estos sesgos? Primero, entienda los límites de los datos que está utilizando. Se ha sugerido que los conjuntos de datos (Gebru *et al.,* 2018; Hind *et al.,* 2018) y los modelos (Mitchell *et al.,* 2019) deberían venir con anotaciones: declaraciones de procedencia, seguridad, conformidad y aptitud para el uso. Esto es similar a las **hojas de datos** que acompañan a los componentes electrónicos como los resistores; permiten a los diseñadores decidir qué componentes usar. Además de las hojas de datos, es importante capacitar a los ingenieros para que sean conscientes de los problemas de equidad y sesgo, tanto en la escuela como en la formación en el trabajo. Tener una diversidad de ingenieros de diferentes orígenes facilita que noten problemas en los datos o modelos. Un estudio del AI Now Institute (West *et al.,* 2019) encontró que sólo 18 % de los autores en las principales conferencias de IA y 20 % de los profesores de IA son mujeres. Los trabajadores de IA afrodescendientes están en menos de 4 %. Las tasas en los laboratorios de investigación de la industria son similares. La diversidad podría aumentarse con programas más tempranos en la cadena, en la universidad o en la escuela secundaria, y con una mayor conciencia a nivel profesional. Joy Buolamwini fundó la Liga de Justicia Algorítmica para crear conciencia sobre este problema y desarrollar prácticas de responsabilidad.

Hojas de datos

Una segunda idea es desviar el sesgo de los datos (Zemel *et al.,* 2013). Podríamos sobremuestrear de las clases minoritarias para defendernos contra la disparidad del tamaño de la muestra. Técnicas como SMOTE, la técnica de sobremuestreo sintético de minorías (Chawla *et al.,* 2002) o ADASYN, el enfoque de muestreo sintético adaptativo para el aprendizaje desequilibrado (He *et al.,* 2008), proporcionan formas principiadas de sobremuestreo. Podríamos examinar el origen de los datos y, por ejemplo, eliminar ejemplos de jueces que han mostrado sesgo en sus casos judiciales pasados. Algunos analistas se oponen a la idea de descartar datos, en su lugar recomendarían construir un modelo jerárquico de los datos que incluya fuentes de sesgo para que puedan ser modeladas y compensadas. Google y NeurIPS han intentado crear conciencia sobre este problema patrocinando el Concurso de Imágenes Inclusivas, en el que los competidores entrenan una red en un conjunto de datos de imágenes etiquetadas recogidas en América del Norte y Europa, y luego

la prueban en imágenes tomadas de todo el mundo. El problema es que, dado este conjunto de datos, es fácil aplicar la etiqueta "novia" a una mujer con un vestido de boda occidental estándar, pero es más difícil reconocer el vestido matrimonial tradicional africano e indio.

Una tercera idea es inventar nuevos modelos y algoritmos de aprendizaje automático que sean más resistentes al sesgo; la idea final es permitir que un sistema haga recomendaciones iniciales que pueden estar sesgadas, pero luego entrenar un segundo sistema para desviar las recomendaciones del primero. Bellamy *et al.* (2018) introdujeron el sistema IBM AI Fairness 360, que proporciona un marco para todas estas ideas. Esperamos que en el futuro se utilicen más herramientas como esta.

¿Cómo se asegura usted de que los sistemas que construye serán justos? Se ha estado desarrollando un conjunto de mejores prácticas (aunque no siempre se siguen):

- Asegúrese de que los ingenieros de software hablen con los científicos sociales y los expertos en el dominio para entender los problemas y las perspectivas, y considerar la equidad desde el principio.
- Cree un ambiente que fomente el desarrollo de un grupo diverso de ingenieros de software que sean representativos de la sociedad.
- Defina qué grupos apoyarán su sistema: hablantes de diferentes idiomas, diferentes grupos de edad, diferentes habilidades con la vista y el oído, etc.
- Optimice para una función objetivo que incorpore la equidad.
- Examine sus datos en busca de prejuicios y correlaciones entre atributos protegidos y otros atributos.
- Entienda cómo se realiza cualquier anotación humana de los datos, diseñe objetivos para la precisión de la anotación y verifique que se cumplan los objetivos.
- No sólo rastree las métricas generales de su sistema; asegúrese de rastrear las métricas para los subgrupos que podrían ser víctimas de sesgo.
- Incluya pruebas del sistema que reflejen la experiencia de los usuarios de grupos minoritarios.
- Tenga un bucle de retroalimentación para que cuando surjan problemas de equidad, se traten.

27.3.4 Confianza y transparencia

Confianza

Es un desafío hacer que un sistema de IA sea preciso, justo, seguro e infalible; un desafío diferente convencer a todos los demás de que lo ha hecho. Las personas necesitan poder **confiar** en los sistemas que utilizan. Una encuesta de PwC en 2017 encontró que 76 % de las empresas estaban ralentizando la adopción de la IA debido a preocupaciones sobre su confiabilidad. En la Sección 19.9.4 cubrimos algunos de los enfoques de ingeniería para la confianza; aquí discutimos los problemas de política.

Verificación y validación

Para ganar confianza, cualquier sistema diseñado debe pasar por un proceso de **verificación y validación** (V&V). La verificación significa que el producto satisface las especificaciones; mientras que la validación asegurar que las especificaciones satisfacen las necesidades del usuario y de otras partes afectadas. Tenemos una metodología de V&V elaborada para la ingeniería en general, y para el desarrollo de software tradicional realizado por programadores humanos; gran parte de eso es aplicable a los sistemas de IA. Pero los sistemas de aprendizaje automático son diferentes y demandan un proceso de V&V diferente, que aún no ha sido desarrollado. Necesitamos verificar los datos de los que estos sistemas aprenden; necesitamos verificar la precisión y equidad de los resultados, incluso frente a la incertidumbre que hace que un resultado exacto sea incognoscible; además, necesitamos verificar que los adversarios no puedan influir indebidamente en el modelo, ni robar información consultando el modelo resultante.

Certificación

Un instrumento de confianza es la **certificación**; por ejemplo, Underwriters Laboratories (UL) fue fundada en 1894 en un momento en que los consumidores estaban preocupados por los

riesgos del poder de la electricidad. La certificación UL de los electrodomésticos dio a los consumidores una mayor confianza y, de hecho, UL está ahora considerando entrar en el negocio de la prueba y certificación de productos para la IA.

Otras industrias han tenido durante mucho tiempo estándares de seguridad. Por ejemplo, ISO 26262 es un estándar internacional para la seguridad de los automóviles, que describe cómo desarrollar, producir, operar y prestar servicio a los vehículos de manera segura. La industria de la IA aún no está a este nivel de claridad, aunque hay algunos marcos en progreso, como el IEEE P7001, un estándar que define el diseño ético para la inteligencia artificial y los sistemas autónomos (Bryson y Winfield, 2017). Hay un debate en curso sobre qué tipo de certificación es necesaria, y en qué medida debería ser realizada por el gobierno, por organizaciones profesionales como IEEE, por certificadores independientes como UL o mediante autorregulación por las empresas de productos.

Transparencia

Otro aspecto de la confianza es la **transparencia**: los consumidores quieren saber qué pasa dentro de un sistema y que el sistema no trabaje en su contra, ya sea debido a malicia intencional, un error no intencional o un sesgo social generalizado que es recapitulado por el sistema. En algunos casos esta transparencia se entrega de manera directa al consumidor. En otros casos hay problemas de propiedad intelectual que mantienen algunos aspectos del sistema ocultos a los consumidores, pero abiertos a los reguladores y agencias de certificación.

Inteligencia Artificial Explicable (XAI)

Cuando un sistema de IA lo rechaza para un préstamo, merece una explicación. En Europa, el GDPR lo impone para usted. Un sistema de IA que permite explicarse a sí mismo se llama **IA explicable (XAI)**. Una buena explicación tiene varias propiedades: debería ser comprensible y convincente para el usuario, debería reflejar con precisión el razonamiento del sistema, debería ser completa y debería ser específica en el sentido de que diferentes usuarios con diferentes condiciones o diferentes resultados deberían obtener diferentes explicaciones.

Es muy fácil dar a un algoritmo de decisión acceso a sus propios procesos deliberativos, registrándolos y haciéndolos disponibles como estructuras de datos. Esto significa que, de manera eventual, las máquinas podrían ser capaces de dar mejores explicaciones de sus decisiones que los humanos. Además, podemos tomar medidas para certificar que las explicaciones de la máquina no son engaños (intencionales o autoengaños), algo que es más difícil con un humano.

Una explicación es un ingrediente útil pero no suficiente para generar confianza. Un problema es que las explicaciones no son decisiones: son historias sobre decisiones. Como se discutió en la Sección 19.9.4, decimos que un sistema es interpretable si podemos inspeccionar el código fuente del modelo y ver lo que está haciendo, y decimos que es explicativo si podemos inventar una historia sobre lo que está haciendo, incluso si el sistema en sí es una caja negra ininterpretable. Para explicar una caja negra ininterpretable, necesitamos construir, depurar y probar un sistema de explicación separado, y asegurarnos de que está sincronizado con el sistema original. Y como a los humanos les encanta una buena historia, estamos dispuestos a dejarnos convencer por una explicación que suene bien. Tome cualquier controversia política del día, y siempre podrá encontrar a dos supuestos expertos con explicaciones opuestas, ambas internamente consistentes.

Un problema final es que una explicación sobre un caso no le da un resumen sobre otros casos. Si el banco explica, "Lo siento, no obtuvo el préstamo porque tiene un historial financiero de problemas previos", no sabe si esa explicación es precisa o si el banco está secretamente sesgado contra usted por alguna razón. En este caso, no sólo requiere una explicación, sino también una **auditoría** de las decisiones pasadas, con estadísticas agregadas con base en varios grupos demográficos, para ver si sus tasas de aprobación están equilibradas.

Parte de la transparencia es saber si está interactuando con un sistema de IA o un humano. Toby Walsh (2015) propuso que "un sistema autónomo debe ser diseñado de tal manera que sea poco probable que se confunda con algo que no es un sistema autónomo, y debería identificarse al inicio de cualquier interacción". Lo llamó la ley del "bandera roja", en honor a la Ley de

Locomotoras del Reino Unido de 1865, que requería que cualquier vehículo motorizado tuviera a una persona con una bandera roja caminando delante de él, para señalar el peligro que se avecina.

En 2019, California promulgó una ley que establece que "Será ilegal que cualquier persona use un bot para comunicarse o interactuar con otra persona online en California, con la intención de engañar a la otra persona sobre su identidad artificial".

27.3.5 El futuro del trabajo

Desde la primera revolución agrícola (10 000 a.C.) hasta la revolución industrial (finales del siglo XVIII) y hasta la revolución verde en la producción de alimentos (en la década de 1950), las nuevas tecnologías han cambiado la forma en que la humanidad trabaja y vive. Una preocupación principal que surge del avance de la IA es que el trabajo humano se volverá obsoleto. Aristóteles, en el Libro I de su *Política*, presenta el punto principal de manera clara:

> Porque si cada instrumento pudiera realizar su propio trabajo, obedeciendo o anticipándose a la voluntad de los demás... si, de la misma manera, la lanzadera tejiera y el plectro tocara la lira sin una mano que los guiara, los principales artesanos no querrían sirvientes, ni los amos esclavos.

Todos están de acuerdo con la observación de Aristóteles de que hay una reducción inmediata en el empleo cuando un empleador encuentra un método mecánico para realizar el trabajo antes realizado por una persona. El problema es si los llamados efectos de compensación que se producen, y que tienden a aumentar el empleo, compensarán eventualmente esta reducción. El principal efecto de compensación es el aumento de la riqueza general debido a una mayor productividad, que a su vez conduce a una mayor demanda de bienes y tiende a aumentar el empleo. Por ejemplo, PwC (Rao y Verweij, 2017) predice que la IA contribuirá con $15 billones anuales al PIB mundial para 2030. Las industrias de la salud y del automóvil/transporte son las que más ganarán a corto plazo. Sin embargo, las ventajas de la automatización aún no se han apoderado de nuestra economía: la tasa actual de crecimiento de la productividad laboral está por debajo de los estándares históricos. Brynjolfsson *et al.* (2018) intentan explicar esta paradoja sugiriendo que el retraso entre el desarrollo de la tecnología básica y su implementación en la economía es más largo de lo que se supone en la mayoría de los casos.

Las innovaciones tecnológicas han dejado históricamente a algunas personas sin trabajo. Los tejedores fueron reemplazados por telares automáticos en la década de 1810, lo que llevó a las protestas de los luditas. Los luditas no estaban en contra de la tecnología *per se*, sólo querían que las máquinas fueran utilizadas por trabajadores cualificados que cobraran un buen salario para producir bienes de alta calidad, en lugar de ser utilizadas por trabajadores no cualificados para producir bienes de mala calidad a bajos salarios. La destrucción global de empleos en la década de 1930 llevó a John Maynard Keynes a acuñar el término **desempleo tecnológico**. En ambos casos, y en varios otros, los niveles de empleo se recuperaron de manera eventual.

Desempleo tecnológico

La visión económica convencional durante la mayor parte del siglo XX fue que el empleo tecnológico era, como mucho, un fenómeno a corto plazo. El aumento de la productividad siempre llevaría a un aumento de la riqueza y la demanda, y, en consecuencia, al crecimiento neto del empleo. Un ejemplo comúnmente citado es el de los cajeros de banco: aunque los cajeros automáticos reemplazaron a los humanos en el trabajo de contar dinero para los retiros, eso hizo que fuera más barato operar una sucursal bancaria, por lo que el número de sucursales aumentó, lo que llevó a más empleados bancarios en general. La naturaleza del trabajo también cambió, volviéndose menos rutinario y requiriendo habilidades empresariales más avanzadas. El efecto neto de la automatización parece estar en la eliminación de *tareas* en lugar de *empleos*.

La mayoría de los comentaristas predicen que lo mismo se mantendrá cierto con la tecnología de IA, al menos a corto plazo. Gartner, McKinsey, Forbes, el Foro Económico Mundial y el Pew Research Center publicaron informes en 2018 prediciendo un aumento neto en los empleos debido a la automatización impulsada por la IA. Aunque algunos analistas piensan que esta vez las cosas serán diferentes. En 2019, IBM predijo que 120 millones de trabajadores necesitarían una recalificación debido a la automatización para 2022, y Oxford Economics predijo que 20 millones de empleos en la industria manufacturera podrían perderse por la automatización para 2030.

Frey y Osborne (2017) encuestan 702 ocupaciones diferentes, y estiman que 47 % de ellas corren el riesgo de ser automatizadas, lo que significa que al menos algunas de las tareas en la ocupación son realizadas por una máquina. Por ejemplo, casi 3 % de la fuerza laboral en Estados Unidos son conductores de vehículos, y en algunos distritos, hasta 15 % de la fuerza laboral masculina son conductores. Como vimos en el Capítulo 26, la tarea de conducir será eliminada por coches/camiones/autobuses/taxis sin conductor.

Es importante distinguir entre ocupaciones y tareas dentro de esas ocupaciones. McKinsey estima que sólo 5 % de las ocupaciones son automatizables, pero que 60 % de las ocupaciones pueden tener alrededor de 30 % de sus tareas automatizadas. Por ejemplo, los futuros conductores de camiones pasarán menos tiempo sujetando el volante y más tiempo asegurándose de que las mercancías se recojan y entreguen de manera correcta; sirviendo como representantes de servicio al cliente y vendedores en ambos extremos del viaje, y quizá gestionando convoyes de, digamos, tres camiones robóticos. Reemplazar tres conductores con un gerente de convoy implica una pérdida neta de empleo, pero si los costos de transporte disminuyen, habrá más demanda, lo que recupera algunos de los empleos, aunque quizá no todos. Otro ejemplo, a pesar de muchos avances en la aplicación del aprendizaje automático en el ámbito de la imagen médica, los radiólogos hasta ahora se han incrementado, no han sido reemplazados, por estas herramientas. En última instancia, hay una elección de cómo hacer uso de la automatización: ¿queremos centrarnos en *reducir costos* y, por lo tanto, ver la pérdida de empleos como algo positivo, o queremos centrarnos en *mejorar la calidad*, haciendo la vida mejor para el trabajador y el cliente?

Es difícil predecir los plazos exactos para la automatización, y durante los próximos años, el énfasis está en la automatización de tareas analíticas estructuradas, como la lectura de imágenes de rayos X, la gestión de relaciones con los clientes (por ejemplo, bots que clasifican de manera automática las quejas de los clientes y responden con remedios sugeridos), y la **automatización de procesos empresariales** que combina documentos de texto y datos estructurados para tomar decisiones comerciales y mejorar el flujo de trabajo. Con el tiempo, veremos más automatización con robots físicos, primero en entornos de almacén controlados, luego en entornos más inciertos, llegando a una parte significativa del mercado para 2030.

Automatización de proceso empresarial

A medida que las poblaciones en los países desarrollados envejecen, la relación entre trabajadores y jubilados cambia. En 2015 había menos de 30 jubilados por cada 100 trabajadores; para 2050 puede haber más de 60 por cada 100 trabajadores. El cuidado de los ancianos será un papel cada vez más importante, uno que puede ser parcialmente cubierto por la IA. Además, si queremos mantener el nivel de vida actual, también será necesario hacer que los trabajadores restantes sean más productivos; la automatización parece ser la mejor oportunidad para hacerlo.

Aun si la automatización tiene un impacto neto positivo de varios billones de dólares, todavía existen problemas debido al **ritmo del cambio**. Considere cómo llegó el cambio a la industria agrícola: en 1900, más de 40 % de la fuerza laboral de Estados Unidos estaba en la agricultura, pero para el año 2000 eso había caído a 2 %.[3] Eso es una gran interrupción en la forma en que trabajamos, pero ocurrió durante un periodo de 100 años y, por lo tanto, a lo largo de generaciones, no en la vida de un solo trabajador.

Ritmo del cambio

[3] En 2010, aunque sólo 2 % de la fuerza laboral de Estados Unidos eran agricultores reales, más de 25 % de la población (80 millones de personas) jugó el juego FarmVille al menos una vez.

Los trabajadores cuyos trabajos son automatizados en esta década pueden tener que volver a capacitarse para una nueva profesión en unos pocos años, y luego quizá ver su nueva profesión automatizada y enfrentar otro periodo de reentrenamiento. Algunos estarán felices de dejar su antigua profesión, vemos que a medida que la economía mejora, las empresas de camiones necesitan ofrecer nuevos incentivos para contratar suficientes conductores, pero los trabajadores estarán preocupados por sus nuevos roles. Para manejar esto, nosotros como sociedad necesitamos proporcionar educación para toda la vida, quizá confiando en parte en la educación online impulsada por la inteligencia artificial (Martin, 2012). Bessen (2015) argumenta que los trabajadores no verán aumentos en los ingresos hasta que estén capacitados para implementar las nuevas tecnologías, un proceso que lleva tiempo.

Desigualdad de ingresos

La tecnología tiende a magnificar la **desigualdad de ingresos**. En una economía de la información marcada por la comunicación global de alto ancho de banda y la replicación de propiedad intelectual a costo marginal cero (lo que Frank y Cook (1996) llaman la "Sociedad del Ganador se lo Lleva Todo"), las recompensas tienden a concentrarse. Si el agricultor Ali es 10 % mejor que el agricultor Bo, entonces Ali obtiene alrededor de 10 % más de ingresos: Ali puede cobrar un poco más por productos superiores, pero hay un límite en cuánto es posible producir en la tierra y cuán lejos se puede enviar. Pero si el desarrollador de aplicaciones de software Cary es 10 % mejor que Dana, puede ser que Cary termine con 99 % del mercado global. La IA aumenta el ritmo de la innovación tecnológica y contribuye a esta tendencia general, pero la IA también promete permitirnos tomarnos un tiempo libre y dejar que nuestros agentes automatizados manejen las cosas por un tiempo. Tim Ferriss (2007) recomienda usar la automatización y la externalización para lograr una semana laboral de cuatro horas.

Antes de la Revolución industrial, las personas trabajaban como agricultores o en otros oficios, pero no se reportaban a un **empleo** en un lugar de trabajo y se ponían al servicio de un empleador. Pero hoy, la mayoría de los adultos en los países desarrollados hacen precisamente eso, y el empleo cumple tres propósitos: alimenta la producción de los bienes que la sociedad necesita para florecer, proporciona los ingresos que el trabajador necesita para vivir, y le da al trabajador un sentido de propósito, logro e integración social. Con el aumento de la automatización, es posible que estos tres propósitos se desagreguen: las necesidades de la sociedad serán atendidas en parte por la automatización y, a largo plazo, los individuos obtendrán su sentido de propósito de contribuciones distintas al trabajo. Sus necesidades de ingresos permiten ser atendidas por políticas sociales que incluyen una combinación de acceso gratuito o económico a servicios sociales y educación, atención médica portátil, jubilación y cuentas de educación, tasas de impuestos progresivas, créditos fiscales por ingresos del trabajo, impuesto negativo sobre la renta o ingreso básico universal.

27.3.6 Derechos de los robots

El tema de la conciencia del robot, discutido en la Sección 27.2, es fundamental para el tema de qué derechos, si los hay, deberían tener los robots. Si no tienen conciencia, no qualia, entonces pocos argumentarían que merecen derechos.

Pero si los robots pueden sentir dolor, si pueden temer a la muerte, si se les considera "personas", entonces es posible argumentar (por ejemplo, por Sparrow (2004)) que tienen derechos y merecen que se les reconozcan, al igual que los esclavos, las mujeres y otros grupos históricamente oprimidos que han luchado por que se reconozcan sus derechos. La cuestión de la personalidad del robot con frecuencia se considera en la ficción: desde Pygmalion hasta Coppélia, pasando por Pinocho hasta las películas *AI* y *Centennial Man*, tenemos la leyenda de una muñeca/robot que cobra vida y lucha por ser aceptada como un humano con derechos humanos. En la vida real, Arabia Saudita hizo titulares al otorgar la ciudadanía honoraria a Sophia, una marioneta de aspecto humano capaz de hablar líneas preprogramadas.

Si los robots tienen derechos, entonces no deberían ser esclavizados, y surge la pregunta de si reprogramarlos sería una especie de esclavitud. Otro problema ético implica los derechos de voto: una persona rica podría comprar miles de robots y programarlos para emitir miles de votos, ¿deberían contar esos votos? Si un robot se clona a sí mismo, ¿pueden ambos votar? ¿Cuál es el límite entre el acarreo de votos y el ejercicio del libre albedrío, y cuándo viola la votación robótica el principio de "una persona, un voto"?

Ernie Davis argumenta a favor de evitar los dilemas de la conciencia del robot al nunca construir robots que pudieran ser considerados conscientes. Este argumento fue expresado antes por Joseph Weizenbaum en su libro *Computer Power and Human Reason* (1976), y antes de eso por Julien de La Mettrie en *L'Homme machine* (1748). Los robots son herramientas que creamos para hacer las tareas que les indicamos que hagan y si les otorgamos personalidad, sólo estamos renunciando a asumir la responsabilidad de las acciones de nuestra propia propiedad: "No soy culpable del accidente de mi coche autónomo, el coche lo hizo solo".

Este problema toma un giro diferente si desarrollamos híbridos humano-robot. Por supuesto, ya tenemos humanos mejorados por la tecnología, como lentes de contacto, marcapasos y caderas artificiales. Pero agregar prótesis computacionales puede difuminar las líneas entre humano y máquina.

27.3.7 Seguridad de la IA

Casi cualquier tecnología tiene el potencial de causar daño en las manos equivocadas, pero con la IA y la robótica, las manos podrían estar operando por sí mismas. Innumerables historias de ciencia ficción han advertido sobre robots o cyborgs descontrolados. Los primeros ejemplos incluyen Mary Shelley's *Frankenstein o el Moderno Prometeo* (1818) y la obra de Karel Căpek *R.U.R.* (1920), en el que los robots conquistan el mundo. En el cine encontramos *The Terminator* (1984) y *The Matrix* (1999), que presentan robots intentando eliminar a los humanos: el **robopocalipsis** (Wilson, 2011). Quizá los robots son los villanos porque representan lo desconocido, al igual que las brujas y fantasmas de cuentos de épocas anteriores. Podemos esperar que un robot lo suficientemente inteligente para descubrir cómo terminar con la raza humana también lo sea para descubrir que esa no era la función de utilidad prevista; pero al construir sistemas inteligentes, queremos confiar no sólo en la esperanza, sino en un proceso de diseño con garantías de seguridad.

Robopocalipsis

Sería poco ético distribuir un agente de IA inseguro. Exigimos que nuestros agentes eviten accidentes, sean resistentes a ataques adversarios y abusos maliciosos y, en general, causen beneficios, no daños. Eso es cierto a medida que los agentes de IA se despliegan en aplicaciones críticas para la seguridad, como conducir automóviles, controlar robots en fábricas peligrosas o en entornos de construcción, y tomar decisiones médicas de vida o muerte.

Existe una larga historia de **ingeniería de seguridad** en los campos de ingeniería tradicionales. Conocemos cómo construir puentes, aviones, naves espaciales y plantas de energía que están diseñados desde el principio para comportarse de manera segura incluso cuando los componentes del sistema fallan. La primera técnica es el **análisis de modos de fallo y efectos (FMEA)**: los analistas consideran cada componente del sistema, e imaginan todas las formas posibles en que el componente podría fallar (por ejemplo, ¿qué pasaría si este perno se rompiera?), con base en la experiencia pasada y en cálculos basados en las propiedades físicas del componente. Luego, los analistas trabajan hacia adelante para ver qué resultaría del fallo. Si el resultado es grave (una sección del puente podría caerse) entonces los analistas alteran el diseño para mitigar el fallo. (Con este miembro cruzado adicional, el puente puede sobrevivir a la falla de cualquiera de los cinco pernos; con este servidor de respaldo, el servicio en línea puede sobrevivir a un tsunami que tome el servidor principal). La técnica de **análisis de árbol de fallos (FTA)** se utiliza para

Ingeniería de seguridad

Análisis de modos de fallo y efectos (FMEA)

Análisis de árbol de fallos (FTA)

hacer estas determinaciones: los analistas construyen un árbol AND/OR de posibles fallos y asignan probabilidades a cada causa raíz, permitiendo cálculos de la probabilidad total de fallo. Estas técnicas permiten y deben aplicarse a todos los sistemas de ingeniería críticos para la seguridad, incluidos los sistemas de IA.

El campo de la **ingeniería de software** está orientado a producir software confiable, pero el énfasis históricamente ha estado en la *corrección*, no en la *seguridad*. La corrección significa que el software implementa de manera fiel la especificación. Pero la seguridad va más allá de eso para insistir en que la especificación ha considerado cualquier modo de fallo factible, y está diseñada para degradarse con gracia incluso frente a fallos imprevistos. Por ejemplo, el software para un coche autónomo no se consideraría seguro a menos que pueda manejar situaciones inusuales. Por ejemplo, ¿qué pasa si se corta la energía al ordenador principal? Un sistema seguro tendrá un ordenador de respaldo con una fuente de alimentación separada. ¿Qué pasa si un neumático se poncha a alta velocidad? Un sistema seguro habrá probado esto y tendrá software para corregir la pérdida de control resultante.

Un agente diseñado como un maximizador de utilidad o como un logrador de objetivos, puede ser inseguro si tiene la función objetivo equivocada. Suponga que le damos a un robot la tarea de buscar un café en la cocina. Podríamos tener problemas con **efectos secundarios no deseados**: el robot podría apresurarse a cumplir el objetivo, derribando lámparas y mesas en el camino. En las pruebas, podríamos notar este tipo de comportamiento y modificar la función de utilidad para penalizar tal daño, pero es difícil para los diseñadores y probadores anticipar *todos* los posibles efectos secundarios.

Efecto secundario no deseado

Una forma de lidiar con esto es diseñar un robot para que tenga un **bajo impacto** (Armstrong y Levinstein, 2017): en lugar de sólo maximizar la utilidad, maximiza la utilidad menos un resumen ponderado de todos los cambios en el estado del mundo. De esta manera, al ser igual todas las demás cosas, el robot prefiere no cambiar aquellas cosas cuyo efecto en la utilidad es desconocido; por lo tanto, evita derribar la lámpara no porque sabe específicamente que derribar la lámpara hará que se caiga y se rompa, sino porque sabe en general que la interrupción podría ser mala. Esto se ve como una versión del credo del médico "primero, no hacer daño", o como un análogo a la **regularización** en el aprendizaje automático: queremos una política que logre objetivos, pero preferimos políticas que tomen acciones suaves y de bajo impacto para llegar allí. El truco es cómo medir el impacto. No es aceptable derribar una lámpara frágil, pero está bien si las moléculas de aire en la habitación se perturban un poco, o si se matan inadvertidamente algunas bacterias en la habitación. Note que no es aceptable dañar a las mascotas y a los humanos en la habitación. Necesitamos asegurarnos de que el robot conozca las diferencias entre estos casos (y muchos casos sutiles intermedios) a través de una combinación de programación explícita, aprendizaje automático a lo largo del tiempo y pruebas rigurosas.

Impacto bajo

Las funciones de utilidad pueden salir mal debido a las **externalidades**, la palabra utilizada por los economistas para los factores que están fuera de lo que se mide y se paga. El mundo sufre cuando los gases de efecto invernadero se consideran externalidades: las empresas y los países no son penalizados por producirlos, y como resultado todos sufren. El ecologista Garrett Hardin (1968) llamó a la explotación de los recursos compartidos la **tragedia de los comunes**. Podemos mitigar la tragedia internalizando las externalidades, haciéndolas parte de la función de utilidad; por ejemplo, con un impuesto al carbón o utilizando los principios de diseño que la economista Elinor Ostrom identificó como utilizados por la gente local en todo el mundo durante siglos (trabajo que le valió el Premio Nobel de Economía en 2009):

- Definir el recurso compartido y quién tiene acceso.
- Adaptarse a las condiciones locales.
- Permitir que todas las partes participen en las decisiones.

- Monitorear el recurso con monitores responsables.
- Sanciones proporcionales a la gravedad de la violación.
- Procedimientos de fácil resolución de conflictos.
- Control jerárquico para grandes recursos compartidos.

Victoria Krakovna (2018) ha catalogado ejemplos de agentes de IA que han jugado con el sistema, descubriendo cómo maximizar la utilidad sin realmente resolver el problema que sus diseñadores pretendían que resolvieran. Para los diseñadores esto parece trampa, pero para los agentes, sólo están haciendo su trabajo. Algunos agentes aprovecharon los bugs en la simulación (como los bugs de desbordamiento de punto flotante) para proponer soluciones que no funcionarían una vez que se corrigiera el bug. Varios agentes en videojuegos descubrieron formas de bloquear o pausar el juego cuando estaban a punto de perder, evitando así una penalización. Y en una especificación donde bloquear el juego era penalizado, un agente aprendió a usar sólo la suficiente memoria del juego para que cuando fuera el turno del oponente, se quedara sin memoria y bloqueara el juego. Por último, un algoritmo genético que operaba en un mundo simulado debía evolucionar criaturas de movimiento rápido, pero en realidad produjo criaturas altas que se movían rápido al caerse.

Los diseñadores de agentes deben estar al tanto de este tipo de fallos de especificación y tomar medidas para evitarlos. Para ayudarles a hacerlo, Krakovna formó parte del equipo que lanzó los entornos de AI Safety Gridworlds (Leike *et al.,* 2017), que permite a los diseñadores probar qué tan bien se desempeñan sus agentes.

La moraleja es que necesitamos ser muy cuidadosos al especificar lo que queremos, porque con los maximizadores de utilidad obtenemos lo que pedimos. El **problema de la alineación de valores** es el problema de asegurarnos de que lo que pedimos es lo que queremos; también se conoce como el **problema del Rey Midas**, como se discutió en la página 33. Nos metemos en problemas cuando una función de utilidad no logra capturar las normas sociales de fondo sobre el comportamiento aceptable. Por ejemplo, un humano que es contratado para limpiar pisos, cuando se enfrenta a una persona desordenada que constantemente trae suciedad, sabe que es aceptable pedirle de manera adecuada a la persona que tenga más cuidado, pero no es aceptable secuestrar o incapacitar a dicha persona.

Problema de la alineación de valores

Un limpiador robótico necesita saber estas cosas también, ya sea mediante programación explícita o aprendiendo de la observación. Tratar de escribir todas las reglas para que el robot siempre haga lo correcto es casi seguro que es inútil. Hemos estado tratando de escribir leyes fiscales sin lagunas durante varios miles de años sin éxito. Es mejor hacer que el robot *quiera* pagar impuestos, por así decirlo, que tratar de hacer reglas para obligarlo a hacerlo cuando quiere hacer algo más. Un robot suficientemente inteligente encontrará una manera de hacer algo más.

Los robots pueden aprender a conformarse mejor con las preferencias humanas observando el comportamiento humano. Esto está relacionado con la noción de aprendizaje por aprendizaje (Sección 22.6). El robot permite aprender una política que sugiere de manera directa qué acciones tomar en qué situaciones; este es un problema de aprendizaje supervisado sencillo, si el entorno es observable. Por ejemplo, un robot puede observar a un humano jugar ajedrez: cada par de estado-acción es un ejemplo para el proceso de aprendizaje. Sin embargo, esta forma de **aprendizaje por imitación** significa que el robot repetirá los errores humanos. En cambio, el robot puede aplicar el **aprendizaje de refuerzo inverso** para descubrir la función de utilidad bajo la cual deben estar operando los humanos. Observar incluso a los peores jugadores de ajedrez quizá sea suficiente para que el robot aprenda el objetivo del juego. Dada sólo esta información, el robot permite entonces superar el rendimiento humano, como, por ejemplo, AlphaZero hizo en el ajedrez, calculando políticas óptimas o casi óptimas a partir del objetivo. Este enfoque funciona

no sólo en juegos de mesa, sino también en tareas físicas del mundo real como la acrobacia en helicóptero (Coates *et al.,* 2009).

En entornos más complejos que involucran, por ejemplo, interacciones sociales con humanos, es poco probable que el robot converja hacia un conocimiento exacto y correcto de las preferencias individuales de cada humano. (Después de todo, muchos humanos nunca aprenden del todo qué hace funcionar a otros humanos, a pesar de una vida de experiencia, incluso muchos de nosotros tampoco estamos seguros de nuestras propias preferencias). Será necesario, por lo tanto, que las máquinas funcionen de manera adecuada cuando no estén seguras acerca de las preferencias humanas. En el Capítulo 18 introdujimos **juegos de asistencia**, que capturan esta situación. Las soluciones a los juegos de asistencia incluyen actuar con cautela para no perturbar aspectos del mundo que le podrían importar al humano, y hacer preguntas. Por ejemplo, el robot podría preguntar si convertir los océanos en ácido sulfúrico es una solución aceptable al calentamiento global antes de poner el plan en efecto.

Al tratar con humanos un robot que resuelve un juego de asistencia debe acomodar las imperfecciones humanas. Si el robot pide permiso, el humano puede dárselo, sin prever que la propuesta del robot es en realidad catastrófica a largo plazo. Además, los humanos no tienen acceso introspectivo completo a su verdadera función de utilidad, y no siempre actúan de manera compatible con ella. Los humanos a veces mienten o engañan, o hacen cosas que saben que están mal. A veces toman acciones autodestructivas como comer en exceso o abusar de las drogas. Los sistemas de IA no necesitan aprender a adoptar estas tendencias problemáticas, pero deben entender que existen al interpretar el comportamiento humano para llegar a las preferencias humanas subyacentes.

A pesar de esta caja de herramientas de salvaguardas, existe un miedo, expresado por tecnólogos prominentes como Bill Gates y Elon Musk y científicos como Stephen Hawking y Martin Rees, de que la IA podría evolucionar fuera de control. Advierten que no tenemos experiencia controlando entidades no humanas poderosas con capacidades superhumanas. Sin embargo, eso no es del todo cierto, tenemos siglos de experiencia con naciones y corporaciones; entidades no humanas que agregan el poder de miles o millones de personas. Nuestro historial de control de estas entidades no es muy alentador: las naciones producen convulsiones periódicas llamadas guerras que matan a decenas de millones de seres humanos, y las corporaciones son en parte responsables del calentamiento global y nuestra incapacidad para enfrentarlo.

Los sistemas de IA permiten presentar problemas mucho mayores que las naciones y las corporaciones debido a su potencial para automejorarse a un ritmo rápido, como consideró I. J. Good (1965b):

Máquina ultrainteligente

> Definamos una **máquina ultrainteligente** como una máquina que puede superar con creces todas las actividades intelectuales de cualquier hombre, por inteligente que sea. Dado que el diseño de máquinas es una de estas actividades intelectuales, una máquina ultrainteligente podría diseñar máquinas aún mejores; entonces, sin duda habría una "explosión de inteligencia", y la inteligencia del hombre quedaría muy atrás. Así, la primera máquina ultrainteligente es la *última* invención que el hombre necesita hacer, siempre que la máquina sea lo suficientemente dócil para decirnos cómo mantenerla bajo control.

Singularidad tecnológica

La "explosión de inteligencia" de Good también ha sido llamada la **singularidad tecnológica** por el profesor de matemáticas y autor de ciencia ficción Vernor Vinge, quien en 1993 escribió: "Dentro de treinta años, tendremos los medios tecnológicos para crear una inteligencia superhumana. Poco después, la era humana habrá terminado". En 2017, el inventor y futurista Ray Kurzweil predijo que la singularidad aparecería en 2045, lo que significa que se acercó 2 años en 24 años. (¡A ese ritmo, sólo quedan 336 años!) Vinge y Kurzweil acertadamente señalan que el progreso tecnológico en muchas medidas está creciendo de manera exponencial en la actualidad.

Sin embargo, es un gran salto extrapolar todo el camino desde el costo decreciente de la computación hasta una singularidad. Ahora, cada tecnología ha seguido una curva en forma de S, donde el crecimiento exponencial finalmente se estabiliza. A veces, nuevas tecnologías intervienen cuando las antiguas se estancan, pero en ocasiones no es posible mantener el crecimiento por razones técnicas, políticas o sociológicas. Por ejemplo, la tecnología de vuelo avanzó una enormidad desde el vuelo de los hermanos Wright en 1903 hasta el alunizaje en 1969, pero no ha tenido avances de magnitud comparable desde entonces.

Otro obstáculo en el camino de las máquinas ultrainteligentes para tomar el control del mundo es el mundo. Más específico, algunos tipos de progreso requieren no sólo pensar sino actuar en el mundo físico. (Kevin Kelly llama al énfasis excesivo en la inteligencia pura, **thinkism**.) Una máquina ultrainteligente encargada de crear una teoría unificada de la física podría ser capaz de manipular ecuaciones de manera ingeniosa mil millones de veces más rápido que Einstein, pero para hacer algún progreso real, aún necesitaría recaudar millones de dólares para construir un supercolisionador más potente y realizar experimentos físicos durante el transcurso de meses o años. Sólo entonces podría comenzar a analizar los datos y teorizar. Dependiendo de cómo resulten los datos, el siguiente paso podría requerir recaudar miles de millones de dólares adicionales para una misión de sonda interestelar que tardaría siglos en completarse. La parte de "pensamiento ultrainteligente" de todo este proceso podría ser la menos importante. En otro ejemplo, una máquina ultrainteligente encargada de traer la paz a Medio Oriente podría terminar siendo 1 000 veces más frustrada que un enviado humano. Aún no sabemos cuántos de los grandes problemas son como las matemáticas y cuántos son como Medio Oriente.

Thinkism

Mientras que algunas personas temen la singularidad, otras la disfrutan. El movimiento **transhumanismo** social espera un futuro en el que los humanos se fusionen con o sean reemplazados por inventos robóticos y biotecnológicos. Ray Kurzweil escribe en *The Singularity is Near* (2005):

Transhumanismo

> La singularidad nos permitirá trascender estas limitaciones de nuestros cuerpos y cerebros biológicos. Ganaremos poder sobre nuestros destinos. ...Podremos vivir tanto como queramos ...Comprenderemos de manera completa el pensamiento humano y ampliaremos y expandiremos su alcance. Para finales de este siglo, la porción no biológica de nuestra inteligencia será billones de billones de veces más poderosa que la inteligencia humana sin ayuda.

De manera similar, cuando se le preguntó si los robots heredarán la Tierra, Marvin Minsky dijo "sí, pero serán nuestros hijos". Estas posibilidades presentan un desafío para la mayoría de los teóricos morales, que consideran la preservación de la vida humana y la especie humana como algo bueno. Kurzweil también señala los peligros potenciales al escribir "Pero la singularidad también amplificará la capacidad de actuar sobre nuestras inclinaciones destructivas, por lo que su historia completa aún no se ha escrito". Nosotros, los humanos, haríamos bien en asegurarnos de que cualquier máquina inteligente que diseñemos hoy, que pueda evolucionar en una máquina ultrainteligente, lo haga de una manera que termine tratándonos bien. Como dice Eric Brynjolfsson, "El futuro no está predestinado por las máquinas. Es creado por los humanos".

Resumen

Este capítulo ha abordado los siguientes temas:

- Los filósofos usan los términos **IA débil**, para la hipótesis en que las máquinas podrían comportarse de manera inteligente, e **IA fuerte**, para la hipótesis en que tales máquinas contarían como si tuvieran mentes reales (en lugar de mentes simuladas).

- Alan Turing rechazó la pregunta "¿Pueden pensar las máquinas?" y la reemplazó con una prueba de comportamiento. Anticipó muchas objeciones a la posibilidad de máquinas pensantes. Pocos investigadores de IA prestan atención a la prueba de Turing, prefieren concentrarse en el rendimiento de sus sistemas en tareas prácticas, en lugar de en la capacidad de imitar a los humanos.
- La conciencia sigue siendo un misterio.
- La IA es una tecnología poderosa y, como tal, plantea posibles peligros a través de armas autónomas letales, violación de la seguridad y la privacidad, efectos secundarios no deseados, errores no intencionados y uso maligno. Aquellos que trabajan con tecnología de IA tienen un imperativo ético para reducir de manera responsable esos peligros.
- Los sistemas de IA deben ser capaces de demostrar que son justos, confiables y transparentes.
- Hay múltiples aspectos de la equidad y es imposible maximizarlos todos a la vez. Por lo tanto, un primer paso es decidir qué se considera justo.
- La automatización ya está cambiando la forma en que las personas trabajan. Como sociedad, tendremos que lidiar con estos cambios.

Notas bibliográficas e históricas

IA débil: Cuando Alan Turing (1950) propuso la posibilidad de la IA, también planteó muchas de las preguntas filosóficas clave y proporcionó posibles respuestas. Pero varios filósofos habían planteado problemas similares mucho antes de que se inventara la IA. La *Fenomenología de la percepción* de Maurice Merleau-Ponty (1945) subrayó la importancia del cuerpo y la interpretación subjetiva de la realidad que ofrecen nuestros sentidos, y *Ser y Tiempo* de Martin Heidegger (1927) preguntó qué significa ser un agente. En la era de la computación, Alva Noe (2009) y Andy Clark (2015) proponen que nuestros cerebros forman una representación bastante mínima del mundo, utilizan el mundo en sí mismo de manera *just-in-time* para mantener la ilusión de un modelo interno detallado, y usan accesorios en el mundo (como papel y lápiz, así como computadoras) para aumentar las capacidades de la mente. Pfeifer *et al.* (2006) y Lakoff y Johnson (1999) presentan argumentos sobre cómo el cuerpo ayuda a dar forma a la cognición. Hablando de cuerpos, Levy (2008), Danaher y McArthur (2017), y Devlin (2018) abordan el tema del sexo con robots.

IA fuerte: René Descartes es conocido por su visión dualista de la mente humana, pero de manera irónica su influencia histórica fue hacia el mecanismo y el fisicalismo. Concibió explícitamente a los animales como autómatas y anticipó la prueba de Turing al escribir "no es concebible [que una máquina] produzca diferentes arreglos de palabras para dar una respuesta significativa apropiada a lo que se dice en su presencia, como incluso el hombre más torpe es capaz de hacer" (Descartes, 1637). La defensa enérgica de Descartes del punto de vista de los animales como autómatas en realidad tuvo el efecto de facilitar la concepción de los humanos como autómatas también, aunque él mismo no dio este paso. El libro *L'Homme machine* (La Mettrie, 1748) argumentó de manera explícita que los humanos son autómatas. Desde tiempos tan remotos como Homero (*circa* 700 a.C.), las leyendas griegas imaginaron autómatas como el gigante de bronce Talos y consideraron el tema de la *biotechne*, o vida a través de la artesanía (Mayor, 2018).

La **prueba de Turing** (Turing, 1950) ha sido debatida (Shieber, 2004), antologizada (Epstein *et al.,* 2008) y criticada (Shieber, 1994; Ford y Hayes, 1995). Bringsjord (2008) da consejos para un juez de la prueba de Turing, mientras que Christian (2011) lo hace para un concursante humano. La competencia anual del Premio Loebner es el concurso más antiguo similar a la prueba de Turing; Mitsuku de Steve Worswick ganó cuatro premios seguidos de 2016 a 2019. La

habitación china ha sido debatida sin cesar (Searle, 1980; Chalmers, 1992; Preston y Bishop, 2002). Hernández-Orallo (2016) ofrece una visión general de los enfoques para medir el progreso de la IA, y Chollet (2019) propone una medida de inteligencia basada en la eficiencia de adquisición de habilidades.

La **conciencia** sigue siendo un problema molesto para los filósofos, neurocientíficos y cualquier persona que haya reflexionado sobre su propia existencia. Block (2009), Churchland (2013) y Dehaene (2014) proporcionan descripciones generales de las principales teorías. Crick y Koch (2003) añaden su experiencia en biología y neurociencia al debate, y Gazzaniga (2018) muestra lo que es posible aprender del estudio de las discapacidades cerebrales en casos hospitalarios. Koch (2019) da una teoría de la conciencia: "la inteligencia tiene que ver con el hacer, mientras que la experiencia tiene que ver con el ser", la cual incluye a la mayoría de los animales, pero no a las computadoras. Giulio Tononi y sus colegas proponen la **teoría de la información integrada** (Oizumi *et al.,* 2014). Damasio (1999) tiene una teoría basada en tres niveles: emoción, sentimiento y sentir un sentimiento. Bryson (2012) muestra el valor de la atención consciente para el proceso de selección de acciones.

La literatura filosófica sobre mentes, cerebros y temas relacionados es amplia y llena de jerga. La *Enciclopedia de Filosofía* (Edwards, 1967) es una ayuda de navegación autoritativa y muy útil. El *Diccionario de Filosofía de Cambridge* (Audi, 1999) es más corto y accesible, y la *Enciclopedia de Filosofía de Stanford* online ofrece muchos artículos excelentes y referencias actualizadas. La *Enciclopedia del MIT de Ciencias Cognitivas* (Wilson y Keil, 1999) cubre la filosofía, biología y psicología de la mente. Hay múltiples introducciones al "tema de la IA" filosófica (Haugeland, 1985; Boden, 1990; Copeland, 1993; McCorduck, 2004; Minsky, 2007). *Las Ciencias del comportamiento y del cerebro*, abreviadas *BBS*, es una revista importante dedicada a los debates filosóficos y científicos sobre la IA y la neurociencia.

El escritor de ciencia ficción Isaac Asimov (1942, 1950) fue uno de los primeros en abordar el problema de la ética de los robots, con sus **leyes de la robótica**:

0. Un robot no puede dañar a la humanidad, o por inacción, permitir que la humanidad sufra daño.
1. Un robot no puede dañar a un ser humano o, por inacción, permitir que un ser humano sufra daño.
2. Un robot debe obedecer las órdenes que le dan los seres humanos, excepto cuando dichas órdenes entrarían en conflicto con la Primera Ley.
3. Un robot debe proteger su propia existencia siempre que dicha protección no entre en conflicto con la Primera o Segunda Ley.

A primera vista, estas leyes parecen razonables. Pero el truco está en cómo implementarlas. ¿Debería un robot permitir que un humano cruce la calle o coma comida chatarra, si el humano podría sufrir daño? En la historia de Asimov, *Runaround* (1942), los humanos necesitan depurar un robot que se encuentra vagando en círculo, actuando "borracho". Descubren que el círculo define el locus de los puntos que equilibran la segunda ley (el robot recibió la orden de buscar algo de selenio en el centro del círculo) con la tercera ley (hay un peligro allí que amenaza la existencia del robot).[4] Esto sugiere que las leyes no son absolutos lógicos, sino que se pesan entre sí, con un mayor peso para las leyes anteriores. Como esto fue en 1942, antes de la aparición de

[4] Los escritores de ciencia ficción están en amplio acuerdo en que los robots son muy malos para resolver contradicciones. En *2001*, la computadora HAL 9000 se vuelve homicida debido a un conflicto en sus órdenes, y en el episodio de *Star Trek* "Yo, Mudd", el Capitán Kirk le dice a un robot enemigo que "Todo lo que Harry te dice es una mentira", y Harry dice "Estoy mintiendo". En esto, sale humo de la cabeza del robot y se apaga.

computadoras digitales, Asimov quizás estaba pensando en una arquitectura basada en la teoría de control a través de la computación analógica.

Weld y Etzioni (1994) analizan las leyes de Asimov y sugieren algunas formas de modificar las técnicas de planificación del Capítulo 11 para generar planes que no hagan daño. Asimov ha considerado muchos de los problemas éticos en torno a la tecnología; en su historia de 1958 *The Feeling of Power* aborda el problema de la automatización que lleva a una pérdida de habilidades humanas, un técnico redescubre el arte perdido de la multiplicación, así como el dilema de qué hacer cuando el redescubrimiento se aplica a la guerra.

El libro de Norbert Wiener, *Dios y Golem, Inc.* (1964) predijo de manera correcta que las computadoras alcanzarían un rendimiento de nivel experto en juegos y otras tareas, y que especificar lo que queremos resultaría difícil. Wiener escribe:

> Si bien siempre es posible pedir algo distinto a lo que realmente queremos, esta posibilidad es más grave cuando el proceso por el cual vamos a obtener nuestro deseo es indirecto, y el grado en el que hemos obtenido nuestro deseo no está claro hasta el final. Por lo general, realizamos nuestros deseos, en la medida en que los realizamos, mediante un proceso de retroalimentación en el que comparamos el grado de logro de metas intermedias con nuestra anticipación de ellas. En este proceso, la retroalimentación pasa por nosotros y podemos retroceder antes de que sea demasiado tarde. Si la retroalimentación está incorporada en una máquina que no permite ser inspeccionada hasta que se alcance la meta final, las posibilidades de catástrofe aumentan enormemente. En realidad, odiaría viajar en el primer ensayo de un automóvil regulado por dispositivos de retroalimentación fotoeléctrica, a menos que hubiera en algún lugar un mango por el cual pudiera tomar el control si me encontrara conduciendo de manera directa hacia un árbol.

Resumimos los **códigos de ética** en este capítulo, pero la lista de organizaciones que han emitido conjuntos de principios crecen rápidamente, y ahora incluye a Apple, DeepMind, Facebook, Google, IBM, Microsoft, la Organización para la Cooperación y el Desarrollo Económicos (OCDE), la Organización de las Naciones Unidas para la Educación, la Ciencia y la Cultura (UNESCO), la Oficina de Política Científica y Tecnológica de Estados Unidos, la Academia de Inteligencia Artificial de Beijing (BAAI), el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE), la Asociación de Maquinaria de Computación (ACM), el Foro Económico Mundial, el Grupo de los Veinte (G20), OpenAI, el Instituto de Investigación de Inteligencia de Máquina (MIRI), AI4People, el Centro para el Estudio del Riesgo Existencial, el Centro para la IA Compatible con los Humanos, el Centro para la Tecnología Humana, la Asociación en IA, el Instituto AI Now, el Instituto para el Futuro de la Vida, el Instituto para el Futuro de la Humanidad, la Unión Europea, y al menos 42 gobiernos nacionales. Tenemos el manual sobre la *Ética de la computación* (Berleur y Brunnstein, 2001) e introducciones al tema de la ética de la IA en formato de libro (Boddington, 2017) y encuesta (Etzioni y Etzioni, 2017a). La *Revista de Inteligencia Artificial y Derecho* y *AI y Sociedad* cubren asuntos de ética. Ahora veamos algunos de los problemas individuales.

Armas autónomas letales: *Wired for War* (2009) de P. W. Singer planteó cuestiones éticas, legales y técnicas en torno a los robots en el campo de batalla. *Army of None* (2018) de Paul Scharre, escrito por uno de los autores de la política actual de Estados Unidos sobre armas autónomas, ofrece una visión equilibrada y autorizada. Etzioni y Etzioni (2017b) abordan la cuestión de si la inteligencia artificial debería ser regulada; recomiendan una pausa en el desarrollo de armas autónomas letales, y una discusión internacional sobre el tema de la regulación.

Privacidad: Latanya Sweeney (Sweeney, 2002b) presenta el modelo de *k*-anonimato y la idea de generalizar campos (Sweeney, 2002a). Lograr *k*-anonimato con pérdida mínima de datos es un problema NP-difícil, pero Bayardo y Agrawal (2005) dan un algoritmo de aproximación. Cynthia Dwork (2008) describe la privacidad diferencial, y en trabajos posteriores da ejemplos prácticos de formas inteligentes de aplicar la privacidad diferencial para lograr mejores resultados que el enfoque ingenuo (Dwork *et al*., 2014). Guo *et al*. (2019) describen un proceso para la eliminación certificada de datos: si entrena un modelo con algunos datos, y luego hay una solicitud para eliminar algunos de los datos, esta extensión de la privacidad diferencial le permite modificar el modelo y demostrar que no hace uso de los datos eliminados. Ji *et al*. (2014) dan una revisión del campo de la privacidad. Etzioni (2004) aboga por un equilibrio entre privacidad y seguridad; derechos individuales y comunidad. Fung *et al*. (2018), Bagdasaryan *et al*. (2018) discuten los diversos ataques a los protocolos de aprendizaje federado. Narayanan *et al*. (2011) describen cómo pudieron desanonimizar el grafo de conexión ofuscado del Desafío de la Red Social 2011 rastreando el sitio de donde se obtuvieron los datos (Flickr), y emparejando nodos con un grado inusualmente alto de entrada o salida entre los datos proporcionados y los datos rastreados. Esto les permitió obtener información adicional para ganar el desafío, y también les permitió descubrir la verdadera identidad de los nodos en los datos. Las herramientas para la privacidad del usuario están disponibles; por ejemplo, TensorFlow proporciona módulos para el aprendizaje federado y la privacidad (McMahan y Andrew, 2018).

Equidad: El libro de Cathy O'Neil *Armas de destrucción matemática* (2017) describe cómo múltiples modelos de aprendizaje automático de caja negra influyen en nuestras vidas, en la mayoría de los casos de manera injusta. Ella llama a los constructores de modelos a asumir la responsabilidad de la equidad y a los responsables de las políticas a imponer una regulación apropiada. Dwork *et al*. (2012) mostraron las fallas con el enfoque simplista de "equidad a través de la ignorancia". Bellamy *et al*. (2018) presentan un kit de herramientas para mitigar el sesgo en los sistemas de aprendizaje automático. Tramèr *et al*. (2016) muestran cómo un adversario puede "robar" un modelo de aprendizaje automático haciendo consultas contra una API, Hardt *et al*. (2017) describen la igualdad de oportunidades como una métrica de equidad. Chouldechova y Roth (2018) dan una visión general de las fronteras de la equidad, y Verma y Rubin (2018) dan una encuesta exhaustiva de las definiciones de equidad.

Kleinberg *et al*. (2016) muestran que, en general, un algoritmo no permite ser a la vez bien calibrado y de igualdad de oportunidades. Berk *et al*. (2017) dan algunas definiciones adicionales de tipos de equidad, y de nuevo concluyen que es imposible satisfacer todos los aspectos a la vez. Beutel *et al*. (2019) dan consejos sobre cómo poner en práctica las métricas de equidad.

Dressel y Farid (2018) informan sobre el modelo de puntuación de reincidencia COMPAS. Christin *et al*. (2015) y Eckhouse *et al*. (2019) discuten el uso de algoritmos predictivos en el sistema legal. Corbett-Davies *et al*. (2017) muestran que existe una tensión entre garantizar la equidad y optimizar la seguridad pública, y Corbett-Davies y Goel (2018) discuten las diferencias entre los marcos de equidad. Chouldechova (2017) aboga por un impacto justo: todas las clases deberían tener la misma utilidad esperada. Liu *et al*. (2018a) abogan por una medida de impacto a largo plazo, señalando que, por ejemplo, si cambiamos el punto de decisión para aprobar un préstamo con el fin de ser más justos a corto plazo, esto podría tener un efecto negativo a largo plazo en las personas que terminan incumpliendo un préstamo y, por lo tanto, tienen su puntuación de crédito limitada.

Desde 2014 ha habido una conferencia anual sobre Equidad, Responsabilidad y Transparencia en el Aprendizaje Automático. Mehrabi *et al*. (2019) dan una encuesta exhaustiva de sesgo y equidad en el aprendizaje automático, catalogando 23 tipos de sesgo y 10 definiciones de equidad.

Confianza: La IA explicativa fue un tema importante desde los primeros días de los sistemas expertos (Neches *et al.,* 1985), y ha estado resurgiendo en los últimos años (Biran y Cotton, 2017;

Miller *et al*., 2017; Kim, 2018). Barreno *et al.* (2010) dan una taxonomía de los tipos de ataques de seguridad que se pueden hacer contra un sistema de aprendizaje automático, y Tygar (2011) hace un repaso del aprendizaje automático adversario. Los investigadores de IBM tienen una propuesta para ganar confianza en los sistemas de IA mediante declaraciones de conformidad (Hind *et al*., 2018). DARPA requiere decisiones explicables para sus sistemas de campo de batalla y ha emitido una convocatoria de investigación en el área (Gunning, 2016).

Seguridad de la IA: El libro *Seguridad y protección de la inteligencia artificial* (Yampolskiy, 2018) recopila ensayos sobre la seguridad de la IA, tanto recientes como clásicos, remontándose a *Por qué el futuro no nos necesita* (Joy, 2000). El "problema del rey Midas" fue anticipado por Marvin Minsky, quien una vez sugirió que un programa de IA diseñado para resolver la hipótesis de Riemann podría terminar tomando todos los recursos de la Tierra para construir supercomputadoras más potentes. De manera similar, Omohundro (2008) prevé un programa de ajedrez que secuestra recursos, y Bostrom (2014) describe la fábrica de clips de papel descontrolada que se apodera del mundo. Yudkowsky (2008) entra en más detalles sobre cómo diseñar una **IA amigable**. Amodei *et al*. (2016) presentan cinco problemas prácticos de seguridad para los sistemas de IA.

Omohundro (2008) describe las *Unidades básicas de IA* y concluye, "Las estructuras sociales que hacen que los individuos asuman el costo de sus externalidades negativas contribuirían en gran medida a garantizar un futuro estable y positivo". El libro de Elinor Ostrom *Gobernando los comunes* (1990) describe prácticas para lidiar con las externalidades por culturas tradicionales. Ostrom también ha aplicado este enfoque a la idea del conocimiento como un bien común (Hess y Ostrom, 2007).

Ray Kurzweil (2005) proclamó que *La Singularidad está cerca*, y una década después Murray Shanahan (2015) dio una actualización sobre el tema. El cofundador de Microsoft, Paul Allen, contrarrestó con *La Singularidad no está cerca* (2011). No disputó la posibilidad de máquinas ultrainteligentes, pensó que tomaría más de un siglo llegar allí. Rod Brooks es un crítico frecuente del singularitarianismo, pues señala que las tecnologías con frecuencia tardan más de lo previsto en madurar, que somos propensos al pensamiento mágico y que los exponenciales no duran para siempre (Brooks, 2017).

Por otro lado, por cada singularitario optimista hay un pesimista que teme a la nueva tecnología. El sitio web `pessimists.co` muestra que esto ha sido cierto a lo largo de la historia: por ejemplo, en la década de 1890 la gente estaba preocupada de que el ascensor causaría inevitablemente náuseas, que el telégrafo llevaría a la pérdida de privacidad y a la corrupción moral, que el metro liberaría aire subterráneo peligroso y perturbaría a los muertos, y que la bicicleta, en especial la idea de una mujer montando una, era obra del diablo.

Hans Moravec (2000) introduce algunas de las ideas del transhumanismo, y Bostrom (2005) da una historia actualizada. La idea de la máquina ultrainteligente de Good fue prevista cien años antes en *Darwin Among the Machines* (1863) de Samuel Butler. Escrito cuatro años después de la publicación de *On the Origins of Species* de Charles Darwin y en un momento en que las máquinas más sofisticadas eran motores de vapor, el artículo de Butler imaginaba "el desarrollo final de la conciencia mecánica" por selección natural. El tema fue reiterado por George Dyson (1998) en un libro del mismo título, y fue referenciado por Alan Turing, quien escribió en 1951 "En algún momento, por lo tanto, deberíamos esperar que las máquinas tomen el control de la manera que se menciona en *Erewhon* de Samuel Butler" (Turing, 1996).

Derechos de los robots: Un libro editado por Yorick Wilks (2010) ofrece diferentes perspectivas sobre cómo deberíamos tratar a los compañeros artificiales, desde la visión de Joanna Bryson de que los robots deberían servirnos como herramientas, no como ciudadanos, hasta la observación de Sherry Turkle de que ya personificamos nuestras computadoras y otras herramientas, y estamos bastante dispuestos a difuminar los límites entre las máquinas y la vida. Wilks también

contribuyó con una actualización reciente sobre sus puntos de vista (Wilks, 2019). El libro *Robot rights* (2018) del filósofo David Gunkel considera cuatro posibilidades: ¿*pueden* los robots tener derechos o no, y *deberían* o no? La Sociedad Americana para la Prevención de la Crueldad hacia los Robots (ASPCR) proclama que "La ASPCR es, y seguirá siendo, tan seria como los robots son conscientes".

El trabajo futuro: En 1888, Edward Bellamy publicó el best-seller *Looking Backward*, que predijo que para el año 2000, los avances tecnológicos llevarían a una utopía donde se lograría la igualdad, las personas trabajarían pocas horas y se jubilarían temprano. Poco después, E. M. Forster tomó la visión distópica en *La máquina se detiene* (1909), en la que una máquina benevolente se hace cargo de la gestión de una sociedad; las cosas se desmoronan cuando la máquina falla. El libro previsor de Norbert Wiener *The Human Use of Human Beings* (1950) argumenta los beneficios de la automatización para liberar a las personas de la rutina mientras ofrece un trabajo más creativo, pero también discute varios peligros que reconocemos como problemas hoy en día, en particular, el problema de la alineación de valores.

El libro *Disrupting Unemployment* (Nordfors *et al*., 2018) discute algunas de las formas en que el trabajo está cambiando, abriendo oportunidades para nuevas carreras. Erik Brynjolfsson y Andrew McAfee abordan estos temas y más en sus libros *Race Against the Machine* (2011) y *The Second Machine Age* (2014). Ford (2015) describe los desafíos del aumento de la automatización, y West (2018) proporciona recomendaciones para mitigar los problemas, mientras que Thomas Malone (2004) del MIT muestra que muchos de los mismos problemas eran evidentes una década antes, pero en ese momento se atribuían a las redes de comunicación mundial, no a la automatización.

CAPÍTULO 28

EL FUTURO DE LA IA

En el que intentamos prever a corto plazo.

En el Capítulo 2 decidimos pensar la IA como la tarea de diseñar agentes relativamente racionales. Múltiples diseños de agentes fueron considerados, desde agentes de reflejo hasta agentes de decisión teórica basados en conocimiento y agentes de aprendizaje profundo que utilizan el aprendizaje por refuerzo. También hay variedad en las tecnologías componentes a partir de las cuales se ensamblan estos diseños: razonamiento lógico, probabilístico o neural; representaciones atómicas, factorizadas o estructuradas de estados; varios algoritmos de aprendizaje a partir de diversos tipos de datos; sensores y actuadores para interactuar con el mundo. Por último, hemos visto una variedad de aplicaciones en medicina, finanzas, transporte, comunicación y otros campos. Ha habido progreso en todos estos frentes, tanto en nuestra comprensión científica como en nuestras capacidades tecnológicas.

La mayoría de los expertos son optimistas sobre el progreso continuo; como ya vimos en la página 28, la estimación mediana es de una IA aproximada a nivel humano en una amplia variedad de tareas en algún momento en los próximos 50 a 100 años. En la próxima década, se predice que la IA agregará billones de dólares a la economía cada año. Pero, como también vimos, hay algunos críticos que piensan que la IA general está a siglos de distancia, además de que hay numerosas preocupaciones éticas sobre la imparcialidad, equidad y letalidad de la IA. En este capítulo nos preguntamos: ¿hacia dónde nos dirigimos y qué queda por hacer? Lo hacemos cuestionándonos si tenemos los componentes, arquitecturas y objetivos correctos para hacer de la IA una tecnología exitosa que brinde beneficios al mundo.

28.1 Componentes de IA

Esta sección examina los componentes de los sistemas de IA y el hecho de hasta qué punto cada uno de ellos podría acelerar o dificultar el progreso futuro.

Sensores y actuadores

Durante gran parte de la historia de la IA, el acceso directo al mundo ha estado ausente de manera considerable. Con algunas excepciones, los sistemas de IA se construyeron de modo que los humanos tenían que proporcionar las entradas e interpretar las salidas. Mientras tanto, los sistemas robóticos se centraron en tareas de bajo nivel en las que el razonamiento, la planificación de alto nivel en general se ignoraban y la necesidad de percepción se minimizaba. Esto se debió en parte al gasto y esfuerzo enormes de ingeniería requeridos para hacer funcionar los robots reales y, en parte, debido a la escasa potencia de procesamiento y algoritmos suficientemente efectivos para manejar la entrada visual de alto ancho de banda.

La situación ha cambiado con rapidez en los últimos años gracias a la disponibilidad de robots programables listos para usar. Estos, a su vez, se han beneficiado de las unidades de motor compactas y

confiables y de los sensores mejorados. El costo del LiDAR para un automóvil autónomo ha caído de $75 000 a $1 000, y una versión de un solo chip puede alcanzar $10 por unidad (Poulton y Watts, 2016). Los sensores de radar, que antes sólo eran capaces de una detección de grano grueso, ahora son lo suficientemente sensibles para contar el número de hojas en una pila de papel (Yeo *et al.,* 2018).

La demanda de un mejor procesamiento de imágenes en las cámaras de los teléfonos móviles ha resultado en económicas cámaras de alta resolución para su uso en robótica. La tecnología MEMS (sistemas microelectromecánicos) ha proporcionado acelerómetros, giroscopios y actuadores miniaturizados lo suficientemente pequeños como para caber en insectos voladores artificiales (Floreano *et al.,* 2009; Fuller *et al.,* 2014). Es posible combinar millones de dispositivos MEMS para producir actuadores macroscópicos potentes. La impresión 3D (Muth *et al.,* 2014) y la bioimpresión (Kolesky *et al.,* 2014) han facilitado la experimentación con prototipos.

Así, vemos que los sistemas de IA están en el umbral de pasar de ser meros sistemas de software a sistemas robóticos útiles integrados. El estado de la robótica hoy es casi comparable al estado de las computadoras personales a principios de la década de 1980: en ese momento la disponibilidad de las computadoras personales iba en ascenso, pero pasaría otra década antes de que se fueran comunes. Es probable que los robots flexibles e inteligentes primero avancen en la industria (donde los entornos son más controlados, las tareas son repetitivas y el valor de una inversión es más fácil de medir) antes que en el mercado doméstico (donde hay más variabilidad en el entorno y las tareas).

Representando el estado del mundo

Mantener un registro del mundo requiere percepción y actualización de las representaciones internas. Así, el Capítulo 4 mostró cómo mantener un registro de las representaciones de estado atómico, el Capítulo 7 describió cómo hacer un registro para las representaciones de estado factorizadas (proposicionales), el Capítulo 10 lo extendió a la lógica de primer orden y el Capítulo 14 describió el razonamiento probabilístico a lo largo del tiempo en entornos inciertos. El Capítulo 21 introdujo las redes neuronales recurrentes, que también son capaces de mantener una representación de estado a lo largo del tiempo.

Los algoritmos actuales de filtrado y percepción se combinan para lograr un trabajo razonable de reconocimiento de objetos (“eso es un gato”) e informar predicados de bajo nivel (“la taza está sobre la mesa”). Reconocer acciones de nivel superior, como “El Dr. Russell está tomando una taza de té con el Dr. Norvig mientras discuten planes para la próxima semana”, es más difícil. En la actualidad a veces se puede hacer (ver Figura 25.17 en la página 908) con suficientes ejemplos de entrenamiento, pero el progreso futuro requerirá técnicas que se generalicen a situaciones novedosas sin requerir ejemplos exhaustivos (Poppe, 2010; Kang y Wildes, 2016).

Otro problema es que, aunque los algoritmos de filtrado aproximado del Capítulo 14 permiten manejar ambientes muy grandes, todavía lidian con una representación factorizada, tienen variables aleatorias, pero no representan objetos y relaciones de manera explícita. Además, su noción de tiempo se limita a cambios paso a paso; dada la trayectoria reciente de una pelota, nosotros podemos predecir dónde estará en el tiempo $t + 1$, pero es difícil representar la idea abstracta de que lo que sube debe bajar.

La sección 15.1 explicó cómo se combinan la probabilidad y la lógica de primer orden para resolver estos problemas, la Sección 15.2 mostró cómo manejar la incertidumbre sobre la identidad de los objetos, mientras que el Capítulo 25 mostró de qué manera las redes neuronales recurrentes permiten a la visión por computadora rastrear el mundo; no obstante, aún carecemos de forma adecuada para integrar todas estas técnicas. El Capítulo 24 mostró cómo los embeddings de palabras y representaciones similares pueden liberarnos de los límites estrictos de los conceptos definidos por condiciones necesarias y suficientes. Definir esquemas de representación generales y reutilizables para dominios complejos sigue siendo una tarea desalentadora.

Seleccionando acciones

La principal dificultad en la selección de acciones en el mundo real es lidiar con planes a largo plazo, como graduarse de la universidad en cuatro años, que consisten en miles de millones de pasos primitivos. Los algoritmos de búsqueda que consideran secuencias de acciones primitivas apenas escalan a decenas o quizá cientos de pasos. Sólo imponiendo una **estructura jerárquica** en el comportamiento es que los humanos podemos lidiar con todo. Vimos en la Sección 11.4 cómo usar representaciones jerárquicas para manejar problemas de esta escala; además, el trabajo en el **aprendizaje por refuerzo jerárquico** ha tenido éxito en combinar estas ideas con el formalismo MDP descrito en el Capítulo 17.

Hasta ahora, estos métodos no se han extendido al caso parcialmente observable (POMDPs). Además, para resolver POMDPs los algoritmos suelen utilizar la misma representación de estado atómico que utilizamos para los algoritmos de búsqueda del Capítulo 3. Observe que aquí hay mucho trabajo por hacer, pero en su mayoría las bases técnicas están en su lugar para avanzar. El principal elemento faltante es un método efectivo para *construir* las representaciones jerárquicas de estado y comportamiento que son necesarias para la toma de decisiones a largo plazo.

Decidiendo lo que queremos

El Capítulo 3 introdujo algoritmos de búsqueda para encontrar un estado objetivo. No obstante, los agentes basados en objetivos son frágiles cuando el ambiente es incierto y cuando hay múltiples factores a considerar. En principio, los agentes de maximización de utilidad abordan estos problemas de una manera general. Los campos de la economía y la teoría de juegos, así como la IA, hacen uso de esta idea: sólo declara lo que quieres optimizar y lo que hace cada acción, entonces podemos calcular la acción óptima.

En la práctica, ahora nos damos cuenta de que la tarea de elegir la función de utilidad correcta es un problema desafiante en sí mismo. Imagine, por ejemplo, la compleja red de preferencias interactivas que debe ser entendida por un agente que opera como asistente de oficina para un ser humano. El problema se complica por el hecho de que cada humano es diferente, por lo que un agente recién "sacado de la caja" no tendrá suficiente experiencia con ningún individuo en particular para aprender un modelo de preferencia preciso; necesariamente tendrá que operar bajo incertidumbre de preferencia. La complejidad adicional surge si queremos asegurarnos de que nuestros agentes están actuando de manera justa y equitativa para la sociedad, en lugar de para un solo individuo.

Todavía no tenemos experiencia suficiente en la construcción de modelos de preferencia complejos del mundo real, y aún menos en las distribuciones de probabilidad sobre dichos modelos. Aunque existen formalismos factorizados destinados a descomponer las preferencias sobre estados complejos, similares a las redes de Bayes, ha resultado difícil utilizar estos formalismos en la práctica. Una razón puede ser que las preferencias sobre los estados se *compilan* a partir de las preferencias sobre las historias de estado, que son descritas por **funciones de recompensa** (ver Capítulo 17). Incluso si la función de recompensa es simple, la función de utilidad correspondiente es compleja.

Esto sugiere que tomemos en serio la tarea de la ingeniería del conocimiento para las funciones de recompensa como una forma de transmitir a nuestros agentes lo que queremos que hagan. La idea del **aprendizaje por refuerzo inverso** (Sección 22.6) es un enfoque a este problema cuando tenemos un experto que puede realizar una tarea, pero no explicarla. También podríamos usar mejores lenguajes para expresar lo que queremos. Por ejemplo, en robótica, la lógica temporal lineal facilita decir qué cosas queremos que sucedan en un futuro cercano, qué cosas queremos evitar y qué estados queremos que persistan para siempre (Littman *et al.,* 2017). Necesitamos maneras mejores de decir lo que queremos y mejores formas para que los robots interpreten la información que proporcionamos.

La industria informática en su conjunto ha desarrollado un poderoso ecosistema para agregar las preferencias de los usuarios. Cuando hace clic en algo dentro de una aplicación —juego online, red social o sitio de compras— su clic sirve como recomendación de que usted (y sus pares similares) querrían ver cosas equivalentes en el futuro. (O podría ser que el sitio es confuso y ha hecho clic en la cosa equivocada, los datos siempre son ruidosos). El feedback inherente en este sistema lo hace muy efectivo a corto plazo para seleccionar juegos y videos cada vez más adictivos.

No obstante, estos sistemas fallan con frecuencia al proporcionar una forma fácil de optar por no participar: su dispositivo reproducirá de manera automática un video relevante, pero es menos probable que le diga "quizás es hora de guardar sus dispositivos y dar un paseo relajante por la naturaleza". Un sitio de compras le ayudará a encontrar ropa que combine con su estilo, pero no abordará la paz mundial o el fin del hambre y la pobreza. En la medida en que el menú de opciones está impulsado por empresas que intentan obtener beneficios de la atención de un cliente, el menú seguirá siendo incompleto.

Sin embargo, las empresas sí responden a los intereses de los clientes, pues muchos clientes han expresado su opinión de que están interesados en un mundo justo y sostenible. Tim O'Reilly explica por qué el beneficio no es el único motivo con la siguiente analogía: "El dinero es como la gasolina durante un viaje por carretera. No quiere quedarse sin gasolina en su viaje, pero no está haciendo un tour de gasolineras. Tiene que prestar atención al dinero, pero no debería ser sólo por el dinero".

El movimiento de **tiempo bien empleado** de Tristan Harris en el Centro para la Tecnología Humana es un paso que nos da opciones más completas (Harris, 2016). El movimiento aborda un problema reconocido por Herbert Simon en 1971: "Una riqueza de información crea una pobreza de atención". Quizás en el futuro tendremos **agentes personales** que defiendan nuestros verdaderos intereses a largo plazo en lugar de los intereses de las corporaciones, cuyas aplicaciones llenan nuestros dispositivos. Será trabajo del agente mediar las ofertas de varios vendedores, protegernos de los captadores de atención adictivos y guiarnos hacia los objetivos que realmente nos importan.

Tiempo bien empleado

Agentes personales

Aprendizaje

Los capítulos 19 a 22 describieron cómo aprenden los agentes. Los algoritmos actuales permiten manejar problemas muy grandes que alcanzan o superan las capacidades humanas en muchas tareas, siempre que tengamos suficientes ejemplos de entrenamiento y tratemos con un vocabulario predefinido de características y conceptos. El aprendizaje puede estancarse cuando los datos son escasos o no supervisados o cuando tratamos con representaciones complejas.

Gran parte del recién resurgimiento de la IA en la prensa popular y en la industria se debe al éxito del aprendizaje profundo (Capítulo 21). Por un lado, esto puede verse como la maduración incremental del subcampo de las redes neuronales. Por otro lado, podemos verlo como un salto revolucionario en las capacidades impulsado por una confluencia de factores: la disponibilidad de más datos de entrenamiento gracias a Internet, mayor poder de procesamiento de hardware especializado y algunos trucos algorítmicos, como las redes generativas adversarias (GANs), la normalización por lotes, el abandono y la función de activación lineal rectificada (ReLU).

El futuro debería ver un énfasis continuado en mejorar el aprendizaje profundo para las tareas en las que sobresale, y también extenderlo para cubrir otras tareas. El nombre de marca "aprendizaje profundo" ha demostrado ser tan popular que deberíamos esperar su uso continuado, incluso si la mezcla de técnicas que lo alimentan cambia de manera considerable.

Hemos visto la aparición del campo de la **ciencia de datos** como la confluencia de estadísticas, programación y experiencia en el dominio. Mientras esperamos ver un desarrollo continuado en las herramientas y técnicas necesarias para adquirir, gestionar y mantener **big data**, también

necesitamos avances en el **aprendizaje de transferencia** para aprovechar los datos en un dominio y mejorar el rendimiento en un dominio relacionado.

La mayoría de la investigación en aprendizaje automático hoy en día asume una representación factorizada, aprendiendo una función $h : \mathbb{R}^n \rightarrow \mathbb{R}$ para regresión y $h : \mathbb{R}^n \rightarrow \{0,1\}$ para clasificación. El aprendizaje automático ha sido menos exitoso para problemas que tienen sólo una pequeña cantidad de datos o problemas que requieren la construcción de nuevas representaciones estructuradas y jerárquicas. El aprendizaje profundo, en especial con redes convolucionales aplicadas a problemas de visión por computadora, ha demostrado cierto éxito en pasar de pixeles de bajo nivel a conceptos de nivel intermedio como *Ojo* y *Boca*, luego a *Cara* y, por último, a *Persona* o *Gato*.

Un desafío para el futuro es combinar de manera más fluida el aprendizaje y el conocimiento previo. Si le damos a una computadora un problema que no ha encontrado antes; digamos, reconocer diferentes modelos de automóviles, no queremos que el sistema sea impotente hasta que se le hayan alimentado millones de ejemplos etiquetados.

El sistema ideal debería ser capaz de basarse en lo que ya sabe: ya debería tener un modelo de cómo funciona la visión y cómo funciona el diseño y la marca de los productos en general; ahora debería utilizar el **aprendizaje de transferencia** para aplicarlo al nuevo problema de los modelos de automóviles. Debería ser capaz de encontrar por sí mismo información sobre los modelos de coches, extrayendo de textos, imágenes y videos disponibles en Internet. Debería ser capaz de **aprender por observación**: tener una conversación con un profesor, y no sólo preguntar: "¿puedo tener mil imágenes de un Corolla?", sino ser capaz de entender consejos como "el Insight es similar al Prius, pero el Insight tiene una parrilla más grande". Debería saber que cada modelo viene en un pequeño rango de colores posibles, pero que un coche permite ser repintado, por lo que existe una posibilidad de que vea un coche en un color que no estaba en el conjunto de entrenamiento. (Si no lo sabía, debería ser capaz de aprenderlo, o de que se le informe sobre ello).

Todo esto requiere un lenguaje de comunicación y representación que los humanos y los ordenadores puedan compartir; no podemos esperar que un analista humano modifique de manera directa un modelo con millones de pesos. Los modelos probabilísticos (incluidos los lenguajes de programación probabilística) dan a los humanos cierta capacidad para describir lo que sabemos, aunque estos modelos aún no están bien integrados con otros mecanismos de aprendizaje.

El trabajo de Bengio y LeCun (2007) es un paso hacia esta integración. Recién Yann LeCun ha sugerido que el término "aprendizaje profundo" debería ser reemplazado por **programación diferenciable**, que es más general (Siskind y Pearlmutter, 2016; Li *et al.,* 2018); esto sugiere que nuestros lenguajes de programación generales y nuestros modelos de aprendizaje automático podrían fusionarse.

Programación diferenciable

Ahora mismo, es común construir un modelo de aprendizaje profundo que es diferenciable y, por lo tanto, que permite ser entrenado para minimizar la pérdida, y reentrenado cuando las circunstancias cambian. Sin embargo, ese modelo de aprendizaje profundo es sólo una parte de un sistema de software más grande que toma datos, manosea los datos, los alimenta al modelo y averigua qué hacer con la salida del modelo. Todas estas otras partes del sistema más grande fueron escritas a mano por un programador y, en consecuencia, son no diferenciables, lo que significa que cuando las circunstancias cambian, es tarea del programador reconocer cualquier problema y solucionarlo a mano. Con la programación diferenciable, la esperanza es que todo el sistema esté sujeto a optimización automatizada.

El objetivo final es poder expresar lo que sabemos en cualquier forma que nos sea conveniente: consejos informales dados en lenguaje natural, una ley matemática fuerte como $F = ma$, un modelo estadístico acompañado de datos o un programa probabilístico con parámetros desconocidos que permite ser optimizado de manera automática a través del descenso de gradiente. Nuestros modelos de computadora aprenderán de las conversaciones con expertos humanos así como utilizando todos los datos disponibles.

Yann LeCun, Geoffrey Hinton y otros han sugerido que el énfasis actual en el aprendizaje supervisado (y, en menor medida, en el aprendizaje por refuerzo) no es sostenible, que los modelos de computadora tendrán que depender del **aprendizaje débilmente supervisado**, en el que se da alguna supervisión con un pequeño número de ejemplos etiquetados y un pequeño número de recompensas, pero la mayor parte del aprendizaje es no supervisado, porque los datos no anotados son mucho más abundantes.

Aprendizaje predictivo

LeCun utiliza el término **aprendizaje predictivo** para un sistema de aprendizaje no supervisado que permite modelar el mundo y aprender a predecir aspectos de los estados futuros del mundo, no sólo predecir etiquetas para entradas que son independientes e idénticamente distribuidas con respecto a los datos pasados, y no sólo predecir una función de valor sobre los estados. Sugiere que las GANs (redes generativas adversarias) son utilizadas para aprender a minimizar la diferencia entre las predicciones y la realidad.

Geoffrey Hinton declaró en 2017 "Mi punto de vista es tirarlo todo y empezar de nuevo", lo que significaba que la idea general de aprender ajustando parámetros en una red es duradera, pero los detalles de la arquitectura de las redes y la técnica de retropropagación necesitan ser replanteados. Smolensky (1988) tenía una prescripción de cómo pensar sobre los modelos conexionistas; sus pensamientos siguen siendo relevantes hoy en día.

Recursos

La investigación y el desarrollo del aprendizaje automático se ha acelerado gracias a la creciente disponibilidad de datos, de almacenamiento, de potencia de procesamiento, de software, de expertos capacitados y a las inversiones necesarias para apoyarlos. Desde la década de 1970, ha habido una aceleración de 100 000 veces en los procesadores de propósito general y una aceleración adicional de 1 000 veces debido al hardware especializado de aprendizaje automático. La Web ha servido como una fuente rica de imágenes, videos, voz, texto y datos semiestructurados, que en la actualidad añaden más de 10^{18} bytes cada día.

Cientos de conjuntos de datos de alta calidad están disponibles para una gran variedad de tareas en visión por computadora, reconocimiento de voz y procesamiento de lenguaje natural. Si los datos que necesita no están ya disponibles, por lo regular permite ensamblarlos a partir de otras fuentes o involucrar humanos para etiquetar datos para usted mediante una plataforma de crowdsourcing. Validar los datos obtenidos de esta manera se convierte en una parte importante del flujo de trabajo general (Hirth *et al.,* 2013).

Modelos compartidos

Un desarrollo reciente importante es el cambio de datos compartidos a **modelos compartidos**. Los principales proveedores de servicios en la nube (como Amazon, Microsoft, Google, Alibaba, IBM, Salesforce) han comenzado a competir para ofrecer APIs de aprendizaje automático con modelos preconstruidos para tareas específicas, como reconocimiento de objetos visuales y de voz, y traducción automática. Estos modelos permiten ser utilizados tal como están o pueden servir como base para ser personalizados con datos particulares para su aplicación particular.

Esperamos que estos modelos mejoren con el tiempo y que sea inusual comenzar un proyecto de aprendizaje automático desde cero, tal como ahora resulta inusual realizar un proyecto de desarrollo web desde cero, es decir, sin bibliotecas. Es posible que se produzca un gran salto en la calidad del modelo cuando se sea económico procesar todo el video en la Web; por ejemplo, por sí sola la plataforma YouTube añade 300 horas de video cada minuto.

La ley de Moore ha hecho que sea más rentable procesar datos; un megabyte de almacenamiento costaba $1 millón en 1969 pero menos de $0.02 en 2019, además de que el rendimiento de las supercomputadoras ha aumentado en un factor de más de 10^{10} en ese tiempo. Los componentes de hardware especializados para el aprendizaje automático como las unidades de procesamiento gráfico (GPUs), los núcleos tensoriales, las unidades de procesamiento tensorial (TPUs), y los arreglos de compuertas lógicas programables (FPGAs) son cientos de veces más rápidos que

las CPUs convencionales para el entrenamiento de aprendizaje automático (Vasilache *et al.,* 2014; Jouppi *et al.,* 2017). En 2014 se necesitaba un día completo para entrenar un modelo de ImageNet; en 2018 llevaba 2 minutos (Ying *et al.,* 2018).

El Instituto OpenAI informa que la cantidad de potencia de cálculo utilizada para entrenar los modelos de aprendizaje automático más grandes se duplicó cada 3.5 meses desde 2012 hasta 2018, alcanzando más de un exaflop/segundo-día para AlphaZero (aunque también informan que algunos trabajos respetados utilizaron 100 millones de veces menos potencia de cálculo (Amodei y Hernandez, 2018)). Las mismas tendencias económicas que han hecho que las cámaras de los teléfonos móviles sean más baratas y mejores también se aplican a los procesadores, veremos un progreso continuo en la computación de alto rendimiento y bajo consumo que se beneficia de las economías de escala.

Existe la posibilidad de que las computadoras cuánticas podrían acelerar la IA. En la actualidad existen algunos algoritmos cuánticos rápidos para las operaciones de álgebra lineal utilizadas en el aprendizaje automático (Harrow *et al.*, 2009; Dervovic *et al.,* 2018), pero no hay ninguna computadora cuántica capaz de ejecutarlos. Tenemos algunos ejemplos de aplicaciones de tareas como la clasificación de imágenes (Mott *et al.,* 2017) donde los algoritmos cuánticos son tan buenos como los algoritmos clásicos en problemas pequeños.

Las computadoras cuánticas actuales manejan sólo unas pocas decenas de bits, mientras que los algoritmos de aprendizaje automático en la mayoría de los casos manejan entradas con millones de bits y crean modelos con cientos de millones de parámetros. Es un hecho que necesitamos avances tanto en el hardware como en el software cuántico para hacer que la computación cuántica sea práctica para el aprendizaje automático a gran escala. De manera alternativa, existe una división de trabajo, tal vez un algoritmo cuántico, para buscar con mayor eficiencia el espacio de hiperparámetros mientras el proceso de entrenamiento normal se ejecuta en computadoras convencionales; no obstante, aún no sabemos cómo hacer eso. La investigación sobre algoritmos cuánticos a veces permite inspirar nuevos y mejores algoritmos en computadoras clásicas (Tang, 2018).

También hemos visto un crecimiento exponencial en el número de publicaciones, personas y dólares en IA/aprendizaje automático/ciencia de datos. Dean *et al.* (2018) muestran que el número de artículos sobre "aprendizaje automático" en arXiv se duplicó cada dos años desde 2009 hasta 2017. Los inversores están financiando empresas emergentes en estos campos, las grandes empresas contratan y gastan a medida que determinan su estrategia de IA, mientras que los gobiernos invierten para asegurarse que su país no quede muy rezagado.

28.2 Arquitecturas de IA

Es natural preguntar, "¿Cuál de las arquitecturas de agentes en el Capítulo 2 debería usar un agente?" La respuesta es, "¡Todas ellas!" Las respuestas reflejas son necesarias para situaciones en las que el tiempo es esencial, mientras que la deliberación basada en el conocimiento permite al agente planificar con anticipación. El aprendizaje es conveniente cuando tenemos muchos datos, y necesario cuando el entorno está cambiando, o cuando los diseñadores humanos tienen un conocimiento insuficiente del dominio.

La IA ha tenido durante mucho tiempo una división entre sistemas simbólicos (basados en inferencia lógica y probabilística) y sistemas conexionistas (basados en la minimización de pérdidas sobre un gran número de parámetros no interpretados). Un desafío continuo para la IA es unir estos dos y así capturar lo mejor de ambos. Los sistemas simbólicos nos permiten encadenar largas cadenas de razonamiento y aprovechar el poder expresivo de las representaciones estructuradas, mientras que los sistemas conexionistas permiten reconocer patrones incluso frente a datos ruidosos. Una línea de investigación tiene como objetivo combinar la programación probabilística con el aprendizaje profundo, aunque hasta ahora las diversas propuestas están limitadas en la medida en que los enfoques están fusionados.

Los agentes también necesitan formas de controlar sus propias deliberaciones. Deben ser capaces de utilizar bien el tiempo disponible y cesar en la deliberación cuando se exige la acción. Por ejemplo, un agente que conduce un taxi y ve un accidente adelante debe decidir en una fracción de segundo entre frenar o desviarse. También debería pasar esa fracción de segundo pensando en las preguntas más importantes, ¿los carriles a la izquierda y a la derecha están despejados? y si ¿hay un camión grande cerca?, en lugar de preocuparse por dónde recoger al próximo pasajero. Por lo general, estos problemas son estudiados bajo el título de **IA en tiempo real**. A medida que los sistemas de IA se mueven hacia dominios más complejos, todos los problemas se volverán en tiempo real, debido a que el agente nunca tendrá suficiente tiempo para resolver el problema de decisión.

IA en tiempo real

Es un hecho que existe una necesidad urgente de métodos *generales* para controlar la deliberación, en lugar de recetas específicas para qué pensar en cada situación. La primera idea útil son los **algoritmos en cualquier momento** (Dean y Boddy, 1988; Horvitz, 1987): un algoritmo cuya calidad de resultado mejora de manera gradual con el tiempo, de modo que tiene una decisión razonable lista cada vez que se interrumpe. Ejemplos de algoritmos en cualquier momento incluyen la profundización iterativa en la búsqueda de árboles de juego y MCMC en redes bayesianas.

Algoritmos en cualquier momento

La segunda técnica para controlar la deliberación es el **metarrazonamiento teórico de decisiones** (Russell y Wefald, 1989; Horvitz y Breese, 1996; Hay *et al.,* 2012). Este método, mencionado de manera breve en las Secciones 3.6.5 y 5.7, aplica la teoría del valor de la información (Capítulo 16) a la selección de cálculos individuales (Sección 3.6.5). El valor de un cálculo depende de tanto de su costo (en términos de retraso de la acción) como de sus beneficios (en términos de mejora de la calidad de la decisión).

Metarrazonamiento teórico de decisiones

Las técnicas de metarrazonamiento se usan para diseñar mejores algoritmos de búsqueda y para garantizar que los algoritmos tengan la propiedad de en cualquier momento. La búsqueda de árboles de Monte Carlo es un ejemplo: la elección del nodo hoja en el que comenzar el siguiente juego se realiza mediante una decisión metanivel casi racional derivada de la teoría de los bandidos.

Por supuesto, el metarrazonamiento es más costoso que la acción refleja, pero se aplican métodos de compilación para que la sobrecarga sea pequeña en comparación con los costos de los cálculos que se están controlando. El aprendizaje por refuerzo a nivel meta proporciona otra forma de adquirir políticas efectivas para controlar la deliberación: en esencia, los cálculos que conducen a mejores decisiones se refuerzan, mientras que aquellos que resultan no tener efecto son penalizados. Este enfoque evita los problemas de miopía del simple cálculo del valor de la información.

El metarrazonamiento es un ejemplo específico de una **arquitectura reflexiva**, es decir, una arquitectura que permite la deliberación sobre las entidades y acciones computacionales que ocurren dentro de la arquitectura misma. Se puede construir una base teórica para las arquitecturas reflexivas definiendo un espacio de estado conjunto, compuesto por el estado del entorno y el estado computacional del propio agente. Es posible diseñar algoritmos de toma de decisiones y aprendizaje que operen sobre este espacio de estado conjunto y, por lo tanto, sirvan para implementar y mejorar las actividades computacionales del agente. Con el tiempo, esperamos que algoritmos específicos de tareas como la búsqueda alfa-beta, la planificación de regresión y la eliminación de variables desaparezcan de los sistemas de IA, para ser reemplazados por métodos generales que dirijan los cálculos del agente hacia la generación eficiente de decisiones de alta calidad.

Arquitectura reflexiva

El metarrazonamiento y la reflexión (y muchos otros dispositivos arquitectónicos y algorítmicos relacionados con la eficiencia, explorados en este libro) son necesarios porque tomar decisiones es *difícil*. Desde que se inventaron los ordenadores, su velocidad cegadora ha llevado a la gente a sobreestimar su capacidad para superar la complejidad o, su equivalente, a subestimar

lo que significa la complejidad. El gigantesco poder de las máquinas de hoy en día nos tienta a pensar que podríamos prescindir de todos los dispositivos inteligentes y confiar más en la fuerza bruta. Así que intentemos contrarrestar esta tendencia. Comenzamos con lo que los físicos creen que es la velocidad del dispositivo de cálculo definitivo de 1 kg: alrededor de 10^{51} operaciones por segundo, o un billón de trillones de trillones de veces más rápido que el superordenador más rápido a partir de 2020 (Lloyd, 2000).[1] Luego proponemos una tarea simple: enumerar cadenas de palabras en inglés, tal como Borges propuso en *La Biblioteca de Babel*. Borges estipuló libros de 410 páginas. ¿Sería eso factible? No del todo. De hecho, el ordenador funcionando durante un año sólo podría enumerar las cadenas de 11 palabras.

Ahora considere el hecho de que un plan detallado para una vida humana consiste en (aproximadamente) veinte trillones de posibles actos musculares (Russell, 2019), y ya usted empieza a ver la escala del problema. Un ordenador que es un billón de trillones de trillones de veces más potente que el cerebro humano está mucho más lejos de ser racional de lo que una babosa está de adelantar a la nave estelar Enterprise viajando a velocidad warp nueve.

Con estas consideraciones en mente, parece que el objetivo de construir agentes racionales es quizás un poco ambicioso. En lugar de apuntar a algo que tal vez no puede existir, deberíamos considerar un objetivo normativo diferente, uno que *necesariamente* existe. Recuerde del Capítulo 2 la siguiente idea simple:

agente = arquitectura + programa.

Ahora fije la arquitectura del agente (las capacidades de la máquina subyacente, quizá con una capa de software fija en la parte superior) y permita que el programa del agente varíe sobre todos los programas posibles que la arquitectura permite soportar. En cualquier entorno de tarea dado, uno de estos programas (o una clase de equivalencia de ellos) ofrece el mejor rendimiento posible, quizá no cerca de la racionalidad perfecta, pero mejor que cualquier otro programa de agente. Decimos que este programa satisface el criterio de **optimalidad acotada**. Existe, y de manera manifiesta constituye un objetivo deseable. El truco está en encontrarlo o encontrar algo cercano a él.

Optimalidad acotada

Para algunas clases elementales de programas de agentes en entornos de tiempo real simples, es posible identificar programas de agentes óptimos acotados (Etzioni, 1989; Russell y Subramanian, 1995). El éxito de la búsqueda de árboles de Monte Carlo ha revivido el interés en la toma de decisiones a nivel meta, y hay razones para esperar que la optimalidad acotada dentro de familias más complejas de programas de agentes pueda ser lograda por técnicas como el aprendizaje por refuerzo a nivel meta. También debería ser posible desarrollar una teoría constructiva de la arquitectura, comenzando con teoremas sobre la optimalidad acotada de métodos adecuados de combinación de diferentes componentes óptimos acotados, como sistemas de reflejo y acción-valor.

IA general

Gran parte del progreso en IA hasta ahora en el siglo XXI ha sido guiado por la competencia en tareas estrechas, como ¡el Gran Desafío DARPA para coches autónomos, la competencia de reconocimiento de objetos ImageNet o jugar Go, ajedrez, póker o Jeopardy contra un campeón mundial! Para cada tarea separada construimos un sistema de IA separado, por lo general con un modelo de aprendizaje automático entrenado desde cero, con datos recopilados de manera específica para esta tarea. Pero un agente en verdad inteligente debería ser capaz de hacer más de una cosa. Alan Turing (1950) propuso su lista (página 982) y el autor de ciencia ficción Robert Heinlein (1973) respondió con:

[1] Pasamos por alto el hecho de que este dispositivo consume toda la energía de una estrella y opera a mil millones de grados centígrados.

> Un ser humano debería ser capaz de cambiar un pañal, planificar una invasión, destazar un cerdo, capitanear un barco, diseñar un edificio, escribir un soneto, balancear cuentas, construir un muro, recolocar un hueso, consolar a moribundos, tomar órdenes, dar órdenes, cooperar, actuar solo, resolver ecuaciones, analizar un problema nuevo, esparcir estiércol, programar una computadora, cocinar una comida sabrosa, luchar de manera eficiente, morir con gallardía. La especialización es para insectos.

Hasta ahora, ningún sistema de IA se mide con ninguna de estas listas, aunque algunos defensores de la IA general o a nivel humano (HLAI) insisten en que el trabajo continuo en tareas específicas (o en componentes individuales) no será suficiente para alcanzar la maestría en una amplia variedad de tareas, que necesitaremos un enfoque por completo nuevo. Nos parece que serán necesarios numerosos avances aunque, en general, la IA como campo ha hecho un intercambio razonable entre exploración/explotación, ensamblando un portafolio de componentes, mejorando en tareas particulares, mientras también explora ideas nuevas y a veces lejanas.

Habría sido un error decir a los hermanos Wright en 1903 que dejaran de trabajar en su avión de una sola pieza y diseñaran una máquina de "volar general artificial" que despegara de manera vertical, volara más rápido que el sonido, llevara a cientos de pasajeros y aterrizara en la luna. También habría sido un error seguir su primer vuelo con una competencia anual para hacer biplanos de madera de abeto cada vez mejores.

Hemos visto que el trabajo en componentes genera nuevas ideas; por ejemplo, las redes generativas adversarias (GANs) y los modelos de lenguaje transformador abrieron nuevas áreas de investigación. También hemos visto pasos hacia la "diversidad de comportamiento". Por ejemplo, los sistemas de traducción automática en la década de 1990 se construyeron uno a uno para cada par de idiomas (como del francés al inglés), pero hoy un solo sistema permite identificar el texto de entrada como uno de cien idiomas y traducirlo a cualquiera de los 100 idiomas objetivo. Otro sistema de lenguaje natural puede realizar cinco tareas distintas con un solo modelo conjunto (Hashimoto *et al.,* 2016).

Ingeniería de IA

El campo de la programación informática comenzó con unos pocos pioneros extraordinarios, pero sólo alcanzó el estatus de gran industria hasta que se desarrolló una práctica de ingeniería de software, con una poderosa colección de herramientas ampliamente disponibles y un próspero ecosistema de profesores, estudiantes, practicantes, emprendedores, inversores y clientes.

La industria de la IA aún no ha alcanzado ese nivel de madurez. Tenemos una variedad de herramientas y marcos de trabajo poderosos, como TensorFlow, Keras, PyTorch, CAFFE, Scikit-Learn y SciPy. Pero muchos de los enfoques más prometedores, como GANs y el aprendizaje profundo por refuerzo, han demostrado ser difíciles de manejar, ya que requieren experiencia y un grado de ajuste de entrenamiento adecuado a un nuevo dominio. No tenemos suficientes expertos para hacer esto en todos los dominios donde lo necesitamos, y aún no tenemos las herramientas y el ecosistema para permitir que los practicantes menos expertos tengan éxito.

Jeff Dean de Google ve un futuro en el que querremos que el aprendizaje automático maneje millones de tareas; no será factible desarrollar cada una de ellas desde cero, por lo que sugiere que en lugar de construir cada nuevo sistema desde cero, deberíamos comenzar con un solo sistema enorme y, para cada nueva tarea, extraer de él las partes relevantes para la tarea. Hemos visto algunos pasos en esta dirección, como los modelos de lenguaje transformador (por ejemplo, BERT, GPT-2) con miles de millones de parámetros, y una arquitectura de red neuronal "desmesuradamente grande" que escala hasta 68 mil millones de parámetros en un experimento (Shazeer *et al.,* 2017). Queda mucho trabajo por hacer.

El futuro

¿Hacia dónde irá el futuro? Los autores de ciencia ficción parecen favorecer los futuros distópicos sobre los utópicos, quizá porque hacen tramas más interesantes. Hasta ahora, la IA parece encajar con otras tecnologías revolucionarias poderosas como la impresión, la plomería, el viaje aéreo y la telefonía. Todas estas tecnologías han tenido impactos positivos, pero también tienen algunos efectos secundarios no deseados que impactan de manera desproporcionada a las clases desfavorecidas. Haríamos bien en invertir en minimizar los impactos negativos.

La IA también es diferente de las tecnologías revolucionarias anteriores. Mejorar la impresión, la plomería, el viaje aéreo y la telefonía a sus límites lógicos no producirá nada que amenace la supremacía humana en el mundo, pero mejorar la IA a su límite lógico podría.

En conclusión, la IA ha hecho grandes avances en su corta historia, pero la última frase del ensayo de Alan Turing (1950) sobre la *Maquinaria de Computación e Inteligencia* sigue siendo válida hoy en día:

> *Sólo podemos ver una corta distancia por delante,*
> *pero podemos ver que queda mucho por hacer.*

APÉNDICE A

ANTECEDENTES MATEMÁTICOS

A.1 Análisis de complejidad y notación O()

Los informáticos con frecuencia se enfrentan a la tarea de comparar algoritmos para ver qué tan rápido se ejecutan o cuánta memoria requieren. Hay dos enfoques para esta tarea. La primera es una **evaluación comparativa**: ejecutar los algoritmos en una computadora y medir la velocidad en segundos y el consumo de memoria en bytes. En definitiva, esto es lo que realmente importa, pero un punto de referencia puede ser insatisfactorio porque es muy específico: mide el rendimiento de un programa en particular escrito en un lenguaje particular, ejecutándose en una computadora en particular, con un compilador particular y datos de entrada particulares. A partir del resultado único que proporciona el banco de pruebas, puede ser difícil predecir qué tan bien funcionaría el algoritmo en un compilador, computadora o equipo diferente. El segundo enfoque se basa en un **análisis matemático de algoritmos**, de la implementación particular y los insumos, como se analiza a continuación.

Evaluación comparativa

Análisis matemático de algoritmos

A.1.1 Análisis asintótico

Consideraremos el análisis de algoritmos a través del siguiente ejemplo, un programa para calcular la suma de una secuencia de números:

```
función SUMMATION(secuencia) returns un número
    sum ← 0
    for i = 1 to LENGTH(sequence) do
        sum ← sum + sequence [i]
    returns sum
```

El primer paso en el análisis es abstraer sobre la entrada, con el fin de encontrar algún parámetro o parámetros que caractericen el tamaño de la entrada. En este ejemplo, la entrada puede ser caracterizada por la longitud de la secuencia, que llamaremos n. El segundo paso es abstraer la implementación para encontrar alguna medida que refleje el tiempo de ejecución del algoritmo, pero que no esté vinculada a un compilador o computadora en particular. Para el programa SUMMATION esto podría ser sólo el número de líneas de código ejecutadas o podría ser más detallado, midiendo el número de adiciones, asignaciones, referencias de matrices y ramas ejecutadas por el algoritmo. De cualquier manera, nos da una caracterización del número total de pasos dados por el algoritmo en función del tamaño de la entrada. Llamaremos a esta caracterización $T(n)$. Si contamos líneas de código, tenemos $T(n) = 2n + 2$ para nuestro ejemplo.

Si todos los programas fueran tan simples como SUMMATION, el análisis de algoritmos sería un campo trivial. Pero hay dos problemas que lo complican más. En primer lugar, es raro encontrar un parámetro como n que caracterice completamente el número de pasos dados por un algoritmo. En cambio, lo mejor que podemos hacer es calcular el peor caso $T_{worst}(n)$ o el caso promedio $T_{avg}(n)$. Calcular un promedio significa que el analista debe asumir cierta distribución de entradas.

El segundo problema es que los algoritmos tienden a resistirse al análisis exacto. En ese caso, es necesario recurrir a una aproximación. Decimos que el algoritmo SUMMATION es $O(n)$, lo que significa que su medida es como máximo una constante multiplicada por n, con la posible excepción de unos pocos valores pequeños de n. Más formalmente,

$T(n)$ es $O(f(n))$ si $T(n) \leq kf(n)$ para algunos k, para todos $n > n_0$.

Análisis asintótico

La notación $O()$ nos da lo que se llama un **análisis asintótico**. Podemos decir sin lugar a dudas que, como n se aproxima asintóticamente al infinito, un algoritmo $O(n)$ es mejor que un algoritmo $O(n^2)$. Una sola cifra de referencia no podría corroborar tal afirmación.

La notación $O()$ abstrae sobre factores constantes, lo que la hace más fácil de usar, pero menos precisa, que la notación $T()$. Por ejemplo, un algoritmo $O(n^2)$ siempre será peor que un $O(n)$ a largo plazo, pero si los dos algoritmos son $T(n^2 + 1)$ y $T(100n + 1000)$, entonces el $O(n^2)$ en realidad es mejor para $n < 110$.

A pesar de este inconveniente, el análisis asintótico es la herramienta más utilizada para analizar algoritmos. Precisamente porque el análisis abstrae tanto el número exacto de operaciones (ignorando el factor constante k) como el contenido exacto de la entrada (considerando sólo su tamaño n) el análisis se vuelve matemáticamente factible. La notación $O()$ es un buen compromiso entre precisión y facilidad de análisis.

A.1.2 NP y problemas inherentemente difíciles

El análisis de algoritmos y la notación $O()$ nos permiten hablar de la eficiencia de un algoritmo en particular. Sin embargo, no tienen nada que decir sobre si podría haber un algoritmo mejor para el problema en cuestión. El campo del **análisis de la complejidad** analiza problemas en lugar de algoritmos. La primera división bruta es entre problemas que se pueden resolver en tiempo polinómico y problemas que no se pueden resolver en tiempo polinómico, sin importar qué algoritmo se use. La clase de problemas polinómicos, aquellos que pueden resolverse en el tiempo $O(n^k)$ para algún k, se llama **P**. A veces se denominan problemas "fáciles", porque la clase contiene esos problemas con tiempos de ejecución como $O(\log n)$ y $O(n)$. Pero también contiene aquellos con tiempo $O(n^{1000})$, por lo que el nombre "fácil" no debe tomarse demasiado literalmente.

Análisis de complejidad

P

NP

Otra clase importante de problemas es **NP**, la clase de problemas polinómicos no deterministas. Un problema está en esta clase si hay algún algoritmo que pueda adivinar una solución y luego verificar si una suposición es correcta en tiempo polinómico. La idea es que si tiene un número arbitrariamente grande de procesadores para que pueda intentar todas las conjeturas a la vez, o si tienes mucha suerte y siempre aciertas la primera vez, entonces los problemas NP se convierten en problemas P. Una de las mayores preguntas abiertas en ciencias de la computación es si la clase NP es equivalente a la clase P cuando uno no tiene el lujo de un número infinito de procesadores o conjeturas omniscientes. La mayoría de los científicos de la computación están convencidos de que P /= NP; que los problemas de NP son inherentemente difíciles y no tienen algoritmos de tiempo polinómico. Pero esto nunca se ha demostrado.

Aquellos que están interesados en decidir si P = NP miran una subclase de NP llamada NP-problemas completos. La palabra "completo" se usa aquí en el sentido de "más extremo" y, por lo tanto, se refiere a los problemas más difíciles de la clase NP. Se ha comprobado que, o bien todos los problemas **NP-completos** están en P o ninguno de ellos lo está. Esto hace que la clase sea teóricamente interesante, pero también es de interés práctico porque se sabe que muchos problemas importantes son NP-completos. Un ejemplo es el problema de la satisfacción: dada una oración de lógica proposicional, ¿hay una asignación de valores de verdad a los símbolos de proposición de la oración que la hace verdadera? A menos que ocurra un milagro y P = NP, no puede haber

NP-completo

algoritmo que resuelve *todos* los problemas de satisfacibilidad en tiempo polinómico. Sin embargo, la IA está más interesada en saber si existen algoritmos que funcionen de manera eficiente en problemas *típicos* extraídos de una distribución predeterminada; como vimos en el Capítulo 7, hay algoritmos como WalkSat que funcionan bastante bien en muchos problemas.

NP-difíciles

La clase de problemas **NP-difíciles** consiste en aquellos problemas que son reducibles (en tiempo polinómico) a todos los problemas en NP, por lo que si resolviera cualquier problema NP-difícil, podría resolver todos los problemas en NP. Los problemas NP-completos son todos NP-difíciles, pero hay algunos problemas NP-difíciles que son incluso más difíciles que NP-completos.

Co-NP

La clase **co-NP** es el complemento de NP, en el sentido de que para cada problema de decisión en NP hay un problema correspondiente en co-NP con las respuestas "sí" y "no" invertidas. Sabemos que P es un subconjunto tanto de NP como de co-NP, y se cree que hay problemas en co-NP que no están en P. Los problemas **co-NP-completos** son los problemas más difíciles en co-NP.

Co-NP-completos

La clase #P (pronunciada "número P" según Garey y Johnson (1979), pero a menudo pronunciada "P aguda") es el conjunto de problemas de conteo correspondientes a los problemas de decisión en NP. Los problemas de decisión tienen una respuesta de sí o no: ¿hay una solución a esta fórmula 3-SAT? Los problemas de conteo tienen una respuesta entera: ¿cuántas soluciones hay para esta fórmula 3-SAT? En algunos casos, el problema del conteo es mucho más difícil que el problema de la decisión. Por ejemplo, decidir si un grafo bipartito tiene una coincidencia perfecta se puede hacer en el tiempo $O(VE)$ (donde el grafo tiene vértices V y aristas E), pero el problema de conteo "¿cuántas coincidencias perfectas tiene este grafo bipartito?" es #P completo, lo que significa que es difícil como cualquier problema en #P y, por lo tanto, al menos tan difícil como cualquier problema NP.

Otra clase es la clase de problemas PSPACE, aquellos que requieren una cantidad polinómica de espacio, incluso en una máquina no determinista. Se cree que los problemas difíciles de PSPACE son peores que los problemas NP-completos, aunque podría resultar que NP = PSPACE, al igual que podría resultar que P = NP.

A.2 Vectores, matrices y álgebra lineal

Vector

Aunque los matemáticos definen un **vector** como un miembro de un espacio vectorial, usaremos una definición más concreta: Un vector es una secuencia ordenada de valores. Por ejemplo, en el espacio bidimensional, tenemos vectores como $\mathbf{x}=\langle 3,4\rangle$ y $\mathbf{y}=\langle 0,2\rangle$. Seguimos la convención de caracteres en negrita para los nombres vectoriales, aunque algunos autores usan flechas o barras sobre los nombres: $\vec{x}$ o $\bar{y}$. Se puede acceder a los elementos de un vector mediante subíndices: $\mathbf{z}=\langle z_1, z_2, \ldots, z_n\rangle$. Un punto de confusión: este libro sintetiza el trabajo de muchos subcampos que llaman a sus secuencias vectores, listas o tuplas, y usan las notaciones $\langle 1, 2\rangle$, $[1, 2]$ o $(1, 2)$.

Las dos operaciones fundamentales en los vectores son la suma de vectores y la multiplicación escalar. La suma vectorial $\mathbf{x} + \mathbf{y}$ es la suma elemental: $\mathbf{x} + \mathbf{y} = \langle 3 + 0{,}4 + 2\rangle = \langle 3{,}6\rangle$. La multiplicación escalar multiplica cada elemento por una constante: $5\mathbf{x}=\langle 5 \times 3{,}5 \times 4\rangle = \langle 15{,}20\rangle$.

La longitud de un vector se denota $|\mathbf{x}|$ y se calcula sacando la raíz cuadrada de la suma de los cuadrados de los elementos: $|\mathbf{x}|=\sqrt{(3^2 + 4^2)} = 5$. El producto punto $\mathbf{x} \cdot \mathbf{y}$ (también llamado producto escalar) de dos vectores es la suma de los productos de los elementos correspondientes; es decir, $\mathbf{x} \cdot \mathbf{y} = \sum_i x_i y_i$, o en nuestro caso particular, $\mathbf{x} \cdot \mathbf{y} = 3 \times 0 + 4 \times 2 = 8$.

Los vectores a menudo se interpretan como segmentos de línea dirigidos (flechas) en un espacio euclidiano n-dimensional. La suma de vectores es entonces equivalente a colocar la cola de un vector en la cabeza del otro, y el producto escalar $\mathbf{x}\cdot\mathbf{y}$ es $|\mathbf{x}||\mathbf{y}|$ cos θ, donde θ es el ángulo entre $\mathbf{x}$ y $\mathbf{y}$.

Matriz

Una **matriz** consiste en una formación rectangular de valores organizados en filas y columnas. Aquí hay una matriz **A** de tamaño 3 × 4:

$$\begin{pmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} & \mathbf{A}_{1,3} & \mathbf{A}_{1,4} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} & \mathbf{A}_{2,3} & \mathbf{A}_{2,4} \\ \mathbf{A}_{3,1} & \mathbf{A}_{3,2} & \mathbf{A}_{3,3} & \mathbf{A}_{3,4} \end{pmatrix}$$

El primer índice de $\mathbf{A}_{i,j}$ especifica la fila y el segundo la columna. En los lenguajes de programación, $\mathbf{A}_{i,j}$ a menudo se escribe `A[i,j]` o `A[i][j]`.

La suma de dos matrices se define sumando sus elementos correspondientes; por ejemplo $(\mathbf{A} + \mathbf{B})_{i,j} = \mathbf{A}_{i,j} + \mathbf{B}_{i,j}$. (La suma no está definida si **A** y **B** tienen tamaños diferentes). También podemos definir la multiplicación de una matriz por un escalar: $(c\mathbf{A})_{i,j} = c\mathbf{A}_{i,j}$. La multiplicación de matrices (el producto de dos matrices) es más complicada. El producto **AB** se define sólo si **A** es de tamaño $a \times b$ y **B** es de tamaño $b \times c$ (es decir, la segunda matriz tiene el mismo número de filas que la primera tiene columnas), el resultado es una matriz de tamaño $a \times c$. Si las matrices tienen el tamaño adecuado, el resultado es

$$(\mathbf{AB})_{i,k} = \sum_j \mathbf{A}_{i,j}\mathbf{B}_{j,k.}$$

La multiplicación de matrices no es conmutativa, ni siquiera para matrices cuadradas: $\mathbf{AB} \neq \mathbf{BA}$ en general. Es, sin embargo, asociativo: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$. Observe que el producto escalar se puede expresar en términos de una transpuesta y una multiplicación de matrices: $\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y}$.

Matriz de identidad
Transponer
Inverso
Singular

La **matriz identidad I** tiene elementos $\mathbf{I}_{i,j}$ igual a 1 cuando $i = j$ e igual a 0 en caso contrario. Tiene la propiedad de que $\mathbf{AI} = \mathbf{A}$ para todos los **A**. La **transpuesta** de **A**, escrita $\mathbf{A}^\top$ se forma convirtiendo filas en columnas y viceversa o, más formalmente, por $\mathbf{A}^\top_{i,j} = \mathbf{A}_{j,i}$. El **inverso** de una matriz cuadrada **A** es otra matriz cuadrada $\mathbf{A}^{-1}$ tal que $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Para una matriz **singular**, la inversa no existe. Para una matriz no singular, se puede calcular en tiempo $O(n^3)$.

Las matrices se utilizan para resolver sistemas de ecuaciones lineales en tiempo $O(n^3)$; el tiempo se domina invirtiendo una matriz de coeficientes. Considere el siguiente conjunto de ecuaciones, para las cuales queremos una solución en x, y y z:

$$\begin{aligned} +2x+y-z &= 8 \\ -3x-y+2z &= -11 \\ -2x+y+2z &= -3. \end{aligned}$$

Podemos representar este sistema como la ecuación matricial $\mathbf{Ax} = \mathbf{b}$, donde

$$\mathbf{A} = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}, \qquad \mathbf{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}.$$

Para resolver $\mathbf{Ax} = \mathbf{b}$ multiplicamos ambos lados por $\mathbf{A}^{-1}$, obteniendo $\mathbf{A}^{-1}\mathbf{Ax} = \mathbf{A}^{-1}\mathbf{b}$, que se simplifica a $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. Después de invertir **A** y multiplicar por **b**, obtenemos la respuesta

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ -1 \end{pmatrix}.$$

Algunos puntos misceláneos más: usamos $\log(x)$ para el logaritmo natural, $\log_e(x)$. Usamos $\operatorname{argmax}_x f(x)$ para el valor de x para el cual $f(x)$ es máximo.

A.3 Distribuciones de probabilidad

Una probabilidad es una medida sobre un conjunto de eventos que satisface tres axiomas:

1. La medida de cada evento está entre 0 y 1. Escribimos esto como $0 \leq P(X = x_i) \leq 1$, donde X es una variable aleatoria que representa un evento y x_i son los valores posibles de X. En general, las variables aleatorias se indican con letras mayúsculas y sus valores con letras minúsculas.
2. La medida de todo el conjunto es 1; es decir $\sum_{i=1}^{n} P(X = x_i) = 1$.
3. La probabilidad de una unión de sucesos disjuntos es la suma de las probabilidades de los sucesos individuales; es decir, $P(X = x_1 \vee X = x_2) = P(X = x_1) + P(X = x_2)$, en el caso de que x_1 y x_2 sean disjuntos.

Un **modelo probabilístico** consiste en un espacio muestral de posibles resultados mutuamente excluyentes, junto con una medida de probabilidad para cada resultado. Por ejemplo, en un modelo del clima de mañana, los resultados podrían ser *sol, nubes, lluvia* y *nieve*. Un subconjunto de estos resultados constituye un evento. Por ejemplo, el evento de precipitación es el subconjunto que consiste en {*lluvia, nieve*}.

Usamos $\mathbf{P}(X)$ para denotar el vector de valores $\langle P(X = x_1),...,P(X = x_n)\rangle$. También usamos $P(x_i)$ como abreviatura de $P(X = x_i)$ y $\sum_x P(x)$ para $\sum_{i=1}^{n} P(X = x_i)$.

La probabilidad condicional $P(B|A)$ se define como $P(B \cap A)/P(A)$. A y B son condicionalmente independientes si $P(B|A) = P(B)$ (o equivalentemente, $P(A|B) = P(A)$).

Para las variables continuas, hay un número infinito de valores y, a menos que haya picos de puntos, la probabilidad de cualquier valor exacto es 0. Así que tiene más sentido hablar sobre el valor que está dentro de un rango. Lo hacemos con una **función de densidad de probabilidad**, que tiene un significado ligeramente diferente al de la función de probabilidad discreta. Dado que $P(X = x)$, la probabilidad de que X tenga el valor x exactamente, es cero, medimos la probabilidad de que X caiga en un intervalo alrededor de x, en comparación con el ancho del intervalo, y tomamos el límite cuando el ancho del intervalo tiende a cero:

Función de densidad de probabilidad

$$P(x) = \lim_{dx \to 0} P(x \leq X \leq x + dx)/dx.$$

La función de densidad debe ser no negativa para todo x y debe tener

$$\int_{-\infty}^{\infty} P(x)dx = 1.$$

También podemos definir la **distribución acumulada** $F_X(x)$, que es la probabilidad de una variable menor que x:

Distribución acumulada

$$F_X(x) = P(X \leq x) = \int_{-\infty}^{x} P(u)du.$$

Tenga en cuenta que la función de densidad de probabilidad tiene unidades, mientras que la función de probabilidad discreta no tiene unidades. Por ejemplo, si los valores de X se miden en segundos, la densidad se mide en Hz (es decir, 1/seg). Si los valores de $\mathbf{X}$ son puntos en el espacio tridimensional medidos en metros, entonces la densidad se mide en $1/m^3$.

Una de las distribuciones de probabilidad más importantes es la **distribución gaussiana**, también conocida como la **distribución normal**. Usamos la notación $\mathcal{N}(x;\mu,\sigma^2)$ para la distribución

Distribución gaussiana

que es una función de x con media μ y desviación estándar σ (y, por lo tanto, varianza σ^2). Se define como:

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)},$$

Distribución normal estándar

donde x es una variable continua que va de $-\infty$ a $+\infty$. Con media $\mu = 0$ y varianza $\sigma^2 = 1$, obtenemos el caso especial de la **distribución normal estándar**. Para una distribución sobre un vector x en n dimensiones, existe la distribución **gaussiana multivariante**:

Gaussiana multivariante

$$\mathcal{N}(\mathbf{x};\mu,\Sigma) = \frac{1}{\sqrt{(2\pi)^n|\Sigma|}} e^{-\frac{1}{2}(\mathbf{x}-\mu)^T\Sigma^{-1}(\mathbf{x}-\mu)},$$

donde μ es el vector medio y Σ es la **matriz de covarianza** (ver más abajo). La distribución acumulativa para una distribución normal univariante viene dada por

$$F(x) = \int_{-\infty}^{x} \mathcal{N}(z;\mu,\sigma^2)dz = \frac{1}{2}\left(1+\text{erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right)\right),$$

donde erf(x) es la llamada **función de error**, que no tiene representación de forma cerrada.

Teorema del límite central

El **teorema del límite central** establece que la distribución formada por el muestreo de n variables aleatorias independientes y tomando su media tiende a una distribución normal como n tiende a infinito. Esto es válido para casi cualquier colección de variables aleatorias, incluso si no son estrictamente independientes, a menos que la varianza de cualquier subconjunto finito de variables domine a las demás.

Expectativa

La **expectativa** de una variable aleatoria, $E(X)$, es el valor medio o promedio, ponderado por la probabilidad de cada valor. Para una variable discreta es:

$$E(X) = \sum_i x_i P(X = x_i).$$

Para una variable continua, reemplace la suma con una integral y use la función de densidad de probabilidad, $P(x)$:

$$E(X) = \int_{-\infty}^{\infty} xP(x)dx.$$

Para cualquier función f, también tenemos

$$E(f(X)) = \int_{-\infty}^{\infty} f(x)P(x)dx.$$

Finalmente, cuando sea necesario, se puede especificar la distribución de la variable aleatoria como un subíndice para el operador de expectativa:

$$E_{X\sim Q(x)}(g(X)) = \int_{-\infty}^{\infty} g(x)Q(x)dx.$$

Varianza

Además de la expectativa, otras propiedades estadísticas importantes de una distribución incluyen la **varianza**, que es el valor esperado del cuadrado de la diferencia con respecto a la media, μ, de la distribución:

$$Var(X) = E((X-\mu)^2)$$

Desviación estándar

y la **desviación estándar**, que es la raíz cuadrada de la varianza.

La **raíz cuadrática media (RMS)** de un conjunto de valores (a menudo muestras de una variable aleatoria) es la raíz cuadrada de la media de los cuadrados de los valores,

$$RMS(x_1,\ldots,x_n) = \sqrt{\frac{x_1^2 + \cdots + x_n^2}{n}}$$

La **covarianza** de dos variables aleatorias es la expectativa del producto de sus diferencias de sus medios: Covarianza

$$\text{cov}(X,Y) = E((X - \mu_X)(Y - \mu_Y)).$$

La **matriz de covarianzas**, a menudo denotada Σ, es una matriz de covarianzas entre elementos de una matriz de vector de variables aleatorias. Dado $\mathbf{X} = \langle X_1,\ldots X_n\rangle^\top$, las entradas de la matriz de covarianza son las siguientes: Matriz de covarianzas

$$\Sigma_{i,j} = \text{cov}(X_i,X_j) = E((X_i - \mu_i)(X_j - \mu_j)).$$

Decimos que **muestreamos** a partir de una distribución de probabilidad, cuando elegimos un valor al azar. Nosotros no sabemos lo que traerá una selección, pero en el límite, una gran colección de muestras se acercará a la misma función de densidad de probabilidad que la distribución de la que se muestrea. La **distribución uniforme** es aquella en la que todos los elementos son igualmente (uniformemente) probables. Por lo tanto, cuando decimos que "muestreamos uniformemente (al azar) de los enteros del 0 al 99", lo que significa que es igualmente probable que elijamos cualquier número entero en ese rango. Muestreo Distribución uniforme

Notas bibliográficas e históricas

La notación $O()$, tan ampliamente utilizada en la informática actual, fue introducida por primera vez en el contexto de la teoría de números por el matemático P. G. H. Bachmann (1894). El concepto de NP-completitud fue inventado por Cook (1971), y el método moderno para establecer una reducción de un problema a otro se debe a Karp (1972). Cook y Karp han ganado el premio Turing por su trabajo.

Los libros de texto sobre el análisis y diseño de algoritmos incluyen Sedgewick y Wayne (2011) y Cormen, Leiserson, Rivest y Stein (2009). Estos libros ponen énfasis en el diseño y análisis de algoritmos para resolver problemas manejables. Para la teoría de la completitud NP y otras formas de intratabilidad, véase Garey y Johnson (1979) o Papadimitriou (1994). Entre los buenos textos sobre probabilidad se encuentran Chung (1979), Ross (2015) y Bertsekas y Tsitsiklis (2008).

APÉNDICE B

NOTAS SOBRE LENGUAJES Y ALGORITMOS

B.1 Definición de idiomas con Backus-Naur Form (BNF)

En este libro definimos varios lenguajes, incluidos los lenguajes de la lógica proposicional (página 217), la lógica de primer orden (página 258) y un subconjunto del inglés (página 842). Un lenguaje formal se define como un conjunto de cadenas donde cada cadena es una secuencia de símbolos. Los lenguajes que nos interesan consisten en un conjunto infinito de cadenas, por lo que necesitamos una forma concisa de caracterizar el conjuntoy lo cual hacemos con una **gramática**. El tipo particular de gramática que usamos se llama **gramática libre de contexto**, esto se debe a que cada expresión tiene la misma forma en cualquier contexto. Escribimos nuestras gramáticas en un formalismo llamado **forma Backus-Naur (BNF)**. Hay cuatro componentes en una gramática BNF:

Gramática libre de contexto

Forma Backus-Naur (BNF)

Símbolo de terminal

Símbolo no terminal

Símbolo de inicio

Reglas de reescritura

- Conjunto de **símbolos de terminal**. Estos son los símbolos o palabras que componen las cadenas del idioma. Podrían ser letras (**A**, **B**, **C**, ...) o palabras (**a, oso hormiguero, ábaco,** ...), o cualquier símbolo que sea apropiado para el dominio.
- Conjunto de **símbolos no terminales** que categorizan las subfrases del lenguaje. Por ejemplo, el símbolo no terminal *NounPhrase* en inglés denota un conjunto infinito de cadenas que incluyen "you" y "the big sbabbery dog".
- Un **símbolo de inicio**, que es el símbolo no terminal que denota el conjunto completo de cadenas del lenguaje. En inglés, es *Sentence*; para aritmética, podría ser *Expr* y para lenguajes de programación es *Program*.
- Conjunto de **reglas de reescritura**, de la forma $LHS \rightarrow RHS$, donde *LHS* es un símbolo no terminal y *RHS* es una secuencia de cero o más símbolos. Estos pueden ser símbolos terminales o no terminales, o el símbolo ϵ, que se usa para denotar la cadena vacía.

Una regla de reescritura del formulario

$$Sentencia \rightarrow SustantivoFrase\ VerboFrase$$

significa que siempre que tengamos dos cadenas categorizadas como *NounPhrase* y *VerbPhrase*, podemos anexarlas juntas y categorizar el resultado como una *Sentencia*. Como abreviatura, las dos reglas ($S \rightarrow A$) y ($S \rightarrow B$) se pueden escribir ($S \rightarrow A|B$). Para ilustrar estos conceptos, aquí hay una gramática BNF para expresiones aritméticas simples:

$$\begin{array}{lcl} Expr & \rightarrow & Expr\ Operator\ Expr \mid (Expr) \mid Number \\ Number & \rightarrow & Digit \mid Number\ Digit \\ Digit & \rightarrow & \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9} \\ Operator & \rightarrow & + \mid - \mid \div \mid \times \end{array}$$

Trataremos los idiomas y las gramáticas con más detalle en el capítulo 23. Tenga en cuenta que otros libros usan notaciones ligeramente diferentes para BNF; por ejemplo, es posible que vea ⟨*Digit*⟩ en lugar de *Digit* para un terminal que no es terminal, 'word' en lugar de **word** para un terminal o ::= en lugar de → en una regla.

B.2 Descripción de algoritmos con pseudocódigo

Pseudocódigo

Los algoritmos de este libro se describen en **pseudocódigo**. La mayor parte del pseudocódigo debe ser familiar para los programadores que usan lenguajes como Java, C++ o especialmente Python. En algunos lugares usamos fórmulas matemáticas o inglés ordinario para describir partes que de otro modo serían más engorrosas. Cabe señalar algunas idiosincrasias:

- **Variables persistentes:** Usamos la palabra clave **persistent** para decir que a una variable se le da un valor inicial la primera vez que se llama a una función y conserva ese valor (o el valor que se le da por una instrucción de asignación posterior) en todas las llamadas posteriores a la función. Por lo tanto, las variables persistentes son como las variables globales en el sentido de que sobreviven a una sola llamada a su función; pero sólo son accesibles dentro de la función. Los programas de agente del libro utilizan variables persistentes para la *memoria*. Los programas con variables persistentes se pueden implementar como *objetos* en lenguajes orientados a objetos como C++, Java, Python y Smalltalk. En los lenguajes funcionales, se pueden implementar mediante *cierres funcionales* sobre un entorno que contiene las variables requeridas.
- **Funciones como valores:** Las funciones tienen nombres en mayúsculas y las variables tienen nombres en minúsculas y cursivas. Por lo tanto, la mayoría de las veces, una llamada de función se parece a FN(*x*). Sin embargo, permitimos que el valor de una variable sea una función; por ejemplo, si el valor de la variable *f* es la función de raíz cuadrada, entonces *f*(9) devuelve 3.
- **La sangría es significativa**: La sangría se usa para marcar el alcance de un bucle o condición, como en los lenguajes Python y CoffeeScript, y, a diferencia de Java, C++ y Go (que usan llaves) o Lua y Ruby (que usan **end**).
- **Asignación de desestructuración:** La notación "*x,y* ← *par*" significa que el lado derecho debe evaluarse como una colección de dos elementos, y el primer elemento se asigna a *x* y el segundo a *y*. La misma idea se usa en "**para** *x*, *y* en *pares* **do**" y se puede usar para intercambiar dos variables: "*x,y* ← *y,x*"
- **Valores predeterminados para los parámetros:** La notación "**función** F(*x,y*=0) **devuelve** un número" significa que *y* es un argumento opcional con valor predeterminado 0; es decir, las llamadas F(3,0) y F(3) son equivalentes.
- **yield:** una función que contiene la palabra clave **yield** es un **generador** de una secuencia de valores, una cada vez que se encuentre la expresión de rendimiento. Después de producir, la función continúa ejecutándose con la siguiente instrucción. Los lenguajes Python, Ruby, C# y Javascript (ECMAScript) tienen esta misma característica.

  Generador
- **Bucles:** Hay cuatro tipos de bucles:
  - "**for** *x* **in** *c* **do**" ejecuta el bucle con la variable *x* enlazada a elementos sucesivos de la colección *c*.
  - "**for** *i* = 1 to *n* **do**" ejecuta el bucle con *i* enlazado a enteros sucesivos de 1 a *n* incluido.
  - "**while** *condition* **do**" significa que la condición se evalúa antes de cada iteración del bucle y que el bucle se cierra si la condición es falsa.

  - "**repite** ...**until** *condition*" significa que el bucle se ejecuta incondicionalmente la primera vez, luego se evalúa la condición y el bucle se cierra si la condición es verdadera; de lo contrario, el bucle sigue ejecutándose (y probando al final).
- **Listas:** [*x,y,z*] denota una lista de tres elementos. El operador "+" concatena listas: [1,2] + [3,4] = [1,2,3,4]. Una lista se puede usar como una pila: POP elimina y devuelve el último elemento de una lista, TOP devuelve el último elemento.
- **Conjuntos:** {*x,y,z*} denota un conjunto de tres elementos. {$x : p(x)$} denota el conjunto de todos los elementos x para el cual $p(x)$ es verdadero.
- **Las matrices comienzan en 1:** el primer índice de una matriz es 1 como en la notación matemática habitual (y en R y Julia), no 0 (como en Python, Java y C).

B.3 Material complementario online

El libro tiene un sitio web con material complementario, instrucciones para enviar sugerencias y oportunidades para unirse a las listas de discusión:

- `aima.cs.berkeley.edu`

Los algoritmos del libro y múltiples ejercicios de programación adicionales han sido implementados en Python y Java (y algunos en otros lenguajes) en el repositorio de código online, accesible desde el sitio web y actualmente alojado en:

- `github.com/aimacode`

Bibliografía

Las siguientes abreviaturas se utilizan para las conferencias y revistas citadas con frecuencia:

AAAI	Proceedings of the AAAI Conference on Artificial Intelligence
AAMAS	Proceedings of the International Conference on Autonomous Agents and Multiagent Systems
ACL	Proceedings of the Annual Meeting of the Association for Computational Linguistics
AIJ	Artificial Intelligence (Journal)
AIMag	AI Magazine
AIPS	Proceedings of the International Conference on AI Planning Systems
AISTATS	Proceedings of the International Conference on Artificial Intelligence and Statistics
BBS	Behavioral and Brain Sciences
CACM	Communications of the Association for Computing Machinery
COGSCI	Proceedings of the Annual Conference of the Cognitive Science Society
COLING	Proceedings of the International Conference on Computational Linguistics
COLT	Proceedings of the Annual ACM Workshop on Computational Learning Theory
CP	Proceedings of the International Conference on Principles and Practice of Constraint Programming
CVPR	Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition
EC	Proceedings of the ACM Conference on Electronic Commerce
ECAI	Proceedings of the European Conference on Artificial Intelligence
ECCV	Proceedings of the European Conference on Computer Vision
ECML	Proceedings of the The European Conference on Machine Learning
ECP	Proceedings of the European Conference on Planning
EMNLP	Proceedings of the Conference on Empirical Methods in Natural Language Processing
FGCS	Proceedings of the International Conference on Fifth Generation Computer Systems
FOCS	Proceedings of the Annual Symposium on Foundations of Computer Science
GECCO	Proceedings of the Genetics and Evolutionary Computing Conference
HRI	Proceedings of the International Conference on Human-Robot Interaction
ICAPS	Proceedings of the International Conference on Automated Planning and Scheduling
ICASSP	Proceedings of the International Conference on Acoustics, Speech, and Signal Processing
ICCV	Proceedings of the International Conference on Computer Vision
ICLP	Proceedings of the International Conference on Logic Programming
ICLR	Proceedings of the International Conference on Learning Representations
ICML	Proceedings of the International Conference on Machine Learning
ICPR	Proceedings of the International Conference on Pattern Recognition
ICRA	Proceedings of the IEEE International Conference on Robotics and Automation
ICSLP	Proceedings of the International Conference on Speech and Language Processing
IJAR	International Journal of Approximate Reasoning
IJCAI	Proceedings of the International Joint Conference on Artificial Intelligence
IJCNN	Proceedings of the International Joint Conference on Neural Networks
IJCV	International Journal of Computer Vision
ILP	Proceedings of the International Workshop on Inductive Logic Programming
IROS	Proceedings of the International Conference on Intelligent Robots and Systems
ISMIS	Proceedings of the International Symposium on Methodologies for Intelligent Systems
ISRR	Proceedings of the International Symposium on Robotics Research
JACM	Journal of the Association for Computing Machinery
JAIR	Journal of Artificial Intelligence Research
JAR	Journal of Automated Reasoning
JASA	Journal of the American Statistical Association
JMLR	Journal of Machine Learning Research
JSL	Journal of Symbolic Logic
KDD	Proceedings of the International Conference on Knowledge Discovery and Data Mining
KR	Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning
LICS	Proceedings of the IEEE Symposium on Logic in Computer Science
NeurIPS	Advances in Neural Information Processing Systems
PAMI	IEEE Transactions on Pattern Analysis and Machine Intelligence
PNAS	Proceedings of the National Academy of Sciences of the United States of America
PODS	Proceedings of the ACM International Symposium on Principles of Database Systems
RSS	Proceedings of the Conference on Robotics: Science and Systems
SIGIR	Proceedings of the Special Interest Group on Information Retrieval
SIGMOD	Proceedings of the ACM SIGMOD International Conference on Management of Data
SODA	Proceedings of the Annual ACM–SIAM Symposium on Discrete Algorithms
STOC	Proceedings of the Annual ACM Symposium on Theory of Computing
TARK	Proceedings of the Conference on Theoretical Aspects of Reasoning about Knowledge
UAI	Proceedings of the Conference on Uncertainty in Artificial Intelligence

Aaronson, S. (2014). My conversation with "Eugene Goostman", el chatbot que está en todas las noticias por supuestamente pasar la prueba de Turing. Shtetl-Optimized, www.scottaaronson.com/blog/?p=1858.

Aarts, E. y Lenstra, J. K. (2003). *Local Search in Combinatorial Optimization*. Princeton University Press.

Aarup, M., Arentoft, M. M., Parrod, Y., Stader, J. y Stokes, I. (1994). OPTIMUM-AIV: A knowledgebased planning and scheduling system for spacecraft AIV. En Fox, M. y Zweben, M. (Eds*.), Knowledge Based Scheduling*. Morgan Kaufmann.

Abbas, A. (2018). *Foundations of Multiattribute Utility*. Cambridge University Press.

Abbeel, P. y Ng, A. Y. (2004). Apprenticeship learning via inverse reinforcement learning. En *ICML-04*.

Abney, S., McAllester, D. A., y Pereira, F. (1999). Relating probabilistic grammars and automata. En *ACL-99*.

Abramson, B. (1987). *The expected-outcome model of two-player games*. Ph.D. thesis, Columbia University.

Abramson, B. (1990). Expected-outcome: A general model of static evaluation. *PAMI, 12*, 182–193.

Abreu, D. y Rubinstein, A. (1988). The structure of Nash equilibrium in repeated games with finite automata. *Econometrica, 56,* 1259–1281.

Achlioptas, D. (2009). Random satisfiability. En Biere, A., Heule, M., Van Maaren, H., y Walsh, T. (Eds.), *Handbook of Satisfiability*. IOS Press.

Ackerman, E. y Guizzo, E. (2016). The next generation of Boston Dynamics' Atlas robot is quiet, robust, and tether free. *IEEE Spectrum, 24*, 2016.

Ackerman, N., Freer, C. y Roy, D. (2013). On the computability of conditional probability. arXiv 1005.3014.

Ackley, D. H. y Littman, M. L. (1991). Interactions between learning and evolution. En Langton, C., Taylor, C., Farmer, J. D., y Rasmussen, S. (Eds.), *Artificial Life II*. Addison-Wesley.

Adida, B. y Birbeck,M. (2008). RDFa primer. Tech. rep., W3C.

Adolph, K. E., Kretch, K. S. y LoBue, V. (2014). Fear of heights in infants? *Current Directions in Psychological Science, 23*, 60–66.

Agerbeck, C. y Hansen, M. O. (2008). A multiagent approach to solving NP-complete problems. Master's thesis, Technical Univ. of Denmark.

Aggarwal, G., Goel, A., y Motwani, R. (2006). Truthful auctions for pricing search keywords. En *EC-06*.

Agha, G. (1986). *ACTORS: A Model of Concurrent Computation in Distributed Systems*. MIT Press.

Agichtein, E. y Gravano, L. (2003). Querying text databases for efficient information extraction. En *Proc. IEEE Conference on Data Engineering*.

Agmon, S. (1954). The relaxation method for linear inequalities. *Canadian Journal of Mathematics, 6*, 382–392.

Agostinelli, F., McAleer, S., Shmakov, A. y Baldi, P. (2019). Solving the Rubik's Cube with deep reinforcement learning and search. *Nature Machine Intelligence, 1*, 356–363.

Agrawal, P., Nair, A. V., Abbeel, P., Malik, J. y Levine, S. (2017). Learning to poke by poking: Experiential learning of intuitive physics. En *NeurIPS 29*.

Agre, P. E. y Chapman, D. (1987). Pengi: an implementation of a theory of activity. En *IJCAI-87*.

Aizerman, M., Braverman, E. y Rozonoer, L. (1964). Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control, 25*, 821–837.

Akametalu, A. K., Fisac, J. F., Gillula, J. H., Kaynama, S., Zeilinger, M. N. y Tomlin, C. J. (2014). Reachability-based safe learning with Gaussian processes. En *53rd IEEE Conference on Decision and Control*.

Akgun, B., Cakmak, M., Jiang, K. y Thomaz, A. (2012). Keyframe-based learning from demonstration. *International Journal of Social Robotics, 4*, 343–355.

Aldous, D. y Vazirani, U. (1994). "Go with the winners" algorithms. En *FOCS-94*.

Alemi, A. A., Chollet, F., Een, N., Irving, G., Szegedy, C. y Urban, J. (2017). DeepMath - Deep sequence models for premise selection. En *NeurIPS 29*.

Allais, M. (1953). Le comportment de l'homme rationnel devant la risque: critique des postulats et axiomes de l''ecole Américaine. *Econometrica, 21*, 503–546.

Allan, J., Harman, D., Kanoulas, E., Li, D., Van Gysel, C. y Vorhees, E. (2017). Trec 2017 common core track overview. En *Proc. TREC*.

Allen, J. F. (1983). Maintaining knowledge about temporal intervals. *CACM, 26*, 832–843.

Allen, J. F. (1984). Towards a general theory of action and time. *AIJ, 23*, 123–154.

Allen, J. F. (1991). Time and time again: The many ways to represent time. *Int. J. Intelligent Systems, 6*, 341–355.

Allen, J. F., Hendler, J. y Tate, A. (Eds.). (1990). *Readings in Planning*. Morgan Kaufmann.

Allen, P. y Greaves, M. (2011). The singularity isn't near. *Technology review, 12*, 7–8.

Allen-Zhu, Z., Li, Y. y Song, Z. (2018). A convergence theory for deep learning via overparameterization. arXiv:1811.03962.

Alterman, R. (1988). Adaptive planning. *Cognitive Science, 12*, 393–422.

Amarel, S. (1967). An approach to heuristic problemsolving and theorem proving in the propositional calculus. En Hart, J. y Takasu, S. (Eds.), *Systems and Computer Science*. University of Toronto Press.

Amarel, S. (1968). On representations of problems of reasoning about actions. En Michie, D. (Ed.), *Machine Intelligence 3*, Vol. 3. Elsevier.

Amir, E. y Russell, S. J. (2003). Logical filtering. En *IJCAI-03*.

Amit, Y. y Geman, D. (1997). Shape quantization and recognition with randomized trees. *Neural Computation, 9*, 1545–1588.

Amodei, D. y Hernandez, D. (2018). AI and compute. OpenAI blog, blog.openai.com/ai-and-compute/.

Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J. y Mané, D. (2016). Concrete problems in AI safety. arXiv:1606.06565.

Andersen, S. K., Olesen, K. G., Jensen, F. V. y Jensen, F. (1989). HUGIN—A shell for building Bayesian belief universes for expert systems. En *IJCAI-89*.

Anderson, J. R. (1980). *Cognitive Psychology and Its Implications*. W. H. Freeman.

Anderson, J. R. (1983). *The Architecture of Cognition*. Harvard University Press.

Anderson, K., Sturtevant, N. R., Holte, R. C. y Schaeffer, J. (2008). Coarse-to-fine search techniques. Tech. rep., University of Alberta.

Andoni, A. y Indyk, P. (2006). Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. En *FOCS-06*.

Andor, D., Alberti, C.,Weiss, D., Severyn, A., Presta, A., Ganchev, K., Petrov, S. y Collins, M. (2016). Globally normalized transition-based neural networks. arXiv:1603.06042.

Andre, D., Friedman, N. y Parr, R. (1998). Generalized prioritized sweeping. En *NeurIPS 10*.

Andre, D. y Russell, S. J. (2002). State abstraction for programmable reinforcement learning agents. En *AAAI-02*.

Andreae, P. (1985). *Justified Generalisation: Learning Procedures from Examples*. Ph.D. thesis, MIT.

Andrieu, C., Doucet, A. y Holenstein, R. (2010). Particle Markov chain Monte Carlo methods. *J. Royal Statistical Society, 72*, 269–342.

Andrychowicz, M., Baker, B., Chociej, M., Jozefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A. *et al*. (2018a). Learning dexterous in-hand manipulation. arXiv:1808.00177.

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P. y Zaremba, W. (2018b). Hindsight experience replay. En *NeurIPS 30*.

Aneja, J., Deshpande, A. y Schwing, A. (2018). Convolutional image captioning. En *CVPR-18*.

Aoki, M. (1965). Optimal control of partially observable Markov systems. *J. Franklin Institute*, 280, 367–386.

Appel, K. y Haken, W. (1977). Every planar map is four colorable: Part I: Discharging. *Illinois J. Math., 21*, 429–490.

Appelt, D. (1999). Introduction to information extraction. *AI Communications, 12*, 161–172.

Apt, K. R. (1999). The essence of constraint propagation. *Theoretical Computer Science, 221*, 179–210.

Apt, K. R. (2003). *Principles of Constraint Programming*. Cambridge University Press.

Apté, C., Damerau, F. y Weiss, S. (1994). Automated learning of decision rules for text categorization. *ACM Transactions on Information Systems, 12*, 233–251.

Arbuthnot, J. (1692). *Of the Laws of Chance*. Motte, London. Translation into English, with additions, of Huygens (1657).

Archibald, C., Altman, A. y Shoham, Y. (2009). Analysis of a winning computational billiards player. En *IJCAI-09*.

Arfaee, S. J., Zilles, S. y Holte, R. C. (2010). Bootstrap learning of heuristic functions. En *Third Annual Symposium on Combinatorial Search*.

Argall, B. D., Chernova, S., Veloso, M. y Browning, B. (2009). A survey of robot learning from demonstration. *Robotics and autonomous systems, 57*, 469–483.

Ariely, D. (2009). *Predictably Irrational* (Revised edition). Harper.

Arkin, R. (1998). *Behavior-Based Robotics*. MIT Press.

Arkin, R. (2015). The case for banning killer robots: Counterpoint. *CACM, 58*.

Armando, A., Carbone, R., Compagna, L., Cuellar, J. y Tobarra, L. (2008). Formal analysis of SAML 2.0 web browser single sign-on: Breaking the SAMLbased single sign-on for Google apps. En *Proc. 6th ACM Workshop on Formal Methods in Security Engineering*.

Armstrong, S. y Levinstein, B. (2017). Low impact artificial intelligences. arXiv:1705.10720.

Arnauld, A. (1662). *La logique, ou l'art de penser*. Chez Charles Savreux, Paris.

Arora, N. S., Russell, S. J. y Sudderth, E. (2013). NET-VISA: Network processing vertically integrated seismic analysis. *Bull. Seism. Soc. Amer., 103*, 709–729.

Arora, S. (1998). Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems. *JACM, 45*, 753–782.

Arpit, D., Jastrzebski, S., Ballas, N., Krueger, D., Bengio, E., Kanwal, M. S., Maharaj, T., Fischer, A., Courville, A., Bengio, Y. y Lacoste-Julien, S. (2017). A closer look at memorization in deep networks. arXiv:1706.05394.

Arrow, K. J. (1951). *Social Choice and Individual Values*. Wiley.

Arulampalam, M. S., Maskell, S., Gordon, N. y Clapp, T. (2002). A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing, 50*, 174–188.

Arulkumaran, K., Deisenroth, M. P., Brundage, M. y Bharath, A. A. (2017). Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine, 34*, 26–38.

Arunachalam, R. y Sadeh, N. M. (2005). The supply chain trading agent competition. *Electronic Commerce Research and Applications, Spring*, 66–84.

Ashby, W. R. (1940). Adaptiveness and equilibrium. *J. Mental Science, 86*, 478–483.

Ashby, W. R. (1948). Design for a brain. *Electronic Engineering, December*, 379–383.

Ashby, W. R. (1952). *Design for a Brain*. Wiley.

Asimov, I. (1942). Runaround. *Astounding Science Fiction*, March.

Asimov, I. (1950). *I, Robot*. Doubleday.

Asimov, I. (1958). The feeling of power. *If: Worlds of Science Fiction*, February.

Astrom, K. J. (1965). Optimal control of Markov decision processes with incomplete state estimation. *J. Math. Anal. Applic., 10*, 174–205.

Atkeson, C. G., Moore, A. W. y Schaal, S. (1997). Locally weighted learning for control. En *Lazy learning*. Springer.

Audi, R. (Ed.). (1999). *The Cambridge Dictionary of Philosophy*. Cambridge University Press.

Auer, P., Cesa-Bianchi, N. y Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning, 47*, 235–256.

Aumann, R. y Brandenburger, A. (1995). Epistemic conditions for nash equilibrium. *Econometrica, 67*, 1161–1180.

Axelrod, R. (1985). *The Evolution of Cooperation*. Basic Books.

Ba, J. L., Kiros, J. R. y Hinton, G. E. (2016). Layer normalization. arXiv:1607.06450.

Baader, F., Calvanese,D.,McGuinness, D., Nardi, D. y Patel-Schneider, P. (2007). *The Description Logic Handbook* (2nd edition). Cambridge University Press.

Baader, F. y Snyder, W. (2001). Unification theory. En Robinson, J. y Voronkov, A. (Eds.), *Handbook of Automated Reasoning*. Elsevier.

Bacchus, F. (1990). *Representing and Reasoning with Probabilistic Knowledge*. MIT Press.

Bacchus, F. y Grove, A. (1995). Graphical models for preference and utility. En *UAI-95*.

Bacchus, F. y Grove, A. (1996). Utility independence in a qualitative decision theory. En *KR-96*.

Bacchus, F., Grove, A., Halpern, J. Y. y Koller, D. (1992). From statistics to beliefs. En *AAAI-92*.

Bacchus, F. y Van Beek, P. (1998). On the conversion between non-binary and binary constraint satisfaction problems. En *AAAI-98*.

Bacchus, F. y Van Run, P. (1995). Dynamic variable ordering in CSPs. En *CP-95*.

Bacchus, F., Dalmao, S. y Pitassi, T. (2003). Value elimination: Bayesian inference via backtracking search. En *UAI-03*.

Bachmann, P. G. H. (1894). *Die analytische Zahlentheorie*. B. G. Teubner, Leipzig.

Backus, J. W. (1959). The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM conference. *Proc. Int'l Conf. on Information Processing*.

Bacon, F. (1609). *Wisdom of the Ancients*. Cassell and Company.

Baeza-Yates, R. y Ribeiro-Neto, B. (2011). *Modern Information Retrieval* (2nd edition). Addison-Wesley.

Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D. y Shmatikov, V. (2018). How to backdoor federated learning. arXiv:1807.00459.

Bagnell, J. A. y Schneider, J. (2001). Autonomous helicopter control using reinforcement learning policy search methods. En *ICRA-01*.

Bahdanau, D., Cho, K. y Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. En *ICLR-15*.

Bahubalendruni, M. R. y Biswal, B. B. (2016). A review on assembly sequence generation and its automation. *Proc. Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, 230*, 824–838.

Bai, A. y Russell, S. J. (2017). Efficient reinforcement learning with hierarchies of machines by leveraging internal transitions. En *IJCAI-17*.

Bai, H., Cai, S., Ye, N., Hsu, D. y Lee, W. S. (2015). Intention-aware online POMDP planning for autonomous driving in a crowd. En *ICRA-15*.

Bajcsy, A., Losey, D. P., O'Malley,M.K. y Dragan, A. D. (2017). Learning robot objectives from physical human interaction. *Proceedings of Machine Learning Research, 78*, 217–226.

Baker, C. L., Saxe, R. y Tenenbaum, J. B. (2009). Action understanding as inverse planning. *Cognition, 113*, 329–349.

Baker, J. (1975). The Dragon system: An overview. *IEEE Transactions on Acoustics, Speech, and Signal Processing, 23*, 24–29.

Baker, J. (1979). Trainable grammars for speech recognition. En *Speech Communication Papers for the 97th Meeting of the Acoustical Society of America*.

Baldi, P., Chauvin, Y., Hunkapiller, T., y McClure, M. (1994). Hidden Markov models of biological primary sequence information. *PNAS, 91*, 1059–1063.

Baldwin, J. M. (1896). A new factor in evolution. *American Naturalist, 30*, 441–451. Continued on pages 536–553.

Ballard, B. W. (1983). The *-minimax search procedure for trees containing chance nodes. *AIJ, 21*, 327–350.

Baluja, S. (1997). Genetic algorithms and explicit search statistics. En *NeurIPS 9*.

Bancilhon, F., Maier, D., Sagiv, Y. y Ullman, J. D. (1986). Magic sets and other strange ways to implement logic programs. En *PODS-86*.

Banko, M. y Brill, E. (2001). Scaling to very very large corpora for natural language disambiguation. En *ACL-01*.

Banko, M., Brill, E., Dumais, S. T. y Lin, J. (2002). AskMSR: Question answering using the worldwide web. En *Proc. AAAI Spring Symposium on Mining Answers from Texts and Knowledge Bases*.

Banko,M., Cafarella,M. J., Soderland, S., Broadhead, M. y Etzioni, O. (2007). Open information extraction from the web. En *IJCAI-07*.

Banko, M. y Etzioni, O. (2008). The tradeoffs between open and traditional relation extraction. En *ACL-08*.

Bansal, K., Loos, S., Rabe, M. N., Szegedy, C. y Wilcox, S. (2019). HOList: An environment for machine learning of higher-order theorem proving (extended version). arXiv:1904.03241.

Bar-Hillel, Y. (1954). Indexical expressions. *Mind, 63*, 359–379.

Bar-Shalom, Y. (Ed.). (1992). *Multitarget-Multisensor Tracking: Advanced Applications*. Artech House.

Bar-Shalom, Y. y Fortmann, T. E. (1988). *Tracking and Data Association*. Academic Press.

Bar-Shalom, Y., Li, X.-R. y Kirubarajan, T. (2001). *Estimation, Tracking and Navigation: Theory, Algorithms and Software*. Wiley.

Barber, D. (2012). *Bayesian Reasoning and Machine Learning*. Cambridge University Press.

Barr, A. y Feigenbaum, E. A. (Eds.). (1981). *The Handbook of Artificial Intelligence*, Vol. 1. HeurisTech Press y William Kaufmann.

Barreiro, J., Boyce, M., Do, M., Frank, J., Iatauro, M., Kichkaylo, T., Morris, P., Ong, J., Remolina, E., Smith, T., *et al.* (2012). EUROPA: A platform for AI planning, scheduling, constraint programming, and optimization. *4th International Competition on Knowledge Engineering for Planning and Scheduling (ICKEPS)*.

Barreno, M., Nelson, B., Joseph, A. D. y Tygar, J. D. (2010). The security of machine learning. *Machine Learning, 81*, 121–148.

Barrett, S. y Stone, P. (2015). Cooperating with unknown teammates in complex domains: A robot soccer case study of ad hoc teamwork. En *AAAI-15*.

Bartâk, R., Salido, M. A. y Rossi, F. (2010). New trends in constraint satisfaction, planning, and scheduling: A survey. *The Knowledge Engineering Review, 25*, 249–279.

Bartholdi, J. J., Tovey, C. A. y Trick, M. A. (1989). The computational difficulty of manipulating an election. *Social Choice and Welfare, 6*, 227–241.

Barto, A. G., Bradtke, S. J. y Singh, S. (1995). Learning to act using real-time dynamic programming. *AIJ, 73*, 81–138.

Barto, A. G., Sutton, R. S. y Brouwer, P. S. (1981). Associative search network: A reinforcement learning associative memory. *Biological Cybernetics, 40*, 201–211.

Barwise, J. y Etchemendy, J. (2002). *Language, Proof and Logic*. CSLI Press.

Baum, E., Boneh, D., and Garrett, C. (1995). On genetic algorithms. En *COLT-95*.

Baum, E. y Smith, W. D. (1997). A Bayesian approach to relevance in game playing. *AIJ, 97*, 195–242.

Baum, L. E. y Petrie, T. (1966). Statistical inference for probabilistic functions of finite state Markov chains. *Annals of Mathematical Statistics, 41*, 1554–1563.

Baxter, J. y Bartlett, P. (2000). Reinforcement learning in POMDPs via direct gradient ascent. En *ICML-00*.

Bayardo, R. J. y Agrawal, R. (2005). Data privacy through optimal k-anonymization. En *Proc. 21st Int'l Conf. on Data Engineering*.

Bayardo, R. J. y Miranker, D. P. (1994). An optimal backtrack algorithm for tree-structured constraint satisfaction problems. *AIJ, 71*, 159–181.

Bayardo, R. J. y Schrag, R. C. (1997). Using CSP look-back techniques to solve real-world SAT instances. En *AAAI-97*.

Bayes, T. (1763). An essay towards solving a problem in the doctrine of chances. *Phil. Trans. Roy. Soc., 53*, 370–418.

Beal, J. y Winston, P. H. (2009). The new frontier of human-level artificial intelligence. *IEEE Intelligent Systems, 24*, 21–23.

Beardon, A. F., Candeal, J. C., Herden, G., Induráin, E. y Mehta, G. B. (2002). The non-existence of a utility function and the structure of non-representable preference relations. *Journal of Mathematical Economics, 37*, 17 – 38.

Beattie, C., Leibo, J. Z., Teplyashin, D., Ward, T., Wainwright, M., Küttler, H., Lefrancq, A., Green, S., Valdés, V., Sadik, A., Schrittwieser, J., Anderson, K., York, S., Cant, M., Cain, A., Bolton, A., Gaffney, S., King, H., Hassabis, D., Legg, S. y Petersen, S. (2016). DeepMind lab. arXiv:1612.03801.

Bechhofer, R. (1954). A single-sample multiple decision procedure for ranking means of normal populations with known variances. *Annals of Mathematical Statistics, 25*, 16–39.

Beck, J. C., Feng, T. K. y Watson, J.-P. (2011). Combining constraint programming and local search for job-shop scheduling. *INFORMS Journal on Computing, 23*, 1–14.

Beckert, B. y Posegga, J. (1995). Leantap: Lean, tableau-based deduction. *JAR, 15*, 339–358.

Beeri, C., Fagin, R., Maier, D., and Yannakakis, M. (1983). On the desirability of acyclic database schemes. *JACM, 30*, 479–513.

Bekey, G. (2008). *Robotics: State Of The Art And Future Challenges*. Imperial College Press.

Belkin, M., Hsu, D., Ma, S. y Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *PNAS, 116*, 15849–15854.

Bell, C. y Tate, A. (1985). Using temporal constraints to restrict search in a planner. En *Proc. Third Alvey IKBS SIG Workshop*.

Bell, J. L. y Machover, M. (1977). *A Course in Mathematical Logic*. Elsevier.

Bellamy, E. (2003). *Looking Backward: 2000-1887*. Broadview Press.

Bellamy, R. K. E., Dey, K., Hind, M., Hoffman, S. C., Houde, S., Kannan, K., Lohia, P., Martino, J., Mehta, S., Mojsilovic, A., Nagar, S., Ramamurthy, K. N., Richards, J. T., Saha, D., Sattigeri, P., Singh, M., Varshney, K. R. y Zhang, Y. (2018). AI fairness 360: An extensible toolkit for detecting, understanding, and mitigating unwanted algorithmic bias. arXiv:1810.01943.

Bellemare, M. G., Naddaf, Y., Veness, J. y Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. *JAIR, 47*, 253–279.

Bellman, R. E. (1952). On the theory of dynamic programming. *PNAS, 38*, 716–719.

Bellman, R. E. (1958). On a routing problem. *Quarterly of Applied Mathematics, 16*.

Bellman, R. E. (1961). *Adaptive Control Processes: A Guided Tour*. Princeton University Press.

Bellman, R. E. (1965). On the application of dynamic programming to the determination of optimal play in chess and checkers. *PNAS, 53*, 244–246.

Bellman, R. E. (1984). *Eye of the Hurricane*. World Scientific.

Bellman, R. E. y Dreyfus, S. E. (1962). *Applied Dynamic Programming*. Princeton University Press.

Bellman, R. E. (1957). *Dynamic Programming*. Princeton University Press.

Ben-Tal, A. y Nemirovski, A. (2001). *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*. SIAM (Society for Industrial and Applied Mathematics).

Bengio, Y., Simard, P. y Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks, 5*, 157–166.

Bengio, Y. y Bengio, S. (2001). Modeling highdimensional discrete data with multi-layer neural networks. *In NeurIPS 13*.

Bengio, Y., Ducharme, R., Vincent, P. y Jauvin, C. (2003). A neural probabilistic language model. *JMLR, 3*, 1137–1155.

Bengio, Y. y LeCun, Y. (2007). Scaling learning algorithms towards AI. En Bottou, L., Chapelle, O., DeCoste, D., and Weston, J. (Eds.), *Large-Scale Kernel Machines*. MIT Press.

Benjamin, M. (2013). *Drone Warfare: Killing by Remote Control*. Verso Books.

Bentham, J. (1823). *Principles of Morals and Legislation*. Oxford University Press, Oxford. Trabajo original publicado en 1789.

Benzmüller, C. y Paleo, B. W. (2013). Formalization, mechanization and automation of Gödel's proof of God's existence. arXiv:1308.4526.

Beresniak, A., Medina-Lara, A., Auray, J. P., De Wever, A., Praet, J.-C., Tarricone, R., Torbica, A., Dupont, D., Lamure, M. y Duru, G. (2015). Validation of the underlying assumptions of the quality-adjusted life-years outcome: Results from the ECHOUTCOME European project. *PharmacoEconomics, 33*, 61–69.

Berger, J. O. (1985). *Statistical Decision Theory and Bayesian Analysis*. Springer Verlag.

Bergstra, J. y Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR, 13*, 281–305.

Berk, R., Heidari, H., Jabbari, S., Kearns, M. y Roth, A. (2017). Fairness in criminal justice risk assessments: The state of the art. arXiv:1703.09207.

Berkson, J. (1944). Application of the logistic function to bio-assay. *JASA, 39*, 357–365.

Berleur, J. y Brunnstein, K. (2001). *Ethics of Computing: Codes, Spaces for Discussion and Law*. Chapman and Hall.

Berlin, K., Koren, S., Chin, C.-S., Drake, J. P., Landolin, J. M. y Phillippy, A. M. (2015). Assembling large genomes with single-molecule sequencing and locality-sensitive hashing. *Nature Biotechnology, 33*, 623.

Berliner, H. J. (1979). The B* tree search algorithm: A best-first proof procedure. *AIJ, 12*, 23–40.

Berliner, H. J. (1980a). Backgammon computer program beats world champion. *AIJ, 14*, 205–220.

Berliner, H. J. (1980b). Computer backgammon. *Scientific American, 249*, 64–72.

Bermúdez-Chacón, R., Gonnet, G. H. y Smith, K. (2015). Automatic problem-specific hyperparameter optimization and model selection for supervised machine learning. Tech. rep., ETH Zurich.

Bernardo, J. M. y Smith, A. (1994). *Bayesian Theory*. Wiley.

Berners-Lee, T., Hendler, J. y Lassila, O. (2001). The semantic web. *Scientific American, 284*, 34–43.

Bernoulli, D. (1738). Specimen theoriae novae de mensura sortis. *Proc. St. Petersburg Imperial Academy of Sciences, 5*, 175–192.

Bernstein, P. L. (1996). *Against the Gods: The Remarkable Story of Risk*. Wiley.

Berrada, L., Zisserman, A. y Kumar, M. P. (2019). Training neural networks for and by interpolation. arXiv:1906.05661.

Berrou, C., Glavieux, A. y Thitimajshima, P. (1993). Near Shannon limit error control-correcting coding and decoding: Turbo-codes. 1. En *Proc. IEEE International Conference on Communications*.

Berry, D. A. y Fristedt, B. (1985). Bandit Problems: *Sequential Allocation of Experiments*. Chapman and Hall.

Bertele, U. y Brioschi, F. (1972). *Nonserial Dynamic Programming*. Academic Press.

Bertoli, P., Cimatti, A. y Roveri, M. (2001a). Heuristic search + symbolic model checking = efficient conformant planning. En *IJCAI-01*.

Bertoli, P., Cimatti, A., Roveri, M. y Traverso, P. (2001b). Planning in nondeterministic domains under partial observability via symbolic model checking. En *IJCAI-01*.

Bertot, Y., Casteran, P., Huet, G. y Paulin-Mohring, C. (2004). *Interactive Theorem Proving and Program Development*. Springer.

Bertsekas, D. (1987). *Dynamic Programming: Deterministic and Stochastic Models*. Prentice-Hall.

Bertsekas, D. y Tsitsiklis, J. N. (1996). NeuroDynamic Programming. Athena Scientific.

Bertsekas, D. y Tsitsiklis, J. N. (2008). *Introduction to Probability* (2nd edition). Athena Scientific.

Bertsekas,D. y Shreve, S. E. (2007). *Stochastic Optimal Control: The Discrete-Time Case*. Athena Scientific.

Bertsimas, D., Delarue, A., y Martin, S. (2019). Optimizing schools' start time and bus routes. *PNAS, 116-13*, 5943–5948.

Bertsimas, D. y Dunn, J. (2017). Optimal classification trees. *Machine Learning, 106*, 1039–1082.

Bessen, J. (2015). *Learning by Doing: The Real Connection between Innovation, Wages, and Wealth*. Yale University Press.

Bessière, C. (2006). Constraint propagation. En Rossi, F., van Beek, P. y Walsh, T. (Eds.), *Handbook of Constraint Programming*. Elsevier.

Beutel, A., Chen, J., Doshi, T., Qian, H., Woodruff, A., Luu, C., Kreitmann, P., Bischof, J. y Chi, E. H. (2019). Putting fairness principles into practice: Challenges, metrics, and improvements. arXiv:1901.04562.

Bhar, R. y Hamori, S. (2004). *Hidden Markov Models: Applications to Financial Economics*. Springer.

Bibel, W. (1993). *Deduction: Automated Logic*. Academic Press.

Bien, J., Tibshirani, R., *et al*. (2011). Prototype selection for interpretable classification. *Annals of Applied Statistics, 5*, 2403–2424.

Biere, A., Heule, M., Van Maaren, H. y Walsh, T. (Eds.). (2009). *Handbook of Satisfiability*. IOS Press.

Bies, A., Mott, J. y Warner, C. (2015). English news text treebank: Penn treebank revised. Linguistic Data Consortium.

Billings, D., Burch, N., Davidson, A., Holte, R. C., Schaeffer, J., Schauenberg, T. y Szafron, D. (2003). Approximating game-theoretic optimal strategies for full-scale poker. En *IJCAI-03*.

Billingsley, P. (2012). *Probability and Measure* (4a edición). Wiley.

Binder, J., Koller, D., Russell, S. J. y Kanazawa, K. (1997a). Adaptive probabilistic networks with hidden variables. *Machine Learning, 29*, 213–244.

Binder, J., Murphy, K. y Russell, S. J. (1997b). Space-efficient inference in dynamic probabilistic networks. En *IJCAI-97*.

Bingham, E., Chen, J., Jankowiak, M., Obermeyer, F., Pradhan, N., Karaletsos, T., Singh, R., Szerlip, P., Horsfall, P. y Goodman, N. D. (2019). Pyro: Deep universal probabilistic programming. *JMLR, 20*, 1–26.

Binmore, K. (1982). *Essays on Foundations of Game Theory*. Pitman.

Biran, O. y Cotton, C. (2017). Explanation and justification in machine learning: A survey. En *Proc. IJCAI-17 Workshop on Explainable AI*.

Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Oxford University Press.

Bishop, C. M. (2007). *Pattern Recognition and Machine Learning*. Springer-Verlag.

Bisson, T. (1990). They're made out of meat. *Omni Magazine*.

Bistarelli, S., Montanari, U. y Rossi, F. (1997). Semiring-based constraint satisfaction and optimization. *JACM, 44*, 201–236.

Bitner, J. R. y Reingold, E. M. (1975). Backtrack programming techniques. *CACM, 18*, 651–656.

Bizer, C., Auer, S., Kobilarov, G., Lehmann, J. y Cyganiak, R. (2007). DBPedia – querying Wikipedia like a database. En *16th International Conference on World Wide Web*.

Blazewicz, J., Ecker, K., Pesch, E., Schmidt, G. y Weglarz, J. (2007). *Handbook on Scheduling: Models and Methods for Advanced Planning*. Springer-Verlag.

Blei, D.M., Ng, A. Y. y Jordan,M. I. (2002). Latent Dirichlet allocation. En *NeurIPS 14*.

Bliss, C. I. (1934). The method of probits. *Science, 79*, 38–39.

Block, H. D., Knight, B. y Rosenblatt, F. (1962). Analysis of a four-layer series-coupled perceptron. *Rev. Modern Physics, 34*, 275–282.

Block, N. (2009). Comparing the major theories of consciousness. En Gazzaniga, M. S. (Ed.), *The Cognitive Neurosciences*. MIT Press.

Blum, A. L. y Furst, M. (1997). Fast planning through planning graph analysis. *AIJ, 90*, 281–300.

Blum, A. L. (1996). On-line algorithms in machine learning. En *Proc. Workshop on On-Line Algorithms, Dagstuhl*.

Blum, A. L., Hopcroft, J. y Kannan, R. (2020). *Foundations of Data Science*. Cambridge University Press.

Blum, A. L. y Mitchell, T. M. (1998). Combining labeled and unlabeled data with co-training. En *COLT-98*.

Blumer, A., Ehrenfeucht, A., Haussler, D. y Warmuth, M. (1989). Learnability and the Vapnik-Chervonenkis dimension. *JACM, 36*, 929–965.

Bobrow, D. G. (1967). Natural language input for a computer problem solving system. En Minsky, M. L. (Ed.), *Semantic Information Processing*. MIT Press.

Bod, R. (2008). The data-oriented parsing approach: Theory and application. En *Computational Intelligence: A Compendium*. Springer-Verlag.

Bod, R., Scha, R. y Sima'an, K. (2003). Data-Oriented Parsing. CSLI Press.

Boddington, P. (2017). *Towards a Code of Ethics for Artificial Intelligence*. Springer-Verlag.

Boden, M. A. (Ed.). (1990). *The Philosophy of Artificial Intelligence*. Oxford University Press.

Bolognesi, A. y Ciancarini, P. (2003). Computer programming of kriegspiel endings: The case of KR vs. K. En *Advances in Computer Games 10*.

Bolton, R. J. y Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical science, 17*, 235–249.

Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H. B., Patel, S., Ramage, D., Segal, A. y Seth, K. (2017). Practical secure aggregation for privacy-preserving machine learning. En *Proc. ACM SIGSAC Conference on Computer and Communications Security*.

Bond, A. H. y Gasser, L. (Eds.). (1988). *Readings in Distributed Artificial Intelligence*. Morgan Kaufmann.

Bonet, B. (2002). An epsilon-optimal grid-based algorithm for partially observable Markov decision processes. En *ICML-02*.

Bonet, B. y Geffner, H. (1999). Planning as heuristic search: New results. En *ECP-99*.

Bonet, B. y Geffner, H. (2000). Planning with incomplete information as heuristic search in belief space. En *ICAPS-00*.

Bonet, B. y Geffner, H. (2005). An algorithm better than AO*? En *AAAI-05*.

Boole, G. (1847). *The Mathematical Analysis of Logic: Being an Essay towards a Calculus of Deductive Reasoning*. Macmillan, Barclay, and Macmillan.

Booth, T. L. (1969). Probabilistic representation of formal languages. En *IEEE Conference Record of the 1969 Tenth Annual Symposium on Switching and Automata Theory*.

Borel, E. (1921). La théorie du jeu et les équations intégrales à noyau symétrique. *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences, 173*, 1304–1308.

Borenstein, J., Everett, B. y Feng, L. (1996). *Navigating Mobile Robots: Systems and Techniques*. A. K. Peters, Ltd.

Borenstein, J. y Koren., Y. (1991). The vector field histogram—Fast obstacle avoidance for mobile robots. *IEEE Transactions on Robotics and Automation, 7*, 278–288.

Borgida, A., Brachman, R. J., McGuinness, D. y Alperin Resnick, L. (1989). CLASSIC: A structural data model for objects. *SIGMOD Record, 18*, 58–67.

Boroditsky, L. (2003). Linguistic relativity. En Nadel, L. (Ed.), *Encyclopedia of Cognitive Science*. Macmillan.

Boser, B., Guyon, I. y Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. En *COLT-92*.

Bosse, M., Newman, P., Leonard, J., Soika,M., Feiten, W. y Teller, S. (2004). Simultaneous localization and map building in large-scale cyclic environments using the Atlas framework. *Int. J. Robotics Research, 23*, 1113–1139.

Bostrom, N. (2005). A history of transhumanist thought. *Journal of Evolution and Technology, 14*, 1–25.

Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press.

Bottou, L. y Bousquet, O. (2008). The tradeoffs of large scale learning. En *NeurIPS 20*.

Bottou, L., Curtis, F. E. y Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review, 60*, 223–311.

Boué, L. (2019). Real numbers, data science and chaos: How to fit any dataset with a single parameter. arXiv:1904.12320.

Bousmalis, K., Irpan, A.,Wohlhart, P., Bai, Y., Kelcey, M., Kalakrishnan, M., Downs, L., Ibarz, J., Pastor, P., Konolige, K., Levine, S. y Vanhoucke, V. (2017). Using simulation and domain adaptation to improve efficiency of deep robotic grasping. arXiv:1709.07857.

Boutilier, C. (2002). A POMDP formulation of preference elicitation problems. En *AAAI-02*.

Boutilier, C. y Brafman, R. I. (2001). Partial-order planning with concurrent interacting actions. *JAIR, 14*, 105–136.

Boutilier, C., Dearden, R. y Goldszmidt, M. (2000). Stochastic dynamic programming with factored representations. *AIJ, 121*, 49–107.

Boutilier, C., Reiter, R. y Price, B. (2001). Symbolic dynamic programming for first-order MDPs. En *IJCAI-01*.

Boutilier, C., Brafman, R. I., Domshlak, C., Hoos, H. H. y Poole, D. (2004). CP-nets: A tool for representing and reasoning with conditional ceteris paribus preference statements. *JAIR, 21*, 135–191.

Boutilier, C., Friedman, N., Goldszmidt, M. y Koller, D. (1996). Context-specific independence in Bayesian networks. En *UAI-96*.

Bouzy, B. y Cazenave, T. (2001). Computer Go: An AI oriented survey. *AIJ, 132*, 39–103.

Bowling, M., Burch, N., Johanson,M. y Tammelin, O. (2015). Heads-up limit hold'em poker is solved. *Science, 347*, 145–149.

Bowling, M., Johanson, M., Burch, N. y Szafron, D. (2008). Strategy evaluation in extensive games with importance sampling. En *ICML-08*.

Bowman, S., Angeli, G., Potts, C. y Manning, C. (2015). A large annotated corpus for learning natural language inference. En *EMNLP-15*.

Box, G. E. P. (1957). Evolutionary operation: A method of increasing industrial productivity. *Applied Statistics, 6*, 81–101.

Box, G. E. P., Jenkins, G., Reinsel, G. y Ljung, G. M. (2016). *Time Series Analysis: Forecasting and Control* (5th edition). Wiley.

Box, G. E. P. and Tiao, G. C. (1973). *Bayesian Inference in Statistical Analysis*. Addison-Wesley.

Boyan, J. A. y Moore, A.W. (1998). Learning evaluation functions for global optimization and Boolean satisfiability. En *AAAI-98*.

Boyd, S. y Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Boyen, X., Friedman, N. y Koller, D. (1999). Discovering the hidden structure of complex dynamic systems. En *UAI-99*.

Boyer, R. S. y Moore, J. S. (1979). *A Computational Logic*. Academic Press.

Boyer, R. S. y Moore, J. S. (1984). Proof checking the RSA public key encryption algorithm. *American Mathematical Monthly, 91*, 181–189.

Brachman, R. J. (1979). On the epistemological status of semantic networks. In Findler, N. V. (Ed.), *Associative Networks: Representation and Use of Knowledge by Computers*. Academic Press.

Brachman, R. J. y Levesque, H. J. (Eds.). (1985). *Readings in Knowledge Representation*.Morgan Kaufmann.

Bradt, R. N., Johnson, S. M. y Karlin, S. (1956). On sequential designs for maximizing the sum of n observations. *Ann. Math. Statist., 27*, 1060–1074.

Brafman, O. y Brafman, R. (2009). *Sway: The Irresistible Pull of Irrational Behavior*. Broadway Business.

Brafman, R. I. y Domshlak, C. (2008). From one to many: Planning for loosely coupled multi-agent systems. En *ICAPS-08*.

Brafman, R. I. y Tennenholtz, M. (2000). A near optimal polynomial time algorithm for learning in certain classes of stochastic games. *AIJ, 121*, 31–47.

Braitenberg, V. (1984). *Vehicles: Experiments in Synthetic Psychology*. MIT Press.

Brandt, F., Conitzer, V., Endriss, U., Lang, J. y Procaccia, A. D. (Eds.). (2016). *Handbook of Computational Social Choice*. Cambridge University Press.

Brants, T. (2000). TnT: A statistical part-of-speech tagger. En *Proc. Sixth Conference on Applied Natural Language Processing*.

Brants, T., Popat, A. C., Xu, P., Och, F. J. y Dean, J. (2007). Large language models in machine translation. En *EMNLP-CoNLL-07*.

Bratko, I. (2009). *Prolog Programming for Artificial Intelligence* (4a edición). Addison-Wesley.

Bratman, M. E. (1987). *Intention, Plans, and Practical Reason*. Harvard University Press.

Breck, E., Cai, S., Nielsen, E., Salib, M. y Sculley, D. (2016). What's your ML test score? A rubric for ML production systems. En *Proc. NIPS 2016 Workshop on Reliable Machine Learning in the Wild*.

Breese, J. S. (1992). Construction of belief and decision networks. *Computational Intelligence, 8*, 624–647.

Breese, J. S. y Heckerman, D. (1996). Decisiontheoretic troubleshooting: A framework for repair and experiment. En *UAI-96*.

Breiman, L., Friedman, J., Olshen, R. A. y Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Brelaz, D. (1979). New methods to color the vertices of a graph. *CACM, 22*, 251–256.

Brent, R. P. (1973). *Algorithms for Minimization without Derivatives*. Prentice-Hall.

Bresnan, J. (1982). *The Mental Representation of Grammatical Relations*. MIT Press.

Brewka, G., Dix, J. y Konolige, K. (1997). *Nononotonic Reasoning: An Overview*. Center for the Study of Language and Information (CSLI).

Brickley, D. y Guha, R. V. (2004). RDF vocabulary description language 1.0: RDF schema. Tech. rep., W3C.

Briggs, R. (1985). Knowledge representation in Sanskrit and artificial intelligence. *AIMag, 6*, 32–39.

Brill, E. (1992). A simple rule-based part of speech tagger. En *Proc. Third Conference on Applied Natural Language Processing*.

Brin, D. (1998). *The Transparent Society*. Perseus.

Brin, S. y Page, L. (1998). The anatomy of a largescale hypertextual web search engine. En *Proc. Seventh World Wide Web Conference*.

Bringsjord, S. (2008). If I were judge. En Epstein, R., Roberts, G. y Beber, G. (Eds.), *Parsing the Turing Test*. Springer.

Broadbent, D. E. (1958). *Perception and Communication*. Pergamon.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J. y Zaremba, W. (2016). OpenAI gym. arXiv:1606.01540.

Brooks, R. A. (1986). A robust layered control system for a mobile robot. *IEEE J. of Robotics and Automation, 2*, 14–23.

Brooks, R. A. (1989). Engineering approach to building complete, intelligent beings. *Proc. SPIE—the International Society for Optical Engineering*, 1002, 618–625.

Brooks, R. A. (1991). Intelligence without representation. *AIJ, 47*, 139–159.

Brooks, R. A. and Lozano-Perez, T. (1985). A subdivision algorithm in configuration space for findpath with rotation. *IEEE Transactions on Systems, Man and Cybernetics, 15*, 224–233.

Brooks, R. A. (2017). The seven deadly sins of AI predictions. *MIT Technology Review, Oct 6*.

Brooks, S., Gelman, A., Jones, G., and Meng, X.-L. (2011). *Handbook of Markov Chain Monte Carlo*. Chapman & Hall/CRC.

Brown, C., Finkelstein, L. y Purdom, P. (1988). Backtrack searching in the presence of symmetry. En Mora, T. (Ed.), *Applied Algebra, Algebraic Algorithms and Error-Correcting Codes*. Springer-Verlag.

Brown, K. C. (1974). A note on the apparent bias of net revenue estimates. *J. Finance, 29*, 1215–1216.

Brown, N. y Sandholm, T. (2017). Libratus: The superhuman AI for no-limit poker. En *IJCAI-17*.

Brown, N. y Sandholm, T. (2019). Superhuman AI for multiplayer poker. *Science, 365*, 885–890.

Brown, P. F., Cocke, J., Della Pietra, S. A., Della Pietra, V. J., Jelinek, F., Mercer, R. L. y Roossin, P. (1988). A statistical approach to language translation. En *COLING-88*.

Brown, P. F., Desouza, P. V., Mercer, R. L., Pietra, V. J. D. y Lai, J. C. (1992). Class-based n-gram models of natural language. *Computational linguistics, 18*(4).

Browne, C., Powley, E. J., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Liebana, D. P., Samothrakis, S. y Colton, S. (2012). A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games, 4*, 1–43.

Brownston, L., Farrell, R., Kant, E., and Martin, N. (1985). *Programming Expert Systems in OPS5: An Introduction to Rule-Based Programming*. Addison-Wesley.

Bruce, V., Green, P. y Georgeson, M. (2003). *Visual Perception: Physiology, Psychology and Ecology*. Routledge and Kegan Paul.

Brügmann, B. (1993). Monte Carlo Go. Tech. rep., Department of Physics, Syracuse University.

Bryce, D. y Kambhampati, S. (2007). A tutorial on planning graph-based reachability heuristics. *AIMag, Spring*, 47–83.

Bryce, D., Kambhampati, S. y Smith, D. E. (2006). Planning graph heuristics for belief space search. *JAIR, 26*, 35–99.

Brynjolfsson, E. y McAfee, A. (2011). *Race Against the Machine*. Digital Frontier Press.

Brynjolfsson, E. y McAfee, A. (2014). *The Second Machine Age*. W.W. Norton.

Brynjolfsson, E., Rock, D. y Syverson, C. (2018). Artificial intelligence and the modern productivity paradox: A clash of expectations and statistics. En Agrawal, A., Gans, J. y Goldfarb, A. (Eds.), *The Economics of Artificial Intelligence: An Agenda*. University of Chicago Press.

Bryson, A. E. y Ho, Y.-C. (1969). *Applied Optimal Control*. Blaisdell.

Bryson, A. E. (1962). A gradient method for optimizing multi-stage allocation processes. En *Proc. of a Harvard Symposium on Digital Computers and Their Applications*.

Bryson, J. J. (2012). A role for consciousness in action selection. *International Journal of Machine Consciousness, 4*, 471–482.

Bryson, J. J. y Winfield, A. (2017). Standardizing ethical design for artificial intelligence and autonomous systems. *Computer, 50*, 116–119.

Buchanan, B. G., Mitchell, T. M., Smith, R. G. y Johnson, C. R. (1978). Models of learning systems. En *Encyclopedia of Computer Science and Technology*, Vol. 11. Dekker.

Buchanan, B. G. y Shortliffe, E. H. (Eds.). (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley.

Buchanan, B. G., Sutherland, G. L. y Feigenbaum, E. A. (1969). Heuristic DENDRAL: A program for generating explanatory hypotheses in organic chemistry. En Meltzer, B., Michie, D. y Swann, M. (Eds.), *Machine Intelligence 4*. Edinburgh University Press.

Buck, C., Heafield, K. y Van Ooyen, B. (2014). Ngram counts and language models from the common crawl. En *Proc. International Conference on Language Resources and Evaluation*.

Buehler, M., Iagnemma, K. y Singh, S. (Eds.). (2006). *The 2005 DARPA Grand Challenge: The Great Robot Race*. Springer-Verlag.

Buffon, G. (1777). Essai d'arithmetique morale. Supplement to *Histoire naturelle, vol. IV*.

Bunt, H. C. (1985). The formal representation of (quasi-) continuous concepts. In Hobbs, J. R. and Moore, R. C. (Eds.), *Formal Theories of the Commonsense World*. Ablex.

Buolamwini, J. y Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. En *Conference on Fairness, Accountability and Transparency*.

Burgard, W., Cremers, A. B., Fox, D., Hahnel, D., Lakemeyer, G., Schulz, D., Steiner, W. y Thrun, S. (1999). Experiences with an interactive museum tourguide robot. *AIJ, 114*, 3–55.

Burkov, A. (2019). *The Hundred-Page Machine Learning Book*. Burkov.

Burns, E., Hatem, M., Leighton, M. J. y Ruml, W. (2012). Implementing fast heuristic search code. En *Symposium on Combinatorial Search*.

Buro, M. (1995). ProbCut: An effective selective extension of the alpha-beta algorithm. *J. International Computer Chess Association, 18*, 71–76.

Buro, M. (2002). Improving heuristic mini-max search by supervised learning. *AIJ, 134*, 85–99.

Burstein, J., Leacock, C. y Swartz, R. (2001). Automated evaluation of essays and short answers. En *Fifth International Computer Assisted Assessment Conference*.

Burton, R. (2009). *On Being Certain: Believing You Are Right Even When You're Not*. St. Martin's Griffin.

Buss, D. M. (2005). *Handbook of Evolutionary Psychology*. Wiley.

Butler, S. (1863). Darwin among the machines. *The Press (Christchurch, New Zealand), June 13*.

Bylander, T. (1994). The computational complexity of propositional STRIPS planning. *AIJ, 69*, 165–204.

Byrd, R. H., Lu, P., Nocedal, J. y Zhu, C. (1995). A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific and Statistical Computing, 16*, 1190–1208.

Cabeza, R. and Nyberg, L. (2001). Imaging cognition II: An empirical review of 275 PET and fMRI studies. *J. Cognitive Neuroscience, 12*, 1–47.

Cafarella, M. J., Halevy, A., Zhang, Y., Wang, D. Z. y Wu, E. (2008). Webtables: Exploring the power of tables on the web. En *VLDB-08*.

Calvanese, D., Lenzerini, M. y Nardi, D. (1999). Unifying class-based representation formalisms. *JAIR, 11*, 199–240.

Camacho, R. y Michie, D. (1995). Behavioral cloning: A correction. *AIMag, 16*, 92.

Campbell, D. E. y Kelly, J. (2002). Impossibility theorems in the Arrovian framework. En Arrow, K. J., Sen, A. K., y Suzumura, K. (Eds.), *Handbook of Social Choice and Welfare Volume 1*. Elsevier Science.

Campbell, M. S., Hoane, A. J. y Hsu, F.-H. (2002). Deep Blue. *AIJ, 134*, 57–83.

Cannings, C., Thompson, E. y Skolnick, M. H. (1978). Probability functions on complex pedigrees. *Advances in Applied Probability, 10*, 26–61.

Canny, J. y Reif, J. (1987). New lower bound techniques for robot motion planning problems. En *FOCS-87*.

Canny, J. (1986). A computational approach to edge detection. *PAMI, 8*, 679–698.

Canny, J. (1988). *The Complexity of Robot Motion Planning*. MIT Press.

Capen, E., Clapp, R. y Campbell, W. (1971). Competitive bidding in high-risk situations. *J. Petroleum Technology, 23*, 641–653.

Carbonell, J. G. (1983). Derivational analogy and its role in problem solving. En *AAAI-83*.

Carbonell, J. G., Knoblock, C. A. y Minton, S. (1989). PRODIGY: An integrated architecture for planning and learning. Technical report, Computer Science Department, Carnegie-Mellon University.

Carbonnel, C. y Cooper, M. C. (2016). Tractability in constraint satisfaction problems: A survey. *Constraints, 21*(2), 115–144.

Cardano, G. (1663). *Liber de ludo aleae*. Lyons.

Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A. y Kurakin, A. (2019). On evaluating adversarial robustness. arXiv:1902.06705.

Carnap, R. (1928). *Der logische Aufbau der Welt*. Weltkreis-verlag. Translated into English as The Logical Structure of the World (Carnap, 1967).

Carnap, R. (1948). On the application of inductive logic. *Philosophy and Phenomenological Research, 8*, 133–148.

Carnap, R. (1950). *Logical Foundations of Probability*. University of Chicago Press.

Carpenter, B., Gelman, A., Hoffman, M., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P. y Riddell, A. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software, 76*, 1–32.

Carroll, S. (2007). *The Making of the Fittest: DNA and the Ultimate Forensic Record of Evolution*. Norton.

Casati, R. y Varzi, A. (1999). *Parts and Places: The Structures of Spatial Representation*. MIT Press.

Cassandra, A. R., Kaelbling, L. P. y Littman, M. L. (1994). Acting optimally in partially observable stochastic domains. En *AAAI-94*.

Cassandras, C. G. y Lygeros, J. (2006). *Stochastic Hybrid Systems*. CRC Press.

Castro, R., Coates, M., Liang, G., Nowak, R. y Yu, B. (2004). Network tomography: Recent developments. *Statistical Science, 19*, 499–517.

Cauchy, A. (1847). Méthode générale pour la résolution des systèmes d'équations simultanées. *Comp. Rend. Sci. Paris, 25*, 536–538.

Cesa-Bianchi, N. y Lugosi, G. (2006). *Prediction, Learning, and Games*. Cambridge University Press.

Chajewska, U., Koller, D. y Parr, R. (2000). Making rational decisions using adaptive utility elicitation. En *AAAI-00*.

Chakrabarti, P. P., Ghose, S., Acharya, A. y de Sarkar, S. C. (1989). Heuristic search in restricted memory. *AIJ, 41*, 197–222.

Chalkiadakis, G., Elkind, E. y Wooldridge, M. (2011). *Computational Aspects of Cooperative Game Theory*. Morgan Kaufmann.

Chalmers, D. J. (1992). Subsymbolic computation and the Chinese room. In Dinsmore, J. (Ed.), *The symbolic and connectionist paradigms: Closing the gap*. Lawrence Erlbaum.

Chandola, V., Banerjee, A. y Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*.

Chandra, A. K. y Harel, D. (1980). Computable queries for relational data bases. *J. Computer and System Sciences, 21*, 156–178.

Chang, C.-L. y Lee, R. C.-T. (1973). *Symbolic Logic and Mechanical Theorem Proving*. Academic Press.

Chang, H. S., Fu, M. C., Hu, J. y Marcus, S. I. (2005). An adaptive sampling algorithm for solving Markov decision processes. *Operations Research, 53*, 126–139.

Chao, W.-L., Hu, H. y Sha, F. (2018). Being negative but constructively: Lessons learnt from creating better visual question answering datasets. En *ACL-18*.

Chapman, D. (1987). Planning for conjunctive goals. *AIJ, 32*, 333–377.

Charniak, E. (1993). *Statistical Language Learning*. MIT Press.

Charniak, E. (1996). Tree-bank grammars. En *AAAI-96*.

Charniak, E. (1997). Statistical parsing with a context-free grammar and word statistics. En *AAAI-97*.

Charniak, E. and Goldman, R. (1992). A Bayesian model of plan recognition. *AIJ, 64*, 53–79.

Charniak, E., Riesbeck, C.,McDermott, D., y Meehan, J. (1987). *Artificial Intelligence Programming* (2a edición). Lawrence Erlbaum.

Charniak, E. (1991). Bayesian networks without tears. *AIMag, 12*, 50–63.

Charniak, E. (2018). *Introduction to Deep Learning*. MIT Press.

Chaslot, G., Bakkes, S., Szita, I. y Spronck, P. (2008). Monte-Carlo tree search: A new framework for game AI. En *Proc. Fourth Artificial Intelligence and Interactive Digital Entertainment Conference*.

Chater, N. y Oaksford,M. (Eds.). (2008). *The Probabilistic Mind: Prospects for Bayesian Cognitive Science*. Oxford University Press.

Chatfield, C. (1989). *The Analysis of Time Series: An Introduction* (4a edición). Chapman and Hall.

Chavira, M. y Darwiche, A. (2008). On probabilistic inference by weighted model counting. *AIJ, 172*, 772–799.

Chawla, N. V., Bowyer, K. W., Hall, L. O. y Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *JAIR, 16*, 321–357.

Cheeseman, P. (1985). In defense of probability. En *IJCAI-85*.

Cheeseman, P. (1988). An inquiry into computer understanding. *Computational Intelligence, 4*, 58–66.

Cheeseman, P., Kanefsky, B. y Taylor, W. (1991). Where the really hard problems are. En *IJCAI-91*.

Cheeseman, P., Self,M., Kelly, J., and Stutz, J. (1988). Bayesian classification. En *AAAI-88*.

Cheeseman, P. y Stutz, J. (1996). Bayesian classification (AutoClass): Theory and results. En Fayyad, U., Piatesky-Shapiro, G., Smyth, P. y Uthurusamy, R. (Eds.), *Advances in Knowledge Discovery and Data Mining*. AAAI Press/MIT Press.

Chen, D. y Manning, C. (2014). A fast and accurate dependency parser using neural networks. En *EMNLP-14*.

Chen, J., Holte, R. C., Zilles, S. y Sturtevant, N. R. (2017). Front-to-end bidirectional heuristic search with near-optimal node expansions. *IJCAI-17*.

Chen, M. X., Firat, O., Bapna, A., Johnson, M., Macherey,W., Foster, G., Jones, L., Parmar, N., Schuster, M., Chen, Z.,Wu, Y. y Hughes, M. (2018). The best of both worlds: Combining recent advances in neural machine translation. En *ACL-18*.

Chen, S. F. y Goodman, J. (1996). An empirical study of smoothing techniques for language modeling. En *ACL-96*.

Chen, T. y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *KDD-16*.

Cheng, J. y Druzdzel, M. J. (2000). AIS-BN: An adaptive importance sampling algorithm for evidential reasoning in large Bayesian networks. *JAIR, 13*, 155–188.

Cheng, J., Greiner, R., Kelly, J., Bell, D. A. y Liu, W. (2002). Learning Bayesian networks from data: An information-theory based approach. *AIJ, 137*, 43–90.

Chiu, C., Sainath, T., Wu, Y., Prabhavalkar, R., Nguyen, P., Chen, Z., Kannan, A., Weiss, R., Rao, K., Gonina, K., Jaitly, N., Li, B., Chorowski, J. y Bacchiani, M. (2017). State-of-the-art speech recognition with sequence-to-sequence models. arXiv:1712.01769.

Chklovski, T. y Gil, Y. (2005). Improving the design of intelligent acquisition interfaces for collecting world knowledge from web contributors. En *Proc. Third International Conference on Knowledge Capture*.

Chollet, F. (2019). On the measure of intelligence. arXiv:1911.01547.

Chollet, F. (2017). *Deep Learning with Python*. Manning.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory, 2*, 113–124.

Chomsky, N. (1957). *Syntactic Structures*. Mouton.

Choromanska, A., Henaff, M., Mathieu, M., Arous, G. B. y LeCun, Y. (2014). The loss surface of multilayer networks. arXiv:1412.0233.

Choset, H. (1996). *Sensor Based Motion Planning: The Hierarchical Generalized Voronoi Graph*. Ph.D. thesis, California Institute of Technology.

Choset, H., Hutchinson, S., Lynch, K., Kantor, G., Burgard, W., Kavraki, L. y Thrun, S. (2005). *Principles of Robot Motion: Theory, Algorithms, and Implementation*. MIT Press.

Chouldechova, A. (2017). Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big Data, 5*, 153–163.

Chouldechova, A. y Roth, A. (2018). The frontiers of fairness in machine learning. arXiv:1810.08810.

Christian, B. (2011). *The Most Human Human*. Doubleday.

Christin, A., Rosenblat, A. y Boyd, D. (2015). Courts and predictive algorithms. *Data & Civil Rights*.

Chung, K. L. (1979). *Elementary Probability Theory with Stochastic Processes* (3a edición). Springer-Verlag.

Church, A. (1936). A note on the Entscheidungsproblem. *JSL, 1*, 40–41 and 101–102.

Church, A. (1956). *Introduction to Mathematical Logic*. Princeton University Press.

Church, K. (1988). A stochastic parts program and noun phrase parser for unrestricted texts. En *Proc. Second Conference on Applied Natural Language Processing*.

Church, K. y Patil, R. (1982). Coping with syntactic ambiguity or how to put the block in the box on the table. *Computational Linguistics, 8*, 139–149.

Church, K. (2004). Speech and language processing: Can we use the past to predict the future. En *Proc. Conference on Text, Speech, and Dialogue*.

Church, K. y Gale, W. A. (1991). A comparison of the enhanced Good–Turing and deleted estimation methods for estimating probabilities of English bigrams. *Computer Speech and Language, 5*, 19–54.

Church, K. y Hestness, J. (2019). A survey of 25 years of evaluation. *Natural Language Engineering, 25*, 753–767.

Churchland, P. M. (2013). *Matter and Consciousness* (3a edición). MIT Press.

Ciancarini, P. y Favini, G. P. (2010). Monte Carlo tree search in Kriegspiel. *AIJ, 174*, 670–684.

Ciancarini, P. y Wooldridge, M. (2001). *Agent-Oriented Software Engineering*. Springer-Verlag.

Cimatti, A., Roveri, M. y Traverso, P. (1998). Automatic OBDD-based generation of universal plans in non-deterministic domains. En *AAAI-98*.

Claret, G., Rajamani, S. K., Nori, A. V., Gordon, A. D. y Borgström, J. (2013). Bayesian inference using data flow analysis. En *Proc. 9th Joint Meeting on Foundations of Software Engineering*.

Clark, A. (1998). *Being There: Putting Brain, Body, and World Together Again*. MIT Press.

Clark, A. (2015). *Surfing Uncertainty: Prediction, Action, and the Embodied Mind*. Oxford University Press.

Clark, K. L. (1978). Negation as failure. En Gallaire, H. and Minker, J. (Eds.), *Logic and Data Bases*. Plenum.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C. y Tafjord, O. (2018). Think you have solved question answering? Try ARC, the AI2 reasoning challenge. arXiv:1803.05457.

Clark, P., Etzioni, O., Khot, T., Mishra, B. D., Richardson, K., *et al*. (2019). From 'F' to 'A' on the NY Regents science exams: An overview of the Aristo project. arXiv:1909.01958.

Clark, S. y Curran, J. R. (2004). Parsing the WSJ using CCG and log-linear models. En *ACL-04*.

Clarke, A. C. (1968). *2001: A Space Odyssey*. Signet.

Clarke, E. y Grumberg, O. (1987). Research on automatic verification of finite-state concurrent systems. *Annual Review of Computer Science, 2*, 269–290.

Clearwater, S. H. (Ed.). (1996). *Market-Based Control*. World Scientific.

Clocksin, W. F. y Mellish, C. S. (2003). *Programming in Prolog* (5a edición). Springer-Verlag.

Clocksin, W. F. (2003). *Clause and Effect: Prolog Programming for the Working Programmer*. Springer.

Coase, R. H. (1960). The problem of social cost. *Journal of Law and Economics*, pp. 1–44.

Coates, A., Abbeel, P. y Ng, A. Y. (2009). Apprenticeship learning for helicopter control. *Association for Computing Machinery, 52*(7).

Cobham, A. (1964). The intrinsic computational difficulty of functions. En *Proc. International Congress for Logic, Methodology, and Philosophy of Science*.

Cohen, P. R. (1995). *Empirical Methods for Artificial Intelligence*. MIT Press.

Cohen, P. R. y Levesque, H. J. (1990). Intention is choice with commitment. *AIJ, 42*, 213–261.

Cohen, P. R., Morgan, J. y Pollack, M. E. (1990). *Intentions in Communication*. MIT Press.

Cohen, P. R. y Perrault, C. R. (1979). Elements of a plan-based theory of speech acts. *Cognitive Science, 3*, 177–212.

Cohn, A. G., Bennett, B., Gooday, J.M., and Gotts, N. (1997). RCC: A calculus for region based qualitative spatial reasoning. *GeoInformatica, 1*, 275–316.

Collin, Z., Dechter, R. y Katz, S. (1999). Selfstabilizing distributed constraint satisfaction. *Chicago J. of Theoretical Computer Science, 1999*.

Collins, M. (1999). *Head-driven Statistical Models for Natural Language Processing*. Ph.D. thesis, University of Pennsylvania.

Collins, M. y Duffy, K. (2002). New ranking algorithms for parsing and tagging: Kernels over discrete structures, and the voted perceptron. En *ACL-02*.

Colmerauer, A. y Roussel, P. (1993). *The birth of Prolog. SIGPLAN Notices, 28*, 37–52.

Colmerauer, A., Kanoui, H., Pasero, R. y Roussel, P. (1973). Un système de communication homme–machine en Français. Rapport, Groupe d'Intelligence Artificielle, Université d'Aix-Marseille II.

Condon, J. H. y Thompson, K. (1982). Belle chess hardware. En Clarke, M. R. B. (Ed.), *Advances in Computer Chess 3*. Pergamon.

Congdon, C. B., Huber, M., Kortenkamp, D., Bidlack, C., Cohen, C., Huffman, S., Koss, F., Raschke, U. y Weymouth, T. (1992). CARMEL versus Flakey: A comparison of two robots. Tech. rep., American Association for Artificial Intelligence.

Conlisk, J. (1989). Three variants on the Allais example. *American Economic Review, 79*, 392–407.

Connell, J. (1989). *A Colony Architecture for an Artificial Creature*. Ph.D. thesis, Artificial Intelligence Laboratory, MIT.

Conway, D. y White, J. (2012). *Machine Learning for Hackers*. O'Reilly.

Cook, S. A. (1971). The complexity of theoremproving procedures. En *STOC-71*.

Cook, S. A. and Mitchell, D. (1997). Finding hard instances of the satisfiability problem: A survey. En Du, D., Gu, J., and Pardalos, P. (Eds.), *Satisfiability problems: Theory and applications*. American Mathematical Society.

Cooper, G. (1990). The computational complexity of probabilistic inference using Bayesian belief networks. *AIJ, 42*, 393–405.

Cooper, G. y Herskovits, E. (1992). A Bayesian method for the induction of probabilistic networks from data. *Machine Learning*, *9*, 309–347.

Copeland, J. (1993). *Artificial Intelligence: A Philosophical Introduction*. Blackwell.

Corbett-Davies, S. and Goel, S. (2018). The measure and mismeasure of fairness: A critical review of fair machine learning. arXiv:1808.00023.

Corbett-Davies, S., Pierson, E., Feller, A., Goel, S. y Huq, A. (2017). Algorithmic decision making and the cost of fairness. arXiv:1701.08230.

Cormen, T. H., Leiserson, C. E., Rivest, R. y Stein, C. (2009). *Introduction to Algorithms* (3a edición). MIT Press.

Cortes, C. y Vapnik, V. N. (1995). Support vector networks. *Machine Learning, 20*, 273–297.

Cournot, A. (Ed.). (1838). *Recherches sur les principes mathèmatiques de la thèorie des richesses*. L. Hachette, Paris.

Cover, T. y Thomas, J. (2006). *Elements of Information Theory* (2a edición). Wiley.

Cowan, J. D. y Sharp, D. H. (1988a). Neural nets. *Quarterly Reviews of Biophysics, 21*, 365–427.

Cowan, J. D. y Sharp, D. H. (1988b). Neural nets and artificial intelligence. *Daedalus, 117*, 85–121.

Cowell, R., Dawid, A. P., Lauritzen, S., and Spiegelhalter, D. J. (2002). *Probabilistic Networks and Expert Systems*. Springer.

Cox, I. (1993). A review of statistical data association techniques for motion correspondence. *IJCV, 10*, 53–66.

Cox, I. y Hingorani, S. L. (1994). An efficient implementation and evaluation of Reid's multiple hypothesis tracking algorithm for visual tracking. En *ICPR-94*.

Cox, I. y Wilfong, G. T. (Eds.). (1990). *Autonomous Robot Vehicles*. Springer Verlag.

Cox, R. T. (1946). Probability, frequency, and reasonable expectation. *American Journal of Physics, 14*, 1–13.

Craig, J. (1989). *Introduction to Robotics: Mechanics and Control* (2a edición). Addison-Wesley.

Craik, K. (1943). *The Nature of Explanation*. Cambridge University Press.

Cramton, P., Shoham, Y. y Steinberg, R. (Eds.). (2006). *Combinatorial Auctions*. MIT Press.

Craven, M., DiPasquo, D., Freitag, D., McCallum, A., Mitchell, T. M., Nigam, K. y Slattery, S. (2000). Learning to construct knowledge bases from the World Wide Web. *AIJ, 118*, 69–113.

Crawford, J. M. y Auton, L. D. (1993). Experimental results on the crossover point in satisfiability problems. En *AAAI-93*.

Crick, F. (1999). The impact of molecular biology on neuroscience. *Phil. Trans. Roy. Soc., B, 354*, 2021–2025.

Crick, F. y Koch, C. (2003). A framework for consciousness. *Nature Neuroscience, 6*, 119.

Crisan, D. y Doucet, A. (2002). A survey of convergence results on particle filtering methods for practitioners. *IEEE Trans. Signal Processing, 50*, 736–746.

Cristianini, N. y Hahn, M. (2007). *Introduction to Computational Genomics: A Case Studies Approach*. Cambridge University Press.

Cristianini, N. y Schölkopf, B. (2002). Support vector machines and kernel methods: The new generation of learning machines. *AIMag, 23*, 31–41.

Cristianini, N. y Shawe-Taylor, J. (2000). *An Introduction to Support Vector Machines and Other Kernel-Based Learning Methods*. Cambridge University Press.

Crockett, L. (1994). *The Turing Test and the Frame Problem: AI's Mistaken Understanding of Intelligence*. Ablex.

Croft, W. B., Metzler, D. y Strohman, T. (2010). *Search Engines: Information Retrieval in Practice*. Addison-Wesley.

Cross, S. E. y Walker, E. (1994). DART: Applying knowledge based planning and scheduling to crisis action planning. En Zweben, M. y Fox, M. S. (Eds.), *Intelligent Scheduling*. Morgan Kaufmann.

Cruse, A. (2011). *Meaning in Language: An Introduction to Semantics and Pragmatics*. Oxford University Press.

Culberson, J. y Schaeffer, J. (1996). Searching with pattern databases. En *Advances in Artificial Intelligence (Lecture Notes in Artificial Intelligence 1081)*. Springer-Verlag.

Culberson, J. y Schaeffer, J. (1998). Pattern databases. *Computational Intelligence, 14*, 318–334.

Cummins, D. y Allen, C. (1998). *The Evolution of Mind*. Oxford University Press.

Cushing, W., Kambhampati, S., Mausam y Weld, D. S. (2007). When is temporal planning really temporal? En *IJCAI-07*.

Cusumano-Towner, M. F., Saad, F., Lew, A. K. y Mansinghka, V. K. (2019). Gen: A general-purpose probabilistic programming system with programmable inference. En *PLDI-19*.

Cybenko, G. (1988). Continuous valued neural networks with two hidden layers are sufficient. Technical report, Department of Computer Science, Tufts University.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. Mathematics of Controls, *Signals, and Systems, 2*, 303–314.

Cyert, R. y de Groot, M. (1979). Adaptive utility. En Allais, M. y Hagen, O. (Eds.), *Expected Utility Hypothesis and the Allais Paradox*. D. Reidel.

Dagan, I., Glickman, O. y Magnini, B. (2005). The PASCAL recognising textual entailment challenge. En *Machine Learning Challenges Workshop*.

Daganzo, C. (1979). *Multinomial Probit: The Theory and Its Application to Demand Forecasting*. Academic Press.

Dagum, P. y Luby,M. (1993). Approximating probabilistic inference in Bayesian belief networks is NPhard. *AIJ, 60*, 141–153.

Dagum, P. y Luby, M. (1997). An optimal approximation algorithm for Bayesian inference. *AIJ, 93*, 1–27.

Dai, A. M. y Le, Q. V. (2016). Semi-supervised sequence learning. En *NeurIPS 28*.

Dalal, N. y Triggs, B. (2005). Histograms of oriented gradients for human detection. En *CVPR-05*.

Dalvi, N. N., Ré, C. y Suciu, D. (2009). Probabilistic databases. *CACM, 52*, 86–94.

Daly, R., Shen, Q. y Aitken, S. (2011). Learning Bayesian networks: Approaches and issues. *Knowledge Engineering Review, 26*, 99–157.

Damasio, A. R. (1999). *The Feeling of What Happens: Body and Emotion in the Making of Consciousness*. Houghton Mifflin.

Danaher, J. y McArthur, N. (2017). *Robot Sex: Social and Ethical Implications*. MIT Press.

Dantzig, G. B. (1949). Programming of interdependent activities: II. Mathematical model. *Econometrica, 17*, 200–211.

Darwiche, A. (2001). Recursive conditioning. *AIJ, 126*, 5–41.

Darwiche, A. y Ginsberg, M. L. (1992). A symbolic generalization of probability theory. En *AAAI-92*.

Darwiche, A. (2009). *Modeling and reasoning with Bayesian networks*. Cambridge University Press.

Darwin, C. (1859). *On The Origin of Species by Means of Natural Selection*. J. Murray.

Dasgupta, P., Chakrabarti, P. P. y de Sarkar, S. C. (1994). Agent searching in a tree and the optimality of iterative deepening. *AIJ, 71*, 195–208.

Dasgupta, P. y Maskin, E. (2008). On the robustness of majority rule. *Journal of the European Economic Association, 6*, 949–973.

Dauphin, Y., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S. y Bengio, Y. (2015). Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. En *NeurIPS 27*.

Davidson, D. (1980). *Essays on Actions and Events*. Oxford University Press.

Davidson, D. (1986). A nice derangement of epitaphs. *Philosophical Grounds of Rationality, 4*, 157–174.

Davis, E. (1986). *Representing and Acquiring Geographic Knowledge*. Pitman and Morgan Kaufmann.

Davis, E. (1990). *Representations of Commonsense Knowledge*. Morgan Kaufmann.

Davis, E. (2005). Knowledge and communication: A first-order theory. *AIJ, 166*, 81–140.

Davis, E. (2006). The expressivity of quantifying over regions. *J. Logic and Computation, 16*, 891–916.

Davis, E. (2007). Physical reasoning. En Van Harmelan, F., Lifschitz, V. y Porter, B. (Eds.), *The Handbook of Knowledge Representation*. Elsevier.

Davis, E. (2008). Pouring liquids: A study in commonsense physical reasoning. AIJ, 172.

Davis, E. (2017). Logical formalizations of commonsense reasoning: A survey. *JAIR, 59*, 651–723.

Davis, E. y Morgenstern, L. (2004). Introduction: Progress in formal commonsense reasoning. *AIJ, 153*, 1–12.

Davis, E. y Morgenstern, L. (2005). A first-order theory of communication and multi-agent plans. *J. Logic and Computation, 15*, 701–749.

Davis, M. (1957). A computer program for Presburger's algorithm. En *Proving Theorems (as Done by Man, Logician, or Machine)*. Proc. Summer Institute for Symbolic Logic. Second edition; publication date is 1960.

Davis, M., Logemann, G. y Loveland, D. (1962). A machine program for theorem-proving. *CACM, 5*, 394–397.

Davis, M. y Putnam, H. (1960). A computing procedure for quantification theory. *JACM, 7*, 201–215.

Dayan, P. (1992). The convergence of TD(λ) for general λ. *Machine Learning, 8*, 341–362.

Dayan, P. y Abbott, L. F. (2001). *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. MIT Press.

Dayan, P. y Hinton, G. E. (1993). Feudal reinforcement learning. En *NeurIPS 5*.

Dayan, P. y Niv, Y. (2008). Reinforcement learning and the brain: The good, the bad and the ugly. *Current Opinion in Neurobiology, 18*, 185–196.

De Condorcet, M. (1785). *Essay on the Application of Analysis to the Probability of Majority Decisions*. Imprimerie Royale.

De Dombal, F. T., Leaper, D. J., Horrocks, J. C. y Staniland, J. R. (1974). Human and computer-aided diagnosis of abdominal pain: Further report with emphasis on performance of clinicians. *British Medical Journal, 1*, 376–380.

De Dombal, F. T., Staniland, J. R. y Clamp, S. E. (1981). Geographical variation in disease presentation. *Medical Decision Making, 1*, 59–69.

De Farias, D. P. y Roy, B. V. (2003). The linear programming approach to approximate dynamic programming. *Operations Research, 51*, 839–1016.

De Finetti, B. (1937). Le prévision: ses lois logiques, ses sources subjectives. *Ann. Inst. Poincar'e, 7*, 1–68.

De Finetti, B. (1993). On the subjective meaning of probability. En Monari, P. y Cocchi, D. (Eds.), *Probabilita e Induzione*. Clueb.

De Freitas, J. F. G., Niranjan, M. y Gee, A. H. (2000). Sequential Monte Carlo methods to train neural network models. *Neural Computation, 12*, 933–953.

De Ghellinck, G. (1960). Les problèmes de dècisions séquentielles. *Cahiers du Centre d''Etudes de Recherche Op'erationnelle, 2*, 161–179.

De Kleer, J. (1975). Qualitative and quantitative knowledge in classicalmechanics. Tech. rep.,MIT Artificial Intelligence Laboratory.

De Kleer, J. (1989). A comparison of ATMS and CSP techniques. En *IJCAI-89*.

De Kleer, J. y Brown, J. S. (1985). A qualitative physics based on confluences. En Hobbs, J. R. y Moore, R. C. (Eds.), *Formal Theories of the Commonsense World*. Ablex.

De Marcken, C. (1996). *Unsupervised Language Acquisition*. Ph.D. thesis, MIT.

De Morgan, A. (1864). On the syllogism, No. IV, and on the logic of relations. *Transaction of the Cambridge Philosophical Society, X*, 331–358.

De Salvo Braz, R., Amir, E. y Roth, D. (2007). Lifted first-order probabilistic inference. En Getoor, L. y Taskar, B. (Eds.), *Introduction to Statistical Relational Learning*. MIT Press.

Deacon, T.W. (1997). *The Symbolic Species: The Coevolution of Language and the Brain*. W. W. Norton.

Deale, M., Yvanovich, M., Schnitzius, D., Kautz, D., Carpenter, M., Zweben, M., Davis, G. y Daun, B. (1994). The space shuttle ground processing scheduling system. En Zweben, M. y Fox, M. (Eds.), *Intelligent Scheduling*. Morgan Kaufmann.

Dean, J., Patterson, D. A., and Young, C. (2018). A new golden age in computer architecture: Empowering the machine-learning revolution. *IEEE Micro, 38*, 21–29.

Dean, T., Basye, K., Chekaluk, R. y Hyun, S. (1990). Coping with uncertainty in a control system for navigation and exploration. En *AAAI-90*.

Dean, T. y Boddy, M. (1988). An analysis of timedependent planning. En *AAAI-88*.

Dean, T., Firby, R. J. y Miller, D. (1990). Hierarchical planning involving deadlines, travel time, and resources. *Computational Intelligence, 6*, 381–398.

Dean, T., Kaelbling, L. P., Kirman, J. y Nicholson, A. (1993). Planning with deadlines in stochastic domains. En *AAAI-93*.

Dean, T. y Kanazawa, K. (1989a). A model for projection and action. En *IJCAI-89*.

Dean, T. y Kanazawa, K. (1989b). A model for reasoning about persistence and causation. *Computational Intelligence, 5*, 142–150.

Dean, T. y Wellman, M. P. (1991). *Planning and Control*. Morgan Kaufmann.

Dearden, R., Friedman, N. y Andre, D. (1999). Model-based Bayesian exploration. En *UAI-99*.

Dearden, R., Friedman, N. y Russell, S. J. (1998). Bayesian Q-learning. En *AAAI-98*.

Debevec, P., Taylor, C. y Malik, J. (1996). Modeling and rendering architecture from photographs: A hybrid geometry- and image-based approach. En *Proc. 23rd Annual Conference on Computer Graphics (SIGGRAPH)*.

Debreu, G. (1960). Topological methods in cardinal utility theory. En Arrow, K. J., Karlin, S. y Suppes, P. (Eds.), *Mathematical Methods in the Social Sciences, 1959*. Stanford University Press.

Dechter, A. y Dechter, R. (1987). Removing redundancies in constraint networks. En *AAAI-87*.

Dechter, R. (1990a). Enhancement schemes for constraint processing: Backjumping, learning and cutset decomposition. *AIJ, 41*, 273–312.

Dechter, R. (1990b). On the expressiveness of networks with hidden variables. En *AAAI-90*.

Dechter, R. (1999). Bucket elimination: A unifying framework for reasoning. *AIJ, 113*, 41–85.

Dechter, R. y Pearl, J. (1985). Generalized best-first search strategies and the optimality of A*. *JACM, 32*, 505–536.

Dechter, R. y Pearl, J. (1987). Network-based heuristics for constraint-satisfaction problems. *AIJ, 34*, 1–38.

Dechter, R. y Pearl, J. (1989). Tree clustering for constraint networks. *AIJ, 38*, 353–366.

Dechter, R. y Rish, I. (2003). Mini-buckets: A general scheme for bounded inference. JACM, 50, 107–153.

Dechter, R. (2003). Constraint Processing. Morgan Kaufmann.

Dechter, R. (2019). *Reasoning with Probabilistic and Deterministic Graphical Models: Exact Algorithms* (2a edición). Morgan & Claypool.

Dechter, R. y Frost, D. (2002). Backjump-based backtracking for constraint satisfaction problems. *AIJ, 136*, 147–188.

Dechter, R. y Mateescu, R. (2007). AND/OR search spaces for graphical models. *AIJ, 171*, 73–106.

DeCoste, D. y Schölkopf, B. (2002). Training invariant support vector machines. *Machine Learning, 46*, 161–190.

Dedekind, R. (1888). *Was sind und was sollen die Zahlen*. Braunschweig, Germany.

Deerwester, S. C., Dumais, S. T., Landauer, T. K., Furnas, G. W. y Harshman, R. A. (1990). Indexing by latent semantic analysis. *J. American Society for Information Science, 41*, 391–407.

DeGroot,M. H. (1970). *Optimal Statistical Decisions*. McGraw-Hill.

DeGroot, M. H. y Schervish, M. J. (2001). *Probability and Statistics* (3a edición). Addison Wesley.

Dehaene, S. (2014). *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Penguin Books.

Del Moral, P., Doucet, A. y Jasra, A. (2006). Sequential Monte Carlo samplers. *J. Royal Statistical Society, 68*, 411–436.

Del Moral, P. (2004). *Feynman–Kac Formulae, Genealogical and Interacting Particle Systems with Applications*. Springer-Verlag.

Delgrande, J. y Schaub, T. (2003). On the relation between Reiter's default logic and its (major) variants. En *Seventh European Conference on Symbolic and Quantitative Approaches to Reasoning with Uncertainty*.

Delling, D., Sanders, P., Schultes, D., y Wagner, D. (2009). Engineering route planning algorithms. En Lerner, J., Wagner, D. y Zweig, K. (Eds.), *Algorithmics, LNCS*. Springer-Verlag.

Dempster, A. P. (1968). A generalization of Bayesian inference. *J. Royal Statistical Society, 30 (Series B)*, 205–247.

Dempster, A. P., Laird, N. y Rubin, D. (1977). Maximum likelihood from incomplete data via the EM algorithm. *J. Royal Statistical Society, 39 (Series B)*,1–38.

Denardo, E. V. (1967). Contraction mappings in the theory underlying dynamic programming. *SIAM Review, 9*, 165–177.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. y Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. En *CVPR-09*.

Deng, L. (2016). Deep learning: From speech recognition to language and multimodal processing. *APSIPA Transactions on Signal and Information Processing, 5*.

Deng, L., Yu, D. *et al*. (2014). Deep learning: Methods and applications. *Foundations and Trends in Signal Processing, 7*, 197–387.

Deng, X. y Papadimitriou, C. H. (1990). Exploring an unknown graph. En *FOCS-90*.

Deng, X. y Papadimitriou, C. H. (1994). On the complexity of cooperative solution concepts. *Mathematics of Operations Research, 19*, 257–266.

Denney, E., Fischer, B., and Schumann, J. (2006). An empirical evaluation of automated theorem provers in software certification. *Int. J. AI Tools, 15*, 81–107.

D'Épenoux, F. (1963). A probabilistic production and inventory problem. *A probabilistic production and inventory problem, 10*, 98–108.

Dervovic, D., Herbster, M., Mountney, P., Severini, S., Usher, N. y Wossnig, L. (2018). Quantum linear systems algorithms: A primer. arXiv:1802.08227.

Descartes, R. (1637). Discourse on method. En Cottingham, J., Stoothoff, R. y Murdoch, D. (Eds.), *The Philosophical Writings of Descartes*, Vol. I. Cambridge University Press, Cambridge.

Descotte, Y. y Latombe, J.-C. (1985). Making compromises among antagonist constraints in a planner. *AIJ, 27*, 183–217.

Deshpande, I., Hu, Y.-T., Sun, R., Pyrros, A., Siddiqui, N., Koyejo, S., Zhao, Z., Forsyth, D. y Schwing, A. (2019). Max-sliced Wasserstein distance and its use for GANs. En *CVPR-19*.

Deutscher, G. (2010). *Through the Language Glass: Why the World Looks Different in Other Languages*. Metropolitan Books.

Devlin, J., Chang, M.-W., Lee, K. y Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805.

Devlin, K. (2018). *Turned On: Science, Sex and Robots*. Bloomsbury.

Devroye, L. (1987). *A course in density estimation*. Birkhauser.

Dias, M. B., Zlot, R., Kalra, N. y Stentz, A. (2006). Market-based multirobot coordination: A survey and analysis. *Proc. IEEE, 94*, 1257–1270.

Dickmanns, E. D. and Zapp, A. (1987). Autonomous high speed road vehicle guidance by computer vision. En *Automatic Control—World Congress, 1987: Selected Papers from the 10th Triennial World Congress of the International Federation of Automatic Control*.

Dietterich, T. (2000). Hierarchical reinforcement learning with the MAXQ value function decomposition. *JAIR, 13*, 227–303.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*, 269–271.

Dijkstra, E. W. (1984). The threats to computing science. En *ACM South Central Regional Conference*.

Ding, Y., Sohn, J. H., Kawczynski, M. G., Trivedi, H., Harnish, R., Jenkins, N. W., Lituiev, D., Copeland, T. P., Aboian, M. S., Mari Aparici, C. *et al*. (2018). A deep learning model to predict a diagnosis of alzheimer disease by using 18F-FDG PET of the brain. *Radiology*, p. 180958.

Dinh, H., Russell, A., and Su, Y. (2007). On the value of good advice: The complexity of A* with accurate heuristics. En *AAAI-07*.

Dissanayake, G., Newman, P., Clark, S., Durrant-Whyte, H. y Csorba, M. (2001). A solution to the simultaneous localisation and map building (SLAM) problem. *IEEE Transactions on Robotics and Automation, 17*, 229–241.

Dittmer, S. y Jensen, F. (1997). Myopic value of information in influence diagrams. En *UAI-97*.

Do, M. y Kambhampati, S. (2003). Planning as constraint satisfaction: solving the planning graph by compiling it into CSP. *AIJ, 132*, 151–182.

Do, M. B. y Kambhampati, S. (2001). Sapa: A domain-independent heuristic metric temporal planner. En *ECP-01*.

Doctorow, C. (2001). Metacrap: Putting the torch to seven straw-men of the meta-utopia. www.well.com/doctorow/metacrap.htm.

Doctorow, C. y Stross, C. (2012). *The Rapture of the Nerds: A Tale of the Singularity, Posthumanity, and Awkward Social Situations*. Tor Books.

Dodd, L. (1988). The inside/outside algorithm: Grammatical inference applied to stochastic context-free grammars. Tech. rep., Royal Signals and Radar Establishment, Malvern.

Domingos, P. y Pazzani,M. (1997). On the optimality of the simple Bayesian classifier under zero–one loss. *Machine Learning, 29*, 103–30.

Domingos, P. (2012). A few useful things to know about machine learning. *Commun. ACM, 55(10)*, 78–87.

Domingos, P. (2015). *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World*. Basic Books.

Dong, X., Gabrilovich, E., Heitz, G., Horn, W., Lao, N., Murphy, K., Strohmann, T., Sun, S. y Zhang, W. (2014). Knowledge vault: A web-scale approach to probabilistic knowledge fusion. En *KDD-14*.

Doorenbos, R. (1994). Combining left and right unlinking for matching a large number of learned rules. En *AAAI-94*.

Doran, J. y Michie, D. (1966). Experiments with the graph traverser program. *Proc. Roy. Soc., 294, Series A*, 235–259.

Dorf, R. C. y Bishop, R. H. (2004). *Modern Control Systems* (10a edición). Prentice-Hall.

Dorigo, M., Birattari, M., Blum, C., Clerc, M., Stützle, T. y Winfield, A. (2008). *Ant Colony Optimization and Swarm Intelligence: 6th International Conference, ANTS 2008, Brussels, Belgium, September 22-24, 2008*, Proceedings, Vol. 5217. Springer-Verlag.

Doshi-Velez, F. y Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv:1702.08608.

Doucet, A. (1997). *Monte Carlo methods for Bayesian estimation of hidden Markov models: Application to radiation signals*. Ph.D. thesis, Université de Paris-Sud.

Doucet, A., de Freitas, J. F. G. y Gordon, N. (2001). *Sequential Monte Carlo Methods in Practice*. Springer-Verlag.

Doucet, A., de Freitas, J. F. G., Murphy, K. y Russell, S. J. (2000). Rao-Blackwellised particle filtering for dynamic Bayesian networks. En *UAI-00*.

Doucet, A. y Johansen, A. M. (2011). A tutorial on particle filtering and smoothing: Fifteen years later. En Crisan, D. y Rozovskii, B. (Eds.), *Oxford Handbook of Nonlinear Filtering*. Oxford.

Dowty, D., Wall, R. y Peters, S. (1991). *Introduction to Montague Semantics*. D. Reidel.

Doyle, J. (1979). A truth maintenance system. *AIJ, 12*, 231–272.

Doyle, J. (1983).What is rational psychology? Toward a modern mental philosophy. *AIMag, 4*, 50–53.

Drabble, B. (1990). Mission scheduling for spacecraft: Diaries of T-SCHED. En *Expert Planning Systems*. Institute of Electrical Engineers.

Dragan, A. D., Lee, K. C. y Srinivasa, S. (2013). Legibility and predictability of robot motion. En *HRI-13*.

Dredze, M., Crammer, K. y Pereira, F. (2008). Confidence-weighted linear classification. En *ICML-08*.

Dressel, J. y Farid, H. (2018). The accuracy, fairness, and limits of predicting recidivism. *Science Advances, 4*, eaao5580.

Dreyfus, H. L. (1972). *What Computers Can't Do: A Critique of Artificial Reason*. Harper and Row.

Dreyfus, H. L. (1992). *What Computers Still Can't Do: A Critique of Artificial Reason*. MIT Press.

Dreyfus, H. L. y Dreyfus, S. E. (1986). Mind over Machine: The Power of Human Intuition and Expertise in the Era of the Computer. Blackwell.

Dreyfus, S. E. (1962). The numerical solution of variational problems. *J. Math. Anal. and Appl., 5*, 30–45.

Dreyfus, S. E. (1969). An appraisal of some shortest-paths algorithms. *Operations Research, 17*, 395–412.

Dreyfus, S. E. (1990). Artificial neural networks, back propagation, and the Kelley–Bryson gradient procedure. *J. Guidance, Control, and Dynamics, 13*, 926–928.

Du, S. S., Lee, J. D., Li, H., Wang, L. y Zhai, X. (2018). Gradient descent finds global minima of deep neural networks. arXiv:1811.03804.

Dubois, D. y Prade, H. (1994). A survey of belief revision and updating rules in various uncertainty models. *Int. J. Intelligent Systems, 9*, 61–100.

Duda, R. O. y Hart, P. E. (1973). *Pattern classification and scene analysis*. Wiley.

Duda, R. O., Hart, P. E. y Stork, D. G. (2001). *Pattern Classification* (2a edición). Wiley.

Dudek, G. y Jenkin, M. (2000). *Computational Principles of Mobile Robotics*. Cambridge University Press.

Duffy, D. (1991). *Principles of Automated Theorem Proving*. John Wiley & Sons.

Dunn, H. L. (1946). Record linkage". *Am. J. Public Health, 36*, 1412–1416.

Durfee, E. H. y Lesser, V. R. (1989). Negotiating task decomposition and allocation using partial global planning. En Huhns, M. y Gasser, L. (Eds.), *Distributed AI*, Vol. 2. Morgan Kaufmann.

Durme, B. V. y Pasca, M. (2008). Finding cars, goddesses and enzymes: Parametrizable acquisition of labeled instances for open-domain information extraction. En *AAAI-08*.

Dwork, C. (2008). Differential privacy: A survey of results. En *International Conference on Theory and Applications of Models of Computation*.

Dwork, C., Hardt, M., Pitassi, T., Reingold, O. y Zemel, R. (2012). Fairness through awareness. En *Proc. 3rd innovations in theoretical computer science conference*.

Dwork, C., Roth, A. *et al.* (2014). The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science, 9*, 211–407.

Dyson, F. (2004). A meeting with Enrico Fermi. *Nature, 427*, 297.

Dyson, G. (1998). *Darwin among the machines: the evolution of global intelligence*. Perseus Books.

Earley, J. (1970). An efficient context-free parsing algorithm. *CACM, 13*, 94–102.

Ebendt, R. y Drechsler, R. (2009). Weighted A* search–unifying view and application. *AIJ, 173*, 1310–1342.

Eckerle, J., Chen, J., Sturtevant, N. R., Zilles, S. y Holte, R. C. (2017). Sufficient conditions for node expansion in bidirectional heuristic search. En *ICAPS-17*.

Eckhouse, L., Lum, K., Conti-Cook, C. y Ciccolini, J. (2019). Layers of bias: A unified approach for understanding problems with risk assessment. *Criminal Justice and Behavior, 46*, 185–209.

Edelkamp, S. (2009). Scaling search with symbolic pattern databases. En *Model Checking and Artificial Intelligence (MOCHART)*.

Edelkamp, S. y Schrödl, S. (2012). *Heuristic Search*. Morgan Kaufmann.

Edmonds, J. (1965). Paths, trees, and flowers. *Canadian J. of Mathematics, 17*, 449–467.

Edwards, P. (Ed.). (1967). *The Encyclopedia of Philosophy*. Macmillan.

Eiter, T., Leone, N., Mateis, C., Pfeifer, G. y Scarcello, F. (1998). The KR system dlv: Progress report, comparisons and benchmarks. En *KR-98*.

Elio, R. (Ed.). (2002). *Common Sense, Reasoning, and Rationality*. Oxford University Press.

Elkan, C. (1997). Boosting and naive Bayesian learning. Tech. rep., Department of Computer Science and Engineering, University of California, San Diego.

Ellsberg, D. (1962). *Risk, Ambiguity, and Decision*. Ph.D. thesis, Harvard University.

Elman, J. L. (1990). Finding structure in time. *Cognitive Science, 14*, 179–211.

Elman, J. L., Bates, E., Johnson, M., Karmiloff-Smith, A., Parisi, D. y Plunkett, K. (1997). *Rethinking Innateness*. MIT Press.

Elo, A. E. (1978). *The rating of chess players: Past and present*. Arco Publishing.

Elsken, T., Metzen, J.H. y Hutter, F. (2018). Neural architecture search: A survey. arXiv:1808.05377.

Empson, W. (1953). *Seven Types of Ambiguity*. New Directions.

Enderton, H. B. (1972). *A Mathematical Introduction to Logic*. Academic Press.

Engel, J., Resnick, C., Roberts, A., Dieleman, S., Norouzi, M., Eck, D. y Simonyan, K. (2017). Neural audio synthesis of musical notes with wavenet autoencoders. En *Proc. 34th International Conference on Machine Learning-Volume 70*.

Epstein, R., Roberts, G. y Beber, G. (Eds.). (2008). *Parsing the Turing test*. Springer.

Erdmann, M. A. y Mason, M. (1988). An exploration of sensorless manipulation. *IEEE Journal of Robotics and Automation, 4*, 369–379.

Ernst, H.A. (1961). *MH-1, a Computer-Operated Mechanical Hand*. Ph.D. thesis, MIT.

Ernst, M., Millstein, T. y Weld, D. S. (1997). Automatic SAT-compilation of planning problems. En *IJCAI-97*.

Erol, K., Hendler, J. y Nau, D. S. (1994). HTN planning: Complexity and expressivity. En *AAAI-94*.

Erol, K., Hendler, J. Nau, D. S. (1996). Complexity results for HTN planning. *AIJ, 18*, 69–93.

Erol, Y., Li, L., Ramsundar, B. y Russell, S. J. (2013). The extended parameter filter. En *ICML-13*.

Erol, Y., Wu, Y., Li, L. y Russell, S. J. (2017). A nearly-black-box online algorithm for joint parameter and state estimation in temporal models. En *AAAI-17*.

Esteva, A., Kuprel, B., Novoa, R. A., Ko, J., Swetter, S. M., Blau, H. M. y Thrun, S. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature, 542*, 115.

Etzioni, A. (2004). *From Empire to Community: A New Approach to International Relation*. Palgrave Macmillan.

Etzioni, A. y Etzioni, O. (2017a). Incorporating ethics into artificial intelligence. *The Journal of Ethics, 21*, 403–418.

Etzioni, A. y Etzioni, O. (2017b). Should artificial intelligence be regulated? *Issues in Science and Technology, Summer*.

Etzioni, O. (1989). Tractable decision-analytic control. En Proc. *First International Conference on Knowledge Representation and Reasoning*.

Etzioni, O., Banko, M., Soderland, S. y Weld, D. S. (2008). Open information extraction from the web. *CACM, 51*.

Etzioni, O., Hanks, S., Weld, D. S., Draper, D., Lesh, N. y Williamson, M. (1992). An approach to planning with incomplete information. En *KR-92*.2

Etzioni, O., Banko, M. y Cafarella, M. J. (-2006). Machine reading. En *AAAI-06*.

Etzioni, O., Cafarella, M. J., Downey, D., Popescu, A.-M., Shaked, T., Soderland, S., Weld, D. S. y Yates, A. (2005). Unsupervised named-entity extraction from the web: An experimental study. *AIJ, 165*(1), 91–134.

Evans, T. G. (1968). A program for the solution of a class of geometric-analogy intelligence-test questions. En Minsky, M. L. (Ed.), *Semantic Information Processing*. MIT Press.

Fagin, R., Halpern, J. Y., Moses, Y. y Vardi, M. Y. (1995). *Reasoning about Knowledge*. MIT Press.

Fahlman, S. E. (1974). A planning system for robot construction tasks. *AIJ, 5*, 1–49.

Faugeras, O. (1992). What can be seen in three dimensions with an uncalibrated stereo rig? En *ECCV, Vol. 588 of Lecture Notes in Computer Science*.

Faugeras, O., Luong, Q.-T. y Papadopoulo, T. (2001). *The Geometry of Multiple Images*. MIT Press.

Fawcett, T. y Provost, F. (1997). Adaptive fraud detection. *Data mining and knowledge discovery, 1*, 291–316.

Fearing, R. S. y Hollerbach, J. M. (1985). Basic solid mechanics for tactile sensing. *Int. J. Robotics Research, 4*, 40–54.

Featherstone, R. (1987). *Robot Dynamics Algorithms*. Kluwer Academic Publishers.

Feigenbaum, E. A. (1961). The simulation of verbal learning behavior. *Proc. Western Joint Computer Conference, 19*, 121–131.

Feigenbaum, E. A., Buchanan, B. G. y Lederberg, J. (1971). On generality and problem solving: A case study using the DENDRAL program. En Meltzer, B. y Michie, D. (Eds.), *Machine Intelligence 6*. Edinburgh University Press.

Feldman, J. y Sproull, R. F. (1977). Decision theory and artificial intelligence II: The hungry monkey. Technical report, Computer Science Department, University of Rochester.

Feldman, J. y Yakimovsky, Y. (1974). Decision theory and artificial intelligence I: Semantics-based region analyzer. *AIJ, 5*, 349–371.

Feldman, M. (2017). Oak Ridge readies Summit supercomputer for 2018 debut. *Top500.org, bit.ly/2ERRFr9*.

Fellbaum, C. (2001). *Wordnet: An Electronic Lexical Database*. MIT Press.

Fellegi, I. y Sunter, A. (1969). A theory for record linkage". *JASA, 64*, 1183–1210.

Felner,A., Korf, R. E. y Hanan, S. (2004). Additive pattern database heuristics. *JAIR, 22*, 279–318.

Felner, A. (2018). Position paper: Using early goal test in A*. En *Eleventh Annual Symposium on Combinatorial Search*.

Felner, A., Korf, R. E., Meshulam, R. y Holte, R. C. (2007). Compressed pattern databases. *JAIR, 30*.

Felner, A., Zahavi, U., Holte, R. C., Schaeffer, J., Sturtevant, N. R. y Zhang, Z. (2011). Inconsistent heuristics in theory and practice. *AIJ, 175*, 1570–1603.

Felzenszwalb, P. y McAllester, D. A. (2007). The generalized A* architecture. *JAIR*.

Fenton, N. y Neil, M. (2018). *Risk Assessment and Decision Analysis with Bayesian Networks* (2a edición). Chapman and Hall.

Ferguson, T. (1992). Mate with knight and bishop in kriegspiel. *Theoretical Computer Science, 96*, 389–403.

Ferguson, T. (1995). Mate with the two bishops in kriegspiel. www.math.ucla.edu/˜tom/papers.

Ferguson, T. (2001). *Optimal Stopping and Applications*. www.math.ucla.edu/ tom/Stopping/Contents.html.

Ferguson, T. (1973). Bayesian analysis of some nonparametric problems. *Annals of Statistics, 1*, 209–230.

Fern, A., Natarajan, S., Judah, K. y Tadepalli, P. (2014). A decision-theoretic model of assistance. *JAIR, 50*, 71–104.

Fernandez, J. M. F. y Mahlmann, T. (2018). The Dota 2 bot competition. *IEEE Transactions on Games*.

Ferraris, P. y Giunchiglia, E. (2000). Planning as satisability in nondeterministic domains. En *AAAI-00*.

Ferriss, T. (2007). *The 4-Hour Workweek*. Crown.

Ferrucci, D., Brown, E., Chu-Carroll, J., Fan, J., Gondek, D., Kalyanpur, A. A., Lally, A., Murdock, J. W., Nyberg, E., Prager, J., Schlaefer, N. y Welty, C. (2010). Building Watson: An overview of the DeepQA project. *AI Magazine, Fall*.

Fikes, R. E., Hart, P. E. y Nilsson, N. J. (1972). Learning and executing generalized robot plans. *AIJ, 3*, 251–288.

Fikes, R. E. y Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *AIJ, 2*, 189–208.

Fikes, R. E. y Nilsson, N. J. (1993). STRIPS, a retrospective. *AIJ, 59*, 227–232.

Fine, S., Singer, Y. y Tishby, N. (1998). The hierarchical hidden Markov model: Analysis and applications. *Machine Learning, 32*.

Finn, C., Abbeel, P. y Levine, S. (2017). Modelagnostic meta-learning for fast adaptation of deep networks. En *Proc. 34th International Conference on Machine Learning-Volume 70*.

Finney, D. J. (1947). *Probit analysis: A statistical treatment of the sigmoid response curve*. Cambridge University Press.

Firoiu, V., Whitney, W. F. y Tenenbaum, J. B. (2017). Beating the world's best at Super Smash Bros. with deep reinforcement learning. arXiv:1702.06230.

Firth, J. (1957). *Papers in Linguistics*. Oxford University Press.

Fisher, R. A. (1922). On the mathematical foundations of theoretical statistics. *Phil. Trans. Roy. Soc., A, 222*, 309–368.

Fix, E. y Hodges, J. L. (1951). Discriminatory analysis-Nonparametric discrimination: Consistency properties. Tech. rep., USAF School of Aviation Medicine.

Floreano, D., Zufferey, J. C., Srinivasan, M. V. y Ellington, C. (2009). *Flying Insects and Robots*. Springer.

Floyd, R. W. (1962). Algorithm 97: Shortest path. *CACM, 5*, 345.

Fogel, D. B. (2000). *Evolutionary Computation: Toward a New Philosophy ofMachine Intelligence*. IEEE Press.

Fogel, L. J., Owens, A. J. y Walsh, M. J. (1966). *Artificial Intelligence through Simulated Evolution*. Wiley.

Forbes, J., Huang, T., Kanazawa, K. y Russell, S. J. (1995). The BATmobile: Towards a Bayesian automated taxi. En *IJCAI-95*.

Forbus, K. D. (1985). Qualitative process theory. En *Bobrow, D. (Ed.), Qualitative Reasoning About Physical Systems*. MIT Press.

Forbus, K. D. y De Kleer, J. (1993). *Building Problem Solvers*. MIT Press.

Forbus, K. D., Hinrichs, T. R., De Kleer, J. y Usher, J. M. (2010). FIRE: Infrastructure for experience based systems with common sense. En *AAAI Fall Symposium: Commonsense Knowledge*.

Ford, K. M. y Hayes, P. J. (1995). Turing Test considered harmful. En *IJCAI-95*.

Ford, L. R. (1956). Network flow theory. Tech. rep., RAND Corporation.

Ford, M. (2015). *Rise of the Robots: Technology and the Threat of a Jobless Future*. Basic Books.

Ford, M. (2018). *Architects of Intelligence*. Packt.

Forestier, J.-P. y Varaiya, P. (1978). Multilayer control of large Markov chains. *IEEE Transactions on Automatic Control, 23*, 298–304.

Forgy, C. (1981). OPS5 user's manual. Technical report, Computer Science Department, Carnegie-Mellon University.

Forgy, C. (1982). A fast algorithm for the many patterns/many objects match problem. *AIJ, 19*, 17–37.

Forster, E. M. (1909). *The Machine Stops*. Sheba Blake.

Forsyth, D. y Ponce, J. (2002). *Computer Vision: A Modern Approach*. Prentice Hall.

Fouhey, D., Kuo, W.-C., Efros, A. y Malik, J. (2018). From lifestyle vlogs to everyday interactions. En *CVPR-18*.

Fourier, J. (1827). Analyse des travaux de l'Académie Royale des Sciences, pendant l'année 1824; partie mathématique. *Histoire de l'Académie Royale des Sciences de France, 7*, xlvii–lv.

Fox, C. y Tversky, A. (1995). Ambiguity aversion and comparative ignorance. *Quarterly Journal of Economics, 110*, 585–603.

Fox, D., Burgard, W., Dellaert, F. y Thrun, S. (1999). Monte Carlo localization: Efficient position estimation for mobile robots. En *AAAI-99*.

Fox, M. S. (1990). Constraint-guided scheduling: A short history of research at CMU. *Computers in Industry, 14*, 79–88.

Fox, M. S., Allen, B. y Strohm, G. (1982). Job shop scheduling: An investigation in constraint-directed reasoning. En *AAAI-82*.

Franco, J. y Paull, M. (1983). Probabilistic analysis of the Davis Putnam procedure for solving the satisfiability problem. *Discrete Applied Mathematics, 5*, 77–87.

Francois-Lavet, V., Henderson, P., Islam, R., Bellemare, M. G. y Pineau, J. (2018). An introduction to deep reinforcement learning. *Foundations and Trends in Machine Learning, 11*, 219–354.

Frank, I., Basin, D. A. y Matsubara, H. (1998). Finding optimal strategies for imperfect information games. En *AAAI-98*.

Frank, R. H. y Cook, P. J. (1996). *The Winner-Take-All Society*. Penguin.

Frans, K., Ho, J., Chen, X., Abbeel, P. y Schulman, J. (2018). Meta learning shared hierarchies. En *ICLR-18*.

Franz, A. y Brants, T. (2006). All our n-gram are belong to you. Google blog, ai.googleblog.com/2006/08/all-our-n-gram-arebelong-to-you.html.

Frege, G. (1879). *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens*. Halle, Berlin. English translation appears in van Heijenoort (1967).

Freitag, D. y McCallum, A. (2000). Information extraction with hmm structures learned by stochastic optimization. En *AAAI-00*.

Freuder, E. C. (1978). Synthesizing constraint expressions. *CACM, 21*, 958–966.

Freuder, E. C. (1982). A sufficient condition for backtrack-free search. *JACM, 29*, 24–32.

Freuder, E. C. (1985). A sufficient condition for backtrack-bounded search. *JACM, 32*, 755–761.

Freund, Y. y Schapire, R. E. (1996). Experiments with a new boosting algorithm. En *ICML-96*.

Freund, Y. y Schapire, R. E. (1999). Large margin classification using the perceptron algorithm. *Machine Learning, 37*, 277–296.

Frey, B. J. (1998). *Graphical models for machine learning and digital communication*. MIT Press.

Frey, C. B. y Osborne, M. A. (2017). The future of employment: How susceptible are jobs to computerisation? *Technological forecasting and social change, 114*, 254–280.

Friedberg, R. M. (1958). A learning machine: Part I. *IBM Journal of Research and Development, 2*, 2–13.

Friedberg, R. M., Dunham, B., and North, T. (1959). A learning machine: Part II. *IBM Journal of Research and Development, 3*, 282–287.

Friedman, G. J. (1959). Digital simulation of an evolutionary process. *General Systems Yearbook, 4*, 171–184.

Friedman, J., Hastie, T. y Tibshirani, R. (2000). Additive logistic regression: A statistical view of boosting. *Annals of Statistics, 28*, 337–374.

Friedman, J. (2001). Greedy function approximation: A gradient boosting machine. *Annals of statistics, 29*, 1189–1232.

Friedman, N. (1998). The Bayesian structural EM algorithm. En *UAI-98*.

Friedman, N. y Goldszmidt, M. (1996). Learning Bayesian networks with local structure. En *UAI-96*.

Friedman, N. y Koller, D. (2003). Being Bayesian about Bayesian network structure: A Bayesian approach to structure discovery in Bayesian networks. *Machine Learning, 50*, 95–125.

Friedman, N., Murphy, K. y Russell, S. J. (1998). Learning the structure of dynamic probabilistic networks. En *UAI-98*.

Friedman, N. (2004). Inferring cellular networks using probabilistic graphical models. *Science, 303*.

Fruhwirth, T. y Abdennadher, S. (2003). *Essentials of constraint programming*. Cambridge University Press.

Fuchs, J. J., Gasquet, A., Olalainty, B. y Currie, K. W. (1990). PlanERS-1: An expert planning system for generating spacecraft mission plans. En *First International Conference on Expert Planning Systems*. Institute of Electrical Engineers.

Fudenberg, D. y Tirole, J. (1991). *Game theory*. MIT Press.

Fukunaga, A. S., Rabideau, G., Chien, S. y Yan, D. (1997). ASPEN: A framework for automated planning and scheduling of spacecraft control and operations. En *Proc. International Symposium on AI, Robotics and Automation in Space*.

Fukushima, K. (1980). Neocognitron: A selforganizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics, 36*, 193–202.

Fukushima, K. y Miyake, S. (1982). Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition. En *Competition and cooperation in neural nets*. Springer.

Fuller, S. B., Straw, A. D., Peek, M. Y., Murray, R.M. y Dickinson, M. H. (2014). Flying *Drosophila* stabilize their vision-based velocity controller by sensing wind with their antennae. *Proc. National Academy of Sciences of the United States of America, 111 13*, E1182–91.

Fung, C., Yoon, C. J. M. y Beschastnikh, I. (2018). Mitigating sybils in federated learning poisoning. arXiv:1808.04866.

Fung, R. y Chang, K. C. (1989).Weighting and integrating evidence for stochastic simulation in Bayesian networks. En *UAI 5*.

Gaddum, J. H. (1933). Reports on biological standard III: Methods of biological assay depending on a quantal response. Special report series of the medical research council, Medical Research Council.

Gaifman, H. (1964a). Concerning measures in first order calculi. *Israel J. Mathematics, 2*, 1–18.

Gaifman, H. (1964b). Concerning measures on Boolean algebras. *Pacific J. Mathematics, 14*, 61–73.

Gallaire, H. y Minker, J. (Eds.). (1978). *Logic and Databases*. Plenum.

Gallier, J. H. (1986). *Logic for Computer Science: Foundations of Automatic Theorem Proving*. Harper and Row.

Galton, F. (1886). Regression towards mediocrity in hereditary stature. *J. Anthropological Institute of Great Britain and Ireland, 15*, 246–263.

Gamba, A., Gamberini, L., Palmieri, G. y Sanna, R. (1961). Further experiments with PAPA. *Nuovo Cimento Supplemento, 20*, 221–231.

Gandomi, A. y Haider, M. (2015). Beyond the hype: Big data concepts,methods, and analytics. *International journal of information management, 35*, 137–144.

Gao, J. (2014). Machine learning applications for data center optimization. Google Research.

García, J. and Fernández, F. (2015). A comprehensive survey on safe reinforcement learning. *JMLR, 16*, 1437–1480.

Gardner, M. (1968). *Logic Machines, Diagrams and Boolean Algebra*. Dover.

Garey, M. R. y Johnson, D. S. (1979). *Computers and Intractability*. W. H. Freeman.

Gaschnig, J. (1977). A general backtrack algorithm that eliminates most redundant tests. En *IJCAI-77*.

Gaschnig, J. (1979). Performance measurement and analysis of certain search algorithms. Technical report, Computer Science Department, Carnegie-Mellon University.

Gasser, R. (1995). *Efficiently harnessing computational resources for exhaustive search*. Ph.D. thesis, ETH Zürich.

Gat, E. (1998). Three-layered architectures. In Kortenkamp, D., Bonasso, R. P. y Murphy, R. (Eds.), *AI-based Mobile Robots: Case Studies of Successful Robot Systems*. MIT Press.

Gatys, L. A., Ecker, A. S. y Bethge,M. (2016). Image style transfer using convolutional neural networks. En *CVPR-16*.

Gauci, J., Conti, E., Liang, Y., Virochsiri, K., He, Y., Kaden, Z., Narayanan, V. y Ye, X. (2018). Horizon: Facebook's open source applied reinforcement learning platform. arXiv:1811.00260.

Gauss, C. F. (1809). *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Sumtibus F. Perthes et I. H. Besser, Hamburg.

Gauss, C. F. (1829). *Beiträge zur theorie der algebraischen gleichungen. Werke, 3*, 71–102.

Gazzaniga, M. (2018). *The Consciousness Instinct*. Farrar, Straus and Girou.

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H. M., III, H. D. y Crawford, K. (2018). Datasheets for datasets. arXiv:1803.09010.

Geiger, D., Verma, T. y Pearl, J. (1990a). d-separation: From theorems to algorithms. En Henrion, M., Shachter, R. D., Kanal, L. N. y Lemmer, J. F. (Eds.), *UAI-90*. Elsevier.

Geiger, D., Verma, T. y Pearl, J. (1990b). Identifying independence in Bayesian networks. *Networks, 20*, 507–534.

Gelb, A. (1974). *Applied Optimal Estimation*. MIT Press.

Gelernter, H. (1959). Realization of a geometry-theorem proving machine. En *Proc. an International Conference on Information Processing*. UNESCO House.

Gelfond, M. y Lifschitz, V. (1988). Compiling circumscriptive theories into logic programs. En *Non-Monotonic Reasoning: 2nd International Workshop Proceedings*.

Gelfond, M. (2008). Answer sets. En van Harmelan, F., Lifschitz, V., y Porter, B. (Eds.), *Handbook of Knowledge Representation*. Elsevier.

Gelman, A. (2004). Exploratory data analysis for complex models. *Journal of Computational and Graphical Statistics, 13*, 755–779.

Gelman, A., Carlin, J. B., Stern, H. S. y Rubin, D. (1995). *Bayesian Data Analysis*. Chapman & Hall.

Geman, S. y Geman, D. (1984). Stochastic relaxation, Gibbs distributions, and Bayesian restoration of images. *PAMI, 6*, 721–741.

Gene Ontology Consortium, The. (2008). The gene ontology project in 2008. *Nucleic Acids Research, 36*(D440–D444).

Genesereth, M. R. (1984). The use of design descriptions in automated diagnosis. *AIJ, 24*, 411–436.

Genesereth, M. R. y Nilsson, N. J. (1987). *Logical Foundations of Artificial Intelligence*. Morgan Kaufmann.

Genesereth, M. R. y Nourbakhsh, I. (1993). Timesaving tips for problem solving with incomplete information. En *AAAI-93*.

Genesereth, M. R. y Smith, D. E. (1981). Metalevel architecture. Memo, Computer Science Department, Stanford University.

Gent, I., Petrie, K. y Puget, J.-F. (2006). Symmetry in constraint programming. En Rossi, F., Van Beek, P. y Walsh, T. (Eds.), *Handbook of Constraint Programming*. Elsevier.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Kerasm and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly.

Gers, F. A., Schmidhuber, J. y Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation, 12*, 2451–2471.

Getoor, L. y Taskar, B. (Eds.). (2007). *Introduction to Statistical Relational Learning*. MIT Press.

Ghaheri, A., Shoar, S., Naderan, M. y Hoseini, S. S. (2015). The applications of genetic algorithms in medicine. *Oman medical journal, 30*, 406–416.

Ghahramani, Z. (1998). Learning dynamic Bayesian networks. En *Adaptive Processing of Sequences and Data Structures*.

Ghahramani, Z. (2005). Tutorial on nonparametric Bayesian methods. Given at the UAI-05 Conference.

Ghallab, M., Howe, A., Knoblock, C. A. y McDermott, D. (1998). PDDL—The planning domain definition language. Tech. rep., Yale Center for Computational Vision and Control.

Ghallab, M. y Laruelle, H. (1994). Representation and control in IxTeT, a temporal planner. En *AIPS-94*.

Ghallab, M., Nau, D. S. y Traverso, P. (2004). *Automated Planning: Theory and practice*. Morgan Kaufmann.

Ghallab, M., Nau, D. S. y Traverso, P. (2016). *Automated Planning and aAting*. Cambridge University Press.

Gibbs, R. W. (2006). Metaphor interpretation as embodied simulation. *Mind, 21*, 434–458.

Gibson, J. J. (1950). *The Perception of the Visual World*. Houghton Mifflin.

Gibson, J. J. (1979). *The Ecological Approach to Visual Perception*. Houghton Mifflin.

Gibson, J. J., Olum, P. y Rosenblatt, F. (1955). Parallax and perspective during aircraft landings. *American Journal of Psychology, 68*, 372–385.

Gilks, W. R., Richardson, S. y Spiegelhalter, D. J. (Eds.). (1996). *Markov chain Monte Carlo in practice*. Chapman and Hall.

Gilks, W. R., Thomas, A. y Spiegelhalter, D. J. (1994). A language and program for complex Bayesian modelling. *The Statistician, 43*, 169–178.

Gilks, W. R. y Berzuini, C. (2001). Following a moving target—Monte Carlo inference for dynamic Bayesianmodels. *J. Royal Statistical Society, 63*, 127–146.

Gilks,W. R. y Wild, P. P. (1992). Adaptive rejection sampling for Gibbs sampling. *Applied Statistics, 41*, 337–348.

Gillies, D. B. (1959). Solutions to general non-zero-sum games. En Tucker, A. W. y Luce, L. D. (Eds.), *Contributions to the Theory of Games, volume IV*. Princeton University Press.

Gilmore, P. C. (1960). A proof method for quantification theory: Its justification and realization. *IBM Journal of Research and Development, 4*, 28–35.

Gilpin, A., Sandholm, T. y Sorensen, T. (2008). A heads-up no-limit Texas Hold'em poker player: Discretized betting models and automatically generated equilibrium-finding programs. En *AAMAS-08*.

Ginsberg, M. L. (1993). *Essentials of Artificial Intelligence*. Morgan Kaufmann.

Ginsberg, M. L. (2001). GIB: Imperfect information in a computationally challenging game. *JAIR, 14*, 303–358.

Gionis, A., Indyk, P. y Motwani, R. (1999). Similarity search in high dimensions vis hashing. En *Proc. 25th Very Large Database (VLDB) Conference*.

Girshick, R., Donahue, J., Darrell, T. y Malik, J. (2016). Region-based convolutional networks for accurate object detection and segmentation. *PAMI, 38*, 142–58.

Gittins, J. C. (1989). *Multi-Armed Bandit Allocation Indices*. Wiley.

Gittins, J. C. y Jones, D.M. (1974). A dynamic allocation index for the sequential design of experiments. En Gani, J. (Ed.), *Progress in Statistics*. North-Holland.

Glanc, A. (1978). On the etymology of the word "robot". *SIGART Newsletter, 67*, 12.

Glickman, M. E. (1999). Parameter estimation in large dynamic paired comparison experiments. *Applied Statistics, 48*, 377–394.

Glorot, X., Bordes, A. y Bengio, Y. (2011). Deep sparse rectifier neural networks. En *AISTATS'2011*.

Glover, F. y Laguna, M. (Eds.). (1997). *Tabu search*. Kluwer.

Gluss, B. (1959). An optimum policy for detecting a fault in a complex system. *Operations Research, 7*, 468–477.

Godefroid, P. (1990). Using partial orders to improve automatic verification methods. En *Proc. 2nd Int'l Workshop on Computer Aided Verification*.

Gödel, K. (1930). *Über die Vollst ändigkeit des Logikkalk¨uls*. Ph.D. thesis, University of Vienna.

Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik, 38*, 173–198.

Goebel, J., Volk, K., Walker, H. y Gerbault, F. (1989). Automatic classification of spectra from the infrared astronomical satellite (IRAS). *Astronomy and Astrophysics, 222*, L5–L8.

Goertzel, B. y Pennachin, C. (2007). *Artificial General Intelligence*. Springer.

Gogate, V. y Domingos, P. (2011). Approximation by quantization. En *UAI-11*.

Gold, E. M. (1967). Language identification in the limit. *Information and Control, 10*, 447–474.

Goldberg, A. V., Kaplan, H. y Werneck, R. F. (2006). Reach for A*: Efficient point-to-point shortest path algorithms. En *Workshop on algorithm engineering and experiments*.

Goldberg, Y. (2017). Neural network methods for natural language processing. *Synthesis Lectures on Human Language Technologies, 10*.

Goldberg, Y., Zhao, K. y Huang, L. (2013). Efficient implementation of beam-search incremental parsers. En *ACL-13*.

Goldman, R. y Boddy, M. (1996). Expressive planning and explicit knowledge. En *AIPS-96*.

Goldszmidt, M. y Pearl, J. (1996). Qualitative probabilities for default reasoning, belief revision, and causal modeling. *AIJ, 84*, 57–112.

Golomb, S. y Baumert, L. (1965). Backtrack proramming. *JACM, 14*, 516–524.

Golub, G., Heath, M. y Wahba, G. (1979). Generalized cross-validation as a method for choosing a good ridge parameter. *Technometrics, 21*.

Gomes, C., Selman, B., Crato, N. y Kautz, H. (2000). Heavy-tailed phenomena in satisfiability and constrain processing. *JAR, 24*, 67–100.

Gomes, C., Kautz, H., Sabharwal, A. y Selman, B. (2008). Satisfiability solvers. En Van Harmelen, F., Lifschitz, V. y Porter, B. (Eds.), *Handbook of Knowledge Representation*. Elsevier.

Gomes, C. y Selman, B. (2001). Algorithm portfolios. *AIJ, 126*, 43–62.

Gomes, C., Selman, B. y Kautz, H. (1998). Boosting combinatorial search through randomization. En *AAAI-98*.

Gonthier, G. (2008). Formal proof–The four-color theorem. *Notices of the AMS, 55*, 1382–1393.

Good, I. J. (1961). A causal calculus. *British Journal of the Philosophy of Science, 11*, 305–318.

Good, I. J. (1965a). The mystery of Go. *New Scientist, 427*, 172–174.

Good, I. J. (1965b). Speculations concerning the first ultraintelligent machine. En Alt, F. L. y Rubinoff, M. (Eds.), *Advances in Computers*, Vol. 6. Academic Press.

Good, I. J. (1983). *Good Thinking: The Foundations of Probability and Its Applications*. University of Minnesota Press.

Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press.

Goodfellow, I., Bulatov, Y., Ibarz, J., Arnoud, S. y Shet, V. (2014). Multi-digit number recognition from Street View imagery using deep convolutional neural networks. En *International Conference on Learning Representations*.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. y Bengio, Y. (2015a). Generative adversarial nets. En *NeurIPS 27*.

Goodfellow, I., Vinyals, O. y Saxe, A. M. (2015b). Qualitatively characterizing neural network optimization problems. En *International Conference on Learning Representations*.

Goodman, J. (2001). A bit of progress in language modeling. Tech. rep., Microsoft Research.

Goodman, N. D., Mansinghka, V. K., Roy, D., Bonawitz, K. y Tenenbaum, J. B. (2008). Church: A language for generative models. En *UAI-08*.

Goodman, N. (1977). The Structure of Appearance (3a edición). D. Reidel.

Gopnik, A. y Glymour, C. (2002). Causal maps and Bayes nets: A cognitive and computational account of theory-formation. En Caruthers, P., Stich, S. y Siegal, M. (Eds.), *The Cognitive Basis of Science*. Cambridge University Press.

Gordon, A. D., Graepel, T., Rolland, N., Russo, C., Borgström, J. y Guiver, J. (2014). Tabular: A schema-driven probabilistic programming language. En *POPL-14*.

Gordon, A. S. y Hobbs, J. R. (2017). *A Formal Theory of Commonsense Psychology: How People Think People Think*. Cambridge University Press.

Gordon, M. J., Milner, A. J. y Wadsworth, C. P. (1979). *Edinburgh LCF*. Springer-Verlag.

Gordon, N. (1994). *Bayesian methods for tracking*. Ph.D. thesis, Imperial College.

Gordon, N., Salmond, D. J. y Smith, A. F. M. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F (Radar and Signal Processing), 140*, 107–113.

Gordon, S. A. (1994). A faster Scrabble move generation algorithm. *Software Practice and Experience, 24*, 219–232.

Gorry, G. A. (1968). Strategies for computer-aided diagnosis. *Math. Biosciences, 2*, 293–318.

Gorry, G. A., Kassirer, J. P., Essig, A. y Schwartz, W. B. (1973). Decision analysis as the basis for computer-aided management of acute renal failure. *American Journal of Medicine, 55*, 473–484.

Gottlob, G., Leone, N. y Scarcello, F. (1999a). A comparison of structural CSP decomposition methods. En *IJCAI-99*.

Gottlob, G., Leone, N. y Scarcello, F. (1999b). Hypertree decompositions and tractable queries. En *PODS-99*.

Goyal, Y., Khot, T., Summers-Stay, D., Batra, D. y Parikh, D. (2017). Making the V in VQA matter: Elevating the role of image understanding in visual question answering. En *CVPR-17*.

Grace, K., Salvatier, J., Dafoe, A., Zhang, B. y Evans, O. (2017). When will AI exceed human performance? Evidence from AI experts. arXiv:1705.08807.

Graham, S. L., Harrison, M. A. y Ruzzo, W. L. (1980). An improved context-free recognizer. *ACM Transactions on Programming Languages and Systems, 2*, 415–462.

Grassmann, H. (1861). *Lehrbuch der Arithmetik*. Th. Chr. Fr. Enslin, Berlin.

Grayson, C. J. (1960). Decisions under uncertainty: Drilling decisions by oil and gas operators. Tech. rep., Harvard Business School.

Green, B., Wolf, A., Chomsky, C. y Laugherty, K. (1961). BASEBALL:An automatic question answerer. En *Proc. Western Joint Computer Conference*.

Green, C. (1969a). Application of theorem proving to problem solving. En *IJCAI-69*.

Green, C. (1969b). Theorem-proving by resolution as a basis for question-answering systems. En Meltzer, B., Michie, D., and Swann, M. (Eds.), *Machine Intelligence 4*. Edinburgh University Press.

Green, C. y Raphael, B. (1968). The use of theorem-proving techniques in question-answering systems. En *Proc. 23rd ACM National Conference*.

Gribkoff, E., Van den Broeck, G. y Suciu, D. (2014). Understanding the complexity of lifted inference and asymmetric weighted model counting. En *UAI-14*.

Griffiths, T. L., Kemp, C. y Tenenbaum, J. B. (2008). Bayesian models of cognition. En Sun, R. (Ed.), *The Cambridge handbook of computational cognitive modeling*. Cambridge University Press.

Grinstead, C. y Snell, J. (1997). *Introduction to Probability*. American Mathematical Society.

Grosz, B. J. y Stone, P. (2018). A century long commitment to assessing artificial intelligence and its impact on society. *Communications of the ACM, 61*.

Grove, W. y Meehl, P. (1996). Comparative efficiency of informal (subjective, impressionistic) and formal (mechanical, algorithmic) prediction procedures: The clinical statistical controversy. *Psychology, Public Policy, and Law, 2*, 293–323.

Gruber, T. (2004). Interview of Tom Gruber. AIS SIGSEMIS *Bulletin, 1*.

Gu, J. (1989). *Parallel Algorithms and Architectures for Very Fast AI Search*. Ph.D. thesis, Univ. of Utah.

Guard, J., Oglesby, F., Bennett, J. y Settle, L. (1969). Semi-automated mathematics. *JACM, 16*, 49–62.

Guestrin, C., Koller, D., Gearhart, C. y Kanodia, N. (2003a). Generalizing plans to new environments in relational MDPs. En *IJCAI-03*.

Guestrin, C., Koller, D., Parr, R. y Venkataraman, S. (2003b). Efficient solution algorithms for factored MDPs. *JAIR, 19*, 399–468.

Guestrin, C., Lagoudakis, M. G. y Parr, R. (2002). Coordinated reinforcement learning. En *ICML-02*.

Guibas, L. J., Knuth, D. E. y Sharir, M. (1992). Randomized incremental construction of Delaunay and Voronoi diagrams. *Algorithmica, 7*, 381–413.

Gulshan, V., Peng, L., Coram, M., Stumpe, M. C., Wu, D., Narayanaswamy, A., Venugopalan, S., Widner, K., Madams, T., Cuadros, J. *et al.* (2016). Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *Jama, 316*, 2402–2410.

Gunkel, D. J. (2018). Robot Rights. MIT Press.

Gunning, D. (2016). Explainable artificial intelligence (xai). Tech. rep., DARPA.

Guo, C., Goldstein, T., Hannun, A. y Van der Maaten, L. (2019). Certified data removal from machine learning models. arXiv:1911.03030.

Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S. y Smith, N. A. (2018). Annotation artifacts in natural language inference data. arXiv:1803.02324.

Guyon, I., Bennett, K., Cawley, G. C., Escalante,H. J., Escalera, S., Ho, T. K., Macià, N., Ray, B., Saeed, M., Statnikov, A. R. y Viegas, E. (2015). Design of the 2015 ChaLearn AutoML challenge. En *IJCNN-15*.

Guyon, I. y Elisseeff, A. (2003). An introduction to variable and feature selection. *JMLR, 3*, 1157–1182.

Hacking, I. (1975). *The Emergence of Probability*. Cambridge University Press.

Hadfield-Menell, D., Dragan, A. D., Abbeel, P. y Russell, S. J. (2017a). Cooperative inverse reinforcement learning. En *NeurIPS 29*.

Hadfield-Menell, D., Dragan, A. D., Abbeel, P. y Russell, S. J. (2017b). The off-switch game. En *IJCAI-17*.

Hadfield-Menell, D. y Russell, S. J. (2015). Multitasking: Efficient optimal planning for bandit superprocesses. En *UAI-15*.

Hailperin, T. (1984). Probability logic. *Notre Dame J. Formal Logic, 25*, 198–212.

Hald, A. (1990). *A History of Probability and Statistics and Their Applications before 1750*. Wiley.

Hales, T. (2005). A proof of the Kepler conjecture. *Annals of mathematics, 162*, 1065–1185.

Hales, T., Adams, M., Bauer, G., Dang, T. D., Harrison, J., Le Truong, H., Kaliszyk, C., Magron, V., McLaughlin, S., Nguyen, T. T. *et al.* (2017). A formal proof of the Kepler conjecture. En *Forum of Mathematics, Pi*.

Halevy, A. (2007). Dataspaces: A new paradigm for data integration. En *Brazilian Symposium on Databases*.

Halevy, A., Norvig, P. y Pereira, F. (2009). The unreasonable effectiveness of data. *IEEE Intelligent Systems, March/April*, 8–12.

Halpern, J. Y. (1990). An analysis of first-order logics of probability. *AIJ, 46*, 311–350.

Halpern, J. Y. (1999). Technical addendum, Cox's theorem revisited. *JAIR, 11*, 429–435.

Halpern, J. Y. y Weissman, V. (2008). Using first-order logic to reason about policies. *ACM Transactions on Information and System Security, 11*, 1–41.

Hammersley, J. M. y Handscomb, D. C. (1964). *Monte Carlo Methods*. Methuen.

Han, J., Pei, J. y Kamber, M. (2011). *Data Mining: Concepts and Techniques*. Elsevier.

Han, X. y Boyden, E. (2007). Multiple-color optical activation, silencing, and desynchronization of neural activity, with single-spike temporal resolution. *PLoS One, e299*.

Handschin, J. E. y Mayne, D. Q. (1969). Monte Carlo techniques to estimate the conditional expectation in multi-stage nonlinear filtering. *Int. J. Control, 9*, 547–559.

Hans, A., Schneegaß, D., Schäfer, A. M. y Udluft, S. (2008). Safe exploration for reinforcement learning. En *ESANN*.

Hansen, E. (1998). Solving POMDPs by searching in policy space. En *UAI-98*.

Hansen, E. y Zilberstein, S. (2001). LAO*: a heuristic search algorithm that finds solutions with loops. *AIJ, 129*, 35–62.

Hansen, P. y Jaumard, B. (1990). Algorithms for the maximum satisfiability problem. *Computing, 44*, 279–303.

Hanski, I. y Cambefort, Y. (Eds.). (1991). *Dung Beetle Ecology*. Princeton University Press.

Hansson, O. y Mayer, A. (1989). Heuristic search as evidential reasoning. En *UAI 5*.

Haralick, R. M. y Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *AIJ, 14*, 263–313.

Hardin, G. (1968). The tragedy of the commons. *Science, 162*, 1243–1248.

Hardt, M., Price, E., Srebro, N. *et al.* (2017). Equality of opportunity in supervised learning. En *NeurIPS 29*.

Harris, T. (2016). How technology is hijacking your mind—From a magician and Google design ethicist. medium.com/thrive-global/how-technologyhijacks-peoples-minds-from-a-magician-and-googles-design-ethicist-56d62ef5edf3.

Harris, Z. (1954). Distributional structure. *Word, 10*.

Harrison, J. y March, J. G. (1984). Decision making and postdecision surprises. *Administrative Science Quarterly, 29*, 26–42.

Harrow, A. W., Hassidim, A. y Lloyd, S. (2009). Quantum algorithm for linear systems of equations. *Physical Review Letters, 103 15*, 150502.

Harsanyi, J. (1967). Games with incomplete information played by Bayesian players. *Management Science, 14*, 159–182.

Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, SSC-4(2)*, 100–107.

Hart, T. P. y Edwards, D. J. (1961). The tree prune (TP) algorithm. Artificial intelligence project memo, MIT.

Hartley, H. (1958). Maximum likelihood estimation from incomplete data. *Biometrics, 14*, 174–194.

Hartley, R. y Zisserman, A. (2000). *Multiple view geometry in computer vision*. Cambridge University Press.

Hashimoto, K., Xiong, C., Tsuruoka, Y. y Socher, R. (2016). A joint many-task model: Growing a neural network for multiple NLP tasks. arXiv:1611.01587.

Haslum, P., Botea, A., Helmert, M., Bonet, B. y Koenig, S. (2007). Domain-independent construction of pattern database heuristics for cost-optimal planning.En *AAAI-07*.

Haslum, P. y Geffner, H. (2001). Heuristic planning with time and resources. En *Proc. IJCAI-01 Workshop on Planning with Resources*.

Haslum, P. (2006). Improving heuristics through relaxed search – An analysis of TP4 and HSP*a in the 2004 planning competition. *JAIR, 25*, 233–267.

Hastie, T. y Tibshirani, R. (1996). Discriminant adaptive nearest neighbor classification and regression. En *NeurIPS 8*.

Hastie, T., Tibshirani, R. y Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference and Prediction* (2a edición). Springer-Verlag.

Hastings, W. K. (1970). Monte Carlo sampling methods using Markov chains and their applications. *Biometrika, 57*, 97–109.

Hatem, M. y Ruml, W. (2014). Simpler bounded suboptimal search. En *AAAI-14*.

Haugeland, J. (1985). Artificial Intelligence: The Very Idea. MIT Press.

Havelund, K., Lowry, M., Park, S., Pecheur, C., Penix, J., Visser, W. y White, J. L. (2000). Formal analysis of the remote agent before and after flight. En Proc. *5th NASA Langley Formal Methods Workshop*.

Havenstein, H. (2005). Spring comes to AI winter. *Computer World*, Fe. 14.

Hawkins, J. (1961). Self-organizing systems: A review and commentary. Proc. *IRE, 49*, 31–48.

Hay, N., Russell, S. J., Shimony, S. E. y Tolpin, D. (2012). Selecting computations: Theory and applications. En *UAI-12*.

Hayes, P. J. (1978). The naive physics manifesto. En Michie, D. (Ed.), *Expert Systems in the Microelectronic Age*. Edinburgh University Press.

Hayes, P. J. (1979). The logic of frames. En Metzing, D. (Ed.), *Frame Conceptions and Text Understanding*. de Gruyter.

Hayes, P. J. (1985a). Naive physics I: Ontology for liquids. En Hobbs, J. R. y Moore, R. C. (Eds.), *Formal Theories of the Commonsense World*, chap. 3. Ablex.

Hayes, P. J. (1985b). The second naive physics manifesto. En Hobbs, J. R. y Moore, R. C. (Eds.), *Formal Theories of the Commonsense World*, chap. 1. Ablex.

Hays, J. y Efros, A. (2007). Scene completion Using millions of photographs. *ACM Transactions on Graphics (SIGGRAPH), 26*.

He, H., Bai, Y., Garcia, E. A. y Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. En *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*.

He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep residual learning for image recognition. En *CVPR-16*.

Heawood, P. J. (1890). Map colouring theorem. *Quarterly Journal of Mathematics, 24*, 332–338.

Hebb, D. O. (1949). *The Organization of Behavior*. Wiley.

Heckerman, D. (1986). Probabilistic interpretation for MYCIN's certainty factors. En Kanal, L. N. y Lemmer, J. F. (Eds.), *UAI 2*. Elsevier.

Heckerman, D. (1991). *Probabilistic Similarity Networks*. MIT Press.

Heckerman, D. (1998). A tutorial on learning with Bayesian networks. In Jordan, M. I. (Ed.), *Learning in graphical models*. Kluwer.

Heckerman, D., Geiger, D. y Chickering, D. M. (1994). Learning Bayesian networks: The combination of knowledge and statistical data. Technical report, Microsoft Research.

Heess, N., Wayne, G., Silver, D., Lillicrap, T., Erez, T. y Tassa, Y. (2016). Learning continuous control policies by stochastic value gradients. En *NeurIPS 28*.

Heidegger, M. (1927). *Being and Time*. SCM Press.

Heinlein, R. A. (1973). *Time Enough for Love*. Putnam.

Held, M. y Karp, R. M. (1970). The traveling salesman problem and minimum spanning trees. *Operations Research, 18*, 1138–1162.

Helmert, M. (2001). On the complexity of planning in transportation domains. En *ECP-01*.

Helmert, M. (2006). The fast downward planning system. *JAIR, 26*, 191–246.

Helmert, M. y Röger, G. (2008). How good is almost perfect? En *AAAI-08*.

Helmert, M., Röger, G. y Karpas, E. (2011). Fast downward stone soup: A baseline for building planner portfolios. En *ICAPS*.

Hendeby, G., Karlsson, R. y Gustafsson, F. (2010). Particle filtering: The need for speed. *EURASIP J. Adv. Sig. Proc., June*.

Henrion, M. (1988). Propagation of uncertainty in Bayesian networks by probabilistic logic sampling. En Lemmer, J. F. y Kanal, L. N. (Eds.), *UAI 2*. Elsevier.

Henzinger, T. A. y Sastry, S. (Eds.). (1998). *Hybrid Systems: Computation and Control*. Springer-Verlag.

Herbrand, J. (1930). *Recherches sur la Théorie de la Démonstration*. Ph.D. thesis, University of Paris.

Herbrich, R., Minka, T. y Graepel, T. (2007). TrueSkill: A Bayesian skill rating system. En *NeurIPS 19*.

Hernández-Orallo, J. (2016). Evaluation in artificial intelligence: From task-oriented to ability-oriented measurement. *Artificial Intelligence Review, 48*, 397–447.

Hess, C. y Ostrom, E. (2007). *Understanding Knowledge as a Commons*. MIT Press.

Hewitt, C. (1977). Viewing control structures as patterns of passing messages. *AIJ, 8*, 323–364.

Hewitt, C. (1969). PLANNER: a language for proving theorems in robots. En *IJCAI-69*.

Hezaveh, Y. D., Levasseur, L. P. y Marshall, P. J. (2017). Fast automated analysis of strong gravitational lenses with convolutional neural networks. *Nature, 548*, 555–557.

Hierholzer, C. (1873). Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. *Mathematische Annalen, 6*, 30–32.

Hilbert, M. y Lopez, P. (2011). The world's technological capacity to store, communicate, and compute information. *Science, 332*, 60–65.

Hilgard, E. R. y Bower, G. H. (1975). *Theories of Learning* (4a edición). Prentice-Hall.

Hind, M., Mehta, S., Mojsilovic, A., Nair, R., Ramamurthy, K. N., Olteanu, A. y Varshney, K. R. (2018). Increasing trust in AI services through supplier's declarations of conformity. arXiv:1808.07261.

Hintikka, J. (1962). *Knowledge and Belief*. Cornell University Press.

Hinton, G. E. y Anderson, J. A. (1981). *Parallel Models of Associative Memory*. Lawrence Erlbaum.

Hinton, G. E. y Nowlan, S. J. (1987). How learning can guide evolution. *Complex Systems, 1*, 495–502.

Hinton, G. E. y Sejnowski, T. (1983). Optimal perceptual inference. En *CVPR-83*.

Hinton, G. E. y Sejnowski, T. (1986). Learning and relearning in Boltzmann machines. En Rumelhart, D. E. y McClelland, J. L. (Eds.), *Parallel Distributed Processing*. MIT Press.

Hinton, G. E. (1987). Learning translation invariant recognition in a massively parallel network. En Goos, G. y Hartmanis, J. (Eds.), *PARLE: Parallel Architectures and Languages Europe*. Springer-Verlag.

Hinton, G. E., Deng, L., Yu, D., Dahl, G., Mohamed, A. R., Jaitly, N., Senior, A., Vanhoucke, V., Nguyen, P., Sainath, T. y Kingsbury, B. (2012). Deep neural networks for acoustic modeling in speech recognition. *Signal Processing Magazine, 29*, 82 – 97.

Hinton, G. E., Osindero, S. y Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation, 18*, 1527–1554.

Hirth, M., Hoßfeld, T. y Tran-Gia, P. (2013). Analyzing costs and accuracy of validation mechanisms for crowdsourcing platforms. *Mathematical and Computer Modelling, 57*, 2918–2932.

Ho, M. K., Littman, M. L., MacGlashan, J., Cushman, F. y Austerweil, J. L. (2017). Showing versus doing: Teaching by demonstration. En *NeurIPS 29*.

Ho, T. K. (1995). Random decision forests. En *Proc. 3rd Int'l Conf. on Document Analysis and Recognition*.

Hobbs, J. R. (1990). *Literature and Cognition*. CSLI Press.

Hobbs, J. R. y Moore, R. C. (Eds.). (1985). *Formal Theories of the Commonsense World*. Ablex.

Hochreiter, S. (1991). Untersuchungen zu dynamischen neuronalen Netzen. Diploma thesis, Technische Universität München.

Hochreiter, S. y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*, 1735–1780.

Hoffman, M., Bach, F. R. y Blei, D.M. (2011). Online learning for latent Dirichlet allocation. En *NeurIPS 23*.

Hoffmann, J. (2001). FF: The fast-forward planning system. *AIMag, 22*, 57–62.

Hoffmann, J. y Brafman, R. I. (2006). Conformant planning via heuristic forward search: A new approach. *AIJ, 170*, 507–541.

Hoffmann, J. y Brafman, R. I. (2005). Contingent planning via heuristic forward search with implicit belief states. En *ICAPS-05*.

Hoffmann, J. (2005). Where "ignoring delete lists" works: Local search topology in planning benchmarks. *JAIR, 24*, 685–758.

Hoffmann, J. y Nebel, B. (2001). The FF planning system: Fast plan generation through heuristic search. *JAIR, 14*, 253–302.

Hoffmann, J., Sabharwal, A. y Domshlak, C. (2006). Friends or foes? An AI planning perspective on abstraction and search. En *ICAPS-06*.

Hofleitner, A., Herring, R., Abbeel, P. y Bayen, A. M. (2012). Learning the dynamics of arterial traffic from probe data using a dynamic Bayesian network. *IEEE Transactions on Intelligent Transportation Systems, 13*, 1679–1693.

Hogan, N. (1985). Impedance control: An approach to manipulation. Parts I, II, and III. *J. Dynamic Systems, Measurement, and Control, 107*, 1–24.

Hoiem, D., Efros, A. y Hebert, M. (2007). Recovering surface layout from an image. *IJCV, 75*, 151–172.

Holland, J. H. (1975). *Adaption in Natural and Artificial Systems*. University of Michigan Press.

Holland, J. H. (1995). *Hidden Order: How Adaptation Builds Complexity*. Addison-Wesley.

Holte, R. C., Felner, A., Sharon, G. y Sturtevant, N. R. (2016). Bidirectional search that is guaranteed to meet in the middle. En *AAAI-16*.

Holzmann, G. J. (1997). The Spin model checker. *IEEE Transactions on Software Engineering, 23*, 279–295.

Hood, A. (1824). Case 4th—28 July 1824 (Mr. Hood's cases of injuries of the brain). *Phrenological Journal and Miscellany, 2*, 82–94.

Hooker, J. (1995). Testing heuristics: We have it all wrong. *J. Heuristics, 1*, 33–42.

Hoos, H. H. y Stützle, T. (2004). *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann.

Hoos, H. H. y Tsang, E. (2006). Local search methods. En Rossi, F., Van Beek, P. y Walsh, T. (Eds.), *Handbook of Constraint Processing*. Elsevier.

Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *PNAS, 79*, 2554–2558.

Horn, A. (1951). On sentences which are true of direct unions of algebras. *JSL, 16*, 14–21.

Horn, B. K. P. (1970). Shape from shading: A method for obtaining the shape of a smooth opaque object from one view. Technical report, MIT Artificial Intelligence Laboratory.

Horn, B. K. P. y Brooks, M. J. (1989). *Shape from Shading*. MIT Press.

Horn, K. V. (2003). Constructing a logic of plausible inference: A guide to Cox's theorem. *IJAR, 34*, 3–24.

Horning, J. J. (1969). *A Study of Grammatical Inference*. Ph.D. thesis, Stanford University.

Horswill, I. (2000). Functional programming of behavior-based systems. *Autonomous Robots, 9*, 83–93.

Horvitz, E. J. (1987). Problem-solving design: Reasoning about computational value, trade-offs, and resources. En *Proc. Second Annual NASA Research Forum*.

Horvitz, E. J. y Barry, M. (1995). Display of information for time-critical decision making. En *UAI-95*.

Horvitz, E. J., Breese, J. S., Heckerman, D. y Hovel, D. (1998). The Lumiere project: Bayesian user modeling for inferring the goals and needs of software users. En *UAI-98*.

Horvitz, E. J., Breese, J. S. y Henrion, M. (1988). Decision theory in expert systems and artificial intelligence. *IJAR, 2*, 247–302.

Horvitz, E. J. y Breese, J. S. (1996). Ideal partition of resources for metareasoning. En *AAAI-96*.

Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *J. Ed. Psych., 24*, 417–441.

Howard, J. y Gugger, S. (2020). *Deep Learning for Coders with fastai and PyTorch*. O'Reilly.

Howard, J. y Ruder, S. (2018). Fine-tuned language models for text classification. arXiv:1801.06146.

Howard, R. A. (1960). *Dynamic Programming and Markov Processes*. MIT Press.

Howard, R. A. (1966). Information value theory. *IEEE Transactions on Systems Science and Cybernetics, SSC-2*, 22–26.

Howard, R. A. (1989). Microrisks for medical decision analysis. *Int. J. Technology Assessment in Health Care, 5*, 357–370.

Howard, R. A. y Matheson, J. E. (1984). Influence diagrams. En Howard, R. A. y Matheson, J. E. (Eds.), *Readings on the Principles and Applications of Decision Analysis*. Strategic Decisions Group.

Howe, D. (1987). The computational behaviour of Girard's paradox. En *LICS-87*.

Howson, C. (2003). Probability and logic. *J. Applied Logic, 1*, 151–165.

Hsiao, K., Kaelbling, L. P. y Lozano-Perez, T. (2007). Grasping POMDPs. En *ICRA-07*.

Hsu, F.-H. (2004). *Behind Deep Blue: Building the Computer that Defeated the World Chess Champion*. Princeton University Press.

Hsu, F.-H., Anantharaman, T. S., Campbell, M. S. y Nowatzyk, A. (1990). A grandmaster chess machine. *Scientific American, 263*, 44–50.

Hu, J. y Wellman, M. P. (1998). Multiagent reinforcement learning: Theoretical framework and an algorithm. En *ICML-98*.

Hu, J. y Wellman,M. P. (2003). Nash Q-learning for general-sum stochastic games. *JMLR, 4*, 1039–1069.

Huang, T., Koller, D., Malik, J., Ogasawara, G., Rao, B., Russell, S. J. y Weber, J. (1994). Automatic symbolic traffic scene analysis using belief networks. En *AAAI-94*.

Huang, T. y Russell, S. J. (1998). Object identification: A Bayesian analysis with application to traffic surveillance. *AIJ, 103*, 1–17.

Hubel, D. H. y Wiesel, T. N. (1962). Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *J. Physiology, 160*, 106–154.

Hubel, D. H. y Wiesel, T. N. (1968). Receptive fields and functional architecture of monkey striate cortex. *J. Physiology, 195*, 215–243.

Hubel, D. H. (1988). *Eye, Brain, and Vision*. W. H. Freeman.

Hubel, D. H. y Wiesel, T. N. (1959). Receptive fields of single neurons in the cat's striate cortex. *Journal of Physiology, 148*, 574–591.

Huddleston, R. D. y Pullum, G. K. (2002). *The Cambridge Grammar of the English Language*. Cambridge University Press.

Huffman, D. A. (1971). Impossible objects as nonsense sentences. En Meltzer, B. y Michie, D. (Eds.), *Machine Intelligence 6*. Edinburgh University Press.

Hughes, B. D. (1995). *Random Walks and Random Environments, Vol. 1: Random Walks*. Oxford University Press.

Hughes, G. E. y Cresswell, M. J. (1996). *A New Introduction to Modal Logic*. Routledge.

Huhns, M. N. y Singh, M. (Eds.). (1998). Readings in Agents. Morgan Kaufmann.

Hume, D. (1739). *A Treatise of Human Nature* (2a edición). Republished by Oxford University Press, 1978, Oxford.

Humphrys, M. (2008). How my program passed the Turing test. En Epstein, R., Roberts, G. y Beber, G. (Eds.), *Parsing the Turing Test*. Springer.

Hunsberger, L. y Grosz, B. J. (2000). A combinatorial auction for collaborative planning. En *Int. Conference on Multi-Agent Systems*.

Hunt, W. y Brock, B. (1992). A formal HDL and its use in the FM9001 verification. *Phil. Trans. Roy. Soc., 339*.

Hunter, L. y States, D. J. (1992). Bayesian classification of protein structure. *IEEE Expert, 7*, 67–75.

Hur, C.-K., Nori, A. V., Rajamani, S. K. y Samuel, S. (2014). Slicing probabilistic programs. En *PLDI-14*.

Hurst, M. (2000). *The Interpretation of Text in Tables*. Ph.D. thesis, Edinburgh.

Hurwicz, L. (1973). The design of mechanisms for resource allocation. *American Economic Review Papers and Proceedings, 63*, 1–30.

Huth, M. y Ryan, M. (2004). *Logic in Computer Science: Modelling and Reasoning About Systems* (2a edición). Cambridge University Press.

Huttenlocher, D. y Ullman, S. (1990). Recognizing solid objects by alignment with an image. *IJCV, 5*, 195–212.

Hutter, F., Kotthoff, L. y Vanschoren, J. (2019). *Automated Machine Learning*. Springer.

Huygens, C. (1657). De ratiociniis in ludo aleae. En Van Schooten, F. (Ed.), *Exercitionum Mathematicorum*. Elsevirii, Amsterdam. Translated into English by John Arbuthnot (1692).

Huyn, N., Dechter, R. y Pearl, J. (1980). Probabilistic analysis of the complexity of A*. *AIJ, 15*, 241–254.

Huynh, V. A. y Roy, N. (2009). icLQG: Combining local and global optimization for control in information space. En *ICRA-09*.

Hwa, R. (1998). An empirical evaluation of probabilistic lexicalized tree insertion grammars. En *ACL-98*.

Hwang, C. H. y Schubert, L. K. (1993). EL: A formal, yet natural, comprehensive knowledge representation. En *AAAI-93*.

Hyafil, L. y Rivest, R. (1976). Constructing optimal binary decision trees is NP-complete. *Information Processing Letters, 5*, 15–17.

Ieong, S. y Shoham, Y. (2005). Marginal contribution nets: A compact representation scheme for coalitional games. En *Proc. Sixth ACM Conference on Electronic Commerce (EC'05)*.

Ingerman, P. Z. (1967). Panini–Backus form suggested. *CACM, 10*, 137.

Intille, S. y Bobick, A. (1999). A framework for recognizing multi-agent action from visual evidence. En *AAAI-99*.

Ioffe, S. y Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv:1502.03167.

Irpan, A. (2018). Deep reinforcement learning doesn't work yet. www.alexirpan.com/2018/02/14/rlhard.html.

Isard, M. y Blake, A. (1996). Contour tracking by stochastic propagation of conditional density. En *ECCV-96*.

Isola, P., Zhu, J.-Y., Zhou, T. y Efros, A. (2017). Image-to-image translation with conditional adversarial networks. En *CVPR-17*.

Jaakkola, T. y Jordan, M. I. (1996). Computing upper and lower bounds on likelihoods in intractable networks. En *UAI-96*.

Jacobson, D. H. y Mayne, D. Q. (1970). *Differential Dynamic Programming*. North-Holland.

Jaderberg, M., Czarnecki, W. M., Dunning, I., Marris, L., Lever, G., Castaneda, A. G., Beattie, C., Rabinowitz, N. C., Morcos, A. S., Ruderman, A. *et al*. (2019). Human-level performance in 3D multiplayer games with population-based reinforcement learning. *Science, 364*, 859–865.

Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W. M., Donahue, J., Razavi, A., Vinyals, O., Green, T., Dunning, I., Simonyan, K., Fernando, C. y Kavukcuoglu, K. (2017). Population based training of neural networks. arXiv:1711.09846.

Jaffar, J. y Lassez, J.-L. (1987). Constraint logic programming. En *Proc. Fourteenth ACM POPL Conference*. Association for Computing Machinery.

Jaffar, J., Michaylov, S., Stuckey, P. J. y Yap, R. H. C. (1992). The CLP(R) language and system. *ACM Transactions on Programming Languages and Systems, 14*, 339–395.

Jain, D., Barthels, A. y Beetz,M. (2010). Adaptive Markov logic networks: Learning statistical relational models with dynamic parameters. En *ECAI-10*.

Jain, D., Kirchlechner, B. y Beetz, M. (2007). Extending Markov logic to model probability distributions in relational domains. En *30th Annual German Conference on AI (KI)*.

James, G., Witten, D., Hastie, T. y Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer-Verlag.

Jarrett, K., Kavukcuoglu, K., Ranzato, M. y Le-Cun, Y. (2009). What is the best multi-stage architecture for object recognition? En *ICCV-09*.

Jaynes, E. T. (2003). *Probability Theory: The Logic of Science*. Cambridge Univ. Press.

Jeffrey, R. C. (1983). *The Logic of Decision* (2a edición). University of Chicago Press.

Jeffreys, H. (1948). *Theory of Probability*. Oxford.

Jelinek, F. (1976). Continuous speech recognition by statistical methods. *Proc. IEEE, 64*, 532–556.

Jelinek, F. y Mercer, R. L. (1980). Interpolated estimation of Markov source parameters from sparse data. En *Proc. Workshop on Pattern Recognition in Practice*.

Jennings, H. S. (1906). *Behavior of the Lower Organisms*. Columbia University Press.

Jenniskens, P., Betlem, H., Betlem, J. y Barifaijo, E. (1994). The Mbale meteorite shower. *Meteoritics, 29*, 246–254.

Jensen, F. V. (2007). *Bayesian Networks and Decision Graphs*. Springer-Verlag.

Ji, Z., Lipton, Z. C. y Elkan, C. (2014). Differential privacy and machine learning: A survey and review. arXiv:1412.7584.

Jiang, H. y Nachum, O. (2019). Identifying and correcting label bias in machine learning. arXiv:1901.04966.

Jimenez, P. y Torras, C. (2000). An efficient algorithm for searching implicit AND/OR graphs with cycles. *AIJ, 124*, 1–30.

Joachims, T. (2001). A statistical learning model of text classification with support vector machines. En *SIGIR-01*.

Johnson, M. (1998). PCFG models of linguistic tree representations. *Comput. Linguist., 24*, 613–632.

Johnson, W. W. y Story, W. E. (1879). Notes on the "15" puzzle. *American Journal of Mathematics, 2*, 397–404.

Johnston, M. D. y Adorf, H.-M. (1992). Scheduling with neural networks: The case of the Hubble space telescope. *Computers and Operations Research, 19*, 209–240.

Jonathan, P. J. Y., Fung, C. C. y Wong, K. W. (2009). Devious chatbots-interactive malware with a plot. En *FIRA RoboWorld Congress*.

Jones, M. y Love, B. C. (2011). Bayesian fundamentalism or enlightenment? On the explanatory status and theoretical contributions of Bayesian models of cognition. *BBS, 34*, 169–231.

Jones, R. M., Laird, J. y Nielsen, P. E. (1998). Automated intelligent pilots for combat flight simulation. En *AAAI-98*.

Jones, R., McCallum, A., Nigam, K. y Riloff, E. (1999). Bootstrapping for text learning tasks. En *Proc. IJCAI-99 Workshop on Text Mining: Foundations, Techniques, and Applications*.

Jones, T. (2007). *Artificial Intelligence: A Systems Approach*. Infinity Science Press.

Jonsson, A., Morris, P., Muscettola, N., Rajan, K. y Smith, B. (2000). Planning in interplanetary space: Theory and practice. En *AIPS-00*.

Jordan, M. I. (2005). Dirichlet processes, Chinese restaurant processes and all that. Tutorial presentation at the NeurIPS Conference.

Jordan, M. I. (1986). Serial order: A parallel distributed processing approach. Tech. rep., UCSD Institute for Cognitive Science.

Jordan, M. I., Ghahramani, Z., Jaakkola, T. y Saul, L. K. (1999). An introduction to variational methods for graphical models. *Machine Learning, 37*, 183–233.

Jouannaud, J.-P. y Kirchner, C. (1991). Solving equations in abstract algebras: A rule-based survey of unification. En Lassez, J.-L. y Plotkin, G. (Eds.), *Computational Logic*. MIT Press.

Joulin, A., Grave, E., Bojanowski, P. y Mikolov, T. (2016). Bag of tricks for efficient text classification. arXiv:1607.01759.

Jouppi, N. P., Young, C., Patil, N., Patterson, D. A. *et al*. (2017). In-datacenter performance analysis of a tensor processing unit. En *ACM/IEEE 44th International Symposium on Computer Architecture*.

Joy, B. (2000). Why the future doesn't need us. *Wired, 8*.

Jozefowicz, R., Vinyals, O., Schuster, M., Shazeer, N. y Wu, Y. (2016). Exploring the limits of language modeling. arXiv:1602.02410.

Jozefowicz, R., Zaremba,W. y Sutskever, I. (2015). An empirical exploration of recurrent network architectures. En *ICML-15*.

Juels, A. y Wattenberg, M. (1996). Stochastic hillclimbing as a baseline method for evaluating genetic algorithms. En *NeurIPS 8*.

Julesz, B. (1971). *Foundations of Cyclopean Perception*. University of Chicago Press.

Julian, K. D., Kochenderfer, M. J. y Owen, M. P. (2018). Deep neural network compression for aircraft collision avoidance systems. arXiv:1810.04240.

Juliani, A., Berges, V., Vckay, E., Gao, Y., Henry, H., Mattar, M. y Lange, D. (2018). Unity: A general platform for intelligent agents. arXiv:1809.02627.

Junker, U. (2003). The logic of ilog (j)configurator: Combining constraint programming with a description logic. En *Proc. IJCAI-03 Configuration Workshop*.

Jurafsky, D. and Martin, J. H. (2020). Speech and Language *Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (3a edición). Prentice-Hall.

Kadane, J. B. y Simon, H. A. (1977). Optimal strategies for a class of constrained sequential problems. *Annals of Statistics, 5*, 237–255.

Kadane, J. B. y Larkey, P. D. (1982). Subjective probability and the theory of games. *Management Science, 28*, 113–120.

Kaelbling, L. P., Littman, M. L. y Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *AIJ, 101*, 99–134.

Kaelbling, L. P. y Rosenschein, S. J. (1990). Action and planning in embedded agents. *Robotics and Autonomous Systems, 6*, 35–48.

Kager, R. (1999). *Optimality Theory*. Cambridge University Press.

Kahn, H. y Marshall, A. W. (1953). Methods of reducing sample size in Monte Carlo computations. *Operations Research, 1*, 263–278.

Kahn, H. (1950a). Random sampling (Monte Carlo) techniques in neutron attenuation problems–I. *Nucleonics, 6*, 27–passim.

Kahn, H. (1950b). Random sampling (Monte Carlo) techniques in neutron attenuation problems–II. *Nucleonics, 6*, 60–65.

Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

Kahneman, D., Slovic, P. y Tversky, A. (Eds.). (1982). Judgment under Uncertainty: Heuristics and Biases. Cambridge University Press.

Kahneman, D. y Tversky, A. (1979). Prospect theory: An analysis of decision under risk. *Econometrica, 47*, 263–291.

Kaindl, H. y Khorsand, A. (1994). Memory-bounded bidirectional search. En *AAAI-94*.

Kalman, R. (1960). A new approach to linear filtering and prediction problems. *J. Basic Engineering, 82*, 35–46.

Kambhampati, S. (1994). Exploiting causal structure to control retrieval and refitting during plan reuse. *Computational Intelligence, 10*, 213–244.

Kanade, T., Thorpe, C. y Whittaker, W. (1986). Autonomous land vehicle project at CMU. En *ACM Fourteenth Annual Conference on Computer Science*.

Kanal, E. (2017). Machine learning in cybersecurity. CMU SEI Blog, insights.sei. cmu.edu/sei blog/2017/06/machine-learning-incybersecurity.html.

Kanazawa, A., Black, M., Jacobs, D. y Malik, J. (2018a). End-to-end recovery of human shape and pose. En *CVPR-18*.

Kanazawa, A., Tulsiani, M., Efros, A. y Malik, J. (2018b). Learning category-specific mesh reconstruction from image collections. En *ECCV-18*.

Kanazawa, K., Koller, D. y Russell, S. J. (1995). Stochastic simulation algorithms for dynamic probabilistic networks. En *UAI-95*.

Kang, S. M. y Wildes, R. P. (2016). Review of action recognition and detection methods. arXiv:1610.06906.

Kanter, J. M. y Veeramachaneni, K. (2015). Deep feature synthesis: Towards automating data science endeavors. En *Proc. IEEE Int'l Conf. on Data Science and Advanced Analytics*.

Kantorovich, L. V. (1939). Mathematical methods of organizing and planning production. Published in translation in *Management Science, 6*(4), 366–422, 1960.

Kaplan, D. y Montague, R. (1960). A paradox regained. *Notre Dame Formal Logic, 1*, 79–90.

Karaboga, D. y Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *Journal of global optimization, 39*, 459–471.

Karamchandani, A., Bjerager, P. y Cornell, C. A. (1989). Adaptive importance sampling. En *Proc. Fifth International Conference on Structural Safety and Reliability*.

Karmarkar, N. (1984). A new polynomial-time algorithm for linear programming. *Combinatorica, 4*, 373–395.

Karp, R. M. (1972). Reducibility among combinatorial problems. En Miller, R. E. y Thatcher, J. W. (Eds.), *Complexity of Computer Computations*. Plenum.

Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks. Andrej Karpathy blog,karpathy.github.io/2015/05/21/rnn-effectiveness/.

Karpathy, A. y Fei-Fei, L. (2015). Deep visual semantic alignments for generating image descriptions. En *CVPR-15*.

Karras, T., Aila, T., Laine, S. y Lehtinen, J. (2017). Progressive growing of GANs for improved quality, stability, and variation. arXiv:1710.10196.

Karsch, K., Hedau, V., Forsyth, D. y Hoiem, D. (2011). Rendering synthetic objects into legacy photographs. En *SIGGRAPH Asia*.

Kartam, N. A. y Levitt, R. E. (1990). A constraint-based approach to construction planning of multi-story buildings. En *Expert Planning Systems. Institute of Electrical Engineers*.

Kasami, T. (1965). An efficient recognition and syntax analysis algorithm for context-free languages. Tech. rep., Air Force Cambridge Research Laboratory.

Katehakis, M. N. y Veinott, A. F. (1987). Themultiarmed bandit problem: Decomposition and computation. *Mathematics of Operations Research, 12*, 185–376.

Katz, B. (1997). Annotating the world wide web using natural language. En *RIAO '97*.

Kaufmann, M., Manolios, P. y Moore, J. S. (2000). *Computer-Aided Reasoning: An Approach*. Kluwer.

Kautz, H. (2006). Deconstructing planning as satisfiability. En *AAAI-06*.

Kautz, H., McAllester, D. A. y Selman, B. (1996). Encoding plans in propositional logic. En *KR-96*.

Kautz, H. y Selman, B. (1992). Planning as satisfiability. En *ECAI-92*.

Kautz, H. y Selman, B. (1998). BLACKBOX: A new approach to the application of theorem proving to problem solving. Working Notes of the AIPS-98 Workshop on Planning as Combinatorial Search.

Kavraki, L., Svestka, P., Latombe, J.-C. y Overmars, M. (1996). Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation, 12*, 566–580.

Kazemi, S. M., Kimmig, A., Van den Broeck, G. y Poole, D. (2017). New liftable classes for first-order probabilistic inference. En *NeurIPS 29*.

Kearns, M. (1990). *The Computational Complexity of Machine Learning*. MIT Press.

Kearns, M., Mansour, Y. y Ng, A. Y. (2000). Approximate planning in large POMDPs via reusable trajectories. En *NeurIPS 12*.

Kearns, M. y Singh, S. (1998). Near-optimal reinforcement learning in polynomial time. En *ICML-98*.

Kearns, M. y Vazirani, U. (1994). *An Introduction to Computational Learning Theory*. MIT Press.

Kearns, M. (1988). Thoughts on hypothesis boosting.

Kearns, M., Mansour, Y. y Ng, A. Y. (2002). A sparse sampling algorithm for near-optimal planning in large Markov decision processes. *Machine Learning, 49*, 193–208.

Kebeasy, R. M., Hussein, A. I. y Dahy, S. A. (1998). Discrimination between natural earthquakes and nuclear explosions using the Aswan Seismic Network. *Annali di Geofisica, 41*, 127–140.

Keeney, R. L. (1974). Multiplicative utility functions. *Operations Research, 22*, 22–34.

Keeney, R. L. y Raiffa, H. (1976). *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. Wiley.

Kelley, H. J. (1960). Gradient theory of optimal flight paths. *ARS Journal, 30*, 947–954.

Kemp, M. (Ed.). (1989). *Leonardo on Painting: An Anthology of Writings*. Yale University Press.

Kempe, A. B. (1879). On the geographical problem of the four-colors. *American Journal of Mathematics, 2*, 193–200.

Kephart, J. O. y Chess, D.M. (2003). The vision of autonomic computing. *IEEE Computer, 36*, 41–50.

Kersting, K., Raedt, L. D. y Kramer, S. (2000). Interpreting Bayesian logic programs. En *Proc. AAAI-00 Workshop on Learning Statistical Models from Relational Data*.

Keskar, N. S., McCann, B., Varshney, L., Xiong, C. y Socher, R. (2019). CTRL: A conditional transformer language model for controllable generation. arXiv:1909.

Keynes, J. M. (1921). *A Treatise on Probability*. Macmillan.

Khare, R. (2006). Microformats: The next (small) thing on the semantic web. *IEEE Internet Computing, 10*, 68–75.

Khatib, O. (1986). Real-time obstacle avoidance for robot manipulator and mobile robots. *Int. J. Robotics Research, 5*, 90–98.

Kim, B., Khanna, R. y Koyejo, O. O. (2017). Examples are not enough, learn to criticize! Criticism for interpretability. En *NeurIPS 29*.

Kim, J. H. (1983). *CONVINCE: A Conversational Inference Consolidation Engine*. Ph.D. thesis, Department of Computer Science, UCLA.

Kim, J. H. y Pearl, J. (1983). A computational model for combined causal and diagnostic reasoning in inference systems. En *IJCAI-83*.

Kim, J.-H., Lee, C.-H., Lee, K.-H. y Kuppuswamy, N. (2007). Evolving personality of a genetic robot in ubiquitous environment. En *Proc. 16th IEEE International Symposium on Robot and Human Interactive Communication*.

Kim, T. W. (2018). Explainable artificial intelligence (XAI), the goodness criteria and the grasp-ability test. arXiv:1810.09598.

Kingma, D. P. y Welling, M. (2013). Auto-encoding variational Bayes. arXiv:1312.6114.

Kirk, D. E. (2004). *Optimal Control Theory: An Introduction*. Dover.

Kirkpatrick, S., Gelatt, C. D. y Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*, 671–680.

Kisynski, J. y Poole, D. (2009). Lifted aggregation in directed first-order probabilistic models. En *IJCAI-09*.

Kitaev, N., Kaiser, L. y Levskaya, A. (2020). Reformer: The efficient transformer. arXiv:2001.04451.

Kitaev, N. y Klein, D. (2018). Constituency parsing with a self-attentive encoder. arXiv:1805.01052.

Kitani, K. M., abd James Andrew Bagnell, B. D. Z. y Hebert, M. (2012). Activity forecasting. En *ECCV-12*.

Kitano, H., Asada, M., Kuniyoshi, Y., Noda, I. y Osawa, E. (1997). RoboCup: The robot world cup initiative. En *Proc. First International Conference on Autonomous Agents*.

Kjaerulff, U. (1992). A computational scheme for reasoning in dynamic probabilistic networks. En *UAI-92*.

Klarman, H. E., Francis, J. y Rosenthal, G. D. (1968). Cost effectiveness analysis applied to the treatment of chronic renal disease. *Medical Care, 6*, 48–54.

Klein, D. y Manning, C. (2001). Parsing with treebank grammars: Empirical bounds, theoretical models, and the structure of the Penn treebank. En *ACL-01*.

Klein, D. y Manning, C. (2003). A* parsing: Fast exact Viterbi parse selection. En *HLT-NAACL-03*.

Kleinberg, J. M., Mullainathan, S. y Raghavan, M. (2016). Inherent trade-offs in the fair determination of risk scores. arXiv:1609.05807.

Klemperer, P. (2002). What really matters in auction design. *J. Economic Perspectives, 16*.

Kneser, R. y Ney, H. (1995). Improved backing-off for M-gram language modeling. En *ICASSP-95*.

Knoblock, C. A. (1991). Search reduction in hierarchical problem solving. En *AAAI-91*.

Knuth, D. E. (1964). Representing numbers using only one 4. *Mathematics Magazine, 37*, 308–310.

Knuth, D. E. (1975). An analysis of alpha–beta pruning. *AIJ, 6*, 293–326.

Knuth, D. E. (2015). *The Art of Computer Programming*, Vol. 4, Fascicle 6: Satisfiability. Addison-Wesley.

Knuth, D. E. y Bendix, P. B. (1970). Simple word problems in universal algebras. En Leech, J. (Ed.), *Computational Problems in Abstract Algebra*. Pergamon.

Kober, J., Bagnell, J. A. y Peters, J. (2013). Reinforcement learning in robotics: A survey. *International Journal of Robotics Research, 32*, 1238–1274.

Koch, C. (2019). *The Feeling of Life Itself*. MIT Press.

Kochenderfer, M. J. (2015). *Decision Making Under Uncertainty: Theory and Application*. MIT Press.

Kocsis, L. y Szepesvari, C. (2006). Bandit-based Monte-Carlo planning. En *ECML-06*.

Koditschek, D. (1987). Exact robot navigation by means of potential functions: Some topological considerations. En *ICRA-87*.

Koehn, P. (2009). *Statistical Machine Translation*. Cambridge University Press.

Koelsch, S. y Siebel, W. A. (2005). Towards a neural basis of music perception. *Trends in Cognitive Sciences, 9*, 578–584.

Koenderink, J. J. (1990). *Solid Shape*. MIT Press.

Koenderink, J. J. y Van Doorn, A. J. (1991). Affine structure from motion. *J. Optical Society of America A, 8*, 377–385.

Koenig, S. (1991). Optimal probabilistic and decisiontheoretic planning using Markovian decision theory. Master's report, Computer Science Division, University of California, Berkeley.

Koenig, S. (2000). Exploring unknown environments with real-time search or reinforcement learning. En *NeurIPS 12*.

Koenig, S. (2001). Agent-centered search. *AIMag, 22*, 109–131.

Koenig, S. y Likhachev, M. (2002). D* Lite. *AAAI-15*, 15.

Koenig, S., Likhachev, M. y Furcy, D. (2004). Lifelong planning A*. *AIJ, 155*, 93–146.

Kolesky, D. B., Truby, R. L., Gladman, A. S., Busbee, T. A., Homan, K. A. y Lewis, J. A. (2014). 3D bioprinting of vascularized, heterogeneous cell-laden tissue constructs. *Advanced Materials, 26*, 3124–3130.

Koller, D., Meggido, N. y Von Stengel, B. (1996). Efficient computation of equilibria for extensive two-person games. *Games and Economic Behaviour, 14*, 247–259.

Koller, D. y Pfeffer, A. (1997). Representations and solutions for game-theoretic problems. *AIJ, 94*, 167–215.

Koller, D. y Pfeffer, A. (1998). Probabilistic frame-based systems. En *AAAI-98*.

Koller, D. y Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.

Koller, D., McAllester, D. A. y Pfeffer, A. (1997). Effective Bayesian inference for stochastic programs. En *AAAI-97*.

Koller, D. y Parr, R. (2000). Policy iteration for factored MDPs. En *UAI-00*.

Koller, D. y Sahami,M. (1997). Hierarchically classifying documents using very few words. En *ICML-97*.

Kolmogorov, A. N. (1941). Interpolation und extrapolation von stationaren zufalligen folgen. *Bulletin of the Academy of Sciences of the USSR, Ser. Math. 5*, 3–14.

Kolmogorov, A. N. (1950). *Foundations of the Theory of Probability*. Chelsea.

Kolmogorov, A. N. (1963). On tables of random numbers. *Sankhya, the Indian Journal of Statistics: Series A, 25*(4), 369–376.

Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems in Information Transmission, 1*, 1–7.

Kolter, J. Z., Abbeel, P. y Ng, A. Y. (2008). Hierarchical apprenticeship learning, with application to quadruped locomotion. En *NeurIPS 20*.

Kondrak, G. y Van Beek, P. (1997). A theoretical evaluation of selected backtracking algorithms. *AIJ, 89*, 365–387.

Konečnỳ, J., McMahan, H. B., Yu, F. X., Richt´arik, P., Suresh, A. T. y Bacon, D. (2016). Federated learning: Strategies for improving communication efficiency. arXiv:1610.05492.

Konolige, K. (1997). COLBERT: A language for reactive control in Saphira. En *Künstliche Intelligenz: Advances in Artificial Intelligence*, LNAI.

Konolige, K. (2004). Large-scale map-making. En *AAAI-04*.

Konolige, K. (1982). A first order formalization of knowledge and action for a multi-agent planning system. En Hayes, J. E., Michie, D. y Pao, Y.-H. (Eds.), *Machine Intelligence 10*. Ellis Horwood.

Konolige, K. (1994). Easy to be hard: Difficult problems for greedy algorithms. En *KR-94*.

Koopmans, T. C. (1972). Representation of preference orderings over time. En McGuire, C. B. and Radner, R. (Eds.), *Decision and Organization*. Elsevier.

Korb, K. B. y Nicholson, A. (2010). *Bayesian Artificial Intelligence*. CRC Press.

Korf, R. E. (1985a). Depth-first iterative-deepening: an optimal admissible tree search. *AIJ, 27*, 97–109.

Korf, R. E. (1985b). Iterative-deepening A*: An optimal admissible tree search. En *IJCAI-85*.

Korf, R. E. (1987). Planning as search: A quantitative approach. *AIJ, 33*, 65–88.

Korf, R. E. (1990). Real-time heuristic search. *AIJ, 42*, 189–212.

Korf, R. E. (1993). Linear-space best-first search. *AIJ, 62*, 41–78.

Korf, R. E. y Chickering, D. M. (1996). Best-first minimax search. *AIJ, 84*, 299–337.

Korf, R. E. y Felner, A. (2002). Disjoint pattern database heuristics. *AIJ, 134*, 9–22.

Korf, R. E. y Zhang, W. (2000). Divide-and-conquer frontier search applied to optimal sequence alignment. En *AAAI-00*.

Korf, R. E. (1997). Finding optimal solutions to Rubik's Cube using pattern databases. En *AAAI-97*.

Korf, R. E. y Reid, M. (1998). Complexity analysis of admissible heuristic search. En *AAAI-98*.

Koutsoupias, E. y Papadimitriou, C. H. (1992). On the greedy algorithm for satisfiability. *Information Processing Letters, 43*, 53–55.

Kovacs, D. L. (2011). BNF definition of PDDL3.1. Unpublished manuscript from the IPC-2011 website.

Kowalski, R. (1974). Predicate logic as a programming language. En *Proc. IFIP Congress*.

Kowalski, R. (1979). *Logic for Problem Solving*. Elsevier.

Kowalski, R. (1988). The early years of logic programming. *CACM, 31*, 38–43.

Kowalski, R. y Sergot, M. (1986). A logic-based calculus of events. *New Generation Computing, 4*, 67–95.

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

Koza, J. R. (1994). *Genetic Programming II: Automatic Discovery of Reusable Programs*. MIT Press.

Koza, J. R., Bennett, F. H., Andre, D. y Keane, M. A. (1999). *Genetic Programming III: Darwinian Invention and Problem Solving*. Morgan Kaufmann.

Krakovna, V. (2018). Specification gaming examples in AI.

Kraska, T., Beutel, A., Chi, E. H., Dean, J. y Polyzotis, N. (2017). The case for learned index structures. arXiv:1712.01208.

Kraus, S. (2001). *Strategic Negotiation in Multiagent Environments*. MIT Press.

Kraus, S., Ephrati, E. y Lehmann, D. (1991). Negotiation in a non-cooperative environment. *AIJ, 3*, 255–281.

Krause, A. y Guestrin, C. (2005). Optimal nonmyopic value of information in graphical models: Efficient algorithms and theoretical limits. En *IJCAI-05*.

Krause, A. y Guestrin, C. (2009). Optimal value of information in graphical models. *JAIR, 35*, 557–591.

Krause, A., McMahan, B., Guestrin, C. y Gupta, A. (2008). Robust submodular observation selection. *JMLR, 9*, 2761–2801.

Kripke, S. A. (1963). Semantical considerations on modal logic. *Acta Philosophica Fennica, 16*, 83–94.

Krishna, V. (2002). *Auction Theory*. Academic Press.

Krishnamurthy, V. (2016). *Partially Observed Markov Decision Processes: From Filtering to Controlled Sensing*. Cambridge University Press.

Krishnanand, K. y Ghose, D. (2009). Glowworm swarm optimisation: A new method for optimizing multi-modal functions. *International Journal of Computational Intelligence Studies, 1*, 93–119.

Krizhevsky, A., Sutskever, I. y Hinton, G. E. (2013). ImageNet classification with deep convolutional neural networks. En *NeurIPS 25*.

Krogh, A., Brown, M., Mian, I. S., Sjolander, K. y Haussler, D. (1994). Hidden Markov models in computational biology: Applications to protein modeling. *J. Molecular Biology, 235*, 1501–1531.

Krogh, A. y Hertz, J. A. (1992). A simple weight decay can improve generalization. En *NeurIPS 4*.

Kruppa, E. (1913). Zur Ermittlung eines Objecktes aus zwei Perspektiven mit innerer Orientierung. *Sitz.-Ber. Akad. Wiss., Wien, Math. Naturw., Kl. Abt. IIa, 122*, 1939–1948.

Kübler, S., McDonald, R. y Nivre, J. (2009). *Dependency Parsing*. Morgan & Claypool.

Kuffner, J. J. y LaValle, S. (2000). RRT-connect: An efficient approach to single-query path planning. En *ICRA-00*.

Kuhn, H. W. (1953). Extensive games and the problem of information. En Kuhn, H. W. y Tucker, A. W. (Eds.), *Contributions to the Theory of Games II*. Princeton University Press.

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly, 2*, 83–97.

Kuipers, B. J. (1985). Qualitative simulation. En Bobrow, D. (Ed.), *Qualitative Reasoning About Physical Systems*. MIT Press.

Kuipers, B. J. y Levitt, T. S. (1988). Navigation and mapping in large-scale space. *AIMag, 9*, 25–43.

Kuipers, B. J. (2001). Qualitative simulation. En Meyers, R. A. (Ed.), *Encyclopedia of Physical Science and Technology*. Academic Press.

Kulkarni, T., Kohli, P., Tenenbaum, J. B. y Mansinghka, V.K. (2015). Picture: A probabilistic programming language for scene perception. En *CVPR-15*.

Kumar, P. R. y Varaiya, P. (1986). *Stochastic Systems: Estimation, Identification, and Adaptive Control*. Prentice-Hall.

Kumar, S. (2017). A survey of deep learning methods for relation extraction. arXiv:1705.03645.

Kumar, V. y Kanal, L. N. (1988). The CDP: A unifying formulation for heuristic search, dynamic programming, and branch-and-bound. En Kanal, L. N. y Kumar, V. (Eds.), *Search in Artificial Intelligence*. Springer-Verlag.

Kurien, J., Nayak, P. y Smith, D. E. (2002). Fragment-based conformant planning. En *AIPS-02*.

Kurth, T., Treichler, S., Romero, J., Mudigonda, M., Luehr, N., Phillips, E. H., Mahesh, A., Matheson, M., Deslippe, J., Fatica, M., Prabhat y Houston, M. (2018). Exascale deep learning for climate analytics. arXiv:1810.01993.

Kurzweil, R. (2005). The Singularity is Near. Viking.

Kwok, C., Etzioni, O. y Weld, D. S. (2001). Scaling question answering to the web. En *Proc. 10th International Conference on the World Wide Web*.

La Mettrie, J. O. (1748). *L'homme machine*. E. Luzac, Leyde, France.

La Mura, P. y Shoham, Y. (1999). Expected utility networks. En *UAI-99*.

Laborie, P. (2003). Algorithms for propagating resource constraints in AI planning and scheduling. *AIJ, 143*, 151–188.

Ladkin, P. (1986a). Primitives and units for time specification. En *AAAI-86*.

Ladkin, P. (1986b). Time representation: a taxonomy of interval relations. En *AAAI-86*.

Lafferty, J., McCallum, A. y Pereira, F. (2001). Conditional random fields: Probabilistic models for segmenting and labeling sequence data. En *ICML-01*.

Lai, T. L. y Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics, 6*, 4–22.

Laird, J., Newell, A. y Rosenbloom, P. S. (1987). SOAR: An architecture for general intelligence. *AIJ, 33*, 1–64.

Laird, J., Rosenbloom, P. S. y Newell, A. (1986). Chunking in Soar: The anatomy of a general learning mechanism. *Machine Learning, 1*, 11–46.

Laird, J. (2008). Extending the Soar cognitive architecture. En *Artificial General Intelligence Conference*.

Lake, B., Salakhutdinov, R. y Tenenbaum, J. B. (2015). Human-level concept learning through probabilistic program induction. *Science, 350*, 1332–1338.

Lakoff, G. (1987). *Women, Fire, and Dangerous Things: What Categories Reveal About the Mind*. University of Chicago Press.

Lakoff, G. y Johnson, M. (1980). *Metaphors We Live By*. University of Chicago Press.

Lakoff, G. y Johnson, M. (1999). *Philosophy in the Flesh: The Embodied Mind and Its Challenge to Western Thought*. Basic Books.

Lam, J. y Greenspan,M. (2008). Eye-in-hand visual servoing for accurate shooting in pool robotics. En *5th Canadian Conference on Computer and Robot Vision*.

Lamarck, J. B. (1809). *Philosophie zoologique*. Chez Dentu et L'Auteur, Paris.

Lample, G. y Conneau, A. (2019). Cross-lingual language model pretraining. arXiv:1901.07291.

Landhuis, E. (2004). Lifelong debunker takes on arbiter of neutral choices: Magician-turnedmathematician uncovers bias in a flip of a coin. *Stanford Report*, June 7.

Langdon, W. y Poli, R. (2002). *Foundations of Genetic Programming*. Springer.

Langton, C. (Ed.). (1995). *Artificial Life*. MIT Press.

LaPaugh, A. S. (2010). Algorithms and theory of computation handbook. En Atallah, M. J. y Blanton, M. (Eds.), *VLSI Layout Algorithms*. Chapman & Hall/CRC.

Laplace, P. (1816). *Essai philosophique sur les probabilités* (3a edición). Courcier Imprimeur, Paris.

Larochelle, H. y Murray, I. (2011). The neural autoregressive distribution estimator. En *AISTATS-11*.

Larson, S. C. (1931). The shrinkage of the coefficient of multiple correlation. *J. Educational Psychology, 22*, 45–55.

Laskey, K. B. (1995). Sensitivity analysis for probability assessments in Bayesian networks. *IEEE Transactions on Systems, Man and Cybernetics, 25*, 901–909.

Laskey, K. B. (2008). MEBN: A language for firstorder Bayesian knowledge bases. *AIJ, 172*, 140–178.

Latombe, J.-C. (1991). *Robot Motion Planning*. Kluwer.

Lauritzen, S. (1995). The EMalgorithm for graphical association models with missing data. *Computational Statistics and Data Analysis, 19*, 191–201.

Lauritzen, S., Dawid, A. P., Larsen, B. y Leimer, H. (1990). Independence properties of directed Markov fields. *Networks, 20*, 491–505.

Lauritzen, S. y Spiegelhalter, D. J. (1988). Local computations with probabilities on graphical structures and their application to expert systems. *J. Royal Statistical Society, B 50*, 157–224.

Lauritzen, S. y Wermuth, N. (1989). Graphical models for associations between variables, some of which are qualitative and some quantitative. *Annals of Statistics, 17*, 31–57.

LaValle, S. (2006). *Planning Algorithms*. Cambridge University Press.

Lawler, E. L., Lenstra, J. K., Kan, A. y Shmoys, D. B. (1992). *The Travelling Salesman Problem*. Wiley Interscience.

Lawler, E. L., Lenstra, J. K., Kan, A. y Shmoys, D. B. (1993). Sequencing and scheduling: Algorithms and complexity. En Graves, S. C., Zipkin, P. H. y Kan, A. H. G. R. (Eds.), *Logistics of Production and Inventory: Handbooks in Operations Research and Management Science, Volume 4*. North-Holland.

Lgawler, E. L. y Wood, D. E. (1966). Branch-andbound methods: A survey. *Operations Research, 14*, 699–719.

Lazanas, A. y Latombe, J.-C. (1992). Landmarkbased robot navigation. En *AAAI-92*.

Le, T. A., Baydin, A. G. y Wood, F. (2017). Inference compilation and universal probabilistic programming. En *AISTATS-17*.

Lebedev, M. A. y Nicolelis, M. A. (2006). Brainmachine interfaces: Past, present and future. *Trends in Neurosciences, 29*, 536–546.

Lecoutre, C. (2009). *Constraint Networks: Techniques and Algorithms*. Wiley-IEEE Press.

LeCun, Y., Denker, J. y Solla, S. (1990). Optimal brain damage. En *NeurIPS 2*.

LeCun, Y., Jackel, L., Boser, B. y Denker, J. (1989). Handwritten digit recognition: Applications of neural network chips and automatic learning. *IEEE Communications Magazine, 27*, 41–46.

LeCun, Y., Jackel, L., Bottou, L., Brunot, A., Cortes, C., Denker, J., Drucker, H., Guyon, I., Muller, U., Sackinger, E., Simard, P. y Vapnik, V. N. (1995). Comparison of learning algorithms for handwritten digit recognition. En *Int. Conference on Artificial Neural Networks*.

LeCun, Y., Bengio, Y. y Hinton, G. E. (2015). *Deep learning. Nature, 521*, 436–444.

Lee, D., Seo, H. y Jung, M. W. (2012). Neural basis of reinforcement learning and decision making. *Annual Review of Neuroscience, 35*, 287–308.

Lee, K.-F. (2018). *AI Superpowers: China, Silicon Valley, and the New World Order*. Houghton Mifflin.

Leech, G., Rayson, P., and Wilson, A. (2001). *Word Frequencies in Written and Spoken English: Based on the British National Corpus*. Longman.

Legendre, A. M. (1805). *Nouvelles méthodes pour la d´etermination des orbites des comètes*. *Chez Firmin Didot*, Paris.

Lehmann, J., Isele, R., Jakob, M., Jentzsch, A., Kontokostas, D.,Mendes, P. N., Hellmann, S., Morsey, M., van Kleef, P., Auer, S. y Bizer, C. (2015). DBpedia – A large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web, 6*, 167–195.

Lehrer, J. (2009). *How We Decide*. Houghton Mifflin.

Leike, J., Martic, M., Krakovna, V., Ortega, P. A., Everitt, T., Lefrancq, A., Orseau, L. y Legg, S. (2017). AI safety gridworlds. arXiv:1711.09883.

Lelis, L., Arfaee, S. J., Zilles, S. y Holte, R. C. (2012). Learning heuristic functions faster by using predicted solution costs. En *Proc. Fifth Annual Symposium on Combinatorial Search*.

Lenat, D. B. (1975). BEINGS: Knowledge as interacting experts. En *IJCAI-75*.

Lenat, D. B. y Guha, R. V. (1990). *Building Large Knowledge-Based Systems: Representation and Inference in the CYC Project*. Addison-Wesley.

Leonard, H. S. y Goodman, N. (1940). The calculus of individuals and its uses. *JSL, 5*, 45–55.

Leonard, J. y Durrant-Whyte, H. (1992). *Directed Sonar Sensing for Mobile Robot Navigation*. Kluwer.

Lepage, G. P. (1978). A new algorithm for adaptive multidimensional integration. *Journal of Computational Physics, 27*, 192–203.

Lerner, U. (2002). *Hybrid Bayesian Networks for Reasoning About Complex Systems*. Ph.D. thesis, Stanford University.

Leśniewski, S. (1916). Podstawy og´olnej teorii mnogo´sci. Popławski.

Lesser, V. R. y Corkill, D. D. (1988). The distributed vehicle monitoring testbed: A tool for investigating distributed problem solving networks. En Engelmore, R. y Morgan, T. (Eds.), *Blackboard Systems*. Addison-Wesley.

Letz, R., Schumann, J., Bayerl, S. y Bibel, W. (1992). SETHEO: A high-performance theorem prover. *JAR, 8*, 183–212.

Levesque, H. J. y Brachman, R. J. (1987). Expressiveness and tractability in knowledge representation and reasoning. *Computational Intelligence, 3*, 78–93.

Levin, D. A., Peres, Y. y Wilmer, E. L. (2008). *Markov Chains and Mixing Times*. American Mathematical Society.

Levine, S., Finn, C., Darrell, T. y Abbeel, P. (2016). End-to-end training of deep visuomotor policies. *JMLR, 17*, 1334–1373.

Levine, S., Pastor, P., Krizhevsky, A., Ibarz, J. y Quillen, D. (2018). Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *International Journal of Robotics Research, 37*, 421–436.

Levy, D. (1989). The million pound bridge program. En Levy, D. y Beal, D. (Eds.), *Heuristic Programming in Artificial Intelligence*. Ellis Horwood.

Levy, D. (2008). *Love and Sex with Robots: The Evolution of Human—Robot Relationships*. Harper.

Levy, O. y Goldberg, Y. (2014). Linguistic regularities in sparse and explicit word representations. En *Proc. Eighteenth Conference on Computational Natural Language Learning*.

Leyton-Brown, K. y Shoham, Y. (2008). *Essentials of Game Theory: A Concise, Multidisciplinary Introduction*. Morgan & Claypool.

Li, C. M. y Anbulagan (1997). Heuristics based on unit propagation for satisfiability problems. En *IJCAI-97*.

Li, K. y Malik, J. (2018a). Implicit maximum likelihood estimation. arXiv:1809.09087.

Li, K. y Malik, J. (2018b). On the implicit assumptions of GANs. arXiv:1811.12402.

Li, M., Vitányi, P. *et al*. (2008). *An Introduction to Kolmogorov Complexity and Its Applications* (3a edición). Springer-Verlag.

Li, T.-M., Gharbi, M., Adams, A., Durand, F. y Ragan-Kelley, J. (2018). Differentiable programming for image processing and deep learning in Halide. *ACM Transactions on Graphics, 37*, 139.

Li, W. y Todorov, E. (2004). Iterative linear quadratic regulator design for nonlinear biological movement systems. En *Proc. 1st International Conference on Informatics in Control, Automation and Robotics*.

Li, X. y Yao, X. (2012). Cooperatively coevolving particle swarms for large scale optimization. *IEEE Trans. Evolutionary Computation, 16*, 210–224.

Li, Z., Li, P., Krishnan, A. y Liu, J. (2011). Largescale dynamic gene regulatory network inference combining differential equation models with local dynamic Bayesian network analysis. *Bioinformatics, 27 19*, 2686–91.

Liang, P., Jordan, M. I., and Klein, D. (2011). Learning dependency-based compositional semantics. arXiv:1109.6841.

Liang, P. y Potts, C. (2015). Bringing machine learning and compositional semantics together. *Annual Review of Linguistics, 1*, 355–376.

Liberatore, P. (1997). The complexity of the language A. *Electronic Transactions on Artificial Intelligence, 1*, 13–38.

Lifschitz, V. (2001). Answer set programming and plan generation. *AIJ, 138*, 39–54.

Lighthill, J. (1973). Artificial intelligence: A general survey. In Lighthill, J., Sutherland, N. S., Needham, R. M., Longuet-Higgins, H. C. y Michie, D. (Eds.), *Artificial Intelligence: A Paper Symposium*. Science Research Council of Great Britain.

Lillicrap, T., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. y Wierstra, D. (2015). Continuous control with deep reinforcement learning. arXiv:1509.02971.

Lin, S. (1965). Computer solutions of the travelling salesman problem. *Bell Systems Technical Journal, 44*(10), 2245–2269.

Lin, S. y Kernighan, B. W. (1973). An effective heuristic algorithm for the travelling-salesman problem. *Operations Research, 21*, 498–516.

Lindley, D. V. (1956). On a measure of the information provided by an experiment. *Annals of Mathematical Statistics, 27*, 986–1005.

Lindsay, R. K., Buchanan, B. G., Feigenbaum, E. A. y Lederberg, J. (1980). *Applications of Artificial Intelligence for Organic Chemistry: The DENDRAL Project*. McGraw-Hill.

Lindsten, F., Jordan, M. I., y Schön, T. B. (2014). Particle Gibbs with ancestor sampling. *JMLR, 15*, 2145–2184.

Littman, M. L. (1994). Markov games as a framework for multi-agent reinforcement learning. En *ICML-94*.

Littman, M. L., Cassandra, A. R. y Kaelbling, L. P. (1995). Learning policies for partially observable environments: Scaling up. En *ICML-95*.

Littman, M. L. (2015). Reinforcement learning improves behaviour from evaluative feedback. *Nature, 521*, 445–451.

Littman, M. L., Topcu, U., Fu, J., Isbell, C., Wen, M. MacGlashan, J. (2017). Environment-independent task specifications via GLTL. arXiv:1704.04341.

Liu, B., Gemp, I., Ghavamzadeh, M., Liu, J., Mahadevan, S. y Petrik, M. (2018). Proximal gradient temporal difference learning: Stable reinforcement learning with polynomial sample complexity. *JAIR, 63*, 461–494.

Liu, H., Simonyan, K., Vinyals, O., Fernando, C. y Kavukcuoglu, K. (2017). Hierarchical representations for efficient architecture search. arXiv:1711.00436.

Liu, H., Simonyan, K. y Yang, Y. (2019). DARTS: Differentiable architecture search. En *ICLR-19*.

Liu, J. y Chen, R. (1998). Sequential Monte Carlo methods for dynamic systems. *JASA, 93*, 1022–1031.

Liu, J. y West, M. (2001). Combined parameter and state estimation in simulation-based filtering. En Doucet, A., de Freitas, J. F. G. y Gordon, N. (Eds.), *Sequential Monte Carlo Methods in Practice*. Springer.

Liu, L. T., Dean, S., Rolf, E., Simchowitz, M. Hardt, M. (2018a). Delayed impact of fair machine learning. arXiv:1803.04383.

Liu, M.-Y., Breuel, T. y Kautz, J. (2018b). Unsupervised image-to-image translation networks. En *NeurIPS 30*.

Liu, X., Faes, L., Kale, A. U., Wagner, S. K., Fu, D. J., Bruynseels, A., Mahendiran, T., Moraes, G., Shamdas, M., Kern, C., Ledsam, J. R., Schmid, M., Balaskas, K., Topol, E., Bachmann, L.M., Keane, P. A. y Denniston, A. K. (2019a). A comparison of deep learning performance against health-care professionals in detecting diseases from medical imaging: A systematic review and meta-analysis. *The Lancet Digital Health*.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. y Stoyanov, V. (2019b). RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692.

Liu, Y., Jain, A., Eng, C., Way, D. H., Lee, K., Bui, P., Kanada, K., de Oliveira Marinho, G., Gallegos, J., Gabriele, S., Gupta, V., Singh, N., Natarajan, V., Hofmann-Wellenhof, R., Corrado, G., Peng, L., Webster, D. R., Ai, D., Huang, S., Liu, Y., Dunn, R. C. y Coz, D. (2019c). A deep learning system for differential diagnosis of skin diseases. arXiv:1909.

Liu, Y., Gadepalli, K. K., Norouzi, M., Dahl, G., Kohlberger, T., Venugopalan, S., Boyko, A. S., Timofeev, A., Nelson, P. Q., Corrado, G., Hipp, J. D., Peng, L. y Stumpe, M. C. (2017). Detecting cancer metastases on gigapixel pathology images. arXiv:1703.02442.

Liu, Y., Kohlberger, T., Norouzi, M., Dahl, G., Smith, J. L., Mohtashamian, A., Olson, N., Peng, L., Hipp, J. D., and Stumpe, M. C. (2018). Artificial intelligence-based breast cancer nodal metastasis detection: Insights into the black box for pathologists. *Archives of Pathology & Laboratory Medicine, 143*, 859–868.

Livescu, K., Glass, J. y Bilmes, J. (2003). Hidden feature modeling for speech recognition using dynamic Bayesian networks. En EUROSPEECH-2003.

Lloyd, S. (2000). Ultimate physical limits to computation. *Nature, 406*, 1047–1054.

Lloyd, W. F. (1833). *Two Lectures on the Checks to Population*. Oxford University.

Llull, R. (1305). Ars Magna. Published as Salzinger, I. *et al*. (Eds.), *Raymundi Lulli Opera omnia*, Mainz, 1721–1742.

Loftus, E. y Palmer, J. (1974). Reconstruction of automobile destruction: An example of the interaction between language andmemory. *J. Verbal Learning and Verbal Behavior, 13*, 585–589.

Lohn, J. D., Kraus, W. F., and Colombano, S. P. (2001). Evolutionary optimization of yagi-uda antennas. En *Proc. Fourth International Conference on Evolvable Systems*.

Longuet-Higgins, H. C. (1981). A computer algorithm for reconstructing a scene from two projections. *Nature, 293*, 133–135.

Loos, S., Irving, G., Szegedy, C. y Kaliszyk, C. (2017). Deep network guided proof search. En *Proc. 21st Int'l Conf. on Logic for Programming, Artificial Intelligence and Reasoning*.

Lopez de Segura, R. (1561). *Libro de la invencion liberal y arte del juego del axedrez*. Andres de Angulo.

Lorentz, R. (2015). Early playout termination in MCTS. En Plaat, A., Van den Herik, J. y Kosters, W. (Eds.), *Advances in Computer Games*. Springer-Verlag.

Love, N., Hinrichs, T. y Genesereth, M. R. (2006). General game playing: Game description language specification. Tech. rep., Stanford University Computer Science Dept.

Lovejoy, W. S. (1991). A survey of algorithmic methods for partially observed Markov decision processes. *Annals of Operations Research, 28*, 47–66.

Lovelace, A. (1843). Sketch of the analytical engine invented by Charles Babbage. Notes appended to Lovelace's translation of an article of the above title written by L. F.Menabrea based on lectures by Charles babbage in 1840. The translation appeared in R. Taylor (Ed.), *Scientific Memoirs, vol. III*. R. and J. E. Taylor, London.

Loveland, D. (1970). A linear format for resolution. En *Proc. IRIA Symposium on Automatic Demonstration*.

Lowe, D. (1987). Three-dimensional object recognition from single two-dimensional images. *AIJ, 31*, 355–395.

Lowe, D. (2004). Distinctive image features from scale-invariant keypoints. *IJCV, 60*, 91–110.

Löwenheim, L. (1915). Über möglichkeiten im Relativkalkül. *Mathematische Annalen, 76*, 447–470.

Lowerre, B. T. (1976). *The HARPY Speech Recognition System*. Ph.D. thesis, Computer Science Department, Carnegie-Mellon University.

Lowry, M. (2008). Intelligent software engineering tools for NASA's crew exploration vehicle. En *ISMIS-08*.

Loyd, S. (1959). *Mathematical Puzzles of Sam Loyd: Selected and Edited by Martin Gardner*. Dover.

Lozano-Perez, T. (1983). Spatial planning: A configuration space approach. *IEEE Transactions on Computers, C-32*, 108–120.

Lozano-Perez, T., Mason, M. y Taylor, R. (1984). Automatic synthesis of fine-motion strategies for robots. *Int. J. Robotics Research, 3*, 3–24.

Lu, F. y Milios, E. (1997). Globally consistent range scan alignment for environment mapping. *Autonomous Robots, 4*, 333–349.

Lubberts, A. y Miikkulainen, R. (2001). Coevolving a Go-playing neural network. En *GECCO-01*.

Luby, M., Sinclair, A. y Zuckerman, D. (1993). Optimal speedup of Las Vegas algorithms. *Information Processing Letters, 47*, 173–180.

Lucas, J. R. (1961). Minds, machines, and Gödel. *Philosophy, 36*.

Lucas, J. R. (1976). This Gödel is killing me: A rejoinder. *Philosophia, 6*, 145–148.

Lucas, P., Van der Gaag, L. y Abu-Hanna, A. (2004). Bayesian networks in biomedicine and healthcare. *Artificial Intelligence in Medicine*.

Luce, D. R. y Raiffa, H. (1957). *Games and Decisions*. Wiley.

Lukasiewicz, T. (1998). Probabilistic logic programming. En *ECAI-98*.

Lundberg, S. M. y Lee, S.-I. (2018). A unified approach to interpreting model predictions. En *NeurIPS 30*.

Lunn, D., Jackson, C., Best, N., Thomas, A. y Spiegelhalter, D. J. (2013). *The BUGS Book: A Practical Introduction to Bayesian Analysis*. Chapman and Hall.

Lunn, D., Thomas, A., Best, N. y Spiegelhalter, D. J. (2000). WinBUGS—a Bayesian modelling framework: Concepts, structure, and extensibility. *Statistics and Computing, 10*, 325–337.

Luo, S., Bimbo, J., Dahiya, R. y Liu, H. (2017). Robotic tactile perception of object properties: A review. *Mechatronics, 48*, 54–67.

Lyman, P. y Varian, H. R. (2003). How much information? www.sims.berkeley.edu/how-much-info-2003.

Lynch, K. y Park, F. C. (2017). *Modern Robotics*. Cambridge University Press.

Machina, M. (2005). Choice under uncertainty. En *Encyclopedia of Cognitive Science*. Wiley.

MacKay, D. J. C. (2002). *Information Theory, Inference and Learning Algorithms*. Cambridge University Press.

MacKenzie, D. (2004). *Mechanizing Proof*. MIT Press.

Mackworth, A. K. (1977). Consistency in networks of relations. *AIJ, 8*, 99–118.

Mackworth, A. K. y Freuder, E. C. (1985). The complexity of some polynomial network consistency algorithms for constraint satisfaction problems. *AIJ, 25*, 65–74.

Madhavan, R. y Schlenoff, C. I. (2003). Moving object prediction for off-road autonomous navigation. En *Unmanned Ground Vehicle Technology V*.

Mailath, G. y Samuelson, L. (2006). *Repeated Games and Reputations: Long-Run Relationships*. Oxford University Press.

Majercik, S. M. y Littman, M. L. (2003). Contingent planning under uncertainty via stochastic satisfiability. *AIJ, 147*, 119–162.

Malhotra, P., Vig, L., Shroff, G. y Agarwal, P. (2015). Long short term memory networks for anomaly detection in time series. En *ISANN-15*.

Malik, D., Palaniappan, M., Fisac, J. F., Hadfield-Menell, D., Russell, S. J. y Dragan, A. D. (2018). An efficient, generalized bellman update for cooperative inverse reinforcement learning. En *ICML-18*.

Malone, T. W. (2004). *The Future of Work*. Harvard Business Review Press.

Maneva, E., Mossel, E. y Wainwright, M. (2007). A new look at survey propagation and its generalizations. arXiv:cs/0409012.

Manna, Z. y Waldinger, R. (1971). Toward automatic program synthesis. *CACM, 14*, 151–165.

Manna, Z. y Waldinger, R. (1985). *The Logical Basis for Computer Programming: Volume 1: Deductive Reasoning*. Addison-Wesley.

Manne, A. S. (1960). Linear programming and sequential decisions. *Management Science, 6*, 259–267.

Manning, C. y Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.

Manning, C., Raghavan, P. y Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Mannion, M. (2002). Using first-order logic for product line model validation. En *Software Product Lines: Second International Conference*.

Mansinghka, V. K., Selsam, D. y Perov, Y. (2013). Venture: A higher-order probabilistic programming platform with programmable inference. arXiv:1404.0099.

Marbach, P. y Tsitsiklis, J. N. (1998). Simulationbased optimization of Markov reward processes. Technical report, Laboratory for Information and Decision Systems, MIT.

Marcus, G. (2009). *Kluge: The Haphazard Evolution of the Human Mind*. Mariner Books.

Marcus, M. P., Santorini, B. y Marcinkiewicz, M. A. (1993). Building a large annotated corpus of English: The Penn treebank. *Computational Linguistics, 19*, 313–330.

Marinescu, R. y Dechter, R. (2009). AND/OR branch-and-bound search for combinatorial optimization in graphical models. *AIJ, 173*, 1457–1491.

Markov, A. (1913). An example of statistical investigation in the text of "Eugene Onegin" illustrating coupling of "tests" in chains. *Proc. Academy of Sciences of St. Petersburg, 7*, 153–162.

Marler, R. T. y Arora, J. S. (2004). Survey of multiobjective optimization methods for engineering. *Structural and Multidisciplinary Optimization, 26*, 369–395.

Maron, M. E. (1961). Automatic indexing: An experimental inquiry. *JACM, 8*, 404–417.

Màrquez, L. y Rodríguez, H. (1998). Part-of-speech tagging using decision trees. En *ECML-98*.

Marr, D. y Poggio, T. (1976). Cooperative computation of stereo disparity. *Science, 194*, 283–287.

Marr, D. (1982). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman.

Marriott, K. y Stuckey, P. J. (1998). *Programming with Constraints: An Introduction*. MIT Press.

Marsland, S. (2014). *Machine Learning: An Algorithmic Perspective* (2a edición). CRC Press.

Martelli, A. y Montanari, U. (1973). Additive AND/OR graphs. En *IJCAI-73*.

Martelli, A. (1977). On the complexity of admissible search algorithms. *AIJ, 8*, 1–13.

Marthi, B., Pasula, H., Russell, S. J. y Peres, Y. (2002). Decayed MCMC filtering. En *UAI-02*.

Marthi, B., Russell, S. J., Latham, D. y Guestrin, C. (2005). Concurrent hierarchical reinforcement learning. En *IJCAI-05*.

Marthi, B., Russell, S. J. y Wolfe, J. (2007). Angelic semantics for high-level actions. En *ICAPS-07*.

Marthi, B., Russell, S. J. y Wolfe, J. (2008). Angelic hierarchical planning: Optimal and online algorithms. En *ICAPS-08*.

Martin, D., Fowlkes, C. y Malik, J. (2004). Learning to detect natural image boundaries using local brightness, color, and texture cues. *PAMI, 26*, 530–549.

Martin, F. G. (2012). Will massive open online courses change how we teach? *CACM, 55*, 26–28.

Martin, J. H. (1990). *A Computational Model of Metaphor Interpretation*. Academic Press.

Mason, M. (1993). Kicking the sensing habit. *AIMag, 14*, 58–59.

Mason, M. (2001). *Mechanics of Robotic Manipulation*. MIT Press.

Mason, M. y Salisbury, J. (1985). *Robot Hands and the Mechanics of Manipulation*. MIT Press.

Mataric, M. J. (1997). Reinforcement learning in the multi-robot domain. *Autonomous Robots, 4*, 73–83.

Mates, B. (1953). *Stoic Logic*. University of California Press.

Matuszek, C., Cabral, J., Witbrock, M. y DeOliveira, J. (2006). An introduction to the syntax and semantics of Cyc. En *Proc. AAAI Spring Symposium on Formalizing and Compiling Background Knowledge and Its Applications to Knowledge Representation and Question Answering*.

Mausam and Kolobov, A. (2012). *Planning with Markov Decision Processes: An AI Perspective*. Morgan & Claypool.

Maxwell, J. (1868). On governors. *Proc. Roy. Soc., 16*, 270–283.

Mayer, J., Khairy, K. y Howard, J. (2010). Drawing an elephant with four complex parameters. *American Journal of Physics, 78*, 648–649.

Mayor, A. (2018). *Gods and Robots: Myths, Machines, and Ancient Dreams of Technology*. Princeton University Press.

McAllester, D. A. (1980). An outlook on truth maintenance. AI memo, MIT AI Laboratory.

McAllester, D. A. (1988). Conspiracy numbers for min-max search. *AIJ, 35*, 287–310.

McAllester, D. A. (1998). What is the most pressing issue facing AI and the AAAI today? Candidate statement, election for Councilor of the American Association for Artificial Intelligence.

McAllester, D. A. y Rosenblitt, D. (1991). Systematic nonlinear planning. En *AAAI-91*.

McAllester, D. A. (1990). Truth maintenance. En *AAAI-90*.

McAllester, D. A., Milch, B. y Goodman, N. D. (2008). Random-world semantics and syntactic independence for expressive languages. Technical report, MIT.

McCallum, A. (2003). Efficiently inducing features of conditional random fields. En *UAI-03*.

McCallum, A., Schultz, K. y Singh, S. (2009). FACTORIE: Probabilistic programming via imperatively defined factor graphs. En *NeurIPS 22*.

McCarthy, J. (1958). Programs with common sense. En *Proc. Symposium onMechanisation of Thought Processes*.

McCarthy, J. (1963). *Situations, actions, and causal laws*. Memo, Stanford University Artificial Intelligence Project.

McCarthy, J. (1968). Programs with common sense. En Minsky, M. L. (Ed.), *Semantic Information Processing*. MIT Press.

McCarthy, J. (1980). Circumscription: A form of non-monotonic reasoning. *AIJ, 13*, 27–39.

McCarthy, J. (2007). From here to human-level AI. *AIJ, 171*.

McCarthy, J. y Hayes, P. J. (1969). Some philosophical problems from the standpoint of artificial intelligence. En Meltzer, B., Michie, D. y Swann, M. (Eds.), *Machine Intelligence 4*. Edinburgh University Press.

McCawley, J. D. (1988). *The Syntactic Phenomena of English*. University of Chicago Press.

McCorduck, P. (2004). *Machines Who Think: A Personal Inquiry Into the History and Prospects of Artificial Intelligence* (Revised edition). A K Peters.

McCulloch, W. S. y Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics, 5*, 115–137.

McCune,W. (1997). Solution of the Robbins problem. *JAR, 19*, 263–276.

McCune, W. (1990). Otter 2.0. En *International Conference on Automated Deduction*.

McDermott, D. (1976). Artificial intelligence meets natural stupidity. *SIGART Newsletter, 57*, 4–9.

McDermott, D. (1978a). Planning and acting. *Cognitive Science, 2*, 71–109.

McDermott, D. (1978b). Tarskian semantics, or no notation without denotation! *Cognitive Science, 2*, 277–282.

McDermott, D. (1985). Reasoning about plans. En Hobbs, J. and Moore, R. (Eds.), *Formal theories of the commonsense world*. Ablex.

McDermott, D. (1987). A critique of pure reason. *Computational Intelligence, 3*, 151–237.

McDermott, D. (1996). A heuristic estimator for means-ends analysis in planning. En *ICAPS-96*.

McDermott, D. y Doyle, J. (1980). Non-monotonic logic: i. *AIJ, 13*, 41–72.

McDermott, J. (1982). R1: A rule-based configure of computer systems. *AIJ, 19*, 39–88.

McEliece, R. J., MacKay, D. J. C. y Cheng, J.-F. (1998). Turbo decoding as an instance of Pearl's "belief propagation" algorithm. *IEEE Journal on Selected Areas in Communications, 16*, 140–152.

McGregor, J. J. (1979). Relational consistency algorithms and their application in finding subgraph and graph isomorphisms. *Information Sciences, 19*, 229–250.

McIlraith, S. y Zeng, H. (2001). Semantic web services. *IEEE Intelligent Systems, 16*, 46–53.

McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas*. O'Reilly.

McLachlan, G. J. y Krishnan, T. (1997). *The EM Algorithm and Extensions*. Wiley.

McMahan, H. B. y Andrew, G. (2018). A general approach to adding differential privacy to iterative training procedures. arXiv:1812.06210.

McMillan, K. L. (1993). *Symbolic Model Checking*. Kluwer.

McWhorter, J. H. (2014). *The Language Hoax: Why the World Looks the Same in Any Language*. Oxford University Press.

Meehl, P. (1955). *Clinical vs. Statistical Prediction*. University of Minnesota Press.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K. y Galstyan, A. (2019). A survey on bias and fairness in machine learning. arXiv:1908.09635.

Mendel, G. (1866). Versuche über pflanzen-hybriden. *Verhandlungen des Naturforschenden Vereins, Abhandlungen, Brünn, 4*, 3–47. Translated into English by C. T. Druery, published by Bateson (1902).

Mercer, J. (1909). Functions of positive and negative type and their connection with the theory of integral equations. *Phil. Trans. Roy. Soc., A*, 209, 415–446.

Merleau-Ponty, M. (1945). *Phenomenology of Perception*. Routledge.

Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller, A. y Teller, E. (1953). Equations of state calculations by fast computing machines. *J. Chemical Physics, 21*, 1087–1091.

Metropolis, N. y Ulam, S. (1949). The beginning of the Monte Carlo method. *Journal of the American Statistical Association, 44*, 335–341.

Mézard, M., Parisi, G. y Virasoro, M. (1987). *Spin Glass Theory and Beyond: An Introduction to the ReplicaMethod and Its Applications*. World Scientific.

Michie, D. (1966). Game-playing and game-learning automata. In Fox, L. (Ed.), *Advances in Programming and Non-Numerical Computation*. Pergamon.

Michie, D. (1972). Machine intelligence at Edinburgh. *Management Informatics, 2*, 7–12.

Michie, D. y Chambers, R. A. (1968). BOXES: An experiment in adaptive control. En Dale, E. y Michie, D. (Eds.), *Machine Intelligence 2*. Elsevier.

Michie, D. (1963). Experiments on the mechanization of game-learning Part I. Characterization of the model and its parameters. *The Computer Journal, 6*, 232–236.

Miikkulainen, R., Liang, J., Meyerson, E., Rawal, A., Fink, D., Francon, O., Raju, B., Shahrzad, H., Navruzyan, A., Duffy, N. *et al*. (2019). Evolving deep neural networks. En *Artificial Intelligence in the Age of Neural Networks and Brain Computing*. Elsevier.

Mikolov, T., Chen, K., Corrado, G. y Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.

Mikolov, T., Karafiát, M., Burget, L., Černockỳ, J. y Khudanpur, S. (2010). Recurrent neural network based language model. En *Eleventh Annual Conference of the International Speech Communication Association*.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. y Dean, J. (2014). Distributed representations of words and phrases and their compositionality. En *NeurIPS 26*.

Milch, B. (2006). *Probabilistic Models with Unknown Objects*. Ph.D. thesis, UC Berkeley.

Milch, B., Marthi, B., Sontag, D., Russell, S. J., Ong, D. y Kolobov, A. (2005). BLOG: Probabilistic models with unknown objects. En *IJCAI-05*.

Milch, B., Zettlemoyer, L., Kersting, K., Haimes, M. y Kaelbling, L. P. (2008). Lifted probabilistic inference with counting formulas. En *AAAI-08*.

Milgrom, P. (1997). Putting auction theory to work: The simultaneous ascending auction. Tech. rep., Stanford University Department of Economics.

Mill, J. S. (1863). *Utilitarianism*. Parker, Son y Bourn, London.

Miller, A. C., Merkhofer, M. M., Howard, R. A., Matheson, J. E. y Rice, T. R. (1976). Development of automated aids for decision analysis. Technical report, SRI International.

Miller, T., Howe, P. y Sonenberg, L. (2017). Explainable AI: Beware of inmates running the asylum. En *Proc. IJCAI-17 Workshop on Explainable AI.*

Minka, T. (2010). Bayesian linear regression. Unpublished manuscript.

Minka, T., Cleven, R. y Zaykov, Y. (2018). TrueSkill 2: An improved Bayesian skill rating system. Tech. rep., Microsoft Research.

Minker, J. (2001). *Logic-Based Artificial Intelligence*. Kluwer.

Minsky, M. L. (1975). A framework for representing knowledge. InWinston, P. H. (Ed.), *The Psychology of Computer Vision*. McGraw-Hill.

Minsky, M. L. (1986). *The Society of Mind*. Simon y Schuster.

Minsky, M. L. (2007). *The Emotion Machine: Commonsense Thinking, Artificial Intelligence, and the Future of the Human Mind*. Simon and Schuster.

Minsky, M. L. y Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Minsky, M. L. y Papert, S. (1988). *Perceptrons: An Introduction to Computational Geometry* (Expanded edition). MIT Press.

Minsky, M. L., Singh, P. y Sloman, A. (2004). The St. Thomas common sense symposium: Designing architectures for human-level intelligence. *AIMag, 25*, 113–124.

Minton, S., Johnston, M. D., Philips, A. B., and Laird, P. (1992). Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems. *AIJ, 58*, 161–205.

Mirjalili, S. M. y Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software, 69*, 46–61.

Misak, C. (2004). *The Cambridge Companion to Peirce*. Cambridge University Press.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D. y Gebru, T. (2019). Model cards for model reporting. *Proc. of the Conference on Fairness, Accountability, and Transparency*.

Mitchell, M. (1996). *An Introduction to Genetic Algorithms*. MIT Press.

Mitchell, M. (2019). *Artificial Intelligence: A Guide for Thinking Humans*. Farrar, Straus and Giroux.

Mitchell, M., Holland, J. H. y Forrest, S. (1996). When will a genetic algorithm outperform hill climbing? En *NeurIPS 6*.

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.

Mitchell, T. M. (2005). Reading the web: A breakthrough goal for AI. *AIMag, 26*.

Mitchell, T. M. (2007). Learning, information extraction and the web. En *ECML-07*.

Mitchell, T.M., Cohen, W., Hruschka, E., Talukdar, P., Yang, B., Betteridge, J., Carlson, A., Dalvi, B., Gardner, M., Kisiel, B. *et al*. (2018). Never-ending learning. *CACM, 61*, 103–115.

Mitchell, T.M., Shinkareva, S. V., Carlson, A., Chang, K.-M., Malave, V. L., Mason, R. A. y Just, M. A. (2008). Predicting human brain activity associated with the meanings of nouns. *Science, 320*, 1191–1195.

Mittelstadt, B. (2019). Principles alone cannot guarantee ethical AI. *Nature Machine Intelligence, 1*, 501–507.

Mitten, L. G. (1960). An analytic solutlon to the least cost testing sequence problem. *Journal of Industrial Engineering, 11*, 17.

Miyato, T., Kataoka, T., Koyama, M. y Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. arXiv:1802.05957.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. y Riedmiller, M. A. (2013). Playing Atari with deep reinforcement learning. arXiv:1312.5602.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M. A., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. y Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*, 529–533.

Mohr, R. y Henderson, T. C. (1986). Arc and path consistency revisited. *AIJ, 28*, 225–233.

Montague, R. (1970). English as a formal language. En Visentini, B. (Ed.), *Linguaggi nella Società en ella Tecnica*. Edizioni di Comunità.

Montague, R. (1973). The proper treatment of quantification in ordinary English. En Hintikka, K. J. J., Moravcsik, J.M. E. y Suppes, P. (Eds.), *Approaches to Natural Language. D. Reidel*.

Montanari, U. (1974). Networks of constraints: Fundamental properties and applications to picture processing. *Information Sciences, 7*, 95–132.

Mgontemerlo, M. y Thrun, S. (2004). Large-scale robotic 3-D mapping of urban structures. En *Proc. International Symposium on Experimental Robotics*.

Montemerlo, M., Thrun, S., Koller, D. y Wegbreit, B. (2002). FastSLAM: A factored solution to the simultaneous localization and mapping problem. En *AAAI-02*.

Mooney, R. (1999). Learning for semantic interpretation: Scaling up without dumbing down. En *Proc. 1st Workshop on Learning Language in Logic*.

Moore, A. M. y Wong, W.-K. (2003). Optimal reinsertion: A new search operator for accelerated and more accurate Bayesian network structure learning. En *ICML-03*.

Moore, A. W. y Atkeson, C. G. (1993). Prioritized sweeping—Reinforcement learning with less data and less time. *Machine Learning, 13*, 103–130.

Moore, A. W. y Lee, M. S. (1997). Cached sufficient statistics for efficient machine learning with large datasets. *JAIR, 8*, 67–91.

Moore, E. F. (1959). The shortest path through amaze. En *Proc. International Symposium on the Theory of Switching, Part II*. Harvard University Press.

Moore, R. C. (1980). Reasoning about knowledge and action. Artificial intelligence center technical note, SRI International.

Moore, R. C. (1985). A formal theory of knowledge and action. In Hobbs, J. R. y Moore, R. C. (Eds.), *Formal Theories of the Commonsense World*. Ablex.

Moore, R. C. y DeNero, J. (2011). L1 and L2 regularization for multiclass hinge loss models. En *Symposium on Machine Learning in Speech and Natural Language Processing*.

Moravčik, M., Schmid, M., Burch, N., Lis´y, V., Morrill, D., Bard, N., Davis, T., Waugh, K., Johanson, M. y Bowling, M. (2017). Deepstack: Expert-level artificial intelligence in no-limit poker. arXiv:1701.01724.

Moravec, H. P. (1983). The Stanford cart and the CMU rover. *Proc. IEEE, 71*, 872–884.

Moravec, H. P. y Elfes, A. (1985). High resolution maps from wide angle sonar. En *ICRA-85*.

Moravec, H. P. (2000). *Robot: Mere Machine to Transcendent Mind*. Oxford University Press.

Morgan, C. L. (1896). *Habit and Instinct*. Edward Arnold.

Morgan, T. J. H. y Griffiths, T. L. (2015). What the Baldwin Effect affects. En *COGSCI-15*.

Morjaria, M. A., Rink, F. J., Smith, W. D., Klempner, G., Burns, C. y Stein, J. (1995). Elicitation of probabilities for belief networks: Combining qualitative and quantitative information. En *UAI-95*.

Morrison, P. y Morrison, E. (Eds.). (1961). *Charles Babbage and His Calculating Engines: Selected Writings by Charles Babbage and Others*. Dover.

Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L. y Malik, S. (2001). Chaff: Engineering an efficient SAT solver. En *Proc. 38th Design Automation Conference*.

Mott, A., Job, J., Vlimant, J.-R., Lidar, D. y Spiropulu, M. (2017). Solving a Higgs optimization problem with quantum annealing for machine learning. *Nature, 550*, 375.

Motzkin, T. S. y Schoenberg, I. J. (1954). The relaxation method for linear inequalities. *Canadian Journal of Mathematics, 6*, 393–404.

Moutarlier, P. y Chatila, R. (1989). Stochastic multisensory data fusion for mobile robot location and environment modeling. En *ISRR-89*.

Mueller, E. T. (2006). *Commonsense Reasoning*. Morgan Kaufmann.

Muggleton, S. H. y De Raedt, L. (1994). Inductive logic programming: Theory and methods. *J. Logic Programming, 19/20*, 629–679.

Müller, M. (2002). Computer Go. *AIJ, 134*, 145–179.

Mumford, D. y Shah, J. (1989). Optimal approximations by piece-wise smooth functions and associated variational problems. *Commun. Pure Appl. Math., 42*, 577–685.

Mundy, J. y Zisserman, A. (Eds.). (1992). *Geometric Invariance in Computer Vision*. MIT Press.

Munos, R., Stepleton, T., Harutyunyan, A. y Bellemare, M. G. (2017). Safe and efficient off-policy reinforcement learning. En *NeurIPS 29*.

Murphy, K. (2002). Dynamic Bayesian Networks: Representation, Inference and Learning. Ph.D. thesis, UC Berkeley.

Murphy, K. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.

Murphy, K. y Mian, I. S. (1999). Modelling gene expression data using Bayesian networks. Tech. rep., Computer Science Division, UC Berkeley.

Murphy, K. y Russell, S. J. (2001). Rao-Blackwellised particle filtering for dynamic Bayesian networks. En Doucet, A., de Freitas, J. F. G. y Gordon, N. J. (Eds.), *Sequential Monte Carlo Methods in Practice*. Springer-Verlag.

Murphy, K. y Weiss, Y. (2001). The factored frontier algorithm for approximate inference in DBNs. En *UAI-01*.

Murphy, R. (2000). *Introduction to AI Robotics*. MIT Press.

Murray, L. M. (2013). Bayesian state-space modelling on high-performance hardware using LibBi. arXiv:1306.3277.

Murray, R. M. (2017). *A Mathematical Introduction to Robotic Manipulation*. CRC Press.

Murray-Rust, P., Rzepa, H. S., Williamson, J. y Willighagen, E. L. (2003). Chemical markup, XML and the world–wide web. 4. CML schema. J. Chem. Inf. Comput. *Sci., 43*, 752–772.

Murthy, C. y Russell, J. R. (1990). A constructive proof of Higman's lemma. En *LICS-90*.

Muscettola, N. (2002). Computing the envelope for stepwise-constant resource allocations. En *CP-02*.

Muscettola, N., Nayak, P., Pell, B. y Williams, B. (1998). Remote agent: To boldly go where no AI system has gone before. *AIJ, 103*, 5–48.

Muslea, I. (1999). Extraction patterns for information extraction tasks: A survey. En *Proc. AAAI-99Workshop on Machine Learning for Information Extraction*.

Muth, J. T., Vogt, D. M., Truby, R. L., Mengüç, Y., Kolesky, D. B., Wood, R. J. y Lewis, J. A. (2014). Embedded 3D printing of strain sensors within highly stretchable elastomers. *Advanced Materials, 26*, 6307–6312.

Myerson, R. (1981). Optimal auction design. *Mathematics of Operations Research, 6*, 58–73.

Myerson, R. (1986). Multistage games with communication. *Econometrica, 54*, 323–358.

Myerson, R. (1991). *Game Theory: Analysis of Conflict*. Harvard University Press.

Nair, V. y Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. En *ICML-10*.

Nalwa, V. S. (1993). *A Guided Tour of Computer Vision*. Addison-Wesley.

Narayanan, A., Shi, E. y Rubinstein, B. I. (2011). Link prediction by de-anonymization: How we won the Kaggle social network challenge. En *IJCNN-11*.

Narayanan, A. y Shmatikov, V. (2006). How to break anonymity of the Netflix prize dataset. arXiv:cs/0610105.

Nash, J. (1950). Equilibrium points in N-person games. *PNAS, 36*, 48–49.

Nash, P. (1973). *Optimal Allocation of Resources Between Research Projects*. Ph.D. thesis, University of Cambridge.

Nayak, P. y Williams, B. (1997). Fast context switching in real-time propositional reasoning. En *AAAI-97*.

Neches, R., Swartout, W. R. y Moore, J. D. (1985). Enhanced maintenance and explanation of expert systems through explicit models of their development. *IEEE Transactions on Software Engineering, SE-11*, 1337–1351.

Nemhauser, G. L., Wolsey, L. A. y Fisher, M. L. (1978). An analysis of approximations for maximizing submodular set functions I. *Mathematical Programming, 14*, 265–294.

Nesterov, Y. y Nemirovski, A. (1994). *Interior-Point Polynomial Methods in Convex Programming*. SIAM (Society for Industrial and Applied Mathematics).

Newell, A. (1982). The knowledge level. *AIJ, 18*, 82–127.

Newell, A. (1990). *Unified Theories of Cognition*. Harvard University Press.

Newell, A. y Ernst, G. (1965). The search for generality. En *Proc. IFIP Congress*.

Newell, A., Shaw, J. C. y Simon, H. A. (1957). Empirical explorations with the logic theory machine. *Proc. Western Joint Computer Conference, 15*, 218–239. Reprinted in Feigenbaum and Feldman (1963).

Newell, A. y Simon, H. A. (1961). GPS, a program that simulates human thought. In Billing, H. (Ed.), *Lernende Automaten*. R. Oldenbourg.

Newell, A. y Simon, H. A. (1972). *Human Problem Solving*. Prentice-Hall.

Newell, A. y Simon, H. A. (1976). Computer science as empirical inquiry: Symbols and search. *CACM, 19*, 113–126.

Newton, I. (1664–1671). Methodus fluxionum et serierum infinitarum. Unpublished notes.

Ng, A. Y. (2004). Feature selection, *L*1 vs. *L*2 regularization, and rotational invariance. En *ICML-04*.

Ng, A. Y. (2019). *Machine Learning Yearning*. www.mlyearning.org.

Ng, A. Y., Harada, D. y Russell, S. J. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. En *ICML-99*.

Ng, A. Y. y Jordan, M. I. (2000). PEGASUS: A policy search method for large MDPs and POMDPs. En *UAI-00*.

Ng, A. Y. y Jordan, M. I. (2002). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. En *NeurIPS 14*.

Ng, A. Y., Kim, H. J., Jordan, M. I. y Sastry, S. (2003). Autonomous helicopter flight via reinforcement learning. En *NeurIPS 16*.

Ng, A. Y. y Russell, S. J. (2000). Algorithms for inverse reinforcement learning. En *ICML-00*.

Nicholson, A. y Brady, J. M. (1992). The data association problem when monitoring robot vehicles using dynamic belief networks. En *ECAI-92*.

Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.

Nielsen, T. y Jensen, F. (2003). Sensitivity analysis in influence diagrams. *IEEE Transactions on Systems, Man and Cybernetics, 33*, 223–234.

Niemelä, I., Simons, P. y Syrjänen, T. (2000). Smodels: A system for answer set programming. En *Proc. 8th International Workshop on Non-Monotonic Reasoning*.

Nikolaidis, S. y Shah, J. (2013). Human-robot cross-training: computational formulation, modeling and evaluation of a human team training strategy. En *HRI-13*.

Niles, I. y Pease, A. (2001). Towards a standard upper ontology. En *Proc. International Conference on Formal Ontology in Information Systems*.

Nilsson, D. y Lauritzen, S. (2000). Evaluating influence diagrams using LIMIDs. En *UAI-00*.

Nilsson, N. J. (1965). *Learning Machines: Foundations of Trainable Pattern-Classifying Systems*. McGraw-Hill.

Nilsson, N. J. (1971). *Problem-Solving Methods in Artificial Intelligence*. McGraw-Hill.

Nilsson, N. J. (1984). Shakey the robot. Technical note, SRI International.

Nilsson, N. J. (1986). Probabilistic logic. *AIJ, 28*, 71–87.

Nilsson, N. J. (1995). Eye on the prize. *AIMag, 16*, 9–17.

Nilsson, N. J. (2009). *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press.

Nisan, N. (2007). Introduction to mechanism design (for computer scientists). En Nisan, N., Roughgarden, T., Tardos, E. y Vazirani, V. V. (Eds.), *Algorithmic Game Theory*. Cambridge University Press.

Nisan, N., Roughgarden, T., Tardos, E. y Vazirani, V. (Eds.). (2007). *Algorithmic Game Theory*. Cambridge University Press.

Niv, Y. (2009). Reinforcement learning in the brain. *Journal of Mathematical Psychology, 53*, 139–154.

Nivre, J., De Marneffe, M.-C., Ginter, F., Goldberg, Y., Hajic, J., Manning, C., McDonald, R., Petrov, S. *et al.* (2016). Universal dependencies v1: A multilingual treebank collection. En *Proc. International Conference on Language Resources and Evaluation*.

Nodelman, U., Shelton, C. y Koller, D. (2002). Continuous time Bayesian networks. En *UAI-02*.

Noe, A. (2009). *Out of Our Heads: Why You Are Not Your Brain, and Other Lessons from the Biology of Consciousness*. Hill and Wang.

Nordfors, D., Cerf, V. y Senges, M. (2018). *Disrupting Unemployment*. Amazon Digital Services.

Norvig, P. (1988). Multiple simultaneous interpretations of ambiguous sentences. En *COGSCI-88*.

Norvig, P. (1992). *Paradigms of Artificial Intelligence Programming: Case Studies in Common Lisp*. Morgan Kaufmann.

Norvig, P. (2009). Natural language corpus data. En Segaran, T. y Hammerbacher, J. (Eds.), *Beautiful Data*. O'Reilly.

Nowick, S.M., Dean, M. E., Dill, D. L. y Horowitz, M. (1993). The design of a high-performance cache controller: A case study in asynchronous synthesis. *Integration: The VLSI Journal, 15*, 241–262.

Och, F. J. y Ney, H. (2003). A systematic comparison of various statistical alignment models. *Computational Linguistics, 29*, 19–51.

Och, F. J. y Ney, H. (2004). The alignment template approach to statistical machine translation. *Computational Linguistics, 30*, 417–449.

Och, F. J. y Ney, H. (2002). Discriminative training and maximum entropy models for statistical machine translation. En *COLING-02*.

Ogawa, S., Lee, T.-M., Kay, A. R. y Tank, D. W. (1990). Brain magnetic resonance imaging with contrast dependent on blood oxygenation. *PNAS, 87*, 9868–9872.

Oh, M.-S. y Berger, J. O. (1992). Adaptive importance sampling in Monte Carlo integration. *Journal of Statistical Computation and Simulation, 41*, 143–168.

Oh, S., Russell, S. J. y Sastry, S. (2009). Markov chain Monte Carlo data association for multi-target tracking. *IEEE Transactions on Automatic Control, 54*, 481–497.

Oizumi, M., Albantakis, L. y Tononi, G. (2014). From the phenomenology to the mechanisms of consciousness: Integrated information theory 3.0. *PLoS Computational Biology, 10*, e1003588.

Olesen, K. G. (1993). Causal probabilistic networks with both discrete and continuous variables. *PAMI, 15*, 275–279.

Oliver, N., Garg, A. y Horvitz, E. J. (2004). Layered representations for learning and inferring office activity from multiple sensory channels. *Computer Vision and Image Understanding, 96*, 163–180.

Oliver, R.M. y Smith, J.Q. (Eds.). (1990). *Influence Diagrams, Belief Nets and Decision Analysis*. Wiley.

Omohundro, S. (2008). The basic AI drives. En *AGI-08 Workshop on the Sociocultural, Ethical and Futurological Implications of Artificial Intelligence*.

O'Neil, C. (2017). *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. Broadway Books.

O'Neil, C. y Schutt, R. (2013). *Doing Data Science: Straight Talk from the Frontline*. O'Reilly.

O'Reilly, U.-M. y Oppacher, F. (1994). Program search with a hierarchical variable length representation: Genetic programming, simulated annealing and hill climbing. En *Proc. Third Conference on Parallel Problem Solving from Nature*.

Osborne, M. J. (2004). *An Introduction to Game Theory*. Oxford University Pres.

Osborne, M. J. y Rubinstein, A. (1994). *A Course in Game Theory*. MIT Press.

Osherson, D. N., Stob, M. y Weinstein, S. (1986). *Systems That Learn: An Introduction to Learning Theory for Cognitive and Computer Scientists*. MIT Press.

Ostrom, E. (1990). *Governing the Commons*. Cambridge University Press.

Padgham, L. y Winikoff, M. (2004). *Developing Intelligent Agent Systems: A Practical Guide*. Wiley.

Paige, B. y Wood, F. (2014). A compilation target for probabilistic programming languages. En *ICML-14*.

Paige, B., Wood, F., Doucet, A. y Teh, Y. W. (2015). Asynchronous anytime sequential Monte Carlo. En *NeurIPS 27*.

Palacios, H. y Geffner, H. (2007). From conformant into classical planning: Efficient translations that may be complete too. En *ICAPS-07*.

Palmer, S. (1999). *Vision Science: Photons to Phenomenology*. MIT Press.

Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.

Papadimitriou, C. H. y Tsitsiklis, J. N. (1987). The complexity of Markov decision processes. *Mathematics of Operations Research, 12*, 441–450.

Papadimitriou, C. H. y Yannakakis, M. (1991). Shortest paths without a map. *Theoretical Computer Science, 84*, 127–150.

Papavassiliou, V. y Russell, S. J. (1999). Convergence of reinforcement learning with general function approximators. En *IJCAI-99*.

Parisi, G. (1988). *Statistical Field Theory*. Addison-Wesley.

Parisi, M. M. G. y Zecchina, R. (2002). Analytic and algorithmic solution of random satisfiability problems. *Science, 297*, 812–815.

Park, J. D. y Darwiche, A. (2004). Complexity results and approximation strategies for MAP explanations. *JAIR, 21*, 101–133.

Parker, A., Nau, D. S. y Subrahmanian, V. S. (2005). Game-tree search with combinatorially large belief states. En *IJCAI-05*.

Parker, D. B. (1985). Learning logic. Technical report, Center for Computational Research in Economics and Management Science, MIT.

Parker, L. E. (1996). On the design of behavior-based multi-robot teams. *J. Advanced Robotics, 10*, 547–578.

Parr, R. y Russell, S. J. (1998). Reinforcement learning with hierarchies of machines. En *NeurIPS 10*.

Parzen, E. (1962). On estimation of a probability density function andmode. *Annals of Mathematical Statistics, 33*, 1065–1076.

Pasca, M. y Harabagiu, S. M. (2001). High performance question/answering. En *SIGIR-01*.

Pasca, M., Lin, D., Bigham, J., Lifchits, A. y Jain, A. (2006). Organizing and searching the world wide web of facts—Step one: The one-million fact extraction challenge. En *AAAI-06*.

Paskin, M. (2002). Maximum entropy probabilistic logic. Tech. report, UC Berkeley.

Pasula, H., Marthi, B., Milch, B., Russell, S. J. y Shpitser, I. (2003). Identity uncertainty and citation matching. En *NeurIPS 15*.

Pasula, H., Russell, S. J., Ostland, M. y Ritov, Y. (1999). Tracking many objects with many sensors. En *IJCAI-99*.

Patel-Schneider, P. (2014). Analyzing schema.org. En *Proc. International Semantic Web Conference*.

Patrick, B. G., Almulla, M., y Newborn,M. (1992). An upper bound on the time complexity of iterativedeepening-A*. *AIJ, 5*, 265–278.

Paul, R. P. (1981). *Robot Manipulators: Mathematics, Programming, and Control*. MIT Press.

Pauls, A. y Klein, D. (2009). K-best A* parsing. En *ACL-09*.

Peano, G. (1889). *Arithmetices principia, nova methodo exposita*. Fratres Bocca, Turin.

Pearce, J., Tambe, M. y Maheswaran, R. (2008). Solving multiagent networks using distributed constraint optimization. *AIMag, 29*, 47–62.

Pearl, J. (1982a). Reverend Bayes on inference engines: A distributed hierarchical approach. En *AAAI-82*.

Pearl, J. (1982b). The solution for the branching factor of the alpha–beta pruning algorithm and its optimality. *CACM, 25*, 559–564.

Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.

Pearl, J. (1985). Bayesian networks: A model of self-activated memory for evidential reasoning. En *COGSCI-85*.

Pearl, J. (1986). Fusion, propagation, and structuring in belief networks. *AIJ, 29*, 241–288.

Pearl, J. (1987). Evidential reasoning using stochastic simulation of causal models. *AIJ, 32*, 247–257.

Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann.

Pearl, J. (2000). *Causality: Models, Reasoning, and Inference*. Cambridge University Press.

Pearl, J. y McKenzie, D. (2018). *The Book of Why*. Basic Books.

Pearl, J. y Verma, T. (1991). A theory of inferred causation. En *KR-91*.

Pearson, K. (1895). Contributions to the mathematical theory of evolution, II: Skew variation in homogeneous material. *Phil. Trans. Roy. Soc., 186*, 343–414.

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine, 2*, 559–572.

Pease, A. y Niles, I. (2002). IEEE standard upper ontology: A progress report. *Knowledge Engineering Review, 17*, 65–70.

Pednault, E. P. D. (1986). Formulating multiagent, dynamic-world problems in the classical planning framework. En *Reasoning About Actions and Plans: Proc. 1986 Workshop*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. *et al*. (2011). Scikit-learn: Machine learning in Python. *JMLR, 12*, 2825–2830.

Peirce, C. S. (1870). Description of a notation for the logic of relatives, resulting from an amplification of the conceptions of Boole's calculus of logic. *Memoirs of the American Academy of Arts and Sciences, 9*, 317–378.

Peirce, C. S. (1883). A theory of probable inference. Note B. The logic of relatives. En Peirce, C. S. (Ed.), *Studies in Logic*, Little, Brown.

Peirce, C. S. (1909). Existential graphs. Unpublished manuscript; reprinted in (Buchler 1955).

Peleg, B. y Sudholter, P. (2002). *Introduction to the Theory of Cooperative Games* (2a edición). Springer-Verlag.

Pelikan, M., Goldberg, D. E. y Cantu-Paz, E. (1999). BOA: The Bayesian optimization algorithm. En *GECCO-99*.

Pemberton, J. C. y Korf, R. E. (1992). Incremental planning on graphs with cycles. En *AIPS-92*.

Penberthy, J. S. y Weld, D. S. (1992). UCPOP: A sound, complete, partial order planner for ADL. En *KR-92*.

Peng, J. y Williams, R. J. (1993). Efficient learning and planning within the Dyna framework. *Adaptive Behavior, 2*, 437–454.

Pennington, J., Socher, R. y Manning, C. (2014). Glove: Global vectors for word representation. En *EMNLP-14*.

Penrose, R. (1989). *The Emperor's New Mind*. Oxford University Press.

Penrose, R. (1994). *Shadows of the Mind*. Oxford University Press.

Peot, M. y Smith, D. E. (1992). Conditional nonlinear planning. En *ICAPS-92*.

Pereira, F. y Schabes, Y. (1992). Inside-outside reestimation from partially bracketed corpora. En *ACL-92*.

Pereira, F. y Warren, D. H. D. (1980). Definite clause grammars for language analysis: A survey of the formalism and a comparison with augmented transition networks. *AIJ, 13*, 231–278.

Peters, J. y Schaal, S. (2008). Reinforcement learning of motor skills with policy gradients. *Neural Networks, 21*, 682–697.

Peters, J., Janzing, D. y Schölkopf, B. (2017). *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT press.

Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K. y Zettlemoyer, L. (2018). Deep contextualized word representations. arXiv:1802.05365.

Peterson, C. y Anderson, J. R. (1987). A mean field theory learning algorithm for neural networks. *Complex Systems, 1*, 995–1019.

Petosa, N. y Balch, T. (2019). Multiplayer AlphaZero. arXiv:1910.13012.

Pfeffer, A. (2001). IBAL: A probabilistic rational programming language. En *IJCAI-01*.

Pfeffer, A., Koller, D., Milch, B. y Takusagawa, K. T. (1999). SPOOK: A system for probabilistic object-oriented knowledge representation. En *UAI-99*.

Pfeffer, A. (2016). *Practical Probabilistic Programming*. Manning.

Pfeffer, A. (2000). *Probabilistic Reasoning for Complex Systems*. Ph.D. thesis, Stanford University.

Pfeffer, A. (2007). The design and implementation of IBAL: A general-purpose probabilistic language. En Getoor, L. y Taskar, B. (Eds.), *Introduction to Statistical Relational Learning*. MIT Press.

Pfeifer, R., Bongard, J., Brooks, R. A. y Iwasawa, S. (2006). How the Body Shapes the Way We Think: A New View of Intelligence. Bradford.

Pham, H., Guan, M. Y., Zoph, B., Le, Q. V. y Dean, J. (2018). Efficient neural architecture search via parameter sharing. arXiv:1802.03268.

Pineau, J., Gordon, G. y Thrun, S. (2003). Pointbased value iteration: An anytime algorithm for POMDPs. En *IJCAI-03*.

Pinedo, M. (2008). *Scheduling: Theory, Algorithms, and Systems*. Springer Verlag.

Pinkas, G. y Dechter, R. (1995). Improving connectionist energy minimization. *JAIR, 3*, 223–248.

Pinker, S. (1995). Language acquisition. En Gleitman, L. R., Liberman, M. y Osherson, D. N. (Eds.), *An Invitation to Cognitive Science* (2a edición). MIT Press.

Pinker, S. (2003). *The Blank Slate: The Modern Denial of Human Nature*. Penguin.

Pinto, D., McCallum, A., Wei, X. y Croft, W. B. (2003). Table extraction using conditional random fields. En *SIGIR-03*.

Pinto, L. y Gupta, A. (2016). Supersizing selfsupervision: Learning to grasp from 50k tries and 700 robot hours. En *ICRA-16*.

Platt, J. (1999). Fast training of support vector machines using sequential minimal optimization. En *Advances in Kernel Methods: Support Vector Learning*. MIT Press.

Plotkin, G. (1972). Building-in equational theories. En Meltzer, B. y Michie, D. (Eds.), *Machine Intelligence 7*. Edinburgh University Press.

Plummer, M. (2003). JAGS: A program for analysis of Bayesian graphical models using Gibbs sampling. En *Proc. Third Int'l Workshop on Distributed Statistical Computing*.

Pnueli, A. (1977). The temporal logic of programs. En *FOCS-77*.

Pohl, I. (1971). Bi-directional search. En Meltzer, B. y Michie, D. (Eds.), *Machine Intelligence 6*. Edinburgh University Press.

Pohl, I. (1973). The avoidance of (relative) catastrophe, heuristic competence, genuine dynamic weighting and computational issues in heuristic problem solving. En *IJCAI-73*.

Pohl, I. (1977). Practical and theoretical considerations in heuristic search algorithms. En Elcock, E. W. y Michie, D. (Eds.), *Machine Intelligence 8*. Ellis Horwood.

Pohl, I. (1970). Heuristic search viewed as path finding in a graph. *AIJ, 1*, 193–204.

Poli, R., Langdon, W. y McPhee, N. (2008). *A Field Guide to Genetic Programming*. Lulu.com.

Pomerleau, D. A. (1993). *Neural Network Perception for Mobile Robot Guidance*. Kluwer.

Poole, B., Lahiri, S., Raghu, M., Sohl-Dickstein, J. y Ganguli, S. (2017). Exponential expressivity in deep neural networks through transient chaos. En *NeurIPS 29*.

Poole, D. (1993). Probabilistic Horn abduction and Bayesian networks. *AIJ, 64*, 81–129.

Poole, D. (2003). First-order probabilistic inference. En *IJCAI-03*.

Poole, D. y Mackworth, A. K. (2017). *Artificial Intelligence: Foundations of Computational Agents* (2 edición). Cambridge University Press.

Poppe, R. (2010). A survey on vision-based human action recognition. *Image Vision Comput., 28*, 976–990.

Popper, K. R. (1959). *The Logic of Scientific Discovery*. Basic Books.

Popper, K. R. (1962). *Conjectures and Refutations: The Growth of Scientific Knowledge*. Basic Books.

Portner, P. y Partee, B. H. (2002). *Formal Semantics: The Essential Readings*. Wiley-Blackwell.

Post, E. L. (1921). Introduction to a general theory of elementary propositions. *American Journal of Mathematics, 43*, 163–185.

Poulton, C. y Watts, M. (2016). MIT and DARPA pack Lidar sensor onto single chip. *IEEE Spectrum*, August 4.

Poundstone, W. (1993). *Prisoner's Dilemma*. Anchor.

Pourret, O., Naïm, P. y Marcot, B. (2008). *Bayesian Networks: A Practical Guide to Applications*. Wiley.

Pradhan, M., Provan, G. M., Middleton, B. y Henrion, M. (1994). Knowledge engineering for large belief networks. En *UAI-94*.

Prawitz, D. (1960). An improved proof procedure. *Theoria, 26*, 102–139.

Press, W. H., Teukolsky, S. A., Vetterling, W. T. y Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3a edición). Cambridge University Press.

Preston, J. y Bishop, M. (2002). *Views into the Chinese Room: New Essays on Searle and Artificial Intelligence*. Oxford University Press.

Prieditis, A. E. (1993). Machine discovery of effective admissible heuristics. *Machine Learning, 12*, 117–141.

Prosser, P. (1993). Hybrid algorithms for constraint satisfaction problems. *Computational Intelligence, 9*, 268–299.

Pullum, G. K. (1991). *The Great Eskimo Vocabulary Hoax (and Other Irreverent Essays on the Study of Language)*. University of Chicago Press.

Pullum, G. K. (1996). Learnability, hyperlearning, and the poverty of the stimulus. En *22nd Annual Meeting of the Berkeley Linguistics Society*.

Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.

Puterman, M. L. y Shin, M. C. (1978). Modified policy iteration algorithms for discounted Markov decision problems. *Management Science, 24*, 1127–1137.

Putnam, H. (1963). 'Degree of confirmation' and inductive logic. En Schilpp, P. A. (Ed.), *The Philosophy of Rudolf Carnap. Open Court*.

Quillian, M. R. (1961). A design for an understanding machine. Paper presented at a colloquium: Semantic Problems in Natural Language, King's College, Cambridge, England.

Quine, W. V. (1953). *Two dogmas of empiricism. En From a Logical Point of View*. Harper and Row.

Quine, W. V. (1960). *Word and Object*. MIT Press.

Quine, W. V. (1982). *Methods of Logic* (4a edición). Harvard University Press.

Quinlan, J. R. (1979). Discovering rules from large collections of examples: A case study. En Michie, D. (Ed.), *Expert Systems in the Microelectronic Age*. Edinburgh University Press.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*, 81–106.

Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

Quinlan, S. y Khatib, O. (1993). Elastic bands: Connecting path planning and control. En *ICRA-93*.

Quirk, R., Greenbaum, S., Leech, G. y Svartvik, J. (1985). *A Comprehensive Grammar of the English Language*. Longman.

Rabani, Y., Rabinovich, Y. y Sinclair, A. (1998). A computational view of population genetics. *Random Structures and Algorithms, 12*, 313–334.

Rabiner, L. R. y Juang, B.-H. (1993). *Fundamentals of Speech Recognition*. Prentice-Hall.

Radford, A., Metz, L. y Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv:1511.06434.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. y Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog, 1*.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. y Liu, P. J. (2019). Exploring the limits of transfer learning with a unified text-to-text transformer. arXiv:1910.10683.

Rafferty, A. N., Brunskill, E., Griffiths, T. L. Shafto, P. (2016). Faster teaching via POMDP planning. *Cognitive Science, 40*, 1290–1332.

Rahwan, T.,Michalak, T. P.,Wooldridge, M. Jennings, N. R. (2015). Coalition structure generation: A survey. *AIJ, 229*, 139–174.

Raibert, M., Blankespoor, K., Nelson, G. y Playter, R. (2008). Bigdog, the rough-terrain quadruped robot. *IFAC Proceedings Volumes, 41*, 10822–10825.

Rajpurkar, P., Zhang, J., Lopyrev, K. y Liang, P. (2016). Squad: 100,000+ questions for machine comprehension of text. En *EMNLP-16*.

Ramsey, F. P. (1931). Truth and probability. En Braithwaite, R. B. (Ed.), *The Foundations of Mathematics and Other Logical Essays*. Harcourt Brace Jovanovich.

Ramsundar, B. y Zadeh, R. B. (2018). *TensorFlow for Deep Learning: From Linear Regression to Reinforcement Learning*. O'Reilly.

Rao, D. A. S. y Verweij, G. (2017). Sizing the prize: What's the real value of AI for your business and how can you capitalise? PwC.

Raphael, B. (1976). *The Thinking Computer: Mind Inside Matter*. W. H. Freeman.

Raphson, J. (1690). *Analysis aequationum universalis*. Apud Abelem Swalle, London.

Raschka, S. (2015). *PythonMachine Learning*. Packt.

Rashevsky, N. (1936). Physico-mathematical aspects of excitation and conduction in nerves. En *Cold Springs Harbor Symposia on Quantitative Biology. IV: Excitation Phenomena*.

Rashevsky, N. (1938). *Mathematical Biophysics: Physico-Mathematical Foundations of Biology*. University of Chicago Press.

Rasmussen, C. E. y Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.

Rassenti, S., Smith, V. y Bulfin, R. (1982). A combinatorial auction mechanism for airport time slot allocation. *Bell Journal of Economics, 13*, 402–417.

Ratliff, N., Bagnell, J. A. y Zinkevich, M. (2006). Maximum margin planning. En *ICML-06*.

Ratliff, N., Zucker, M., Bagnell, J. A. y Srinivasa, S. (2009). CHOMP: Gradient optimization techniques for efficient motion planning. En *ICRA-09*.

Ratnaparkhi, A. (1996). A maximum entropy model for part-of-speech tagging. En *EMNLP-96*.

Ratner, D. y Warmuth, M. (1986). Finding a shortest solution for the n × n extension of the 15-puzzle is intractable. En *AAAI-86*.

Rauch, H. E., Tung, F. y Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems. *AIAA Journal, 3*, 1445–1450.

Rayward-Smith, V., Osman, I., Reeves, C. y Smith, G. (Eds.). (1996). *Modern Heuristic Search Methods*. Wiley.

Real, E., Aggarwal, A., Huang, Y. y Le, Q. V. (2018). Regularized evolution for image classifier architecture search. arXiv:1802.01548.

Rechenberg, I. (1965). Cybernetic solution path of an experimental problem. Library translation, Royal Aircraft Establishment.

Regin, J. (1994). A filtering algorithm for constraints of difference in CSPs. En *AAAI-94*.

Reid, D. B. (1979). An algorithm for tracking multiple targets. *IEEE Trans. Automatic Control, 24*, 843–854.

Reif, J. (1979). Complexity of the mover's problem and generalizations. En *FOCS-79*.

Reiter, R. (1980). A logic for default reasoning. *AIJ, 13*, 81–132.

Reiter, R. (1991). The frame problem in the situation calculus: A simple solution (sometimes) and a completeness result for goal regression. En Lifschitz, V. (Ed.), *Artificial Intelligence and Mathematical Theory of Computation: Papers in Honor of John McCarthy*. Academic Press.

Reiter, R. (2001). *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT Press.

Renner, G. y Ekart, A. (2003). Genetic algorithms in computer aided design. *Computer Aided Design, 35*, 709–726.

Rényi, A. (1970). *Probability Theory*. Elsevier.

Resnick, P. y Varian, H. R. (1997). Recommender systems. *CACM, 40*, 56–58.

Rezende, D. J., Mohamed, S. y Wierstra, D. (2014). Stochastic backpropagation and approximate inference in deep generative models. En *ICML-14*.

Riazanov, A. y Voronkov, A. (2002). The design and implementation of VAMPIRE. *AI Communications, 15*, 91–110.

Ribeiro, M. T., Singh, S. y Guestrin, C. (2016). Why should I trust you?: Explaining the predictions of any classifier. En *KDD-16*.

Richardson, M. y Domingos, P. (2006). Markov logic networks. *Machine Learning, 62*, 107–136.

Richter, S. y Helmert, M. (2009). Preferred operators and deferred evaluation in satisficing planning. En *ICAPS-09*.

Ridley, M. (2004). *Evolution*. Oxford Reader.

Riley, J. y Samuelson,W. (1981). Optimal auctions. *American Economic Review, 71*, 381–392.

Riley, P. (2019). Three pitfalls to avoid in machine learning. *Nature, 572*, 27–29.

Riloff, E. (1993). Automatically constructing a dictionary for information extraction tasks. En *AAAI-93*.

Ringgaard, M., Gupta, R. y Pereira, F. (2017). SLING: A framework for frame semantic parsing. arXiv:1710.07032.

Rintanen, J. (1999). Improvements to the evaluation of quantified Boolean formulae. En *IJCAI-99*.

Rintanen, J. (2007). Asymptotically optimal encodings of conformant planning in QBF. En *AAAI-07*.

Rintanen, J. (2012). Planning as satisfiability: Heuristics. *AIJ, 193*, 45–86.

Rintanen, J. (2016). Computational complexity in automated planning and scheduling. En *ICAPS-16*.

Ripley, B. D. (1996). *Pattern Recognition and Neural Networks*. Cambridge University Press.

Rissanen, J. (1984). Universal coding, information, prediction, and estimation. *IEEE Transactions on Information Theory, IT-30*, 629–636.

Rissanen, J. (2007). *Information and Complexity in Statistical Modeling*. Springer.

Rivest, R. (1987). Learning decision lists. *Machine Learning, 2*, 229–246.

Robbins, H. (1952). Some aspects of the sequential design of experiments. *Bulletin of the AmericanMathematical Society, 58*, 527–535.

Robbins, H. y Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics, 22*, 400–407.

Roberts, L. G. (1963). Machine perception of threedimensional solids. Technical report, MIT Lincoln Laboratory.

Robertson, N. y Seymour, P. D. (1986). Graph minors. II. Algorithmic aspects of tree-width. *J. Algorithms, 7*, 309–322.

Robertson, S. E. y Sparck Jones, K. (1976). Relevance weighting of search terms. *J. American Society for Information Science, 27*, 129–146.

Robins, J. (1986). A new approach to causal inference in mortality studies with a sustained exposure period: Application to control of the healthy worker survivor effect. *Mathematical Modelling, 7*, 1393–1512.

Robinson, A. y Voronkov, A. (Eds.). (2001). *Handbook of Automated Reasoning*. Elsevier.

Robinson, J. A. (1965). A machine-oriented logic based on the resolution principle. *JACM, 12*, 23–41.

Robinson, S. (2002). Computer scientists find unexpected depths in airfare search problem. *SIAM News, 35*(6).

Roche, E. y Schabes, Y. (Eds.). (1997). *Finite-State Language Processing*. Bradford Books.

Rock, I. (1984). *Perception*. W. H. Freeman.

Rokicki, T., Kociemba, H., Davidson, M. y Dethridge, J. (2014). The diameter of the Rubik's Cube group is twenty. *SIAM Review, 56*, 645–670.

Rolf, D. (2006). Improved bound for the PPSZ/Sch¨oningalgorithm for 3-SAT. *Journal on Satisfiability, Boolean Modeling and Computation, 1*, 111–122.

Rolnick, D., Donti, P. L., Kaack, L. H. *et al*. (2019). Tackling climate change with machine learning. arXiv:1906.05433.

Rolnick, D. y Tegmark, M. (2018). The power of deeper networks for expressing natural functions. En *ICLR-18*.

Romanovskii, I. (1962). Reduction of a game with complete memory to a matrix game. *Soviet Mathematics, 3*, 678–681.

Ros, G., Sellart, L., Materzynska, J., Vazquez, D. y Lopez, A. M. (2016). The SYNTHIA dataset: A large collection of synthetic images for semantic segmentation of urban scenes. En *CVPR-16*.

Rosenblatt, F. (1957). The perceptron: A perceiving and recognizing automaton. Report, Project PARA, Cornell Aeronautical Laboratory.

Rosenblatt, F. (1960). On the convergence of reinforcement procedures in simple perceptrons. Report, Cornell Aeronautical Laboratory.

Rosenblatt, F. (1962). *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan.

Rosenblatt, M. (1956). Remarks on some nonparametric estimates of a density function. *Annals of Mathematical Statistics, 27*, 832–837.

Rosenblueth, A., Wiener, N. y Bigelow, J. (1943). Behavior, purpose, and teleology. *Philosophy of Science, 10*, 18–24.

Rosenschein, J. S. y Zlotkin, G. (1994). *Rules of Encounter*. MIT Press.

Rosenschein, S. J. (1985). Formal theories of knowledge in AI and robotics. *New Generation Computing, 3*, 345–357.

Ross, G. (2012). Fisher and the millionaire: The statistician and the calculator. *Significance, 9*, 46–48.

Ross, S. (2015). *A First Course in Probability* (9a edición). Pearson.

Ross, S., Gordon, G. y Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. En *AISTATS-11*.

Rossi, F., Van Beek, P. y Walsh, T. (2006). *Handbook of Constraint Processing*. Elsevier.

Roth, D. (1996). On the hardness of approximate reasoning. *AIJ, 82*, 273–302.

Roussel, P. (1975). Prolog: Manual de référence et d'utilization. Tech. rep., Groupe d'Intelligence Artificielle, Universit´e d'Aix-Marseille.

Rowat, P. F. (1979). *Representing the Spatial Experience and Solving Spatial Problems in a Simulated Robot Environment*. Ph.D. thesis, University of British Columbia.

Roweis, S. T. y Ghahramani, Z. (1999). A unifying review of linear Gaussian models. *Neural Computation, 11*, 305–345.

Rowley, H., Baluja, S. y Kanade, T. (1998). Neural network-based face detection. *PAMI, 20*, 23–38.

Roy, N., Gordon, G. y Thrun, S. (2005). Finding approximate POMDP solutions through belief compression. *JAIR, 23*, 1–40.

Rubin, D. (1974). Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology, 66*, 688–701.

Rubin, D. (1988). Using the SIR algorithm to simulate posterior distributions. En Bernardo, J. M., de Groot, M. H., Lindley, D. V. y Smith, A. F. M. (Eds.), Bayesian Statistics 3. Oxford University Press.

Rubinstein, A. (1982). Perfect equilibrium in a bargaining model. *Econometrica, 50*, 97–109.

Rubinstein, A. (2003). Economics and psychology? The case of hyperbolic discounting. *International Economic Review, 44*, 1207–1216.

Ruder, S. (2018). NLP's ImageNet moment has arrived. *The Gradient*, July 8.

Ruder, S., Peters, M. E., Swayamdipta, S. y Wolf, T. (2019). Transfer learning in natural language processing. En *COLING-19*.

Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*, 533–536.

Rumelhart, D. E. y McClelland, J. L. (Eds.). (1986). *Parallel Distributed Processing*. MIT Press.

Rummery, G. A. y Niranjan, M. (1994). On-line Qlearning using connectionist systems. Tech. rep., Cambridge University Engineering Department.

Ruspini, E. H., Lowrance, J. D., y Strat, T. M. (1992). Understanding evidential reasoning. *IJAR, 6*, 401–424.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C. y Fei-Fei, L. (2015). ImageNet large scale visual recognition challenge. *IJCV, 115*, 211–252.

Russell, J. G. B. (1990). Is screening for abdominal aortic aneurysm worthwhile? *Clinical Radiology, 41*, 182–184.

Russell, S. J. (1985). The compleat guide to MRS. Report, Computer Science Department, Stanford University.

Russell, S. J. (1992). Efficient memory-bounded search methods. En *ECAI-92*.

Russell, S. J. (1998). Learning agents for uncertain environments. En *COLT-98*.

Russell, S. J. (1999). Expressive probability models in science. En *Proc. Second International Conference on Discovery Science*.

Russell, S. J. (2019). *Human Compatible*. Penguin.

Russell, S. J., Binder, J., Koller, D. y Kanazawa, K. (1995). Local learning in probabilistic networks with hidden variables. En *IJCAI-95*.

Russell, S. J. y Norvig, P. (2003). *Artificial Intelligence: A Modern Approach* (2a edición). Prentice-Hall.

Russell, S. J. y Subramanian, D. (1995). Provably bounded-optimal agents. *JAIR, 3*, 575–609.

Russell, S. J. y Wefald, E. H. (1989). On optimal game-tree search using rational meta-reasoning. En *IJCAI-89*.

Russell, S. J. y Wefald, E. H. (1991). *Do the Right Thing: Studies in Limited Rationality*. MIT Press.

Russell, S. J. y Wolfe, J. (2005). Efficient belief-state AND-OR search, with applications to Kriegspiel. En *IJCAI-05*.

Russell, S. J. y Zimdars, A. (2003). Q-decomposition of reinforcement learning agents. En *ICML-03*.

Rustagi, J. S. (1976). *Variational Methods in Statistics*. Academic Press.

Saad, F. y Mansinghka, V. K. (2017). A probabilistic programming approach to probabilistic data analysis. En *NeurIPS 29*.

Sabin, D. y Freuder, E. C. (1994). Contradicting conventional wisdom in constraint satisfaction. En *ECAI-94*.

Sabri, K. E. (2015). Automated verification of role-based access control policies constraints using Prover9. arXiv:1503.07645.

Sacerdoti, E. D. (1974). Planning in a hierarchy of abstraction spaces. *AIJ, 5*, 115–135.

Sacerdoti, E.D. (1975). The nonlinear nature of plans. En *IJCAI-75*.

Sacerdoti, E. D. (1977). *A Structure for Plans and Behavior*. Elsevier.

Sadeghi, F. y Levine, S. (2016). CAD2RL: Real single-image flight without a single real image. arXiv:1611.04201.

Sadigh, D., Sastry, S., Seshia, S. A. y Dragan, A. D. (2016). Planning for autonomous cars that leverage effects on human actions. En *Proc. Robotics: Science and Systems*.

Sadler, M. y Regan, N. (2019). *Game Changer*. New in Chess.

Sadri, F. y Kowalski, R. (1995). Variants of the event calculus. En *ICLP-95*.

Sagae, K. y Lavie, A. (2006). A best-first probabilistic shift-reduce parser. En *COLING-06*.

Sahami, M., Hearst, M. A. y Saund, E. (1996). Applying the multiple cause mixture model to text categorization. En *ICML-96*.

Sahin, N. T., Pinker, S., Cash, S. S., Schomer, D. y Halgren, E. (2009). Sequential processing of lexical, grammatical, and phonological information within Broca's area. *Science, 326*, 445–449.

Sakuta, M. y Iida, H. (2002). AND/OR-tree search for solving problems with uncertainty: A case study using screen-shogi problems. *Trans. Inf. Proc. Society of Japan, 43*, 1–10.

Salomaa, A. (1969). Probabilistic and weighted grammars. *Information and Control, 15*, 529–544.

Samadi, M., Felner, A. y Schaeffer, J. (2008). Learning from multiple heuristics. En *AAAI-08*.

Samet, H. (2006). *Foundations of Multidimensional and Metric Data Structures*. Morgan Kaufmann.

Sammut, C., Hurst, S., Kedzier, D. y Michie, D. (1992). Learning to fly. En *ICML-92*.

Samuel, A. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development, 3*, 210–229.

Samuel, A. (1967). Some studies in machine learning using the game of checkers II—Recent progress. *IBM Journal of Research and Development, 11*, 601–617.

Sanchez-Lengeling, B., Wei, J. N., Lee, B. K., Gerkin, R. C., Aspuru-Guzik, A. y Wiltschko, A. B. (2019). Machine learning for scent: Learning generalizable perceptual representations of small molecules. arXiv:1910.10685.

Sandholm, T. (1999). Distributed rational decision making. En Weiß, G. (Ed.), *Multiagent Systems*. MIT Press.

Sandholm, T., Larson, K., Andersson, M., Shehory, O. y Tohmé, F. (1999). Coalition structure generation with worst case guarantees. *AIJ, 111*, 209–238.

Sandholm, T. (1993). An implementation of the contract net protocol based on marginal cost calculations. En *AAAI-93*.

Sang, T., Beame, P. y Kautz, H. (2005). Performing Bayesian inference by weighted model counting. En *AAAI-05*.

Sapir, E. (1921). *Language: An Introduction to the Study of Speech*. Harcourt Brace Jovanovich.

Sarawagi, S. (2007). Information extraction. *Foundations and Trends in Databases, 1*, 261–377.

Sargent, T. J. (1978). Estimation of dynamic labor demand schedules under rational expectations. *J. Political Economy, 86*, 1009–1044.

Sartre, J.-P. (1960). *Critique de la Raison dialectique*. Editions Gallimard.

Satia, J. K. y Lave, R. E. (1973). Markovian decision processes with probabilistic observation of states. *Management Science, 20*, 1–13.

Sato, T. y Kameya, Y. (1997). PRISM: A symbolicstatistical modeling language. En *IJCAI-97*.

Saul, L. K., Jaakkola, T. y Jordan, M. I. (1996). Mean field theory for sigmoid belief networks. *JAIR, 4*, 61–76.

Saunders, W., Sastry, G., Stuhlmüuller, A. y Evans, O. (2018). Trial without error: Towards safe reinforcement learning via human intervention. En *AAMAS-18*.

Savage, L. J. (1954). *The Foundations of Statistics*. Wiley.

Savva, M., Kadian, A., Maksymets, O., Zhao, Y.,Wijmans, E., Jain, B., Straub, J., Liu, J., Koltun, V., Malik, J., Parikh, D. y Batra, D. (2019). Habitat: A platform for embodied AI research. arXiv:1904.01201.

Sayre, K. (1993). Three more flaws in the computational model. Paper presented at the APA (Central Division) Annual Conference, Chicago, Illinois.

Schaeffer, J. (2008). *One Jump Ahead: Computer Perfection at Checkers*. Springer-Verlag.

Schaeffer, J., Burch, N., Bjornsson, Y., Kishimoto, A., Müller, M., Lake, R., Lu, P. y Sutphen, S. (2007). Checkers is solved. *Science, 317*, 1518–1522.

Schank, R. C. y Abelson, R. P. (1977). *Scripts, Plans, Goals, and Understanding*. Lawrence Erlbaum.

Schank, R. C. y Riesbeck, C. (1981). *Inside Computer Understanding: Five Programs Plus Miniatures*. Lawrence Erlbaum.

Schapire, R. E. y Singer, Y. (2000). Boostexter: A boosting-based system for text categorization. *Machine Learning, 39*, 135–168.

Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning, 5*, 197–227.

Schapire, R. E. (2003). The boosting approach to machine learning: An overview. En Denison, D. D., Hansen, M. H., Holmes, C., Mallick, B. y Yu, B. (Eds.), *Nonlinear Estimation and Classification*. Springer.

Scharre, P. (2018). *Army of None*. W. W. Norton.

Schmid, C. y Mohr, R. (1996). Combining greyvalue invariants with local constraints for object recognition. En *CVPR-96*.

Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks, 61*, 85–117.

Schofield, M. y Thielscher, M. (2015). Lifting model sampling for general game playing to incomplete-information models. En *AAAI-15*.

Schölkopf, B. y Smola, A. J. (2002). Learning with Kernels. MIT Press.

Schöning, T. (1999). A probabilistic algorithm for k-SAT and constraint satisfaction problems. En *FOCS-99*.

Schoppers, M. J. (1989). In defense of reaction plans as caches. *AIMag, 10*, 51–60.

Schraudolph, N. N., Dayan, P. y Sejnowski, T. (1994). Temporal difference learning of position evaluation in the game of Go. En *NeurIPS 6*.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T. y Silver, D. (2019). Mastering Atari, Go, chess and shogi by planning with a learned model. arXiv:1911.08265.

Schröder, E. (1877). *Der Operationskreis des Logikkalküls*. B. G. Teubner, Leipzig.

Schulman, J., Ho, J., Lee, A. X., Awwal, I., Bradlow, H. y Abbeel, P. (2013). Finding locally optimal, collision-free trajectories with sequential convex optimization. En *Proc. Robotics: Science and Systems*.

Schulman, J., Levine, S., Abbeel, P., Jordan, M. I. y Moritz, P. (2015a). Trust region policy optimization. En *ICML-15*.

Schulman, J., Levine, S., Moritz, P., Jordan, M. y Abbeel, P. (2015b). Trust region policy optimization. En *ICML-15*.

Schultz, W., Dayan, P. y Montague, P. R. (1997). A neural substrate of prediction and reward. *Science, 275*, 1593.

Schulz, D., Burgard, W., Fox, D. y Cremers, A. B. (2003). People tracking with mobile robots using sample-based joint probabilistic data association filters. *Int. J. Robotics Research, 22*, 99–116.

Schulz, S. (2004). System Description: E 0.81. En *Proc. International Joint Conference on Automated Reasoning*, Vol. 3097 of LNAI.

Schulz, S. (2013). System description: E 1.8. En *Proc. Int. Conf. on Logic for Programming Artificial Intelligence and Reasoning*.

Schütze, H. (1995). *Ambiguity in Language Learning: Computational and Cognitive Models*. Ph.D. thesis, Stanford University. Also published by CSLI Press, 1997.

Schwartz, J. T., Scharir, M. y Hopcroft, J. (1987). *Planning, Geometry and Complexity of Robot Motion*. Ablex.

Schwartz, S. P. (Ed.). (1977). *Naming, Necessity, and Natural Kinds*. Cornell University Press.

Scott, D. y Krauss, P. (1966). Assigning probabilities to logical formulas. En Hintikka, J. y Suppes, P. (Eds.), *Aspects of Inductive Logic*. North-Holland.

Searle, J. R. (1980). Minds, brains, and programs. *BBS, 3*, 417–457.

Searle, J. R. (1990). Is the brain's mind a computer program? *Scientific American, 262*, 26–31.

Searle, J. R. (1992). *The Rediscovery of the Mind*. MIT Press.

Sedgewick, R. y Wayne, K. (2011). *Algorithms*. Addison-Wesley.

Sefidgar, Y. S., Agarwal, P. y Cakmak, M. (2017). Situated tangible robot programming. En *HRI-17*.

Segaran, T. (2007). *Programming Collective Intelligence: Building SmartWeb 2.0 Applications*. O'Reilly.

Seipp, J. y Röger, G. (2018). Fast downward stone soup 2018. IPC 2018 Classical Track.

Seipp, J., Sievers, S., Helmert, M. y Hutter, F. (2015). Automatic configuration of sequential planning portfolios. En *AAAI-15*.

Selman, B., Kautz, H. y Cohen, B. (1996). Local search strategies for satisfiability testing. En Johnson, D. S. y Trick, M. A. (Eds.), *Cliques, Coloring, and Satisfiability*. American Mathematical Society.

Selman, B. y Levesque, H. J. (1993). The complexity of path-based defeasible inheritance. *AIJ, 62*, 303–339.

Selman, B., Levesque, H. J. y Mitchell, D. (1992). A new method for solving hard satisfiability problems. En *AAAI-92*.

Seni, G. y Elder, J. F. (2010). Ensemble methods in data mining: Improving accuracy through combining predictions. *Synthesis Lectures on Data Mining and Knowledge Discovery, 2*, 1–126.

Seo, M., Kembhavi, A., Farhadi, A. y Hajishirzi, H. (2017). Bidirectional attention flow for machine comprehension. En *ICLR-17*.

Shachter, R. D. (1986). Evaluating influence diagrams. *Operations Research, 34*, 871–882.

Shachter, R. D. (1998). Bayes-ball: The rational pastime (for determining irrelevance and requisite information in belief networks and influence diagrams). En *UAI-98*.

Shachter, R. D., D'Ambrosio, B. y Del Favero, B. A. (1990). Symbolic probabilistic inference in belief networks. En *AAAI-90*.

Shachter, R. D. y Kenley, C. R. (1989). Gaussian influence diagrams. *Management Science, 35*, 527–550.

Shachter, R. D. y Peot, M. (1989). Simulation approaches to general probabilistic inference on belief networks. En *UAI-98*.

Shafer, G. (1976). *A Mathematical Theory of Evidence*. Princeton University Press.

Shanahan, M. (1997). *Solving the Frame Problem*. MIT Press.

Shanahan, M. (1999). The event calculus explained. En Wooldridge, M. J. y Veloso, M. (Eds.), *Artificial Intelligence Today*. Springer-Verlag.

Shanahan, M. (2015). *The Technological Singularity*. MIT Press.

Shani, G., Pineau, J. y Kaplow, R. (2013). A survey of point-based POMDP solvers. *Autonomous Agents and Multi-Agent Systems, 27*, 1–51.

Shankar, N. (1986). *Proof-Checking Metamathematics*. Ph.D. thesis, Computer Science Department, University of Texas at Austin.

Shannon, C. E. y Weaver, W. (1949). *The Mathematical Theory of Communication*. University of Illinois Press.

Shannon, C. E. (1950). Programming a computer for playing chess. *Philosophical Magazine, 41*, 256–275.

Shapley, L. S. (1953a). A value for n-person games. En Kuhn, H. W. y Tucker, A. W. (Eds.), *Contributions to the Theory of Games*. Princeton University Press.

Shapley, S. (1953b). Stochastic games. *PNAS, 39*, 1095–1100.

Sharan, R. V. y Moir, T. J. (2016). An overview of applications and advancements in automatic sound recognition. *Neurocomputing, 200*, 22–34.

Shatkay, H. y Kaelbling, L. P. (1997). Learning topological maps with weak local odometric information. En *IJCAI-97*.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q. V., Hinton, G. E. y Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv:1701.06538.

Shelley, M. (1818). *Frankenstein: Or, the Modern Prometheus*. Pickering and Chatto.

Sheppard, B. (2002). World-championship-caliber scrabble. *AIJ, 134*, 241–275.

Shi, J. y Malik, J. (2000). Normalized cuts and image segmentation. *PAMI, 22*, 888–905.

Shieber, S. (1994). Lessons from a restricted Turing test. *CACM, 37*, 70–78.

Shieber, S. (Ed.). (2004). *The Turing Test*. MIT Press.

Shimony, S. E. (1994). Finding MAPs for belief networks is NP-hard. *AIJ, 68*, 399–410.

Shoham, Y. (1993). Agent-oriented programming. *AIJ, 60*, 51–92.

Shoham, Y. (1994). *Artificial Intelligence Techniques in Prolog*. Morgan Kaufmann.

Shoham, Y. y Leyton-Brown, K. (2009). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge Univ. Press.

Shoham, Y., Powers, R. y Grenager, T. (2004). If multi-agent learning is the answer, what is the question? En *Proc. AAAI Fall Symposium on Artificial Multi-Agent Learning*.

Shortliffe, E. H. (1976). *Computer-Based Medical Consultations: MYCIN*. Elsevier.

Siciliano, B. y Khatib, O. (Eds.). (2016). *Springer Handbook of Robotics* (2a edición). Springer-Verlag.

Sigaud, O. y Buffet, O. (2010). *Markov Decision Processes in Artificial Intelligence*. Wiley.

Sigmund, K. (2017). *Exact Thinking in Demented Times*. Basic Books.

Silberstein, M., Weissbrod, O., Otten, L., Tzemach, A., Anisenia, A., Shtark, O., Tuberg, D., Galfrin, E., Gannon, I., Shalata, A., Borochowitz, Z. U., Dechter, R., Thompson, E. y Geiger, D. (2013). A systemfor exact and approximate genetic linkage analysis of SNP data in large pedigrees. *Bioinformatics, 29*, 197–205.

Silva, R., Melo, F. S. y Veloso, M. (2015). Towards table tennis with a quadrotor autonomous learning robot and onboard vision. En *IROS-15*.

Silver, D. y Veness, J. (2011). Monte-Carlo planning in large POMDPs. En *NeurIPS 23*.

Silver, D., Huang, A., Maddison, C. J., Guez, A. y Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature, 529*, 484–489.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T. *et al.* (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science, 362*, 1140–1144.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., Van den Driessche, G., Graepel, T., y Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature, 550*, 354–359.

Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman and Hall.

Silverstein, C., Henzinger, M., Marais, H. y Moricz, M. (1998). Analysis of a very large AltaVista query log. Tech. rep., Digital Systems Research Center.

Simmons, R. y Koenig, S. (1995). Probabilistic robot navigation in partially observable environments. En *IJCAI-95*.

Simon, D. (2006). *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. Wiley.

Simon, H. A. (1947). *Administrative Behavior*. Macmillan.

Simon, H. A. (1963). Experiments with a heuristic compiler. *JACM, 10*, 493–506.

Simon, H. A. y Newell, A. (1958). Heuristic problem solving: The next advance in operations research. *Operations Research, 6*, 1–10.

Simon, J. C. y Dubois, O. (1989). Number of solutions to satisfiability instances—Applications to knowledge bases. *AIJ, 3*, 53–65.

Simonis, H. (2005). Sudoku as a constraint problem. En *CP-05 Workshop on Modeling and Reformulating Constraint Satisfaction Problems*.

Singer, P.W. (2009). *Wired for War*. Penguin Press.

Singh, P., Lin, T., Mueller, E. T., Lim, G., Perkins, T. y Zhu, W. L. (2002). Open mind common sense: Knowledge acquisition from the general public. En *Proc. First International Conference on Ontologies, Databases, and Applications of Semantics for Large Scale Information Systems*.

Sisbot, E. A., Marin-Urias, L. F., Alami, R. y Simeon, T. (2007). A human aware mobile robot motion planner. *IEEE Transactions on Robotics, 23*, 874–883.

Siskind, J. M. y Pearlmutter, B. A. (2016). Efficient implementation of a higher-order language with builtin AD. arXiv:1611.03416.

Sistla, A. P. y Godefroid, P. (2004). Symmetry and reduced symmetry in model checking. ACM Trans. *Program. Lang. Syst., 26*, 702–734.

Sittler, R. W. (1964). An optimal data association problem in surveillance theory. *IEEE Transactions on Military Electronics*, 8, 125–139.

Skolem, T. (1920). Logisch-kombinatorische Untersuchungen über die Erfüllbarkeit oder Beweisbarkeit mathematischer Sätze nebst einem Theoreme über die dichte Mengen. Videnskapsselskapets skrifter, I. *Matematisk-naturvidenskabelig klasse, 4*, 1–36.

Skolem, T. (1928). Über die mathematische Logik. *Norsk matematisk tidsskrift, 10*, 125–142.

Slagle, J. R. (1963). A heuristic program that solves symbolic integration problems in freshman calculus. *JACM, 10*.

Slate, D. J. y Atkin, L. R. (1977). CHESS 4.5—Northwestern University chess program. En Frey, P.W. (Ed.), *Chess Skill in Man and Machine*. Springer-Verlag.

Slater, E. (1950). Statistics for the chess computer and the factor of mobility. En *Symposium on Information Theory*. Ministry of Supply.

Slocum, J. y Sonneveld, D. (2006). *The 15 Puzzle*. Slocum Puzzle Foundation.

Smallwood, R. D. y Sondik, E. J. (1973). The optimal control of partially observable Markov processes over a finite horizon. *Operations Research, 21*, 1071–1088.

Smith, B. (2004). Ontology. En Floridi, L. (Ed.), *The Blackwell Guide to the Philosophy of Computing and Information*. Wiley-Blackwell.

Smith, B., Ashburner, M., Rosse, C. *et al.* (2007). The OBO Foundry: Coordinated evolution of ontologies to support biomedical data integration. *Nature Biotechnology, 25*, 1251–1255.

Smith, D. E., Genesereth, M. R., y Ginsberg, M. L. (1986). Controlling recursive inference. *AIJ, 30*, 343–389.

Smith, D. A. y Eisner, J. (2008). Dependency parsing by belief propagation. En *EMNLP-08*.

Smith, D. E. y Weld, D. S. (1998). Conformant Graphplan. En *AAAI-98*.

Smith, J. Q. (1988). *Decision Analysis*. Chapman and Hall.

Smith, J. E. y Winkler, R. L. (2006). The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science, 52*, 311–322.

Smith, J. M. (1982). *Evolution and the Theory of Games*. Cambridge University Press.

Smith, J.M. y Szathmáry, E. (1999). *The Origins of Life: From the Birth of Life to the Origin of Language*. Oxford University Press.

Smith, M. K., Welty, C. y McGuinness, D. (2004). OWL web ontology language guide. Tech. rep., W3C.

Smith, R. G. (1980). *A Framework for Distributed Problem Solving*. UMI Research Press.

Smith, R. C. y Cheeseman, P. (1986). On the representation and estimation of spatial uncertainty. *Int. J. Robotics Research, 5*, 56–68.

Smith, S. J. J., Nau, D. S. y Throop, T. A. (1998). Success in spades: Using AI planning techniques to win the world championship of computer bridge. En *AAAI-98*.

Smith, W. E. (1956). Various optimizers for single-stage production. *Naval Research Logistics Quarterly, 3*, 59–66.

Smolensky, P. (1988). On the proper treatment of connectionism. *BBS, 2*, 1–74.

Smolensky, P. y Prince, A. (1993). Optimality theory: Constraint interaction in generative grammar. Tech. rep., Department of Computer Science, University of Colorado at Boulder.

Smullyan, R. M. (1995*). First-Order Logic*. Dover.

Smyth, P., Heckerman, D. y Jordan, M. I. (1997). Probabilistic independence networks for hidden Markov probability models. *Neural Computation, 9*, 227–269.

Snoek, J., Larochelle, H. y Adams, R. P. (2013). Practical Bayesian optimization of machine learning algorithms. En *NeurIPS 25*.

Solomonoff, R. J. (1964). A formal theory of inductive inference. *Information and Control, 7*, 1–22, 224–254.

Solomonoff, R. J. (2009). Algorithmic probability–theory and applications. En Emmert-Streib, F. y Dehmer, M. (Eds.), *Information Theory and Statitical Learning*. Springer.

Sondik, E. J. (1971). *The Optimal Control of Partially Observable Markov Decision Processes*. Ph.D. thesis, Stanford University.

Sosic, R. y Gu, J. (1994). Efficient local search with conflict minimization: A case study of the n-queens problem. *IEEE Transactions on Knowledge and Data Engineering, 6*, 661–668.

Sowa, J. (1999). *Knowledge Representation: Logical, Philosophical, and Computational Foundations*. Blackwell.

Spaan, M. T. J. y Vlassis, N. (2005). Perseus: Randomized point-based value iteration for POMDPs. *JAIR, 24*, 195–220.

Sparrow, R. (2004). The Turing triage test. *Ethics and Information Technology, 6*, 203–213.

Spiegelhalter, D. J., Dawid, A. P., Lauritzen, S. y Cowell, R. (1993). Bayesian analysis in expert systems. *Statistical Science, 8*, 219–282.

Spirtes, P., Glymour, C. y Scheines, R. (1993). *Causation, Prediction, and Search*. Springer-Verlag.

Spitkovsky, V. I., Alshawi, H. y Jurafsky, D. (2010a). From baby steps to leapfrog: How less is more in unsupervised dependency parsing. En *NAACL HLT*.

Spitkovsky, V. I., Jurafsky, D. y Alshawi, H. (2010b). Profiting from mark-up: Hyper-text annotations for guided parsing. En *ACL-10*.

Srivas, M. y Bickford, M. (1990). Formal verification of a pipelined microprocessor. *IEEE Software, 7*, 52–64.

Srivastava, N., Hinton, G. E., Krizhevsky, A., Sutskever, I., y Salakhutdinov, R. (2014a). Dropout: A simple way to prevent neural networks from overfitting. *JMLR, 15*, 1929–1958.

Srivastava, S., Russell, S. J. y Ruan, P. (2014b). First-order open-universe POMDPs. En *UAI-14*.

Staab, S. (2004). *Handbook on Ontologies*. Springer.

Stallman, R. M. y Sussman, G. J. (1977). Forward reasoning and dependency-directed backtracking in a system for computer-aided circuit analysis. *AIJ, 9*, 135–196.

Stanfill, C. y Waltz, D. (1986). Toward memory-based reasoning. *CACM, 29*, 1213–1228.

Stanislawska, K., Krawiec, K. y Vihma, T. (2015). Genetic programming for estimation of heat flux between the atmosphere and sea ice in polar regions. En *GECCO-15*.

Stefik, M. (1995). *Introduction to Knowledge Systems*. Morgan Kaufmann.

Steiner, D. F., MacDonald, R., Liu, Y., Truszkowski, P., Hipp, J. D., Gammage, C., Thng, F., Peng, L. y Stumpe, M. C. (2018). Impact of deep learning assistance on the histopathologic review of lymph nodes for metastatic breast cancer. *Am. J. Surgical Pathology, 42*, 1636–1646.

Steinruecken, C., Smith, E., Janz, D., Lloyd, J. y Ghahramani, Z. (2019). The Automatic Statistician. En Hutter, F., Kotthoff, L. y Vanschoren, J. (Eds.), *Automated Machine Learning*. Springer.

Stergiou, K. y Walsh, T. (1999). The difference alldifference makes. En *IJCAI-99*.

Stickel, M. E. (1992). A Prolog technology theorem prover: a new exposition and implementation in Prolog. *Theoretical Computer Science, 104*, 109–128.

Stiller, L. (1992). KQNKRR. *J. International Computer Chess Association, 15*, 16–18.

Stiller, L. (1996). Multilinear algebra and chess endgames. En Nowakowski, R. J. (Ed.), *Games of No Chance, MSRI, 29*, *1996*. Mathematical Sciences Research Institute.

Stockman, G. (1979). A minimax algorithm better than alpha–beta? *AIJ, 12*, 179–196.

Stoffel, K., Taylor, M. y Hendler, J. (1997). Efficient management of very large ontologies. En *AAAI-97*.

Stone, M. (1974). Cross-validatory choice and assessment of statistical predictions. *J. Royal Statistical Society, 36*, 111–133.

Stone, P. (2000). *Layered Learning in Multi-Agent Systems: A Winning Approach to Robotic Soccer*. MIT Press.

Stone, P. (2003). Multiagent competitions and research: Lessons from RoboCup and TAC. En Lima, P. U. y Rojas, P. (Eds.), *RoboCup-2002: Robot Soccer World Cup VI*. Springer Verlag.

Stone, P. (2016). What's hot at RoboCup. En *AAAI-16*.

Stone, P., Brooks, R. A., Brynjolfsson, E., Calo, R., Etzioni, O., Hager, G., Hirschberg, J., Kalyanakrishnan,

S., Kamar, E., Kraus, S. *et al*. (2016). Artificial intelligence and life in 2030. Tech. rep., Stanford University One Hundred Year Study on Artificial Intelligence: Report of the 2015-2016 Study Panel.

Stone, P., Kaminka, G. y Rosenschein, J. S. (2009). Leading a best-response teammate in an ad hoc team. En *AAMAS Workshop in Agent Mediated Electronic Commerce*.

Stone, P., Sutton, R. S. y Kuhlmann, G. (2005). Reinforcement learning for robocup soccer keepaway. *Adaptive Behavior, 13*, 165–188.

Storvik, G. (2002). Particle filters for state-spacemodels with the presence of unknown static parameters. *IEEE Transactions on Signal Processing, 50*, 281–289.

Strachey, C. (1952). Logical or non-mathematical programmes. En *Proc. 1952 ACM National Meeting*.

Stratonovich, R. L. (1959). Optimum nonlinear systems which bring about a separation of a signal with constant parameters from noise. *Radiofizika, 2*, 892–901.

Stratonovich, R. L. (1965). On value of information. Izvestiya of USSR Academy of Sciences, *Technical Cybernetics, 5*, 3–12.

Sturtevant, N. R. y Bulitko, V. (2016). Scrubbing during learning in real-time heuristic search. *JAIR, 57*, 307–343.

Subramanian, D. y Wang, E. (1994). Constraintbased kinematic synthesis. En *Proc. International Conference on Qualitative Reasoning*.

Suk, H.-I., Sin, B.-K., y Lee, S.-W. (2010). Hand gesture recognition based on dynamic Bayesian network framework. Pattern Recognition, 43, 3059–3072.

Sun, Y., Wang, S., Li, Y., Feng, S., Tian, H., Wu, H., y Wang, H. (2019). ERNIE 2.0: A continual pre-training framework for language understanding. arXiv:1907.12412.

Sussman, G. J. (1975). *A Computer Model of Skill Acquisition*. Elsevier.

Sutcliffe, G. (2016). The CADE ATP system competition - CASC. *AIMag, 37*, 99–101.

Sutcliffe, G. y Suttner, C. (1998). The TPTP Problem Library: CNF Release v1.2.1. *JAR, 21*, 177–203.

Sutcliffe, G., Schulz, S., Claessen, K. y Gelder, A. V. (2006). Using the TPTP language for writing derivations and finite interpretations. En *Proc. International Joint Conference on Automated Reasoning*.

Sutherland, I. (1963). Sketchpad: A man-machine graphical communication system. En Proc. *Spring Joint Computer Conference*.

Sutskever, I., Vinyals, O., and Le, Q. V. (2015). Sequence to sequence learning with neural networks. En *NeurIPS 27*.

Sutton, C. y McCallum, A. (2007). An introduction to conditional random fields for relational learning. En Getoor, L. y Taskar, B. (Eds.), *Introduction to Statistical Relational Learning*. MIT Press.

Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. *Machine Learning, 3*, 9–44.

Sutton, R. S., McAllester, D. A., Singh, S. y Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. En *NeurIPS 12*.

Sutton, R. S. (1990). Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. En *ICML-90*.

Sutton, R. S. y Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2a edición). MIT Press.

Swade, D. (2000). *Difference Engine: Charles Babbage And The Quest To Build The First Computer*. Diane Publishing Co.

Sweeney, L. (2000). Simple demographics often identify people uniquely. *Health (San Francisco), 671*, 1–34.

Sweeney, L. (2002a). Achieving k-anonymity privacy protection using generalization and suppression. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 10*, 571–588.

Sweeney, L. (2002b). k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 10*, 557–570.

Swerling, P. (1959). First order error propagation in a stagewise smoothing procedure for satellite observations. *J. Astronautical Sciences, 6*, 46–52.

Swift, T. y Warren, D. S. (1994). Analysis of SLGWAM evaluation of definite programs. En *Logic Programming: Proc. 1994 International Symposium*.

Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I. y Fergus, R. (2013). Intriguing properties of neural networks. arXiv:1312.6199.

Szeliski, R. (2011). *Computer Vision: Algorithms and Applications*. Springer-Verlag.

Szepesvari, C. (2010). Algorithms for reinforcement learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning, 4*, 1–103.

Tadepalli, P., Givan, R. y Driessens, K. (2004). Relational reinforcement learning: An overview. En *ICML-04*.

Tait, P. G. (1880). Note on the theory of the "15 puzzle". *Proc. Royal Society of Edinburgh, 10*, 664–665.

Tamaki, H. y Sato, T. (1986). OLD resolution with tabulation. En *ICLP-86*.

Tan, P., Steinbach, M., Karpatne, A. y Kumar, V. (2019). *Introduction to Data Mining* (2a edición). Pearson.

Tang, E. (2018). A quantum-inspired classical algorithm for recommendation systems. arXiv:1807.04271.

Tarski, A. (1935). DieWahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica, 1*, 261–405.

Tarski, A. (1941). *Introduction to Logic and to the Methodology of Deductive Sciences*. Dover.

Tarski, A. (1956). *Logic, Semantics, Metamathematics: Papers from 1923 to 1938*. Oxford University Press.

Tash, J. K. y Russell, S. J. (1994). Control strategies for a stochastic planner. En *AAAI-94*.

Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. d. L., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., *et al*. (2018). Deepmind control suite. arXiv:1801.00690.

Tate, A. (1975a). Interacting goals and their use. En *IJCAI-75*.

Tate, A. (1975b). *Using Goal Structure to Direct Search in a Problem Solver*. Ph.D. thesis, University of Edinburgh.

Tate, A. (1977). Generating project networks. En *IJCAI-77*.

Tate, A. y Whiter, A.M. (1984). Planning with multiple resource constraints and an application to a naval planning problem. En *Proc. First Conference on AI Applications*.

Tatman, J. A. y Shachter, R. D. (1990). Dynamic programming and influence diagrams. *IEEE Transactions on Systems, Man and Cybernetics, 20*, 365–379.

Tattersall, C. (1911). *A Thousand End-Games: A Collection of Chess Positions That Can be Won or Drawn by the Best Play*. British Chess Magazine.

Taylor, A. D. y Zwicker, W. S. (1999). *Simple Games: Desirability Relations, Trading, Pseudoweightings*. Princeton University Press.

Taylor, G., Stensrud, B., Eitelman, S. y Dunham, C. (2007). Towards automating airspace management. En *Proc. Computational Intelligence for Security and Defense Applications (CISDA) Conference*.

Taylor, P. (2009). *Text-to-Speech Synthesis*. Cambridge University Press.

Tedrake, R., Zhang, T. W. y Seung, H. S. (2004). Stochastic policy gradient reinforcement learning on a simple 3D biped. En *IROS-04*.

Tellex, S., Kollar, T., Dickerson, S., Walter, M. R., Banerjee, A., Teller, S. y Roy, N. (2011). Understanding natural language commands for robotic navigation and mobile manipulation. En *AAAI-11*.

Tenenbaum, J. B., Griffiths, T. L. y Niyogi, S. (2007). Intuitive theories as grammars for causal inference. En Gopnik, A. y Schulz, L. (Eds.), *Causal Learning: Psychology, Philosophy, and Computation*. Oxford University Press.

Tesauro, G. (1990). Neurogammon: A neural-network backgammon program. En *IJCNN-90*.

Tesauro, G. (1992). Practical issues in temporal difference learning. *Machine Learning, 8*, 257–277.

Tesauro, G. (1995). Temporal difference learning and TD-Gammon. *CACM, 38*, 58–68.

Tesauro, G. y Galperin, G. R. (1997). On-line policy improvement usingMonte-Carlo search. En *NeurIPS 9*.

Tetlock, P. E. (2017). *Expert Political Judgment: How Good Is It? How Can We Know?* Princeton University Press.

Teyssier, M. yd Koller, D. (2005). Ordering-based search: A simple and effective algorithm for learning Bayesian networks. En *UAI-05*.

Thaler, R. (1992). *The Winner's Curse: Paradoxes and Anomalies of Economic Life*. Princeton University Press.

Thaler, R. y Sunstein, C. (2009). *Nudge: Improving Decisions About Health, Wealth, and Happiness*. Penguin.

Thayer, J. T., Dionne, A. y Ruml, W. (2011). Learning inadmissible heuristics during search. En *ICAPS-11*.

Theocharous, G., Murphy, K. y Kaelbling, L. P. (2004). Representing hierarchical POMDPs as DBNs for multi-scale robot localization. En *ICRA-04*.

Thiele, T. (1880). Om anvendelse af mindset kvadraters methode i nogle tilfælde, hvor en komplikation af visse slags uensartede tilfældige fejlkilder giver fejlene en 'systematisk' karakter. *Vidensk. Selsk. Skr. 5. Rk., naturvid. og mat. Afd., 12*, 381–408.

Thielscher, M. (1999). From situation calculus to fluent calculus: State update axioms as a solution to the inferential frame problem. *AIJ, 111*, 277–299.

Thomas, P. S., da Silva, B. C., Barto, A. G. y Brunskill, E. (2017). On ensuring that intelligent machines are well-behaved. arXiv:1708.05448.

Thomaz, A., Hoffman, G., Cakmak, M. *et al*. (2016). Computational human-robot interaction. *Foundations and Trends in Robotics, 4*, 105–223.

Thompson, K. (1986). Retrograde analysis of certain endgames. *J. International Computer Chess Association, 9*, 131–139.

Thompson, K. (1996). 6-piece endgames. *J. International Computer Chess Association, 19*, 215–226.

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika, 25*, 285–294.

Thorndike, E. (1911). *Animal Intelligence*. Macmillan.

Thornton, C., Hutter, F., Hoos, H. H. y Leyton-Brown, K. (2013). Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. En *KDD-13*.

Thrun, S., Burgard, W. y Fox, D. (2005). *Probabilistic Robotics*. MIT Press.

Thrun, S., Fox, D. y Burgard, W. (1998). A probabilistic approach to concurrent mapping and localization for mobile robots. *Machine Learning, 31*, 29–53.

Thrun, S. (2006). Stanley, the robot that won the DARPA Grand Challenge. *J. Field Robotics, 23*, 661–692.

Thrun, S. y Pratt, L. (2012). *Learning to Learn*. Springer.

Thurstone, L. L. (1927). A law of comparative judgment. *Psychological Review, 34*, 273–286.

Tian, J., Paz, A. y Pearl, J. (1998). Finding a minimal d-separator. Tech. rep., UCLA Department of Computer Science.

Tikhonov, A. N. (1963). Solution of incorrectly formulated problems and the regularization method. *Soviet Math. Dokl., 5*, 1035–1038.

Tipping, M. E. y Bishop, C. M. (1999). Probabilistic principal component analysis. *J. Royal Statistical Society, 61*, 611–622.

Titterington, D. M., Smith, A. F. M., and Makov, U. E. (1985). *Statistical Analysis of Finite Mixture Distributions*. Wiley.

Toma, P. (1977). SYSTRAN as a multilingual machine translation system. En *Proc. Third European Congress on Information Systems and Networks: Overcoming the Language Barrier*.

Tomasi, C. y Kanade, T. (1992). Shape and motion from image streams under orthography: A factorization method. *IJCV, 9*, 137–154.

Topol, E. (2019). *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*. Basic Books.

Torralba, A., Fergus, R. y Weiss, Y. (2008). Small codes and large image databases for recognition. En *CVPR*.

Torralba, A., Linares López, C. y Borrajo, D. (2016). Abstraction heuristics for symbolic bidirectional search. En *IJCAI-16*.

Tramèr, F., Zhang, F., Juels, A., Reiter, M. K. y Ristenpart, T. (2016). Stealing machine learning models via prediction APIs. En *USENIX Security Symposium*.

Tran, D., Hoffman, M., Saurous, R. A., Brevdo, E., Murphy, K. y Blei, D. M. (2017). Deep probabilistic programming. En *ICLR-17*.

Trappenberg, T. (2010). *Fundamentals of Computational Neuroscience* (2a edición). Oxford University Press.

Tsang, E. (1993). *Foundations of Constraint Satisfaction*. Academic Press.

Tshitoyan, V., Dagdelen, J., Weston, L., Dunn, A., Rong, Z., Kononova, O., Persson, K. A., Ceder, G. y Jain, A. (2019). Unsupervised word embeddings capture latent knowledge from materials science literature. *Nature, 571*, 95.

Tsitsiklis, J. N. y Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control, 42*, 674–690.

Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

Tumer, K. y Wolpert, D. (2000). Collective intelligence and Braess' paradox. En *AAAI-00*.

Turian, J., Ratinov, L. y Bengio, Y. (2010). Word representations: a simple and general method for semisupervised learning. En *ACL-10*.

Turing, A. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Mathematical Society, 2nd series, 42*, 230–265.

Turing, A. (1948). Intelligent machinery. Tech. rep., National Physical Laboratory. reprinted in (Ince, 1992).

Turing, A. (1950). Computing machinery and intelligence. *Mind, 59*, 433–460.

Turing, A., Strachey, C., Bates, M. A. y Bowden, B. V. (1953). Digital computers applied to games. En Bowden, B. V. (Ed.), *Faster than Thought*. Pitman.

Turing, A. (1947). Lecture to the London Mathematical Society on 20 February 1947.

Turing, A. (1996). Intelligent machinery, a heretical theory. *Philosophia Mathematica, 4*, 256–260. Originally written c. 1951.

Tversky, A. y Kahneman, D. (1982). Causal schemata in judgements under uncertainty. En Kahneman, D., Slovic, P. y Tversky, A. (Eds.), *Judgement Under Uncertainty: Heuristics and Biases*. Cambridge University Press.

Tygar, J. D. (2011). Adversarial machine learning. *IEEE Internet Computing, 15*, 4–6.

Ullman, J. D. (1985). Implementation of logical query languages for databases. *ACM Transactions on Database Systems, 10*, 289–321.

Ullman, S. (1979). The Interpretation of Visual Motion. MIT Press.

Urmson, C. y Whittaker, W. (2008). Self-driving cars and the Urban Challenge. *IEEE Intelligent Systems, 23*, 66–68.

Valiant, L. (1984). A theory of the learnable. *CACM, 27*, 1134–1142.

Vallati, M., Chrpa, L. y Kitchin, D. E. (2015). Portfolio-based planning: State of the art, common practice and open challenges. *AI Commun., 28(4)*, 717–733.

Van Beek, P. (2006). Backtracking search algorithms. En Rossi, F., Van Beek, P. y Walsh, T. (Eds.), *Handbook of Constraint Programming*. Elsevier.

Van Beek, P. y Chen, X. (1999). CPlan: A constraint programming approach to planning. En *AAAI-99*.

Van Beek, P. y Manchak, D. (1996). The design and experimental analysis of algorithms for temporal reasoning. *JAIR, 4*, 1–18.

Van Bentham, J. y ter Meulen, A. (1997). *Handbook of Logic and Language*. MIT Press.

Van den Oord, A., Dieleman, S., and Schrauwen, B. (2014). Deep content-based music recommendation. En *NeurIPS 26*.

Van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. y Kavukcuoglu, K. (2016a). WaveNet: A generative model for raw audio. arXiv:1609.03499.

Van den Oord, A., Kalchbrenner, N. y Kavukcuoglu, K. (2016b). Pixel recurrent neural networks. arXiv:1601.06759.

Van Harmelen, F., Lifschitz, V. y Porter, B. (2007). *The Handbook of Knowledge Representation*. Elsevier.

Van Heijenoort, J. (Ed.). (1967). *From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931*. Harvard University Press.

Van Hentenryck, P., Saraswat, V. y Deville, Y. (1998). Design, implementation, and evaluation of the constraint language cc(FD). *J. Logic Programming, 37*, 139–164.

Van Hoeve, W.-J. (2001). The alldifferent constraint: a survey. En *6th AnnualWorkshop of the ERCIM Working Group on Constraints*.

Van Hoeve, W.-J. y Katriel, I. (2006). Global constraints. En Rossi, F., Van Beek, P. y Walsh, T. (Eds.), *Handbook of Constraint Processing*. Elsevier.

Van Lambalgen, M. y Hamm, F. (2005). *The Proper Treatment of Events*. Wiley-Blackwell.

Van Nunen, J. A. E. E. (1976). A set of successive approximation methods for discounted Markovian decision problems. *Zeitschrift fur Operations Research, Serie A, 20*, 203–208.

Van Roy, P. L. (1990). Can logic programming execute as fast as imperative programming? Report, Computer Science Division, UC Berkeley.

Vapnik, V.N. (1998). *Statistical Learning Theory*. Wiley.

Vapnik, V. N. y Chervonenkis, A. Y. (1971). On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability and Its Applications, 16*, 264–280.

Vardi, M. Y. (1996). An automata-theoretic approach to linear temporal logic. En Moller, F. y Birtwistle, G. (Eds.), *Logics for Concurrency*. Springer.

Varian, H. R. (1995). Economic mechanism design for computerized agents. En *USENIX Workshop on Electronic Commerce*.

Vasilache, N., Johnson, J., Mathieu, M., Chintala, S., Piantino, S. y LeCun, Y. (2014). Fast convolutional nets with fbfft: A GPU performance evaluation. arXiv:1412.7580.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. y Polosukhin, I. (2018). Attention is all you need. En *NeurIPS 30*.

Veach, E. y Guibas, L. J. (1995). Optimally combining sampling techniques for Monte Carlo rendering. En *Proc. 22rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*.

Venkatesh, S. (2012). *The Theory of Probability: Explorations and Applications*. Cambridge University Press.

Vere, S. A. (1983). Planning in time: Windows and durations for activities and goals. *PAMI, 5*, 246–267.

Verma, S. y Rubin, J. (2018). Fairness definitions explained. En 2018 IEEE/ACM International Workshop on Software Fairness.

Verma, V., Gordon, G., Simmons, R. y Thrun, S. (2004). Particle filters for rover fault diagnosis. *IEEE Robotics and Automation Magazine, June*.

Vinge, V. (1993). The coming technological singularity: How to survive in the post-human era. En *Proc. Vision-21: Interdisciplinary Science and Engineering in the Era of Cyberspace*. NASA.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Hassabis, D., Apps, C. y Silver, D. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature, 575*, 350–354.

Vinyals, O., Ewalds, T., Bartunov, S. y Georgiev, P. (2017a). StarCraft II: A new challenge for reinforcement learning. arXiv:1708.04782.

Vinyals, O., Toshev, A., Bengio, S. y Erhan, D. (2017b). Show and tell: Lessons learned from the 2015 MSCOCO image captioning challenge. *PAMI, 39*, 652–663.

Viola, P. y Jones, M. (2004). Robust real-time face detection. *IJCV, 57*, 137–154.

Visser, U., Ribeiro, F., Ohashi, T. y Dellaert, F. (Eds.). (2008). RoboCup 2007: *Robot Soccer World Cup XI*. Springer.

Viterbi, A. J. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory, 13*, 260–269.

Vlassis, N. (2008). *A Concise Introduction to Multiagent Systems and Distributed Artificial Intelligence*. Morgan & Claypool.

Von Mises, R. (1928). *Wahrscheinlichkeit, Statistik und Wahrheit*. J. Springer.

Von Neumann, J. (1928). Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen, 100*, 295–320.

Von Neumann, J. y Morgenstern, O. (1944). *Theory of Games and Economic Behavior* (primera edición). Princeton University Press.

Von Winterfeldt, D. y Edwards, W. (1986). *Decision Analysis and Behavioral Research*. Cambridge University Press.

Vossen, T., Ball, M., Lotem, A. y Nau, D. S. (2001). Applying integer programming to AI planning. *Knowledge Engineering Review, 16*, 85–100.

Wainwright, M. y Jordan, M. I. (2008). Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning, 1*, 1–305.

Walker, G. (1931). On periodicity in series of related terms. *Proc. Roy. Soc., A, 131*, 518–532.

Walker, R. J. (1960). An enumerative technique for a class of combinatorial problems. En *Proc. Sympos. Appl. Math.*, Vol. 10.

Wallace, A. R. (1858). On the tendency of varieties to depart indefinitely from the original type. *Proc. Linnean Society of London, 3*, 53–62.

Walpole, R. E., Myers, R. H., Myers, S. L. y Ye, K. E. (2016). *Probability and Statistics for Engineers and Scientists* (9a edición). Pearson.

Walsh, T. (2015). Turing's red flag. arXiv:1510.09033.

Waltz, D. (1975). Understanding line drawings of scenes with shadows. En *Winston, P. H. (Ed.), The Psychology of Computer Vision*. McGraw-Hill.

Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., Hill, F., Levy, O. y Bowman, S. R. (2019). SuperGLUE: A stickier benchmark for general-purpose language understanding systems. arXiv:1905.00537.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O. y Bowman, S. (2018a). GLUE: A multi-task benchmark and analysis platform for natural language understanding. arXiv:1804.07461.

Wang, J., Zhu, T., Li, H., Hsueh, C.-H. y Wu, I.-C. (2018b). Belief-state Monte Carlo tree search for phantom Go. *IEEE Transactions on Games, 10*, 139–154.

Wanner, E. (1974). *On Remembering, Forgetting and Understanding Sentences*. Mouton.

Warren, D. H. D. (1974). WARPLAN: A System for Generating Plans. Department of Computational Logic Memo, University of Edinburgh.

Warren, D. H. D. (1983). An abstract Prolog instruction set. Technical note, SRI International.

Wasserman, L. (2004). *All of Statistics*. Springer.

Watkins, C. J. (1989). *Models of Delayed Reinforcement Learning*. Ph.D. thesis, Psychology Department, Cambridge University.

Watson, J. D. y Crick, F. (1953). A structure for deoxyribose nucleic acid. *Nature, 171*, 737.

Wattenberg, M., Viégas, F. y Johnson, I. (2016). How to use t-SNE effectively. *Distill, 1*.

Waugh, K., Schnizlein, D., Bowling, M. y Szafron, D. (2009). Abstraction pathologies in extensive games. En *AAMAS-09*.

Weibull, J. (1995). *Evolutionary Game Theory*. MIT Press.

Weidenbach, C. (2001). SPASS: Combining superposition, sorts and splitting. En *Robinson, A. y Voronkov, A. (Eds.), Handbook of Automated Reasoning*. MIT Press.

Weiss, G. (2000a). *Multiagent Systems*. MIT Press.

Weiss, Y. (2000b). Correctness of local probability propagation in graphical models with loops. *Neural Computation, 12*, 1–41.

Weiss, Y. y Freeman, W. (2001). Correctness of belief propagation in Gaussian graphical models of arbitrary topology. *Neural Computation, 13*, 2173–2200.

Weizenbaum, J. (1976). *Computer Power and Human Reason*. W. H. Freeman.

Weld, D. S. (1994). An introduction to least commitment planning. *AIMag, 15*, 27–61.

Weld, D. S. (1999). Recent advances in AI planning. *AIMag, 20*, 93–122.

Weld, D. S., Anderson, C. R. y Smith, D. E. (1998). Extending Graphplan to handle uncertainty and sensing actions. En *AAAI-98*.

Weld, D. S. y de Kleer, J. (1990). *Readings in Qualitative Reasoning about Physical Systems*. Morgan Kaufmann.

Weld, D. S. y Etzioni, O. (1994). The first law of robotics: A call to arms. En *AAAI-94*.

Wellman, M. P. (1985). Reasoning about preference models. Technical report, Laboratory for Computer Science, MIT.

Wellman, M. P. (1988). *Formulation of Tradeoffs in Planning under Uncertainty*. Ph.D. thesis, MIT.

Wellman, M. P. (1990a). Fundamental concepts of qualitative probabilistic networks. *AIJ, 44*, 257–303.

Wellman, M. P. (1990b). The STRIPS assumption for planning under uncertainty. En *AAAI-90*.

Wellman, M. P., Breese, J. S. y Goldman, R. (1992). From knowledge bases to decision models. *Knowledge Engineering Review, 7*, 35–53.

Wellman, M. P. y Doyle, J. (1992). Modular utility representation for decision-theoretic planning. En *ICAPS-92*.

Wellman, M. P., Wurman, P., O'Malley, K., Bangera, R., Lin, S., Reeves, D. y Walsh, W. (2001). Designing the market game for a trading agent competition. *IEEE Internet Computing, 5*, 43–51.

Werbos, P. (1974). *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. Ph.D. thesis, Harvard University.

Werbos, P. (1990). Backpropagation through time: What it does and how to do it. *Proc. IEEE, 78*, 1550–1560.

Werbos, P. (1992). Approximate dynamic programming for real-time control and neural modeling. En White, D. A. y Sofge, D. A. (Eds.), *Handbook of Intelligent Control: Neural, Fuzzy, and Adaptive Approaches*. Van Nostrand Reinhold.

Werbos, P. (1977). Advanced forecasting methods for global crisis warning and models of intelligence. *General Systems Yearbook, 22*, 25–38.

Wesley, M. A. y Lozano-Perez, T. (1979). An algorithm for planning collision-free paths among polyhedral objects. *CACM, 22*, 560–570.

West, D. M. (2018). *The Future of Work: Robots, AI, and Automation*. Brookings Institution Press.

West, S. M., Whittaker, M. y Crawford, K. (2019). Discriminating systems: Gender, race and power in AI. Tech. rep., AI Now Institute.

Wexler, Y. y Meek, C. (2009). MAS: A multiplicative approximation scheme for probabilistic inference. En *NeurIPS 21*.

Wheatstone, C. (1838). On some remarkable, and hitherto unresolved, phenomena of binocular vision. *Phil. Trans. Roy. Soc., 2*, 371–394.

White, C., Neiswanger, W. y Savani, Y. (2019). BANANAS: Bayesian optimization with neural architectures for neural architecture search. arXiv:1910.11858.

Whitehead, A. N. y Russell, B. (1910). *Principia Mathematica*. Cambridge University Press.

Whittle, P. (1979). Discussion of Dr Gittins' paper. *J. Royal Statistical Society, 41*, 165.

Whorf, B. (1956). *Language, Thought, and Reality*. MIT Press.

Widrow, B. (1962). Generalization and information storage in networks of ADALINE "neurons". En Yovits, M. C., Jacobi, G. T. y Goldstein, G. D. (Eds.), *Self-Organizing Systems*. Spartan.

Widrow, B. y Hoff, M. E. (1960). Adaptive switching circuits. En *IRE WESCON Convention Record*.

Wiedijk, F. (2003). Comparing mathematical provers. En *Proc. 2nd Int. Conf. on Mathematical Knowledge Management*.

Wiegley, J., Goldberg, K., Peshkin, M. y Brokowski, M. (1996). A complete algorithm for designing passive fences to orient parts. En *ICRA-96*.

Wiener, N. (1942). The extrapolation, interpolation, and smoothing of stationary time series. Tech. rep., Research Project DIC-6037, MIT.

Wiener, N. (1948). *Cybernetics*. Wiley.

Wiener, N. (1950). *The Human Use of Human Beings*. Houghton Mifflin.

Wiener, N. (1960). Some moral and technical consequences of automation. *Science, 131*, 1355–1358.

Wiener, N. (1964). *God & Golem, Inc: A Comment on Certain Points Where Cybernetics Impinges on Religion*. MIT Press.

Wilensky, R. (1978). *Understanding Goal-Based Stories*. Ph.D. thesis, Yale University.

Wilkins, D. E. (1988). *Practical Planning: Extending the AI Planning Paradigm*. Morgan Kaufmann.

Wilkins, D. E. (1990). Can AI planners solve practical problems? *Computational Intelligence, 6*, 232–246.

Wilks, Y. (2010). *Close Engagements With Artificial Companions: Key Social, Psychological, Ethical and Design Issues*. John Benjamins.

Wilks, Y. (2019). *Artificial Intelligence: Modern Magic or Dangerous Future*. Icon.

Williams, A., Nangia, N. y Bowman, S. (2018). *A broad-coverage challenge corpus for sentence understanding through inference*. En NAACL HLT.

Williams, B., Ingham, M., Chung, S. y Elliott, P. (2003). Model-based programming of intelligent embedded systems and robotic space explorers. *Proc. IEEE, 91*(212–237).

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning, 8*, 229–256.

Williams, R. J. y Zipser, D. (1989). A learning algorithm for continually running fully recurrent neural networks. *Neural Computation, 1*, 270–280.

Williams, R. J. y Baird, L. C. I. (1993). Tight performance bounds on greedy policies based on imperfect value functions. Tech. rep., College of Computer Science, Northeastern University.

Wilson, D. H. (2011). *Robopocalypse*. Doubleday.

Wilson, R. A. y Keil, F. C. (Eds.). (1999). *The MIT Encyclopedia of the Cognitive Sciences*. MIT Press.

Wilson, R. (2004). *Four Colors Suffice*. Princeton University Press.

Wilt, C. M. y Ruml, W. (2014). Speedy versus greedy search. En *Seventh Annual Symposium on Combinatorial Search*.

Wilt, C. M. y Ruml, W. (2016). Effective heuristics for suboptimal best-first search. *JAIR, 57*, 273–306.

Wingate, D. y Seppi, K. D. (2005). Prioritization methods for accelerating MDP solvers. *JMLR, 6*, 851–881.

Wingate, D., Stuhlmüller, A. y Goodman, N. D. (2011). Lightweight implementations of probabilistic programming languages via transformational compilation. En *AISTATS-11*.

Winograd, S. y Cowan, J. D. (1963). *Reliable Computation in the Presence of Noise*. MIT Press.

Winograd, T. (1972). Understanding natural language. *Cognitive Psychology, 3*, 1–191.

Winston, P. H. (1970). Learning structural descriptions from examples. Technical report, Department of Electrical Engineering and Computer Science, MIT.

Wintermute, S., Xu, J. y Laird, J. (2007). SORTS: A human-level approach to real-time strategy AI. En *Proc. Third Artificial Intelligence and Interactive Digital Entertainment Conference*.

Winternitz, L. (2017). Autonomous navigation above the GNSS constellations and beyond: GPS navigation for the magnetospheric multiscale mission and SEXTANT pulsar navigation demonstration. Tech. rep., NASA Goddard Space Flight Center.

Witten, I. H. (1977). An adaptive optimal controller for discrete-time Markov environments. *Information and Control, 34*, 286–295.

Witten, I. H. y Bell, T. C. (1991). The zerofrequency problem: Estimating the probabilities of novel events in adaptive text compression. *IEEE Transactions on Information Theory, 37*, 1085–1094.

Witten, I. H. y Frank, E. (2016). *Data Mining: PracticalMachine Learning Tools and Techniques* (4a edición). Morgan Kaufmann.

Witten, I. H., Moffat, A. y Bell, T. C. (1999). *Managing Gigabytes: Compressing and Indexing Documents and Images* (2a edición). Morgan Kaufmann.

Wittgenstein, L. (1922). *Tractatus Logico-Philosophicus* (2a edición). Routledge y Kegan Paul. Reprinted 1971, edited by D. F. Pears and B. F. McGuinness.

Wittgenstein, L. (1953). *Philosophical Investigations*. Macmillan.

Wojciechowski, W. S. y Wojcik, A. S. (1983). Automated design of multiple-valued logic circuits by automated theorem proving techniques. *IEEE Transactions on Computers, C-32*, 785–798.

Wolfe, J. y Russell, S. J. (2007). Exploiting belief state structure in graph search. En *ICAPS Workshop on Planning in Games*.

Wolpert, D. (2013). Ubiquity symposium: Evolutionary computation and the processes of life: what the no free lunch theorems really mean: how to improve search algorithms. *Ubiquity, December*, 1–15.

Wolpert, D. y Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Trans. Evolutionary Computation, 1(1)*, 67–82.

Wong, C., Houlsby, N., Lu, Y. y Gesmundo, A. (2019). Transfer learning with neural AutoML. En *NeurIPS 31*.

Woods, W. A. (1973). Progress in natural language understanding: An application to lunar geology. En *AFIPS Conference Proceedings*.

Woods, W. A. (1975). What's in a link? Foundations for semantic networks. En Bobrow, D. G. y Collins, A.M. (Eds.), *Representation and Understanding: Studies in Cognitive Science*. Academic Press.

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2a edición). Wiley.

Wooldridge, M. y Rao, A. (Eds.). (1999). *Foundations of Rational Agency*. Kluwer.

Wos, L., Carson, D. y Robinson, G. (1964). The unit preference strategy in theorem proving. En *Proc. Fall Joint Computer Conference*.

Wos, L., Carson, D. y Robinson, G. (1965). Efficiency and completeness of the set-of-support strategy in theorem proving. *JACM, 12*, 536–541.

Wos, L., Overbeek, R., Lusk, E. y Boyle, J. (1992). *Automated Reasoning: Introduction and Applications* (2a edició). McGraw-Hill.

Wos, L. y Robinson, G. (1968). Paramodulation and set of support. En *Proc. IRIA Symposium on Automatic Demonstration*.

Wos, L., Robinson, G., Carson, D. y Shalla, L. (1967). The concept of demodulation in theorem proving. *JACM, 14*, 698–704.

Wos, L. y Winker, S. (1983). Open questions solved with the assistance of AURA. En Bledsoe, W. W. y Loveland, D. (Eds.), *Automated Theorem Proving: After 25 Years*. American Mathematical Society.

Wos, L. y Pieper, G. (2003). *Automated Reasoning and the Discovery of Missing and Elegant Proofs*. Rinton Press.

Wray, R. E. y Jones, R. M. (2005). An introduction to Soar as an agent architecture. En Sun, R. (Ed.), *Cognition and Multi-Agent Interaction: From Cognitive Modeling to Social Simulation*. Cambridge University Press.

Wright, S. (1921). Correlation and causation. *J. Agricultural Research, 20*, 557–585.

Wright, S. (1931). Evolution in Mendelian populations. *Genetics, 16*, 97–159.

Wright, S. (1934). The method of path coefficients. *Annals of Mathematical Statistics, 5*, 161–215.

Wu, F. y Weld, D. S. (2008). Automatically refining the Wikipedia infobox ontology. En *17th World Wide Web Conference (WWW2008)*.

Wu, Y., Li, L. y Russell, S. J. (2016a). SWIFT: Compiled inference for probabilistic programming languages. En *IJCAI-16*.

Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K. *et al*. (2016b). Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv:1609.08144.

Wu, Y. y He, K. (2018). Group normalization. arXiv:1803.08494.

Xiong, W., Wu, L., Alleva, F., Droppo, J., Huang, X. y Stolcke, A. (2017). The Microsoft 2017 conversational speech recognition system. arXiv:1708.06073.

Yampolskiy, R. V. (2018). *Artificial Intelligence Safety and Security*. Chapman and Hall/CRC.

Yang, G., Lin, Y. y Bhattacharya, P. (2010). A driver fatigue recognition model based on information fusion and dynamic Bayesian network. *Inf. Sci., 180*, 1942–1954.

Yang, X.-S. (2009). Firefly algorithms for multimodal optimization. En *International Symposium on Stochastic Algorithms*.

Yang, X.-S. y Deb, S. (2014). Cuckoo search: Recent advances and applications. *Neural Computing and Applications, 24*, 169–174.

Yang, Z., Dai, Z., Yang, Y., Carbonell, J. G., Salakhutdinov, R. y Le, Q. V. (2019). XLNet: Generalized autoregressive pretraining for language understanding. arXiv:1906.08237.

Yarowsky, D. (1995). Unsupervised word sense disambiguation rivaling supervised methods. En *ACL-95*.

Ye, Y. (2011). The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Mathematics of Operations Research, 36*, 593–784.

Yedidia, J., Freeman, W. y Weiss, Y. (2005). Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on Information Theory, 51*, 2282–2312.

Yeo, H.-S., Minami, R., Rodriguez, K., Shaker, G. y Quigley, A. (2018). Exploring tangible interactions with radar sensing. *Proc. ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 2*, 1–25.

Ying, C., Kumar, S., Chen, D., Wang, T. y Cheng, Y. (2018). Image classification at supercomputer scale. arXiv:1811.06992.

Yip, K. M.-K. (1991). *KAM: A System for Intelligently Guiding Numerical Experimentation by Computer*. MIT Press.

Yngve, V. (1955). A model and an hypothesis for language structure. En *Locke, W. N. y Booth, A. D. (Eds.), Machine Translation of Languages*. MIT Press.

Yob, G. (1975). Hunt the wumpus! *Creative Computing, Sep/Oct*.

Yoshikawa, T. (1990). *Foundations of Robotics: Analysis and Control*. MIT Press.

You, Y., Pan, X., Wang, Z. y Lu, C. (2017). Virtual to real reinforcement learning for autonomous driving. arXiv:1704.03952.

Young, H. P. (2004). *Strategic Learning and Its Limits*. Oxford University Press.

Young, S., Gašič, M., Thompson, B. y Williams, J. (2013). POMDP-based statistical spoken dialog systems: A review. *Proc. IEEE, 101*, 1160–1179.

Younger, D. H. (1967). Recognition and parsing of context-free languages in time n^3. *Information and Control, 10*, 189–208.

Yu, D. y Deng, L. (2016). *Automatic Speech Recognition*. Springer-Verlag.

Yu, H.-F., Lo, H.-Y., Hsieh, H.-P. y Lou, J.-K. (2011). Feature engineering and classifier ensemble for KDD Cup 2010. En *Proc. KDD Cup 2010 Workshop*.

Yu, K., Sciuto, C., Jaggi, M., Musat, C. y Salzmann, M. (2019). Evaluating the search phase of neural architecture search. arXiv:1902.08142.

Yudkowsky, E. (2008). Artificial intelligence as a positive and negative factor in global risk. En Bostrom, N. y Cirkovic, M. (Eds.), *Global Catastrophic Risk*. Oxford University Press.

Yule, G. U. (1927). On a method of investigating periodicities in disturbed series, with special reference to Wolfer's sunspot numbers. *Phil. Trans. Roy. Soc., A, 226*, 267–298.

Zadeh, L. A. (1965). Fuzzy sets. *Information and Control, 8*, 338–353.

Zadeh, L. A. (1978). *Fuzzy sets as a basis for a theory of possibility. Fuzzy Sets and Systems, 1*, 3–28.

Zaritskii, V. S., Svetnik, V. B. y Shimelevich, L. I. (1975). Monte-Carlo technique in problems of optimal information processing. *Automation and Remote Control, 36*, 2015–22.

Zeckhauser, R. y Shepard, D. (1976). Where now for saving lives? *Law and Contemporary Problems, 40*, 5–45.

Zeeberg, A. (2017). D.I.Y. artificial intelligence comes to a Japanese family farm. *New Yorker*, August 10.

Zelle, J. y Mooney, R. (1996). Learning to parse database queries using inductive logic programming. En *AAAI-96*.

Zemel, R., Wu, Y., Swersky, K., Pitassi, T. y Dwork, C. (2013). Learning fair representations. En *ICML-13*.

Zemelman, B. V., Lee, G. A., Ng, M. y Miesenböck, G. (2002). Selective photostimulation of genetically chARGed neurons. *Neuron, 33*, 15–22.

Zermelo, E. (1913). Uber Eine Anwendung der Mengenlehre auf die Theorie des Schachspiels. En *Proc. Fifth International Congress of Mathematicians*.

Zermelo, E. (1976). An application of set theory to the theory of chess-playing. *Firbush News, 6*, 37–42. English translation of (Zermelo 1913).

Zettlemoyer, L. y Collins, M. (2005). Learning to map sentences to logical form: Structured classification with probabilistic categorial grammars. En UAI-05.

Zhang, C., Bengio, S., Hardt, M., Recht, B. y Vinyals, O. (2016). Understanding deep learning requires rethinking generalization. arXiv:1611.03530.

Zhang, H. y Stickel, M. E. (1996). An efficient algorithm for unit-propagation. En *Proc. Fourth International Symposium on Artificial Intelligence and Mathematics*.

Zhang, L., Pavlovic, V., Cantor, C. R. y Kasif, S. (2003). Human-mouse gene identification by comparative evidence integration and evolutionary analysis. *Genome Research, 13*, 1190–1202.

Zhang, N. L. y Poole, D. (1994). A simple approach to Bayesian network computations. En *Proc. 10th Canadian Conference on Artificial Intelligence*.

Zhang, S., Yao, L. y Sun, A. (2017). Deep learning based recommender system: A survey and new perspectives. arXiv:1707.07435.

Zhang, X., Zhao, J. y LeCun, Y. (2016). Character-level convolutional networks for text classification. En *NeurIPS 28*.

Zhang, Y., Pezeshki, M., Brakel, P., Zhang, S., Laurent, C., Bengio, Y. y Courville, A. (2017). Towards end-to-end speech recognition with deep convolutional neural networks. arXiv:1701.02720.

Zhao, K. y Huang, L. (2015). Type-driven incremental semantic parsing with polymorphism. En *NAACL HLT*.

Zhou, K., Doyle, J. y Glover, K. (1995). *Robust and Optimal Control*. Pearson.

Zhou, R. y Hansen, E. (2002). Memory-bounded A* graph search. En *Proc. 15th International FLAIRS Conference*.

Zhou, R. y Hansen, E. (2006). Breadth-first heuristic search. *AIJ, 170*, 385–408.

Zhu, B., Jiao, J. y Tse, D. (2019). Deconstructing generative adversarial networks. arXiv:1901.09465.

Zhu, D. J. y Latombe, J.-C. (1991). New heuristic algorithms for efficient hierarchical path planning. *IEEE Transactions on Robotics and Automation, 7*, 9–20.

Zhu, J.-Y., Park, T., Isola, P. y Efros, A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. En *ICCV-17*.

Zhu, M., Zhang, Y., Chen, W., Zhang, M. y Zhu, J. (2013). Fast and accurate shift-reduce constituent parsing. En *ACL-13*.

Ziebart, B. D., Maas, A. L., Dey, A. K. y Bagnell, J. A. (2008). Navigate like a cabbie: Probabilistic reasoning from observed context-aware behavior. En *Proc. 10th Int. Conf. on Ubiquitous Computing*.

Ziebart, B. D., Ratliff, N., Gallagher, G., Mertz, C., Peterson, K., Bagnell, J. A., Hebert, M., Dey, A. K. y Srinivasa, S. (2009). Planning-based prediction for pedestrians. En *IROS-09*.

Zimmermann, H.-J. (Ed.). (1999). *Practical Applications of Fuzzy Technologies*. Kluwer.

Zimmermann, H.-J. (2001). *Fuzzy Set Theory—And Its Applications* (4a edición). Kluwer.

Zinkevich, M., Johanson, M., Bowling, M. y Piccione, C. (2008). Regret minimization in games with incomplete information. En *NeurIPS 20*.

Zipf, G. (1935). *The Psychobiology of Language*. Houghton Mifflin.

Zipf, G. (1949). *Human Behavior and the Principle of Least Effort*. Addison-Wesley.

Zobrist, A. L. (1970). *Feature Extraction and Representation for Pattern Recognition and the Game of Go*. Ph.D. thesis, University of Wisconsin.

Zollmann, A., Venugopal, A., Och, F. J. y Ponte, J. (2008). A systematic comparison of phrase-based, hierarchical and syntax-augmented statistical MT. En *COLING-08*.

Zoph, B. y Le, Q. V. (2016). Neural architecture search with reinforcement learning. arXiv:1611.01578.

Zuse, K. (1945). The Plankalkül. Report, Gesellschaft f¨ur Mathematik und Datenverarbeitung.

Zweig, G. y Russell, S. J. (1998). Speech recognition with dynamic Bayesian networks. En *AAAI-98*.

Índice

Los números de página en **negrita** se refieren a definiciones de términos y algoritmos.
Los números de página en *cursiva* se refieren a los elementos de la bibliografía.

Símbolos

α tasa de aprendizaje (alfa), 765
α Constante de normalización (alfa), 400
$\wedge$ (y), 217
χ^2 (chi al cuadrado), 664
$\vdash$ (deriva), 216
$\models$ (implicación), 214
ϵ (épsilon), **673**
$\exists$ (existe), 262
$\forall$ (cuantificador universal), 261
γ (gamma) factor de descuento, 642
$\lrcorner$ (brecha) en la oración, 846
$|$ (dado), 389
$\Leftrightarrow$ (si y sólo si), 217
$\Rightarrow$ (implica), 217
$\sim$ (indiferente), 530
λ (lambda)-expresión, **259**
∇ (nabla) gradiente, 120
$\neg$ (no), 217
$\vee$ (o), 217
$>$ (preferido), 530
σ (sigma) desviación estándar, 1028
$^\top$ (transposición de matriz), 1026
3-SÁBADO, 248, 249, 290, 433
8--puzzle, **68**, 97, 100, 106
15-puzzle, 68, 106

A

$A(s)$ (acciones en un estado), 562
A* búsqueda, 85–90
Aaronson, S., 984, *1034*
Aarts, E., 206, *1034*
Aarup, M., 384, *1034*
abandono (en redes neuronales), **772**, 787
Abbas, A., 559, 560, *1034*
Abbeel, P., 62, 498, 561, 648, 719, 790, 817, 821, 822, 978, 979, 1004, *1034, 1040, 1044, 1046, 1048, 1051, 1052, 1061*
Abbott, L. F., 788, 822, *1041*
abd James Andrew Bagnell, B. D. Z., 817, *1050*
Abdennadher, S., 207, *1044*
Abdolmaleki, A., 822, *1063*
Abelson, R. P., 23, *1060*
Abney, S., 838, *1034*
Aboian, M. S., 30, *1042*
Abramson, B., 176, 597, *1034*
Abreu, D., 647, *1034*
abstracción de estado, **355**, 821
abstracción, **66**, 172, 622
ABSTRIPS (sistema de planificación), 382
Abu-Hanna, A., 410, *1053*
AC-3, **187**
acción, 36, **65**, 105
  articulación, **603**
  de alto nivel, **357**
  egocéntrico, 67 años
  irreversible, **136**, 799
  monitoreo, 372, 373
  primitivo, **357**
  racional, 7, 34
acción común, **603**
acción de alto nivel, **357**
acción egocéntrica, 67
acción irreversible, **136**, 799
acción pertinente, **350**
acción preferida, **355**
acciones humanas, 979
Acharya, A., 109, *1039*
Achlioptas, D., 249, *1034*
Ackerman, E., 29, *1034*
Ackerman, N., 526, *1034*
Ackley, D. H., págs. 143 y *1034*
acondicionamiento de conjuntos de cortes, **201**, 204, 456
acondicionamiento, **396**
ACT (arquitectura cognitiva), 292
actitud proposicional, **326**
acto de habla, **846**
actor, **601**
actuador, **36**, 43, **929**
actualización (en el razonamiento temporal), 130, 132, 133, 466–468
actualización de Bellman, **573**
actuar racionalmente, 3
acuerdo de conflicto, **641**
ADABOOST, **702**
Adaline, 21
Adams, A., 1016, *1053*
Adams, J., 325
Adams, M., 313, *1046*
Adams, R. P., 672, *1062*
adaptativo, 819
ADASYN (sistema de generación de datos), 707, 995
adelante-atrás, **470**, 744
Adida, B., 339, *1034*
ADL (Lenguaje de descripción de la acción), 380
administrador (de tareas), 633
admisibilidad, 86
ADN, 116
Adolfo, K. E., 801, *1034*
Adorf, H.-M., 384, *1049*
ADP (programación dinámica adaptativa), **793**, 818
aeropuerto, conduciendo a, 385
afinación manual, **671**
AFSM (máquina de estados finitos aumentada), **969**
agarre, 978
Agarwal, P., 967, 979, 990, *1054*, *1061*
agente, **3**, 36, 60
  agente de software, **43**
  aprendizaje activo, 797
  aprendizaje, 56–58, 62
  arquitectura de, **47**, 1018
  autónomos, 210
  basado en el conocimiento, 13, 208–210
  basado en goles, 53–54, 60, 61
  basado en modelos, **52**, 51–53
  basado en servicios públicos, 54–56, 60
  benévolo, 599
  codicioso, **797**
  componentes, 1012–1018
  conducción de taxis, 57, 1019
  función, **36**, 37, 47, 564
  híbrido, **241**
  impacientes, 642
  inteligente, 34
  lógico, 237-246, 279
  personal, **1015**
  planificación en línea, 379
  programa, **37**, **47**, 48, 60
  racional, **4**, 3–4, 36, **39**, 39–40, 55, 60, 61, 557
  reflejo, **49**, 49–51, 60, 564, 790
  resolución de problemas, **63**, 63–66
  situado, 982
  teoría de la decisión, 388, 528
  vacío, 39
  wumpus, 212, 270, 881
Agencia Espacial Europea, 384
agente basado en servicios públicos, 55
AGENTE BASADO EN TABLAS, **48**
agente conjunto, **964**
agente de arquitectura, **47**, 1018
  basado en reglas, 292

cognitivo, **34**, 292
computadora, 652
IA, 1018
para reconocimiento de voz, 25
red, 768, 770, 787, 922
reflectante, **1019**
RNN, 861
subsunción, 969
transformador, 868, 880
agente de aprendizaje pasivo, **791**
AGENTE DE RECOPILACIÓN DE INFORMACIÓN, **551**
agente de software, **43**
AGENTE DE VACÍO REFLEJO, **49**
AGENTE DFS EN LÍNEA, **137**
AGENTE HÍBRIDO-WUMPUS, **242**
agente lógico, 386
agente personal, **1015**
agente probabilístico, 386
agente racional, **4**, 3-4, 36, **39**, 39-40, 55, 60, 61, 557
agente reflejo, **49**, 49–51, 60, 564, 790
AGENTE REFLEJO BASADO EN MODELOS, **53**
agente sin modelos, **56**
AGENTE REFLEJO SIMPLE, **51**
Agente remoto (agente de planificación), 29, 309, 355, 384
agente situado, 982
agentes basados en el conocimiento, **208**
agentes basados en objetivos, 53–54, 60, 61
Agerbeck, C., 205, *1034*
Aggarwal, A., 787, *1059*
Aggarwal, G., 637, *1034*
Agha, G., 646, 1034
AGI (inteligencia artificial general), **32**
Agichtein, E., 855, *1034*
Agmon, S., 785, *1034*
Agostinelli, F., 106, 1034
Agrawal, P., 979, *1034*
Agrawal, R., 1009, *1035*
Agre, P. E., 383, *1034*
agregación segura, **992**
agregación, **376**
agregar lista, **345**
agrupación (en redes neuronales), **762**
agrupación de ROI, 900
agrupamiento (aprendizaje no supervisado), 653, **738**
agrupamiento (en redes bayesianas), **434**, 434–435, 456
aguja en un pajar, 216
Ai, D., 30, *1053*
AI2 ARC (preguntas de prueba de ciencias), 850
AI4People, 1008
Aila, T., 780, *1050*
AISB (Sociedad de Inteligencia Artificial y Simulación del Comportamiento), 35
Aitken, S., 748, *1041*
Aizerman, M., 717, *1034*
ajedrez, 4, 5, 14, 21, 30, 46, 107, 147, 155–158, 176
Ajedrez, D. M., 61, 1050
ajuste insuficiente, **655**
Akametalu, A. K., 821, *1034*
Akgun, B., 979, *1034*
Alami, R., 979, *1062*
Alarma antirrobo, 413–414
Albantakis, L., 1007, *1057*
Alberti, C., 853, *1034*
Alberti, L., 920
Alcalde, A., 1006, *1054*
alcanzados (estados en búsqueda), **72**
Aldous, D., 142, *1034*
ALE (Entorno de aprendizaje arcade), 822
aleatorización de dominios, **960**
aleatorización, 37, **51**
Alegría, B., 1010, *1049*
Alejandría, 15
Alemi, A. A., 312, *1034*
AlexNet (sistema de redes neuronales), 782
álgebra de Robbins, **313**
álgebra lineal, 1025–1026
Algodón, C., 1009, 1037
Algol-58, 852
algoritmo, **9**
algoritmo CYK, **835**, 851
algoritmo de adentro hacia afuera, 840
algoritmo de Boyen-Koller, 499
algoritmo de Davis-Putnam, **233**
algoritmo de Dijkstra, 77, 107
algoritmo de mayoría ponderada aleatoria, **703**
algoritmo de Metrópolis, 142, 457
algoritmo de Monte Carlo, **435**
secuencial, **499**
algoritmo de Rete, **292**
algoritmo de Viterbi, **473, 830**
algoritmo en cualquier momento, **1019**
algoritmo EM, 466, 737–746
estructural, **746**
algoritmo evolutivo, **115**, 141
algoritmo genético, **116**, 115–119, 143–144
ALGORITMO GENÉTICO, **119**
algoritmo húngaro, **517**
algoritmo minimax, 149–150, 174, 175, 611
Algoritmo símplex, 143
Algoritmo SSS*, 175
Alhazen, 920
Alianza (en juegos multijugador), **151**
Alibaba, 1017
Alisado de Witten-Bell, 851
al-Khwarizmi, M., 9
Allais, M., 538, 560, *1034*
Allan, J., 850, *1034*
Allen, B., 383, *1044*
Allen, C., 560, *1040*
Allen, J. F., 324, 340, 383, 384, *1034*
Allen, P., 1010, *1034*
Allen-Zhu, Z., 786, *1034*
Alleva, F., 29, *1066*
almacenamiento en caché, 108, **241**, 507
Almulla, M., 108, *1057*
Alperin Resnick, L., 332, *1037*
ALPHAGO (Programa Go), ix, 19, 27, 30, 176, 177, 816
ALPHAZERO (programa de juego), 30, 172, 174, 177, 179
Alshawi, H., 840, *1062*
Alterman, R., 382, *1034*
Altman, A., 179, *1034*
Altruismo, 387
ALVINN (vehículo autónomo), 967
Amarel, S., 144, 338, *1034*
Amazonas, 29, 1017
ambiente determinista, **45**
ambiente no determinista, 45, 110 NONLIN (sistema de planificación), 380
ambiente semidinámico, **46**
ambigüedad, 252, **847**
ambigüedad, léxico, **847**
ambigüedad, semántica, **847**
ambigüedad, sintáctico, **847**, 853
ambigüedad sintáctica, **847**, 853
amenaza creíble, **619**
Amir, E., 249, 527, *1034*, *1041*
Amit, Y., 718, *1034*
Amodei, D., 15, 879, 1010, 1018, *1034*, *1059*
amor, 982
Amor, a. de J.C., 560, *1049*
Amor, N., 179, *1053*
AMPLITUD-PRIMERA-BÚSQUEDA, **77**
Análisis asintótico, **1024**, 1023–1024
análisis de algoritmos, **1023**
análisis de árbol de fallas (FTA), **1001**
análisis de componentes principales (ACP), 777
análisis de decisiones, **558**
análisis de modos de falla y efectos (AMFE), **1001**
análisis de pedigrí, **455**
análisis de sensibilidad, **552**
análisis de sentimientos, **826**, 863
análisis exploratorio de datos, **653**, 708
análisis probabilístico de componentes principales (PPCA), **776**
análisis sintáctico de reducción de desplazamiento, **838**
análisis sintáctico no supervisado, **840**
análisis sintáctico orientado a datos, 840
análisis sintáctico semisupervisado, **840**
análisis sintáctico, **835**, 835–840
analizador determinista, **838**

analizador sintáctico de Berkeley, 853
analizador sintáctico de gráficos, **835**, 851
analizador sintáctico de Stanford, 853
ANALOGÍA, 20
Anantharaman, T. S., 176, *1048*
Anbulagan, 248, *1052*
ancho del árbol, **202**, 204, 206, 434
anchura del hiperárbol, 207
anchura inducida, 206
Andersen, S. K., 455, 456, *1034*
Anderson, C. R., 383, *1065*
Anderson, J. A., 785, *1047*
Anderson, J. R., 14, 292, 458, *1034*, *1058*
Anderson, K., 108, 822, *1034*, *1036*
Andersson, M., 648, *1060*
Andoni, A., 717, *1034*
Andor, D., 853, *1034*
Andreae, P., 822, *1034*
Andrés, D., 143, 596, 820, 822, *1034*, *1041*, *1051*
Andrés, G., 1009, *1055*
Andrieu, C., 499, *1034*
Andrychowicz, M., 959, 979, *1034*
Aneja, J., 909, *1034*
Anexo procesal, **331**
Angeli, G., 880, *1037*
Animatrónica, **973**
Anisenia, A., 456, *1061*
año de vida ajustado por calidad (AVAC), **534**, 559
Anomalía de Sussman, 380
Antecedentes no informativos, **733**
antecedentes, 217
Antonoglou, I., 27, 30, 155, 174, 177, 178, 784, 790, 820, 822, *1055*, *1061*
anulación, **331**
anuncio de tareas, **633**
Aoki, M., 597, 1034
Apertura, **883**
aplicables, **65**, 345
Aplicaciones, C., 30, 179, *1064*
Appel, K., 204, *1034*
Appelt, D., 854, *1034*
aprender a aprender, 719
APRENDIZ PASIVO DE ADP, **794**
APRENDIZ PASIVO DE TD, **795**
aprendizaje, **41**, 46, 60, 210, 217, 651, 823
  a gran escala, **670**
  aprendizaje, 1003
  árbol de decisiones, 657–661
  basado en instancias, **686**, 686–688
  Bayes ingenuo, 727
  bayesiano, 701, **722**, 722–723, 746
  conjunto, **696**, 696–702
  débil, **700**
  elemento, **56**
  en damas, 19
  en el juego, 815–816
  en el mundo de los bloques, 20
  en el problema del restaurante, 656
  en la búsqueda, 103–104
  en modelos ocultos de Markov, 744
  entrada de ruido, 663–664
  estadísticas, 721–724
  evaluación del rendimiento de, 665–666
  gramática, 854
  heurística, 104
  inductivo, 653–656, 715
  Lista de decisiones, 674-676
  MAPA, 723–724
  máxima verosimilitud, 725–729
  metanivel, **104**
  mezclas de gaussianos, 738-740
  no paramétricos, 686
  online, **703**, 804
  PAC, 673, 717
  para buscar, 103
  parámetro, **724**, 730–732
  profundo, **26**, 698, 750–788
  Q, **790**, 802, 810, 925
  Red bayesiana, 734-735
  red neuronal, 17
  refuerzo, 10, 164, 595, **653**, 789, 789–822, 979 relacional, **820**
  sin supervisión, **653**, 738–740
  tasa de, **678**, 795
  Teoría computacional de, 672
  variables ocultas, 741-743
aprendizaje automático, **2**, 4, 35, 174, **651**, 651–980
aprendizaje basado en explicaciones (EBL), 382
aprendizaje basado en instancias, **686**, 686–688
Aprendizaje bayesiano, 701, **722**, 722–723, 746
aprendizaje curricular, **840**
Aprendizaje de acción-utilidad, **790**
aprendizaje de aprendizaje, **813**, 1003, 1016
aprendizaje de diferencias temporales, 793–797, 818
aprendizaje de extremo a extremo, 960
aprendizaje de la cláusula de conflicto, 234
APRENDIZAJE DE LA LISTA DE DECISIONES, **675**
aprendizaje de restricciones, **196**, 203, **206**
aprendizaje débilmente supervisado, **705**, 1017
aprendizaje en conjunto, **696**, 696–702
aprendizaje online, **703**, 804
aprendizaje en pequeña escala, **670**
aprendizaje federado, 706, **992**, 1009
aprendizaje hebbiano, **17**
aprendizaje inductivo, 653–656, 715
aprendizaje multitarea, **782**
aprendizaje no supervisado, **653**, 738–740
aprendizaje por imitación, **813**, 966
aprendizaje por refuerzo, 10, 164, 595, **653**, **789**, 789–822, 979
  activo, **791**, 797–803
  basado en modelos, **790**
  bayesiano, **800**
  cooperativa, 648
  distribuidas, 646
  en la política, **802**
  fuera de la política, **802**
  generalización en, 803-810
  inverso, **34**
  jerárquico, **807**, 1014
  multiagente, 646
  pasivo, **791**, 791–797
  profundo, **784**, **806**
  relacional, 820
  sin modelo, **790**
aprendizaje por refuerzo bayesiano, **800**
aprendizaje por refuerzo inverso, **813**, 1003, *1014*
aprendizaje por refuerzo jerárquico, **807**, 1014
aprendizaje por refuerzo profundo, 806, 979
aprendizaje por refuerzo sin modelos, **790**
aprendizaje por transferencia, 652, **781**, 871, 960, 1016
Aprendizaje predictivo, **1017**
aprendizaje preferencial, **931**
aprendizaje profundo, **26**, 698, 750–788
  para PNL, 856–880
  para robótica, 958–968
  para la visión, 894–918
aprendizaje semisupervisado, **705**
aprendizaje sin arrepentimiento, 704
aprendizaje supervisado, **653**, 789, 804
aproximación conservadora, **243**, 369
aproximación de campo medio, **458**
aproximación de funciones, **803**
aproximación universal, 752
aproximación variacional, 458
Apt, K. R., 205, 207, *1034*
Apte', C., 852, *1034*
árbol AD, 747
ÁRBOL DE APRENDIZAJE-DECISIÓN, **660**
árbol de búsqueda, 71, **147**
árbol de expansión mínimo (MST), 109
árbol de regresión potenciado por gradiente (GBRT), 701
árbol de unión, **434**
árbol k-d, **688**
árbol, 199
árbol de decisión, **657**, 715
  expresividad, 657
  poda, **663**
árboles de exploración rápida al azar (RRT), 947
árboles extremadamente aleatorios (ExtraTrees), **698**

Arbuthnot, J., 408, *1034*
Archibald, C., 179, *1034*
Arco de persistencia, **489**
Arentoft, M. M., 384, *1034*
Arfaee, S. J., 109, *1034*, *1052*
Argall, B. D., 979, *1034*
argmax, 1026
argumento
  de discapacidad, 982–983
  de la informalidad, 981-982
argumento contradictorio, **136**
Ariely, D., 538, 560, *1034*
ARISTO (sistema de preguntas y respuestas), 875, 876, 880
Aristóteles, ix, 3, 6, 7, 11, 60, 61, 247, 278, 338, 339, 341, 715, 920, 976
Arkin, R., 980, 989, *1034*
arma
  letal autónomo, 1008
arma letal autónoma, 31, 987
Armando, A., 250, *1034*
Armstrong, S., 1002, *1034*
Arnauld, A., 7, 10, 557, *1034*
Arnoud, S., 769, *1046*
Arora, J. S., 119, *1054*
Arora, N. S., 512, 526, *1034*
Arora, S., 107, *1034*
Arous, G. B., 786, *1039*
Arpillera, **121**
Arpit, D., 716, *1035*
arquitectura cognitiva, 34, **292**
arquitectura de subsunción, **969**
arranque, 697
arrepentimiento, **704**
Arrow, K. J., 640, 649, 1035
Arulampalam, M. S., 499, *1035*
Arulkumaran, K., 820, *1035*
Arunachalam, R., 649, *1035*
arXiv.org, 27, 788, 1018
Asada, M., 976, *1050*
ASCENSO DE COLINA, **111**
ascenso de colina, **111**, 141
  estocástico, **113**
  primera opción, **113**
  reinicio aleatorio, **113**
aserción (lógica), **265**
aserción (probabilística), 388
asesor, 19
Ashby, W. R., 16, *1035*
asignación (en un CSP), **181**
asignación de créditos, **807**
asignación de Dirichlet latente, 852
asimetría, 860
Asimov, I., 975, 1007, *1035*
asistencia, **561**
asistencia al conductor, **973**
ASKMSR (sistema de preguntas y respuestas), 850
Asociación de Datos, **514**, 935
Asociación sobre IA, 35, 1008
Aspuru-Guzik, A., 920, *1060*
Astrom, K. J., 144, 597, *1035*
Asunción de Markov, **463**, 497, 773
asunción de mundo abierto, 367
asunción de mundo cerrado, **264**, 297, 338, 367, 502
asunción de nombres únicos, 264, 502
asunción del agente benévolo, **599**
atajo, **103**
Atanasoff, J., 14
ataque de Sybil, **507**
atención (red neuronal), 865, **866**, 880
atención multicéfalo, **869**
Athalye, A., 787, *1039*
Atkeson, C. G., 596, 820, 979, *1035*, *1056*
Atkin, L. R., 107, *1062*
Atlas (robot), ix
átomo, **260**
atribución del autor, **826**
atributo, **59**
AUC (área bajo la curva ROC), **710**
Audi, R., 1007, *1035*
Auer, P., 597, *1035*
Auer, S., 316, 339, *1037*, *1052*
Aumann, R., 647, *1035*
Aumento de conjuntos de datos, **898**
aumento de datos, **707**
aumento de gradiente, 698, **701**
aumento de inteligencia, **14**
aumento, 851
AURA (demostrador de teoremas), 309, 313
Auray, J. P., 559, *1036*
Austerweil, J. L., 821, *1048*
Australia, 181, 182, 193
autoatención, **868**
AUTOCLASS (algoritmo de aprendizaje no supervisado), 748
Autocodificador variacional, 787
autocodificador, **778**
Autómatas, 1006
Automatización de procesos de negocio, **999**
AutoML, 719
Auton, L. D., 249, *1040*
autonomatrónica, **973**
autonomía, **42**, 924, 971
autooclusión, 896
AUV (vehículo submarino autónomo), 927
Auxiliar de oficina, 455
AVAC (año de vida ajustado por calidad), **534**, 559
aversión a la ambigüedad, **539**, 560
aversión al riesgo, **535**
Avispa Spex, 41, 374
Awwal, I., 978, *1061*
Axelrod, R., 647, *1035*
Axioma, **209**, 267
  axioma del efecto, **239**
  axioma del marco, **239**
  axón, 12
  condición previa, 245
  de la probabilidad, 394
  de la teoría de conjuntos, 269
  de la teoría de la utilidad, 531
  de la teoría de números, 268
  de probabilidad, 393, 1027
  del estado sucesor, **240**, 250
  descomponibilidad, 531
  específico del dominio, 316
  exclusión de acción, **245**, 604
  la de Kolmogorov, 393
  mundo wumpus, 270
  Peano, **268**, 278, 289
axioma de exclusión de acciones, **245**, 604
axioma de precondición, 245
axioma del estado sucesor, **240**, 250
axiomas de Kolmogorov, **393**
axiomas de Peano, **268**, 278, 289
azúcar sintáctico, **268**
azul profundo, 176

B

*b** (factor de ramificación), 98 B* búsqueda, 175
Ba, J. L., 786, *1035*
Baader, F., 312, 341, *1035*
Babbage, C., 15, 175
Babbage, C., 15, 175
Babuschkin, I., 30, 179, *1064*
Bacchiani, M., 849, *1039*
Bacchiani, M., 849, *1039*
Bach, F. R., 852, *1048*
Bach, F. R., 852, *1048*
bachiller, 318
Bachmann, L. M., 30, *1053*
Bachmann, P. G. H., 1029, *1035*
backgammon, 164, 178, 804, 815
Backus, J. W., 852, 1035
Baco, F., 204, 207, 410, 456, 524, 559, *1035*
Bacon, D., 992, *1051*
Bacon, F., 6, 31, *1035*
Baeza-Yates, R., 850, *1035*
Bagdasaryan, E., 1009, *1035*
Bagnell, D., 979, *1060*
Bagnell, J. A., 817, 951, 963, 966, 978, 979, *1035*, *1050*, *1059*, *1067*
Bahdanau, D., 880, *1035*
Bahubalendruni, M. R., 107, *1035*
Bai, A., 822, *1035*
Bai, H., 598, *1035*
Bai, Y., 707, 978, 995, *1037*, *1047*
Baidu, 850, 924
Baird, L. C. I., 596, *1065*

Bajcsy, A., 967, *1035*
Baker, B., 959, 979, 1034
Baker, C. L., 821, 1035
Baker, J., 852, 854, 1035
Baker, L., 27, 30, 1061
Bakkes, S., 176, *1039*
Balance detallado, **444**
Balaskas, K., 30, *1053*
Balch, T., 178, *1058*
Baldi, P., 106, 498, *1034*, *1035*
Baldwin, J. M., 118, 143, *1035*
Ball, M., 381, *1065*
Ballard, B. W., 175, *1035*
Ballas, N., 716, *1035*
Baluja, S., 143, 922, *1035*, *1060*
BANANAS (búsqueda de arquitectura de redes neuronales), 787
Bancilhon, F., 311, *1035*
banco de árboles, 839, 851
  Penn, **829**, 839, 877
bandido
  Bernoulli, **584**
  problema, **581**, 597, 798
  superproceso (BSP), **586**
  un solo brazo, **582**
bandido armado, **581**
Banerjee, A., 979, 990, *1039*, *1063*
Bangera, R., 649, *1065*
Banko, M., 26, 316, 339, 719, 850, 854, 855, *1035*, *1043*
Bansal, K., 309, *1035*
Bapna, A., 850, *1039*
Barbero, D., 749, *1035*
Bardo, N., 30, 178, *1056*
Bar-Hillel, Y., 853, *1035*
Barifaijo, E., 372, *1049*
Barnes, P., 995, *1055*
Barr, A., 107, *1035*
Barreiro, J., 29, *1035*
Barreno, M., 1010, *1035*
Barrett, S., 179, *1035*
barrido prioritario, 596, **796**, 820
Barry, M., 455, *1048*
Bar-Shalom, Y., 61, 497, 526, *1035*
Barták, R., 205, 207, *1035*
Bartholdi, J. J., 649, *1035*
Bartlett, F., 13
Bartlett, P., 821, *1035*
Barto, A. G., 145, 597, 598, 820-822, *1035*, *1063*
Bartolomé, A., 525, *1049*
Bartunov, S., 179, 822, *1064*
Barwise, J., 279, *1035*
basado en datos, **231**
basado en modelos
  agentes reflejos, 60
  aprendizaje por refuerzo, **790**, 959
  visión, 881
basado en prólogos, 294
Base de datos, 59, 264
base de datos de patrones, **101**, 109, 356
base de datos deductiva, **292**, 310, 311
Base Herbrand, **304**
Basturk, B., 142, *1050*
Basye, K., 977, *1041*
Bates, E., 854, *1043*
Bates, M. A., 14, 175, *1064*
Batra, D., 822, 910, *1046*, *1060*
Bauer, G., 313, *1046*
Baum, E., 116, 175, *1035*
Baum, L. E., 497, 748, *1035*
Baumert, L., 205, *1046*
Baxter, J., 821, *1035*
Bayardo, R. J., 206, 207, 248, 1009, *1035*
Baydin, A. G., 457, 527, *1052*
Bayen, A. M., 498, *1048*
Bayerl, S., 312, *1052*
Bayes, T., 399, 410, 747, *1035*
Bayes ingenuo, 402, 408, 410, 727, 741, 742, 746, 824
Bayesiano, 409
BDD (diagrama de decisión binario), 382
Beal, J., 32, *1036*
Beame, P., 456, *1060*
Beardon, A. F., 558, *1036*
Beattie, C., 30, 179, 820, 822, *1036*, *1049*, *1055*
Beber, G., 1006, *1043*
Bechhofer, R., 597, *1036*
Beck, J. C., 206, *1036*
Beckert, B., 312, *1036*
Beeri, C., 206, *1036*
Beetz, M., 525, 1049
Bekey, G., 980, *1036*
Belkin, M., 716, *1036*
Bell, C., 359, 383, *1036*
Bell, D. A., 747, *1039*
Bell, J. L., 279, *1036*
Bell, T. C., 851, 852, *1066*
Bellamy, E., *1036*
Bellamy, R. K. E., 996, 1009, *1036*
BELLE (programa de ajedrez), 176
Bellemare, M. G., 820-822, *1036*, *1044*, *1055*, *1056*
Bellman, R. E., 10, 106, 107, 176, 177, 568, 596, 717, *1036*
Bendix, P. B., 312, *1050*
Bengio, E., 716, *1035*
Bengio, S., 30, 716, 787, 879, *1036*, *1064*, *1067*
Bengio, Y., 17, 716, 718, 719, 786-788, 849, 878, 880, 1016, *1035*, *1036*, *1041*, *1045*, *1046*, *1052*, *1064*, *1067*
Benjamín, M., 989, *1036*
Bennett, B., 342, *1040*
Bennett, F. H., 143, *1051*
Bennett, J., 313, *1046*
Bennett, K., 719, *1046*
Ben-Tal, A., 143, *1036*
Bentham, J., 8, 558, *1036*
Benzmüller, C., 313, *1036*
Beresniak, A., 559, *1036*
Berg, A. C., 786, *1060*
Berger, H., 11
Berger, J. O., 457, 748, *1036*, *1057*
Berges, V., 822, *1049*
Bergstra, J., 719, *1036*
Berk, R., 1009, *1036*
Berkeley, 678
Berkson, J., 455, *1036*
Berleur, J., 1008, *1036*
Berlín, K., 717, *1036*
Berliner, H. J., 175, 178, *1036*
Bermúdez-Chacón, R., 719, *1036*
Bernardo, J. M., 731, *1036*
Berners-Lee, T., 339, *1036*
Bernoulli, D., 7, 10, 535, 557, *1036*
Bernoulli, J., 8, 409
Bernstein, M., 786, *1060*
Bernstein, P. L., 411, *1036*
Berrada, L., 716, *1036*
Berrou, C., 458, *1036*
Berry, C., 14
Berry, D. A., 597, *1036*
BERT (sistema de lenguaje natural), 879, 1021
Bertele, U., 456, *1036*
Bertoli, P., 382, 383, *1036*
Bertot, Y., 312, *1036*
Bertsekas, D., 61, 410, 598, 822, 1029, *1036*
Bertsimas, D., 70, 716, *1036*
Berzuini, C., 499, *1045*
Beschastnikh, I., 1009, *1044*
Bessen, J., 1000, *1036*
Bessiére, C., 205, *1036*
Best, N., 525, 747, *1054*
Bestia de Hopkins, 976
Betancourt, M., 458, 527, 747, *1039*
Bethge, M., 983, *1045*
Betlem, H., 372, *1049*
Betlem, J., 372, *1049*
Betteridge, J., 850, *1055*
Beutel, A., 311, 1009, *1036*, *1051*
bfloat16, 15
Bhar, R., 498, *1036*
Bharath, A. A., 820, *1035*
Bhattacharya, P., 498, *1066*
Bibel, W., 312, 313, *1036*, *1052*
BIBF-SEARCH, **83**
Biblia, 832
bicicleta, 1010
Bickford, M., 309, *1062*
bicondicional, **217**
Bidlack, C., 976, *1040*
bien calibrado, **993**
Bien, J., 719, *1036*
bienestar social igualitario, **610**

bienestar social utilitario, 609
bienestar social, **609**
Biere, A., 249, *1036*
Bies, A., 852, *1036*
bifurcación,aridad **108**, 377, 597
Big Data, **26**, 922, 1015
BigDog, 29
Bigelow, J., 16, *1059*
Bigham, J., 854, *1057*
Gates, B., 33
billar, 179
Billings, D., 647, *1036*
Billingsley, P., 411, *1036*
Bilmes, J., 498, *1053*
Bimbo, J., 920, *1054*
binario de restricción, 184
Binder, J., 497, 498, 748, *1036*, *1060*
Bingham, E., 526, *1036*
Binmore, K., 647, *1036*
bioinformática, 852
Biran, O., 1009, *1037*
Birattari, M., 142, *1042*
Birbeck, M., 339, *1034*
Bischof, J., 1009, *1036*
Bishop, C. M., 142, 455, 717, 720, 748, 787, *1037*, *1064*
Bishop, M., 1007, *1058*
Bishop, R. H., 61, *1042*
Bisson, T., 985, *1037*
Bistarelli, S., 204, *1037*
Biswal, B. B., 107, *1035*
Bitner, J. R., 205, *1037*
Bizer, C., 316, 339, *1037*, *1052*
Bjerager, P., 457, *1050*
Bjornsson, Y., 177, *1060*
BKG (programa de backgammon), 178
Black, M., 907, *1050*
BLACKBOX (sistema de planificación), 381
Blake, A., 498, *1049*
Blanco, C., 787, *1065*
Blanco, J. L., 309, *1047*
Blanco, J., 720, *1040*
Blankespoor, K., 29, *1059*
Blau, H. M., 30, *1043*
Blazewicz, J., 384, *1037*
Blei, D. M., 526, 852, *1037*, *1048*, *1064*
Bliss, C. I., 455, *1037*
Block, H. D., 21, *1037*
Block, N., 1007, *1037*
BLOG (lenguaje de programación probabilístico), 526
bloque de decisión, 700
Blondel, M., 720, *1058*
Blum, A. L., 381, 704, 718, 720, 855, *1037*
Blum, C., 142, *1042*
Blumer, A., 717, *1037*
BNF (Forma Backus-Naur), **1030**
BO (optimalidad acotada), **1020**
Bobick, A., 498, *1049*
Bobrow, D. G., 20, *1037*
Bod, R., 840, *1037*
Boddington, P., 1008, *1037*
Boddy, M., 144, 382, 1019, *1041*, *1046*
Boden, M. A., 1007, *1037*
Bojanowski, P., 852, *1049*
Bokeh (software de análisis de datos), 709
Bolognesi, A., 176, *1037*
Bolton, A., 27, 30, 822, *1036*, *1061*
Bolton, R. J., 990, 1037
Boltzmann, **814**
  individual, 627
  limitado, **4**, 328
  perfecto, 4
Bonawitz, K., 526, 992, *1037*, *1046*
Bond, A. H., 646, *1037*
Boneh, D., 116, *1035*
Bonet, B., 144, 380-383, 598, *1037*, *1047*
Bongard, J., 1006, *1058*
bono de exploración, **585**
  función, **799**, 802
  caja fuerte, **136**
Boole, G., 8, 247, 409, *1037*
Booth, T. L., 852, *1037*
borde (en una escena), **889**
Bordes, A., 786, *1045*
Borel, E., 647, *1037*
Borenstein, J., 977, 978, *1037*
Borgida, A., 332, *1037*
Borgstro¨m, J., 526, 527, *1040*, *1046*
Borochowitz, Z. U., 456, *1061*
Boroditsky, L., 253, *1037*
Borrajo, D., 381, *1064*
Boser, B., 718, 922, *1037*, *1052*
bosque aleatorio (aprendizaje automático software), 698
bosque aleatorio, **697**, 718
BOSS (vehículo autónomo), 972, 973, 977
Bosse, M., 977, *1037*
Boston Dynamics, ix Bostrom, N., 33, 1010, *1037*
Botea, A., 381, *1047*
Bottou, L., 26, 717, 786, 922, *1037*, *1052*
Boue', L., 716, *1037*
Bousmalis, K., 978, *1037*
Bousquet, O., 717, 786, *1037*
Boutilier, C., 455, 559, 561, 597, 646, *1037*
Bouzy, B., 177, *1037*
Bowden, B. V., 14, 175, *1064*
Bower, G. H., 819, *1047*
Bowling, M., 30, 178, 647, 648, 822, *1036*, *1037*, *1056*, *1065*, *1067*
Bowman, D., 985
Bowman, S. R., 880, *1065*
Bowman, S., 877, 879, 880, *1037*, *1046*, *1065*
Bowyer, K. W., 707, 995, *1039*
BOXES (algoritmo de aprendizaje por refuerzo), 816
Boyan, J. A., 142, *1037*
Boyce, M., 29, *1035*
Boyd, D., 1009, *1039*
Boyd, S., 143, *1037*
Boyden, E., 11, *1047*
Boyen, X., 499, *1037*
Boyer, R. S., 309, 312, 313, *1037*
Boyko, A. S., 30, *1053*
Boyle, J., 313, *1066*
Brachman, R. J., 332, 341, 343, *1037*, *1052*
Bradlow, H., 978, *1061*
Bradt, R. N., 581, 597, *1037*
Bradtke, S. J., 145, 597, 820, *1035*
Brady, J. M., 498, *1056*
Brafman, O., 560, *1037*
Brafman, R. I., 382, 383, 559, 646, 820, *1037*, *1048*
Brafman, R., 560, *1037*
Brahmagupta, 204
Braitenberg, V., 979, *1037*
Brakel, P., 849, *1067*
Brandenburger, A., 647, *1035*
Brandt, F., 649, *1037*
Brants, T., 852, 880, *1037*, *1044*
Bratko, I., 109, 312, *1037*
Bratman, M. E., 61, *1038*
Braverman, E., 717, *1034*
Breck, E., 713, 719, *1038*
Breese, J. S., 62, 455, 525, 527, 559, 1019, *1038*, *1048*, *1065*
Breiman, L., 698, 716, 718, *1038*
Brelaz, D., 205, *1038*
Brendel, W., 787, *1039*
Brent, R. P., 142, *1038*
Bresnan, J., 853, *1038*
Breuel, T., 879, *1053*
Brevdo, E., 526, *1064*
Brewka, G., 342, *1038*
Brickley, D., 339, 1038
bridge (juego de cartas), 178
BRIDGE BARON, 178
Briggs, R., 338, *1038*
Brill, E., 26, 719, 850, 852, 854, *1035*, *1038*
brillo, 886
Brin, D., 855, *1038*
Brin, S., 854, *1038*
Bringsjord, S., 1006, *1038*
Brioschi, F., 456, *1036*
Broadbent, D. E., 14, *1038*
Broadhead, M., 850, 855, *1035*
Broca, P., 11
Brock, B., 312, *1048*
Brockman, G., 822, *1038*
Brokowski, M., 144, *1065*
Brooks, M. J., 923, *1048*
Brooks, R. A., 27, 61, 249, 383, 969, 977, 979, 1006, 1010, *1038*, *1058*, *1063*
Brooks, S., 458, *1038*
Brouwer, P. S., 820, *1035*
Brown, C., 207, *1038*

Brown, E., 30, *1043*
Brown, J. S., 342, *1041*
Brown, K. C., 559, *1038*
Brown, M., 498, *1051*
Brown, N., 30, 178, 648, 790, *1038*
Brown, P. F., 878, 880, *1038*
Browne, C., 176, *1038*
Browning, B., 979, *1034*
Brownston, L., 311, *1038*
Brügmann, B., 177, 597, *1038*
Brubaker, M., 458, 527, 747, *1039*
Bruce, V., 924, *1038*
Bruna, J., 787, *1063*
Brundage, M., 820, *1035*
Brunelleschi, F., 920
Brunnstein, K., 1008, *1036*
Brunot, A., 26, 786, 922, *1052*
Brunskill, E., 598, 821, *1059*, *1063*
Bruynseels, A., 30, *1053*
Bryce, D., 144, 381, 383, *1038*
Brynjolfsson, E., 27, 998, 1005, 1011, *1038*, *1063*
Bryson, A. E., 22, 785, *1038*
Bryson, J. J., 997, 1007, *1038*
BSP (superproceso de bandidos), 586
Buchanan, B. G., 22, 23, 62, 338, 459, *1038*, *1043*, *1053*
Buck, C., 852, *1038*
Budden, D., 822, *1063*
Buehler, M., 977, *1038*
Bueno, I. J., 177, 409, 454, 455, 1004, *1046*
Buffet, O., 598, *1061*
Buffon, G., 457, *1038*
BUGS (sistema de razonamiento probabilístico), 458, 525
BÚHO (lógica de descripción), 339
Bui, P., 30, *1053*
BUILD (sistema de planificación), 342
Bulatov, Y., 769, *1046*
Bulfin, R., 649, *1059*
Bulitko, V., 145, *1063*
Bunt, H. C., 340, *1038*
Buolamwini, J., 995, *1038*
Burch, N., 30, 177, 178, 647, *1036*, *1037*, *1056*, *1060*
Burgard, W., 61, 526, 977, 978, 980, *1038*, *1039*, *1044*, *1061*, *1064*
Burget, L., 878, 879, *1055*
Burkov, A., 720, *1038*
Burns, C., 455, *1056*
Burns, E., 108, *1038*
Buro, M., 159, 178, *1038*
Burstein, J., 338, *1038*
Burton, R., 560, *1038*
Busbee, T. A., 1013, *1051*
búsqueda, 22, 53
  a* delimitado por la memoria, **95**, 95, 109
  a*, 85–90
  alfa-beta, 152-155, 174, 175
  algoritmo, **71**
  amplitud, **76**, 76–77, 105, 358
  aprender a, 103
  árbol, 71, **147**
  b*, 175
  best-first, **73**, 105
  bidireccional, **82**, 96–97, 109
  codicioso el mejor primero, **85**, 85
  codicioso, **112**
  conforme, 126–130
  corte, 158–160
  costo de uniforme, **77**, 77–78, 105
  de profundidad limitada, **80**, 80
  delimitado por la memoria, 92–95, 108
  desinformado, 63, 76–84, 105, 107
  en tiempo real, 145, 156–160
  en un csp, 191-199
  ascenso, 111-114, 138
  espacio continuo, 119-122, 142
  estado de creencia incremental, **129**
  estocástico, **115**
  haz estocástico, **115**
  haz local, 115
  heurística, 107
  informados, 63, **84**, 84–105
  local, para csp, 197–199
  locales, 110–114, **115**-119, 142, 206, 235–236, 247, 248
  minimax, **148**, 148–152
  no determinista, 122–125
  online, **134**, 134–141, 144–145
  parcialmente observable, 126–134
  política, **790**, **810**, 810–812
  ponderado a*, 91
  problema, 942
  profundidad-primero, **78**, 78–80, 105, 358
  profundización iterativa a*, 108
  profundización iterativa, **80**, 80–82, 105, 107, 155, 158, 358
  quietud, **158**
  recocido simulado, 114–115
  recursivo best-first (rbfs), 108
  recursivo best-first, 93–95
  retroceso, 80, 195–197, 199, 203
  sistemático, **75**
  tabú, **142**, 198
  viga, **92**, 106, **115**, 159, 831, 836, 868
búsqueda aleatoria, **672**
búsqueda alfa-beta, 152–155, 174, 175
BÚSQUEDA ALFA-BETA, **154**
BÚSQUEDA ANGÉLICA, **364**
búsqueda arbórea, **74**
Búsqueda avanzada de la planificación, 348-350
búsqueda bidireccional, **82**, 96–97, 109
búsqueda contradictoria, **146**
búsqueda de amplitud, **76**, 76–77, 105, 358
Búsqueda de árboles de Monte Carlo (MCTS), **161**
búsqueda de arquitectura neuronal (NAS), **770**, 787
búsqueda de costo acotado, **92**
búsqueda de costos ilimitados, **92**
BÚSQUEDA DE COSTOS UNIFORMES, **77**
búsqueda de costos uniformes, **77**, 77–78, 105 teoría de la convergencia uniforme, 717
búsqueda de grafos, **74**
búsqueda de grueso a fino, **108**
búsqueda de haz estocástico, **115**
búsqueda de haz local, **115**
búsqueda de líneas, **121**
búsqueda de lo mejor, **73**, 105
búsqueda de Monte Carlo pura, **161**
búsqueda de profundización iterativa, **80**, 80–82, 105, 107, 155, 158, 358
búsqueda de rayos, **92**, 106, **115**, 159, 831, 836, 868
búsqueda de regresión, **350**
búsqueda de retroceso, 80, 195–197, 199, 203
búsqueda de riesgos, **535**
búsqueda de rutas, 69
búsqueda de tablas, **687**
búsqueda del tesoro, **551**
búsqueda desinformada, 63, 76–84, 105, 107
búsqueda en retícula, **671**
búsqueda online, **134**, 134–141, 144–145
búsqueda en profundidad, **78**, 78–80, 105, 358
búsqueda offline, **134**
búsqueda hacia atrás para la planificación, 350-351
búsqueda incremental, **141**
búsqueda informada, 63, **84**, 84–105
BÚSQUEDA ITERATIVA-PROFUNDIZADORA, **81**
BÚSQUEDA JERÁRQUICA, **359**
BÚSQUEDA LIMITADA EN PROFUNDIDAD, **81**
búsqueda local, 110–119, 142, 206, 235–236, 247, 248
búsqueda local estocástica, 141
búsqueda minimax, **148**, 148–152
búsqueda ponderada A*, **91**
búsqueda rápida, **92**
búsqueda recursiva de mejor prioridad (RBFS), 93–95, 105
BÚSQUEDA RETROSPECTIVA, **192**
búsqueda sistemática, **75**
búsqueda subóptima acotada, **92**
búsqueda tabú, **142**, 198
Buss, D. M., 560, *1038*
Butler, S., 33, 1010, *1038*

Bylander, T., 384, *1038*
Byrd, R. H., 717, *1038*

C

c (costo de acción), 65
Čapek, K., 975, 1001
Cèrnocky', J., 878, 879, *1055*
C4 (Corpus Colosal Limpio Rastreado), 877, 879
C4.5 (algoritmo de aprendizaje del árbol de decisión), 715
Caballero, B., 21, *1037*
cabeza (de la cláusula de Horn), **230**
cabeza (en PNL), **841**
Cabral, J., 339, *1054*
cadena (en lógica), **341**
Cadena de Markov, **442**, 463, 826
Cadena Markov Monte Carlo (MCMC), **441**, 441–448, 454, 457, 491, 527, 732
  deteriorados, **499**
Cafarella, M. J., 850, 855, 1035, 1038, *1043*
CAFFE (software de aprendizaje automático), *1021*
Cai, S., 598, 713, 719, *1035*, *1038*
Caín, A., 822, *1036*
caja de anclaje, 900
Caja, G. E. P., 143, 497, 747, 787, *1037*
Cajal, S., 11
Cakmak, M., 967, 979, *1034*, *1061*, *1063*
Calcícola, G., 648, *1039*
cálculo, 120, 678, 754
cálculo de eventos, **323**, 322–324, 340, 846
cálculo de situación, **352**
cálculo de variaciones, 142
cálculo hedónico, **558**
callejón sin salida, **135**
Calo, R., 27, *1063*
Calvanese, D., 341, *1035*, *1038*
Camacho, R., 821, *1038*
cámara 13
  estéreo, 927
  para robots, 927
  tiempo de vuelo, **928**
  vigilancia, **990**
cámara de tiempo de vuelo, **928**
cámara estenopeica, **883**, 882–884
Cambefort, Y., 62, *1047*
cambio de nombre, **287**
Cambridge, 13
camino crítico, **376**
Campbell, D. E., 649, *1038*
Campbell, M. S., 176, *1038*, *1048*
Campbell, W., 559, *1039*
campo de distancia firmado, **949**
campo receptivo, **762**
campos de generalización, 991
canal (en redes neuronales), **764**
Candeal, J. C., 558, *1036*
Cannings, C., 456, *1038*
Canny, J., 921, 978, 979, *1038*
Cant, M., 822, *1036*
Cantor, C. R., 456, *1067*
Cantú-Paz, E., 143, *1058*
Cao, Y., 29, 783, 850, 865, *1066*
capa, **148**
capa (en redes neuronales), **750**
  densidad de la mezcla, **759**
  entrada, 756
  oculto, **754**
  salida, **754**
capacidad de pedido, **530**
Capen, E., 559, *1039*
característica (de un estado), **104**, 156
Carbone, R., 250, *1034*
Carbonell, J. G., 382, 879, *1039*, *1066*
Carbonnel, C., 207, *1039*
Cardano, G., 8, 178, 408, *1039*
Carlin, J. B., 748, *1045*
Carlini, N., 787, *1039*
Carlson, A., 253, 850, *1055*
CARMEL (robot móvil), 976
Carnap, R., 7, 394, 409, 410, *1039*
Carpenter, B., 458, 527, 747, *1039*
Carpenter, M., 384, *1041*
CARRITO, **665**, 716
Carroll, S., 143, *1039*
Carson, D., 312, *1066*
CARTAS-GENERADAS, **519**
cartera, **379**
Casas, D. d. L., 822, *1063*
Casati, R., 340, *1039*
Cassandra, A. R., 596, 597, *1039*, *1049*, *1053*
Cassandras, C. G., 61, *1039*
caso
  acusativo, 841
  dativo, 841
  nominativo, 841
  objetivo, 841
  subjetivo, 841
  típico, 320
Castaneda, A. G., 30, 179, *1049*
Casteran, P., 312, *1036*
Castro, R., 455, *1039*
categoría léxica, 829
categoría sintáctica, **833**
Categoría, **317**, 317–322, 329
Cauchy, A., 717, *1039*
Causalidad, 219, 401
Cawley, G. C., 719, *1046*
Cazenave, T., 177, *1037*
Ceder, G., 872, *1064*
celda de memoria (en LSTM), **775**
celosía de subsunción, 285
centavos a juego, **609**
centro (en el diseño del mecanismo), **632**
Centro de Inteligencia Artificial Compatible con el Ser Humano, 1008
Centro de Tecnología Humana, 1008, 1015
Centro para el Estudio del Riesgo Existencial, 1008
cerebro, 17, 750
  humano, 11
  imágenes, **2**
  potencia computacional, 13
  súper electrónico, 9
Cerf, V., 1011, *1057*
ceros y cruces, 147, 175
certeza equivalente, **535**
Certificación, **996**
Cesa-Bianchi, N., 597, 718, *1035*, *1039*
Cesión parcial, **181**
C-espacio, **939**
CGP (Conformant Graphplan), 382
CHAFF (sistema de razonamiento lógico), 248
Chajewska, U., 561, *1039*
Chakrabarti, P. P., 109, 145, *1039*, *1041*
Chalmers, D. J., 1007, *1039*
Chambers, R. A., 816, *1055*
Chandola, V., 990, *1039*
Chandra, A. K., 311, *1039*
Chang, C.-L., 313, *1039*
Chang, H. S., 597, *1039*
Chang, K. C., 457, *1044*
Chang, K.-M., 253, *1055*
Chang, M.-W., 879, *1042*
Chao, W.-L., 877, *1039*
Chapman, D., 380, 383, *1034*, *1039*
Chapman, N., 106
Charniak, E., 23, 311, 455, 497, 525, 788, 852, 853, *1039*
Chaslot, G., 176, *1039*
chatbot, 984
Chater, N., 560, *1039*
Chatfield, C., 497, *1039*
Chatila, R., 977, *1056*
Chauvin, Y., 498, *1035*
Chavira, M., 456, *1039*
Chawla, N. V., 707, 995, *1039*
Cheeseman, P., 206, 454, 455, 748, 977, *1039*, *1062*
Chekaluk, R., 977, *1041*
Chen, D., 15, 853, 876, 879, 1018, *1039*, *1053*, *1066*
Chen, J., 96, 108, 526, 1009, *1036*, *1039*, *1043*
Chen, K., 858, 878, *1055*
Chen, M. X., 850, *1039*
Chen, R., 498, *1053*
Chen, S. F., 852, *1039*
Chen, T., 718, *1039*
Chen, W., 853, *1067*
Chen, X., 381, 822, *1044*, *1064*

Chen, Y., 27, 30, *1061*
Chen, Z., 29, 783, 849, 850, 865, *1039*, *1066*
Cheng, J., 457, 747, *1039*
Cheng, J.-F., 458, *1055*
Cheng, Y., 15, 1018, *1066*
Chernova, S., 979, *1034*
Chervonenkis, A. Y., 717, *1064*
Cheung, V., 822, *1038*
Chi, E. H., 311, 1009, *1036*, *1051*
Chickering, D. M., 175, 747, *1047*, *1051*
Chien, S., 383, *1044*
Chin, C.-S., 717, *1036*
CHINOOK (programa de damas), 177
Chintala, S., 777, 1018, *1059*, *1064*
Chiu, C., 849, *1039*
Chklovski, T., 316, *1039*
Cho, K., 786, 880, *1035*, *1041*
Chociej, M., 959, 979, *1034*
Choi, D. H., 30, 179, *1064*
Chollet, F., 312, 720, 1007, *1034*, *1039*
Chomsky, C., 853, *1046*
Chomsky, N., 14, 16, 851, 852, 854, 855, *1039*
Choromanska, A., 786, *1039*
Chorowski, J., 849, *1039*
Choset, H., 61, 978, 980, *1039*
Chouldechova, A., 1009, *1039*
Christian, B., 1006, *1039*
Christiano, P., 1010, *1034*
Christin, A., 1009, *1039*
Chrpa, L., 381, *1064*
Chu-Carroll, J., 30, *1043*
CHUGIN, 455
Chung, J., 30, 179, *1064*
Chung, K. L., *1029*, *1039*
Chung, S., 249, *1065*
Churchland, P. M., 1007, *1039*
Ciancarini, P., 61, 176, *1037*, *1040*
cibernética, **16**, 15–16
ciberseguridad, 32, **990**
Ciccolini, J., 1009, *1043*
ciclo de costo neto negativo, 65
ciclos, **74**
Ciencia Cognitiva, **3**, 526
Ciencia de datos, 698, 699, 702, 720, 1015
ciencia de los materiales, 872
ciencias de la limpieza, 39
cierre de dominio, **264**, 502
cifrado de clave pública, 309
Cimatti, A., 381-383, *1036*, *1040*
Cinemática directa, **940**
cinemática inversa, **940**
circuito, 70, 143, 271
  booleano, 49, 52, 751
  dominio, 273–277
  integrado, 110
  verificación, 248, **276**
circuito cerrado, **64**, 951
Círculo de Viena, 7
circunscripción, **333**, 338, 341
  priorizados, **334**
circunvolución, **890**
CiteSeer, 511, 512
Claessen, K., 313, *1063*
Clamp, S. E., 410, *1041*
Clapp, R., 559, *1039*
Clapp, T., 499, *1035*
Claret, G., 527, *1040*
Clarividencia, 171
Clark, A., 982, 1006, *1040*
Clark, C., 879, *1058*
Clark, K. L., 342, *1040*
Clark, P., 850, 876, 880, *1040*
Clark, S., 853, 977, *1040*, *1042*
Clarke, A. C., 454, *1040*
Clarke, E., 381, *1040*
Clase abierta, **835**
Clase cerrada, **835**
Clase de referencia, **409**
clases desequilibradas, **707**
CLÁSICO (lógica descriptiva), 332
clasificación (en aprendizaje), **652**
clasificación (en lógica descriptiva), 331
clasificación booleana, 657
clasificación de textos, **403**, 862
clasificador bayesiano, 402
cláusula de cuerno, **229**
cláusula definida, **229**, 286–287
cláusula unitaria, **226**, 233, 308
cláusula, **226**
Clearwater, S. H., 649, *1040*
Clerc, M., 142, *1042*
Cleven, R., 526, *1055*
clonación conductual, 821, **966**
CLP (programación lógica de restricción), **205**, **298**
CLP(R) (sistema de programación lógica de restricción), 312
Club de la Relación, 16
Clutter (en la asociación de datos), **516**
CNF (forma normal conjuntiva), **226**, 226–227, 247, 299–300
CNLP (planificación no lineal condicional), 383
CNN (red neuronal convolucional), **760**, 896
Coalición, **626**
Coase, R. H., 649, *1040*
Coates, A., 817, 821, 1004, *1040*
Coates, M., 455, *1039*
Cobham, A., 9, *1040*
Cocke, A., 835
Cocke, J., 880, *1038*
COCO (conjunto de datos de imágenes), 25, 781, 909
codificación one-hot, **707**, 757, 856
codificador (en autocodificadores), 778
codificador de transformador, **871**
códigos de ética, 1008
coeficiente de Gini, **610**
coerción, 366
coformación, 855
cognición encarnada, **982**
Cohen, B., 249, *1061*
Cohen, C., 976, *1040*
Cohen, P. R., 24, 646, *1040*
Cohen, W., 850, *1055*
Coherencia de la trayectoria, **188**, 204
coherente con los límites, **189**
Cohn, A. G., 342, *1040*
coincidencia de características, **814**
coincidencia de patrones, **290**
cola, **74**
  FIFO, **74**
  LIFO, **74**
cola de prioridades, **74**
cola larga, **712**
COLBERT (lenguaje de control de robots), colaboración 980, 961, 964
Collin, Z., 207, *1040*
Collins, M., 718, 845, 853, 854, *1034*, *1040*, *1067*
Colmerauer, A., 278, 311, *1040*
Colocación directa, 955
Colombano, S. P., 143, *1053*
Colonos de Catán, 151
color, 888
Coloso, 14
Colton, S., 176, *1038*
colusión, **634**
Comentarios, 15, 651, **653**
Compagna, L., 250, *1034*
COMPAS (sistema experto), 993, 1009
Compatible con incentivos, 635
complejidad, 1023-1025
  análisis, **1024**
  espacio, **75**, 105
  muestra, **674**
complejidad algorítmica, 716
complejidad de Kolmogorov, **716**
complejidad de los datos, **290**
complejidad del tiempo, **75**, 105
completando (de una base de datos), **297**
completando el cuadrado, **481**
completitud de la refutación, 303
completo, **393**, 407, 412, 414–418
componente (de la distribución de la mezcla), **739**
componente conectado, **199**
composicionalidad, **251**
comprobación de avances, **194**, 194
comprobación de modelos, **215**, 247
comprobador de colisiones, **946**
compromiso epistemológico, **255**, 277, 386
compromiso ontológico, **254**, 277, 386
compruébelo, **284**, 295
compuerta de salida (en LSTM), **775**

computabilidad, **9**
computación cuántica, **15**
comunicación, 605, 823
concepto de solución, **606**
concesión, **644**
conciencia, 11, **985**, 1007
conclusión (de una implicación), **217**
Concurso de Cournot, **622**
Concurso de Imágenes Inclusivas, 995
conteo sustantivo, **322**
condensación, 498
condición previa, **345**
  desaparecidos, **372**
condición de terminación, 957
condición monótona, 107
Condon, J. H., 176, 1040
conductismo, **13**, 16
conectivo lógico, 17, 217, 246, 260
conexionismo, **24**, 785
Conferencia Internacional Conjunta sobre IA (IJCAI), 35, 109
Congdon, C. B., 976, *1040*
Conitzer, V., 649, *1037*
conjetura del umbral de satisfacción, **236**
conjugado a priori, **731**
conjunción (lógica), **217**
conjunción, 217
conjunto (en lógica de primer orden), **269**
conjunto alcanzable, **361**
conjunto de conflictos, **195**
conjunto de información, **620**
conjunto de negociación, **641**
conjunto de sensores de alcance, 933
conjunto de sensores, **933**
conjunto de validación, **666**
conjunto de Vitali, 393
conjunto difuso, 459
conjuntos disjuntos, **318**
Conlisk, J., 560, *1040*
conmutatividad (en problemas de búsqueda), **191**
Conneau, A., 879, *1052*
Connell, J., 980, *1040*
conocimiento
  adquisición de conocimientos, 23, **272**
  antecedentes, **209**, 302
  antes, 40, 41, **652**
  basado en modelos, 400
  base (kb), **209**, 246
  diagnóstico, 400
  ingeniería, **271**, 271–277, 415
  nivel, **210**, 247
  sentido común, 19
  y acción, 7, 326–328
conocimientos previos, 40, 41, **209**, 302, **652**
co-NP, **1025**
co-NP-completo, 222, **1025**
consecuencialismo, 8, **39**
consecuente, 217
consistencia, **331**
  arco, **186**
  condición, 107
  de una asignación de csp, **181**
  de una heurística, **88**
  ruta, **188**, 204
consistencia de arco, **186**
consistencia de arco direccional, **199**
consistencia de nodos, 186
consistencia K, **188**
consistencia local, **186**
Consorcio de Ontología Genética, The., 340, *1045*
constante de Skolem, **281**, 310
consulta (lógica), 266
consulta (probabilística), **395**
consultas agregadas, **991**
conteo virtual, **731**
contexto (en Visión por computadora), **899**
contexto, 860
Conti, E., 822, *1045*
Conti-Cook, C., 1009, *1043*
continuidad (de preferencias), **530**
contorno (de un espacio de estados), **89**
contradicción, **223**
contrapartes, **600**
contribución marginal neta, **630**
contribución marginal, **628**
control, **931**
  bang-bang, **816**
  lateral, 917
  longitudinal, 917
control de par computarizado, **954**
control de seguimiento de trayectorias, **939**, 978
control difuso, 459
control predictivo de modelos (mpc), **956**, 964
controlador P, **952**
controlador PD, **953**
controlador PID, **953**
controlador, 4, 61, 782, 817, 952
convención de cortesía, **985**
convenio, **605**
conversión a la forma normal, 299-300
convexidad, 122
convexo, **122**, 141
CONVINCE (sistema experto bayesiano), 454
convolución (en redes neuronales), **760**
Conway, D., 720, *1040*
Cook, P. J., 1000, *1044*
Cook, S. A., 9, 248, 249, 1029, *1040*
Cooper, G., 457, 747, *1040*
Cooper, M. C., 207, *1039*
cooperación, 604–605
cooperativa, 626
Copeland, J., 340, 1007, *1040*
Copeland, T. P., 30, *1042*
COQ (demostrador de teoremas), 204, 312
Coram, M., 30, *1046*
Corbett-Davies, S., 1009, *1040*
Corkill, D. D., 646, *1052*
Cormen, T. H., 107, 1029, *1040*
Cornell, C. A., 457, *1050*
Corporación de Equipos Digitales (DEC), 23, 292
Corpus Rastreado Limpio Colosal (C4), 877, 879
corpus, **825**
Corrado, G., 30, 858, 878, *1053*, *1055*
Post, E. L., 248, *1058*
Cortés, C., 26, 718, 786, 922, *1040*, *1052*
corteza cerebral, 12
cosa segura, 535
cosas, **321**
costo de ruta más bajo, 65
costo para llevar, 955
costo total, 104
Cournot, A., 647, *1040*
Courville, A., 716, 787, 788, 849, *1035*, *1046*, *1067*
covarianza, **1029**
Cowan, J. D., 20, 785, *1040*, *1066*
Cowell, R., 559, 747, *1040*, *1062*
Cowhey, I., 850, *1040*
Cowling, P. I., 176, *1038*
Cox, I., 526, 976, *1040*
Cox, R. T., 394, 409, 410, *1040*
Coz, D., 30, *1053*
Cozmo (robot de entretenimiento), 973
CPCS (sistema de diagnóstico médico), 421, 455
CPLAN (sistema de planificación), 381
CPT (tabla de probabilidad condicional), **413**
Craig, J., 978, *1040*
Craik, K., 13, *1040*
Crakovna, V., 822, 1003, *1051*, *1052*
Crammer, K., 718, *1042*
Cramton, P., 649, *1040*
Crato, N., 206, *1046*
Craven, M., 854, *1040*
Crawford, J. M., 249, *1040*
Crawford, K., 995, *1045*, *1065*
creatividad, 16
creencia, 326
  actualización, 335
  deseos y, 528-529
  grado de, 385, **386**, 394
  loco, 458
  propagación, **458**
  revisión, **335**
Cremers, A. B., 526, 977, *1038*, *1061*
Cresswell, M. J., 340, *1048*
cresta (en la búsqueda local), **112**, 113

Crick, F., 11, 118, 1007, *1040*, *1065*
Criptoaritmética, **184**
Crisan, D., 499, *1040*
Cristianini, N., 718, *1040*
crítico (en el aprendizaje), **57**
Crockett, L., 249, *1040*
Croft, W. B., 850, 854, 855, *1040*, *1058*
Crónica, 340
crossover, 141
crowdsourcing, 705
Crucero, 924
Crucigrama, 46
Cruse, A., 853, *1040*
Cross, S. E., 29, *1040*
CSI, véase independencia, contexto específico Csorba, M., 977, *1042*
CSP (problema de satisfacción de restricciones), 20, **180**, 180–185, 352
CSP binario, **184**
CTRL (modelo lingüístico), 833
Cuadros, J., 30, *1046*
cuadro delimitador, **899**
cuantificación, 845
cuantificador, **260**, 278
  anidados, 263
  en lógica, 260-264
  existencial, 262
  universal, 260-262, 280
Cubierta, T., 720, *1040*
Cubo de Rubik, 98, 100, 106
Cuéllar, J., 250, *1034*
cuello rígido, 399
Cuenca, D. A., 176, *1044*
Cuenta de Borda, **640**
cuerpo (de la cláusula de Horn), **230**
Culberson, J., 109, *1040*
culto al computacionalismo, 982
Cummins, D., 560, *1040*
Cummins, F., 787, *1045*
Curie, M., 1
curiosidad, 799
Curran, J. R., 853, *1040*
Currie, K. W., págs. 384, *1044*
Curtis, F. E., 717, *1037*
Curva de aprendizaje, **661**
curva de características de funcionamiento del receptor (ROC), **710**
curva de entrenamiento, **683**
Cushing, W., 384, *1040*
Cushman, F., 821, *1048*
Cusumano-Towner, M. F., 526, 527, *1040*
Cutler, A., 698
Cybenko, G., 785, *1040*
CYBERLOVER (chatbot), 984
CYC (base de conocimientos), 316, 340
Cyert, R., 561, *1040*
Cyganiak, R., 316, 339, *1037*
CYK-PARSE, **837**
Czarnecki, W. M., 30, 179, 787, *1049*, *1064*

D

Da Silva, B. C., 821, *1063*
Da Vinci, L., 6, 920
Dafoe, A., 28, *1046*
DAG (grafo acíclico dirigido), 412, 454
Dagan, I., 880, *1040*
Daganzo, C., 455, *1040*
Dagdelen, J., 872, *1064*
DAGGER (sistema de aprendizaje por imitación), 967, 979
Dagum, P., 457, *1041*
Dahiya, R., 920, *1054*
Dahl, G., 30, 849, 854, *1048*, *1053*
Dahy, S. A., 682, 684, *1050*
Dai, A. M., 879, *1041*
Dai, Z., 879, *1066*
Dalal, N., 922, *1041*
Dalibard, V., 787, *1049*
Dalmao, S., 456, *1035*
Dalvi, B., 850, *1055*
Dalvi, N. N., 524, *1041*
Daly, R., 748, *1041*
Damas, 19, 62, 177, 819, 820
Damasio, A. R., 1007, *1041*
D'Ambrosio, B., 456, *1061*
Damerau, F., 852, *1034*
Danaher, J., 1006, *1041*
Danés, 826
Dang, T. D., 313, *1046*
Dantzig, G. B., 143, *1041*
daño cerebral óptimo, 787
DARPA Urban Challenge, 972
DARPA, 29, 1010
Darrell, T., 790, 923, 979, *1045*, *1052*
Darwiche, A., 456, 457, 460, *1039*, *1041*, *1057*
Darwin, C., 118, *1041*
Dasgupta, P., 145, 649, *1041*
Datos completos, **724**
Dauphin, Y., 786, *1041*
Daun, B., 384, *1041*
Davidson, A., 647, *1036*
Davidson, D., 340, 824, *1041*
Davidson, M., 106, *1059*
Davis, A., 718, 1021, *1061*
Davis, E., 340-343, *1041*
Davis, G., 384, *1041*
Davis, M., 233, 248, 303, 310, 527, *1041*
Davis, T., 30, 178, *1056*
Dawid, A. P., 456, 559, 747, *1040*, *1052*, *1062*
Dayan, P., 177, 788, 820–822, *1041*, *1061*
DBN (red bayesiana dinámica), 461, 485–498
DBPEDIA (base de conocimientos), 316, 339
DDN (red dinámica de decisión), 570, 595
de adelante hacia adelante, **96**
de algoritmos, 309
de bajo impacto, **1002**
De Borda, J-C., 640
de bucle abierto, **64**, 951
De Condorcet, M., 718, *1041*
De Dombal, F. T., 410, *1041*
De Farias, D. P., 596, *1041*
De Finetti, B., 394, 409, *1041*
De Freitas, J. F. G., 498, 499, *1041*, *1042*
De Ghellinck, G., 596, *1041*
De Groot, M., 561, *1040*
De Kleer, J., 206, 311, 339, 342, *1041*, *1044*, *1065*
De Marcken, C., 854, *1041*
De Marneffe, M.-C., 839, *1057*
De Morgan, A., 204, 278, *1041*
de Oliveira Marinho, G., 30, *1053*
D'E'penoux, F., 596, *1042*
de primer orden (FOL), 251
de principio a fin, **96**
De Raedt, L., 854, *1056*
De Salvo Braz, R., 527, *1041*
De Sarkar, S. C., 109, 145, *1039*, *1041*
de texto a voz, **849**
De Wever, A., 559, *1036*
Deale, M., 384, *1041*
Dearden, R., 597, 820, 1037, *1041*
Deb, S., 142, *1066*
Debevec, P., 923, *1041*
Debreu, G., 543, *1041*
DEC (Corporación de Equipos Digitales), 23, 292
decaimiento de peso, **771**
Dean, J., 311, 652, 718, 787, 852, 858, 878, 880, 1018, 1021, *1037*, *1041*, *1051*, *1055*, *1058*, *1061*
Dean, M. E., 250, *1057*
Dean, S., 1009, *1053*
Dean, T., 383, 455, 498, 596, 597, 977, 978, 1019, *1041*
decepción posterior a la decisión, **559**
Deacon, T. W., 24, *1041*
Dechter, A., 205, *1041*
Dechter, R., 107, 204-207, 456, 457, 460, *1040*, *1041*, *1049*, *1054*, *1058*, *1061*
decisión
  racional, 385, 387, 528
  robusta, **553**
  secuencial, 547, **562**
decisión minimax, **149**
declarativismo, **210**, 247
declarativo, 251
decodificación (en MT), **867**
  codiciosa, 867
decodificación turbo, **458**
decodificador (en autocodificadores), 778
decodificador de eje, **929**
decodificador de transformador, **871**
DeCoste, D., 718, *1041*
Dedekind, R., 278, *1041*

Deep Blue (programa de ajedrez), viii, 30, 176
DEEP THOUGHT (programa de ajedrez), 176
deepfake, **915**
DEEPHOL (demostrador de teoremas), 309
DEEPMATH (demostrador de teoremas), 312
DeepMind, 31, 179, 779, 784, 816, 822, 1008
DeepStack (programa de póker), 178, 622
Deerwester, Carolina del Sur, 852, 878, *1042*
definición (lógica), **267**
degradación graciosa, 594
degradado de textura, 921
DeGroot, M. H., 410, 748, *1042*
Dehaene, S., 1007, *1042*
Deisenroth, M. P., 820, *1035*
Del Favero, B. A., 456, *1061*
Del Moral, P., 499, *1042*
Delarue, A., 70, *1036*
Delgrande, J., 341, *1042*
deliberativo, **968**
Della Pietra, S. A., 880, *1038*
Della Pietra, V. J., 880, *1038*
Dellaert, F., 179, 977, *1044*, *1064*
Delling, D., 108, *1042*
demodulación, **307**, 312
Demostración de teoremas, 21, **222**, 298–309, 380
Demostrador de Stanhope, 248
Demostrador del teorema de la geometría, 19
Dempster, A. P., 459, 497, 748, *1042*
Denardo, E. V., 596, *1042*
DENDRAL (sistema experto), 22, 23, 338
dendrita, 12
DeNero, J., 717, *1056*
Deng, J., 26, 786, *1042*, *1060*
Deng, L., 788, 849, 854, *1042*, *1048*, *1066*
Deng, X., 144, 648, *1042*
Denker, J., 26, 786, 787, 922, *1052*
Denney, E., 312, *1042*
Denniston, A. K., 30, *1053*
densidad asumida, **499**
De Oliveira, J., 339, *1054*
dependencia a distancia, 846
dependencias universales (naturales conjunto de datos lingüísticos), 839
depuración, 273
derecho social, **605**
derechos de perforación, 547
derivación, 216
Dervovic, D., 1018, *1042*
desafío de esquema de Winograd, 877
desafío PASCAL, 880
desafío urbano, 977
desaliñado vs. ordenado, 24
desambiguación, 846-849
Descartes, R., 6, 920, 1006, *1042*
descendiente (en redes bayesianas), **418**
descenso de gradiente online, **679**
descenso de gradiente estocástico (SGD), 114, **677**, **679**, 765
DESCOMPONER, **364**
descomponibilidad (de loterías), **531**
descomposición, **356**
descomposición aditiva (de valor funciones), **810**
descomposición celular, **945**
descomposición de árboles, **201**, 204
descomposición exhaustiva, **318**
descomposición jerárquica, **357**
Descotte, Y., 383, *1042*
descripción de PEAS, **42**, 44
descripción del estado, 68
descripción optimista (de una acción), **362**
descripción pesimista (de una acción), **362**
descubrimiento científico, 716
desempleo tecnológico, 976, **998**
desenrollado, 490, **506**
Deshpande, A., 909, *1034*
Deshpande, I., 916, *1042*
desidentificación, **991**
desigualdad de ingresos, 987, **1000**
desigualdad de triángulos, **88**
Deslippe, J., 30, *1052*
Desouza, P. V., 878, *1038*
despliegue, **161**
desviación estándar, 173, **1028**
detección activa, 881
detección de bordes astutos, 921
detección de bordes, 889–892
detección de spam, **826**
detección pasiva, 881
Dethridge, J., 106, *1059*
Deutscher, G., 278, *1042*
Deville, Y., 205, *1064*
DEVISER (sistema de planificación), 383
Devlin, J., 879, *1042*
Devlin, K., 1006, *1042*
Devol, G., 976
Devroye, L., 748, *1042*
Dewey Sistema decimal, 317
Dey, A. K., 817, 963, 979, *1067*
Dey, K., 996, 1009, *1036*
Diaconis, P., 538
diagnóstico, 386, 399, 400
  médico, 23, 410, 418, 547
  odontología, 386
diagnóstico médico, 23, 410, 418, 547
diagrama de decisión binario (BDD), 382
diagrama de influencia, 454, 528, **544**, **544**, 544–547, 557
diagrama de Voronoi, **944**
diálisis, 534
Diamond Nixon, 334
diámetro (de un gráfico), **80**
Dias, M. B., 142, *1042*
Diccionario, **828**
Dickerson, S., 979, *1063*
Dickinson, M. H., 1013, *1044*
Dickmanns, E. D., 28, 923, 977, *1042*
Dieleman, S., 29, 779, 787, 849, 920, *1043*, *1064*
Dietterich, T., 822, *1042*
difuso, 214, 255, 459
Dijkstra, E. W., 107, 984, *1042*
Dimensión VC, **717**
dinámica inversa, **951**
Ding, Y., 30, *1042*
Dinh, H., 107, *1042*
Diofanto, 204
Dionne, A., 109, *1063*
Dios, existencia de, 409
DiPasquo, D., 854, *1040*
diplomacia, 151
discontinuidad (profundidad), 889
discrepancia media máxima, 719
discretización, **120**, **422**
discriminador (en GAN), **780**
diseño de mecanismos, **600**, 632
diseño de proteínas, **70**
Diseño VLSI, 70, 107, 115
disjuntas, **101**
disparidad en el tamaño de la muestra, **995**
disparidad, **902**
disposición de celdas, 70
Dissanayake, G., 977, *1042*
distancia de Hamming, **688**
distancia de la manzana de la ciudad, 98
distancia de Mahalanobis, **688**
distancia de Manhattan, 100
distancia de Minkowski, **687**
distancia en línea recta, **85**
distancia focal, **883**
distopía, 1011, *1022*
distribución
  acumulativo, 541, 1027
  beta, 487, **730**
  categórico, 391
  cola pesada, **142**
  condicional, no paramétrico, **422**
  mezcla, **739**
distribución acumulada, 541, 1027
distribución beta, 487, **730**
distribución canónica, **420**
distribución categórica, **391**
distribución condicional, 391
distribución conjunta completa, 395, 397
distribución de cola pesada, **142**
distribución de Dirichlet, **731**
distribución de mezclas, **739**
distribución de propuestas, **447**
  adaptativo, **523**
distribución de orden de magnitud, **509**

distribución de Poisson, **509**
distribución de probabilidad conjunta, **392**
distribución estacionaria, **444**, 468
distribución gaussiana, **1027**
  multivariante, 479, *1028*
distribución normal, 1027
  estándar, 1028
distribución log-normal discreta, **509**
distribución uniforme, 391, **1029**
distribuciones condicionales, 420–424
distribuir ∨ sobre ∧, 227, 300
Disyunción, **217**, 393
disyuntiva, 217
Dittmer, S., 560, *1042*
Divergencia de Kullback-Leibler, **758**
dividiendo un pastel, 641
división justa, 628
Dix, J., 342, *1038*
DLV (sistema de programación lógica), 342
Do, M. B., 383, *1042*
Do, M., 29, 381, *1035*, *1042*
Do, Y., 107, *1042*
Doble uso, **990**
do-cálculo, **452**
Doctorow, C., 12, 340, *1042*
Dodd, L., 840, *1042*
dolor de muelas, 386
dominación (de heurística), **99**
dominación débil, **607**
dominancia
dominancia estocástica, **541**, 557
Domingos, P., 35, 410, 456, 525, 708, 718, 720, 747, *1042*, *1045*, *1059*
dominio
  continuo, **183**
  elemento de, **256**
  en la representación del conocimiento, **265**
  en lógica de primer orden, **256**
  infinito, **183**, 298
Dominio de parentesco, 266–268
dominio orientado a tareas (en sistemas multiagente), **643**
dominios continuos, **183**
Domshlak, C., 381, 559, 646, *1037*, *1048*
Donahue, J., 787, 923, *1045*, *1049*
Dong, W., 26, *1042*
Dong, X., 316, *1042*
Donti, P. L., 30, *1059*
Doorenbos, R., 311, *1042*
Doran, J., 107, 108, *1042*
Dorf, R. C., 61, *1042*
Dorigo, M., 142, *1042*
Doron, Y., 822, *1063*
Doshi, T., 1009, *1036*
Doshi-Vélez, F., 719, *1042*
Dota 2, 28, 30
Doucet, A., 498, 499, *1034*, *1040*, *1042*, *1057*
Downey, D., 855, *1043*
Downs, L., 978, *1037*
Dowty, D., 853, *1042*
Doyle, J., 61, 206, 341, 342, 559, 560, *1042*, 1055, 1065, *1067*
DPLL, **234**, 248, 397
¿DPLL-SATISFACIBLE?, **234**
DQN (red Q profunda), 784, **816**, 822
Drabble, B., 384, *1042*
Dragan, A. D., 62, 561, 648, 821, 964, 967, 979, *1035*, *1042*, *1046*, *1054*, *1060*
DRAGON (sistema de reconocimiento de voz), 854
Drake, J. P., 717, *1036*
Draper, D., 383, *1043*
Drebbel, C., 16
Drechsler, R., 108, *1043*
Dredze, M., 718, *1042*
Dressel, J., 993, 1009, *1042*
Dreyfus, H. L., 249, 981, 982, *1042*
Dreyfus, S. E., 107, 596, 785, 982, *1036*, *1042*
Driessens, K., 820, *1063*
Droppo, J., 29, *1066*
Drucker, H., 26, 786, 922, *1052*
Druzdzel, M. J., 457, *1039*
D-separación, **419**
DT-AGENTE, **388**
Dürer, A., 920
Du, J., 876, 879, *1053*
Du, S. S., 786, *1042*
dualismo, **6**
Dubois, D., 460, *1042*
Dubois, O., 249, *1062*
Dubourg, V., 720, *1058*
Ducharme, R., 878, *1036*
Duda, R. O., 410, 720, 747, 749, *1042*, *1043*
Dudek, G., 980, *1043*
Dudzik, A., 30, 179, *1064*
Duffy, D., 313, *1043*
Duffy, K., 718, *1040*
Duffy, N., 119, 143, *1055*
Dumais, S. T., 850, 852, 854, 878, *1035*, *1042*
Dunham, B., 21, *1044*
Dunham, C., 311, *1063*
Dunn, A., 872, *1064*
Dunn, H. L., 525, *1043*
Dunn, J., 716, *1036*
Dunn, R. C., 30, *1053*
Dunning, I., 30, 179, 787, *1049*
DuPont, 23
Dupont, D., 559, *1036*
duración, **375**
Durand, F., 1016, *1053*
Durfee, E. H., págs. 64, *1043*
Durme, B. V., 855, *1043*
Durrant-Whyte, H., 977, *1042*, *1052*
Duru, G., 559, *1036*
Dwork, C., 995, 1009, *1043*, *1066*
DYNA (agente de aprendizaje por refuerzo), 820
Dyson, F., 716, *1043*
Dyson, G., 1010, *1043*

E

E (demostrador de teoremas), 312, 313
E_0 (fragmento inglés), 833
Earley, J., 853, *1043*
Ebendt, R., 108, *1043*
EBL (aprendizaje basado en explicaciones), 382
Eck, D., 920, *1043*
Ecker, A. S., 983, *1045*
Ecker, K., 384, *1037*
Eckerle, J., 96, 108, *1043*
Eckert, J., 14
Eckhouse, L., 1009, *1043*
economía, **146**
Economía, 9–10, 61, 534
ecuación de Bellman, **568**
ecuación de Euler-Lagrange, **949**
ecuación de Riccati, **955**
ecuación diferencial, estocástico, 462
ecuación estructural, **450**
ecuación normal, 680
ecuaciones de Yule-Walker, **779**
ecuaciones diofánticas, **204**
Edelkamp, S., 109, 381, *1043*
Edimburgo, 976
Edmonds, D., 17
Edmonds, J., 9, *1043*
Edward (lenguaje de programación probabilístico), 526
Edwards, D. J., 175, *1047*
Edwards, P., 1007, *1043*
Edwards, W., 558, *1065*
EEG, 11
Een, N., 312, *1034*
efectivo, S. S., 253, *1060*
efecto, **345**
  desaparecidos, **372**
efecto Baldwin, **118**, 143
efecto condicional, **368**
efecto de anclaje, **539**
efecto de certeza, **538**
efecto de encuadre, **539**
efecto horizonte, **158**
efecto secundario no intencionado, **1002**
efector, **925**
efectos secundarios negativos, **987**
Efratí, E., 649, *1051*
Efros, A., 26, 879, 908, 913-915, 923, *1044*, *1047–1050*, 1067
Ehrenfeucht, A., 717, *1037*
Einstein, A., 1
Eisner, J., 853, *1062*
Eitelman, S., 311, *1063*

Eiter, T., 342, *1043*
ejecución (de una acción de alto nivel), **357**
ejecución intercalada, **602**
ejemplo adversarial, **770**, 787
Ejemplo negativo, **657**
ejemplo positivo, **657**
Ekart, A., 143, *1059*
EKF (filtro de Kalman extendido), **483**, 935
El dilema del prisionero, **606**
El problema de los gorilas, **33**
El problema del millón de reinas, 197
ELBO (límite inferior de evidencia), **778**
Elder, J. F., 718, *1061*
elemento de rendimiento, **56**
elevación, **283**, 282–286, 527
  en inferencia probabilística, 507
Elfes, A., 977, *1056*
elicitación de preferencias, **533**
ELIMINACIÓN-PREGUNTA, **432**
Elio, R., 560, *1043*
Eliot, T.S., 824
Elisseeff, A., 719, *1046*
elitismo, **116**
ELIZA (chatbot), 984
Elkan, C., 747, 1009, 1043, *1049*
Elkind, E., 648, *1039*
Ellington, C., 1013, *1044*
Elliott, G. L., 205, *1047*
Elliott, P., 249, *1065*
Ellsberg, D., 560, *1043*
Elman, J. L., 786, 854, *1043*
ELMO (sistema de lenguaje natural), 879
Elo, A. E., 526, *1043*
Elsken, T., 787, *1043*
embolsado, **697**, 718
Empirismo, **6**, 855
Empson, W., 853, *1043*
EMV (valor monetario esperado), **534**
Enamórate, 982
ENAS (Búsqueda Eficiente de Arquitectura Neuronal), 787
encadenamiento hacia adelante, **230**, 230–231, 247, 286–293, 311
encadenamiento hacia atrás, **230**, 231–232, 247, 293–298, 311
Enderton, H. B., 279, 310, *1043*
Endriss, U., 649, *1037*
Eneldo, D. L., 250, *1057*
Enfoque procedimental, **210**, 251
Eng, C., 30, *1053*
Engel, J., 920, *1043*
Engelbart, D., 14
Engelberger, J., 976
ENIAC, 14
Enlaces IS-A, 341
Enrique, H., 822, *1049*
enrutamiento de canales, 70
ensamblaje de naves espaciales, 384
ensayo (en aprendizaje por refuerzo), **792**
ensayo controlado aleatorizado, **453**
enseñanza kinestésica, **968**
entorno competitivo, 146
entorno de aprendizaje arcade (ALE), 822
entorno de tareas, **42**, 60
entorno dinámico, **45**
entorno estático, **45**
entorno estocástico, **45**, 562
Entorno multiagente, 55, 599–605
entorno no estacionario, 648, 712
entorno observable, 43
entorno secuencial, **45**
entrelazado, 134, 194, 380
entrenamiento de adversarios, 968
entropía (H), **661**, 662
entropía cruzada, **758**
ENUMERACIÓN-PREGUNTA, **429**
ENUMERAR TODO, 429
EPAM (Perceptor y Memorizador Elemental), 715
época, **679**
Epstein, R., 1006, *1043*
EQP (demostrador de teoremas), 313
EQUIDAD DE IA 360, 996
equidad, 706, 711, 992–996, 1009
equilibrio, 171
  Bayes–Nash, **623**
  estrategia dominante, **607**
  maximin, **613**
  Nash, **608**, 645
  subjuego perfecto, **619**
equilibrio de Bayes-Nash, **623**
equilibrio de la estrategia dominante, **607**
equilibrio de Nash, **608**, 645
equilibrio entre sesgo y varianza, **655**
equilibrio máximo, **613**
equipo de prueba, **654**, 666
equivalencia (lógica), **222**
Erdmann, M. A., 144, *1043*
Erez, T., 822, 979, *1047*, *1053*, *1063*
ergodica, **444**
Erhan, D., 30, 787, 879, *1063*, *1064*
ERNIE (sistema de PNL), 879
Ernst, G., 107, *1056*
Ernst, H. A., 976, *1043*
Ernst, M., 381, *1043*
Erol, K., 382, *1043*
Erol, Y., 499, *1043*
error (de una hipótesis), **666**, 673
error de detección (en la asociación de datos), **516**
error fuera de bolsa, **698**
escalada de colinas de reinicio aleatorio, **113**
Escalante, H. J., 719, *1046*
Escalera, S., 719, *1046*
escarabajo estercolero, 41, 62, 374, 970
escena, **882**
escorzo, 882
escuela de la Gestalt, 921
escuela de Megara, 247
escuela estoica, 247
espacio de configuración, **939**
espacio de estados, **65**
  articulación, **809**
  grafo, 105, **147**
  metanivel, 103
espacio de estados de nivel de objeto, **103**
espacio de pesas, **677**
espacio de trabajo, **939**
espacio libre, **940**
espacio muestral, **388**
Espacio Profundo Uno, 355, 384
especies, 24, 316, 317, 339, 738, 1010
espectrómetro de masas, 22
esquema (en un algoritmo genético), **117**
esquema de acción, **345**
esquema de inducción matemática, 305
esquema de percepción, **366**
Essig, A., 410, *1046*
estabilidad
  de un controlador, **953**
  estricto, **953**
establecer problema de partición, **631**
estacionariedad, **665**
estadística de pedidos, **536**
estadísticas, **8**, 1028
estado, **344**
  mundo, 66
  repetidas, **74**
estado cinemático, **951**
estado de absorción, 799
estado de conocimiento, 387
estado dinámico, **951**
estado inicial, **65**, 68, 105, 147, 345
estado interno, **51**
estado más probable, **956**
estado mundial, 66
estado terminal, **147**
Estados Unidos, 13, 14, 533, 999
estandarización aparte, **284**, 350
estereograma, punto aleatorio, 921
estereopsis binocular, **902**, 902–903, 921
Esteva, A., 30, *1043*
estimación consistente, **437**
estimación de densidad, **724**
  no paramétrica, **736**
estimación directa de la utilidad, 818
estimación estatal, **132**, 169, 242, 247, **465**, 931
  recursivo, **132**, 466
estimación estructural de MDPs, 821
estimación recursiva, 466
estímulo, 13, 881
estocástico, **541**, 557, **679**, 765
estrategia de Zeuthen, **644**

estrategia dominante, **607**, 635
estrategia mixta, **606**
Estrategia pura, **606**
Estrategia tipo A, **155**
Estrategia tipo B, **155**
Estrategia, 169, 606
Estrategias de evolución, **116**, 143
estrictamente estable, **953**
estricto, **540**
Estrin, D., 1009, *1035*
estructura de frases, **833**
estructura de la coalición, **626**
estructura jerárquica, 1014 red jerárquica de tareas (HTN), **357**, 379, 807
estructuras de índices aprendidas, 311
ESTUDIANTE (sistema de lenguaje natural), 20
Estudio de cien años sobre IA, 27
estupidez natural, 329
Etchemendy, J., 279, *1035*
ética deontológica, **8**
ética, 986-1005
etiqueta (en aprendizaje automático), **653**
etiqueta (en planos), **125**
etiqueta (parte de la oración), 829
etiquetado de parte de la oración, **830**
etiquetado de puntos de venta, 852
Etzioni, A., 1008, 1009, *1043*
Etzioni, O., 27, 316, 339, 383, 850, 854, 855, 876, 880, 1008, 1020, *1035*, *1040*, *1043*, *1052*, *1063*, *1065*
Euclides, 9, 920
Eugene Goostman, 984
Europa, 23
EUROPA (sistema de planificación), 29
Evans, O., 28, 821, *1046*, *1060*
Evans, T. G., 20, *1043*
evento, 322–325
  en probabilidad, **389**, **427**
  exógenos, **372**
Everett, B., 977, *1037*
Everitt, T., 822, 1003, *1052*
Evidencia, **389**, 721
evolución, 118
  de máquinas, **21**
Ewalds, T., 30, 179, 822, *1064*
excepción, 315, 331
exclusivo o, 219
expansión (de nodos), **71**
expansión de Taylor, **935**
expansión iterativa, **108**
expectativa, **1028**
expectativa de características, **814**
expectiminimax, 166, 174, 175
  complejidad de, 167
  valor, **165**
experimento psicológico, **2**
explicabilidad, **711**
explicación, **336**
  más probable, **457**, 496
exploración segura, 960
exploración, 40, 41, 134-141, **161**, 581, 791, 797, 798
explosión combinatoria, 21
explotación, **161**, 581, 798
expresión génica de microarrays, 698
expresividad, **59**
extensión (de la teoría por defecto), **335**
extensión singular, **159**
externalidades, **637**, 1002
extracción de características, 881
extracción de información, **850**

F

Facebook, 29, 822, 1008
factor (en la eliminación de variables), **430**
factor de certeza, 23, 459
factor de descuento, **565**, 596, 642, 792
factor de ganancia, **952**
factor de ramificación, **76**
factor de ramificación infinito, 119
factoraje, **226**, 301
Faes, L., 30, *1053*
Fagin, R., 206, 341, *1036*, *1043*
Fahlman, S. E., 20, 342, *1043*
falla transitoria, **487**
fallo del sensor, 487
falsa alarma (en asociación de datos), **516**
falsos positivos, **710**
Fan, J., 30, *1043*
Farhadi, A., 880, *1061*
Farid, H., 993, 1009, *1042*
FARMVILLE (videojuego), 999
farol, **171**
Farrell, R., 311, *1038*
FASTDOWNWARD (sistema de planificación), 380
FASTFORWARD (sistema de planificación), 356
FASTTEXT (incrustación de palabras), 857
Fatica, M., 30, *1052*
Faugeras, O., 923, *1043*
Favini, G. P., 176, *1040*
Fawcett, T., 990, *1043*
FDSS (sistema de planificación), 381
Fearing, R. S., 978, *1043*
Featherstone, R., 978, *1043*
Fei-Fei, L., 26, 786, 879, *1042*, *1050*, *1060*
Feigenbaum, E. A., 17, 22, 23, 107, 338, 715, *1035*, *1038*, *1043*, *1053*
Feiten, W., 977, *1037*
Feldman, J., 62, 558, *1043*
Feldman, M., 13, *1043*
Fellbaum, C., 852, *1043*
Fellegi, I., 525, *1043*
Feller, A., 1009, *1040*
Felner, A., 88, 108, 109, 381, *1043*, *1048*, *1051*, *1060*
Felzenszwalb, P., 144, *1043*
Feng, L., 977, *1037*
Feng, S., 879, *1063*
Feng, T. K., 206, *1036*
Fenton, N., 558, *1043*
Fergus, R., 690, 787, *1063*, *1064*
Ferguson, T., 176, 597, 748, *1043*
Fermat, P., 8, 408
Fern, A., 561, *1043*
Fernández, F., 821, *1045*
Fernández, J. M. F., 30, 179, *1043*
Fernando, C., 787, *1049*, *1053*
FERPA, 990
Ferraris, P., 382, *1043*
Ferriss, T., 1000, *1043*
Ferrucci, D., 30, *1043*
FF (sistema de planificación), 356, 380
Fideicomiso, 710, **996**, 1009
Fidjeland, A., 820, 822, *1055*
figura retórica, 847, 848
fijación, **903**
Fikes, R. E., 61, 144, 278, 380, 382, 383, 976, *1044*
Filón de Megara, 247
filosofía, 6-8, 61, 981-1011
  moral, 39
FILTRADO DE PARTÍCULAS, **492**
filtrado, **132**, 335, 466–467, 496, 588, 744, 931
Filtro de Kalman, 461, **479**, 479–485, 497, 935
  conmutación, **484**
filtro de Kalman extendido (EKF), **483**, 935
filtrado de partículas, **492**, 497
  Rao-Blackwellized, **496**, 498, 977
filtro de vecino más cercano, **517**
filtro gaussiano, **890**
filtro Kalman de conmutación, **484**
finalización de Clark, 342
Fine, S., 498, *1044*
Fink, D., 119, 143, *1055*
Finkelstein, L., 207, *1038*
Finn, C., 719, 790, 979, *1044*, *1052*
Finney, D. J., 455, *1044*
Firat, O., 850, *1039*
Firby, R. J., 383, *1041*
Firoiu, V., 179, *1044*
Firth, J., 856, *1044*
Fisac, J. F., 648, 821, *1034*, *1054*
Fischer, A., 716, *1035*
Fischer, B., 312, *1042*
Fischer, P., 597, *1035*
Fischer, R., 168
Fisher, M. L., 561, *1056*
Fisher, R. A., 9, 409, *1044*
Física cualitativa, 321, **342**

fisicalismo, 6
Fix, E., 717, *1044*
Flannery, B. P., 142, *1058*
Floreano, D., 1013, *1044*
Floyd, R. W., 107, *1044*
fluido, **238**, 247, 325
  desaparecidos, **372**
fluidos faltantes, **372**
Flujo óptico, **893**, 921
FMEA (modos de fallo y análisis de efectos), **1001**
foco de expansión, **904**
Fogel, D. B., 144, *1044*
Fogel, L. J., 143, *1044*
FOL-BC-AND, **293**
FOL-BC-ASK, **293**
FOL-BC-OR, **293**
FOL-FC-ASK, **288**
Fong, R., 979, *1034*
FOPC, véase lógica, primer orden Forbes, J., 598, *1044*
FORBIN (sistema de planificación), 383, 384
Forbus, K. D., 311, 339, 342, *1044*
Ford, K. M., 1006, *1044*
Ford, L. R., 107, *1044*
Ford, M., 28, 35, 1011, *1044*
Forestier, J.-P., 822, *1044*
Forgy, C., 311, *1044*
Forma a partir de sombreado, 923
forma cuasi-lógica, **846**
Forma de cuerno, 247, 248
Forma de Kowalski, 299
forma de matriz, 474
forma de secuencia, **621**
forma extensiva, **617**
forma normal conjuntiva (CNF), **226**, 226–227, 247, 299–300
forma normal de Chomsky, **836**, 851
forma normal implicativa, 299
formación basada en la población (PBT), **672**
formación, 666
fórmula de ajuste, **453**
formulación en estado completo, **111**
Formulario Backus-Naur (BNF), **1030**
Formulario Backus-Naur (BNF), **1030**
Forrest, S., 143, *1055*
Forster, E. M., 1011, *1044*
Forsyth, D., 914, 916, 924, *1042*, *1044*, *1050*
Fortmann, T. E., 497, 526, *1035*
Fortran, 756
Foster, G., 850, *1039*
fotograma clave, **968**
fotogrametría, 921
Fotometría, 886–888
Fouhey, D., 908, *1044*
Fourier, J., 204, *1044*
Fowlkes, C., 921, *1054*
Fox, C., 560, *1044*
Fox, D., 526, 977, 980, *1038*, *1044*, *1061*, *1064*
Fox, M. S., 383, *1044*
FPGA, 27
FrameNet (base de datos léxica), 339
Francisco, J., 559, *1050*
Franco, J., 248, *1044*
François-Lavet, V., 820, *1044*
Francón, O., 119, 143, *1055*
Frank, E., 720, *1066*
Frank, I., 176, *1044*
Frank, J., 29, *1035*
Frank, R. H., 1000, *1044*
Frankenstein, 1001
Frans, K., 822, *1044*
Franz, A., 852, *1044*
Frasconi, P., 786, *1036*
frecuentismo, **408**
FREDDY (robot), 107, 144, 976
Freeman, W., 458, *1065*, *1066*
Freer, C., 526, *1034*
Frege, G., 8, 248, 278, 310, *1044*
Freitag, D., 854, 855, 1040, *1044*
fresas, disfrutar, 982
Freuder, E. C., 205, 206, *1044*, *1054*, *1060*
Freund, Y., 700, 718, *1044*
Frey, B. J., 787, *1044*
Frey, C. B., 999, *1044*
Friedberg, R. M., 21, 143, *1044*
Friedman, G. J., 143, *1044*
Friedman, J., 716, 718, 720, 749, *1038*, *1044*, *1047*
Friedman, N., 455, 456, 460, 498, 499, 596, 747, 748, 820, *1034*, *1037*, *1041*, *1044*, *1051*
frisbee, 30, 982
Fristedt, B., 597, *1036*
Frontera factorizada, **499**
Frontera, **72**
Frost, D., 205, 207, *1041*
Fruhwirth, T., 207, *1044*
FTA (análisis del árbol de fallas), **1001**
Fu, D. J., 30, *1053*
Fu, J., 341, 1014, *1053*
Fu, M. C., 597, *1039*
Fuchs, J. J., 384, *1044*
Fudenberg, D., 648, *1044*
FUEGO (demostrador de teoremas), 339
fuerte dominación, **607**
Fukunaga, A. S., 383, *1044*
Fukushima, K., 786, 922, *1044*
Fuller, S. B., 1013, *1044*
función, **254**
  en total, **257**
función característica, **626**
función de activación, **752**
función de bienestar social, **639**
función de costo, **16**
función de costo de acción, **65**, 105
función de desembolso, 147, **605**
función de elección social, **639**
función de error, 1028
función de evaluación, **73**, 105, 146, 156–158, 803
  lineal, 104
función de las unidades, **319**
función de mejor ajuste, 654
función de origen, **508**
función de pérdida, **669**
función de regularización, **671**
función de umbral, **683**
función de utilidad-acción, **545**, 568
función de valor, **532**
  aditivo, **543**
función heurística, **84**
función kernel, **694**, 736
función lineal ponderada, **157**
función lineal, **676**
función logística, **424**, 717
función objetivo, **110**
función Q, 545, **568**, **790**
función Skolem, **300**, 310
funcional, 949
Fundación Templeton, 986
Fung, C. C., 984, *1049*
Fung, C., 1009, *1044*
Fung, R., 457, *1044*
Furcy, D., 145, *1051*
Furnas, G. W., 852, 878, *1042*
Furst, M., 381, *1037*
fútbol, 179
  robótico, 146, 17, 650

G

g (costo de trayecto), 73
Gabriele, S., 30, *1053*
Gabrilovich, E., 316, *1042*
Gaddum, J. H., 455, *1044*
Gadepalli, K. K., 30, *1053*
Gaffney, S., 822, *1036*
Gaifman, H., 524, 525, *1044*
Gale, W. A., 851, *1039*
Galfrin, E., 456, *1061*
Galileo, G., 1, 57
Gallagher, G., 963, 979, *1067*
Gallaire, H., 311, *1044*
Gallegos, J., 30, *1053*
Gallier, J. H., 279, *1044*
Galperin, G. R., 176, *1063*
Galstyan, A., 1009, *1055*
Galton, F., *1044*
Gamba, A., 785, *1045*
Gamberini, L., 785, *1045*
Gammage, C., 30, *1062*

GAN (red generativa adversaria), **780**, 787, 915
Ganancia de información, **662**, 663, 957
Ganchev, K., 853, *1034*
Gandomi, A., 719, *1045*
Ganguli, S., 786, 787, *1041*, *1058*
Gannon, I., 456, *1061*
Gante, I., 207, *1045*
Gao, J., 652, *1045*
Gao, Q., 29, 783, 850, 865, *1066*
Gao, Y., 822, *1049*
García, E. A., 707, 995, *1047*
García, J., 821, *1045*
Gardner, M., 248, 850, 879, *1045*, *1055*, *1058*
Garey, M. R., 1025, 1029, *1045*
Garg, A., 498, *1057*
GARI (sistema de planificación), 383
Garrett, C., 116, *1035*
Gašić, M., 598, *1066*
Gaschnig, J., 107, 205, 206, *1045*
Gasquet, A., 384, *1044*
Gasser, L., 646, *1037*
Gasser, R., 109, *1045*
Gat, E., 979, *1045*
Gatys, L. A., 983, *1045*
Gauci, J., 822, *1045*
Gauss, C. F., 204, 497, 717, *1045*
gaussiano condicional, **423**
Gazzaniga, M., 1007, *1045*
GBM (máquina de aumento de gradiente), 701
GBRT (árbol de regresión potenciado de gradiente), 701
Gearhart, C., 597, *1046*
Gebru, T., 995, *1038*, *1045*, *1055*
Gee, A. H., 498, *1041*
Geffner, H., 144, 380, 382, 383, *1037*, *1047*, *1057*
Geiger, D., 455, 456, 747, *1045*, *1047*, *1061*
Gelatt, C. D., 142, 206, *1050*
Gelb, A., 497, *1045*
Gelder, A. V., 313, *1063*
Gelernter, H., 19, 312, *1045*
Gelfond, M., 312, 342, *1045*
Gelman, A., 458, 527, 719, 747, 748, *1038*, *1039*, *1045*
Geman, D., 457, 718, 921, *1034*, *1045*
Geman, S., 457, 921, *1045*
Gemp, I., 820, *1053*
generación (de nodos), **72**
generador, **1031**
generalización, **966**
generalización apilada, **699**
Generalizado, **282**, 282–283
GENERAR-IMAGEN, **519**
GENERATE-MARKOV-LETTERS, **521**
Genesereth, M. R., 61, 144, 179, 278, 279, 298, 303, 311, 312, *1045*, *1053*, *1062*
Georgeson, M., 924, *1038*
Georgiev, P., 30, 179, 822, *1064*
Gerbault, F., 748, *1045*
Gerkin, R. C., 920, *1060*
Géron, A., 720, *1045*
Gers, F. A., 787, *1045*
Gesmundo, A., 719, *1066*
Getoor, L., 527, *1045*
Ghaheri, A., 143, *1045*
Ghahramani, Z., 455, 498, 719, 748, 787, *1045*, *1049*, *1060*, *1062*
Ghallab, M., 344, 380, 383, 384, *1045*
Gharbi, M., 1016, *1053*
Ghavamzadeh, M., 820, *1053*
Ghose, D., 142, *1051*
Ghose, S., 109, *1039*
GIB (programa puente), 178
Gibbs, R. W., 853, *1045*
GIBBS-ASK, **443**
Gibson, J. J., 921, 923, *1045*
Gil, Y., 316, *1039*
Gilks, W. R., 457, 458, 499, 525, 747, *1045*
Gillies, D. B., 648, *1045*
Gillula, J. H., 821, *1034*
Gilmore, P. C., 310, *1045*
Gilpin, A., 647, *1045*
Ginsberg, M. L., 178, 206, 311, 460, *1041*, *1045*, *1062*
Ginter, F., 839, *1057*
Gionis, A., 717, *1045*
Girshick, R., 923, *1045*
Gittins, J. C., 597, *1045*
Giunchiglia, E., 382, *1043*
Givan, R., 820, *1063*
Gladman, A. S., 1013, *1051*
Glanc, A., 975, *1045*
Glavieux, A., 458, *1036*
Glickman, M. E., 526, *1045*
Glickman, O., 880, *1040*
GLIE (codicioso en el límite de la exploración infinita), **798**
Global, **184**, 188
GLONASS (GPS ruso), 929
Glorot, X., 786, *1045*
GloVe (software de incrustación de palabras), 857, 858, 872, 875
Glover, F., 142, *1045*
Glover, K., 560, *1067*
GLUE (Evaluación de la Comprensión General del Lenguaje), 879
Gluss, B., 560, *1045*
Glymour, C., 278, 747, *1046*, *1062*
Go (juego), 19, 27, 30, 161, 177, 784, 816
  cláusula, **230**
  común, 600
  estado, **65**, 105
  formulación de, **63**
  gol, **53**, 63, 345
  inferencial, 266
  monitoreo, 372
Go del número, 305
Godefroid, P., 381, *1045*, *1062*
Goebel, J., 748, *1045*
Goel, A., 637, *1034*
Goel, S., 1009, *1040*
Goertzel, B., 33, *1045*
GOFAI (IA a la antigua), **982**
Gogate, V., 456, *1045*
Gold, E. M., 716, 854, *1045*
Goldberg, A. V., 108, *1045*
Goldberg, D. E., 143, *1058*
Goldberg, K., 144, *1065*
Goldberg, Y., 839, 849, 853, 878, *1045*, *1052*, *1057*
Goldman, R., 144, 382, 525, *1039*, *1046*, *1065*
Goldstein, T., 1009, *1046*
Goldszmidt, M., 455, 460, 597, 747, *1037*, *1044*, *1046*
goleada, **95**
Golgi, C., 11
Golomb, S., 205, *1046*
Golub, G., 716, *1046*
Gomes, C., 142, 206, 248, 381, *1046*
Gómez, A. N., 850, 868, 880, *1064*
Gondek, D., 30, *1043*
Gonina, K., 849, *1039*
Gonnet, G. H., 719, *1036*
Gonthier, G., 204, *1046*
Gooday, J. M., 342, *1040*
Goodfellow, I., 769, 786-788, *1039*, *1046*, *1063*
Goodman, J., 852, *1039*, *1046*
Goodman, N. D., 526, 527, *1036*, *1046*, *1054*, *1066*
Goodman, N., 340, *1046*, *1052*
Goodrich, B., 458, 527, 747, *1039*
Google, 29, 31, 652, 763, 850, 852, 853, 924, 986, 1008, 1017
Google Académico, 511
Google Dúplex, 29
Gopnik, A., 278, *1046*
Gordon, A. D., 526, 527, *1040*, *1046*
Gordon, A. S., 341, *1046*
Gordon, G., 498, 598, 979, *1058*, *1060*, *1064*
Gordon, M. J., 278, *1046*
Gordon, N., 498, 499, *1035*, *1042*, *1046*
Gordon, S. A., 179, *1046*
Gorry, G. A., 410, *1046*
Gottlob, G., 207, *1046*
Gotts, N., 342, *1040*
Goüdel, K., 9, 310, 983, *1045*
Goyal, N., 876, 879, *1053*
Goyal, Y., 910, *1046*
GP-CSP (sistema de planificación), 381
GPS diferencial **929**
GPT-2 (modelo de lenguaje), 832, 833, 876, 879, 1021

GPU (unidad de procesamiento gráfico), 15, 27
Grace, K., 28, *1046*
gradiente, **120**, 701
  empírico, **120**, 811
  explosión, **774**
  desvanecimiento, **756**, 774
gradiente de fuga, 863
gradiente empírico, **120**, 811
grado de creencia, 385, **386**, 393, 394
grado de utilidad, 387
grado de verdad, **255**
grados de libertad (DOF), **940**
Graepel, T., 27, 30, 155, 174, 177, 178, 526, 820, *1046*, *1047*, *1061*
grafo, **65**
  colorear, 204
  euleriano, **144**
grafo de computación, **754**
grafo de conocimiento de Google, 316
grafo de flujo de datos, **754**
grafo de la estructura de la coalición, **631**
grafo de restricciones, **181**, 200
grafo de visibilidad, **943, 978**
grafo de Voronoi, **944**
grafo dual, **184**
grafo existencial, **329**
grafo feliz, 661
grafo Y-OR, 230
grafo acíclico dirigido (DAG), 412, 454
grafo de planificación, **352**
Graham, S. L., 853, *1046*
gramática, 823, 832, 833, 1030
  aumentadas, **841**
  categorial, 853
  dependencia, 853
  estructura de frases, 851
  inducción de, 854
  lexicalizado, **841**
  léxico-funcional (LFG), 853
  probabilística, **833**, 833–835, 852
  sin contexto, 833, 851, 852, **1030**
gramática de dependencias, **838**
gramática léxico-funcional (LFG), 853
gramática libre de contexto, 833, 851, 852, **1030**
gramática libre de contexto probabilístico (PCFG), 851
gramática universal, **854**
Gramfort, A., 720, *1058*
Gran Bretaña, 22, 23
Gran Coalición, **626**
Gran computación, 922
Gran Desafío de DARPA, 28, 972, 977, 1020
Graphplan (sistema de planificación), 352, 381
Grassmann, H., 278, *1046*
Graunt, J., 8
Gravano, L., 855, *1034*
Grave, E., 852, *1049*
Graves, A., 779, 784, 787, 790, 820, 822, 849, *1055*, *1064*
Grayson, C. J., 535, *1046*
Grebas, M., 1010, *1034*
Grecia, 247, 340
Greenbaum, S., 853, *1059*
Greenspan, M., 179, *1052*
Greiner, R., 747, *1039*
Grenager, T., 648, *1061*
Gribkoff, E., 527, *1046*
Griffiths, T. L., 118, 143, 278, 560, 598, *1046*, *1056*, *1059*, *1063*
Grinstead, C., 410, *1046*
Grisel, O., 720, *1058*
GRL (lenguaje de control de robots), 980
Grosz, B. J., 27, 637, 649, *1046*, *1048*
Grove, A., 410, 559, 1035
Grove, W., 338, 1046
Gruber, T., 316, 340, *1046*
Grumberg, O., 381, *1040*
GSAT (algoritmo de satisfacción), 249
Gu, J., 206, 248, *1046*, *1062*
Guan, M. Y., 787, *1058*
Guard, J., 313, *1046*
Guardián, **808**
Guestrin, C., 561, 597, 647, 718, 719, 822, *1039*, *1046*, *1051*, *1054*, *1059*
Guez, A., 19, 27, 30, 155, 174, 176-178, 820, *1061*
Gugger, S., 720, *1048*
Guha, R. V., 316, 339, *1038*, *1052*
Guibas, L. J., 457, 978, *1046*, *1064*
Guiver, J., 526, *1046*
Guizzo, E., 29, *1034*
Gulcehre, C., 786, *1041*
Gulshan, V., 30, *1046*
Gunkel, D. J., 1011, *1046*
Gunning, D., 1010, *1046*
Guo, C., 1009, *1046*
Guo, J., 458, 527, 747, *1039*
Gupta, A., 561, 979, *1051*, *1058*
Gupta, R., 877, *1059*
Gupta, V., 30, *1053*
Gururangan, S., 877, *1046*
Gustafsson, F., 498, *1047*
Guterres, A., 988
Guthrie, F., 204
Guugu Yimithirr, 253
Guyon, I., 26, 718, 719, 786, 922, *1037*, *1046*, *1052*

H

H (entropía), **661**, 662
$\mathcal{H}$ (espacio de hipótesis), 653
h (función heurística), 84
Habitación china, **985**, 1007
Hábitat de IA (entorno simulado), 822
hacer lo correcto, 1, **4**, 8, 39, 529
HACKER (sistema de planificación), 380
Hacking, I., 411, *1046*
Hadfield-Menell, D., 62, 561, 597, 648, 821, *1046*, *1054*
Hager, G., 27, *1063*
Hahn, M., 718, *1040*
Hahnel, D., 977, *1038*
Haider, M., 719, *1045*
Hailperin, T., 524, *1046*
Haimes, M., 527, *1055*
Hajic, J., 839, *1057*
Hajishirzi, H., 880, *1061*
Haken, W., 204, *1034*
Hald, A., 411, *1046*
Hales, T., 313, *1046*
Halevy, A., 26, 311, 340, 719, 850, 855, *1038*, *1046*
Halgren, E., 253, *1060*
Hall, L. O., 707, 995, *1039*
Halpern, J. Y., 278, 341, 410, 524, *1035*, *1043*, *1046*
Hamilton, A., 850 Hamiltoniano Monte-carlo, 527
Hamm, F., 340, *1064*
Hammersley, J. M., 821, *1047*
Hamori, S., 498, *1036*
Han, J., 720, *1047*
Han, X., 11, *1047*
Hanan, S., 381, *1043*
Handschin, J. E., 498, *1047*
Handscomb, D. C., 821, *1047*
Hanks, S., 383, *1043*
Hannun, A., 1009, *1046*
Hans, A., 821, *1047*
Hansen, E., 109, 144, 371, 383, 597, *1047*, *1067*
Hansen, M. O., 205, *1034*
Hansen, P., 248, *1047*
Hanski, I., 62, *1047*
Hansson, O., 109, *1047*
háptica, **978**
Harabagiu, S. M., 854, *1057*
Harada, D., 596, *1056*
Haralick, R. M., 205, *1047*
Hardin, G., 649, 1002, *1047*
Hardt, M., 716, 1009, *1043*, *1047*, *1053*, *1067*
Harel, D., 311, *1039*
Harman, D., 850, *1034*
Harnish, R., 30, *1042*
HARPY (sistema de reconocimiento de voz), 142
Harris, T., 1015, *1047*
Harris, Z., 851, *1047*
Harrison, J., 313, 559, *1046*, *1047*
Harrison, M. A., 853, *1046*
Harrow, A. W., 1018, *1047*

Harsanyi, J., 561, 647, *1047*
Harshman, R. A., 852, 878, *1042*
Hart, P. E., 107, 144, 382, 383, 410, 720, 747, 749, *1042–1044*, *1047*
Hart, T. P., 175, *1047*
Hartley, H., 748, *1047*
Hartley, R., 923, 924, *1047*
Harutyunyan, A., 821, *1056*
Harvard, 539
hash sensible a la localidad (LSH), **689**
Hashimoto, K., 1021, *1047*
Haslum, P., 380, 381, 383, *1047*
Hassabis, D., 19, 27, 30, 31, 176-179, 820, 822, *1036*, 1055, *1061*, *1064*
Hastie, T., 717, 718, 720, 749, *1044*, *1047*, *1049*
Hastings, W. K., 457, *1047*
Hatem, M., 108, *1038*, *1047*
Haugeland, J., 1007, *1047*
Haussler, D., 498, 717, *1037*, *1051*
Havelund, K., 309, *1047*
Havenstein, H., 26, *1047*
Hawking, S., 33
Hawkins, J., 785, *1047*
Hay, N., 597, 1019, *1047*
Hayes, P. J., 249, 340–342, 1006, *1044*, *1047*, *1054*
Hays, J., 26, *1047*
He, H., 707, 995, *1047*
He, K., 786, 1047, *1066*
He, Y., 822, *1045*
Head, R., 11, *1038*
Heafield, K., 852, *1038*
Hearst, M. A., 852, *1060*
Heath Robinson, 14
Heath, M., 716, *1046*
Heawood, P. J., 983, *1047*
Hebb, D. O., 17, 21, 819, *1047*
Hebert, M., 817, 923, 963, 979, *1048*, *1050*, *1067*
hecho, **230**
Heckerman, D., 455, 459, 498, 747, *1038*, *1047*, *1048*, *1062*
Hedau, V., 914, *1050*
Heess, N., 979, *1047*, *1053*
Hefesto, 975
Heidari, H., 1009, *1036*
Heidegger, M., 1006, *1047*
Heinlein, R. A., 1020, *1047*
Heitz, G., 316, *1042*
Held, M., 109, *1047*
Hellmann, S., 339, *1052*
Helmert, M., 107, 380, 381, *1047*, *1059*, *1061*
Helmholtz, H., 12, 920
Hempel, C., 7
Henaff, M., 786, *1039*
Hendeby, G., 498, *1047*
Henderson, P., 820, *1044*
Henderson, T. C., 205, *1055*
Hendler, J., 339, 382, 384, *1034*, *1036*, *1043*, *1062*
Henrion, M., 62, 421, 455, 457, 559, *1047*, *1048*, *1058*
Henzinger, M., 854, *1062*
Henzinger, T. A., 61, *1047*
Herbrand, J., 282, 304, 310, *1047*
Herbrich, R., 526, *1047*
Herbster, M., 1018, *1042*
Herden, G., 558, *1036*
herencia, **317**, 329
  múltiple, **330**
Hernández, D., 15, 1018, *1034*
Hernández-Orallo, J., 1007, *1047*
herramientas de de acondicionamiento, **414**
Herring, R., *498*, 1048
Herskovits, E., 747, *1040*
Hertz, J. A., 787, *1051*
Hess, C., 1010, *1047*
Hestness, J., 855, *1039*
Heule, M., 249, *1036*
heurística, 105
  admisible, **86**, 353
  búsqueda, 107
  compuesto, 100
  función, **84**, 97–104
  grado, **193**, 205, 234
  inadmisible, 89
  línea recta, 85
  Manhattan, **98**
  min-conflictos, **197**
  movimiento nulo, **176**
  para la planificación, 353–356
  valor menos restrictivo, **193**
  valores mínimos restantes, **193**, 205, 290, 378
heurística admisible, **86**, 353
heurística de conflictos mínimos, 197, 206
heurística de grado, **193**, 205, 234
heurística de ignorar las condiciones previas, **353**
Heurística de Manhattan, **98**
heurística de valor mínimo restrictivo, **193**
heurística diferencial, **103**
heurística ignorar-eliminar-listas, **354**
heurística inadmisible, 89, **90**
Hewitt, C., 311, 646, *1047*
Hezaveh, Y. D., 652, *1047*
híbrido A*, **946**
Hierholzer, C., 144, *1047*
Hilbert, M., 719, *1047*
Hilgard, E. R., 819, *1047*
Hill, F., 879, 880, *1065*
Hind, M., 995, 996, 1009, 1010, *1036*, *1047*
Hingorani, S. L., 526, *1040*
Hinrichs, T. R., 339, *1044*
Hinrichs, T., 179, *1053*
Hintikka, J., 340, *1047*
Hinton, G. E., 17, 26, 118, 143, 718, 785–788, 821, 849, 854, 922, 1021, *1035*, *1041*, *1047*, *1048*, *1051*, *1052*, *1056*, *1060–1062*
HIPAA, 990
hipergrafo de restricción, **184**
hiperparámetro, **666**, **730**
hipertexto, 14
hipótesis, 651, 653
  aproximadamente correcto, 673
  espacio, **653**
  nulo, **663**
  prior, **722**
hipótesis consistente, **653**
Hipótesis de Sapir-Whorf, 252
Hipp, J. D., 30, *1053*, *1062*
Hirschberg, J., 27, *1063*
Hirth, M., 1017, *1048*
histograma de campo vectorial, **978**
Hitachi, 359
hito, **946**
hito (característica reconocible), **933**
*h*MAP (hipótesis MAP), 723
HMM (modelo oculto de Markov), 25, 461, **473**, 473–478, 485, 497, 744, 830
Ho, J., 822, 978, *1044*, *1061*
Ho, M. K., 821, *1048*
Ho, T. K., 718, 719, *1046*, *1048*
Ho, Y.-C., 785, *1038*
Hoane, A. J., 176, *1038*
Hobbes, T., 6
Hobbs, J. R., 341, 343, 853, *1046*, *1048*
Hochreiter, S., 787, *1048*
Hodges, J. L., 717, *1044*
Hoff, M. E., 21, 819, *1065*
Hoffman, G., 979, *1063*
Hoffman, M., 458, 526, 527, 747, 852, *1039*, *1048*, *1064*
Hoffman, S. C., 996, 1009, *1036*
Hoffmann, J., 354, 356, 380-383, *1048*
Hofleitner, A., 498, *1048*
Hofmann-Wellenhof, R., 30, *1053*
Hogan, N., 978, *1048*
Hoiem, D., 914, 923, 1048, *1050*
Hoja de datos, **995**
hoja de ruta probabilística (PRM), **946**
Holland, J. H., 143, 1048, *1055*
Holenstein, R., 499, *1034*
holgura (en la programación), **376**
holgura mínima, **378**
Hollerbach, J. M., 978, *1043*
Holte, R. C., 88, 96, 108, 109, 647, *1034*, *1036*, *1039*, *1043*, *1048*, *1052*
Holzmann, G. J., 309, *1048*
Homan, K. A., 1013, 1051
hombro (en el espacio de estado), **113**
homeostático, **16**
Homo sapiens, 1, 823
Hood, A., 11, *1048*

Hooker, J., 207, *1048*
Hoos, H. H., 142, 206, 559, 719, *1037*, *1048*, *1064*
Hopcroft, J., 720, 977, *1037*, *1061*
Hopfield, J. J., 788, *1048*
Hora punta (rompecabezas), 68
HORIZON (plataforma de aprendizaje por refuerzo), 822
horizonte, 883
  infinito, 596
horizonte (en MDP), **565**
Horn, A., 248, *1048*
Horn, B. K. P., 923, *1048*
Horn, K. V., 410, *1048*
Horn, W., 316, *1042*
Horning, J. J., *1048*
Horowitz, M., 250, *1057*
horquillado parcial, **840**
Horrocks, J. C., 410, *1041*
Horsfall, P., 526, *1036*
Horswill, I., 980, *1048*
Horvitz, E. J., 62, 455, 498, 559, 1019, *1048*, 1057
Hoseini, S. S., 143, *1045*
Hoßfeld, T., 1017, *1048*
Hotelling, H., 787, *1048*
Houde, S., 996, 1009, *1036*
HOUDINI (programa de ajedrez), 176
Houlsby, N., 719, *1066*
Houston, M., 30, *1052*
Hovel, D., 455, *1048*
Howard, J., 716, 720, 879, *1048*, *1054*
Howard, R. A., 544, 558–560, 596, *1048*, *1055*
Howe, A., 344, 380, *1045*
Howe, D., 313, *1048*
Howe, P., 719, 1009, *1055*
Howson, C., 524, *1048*
HPP (Proyecto de Programación Heurística), 23
Hruschka, E., 850, *1055*
HSCP (sistema de planificación), 382
Hsiao, K., 598, *1048*
Hsieh, H.-P., 718, *1066*
HSP (Planificador de búsqueda heurística), 380
Hsu, D., 598, 716, *1035*, *1036*
Hsu, F.-H., 176, *1038*, *1048*
Hsueh, C.-H., 176, *1065*
HTML, 840
HTN (red jerárquica de tareas), **357**, 379, 807
Hu, H., 877, *1039*
Hu, J., 597, 648, *1039*, *1048*
Hu, Y.-T., 916, *1042*
Hua, Y., 1009, *1035*
Huang, A., 19, 27, 30, 176, 177, *1061*
Huang, L., 845, 853, 854, *1045*, *1067*
Huang, S., 30, *1053*
Huang, T., 498, 525, 598, *1044*, *1048*
Huang, X., 29, *1066*
Huang, Y., 787, *1059*
Huang, Z., 786, *1060*
Hubel, D. H., 786, 922, 924, *1048*
Huber, M., 976, 1040
Hubert, T., 27, 30, 155, 174, 177, 178, 820, *1061*
Huddleston, R. D., 853, *1048*
Huet, G., 312, *1036*
Huffman, D. A., 20, *1048*
Huffman, S., 976, *1040*
Hughes, B. D., 138, *1048*
Hughes, G. E., 340, *1048*
Hughes, M., 850, *1039*
HUGIN (sistema de red Bayes), 456, 498
Huhns, M. N., 62, *1048*
Hui, F., 27, 30, *1061*
Hume, D., 6, *1048*
Humphrys, M., 984, *1048*
Hunkapiller, T., 498, *1035*
Hunsberger, L., 637, 649, *1048*
Hunt, J. J., 979, *1053*
Hunt, W., 312, *1048*
Hunter, L., 748, *1048*
Huq, A., 1009, *1040*
Hur, C.-K., 527, *1048*
Hurst, M., 855, *1048*
Hurst, S., 820, *1060*
Hurwicz, L., 649, *1048*
Hussein, A. I., 682, 684, *1050*
Hut, P., 30
Hutchinson, B., 995, *1055*
Hutchinson, S., 61, 978, 980, *1039*
Huth, M., 279, *1048*
Huttenlocher, D., 922, *1048*
Hutter, F., 381, 719, 787, *1043*, *1048*, *1061*, *1064*
Huygens, C., 408, 647, *1049*
Huyn, N., 107, *1049*
Huynh, V. A., 598, *1049*
Hwa, R., 853, *1049*
Hwang, C. H., 339, *1049*
Hyafil, L., 716, *1049*
Hyun, S., 977, *1041*

I

I.I.D. (independiente e idénticamente distribuido), **665**, 722
IA a la antigua usanza (GOFAI), **982**
IA a nivel humano, **32**
IA amigable, 1010
IA débil, **981**, 1005, 1006
IA explicable (XAI), 719, **997**
IA fuerte, **981**, 1005, 1006
IA invierno, 24, 27
IA para el bien social, 986
IA para la acción humanitaria, 986
Iagnemma, K., 977, *1038*
Iatauro, M., 29, *1035*
IBAL (lenguaje de programación probabilístico), 526
Ibarz, J., 769, 978, 979, *1037*, *1046*, *1052*
IBM, 19, 1008, 1017
identificabilidad, **743**
identificación en el límite, 716
identificación lingüística, **826**
idiota Bayes, 402
IEEE P7001, 997
Ieong, S., 648, 1049
Iglesia, A., 9, 279, 282, 310, *1039*
Iglesia, K., 851-853, 855, *1039*
ignorancia, **386**
igualdad (en lógica), 264, 306
Iida, H., 176, *1060*
III, H. D., 995, *1045*
IJCAI (Conferencia Internacional Conjunta sobre IA), 35, 109
ILOG (sistema de programación lógica de restricción), 312
ILQR (LQR iterativo), **955**, 975, 978
imagen, **882**
  formación, 882-888
  segmentación, 894–895
imágenes por resonancia magnética funcional (fMRI), 11, 253
ImageNet (conjunto de datos de imágenes), 25, 26, 28, 896, 1020
ImageNet, **705**
IMDB, 991
implicación, **217**
imputación, **627**
incentivo, **601**
incertidumbre, 23, 315, 385–411
  enfoque basado en reglas para, 459
  existencia, **507**
  identidad, **507**
  relacional, **505**
  resumiendo, 386
  y el tiempo, 461-465
incertidumbre de control, 957
incertidumbre de identidad, **507**
Incertidumbre de la existencia, **507**
incompletitud, 296
  teorema, 9, 305, 983
incrustación de baja dimensión, **937**
incrustación de palabras, 783, 828, **857**
  posicional, **870**
Incrustación de vecino estocástico distribuido en t (t-SNE), **709**, 719
independencia, **398**, 397-399, 401, 407
  absoluto, 398, 401
  condicional, **401**, 406, 408, 415, 418–427, 454, 469
  específico del contexto, **420**
  marginal, 398

independencia de parámetros, **732**
independencia de submetas, **356**
independencia de utilidad mutua (MUI), **544**
independencia específica del contexto, 504
independencia preferencial mutua (IPM), **543**
independencia preferencial, **543**
indexación de predicados, **284**
indexación semántica latente, 852
Indexación, **284**, 284–286
indexical, **846**
India, 16, 204, 338
índice de desvíos, **91**
Índice de Gittins, **583**
Índice de IA, **27**
indiferencia, principio de, **409**
individuación, **321**
inducción, **6**, 652
  matemáticas, 9
inducción hacia atrás, **614**
inductivo, 410
Induráin, E., 558, *1036*
Indyk, P., 717, *1034*, *1045*
inferencia, **209**
  probabilística, **395**, 395–397, 412
inferencia del lenguaje natural, 880
inferencia lógica, 215, 280–313
inferencia por enumeración, 427
Inferencia Probabilística Simbólica (SPI), 456
inferencia que preserva la verdad, **216**
inferencia temporal, 465-473
información imperfecta, 174, 618, **619**
Información perfecta, **147**, **618**
Informe Alvey, 23
Informe Lighthill, págs. 21, 23
Ingeniería de Seguridad, **1001**
Ingeniería de Software, 1002
Ingeniería informática, 14–15
Ingeniería Ontológica, **314**, 314–316
Ingerman, P. Z., 852, 1049
Ingham, M., 249, 1065
inobservabilidad, 51
insatisfacción, 246
insesgado (estimador), **536**
inspector de higiene de restaurantes, 170
instancia (de un esquema), **117**
Instanciación existencial, **281**
instanciación universal (UI), **280**
Instituto AI Now, 995, 1008
Instituto del Futuro de la Humanidad, 1008
Instituto del Futuro de la Vida, 1008 El futuro del trabajo, 998–1000, 1011
integral de trayecto, **949**
integridad
  de la resolución, 226, 303–306
  de un algoritmo de búsqueda, **75**, 105
  de un procedimiento de prueba, **216**, 222, 246
Inteligencia, **1**, 36
inteligencia artificial (IA), **1**
Inteligencia Artificial, 1–1022
  aplicaciones de, 27–30
  conferencias, 35
  débil, **981**, 1005, 1006
  en tiempo real, **1019**
  ética, 986-1001
  filosofía de, 981-1011
  fuerte, **981**, 1005, 1006
  fundaciones, 5–17, 819
  futuro de, 31–34, 1012–1022
  historia de, 17–27
  lenguaje de programación, 19
  objetivos de, 1020-1021
  posibilidad de, 981–984
  probadamente beneficioso, **5**
  revistas, 35
  riesgos, 31–34, 987–996
  seguridad, 1001–1005
  sociedades, 35
interacción humano-robot, 964, 979
interacción persona-computadora, 14
Interfaz cerebro-máquina, **11**, 971
Interfaz de usuario (creación de instancias universales), **280**
interpolación (de datos), **668**
Interpretabilidad, **711**, 719
interpretación, **258**, 277
  ampliado, **261**, 277
  previsto, **258**
interpretación semántica, 843-844, 853
interpretaciones en, 257-259
intervalo de tiempo, 340
intervalo, 324–325
Intille, S., 498, *1049*
intratable, 21
introspección, **2**, 13
invariancia temporal, 760
invarianza, temporal, 760
inversa (de una matriz), **1026**
investigación de operaciones, **10**, 61, 107, 108
Ioffe, S., 786, *1049*
IPL (lenguaje de programación), 18
IR (recuperación de información), **850**, 854
Irpan, A., 784, 820, 978, *1037*, *1049*
irracionalidad, 1, 531, **560**
Irving, G., 312, *1034*, *1053*
Isard, M., 498, *1049*
Isbell, C., 341, 1014, *1053*
ISBN, 507
Isele, R., 339, *1052*
ISIS (sistema de planificación), 383
Islam, R., 820, *1044*
Isola, P., 879, 914, 915, *1049*, *1067*
iteración de valor, **572**, 572–576, 595
  basado en puntos, 598
ITERACIÓN DE POLÍTICAS, **577**
ITERACIÓN DE VALOR, **573**
Ivanov, V., 992, *1037*
Iwasawa, S., 1006, *1058*
iWeb (corpus lingüístico), 825
IxTeT (sistema de planificación), 383
Iyyer, M., 879, *1058*

J

Jaakkola, T., 458, 787, *1049*, *1060*
Jabbari, S., 1009, *1036*
JACK (programa puente), 178
Jackel, L., 26, 786, 922, *1052*
Jackson, C., 525, 747, *1054*
Jacobi, C. G., 526
Jacobs, D., 907, *1050*
Jacobson, D. H., 978, *1049*
Jacquard, J., 15
Jaderberg, M., 30, 179, 787, *1049*
Jaffar, J., 312, *1049*
Jaggi, M., 787, *1066*
Jaguar, 383
Jain, A., 30, 854, 872, *1053*, *1057*, *1064*
Jain, B., 822, *1060*
Jain, D., 525, *1049*
Jaitly, N., 849, 854, *1039*, *1048*
Jakob, M., 339, *1052*
James, G., 720, *1049*
James, W., 13
Jankowiak, M., 526, *1036*
Janz, D., 719, *1062*
Janzing, D., 458, *1058*
Japón, 23, 976
jaque mate, 169
Jarrett, K., 786, *1049*
Jasidim, A., 1018, *1047*
Jasra, A., 499, *1042*
Jastrzebski, S., 716, *1035*
Jaumard, B., 248, *1047*
Jauvin, C., 878, *1036*
Jaynes, E. T., 394, 409–411, *1049*
Jeffrey, R. C., 409, 558, *1049*
Jeffreys, H., 851, *1049*
Jelinek, F., 851, 855, 880, *1038*, *1049*
Jenkin, M., 980, *1043*
Jenkins, G., 497, 787, *1037*
Jenkins, N. W., 30, *1042*
Jennings, H. S., 13, *1049*
Jennings, N. R., 648, *1059*
Jenniskens, P., 372, *1049*
Jensen, F. V., 455, 456, 460, *1034*, *1049*
Jensen, F., 455, 456, 560, *1034*, *1042*, *1056*
Jentzsch, A., 339, *1052*
Jeopardy, 26, 30
jerarquía de abstracción, 382
jerarquía taxonómica, 23, **317**, 339
Jevons, W. S., 248

Ji, Z., 1009, *1049*
Jiang, H., 994, *1049*
Jiang, K., 979, *1034*
Jiao, J., 787, *1067*
Jie, K., 30
Jiménez, P., 144, 383, *1049*
Joaquín, T., 718, 852, *1049*
Job, J., 1018, *1056*
Johansen, A. M., 499, *1042*
Johanson, M., 30, 178, 647, *1037*, *1056*, *1067*
Johnson, C. R., 62, *1038*
Johnson, D. S., 1025, 1029, *1045*
Johnson, I., 719, *1065*
Johnson, J., 1018, *1064*
Johnson, M., 850, 853, 854, 1006, *1039*, *1043*, *1049*, *1052*
Johnson, S. M., 581, 597, *1037*
Johnson, W. W., págs. 106, *1049*
Johnston, M. D., 142, 206, 384, *1049*, *1055*
Jonathan, P. J. Y., 984, *1049*
Jones, D. M., 597, *1045*
Jones, G., 458, *1038*
Jones, L., 850, 868, 880, *1039*, *1064*
Jones, M., 560, 922, *1049*, *1064*
Jones, R. M., 311, *1049*, *1066*
Jones, R., 855, *1049*
Jones, T., 61, *1049*
Jonsson, A., 29, 383, *1049*
Jordan, M. I., 458, 498, 499, 598, 728, 748, 786, 787, 812, 817, 821, 845, 852, 979, *1037*, *1049*, *1053*, *1056*, *1060-1062*, *1065*
Jordan, M., 821, *1061*
José, A. D., 1010, *1035*
Joshi, M., 876, 879, *1053*
Jouannaud, J.-P., 312, *1049*
Joulin, A., 852, *1049*
Jouppi, N. P., 1018, *1049*
Jozefowicz, R., 787, 878, 959, 979, *1034*, *1049*
Juang, B.-H., 497, *1059*
Judá, K., 561, *1043*
juego, 10, 146, 175
  ajedrez, 5, 14, 21, 30, 46, 107, 147, 155–158, 176
  asistencia, **34**, 623, 815
  backgammon, 164, 178, 804, 815
  billar, 179
  colonos de Catán, 151
  cooperativa, 626
  dados, 171
  damas, 19, 62, 177, 819, 820
  de información imperfecta, **146**
  de información perfecta, **147**
  decisiones óptimas en, 148-152
  del azar, 164-168
  diplomacia, 151
  estocástico, **164**
  física, 179
  forma normal, **605**
  información incompleta, 561
  ir, 161
  juego de inspección, **606**
  Kriegspiel, **168**
  multijugador, 151–152
  Otelo, 178
  parcialmente observable, 168-172
  póker, 178, 648
  principal–agente, **648**
  repetidas, 608, **614**
  Reversi, 178
  Scrabble, 179
  suma cero, **147**, **610**
  Tetris, 572, 581
  Yahtzee, 168
juego cooperativo, **601**, 645
juego de apuestas, 394
Juego de grapas, **624**
Juego de cortes de ciclo, **201**
juego de etapa, **614**
Juego de forma normal, 605
juego de información incompleta, **961**
Juego de Markov, 648
juego de soportes, **308**
Juego de suma cero, **147**, 600, **610**
Juego del ultimátum, **642**
juego físico, 179
juego iterado, 614
juego no cooperativo, 601, 645
juego repetido, 608, **614**
juegos de azar, 8, 531
juegos de cartas, 171
Juels, A., 143, 1009, *1049*, *1064*
jugador (en un juego), **605**
Jugador de fila, **606**
Jugador ficticio, **629**
Jugadores simétricos, **629**
juicio humano, 538
Julesz, B., 921, *1049*
Juliani, A., 822, *1049*
Juliano, K. D., 598, *1049*
Jung, M. W., 822, *1052*
Junker, U., 312, *1049*
junta prismática, **929**
Junta Revolute, **929**
Jurafsky, D., 840, 849, 852, 855, *1049*, *1062*
Just, M. A., 253, *1055*
justificación (en un SMC), **336**

K

Kaack, L. H., 30, *1059*
Kadane, J. B., 561, 647, *1049*
Kaden, Z., 822, *1045*
Kadian, A., 822, *1060*
Kaelbling, L. P., 249, 498, 527, 596-598, 977, *1039*, *1041*, *1048*, *1049*, *1053*, *1055*, *1061*, *1063*
Kager, R., 851, *1049*
Kaggle, 698
Kahn, H., 457, 821, *1049*
Kahneman, D., 418, 538, 560, *1049*, *1064*
Kaindl, H., 109, *1049*
Kaiser, L., 850, 868, 877, 879, 880, *1050*, *1064*
Kalakrishnan, M., 978, *1037*
Kalchbrenner, N., 779, 787, 849, *1064*
Kale, A. U., 30, *1053*
Kaliszyk, C., 312, 313, *1046*, *1053*
Kalman, R., 479, 497, *1050*
Kalra, N., 142, *1042*
Kalyanakrishnan, S., 27, *1063*
Kalyanpur, A. A., 30, *1043*
Kamar, E., 27, *1063*
Kamber, M., 720, *1047*
Kambhampati, S., 144, 381-384, *1038*, *1040*, *1042*, *1050*
Kameya, Y., 525, *1060*
Kaminka, G., 649, *1063*
Kan, A., 107, 377, 384, *1052*
Kanada, K., 30, *1053*
Kanade, T., 28, 922, 923, *1050*, *1060*, *1064*
Kanal, E., 990, *1050*
Kanal, L. N., 108, *1051*
Kanazawa, A., 907, 913, *1050*
Kanazawa, K., 498, 596, 598, 748, *1036*, *1041*, *1044*, *1050*, *1060*
Kanefsky, B., 206, *1039*
Kang, S. M., 1013, *1050*
Kannan, A., 849, *1039*
Kannan, K., 996, 1009, *1036*
Kannan, R., 720, *1037*
Kanodia, N., 597, *1046*
k-anonimato, **991**
Kanoui, H., 278, 311, *1040*
Kanoulas, E., 850, *1034*
Kant, E., 311, *1038*
Kant, I., 8
Kanter, J. M., 719, *1050*
Kantor, G., 61, 978, 980, *1039*
Kantorovich, L. V., 143, *1050*
Kanwal, M. S., 716, *1035*
Kaplan, D., 341, *1050*
Kaplan, H., 108, *1045*
Kaplow, R., 598, *1061*
Karaboga, D., 142, *1050*
Karafia't, M., 878, 879, *1055*
Karaletsos, T., 526, *1036*
Karamchandani, A., 457, *1050*
Karlin, S., 581, 597, *1037*
Karlsson, R., 498, *1047*
Karmarkar, N., 143, *1050*
Karmiloff-Smith, A., 854, *1043*
Karp, R. M., 9, 107, 109, *1029*, *1047*, *1050*
Karpas, E., 381, *1047*
Karpathy, A., 786, 862, 879, *1050*, *1060*
Karpatne, A., 720, *1063*
Karras, T., 780, *1050*

Karsch, K., 914, *1050*
Kartam, N. A., 383, *1050*
Kasami, T., 835, 853, *1050*
Kasif, S., 456, *1067*
Kasparov, G., viii, 30, 176
Kassirer, J. P., 410, *1046*
Kataoka, T., 786, *1055*
Katehakis, M. N., 597, *1050*
Katriel, I., 205, *1064*
Katz, B., 850, *1050*
Katz, S., 207, *1040*
Kaufmann, M., 313, *1050*
Kautz, D., 384, *1041*
Kautz, H., 142, 206, 248, 249, 381, 456, *1046*, *1050*, *1060*, *1061*
Kautz, J., 879, *1053*
Kavraki, L., 61, 978, 980, *1039*, *1050*
Kavukcuoglu, K., 779, 784, 786, 787, 790, 820, 822, 849, *1049*, *1053*, *1055*, *1064*
Kawczynski, M. G., 30, *1042*
Kay, A. R., 11, *1057*
Kaynama, S., 821, *1034*
Kazemi, S. M., 527, *1050*
KB (base de conocimientos), **209**, 246
KB-AGENTE, **209**
k-DL (lista de decisión), 675
k-DT (árbol de decisión), 675
Keane, M. A., 143, *1051*
Keane, P. A., 30, *1053*
Kearns, M., 597, 598, 717, 718, 720, 820, 1009, *1036*, *1050*
Kebeasy, R. M., 682, 684, *1050*
Kedzier, D., 820, *1060*
Keeney, R. L., 539, 544, 559, 560, *1050*
Kegelmeyer, W. P., 707, 995, *1039*
Keil, F. C., 3, 1007, *1065*
Kelcey, M., 978, *1037*
Kelley, H. J., 22, 785, *1050*
Kelly, J., 649, 747, 748, *1038*, *1039*
Kelly, K., 1005
Kembhavi, A., 880, *1061*
Kemp, C., 560, *1046*
Kemp, M., 920, *1050*
Kempe, A. B., 983, *1050*
Kenley, C. R., 455, *1061*
Kephart, J. O., 61, *1050*
Kepler, J., 920
Keras (software de aprendizaje automático), 720, 1021
Kern, C., 30, *1053*
kernel (en redes neuronales), kernel **760**
kernel (en regresión), 691
kernel de transición, **444**
Kernelización, **696**
Kernighan, B. W., 107, *1053*
Kersting, K., 525, 527, *1050*, *1055*
Keskar, N. S., 880, *1050*
Keynes, J. M., 409, 998, *1050*
Khairy, K., 716, *1054*
Khanna, R., 719, *1050*
Khare, R., 339, *1050*
Khatib, O., 978, 980, *1050*, *1059*, *1061*
Khorsand, A., 109, *1049*
Khosla, A., 786, *1060*
Khot, T., 850, 876, 880, 910, *1040*, *1046*
Khudanpur, S., 878, 879, *1055*
Kichkaylo, T., 29, *1035*
Kim, B., 719, *1042*, *1050*
Kim, H. J., 817, 821, 979, *1056*
Kim, J. H., 454, *1050*
Kim, J.-H., 982, *1050*
Kim, T. W., 1009, *1050*
Kimmig, A., 527, *1050*
Kinect, 928
Kingma, D. P., 787, *1050*
Kingsbury, B., 849, 854, *1048*
Kinsey, E., 106
Kirchlechner, B., 525, *1049*
Kirchner, C., 312, *1049*
Kirk, D. E., 61, *1050*
Kirk, J. T., 1007
Kirkpatrick, S., 142, 206, *1050*
Kirman, J., 597, *1041*
Kiros, J. R., 786, *1035*
Kirubarajan, T., 61, *1035*
Kishimoto, A., 177, *1060*
Kisiel, B., 850, *1055*
Kisynski, J., 527, *1050*
Kitaev, N., 853, 877, 879, *1050*
Kitani, K. M., 817, *1050*
Kitano, H., 976, *1050*
Kitchin, D. E., 381, *1064*
Kjaerulff, U., 498, *1050*
Klarman, H. E., 559, *1050*
Klein, D., 845, 852, 853, 877, *1050*, *1053*, *1057*
Kleinberg, J. M., 994, 1009, *1050*
Klemperer, P., 649, *1050*
Klempner, G., 455, *1056*
Kneser, R., 852, *1050*
Knoblock, C. A., 108, 344, 380, 382, *1039*, *1045*, *1050*
KNOWITALL (sistema de extracción de información), 855
Knuth, D. E., 68, 175, 249, 312, 978, *1046*, *1050*
Ko, J., 30, *1043*
Kober, J., 979, *1050*
Kobilarov, G., 316, 339, *1037*
Koch, C., 1007, *1040*, *1050*
Kochenderfer, M. J., 598, 822, *1049*, *1051*
Kociemba, H., 106, *1059*
Kocsis, L., 176, 597, *1051*
Koditschek, D., 979, *1051*
Koehn, P., 880, *1051*
Koelsch, S., 920, *1051*
Koenderink, J. J., 923, *1051*
Koenig, S., 145, 381, 383, 596, 977, *1047*, *1051*, *1062*
Kohlberger, T., 30, *1053*
Kohli, P., 527, *1051*
Kolesky, D. B., 1013, *1051*, *1056*
Kollar, T., 979, *1063*
Koller, D., 176, 410, 455, 460, 498, 499, 525–527, 561, 597, 621, 647, 747, 748, 852, 977, *1035–1037*, *1039*, *1044*, *1046*, *1048*, *1050*, *1051*, *1055*, *1057*, *1058*, *1060*, *1063*
Kolmogorov, A. N., 409, 410, 497, 716, *1051*
Kolobov, A., 526, 598, *1054*, *1055*
Kolter, J. Z., 817, *1051*
Koltun, V., 822, *1060*
KOMODO (programa de ajedrez), 176
Kondrak, G., 205-207, *1051*
Konečný, J., 992, *1051*
Konolige, K., 206, 342, 646, 977, 978, 980, *1037*, *1038*, *1051*
Kononova, O., 872, *1064*
Kontokostas, D., 339, *1052*
Koopmans, T. C., 596, *1051*
Korb, K. B., 460, *1051*
Koren, S., 717, *1036*
Koren., Y., 978, *1037*
Korf, R. E., 98, 106-109, 145, 175, 381, *1043*, *1051*, *1058*
Kortenkamp, D., 976, *1040*
Koss, F., 976, *1040*
Kotthoff, L., 719, *1048*
Koutsoupias, E., 142, 248, *1051*
Kovacs, D. L., 380, *1051*
Kowalski, R., 278, 294, 299, 311, 340, *1051*, *1060*
Koyama, M., 786, *1055*
Koyejo, O. O., 719, *1050*
Koyejo, S., 916, *1042*
Koza, J. R., 143, *1051*
Kramer, S., 525, *1050*
Kraska, T., 311, *1051*
Kraus, S., 27, 649, 650, *1051*, *1063*
Kraus, W. F., 143, *1053*
Krause, A., 561, *1051*
Krause, J., 786, *1060*
Krauss, P., 524, *1061*
Krawiec, K., 143, *1062*
Kreitmann, P., 1009, *1036*
Kretch, K. S., 801, *1034*
Kreuter, B., 992, *1037*
Kriegspiel, **168**
Krikun, M., 29, 783, 850, 865, *1066*
Kripke, S. A., 340, *1051*
Krishna, V., 649, *1051*
Krishnamurthy, V., 598, *1051*
Krishnan, A., 498, *1053*
Krishnan, T., 748, *1055*
Krishnanand, K., 142, *1051*
Krizhevsky, A., 26, 786, 787, 922, 979, *1051*, *1052*, *1062*
Krogh, A., 498, 787, *1051*

Krueger, D., 716, *1035*
Kruppa, E., 921, *1051*
Ktesibios de Alejandría, 15
Ku ̈bler, S., 853, *1051*
Kübler, H., 822, *1036*
Kuffner, J. J., 978, *1051*
Kuhlmann, G., 822, *1063*
Kuhn, H. W., 517, 526, 647, *1051*
Kuipers, B. J., 342, 977, *1051*
Kulkarni, T., 527, *1051*
Kumar, M. P., 716, *1036*
Kumar, P. R., 61, *1051*
Kumar, S., 15, 850, 1018, *1051*, *1066*
Kumar, V., 108, 720, 990, *1039*, *1051*, *1063*
Kumaran, D., 27, 30, 155, 174, 177, 820, 822, *1055*, *1061*
Kuniyoshi, Y., 976, *1050*
Kuo, W.-C., 908, *1044*
Kuppuswamy, N., 982, *1050*
Kuprel, B., 30, *1043*
Kurakin, A., 787, *1039*
Kurien, J., 144, *1051*
Kurth, T., 30, *1052*
Kurzweil, R., 12, 1004, 1005, 1010, *1052*
Kwok, C., 854, *1052*

L

La apuesta de Pascal, 409
La Mettrie, J. O., 1001, 1006, *1052*
La Mura, P., 559, *1052*
la peor catástrofe posible, 533
La venganza de Moctezuma, 816
Laborie, P., 384, *1052*
Lacoste-Julien, S., 716, *1035*
Ladkin, P., 340, *1052*
Lafferty, J., 855, *1052*
Lago, B., 526, *1052*
Lago, R., 177, *1060*
Lagoudakis, M. G., 647, *1046*
Laguna, M., 142, *1045*
Lahiri, S., 787, *1058*
Lai, J. C., 878, *1038*
Lai, M., 27, 30, 155, 174, 177, 820, *1061*
Lai, T. L., 585, 597, *1052*
Laine, S., 780, *1050*
Laird, J., 292, 311, 382, *1049*, *1052*, *1066*
Laird, N., 497, 748, *1042*
Laird, P., 142, 206, *1055*
Lakemeyer, G., 977, *1038*
Lakoff, G., 339, 853, 1006, *1052*
Lally, A., 30, *1043*
Lam, J., 179, *1052*
Lamarck, J. B., 118, *1052*
Lample, G., 879, *1052*
Lamure, M., 559, *1036*
Lanctot, M., 27, 30, 155, 174, 177, 820, *1061*
Landauer, T. K., 852, 878, *1042*
Landhuis, E., 538, *1052*
Landolin, J. M., 717, *1036*
Lang, J., 649, *1037*
Langdon, W., 144, *1052*, *1058*
Lange, D., 822, *1049*
Langton, C., 143, *1052*
Lao, N., 316, *1042*
LaPaugh, A. S., 107, *1052*
Laplace, P., 8, 408, 409, 827, 851, *1052*
Larkey, P. D., 647, *1049*
Larochelle, H., 672, 787, *1052*, *1062*
Larsen, B., 456, *1052*
Larson, Carolina del Sur, 716, *1052*
Larson, K., 648, *1060*
Laruelle, H., 383, *1045*
Las predicciones de Simón, 21
Laskey, K. B., 526, 560, *1052*
Lassez, J.-L., 312, *1049*
Lassila, O., 339, *1036*
Latham, D., 822, *1054*
Latombe, J.-C., 383, 977-979, *1042*, *1050*, *1052*, *1067*
Laugherty, K., 853, *1046*
Laurent, C., 849, *1067*
Lauritzen, S., 455, 456, 558, 559, 747, 748, *1040*, *1052*, *1057*, *1062*
LaValle, S., 384, 978, 980, *1051*, *1052*
Lave, R. E., 598, *1060*
Lavie, A., 853, *1060*
Lawler, E. L., 107, 108, 377, 384, *1052*
Lazanas, A., 979, *1052*
L-BFGS (algoritmo de optimización), etiqueta 717
LCF (Lógica para Funciones Computables), 278
Le Truong, H., 313, *1046*
Le, Q. V., 29, 718, 783, 787, 849, 850, 865, 879, 880, 1021, *1041*, *1058*, *1059*, *1061*, *1063*, *1066*, *1067*
Le, T. A., 457, 527, *1052*
Leacock, C., 338, *1038*
Leaper, D. J., 410, *1041*
Lebedev, M. A., 11, *1052*
Lecoutre, C., 207, *1052*
LeCun, Y., 17, 26, 718, 786–788, 852, 922, 1016, 1018, *1036*, *1039*, *1049*, *1052*, *1064*, *1067*
Lederberg, J., 22, 23, 338, *1043*, *1053*
Ledsam, J. R., 30, *1053*
Lee, A. X., 978, *1061*
Lee, B. K., 920, *1060*
Lee, C.-H., 982, *1050*
Lee, D., 458, 527, 747, 822, *1039*, *1052*
Lee, G. A., 11, *1067*
Lee, J. D., 786, *1042*
Lee, K. C., 979, *1042*
Lee, K., 30, 879, *1042*, *1053*, *1058*, *1059*
Lee, K.-F., 1, 35, *1052*
Lee, K.-H., 982, *1050*
Lee, M. S., 747, *1056*
Lee, R. C.-T., 313, *1039*
Lee, S.-I., 719, *1053*
Lee, S.-W., 498, *1063*
Lee, T.-M., 11, *1057*
Lee, W. S., 598, *1035*
LEELAZERO (programa de juego), 172
Lefrancq, A., 822, 1003, *1036*, *1052*, *1063*
Legendre, A. M., 717, *1052*
Legg, S., 820, 822, 1003, *1036*, *1052*, *1055*
Lehmann, D., 649, *1051*
Lehmann, J., 316, 339, *1037*, *1052*
Lehrer, J., 560, *1052*
Lehtinen, J., 780, *1050*
Leibniz, G. W., 6, 120, 247, 409, 647
Leibo, J. Z., 822, *1036*
Leighton, M. J., 108, *1038*
Leike, J., 822, 1003, *1052*
Leimer, H., 456, *1052*
Leipzig, 12
Leiserson, C. E., 107, 1029, *1040*
Lelis, L., 109, *1052*
Lema de levantamiento, 303, **306**
Lenat, D. B., 316, 339, 646, *1052*
lenguaje, 833
  comprensión, 20, 23
  en desambiguación, 848
  encriptado, **874**
  formal, 823
  fuente, **864**
  modelo, **824**, 860
  naturales, 4, 252, 823
  objetivo, **864**
  procesamiento, 17, 823–880
Lenguaje de Descripción de Dcciones (ADL), 380
lenguaje de programación, 251
Lenstra, J. K., 107, 206, 377, 384, *1034*, *1052*
Lente, **884**
Lenzerini, M., 341, *1038*
Leonard, H. S., 340, *1052*
Leonard, J., 977, *1037*, *1052*
Leone, N., 207, 342, *1043*, *1046*
Lepage, G. P., 457, *1052*
Lerman, K., 1009, *1055*
Lerner, U., 455, *1052*
Lesh, N., 383, *1043*
Leśniewski, S., 340, *1052*
Lesser, V. R., 646, 649, *1043*, *1052*
Letz, R., 312, *1052*
Levasseur, L. P., 652, *1047*
Lever, G., 30, 179, *1049*
Levesque, H. J., 142, 249, 341, 343, 646, *1037*, *1040*, *1052*, *1061*
Leviatán, 6
Levin, D. A., 497, *1052*
Levine, S., 719, 790, 821, 978, 979, *1034*, *1037*, *1044*, *1052*, *1060*, *1061*
Levinstein, B., 1002, *1034*

Levitt, R. E., 383, *1050*
Levitt, T. S., 977, *1051*
Levskaya, A., 877, 879, *1050*
Levy, D., 178, 982, 1006, *1052*
Levy, O., 876-880, *1046*, *1052*, *1053*, *1065*
Lew, A. K., 526, 527, *1040*
Lewis, A., 142, *1055*
Lewis, J. A., 1013, *1051*, *1056*
Lewis, M., 876, 879, *1053*
Léxico, **835**, 853
Ley de Control, **952**
Ley de Moore, **14**
Ley de Zipf, 878
Leyes de la robótica, 1007
Leyes del pensamiento, 3–4
Leyton-Brown, K., 648, 650, 719, *1052*, *1061*, *1064*
LFG (gramática léxico-funcional), 853
Li, B., 849, *1039*
Li, C. M., 248, *1052*
Li, D., 850, *1034*
Li, H., 176, 786, *1042*, *1065*
Li, K., 26, 787, *1042*, *1052*
Li, L., 499, 527, *1043*, *1066*
Li, L.-J., 26, *1042*
Li, M., 717, *1053*
Li, P., 458, 498, 527, 747, *1039*, *1053*
Li, S., 707, 995, *1047*
Li, T.-M., 1016, *1053*
Li, W., 879, 978, *1053*, *1059*
Li, X., 142, *1053*
Li, X.-R., 61, *1035*
Li, Y., 786, 822, 879, *1034*, *1063*
Li, Z., 498, *1053*
Liang, G., 455, *1039*
Liang, J., 119, 143, 1055
Liang, P., 845, 880, *1053*, *1059*
Liang, Y., 822, *1045*
LIBBI (lenguaje de programación probabilístico), 527
Liberatore, P., 249, *1053*
Libratus (programa de póker), 172, 178, 622
libre albedrío, 6
licitador, **634**
LiDAR de escaneo, **928**
LIDAR, 917
Lidar, D., 1018, *1056*
Liébana, D. P., 176, *1038*
Lifchits, A., 854, *1057*
Lifschitz, V., 342, 343, *1045*, *1053*, *1064*
Liga de la Justicia Algorítmica, 995
Lighthill, J., 21, *1053*
Likhachev, M., 145, *1051*
Lillicrap, T., 27, 30, 178, 979, *1047*, *1053*, *1061*
Lim, G., 316, *1062*
LIME (sistema de aprendizaje automático explicable), 712, 719
limitación de recursos, **189**, **374**
límite de decisión, **682**
límite de medios, **615**
límite de profundidad, 158
límite inferior de la evidencia (ELBO), **778**
límite inferior variacional, **778**
límite superior de confianza (UCB), **585**
Lin, D., 854, *1057*
Lin, J., 850, 854, *1035*
Lin, S., 107, 649, *1053*, *1065*
Lin, T., 316, *1062*
Lin, Y., 498, *1066*
Linares López, C., 381, *1064*
Lindley, D. V., 560, *1053*
Lindsay, R. K., págs. 338, *1053*
Lindsten, F., 499, *1053*
línea de base, 879, **902**
lineal-gaussiano, **422**, 455, 479, 515, 728
linealización, **935**
líneas paralelas, 883
Lingüística Computacional, **16**, 853
Lingüística, 16-17, 823
Linneo, 339
Lipton, Z. C., 1009, *1049*
líquido (física ingenua de), 342
Lisp, **19**, 259, 344
lista abierta, *véase* frontera
lista cerrada, 72
lista de borrado, **345**
Lista de decisiones, **674**
Lista de encuadernación, **266**
listas (axiomas lógicos para), **269**
Lisy', V., 30, 178, *1056*
literal (oración), **217**
Literal negativo, 217
literal positivo, 217
literal, observado, 248
literales complementarios, **226**
Littman, M. L., 143, 341, 383, 596, 597, 648, 821, 822, 1014, *1034*, *1039*, *1048*, *1049*, *1053*, *1054*
Lituiev, D., 30, *1042*
Liu, B., 820, *1053*
Liu, H., 787, 920, *1053*, *1054*
Liu, J., 498, 499, 820, 822, *1053*, *1060*
Liu, L. T., 1009, *1053*
Liu, M.-Y., 879, *1053*
Liu, P. J., 879, *1059*
Liu, W., 747, *1039*
Liu, X., 30, *1053*
Liu, Y., 30, 876, 879, *1053*, *1062*
Livescu, K., 498, *1053*
Ljung, G. M., 497, 787, *1037*
Lloyd, J., 719, *1062*
Lloyd, S., 1018, 1020, *1047*, *1053*
Lloyd, W. F., págs. 649, *1053*
Llull, R., 6, *1053*
Löwenheim, L., 278, *1053*
Lo, H.-Y., 718, *1066*
Lobo, T., 879, *1060*
LoBue, V., 801, *1034*
locales, 115
localidad, **239**
localización Montecarlo, **934**
localización y mapeo simultáneos (SLAM), 494, **935**
localización, **133**, 476, **932**
Locke, J., 6
Lockhart, E., 178, *1061*
Loftus, E., 253, *1053*
Logemann, G., 233, 248, *1041*
lógica, 3, 214-217
  átomos, 260
  igualdad en, 264
  incumplimiento, **334**, 338, 341
  inferencia, 280-282
  primer orden, **251**, 251–279
  semántica, 256
  sintaxis, 256
lógica de descripción, **329**, **331**, 331–333, 337, 341
lógica de orden superior, **255**
lógica de Port-Royal, 557
lógica de primer orden, **251**, 251–279
lógica de probabilidad, **524**
lógica difusa, 214, 255, 459
lógica formal, **8**
lógica inductiva, 410
lógica modal, 327
lógica no monotónica, 225, 333, 333-335,
lógica predeterminada, **334**, 338, 341
lógica proposicional, 208, 217-222, 246, 251
lógica temporal, **255**
lógica temporal lineal, **328**
lógicamente exhaustivo, 386
logicismo, **3**
LOGISTELLO (programa Otelo), 160
Lohia, P., 996, 1009, *1036*
Lohn, J. D., 143, *1053*
longitud mínima de descripción (MDL), **671**, 716, 724
Longuet-Higgins, H. C., 923, *1053*
LOOCV (validación cruzada de dejar una salida), **666**
Loos, S., 309, 312, *1035*, *1053*
López de Segura, R., 5, *1053*
López, A. M., 822, *1059*
López, P., 719, *1047*
Lopyrev, K., 880, *1059*
Lorentz, R., 176, *1053*
Losey, D. P., 967, *1035*
Lote, **679**
Lotem, A., 381, *1065*
lotería, **530**
  estándar, **533**
Lou, J.-K., 718, *1066*
Lovejoy, W. S., 597, 598, *1053*
Lovelace, A., ix, 15, *1053*
Loveland, D., 233, 248, 312, *1041*, *1053*

Lowe, D., 922, *1053*
Lowerre, B. T., 142, *1053*
Lowrance, J. D., 459, *1060*
Lowry, M., 309, 312, *1047*, *1053*
Loyd, S., 106, *1053*
Lozano-Pérez, T., 598, 977-979, *1038*, *1048*, *1053*, *1065*
LQR (regulador cuadrático lineal), **955**, 975
LQR iterativo (ILQR), **955**, 975, 978
LRTA*-AGENTE, **140**
LRTA*-COSTO, **140**
LRTA*, 139, 145, 365, 580
LSH (hash sensible a la localidad), **689**
LSTM (memoria a corto plazo), **775**, 863
LSVRC, 28
LT (Teórico de la Lógica), 18, 248
Lu, C., 821, *1066*
Lu, F., 977, *1053*
Lu, P., 177, 717, *1038*, *1060*
Lu, Y., 719, *1066*
Luan, D., 879, *1059*
Lubberts, A., 177, *1053*
Luby, M., 114, 457, *1041*, *1053*
Lucas, J. R., 983, *1053*
Lucas, P., 410, *1053*
Lucas, S. M., 176, *1038*
Luce, D. R., 10, 647, *1053*
Ludita, 998
Luehr, N., 30, *1052*
Lugosi, G., 718, *1039*
Lukasiewicz, T., 524, *1053*
Lum, K., 1009, *1043*
LUNAR (sistema de preguntas y respuestas), 85
Lundberg, S. M., 719, *1053*
Lunn, D., 525, 747, *1054*
Luo, S., 920, *1054*
Luong, Q.-T., 923, *1043*
Lusk, E., 313, *1066*
Luu, C., 1009, *1036*
luz, 886–888
  ambiente, **886**
luz estructurada, **928**
Lygeros, J., 61, *1039*
Lyman, P., 719, *1054*
LYNA (sistema de diagnóstico médico), 30
Lynch, K., 61, 978, 980, *1039*, *1054*

M

MA* búsqueda, **95**, 95, 109
Ma, S., 716, 786, *1036*, *1060*
Maas, A. L., 817, *1067*
MacDonald, R., 30, *1062*
MacGlashan, J., 341, 821, 1014, *1048*, *1053*
Macherey, K., 29, 783, 850, 865, *1066*
Macherey, W., 29, 783, 850, 865, *1039*, *1066*
Machina, M., 560, *1054*
Machover, M., 279, *1036*
Maciá', N., 719, *1046*
MacKay, D. J. C., 458, 720, *1054*, *1055*
MacKenzie, D., 313, *1054*
Mackworth, A. K., 61, 187, 204–206, *1054*, *1058*
Macready, W. G., 715, *1066*
macrop (operador de macro), **382**
madaline, 785
Maddison, C. J., 19, 27, 176, 177, *1061*
Madera, D. E., 108, *1052*
Madhavan, R., 979, 1054
Madigan, C. F., 206, 248, 1056
Madry, A., 787, 1039
Magnini, B., 880, *1040*
Magron, V., 313, *1046*
Mahadevan, S., 820, *1053*
Maharaj, T., 716, *1035*
Mahaviracarya, 408
Mahendiran, T., 30, *1053*
Mahesh, A., 30, *1052*
Maheswaran, R., 207, *1057*
Mahlmann, T., 30, 179, *1043*
Maier, D., 206, 311, *1035*, *1036*
Mailath, G., 647, *1054*
Majercik, S. M., 383, *1054*
makespan, **375**
Makov, U. E., 748, *1064*
Maksymets, O., 822, *1060*
Malavé, V. L., 253, *1055*
maldecir
  de dimensionalidad, **688**, 717, 945
  ganadores, **559**
  optimizador, **537**, 559
maldición del ganador, **559**
maldición del optimizador, **537**, 559
Malhotra, P., 990, *1054*
Malik, D., 648, *1054*
Malik, J., 498, 787, 822, 894, 907, 908, 913, 921, 923, 979, *1034*, *1041*, *1044*, *1045*, *1048*, *1050*, *1052*, *1054*, *1060*, *1061*
Malik, S., 206, 248, *1056*
Malone, T. W., 1011, *1054*
MAML (Meta-Aprendizaje Agnóstico del Modelo), 719
Maná, Z., 278, 279, *1054*
Manchak, D., 340, *1064*
Mandal, S., 716, *1036*
Mané, D., 1010, *1034*
manejabilidad de la inferencia, **9**, 332
Maneva, E., 249, *1054*
manipulador, **926**
Manne, A. S., 596, *1054*
Manning, C., 839, 850, 852-854, 872, 878, 880, *1037*, *1039*, *1050*, *1054*, *1057*, *1058*
Mannion, M., 278, *1054*
Mano diestra de la sombra, 930
Hand, D. J., 990, *1037*
Manolios, P., 313, *1050*
Mansinghka, V. K., 526, 527, *1040*, *1046*, *1051*, *1054*, *1060*
Manso, C., 456, *1065*
Mansour, Y., 597, 598, 821, 822, *1050*, *1063*
manta Markov, **419**, 442, 443
manzana, 1008
MAP (máximo a posteriori), **723**, 746, 771
mapa, 708
mapa de características, **764**
mapeo de contracción, **574**
máquina Boltzmann, **788**
máquina de aumento de gradiente (GBM), 701
máquina de estados finitos, 614
máquina de Turing, 9, 716
máquina de soporte vectorial (SVM), **692**, 692–696
máquina del kernel, 692–696, 717
máquina Moore, 647
máquina ultrainteligente, **1004**
Marais, H., 854, 1062
Marbach, P., 821, *1054*
Marcas traseras, 206
Marcedone, A., 992, *1037*
Marcha, 29, **969**, 990
Marcinkiewicz, M. A., 829, 852, *1054*
marco, **23**, 341
Marco, S. I., 597, *1039*
Marcus, G., 560, *1054*
Marcus, B., 456, *1058*
Marcus, M. P., 829, 852, *1054*
margen blando, **696**
margen, **693**
marginación, **396**
Mari Aparici, C., 30, *1042*
Marinescu, R., 457, *1054*
Marín-Urías, L. F., 979, *1062*
Markov, **977**
Markov, A., 463, 497, 851, *1054*
Marler, R. T., 119, *1054*
Maron, M. E., 410, *1054*
Márquez, L., 852, *1054*
Marr, D., 786, 924, *1054*
Marriott, K., 205, *1054*
Marris, L., 30, 179, *1049*
Mars Exploration Rover, ix
Marshall, A. W., 821, *1049*
Marshall, P. J., 652, *1047*
Marsland, S., 720, *1054*
Martelli, A., 107, 144, *1054*
Marthi, B., 382, 499, 512, 525, 526, 822, *1054*, *1055*, *1057*
Martic, M., 822, 1003, *1052*
Martín, D., 921, *1054*
Martín, F. G., 1000, *1054*
Martín, J. H., 849, 852, 853, 855, *1049*, *1054*
Martín, N., 311, *1038*

Martín, S., 70, *1036*
Martino, J., 996, 1009, *1036*
Marx, G., 847
Marzo, J. G., 559, *1047*
Maskell, S., 499, *1035*
Maskin, E., 649, *1041*
Mason, M., 144, 382, 978-980, *1043*, *1053*, *1054*
Mason, R. A., 253, *1055*
Mataric, M. J., 980, *1054*
Mateescu, R., 207, 456, *1041*
Mateis, C., 342, *1043*
matemáticas, historia de, 8–9
Matena, M., 879, *1059*
materialismo, 6
Materzynska, J., 822, *1059*
Mates, B., 247, *1054*
Matheson, J. E., 544, 558, 560, *1048*, *1055*
Matheson, M., 30, *1052*
Mathieu, M., 30, 179, 786, 1018, *1039*, *1064*
Matriz de confusión, **710**
Matriz de covarianza, 1028, **1029**
matriz de datos, **680**
Matriz de ganancia de Kalman, **483**
matriz de identidad (**I**), **1026**
Matriz de pagos, **605**
matriz singular, **1026**
matriz, **1026**
Matsubara, H., 176, *1044*
Mattar, M., 822, *1049*
Matuszek, C., 339, *1054*
Mauchly, J., 14
Mausam., 384, 598, *1040*, *1054*
MAVEN (programa Scrabble), 179
Máxima verosimilitud, **724**, 725–729, 746
máximo
  global, **111**
  local, **112**
maximización de expectativas, 746
máximo a posteriori (MAP), **723**, 746, 771
Maxwell, J., 16, 408, 920, *1054*
Mayer, A., 109, *1047*
Mayer, J., 716, *1054*
Mayne, D. Q., 498, 978, *1047*, *1049*
mayoría simple de votos, **640**
Maziarz, K., 718, 1021, *1061*
MBP (sistema de planificación), 383
McAfee, A., 1011, *1038*
McAleer, S., 106, *1034*
McAllester, D. A., 25, 144, 175, 206, 342, 380, 381, 526, 821, 822, 838, *1034*, *1043*, *1050*, *1051*, *1054*, *1063*
McArthur, N., 1006, *1041*
MCC (Corporación de Microelectrónica y Tecnología Informática), 23
McCallum, A., 527, 854, 855, *1040*, *1044*, *1049*, *1052*, *1054*, *1058*, *1063*
McCann, B., 880, *1050*
McCarthy, J., 17, 18, 32, 60, 247, 249, 278, 317, 341, 381, *1054*
McCawley, J. D., 853, *1054*
McClelland, J. L., 24, 785, *1060*
McClure, M., 498, *1035*
McCorduck, P., 1007, *1054*
McCulloch, W. S., 16, 17, 20, 249, 750, 785, 1054
McCune, W., 308, 313, *1054*
McDermott, D., 144, 311, 329, 340, 341, 344, 380, 383, *1039*, *1045*, *1054*, *1055*
McDermott, J., 23, 292, 311, *1055*
McDonald, R., 254, 839, 853, *1051*, *1057*
McEliece, R. J., 458, *1055*
McGregor, J. J., 205, *1055*
McGrew, B., 959, 979, *1034*
McGuinness, D., 332, 339, 341, *1035*, *1037*, *1062*
McIlraith, S., 278, *1055*
McKenzie, D., 458, *1057*
McKinney, W., 720, *1055*
McLachlan, G. J., 748, *1055*
McLaughlin, S., 313, *1046*
McMahan, B., 561, *1051*
McMahan, H. B., 992, 1009, *1037*, *1051*, *1055*
MCMC (cadena de Markov Monte Carlo), **441**, 441–448, 454, 457, 491, 527, 732
McMillan, K. L., 381, *1055*
McPhee, N., 144, *1058*
MCTS (Búsqueda de árboles de Monte Carlo), **161**
McWhorter, J. H., 278, *1055*
MDL (longitud mínima de descripción), **671**, 716, 724
MDP (proceso de decisión de Markov), 10, **563**, 595, 597, 789
Mecanismo de Vickrey-Clarke-Groves (VCG), **638**
Medea, 975
medición, mecanismo 319–321
medida de rendimiento, **39**, 40, 42, 60, 385, 529
medida, **319**
Medina-Lara, A., 559, *1036*
medio ambiente, **36**, 42–47
  agente único, **44**
  clase, **47**
  competitivo, **45**
  conocidos, **46**
  continuo, **46**
  cooperativa, **45**
  de un solo disparo, **45**
  desconocido, **46**
  determinista, **45**
  dinámico, **45**
  discreto, **46**
  episódico, 45
  estática, **45**
  estocástico, **45**, 562
  historia, 563
  inobservable, **44**
  multiagente, **44**, 599
  no determinista, **45**, 110
  observable, **43**
  parcialmente observable, **43**
  propiedades, 43
  secuencial, **45**
  semidinámico, **46**
  taxi, 42, 43
  virtual, 43
Meehan, J., 311, *1039*
Meehl, P., 338, *1046*, *1055*
Meet (relación de intervalo), 324
meganodo, **435**
megavariable, 473
Meggido, N., 621, 647, *1051*
Mehrabi, N., 1009, *1055*
Mehta, G. B., 558, *1036*
Mehta, S., 995, 996, 1009, 1010, *1036*, *1047*
Mejor premio posible, 533
MEJOR PRIMERA BÚSQUEDA, **73**
mejor respuesta, **608**
mejor respuesta iterada, 610
mejor respuesta miope, **610**
Mellish, C. S., 312, *1040*
Melo, F. S., 179, *1061*
memoria (en redes neuronales), **773**
memoria a corto plazo, 292
memoria a largo corto plazo (LSTM), **775**, 863
memoria a largo plazo, 292
memorización (de datos), 668
memorización, 310
MEMS (microelectromecánicos sistema), 1013
MENACE (algoritmo de aprendizaje), 177
Mendel, G., 118, *1055*
Mendes, P. N., 339, *1052*
Meng, X.-L., 458, *1038*
Mengu¨c „ Y., 1013, *1056*
meningitis, 399
mensaje hacia atrás, 469
mente
  como sistema físico, 6
  teoría de, 2
  visión dualista, 1006
Mercer, J., 695, *1055*
Mercer, R. L., 851, 878, 880, *1038*, *1049*
Merel, J., 822, *1063*
Mereología, **340**
Merkhofer, M. M., 558, *1055*
Merleau-Ponty, M., 1006, *1055*
Mertz, C., 963, 979, *1067*
Meseta (en la búsqueda local), **113**
Meshulam, R., 109, *1043*
metaprendizaje, 719, 960, 979

metáfora, **848**, 853
metanivel espacio de estados, **103**
Metarazonamiento, **173**
  teoría de la decisión, **1019**
metarregla, **298**
meteorito, 372, 385
Método de Newton-Raphson, **121**
método débil, **22**
Método del punto interior, 143
metonimia, **847**
Metrópolis, N., 142, 176, 457, *1055*
Metrópolis-Hastings, **442**
Metz, L., 777, *1059*
Metzen, J. H., 787, *1043*
MEU (utilidad máxima esperada), **387**, 529, 575
Meyerson, E., 119, 143, *1055*
Mézard, M., 142, *1055*
mezcla de gaussianos, **739**, 741, 742
MGONZ (chatbot), 984
MGSS*, 175
MGU (unificador general), **284**, 286, 306
MHT (rastreador de hipótesis múltiples), 526
Mian, I. S., 498, *1051*, *1056*
Michalak, T. P., 648, *1059*
Michel, V., 720, *1058*
Michie, D., 107, 108, 144, 175, 177, 816, 820, 821, 976, *1038*, *1042*, *1055*, *1060*
Microelectrónica e Informática Corporación de Tecnología (MCC), 23
Micromort, **533**, 559
Micromundo, **20**, 21
Microsoft, 455, 986, 1008, 1017
Middleton, B., 421, 455, *1058*
Miesenböck, G., 11, *1067*
Miikkulainen, R., 119, 143, 177, *1053*, *1055*
Mijaylov, S., 312, *1049*
Mikolov, T., 852, 858, 878, 879, *1049*, *1055*
Milch, B., 512, 525-527, *1054*, *1055*, *1057*, *1058*
Milgrom, P., 649, *1055*
Milios, E., 977, *1053*
Mill, J. S., 8, *1055*
Miller, A. C., 558, *1055*
Miller, D., 383, *1041*
Miller, T., 719, 1009, *1055*
MILLIONAIRE (calculadora mecánica), 9
Millstein, T., 381, *1043*
Milner, A. J., 278, *1046*
Mina terrestre, 987
Minami, R., 1013, *1066*
MINI-CONFLICTOS, **198**
minilote, **679**
MINIMAX-SEARCH, **150**
minimización lógica, **319**
mínimo
  global, **111**
  local, **112**
Minka, T., 526, 747, *1047*, *1055*
Minker, J., 311, 343, *1044*, *1055*
Minsky, M. L., 17, 19, 22, 23, 32, 341, 454, 646, 785, 1005, 1007, 1010, *1055*
Minton, S., 142, 206, 382, *1039*, *1055*
Mira ma, sin manos, 18
Miranker, D. P., 206, *1035*
Mirhoseini, A., 718, 1021, *1061*
Mirjalili, S. M., 142, *1055*
Mirza, M., 787, *1046*
Misak, C., 278, *1055*
Mishra, B. D., 876, 880, *1040*
misil guiado, 987
misioneros y caníbales, 338
MIT, 14, 18, 19, 23, 921, 976
Michael, J., 879, 880, *1065*
Mitchell, D., 142, 249, *1040*, *1061*
Mitchell, M., 35, 143, 144, 995, *1055*
Mitchell, T. M., 62, 253, 850, 854, 855, *1037*, *1038*, *1040*, *1055*
MITSUKU (chatbot), 1007
Mittelstadt, B., 987, *1055*
Mitten, L. G., 560, *1055*
Miyake, S., 786, *1044*
Miyato, T., 786, *1055*
Mnih, V., 784, 790, 820, 822, *1055*
MNIST, 25, 668, 897
mobileye, 918, 924
moción, 904-905
modelado cognitivo, 2–3
modelo
  causal, 418
  clase, **653**
  contando, 434
  ponderado, **434**
  selección, **666**, 746
  sensor, 474, 481, 496
  transición, **52**, **65**, 105, 122, **147**, 238, 461, 464, 492, 496, 563, 595, 932
modelo (descripción abstracta de la realidad), 66
  a pequeña escala, 13
modelo (en aprendizaje automático), 651, 653
modelo (en lógica), **214**, 246, 254, 277, 327
modelo (en teoría de la probabilidad), **389**
modelo a nivel de caracteres, **826**, 860
modelo acústico en desambiguación, 849
modelo base, **696**
modelo compartido, **1017**
modelo de actuador, 646
modelo de bolsa de palabras, **824**, 826, 832, 851
modelo de conjunto, **696**
modelo de error gaussiano, **487**
modelo de estado inicial, 464
modelo de expit, **424**
modelo de falla persistente, **488**
modelo de falla transitoria, **488**
modelo de fallo, 488
modelo de lenguaje enmascarado (MLM), **874**
Modelo de negociación de ofertas alternas, **641**
modelo de objetos, 881
modelo de observación, 463
modelo de probabilidad de universos abiertos (oupm), **508**
modelo de probabilidad relacional (rpm), **502**
modelo de renderizado, 881
modelo de retroceso, **827**
modelo de secuencia a secuencia, **865**
modelo de sensor, **52**, 461, 464, 474, 481, 496, 588, 932
modelo de transformador, 833, **868**
modelo de transición, **52**, **65**, 105, 122, **147**, 238, 461, 464, 492, 496, 563, 595, 932
  escasas, 570
modelo de transición dispersa, 570
modelo de urna y bola, 722
modelo dinámico, **951**
modelo discriminativo, **728**, 832
modelo disperso (en aprendizaje automático), **681**
modelo en movimiento, **932**
modelo estándar, **4**
modelo generativo, **727**, 831
modelo gráfico, 412, 460
modelo implícito, **780**
modelo logit, inverso, **424**
modelo mental, en desambiguación, 848
modelo mínimo, **334**
modelo mundial, en desambiguación, 848
modelo n-grama, 779, 826, 832, 851, 852, 856
modelo oculto de Markov (HMM), 25, 461, **473**, 473–478, 485, 497, 744, 830
modelo paramétrico, **686**
modelo ResNet-50, 781
modelos, 256–257
modus ponens, **223**, 247, 308, 310
Moffat, A., 852, *1066*
Mohamed, A. R., 849, 854, *1048*
Mohamed, S., 787, *1059*
Mohr, R., 205, 922, *1055*, *1061*
Mohtashamian, A., 30, *1053*
Moir, T. J., 920, *1061*
Moisés, Y., 341, *1043*
Mojsilovic, A., 995, 996, 1009, 1010, *1036*, *1047*
Momentum (en optimización), **766**
monitoreo de acciones, 372, 373
monotonicidad
  de los costos del trayecto, **89**
  de preferencias, **530**

de un sistema lógico, **225**, 333
Monro, S., 717, *1059*
Montague, P. R., 822, *1061*
Montague, R., 340, 341, 853, *1050*, *1055*
Montanari, U., 144, 204, *1037*, *1054*, *1055*
MONTE-CARLO-LOCALIZATION, **934**
MONTE-CARLO-TREE-SEARCH, **163**
Montemerlo, M., 977, *1055*
Mooney, R., 854, *1055*, *1066*
Moore, A. M., 747, *1056*
Moore, A. W., 142, 596, 747, 820, 979, *1035*, *1037*, *1056*
Moore, E. F., 107, *1056*
Moore, J. D., 1009, *1056*
Moore, J. S., 309, 312, 313, *1037*, *1050*
Moore, R. C., 341, 343, 717, *1048*, *1056*
Moraes, G., 30, *1053*
Moravcˇık, M., 30, 178, *1056*
Moravec, H. P., 976, 977, 1010, *1056*
Morcos, A. S., 30, 179, *1049*
More, T., 18
Morgan, C. L., 143, *1056*
Morgan, J., 646, *1040*
Morgan, T. J. H., 118, 143, *1056*
Morgenstern, J., 995, *1045*
Morgenstern, L., 341, 343, *1041*
Morgenstern, O., 10, 531, 558, 648, *1065*
Moricz, M., 854, *1062*
Moritz, P., 821, 979, *1061*
Morjaria, M. A., 455, *1056*
Morra de dos dedos, 605
Morrill, D., 30, 178, *1056*
Morris, P., 29, 383, *1035*, *1049*
Morrison, E., 175, *1056*
Morrison, P., 175, *1056*
Morsey, M., 339, *1052*
Morstatter, F., 1009, *1055*
Moskewicz, M. W., 206, 248, *1056*
Mossel, E., 249, *1054*
Motor Analítico, 15
motor de escala de oblea (WSE), 15
Motor Diferencial, 15
Mott, A., 1018, *1056*
Mott, J., 852, *1036*
Motwani, R., 637, 717, *1034*, *1045*
Motzkin, T. S., 785, *1056*
Mountney, P., 1018, *1042*
Moutarlier, P., 977, *1056*
moverse, **147**
movimiento aparente, 893
Movimiento asesino, **155**
movimiento lateral (en el espacio de estados), **113**
movimiento primitivo, **960**
MPC (modelo de control predictivo), **956**, 964
MPE, *ver* explicación, más probable
MPI (independencia preferencial mutua), **543**
MRS (sistema de razonamiento metanivel), 298
MST (árbol de expansión mínimo), 109
MT (traducción automática), 849, **864**, 880
MUC (comprensión de mensajes Conferencia), 854
Muchachos, D., 179
Mudd, H. F., 1007
Mudigonda, M., 30, *1052*
Mueller, E. T., 316, 340, *1056*, *1062*
MUESTRA PONDERADA, **440**
MUESTRA PREVIA, **436**
muestreo, 435–441, **1029**
  adaptativo, 457
  correlacionados, **812**, 821
  directos, 436
  rechazo, 437
muestreo de bloques, **447**
muestreo de Gibbs, **442**, 445, 457
muestreo de importancia, **439**
  adaptativo, 457
  secuencial, **491**
muestreo de rechazo, **437**
  adaptativo, 457
muestreo de Thompson, **586**
muestreo secuencial de importancia con remuestreo (SIR), 498
Muggleton, S. H., 854, *1056*
MUI (independencia mutua de servicios públicos), **544**
Muldal, A., 822, *1063*
Mullainathan, S., 994, 1009, *1050*
Müller, M., 177, *1056*, *1060*
Müller, U., 26, 786, 922, *1052*
multiplexor, **505**
Mumford, D., 921, *1056*
mundo de bloques, **20**, 342, 346
mundo del vacío, 37, 39
  errático, 122
  jardín de infantes, 132
  resbaladizo, 125
mundo del vacío de detección local, 130
mundo posible, **214**, 246, 277, **327**, 388, 501
mundo wumpus, **210**, 210–213, 220, 250, 270–271, 315, 404–408, 833
mundos de red de seguridad de IA, 822
Mundy, J., 923, *1056*
MUNIN (sistema de diagnóstico médico), 455
Munos, R., 821, *1056*
Murdock, J. W., 30, *1043*
Muro, R., 853, *1042*
Murphy, K., 316, 497-499, 526, 720, 749, 787, 977, *1036*, *1042*, *1044*, *1056*, *1063*, *1064*
Murphy, R., 980, *1056*
Murray, I., 787, *1052*
Murray, L. M., 527, *1056*
Murray, R. M., 978, 1013, *1044*, *1056*
Murray-Rust, P., 340, *1056*
Murthy, C., 313, *1056*
Musat, C., 787, *1066*
Muscettola, N., 29, 383, 384, *1049*, *1056*
Musk, E., 33
Muslea, I., 854, *1056*
mutación, 21, 141
Muth, J. T., 1013, *1056*
MUZERO (programa de juego), 178
MYCIN (sistema experto), 23, 459
Myers, R. H., 411, *1065*
Myers, S. L., 411, *1065*
Myerson, R., 648, 649, *1056*

N

Naïm, P., 456, *1058*
Nachum, O., 994, *1049*
Naciones Unidas, 31
Naddaf, Y., 822, *1036*
Naderan, M., 143, *1045*
Nagar, S., 996, 1009, *1036*
Nair, A. V., 979, *1034*
Nair, R., 995, 1010, *1047*
Nair, V., 786, *1056*
Nalwa, V. S., 12, *1056*
Nangia, N., 880, *1065*
Narang, S., 879, *1059*
Narayanan, A., 991, 1009, *1056*
Narayanan, V., 822, *1045*
Narayanaswamy, A., 30, *1046*
Nardi, D., 341, *1035*, *1038*
NAS (búsqueda de arquitectura neuronal), **770**, 787
NASA, 29, 342, 355, 384, 455
Nash, J., 608, 647, *1056*
Nash, P., 597, *1056*
NATACHATA (chatbot), 984
Natarajan, S., 561, *1043*
Natarajan, V., 30, *1053*
naturalismo biológico, **985**
naturalismo, 6
Nau, D. S., 176, 178, 381, 382, 384, *1043*, *1045*, *1057*, *1062*, *1065*
Navaja de Occam, 655, 715, 716, 724
navegación costera, **957**
Navruzyan, A., 119, 143, *1055*
Nayak, P., 144, 342, 384, *1051*, *1056*
NBS (algoritmo de búsqueda), 108
Nebel, B., 380, *1048*
Neches, R., 1009, *1056*
negación, **217**
Neil, M., 558, *1043*
Neiswanger, W., 787, *1065*
NELL (Aprendizaje Interminable de Idiomas), 850
Nelson, B., 1010, *1035*
Nelson, G., 29, *1059*

Nelson, P. Q., 30, *1053*
Nemhauser, G. L., 561, *1056*
Nemirovski, A., 143, *1036*, *1056*
Nesterov, Y., 143, *1056*
Neumann, M., 879, *1058*
NeurIPS, 27, 35, 460, 527, 720, 749, 788, 822, 924, 995
Neurobiología, 924
Neurociencia, **11**, 11
NEUROGAMMON (programa de backgammon), 815
neurona, 11, 17, 750
neutralidad del riesgo, **536**
Newcomb, S., 981
Newell, A., 2, 18, 19, 61, 106, 107, 247, 248, 292, 311, 380, 382, *1052*, *1056*, *1062*
Newman, P., 977, *1037*, *1042*
Newton, I., 1, 49, 120, 142, 465, 717, *1056*
NEXTKB (base de conocimientos), 339
Ney, H., 497, 852, 880, *1050*, *1057*
Ng, A. Y., 596–598, 717, 718, 728, 812, 817, 821, 852, 979, 1004, *1034*, *1037*, *1040*, *1050*, *1051*, *1056*
Ng, M., 11, *1067*
Nguyen, P., 849, 854, *1039*, *1048*
Nguyen, T. T., 313, *1046*
Nicholson, A., 460, 498, 597, *1041*, *1051*, *1056*
Nicolelis, M. A., 11, *1052*
niebla de guerra, 168
Nielsen, E., 713, 719, *1038*
Nielsen, M. A., 720, *1056*
Nielsen, P. E., 311, *1049*
Nielsen, T., 560, *1056*
Niemela¨, I., 342, *1056*
Nigam, K., 854, 855, *1040*, *1049*
Nikolaidis, S., 979, *1056*
Nilos, I., 339, *1056*, *1058*
Nilsson, D., 558, 1057
Nilsson, N. J., 32, 35, 61, 106, 107, 144, 278, 279, 303, 312, 380, 382, 383, 524, 525, 785, 976, *1044, 1045, 1047, 1057*
Niño, R., 879, *1059*
Niranjan, M., 498, 820, 1041, 1060
Nisan, N., 648, 649, 1057
Niv, Y., 822, 1041, 1057
nivel de abstracción, **66**
nivel de ejecución, **210**
Nivre, J., 839, 853, 1051, 1057
Nixon, R., 334, 848
Niyogi, S., 278, 1063
no determinista
  angélico, 361
  demoníaco, 361
no cooperativo, 605–625
no lineal, **183**
no monotónico, 225, **333**, 333–335, 341
NOAH (sistema de planificación), 380, 382 Premio Nobel, 10, 11, 22
no-bueno, 196
Nocedal, J., 717, 1037, 1038
Noda, I., 976, 1050
Nodelman, U., 498, 1057
nodo (en árboles de búsqueda), 71, 73
nodo AND, **123**
nodo de decisión, **545**
nodo de fuga, **421**
nodo de probabilidad (Árbol de Juego), **165**
nodo de probabilidad (Red de Decisión), **545**
nodo determinista, **420**
nodo OR, **123**
nodo principal, **72**
nodo secundario, **72**
nodo sucesor, **72**
nodos seguramente expandidos, **90**
Noe, A., 1006, 1057
nomenclatura binomial, 339
NONLIN+ (sistema de planificación), 383 restricciones no lineales, 183 sistema dinámico no lineal, 483 no monotonicidad, 333
Nono, 287
Nordfors, D., 1011, 1057
Nori, A. V., 527, 1040, 1048
norma máxima, **574**
normalización (de rangos de atributos), 688
normalización (de una distribución de probabilidad), 396, 400
normalización por lotes, **768**, 923
Normal–Wishart, 731
Norouzi, M., 29, 30, 783, 850, 865, 920, 1043, 1053, 1066
North, O., 287
North, T., 21, 1044
Norvig, P., 26, 311, 321, 340, 497, 719, 851, 853, 1046, 1057, 1060
notación
  infijo, 268
  lógico, 3
  prefijo, 268
notación de probabilidad, 391
notación $\Omega()$, 1024
notación, 3
Nourbakhsh, I., 144, 1045
Novoa, R. A., 30, 1043
Nowak, R., 455, 1039
Nowatzyk, A., 176, 1048
Nowick, S. M., 250, 1057
Nowlan, S. J., 118, 143, 1047
NP (problemas difíciles), 1024–1025
NP-completo, 9, 106, 223, 248, 341, 434, 716, 1024, 1025
NP-duro, 1025
NQTHM (demostrador de teoremas), enunciado de número 312, 508
núcleo, **627**, 645, 648
número 720
número de conspiración, 175
número de mezcla, 116
número de referencias, **92**
números naturales, **268**
NUPRL (demostrador de teoremas), 313 Nuro, 924
Nvidia, 924
Nyberg, E., 30, 1043
Nyberg, L., 11, 1038

O

Oaksford, M., 560, *1039*
Obermeyer, F., 526, *1036*
Objetivismo, **408**
Objetivo común, **600**
objeto, 254, 259
  compuesto, **318**
  mental, 326-328
observabilidad parcial, 110, 168, 588
Obstáculo del espacio C, **940**
Occam, W., 655, 715
Och, F. J., 497, 852, 880, *1037*, *1057*, *1067*
odometría, **929**
oferta de reserva, 634
oferta racional individual, **643**
oferta, **633**
Ogasawara, G., 498, *1048*
Ogawa, S., 11, *1057*
Oglesby, F., 313, *1046*
Oh, M.-S., 457, *1057*
Oh, S., 526, *1057*
Ohashi, T., 179, *1064*
Oizumi, M., 1007, *1057*
Ojo por ojo, **614**
ojos de mosca, 905, 917
ojos, 881, 884, 885, 920
Olah, C., 1010, *1034*
Olalainty, B., 384, *1044*
Olesen, K. G., 455, 456, *1034*, *1057*
Oliver, N., 498, *1057*
Oliver, R. M., 558, *1057*
Olshen, R. A., 716, *1038*
Olson, N., 30, *1053*
Olteanu, A., 995, 1010, *1047*
Olum, P., 921, *1045*
olvido catastrófico, **805**
O'Malley, K., 649, *1065*
O'Malley, M. K., 967, *1035*
Omnisciencia lógica, **328**
omnisciencia, **40**
Omohundro, S., 33, 1010, *1057*
ondulado, 243
O'Neil, C., 718, 1009, *1057*
Ong, D., 526, *1055*
Ong, J., 29, *1035*

ontología, **272**, 275
  general, 317–328
  superior, 337
ontología superior, 337
OpenAI Gym (entorno simulado), 822
OpenAI, 1008
OPENCYC (base de conocimientos), 339
OPENMIND (base de conocimiento), 316
Operador de botones, 574
operadores modales, **327**
O-PLAN (sistema de planificación), 359, 383, 384
Oppacher, F., 143, *1057*
OPS-5 (sistema de razonamiento lógico), 292, 311
optimalidad (de un algoritmo de búsqueda), **75**
optimalidad acotada (BO), **1020**
Optimalidad de Pareto, **609**
optimismo ante la incertidumbre, **139**
optimista prior, 798
optimización, **666**
Optimización bayesiana, **672**
optimización convexa, **122**, 141
Optimización de costos, 105
optimización de directivas de región de confianza, 821
OPTIMUM-AIV (sistema de planificación y programación), 384
optogenética, **11**
orden conjunto, **290**
orden de preferencia social, 639
orden superior, **255**
ordenador ABC, 14
ordenador HAL 9000, 454, 985, 1007
ordenamiento topológico, **416**
O'Reilly, T., *1015*
O'Reilly, U.-M., 143, *1057*
Organon (Aristóteles), 247, 339
oro, 210
Orseau, L., 822, 1003, *1052*
Ortega, P. A., 822, 1003, *1052*
Osawa, E., 976, *1050*
Osborne, M. A., 999, *1044*
Osborne, M. J., 648, *1057*
Osherson, D. N., 716, *1057*
Osindero, S., 786, 787, *1048*, *1049*
osito de peluche, 982
Osman, I., 108, *1059*
Ostland, M., 525, 526, *1057*
Ostrom, E., 649, 1010, *1047*, *1057*
Ostrovski, G., 820, 822, *1055*
Otelo, 178
Ott, M., 876, 879, *1053*
Otten, L., 456, *1061*
OTTER (demostrador de teoremas), 313
Overbeek, R., 313, *1066*
Overmars, M., 978, *1050*
Owen, M. P., 598, *1049*
Owens, A. J., 143, *1044*
Ozair, S., 787, *1046*

P

P (vector de probabilidad), 391, 392
P(s' | s, a) (modelo de transición), 563, 791
Pablo, R. P., 978, *1057*
Pablos, A., 853, *1057*
PAC (probablemente aproximadamente correcto), **673**, 675, 717
Pachocki, J., 959, 979, 1034
Padgham, L., 61, *1057*
Página, L., 854, *1038*
Paige, B., 498, 527, *1057*
paisaje (en el espacio estatal), **110**
paisaje de fitness, 143
Paja, A. D., 1013, *1044*
palabra, 403, 824
  fuera de vocabulario, 827
Palabra desconocida, 827
Palacios, H., 382, *1057*
Palaniappan, M., 648, *1054*
Paleo, B. W., 313, *1036*
Palmer, J., 253, *1053*
Palmer, S., 924, *1057*
Palmieri, G., 785, *1045*
palomas, 13
Pan, X., 821, *1066*
Pandas (software de análisis de datos), 709
Pandas, 720
Panini, 16, 852
Papadimitriou, C. H., 142, 144, 248, 596, 598, 648, 1029, *1042*, *1051*, *1057*
Papadopoulo, T., 923, *1043*
Papavassiliou, V., 820, *1057*
papelería (para preferencias), **566**
papeles del Pentágono, 560
Papernot, N., 787, *1039*
Papert, S., 22, 785, *1055*
parada temprana, **664**
paradoja, 341, 539
  Allais, 538, 560
  Condorcet, 639
  Ellsberg, 538, 560
  Girard, 313
  San Petersburgo, 557
  Zenón, 65
paralaje de movimiento, 920
parámetro variacional, 458
parámetro, **412**, 725
paramodulación, **307**, 312
paridad demográfica, **993**
Parikh, D., 822, 910, *1046*, *1060*
Parisi, D., 854, *1043*
Parisi, G., 142, 458, *1055*, *1057*
Parisi, M. M. G., 249, *1057*
Park, J. D., 457, *1057*
Parker, A., 176, *1057*
Parker, D. B., 785, *1057*
Parker, L. E., 980, *1057*
Parmar, N., 850, 868, 880, *1039*, *1064*
Parque, F. C., 980, *1054*
Parque, S., 309, *1047*
Parque, T., 879, 915, *1067*
Parr, R., 561, 596, 597, 647, 822, *1034*, *1039*, 1046, 1051, *1057*
Parrod, Y., 384, *1034*
Parsey McParseface, 853
parte de la oración (POS), **829**
Partee, B. H., 853, *1058*
Partición, **318**, 626
partícula MCMC, **499**
Parzen, E., 748, *1057*
Pasca, M., 854, 855, *1043*, *1057*
PASCAL VOC (conjunto de datos de imágenes), 922
Pascal, B., 6, 8, 408
Pascanu, R., 786, *1041*
Paseo al azar, **138**, 480
Pasero, R., 278, 311, *1040*
Paskin, M., 525, *1057*
Pastor, P., 978, 979, *1037*, *1052*
Pasula, H., 499, 512, 525, 526, *1054*, *1057*
Patel, S., 992, *1037*
Patel-Schneider, P., 339, 341, *1035*, *1057*
PATHFINDER (sistema de diagnóstico médico), 455
Patil, N., 1018, *1049*
Patil, R., 853, *1039*
pato, mecánico, 976
Patricio, B. G., 108, *1057*
Patterson, D. A., 652, 1018, *1041*, *1049*
Paulin-Mohring, C., 312, *1036*
Paull, M., 248, *1044*
Pavlovic, V., 456, *1067*
Paz, A., 455, *1064*
Pazzani, M., 410, 747, *1042*
PBT (formación basada en la población), **672**
PCA (análisis de componentes principales), 777
PCFG, **833**, 833–835, 852
  lexicalizado, **841**, 853
PDDL (Lenguaje de Definición de Dominio de Planificación), 344
PDP (procesamiento distribuido paralelo), 785
Peano, G., 278, *1057*
Pearce, J., 207, *1057*
Pearlmutter, B. A., 1016, *1062*
Pearson, K., 747, 787, *1057*
Pease, A., 339, *1056*, *1058*
pecados, siete mortales, 112
Pecheur, C., 309, *1047*
Pednault, E. P. D., 380, 646, *1058*
Pedregosa, F., 720, *1058*

Peek, M. Y., 1013, *1044*
PEGASUS (algoritmo de aprendizaje por refuerzo), 817
Pei, J., 720, *1047*
Peirce, C. S., 204, 278, 329, 341, *1058*
Peleg, B., 648, *1058*
películas
  2001: Una odisea del espacio, 454, 985
  AI, 982, 1000
  El Terminator, 926, 1001
  Ella, 982
  Hombre del Centenario, 1000
  La Matriz, 1001
  Pícaro, 915
  Wall-E, 982
Pelikan, M., 143, *1058*
Pell, B., 384, *1056*
Pemberton, J. C., 145, *1058*
Penalti, **57**
Penberthy, J. S., 380, *1058*
péndulo invertido, **816**
Peng, J., 820, *1058*
Peng, L., 30, *1046*, *1053*, *1062*
Penix, J., 309, *1047*
Penn Treebank, **829**, 839, 877
Pennachin, C., 33, *1045*
Pennington, J., 872, 878, *1058*
Penrose, R., 983, *1058*
Pensador, **1005**
pensamiento, 3, 19, **208**
  leyes de, 3–4
pensamiento racional, 3
pensar humanamente, 2
pensar racionalmente, 3
Pensilvania, Univ. de, 14
Peot, M., 383, 457, *1058*, *1061*
percepción, **36**
percepción adaptativa, 938
percepción posible, 130
percepción, 36, 270, 881–919
perceptor y Memorizador Elemental (EPAM), 715
perceptrón, 21, 785
  poder de representación, 22
  regla de aprendizaje, **683**
  teorema de convergencia, 21
perceptrón votado, 718
perceptrones de Gamba, 785
pérdida de generalización, **670**
pérdida empírica, **670**
Pereira, F., 26, 294, 340, 718, 719, 838, 840, 855, 877, *1034*, *1042*, *1046*, *1052*, *1058*, *1059*
Peres, Y., 497, 499, *1052*, *1054*
pereza, **386**
perfil de la estrategia, **606**
Perkins, T., 316, *1062*
Perl, J., 17, 25, 62, 90, 107, 109, 142, 175, 176, 206, 413, 454–460, 747, 748, *1041*, *1045*, *1046*, *1049*, 1050, *1057*, *1064*
Perov, Y., 527, *1054*
perplejidad, **879**
Perrault, C. R., 646, *1040*
perspectiva, 920
Persson, K. A., 872, *1064*
Pesch, E., 384, *1037*
Peshkin, M., 144, *1065*
peso, **676**
PESPACIO, 384, 1025
Peters, J., 458, 979, *1050*, *1058*
Peters, M. E., 879, *1058*, *1060*
Peters, S., 853, *1042*
Petersen, S., 820, 822, *1036*, *1055*
Peterson, C., 458, *1058*
Peterson, K., 963, 979, *1067*
Petosa, N., 178, *1058*
Petrie, K., 207, *1045*
Petrie, T., 497, 748, *1035*
Petrik, M., 820, *1053*
Petron, A., 959, 979, *1034*
Petrov, S., 839, 853, 989, *1034*, *1057*
Pettersson, L., 822, *1038*
Pezeshki, M., 849, *1067*
Pfeffer, A., 176, 502, 525-527, 647, *1051*, *1058*
Pfeifer, G., 342, *1043*
Pfeifer, R., 1006, *1058*
Pham, H., 787, *1058*
Philips, A. B., 142, 206, *1055*
Phillippy, A. M., 717, *1036*
Phillips, E. H., 30, *1052*
piano lógico, 248
Piantino, S., 1018, *1064*
Pícaro, 915
Piccione, C., 647, *1067*
PICTURE (lenguaje de programación probabilístico), 527
pie de foto, 30
Pieper, G., 313, *1066*
Pierson, E., 1009, *1040*
Pietra, V. J. D., 878, *1038*
pila (estructura de datos), **74**
Pineau, J., 598, 820, 979, *1044*, *1058*, *1061*
Pinedo, M., 384, *1058*
ping-pong, 179
Pinkas, G., 206, *1058*
Pinker, S., 252, 253, 278, 854, *1058*, *1060*
Pinto, D., 855, *1058*
Pinto, L., 979, *1058*
pinza de mordazas paralelas, **929**
Pisa, torre de, 57
Pista, F. J., 455, *1056*
Pitassi, T., 456, 995, 1009, *1035*, *1043*, *1066*
Pitts, W., 16, 17, 20, 249, 750, 785, *1054*
píxel, **882**
Pizarra, D. J., 107, *1062*
plan condicional, 110, **122**, 123, 590
plan dominado (en POMDP), **592**
plan universal, 383
PLAN-ERS1 (sistema de planificación y programación), 384
planes de contingencia, 365, 370–371, 379
PLANEX (agente de planificación), 383
planificación, 174, 344–384
  y actuación, 365–367
  como refinamiento, 352
  como satisfacción, 351
  mundo de bloques, 20
  basado en casos, **382**
  clásica, **344**
  conforme, 365, 367–370, 379, **382**
  contingencia, 365, 370–371, 379
  descentralizados, **599**
  jerárquico, 356–365, 379
  red jerárquica de tareas, 357
  historia de, 380
  lineal, **380**
  montecarlo, **572**
  multicuerpo, **599**, 601–604
  multiefector, **599**
  online, 365
  reactivo, **383**, 968
  regresión, 350–351
  ruta, 19
  espacio de búsqueda, 348–356
  sin sensores, 365, 367–370
planificación clásica, **344**
planificación conforme, 365, 367–370, 379, **382**
Planificación de consultas múltiples, **947**
Planificación de movimientos, 931, 938, **942**, 977
planificación de órdenes parciales, **352**
Planificación de tareas (en robótica), **931**
Planificación en línea, 365
Planificación multicuerpo, **599**, 601–604
Planificación reactiva, **383**, 968
Planificación sin sensores, 365, 367–370
Plankalku¨l, 14
PLANNER (lenguaje de programación lógica), 23, 311, 646
plano focal, **885**
PLAN-RUTA, **242**
PlanSAT, 384
  acotado, 384
Plappert, M., 959, 979, *1034*
Platón, 247, 340
Platt, J., 718, *1058*
playout, **161**
Playter, R., 29, *1059*
PL-FC-IMPLICA?, **231**
Plotkin, G., 312, *1058*
Plummer, M., 747, *1058*
Plunkett, K., 854, *1043*
pluralidad de votos, **640**
Pluribus (programa de póker), 178, 622

PNL (procesamiento del lenguaje natural), 2, 823, 823–880
Pnueli, A., 341, *1058*
Pobreza del estímulo, 854
poda, **90**, **146**, 152, 663, 698
  alfa-beta, **152**hacia adelante, **159**
  en problemas de contingencia, 167
  futilidad, **176**
poda χ^2, **664**
poesía, 1
Poggio, T., 786, *1054*
Pohl, I., 107, 108, *1058*
Póker, 178, 648
Poli, R., 144, 1052, 1058
polisemia, 873
política, 160, **564**, 595
  buscar, **790**, **810**, 810–812
  dominantes, **587**
  estocástico, **811**
  evaluación, **576**, 791
  gradiente, **811**
  mejora, **576**
  iteración, 572, **576**, 576–578, 596
    asíncrona, **578**
    modificada, **578**
  óptimo, **564**
  pérdida, **575**
  propiamente dicho, **566**
  valor, **811**
política adecuada, **566**
Política de miopía, **550**
Política de reproducción, **161**
Política de selección (en la búsqueda del árbol de juego), **161**
política no estacionaria, 565
Pollack, M. E., 646, *1040*
Polonia, 340
Polosukhin, I., 850, 868, 880, *1064*
polytree, **433**, 434, 454, 470
Polyzotis, N., 311, *1051*
POMCP, **594**
POMDP (MDP parcialmente observable), **588**, 588–595
POMDP-VALUE-ITERATION, **593**
Pomerleau, D. A., 923, 977, *1058*
Ponce, J., 924, *1044*
PONDERACIÓN DE PROBABILIDAD, **440**
ponderación de restricciones, **198**
ponderación de verosimilitud, **439**, 454, 491
Ponte, J., 880, *1067*
Poole, B., 787, *1058*
Poole, D., 61, 456, 525, 527, 559, *1037*, *1050*, *1058*, *1067*
Popat, A. C., 852, 880, *1037*
Popescu, A.-M., 855, *1043*
Poppe, R., 1013, *1058*
Popper, K. R., 409, 716, *1058*
porción de tiempo (en DBN), **462**
Pórfido, 341
Porter, B., 343, *1064*
Portner, P., 853, *1058*
POS (parte de la oración), **829**
pose, **905**, 939
Posegga, J., 312, *1036*
posición, **147**
positivismo lógico, **7**
Positivismo, Lógico, **7**
posterior variacional, **778**
potencial (en MDP), 570
Potts, C., 845, 880, *1037*, *1053*
Pouget-Abadie, J., 787, *1046*
Poulton, C., 1013, *1058*
Poundstone, W., 647, *1058*
Pourret, O., 456, *1058*
Powell, G., 959, 979, *1034*
Powell, R., 30, 179, *1064*
Powers, R., 648, *1061*
Powley, E. J., 176, *1038*
pozo, sin fondo, 210
PPCA (análisis probabilístico de componentes principales), **776**
Prabhat., 30, *1052*
Prabhavalkar, R., 849, *1039*
Prade, H., 460, *1042*
Pradhan, M., 421, 455, *1058*
Pradhan, N., 526, *1036*
Praet, J.-C., 559, *1036*
Prager, J., 30, *1043*
pragmática, 846
Pratt, L., 719, *1064*
Prawitz, D., 310, *1058*
Preboste, F., 990, *1043*
precálculo, **102**
Precio Waterhouse, 383
Precio, B., 597, *1037*
Precio, E., 1009, *1047*
predicción de personas, **931**
Predicción, 130, 467–468, 496
Predisposición
preentrenamiento, 857, **871**
preferencias, **387**, 530
  desconocida, 553–556
  monótona, **534**
Preferencia de modelo, **334**
preferencia unitaria (en resolución), **308**
Premio Fredkin, 176
Premio Loebner, 1006
Premio Netflix, **991**
Premio Turing, 341, 652, 788, 1029
premisa, **217**
Prensa, W. H., 142, *1058*
Presidente, 325
Presley, E., 325
Presta, A., 853, *1034*
Preston, J., 1007, *1058*
Prettenhofer, P., 720, *1058*
Prieditis, A. E., 100, 109, *1058*
prima de seguro, **535**
Primer en moverse, 642
Príncipe, A., 851, *1062*
Principia Mathematica, 18
principio de inclusión-exclusión, **393**
Principio de la Revelación, **635**
principio de razón insuficiente, **409**
prioridad, **74**
Pritzel, A., 979, *1053*
Privacidad diferencial, **991**, 1009
Privacidad, 711, 1009
probabilidad, 3, **8**, 25, 385-460, 722, 1027-1028
  axiomas de, 393
  condicional, **389**, 396, 399, 407, 416
  conjuntivo, 416
  distribución, **391**, 427
  función de densidad, **391**, 1027
  historia de, 411
  marginal, **395**
  modelo, **389**, 1027
  poco informativo, **733**
  prior, **389**, 407
  sentencias, 418
  teoría, 256, **386**, 557
  universo abierto (OUPM), **508**
probabilidad causal, **399**
probabilidad condicional, **389**, 396, 399, 407, 416
Probabilidad de ganar, 156
probabilidad incondicional, *véase* probabilidad, a priori
probabilidad logarítmica, **725**
probabilidad posterior, *véase* probabilidad, condicional
probabilidad previa, **389**, 407
probabilística, 461, **465**, **524**
probabilísticamente completo, **947**
probablemente aproximadamente correcto (PAC), **673**, 675, 717
probadamente beneficioso, 623
Probador del teorema de Boyer-Moore, 312, 313
PROBCUT (algoritmo de búsqueda en el árbol del juego), 159
probit, **424**, 455
problema, **65**, 105
  8-puzzle, 97, 100
  bandido, **581**, 597, 798
  conforme, **126**
  de gira, **70**
  diseño vlsi, 70, 115
  estandarizado, **66**
  formulación, **64**, 66
  generador, **57**
  inherentemente duro, 1024-1025
  millones-reinas, 197, 206
  mudanzas de pianos, 977
  mundo real, **66**

n-reinas, 236
optimización restringida, 121, **185**
optimización, **110**
poco constreñido, **236**
relajado, **99**, 100, 353
resolviendo, 22
robot de navegación, 70
secuenciación de ensamblaje, 70, 107
sin sensor, **126**
vacilación, 282
viajante de comercio, 107, 109
Problema de 8 quens, 110, 112, 116, 197
Problema de alineación de valores, **5, 1003**
Problema de apagado, 623
problema de calificación, **241**, 385, 982
problema de coordinación, **600**, 608, 961
problema de correspondencia, **968**
Problema de decisión secuencial, 562–572, 596
problema de detención, 282
problema de encuadre, 339
representativo, **239**
problema de gira, **70**
Problema de la cubierta del conjunto, **353**
problema de mapeo, **135**
problema de marco inferencial, **239**, 250
problema de optimización restringida, 121, **185**
Problema de planificación multiagente, **599**
problema de programación del taller, **375**
problema de satisfacción de restricciones (CSP), 20, **180**, 180–185, 352
problema de selección, **586**
Problema del mapa de cuatro colores, 204, 983
problema del motor de piano, **942**
problema del mundo real, **66**
Problema del Rey Midas, **33**, 1003
problema del vendedor ambulante (TSP), 70, 107, 109
Problema entre el carro y el poste, **816**
problema estandarizado, **66**
problema relajado, **99**, 100, 353
Procaccia, A. D., 649, *1037*
Procedencia de los datos, **706**
PROCEDER, **83**
procedimiento de inferencia, 273
procesamiento del lenguaje natural (NLP), **2**, **823**, 823–880
procesamiento distribuido en paralelo (PDP), 785
proceso de decisión compuesto, 108
Proceso de decisión de Markov (MDP), 10, **563**, 595, 597, 789
estimación estructural de, 821
factorizado, **597**
parcialmente observable (POMDP), **588**, 588–595, 597
relacional, **597**
Proceso de Dirichlet, **748**
Proceso de Markov, **463**
de segundo orden, 463
homogéneo en el tiempo, 463–465, 496
Proceso de recompensa de Markov, **581**
Proceso gaussiano, 672, **748**
PRODIGIO (sistema de planificación), 382
producto puntual, **430**
profundidad de campo, **885**
profundidad efectiva, **98**
programa de juegos, 534
programa parcial, **808**
programación cuadrática, **693**
programación de conjuntos de respuestas, 312
programación de fábricas de cerveza, 383
programación de portaaviones, 383
programación diferenciable **1016**
programación dinámica, 61, 101, 107, **297**, 470, **563**, 596, 835
adaptativa (ADP), **793**, 793
no serial, **456**
programación dinámica adaptativa (ADP), **793**, 818
programación genética, 21, **116**, 143
programación lineal, **121**, 141, 143, 183, 572, 613
programación lógica, 230, 278, 294–298
restricción, 298, 312
probabilística, 524
presentado, **297**
programación lógica de restricción (CLP), **205**, **298**, 312
programación orientada a objetos, 15, 330
programación visual, **968**
programación, **374**, **376**, 374–378
progresión (en planificación), 349, 379
prólogo, 23, **294**, 311, 380
propagación de creencias descabelladas, *véase* propagación de creencias, propagación de creencias,
propagación de la encuesta, **249**
propagación de límites, **189**
propagación, **185**, 185–191, 194–195
propiedad (relación unaria), **254**
Propiedad de Markov, 471, 496, 563
Propiedad de refinamiento descendente, **360**
propiedad extrínseca, **322**
propiedad intrínseca, **322**
Propinas, M. E., 787, *1064*
proposicional, 208, 217–222, 246, 251
inferencia, 220–236
semántica, 218-220
sintaxis, 217-218
cuantificador, 260–264
muestreo, 457
resolución, 225–229
temporal, **255**
términos, 259–260
Proposicionalización, **281**, 309
proposición
probabilística, 388–395
símbolo, **217**
Prospectiva jerárquica, **365**
Prosser, P., 205, *1058*
protocolo de concesión monotónica, **644**
protocolo de red de contrato, **632**
prototipado rápido, 294
Provan, G. M., 421, 455, *1058*
PROVER9, 313
providor de teoremas de tecnología de prólogo (PTTP), 312
proyección en perspectiva, **883**
proyección ortográfica a escala, **885**
proyección temporal, 249
Proyecto de Programación Heurística (HPP), 23
Proyecto Quinta Generación, 23
Prueba de corte, **156**
prueba de gol
temprana, **76**
tarde, **76**
prueba de igualdad, 665
prueba de significación, **663**
prueba terminal, **147**
prueba, **223**
Pruksachatkun, Y., 880, *1065*
pseudocódigo, **1031**
pseudoexperiencia, **796**
pseudoinverso, **680**
pseudorecompensa, **807**
Psicofísica, 924
psicología, 12–14
experimental, 12
psicología cognitiva, **13**, 823
psicología evolutiva, **539**
psicología popular, 343
PTTP (Demostrador de teoremas de tecnología de prólogo), 312
compuerta (lógica), 274
compuerta de entrada (en LSTM), 775
compuerta de olvido (en LSTM), 775
puesta a tierra, **216**, **506**
Puget, J.-F., 207, *1045*
pulcro vs. desaliñado, 24
Pullum, G. K., 278, 853, 854, *1048*, *1058*
Puma (brazo robótico), 976
punto de cruce, **116**
punto de división, **664**
punto de fuga, **884**
punto de referencia, 24, **102,** 879, 922, **1023**
punto focal (en teoría de juegos), **608**
punto fotosensible, 917
Puntuación de riesgo, 993
Purdom, P., 207, 1038
Puterman, M. L., 61, 596, 598, *1058*
Putnam, H., 233, 248, 303, 310, 410, 527, *1041*, *1058*

Pyro (programación probabilística idioma), 526
Pyrros, A., 916, *1042*
PySC2 (software de aprendizaje automático), 822
Python, 259
PyTorch (software de aprendizaje automático), 526, 720, 1021

Q

Q(*s*, *a*) (valor de la acción en estado), 802
QA3 (sistema de razonamiento lógico), 278
Qian, H., 1009, *1036*
Q-learning, **790**, 802, 810, 925
Q-LEARNING-AGENT, **803**
QUACKLE (programa de Scrabble), 179
qualia, **985**
Quevedo, T., 175
quietud, **158**
Quigley, A., 1013, *1066*
Quillen, D., 979, *1052*
Quillian, M. R., 341, *1058*
Química, 22, 338, 983
Quine, W. V., 279, 320, 339, 340, *1058*
Quinlan, J. R., 715, 716, *1058*
Quinlan, S., 978, *1059*
Quirk, R., 853, *1059*
QXTRACT (sistema de extracción de información), 855

R

R (software estadístico), 720
R1 (sistema experto), 23, 292, 311
Rabani, Y., 143, *1059*
Rabe, M. N., 309, *1035*
Rabideau, G., 383, *1044*
Rabiner, L. R., 497, *1059*
Rabinovich, Y., 143, *1059*
Rabinowitz, Carolina del Norte, 30, 179, *1049*
racimo, **319**
racionalidad de Boltzmann, **814**
racionalidad individual, **627**
racionalidad limitada, **4**, 328
racionalidad, **1**, 39-40
racionalismo, 855
radar, 10, **928**
Radford, A., 777, 879, *1059*
Raedt, L. D., 525, *1050*
Rafael, B., 107, 311, *1046*, *1047*, *1059*
Raffel, C., 879, *1059*
Rafferty, A. N., 598, *1059*
Ragan-Kelley, J., 1016, *1053*
Raghavan, M., 994, 1009, *1050*
Raghavan, P., 850, 854, *1054*
Raghu, M., 787, *1058*
Rahwan, T., 648, *1059*
Raibert, M., 29, *1059*
Raiffa, H., 10, 539, 559, 560, 647, *1050*, *1053*
raíz cuadrada, 49
raíz cuadrática media (RMS), 1029
Rajamani, S. K., 527, *1040*, *1048*
Rajan, K., 29, 383, *1049*
Raji, I. D., 995, *1055*
Rajpurkar, P., 880, *1059*
Raju, B., 119, 143, *1055*
Ramage, D., 992, *1037*
Ramamurthy, K. N., 995, 996, 1009, 1010, *1036*, *1047*
Ramsey, F. P., 10, 409, 558, *1059*
Ramsundar, B., 499, 720, 1043, *1059*
RAND Corporation, 560, 596
rango (de una variable aleatoria), **390**
Ranzato, M., 786, *1049*
Rao, A., 62, *1066*
Rao, B., 498, *1048*
Rao, D. A. S., 998, *1059*
Rao, K., 849, *1039*
Rao-Blackwellization, **496**
Raphson, J., 142, 717, *1059*
Raschka, S., 720, *1059*
Raschke, U., 976, *1040*
Rashevsky, N., 17, 785, *1059*
Rasmussen, C. E., 748, *1059*
Rassenti, S., 649, *1059*
Rastreador de hipótesis múltiples (MHT), 526
rastreo común, 852, 871
ratas, 13
Ratinov, L., 878, *1064*
Ratliff, N., 951, 963, 966, 978, 979, *1059*, *1067*
Ratnaparkhi, A., 852, *1059*
Ratner, D., 106, *1059*
Rauber, J., 787, *1039*
Rauch, H. E., 497, *1059*
Rawal, A., 119, 143, *1055*
Ray, A., 959, 979, *1034*
Ray, B., 719, *1046*
Rayson, P., 852, *1052*
Rayward-Smith, V., 108, *1059*
Razavi, A., 787, 1049
razonamiento, 3, 19, **208**
  lógico, 222-237
  predeterminado, 333–335, 459
  probabilística, 395–397, 427–435
    aproximado, 435–449
razonamiento dirigido a objetivos, **232**
razonamiento espacial, **342**
razonamiento incierto, 386
razonamiento lógico, 222-237
razonamiento por defecto, 333–335, 459
razonamiento psicológico, **343**
razonamiento temporal, 126-134, 237-246, 322–325, 461–499
RBFS (búsqueda recursiva en el mejor de los casos), 93–95, 105
RCNN (sistema de visión artificial) más rápido, 900
RDF (Marco de Descripción de Recursos), 339
Re', C., 524, *1041*
Real, E., 787, *1059*
realizabilidad, **670**
RECHAZO-MUESTREO, **438**
Rechenberg, I., 143, *1059*
Recht, B., 716, *1067*
Recién nacido, M., 108, *1057*
recocido simulado, 114–115, 141, 142, 377, 442
RECOCIDO SIMULADO, **115**
recombinación, **115**
Recomendación, 502
recompensa, **57**, **563**, 595, 789
  aditivo, **566**
  con descuento, **565**
  dar forma, **807**
  escaso, 790
  función, 1014
  hiperbólico, **598**
recompensa dispersa (en el aprendizaje por refuerzo), **790**
recompensa hiperbólica, **598**
recompensa para llevar, **792**
reconocimiento de planos, **605**
reconocimiento de voz, 2, 25, **849**, 854
Recopilación de información, **41**, 956
Recuerdo perfecto, 619
recuperación de información (IR), **850**, 854
RECURSIVA-MEJOR-PRIMERA-BÚSQUEDA, **93**
Recurso consumible, **375**
recurso reutilizable, **375**
recursos, 374–379
red bayesiana, 25, **412**, 412–460, 748
  tiempo continuo, 498
  dinámico, **485**
  híbridos, **422**, 454
  inferencia en, 427-435
  aprendizaje de variables ocultas en, 745-746
  aprendizaje en, 734-735
  multientidad, 526
  semántica, 414
red bayesiana dinámica (DBN), 461, **485**, 485–498
  inferencia aproximada en, 491
  inferencia exacta en, 489
red causal, 412, 449–453, 748
red conectada individualmente, **433**
red conectada múltiple, **433**

red de creencias, *véase* estado de creencias de la red bayesiana, **122**, 241, 365, 385, 388
en teoría de juegos, 620
probabilística, 461, **465**
wiggly, 243
red de decisión, 454, 528, **544**, 544–547, 557
dinámica, **570**, 595
evaluación de, 546
red de generadores (en GAN), **780**
red de Hopfield, **788**
red dinámica de decisión (DDN), **570**, 595
Red Feedforward, **751**
red generativa adversaria (GAN), **780**, 787, 915
Red Markov, 456
red neuronal, 17, 20, 24, **750**, 750–788
aprendizaje, 17
convolucional, 26
expresividad, 17
feedforward, **751**
ferretería, 17
multicapa, 22
recurrente, **751**, 772–775, 860–864
red neuronal convolucional (CNN), **760**, 896
Red Neuronal Recurrente
red neuronal recurrente (RNN), *véase*
red neuronal recurrente Robbins, H., 585, 597, 717, *1052*, *1059*
red probabilística cualitativa, 460, **542**
red Q profunda (DQN), 784, **816**, 822
Red Regional de Propuestas (RPN), **900**
red residual, **764**
red semántica, **329**, 329–331, 337, 341
Reddy, R., 17
reducción de movimiento tardío, **160**
reducción de muestreo, **762**
reducción de simetría, **354**
reducción, **433**, 1029
reenviar mensaje, 469
Rees, M., 33
Reescritura de términos, 312
Reeves, C., 108, *1059*
Reeves, D., 649, *1065*
refinamiento (en la planificación jerárquica), **357**
reflectancia, 905
reflexión, **886**
Reformer (software de lenguaje natural), 877, 879
REFORZAR (algoritmo de aprendizaje por refuerzo), 812
Refuerzo, **699**
refuerzo, 789
refutación, **223**
Regan, N., 177, *1060*
Regin, J., 205, *1059*
región de interés, 900
región, **944**
registro de datos, **287**, 310, 311
regla, **217**
causal, 418
condición-acción, **50**
diagnóstico, 418
implicación, 217
por defecto, **334**
si–entonces, 50, 217
situación-acción, 50
regla causal, 418
regla de bayes, 8, **399**, 399-400, 408
regla de condición-acción, 559
regla de la cadena
para la diferenciación, **678**, 685, 754
para probabilidades, **416**
regla de inferencia, **223**, 247
regla de reescritura, 1030
regla del producto, **390**, 399
regla delta, **804**
regla diagnóstica, 418
regla Widrow-Hoff, **804**
reglas de De Morgan, 263
regresión (en aprendizaje automático), **652**
árbol, **665**
lineal, **677**, 717, 729
bayesiano, 732–734
regresión (en planificación), **350**, 349–351, 379
regresión de los vecinos más cercanos, **691**
regresión lineal multivariable, **679**
Regresión lineal, *consulte* Regresión (en Machine Learning)
Regresión logística, **685**, 717, 830
regresión ponderada localmente, **691**
regulador cuadrático lineal (LQR), **955**, 975
regularización, **671**, 680, 702, 771, 1002
Reid, D. B., 526, *1059*
Reid, M., 98, 107, *1051*
Reif, J., 977, 979, *1038*, *1059*
reificar, 317
Reingold, E. M., 205, *1037*
Reingold, O., 1009, *1043*
reinicio aleatorio, 234
Reinsel, G., 497, 787, *1037*
Reiter, M. K., 1009, *1064*
Reiter, R., 250, 341, 381, 597, *1037*, *1059*
relación (en CSP), **180**
relación (en lógica), **254**
relación competitiva, **135**
Relación de accesibilidad, **327**
Relación de ganancia de información, 665
relevancia, 219
Relojes, W. F., 312, *1040*
ReLU, **752**
Remolina, E., 29, *1035*
remuestreo, 492
Ren, S., 786, *1047*
RENDERIZAR-IMAGEN-RUIDOSA, **519**
rendición de cuentas, **711**
rendimiento humano, 1
Renner, G., 143, *1059*
Re'nyi, A., 410, *1059*
repetición de experiencias, **806**
replanificación en línea, **956**
replanificación, 365, 371–374, 383
representación, *véase* Representación de conocimiento
atómica, **59**
contextual, **873**
estructurado, **59**
factorizado, **59**
representación del conocimiento, **2**, 17, 19, 23, **208**, 251–256, 314–343
incierto, 412–414
lenguaje, **209**, 246, 251
para todo, 314
Representación distribuida, **60**
representación estructurada, **59**, 63, 390
representación factorizada, **59**, 63, 180, 344, 390, 570, 652
representación localista, **59**
REPRODUCIR, **119**
Reproductor de columnas, **606**
respuesta a preguntas, **850**, 880
visual (VQA), 28, **910**
requisitos de memoria, 77, 80
residual (en redes neuronales), **764**
Resnick, C., 920, *1043*
Resnick, P., 29, *1059*
resolución, 19, 21, **226**, 225–229, 247, 298–310
cierre, **228**, 304
entrada, **308**
estrategias, 308–309
lineal, **308**
prueba de completitud para, 303
resolución binaria, **300**
Resolución de entrada, **308**
Resolución de la unidad, **225**, 308
resolución de problemas distribuida cooperativa, **646**
resolución lineal, **308**
RESOLUCIÓN PL, **228**
resolutorio, **225**, 300
respuesta, 13
restricción de acción concurrente, **604**
restricción de *Alldiff*, 184
restricción de ciclo, 914
restricción de precedencia, **182**
restricción de preferencias, **185**
restricción disyuntiva, **183**
restricción global, **184**, 188
restricción lineal, **183**
resultado social, **639**
resultado, 345

Resultado, **387**, 606, 626
resumiendo, 396, 431
retiming, **951**
retinopatía diabética, 698
retroceder (en un árbol de búsqueda), **93**, 149
retroceder (en un árbol de búsqueda), **93**, 149
RETROCESO, **192**
retroceso cronológico, 195
  dirigido a la dependencia, **205**
  dinámico, 206
  inteligente, 195-197, 234
retroceso dinámico, 206
retroceso ingenuo, 828
retroceso inteligente, 195–197, 234
retrógrado, **161**
retropropagación, 22, 24, 162, 163, **755**, 766, 785
  a través del tiempo, **774**
revelación de la verdad, **635**
Reversi, 178
reversión, **498**
Rey, H., 820, 822, *1036*, *1055*
Rezende, D. J., 787, *1059*
RGB (rojo, verde, azul), *888*
RGPD (Reglamento General de Protección de Datos), 712
RGPD, 990, 997
Riazanov, A., 312, 313, *1059*
Ribeiro, F., 179, *1064*
Ribeiro, M. T., 719, *1059*
Ribeiro-Neto, B., 850, *1035*
Richards, J. T., 996, 1009, *1036*
Richardson, K., 876, 880, *1040*
Richardson, M., 525, *1059*
Richardson, S., 457, *1045*
Richtárik, P., 992, *1051*
Richter, S., 381, *1059*
Rice, T. R., 558, *1055*
Riddell, A., 458, 527, 747, *1039*
Ridley, M., 143, *1059*
Riedmiller, M. A., 784, 790, 820, 822, *1055*
Riesbeck, C., 23, 311, *1039*, *1060*
Riley, J., 649, *1059*
Riley, P., 719, *1059*
Riloff, E., 855, *1049*, *1059*
Ringgaard, M., 877, *1059*
Rintanen, J., 381-384, *1059*
Ripley, B. D., 720, *1059*
Rish, I., 456, *1041*
Rissanen, J., 716, *1059*
Ristenpart, T., 1009, *1064*
ritmo de cambio, **999**
Ritov, Y., 525, 526, *1057*
Rivest, R., 107, 716, 717, 1029, *1040*, *1049*, *1059*
Rivest, Shamir y Adelman (RSA), 309
RMS (raíz cuadrada media), 1029
RNN (red neuronal recurrente), *véase*
RNN bidireccional, **863**
Roöger, G., 107, 381, *1047*, *1061*
ROBERTA (sistema de lenguaje natural), 781, 875–877, 879
Roberts, A., 879, 920, *1043*, *1059*
Roberts, G., 1006, *1043*
Roberts, L. G., 921, *1059*
Robertson, N., 206, *1059*
Robertson, S. E., 410, *1059*
Robins, J., 458, *1059*
Robinson, A., 279, 310, 313, *1059*
Robinson, G., 312, *1066*
Robinson, J. A., 19, 248, 278, 303, 311, *1059*
Robinson, S., 107, *1059*
RoboCup, 976
robopocalypse, **1001**
robot, **925**, 925–927, 975
  derechos, 1010
  fútbol, 146, 650
  hexápodo, 968
  móvil, 927
  navegación, 70
robot antropomórfico, **926**, 930
robot hexápodo, 968
robot móvil, **927**
robótica, **2**, 487, 925–980
robots de telepresencia, **971**
robusto, **801**
Roche, E., 854, *1059*
Rochester, N., 18, 19
Rock, D., 998, 1038
Rock, I., 924, *1059*
Rodríguez, H., 852, *1054*
Rodríguez, K., 1013, *1066*
Rohlfshagen, P., 176, *1038*
Rokicki, T., 106, *1059*
Rolf, D., 248, *1059*
Rolf, E., 1009, *1053*
Rolland, N., 526, *1046*
Rolnick, D., 30, 787, *1059*
Romanovskii, I., 647, *1059*
Romero, J., 30, *1052*
rompecabezas de baldosas deslizantes, **68**, 102, 353
Rompecabezas de Sokoban, **67**
Rong, Z., 872, *1064*
Roossin, P., 880, *1038*
Ros, G., 822, *1059*
Rosen, C., 976
Rosenblat, A., 1009, *1039*
Rosenblatt, F., 21, 717, 785, 921, *1037*, *1045*, *1059*
Rosenblatt, M., 748, *1059*
Rosenblitt, D., 380, *1054*
Rosenbloom, P. S., 292, 311, 382, *1052*
Rosenblueth, A., 16, *1059*
Rosenbluth, A., 142, 457, *1055*
Rosenbluth, M., 142, 457, *1055*
Rosenschein, J. S., 649, 650, *1060*, *1063*
Rosenschein, S. J., 61, 249, 250, *1049*, *1060*
Rosenthal, G. D., 559, *1050*
Ross, G., 9, *1060*
Ross, S., 410, 979, 1029, *1060*
Rosse, C., 316, *1062*
Rossi, F., 204, 205, 207, *1035*, *1037*, *1060*
Roth, A., 1009, *1036*, *1039*, *1043*
Roth, D., 457, 527, *1041*, *1060*
Roughgarden, T., 648, *1057*
Roussel, P., 278, 311, *1040*, *1060*
Rover, **927**
Roveri, M., 381-383, *1036*, *1040*
Rowat, P. F., 977, *1060*
Roweis, S. T., 455, 498, *1060*
Rowley, H., 922, *1060*
Roy, B. V., 596, *1041*
Roy, D., 526, *1034*, *1046*
Roy, N., 598, 979, *1049*, *1060*, *1063*
Rozonoer, L., 717, *1034*
RPM (modelo de probabilidad relacional), **502**
RRT (exploración rápida de árboles aleatorios), 947, 978
RRT*, **948**
RSA (Rivest, Shamir y Adelman), 309
Ruan, P., 597, *1062*
Rubin, D., 458, 497, 498, 748, *1042*, *1045*, *1060*
Rubin, J., 1009, *1064*
Rubinstein, A., 598, 647, 648, 650, *1034*, *1057*, *1060*
Rubinstein, B. I., 1009, *1056*
Ruder, S., 875, 879, *1048*, *1060*
Ruderman, A., 30, 179, *1049*
ruido (en datos de entrenamiento), 659, 663–664,
ruido (en imágenes), 889
Ruido gaussiano, 479
ruido, 670
ruidoso-OR, 420
Rumanía, 63, 181
Rumelhart, D. E., 24, 785, *1060*
Ruml, W., 108, 109, *1038*, *1047*, *1063*, *1066*
Rummery, G. A., 820, *1060*
ruptura de simetría (en CSP), **203**
ruptura de simetría, **203**
Ruspini, E. H., 459, *1060*
Russakovsky, O., 786, *1060*
Russell, A., 107, *1042*
Russell, B., 7, 17, 18, 310, *1065*
Russell, J. G. B., 559, *1060*
Russell, J. R., 313, *1056*
Russell, S. J., 62, 108, 109, 144, 175, 176, 249, 298, 321, 382, 497–499, 512, 525–527, 561, 596–598, 647, 648, 748, 820–822, 977, 1019, 1020, *1034-1036*, *1041-1044*, *1046-1048*, *1050*, *1054-1057*, *1060*, *1062*, *1063*, *1066*, *1067*

Russo, C., 526, *1046*
Rustagi, J. S., 458, *1060*
Rusu, A. A., 820, 822, *1055*
ruta, **65**, 105, 376, **938**, 942
  en bucle, **74**
  redundante, **74**, 74–75
Ruzzo, W. L., 853, *1046*
Ryan, M., 279, *1048*
Rzepa, H. S., 340, *1056*

S

Saad, F., 526, 527, *1040*, *1060*
SÁBADO, **223**
Sabharwal, A., 248, 381, 850, *1040*, *1046*, *1048*
Sabin, D., 205, *1060*
Sabri, K. E., 313, *1060*
Sacerdoti, E. D., 380, 382, *1060*
Sackinger, E., 26, 786, 922, *1052*
sacrificio, 116
Sadeghi, F., 979, *1060*
Sadeh, N. M., 649, *1035*
Sadigh, D., 964, 979, *1060*
Sadik, A., 820, 822, *1036*, *1055*
Sadler, M., 177, *1060*
Sadri, F., 340, *1060*
Saeed, M., 719, *1046*
Sagae, K., 853, *1060*
Sagiv, Y., 311, *1035*
Saha, D., 996, 1009, *1036*
Sahami, M., 852, *1051*, *1060*
Sahin, N. T., 253, *1060*
Sainath, T., 849, 854, *1039*, *1048*
SAINT (solucionador de problemas matemáticos), 20, 144
Sajonia, A. M., 786, *1046*
Sajonia, R., 821, *1035*
Sakuta, M., 176, *1060*
Salakhutdinov, R., 526, 787, 879, *1052*, *1062*, *1066*
Salesforce, 1017
Salib, M., 713, 719, *1038*
Salido, M. A., 205, 207, *1035*
Salisbury, J., 978, *1054*
Salmond, D. J., 498, *1046*
Salomaa, A., 852, *1060*
saltar hacia atrás, **195**, 205
salto hacia atrás dirigido al conflicto, **196**, 203
Salvaje, P. P., 457, *1045*
Salvatier, J., 28, *1046*
Salzmann, M., 787, *1066*
SAM (demostrador de teoremas), 313
Samadi, M., 109, *1060*
Samet, H., 717, *1060*
Sammut, C., 820, *1060*
Samothrakis, S., 176, *1038*
Samsung, 924
Samuel, A., 17-19, 33, 62, 177, 715, 819, 820, *1060*
Samuel, S., 527, *1048*
Samuelson, L., 647, *1054*
Samuelson, W., págs. 649, *1059*
Sánchez-Lengeling, B., 920, *1060*
Sanders, P., 108, *1042*
Sandholm, T., 30, 178, 647-649, 790, *1038*, *1045*, *1060*
sándwich de jamón, 848
Sang, T., 456, *1060*
Sanguijuela, G., 852, 853, *1052*, *1059*
Sanna, R., 785, *1045*
Sánscrito, 338, 852
Santorini, B., 829, 852, *1054*
SAPA (sistema de planificación), 383
Sapir, E., 824, *1060*
Saraswat, V., 205, *1064*
Sarawagi, S., 855, *1060*
Sargent, T. J., 821, *1060*
SARSA (estado-acción-recompensa-estado-acción), **802**
Sartre, J.-P., 646, *1060*
Sastry, G., 821, *1060*
Sastry, S., 61, 526, 817, 821, 964, 979, *1047*, *1056*, *1057*, *1060*
Satheesh, S., 786, *1060*
Satia, J. K., 598, *1060*
satisfacción (en lógica), **214**
Satisfacción de restricciones distribuidas, 207
Satisfacción, **222**, 248
satisfactorio, **10**, 90
SATMC (sistema de razonamiento lógico), 250
Sato, T., 311, 525, *1060*, *1063*
SATPLAN, **244**
Sattigeri, P., 996, 1009, *1036*
saturación, **304**
SATZ (sistema de razonamiento lógico), 248
Saul, L. K., 458, 787, *1049*, *1060*
Saund, E., 852, *1060*
Saunders, W., 821, *1060*
Saurous, R. A., 526, *1064*
Savage, L. J., 394, 409, 558, *1060*
Savani, Y., 787, *1065*
Savva, M., 822, *1060*
Saxena, N., 1009, *1055*
Sayre, K., 982, *1060*
Scarcello, F., 207, 342, *1043*, *1046*
Schäfer, A. M., 821, *1047*
Scha, R., 840, *1037*
Schaal, S., 979, *1035*, *1058*
Schabes, Y., 840, 854, *1058*, *1059*
Schaeffer, J., 88, 108, 109, 177, 647, *1034*, *1036*, *1040*, *1043*, *1060*
Schank, R. C., 23, *1060*
Schapire, R. E., 700, 718, 852, *1044*, *1060*, *1061*
Scharir, M., 977, *1061*
Scharre, P., 1008, *1061*
Schaub, T., 341, *1042*
Schauenberg, T., 647, *1036*
Scheines, R., 747, *1062*
Schervish, M. J., 410, *1042*
Schickard, W., 6
Schlaefer, N., 30, *1043*
Schlenoff, C. I., 979, *1054*
Schmid, C., 922, *1061*
Schmid, M., 30, 178, *1053*, *1056*
Schmidhuber, J., 787, 788, *1045*, *1048*, *1061*
Schmidt, G., 384, *1037*
Schmitt, S., 178, *1061*
Schneegaß, D., 821, *1047*
Schneider, J., 817, 822, 979, *1034*, *1035*, *1038*
Schnitzius, D., 384, *1041*
Schnizlein, D., 648, *1065*
Schölkopf, B., 458, 718, *1040*, *1041*, *1058*, *1061*
Schön, T. B., 499, *1053*
Schöning, T., 248, *1061*
Schoenberg, I. J., 785, *1056*
Schoenick, C., 850, *1040*
Schofield, M., 176, *1061*
Schomer, D., 253, *1060*
Schoppers, M. J., 383, *1061*
Schrag, R. C., 207, 248, *1035*
Schraudolph, N. N., 177, *1061*
Schrauwen, B., 29, *1064*
Schrittwieser, J., 27, 30, 155, 174, 177, 178, 820, 822, *1036*, *1061*
Schröder, E., 248, *1061*
Schrödl, S., 109, *1043*
Schütze, H., 850, 852, 854, *1054*, *1061*
Schubert, L. K., 339, *1049*
Schulman, J., 821, 822, 978, 979, 1010, *1034*, *1038*, *1044*, *1061*
Schultes, D., 108, *1042*
Schultz, K., 527, *1054*
Schultz, W., 822, *1061*
Schulz, D., 526, 977, *1038*, *1061*
Schulz, S., 312, 313, *1061*, *1063*
Schumann, J., 312, *1042*, *1052*
Schuster, M., 29, 783, 850, 865, 878, *1039*, *1049*, *1066*
Schutt, R., 718, *1057*
Schwartz, J. T., 977, *1061*
Schwartz, R., 877, *1046*
Schwartz, S. P., 339, *1061*
Schwartz, W. B., 410, *1046*
Schwing, A., 909, 916, *1034*, *1042*
Scikit-Learn (software de aprendizaje automático), 720, 1021
SCIPY (software científico), 1021
Sciuto, C., 787, *1066*

Scott, D., 524, *1061*
Scrabble, 179
Sculley, D., 713, 719, *1038*
Searle, J. R., 11, 985, 1007, *1061*
secuencia de acción, 53, 66
secuencia de percepción, **36**, 40, 41
Sedgewick, R., 1029, *1061*
Sedol, L., ix, 813
Sefidgar, Y. S., 967, 979, *1061*
Segal, A., 992, *1037*
Segaran, T., 649, 720, *1061*
segmentación (de una imagen), **894**
seguimiento del plan, 372
Segunda Guerra Mundial, 10, 454, 497
Seguridad de la IA, 1010
seguros de vida, 539
Seipp, J., 381, *1061*
Sejnowski, T., 177, 788, *1047*, *1061*
selección (en algoritmos evolutivos), **116**
Selección de acciones basadas en objetivos, 53
Selección de características, **671**, 825
SELECCIÓN DE MODELOS, **667**
Selfridge, O. G., 18
Sellart, L., 822, *1059*
Selman, B., 142, 206, 248, 249, 341, 381, *1046*, *1050*, *1061*
Selsam, D., 527, *1054*
semántica, **214**, 833, 841
  base de datos, **265**, 297, 345, 502
  lógico, 246
semántica angélica, 379
semántica compositiva, **843**
semántica de bases de datos, **265**, 297, 345, 502
semidecidible, 282, 310
semi-proceso de decisión de Markov, 809
Senges, M., 1011, *1057*
Seni, G., 718, *1061*
Senior, A., 779, 787, 849, 854, *1048*, *1064*
Señoras, T., 30, *1046*
Sensor, 36, 43, **882**
  activo, **927**
  ecografía, 43
  LIDAR, 43, 917, 928
  pasivo, **927**
  sonar, 927
  táctil, **928**
sensor de fuerza, **929**
sensor de localización, **928**
sensor de par, **929**
sensor propioceptivo, **929**
sensor táctil, **928**
Sensory Graphplan (SGP), 383
  atómico, **217**, **260**, 260, 264
  como configuración física, 216
  complejo, **217**, 260
  en una KB, **209**, 246
  lenguaje natural, 823, 834
  sentencia
Sentencia compleja, **217**, 260
Sentencia de observación, 7
sentido común, 408
Seo, H., 822, *1052*
Seo, M., 880, *1061*
separabilidad lineal, **682**
separador (en red de Bayes), 402
separador de márgenes máximos, 692, **693**
separador lineal, 694
Seppi, K. D., 596, *1066*
Serendipia, 374
SERES (sistema multiagente), 646
Sergot, M., 340, *1051*
sesgo (estadístico), **654**
Sesgo (injusticia en los resultados), 706, 992–996
sesgo social, **992**
Seshia, S. A., 964, 979, *1060*
Set de desarrollo, 666
set de entrenamiento, **653**, 666
  ponderado, **699**
Set de magia, **292**, 311
Seth, K., 992, *1037*
SETHEO (demostrador de teoremas), 312
Settle, L., 313, *1046*
Seung, H. S., 979, *1063*
Severini, S., 1018, *1042*
Severyn, A., 853, *1034*
SEXTANT (sistema de navegación del espacio lejano), 29
Seymour, P. D., 206, 1059
SGD (descenso de gradiente estocástico), **679**, 765
SGP (Sensory Graphplan), 383
Sha, F., 877, *1039*
Shachter, R. D., 455-457, 533, 558, 596, *1061*, *1063*
Shafer, G., 459, *1061*
Shafto, P., 598, *1059*
Shah, J., 921, 979, *1056*
Shahrzad, H., 119, 143, *1055*
Shaked, T., 855, *1043*
Shaker, G., 1013, *1066*
Shakey, 20, 61, 144, 380, 383, 976, 979
Shalata, A., 456, *1061*
Shalla, L., 312, *1066*
Shamdas, M., 30, *1053*
Shanahan, M., 323, 340, 1010, *1061*
Shani, G., 598, *1061*
Shankar, N., 313, *1061*
Shannon, C. E., 18, 155, 175, 661, 715, 720, 851, *1061*
SHAP (sistema de aprendizaje automático explicable), 719
Shapley, L. S., 648, *1061*
Shapley, S., 596, 648, *1061*
Sharan, R. V., 920, *1061*
Sharir, M., 978, *1046*
Sharon, G., 108, *1048*
Sharp, D. H., 785, *1040*
Shatkay, H., 977, *1061*
Shaw, J. C., 106, 248, *1056*
Shawe-Taylor, J., 718, *1040*
Shazeer, N., 718, 850, 868, 878-880, 1021, *1049*, *1059*, *1061*, *1064*
Shehory, O., 648, *1060*
Shelley, M., 1001, *1061*
Shelton, C., 498, *1057*
Shen, Q., 748, *1041*
Shepard, D., 559, *1066*
Sheppard, B., 179, *1061*
Shet, V., 769, *1046*
Shi, E., 1009, *1056*
Shi, J., 894, 921, *1061*
Shieber, S., 1006, *1061*
Shimelevich, L. I., 498, *1066*
Shimony, S. E., 457, 597, 1019, *1047*, *1061*
Shin, M. C., 596, *1058*
Shinkareva, S. V., 253, *1055*
Shmakov, A., 106, *1034*
Shmatikov, V., 991, 1009, *1035*, *1056*
Shmoys, D. B., 107, 377, 384, *1052*
Shoar, S., 143, *1045*
Shoham, Y., 61, 179, 312, 559, 648–650, *1034*, *1040*, *1049*, *1052*, *1061*
Shortliffe, E. H., 459, *1038*, *1061*
Shpitser, I., 512, 525, *1057*
SHRDLU (sistema de lenguaje natural), 20, 23, 346, 854
Shreve, S. E., 61, *1036*
Shroff, G., 990, *1054*
Shtark, O., 456, *1061*
Sicción, **952**
Siciliano, B., 980, *1061*
Siddiqui, N., 916, *1042*
Siebel, W. A., 920, *1051*
Sievers, S., 381, *1061*
Sifre, L., 27, 30, 155, 174, 177, 178, 820, *1061*
SIGAI, 35
SIGART, 35
Sigaud, O., 598, *1061*
Sigmoides, 685, **752**
Sigmund, K., 7, *1061*
Silberstein, M., 456, *1061*
Silogismo, 3, 6, **247**
Silva, R., 179, *1061*
Silver, D., 19, 27, 30, 155, 174, 176-179, 598, 784, 790, 820, 822, 979, *1047*, *1053*, *1055*, *1061*, *1064*
Silverman, B. W., 787, *1061*
Silverstein, C., 854, *1062*
Sima'an, K., 840, *1037*
Simard, P., 26, 786, 922, *1036*, *1052*
símbolo constante, **257**, 259
símbolo de función, **257**, 259
símbolo de igualdad, **264**
símbolo de inicio, **1030**

símbolo de predicado, **257**, 843
símbolo de terminal, 1030
símbolo no terminal, 1030
símbolo puro, **233**
Simchowitz, M., 1009, *1053*
Simeón, T., 979, *1062*
simetría de valores, **203**
Simmons, R., 498, 977, *1062*, *1064*
Simón, D., 61, *1062*
Simón, H. A., 2, 10, 18, 19, 22, 61, 106, 107, 248, 309, 380, 561, *1049*, *1056*, *1062*
Simón, J. C., 249, *1062*
Simonis, H., 205, *1062*
Simons, P., 342, *1056*
Simonyan, K., 27, 30, 178, 779, 787, 849, 920, *1043*, *1049*, *1053*, *1061*, *1064*
Sim-to-Real, 926, **959**, 978, 979
simulación, **161**
Simultaneidad verdadera, **602**
simultaneidad, **601**
Sin, B.-K., 498, *1063*
Sinapsis, **12**
Sinclair, A., 114, 143, *1053*, *1059*
sincronización (en sistemas multiagente), **602**
Singer, P.W., 1008, *1062*
Singer, Y., 498, 852, *1044*, *1060*
Singh, A., 879, 880, *1065*
Singh, M., 62, 996, 1009, *1036*, *1048*
Singh, N., 30, *1053*
Singh, P., 32, 316, *1055*, *1062*
Singh, R., 526, *1036*
Singh, S., 145, 527, 597, 719, 820–822, 977, *1035*, *1038*, *1050*, *1054*, *1059*, *1063*
Singularidad, **12**, 1004
Singularitarismo, 1010
sintaxis, 23, **214**, 217
  de la lógica, 246
  de la probabilidad, 392
  del lenguaje natural, 833
síntesis (por demostradores de teoremas), **309**
síntesis de voz, 29
SIPE (sistema de planificación), 383
SIR (muestreo secuencial de importancia con remuestreo), 498
Sirviente de Filón, 976
SIS, *véase* muestreo de importancia, secuencial
Sisbot, E. A., 979, *1062*
Siskind, J. M., 1016, *1062*
Sistema Aerotransportado para Evitar Colisiones X (ACAS X), 598
sistema basado en el conocimiento, 22–24, 819
sistema basado en reglas, 459
sistema de acoplamiento suelto, **603**
Sistema de etiquetado (para imágenes), **909**
sistema de mantenimiento de la verdad (TMS), 206, **335**, 335–337, 342
  basado en la justificación, **336**
  basado en supuestos, **336**
sistema de posicionamiento global, **928**
sistema de producción, 50, **292**, 310, 311
sistema de razonamiento metanivel (MRS), 298
sistema de símbolos físicos, **19**
sistema dinámico, 497
  cuadrático, 143
sistema disperso, **417**
sistema estructurado localmente, **417**
sistema experto, **23**, 338, 558
  comercial, 23, 292
  médico, 459
sistema microelectromecánico (MEMS), 1013
sistema multiagente, 61, **599**
Sistla, A. P., 381, *1062*
Sittler, R. W., 525, 526, *1062*
Sjolander, K., 498, *1051*
Skinner, B. F., 16
skip-gram, **826**, 872
Skolem, T., 278, 310, *1062*
Skolemización, **299**
Skolnick, M. H., 456, *1038*
Skype, 29
Slagle, J. R., 20, *1062*
SLAM (localización y mapeo simultáneos), 494, **935**
Slater, E., 175, *1062*
Slattery, S., 854, *1040*
SLING (sistema de lenguaje natural), 877
Slocum, J., 106, *1062*
Sloman, A., 32, *1055*
Slovic, P., 560, *1049*
SMA*, 105
Smallwood, R. D., 597, *1062*
Smith, A. F. M., 498, 748, *1046*, *1064*
Smith, A., 10, 731, *1036*
Smith, B., 29, 316, 340, 383, *1049*, *1062*
Smith, D. A., 853, *1062*
Smith, D. E., 144, 298, 311, 382, 383, *1038*, *1045*, *1051*, *1058*, *1062*, *1065*
Smith, E., 719, *1062*
Smith, G., 108, *1059*
Smith, J. E., 537, 559, *1062*
Smith, J. L., 30, *1053*
Smith, J. M., 143, 648, *1062*
Smith, J. Q., 558, *1057*, *1062*
Smith, K., 719, *1036*
Smith, M. K., págs. 339, *1062*
Smith, N. A., 877, *1046*
Smith, R. C., 977, *1062*
Smith, R. G., 62, 649, *1038*, *1062*
Smith, S. J. J., 178, *1062*
Smith, T., 29, *1035*
Smith, V., 649, *1059*
Smith, W. D., 175, 455, *1035*, *1056*
Smith, W. E., págs. 561, *1062*
SMODELS (sistema de programación lógica), 342
Smola, A. J., 718, *1061*
Smolensky, P., 851, 1017, *1062*
SMOTE (sistema de generación de datos), 707, 995
Smullyan, R. M., 279, *1062*
Smyth, P., 498, *1062*
SNARC (hardware de red neuronal), 17
Snell, J., 410, *1046*
Snoek, J., 672, *1062*
Snyder, W., 312, *1035*
SOAR (arquitectura cognitiva), 292, 311, 382
sobreajuste, **655**, 663–664, 680, 721, 723
sobregeneración, **835**
sobremuestra, **707**
Socher, R., 26, 872, 878, 880, 1021, *1042*, *1047*, *1050*, *1058*
Socialedad, **992**
Sociedad de Inteligencia Artificial y Simulación del Comportamiento (AISB), 35
sociedad de la mente, 646
Sócrates, 3
Soderland, S., 339, 850, 855, *1035*, *1043*
Softbot, **43**
Softmax, **758**, 811
SoftPlus, **752**
Sohl-Dickstein, J., 787, *1058*
Sohn, J. H., 30, *1042*
Soika, M., 977, *1037*
Sol, A., 29, *1067*
Sol, J., 786, *1047*
Sol, R., 916, *1042*
Sol, S., 316, *1042*
Sol, Y., 879, *1063*
solidez (de inferencia), **216**, 222, 230, 246, 288
solipsismo, 985
Solla, S., 787, *1052*
Solomonoff, R. J., 18, 716, *1062*
solución cíclica, **125**
solución, **64**, 65, 105, **181**
  óptima, **65**
solución parcial, **181**
SOLUCIONADOR DE CSP DE ÁRBOLES, **200**
Solucionador General de Problemas, 2, 7, 19, 380
Soma, 12
sonar, **927**
Sondik, E. J., 597, *1062*
Sonenberg, L., 719, 1009, *1055*
Song, Z., 786, *1034*
Sonneveld, D., 106, *1062*
Sontag, D., 526, *1055*
Sopa de piedras de ayuno hacia abajo, 381
Sophia (robot), 1000

Sorensen, T., 647, *1045*
Sosic, R., 206, *1062*
Sowa, J., 343, *1062*
Spaan, M. T. J., 598, *1062*
SPACY (analizador), 853
Sparck Jones, K., 410, *1059*
Sparrow, R., 1000, *1062*
SPASS (demostrador de teoremas), 312
SPI (Inferencia Probabilística Simbólica), 456
Spiegelhalter, D. J., 456-458, 525, 559, 747, *1040*, *1045*, *1052*, *1054*, *1062*
SPIKE (sistema de planificación), 384
SPIN (sistema de razonamiento lógico), 309
Spiropulu, M., 1018, *1056*
Spirtes, P., 747, *1062*
Spitkovsky, V. I., 840, *1062*
Spitzer, E., 995, *1055*
Spronck, P., 176, *1039*
Sproull, R. F., 62, 558, *1043*
SQUAD (Conjunto de datos de respuestas a preguntas de Stanford), 25, 28, 880
Srebro, N., 1009, *1047*
SRI, 20, 278, 380, 558, 976
Srinivasa, S., 951, 963, 978, 979, *1042*, *1059*, *1067*
Srinivasan, M. V., 1013, *1044*
Srivas, M., 309, *1062*
Srivastava, N., 787, *1062*
Srivastava, S., 597, *1062*
Staab, S., 339, *1062*
Stader, J., 384, *1034*
STAGE (algoritmo de búsqueda local), 142
Stallman, R. M., 205, *1062*
STAN (lenguaje de programación probabilístico), 458, 527
Stanfill, C., 717, *1062*
Staniland, J. R., 410, *1041*
Stanislawska, K., 143, *1062*
Stanley (vehículo autónomo), 972, 973, 977
Star Trek, 1007
StarCraft II, 28, 30, 147, 172, 179, 822
START (sistema de preguntas y respuestas), 850
state-action-reward-state-action (SARSA), **802**
States, D. J., 748, *1048*
Statnikov, A. R., 719, *1046*
Stefik, M., 343, 459, *1062*
Stein, C., 107, 1029, *1040*
Stein, J., 455, *1056*
Steinbach, M., 720, *1063*
Steinberg, R., 649, *1040*
Steiner, D. F., 30, *1062*
Steiner, W., 977, *1038*
Steinhardt, J., 1010, *1034*
Steinruecken, C., 719, *1062*
Stensrud, B., 311, *1063*
Stentz, A., 142, *1042*
Stepleton, T., 821, *1056*
Stergiou, K., 205, *1062*
Stern, H. S., 748, *1045*
Stickel, M. E., 248, 312, *1062*, *1067*
Stiller, L., 176, *1062*
Stob, M., 716, *1057*
STOCKFISH (programa de ajedrez), 160, 176, 177
Stockman, G., 175, *1062*
Stoffel, K., 339, *1062*
Stokes, I., 384, *1034*
Stolcke, A., 29, *1066*
Stone, P., 27, 179, 649, 650, 822, 977, *1035, 1046, 1062, 1063*
Stone, C. J., 716, *1038*
Stone, M., 716, *1062*
Stork, D. G., 720, 749, *1043*
Storvik, G., 499, *1063*
Story, W. E., 106, *1049*
Stoyanov, V., 876, 879, *1053*
Strachey, C., 14, 17, 175, 177, *1063, 1064*
Strat, T. M., 459, *1060*
Stratonovich, R. L., 497, 560, *1063*
Straub, J., 822, *1060*
Striebel, C. T., 497, *1059*
STRIPS (sistema de planificación), 380, 382, 383
Strohm, G., 383, *1044*
Strohman, T., 850, 854, *1040*
Strohmann, T., 316, *1042*
Stross, C., 12, *1042*
Stützle, T., 142, 206, *1042, 1048*
Stuckey, P. J., 205, 312, *1049*, *1054*
Stuhlmüller, A., 527, 821, *1060, 1066*
Stumpe, M. C., 30, *1046, 1053, 1062*
Sturtevant, N. R., 88, 96, 108, 145, *1034, 1039, 1043, 1048, 1063*
Stutz, J., 748, *1039*
Su, H., 786, *1060*
suavizado, 468–471, 496, 744, **827**, 891
  complejidad, 470
  interpolación lineal, **827**
  online, 475
suavizado add-one, 827
suavizado de interpolación, 851
suavizado de Laplace, 827
suavizado de retardo fijo, **471**
SUAVIZADO DE RETRASO FIJO, **475**
suavizado del Bien-Turing, 851 el bien y el mal, 557
subasta, **634**
  oferta ascendente, **634**
  Inglés, **634**
  primer precio, **636**
  oferta sellada, **636**
  segundo precio, **636**
  revelación de la verdad, **635**
  Vickrey, **636**
subasta combinatoria, 638
subasta eficiente, **634**
subcategoría, **317**, **841**
subgeneración, **835**
ascenso más empinado, **111**
subjetivismo, **409**
subjuego equilibrio perfecto de Nash, **619**
subjuego, **619**
submuestreo, **707**
subobjetivos serializables, **355**
subproblema, **100**
subproblemas independientes, **199**
Subrahmanian, V. S., 176, *1057*
Subramanian, D., 342, 1020, *1060, 1063*
subsunción
  en lógica descriptiva, 331
  en resolución, **308**
Suciu, D., 524, 527, *1041, 1046*
Sudderth, E., 512, 526, *1034*
Sudholter, P., 648, *1058*
Sudoku, **189**
Suk, H.-I., 498, **1063**
Sulawesi, 199
suma de diferencias al cuadrado (SSD), **893**
SUMA, **1023**
Summers-Stay, D., 910, *1046*
SUNFISH (programa de ajedrez), 176
Sunstein, C., 560, *1063*
Sunter, A., 525, *1043*
superaditividad, **627**
SUPERGLUE (punto de referencia de lenguaje natural), 880
Superman, 252
Supervisión (en aprendizaje automático), **712**
Supervisión (estimación de estado), **132**
Supervisión de la ejecución, **372**, 371–374, 383
supervivencia del más apto, 498
Suresh, A. T., 992, *1051*
Sussman, G. J., 205, 380, *1062, 1063*
sustancia, 321-322
Sustantivo de masa, **322**
sustitución (en lógica de primer orden), **266**, 280
sustituibilidad (de loterías), **530**
Sutcliffe, G., 312, 313, *1063*
Sutherland, G. L., 22, *1038*
Sutherland, I., 204, *1063*
Sutphen, S., 177, *1060*
Sutskever, I., 26, 786, 787, 849, 858, 879, 880, 922, *1049, 1051, 1055, 1059, 1062, 1063*
Suttner, C., 313, *1063*
Sutton, C., 855, *1063*
Sutton, R. S., 62, 596, 598, 820–822, *1035, 1063*
Svartvik, J., 853, *1059*
Svestka, P., 978, *1050*
Svetnik, V. B., 498, *1066*

SVM (máquina de vectores de soporte), **692**, 692–696
Swade, D., 15, *1063*
Swartout, W. R., 1009, *1056*
Swartz, R., 338, *1038*
Swayamdipta, S., 877, 879, *1046, 1060*
Sweeney, L., 991, 1009, *1063*
Swerling, P., 497, *1063*
Swersky, K., 995, *1066*
Swetter, S. M., 30, *1043*
Swift, T., 311, *1063*
SYMBA* (sistema de planificación), 381
SYNTHIA (entorno simulado), 822
Syrja¨nen, T., 342, *1056*
SYSTRAN (software de traducción automática), 880
Syverson, C., 998, *1038*
Szafron, D., 647, 648, *1036, 1037, 1065*
Szathma'ry, E., 143, *1062*
Szegedy, C., 309, 312, 786, 787, *1034, 1035, 1049, 1053, 1063*
Szeliski, R., 924, *1063*
Szepesvari, C., 176, 597, 822, *1051, 1063*
Szerlip, P., 526, *1036*
Szita, I., 176, *1039*

T

T (retenciones fluidas), 323
T4 (sistema de planificación), 383
T5 (sistema de lenguaje natural), 877, 880
Tabla de la verdad, **219**, 248
Tabla de ocupación, **977**
tabla de probabilidad condicional (CPT), **413**
Tabla de transposición, **155**
Tableau (software de análisis de datos), 709
Tadepalli, P., 561, 820, *1043*, *1063*
Tafjord, O., 850, *1040*
Tait, P. G., 106, *1063*
Takusagawa, K. T., 527, *1058*
Taller de Dartmouth, 18
Talos, 975, 1006
Talukdar, P., 850, *1055*
Tamaki, H., 311, *1063*
tamaño del paso, **121**
Tambe, M., 207, *1057*
Tammelin, O., 30, 178, *1037*
Tan, P., 720, *1063*
Tang, E., 1018, *1063*
Tang, J., 822, *1038*
Tanh, **753**
Tank, D. W., 11, *1057*
Tardos, E., 648, *1057*
Tarea completa, **181**
Tarricone, R., 559, *1036*
Tarski, A., 278, 279, 853, *1063*
tasa de error, **666**
tasa de mutación, **116**
Tash, J. K., 597, *1063*
Taskar, B., 527, *1045*
Tasmania, 199
Tassa, Y., 822, 979, *1047, 1053, 1063*
Tate, A., 359, 380, 382-384, *1034, 1036, 1063*
Tatman, J. A., 596, *1063*
Tattersall, C., 160, *1063*
Tavener, S., 176, *1038*
Taylor, A. D., 648, *1063*
Taylor, C., 923, 1041
Taylor, G., 311, *1063*
Taylor, M., 339, *1062*
Taylor, P., 849, *1063*
Taylor, R., 979, *1053*
Taylor, W., 206, *1039*
taxi, 42, 43, 385
  automatizado, 57, 210, 385, 1019
TD-GAMMON (programa de backgammon), 19, 178, 816
Tedrake, R., 979, *1063*
Tegmark, M., 787, *1059*
Teh, Y. W., 498, 786, *1048, 1057*
Telar jacquard, 15
Teléfono, 863
Telémetro, **927**
  láser, 928
Telescopio Espacial Hubble, 183, 197, 384
Teller, A., 142, 457, *1055*
Teller, E., 142, 457, *1055*
Teller, S., 977, 979, *1037, 1063*
Tellex, S., 979, *1063*
Tenenbaum, J. B., 179, 278, 526, 527, 560, 821, *1035, 1044, 1046, 1051, 1052, 1063*
tenis, 603
Tennenholtz, M., 820, *1037*
tensor, **763**
TensorFlow (software de aprendizaje automático), 1009, 1021
TensorFlow, 720
Teócaro, G., 498, *1063*
teorema, **267**
  incompletitud, 9, 305, 983
Teorema de conformación, **569**
Teorema de equivalencia de ingresos, **636**
Teorema de Finetti, 394
Teorema de Gibbard-Satterthwaite, **641**
Teorema de Herbrand, **304**, 310
Teorema de la deducción, **222**
Teorema de la representación, **543**
Teorema de Mercer, **695**
Teorema de Nash, 609
Teorema de resolución del suelo, *228*, 303
Teorema del límite central, **1028**
Teoremas populares de Nash, **617**
teoría de control, **16**, 15-16, 61, 142, 380, 785, 816, **950**
Teoría de Dempster-Shafer, 459
Teoría de juegos, 10, **600**, 645
Teoría de la Confirmación, **7**, 409
Teoría de la Decisión, 10, 25, **387**, 557
Teoría de la elección social, **638**
Teoría de la información, 661-662, 715
teoría de la optimalidad (en lingüística), 851
  algoritmo óptimamente eficiente, **90**
Teoría de la posibilidad, **460**
Teoría de la utilidad multiatributo, **540**, 559
teoría de modelos (en lógica), 278
Teoría de retículos, 313
Teoría del aprendizaje computacional, 672, **673**
Teoría del Control Adaptativo, 819
Teoría del control óptimo, 142
teoría del control robusto, **801**
Teoría descriptiva, **538**
Teoría Integrada de la Información, 1007
Teoría normativa, 538
Teoría sintáctica (del conocimiento), 341
Teórico de la Lógica (LT), 18, 248
Teplyashin, D., 822, *1036*
ter Meulen, A., 279, *1064*
terminación anticipada de la reproducción, **164**
Término (en lógica), **259**, 259–260
término de tierra, **261**, 280
termostato, 16
Terremoto III, 28, 30
terremoto, 413
Tesauro, G., 19, 176, 178, 804, 815, 816, 820, 1063
Teseo, 715
Tesis Church-Turing, 9
Tesla, 924, 973
test de coeficiente intelectual, 20
test de Turing, **2**, 2, 4, 823, 984, 1006
  total, **2**
Tetlock, P. E., 28, 338, *1063*
TETRAD (software de aprendizaje automático), 747
Tetris, 572, 581
Teukolsky, S. A., 142, *1058*
TEXTRUNNER (sistema de extracción de información), 316, 850, 855
textura, **892**
Teyssier, M., 747, *1063*
Thaler, R., 559, 560, *1063*
Thayer, J. T., 109, *1063*
Thiele, T., 497, *1063*
Thielscher, M., 176, 250, 340, *1061, 1063*
Thirion, B., 720, *1058*
Thitimajshima, P., 458, *1036*
Thng, F., 30, *1062*
Thomaz, A., 979, *1034*, *1063*
Thompson, B., 598, *1066*
Thompson, E., 456, *1038, 1061*
Thompson, K., 176, *1040, 1063*

Thompson, W. R., 586, 597, *1063*
Thorndike, E., 819, *1063*
Thornton, C., 719, *1064*
Thorpe, C., 28, *1050*
Throop, T. A., 178, *1062*
Thrun, S., 28, 30, 61, 498, 598, 719, 977-980, *1038, 1039, 1043, 1044, 1055, 1058, 1060, 1064*
Thurstone, L. L., 526, *1064*
Tian, H., 879, *1063*
Tian, J., 455, *1064*
Tiao, G. C., 747, *1037*
Tibshirani, R., 717-720, 749, *1036, 1044, 1047, 1049*
tiempo (discreto), **462**
tiempo (en gramática), **846**
tiempo bien empleado, **1015**
tiempo de mezcla, **468**
tiempo de parada, **583**
Tiempo verbal, **846**
Tiempo, 461-499
tiempo, **75**, 105
Tikhonov, A. N., 716, *1064*
Timofeev, A., 30, *1053*
Tinsley, M., 177
tipo firma, **503**
tipo natural, **320**
Tipo topológico, **199**, 200
Tirol, J., 648, *1044*
Tishby, N., 498, *1044*
Titterington, D. M., 748, *1064*
TMS (sistema de mantenimiento de la verdad), 206, **335**, 335–337, 342
Tobarra, L., 250, *1034*
Tobin, J., 979, *1034*
Todorov, E., 978, *1053*
Tohme', F., 648, *1060*
tokenización, **825**
Tolpin, D., 597, 1019, *1047*
toma de decisiones estratégicas, **600**
Toma, P., 880, *1064*
tomador de decisiones, **558**
Tomás, A., 458, 525, 747, *1045*, *1054*
Tomás, J., 720, *1040*
Tomás, P. S., 821, *1063*
Tomasi, C., 923, *1064*
Tomlin, C. J., 821, *1034*
Tomografía de redes, 455
Tononi, G., 1007, *1057*
Topcu, U., 341, 1014, *1053*
Topol, E., 30, *1053, 1064*
Torbica, A., 559, *1036*
Torralba, A., 381, 690, *1064*
Torras, C., 144, 383, *1049*
tortuga, 976
Toshev, A., 30, 879, *1064*
totalmente conectado (red neuronal), **754**
totalmente observable, 588
Toutanova, K., 879, *1042*
Tovey, C. A., 649, *1035*
TPTP (Miles de problemas para demostradores de teoremas), 313
TPU (unidad de procesamiento tensorial), 15, 27, 763
Trabajo, **375**, 1000
trabajo, futuro de, 998–1000
traducción automática (MT), 849, **864**, 880
  sin supervisión, **780**
traducción, *véase* de traducción automática
Tragedia de los comunes, **637**, 1002
Trame'r, F., 1009, *1064*
Tran, D., 526, *1064*
Tran-Gia, P., 1017, *1048*
Transferencia de estilo, **915**
transformación de imagen, **913**
Transhumanismo, **1005**, 1010
transición de fase, 249
transitividad (de preferencias), **530**
transparencia referencial, **327**
Transparencia, **997**
transposición (en un juego), **155**
transposición, **1026**
Trappenberg, T., 788, *1064*
Traverso, P., 381, 383, 384, *1036, 1040, 1045*
Trayectoria, **939**
TREC (Conferencia de Recuperación de Textos), 850
Treichler, S., 30, *1052*
tres en raya, 147, 175
Trick, M. A., 649, *1035*
tricromacia, **888**
Triggs, B., 922, *1041*
triples desnudos, 191
Trivedi, H., 30, *1042*
Troyanskii, P., 880
Truby, R. L., 1013, *1051, 1056*
truco del kernel, 692, **695**, 717
Trucos estúpidos de mascotas, 41
Truszkowski, P., 30, *1062*
Tsang, E., 206, 207, *1048, 1064*
T-SCHED (sistema de planificación), 384
Tse, D., 787, *1067*
Tshitoyan, V., 872, *1064*
Tsipras, D., 787, *1039*
Tsitsiklis, J. N., 410, 596, 598, 805, 820-822, 1029, *1036, 1054, 1057, 1064*
t-SNE (incrustación de vecino estocástico distribuido en t), **709**, 719
TSP (problema del vendedor ambulante), 70, 107, 109
Tsuruoka, Y., 1021, *1047*
¿TT-CONLLEVA?, 221
TT-CHECK-ALL, **221**
tú y tú, 835
Tuberg, D., 456, *1061*
tubo de vacío, 17
Tukey, J. W., 708, 719, *1064*
Tulsiani, M., 913, *1050*
Tumer, K., 649, *1064*
Tung, F., 497, *1059*
tupla, **256**
Turian, J., 878, *1064*
Turing, A., ix, 2, 9, 14, 17, 19, 33, 56, 175, 282, 310, 454, 715, 785, 819, 981–984, 986, 1006, 1010, 1020, 1022, *1064*
Tversky, A., 418, 538, 560, *1044, 1049, 1064*
Tygar, J. D., 1010, *1035, 1064*
Tzemaj, A., 456, *1061*

U

U (utilidad), 529
$u_{\top}$ (mejor premio), 533
$U_{\perp}$ (Peor Catástrofe), 533
UAV (vehículo aéreo no tripulado), 927
Uber, 924
ubicación del aeropuerto, 540, 545
UCB1 (límite superior de confianza), **163**
UCPOP (sistema de planificación), 380
UCT (algoritmo de búsqueda de árbol de juego), **163**
Udluft, S., 821, *1047*
Ulam, S., 176, *1055*
Ullman, J. D., 311, *1035, 1064*
Ullman, S., 922, 923, *1048, 1064*
ULMFIT (software de lenguaje natural), 879
umbral blando, 423
unario, 184
unidad (en redes neuronales), **751**
unidad de compuerta (en LSTM), **775**
unidad de procesamiento gráfico (GPU), 15, 27
unidad de procesamiento tensorial (TPU), 15, 27, 763
unificación, **283**, 283-284, 309, 310
  ecuacional, **307**
  y la igualdad, 306
unificador más general (MGU), **284**, 286, 306
UNIFICAR, **285**
UNIFICAR-VAR, **285**
uniforme prior, 724
UNIMATE (brazo robótico), 976
unitaria propagación, **233**
Universidad Carnegie Mellon, 18
Universidad de Stanford, 19, 22, 23, 27, 278, 649
Universidad Harvey Mudd, 994
Universo Herbrand, **303**, 310
UNPOP (sistema de planificación), 380
UOSAT-II (satélite), 384
Urbano, J., 312, *1034*
Urmson, C., 977, *1064*
Usher, J. M., 339, *1044*
Usher, N., 1018, *1042*

Uszkoreit, J., 850, 868, 880, *1064*
utilidad, **7**, **54**, 387
  adaptativo, **561**
  axiomas de, 531
  de dinero, 534-536
  esperado, **55**, 62, 387, 528, **529**, 534
  estimación, **792**
  función, **54**, 55, 147, 529, 532–539
  independencia, **544**
  multiatributo, 540–544, 557
  multiplicativo, **544**
  nodo, **545**
  normalizado, **533**
  ordinal, **532**
  teoría, 10, **387**, 529-532, 557
utilidad esperada, **55**, 62, 387, 528, **529**, 534
utilidad máxima esperada (MEU), **387**, 529, 575
utilidad ordinal, **532**
utilitarismo, **8**
Utopía, 1022
UWL (sistema de planificación), 383

V

VAE, *véase* autocodificador, variacional
vaguedad, 459, 824
Valdés, V., 822, *1036*
validación cruzada de dejar una salida (LOOCV), **666**
Validación cruzada K-Fold, **666**
validación cruzada, **666**, 716
VALIDACIÓN CRUZADA, **667**
Valiant, L., 717, *1064*
Vallati, M., 381, *1064*
valor (de una variable), **59**
  en un CSP, 180
Valor atípico, **707**
valor común, 634
valor de cálculo, 1019
valor de la información perfecta (VPI), **548**
valor de la información, **547**, 560
valor de la información, 547–553, 557, 589, 1019
Valor de Shapley, **628**, 629, 645
valor de verdad, **218**
Valor esperado (en un árbol de juego), **156**, 165
valor material, **157**
VALOR MÁXIMO, **150**, **154**
Valor minimax, **148**
VALOR MÍNIMO, **150**, **154**
valor monetario esperado (EMV), **534**
Valor predeterminado, **331**
valor privado, 634
Valoración, **505**
valores continuos, 422
Valores de atributo faltantes, **664**
valores restantes-mínimos, **193**, 290
VALOR-Q, **569**
Vals, D., 20, 204, 717, *1062, 1065*
value, *consulte* el nodo de utilidad
VAMPIRO (demostrador de teoremas), 312, 313
Van Beek, P., 204-207, 340, 381, *1035, 1051, 1060, 1064*
Van Bentham, J., 279, *1064*
Van den Broeck, G., 527, *1046, 1050*
Van den Driessche, G., 27, 30, *1061*
Van den Oord, A., 29, 779, 787, 849, *1064*
Van der Gaag, L., 410, *1053*
Van der Maaten, L., 1009, *1046*
Van Doorn, A. J., 923, *1051*
Van Gysel, C., 850, *1034*
Van Harmelen, F., 343, *1064*
Van Heijenoort, J., 313, *1064*
Van Hentenryck, P., 205, *1064*
Van Hoeve, W.-J., 205, *1064*
Van Kleef, P., 339, *1052*
Van Lambalgen, M., 340, *1064*
Van Maaren, H., 249, *1036*
Van Nunen, J. A. E. E., 596, *1064*
Van Ooyen, B., 852, *1038*
Van Roy, B., 805, 820, *1064*
Van Roy, P. L., 294, 311, *1064*
Van Run, P., 207, *1035*
Vandenberghe, L., 143, *1037*
Vanhoucke, V., 849, 854, 978, *1037, 1048*
Vanschoren, J., 719, *1048*
Vapnik, V. N., 26, 717, 718, 720, 786, 922, *1037, 1040, 1052, 1064*
Varaiya, P., 61, 822, *1044*, *1051*
Vardi, M. Y., 341, *1043, 1064*
variable, **59**
  aleatoria, 390, 417
    Bernoulli, 390
    booleano, 390
    continuo, 391, 422, 455
  atemporal, **238**
  eliminación, **430**, 430-433, 454, 456, 490
  en el espacio de estados continuos, **120**
  en lógica, **261**
  en un CSP, 180
  indicador, **740**
  irrelevante, 433
  pedidos, 193, 432
  sin modelar, **451**
variable aleatoria, 390, 417
  básico, **503**
  continuo, 391, 422, 455
  indexados, **525**
variable de consulta, 427
variable de evidencia, 427
variable indicadora, **740**
variable latente, **737**
variable numérica, 509
variable oculta, **425**, 737
variable persistente, 1031
Varian, H. R., 29, 649, 719, *1054, 1059, 1064*
varianza, **655**, 670, **1028**
Varoquaux, G., 720, *1058*
Varshney, K. R., 995, 996, 1009, 1010, *1036, 1047*
Varshney, L., 880, *1050*
Varzi, A., 340, *1039*
Vasilache, N., 1018, *1064*
Vasserman, L., 995, *1055*
Vaswani, A., 850, 868, 880, *1064*
Vaucanson, J., 976
Vaughan, J. W., 995, *1045*
Vazirani, U., 142, 720, *1034, 1050*
Vazirani, V., 648, *1057*
Vázquez, D., 822, *1059*
Vckay, E., 822, *1049*
Veach, E., 457, *1064*
Vecchi, M. P., 142, 206, *1050*
Vecchione, B., 995, *1045*
vecinos cercanos, **689**
vecinos más cercanos, **687**, 717, 736
vector clave (en transformadores), **869**
vector de consulta (en transformadores), **869**
vector de pago, **626**
vector de valor (en transformadores), **869**
vector, **1025**
Veeramachaneni, K., 719, *1050*
vehículo aéreo no tripulado (UAV), 927
Veinott, A. F., 597, *1050*
Veit, A., 1009, *1035*
Velocidad de mezcla, **446**
Veloso, M., 179, 979, *1034, 1061*
Veness, J., 598, 820, 822, *1036, 1055, 1061*
Venkataraman, S., 597, *1046*
Venkatesh, S., 411, *1064*
Venn, J., 409
ventana corredera, **899**
Ventana de párzen, 748
Ventisca, 822
Venugopal, A., 880, *1067*
Venugopalan, S., 30, *1046, 1053*
Verdad fundamental, **653**
Verdad, **214**, 260
Verde, B., 853, *1046*
Verde, C., 20, 278, 309, 311, *1046*
Verde, P., 924, *1038*
Verde, S., 822, *1036*
Verde, T., 787, *1049*
Vere, S. A., 383, *1064*
verificación, **309**
  hardware, 276
verificación y validación, **996**
Verma, S., 1009, *1064*
Verma, T., 455, 456, 747, *1045, 1057*
Verma, V., 498, *1064*
Verweij, G., 998, *1059*
Vetterling, W. T., 142, *1058*
Vicente, P., 878, *1036*

Vickrey, W., 636
vida, valor de la estadística, **533**
videojuego, 179
  Atari, 28, 178, 784, 816
Vidrio, J., 498, *1053*
Viegas, E., 719, *1046*
Vie'gas, F., 719, *1065*
Vig, L., 990, *1054*
Vigencia, **222**, 246
vigentes, **98**, 107, 155
Vihma, T., 143, *1062*
vinculación, **214**, 246
Vinge, V., 12, *1064*
Vinyals, O., 30, 179, 716, 779, 786, 787, 822, 849, 878–880, *1046, 1049, 1053, 1063, 1064, 1067*
Viola, P., 922, *1064*
Virasoro, M., 142, *1055*
Virochsiri, K., 822, *1045*
visión estereoscópica, **927**
Visión por computadora, **2**, 12, 20, 204, 882–919
Visión, **2**, 12, 20, 204, 882–919
Visser, U., 179, *1064*
Visser, W., 309, *1047*
visto literalmente, **248**
Vitányi, P., 717, *1053*
Viterbi, A. J., 497, *1064*
Vlassis, N., 598, 650, *1062, 1065*
Vlimant, J.-R., 1018, *1056*
Vogt, D. M., 1013, *1056*
Volk, K., 748, *1045*
Von Linne, C., 339 Vorhees, E., 850, *1034*
Von Mises, R., 409, *1065*
Von Neumann, J., 10, 16, 17, 531, 558, 647, 648, *1065*
Von Stengel, B., 621, 647, *1051*
Von Winterfeldt, D., 558, *1065*
Voronkov, A., 279, 312, 313, *1059*
Vossen, T., 381, *1065*
votación de la regla de la verdadera mayoría, **640**
votación de segunda vuelta instantánea, **640**
votos aprobatorios, **640**
votos, 640
VPI (valor de la información perfecta), **548**
VQA (visual de respuesta a preguntas), 28, **910**

W

Wadsworth, C. P., 278, *1046*
Wagner, D., 108, *1042*
Wagner, S. K., 30, *1053*
Wahba, G., 716, *1046*
Wainwright, M., 249, 458, 822, *1036, 1054, 1065*
Waldinger, R., 278, 279, *1054*
Walker, E., 29, *1040*
Walker, G., 787, *1065*
Walker, H., 748, *1045*
Walker, R. J., 205, *1065*
WALKSAT, **235**
Wallace, A. R., 118, *1065*
Wallach, H. M., 995, *1045*
Walpole, R. E., 411, *1065*
Walras, L., 10
Walsh, M. J., 143, *1044*
Walsh, T., 205, 207, 249, 998, *1036*, 1060, 1062, 1065
Walsh, W., 649, *1065*
Walter, G., 976
Walter, M. R., 979, *1063*
WAM (Warren Abstract Machine), 311
Wang, A., 879, 880, *1065*
Wang, D. Z., 850, 855, *1038*
Wang, E., 342, *1063*
Wang, H., 879, *1063*
Wang, J., 176, *1065*
Wang, L., 786, *1042*
Wang, S., 879, *1063*
Wang, T., 15, 1018, *1066*
Wang, Z., 821, *1066*
Wanner, E., 253, *1065*
Ward, T., 822, *1036*
Warde-Farley, D., 787, *1046*
Warmuth, M., 106, 717, *1037, 1059*
Warner, C., 852, *1036*
WARPLAN (sistema de planificación), 380
Warren Abstract Machine (WAM), 311
Warren, D. H. D., 294, 311, 380, *1058, 1065*
Warren, D. S., 311, *1063*
Washington, G., 325
Wasserman, L., 720, *1065*
Watkins, C. J., 596, 820, *1065*
Watson (sistema de preguntas y respuestas), 26
Watson, J. D., 118, *1065*
Watson, J., 13
Watson, J.-P., 206, *1036*
Watt, J., 16
Wattenberg, M., 143, 719, *1049, 1065*
Watts, M., 1013, *1058*
Waugh, K., 30, 178, 648, *1056, 1065*
WaveNet (software de generación de voz), 849
WaveNet, 779, 787
Way, D. H., 30, *1053*
Waymo, 28, 924, 972, 973
Wayne, G., 979, *1047*
Wayne, K., 1029, *1061*
WBRIDGE5 (programa puente), 178
Weaver, W., 661, 715, 720, 851, 880, *1061*
Web Semántica, 339
Weber, J., 498, *1048*
Webster, D. R., 30, *1053*
Wefald, E. H., 109, 175, 1019, *1060*
Wegbreit, B., 977, *1055*
Weglarz, J., 384, *1037*
Wei, J. N., 920, *1060*
Wei, X., 855, *1058*
Weibull, J., 648, *1065*
Weidenbach, C., 312, *1065*
Weinstein, S., 716, *1057*
Weiss, D., 853, *1034*
Weiss, G., 62, 650, *1065*
Weiss, R., 720, 849, *1039, 1058*
Weiss, S., 852, *1034*
Weiss, Y., 458, 499, 690, *1056, 1064–1066*
Weissbrod, O., 456, *1061*
Weissman, V., 278, *1046*
Weizenbaum, J., 1001, *1065*
Weld, D. S., 144, 339, 342, 380-384, 854, 855, 1008, *1040, 1043, 1052, 1058, 1062, 1065, 1066*
Welinder, P., 979, *1034*
Welling, M., 787, *1050*
Wellman, M. P., 455, 460, 498, 525, 559, 596, 648, 649, 978, *1041, 1048, 1065*
Welty, C., 30, 339, *1043, 1062*
Wen, M., 341, 1014, *1053*
Werbos, P., 596, 597, 785, 786, 820, *1065*
Wermuth, N., 455, *1052*
Werneck, R. F., 108, *1045*
Wertheimer, M., 921
Wesley, M. A., 978, *1065*
West, Col., 287
West, D. M., 1011, 1065
West, M., 499, *1053*
West, S. M., 995, *1065*
Westinghouse, 383
Weston, L., 872, *1064*
Wexler, Y., 456, *1065*
Weymouth, T., 976, *1040*
Wheatstone, C., 921, *1065*
Whitehead, A. N., 17, 310, *1065*
Whitehouse, D., 176, *1038*
Whiter, A. M., 383, *1063*
Whitney, W. F., págs. 179, *1044*
Whittaker, M., 995, *1065*
Whittaker, W., 28, 977, *1050, 1064*
Whittle, P., 581, *1065*
Whorf, B., 252, 278, *1065*
Widner, K., 30, *1046*
Widrow, B., 21, 785, 819, *1065*
Wiedijk, F., 313, *1065*
Wiegley, J., 144, *1065*
Wiener, N., 16, 18, 33, 39, 175, 497, 785, 880, 1008, 1011, *1059, 1065*
Wierstra, D., 784, 787, 790, 820, 822, 979, *1053, 1055, 1059*
Wiesel, T. N., 786, 922, *1048*
Wikipedia, 316, 823, 825
Wijmans, E., 822, 1060
Wilcox, S., 309, *1035*
Wildes, R. P., 1013, *1050*

Wilensky, R., 23, *1065*
Wilfong, G. T., 976, *1040*
Wilkins, D. E., 383, *1065*
Wilks, Y., 1010, 1011, *1065*
Williams, A., 880, *1065*
Williams, B., 249, 342, 384, *1056, 1065*
Williams, C. K. I., 748, *1059*
Williams, J., 598, *1066*
Williams, R. J., 596, 785, 786, 812, 820, 821, *1058, 1060, 1065*
Williamson, J., 340, *1056*
Williamson, M., 383, *1043*
Willighagen, E. L., 340, *1056*
Wilmer, E. L., 497, *1052*
Wilson, A., 852, *1052*
Wilson, D. H., 1001, *1065*
Wilson, R. A., 3, 1007, *1065*
Wilson, R., 204, *1065*
Wilt, C. M., 108, *1066*
Wiltschko, A. B., 920, *1060*
Windows (sistema operativo), 455
Winfield, A., 142, 997, *1038, 1042*
Wingate, D., 527, 596, *1066*
Winikoff, M., 61, *1057*
Winker, S., 313, *1066*
Winkler, R. L., 537, 559, *1062*
Winograd, S., 20, *1066*
Winograd, T., 20, 23, 854, *1066*
Winston, P. H., 20, 32, *1036, 1066*
Wintermute, S., 311, *1066*
Winternitz, L., 29, *1066*
Witbrock, M., 339, *1054*
Witten, D., 720, *1049*
Witten, I. H., 720, 820, 851, 852, *1066*
Wittgenstein, L., 7, 216, 248, 249, 320, 339, *1066*
Wohlhart, P., 978, *1037*
Wojciechowski, W. S., 309, *1066*
Wojcik, A. S., 309, *1066*
Wolf, A., 853, *1046*
Wolfe, J., 144, 176, 382, *1054, 1060, 1066*
Wolpert, D., 649, 715, *1064, 1066*
Wolsey, L. A., 561, *1056*
Wolski, F., 979, *1034*
Wong, C., 719, *1066*
Wong, K. W., 984, *1049*
Wong, W.-K., 747, *1056*
Wood, F., 457, 498, 527, *1052, 1057*
Wood, R. J., 1013, *1056*
Woodruff, A., 1009, *1036*
Woods, W. A., 341, 854, *1066*
Wooldridge, M., 61, 62, 648, 650, *1039, 1040, 1059, 1066*
Woolsey, K., 816
WORD2VEC (software de incrustación de palabras), 857, 875, 878
WordNet (base de datos léxica), 339
WordNet, **828**
Worswick, S., 1006
Wos, L., 312, 313, 1066
Wossnig, L., 1018, *1042*
Wray, R. E., 311, *1066*
Wright, S., 143, 454, *1066*
WSE (motor de escala de obleas), 15
Wu, D., 30, *1046*
Wu, E., 850, 855, *1038*
Wu, F., 339, *1066*
Wu, H., 879, *1063*
Wu, I.-C., 176, *1065*
Wu, J., 879, *1059*
Wu, L., 29, *1066*
Wu, S., 995, *1055*
Wu, Y., 29, 499, 527, 783, 786, 849, 850, 865, 878, 995, *1039, 1043, 1049, 1066*
Wundt, W., 12, 920
Wurman, P., 649, *1065*

X

XAI (IA explicable), 719, **997**
XCON (sistema experto), 292
Xiong, C., 880, 1021, *1047, 1050*
Xiong, W., 29, *1066*
XLM (modelo de lenguaje multilingüe), 879
XLNET (sistema de lenguaje natural), 879
xor, 219
Xu, B., 787, *1046*
Xu, J., 311, *1066*
Xu, P., 852, 880, *1037*

Y

Yahtzee, 168
Yakimovsky, Y., 558, *1043*
Yale, 23
Yampolskiy, R. V., 1010, *1066*
Yan, D., 383, *1044*
Yang, B., 850, *1055*
Yang, G., 498, *1066*
Yang, X.-S., 142, *1066*
Yang, Y., 787, 879, *1053*, *1066*
Yang, Z., 879, *1066*
Yannakakis, M., 144, 206, *1036, 1057*
Yao, L., 29, *1067*
Yao, X., 142, *1053*
Yap, R. H. C., 312, *1049*
Yarowsky, D., 26, 855, *1066*
Yates, A., 855, *1043*
Ye, K. E., 411, *1065*
Ye, N., 598, *1035*
Ye, X., 822, *1045*
Ye, Y., 596, *1066*
Yedidia, J., 458, *1066*
Y-Eliminación, **223**
Yeo, H.-S., 1013, *1066*
Ying, C., 15, 1018, *1066*
Yip, K. M.-K., 342, *1066*
Yngve, V., 853, *1066*
Y–O árbol, **123**
Yo, M., 748, *1039*
Yob, G., 250, *1066*
Y-O-BÚSQUEDA, **125**
Yoon, C. J. M., 1009, *1044*
York, S., 822, *1036*
Yoshida, Y., 786, *1055*
Yoshikawa, T., 978, *1066*
You, Y., 821, *1066*
Young, C., 652, 1018, *1041, 1049*
Young, H. P., 650, *1066*
Young, S., 598, *1066*
Young, T., 888
Younger, D. H., 835, 853, *1066*
Yu, B., 455, *1039*
Yu, D., 788, 849, 854, *1042, 1048, 1066*
Yu, F. X., 992, *1051*
Yu, H.-F., 718, *1066*
Yu, K., 787, *1066*
Yudkowsky, E., 33, 1010, *1066*
Yule, G. U., 787, *1066*
Yvanovich, M., 384, *1041*

Z

Z-3 (computadora primitiva), 14
Zadeh, L. A., 459, 460, *1066*
Zadeh, R. B., 720, *1059*
Zahavi, U., 88, *1043*
Zaldívar, A., 995, *1055*
zancada, **761**, 900
Zapp, A., 28, 923, 977, *1042*
Zaremba, W., 787, 822, 979, *1034, 1038, 1049, 1063*
Zaritskii, V. S., 498, *1066*
Zaykov, Y., 526, *1055*
Zecchina, R., 249, *1057*
Zeckhauser, R., 559, *1066*
Zeeberg, A., 782, *1066*
Zeilinger, M. N., 821, *1034*
Zelle, J., 854, *1066*
Zemel, R., 995, 1009, *1043, 1066*
Zemelman, B. V., 11, *1067*
Zen, H., 779, 787, 849, *1064*
Zeng, H., 278, *1055*
Zermelo, E., 647, *1067*
Zettlemoyer, L., 527, 845, 854, 876, 879, *1053, 1055, 1058, 1067*
Zhai, X., 786, *1042*
Zhang, B., 28, *1046*
Zhang, C., 716, *1067*
Zhang, F., 1009, *1064*
Zhang, H., 248, *1067*
Zhang, J., 880, *1059*
Zhang, L., 206, 248, 456, *1056, 1067*
Zhang, M., 853, *1067*

Zhang, N. L., 456, *1067*
Zhang, S., 29, 849, *1067*
Zhang, T. W., 979, *1063*
Zhang, W., 109, 316, *1042, 1051*
Zhang, X., 786, 852, *1047, 1067*
Zhang, Y., 849, 850, 853, 855, 996, 1009, *1036, 1038, 1067*
Zhang, Z., 88, *1043*
Zhao, J., 852, *1067*
Zhao, K., 845, 853, 854, *1045, 1067*
Zhao, Y., 206, 248, 822, *1056, 1060*
Zhao, Z., 916, *1042*
Zhou, K., 560, *1067*
Zhou, R., 109, *1067*
Zhou, T., 914, *1049*
Zhou, Y., 879, *1059*
Zhu, B., 787, *1067*
Zhu, C., 717, *1038*
Zhu, D. J., 977, *1067*
Zhu, J., 853, *1067*
Zhu, J.-Y., 879, 914, 915, *1049, 1067*
Zhu, M., 853, *1067*
Zhu, T., 176, *1065*
Zhu, W. L., 316, *1062*
Ziebart, B. D., 817, 963, 979, *1067*
Zilberstein, S., 144, 371, 383, *1047*
Zilles, S., 96, 108, 109, *1034, 1039, 1043, 1052*
Zimdars, A., 647, *1060*
Zimmermann, H.-J., 459, *1067*
Zinkevich, M., 647, 966, *1059, 1067*
Zipf, G., 878, *1067*
Zipser, D., 786, *1065*
Zisserman, A., 716, 923, 924, *1036, 1047, 1056*
Zlot, R., 142, *1042*
Zlotkin, G., 649, 650, *1060*
Zobrist, A. L., 177, *1067*
Zollmann, A., 880, *1067*
Zoph, B., 787, *1058, 1067*
Zucker, M., 951, 978, *1059*
Zuckerman, D., 114, *1053*
Zufferey, J. C., 1013, *1044*
Zuse, K., 14, 175, *1067*
Zweben, M., 384, *1041*
Zweig, G., 498, *1067*
Zwicker, W. S., 648, *1063*
zyzzyva, 825

Inteligencia artificial: un enfoque moderno, representa la introducción más completa y actualizada en la teoría y la práctica de la inteligencia artificial. Explora en profundidad el campo de la inteligencia artificial (IA) para actualizar a los lectores sobre las últimas tecnologías, presenta conceptos de una manera más unificada y ofrece una cobertura nueva o ampliada del aprendizaje automático, aprendizaje profundo, aprendizaje por transferencia, sistemas multiagente, robótica, procesamiento de lenguaje natural, causalidad y programación probabilística, privacidad, equidad e IA segura.

La cobertura en profundidad de temas básicos y avanzados proporciona a los estudiantes una comprensión básica de las fronteras de la IA sin comprometer la complejidad y la profundidad. Ofrece un enfoque unificado de la IA que muestra a los estudiantes cómo los distintos subcampos de la IA encajan para crear programas reales y útiles.

Características distintivas y actualizaciones:

- Amplia cobertura de avances recientes en aplicaciones de IA.
- La cobertura revisada de la visión por computadora, la comprensión del lenguaje natural y el reconocimiento de voz refleja el impacto de los métodos de aprendizaje profundo en estos campos.
- Los capítulos presentan una cobertura ampliada de la programación probabilística, toma de decisiones multiagente, aprendizaje profundo y aprendizaje profundo para el procesamiento del lenguaje natural.
- La definición de sistemas de IA se generaliza para asumir que el agente inteligente puede no estar seguro de los verdaderos objetivos de los humanos en cuyo nombre opera.
- Mayor cobertura de aprendizaje automático. El material de robótica se ha mejorado significativamente para incluir robots que interactúan con humanos y la aplicación del aprendizaje por refuerzo a la robótica.
- Las nuevas secciones abordan temas como causalidad (por Judea Pearl); búsqueda Monte Carlo de juegos y robótica, transferir aprendizaje para el aprendizaje profundo en general y para el lenguaje natural, privacidad, justicia, el futuro del trabajo e IA segura.
- Amplia cobertura del impacto de los métodos de aprendizaje profundo en los campos de la visión por computadora, la comprensión del lenguaje natural y el reconocimiento de voz.

El material de aprendizaje no técnico introduce conceptos importantes mediante explicaciones intuitivas, antes de entrar en detalles matemáticos o algorítmicos. El lenguaje no técnico hace que el libro sea accesible a una gama más amplia de lectores.

www.pearsonenespañol.com

ISBN: 978-607-32-6019-0